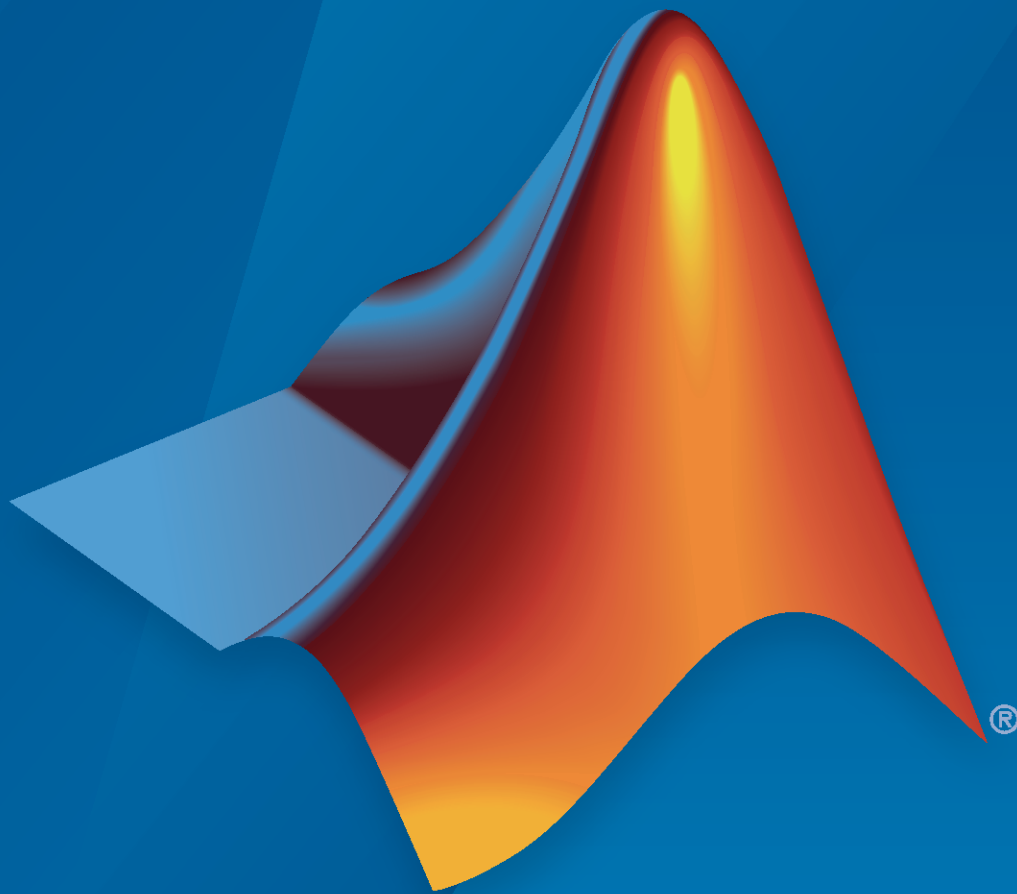


# Symbolic Math Toolbox™

## User's Guide



# MATLAB®

R2026a



## How to Contact MathWorks



Latest news: [www.mathworks.com](http://www.mathworks.com)  
Sales and services: [www.mathworks.com/sales\\_and\\_services](http://www.mathworks.com/sales_and_services)  
User community: [www.mathworks.com/matlabcentral](http://www.mathworks.com/matlabcentral)  
Technical support: [www.mathworks.com/support/contact\\_us](http://www.mathworks.com/support/contact_us)



Phone: 508-647-7000



The MathWorks, Inc.  
1 Apple Hill Drive  
Natick, MA 01760-2098

### *Symbolic Math Toolbox™ User's Guide*

© COPYRIGHT 1993-2026 by The MathWorks, Inc.

The software described in this document is furnished under a license agreement. The software may be used or copied only under the terms of the license agreement. No part of this manual may be photocopied or reproduced in any form without prior written consent from The MathWorks, Inc.

FEDERAL ACQUISITION: This provision applies to all acquisitions of the Program and Documentation by, for, or through the federal government of the United States. By accepting delivery of the Program or Documentation, the government hereby agrees that this software or documentation qualifies as commercial computer software or commercial computer software documentation as such terms are used or defined in FAR 12.212, DFARS Part 227.72, and DFARS 252.227-7014. Accordingly, the terms and conditions of this Agreement and only those rights specified in this Agreement, shall pertain to and govern the use, modification, reproduction, release, performance, display, and disclosure of the Program and Documentation by the federal government (or other entity acquiring for or through the federal government) and shall supersede any conflicting contractual terms or conditions. If this License fails to meet the government's needs or is inconsistent in any respect with federal procurement law, the government agrees to return the Program and Documentation, unused, to The MathWorks, Inc.

### **Trademarks**

MATLAB and Simulink are registered trademarks of The MathWorks, Inc. See [www.mathworks.com/trademarks](http://www.mathworks.com/trademarks) for a list of additional trademarks. Other product or brand names may be trademarks or registered trademarks of their respective holders.

### **Patents**

MathWorks products are protected by one or more U.S. patents. Please see [www.mathworks.com/patents](http://www.mathworks.com/patents) for more information.

## Revision History

|                |                  |                                           |
|----------------|------------------|-------------------------------------------|
| August 1993    | First printing   |                                           |
| October 1994   | Second printing  |                                           |
| May 1997       | Third printing   | Revised for Version 2                     |
| May 2000       | Fourth printing  | Minor changes                             |
| June 2001      | Fifth printing   | Minor changes                             |
| July 2002      | Online only      | Revised for Version 2.1.3 (Release 13)    |
| October 2002   | Online only      | Revised for Version 3.0.1                 |
| December 2002  | Sixth printing   |                                           |
| June 2004      | Seventh printing | Revised for Version 3.1 (Release 14)      |
| October 2004   | Online only      | Revised for Version 3.1.1 (Release 14SP1) |
| March 2005     | Online only      | Revised for Version 3.1.2 (Release 14SP2) |
| September 2005 | Online only      | Revised for Version 3.1.3 (Release 14SP3) |
| March 2006     | Online only      | Revised for Version 3.1.4 (Release 2006a) |
| September 2006 | Online only      | Revised for Version 3.1.5 (Release 2006b) |
| March 2007     | Online only      | Revised for Version 3.2 (Release 2007a)   |
| September 2007 | Online only      | Revised for Version 3.2.2 (Release 2007b) |
| March 2008     | Online only      | Revised for Version 3.2.3 (Release 2008a) |
| October 2008   | Online only      | Revised for Version 5.0 (Release 2008a+)  |
| October 2008   | Online only      | Revised for Version 5.1 (Release 2008b)   |
| November 2008  | Online only      | Revised for Version 4.9 (Release 2007b+)  |
| March 2009     | Online only      | Revised for Version 5.2 (Release 2009a)   |
| September 2009 | Online only      | Revised for Version 5.3 (Release 2009b)   |
| March 2010     | Online only      | Revised for Version 5.4 (Release 2010a)   |
| September 2010 | Online only      | Revised for Version 5.5 (Release 2010b)   |
| April 2011     | Online only      | Revised for Version 5.6 (Release 2011a)   |
| September 2011 | Online only      | Revised for Version 5.7 (Release 2011b)   |
| March 2012     | Online only      | Revised for Version 5.8 (Release 2012a)   |
| September 2012 | Online only      | Revised for Version 5.9 (Release 2012b)   |
| March 2013     | Online only      | Revised for Version 5.10 (Release 2013a)  |
| September 2013 | Online only      | Revised for Version 5.11 (Release 2013b)  |
| March 2014     | Online only      | Revised for Version 6.0 (Release 2014a)   |
| October 2014   | Online only      | Revised for Version 6.1 (Release 2014b)   |
| March 2015     | Online only      | Revised for Version 6.2 (Release 2015a)   |
| September 2015 | Online only      | Revised for Version 6.3 (Release 2015b)   |
| March 2016     | Online only      | Revised for Version 7.0 (Release 2016a)   |
| September 2016 | Online only      | Revised for Version 7.1 (Release 2016b)   |
| March 2017     | Online only      | Revised for Version 7.2 (Release 2017a)   |
| September 2017 | Online only      | Revised for Version 8.0 (Release 2017b)   |
| March 2018     | Online only      | Revised for Version 8.1 (Release 2018a)   |
| September 2018 | Online only      | Revised for Version 8.2 (Release 2018b)   |
| March 2019     | Online only      | Revised for Version 8.3 (Release 2019a)   |
| September 2019 | Online only      | Revised for Version 8.4 (Release 2019b)   |
| March 2020     | Online only      | Revised for Version 8.5 (Release 2020a)   |
| September 2020 | Online only      | Revised for Version 8.6 (Release 2020b)   |
| March 2021     | Online only      | Revised for Version 8.7 (Release 2021a)   |
| September 2021 | Online only      | Revised for Version 9.0 (Release 2021b)   |
| March 2022     | Online only      | Revised for Version 9.1 (Release 2022a)   |
| September 2022 | Online only      | Revised for Version 9.2 (Release 2022b)   |
| March 2023     | Online only      | Revised for Version 9.3 (Release 2023a)   |
| September 2023 | Online only      | Revised for Version 23.2 (R2023b)         |
| March 2024     | Online only      | Revised for Version 24.1 (R2024a)         |
| September 2024 | Online only      | Revised for Version 24.2 (R2024b)         |
| March 2025     | Online only      | Revised for Version 25.1 (R2025a)         |
| September 2025 | Online only      | Rereleased for Version 25.2 (R2025b)      |
| March 2026     | Online only      | Revised for Version 26.1 (R2026a)         |





## 1

## Getting Started

|                                                                          |             |
|--------------------------------------------------------------------------|-------------|
| <b>Symbolic Math Toolbox Product Description</b> .....                   | <b>1-2</b>  |
| <b>Create Symbolic Numbers, Variables, and Expressions</b> .....         | <b>1-3</b>  |
| <b>Create Symbolic Functions</b> .....                                   | <b>1-9</b>  |
| <b>Create Symbolic Matrices</b> .....                                    | <b>1-11</b> |
| Use Existing Symbolic Variables .....                                    | <b>1-11</b> |
| Generate Elements While Creating a Matrix .....                          | <b>1-11</b> |
| Create Matrix of Symbolic Numbers .....                                  | <b>1-12</b> |
| <b>Create Symbolic Matrix Variables</b> .....                            | <b>1-13</b> |
| <b>Use Symbolic Objects to Represent Mathematical Objects</b> .....      | <b>1-20</b> |
| Symbolic Number .....                                                    | <b>1-20</b> |
| Symbolic Scalar Variable, Function, and Expression .....                 | <b>1-21</b> |
| Symbolic Equation .....                                                  | <b>1-21</b> |
| Symbolic Vector and Matrix .....                                         | <b>1-22</b> |
| Symbolic Matrix Variable .....                                           | <b>1-23</b> |
| Symbolic Matrix Function .....                                           | <b>1-25</b> |
| Comparison of Symbolic Objects .....                                     | <b>1-26</b> |
| <b>Perform Symbolic Computations</b> .....                               | <b>1-29</b> |
| Differentiate Symbolic Expressions .....                                 | <b>1-29</b> |
| Integrate Symbolic Expressions .....                                     | <b>1-30</b> |
| Solve Equations .....                                                    | <b>1-31</b> |
| Simplify Symbolic Expressions .....                                      | <b>1-33</b> |
| Substitutions in Symbolic Expressions .....                              | <b>1-34</b> |
| Plot Symbolic Functions .....                                            | <b>1-36</b> |
| <b>Use Assumptions on Symbolic Variables</b> .....                       | <b>1-41</b> |
| <b>Computational Mathematics in Symbolic Math Toolbox</b> .....          | <b>1-44</b> |
| <b>Next Step Suggestions for Symbolic Workflows in Live Editor</b> ..... | <b>1-55</b> |

|                                                                                                     |             |
|-----------------------------------------------------------------------------------------------------|-------------|
| <b>Find Symbolic Variables in Expressions, Functions, and Matrices . . . . .</b>                    | <b>2-2</b>  |
| Find a Default Symbolic Variable . . . . .                                                          | 2-2         |
| <b>Choose syms or sym Function . . . . .</b>                                                        | <b>2-4</b>  |
| <b>Change Output Format of Symbolic and Variable-Precision Arithmetic . . . . .</b>                 | <b>2-8</b>  |
| <b>Change Output Display Format of Symbolic Results in the Live Editor . . . . .</b>                | <b>2-11</b> |
| <b>Add Subscripts, Superscripts, and Accents to Symbolic Variables in the Live Editor . . . . .</b> | <b>2-14</b> |
| <b>Copy and Paste Symbolic Output in Live Editor . . . . .</b>                                      | <b>2-18</b> |
| <b>Check Symbolic Equations, Inequalities, and Conditional Statements . . . . .</b>                 | <b>2-23</b> |
| <b>Numeric to Symbolic Conversion . . . . .</b>                                                     | <b>2-29</b> |
| <b>Choose Numeric or Symbolic Arithmetic . . . . .</b>                                              | <b>2-32</b> |
| Double-Precision Arithmetic . . . . .                                                               | 2-32        |
| Variable-Precision Arithmetic . . . . .                                                             | 2-32        |
| Symbolic Arithmetic . . . . .                                                                       | 2-33        |
| Comparisons of Numeric and Symbolic Arithmetic . . . . .                                            | 2-34        |
| <b>Increase Precision of Numeric Calculations . . . . .</b>                                         | <b>2-36</b> |
| <b>Compute Binomial Coefficients Exactly . . . . .</b>                                              | <b>2-38</b> |
| <b>Find Almost Integers with High-Precision Arithmetic . . . . .</b>                                | <b>2-41</b> |
| <b>Decimal Digits of PI . . . . .</b>                                                               | <b>2-45</b> |
| <b>Units of Measurement Tutorial . . . . .</b>                                                      | <b>2-47</b> |
| <b>Unit Conversions and Unit Systems . . . . .</b>                                                  | <b>2-53</b> |
| Convert Units . . . . .                                                                             | 2-53        |
| Temperature Unit Conversion . . . . .                                                               | 2-54        |
| Convert to SI, CGS, or US Unit Systems . . . . .                                                    | 2-54        |
| Define Custom Unit System from Existing System . . . . .                                            | 2-56        |
| Define Custom Unit System Directly . . . . .                                                        | 2-57        |
| Unit System Definition . . . . .                                                                    | 2-58        |
| <b>Units and Unit Systems List . . . . .</b>                                                        | <b>2-60</b> |
| Units List . . . . .                                                                                | 2-60        |
| SI Unit Prefixes List . . . . .                                                                     | 2-70        |
| Unit Systems List . . . . .                                                                         | 2-71        |
| Defining Constants of SI Units . . . . .                                                            | 2-72        |
| <b>Units in Physics Calculations . . . . .</b>                                                      | <b>2-74</b> |

|                                                                      |              |
|----------------------------------------------------------------------|--------------|
| <b>The Physics of the Damped Harmonic Oscillator . . . . .</b>       | <b>2-78</b>  |
| <b>Evaluate Average Power Delivered by Wind Turbine . . . . .</b>    | <b>2-87</b>  |
| <b>Developing an Algorithm for Undistorting an Image . . . . .</b>   | <b>2-93</b>  |
| <b>Electric Dipole Moment and Radiation Power . . . . .</b>          | <b>2-99</b>  |
| <b>Validate Simulink Model Using Symbolic Math Toolbox . . . . .</b> | <b>2-102</b> |

## Mathematics

### 3

|                                                                                |              |
|--------------------------------------------------------------------------------|--------------|
| <b>Solve Algebraic Equations . . . . .</b>                                     | <b>3-3</b>   |
| <b>Solve System of Algebraic Equations . . . . .</b>                           | <b>3-7</b>   |
| <b>Choose an Approach for Solving Equations Using solve Function . . . . .</b> | <b>3-16</b>  |
| <b>Solve Equations Numerically . . . . .</b>                                   | <b>3-28</b>  |
| <b>Solve System of Linear Equations . . . . .</b>                              | <b>3-36</b>  |
| <b>Select Numeric or Symbolic Solver . . . . .</b>                             | <b>3-39</b>  |
| <b>Solve Parametric Equations in ReturnConditions Mode . . . . .</b>           | <b>3-40</b>  |
| <b>Solve Algebraic Equation Using Live Editor Task . . . . .</b>               | <b>3-42</b>  |
| <b>Solve Differential Equation . . . . .</b>                                   | <b>3-49</b>  |
| <b>Solve a System of Differential Equations . . . . .</b>                      | <b>3-53</b>  |
| <b>Solve a Second-Order Differential Equation Numerically . . . . .</b>        | <b>3-57</b>  |
| <b>Solve Partial Differential Equation of Tsunami Model . . . . .</b>          | <b>3-59</b>  |
| <b>Solve Differential Algebraic Equations (DAEs) . . . . .</b>                 | <b>3-66</b>  |
| <b>Solve Semilinear DAE System . . . . .</b>                                   | <b>3-75</b>  |
| <b>Solve DAEs Using Mass Matrix Solvers . . . . .</b>                          | <b>3-82</b>  |
| <b>Analyze and Manipulate Differential Algebraic Equations . . . . .</b>       | <b>3-87</b>  |
| <b>Derive and Apply Inverse Kinematics to Two-Link Robot Arm . . . . .</b>     | <b>3-94</b>  |
| <b>Gauss-Laguerre Quadrature Evaluation Points and Weights . . . . .</b>       | <b>3-101</b> |
| <b>Simulate a Stochastic Process Using the Feynman-Kac Formula . . . . .</b>   | <b>3-111</b> |

|                                                                                  |              |
|----------------------------------------------------------------------------------|--------------|
| <b>The Black-Scholes Formula for Call Option Price</b> .....                     | <b>3-118</b> |
| <b>Choose Function to Rearrange Expression</b> .....                             | <b>3-123</b> |
| Combine Terms of Same Algebraic Structures .....                                 | <b>3-123</b> |
| Expand Expressions .....                                                         | <b>3-124</b> |
| Factor Expressions .....                                                         | <b>3-125</b> |
| Extract Subexpressions from Expression .....                                     | <b>3-127</b> |
| Collect Terms with Same Powers .....                                             | <b>3-127</b> |
| Rewrite Expressions in Terms of Other Functions .....                            | <b>3-128</b> |
| Compute Partial Fraction Decompositions of Expressions .....                     | <b>3-129</b> |
| Compute Normal Forms of Rational Expressions .....                               | <b>3-129</b> |
| Represent Polynomials Using Horner Nested Forms .....                            | <b>3-130</b> |
| <b>Extract Numerators and Denominators of Rational Expressions</b> .....         | <b>3-131</b> |
| <b>Simplify Symbolic Expressions</b> .....                                       | <b>3-133</b> |
| Simplify Using Options .....                                                     | <b>3-134</b> |
| Simplify Using Assumptions .....                                                 | <b>3-136</b> |
| Simplify Fractions .....                                                         | <b>3-136</b> |
| <b>Simplify Symbolic Expressions Using Live Editor Task</b> .....                | <b>3-138</b> |
| <b>Substitute Parameters in Symbolic Solutions</b> .....                         | <b>3-144</b> |
| <b>Substitute Elements in Symbolic Matrices</b> .....                            | <b>3-146</b> |
| <b>Substitute Scalars with Matrices</b> .....                                    | <b>3-148</b> |
| <b>Substitute Values into Symbolic Expressions Using subs</b> .....              | <b>3-150</b> |
| <b>Abbreviate Common Terms in Long Expressions</b> .....                         | <b>3-157</b> |
| <b>Harmonic Analysis of Transfer Function Output</b> .....                       | <b>3-160</b> |
| <b>Explore Single-Period Asset Arbitrage</b> .....                               | <b>3-166</b> |
| <b>Analytical Solutions of the Inverse Kinematics of a Humanoid Robot</b> .....  | <b>3-170</b> |
| <b>Differentiation</b> .....                                                     | <b>3-181</b> |
| <b>Integration</b> .....                                                         | <b>3-186</b> |
| Integration with Real Parameters .....                                           | <b>3-188</b> |
| Integration with Complex Parameters .....                                        | <b>3-189</b> |
| High-Precision Numerical Integration Using Variable-Precision Arithmetic .....   | <b>3-190</b> |
| <b>Taylor Series</b> .....                                                       | <b>3-192</b> |
| <b>Fourier and Inverse Fourier Transforms</b> .....                              | <b>3-194</b> |
| Fourier Transform Definition .....                                               | <b>3-194</b> |
| Concept: Using Symbolic Workflows .....                                          | <b>3-194</b> |
| Calculate Beam Deflection Using Fourier Transform .....                          | <b>3-194</b> |
| <b>Solve Differential Equations of RLC Circuit Using Laplace Transform</b> ..... | <b>3-198</b> |

|                                                                             |              |
|-----------------------------------------------------------------------------|--------------|
| <b>Solve Difference Equations Using Z-Transform</b> .....                   | <b>3-205</b> |
| <b>Symbolic Summation</b> .....                                             | <b>3-210</b> |
| Comparing symsum and sum .....                                              | <b>3-210</b> |
| Computational Speed of symsum versus sum .....                              | <b>3-210</b> |
| Output Format Differences Between symsum and sum .....                      | <b>3-211</b> |
| <b>Padé Approximant</b> .....                                               | <b>3-212</b> |
| <b>Limits</b> .....                                                         | <b>3-219</b> |
| One-Sided Limits .....                                                      | <b>3-219</b> |
| <b>Find Asymptotes, Critical, and Inflection Points</b> .....               | <b>3-222</b> |
| <b>Functional Derivatives Tutorial</b> .....                                | <b>3-226</b> |
| <b>Learn Calculus in the Live Editor</b> .....                              | <b>3-231</b> |
| <b>Differentiation</b> .....                                                | <b>3-236</b> |
| <b>Integration</b> .....                                                    | <b>3-243</b> |
| <b>Interactive Calculus in Live Editor</b> .....                            | <b>3-247</b> |
| <b>Maxima, Minima, and Inflection Points</b> .....                          | <b>3-249</b> |
| <b>Find Extremum of Multivariate Function and Its Approximation</b> .....   | <b>3-257</b> |
| <b>Padé Approximant of Time-Delay Input</b> .....                           | <b>3-261</b> |
| <b>Heating of Finite Slab</b> .....                                         | <b>3-267</b> |
| <b>Solve Partial Differential Equation of Nonlinear Heat Transfer</b> ..... | <b>3-275</b> |
| <b>Derive Equations of Motion and Simulate Cart-Pole System</b> .....       | <b>3-281</b> |
| <b>Finite Element Formulation for Timoshenko Beam Problem</b> .....         | <b>3-303</b> |
| <b>Analyze Transfer Function of T-Coil Circuit</b> .....                    | <b>3-324</b> |
| <b>Symbolic Matrix Computation</b> .....                                    | <b>3-334</b> |
| <b>Linear Algebraic Operations</b> .....                                    | <b>3-341</b> |
| Symbolic Hilbert Matrix .....                                               | <b>3-341</b> |
| Symbolic Linear Algebra Operations .....                                    | <b>3-341</b> |
| Variable-Precision Arithmetic .....                                         | <b>3-342</b> |
| Symbolic Investigation of Singular Value .....                              | <b>3-343</b> |
| <b>Basic Algebraic Operations</b> .....                                     | <b>3-345</b> |
| <b>Singular Value Decomposition</b> .....                                   | <b>3-347</b> |
| <b>Eigenvalues</b> .....                                                    | <b>3-349</b> |

|                                                                |              |
|----------------------------------------------------------------|--------------|
| <b>Jordan Canonical Form</b> .....                             | <b>3-353</b> |
| <b>Eigenvalues of the Laplace Operator</b> .....               | <b>3-355</b> |
| <b>Hilbert Matrices and Their Inverses</b> .....               | <b>3-365</b> |
| <b>Markov Chain Analysis and Stationary Distribution</b> ..... | <b>3-369</b> |
| <b>Matrix Rotations and Transformations</b> .....              | <b>3-374</b> |
| <b>Clear Assumptions and Reset the Symbolic Engine</b> .....   | <b>3-379</b> |
| Check Assumptions Set on Variables .....                       | <b>3-380</b> |
| Effects of Assumptions on Computations .....                   | <b>3-381</b> |
| <b>Recognize and Avoid Round-Off Errors</b> .....              | <b>3-382</b> |
| Use Symbolic Computations When Possible .....                  | <b>3-382</b> |
| Perform Calculations with Increased Precision .....            | <b>3-382</b> |
| Compare Symbolic and Numeric Results .....                     | <b>3-384</b> |
| Plot the Function or Expression .....                          | <b>3-384</b> |
| <b>Increase Speed by Reducing Precision</b> .....              | <b>3-386</b> |
| <b>Prime Factorizations</b> .....                              | <b>3-388</b> |
| <b>Handling Large Integers to Solve the 196 Problem</b> .....  | <b>3-391</b> |

## Graphics

### 4

|                                                                                             |             |
|---------------------------------------------------------------------------------------------|-------------|
| <b>Create Plots of Symbolic Expressions</b> .....                                           | <b>4-2</b>  |
| <b>Transform Spherical Coordinates to Cartesian Coordinates and Plot Analytically</b> ..... | <b>4-10</b> |
| <b>Tangent Plane and Normal Line of Implicit Surface</b> .....                              | <b>4-17</b> |
| <b>Analytical Plotting with Symbolic Math Toolbox</b> .....                                 | <b>4-21</b> |
| <b>Simulate the Motion of the Periodic Swing of a Pendulum</b> .....                        | <b>4-35</b> |
| <b>Animation and Solution of Double Pendulum Motion</b> .....                               | <b>4-47</b> |
| <b>Animation and Model of Automotive Piston</b> .....                                       | <b>4-54</b> |

## Code Generation

### 5

|                                                                   |            |
|-------------------------------------------------------------------|------------|
| <b>Generate C or Fortran Code from Symbolic Expressions</b> ..... | <b>5-2</b> |
|-------------------------------------------------------------------|------------|

|                                                                                                                       |             |
|-----------------------------------------------------------------------------------------------------------------------|-------------|
| <b>Generate MATLAB Functions from Symbolic Expressions</b> .....                                                      | <b>5-3</b>  |
| Generating a Function Handle .....                                                                                    | <b>5-3</b>  |
| Control the Order of Variables .....                                                                                  | <b>5-3</b>  |
| Generate a File .....                                                                                                 | <b>5-4</b>  |
| Name Output Variables .....                                                                                           | <b>5-5</b>  |
| <b>Generate MATLAB Function Blocks from Symbolic Expressions</b> .....                                                | <b>5-6</b>  |
| Generate and Edit a Block .....                                                                                       | <b>5-6</b>  |
| Control the Order of Input Ports .....                                                                                | <b>5-6</b>  |
| Name the Output Ports .....                                                                                           | <b>5-7</b>  |
| <b>Generate Simscape Equations from Symbolic Expressions</b> .....                                                    | <b>5-8</b>  |
| Convert Algebraic and Differential Equations .....                                                                    | <b>5-8</b>  |
| Limitations .....                                                                                                     | <b>5-9</b>  |
| <b>Deploy Generated MATLAB Functions from Symbolic Expressions with<br/>    MATLAB Compiler</b> .....                 | <b>5-10</b> |
| <b>Generate C Code from Symbolic Expressions Using the MATLAB Coder<br/>    App</b> .....                             | <b>5-15</b> |
| <b>Derive Quadrotor Dynamics for Nonlinear Model Predictive Control</b> ...                                           | <b>5-24</b> |
| <b>Autonomous Underwater Vehicle Pose Estimation Using Analytical<br/>    Jacobians in Custom Sensor Models</b> ..... | <b>5-31</b> |
| <b>Using Symbolic Mathematics with Optimization Toolbox Solvers</b> .....                                             | <b>5-43</b> |
| <b>Improving Accuracy and Performance in Optimization</b> .....                                                       | <b>5-52</b> |
| <b>Analytical Model of Cantilever Truss Structure for Simscape</b> .....                                              | <b>5-57</b> |
| <b>Customize and Extend Simscape Libraries for a Custom DC Motor</b> ....                                             | <b>5-65</b> |
| <b>Estimate Model Parameters of a Symbolically Derived Plant Model in<br/>    Simulink</b> .....                      | <b>5-74</b> |

## Functions





# Getting Started

---

- “Symbolic Math Toolbox Product Description” on page 1-2
- “Create Symbolic Numbers, Variables, and Expressions” on page 1-3
- “Create Symbolic Functions” on page 1-9
- “Create Symbolic Matrices” on page 1-11
- “Create Symbolic Matrix Variables” on page 1-13
- “Use Symbolic Objects to Represent Mathematical Objects” on page 1-20
- “Perform Symbolic Computations” on page 1-29
- “Use Assumptions on Symbolic Variables” on page 1-41
- “Computational Mathematics in Symbolic Math Toolbox” on page 1-44
- “Next Step Suggestions for Symbolic Workflows in Live Editor” on page 1-55

## **Symbolic Math Toolbox Product Description**

### **Perform symbolic math computations**

Symbolic Math Toolbox provides functions for solving, plotting, and manipulating symbolic math equations. You can create, run, and share symbolic math code. In the MATLAB Live Editor, you can get next-step suggestions for symbolic workflows. The toolbox provides functions in common mathematical areas such as calculus, linear algebra, algebraic and differential equations, equation simplification, and equation manipulation.

Symbolic Math Toolbox lets you analytically perform differentiation, integration, simplification, transforms, and equation solving. You can perform dimensional computations and convert between units. Your computations can be performed either analytically or using variable-precision arithmetic, with the results displayed in mathematical typeset.

You can share your symbolic work with other MATLAB users as live scripts or convert them to HTML, Word, LaTeX, or PDF documents. You can generate MATLAB functions, Simulink<sup>®</sup> function blocks, and Simscape<sup>™</sup> equations directly from symbolic expressions.

## Create Symbolic Numbers, Variables, and Expressions

This example shows how to create symbolic numbers, variables, and expressions. To learn how to work with symbolic math, see “Perform Symbolic Computations” on page 1-29.

### Create Symbolic Numbers with Exact Representations

You can create symbolic numbers by using `sym`. Symbolic numbers are exact representations, unlike floating-point numbers.

Create symbolic numbers by using `sym` and compare them to the same floating-point numbers.

```
a1Sym = sym(1/3)
```

```
a1Sym =  
      1  
      3
```

```
a1 = 1/3
```

```
a1 =  
0.3333
```

```
a2Sym = sym(pi)
```

```
a2Sym =  $\pi$ 
```

```
a2 = pi
```

```
a2 =  
3.1416
```

The symbolic numbers are represented in exact rational form, while the floating-point numbers are decimal approximations.

Calculations on symbolic numbers are exact. Demonstrate this exactness by finding `sin(pi)` symbolically and numerically. The symbolic result is exact, while the numeric result is an approximation.

```
bSym = sin(sym(pi))
```

```
bSym = 0
```

```
b = sin(pi)
```

```
b =  
1.2246e-16
```

When you use `sym` on a numeric input, the numeric expression is first evaluated to the MATLAB® default double-precision number that can be less accurate. Then, `sym` is applied on that double-precision number. To represent an exact number without evaluating it to double precision, use a character vector with quotes. For example, create a symbolic number to represent a very large integer exactly.

```
inaccurateNum = sym(123456789012345678)
```

```
inaccurateNum = 123456789012345680
```

```
accurateNum = sym("123456789012345678")  
accurateNum = 123456789012345678
```

You can also create symbolic complex numbers, by specifying the imaginary part of a number as `1i`, `2i`, and so on.

```
sym("1234567 + 1i")  
ans = 1234567 + i
```

To learn more about symbolic representation of numbers, see “Numeric to Symbolic Conversion” on page 2-29.

### Create Symbolic Numbers with Variable Precision

You can create symbolic numbers with variable-precision floating-point arithmetic by using `vpa`. By default, `vpa` calculates values to 32 significant digits.

```
piVpa = vpa(pi)  
piVpa = 3.1415926535897932384626433832795
```

When you use `vpa` on a numeric expression, such as `log(2)`, the expression is first evaluated to the MATLAB default double-precision number that has less than 32 significant digits. Then, `vpa` is applied on that double-precision number, which can be less accurate. For more accurate results, convert double-precision numbers in an expression to symbolic numbers with `sym` and then use `vpa` to evaluate the results with variable precision. For example, find `log(2)` with 17- and 20- digit precision.

```
vpaOnDouble = vpa(log(2))  
vpaOnDouble = 0.69314718055994528622676398299518  
vpaOnSym_17 = vpa(log(sym(2)),17)  
vpaOnSym_17 = 0.69314718055994531  
vpaOnSym_20 = vpa(log(sym(2)),20)  
vpaOnSym_20 = 0.69314718055994530942
```

When you convert large numbers, use quotes to represent them exactly.

```
inaccurateNum = vpa(123456789012345678)  
inaccurateNum = 123456789012345680.0  
accurateNum = vpa("123456789012345678")  
accurateNum = 123456789012345678.0
```

### Create Symbolic Variables

You can create symbolic variables using either `syms` or `sym`. Typical uses of these functions include:

- `sym` - Create numbered symbolic variables, symbolic variables in MATLAB functions, or symbolic numbers whose values differ from their names in the MATLAB workspace.

- `syms` – Create *fresh* symbolic variables for interactive symbolic workflows, that is, for symbolic variable creation at the MATLAB command line or in MATLAB live scripts. A *fresh* symbolic variable does not have any assumptions.

The `syms` command is shorthand for the `sym` syntax, but the two functions handle assumptions differently. `syms` clears the assumptions when creating variables. However, recreating a variable using `sym` does not clear its assumptions. For more details about the differences of these two functions, see “Choose `syms` or `sym` Function” on page 2-4.

Create the symbolic variables `x` and `y` using `syms` and `sym`, respectively.

```
syms x
y = sym("y")

y = y
```

The first command creates a symbolic variable `x` in the MATLAB workspace with the value `x` assigned to the variable `x`. The second command creates a symbolic variable `y` with the value `y`.

With `syms`, you can create multiple variables in one command. Create the variables `a`, `b`, and `c`.

```
syms a b c
```

### Create Array of Symbolic Variables

If you want to create a MATLAB array of numbered symbolic variables, you can use the `sym` or the `syms` syntax.

Use `sym` to create an array of many numbered symbolic variables. Clear the workspace. Create a row vector containing the symbolic variables  $a_1, \dots, a_{10}$  and assign it to the MATLAB variable `A`. Display the variable in the MATLAB workspace.

```
clear
A = sym("a",[1 10])

A = (a1 a2 a3 a4 a5 a6 a7 a8 a9 a10)
```

```
whos
```

| Name | Size | Bytes | Class | Attributes |
|------|------|-------|-------|------------|
| A    | 1x10 | 8     | sym   |            |

`A` is a 1-by-10 array of 10 automatically generated elements. These generated elements of `A` do not appear in the MATLAB workspace.

Use `syms` to create many fresh symbolic variables with corresponding variable names in the MATLAB workspace. Clear the workspace. Create the fresh symbolic variables `a1`, ..., `a10`. Display the variables in the MATLAB workspace.

```
clear
syms a [1 10]
whos
```

| Name | Size | Bytes | Class | Attributes |
|------|------|-------|-------|------------|
| a    | 1x10 | 8     | sym   |            |

|     |     |   |     |
|-----|-----|---|-----|
| a1  | 1x1 | 8 | sym |
| a10 | 1x1 | 8 | sym |
| a2  | 1x1 | 8 | sym |
| a3  | 1x1 | 8 | sym |
| a4  | 1x1 | 8 | sym |
| a5  | 1x1 | 8 | sym |
| a6  | 1x1 | 8 | sym |
| a7  | 1x1 | 8 | sym |
| a8  | 1x1 | 8 | sym |
| a9  | 1x1 | 8 | sym |

The MATLAB workspace contains 10 MATLAB variables in the workspace that are symbolic variables.

The `syms` command is a convenient shorthand for the `sym` syntax, and its typical use is to create fresh symbolic variables for interactive symbolic workflows. Use the `sym` syntax to create the following:

- Symbolic variables in MATLAB functions
- Many numbered symbolic variables
- Symbolic variable whose value differs from its name in the MATLAB workspace
- Symbolic number, such as `sym(3)/11`
- Symbolic variable that inherits the assumptions from a previously used symbolic variable having the same name

### Create Symbolic Expressions

Suppose you want to use a symbolic variable to represent the golden ratio  $\varphi = \frac{1+\sqrt{5}}{2}$ .

Use `sym` to create the golden ratio.

```
phi = (1 + sqrt(sym(5)))/2;
```

Now you can perform various mathematical operations on `phi`. For example:

```
f = phi^2 - phi - 1
```

```
f =  

$$\left(\frac{\sqrt{5}}{2} + \frac{1}{2}\right)^2 - \frac{\sqrt{5}}{2} - \frac{3}{2}$$

```

Next, suppose you want to study the quadratic function  $f = ax^2 + bx + c$ . First, create the symbolic variables `a`, `b`, `c`, and `x`.

```
syms a b c x
```

Then, create a symbolic expression `f` that represents the arithmetical expression  $ax^2 + bx + c$ .

```
f = a*x^2 + b*x + c
```

```
f =  $ax^2 + bx + c$ 
```

Solve the quadratic equation  $f = 0$  for `x` by using `solve`.

```
x_0 = solve(f == 0,x)
```

```
x_0 =
```

$$\left( \begin{array}{c} -\frac{b + \sqrt{b^2 - 4ac}}{2a} \\ -\frac{b - \sqrt{b^2 - 4ac}}{2a} \end{array} \right)$$

You can also apply a mathematical function to an arithmetical expression. For example, apply the Bessel function of the first kind  $J_0$  to the arithmetical expression  $f$  and find its derivative with respect to  $x$ .

```
J_0 = besselj(0,f)
```

```
J_0 = J0(a x^2 + b x + c)
```

```
DJ_0 = diff(J_0,x)
```

```
DJ_0 = -J1(a x^2 + b x + c) (b + 2 a x)
```

To create a symbolic number in a symbolic expression, use `sym`. Do not use `syms` to create a symbolic expression that is a constant. For example, to create an expression whose value is 5, enter `f = sym(5)`. The command `f = 5` does not define `f` as a symbolic expression.

You can also create symbolic expressions from strings by using `str2sym` when reading expressions from text files or when specifying numbers exactly.

### Reuse Names of Symbolic Objects

If you set a variable equal to a symbolic expression and then apply the `syms` command to the variable, MATLAB removes the previously defined expression from the variable.

For example, create a symbolic expression `f`.

```
syms a b
f = a + b
```

```
f = a + b
```

If you recreate `f`, then MATLAB removes the value `a + b` from the expression `f`.

```
syms f
f
```

```
f = f
```

You can use the `syms` command to clear variables of definitions that you previously assigned to them in your MATLAB session. `syms` clears the assumptions of the variables. These assumptions (which can be real, integer, rational, and positive) are stored separately from the symbolic object. However, recreating a variable using `sym` does not clear its assumptions. For more information, see “Use Assumptions on Symbolic Variables” on page 1-41.

### See Also

`sym` | `syms` | `vpa`

## More About

- “Create Symbolic Functions” on page 1-9
- “Create Symbolic Matrices” on page 1-11
- “Create Symbolic Matrix Variables” on page 1-13
- “Use Symbolic Objects to Represent Mathematical Objects” on page 1-20
- “Choose syms or sym Function” on page 2-4
- “Perform Symbolic Computations” on page 1-29
- “Choose Numeric or Symbolic Arithmetic” on page 2-32
- “Add Subscripts, Superscripts, and Accents to Symbolic Variables in the Live Editor” on page 2-14



## Create Symbolic Functions

Symbolic functions represent math functions. Use symbolic functions for differentiation, integration, solving ODEs, and other math operations. Create symbolic functions by using `syms`.

---

**Note** Symbolic functions must be functions of symbolic variables. The Symbolic Math Toolbox currently does not support composite symbolic functions, or symbolic functions that are functions of another symbolic functions.

---

Create a symbolic function `f` with variables `x` and `y` by using `syms`. Creating `f` automatically creates `x` and `y`.

```
syms f(x,y)
```

Assign a mathematical expression to `f`.

```
f(x,y) = x^2*y
```

```
f(x, y) =  
x^2*y
```

Find the value of `f` at `(3,2)`.

```
f(3,2)
```

```
ans =  
18
```

Symbolic functions accept array inputs. Calculate `f` for multiple values of `x` and `y`.

```
xVal = 1:5;  
yVal = 3:7;  
f(xVal,yVal)
```

```
ans =  
[ 3, 16, 45, 96, 175]
```

You can differentiate symbolic functions, integrate or simplify them, substitute their arguments with values, and perform other mathematical operations. For example, find the derivative of `f(x,y)` with respect to `x`. The result `dfx` is also a symbolic function.

```
dfx = diff(f,x)
```

```
dfx(x,y) =  
2*x*y
```

Calculate `df(x,y)` at `x = y + 1`.

```
dfx(y+1,y)
```

```
ans =  
2*y*(y + 1)
```

If you are creating a constant function, such as `f(x,y) = 1`, you must first create `f(x,y)`. If you do not create `f(x,y)`, then the assignment `f(x,y) = 1` throws an error.

### See Also

### More About

- “Create Symbolic Numbers, Variables, and Expressions” on page 1-3
- “Create Symbolic Matrices” on page 1-11
- “Create Symbolic Matrix Variables” on page 1-13
- “Use Symbolic Objects to Represent Mathematical Objects” on page 1-20
- “Perform Symbolic Computations” on page 1-29
- “Use Assumptions on Symbolic Variables” on page 1-41

## Create Symbolic Matrices

### In this section...

“Use Existing Symbolic Variables” on page 1-11

“Generate Elements While Creating a Matrix” on page 1-11

“Create Matrix of Symbolic Numbers” on page 1-12

### Use Existing Symbolic Variables

A circulant matrix has the property that each row is obtained from the previous one by cyclically permuting the entries one step forward. For example, create the symbolic circulant matrix whose elements are *a*, *b*, and *c*, using the commands:

```
syms a b c
A = [a b c; c a b; b c a]

A =
[ a, b, c]
[ c, a, b]
[ b, c, a]
```

Since matrix *A* is circulant, the sum of elements over each row and each column is the same. Find the sum of all the elements of the first row:

```
sum(A(1,:))

ans =
a + b + c
```

To check if the sum of the elements of the first row equals the sum of the elements of the second column, use the `isAlways` function:

```
isAlways(sum(A(1,:)) == sum(A(:,2)))
```

The sums are equal:

```
ans =
    logical
     1
```

From this example, you can see that using symbolic objects is very similar to using regular MATLAB numeric objects.

### Generate Elements While Creating a Matrix

The `sym` function also lets you define a symbolic matrix or vector without having to define its elements in advance. In this case, the `sym` function generates the elements of a symbolic matrix at the same time that it creates a matrix. The function presents all generated elements using the same form: the base (which must be a valid variable name), a row index, and a column index. Use the first argument of `sym` to specify the base for the names of generated elements. You can use any valid variable name as a base. To check whether the name is a valid variable name, use the `isvarname` function. By default, `sym` separates a row index and a column index by underscore. For example, create the 2-by-4 matrix *A* with the elements *A*<sub>1,1</sub>, ..., *A*<sub>2,4</sub>:

```
A = sym('A', [2 4])
```

```
A =  
[ A1_1, A1_2, A1_3, A1_4]  
[ A2_1, A2_2, A2_3, A2_4]
```

To control the format of the generated names of matrix elements, use %d in the first argument:

```
A = sym('A%d%d', [2 4])
```

```
A =  
[ A11, A12, A13, A14]  
[ A21, A22, A23, A24]
```

## Create Matrix of Symbolic Numbers

A particularly effective use of `sym` is to convert a matrix from numeric to symbolic form. The command

```
A = hilb(3)
```

generates the 3-by-3 Hilbert matrix:

```
A =  
    1.0000    0.5000    0.3333  
    0.5000    0.3333    0.2500  
    0.3333    0.2500    0.2000
```

By applying `sym` to `A`

```
A = sym(A)
```

you can obtain the precise symbolic form of the 3-by-3 Hilbert matrix:

```
A =  
[ 1, 1/2, 1/3]  
[ 1/2, 1/3, 1/4]  
[ 1/3, 1/4, 1/5]
```

For more information on numeric to symbolic conversions, see “Numeric to Symbolic Conversion” on page 2-29.

## See Also

### More About

- “Create Symbolic Numbers, Variables, and Expressions” on page 1-3
- “Create Symbolic Functions” on page 1-9
- “Create Symbolic Matrix Variables” on page 1-13
- “Use Symbolic Objects to Represent Mathematical Objects” on page 1-20
- “Perform Symbolic Computations” on page 1-29
- “Use Assumptions on Symbolic Variables” on page 1-41

## Create Symbolic Matrix Variables

Since R2021a

Symbolic matrix variables represent matrices, vectors, and scalars in compact matrix notation. When mathematical formulas involve matrices and vectors, writing them using symbolic matrix variables is more concise and clear than writing them componentwise. When you do this, you can take vector-based expressions and equations from textbooks, enter them in Symbolic Math Toolbox™, perform mathematical operations on them, and derive further equations from them.

Derived equations involving symbolic matrix variables are displayed in typeset as they would be in textbooks. For example, create three symbolic matrix variables  $A$ ,  $x$ , and  $y$  by using `syms`. Find the differential of the expression  $y^T A x$  with respect to the vector  $x$ .

```
syms A [3 4] matrix
syms x [4 1] matrix
syms y [3 1] matrix
eq = y.'*A*x
```

```
eq =  $y^T A x$ 
```

```
D = diff(eq,x)
```

```
D =  $y^T A$ 
```

### Comparison Between Matrix of Symbolic Scalar Variables and Symbolic Matrix Variables

Symbolic matrix variables are an alternative to symbolic scalar variables. The two options are of different types and displayed differently.

For example, create two 2-by-3 matrices of symbolic scalar variables by using `syms`. For brevity, matrices of symbolic scalar variables are sometimes called *symbolic matrices*. These matrices are displayed by listing their components.

```
syms A B [2 3]
A
```

```
A =
 $\begin{pmatrix} A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,1} & A_{2,2} & A_{2,3} \end{pmatrix}$ 
```

```
B
```

```
B =
 $\begin{pmatrix} B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,1} & B_{2,2} & B_{2,3} \end{pmatrix}$ 
```

A matrix of symbolic scalar variables is of type `sym`.

```
class(A)
```

```
ans =
'sym'
```

Applying symbolic math operations to these matrices can result in a complex solution expressed in terms of the matrix components. For example, multiply the matrices  $A$  and  $B'$ .

```
C = A*B'
```

```
C =
```

$$\begin{pmatrix} A_{1,1}\overline{B_{1,1}} + A_{1,2}\overline{B_{1,2}} + A_{1,3}\overline{B_{1,3}} & A_{1,1}\overline{B_{2,1}} + A_{1,2}\overline{B_{2,2}} + A_{1,3}\overline{B_{2,3}} \\ A_{2,1}\overline{B_{1,1}} + A_{2,2}\overline{B_{1,2}} + A_{2,3}\overline{B_{1,3}} & A_{2,1}\overline{B_{2,1}} + A_{2,2}\overline{B_{2,2}} + A_{2,3}\overline{B_{2,3}} \end{pmatrix}$$

To create symbolic matrix variables of the same size, use the `syms` command followed by the variable names, their size, and the `matrix` keyword. Symbolic matrix variables are displayed in bold to distinguish them from symbolic scalar variables.

```
syms A B [2 3] matrix
A
```

```
A = A
```

```
B
```

```
B = B
```

Symbolic matrix variables are of type `symmatrix`.

```
class(A)
```

```
ans =
'symmatrix'
```

Applying symbolic math operations to symbolic matrix variables results in a concise display. For example, multiply  $A$  and  $B'$ .

```
C = A*B'
```

```
C = A(\overline{B})^T
```

## Mathematical Operations with Symbolic Matrix Variables

Symbolic matrix variables are recognized as noncommutative objects. They support common math operations, and you can use these operations to build symbolic matrix variable expressions.

```
syms A B [2 2] matrix
A*B - B*A
```

```
ans = AB - BA
```

For example, check the commutation relation for multiplication between two symbolic matrix variables.

```
isequal(A*B,B*A)
```

```
ans = logical
0
```

Check the commutation relation for addition.

```
isequal(A+B,B+A)
```

```
ans = logical
      1
```

If an operation has any arguments of type `symmatrix`, the result is automatically converted to type `symmatrix`. For example, multiply a matrix `A` that is represented by symbolic matrix variable and a scalar `c` that is represented by symbolic scalar variable. The result is of type `symmatrix`.

```
syms A [2 2] matrix
syms c
class(A)
```

```
ans =
'symmatrix'
```

```
class(c)
```

```
ans =
'sym'
```

```
M = c*A
```

```
M = c A
```

```
class(M)
```

```
ans =
'symmatrix'
```

Multiply three matrices that are represented by symbolic matrix variables. The result `X` is a `symmatrix` object.

```
syms V [2 1] matrix
X = V.'*A*V
```

```
X = VT A V
```

```
class(X)
```

```
ans =
'symmatrix'
```

You can pass `symmatrix` objects as arguments to math functions. For example, perform a mathematical operation to `X` by taking the differential of `X` with respect to `V`.

```
diff(X,V)
```

```
ans = VT AT + VT A
```

### Create Symbolic Matrix Variable from Array of Symbolic Scalar Variables

You can convert an array of symbolic scalar variables to a single symbolic matrix variable using the `symmatrix` function. Symbolic matrix variables that are converted in this way are displayed elementwise.

```
syms A [3 4]
class(A)
```

```
ans =
'sym'
```

```
B = symmatrix(A)
```

```
B =
```

$$\Sigma_1$$

where

$$\Sigma_1 = \begin{pmatrix} A_{1,1} & A_{1,2} & A_{1,3} & A_{1,4} \\ A_{2,1} & A_{2,2} & A_{2,3} & A_{2,4} \\ A_{3,1} & A_{3,2} & A_{3,3} & A_{3,4} \end{pmatrix}$$

```
class(B)
```

```
ans =
```

```
'symmatrix'
```

### Convert Symbolic Matrix Variable into Array of Symbolic Scalar Variables

You can create symbolic matrix variables, derive equations, and then convert the result to arrays of symbolic scalar variables using the `symmatrix2sym` function.

For example, find the matrix product of two symbolic matrix variables A and B. The result X is of type `symmatrix`.

```
syms A B [2 2] matrix
```

```
X = A*B
```

```
X = AB
```

```
class(X)
```

```
ans =
```

```
'symmatrix'
```

Convert the symbolic matrix variable X to array of symbolic scalar variables. The converted matrix Y is of type `sym`.

```
Y = symmatrix2sym(X)
```

```
Y =
```

$$\begin{pmatrix} A_{1,1} B_{1,1} + A_{1,2} B_{2,1} & A_{1,1} B_{1,2} + A_{1,2} B_{2,2} \\ A_{2,1} B_{1,1} + A_{2,2} B_{2,1} & A_{2,1} B_{1,2} + A_{2,2} B_{2,2} \end{pmatrix}$$

```
class(Y)
```

```
ans =
```

```
'sym'
```

Check that the product obtained by converting symbolic matrix variables is equal to the product of two arrays of symbolic scalar variables.

```
syms A B [2 2]
```

```
isequal(Y,A*B)
```

```
ans = logical
```

```
1
```



## Indexing into Symbolic Matrix Variables

Indexing into a symbolic matrix variable returns corresponding matrix elements in the form of another symbolic matrix variable.

```
syms A [2 3] matrix
a = A(2,3)
```

```
a =  $A_{2,3}$ 
```

```
class(a)
```

```
ans =
'symmatrix'
```

Alternatively, convert the symbolic matrix variable A to a matrix of symbolic scalar variables. Then, index into that matrix.

```
Asym = symmatrix2sym(A)
```

```
Asym =

$$\begin{pmatrix} A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,1} & A_{2,2} & A_{2,3} \end{pmatrix}$$

```

```
asym = Asym(2,3)
```

```
asym =  $A_{2,3}$ 
```

```
class(asym)
```

```
ans =
'sym'
```

Note that both results are equal.

```
isequal(a,symmatrix(asym))
```

```
ans = logical
      1
```

## Display of Operations Involving Symbolic Matrix Variables

Matrices like those returned by `eye`, `zeros`, and `ones` often have special meaning with specific notation in symbolic workflows. Declaring these matrices as symbolic matrix variables display the matrices in bold along with the matrix dimensions.

```
symmatrix(eye(3))
```

```
ans =  $I_3$ 
```

```
symmatrix(zeros(2,3))
```

```
ans =  $0_{2,3}$ 
```

```
symmatrix(ones(3,5))
```

```
ans =  $1_{3,5}$ 
```

If the inputs to a componentwise operation in MATLAB® are symbolic matrix variables, so is the output. These operations are displayed in special notations which follow conventions from textbooks.

```
syms A B [3 3] matrix
A.*B
```

```
ans =  $A \odot B$ 
```

```
A./B
```

```
ans =  $A \oslash B$ 
```

```
A.\B
```

```
ans =  $B \oslash A$ 
```

```
A.*hilb(3)
```

```
ans =  
 $A \odot \Sigma_1$ 
```

```
where
```

$$\Sigma_1 = \begin{pmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \end{pmatrix}$$

```
A.^(2*ones(3))
```

```
ans =  $A^{\circ 2 \mathbf{1}_{3,3}}$ 
```

```
A.^B
```

```
ans =  $A^{\circ B}$ 
```

```
kron(A,B)
```

```
ans =  $A \otimes B$ 
```

```
adjoint(A)
```

```
ans =  $\text{adj}(A)$ 
```

```
trace(A)
```

```
ans =  $\text{Tr}(A)$ 
```

## See Also

syms | symmatrix | symmatrix2sym

## More About

- “Create Symbolic Numbers, Variables, and Expressions” on page 1-3
- “Create Symbolic Functions” on page 1-9

- “Create Symbolic Matrices” on page 1-11
- “Use Symbolic Objects to Represent Mathematical Objects” on page 1-20
- “Perform Symbolic Computations” on page 1-29

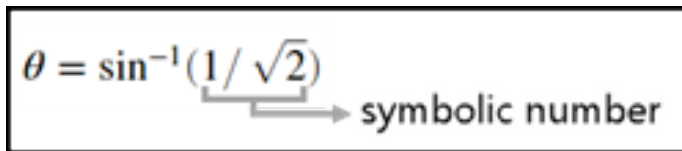
## Use Symbolic Objects to Represent Mathematical Objects

To solve mathematical problems with Symbolic Math Toolbox, you can define symbolic objects to represent various mathematical objects. This example discusses the usage of these symbolic objects in the Command Window:

- symbolic number
- symbolic scalar variable, function, and expression
- symbolic equation
- symbolic vector and matrix
- symbolic matrix variable
- symbolic matrix function

### Symbolic Number

Defining a number as a symbolic number instructs MATLAB to treat the number as an exact form instead of using a numeric approximation. For example, use a symbolic number to represent the argument of an inverse trigonometric function  $\theta = \sin^{-1}(1/\sqrt{2})$ .


$$\theta = \sin^{-1}\left(\frac{1}{\sqrt{2}}\right) \rightarrow \text{symbolic number}$$

Create the symbolic number  $1/\sqrt{2}$  using `sym`, and assign it to `a`.

```
a = sym(1/sqrt(2))
```

```
a =  
2^(1/2)/2
```

Find the inverse sine of `a`. The result is the symbolic number `pi/4`.

```
thetaSym = asin(a)
```

```
thetaSym =  
pi/4
```

You can convert a symbolic number to variable-precision arithmetic by using `vpa`. The result is a decimal number with 32 significant digits.

```
thetaVpa = vpa(thetaSym)
```

```
thetaVpa =  
0.78539816339744830961566084581988
```

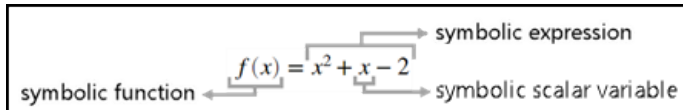
To convert the symbolic number to a double-precision number, use `double`. For more information about whether to use numeric or symbolic arithmetic, see “Choose Numeric or Symbolic Arithmetic” on page 2-32.

```
thetaDouble = double(thetaSym)
```

```
thetaDouble =
0.7854
```

## Symbolic Scalar Variable, Function, and Expression

Defining variables, functions, and expressions as symbolic objects enables you to perform algebraic operations with those symbolic objects, including simplifying formulas and solving equations. For example, use a symbolic scalar variable, function, and expression to represent the quadratic function  $f(x) = x^2 + x - 2$ . For brevity, a symbolic scalar variable is also called a *symbolic variable*.



Create a symbolic scalar variable  $x$  using `syms`. You can also use `sym` to create a symbolic scalar variable. For more information about whether to use `syms` or `sym`, see “Choose `syms` or `sym` Function” on page 2-4.

Define a *symbolic expression*  $x^2 + x - 2$  to represent the right side of the quadratic equation and assign it to  $f(x)$ . The identifier  $f(x)$  now refers to a *symbolic function* that represents the quadratic function. A symbolic function accepts scalars as input arguments.

```
syms x
f(x) = x^2 + x - 2

f(x) =
x^2 + x - 2
```

You can then evaluate the quadratic function by providing its input argument inside the parentheses. For example, evaluate  $f(2)$ .

```
fVal = f(2)

fVal =
4
```

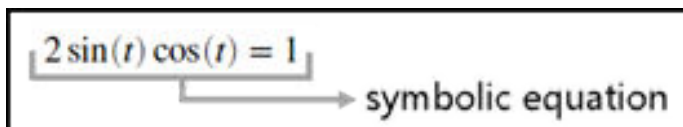
You can also solve the quadratic equation  $f(x) = 0$ . Use `solve` to find the roots of the quadratic equation. `solve` returns the two solutions as a vector of two symbolic numbers.

```
sols = solve(f)

sols =
-2
1
```

## Symbolic Equation

Defining a mathematical equation as a symbolic equation enables you to find the solution of the equation. For example, use a symbolic equation to solve the trigonometric problem  $2\sin(t)\cos(t) = 1$ .



Create a symbolic function  $g(t)$  using `syms`. Assign the symbolic expression  $2*\sin(t)*\cos(t)$  to  $g(t)$ .

```
syms g(t)
g(t) = 2*sin(t)*cos(t)

g(t) =
2*cos(t)*sin(t)
```

To define the equation, use the `==` operator and assign the mathematical relation  $g(t) == 1$  to `eqn`. The identifier `eqn` is a symbolic equation that represents the trigonometric problem.

```
eqn = g(t) == 1

eqn =
2*cos(t)*sin(t) == 1
```

Use `solve` to find the solution of the trigonometric problem.

```
sol = solve(eqn)

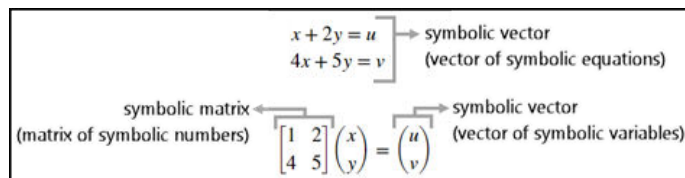
sol =
pi/4
```

## Symbolic Vector and Matrix

Use a symbolic vector and matrix to represent and solve a system of linear equations.

$$\begin{aligned}x + 2y &= u \\ 4x + 5y &= v\end{aligned}$$

You can represent the system of equations as a vector of two symbolic equations. You can also represent the system of equations as a matrix problem involving a matrix of symbolic numbers and a vector of symbolic variables. For brevity, any vector of symbolic objects is called a *symbolic vector* and any matrix of symbolic objects is called a *symbolic matrix*.



Create two symbolic equations `eq1` and `eq2`. Combine the two equations into a symbolic vector.

```
syms u v x y
eq1 = x + 2*y == u;
eq2 = 4*x + 5*y == v;
eqns = [eq1, eq2]

eqns =
[x + 2*y == u, 4*x + 5*y == v]
```

Use `solve` to find the solutions of the system of equations represented by `eqns`. `solve` returns a structure `S` with fields named after each of the variables in the equations. You can access the solutions using dot notation, as `S.x` and `S.y`.

```
S = solve(eqns);
S.x
```

```
ans =
(2*v)/3 - (5*u)/3
```

```
S.y
```

```
ans =
(4*u)/3 - v/3
```

Another way to solve the system of linear equations is to convert it to matrix form. Use `equationsToMatrix` to convert the system of equations to matrix form and assign the output to `A` and `b`. Here, `A` is a symbolic matrix and `b` is a symbolic vector. Solve the matrix problem by using the matrix division `\` operator.

```
[A,b] = equationsToMatrix(eqns,x,y)
```

```
A =
[1, 2]
[4, 5]
```

```
b =
u
v
```

```
sols = A\b
```

```
sols =
(2*v)/3 - (5*u)/3
(4*u)/3 - v/3
```

## Symbolic Matrix Variable

Since R2021a

Use symbolic matrix variables to evaluate differentials with respect to vectors.

$$\alpha = \mathbf{y}^T \mathbf{A} \mathbf{x}$$

$$\frac{\partial \alpha}{\partial \mathbf{x}} = \mathbf{y}^T \mathbf{A}$$

$$\frac{\partial \alpha}{\partial \mathbf{y}} = \mathbf{x}^T \mathbf{A}^T$$

Symbolic matrix variables represent matrices, vectors, and scalars in compact matrix notation. Symbolic matrix variables offer a concise display in typeset and show mathematical formulas with more clarity. You can enter vector- and matrix-based expressions as symbolic matrix variables in Symbolic Math Toolbox.

$$\alpha = \mathbf{y}^T \mathbf{A} \mathbf{x}$$

$$\frac{\partial \alpha}{\partial \mathbf{x}} = \mathbf{y}^T \mathbf{A}$$

$$\frac{\partial \alpha}{\partial \mathbf{y}} = \mathbf{x}^T \mathbf{A}^T$$

Create three symbolic matrix variables  $\mathbf{x}$ ,  $\mathbf{y}$ , and  $\mathbf{A}$  using the `syms` command with the `matrix` syntax. Nonscalar symbolic matrix variables are displayed as bold characters in the Command Window and in the Live Editor.

```
syms x [4 1] matrix
syms y [3 1] matrix
syms A [3 4] matrix
x
y
A
```

```
x =
x
```

```
y =
y
```

```
A =
A
```

Define `alpha`. Find the differential of `alpha` with respect to the vectors  $\mathbf{x}$  and  $\mathbf{y}$  that are represented by the symbolic matrix variables  $\mathbf{x}$  and  $\mathbf{y}$ .

```
alpha = y.'*A*x
```

```
alpha =
y.'*A*x
```

```
Dx = diff(alpha,x)
```

```
Dx =
y.'*A
```

```
Dy = diff(alpha,y)
```

```
Dy =
x.'*A.'
```

Substitute  $\mathbf{y}$  with `[1; 2; 3]` in `Dx` and substitute  $\mathbf{x}$  with `[-1; 2; 0; 1]` in `Dy` using `subs`. When evaluating a symbolic expression, you must substitute values that have the same size as the defined symbolic matrix variables.

```
Dx = subs(Dx,y,[1; 2; 3])
```



```
Dx =
symmatrix([1;2;3]).'*A

Dy = subs(Dy,x,[-1; 2; 0; 1])

Dx =
symmatrix([-1;2;0;1]).'*A.'
```

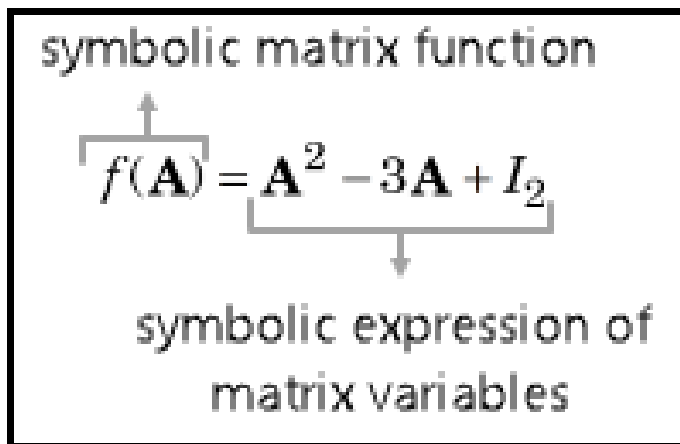
## Symbolic Matrix Function

Since R2022a

Use a symbolic matrix function to evaluate a matrix polynomial.

$$f(\mathbf{A}) = \mathbf{A}^2 - 3\mathbf{A} + \mathbf{I}_2$$

A symbolic matrix function represents a parameter-dependent function that accepts matrices, vectors, and scalars as input arguments. Symbolic matrix function operates on matrices in compact matrix notation, offering a concise display in typeset and showing mathematical formulas with more clarity. You can enter vector- and matrix-based formulas as symbolic matrix functions in Symbolic Math Toolbox.



Create a 2-by-2 symbolic matrix variable  $\mathbf{A}$  using the `syms` command with the `matrix` syntax. Create a symbolic matrix function  $f(\mathbf{A})$  that accepts  $\mathbf{A}$  as an input argument using the `syms` command with the `matrix keepargs` syntax to keep the previous definition of  $\mathbf{A}$ .

```
syms A 2 matrix
syms f(A) 2 matrix keepargs
```

Assign the polynomial expression to the symbolic matrix function.

```
f(A) = A^2 - 3*A + 2*eye(2)

f(A) =

2*symmatrix(eye(2)) - 3*A + A^2
```

Evaluate the function for the matrix value  $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ -2 & -1 \end{bmatrix}$ . When evaluating a symbolic matrix function, you must substitute values that have the same size as the defined input arguments.

```
fEval = f([1 2; -2 -1])
```

```
fEval =
- 3*symmatrix([1,2;-2,-1]) + symmatrix([1,2;-2,-1])^2 + 2*symmatrix(eye(2))
```

Convert the evaluated function from the `symmatrix` data type to the `sym` data type.

```
fSym = symmatrix2sym(fEval)
```

```
fSym =
[-4, -6]
[ 6,  2]
```

## Comparison of Symbolic Objects

This table compares the symbolic objects that are available in Symbolic Math Toolbox.

| Symbolic Object          | Examples of MATLAB Command                                                                                | Size of Symbolic Object                                                                                                                                                                                                                                     | Data Type                                                                                                                                                                                                                                    |
|--------------------------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Symbolic number          | <pre>a = 1/sqrt(sym(2)) theta = asin(a)  a = 2^(1/2)/2  theta = pi/4</pre>                                | 1-by-1                                                                                                                                                                                                                                                      | sym                                                                                                                                                                                                                                          |
| Symbolic scalar variable | <code>syms x y u v</code>                                                                                 | 1-by-1                                                                                                                                                                                                                                                      | sym                                                                                                                                                                                                                                          |
| Symbolic function        | <pre>syms x f(x) = x^2 + x - 2 syms g(t) [1 3] g  f(x) = x^2 + x - 2  g(t) = [g1(t), g2(t), g3(t)]</pre>  | <ul style="list-style-type: none"> <li>Size of unevaluated function, such as <code>size(g)</code>, is 1-by-1.</li> <li>Size of evaluated function, such as <code>size(g(t))</code>, is m-by-n, where m is the row size and n is the column size.</li> </ul> | <ul style="list-style-type: none"> <li>Data type of unevaluated function, such as <code>class(g)</code>, is <code>symfun</code>.</li> <li>Data type of evaluated function, such as <code>class(g(t))</code>, is <code>sym</code>.</li> </ul> |
| Symbolic expression      | <pre>syms x expr = x^2 + x - 2 expr2 = 2*sin(x)*cos(x)  expr = x^2 + x - 2  expr2 = 2*cos(x)*sin(x)</pre> | 1-by-1                                                                                                                                                                                                                                                      | sym                                                                                                                                                                                                                                          |

| Symbolic Object                            | Examples of MATLAB Command                                                                                | Size of Symbolic Object                                            | Data Type |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|-----------|
| Symbolic equation                          | <pre>syms u v x y eq1 = x + 2*y == u eq2 = 4*x + 5*y == v  eq1 = x + 2*y == u  eq2 = 4*x + 5*y == v</pre> | 1-by-1                                                             | sym       |
| Symbolic vector                            | <pre>syms u v b = [u v]  b = [u, v]</pre>                                                                 | 1-by-n or m-by-1, where m is the row size and n is the column size | sym       |
| Symbolic matrix                            | <pre>syms A x y A = [x y; x*y y^2]  A = [ x,   y] [x*y, y^2]</pre>                                        | m-by-n, where m is the row size and n is the column size           | sym       |
| Symbolic multidimensional array            | <pre>syms A [2 1 2] A  A(:,:,1) = A1_1 A2_1  A(:,:,2) = A1_2 A2_2</pre>                                   | sz1-by-sz2-...-szn, where szn is the size of the nth dimension     | sym       |
| Symbolic matrix variable<br>(since R2021a) | <pre>syms A B [2 3] matrix A B  A = A  B = B</pre>                                                        | m-by-n, where m is the row size and n is the column size           | symmatrix |

| Symbolic Object                            | Examples of MATLAB Command                                                                             | Size of Symbolic Object                                                                                                                                                                                                                                              | Data Type                                                                                                                                                                                                                                                         |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Symbolic matrix function<br>(since R2022a) | <pre>syms X Y [2 2] matrix syms f(X,Y) [2 2] matrix keep f(X,Y) = X*Y - Y*X  f(X, Y) = X*Y - Y*X</pre> | <ul style="list-style-type: none"> <li>Size of unevaluated matrix function, such as <code>size(f)</code>, is 1-by-1.</li> <li>Size of evaluated function, such as <code>size(f(X,Y))</code>, is m-by-n, where m is the row size and n is the column size.</li> </ul> | <ul style="list-style-type: none"> <li>Data type of unevaluated matrix function, such as <code>class(f)</code>, is <code>symfunmatrix</code>.</li> <li>Data type of evaluated function, such as <code>class(f(X,Y))</code>, is <code>symmatrix</code>.</li> </ul> |

## See Also

`syms` | `sym` | `symfun` | `symmatrix` | `symfunmatrix` | `symfunmatrix2symfun` | `symmatrix2sym` | `str2sym`

## More About

- “Create Symbolic Numbers, Variables, and Expressions” on page 1-3
- “Create Symbolic Functions” on page 1-9
- “Create Symbolic Matrices” on page 1-11
- “Create Symbolic Matrix Variables” on page 1-13
- “Choose `syms` or `sym` Function” on page 2-4
- “Choose Numeric or Symbolic Arithmetic” on page 2-32

# Perform Symbolic Computations

**In this section...**

“Differentiate Symbolic Expressions” on page 1-29  
“Integrate Symbolic Expressions” on page 1-30  
“Solve Equations” on page 1-31  
“Simplify Symbolic Expressions” on page 1-33  
“Substitutions in Symbolic Expressions” on page 1-34  
“Plot Symbolic Functions” on page 1-36

## Differentiate Symbolic Expressions

With the Symbolic Math Toolbox software, you can find

- Derivatives of single-variable expressions
- Partial derivatives
- Second and higher order derivatives
- Mixed derivatives

For in-depth information on taking symbolic derivatives see “Differentiation” on page 3-181.

### Expressions with One Variable

To differentiate a symbolic expression, use the `diff` command. The following example illustrates how to take a first derivative of a symbolic expression:

```
syms x
f = sin(x)^2;
diff(f)

ans =
2*cos(x)*sin(x)
```

### Partial Derivatives

For multivariable expressions, you can specify the differentiation variable. If you do not specify any variable, MATLAB chooses a default variable by its proximity to the letter `x`:

```
syms x y
f = sin(x)^2 + cos(y)^2;
diff(f)

ans =
2*cos(x)*sin(x)
```

For the complete set of rules MATLAB applies for choosing a default variable, see “Find a Default Symbolic Variable” on page 2-2.

To differentiate the symbolic expression `f` with respect to a variable `y`, enter:

```
syms x y
f = sin(x)^2 + cos(y)^2;
diff(f, y)

ans =
-2*cos(y)*sin(y)
```

## Second Partial and Mixed Derivatives

To take a second derivative of the symbolic expression `f` with respect to a variable `y`, enter:

```
syms x y
f = sin(x)^2 + cos(y)^2;
diff(f, y, 2)

ans =
2*sin(y)^2 - 2*cos(y)^2
```

You get the same result by taking derivative twice: `diff(diff(f, y))`. To take mixed derivatives, use two differentiation commands. For example:

```
syms x y
f = sin(x)^2 + cos(y)^2;
diff(diff(f, y), x)

ans =
0
```

## Integrate Symbolic Expressions

You can perform symbolic integration including:

- Indefinite and definite integration
- Integration of multivariable expressions

For in-depth information on the `int` command including integration with real and complex parameters, see “Integration” on page 3-186.

### Indefinite Integrals of One-Variable Expressions

Suppose you want to integrate a symbolic expression. The first step is to create the symbolic expression:

```
syms x
f = sin(x)^2;
```

To find the indefinite integral, enter

```
int(f)

ans =
x/2 - sin(2*x)/4
```

### Indefinite Integrals of Multivariable Expressions

If the expression depends on multiple symbolic variables, you can designate a variable of integration. If you do not specify any variable, MATLAB chooses a default variable by the proximity to the letter `x`:

```
syms x y n
f = x^n + y^n;
int(f)

ans =
x*y^n + (x*x^n)/(n + 1)
```

For the complete set of rules MATLAB applies for choosing a default variable, see “Find a Default Symbolic Variable” on page 2-2.

You also can integrate the expression  $f = x^n + y^n$  with respect to  $y$

```
syms x y n
f = x^n + y^n;
int(f, y)

ans =
x^n*y + (y*y^n)/(n + 1)
```

If the integration variable is  $n$ , enter

```
syms x y n
f = x^n + y^n;
int(f, n)

ans =
x^n/log(x) + y^n/log(y)
```

## Definite Integrals

To find a definite integral, pass the limits of integration as the final two arguments of the `int` function:

```
syms x y n
f = x^n + y^n;
int(f, 1, 10)

ans =
piecewise(n == -1, log(10) + 9/y, n ~= -1,...
(10*10^n - 1)/(n + 1) + 9*y^n)
```

## If MATLAB Cannot Find a Closed Form of an Integral

If the `int` function cannot compute an integral, it returns an unresolved integral:

```
syms x
int(sin(sinh(x)))

ans =
int(sin(sinh(x)), x)
```

## Solve Equations

You can solve different types of symbolic equations including:

- Algebraic equations with one symbolic variable
- Algebraic equations with several symbolic variables

- Systems of algebraic equations

For in-depth information on solving symbolic equations including differential equations, see “Equation Solving”.

### **Solve Algebraic Equations with One Symbolic Variable**

Use the double equal sign (==) to define an equation. Then you can solve the equation by calling the solve function. For example, solve this equation:

```
syms x
solve(x^3 - 6*x^2 == 6 - 11*x)
```

```
ans =
     1
     2
     3
```

If you do not specify the right side of the equation, solve assumes that it is zero:

```
syms x
solve(x^3 - 6*x^2 + 11*x - 6)
```

```
ans =
     1
     2
     3
```

### **Solve Algebraic Equations with Several Symbolic Variables**

If an equation contains several symbolic variables, you can specify a variable for which this equation should be solved. For example, solve this multivariable equation with respect to y:

```
syms x y
solve(6*x^2 - 6*x^2*y + x*y^2 - x*y + y^3 - y^2 == 0, y)
```

```
ans =
     1
    2*x
   -3*x
```

If you do not specify any variable, you get the solution of an equation for the alphabetically closest to x variable. For the complete set of rules MATLAB applies for choosing a default variable see “Find a Default Symbolic Variable” on page 2-2.

### **Solve Systems of Algebraic Equations**

You also can solve systems of equations. For example:

```
syms x y z
[x, y, z] = solve(z == 4*x, x == y, z == x^2 + y^2)
```

```
x =
     0
     2
```

```
y =
     0
```



```

2
z =
0
8

```

## Simplify Symbolic Expressions

Symbolic Math Toolbox provides a set of simplification functions allowing you to manipulate the output of a symbolic expression. For example, the following polynomial of the golden ratio `phi`

```

phi = (1 + sqrt(sym(5)))/2;
f = phi^2 - phi - 1

```

returns

```

f =
(5^(1/2)/2 + 1/2)^2 - 5^(1/2)/2 - 3/2

```

You can simplify this answer by entering

```
simplify(f)
```

and get a very short answer:

```

ans =
0

```

Symbolic simplification is not always so straightforward. There is no universal simplification function, because the meaning of a simplest representation of a symbolic expression cannot be defined clearly. Different problems require different forms of the same mathematical expression. Knowing what form is more effective for solving your particular problem, you can choose the appropriate simplification function.

For example, to show the order of a polynomial or symbolically differentiate or integrate a polynomial, use the standard polynomial form with all the parentheses multiplied out and all the similar terms summed up. To rewrite a polynomial in the standard form, use the `expand` function:

```

syms x
f = (x^2 - 1)*(x^4 + x^3 + x^2 + x + 1)*(x^4 - x^3 + x^2 - x + 1);
expand(f)

ans =
x^10 - 1

```

The `factor` simplification function shows the polynomial roots. If a polynomial cannot be factored over the rational numbers, the output of the `factor` function is the standard polynomial form. For example, to factor the third-order polynomial, enter:

```

syms x
g = x^3 + 6*x^2 + 11*x + 6;
factor(g)

ans =
[ x + 3, x + 2, x + 1]

```

The nested (Horner) representation of a polynomial is the most efficient for numerical evaluations:

```
syms x
h = x^5 + x^4 + x^3 + x^2 + x;
horner(h)

ans =
x*(x*(x*(x*(x + 1) + 1) + 1) + 1)
```

For a list of Symbolic Math Toolbox simplification functions, see “Choose Function to Rearrange Expression” on page 3-123.

## Substitutions in Symbolic Expressions

### Substitute Symbolic Variables with Numbers

You can substitute a symbolic variable with a numeric value by using the `subs` function. For example, evaluate the symbolic expression `f` at the point  $x = 1/3$ :

```
syms x
f = 2*x^2 - 3*x + 1;
subs(f, 1/3)

ans =
2/9
```

The `subs` function does not change the original expression `f`:

```
f

f =
2*x^2 - 3*x + 1
```

### Substitute in Multivariate Expressions

When your expression contains more than one variable, you can specify the variable for which you want to make the substitution. For example, to substitute the value  $x = 3$  in the symbolic expression

```
syms x y
f = x^2*y + 5*x*sqrt(y);
```

enter the command

```
subs(f, x, 3)

ans =
9*y + 15*y^(1/2)
```

### Substitute One Symbolic Variable for Another

You also can substitute one symbolic variable for another symbolic variable. For example to replace the variable `y` with the variable `x`, enter

```
subs(f, y, x)

ans =
x^3 + 5*x^(3/2)
```

## Substitute a Matrix into a Polynomial

You can also substitute a matrix into a symbolic polynomial with numeric coefficients. There are two ways to substitute a matrix into a polynomial: element by element and according to matrix multiplication rules.

### Element-by-Element Substitution

To substitute a matrix at each element, use the `subs` command:

```
syms x
f = x^3 - 15*x^2 - 24*x + 350;
A = [1 2 3; 4 5 6];
subs(f,A)

ans =
[ 312, 250, 170]
[ 78, -20, -118]
```

You can do element-by-element substitution for rectangular or square matrices.

### Substitution in a Matrix Sense

If you want to substitute a matrix into a polynomial using standard matrix multiplication rules, a matrix must be square. For example, you can substitute the magic square `A` into a polynomial `f`:

- 1 Create the polynomial:

```
syms x
f = x^3 - 15*x^2 - 24*x + 350;
```

- 2 Create the magic square matrix:

```
A = magic(3)

A =
     8     1     6
     3     5     7
     4     9     2
```

- 3 Get a row vector containing the numeric coefficients of the polynomial `f`:

```
b = sym2poly(f)

b =
     1    -15    -24    350
```

- 4 Substitute the magic square matrix `A` into the polynomial `f`. Matrix `A` replaces all occurrences of `x` in the polynomial. The constant times the identity matrix `eye(3)` replaces the constant term of `f`:

```
A^3 - 15*A^2 - 24*A + 350*eye(3)

ans =
    -10     0     0
     0    -10     0
     0     0    -10
```

The `polyvalm` command provides an easy way to obtain the same result:

```
polyvalm(b,A)
```

```
ans =  
    -10     0     0  
     0    -10     0  
     0     0    -10
```

### Substitute the Elements of a Symbolic Matrix

To substitute a set of elements in a symbolic matrix, also use the `subs` command. Suppose you want to replace some of the elements of a symbolic circulant matrix `A`

```
syms a b c  
A = [a b c; c a b; b c a]  
  
A =  
[ a, b, c]  
[ c, a, b]  
[ b, c, a]
```

To replace the (2, 1) element of `A` with `beta` and the variable `b` throughout the matrix with variable `alpha`, enter

```
alpha = sym('alpha');  
beta = sym('beta');  
A(2,1) = beta;  
A = subs(A,b,alpha)
```

The result is the matrix:

```
A =  
[      a, alpha,      c]  
[  beta,      a, alpha]  
[ alpha,      c,      a]
```

For more information, see “Substitute Elements in Symbolic Matrices” on page 3-146.

## Plot Symbolic Functions

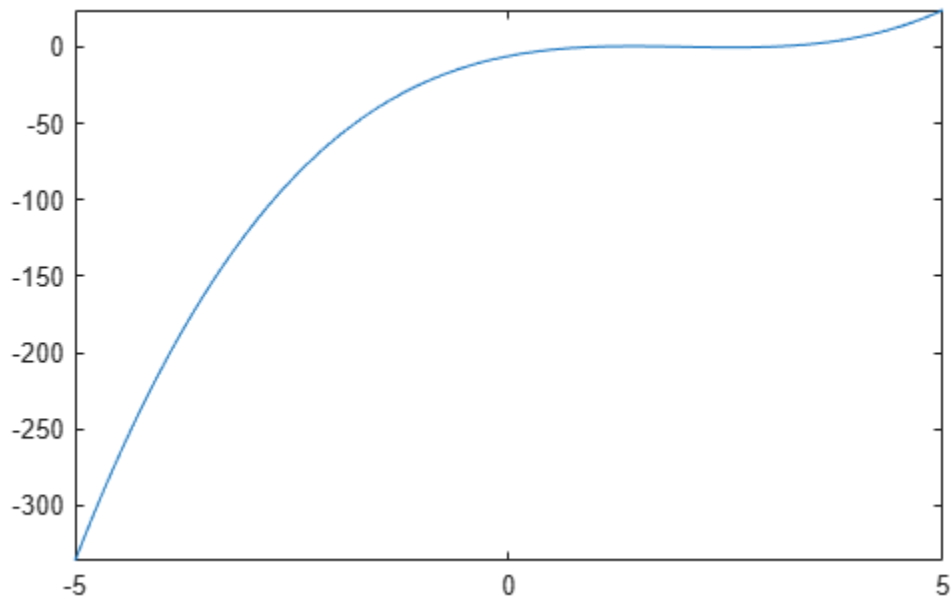
Symbolic Math Toolbox provides the plotting functions:

- `fplot` to create 2-D plots of symbolic expressions, equations, or functions in Cartesian coordinates.
- `fplot3` to create 3-D parametric plots.
- `fpolarplot` to create plots in polar coordinates.
- `fsurf` to create surface plots.
- `fcontour` to create contour plots.
- `fmesh` to create mesh plots.

### Explicit Function Plot

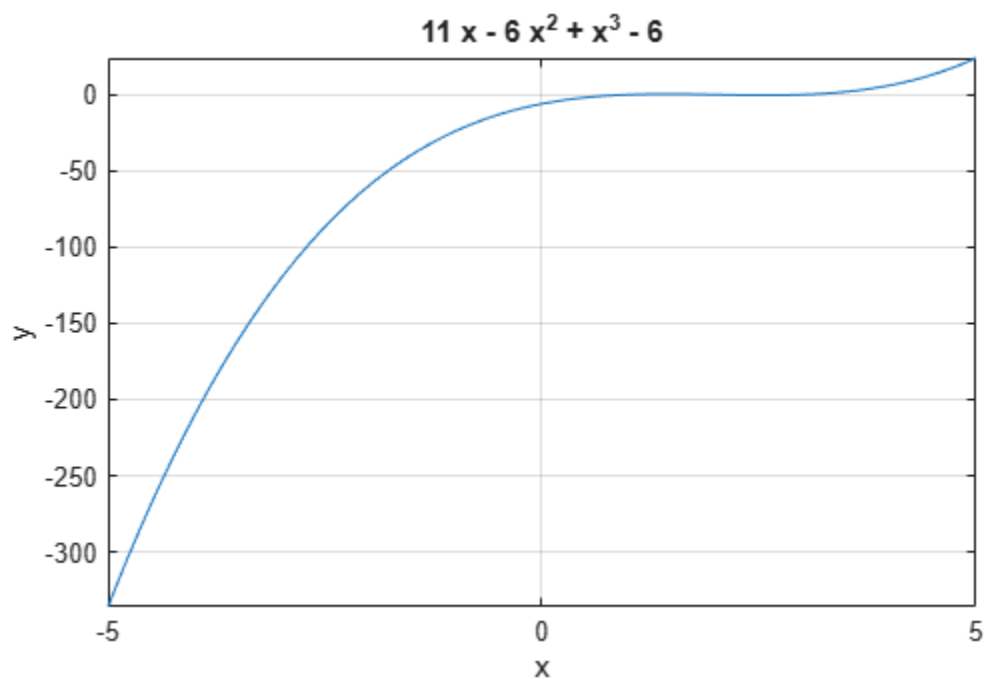
Create a 2-D line plot by using `fplot`. Plot the expression  $x^3 - 6x^2 + 11x - 6$ .

```
syms x  
f = x^3 - 6*x^2 + 11*x - 6;  
fplot(f)
```



Add labels for the x- and y-axes. Generate the title by using `texlabel(f)`. Show the grid by using `grid on`. For details, see “Add Title and Axis Labels to Chart”.

```
xlabel('x')  
ylabel('y')  
title(texlabel(f))  
grid on
```

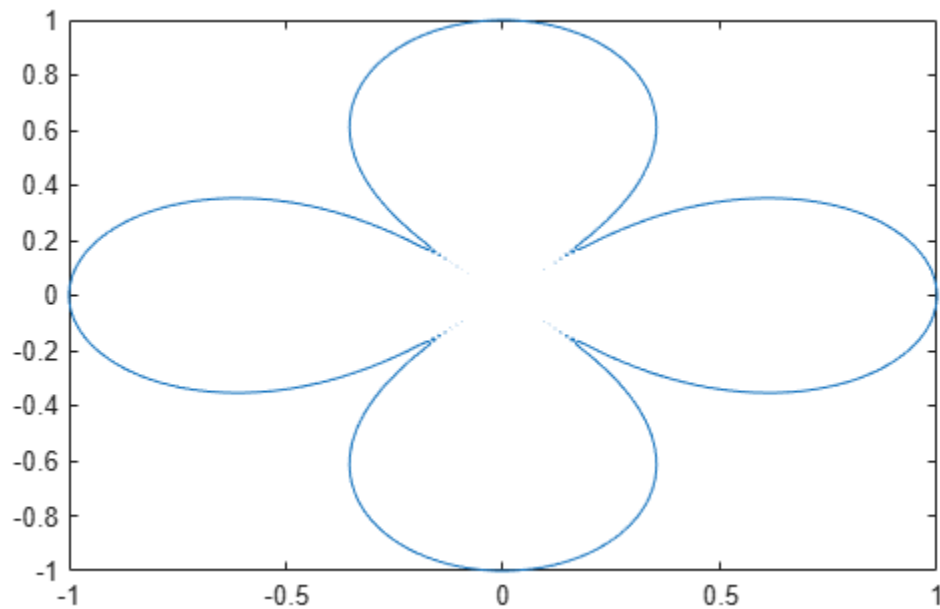


### Implicit Function Plot

Plot equations and implicit functions using `fimplicit`.

Plot the equation  $(x^2 + y^2)^4 = (x^2 - y^2)^2$  over  $-1 < x < 1$ .

```
syms x y
eqn = (x^2 + y^2)^4 == (x^2 - y^2)^2;
fimplicit(eqn, [-1 1])
```



### 3-D Plot

Plot 3-D parametric lines by using `fplot3`.

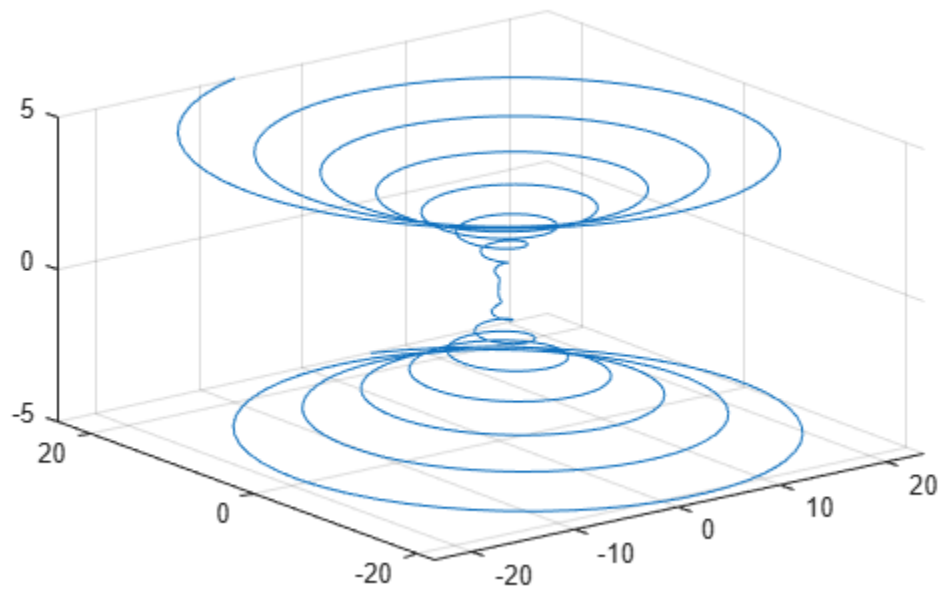
Plot the parametric line

$$x = t^2 \sin(10t)$$

$$y = t^2 \cos(10t)$$

$$z = t.$$

```
syms t
fplot3(t^2*sin(10*t), t^2*cos(10*t), t)
```

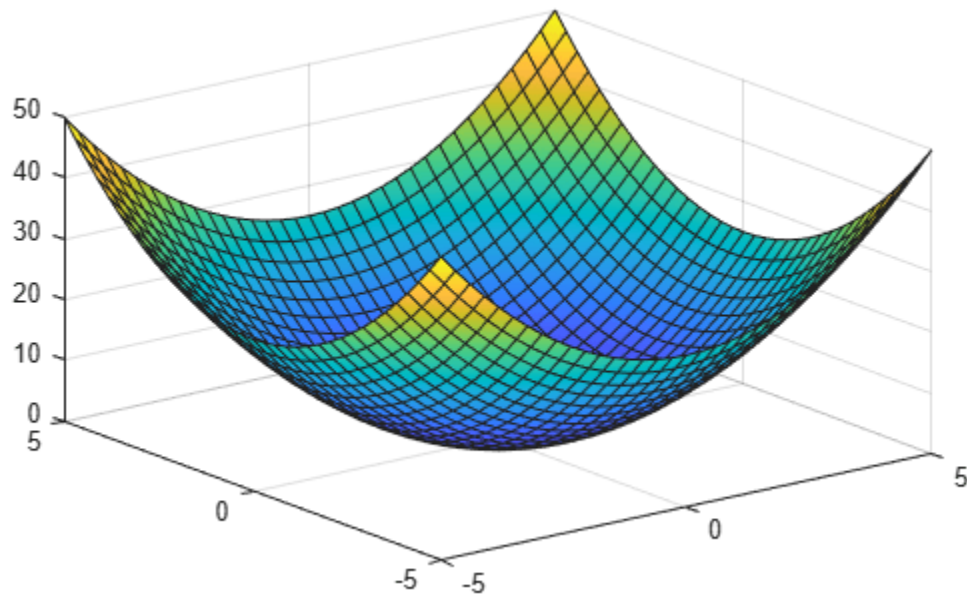


### Create Surface Plot

Create a 3-D surface by using `fsurf`.

Plot the paraboloid  $z = x^2 + y^2$ .

```
syms x y  
fsurf(x^2 + y^2)
```



## See Also

### More About

- “Create Symbolic Numbers, Variables, and Expressions” on page 1-3
- “Create Symbolic Functions” on page 1-9
- “Create Symbolic Matrices” on page 1-11
- “Use Assumptions on Symbolic Variables” on page 1-41



## Use Assumptions on Symbolic Variables

You can set mathematical assumptions or conditions on symbolic variables to restrict the domain and range of the variables. This example shows how to set, check, and clear assumptions.

### Default Assumption

In Symbolic Math Toolbox™, symbolic variables are complex variables by default. For example, if you create  $z$  as a symbolic variable, then Symbolic Math Toolbox assumes that  $z$  is a complex variable.

```
syms z
```

You can check if a symbolic variable is assumed to be complex by using `assumptions`. If  $z$  is complex, then `assumptions(z)` returns an empty symbolic object.

```
assumptions(z)
```

```
ans =
```

```
Empty sym: 1-by-0
```

### Set Assumptions

To set mathematical assumptions or conditions on symbolic variables, use the `assume` function. For example, assume that the variable  $x$  is larger than 2.

```
syms x
assume(x > 2)
```

`assume` replaces all previous assumptions on the variable with the new assumption. If you want to add a new assumption to the existing assumptions, then use `assumeAlso`. For example, add the assumption that  $x$  is also an integer. Now the variable  $x$  is an integer larger than 2.

```
assumeAlso(x, "integer")
```

`assume` and `assumeAlso` let you state that a variable or an expression belongs to these sets: real numbers, rational numbers, integers, and positive numbers. You can also use `assume` and `assumeAlso` to set mathematical conditions on symbolic variables or expressions, such as  $x > 2$ .

Alternatively, you can set assumptions when declaring symbolic variables using `sym` or `syms`, where the assumptions are limited to these sets: real numbers, rational numbers, integers, and positive numbers. For example, create the real symbolic variables  $a$  and  $b$ , and the positive symbolic variable  $c$  using `sym`.

```
a = sym("a", "real");
b = sym("b", "real");
c = sym("c", "positive");
```

You can also use `syms` to create these symbolic variables with assumptions.

```
syms a b real
syms c positive
```

## Check Existing Assumptions

To see all assumptions set on a symbolic variable, use the `assumptions` function with the name of the variable as an input argument. For example, this command returns the assumptions currently set on the variable `x`.

```
assumptions(x)

ans = (x ∈ ℤ 2 < x)
```

To see all assumptions set on all symbolic variables in the MATLAB® workspace, use `assumptions` without input arguments.

```
assumptions

ans = (a ∈ ℝ b ∈ ℝ x ∈ ℤ 0 < c 2 < x)
```

## Delete Symbolic Objects and Their Assumptions

Symbolic objects and their assumptions are stored separately. When you set an assumption that the symbolic variable `x` is real, you actually create a symbolic object `x` and the assumption that the object is real. The object is stored in the MATLAB workspace, and the assumption is stored in the symbolic engine. When you delete a symbolic object from the MATLAB workspace using `clear`, the assumption that `x` is real stays in the symbolic engine.

```
syms x
assume(x, "real")
clear x
```

If you recreate a variable using `sym`, then the existing assumptions apply. If you recreate a variable using `syms`, then the existing assumptions on that variable are cleared.

For example, the assumption that `x` is real causes the polynomial  $x^2 + 1 == 0$  to have no roots.

```
x = sym("x");
solve(x^2 + 1 == 0, x)
```

```
ans =

Empty sym: 0-by-1
```

The complex roots of this polynomial do not appear as solutions because the symbolic variable `x` still has the assumption that `x` is real stored in the symbolic engine. To clear the assumption, recreate the variable using `syms`.

```
syms x
```

Solve the same polynomial equation again.

```
solve(x^2 + 1 == 0, x)
```

```
ans =

(-i)
(i)
```

If you want to delete both the symbolic object and its assumption, first clear the assumption on the symbolic object using `assume(x,"clear")` and then clear the object using `clear x`.

```
assume(x,"clear")  
clear x
```

For details on clearing symbolic variables, see “Clear Assumptions and Reset the Symbolic Engine” on page 3-379.

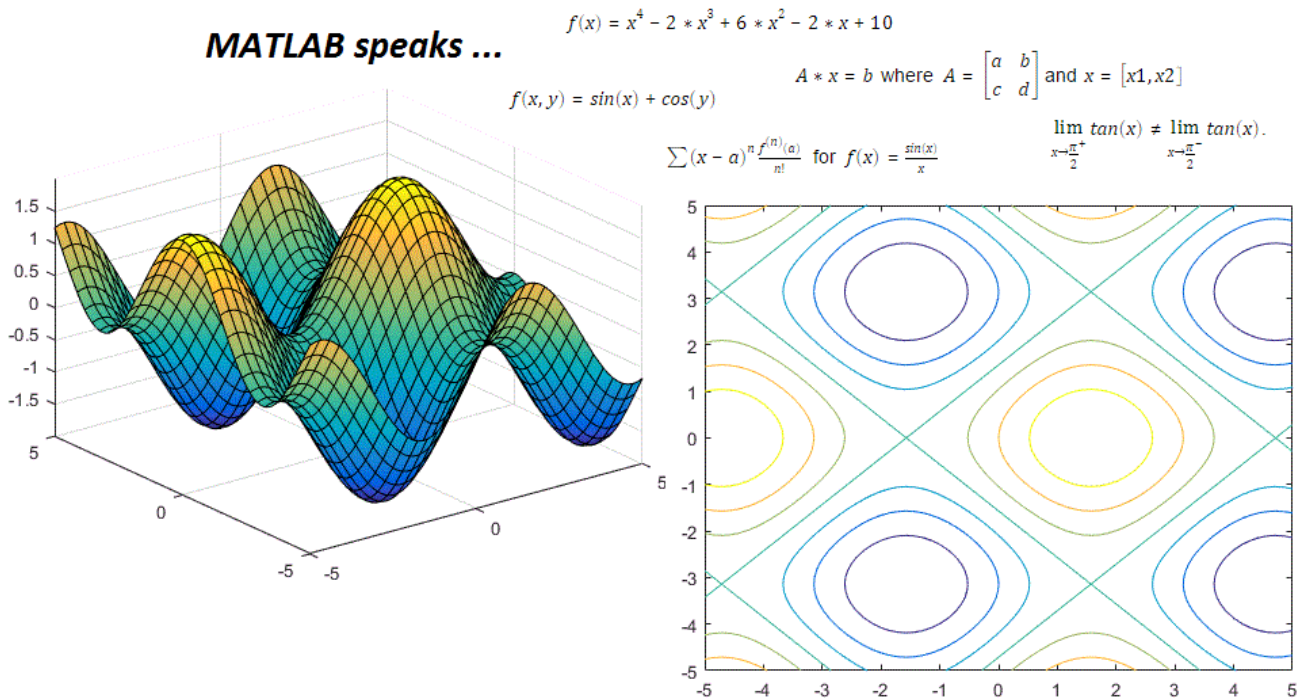
## See Also

### More About

- “Create Symbolic Numbers, Variables, and Expressions” on page 1-3
- “Create Symbolic Functions” on page 1-9
- “Create Symbolic Matrices” on page 1-11
- “Perform Symbolic Computations” on page 1-29

## Computational Mathematics in Symbolic Math Toolbox

This example provides an overview of the Symbolic Math Toolbox™ which offers a complete set of tools for computational and analytical mathematics.



This example includes

- Variables, Expressions, Functions and Equations
- Substitution and Solving
- Simplification and Manipulation
- Calculus (Differentiation, Integration, Limits, Series)
- Differential Equations
- Linear Algebra
- Graphics

For more details see “Get Started with Symbolic Math Toolbox”. For more details on documenting and sharing your mathematics see “Create Live Scripts in the Live Editor”.

### Variables, Expressions, Functions and Equations

Variables in MATLAB® are by default double-precision. The Symbolic Math Toolbox extends this by allowing you to express numbers in exact symbolic form using `sym` and with variable-precision using `vpa`.

```
pi/6 + pi/4
```

```
ans =  
1.3090
```

```
sym(pi/6) + sym(pi/4)
```

```
ans =  
 $\frac{5\pi}{12}$ 
```

```
vpa(pi/6) + vpa(pi/4)
```

```
ans = 1.3089969389957471826927680763665
```

Symbolic variables can be used in mathematical expressions, functions and equations including trigonometric, logarithmic, exponential, and special functions. You can create symbolic expressions and perform mathematical calculations on them.

```
syms x y  
log(x) + exp(y)
```

```
ans = ey + log(x)
```

You can also create piecewise functions.

```
y(x) = piecewise(x<0, -1, x>0, 1)
```

```
y(x) =  
 $\begin{cases} -1 & \text{if } x < 0 \\ 1 & \text{if } 0 < x \end{cases}$ 
```

Create and evaluate “Create Symbolic Functions” on page 1-9. Find the value of  $f$  at  $x = -5$ .

```
syms f(x)  
f(x) = x^4-2*x^3+6*x^2-2*x+10
```

```
f(x) = x4 - 2 x3 + 6 x2 - 2 x + 10
```

```
f(-5)
```

```
ans = 1045
```

Find the intersection between lines  $y_1$  and  $y_2$  using `solve`. Equate the lines using the `==` operator.

```
syms y1 y2  
y1 = x+3; y2 = 3*x;  
solve(y1 == y2)
```

```
ans =  
 $\frac{3}{2}$ 
```

Make `assume` on symbolic variables. There are 4 solutions to  $x^4 = 1$ , two real and two complex. Assuming that  $x$  is real and  $x > 0$ , there is only one solution.

```
syms x  
solve(x^4 == 1)
```

```
ans =
```

$$\begin{pmatrix} -1 \\ 1 \\ -i \\ i \end{pmatrix}$$

```
assume(x, 'real')
assumeAlso( x > 0)
assumptions(x)
```

```
ans = (x ∈ ℝ 0 < x)
```

```
solve(x^4 == 1)
```

```
ans = 1
```

```
assume(x, 'clear')
```

### Substitution and Solving

The Symbolic Math Toolbox supports evaluation of mathematical functions by substituting for any part of an expression using `subs`. You can substitute numeric values, other symbolic variables or expressions, vectors, or matrices. The Symbolic Math Toolbox supports the solving of equations and systems of equations using `solve`. It supports solving multivariate equations, solving inequalities and solving with assumptions. Solutions can be found symbolically or numerically with high precision by using variable-precision arithmetic.

Make substitutions with your symbolic variables. Substitute  $x = x_0 - 1$  into  $x^2 + 1$

```
syms x x0
subs(x^2+1,x,x0-1)
```

```
ans = (x0 - 1)^2 + 1
```

Substitute multiple values. For example, evaluate  $\cos(a) + \sin(b) - e^{2C}$  by substituting

$$a = \frac{\pi}{2}, b = \frac{\pi}{4}, c = -1.$$

```
syms a b c
subs(cos(a) + sin(b) - exp(2*c), [a b c], [pi/2 pi/4 -1])
```

```
ans =
     $\frac{\sqrt{2}}{2} - e^{-2}$ 
```

Create and solve equations. Find the zeros of  $9x^2 - 1 = 0$ .

```
solve(9*x^2 - 1 == 0)
```

```
ans =
     $\begin{pmatrix} -\frac{1}{3} \\ \frac{1}{3} \end{pmatrix}$ 
```

Solve the general quadratic equation  $ax^2 + bx + c = 0$  and use `subs` to evaluate that solution for  $a = 9, b = 0, c = -1$ .

```
eqn = a*x^2 + b*x + c == 0;
sol = solve(eqn)
```

```
sol =
```

$$\begin{pmatrix} -\frac{b + \sqrt{b^2 - 4ac}}{2a} \\ -\frac{b - \sqrt{b^2 - 4ac}}{2a} \end{pmatrix}$$

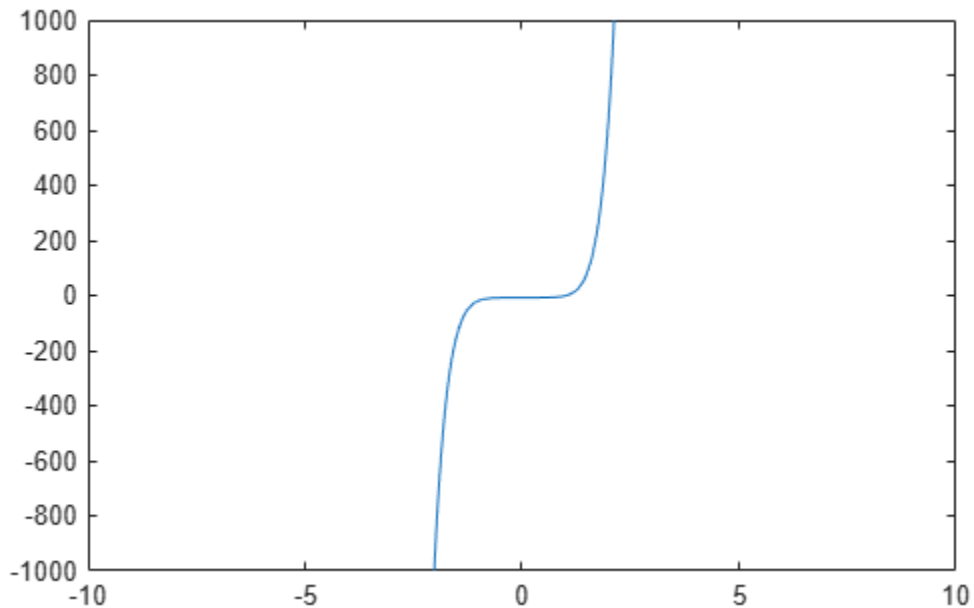
```
subs(sol,[a b c],[9 0 -1])
```

```
ans =
```

$$\begin{pmatrix} -\frac{1}{3} \\ \frac{1}{3} \end{pmatrix}$$

Solve equations symbolically or with variable-precision arithmetic when exact results or high precision is needed. The graph of  $f(x) = 6x^7 - 2x^6 + 3x^3 - 8$  is very flat near its root.

```
syms x f(x)
assume(x>0)
f(x) = 6*x^7-2*x^6+3*x^3-8;
fplot(f)
xlim([-10 10])
ylim([-1e3 1e3])
```



```
doubleSol = roots([6 -2 0 0 3 0 0 -8]) % double-precision
```

```
doubleSol = 7×1 complex
```

```
1.0240 + 0.0000i
0.7652 + 0.8319i
```

```

0.7652 - 0.8319i
-0.8808 + 0.5043i
-0.8808 - 0.5043i
-0.2297 + 0.9677i
-0.2297 - 0.9677i

```

`symsSol = solve(f) % exact.` The `roots` object stores the zeros for symbolic computations

```
symsSol =
```

```

root(z^7 - z^6/3 + z^3/2 - 4/3, z, 5)

```

`vpaSol = vpasolve(f) % variable-precision`

```
vpaSol =
```

```

1.0240240759053702941448316563337
-0.88080620051762149639205672298326 + 0.50434058840127584376331806592405 i
-0.88080620051762149639205672298326 - 0.50434058840127584376331806592405 i
-0.22974795226118163963098570610724 + 0.96774615576744031073999010695171 i
-0.22974795226118163963098570610724 - 0.96774615576744031073999010695171 i
0.7652087814927846556172932675903 + 0.83187331431049713218367239317121 i
0.7652087814927846556172932675903 - 0.83187331431049713218367239317121 i

```

## Simplification and Manipulation

The Symbolic Math Toolbox supports the “Formula Manipulation and Simplification” of mathematical functions. Most mathematical expressions can be represented in different, but mathematically equivalent forms and the Symbolic Math Toolbox supports a number of operations, including factoring or expanding expressions, combining terms, rewriting or rearranging expressions, and simplification based on assumptions.

Perform polynomial multiplication and simplify the results, show that

$(x - 1)(x + 1)(x^2 + x + 1)(x^2 + 1)(x^2 - x + 1)(x^4 - x^2 + 1)$  simplifies to  $x^{12} - 1$ .

```
simplify((x - 1)*(x + 1)*(x^2 + x + 1)*(x^2 + 1)*(x^2 - x + 1)*(x^4 - x^2 + 1))
```

```
ans = x^12 - 1
```

Apply trigonometric identities to simplifications, for example  $\sin^2(x) = \frac{1 - \cos(2x)}{2}$ .

```
combine(2*sin(x)*cos(x) + (1- cos(2*x))/2 + cos(x)^2, 'sincos')
```

```
ans = sin(2 x) + 1
```

Factor or expand multivariate polynomials.

```
syms x y
factor(y^6-x^6)
```

```
ans = (-1 x - y x + y x^2 + x y + y^2 x^2 - x y + y^2)
```

```
f(x) = (x^3 + 7);
expand(f(y-1))
```

```
ans = y^3 - 3 y^2 + 3 y + 6
```



Find the functional composition  $f(g(x))$ .

```
f(x) = sqrt(log(x));
g(x) = sqrt(1-x);
h = compose(g,f,x)
```

$$h(x) = \sqrt{1 - \sqrt{\log(x)}}$$

### Calculus (Differentiation, Integration, Limits, Series, etc)

The Symbolic Math Toolbox has a full set of calculus tools for applied mathematics. It can perform multivariate symbolic integration and differentiation. It can generate, manipulate, and perform calculations with series.

Find the derivative of  $\frac{d}{dx}(\sin(x))$ .

```
diff(sin(x))
```

```
ans = cos(x)
```

Find the derivative of  $\frac{d}{dx}(x^2 + \sin(2x^4) + 1)$  using the chain rule.

```
diff(x^2+sin(2*x^4)+1,x)
```

```
ans = 2 x + 8 x^3 cos(2 x^4)
```

Find the indefinite integral  $\int f(x) dx$  for  $f(x) = e^{-\frac{x^2}{2}}$ .

```
int(exp(-x^2/2),x)
```

```
ans =
      sqrt(2) sqrt(pi) erf(sqrt(2) x / 2)
      -----
              2
```

Find the definite integral  $\int_a^b f(x) dx$  for  $f(x) = x \log(1+x)$  from 0 to 1.

```
int(x*log(1+x),0,1)
```

```
ans =
      1
      -
      4
```

Show that  $\frac{\sin(x)}{x} = 1$  at  $x = 0$  by computing the Taylor series expansion  $\sum (x-a)^n \frac{f^{(n)}(a)}{n!}$  for

$f(x) = \frac{\sin(x)}{x}$  around the point  $x = 0$ .

```
syms x
T = taylor(sin(x)/x)
```

```
T =
      x^4
      ---
     120
      -
      x^2
      ---
      6
      + 1
```

```
subs(T,x,0)
```

```
ans = 1
```

Show that  $\tan(x)$  is discontinuous at  $x = \frac{\pi}{2}$  by showing that the left and right limits are not equal.

$$\lim_{x \rightarrow \frac{\pi}{2}^+} \tan(x) \neq \lim_{x \rightarrow \frac{\pi}{2}^-} \tan(x).$$

```
limit(tan(x),x,pi/2,'left')
```

```
ans = ∞
```

```
limit(tan(x),x,pi/2,'right')
```

```
ans = - ∞
```

```
limit(tan(x),x,pi/2)
```

```
ans = NaN
```

### Differential Equations

The Symbolic Math Toolbox can analytically solve a system of ordinary differential equations using `dsolve`. For more information, see “Solve a System of Differential Equations” on page 3-53.

Solve the first order ODE  $\frac{dy}{dx} = -ay$ .

```
syms a b y(x)
dsolve(diff(y) == -a*y)
```

```
ans = C1 e-a x
```

Solve the same ODE with the initial condition  $y(0) = b$ .

```
dsolve(diff(y)== -a*y,y(0)==b)
```

```
ans = b e-a x
```

Solve the system of coupled first order ODEs  $\frac{dx}{dt} = y$  and  $\frac{dy}{dt} = -x$ .

```
syms x(t) y(t)
z = dsolve(diff(x) == y, diff(y) == -x);
disp([z.x;z.y])
```

$$\begin{pmatrix} C_1 \cos(t) + C_2 \sin(t) \\ C_2 \cos(t) - C_1 \sin(t) \end{pmatrix}$$

### Linear Algebra

The Symbolic Math Toolbox can work with symbolic vectors and matrices. It can compute `eig` of symbolic matrices.

Perform matrix multiplication  $Ax = b$  where  $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$  and  $x = [x_1, x_2]$

```
syms a b c d
syms x1 x2
x = [x1; x2];
A = [a b ; c d];
b = A*x
```

$$b = \begin{pmatrix} a x_1 + b x_2 \\ c x_1 + d x_2 \end{pmatrix}$$

Find the determinant of A.

```
det(A)
```

```
ans = a d - b c
```

Find the eigenvalues of A.

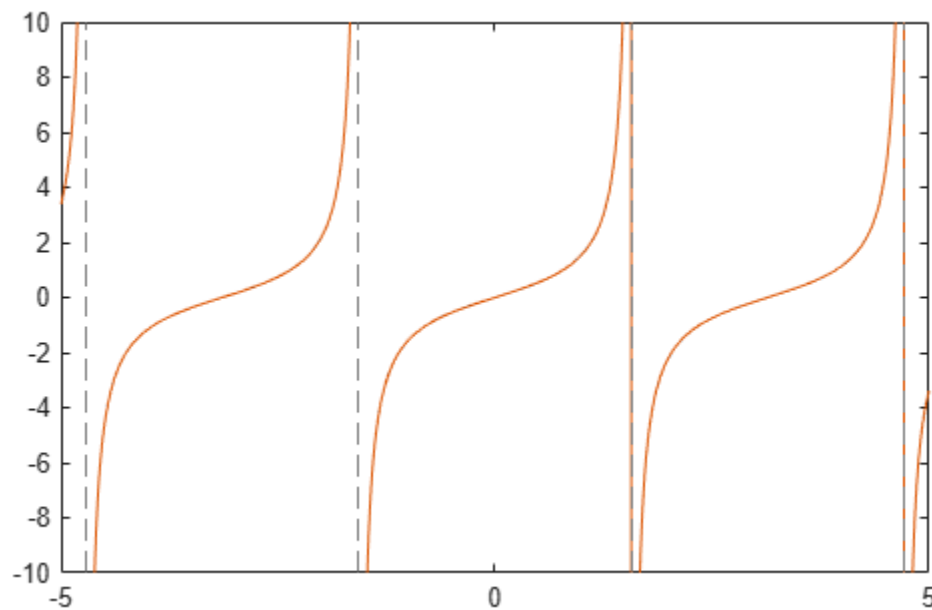
```
lambda = eig(A)
```

$$\lambda = \begin{pmatrix} \frac{a}{2} + \frac{d}{2} - \frac{\sqrt{a^2 - 2 a d + d^2 + 4 b c}}{2} \\ \frac{a}{2} + \frac{d}{2} + \frac{\sqrt{a^2 - 2 a d + d^2 + 4 b c}}{2} \end{pmatrix}$$

## Graphics

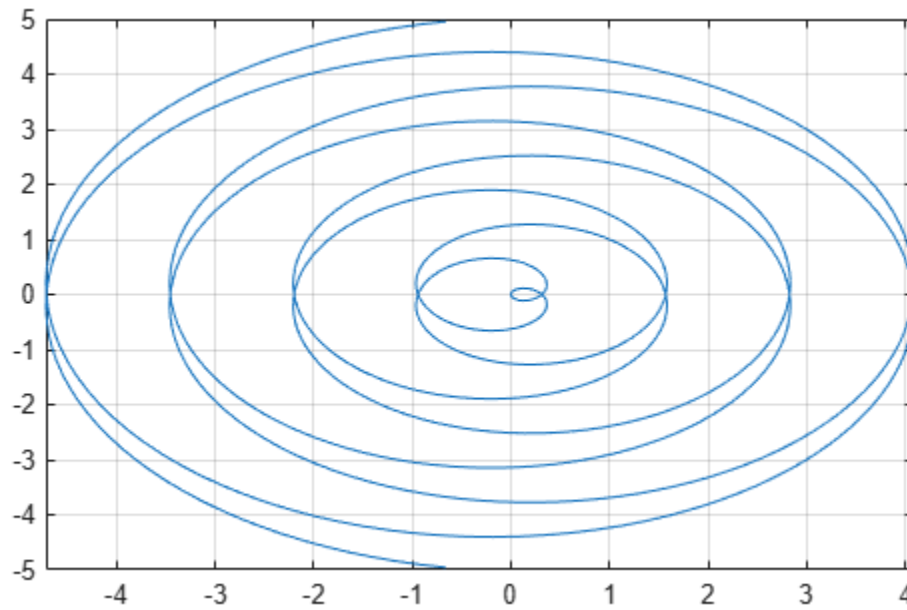
The Symbolic Math Toolbox supports analytical plotting in 2D and 3D.

```
fplot(tan(x))
```



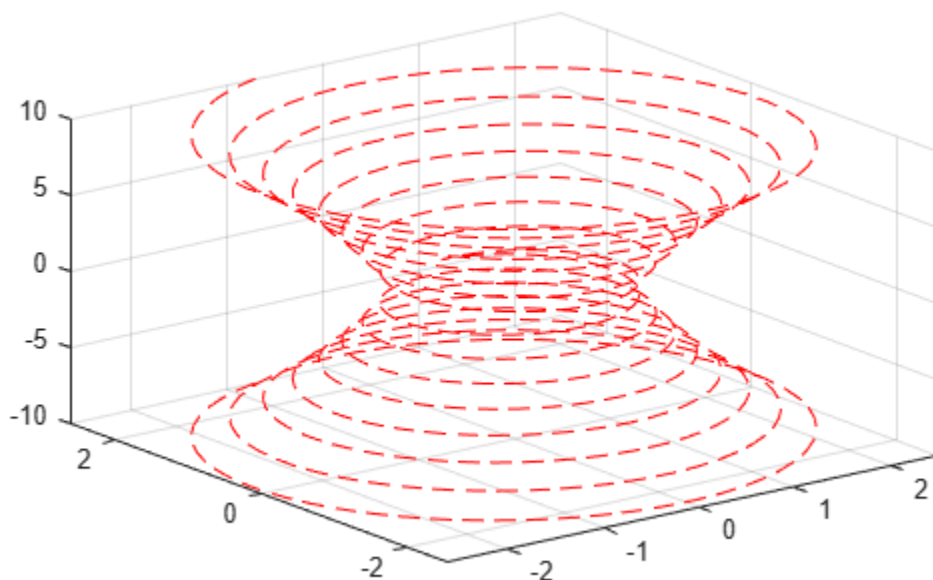
Plot the parametric curve  $x(t) = t \sin(5t)$  and  $y(t) = t \cos(5t)$ .

```
syms t
x = t*sin(5*t);
y = t*cos(5*t);
fplot(x, y)
grid on
```



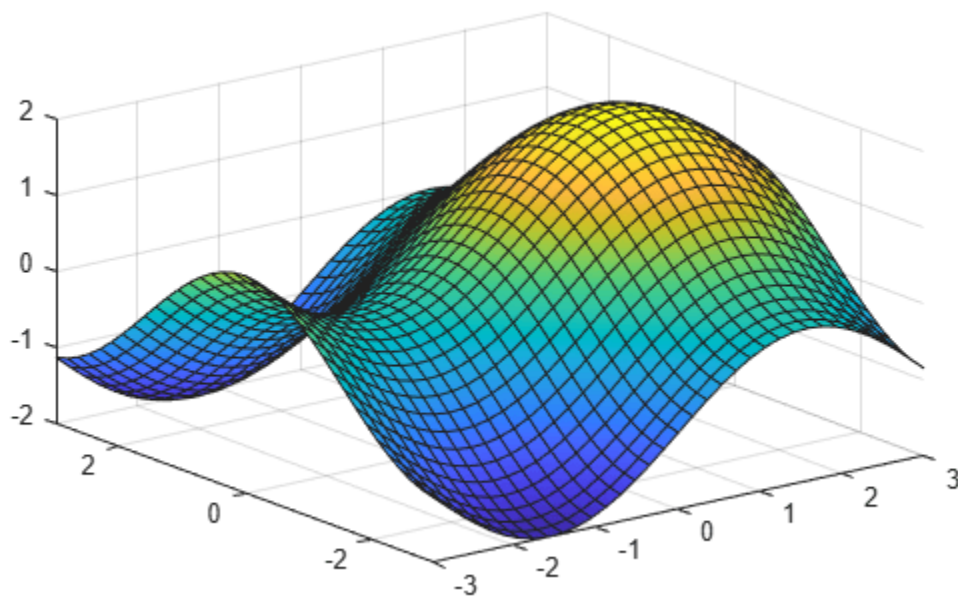
Plot the 3D parametric curve  $x(t) = e^{\frac{|t|}{10}}\sin(5|t|)$ ,  $y(t) = e^{\frac{|t|}{10}}\cos(5|t|)$  and  $z(t) = t$  from  $[-10, 10]$  with a dashed red line.

```
syms t
xt = exp(abs(t)/10).*sin(5*abs(t));
yt = exp(abs(t)/10).*cos(5*abs(t));
zt = t;
h = fplot3(xt,yt,zt, [-10,10], '--r');
```



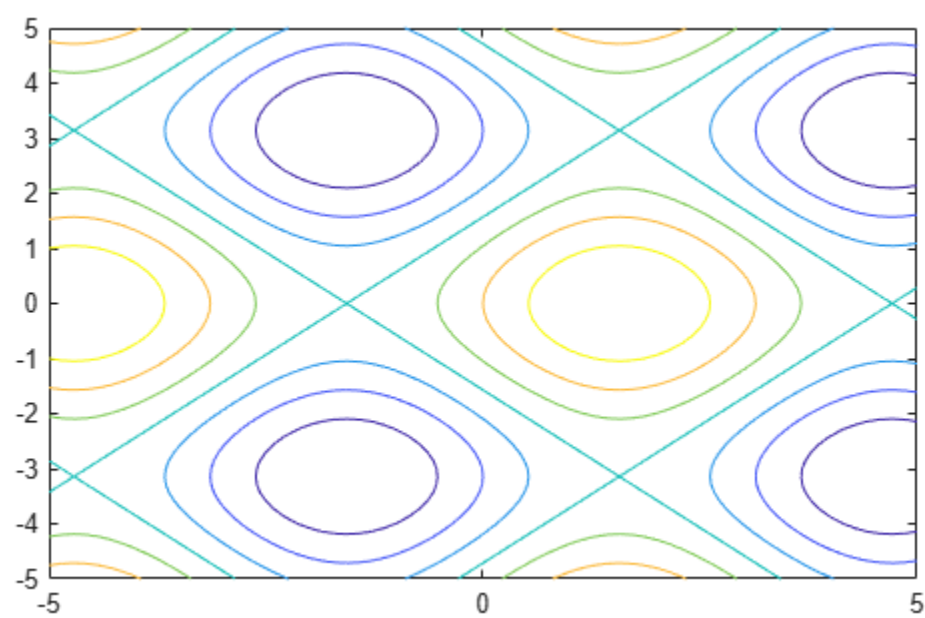
Plot the 3D surface  $f(x, y) = \sin(x) + \cos(y)$ .

```
syms x y
fsurf(sin(x) + cos(y))
```



Plot the 2D contours of the same surface.

```
fcontour(sin(x) + cos(y))
```



## Next Step Suggestions for Symbolic Workflows in Live Editor

Starting in R2021b, when you run code that generates a symbolic output in the Live Editor, Live Editor provides a context menu with next step suggestions that are specific to the output. To open the suggestion menu, you can point your mouse to the symbolic output and click the three-dot icon , or you can right-click the symbolic output. When reopening your symbolic workflow live script, run the code again to get the next step suggestions.

Starting in R2023b, you can also use keyboard shortcuts to navigate to the next step suggestions after you run your code. To move between code and output in the Live Editor, use the **Up/Down Arrow** when the output is inline, or use **Ctrl+Shift+O** (on macOS systems, use **Option+Command+O** instead) when the output is on the right. Activate the symbolic output by pressing the **Return** key. Use the shortcut **Shift+F10** to open the suggestion menus.

Using these suggestion menus, you can insert and execute function calls or Live Editor tasks into live scripts. Typical uses of these suggestion menus include:

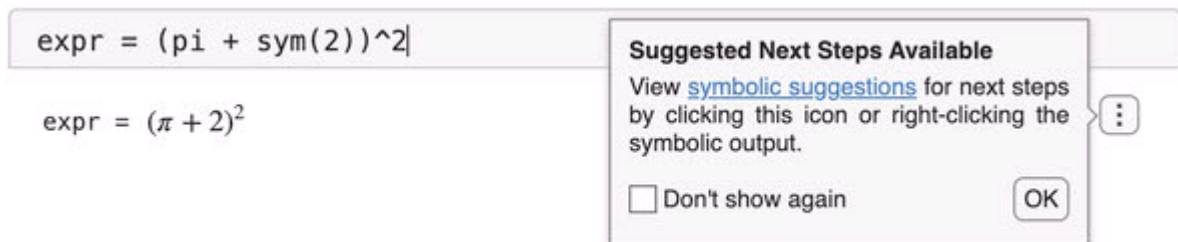
- Evaluate and convert symbolic expressions to numeric values.
- Simplify and manipulate, or solve symbolic math equations.
- Plot symbolic expressions.
- Perform matrix and vector operations, such as finding the inverse and determinant of a matrix, and finding the Jacobian and curl of a vector.
- Perform calculus functions, such as differentiation, integration, transforms, and solving differential equations.
- Convert between units of measurement and verify unit dimensions.

For example, in a new live script, create a symbolic expression.

```
expr = (pi + sym(2))^2
```

```
expr = ( $\pi + 2$ )2
```

Click **Run** to see the result, which is a symbolic output. When you first run code that generates a symbolic output, Live Editor shows the three-dot icon  for suggested next steps with a pop-up notification. You can also point to the symbolic output to bring up the three-dot icon. To suppress the pop-up notification in the rest of your workflow, select **Don't show again**.

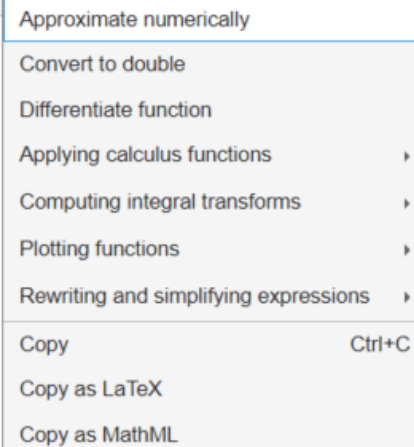


Click the three-dot icon  located to the right of the output to bring up the suggestion menu. When you point to a menu item, Live Editor gives you a preview of what happens when you select the menu

item. For instance, if you point to **Approximate numerically**, you see the new line of code it suggests.

```
expr = (pi + sym(2))^2
% Approximate numerically using variable precision floating point arithmetic
var = vpa(expr)
```

expr =  $(\pi + 2)^2$



Select **Approximate numerically** to add the suggested new line of code. Live Editor inserts the `vpa` function into the code region and automatically runs the current section to evaluate the expression numerically.

```
var = vpa(expr)
```

```
var = 26.435975015448531572685064532994
```

As another example, create a symbolic equation. Run a live script to generate the symbolic output. Solve the equation numerically using symbolic suggestions for next steps.

```
syms x a
eqn = (2*x^2 + a)/(x + 1) == 3
```

eqn =  $\frac{2x^2 + a}{x + 1} = 3$

To open the suggestion menu, you can also right-click the symbolic output. Select **Solving equations > Solve equation numerically**.



```
syms x a
eqn = (2*x^2 + a)/(x + 1) == 3
% Compute numeric solution of algebraic equation using variable precision floating point arithmetic
var2 = vpasolve(eqn,x)
```

eqn =  $\frac{2x^2 + a}{x + 1} = 3$

- Approximate numerically
- Solve equation analytically
- Substitute variables
- Plotting functions
- Rewriting and simplifying expressions
- Solving equations
  - Extract left side
  - Extract right side
  - Isolate variable
  - Solve equation numerically**
- Copy Ctrl+C
- Copy as LaTeX
- Copy as MathML
- Copy Output
- Copy All Output
- Clear Output
- Clear All Output
- Move Focus Between Code and Output Ctrl+Shift+O

When you click the **Solve equation numerically** suggestion, Live Editor inserts the `vpasolve` function into the code region. Live Editor then automatically runs the current section to solve the equation numerically.

```
var2 = vpasolve(eqn,x)
```

```
var2 =
(0.75 - 0.25√33.0 - 8.0 a)
(0.25√33.0 - 8.0 a + 0.75)
```

The following sections provide more examples showing how to use the interactive suggestion menus in symbolic workflows.

### Simplify Symbolic Expression

Create a symbolic expression that contains exponential functions and imaginary numbers. Run the following code to generate the symbolic output.

```
syms x
expr = 1i*(exp(-1i*x) - exp(1i*x))/(exp(-1i*x) + exp(1i*x))
```

expr =  $\frac{e^{-xi} i - e^{xi} i}{e^{-xi} + e^{xi}}$

To simplify the expression, right-click the symbolic output and select **Rewriting and simplifying expressions > Simplify expression**.

```
syms x
expr = 1i*(exp(-1i*x) - exp(1i*x))/(exp(-1i*x) + exp(1i*x))
% Simplify each element of matrix or expression
% The Simplify Symbolic Expression task will be inserted here with Method set to 'Simplify'
```

expr =

$$\frac{e^{-xi}i - e^{xi}i}{e^{-xi} + e^{xi}}$$

|                                       |              |
|---------------------------------------|--------------|
| Substitute variables                  |              |
| Applying calculus functions           | ▸            |
| Computing integral transforms         | ▸            |
| Converting numbers                    | ▸            |
| Rewriting and simplifying expressions | ▸            |
| Solving equations                     | ▸            |
| Copy                                  | Ctrl+C       |
| Copy as LaTeX                         |              |
| Copy as MathML                        |              |
| Copy Output                           |              |
| Copy All Output                       |              |
| Clear Output                          |              |
| Clear All Output                      |              |
| Move Focus Between Code and Output    | Ctrl+Shift+O |

|                            |
|----------------------------|
| Combine expression         |
| Expand expression          |
| Rewrite expression         |
| <b>Simplify expression</b> |
| Simplify fractions         |

Live Editor inserts and applies the Simplify Symbolic Expression task to interactively simplify or rearrange symbolic expressions. Change the computational effort to **Medium** to get a simpler result.

Simplify Symbolic Expression

▶

☒ Autorun
 

?

⋮

simplifiedExpr = Simplified expression **expr** using Simplify

Select expression

Expression

expr ▼

Specify simplification method

Method

Simplify ▼

Effort

Medium ▼

Minimum

Low

Medium

High

Full

Display result

☐ Expression

☒ Simplified expression

▶ Show code

simplifiedExpr = tan(x)

### Substitute Coefficients and Solve Quadratic Equation

Create a quadratic equation with coefficients  $a$ ,  $b$ , and  $c$ . Run the following code to generate the symbolic output.

```
syms a b c x
eqn = a*x^2 + b*x + c == 0

eqn =  $ax^2 + bx + c = 0$ 
```

To substitute the coefficients in the equation, right-click the output and select **Substitute variables**.

```
syms a b c x
eqn = a*x^2 + b*x + c == 0
% Substitute variables for values in expression or matrix: replace '0's and run code manually
var3 = subs(eqn,[a,b,c,x],[0,0,0,0])
```

eqn =  $a x^2 + b x + c = 0$

- Approximate numerically
- Solve equation analytically
- Substitute variables
- Rewriting and simplifying expressions ▶
- Solving equations ▶
- Copy Ctrl+C
- Copy as LaTeX
- Copy as MathML
- Copy Output
- Copy All Output
- Clear Output
- Clear All Output
- Move Focus Between Code and Output Ctrl+Shift+O

Live Editor inserts the `subs` function to substitute the coefficients and variables in the equations. For the `subs` function, Live Editor does not run the function automatically. To substitute  $a = 3$ ,  $b = 2$ , and  $c = 0$ , change the second and third arguments of the `subs` function to `[a,b,c]` and `[3,2,0]`. Run the Live Editor section afterward to apply the `subs` function.

```
var3 = subs(eqn,[a,b,c],[3,2,0])
```

var3 =  $3 x^2 + 2 x = 0$

To solve the quadratic equation, right-click the output and select **Solve equation analytically**.

```
var3 = subs(eqn,[a,b,c],[3,2,0])  
% Compute analytical solution of algebraic equation  
% The Solve Symbolic Equation task will be inserted here
```

$\text{var3} = 3x^2 + 2x = 0$

- Approximate numerically
- Solve equation analytically
- Substitute variables
- Plotting functions ▶
- Rewriting and simplifying expressions ▶
- Solving equations ▶
- Copy Ctrl+C
- Copy as LaTeX
- Copy as MathML
- Copy Output
- Copy All Output
- Clear Output
- Clear All Output
- Move Focus Between Code and Output Ctrl+Shift+O

Live Editor inserts and applies the Solve Symbolic Equation task to interactively find analytic solutions of symbolic equations.

Solve Symbolic Equation

▶

☒ Autorun

?

⋮

solution

=

Analytic solution of equation **var3** with respect to x

Select equation

Equation

var3 ▼

Variables

x ▼

for example: x

Specify solver options

☐ Return real solutions
 ☐ Return conditions
 ☐ Ignore analytic constraints

☐ Return one solution
 ☐ Expand all roots
 ☐ Ignore properties

Display result

☐ Equation
 ☒ Solution

▶ Show code

solution =

$$\begin{pmatrix} -\frac{2}{3} \\ 0 \end{pmatrix}$$

### Plot Explicit and Implicit Functions

Create three symbolic variables  $x$ ,  $y$ , and  $z$  and a sinusoidal function. Run the following code to generate the symbolic output.

```
syms x y z
f = sin(2*x)

f = sin(2 x)
```

To plot the sinusoidal function, right-click the output and select **Plotting functions > Plot function**.

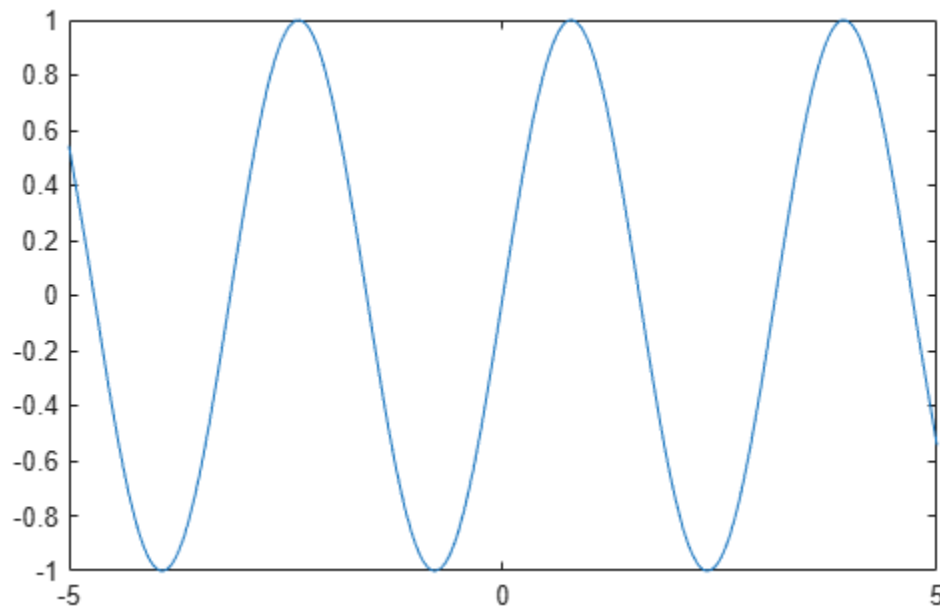
```
syms x y z
f = sin(2*x)
% Plot function f(x) or expression
fplot(f)
```

f = sin(2 x)

- Approximate numerically
- Substitute variables
- Applying calculus functions ›
- Computing integral transforms ›
- Plotting functions ›
  - Plot function
  - Plot implicit equation in 2-D
  - Plot implicit equation in 3-D
- Rewriting and simplifying expressions ›
- Solving equations ›
- Copy Ctrl+C
- Copy as LaTeX
- Copy as MathML
- Copy Output
- Copy All Output
- Clear Output
- Clear All Output
- Move Focus Between Code and Output Ctrl+Shift+O

Live Editor plots the sinusoidal function using `fplot`.

`fplot(f)`



Next, create an equation that represents a hyperbola. Run the following code to generate the symbolic output.

```
eqn = x^2 - y^2 == 1
```

```
eqn =  $x^2 - y^2 = 1$ 
```

To plot the sinusoidal function, right-click the output and select **Plotting functions > Plot implicit equation in 2-D**.



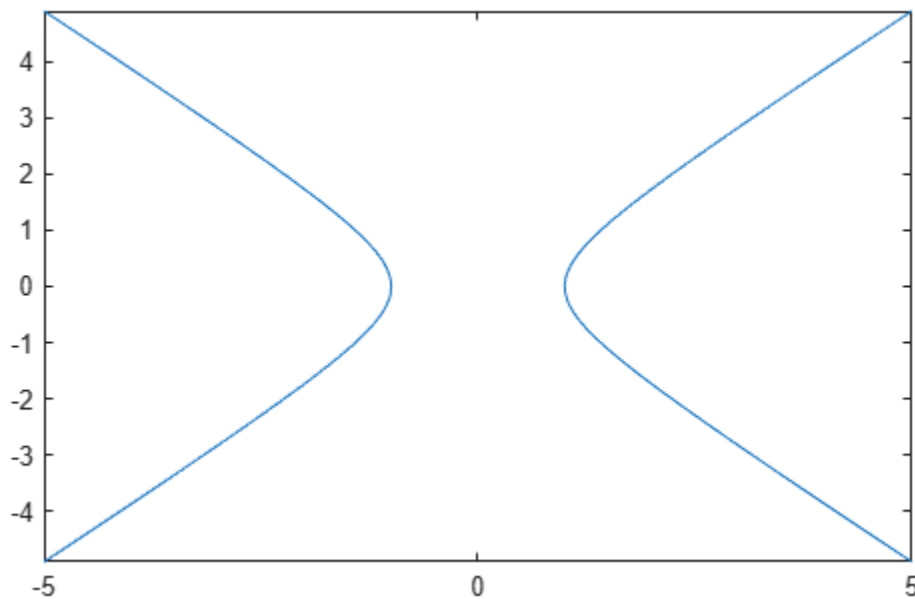
```
eqn = x^2 - y^2 == 1
% Plot 2-D implicit equation or function f(x,y)
fimplicit(eqn)
```

eqn =  $x^2 - y^2 = 1$

- Approximate numerically
- Solve equation analytically
- Substitute variables
- Plotting functions
  - Plot implicit equation in 2-D
  - Plot implicit equation in 3-D
- Rewriting and simplifying expressions
- Solving equations
- Copy Ctrl+C
- Copy as LaTeX
- Copy as MathML
- Copy Output
- Copy All Output
- Clear Output
- Clear All Output
- Move Focus Between Code and Output Ctrl+Shift+O

Live Editor plots the hyperbola using `fimplicit`.

```
fimplicit(eqn)
```



Next, create a symbolic function that represents a torus. Run the following code to generate the symbolic output.

$$f(x,y,z) = (x^2 + y^2 + z^2 + 3^2 - 2^2)^2 - 4*3^2*(x^2+y^2)$$

$$f(x, y, z) = (x^2 + y^2 + z^2 + 5)^2 - 36x^2 - 36y^2$$

To plot the torus, right-click the output and select **Plot implicit equation in 3-D**.

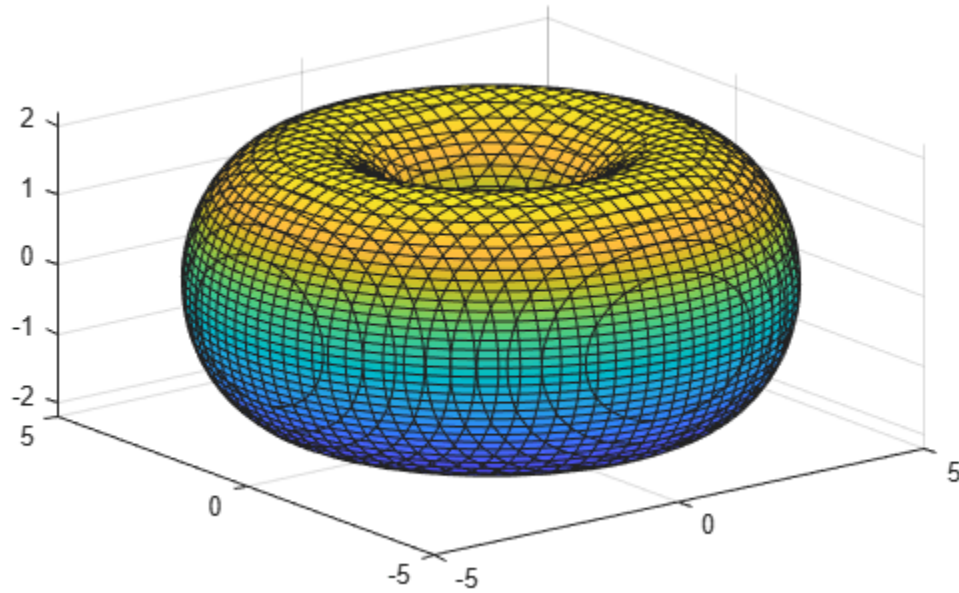
```
f(x,y,z) = (x^2 + y^2 + z^2 + 3^2 - 2^2)^2 - 4*3^2*(x^2+y^2)
% Plot 3-D implicit equation or function f(x,y,z)
fimplicit3(f)
```

$$f(x, y, z) = (x^2 + y^2 + z^2 + 5)^2 - 36x^2 - 36y^2$$

|                                                    |        |
|----------------------------------------------------|--------|
| Approximate numerically                            |        |
| Plot implicit equation in 3-D                      |        |
| Substitute variables                               |        |
| Applying calculus functions ▶                      |        |
| Computing integral transforms ▶                    |        |
| Rewriting and simplifying expressions ▶            |        |
| Solving equations ▶                                |        |
| Copy                                               | Ctrl+C |
| Copy as LaTeX                                      |        |
| Copy as MathML                                     |        |
| Copy Output                                        |        |
| Copy All Output                                    |        |
| Clear Output                                       |        |
| Clear All Output                                   |        |
| Move Focus Between Code and Output    Ctrl+Shift+O |        |

Live Editor plots the torus using `fimplicit3`.

`fimplicit3(f)`



### Apply Matrix Functions

Create three symbolic variables  $x$ ,  $y$ , and  $z$  and a symbolic matrix. Run the following code to generate the symbolic output.

```
syms x y z
M1 = sym([x^2 + a, x; y + 2, 3*y^2])
```

M1 =

$$\begin{pmatrix} x^2 + a & x \\ y + 2 & 3y^2 \end{pmatrix}$$

To invert the matrix, right-click the output and select **Applying matrix functions > Invert matrix**. Note that the **Applying matrix functions > Invert matrix** suggestion is available only if the symbolic output is a symbolic matrix.

```
syms x y z
M1 = sym([x^2 + a, x; y + 2, 3*y^2])
% Invert matrix
var4 = inv(M1)
```

M1 =

$$\begin{pmatrix} x^2 + a & x \\ y + 2 & 3y^2 \end{pmatrix}$$

|                                       |              |
|---------------------------------------|--------------|
| Approximate numerically               |              |
| Substitute variables                  |              |
| Applying calculus functions           | ▸            |
| Applying matrix functions             | ▸            |
| Computing integral transforms         | ▸            |
| Rewriting and simplifying expressions | ▸            |
| Solving equations                     | ▸            |
| Copy                                  | Ctrl+C       |
| Copy as LaTeX                         |              |
| Copy as MathML                        |              |
| Copy Output                           |              |
| Copy All Output                       |              |
| Clear Output                          |              |
| Clear All Output                      |              |
| Move Focus Between Code and Output    | Ctrl+Shift+O |

Compute determinant  
 Invert matrix

Live Editor inserts and applies the `inv` function to invert the matrix.

```
var4 = inv(M1)
```

var4 =

$$\begin{pmatrix} -\frac{3y^2}{\sigma_1} & \frac{x}{\sigma_1} \\ \frac{y+2}{\sigma_1} & -\frac{x^2+a}{\sigma_1} \end{pmatrix}$$

where

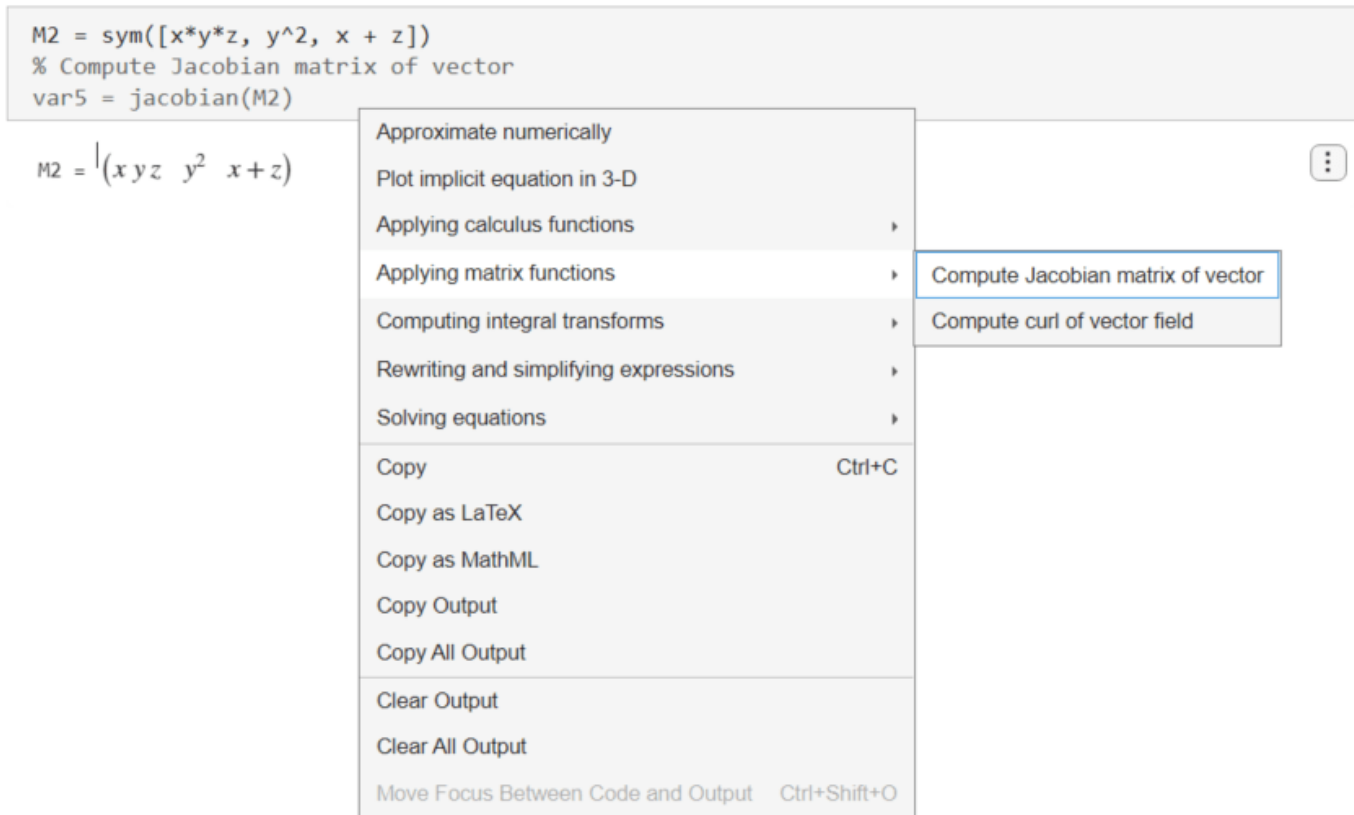
$$\sigma_1 = -3x^2y^2 + xy + 2x - 3ay^2$$

Next, create a 1-by-3 symbolic vector. Run the following code to generate the symbolic output.

```
M2 = sym([x*y*z, y^2, x + z])
```

M2 =  $(xyz \ y^2 \ x + z)$

To compute the Jacobian of the vector, right-click the output and select **Applying matrix functions > Compute Jacobian matrix of vector**. Note that the **Applying matrix functions > Compute Jacobian matrix of vector** suggestion is available only if the symbolic output is a symbolic vector.



Live Editor inserts and applies the `jacobian` function to compute the Jacobian of the vector.

```
var5 = jacobian(M2)
```

```
var5 =
```

$$\begin{pmatrix} y z & x z & x y \\ 0 & 2 y & 0 \\ 1 & 0 & 1 \end{pmatrix}$$

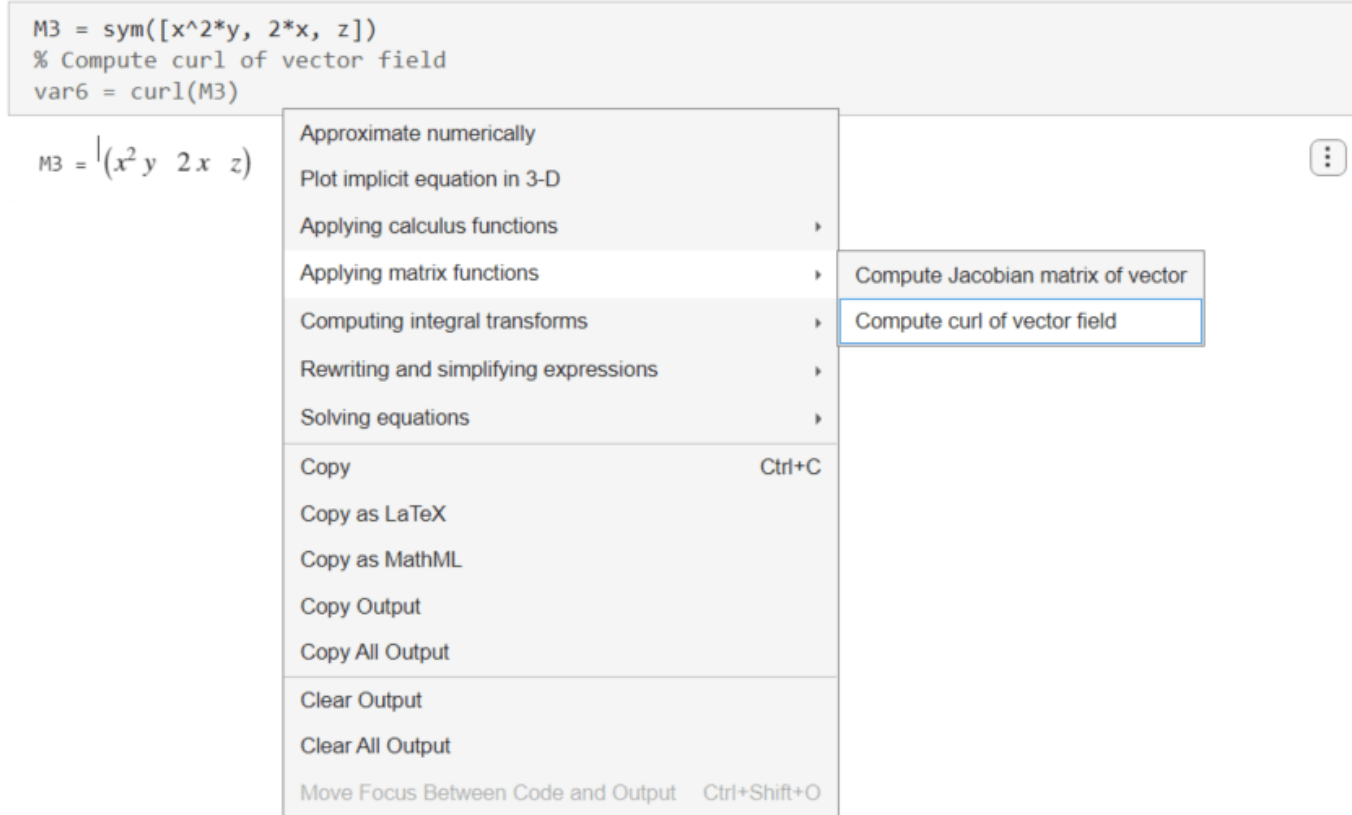
Next, create another 1-by-3 symbolic vector. Run the following code to generate the symbolic output.

```
M3 = sym([x^2*y, 2*x, z])
```

```
M3 =
```

$$\begin{pmatrix} x^2 y & 2 x & z \end{pmatrix}$$

To compute the curl of the vector, right-click the output and select **Applying matrix functions > Compute curl of vector field**.



Live Editor inserts and applies the `curl` function to compute the curl of the vector.

```
var6 = curl(M3)
```

```
var6 =
```

$$\begin{pmatrix} 0 \\ 0 \\ 2 - x^2 \end{pmatrix}$$

### Solve Differential Equation, Compute Integral Transform, and Find Poles

Create a second-order differential equation. Run the following code to generate the symbolic output.

```
syms f(x) s
eqn = diff(f,x,x) == -9*f
```

```
eqn(x) =
```

$$\frac{\partial^2}{\partial x^2} f(x) = -9 f(x)$$

To solve the differential equation, right-click the output and select **Solve differential equation**. Note that the **Solve differential equation** suggestion is available only if the symbolic output is a differential equation.

```
syms f(x) s
eqn = diff(f,x,x) == -9*f
% Compute analytical solution of ordinary differential equations
var7 = dsolve(eqn)
```

eqn(x) =  
 $\frac{\partial^2}{\partial x^2} f(x) = -9 f(x)$

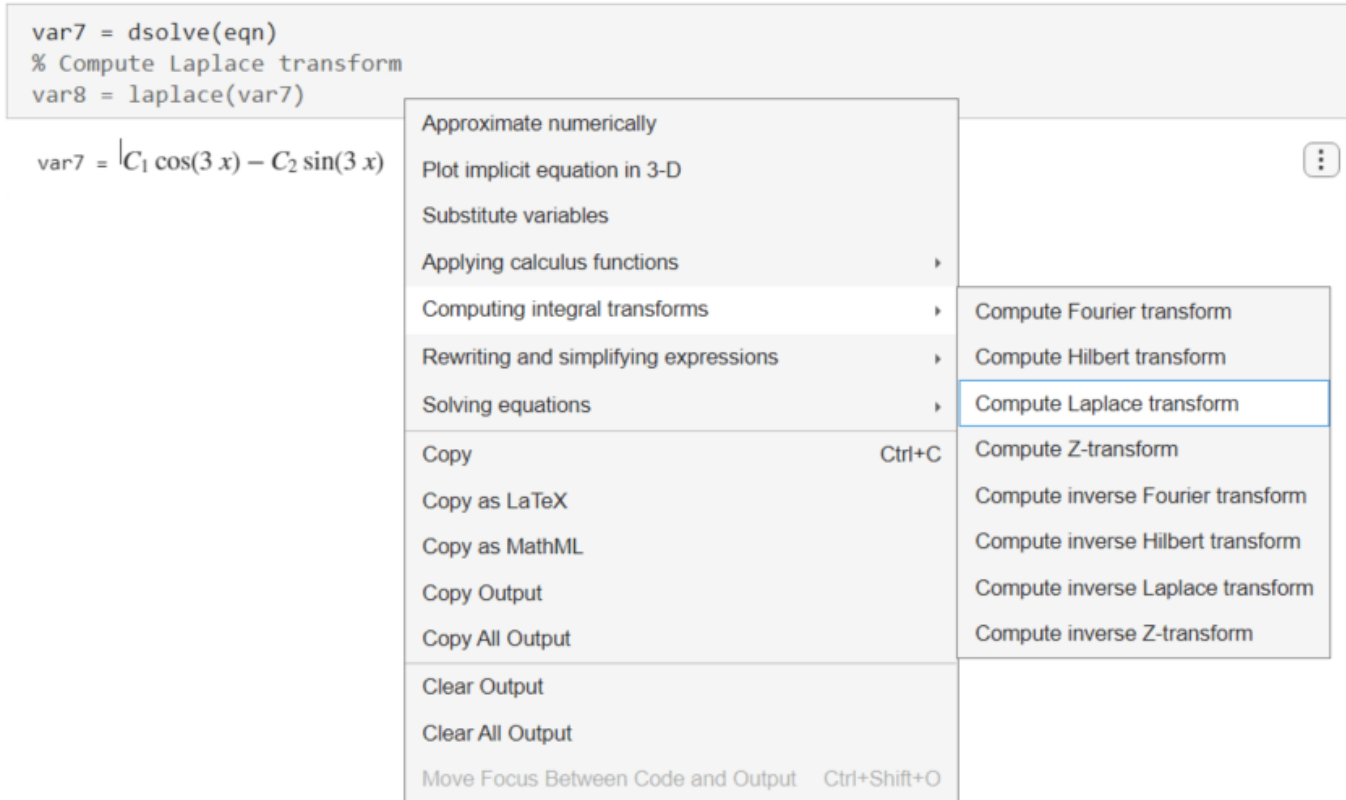
- Approximate numerically
- Simplify expression
- Solve differential equation
- Substitute variables
- Rewriting and simplifying expressions ›
- Solving equations ›
- Copy Ctrl+C
- Copy as LaTeX
- Copy as MathML
- Copy Output
- Copy All Output
- Clear Output
- Clear All Output
- Move Focus Between Code and Output Ctrl+Shift+O

Live Editor inserts and applies the `dsolve` function to solve the differential equation.

```
var7 = dsolve(eqn)
```

```
var7 = C1 cos(3 x) - C2 sin(3 x)
```

Next, find the Laplace transform of the solution. Right-click the output of `dsolve` and select **Computing integral transforms > Compute Laplace transform**.



Live Editor inserts and applies the `laplace` function to compute the Laplace transform.

```
var8 = laplace(var7)
```

```
var8 =
```

$$\frac{C_1 s}{s^2 + 9} - \frac{3 C_2}{s^2 + 9}$$

Finally, find the poles of the Laplace transform. Right-click the output of the Laplace transform and select **Applying calculus functions > Compute poles of function**.



```
var8 = laplace(var7)
% Compute poles of symbolic function
var9 = poles(var8,s)
```

$$\text{var8} = \frac{C_1 s}{s^2 + 9} - \frac{3 C_2}{s^2 + 9}$$

|                                       |                                  |
|---------------------------------------|----------------------------------|
| Approximate numerically               |                                  |
| Plot implicit equation in 3-D         |                                  |
| Substitute variables                  |                                  |
| Applying calculus functions           | › Compute Hessian matrix         |
| Computing integral transforms         | › Compute Taylor expansion       |
| Rewriting and simplifying expressions | › Compute gradient               |
| Solving equations                     | › Compute limit value            |
| Copy                                  | Ctrl+C Compute poles of function |
| Copy as LaTeX                         | Compute series expansion         |
| Copy as MathML                        | Differentiate function           |
| Copy Output                           | Integrate function               |
| Copy All Output                       |                                  |
| Clear Output                          |                                  |
| Clear All Output                      |                                  |
| Move Focus Between Code and Output    | Ctrl+Shift+O                     |

Live Editor inserts and applies the `poles` function to compute the poles.

```
var9 = poles(var8,s)
```

$$\text{var9} = \begin{pmatrix} -3i \\ 3i \end{pmatrix}$$

### Convert Units and Check Consistency of Units

Create a ratio of two lengths with different units. Run the following code to generate the symbolic output.

```
u = symunit;
ratio = (7*u.mi)/(420*u.ft)
```

$$\text{ratio} = \frac{1}{60} \frac{\text{mi}}{\text{ft}}$$

Convert the units to the meter-gram-second system. Right-click the output and select **Applying physical unit functions > Convert units**. Note that the **Applying physical unit functions** suggestion is available only if the symbolic output contains symbolic units.

```

u = symunit;
ratio = (7*u.mi)/(420*u.ft)
% Convert physical units in expression
var10 = unitConvert(ratio,[symunit("m"),symunit("g"),symunit("s")])

```

ratio =  
 $\frac{1}{60} \frac{\text{mi}}{\text{ft}}$

|                                       |              |
|---------------------------------------|--------------|
| Approximate numerically               |              |
| Differentiate function                |              |
| Applying calculus functions           | ▶            |
| Applying physical unit functions      | ▶            |
| Computing integral transforms         | ▶            |
| Plotting functions                    | ▶            |
| Rewriting and simplifying expressions | ▶            |
| Copy                                  | Ctrl+C       |
| Copy as LaTeX                         |              |
| Copy as MathML                        |              |
| Copy Output                           |              |
| Copy All Output                       |              |
| Clear Output                          |              |
| Clear All Output                      |              |
| Move Focus Between Code and Output    | Ctrl+Shift+O |

|                            |
|----------------------------|
| Check consistency of units |
| Convert units              |
| Separate value from unit   |

Live Editor inserts and applies the `unitConvert` function to convert the units to the meter-gram-second system.

```

var10 = unitConvert(ratio,[symunit('m'),symunit('g'),symunit('s')])
var10 = 88

```

Next, create two symbolic expressions `x` and `y` that describe the *x*- and *y*-coordinates of a moving projectile. Create a symbolic equation `r` that compares the units of `x` and `y` to the length units `m` and `ft`. Run the following code to generate the symbolic output.

```

syms theta ts x y r
g = 9.81*u.m/u.s^2;
v = 10*u.m/u.s;
t = ts*u.s;
x = v*cos(theta)*t;
y = v*sin(theta)*t + (-g*t^2)/2;
r = [x == u.m y == u.ft]

r =

$$\left( 10 \, ts \cos(\theta) \, \text{m} = \text{m} \, 10 \, ts \sin(\theta) \, \text{m} - \frac{981 \, ts^2}{200} \, \text{m} = \text{ft} \right)$$


```

Check the consistency of units in `r`. Right-click the output and select **Applying physical unit functions > Check consistency of units**.

```

syms theta ts x y r
g = 9.81*u.m/u.s^2;
v = 10*u.m/u.s;
t = ts*u.s;
x = v*cos(theta)*t;
y = v*sin(theta)*t + (-g*t^2)/2;
r = [x == u.m y == u.ft]
% Check for compatibility and consistency of physical units
var11 = checkUnits(r)

```

r =

$$\left( 10 \text{ ts } \cos(\theta) \text{ m} = \text{m} \quad 10 \text{ ts } \sin(\theta) \text{ m} - \frac{981 \text{ ts}^2}{200} \text{ m} = \text{ft} \right)$$

Check consistency of units

Convert units

Separate value from unit

Approximate numerically

Substitute variables

Applying physical unit functions

Plotting functions

Rewriting and simplifying expressions

Solving equations

Copy

Ctrl+C

Copy as LaTeX

Copy as MathML

Copy Output

Copy All Output

Clear Output

Clear All Output

Move Focus Between Code and Output Ctrl+Shift+O

Live Editor inserts and applies the `checkUnits` function to check the consistency and compatibility of the units in `r`.

The `checkUnits` function returns a structure containing the fields `Consistent` and `Compatible`. The `Consistent` field returns logical `1(true)` if all terms in `r` have the same dimension and same unit with a conversion factor of 1. The `Compatible` field returns logical `1(true)` if all terms have the same dimension, but not necessarily the same unit.

```
var11 = checkUnits(r)
```

```
var11 = struct with fields:
```

```
Consistent: [1 0]
```

```
Compatible: [1 1]
```



# Symbolic Computations in MATLAB

---

- “Find Symbolic Variables in Expressions, Functions, and Matrices” on page 2-2
- “Choose syms or sym Function” on page 2-4
- “Change Output Format of Symbolic and Variable-Precision Arithmetic” on page 2-8
- “Change Output Display Format of Symbolic Results in the Live Editor” on page 2-11
- “Add Subscripts, Superscripts, and Accents to Symbolic Variables in the Live Editor” on page 2-14
- “Copy and Paste Symbolic Output in Live Editor” on page 2-18
- “Check Symbolic Equations, Inequalities, and Conditional Statements” on page 2-23
- “Numeric to Symbolic Conversion” on page 2-29
- “Choose Numeric or Symbolic Arithmetic” on page 2-32
- “Increase Precision of Numeric Calculations” on page 2-36
- “Compute Binomial Coefficients Exactly” on page 2-38
- “Find Almost Integers with High-Precision Arithmetic” on page 2-41
- “Decimal Digits of PI” on page 2-45
- “Units of Measurement Tutorial” on page 2-47
- “Unit Conversions and Unit Systems” on page 2-53
- “Units and Unit Systems List” on page 2-60
- “Units in Physics Calculations” on page 2-74
- “The Physics of the Damped Harmonic Oscillator” on page 2-78
- “Evaluate Average Power Delivered by Wind Turbine” on page 2-87
- “Developing an Algorithm for Undistorting an Image” on page 2-93
- “Electric Dipole Moment and Radiation Power” on page 2-99
- “Validate Simulink Model Using Symbolic Math Toolbox” on page 2-102

## Find Symbolic Variables in Expressions, Functions, and Matrices

To find symbolic variables in an expression, function, or matrix, use `symvar`. For example, find all symbolic variables in symbolic expressions `f` and `g`:

```
syms a b n t x
f = x^n;
g = sin(a*t + b);
symvar(f)
```

```
ans =
[ n, x]
```

Here, `symvar` sorts all returned variables alphabetically. Similarly, you can find the symbolic variables in `g` by entering:

```
symvar(g)
```

```
ans =
[ a, b, t]
```

`symvar` also can return the first `n` symbolic variables found in a symbolic expression, matrix, or function. To specify the number of symbolic variables that you want `symvar` to return, use the second parameter of `symvar`. For example, return the first two variables found in symbolic expression `g`:

```
symvar(g, 2)
```

```
ans =
[ b, t]
```

Notice that the first two variables in this case are not `a` and `b`. When you call `symvar` with two arguments, it finds symbolic variables by their proximity to `x` before sorting them alphabetically.

When you call `symvar` on a symbolic function, `symvar` returns the function inputs before other variables.

```
syms x y w z
f(w, z) = x*w + y*z;
symvar(f)
```

```
ans =
[ w, z, x, y]
```

When called with two arguments for symbolic functions, `symvar` also follows this behavior.

```
symvar(f, 2)
```

```
ans =
[ w, z]
```

### Find a Default Symbolic Variable

If you do not specify an independent variable when performing substitution, differentiation, or integration, MATLAB uses a default variable. The default variable is typically the one closest alphabetically to `x` or, for symbolic functions, the first input argument of a function. To find which variable is chosen as a default variable, use the `symvar(f, 1)` command. For example:

```
syms s t
f = s + t;
symvar(f, 1)
```

```
ans =
t
```

```
syms sx tx
f = sx + tx;
symvar(f, 1)
```

```
ans =
tx
```

For more information on choosing the default symbolic variable, see `symvar`.

## Choose syms or sym Function

In Symbolic Math Toolbox™, you can create symbolic objects by using either `syms` or `sym`. These two functions are conceptually different.

- The `syms` function *creates* a symbolic object that is automatically assigned to a MATLAB® variable with the same name.
- The `sym` function *refers* to a symbolic object that can be assigned to a MATLAB variable with the same name or a different name.

The following examples discuss the differences between the `syms` and `sym` functions. For more examples on the use cases of each function, see `syms` or `sym`.

### Assign Symbolic Variables to MATLAB Variables

The `syms` function *creates* a variable dynamically. For example, the command `syms x` creates the symbolic variable `x` and automatically assigns it to a MATLAB variable with the same name.

```
syms x
x
```

```
x = x
```

You can then use the variable `x` in the MATLAB workspace for symbolic workflow, such as finding the roots of a polynomial.

```
f = x^2 + x - 6
```

```
f = x2 + x - 6
```

```
x0 = solve(f)
```

```
x0 =  
    (-3)  
     2
```

The `sym` function *refers* to a symbolic variable, which you can then assign to a MATLAB variable with a different name. For example, the command `f1 = sym('x')` refers to the symbolic variable `x` and assigns it to the MATLAB variable `f1`.

```
clear  
f1 = sym('x')
```

```
f1 = x
```

You can then use the variable `f1` in the MATLAB workspace for symbolic workflow, such as finding the zeros of a sine function.

```
f2 = sin(f1)
```

```
f2 = sin(x)
```

```
[solx,parameters,conditions] = solve(f2,f1,'ReturnConditions',true)
```

```
solx =  $\pi k$ 
```

```
parameters = k
```



```
conditions =  $k \in \mathbb{Z}$ 
```

### Create Symbolic Number

Use the `syms` function to *create* a symbolic variable `x` and automatically assign it to a MATLAB variable `x`. When you assign a number to the MATLAB variable `x`, the number is represented in double-precision and this assignment overwrites the previous assignment to a symbolic variable. The class of `x` becomes `double`.

```
syms x
x = 1/33
```

```
x =
0.0303
```

```
class(x)
```

```
ans =
'double'
```

Use the `sym` function to *refer* to an exact symbolic number without floating-point approximation. You can then assign this number to the MATLAB variable `x`. The class of `x` is `sym`.

```
x = sym('1/33')
```

```
x =
 $\frac{1}{33}$ 
```

```
class(x)
```

```
ans =
'sym'
```

### Create Symbolic Variable with Assumptions

When you create a symbolic variable with an assumption, MATLAB stores the symbolic variable and its assumption separately.

Use `syms` to *create* a symbolic variable that is assigned to a MATLAB variable with the same name. You get a fresh symbolic variable with no assumptions. If you declare a variable using `syms`, existing assumptions are cleared.

```
syms x positive
syms x
assumptions
```

```
ans =
```

```
Empty sym: 1-by-0
```

Use `sym` to *refer* to an existing symbolic variable. If this symbolic variable was used in your MATLAB session before, then `sym` refers to it and its current assumption. If it was not used before, then `sym` creates it with no assumptions.

```
syms x positive
x = sym('x');
assumptions

ans = 0 < x
```

### Create Many Symbolic Variables

To *create* many symbolic variables simultaneously, using the `syms` function is more convenient. You can create multiple variables in one line of code.

```
syms a b c
```

When you use `sym`, you have to declare MATLAB variables one by one and *refer* them to the corresponding symbolic variables.

```
a = sym('a');
b = sym('b');
c = sym('c');
```

### Create Array of Symbolic Variables

To declare a symbolic array that contains symbolic variables as its elements, you can use either `syms` or `sym`.

The command `syms a [1 3]` *creates* a 1-by-3 symbolic array `a` and the symbolic variables `a1`, `a2`, and `a3` in the workspace. The symbolic variables `a1`, `a2`, and `a3` are automatically assigned to the symbolic array `a`.

```
clear
syms a [1 3]
a
```

```
a = (a1 a2 a3)
```

```
whos
```

| Name | Size | Bytes | Class | Attributes |
|------|------|-------|-------|------------|
| a    | 1x3  | 8     | sym   |            |
| a1   | 1x1  | 8     | sym   |            |
| a2   | 1x1  | 8     | sym   |            |
| a3   | 1x1  | 8     | sym   |            |

The command `a = sym('a',[1 3])` *refers* to the symbolic variables `a1`, `a2`, and `a3`, which are assigned to the symbolic array `a` in the workspace. The elements `a1`, `a2`, and `a3` are not created in the workspace.

```
clear
a = sym('a',[1 3])
```

```
a = (a1 a2 a3)
```

```
whos
```

| Name | Size | Bytes | Class | Attributes |
|------|------|-------|-------|------------|
| a    | 1x3  | 8     | sym   |            |

## Create Symbolic Variable in Parallel For Loop

To create a symbolic variable in each worker for parallel computation, use `sym`. For example, you can create a symbolic variable `k` in a `parfor` loop that performs sums of series using `symsum`.

```
S = zeros(1,10);
parfor i = 1:10
    k = sym('k');
    S(i) = symsum(1/k^i,k,1,Inf);
end
```

```
Starting parallel pool (parpool) using the 'Processes' profile ...
Connected to the parallel pool (number of workers: 6).
```

You cannot use `syms` to create a symbolic variable in a `parfor` loop because it modifies the global state of the workspace.

## Create Symbolic Variable in Function

To declare a symbolic variable within a function, use `sym`. For example, you can explicitly define a MATLAB variable `x` in the parent function workspace and refer `x` to a symbolic variable with the same name.

```
function primaryFx
    x = sym('x')
    function nestedFx
        ...
    end
end
```

Functions make the workspace static, so you cannot dynamically add variables using `syms`.

## See Also

### Related Examples

- “Create Symbolic Numbers, Variables, and Expressions” on page 1-3
- “Find Symbolic Variables in Expressions, Functions, and Matrices” on page 2-2
- “Use Assumptions on Symbolic Variables” on page 1-41

## Change Output Format of Symbolic and Variable-Precision Arithmetic

Symbolic Math Toolbox™ enables you to perform analytical calculations by using symbolic numbers, variables, functions, and expressions to represent mathematical objects in their exact form. You can also use variable-precision arithmetic to specify the significant digits used in symbolic computations. However, symbolic outputs in their analytical form can be long and complicated. You can change the display format of symbolic outputs by changing the settings for the toolbox.

This example shows how to:

- Use `sym` and `syms` to create symbolic numbers, variables, functions, and expressions to perform analytical calculations.
- Use `digits` and `vpa` to evaluate symbolic objects with variable-precision arithmetic to specified significant digits.
- Use `sympref` to set the output display format of symbolic objects to a short, fixed-decimal format with four digits after the decimal point, without affecting the precision used in symbolic computations.

If you want to use the results of your symbolic computations in numeric computations using MATLAB® (without Symbolic Math Toolbox), you can convert symbolic values to numeric values by using the `double` and `matlabFunction` functions.

### Symbolic Arithmetic

Symbolic Math Toolbox provides the `sym` and `syms` functions to create symbolic objects for symbolic computations. In symbolic arithmetic, you can perform analytical computations involving numbers and variables in their exact form, such as  $5/11$ ,  $\pi$ , or  $x^{(1/2)}$ .

For example, create the number  $\sqrt{\frac{11}{35}}$  in its exact form by using `sym`.

```
a = sqrt(sym(11)/35)
```

```
a =  

$$\frac{\sqrt{11}\sqrt{35}}{35}$$

```

Create a symbolic variable `x` using `syms`. Create a symbolic function `y(x)` that takes the variable `x` as an input argument.

```
syms x  
y(x) = a*x^2
```

```
y(x) =  

$$\frac{\sqrt{11}\sqrt{35}x^2}{35}$$

```

Evaluate `y` for `x = sqrt(35)`.

```
y_eval = y(sqrt(35))
```

```
y_eval =  $\sqrt{11}\sqrt{35}$ 
```

## Variable-Precision Arithmetic

To specify the number of significant digits when performing calculations in Symbolic Math Toolbox with variable-precision arithmetic, you can use the `digits` and `vpa` functions.

For example, specify 10 significant digits and evaluate the previous symbolic values and functions to the specified precision.

```
digits(10)
a_vpa = vpa(a)

a_vpa = 0.5606119106

y_vpa(x) = vpa(y)

y_vpa(x) = 0.5606119106 x2

y_vpaeval = y_vpa(sqrt(35))

y_vpaeval = 19.62141687
```

## Change Output Display Format

The results of symbolic computations in their analytical form can be long and complicated. To change the display format of symbolic results, you can use the `sympref` function. This function sets the settings used in symbolic computations.

For example, set the "FloatingPointOutput" setting to `true`. Then display the values and functions from the previous symbolic and variable-precision arithmetic examples. The output of all symbolic calculations are now in the short, fixed-decimal format with four digits after the decimal point.

```
sympref("FloatingPointOutput",true);
a

a = 0.5606

a_vpa

a_vpa = 0.5606

y

y(x) = 0.5606 x2

y_vpa

y_vpa(x) = 0.5606 x2

y_eval

y_eval = 19.6214

y_vpaeval

y_vpaeval = 19.6214
```

Setting "FloatingPointOutput" to `true` affects only the output display format and does not affect the floating-point precision in symbolic computations. For example, restore the default symbolic

settings by using `sympref("default")`. The variables `y_eval` and `y_vpaeval` still contain the exact symbolic arithmetic and 10-digit variable-precision arithmetic, respectively.

```
sympref("default");  
y_eval  
  
y_eval =  $\sqrt{11}\sqrt{35}$   
y_vpaeval  
  
y_vpaeval = 19.62141687
```

### Convert Symbolic Values to Numeric Values

You can also convert symbolic values to the default MATLAB double-precision values.

For example, you can use the `double` function to convert symbolic numbers to double-precision numbers.

```
a_num = double(a)  
  
a_num =  
0.5606
```

You can use `matlabFunction` to generate a MATLAB function that operates on the double data type from an existing symbolic function.

```
y_num = matlabFunction(y)  
  
y_num = function_handle with value:  
@ (x) (sqrt(1.1e+1).*sqrt(3.5e+1).*x.^2)./3.5e+1
```

Evaluate the generated function `y_num` for `x = sqrt(35)`.

```
y_numeval = y_num(sqrt(35))  
  
y_numeval =  
19.6214
```

Setting "FloatingPointOutput" to `true` by using `sympref` affects only the output display format of symbolic results. To change the output display format for MATLAB numeric results, use the `format` function.

### See Also

#### Related Examples

- "Create Symbolic Numbers, Variables, and Expressions" on page 1-3
- "Change Output Display Format of Symbolic Results in the Live Editor" on page 2-11

## Change Output Display Format of Symbolic Results in the Live Editor

This example shows how to modify the output display format of symbolic results in the MATLAB® Live Editor by using the `sympref` function. To demonstrate the use of the function, this example uses a third-degree polynomial.

### Modify Output Order of Third-Degree Polynomial

Create a third-degree polynomial consisting of one variable and three coefficients. Define the variable and coefficients as symbolic variables by using the `syms` command.

```
syms x a b c
f(x) = (a*x^2 + b)*(b*x - a) + c
```

$$f(x) = c - (ax^2 + b)(a - bx)$$

Symbolic settings persist through successive MATLAB sessions. Restore all symbolic settings to the default values. Expand the polynomial and return the output in the default order.

```
sympref('default');
poly = expand(f)
```

$$\text{poly}(x) = -a^2x^2 + abx^3 - ab + b^2x + c$$

The default output format displays the terms of a symbolic polynomial in alphabetical order, without distinguishing the different symbolic variables in each monomial term.

To change the output order of a polynomial, change the `'PolynomialDisplayStyle'` setting. The `'ascend'` option sorts the output in an ascending order based on the standard mathematical notation for polynomials. Here, the variable `x` with the highest order in a monomial term is displayed last.

```
sympref('PolynomialDisplayStyle','ascend');
poly
```

$$\text{poly}(x) = c - ab + b^2x - a^2x^2 + abx^3$$

### Modify Output Display of Polynomial Roots

By default, symbolic results in Live Scripts are typeset in standard mathematical notation, long expressions are abbreviated, and matrices are set in parentheses (round brackets). You can modify the output display format by modifying symbolic settings.

Find the roots or zeros of the third-degree polynomial using `solve`. In Symbolic Math Toolbox™, the `root` function represents the roots of a polynomial.

```
sols = solve(poly,x)
```

```
sols =
```

$$\begin{pmatrix} \text{root}(\sigma_1, z, 1) \\ \text{root}(\sigma_1, z, 2) \\ \text{root}(\sigma_1, z, 3) \end{pmatrix}$$

where

$$\sigma_1 = a b z^3 - a^2 z^2 + b^2 z - a b + c$$

To display the results without being abbreviated, set 'AbbreviateOutput' to false.

```
sympref('AbbreviateOutput', false);
sols
```

$$\text{sols} = \begin{pmatrix} \text{root}(a b z^3 - a^2 z^2 + b^2 z - a b + c, z, 1) \\ \text{root}(a b z^3 - a^2 z^2 + b^2 z - a b + c, z, 2) \\ \text{root}(a b z^3 - a^2 z^2 + b^2 z - a b + c, z, 3) \end{pmatrix}$$

To display the symbolic matrix with square brackets, rather than parentheses, set 'MatrixWithSquareBrackets' to true.

```
sympref('MatrixWithSquareBrackets', true);
sols
```

$$\text{sols} = \begin{bmatrix} \text{root}(a b z^3 - a^2 z^2 + b^2 z - a b + c, z, 1) \\ \text{root}(a b z^3 - a^2 z^2 + b^2 z - a b + c, z, 2) \\ \text{root}(a b z^3 - a^2 z^2 + b^2 z - a b + c, z, 3) \end{bmatrix}$$

To display the results in ASCII characters instead of in typeset mathematical notation, set 'TypesetOutput' to false.

```
sympref('TypesetOutput', false);
sols
```

$$\begin{aligned} \text{sols} = & \\ & \text{root}(a*b*z^3 - a^2*z^2 + b^2*z - a*b + c, z, 1) \\ & \text{root}(a*b*z^3 - a^2*z^2 + b^2*z - a*b + c, z, 2) \\ & \text{root}(a*b*z^3 - a^2*z^2 + b^2*z - a*b + c, z, 3) \end{aligned}$$

The settings you set using `sympref` persist through your current and future MATLAB sessions. Restore the symbolic settings to the default values for the next step.

```
sympref('default');
```

### Display Floating-Point Output of Symbolic Numbers

Replace the polynomial coefficients with symbolic numbers using `subs`. The function returns the solutions without any approximation.



```
numSols = subs(sols,[a b c],[sqrt(2) pi 0.001])
```

```
numSols =  


$$\begin{pmatrix} \text{root}(\sigma_1, z, 1) \\ \text{root}(\sigma_1, z, 2) \\ \text{root}(\sigma_1, z, 3) \end{pmatrix}$$

```

where

$$\sigma_1 = 1000 \pi \sqrt{2} z^3 - 2000 z^2 + 1000 z \pi^2 - 1000 \pi \sqrt{2} + 1$$

To display the results in floating-point format, set `'FloatingPointOutput'` to `true`. This option displays symbolic numbers in fixed-decimal format with 4 digits after the decimal point. For a complex result of class `'sym'`, this setting affects the real and imaginary parts independently.

```
sympref('FloatingPointOutput',true);  
numSols
```

```
numSols =  


$$\begin{pmatrix} 0.4501 \\ 4.6427\text{e-}05 - 1.4904 \text{ i} \\ 4.6427\text{e-}05 + 1.4904 \text{ i} \end{pmatrix}$$

```

The display settings you set do not affect the computation of symbolic results. You can use the `vpa` function to approximate symbolic numbers in floating-point precision with 4 significant digits.

```
vpaSols = vpa(numSols,4)
```

```
vpaSols =  


$$\begin{pmatrix} 0.4501 \\ -1.4904 \text{ i} \\ 1.4904 \text{ i} \end{pmatrix}$$

```

Restore the default value of `'FloatingPointOutput'` by specifying the `'default'` option.

```
sympref('FloatingPointOutput','default');
```

## See Also

### Related Examples

- “Create Symbolic Numbers, Variables, and Expressions” on page 1-3
- “Change Output Format of Symbolic and Variable-Precision Arithmetic” on page 2-8

## Add Subscripts, Superscripts, and Accents to Symbolic Variables in the Live Editor

Starting in R2019a, MATLAB® Live Editor displays symbolic variables with subscripts, superscripts, and accents in standard mathematical notation. This example shows how to add subscripts, superscripts, and accents to symbolic variables in the MATLAB Live Editor.

### Add Subscripts and Superscripts

To add subscripts to symbolic variables in live scripts, append the corresponding index to the variable using one underscore (\_). For example, create two symbolic variables with subscripts using `syms`. Use these variables in an expression.

```
syms F_a F_b
Ftot = F_a + F_b

Ftot =  $F_a + F_b$ 
```

You can also use `sym` to create a symbolic variable with a subscript and assign the variable to a symbolic expression.

```
Fa = sym("F_a")

Fa =  $F_a$ 
```

To add superscripts to symbolic variables, append the corresponding index to the variable using two underscores (\_\_). For example, create two symbolic variables with superscripts.

```
syms F__a F__b
Ftot = F__a + F__b

Ftot =  $F^a + F^b$ 
```

When you assign symbolic variables to an expression, the symbolic expression is displayed in ASCII format.

### Add Accents

To add accents to symbolic variables in live scripts, append the corresponding suffix to the variable using an underscore (\_). For example, create symbolic variables with one dot and two dots over the symbol  $x$ . Use these variables in an equation.

```
syms x x_dot x_ddot c m k
eq1 = m*x_ddot - c*x_dot + k*x == 0

eq1 =  $kx - c\dot{x} + m\ddot{x} = 0$ 
```

When you compute the complex conjugate of a symbolic variable with an accent, a bar notation is added above the variable. For example, find the complex conjugate of `x_dot` using the `conj` function.

```
xConj = conj(x_dot)

xConj =  $\overline{\dot{x}}$ 
```

This accent list shows the supported accent suffixes and the corresponding display output. The dagger accent with † symbol is available since R2022b and the degree accent with ° symbol is available since R2023a.

```
suffix = ["ast"; "dag"; "deg"; "hat"; "tilde"; ...
    "vec"; "bar"; "ubar"; "dot"; "ddot"; "tdot"; ...
    "qdot"; "prime"; "dprime"; "tprime"; "qprime"];
accentList = [suffix, sym("x_" + suffix)]
```

```
accentList =
    ( ast  x*
      dag  x†
      deg  x°
      hat  x̂
      tilde x̃
      vec  x→
      bar  x̄
      ubar x̅
      dot  ẋ
      ddot ẍ
      tdot x̣
      qdot x̤
      prime x'
      dprime x''
      tprime x'''
      qprime x''')
```

When you compute the complex conjugate transpose of a matrix containing symbolic variables, a bar notation is also added above each variable. For example, find the complex conjugate transpose of the symbolic variables in `accentList(:,2)` using the `ctranspose` or `'` function.

```
conjVar = accentList(:,2) '
conjVar =
    (x* x† x° x̂ x̃ x→ x̄ x̅ ẋ ẍ x̣ x̤ x' x'' x''' x''')
```

When you compute the nonconjugate transpose of a matrix containing symbolic variables, the display output is unchanged. For example, find the nonconjugate transpose of the symbolic variables in `accentList(:,2)` using the `transpose` or `.'` function.

```
nonconjVar = accentList(:,2) .'
nonconjVar =
    (x* x† x° x̂ x̃ x→ x̄ x̅ ẋ ẍ x̣ x̤ x' x'' x''' x''')
```

### Add Multiple Subscripts, Superscripts, and Accents

You can create symbolic variables with multiple subscripts, superscripts, and accents. The multiple suffixes are assigned to the symbolic variables from left to right.

Create symbolic variables with multiple subscripts and superscripts. If you add multiple subscripts and superscripts, then the input indices are separated with a comma and displayed from left to right.

```
x1 = sym("x_b_1__a__1")
```

$$x1 = x_{b,1}^{a,1}$$

```
x2 = sym("x__b_1_a__1")
```

$$x2 = x_{1,a}^{b,1}$$

Now create symbolic variables with multiple accents. If you add multiple accents, then the input accents are assigned from left to right to the closest preceding variable or index.

```
v1 = sym("v_prime_vec")
```

$$v1 = \vec{v'}$$

```
v2 = sym("v_vec_prime")
```

$$v2 = \vec{v'}'$$

```
va = sym("v__a_bar_prime")
```

$$va = v^{\vec{a'}}$$

```
vb = sym("v_bar__b_prime")
```

$$vb = \vec{v}^{b'}$$

Adding suffixes to symbolic variables can produce similar output. However, the variables are equal only if their suffixes are also in the same order. For example, create three symbolic variables that produce similar output.

```
syms F_t__a
```

```
F1 = F_t__a
```

$$F1 = F_t^a$$

```
F2 = sym("F_t__a")
```

$$F2 = F_t^a$$

```
F3 = sym("F__a_t")
```

$$F3 = F_t^a$$

Determine if the symbolic variables are equal to each other using the `isequal` function.

```
tf_12 = isequal(F1,F2)
```

$$tf_{12} = \text{logical}$$

```
tf_23 = isequal(F2,F3)
```

```
tf_23 = logical
      0
```

## Add Signs to Subscripts and Superscripts

Starting in R2022b, you can also add signs as suffixes when creating symbolic variables. The supported signs are  $-$ ,  $+$ ,  $\pm$ , and  $\#$ . This sign list shows the supported sign suffixes and the corresponding display output.

```
suffix = ["minus"; "plus"; "plusmn"; "hash"];
signList = [suffix, sym("A_" + suffix)]
```

```
signList =
    (
      minus  A_-
      plus   A_+
      plusmn A_±
      hash   A_#
    )
```

You can add these signs in combinations with other symbols when creating symbolic variables. To combine these signs with other subscripts or superscripts, you can use one underscore ( $_$ ) or two underscores ( $__$ ) with other suffixes, respectively. For example:

```
A1 = sym("A_minus_x_bar")
```

```
A1 = A_ $\bar{x}$ 
```

```
A2 = sym("A__plusmn__c")
```

```
A2 = A $^{\pm c}$ 
```

```
A3 = sym("A_hash_123")
```

```
A3 = A $_{\#123}$ 
```

```
A4 = sym("A_minus_x_plus_y__r__theta")
```

```
A4 = A $_{-x,+y}^{r,\theta}$ 
```

## See Also

### Related Examples

- “Create Symbolic Numbers, Variables, and Expressions” on page 1-3
- “Find Symbolic Variables in Expressions, Functions, and Matrices” on page 2-2
- “Use Assumptions on Symbolic Variables” on page 1-41

## Copy and Paste Symbolic Output in Live Editor

This example shows how to copy symbolic output and paste it as MATLAB® code or equation typeset in the MATLAB Live Editor. To demonstrate this capability, this example uses a cubic (third-degree) polynomial.

### Copy Output and Paste as MATLAB Code

Solve the cubic polynomial  $x^3 + bx + c = 0$ . The solutions are displayed in terms of the abbreviated expression  $\sigma_1$ .

```
syms b c x
S = solve(x^3 + b*x + c == 0, x, "MaxDegree", 3)
```

S =

$$\begin{pmatrix} \sigma_1 - \frac{b}{3\sigma_1} \\ \frac{b}{6\sigma_1} - \frac{\sigma_1}{2} - \frac{\sqrt{3}\left(\frac{b}{3\sigma_1} + \sigma_1\right)i}{2} \\ \frac{b}{6\sigma_1} - \frac{\sigma_1}{2} + \frac{\sqrt{3}\left(\frac{b}{3\sigma_1} + \sigma_1\right)i}{2} \end{pmatrix}$$

where

$$\sigma_1 = \left( \sqrt{\frac{b^3}{27} + \frac{c^2}{4}} - \frac{c}{2} \right)^{1/3}$$

After you run your code, right-click the symbolic output. Select **Copy Output** to copy the symbolic expressions that represent the roots of the cubic polynomial.

Starting in R2023b, you can also use keyboard shortcuts to navigate to the symbolic output. To move between code and output in the Live Editor, use the **Up/Down Arrow** when the output is inline, or use **Ctrl+Shift+O** (on macOS systems, use **Option+Command+O** instead) when the output is on the right. Use the shortcut **Shift+F10** to open the menu selection to copy output.

The screenshot shows the MATLAB Live Editor interface. The script in the editor is:

```
1 syms b c x
2 S = solve(x^3 + b*x + c == 0, x, "MaxDegree", 3)
```

The output area displays the symbolic solution  $S =$  as a vector of three roots. The first root is  $\sigma_1 - \frac{b}{3\sigma_1}$ . The second and third roots are complex conjugates involving  $\sqrt{3}$  and  $i$ . Below the roots, it says "where" and defines  $\sigma_1 = \left( \sqrt{\frac{b^3}{27} + \frac{c^2}{4}} - \frac{c}{2} \right)^{1/3}$ . A context menu is open over the output, with the "Copy" option selected, which is also labeled with "Ctrl+C". Other options in the menu include "Compute Jacobian matrix of vector", "Applying calculus functions", "Computing integral transforms", "Converting numbers", "Rewriting and simplifying expressions", "Solving equations", "Copy as LaTeX", "Copy as MathML", "Copy Output", "Copy All Output", "Clear Output", "Clear All Output", and "Move Focus Between Code and Output" (labeled "Ctrl+Shift+O").

Insert code in the live script and assign the polynomial roots to the variable `Sol`. Then paste the output as MATLAB code using **Ctrl + V** (or right-click and select **Paste**). Pasting the output as MATLAB code automatically expands the abbreviated expression.

```
Sol = [(sqrt(b^3/27 + c^2/4) - c/2)^sym(1/3) - b/(3*(sqrt(b^3/27 + c^2/4) - c/2)^sym(1/3));
      b/(6*(sqrt(b^3/27 + c^2/4) - c/2)^sym(1/3)) - (sqrt(b^3/27 + c^2/4) - c/2)^sym(1/3)/2 - (s
      b/(6*(sqrt(b^3/27 + c^2/4) - c/2)^sym(1/3)) - (sqrt(b^3/27 + c^2/4) - c/2)^sym(1/3)/2 + (s

Sol =
```

$$\begin{pmatrix} \sigma_1 - \frac{b}{3\sigma_1} \\ \frac{b}{6\sigma_1} - \frac{\sigma_1}{2} - \frac{\sqrt{3}\left(\frac{b}{3\sigma_1} + \sigma_1\right)i}{2} \\ \frac{b}{6\sigma_1} - \frac{\sigma_1}{2} + \frac{\sqrt{3}\left(\frac{b}{3\sigma_1} + \sigma_1\right)i}{2} \end{pmatrix}$$

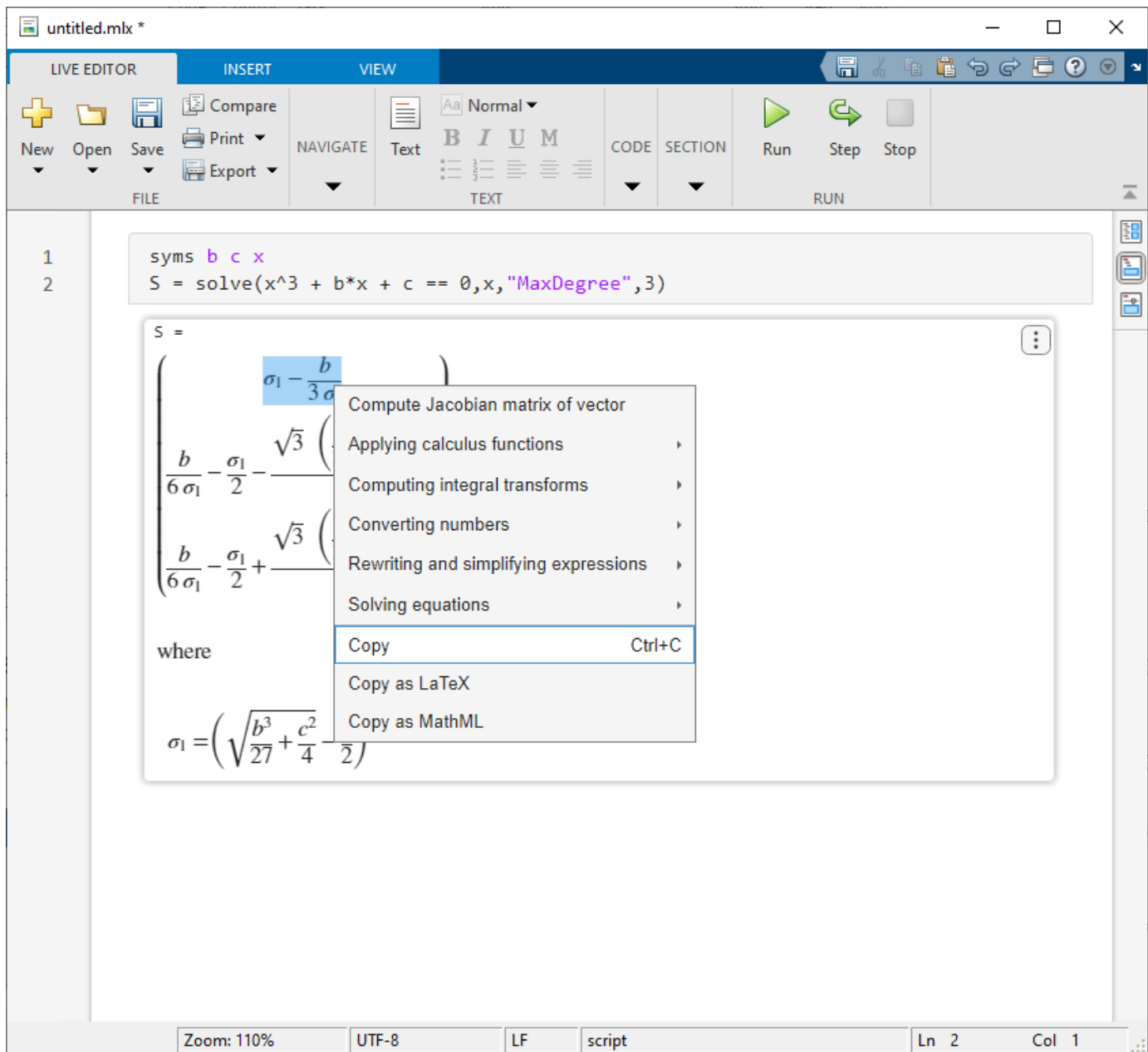
where

$$\sigma_1 = \left( \sqrt{\frac{b^3}{27} + \frac{c^2}{4}} - \frac{c}{2} \right)^{1/3}$$

### Copy Selected Output and Paste as MATLAB Code

Select the first solution of the cubic polynomial. When selecting a subexpression, you can copy and paste only the subexpression that is on the right side of the equal sign. Right-click the existing selection and choose **Copy (Ctrl + C)** in the context menu.





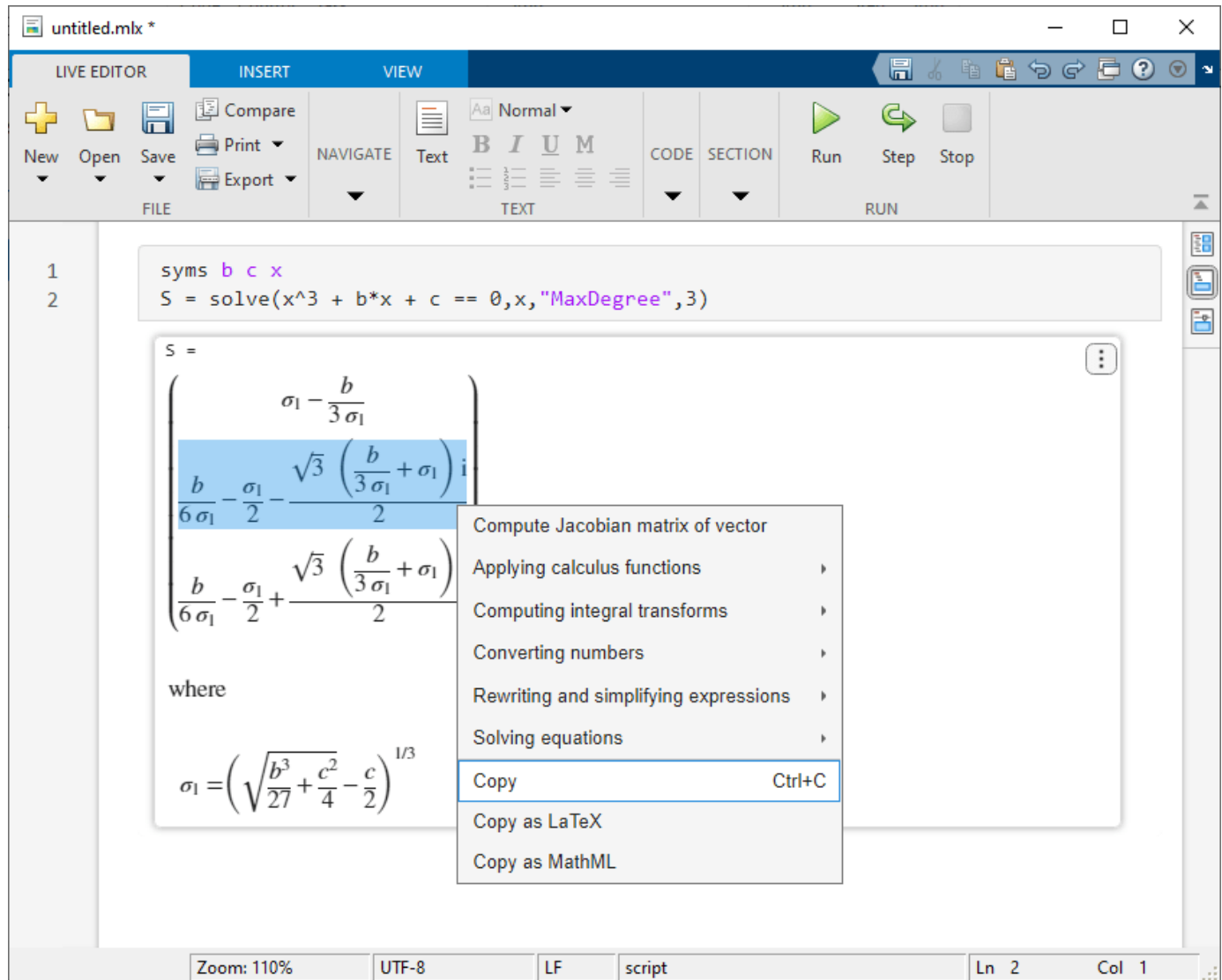
Insert code in the live script and assign the first root of the polynomial to the variable S1. Then paste the output as MATLAB code using **Ctrl + V** (or right-click and select **Paste**). Pasting the output as MATLAB code automatically expands the abbreviated expression.

S1 = (sqrt(b^3/27 + c^2/4) - c/2)^sym(1/3) - b/(3\*(sqrt(b^3/27 + c^2/4) - c/2)^sym(1/3))

$$S1 = \left( \sqrt{\frac{b^3}{27} + \frac{c^2}{4}} - \frac{c}{2} \right)^{1/3} - \frac{b}{3 \left( \sqrt{\frac{b^3}{27} + \frac{c^2}{4}} - \frac{c}{2} \right)^{1/3}}$$

### Copy Selected Output and Paste as Equation Typeset

You can also paste a selection as equation typeset. Select the second solution of the cubic polynomial. Right-click the selection and choose **Copy (Ctrl + C)** in the context menu.



Then paste the selection as equation typeset in the live script using **Ctrl + V** (or right-click and select **Paste**). The equation typeset is rendered as an editable equation. Note that when you paste the output as equation typeset, abbreviated expressions are not expanded.

$$\frac{b}{6\sigma_1} - \frac{\sigma_1}{2} - \frac{\sqrt{3}\left(\frac{b}{3\sigma_1} + \sigma_1\right)i}{2}$$

## Check Symbolic Equations, Inequalities, and Conditional Statements

Symbolic Math Toolbox™ provides several functions to check symbolic equations, inequalities, and conditional statements that involve symbolic objects. This example discusses the use cases of these functions:

- Use `isequal` to check if two symbolic inputs are equal (from a coding perspective).
- Use `logical` to check if symbolic conditions involving relational operators (such as `&`, `|`, `>`, `~`, `=`, and so on) are true.
- Use `isAlways` to check if symbolic conditions are always mathematically true.

For more details, the descriptions of these functions are:

- `isequal(A,B)` checks if `A` and `B` are the same size and their contents are equal (from a coding perspective). `isequal` is useful only to check equality between two expressions without applying mathematical transformations and simplifications. `isequal` returns a scalar logical value `1` (`true`) if `A` and `B` are the same expressions. Otherwise, it returns logical `0` (`false`). Note that `isequal` does not consider NaN (not a number) values as equal. To consider NaN values as equal, you can use `isequaln`.
- `logical(cond)` checks if the symbolic statements in `cond` hold true without applying mathematical transformations and simplifications. It also ignores assumptions on symbolic variables. `logical` returns a logical array with elements `1` (`true`) for the elements in `cond` that are true and `0` (`false`) for the elements in `cond` that are false.
- `isAlways(cond)` checks if the symbolic statements in `cond` are always true for all possible values of the symbolic variables in `cond`. When verifying `cond`, `isAlways` applies mathematical transformations and simplifications. `isAlways` also considers all assumptions on the variables in `cond`. `isAlways` returns a logical array with elements `1` (`true`) for the elements in `cond` that are mathematically true and `0` (`false`) for the elements in `cond` that are not mathematically true. In almost all cases, you can use `isAlways` to check symbolic equalities, inequalities, and conditional statements.

### Check If Expressions Are Equal

`isequal(a,b)` only checks if `a` and `b` have the same contents but does not check if they are mathematically equal. If you use `isequal` to check different expressions, such as  $(x + 1)^2$  and  $x^2 + 2x + 1$ , then it returns `0` (`false`), even though they are mathematically equal.

```
syms x
tf = isequal((x+1)^2,x^2+2*x+1)

tf = logical
    0
```

Expand the expression  $(x + 1)^2$ , and use `isequal` to test if the expanded expression is equal to  $x^2 + 2x + 1$ .

```
expr = expand((x+1)^2)

expr = x^2 + 2*x + 1
```

```
tf = isequal(expr,x^2+2*x+1)
```

```
tf = logical
    1
```

Next, check if the equation  $\tan(x) = \frac{\sin(x)}{\cos(x)}$  is true for all values of  $x$  by using `isAlways`.

```
tf = isAlways(tan(x) == sin(x)/cos(x))
```

```
tf = logical
    1
```

Test if the expressions  $\tan(x)$  and  $\frac{\sin(x)}{\cos(x)}$  are equal. The `isequal` function returns 0 (false) because the expressions are different, even though they are mathematically equal.

```
tf = isequal(tan(x),sin(x)/cos(x))
```

```
tf = logical
    0
```

Rewrite the expression  $\tan(x)$  in terms of  $\sin(x)$  and  $\cos(x)$ . Test if `rewrite` correctly rewrites  $\tan(x)$  as  $\frac{\sin(x)}{\cos(x)}$ .

```
expr = rewrite(tan(x),"sincos")
```

```
expr =
    sin(x)
    cos(x)
```

```
tf = isequal(expr,sin(x)/cos(x))
```

```
tf = logical
    1
```

### Check Equation with and Without Simplifications

To check an equation that requires simplifications, use `isAlways`. For example, check the equality of  $x + 1$  and  $(x^2 + 2x + 1)/(x + 1)$ .

```
syms x
tf = isAlways(x+1 == (x^2+2*x+1)/(x+1))
```

```
tf = logical
    1
```

If you use `logical` to check an equality with different expressions on both sides, then it returns 0 (false).

```
tf = logical(x+1 == (x^2+2*x+1)/(x+1))
```

```
tf = logical
    0
```

Simplify the condition represented by the symbolic equation using `simplify`. The `simplify` function returns the symbolic logical constant `symtrue` because the equation is always true for all values of  $x$ .

```
cond = simplify(x+1 == (x^2+2*x+1)/(x+1))
cond = symtrue
```

Using `logical` on `symtrue` converts it to logical 1 (true).

```
tf = logical(cond)
tf = logical
    1
```

As shown in the previous section, if you use `isequal` to check expressions that are different, then it returns 0 (false).

```
tf = isequal(x+1, (x^2+2*x+1)/(x+1))
tf = logical
    0
```

Simplify the expression  $(x^2 + 2x + 1)/(x + 1)$ . Use `isequal` to check if the simplified expression is equal to  $x + 1$ .

```
expr = simplify((x^2+2*x+1)/(x+1))
expr = x+1
tf = isequal(x+1,expr)
tf = logical
    1
```

### Check Equation with Assumptions

Check if the equation  $\sin(2n\pi) = 0$  holds true for all integers  $n$ . When you create  $n$  as a symbolic variable, Symbolic Math Toolbox treats it as a general complex quantity. To test if the equation holds true for integers, set an assumption on  $n$  and check the equation using `isAlways`.

```
syms n
assume(n, "integer")
tf = isAlways(sin(2*n*pi) == 0)
tf = logical
    1
```

Note that `logical` ignores assumptions on variables. It returns logical 0 (false) in this case.

```
tf = logical(sin(2*n*pi) == 0)
tf = logical
    0
```

### Check Conditions Involving Equations and Inequalities

To check conditions involving equations and inequalities, you can use `logical` or `isAlways`. However, `logical` does not apply mathematical transformations and simplifications when checking the conditions.

For example, test the condition  $1 < 2$  AND  $\exp(\log(x)) = x$ . Note that if a condition uses other functions, such as `exp` and `log`, then these functions are evaluated when defining the condition.

```
syms x
cond1 = 1 < 2 & exp(log(x)) == x

cond1 = x = x
```

Check this condition by using `isAlways`.

```
tf = isAlways(cond1)

tf = logical
    1
```

You can also use `logical` to check a condition that does not require mathematical transformations and simplifications.

```
tf = logical(cond1)

tf = logical
    1
```

Do not use `logical` to check if a condition holds true when mathematical transformations are required. For example, `logical` returns an error when testing the conditional statement  $\sin(x)^2 + \cos(x)^2 = 1$  OR  $x^2 > 0$ . Instead, use `isAlways` to test this conditional statement.

```
cond2 = sin(x)^2 + cos(x)^2 == 1 | x^2 > 0

cond2 = 0 < x^2 ∨ cos(x)^2 + sin(x)^2 = 1

tf = isAlways(cond2)

tf = logical
    1
```

### Check Multiple Conditions

To check multiple conditions, you can represent them as a symbolic array.

For example, create two symbolic arrays where each array has three different expressions.

```
syms x
expr1 = [tan(x); x+1; exp(log(x))]
```

$$\text{expr1} = \begin{pmatrix} \tan(x) \\ x + 1 \\ x \end{pmatrix}$$

```
expr2 = [sin(x)/cos(x); (x^2+2*x+1)/(x+1); x]
```

$$\text{expr2} = \begin{pmatrix} \frac{\sin(x)}{\cos(x)} \\ \frac{x^2 + 2x + 1}{x + 1} \\ x \end{pmatrix}$$

To compare these expressions, create a symbolic array of conditional statements using the relational operator `==`.

```
cond = expr1 == expr2
```

$$\text{cond} = \begin{pmatrix} \tan(x) = \frac{\sin(x)}{\cos(x)} \\ x + 1 = \frac{x^2 + 2x + 1}{x + 1} \\ x = x \end{pmatrix}$$

Check if these multiple conditions are always mathematically true using `isAlways`. `isAlways` returns a 3-by-1 array with logical values 1 (`true`) because each condition is mathematically true.

```
tf = isAlways(cond)
```

```
tf = 3×1 logical array
```

```
1
1
1
```

Check if these conditions hold true using `logical`. `logical` returns a 3-by-1 array, where the first two elements are 0 (`false`) because `logical` does not apply mathematical transformations or simplifications.

```
tf = logical(cond)
```

```
tf = 3×1 logical array
```

```
0
0
1
```

Check if each corresponding element in the two 3-by-1 symbolic arrays, `expr1` and `expr2`, is equal using `isequal`. `isequal` returns a logical scalar 0 (`false`) because some of the corresponding elements are not equal.

```
tf = isequal(expr1,expr2)
```

```
tf = logical
0
```

Next, simplify the second symbolic array using `simplify`.

```
expr2 = simplify(expr2,Steps=10)
```

```
expr2 =  

$$\begin{pmatrix} \tan(x) \\ x+1 \\ x \end{pmatrix}$$

```

Check if each simplified expression in `expr2` is equal to the corresponding expression in `expr1` using `logical`.

```
tf = logical(expr1 == expr2)
```

```
tf = 3×1 logical array
```

```
1  
1  
1
```

Check if all simplified expressions in `expr2` are equal to `expr1` using `isequal`.

```
tf = isequal(expr1,expr2)
```

```
tf = logical  
1
```

## See Also

`isequal` | `logical` | `isAlways` | `isequaln`



## Numeric to Symbolic Conversion

This topic shows how Symbolic Math Toolbox™ converts numbers into symbolic form. For an overview of symbolic and numeric arithmetic, see “Choose Numeric or Symbolic Arithmetic” on page 2-32.

To convert numeric input to symbolic form, use the `sym` command. By default, `sym` returns a rational approximation of a numeric expression.

```
t = 0.1;
sym(t)
```

```
ans =
    1
   10
```

`sym` determines that the double-precision value `0.1` approximates the exact symbolic value  $\frac{1}{10}$ . In general, `sym` tries to correct the round-off error in floating-point inputs to return the exact symbolic form. Specifically, `sym` corrects round-off error in numeric inputs that match the forms  $\frac{p}{q}$ ,  $\frac{p\pi}{q}$ ,  $\left(\frac{p}{q}\right)^{\frac{1}{2}}$ ,  $2^q$ , and  $10^q$ , where  $p$  and  $q$  are modest-sized integers.

For these forms, demonstrate that `sym` converts floating-point inputs to the exact symbolic form. First, numerically approximate  $\frac{1}{7}$ ,  $\pi$ , and  $\frac{1}{\sqrt{2}}$ .

```
N = [1/7 pi 1/sqrt(2)]
```

```
N = 1×3
    0.1429    3.1416    0.7071
```

Convert the numeric approximations to exact symbolic form. `sym` corrects the round-off error.

```
S = sym(N)
```

```
S =
    (1/7 pi sqrt(2))
```

You can force `sym` to accept the input as is by placing the input in quotes. Demonstrate this behavior on the previous input `0.142857142857143`. The `sym` function does not convert the input to  $1/7$ .

```
sym('0.142857142857143')
```

```
ans = 0.142857142857143
```

When you convert large numbers, use quotes to exactly represent them. Demonstrate this behavior by comparing `sym(13333333333333333333)` with `sym('13333333333333333333')`.

```
sym(13333333333333333333)
```

```
ans = 133333333333333333248
```

```
sym('13333333333333333333')
```

```
ans = 13333333333333333333
```

You can specify the technique used by `sym` to convert floating-point numbers using the optional second argument, which can be `'f'`, `'r'`, `'e'`, or `'d'`. The default flag is `'r'`, for rational form.

### Conversion to Rational Symbolic Form

Convert input to exact rational form by calling `sym` with the `'r'` flag. This is the default behavior when you call `sym` without flags.

```
t = 0.1;
sym(t, 'r')
```

```
ans =
    1
   10
```

### Conversion by Using Floating-Point Expansion

If you call `sym` with the flag `'f'`, `sym` converts double-precision, floating-point numbers to their numeric value by using  $N \cdot 2^e$ , where  $N$  and  $e$  are the exponent and mantissa respectively.

Convert `t` by using a floating-point expansion.

```
sym(t, 'f')
```

```
ans =
    3602879701896397
   36028797018963968
```

### Conversion to Rational Symbolic Form with Error Term

If you call `sym` with the flag `'e'`, `sym` returns the rational form of `t` plus the error between the estimated, exact value for `t` and its floating-point representation. This error is expressed in terms of `eps` (the floating-point relative precision).

Convert `t` to symbolic form. Return the error between its estimated symbolic form and its floating-point value.

```
sym(t, 'e')
```

```
ans =
    eps
   40 + 1
   10
```

The error term `eps/40` is the difference between `sym('0.1')` and `sym(0.1)`.

### Conversion to Decimal Form

If you call `sym` with the flag `'d'`, `sym` returns the decimal expansion of the input. The `digits` function specifies the number of significant digits used. The default value of `digits` is 32.

```
sym(t, 'd')
```

```
ans = 0.10000000000000000555111512312578
```

Change the number of significant digits by using `digits`.

```
digitsOld = digits(7);
sym(t, 'd')
```

```
ans = 0.1
```

For further calculations, restore the old value of `digits`.

```
digits(digitsOld)
```

### Conversion to Variable Precision

You can create symbolic numbers with variable-precision floating-point arithmetic by using `vpa`. By default, `vpa` calculates values to 32 significant digits.

```
piVpa = vpa(pi)
```

```
piVpa = 3.1415926535897932384626433832795
```

When you use `vpa` on a numeric input, such as `log(2)`, the numeric expression is first evaluated to the MATLAB® default double-precision number that has less than 32 significant digits. Then, `vpa` is applied on that double-precision number, which can be less accurate. For more accurate results, convert numeric expressions to symbolic expressions with `sym` and then use `vpa` to evaluate the results with variable precision. For example, find `log(2)` with 17 and 20 digits precision.

```
vpaOnDouble = vpa(log(2))
```

```
vpaOnDouble = 0.69314718055994528622676398299518
```

```
vpaOnSym_17 = vpa(log(sym(2)), 17)
```

```
vpaOnSym_17 = 0.69314718055994531
```

```
vpaOnSym_20 = vpa(log(sym(2)), 20)
```

```
vpaOnSym_20 = 0.69314718055994530942
```

When you convert large numbers, use quotes to exactly represent them.

```
inaccurateNum = vpa(123456789012345678)
```

```
inaccurateNum = 123456789012345680.0
```

```
accurateNum = vpa('123456789012345678')
```

```
accurateNum = 123456789012345678.0
```

## See Also

### More About

- “Conversion Between Symbolic and Numeric”

## Choose Numeric or Symbolic Arithmetic

Symbolic Math Toolbox operates on numbers by using either numeric or symbolic arithmetic. In numeric arithmetic, you represent numbers in floating-point format using either double precision or variable precision. In symbolic arithmetic, you represent numbers in their exact form. This topic compares double-precision, variable-precision, and symbolic arithmetic.

### Double-Precision Arithmetic

Numeric computations in MATLAB use double-precision arithmetic by default. For example, evaluate the expressions  $10001/1001$ ,  $\pi$ , and  $\sqrt{2}$ . The results are converted to double-precision values.

```
x = 10001/1001
y = pi
z = sqrt(2)
```

```
x =
    9.9910
```

```
y =
    3.1416
```

```
z =
    1.4142
```

For more information about double-precision arithmetic, see “Floating-Point Numbers”. This arithmetic is recommended when you do not have Symbolic Math Toolbox or are using functions that do not accept symbolic input. Otherwise, exact symbolic arithmetic and variable-precision arithmetic are recommended. To convert a symbolic value to double precision, use the `double` function.

### Variable-Precision Arithmetic

Variable-precision arithmetic using `vpa` is the recommended approach for numeric calculations in Symbolic Math Toolbox. You can specify the number of significant digits when performing calculations with variable-precision arithmetic.

For example, use `vpa` to evaluate the fraction  $10001/1001$ . By default, `vpa` evaluates inputs to 32 significant digits. Approximate the fraction  $10001/1001$  to at least 32 significant digits.

```
vpa(10001/1001)
```

```
ans =
9.991008991008991008991008991009
```

Approximate the fraction to at least 8 significant digits. Change the number of significant digits by using the `digits` function.

```
digits(8);
vpa(10001/1001)
```

```
ans =
9.991009
```

In variable-precision arithmetic, you can increase the number of significant digits on page 2-36 for greater precision. Alternatively, you can decrease the number of significant digits on page 3-386 for faster computations and decreased memory usage.

## Symbolic Arithmetic

Symbolic Math Toolbox provides the `sym` and `syms` functions to perform exact symbolic computations on page 1-29. In symbolic arithmetic, you can perform computations involving numbers and variables in their exact form, such as  $x/2$ ,  $2^{(1/2)}$ , or  $\pi$ . The following three examples show several calculations that are performed in symbolic arithmetic.

### Express Irrational Numbers

Use `sym` to create symbolic numbers. Express the irrational numbers  $\pi$  and  $\sqrt{2}$  in symbolic form.

```
x = sym(pi)
y = sqrt(sym(2))

x =
pi

y =
2^(1/2)
```

### Perform Calculations with Large Integers

When you declare a number, MATLAB automatically converts the number to double precision. For example, declare the integer 80435758145817515 as the input argument of `sym`. The number loses its accuracy since it is bigger than the largest consecutive integer `flintmax` in double precision, which is  $2^{53}$ .

```
Z = 80435758145817515
Zinaccurate = sym(80435758145817515)

Z =
8.0436e+16

Zinaccurate =
80435758145817520
```

To declare a large integer as symbolic number accurately, use a character vector with single quotation marks as the input argument of `sym`.

```
Zaccurate = sym('80435758145817515')

Zaccurate =
80435758145817515
```

You can then perform calculations with large integers using symbolic arithmetic accurately. For example, evaluate the sum of the cubes of three large integers.

```
Z1 = sym('80435758145817515')
Z2 = sym('12602123297335631')
Z3 = sym('-80538738812075974')
Zsum = Z1^3 + Z2^3 + Z3^3

Z1 =
80435758145817515
```

```
Z2 =
12602123297335631

Z3 =
-80538738812075974

Zsum =
42
```

### Solve Mathematical Equations

With symbolic arithmetic, you can solve a mathematical equation. For example, solve the quadratic equation  $ax^2 + bx + c = 0$ . Use `syms` to declare the variable  $x$  and the coefficients  $a$ ,  $b$ , and  $c$  in the quadratic equation.

```
syms a b c x
eqn = a*x^2 + b*x + c == 0;
```

Find the solutions using `solve` and return them as symbolic expressions.

```
sols = solve(eqn,x)

sols =
-(b + (b^2 - 4*a*c)^(1/2))/(2*a)
-(b - (b^2 - 4*a*c)^(1/2))/(2*a)
```

Use `subs` to substitute symbolic values for the coefficients. Set  $a = 1$ ,  $b = 2$ , and  $c = 3$ . Return the solutions of the quadratic equation as symbolic numbers.

```
solsSym = subs(sols,[a b c],[1 2 3])

solsSym =
- (8^(1/2)*1i)/2 - 1
(8^(1/2)*1i)/2 - 1
```

You can then convert the symbolic solutions to floating-point format in double precision or variable precision.

```
digits(32);
solsDouble = double(solsSym)
solsVpa = vpa(solsSym)

solsDouble =
-1.0000 - 1.4142i
-1.0000 + 1.4142i

solsVpa =
- 1.0 - 1.4142135623730950488016887242097i
- 1.0 + 1.4142135623730950488016887242097i
```

### Comparisons of Numeric and Symbolic Arithmetic

The table below compares double-precision, variable-precision, and symbolic arithmetic.

|                                          | Double Precision                                                      | Variable Precision                                                                                                        | Symbolic                                                                                                                  |
|------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Example 1:<br>Evaluate $\sin(\pi)$       | <pre>a = pi sin(pi)  a =     3.1416 ans =     1.2246e-16</pre>        | <pre>b = vpa(pi) sin(b)  b =     3.1415926535897932384626433832795 ans =    -3.21010830131003960695407145883568e-40</pre> | <pre>c = sym(pi) sin(c)  c =     3.1415926535897932384626433832795 ans =    -3.21010830131003960695407145883568e-40</pre> |
| Example 2:<br>Evaluate $1 - 3*(4/3 - 1)$ | <pre>a = 4/3 1 - 3*(a - 1)  a =     1.3333 ans =     2.2204e-16</pre> | <pre>digits(16); b = vpa(4/3) 1 - 3*(b - 1)  b =     1.3333333333333333 ans =     3.308722450212111e-24</pre>             | <pre>c = sym(4)/3 1 - 3*(c - 1)  c =     4/3 ans =     0</pre>                                                            |
| Functions Used                           | double                                                                | vpa<br>digits                                                                                                             | sym                                                                                                                       |
| Data Type                                | double                                                                | sym                                                                                                                       | sym                                                                                                                       |
| Round-Off Errors                         | Yes, the answer has 16 digits of precision.                           | Yes, the number of digits depends on the precision used.                                                                  | No, the results are exact.                                                                                                |
| Speed                                    | Faster                                                                | Faster, depending on the precision used                                                                                   | Slowest                                                                                                                   |
| Memory Usage                             | Least                                                                 | Variable, depending on the precision used                                                                                 | Greatest                                                                                                                  |

## See Also

## More About

- “Numeric to Symbolic Conversion” on page 2-29
- “Increase Precision of Numeric Calculations” on page 2-36
- “Find Almost Integers with High-Precision Arithmetic” on page 2-41

## Increase Precision of Numeric Calculations

By default, MATLAB uses 16 digits of precision. For higher precision, use the `vpa` function in Symbolic Math Toolbox. `vpa` provides variable precision which can be increased without limit.

When you choose variable-precision arithmetic, by default, `vpa` uses 32 significant decimal digits of precision. For details, see “Choose Numeric or Symbolic Arithmetic” on page 2-32. You can set a higher precision by using the `digits` function.

Approximate a sum using the default precision of 32 digits. If at least one input is wrapped with `vpa`, all other inputs are converted to variable precision automatically.

```
vpa(1/3) + 1/2
```

```
ans =  
0.83333333333333333333333333333333
```

You must wrap all inner inputs with `vpa`, such as `exp(vpa(200))`. Otherwise, the inputs are automatically converted to double by MATLAB.

Increase the precision to 50 digits by using `digits` and save the old value of `digits` in `digitsOld`. Repeat the sum.

```
digitsOld = digits(50);  
sum50 = vpa(1/3) + 1/2
```

```
sum50 =  
0.833333333333333333333333333333333333333333333333333
```

Restore the old value of `digits` for further calculations.

```
digits(digitsOld)
```

---

**Note** `vpa` output is symbolic. To use symbolic output with a MATLAB function that does not accept symbolic values, convert symbolic values to double precision by using `double`.

---

Check the current `digits` setting by calling `digits`.

```
digits  
Digits = 32
```

Change the precision for a single `vpa` call by specifying the precision as the second input to `vpa`. This call does not affect `digits`. For example, approximate `pi` with 100 digits.

```
vpa(pi,100)  
  
ans =  
3.14159265358979323846264338327950288419716939937510582097494  
4592307816406286208998628034825342117068  
  
digits    % digits remains 32  
Digits = 32
```

Variable precision can be increased arbitrarily. Find `pi` to 500 digits.



```
digitsOld = digits(500);  
vpa(pi)  
digits(digitsOld)  
  
ans =  
3.1415926535897932384626433832795028841971693993751058209749  
445923078164062862089986280348253421170679821480865132823066  
470938446095505822317253594081284811174502841027019385211055  
596446229489549303819644288109756659334461284756482337867831  
652712019091456485669234603486104543266482133936072602491412  
737245870066063155881748815209209628292540917153643678925903  
600113305305488204665213841469519415116094330572703657595919  
530921861173819326117931051185480744623799627495673518857527  
248912279381830119491
```

`digits` and `vpa` control the number of *significant* decimal digits. For example, approximating  $1/111$  with four-digit accuracy returns six digits after the decimal point because the first two digits are zeros.

```
vpa(1/111,4)
```

```
ans =  
0.009009
```

---

**Note** If you want to increase performance by *decreasing* precision, see “Increase Speed by Reducing Precision” on page 3-386.

---

## Compute Binomial Coefficients Exactly

This example shows how to get precise values for binomial coefficients and find probabilities in coin-tossing experiments using the Symbolic Math Toolbox™.



Define the symbolic function,  $P(n, k)$ , that computes the probability for the heads to come up exactly  $k$  times out of  $n$  tosses.

```
syms P(n,k)
P(n,k) = nchoosek(n,k)/2^n
```

$$P(n, k) = \frac{\binom{n}{k}}{2^n}$$

Suppose you toss a coin 2000 times. The probability that heads comes up in half of the tosses is  $P(n, n/2)$ , where  $n = 2000$ . The result is a rational expression with large numbers in both the numerator and denominator. Symbolic Math Toolbox returns the exact result.

```
n = 2000;
central = P(n, n/2)
```

```
central =
    320023691717107767864869141090753913186941388747093286534434787136210654094075586848270780341032
    1793954211366022694113801876840128100034871409513586250746316776290259783425578615401030447369541
```

Approximate this rational number with 10-digit accuracy using `digits` and `vpa`.

```
previous_digits = digits(10);
vpa(central)
```

```
ans = 0.01783901115
```

Compute the probability that the number of "heads" differs from the expected value by no more than two standard deviations.

```
sigma = sqrt(n/4);
withinTwoSigma = symsum(P(n, k), k, ceil(n/2 - 2*sigma), floor(n/2 + 2*sigma))
```

```
withinTwoSigma =
1368352466395056520289440683203474916723919590470093450966749985815252789703206921185078166194364
1435163369092818155291041501472102480027897127610869000597053421032207826740462892320824357895632
```

Approximate the result with a floating-point number.

```
vpa(withinTwoSigma)
```

```
ans = 0.9534471795
```

Compare this result with the following two estimates derived from the cumulative distribution function (cdf) of the normal distribution. It turns out that taking the midpoint between the first integer outside and the first integer inside the two-sigma interval gives a more precise result than using the two-sigma interval itself.

```
syms cdf(x)
cdf(x) = 1/2 * (1 + erf((x - n/2)/sqrt(sym(n/2))))
```

```
cdf(x) =

$$\frac{\operatorname{erf}\left(\frac{\sqrt{10}(x - 1000)}{100}\right)}{2} + \frac{1}{2}$$

```

```
estimate1 = vpa(cdf(n/2 + 2*sigma) - cdf(n/2 - 2*sigma))
```

```
estimate1 = 0.9544997361
```

```
estimate2 = vpa(cdf(floor(n/2 + 2*sigma) + 1/2) - ...
cdf(ceil(n/2 - 2*sigma) - 1/2))
```

```
estimate2 = 0.9534201342
```

```
error1 = vpa(estimate1 - withinTwoSigma)
```

```
error1 = 0.001052556595
```

```
error2 = vpa(estimate2 - withinTwoSigma)
```

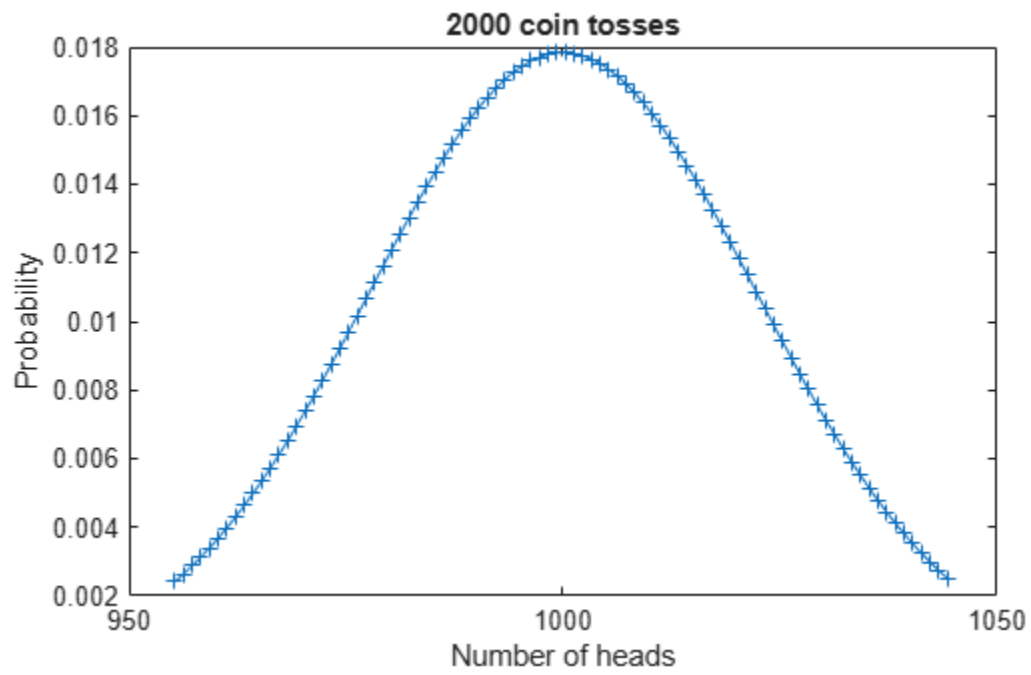
```
error2 = -0.00002704525904
```

Restore the default precision for floating-point computations.

```
digits(previous_digits);
```

Plot this distribution for  $k$  within the  $2\sigma$ -interval. The plot is a Gaussian curve.

```
k = n/2 + (-2*sigma:2*sigma);
plot(k, P(n,k), '-+');
title('2000 coin tosses');
xlabel('Number of heads');
ylabel('Probability');
```



## Find Almost Integers with High-Precision Arithmetic

Algorithms for finding integer relations [1], such as the PSLQ algorithm, require high-precision arithmetic to produce accurate results. The higher the precision of the inputs to the algorithm, the greater the level of confidence that the algorithm can find an integer relation that is not just a numerical artifact. The PSLQ algorithm can encounter false positives, such as those caused by almost integers.

This example shows how to find almost integers, or numbers that are very close to integers, using variable-precision arithmetic in Symbolic Math Toolbox™. This example searches for almost integers (or near-integers) that have the form  $\exp(\pi\sqrt{n})$  or  $\exp(\pi n)$  for the integers  $n = 1, \dots, 200$ .

First, consider a well-known example of an almost integer [2] that is the real number  $\exp(\pi\sqrt{163})$ . Create this real number as an exact symbolic number.

```
r = exp(pi*sqrt(sym(163)))
```

```
r = eπ√163
```

Evaluate this number with variable-precision arithmetic using `vpa`. By default, `vpa` calculates values to 32 significant digits.

```
vpa(r)
```

```
ans = 262537412640768743.99999999999925
```

You can change the number of significant digits to a higher precision by using `digits`. Evaluate the same number to 40 significant digits.

```
digits(40)
```

```
vpa(r)
```

```
ans = 262537412640768743.9999999999992500725972
```

This number is very close to an integer. Find the difference between this real number and its nearest integer. Use `vpa` to evaluate this result to 40 significant digits.

```
dr = vpa(round(r) - r)
```

```
dr = 0.00000000000007499274028018143151532171817442410122968
```

Next, search for almost integers that have the form  $\exp(\pi\sqrt{n})$  for the integers  $n = 1, \dots, 200$ . Create these numbers as exact symbolic numbers.

```
A = exp(pi*sqrt(sym(1:200)));
```

Set the number of significant digits to the number of digits in the integer part of  $\exp(\pi\sqrt{200})$  plus 20 decimal points.

```
d = log10(A(end));  
digits(ceil(d) + 20)
```

Evaluate the differences between this series of numbers and their nearest integers. Find the almost integers with rounding errors that are less than 0.0001. Show the results in exact symbolic form.

```
B = vpa(round(A)-A);
A_nearint = A(abs(B)<0.0001)'
```

```
A_nearint =
```

$$\begin{pmatrix} e^{\pi\sqrt{37}} \\ e^{\pi\sqrt{58}} \\ e^{\pi\sqrt{67}} \\ e^{\pi\sqrt{163}} \end{pmatrix}$$

Evaluate the almost integers with a precision of at least 20 decimal points.

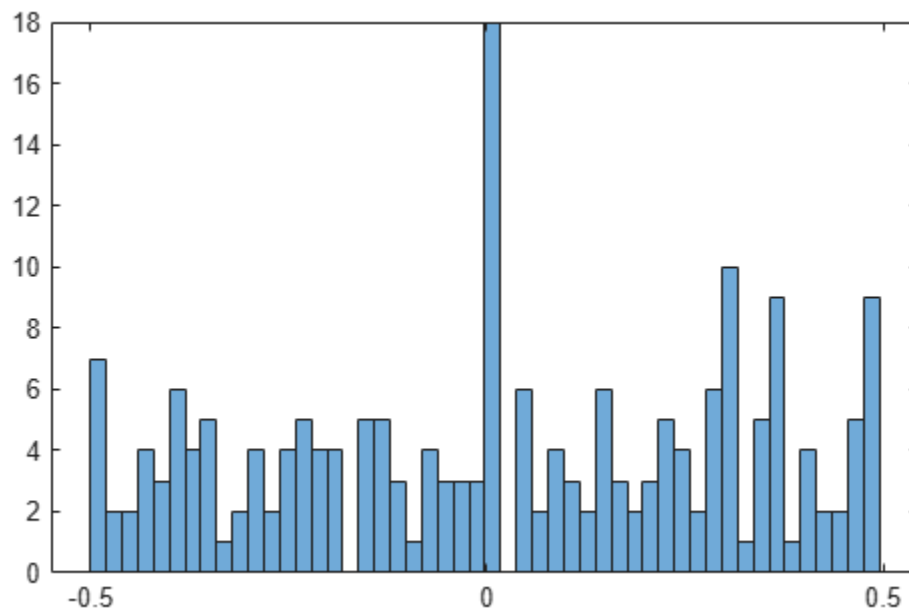
```
A_nearint = vpa(A_nearint)
```

```
A_nearint =
```

$$\begin{pmatrix} 199148647.9999780465518567665009238753359 \\ 24591257751.99999982221324146957619235527 \\ 147197952743.9999986624542245068292613126 \\ 262537412640768743.999999999992500725972 \end{pmatrix}$$

Plot the histogram of the differences to show their distribution. The distribution has many occurrences of differences that are close to zero, where the form  $\exp(\pi\sqrt{n})$  is an almost integer.

```
histogram(double(B),50)
```



Next, search for almost integers that have the form  $\exp(\pi n)$  for the integers  $n = 1, \dots, 200$ . Create these numbers as exact symbolic numbers.

```
A = exp(sym(pi)*(1:200));
```

Set the number of significant digits to the number of digits in the integer part of  $\exp(\pi \cdot 200)$  plus 20 decimal points.

```
d = log10(A(end));
digits(ceil(d) + 20)
```

Evaluate the differences between this series of numbers and their nearest integers. Find the almost integers with rounding errors that are less than 0.0001. The result is an empty sym array, which means there is no almost integer that satisfies this condition.

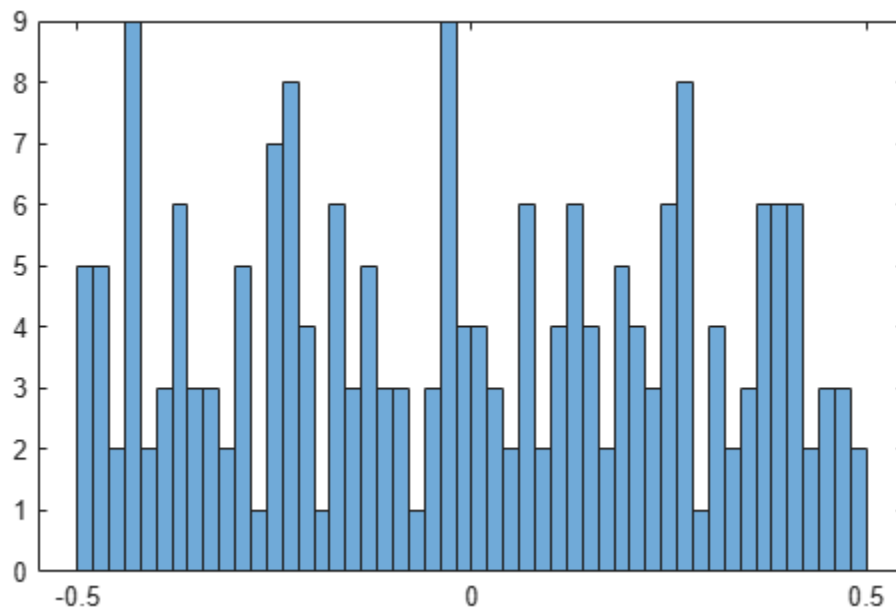
```
B = vpa(round(A)-A);
A_nearint = A(abs(B)<0.0001)
```

```
A_nearint =
```

```
Empty sym: 1-by-0
```

Plot the histogram of the differences. The histogram is relatively evenly distributed and shows that the form  $\exp(\pi n)$  does not have many occurrences of almost integers. For this specific example, there is no almost integer with a rounding error that is less than 0.0001.

```
histogram(double(B),50)
```



Finally, restore the default precision of 32 significant digits for further calculations.

```
digits(32)
```

## References

[1] "Integer Relation Algorithm." In Wikipedia, April 9, 2022. [https://en.wikipedia.org/w/index.php?title=Integer\\_relation\\_algorithm&oldid=1081697113](https://en.wikipedia.org/w/index.php?title=Integer_relation_algorithm&oldid=1081697113).

[2] “Almost Integer.” In Wikipedia, December 4, 2021. [https://en.wikipedia.org/w/index.php?title=Almost\\_integer&oldid=1058543590](https://en.wikipedia.org/w/index.php?title=Almost_integer&oldid=1058543590).

### See Also

`sym` | `vpa` | `digits`

### More About

- “Choose Numeric or Symbolic Arithmetic” on page 2-32
- “Numeric to Symbolic Conversion” on page 2-29
- “Increase Precision of Numeric Calculations” on page 2-36



## Decimal Digits of PI

This example shows how to use variable-precision arithmetic to investigate the decimal digits of  $\pi$  using Symbolic Math Toolbox™.

```
 $\pi \approx 3$ 
.141592653589793238462643383279502884197169399375105820974944592307816406286208998628034
...
.
```

**Joke:** What do you get when you take the sun and divide its circumference by its diameter?

**Answer:**  $\pi$  in the sky!

It is an old game to search for one's birthday or telephone number in the decimal digits of  $\pi$ . The precision of the built-in datatypes suffices to obtain a few digits only:

```
num2str(pi, 100000)

ans =
'3.141592653589793115997963468544185161590576171875'
```

The function `vpa` uses variable-precision to convert symbolic expressions into symbolic floating-point numbers. Convert `pi` to a floating-point number using `vpa`. Increase the precision of `vpa` using `digits`.

```
digits(5000);
a = vpa(pi)

a = 3.141592653589793238462643383279502884197169399375105820974944592307816406286208998628034825342117
```

To convert a variable-precision number into a string, use the function `char`.

```
c = char(a);
```

Search for your phone number in the digits of `pi`.

```
strfind(c, '1185480')

ans =
447
```

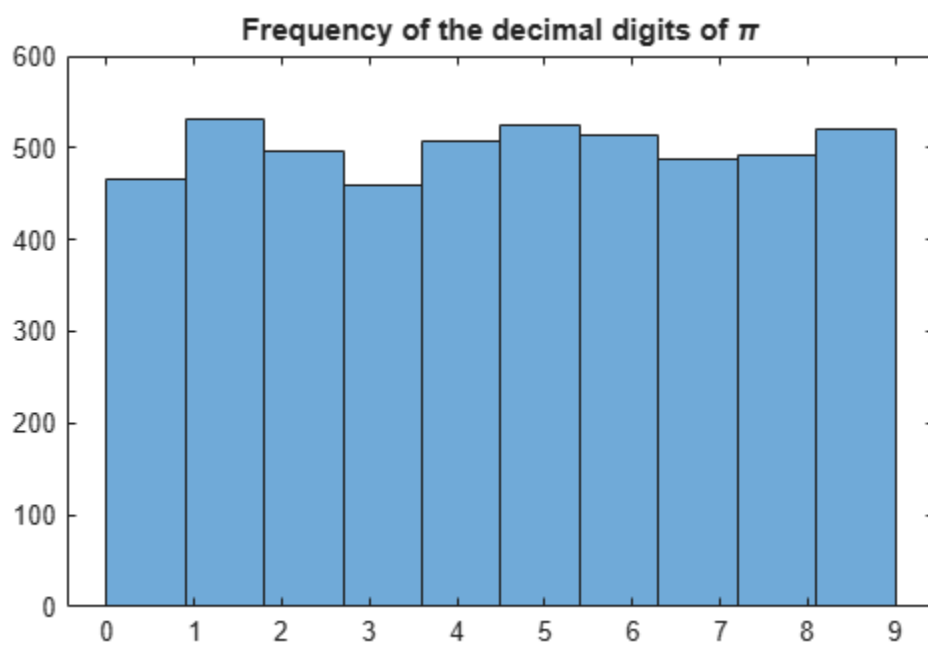
It is common belief that all digits occur asymptotically equally often in the decimal expansion of  $\pi$ , but no proof exists yet. Find the decimal point:

```
pos = strfind(c, '.')

pos =
2
```

Convert the decimal digits to numbers, and plot a histogram of their frequency:

```
d = arrayfun(@str2num, c(pos+1:end));
histogram(d, 10);
title('Frequency of the decimal digits of \pi');
```



# Units of Measurement Tutorial

Use units of measurement with Symbolic Math Toolbox™. This page shows how to define units, use units in equations (including differential equations), and verify the dimensions of expressions.

## Define and Convert Units

Load units by using `symunit`.

```
u = symunit;
```

Specify a unit by using `u.unit`. For example, specify a distance of 5 meters, a weight of 50 kilograms, and a speed of 10 kilometers per hour.

```
d = 5*u.m
```

```
d = 5 m
```

```
w = 50*u.kg
```

```
w = 50 kg
```

```
s = 10*u.km/u.hr
```

```
s =  
10  $\frac{\text{km}}{\text{h}}$ 
```

You can use tab expansion to find names of units. Type `u.`, press **Tab**, and continue typing.

Units are treated like other symbolic expressions and can be used in any standard operation or function. Units are not automatically simplified, which provides flexibility. Common alternate names for units are supported. Plurals are not supported.

Add 500 meters and 2 kilometers. The resulting distance is not automatically simplified.

```
d = 500*u.m + 2*u.km
```

```
d = 2 km + 500 m
```

Simplify `d` by using `simplify`. The `simplify` function chooses the unit to return.

```
d = simplify(d)
```

```
d = 2500 m
```

Convert `d` to a specific unit by using `unitConvert`. Convert `d` to kilometers.

```
d = unitConvert(d,u.km)
```

```
d =  
 $\frac{5}{2}$  km
```

For more unit conversion and unit system options, see “Unit Conversions and Unit Systems” on page 2-53.

Find the speed if the distance `d` is traveled in 50 seconds. The result has the correct units.

```
t = 50*u.s;
s = d/t
```

```
s =
    1 km
   20 s
```

### Use Temperature Units in Absolute or Difference Forms

By default, temperatures are assumed to represent differences and not absolute measurements. For example, `5*u.Celsius` is assumed to represent a temperature difference of 5 degrees Celsius. This assumption allows arithmetical operations on temperature values.

To represent absolute temperatures, use kelvin, so that you do not have to distinguish an absolute temperature from a temperature difference.

Convert 23 degrees Celsius to kelvin, treating the temperature first as a temperature difference and then as an absolute temperature.

```
u = symunit;
T = 23*u.Celsius;
diffK = unitConvert(T,u.K)
```

```
diffK = 23 K
```

```
absK = unitConvert(T,u.K,'Temperature','absolute')
```

```
absK =
    5923
   20 K
```

Because the value 0 times a symbolic unit is returned as a dimensionless 0 when using `symunit`, you can use a cell array to represent 0 degrees.

For example, convert 0 degrees Celsius to degrees Fahrenheit.

```
TC = {0,u.Celsius};
TF = unitConvert(TC,u.Fahrenheit,'Temperature','Absolute')
```

```
TF = 32 °F
```

### Verify Dimensions

In longer expressions, visually checking for units is difficult. You can check the dimensions of expressions automatically by verifying the dimensions of an equation.

First, define the kinematic equation  $v^2 = v_0^2 + 2as$ , where  $v$  represents velocity,  $a$  represents acceleration, and  $s$  represents distance. Assume  $s$  is in kilometers and all other units are in SI base units. To demonstrate dimension checking, the units of  $a$  are intentionally set incorrectly.

```
syms v v0 a s
u = symunit;
eqn = (v*u.m/u.s)^2 == (v0*u.m/u.s)^2 + 2*a*u.m/u.s*s*u.km
```

```
eqn =
    v^2 m^2
      s^2 = v0^2 m^2
            s^2 + 2 a s km m
                  s
```

Observe the units that appear in `eqn` by using `findUnits`. The returned units show that both kilometers and meters are used to represent distance.

```
findUnits(eqn)
```

```
ans = (km m s)
```

Check if the units have the same dimensions (such as length or time) by using `checkUnits` with the 'Compatible' input. MATLAB® assumes symbolic variables are dimensionless. `checkUnits` returns logical 0 (false), meaning the units are incompatible and not of the same physical dimensions.

```
checkUnits(eqn, 'Compatible')
```

```
ans = logical
      0
```

Looking at `eqn`, the acceleration `a` has incorrect units. Correct the units and recheck for compatibility again. `eqn` now has compatible units.

```
eqn = (v*u.m/u.s)^2 == (v0*u.m/u.s)^2 + 2*a*u.m/u.s^2*s*u.km;
checkUnits(eqn, 'Compatible')
```

```
ans = logical
      1
```

Now, to check that each dimension is consistently represented by the same unit, use `checkUnits` with the 'Consistent' input. `checkUnits` returns logical 0 (false) because meters and kilometers are both used to represent distance in `eqn`.

```
checkUnits(eqn, 'Consistent')
```

```
ans = logical
      0
```

Convert `eqn` to SI base units to make the units consistent. Run `checkUnits` again. `eqn` has both compatible and consistent units.

```
eqn = unitConvert(eqn, 'SI')
```

```
eqn =
      v^2  $\frac{\text{m}^2}{\text{s}^2}$  = v0^2  $\frac{\text{m}^2}{\text{s}^2}$  + 2000 a s  $\frac{\text{m}^2}{\text{s}^2}$ 
```

```
checkUnits(eqn)
```

```
ans = struct with fields:
    Consistent: 1
    Compatible: 1
```

After you finish working with units and only need the dimensionless equation or expression, separate the units and the equation by using `separateUnits`.

```
eqn = separateUnits(eqn)
```

$$\text{eqn} = v^2 = v_0^2 + 2000 \, a \, s$$

To calculate numeric values from your expression, substitute for symbolic variables using `subs`, and convert to numeric values using `double` or `vpa`.

Solve `eqn` for `v`. Then find the value of `v` where  $v_0 = 5$ ,  $a = 2.5$ , and  $s = 10$ . Convert the result to `double`.

```
v = solve(eqn,v);
v = v(2);           % choose the positive solution
vSol = subs(v,[v0 a s],[5 2.5 10]);
vSol = double(vSol)

vSol =
223.6627
```

### Use Units in Differential Equations

Use units in differential equations just as in standard equations. This section shows how to use units in differential equations by deriving the velocity relations  $v = v_0 + at$  and  $v^2 = v_0^2 + 2as$  starting from the definition of acceleration  $a = \frac{dv}{dt}$ .

Represent the definition of acceleration symbolically using SI units. Given that the velocity `V` has units, you must differentiate `V` with respect to the correct units as `T = t*u.s` and not just `t`.

```
syms V(t) a
u = symunit;
T = t*u.s;           % time in seconds
A = a*u.m/u.s^2;     % acceleration in meters per second
eqn1 = A == diff(V,T)
```

$$\text{eqn1}(t) = a \frac{\text{m}}{\text{s}^2} = \frac{\partial}{\partial t} V(t) \frac{1}{\text{s}}$$

Because the velocity `V` is unknown and does not have units, `eqn1` has incompatible and inconsistent units.

```
checkUnits(eqn1)

ans = struct with fields:
    Consistent: 0
    Compatible: 0
```

Solve `eqn1` for `V` with the condition that the initial velocity is  $v_0$ . The result is the equation  $v(t) = v_0 + at$ .

```
syms v_0
cond = V(0) == v_0*u.m/u.s;
eqn2 = V == dsolve(eqn1,cond)
```

$$\text{eqn2}(t) = V(t) = (v_0 \, \text{m} + a \, t \, \text{m}) \frac{1}{\text{s}}$$

Check that the result has the correct dimensions by substituting `rhs(eqn2)` into `eqn1` and using `checkUnits`.

```
checkUnits(subs(eqn1,V,rhs(eqn2)))
```

```
ans = struct with fields:
    Consistent: 1
    Compatible: 1
```

Now, derive  $v^2 = v_0^2 + 2as$ . Because velocity is the rate of change of distance, substitute `V` with the derivative of distance `S`. Again, given that `S` has units, you must differentiate `S` with respect to the correct units as `T = t*u.s` and not just `t`.

```
syms S(t)
eqn2 = subs(eqn2,V,diff(S,T))
```

```
eqn2(t) =
    
$$\frac{\partial}{\partial t} S(t) \frac{1}{s} = (v_0 m + a t m) \frac{1}{s}$$

```

Solve `eqn2` with the condition that the initial distance covered is 0. Get the expected form of `S` by using `expand`.

```
cond2 = S(0) == 0;
eqn3 = S == dsolve(eqn2,cond2);
eqn3 = expand(eqn3)
```

```
eqn3(t) =
    
$$S(t) = \frac{a t^2}{2} m + v_0 t m$$

```

You can use this equation with the units in symbolic workflows. Alternatively, you can remove the units by returning the right side using `rhs`, separating units by using `separateUnits`, and using the resulting unitless expression.

```
[S units] = separateUnits(rhs(eqn3))
```

```
S(t) =
    
$$\frac{t(2 v_0 + a t)}{2}$$

```

```
units(t) = m
```

When you need to calculate numeric values from your expression, substitute for symbolic variables using `subs`, and convert to numeric values using `double` or `vpa`.

Find the distance traveled in 8 seconds where  $v_0 = 20$  and  $a = 1.3$ . Convert the result to double.

```
S = subs(S,[v_0 a],[20 1.3]);
dist = S(8);
dist = double(dist)
```

```
dist =  
201.6000
```

### See Also

[checkUnits](#) | [findUnits](#) | [isUnit](#) | [newUnit](#) | [separateUnits](#) | [symunit2str](#) | [unitConversionFactor](#) | [unitConvert](#)

### More About

- “Unit Conversions and Unit Systems” on page 2-53
- “Units and Unit Systems List” on page 2-60

### External Websites

- The International System of Units (SI)



## Unit Conversions and Unit Systems

Convert between units with Symbolic Math Toolbox. This page shows conversions between units and between systems of units, such as SI, CGS, or a user-defined unit system.

### Convert Units

Convert between units by using `unitConvert`.

Convert 1.2 meters to centimeters.

```
u = symunit;
len = 1.2*u.m;
len = unitConvert(len,u.cm)
```

```
len =
120*[cm]
```

Convert `len` to inches. The result is in exact symbolic form. Separate units and convert to double.

```
len = unitConvert(len,u.in)

len =
(6000/127)*[in]

[len units] = separateUnits(len);
len = double(len)

len =
47.2441
```

Calculate the force needed to accelerate a mass of 5 kg at 2 m/s<sup>2</sup>.

```
m = 5*u.kg;
a = 2*u.m/u.s^2;
F = m*a

F =
10*([kg]*[m])/[s]^2)
```

Convert the result to newton.

```
F = unitConvert(F,u.N)

F =
10*[N]
```

---

**Tip** Use tab expansion to find names of units. Type `u.`, press **Tab**, and continue typing.

---

Calculate the energy when force `F` is applied for 3 meters. Convert the result to joule.

```
d = 3*u.m;
E = F*d

E =
30*[N]*[m]
```

```
E = unitConvert(E,u.J)
```

```
E =  
30*[J]
```

Convert E to kilowatt-hour.

```
E = unitConvert(E,u.kWh)
```

```
E =  
(1/120000)*[kWh]
```

## Temperature Unit Conversion

Temperatures can represent either absolute temperatures or temperature differences. By default, temperatures are assumed to be differences. Convert temperatures assuming temperatures are absolute by specifying the 'Temperature' input as 'absolute'.

Convert 23 degrees Celsius to degrees Kelvin, first as a temperature difference and then as an absolute temperature.

```
u = symunit;  
T = 23*u.Celsius;  
relK = unitConvert(T,u.K,'Temperature','difference')
```

```
relK =  
23*[K]
```

```
absK = unitConvert(T,u.K,'Temperature','absolute')
```

```
absK =  
(5923/20)*[K]
```

Because the value 0 is dimensionless and 0 degrees cannot be represented, convert 0 degrees between temperature units by using cell input.

Convert 0 degrees Celsius to degrees Fahrenheit.

```
tC = {0,u.Celsius};  
tF = unitConvert(tC,u.Fahrenheit,'Temperature','Absolute')
```

```
tF =  
32*[Fahrenheit]
```

## Convert to SI, CGS, or US Unit Systems

Automatically convert to the correct units by converting to a unit system. Further, converting to the *derived* units of a unit system attempts to select convenient units. Available unit systems include SI, CGS, and US. For all unit systems, see “Unit Systems List” on page 2-71. In addition, you can define custom unit systems.

Calculate the force due to a 5 kg mass accelerating at 2 m/s<sup>2</sup>. The resulting units are hard to read. Convert them to convenient units by specifying the SI and Derived options. `unitConvert` automatically chooses the correct units of newton.

```
u = symunit;  
m = 5*u.kg;
```

```
a = 2*u.m/u.s^2;
F = m*a
```

```
F =
10*([kg]*[m])/[s]^2)
```

```
F = unitConvert(F,'SI','Derived')
```

```
F =
10*[N]
```

Convert F to US units. By default, the converted units are base units. For convenience, also convert into derived units by specifying the `Derived` option. The derived units are easier to read.

```
F = unitConvert(F,'US')
```

```
F =
(1250000000000/17281869297)*([ft]*[lbm])/[s]^2)
```

```
F = unitConvert(F,'US','Derived')
```

```
F =
(20000000000000/8896443230521)*[lbf]
```

Convert F to CGS derived units.

```
F = unitConvert(F,'CGS','Derived')
```

```
F =
1000000*[dyn]
```

Convert a specification in SI to US derived units. Specify the temperatures as absolute.

```
loadCell = [ 3*u.kg;      % capacity
             50*u.mm;     % length
             15*u.mm;     % width
             10*u.mm;     % height
             -10*u.Celsius; % minimum temperature
             40*u.Celsius; % maximum temperature
            ];
loadCell = unitConvert(loadCell,'US','derived','Temperature','absolute')

loadCell =
(3000000000/45359237)*[lbm]
(125/762)*[ft]
(25/508)*[ft]
(25/762)*[ft]
14*[Fahrenheit]
104*[Fahrenheit]
```

If `unitConvert` does not choose your preferred unit, then adjust the result with further `unitConvert` commands. Here, inches are more convenient than feet. Convert the result to inches.

```
loadCell = unitConvert(loadCell,u.inch)
```

```
loadCell =
(3000000000/45359237)*[lbm]
(250/127)*[in]
(75/127)*[in]
(50/127)*[in]
```

```
14*[Fahrenheit]
104*[Fahrenheit]
```

The exact symbolic values are hard to read. Separate the units and convert to double.

```
[loadCellDouble loadCellUnits] = separateUnits(loadCell);
loadCellDouble = double(loadCellDouble)
```

```
loadCellDouble =
    6.6139
    1.9685
    0.5906
    0.3937
   14.0000
  104.0000
```

Alternatively, approximate the result to high precision by using `vpa`. The `vpa` function also keeps the symbolic units because it returns symbolic output.

```
loadCell = vpa(loadCell)

loadCell =
 6.6138678655463274216892140403508*[lbm]
 1.968503937007874015748031496063*[in]
 0.5905511811023622047244094488189*[in]
 0.3937007874015748031496062992126*[in]
          14.0*[Fahrenheit]
          104.0*[Fahrenheit]
```

Convert five acres (ac), whose unit is a US survey acre, to metric area.

```
u = symunit;
area = 5*u.ac_US;
area = unitConvert(area, 'SI')

area =

(313632000000/15499969)*[m]^2
```

## Define Custom Unit System from Existing System

Custom unit systems provide flexibility in converting units. You can easily define a custom unit system by modifying a default unit system. Alternatively, you can define the system directly. For definitions of unit system, base units, and derived units, see “Unit System Definition” on page 2-58.

In photonics, commonly used units are nanosecond (ns), electron volt (eV), and nanometer (nm). Define a unit system with these units by modifying the SI unit system. Get SI base and derived units by using `baseUnits` and `derivedUnits`. Modify the units by using `subs`.

```
u = symunit;
bunits = baseUnits('SI');
bunits = subs(bunits,[u.m u.s],[u.nm u.ns])

bunits =
[ [kg], [ns], [nm], [A], [cd], [mol], [K]]

dunits = derivedUnits('SI');
dunits = subs(dunits,u.J,u.eV)
```

```
dunits =
[ [F], [C], [S], [H], [V], [eV], [N], [lx], [lm], [Wb], [W], [Pa],...
  [Ohm], [T], [Gy], [Bq], [Sv], [Hz], [kat], [rad], [sr], [Celsius]]
```

---

**Note** Do not define variables called `baseUnits` and `derivedUnits` because the variables prevent access to the `baseUnits` and `derivedUnits` functions.

---

Define the new unit system by using `newUnitSystem`.

```
phSys = newUnitSystem('photonics',bunits,dunits)
```

```
phSys =
  "photonics"
```

Calculate the energy of a photon of frequency 1 GHz and convert the result to derived units of the `phSys` system. The result is in electron volts.

```
f = 1*u.GHz;
E = u.h_c*f;
E = unitConvert(E,phSys,'Derived')
```

```
E =
(44173801/10681177560000)*[eV]
```

The exact symbolic result is hard to read. Separate the units and convert to double.

```
[E Eunits] = separateUnits(E);
E = double(E)
```

```
E =
  4.1357e-06
```

After completing calculations, remove the unit system.

```
removeUnitSystem(phSys)
```

## Define Custom Unit System Directly

Define a custom unit system for atomic units (au).

Define these base units:

| Dimension         | Unit                  | Implementation                         |
|-------------------|-----------------------|----------------------------------------|
| Mass              | Electron rest mass    | <code>u.m_e</code>                     |
| Elementary charge | Electron charge       | <code>u.e</code>                       |
| Length            | Bohr radius ( $a_0$ ) | <code>u.Bohr</code>                    |
| Time              | $\hbar/E_h$           | Define by using <code>newUnit</code> . |

```
u = symunit;
t_au = newUnit('t_au',u.hbar/u.E_h);
bunits = [u.m_e u.e u.Bohr u.t_au]
```

```
bunits =
[ [m_e], [e], [a_0], [t_au]]
```

Define these derived units:

| Dimension              | Unit                          | Implementation                         |
|------------------------|-------------------------------|----------------------------------------|
| Angular momentum       | Reduced Planck's constant     | <code>u.hbar</code>                    |
| Energy                 | Hartree                       | <code>u.E_h</code>                     |
| Electric dipole moment | $ea_0$                        | Define by using <code>newUnit</code> . |
| Magnetic dipole moment | Bohr magneton = $e\hbar/2m_e$ | Define by using <code>newUnit</code> . |
| Electric potential     | $E_h/e$                       | Define by using <code>newUnit</code> . |

```
edm_au = newUnit('edm_au',u.e*u.bohr);
mdm_au = newUnit('mdm_au', u.e*u.hbar/(2*u.me));
ep_au = newUnit('ep_au', u.E_h/u.e);
dunits = [u.hbar u.E_h u.edm_au u.mdm_au u.ep_au]
```

```
dunits =
[ [h_bar], [E_h], [edm_au], [mdm_au], [ep_au]]
```

Define the unit system.

```
auSys = newUnitSystem('atomicUnits',bunits,dunits)

auSys =
    "atomicUnits"
```

Convert the properties of a proton to atomic units.

```
proton = [ 1.672621923e-27*u.kg;      % mass
           1.6021766208e-19*u.C;      % charge
           5.4e-24*u.e*u.cm;         % electric dipole moment
           1.4106067873e-26*u.J/u.T;  % magnetic dipole moment
           ];
proton = unitConvert(proton,auSys,'Derived')

proton =
           1836.1526726825404620381265471117*[m_e]
           0.99999999176120807953267071600981*[e]
    0.000000000000000010204521072979158730257341288851*[edm_au]
           0.00048415958374162452452052339364507*pi*[mdm_au]
```

After completing calculations, remove the unit system and the added units.

```
removeUnitSystem(auSys)
removeUnit([u.t_au u.edm_au u.mdm_au u.ep_au])
```

## Unit System Definition

A unit system is a collection of base units and derived units that follows these rules:

- Base units must be independent in terms of the dimensions mass, time, length, electric current, luminous intensity, amount of substance, and temperature. Therefore, a unit system has up to 7 base units. As long as the independence is satisfied, any unit can be a base unit, including units such as newton or watt.
- A unit system can have less than 7 base units. For example, mechanical systems need base units only for the dimensions length, mass, and time.

- Derived units in a unit system must have a representation in terms of the products of powers of the base units for that system. Unlike base units, derived units do not have to be independent.
- Derived units are optional and added for convenience of representation. For example,  $\text{kg m/s}^2$  is abbreviated by newton.
- An example of a unit system is the SI unit system, which has 7 base units: kilogram, second, meter, ampere, candela, mol, and kelvin. There are 22 derived units found by calling `derivedUnits('SI')`.

## See Also

`baseUnits` | `derivedUnits` | `newUnitSystem` | `removeUnit` | `removeUnitSystem` | `symunit` | `unitConvert`

## More About

- “Units of Measurement Tutorial” on page 2-47
- “Units and Unit Systems List” on page 2-60

## External Websites

- The International System of Units (SI)

## Units and Unit Systems List

List of units, SI unit prefixes, and unit systems in Symbolic Math Toolbox. For details, see “Units of Measurement Tutorial” on page 2-47. Common alternate names for units are supported and map to the names listed here. Plurals are not supported.

| In this section...                            |
|-----------------------------------------------|
| “Units List” on page 2-60                     |
| “SI Unit Prefixes List” on page 2-70          |
| “Unit Systems List” on page 2-71              |
| “Defining Constants of SI Units” on page 2-72 |

### Units List

#### Length

- `Ao` - angstrom
- `a_0` - Bohr radius
- `au` - astronomical unit
- `ch` - chain
- `ft` - foot
- `ft_US` - US survey foot
- `ftm` - fathom
- `fur` - furlong
- `gg` - gauge
- `hand` - hand
- `in` - inch
- `inm` - international nautical mile
- `land` - league
- `li` - link
- `line` - line
- `ly` - light-year
- `m` - meter (SI)
- `mi` - mile
- `mi_US` - US survey mile
- `mil` - mil
- `nmile` - British imperial nautical mile
- `pc` - parsec
- `pica` - pica
- `pica_US` - US customary pica
- `pt` - point



- pt\_US - US customary point
- rod - rod
- span - span
- xu - x unit
- xu\_Cu - x unit (copper)
- xu\_Mo - x unit (molybdenum)
- yd - yard

**Mass**

- Mt - metric megaton
- ct - carat
- cwt - US customary short hundredweight
- cwt\_UK - British imperial short hundredweight
- dalton - atomic mass constant
- dr - dram
- g - gram (SI)
- gr - grain
- hyl - hyl
- kt - metric kiloton
- lbm - pound mass
- m\_e - electron mass
- m\_p - proton mass
- m\_u - atomic mass constant
- oz - ounce
- quarter - quarter
- slug - slug
- stone - stone
- t - metric ton
- tn - US customary short ton
- ton\_UK - British imperial ton

**Time**

- d - day
- fortnight - 14 days
- h - hour
- min - minute
- month\_30 - 30-day month
- s - second (SI)
- week - 7-day week
- year\_360 - 360-day year

- `year_Julian` - Julian year
- `year_Gregorian` - Gregorian year

### **Absorbed Dose or Dose Equivalent**

- `Gy` - gray (SI)
- `Rad` - absorbed radiation dose
- `Sv` - sievert (SI)
- `rem` - roentgen equivalent man

### **Acceleration**

- `Gal` - gal
- `g_n` - earth gravitational acceleration

### **Activity**

- `Bq` - becquerel (SI)
- `Ci` - curie

### **Amount of Substance**

- `item` - number of items
- `mol` - mole (SI)
- `molecule` - number of molecules

### **Angular Momentum**

- `Nms` - newton meter second
- `h_bar` - reduced Planck constant
- `h_c` - Planck constant

### **Area**

- `a` - are
- `ac` - acre
- `ac_US` - US survey acre
- `barn` - barn
- `circ_mil` - circular mil
- `circ_inch` - circular inch
- `ha` - metric hectare
- `ha_US` - US survey hectare
- `ro` - rood
- `twp` - township

### **Capacitance**

- `F` - farad (SI)

- abF - abfarad
- statF - statfarad

**Capacitance Per Length**

- $\epsilon_0$  - vacuum electric permittivity or electric constant

**Catalytic Activity**

- kat - katal (SI)

**Conductance**

- $G_0$  - conductance quantum
- S - siemens (SI)
- abS - absiemens
- statS - statsiemens

**Data Transfer Rate**

- Bd - baud
- bps - bit per second

**Digital Information**

- B - byte
- bit - basic unit of information

**Dimensionless**

- $\alpha$  - fine-structure constant
- $g_e$  - electron g-factor
- $g_p$  - proton g-factor

**Dose Equivalent**

- Sv - sievert (SI)

**Dynamic Viscosity**

- P - poise
- reyn - reynolds

**Electric Charge**

- C - coulomb (SI)
- Fr - franklin
- abC - abcoulomb
- e - elementary charge
- statC - statcoulomb

**Electric Charge Per Mole of Electrons**

- `F_c` - Faraday constant

**Electric Current**

- `A` - ampere (SI)
- `Bi` - biot
- `abA` - abampere
- `statA` - statampere

**Electric Dipole Moment**

- `debye` - debye

**Electric Potential or Electromotive Force**

- `V` - volt (SI)
- `abV` - abvolt
- `statV` - statvolt

**Energy or Work or Heat**

- `Btu_IT` - British thermal unit (International Table)
- `Btu_th` - British thermal unit (thermochemical)
- `E_h` - Hartree energy
- `J` - joule (SI)
- `Nm` - newton meter
- `Wh` - watt hour
- `Ws` - watt second
- `cal_4` - calorie (4 degree Celsius)
- `cal_20` - calorie (20 degree Celsius)
- `cal_15` - calorie (15 degree Celsius)
- `cal_IT` - calorie (International Table)
- `cal_th` - calorie (thermochemical)
- `cal_mean` - calorie (mean)
- `eV` - electronvolt
- `erg` - erg
- `kcal_4` - kilocalorie (4 degree Celsius)
- `kcal_20` - kilocalorie (20 degree Celsius)
- `kcal_15` - kilocalorie (15 degree Celsius)
- `kcal_IT` - kilocalorie (International Table)
- `kcal_th` - kilocalorie (thermochemical)
- `kcal_mean` - kilocalorie (mean)
- `kpm` - kilopond meter

- therm - therm

**Energy Per Temperature**

- k<sub>B</sub> - Boltzmann constant

**Energy Per Temperature Per Amount of Substance**

- R<sub>c</sub> - molar gas constant

**European Currency**

- Cent - cent
- EUR - Euro

**Flow Rate**

- gpm - US customary gallon per minute
- gpm<sub>UK</sub> - British imperial gallon per minute
- lpm - liter per minute

**Force**

- N - newton (SI)
- dyn - dyne
- kgf - kilogram force
- kip - kip
- kp - kilopond
- lbf - pound force
- ozf - ounce force
- p - pond
- pdl - poundal
- sn - sthene
- tonf - short ton force

**Former European Currency**

- ATS - Austrian Schilling
- BEF - Belgian Franc
- DM - German Mark
- ESP - Spanish Peseta
- FIM - Finnish Markka
- FRF - French Franc
- IEP - Irish Pound
- ITL - Italian Lire
- LUF - Luxembourgian Franc
- NLG - Dutch Gulden

- PTE - Portuguese Escudo

### **Frequency**

- Hz - hertz (SI)
- dv\_Cs - caesium hyperfine transition frequency

### **Frequency of Rotation**

- rpm - revolution per minute
- rps - revolution per second

### **Fuel Consumption**

- l\_100km - liter per 100 km

### **Fuel Economy**

- mpg - mile per gallon

### **Gravity**

- G\_c - Newtonian constant of gravitation

### **Illuminance**

- lx - lux (SI)
- nx - nox
- ph - phot

### **Inductance**

- H - henry (SI)
- abH - abhenry
- statH - stathenry

### **Inductance Per Length**

- mu\_0 - vacuum magnetic permeability

### **Intensity Per Fourth Power of Absolute Temperature**

- sigma - Stefan-Boltzmann constant

### **Ionising Dosage**

- R - roentgen

### **Kinematic Viscosity**

- St - stokes
- newt - newt

**Luminance**

- asb - apostilb
- sb - stilb

**Luminous Efficacy**

- $K_{cd}$  - luminous efficacy of a defined visible radiation

**Luminous Flux**

- lm - lumen (SI)

**Luminous Intensity**

- cd - candela (SI)
- cp - candlepower

**Magnetic Field Strength**

- Oe - oersted

**Magnetic Flux**

- Mx - maxwell
- Wb - weber (SI)
- abWb - abweber
- statWb - statweber
- $\phi_0$  - magnetic flux quantum

**Magnetic Flux Density**

- G - gauss
- T - tesla (SI)
- abT - abtesla
- statT - stattesla

**Magnetic Force**

- Gb - gilbert

**Mass Per Length**

- den - denier
- tex - filament tex

**Particle Magnetic Moment Per Angular Momentum**

- $\gamma_e$  - electron gyromagnetic ratio
- $\gamma_p$  - proton gyromagnetic ratio

**Particle Per Amount of Substance**

- `N_A` - Avogadro constant

**Plane Angle**

- `arcsec` - arcsecond
- `arcmin` - arcminute
- `deg` - degree
- `rad` - radian (SI)
- `rev` - revolution

**Power or Heat Flow Rate**

- `HP_E` - electrical horsepower
- `HP_I` - mechanical horsepower
- `HP_UK` - British imperial horsepower
- `HP_DIN` - metric horsepower (DIN 66036)
- `PS_SAE` - net horsepower (SAE J1349)
- `PS_DIN` - horsepower (DIN 70020)
- `W` - watt (SI)
- `poncelet` - poncelet

**Pressure or Stress**

- `Ba` - barye
- `Pa` - pascal (SI)
- `Torr` - torr
- `at` - technical atmosphere
- `atm` - standard atmosphere
- `bar` - bar
- `cmHg` - centimeter of mercury (conventional)
- `cmH2O` - centimeter of water (conventional)
- `ftHg` - foot of mercury (conventional)
- `ftH2O` - foot of water (conventional)
- `inHg` - inch of mercury (conventional)
- `inH2O` - inch of water (conventional)
- `ksf` - kip per square foot
- `ksi` - kip per square inch
- `mH2O` - meter of water (conventional)
- `mHg` - meter of mercury (conventional)
- `mmHg` - millimeter of mercury (conventional)
- `mmH2O` - millimeter of water (conventional)
- `psf` - pound force per square foot



- psi - pound force per square inch
- pz - pieze

**Quantum Resistance**

- $R_K$  - von Klitzing constant

**Radiation**

- lan - langley

**Reciprocal Length**

- kayser - kayser
- $R_{\infty}$  - Rydberg constant

**Refractive Power of Lenses**

- dpt - diopter

**Resistance**

- Ohm - ohm (SI)
- abOhm - abohm
- statOhm - statohm

**Solid Angle**

- sr - steradian (SI)

**Substance Per Volume**

- molarity - molarity

**Temperature**

- Celsius - degree Celsius (SI)
- Fahrenheit - degree Fahrenheit
- K - kelvin (SI)
- Rankine - degree Rankine
- Reaumur - degree Reaumur

**Velocity**

- Kyne - kyne
- $c_0$  - speed of light in vacuum
- fpm - foot per minute
- fps - foot per second
- kmh - kilometer per hour
- knot<sub>UK</sub> - British imperial knot
- kts - international knot

- mach - speed of sound
- mph - mile per hour

**Volume**

- barrel - barrel
- bbl - US customary dry barrel
- bu\_UK - British imperial bushel
- chaldron - chaldron
- dry\_bu - US customary dry bushel
- dry\_gal - US customary dry gallon
- dry\_pk - US customary dry peck
- dry\_pt - US customary dry pint
- dry\_qt - US customary dry quart
- fldr - US customary fluid dram
- fldr\_UK - British imperial fluid drachm (dram)
- floz - US customary fluid ounce
- floz\_UK - British imperial fluid ounce
- gal - US customary liquid gallon
- gal\_UK - British imperial gallon
- gill - US customary fluid gill
- gill\_UK - British imperial gill
- igal - British imperial gallon
- l - liter
- liq\_pt - US customary liquid pint
- liq\_qt - US customary liquid quart
- minim - US customary minim
- minim\_UK - British imperial minim
- pint - US customary liquid pint
- pint\_UK - British imperial pint
- pk\_UK - British imperial peck
- pottle - British imperial pottle
- qt\_UK - British imperial quart
- quart - US customary liquid quart

**SI Unit Prefixes List**

SI unit prefixes in Symbolic Math Toolbox. Every unit marked by (SI) in the units list accepts SI prefixes. For example, m accepts nm, mcm, mm, cm, km and so on.

| Prefix           | Input Forms    | Example using meters u.m<br>where u = symunit |
|------------------|----------------|-----------------------------------------------|
| Yotta $10^{24}$  | Y, yotta       | u.Ym, u.yottam                                |
| Zetta $10^{21}$  | Z, zetta       | u.Zm, u.zettam                                |
| Exa $10^{18}$    | E, exa         | u.Em, u.exam                                  |
| Peta $10^{15}$   | P, peta        | u.Pm, u.petam                                 |
| Tera $10^{12}$   | T, tera        | u.Tm, u.teram                                 |
| Giga $10^9$      | G, giga        | u.gigam, u.Gm                                 |
| Mega $10^6$      | M, mega        | u.Mm, u.megam                                 |
| Kilo $10^3$      | k, kilo        | u.km, u.kilom                                 |
| Hecto $10^2$     | h, hecto       | u.hm, u.hectom                                |
| Deka $10^1$      | da, deka, deca | u.dam, u.dekam, u.decam                       |
| Deci $10^{-1}$   | d, deci        | u.dm, u.decim                                 |
| Centi $10^{-2}$  | c, centi       | u.cm, u.centim                                |
| Milli $10^{-3}$  | m, milli       | u.mm, u.millim                                |
| Micro $10^{-6}$  | mc, micro, u   | u.mcm, u.microm, u.um                         |
| Nano $10^{-9}$   | n, nano        | u.nm, u.nanom                                 |
| Pico $10^{-12}$  | p, pico        | u.pm, u.picom                                 |
| Femto $10^{-15}$ | f, femto       | u.fm, u.femtom                                |
| Atto $10^{-18}$  | a, atto        | u.am, u.attom                                 |
| Zepto $10^{-21}$ | z, zepto       | u.zm, u.zeptom                                |
| Yocto $10^{-24}$ | y, yocto       | u.ym, u.yoctom                                |

## Unit Systems List

Available unit systems in Symbolic Math Toolbox are listed below. For details, see “Unit Conversions and Unit Systems” on page 2-53.

| Unit System       | Base Units                                                   | Derived Units                                                                                                                         |
|-------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| SI units ('SI')   | baseUnits('SI')<br><br>ans =<br>[ [kg], [s], [m], [A], [cd], | derivedUnits('SI')<br><br>ans =<br>[[m], [K], [s], [H], [V], [J], [N], [l],<br>[Pa], [Ohm], [T], [Gy], [Bq], [Sv], [H],<br>[Celsius]] |
| CGS units ('CGS') | baseUnits('CGS')<br><br>ans =<br>[ [cm], [g], [s], [K]]      | derivedUnits('CGS')<br><br>ans =<br>[ [Gal], [dyn], [erg], [Ba], [P], [St],                                                           |





## Units in Physics Calculations

This example shows how to work with units in physics calculations. Calculate the terminal velocity of a falling paratrooper in both SI and imperial units. Solve the motion of the paratrooper, taking into account the gravitational force and the drag force.

### Introduction

Imagine a paratrooper jumping out of an airplane. Assume there are only two forces acting on the paratrooper: the gravitational force and an opposing drag force from the parachute. The drag force is proportional to the velocity squared of the paratrooper.



The net force acting on the paratrooper can be expressed as

mass · acceleration = drag force – gravitational force,

$$m \frac{\partial}{\partial t} v(t) = c_d v(t)^2 - m g,$$

where

- $m$  is the mass of the paratrooper
- $g$  is the gravitational acceleration
- $v(t)$  is the velocity of the paratrooper
- $c_d$  is the drag constant

### Define and Solve Equation of Motion

Define the differential equation describing the equation of motion.

```
syms g m c_d  
syms v(t)  
eq = m*diff(v(t),t) + m*g == c_d*v(t)^2
```

```
eq =  
m  $\frac{\partial}{\partial t}$  v(t) + g m = c_d v(t)^2
```

Assume that the parachute opens immediately at  $t = 0$  so that the equation `eq` is valid for all values of  $t \geq 0$ . Solve the differential equation analytically using `dsolve` with the initial condition  $v(0) = 0$ . The solution represents the velocity of the paratrooper as a function of time.

```
velocity = simplify(dsolve(eq, v(0) == 0))
```

```
velocity =
```

$$-\frac{\sqrt{g} \sqrt{m} \tanh\left(\frac{\sqrt{c_d} \sqrt{g} t}{\sqrt{m}}\right)}{\sqrt{c_d}}$$

### Find Unit of Drag Constant

Find the SI unit of the drag constant  $c_d$ .

The SI unit of force is the Newton (N). In terms of the base units, the Newton is  $\left(\frac{\text{kg} \cdot \text{m}}{\text{s}^2}\right)$ . Since these are equivalent, they have a unit conversion factor of 1.

```
u = symunit;
unitConversionFactor(u.N, u.kg*u.m/u.s^2)
```

```
ans = 1
```

The drag force  $c_d v(t)^2$  must have the same unit in Newton (N) as the gravitational force  $m g$ . Using dimensional analysis, solve for the unit of  $c_d$ .

```
syms drag_units_SI
drag_units_SI = simplify(solve(drag_units_SI * (u.m / u.s)^2 == u.N))
```

```
drag_units_SI =
```

$$\frac{\text{kg}}{\text{m}}$$

### Estimate Terminal Velocity

Describe the motion of the paratrooper by defining the following values.

- Mass of the paratrooper  $m = 70 \text{ kg}$
- Gravitational acceleration  $g = 9.81 \text{ m/s}^2$
- Drag coefficient  $c_d = 40 \text{ kg/m}$

Substitute these values into the velocity equation and simplify the result.

```
vel_SI = subs(velocity,[g,m,c_d],[9.81*u.m/u.s^2, 70*u.kg, 40*drag_units_SI])
```

```
vel_SI =
```

$$-\frac{\tanh\left(\frac{t \sqrt{40 \frac{\text{kg}}{\text{m}}} \sqrt{\frac{981 \text{ m}}{100 \text{ s}^2}}}{\sqrt{70 \text{ kg}}}\right) \sqrt{70 \text{ kg}} \sqrt{\frac{981 \text{ m}}{100 \text{ s}^2}}}{\sqrt{40 \frac{\text{kg}}{\text{m}}}}$$

```
vel_SI = simplify(vel_SI)
```

```
vel_SI =
```

$$-\frac{3\sqrt{763}\tanh\left(\frac{3\sqrt{763}t}{35}\frac{1}{s}\right)}{20}\frac{m}{s}$$

Compute a numerical approximation of the velocity to 3 significant digits.

```
digits(3)
vel_SI = vpa(vel_SI)

vel_SI =
    -4.14 tanh(2.37 t  $\frac{1}{s}$ )  $\frac{m}{s}$ 
```

The paratrooper approaches a constant velocity when the gravitational force is balanced by the drag force. This is called the terminal velocity and it occurs when the drag force from the parachute cancels out the gravitational force (there is no further acceleration). Find the terminal velocity by taking the limit of  $t \rightarrow \infty$ .

```
vel_term_SI = limit(vel_SI, t, Inf)

vel_term_SI =
    -4.14  $\frac{m}{s}$ 
```

### Convert Velocity to Imperial Units

Finally, convert the velocity function from SI units to imperial units.

```
vel_Imperial = rewrite(vel_SI,u.ft)

vel_Imperial =
    -13.6 tanh(2.37 t  $\frac{1}{s}$ )  $\frac{ft}{s}$ 
```

Convert the terminal velocity.

```
vel_term_Imperial = rewrite(vel_term_SI,u.ft)

vel_term_Imperial =
    -13.6  $\frac{ft}{s}$ 
```

### Plot Velocity over Time

To plot the velocity as a function of time, express the time  $t$  in seconds and replace  $t$  by  $T$  s, where  $T$  is a dimensionless symbolic variable.

```
syms T
vel_SI = subs(vel_SI, t, T*u.s)

vel_SI =
    -4.14 tanh(2.37 T)  $\frac{m}{s}$ 

vel_Imperial = rewrite(vel_SI, u.ft)

vel_Imperial =
    -13.6 tanh(2.37 T)  $\frac{ft}{s}$ 
```

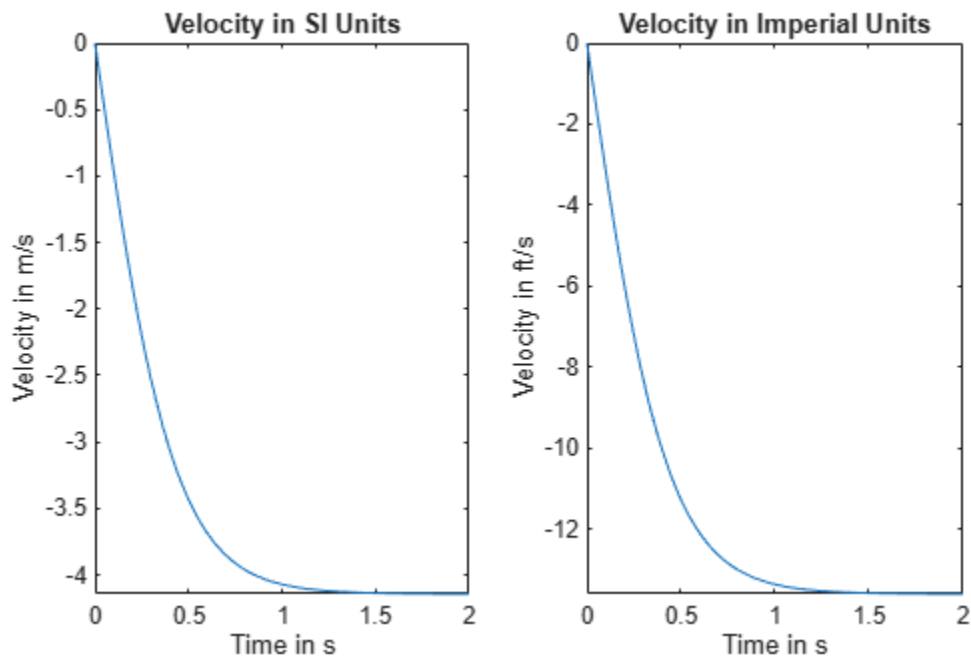


Separate the expression from the units by using `separateUnits`. Plot the expression using `fplot`. Convert the units to strings for use as plot labels by using `symunit2str`.

```
[data_SI, units_SI] = separateUnits(vel_SI);
[data_Imperial, units_Imperial] = separateUnits(vel_Imperial);
```

The velocity of the paratrooper approaches steady state when  $t > 1$ . Show how the velocity approaches terminal velocity by plotting the velocity over the range  $0 \leq T \leq 2$ .

```
subplot(1,2,1)
fplot(data_SI,[0 2])
title('Velocity in SI Units')
xlabel('Time in s')
ylabel(['Velocity in ' symunit2str(units_SI)])
subplot(1,2,2)
fplot(data_Imperial,[0 2])
title('Velocity in Imperial Units')
xlabel('Time in s')
ylabel(['Velocity in ' symunit2str(units_Imperial)])
```



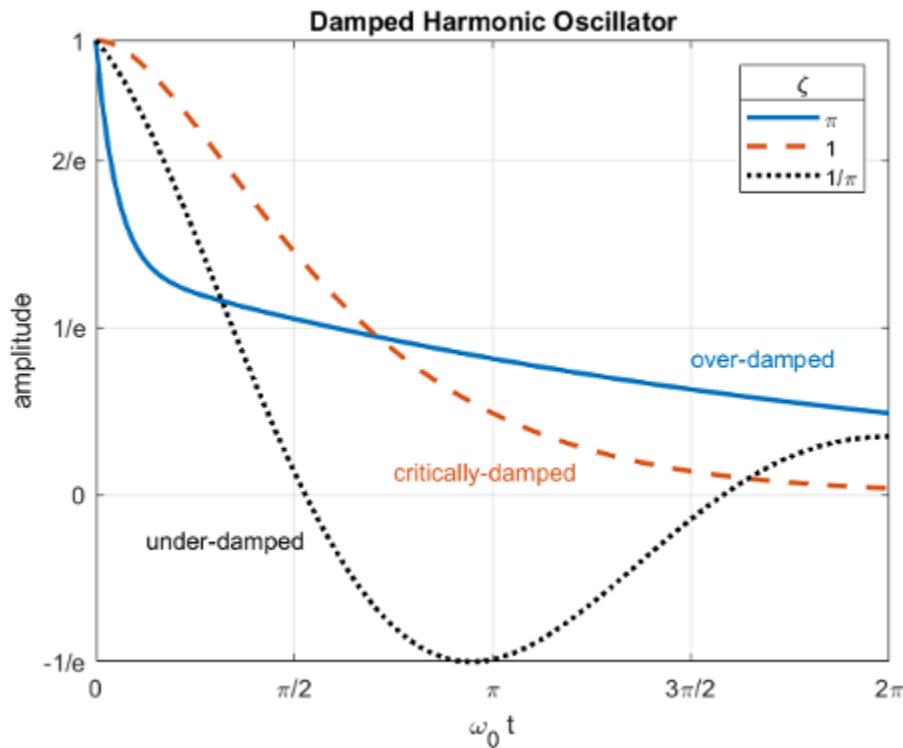
## See Also

### Functions

`symunit` | `unitConversionFactor` | `rewrite` | `separateUnits`

## The Physics of the Damped Harmonic Oscillator

This example explores the physics of the damped harmonic oscillator by solving the equations of motion in the case of no driving forces. This example investigates the cases of underdamping, overdamping, and critical damping.

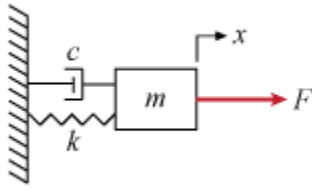


### Contents

- 1 Derive Equation of Motion
- 2 Solve the Equation of Motion ( $F = 0$ )
- 3 Underdamped Case ( $\zeta < 1$ )
- 4 Overdamped Case ( $\zeta > 1$ )
- 5 Critically Damped Case ( $\zeta = 1$ )
- 6 Conclusion

#### 1. Derive Equation of Motion

Consider a forced harmonic oscillator with damping shown below. Model the resistance force as proportional to the speed with which the oscillator moves.



Define the equation of motion where

- $m$  is the mass
- $c$  is the damping coefficient
- $k$  is the spring constant
- $F$  is a driving force

```
syms x(t) m c k F(t)
eq = m*diff(x,t,t) + c*diff(x,t) + k*x == F
```

```
eq(t) =
      2      2
m   $\frac{\partial}{\partial t}$  x(t) + c  $\frac{\partial}{\partial t}$  x(t) + k x(t) = F(t)
```

Rewrite the equation using  $c = m \gamma$  and  $k = m \omega_0^2$ .

```
syms gamma omega_0
eq = subs(eq,[c k],[m*gamma m*omega_0^2])
```

```
eq(t) =
      2      2
m   $\frac{\partial}{\partial t}$  x(t) + gamma m  $\frac{\partial}{\partial t}$  x(t) + m x(t) omega_0^2 = F(t)
```

Divide out the mass  $m$ . Now we have the equation in a convenient form to analyze.

```
eq = collect(eq,m)/m
```

```
eq(t) =
      2      2
 $\frac{\partial}{\partial t}$  x(t) + gamma  $\frac{\partial}{\partial t}$  x(t) + x(t) omega_0^2 =  $\frac{F(t)}{m}$ 
```

## 2. Solve the Equation of Motion where $F = 0$

Solve the equation of motion using `dsolve` in the case of no external forces where  $F = 0$ . Use the initial conditions of unit displacement and zero velocity.

```
vel = diff(x,t);
cond = [x(0) == 1, vel(0) == 0];
eq = subs(eq,F,0);
sol = dsolve(eq,cond)
```

```
sol =
```

$$\frac{e^{-t\left(\frac{\gamma}{2} - \frac{\sigma_1}{2}\right)} (\gamma + \sigma_1)}{2 \sigma_1} - \frac{e^{-t\left(\frac{\gamma}{2} + \frac{\sigma_1}{2}\right)} (\gamma - \sigma_1)}{2 \sigma_1}$$

where

$$\sigma_1 = \sqrt{(\gamma - 2 \omega_0) (\gamma + 2 \omega_0)}$$

Examine how to simplify the solution by expanding it.

```
sol = expand(sol)
```

```
sol =
```

$$\frac{\sigma_1 \sigma_2}{2} + \frac{\sigma_1 e^{\frac{t \sqrt{\gamma^2 - 4 \omega_0^2}}{2}}}{2} - \frac{\gamma \sigma_1 \sigma_2}{2 \sqrt{\gamma^2 - 4 \omega_0^2}} + \frac{\gamma \sigma_1 e^{\frac{t \sqrt{\gamma^2 - 4 \omega_0^2}}{2}}}{2 \sqrt{\gamma^2 - 4 \omega_0^2}}$$

where

$$\sigma_1 = e^{-\frac{\gamma t}{2}}$$

$$\sigma_2 = e^{-\frac{t \sqrt{\gamma^2 - 4 \omega_0^2}}{2}}$$

Notice that each term has a factor of  $\sigma_1$ , or  $e^{-\gamma t/2}$ , use `collect` to gather these terms

```
sol = collect(sol,exp(-gamma*t/2))
```

```
sol =
```

$$\left( \frac{e^{-\sigma_1}}{2} + \frac{e^{\sigma_1}}{2} - \frac{\gamma e^{-\sigma_1}}{2 \sqrt{\gamma^2 - 4 \omega_0^2}} + \frac{\gamma e^{\sigma_1}}{2 \sqrt{\gamma^2 - 4 \omega_0^2}} \right) e^{-\frac{\gamma t}{2}}$$

where

$$\sigma_1 = \frac{t \sqrt{\gamma^2 - 4 \omega_0^2}}{2}$$

The term  $\sqrt{\gamma^2 - 4 \omega_0^2}$  appears in various parts of the solution. Rewrite it in a simpler form by introducing the damping ratio  $\zeta \equiv \frac{\gamma}{2 \omega_0}$ .

Substituting  $\zeta$  into the term above gives:

$$\sqrt{\gamma^2 - 4 \omega_0^2} = 2 \omega_0 \sqrt{\left(\frac{\gamma}{2 \omega_0}\right)^2 - 1} = 2 \omega_0 \sqrt{\zeta^2 - 1},$$

```
syms zeta;
```

```
sol = subs(sol, ...  
    sqrt(gamma^2 - 4*omega_0^2), ...  
    2*omega_0*sqrt(zeta^2-1))
```

```
sol =
```

$$e^{-\frac{\gamma t}{2}} \left( \frac{\sigma_2}{2} + \frac{\sigma_1}{2} + \frac{\gamma \sigma_2}{4 \omega_0 \sqrt{\zeta^2 - 1}} - \frac{\gamma \sigma_1}{4 \omega_0 \sqrt{\zeta^2 - 1}} \right)$$

where

$$\sigma_1 = e^{-\omega_0 t \sqrt{\zeta^2 - 1}}$$

$$\sigma_2 = e^{\omega_0 t \sqrt{\zeta^2 - 1}}$$

Further simplify the solution by substituting  $\gamma$  in terms of  $\omega_0$  and  $\zeta$ ,

`sol = subs(sol,gamma,2*zeta*omega_0)`

`sol =`

$$e^{-\omega_0 t \zeta} \left( \frac{\sigma_2}{2} + \frac{\sigma_1}{2} + \frac{\zeta \sigma_2}{2 \sqrt{\zeta^2 - 1}} - \frac{\zeta \sigma_1}{2 \sqrt{\zeta^2 - 1}} \right)$$

where

$$\sigma_1 = e^{-\omega_0 t \sqrt{\zeta^2 - 1}}$$

$$\sigma_2 = e^{\omega_0 t \sqrt{\zeta^2 - 1}}$$

We have derived the general solution for the motion of the damped harmonic oscillator with no driving forces. Next, we will explore three special cases of the damping ratio  $\zeta$  where the motion takes on simpler forms. These cases are called

- underdamped ( $\zeta < 1$ ),
- overdamped ( $\zeta > 1$ ), and
- critically damped ( $\zeta = 1$ ).

### 3. Underdamped Case ( $\zeta < 1$ )

If  $\zeta < 1$ , then  $\sqrt{\zeta^2 - 1} = i\sqrt{1 - \zeta^2}$  is purely imaginary.

`solUnder = subs(sol,sqrt(zeta^2-1),1i*sqrt(1-zeta^2))`

`solUnder =`

$$e^{-\omega_0 t \zeta} \left( \frac{\sigma_1}{2} + \frac{\sigma_2}{2} + \frac{\zeta \sigma_1 i}{2 \sqrt{1 - \zeta^2}} - \frac{\zeta \sigma_2 i}{2 \sqrt{1 - \zeta^2}} \right)$$

where

$$\sigma_1 = e^{-\omega_0 t \sqrt{1 - \zeta^2} i}$$

$$\sigma_2 = e^{\omega_0 t \sqrt{1 - \zeta^2} i}$$

Notice the terms  $e^{i \omega_0 t \sqrt{1 - \zeta^2}} \pm e^{-i \omega_0 t \sqrt{1 - \zeta^2}}$  in the above equation and recall the identity  $e^{i x} = \cos(x) + i \sin(x)$ .

Rewrite the solution in terms of cos.

```
solUnder = coeffs(solUnder,zeta);
solUnder = solUnder(1);
c = exp(-omega_0*zeta*t);
solUnder = c*rewrite(solUnder/c,"cos")
```

$$\text{solUnder} = e^{-\omega_0 t \zeta} \cos(\omega_0 t \sqrt{1 - \zeta^2})$$

```
solUnder(t,omega_0,zeta) = solUnder
```

$$\text{solUnder}(t, \omega_0, \zeta) = e^{-\omega_0 t \zeta} \cos(\omega_0 t \sqrt{1 - \zeta^2})$$

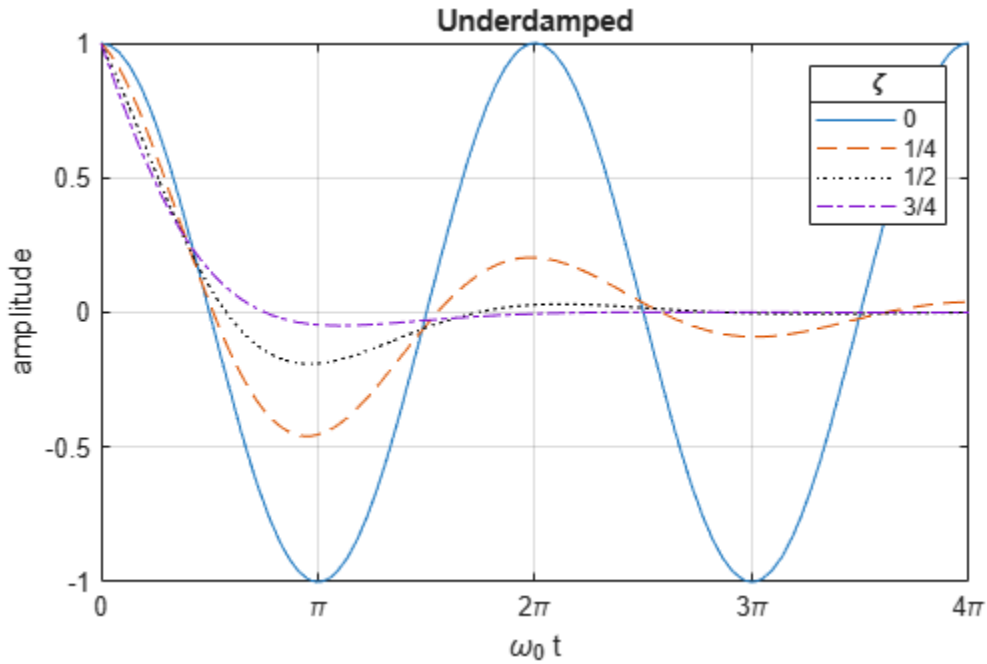
The system oscillates at a natural frequency of  $\omega_0 \sqrt{1 - \zeta^2}$  and decays at an exponential rate of  $1/\omega_0 \zeta$ .

Plot the solution with `fplot` as a function of  $\omega_0 t$  and  $\zeta$ .

```
z = [0 1/4 1/2 3/4];
w = 1;
T = 4*pi;
lineStyle = {"-", "-.", ":k", "-."};

fplot(@(t) solUnder(t,w,z(1)),[0 T],lineStyle{1});

hold on;
for k = 2:numel(z)
    fplot(@(t) solUnder(t,w,z(k)),[0 T],lineStyle{k});
end
hold off;
grid on;
xticks(T*linspace(0,1,5));
xticklabels({"0", "\pi", "2\pi", "3\pi", "4\pi"});
xlabel("\omega_0 t");
ylabel("amplitude");
lgd = legend("0", "1/4", "1/2", "3/4");
title(lgd, "\zeta");
title("Underdamped");
```



#### 4. Overdamped Case ( $\zeta > 1$ )

If  $\zeta > 1$ , then  $\sqrt{\zeta^2 - 1}$  is purely real and the solution can be rewritten as

`sol0ver = sol`

`sol0ver =`

$$e^{-\omega_0 t \zeta} \left( \frac{\sigma_2}{2} + \frac{\sigma_1}{2} + \frac{\zeta \sigma_2}{2\sqrt{\zeta^2 - 1}} - \frac{\zeta \sigma_1}{2\sqrt{\zeta^2 - 1}} \right)$$

where

$$\sigma_1 = e^{-\omega_0 t \sqrt{\zeta^2 - 1}}$$

$$\sigma_2 = e^{\omega_0 t \sqrt{\zeta^2 - 1}}$$

`sol0ver = coeffs(sol0ver, zeta);`

`sol0ver = sol0ver(1)`

`sol0ver =`

$$e^{-\omega_0 t \zeta} \left( \frac{e^{\omega_0 t \sqrt{\zeta^2 - 1}}}{2} + \frac{e^{-\omega_0 t \sqrt{\zeta^2 - 1}}}{2} \right)$$

Notice the terms  $\frac{(e^{\omega_0 t \sqrt{\zeta^2 - 1}} + e^{-\omega_0 t \sqrt{\zeta^2 - 1}})}{2}$  and recall the identity  $\cosh(x) = \frac{e^x + e^{-x}}{2}$ .

Rewrite the expression in terms of cosh.

`c = exp(-omega_0*t*zeta);`

`sol0ver = c*rewrite(sol0ver/c, "cosh")`

$$\text{sol0ver} = \cosh(\omega_0 t \sqrt{\zeta^2 - 1}) e^{-\omega_0 t \zeta}$$

$$\text{sol0ver}(t, \omega_0, \zeta) = \text{sol0ver}$$

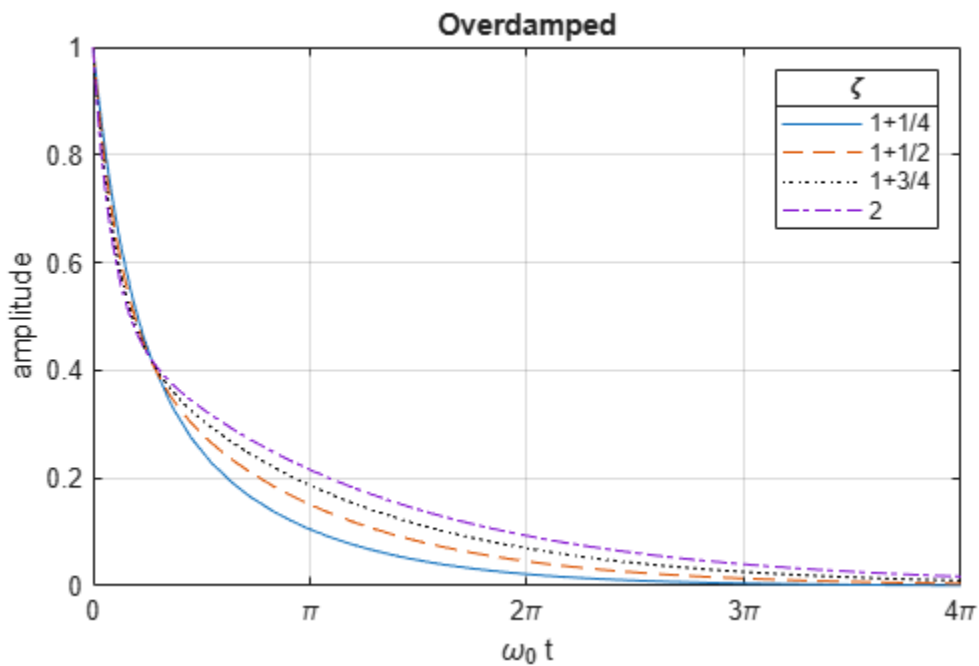
$$\text{sol0ver}(t, \omega_0, \zeta) = \cosh(\omega_0 t \sqrt{\zeta^2 - 1}) e^{-\omega_0 t \zeta}$$

Plot the solution to see that it decays without oscillating.

```
z = 1 + [1/4 1/2 3/4 1];
w = 1;
T = 4*pi;
lineStyle = {"-", "--", ":k", "-."};

fplot(@(t) sol0ver(t,w,z(1)),[0 T],lineStyle{1});

hold on;
for k = 2:numel(z)
    fplot(@(t) sol0ver(t,w,z(k)),[0 T],lineStyle{k});
end
hold off;
grid on;
xticks(T*linspace(0,1,5));
xticklabels({"0", "\pi", "2\pi", "3\pi", "4\pi"});
xlabel("\omega_0 t");
ylabel("amplitude");
lgd = legend("1+1/4", "1+1/2", "1+3/4", "2");
title(lgd, "\zeta");
title("Overdamped");
```



### 5. Critically Damped Case ( $\zeta = 1$ )

If  $\zeta = 1$ , then the solution simplifies to



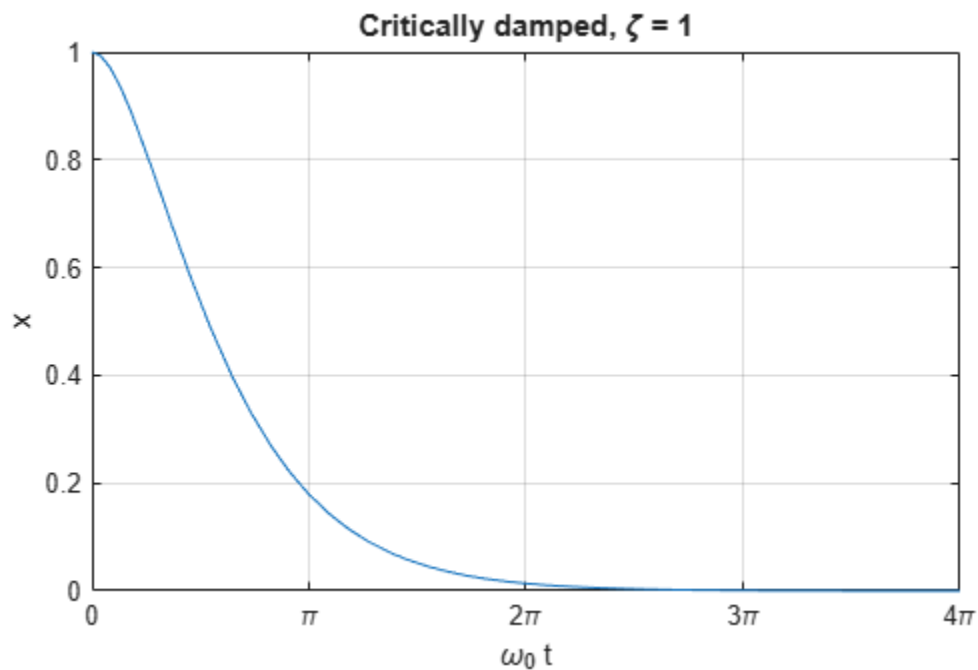
```
solCritical(t,omega_0) = limit(sol,zeta,1)
```

```
solCritical(t, omega_0) = e-ω0t (ω0t + 1)
```

Plot the solution for the critically damped case.

```
w = 1;
T = 4*pi;

fplot(solCritical(t, w), [0 T])
xlabel("\omega_0 t");
ylabel("x");
title("Critically damped, \zeta = 1");
grid on;
xticks(T*linspace(0,1,5));
xticklabels({"0", "\pi", "2\pi", "3\pi", "4\pi"});
```



## 6. Conclusion

We have examined the different damping states for the harmonic oscillator by solving the ODEs which represents its motion using the damping ratio  $\zeta$ . Plot all three cases together to compare and contrast them.

```
zOver = pi;
zUnder = 1/zOver;
w = 1;
T = 2*pi;
lineStyle = {"-", "--", ":k"};

fplot(@(t) solOver(t,w,zOver),[0 T],lineStyle{1},LineWidth=2);
hold on;
fplot(solCritical(t, w), [0 T], lineStyle{2},LineWidth=2)
fplot(@(t) solUnder(t,w,zUnder),[0 T],lineStyle{3},LineWidth=2);
```

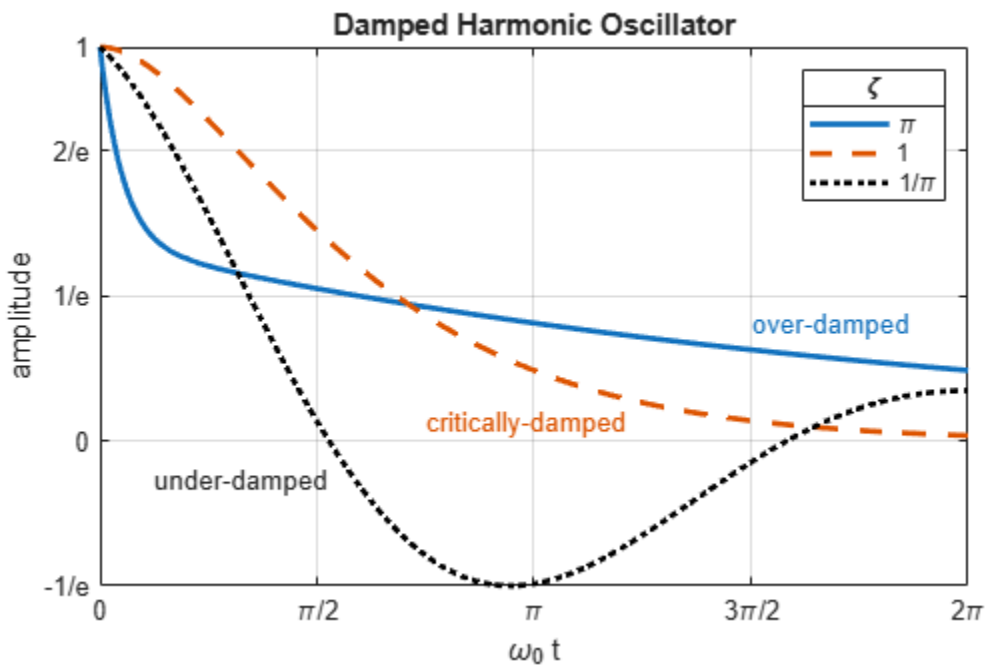
```

hold off;

textColor = lines(3);
text(3*pi/2,0.3,"over-damped",Color=textColor(1,:));
text(pi*3/4,0.05,"critically-damped",Color=textColor(2,:));
text(pi/8,-0.1,"under-damped");

grid on;
xlabel("\omega_0 t");
ylabel("amplitude");
xticks(T*linspace(0,1,5));
xticklabels({"0","\pi/2","\pi","3\pi/2","2\pi"});
yticks((1/exp(1))*[-1 0 1 2 exp(1)]);
yticklabels({"-1/e","0","1/e","2/e","1"});
lgd = legend("\pi","1","1/\pi");
title(lgd,"\zeta");
title("Damped Harmonic Oscillator");

```



## See Also

### Functions

diff | subs | dsolve | fplot | collect | expand | rewrite

## Evaluate Average Power Delivered by Wind Turbine

This example shows how to compute the average power output of a wind turbine using Symbolic Math Toolbox™ and Statistics and Machine Learning Toolbox™. In this example, you define a parametric analytical expression that models the conversion of wind energy to turbine power. Then you evaluate the average power output by assuming that the wind speed follows a Weibull distribution.



### Define Model for Wind Turbine

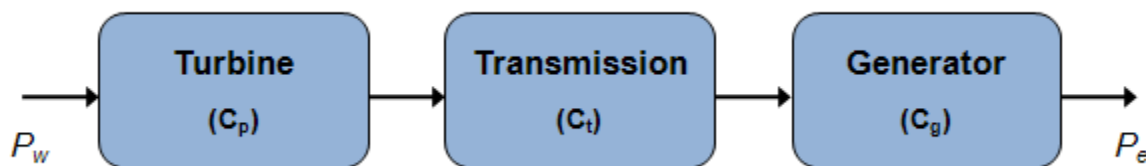
You can estimate the total power delivered to a wind turbine by taking the derivative of the wind kinetic energy.

$$P_w = \frac{1}{2} \frac{d}{dt} (\rho A u^3) = \frac{\rho A u^3}{2}$$

The parameters of this equation are:

- $A$  - The swept area of turbine blades, in  $\text{m}^2$
- $\rho$  - The air density, in  $\text{kg}/\text{m}^3$
- $u$  - The wind speed, in  $\text{m}/\text{s}$

The process of converting wind energy to electrical power results in efficiency losses, as described in this diagram.



This equation describes the electrical power output of a practical wind turbine  $P_e$ .

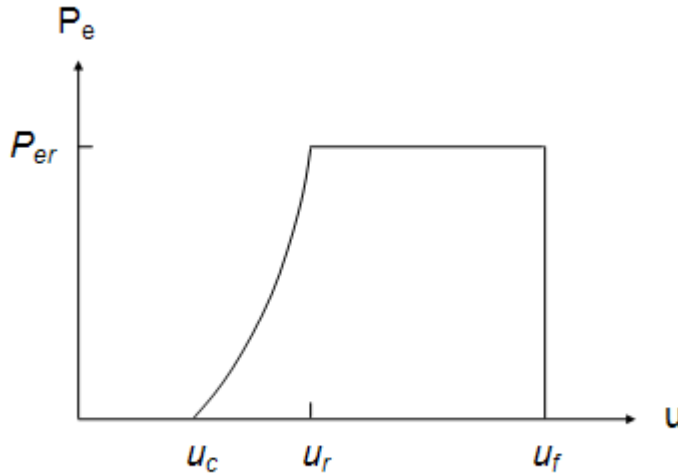
$$P_e(u) = \frac{C_{\text{tot}}(u) \rho A u^3}{2}$$

where  $C_{\text{tot}}(u) = C_p C_t C_g$  is the overall efficiency or conversion factor of the wind energy that depends on the wind speed.

The overall efficiency is between 0.3 and 0.5, varying with both wind speed and rotational speed of the turbine. For a fixed rotational speed, the electrical power delivered by the wind turbine reaches its maximum ( $P_{er}$ ) at a rated wind speed. The overall efficiency at this rated wind speed is expressed as  $C_{\text{totR}}$ .

$$P_{er} = \frac{C_{\text{totR}} \rho A u^3}{2}$$

Assuming a fixed rotational speed, the electrical power output of the wind turbine depends on the wind speed. This graph estimates the profile of  $P_e$  as a function of  $u$ .



The parameters of the graph are:

- $u_c$  - The cut-in speed, or the speed at which the electrical power output rises above zero and power production starts
- $u_r$  - The rated wind speed
- $u_f$  - The furling wind speed, or the speed at which the turbine shuts down to prevent structural damage

The profile of the power output increases between  $u_c$  and  $u_r$ . In this region, the power output  $P_e$  depends on  $u$  exponentially, that is, raised to the power of  $k$ . In the region between  $u_r$  and  $u_f$ , the power output is at a constant maximum value of  $P_{er}$ . For all other conditions, the power output is zero.

### Define Piecewise Function for Power Output

Describe the turbine power output by defining a piecewise function. Assume that wind speed  $u$  and the parameters  $u_c$ ,  $u_r$ ,  $u_f$ ,  $k$ , and  $P_{er}$  are positive real numbers. Also assume that the cut-in speed is less than the rated wind speed, which is less than the furling wind speed.

```
syms C_1 C_2
syms u u_c u_r u_f k P_er positive real
assume(0 < u_c < u_r < u_f)
Pe(u) = piecewise(u < u_c, 0, ...
                  u_c <= u <= u_r, C_1 + C_2*u^k, ...
                  u_r < u <= u_f, P_er, ...
                  u_f < u, 0)
```

$$Pe(u) = \begin{cases} 0 & \text{if } u < u_c \\ C_1 + C_2 u^k & \text{if } u_c \leq u \wedge u \leq u_r \\ P_{er} & \text{if } u \leq u_f \wedge u_r < u \\ 0 & \text{if } u_f < u \end{cases}$$

Based on the conditions where  $P_e(u_c) = 0$  and  $P_e(u_r) = P_{er}$ , solve for the coefficients  $C_1$  and  $C_2$ .

```
sol = solve([Pe(u_c) == 0; Pe(u_r) == P_er], [C_1 C_2]);
```

Substitute the solutions for  $C_1$  and  $C_2$  into the piecewise function.

```
Pe = subs(Pe, {C_1, C_2}, {sol.C_1, sol.C_2})
```

$$Pe(u) = \begin{cases} 0 & \text{if } u < u_c \\ \frac{P_{er} u_c^k}{u_c^k - u_r^k} - \frac{P_{er} u^k}{u_c^k - u_r^k} & \text{if } u_c \leq u \wedge u \leq u_r \\ P_{er} & \text{if } u \leq u_f \wedge u_r < u \\ 0 & \text{if } u_f < u \end{cases}$$

### Define Profile of Wind Speed

The rated power output indicates how much power a wind turbine is capable of producing. However, the turbine power output depends on the wind speed. To calculate the average power produced by the turbine, you need to account for external wind conditions. In this example, use the Weibull distribution to model the wind speed. The wind profile follows the Weibull probability density function.

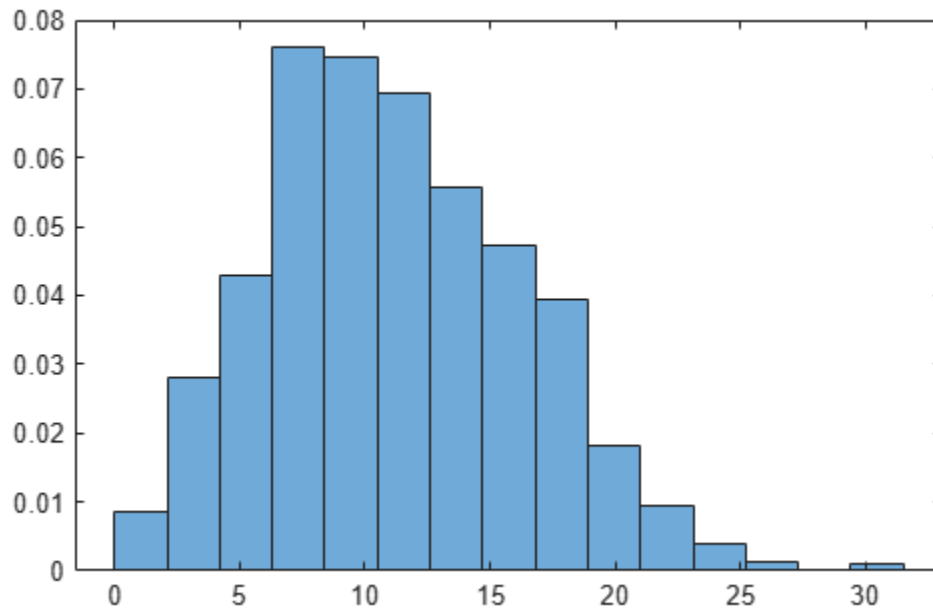
$$f(u) = \frac{\left(\frac{b}{a}\right)\left(\frac{u}{a}\right)^{b-1}}{e\left(\frac{u}{a}\right)^b} \text{ if } u \geq 0$$

The parameters  $a$  and  $b$  represent scale and shape, respectively. In general, a larger value of  $a$  indicates a higher median wind speed, and a larger value of  $b$  indicates a reduced variability in the wind speed.

To generate random numbers from the “Weibull Distribution” (Statistics and Machine Learning Toolbox), you can use the `wblrnd` function. For example, choose the parameters  $a = 12.5$  and

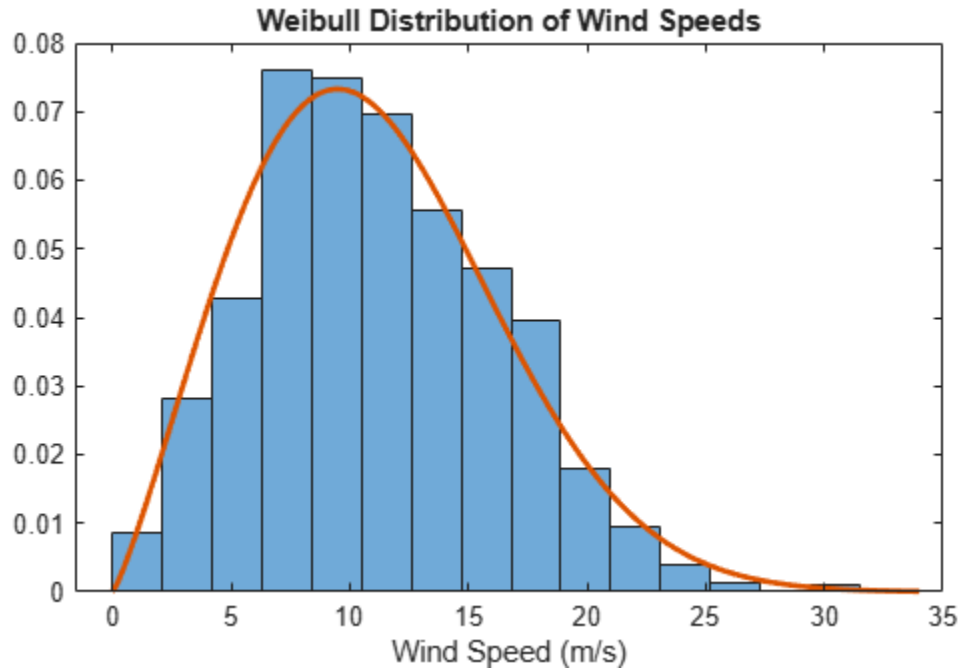
$b = 2$ . 2. Generate 1000 random numbers that follow the Weibull distribution. Plot these random numbers as a histogram that is normalized to the probability density function.

```
a = 12.5;
b = 2.2;
N = 1000;
r = wblrnd(a,b,[1 N]);
histogram(r,15,Normalization="pdf")
```



To generate the Weibull probability density function (pdf), use `wblpdf`. Plot the pdf and compare it with the histogram plot of the sampled random numbers.

```
hold on
x = linspace(0,34,N);
y = wblpdf(x,a,b);
plot(x,y,LineWidth=2)
hold off
title("Weibull Distribution of Wind Speeds")
xlabel("Wind Speed (m/s)")
```



### Calculate Average Power Output

You can calculate the average power output of a wind turbine by using this integral.

$$\bar{P}_e = \int_0^{\infty} P_e(u) f(u) du$$

Define the Weibull pdf analytically, and assume the parameters  $a$  and  $b$  are positive real numbers.

`syms a b positive real`

`f(u) = (b/a)*(u/a)^(b-1)/exp((u/a)^b)`

`f(u) =`

$$\frac{b e^{-\left(\frac{u}{a}\right)^b} \left(\frac{u}{a}\right)^{b-1}}{a}$$

Find the integrand you can use to evaluate the average power output.

`Pintegrand(u) = simplify(Pe(u)*f(u))`

`Pintegrand(u) =`

$$\left\{ \begin{array}{ll} 0 & \text{if } u < u_c \\ -\frac{P_{er} b u^{b-1} e^{-\frac{u^b}{a^b}} (u^k - u_c^k)}{a^b (u_c^k - u_r^k)} & \text{if } u_c \leq u \wedge u \leq u_r \\ \frac{P_{er} b u^{b-1} e^{-\left(\frac{u}{a}\right)^b}}{a^b} & \text{if } u \leq u_f \wedge u_r < u \\ 0 & \text{if } u_f < u \end{array} \right.$$

Perform the integration with respect to the wind speed  $u$  with the bounds from 0 to  $\infty$  by using `int`. The resulting analytical expression represents the average power output of the wind turbine.

```
Pav = int(Pintegrand,u,0,Inf)
```

```
Pav =
```

$$\int_{u_c}^{u_r} \left( -\frac{P_{\text{er}} b u^{b-1} e^{-\frac{u^b}{a^b}} (u^k - u_c^k)}{a^b (u_c^k - u_r^k)} \right) du - P_{\text{er}} \left( e^{-\left(\frac{u_f}{a}\right)^b} - e^{-\left(\frac{u_r}{a}\right)^b} \right)$$

To evaluate the average power output for specific parameter values, you can use the `subs` function. For example, find the average power output for these parameters.

```
params.a = 12.5;
```

```
params.b = 2.2;
```

```
params.k = 2;
```

```
params.uc = 5;
```

```
params.ur = 15;
```

```
params.uf = 40;
```

```
params.Per = 2e5;
```

```
Pav = subs(Pav,{a b k u_c u_r u_f P_er},{params.a params.b params.k params.uc params.ur params.uf params.Per})
```

```
Pav =
```

$$200000 e^{-\frac{36 \cdot 5^{4/5} 6^{1/5}}{125}} - 200000 e^{-\sigma_1} - 200000 e^{-\frac{256 \cdot 5^{4/5} 16^{1/5}}{125}} + \frac{1562500 \Gamma\left(\frac{10}{11}, \frac{4 \cdot 1250^{1/5}}{125}\right)}{11} - \frac{1562500 \Gamma\left(\frac{10}{11}, \sigma_1\right)}{11}$$

where

$$\sigma_1 = \frac{36 \cdot 3750^{1/5}}{125}$$

The result is an exact symbolic number that involves the exponential function and the gamma function. You can use `double` to convert the symbolic number to a double-precision number.

```
Pav_num = double(Pav)
```

```
Pav_num =
```

```
9.7744e+04
```

You can use the parametric expression defined in this example to evaluate various wind turbine configurations and wind farm sites. For more information, see [Wind Resource Assessment](#).

## See Also

`syms` | `assume` | `piecewise` | `int` | `subs`

## Related Examples

- “Perform Symbolic Computations” on page 1-29
- “Working with Probability Distributions” (Statistics and Machine Learning Toolbox)



## Developing an Algorithm for Undistorting an Image

This example develops a mathematical model using the Symbolic Math Toolbox™ to undistort an image and features a local function in the live script.

### Background

Any real world point  $P(X, Y, Z)$  can be defined with respect to some 3-D world origin.

Relative to a camera lens, this 3-D point can be defined as  $p_0$ , which is obtained by rotating and translating  $P$ .

$$p_0 = (x_0, y_0, z_0) = RP + t$$

The 3-D point  $p_0$  is then projected into the camera's image plane as a 2D point,  $(x_1, y_1)$ .

$$x_1 = \frac{x_0}{z_0}, y_1 = \frac{y_0}{z_0}$$

When a camera captures an image, it does not precisely capture the real points, but rather a slightly distorted version of the real points which can be denoted  $(x_2, y_2)$ . The distorted points can be described using the following function:

$$x_2 = x_1 (1 + k_1 r^2 + k_2 r^4) + 2 p_1 x_1 y_1 + p_2 (r^2 + 2 x_1^2)$$

$$y_2 = y_1 (1 + k_1 r^2 + k_2 r^4) + 2 p_2 x_1 y_1 + p_1 (r^2 + 2 y_1^2)$$

where:

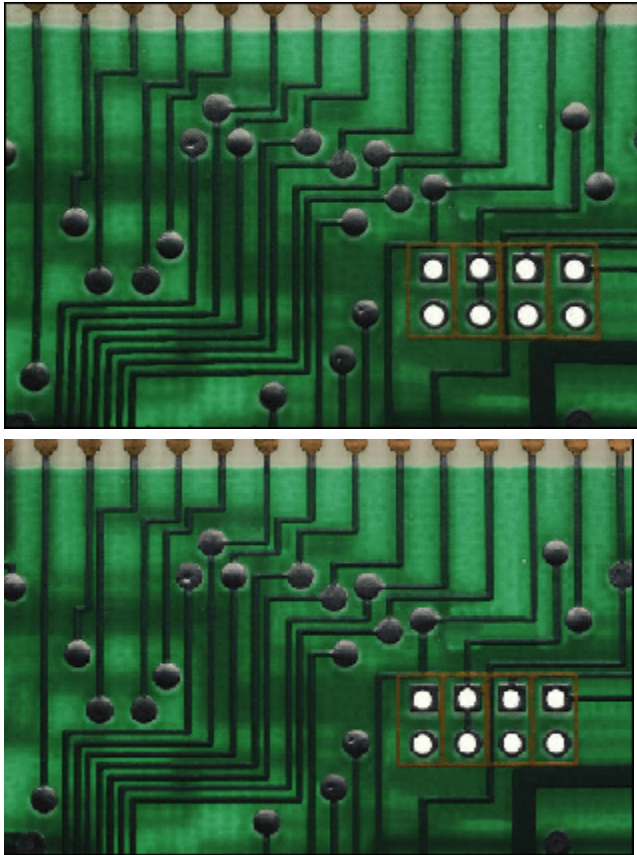
$k_1, k_2$  = radial distortion coefficients of the lens

$p_1, p_2$  = tangential distortion coefficients of the lens

$$r = \sqrt{x_1^2 + y_1^2}$$

### Distortion

An example of lens distortion is shown below; original distorted image (left) and undistorted image (right).



Note the curvature of the lines toward the edges of the first image. For applications such as image reconstruction and tracking, it is important to know the real world location of points. When we have a distorted image, we know the distorted pixel locations  $(x_2, y_2)$ . It's our goal to determine the undistorted pixel locations  $(x_1, y_1)$  given  $(x_2, y_2)$  and the distortion coefficients of the particular lens.

While otherwise straightforward, the nonlinear nature of the lens distortion makes the problem challenging.

### Define Distortion Model

We begin by defining our distortion model:

```
% Parameters
syms k_1 k_2 p_1 p_2 real
syms r x y
distortionX = subs(x*(1 + k_1*r^2 + k_2*r^4) + 2*p_1*x*y + p_2*(r^2 + 2*x^2), r, sqrt(x^2 + y^2))

distortionX = p_2(3*x^2 + y^2) + x(k_2(x^2 + y^2)^2 + k_1(x^2 + y^2) + 1) + 2*p_1*x*y

distortionY = subs(y*(1 + k_1*r^2 + k_2*r^4) + 2*p_2*x*y + p_1*(r^2 + 2*y^2), r, sqrt(x^2 + y^2))

distortionY = p_1(x^2 + 3*y^2) + y(k_2(x^2 + y^2)^2 + k_1(x^2 + y^2) + 1) + 2*p_2*x*y
```

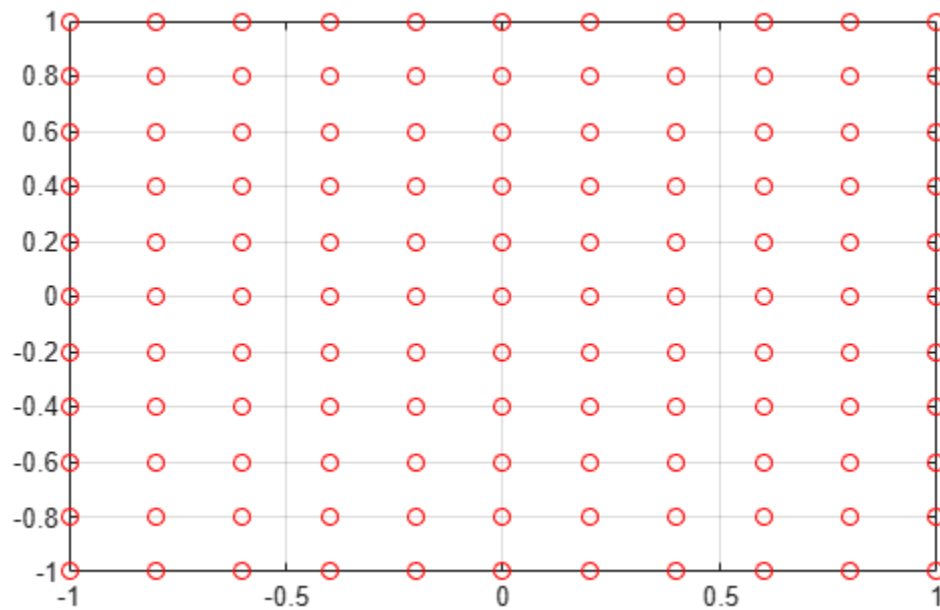
**Radial Distortion**  $k_1 = 0$

We plot a grid of pixel locations assuming our lens has a radial distortion coefficient  $k_1 = 0$ . Note that distortion is smallest near the center of the image and largest near the edges.

```
% Set Parameters
parameters = [k_1 k_2 p_1 p_2];
parameterValues = [0 0 0 0];
plotLensDistortion(distortionX,distortionY,parameters,parameterValues)

spacing =
0.2000

distortionX = x
distortionY = y
```



### Radial Distortion $k_1 = 0.15$

Explore the sensitivity to changes in  $k_1$ .

```
% Set Parameters
parameters = [k_1 k_2 p_1 p_2];
parameterValues = [0.15 0 0 0];
plotLensDistortion(distortionX,distortionY,parameters,parameterValues)

spacing =
0.2000

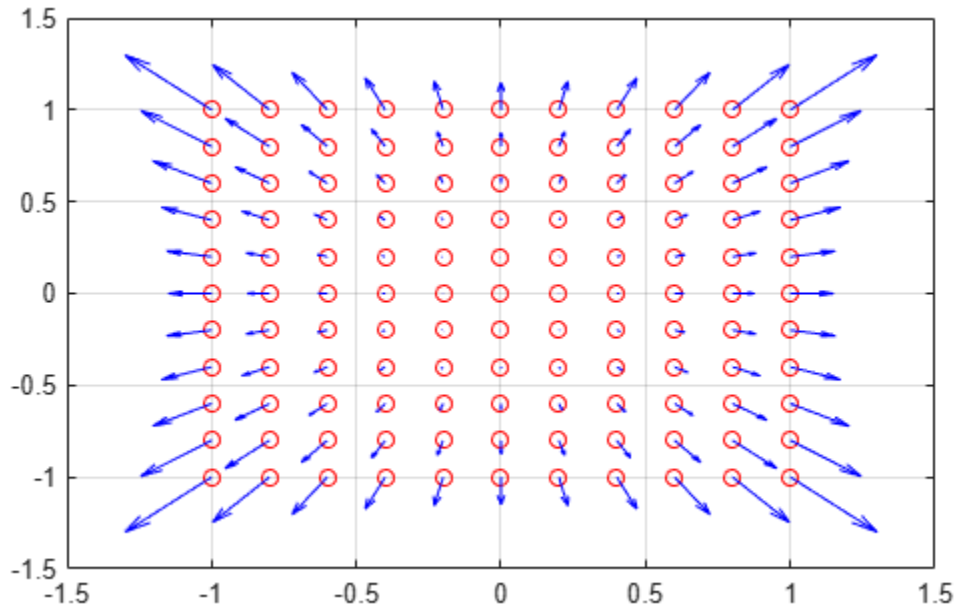
distortionX =

$$x \left( \frac{3x^2}{20} + \frac{3y^2}{20} + 1 \right)$$


distortionY =

$$y \left( \frac{3x^2}{20} + \frac{3y^2}{20} + 1 \right)$$

```



### Calculate Inverse Distortion Model

Given a camera's lens distortion coefficients and a set of distorted pixel locations  $(x_2, y_2)$ , we want to be able to calculate the undistorted pixel locations  $(x_1, y_1)$ . We will look at the specific case where all distortion coefficients are zero except for  $k_1$  which equals 0.2.

We begin by defining the distortion coefficients.

```
syms X Y positive
eq1 = X == distortionX
```

$$\text{eq1} = X = p_2 (3x^2 + y^2) + x (k_2 (x^2 + y^2)^2 + k_1 (x^2 + y^2) + 1) + 2p_1 xy$$

```
eq2 = Y == distortionY
```

$$\text{eq2} = Y = p_1 (x^2 + 3y^2) + y (k_2 (x^2 + y^2)^2 + k_1 (x^2 + y^2) + 1) + 2p_2 xy$$

We define the distortion equations for given distortion coefficients, and solve for the undistorted pixel locations  $(x_1, y_1)$ .

```
parameters = [k_1 k_2 p_1 p_2];
parameterValues = [0.2 0 0 0];
eq1 = expand(subs(eq1,parameters,parameterValues))
```

$$\text{eq1} = X = \frac{x^3}{5} + \frac{xy^2}{5} + x$$

```
eq2 = expand(subs(eq2,parameters,parameterValues))
```

```
eq2 =
```

$$Y = \frac{x^2 y}{5} + \frac{y^3}{5} + y$$

```
Result = solve([eq1, eq2],[x,y],MaxDegree=3,Real=true)
```

```
Result = struct with fields:
```

```
  x: (X*((5*Y^3)/(2*(X^2 + Y^2))) + ((25*Y^6)/(4*(X^2 + Y^2)^2) + (125*Y^6)/(27*(X^2 + Y^2)^3))
  y: ((5*Y^3)/(2*(X^2 + Y^2))) + ((25*Y^6)/(4*(X^2 + Y^2)^2) + (125*Y^6)/(27*(X^2 + Y^2)^3))^(1/3)
```

Since element 1 is the only real solution, we will extract that expression into its own variable.

```
[Result.x Result.y]
```

```
ans =
  (X σ1 / Y) σ1
```

where

$$\sigma_1 = \sigma_2 - \frac{5 Y^2}{3 (X^2 + Y^2) \sigma_2}$$

$$\sigma_2 = \left( \frac{5 Y^3}{2 (X^2 + Y^2)} + \sqrt{\frac{25 Y^6}{4 (X^2 + Y^2)^2} + \frac{125 Y^6}{27 (X^2 + Y^2)^3}} \right)^{1/3}$$

Now we have analytical expressions for the pixel locations X and Y which we can use to undistort our images.

### Function for Drawing the Lens Distortion

```
function plotLensDistortion(distortionX,distortionY,parameters,parameterValues)
% distortionX is the expression describing the distorted x coordinate
% distortionY is the expression describing the distorted y coordinate
% k1 and k2 are the radial distortion coefficients
% p1 and p2 are the tangential distortion coefficients

syms x y

% This is the grid spacing over the image
spacing = 0.2

% Inspect and parametrically substitute in the values for k_1 k_2 p_1 p_2
distortionX = subs(distortionX,parameters,parameterValues)
distortionY = subs(distortionY,parameters,parameterValues)

% Loop over the grid
for x_i = -1:spacing:1
    for y_j = -1:spacing:1

        % Compute the distorted location
        xout = subs(distortionX, {x,y}, {x_i,y_j});
        yout = subs(distortionY, {x,y}, {x_i,y_j});

        % Plot the original point
        plot(x_i,y_j,"ro")
```

```
        hold on

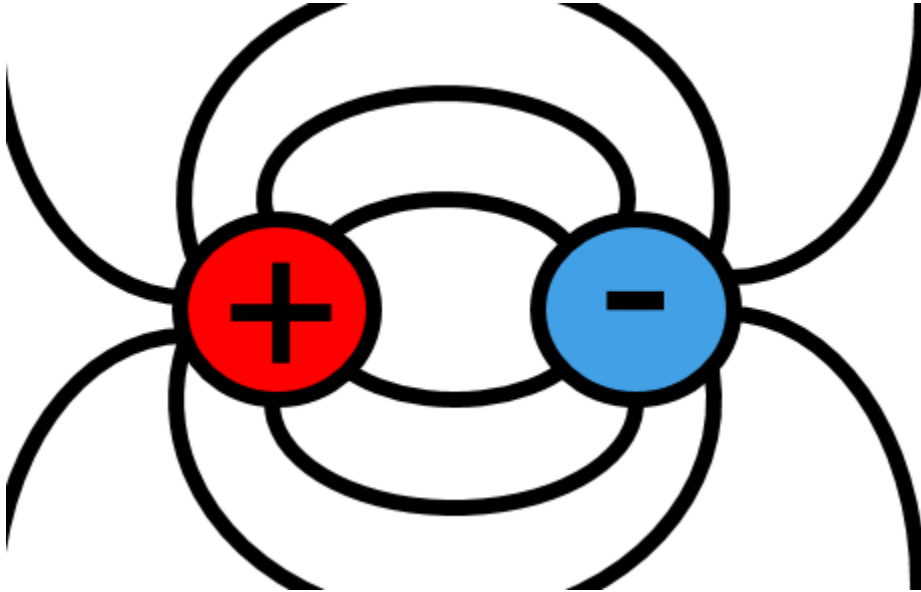
        % Draw the distortion direction with Quiver
        p1 = [x_i,y_j];           % First Point
        p2 = [xout,yout];         % Second Point
        dp = p2-p1;               % Difference
        quiver(p1(1),p1(2),dp(1),dp(2),AutoScale="off",MaxHeadSize=1,Color=[0 0 1])

    end
end
hold off
grid on

end
```

## Electric Dipole Moment and Radiation Power

This example finds the average radiation power of two attracting charges moving in an elliptical orbit (an *electric dipole*).



### Common Center of Mass

The two opposite charges,  $e_1$  and  $e_2$ , form an electric dipole. The masses of the charged particles are  $m_1$  and  $m_2$ , respectively. For the common center of mass  $m_1 r_1 + m_2 r_2 = 0$ , where  $r_1$  and  $r_2$  are distance vectors to the charged particles. The distance between charged particles is  $r = r_1 - r_2$ .

```
syms m1 m2 e1 e2 r1 r2 r
[r1,r2] = solve(m1*r1 + m2*r2 == 0, r == r1 - r2, r1, r2)
```

$$r_1 = \frac{m_2 r}{m_1 + m_2}$$

$$r_2 = -\frac{m_1 r}{m_1 + m_2}$$

### Dipole Moment

Find the dipole moment of this system:

```
d = e1*r1 + e2*r2;
simplify(d)
```

$$\text{ans} = \frac{r(e_1 m_2 - e_2 m_1)}{m_1 + m_2}$$

### Radiation Power per Unit of Time

According to the Larmor formula, the total power radiated in a unit of time is  $J = \frac{2}{3c^3} \ddot{d}^2$ , or, in terms of the distance between the charged particles,  $J = \frac{2}{3c^3} \frac{m_1 m_2}{m_1 + m_2} \left( \frac{e_1}{m_1} - \frac{e_2}{m_2} \right)^2 \ddot{r}^2$ . Here dot means a time derivative. Coulomb's law  $m \ddot{r} = -\frac{\alpha}{r^2}$  lets you find the values of acceleration  $\ddot{r}$  in terms of the reduced mass of the system,  $m = \frac{m_1 m_2}{m_1 + m_2}$ , and the product of the charges of the particles,  $\alpha = |e_1 e_2|$ .

```
alpha = sym('alpha');
syms m c
m = m1*m2/(m1 + m2);
r2 = -alpha/(m*r^2);

J = simplify(subs(2/(3*c^3)*d^2, r, r2))

J =
```

$$\frac{2 \alpha^2 (e_1 m_2 - e_2 m_1)^2}{3 c^3 m_1^2 m_2^2 r^4}$$

### Parameters of the Elliptical Orbit

The major semiaxis  $a$  and eccentricity  $e$  of an elliptical orbit are given by the following expressions, where  $E$  is the total orbital energy, and  $L = m r^2 \dot{\phi}$  is the angular momentum.

```
syms E L phi
a = alpha/(2*E)

a =
```

$$\frac{\alpha}{2 E}$$

```
eccentricity = sqrt(1-2*E*L^2/(m*alpha^2))

eccentricity =
```

$$\sqrt{1 - \frac{2 E L^2 (m_1 + m_2)}{\alpha^2 m_1 m_2}}$$

The equation of an elliptical orbit,  $1 + e \cos \phi = a(1 - e^2)/r$ , lets you express the distance  $r$  in terms of the angle  $\phi$ .

```
r = a*(1 - eccentricity^2)/(1 + eccentricity*cos(phi));
```

### Averaged Radiation Power

The average radiation power of two charged particles moving in an elliptical orbit is an integral of the radiation power over one full cycle of motion, normalized by the period of motion,  $J_{avg} = 1/T \int_0^T J dt$ .

The period of motions  $T$  is

```
T = 2*pi*sqrt(m*a^3/alpha);
```



Changing the integration variable  $t$  to  $\phi$ , you get the following result. Use the `simplify` function to get a shorter integration result. Here, use `subs` to evaluate  $J$ .

```
J = subs(J);
Javg = simplify(1/T*int(J*m*r^2/L, phi, 0, 2*pi))
```

$$J_{\text{avg}} = \frac{2\sqrt{2}\alpha^2(e_1 m_2 - e_2 m_1)^2(2EL^2 m_1 + 2EL^2 m_2 - 3\alpha^2 m_1 m_2)}{3L^5 c^3 (m_1 + m_2)^3 \sqrt{\frac{\alpha^2 m_1 m_2}{E^3 (m_1 + m_2)}}}$$

### If One Particle is Much Heavier Than the Other

Estimate the average radiation power of the electric dipole with one particle much heavier than the other,  $m_1 \gg m_2$ . For this, compute the limit of the expression for radiation power, assuming that  $m_1$  tends to infinity.

```
limJ = limit(Javg, m1, Inf);
simplify(limJ)
```

$$\text{ans} = \frac{2\sqrt{2}\alpha^2 e_2^2 (2EL^2 - 3\alpha^2 m_2)}{3L^5 c^3 \sqrt{\frac{\alpha^2 m_2}{E^3}}}$$

## Validate Simulink Model Using Symbolic Math Toolbox

This example shows how to model a bouncing ball, which is a classical hybrid dynamic system. This model includes both continuous dynamics and discrete transitions. It uses the Symbolic Math Toolbox™ to help explain some of the theory behind ODE solving in the “Simulation of Bouncing Ball” (Simulink).

### Assumptions

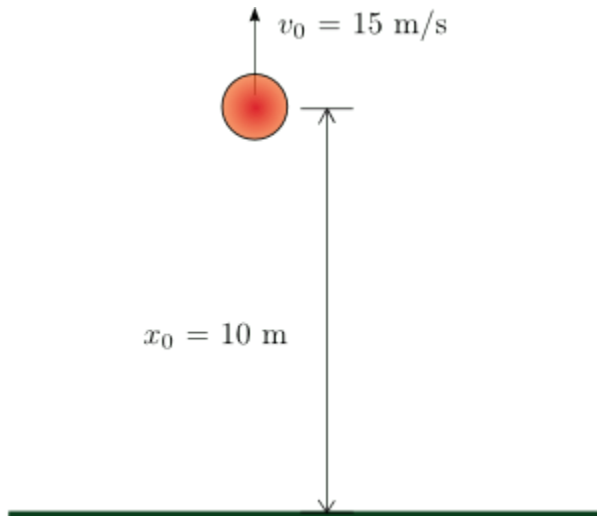
- The ball rebounds with no angle
- There is no drag
- Height at time  $t=0$  is 10 m
- Thrown with upwards velocity of 15 m/s

### Derivation

The coefficient of restitution is defined as

$$C_r = v_b - v_a / u_a - u_b$$

where  $v$  is the speed of object before impact and  $u$  is the speed after.



We split the second order differential equation

$$\frac{d^2 h}{dt^2} = -g$$

into

$$\frac{dh}{dt} = v \text{ discretized as } \frac{h(t + \Delta t) - h(t)}{\Delta t} = v(t)$$

and

$$\frac{dv}{dt} = -g \text{ discretized as } \frac{v(t + \Delta t) - v(t)}{\Delta t} = -g$$

We will use basic 1st order numerical integration using forward Euler.

$$h(t + \Delta t) = h(t) + v(t)\Delta t$$

$$v(t + \Delta t) = v(t) - g\Delta t$$

### Solve the Problem Analytically

Using the Symbolic Math Toolbox, we can approach the problem analytically. This allows us to solve certain problems more easily, such as determining precisely when the ball first hits the ground (below).

Declare our symbolic variables.

```
syms g t H(t) h_0 v_0
```

Split the second order differential equation  $\frac{d^2h}{dt^2} = -g$  into  $\frac{dh}{dt} = v$  and  $\frac{dv}{dt} = -g$ .

```
Dh = diff(H);
D2h = diff(H, 2) == g
```

$$D2h(t) = \frac{\partial^2}{\partial t^2} H(t) = g$$

Solve the ODE using `dsolve`:

```
eqn = dsolve(D2h, H(0) == h_0, Dh(0) == v_0)
```

$$\text{eqn} = \frac{g t^2}{2} + v_0 t + h_0$$

Parametrically explore the parabolic profile of motion using `subs`:

```
eqn = subs(eqn, [h_0, v_0, g], [10, 15, -9.81])
```

$$\text{eqn} = -\frac{981 t^2}{200} + 15 t + 10$$

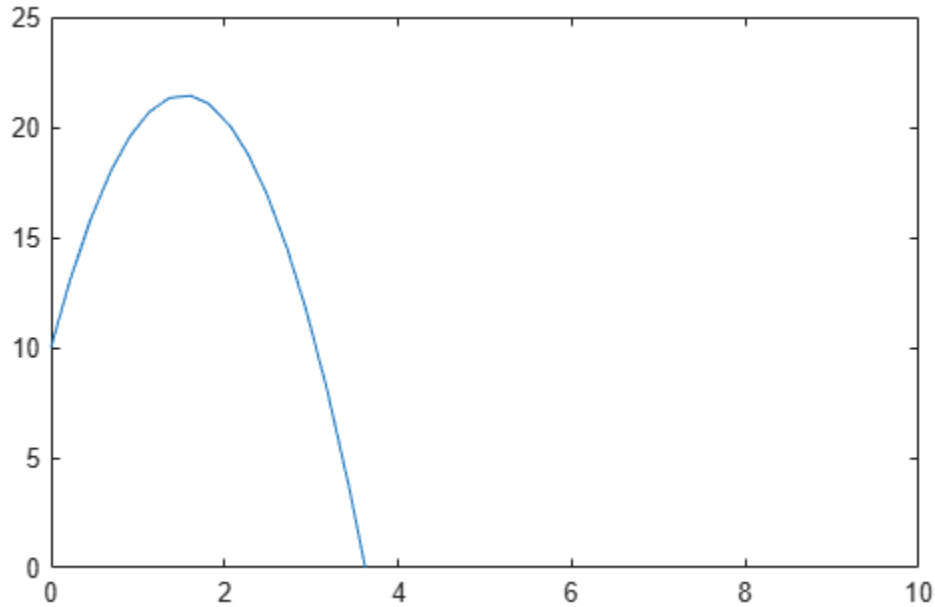
Find the time at which the ball hits the ground by solving for zero:

```
assume(t > 0)
tHit = solve(eqn == 0)
```

$$tHit = \frac{20\sqrt{5}\sqrt{26}}{109} + \frac{500}{327}$$

Visualize the solution:

```
fplot(eqn,[0 10])
ylim([0 25])
```



Format your exact results using variable precision arithmetic with `vpa`:

```
disp(['The ball with an initial height of 10m and velocity of 15m/s will hit the ground at ' char
```

The ball with an initial height of 10m and velocity of 15m/s will hit the ground at 3.621 seconds

### Solve the Problem Numerically

Setup Simulation Parameters

Properties of the ball

```
c_bounce = .9;           % Bouncing's coefficient of restitution; perfect restitution would be 1
```

Properties of the simulation

```
gravity = 9.8;           % Gravity's acceleration (m/s)
height_0 = 10;           % Initial height at time t=0 (m)
velocity_0=15;           % Initial velocity at time t=0 (m/s)
```

Declaring the simulation time step

```
dt = 0.05;               % Animation timestep (s)
t_final = 25;            % Simulate period (s)
t = 0:dt:t_final;        % Timespan
N = length(t);           % Number of iterations
```

Initialize simulation quantities

```
h=[];                    % Height of the ball as a function of time (m)
v=[];                    % Velocity of the ball (m/sec) as a function of time (m/s)
```

```
h(1)=height_0;
v(1)=velocity_0;
```

Simulate the bouncing ball (we will use basic 1st order numerical integration using forward Euler):

```
for i=1:N-1

    v(i+1)=v(i)-gravity*dt;
    h(i+1)=h(i)+v(i)*dt;

    % When the ball bounces (the height is less than 0),
    % reverse the velocity and recalculate the position.
    % Using the coefficient of restitution
    if h(i+1) < 0

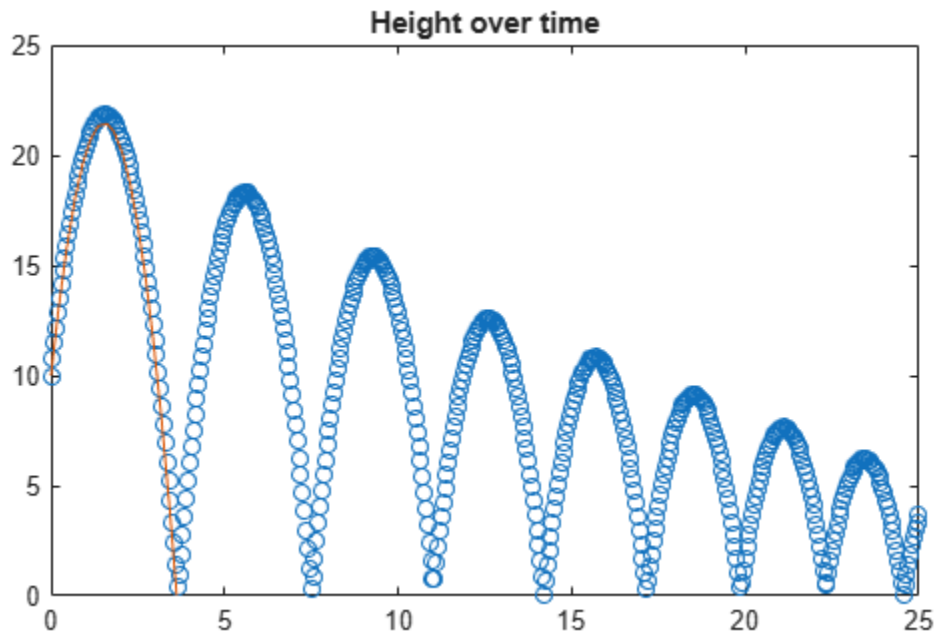
        v(i)=-v(i)*c_bounce;
        v(i+1)=v(i)-gravity*dt;
        h(i+1)=0+v(i)*dt;

    end

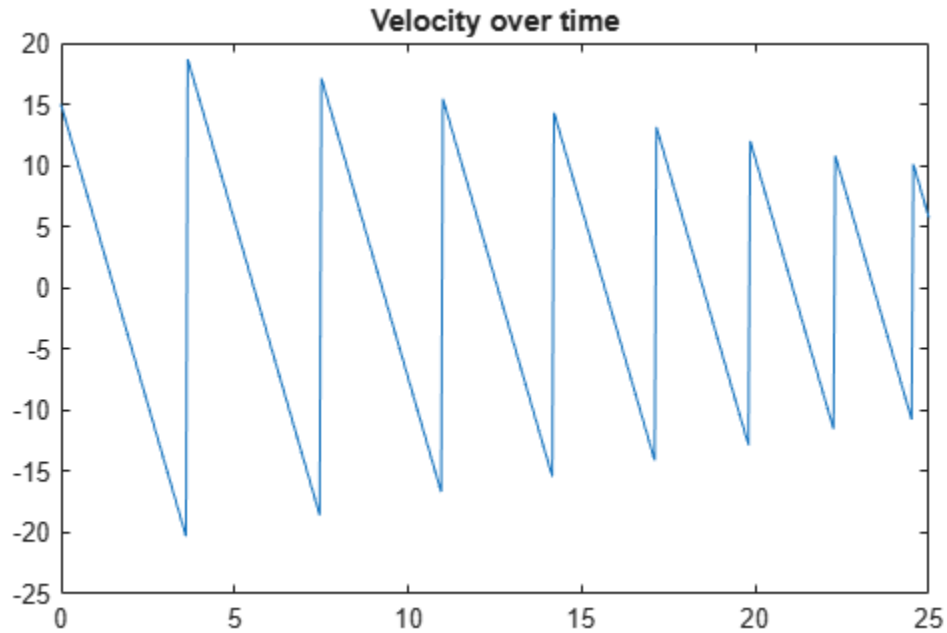
end
```

Visualize and validate the simulation

```
plot(t,h,'o')
hold on
fplot(eqn,[0 10])
title('Height over time')
ylim([0 25])
hold off
```



```
plot(t,v)
title('Velocity over time')
```



### Validate Numerics with Analytics

**Compare your analytical results with your numeric results.**

As a reminder the time of impact was:

```
disp(['The ball with an initial height of 10m and velocity of 15m/s will hit the ground at ' char
```

```
The ball with an initial height of 10m and velocity of 15m/s will hit the ground at 3.621 seconds
```

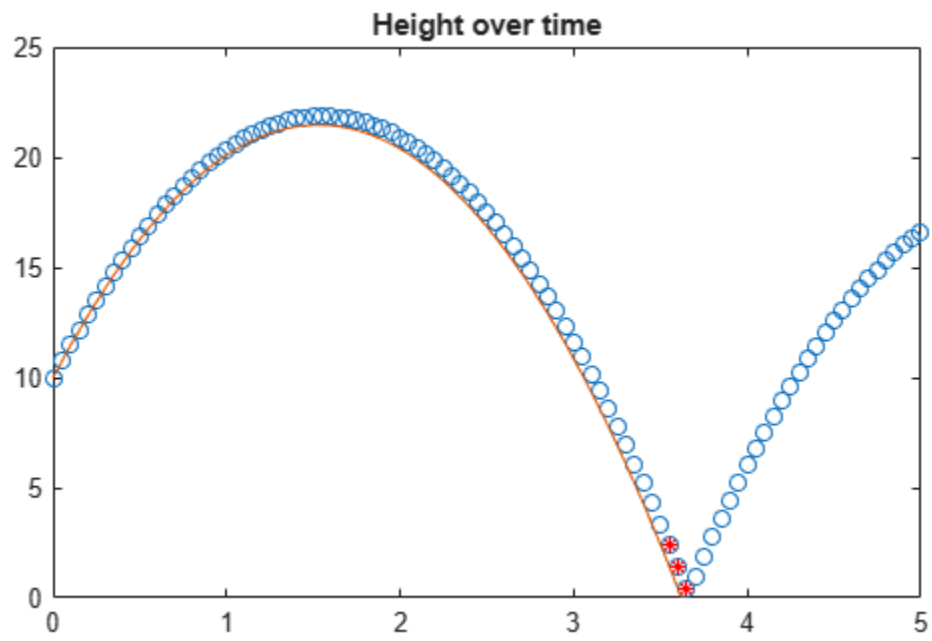
From the numerical simulation we can find the closest value in the simulation when  $h(t_i) \approx 0$

```
i = ceil(double(tHit/dt));
t([i-1 i i+1])
```

```
ans = 1×3
```

```
3.5500    3.6000    3.6500
```

```
plot(t,h,'o')
hold on
fplot(eqn,[0 10])
plot(t([i-1 i i+1 ]),h([i-1 i i+1 ]),'*R')
title('Height over time')
xlim([0 5])
ylim([0 25])
hold off
```







# Mathematics

---

- “Solve Algebraic Equations” on page 3-3
- “Solve System of Algebraic Equations” on page 3-7
- “Choose an Approach for Solving Equations Using solve Function” on page 3-16
- “Solve Equations Numerically” on page 3-28
- “Solve System of Linear Equations” on page 3-36
- “Select Numeric or Symbolic Solver” on page 3-39
- “Solve Parametric Equations in ReturnConditions Mode” on page 3-40
- “Solve Algebraic Equation Using Live Editor Task” on page 3-42
- “Solve Differential Equation” on page 3-49
- “Solve a System of Differential Equations” on page 3-53
- “Solve a Second-Order Differential Equation Numerically” on page 3-57
- “Solve Partial Differential Equation of Tsunami Model” on page 3-59
- “Solve Differential Algebraic Equations (DAEs)” on page 3-66
- “Solve Semilinear DAE System” on page 3-75
- “Solve DAEs Using Mass Matrix Solvers” on page 3-82
- “Analyze and Manipulate Differential Algebraic Equations” on page 3-87
- “Derive and Apply Inverse Kinematics to Two-Link Robot Arm” on page 3-94
- “Gauss-Laguerre Quadrature Evaluation Points and Weights” on page 3-101
- “Simulate a Stochastic Process Using the Feynman-Kac Formula” on page 3-111
- “The Black-Scholes Formula for Call Option Price” on page 3-118
- “Choose Function to Rearrange Expression” on page 3-123
- “Extract Numerators and Denominators of Rational Expressions” on page 3-131
- “Simplify Symbolic Expressions” on page 3-133
- “Simplify Symbolic Expressions Using Live Editor Task” on page 3-138
- “Substitute Parameters in Symbolic Solutions” on page 3-144
- “Substitute Elements in Symbolic Matrices” on page 3-146
- “Substitute Scalars with Matrices” on page 3-148
- “Substitute Values into Symbolic Expressions Using subs” on page 3-150
- “Abbreviate Common Terms in Long Expressions” on page 3-157
- “Harmonic Analysis of Transfer Function Output” on page 3-160
- “Explore Single-Period Asset Arbitrage” on page 3-166
- “Analytical Solutions of the Inverse Kinematics of a Humanoid Robot” on page 3-170
- “Differentiation” on page 3-181
- “Integration” on page 3-186
- “Taylor Series” on page 3-192

- “Fourier and Inverse Fourier Transforms” on page 3-194
- “Solve Differential Equations of RLC Circuit Using Laplace Transform” on page 3-198
- “Solve Difference Equations Using Z-Transform” on page 3-205
- “Symbolic Summation” on page 3-210
- “Padé Approximant” on page 3-212
- “Limits” on page 3-219
- “Find Asymptotes, Critical, and Inflection Points” on page 3-222
- “Functional Derivatives Tutorial” on page 3-226
- “Learn Calculus in the Live Editor” on page 3-231
- “Differentiation” on page 3-236
- “Integration” on page 3-243
- “Interactive Calculus in Live Editor” on page 3-247
- “Maxima, Minima, and Inflection Points” on page 3-249
- “Find Extremum of Multivariate Function and Its Approximation” on page 3-257
- “Padé Approximant of Time-Delay Input” on page 3-261
- “Heating of Finite Slab” on page 3-267
- “Solve Partial Differential Equation of Nonlinear Heat Transfer” on page 3-275
- “Derive Equations of Motion and Simulate Cart-Pole System” on page 3-281
- “Finite Element Formulation for Timoshenko Beam Problem” on page 3-303
- “Analyze Transfer Function of T-Coil Circuit” on page 3-324
- “Symbolic Matrix Computation” on page 3-334
- “Linear Algebraic Operations” on page 3-341
- “Basic Algebraic Operations” on page 3-345
- “Singular Value Decomposition” on page 3-347
- “Eigenvalues” on page 3-349
- “Jordan Canonical Form” on page 3-353
- “Eigenvalues of the Laplace Operator” on page 3-355
- “Hilbert Matrices and Their Inverses” on page 3-365
- “Markov Chain Analysis and Stationary Distribution” on page 3-369
- “Matrix Rotations and Transformations” on page 3-374
- “Clear Assumptions and Reset the Symbolic Engine” on page 3-379
- “Recognize and Avoid Round-Off Errors” on page 3-382
- “Increase Speed by Reducing Precision” on page 3-386
- “Prime Factorizations” on page 3-388
- “Handling Large Integers to Solve the 196 Problem” on page 3-391

## Solve Algebraic Equations

Symbolic Math Toolbox™ offers both symbolic and numeric equation solvers. This topic shows you how to solve an equation symbolically using the symbolic solver `solve`. To compare symbolic and numeric solvers, see “Select Numeric or Symbolic Solver” on page 3-39.

### Solve an Equation

If `eqn` is an equation, `solve(eqn,x)` solves `eqn` for the symbolic variable `x`.

Use the `==` operator to specify the familiar quadratic equation and solve it using `solve`.

```
syms a b c x
eqn = a*x^2 + b*x + c == 0;
solx = solve(eqn, x)
```

$$\text{solx} = \begin{pmatrix} -\frac{b + \sqrt{b^2 - 4ac}}{2a} \\ -\frac{b - \sqrt{b^2 - 4ac}}{2a} \end{pmatrix}$$

`solx` is a symbolic vector containing the two solutions of the quadratic equation. If the input `eqn` is an expression and not an equation, `solve` solves the equation `eqn == 0`.

To solve for a variable other than `x`, specify that variable instead. For example, solve `eqn` for `b`.

```
solb = solve(eqn,b)
```

$$\text{solb} = -\frac{ax^2 + c}{x}$$

If you do not specify a variable, `solve` uses `symvar` to select the variable to solve for. For example, `solve(eqn)` solves `eqn` for `x`.

### Return the Full Solution to an Equation

`solve` does not automatically return all solutions of an equation. Solve the equation `cos(x) == -sin(x)`. The `solve` function returns one of many solutions.

```
syms x
solx = solve(cos(x) == -sin(x), x)
```

$$\text{solx} = -\frac{\pi}{4}$$

To return all solutions along with the parameters in the solution and the conditions on the solution, set the `ReturnConditions` option to `true`. Solve the same equation for the full solution. Provide three output variables: for the solution to `x`, for the parameters in the solution, and for the conditions on the solution.

```
syms x
[solx,param,cond] = solve(cos(x) == -sin(x), x, ReturnConditions=true)
```

```
solx =
     $\pi k - \frac{\pi}{4}$ 

param = k

cond =  $k \in \mathbb{Z}$ 
```

`solx` contains the solution for  $x$ , which is  $\pi k - \pi/4$ . The `param` variable specifies the parameter in the solution, which is  $k$ . The `cond` variable specifies the condition  $k \in \mathbb{Z}$  on the solution, which means  $k$  must be an integer. Thus, `solve` returns a periodic solution starting at  $\pi/4$  which repeats at intervals of  $\pi*k$ , where  $k$  is an integer.

### Work with the Full Solution, Parameters, and Conditions Returned by solve

You can use the solutions, parameters, and conditions returned by `solve` to find solutions within an interval or under additional conditions.

To find values of  $x$  in the interval  $-2*\pi < x < 2*\pi$ , solve `solx` for  $k$  within that interval under the condition `cond`. Assume the condition `cond` using `assume`.

```
assume(cond)
solk = solve(-2*pi < solx, solx < 2*pi, param)
```

```
solk =
     $\begin{pmatrix} -1 \\ 0 \\ 1 \\ 2 \end{pmatrix}$ 
```

To find values of  $x$  corresponding to these values of  $k$ , use `subs` to substitute for  $k$  in `solx`.

```
xvalues = subs(solx, solk)
```

```
xvalues =
     $\begin{pmatrix} -\frac{5\pi}{4} \\ -\frac{\pi}{4} \\ \frac{3\pi}{4} \\ \frac{7\pi}{4} \end{pmatrix}$ 
```

To convert these symbolic values into numeric values for use in numeric calculations, use `vpa`.

```
xvalues = vpa(xvalues)
```

```
xvalues =
     $\begin{pmatrix} -3.9269908169872415480783042290994 \\ -0.78539816339744830961566084581988 \\ 2.3561944901923449288469825374596 \\ 5.4977871437821381673096259207391 \end{pmatrix}$ 
```

### Visualize and Plot Solutions Returned by solve

The previous sections used `solve` to solve the equation  $\cos(x) == -\sin(x)$ . The solution to this equation can be visualized using plotting functions such as `fplot` and `scatter`.

Plot both sides of equation  $\cos(x) == -\sin(x)$ .

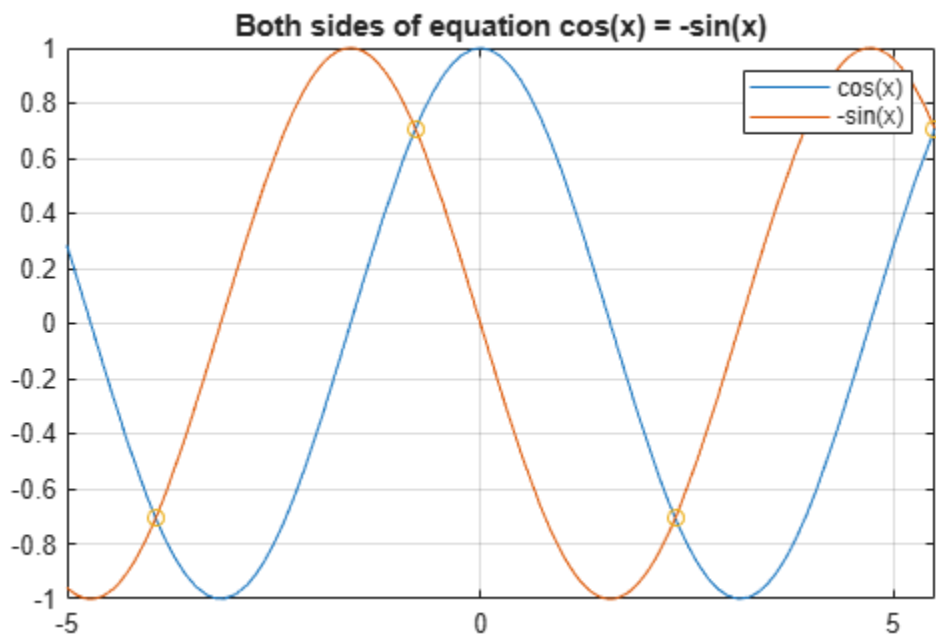
```
fplot(cos(x))
hold on
grid on
fplot(-sin(x))
title("Both sides of equation cos(x) = -sin(x)")
legend("cos(x)", "-sin(x)", AutoUpdate="off")
```

Calculate the values of the functions at the values of  $x$ , and superimpose the solutions as points using `scatter`.

```
yvalues = cos(xvalues)
```

```
yvalues =
    [-0.70710678118654752440084436210485
     0.70710678118654752440084436210485
    -0.70710678118654752440084436210485
     0.70710678118654752440084436210485]
```

```
scatter(xvalues, yvalues)
```



As expected, the solutions appear at the intersection of the two plots.

**Simplify Complicated Results and Improve Performance**

If results look complicated, `solve` is stuck, or if you want to improve performance, see, “Choose an Approach for Solving Equations Using `solve` Function” on page 3-16.

**See Also****Related Examples**

- “Solve System of Linear Equations” on page 3-36
- “Solve Differential Equation” on page 3-49
- “Solve Differential Algebraic Equations (DAEs)” on page 3-66

## Solve System of Algebraic Equations

This topic shows you how to solve a system of equations symbolically using Symbolic Math Toolbox™. This toolbox offers both numeric and symbolic equation solvers. For a comparison of numeric and symbolic solvers, see “Select Numeric or Symbolic Solver” on page 3-39.

### Handle the Output of solve

Suppose you have the system

$$x^2y^2 = 0$$

$$x - \frac{y}{2} = a,$$

and you want to solve for  $x$  and  $y$ . First, create the necessary symbolic objects.

```
syms x y a
```

There are several ways to address the output of `solve`. One way is to use a two-output call. The call returns the following.

```
[solx,soly] = solve(x^2*y^2 == 0, x-y/2 == a)
```

$$\text{solx} = \begin{pmatrix} a \\ 0 \end{pmatrix}$$

$$\text{soly} = \begin{pmatrix} 0 \\ -2a \end{pmatrix}$$

Modify the first equation to  $x^2y^2 = 1$ . The new system has more solutions. Four distinct solutions are produced.

```
[solx,soly] = solve(x^2*y^2 == 1, x-y/2 == a)
```

$$\text{solx} = \begin{pmatrix} \frac{a}{2} - \frac{\sqrt{a^2 - 2}}{2} \\ \frac{a}{2} - \frac{\sqrt{a^2 + 2}}{2} \\ \frac{a}{2} + \frac{\sqrt{a^2 - 2}}{2} \\ \frac{a}{2} + \frac{\sqrt{a^2 + 2}}{2} \end{pmatrix}$$

$$\text{soly} = \begin{pmatrix} -a - \sqrt{a^2 - 2} \\ -a - \sqrt{a^2 + 2} \\ \sqrt{a^2 - 2} - a \\ \sqrt{a^2 + 2} - a \end{pmatrix}$$

Since you did not specify the dependent variables, `solve` uses `symvar` to determine the variables.

This way of assigning output from `solve` is quite successful for “small” systems. For instance, if you have a 10-by-10 system of equations, typing the following is both awkward and time consuming.

```
[x1,x2,x3,x4,x5,x6,x7,x8,x9,x10] = solve(...)
```

To circumvent this difficulty, `solve` can return a structure whose fields are the solutions. For example, solve the system of equations  $u^2 - v^2 = a^2$ ,  $u + v = 1$ ,  $a^2 - 2a = 3$ . The solver returns its results enclosed in a structure.

```
syms u v a
S = solve(u^2 - v^2 == a^2, u + v == 1, a^2 - 2*a == 3)

S = struct with fields:
    a: [2×1 sym]
    u: [2×1 sym]
    v: [2×1 sym]
```

The solutions for `a` reside in the “`a-field`” of `S`.

```
S.a
```

```
ans =
    (-1)
     3
```

Similar comments apply to the solutions for `u` and `v`. The structure `S` can now be manipulated by the field and index to access a particular portion of the solution. For example, to examine the second solution, you can use the following statement to extract the second component of each field.

```
s2 = [S.a(2),S.u(2),S.v(2)]

s2 = (3 5 -4)
```

The following statement creates the solution matrix `M` whose rows comprise the distinct solutions of the system.

```
M = [S.a,S.u,S.v]

M =
    (-1 1 0)
     3 5 -4
```

Clear `solx` and `soly` for further use.

```
clear solx soly
```

### Solve a Linear System of Equations

Linear systems of equations can also be solved using matrix division. For example, solve this system.

```
clear u v x y
syms u v x y
eqns = [x + 2*y == u, 4*x + 5*y == v];
S = solve(eqns);
sol = [S.x;S.y]
```



```
sol =
    (2 v - 5 u)
    (3      3)
    (4 u - v)
    (3      3)

[A,b] = equationsToMatrix(eqns,x,y);
z = A\b

z =
    (2 v - 5 u)
    (3      3)
    (4 u - v)
    (3      3)
```

Thus, `sol` and `z` produce the same solution, although the results are assigned to different variables.

### Return the Full Solution of a System of Equations

`solve` does not automatically return all solutions of an equation. To return all solutions along with the parameters in the solution and the conditions on the solution, set the `ReturnConditions` option to `true`.

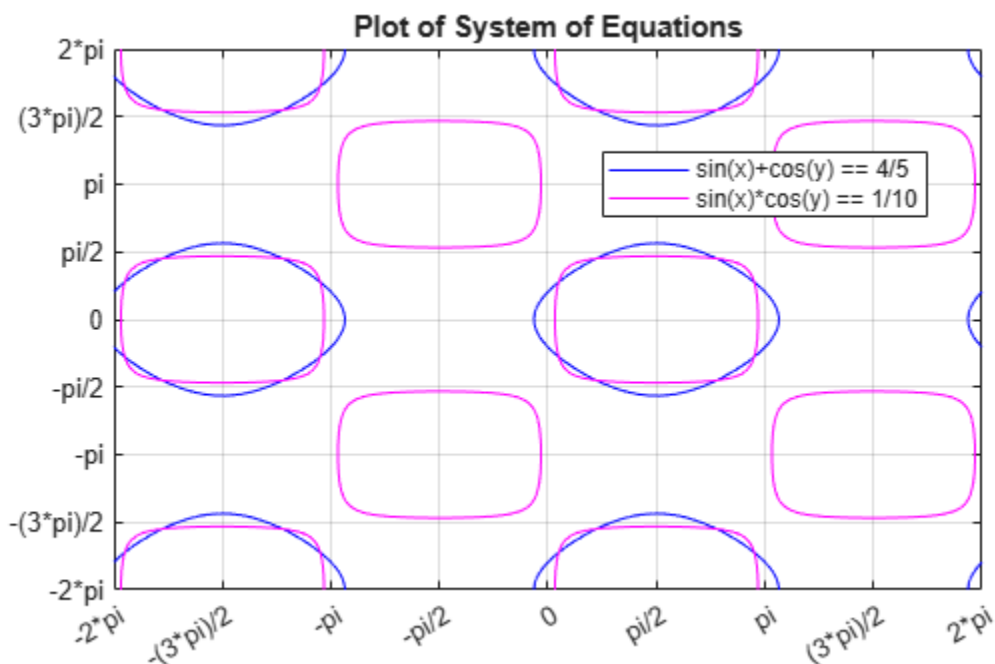
Consider the following system of equations:

$$\sin(x) + \cos(y) = \frac{4}{5}$$

$$\sin(x) \cos(y) = \frac{1}{10}$$

Visualize the system of equations using `fimplicit`. To set the x-axis and y-axis values in terms of  $\pi$ , get the axes handles using `axes` in `a`. Create the symbolic array `S` of the values  $-2\pi$  to  $2\pi$  at intervals of  $\pi/2$ . To set the ticks to `S`, use the `XTick` and `YTick` properties of `a`. To set the labels for the x-and y-axes, convert `S` to character vectors. Use `arrayfun` to apply `char` to every element of `S` to return `T`. Set the `XTickLabel` and `YTickLabel` properties of `a` to `T`.

```
syms x y
eqn1 = sin(x)+cos(y) == 4/5;
eqn2 = sin(x)*cos(y) == 1/10;
a = axes;
fimplicit(eqn1,[-2*pi 2*pi],"b");
hold on
grid on
fimplicit(eqn2,[-2*pi 2*pi],"m");
L = sym(-2*pi:pi/2:2*pi);
a.XTick = double(L);
a.YTick = double(L);
M = arrayfun(@char, L, UniformOutput=false);
a.XTickLabel = M;
a.YTickLabel = M;
title("Plot of System of Equations")
legend("sin(x)+cos(y) == 4/5","sin(x)*cos(y) == 1/10",...
    Location="best",AutoUpdate="off")
```



The solutions lie at the intersection of the two plots. This shows the system has repeated, periodic solutions. To solve this system of equations for the full solution set, use `solve` and set the `ReturnConditions` option to `true`.

```
S = solve(eqn1,eqn2,ReturnConditions=true)
```

```
S = struct with fields:
      x: [2×1 sym]
      y: [2×1 sym]
  parameters: [z    z1]
  conditions: [2×1 sym]
```

`solve` returns a structure `S` with the fields `S.x` for the solution to `x`, `S.y` for the solution to `y`, `S.parameters` for the parameters in the solution, and `S.conditions` for the conditions on the solution. Elements of the same index in `S.x`, `S.y`, and `S.conditions` form a solution. Thus, `S.x(1)`, `S.y(1)`, and `S.conditions(1)` form one solution to the system of equations. The parameters in `S.parameters` can appear in all solutions.

Index into `S` to return the solutions, parameters, and conditions.

`S.x`

```
ans =
    (z1)
    (z1)
```

`S.y`

```
ans =
    (z)
    (z)
```

S.parameters

ans = (z z<sub>1</sub>)

S.conditions

ans = 
$$\left( \left( \frac{z + \arccos(\sigma_3)}{2\pi} \in \mathbb{Z} \vee \frac{z - \arccos(\sigma_3)}{2\pi} \in \mathbb{Z} \right) \wedge \left( -\frac{\pi - z_1 + \sigma_1}{2\pi} \in \mathbb{Z} \vee \frac{z_1 + \sigma_1}{2\pi} \in \mathbb{Z} \right) \right) \\ \left( \left( \frac{z_1 - \pi + \arcsin(\sigma_3)}{2\pi} \in \mathbb{Z} \vee \frac{z_1 - \arcsin(\sigma_3)}{2\pi} \in \mathbb{Z} \right) \wedge \left( \frac{z + \sigma_2}{2\pi} \in \mathbb{Z} \vee \frac{z - \sigma_2}{2\pi} \in \mathbb{Z} \right) \right)$$

where

$$\sigma_1 = \arcsin\left(\frac{\sqrt{6}}{10} - \frac{2}{5}\right)$$

$$\sigma_2 = \arccos\left(\frac{2}{5} - \frac{\sqrt{6}}{10}\right)$$

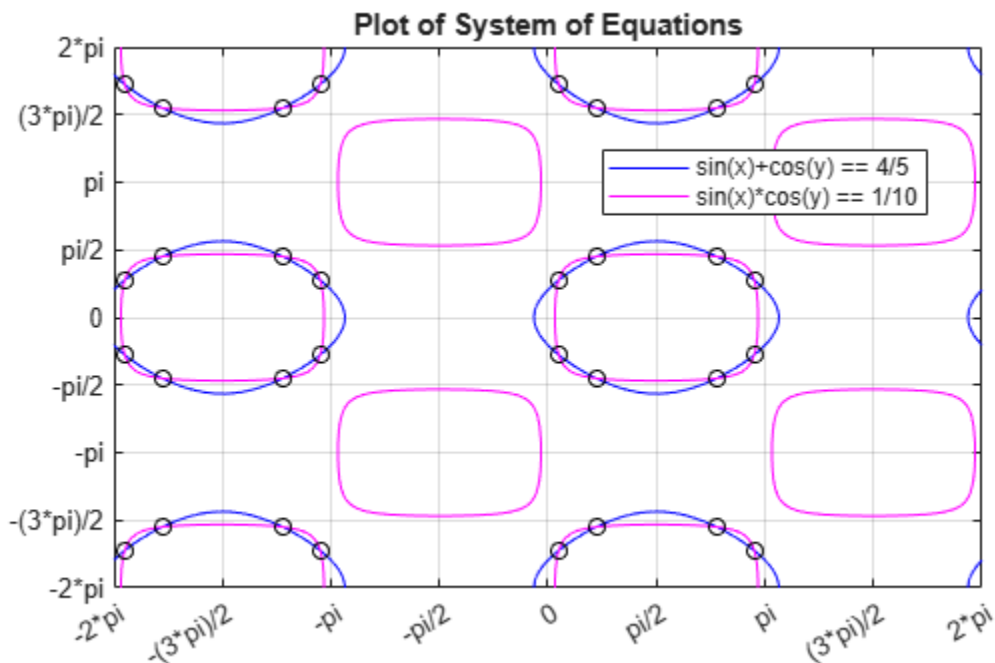
$$\sigma_3 = \frac{\sqrt{6}}{10} + \frac{2}{5}$$

### Solve a System of Equations Under Conditions

To solve the system of equations under conditions, specify the conditions in the input to solve.

Solve the system of equations considered above for x and y in the interval -2\*pi to 2\*pi. Overlay the solutions on the plot using scatter.

```
Srange = solve(eqn1, eqn2, -2*pi < x, x < 2*pi, -2*pi < y, y < 2*pi, ReturnConditions=true);
scatter(Srange.x, Srange.y, "k")
```



### Work with Solutions, Parameters, and Conditions Returned by solve

You can use the solutions, parameters, and conditions returned by `solve` to find solutions within an interval or under additional conditions. This section has the same goal as the previous section, to solve the system of equations within a search range, but with a different approach. Instead of placing conditions directly, it shows how to work with the parameters and conditions returned by `solve`.

For the full solution `S` of the system of equations, find values of `x` and `y` in the interval  $-2\pi$  to  $2\pi$  by solving the solutions `S.x` and `S.y` for the parameters `S.parameters` within that interval under the condition `S.conditions`.

Before solving for `x` and `y` in the interval, assume the conditions in `S.conditions` using `assume` so that the solutions returned satisfy the condition. Assume the conditions for the first solution.

```
assume(S.conditions(1))
```

Find the parameters in `S.x` and `S.y`.

```
paramx = intersect(symvar(S.x),S.parameters)
```

```
paramx = z1
```

```
paramy = intersect(symvar(S.y),S.parameters)
```

```
paramy = z
```

Solve the first solution of `x` for the parameter `paramx`.

```
solparamx(1,:) = solve(S.x(1) > -2*pi, S.x(1) < 2*pi, paramx)
```

```
solparamx =  


$$\left( \pi + \arcsin\left(\frac{\sqrt{6}}{10} - \frac{2}{5}\right) \arcsin\left(\frac{\sqrt{6}}{10} - \frac{2}{5}\right) - \pi - \arcsin\left(\frac{\sqrt{6}}{10} - \frac{2}{5}\right) - 2\pi - \arcsin\left(\frac{\sqrt{6}}{10} - \frac{2}{5}\right) \right)$$

```

Similarly, solve the first solution of `y` for `paramy`.

```
solparamy(1,:) = solve(S.y(1) > -2*pi, S.y(1) < 2*pi, paramy)
```

```
solparamy =  


$$\left( \arccos\left(\frac{\sqrt{6}}{10} + \frac{2}{5}\right) \arccos\left(\frac{\sqrt{6}}{10} + \frac{2}{5}\right) - 2\pi - \arccos\left(\frac{\sqrt{6}}{10} + \frac{2}{5}\right) 2\pi - \arccos\left(\frac{\sqrt{6}}{10} + \frac{2}{5}\right) \right)$$

```

Clear the assumptions set by `S.conditions(1)` using `assume`. Call `assumptions` to check that the assumptions are cleared.

```
assume(S.parameters,"clear")  
assumptions
```

```
ans =
```

```
Empty sym: 1-by-0
```

Assume the conditions for the second solution.

```
assume(S.conditions(2))
```

Solve the second solution to `x` and `y` for the parameters `paramx` and `paramy`.

```
solparamx(2,:) = solve(S.x(2) > -2*pi, S.x(2) < 2*pi, paramx)
```

```
solparamx =  


$$\begin{pmatrix} \pi + \sigma_1 & \sigma_1 - \pi & -\sigma_1 & -2\pi - \sigma_1 \\ \sigma_2 & \pi - \sigma_2 & \sigma_2 - 2\pi & -\pi - \sigma_2 \end{pmatrix}$$

```

where

$$\sigma_1 = \arcsin\left(\frac{\sqrt{6}}{10} - \frac{2}{5}\right)$$

$$\sigma_2 = \arcsin\left(\frac{\sqrt{6}}{10} + \frac{2}{5}\right)$$

```
solparamy(2,:) = solve(S.y(2) > -2*pi, S.y(2) < 2*pi, paramy)
```

```
solparamy =  


$$\begin{pmatrix} \sigma_2 & \sigma_2 - 2\pi & -\sigma_2 & 2\pi - \sigma_2 \\ \sigma_1 & \sigma_1 - 2\pi & -\sigma_1 & 2\pi - \sigma_1 \end{pmatrix}$$

```

where

$$\sigma_1 = \arccos\left(\frac{2}{5} - \frac{\sqrt{6}}{10}\right)$$

$$\sigma_2 = \arccos\left(\frac{\sqrt{6}}{10} + \frac{2}{5}\right)$$

The first rows of **paramx** and **paramy** form the first solution to the system of equations, and the second rows form the second solution.

To find the values of **x** and **y** for these values of **paramx** and **paramy**, use **subs** to substitute for **paramx** and **paramy** in **S.x** and **S.y**.

```
solx(1,:) = subs(S.x(1), paramx, solparamx(1,:));  
solx(2,:) = subs(S.x(2), paramx, solparamx(2,:))
```

```
solx =  


$$\begin{pmatrix} \pi + \sigma_1 & \sigma_1 - \pi & -\sigma_1 & -2\pi - \sigma_1 \\ \sigma_2 & \pi - \sigma_2 & \sigma_2 - 2\pi & -\pi - \sigma_2 \end{pmatrix}$$

```

where

$$\sigma_1 = \arcsin\left(\frac{\sqrt{6}}{10} - \frac{2}{5}\right)$$

$$\sigma_2 = \arcsin\left(\frac{\sqrt{6}}{10} + \frac{2}{5}\right)$$

```
soly(1,:) = subs(S.y(1), paramy, solparamy(1,:));  
soly(2,:) = subs(S.y(2), paramy, solparamy(2,:))
```

```
soly =
```

$$\begin{pmatrix} \sigma_2 & \sigma_2 - 2\pi & -\sigma_2 & 2\pi - \sigma_2 \\ \sigma_1 & \sigma_1 - 2\pi & -\sigma_1 & 2\pi - \sigma_1 \end{pmatrix}$$

where

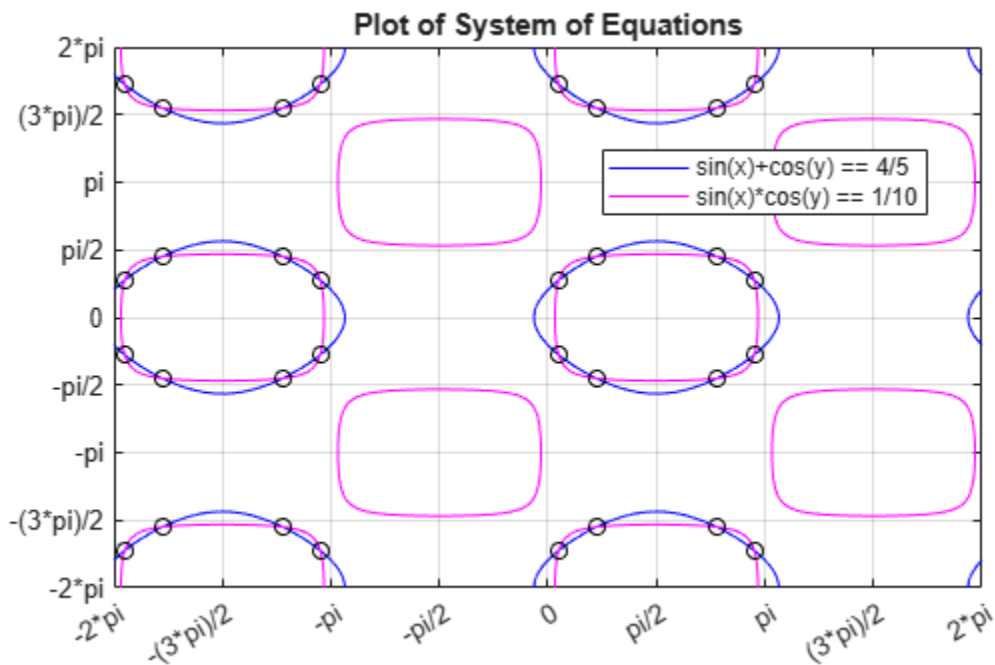
$$\sigma_1 = \arccos\left(\frac{2}{5} - \frac{\sqrt{6}}{10}\right)$$

$$\sigma_2 = \arccos\left(\frac{\sqrt{6}}{10} + \frac{2}{5}\right)$$

Note that `solx` and `soly` are the two sets of solutions to `x` and to `y`. The full sets of solutions to the system of equations are the two sets of points formed by all possible combinations of the values in `solx` and `soly`.

Plot these two sets of points using `scatter`. Overlay them on the plot of the equations. As expected, the solutions appear at the intersection of the plots of the two equations.

```
for i = 1:length(solx(1,:))
    for j = 1:length(soly(1,:))
        scatter(solx(1,i), soly(1,j), "k")
        scatter(solx(2,i), soly(2,j), "k")
    end
end
```



### Convert Symbolic Results to Numeric Values

Symbolic calculations provide exact accuracy, while numeric calculations are approximations. Despite this loss of accuracy, you might need to convert symbolic results to numeric approximations for use in numeric calculations. For a high-accuracy conversion, use variable-precision arithmetic provided by the `vpa` function. For standard accuracy and better performance, convert to double precision using `double`.

Use `vpa` to convert the symbolic solutions `solx` and `soly` to numeric form.

```
solxvpa = vpa(solx)
```

```
solxvpa =  
( 2.9859135500977407388300518406219 -3.2972717570818457380952349259371 0.15567910349205249963259154  
 0.70095651347102524787213653614929 2.4406361401187679905905068471302 -5.5822287937085612290531502  
)
```

```
solyvpa = vpa(soly)
```

```
solyvpa =  
( 0.86983981332387137135918515549046 -5.4133454938557151055661016110685 -0.8698398133238713713591851  
 1.4151172233028441195987301489821 -4.8680680838767423573265566175769 -1.4151172233028441195987301  
)
```

### **Simplify Complicated Results and Improve Performance**

If results look complicated, `solve` is stuck, or if you want to improve performance, see, “Choose an Approach for Solving Equations Using `solve` Function” on page 3-16.

## Choose an Approach for Solving Equations Using solve Function

Symbolic Math Toolbox™ provides the `solve` function for solving a mathematical equation or a system of equations. The `solve` function can return unique solutions, no solutions, or infinitely many solutions that include parameterizations and conditions on the unknowns. However, even when unique solutions exist, the toolbox might not express the solutions in a symbolic form explicitly. For these reasons, you can choose from several approaches to solving equations:

- Find a complete set of solutions, including parameters and conditions on solutions.
- Find numerical solutions with arbitrary precision.
- Find solutions for equations that include the definition of a symbolic function.
- Find other forms of solutions.

This example discusses these approaches. If you want to solve a system of ordinary differential equations, use `dsolve` instead.

### Approaches for Solving Equations

| When to Use                                                                                                                                                             | Approach                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| You want to find a complete set of solutions, including parameters and conditions on solutions. The system of equations to be solved can be consistent or inconsistent. | Use the <code>solve</code> function with <code>ReturnConditions</code> set to <code>true</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| You want to find numerical solutions with arbitrary precision when <code>solve</code> cannot find a symbolic solution.                                                  | Use the <code>vpasolve</code> function instead.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| You want to solve a system of equations that includes the definition of a symbolic function.                                                                            | Define the symbolic function outside the system of equations. Solve the system with necessary substitutions to rewrite the formula of the function.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| You want to find other forms of solutions.                                                                                                                              | Use the <code>solve</code> function with name-value arguments: <ul style="list-style-type: none"> <li>• To return only real solutions, specify <code>Real</code> as <code>true</code>.</li> <li>• To solve inequalities including parameters and conditions on solutions, specify <code>IgnorePr</code> as <code>true</code>.</li> <li>• To ignore the assumptions on the unknowns, specify <code>IgnorePr</code> as <code>true</code>.</li> <li>• To apply mathematical rules that are assumed to be valid, specify <code>IgnorePr</code> as <code>true</code>.</li> <li>• To find analytical solutions for polynomials with a degree less than 5, specify <code>MaxDegree</code> as a positive integer less than 5.</li> <li>• To return one solution out of many possible solutions, specify <code>PrincipalSolution</code> as <code>true</code>.</li> </ul> |

The `solve` function attempts to find symbolic solutions to your equations. Although specifying `ReturnConditions` as `true` is the most comprehensive approach, it is generally the slowest. To



increase computational speed, reduce the number of symbolic variables in your equations by substituting given values for some variables before using `solve`. For further speed improvements, if you are interested only in the solutions without parameterization, consider using the numeric solver `vpasolve` with a lower precision setting. To use another numeric solver without Symbolic Math Toolbox, you can convert symbolic expressions to MATLAB® functions by using `matlabFunction`. This conversion allows you to use the resulting function with other solvers in different MATLAB products, such as `fsolve` (Optimization Toolbox) in Optimization Toolbox™.

### Include Parameters and Conditions on Solutions

A system of equations consists of a specific number of equations and unknowns. In mathematics, these systems can be categorized as:

- Overdetermined system — More equations than unknowns
- Underdetermined system — Fewer equations than unknowns
- Exactly determined system — An equal number of equations and unknowns

However, the existence of solutions for these systems is not primarily dependent on the number of equations and unknowns. A system is considered **consistent** if at least one set of values for the unknowns satisfies all the equations. In this case, the solution might consist of specific values or involve free parameters. In contrast, a system is considered **inconsistent** if no set of values for the unknowns satisfies all the equations [1] on page 3-27.

### Consistent and Inconsistent Systems of Equations

A consistent system can involve free parameters in the solutions. To find solutions that include free parameters, specify the `ReturnConditions` name-value argument as `true` when using `solve`.

For example, consider the following system of two equations with three unknowns. This underdetermined system is consistent because a set of solutions for the three unknowns exists. The set of solutions is  $z = 1$  (as seen by subtracting the first equation from the second) and any values of  $x$  and  $y$  that satisfy  $x + y = 2$ .

```
syms x y z
eq1 = x + y + z == 3;
eq2 = x + y + 2*z == 4;
eqns = [eq1; eq2]
```

```
eqns =

$$\begin{pmatrix} x + y + z = 3 \\ x + y + 2z = 4 \end{pmatrix}$$

```

Solve this system by using `solve`. The solver returns one solution from the set of solutions. Here, the solver chooses the solution  $z = 1$ ,  $x = 2$ , and  $y = 0$ , which satisfies  $x + y = 2$ .

```
sols = solve(eqns,[x y z])

sols = struct with fields:
    x: 2
    y: 0
    z: 1
```

To find a complete set of solutions that includes free parameters, specify `ReturnConditions` as `true`. Here, the solver returns  $y = z1$  as a free parameter, which can take any complex value.

```
sols = solve(eqns,[x y z],ReturnConditions=true)
```

```
sols = struct with fields:
      x: 2 - z1
      y: z1
      z: 1
parameters: z1
conditions: symtrue
```

Next, consider the following system of equations with two unknowns. This overdetermined system is inconsistent because the last equation contradicts the other two equations (as seen by summing the first two equations).

```
syms x y
eq1 = x + y == 3;
eq2 = x + 2*y == 7;
eq3 = 2*x + 3*y == 11;
eqns = [eq1; eq2; eq3]
```

```
eqns =

$$\begin{pmatrix} x + y = 3 \\ x + 2y = 7 \\ 2x + 3y = 11 \end{pmatrix}$$

```

The `solve` function returns an empty solution for this system.

```
sols = solve(eqns)
```

```
sols = struct with fields:
      x: [0x1 sym]
      y: [0x1 sym]
```

Even if you specify `ReturnConditions` as `true`, `solve` returns an empty solution. These outputs confirm that the system is inconsistent and does not have a solution.

```
sols = solve(eqns,[x y],ReturnConditions=true)
```

```
sols = struct with fields:
      x: [0x1 sym]
      y: [0x1 sym]
parameters: [1x0 sym]
conditions: [0x1 sym]
```

For comparison, solving only two of the three equations yields a solution because no contradiction exists between the two equations.

```
sols = solve([eq1; eq3])
```

```
sols = struct with fields:
      x: -2
      y: 5
```

### Different Parameterizations When Specifying Different Variable Orders

When the solution of a system of equations involves a free parameter, specifying different variable orders to solve for when using `solve` affects the choice of the unknown to be parameterized.

For example, define a system of equations where the third equation is the result of subtracting the first equation from the second equation.

```
syms x y z
eq1 = z == 2*x + y;
eq2 = 2*z == 3*x - y;
eq3 = z == x - 2*y;
eqns = [eq1; eq2; eq3];
```

Solve for `x` and `y`. The solver returns the solutions in terms of the other unknown `z`.

```
sols = solve(eqns,x,y)

sols = struct with fields:
  x: (3*z)/5
  y: -z/5
```

Next, solve for `y` and `z`. The solver returns the solutions in terms of the other unknown `x`.

```
sols = solve(eqns,y,z)

sols = struct with fields:
  y: -x/3
  z: (5*x)/3
```

Now, if you solve for `x`, `y`, and `z`, then `solve` returns one solution from the set of solutions that satisfies the system. Here, `solve` returns `x = 0`, `y = 0`, and `z = 0`.

```
sols = solve(eqns,x,y,z)

sols = struct with fields:
  x: 0
  y: 0
  z: 0
```

To find a complete set of solutions that satisfies the system, specify `ReturnConditions` as `true`. Here, one unknown has to be parameterized, and the solver selects the last variable you specify in the second input argument to be parameterized. The solver parameterizes the unknown `z`, which can have any complex value.

```
sols = solve(eqns,[x y z],ReturnConditions=true)

sols = struct with fields:
  x: (3*z1)/5
  y: -z1/5
  z: z1
  parameters: z1
  conditions: symtrue
```

For comparison, change the order of the unknowns to solve for in the second input argument. Here, the solver parameterizes the unknown  $x$ .

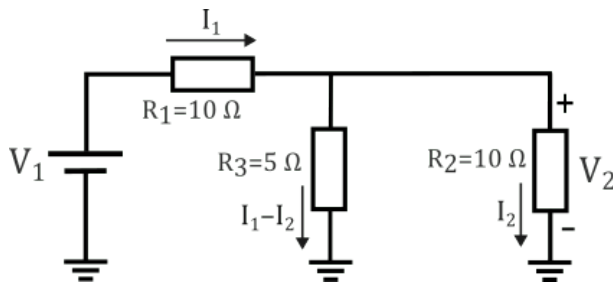
```
sols = solve(eqns,[y z x],ReturnConditions=true)

sols = struct with fields:
    y: -z1/3
    z: (5*z1)/3
    x: z1
    parameters: z1
    conditions: symtrue
```

### Redundant System of Equations

Consider the following circuit diagram. Define the variables that describe the circuit:

- $V_1$  is the input voltage.
- $V_2$  is the output voltage.
- $R_1$ ,  $R_2$ , and  $R_3$  are resistors.
- $I_1$  and  $I_2$  are currents that flow on specific paths of the circuit.



Apply Kirchhoff's voltage and current laws to obtain the system of equations that represents the circuit.

```
syms V1 V2 I1 I2 R1 R2 R3
eqn1 = -V1 + I1*R1 + (I1-I2)*R3 == 0;
eqn2 = -V1 + I1*R1 + I2*R2 == 0;
eqn3 = -(I1-I2)*R3 + I2*R2 == 0;
eqn4 = V2 == I2*R2;
eqns = [eqn1; eqn2; eqn3; eqn4]
```

```
eqns =

$$\begin{pmatrix} R_3 (I_1 - I_2) - V_1 + I_1 R_1 = 0 \\ I_1 R_1 - V_1 + I_2 R_2 = 0 \\ I_2 R_2 - R_3 (I_1 - I_2) = 0 \\ V_2 = I_2 R_2 \end{pmatrix}$$

```

This system of equations has one equation that is redundant—you can obtain the third equation by subtracting the second equation from the first equation.

For this reason, the system has only three linearly independent equations. You can solve for three unknowns of this system to obtain the result in terms of the other variables that describe the circuit. For example, solve for  $I_1$ ,  $I_2$ , and  $V_2$ . The result is in terms of  $V_1$ ,  $R_1$ ,  $R_2$ , and  $R_3$ .

```
sols = solve(eqns,I1,I2,V2)

sols = struct with fields:
    I1: (V1*(R2 + R3))/(R1*R2 + R1*R3 + R2*R3)
    I2: (R3*V1)/(R1*R2 + R1*R3 + R2*R3)
    V2: (R2*R3*V1)/(R1*R2 + R1*R3 + R2*R3)
```

You can also solve for I1, I2, and R1. The result is in terms of V1, V2, R2, and R3.

```
sols = solve(eqns,I1,I2,R1)

sols = struct with fields:
    I1: (V2*(R2 + R3))/(R2*R3)
    I2: V2/R2
    R1: (R2*R3*V1 - R2*R3*V2)/(R2*V2 + R3*V2)
```

If you solve for four unknowns from this system of four equations, where one equation is redundant without specifying `ReturnConditions` as `true`, then the solver chooses one solution from the set of solutions that satisfies the system. For example, solving for I1, I2, V1, and V2 results in a solution of 0 for these unknowns.

```
sols = solve(eqns,I1,I2,V1,V2)

sols = struct with fields:
    I1: 0
    I2: 0
    V1: 0
    V2: 0
```

For comparison, if you solve for I1, I2, V1, and R2, then the solver chooses a solution where R2 is 1.

```
sols = solve(eqns,I1,I2,V1,R2)

sols = struct with fields:
    I1: (V2 + R3*V2)/R3
    I2: V2
    V1: (R1*V2 + R3*V2 + R1*R3*V2)/R3
    R2: 1
```

To find a complete set of solutions that satisfies the system, along with the free parameters and conditions on the solutions, specify `ReturnConditions` as `true`. Here, solving for I1, I2, V1, and V2 results in a set of solutions where the solver parameterizes the variable V2.

```
sols = solve(eqns,I1,I2,V1,V2,ReturnConditions=true)

sols = struct with fields:
    I1: (z*(R2 + R3))/(R2*R3)
    I2: z/R2
    V1: (z*(R1*R2 + R1*R3 + R2*R3))/(R2*R3)
    V2: z
    parameters: z
    conditions: R1 + R3 ~= 0 & R2 ~= 0 & R3 ~= 0 & R1*R2 + R1*R3 + R2*R3 ~= 0
```

However, if your system of equations is more complicated and involves more unknowns, the solver can be slow, especially if `ReturnConditions` is set to `true`. In such cases, if you know the values of

the other unknowns, you can substitute these values before using `solve` to speed up the solver computation.

```
params.R1 = 10;
params.R2 = 10;
params.R3 = 5;
eqnSubbed = vpa(subs(eqns,params));
sols = solve(eqnSubbed,I1,I2,V1,V2,ReturnConditions=true)

sols = struct with fields:
    I1: 0.3*z
    I2: 0.1*z
    V1: 4.0*z
    V2: z
    parameters: z
    conditions: symtrue
```

### Find Numerical Solutions

If `solve` cannot find an explicit solution, it can issue a warning and return an empty solution. To overcome this issue, specify the appropriate number of unknowns to solve for to make the system of equations consistent. You can also use `vpsolve` to find numerical solutions within specific digits of precision.

### If `solve` Cannot Find Explicit Solution

Define a system of equations that involve trigonometric functions.

```
syms x y z
eqns = [x^2*z*(y-1) == 0; sin(sqrt(3)*x) == z; cos(x) == 1]
```

```
eqns =

$$\begin{cases} x^2 z (y - 1) = 0 \\ \sin(\sqrt{3} x) = z \\ \cos(x) = 1 \end{cases}$$

```

Solve for only `x`. The solver issues a warning and returns an empty solution.

```
sols = solve(eqns,x)
```

Warning: Unable to find explicit solution. For options, see [```
sols =
```](matlab:web(fullfile(docroot</a></p></div><div data-bbox=)

```
Empty sym: 0-by-1
```

Instead, specify `x`, `y`, and `z` as the unknowns to solve for. Here, `solve` returns two valid solutions.

```
sols = solve(eqns,[x y z])
```

```
sols = struct with fields:
    x: [2x1 sym]
    y: [2x1 sym]
    z: [2x1 sym]
```

`sols.x`

`ans =`  

$$\begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

`sols.y`

`ans =`  

$$\begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

`sols.z`

`ans =`  

$$\begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

To find the complete set of solutions with parameterization, specify `ReturnConditions` as `true`.

```
sols = solve(eqns,[x y z],ReturnConditions=true)
```

```
sols = struct with fields:
      x: [2×1 sym]
      y: [2×1 sym]
      z: [2×1 sym]
  parameters: [k    z1]
  conditions: [2×1 sym]
```

`sols.x`

`ans =`  

$$\begin{pmatrix} 2\pi k \\ 0 \end{pmatrix}$$

`sols.y`

`ans =`  

$$\begin{pmatrix} 1 \\ z_1 \end{pmatrix}$$

`sols.z`

`ans =`  

$$\begin{pmatrix} \sin(2\pi\sqrt{3}k) \\ 0 \end{pmatrix}$$

`sols.conditions`

`ans =`  

$$\begin{pmatrix} k \in \mathbb{Z} \\ \text{symtrue} \end{pmatrix}$$

If round-off errors are not a significant issue, you can also use `vpasolve` to find a numerical solution. By default, `vpasolve` uses 32 digits of precision. For this example, specify 20 digits of precision. Solve for `x`, `y`, and `z` by specifying an initial guess of 0 for all these variables. Here, the numerical solution is identical to one of the symbolic solutions, where `x = 0`, `y = 0`, and `z = 0`.

```

digits(20)
sols = vpsolve(eqns,[x y z],0)

sols = struct with fields:
    x: 0
    y: 0
    z: 0

```

Specify the initial guess as 1. Here, `vpsolve` returns an approximate solution. This solution satisfies the system of equations within the 20 digits of precision (as seen by substituting the solution into the system of equations).

```

sols = vpsolve(eqns,[x y z],1)

sols = struct with fields:
    x: 0.000000000000000035116234294122756747
    y: 1.0
    z: 0.000000000000000060823101967913224505

```

```
vpa(subs(eqns,sols))
```

```

ans =
    0.0 = 0.0
    0.000000000000000060823101967913224505 = 0.000000000000000060823101967913224505
    1.0 = 1.0

```

Restore the value of `digits` to 32 for further calculations.

```
digits(32)
```

### If `solve` Returns Conditions That Restate the System of Equations

Define the probability density function of the gamma distribution with parameters  $\alpha$  and  $\beta$ .

```

syms x alpha beta positive
pdf = (beta^alpha)/gamma(alpha)*x^(alpha-1)*exp(-beta*x)

```

```

pdf =
    
$$\frac{\beta^\alpha x^{\alpha-1} e^{-\beta x}}{\Gamma(\alpha)}$$


```

Find the mean of this distribution and the cumulative distribution function.

```

syms xc
integratex = int(x*pdf,x,0,xc);
mean = simplify(limit(integratex,xc,Inf,"left"))

```

```

mean =
    
$$\frac{\alpha}{\beta}$$


```

```
cdf(xc) = int(pdf,x,0,xc)
```

```

cdf(xc) =
    
$$1 - \frac{\Gamma(\alpha, \beta xc)}{\Gamma(\alpha)}$$


```



Define two equations that specify the mean of this distribution as 7 and the cumulative distribution function as 0.9 for  $0 < x \leq 10$ .

```
eq1 = mean == 7
```

```
eq1 =  
     $\frac{\alpha}{\beta} = 7$ 
```

```
eq2 = cdf(10) == 0.9
```

```
eq2 =  
     $1 - \frac{\Gamma(\alpha, 10\beta)}{\Gamma(\alpha)} = \frac{9}{10}$ 
```

Solve for the parameters  $\alpha$  and  $\beta$  using `solve`. Here, `solve` issues a warning and returns parameterized solutions.

```
sols = solve([eq1; eq2],[alpha beta])
```

Warning: Solutions are parameterized by the symbols: [z, z1], z1. To include parameters and conditions

Warning: Solutions are only valid under certain conditions. To include parameters and conditions

```
sols = struct with fields:  
    alpha: z  
    beta: z1
```

Include parameters and conditions on the solutions by specifying `ReturnConditions` as `true`. Here, the conditions on the parameterized solution are a restatement of the equations to be solved. In other words, `solve` cannot find the explicit symbolic solutions.

```
sols = solve([eq1; eq2],[alpha beta],ReturnConditions=true)
```

```
sols = struct with fields:  
    alpha: z  
    beta: z1  
    parameters: [z    z1]  
    conditions: 1/10 - igamma(z, 10*z1)/gamma(z) == 0 & z/z1 - 7 == 0 & 0 < z & 0 < z1
```

Instead, use `vpasolve` to find a numerical solution for the two equations. To find nonnegative solutions, set the search interval as `[0 Inf]` for both  $\alpha$  and  $\beta$ .

```
sols = vpasolve([eq1; eq2],[alpha beta],[0 Inf; 0 Inf])
```

```
sols = struct with fields:  
    alpha: 9.6391484249254912862751301540044  
    beta: 1.3770212035607844694678757362863
```

### If solve Cannot Find Symbolic Solution and Returns Numerical Solution Instead

The `solve` function attempts to find symbolic solutions to your equations. If `solve` cannot symbolically solve an equation, it tries to find a numerical solution using `vpasolve`. By default, the `vpasolve` function returns the first solution found.

Solve the equation  $\sin(x) = x^2 - 1$ . `solve` issues a warning and returns a numeric solution because it cannot find a symbolic solution.

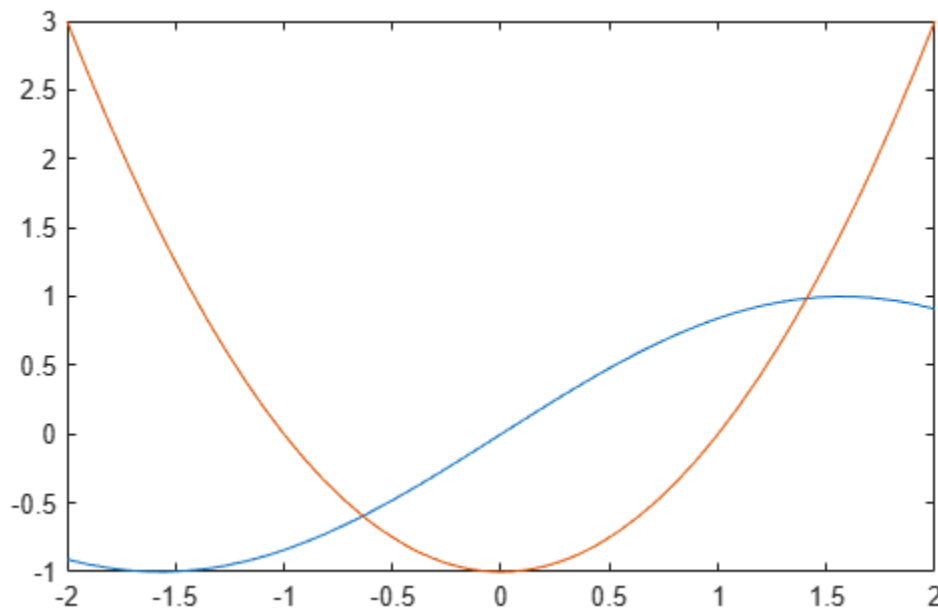
```
syms x
eqn = sin(x) == x^2 - 1;
sol = solve(eqn,x)
```

Warning: Unable to solve symbolically. Returning a numeric solution using `<a href="matlab:web(fu`

```
sol = -0.63673265080528201088799090383828
```

Plot the left and right sides of the equation. The plot shows that the equation also has a positive solution.

```
fplot([lhs(eqn) rhs(eqn)],[-2 2])
```



Find the other solution by directly calling the numeric solver `vpasolve` and specifying the search interval as 0 to 2.

```
sol2 = vpasolve(eqn,x,[0 2])
```

```
sol2 = 1.4096240040025962492355939705895
```

### Define Symbolic Function Outside of Equations

When solving a system of equations that involves symbolic functions, `solve` might return an empty solution. For example, the following equations define the formula for a function  $f(x)$  and the value of the function at  $f(1)$ .

```
syms f(x) a
eq1 = f(x) == 3*x + a;
eq2 = f(1) == 5;
eqns = [eq1; eq2]
```

```
eqns =
    (f(x) = a + 3 x)
    (f(1) = 5)
```

If you use `solve` to find the value of  $a$ , then `solve` returns an empty solution.

```
asol = solve(eqns,a)
```

```
asol =
```

```
Empty sym: 0-by-1
```

Instead, define the formula for the symbolic function  $f(x)$  outside the system of equations. Then, solve for  $a$  by using the evaluated value of  $f(x)$  at 1.

```
syms f(x) a
f(x) = 3*x + a;
asol = solve(f(1) == 5)

asol = 2
```

Once you have the solution for  $a$ , you can substitute it into the original formula.

```
fsol = subs(f(x),a,asol)

fsol = 3 x + 2
```

## References

[1] "Consistent and Inconsistent Equations." In *Wikipedia*, September 3, 2024. [https://en.wikipedia.org/w/index.php?title=Consistent\\_and\\_inconsistent\\_equations&oldid=1243721077](https://en.wikipedia.org/w/index.php?title=Consistent_and_inconsistent_equations&oldid=1243721077).

[2] "Overdetermined System." In *Wikipedia*, July 22, 2024. [https://en.wikipedia.org/w/index.php?title=Overdetermined\\_system&oldid=1235933306](https://en.wikipedia.org/w/index.php?title=Overdetermined_system&oldid=1235933306).

[3] "Underdetermined System." In *Wikipedia*, August 1, 2024. [https://en.wikipedia.org/w/index.php?title=Underdetermined\\_system&oldid=1237892077](https://en.wikipedia.org/w/index.php?title=Underdetermined_system&oldid=1237892077).

## See Also

### Functions

`solve` | `vpasolve` | `dsolve` | `linsolve`

## Related Examples

- "Solve Equations Numerically" on page 3-28
- "Solve System of Linear Equations" on page 3-36
- "Select Numeric or Symbolic Solver" on page 3-39
- "Solve Parametric Equations in ReturnConditions Mode" on page 3-40

## Solve Equations Numerically

Symbolic Math Toolbox™ offers both numeric and symbolic equation solvers. For a comparison of numeric and symbolic solvers, see “Select Numeric or Symbolic Solver” on page 3-39. An equation or a system of equations can have multiple solutions. To find these solutions numerically, use the function `vpasolve`. For polynomial equations, `vpasolve` returns all solutions. For nonpolynomial equations, `vpasolve` returns the first solution it finds. These examples show you how to use `vpasolve` to find solutions to both polynomial and nonpolynomial equations, and how to obtain these solutions to arbitrary precision.

### Find All Roots of a Polynomial Function

Use `vpasolve` to find all the solutions to the function  $f(x) = 6x^7 - 2x^6 + 3x^3 - 8$ .

```
syms f(x)
f(x) = 6*x^7-2*x^6+3*x^3-8;
sol = vpasolve(f)

sol =
```

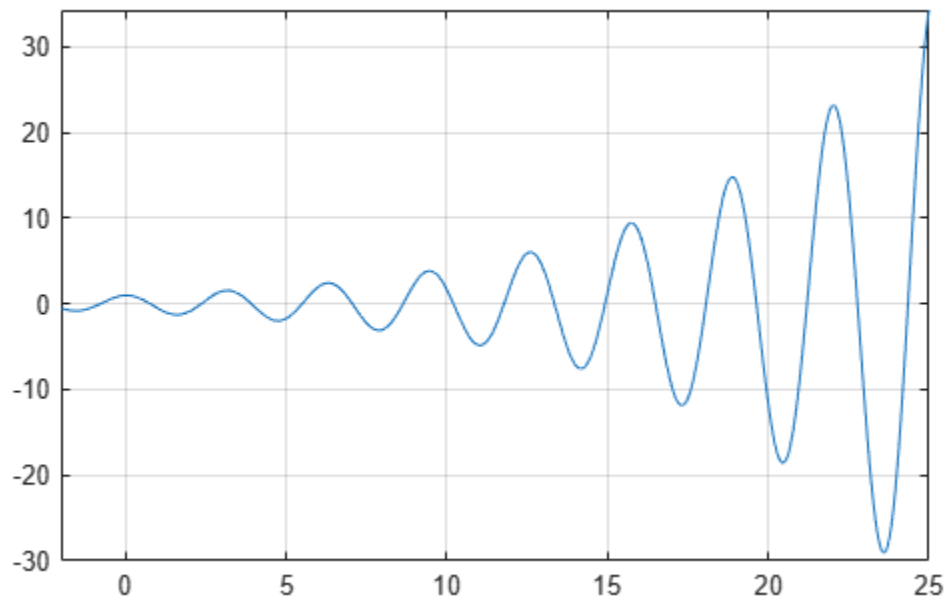
$$\begin{pmatrix} 1.0240240759053702941448316563337 \\ -0.88080620051762149639205672298326 + 0.50434058840127584376331806592405 i \\ -0.88080620051762149639205672298326 - 0.50434058840127584376331806592405 i \\ -0.22974795226118163963098570610724 + 0.96774615576744031073999010695171 i \\ -0.22974795226118163963098570610724 - 0.96774615576744031073999010695171 i \\ 0.7652087814927846556172932675903 + 0.83187331431049713218367239317121 i \\ 0.7652087814927846556172932675903 - 0.83187331431049713218367239317121 i \end{pmatrix}$$

`vpasolve` returns seven roots of the function, as expected, because the function is a polynomial of degree seven.

### Find Zeros of a Nonpolynomial Function Using Search Ranges and Starting Points

A plot of the function  $f(x) = e^{(x/7)}\cos(2x)$  reveals periodic zeros, with increasing slopes at the zero points as  $x$  increases.

```
syms x
h = fplot(exp(x/7)*cos(2*x), [-2 25]);
grid on
```



Use `vpasolve` to find a zero of the function `f`. Note that `vpasolve` returns only one solution of a nonpolynomial equation, even if multiple solutions exist. On repeated calls, `vpasolve` returns the same result.

```
f = exp(x/7)*cos(2*x);
for k = 1:3
    vpasolve(f,x)
end

ans = -7.0685834705770347865409476123789
ans = -7.0685834705770347865409476123789
ans = -7.0685834705770347865409476123789
```

To find multiple solutions, set the option `'Random'` to `true`. This makes `vpasolve` choose starting points randomly. For information on the algorithm that chooses random starting points, see “Algorithms” on page 6-1646 on the `vpasolve` page.

```
for k = 1:3
    vpasolve(f,x,'Random',true)
end

ans = -226.98006922186256147892598444194
ans = 98.174770424681038701957605727484
ans = 52.621676947629036744249276669932
```

To find a zero close to  $x = 10$ , set the starting point to 10.

```
vpasolve(f,x,10)

ans = 10.210176124166828025003590995658
```

To find a zero close to  $x = 1000$ , set the starting point to 1000.

```
vpasolve(f,x,1000)
```

```
ans = 999.8118620049516981407362567287
```

To find a zero in the range  $15 \leq x \leq 25$ , set the search range to [15 25].

```
vpasolve(f,x,[15 25])
```

```
ans = 21.205750411731104359622842837137
```

To find multiple zeros in the range [15 25], you cannot call `vpasolve` repeatedly because it returns the same result on each call, as previously shown. Instead, set the search range and set 'Random' to true.

```
for k = 1:3
    vpasolve(f,x,[15 25],'Random',true)
end
```

```
ans = 21.205750411731104359622842837137
```

```
ans = 21.205750411731104359622842837137
```

```
ans = 16.493361431346414501928877762217
```

Because 'Random' selects starting points randomly, the same solution might be found on successive calls.

### Find All Zeros in a Specified Search Range

Create a function `findzeros` to systematically find all zeros for  $f$  in a given search range, within a specified error tolerance. The function starts with the input search range and calls `vpasolve` to find a zero. Then, it splits the search range into two around the zero value and recursively calls itself with the new search ranges as inputs to find more zeros.

The function is explained section by section here.

Declare the function with the three inputs and one output. The first input is the function, the second input is the range, and the optional third input allows you to specify the error between a zero and the higher and lower bounds generated from it.

```
function sol = findzeros(f,range,err)
```

If you do not specify the optional argument for error tolerance, `findzeros` sets `err` to 0.001.

```
if nargin < 2
    err = 1e-3;
end
```

Find a zero in the search range using `vpasolve`.

```
sol = vpasolve(f,range);
```

If `vpasolve` does not find a zero, exit.

```
if isempty(sol)
    return
```

If `vpasolve` finds a zero, split the search range into two search ranges above and below the zero.

```
else
    lowLimit = sol-err;
    highLimit = sol+err;
```

Call `findzeros` with the lower search range. If `findzeros` returns zeros, copy the values into the solution array and sort them.

```
temp = findzeros(f,[range(1) lowLimit],1);
if ~isempty(temp)
    sol = sort([sol temp]);
end
```

Call `findzeros` with the higher search range. If `findzeros` returns zeros, copy the values into the solution array and sort them.

```
temp = findzeros(f,[highLimit range(2)],1);
if ~isempty(temp)
    sol = sort([sol temp]);
end
return
end
end
```

The entire function `findzeros` is as follows. Save this function as `findzeros.m` in the current folder.

```
function sol = findzeros(f,range,err)
if nargin < 3
    err = 1e-3;
end
sol = vpasolve(f,range);
if isempty(sol)
    return
else
    lowLimit = sol-err;
    highLimit = sol+err;
    temp = findzeros(f,[range(1) lowLimit],1);
    if ~isempty(temp)
        sol = sort([sol temp]);
    end
    temp = findzeros(f,[highLimit range(2)],1);
    if ~isempty(temp)
        sol = sort([sol temp]);
    end
    return
end
end
```

Call `findzeros` with search range `[15 25]` to find all zeros in that range for  $f(x) = \exp(x/7) \cdot \cos(2x)$ , within the default error tolerance.

```
syms f(x)
f(x) = exp(x/7)*cos(2*x);
sol = findzeros(f,[15 25])'

sol =
```

```
(16.493361431346414501928877762217)
(18.064157758141311121160199453857)
(19.634954084936207740391521145497)
(21.205750411731104359622842837137)
(22.776546738526000978854164528776)
(24.347343065320897598085486220416)
```

### Obtain Solutions to Arbitrary Precision

Use `digits` to set the precision of the solutions returned by `vpasolve`. By default, `vpasolve` returns solutions to a precision of 32 significant figures.

```
f = exp(x/7)*cos(2*x);
vpasolve(f)

ans = -7.0685834705770347865409476123789
```

Use `digits` to increase the precision to 64 significant figures. When modifying `digits`, ensure that you save its current value so that you can restore it.

```
digitsOld = digits;
digits(64)
vpasolve(f)

ans = -7.068583470577034786540947612378881489443631148593988097193625333
```

Next, change the precision of the solutions to 16 significant figures.

```
digits(16)
```

### Solve Multivariate Equations Using Search Ranges

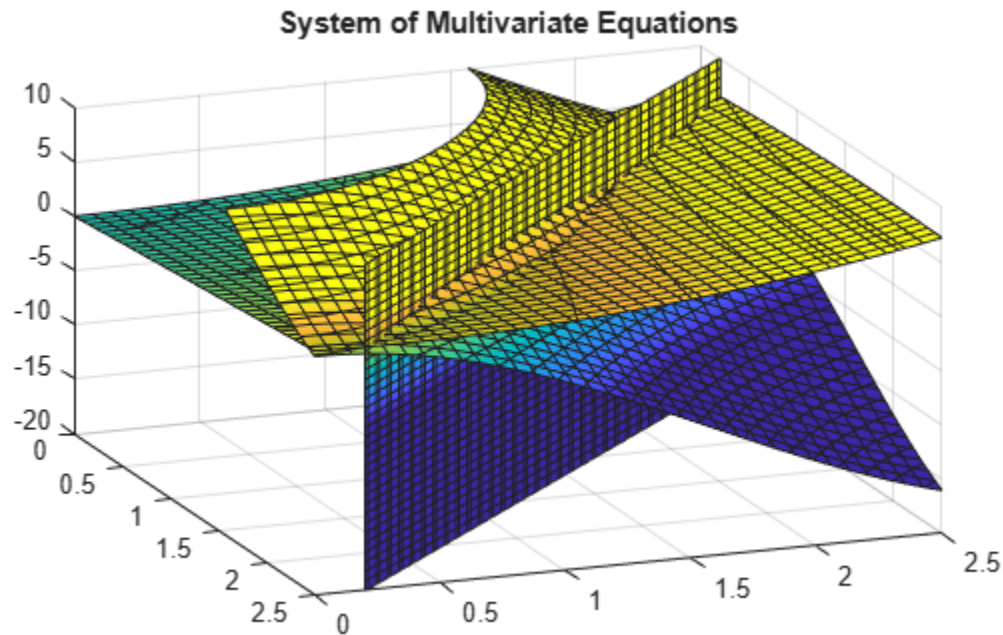
Consider the following system of equations.

$$\begin{aligned} z &= 10(\cos(x) + \cos(y)) \\ z &= x + y^2 - 0.1x^2y \\ x + y - 2.7 &= 0 \end{aligned}$$

A plot of the equations for  $0 \leq x \leq 2.5$  and  $0 \leq y \leq 2.5$  shows that the three surfaces intersect in two points. To better visualize the plot, change the line of sight using `view`.

```
syms x y z
eqn1 = z == 10*(cos(x) + cos(y));
eqn2 = z == x+y^2-0.1*x^2*y;
eqn3 = x+y-2.7 == 0;
equations = [eqn1 eqn2 eqn3];
fimplicit3(equations)
axis([0 2.5 0 2.5 -20 10])
title('System of Multivariate Equations')
view(69,28)
```



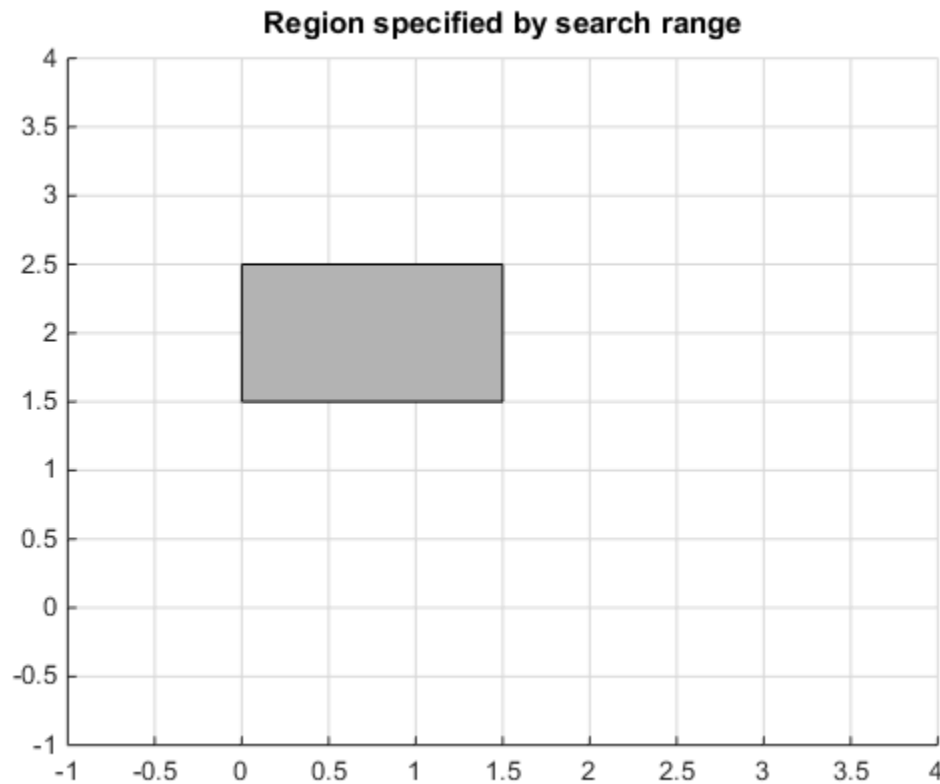


Use `vpasolve` to find a point where the surfaces intersect. The function `vpasolve` returns a structure. To access the x-, y-, and z-values of the solution, index into the structure.

```
sol = vpasolve(equations);  
[sol.x sol.y sol.z]
```

```
ans = (2.369747722454798 0.3302522775452021 2.293354376823228)
```

To search a region of the solution space, specify search ranges for the variables. If you specify the ranges  $0 \leq x \leq 1.5$  and  $1.5 \leq y \leq 2.5$ , then `vpasolve` function searches the bounded area shown.



Use `vpasolve` to find a solution for this search range. To omit a search range for  $z$ , set the third search range to `[NaN NaN]`.

```
vars = [x y z];
range = [0 1.5; 1.5 2.5; NaN NaN];
sol = vpasolve(equations,vars,range);
[sol.x sol.y sol.z]
```

```
ans = (0.9106266172563336 1.789373382743666 3.964101572135625)
```

To find multiple solutions, set the `'Random'` option to `true`. This makes `vpasolve` use random starting points on successive runs. The `'Random'` option can be used in conjunction with search ranges to make `vpasolve` use random starting points within a search range. Because `'Random'` selects starting points randomly, the same solution might be found on successive calls. Call `vpasolve` repeatedly to ensure you find both solutions.

```
clear sol
range = [0 3; 0 3; NaN NaN];
for k = 1:5
    temp = vpasolve(equations,vars,range,'Random',true);
    sol(k,1) = temp.x;
    sol(k,2) = temp.y;
    sol(k,3) = temp.z;
end
sol

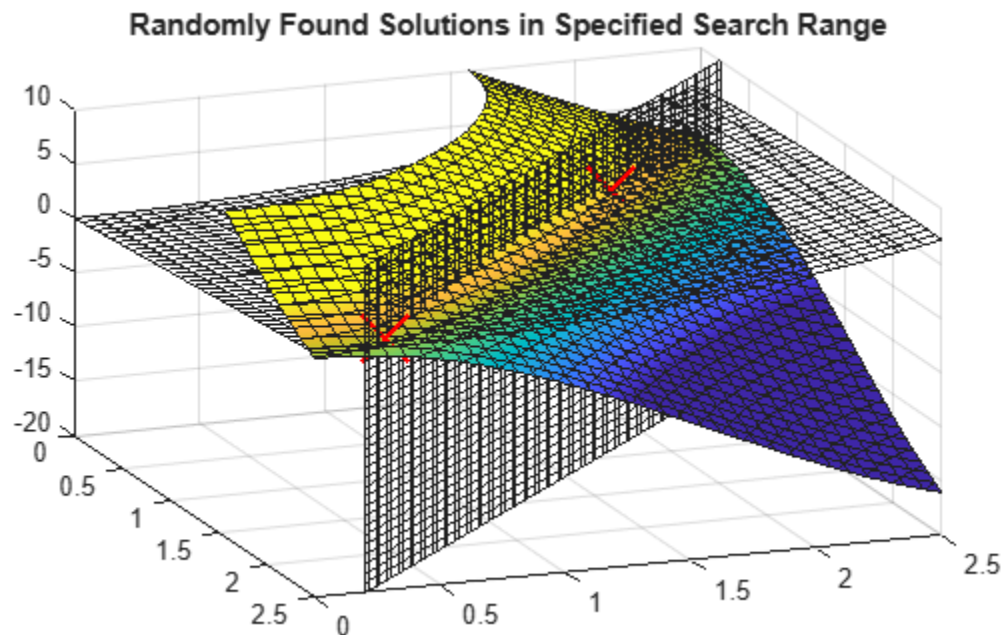
sol =
```

```
( 2.369747722454798  0.3302522775452021  2.293354376823228)
( 2.369747722454798  0.3302522775452021  2.293354376823228)
( 2.369747722454798  0.330252277545202  2.293354376823228)
( 0.9106266172563336  1.789373382743666  3.964101572135625)
( 0.9106266172563336  1.789373382743666  3.964101572135625)
```

Plot the equations. Superimpose the solutions as a scatter plot of points with yellow X markers using `scatter3`. To better visualize the plot, make two of the surfaces transparent using `alpha` and change the line of sight using `view`.

`vpasolve` finds solutions at the intersection of the surfaces formed by the equations as shown.

```
clf
ax = axes;
h = fimplicit3(equations);
h(2).FaceAlpha = 0;
h(3).FaceAlpha = 0;
axis([0 2.5 0 2.5 -20 10])
hold on
scatter3(sol(:,1),sol(:,2),sol(:,3),600,'red','X','LineWidth',2)
title('Randomly Found Solutions in Specified Search Range')
cz = ax.Children;
view(69,28)
hold off
```



Lastly, restore the old value of `digits` for further calculations.

```
digits(digitsOld)
```

## Solve System of Linear Equations

This example shows how to solve a system of linear equations using the Symbolic Math Toolbox™.

### Solve System of Linear Equations Using `linsolve`

A system of linear equations

$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1$$

$$a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2$$

...

$$a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n = b_m$$

can be represented as the matrix equation  $A\vec{x} = \vec{b}$ , where  $A$  is the coefficient matrix

$$A = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \dots & a_{mn} \end{bmatrix}$$

and  $\vec{b}$  is the vector containing the right sides of equations,

$$\vec{b} = \begin{bmatrix} b_1 \\ \vdots \\ b_m \end{bmatrix}$$

If you do not have the system of linear equations in the form  $AX = B$ , use `equationsToMatrix` to convert the equations into this form. Consider the following system.

$$2x + y + z = 2$$

$$-x + y - z = 3$$

$$x + 2y + 3z = -10$$

Define the system of equations.

```
syms x y z
eqn1 = 2*x + y + z == 2;
eqn2 = -x + y - z == 3;
eqn3 = x + 2*y + 3*z == -10;
```

Use `equationsToMatrix` to convert the equations into the form  $AX = B$ . The second input to `equationsToMatrix` specifies the independent variables in the equations.

```
[A,B] = equationsToMatrix([eqn1,eqn2,eqn3],[x,y,z])
```

$$A = \begin{pmatrix} 2 & 1 & 1 \\ -1 & 1 & -1 \\ 1 & 2 & 3 \end{pmatrix}$$

B =

$$\begin{pmatrix} 2 \\ 3 \\ -10 \end{pmatrix}$$

Use `linsolve` to solve  $AX = B$  for the vector of unknowns  $X$ .

```
X = linsolve(A,B)
```

$$X = \begin{pmatrix} 3 \\ 1 \\ -5 \end{pmatrix}$$

From the result in  $X$ , the solutions of the system are  $x = 3$ ,  $y = 1$ , and  $z = -5$ .

### Solve System of Linear Equations Using `solve`

Use `solve` instead of `linsolve` if you have the equations in the form of expressions and not a matrix of coefficients. Consider the same system of linear equations.

$$\begin{aligned} 2x + y + z &= 2 \\ -x + y - z &= 3 \\ x + 2y + 3z &= -10 \end{aligned}$$

Define the system of equations.

```
syms x y z
eqn1 = 2*x + y + z == 2;
eqn2 = -x + y - z == 3;
eqn3 = x + 2*y + 3*z == -10;
```

Solve the system of equations using `solve`. The inputs to `solve` are a vector of equations, and a vector of variables to solve the equations for.

```
sol = solve([eqn1,eqn2,eqn3],[x,y,z]);
```

`solve` returns the solutions in a structure array. To access the solutions, index into the array.

```
xSol = sol.x
```

```
xSol = 3
```

```
ySol = sol.y
```

```
ySol = 1
```

```
zSol = sol.z
```

```
zSol = -5
```

### See Also

### More About

- “Solve Algebraic Equations” on page 3-3

- “Solve System of Algebraic Equations” on page 3-7

## Select Numeric or Symbolic Solver

You can solve equations to obtain a symbolic or numeric answer. For example, a solution to  $\cos(x) = -1$  is  $\pi$  in symbolic form and 3.14159 in numeric form. The symbolic solution is exact, while the numeric solution approximates the exact symbolic solution. Symbolic Math Toolbox offers both symbolic and numeric equation solvers. This table can help you choose either the symbolic solver (`solve`) or the numeric solver (`vpasolve`). A possible strategy is to try the symbolic solver first, and use the numeric solver if the symbolic solver is stuck.

| Solve Equations Symbolically Using <code>solve</code>                                | Solve Equations Numerically Using <code>vpasolve</code>                                                                                    |
|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Returns exact solutions. Solutions can then be approximated using <code>vpa</code> . | Returns approximate solutions. Precision can be controlled arbitrarily using <code>digits</code> .                                         |
| Returns a general form of the solution.                                              | For polynomial equations, returns all numeric solutions that exist. For nonpolynomial equations, returns the first numeric solution found. |
| General form allows insight into the solution.                                       | Numeric solutions provide less insight.                                                                                                    |
| Runs slower.                                                                         | Runs faster.                                                                                                                               |
| Search ranges can be specified using inequalities.                                   | Search ranges and starting points can be specified.                                                                                        |
| <code>solve</code> solves equations and inequalities that contain parameters.        | <code>vpasolve</code> does not solve inequalities, nor does it solve equations that contain parameters.                                    |
| <code>solve</code> can return parameterized solutions.                               | <code>vpasolve</code> does not return parameterized solutions.                                                                             |

`vpasolve` uses variable-precision arithmetic. You can control precision arbitrarily using `digits`. For examples, see “Increase Precision of Numeric Calculations” on page 2-36.

### See Also

`solve` | `vpasolve`

### Related Examples

- “Solve Algebraic Equations” on page 3-3
- “Solve Equations Numerically” on page 3-28
- “Solve System of Linear Equations” on page 3-36

## Solve Parametric Equations in ReturnConditions Mode

This example shows you how to solve parameterized algebraic equations using the Symbolic Math Toolbox™.

To solve algebraic equations symbolically, use the `solve` function. The `solve` function can provide complete information about all solutions of an equation, even if there are infinitely many, by introducing a parameterization. It can also provide information under which conditions these solutions are valid. To obtain this information, set the option `ReturnConditions` to `true`.

Solve the equation  $\sin(C*x) = 1$ . Specify  $x$  as the variable to solve for. The `solve` function handles  $C$  as a constant. Provide three output variables for the solution, the newly generated parameters in the solution, and the conditions on the solution.

```
syms C x
eq = sin(C*x) == 1;
[solx,params,conds] = solve(eq,x,ReturnConditions=true)
```

```
solx =

$$\frac{\frac{\pi}{2} + 2\pi k}{C}$$

```

```
params = k
```

```
conds = k ∈ ℤ ∧ C ≠ 0
```

To verify the solution, substitute the solution into the equation using `subs`. To work on under the assumptions in `conds` for the rest of this example, use `assume`. Test the solution using `isAlways`. The `isAlways` function returns logical 1 (`true`) indicating that the solution always holds under the given assumptions.

```
SolutionCorrect = subs(eq,x,solx)
```

```
SolutionCorrect =

$$\sin\left(\frac{\pi}{2} + 2\pi k\right) = 1$$

```

```
assume(conds)
isAlways(SolutionCorrect)
```

```
ans = logical
      1
```

To obtain one solution out of the infinitely many solutions, find a value of the parameters `params` by solving the conditions `conds` for the parameters; do not specify the `ReturnConditions` option. Substitute this value of  $k$  into the solution using `subs` to obtain a solution out of the solution set.

```
k0 = solve(conds,params)
```

```
k0 = 0
```

```
subs(solx,params,k0)
```

```
ans =
```



$$\frac{\pi}{2C}$$

To obtain a parameter value that satisfies a certain condition, add the condition to the input to `solve`. Find a value of the parameter greater than  $99/4$  and substitute in to find the solution.

```
k1 = solve([conds,params > 99/4],params)
```

```
k1 = 26
```

```
subs(solx,params,k1)
```

```
ans =  
 $\frac{105\pi}{2C}$ 
```

To find a solution in a specified interval, you can solve the original equation with the inequalities that specify the interval.

```
[solx1,params1,conds1] = solve([eq,x > 2,x < 7],x,ReturnConditions=true)
```

```
solx1 =  
 $\frac{\pi + 4\pi k}{2C}$ 
```

```
params1 = k
```

```
conds1 = (0 < C ∧ 4C < π + 4πk ∧ π + 4πk < 14C) ∨ (C < 0 ∧ π + 4πk < 4C ∧ 14C < π + 4πk)
```

Alternatively, you can also use the existing solution, and restrict it with additional conditions. Note that while the condition changes, the solution remains the same. The `solve` function expresses `solx` and `solx1` with different parameterizations, although they are equivalent.

```
[~,~,conds2] = solve(x == solx,x < 7,x > 2,x,ReturnConditions=true)
```

```
conds2 =  
 $\frac{4}{\pi} < \frac{4k+1}{C} \wedge \frac{4k+1}{C} < \frac{14}{\pi}$ 
```

Obtain those parameter values that satisfy the new condition, for a particular value of the constant  $C$ .

```
conds3 = subs(conds2,C,5)
```

```
conds3 =  
 $\frac{4}{\pi} < \frac{4k}{5} + \frac{1}{5} \wedge \frac{4k}{5} + \frac{1}{5} < \frac{14}{\pi}$ 
```

```
solve(conds3,params)
```

```
ans =  
 $\begin{pmatrix} 2 \\ 3 \\ 4 \\ 5 \end{pmatrix}$ 
```

## Solve Algebraic Equation Using Live Editor Task

You can interactively solve algebraic equations to obtain symbolic solutions using the Solve Symbolic Equation task in the Live Editor. For more information on Live Editor tasks, see “Add Interactive Tasks to a Live Script”.

These examples show you how to find the solutions of

- a trigonometric equation
- a cubic equation
- a system of cubic and linear equations

### Solve a Trigonometric Equation

Find the solution of the trigonometric equation  $\sin(x) + \cos(x) = 0$  with the assumption that  $x > \pi/2$ .

First, go to the **Home** tab, and create a live script by clicking  **New Live Script**. Define the symbolic variable  $x$ , and use the `==` operator to declare the equality sign of the trigonometric equation. Use `assume` to set the assumption on  $x$ .

```
syms x
eqn = sin(x) + cos(x) == 0;
assume(x > pi/2);
```

In the **Live Editor** tab, run the code by clicking  **Run** to store  $x$  with its assumption and `eqn` into the current workspace.

Next, open the **Solve Symbolic Equation** task by selecting **Task > Solve Symbolic Equation** in the **Live Editor** tab. To find the solution of the trigonometric equation, select the symbolic equation `eqn` from the workspace. Specify  $x$  as the variable to solve for. Select the **Return conditions** option to

return the general solution and the analytic constraints under which it holds.

Solve Symbolic Equation


☒ Autorun




solution = Analytic solution of equation **eqn** with respect to x with conditions

Select equation

Equation

eqn

Variables

x

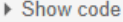
for example: x

Specify solver options

☐ Return real solutions
 ☒ Return conditions
 ☐ Ignore analytic constraints
 ☐ Return one solution
 ☐ Expand all roots
 ☐ Ignore properties

Display result

☒ Equation
 ☒ Solution



```

eqn = cos(x) + sin(x) = 0

solution = struct with fields:
    x: pi*k - pi/4
    parameters: k
    conditions: in(k, 'integer') & 1 <= k
where
x = pi*k - pi/4 for all k with k in Z and 1 <= k

```

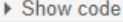
You can ignore the assumption on x by selecting the **Ignore properties** option. Return the solution without using the assumption that  $x > \pi/2$ .

Specify solver options

☐ Return real solutions
 ☒ Return conditions
 ☐ Ignore analytic constraints
 ☐ Return one solution
 ☐ Expand all roots
 ☒ Ignore properties

Display result

☒ Equation
 ☒ Solution



```

eqn = cos(x) + sin(x) = 0

solution = struct with fields:
    x: pi*k - pi/4
    parameters: k
    conditions: in(k, 'integer')
where
x = pi*k - pi/4 for all k with k in Z

```



The solver returns the symbolic solutions in terms of the `root` function. To express the root function in terms of square roots, select the **Expand all roots** option.

#### Specify solver options

- ☐ Return real solutions    ☐ Return conditions    ☐ Ignore analytic constraints  
☐ Return one solution    ☒ Expand all roots    ☐ Ignore properties

#### Display result

- ☒ Equation    ☒ Solution

► Show code

$$\text{cubicEquation} = x^3 - 2x^2 + y = 0$$

solution =

$$\begin{pmatrix} \frac{4}{9\sigma_2} + \sigma_2 + \frac{2}{3} \\ \frac{2}{3} - \frac{\sigma_2}{2} - \frac{2}{9\sigma_2} - \sigma_1 \\ \frac{2}{3} - \frac{\sigma_2}{2} - \frac{2}{9\sigma_2} + \sigma_1 \end{pmatrix}$$

where

$$\sigma_1 = \frac{\sqrt{3} \left( \frac{4}{9\sigma_2} - \sigma_2 \right) i}{2}$$

$$\sigma_2 = \left( \sqrt{\left( \frac{y}{2} - \frac{8}{27} \right)^2 - \frac{64}{729}} - \frac{y}{2} + \frac{8}{27} \right)^{1/3}$$

### Solve a System of Equations

Solve the system of cubic and linear equations:

$$x^3 - 2x^2 + y = 0$$

$$y = 4x - 8$$

Define the symbolic variables `x` and `y` using `syms`. Use the `==` operator to declare the equality sign of the equations. To declare the system of equations, combine the two symbolic equations into an array.

```
syms x y
cubicEquation = x^3 - 2*x^2 + y == 0;
linearEquation = y == 4*x - 8;
systemEquations = [cubicEquation linearEquation];
```

To find the solution of the system of equations, select the symbolic equation `systemEquations` from the workspace. Specify `x` and `y` as the variables to solve for.

### Solve Symbolic Equation

▶ ☒ Autorun | ? ⋮

solution

= Analytic solution of equation `systemEquations` with respect to `x,y`

Select equation

Equation systemEquations ▼ Variables x,y ▼ *for example: x,y*

Specify solver options

☐ Return real solutions
 ☐ Return conditions
 ☐ Ignore analytic constraints

☐ Return one solution
 ☐ Expand all roots
 ☐ Ignore properties

Display result

☒ Equation
 ☒ Solution

▶ Show code

```

systemEquations = (x^3 - 2 x^2 + y = 0  y = 4 x - 8)

solution = struct with fields:
    x: [3x1 sym]
    y: [3x1 sym]
where
(x(1) y(1)) = (2 0)
(x(2) y(2)) = (-2 i  -8 - 8 i)
(x(3) y(3)) = (2 i  -8 + 8 i)

```

The solver returns real and complex solutions. To show real solutions only, select the **Return real solutions** option.

Specify solver options

☒ Return real solutions
 ☐ Return conditions
 ☐ Ignore analytic constraints

☐ Return one solution
 ☐ Expand all roots
 ☐ Ignore properties

Display result

☒ Equation
 ☒ Solution

▶ Show code

```

systemEquations = (x^3 - 2 x^2 + y = 0  y = 4 x - 8)

solution = struct with fields:
    x: 2
    y: 0
where
(x y) = (2 0)

```

## Generate Code

To view the code that a task used, click  **Show code** at the bottom of the task window. The task displays the code block, which you can cut and paste to use or modify later in the existing script or a different program. For example:

Display result
☒ Equation ☒ Solution

▼ Hide code

```

% Compute analytic solution of a symbolic equation
systemEquations

systemEquations = (x^3 - 2 x^2 + y = 0  y = 4 x - 8)

solution = solve(systemEquations,[x,y],"Real",true);
% Display symbolic solution returned by solve
displaySymSolution(solution);

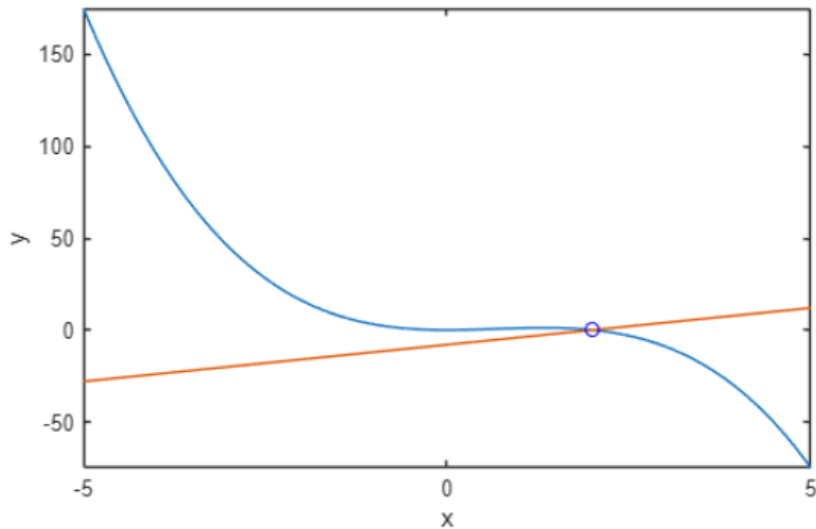
solution = struct with fields:
    x: 2
    y: 0
where
(x y) = (2 0)

```

Because the underlying code is now part of your live script, you can continue to use the solutions generated by the task for further processing. For example, you can plot the system of equations and

their real-valued solution.

```
fplot(-x^3 + 2*x^2)
xlabel("x")
ylabel("y")
hold on
fplot(4*x - 8)
plot(solution.x,solution.y,"bo")
```



## See Also

### Live Editor Tasks

[Simplify Symbolic Expression](#) | [Solve Symbolic Equation](#)

### Functions

`solve`

## Related Examples

- [“Add Interactive Tasks to a Live Script”](#)
- [“Solve Algebraic Equations”](#) on page 3-3



## Solve Differential Equation

Solve a differential equation analytically by using the `dsolve` function, with or without initial conditions. To solve a system of differential equations, see “Solve a System of Differential Equations” on page 3-53.

### First-Order Linear ODE

Solve this differential equation.

$$\frac{dy}{dt} = ty$$

First, represent  $y$  by using `syms` to create the symbolic function  $y(t)$ .

```
syms y(t)
```

Define the equation using `==` and represent differentiation using the `diff` function.

```
ode = diff(y,t) == t*y
```

$$\text{ode}(t) = \frac{\partial}{\partial t} y(t) = t y(t)$$

Solve the equation using `dsolve`.

```
ySol(t) = dsolve(ode)
```

$$ySol(t) = C_1 \frac{t^2}{e^{\frac{t^2}{2}}}$$

### Solve Differential Equation with Condition

In the previous solution, the constant  $C_1$  appears because no condition was specified. Solve the equation with the initial condition  $y(0) == 2$ . The `dsolve` function finds a value of  $C_1$  that satisfies the condition.

```
cond = y(0) == 2;
ySol(t) = dsolve(ode,cond)
```

$$ySol(t) = \frac{t^2}{2 e^{\frac{t^2}{2}}}$$

If `dsolve` cannot solve your equation, then try solving the equation numerically. See “Solve a Second-Order Differential Equation Numerically” on page 3-57.

### Nonlinear Differential Equation with Initial Condition

Solve this nonlinear differential equation with an initial condition. The equation has multiple solutions.

$$\left(\frac{dy}{dt} + y\right)^2 = 1$$

$$y(0) = 0$$

```
syms y(t)
ode = (diff(y,t)+y)^2 == 1;
cond = y(0) == 0;
ySol(t) = dsolve(ode,cond)
```

```
ySol(t) =
    (e^{-t} - 1)
    (1 - e^{-t})
```

### Second-Order ODE with Initial Conditions

Solve this second-order differential equation with two initial conditions.

$$\frac{d^2y}{dx^2} = \cos(2x) - y$$

$$y(0) = 1$$

$$y'(0) = 0$$

Define the equation and conditions. The second initial condition involves the first derivative of  $y$ . Represent the derivative by creating the symbolic function  $Dy = \text{diff}(y)$  and then define the condition using  $Dy(0) == 0$ .

```
syms y(x)
Dy = diff(y);

ode = diff(y,x,2) == cos(2*x)-y;
cond1 = y(0) == 1;
cond2 = Dy(0) == 0;
```

Solve `ode` for  $y$ . Simplify the solution using the `simplify` function.

```
conds = [cond1 cond2];
ySol(x) = dsolve(ode,conds);
ySol = simplify(ySol)
```

```
ySol(x) =
    1 - \frac{8 \sin\left(\frac{x}{2}\right)^4}{3}
```

### Third-Order ODE with Initial Conditions

Solve this third-order differential equation with three initial conditions.

$$\frac{d^3u}{dx^3} = u$$

$$u(0) = 1$$

$$u'(0) = -1$$

$$u''(0) = \pi$$

Because the initial conditions contain the first- and second-order derivatives, create two symbolic functions,  $Du = \text{diff}(u,x)$  and  $D2u = \text{diff}(u,x,2)$ , to specify the initial conditions.

```
syms u(x)
Du = diff(u,x);
D2u = diff(u,x,2);
```

Create the equation and initial conditions, and solve it.

```
ode = diff(u,x,3) == u;
cond1 = u(0) == 1;
cond2 = Du(0) == -1;
cond3 = D2u(0) == pi;
conds = [cond1 cond2 cond3];
```

```
uSol(x) = dsolve(ode,conds)
```

```
uSol(x) =
```

$$\frac{\pi}{3} e^x - e^{-\frac{x}{2}} \cos\left(\frac{\sqrt{3}x}{2}\right) \left(\frac{\pi}{3} - 1\right) - \frac{\sqrt{3} e^{-\frac{x}{2}} \sin\left(\frac{\sqrt{3}x}{2}\right) (\pi + 1)}{3}$$

### More ODE Examples

This table shows examples of differential equations and their Symbolic Math Toolbox™ syntax.

| Differential Equation                               | Commands Using Symbolic Math Toolbox                                                                                                                                      |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\frac{dy}{dt} + 4y(t) = e^{-t}$<br>$y(0) = 1$      | <pre>&gt;&gt; syms y(t) &gt;&gt; ode = diff(y)+4*y == exp(-t); &gt;&gt; cond = y(0) == 1; &gt;&gt; ySol(t) = dsolve(ode,cond) ySol(t) = exp(-t)/3 + (2*exp(-4*t))/3</pre> |
| $2x^2 \frac{d^2y}{dx^2} + 3x \frac{dy}{dx} - y = 0$ | <pre>&gt;&gt; syms y(x) &gt;&gt; ode = 2*x^2*diff(y,x,2)+3*x*diff(y,x)-y == 0; &gt;&gt; ySol(x) = dsolve(ode) ySol(x) = C1/(3*x) + C2*x^(1/2)</pre>                       |
| The Airy equation.<br>$\frac{d^2y}{dx^2} = x y(x)$  | <pre>&gt;&gt; syms y(x) &gt;&gt; ode = diff(y,x,2) == x*y; &gt;&gt; ySol(x) = dsolve(ode) ySol(x) = C1*airy(0,x) + C2*airy(2,x)</pre>                                     |

| Differential Equation                                                              | Commands Using Symbolic Math Toolbox                                                                                                                                                                                                                                                             |
|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Puiseux series solution.<br>$(x^2 + 1)\frac{d^2y}{dx^2} - 2x\frac{dy}{dx} + y = 0$ | >> syms y(x) a<br><br>>> ode = (x^2+1)*diff(y,x,2)-2*x*diff(y,x)+y<br>== 0;<br><br>>> Dy = diff(y,x);<br><br>>> cond = [Dy(0) == a; y(0) == 5];<br><br>>> ySol(x) =<br>dsolve(ode,cond,ExpansionPoint=0)<br><br>ySol(x) =<br><br>- (a*x^5)/120 - (5*x^4)/24 + (a*x^3)/6 -<br>(5*x^2)/2 + a*x + 5 |

See Also

dsolve | odeFunction | odeToVectorField | reduceDifferentialOrder | daeFunction

Related Examples

- “Solve a System of Differential Equations” on page 3-53
- “Solve a Second-Order Differential Equation Numerically” on page 3-57
- “Solve Differential Algebraic Equations (DAEs)” on page 3-66

## Solve a System of Differential Equations

Solve a system of several ordinary differential equations in several variables by using the `dsolve` function, with or without initial conditions. To solve a single differential equation, see “Solve Differential Equation” on page 3-49.

### Solve System of Differential Equations

Solve this system of linear first-order differential equations.

$$\begin{aligned}\frac{du}{dt} &= 3u + 4v, \\ \frac{dv}{dt} &= -4u + 3v.\end{aligned}$$

First, represent  $u$  and  $v$  by using `syms` to create the symbolic functions  $u(t)$  and  $v(t)$ .

```
syms u(t) v(t)
```

Define the equations using `==` and represent differentiation using the `diff` function.

```
ode1 = diff(u) == 3*u + 4*v;
ode2 = diff(v) == -4*u + 3*v;
odes = [ode1; ode2]
```

$$\text{odes}(t) = \begin{pmatrix} \frac{\partial}{\partial t} u(t) = 3u(t) + 4v(t) \\ \frac{\partial}{\partial t} v(t) = 3v(t) - 4u(t) \end{pmatrix}$$

Solve the system using the `dsolve` function which returns the solutions as elements of a structure.

```
S = dsolve(odes)
```

```
S = struct with fields:
  v: C1*cos(4*t)*exp(3*t) - C2*sin(4*t)*exp(3*t)
  u: C2*cos(4*t)*exp(3*t) + C1*sin(4*t)*exp(3*t)
```

If `dsolve` cannot solve your equation, then try solving the equation numerically. See “Solve a Second-Order Differential Equation Numerically” on page 3-57.

To access  $u(t)$  and  $v(t)$ , index into the structure  $S$ .

```
uSol(t) = S.u
```

$$uSol(t) = C_2 \cos(4t) e^{3t} + C_1 \sin(4t) e^{3t}$$

```
vSol(t) = S.v
```

$$vSol(t) = C_1 \cos(4t) e^{3t} - C_2 \sin(4t) e^{3t}$$

Alternatively, store  $u(t)$  and  $v(t)$  directly by providing multiple output arguments.

```
[uSol(t),vSol(t)] = dsolve(odes)
```

$$uSol(t) = C_2 \cos(4t) e^{3t} + C_1 \sin(4t) e^{3t}$$

$$vSol(t) = C_1 \cos(4t) e^{3t} - C_2 \sin(4t) e^{3t}$$

The constants  $C_1$  and  $C_2$  appear because no conditions are specified. Solve the system with the initial conditions  $u(0) == 0$  and  $v(0) == 1$ . The `dsolve` function finds values for the constants that satisfy these conditions.

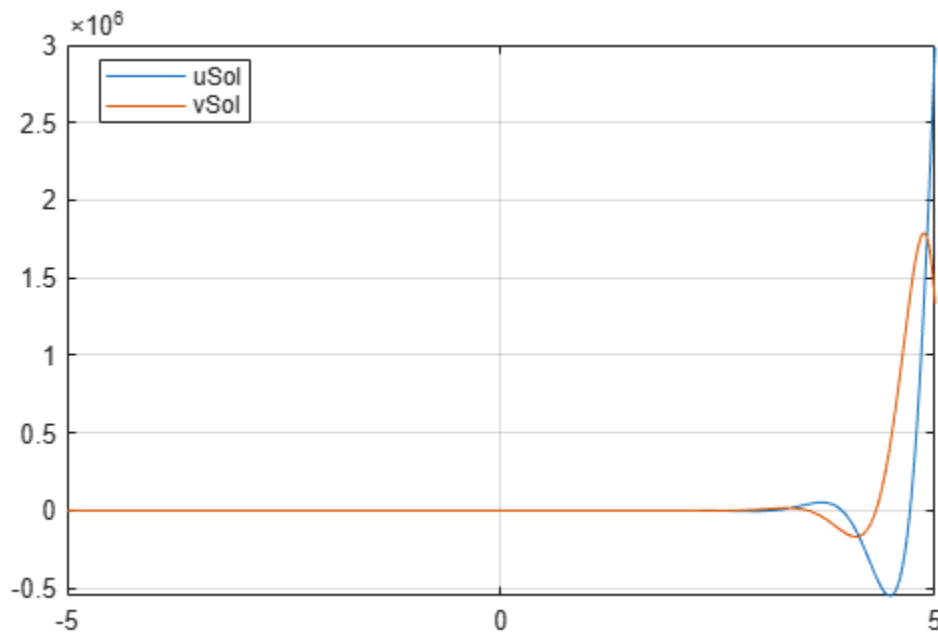
```
cond1 = u(0) == 0;
cond2 = v(0) == 1;
conds = [cond1; cond2];
[uSol(t), vSol(t)] = dsolve(odes, conds)
```

$$uSol(t) = \sin(4t) e^{3t}$$

$$vSol(t) = \cos(4t) e^{3t}$$

Visualize the solution using `fplot`.

```
fplot(uSol)
hold on
fplot(vSol)
grid on
legend("uSol", "vSol", Location="best")
```



### Solve Differential Equations in Matrix Form

Solve differential equations in matrix form by using `dsolve`.

Consider this system of differential equations.

$$\begin{aligned}\frac{dx}{dt} &= x + 2y + 1, \\ \frac{dy}{dt} &= -x + y + t.\end{aligned}$$

The matrix form of the system is

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} 1 \\ t \end{bmatrix}.$$

Let

$$Y = \begin{bmatrix} x \\ y \end{bmatrix}, A = \begin{bmatrix} 1 & 2 \\ -1 & 1 \end{bmatrix}, B = \begin{bmatrix} 1 \\ t \end{bmatrix}.$$

The system is now  $Y' = A Y + B$ .

Define these matrices and the matrix equation.

```
syms x(t) y(t)
A = [1 2; -1 1];
B = [1; t];
Y = [x; y];
odes = diff(Y) == A*Y + B
```

$$\text{odes}(t) = \begin{pmatrix} \frac{\partial}{\partial t} x(t) = x(t) + 2y(t) + 1 \\ \frac{\partial}{\partial t} y(t) = t - x(t) + y(t) \end{pmatrix}$$

Solve the matrix equation using `dsolve`. Simplify the solution by using the `simplify` function.

```
[xSol(t),ySol(t)] = dsolve(odes);
xSol(t) = simplify(xSol(t))

xSol(t) =
    \frac{2t}{3} + \sqrt{2}C_2e^t\cos(\sqrt{2}t) + \sqrt{2}C_1e^t\sin(\sqrt{2}t) + \frac{1}{9}

ySol(t) = simplify(ySol(t))

ySol(t) =
    C_1e^t\cos(\sqrt{2}t) - \frac{t}{3} - C_2e^t\sin(\sqrt{2}t) - \frac{2}{9}
```

The constants  $C_1$  and  $C_2$  appear because no conditions are specified.

Solve the system with the initial conditions  $u(0) = 2$  and  $v(0) = -1$ . When specifying equations in matrix form, you must specify initial conditions in matrix form too. `dsolve` finds values for the constants that satisfy these conditions.

```
C = Y(0) == [2;-1];
[xSol(t),ySol(t)] = dsolve(odes,C)

xSol(t) =
```

$$\sqrt{2} e^t \sigma_2 \left( \frac{17\sqrt{2}}{18} + \frac{e^{-t} (4\sigma_1 + \sqrt{2}\sigma_2 + 6t\sigma_1 + 6\sqrt{2}t\sigma_2)}{18} \right) - \sqrt{2} e^t \sigma_1 \left( \frac{e^{-t} (4\sigma_2 - \sqrt{2}\sigma_1 + 6t\sigma_2 - 6\sqrt{2}t\sigma_1)}{18} + \frac{7}{9} \right)$$

where

$$\sigma_1 = \sin(\sqrt{2}t)$$

$$\sigma_2 = \cos(\sqrt{2}t)$$

$ySol(t) =$

$$-e^t \sigma_1 \left( \frac{17\sqrt{2}}{18} + \frac{e^{-t} (4\sigma_1 + \sqrt{2}\sigma_2 + 6t\sigma_1 + 6\sqrt{2}t\sigma_2)}{18} \right) - e^t \sigma_2 \left( \frac{e^{-t} (4\sigma_2 - \sqrt{2}\sigma_1 + 6t\sigma_2 - 6\sqrt{2}t\sigma_1)}{18} + \frac{7}{9} \right)$$

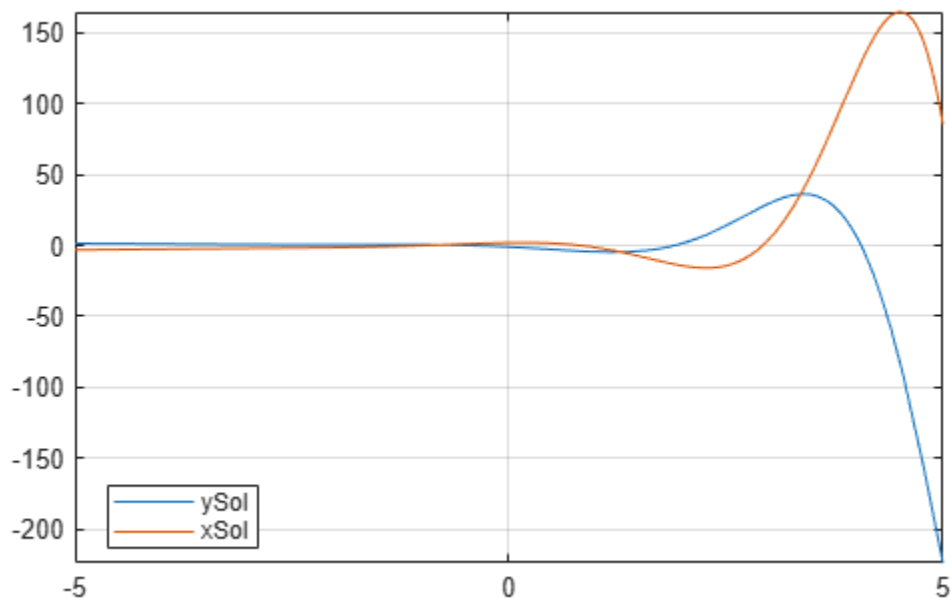
where

$$\sigma_1 = \sin(\sqrt{2}t)$$

$$\sigma_2 = \cos(\sqrt{2}t)$$

Visualize the solution using `fplot`.

```
figure
fplot(ySol)
hold on
fplot(xSol)
grid on
legend("ySol", "xSol", Location="best")
```



## See Also

"Solve Differential Equation" on page 3-49



## Solve a Second-Order Differential Equation Numerically

This example shows you how to convert a second-order differential equation into a system of differential equations that can be solved using the numerical solver `ode45` of MATLAB®.

A typical approach to solving higher-order ordinary differential equations is to convert them to systems of first-order differential equations, and then solve those systems. The example uses Symbolic Math Toolbox™ to convert a second-order ODE to a system of first-order ODEs. Then it uses the MATLAB solver `ode45` to solve the system.

### Rewrite the Second-Order ODE as a System of First-Order ODEs

Use `odeToVectorField` to rewrite this second-order differential equation

$$\frac{d^2 y}{dt^2} = (1 - y^2) \frac{dy}{dt} - y$$

using a change of variables. Let  $y(t) = Y_1$  and  $\frac{dy}{dt} = Y_2$  such that differentiating both equations we obtain a system of first-order differential equations.

$$\begin{aligned} \frac{dY_1}{dt} &= Y_2 \\ \frac{dY_2}{dt} &= -(Y_1^2 - 1)Y_2 - Y_1 \end{aligned}$$

```
syms y(t)
[V] = odeToVectorField(diff(y, 2) == (1 - y^2)*diff(y) - y)

V =
```

$$\begin{pmatrix} Y_2 \\ -(Y_1^2 - 1)Y_2 - Y_1 \end{pmatrix}$$

### Generate MATLAB Function

The MATLAB ODE solvers do not accept symbolic expressions as an input. Therefore, before you can use a MATLAB ODE solver to solve the system, you must convert that system to a MATLAB function. Generate a MATLAB function from this system of first-order differential equations using `matlabFunction` with `V` as an input.

```
M = matlabFunction(V, 'vars', {'t', 'Y'})

M = function_handle with value:
    @(t,Y)[Y(2); -(Y(1).^2-1.0).*Y(2)-Y(1)]
```

### Solve the System of First-Order ODEs

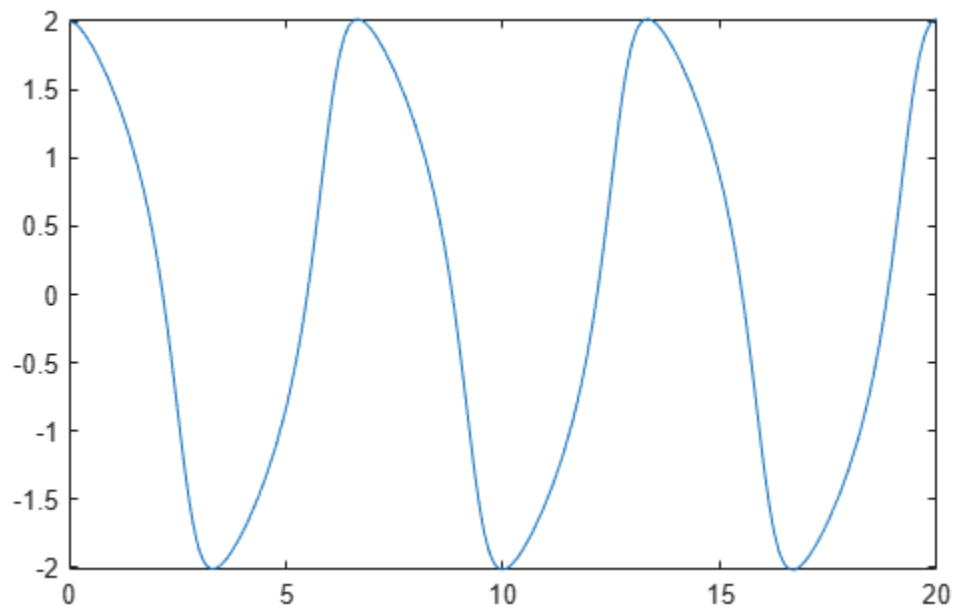
To solve this system, call the MATLAB `ode45` numerical solver using the generated MATLAB function as an input.

```
sol = ode45(M,[0 20],[2 0]);
```

**Plot the Solution**

Plot the solution using `linspace` to generate 100 points in the interval  $[0,20]$  and `deval` to evaluate the solution for each point.

```
fplot(@(x)deval(sol,x,1), [0, 20])
```

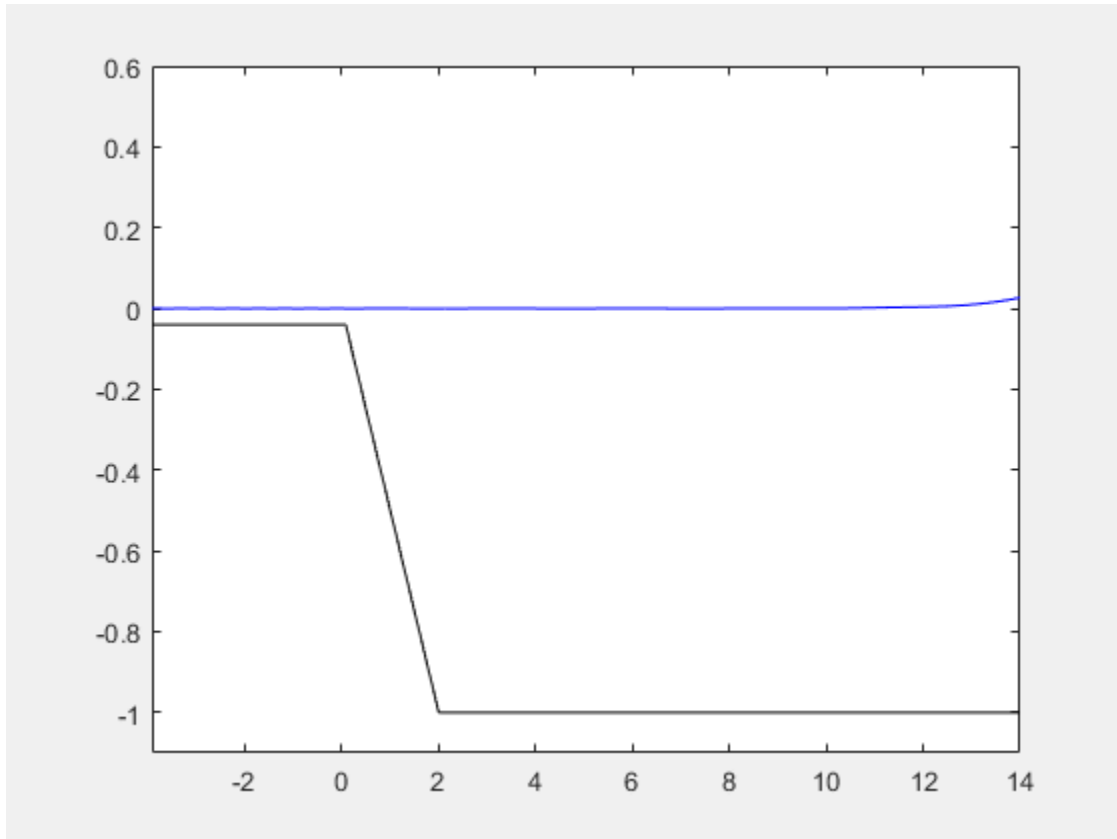
**See Also**

[dsolve](#) | [matlabFunction](#) | [ode45](#) | [odeToVectorField](#)

## Solve Partial Differential Equation of Tsunami Model

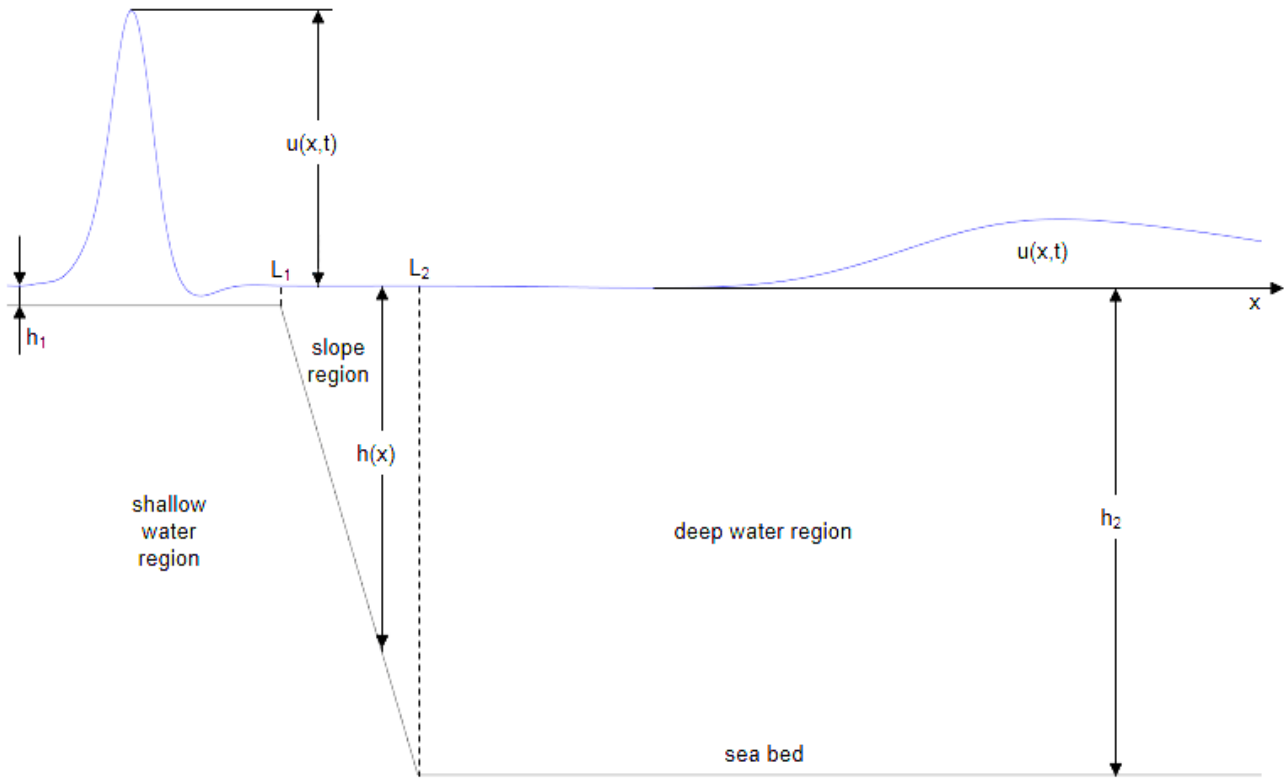
This example simulates a tsunami wave phenomenon by solving the partial differential equations that model the tsunami.

This simulation is a simplified visualization of the phenomenon, and is based on a paper by Goring and Raichlen [1].



### Mathematics of the Tsunami Model

A solitary wave (a soliton solution of the Korteweg-de Vries equation) travels at a constant speed from the right to the left along a canal of constant depth. This corresponds to a tsunami traveling over deep sea. At the left end of the canal, there is a slope simulating the continental shelf. After reaching the slope, the solitary wave begins to increase its height. When the water becomes very shallow, most of the wave is reflected back into the canal. However, a narrow but high peak of water arises at the end of the slope and proceeds with reduced speed in the original direction of the incident wave. This is the tsunami that finally hits the shore, causing disastrous destruction along the coastline. The speed of the wave nearing the shore is comparatively small. The wave eventually starts to break.



Using linear dispersionless water theory, the height  $u(x, t)$  of a free surface wave above the undisturbed water level in a one-dimensional canal of varying depth  $h(x)$  is the solution of the following partial differential equation. (See [2].)

$$u_{tt} = g(h u_x)_x$$

In this formula, subscripts denote partial derivatives, and  $g = 9.81 \text{ m/s}^2$  is the gravitational acceleration.

Consider a wave crossing a linear slope  $h(x)$  from a region with the constant depth  $h_2$  to a region with the constant depth  $h_1 \ll h_2$ . The Fourier transformation with respect to  $t$  turns the water wave partial differential equation to the following ordinary differential equation for the Fourier mode

$$u(x, t) = U(x, \omega) e^{i\omega t}.$$

$$-\omega^2 U = g(h U_x)_x$$

For the regions with constant depth  $h$ , the Fourier modes are traveling waves propagating in opposite directions with constant speed  $c = \sqrt{gh}$ .

$$u(x, t) = C_1(\omega) e^{i\omega(t + x/c)} + C_2(\omega) e^{i\omega(t - x/c)}$$

The solution  $u_2(x, t) = e^{i\omega(t + x/c_2)} + R(\omega) e^{i\omega(t - x/c_2)}$  for the deep water region is the superposition of two waves:

- a wave traveling to the left with constant speed  $c_2 = \sqrt{g h_2}$
- a wave traveling to the right with an amplitude given by the frequency dependent reflection coefficient  $R(\omega)$

This choice of  $u_2$  satisfies the wave equation in the deep water region for any  $R(\omega)$ .

The solution  $u_1(x, t) = T(\omega) e^{i\omega(t + x/c_1)}$  for the shallow water region is a transmitted wave traveling to the left with the constant speed  $c_1 = \sqrt{g h_1}$ . This choice of  $u_1$  satisfies the wave equation in the shallow water region for any transmission coefficient  $T(\omega)$ .

For the transition region (the slope), use  $u(x, t) = U(x, \omega) e^{i\omega t}$ .

### Parameters and Solutions of the Tsunami Model in Symbolic Math Toolbox

Define the parameters of the tsunami model as follows. Disregard the dependency on the frequency  $\omega$  in the following notations:  $R = R(\omega)$ ,  $T = T(\omega)$ ,  $U(x) = U(x, \omega)$ .

```
syms L H depthratio g positive
syms x t w T R U(x)

L1 = depthratio*L;
L2 = L;

h1 = depthratio*H;
h2 = H;
h(x) = x*H/L;

c1 = sqrt(g*h1);
c2 = sqrt(g*h2);

u(x,t) = U(x)*exp(1i*w*t);
u1(x,t) = T*exp(1i*w*(t + x/c1));
u2(x,t) = exp(1i*w*(t + x/c2)) + R*exp(1i*w*(t - x/c2));
```

In the transition region over the linear slope, use `dsolve` to solve the ODE for the Fourier transform  $U$  of  $u$ .

```
wavePDE(x,t) = diff(u,t,t) - g*diff(h(x)*diff(u,x),x);
slopeODE(x) = wavePDE(x,0);
U(x) = dsolve(slopeODE);
```

The solution  $U$  is a complicated expression involving Bessel functions. It contains two arbitrary "constants" that depend on  $\omega$ .

```
Const = setdiff(symvar(U), sym([depthratio,g,H,L,x,w]))
```

```
Const = (C1 C2)
```

For any Fourier mode, the overall solution must be a continuously differentiable function of  $x$ . Hence, the function values and the derivatives must match at the seam points  $L_1$  and  $L_2$ . This provides four linear equations for  $T$ ,  $R$ , and the two constants in  $U$ .

```
du1(x) = diff(u1(x,0),x);
du2(x) = diff(u2(x,0),x);
dU(x) = diff(U(x),x);
```

```
eqs = [ U(L1) == u1(L1,0), U(L2) == u2(L2,0),...
        dU(L1) == du1(L1), dU(L2) == du2(L2)];
unknowns = [Const(1),Const(2),R,T];
```

Solve these equations.

```
[Cvalue1, Cvalue2, R, T] = solve(eqs, unknowns);
```

Substitute the results back into  $R$ ,  $T$ , and  $U$ .

```
U(x) = subs(U(x), {Const(1),Const(2)}, {Cvalue1,Cvalue2});
```

You cannot directly evaluate the solution for  $\omega = 0$  because both numerator and denominator of the corresponding expressions vanish. Instead, find the low frequency limits of these expressions.

```
simplify(limit(U(x), w, 0))
```

```
ans =
      2
  -----
  sqrt(depthratio + 1)
```

```
simplify(limit(R, w, 0))
```

```
ans =
  -sqrt(depthratio - 1)
  -----
  sqrt(depthratio + 1)
```

```
simplify(limit(T, w, 0))
```

```
ans =
      2
  -----
  sqrt(depthratio + 1)
```

These limits are remarkably simple. They only depend on the ratio of the depth values defining the slope.

### Substitute Symbolic Parameters with Numeric Values

For the following computations, use these numerical values for the symbolic parameters.

- Gravitational acceleration:  $g = 9.81 \text{ m/sec}^2$
- Depth of the canal:  $H = 1 \text{ m}$
- Depth ratio between the shallow and the deep regions:  $depthratio = 0.04$
- Length of the slope region:  $L = 2 \text{ m}$

```
g = 9.81;
L = 2;
H = 1;
depthratio = 0.04;
```

```
h1 = depthratio*H;
h2 = H;
```

```
L1 = depthratio*L;
L2 = L;
```

```
c1 = sqrt(g*h1);
c2 = sqrt(g*h2);
```

Define the incoming soliton of amplitude  $A$  traveling to the left with constant speed  $c_2$  in the deep water region.

```
A = 0.3;
soliton = @(x,t) A.*sech(sqrt(3/4*g*A/H)*(x/c2+t)).^2;
```

Choose  $Nt$  sample points for  $t$ . The time scale is chosen as a multiple of the (temporal) width of the incoming soliton. Store the corresponding discretized frequencies of the Fourier transform in  $W$ .

```
Nt = 64;
TimeScale = 40*sqrt(4/3*H/A/g);
W = [0, 1:Nt/2 - 1, -(Nt/2 - 1):-1]*2*pi/TimeScale;
```

Choose  $Nx$  sample points in  $x$  direction for each region. Create sample points  $X1$  for the shallow water region,  $X2$  for the deep water region, and  $X12$  for the slope region.

```
Nx = 100;
x_min = L1 - 4;
x_max = L2 + 12;
X12 = linspace(L1, L2, Nx);
X1 = linspace(x_min, L1, Nx);
X2 = linspace(L2, x_max, Nx);
```

Compute the Fourier transform of the incoming soliton on a time grid of  $Nt$  equidistant sample points.

```
S = fft(soliton(-0.8*TimeScale*c2, linspace(0,TimeScale,2*(Nt/2)-1)))';
S = repmat(S,1,Nx);
```

Construct a traveling wave solution in the deep water region based on the Fourier data in  $S$ .

```
soliton = real(ifft(S .* exp(1i*W*X2/c2)));
```

Convert the Fourier modes of the reflected wave in the deep water region to numerical values over a grid in  $(x, \omega)$  space. Multiply these values with the Fourier coefficients in  $S$  and use the function `ifft` to compute the reflected wave in  $(x, t)$  space. Note that the first row of the numeric data  $R$  consists of NaN values because proper numerical evaluation of the symbolic data  $R$  for  $\omega = 0$  is not possible. Define the values in the first row of  $R$  as the low frequency limits.

```
R = double(subs(subs(vpa(subs(R)), w, W), x, X2));
R(1,:) = double((1-sqrt(depthratio)) / (1+sqrt(depthratio)));
reflectedWave = real(ifft(S .* R .* exp(-1i*W*X2/c2)));
```

Use the same approach for the transmitted wave in the shallow water region.

```
T = double(subs(subs(vpa(subs(T)), w, W), x, X1));
T(1,:) = double(2/(1+sqrt(depthratio)));
transmittedWave = real(ifft(S .* T .* exp(1i*W*X1/c1)));
```

Also, use this approach for the slope region.

```
U12 = double(subs(subs(vpa(subs(U(x))), w, W), x, X12));
U12(1,:) = double(2/(1+sqrt(depthratio)));
U12 = real(ifft(S .* U12));
```

### Plot the Solution

For a smoother animation, generate additional sample points using trigonometric interpolation along the columns of the plot data.

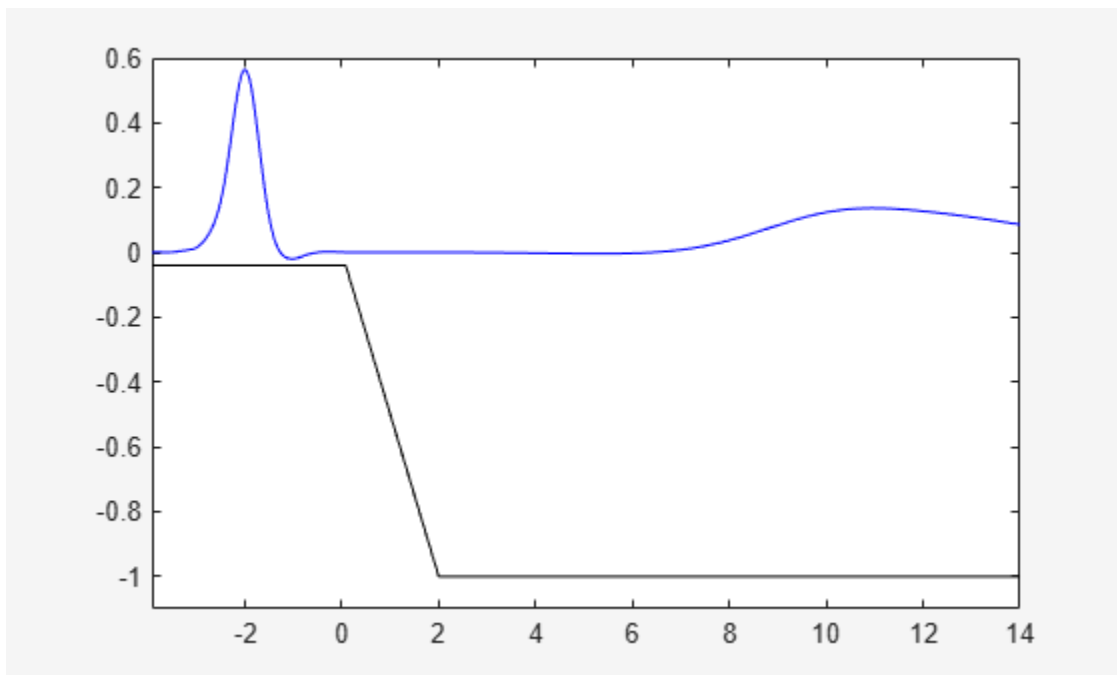
```
soliton = interpft(soliton, 10*Nt);
reflectedWave = interpft(reflectedWave, 10*Nt);
U12 = interpft(U12, 10*Nt);
transmittedWave = interpft(transmittedWave, 10*Nt);
```

Create an animated plot of the solution that shows-up in a separate figure window.

```
figure('Visible', 'on');
plot([x_min, L1, L2, x_max], [-h1, -h1, -h2, -h2], 'Color', 'black')
axis([x_min, x_max, -H-0.1, 0.6])
hold on

line1 = plot(X1,transmittedWave(1,:), 'Color', 'blue');
line12 = plot(X12,U12(1,:), 'Color', 'blue');
line2 = plot(X2,soliton(1,:) + reflectedWave(1,:), 'Color', 'blue');

for t = 2 : size(soliton, 1)*0.35
    line1.YData = transmittedWave(t,:);
    line12.YData = U12(t,:);
    line2.YData = soliton(t,:) + reflectedWave(t,:);
    pause(0.1)
end
```



### More About Tsunamis

In real life, tsunamis have a wavelength of hundreds of kilometers, often traveling at speeds of more than 500 km/hour. (Note that the average depth of the ocean is about 4 km, corresponding to a speed of  $\sqrt{gh} \approx 700 \text{ km/hour}$ .) Over deep sea, the amplitude is rather small, often about 0.5 m or less.



When propagating onto the shelf, however, tsunamis increase their height dramatically: amplitudes of up to 30 m and more were reported.

One interesting phenomenon is that although tsunamis typically approach the coastline as a wave front extending for hundreds of kilometers perpendicular to the direction in which they travel, they do not cause uniform damage along the coast. At some points they cause disasters, whereas only moderate wave phenomena are observed at other places. This is caused by different slopes from the sea bed to the continental shelf. In fact, very steep slopes cause most of the tsunami to be reflected back into the region of deep water, whereas small slopes reflect less of the wave, transmitting a narrow but high wave carrying much energy.

Run the simulation for different values of  $L$ , which correspond to different slopes. The steeper the slope, the lower and less powerful the wave that is transmitted.

Note that this model ignores the dispersion and friction effects. On the shelf, the simulation loses its physical meaning. Here, friction effects are important, causing breaking of the waves.

### References

[1] Derek G. Goring and F. Raichlen, Tsunamis - The Propagation of Long Waves onto a Shelf, *Journal of Waterway, Port, Coastal and Ocean Engineering* 118(1), 1992, pp. 41 - 63.

[2] H. Lamb, *Hydrodynamics*, Dover, 1932.

### See Also

#### Functions

`diff` | `dsolve` | `solve` | `subs`

## Solve Differential Algebraic Equations (DAEs)

This example shows how to solve differential algebraic equations (DAEs) by using MATLAB® and Symbolic Math Toolbox™.

Differential algebraic equations involving functions, or state variables,  $x(t) = [x_1(t), \dots, x_n(t)]$ , have the form

$$F(t, x(t), \dot{x}(t)) = 0$$

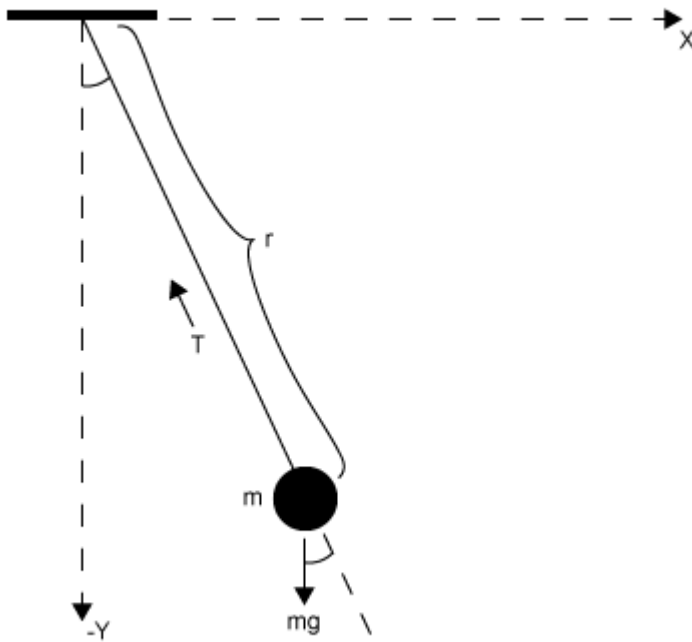
where  $t$  is the independent variable. The number of equations  $F = [F_1, \dots, F_n]$  must match the number of state variables  $x(t) = [x_1(t), \dots, x_n(t)]$ .

Because most DAE systems are not suitable for direct input to MATLAB solvers, such as `ode15i`, first convert them to a suitable form by using Symbolic Math Toolbox functionality. This functionality reduces the differential index (number of differentiations needed to reduce the system to ODEs) of the DAEs to 1 or 0, and then converts the DAE system to numeric function handles suitable for MATLAB solvers. Then, use MATLAB solvers, such as `ode15i`, `ode15s`, or `ode23t`, to solve the DAEs.

Solve your DAE system by completing these steps.

### Step 1: Specify Equations and Variables

The following figure shows the DAE workflow by solving the DAEs for a pendulum.



The state variables are:

- Horizontal position of pendulum  $x(t)$
- Vertical position of pendulum  $y(t)$
- Tension force on pendulum string  $T(t)$

The other variables are:

- Pendulum mass  $m$
- Pendulum length  $r$
- Gravitational constant  $g$

The DAE system of equations is:

$$m \frac{d^2 x}{dt^2} = T(t) \frac{x(t)}{r}$$

$$m \frac{d^2 y}{dt^2} = T(t) \frac{y(t)}{r} - mg$$

$$x(t)^2 + y(t)^2 = r^2$$

Specify independent variables and state variables by using `syms`.

```
syms x(t) y(t) T(t) m r g
```

Specify equations by using the `==` operator.

```
eqn1 = m*diff(x(t),2) == T(t)/r*x(t);
eqn2 = m*diff(y(t),2) == T(t)/r*y(t) - m*g;
eqn3 = x(t)^2 + y(t)^2 == r^2;
eqns = [eqn1 eqn2 eqn3];
```

Place the state variables in a column vector. Store the number of original variables for reference.

```
vars = [x(t); y(t); T(t)];
origVars = length(vars);
```

## Step 2: Reduce Differential Order

### 2.1 (Optional) Check Incidence of Variables

This step is *optional*. You can check where variables occur in the DAE system by viewing the incidence matrix. This step finds any variables that do not occur in your input and can be removed from the `vars` vector.

Display the incidence matrix by using `incidenceMatrix`. The output of `incidenceMatrix` has a row for each equation and a column for each variable. Because the system has three equations and three state variables, `incidenceMatrix` returns a 3-by-3 matrix. The matrix has 1s and 0s, where 1s represent the occurrence of a state variable. For example, the 1 in position (2,3) means the second equation contains the third state variable  $T(t)$ .

```
M = incidenceMatrix(eqns,vars)
```

```
M = 3×3
```

```
     1     0     1
```

$$\begin{array}{ccc} 0 & 1 & 1 \\ 1 & 1 & 0 \end{array}$$

If a column of the incidence matrix is all 0s, then that state variable does not occur in the DAE system and should be removed.

## 2.2 Reduce Differential Order

The *differential order* of a DAE system is the highest differential order of its equations. To solve DAEs using MATLAB, the differential order must be reduced to 1. Here, the first and second equations have second-order derivatives of  $x(t)$  and  $y(t)$ . Thus, the differential order is 2.

Reduce the system to a first-order system by using `reduceDifferentialOrder`. The `reduceDifferentialOrder` function substitutes derivatives with new variables, such as  $Dxt(t)$  and  $Dyt(t)$ . The right side of the expressions in `eqns` is 0.

```
[eqns,vars] = reduceDifferentialOrder(eqns,vars)
```

$$\text{eqns} = \begin{pmatrix} m \frac{\partial}{\partial t} Dxt(t) - \frac{T(t)x(t)}{r} \\ g m + m \frac{\partial}{\partial t} Dyt(t) - \frac{T(t)y(t)}{r} \\ -r^2 + x(t)^2 + y(t)^2 \\ Dxt(t) - \frac{\partial}{\partial t} x(t) \\ Dyt(t) - \frac{\partial}{\partial t} y(t) \end{pmatrix}$$

$$\text{vars} = \begin{pmatrix} x(t) \\ y(t) \\ T(t) \\ Dxt(t) \\ Dyt(t) \end{pmatrix}$$

## Step 3: Check and Reduce Differential Index

### 3.1 Check Differential Index of System

Check the differential index of the DAE system by using `isLowIndexDAE`. If the index is 0 or 1, then `isLowIndexDAE` returns logical 1 (true) and you can skip step 3.2 and go to Step 4. Convert DAE Systems to MATLAB Function Handles. Here, `isLowIndexDAE` returns logical 0 (false), which means the differential index is greater than 1 and must be reduced.

```
isLowIndexDAE(eqns,vars)
```

```
ans = logical
      0
```

### 3.2 Reduce Differential Index with reduceDAEIndex

To reduce the differential index, the `reduceDAEIndex` function adds new equations that are derived from the input equations, and then replaces higher-order derivatives with new variables. If

`reduceDAEIndex` fails and issues a warning, then use the alternative function `reduceDAEtoODE` as described in the workflow “Solve Semilinear DAE System” on page 3-75.

Reduce the differential index of the DAEs described by `eqns` and `vars`.

```
[DAEs,DAEvars] = reduceDAEIndex(eqns,vars)
```

$$\text{DAEs} = \begin{pmatrix} m \, \text{Dxtt}(t) - \frac{T(t) \, x(t)}{r} \\ g \, m + m \, \text{Dytt}(t) - \frac{T(t) \, y(t)}{r} \\ -r^2 + x(t)^2 + y(t)^2 \\ \text{Dxt}(t) - \text{Dxt}_1(t) \\ \text{Dyt}(t) - \text{Dyt}_1(t) \\ 2 \, \text{Dxt}_1(t) \, x(t) + 2 \, \text{Dyt}_1(t) \, y(t) \\ 2 \, y(t) \, \frac{\partial}{\partial t} \, \text{Dyt}_1(t) + 2 \, \text{Dxt}_1(t)^2 + 2 \, \text{Dyt}_1(t)^2 + 2 \, \text{Dxt}_1(t) \, x(t) \\ \text{Dxtt}(t) - \text{Dxt}_1(t) \\ \text{Dytt}(t) - \frac{\partial}{\partial t} \, \text{Dyt}_1(t) \\ \text{Dyt}_1(t) - \frac{\partial}{\partial t} \, y(t) \end{pmatrix}$$

$$\text{DAEvars} = \begin{pmatrix} x(t) \\ y(t) \\ T(t) \\ \text{Dxt}(t) \\ \text{Dyt}(t) \\ \text{Dytt}(t) \\ \text{Dxtt}(t) \\ \text{Dxt}_1(t) \\ \text{Dyt}_1(t) \\ \text{Dxt}_1(t) \end{pmatrix}$$

If `reduceDAEIndex` issues an error or a warning, use the alternative workflow described in “Solve Semilinear DAE System” on page 3-75.

Often, `reduceDAEIndex` introduces redundant equations and variables that can be eliminated. Eliminate redundant equations and variables using `reduceRedundancies`.

```
[DAEs,DAEvars] = reduceRedundancies(DAEs,DAEvars)
```

```
DAEs =
```

$$\begin{pmatrix} -\frac{T(t)x(t) - m r Dxtt(t)}{r} \\ \frac{g m r - T(t)y(t) + m r Dytt(t)}{r} \\ -r^2 + x(t)^2 + y(t)^2 \\ 2 Dxt(t)x(t) + 2 Dyt(t)y(t) \\ 2 Dxt(t)^2 + 2 Dyt(t)^2 + 2 Dxtt(t)x(t) + 2 Dytt(t)y(t) \\ Dytt(t) - \frac{\partial}{\partial t} Dyt(t) \\ Dyt(t) - \frac{\partial}{\partial t} y(t) \end{pmatrix}$$

DAEvars =

$$\begin{pmatrix} x(t) \\ y(t) \\ T(t) \\ Dxt(t) \\ Dyt(t) \\ Dytt(t) \\ Dxtt(t) \end{pmatrix}$$

Check the differential index of the new system. Now, `isLowIndexDAE` returns logical 1 (`true`), which means that the differential index of the system is 0 or 1.

```
isLowIndexDAE(DAEs, DAEvars)
```

```
ans = logical
      1
```

#### Step 4: Convert DAE Systems to MATLAB Function Handles

This step creates function handles for the MATLAB ODE solver `ode15i`, which is a general purpose solver. To use specialized mass matrix solvers such as `ode15s` and `ode23t`, see “Solve DAEs Using Mass Matrix Solvers” on page 3-82 and “Choose an ODE Solver”.

`reduceDAEIndex` outputs a vector of equations in DAEs and a vector of variables in DAEvars. To use `ode15i`, you need a function handle that describes the DAE system.

First, the equations in DAEs can contain symbolic parameters that are not specified in the vector of variables DAEvars. Find these parameters by using `setdiff` on the output of `symvar` from DAEs and DAEvars.

```
pDAEs = symvar(DAEs);
pDAEvars = symvar(DAEvars);
extraParams = setdiff(pDAEs, pDAEvars)

extraParams = (g m r)
```

The extra parameters that you need to specify are the mass `m`, radius `r`, and gravitational constant `g`.

Create the function handle by using `daeFunction`. Specify the extra symbolic parameters as additional input arguments of `daeFunction`.

```
f = daeFunction(DAEs, DAEvars, g, m, r);
```

The rest of the workflow is purely numerical. Set the parameter values and create the function handle for `ode15i`.

```
gVal = 9.81;
mVal = 1;
rVal = 1;
F = @(t,Y,YP) f(t,Y,YP,gVal,mVal,rVal);
```

### Step 5: Find Initial Conditions for Solvers

The `ode15i` solver requires initial values for all variables in the function handle. Find initial values that satisfy the equations by using the MATLAB `decic` function. `decic` accepts guesses (which might not satisfy the equations) for the initial conditions and tries to find satisfactory initial conditions using those guesses. `decic` can fail, in which case you must manually supply consistent initial values for your problem.

First, check the variables in `DAEvars`.

`DAEvars`

```
DAEvars =
    (
      x(t)
      y(t)
      T(t)
      Dxt(t)
      Dyt(t)
      Dytt(t)
      Dxtt(t)
    )
```

Here, `Dxt(t)` is the first derivative of `x(t)`, `Dytt(t)` is the second derivative of `y(t)`, and so on. There are 7 variables in a 7-by-1 vector. Therefore, guesses for initial values of variables and their derivatives must also be 7-by-1 vectors.

Assume the initial angular displacement of the pendulum is  $30^\circ$  or  $\pi/6$ , and the origin of the coordinates is at the suspension point of the pendulum. Given that we used a radius `rVal` of 1, the initial horizontal position `x(t)` is `rVal*sin(pi/6)`. The initial vertical position `y(t)` is `-rVal*cos(pi/6)`. Specify these initial values of the variables in the vector `y0est`.

Arbitrarily set the initial values of the remaining variables and their derivatives to 0. These are not good guesses. However, they suffice for this problem. In your problem, if `decic` errors, then provide better guesses and refer to `decic`.

```
y0est = [rVal*sin(pi/6); -rVal*cos(pi/6); 0; 0; 0; 0; 0];
yp0est = zeros(7,1);
```

Create an option set that specifies numerical tolerances for the numerical search.

```
opt = odeset(RelTol=1e-7,AbsTol=1e-7);
```

Find consistent initial values for the variables and their derivatives by using `decic`.

```
[y0,yp0] = decic(F,0,y0est,[],yp0est,[],opt)
```

```
y0 = 7x1
```

```

    0.4771
   -0.8788
   -8.6214
         0
   -0.0000
   -2.2333
   -4.1135

yp0 = 7×1

         0
   -0.0000
         0
         0
   -2.2333
         0
         0

```

### Step 6: Solve DAEs Using ode15i

Solve the system integrating over the time span  $0 \leq t \leq 0.5$ . Add the grid lines and the legend to the plot.

```

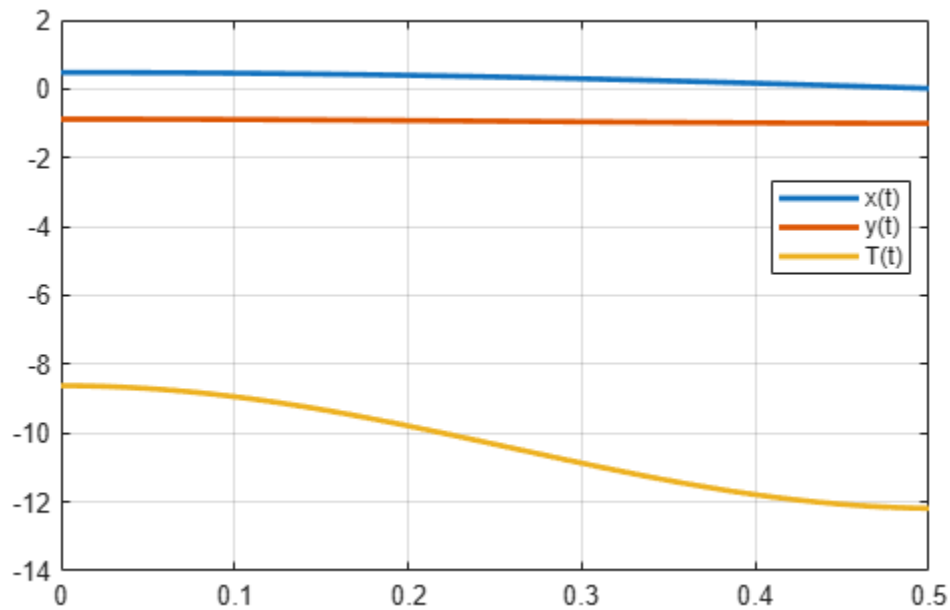
[tSol,ySol] = ode15i(F,[0 0.5],y0,yp0,opt);
plot(tSol,ySol(:,1:origVars),LineWidth=2)

for k = 1:origVars
    S{k} = char(DAEvars(k));
end

legend(S,Location="best")
grid on

```





Solve the system for different parameter values by setting the new value and regenerating the function handle and initial conditions.

Set `rVal` to 2 and regenerate the function handle and initial conditions.

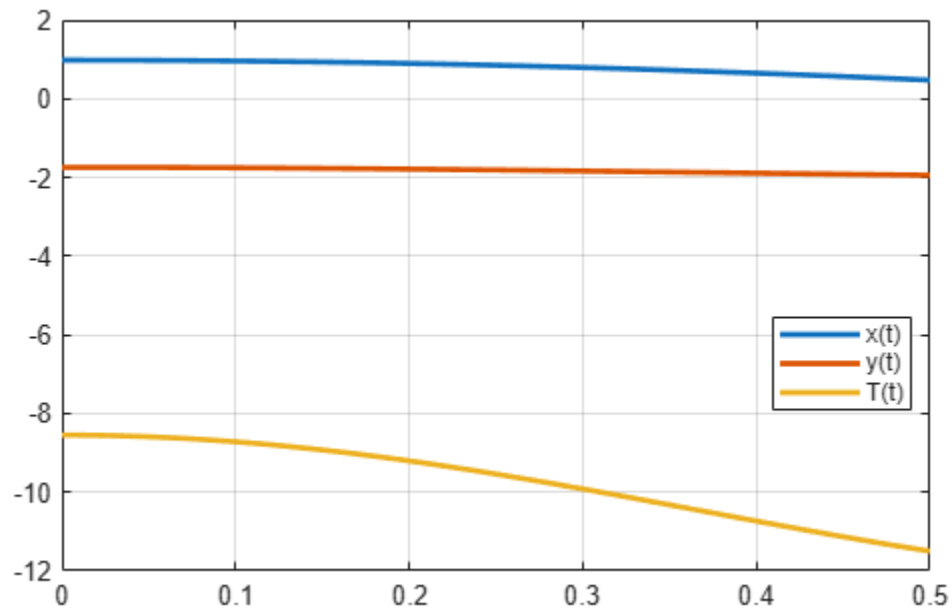
```
rVal = 2;
F = @(t,Y,YP) f(t,Y,YP,gVal,mVal,rVal);

y0est = [rVal*sin(pi/6); -rVal*cos(pi/6); 0; 0; 0; 0; 0];
[y0,yp0] = decic(F,0,y0est,[],yp0est,[],opt);
```

Solve the system for the new parameter value.

```
[tSol,y] = ode15i(F,[0 0.5],y0,yp0,opt);
plot(tSol,y(:,1:origVars),LineWidth=2)
```

```
for k = 1:origVars
    S{k} = char(DAEvars(k));
end
legend(S,Location="best")
grid on
```



## See Also

### Related Examples

- “Solve Semilinear DAE System” on page 3-75
- “Solve DAEs Using Mass Matrix Solvers” on page 3-82

## Solve Semilinear DAE System

This workflow is an alternative workflow to solving semilinear differential algebraic equations (DAEs), used only if `reduceDAEIndex` failed in the standard workflow with the warning message: The index of the reduced DAEs is larger than 1. For the standard workflow, see “Solve Differential Algebraic Equations (DAEs)” on page 3-66.

Complete steps 1 and 2 in “Solve Differential Algebraic Equations (DAEs)” on page 3-66 before beginning other steps. Then, in step 3, if `reduceDAEIndex` fails, reduce the differential index using `reduceDAETooDE`. The advantage of `reduceDAETooDE` is that it reliably reduces semilinear DAEs to ODEs (DAEs of index 0). However, this function is slower and works only on semilinear DAE systems. `reduceDAETooDE` can fail if the system is not semilinear.

To solve your DAE system, complete these steps.

### Step 1. Specify Equations and Variables

The DAE system of equations is:

$$\begin{aligned} m \frac{d^2 x}{dt^2} &= T(t) \frac{x(t)}{r} \\ m \frac{d^2 y}{dt^2} &= T(t) \frac{y(t)}{r} - mg \\ x(t)^2 + y(t)^2 &= r^2 \end{aligned}$$

Specify independent variables and state variables by using `syms`.

```
syms x(t) y(t) T(t) m r g
```

Specify equations by using the `==` operator.

```
eqn1 = m*diff(x(t),2) == T(t)/r*x(t);
eqn2 = m*diff(y(t),2) == T(t)/r*y(t) - m*g;
eqn3 = x(t)^2 + y(t)^2 == r^2;
eqns = [eqn1 eqn2 eqn3];
```

Place the state variables in a column vector. Store the number of original variables for reference.

```
vars = [x(t); y(t); T(t)];
origVars = length(vars);
```

### Step 2. Reduce Differential Order

The *differential order* of a DAE system is the highest differential order of its equations. To solve DAEs using MATLAB®, the differential order must be reduced to 1. Here, the first and second equations have second-order derivatives of  $x(t)$  and  $y(t)$ . Thus, the differential order is 2.

Reduce the system to a first-order system by using `reduceDifferentialOrder`. The `reduceDifferentialOrder` function substitutes derivatives with new variables, such as  $Dx(t)$  and  $Dy(t)$ . The right side of the expressions in `eqns` is 0.

```
[eqns,vars] = reduceDifferentialOrder(eqns,vars)
```

$$\text{eqns} = \begin{pmatrix} m \frac{\partial}{\partial t} \text{Dxt}(t) - \frac{T(t) x(t)}{r} \\ g m + m \frac{\partial}{\partial t} \text{Dyt}(t) - \frac{T(t) y(t)}{r} \\ -r^2 + x(t)^2 + y(t)^2 \\ \text{Dxt}(t) - \frac{\partial}{\partial t} x(t) \\ \text{Dyt}(t) - \frac{\partial}{\partial t} y(t) \end{pmatrix}$$

$$\text{vars} = \begin{pmatrix} x(t) \\ y(t) \\ T(t) \\ \text{Dxt}(t) \\ \text{Dyt}(t) \end{pmatrix}$$

### Step 3. Reduce Differential Index with reduceDAEToODE

To reduce the differential index of the DAEs described by `eqns` and `vars`, use `reduceDAEToODE`. To reduce the index, `reduceDAEToODE` adds new variables and equations to the system. `reduceDAEToODE` also returns constraints, which are conditions that help find initial values to ensure that the resulting ODEs are equal to the initial DAEs.

$$[\text{ODEs}, \text{constraints}] = \text{reduceDAEToODE}(\text{eqns}, \text{vars})$$

$$\text{ODEs} = \begin{pmatrix} \text{Dxt}(t) - \frac{\partial}{\partial t} x(t) \\ \text{Dyt}(t) - \frac{\partial}{\partial t} y(t) \\ m \frac{\partial}{\partial t} \text{Dxt}(t) - \frac{T(t) x(t)}{r} \\ m \frac{\partial}{\partial t} \text{Dyt}(t) - \frac{T(t) y(t) - g m r}{r} \\ - (4 T(t) y(t) - 2 g m r) \frac{\partial}{\partial t} y(t) - (2 x(t)^2 + 2 y(t)^2) \frac{\partial}{\partial t} T(t) - 4 T(t) x(t) \frac{\partial}{\partial t} x(t) - 4 m r \text{Dxt}(t) \frac{\partial}{\partial t} \text{Dxt}(t) - 4 m r \text{Dyt}(t) \frac{\partial}{\partial t} \text{Dyt}(t) \end{pmatrix}$$

$$\text{constraints} = \begin{pmatrix} 2 \text{Dxt}(t) x(t) + 2 \text{Dyt}(t) y(t) \\ r^2 - x(t)^2 - y(t)^2 \\ -2 m r \text{Dxt}(t)^2 - 2 m r \text{Dyt}(t)^2 - 2 T(t) x(t)^2 - 2 T(t) y(t)^2 + 2 g m r y(t) \end{pmatrix}$$

#### Step 4. ODEs to Function Handles for ode15s and ode23t

From the output of `reduceDAEToODE`, you have a vector of equations in `ODEs` and a vector of variables in `vars`. To use `ode15s` or `ode23t`, you need two function handles: one representing the mass matrix of the ODE system, and the other representing the vector containing the right sides of the mass matrix equations. These function handles are the equivalent mass matrix representation of the ODE system where  $M(t,y(t))y'(t) = f(t,y(t))$ .

Find these function handles by using `massMatrixForm` to get the mass matrix `massM` ( $M$  in the equation) and right sides `f`.

```
[massM,f] = massMatrixForm(ODEs,vars)
```

$$\text{massM} = \begin{pmatrix} -1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & m & 0 \\ 0 & 0 & 0 & 0 & m \\ -4 T(t) x(t) / 2 g m r - 4 T(t) y(t) / 2 x(t)^2 - 2 y(t)^2 & -4 m r Dxt(t) & -4 m r Dyt(t) \end{pmatrix}$$

$$f = \begin{pmatrix} -Dxt(t) \\ -Dyt(t) \\ \frac{T(t) x(t)}{r} \\ \frac{T(t) y(t) - g m r}{r} \\ 0 \end{pmatrix}$$

The equations in `ODEs` can contain symbolic parameters that are not specified in the vector of variables `vars`. Find these parameters by using `setdiff` on the output of `symvar` from `ODEs` and `vars`.

```
pODEs = symvar(ODEs);
pvars = symvar(vars);
extraParams = setdiff(pODEs,pvars)
```

```
extraParams = (g m r)
```

The extra parameters that you need to specify are the mass  $m$ , radius  $r$ , and gravitational constant  $g$ .

Convert `massM` and `f` to function handles using `odeFunction`. Specify the extra symbolic parameters as additional inputs to `odeFunction`.

```
massM = odeFunction(massM,vars,m,r,g);
f = odeFunction(f,vars,m,r,g);
```

The rest of the workflow is purely numerical. Set the parameter values and substitute the parameter values in `DAEs` and `constraints`.

```
mVal = 1;
rVal = 1;
gVal = 9.81;
ODEsNumeric = subs(ODEs,[m r g],[mVal rVal gVal]);
constraintsNumeric = subs(constraints,[m r g],[mVal rVal gVal]);
```

Create the function handle suitable for input to `ode15s` or `ode23s`.

```
M = @(t,Y) massM(t,Y,mVal,rVal,gVal);
F = @(t,Y) f(t,Y,mVal,rVal,gVal);
```

### Step 5. Initial Conditions for `ode15s` and `ode23t`

The solvers require initial values for all variables in the function handle. Find initial values that satisfy the equations by using the MATLAB `decic` function. The `decic` accepts guesses (which might not satisfy the equations) for the initial conditions and tries to find satisfactory initial conditions using those guesses. `decic` can fail, in which case you must manually supply consistent initial values for your problem.

First, check the variables in `vars`.

```
vars
```

```
vars =
    x(t)
    y(t)
    T(t)
    Dxt(t)
    Dyt(t)
```

Here, `Dxt(t)` is the first derivative of `x(t)`, and so on. There are 5 variables in a 5-by-1 vector. Therefore, guesses for initial values of variables and their derivatives must also be 5-by-1 vectors.

Assume the initial angular displacement of the pendulum is  $30^\circ$  or  $\pi/6$ , and the origin of the coordinates is at the suspension point of the pendulum. Given that we used a radius `rVal` of 1, the initial horizontal position `x(t)` is `rVal*sin(pi/6)`. The initial vertical position `y(t)` is `-rVal*cos(pi/6)`. Specify these initial values of the variables in the vector `y0est`.

Arbitrarily set the initial values of the remaining variables and their derivatives to 0. These are not good guesses. However, they suffice for this problem. In your problem, if `decic` errors, then provide better guesses and refer to the `decic` page.

```
y0est = [rVal*sin(pi/6); -rVal*cos(pi/6); 0; 0; 0];
yp0est = zeros(5,1);
```

Create an option set that contains the mass matrix `M` of the system and specifies numerical tolerances for the numerical search.

```
opt = odeset(Mass=M,RelTol=1e-7,AbsTol=1e-7);
```

Find initial values consistent with the system of ODEs and with the algebraic constraints by using `decic`. The parameter `[1,0,0,0,1]` in this function call fixes the first and the last element in `y0est`, so that `decic` does not change them during the numerical search. Here, this fixing is necessary to ensure `decic` finds satisfactory initial conditions.

```
[y0,yp0] = decic(ODEsNumeric,vars,constraintsNumeric,0, ...
    y0est,[1,0,0,0,1],yp0est,opt)
```

```
y0 = 5×1
    0.5000
   -0.8660
```

```
-8.4957
-0.0000
0
```

```
yp0 = 5×1
```

```
-0.0000
0
-0.0000
-4.2479
-2.4525
```

Now create an option set that contains the mass matrix  $M$  of the system and the vector  $yp0$  of consistent initial values for the derivatives. You will use this option set when solving the system.

```
opt = odeset(opt,InitialSlope=yp0);
```

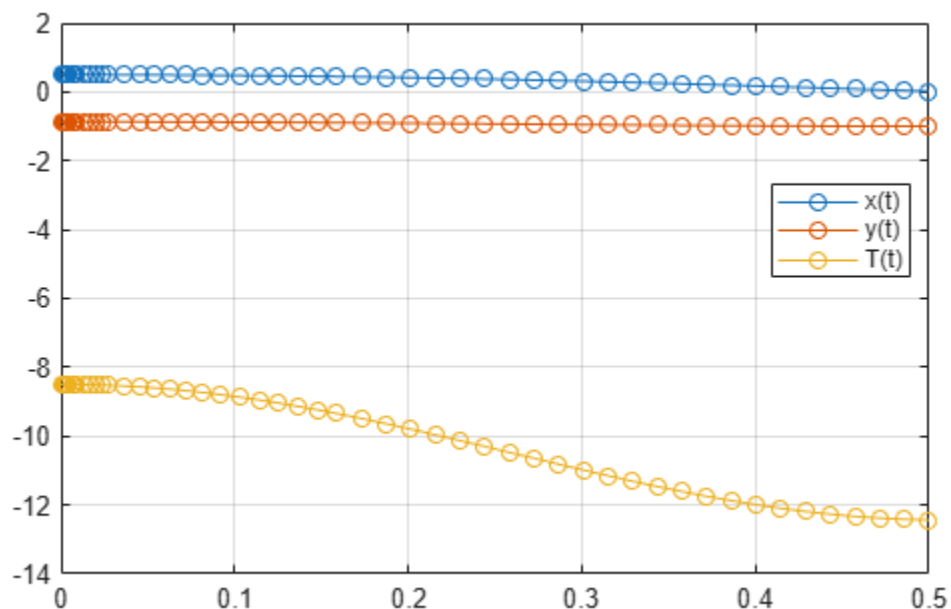
### Step 6. Solve an ODE System with ode15s or ode23t

Solve the system integrating over the time span  $0 \leq t \leq 0.5$ . Add the grid lines and the legend to the plot. Use ode23s by replacing ode15s with ode23s.

```
[tSol,ySol] = ode15s(F,[0 0.5],y0,opt);
plot(tSol,ySol(:,1:origVars),'-o')
```

```
for k = 1:origVars
    S{k} = char(vars(k));
end
```

```
legend(S, Location="best")
grid on
```



Solve the system for different parameter values by setting the new value and regenerating the function handle and initial conditions.

Set `rVal` to 2 and repeat the steps.

```
rVal = 2;

ODEsNumeric = subs(ODEs,[m r g],[mVal rVal gVal]);
constraintsNumeric = subs(constraints,[m r g],[mVal rVal gVal]);
M = @(t,Y) massM(t,Y,mVal,rVal,gVal);
F = @(t,Y) f(t,Y,mVal,rVal,gVal);

y0est = [rVal*sin(pi/6); -rVal*cos(pi/6); 0; 0; 0];
opt = odeset(Mass=M,RelTol=1e-7,AbsTol=1e-7);
[y0, yp0] = decic(ODEsNumeric,vars,constraintsNumeric,0, ...
    y0est,[1,0,0,0,1],yp0est,opt);

opt = odeset(opt,InitialSlope=yp0);

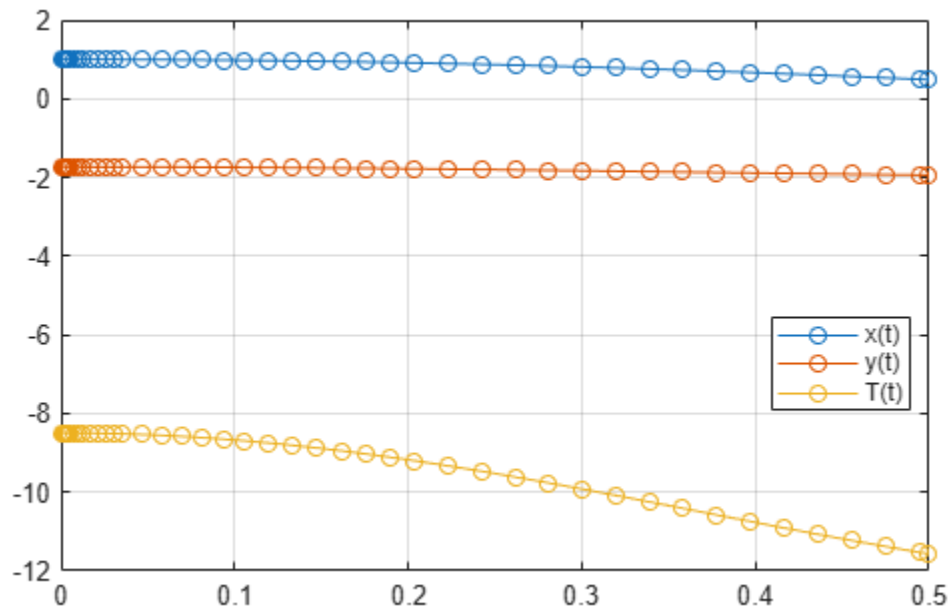
Solve the system for the new parameter value.

[tSol,ySol] = ode15s(F,[0, 0.5],y0,opt);
plot(tSol,ySol(:,1:origVars),'-o')

for k = 1:origVars
    S{k} = char(vars(k));
end

legend(S,Location="best")
grid on
```





### See Also

[daeFunction](#) | [decic](#) | [findDecoupledBlocks](#) | [incidenceMatrix](#) | [isLowIndexDAE](#) | [massMatrixForm](#) | [odeFunction](#) | [reduceDAEIndex](#) | [reduceDAEToODE](#) | [reduceDifferentialOrder](#) | [reduceRedundancies](#)

### Related Examples

- “Solve Differential Algebraic Equations (DAEs)” on page 3-66
- “Solve DAEs Using Mass Matrix Solvers” on page 3-82

## Solve DAEs Using Mass Matrix Solvers

Solve differential algebraic equations by using one of the mass matrix solvers available in MATLAB®. To use this workflow, first complete steps 1, 2, and 3 from “Solve Differential Algebraic Equations (DAEs)” on page 3-66. Then, use a mass matrix solver instead of `ode15i`.

This example demonstrates the use of `ode15s` or `ode23t`. For details on the other solvers, see “Choose an ODE Solver” and adapt the workflow on this page.

### Step 1. Convert DAEs to Function Handles

The code from steps 1 to 3 of “Solve Differential Algebraic Equations (DAEs)” on page 3-66 is:

```
syms x(t) y(t) T(t) m r g
eqn1 = m*diff(x(t), 2) == T(t)/r*x(t);
eqn2 = m*diff(y(t), 2) == T(t)/r*y(t) - m*g;
eqn3 = x(t)^2 + y(t)^2 == r^2;
eqns = [eqn1 eqn2 eqn3];
vars = [x(t); y(t); T(t)];
origVars = length(vars);
[eqns,vars] = reduceDifferentialOrder(eqns,vars);
[DAEs,DAEvars] = reduceDAEIndex(eqns,vars);
[DAEs,DAEvars] = reduceRedundancies(DAEs,DAEvars);
```

From the output of `reduceDAEIndex` and `reduceRedundancies`, you have a vector of equations `DAEs` and a vector of variables `DAEvars`. To use `ode15s` or `ode23t`, you need two function handles: one representing the mass matrix of a DAE system, and the other representing the right sides of the mass matrix equations. These function handles form the equivalent mass matrix representation of the ODE system where  $M(t, y(t)) y'(t) = f(t, y(t))$ .

Find these function handles by using `massMatrixForm` to get the mass matrix `M` and the right sides `F`.

```
[M,f] = massMatrixForm(DAEs,DAEvars)
```

```
M =
    (
    0 0 0 0 0 0 0
    0 0 0 0 0 0 0
    0 0 0 0 0 0 0
    0 0 0 0 0 0 0
    0 0 0 0 0 0 0
    0 0 0 0 -1 0 0
    0 -1 0 0 0 0 0
    )
```

```
f =
```

$$\begin{pmatrix} \frac{T(t)x(t) - m r Dxtt(t)}{r} \\ -\frac{g m r - T(t)y(t) + m r Dytt(t)}{r} \\ r^2 - x(t)^2 - y(t)^2 \\ -2 Dxt(t)x(t) - 2 Dyt(t)y(t) \\ -2 Dxt(t)^2 - 2 Dyt(t)^2 - 2 Dxtt(t)x(t) - 2 Dytt(t)y(t) \\ -Dytt(t) \\ -Dyt(t) \end{pmatrix}$$

The equations in DAEs can contain symbolic parameters that are not specified in the vector of variables `DAEvars`. Find these parameters by using `setdiff` on the output of `symvar` from `DAEs` and `DAEvars`.

```
pDAEs = symvar(DAEs);
pDAEvars = symvar(DAEvars);
extraParams = setdiff(pDAEs,pDAEvars)
```

```
extraParams = (g m r)
```

The mass matrix `M` does not have these extra parameters. Therefore, convert `M` directly to a function handle by using `odeFunction`.

```
M = odeFunction(M,DAEvars);
```

Convert `f` to a function handle. Specify the extra parameters as additional inputs to `odeFunction`.

```
f = odeFunction(f,DAEvars,g,m,r);
```

The rest of the workflow is purely numerical. Set parameter values and create the function handle.

```
gVal = 9.81;
mVal = 1;
rVal = 1;
F = @(t,Y) f(t,Y,gVal,mVal,rVal);
```

## Step 2. Find Initial Conditions

The solvers require initial values for all variables in the function handle. Find initial values that satisfy the equations by using the MATLAB `decic` function. The `decic` accepts guesses (which might not satisfy the equations) for the initial conditions, and tries to find satisfactory initial conditions using those guesses. `decic` can fail, in which case you must manually supply consistent initial values for your problem.

First, check the variables in `DAEvars`.

```
DAEvars
```

```
DAEvars =
    x(t)
    y(t)
    T(t)
    Dxt(t)
    Dyt(t)
    Dytt(t)
    Dxtt(t)
```

Here,  $Dx(t)$  is the first derivative of  $x(t)$ ,  $Dy(t)$  is the second derivative of  $y(t)$ , and so on. There are 7 variables in a 7-by-1 vector. Thus, guesses for initial values of variables and their derivatives must also be 7-by-1 vectors.

Assume the initial angular displacement of the pendulum is  $30^\circ$  or  $\pi/6$ , and the origin of the coordinates is at the suspension point of the pendulum. Given that we used a radius `rVal` of 1, the initial horizontal position  $x(t)$  is  $rVal \cdot \sin(\pi/6)$ . The initial vertical position  $y(t)$  is  $-rVal \cdot \cos(\pi/6)$ . Specify these initial values of the variables in the vector `y0est`.

Arbitrarily set the initial values of the remaining variables and their derivatives to 0. These are not good guesses. However, they suffice for our problem. In your problem, if `decic` errors, then provide better guesses and refer to the `decic` page.

```
y0est = [rVal*sin(pi/6); -rVal*cos(pi/6); 0; 0; 0; 0; 0];
yp0est = zeros(7,1);
```

Create an option set that contains the mass matrix `M` and initial guesses `yp0est`, and specifies numerical tolerances for the numerical search.

```
opt = odeset(Mass=M,InitialSlope=yp0est, ...
    RelTol=1e-7,AbsTol=1e-7);
```

Find consistent initial values for the variables and their derivatives by using the MATLAB `decic` function. The first argument of `decic` must be a function handle describing the DAE as  $f(t, y, yp) = f(t, y, y') = 0$ . In terms of `M` and `F`, this means  $f(t, y, yp) = M(t, y) * yp - F(t, y)$ .

```
implicitDAE = @(t,y,yp) M(t,y)*yp - F(t,y);
[y0,yp0] = decic(implicitDAE,0,y0est,[],yp0est,[],opt)
```

```
y0 = 7×1
    0.4771
   -0.8788
   -8.6214
         0
   -0.0000
   -2.2333
   -4.1135

yp0 = 7×1
         0
   -0.0000
         0
         0
   -2.2333
         0
         0
```

Now create an option set that contains the mass matrix `M` of the system and the vector `yp0` of consistent initial values for the derivatives. You will use this option set when solving the system.

```
opt = odeset(opt,InitialSlope=yp0);
```

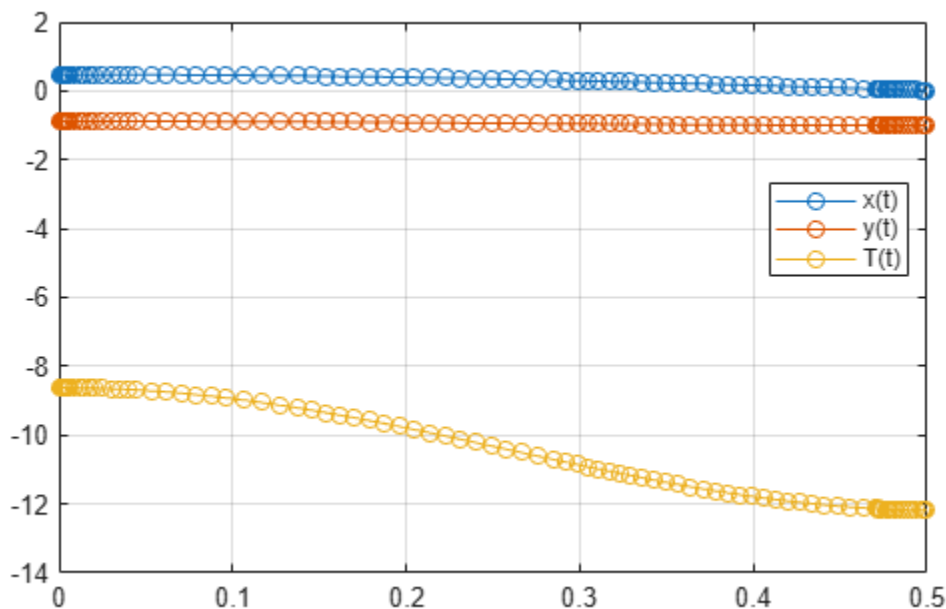
### Step 3. Solve DAE System

Solve the system integrating over the time span  $0 \leq t \leq 0.5$ . Add the grid lines and the legend to the plot. The code uses `ode15s`. Instead, you can also use `ode23t` by replacing `ode15s` with `ode23t`.

```
[tSol,ySol] = ode15s(F,[0, 0.5],y0,opt);
plot(tSol,ySol(:,1:origVars),'-o')
```

```
for k = 1:origVars
    S{k} = char(DAEvars(k));
end
```

```
legend(S,Location="best")
grid on
```



Solve the system for different parameter values by setting the new value and regenerating the function handle and initial conditions.

Set `rVal` to 2 and regenerate the function handle and initial conditions.

```
rVal = 2;

F = @(t,Y) f(t,Y,gVal,mVal,rVal);
y0est = [rVal*sin(pi/6); -rVal*cos(pi/6); 0; 0; 0; 0; 0; 0];
implicitDAE = @(t,y,yp) M(t,y)*yp - F(t,y);
[y0,yp0] = decic(implicitDAE,0,y0est,[],yp0est,[],opt);

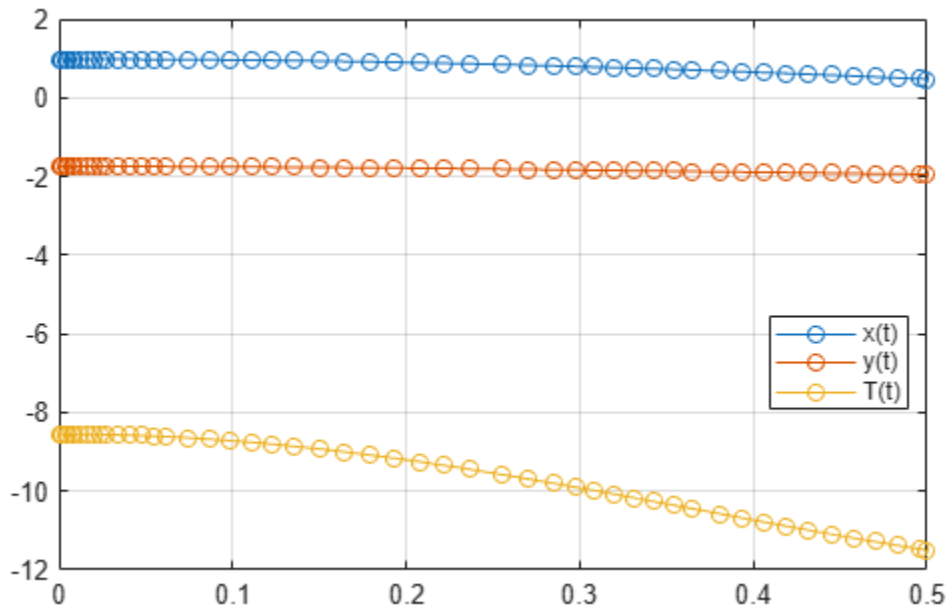
opt = odeset(opt,InitialSlope=yp0);
```

Solve the system for the new parameter value.

```
[tSol,ySol] = ode15s(F,[0, 0.5],y0,opt);
plot(tSol,ySol(:,1:origVars),'-o')
```

```
for k = 1:origVars
    S{k} = char(DAEvars(k));
end

legend(S,Location="best")
grid on
```



## See Also

### Related Examples

- “Solve Differential Algebraic Equations (DAEs)” on page 3-66
- “Solve Semilinear DAE System” on page 3-75

## Analyze and Manipulate Differential Algebraic Equations

This example shows how to solve differential algebraic equations (DAEs) of high differential index using Symbolic Math Toolbox™.

Engineers often specify the behavior of their physical objects (mechanical systems, electrical devices, and so on) by a mixture of differential equations and algebraic equations. MATLAB® provides special numerical solvers, such as `ode15i` and `ode15s`, capable of integrating such DAEs — provided that their differential index does not exceed 1.

This example shows the workflow from setting up the model as a system of differential equations with algebraic constraints to the numerical simulation. The following Symbolic Math Toolbox functions are used.

- `daeFunction`
- `findDecoupledBlocks`
- `incidenceMatrix`
- `isOfLowDAEIndex`
- `reduceDifferentialOrder`
- `massMatrixForm`
- `reduceDAEIndex`
- `reduceDAETooDE`
- `reduceRedundancies`
- `sym/decic`

### Define Parameters of the Model

Consider a 2-D physical pendulum, consisting of a mass  $m$  attached to the origin by a string of constant length  $r$ . Only the gravitational acceleration  $g = 9.81 \text{ m/s}^2$  acts on the mass. The model consists of second-order differential equation for the position  $(x(t), y(t))$  of the mass with an unknown force  $F(t)$  inside the string which serves for keeping the mass on the circle. The force is directed along the string.

```
syms x(t) y(t) F(t) m g r
eqs = [m*diff(x(t),t,t) == F(t)/r*x(t);
       m*diff(y(t),t,t) == F(t)/r*y(t) - m*g;
       x(t)^2 + y(t)^2 == r^2]
```

$$\text{eqs} = \begin{pmatrix} m \frac{\partial^2}{\partial t^2} x(t) = \frac{F(t) x(t)}{r} \\ m \frac{\partial^2}{\partial t^2} y(t) = \frac{F(t) y(t)}{r} - g m \\ x(t)^2 + y(t)^2 = r^2 \end{pmatrix}$$

```
vars = [x(t), y(t), F(t)]
```

```
vars = (x(t) y(t) F(t))
```

Rewrite this DAE system to a system of first-order differential algebraic equations.

```
[eqs,vars,newVars] = reduceDifferentialOrder(eqs,vars)
```

$$\text{eqs} = \begin{pmatrix} m \frac{\partial}{\partial t} \text{Dxt}(t) - \frac{F(t) x(t)}{r} \\ g m + m \frac{\partial}{\partial t} \text{Dyt}(t) - \frac{F(t) y(t)}{r} \\ -r^2 + x(t)^2 + y(t)^2 \\ \text{Dxt}(t) - \frac{\partial}{\partial t} x(t) \\ \text{Dyt}(t) - \frac{\partial}{\partial t} y(t) \end{pmatrix}$$

$$\text{vars} = \begin{pmatrix} x(t) \\ y(t) \\ F(t) \\ \text{Dxt}(t) \\ \text{Dyt}(t) \end{pmatrix}$$

$$\text{newVars} = \begin{pmatrix} \text{Dxt}(t) \frac{\partial}{\partial t} x(t) \\ \text{Dyt}(t) \frac{\partial}{\partial t} y(t) \end{pmatrix}$$

### Try Solving the High-Index DAE System

Before you can use a numerical MATLAB solver, such as `ode15i`, you must follow these steps.

- 1 Convert the system of DAEs to a MATLAB function handle.
- 2 Choose numerical values for symbolic parameters of the system.
- 3 Set consistent initial conditions.

To convert a DAE system to a MATLAB function handle, use `daeFunction`.

```
F = daeFunction(eqs,vars,[m g r])
```

*F = function\_handle with value:*

```
@(t,in2,in3,in4)[in3(4,:).*in4(:,1)-(in2(3,:).*in2(1,:))./in4(:,3);in3(5,:).*in4(:,1)+in4(:,
```

Assign numerical values to the symbolic parameters of the system:  $m = 1\text{kg}$ ,  $g = 9.18\text{m/s}^2$ , and  $r = 1\text{m}$ .

```
f = @(t,y,yp) F(t,y,yp,[1 9.81 1])
```

*f = function\_handle with value:*

```
@(t,y,yp)F(t,y,yp,[1,9.81,1])
```

The function handle `f` is a suitable input for the numerical solver `ode15i`. The next step is to compute consistent initial conditions. Use `odeset` to set numerical tolerances. Then use the MATLAB `decic` function to compute consistent initial conditions `y0`, `yp0` for the positions and the derivatives at time `t0 = 0`.



```
opt = odeset(RelTol=1e-4,AbsTol=1e-4);
t0 = 0;
[y0,yp0] = decic(f,t0,[0.98;-0.21;zeros(3,1)],[],zeros(5,1),[],opt)
```

```
y0 = 5×1

    0.9777
   -0.2100
         0
         0
         0
```

```
yp0 = 5×1

         0
         0
         0
         0
   -9.8100
```

Test the initial conditions:

```
f(t0, y0, yp0)
```

```
ans = 5×1
10-16 ×

         0
         0
   -0.3469
         0
         0
```

Now you can use `ode15i` to try solving the system. When you call `ode15i`, the integration stops immediately and issues the following warnings.

Warning: Matrix is singular, close to singular or badly scaled.

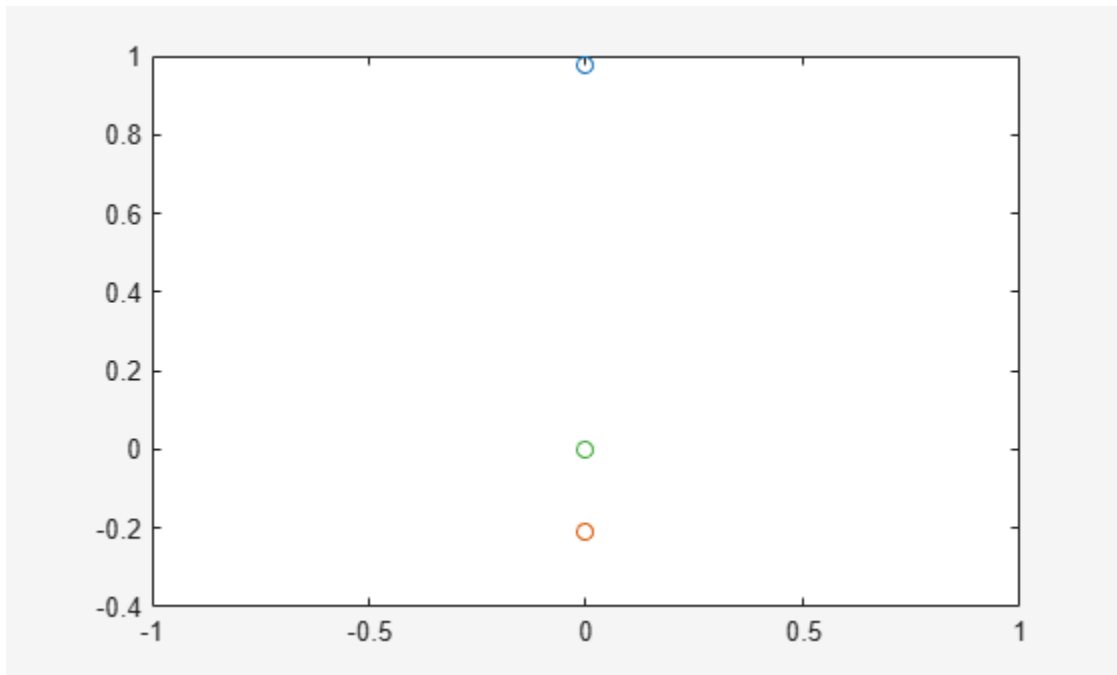
Results may be inaccurate. RCOND = NaN.

Warning: Failure at t=0.000000e+00.

Unable to meet integration tolerances without reducing the step size below the smallest value allowed (0.000000e+00) at time t.

For this example, `ode15i` issues these warnings multiple times. For readability, disable warnings by using `warning("off","all")` before calling `ode15i` and then enable them again.

```
tfinal = 0.5;
s = warning("off","all");
ode15i(f,[t0,tfinal],y0,yp0,opt);
```



```
warning(s)
```

### Analyze and Adjust the DAE System

Check the differential index of the DAE system.

```
isLowIndexDAE(eqs,vars)
```

```
ans = logical
      0
```

This result explains why `ode15i` cannot solve this system. This function requires the input DAE system to be of differential index 0 or 1. Reduce the differential index by extending the model to an equivalent larger DAE system that includes some hidden algebraic constraints.

```
[eqs,vars,newVars,index] = reduceDAEIndex(eqs,vars)
```

```
eqs =
```

$$\begin{pmatrix} m \operatorname{Dxtt}(t) - \frac{F(t) x(t)}{r} \\ g m + m \operatorname{Dytt}(t) - \frac{F(t) y(t)}{r} \\ -r^2 + x(t)^2 + y(t)^2 \\ \operatorname{Dxt}(t) - \operatorname{Dxt}_1(t) \\ \operatorname{Dyt}(t) - \operatorname{Dyt}_1(t) \\ 2 \operatorname{Dxt}_1(t) x(t) + 2 \operatorname{Dyt}_1(t) y(t) \\ 2 y(t) \frac{\partial}{\partial t} \operatorname{Dyt}_1(t) + 2 \operatorname{Dxt}_1(t)^2 + 2 \operatorname{Dyt}_1(t)^2 + 2 \operatorname{Dxt}_1(t) x(t) \\ \operatorname{Dxtt}(t) - \operatorname{Dxt}_1(t) \\ \operatorname{Dytt}(t) - \frac{\partial}{\partial t} \operatorname{Dyt}_1(t) \\ \operatorname{Dyt}_1(t) - \frac{\partial}{\partial t} y(t) \end{pmatrix}$$

vars =

$$\begin{pmatrix} x(t) \\ y(t) \\ F(t) \\ \operatorname{Dxt}(t) \\ \operatorname{Dyt}(t) \\ \operatorname{Dytt}(t) \\ \operatorname{Dxtt}(t) \\ \operatorname{Dxt}_1(t) \\ \operatorname{Dyt}_1(t) \\ \operatorname{Dxt}_1(t) \end{pmatrix}$$

newVars =

$$\begin{pmatrix} \operatorname{Dytt}(t) \frac{\partial}{\partial t} \operatorname{Dyt}(t) \\ \operatorname{Dxtt}(t) \frac{\partial}{\partial t} \operatorname{Dxt}(t) \\ \operatorname{Dxt}_1(t) \frac{\partial}{\partial t} x(t) \\ \operatorname{Dyt}_1(t) \frac{\partial}{\partial t} y(t) \\ \operatorname{Dxt}_1(t) \frac{\partial^2}{\partial t^2} x(t) \end{pmatrix}$$

index =

3

The fourth output shows that the differential index of the original model is three. Simplify the new system.

[eqs,vars,S] = reduceRedundancies(eqs,vars)

eqs =

```


$$\begin{pmatrix} -\frac{F(t)x(t) - m r Dxtt(t)}{r} \\ \frac{g m r - F(t)y(t) + m r Dytt(t)}{r} \\ -r^2 + x(t)^2 + y(t)^2 \\ 2 Dxt(t)x(t) + 2 Dyt(t)y(t) \\ 2 Dxt(t)^2 + 2 Dyt(t)^2 + 2 Dxtt(t)x(t) + 2 Dytt(t)y(t) \\ Dytt(t) - \frac{\partial}{\partial t} Dyt(t) \\ Dyt(t) - \frac{\partial}{\partial t} y(t) \end{pmatrix}$$

vars =

$$\begin{pmatrix} x(t) \\ y(t) \\ F(t) \\ Dxt(t) \\ Dyt(t) \\ Dytt(t) \\ Dxtt(t) \end{pmatrix}$$

S = struct with fields:
    solvedEquations: [3x1 sym]
    constantVariables: [0x2 sym]
    replacedVariables: [3x2 sym]
    otherEquations: [0x1 sym]

```

Check if the new system has a low differential index (0 or 1).

```
isLowIndexDAE(eqs,vars)
```

```
ans = logical
      1
```

### Solve the Low-Index DAE System

Generate a MATLAB function handle that replaces the symbolic parameters by numerical values.

```
F = daeFunction(eqs,vars,[m g r])
```

```
F = function_handle with value:
    @(t,in2,in3,in4)[- (in2(3,:).*in2(1,:)-in2(7,:).*in4(:,1).*in4(:,3))./in4(:,3); (-in2(3,:).*in
```

```
f = @(t,y,yp) F(t,y,yp,[1 9.81 1])
```

```
f = function_handle with value:
    @(t,y,yp)F(t,y,yp,[1,9.81,1])
```

Compute consistent initial conditions for the index reduced by the MATLAB `decic` function. Here, `opt` is the options structure that sets numerical tolerances. You already computed it using `odeset`.

```
[y0,yp0] = decic(f,t0,[0.98;-0.21;zeros(5,1)],[],zeros(7,1),[],opt)
```

```

y0 = 7×1
    0.9779
   -0.2093
   -2.0528
    0.0000
     0
   -9.3804
   -2.0074

```

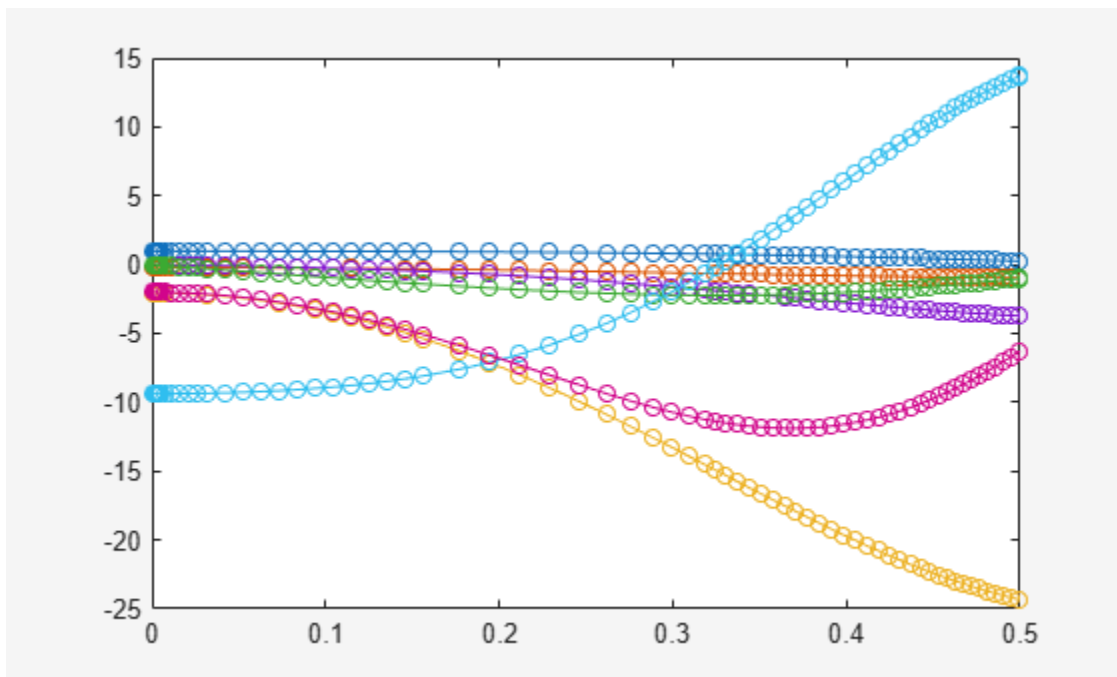
```

yp0 = 7×1
     0
     0
     0
     0
   -9.3804
     0
     0

```

Solve the system and plot the solution.

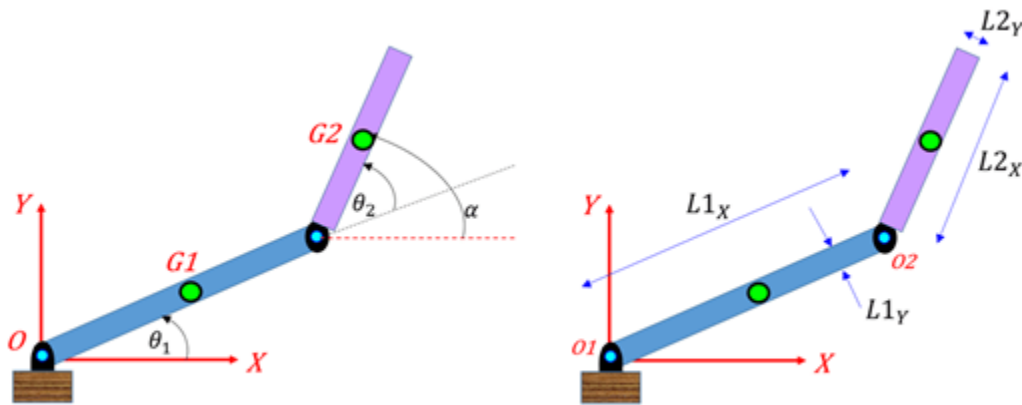
```
ode15i(f,[t0 tfinal],y0,yp0,opt)
```



## Derive and Apply Inverse Kinematics to Two-Link Robot Arm

This example shows how to derive and apply inverse kinematics to a two-link robot arm by using MATLAB® and Symbolic Math Toolbox™.

The example defines the joint parameters and end-effector locations symbolically, calculates and visualizes the forward and inverse kinematics solutions, and finds the system Jacobian, which is useful for simulating the motion of the robot arm.



### Step 1: Define Geometric Parameters

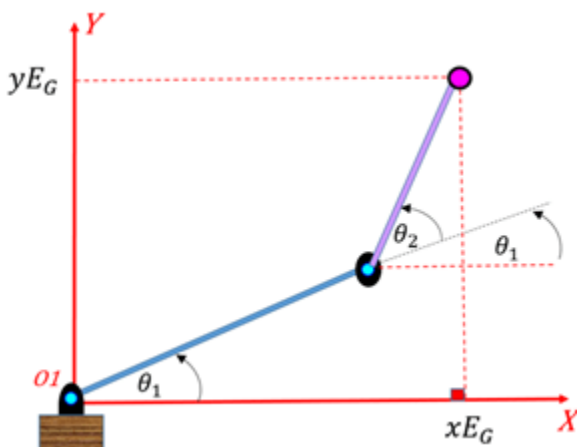
Define the link lengths, joint angles and end-effector locations of the robots as symbolic variables.

```
syms L_1 L_2 theta_1 theta_2 XE YE
```

Specify values for the link lengths of the robot.

```
L1 = 1;  
L2 = 0.5;
```

### Step 2: Define X and Y Coordinates of End Effector



Define the X and Y coordinates of the end-effector as a function of the joint angles ( $\theta_1, \theta_2$ ).

$$XE\_RHS = L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2)$$

$$XE\_RHS = L_2 \cos(\theta_1 + \theta_2) + L_1 \cos(\theta_1)$$

$$YE\_RHS = L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2)$$

$$YE\_RHS = L_2 \sin(\theta_1 + \theta_2) + L_1 \sin(\theta_1)$$

Convert the symbolic expressions into MATLAB functions.

```
XE_MLF = matlabFunction(XE_RHS,Vars=[L_1 L_2 theta_1 theta_2]);
```

```
YE_MLF = matlabFunction(YE_RHS,Vars=[L_1 L_2 theta_1 theta_2]);
```

### Step 3: Calculate and Visualize Forward Kinematics

Forward kinematics transforms the joint angles into end-effector locations:

$(\theta_1, \theta_2) \rightarrow f(\theta_1, \theta_2) \rightarrow (X_E, Y_E)$ . Given specific joint-angle values, use forward kinematics to calculate the end-effector locations.

Specify the input values of the joint angles as  $0^\circ < \theta_1 < 90^\circ$  and  $-180^\circ < \theta_2 < 180^\circ$ .

```
t1_degs_row = linspace(0,90,100);
t2_degs_row = linspace(-180,180,100);
[tt1_degs,tt2_degs] = meshgrid(t1_degs_row,t2_degs_row);
```

Convert the angle units from degrees to radians.

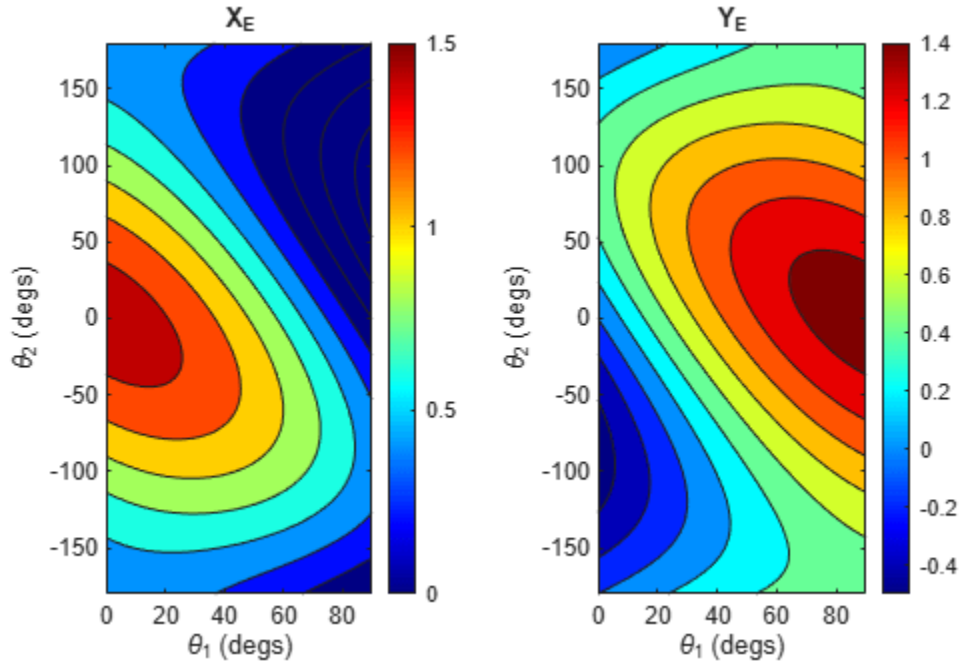
```
tt1_rads = deg2rad(tt1_degs);
tt2_rads = deg2rad(tt2_degs);
```

Calculate the X and Y coordinates using the MATLAB functions XE\_MLF and YE\_MLF, respectively.

```
X_mat = XE_MLF(L1,L2,tt1_rads,tt2_rads);
Y_mat = YE_MLF(L1,L2,tt1_rads,tt2_rads);
```

Visualize the X and Y coordinates using the helper function plot\_XY\_given\_theta\_2dof.

```
plot_XY_given_theta_2dof(tt1_degs,tt2_degs,X_mat,Y_mat,(L1+L2))
```



#### Step 4: Find Inverse Kinematics

Inverse kinematics transforms the end-effector locations into joint angles:

$(X_E, Y_E) \rightarrow g(X_E, Y_E) \rightarrow (\theta_1, \theta_2)$ . Find the inverse kinematics from the forward kinematics equations.

Define the forward kinematics equations.

$X_{E\_EQ} = X_E == X_{E\_RHS};$

$Y_{E\_EQ} = Y_E == Y_{E\_RHS};$

Solve for  $\theta_1$  and  $\theta_2$ .

$S = \text{solve}([X_{E\_EQ} \ Y_{E\_EQ}], [\text{theta}_1 \ \text{theta}_2])$

$S = \text{struct with fields:}$   
 $\text{theta}_1: [2 \times 1 \ \text{sym}]$   
 $\text{theta}_2: [2 \times 1 \ \text{sym}]$

The structure  $S$  represents the multiple solutions for  $\theta_1$  and  $\theta_2$ . Show the pair of solutions for  $\theta_1$ .

$\text{simplify}(S.\text{theta}_1)$

$\text{ans} =$

$$\begin{pmatrix} 2 \operatorname{atan}\left(\frac{2 L_1 Y_E + \sigma_1}{L_1^2 + 2 L_1 X_E - L_2^2 + X_E^2 + Y_E^2}\right) \\ 2 \operatorname{atan}\left(\frac{2 L_1 Y_E - \sigma_1}{L_1^2 + 2 L_1 X_E - L_2^2 + X_E^2 + Y_E^2}\right) \end{pmatrix}$$

where

$$\sigma_1 = \sqrt{-L_1^4 + 2 L_1^2 L_2^2 + 2 L_1^2 X_E^2 + 2 L_1^2 Y_E^2 - L_2^4 + 2 L_2^2 X_E^2 + 2 L_2^2 Y_E^2 - X_E^4 - 2 X_E^2 Y_E^2 - Y_E^4}$$



Show the pair of solutions for  $\theta_2$ .

```
simplify(S.theta_2)
```

```
ans =  
       $\begin{pmatrix} -\sigma_1 \\ \sigma_1 \end{pmatrix}$   
      where
```

$$\sigma_1 = 2 \operatorname{atan} \left( \frac{\sqrt{(-L_1^2 + 2 L_1 L_2 - L_2^2 + XE^2 + YE^2)(L_1^2 + 2 L_1 L_2 + L_2^2 - XE^2 - YE^2)}}{-L_1^2 + 2 L_1 L_2 - L_2^2 + XE^2 + YE^2} \right)$$

Convert the solutions into MATLAB functions that you can use later. The functions TH1\_MLF and TH2\_MLF represent the inverse kinematics.

```
TH1_MLF{1} = matlabFunction(S.theta_1(1),Vars=[L_1 L_2 XE YE]);  
TH1_MLF{2} = matlabFunction(S.theta_1(2),Vars=[L_1 L_2 XE YE]);  
TH2_MLF{1} = matlabFunction(S.theta_2(1),Vars=[L_1 L_2 XE YE]);  
TH2_MLF{2} = matlabFunction(S.theta_2(2),Vars=[L_1 L_2 XE YE]);
```

### Step 5: Calculate and Visualize Inverse Kinematics

Use the inverse kinematics to compute  $\theta_1$  and  $\theta_2$  from the X and Y coordinates.

Define the grid points of the X and Y coordinates.

```
[xmat,ymat] = meshgrid(0:0.01:1.5,0:0.01:1.5);
```

Calculate the angles  $\theta_1$  and  $\theta_2$  using the MATLAB functions TH1\_MLF{1} and TH2\_MLF{1}, respectively.

```
tmp_th1_mat = TH1_MLF{1}(L1,L2,xmat,ymat);  
tmp_th2_mat = TH2_MLF{1}(L1,L2,xmat,ymat);
```

Convert the angle units from radians to degrees.

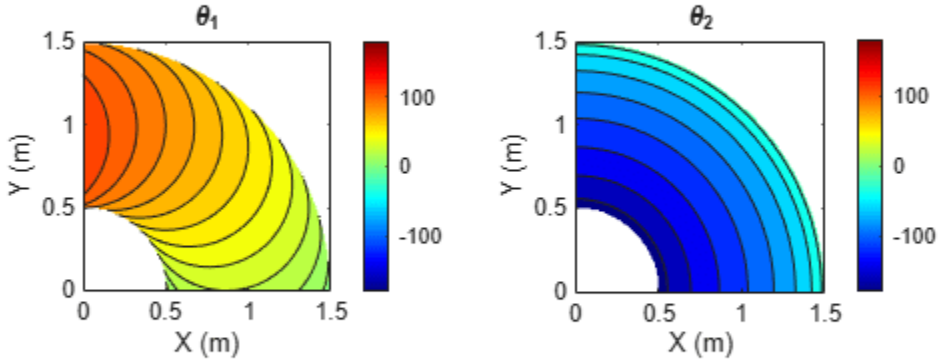
```
tmp_th1_mat = rad2deg(tmp_th1_mat);  
tmp_th2_mat = rad2deg(tmp_th2_mat);
```

Some of the input coordinates, such as (X,Y) = (1.5,1.5), are beyond the reachable workspace of the end effector. The inverse kinematics solutions can generate some imaginary theta values that require correction. Correct the imaginary theta values.

```
th1_mat = NaN(size(tmp_th1_mat));  
th2_mat = NaN(size(tmp_th2_mat));  
  
tf_mat = imag(tmp_th1_mat) == 0;  
th1_mat(tf_mat) = real(tmp_th1_mat(tf_mat));  
  
tf_mat = imag(tmp_th2_mat) == 0;  
th2_mat(tf_mat) = real(tmp_th2_mat(tf_mat));
```

Visualize the angles  $\theta_1$  and  $\theta_2$  using the helper function plot\_theta\_given\_XY\_2dof.

```
plot_theta_given_XY_2dof(xmat,ymat,th1_mat,th2_mat)
```



### Step 6: Compute System Jacobian

The definition of the system Jacobian is:

$$J = \frac{d(X, Y)}{d(\theta_1, \theta_2)} = \begin{pmatrix} \frac{dX}{d\theta_1} & \frac{dX}{d\theta_2} \\ \frac{dY}{d\theta_1} & \frac{dY}{d\theta_2} \end{pmatrix}$$

```
the_J = jacobian([XE_RHS YE_RHS],[theta_1 theta_2])
```

$$\text{the\_J} = \begin{pmatrix} -L_2 \sin(\theta_1 + \theta_2) - L_1 \sin(\theta_1) & -L_2 \sin(\theta_1 + \theta_2) \\ L_2 \cos(\theta_1 + \theta_2) + L_1 \cos(\theta_1) & L_2 \cos(\theta_1 + \theta_2) \end{pmatrix}$$

You can relate the joint velocity to the end-effector velocity and the other way around, by using the system Jacobian  $J$  and its Moore-Penrose pseudoinverse  $J^+$ :

$$\begin{pmatrix} \frac{dX}{dt} \\ \frac{dY}{dt} \end{pmatrix} = \begin{pmatrix} \frac{dX}{d\theta_1} & \frac{dX}{d\theta_2} \\ \frac{dY}{d\theta_1} & \frac{dY}{d\theta_2} \end{pmatrix} \cdot \begin{pmatrix} \frac{d\theta_1}{dt} \\ \frac{d\theta_2}{dt} \end{pmatrix}$$

$$\begin{pmatrix} \frac{dX}{dt} \\ \frac{dY}{dt} \end{pmatrix} = J \cdot \begin{pmatrix} \frac{d\theta_1}{dt} \\ \frac{d\theta_2}{dt} \end{pmatrix}$$

$$\begin{pmatrix} \frac{d\theta_1}{dt} \\ \frac{d\theta_2}{dt} \end{pmatrix} = J^+ \cdot \begin{pmatrix} \frac{dX}{dt} \\ \frac{dY}{dt} \end{pmatrix}$$

You can also convert the symbolic expression of the Jacobian to a MATLAB function block. Simulate the end-effector locations of the robot on a trajectory by defining the multiple waypoints as inputs to a Simulink® model. The Simulink model can calculate a motion-profile based on the joint angle values to reach each waypoint in the trajectory. For more details, see Inverse Kinematics of a 2-link Robot Arm and Teaching Rigid Body Dynamics.

### Helper Functions

```
function plot_theta_given_XY_2dof(X_mat,Y_mat,theta_1_mat_degs,...
                                theta_2_mat_degs)

xlab_str = "X (m)";
ylab_str = "Y (m)";

figure;
hax(1) = subplot(1,2,1);
    contourf(X_mat,Y_mat,theta_1_mat_degs);
    clim(hax(1),[-180 180]);
    colormap(gca,"jet"); colorbar
    xlabel(xlab_str,Interpreter="tex");
    ylabel(ylab_str,Interpreter="tex");
    title(hax(1),"\theta_1",Interpreter="tex")
    axis("equal")
hax(2) = subplot(1,2,2);
    contourf(X_mat,Y_mat,theta_2_mat_degs);
    clim(hax(2),[-180 180]);
    colormap(gca,"jet"); colorbar
    xlabel(xlab_str,Interpreter="tex");
    ylabel(ylab_str,Interpreter="tex");
    title(hax(2),"\theta_2",Interpreter="tex")
    axis("equal")

end

function plot_XY_given_theta_2dof(theta_1_mat_degs,theta_2_mat_degs,...
                                X_mat,Y_mat,a_cmax)

xlab_str = "\theta_1 (degs)";
ylab_str = "\theta_2 (degs)";

figure;
hax(1) = subplot(1,2,1);
    contourf(theta_1_mat_degs,theta_2_mat_degs,X_mat);
    clim(hax(1),[0 a_cmax]);
    colormap(gca,"jet"); colorbar
    xlabel(xlab_str,Interpreter="tex");
    ylabel(ylab_str,Interpreter="tex");
    title(hax(1),"X_E",Interpreter="tex")
hax(2) = subplot(1,2,2);
    contourf(theta_1_mat_degs,theta_2_mat_degs,Y_mat);
    clim(hax(1),[0 a_cmax]);
```

```
colormap(gca,"jet"); colorbar
xlabel(xlab_str,Interpreter="tex");
ylabel(ylab_str,Interpreter="tex");
title(hax(2),"Y_E",Interpreter="tex")

end
```

## See Also

### Functions

matlabFunction | solve | simplify | jacobian

## Gauss-Laguerre Quadrature Evaluation Points and Weights

This example shows how to solve polynomial equations and systems of equations, and work with the results using Symbolic Math Toolbox™.

Gaussian quadrature rules approximate an integral by sums  $\int_a^b f(t)w(t)dt \approx \sum_{i=1}^n f(x_i)\alpha_i$ . Here, the  $x_i$  and  $\alpha_i$  are parameters of the method, depending on  $n$  but not on  $f$ . They follow from the choice of the weight function  $w(t)$ , as follows. Associated to the weight function is a family of orthogonal polynomials. The polynomials' roots are the evaluation points  $x_i$ . Finally, the weights  $\alpha_i$  are determined by the condition that the method be correct for polynomials of small degree. Consider the weight function  $w(t) = \exp(-t)$  on the interval  $[0, \infty]$ . This case is known as the Gauss-Laguerre quadrature.

```
syms t
n = 4;
w(t) = exp(-t);
```

Assume you know the first  $n$  members of the family of orthogonal polynomials. In case of the quadrature rule considered here, they turn out to be the Laguerre polynomials.

```
F = laguerreL(0:n-1, t)
```

```
F =
( 1 1 - t  t^2/2 - 2 t + 1 - t^3/6 + 3 t^2/2 - 3 t + 1 )
```

Let  $L$  be the  $n + 1$  st polynomial, the coefficients of which are still to be determined.

```
X = sym('X', [1, n+1])
```

```
X = (X1 X2 X3 X4 X5)
```

```
L = poly2sym(X, t)
```

```
L = X1 t^4 + X2 t^3 + X3 t^2 + X4 t + X5
```

Represent the orthogonality relations between the Laguerre polynomials  $F$  and  $L$  in a system of equations `sys`.

```
sys = [int(F.*L.*w(t), t, 0, inf) == 0]
```

```
sys = (24 X1 + 6 X2 + 2 X3 + X4 + X5 = 0 -96 X1 - 18 X2 - 4 X3 - X4 = 0 144 X1 + 18 X2 + 2 X3 = 0 -96 X1 - 6 X2 = 0
```

Add the condition that the polynomial have norm 1.

```
sys = [sys, int(L^2.*w(t), 0, inf) == 1]
```

```
sys = (24 X1 + 6 X2 + 2 X3 + X4 + X5 = 0 -96 X1 - 18 X2 - 4 X3 - X4 = 0 144 X1 + 18 X2 + 2 X3 = 0 -96 X1 - 6 X2 = 0 4032 X1^2 + 144 X1 X2 + 18 X2^2 + 2 X3^2 + X4^2 + X5^2 = 1)
```

Solve for the coefficients of  $L$ .

```
S = solve(sys, X)
```

```
S = struct with fields:
    X1: [2×1 sym]
    X2: [2×1 sym]
    X3: [2×1 sym]
    X4: [2×1 sym]
    X5: [2×1 sym]
```

`solve` returns the two solutions in a structure array. Display the solutions.

```
structfun(@display, S)
```

```
ans =
    (
        -1/24
        1/24
    )
```

```
ans =
    (
        2/3
        -2/3
    )
```

```
ans =
    (-3
     3)
```

```
ans =
    (
        4
        -4
    )
```

```
ans =
    (-1
     1)
```

Make the solution unique by imposing an extra condition that the first coefficient be positive:

```
sys = [sys, X(1)>0];
S = solve(sys, X)
```

```
S = struct with fields:
    X1: 1/24
    X2: -2/3
    X3: 3
    X4: -4
    X5: 1
```

Substitute the solution into `L`.

```
L = subs(L, S)
```

```
L =
    t^4/24 - 2t^3/3 + 3t^2 - 4t + 1
```

As expected, this polynomial is the  $|n|$ th Laguerre polynomial:

```
laguerreL(n, t)
```

```
ans =

$$\frac{t^4}{24} - \frac{2t^3}{3} + 3t^2 - 4t + 1$$

```

The evaluation points  $x_i$  are the roots of the polynomial  $L$ . Solve  $L$  for the evaluation points. The roots are expressed in terms of the `root` function.

```
x = solve(L)
```

```
x =

$$\begin{pmatrix} \text{root}(\sigma_1, z, 1) \\ \text{root}(\sigma_1, z, 2) \\ \text{root}(\sigma_1, z, 3) \\ \text{root}(\sigma_1, z, 4) \end{pmatrix}$$

```

where

$$\sigma_1 = z^4 - 16z^3 + 72z^2 - 96z + 24$$

The form of the solutions might suggest that nothing has been achieved, but various operations are available on them. Compute floating-point approximations using `vpa`:

```
vpa(x)
```

```
ans =

$$\begin{pmatrix} 0.32254768961939231180036145910437 \\ 1.7457611011583465756868167125179 \\ 4.5366202969211279832792853849571 \\ 9.3950709123011331292335364434205 \end{pmatrix}$$

```

Some spurious imaginary parts might occur. Prove symbolically that the roots are real numbers:

```
isAlways(in(x, 'real'))
```

```
ans = 4x1 logical array
```

```

1
1
1
1
```

For polynomials of degree less than or equal to 4, you can use `MaxDegree` to obtain the solutions in terms of nested radicals instead in terms of `root`. However, subsequent operations on results of this form would be slow.

```
xradical = solve(L, 'MaxDegree', 4)
```

```
xradical =
```

$$\begin{pmatrix} 4 - \sigma_1 - \sigma_3 \\ 4 + \sigma_1 - \sigma_3 \\ \sigma_3 - \sigma_2 + 4 \\ \sigma_3 + \sigma_2 + 4 \end{pmatrix}$$

where

$$\sigma_1 = \frac{\sqrt{96 \sigma_6 \sigma_4 - 3 \sigma_5 \sigma_4 - 288 \sigma_4 - 512 \sqrt{3} \sqrt{6} \sqrt{6} + \sqrt{3} \sqrt{6} i}}{2 (768 + 128 \sqrt{3} \sqrt{6} i)^{1/6} (144 \sigma_6 + 9 \sigma_5 + 864)^{1/4}}$$

$$\sigma_2 = \frac{\sqrt{96 \sigma_6 \sigma_4 - 3 \sigma_5 \sigma_4 - 288 \sigma_4 + 512 \sqrt{3} \sqrt{6} \sqrt{6} + \sqrt{3} \sqrt{6} i}}{2 (768 + 128 \sqrt{3} \sqrt{6} i)^{1/6} (144 \sigma_6 + 9 \sigma_5 + 864)^{1/4}}$$

$$\sigma_3 = \frac{\sigma_4}{2 (768 + 128 \sqrt{3} \sqrt{6} i)^{1/6}}$$

$$\sigma_4 = \sqrt{16 \sigma_6 + \sigma_5 + 96}$$

$$\sigma_5 = (768 + 128 \sqrt{3} \sqrt{6} i)^{2/3}$$

$$\sigma_6 = (768 + 128 \sqrt{3} \sqrt{6} i)^{1/3}$$

The weights  $\alpha_i$  are given by the condition that for polynomials of degree less than  $n$ , the quadrature rule must produce exact results. It is sufficient if this holds for a basis of the vector space of these polynomials. This condition results in a system of four equations in four variables.

```
y = sym('y', [n, 1]);
sys = sym(zeros(n));
for k=0:n-1
    sys(k+1) = sum(y.*(x.^k)) == int(t^k * w(t), t, 0, inf);
end
sys
```

$$\text{sys} = \begin{pmatrix} y_1 + y_2 + y_3 + y_4 = 1 & 0 & 0 & 0 \\ y_1 \text{root}(\sigma_1, z, 1) + y_2 \text{root}(\sigma_1, z, 2) + y_3 \text{root}(\sigma_1, z, 3) + y_4 \text{root}(\sigma_1, z, 4) = 1 & 0 & 0 & 0 \\ y_1 \text{root}(\sigma_1, z, 1)^2 + y_2 \text{root}(\sigma_1, z, 2)^2 + y_3 \text{root}(\sigma_1, z, 3)^2 + y_4 \text{root}(\sigma_1, z, 4)^2 = 2 & 0 & 0 & 0 \\ y_1 \text{root}(\sigma_1, z, 1)^3 + y_2 \text{root}(\sigma_1, z, 2)^3 + y_3 \text{root}(\sigma_1, z, 3)^3 + y_4 \text{root}(\sigma_1, z, 4)^3 = 6 & 0 & 0 & 0 \end{pmatrix}$$

where

$$\sigma_1 = z^4 - 16 z^3 + 72 z^2 - 96 z + 24$$

Solve the system both numerically and symbolically. The solution is the desired vector of weights  $\alpha_i$ .

```
[a1, a2, a3, a4] = vpasolve(sys, y)
```

```
a1 = 0.60315410434163360163596602381808
```

```
a2 = 0.35741869243779968664149201745809
```

```
a3 = 0.03888790851500538427243816815621
```

```
a4 = 0.00053929470556132745010379056762059
```



```
[alpha1, alpha2, alpha3, alpha4] = solve(sys, y)
```

$$\text{alpha1} = -\frac{\sigma_3 \sigma_2 + \sigma_3 \sigma_1 + \sigma_2 \sigma_1 - \sigma_3 \sigma_2 \sigma_1 - 2 \sigma_3 - 2 \sigma_2 - 2 \sigma_1 + 6}{(\sigma_4 - \sigma_3)(\sigma_4 \sigma_2 + \sigma_4 \sigma_1 - \sigma_2 \sigma_1 - \sigma_4^2)}$$

where

$$\sigma_1 = \text{root}(z^4 - 16z^3 + 72z^2 - 96z + 24, z, 4)$$

$$\sigma_2 = \text{root}(z^4 - 16z^3 + 72z^2 - 96z + 24, z, 3)$$

$$\sigma_3 = \text{root}(z^4 - 16z^3 + 72z^2 - 96z + 24, z, 2)$$

$$\sigma_4 = \text{root}(z^4 - 16z^3 + 72z^2 - 96z + 24, z, 1)$$

$$\text{alpha2} = \frac{\text{root}(\sigma_1, z, 1) \text{root}(\sigma_1, z, 3) + \text{root}(\sigma_1, z, 1) \text{root}(\sigma_1, z, 4) + \text{root}(\sigma_1, z, 3) \text{root}(\sigma_1, z, 4) - \text{root}(\sigma_1, z, 1) \text{root}(\sigma_1, z, 3) \text{root}(\sigma_1, z, 4)}{(\text{root}(\sigma_1, z, 2) - \text{root}(\sigma_1, z, 1))(\text{root}(\sigma_1, z, 2) - \text{root}(\sigma_1, z, 3))(\text{root}(\sigma_1, z, 2) - \text{root}(\sigma_1, z, 4))}$$

where

$$\sigma_1 = z^4 - 16z^3 + 72z^2 - 96z + 24$$

$$\text{alpha3} = \frac{\sigma_3 \sigma_2 + \sigma_3 \sigma_1 + \sigma_2 \sigma_1 - \sigma_3 \sigma_2 \sigma_1 - 2 \sigma_3 - 2 \sigma_2 - 2 \sigma_1 + 6}{(\sigma_4 - \sigma_1)(\sigma_3 \sigma_2 - \sigma_3 \sigma_4 - \sigma_2 \sigma_4 + \sigma_4^2)}$$

where

$$\sigma_1 = \text{root}(z^4 - 16z^3 + 72z^2 - 96z + 24, z, 4)$$

$$\sigma_2 = \text{root}(z^4 - 16z^3 + 72z^2 - 96z + 24, z, 2)$$

$$\sigma_3 = \text{root}(z^4 - 16z^3 + 72z^2 - 96z + 24, z, 1)$$

$$\sigma_4 = \text{root}(z^4 - 16z^3 + 72z^2 - 96z + 24, z, 3)$$

$$\text{alpha4} = -\frac{\sigma_3 \sigma_2 + \sigma_3 \sigma_1 + \sigma_2 \sigma_1 - \sigma_3 \sigma_2 \sigma_1 - 2 \sigma_3 - 2 \sigma_2 - 2 \sigma_1 + 6}{\sigma_4^2 \sigma_3 + \sigma_4^2 \sigma_2 + \sigma_4^2 \sigma_1 - \sigma_4^3 + \sigma_3 \sigma_2 \sigma_1 - \sigma_3 \sigma_2 \sigma_4 - \sigma_3 \sigma_1 \sigma_4 - \sigma_2 \sigma_1 \sigma_4}$$

where

$$\sigma_1 = \text{root}(z^4 - 16z^3 + 72z^2 - 96z + 24, z, 3)$$

$$\sigma_2 = \text{root}(z^4 - 16z^3 + 72z^2 - 96z + 24, z, 2)$$

$$\sigma_3 = \text{root}(z^4 - 16z^3 + 72z^2 - 96z + 24, z, 1)$$

$$\sigma_4 = \text{root}(z^4 - 16z^3 + 72z^2 - 96z + 24, z, 4)$$

Alternatively, you can also obtain the solution as a structure by giving only one output argument.

```
S = solve(sys, y)
```

```
S = struct with fields:
  y1: -(root(z^4 - 16*z^3 + 72*z^2 - 96*z + 24, z, 2)*root(z^4 - 16*z^3 + 72*z^2 - 96*z + 24, z, 1))
  y2: (root(z^4 - 16*z^3 + 72*z^2 - 96*z + 24, z, 1)*root(z^4 - 16*z^3 + 72*z^2 - 96*z + 24, z, 2))
  y3: (root(z^4 - 16*z^3 + 72*z^2 - 96*z + 24, z, 1)*root(z^4 - 16*z^3 + 72*z^2 - 96*z + 24, z, 1))
  y4: -(root(z^4 - 16*z^3 + 72*z^2 - 96*z + 24, z, 1)*root(z^4 - 16*z^3 + 72*z^2 - 96*z + 24, z, 2))
```

```
structfun(@double, S)
```

```
ans = 4×1
```

```
0.6032
0.3574
0.0389
0.0005
```

Convert the structure S to a symbolic array:

```
Scell = struct2cell(S);
alpha = transpose([Scell{:}])
```

```
alpha =
```

$$\left( \begin{array}{c} -\frac{\sigma_3 + \sigma_{15} + \sigma_2 - \sigma_6 - \sigma_{12} - \sigma_{11} - \sigma_{10} + 6}{(\text{root}(\sigma_{16}, z, 1) - \text{root}(\sigma_{16}, z, 2)) (\sigma_1 + \sigma_4 - \sigma_2 - \text{root}(\sigma_{16}, z, 1)^2)} \\ \frac{\sigma_1 + \sigma_4 + \sigma_2 - \sigma_7 - \sigma_{13} - \sigma_{11} - \sigma_{10} + 6}{(\text{root}(\sigma_{16}, z, 2) - \text{root}(\sigma_{16}, z, 1)) (\text{root}(\sigma_{16}, z, 2) - \text{root}(\sigma_{16}, z, 3)) (\text{root}(\sigma_{16}, z, 2) - \text{root}(\sigma_{16}, z, 4))} \\ \frac{\sigma_5 + \sigma_4 + \sigma_{15} - \sigma_8 - \sigma_{13} - \sigma_{12} - \sigma_{10} + 6}{(\text{root}(\sigma_{16}, z, 3) - \text{root}(\sigma_{16}, z, 4)) (\sigma_5 - \sigma_1 - \sigma_3 + \text{root}(\sigma_{16}, z, 3)^2)} \\ -\frac{\sigma_5 + \sigma_1 + \sigma_3 - \sigma_9 - \sigma_{13} - \sigma_{12} - \sigma_{11} + 6}{\sigma_{14} \text{root}(\sigma_{16}, z, 1) + \sigma_{14} \text{root}(\sigma_{16}, z, 2) + \sigma_{14} \text{root}(\sigma_{16}, z, 3) - \text{root}(\sigma_{16}, z, 4)^3 + \sigma_9 - \sigma_8 - \sigma_7 - \sigma_6} \end{array} \right)$$

where

$$\sigma_1 = \text{root}(\sigma_{16}, z, 1) \text{root}(\sigma_{16}, z, 3)$$

$$\sigma_2 = \text{root}(\sigma_{16}, z, 3) \text{root}(\sigma_{16}, z, 4)$$

$$\sigma_3 = \text{root}(\sigma_{16}, z, 2) \text{root}(\sigma_{16}, z, 3)$$

$$\sigma_4 = \text{root}(\sigma_{16}, z, 1) \text{root}(\sigma_{16}, z, 4)$$

$$\sigma_5 = \text{root}(\sigma_{16}, z, 1) \text{root}(\sigma_{16}, z, 2)$$

$$\sigma_6 = \text{root}(\sigma_{16}, z, 2) \text{root}(\sigma_{16}, z, 3) \text{root}(\sigma_{16}, z, 4)$$

$$\sigma_7 = \text{root}(\sigma_{16}, z, 1) \text{root}(\sigma_{16}, z, 3) \text{root}(\sigma_{16}, z, 4)$$

$$\sigma_8 = \text{root}(\sigma_{16}, z, 1) \text{root}(\sigma_{16}, z, 2) \text{root}(\sigma_{16}, z, 4)$$

$$\sigma_9 = \text{root}(\sigma_{16}, z, 1) \text{root}(\sigma_{16}, z, 2) \text{root}(\sigma_{16}, z, 3)$$

$$\sigma_{10} = 2 \text{root}(\sigma_{16}, z, 4)$$

$$\sigma_{11} = 2 \text{root}(\sigma_{16}, z, 3)$$

$$\sigma_{12} = 2 \text{root}(\sigma_{16}, z, 2)$$

$$\sigma_{13} = 2 \text{root}(\sigma_{16}, z, 1)$$

$$\sigma_{14} = \text{root}(\sigma_{16}, z, 4)^2$$

$$\sigma_{15} = \text{root}(\sigma_{16}, z, 2) \text{root}(\sigma_{16}, z, 4)$$

$$\sigma_{16} = z^4 - 16z^3 + 72z^2 - 96z + 24$$

The symbolic solution looks complicated. Simplify it, and convert it into a floating point vector:

`alpha = simplify(alpha)`

`alpha =`

$$\begin{pmatrix} \frac{\text{root}(\sigma_1, z, 1)^2}{72} - \frac{29 \text{root}(\sigma_1, z, 1)}{144} + \frac{2}{3} \\ \frac{\text{root}(\sigma_1, z, 2)^2}{72} - \frac{29 \text{root}(\sigma_1, z, 2)}{144} + \frac{2}{3} \\ \frac{\text{root}(\sigma_1, z, 3)^2}{72} - \frac{29 \text{root}(\sigma_1, z, 3)}{144} + \frac{2}{3} \\ \frac{\text{root}(\sigma_1, z, 4)^2}{72} - \frac{29 \text{root}(\sigma_1, z, 4)}{144} + \frac{2}{3} \end{pmatrix}$$

where

$$\sigma_1 = z^4 - 16 z^3 + 72 z^2 - 96 z + 24$$

`vpa(alpha)`

$$\text{ans} = \begin{pmatrix} 0.60315410434163360163596602381808 \\ 0.35741869243779968664149201745809 \\ 0.03888790851500538427243816815621 \\ 0.00053929470556132745010379056762059 \end{pmatrix}$$

Increase the readability by replacing the occurrences of the roots `x` in `alpha` by abbreviations:

`subs(alpha, x, sym('R', [4, 1]))`

$$\text{ans} = \begin{pmatrix} \frac{R_1^2}{72} - \frac{29 R_1}{144} + \frac{2}{3} \\ \frac{\text{root}(\sigma_1, z_1, 2)^2}{72} - \frac{29 \text{root}(\sigma_1, z_1, 2)}{144} + \frac{2}{3} \\ \frac{\text{root}(\sigma_1, z_1, 3)^2}{72} - \frac{29 \text{root}(\sigma_1, z_1, 3)}{144} + \frac{2}{3} \\ \frac{\text{root}(\sigma_1, z_1, 4)^2}{72} - \frac{29 \text{root}(\sigma_1, z_1, 4)}{144} + \frac{2}{3} \end{pmatrix}$$

where

$$\sigma_1 = z_1^4 - 16 z_1^3 + 72 z_1^2 - 96 z_1 + 24$$

Sum the weights to show that their sum equals 1:

`simplify(sum(alpha))`

$$\text{ans} = 1$$

A different method to obtain the weights of a quadrature rule is to compute them using the formula

$\alpha_i = \int_a^b w(t) \prod_{j \neq i} \frac{t - x_j}{x_i - x_j} dt$ . Do this for  $i = 1$ . It leads to the same result as the other method:

`int(w(t) * prod((t - x(2:4)) ./ (x(1) - x(2:4))), t, 0, inf)`

$$\text{ans} =$$

$$\frac{\text{root}(z^4 - 16z^3 + 72z^2 - 96z + 24, z, 1)^2}{72} - \frac{29 \text{root}(z^4 - 16z^3 + 72z^2 - 96z + 24, z, 1)}{144} + \frac{2}{3}$$

The quadrature rule produces exact results even for all polynomials of degree less than or equal to  $2n - 1$ , but not for  $t^{2n}$ .

```
simplify(sum(alpha.*(x.^(2*n-1))) -int(t^(2*n-1)*w(t), t, 0, inf))
```

```
ans = 0
```

```
simplify(sum(alpha.*(x.^(2*n))) -int(t^(2*n)*w(t), t, 0, inf))
```

```
ans = -576
```

Apply the quadrature rule to the cosine, and compare with the exact result:

```
vpa(sum(alpha.*(cos(x))))
```

```
ans = 0.50249370546067059229918484198931
```

```
int(cos(t)*w(t), t, 0, inf)
```

```
ans =  
1/2
```

For powers of the cosine, the error oscillates between odd and even powers:

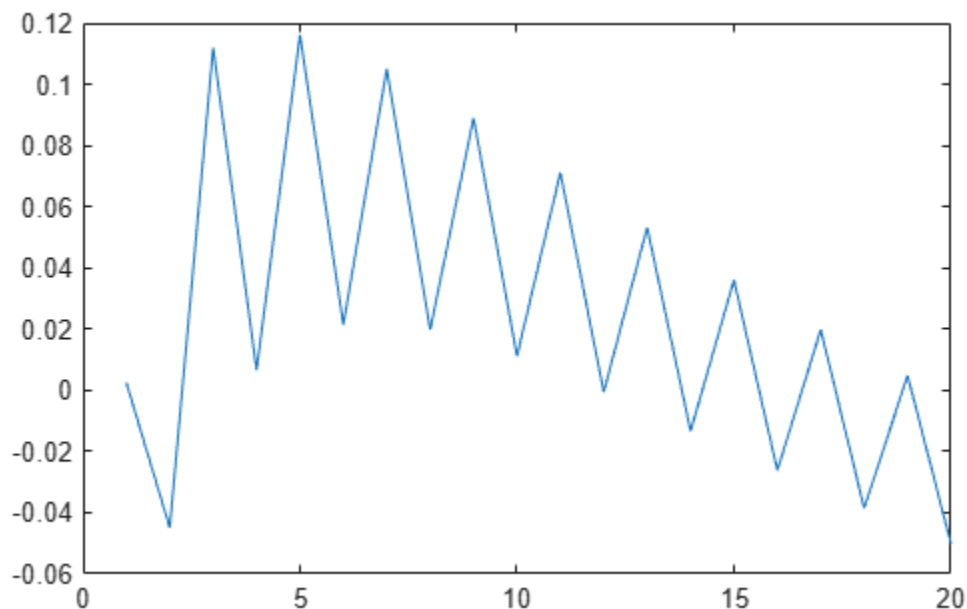
```
errors = zeros(1, 20);
```

```
for k=1:20
```

```
    errors(k) = double(sum(alpha.*(cos(x).^k)) -int(cos(t)^k*w(t), t, 0, inf));
```

```
end
```

```
plot(real(errors))
```





# Simulate a Stochastic Process Using the Feynman-Kac Formula

This example obtains the partial differential equation that describes the expected final price of an asset whose price is a stochastic process given by a stochastic differential equation.

The steps involved are:

- 1 Define Parameters of the Model Using Stochastic Differential Equations
- 2 Apply Ito's Rule
- 3 Solve Feynman-Kac Equation
- 4 Compute Expected Time to Sell Asset

## 1. Define Parameters of the Model Using Stochastic Differential Equations

A model for the price of an asset  $X(t)$  defined in the time interval  $[0, T]$  is a stochastic process defined by a stochastic differential equation of the form  $dX = \mu(t, X)dt + \sigma(t, X)dB(t)$ , where  $B(t)$  is the Wiener process with unit variance parameter.

Create the symbolic variable  $T$  and the following symbolic functions:

- $r(t)$  is a continuous function representing a spot interest rate. This rate determines the discount factor for the final payoff at the time  $T$ .
- $u(t, x)$  is the expected value of the discounted future price calculated as  $X(T) \exp\left(-\int_t^T r(t) dt\right)$  under the condition  $X(t) = x$ .
- $\mu(t, X)$  and  $\sigma(t, X)$  are drift and diffusion of the stochastic process  $X(t)$ .

```
syms mu(t, X) sigma(t, X) r(t, X) u(t, X) T
```

According to the Feynman-Kac theorem,  $u$  satisfies the partial differential equation

$$\frac{\partial u}{\partial t} + \frac{\sigma^2}{2} \frac{\partial^2 u}{\partial x^2} + \mu \frac{\partial u}{\partial x} - ur = 0 \text{ with a final condition at time } T.$$

```
eq = diff(u, t) + sigma^2*diff(u, X, X)/2 + mu*diff(u, X) - u*r;
```

The numerical solver `pdepe` works with initial conditions. To transform the final condition into an initial condition, apply a time reversal by setting  $u(t, X) = v(T - t, X)$ .

```
syms v(t, X)
eq2 = subs(eq, {u, t}, {v(T - t, X), T - t});
eq2 = feval(symengine, 'rewrite', eq2, 'diff')
```

```
eq2 =
```

$$\frac{\sigma(T-t, X)^2 \frac{\partial^2}{\partial X^2} v(t, X)}{2} + \mu(T-t, X) \frac{\partial}{\partial X} v(t, X) - \frac{\partial}{\partial t} v(t, X) - v(t, X) r(T-t, X)$$

The solver `pdepe` requires the partial differential equation in the following form. Here the coefficients  $c$ ,  $f$ , and  $s$  are functions of  $x$ ,  $t$ ,  $v$ , and  $\partial v / \partial x$ .

$$c \frac{\partial v}{\partial t} = \frac{\partial f}{\partial x} + s$$

To be able to solve the equation eq2 with the pdepe solver, map eq2 to this form. First, extract the coefficients and terms of eq2.

```
syms dvt dvXX
eq3 = subs(eq2, {diff(v, t), diff(v, X, X)}, {dvt, dvXX});
[k,terms] = coeffs(eq3, [dvt, dvXX])
```

$$k = \left( -1 \frac{\sigma(T-t, X)^2}{2} \mu(T-t, X) \frac{\partial}{\partial X} v(t, X) - v(t, X) r(T-t, X) \right)$$

```
terms = (dvt dvXX 1)
```

Now, you can rewrite eq2 as  $k(1)*terms(1) + k(2)*terms(2) + k(3)*terms(3) = 0$ . Moving the term with the time derivative to the left side of the equation, you get

$$\frac{\partial v}{\partial t} = k(2) \frac{\partial^2 v}{\partial X^2} + k(3)$$

Manually integrate the right side by parts. The result is

$$\frac{\partial}{\partial X} \left( k(2) \frac{\partial v}{\partial X} \right) + k(3) - \frac{\partial v}{\partial X} \frac{\partial k(2)}{\partial X}$$

Therefore, to write eq2 in the form suitable for pdepe, use the following parameters:

```
c = 1;
f = k(2) * diff(v, X);
s = k(3) - diff(v, X) * diff(k(2), X);
```

## 2. Apply Ito's Rule

Asset prices follow a multiplicative process. That is, the logarithm of the price can be described in terms of an SDE, but the expected value of the price itself is of interest because it describes the profit, and thus we need an SDE for the latter.

In general, if a stochastic process  $X$  is given in terms of an SDE, then Ito's rule says that the transformed process  $G(t, X)$  satisfies

$$dG = \left( \mu \frac{dG}{dX} + \frac{\sigma^2}{2} \frac{d^2 G}{dX^2} + \frac{dG}{dt} \right) dt + \frac{dG}{dX} \sigma dB(t)$$

We assume that the logarithm of the price is given by a one-dimensional additive Brownian motion, that is,  $\mu$  and  $\sigma$  are constants. Define a function that applies Ito's rule, and use it to transform the additive Brownian motion into a geometric Brownian motion.

```
ito = @(mu, sigma, G, t, X) ...
    deal( mu * diff(G, X) + sigma^2/2 * diff(G, X, X) + diff(G, t), ...
    diff(G, X) * sigma );
```

```
syms mu0 sigma0
[mu1, sigma1] = ito(mu0, sigma0, exp(X), t, X)
```

$$mu1 = \frac{e^X \sigma_0^2}{2} + \mu_0 e^X$$



$$\text{sigma1} = \sigma_0 e^X$$

Replace  $\exp(X)$  by  $Y$ .

```
syms Y
mu1 = subs(mu1, X, log(Y));
sigma1 = subs(sigma1, X, log(Y));
f = f(t, log(Y));
s = s(t, log(Y));
```

For simplicity, assume that the interest rate is zero. This is a special case also known as Kolmogorov backward equation.

```
r0 = 0;

c = subs(c, {mu, sigma}, {mu1, sigma1});
s = subs(s, {mu, sigma, r}, {mu1, sigma1, r0});
f = subs(f, {mu, sigma}, {mu1, sigma1});
```

### 3. Solve Feynman-Kac Equation

Before you can convert symbolic expressions to MATLAB function handles, you must replace function calls, such as  $\text{diff}(v(t, X), X)$  and  $v(t, X)$ , with variables. You can use any valid MATLAB variable names.

```
syms dvdx V;
dvX = diff(v, X);
c = subs(c, {dvX, v}, {dvdX, V});
f = subs(f, {dvX, v}, {dvdX, V});
s = subs(s, {dvX, v}, {dvdX, V});
```

For a flat geometry (translation symmetry), set the following value:

```
m = 0;
```

Also, assign numeric values to symbolic parameters.

```
muvalue = 0;
sigmavalue = 1;

c0 = subs(c, {mu0, sigma0}, {muvalue, sigmavalue});
f0 = subs(f, {mu0, sigma0}, {muvalue, sigmavalue});
s0 = subs(s, {mu0, sigma0}, {muvalue, sigmavalue});
```

Use `matlabFunction` to create a function handle. Pass the coefficients  $c0$ ,  $f0$ , and  $s0$  in the form required by `pdepe`, namely, a function handle with three output arguments.

```
FeynmanKacPde = matlabFunction(c0, f0, s0, 'Vars', [t, Y, V, dvdx])
```

```
FeynmanKacPde = function_handle with value:
@(t,Y,V,dvdx)deal(1.0,(Y.^2.*dvdX)./2.0,(Y.*dvdX)./2.0)
```

As the final condition, take the identity mapping. That is, the payoff at time  $T$  is given by the asset price itself. You can modify this line in order to investigate derivative instruments.

```
FeynmanKacIC = @(x) x;
```

Numerical solving of PDEs can only be applied to a finite domain. Therefore, you must specify a boundary condition. Assume that the asset is sold at the moment when its price rises above or falls below a certain limit, and thus the solution  $v$  has to satisfy  $x - v = 0$  at the boundary points  $x$ . You can choose another boundary condition, for example, you can use  $v = 0$  to model knockout options. The zeroes in the second and fourth output indicate that the boundary condition does not depend on  $\frac{dv}{dx}$ .

```
FeynmanKacBC = @(xl, vl, xr, vr, t) ...  
    deal(xl - vl, 0, xr - vr, 0);
```

Choose the space grid, which is the range of values of the price  $x$ . Set the left boundary to zero: it is not reachable by a geometric random walk. Set the right boundary to one: when the asset price reaches this limit, the asset is sold.

```
xgrid = linspace(0, 1, 101);
```

Choose the time grid. Because of the time reversal applied in the beginning, it denotes the time left until the final time  $T$ .

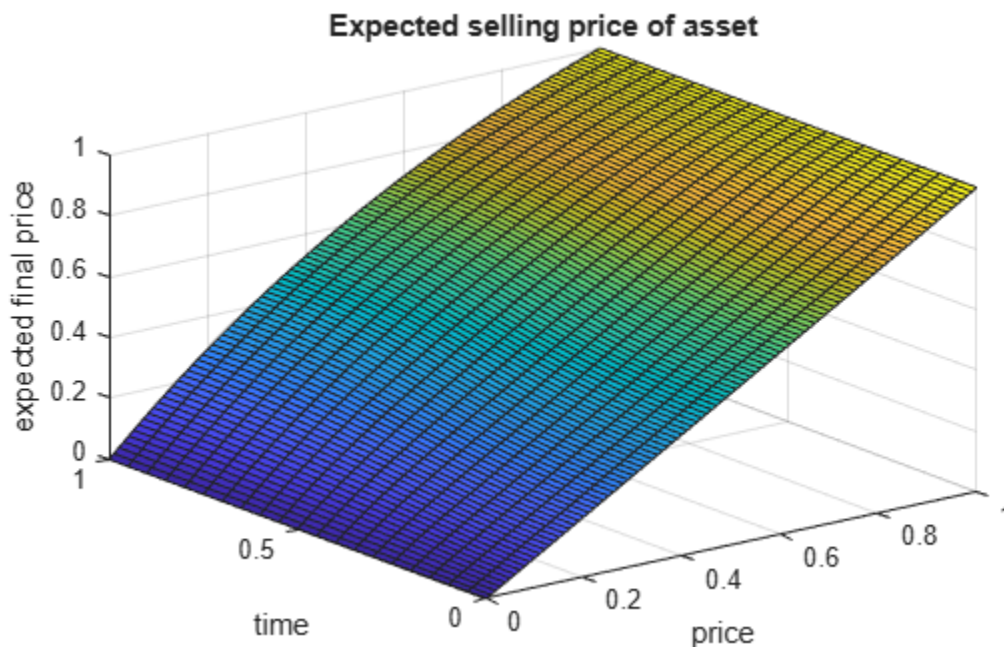
```
tgrid = linspace(0, 1, 21);
```

Solve the equation.

```
sol = pdepe(m, FeynmanKacPde, FeynmanKacIC, FeynmanKacBC, xgrid, tgrid);
```

Plot the solution. The expected selling price depends nearly linearly on the price at time  $t$ , and also weakly on  $t$ .

```
surf(xgrid, tgrid, sol)  
title('Expected selling price of asset');  
xlabel('price');  
ylabel('time');  
zlabel('expected final price');
```

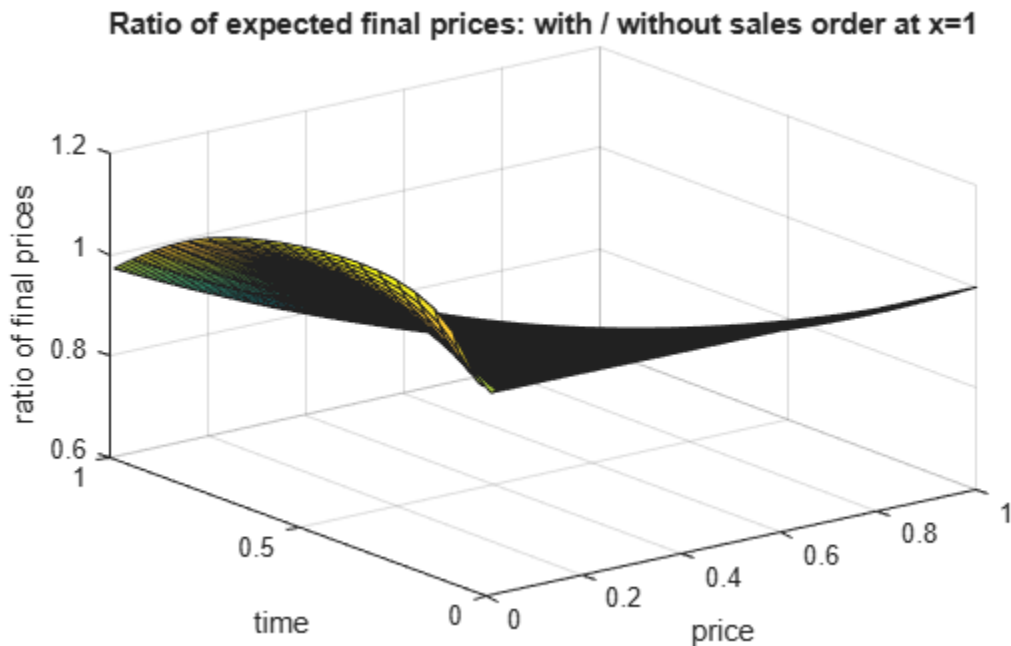


The state of a geometric Brownian motion with drift  $\mu_1$  at time  $t$  is a lognormally distributed random variable with expected value  $\exp(\mu_1 t)$  times its initial value. This describes the expected selling price of an asset that is never sold because of reaching a limit.

```
Expected = transpose(exp(tgrid./2)) * xgrid;
```

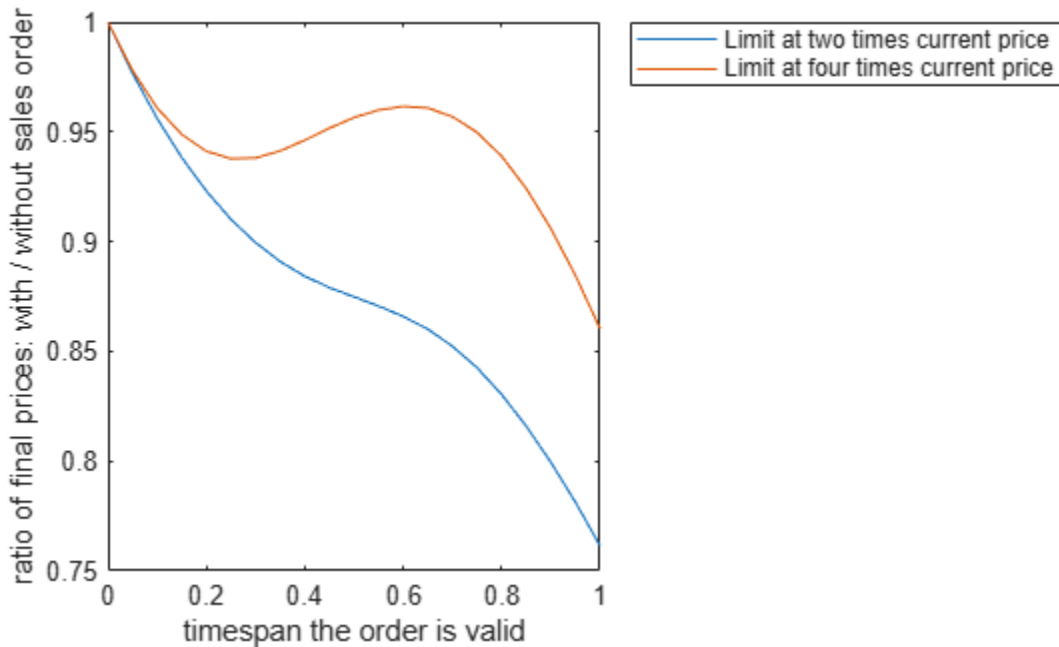
Dividing the solution obtained above by that expected value shows how much profit is lost by selling prematurely at the limit.

```
sol2 = sol./Expected;
surf(xgrid, tgrid, sol2)
title('Ratio of expected final prices: with / without sales order at x=1')
xlabel('price');
ylabel('time');
zlabel('ratio of final prices');
```



For example, plot the ratio of the expected payoff of an asset for which a limit sales order has been placed and the same asset without sales order over a timespan  $T$ , as a function of  $t$ . Consider the case of an order limit of two and four times the current price, respectively.

```
plot(tgrid, sol2(:, xgrid == 1/2));
hold on
plot(tgrid, sol2(:, xgrid == 1/4));
legend('Limit at two times current price', 'Limit at four times current price');
xlabel('timespan the order is valid');
ylabel('ratio of final prices: with / without sales order');
hold off
```



#### 4. Compute Expected Time to Sell Asset

It is a textbook result that the expected exit time when the limit is reached and the asset is sold is given by the following equation:

```
syms y(X)
exitTimeEquation(X) = subs(eq, {r, u}, {0, y(X)}) == -1
```

$$\text{exitTimeEquation}(X) = \frac{\sigma(t, X)^2 \frac{\partial^2}{\partial X^2} y(X)}{2} + \mu(t, X) \frac{\partial}{\partial X} y(X) = -1$$

In addition,  $y$  must be zero at the boundaries. For  $\mu$  and  $\sigma$ , insert the actual stochastic process under consideration:

```
exitTimeGBM = subs(subs(exitTimeEquation, {mu, sigma}, {mu1, sigma1}), Y, X)
```

$$\text{exitTimeGBM}(X) = \left( \frac{X \sigma_0^2}{2} + X \mu_0 \right) \frac{\partial}{\partial X} y(X) + \frac{X^2 \sigma_0^2 \frac{\partial^2}{\partial X^2} y(X)}{2} = -1$$

Solve this problem for arbitrary interval borders  $a$  and  $b$ .

```
syms a b
exitTime = dsolve(exitTimeGBM, y(a) == 0, y(b) == 0)
```

```
exitTime =
```

$$\frac{2 \mu_0 \sigma_4 \log(b) - 2 \mu_0 \sigma_5 \log(a) + a^{\sigma_1} \sigma_0^2 \sigma_4 \sigma_5 - b^{\sigma_1} \sigma_0^2 \sigma_4 \sigma_5}{\sigma_3} - \frac{\log(X)}{\mu_0} + \frac{\sigma_2 (\sigma_6 - \sigma_7 - a^{\sigma_1} \sigma_0^2 \sigma_4 + b^{\sigma_1} \sigma_0^2 \sigma_5)}{\sigma_3} + \frac{X^{\sigma_1} \sigma_0^2}{2 \mu_0^2}$$

where

$$\sigma_1 = \frac{2 \mu_0}{\sigma_0^2}$$

$$\sigma_2 = e^{-\frac{2 \mu_0 \log(X)}{\sigma_0^2}}$$

$$\sigma_3 = 2 \mu_0^2 (\sigma_4 - \sigma_5)$$

$$\sigma_4 = e^{-\frac{\sigma_6}{\sigma_0^2}}$$

$$\sigma_5 = e^{-\frac{\sigma_7}{\sigma_0^2}}$$

$$\sigma_6 = 2 \mu_0 \log(a)$$

$$\sigma_7 = 2 \mu_0 \log(b)$$

Because you cannot substitute  $\mu_0 = 0$  directly, compute the limit at  $\mu_0 \rightarrow 0$  instead.

```
L = limit(subs(exitTime, sigma0, sigmavalue), mu0, muvalue)
```

```
L = -(log(X) - log(a)) (log(X) - log(b))
```

L is undefined at  $a = 0$ . Set the assumption that  $0 < X < 1$ .

```
assume(0 < X & X < 1)
```

Using the value  $b = 1$  for the right border, compute the limit.

```
limit(subs(L, b, 1), a, 0, 'Right')
```

```
ans = ∞
```

The expected exit time is infinite.

## See Also

### Functions

`diff` | `subs` | `coeffs` | `matlabFunction` | `dsolve`

## The Black-Scholes Formula for Call Option Price

This example shows how to calculate the call option price using the Black-Scholes formula. This example uses `vpasolve` to numerically solve the problems of finding the spot price and implied volatility from the Black-Scholes formula.

### Find Call Option Price

The Black-Scholes formula models the price of European call options [1 on page 3-122]. For a non-dividend-paying underlying stock, the parameters of the formula are defined as:

- $S$  is the current stock price or spot price.
- $K$  is the exercise or strike price.
- $\sigma$  is the standard deviation of continuously compounded annual returns of the stock, which is called volatility.
- $T$  is the time for the option to expire in years.
- $r$  is the annualized risk-free interest rate.

The price of a call option  $C$  in terms of the Black-Scholes parameters is

$$C = N(d_1) \times S - N(d_2) \times PV(K),$$

where:

- $d_1 = \frac{1}{\sigma\sqrt{T}} \left[ \log\left(\frac{S}{K}\right) + \left(r + \frac{\sigma^2}{2}\right)T \right]$
- $d_2 = d_1 - \sigma\sqrt{T}$
- $PV(K) = K \exp(-rT)$
- $N(d)$  is the standard normal cumulative distribution function,  $N(d) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^d \exp(-t^2/2) dt$ .

Find the price of a European stock option that expires in three months with an exercise price of \$95. Assume that the underlying stock pays no dividend, trades at \$100, and has a volatility of 50% per annum. The risk-free rate is 1% per annum.

Use `sym` to create symbolic numbers that represent the values of the Black-Scholes parameters.

```
syms t d
S = sym(100);           % current stock price (spot price)
K = sym(95);            % exercise price (strike price)
sigma = sym(0.50);      % volatility of stock
T = sym(3/12);          % expiry time in years
r = sym(0.01);          % annualized risk-free interest rate
```

Calculate the option price without approximation. Create a symbolic function  $N(d)$  that represents the standard normal cumulative distribution function.

```
PV_K = K*exp(-r*T);
d1 = (log(S/K) + (r + sigma^2/2)*T)/(sigma*sqrt(T));
d2 = d1 - sigma*sqrt(T);
N(d) = int(exp(-(t^2)/2), t, -Inf, d)*1/sqrt(2*sym(pi))
```

$$N(d) = \frac{\operatorname{erf}\left(\frac{\sqrt{2}d}{2}\right)}{2} + \frac{1}{2}$$

$$C_{\text{sym}} = N(d_1)*S - N(d_2)*PV\_K$$

$$C_{\text{sym}} =$$

$$50 \operatorname{erf}\left(\frac{\sqrt{2}\left(4 \log\left(\frac{20}{19}\right) + \frac{27}{200}\right)}{2}\right) - 95 e^{-\frac{1}{400}} \left( \frac{\operatorname{erf}\left(\frac{\sqrt{2}\left(4 \log\left(\frac{20}{19}\right) - \frac{23}{200}\right)}{2}\right)}{2} + \frac{1}{2} \right) + 50$$

To obtain the numeric result with variable precision, use `vpa`. By default, `vpa` returns a number with 32 significant digits.

$$C_{\text{vpa}} = \text{vpa}(C_{\text{sym}})$$

$$C_{\text{vpa}} = 12.52792339252145394554497137187$$

To change the precision, use `digits`. The price of the option to 6 significant digits is \$12.5279.

$$\text{digits}(6)$$

$$C_{\text{vpa}} = \text{vpa}(C_{\text{sym}})$$

$$C_{\text{vpa}} = 12.5279$$

### Plot Call Option Price

Next, suppose that for the same stock option the time to expiry changes and the day-to-day stock price is unknown. Find the price of this call option for expiry time  $T$  that varies from 0 to 0.25 years, and spot price  $S$  that varies from \$50 to \$140. Use the values for exercise rate ( $K$ ), volatility ( $\sigma$ ), and interest rate ( $r$ ) from the previous example. In this case, use the time to expiry  $T$  and day-to-day stock price  $S$  as the variable quantities.

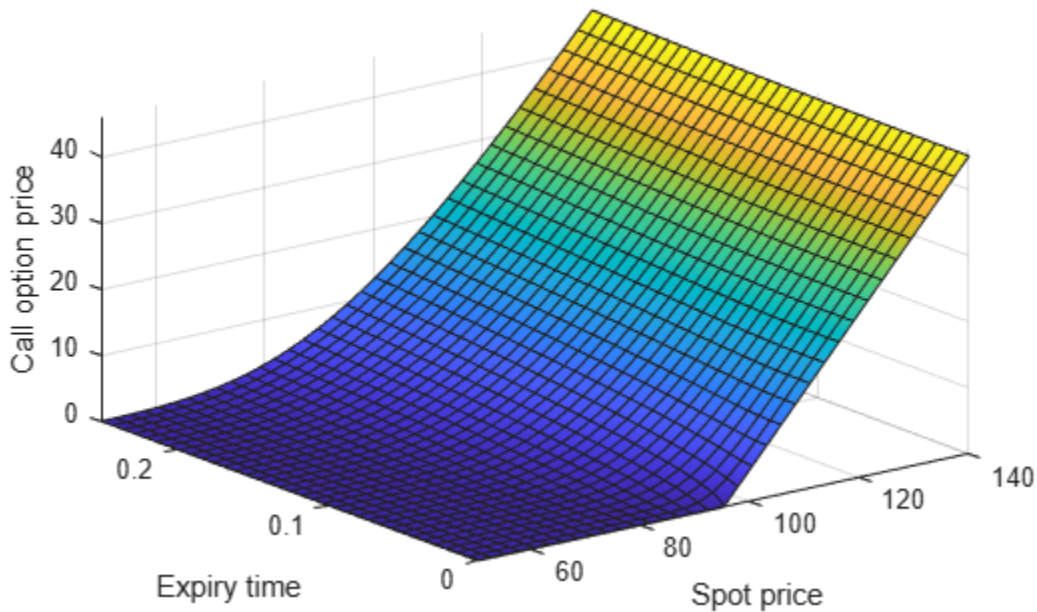
Define the symbolic expression  $C$  to represent the call option price with  $T$  and  $S$  as the unknown variables.

`syms T S`

```
PV_K = K*exp(-r*T);
d1 = (log(S/K) + (r + sigma^2/2)*T)/(sigma*sqrt(T));
d2 = d1 - sigma*sqrt(T);
Nd1 = int(exp(-((t)^2)/2), -Inf, d1)*1/sqrt(2*pi);
Nd2 = int(exp(-((t)^2)/2), -Inf, d2)*1/sqrt(2*pi);
C = Nd1*S - Nd2*PV_K;
```

Plot the call option price as a function of spot price and expiry time.

```
fsurf(C,[50 140 0 0.25])
xlabel('Spot price')
ylabel('Expiry time')
zlabel('Call option price')
```



Calculate the call option price with expiry time 0.1 years and spot price \$105. Use `subs` to substitute the values of  $T$  and  $S$  to the expression  $C$ . Return the price as a numeric result using `vpa`.

```
Csym = subs(C,[T S],[0.1 105]);
Cvpa = vpa(Csym)

Cvpa = 12.5868
```

### Find Spot Price

Consider the case where the option price is changing, and you want to know how this affects the underlying stock price. This is a problem of finding  $S$  from the Black-Scholes formula given the known parameters  $K$ ,  $\sigma$ ,  $T$ ,  $r$ , and  $C$ .

For example, after one month, the price of the same call option now trades at \$15.04 with expiry time of two months. Find the spot price of the underlying stock. Create a symbolic function  $C(S)$  that represents the Black-Scholes formula with the unknown parameter  $S$ .

```
syms C(S) d1(S) d2(S) Nd1(S) Nd2(S)

K = 95;           % exercise price (strike price)
sigma = 0.50;     % volatility of stock
T = 2/12;         % expiry time in years
r = 0.01;         % annualized risk-free interest rate

PV_K = K*exp(-r*T);
d1(S) = (log(S/K) + (r + sigma^2/2)*T)/(sigma*sqrt(T));
d2(S) = d1 - sigma*sqrt(T);
Nd1(S) = int(exp(-((t)^2)/2), -Inf,d1)*1/sqrt(2*pi);
Nd2(S) = int(exp(-((t)^2)/2), -Inf,d2)*1/sqrt(2*pi);
C(S) = Nd1*S - Nd2*PV_K;
```

Use `vpasolve` to numerically solve for the spot price of the underlying stock. Search for solutions only in the positive numbers. The spot price of the underlying stock is \$106.162.



```
S_Sol = vpasolve(C(S) == 15.04,S,[0 Inf])
```

```
S_Sol = 106.162
```

### Find Implied Volatility

Consider the case where the option price is changing and you want to know what is the implied volatility. This is a problem of finding the value of  $\sigma$  from the Black-Scholes formula given the known parameters  $S$ ,  $K$ ,  $T$ ,  $r$ , and  $C$ .

Consider the same stock option that expires in three months with an exercise price of \$95. Assume that the underlying stock trades at \$100, and the risk-free rate is 1% per annum. Find the implied volatility as a function of option price that ranges from \$6 to \$25. Create a vector for the range of the option price. Create a symbolic function  $C(\sigma)$  that represents the Black-Scholes formula with the unknown parameter  $\sigma$ . Use `vpasolve` to numerically solve for the implied volatility.

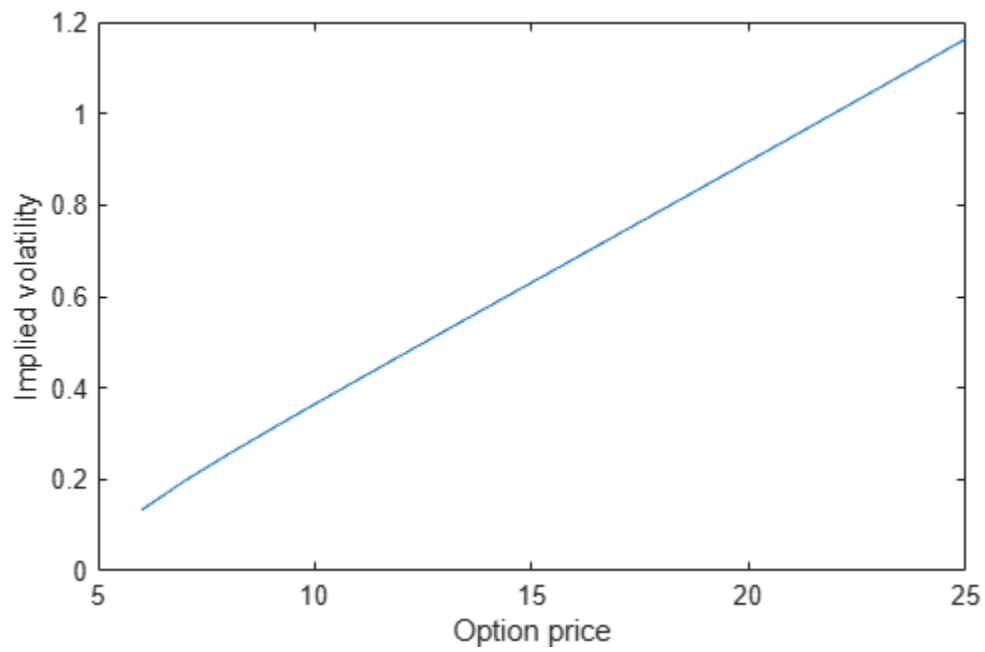
```
syms C(sigma) d1(sigma) d2(sigma) Nd1(sigma) Nd2(sigma)

S = 100;           % current stock price (spot price)
K = 95;            % exercise price (strike price)
T = 3/12;          % expiry time in years
r = 0.01;          % annualized risk-free interest rate
C_Range = 6:25;    % range of option price
sigma_Sol = zeros(size(C_Range));

PV_K = K*exp(-r*T);
d1(sigma) = (log(S/K) + (r + sigma^2/2)*T)/(sigma*sqrt(T));
d2(sigma) = d1 - sigma*sqrt(T);
Nd1(sigma) = int(exp(-((t)^2)/2),-Inf,d1)*1/sqrt(2*pi);
Nd2(sigma) = int(exp(-((t)^2)/2),-Inf,d2)*1/sqrt(2*pi);
C(sigma) = Nd1*S - Nd2*PV_K;
for i = 1:length(C_Range)
    sigma_Sol(i) = vpasolve(C(sigma) == C_Range(i),sigma,[0 Inf]);
end
```

Plot the implied volatility as a function of the option price.

```
plot(C_Range,sigma_Sol)
xlabel('Option price')
ylabel('Implied volatility')
```

**Reference**

[1] [https://en.wikipedia.org/wiki/Black-Scholes\\_model](https://en.wikipedia.org/wiki/Black-Scholes_model)

**See Also****Functions**

vpa | vpasolve | subs

## Choose Function to Rearrange Expression

| Type of Transformation                                                 | Function         |
|------------------------------------------------------------------------|------------------|
| "Combine Terms of Same Algebraic Structures" on page 3-123             | combine          |
| "Expand Expressions" on page 3-124                                     | expand           |
| "Factor Expressions" on page 3-125                                     | factor           |
| "Extract Subexpressions from Expression" on page 3-127                 | children         |
| "Collect Terms with Same Powers" on page 3-127                         | collect          |
| "Rewrite Expressions in Terms of Other Functions" on page 3-128        | rewrite          |
| "Compute Partial Fraction Decompositions of Expressions" on page 3-129 | partfrac         |
| "Compute Normal Forms of Rational Expressions" on page 3-129           | simplifyFraction |
| "Represent Polynomials Using Horner Nested Forms" on page 3-130        | horner           |

### Combine Terms of Same Algebraic Structures

Symbolic Math Toolbox provides the `combine` function for combining subexpressions of an original expression. The `combine` function uses mathematical identities for the functions you specify. For example, combine the trigonometric expression.

```
syms x y
combine(2*sin(x)*cos(x), 'sincos')

ans =
sin(2*x)
```

If you do not specify a target function, `combine` uses the identities for powers wherever these identities are valid:

- $a^b a^c = a^{b+c}$
- $a^c b^c = (a b)^c$
- $(a^b)^c = a^{bc}$

For example, by default the function combines the following square roots.

```
combine(sqrt(2)*sqrt(x))

ans =
(2*x)^(1/2)
```

The function does not combine the square roots `sqrt(x)*sqrt(y)` because the identity is not valid for negative values of variables.

```
combine(sqrt(x)*sqrt(y))
```

```
ans =  
x^(1/2)*y^(1/2)
```

To combine these square roots, use the `IgnoreAnalyticConstraints` option.

```
combine(sqrt(x)*sqrt(y), 'IgnoreAnalyticConstraints', true)
```

```
ans =  
(x*y)^(1/2)
```

`IgnoreAnalyticConstraints` provides a shortcut allowing you to combine expressions under commonly used assumptions about values of the variables. Alternatively, you can set appropriate assumptions on variables explicitly. For example, assume that  $x$  and  $y$  are positive values.

```
assume([x,y], 'positive')  
combine(sqrt(x)*sqrt(y))
```

```
ans =  
(x*y)^(1/2)
```

For further computations, clear the assumptions on  $x$  and  $y$  by recreating them using `syms`.

```
syms x y
```

As target functions, `combine` accepts `atan`, `exp`, `gamma`, `int`, `log`, `sincos`, and `sinhcosh`.

## Expand Expressions

For elementary expressions, use the `expand` function to transform the original expression by multiplying sums of products. This function provides an easy way to expand polynomials.

```
expand((x - 1)*(x - 2)*(x - 3))
```

```
ans =  
x^3 - 6*x^2 + 11*x - 6
```

```
expand(x*(x*(x - 6) + 11) - 6)
```

```
ans =  
x^3 - 6*x^2 + 11*x - 6
```

The function also expands exponential and logarithmic expressions. For example, expand the following expression containing exponentials.

```
expand(exp(x + y)*(x + exp(x - y)))
```

```
ans =  
exp(2*x) + x*exp(x)*exp(y)
```

Expand an expression containing logarithm. Expanding logarithms is not valid for generic complex values, but it is valid for positive values.

```
syms a b c positive  
expand(log(a*b*c))
```

```
ans =  
log(a) + log(b) + log(c)
```

For further computations, clear the assumptions.

```
syms a b c
```

Alternatively, use the `IgnoreAnalyticConstraints` option when expanding logarithms.

```
expand(log(a*b*c),'IgnoreAnalyticConstraints',true)
```

```
ans =  
log(a) + log(b) + log(c)
```

`expand` also works on trigonometric expressions. For example, expand this expression.

```
expand(cos(x + y))
```

```
ans =  
cos(x)*cos(y) - sin(x)*sin(y)
```

`expand` uses mathematical identities between the functions.

```
expand(sin(5*x))
```

```
ans =  
sin(x) - 12*cos(x)^2*sin(x) + 16*cos(x)^4*sin(x)
```

```
expand(cos(3*acos(x)))
```

```
ans =  
4*x^3 - 3*x
```

`expand` works recursively for all subexpressions.

```
expand((sin(3*x) + 1)*(cos(2*x) - 1))
```

```
ans =  
2*sin(x) + 2*cos(x)^2 - 10*cos(x)^2*sin(x) + 8*cos(x)^4*sin(x) - 2
```

To prevent the expansion of all trigonometric, logarithmic, and exponential subexpressions, use the option `ArithmeticOnly`.

```
expand(exp(x + y)*(x + exp(x - y)),'ArithmeticOnly',true)
```

```
ans =  
exp(x - y)*exp(x + y) + x*exp(x + y)
```

```
expand((sin(3*x) + 1)*(cos(2*x) - 1),'ArithmeticOnly',true)
```

```
ans =  
cos(2*x) - sin(3*x) + cos(2*x)*sin(3*x) - 1
```

## Factor Expressions

To return all irreducible factors of an expression, use the `factor` function. For example, find all irreducible polynomial factors of this polynomial expression. The result shows that this polynomial has three roots:  $x = 1$ ,  $x = 2$ , and  $x = 3$ .

```
syms x  
factor(x^3 - 6*x^2 + 11*x - 6)
```

```
ans =  
[ x - 3, x - 1, x - 2]
```

If a polynomial expression is irreducible, `factor` returns the original expression.

```
factor(x^3 - 6*x^2 + 11*x - 5)
```

```
ans =  
x^3 - 6*x^2 + 11*x - 5
```

Find irreducible polynomial factors of the expression  $x^6 + 1$ . By default, `factor` uses factorization over rational numbers keeping rational numbers in their exact symbolic form. The resulting factors for this expression do not show polynomial roots.

```
factor(x^6 + 1)
```

```
ans =  
[ x^2 + 1, x^4 - x^2 + 1]
```

Using other factorization modes lets you factor this expression further. For example, factor the same expression over complex numbers.

```
factor(x^6 + 1, 'FactorMode', 'complex')
```

```
ans =  
[ x + 0.86602540378443864676372317075294 + 0.5i,...  
  x + 0.86602540378443864676372317075294 - 0.5i,...  
  x + 1.0i,...  
  x - 1.0i,...  
  x - 0.86602540378443864676372317075294 + 0.5i,...  
  x - 0.86602540378443864676372317075294 - 0.5i]
```

`factor` also works on expressions other than polynomials and rational expressions. For example, you can factor the following expression that contains logarithm, sine, and cosine functions. Internally, `factor` converts such expressions into polynomials and rational expressions by substituting subexpressions with variables. After computing irreducible factors, the function restores original subexpressions.

```
factor((log(x)^2 - 1)/(cos(x)^2 - sin(x)^2))
```

```
ans =  
[ log(x) - 1, log(x) + 1, 1/(cos(x) - sin(x)), 1/(cos(x) + sin(x))]
```

Use `factor` to factor symbolic integers and symbolic rational numbers.

```
factor(sym(902834092))  
factor(1/sym(210))
```

```
ans =  
[ 2, 2, 47, 379, 12671]
```

```
ans =  
[ 1/2, 1/3, 1/5, 1/7]
```

`factor` also can factor numbers larger than `flintmax` that the MATLAB `factor` cannot. To represent a large number accurately, place the number in quotation marks.

```
factor(sym('41758540882408627201'))
```

```
ans =  
[ 479001599, 87178291199]
```

## Extract Subexpressions from Expression

The `children` function returns the subexpressions of an expression.

Define an expression `f` with several subexpressions.

```
syms x y
f = exp(3*x)*y^3 + exp(2*x)*y^2 + exp(x)*y;
```

Extract the subexpressions of `f` by using `children`.

```
expr = children(f)

expr =
    1x3 cell array
    {[y^2*exp(2*x)]}    {[y^3*exp(3*x)]}    {[y*exp(x)]}
```

You can extract lower-level subexpressions by calling `children` repeatedly on the results.

Extract the subexpressions of `expr{1}` by calling `children` on this expression.

```
expr1 = children(expr{1})

expr1 =
    1x2 cell array
    {[y^2]}    {[exp(2*x)]}
```

Access the contents of the cell array `expr1` using braces.

```
expr1{1}
expr1{2}

ans =
y^2
ans =
exp(2*x)
```

## Collect Terms with Same Powers

If a mathematical expression contains terms with the same powers of a specified variable or expression, the `collect` function reorganizes the expression by grouping such terms. When calling `collect`, specify the variables that the function must consider as unknowns. The `collect` function regards the original expression as a polynomial in the specified unknowns, and groups the coefficients with equal powers. Group the terms of an expression with the equal powers of `x`.

```
syms x y z
expr = x*y^4 + x*z + 2*x^3 + x^2*y*z + ...
      3*x^3*y^4*z^2 + y*z^2 + 5*x*y*z;
collect(expr, x)

ans =
(3*y^4*z^2 + 2)*x^3 + y*z*x^2 + (y^4 + 5*z*y + z)*x + y*z^2
```

Group the terms of the same expression with the equal powers of `y`.

```
collect(expr, y)
```

```
ans =
(3*x^3*z^2 + x)*y^4 + (x^2*z + 5*x*z + z^2)*y + 2*x^3 + z*x
```

Group the terms of the same expression with the equal powers of  $z$ .

```
collect(expr, z)
```

```
ans =
(3*x^3*y^4 + y)*z^2 + (x + 5*x*y + x^2*y)*z + 2*x^3 + x*y^4
```

If you do not specify variables that `collect` must consider as unknowns, the function uses `symvar` to determine the default variable.

```
collect(expr)
```

```
ans =
(3*y^4*z^2 + 2)*x^3 + y*z*x^2 + (y^4 + 5*z*y + z)*x + y*z^2
```

Collect terms of an expression with respect to several unknowns by specifying those unknowns as a vector.

```
collect(expr, [y,z])
```

```
ans =
3*x^3*y^4*z^2 + x*y^4 + y*z^2 + (x^2 + 5*x)*y*z + x*z + 2*x^3
```

## Rewrite Expressions in Terms of Other Functions

To present an expression in terms of a particular function, use `rewrite`. This function uses mathematical identities between functions. For example, rewrite an expression containing trigonometric functions in terms of a particular trigonometric function.

```
syms x
rewrite(sin(x),'tan')
```

```
ans =
(2*tan(x/2))/(tan(x/2)^2 + 1)
```

```
rewrite(cos(x),'tan')
```

```
ans =
-(tan(x/2)^2 - 1)/(tan(x/2)^2 + 1)
```

```
rewrite(sin(2*x) + cos(3*x)^2,'tan')
```

```
ans =
(tan((3*x)/2)^2 - 1)^2/(tan((3*x)/2)^2 + 1)^2 + ...
(2*tan(x))/(tan(x)^2 + 1)
```

Use `rewrite` to express these trigonometric functions in terms of the exponential function.

```
rewrite(sin(x),'exp')
```

```
ans =
(exp(-x*1i)*1i)/2 - (exp(x*1i)*1i)/2
```

```
rewrite(cos(x),'exp')
```

```
ans =
exp(-x*1i)/2 + exp(x*1i)/2
```



Use `rewrite` to express these hyperbolic functions in terms of the exponential function.

```
rewrite(sinh(x), 'exp')
```

```
ans =  
exp(x)/2 - exp(-x)/2
```

```
rewrite(cosh(x), 'exp')
```

```
ans =  
exp(-x)/2 + exp(x)/2
```

`rewrite` also expresses inverse hyperbolic functions in terms of logarithms.

```
rewrite(asinh(x), 'log')
```

```
ans =  
log(x + (x^2 + 1)^(1/2))
```

```
rewrite(acosh(x), 'log')
```

```
ans =  
log(x + (x - 1)^(1/2)*(x + 1)^(1/2))
```

## Compute Partial Fraction Decompositions of Expressions

The `partfrac` function returns a rational expression in the form of a sum of a polynomial and rational terms. In each rational term, the degree of the numerator is smaller than the degree of the denominator. For some expressions, `partfrac` returns visibly simpler forms.

```
syms x  
n = x^6 + 15*x^5 + 94*x^4 + 316*x^3 + 599*x^2 + 602*x + 247;  
d = x^6 + 14*x^5 + 80*x^4 + 238*x^3 + 387*x^2 + 324*x + 108;  
partfrac(n/d, x)
```

```
ans =  
1/(x + 1) + 1/(x + 2)^2 + 1/(x + 3)^3 + 1
```

The denominators in rational terms represent the factored common denominator of the original expression.

```
factor(d)
```

```
ans =  
[ x + 1, x + 2, x + 2, x + 3, x + 3, x + 3]
```

## Compute Normal Forms of Rational Expressions

The `simplifyFraction` function represents the original rational expression as a single rational term with expanded numerator and denominator. The greatest common divisor of the numerator and denominator of the returned expression is 1. This function is more efficient for simplifying fractions than the `simplify` function.

```
syms x y  
simplifyFraction((x^3 + 3*y^2)/(x^2 - y^2) + 3)
```

```
ans =  
(x^3 + 3*x^2)/(x^2 - y^2)
```

`simplifyFraction` cancels common factors that appear in numerator and denominator.

```
simplifyFraction(x^2/(x + y) - y^2/(x + y))
```

```
ans =  
x - y
```

`simplifyFraction` also handles expressions other than polynomials and rational functions. Internally, it converts such expressions into polynomials or rational functions by substituting subexpressions with identifiers. After normalizing the expression with temporary variables, `simplifyFraction` restores the original subexpressions.

```
simplifyFraction((exp(2*x) - exp(2*y))/(exp(x) - exp(y)))
```

```
ans =  
exp(x) + exp(y)
```

## Represent Polynomials Using Horner Nested Forms

The Horner, or nested, form of a polynomial expression is efficient for numerical evaluation because it often involves fewer arithmetical operations compared to other mathematically equivalent forms of the same polynomial. Typically, this form of an expression is numerically stable. To represent a polynomial expression in a nested form, use the `horner` function.

```
syms x  
horner(x^3 - 6*x^2 + 11*x - 6)
```

```
ans =  
x*(x*(x - 6) + 11) - 6
```

If polynomial coefficients are floating-point numbers, the resulting Horner form represents them as rational numbers.

```
horner(1.1 + 2.2*x + 3.3*x^2)
```

```
ans =  
x*((33*x)/10 + 11/5) + 11/10
```

To convert the coefficients in the result to floating-point numbers, use `vpa`.

```
vpa(ans)
```

```
ans =  
x*(3.3*x + 2.2) + 1.1
```

## Extract Numerators and Denominators of Rational Expressions

To extract the numerator and denominator of a rational symbolic expression, use the `numden` function. The first output argument of `numden` is a numerator, the second output argument is a denominator. Use `numden` to find numerators and denominators of symbolic rational numbers.

```
[n,d] = numden(1/sym(3))
```

```
n =  
1
```

```
d =  
3
```

Use `numden` to find numerators and denominators of a symbolic expressions.

```
syms x y  
[n,d] = numden((x^2 - y^2)/(x^2 + y^2))
```

```
n =  
x^2 - y^2
```

```
d =  
x^2 + y^2
```

Use `numden` to find numerators and denominators of symbolic functions. If the input is a symbolic function, `numden` returns the numerator and denominator as symbolic functions.

```
syms f(x) g(x)  
f(x) = sin(x)/x^2;  
g(x) = cos(x)/x;  
[n,d] = numden(f)
```

```
n(x) =  
sin(x)
```

```
d(x) =  
x^2
```

```
[n,d] = numden(f/g)
```

```
n(x) =  
sin(x)
```

```
d(x) =  
x*cos(x)
```

`numden` converts the input to its one-term rational form, such that the greatest common divisor of the numerator and denominator is 1. Then it returns the numerator and denominator of that form of the expression.

```
[n,d] = numden(x/y + y/x)
```

```
n =  
x^2 + y^2
```

```
d =  
x*y
```

`numden` works on vectors and matrices. If an input is a vector or matrix, `numden` returns two vectors or two matrices of the same size as the input. The first vector or matrix contains numerators of each element. The second vector or matrix contains denominators of each element. For example, find numerators and denominators of each element of the 3-by-3 Hilbert matrix.

```
H = sym(hilb(3))
```

```
H =  
[ 1, 1/2, 1/3]  
[ 1/2, 1/3, 1/4]  
[ 1/3, 1/4, 1/5]
```

```
[n,d] = numden(H)
```

```
n =  
[ 1, 1, 1]  
[ 1, 1, 1]  
[ 1, 1, 1]
```

```
d =  
[ 1, 2, 3]  
[ 2, 3, 4]  
[ 3, 4, 5]
```

## Simplify Symbolic Expressions

Simplification of a mathematical expression is not a clearly defined subject. There is no universal idea as to which form of an expression is simplest. The form of a mathematical expression that is simplest for one problem turns out to be complicated or even unsuitable for another problem. For example, the following two mathematical expressions present the same polynomial in different forms:

$$(x + 1)(x - 2)(x + 3)(x - 4),$$

$$x^4 - 2x^3 - 13x^2 + 14x + 24.$$

The first form clearly shows the roots of this polynomial. This form is simpler for working with the roots. The second form serves best when you want to see the coefficients of the polynomial. For example, this form is convenient when you differentiate or integrate polynomials.

If the problem you want to solve requires a particular form of an expression, the best approach is to choose the appropriate simplification function. See “Choose Function to Rearrange Expression” on page 3-123.

Besides specific simplifiers, Symbolic Math Toolbox offers a general simplifier, `simplify`.

If you do not need a particular form of expressions (expanded, factored, or expressed in particular terms), use `simplify` to shorten mathematical expressions. For example, use this simplifier to find a shorter form for a final result of your computations.

`simplify` works on various types of symbolic expressions, such as polynomials, expressions with trigonometric, logarithmic, and special functions. For example, simplify these polynomials.

```
syms x y
simplify((1 - x^2)/(1 - x))
simplify((x - 1)*(x + 1)*(x^2 + x + 1)*(x^2 + 1)*(x^2 - x + 1)*(x^4 - x^2 + 1))

ans =
x + 1

ans =
x^12 - 1
```

Simplify expressions involving trigonometric functions.

```
simplify(cos(x)^(-2) - tan(x)^2)
simplify(cos(x)^2 - sin(x)^2)

ans =
1

ans =
cos(2*x)
```

Simplify expressions involving exponents and logarithms. In the third expression, use `log(sym(3))` instead of `log(3)`. If you use `log(3)`, then MATLAB calculates `log(3)` with the double precision, and then converts the result to a symbolic number.

```
simplify(exp(x)*exp(y))
simplify(exp(x) - exp(x/2)^2)
simplify(log(x) + log(sym(3)) - log(3*x) + (exp(x) - 1)/(exp(x/2) + 1))
```

```
ans =
exp(x + y)
```

```
ans =
0
```

```
ans =
exp(x/2) - 1
```

Simplify expressions involving special functions.

```
simplify(gamma(x + 1) - x*gamma(x))
simplify(besselj(2, x) + besselj(0, x))
```

```
ans =
0
```

```
ans =
(2*besselj(1, x))/x
```

You also can simplify symbolic functions by using `simplify`.

```
syms f(x,y)
f(x,y) = exp(x)*exp(y)
f = simplify(f)
```

```
f(x, y) =
exp(x)*exp(y)
```

```
f(x, y) =
exp(x + y)
```

## Simplify Using Options

By default, `simplify` uses strict simplification rules and ensures that simplified expressions are always mathematically equivalent to initial expressions. For example, it does not combine logarithms for complex values in general.

```
syms x
simplify(log(x^2) + log(x))
```

```
ans =
log(x^2) + log(x)
```

You can apply additional simplification rules which are not correct for all values of parameters and all cases, but using which `simplify` can return shorter results. For this approach, use `IgnoreAnalyticConstraints`. For example, simplifying the same expression with `IgnoreAnalyticConstraints`, you get the result with combined logarithms.

```
simplify(log(x^2) + log(x), 'IgnoreAnalyticConstraints', true)
```

```
ans =
3*log(x)
```

`IgnoreAnalyticConstraints` provides a shortcut allowing you to simplify expressions under commonly used assumptions about values of the variables. Alternatively, you can set appropriate assumptions on variables explicitly. For example, combining logarithms is not valid for complex values

in general. If you assume that  $x$  is a real value, `simplify` combines logarithms without `IgnoreAnalyticConstraints`.

```
assume(x,'real')
simplify(log(x^2) + log(x))

ans =
log(x^3)
```

For further computations, clear the assumption on  $x$  by recreating it using `syms`.

```
syms x
```

Another approach that can improve simplification of an expression or function is the syntax `simplify(f, 'Steps', n)`, where  $n$  is a positive integer that controls how many steps `simplify` takes. Specifying more simplification steps can help you simplify the expression better, but it takes more time. By default,  $n = 1$ . For example, create and simplify this expression. The result is shorter than the original expression, but it can be simplified further.

```
syms x
y = (cos(x)^2 - sin(x)^2)*sin(2*x)*(exp(2*x) - 2*exp(x) + 1)/...
    ((cos(2*x)^2 - sin(2*x)^2)*(exp(2*x) - 1));
simplify(y)

ans =
(sin(4*x)*(exp(x) - 1))/(2*cos(4*x)*(exp(x) + 1))
```

Specify the number of simplification steps for the same expression. First, use 25 steps.

```
simplify(y,'Steps',25)

ans =
(tan(4*x)*(exp(x) - 1))/(2*(exp(x) + 1))
```

Use 50 steps to simplify the expression even further.

```
simplify(y,'Steps',50)

ans =
(tan(4*x)*tanh(x/2))/2
```

Suppose, you already simplified an expression or function, but you want the other forms of the same expression. To do this, you can set the 'All' option to `true`. The syntax `simplify(f, 'Steps', n, 'All', true)` shows other equivalent results of the same expression in the simplification steps.

```
syms x
y = cos(x) + sin(x)
simplify(y,'Steps',10,'All',true)

ans =
2^(1/2)*sin(x + pi/4)
2^(1/2)*cos(x - pi/4)
cos(x) + sin(x)
2^(1/2)*((exp(- x*1i - (pi*1i)/4)*1i)/2 - (exp(x*1i + (pi*1i)/4)*1i)/2)
```

To return even more equivalent results, increase the number of steps to 25.

```
simplify(y,'Steps',25,'All',true)
```

```

ans =
2^(1/2)*sin(x + pi/4)
2^(1/2)*cos(x - pi/4)
cos(x) + sin(x)
-2^(1/2)*(2*sin(x/2 - pi/8)^2 - 1)
2^(1/2)*(exp(- x*1i + (pi*1i)/4)/2 + exp(x*1i - (pi*1i)/4)/2)
2^(1/2)*((exp(- x*1i - (pi*1i)/4)*1i)/2 - (exp(x*1i + (pi*1i)/4)*1i)/2)

```

## Simplify Using Assumptions

Some expressions cannot be simplified in general, but become much shorter under particular assumptions. For example, simplifying this trigonometric expression without additional assumptions returns the original expression.

```

syms n
simplify(sin(2*n*pi))

```

```

ans =
sin(2*pi*n)

```

However, if you assume that variable  $n$  represents an integer, the same trigonometric expression simplifies to 0.

```

assume(n, 'integer')
simplify(sin(2*n*pi))

```

```

ans =
0

```

For further computations, clear the assumption.

```

syms n

```

## Simplify Fractions

You can use the general simplification function, `simplify`, to simplify fractions. However, Symbolic Math Toolbox offers a more efficient function specifically for this task: `simplifyFraction`. The statement `simplifyFraction(f)` represents the expression  $f$  as a fraction, where both the numerator and denominator are polynomials whose greatest common divisor is 1. For example, simplify these expressions.

```

syms x y
simplifyFraction((x^3 - 1)/(x - 1))

ans =
x^2 + x + 1

simplifyFraction((x^3 - x^2*y - x*y^2 + y^3)/(x^3 + y^3))

ans =
(x^2 - 2*x*y + y^2)/(x^2 - x*y + y^2)

```

By default, `simplifyFraction` does not expand expressions in the numerator and denominator of the returned result. To expand the numerator and denominator in the resulting expression, use the `Expand` option. For comparison, first simplify this fraction without `Expand`.

```

simplifyFraction((1 - exp(x)^4)/(1 + exp(x))^4)

```



```
ans =  
(exp(2*x) - exp(3*x) - exp(x) + 1)/(exp(x) + 1)^3
```

Now, simplify the same expressions with Expand.

```
simplifyFraction((1 - exp(x)^4)/(1 + exp(x))^4, 'Expand', true)
```

```
ans =  
(exp(2*x) - exp(3*x) - exp(x) + 1)/(3*exp(2*x) + exp(3*x) + 3*exp(x) + 1)
```

## Simplify Symbolic Expressions Using Live Editor Task

You can interactively simplify or rearrange symbolic expressions using the Simplify Symbolic Expression task in the Live Editor. For more information on Live Editor tasks, see “Add Interactive Tasks to a Live Script”.

This example shows you how to simplify or rearrange various symbolic expressions into the particular form you require by choosing the appropriate method.

### Simplify a Symbolic Expression

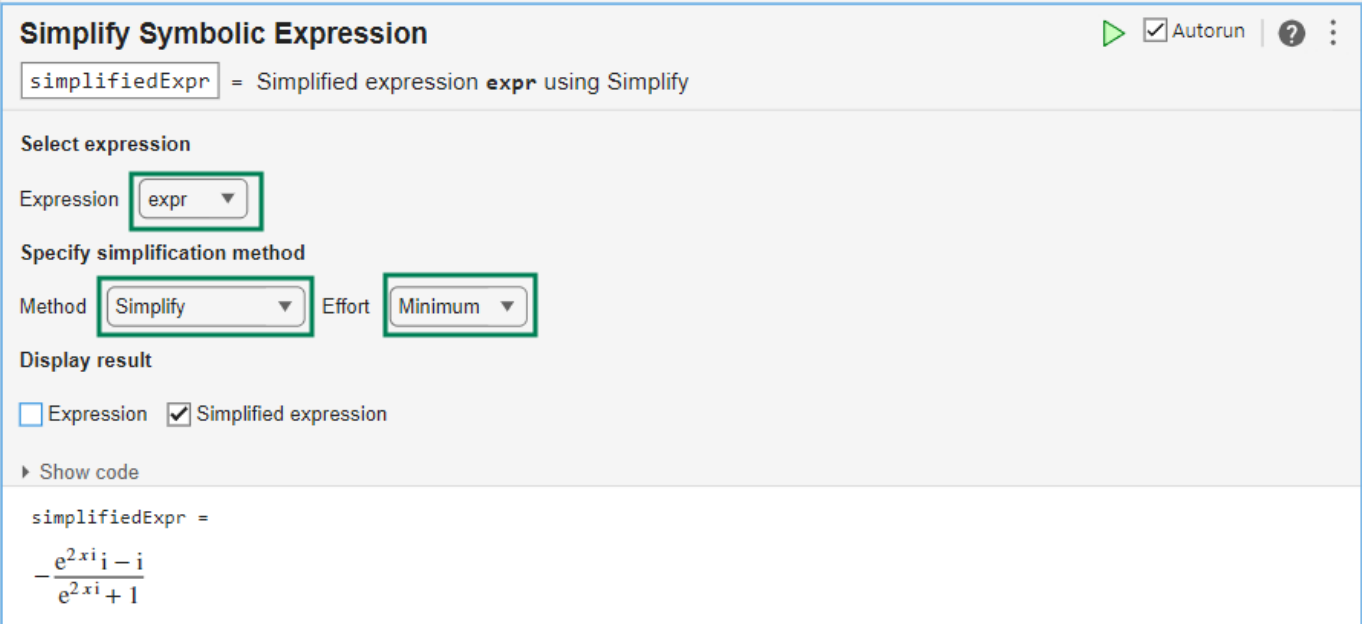
Simplify the expression  $i \frac{e^{-ix} - e^{ix}}{e^{-ix} + e^{ix}}$ .

First, go to the **Home** tab, and create a live script by clicking  **New Live Script**. Define the symbolic variable `x` and declare the expression as a symbolic expression.

```
syms x;
expr = 1i*(exp(-1i*x) - exp(1i*x))/(exp(-1i*x) + exp(1i*x));
```

In the **Live Editor** tab, run the code by clicking  **Run** to store `x` and `expr` into the current workspace.

Next, open the **Simplify Symbolic Expression** task by selecting **Task > Simplify Symbolic Expression** in the **Live Editor** tab. Select the symbolic expression `expr` from the workspace and specify the simplification method as **Simplify**. Select **Minimum** for the computational effort (fastest computation time).



**Simplify Symbolic Expression**  ☒ Autorun  

`simplifiedExpr` = Simplified expression `expr` using Simplify

Select expression

Expression expr ▼

Specify simplification method

Method Simplify ▼ Effort Minimum ▼

Display result

☐ Expression ☒ Simplified expression

► Show code

`simplifiedExpr =`

$$-\frac{e^{2xi}i - i}{e^{2xi} + 1}$$

To get a simpler expression, change the computational effort to **Medium**.

Expression expr ▼

Specify simplification method

Method Simplify ▼ Effort Medium ▼

Display result

☐ Expression ☒ Simplified expression

► Show code

```
simplifiedExpr = tan(x)
```

To experiment with simplifying symbolic expressions, you can repeat the previous steps for other symbolic expressions and simplification methods. You can run the following examples by adding the code to the existing live script or a new live script.

### Simplify a Polynomial Fraction

Simplify the polynomial fraction  $\frac{(x^2 - 1)(x + 1)}{x^2 - 2x + 1}$ .

Declare the polynomial fraction as a symbolic expression.

```
expr2 = ((x^2 - 1)*(x + 1))/(x^2 - 2*x + 1);
```

Select the symbolic expression `expr2` from the workspace and specify the simplification method as `Simplify fraction`.

**Simplify Symbolic Expression** ▶ ☒ Autorun ? ⋮

`simplifiedExpr` = Simplified expression `expr2` using Simplify fraction

Select expression

Expression expr2 ▼

Specify simplification method

Method Simplify fraction ▼ ☐ Expand

Display result

☐ Expression ☒ Simplified expression

► Show code

```
simplifiedExpr =

$$\frac{(x + 1)^2}{x - 1}$$

```

Select the **Expand** option to return the numerator and denominator of the simplified fraction in expanded form.

**Specify simplification method**

Method Simplify fraction ▼ ☒ Expand

**Display result**

☐ Expression ☒ Simplified expression

► Show code

```
simplifiedExpr =
```

$$\frac{x^2 + 2x + 1}{x - 1}$$

### Rewrite an Expression in a Different Form

Rewrite the trigonometric function  $\tan(x)$  in terms of the sine function.

Declare  $\tan(x)$  as a symbolic expression.

```
expr3 = tan(x);
```

Select the symbolic expression `expr3` from the workspace and specify the simplification method as **Rewrite**. Choose `sin` to rewrite  $\tan(x)$  in terms of the sine function.

**Simplify Symbolic Expression** ▶ ☒ Autorun ? ⋮

`simplifiedExpr` = Simplified expression `expr3` using Rewrite

**Select expression**

Expression expr3 ▼

**Specify simplification method**

Method Rewrite ▼ In terms of sin ▼

**Display result**

☐ Expression ☒ Simplified expression

► Show code

```
simplifiedExpr =
```

$$-\frac{\sin(x)}{2 \sin\left(\frac{x}{2}\right)^2 - 1}$$

### Expand a Logarithmic Expression

Expand the expression  $\log\left(\frac{x^3 e^x}{2}\right)$  using the logarithmic identities.

Declare the logarithmic expression as a symbolic expression.

```
expr4 = log(x^3*exp(x)/2);
```

Select the symbolic expression `expr4` from the workspace and specify the simplification method as `Expand`. By default, the symbolic variable `x` in `expr4` is complex when it is initially created. The `Expand` method does not simplify the input expression because the logarithmic identities are not valid for complex values of variables. To apply identities that are convenient but do not always hold for all values of variables, select the **Ignore analytic constraints** option.

☒ Autorun

**Simplify Symbolic Expression**

simplifiedExpr = Simplified expression **expr4** using Expand

Select expression

Expression
 

expr4

Specify simplification method

Method
 

Expand

☐ Multiply out brackets
 

☒ Ignore analytic constraints

Display result

☐ Expression
 ☒ Simplified expression

Show code

simplifiedExpr =  $x - \log(2) + 3 \log(x)$

### Simplify the Sum of Two Integral Expressions

Simplify the sum of two integral expressions:  $\int_a^b x f(x) dx + \int_a^b g(y) dy$ .

First, define `a` and `b` as symbolic variables, and `f(x)` and `g(y)` as symbolic functions. Use the `int` function to represent the integrals.

```
syms a b f(x) g(y)
expr5 = int(x*f(x),x,a,b) + int(g(y),y,a,b);
```

3-141

Select the symbolic expression `expr5` from the workspace and specify the simplification method as `Combine`. Choose `int` as the function to combine.

☒ Autorun

**Simplify Symbolic Expression**

`simplifiedExpr` = Simplified expression **expr5** using `Combine`

**Select expression**

Expression expr5

**Specify simplification method**

Method Combine Applied to int ☐ Ignore analytic constraints

**Display result**

☐ Expression ☒ Simplified expression

► Show code

```
simplifiedExpr =
  \int_a^b (g(x) + x f(x)) dx
```

### Generate Code

To view the code that a task used, click **Show code** at the bottom of the task window. The task displays the code block, which you can cut and paste to use or modify later in the existing script or a different program. For example:

**Display result**

☐ Expression ☒ Simplified expression

▼ Hide code

```
% Compute simplified symbolic expression
simplifiedExpr = combine(expr5,"int")

simplifiedExpr =
  \int_a^b (g(x) + x f(x)) dx
```

Because the underlying code is now part of your live script, you can continue to use the variables created by the task for further processing. For example, define the functions  $f(x)$  and  $g(x)$  as  $f(x) = x$  and  $g(x) = \cos(x)$ . Evaluate the integrals in `simplifiedExpr` by substituting these functions.

```
f(x) = x;  
g(x) = cos(x);  
evalIntegral = subs(simplifiedExpr)
```

```
evalIntegral =
```

$$\sin(b) - \sin(a) - \frac{a^3}{3} + \frac{b^3}{3}$$

## See Also

### Live Editor Tasks

[Simplify Symbolic Expression](#) | [Solve Symbolic Equation](#)

### Functions

[simplify](#) | [simplifyFraction](#) | [combine](#) | [expand](#) | [rewrite](#)

## Related Examples

- [“Add Interactive Tasks to a Live Script”](#)
- [“Choose Function to Rearrange Expression”](#) on page 3-123
- [Simplify Symbolic Expression](#)

## Substitute Parameters in Symbolic Solutions

Solve the following trigonometric equation using the `ReturnConditions` option of the solver to obtain the complete solution. The solver returns the solution, parameters used in the solution, and conditions on those parameters.

```
syms x
eqn = sin(2*x) + cos(x) == 0;
[solx, params, conds] = solve(eqn, x, 'ReturnConditions', true)

solx =
    pi/2 + pi*k
    2*pi*k - pi/6
    (7*pi)/6 + 2*pi*k

params =
    k

conds =
    in(k, 'integer')
    in(k, 'integer')
    in(k, 'integer')
```

Replace the parameter `k` with a new symbolic variable `a`. First, create symbolic variables `k` and `a`. (The solver does not create variable `k` in the MATLAB workspace.)

```
syms k a
```

Now, use the `subs` function to replace `k` by `a` in the solution vector `solx`, parameters `params`, and conditions `conds`.

```
solx = subs(solx, k, a)
params = subs(params, k, a)
conds = subs(conds, k, a)

solx =
    pi/2 + pi*a
    2*pi*a - pi/6
    (7*pi)/6 + 2*pi*a
params =
    a
conds =
    in(a, 'integer')
    in(a, 'integer')
    in(a, 'integer')
```

Suppose, you know that the value of the parameter `a` is 2. Substitute `a` with 2 in the solution vector `solx`.

```
subs(solx, a, 2)
```

```
ans =
    (5*pi)/2
    (23*pi)/6
    (31*pi)/6
```

Alternatively, substitute `params` with 2. This approach returns the same result.





## Substitute Elements in Symbolic Matrices

Create a 2-by-2 matrix **A** with automatically generated elements using `sym`. The generated elements  $A_{1,1}$ ,  $A_{1,2}$ ,  $A_{2,1}$ , and  $A_{2,2}$  do not appear in the MATLAB® workspace.

```
A = sym('A',[2 2])
```

$$A = \begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix}$$

Substitute the element  $A_{1,2}$  with a value 5. Assign the value directly by indexing into the matrix element.

```
A(1,2) = 5
```

$$A = \begin{pmatrix} A_{1,1} & 5 \\ A_{2,1} & A_{2,2} \end{pmatrix}$$

Alternatively, you can create a 2-by-2 matrix using `syms`. Create a matrix **B** using `syms`.

```
syms B [2 2]
```

$$B = \begin{pmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{pmatrix}$$

The generated elements  $B_{1,1}$ ,  $B_{1,2}$ ,  $B_{2,1}$ , and  $B_{2,2}$  appear as symbolic variables `B1_1`, `B1_2`, `B2_1`, and `B2_2` in the MATLAB workspace. Use `subs` to substitute the element of **B** by specifying the variable name. For example, substitute `B2_2` with 4.

```
B = subs(B,B2_2,4)
```

$$B = \begin{pmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & 4 \end{pmatrix}$$

You can also create a matrix by specifying the elements individually. Create a 3-by-3 circulant matrix **M**.

```
syms a b c
M = [a b c; b c a; c a b]
```

$$M = \begin{pmatrix} a & b & c \\ b & c & a \\ c & a & b \end{pmatrix}$$

Replace variable **b** in the matrix **M** by the expression `a + 1`. The `subs` function replaces all **b** elements in matrix **M** with the expression `a + 1`.

```
M = subs(M,b,a+1)
```

$$M = \begin{pmatrix} a & a+1 & c \\ a+1 & c & a \\ c & a & a+1 \end{pmatrix}$$

Next, replace all elements whose value is  $c$  with  $a + 2$ . You can specify the value to replace as  $c$ ,  $M(1,3)$  or  $M(3,1)$ .

$$M = \text{subs}(M, M(1,3), a+2)$$

$$M = \begin{pmatrix} a & a+1 & a+2 \\ a+1 & a+2 & a \\ a+2 & a & a+1 \end{pmatrix}$$

To replace a particular element of a matrix with a new value while keeping all other elements unchanged, use the assignment operation. For example,  $M(1,1) = 2$  replaces only the first element of the matrix  $M$  with the value 2.

Find eigenvalues and eigenvectors of the matrix  $M$ .

$$[V, E] = \text{eig}(M)$$

$$V = \begin{pmatrix} 1 & \frac{\sqrt{3}}{2} - \frac{1}{2} & -\frac{\sqrt{3}}{2} - \frac{1}{2} \\ 1 & -\frac{\sqrt{3}}{2} - \frac{1}{2} & \frac{\sqrt{3}}{2} - \frac{1}{2} \\ 1 & 1 & 1 \end{pmatrix}$$

$$E = \begin{pmatrix} 3a+3 & 0 & 0 \\ 0 & \sqrt{3} & 0 \\ 0 & 0 & -\sqrt{3} \end{pmatrix}$$

Replace the symbolic parameter  $a$  with the value 1.

$$\text{subs}(E, a, 1)$$

$$\text{ans} = \begin{pmatrix} 6 & 0 & 0 \\ 0 & \sqrt{3} & 0 \\ 0 & 0 & -\sqrt{3} \end{pmatrix}$$

## Substitute Scalars with Matrices

Create the following expression representing the sine function.

```
syms w t
f = sin(w*t);
```

Suppose, your task involves creating a matrix whose elements are sine functions with angular velocities represented by a Toeplitz matrix. First, create a 4-by-4 Toeplitz matrix.

```
W = toeplitz(sym([3 2 1 0]))
```

```
W =
[ 3, 2, 1, 0]
[ 2, 3, 2, 1]
[ 1, 2, 3, 2]
[ 0, 1, 2, 3]
```

Next, replace the variable  $w$  in the expression  $f$  with the Toeplitz matrix  $W$ . When you replace a scalar in a symbolic expression with a matrix, `subs` expands the expression into a matrix. In this example, `subs` expands  $f = \sin(w*t)$  into a 4-by-4 matrix whose elements are  $\sin(w*t)$ . Then it replaces  $w$  in that matrix with the corresponding elements of the Toeplitz matrix  $W$ .

```
F = subs(f, w, W)
```

```
F =
[ sin(3*t), sin(2*t), sin(t), 0]
[ sin(2*t), sin(3*t), sin(2*t), sin(t)]
[ sin(t), sin(2*t), sin(3*t), sin(2*t)]
[ 0, sin(t), sin(2*t), sin(3*t)]
```

Find the sum of these sine waves at  $t = \pi$ ,  $t = \pi/2$ ,  $t = \pi/3$ ,  $t = \pi/4$ ,  $t = \pi/5$ , and  $t = \pi/6$ . First, find the sum of all elements of matrix  $F$ . Here, the first call to `sum` returns a row vector containing sums of elements in each column. The second call to `sum` returns the sum of elements of that row vector.

```
S = sum(sum(F))
```

```
S =
6*sin(2*t) + 4*sin(3*t) + 4*sin(t)
```

Now, use `subs` to evaluate  $S$  for particular values of the variable  $t$ .

```
subs(S, t, sym(pi)./[1:6])
```

```
[ 0,...
 0,...
 5*3^(1/2), 4*2^(1/2) + 6,...
 2^(1/2)*(5 - 5^(1/2))^(1/2) + (5*2^(1/2)*(5^(1/2) + 5)^(1/2))/2,...
 3*3^(1/2) + 6]
```

You also can use `subs` to replace a scalar element of a matrix with another matrix. In this case, `subs` expands the matrix to accommodate new elements. For example, replace zero elements of the matrix  $F$  with a column vector  $[1; 2]$ . The original 4-by-4 matrix  $F$  expands to an 8-by-4 matrix. The `subs` function duplicates each row of the original matrix, not only the rows containing zero elements.

```
F = subs(F, 0, [1;2])
```

```
F =  
[ sin(3*t), sin(2*t),  sin(t),      1]  
[ sin(3*t), sin(2*t),  sin(t),      2]  
[ sin(2*t), sin(3*t), sin(2*t),  sin(t)]  
[ sin(2*t), sin(3*t), sin(2*t),  sin(t)]  
[  sin(t), sin(2*t), sin(3*t), sin(2*t)]  
[  sin(t), sin(2*t), sin(3*t), sin(2*t)]  
[      1,  sin(t), sin(2*t), sin(3*t)]  
[      2,  sin(t), sin(2*t), sin(3*t)]
```

## Substitute Values into Symbolic Expressions Using subs

The `subs` function in Symbolic Math Toolbox™ allows you to substitute values into symbolic expressions. The `subs` function is useful for evaluating, simplifying, and manipulating symbolic expressions. You can use `subs` to replace symbolic scalar variables, vectors, matrices, matrix variables, or systems of equations with specific numeric or symbolic values.

These examples show how to use `subs` for a variety of tasks, such as evaluating expressions at given values, simplifying results, verifying solutions to equations, assigning values to variables with different names, and substituting values into vectors and matrices.

### Substitute Value into Expression

To evaluate a symbolic expression at a given value, substitute the value into the expression.

For example, create a symbolic expression using the symbolic variable `x`.

```
syms x
y = x*log(x) - 1;
expr = y/diff(y)
```

```
expr =

$$\frac{x \log(x) - 1}{\log(x) + 1}$$

```

Evaluate the expression `expr` at `x` equal to 2. The result is an exact symbolic number.

```
expr_eval = subs(expr,x,2)
```

```
expr_eval =

$$\frac{2 \log(2) - 1}{\log(2) + 1}$$

```

To convert the symbolic number to double precision, use the `double` function.

```
expr_double = double(expr_eval)
```

```
expr_double =
0.2282
```

If you want to use variable-precision arithmetic, you can use `vpa` and `digits`. To change the output display format, use `sympref`. For more information, see “Change Output Format of Symbolic and Variable-Precision Arithmetic” on page 2-8.

### Substitute and Automatically Simplify Result

When you substitute values in a symbolic expression, the `subs` function attempts to simplify the result.

For example, create a symbolic expression using the symbolic variables `a`, `b`, and `x`.

```
syms a b x
expr = (5*x - sqrt(3))/sqrt(a^2 + b^2) - x
```

```
expr =

$$\frac{5x - \sqrt{3}}{\sqrt{a^2 + b^2}} - x$$

```

Substitute the variables `a` and `b` with the values 3 and 4, respectively.

```
expr_eval = subs(expr,[a b],[3 4])
```

```
expr_eval =  
- $\frac{\sqrt{3}}{5}$ 
```

In this case, the result does not contain the variable `x`. The reason is that the `subs` function simplifies the evaluated expression by removing the terms with the variable `x` that cancel each other out.

### Verify Solutions Using Substitution

You can solve a system of equations symbolically and then verify the solutions using substitution.

For example, create a system of two equations using the symbolic variables `x`, `y`, and `a`.

```
syms x y a  
eqns = [x^2 + y^2 == a; x == y + a]
```

```
eqns =  
( $x^2 + y^2 = a$   
 $x = a + y$ )
```

Solve the equations for the variables `x` and `y` using the `solve` function.

```
sols = solve(eqns,[x y]);  
sols.x
```

```
ans =  
( $\frac{a}{2} - \frac{\sqrt{-a(a-2)}}{2}$   
 $\frac{a}{2} + \frac{\sqrt{-a(a-2)}}{2}$ )
```

```
sols.y
```

```
ans =  
( $-\frac{a}{2} - \frac{\sqrt{-a(a-2)}}{2}$   
 $\frac{\sqrt{-a(a-2)}}{2} - \frac{a}{2}$ )
```

To verify the solutions, substitute the values in `sols` into the original system of equations.

```
eqns_eval = subs(eqns,sols)
```

```
eqns_eval =
```

$$\begin{pmatrix} \sigma_1 = a \\ \sigma_1 = a \\ \sigma_2 = \sigma_2 \\ \sigma_3 = \sigma_3 \end{pmatrix}$$

where

$$\sigma_1 = \sigma_2^2 + \sigma_3^2$$

$$\sigma_2 = \frac{a}{2} - \frac{\sqrt{-a(a-2)}}{2}$$

$$\sigma_3 = \frac{a}{2} + \frac{\sqrt{-a(a-2)}}{2}$$

Here, it is not obvious if the equalities hold true because the substituted expressions might require further simplification. To check if the equalities are always true, you can use the `isAlways` function. For more information, see “Check Symbolic Equations, Inequalities, and Conditional Statements” on page 2-23.

```
tf = isAlways(eqns_eval)
```

```
tf = 4x1 logical array
```

```
1
1
1
1
```

### Use Different Variable Names During Substitution

When substituting values into symbolic expressions, use variables with names that are different from those in the original expression to assign specific values. This best practice helps avoid unintentionally replacing existing symbolic variables.

For example, create a symbolic function that includes the symbolic variables `b` and `x`.

```
syms b x
f(x) = 2*b^3*x^3 - b^2*x^2 + 3*b*x + 1
```

$$f(x) = 2b^3x^3 - b^2x^2 + 3bx + 1$$

To demonstrate the potentially confusing naming practice, assign the value 2 to `b`. The function `f` is still expressed in terms of both `b` and `x`.

```
b = 2;
f
```

$$f(x) = 2b^3x^3 - b^2x^2 + 3bx + 1$$

To substitute the assigned value of `b` into the function `f`, use `subs(f)`.

```
f_eval = subs(f)
```

$$f\_eval(x) = 16x^3 - 4x^2 + 6x + 1$$



Because `b` has the value 2, any new function or expression you create using the variable `b` uses this assigned value.

$$g(x) = x + b$$

$$g(x) = x + 2$$

Instead, the best practice is to assign the value to a new variable with a different name. For example, re-create the symbolic variables `b` and `x`, and the symbolic function `f`.

```
syms b x
f(x) = 2*b^3*x^3 - b^2*x^2 + 3*b*x + 1
```

$$f(x) = 2b^3x^3 - b^2x^2 + 3bx + 1$$

Specify a new variable `bVal` with the assigned value 2.

```
bVal = 2;
```

Substitute `b` in the function `f` with the value `bVal`.

```
f_eval = subs(f,b,bVal)
```

$$f\_eval(x) = 16x^3 - 4x^2 + 6x + 1$$

Now, if you create a new function or expression using the variable `b`, this variable still represents a symbolic variable.

$$g(x) = x + b$$

$$g(x) = b + x$$

### Substitute Scalar Variables with Matrix Values

You can substitute symbolic scalar variables with matrix values in symbolic expressions. The `subs` function automatically expands the expressions and performs element-wise substitutions. If instead you want variables in a symbolic expression to accept matrix values, you can use symbolic matrix variables.

For example, create a symbolic expression using the symbolic scalar variables `A`, `B`, and `t`.

```
syms A B t
expr = B + exp(A*t)
```

$$expr = B + e^{At}$$

Substitute the variables `A` and `B` with 2-by-2 matrix values. To substitute these variables at the same time, specify the variables to substitute and their corresponding values as cell arrays. Here, `subs` automatically expands the original expression to a 2-by-2 size and performs element-wise substitution using the specified values.

```
Aval = [2 3; 1 2];
Bval = [0 1; 2 3];
expr_eval = subs(expr,{A B},{Aval Bval})
```

$$expr\_eval = \begin{pmatrix} e^{2t} & e^{3t} + 1 \\ e^t + 2e^{2t} + 3 & \end{pmatrix}$$

As an alternative, you can define A and B as symbolic matrix variables with a size of 2-by-2. Symbolic matrix variables represent matrices, vectors, and scalars in compact matrix notation. For more information, see `syms` and `symmatrix`.

```
syms A B [2 2] matrix
syms t
expr = B + exp(A*t)
```

$$\text{expr} = e^{tA} + B$$

Substitute the matrix variables A and B with 2-by-2 matrix values.

```
Aval = [2 3; 1 2];
Bval = [0 1; 2 3];
expr_eval = subs(expr, {A B}, {Aval Bval})
```

$$\text{expr\_eval} = e^{t\Sigma_1} + \Sigma_2$$

where

$$\Sigma_1 = \begin{pmatrix} 2 & 3 \\ 1 & 2 \end{pmatrix}$$

$$\Sigma_2 = \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix}$$

When you use symbolic matrix variables of the `symmatrix` data type, the expression is displayed in compact matrix notation. To convert a symbolic matrix variable to a matrix of scalar values of the `sym` data type, use `symmatrix2sym`.

```
expr_eval = symmatrix2sym(expr_eval)
```

$$\text{expr\_eval} = \begin{pmatrix} e^{2t} & e^{3t} + 1 \\ e^t + 2e^{2t} + 3 \end{pmatrix}$$

### Substitute Multiple Vectors with Specified Values

You can substitute multiple vectors of symbolic scalar variables with their specified values by performing the substitutions one by one. To substitute multiple vectors at the same time, you can use symbolic matrix variables.

For example, create a symbolic expression using the arrays of symbolic scalar variables A, B, and t. Here, A is a 1-by-3 vector with elements [A1 A2 A3], and B is a 4-by-1 vector with elements [B1; B2; B3; B4].

```
syms A [1 3]
syms B [4 1]
syms t
C = [A t]*B
```

$$C = B_4 t + A_1 B_1 + A_2 B_2 + A_3 B_3$$

Substitute A and B with their specified values. When you want to substitute multiple vectors of symbolic scalar variables, you must substitute one vector at a time. First, substitute the symbolic vector A.

```
Aval = [1 2 3];
Bval = [0; 1; 2; 3];
C_eval = subs(C,A,Aval)
```

$$C\_eval = B_1 + 2B_2 + 3B_3 + B_4t$$

Next, substitute the symbolic vector B.

```
C_eval = subs(C_eval,B,Bval)
C_eval = 3t + 8
```

As an alternative, you can define A and B as symbolic matrix variables with sizes 1-by-3 and 4-by-1, respectively.

```
syms A [1 3] matrix
syms B [4 1] matrix
syms t
C = [A t]*B
```

$$C = (A \ t)B$$

Substitute the matrix variables A and B with their specified values at the same time.

```
Aval = [1 2 3];
Bval = [0; 1; 2; 3];
C_eval = subs(C,{A B},{Aval Bval})
```

$$C\_eval = (\Sigma_1 \ t) \Sigma_2$$

where

$$\Sigma_1 = (1 \ 2 \ 3)$$

$$\Sigma_2 = \begin{pmatrix} 0 \\ 1 \\ 2 \\ 3 \end{pmatrix}$$

Convert the symbolic matrix variables of type `symmatrix` to matrices of scalar values of type `sym`.

```
C_eval = symmatrix2sym(C_eval)
C_eval = 3t + 8
```

## See Also

`subs` | `solve` | `syms` | `symmatrix` | `symmatrix2sym`

## Related Examples

- “Substitute Elements in Symbolic Matrices” on page 3-146
- “Substitute Scalars with Matrices” on page 3-148
- “Substitute Parameters in Symbolic Solutions” on page 3-144
- “Change Output Format of Symbolic and Variable-Precision Arithmetic” on page 2-8

- “Check Symbolic Equations, Inequalities, and Conditional Statements” on page 2-23

## Abbreviate Common Terms in Long Expressions

Long expressions often contain several instances of the same subexpression. Such expressions look shorter if the same subexpression is replaced with an abbreviation. You can use `sympref` to specify whether or not to use abbreviated output format of symbolic expressions in live scripts.

For example, solve the equation  $\sqrt{x} + \frac{1}{x} = 1$  using `solve`.

```
syms x
sols = solve(sqrt(x) + 1/x == 1, x)
```

```
sols =
```

$$\left( \left( \frac{1}{18\sigma_2} + \frac{\sigma_2}{2} + \frac{1}{3} - \sigma_1 \right)^2 \right)$$

$$\left( \left( \frac{1}{18\sigma_2} + \frac{\sigma_2}{2} + \frac{1}{3} + \sigma_1 \right)^2 \right)$$

where

$$\sigma_1 = \frac{\sqrt{3} \left( \frac{1}{9\sigma_2} - \sigma_2 \right) i}{2}$$

$$\sigma_2 = \left( \frac{25}{54} - \frac{\sqrt{23}\sqrt{108}}{108} \right)^{1/3}$$

The `solve` function returns exact solutions as symbolic expressions. By default, live scripts display symbolic expressions in abbreviated output format. The symbolic setting uses an internal algorithm to choose which subexpressions to abbreviate, which can also include nested abbreviations. For example, the term  $\sigma_1$  contains the subexpression abbreviated as  $\sigma_2$ . The symbolic setting does not provide any options to choose which subexpressions to abbreviate.

You can turn off abbreviated output format by setting `'AbbreviateOutput'` to `false`. The returned result is a long expression that is difficult to read.

```
sympref('AbbreviateOutput', false);
sols
```

```
sols =
```

$$\begin{pmatrix} \left( \frac{1}{18 \left( \frac{25}{54} - \frac{\sqrt{23}\sqrt{108}}{108} \right)^{1/3}} + \frac{\left( \frac{25}{54} - \frac{\sqrt{23}\sqrt{108}}{108} \right)^{1/3}}{2} + \frac{1}{3} - \frac{\sqrt{3} \left( \frac{1}{9 \left( \frac{25}{54} - \frac{\sqrt{23}\sqrt{108}}{108} \right)^{1/3}} - \left( \frac{25}{54} - \frac{\sqrt{23}\sqrt{108}}{108} \right)^{1/3} \right) i}{2} \right)^2 \\ \left( \frac{1}{18 \left( \frac{25}{54} - \frac{\sqrt{23}\sqrt{108}}{108} \right)^{1/3}} + \frac{\left( \frac{25}{54} - \frac{\sqrt{23}\sqrt{108}}{108} \right)^{1/3}}{2} + \frac{1}{3} + \frac{\sqrt{3} \left( \frac{1}{9 \left( \frac{25}{54} - \frac{\sqrt{23}\sqrt{108}}{108} \right)^{1/3}} - \left( \frac{25}{54} - \frac{\sqrt{23}\sqrt{108}}{108} \right)^{1/3} \right) i}{2} \right)^2 \end{pmatrix}$$

The settings you set using `sympref` persist through your current and future MATLAB® sessions. Restore the default values of 'AbbreviateOutput' by specifying the 'default' option.

```
sympref('AbbreviateOutput','default');
```

`subexpr` is another function that you can use to shorten long expressions. This function abbreviates only one common subexpression, and unlike `sympref`, it does not support nested abbreviations. Like `sympref`, `subexpr` also does not let you choose which subexpressions to replace.

Use the second input argument of `subexpr` to specify the variable name that replaces the common subexpression. For example, replace the common subexpression in `sols` with the variable `t`.

```
[sols1,t] = subexpr(sols,'t')
```

```
sols1 =
```

$$\begin{pmatrix} \left( \frac{t}{2} + \frac{1}{18t} + \frac{1}{3} + \frac{\sqrt{3} \left( t - \frac{1}{9t} \right) i}{2} \right)^2 \\ \left( \frac{t}{2} + \frac{1}{18t} + \frac{1}{3} - \frac{\sqrt{3} \left( t - \frac{1}{9t} \right) i}{2} \right)^2 \end{pmatrix}$$

```
t =
```

$$\left( \frac{25}{54} - \frac{\sqrt{23}\sqrt{108}}{108} \right)^{1/3}$$

Although `sympref` and `subexpr` do not provide a way to choose which subexpressions to replace in a solution, you can define these subexpressions as symbolic variables and manually rewrite the solution.

For example, define new symbolic variables `a1` and `a2`.

```
syms a1 a2
```

Rewrite the solutions `sols` in terms of `a1` and `a2` before assigning the values of `a1` and `a2` to avoid evaluating `sols`.

```
sols = [(1/2*a1 + 1/3 + sqrt(3)/2*a2*1i)^2; ...
        (1/2*a1 + 1/3 - sqrt(3)/2*a2*1i)^2]
```

`sols =`

$$\left( \left( \frac{a_1}{2} + \frac{1}{3} + \frac{\sqrt{3} a_2 i}{2} \right)^2 \right)$$

$$\left( \left( \frac{a_1}{2} + \frac{1}{3} - \frac{\sqrt{3} a_2 i}{2} \right)^2 \right)$$

Assign the values  $\left(t + \frac{1}{9t}\right)$  and  $\left(t - \frac{1}{9t}\right)$  to `a1` and `a2`, respectively.

`a1 = t + 1/(9*t)`

`a1 =`

$$\frac{1}{9 \left( \frac{25}{54} - \frac{\sqrt{23} \sqrt{108}}{108} \right)^{1/3}} + \left( \frac{25}{54} - \frac{\sqrt{23} \sqrt{108}}{108} \right)^{1/3}$$

`a2 = t - 1/(9*t)`

`a2 =`

$$\left( \frac{25}{54} - \frac{\sqrt{23} \sqrt{108}}{108} \right)^{1/3} - \frac{1}{9 \left( \frac{25}{54} - \frac{\sqrt{23} \sqrt{108}}{108} \right)^{1/3}}$$

Evaluate `sols` using `subs`. The result is identical to the first output in this example.

`sols_eval = subs(sols)`

`sols_eval =`

$$\left( \left( \frac{1}{18 \sigma_2} + \frac{\sigma_2}{2} + \frac{1}{3} - \sigma_1 \right)^2 \right)$$

$$\left( \left( \frac{1}{18 \sigma_2} + \frac{\sigma_2}{2} + \frac{1}{3} + \sigma_1 \right)^2 \right)$$

where

$$\sigma_1 = \frac{\sqrt{3} \left( \frac{1}{9 \sigma_2} - \sigma_2 \right) i}{2}$$

$$\sigma_2 = \left( \frac{25}{54} - \frac{\sqrt{23} \sqrt{108}}{108} \right)^{1/3}$$

## Harmonic Analysis of Transfer Function Output

This example finds closed-form solutions for the coefficients of frequencies in an output signal. The output signal results from passing an input through an analytical nonlinear transfer function.

This example uses the following Symbolic Math Toolbox™ capabilities:

- Taylor series expansion using `taylor`
- “Formula Manipulation and Simplification”

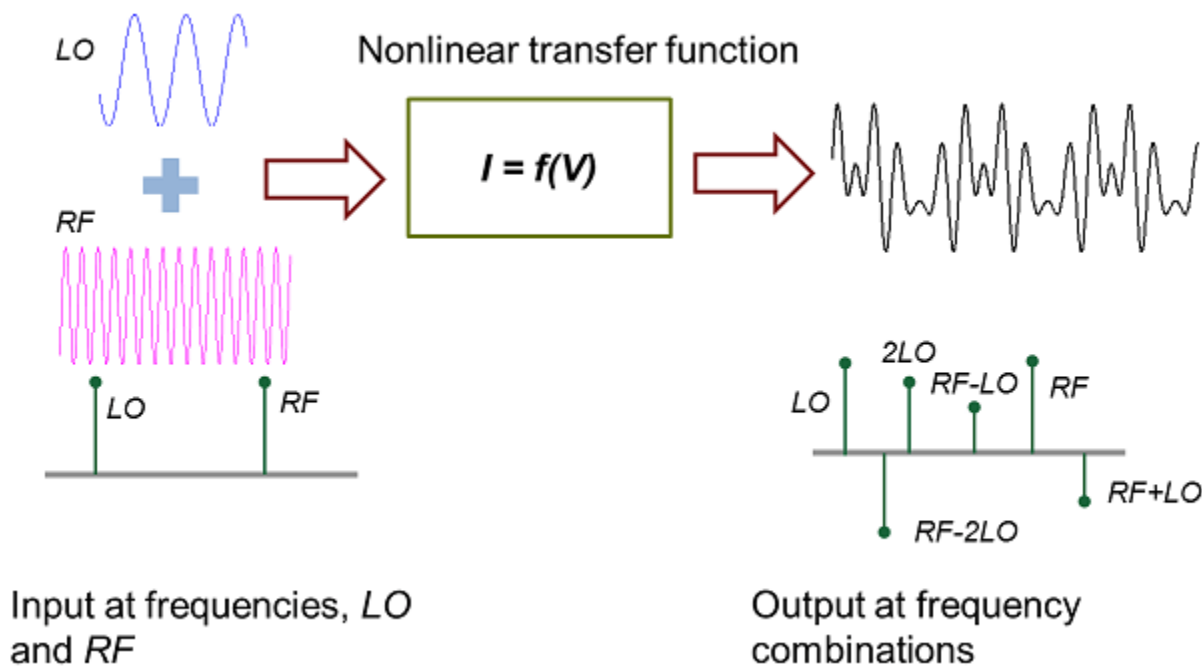
### Motivation

To motivate the solution, we take a simple element from circuit theory: an ideal diode (in forward bias operation). The current,  $I$ , is the output that depends exponentially on the input,  $V$ . Diodes have found use in creating devices such as mixers and amplifiers where understanding the harmonic structure of the output can be useful in characterizing the device and meeting design specifications.

```
syms Is V Vo real;
I = Is*(exp(V/Vo) - 1)
```

$$I = I_s (e^{V/V_o} - 1)$$

If  $V$  is a linear combination of 2 signals at frequencies  $LO$  and  $RF$ , the nonlinear transfer function will mix  $LO$  and  $RF$  to create output with content at combinatorial harmonic frequency combinations:  $f_{reqs} = \{LO, 2LO, RF, 2RF, LO-RF, LO-2RF, \dots\}$ .



The objective of this example is to determine the coefficients of  $f_{reqs}$  in the output.



### Define Input Signal

The input signal is a linear combination of two cosine signals.

```
syms c1 c2 t L0 RF real;
input = c1*cos(L0*t) + c2*cos(RF*t)
```

$$\text{input} = c_1 \cos(L0 t) + c_2 \cos(RF t)$$

### Define Space of Harmonic Frequency Combinations

Below, `harmCombinations` is a combinatorial combination of the integer multiples of the input frequencies `L0` and `RF`. We restrict the space of interest defined by 3 harmonics each in the `L0` and `RF` directions.

```
n = 3;
harmCombinations = [kron((0:n)', ones(n*2+1,1)), repmat((-n:n)', n+1,1)];
freqs = harmCombinations*[L0;RF];
```

The first `n` frequencies are just the negative harmonic frequencies and are therefore redundant considering that the input signal is real.

```
freqs = freqs(n+1:end)
```

```
freqs =
    0
    RF
    2 RF
    3 RF
    LO - 3 RF
    LO - 2 RF
    LO - RF
    LO
    LO + RF
    LO + 2 RF
    LO + 3 RF
    2 LO - 3 RF
    2 LO - 2 RF
    2 LO - RF
    2 LO
    2 LO + RF
    2 LO + 2 RF
    2 LO + 3 RF
    3 LO - 3 RF
    3 LO - 2 RF
    3 LO - RF
    3 LO
    3 LO + RF
    3 LO + 2 RF
    3 LO + 3 RF
```

### Taylor Expansion

To cover the frequency spectrum of interest, a Taylor series of order four for  $I(V)$  is sufficient.

```
s = taylor(I, V, 'Order', 4)
```

$$s = \frac{I_s V^2}{2 V_0^2} + \frac{I_s V^3}{6 V_0^3} + \frac{I_s V}{V_0}$$

Use an input signal combination of the LO and RF frequencies and express  $f$  in terms of  $\cos(LO \cdot t)$  and  $\cos(RF \cdot t)$ .

```
f0 = subs(s, V, input);  
f = expand(f0)
```

$$f = \frac{I_s c_1^2 \sigma_2}{2 V_0^2} + \frac{I_s c_1^3 \cos(LO \cdot t)^3}{6 V_0^3} + \frac{I_s c_2^2 \sigma_1}{2 V_0^2} + \frac{I_s c_2^3 \cos(RF \cdot t)^3}{6 V_0^3} + \frac{I_s c_1 \cos(LO \cdot t)}{V_0} + \frac{I_s c_2 \cos(RF \cdot t)}{V_0} + \frac{I_s c_1 c_2 \cos(LO \cdot t) \cos(RF \cdot t)}{V_0^2}$$

where

$$\sigma_1 = \cos(RF \cdot t)^2$$

$$\sigma_2 = \cos(LO \cdot t)^2$$

Rewrite  $f$  in terms of single powers of cosines.

```
f = combine(f, 'sincos')
```

$$f = \frac{I_s c_1^2}{4 V_0^2} + \frac{I_s c_2^2}{4 V_0^2} + \frac{I_s c_1 \cos(LO \cdot t)}{V_0} + \frac{I_s c_2 \cos(RF \cdot t)}{V_0} + \frac{I_s c_1^2 \cos(2 LO \cdot t)}{4 V_0^2} + \frac{I_s c_1^3 \cos(LO \cdot t)}{8 V_0^3} + \frac{I_s c_1^3 \cos(3 LO \cdot t)}{24 V_0^3} \\ + \frac{I_s c_2^2 \cos(2 RF \cdot t)}{4 V_0^2} + \frac{I_s c_2^3 \cos(RF \cdot t)}{8 V_0^3} + \frac{I_s c_2^3 \cos(3 RF \cdot t)}{24 V_0^3} + \frac{I_s c_1 c_2^2 \cos(LO \cdot t)}{4 V_0^3} + \frac{I_s c_1^2 c_2 \cos(RF \cdot t)}{4 V_0^3} \\ + \frac{I_s c_1 c_2 \cos(LO \cdot t + RF \cdot t)}{2 V_0^2} + \frac{I_s c_1 c_2 \cos(LO \cdot t - RF \cdot t)}{2 V_0^2} + \frac{I_s c_1 c_2^2 \cos(LO \cdot t - 2 RF \cdot t)}{8 V_0^3} \\ + \frac{I_s c_1 c_2^2 \cos(LO \cdot t + 2 RF \cdot t)}{8 V_0^3} + \frac{I_s c_1^2 c_2 \cos(2 LO \cdot t + RF \cdot t)}{8 V_0^3} + \frac{I_s c_1^2 c_2 \cos(2 LO \cdot t - RF \cdot t)}{8 V_0^3}$$

### Extract and Display Coefficients

Get the non-constant i.e. non-DC harmonic frequency terms of the form  $\cos(freq \cdot t)$ .

```
cosFreqs = cos(expand(freqs*t));  
terms = collect(setdiff(cosFreqs', sym(1)));
```

Extract the coefficients for all harmonic frequency terms including DC.

```
newvars = sym('x', [1,numel(terms)]);  
[cx, newvarsx] = coeffs(subs(f,terms,newvars), newvars);  
tx = sym(zeros(1,numel(cx)));  
for k=1:numel(newvarsx)  
    if newvarsx(k) ~= 1  
        tx(k) = terms(newvars == newvarsx(k));
```

```

else
    tx(k) = newvarsx(k);
end
end
cx = simplify(cx);

```

Display the coefficients using table, T. Use cosFreqs as a row identifier.

```

cosFreqs = arrayfun(@char,cosFreqs,'UniformOutput',false);
Frequencies = arrayfun(@char,freqs,'UniformOutput',false);
Coefficients = num2cell(zeros(size(freqs)));
T = table(Frequencies,Coefficients,'RowNames',cosFreqs);

```

Assign cx to the appropriate rows of T corresponding to the cosine terms of tx.

```

nonzeroCosFreqs = arrayfun(@char,tx,'UniformOutput',false).';
T(nonzeroCosFreqs,'Coefficients') = arrayfun(@char,cx,'UniformOutput',false).';

```

Now, remove row names as they are redundant.

```
T.Properties.RowNames = {};
```

Observe that the expressions for the terms are symmetric in LO and RF.

T

```

T=25x2 table
      Frequencies      Coefficients
      _____      _____
{'0'      } {'(Is*(c1^2 + c2^2))/(4*Vo^2)' }
{'RF'     } {'(Is*c2*(8*Vo^2 + 2*c1^2 + c2^2))/(8*Vo^3)' }
{'2*RF'   } {'(Is*c2^2)/(4*Vo^2)' }
{'3*RF'   } {'(Is*c2^3)/(24*Vo^3)' }
{'LO - 3*RF' } {[ 0] }
{'LO - 2*RF' } {'(Is*c1*c2^2)/(8*Vo^3)' }
{'LO - RF'  } {'(Is*c1*c2)/(2*Vo^2)' }
{'LO'      } {'(Is*c1*(8*Vo^2 + c1^2 + 2*c2^2))/(8*Vo^3)' }
{'LO + RF'  } {'(Is*c1*c2)/(2*Vo^2)' }
{'LO + 2*RF' } {'(Is*c1*c2^2)/(8*Vo^3)' }
{'LO + 3*RF' } {[ 0] }
{'2*LO - 3*RF' } {[ 0] }
{'2*LO - 2*RF' } {[ 0] }
{'2*LO - RF'  } {'(Is*c1^2*c2)/(8*Vo^3)' }
{'2*LO'     } {'(Is*c1^2)/(4*Vo^2)' }
{'2*LO + RF'  } {'(Is*c1^2*c2)/(8*Vo^3)' }
⋮

```

### Verify Coefficients

As shown below, the output waveform is reconstructed from the coefficients and has an exact match with the output.

```
simplify(f0 - (dot(tx,cx)))
```

```
ans = 0
```

### Plot Nonlinear Transfer

The following shows the particular nonlinear transfer function analyzed above, in the time and frequency domains, for certain values of the frequencies and voltage ratios. First, extract the data.

```
sample_values = struct('c1',0.4,'c2',1,'L0',800,'RF',13600,'Vo',1,'Is',1);
sample_input = subs(input,sample_values)

sample_input =

$$\frac{2 \cos(800 t)}{5} + \cos(13600 t)$$


sample_output = subs(f,sample_values)

sample_output =

$$\frac{127 \cos(800 t)}{250} + \frac{\cos(1600 t)}{25} + \frac{\cos(2400 t)}{375} + \frac{\cos(12000 t)}{50} + \frac{\cos(12800 t)}{5} + \frac{233 \cos(13600 t)}{200}$$

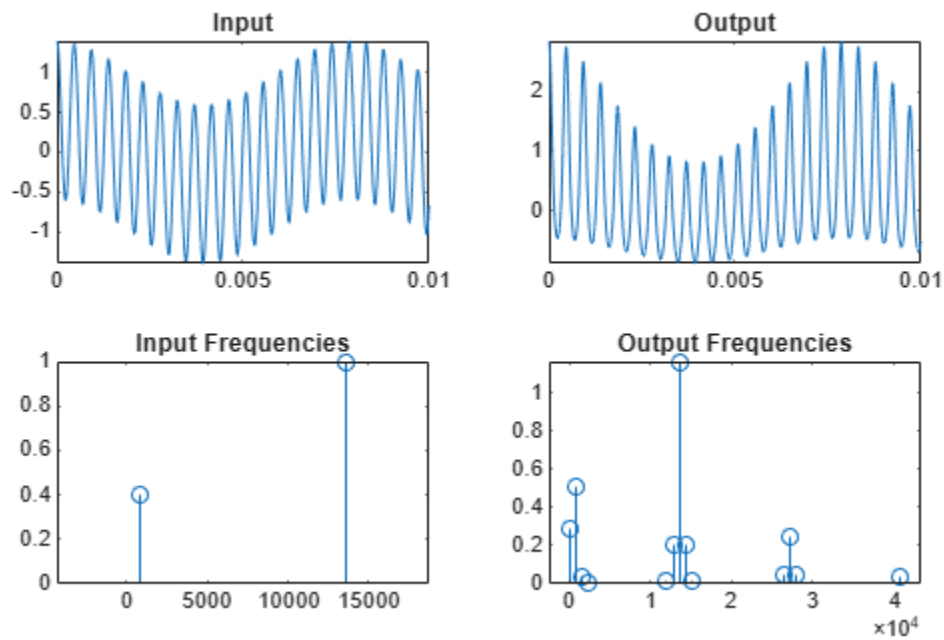

$$+ \frac{\cos(14400 t)}{5} + \frac{\cos(15200 t)}{50} + \frac{\cos(26400 t)}{20} + \frac{\cos(27200 t)}{4} + \frac{\cos(28000 t)}{20} + \frac{\cos(40800 t)}{24} + \frac{29}{100}$$


sample_freqs = zeros(size(tx));
for k=1:numel(tx)
    cosTerm = subs(tx(k),sample_values);
    freq = simplify(acos(cosTerm),'IgnoreAnalyticConstraints',true)/t;
    sample_freqs(k) = double(freq);
end
sample_heights = double(subs(cx,sample_values));
```

Then, use fplot and stem to plot the functions and their harmonic frequencies.

```
subplot(2,2,1);
fplot(sample_input,[0,0.01])
title 'Input'
subplot(2,2,3);
stem([sample_values.L0, sample_values.RF],[sample_values.c1,sample_values.c2]);
title 'Input Frequencies'

subplot(2,2,2);
fplot(sample_output,[0,0.01])
title 'Output'
subplot(2,2,4);
stem(sample_freqs,sample_heights)
title 'Output Frequencies'
```



## See Also

### Related Examples

- “Analyze Transfer Function of T-Coil Circuit” on page 3-324
- “Solve Differential Equations of RLC Circuit Using Laplace Transform” on page 3-198
- “Padé Approximant of Time-Delay Input” on page 3-261
- “Estimate Model Parameters of a Symbolically Derived Plant Model in Simulink” on page 5-74
- “Customize and Extend Simscape Libraries for a Custom DC Motor” on page 5-65

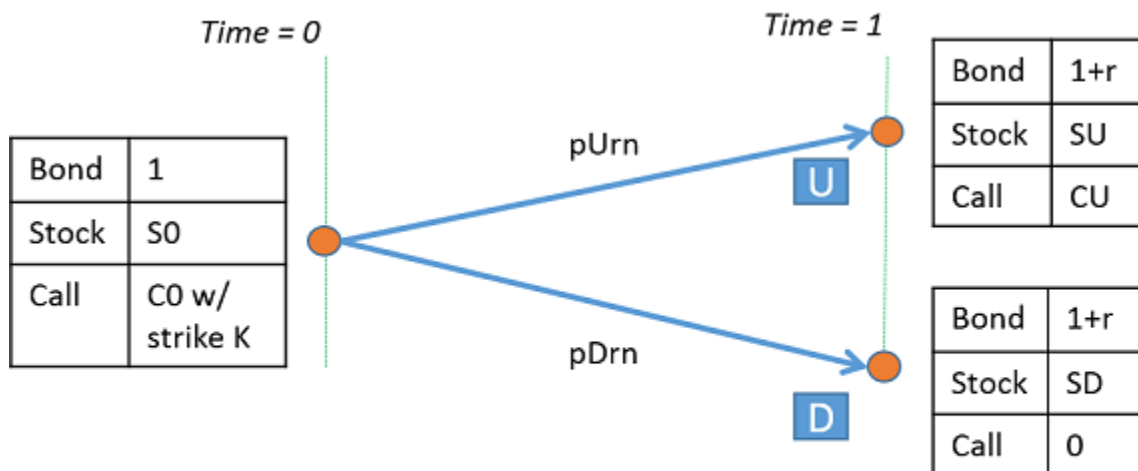
## Explore Single-Period Asset Arbitrage

This example explores basic arbitrage concepts in a single-period, two-state asset portfolio. The portfolio consists of a bond, a long stock, and a long call option on the stock.

It uses these Symbolic Math Toolbox™ functions:

- `equationsToMatrix` to convert a linear system of equations to a matrix.
- `linsolve` to solve the system.
- Symbolic equivalents of standard MATLAB® functions, such as `diag`.

This example symbolically derives the risk-neutral probabilities and call price for a single-period, two-state scenario.



### Two-state, Single-Period Prices for Bond-Stock-Call portfolio

#### Define Parameters of the Portfolio

Create the symbolic variable  $r$  representing the risk-free rate over the period. Set the assumption that  $r$  is a positive value.

```
syms r positive
```

Define the parameters for the beginning of a single period,  $\text{time} = 0$ . Here  $S_0$  is the stock price, and  $C_0$  is the call option price with strike,  $K$ .

```
syms S0 C0 K positive
```

Now, define the parameters for the end of a period,  $\text{time} = 1$ . Label the two possible states at the end of the period as  $U$  (the stock price over this period goes up) and  $D$  (the stock price over this period goes down). Thus,  $S_U$  and  $S_D$  are the stock prices at states  $U$  and  $D$ , and  $C_U$  is the value of the call at state  $U$ . Note that  $S_D < K \leq S_U$ .

```
syms S_U S_D C_U positive
```

The bond price at time = 0 is 1. Note that this example ignores friction costs.

Collect the prices at time = 0 into a column vector.

```
prices = [1 S0 C0]'
```

```
prices =
```

$$\begin{pmatrix} 1 \\ S_0 \\ C_0 \end{pmatrix}$$

Collect the payoffs of the portfolio at time = 1 into the payoff matrix. The columns of payoff correspond to payoffs for states D and U. The rows correspond to payoffs for bond, stock, and call. The payoff for the bond is  $1 + r$ . The payoff for the call in state D is zero since it is not exercised (because  $SD < K$ ).

```
payoff = [(1 + r), (1 + r); SD, SU; 0, CU]
```

```
payoff =
```

$$\begin{pmatrix} r+1 & r+1 \\ SD & SU \\ 0 & CU \end{pmatrix}$$

CU is worth  $SU - K$  in state U. Substitute this value in payoff.

```
payoff = subs(payoff, CU, SU - K)
```

```
payoff =
```

$$\begin{pmatrix} r+1 & r+1 \\ SD & SU \\ 0 & SU - K \end{pmatrix}$$

### Solve for Risk-Neutral Probabilities

Define the probabilities of reaching states U and D.

```
syms pU pD real
```

Under no-arbitrage, eqns == 0 must always hold true with positive pU and pD.

```
eqns = payoff*[pD; pU] - prices
```

```
eqns =
```

$$\begin{pmatrix} pD(r+1) + pU(r+1) - 1 \\ SD pD - S_0 + SU pU \\ -C_0 - pU(K - SU) \end{pmatrix}$$

Transform equations to use risk-neutral probabilities.

```
syms pDrn pUrn real;
```

```
eqns = subs(eqns, [pD; pU], [pDrn; pUrn]/(1 + r))
```

```
eqns =
```

$$\begin{pmatrix} pDrn + pUrn - 1 \\ \frac{SD pDrn}{r+1} - S_0 + \frac{SU pUrn}{r+1} \\ -C_0 - \frac{pUrn(K - SU)}{r+1} \end{pmatrix}$$

The unknown variables are  $p_{Dn}$ ,  $p_{Un}$ , and  $C_0$ . Transform the linear system to a matrix form using these unknown variables.

```
[A, b] = equationsToMatrix(eqns, [pDn, pUn, C0]')
```

$$A = \begin{pmatrix} 1 & 1 & 0 \\ \frac{SD}{r+1} & \frac{SU}{r+1} & 0 \\ 0 & -\frac{K-SU}{r+1} & -1 \end{pmatrix}$$

$$b = \begin{pmatrix} 1 \\ S_0 \\ 0 \end{pmatrix}$$

Using `linsolve`, find the solution for the risk-neutral probabilities and call price.

```
x = linsolve(A, b)
```

$$x = \begin{pmatrix} \frac{S_0 - SU + S_0 r}{SD - SU} \\ -\frac{S_0 - SD + S_0 r}{SD - SU} \\ \frac{(K - SU)(S_0 - SD + S_0 r)}{(SD - SU)(r + 1)} \end{pmatrix}$$

### Verify the Solution

Verify that under risk-neutral probabilities,  $x(1:2)$ , the expected rate of return for the portfolio,  $E\_return$  equals the risk-free rate,  $r$ .

```
E_return = diag(prices)\(payoff - [prices,prices])*x(1:2);  
E_return = simplify(subs(E_return, C0, x(3)))
```

$$E\_return = \begin{pmatrix} r \\ r \\ r \end{pmatrix}$$

### Test for No-Arbitrage Violations

As an example of testing no-arbitrage violations, use the following values:  $r = 5\%$ ,  $S_0 = 100$ , and  $K = 100$ . For  $SU < 105$ , the no-arbitrage condition is violated because  $p_{Dn} = xSol(1)$  is negative ( $SU \geq SD$ ). Further, for any call price other than  $xSol(3)$ , there is arbitrage.

```
xSol = simplify(subs(x, [r,S0,K], [0.05,100,100]))
```

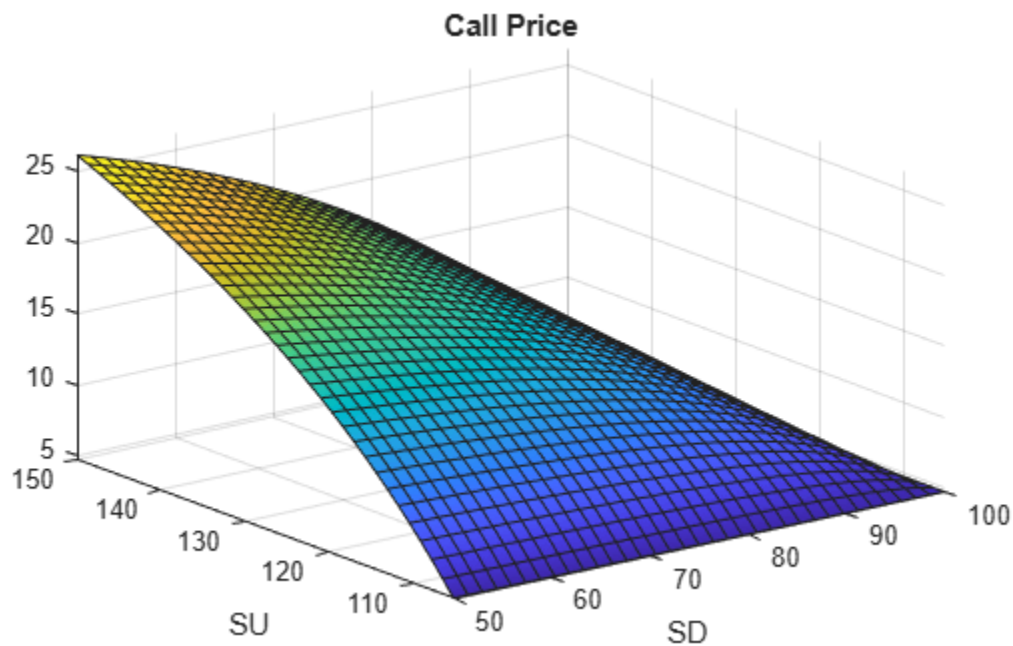
$$xSol = \begin{pmatrix} -\frac{SU - 105}{SD - SU} \\ \frac{SD - 105}{SD - SU} \\ \frac{20(SD - 105)(SU - 100)}{21(SD - SU)} \end{pmatrix}$$



### Plot Call Price as a Surface

Plot the call price,  $C_0 = xSol(3)$ , for  $50 \leq SD \leq 100$  and  $105 \leq SU \leq 150$ . Note that the call is worth more when the "variance" of the underlying stock price is higher for example,  $SD = 50$ ,  $SU = 150$ .

```
fsurf(xSol(3), [50,100,105,150])  
xlabel SD  
ylabel SU  
title 'Call Price'
```



### Reference

Advanced Derivatives, Pricing and Risk Management: Theory, Tools and Programming Applications by Albanese, C., Campolieti, G.

## Analytical Solutions of the Inverse Kinematics of a Humanoid Robot

This example shows how to derive analytical solutions for the inverse kinematics of the head chain of a humanoid robot.

### Step 1: Define Parameters

Describe the kinematics of the head-chain link (the link between the torso and the head) of a humanoid robot using the Denavit-Hartenberg (DH) parameters and notations based on a study by Kofinas et al. [1]. The following transformation defines the head-chain link.

$$T = A_{Base,0} T_{0,1} T_{1,2} R_x R_y A_{2,Head}$$

where:

- $A_{Base,0}$  is the translation from the base (torso) to the joint or reference frame 0
- $T_{0,1}$  is the orientation of reference 1 relative to 0
- $T_{1,2}$  is the orientation of reference 2 relative to 1
- $R_x$  is the roll rotation
- $R_y$  is the pitch rotation
- $A_{2,Head}$  is the translation from reference 2 to the end-effector point (the head)

$T(1:3,4)$  defines the coordinates of the head, which are xc, yc, zc.

In this example, you analytically solve the inverse kinematics problem by returning all orientations of individual joints in the head-chain link given head coordinates of xc, yc, and zc within the reachable space. Then, you convert the analytical results to purely numeric functions for efficiency.

Solving analytically (when doing so is possible) lets you perform computations in real time and avoid singularities, which cause difficulty for numerical methods.

### Step 2: Define Forward Kinematics of Head Chain Using DH Parameters

The function `getKinematicChain` returns the specific kinematic chain for the humanoid robot in terms of symbolic variables. For details on `getKinematicChain`, see the Helper Functions section.

```
kinChain = 'head';
dhInfo = getKinematicChain(kinChain);
T = dhInfo.T
```

$$T = \begin{pmatrix} r_{11} & r_{12} & r_{13} & xc \\ r_{21} & r_{22} & r_{23} & yc \\ r_{31} & r_{32} & r_{33} & zc \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Express the forward kinematics transformation matrix  $T$  as the following sequence of products:  $T = A_{Base0} * T_{01} * T_{12} * R_x * R_y * A_{2Head}$ . Define the individual matrices as follows.

Specify the translation matrix from the torso (base) to joint 0.

$$\text{ABase0} = \text{dhInfo.ABase0}$$

$$\text{ABase0} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \text{NeckOffsetZ} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Specify the transformation matrix from joint 0 to joint 1.

$$\text{T01} = \text{dhInfo.T01}$$

$$\text{T01} = \begin{pmatrix} \cos(\theta_1) & -\sin(\theta_1) & 0 & 0 \\ \sin(\theta_1) & \cos(\theta_1) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Specify the transformation matrix from joint 1 to joint 2.

$$\text{T12} = \text{dhInfo.T12}$$

$$\text{T12} = \begin{pmatrix} \cos\left(\theta_2 - \frac{\pi}{2}\right) & -\sin\left(\theta_2 - \frac{\pi}{2}\right) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -\sin\left(\theta_2 - \frac{\pi}{2}\right) & -\cos\left(\theta_2 - \frac{\pi}{2}\right) & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Specify the roll rotation matrix.

$$\text{Rx} = \text{dhInfo.Rx}$$

$$\text{Rx} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Specify the pitch rotation matrix.

$$\text{Ry} = \text{dhInfo.Ry}$$

$$\text{Ry} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Specify the translation matrix from joint 2 to the head.

$$\text{A2Head} = \text{dhInfo.A2Head}$$

$$A2Head = \begin{pmatrix} 1 & 0 & 0 & CameraX \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & CameraZ \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

The known parameters of this problem are CameraX, CameraZ, NeckOffsetZ, and the positions xc, yc, and zc. The unknown parameters are  $\theta_1$  and  $\theta_2$ . After finding  $\theta_1$  and  $\theta_2$ , you can compute the individual transformations of T. The robot can then achieve the desired position xc, yc, zc.

Although you can see these parameters in the transformation matrices, they do not exist as variables in the MATLAB base workspace. This is because these parameters originate from a function. Functions do not use the base workspace. Each function workspace is separate from the base workspace and all other workspaces to protect the data integrity. Thus, to use these variables outside of the function getKinematicChain, use syms to create them.

```
syms CameraX CameraZ NeckOffsetZ xc yc zc theta_1 theta_2 real;
```

### Step 3: Find Algebraic Solutions for $\theta_1$ and $\theta_2$

Rewrite the equation  $T = ABase0*T01*T12*Rx*Ry*A2Head$ , separating the terms that describe the torso and head movements of the robot:  $inv(T01)*inv(ABase0)*T = T12*Rx*Ry*A2Head$ . Simplify the left and right sides of the equation, and define equations of interest matching the expressions for coordinate positions.

```
LHS = simplify(inv(T01)*inv(ABase0)*T);
RHS = simplify(T12*Rx*Ry*A2Head);
eqns = LHS(1:3,end) - RHS(1:3,end)

eqns =

$$\begin{pmatrix} xc \cos(\theta_1) - CameraZ \sin(\theta_2) - CameraX \cos(\theta_2) + yc \sin(\theta_1) \\ yc \cos(\theta_1) - xc \sin(\theta_1) \\ zc - NeckOffsetZ - CameraZ \cos(\theta_2) + CameraX \sin(\theta_2) \end{pmatrix}$$

```

This system of equations contains two variables for which you want to find solutions,  $\theta_1$  and  $\theta_2$ . However, the equations also imply that you cannot arbitrarily choose xc, yc, and zc. Therefore, also consider yc as a variable. All other unknowns of the system are symbolic parameters.

This example follows a typical algebraic approach for solving inverse kinematics problems [2]. The idea is to get a compact representation of the solution, where the expression of each variable is in terms of parameters and variables for which you already solved the equations. As a first step, find either  $\theta_1$  or  $\theta_2$  in terms of the parameters. Then, express the other variable in terms of the known variable and parameters. To do this, start by identifying the equation that has only one variable.

```
intersect(symvar(eqns(1)), [theta_1, theta_2, yc])

ans = (theta_1 theta_2 yc)

intersect(symvar(eqns(2)), [theta_1, theta_2, yc])

ans = (theta_1 yc)

intersect(symvar(eqns(3)), [theta_1, theta_2, yc])

ans = theta_2
```

The third equation contains only one variable,  $\theta_2$ . Solve this equation first.

```
[theta_2_sol, theta_2_params, theta_2_conds] = ...
    solve(eqns(3), theta_2, 'Real', true, 'ReturnConditions', true)

theta_2_sol =

$$\begin{pmatrix} 2\pi k - 2 \operatorname{atan}\left(\frac{\text{CameraX} - \sqrt{\text{CameraX}^2 + \text{CameraZ}^2 - \text{NeckOffsetZ}^2 + 2 \text{NeckOffsetZ} \text{zc} - \text{zc}^2}}{\text{CameraZ} - \text{NeckOffsetZ} + \text{zc}}\right) \\ 2\pi k - 2 \operatorname{atan}\left(\frac{\text{CameraX} + \sqrt{\text{CameraX}^2 + \text{CameraZ}^2 - \text{NeckOffsetZ}^2 + 2 \text{NeckOffsetZ} \text{zc} - \text{zc}^2}}{\text{CameraZ} - \text{NeckOffsetZ} + \text{zc}}\right) \end{pmatrix}$$


theta_2_params = k

theta_2_conds =

$$\begin{pmatrix} k \in \mathbb{Z} \wedge \text{CameraZ} + \text{zc} \neq \text{NeckOffsetZ} \wedge (\text{NeckOffsetZ} - \text{zc})^2 \leq \text{CameraX}^2 + \text{CameraZ}^2 \\ k \in \mathbb{Z} \wedge \text{CameraZ} + \text{zc} \neq \text{NeckOffsetZ} \wedge (\text{NeckOffsetZ} - \text{zc})^2 \leq \text{CameraX}^2 + \text{CameraZ}^2 \end{pmatrix}$$

```

The solutions have an additive  $2\pi k$  term with the parameter  $k$ . Without loss of generalization, you can set  $k$  equal to 0 in the solutions and conditions.

```
theta_2_sol = subs(theta_2_sol, theta_2_params, 0)

theta_2_sol =

$$\begin{pmatrix} -2 \operatorname{atan}\left(\frac{\text{CameraX} - \sqrt{\text{CameraX}^2 + \text{CameraZ}^2 - \text{NeckOffsetZ}^2 + 2 \text{NeckOffsetZ} \text{zc} - \text{zc}^2}}{\text{CameraZ} - \text{NeckOffsetZ} + \text{zc}}\right) \\ -2 \operatorname{atan}\left(\frac{\text{CameraX} + \sqrt{\text{CameraX}^2 + \text{CameraZ}^2 - \text{NeckOffsetZ}^2 + 2 \text{NeckOffsetZ} \text{zc} - \text{zc}^2}}{\text{CameraZ} - \text{NeckOffsetZ} + \text{zc}}\right) \end{pmatrix}$$


for p = 1:numel(theta_2_conds)
    theta_2_conds(p) = simplify(subs(theta_2_conds(p), theta_2_params, 0));
end
```

Now solve for the variables  $\theta_1$  and  $yc$  in terms of the variable  $\theta_2$  and other symbolic parameters.

```
[theta_1_sol, yc_sol, yc_theta_1_params, yc_theta_1_conds] = ...
    solve(eqns(1:2), theta_1, yc, 'Real', true, 'ReturnConditions', true);
```

Display the solutions for  $\theta_1$  and  $yc$ .

```
theta_1_sol
yc_sol
```

$$\begin{pmatrix} 2 \pi k - \sigma_1 \\ \sigma_2 + 2 \pi k \\ 2 \pi k - \sigma_2 \\ 2 \pi k - \frac{\pi}{2} \\ \sigma_1 + 2 \pi k \\ 2 \operatorname{atan}(x) + 2 \pi k \\ \frac{\pi}{2} + 2 \pi k \\ 2 \pi k - \frac{\pi}{2} \\ \frac{\pi}{2} + 2 \pi k \end{pmatrix}$$

where

$$\sigma_1 = 2 \operatorname{atan}\left(\frac{\sqrt{(\operatorname{CameraX} + xc)(\operatorname{CameraX} - xc)}}{\operatorname{CameraX} - xc}\right)$$

$$\sigma_2 = 2 \operatorname{atan}\left(\frac{\frac{\sigma_3}{\sigma_5 + 1} + \frac{\sigma_5 \sigma_3}{\sigma_5 + 1}}{\sigma_4}\right)$$

$$\sigma_3 = \sqrt{-\left(xc - \operatorname{CameraX} + \operatorname{CameraX} \sigma_5 + xc \sigma_5 - 2 \operatorname{CameraZ} \tan\left(\frac{\theta_2}{2}\right)\right) \sigma_4}$$

$$\sigma_4 = \operatorname{CameraX} + xc - \operatorname{CameraX} \sigma_5 + xc \sigma_5 + 2 \operatorname{CameraZ} \tan\left(\frac{\theta_2}{2}\right)$$

$$\sigma_5 = \tan\left(\frac{\theta_2}{2}\right)^2$$

**yc\_sol**

**yc\_sol** =

$$\begin{pmatrix} \sqrt{(\text{CameraX} + \text{xc})(\text{CameraX} - \text{xc})} \\ \sigma_1 \\ -\sigma_1 \\ \text{CameraX} \\ -\sqrt{(\text{CameraX} + \text{xc})(\text{CameraX} - \text{xc})} \\ 0 \\ \sigma_2 \\ -\sigma_2 \\ -\text{CameraX} \end{pmatrix}$$

where

$$\sigma_1 = \frac{\sqrt{-\left(\text{xc} - \text{CameraX} + \sigma_3 + \text{xc} \tan\left(\frac{\theta_2}{2}\right)^2 - \sigma_4\right)\left(\text{CameraX} + \text{xc} - \sigma_3 + \text{xc} \tan\left(\frac{\theta_2}{2}\right)^2 + \sigma_4\right)}}{\tan\left(\frac{\theta_2}{2}\right)^2 + 1}$$

$$\sigma_2 = \frac{-\sigma_3 + \sigma_4 + \text{CameraX}}{\tan\left(\frac{\theta_2}{2}\right)^2 + 1}$$

$$\sigma_3 = \text{CameraX} \tan\left(\frac{\theta_2}{2}\right)^2$$

$$\sigma_4 = 2 \text{ CameraZ} \tan\left(\frac{\theta_2}{2}\right)$$

The solutions depend on two parameters,  $k$  and  $x$ .

`yc_theta_1_params`

`yc_theta_1_params = (k x)`

The solutions have an additive  $2\pi k$  term. Without loss of generalization, you can set  $k$  equal to 0 in the solution for `theta_1_sol` and the conditions `yc_theta_1_conds`.

`theta_1_sol = simplify(subs(theta_1_sol,yc_theta_1_params,[0,0]))`

`theta_1_sol =`

$$\begin{pmatrix} -2 \operatorname{atan}\left(\frac{\sqrt{(\text{CameraX} + \text{xc})(\text{CameraX} - \text{xc})}}{\text{CameraX} - \text{xc}}\right) \\ \sigma_1 \\ -\sigma_1 \\ -\frac{\pi}{2} \\ 2 \operatorname{atan}\left(\frac{\sqrt{(\text{CameraX} + \text{xc})(\text{CameraX} - \text{xc})}}{\text{CameraX} - \text{xc}}\right) \\ 0 \\ \frac{\pi}{2} \\ -\frac{\pi}{2} \\ \frac{\pi}{2} \end{pmatrix}$$

where

$$\sigma_1 = 2 \operatorname{atan}\left(\frac{\sqrt{(\text{CameraX} \cos(\theta_2) - \text{xc} + \text{CameraZ} \sin(\theta_2))(\text{xc} + \text{CameraX} \cos(\theta_2) + \text{CameraZ} \sin(\theta_2))}}{\text{xc} + \text{CameraX} \cos(\theta_2) + \text{CameraZ} \sin(\theta_2)}\right)$$

```
for p = 1:numel(yc_theta_1_conds)
    yc_theta_1_conds(p) = simplify(subs(yc_theta_1_conds(p),yc_theta_1_params,[0,0]));
end
```

A similar substitution is not required for `yc_sol` since there is no dependency on the parameters.

```
intersect(symvar(yc_sol),yc_theta_1_params)
```

```
ans =
```

```
Empty sym: 1-by-0
```

#### Step 4: Verify the Solutions

Starting with an arbitrary set of known numeric values for  $\theta_1$  and  $\theta_2$ , compute the numeric values of the end-effector positions `xc`, `yc`, and `zc` with forward kinematics. Then, work backward from `xc`, `yc`, and `zc` to compute all possible numeric values for  $\theta_1$  and  $\theta_2$  using the inverse kinematics expressions from the previous computation. Verify that one set of the inverse solutions matches the starting set of numeric values for  $\theta_1$  and  $\theta_2$ .

Set the fixed parameters for the robot. The values in this example are for illustration purposes only.

```
CameraXValue = 53.9;
CameraZValue = 67.9;
NeckOffsetZValue = -5;
```

Using forward computations, find the end-effector positions corresponding to the values  $\theta_1$  and  $\theta_2$ .

```
Tfk = ABase0*T01*T12*Rx*Ry*A2Head;
xyz = Tfk(1:3,end);
```

Create a symbolic function for these positions.



```
xyzFunc = symfun(subs(xyz,[CameraX,CameraZ,NeckOffsetZ], ...
    [CameraXValue,CameraZValue,NeckOffsetZValue]),[theta_1,theta_2]);
```

For forward kinematics, specify two variables `theta_1_known` and `theta_2_known` that contain an arbitrary starting set of values for  $\theta_1$  and  $\theta_2$ .

```
theta_1_known = [pi/4,pi/6,pi/8,pi/10];
theta_2_known = [pi/8,-pi/12,pi/16,-pi/10];
numPts = numel(theta_1_known);
num_theta_2 = numel(theta_2_sol);
num_theta_1 = numel(theta_1_sol);
```

Note that there are potentially `num_theta_1` solutions for each `num_theta_2` solution.

Use the following sequence of operations to verify the solutions.

- 1 Loop over (`theta_1_known`, `theta_2_known`). For each point, compute the end positions `x_known`, `y_known`, and `z_known`, which are then "known".
- 2 Loop over the solutions for  $\theta_2$  corresponding to `x_known` and `z_known`. For each solution, check to see if the corresponding condition `cond_theta_2` is valid. If the condition is valid, compute the corresponding numeric solution `theta_2_derived`.
- 3 Loop over the solutions for  $\theta_1$  corresponding to `x_known`, `z_known`, and `theta_2_derived`. For each solution, check to see if the corresponding condition `cond_theta_1` is valid. If the condition is valid, check to see if `y_derived` numerically matches `y_known` within relative and absolute tolerances through the condition `cond_yc`. If this condition is valid, then compute `theta_1_derived`.
- 4 Collect the results in a table `T` for display purposes.

```
T = table([],[],[],[], 'VariableNames', {'theta_1_known', 'theta_2_known', ...
    'theta_1_derived', 'theta_2_derived'});

for ix1 = 1:numPts
    xyz_known = double(xyzFunc(theta_1_known(ix1),theta_2_known(ix1)));
    x_known = xyz_known(1);
    y_known = xyz_known(2);
    z_known = xyz_known(3);
    for ix2 = 1:num_theta_2 % theta_2 loop
        cond_theta_2 = subs(theta_2_conds(ix2), [CameraX,CameraZ,NeckOffsetZ,xc,zc], ...
            [CameraXValue,CameraZValue,NeckOffsetZValue,x_known,z_known]);
        if isAlways(cond_theta_2) % if theta_2 is valid
            theta_2_derived = subs(theta_2_sol(ix2), [CameraX,CameraZ,NeckOffsetZ,xc,zc], ...
                [CameraXValue,CameraZValue,NeckOffsetZValue,x_known,z_known]);
            theta_2_derived = double(theta_2_derived);
            for ix3 = 1:num_theta_1 % theta_1 loop
                cond_theta_1 = subs(yc_theta_1_conds(ix3), [CameraX,CameraZ,NeckOffsetZ,theta_2,xc,zc], ...
                    [CameraXValue,CameraZValue,NeckOffsetZValue,theta_2_derived,x_known,z_known]);
                if isAlways(cond_theta_1) % if theta_1 is valid
                    y_derived = subs(yc_sol(ix3), [CameraX,CameraZ,NeckOffsetZ,theta_2,xc,zc], ...
                        [CameraXValue,CameraZValue,NeckOffsetZValue,theta_2_derived,x_known,z_known]);
                    y_derived = double(y_derived);
                    cond_yc = abs(y_known - y_derived) < max(100*eps, 1e-6*abs(y_known)); % check rounding
                    if isAlways(cond_yc) % if yc is valid
                        theta_1_derived = subs(theta_1_sol(ix3), [CameraX,CameraZ,NeckOffsetZ,theta_2,xc,zc], ...
                            [CameraXValue,CameraZValue,NeckOffsetZValue,theta_2_derived,x_known,z_known]);
                        theta_1_derived = double(theta_1_derived);
```

```

        T = vertcat(T,table(theta_1_known(ix1),theta_2_known(ix1),theta_1_derived,...
            theta_2_derived,'VariableNames',T.Properties.VariableNames));
    end
end
end
end
end
end
T

```

```

T=8x4 table
    theta_1_known    theta_2_known    theta_1_derived    theta_2_derived
    _____    _____    _____    _____
    0.7854          0.3927          0.7854          0.3927
    0.7854          0.3927         -2.3562         -1.7346
    0.5236         -0.2618          0.5236         -0.2618
    0.5236         -0.2618         -2.618         -1.0801
    0.3927          0.19635         0.3927          0.19635
    0.3927          0.19635        -2.7489         -1.5383
    0.31416        -0.31416         0.31416        -0.31416
    0.31416        -0.31416        -2.8274         -1.0278

```

Observe that there are two possible solution pairs ( $\theta_{1\_derived}, \theta_{2\_derived}$ ) obtained using inverse kinematics for each pair of starting angles ( $\theta_{1\_known}, \theta_{2\_known}$ ). One set of the inverse solutions matches the starting angles.

## References

- 1 Kofinas, N., E. Orfanoudakis, and M. G. Lagoudakis. "Complete Analytical Inverse Kinematics for NAO." In *2013 13th International Conference on Autonomous Robot Systems*. Lisbon, Portugal: Robotica, 2013.
- 2 Kendricks, K. "Solving the Inverse Kinematic Robotics Problem: A Comparison Study of the Denavit-Hartenberg Matrix and Groebner Basis Theory." Ph.D. Thesis. Auburn University, Auburn, AL, 2007.

## Helper Functions

```

function kinChain = getKinematicChain(link)
% This function returns the kinematic chain of a humanoid robot
% represented using Denavit-Hartenberg (DH) parameters.
% The function uses as a reference the paper: Complete analytical inverse
% kinematics for NAO by Kofinas, N., Orfanoudakis, E., Lagoudakis, M.G.,
% 2013 13th International Conference on Autonomous Robot Systems
% (Robotica), Publication Year: 2013.
    if nargin < 1
        link = 'head';
    end
    % Notation: A, R, and T are the translation, rotation, and DH
    % transformation matrices, respectively.
    % Specify DH parameters for the desired end configuration.
    syms r11 r12 r13 r21 r22 r23 r31 r32 r33 xc yc zc real;
    R = [r11 r12 r13;r21 r22 r23;r31 r32 r33];
    kinChain.T = [R,[xc;yc;zc];0,0,0,1];
    switch link
        case 'head' % Kinematic chain from torso (base) to head.

```

```

syms CameraX CameraZ NeckOffsetZ real;
% Translation from torso (base) to joint 0.
ABase0 = getA([0 0 NeckOffsetZ]);
% Translation from joint 2 to head.
A2Head = getA([CameraX 0 CameraZ]);
% Transformation from joint 0 to joint 1.
% theta_1 is the rotation angle corresponding to the 0-1 link.
syms theta_1 real;
alpha1 = 0; a1 = 0; d1 = 0;
T01 = getT(a1,alpha1,d1,theta_1);
% Transformation from joint 1 to joint 2.
% theta_2 is the rotation angle corresponding to the 1-2 link.
syms theta_2 real;
piy2 = str2sym('pi/2');
d2 = 0; a2 = 0; alpha2 = -piy2;
T12 = getT(a2,alpha2,d2,theta_2-piy2);
% Rx is the roll rotation.
Rx = getR('x',piy2);
% Ry is the pitch rotation.
Ry = getR('y',piy2);
% Capture the kinematic chain as a string.
% The transformation is from the base to the head.
kinChain.DHChain = 'ABase0*T01*T12*Rx*Ry*A2Head';
kinChain.ABase0 = ABase0;
kinChain.T01 = T01;
kinChain.T12 = T12;
kinChain.Rx = Rx;
kinChain.Ry = Ry;
kinChain.A2Head = A2Head;
case 'lefthand' % Kinematic chain from torso to left hand.
syms ShoulderOffsetY ElbowOffsetY ShoulderOffsetZ real;
ABase0 = getA([0 (ShoulderOffsetY+ElbowOffsetY) ShoulderOffsetZ]);
syms HandOffsetX LowerArmLength real;
AEnd4 = getA([(HandOffsetX+LowerArmLength) 0 0]);
piy2 = str2sym('pi/2');
syms theta_1 real;
alpha1 = -piy2; a1 = 0; d1 = 0;
T01 = getT(a1,alpha1,d1,theta_1);
syms theta_2 real;
d2 = 0; a2 = 0; alpha2 = piy2;
T12 = getT(a2,alpha2,d2,theta_2-piy2);
syms UpperArmLength theta_3 real;
d3 = UpperArmLength; a3 = 0; alpha3 = -piy2;
T32 = getT(a3,alpha3,d3,theta_3);
syms theta_4 real;
d4 = 0; a4 = 0; alpha4 = piy2;
T43 = getT(a4,alpha4,d4,theta_4);
Rz = getR('z',piy2);
kinChain.DHChain = 'ABase0*T01*T12*T32*T43*Rz*AEnd4';
kinChain.ABase0 = ABase0;
kinChain.T01 = T01;
kinChain.T12 = T12;
kinChain.T32 = T32;
kinChain.T43 = T43;
kinChain.Rz = Rz;
kinChain.AEnd4 = AEnd4;
end
end

```

```
function A = getA(vec)
% This function returns the translation matrix.
    A = [1 0 0 vec(1);
         0 1 0 vec(2);
         0 0 1 vec(3);
         0 0 0 1];
end

function R = getR(orientation,theta)
% This function returns the rotation matrix.
    switch orientation
    case 'x'
        R = [1 0 0 0;
              0 cos(theta) -sin(theta) 0;
              0 sin(theta) cos(theta) 0;
              0 0 0 1];
    case 'y'
        R = [cos(theta) 0 sin(theta) 0;
              0 1 0 0;
              -sin(theta) 0 cos(theta) 0;
              0 0 0 1];
    case 'z'
        R = [cos(theta) -sin(theta) 0 0;
              sin(theta) cos(theta) 0 0;
              0 0 1 0;
              0 0 0 1];
    end
end

function T = getT(a,alpha,d,theta)
% This function returns the Denavit-Hartenberg (DH) matrix.
    T = [cos(theta), -sin(theta), 0, a;
          sin(theta)*cos(alpha), cos(theta)*cos(alpha), -sin(alpha), -sin(alpha)*d;
          sin(theta)*sin(alpha), cos(theta)*sin(alpha), cos(alpha), cos(alpha)*d;
          0, 0, 0, 1];
end
```

## See Also

### Functions

[simplify](#) | [solve](#) | [subs](#) | [double](#)

## Differentiation

This examples shows how to find derivatives using Symbolic Math Toolbox™.

First create a symbolic expression.

```
syms x
f = sin(5*x);
```

Differentiate the expression `f` with respect to `x`.

```
Df = diff(f)
Df = 5 cos(5 x)
```

As another example, specify another expression that uses `exp(x)` to represent  $e^x$ .

```
g = exp(x)*cos(x);
```

Differentiate the expression `g`.

```
y = diff(g)
y = e^x cos(x) - e^x sin(x)
```

To find the derivative of `g` for a given value of `x`, substitute `x` for the value using `subs` and return a numerical value using `vpa`. Find the derivative of `g` at `x = 2`.

```
y_eval = vpa(subs(y,x,2))
y_eval = -9.7937820180676088383807818261614
```

To take the second derivative of `g`, use `diff(g,2)`.

```
D2g = diff(g,2)
D2g = -2 e^x sin(x)
```

You can get the same result by taking the derivative twice.

```
D2g = diff(diff(g))
D2g = -2 e^x sin(x)
```

In this example, Symbolic Math Toolbox returns a simplified answer. However, in some cases, the answer is not simplified, in which case you can use the `simplify` command. The next section of this example will discuss the cases of such simplification.

Note that to take the derivative of a constant, you must first define the constant as a symbolic expression.

```
c = sym("5");
Dc = diff(c)
Dc = 0
```

If you use `diff` directly on a constant number, such as 5, the result is an empty array because the number is not a symbolic object but a `double` numeric type.

```
Dc = diff(5)
```

```
Dc =
```

```
[]
```

### Derivatives of Expressions with Several Variables

To differentiate an expression that contains more than one symbolic variable, specify the variable that you want to differentiate with respect to. The `diff` command then calculates the partial derivative of the expression with respect to that variable. For example, specify a symbolic expression with two variables.

```
syms s t
f = sin(s*t);
```

Find the partial derivative  $\partial f / \partial t$  by using `diff` and specifying the variable to differentiate as `t`.

```
Df_t = diff(f,t)
```

```
Df_t = s*cos(s*t)
```

To differentiate `f` with respect to the variable `s`, specify the variable to differentiate as `s`.

```
Df_s = diff(f,s)
```

```
Df_s = t*cos(s*t)
```

If you do not specify a variable to differentiate with respect to, `diff` uses the default variable. Basically, the default variable is the letter closest to `x` in the alphabet. See the complete set of rules in “Find Symbolic Variables in Expressions, Functions, and Matrices” on page 2-2. In the preceding example, `diff(f)` takes the derivative of `f` with respect to `t` because the letter `t` is closer to `x` in the alphabet than the letter `s` is. To determine the default variable that MATLAB® differentiates with respect to, use `symvar`.

```
fvar = symvar(f,1)
```

```
fvar = t
```

Calculate the second derivative of `f` with respect to `t`.

```
D2f = diff(f,t,2)
```

```
D2f = -s^2*sin(s*t)
```

Note that `diff(f,2)` returns the same answer because `t` is the default variable.

### More Examples About Derivatives

To further illustrate the `diff` function on other expressions, define the symbolic variables `a`, `b`, `x`, `n`, `t`, and `theta`.

```
syms a b x n t theta
```

This table illustrates the results of using `diff` on several other expressions.

| <b>f</b>                                                  | <b>diff(f)</b>                                         |
|-----------------------------------------------------------|--------------------------------------------------------|
| <pre>&gt;&gt; syms x n &gt;&gt; f = x^n;</pre>            | <pre>&gt;&gt; Df = diff(f) Df = n*x^(n - 1)</pre>      |
| <pre>&gt;&gt; syms a b t &gt;&gt; f = sin(a*t + b);</pre> | <pre>&gt;&gt; Df = diff(f) Df = a*cos(b + a*t)</pre>   |
| <pre>&gt;&gt; syms theta &gt;&gt; f = exp(i*theta);</pre> | <pre>&gt;&gt; Df = diff(f) Df = exp(theta*1i)*1i</pre> |

You can differentiate the Bessel function of the first kind, `besselj(nu, z)`, with respect to `z`.

```
syms nu z
b = besselj(nu, z);
Db = diff(b)
```

Db =

$$\frac{\nu J_{\nu}(z)}{z} - J_{\nu+1}(z)$$

The `diff` function can also take a symbolic matrix as its input. In this case, the differentiation is done element-by-element.

```
syms a x
A = [cos(a*x), sin(a*x); -sin(a*x), cos(a*x)]
```

A =

$$\begin{pmatrix} \cos(ax) & \sin(ax) \\ -\sin(ax) & \cos(ax) \end{pmatrix}$$

Find the derivative of `A` with respect to `x`.

```
DA = diff(A)
```

DA =

$$\begin{pmatrix} -a \sin(ax) & a \cos(ax) \\ -a \cos(ax) & -a \sin(ax) \end{pmatrix}$$

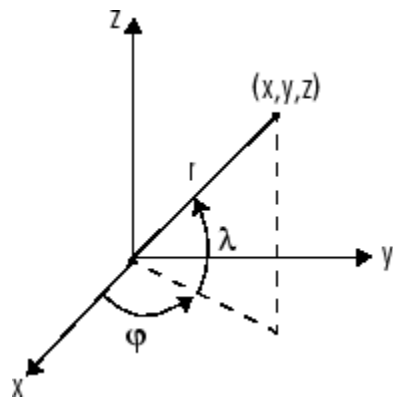
You can also perform differentiation of a vector function with respect to a vector argument. Consider the transformation from Cartesian coordinates  $(x, y, z)$  to spherical coordinates  $(r, \lambda, \varphi)$  as given by

$$x = r \cos \lambda \cos \varphi,$$

$$y = r \cos \lambda \sin \varphi,$$

$$z = r \sin \lambda.$$

Here,  $\lambda$  corresponds to elevation or latitude, while  $\varphi$  denotes azimuth or longitude.



To calculate the Jacobian matrix,  $J$ , of this transformation, use the `jacobian` function. The mathematical notation for  $J$  is

$$J = \frac{\partial(x, y, z)}{\partial(r, \lambda, \varphi)}.$$

For the purposes of computing the Jacobian, use `l` to represent  $\lambda$  and `f` to represent  $\varphi$ . Find the Jacobian.

```
syms r l f
x = r*cos(l)*cos(f);
y = r*cos(l)*sin(f);
z = r*sin(l);
J = jacobian([x; y; z], [r l f])

J =
    (cos(f) cos(l) - r cos(f) sin(l) - r cos(l) sin(f))
    (cos(l) sin(f) - r sin(f) sin(l)  r cos(f) cos(l))
    (sin(l)      r cos(l)      0)
```

Find the determinant of this Jacobian and simplify the result.

```
detJ = simplify(det(J))

detJ = -r^2 cos(l)
```

The arguments of the `jacobian` function can be column or row vectors. Moreover, since the determinant of the Jacobian is a rather complicated trigonometric expression, you can use `simplify` to make trigonometric substitutions and reductions (simplifications).

A table summarizing `diff` and `jacobian` follows.

| Mathematical Operator | Command Using Symbolic Math Toolbox            |
|-----------------------|------------------------------------------------|
| $\frac{df}{dx}$       | <code>diff(f)</code> or <code>diff(f,x)</code> |
| $\frac{df}{da}$       | <code>diff(f,a)</code>                         |
| $\frac{d^2f}{db^2}$   | <code>diff(f,b,2)</code>                       |



| Mathematical Operator                       | Command Using Symbolic Math Toolbox      |
|---------------------------------------------|------------------------------------------|
| $J = \frac{\partial(r, t)}{\partial(u, v)}$ | <code>J = jacobian([r; t],[u; v])</code> |

**See Also**

`diff` | `int` | `jacobian` | `gradient` | `curl` | `laplacian` | `functionalDerivative`

**Teaching Resources**

- Calculus Derivatives (MathWorks Teaching Resources)

## Integration

If  $f$  is a symbolic expression, then

`int(f)`

attempts to find another symbolic expression,  $F$ , so that  $\text{diff}(F) = f$ . That is, `int(f)` returns the indefinite integral or antiderivative of  $f$  (provided one exists in closed form). Similar to differentiation,

`int(f,v)`

uses the symbolic object  $v$  as the variable of integration, rather than the variable determined by `symvar`. See how `int` works by looking at this table.

| Mathematical Operation                                                                                           | MATLAB Command                                                                |
|------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| $\int x^n dx = \begin{cases} \log(x) & \text{if } n = -1 \\ \frac{x^{n+1}}{n+1} & \text{otherwise.} \end{cases}$ | <code>int(x^n)</code> or <code>int(x^n,x)</code>                              |
| $\int_0^{\pi/2} \sin(2x) dx = 1$                                                                                 | <code>int(sin(2*x), 0, pi/2)</code> or <code>int(sin(2*x), x, 0, pi/2)</code> |
| $g = \cos(at + b)$<br>$\int g(t) dt = \sin(at + b)/a$                                                            | <code>g = cos(a*t + b)</code> <code>int(g)</code> or <code>int(g, t)</code>   |
| $\int J_1(z) dz = -J_0(z)$                                                                                       | <code>int(besselj(1, z))</code> or <code>int(besselj(1, z), z)</code>         |

In contrast to differentiation, symbolic integration is a more complicated task. A number of difficulties can arise in computing the integral:

- The antiderivative,  $F$ , may not exist in closed form.
- The antiderivative may define an unfamiliar function.
- The antiderivative may exist, but the software can't find it.
- The software could find the antiderivative on a larger computer, but runs out of time or memory on the available machine.

Nevertheless, in many cases, MATLAB can perform symbolic integration successfully. For example, create the symbolic variables

`syms a b theta x y n u z`

The following table illustrates integration of expressions containing those variables.

| $f$                                            | <code>int(f)</code>                                                                                                                     |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <code>syms x n</code><br><code>f = x^n;</code> | <code>int(f)</code><br><br><code>ans =</code><br><code>piecewise(n == -1, log(x), n ~= -1,...</code><br><code>x^(n + 1)/(n + 1))</code> |

| <b>f</b>                              | <b>int(f)</b>                                          |
|---------------------------------------|--------------------------------------------------------|
| syms y<br>f = y <sup>(-1)</sup> ;     | int(f)<br><br>ans =<br>log(y)                          |
| syms x n<br>f = n <sup>x</sup> ;      | int(f)<br><br>ans =<br>n <sup>x</sup> /log(n)          |
| syms a b theta<br>f = sin(a*theta+b); | int(f)<br><br>ans =<br>-cos(b + a*theta)/a             |
| syms u<br>f = 1/(1+u <sup>2</sup> );  | int(f)<br><br>ans =<br>atan(u)                         |
| syms x<br>f = exp(-x <sup>2</sup> );  | int(f)<br><br>ans =<br>(pi <sup>(1/2)</sup> *erf(x))/2 |

In the last example,  $\exp(-x^2)$ , there is no formula for the integral involving standard calculus expressions, such as trigonometric and exponential functions. In this case, MATLAB returns an answer in terms of the error function `erf`.

If MATLAB is unable to find an answer to the integral of a function `f`, it just returns `int(f)`.

Definite integration is also possible.

| <b>Definite Integral</b> | <b>Command</b>  |
|--------------------------|-----------------|
| $\int_a^b f(x)dx$        | int(f, a, b)    |
| $\int_a^b f(v)dv$        | int(f, v, a, b) |

Here are some additional examples.

| <b>f</b>                       | <b>a, b</b>      | <b>int(f, a, b)</b>                 |
|--------------------------------|------------------|-------------------------------------|
| syms x<br>f = x <sup>7</sup> ; | a = 0;<br>b = 1; | int(f, a, b)<br><br>ans =<br>1/8    |
| syms x<br>f = 1/x;             | a = 1;<br>b = 2; | int(f, a, b)<br><br>ans =<br>log(2) |
| syms x<br>f = log(x)*sqrt(x);  | a = 0;<br>b = 1; | int(f, a, b)<br><br>ans =<br>-4/9   |

| f                             | a, b               | int(f, a, b)                                                                 |
|-------------------------------|--------------------|------------------------------------------------------------------------------|
| syms x<br>f = exp(-x^2);      | a = 0;<br>b = inf; | int(f, a, b)<br><br>ans =<br>pi^(1/2)/2                                      |
| syms z<br>f = besselj(1,z)^2; | a = 0;<br>b = 1;   | int(f, a, b)<br><br>ans =<br>hypergeom([3/2, 3/2],...<br>[2, 5/2, 3], -1)/12 |

For the Bessel function (`besselj`) example, it is possible to compute a numerical approximation to the value of the integral, using the `double` function. The commands

```
syms z
a = int(besselj(1,z)^2,0,1)

return

a =
hypergeom([3/2, 3/2], [2, 5/2, 3], -1)/12

and the command

a = double(a)

returns

a =
0.0717
```

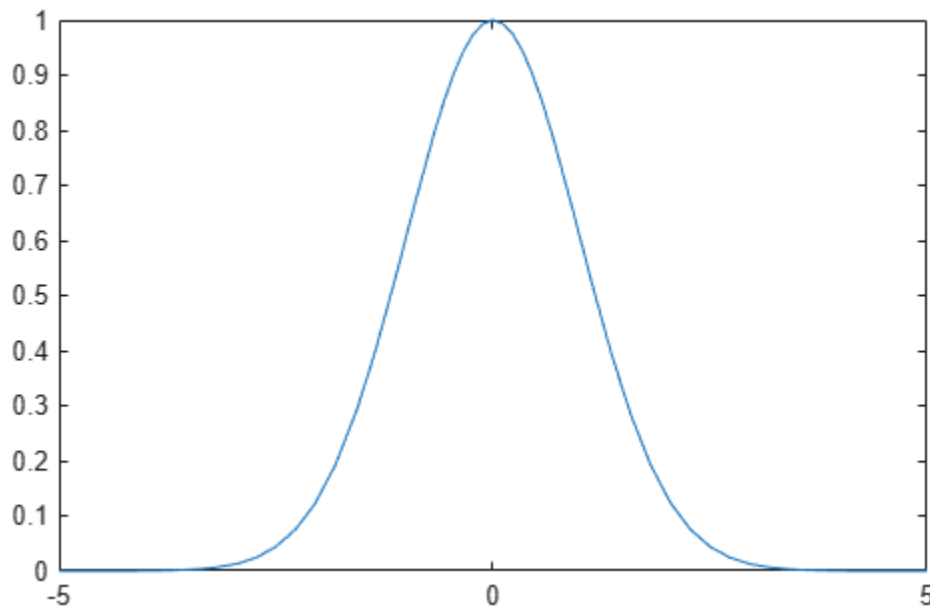
## Integration with Real Parameters

One of the subtleties involved in symbolic integration is the “value” of various parameters. For example, if  $a$  is any positive real number, the expression

$$e^{-ax^2}$$

is the positive, bell shaped curve that tends to 0 as  $x$  tends to  $\pm\infty$ . You can create an example of this curve, for  $a = 1/2$ .

```
syms x
a = sym(1/2);
f = exp(-a*x^2);
fplot(f)
```



However, if you try to calculate the integral

$$\int_{-\infty}^{\infty} e^{-ax^2} dx$$

without assigning a value to  $a$ , MATLAB assumes that  $a$  represents a complex number, and therefore returns a piecewise answer that depends on the argument of  $a$ . If you are only interested in the case when  $a$  is a positive real number, use `assume` to set an assumption on  $a$ :

```
syms a
assume(a > 0)
```

Now you can calculate the preceding integral using the commands

```
syms x
f = exp(-a*x^2);
int(f, x, -inf, inf)
```

This returns

```
ans =
pi^(1/2)/a^(1/2)
```

## Integration with Complex Parameters

To calculate the integral

$$\int_{-\infty}^{\infty} \frac{1}{a^2 + x^2} dx$$

for complex values of  $a$ , enter

```
syms a x
f = 1/(a^2 + x^2);
F = int(f, x, -inf, inf)
```

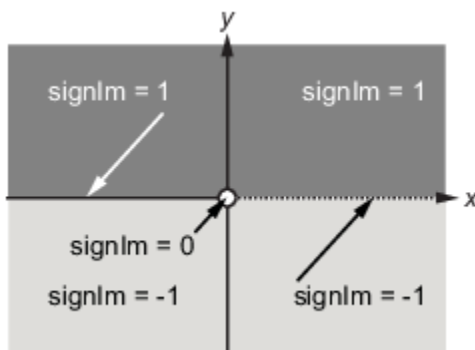
Use `syms` to clear all the assumptions on variables. For more information about symbolic variables and assumptions on them, see “Use Assumptions on Symbolic Variables” on page 1-41.

The preceding commands produce the complex output

```
F =
(pi*signIm(1i/a))/a
```

The function `signIm` is defined as:

$$\text{signIm}(z) = \begin{cases} 1 & \text{if } \text{Im}(z) > 0, \text{ or } \text{Im}(z) = 0 \text{ and } \text{Re}(z) < 0 \\ 0 & \text{if } z = 0 \\ -1 & \text{otherwise.} \end{cases}$$



To evaluate  $F$  at  $a = 1 + i$ , enter

```
g = subs(F, 1 + i)
g =
pi*(1/2 - 1i/2)
double(g)
ans =
1.5708 - 1.5708i
```

## High-Precision Numerical Integration Using Variable-Precision Arithmetic

High-precision numerical integration is implemented in the `vpaintegral` function of the Symbolic Math Toolbox. `vpaintegral` uses variable-precision arithmetic in contrast to the MATLAB `integral` function, which uses double-precision arithmetic.

Integrate `besseli(5,25*u).*exp(-u*25)` by using both `integral` and `vpaintegral`. The `integral` function returns NaN and issues a warning while `vpaintegral` returns the correct result.

```
syms u
f = besseli(5,25*x).*exp(-x*25);
```

```
fun = @(u)besseli(5,25*u).*exp(-u*25);  
  
usingIntegral = integral(fun, 0, 30)  
usingVpintegral = vpintegral(f, 0, 30)  
  
Warning: Infinite or Not-a-Number value encountered.  
usingIntegral =  
    NaN  
  
usingVpintegral =  
    0.688424
```

For more information, see `vpintegral`.

## See Also

`int` | `diff` | `vpintegral`

## Teaching Resources

- [Calculus Integrals \(MathWorks Teaching Resources\)](#)

## Taylor Series

The statements

```
syms x
f = 1/(5 + 4*cos(x));
T = taylor(f, 'Order', 8)
```

return

```
T =
(49*x^6)/131220 + (5*x^4)/1458 + (2*x^2)/81 + 1/9
```

which is all the terms up to, but not including, order eight in the Taylor series for  $f(x)$ :

$$\sum_{n=0}^{\infty} (x-a)^n \frac{f^{(n)}(a)}{n!}.$$

Technically, T is a Maclaurin series, since its expansion point is  $a = 0$ .

These commands

```
syms x
g = exp(x*sin(x));
t = taylor(g, 'ExpansionPoint', 2, 'Order', 12);
```

generate the first 12 nonzero terms of the Taylor series for g about  $x = 2$ .

t is a large expression; enter

```
size(char(t))
```

```
ans =
1      99791
```

to find that t has about 100,000 characters in its printed form. In order to proceed with using t, first simplify its presentation:

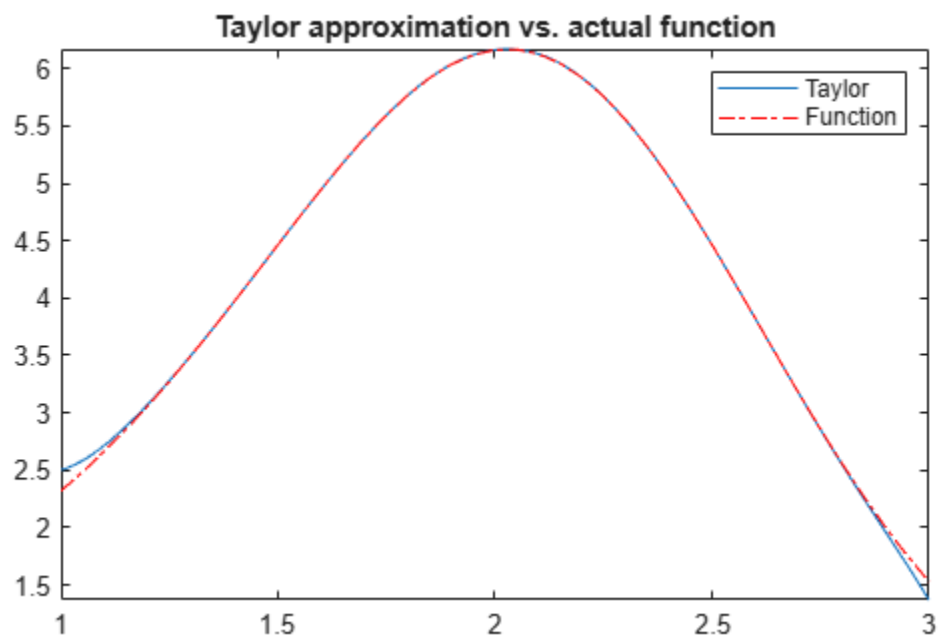
```
t = simplify(t);
size(char(t))
```

```
ans =
1      6988
```

Next, plot these functions together to see how well this Taylor approximation compares to the actual function g:

```
xd = 1:0.05:3;
yd = subs(g,x,xd);
fplot(t, [1, 3])
hold on
plot(xd, yd, 'r-.')
title('Taylor approximation vs. actual function')
legend('Taylor', 'Function')
```





Special thanks is given to Professor Gunnar Bäckström of UMEA in Sweden for this example.

## Fourier and Inverse Fourier Transforms

This page shows the workflow for Fourier and inverse Fourier transforms in Symbolic Math Toolbox. For simple examples, see `fourier` and `ifourier`. Here, the workflow for Fourier transforms is demonstrated by calculating the deflection of a beam due to a force. The associated differential equation is solved by the Fourier transform.

### Fourier Transform Definition

The Fourier transform of  $f(x)$  with respect to  $x$  at  $w$  is

$$F(w) = \int_{-\infty}^{\infty} f(x)e^{-iwx}dx.$$

The inverse Fourier transform is

$$f(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} F(w)e^{iwx}dw.$$

### Concept: Using Symbolic Workflows

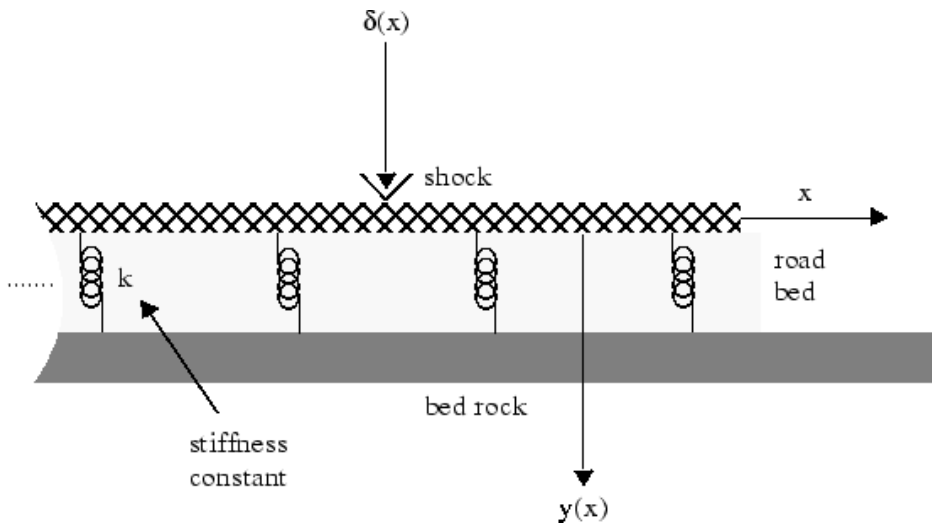
Symbolic workflows keep calculations in the natural symbolic form instead of numeric form. This approach helps you understand the properties of your solution and use exact symbolic values. You substitute numbers in place of symbolic variables only when you require a numeric result or you cannot continue symbolically. For details, see “Choose Numeric or Symbolic Arithmetic” on page 2-32. Typically, the steps are:

- 1 Declare equations.
- 2 Solve equations.
- 3 Substitute values.
- 4 Plot results.
- 5 Analyze results.

## Calculate Beam Deflection Using Fourier Transform

### Define Equations

Fourier transform can be used to solve ordinary and partial differential equations. For example, you can model the deflection of an infinitely long beam resting on an elastic foundation under a point force. A corresponding real-world example is railway tracks on a foundation. The railway tracks are the infinitely long beam while the foundation is elastic.



Let

- $E$  be the elasticity of the beam (or railway track).
- $I$  be the second moment of area of the cross-section of the beam.
- $k$  be the spring stiffness of the foundation.

The differential equation is

$$\frac{d^4 y}{dx^4} + \frac{k}{EI} y = \frac{1}{EI} \delta(x), \quad -\infty < x < \infty.$$

Define the function  $y(x)$  and the variables. Assume  $E$ ,  $I$ , and  $k$  are positive.

```
syms Y(x) w E I k f
assume([E I k] > 0)
```

Assign units to the variables by using `symunit`.

```
u = symunit;
Eu = E*u.Pa;    % Pascal
Iu = I*u.m^4;   % meter^4
ku = k*u.N/u.m^2; % Newton/meter^2
X = x*u.m;
F = f*u.N/u.m;
```

Define the differential equation.

```
eqn = diff(Y,X,4) + ku/(Eu*Iu)*Y == F/(Eu*Iu)
```

```
eqn(x) =
diff(Y(x), x, x, x, x)*(1/[m]^4) + ((k*Y(x))/(E*I))*([N]/([Pa]*[m]^6)) == ...
(f/(E*I))*([N]/([Pa]*[m]^5))
```

Represent the force  $f$  by the Dirac delta function  $\delta(x)$ .

```
eqn = subs(eqn,f,dirac(x))
```

```
eqn(x) =
diff(Y(x), x, x, x, x)*(1/[m]^4) + ((k*Y(x))/(E*I))*([N]/([Pa]*[m]^6)) == ...
(dirac(x)/(E*I))*([N]/([Pa]*[m]^5))
```

### Solve Equations

Calculate the Fourier transform of eqn by using `fourier` on both sides of eqn. The Fourier transform converts differentiation into exponents of  $w$ .

```
eqnFT = fourier(lhs(eqn)) == fourier(rhs(eqn))
```

```
eqnFT =
w^4*fourier(Y(x), x, w)*(1/[m]^4) + ((k*fourier(Y(x), x, w))/(E*I))*([N]/([Pa]*[m]^6)) ...
== (1/(E*I))*([N]/([Pa]*[m]^5))
```

Isolate `fourier(Y(x),x,w)` in the equation.

```
eqnFT = isolate(eqnFT, fourier(Y(x),x,w))
```

```
eqnFT =
fourier(Y(x), x, w) == (1/(E*I*w^4*[Pa]*[m]^2 + k*[N]))*[N]*[m]
```

Calculate  $Y(x)$  by calculating the inverse Fourier transform of the right side. Simplify the result.

```
YSol = ifourier(rhs(eqnFT));
YSol = simplify(YSol)
```

```
YSol =
((exp(-(2^(1/2)*k^(1/4)*abs(x))/(2*E^(1/4)*I^(1/4))))*sin((2*2^(1/2)*k^(1/4)*abs(x) + ...
pi*E^(1/4)*I^(1/4))/(4*E^(1/4)*I^(1/4)))/(2*E^(1/4)*I^(1/4)*k^(3/4)))*[m]
```

Check that `YSol` has the correct dimensions by substituting `YSol` into `eqn` and using the `checkUnits` function. `checkUnits` returns logical 1 (true), meaning `eqn` now has compatible units of the same physical dimensions.

```
checkUnits(subs(eqn,Y,YSol))
```

```
ans =
struct with fields:

    Consistent: 1
    Compatible: 1
```

Separate the expression from the units by using `separateUnits`.

```
YSol = separateUnits(YSol)
```

```
YSol =
(exp(-(2^(1/2)*k^(1/4)*abs(x))/(2*E^(1/4)*I^(1/4))))*sin((2*2^(1/2)*k^(1/4)*abs(x) + ...
pi*E^(1/4)*I^(1/4))/(4*E^(1/4)*I^(1/4)))/(2*E^(1/4)*I^(1/4)*k^(3/4))
```

### Substitute Values

Use the values  $E = 10^6$  Pa,  $I = 10^{-3} \text{ m}^4$ , and  $k = 10^6 \text{ N/m}^2$ . Substitute these values into `YSol` and convert to floating point by using `vpa` with 16 digits of accuracy.

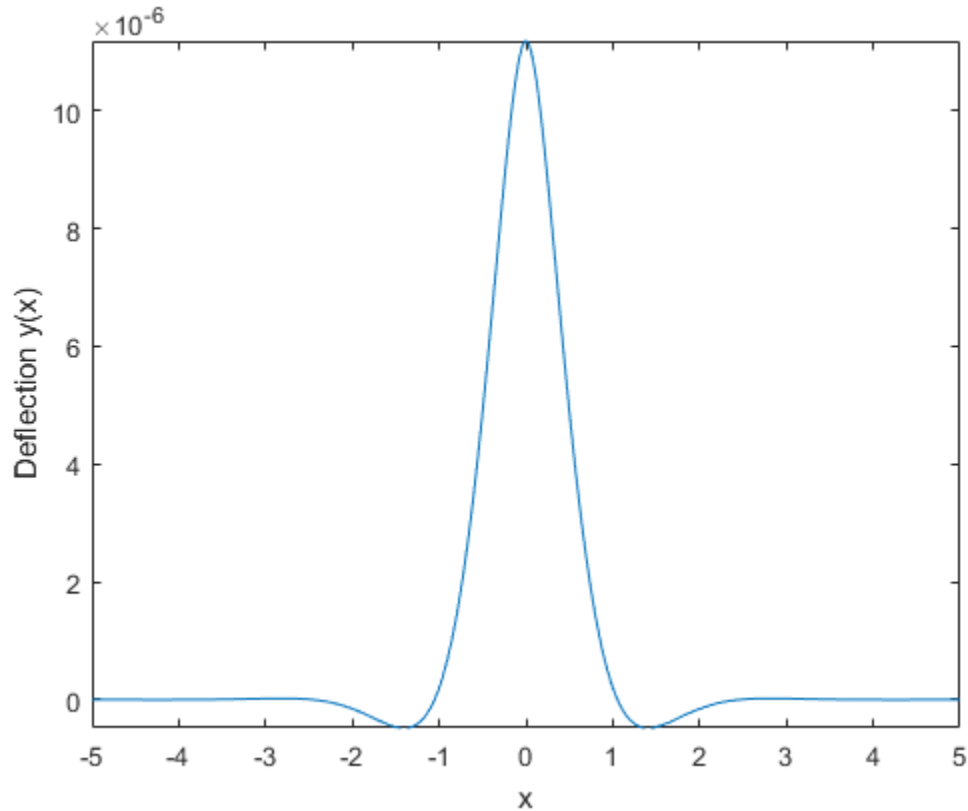
```
values = [1e6 1e-3 1e5];
YSol = subs(YSol,[E I k],values);
YSol = vpa(YSol,16)
```

```
YSol =
0.0000158113883008419*exp(-2.23606797749979*abs(x))*sin(2.23606797749979*abs(x) + ...
0.7853981633974483)
```

### Plot Results

Plot the result by using `fplot`.

```
fplot(YSol)
xlabel('x')
ylabel('Deflection y(x)')
```



### Analyze Results

The plot shows that the deflection of a beam due to a point force is highly localized. The deflection is greatest at the point of impact and then decreases quickly. The symbolic result enables you to analyze the properties of the result, which is not possible with numeric results.

Notice that `YSol` is a product of terms. The term with `sin` shows that the response is vibrating oscillatory behavior. The term with `exp` shows that the oscillatory behavior is quickly damped by the exponential decay as the distance from the point of impact increases.

### See Also

### Teaching Resources

- [Fourier Analysis \(MathWorks Teaching Resources\)](#)

## Solve Differential Equations of RLC Circuit Using Laplace Transform

Solve differential equations of an RLC circuit by using Laplace transforms in Symbolic Math Toolbox™ with this workflow. For simple examples on the Laplace transform, see `laplace` and `ilaplace`.

### Definition: Laplace Transform

The Laplace transform of a function  $f(t)$  is

$$F(s) = \int_0^{\infty} f(t)e^{-ts}dt.$$

### Concept: Using Symbolic Workflows

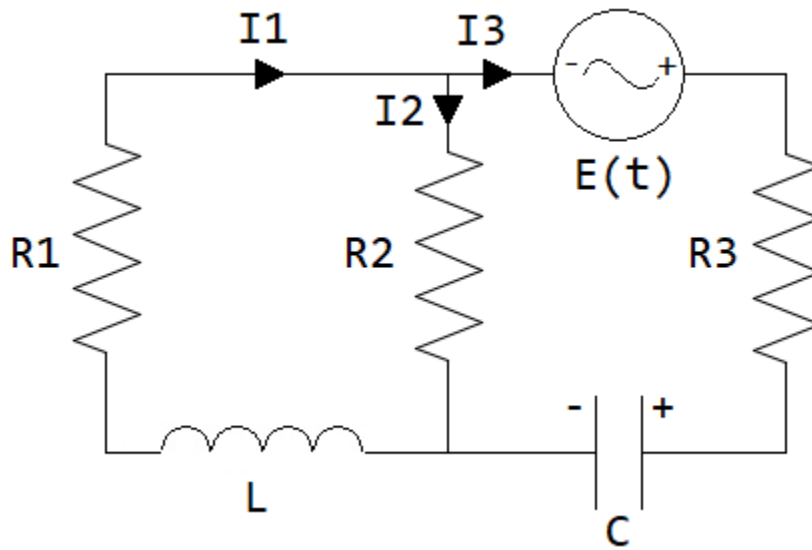
Symbolic workflows keep calculations in the natural symbolic form instead of numeric form. This approach helps you understand the properties of your solution and use exact symbolic values. You substitute numbers in place of symbolic variables only when you require a numeric result or you cannot continue symbolically. For details, see “Choose Numeric or Symbolic Arithmetic” on page 2-32. Typically, the steps are:

- 1 Declare equations.
- 2 Solve equations.
- 3 Substitute values.
- 4 Plot results.
- 5 Analyze results.

### Workflow: Solve RLC Circuit Using Laplace Transform

#### Declare Equations

You can use the Laplace transform to solve differential equations with initial conditions. For example, you can solve resistance-inductor-capacitor (RLC) circuits, such as this circuit.



- Resistances in ohm:  $R_1, R_2, R_3$
- Currents in ampere:  $I_1, I_2, I_3$
- Inductance in henry:  $L$
- Capacitance in farad:  $C$
- AC voltage source in volts:  $E(t)$
- Capacitor charge in coulomb:  $Q(t)$

Apply Kirchhoff's voltage and current laws to get the following equations.

$$I_1 = I_2 + I_3$$

$$L \frac{dI_1}{dt} + I_1 R_1 + I_2 R_2 = 0$$

$$E(t) + I_2 R_2 - \frac{Q}{C} - I_3 R_3 = 0$$

Substitute the relation  $I_3 = dQ/dt$  (which is the rate of the capacitor being charged) to the above equations to get the differential equations for the RLC circuit.

$$\frac{dI_1}{dt} - \frac{R_2}{L} \frac{dQ}{dt} = -\frac{R_1 + R_2}{L} I_1$$

$$\frac{dQ}{dt} = \frac{1}{R_2 + R_3} \left( E(t) - \frac{Q}{C} \right) + \frac{R_2}{R_2 + R_3} I_1$$

Declare the variables. Because the physical quantities have positive values, set the corresponding assumptions on the variables. Let  $E(t)$  be an alternating voltage of 1 V.

```
syms L C I1(t) Q(t) s
R = sym('R%d',[1 3]);
assume([t L C R] > 0)
E(t) = 1*sin(t);           % AC voltage = 1 V
```

Declare the differential equations.

```
dI1 = diff(I1,t);
dQ = diff(Q,t);
eqn1 = dI1 - (R(2)/L)*dQ == -(R(1)+R(2))/L*I1
```

```
eqn1(t) =
```

$$\frac{\partial}{\partial t} I_1(t) - \frac{R_2}{L} \frac{\partial}{\partial t} Q(t) = -\frac{I_1(t) (R_1 + R_2)}{L}$$

```
eqn2 = dQ == (1/(R(2)+R(3))*(E-Q/C)) + R(2)/(R(2)+R(3))*I1
```

```
eqn2(t) =
```

$$\frac{\partial}{\partial t} Q(t) = \frac{\sin(t) - \frac{Q(t)}{C}}{R_2 + R_3} + \frac{R_2 I_1(t)}{R_2 + R_3}$$

### Solve Equations

Compute the Laplace transform of eqn1 and eqn2.

```
eqn1LT = laplace(eqn1,t,s)
```

```
eqn1LT =
```

$$s \operatorname{laplace}(I_1(t), t, s) - I_1(0) + \frac{R_2 (Q(0) - s \operatorname{laplace}(Q(t), t, s))}{L} = -\frac{(R_1 + R_2) \operatorname{laplace}(I_1(t), t, s)}{L}$$

```
eqn2LT = laplace(eqn2,t,s)
```

```
eqn2LT =
```

$$s \operatorname{laplace}(Q(t), t, s) - Q(0) = \frac{R_2 \operatorname{laplace}(I_1(t), t, s)}{R_2 + R_3} + \frac{\frac{C}{s^2 + 1} - \operatorname{laplace}(Q(t), t, s)}{C (R_2 + R_3)}$$

The function `solve` solves only for symbolic variables. Therefore, to use `solve`, first substitute `laplace(I1(t),t,s)` and `laplace(Q(t),t,s)` with the variables `I1_LT` and `Q_LT`.

```
syms I1_LT Q_LT
```

```
eqn1LT = subs(eqn1LT,[laplace(I1,t,s) laplace(Q,t,s)],[I1_LT Q_LT])
```

```
eqn1LT =
```

$$I_{1,LT} s - I_1(0) + \frac{R_2 (Q(0) - Q_{LT} s)}{L} = -\frac{I_{1,LT} (R_1 + R_2)}{L}$$

```
eqn2LT = subs(eqn2LT,[laplace(I1,t,s) laplace(Q,t,s)],[I1_LT Q_LT])
```

```
eqn2LT =
```

$$Q_{LT} s - Q(0) = \frac{I_{1,LT} R_2}{R_2 + R_3} - \frac{Q_{LT} - \frac{C}{s^2 + 1}}{C (R_2 + R_3)}$$

Solve the equations for `I1_LT` and `Q_LT`.

```
eqns = [eqn1LT eqn2LT];
```

```
vars = [I1_LT Q_LT];
```

```
[I1_LT, Q_LT] = solve(eqns,vars)
```

```
I1_LT =
```

$$\frac{L I_1(0) - R_2 Q(0) + C R_2 s + L s^2 I_1(0) - R_2 s^2 Q(0) + C L R_2 s^3 I_1(0) + C L R_3 s^3 I_1(0) + C L R_2 s I_1(0) + C L R_3 s I_1(0)}{(s^2 + 1) (R_1 + R_2 + L s + C L R_2 s^2 + C L R_3 s^2 + C R_1 R_2 s + C R_1 R_3 s + C R_2 R_3 s)}$$



$$Q\_LT = \frac{C(R_1 + R_2 + Ls + LR_2 I_1(0) + R_1 R_2 Q(0) + R_1 R_3 Q(0) + R_2 R_3 Q(0) + LR_2 s^2 I_1(0) + LR_2 s^3 Q(0) + LR_3 s^3 Q(0) + R_1 R_2 s^2 I_1(0) + R_1 R_3 s^2 Q(0) + R_2 R_3 s^2 Q(0) + CL R_2 s^2 + CL R_3 s^2 + CR_1 R_2 s + CR_1 R_3 s + CR_2 R_3)}{(s^2 + 1)(R_1 + R_2 + Ls + CL R_2 s^2 + CL R_3 s^2 + CR_1 R_2 s + CR_1 R_3 s + CR_2 R_3)}$$

Compute  $I_1$  and  $Q$  by computing the inverse Laplace transform of  $I1\_LT$  and  $Q\_LT$ . Simplify the result. Suppress the output because it is long.

```
Ilsol = ilaplace(I1_LT,s,t);
Qsol = ilaplace(Q_LT,s,t);
Ilsol = simplify(Ilsol);
Qsol = simplify(Qsol);
```

### Substitute Values

Before plotting the result, substitute symbolic variables by the numeric values of the circuit elements. Let  $R_1 = 4 \Omega$ ,  $R_2 = 2 \Omega$ ,  $R_3 = 3 \Omega$ ,  $C = 1/4 \text{ F}$ ,  $L = 1.6 \text{ H}$ . Assume that the initial current is  $I_1(0) = 2 \text{ A}$  and the initial charge is  $Q(0) = 2 \text{ C}$ .

```
vars = [R L C I1(0) Q(0)];
values = [4 2 3 1.6 1/4 2 2];
Ilsol = subs(Ilsol,vars,values)
```

$$Ilsol = \frac{200 \cos(t)}{8161} + \frac{405 \sin(t)}{8161} + \frac{16122 e^{-\frac{81t}{40}} \left( \cosh\left(\frac{\sqrt{1761}t}{40}\right) - \frac{742529 \sqrt{1761} \sinh\left(\frac{\sqrt{1761}t}{40}\right)}{14195421} \right)}{8161}$$

```
Qsol = subs(Qsol,vars,values)
```

$$Qsol = \frac{924 \sin(t)}{8161} - \frac{1055 \cos(t)}{8161} + \frac{17377 e^{-\frac{81t}{40}} \left( \cosh\left(\frac{\sqrt{1761}t}{40}\right) + \frac{1109425 \sqrt{1761} \sinh\left(\frac{\sqrt{1761}t}{40}\right)}{30600897} \right)}{8161}$$

### Plot Results

Plot the current  $Ilsol$  and charge  $Qsol$ . Show both the transient and steady state behavior by using two different time intervals:  $0 \leq t \leq 15$  and  $2 \leq t \leq 25$ .

```
subplot(2,2,1)
fplot(Ilsol,[0 15])
title('Current')
ylabel('I1(t)')
xlabel('t')
```

```
subplot(2,2,2)
fplot(Qsol,[0 15])
title('Charge')
ylabel('Q(t)')
xlabel('t')
```

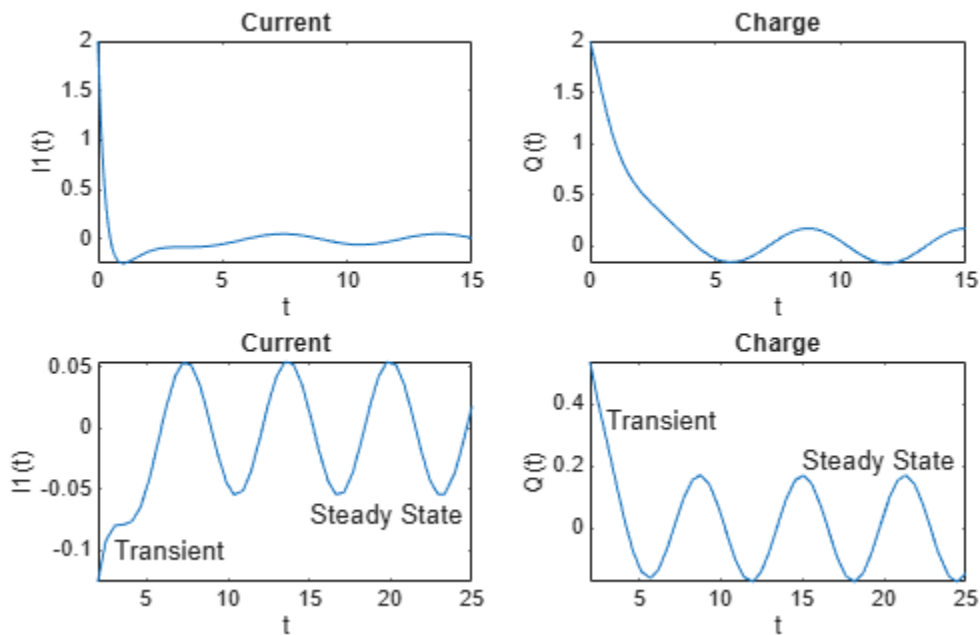
```
subplot(2,2,3)
fplot(Ilsol,[2 25])
title('Current')
```

```

ylabel('I1(t)')
xlabel('t')
text(3,-0.1,'Transient')
text(15,-0.07,'Steady State')

subplot(2,2,4)
fplot(Qsol,[2 25])
title('Charge')
ylabel('Q(t)')
xlabel('t')
text(3,0.35,'Transient')
text(15,0.22,'Steady State')

```



### Analyze Results

Initially, the current and charge decrease exponentially. However, over the long term, they are oscillatory. These behaviors are called "transient" and "steady state", respectively. With the symbolic result, you can analyze the result's properties, which is not possible with numeric results.

Visually inspect `Ilsol` and `Qsol`. They are a sum of terms. Find the terms by using `children`. Then, find the contributions of the terms by plotting them over `[0 15]`. The plots show the transient and steady state terms.

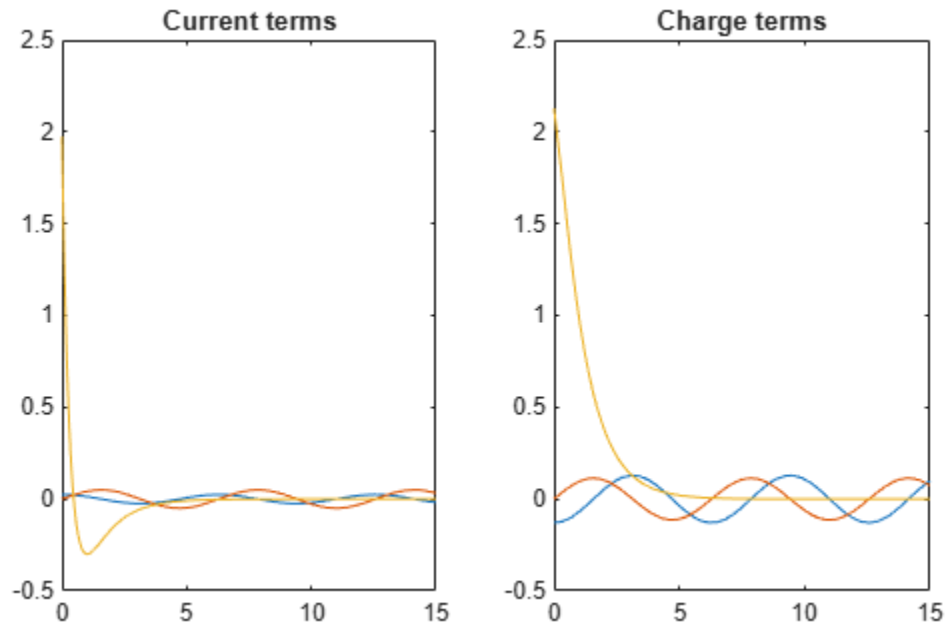
```

Isterms = children(Ilsol);
Isterms = [Isterms{:}];
Qterms = children(Qsol);
Qterms = [Qterms{:}];

figure;
subplot(1,2,1)
fplot(Isterms,[0 15])
ylim([-0.5 2.5])
title('Current terms')

```

```
subplot(1,2,2)
fplot(Qterms,[0 15])
ylim([-0.5 2.5])
title('Charge terms')
```



The plots show that  $I_{lsol}$  has a transient and steady state term, while  $Q_{sol}$  has a transient and two steady state terms. From visual inspection, notice  $I_{lsol}$  and  $Q_{sol}$  have a term containing the  $\exp$  function. Assume that this term causes the transient exponential decay. Separate the transient and steady state terms in  $I_{lsol}$  and  $Q_{sol}$  by checking terms for  $\exp$  using `has`.

```
Iltransient = Ifterms(has(Ifterms,'exp'))
```

```
Iltransient =
```

$$\frac{16122 e^{-\frac{81t}{40}} \left( \cosh\left(\frac{\sqrt{1761}t}{40}\right) - \frac{742529 \sqrt{1761} \sinh\left(\frac{\sqrt{1761}t}{40}\right)}{14195421} \right)}{8161}$$

```
Ilsteadystate = Ifterms(~has(Ifterms,'exp'))
```

```
Ilsteadystate =
```

$$\left( \frac{200 \cos(t)}{8161} \frac{405 \sin(t)}{8161} \right)$$

Similarly, separate  $Q_{sol}$  into transient and steady state terms. This result demonstrates how symbolic calculations help you analyze your problem.

```
Qtransient = Qfterms(has(Qfterms,'exp'))
```

```
Qtransient =
```

$$\frac{17377 e^{-\frac{81 t}{40}} \left( \cosh\left(\frac{\sqrt{1761} t}{40}\right) + \frac{1109425 \sqrt{1761} \sinh\left(\frac{\sqrt{1761} t}{40}\right)}{30600897} \right)}{8161}$$

`Qsteadystate = Qterms(~has(Qterms,'exp'))`

$$Qsteadystate = \left( -\frac{1055 \cos(t)}{8161} \quad \frac{924 \sin(t)}{8161} \right)$$

## See Also

### Related Examples

- “Analyze Transfer Function of T-Coil Circuit” on page 3-324
- “Harmonic Analysis of Transfer Function Output” on page 3-160
- “Padé Approximant of Time-Delay Input” on page 3-261
- “Estimate Model Parameters of a Symbolically Derived Plant Model in Simulink” on page 5-74
- “Customize and Extend Simscape Libraries for a Custom DC Motor” on page 5-65

# Solve Difference Equations Using Z-Transform

Solve difference equations by using Z-transforms in Symbolic Math Toolbox™ with this workflow. For simple examples on the Z-transform, see `ztrans` and `iztrans`.

## Definition: Z-transform

The Z-transform of a function  $f(n)$  is defined as

$$F(z) = \sum_{n=0}^{\infty} \frac{f(n)}{z^n}.$$

## Concept: Using Symbolic Workflows

Symbolic workflows keep calculations in the natural symbolic form instead of numeric form. This approach helps you understand the properties of your solution and use exact symbolic values. You substitute numbers in place of symbolic variables only when you require a numeric result or you cannot continue symbolically. For details, see “Choose Numeric or Symbolic Arithmetic” on page 2-32. Typically, the steps are:

- 1 Declare equations.
- 2 Solve equations.
- 3 Substitute values.
- 4 Plot results.
- 5 Analyze results.

## Workflow: Solve "Rabbit Growth" Problem Using Z-Transform

### Declare Equations

You can use the Z-transform to solve difference equations, such as the well-known "Rabbit Growth" problem. If a pair of rabbits matures in one year, and then produces another pair of rabbits every year, the rabbit population  $p(n)$  at year  $n$  is described by this difference equation.

$$p(n+2) = p(n+1) + p(n)$$

Declare the equation as an expression assuming the right side is 0. Because  $n$  represents years, assume that  $n$  is a positive integer. This assumption simplifies the results.

```
syms p(n) z
assume(n>=0 & in(n,"integer"))
f = p(n+2) - p(n+1) - p(n)

f = p(n+2) - p(n+1) - p(n)
```

### Solve Equations

Find the Z-transform of the equation.

```
fZT = ztrans(f,n,z)
```

```
fZT = z*p(0) - z*ztrans(p(n),n,z) - z*p(1) + z^2*ztrans(p(n),n,z) - z^2*p(0) - ztrans(p(n),n,z)
```

The function `solve` solves only for symbolic variables. Therefore, to use `solve`, first substitute `ztrans(p(n),n,z)` with the variables `pZT`.

```
syms pZT
fZT = subs(fZT,ztrans(p(n),n,z),pZT)

fZT = z*p(0) - pZT - z*p(1) - pZT*z - z^2*p(0) + pZT*z^2
```

Solve for `pZT`.

```
pZT = solve(fZT,pZT)

pZT =

$$-\frac{z p(1) - z p(0) + z^2 p(0)}{-z^2 + z + 1}$$

```

Calculate  $p(n)$  by computing the inverse Z-transform of `pZT`. Simplify the result.

```
pSol = iztrans(pZT,z,n);
pSol = simplify(pSol)

pSol =

$$2(-1)^{n/2} p(1) \cos\left(n\left(\frac{\pi}{2} + \operatorname{asin}\left(\frac{1}{2} i\right)\right)\right) + \frac{2^{2-n} \sqrt{5} (\sqrt{5} + 1)^{n-1} \sigma_1}{5} - \frac{2^{2^{1-n}} \sqrt{5} (1 - \sqrt{5})^{n-1} \sigma_1}{5}$$

where

$$\sigma_1 = \frac{p(0)}{2} - p(1)$$

```

### Substitute Values

To plot the result, first substitute the values of the initial conditions. Let  $p(0)$  and  $p(1)$  be 1 and 2, respectively.

```
pSol = subs(pSol,[p(0) p(1)],[1 2])

pSol =

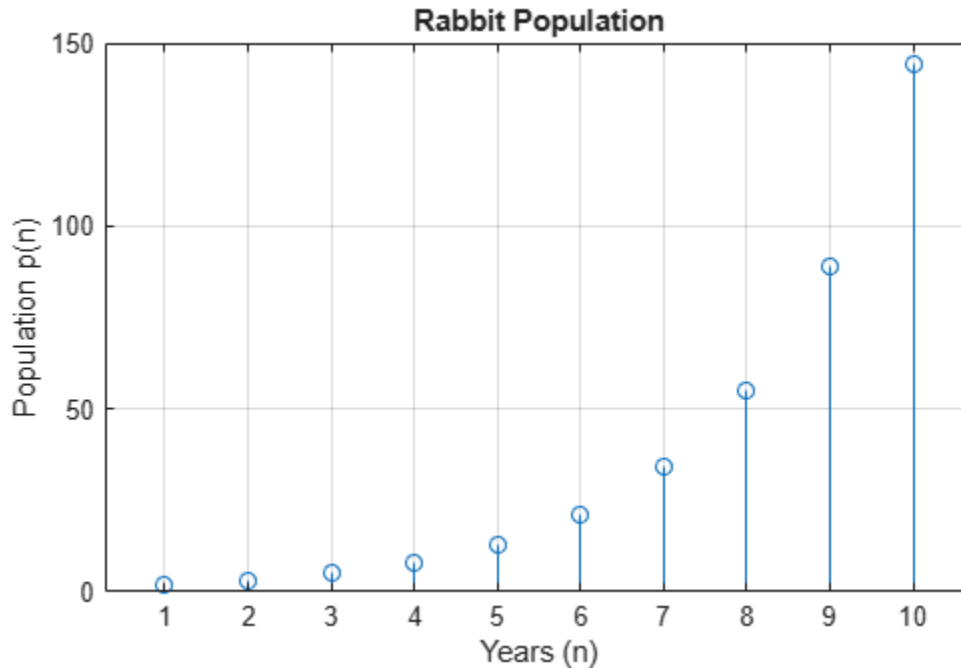
$$4(-1)^{n/2} \cos\left(n\left(\frac{\pi}{2} + \operatorname{asin}\left(\frac{1}{2} i\right)\right)\right) - \frac{3 \cdot 2^{2-n} \sqrt{5} (\sqrt{5} + 1)^{n-1}}{10} + \frac{3 \cdot 2^{1-n} \sqrt{5} (1 - \sqrt{5})^{n-1}}{5}$$

```

### Plot Results

Show the growth in rabbit population over time by plotting `pSol`.

```
nValues = 1:10;
pSolValues = subs(pSol,n,nValues);
pSolValues = double(pSolValues);
pSolValues = real(pSolValues);
stem(nValues,pSolValues)
title("Rabbit Population")
xlabel("Years (n)")
ylabel("Population p(n)")
grid on
```



### Analyze Results

The plot shows that the solution appears to increase exponentially. However, because the solution `pSol` contains many terms, finding the terms that produce this behavior requires analysis.

Because all the functions in `pSol` can be expressed in terms of `exp`, rewrite `pSol` to `exp`. Simplify the result by using `simplify` with 80 additional simplification steps. Now, you can analyze `pSol`.

```
pSol = rewrite(pSol,"exp");
pSol = simplify(pSol,"Steps",80)
```

$$pSol = \frac{2 \cdot 2^n}{(\sqrt{5} - 1)^n} + \frac{2(1 - \sqrt{5})^n}{2^n} - \frac{6\sqrt{5}(\sqrt{5} + 1)^n}{5 \cdot 2^n(\sqrt{5} + 1)} - \frac{6\sqrt{5}(1 - \sqrt{5})^n}{5 \cdot 2^n(\sqrt{5} - 1)}$$

Visually inspect `pSol`. Notice that `pSol` is a sum of terms. Each term is a ratio that can increase or decrease as  $n$  increases. For each term, you can confirm this hypothesis in several ways:

- Check if the limit at  $n = \text{Inf}$  goes to 0 or  $\text{Inf}$  by using `limit`.
- Plot the term for increasing  $n$  and check behavior.
- Calculate the value at a large value of  $n$ .

For simplicity, use the third approach. Calculate the terms at  $n = 100$ , and then verify the approach. First, find the individual terms by using `children`, substitute for  $n$ , and convert to double.

```
pSolTerms = children(pSol);
pSolTerms = [pSolTerms{:}];
pSolTermsDb1 = subs(pSolTerms,n,100);
pSolTermsDb1 = double(pSolTermsDb1)
```

```
pSolTermsDb1 = 1x4
1021 ×
```

1.5841    0.0000    -0.6568    -0.0000

The result shows that some terms are 0 while other terms have a large magnitude. Hypothesize that the large-magnitude terms produce the exponential behavior. Approximate pSol with these terms.

```
idx = abs(pSolTermsDbl)>1; % use arbitrary cutoff
pApprox = pSolTerms(idx);
pApprox = sum(pApprox)
```

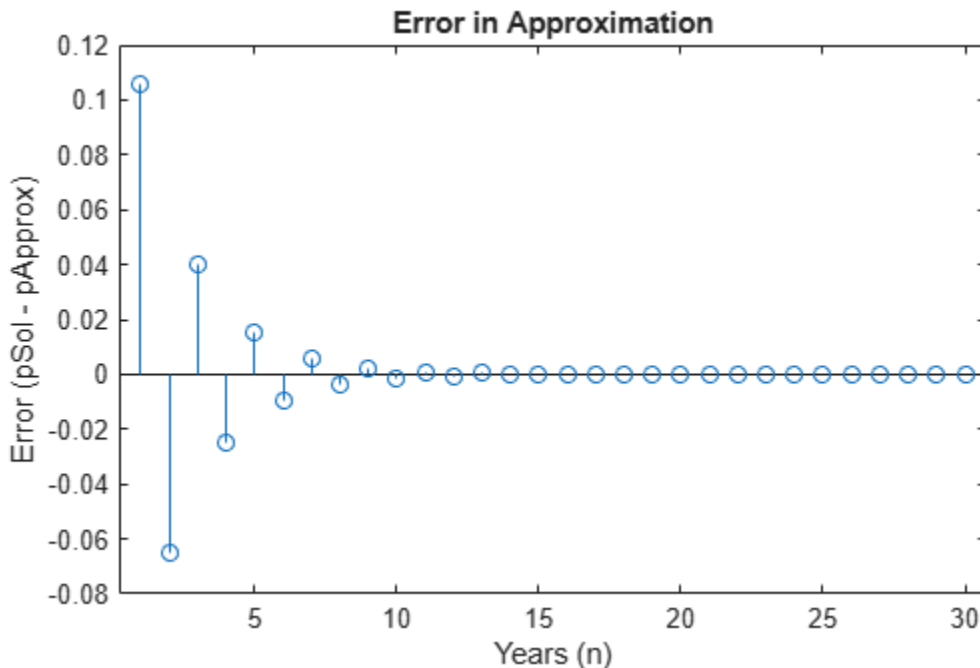
```
pApprox =

$$\frac{2 \cdot 2^n}{(\sqrt{5} - 1)^n} - \frac{6 \sqrt{5} (\sqrt{5} + 1)^n}{5 \cdot 2^n (\sqrt{5} + 1)}$$

```

Verify the hypothesis by plotting the approximation error between pSol and pApprox. As expected, the error goes to 0 as n increases. This result demonstrates how symbolic calculations help you analyze your problem.

```
Perror = pSol - pApprox;
nValues = 1:30;
Perror = subs(Perror,n,nValues);
stem(nValues,Perror)
xlabel("Years (n)")
ylabel("Error (pSol - pApprox)")
title("Error in Approximation")
```



## References

- [1] Andrews, L.C., Shivamoggi, B.K., *Integral Transforms for Engineers and Applied Mathematicians*, Macmillan Publishing Company, New York, 1986
- [2] Crandall, R.E., *Projects in Scientific Computation*, Springer-Verlag Publishers, New York, 1994



- [3] Strang, G., *Introduction to Applied Mathematics*, Wellesley-Cambridge Press, Wellesley, MA, 1986

## Symbolic Summation

Symbolic Math Toolbox provides two functions for calculating sums:

- `sum` finds the sum of elements of symbolic vectors and matrices. Unlike the MATLAB `sum`, the symbolic `sum` function does not work on multidimensional arrays. For details, follow the MATLAB `sum` page.
- `symsum` finds the sum of a symbolic series.

| In this section...                                                                          |
|---------------------------------------------------------------------------------------------|
| "Comparing <code>symsum</code> and <code>sum</code> " on page 3-210                         |
| "Computational Speed of <code>symsum</code> versus <code>sum</code> " on page 3-210         |
| "Output Format Differences Between <code>symsum</code> and <code>sum</code> " on page 3-211 |

### Comparing `symsum` and `sum`

You can find definite sums by using both `sum` and `symsum`. The `sum` function sums the input over a dimension, while the `symsum` function sums the input over an index.

Consider the definite sum  $S = \sum_{k=1}^{10} \frac{1}{k^2}$ . First, find the terms of the definite sum by substituting the index values for  $k$  in the expression. Then, sum the resulting vector using `sum`.

```
syms k
f = 1/k^2;
V = subs(f, k, 1:10)
S_sum = sum(V)

V =
[ 1, 1/4, 1/9, 1/16, 1/25, 1/36, 1/49, 1/64, 1/81, 1/100]
S_sum =
1968329/1270080
```

Find the same sum by using `symsum` by specifying the index and the summation limits. `sum` and `symsum` return identical results.

```
S_symsum = symsum(f, k, 1, 10)

S_symsum =
1968329/1270080
```

### Computational Speed of `symsum` versus `sum`

For summing definite series, `symsum` can be faster than `sum`. For summing an indefinite series, you can only use `symsum`.

You can demonstrate that `symsum` can be faster than `sum` by summing a large definite series such as

$$S = \sum_{k=1}^{100000} k^2.$$

To compare runtimes on your computer, use the following commands.

```
syms k
tic
sum(sym(1:100000).^2);
toc
tic
symsum(k^2, k, 1, 100000);
toc
```

## Output Format Differences Between symsum and sum

symsum can provide a more elegant representation of sums than sum provides. Demonstrate this difference by comparing the function outputs for the definite series  $S = \sum_{k=1}^{10} x^k$ . To simplify the solution, assume  $x > 1$ .

```
syms x
assume(x > 1)
S_sum = sum(x.^(1:10))
S_symsum = symsum(x^k, k, 1, 10)

S_sum =
x^10 + x^9 + x^8 + x^7 + x^6 + x^5 + x^4 + x^3 + x^2 + x
S_symsum =
x^11/(x - 1) - x/(x - 1)
```

Show that the outputs are equal by using `isAlways`. The `isAlways` function returns logical 1 (`true`), meaning that the outputs are equal.

```
isAlways(S_sum == S_symsum)

ans =
    logical
     1
```

For further computations, clear the assumptions.

```
assume(x, 'clear')
```

## Padé Approximant

The Padé approximant of order  $[m, n]$  approximates the function  $f(x)$  around  $x = x_0$  as

$$\frac{a_0 + a_1(x - x_0) + \dots + a_m(x - x_0)^m}{1 + b_1(x - x_0) + \dots + b_n(x - x_0)^n}.$$

The Padé approximant is a rational function formed by a ratio of two power series. Because it is a rational function, it is more accurate than the Taylor series in approximating functions with poles. The Padé approximant is represented by the Symbolic Math Toolbox function `pade`.

When a pole or zero exists at the expansion point  $x = x_0$ , the accuracy of the Padé approximant decreases. To increase accuracy, an alternative form of the Padé approximant can be used which is

$$\frac{(x - x_0)^p (a_0 + a_1(x - x_0) + \dots + a_m(x - x_0)^m)}{1 + b_1(x - x_0) + \dots + b_n(x - x_0)^n}.$$

The `pade` function returns the alternative form of the Padé approximant when you set the `OrderMode` input argument to `Relative`.

The Padé approximant is used in control system theory to model time delays in the response of the system. Time delays arise in systems such as chemical and transport processes where there is a delay between the input and the system response. When these inputs are modeled, they are called dead-time inputs. This example shows how to use the Symbolic Math Toolbox to model the response of a first-order system to dead-time inputs using Padé approximants.

The behavior of a first-order system is described by this differential equation

$$\tau \frac{dy(t)}{dt} + y(t) = ax(t).$$

Enter the differential equation in MATLAB.

```
syms tau a x(t) y(t) xS(s) yS(s) H(s) tmp
F = tau*diff(y)+y == a*x;
```

Find the Laplace transform of  $F$  using `laplace`.

```
F = laplace(F,t,s)
```

```
F = laplace(y(t),t,s) - tau (y(0) - s laplace(y(t),t,s)) = a laplace(x(t),t,s)
```

Assume the response of the system at  $t = 0$  is  $0$ . Use `subs` to substitute for  $y(0) = 0$ .

```
F = subs(F,y(0),0)
```

```
F = laplace(y(t),t,s) + s tau laplace(y(t),t,s) = a laplace(x(t),t,s)
```

To collect common terms, use `simplify`.

```
F = simplify(F)
```

```
F = (s tau + 1) laplace(y(t),t,s) = a laplace(x(t),t,s)
```

For readability, replace the Laplace transforms of  $x(t)$  and  $y(t)$  with  $xS(s)$  and  $yS(s)$ .

```
F = subs(F,[laplace(x(t),t,s) laplace(y(t),t,s)],[xS(s) yS(s)])
```

```
F = yS(s)(s*tau + 1) = a*xS(s)
```

The Laplace transform of the transfer function is  $yS(s)/xS(s)$ . Divide both sides of the equation by  $xS(s)$  and use `subs` to replace  $yS(s)/xS(s)$  with  $H(s)$ .

```
F = F/xS(s);
```

```
F = subs(F,yS(s)/xS(s),H(s))
```

```
F = H(s)(s*tau + 1) = a
```

Solve the equation for  $H(s)$ . Substitute for  $H(s)$  with a dummy variable, solve for the dummy variable using `solve`, and assign the solution back to  $H(s)$ .

```
F = subs(F,H(s),tmp);
```

```
H(s) = solve(F,tmp)
```

```
H(s) =
```

$$\frac{a}{s\tau + 1}$$

The input to the first-order system is a time-delayed step input. To represent a step input, use `heaviside`. Delay the input by three time units. Find the Laplace transform using `laplace`.

```
step = heaviside(t - 3);
```

```
step = laplace(step)
```

```
step =
```

$$\frac{e^{-3s}}{s}$$

Find the response of the system, which is the product of the transfer function and the input.

```
y = H(s)*step
```

```
y =
```

$$\frac{a e^{-3s}}{s(s\tau + 1)}$$

To allow plotting of the response, set parameters `a` and `tau` to their values. For `a` and `tau`, choose values 1 and 3, respectively.

```
y = subs(y,[a tau],[1 3]);
```

```
y = ilaplace(y,s);
```

Find the Padé approximant of order [2 2] of the step input using the `Order` input argument to `pade`.

```
stepPade22 = pade(step,'Order',[2 2])
```

```
stepPade22 =
```

$$\frac{3s^2 - 4s + 2}{2s(s + 1)}$$

Find the response to the input by multiplying the transfer function and the Padé approximant of the input.

```
yPade22 = H(s)*stepPade22
```

$$y_{\text{Pade22}} = \frac{a(3s^2 - 4s + 2)}{2s(s\tau + 1)(s + 1)}$$

Find the inverse Laplace transform of  $y_{\text{Pade22}}$  using `ilaplace`.

```
yPade22 = ilaplace(yPade22,s)
```

```
yPade22 =
```

$$a + \frac{9a}{2\tau - 2}e^{-s} - \frac{ae^{-\frac{s}{\tau}}(2\tau^2 + 4\tau + 3)}{\tau(2\tau - 2)}$$

To plot the response, set parameters  $a$  and  $\tau$  to their values of 1 and 3, respectively.

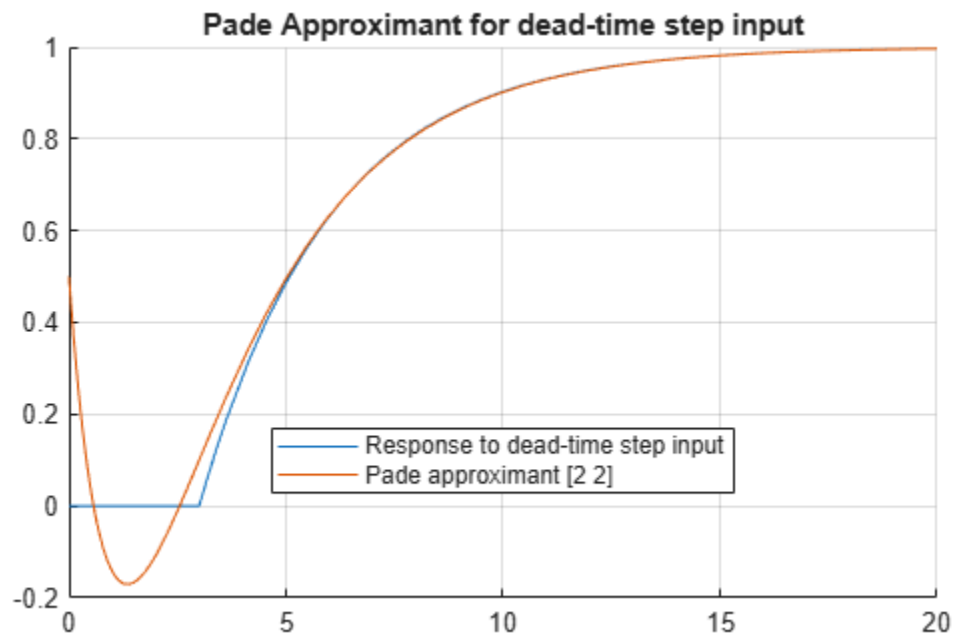
```
yPade22 = subs(yPade22,[a tau],[1 3])
```

```
yPade22 =
```

$$\frac{9}{4}e^{-s} - \frac{11}{4}e^{-\frac{s}{3}} + 1$$

Plot the response of the system  $y$  and the response calculated from the Padé approximant  $y_{\text{Pade22}}$ .

```
hold on
grid on
fplot([y yPade22],[0 20])
title('Padé Approximant for dead-time step input')
legend('Response to dead-time step input',...
       'Padé approximant [2 2]',...
       'Location', 'Best')
```



The [2 2] Padé approximant does not represent the response well because a pole exists at the expansion point of 0. To increase the accuracy of `pade` when there is a pole or zero at the expansion point, set the `OrderMode` input argument to `Relative` and repeat the steps. For details, see `pade`.

```
stepPade22Rel = pade(step, 'Order', [2 2], 'OrderMode', 'Relative')
```

```
stepPade22Rel =
```

$$\frac{3s^2 - 6s + 4}{s(3s^2 + 6s + 4)}$$

```
yPade22Rel = H(s)*stepPade22Rel
```

```
yPade22Rel =
```

$$\frac{a(3s^2 - 6s + 4)}{s(s\tau + 1)(3s^2 + 6s + 4)}$$

```
yPade22Rel = ilaplace(yPade22Rel)
```

```
yPade22Rel =
```

$$a - \frac{a e^{-\frac{t}{\tau}} (4\tau^2 + 6\tau + 3)}{\sigma_1} + \frac{12 a \tau e^{-t} \left( \cos\left(\frac{\sqrt{3}t}{3}\right) - \sqrt{3} \sin\left(\frac{\sqrt{3}t}{3}\right) \left( \frac{36a - 72a\tau}{36a\tau} + 1 \right) \right)}{\sigma_1}$$

where

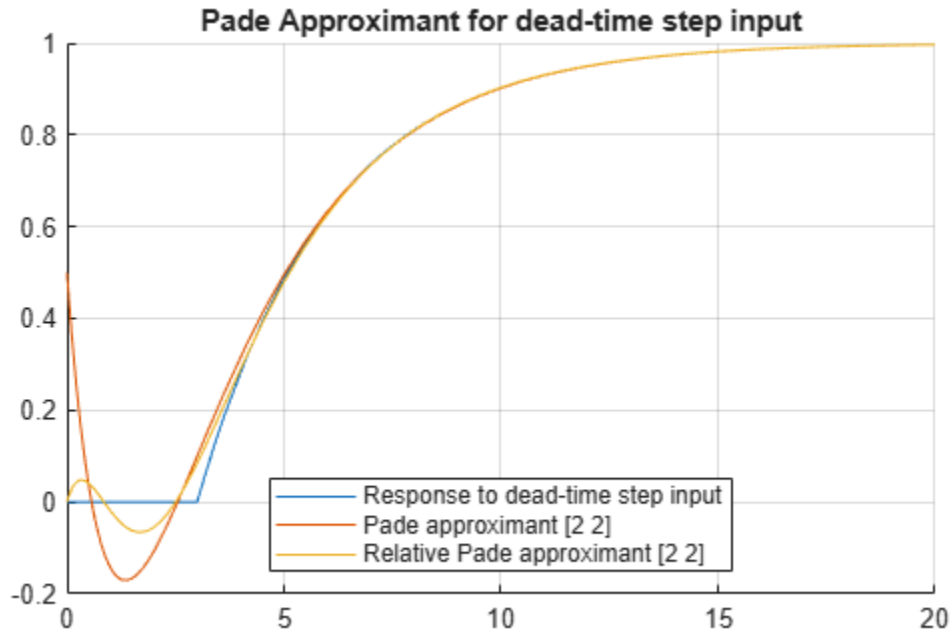
$$\sigma_1 = 4\tau^2 - 6\tau + 3$$

```
yPade22Rel = subs(yPade22Rel, [a tau], [1 3])
```

```
yPade22Rel =
```

$$\frac{12 e^{-t} \left( \cos\left(\frac{\sqrt{3}t}{3}\right) + \frac{2\sqrt{3} \sin\left(\frac{\sqrt{3}t}{3}\right)}{3} \right)}{7} - \frac{19 e^{-\frac{t}{3}}}{7} + 1$$

```
fplot(yPade22Rel, [0 20], 'DisplayName', 'Relative Pade approximant [2 2]')
```



The accuracy of the Padé approximant can also be increased by increasing its order. Increase the order to [4 5] and repeat the steps. The [n-1 n] Padé approximant is better at approximating the response at  $t = 0$  than the [n n] Padé approximant.

```
stepPade45 = pade(step, 'Order', [4 5])
```

```
stepPade45 =  


$$\frac{27s^4 - 180s^3 + 540s^2 - 840s + 560}{s(27s^4 + 180s^3 + 540s^2 + 840s + 560)}$$

```

```
yPade45 = H(s)*stepPade45
```

```
yPade45 =  


$$\frac{a(27s^4 - 180s^3 + 540s^2 - 840s + 560)}{s(s\tau + 1)(27s^4 + 180s^3 + 540s^2 + 840s + 560)}$$

```

```
yPade45 = subs(yPade45,[a tau],[1 3])
```

```
yPade45 =  


$$\frac{27s^4 - 180s^3 + 540s^2 - 840s + 560}{s(3s + 1)(27s^4 + 180s^3 + 540s^2 + 840s + 560)}$$

```

```
yPade45 = ilaplace(yPade45)
```

```
yPade45 =
```



$$\frac{101520 \left( \sum_{k=1}^4 \frac{e^{\sigma_2 t} \sigma_2}{12 (90 \sigma_2 + 45 \sigma_2^2 + 9 \sigma_2^3 + 70)} \right)}{143} - \frac{2721 e^{-\frac{t}{3}}}{1001} + \frac{172560 \left( \sum_{k=1}^4 \frac{e^{\frac{t}{\sigma_1}}}{\sigma_1} \right)}{143} + \frac{294120 \left( \sum_{k=1}^4 \frac{e^{\frac{t}{\sigma_1}} \sigma_2^2}{\sigma_1} \right)}{1001} + \frac{46440}{1001}$$

where

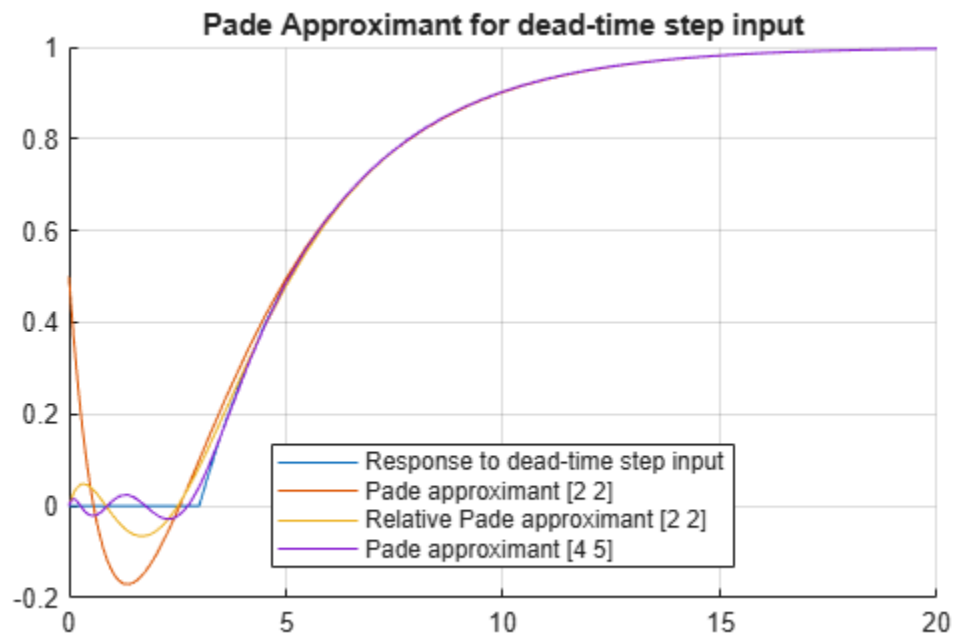
$$\sigma_1 = 12 (9 \sigma_2^3 + 45 \sigma_2^2 + 90 \sigma_2 + 70)$$

$$\sigma_2 = \text{root} \left( z^4 + \frac{20 z^3}{3} + 20 z^2 + \frac{280 z}{9} + \frac{560}{27}, z, k \right)$$

`yPade45 = vpa(yPade45)`

```
yPade45 = 3.2418384981662546679005910164486 e-1.930807068546914778929595950184 t
cos(0.57815608595633583454598214328008 t) - 2.7182817182817182817182817182817
e-0.333333333333333333333333333333333333333333333333333 t - 1.5235567798845363861823092981669
e-1.4025262647864185544037373831494 t cos(1.7716120279045018112388813990878 t)
+ 11.595342871672681856604670597166 e-1.930807068546914778929595950184 t
sin(0.57815608595633583454598214328008 t) - 1.780379837923033426855987436911
e-1.4025262647864185544037373831494 t sin(1.7716120279045018112388813990878 t) + 1.0
```

`fplot(yPade45,[0 20], 'DisplayName', 'Pade approximant [4 5]')`



The following points have been shown:

- Padé approximants can model dead-time step inputs.
- The accuracy of the Padé approximant increases with the increase in the order of the approximant.

- When a pole or zero exists at the expansion point, the Padé approximant is inaccurate about the expansion point. To increase the accuracy of the approximant, set the `OrderMode` option to `Relative`. You can also use increase the order of the denominator relative to the numerator.

## Limits

The fundamental idea in calculus is to make calculations on functions as a variable “gets close to” or approaches a certain value. Recall that the definition of the derivative is given by a limit

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h},$$

provided this limit exists. Symbolic Math Toolbox software enables you to calculate the limits of functions directly. The commands

```
syms h n x
limit((cos(x+h) - cos(x))/h, h, 0)
```

which return

```
ans =
-sin(x)
```

and

```
limit((1 + x/n)^n, n, inf)
```

which returns

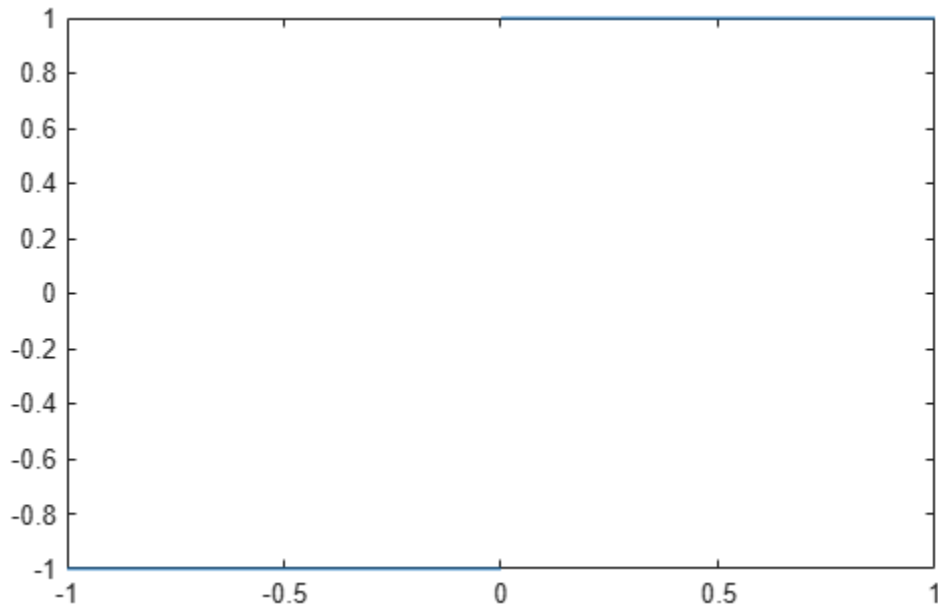
```
ans =
exp(x)
```

illustrate two of the most important limits in mathematics: the derivative (in this case of  $\cos(x)$ ) and the exponential function.

## One-Sided Limits

You can also calculate one-sided limits with Symbolic Math Toolbox software. For example, you can calculate the limit of  $x/|x|$ , whose graph is shown in the following figure, as  $x$  approaches 0 from the left or from the right.

```
syms x
fplot(x/abs(x), [-1 1], 'ShowPoles', 'off')
```



To calculate the limit as  $x$  approaches 0 from the left,

$$\lim_{x \rightarrow 0^-} \frac{x}{|x|},$$

enter

```
syms x
limit(x/abs(x), x, 0, 'left')
```

```
ans =
-1
```

To calculate the limit as  $x$  approaches 0 from the right,

$$\lim_{x \rightarrow 0^+} \frac{x}{|x|} = 1,$$

enter

```
syms x
limit(x/abs(x), x, 0, 'right')
```

```
ans =
1
```

Since the limit from the left does not equal the limit from the right, the two-sided limit does not exist. In the case of undefined limits, MATLAB returns NaN (not a number). For example,

```
syms x
limit(x/abs(x), x, 0)
```

returns

```
ans =
NaN
```

Observe that the default case, `limit(f)` is the same as `limit(f,x,0)`. Explore the options for the `limit` command in this table, where `f` is a function of the symbolic object `x`.

| Mathematical Operation          | MATLAB Command                                             |
|---------------------------------|------------------------------------------------------------|
| $\lim_{x \rightarrow 0} f(x)$   | <code>limit(f)</code>                                      |
| $\lim_{x \rightarrow a} f(x)$   | <code>limit(f, x, a)</code> or<br><code>limit(f, a)</code> |
| $\lim_{x \rightarrow a^-} f(x)$ | <code>limit(f, x, a, 'left')</code>                        |
| $\lim_{x \rightarrow a^+} f(x)$ | <code>limit(f, x, a, 'right')</code>                       |

## Find Asymptotes, Critical, and Inflection Points

This example describes how to analyze a simple function to find its asymptotes, maximum, minimum, and inflection point.

### Define a Function

The function in this example is

$$f(x) = \frac{3x^2 + 6x - 1}{x^2 + x - 3}.$$

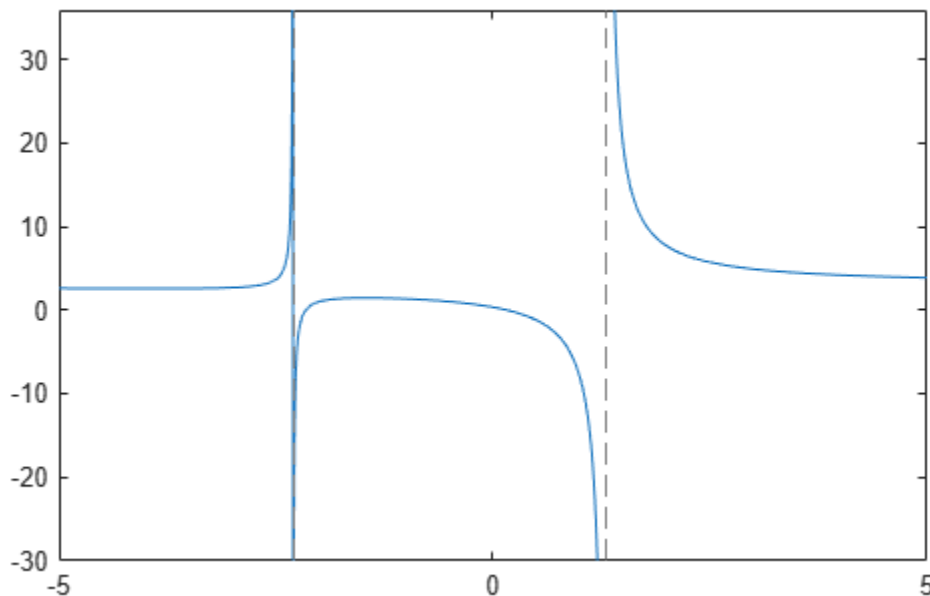
First, create the function.

```
syms x
num = 3*x^2 + 6*x - 1;
denom = x^2 + x - 3;
f = num/denom
```

$$f = \frac{3x^2 + 6x - 1}{x^2 + x - 3}$$

Plot the function by using `fplot`. The `fplot` function automatically shows vertical asymptotes.

```
fplot(f)
```



### Find Asymptotes

To find the horizontal asymptote of  $f$  mathematically, take the limit of  $f$  as  $x$  approaches positive infinity.

```
limit(f,Inf)
```

```
ans = 3
```

The limit as  $x$  approaches negative infinity is also 3. This result means the line  $y = 3$  is a horizontal asymptote to  $f$ .

To find the vertical asymptotes of  $f$ , set the denominator equal to 0 and solve it.

```
roots = solve(denom)
```

```
roots =
```

$$\begin{pmatrix} -\frac{\sqrt{13}}{2} - \frac{1}{2} \\ \frac{\sqrt{13}}{2} - \frac{1}{2} \end{pmatrix}$$

`roots` indicates that the vertical asymptotes are the lines

$$x = \frac{-1 - \sqrt{13}}{2}$$

and

$$x = \frac{-1 + \sqrt{13}}{2}.$$

### Find Maximum and Minimum

You can see from the graph that  $f$  has a local maximum between the points  $x = -2$  and  $x = 0$ . It also has a local minimum between  $x = -6$  and  $x = -2$ . To find the  $x$ -coordinates of the maximum and minimum, first take the derivative of  $f$ .

```
f1 = diff(f)
```

```
f1 =
```

$$\frac{6x+6}{x^2+x-3} - \frac{(2x+1)(3x^2+6x-1)}{(x^2+x-3)^2}$$

To simplify this expression, enter the following.

```
f1 = simplify(f1)
```

```
f1 =
```

$$-\frac{3x^2+16x+17}{(x^2+x-3)^2}$$

Next, set the derivative equal to 0 and solve for the critical points.

```
crit_pts = solve(f1)
```

```
crit_pts =
```

$$\begin{pmatrix} -\frac{\sqrt{13}}{3} - \frac{8}{3} \\ \frac{\sqrt{13}}{3} - \frac{8}{3} \end{pmatrix}$$

As the graph of  $f$  shows, the function has a local minimum at

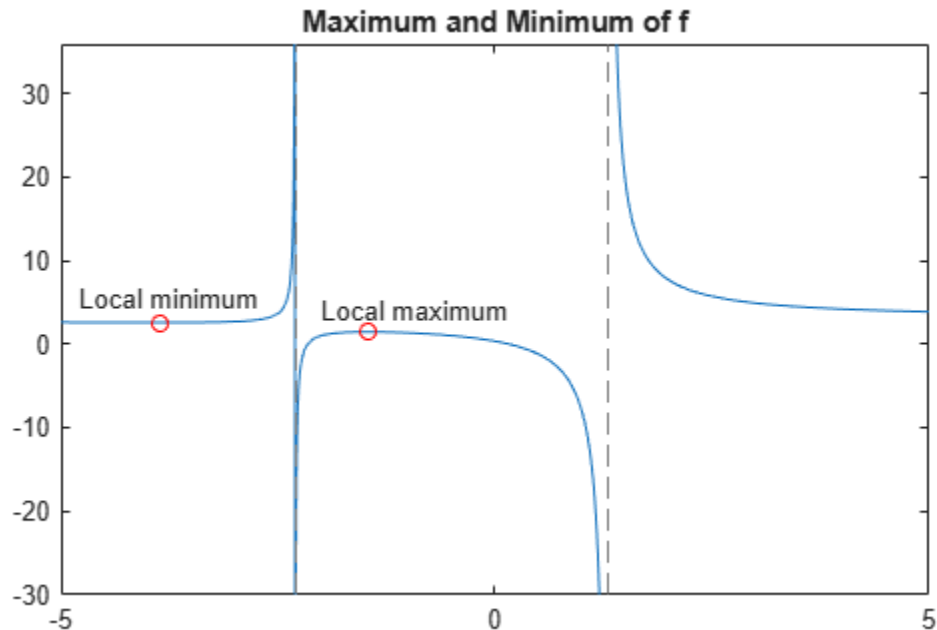
$$x_1 = \frac{-8 - \sqrt{13}}{3}$$

and a local maximum at

$$x_1 = \frac{-8 + \sqrt{13}}{3}.$$

Plot the maximum and minimum of  $f$ .

```
fplot(f)
hold on
plot(double(crit_pts), double(subs(f,crit_pts)), 'ro')
title('Maximum and Minimum of f')
text(-4.8, 5.5, 'Local minimum')
text(-2, 4, 'Local maximum')
hold off
```



### Find Inflection Point

To find the inflection point of  $f$ , set the second derivative equal to 0 and solve for this condition.

```
f2 = diff(f1);
inflec_pt = solve(f2, 'MaxDegree', 3);
double(inflec_pt)
```

```
ans = 3×1 complex
```

```
-5.2635 + 0.0000i
-1.3682 - 0.8511i
-1.3682 + 0.8511i
```



In this example, only the first element is a real number, so this is the only inflection point. MATLAB® does not always return the roots to an equation in the same order.

Instead of selecting the real root by indexing into `inter_pt`, identify the real root by determining which roots have a zero-valued imaginary part.

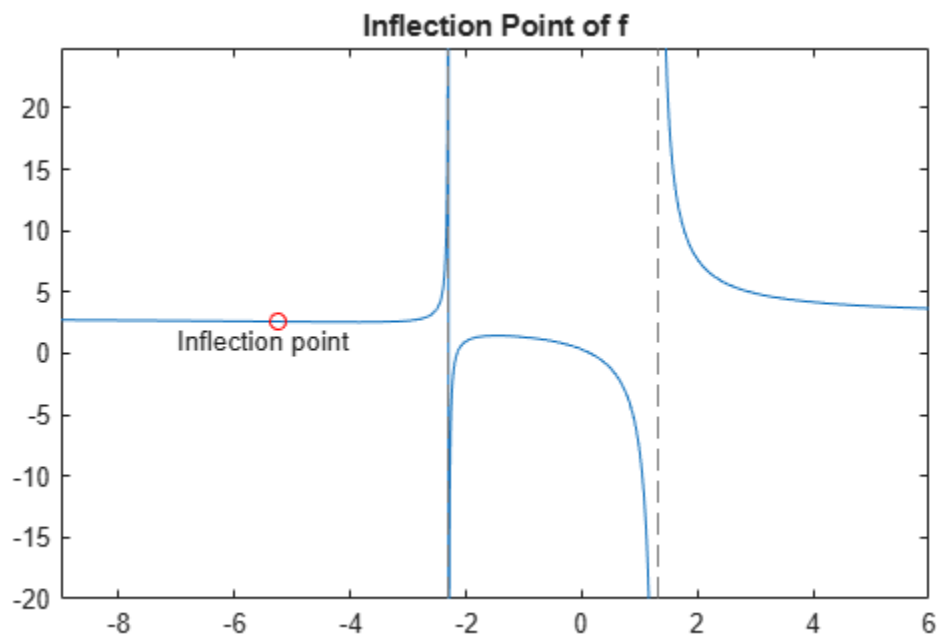
```
idx = imag(double(inflec_pt)) == 0;
inflec_pt = inflec_pt(idx)
```

```
inflec_pt =
```

$$-\frac{13}{9\left(\frac{169}{54} - \frac{\sqrt{2197}}{18}\right)^{1/3}} - \left(\frac{169}{54} - \frac{\sqrt{2197}}{18}\right)^{1/3} - \frac{8}{3}$$

Plot the inflection point. The extra argument `[-9 6]` in `fplot` extends the range of  $x$  values in the plot so that you can see the inflection point more clearly, as the figure shows.

```
fplot(f,[-9 6])
hold on
plot(double(inflec_pt), double(subs(f,inflec_pt)), 'ro')
title('Inflection Point of f')
text(-7,1,'Inflection point')
hold off
```



## Functional Derivatives Tutorial

This example shows how to use functional derivatives in the Symbolic Math Toolbox™ using the example of the wave equation. This example performs calculations symbolically to obtain analytic results. The wave equation for a string fixed at its ends is solved using functional derivatives. A functional derivative is the derivative of a functional with respect to the function that the functional depends on. The Symbolic Math Toolbox implements functional derivatives using the `functionalDerivative` function.

Solving the wave equation is one application of functional derivatives. It describes the motion of waves, from the motion of a string to the propagation of an electromagnetic wave, and is an important equation in physics. You can apply the techniques illustrated in this example to applications in the calculus of variations from solving the Brachistochrone problem to finding minimal surfaces of soap bubbles.

Consider a string of length  $L$  suspended between the two points  $x = 0$  and  $x = L$ . The string has a characteristic density per unit length and a characteristic tension. Define the length, mass density, and tension as constants for later use. For simplicity, set these constants to 1.

```
Length = 1;
Density = 1;
Tension = 1;
```

If the string is in motion, the string's kinetic and potential energy densities are a function of its displacement from rest  $S(x, t)$ , which varies with position  $x$  and time  $t$ . If  $d$  is the mass density per unit length, the kinetic energy density is

$$T = \frac{d}{2} \left( \frac{d}{dt} S(x, t) \right)^2.$$

The potential energy density is

$$V = \frac{r}{2} \left( \frac{d}{dx} S(x, t) \right)^2,$$

where  $r$  is the tension.

Enter these equations in MATLAB®. Since length must be positive, set this assumption. This assumption allows `simplify` to simplify the resulting equations into the expected form.

```
syms S(x,t) d r v L
assume(L>0)
T(x,t) = d/2*diff(S,t)^2;
V(x,t) = r/2*diff(S,x)^2;
```

The action density  $A$  is  $T - V$ . The Principle of Least Action states that action is always minimized. Determine the condition for minimum action by finding the functional derivative of  $A$  with respect to  $S$  using `functionalDerivative` and equate it to zero.

```
A = T-V;
eqn = functionalDerivative(A,S) == 0
eqn(x, t) =
```

$$r \frac{\partial^2}{\partial x^2} S(x, t) - d \frac{\partial^2}{\partial t^2} S(x, t) = 0$$

Simplify the equation using `simplify`. Convert the equation into its expected form by substituting for  $r/d$  with the square of the wave velocity  $v$ .

```
eqn = simplify(eqn)/r;
eqn = subs(eqn, r/d, v^2)
```

```
eqn(x, t) =
```

$$\frac{\frac{\partial^2}{\partial t^2} S(x, t)}{v^2} = \frac{\partial^2}{\partial x^2} S(x, t)$$

Solve the equation using the method of separation of variables. Set  $S(x, t) = U(x)*V(t)$  to separate the dependence on position  $x$  and time  $t$ . Separate both sides of the resulting equation using `children`.

```
syms U(x) V(t)
eqn2 = subs(eqn, S(x, t), U(x)*V(t));
eqn2 = eqn2/(U(x)*V(t))
```

```
eqn2(x, t) =
```

$$\frac{\frac{\partial^2}{\partial t^2} V(t)}{v^2 V(t)} = \frac{\frac{\partial^2}{\partial x^2} U(x)}{U(x)}$$

```
tmp = children(eqn2);
```

Both sides of the equation depend on different variables, yet are equal. This is only possible if each side is a constant. Equate each side to an arbitrary constant  $C$  to get two differential equations.

```
syms C
eqn3 = tmp(1) == C
```

```
eqn3 =
```

$$\frac{\frac{\partial^2}{\partial t^2} V(t)}{v^2 V(t)} = C$$

```
eqn4 = tmp(2) == C
```

```
eqn4 =
```

$$\frac{\frac{\partial^2}{\partial x^2} U(x)}{U(x)} = C$$

Solve the differential equations using `dsolve` with the condition that displacement is 0 at  $x = 0$  and  $t = 0$ . Simplify the equations to their expected form using `simplify` with the `Steps` option set to 50.

```
V(t) = dsolve(eqn3, V(0)==0, t);
U(x) = dsolve(eqn4, U(0)==0, x);
V(t) = simplify(V(t), 'Steps', 50)
```

```
V(t) = -2 C1 sinh(sqrt(C) t v)
```

```
U(x) = simplify(U(x), 'Steps', 50)
```

```
U(x) = 2 C1 sinh(√C x)
```

Obtain the constants in the equations.

```
p1 = setdiff(symvar(U(x)), sym([C, x]))
```

```
p1 = C1
```

```
p2 = setdiff(symvar(V(t)), sym([C, v, t]))
```

```
p2 = C1
```

The string is fixed at the positions  $x = 0$  and  $x = L$ . The condition  $U(0) = 0$  already exists. Apply the boundary condition that  $U(L) = 0$  and solve for  $C$ .

```
eqn_bc = U(L) == 0;
[solC, param, cond] = solve(eqn_bc, C, 'ReturnConditions', true)
```

```
solC =

$$-\frac{k^2 \pi^2}{L^2}$$

```

```
param = k
```

```
cond = k ∈ ℤ ∧ C1 ≠ 0 ∧ 1 ≤ k
```

```
assume(cond)
```

The solution  $S(x, t)$  is the product of  $U(x)$  and  $V(t)$ . Find the solution, and substitute the characteristic values of the string into the solution to obtain the final form of the solution.

```
S(x, t) = U(x)*V(t);
S = subs(S, C, solC);
S = subs(S, [L v], [Length sqrt(Tension/Density)]);
```

The parameters  $p1$  and  $p2$  determine the amplitude of the vibrations. Set  $p1$  and  $p2$  to 1 for simplicity.

```
S = subs(S, [p1 p2], [1 1]);
S = simplify(S, 'Steps', 50)
```

```
S(x, t) = 4 sin(π k t) sin(π k x)
```

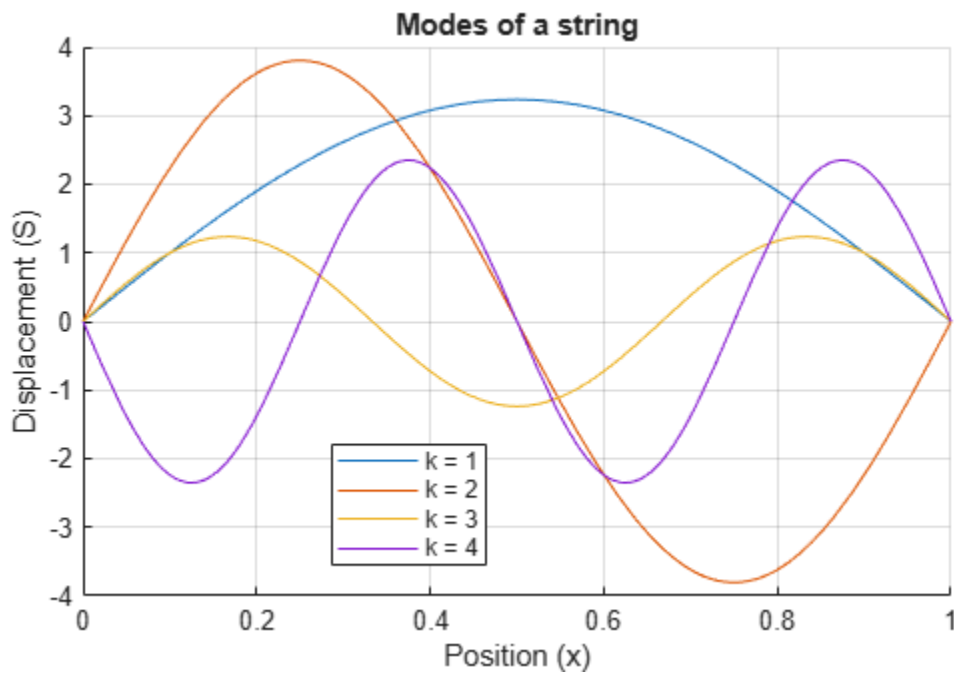
The string has different modes of vibration for different values of  $k$ . Plot the first four modes for an arbitrary value of time  $t$ . Use the `param` argument returned by `solve` to address parameter  $k$ .

```
Splot(x) = S(x, 0.3);
figure(1)
hold on
grid on
ymin = double(coeffs(Splot));
for i = 1:4
    yplot = subs(Splot, param, i);
    fplot(yplot, [0 Length])
end
ylim([-ymin ymin])
```

```

legend('k = 1','k = 2','k = 3','k = 4','Location','best')
xlabel('Position (x)')
ylabel('Displacement (S)')
title('Modes of a string')

```



The wave equation is linear. This means that any linear combination of the allowed modes is a valid solution to the wave equation. Hence, the full solution to the wave equation with the given boundary conditions and initial values is a sum over allowed modes

$$F(x, t) = \sum_{k=n}^m A_k \sin(\pi k t) \sin(\pi k x),$$

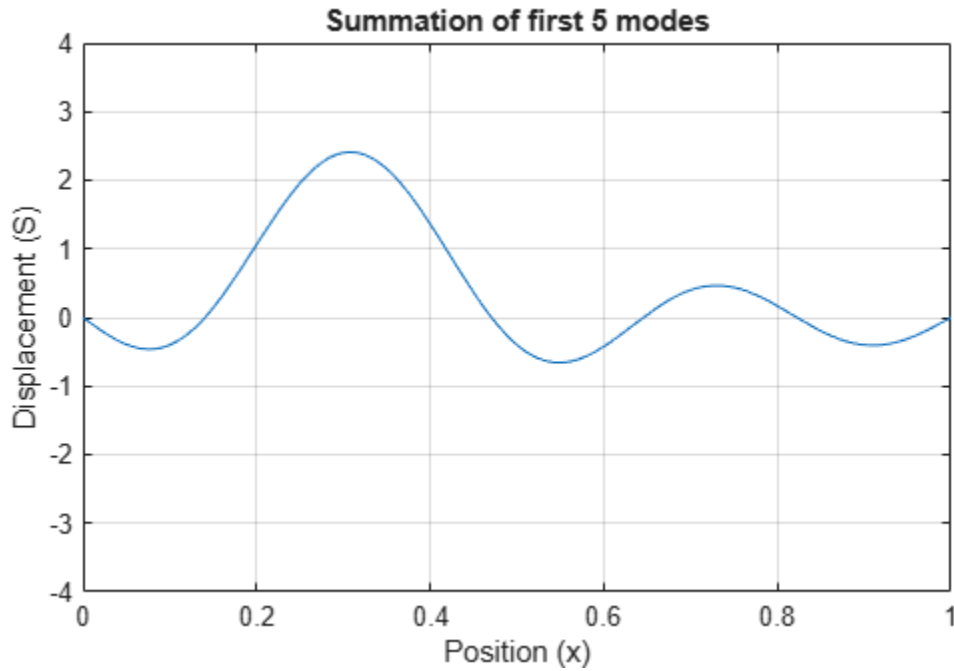
where  $A_k$  denotes arbitrary constants.

Use `symsum` to sum the first five modes of the string. On a new figure, display the resulting waveform at the same instant of time as the previous waveforms for comparison.

```

figure(2)
S5(x) = 1/5*symsum(S,param,1,5);
fplot(subs(S5,t,0.3),[0 Length])
ylim([-ymin ymin])
grid on
xlabel('Position (x)')
ylabel('Displacement (S)')
title('Summation of first 5 modes')

```



The figure shows that summing modes allows you to model a qualitatively different waveform. Here, we specified the initial condition is  $S(x, t = 0) = 0$  for all  $x$ .

You can calculate the values  $A_k$  in the equation  $F(x, t) = \sum_{k=1}^m A_k \sin(\pi k t) \sin(\pi k x)$  by specifying a condition for initial velocity

$$u_t(x, t = 0) = F_t(x, 0).$$

The appropriate summation of modes can represent any waveform, which is the same as using the Fourier series to represent the string's motion.

## Learn Calculus in the Live Editor

Learn calculus and applied mathematics using the Symbolic Math Toolbox™. The example shows introductory functions `fplot` and `diff`.

To manipulate a symbolic variable, create an object of type `syms`.

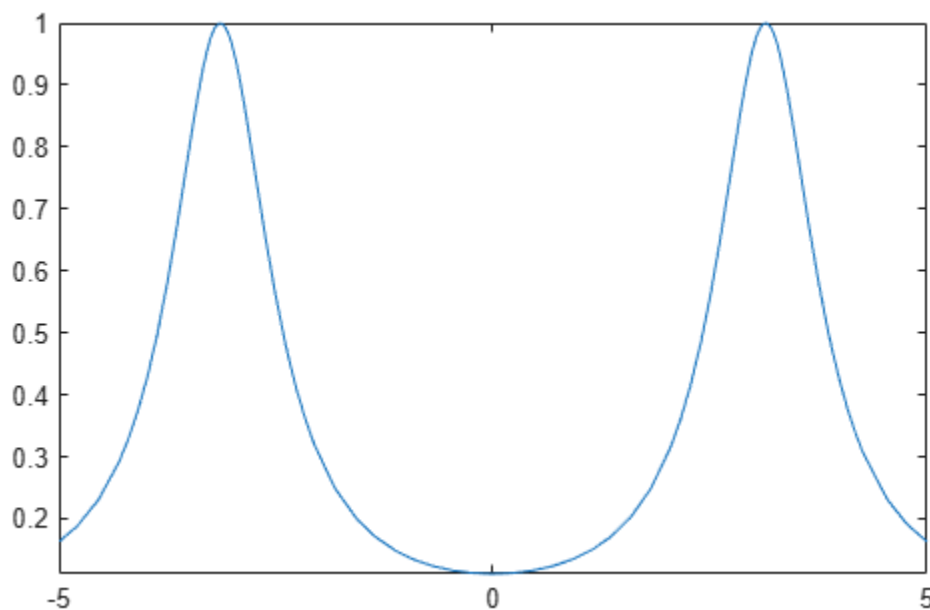
`syms` [x](#)

Once a symbolic variable is defined, you can build and visualize functions with `fplot`.

`f(x) = 1/(5+4*cos(x))`

$$f(x) = \frac{1}{4 \cos(x) + 5}$$

`fplot(f)`



Evaluate the function at  $x = \pi/2$  using math notation.

`f(pi/2)`

$$\text{ans} = \frac{1}{5}$$

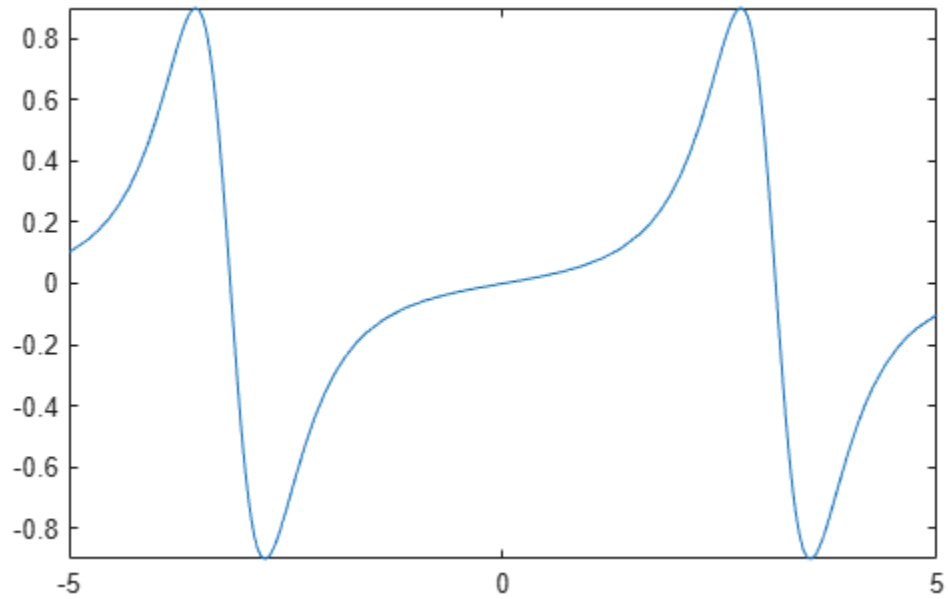
Many functions can work with symbolic variables. For example, `diff` differentiates a function.

`f1 = diff(f)`

$$f1(x) =$$

$$\frac{4 \sin(x)}{(4 \cos(x) + 5)^2}$$

`fplot(f1)`



`diff` can also find the  $N^{th}$  derivative. Here is the second derivative.

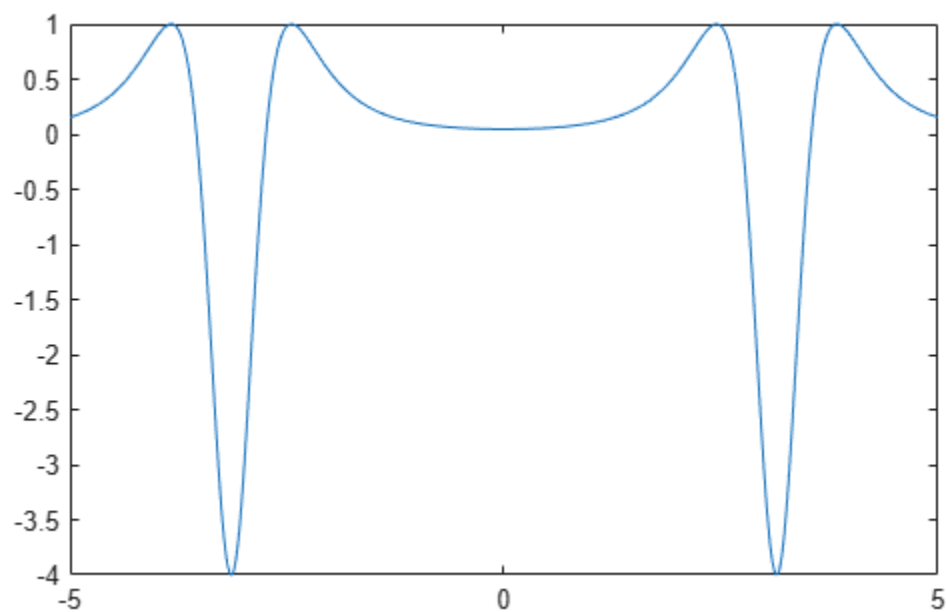
`f2 = diff(f,2)`

$f2(x) =$

$$\frac{4 \cos(x)}{(4 \cos(x) + 5)^2} + \frac{32 \sin(x)^2}{(4 \cos(x) + 5)^3}$$

`fplot(f2)`



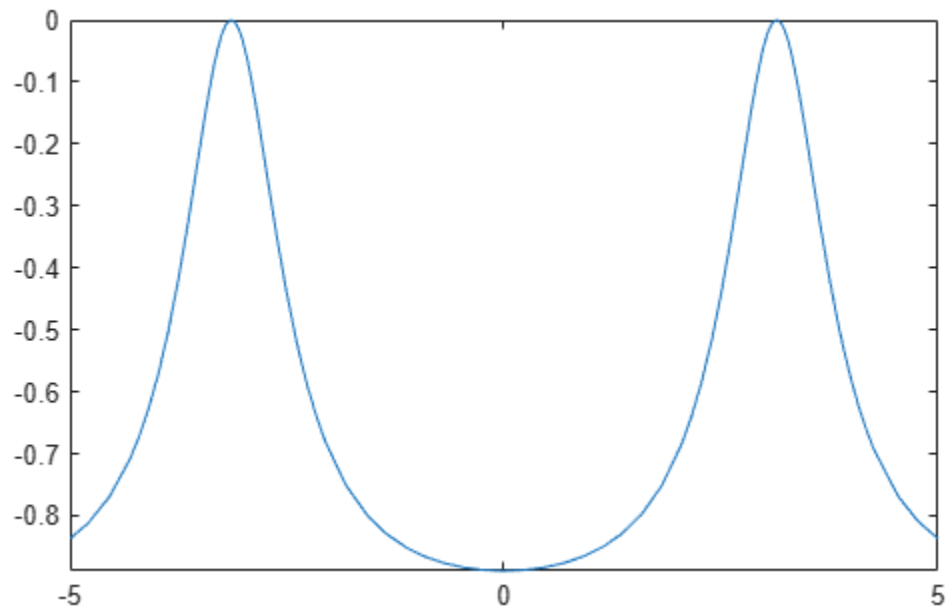


`int` integrates functions of symbolic variables. The following is an attempt to retrieve the original function by integrating the second derivative twice.

```
g = int(int(f2))
```

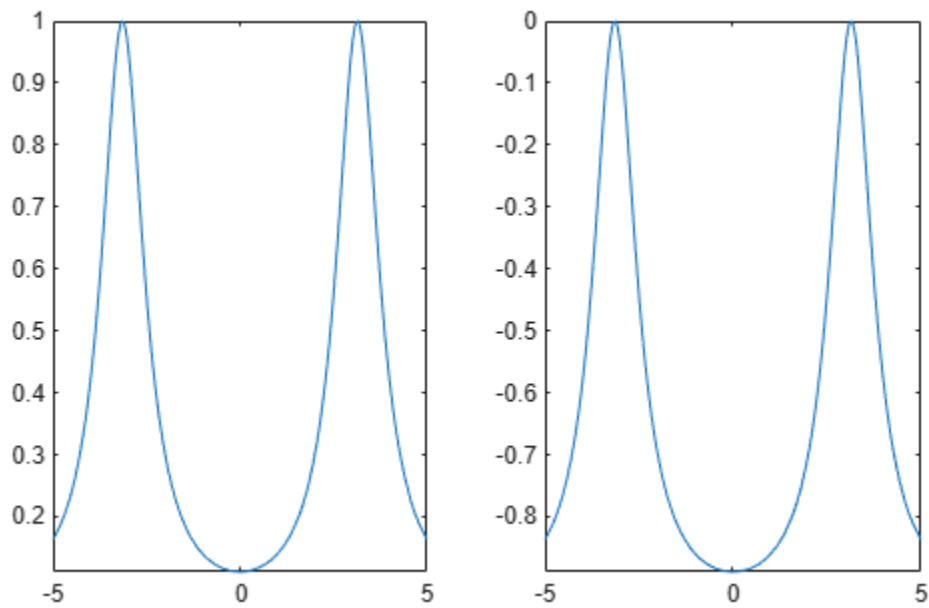
$$g(x) = -\frac{8}{\tan\left(\frac{x}{2}\right)^2 + 9}$$

```
fplot(g)
```



At first glance, the plots for  $f$  and  $g$  look the same. Look carefully, however, at their formulas and their ranges on the y-axis.

```
subplot(1,2,1)
fplot(f)
subplot(1,2,2)
fplot(g)
```



$e$  is the difference between  $f$  and  $g$ . It has a complicated formula, but its graph looks like a constant.

$$e = f - g$$

$$e(x) = \frac{8}{\tan\left(\frac{x}{2}\right)^2 + 9} + \frac{1}{4 \cos(x) + 5}$$

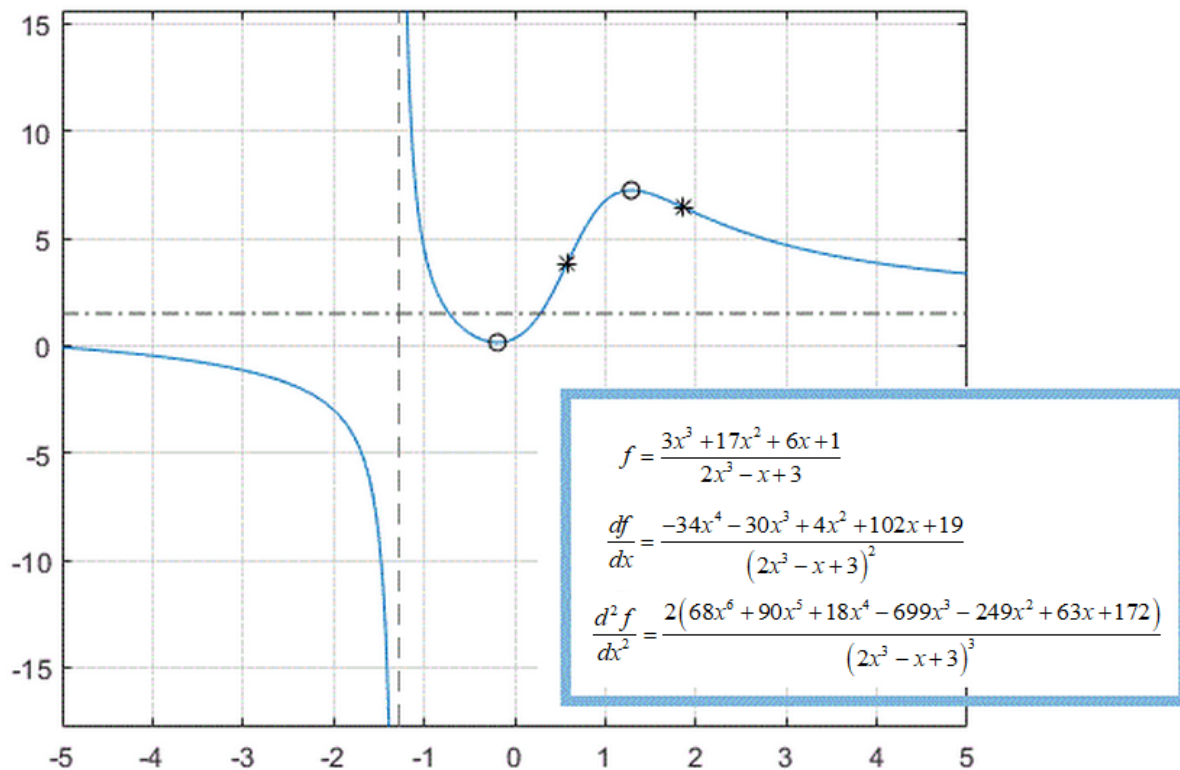
To show that the difference really is a constant, simplify the equation. This confirms that the difference between them really is a constant.

$$e = \text{simplify}(e)$$

$$e(x) = 1$$

## Differentiation

This example shows how to analytically find and evaluate derivatives using Symbolic Math Toolbox™. In the example you will find the 1st and 2nd derivative of  $f(x)$  and use these derivatives to find local maxima, minima and inflection points.



### Find Local Minima and Maxima Using First Derivatives

Computing the first derivative of an expression helps you find local minima and maxima of that expression. Before creating a symbolic expression, create symbolic variables.

`syms x`

By default, solutions that include imaginary components are included in the results. Here, consider only real values of  $x$  by setting the assumption that  $x$  is real.

`assume(x, "real")`

As an example, create a rational expression (such as a fraction where the numerator and denominator are polynomial expressions).

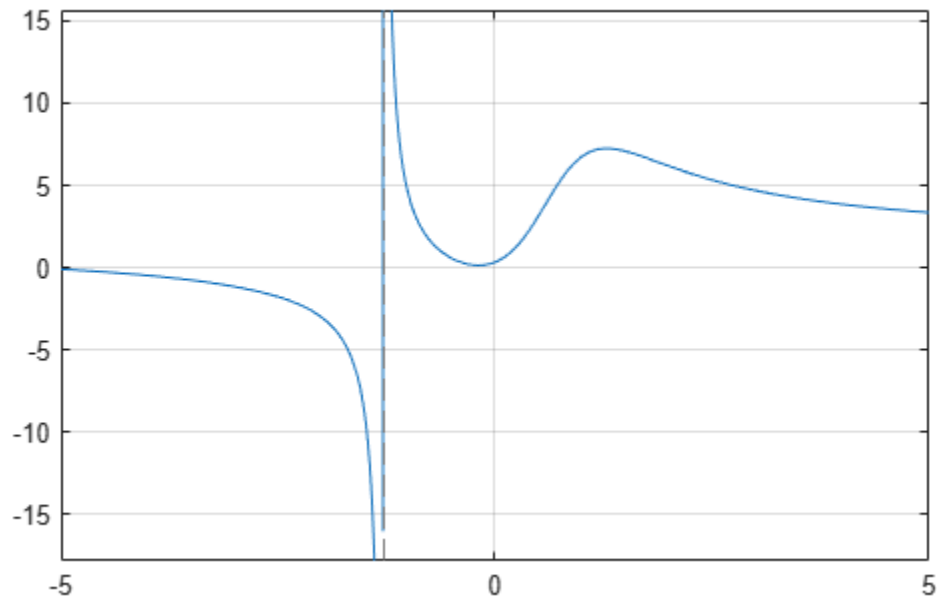
`f = (3*x^3 + 17*x^2 + 6*x + 1)/(2*x^3 - x + 3)`

`f =`  

$$\frac{3x^3 + 17x^2 + 6x + 1}{2x^3 - x + 3}$$

Plotting this expression shows that the expression has horizontal and vertical asymptotes, a local minimum between -1 and 0, and a local maximum between 1 and 2.

```
fplot(f)
grid
```



To find the horizontal asymptote, compute the limits of  $f$  for  $x$  approaching positive and negative infinities. The horizontal asymptote is  $y = 3/2$ .

```
lim_left = limit(f,x,-Inf)
```

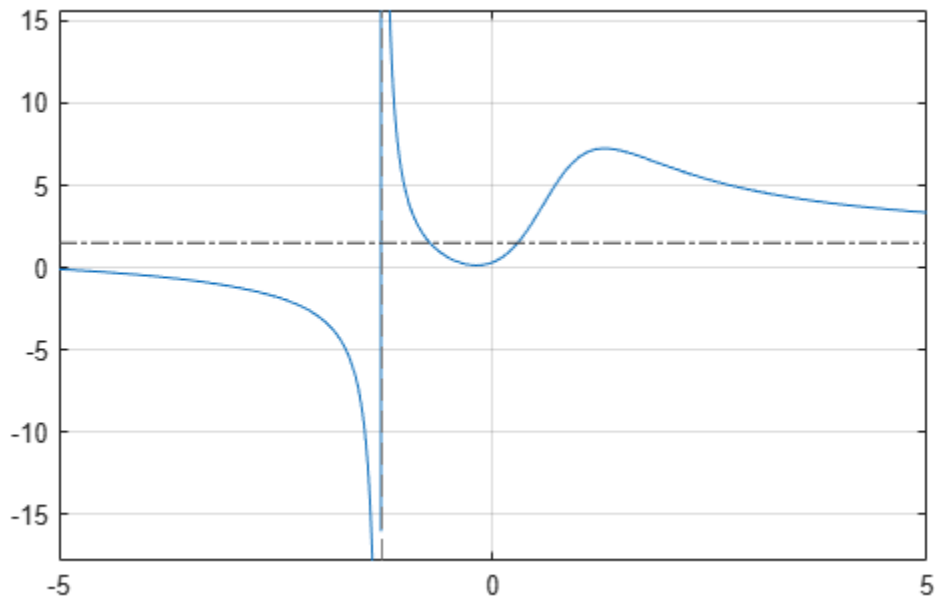
```
lim_left =  
    3  
    2
```

```
lim_right = limit(f,x,Inf)
```

```
lim_right =  
    3  
    2
```

Add this horizontal asymptote to the plot.

```
hold on  
plot(xlim,[lim_right lim_right],LineStyle="-.",Color=[0.25 0.25 0.25])
```



To find the vertical asymptote of  $f$ , find the poles of  $f$ .

```
pole_pos = poles(f,x)
```

```
pole_pos =
```

$$-\frac{1}{6\left(\frac{3}{4}-\frac{\sqrt{241}\sqrt{432}}{432}\right)^{1/3}}-\left(\frac{3}{4}-\frac{\sqrt{241}\sqrt{432}}{432}\right)^{1/3}$$

Approximate the exact solution numerically by using the `double` function.

```
double(pole_pos)
```

```
ans =  
-1.2896
```

Now find the local minimum and maximum of  $f$ . If a point is a local extremum (either minimum or maximum), the first derivative of the expression at that point is equal to zero. Compute the derivative of  $f$  using `diff`.

```
g = diff(f,x)
```

```
g =
```

$$\frac{9x^2+34x+6}{2x^3-x+3}-\frac{(6x^2-1)(3x^3+17x^2+6x+1)}{(2x^3-x+3)^2}$$

To find the local extrema of  $f$ , solve the equation  $g == 0$ .

```
g0 = solve(g,x)
```

```
g0 =
```

$$\left( \frac{\sqrt{\sigma_2}}{6 \sigma_3^{1/6}} - \sigma_1 - \frac{15}{68} \right)$$

$$\left( \frac{\sqrt{\sigma_2}}{6 \sigma_3^{1/6}} + \sigma_1 - \frac{15}{68} \right)$$

where

$$\sigma_1 = \frac{\sqrt{\frac{337491 \sqrt{6} \sqrt{\frac{3 \sqrt{3} \sqrt{178939632355}}{9826} + \frac{2198209}{9826}}}{39304} + \frac{2841 \sigma_3^{1/3} \sqrt{\sigma_2}}{578} - 9 \sigma_3^{2/3} \sqrt{\sigma_2} - \frac{361 \sqrt{\sigma_2}}{289}}{6 \sigma_3^{1/6} \sigma_2^{1/4}}$$

$$\sigma_2 = \frac{2841 \sigma_3^{1/3}}{1156} + 9 \sigma_3^{2/3} + \frac{361}{289}$$

$$\sigma_3 = \frac{\sqrt{3} \sqrt{178939632355}}{176868} + \frac{2198209}{530604}$$

Approximate the exact solution numerically by using the `double` function.

```
double(g0)
```

```
ans = 2×1
```

```
-0.1892  
1.2860
```

The expression `f` has a local maximum at `x = 1.286` and a local minimum at `x = -0.189`. Obtain the function values at these points using `subs`.

```
f0 = subs(f,x,g0)
```

```
f0 =
```

$$\begin{pmatrix} \frac{3\sigma_2 - 17\left(\sigma_5 - \sigma_6 + \frac{15}{68}\right)^2 - \sigma_4 + \sigma_1 + \frac{11}{34}}{\sigma_6 + 2\sigma_2 - \sigma_5 - \frac{219}{68}} \\ - \frac{\sigma_4 + 17\left(\sigma_6 + \sigma_5 - \frac{15}{68}\right)^2 + 3\sigma_3 + \sigma_1 - \frac{11}{34}}{\sigma_6 - 2\sigma_3 + \sigma_5 - \frac{219}{68}} \end{pmatrix}$$

where

$$\sigma_1 = \frac{\sigma_7}{\sigma_9^{1/6} \sigma_8^{1/4}}$$

$$\sigma_2 = \left(\sigma_5 - \sigma_6 + \frac{15}{68}\right)^3$$

$$\sigma_3 = \left(\sigma_6 + \sigma_5 - \frac{15}{68}\right)^3$$

$$\sigma_4 = \frac{\sqrt{\sigma_8}}{\sigma_9^{1/6}}$$

$$\sigma_5 = \frac{\sigma_7}{6\sigma_9^{1/6} \sigma_8^{1/4}}$$

$$\sigma_6 = \frac{\sqrt{\sigma_8}}{6\sigma_9^{1/6}}$$

$$\sigma_7 = \sqrt{\frac{337491\sqrt{6}\sqrt{\frac{3\sqrt{3}\sqrt{178939632355}}{9826} + \frac{2198209}{9826}}}{39304} + \frac{2841\sigma_9^{1/3}\sqrt{\sigma_8}}{578} - 9\sigma_9^{2/3}\sqrt{\sigma_8} - \frac{361\sqrt{\sigma_8}}{289}}$$

$$\sigma_8 = \frac{2841\sigma_9^{1/3}}{1156} + 9\sigma_9^{2/3} + \frac{361}{289}$$

$$\sigma_9 = \frac{\sqrt{3}\sqrt{178939632355}}{176868} + \frac{2198209}{530604}$$

Approximate the exact solution numerically by using the `double` function on the variable `f0`.

```
double(f0)
```

```
ans = 2×1
```

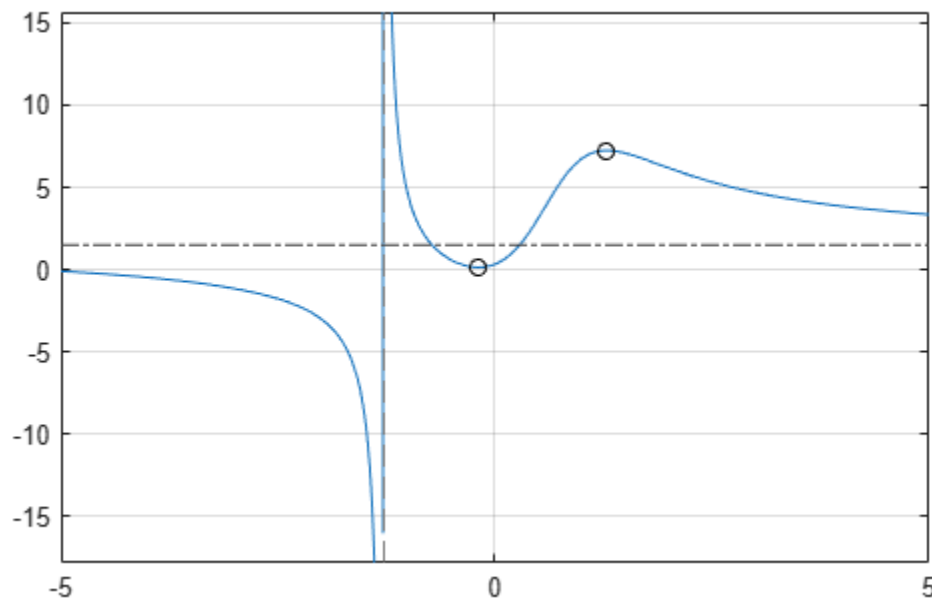
```
0.1427
```

```
7.2410
```

Add point markers to the graph at the extrema.

```
plot(g0,f0,"ok")
```





### Find Inflection Points Using Second Derivatives

Computing the second derivative lets you find inflection points of the expression. The most efficient way to compute second or higher-order derivatives is to use the parameter that specifies the order of the derivative.

```
h = diff(f,x,2)
```

h =

$$\frac{18x + 34}{\sigma_1} - \frac{2(6x^2 - 1)(9x^2 + 34x + 6)}{\sigma_1^2} - \frac{12x\sigma_2}{\sigma_1^2} + \frac{2(6x^2 - 1)^2\sigma_2}{\sigma_1^3}$$

where

$$\sigma_1 = 2x^3 - x + 3$$

$$\sigma_2 = 3x^3 + 17x^2 + 6x + 1$$

Now simplify that result.

```
h = simplify(h)
```

h =

$$\frac{2(68x^6 + 90x^5 + 18x^4 - 699x^3 - 249x^2 + 63x + 172)}{(2x^3 - x + 3)^3}$$

To find inflection points of  $f$ , solve the equation  $h = 0$ . Here, use the numeric solver `vpasolve` to calculate floating-point approximations of the solutions.

```
h0 = vpasolve(h,x)
```

h0 =

```

0.57871842655441748319601085860196
1.8651543689917122385037075917613
-1.4228127856020972275345064554049 - 1.8180342567480118987898749770461 i
-1.4228127856020972275345064554049 + 1.8180342567480118987898749770461 i
-0.46088831805332057449182335801198 + 0.47672261854520359440077796751805 i
-0.46088831805332057449182335801198 - 0.47672261854520359440077796751805 i

```

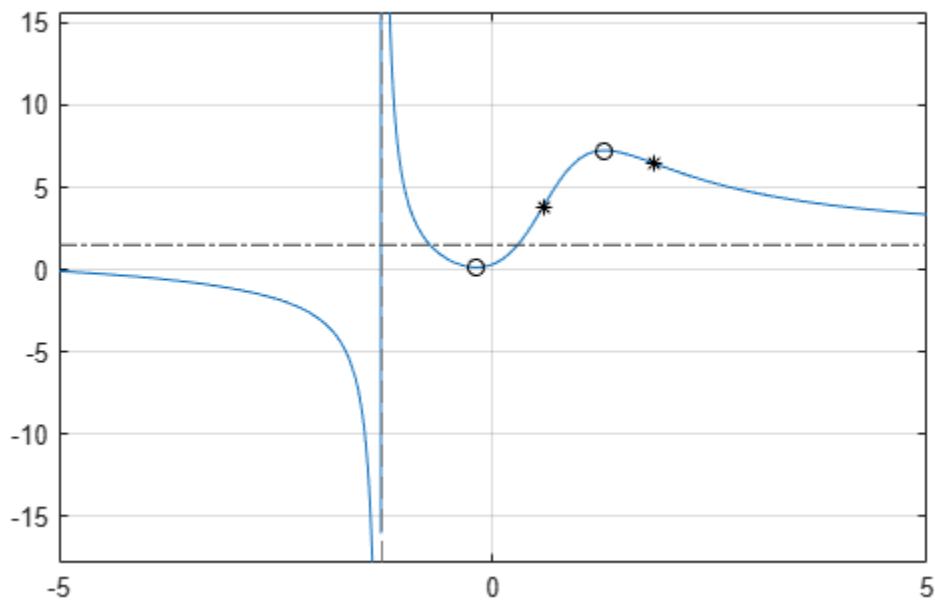
The expression `f` has two inflection points: `x = 1.865` and `x = 0.579`. Note that `vpasolve` also returns complex solutions. Exclude the complex solutions.

```
h0(imag(h0)~=0) = []
```

```
h0 =
(0.57871842655441748319601085860196)
(1.8651543689917122385037075917613)
```

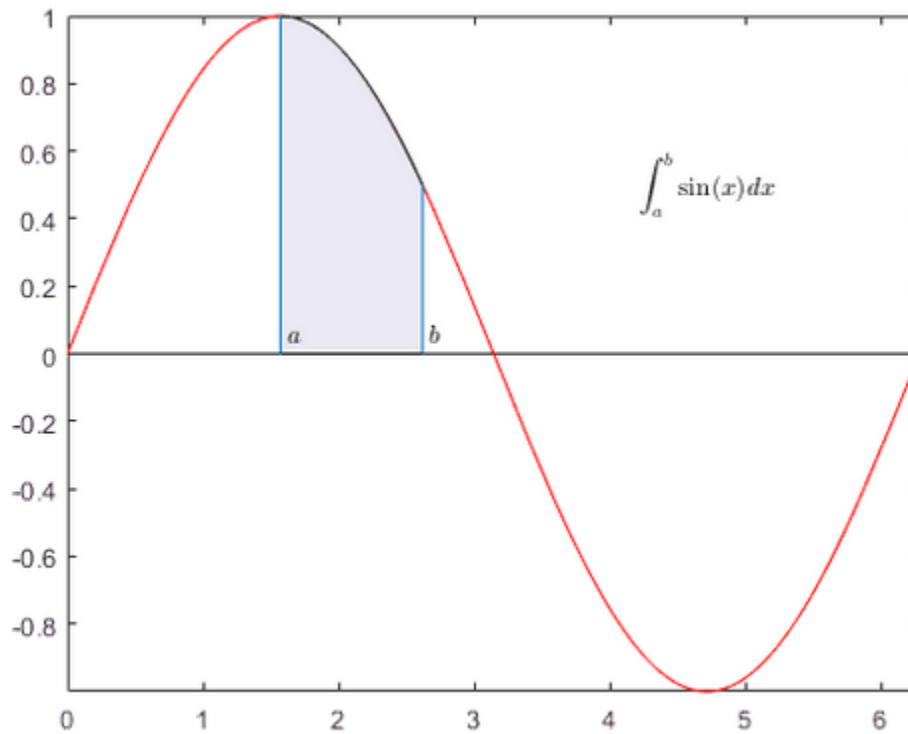
Add markers to the plot showing the inflection points:

```
plot(h0,subs(f,x,h0),'*k')
hold off
```



# Integration

This example shows how to compute definite integrals using Symbolic Math Toolbox™.



## Definite Integral

Show that the definite integral  $\int_a^b f(x) dx$  for  $f(x) = \sin(x)$  on  $\left[\frac{\pi}{2}, \frac{3\pi}{2}\right]$  is 0.

```
syms x
int(sin(x),pi/2,3*pi/2)
```

```
ans = 0
```

## Definite Integrals in Maxima and Minima

To maximize  $F(a) = \int_a^a \sin(ax) \sin(x/a) dx$  for  $a \geq 0$ , first, define the symbolic variables and assume that  $a \geq 0$ .

```
syms a x
assume(a >= 0);
```

Then, define the function to maximize.

```
F = int(sin(a*x)*sin(x/a),x,-a,a)
```

```
F =
```

$$\begin{cases} 1 - \frac{\sin(2)}{2} & \text{if } a = 1 \\ \frac{2a(\sin(a^2)\cos(1) - a^2\cos(a^2)\sin(1))}{a^4 - 1} & \text{if } a \neq 1 \end{cases}$$

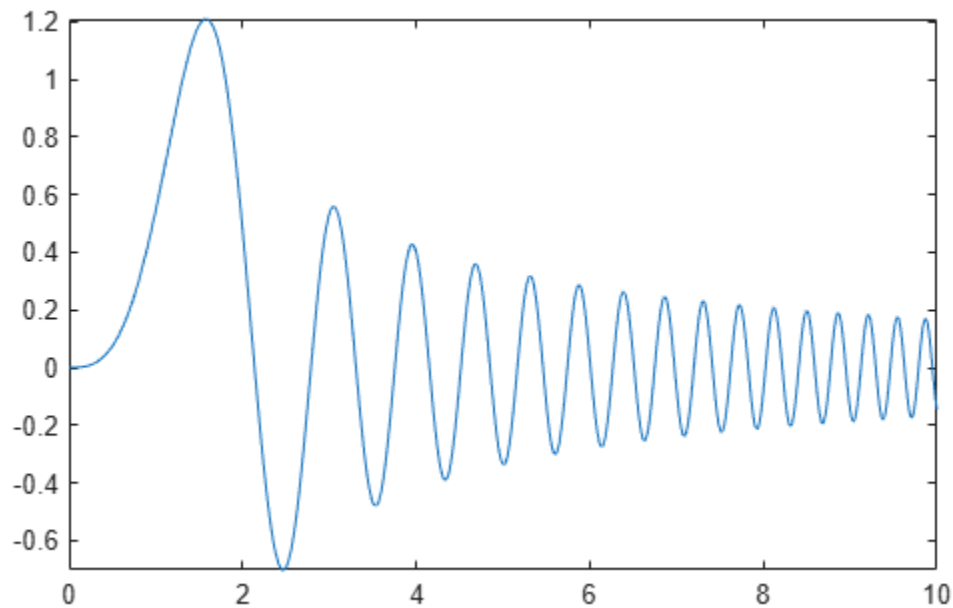
Note the special case here for  $a = 1$ . To make computations easier, use `assumeAlso` to ignore this case.

```
assumeAlso(a ~= 1);
F = int(sin(a*x)*sin(x/a), x, -a, a)
```

$$F = \frac{2a(\sin(a^2)\cos(1) - a^2\cos(a^2)\sin(1))}{a^4 - 1}$$

Create a plot of  $F$  to check its shape.

```
fplot(F,[0 10])
```



Use `diff` to find the derivative of  $F$  with respect to  $a$ .

```
Fa = diff(F,a)
```

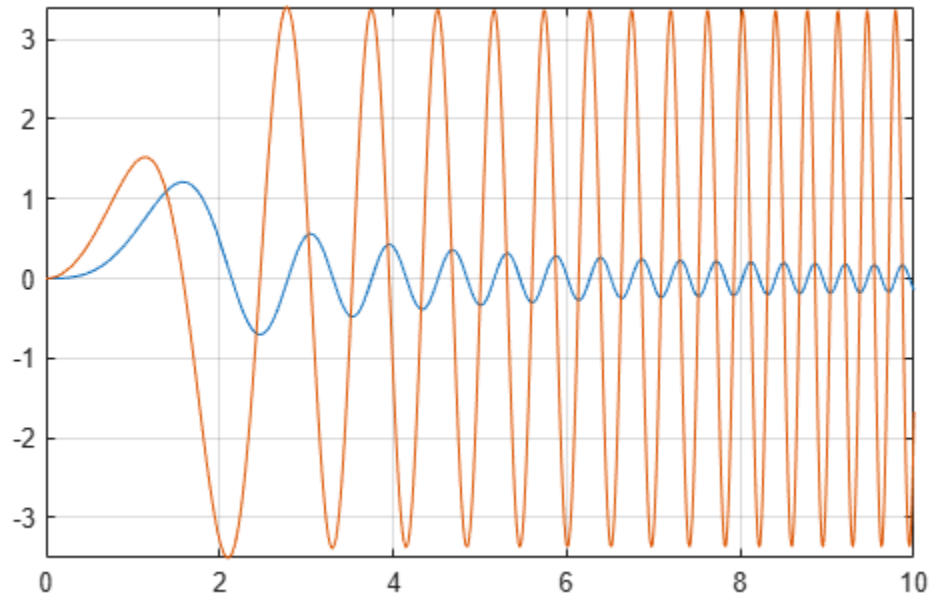
$$Fa = \frac{2\sigma_1}{a^4 - 1} + \frac{2a(2a\cos(a^2)\cos(1) - 2a\cos(a^2)\sin(1) + 2a^3\sin(a^2)\sin(1))}{a^4 - 1} - \frac{8a^4\sigma_1}{(a^4 - 1)^2}$$

where

$$\sigma_1 = \sin(a^2)\cos(1) - a^2\cos(a^2)\sin(1)$$

The zeros of  $Fa$  are the local extrema of  $F$ .

```
hold on
fplot(Fa,[0 10])
grid on
```



The maximum is between 1 and 2. Use `vpasolve` to find an approximation of the zero of  $Fa$  in this interval.

```
a_max = vpasolve(Fa,a,[1,2])
a_max = 1.5782881585233198075558845180583
```

Use `subs` to get the maximal value of the integral.

```
F_max = subs(F,a,a_max)
F_max = 0.36730152527504169588661811770092 cos(1) + 1.2020566879911789986062956284113 sin(1)
```

The result still contains exact numbers  $\sin(1)$  and  $\cos(1)$ . Use `vpa` to replace these by numerical approximations.

```
vpa(F_max)
ans = 1.2099496860938456039155811226054
```

Check that the excluded case  $a = 1$  does not result in a larger value.

```
vpa(int(sin(x)*sin(x),x,-1,1))
ans = 0.54535128658715915230199006704413
```

### Multiple Integration

Numerical integration over higher dimensional areas has special functions.

```
integral2(@(x,y) x.^2-y.^2,0,1,0,1)
```

```
ans =
4.0127e-19
```

There are no such special functions for higher-dimensional symbolic integration. Use nested one-dimensional integrals instead.

```
syms x y
int(int(x^2-y^2,y,0,1),x,0,1)

ans = 0
```

### Line Integrals

Define a vector field  $F$  in 3D space.

```
syms x y z
F(x,y,z) = [x^2*y*z, x*y, 2*y*z];
```

Next, define a curve.

```
syms t
ux(t) = sin(t);
uy(t) = t^2-t;
uz(t) = t;
```

The line integral of  $F$  along the curve  $u$  is defined as  $\int f \cdot du = \int f(ux(t), uy(t), uz(t)) \cdot \frac{du}{dt} dt$ , where the  $\cdot$  on the right-hand-side denotes a scalar product.

Use this definition to compute the line integral for  $t$  from  $[0, 1]$ .

```
F_int = int(F(ux,uy,uz)*diff([ux;uy;uz],t),t,0,1)

F_int =
    19*cos(1)/4 - cos(3)/108 - 12*sin(1) + sin(3)/27 + 395/54
```

Get a numerical approximation of this exact result.

```
vpa(F_int)

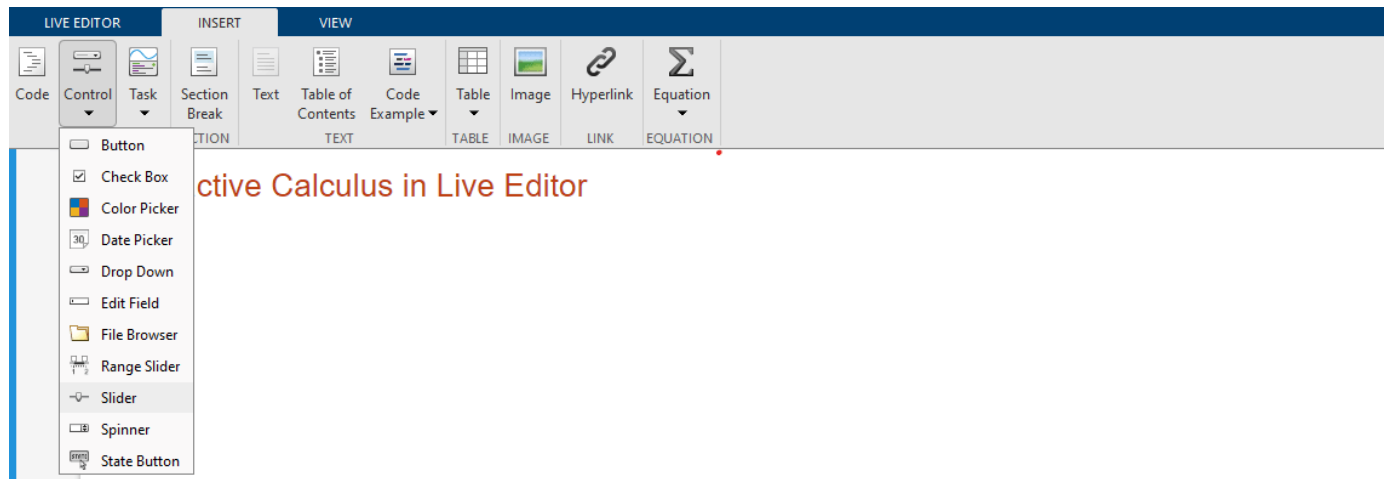
ans = -0.20200778585035447453044423341349
```

# Interactive Calculus in Live Editor

This example shows how you can add interactive controls to solve a calculus problem in a live script.

## Adding Interactive Controls to Your Script

An interactive control can be used to change the values of variables in your live script. To add a numeric slider, go to the **Insert** tab, click the **Control** button, and select **Slider**. For more information, see “Add Interactive Controls to a Live Script”.



## Initialize Variables and Function

Evaluate the integral

$$\int_0^{x_{\text{Max}}} cx^2 dx$$

using Riemann sum approximation.

A Riemann sum is a numerical approximation of the analytical integration by a finite sum of rectangular areas. Use the interactive slider bars to set the upper bound of the integral, the number of rectangles, and the constant factor of the function.

```
syms x;
xMax = 4  ;
numRectangles = 30  ;
c = 2.5  ;
f(x) = c*x^2;
yMax = double(f(xMax));
```

## Visualize the Area Under the Curve Using Riemann Sums

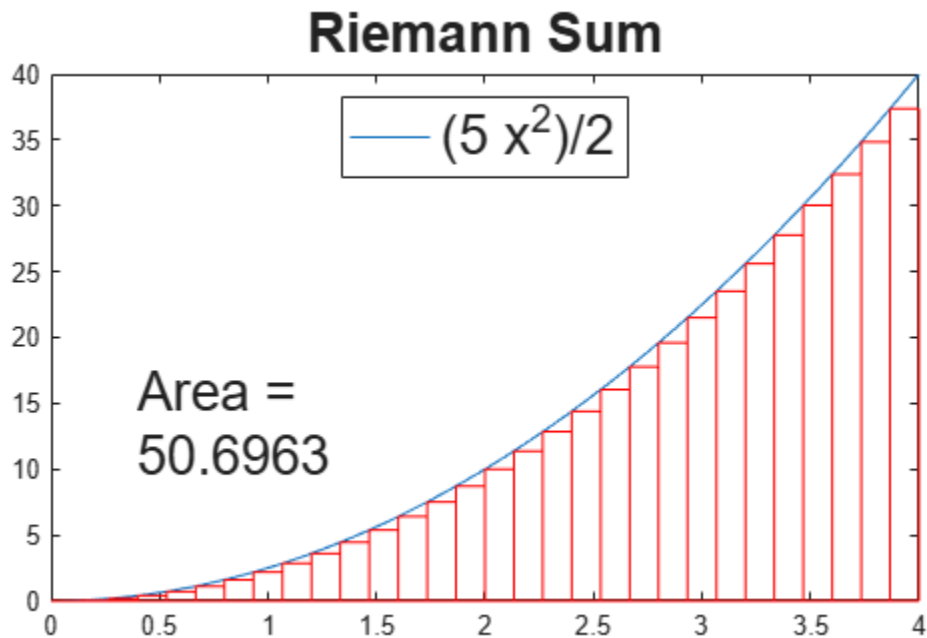
Plot the integrand f.

```
fplot(f);
xlim([0 xMax]); ylim([0 yMax]);
```

```
legend({},Location="north",FontSize=20);
title("Riemann Sum",FontSize=20);
```

Calculate the rectangular areas that approximate the area under the curve of the integral. Plot the rectangles.

```
width = xMax/numRectangles;
sum = 0;
for i = 0:numRectangles-1
    xval = i*width;
    height = double(f(xval));
    rectangle(Position=[xval 0 width height],EdgeColor="r");
    sum = sum + width*height;
end
text(xMax/10,yMax/3,["Area = " num2str(sum)],FontSize=20);
```



### Calculate the Integral Analytically

Calculate the integral analytically. Use `vpa` to numerically approximate the exact symbolic result to 32 significant digits.

```
fInt = int(f,0,xMax)
```

```
fInt =
    160
     3
```

```
vpa(fInt)
```

```
ans = 53.333333333333333333333333333333
```



## Maxima, Minima, and Inflection Points

This demonstration shows how to find extrema of functions using analytical and numerical techniques using the Symbolic Math Toolbox™.

- First Derivatives: Finding Local Minimum and Maximum of the Function
- Second Derivatives: Finding Inflection Points of the Function
- Limits: Functions with Suprema

### First Derivatives: Finding Local Minima and Maxima

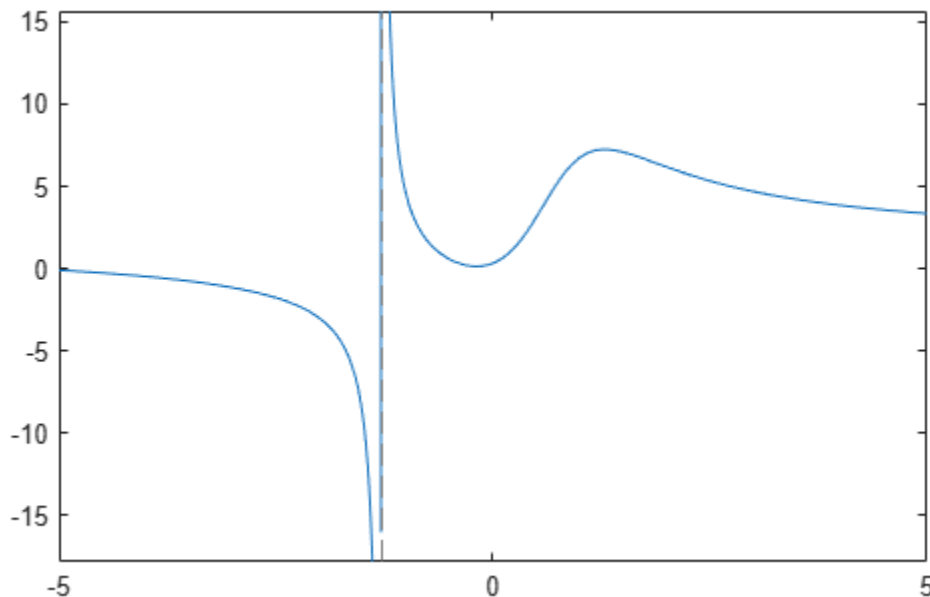
Computing the first derivative of an expression helps you find local minima and maxima of that expression. For example, create a rational expression where the numerator and the denominator are polynomial expressions.

```
syms x
f = (3 * x^3 + 17 * x^2 + 6 * x + 1)/(2 * x^3 + x * -1 + 3)
```

$$f = \frac{3x^3 + 17x^2 + 6x + 1}{2x^3 - x + 3}$$

Plotting this expression shows that it has horizontal and vertical asymptotes, a local minimum between -1 and 0, and a local maximum between 1 and 2.

```
fplot(f)
```



By default, when you operate on this expression, results can include both real and imaginary numbers. If you are interested in real numbers only, you can set the permanent assumption that x

belongs to the set of real numbers. This allows you to avoid complex numbers in the solutions and it also can improve performance.

```
assume(x, 'real')
```

To find a horizontal asymptote, compute the limit of  $f$  for  $x$  approaching positive and negative infinities. The horizontal asymptote is  $x = 3/2$ .

```
[limit(f, x, sym(inf)), limit(f, x, -sym(inf))]
```

```
ans =  
 $\left(\frac{3}{2} \frac{3}{2}\right)$ 
```

To find a vertical asymptote of  $f$ , find the roots of the polynomial expression that represents the denominator of  $f$ .

```
solve(2 * x^3 + x * -1 + 3 == sym(0), x)
```

```
ans =  
root( $z^3 - \frac{z}{2} + \frac{3}{2}$ , z, 3)
```

To get an explicit solution for such equations, try calling the solver with the option `MaxDegree`. The option specifies the maximum degree of polynomials for which the solver tries to return explicit solutions. By default, `MaxDegree` = 2. Increasing this value, you can get explicit solutions for higher-order polynomials. For example, specifying `MaxDegree` = 3 results in an explicit solution.

```
solve(2 * x^3 + x * -1 + 3 == 0, x, 'MaxDegree', 3)
```

```
ans =  

$$-\frac{1}{6\left(\frac{3}{4} - \frac{\sqrt{241}\sqrt{432}}{432}\right)^{1/3}} - \left(\frac{3}{4} - \frac{\sqrt{241}\sqrt{432}}{432}\right)^{1/3}$$

```

You can approximate the exact solution numerically by using the `vpa` function.

```
vpa(ans,6)
```

```
ans = -1.28962
```

Now find the local minimum and maximum of the expression  $f$ . If the point is a local extremum (either minimum or maximum), the first derivative of the expression at that point is equal to zero. To compute the derivative of an expression, use the `diff` function.

```
g = diff(f, x)
```

```
g =  

$$\frac{9x^2 + 34x + 6}{2x^3 - x + 3} - \frac{(6x^2 - 1)(3x^3 + 17x^2 + 6x + 1)}{(2x^3 - x + 3)^2}$$

```

To find the local extrema of  $f$ , solve the equation  $g = 0$ . If you use the `MaxDegree` option, the solver returns the long explicit solution, which can be approximated by using the `vpa` function.

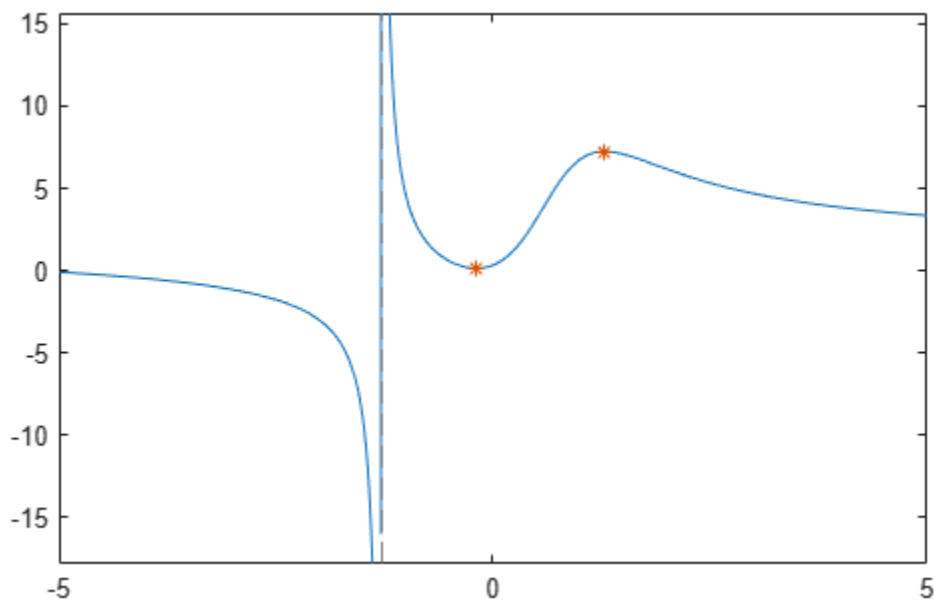
```
solve(g == 0, x, 'MaxDegree', 4);  
extrema = vpa(ans, 6)
```

```
extrema =
```

$$\begin{pmatrix} -0.189245 \\ 1.28598 \end{pmatrix}$$

The plot of the expression  $f$  shows that  $x = -0.189$  is a local minimum of the expression, and  $x = 1.286$  is its local maximum.

```
fplot(f)
hold on
plot(extrema, subs(f,extrema), '*')
hold off
```



### Second Derivatives: Finding Inflection Points

Computing the second derivative lets you find inflection points of the expression.

```
h(x) = simplify(diff(f, x, 2))
```

$$h(x) = \frac{2(68x^6 + 90x^5 + 18x^4 - 699x^3 - 249x^2 + 63x + 172)}{(2x^3 - x + 3)^3}$$

To find inflection points of  $f$ , solve the equation  $h = 0$ . For this equation the symbolic solver returns a complicated result even if you use the `MaxDegree` option.

```
solve(h == 0, x, 'MaxDegree', 4)
```

```
ans =
```

$$\begin{pmatrix} \text{root}(\sigma_1, z, 1) \\ \text{root}(\sigma_1, z, 4) \end{pmatrix}$$

where

$$\sigma_1 = z^6 + \frac{45z^5}{34} + \frac{9z^4}{34} - \frac{699z^3}{68} - \frac{249z^2}{68} + \frac{63z}{68} + \frac{43}{17}$$

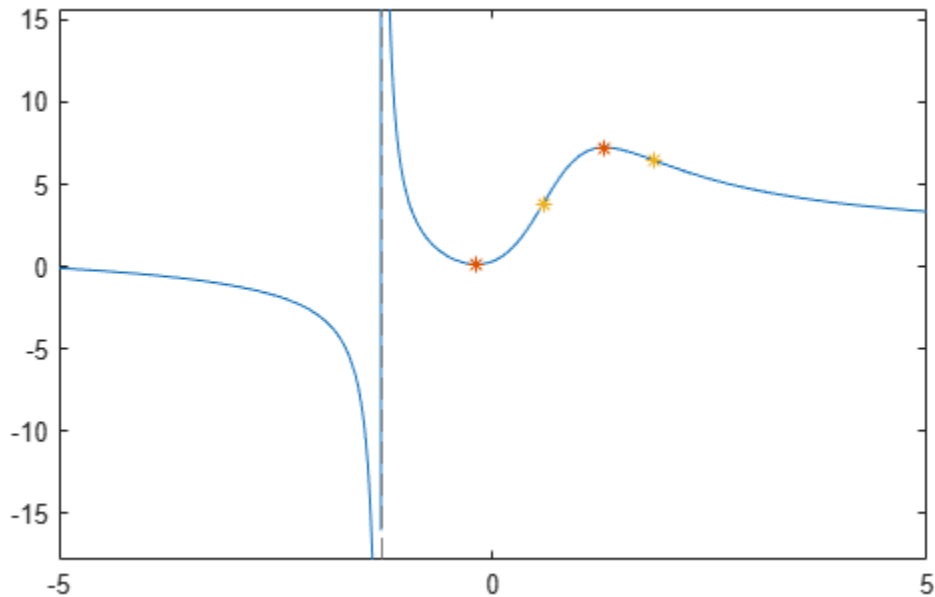
To get the simpler numerical result, solve the equation numerically by using `vpasolve`; specify the search range to restrict the returned results to all real solutions of the expression.

```
inflection = vpasolve(h == 0, x, [-inf, inf])
```

```
inflection =  
(0.57871842655441748319601085860196)  
(1.8651543689917122385037075917613)
```

The expression `f` has two inflection points:  $x = 0.579$  and  $x = 1.865$ .

```
fplot(f)  
hold on  
plot(extrema, subs(f,extrema), '*')  
plot(inflection, subs(f,inflection), '*')  
hold off
```



### Suprema

Not all functions can be treated analytically; the function

$$f(x) = \tan(\sin(x)) - \sin(\tan(x))$$

is very flat at the origin and it oscillates infinitely often near  $-\frac{\pi}{2}$ , becomes linear as it approaches zero and oscillates again near  $\pi$ .

```
f = @(x) tan(sin(x))-sin(tan(x))
```

```
f = function_handle with value:
    @(x)tan(sin(x))-sin(tan(x))
```

```
fplot(f, [-pi , pi])
```

Most important for our purposes here, `fplot` has picked the limit on the y-axes to be

```
ylim
```

```
ans = 1×2
```

```
-2.5573    2.5572
```

### What is happening at $\frac{\pi}{2}$ ?

MATLAB® uses double precision arithmetic, so  $\frac{\pi}{2}$  evaluates to one of the oscillations.

```
pi/2
```

```
ans =
1.5708
```

```
f(pi/2)
```

```
ans =
2.5161
```

The Symbolic Math Toolbox uses exact arithmetic, which shows the function is undefined.

```
syms x
sym(pi/2)
```

```
ans =
    pi
    2
```

```
F = sym(f)
```

```
F = tan(sin(x)) - sin(tan(x))
```

```
subs(F,x,sym(pi/2))
```

```
ans = NaN
```

### Taylor Series

We can also try to look at the value with a Taylor Series.

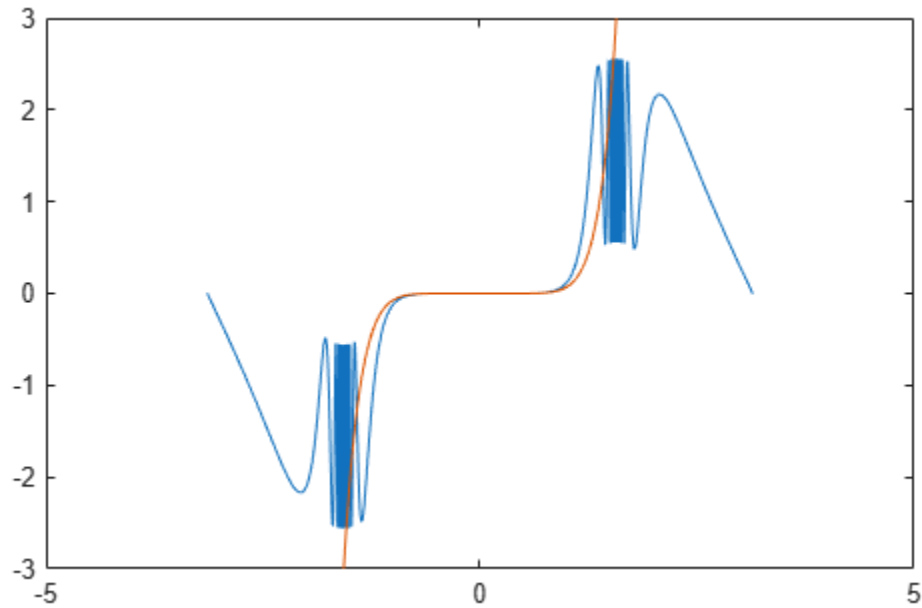
```
T = taylor(F,x,'Order',10,'ExpansionPoint',0)
```

```
T =
    29 x^9
   756
+
    x^7
   30
```

```
vpa(subs(T,x,pi/2))
```

```
ans = 3.0198759869735883213825972535797
```

```
hold on
fplot(T)
ylim ([-3 3])
hold off
```



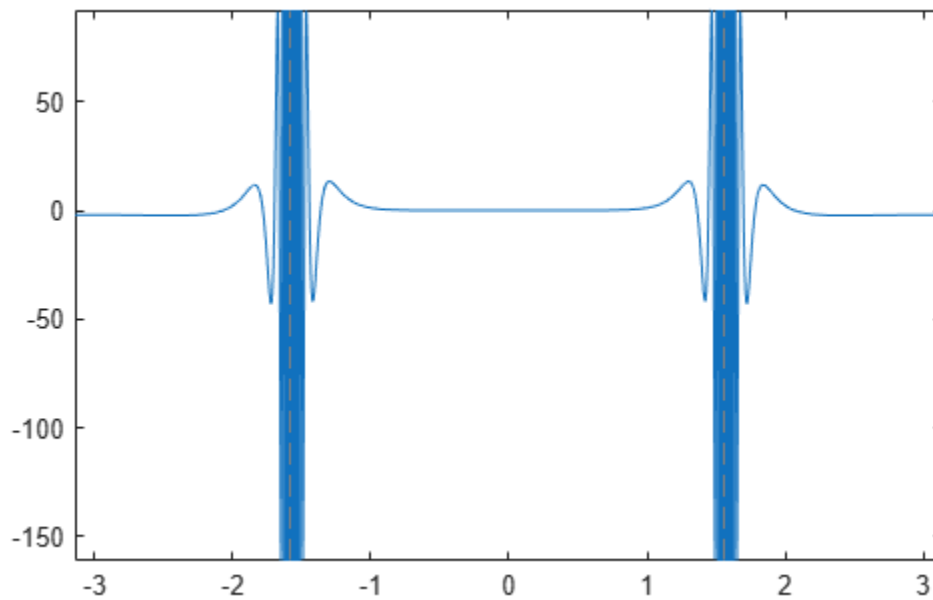
### Calculus

We learn in calculus that a maximum occurs at a zero of the derivative. But this function is not differentiable in the vicinity of  $\frac{\pi}{2}$ . We can analytically differentiate  $f(x)$  using the Symbolic Math Toolbox.

```
diff(F)
```

```
ans = cos(x) (tan(sin(x))^2 + 1) - cos(tan(x)) (tan(x)^2 + 1)
```

```
fplot(diff(F), [-pi , pi])
```



### Sampling

We can sample the function  $N$  times near  $\frac{\pi}{2}$  to get a numerical approximation to the value of the maximum. Is that good enough?

```
N = 100;
xValues = 3*pi/8 + pi/4*rand(1,N)
```

```
xValues = 1×100
```

```
1.8180 1.8895 1.2778 1.8955 1.6748 1.2547 1.3968 1.6076 1.9301 1.9
```

```
ySoln = f(xValues)
```

```
ySoln = 1×100
```

```
0.7260 1.5080 1.5932 1.5614 1.3796 1.3158 2.0658 2.4586 1.8194 1.8
```

```
max(ySoln)
```

```
ans =
2.5387
```

### Proof

Determine the maximum from a mathematical proof.

$$\sin(x) \leq 1$$

so  $\sin(\tan(x)) \leq 1$  and  $\tan(\sin(x)) \leq \tan(1)$  which means consequently

$$f(x) \leq 1 + \tan(1)$$

As  $x \rightarrow \frac{\pi}{2}$ ,  $\tan(x)$  oscillates and blows up; so  $f(x)$  is actually not defined at all at this point as was shown above

$$f(x) < 1 + \tan(1)$$

Now we can take a look at the numerical value.

```
format long  
1 + tan(1)
```

```
ans =  
2.557407724654902
```



## Find Extremum of Multivariate Function and Its Approximation

This example shows how to find the extremum of a multivariate function and its approximation near the extremum point. This example uses symbolic matrix variables to represent the multivariate function and its derivatives. Symbolic matrix variables are available starting in R2021a.

Consider the multivariate function  $f(\mathbf{x}) = \sin(\mathbf{x}^T \mathbf{A} \mathbf{x})$ , where  $\mathbf{x}$  is a 2-by-1 vector and  $\mathbf{A}$  is a 2-by-2 matrix. To find a local extremum of this function, compute a root of the derivative of  $f(\mathbf{x})$ . In other words, find the solution of the derivative  $\nabla f(\mathbf{x}_0) = 0$ .

### Create Function and Find Its Derivative

Create the vector  $\mathbf{x}$  and the matrix  $\mathbf{A}$  as symbolic matrix variables. Define the function  $f(\mathbf{x}) = \sin(\mathbf{x}^T \mathbf{A} \mathbf{x})$ .

```
syms x [2 1] matrix
syms A [2 2] matrix
f = sin(x.'*A*x)
```

$$f = \sin(\mathbf{x}^T \mathbf{A} \mathbf{x})$$

Compute the derivative  $\mathbf{D}$  of the function  $f(\mathbf{x})$  with respect to the vector  $\mathbf{x}$ . The derivative  $\mathbf{D}$  is displayed in compact matrix notation in terms of  $\mathbf{x}$  and  $\mathbf{A}$ .

```
D = diff(f,x)
```

$$\mathbf{D} = \cos(\mathbf{x}^T \mathbf{A} \mathbf{x}) (\mathbf{x}^T \mathbf{A} + \mathbf{x}^T \mathbf{A}^T)$$

### Convert symmatrix Objects to sym Objects

The symbolic matrix variables  $\mathbf{x}$ ,  $\mathbf{A}$ ,  $f$ , and  $\mathbf{D}$  are `symmatrix` objects. These objects represent matrices, vectors, and scalars in compact matrix notation. To show the components of these variables, convert the `symmatrix` objects to `sym` objects using `symmatrix2sym`.

```
xsym = symmatrix2sym(x)
```

$$\mathbf{x}_{\text{sym}} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

```
Asym = symmatrix2sym(A)
```

$$\mathbf{A}_{\text{sym}} = \begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix}$$

```
fsym = symmatrix2sym(f)
```

$$f_{\text{sym}} = \sin(x_1 (A_{1,1} x_1 + A_{1,2} x_2) + x_2 (A_{2,1} x_1 + A_{2,2} x_2))$$

```
Dsym = symmatrix2sym(D)
```

$$\mathbf{D}_{\text{sym}} = \begin{pmatrix} \cos(x_1 (A_{1,1} x_1 + A_{1,2} x_2) + x_2 (A_{2,1} x_1 + A_{2,2} x_2)) (2 A_{1,1} x_1 + A_{1,2} x_2 + A_{2,1} x_2) & \cos(x_1 (A_{1,1} x_1 + A_{1,2} x_2) + x_2 (A_{2,1} x_1 + A_{2,2} x_2)) (A_{1,2} x_1 + A_{2,2} x_2 + A_{2,1} x_1) \\ \cos(x_1 (A_{1,1} x_1 + A_{1,2} x_2) + x_2 (A_{2,1} x_1 + A_{2,2} x_2)) (A_{2,1} x_1 + A_{2,2} x_2 + A_{2,1} x_1) & \cos(x_1 (A_{1,1} x_1 + A_{1,2} x_2) + x_2 (A_{2,1} x_1 + A_{2,2} x_2)) (2 A_{2,1} x_1 + A_{1,2} x_2 + A_{2,2} x_1) \end{pmatrix}$$

### Substitute Numeric Values and Find the Minimum

Suppose you are interested in the case where the value of  $A$  is  $[2 \ -1; \ 0 \ 3]$ . Substitute this value into the function `fsym`.

```
fsym = subs(fsym,Asym,[2 -1; 0 3])
```

```
fsym = sin(3*x2^2 + x1*(2*x1 - x2))
```

Substitute the value of  $A$  into the derivative `Dsym`

```
Dsym = subs(Dsym,Asym,[2 -1; 0 3])
```

```
Dsym = (cos(3*x2^2 + x1*(2*x1 - x2))*(4*x1 - x2) - cos(3*x2^2 + x1*(2*x1 - x2))*(x1 - 6*x2))
```

Then, apply the symbolic function `solve` to get a root of the derivative.

```
[xmin,ymin] = solve(Dsym,xsym,'PrincipalValue',true);
```

```
x0 = [xmin; ymin]
```

```
x0 =  
      (0)  
      (0)
```

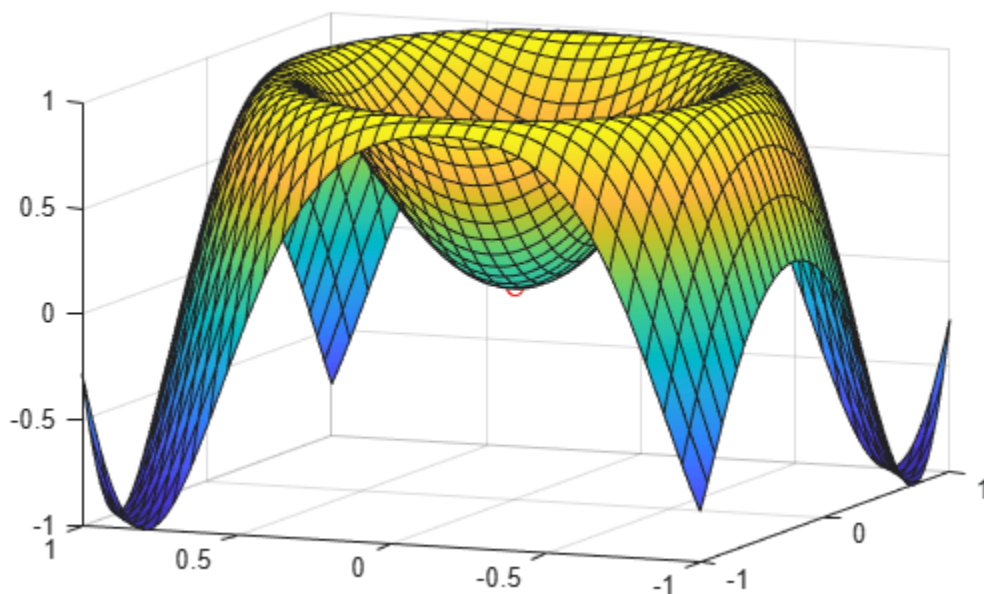
Plot the function  $f(x)$  together with the extremum solution  $x_0$ . Set the plot interval to  $-1 < x_1 < 1$  and  $-1 < x_2 < 1$  as the second argument of `fsurf`. Use `fplot3` to plot the coordinates of the extremum solution.

```
fsurf(fsym,[-1 1 -1 1])
```

```
hold on
```

```
fplot3(xmin,ymin,subs(fsym,xsym,x0),'ro')
```

```
view([-68 13])
```



### Approximate Function Near Its Minimum

You can approximate a multivariate function around a point  $x_0$  with a multinomial using the Taylor expansion.

$$f(x) \approx f(x_0) + \nabla f(x_0) \cdot (x - x_0) + \frac{1}{2}(x - x_0)^T H(f(x_0)) (x - x_0)$$

Here, the term  $\nabla f(x_0)$  is the gradient vector, and  $H(f(x_0))$  is the Hessian matrix of the multivariate function  $f(x)$  calculated at  $x_0$ .

Find the Hessian matrix and return the result as a symbolic matrix variable.

```
H = diff(f,x,x.')
```

$$H = -\sin(x^T A x) (A^T x + A x) (x^T A + x^T A^T) + \cos(x^T A x) (A^T + A)$$

Convert the Hessian matrix  $H(f(x_0))$  to the sym data type, which represents the matrix in its component form. Use subs to evaluate the Hessian matrix for  $A = \begin{bmatrix} 2 & -1 \\ 0 & 3 \end{bmatrix}$  at the minimum point  $x_0$ .

```
Hsym = symmatrix2sym(H);
Hsym = subs(Hsym,Asym,[2 -1; 0 3]);
H0 = subs(Hsym,xsym,x0)
```

$$H_0 = \begin{pmatrix} 4 & -1 \\ -1 & 6 \end{pmatrix}$$

Evaluate the gradient vector  $\nabla f(x_0)$  at  $x_0$ .

```
D0 = subs(Dsym,xsym,x0)
```

$$D_0 = (0 \ 0)$$

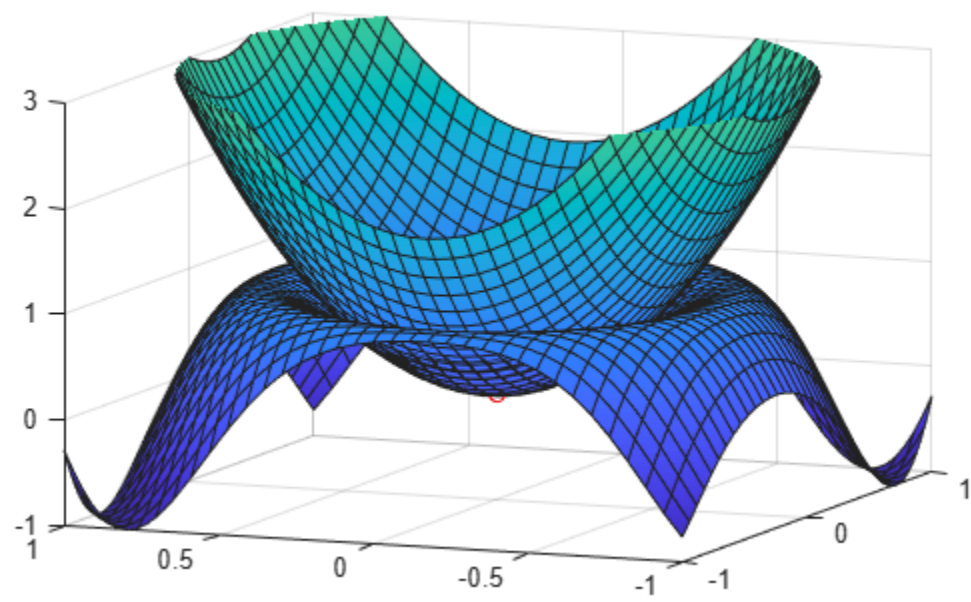
Compute the Taylor approximation to the function near its minimum.

```
fapprox = subs(fsym,xsym,x0) + D0*(xsym-x0) + 1/2*(xsym-x0).'*H0*(xsym-x0)
```

$$f_{\text{approx}} = x_1 \left( 2x_1 - \frac{x_2}{2} \right) - x_2 \left( \frac{x_1}{2} - 3x_2 \right)$$

Plot the function approximation on the same graph that shows  $f(x)$  and  $x_0$ .

```
hold on
fsurf(fapprox,[-1 1 -1 1])
zlim([-1 3])
```



## Padé Approximant of Time-Delay Input

This example shows how to use a Padé approximant in control system theory to model time delays in the response of a first-order system. Time delays arise in systems such as chemical and transport processes where there is a delay between the input and the system response. When these inputs are modeled, they are called dead-time inputs.

This example uses Symbolic Math Toolbox™ to solve for the transfer function of a first-order system and find the system response to dead-time step input using Padé approximant. This example performs calculations symbolically to obtain analytic results.

### Introduction

The Padé approximant of order  $[m, n]$  approximates the function  $f(x)$  around  $x = x_0$  as

$$\frac{a_0 + a_1(x - x_0) + \cdots + a_m(x - x_0)^m}{1 + b_1(x - x_0) + \cdots + b_n(x - x_0)^n}.$$

The Padé approximant is a rational function formed by a ratio of two power series. Because it is a rational function, it is more accurate than the Taylor series in approximating functions with poles. The Padé approximant is represented by the Symbolic Math Toolbox™ function `pade`.

When a pole or zero exists at the expansion point  $x = x_0$ , the accuracy of the Padé approximant decreases. To increase accuracy, use an alternative form of the Padé approximant which is

$$\frac{(x - x_0)^p(a_0 + a_1(x - x_0) + \cdots + a_m(x - x_0)^m)}{1 + b_1(x - x_0) + \cdots + b_n(x - x_0)^n}.$$

The `pade` function returns the alternative form of the Padé approximant when you set the `OrderMode` input argument to `Relative`.

### Find Transfer Function of First-Order System

The behavior of a first-order system is described by this differential equation

$$\tau \frac{dy(t)}{dt} + y(t) = ax(t).$$

Enter the differential equation in MATLAB®.

```
syms tau a x(t) y(t) xS(s) yS(s) H(s) tmp
F = tau*diff(y)+y == a*x;
```

Find the Laplace transform of  $F$  using `laplace`.

```
F = laplace(F,t,s)
```

```
F = laplace(y(t),t,s) - tau (y(0) - s laplace(y(t),t,s)) = a laplace(x(t),t,s)
```

Assume the response of the system at  $t = 0$  is  $0$ . Use `subs` to substitute for  $y(0) = 0$ .

```
F = subs(F,y(0),0)
```

```
F = laplace(y(t),t,s) + s tau laplace(y(t),t,s) = a laplace(x(t),t,s)
```

To collect common terms, use `simplify`.

```
F = simplify(F)
```

```
F = (s*tau + 1) laplace(y(t), t, s) = a laplace(x(t), t, s)
```

For readability, replace the Laplace transforms of  $x(t)$  and  $y(t)$  with  $xS(s)$  and  $yS(s)$ .

```
F = subs(F, [laplace(x(t), t, s) laplace(y(t), t, s)], [xS(s) yS(s)])
```

```
F = yS(s)(s*tau + 1) = a xS(s)
```

The Laplace transform of the transfer function is  $yS(s)/xS(s)$ . Divide both sides of the equation by  $xS(s)$  and use `subs` to replace  $yS(s)/xS(s)$  with  $H(s)$ .

```
F = F/xS(s);
```

```
F = subs(F, yS(s)/xS(s), H(s))
```

```
F = H(s)(s*tau + 1) = a
```

Solve the equation for  $H(s)$ . Substitute for  $H(s)$  with a dummy variable, solve for the dummy variable using `solve`, and assign the solution to `Hsol(s)`.

```
F = subs(F, H(s), tmp);
```

```
Hsol(s) = solve(F, tmp)
```

```
Hsol(s) =
```

$$\frac{a}{s\tau + 1}$$

### Find Response of System to Time-Delayed Step Input

The input to the first-order system is a time-delayed step input. To represent a step input, use `heaviside`. Delay the input by three time units. Find the Laplace transform using `laplace`.

```
step = heaviside(t - 3);
```

```
step = laplace(step)
```

```
step =
```

$$\frac{e^{-3s}}{s}$$

Find the response of the system, which is the product of the transfer function and the input.

```
y = Hsol(s)*step
```

```
y =
```

$$\frac{a e^{-3s}}{s(s\tau + 1)}$$

To allow plotting of the response, set parameters  $a$  and  $\tau$  to specific values. For  $a$  and  $\tau$ , choose values 1 and 3, respectively.

```
y = subs(y, [a tau], [1 3]);
```

```
y = ilaplace(y, s);
```

### Find Response of System Using Padé Approximants

Find the Padé approximant of order  $[2 \ 2]$  of the step input using the `Order` input argument to `pade`.

```
stepPade22 = pade(step, 'Order', [2 2])
```

```
stepPade22 =
```

$$\frac{3s^2 - 4s + 2}{2s(s+1)}$$

Find the response to the input by multiplying the transfer function and the Padé approximant of the input.

```
yPade22 = Hsol(s)*stepPade22
```

```
yPade22 =
```

$$\frac{a(3s^2 - 4s + 2)}{2s(s\tau + 1)(s + 1)}$$

Find the inverse Laplace transform of yPade22 using ilaplace.

```
yPade22 = ilaplace(yPade22,s)
```

```
yPade22 =
```

$$a + \frac{9a}{2\tau - 2}e^{-s} - \frac{ae^{-\frac{s}{\tau}}(2\tau^2 + 4\tau + 3)}{\tau(2\tau - 2)}$$

To plot the response, set parameters a and tau to their values of 1 and 3, respectively.

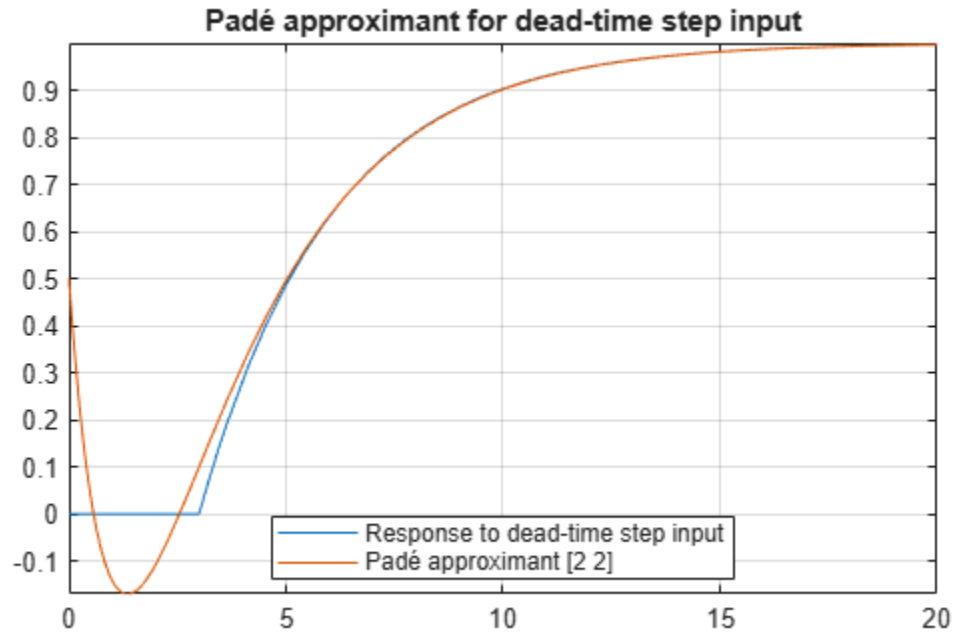
```
yPade22 = subs(yPade22,[a tau],[1 3])
```

```
yPade22 =
```

$$\frac{9}{4}e^{-s} - \frac{11}{4}e^{-\frac{s}{3}} + 1$$

Plot the response of the system y and the response calculated from the Padé approximant yPade22.

```
fplot(y,[0 20])
hold on
fplot(yPade22, [0 20])
grid on
title 'Padé approximant for dead-time step input'
legend('Response to dead-time step input', 'Padé approximant [2 2]',...
       'Location', 'Best');
```



### Increase Accuracy of Padé Approximant Using OrderMode

The [2 2] Padé approximant does not represent the response well because a pole exists at the expansion point of 0. To increase the accuracy of pade when there is a pole or zero at the expansion point, set the OrderMode input argument to Relative and repeat the steps. For details, see pade.

```
stepPade22Rel = pade(step, 'Order', [2 2], 'OrderMode', 'Relative')
```

```
stepPade22Rel =
```

$$\frac{3s^2 - 6s + 4}{s(3s^2 + 6s + 4)}$$

```
yPade22Rel = Hsol(s)*stepPade22Rel
```

```
yPade22Rel =
```

$$\frac{a(3s^2 - 6s + 4)}{s(s\tau + 1)(3s^2 + 6s + 4)}$$

```
yPade22Rel = ilaplace(yPade22Rel);
```

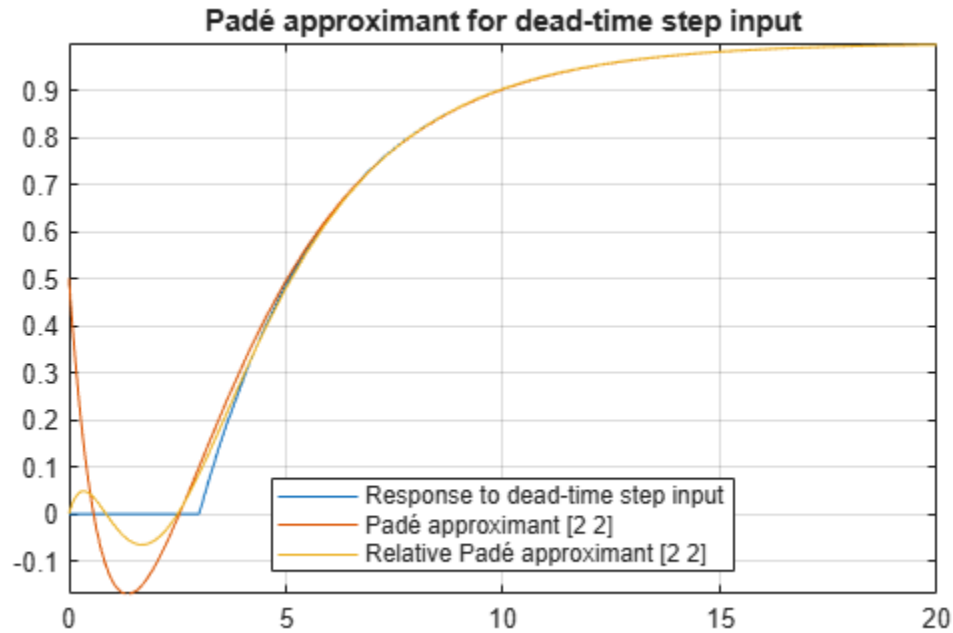
```
yPade22Rel = subs(yPade22Rel, [a tau], [1 3])
```

```
yPade22Rel =
```

$$\frac{12 e^{-t} \left( \cos\left(\frac{\sqrt{3}t}{3}\right) + \frac{2\sqrt{3} \sin\left(\frac{\sqrt{3}t}{3}\right)}{3} \right)}{7} - \frac{19 e^{-\frac{t}{3}}}{7} + 1$$

```
fplot(yPade22Rel, [0 20], 'DisplayName', 'Relative Padé approximant [2 2]')
```





### Increase Accuracy of Padé Approximant by Increasing Order

You can increase the accuracy of the Padé approximant by increasing its order. Increase the order to [4 5] and repeat the steps. The [n-1 n] Padé approximant is better at approximating the response at  $t = 0$  than the [n n] Padé approximant.

```
stepPade45 = pade(step, 'Order', [4 5])
```

```
stepPade45 =  


$$\frac{27s^4 - 180s^3 + 540s^2 - 840s + 560}{s(27s^4 + 180s^3 + 540s^2 + 840s + 560)}$$

```

```
yPade45 = Hsol(s)*stepPade45
```

```
yPade45 =  


$$\frac{a(27s^4 - 180s^3 + 540s^2 - 840s + 560)}{s(s\tau + 1)(27s^4 + 180s^3 + 540s^2 + 840s + 560)}$$

```

```
yPade45 = subs(yPade45,[a tau],[1 3])
```

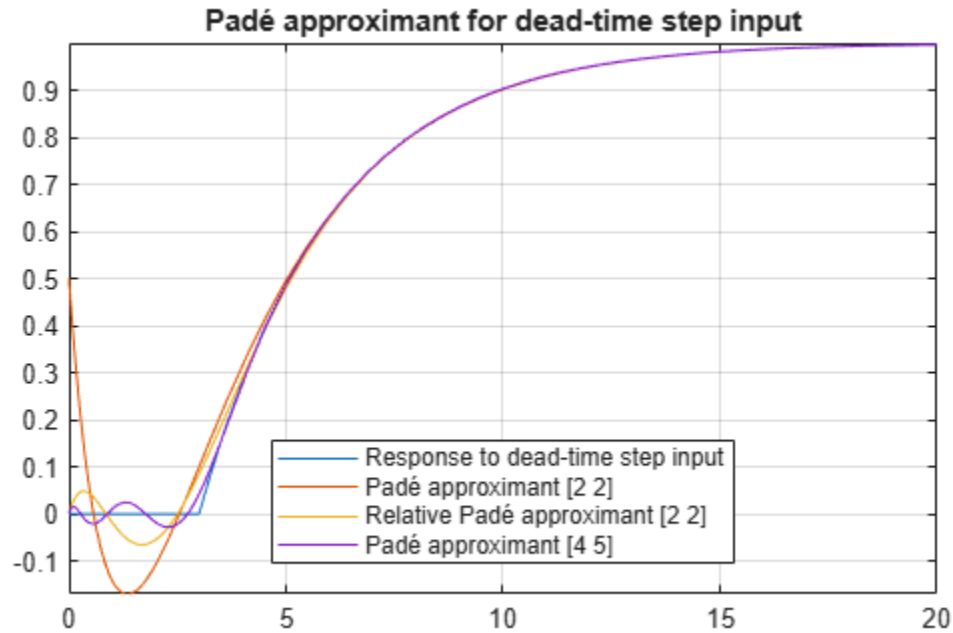
```
yPade45 =  


$$\frac{27s^4 - 180s^3 + 540s^2 - 840s + 560}{s(3s + 1)(27s^4 + 180s^3 + 540s^2 + 840s + 560)}$$

```

Find the inverse Laplace transform of yPade45 using `ilaplace`. Approximate yPade45 numerically using `vpa`. Plot the response calculated from the Padé approximant yPade45.

```
yPade45 = vpa(ilaplace(yPade45));  
fplot(yPade45, [0 20], 'DisplayName', 'Padé approximant [4 5]')
```



## Conclusions

The following points have been shown:

- Padé approximants can model dead-time step inputs.
- The accuracy of the Padé approximant increases with the increase in the order of the approximant.
- When a pole or zero exists at the expansion point, the Padé approximant is inaccurate about the expansion point. To increase the accuracy of the approximant, set the `OrderMode` option to `Relative`. You can also use increase the order of the denominator relative to the numerator.

## See Also

### Functions

`pade` | `diff` | `heaviside` | `laplace` | `ilaplace` | `subs`

## Related Examples

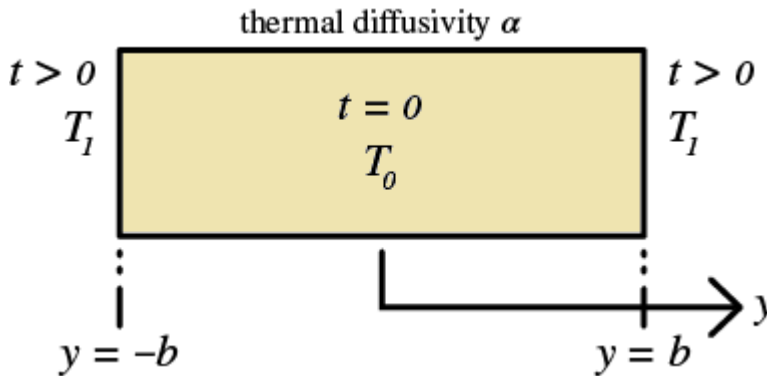
- “Solve Differential Equations of RLC Circuit Using Laplace Transform” on page 3-198
- “Analyze Transfer Function of T-Coil Circuit” on page 3-324
- “Harmonic Analysis of Transfer Function Output” on page 3-160
- “Estimate Model Parameters of a Symbolically Derived Plant Model in Simulink” on page 5-74
- “Customize and Extend Simscape Libraries for a Custom DC Motor” on page 5-65

## Heating of Finite Slab

This example shows how to find the temperature distribution of a one-dimensional finite slab by solving the governing differential equation. The example uses Symbolic Math Toolbox™ to solve the problem of heat transfer analytically in a finite slab as discussed in [1] on page 3-274.

The finite slab has constant thermal properties. Assume that heat transfer is due only to conduction with a given thermal diffusivity  $\alpha = \frac{k}{\rho c_p}$ . The slab is located in the space between  $y = -b$  and  $y = b$ , where heat propagates along the  $y$ -direction. Initially, the slab is at temperature  $T_0$ . At time  $t = 0$ , the edges at  $y = \pm b$  are suddenly raised to temperature  $T_1$  and maintained there. This example solves for the temperature distribution of the slab  $T(y, t)$  as a function of the  $y$ -coordinate and time  $t$  by following these steps:

- Define the heat transfer equation.
- Use the method of separation of variables.
- Apply boundary conditions to find the mode solutions.
- Find the coefficients of the general solution by computing the integral of orthogonal functions.
- Plot the solution for the temperature as a function of the  $y$ -coordinate and time.



### Define Heat Transfer Equation

For a three-dimensional homogeneous medium, the heat transfer equation due to conduction is given by

$$\frac{\partial T}{\partial t} = \alpha \left( \frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} + \frac{\partial^2 T}{\partial z^2} \right).$$

Because heat is transferred along the  $y$ -direction, you can reduce the heat transfer equation to

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial y^2}.$$

Next, define these dimensionless variables:

- Dimensionless temperature  $\Theta = \frac{T_1 - T}{T_1 - T_0}$

- Dimensionless coordinate  $\eta = \frac{y}{b}$
- Dimensionless time  $\tau = \frac{\alpha t}{b^2}$

The differential equation for the heat transfer in terms of these dimensionless variables is

$$\frac{\partial \Theta}{\partial \tau} = \frac{\partial^2 \Theta}{\partial \eta^2},$$

where the initial condition at  $\tau = 0$  is  $\Theta = 1$ , and the boundary conditions at  $\eta = \pm 1$  are  $\Theta = 0$  for  $\tau > 0$ .

Define the main differential equation.

```
syms Theta(eta,tau)
eqMain = diff(Theta,tau) == diff(Theta,eta,eta)

eqMain(eta, tau) =

$$\frac{\partial}{\partial \tau} \Theta(\eta, \tau) = \frac{\partial^2}{\partial \eta^2} \Theta(\eta, \tau)$$

```

### Method of Separation of Variables

Use the method of separation of variables to find the solution for  $\Theta(\eta, \tau)$ . This method separates the spatial and time dependencies by writing them as a product of spatial and time-dependent functions, where  $\Theta(\eta, \tau) = f(\eta)g(\tau)$ .

```
syms f(eta) g(tau)
eqTheta = Theta(eta,tau) == f(eta)*g(tau)

eqTheta = Theta(eta, tau) = f(eta) g(tau)
```

Substitute this ansatz into the differential equation.

```
eqMain = subs(eqMain,lhs(eqTheta),rhs(eqTheta))

eqMain(eta, tau) =

$$f(\eta) \frac{\partial}{\partial \tau} g(\tau) = g(\tau) \frac{\partial^2}{\partial \eta^2} f(\eta)$$

```

Separate the  $f$  and  $g$  terms so that they are be on different sides of the equation.

```
eqMain = eqMain/g(tau)/f(eta)

eqMain(eta, tau) =

$$\frac{\frac{\partial}{\partial \tau} g(\tau)}{g(\tau)} = \frac{\frac{\partial^2}{\partial \eta^2} f(\eta)}{f(\eta)}$$

```

Set each side of the equation to a constant. For convenience, let this constant be  $-c^2$ .

```
syms c real
eq1(tau) = lhs(eqMain) == -c^2

eq1(tau) =
```

$$\frac{\frac{\partial}{\partial \tau} g(\tau)}{g(\tau)} = -c^2$$

```
eq2(eta) = rhs(eqMain) == -c^2
```

$$\text{eq2}(\eta) = \frac{\frac{\partial^2}{\partial \eta^2} f(\eta)}{f(\eta)} = -c^2$$

### Apply Boundary Conditions to Find Mode Solution

Next, find the solutions for  $f(\eta)$  and  $g(\tau)$  by applying the appropriate boundary conditions.

Solve the first-order differential equation for  $g(\tau)$  by using `dsolve`.

```
gsol(tau) = dsolve(eq1)
```

$$\text{gsol}(\tau) = C_1 e^{-c^2 \tau}$$

For  $f(\eta)$ , find the even solution based on the symmetry along the  $y$ -axis. Because the slab is located at  $-b$  to  $b$  and the temperature at both edges is fixed at  $T_1$ , the function  $f(\eta)$  is symmetrical with respect to the origin. Find the general solution using `dsolve`.

```
fsol(eta) = dsolve(eq2)
```

$$\text{fsol}(\eta) = C_1 \cos(c \eta) - C_2 \sin(c \eta)$$

For a well-behaved function, you can also write the general solution as  $f(\eta) = f_{\text{even}}(\eta) + f_{\text{odd}}(\eta)$ . The

even part of this solution is  $f_{\text{even}}(\eta) = \frac{f(\eta) + f(-\eta)}{2}$  and the odd part of this solution is

$f_{\text{odd}}(\eta) = \frac{f(\eta) - f(-\eta)}{2}$ . Define the even solution of  $f(\eta)$ .

```
fsol_even(eta) = (fsol(eta) + fsol(-eta))/2
```

$$\text{fsol\_even}(\eta) = C_1 \cos(c \eta)$$

Symbolic Math Toolbox uses  $C_1$  as a dummy variable to represent a constant. In the solutions for  $f(\eta)$  and  $g(\tau)$  above, the variable  $C_1$  represents different constants. Rewrite the variable  $C_1$  from the even solution of  $f(\eta)$  as  $C_f$ .

```
syms C1 C_f
fsol = subs(fsol_even,C1,C_f)
```

$$\text{fsol}(\eta) = C_f \cos(c \eta)$$

Apply the other boundary condition at  $\eta = \pm 1$ , where  $f(1) = 0$ , and solve for the parameter  $c$ .

```
[csol,params,conds] = solve(fsol(1)==0,c,ReturnConditions=true)
```

$$\text{csol} = \frac{\pi}{2} + \pi k$$

```
params = k
```

$$\text{conds} = k \in \mathbb{Z} \wedge C_f \neq 0$$

The parameter  $c$  in the solutions for  $f(\eta)$  and  $g(\tau)$  depends on an integer parameter  $k$ , where  $c = \pi/2 + \pi k$ .

Substitute the solutions for  $f(\eta)$  and  $g(\tau)$  into  $\Theta(\eta, \tau)$ .

$$\text{eqTheta} = \text{subs}(\text{eqTheta}, \{f, g\}, \{f_{\text{sol}}, g_{\text{sol}}\})$$

$$\text{eqTheta} = \Theta(\eta, \tau) = C_1 C_f e^{-c^2 \tau} \cos(c \eta)$$

Rewrite the  $C_1 C_f$  constant term as  $C_k$ . Rewrite the solution to depend on  $k$  as well. This mode solution uses the index  $k$  for the dimensionless temperature  $\Theta(\eta, \tau)$ .

$$\text{syms } C(k) \ D(k) \\ \text{eqTheta} = \text{subs}(\text{eqTheta}, \{C_1 C_f, c\}, \{C(k), c_{\text{sol}}\})$$

$$\text{eqTheta} =$$

$$\Theta(\eta, \tau) = \cos\left(\eta\left(\frac{\pi}{2} + \pi k\right)\right) e^{-\tau\left(\frac{\pi}{2} + \pi k\right)^2} C(k)$$

### Compute Integral of Orthogonal Functions

Next, construct the general solution of the differential equation from the mode solution. Get the right side of the equation in `eqTheta` to perform further computations.

$$\text{Theta}_k(\eta, \tau, k) = \text{rhs}(\text{eqTheta})$$

$$\text{Theta}_k(\eta, \tau, k) =$$

$$\cos\left(\eta\left(\frac{\pi}{2} + \pi k\right)\right) e^{-\tau\left(\frac{\pi}{2} + \pi k\right)^2} C(k)$$

Following the steps in the textbook [1] on page 3-274, combine the terms with indices  $k$  and  $-(k+1)$ . Simplify these combined terms.

$$\text{Theta}_k2(\eta, \tau, k) = \text{simplify}(\text{Theta}_k(\eta, \tau, k) + \text{Theta}_k(\eta, \tau, -(k+1)))$$

$$\text{Theta}_k2(\eta, \tau, k) =$$

$$e^{-\frac{\tau \pi^2 (2k+1)^2}{4}} \cos\left(\frac{\pi \eta (2k+1)}{2}\right) (C(-k-1) + C(k))$$

Rewrite the general solution by summing all of these modes. Here, define another  $D(k)$  coefficient to represent  $C(-k-1) + C(k)$ . Then, take the summation of the  $k$  indices from 0 to  $\infty$ .

$$\text{Theta}_{\text{sol}} = \text{symsum}(\text{subs}(\text{Theta}_k2, C(-k-1)+C(k), D(k)), k, 0, \text{Inf})$$

$$\text{Theta}_{\text{sol}}(\eta, \tau) =$$

$$\sum_{k=0}^{\infty} e^{-\frac{\tau \pi^2 (2k+1)^2}{4}} \cos\left(\frac{\pi \eta (2k+1)}{2}\right) D(k)$$

Next, find the  $D(k)$  coefficient based on the initial condition of the problem. To do this, set the boundary condition and find the integral of the orthogonal functions that involve the cos function. Set the initial boundary condition  $\Theta(\eta, 0) = 1$ .

$$\text{ThetaBound} = \text{Theta}_{\text{sol}}(\eta, 0) == 1$$

$$\text{ThetaBound} =$$

$$\sum_{k=0}^{\infty} \cos\left(\frac{\pi \eta (2k+1)}{2}\right) D(k) = 1$$

Multiply both sides of the above equation by  $\cos\left(\pi \eta \left(m + \frac{1}{2}\right)\right)$ , and find the integral with respect to  $d\eta$ . Here, the indices  $m$  and  $k$  are integers.

```
syms m k integer
```

```
ThetaBound = ThetaBound*cos((m+1/2)*pi*eta)
```

```
ThetaBound =
```

$$\cos\left(\pi \eta \left(m + \frac{1}{2}\right)\right) \left(\sum_{k=0}^{\infty} \cos\left(\frac{\pi \eta (2k+1)}{2}\right) D(k)\right) = \cos\left(\pi \eta \left(m + \frac{1}{2}\right)\right)$$

```
ThetaBound = int(lhs(ThetaBound),eta,-1,1) == int(rhs(ThetaBound),eta,-1,1)
```

```
ThetaBound =
```

$$\int_1^1 \cos\left(\pi \eta \left(m + \frac{1}{2}\right)\right) \left(\sum_{k=0}^{\infty} \cos\left(\frac{\pi \eta (2k+1)}{2}\right) D(k)\right) d\eta = \frac{2 \sin\left(\pi \left(m + \frac{1}{2}\right)\right)}{\pi \left(m + \frac{1}{2}\right)}$$

This integration involves a symbolic summation series that uses the `symsum` function. Symbolic Math Toolbox does not find this integral explicitly, so you need to investigate the integral of each summation term on the left side of the above equation. Extract each symbolic summation term that has the indices  $m$  and  $k$ . Define this term as a new expression. You can copy and paste from the output above when defining this expression manually.

```
expr_temp = cos(sym(pi)*eta*(m + sym(1/2)))*cos((sym(pi)*eta*(2*k + 1))/2)*D(k)
```

```
expr_temp =
```

$$\cos\left(\pi \eta \left(m + \frac{1}{2}\right)\right) \cos\left(\frac{\pi \eta (2k+1)}{2}\right) D(k)$$

Next, find the integral of the expression over  $\eta$  from  $-1$  to  $1$ .

```
expr_int = int(expr_temp,eta,-1,1)
```

```
expr_int =
```

$$\begin{cases} D(k) & \text{if } k = m \vee k + m = -1 \\ 0 & \text{if } k \neq m \wedge k + m \neq -1 \end{cases}$$

Symbolic Math Toolbox simplifies the integral. To compare this result with the one in the textbook without simplification, check this integral by assuming that  $m$  and  $k$  are equal (as in the first piecewise condition above).

```
expr_temp2 = cos(pi*eta*(m+1/2))*cos(pi*eta*(m+1/2))*D(m)
```

```
expr_temp2 =
```

$$\cos\left(\pi \eta \left(m + \frac{1}{2}\right)\right)^2 D(m)$$

Without simplification, Symbolic Math Toolbox returns the integral as in the textbook example.

```
expr_int2 = int(expr_temp2,eta,-1,1)
```

```
expr_int2 =
```

$$D(m) \left( \frac{\sin\left(2 \pi \left(m + \frac{1}{2}\right)\right)}{2 \pi \left(m + \frac{1}{2}\right)} + 1 \right)$$

Because  $m$  is an integer, you can simplify this result.

```
expr_int2 = simplify(expr_int2)
```

```
expr_int2 = D(m)
```

Next, find the coefficient  $D(m)$  by applying the boundary condition from the previous step. Simplify the equation for  $D(m)$  with the applied boundary condition, and isolate  $D(m)$  on the left side of the equation.

```
ThetaBound = expr_int2 == rhs(ThetaBound)
```

```
ThetaBound =
```

$$D(m) = \frac{2 \sin\left(\pi \left(m + \frac{1}{2}\right)\right)}{\pi \left(m + \frac{1}{2}\right)}$$

```
ThetaBound = isolate(simplify(ThetaBound), D(m))
```

```
ThetaBound =
```

$$D(m) = \frac{4(-1)^m}{\pi + 2\pi m}$$

Change the dummy variable  $m$  to  $k$ . Finally, substitute  $D(k)$  into the general solution. This solution represents the heat transfer of a finite one-dimensional slab.

```
ThetaBound = subs(ThetaBound,m,k);
```

```
Theta_sol = subs(Theta_sol,D,rhs(ThetaBound))
```

```
Theta_sol(eta, tau) =
```

$$4 \left( \sum_{k=0}^{\infty} \frac{(-1)^k e^{-\frac{\tau \pi^2 (2k+1)^2}{4}} \cos\left(\frac{\pi \eta (2k+1)}{2}\right)}{\pi + 2\pi k} \right)$$

### Plot Solution for Temperature

Plot the dimensionless temperature of the slab as a function of  $\eta$  at various values of  $\tau$ . Because the temperature solution involves a summation series with infinite terms, evaluating all these terms when plotting can be slow. To improve the plotting performance, you can approximate the infinite summation series as a summation series of up to  $k = 40$ . Note that the temperature solution is a converging series and numerically stable. For this reason, approximating the series works well to improve the plotting performance.

```
Theta_sol_approx = subs(Theta_sol,Inf,40);
```

Plot the temperature of the slab for various values of  $\tau$  by using `fplot`.

```
% Plot the result as in figure 12.1-1 of reference [1]
```

```
fplot(eta,1-Theta_sol_approx(eta,0.01),[0 1])
```

```
axis equal
```

```
hold on
```

```
fplot(eta,1-Theta_sol_approx(eta,0.04),[0 1])
```

```
fplot(eta,1-Theta_sol_approx(eta,0.1),[0 1])
```

```
fplot(eta,1-Theta_sol_approx(eta,0.2),[0 1])
```

```
fplot(eta,1-Theta_sol_approx(eta,0.4),[0 1])
```

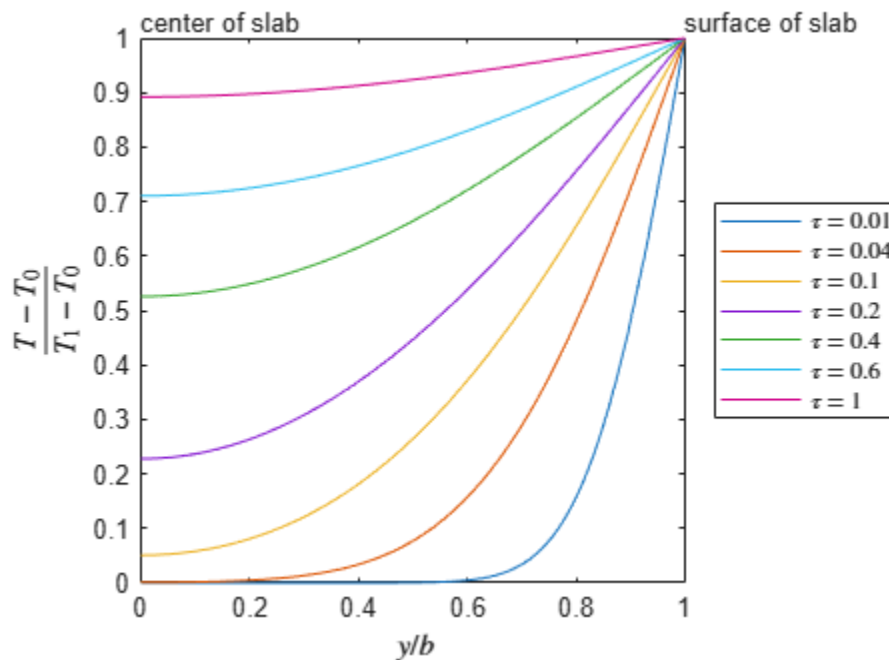
```
fplot(eta,1-Theta_sol_approx(eta,0.6),[0 1])
```



```
fplot(eta,1-Theta_sol_approx(eta,1),[0 1])

text(0,1.03,"center of slab")
text(1,1.03,"surface of slab")
xlabel("$y/b$",interpreter="latex")
ylabel("$\frac{T-T_0}{T_1-T_0}$",interpreter="latex")

legend("$\tau=0.01$","$\tau=0.04$","$\tau=0.1$","$\tau=0.2$","...
"$\tau=0.4$","$\tau=0.6$","$\tau=1$","...
interpreter="latex",Location="eastoutside")
hold off
```



Next, plot the temperature in the SI unit K as a function of the  $y$ -coordinate and time, where  $T_1 = 1000$  K,  $T_0 = 500$  K,  $b = 0.2$  m, and  $\alpha = 10^{-4}$  m<sup>2</sup>/s. Define these parameters and assume the units are in SI.

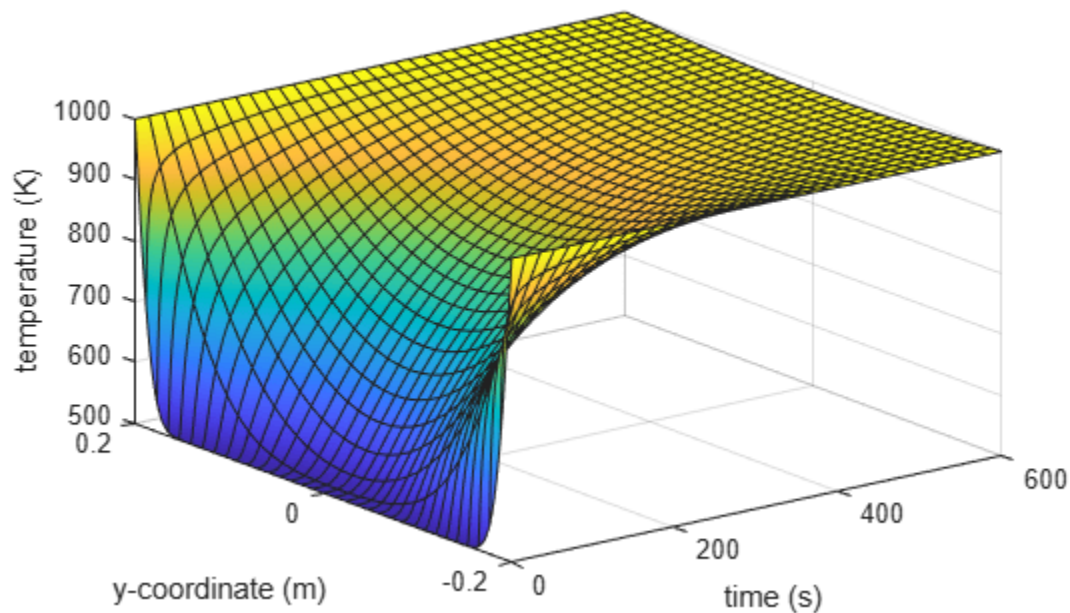
```
T1 = 1000;
T0 = 500;
b = 0.2;
alpha = 10^-4;
```

Define variables for the temperature  $T$ , coordinate  $y$ , and time  $t$ . From the dimensionless temperature solution in `Theta_sol`, find the temperature solution in K by substituting these parameter values. Also, replace the infinite summation terms with a summation term of up to  $k = 40$  to improve the performance when plotting this solution.

```
syms T y t
T_eqn = (T1-T)/(T1-T0) == subs(Theta_sol,{eta,tau},{y/b,alpha*t/b^2});
T_sol = isolate(T_eqn,T);
T_sol = subs(rhs(T_sol),Inf,40);
```

Create a surface plot of the temperature as a function of the y-coordinate and time. You can see that initially the slab is at 500 K with edges at 1000 K, and after about 600 s, the temperature across the slab reaches the steady-state value of 1000 K.

```
figure
fsurf(T_sol,[1 600 -b b])
xlabel("time (s)")
ylabel("y-coordinate (m)")
zlabel("temperature (K)")
```



## References

[1] Bird, R. Byron, Warren E. Stewart, and Edwin N. Lightfoot. *Transport Phenomena*, 2nd ed. (New York: Wiley, 2007), 376-378, Example 12.1-2.

## See Also

`dsolve` | `solve` | `int` | `symsum` | `subs` | `fplot` | `fsurf`

## Related Examples

- "Solve Differential Equation" on page 3-49
- "Integration" on page 3-186
- "Solve Partial Differential Equation of Nonlinear Heat Transfer" on page 3-275

## Solve Partial Differential Equation of Nonlinear Heat Transfer

This example shows how to solve a partial differential equation (PDE) of nonlinear heat transfer in a thin plate.

The plate is square, and its temperature is fixed along the bottom edge. No heat is transferred from the other three edges since the edges are insulated. Heat is transferred from both the top and bottom faces of the plate by convection and radiation. Because radiation is included, the problem is nonlinear.

The purpose of this example is to show how to represent the nonlinear PDE symbolically using Symbolic Math Toolbox™ and solve the PDE problem using finite element analysis in Partial Differential Equation Toolbox™. In this example, perform transient analysis and solve the temperature in the plate as a function of time. The transient analysis shows the time duration until the plate reaches its equilibrium temperature at steady state.

### Heat Transfer Equations for the Plate

The plate has planar dimensions 1 m by 1 m and is 1 cm thick. Because the plate is relatively thin compared with the planar dimensions, the temperature can be assumed to be constant in the thickness direction, and the resulting problem is 2-D.

Assume that convection and radiation heat transfers take place between the two faces of the plate and the environment with a specified ambient temperature.

The amount of heat transferred from each plate face per unit area due to convection is defined as

$$Q_c = h_c(T - T_a),$$

where  $T_a$  is the ambient temperature,  $T$  is the temperature at a particular  $x$  and  $y$  location on the plate surface, and  $h_c$  is a specified convection coefficient.

The amount of heat transferred from each plate face per unit area due to radiation is defined as

$$Q_r = \epsilon\sigma(T^4 - T_a^4),$$

where  $\epsilon$  is the emissivity of the face and  $\sigma$  is the Stefan-Boltzmann constant. Because the heat transferred due to radiation is proportional to the fourth power of the surface temperature, the problem is nonlinear.

The PDE describing the temperature in this thin plate is

$$\rho C_p t_z \frac{\partial T}{\partial t} - k t_z \nabla^2 T + 2Q_c + 2Q_r = 0$$

where  $\rho$  is the material density of the plate,  $C_p$  is its specific heat,  $t_z$  is its plate thickness,  $k$  is its thermal conductivity, and the factors of two account for the heat transfer from both of its faces.

### Define PDE Parameters

Set up the PDE problem by defining the values of the parameters. The plate is composed of copper, which has the following properties.

```

kThermal = 400;           % thermal conductivity of copper, W/(m-K)
rhoCopper = 8960;         % density of copper, kg/m^3
specificHeat = 386;       % specific heat of copper, J/(kg-K)
thick = 0.01;             % plate thickness in meters
stefanBoltz = 5.670373e-8; % Stefan-Boltzmann constant, W/(m^2-K^4)
hCoeff = 1;               % convection coefficient, W/(m^2-K)
tAmbient = 300;           % the ambient temperature
emiss = 0.5;              % emissivity of the plate surface

```

### Extract PDE Coefficients

Define the PDE in symbolic form with the plate temperature as a dependent variable  $T(t, x, y)$ .

```

syms T(t,x,y)
syms eps sig tz hc Ta rho Cp k
Qc = hc*(T - Ta);
Qr = eps*sig*(T^4 - Ta^4);
pdeeq = (rho*Cp*tz*diff(T,t) - k*tz*laplacian(T,[x,y]) + 2*Qc + 2*Qr)

pdeeq(t, x, y) =

```

$$2 \text{eps sig} \left( T(t, x, y)^4 - Ta^4 \right) - k \text{tz} \left( \frac{\partial^2}{\partial x^2} T(t, x, y) + \frac{\partial^2}{\partial y^2} T(t, x, y) \right) - 2 \text{hc} (Ta - T(t, x, y)) + Cp \rho \text{tz} \frac{\partial}{\partial t} T(t, x, y)$$

Now, create coefficients to use as inputs in the PDE model as required by Partial Differential Equation Toolbox. To do this, first extract the coefficients of the symbolic PDE as a structure of symbolic expressions using the `pdeCoefficients` function.

```

symCoeffs = pdeCoefficients(pdeeq,T,'Symbolic',true)

symCoeffs = struct with fields:
    m: 0
    a: 2*hc + 2*eps*sig*T(t, x, y)^3
    c: [2x2 sym]
    f: 2*eps*sig*Ta^4 + 2*hc*Ta
    d: Cp*rho*tz

```

Next, substitute the symbolic variables that represent the PDE parameters with their numeric values.

```

symVars = [eps sig tz hc Ta rho Cp k];
symVals = [emiss stefanBoltz thick hCoeff tAmbient rhoCopper specificHeat kThermal];
symCoeffs = subs(symCoeffs,symVars,symVals);

```

Finally, since the fields `symCoeffs` are symbolic objects, use the `pdeCoefficientsToDouble` function to convert the coefficients to the double data type, which makes them valid inputs for Partial Differential Equation Toolbox.

```

coeffs = pdeCoefficientsToDouble(symCoeffs)

coeffs = struct with fields:
    a: @makeCoefficient/coefficientFunction
    c: [4x1 double]
    m: 0
    d: 3.4586e+04
    f: 1.0593e+03

```

### Specify PDE Model, Geometry, and Coefficients

Now, using Partial Differential Equation Toolbox, solve the PDE problem using finite element analysis based on these coefficients.

First, create the PDE model with a single dependent variable.

```
numberOfPDE = 1;
model = createpde(numberOfPDE);
```

Specify the geometry for the PDE model—in this case, the dimension of the square.

```
width = 1;
height = 1;
```

Define a geometry description matrix. Create the square geometry using the `decsg` (Partial Differential Equation Toolbox) function. For a rectangular geometry, the first row contains 3, and the second row contains 4. The next four rows contain the x-coordinates of the starting points of the edges, and the four rows after that contain the y-coordinates of the starting points of the edges.

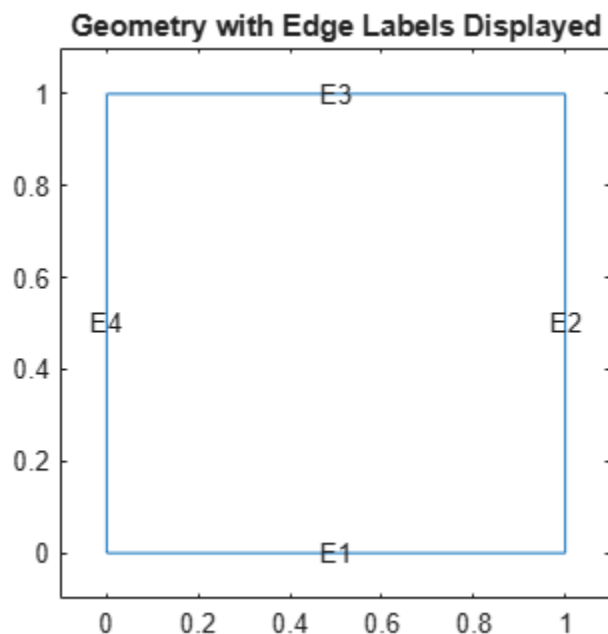
```
gdm = [3 4 0 width width 0 0 0 height height];
g = decsg(gdm, 'S1', ('S1'));
```

Convert the DECSG geometry into a geometry object and include it in the PDE model.

```
geometryFromEdges(model,g);
```

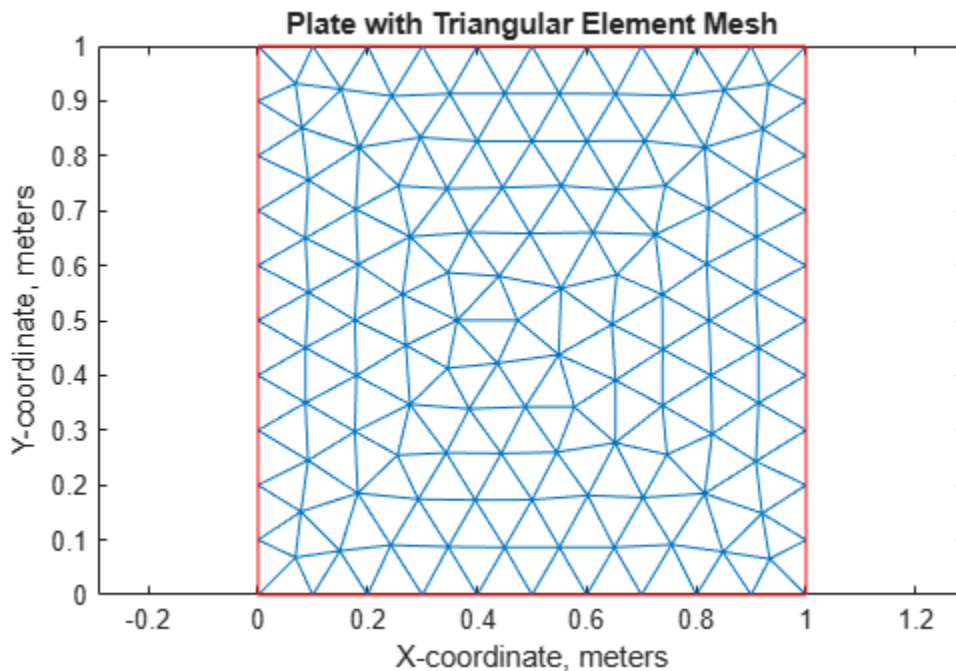
Plot the geometry and display the edge labels.

```
figure;
pdegplot(model, 'EdgeLabels', 'on');
axis([-0.1 1.1 -0.1 1.1]);
title('Geometry with Edge Labels Displayed');
```



Next, create the triangular mesh in the PDE model with a mesh size of approximately 0.1 in each direction.

```
hmax = 0.1; % element size
msh = generateMesh(model,'Hmax',hmax);
figure;
pdeplot(model);
axis equal
title('Plate with Triangular Element Mesh');
xlabel('X-coordinate, meters');
ylabel('Y-coordinate, meters');
```



Specify the coefficients in the PDE model.

```
specifyCoefficients(model,'m',coeffs.m,'d',coeffs.d, ...
    'c',coeffs.c,'a',coeffs.a,'f',coeffs.f);
```

### Find Transient Solution

Apply the boundary conditions. Three of the plate edges are insulated. Because a Neumann boundary condition equal to zero is the default in the finite element formulation, you do not need to set the boundary conditions on these edges. The bottom edge of the plate is fixed at 1000 K. Specify this using a Dirichlet condition on all nodes on the bottom edge (edge E1).

```
applyBoundaryCondition(model,'dirichlet','edge',1,'u',1000);
```

Specify the initial temperature to be 0 K everywhere, except at the bottom edge. Set the initial temperature on the bottom edge E1 to the value of the constant boundary condition, 1000 K.

```
setInitialConditions(model,0);
setInitialConditions(model,1000,'edge',1);
```

Define the time domain to find the transient solution of the PDE problem.

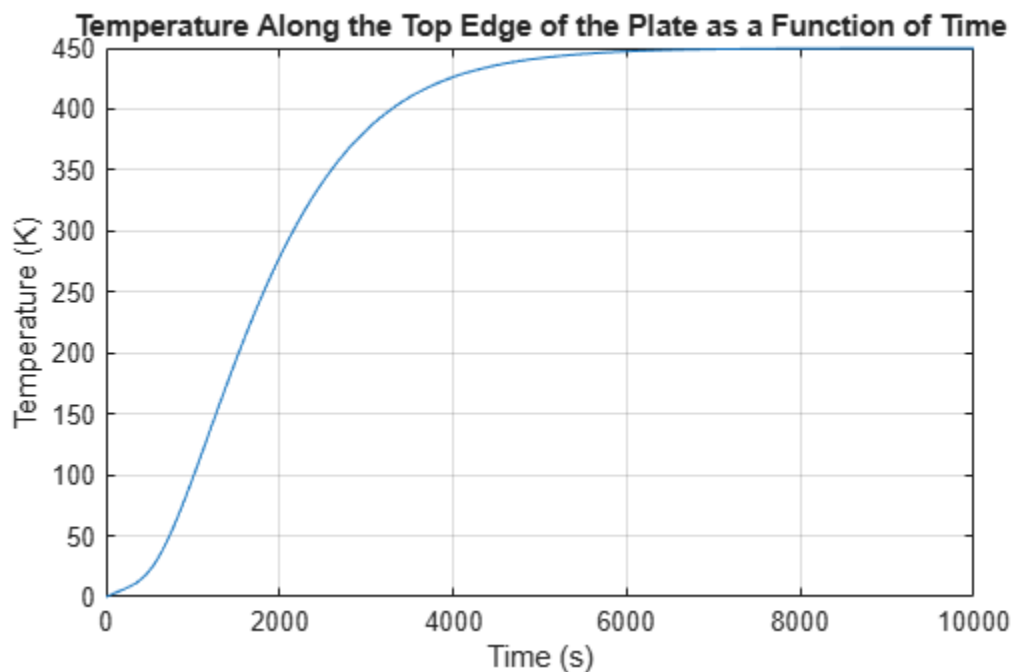
```
endTime = 10000;
tlist = 0:50:endTime;
```

Set the tolerance of the solver options.

```
model.SolverOptions.RelativeTolerance = 1.0e-3;
model.SolverOptions.AbsoluteTolerance = 1.0e-4;
```

Solve the problem using `solvepde`. Plot the temperature along the top edge of the plate as a function of time.

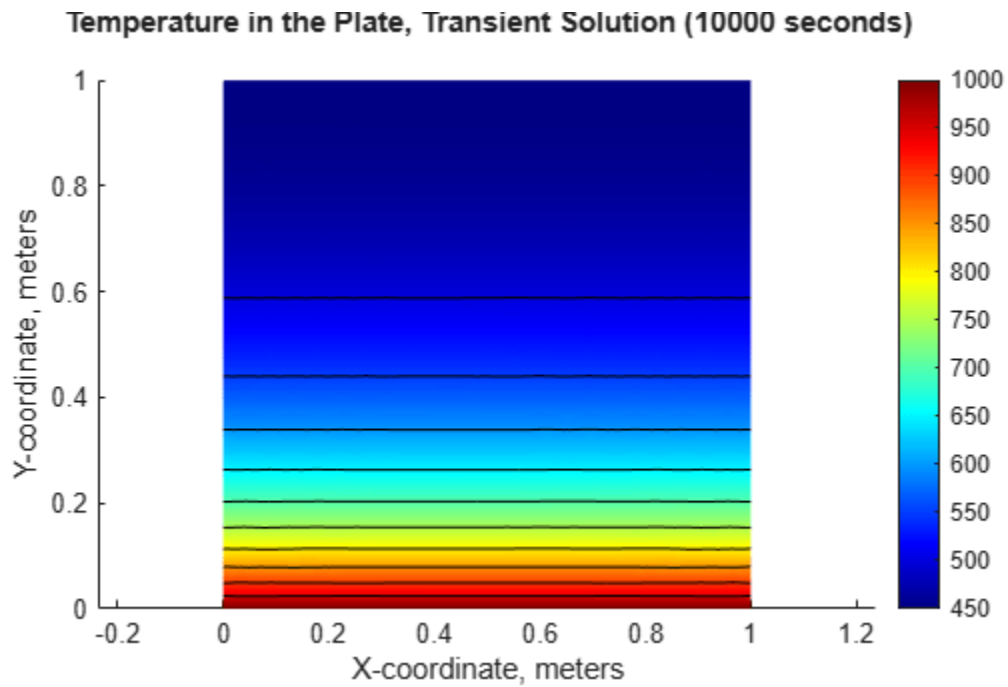
```
R = solvepde(model,tlist);
u = R.NodalSolution;
figure;
plot(tlist,u(3,:));
grid on
title 'Temperature Along the Top Edge of the Plate as a Function of Time'
xlabel 'Time (s)'
ylabel 'Temperature (K)'
```



Based on the plot, the transient solution starts to reach its steady state value after 6000 seconds. The equilibrium temperature of the top edge approaches 450 K after 6000 seconds.

Show the temperature profile of the plate after 10,000 seconds.

```
figure;
pdeplot(model,'XYData',u(:,end),'Contour','on','ColorMap','jet');
title(sprintf('Temperature in the Plate, Transient Solution (%d seconds)\n', ...
    tlist(1,end)));
xlabel 'X-coordinate, meters'
ylabel 'Y-coordinate, meters'
axis equal;
```



Show the temperature at the top edge at 10,000 seconds.

```
u(3,end)
```

```
ans =  
449.7914
```

## See Also

### Functions

`pdeCoefficients` | `pdeCoefficientsToDouble` | `subs`

## Related Examples

- “Perform Symbolic Computations” on page 1-29
- “Heating of Finite Slab” on page 3-267



## Derive Equations of Motion and Simulate Cart-Pole System

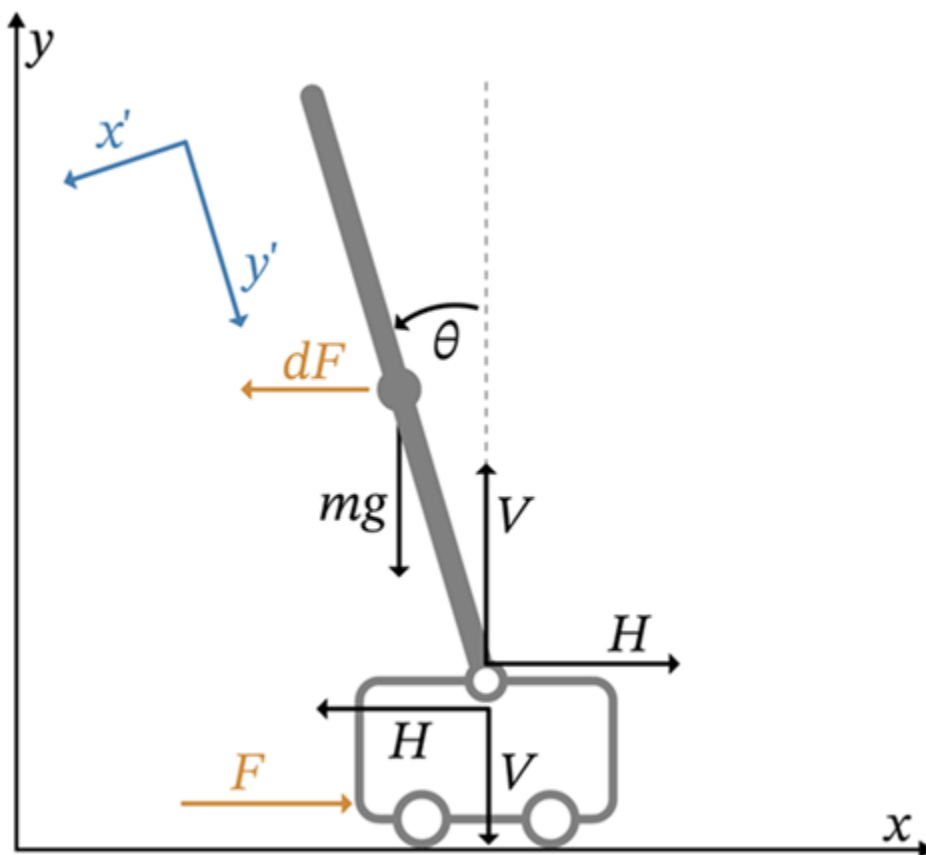
This example shows how to derive the equations of motion for the cart-pole system using Symbolic Math Toolbox™ and then simulate the cart-pole system using the `ode45` solver. In the later sections of the example, you explore how to derive the equations in other forms that can be used to numerically simulate the system (and validate the results) using different tools, such as Simulink®, Simscape™ Multibody™, and Robotics System Toolbox™.

The equations of motion are crucial for modeling and simulating the system in dynamical systems and control theory. The stabilization of the cart-pole system, also known as the inverted pendulum, is one of the most widely used benchmark problems in control design and reinforcement learning [1] on page 3-301.

### Define Cart-Pole System

In the general cart-pole system, the pole is an object with a given mass and moment of inertia. The pole can be a rod with uniform density, or it can be a massless rod with another point mass attached to its end (an inverted pendulum). The cart-pole system includes a special case of an inverted pendulum with an attached object that does not move. Assume a fluid friction force, which also covers the special case where there is no friction. You can extend the assumed fluid friction force to a more complex friction model when needed.

This figure illustrates the cart-pole system discussed in this example.



To derive the equations of motion following Newton's second law, first define the symbolic variables. These parameters and variables describe the cart-pole system:

- $t$  is the time.
- $g$  is the gravitational acceleration.
- $l$  is the distance from the pole-cart attachment to the pole's center of mass.
- $I$  is the moment of inertia of the pole (with respect to the rotation axis at the center of mass).
- $m$  is the mass of the pole.
- $M$  is the mass of the cart.
- $H$  and  $V$  are the horizontal and vertical forces on the pole, respectively. These forces form action and reaction pairs that act on both the pole and the cart. When acting on the pole,  $H$  is positive in the  $x$ -direction (right), while  $V$  is positive in the  $y$ -direction (up).
- $F$  and  $dF$  are external forces acting on the cart and the center of mass of the pole, respectively.
- $\text{theta}(t)$  is the angle that expresses the counterclockwise rotation of the pole with respect to the  $y$ -axis (so  $\text{theta} = 0$  represents the pole standing upright on the cart).
- $x(t)$  is the cart displacement with respect to its initial position along the  $x$ -direction.

In this example, the term  $-b \frac{d}{dt}x(t)$  denotes the friction force on the cart due to the fluid or air surrounding it. If the cart slides on a surface (as opposed to, for example, being on wheels), then you need to use a more accurate model of kinetic friction force.

For an example that uses a different angle notation and includes a pole length of  $2l$ , see [2] on page 3-301.

### Derive Equations of Motion

The equations that represent the cart-pole system can be expressed in various forms, depending on the preferred notation, convention, and calculation sequence. However, it is known that the literature contains inaccuracies (for example, see [2] on page 3-301 and [3] on page 3-301). This example uses an approach to deriving the equations that is similar to the first approach used in [3] on page 3-301, but this section also explains how the equations can be written as in [2] on page 3-301. Later, you solve these equations to determine how the pole angle  $\theta(t)$  and the cart displacement  $x(t)$  evolve over time given some initial conditions.

### Define Symbolic Variables

Define symbolic variables that represent the parameters and variables of the cart-pole system. Because the pole angle and cart displacement can change over time, define  $\text{theta}$  and  $x$  as functions of  $t$ .

```
syms t g l I m M H V F dF theta(t) x(t) b
```

Assume that some of the variables are positive. This simplification makes the representation of expressions involving these variables clearer and helps restrict the returned solutions to feasible domains.

```
assume([t g l m M b] > 0)
```

Also, assume that the other variables are real.

```
assume([I H V F dF], "real")
```

### Derive Equations of Motion in Horizontal Direction

Based on Newton's second law, define an equation to express the horizontal forces acting on the cart. All the forces acting in the x-direction are on the left side of the equation, while the cart's mass multiplied by its acceleration is on the right side.

$$\text{eq1} = F - H - b \cdot \text{diff}(x, 1) == M \cdot \text{diff}(x, 2)$$

$$\text{eq1}(t) =$$

$$-b \frac{\partial}{\partial t} x(t) + F - H = M \frac{\partial^2}{\partial t^2} x(t)$$

Define another equation to express the horizontal forces acting on the pole. Here, the position of the pole's center of mass is expressed in terms of x and theta, and the second derivative of this position (its acceleration) is equal to the total force acting on the pole's end in the horizontal direction.

$$\text{eq2} = H - dF == m \cdot \text{diff}(x - l \cdot \sin(\text{theta}), 2)$$

$$\text{eq2}(t) =$$

$$H - dF = m \left( \frac{\partial^2}{\partial t^2} x(t) + l \sin(\theta(t)) \left( \frac{\partial}{\partial t} \theta(t) \right)^2 - l \cos(\theta(t)) \frac{\partial^2}{\partial t^2} \theta(t) \right)$$

To obtain an equation without the force H, combine eq1 and eq2 into an equation in which the term H is eliminated.

$$\text{mainEq1} = \text{eliminate}([\text{eq1} \text{ eq2}], H) == 0$$

$$\text{mainEq1}(t) =$$

$$F - dF - m \frac{\partial^2}{\partial t^2} x(t) - b \frac{\partial}{\partial t} x(t) - M \frac{\partial^2}{\partial t^2} x(t) - l m \left( \sin(\theta(t)) \left( \frac{\partial}{\partial t} \theta(t) \right)^2 - \cos(\theta(t)) \frac{\partial^2}{\partial t^2} \theta(t) \right) = 0$$

The equation mainEq1 is the first main equation that links the second derivatives of x(t) and theta(t).

### Derive Forces and Moments Acting on Pole

Next, define an equation to express the forces on the pole along the direction normal to the pole (the \*x'-\*direction in the figure). Here, the sum of the forces along x' in the figure is expressed on the left side of the equation. The right side represents the acceleration of the pole's center of mass along x', expressed in terms of x(t) and theta(t).

$$\text{eq3} = -V \cdot \sin(\text{theta}) + m \cdot g \cdot \sin(\text{theta}) - H \cdot \cos(\text{theta}) + dF \cdot \cos(\text{theta}) == m \cdot l \cdot \text{diff}(\text{theta}, 2) - m \cdot \text{diff}$$

$$\text{eq3}(t) =$$

$$dF \cos(\theta(t)) - V \sin(\theta(t)) - H \cos(\theta(t)) + g m \sin(\theta(t)) = l m \frac{\partial^2}{\partial t^2} \theta(t) - m \cos(\theta(t)) \frac{\partial^2}{\partial t^2} x(t)$$

Define another equation to express the pole's angular momentum around its center of mass as a function of its moment of inertia. Here, the sum of moments around the pole's center of mass is equal to its rotational acceleration multiplied by its moment of inertia with respect to its center of mass.

$$\text{eq4} = V \cdot l \cdot \sin(\text{theta}) + H \cdot l \cdot \cos(\text{theta}) == I \cdot \text{diff}(\text{theta}, t, 2)$$

$$\text{eq4}(t) =$$

$$H l \cos(\theta(t)) + V l \sin(\theta(t)) = I \frac{\partial^2}{\partial t^2} \theta(t)$$

To obtain an equation without the forces  $H$  and  $V$ , combine eq3 and eq4 into an equation in which both terms  $H$  and  $V$  are eliminated.

```
mainEq2 = eliminate([eq3 eq4],[H V]) == 0
```

```
mainEq2(t) =
```

$$dF l \cos(\theta(t)) - I \frac{\partial^2}{\partial t^2} \theta(t) - l^2 m \frac{\partial^2}{\partial t^2} \theta(t) + l m \cos(\theta(t)) \frac{\partial^2}{\partial t^2} x(t) + g l m \sin(\theta(t)) = 0$$

The equation mainEq2 is the second main equation that links the second derivatives of  $x(t)$  and  $\theta(t)$ .

### Simple Pendulum Equation

If you set the second derivative of the cart position to zero in the second main equation, the resulting equation represents an inverted pendulum system (a pole attached to a cart that does not move).

```
eqInvPendulum = isolate(subs(mainEq2,diff(x,t,2),0),theta)
```

```
eqInvPendulum =
```

$$dF l \cos(\theta(t)) - I \frac{\partial^2}{\partial t^2} \theta(t) - l^2 m \frac{\partial^2}{\partial t^2} \theta(t) + g l m \sin(\theta(t)) = 0$$

A simple pendulum model (with a clockwise positive angle convention), where no force is applied to the pole ( $dF = 0$ ), is also derived in the example “Simulate the Motion of the Periodic Swing of a Pendulum” on page 4-35 and implemented in Simscape Multibody in the example “Model a Simple Pendulum” (Simscape Multibody).

### Convert Equations to State-Space Representation

The first and second main equations in mainEq1 and mainEq2 describe the dynamics of the cart-pole system. To obtain a state-space representation of the system dynamics, use odeToVectorField on the two main equations. The odeToVectorField function converts higher-order differential equations to a system of first-order differential equations in a vector of state variables

$Y(t) = [Y_1(t) \ Y_2(t) \ \dots \ Y_n(t)]^T$ . By reducing the order of the equations, you can generate numeric functions that simulate the system in MATLAB and Simulink. You can then find the numeric solutions of the system by using the ode45 function.

```
[ssEqs,states] = odeToVectorField([mainEq1 mainEq2])
```

```
ssEqs =
```

$$\begin{pmatrix} Y_2 \\ \frac{l \left( M dF \cos(Y_1) + F m \cos(Y_1) + g m^2 \sin(Y_1) - b m \cos(Y_1) Y_4 + M g m \sin(Y_1) - l m^2 \cos(Y_1) \sin(Y_1) Y_2^2 \right)}{\sigma_1} \\ Y_4 \\ - \frac{I dF - F I + I b Y_4 - F l^2 m + dF l^2 m + l^3 m^2 \sin(Y_1) Y_2^2 - dF l^2 m \cos(Y_1)^2 + b l^2 m Y_4 - g l^2 m^2 \cos(Y_1) \sin(Y_1) + I l m \sin(Y_1)}{\sigma_1} \end{pmatrix}$$

where

$$\sigma_1 = -l^2 m^2 \cos(Y_1)^2 + l^2 m^2 + M l^2 m + I m + I M$$

```
states =
```

$$\begin{pmatrix} \theta \\ D\theta \\ x \\ Dx \end{pmatrix}$$

The first output, `ssEqs`, shows the four first-order differential equations  $[\dot{Y}_1(t) \ \dot{Y}_2(t) \ \dot{Y}_3(t) \ \dot{Y}_4(t)]^T$  in terms of four state variables  $[Y_1(t) \ Y_2(t) \ Y_3(t) \ Y_4(t)]^T$ . The second output, `states`, indicates the relationship between the state variables and the degrees of freedom, `theta(t)` and `x(t)`, which can be expressed as

$$Y_1(t) = \theta(t), \quad Y_2(t) = \frac{\partial}{\partial t}\theta(t) = \dot{\theta}(t), \quad Y_3(t) = x(t), \quad Y_4(t) = \frac{\partial}{\partial t}x(t) = \dot{x}(t).$$

For more information on state-space representations of linear systems within system identification and control settings, see “What Are State-Space Models?” (System Identification Toolbox) and “State-Space Realizations” (Control System Toolbox), respectively.

Next, convert the symbolic form of the system of differential equations to a MATLAB® function by using `matlabFunction`. Specify the `Vars` name-value argument as a string array that represents all quantities in the equations. The generated anonymous function computes the time derivative of the vector `Y(t)` given the current value of time `t` for specified constant values of other known parameters (for more information, see “Anonymous Functions”). Although the generated function does not explicitly use `t` (the time derivatives of the state variables are not explicitly time dependent), you can still specify `t` as an argument for a more general case.

```
Ydot = matlabFunction(ssEqs,Vars=["t","Y","F","dF","I","M","b","g","l","m"]);
```

You can use this generated function to evaluate the time derivative of the state variables given some initial conditions and a set of parameters. For example, set the initial values of all state variables to zero, the forces on the cart and pole to 0.2 N, the moment of inertia to zero, the cart mass to 1 kg, the friction coefficient to zero, the gravitational acceleration to 9.81 m/s<sup>2</sup>, the length from the pole-cart attachment to the pole's center of mass to 0.5 m, and the pole mass to 0.1 kg.

```
Ydot_init = Ydot(0,[0 0 0 0]',0.2,0.2,0,1,0,9.81,0.5,0.1)
```

```
Ydot_init = 4x1
```

```

    0
4.4000
    0
0.2000
```

## Find Open-Loop Solution of Cart-Pole System

In this section, you express the open-loop solution of the cart-pole system in MATLAB, Simulink, and Robotics System Toolbox and find the solution by using the `ode45` solver.

An open-loop solution to the system of differential equations is a solution in which the external forces acting on the system, `F` and `dF`, are independent of the state variables (without any feedback from the state variables of the system). In other words, the external forces do not depend on the state of the system. The forces follow a predefined model without the need to adjust based on actual output or conditions of the system.

### Find Open-Loop Solution in MATLAB

To simulate the system in open loop from 0 to 10 seconds, you can use `ode45`. Because the first argument of `ode45` must be a function that accepts exactly two input arguments, the time and the state vectors (`t` and `Y`), specify another function that takes exactly these two input arguments with defined parameters. Define the other function as `Ydot_eval` that evaluates `Ydot` using specified constant values for all other parameters outside `t` and `Y`. For example, specify the values of `F` as 0.2, `dF` as 0, `I` as 0.1/12, `M` as 1, `b` as 0, `g` as 9.81, `l` as 0.5, and `m` as 0.1.

```
Fval = 0.2;
dFval = 0;
Ival = 0.1/12;
Mval = 1;
bval = 0;
gval = 9.81;
lval = 0.5;
mval = 0.1;
Ydot_eval = @(t,Y) Ydot(t,Y,Fval,dFval,Ival,Mval,bval,gval,lval,mval)
```

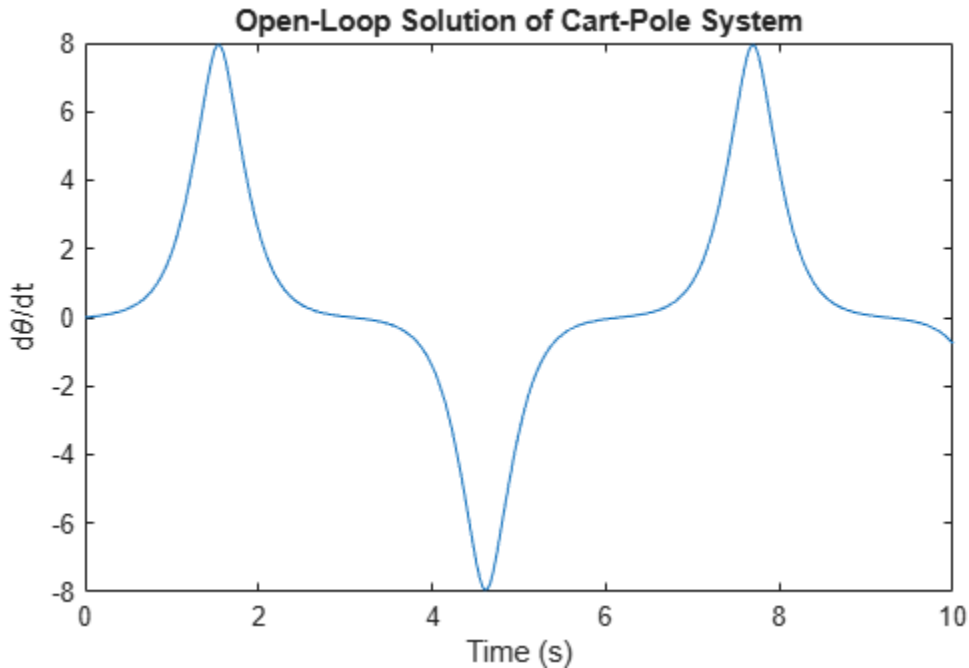
```
Ydot_eval = function_handle with value:
    @(t,Y)Ydot(t,Y,Fval,dFval,Ival,Mval,bval,gval,lval,mval)
```

Define an initial condition where all four state variables are zero. Find the open-loop solution using `ode45`. Because the system starts from an unstable equilibrium, to minimize numerical errors, pass an `odeset` options structure specifying low relative and absolute error tolerances.

```
Y_init = zeros(4,1);
Ysol = ode45(Ydot_eval,[0 10],Y_init,odeset(RelTol=1e-12,AbsTol=1e-12));
```

Plot the solution for the second state variable, which is the pole's angular speed  $\dot{\theta}(t)$ , as a function of time.

```
fig1 = figure;
plot(Ysol.x,Ysol.y(2,:))
xlabel("Time (s)")
ylabel("d\theta/dt")
title("Open-Loop Solution of Cart-Pole System")
```



Note that to find analytical solutions of linear ordinary differential equations that can be expressed in closed-form mathematical expressions, you can use the `dsolve` function. For an example, see “Simulate the Motion of the Periodic Swing of a Pendulum” on page 4-35. However, for nonlinear ordinary differential equations, analytical solutions often do not exist or they exist in formulas that are too long to allow for useful insights into how the solutions interact with all quantities.

### Find Open-Loop Solution in Simulink

Simulink provides a block diagram environment for multidomain simulation that is especially useful for designing and simulating systems containing several interacting components that can be represented by both continuous and discrete-time systems (or systems using different sampling rates). There are several ways to express the cart-pole system in Simulink, which will be discussed in the next sections.

One way to build a Simulink model that express the cart-pole system is to use the state-space equations `ssEqs` obtained previously. First, create a new empty model named `cp_sim`.

```
new_system("cp_sim")
```

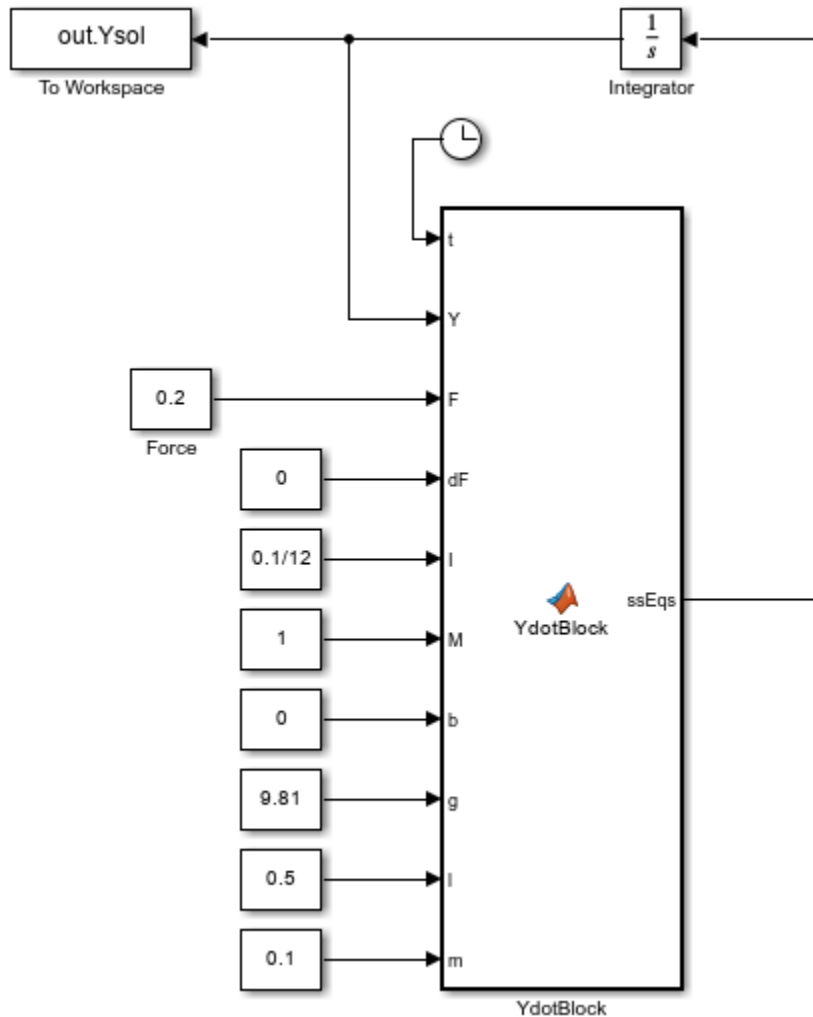
Open the model.

```
open_system("cp_sim")
```

Use the `matlabFunctionBlock` function to generate a Simulink block named `YdotBlock` that calculates the time derivatives of the state variables, with input ports for the current time, the current state variables, and the specified values of the other system parameters. Specify the name and path of the generated block in the `cp_sim` model as the first argument of `matlabFunctionBlock`, the state-space equations as the second argument, and the block input quantities as the third argument. Even if the time derivatives of the state variables do not explicitly depend on time, for this example, include the time variable as a block input.

```
matlabFunctionBlock("cp_sim/YdotBlock",ssEqs,Vars=["t","Y","F","dF","I","M","b","g","l","m"]);
```

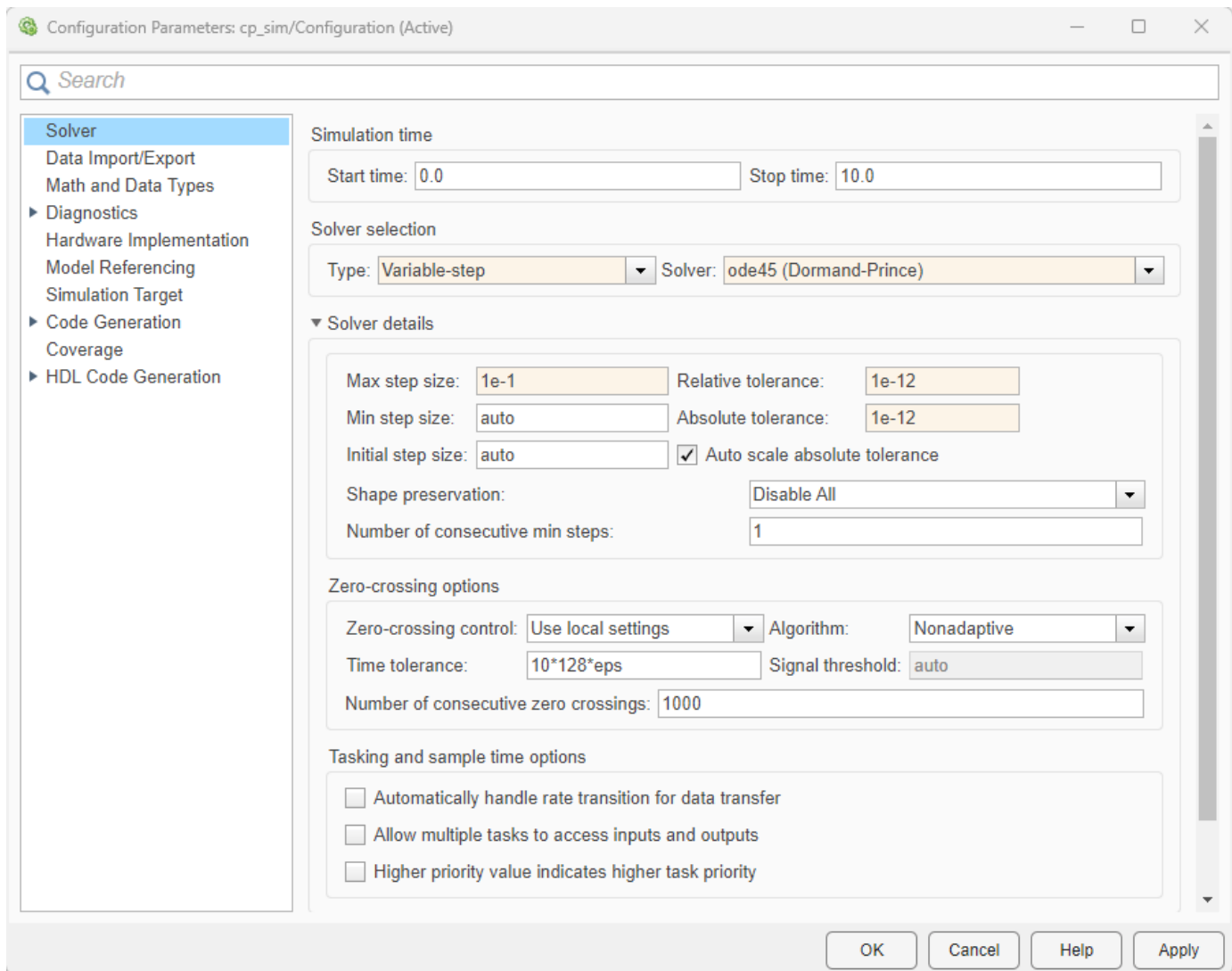
Next, manually add blocks that specify the current time, current state variables, and the values of the other system parameters as inputs to YdotBlock. Create the additional Clock block, Integrator block, and Constant blocks. Add a To Workspace block that collects the values of the state variables into the workspace variable `out.Ysol`. Connect the blocks as shown in the following figure. For this example, the model with the generated block and additional blocks is attached in the file `cp_sim.slx`.



The Integrator block, with the label  $\frac{1}{s}$ , takes the time derivative of the state vector and returns the current value of the state vector `Y`. The Integrator block uses an integration algorithm that can be configured in the **Solver** pane of the Configuration Parameters dialog box. To open the Configuration Parameters dialog box, click **Model Settings** on the **Modeling** tab. Here, the initial condition of the Integrator block is set to zero.

As discussed in the previous section, because this system starts from an unstable equilibrium, select the `ode45` integration and specify low relative and absolute error tolerances to minimize numerical errors.





Finally, simulate the cart-pole system and find the open-loop solution. You can then plot the solution in `out.Ysol` to verify that the results are identical to those obtained using `ode45` in previous subsection.

### Find Open-Loop Solution in Robotics System Toolbox

You can also represent and simulate the cart-pole system in Robotics System Toolbox.

First, create a tree-structured object by using `rigidBodyTree` (Robotics System Toolbox) and define the gravity.

```
rbt = rigidBodyTree(DataFormat="column");
rbt.Gravity = [0 -9.81 0];
```

Create a rigid body object for the cart by using `rigidBody` (Robotics System Toolbox) and define the inertial properties of the cart.

```

cart = rigidBody("cart");
cart.Mass = 1;
cart.Inertia = zeros(1,6);

```

Create a prismatic joint by using `rigidBodyJoint` (Robotics System Toolbox). This joint attaches the cart to the rigid body tree and allows it to move along the x-axis.

```

cart.Joint = rigidBodyJoint("j1","prismatic");
cart.Joint.JointAxis = [1 0 0];

```

Add the cart to the rigid body tree by using `addBody` (Robotics System Toolbox).

```

addBody(rbt, cart, rbt.BaseName);

```

Next, create a rigid body object for the pole and its inertial properties. To follow the base coordinate system of the rigid body tree model, the angular position of the pole (counterclockwise positive) is zero when the pole is in the horizontal position. The pole length spans the x-axis with its center of mass at 0.5 m. In this coordinate system, the moment of inertia of the pole along the y- and z-axes, with respect to the attachment point, is  $\frac{1}{3}m \cdot (2l)^2 = \frac{1}{3}0.1 \cdot 1^2 = \frac{1}{30}$ .

```

rod = rigidBody("rod");
rod.Mass = 0.1;
rod.CenterOfMass = [0.5 0 0];
rod.Inertia = [0 1/30 1/30 0 0 0];

```

Create a revolute joint that attaches the pole to the cart. Specify the fixed transformation for this joint that rotates the pole's coordinate system by  $\pi/2$  along the z-axis. This rotation brings the pole's coordinate system in line with the one shown at the beginning of this example.

```

rod.Joint = rigidBodyJoint("j2","revolute");
setFixedTransform(rod.Joint,eul2tform([pi/2 0 0]));

```

Add the pole to the rigid body tree.

```

addBody(rbt, rod, cart.Name);

```

Next, define the helper function `fd` that takes a vector `q` containing the four state variables (cart's position, pole's position, cart's velocity, and pole's angular velocity) and returns the vector `dqdt` containing the time derivatives of `q`. This function uses the `forwardDynamics` (Robotics System Toolbox) function to calculate the joint accelerations, given the rigid body tree object, the joint positions and velocities, and the external force. You use this helper function as the first argument of the `ode45` function when solving for the dynamics of the system. Specify the external force `F` acting on the cart to be 0.2 N. To simulate the system in a closed loop, you can set `F` as a function of `q`.

```

function dqdt = fd(~,q,rbt)

% Extract joint position, velocity and forces.
jointpos = q(1:2);
jointvel = q(3:4);
jointtau = zeros(2,1);

% Define external forces.
fext = externalForce(rbt,"cart",[0 0 0 0.2 0 0]);

% Calculate double derivatives of x and theta.
qdd = forwardDynamics(rbt,jointpos,jointvel,jointtau,fext);

```

```
% Assemble state vector.
dqdt = [jointvel; qdd];
```

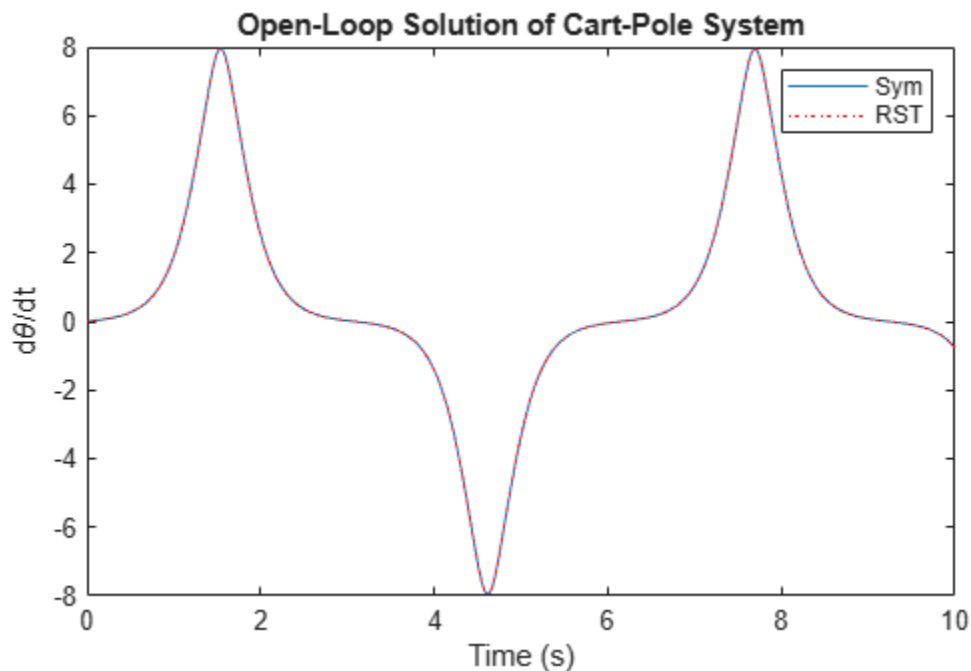
```
end
```

Use `ode45` to simulate the system in open loop from 0 to 10 seconds, starting with an initial condition where all four state variables are zero. Because the first argument of `ode45` must be a function with exactly two input arguments (time and the state vector), you cannot call `fd` directly when using `ode45`. Instead, define an anonymous function that calls `fd` with a specified rigid body tree `rbt`. Because the system starts from an unstable equilibrium, to minimize numerical errors, pass an `odeset` options structure specifying low relative and absolute error tolerances.

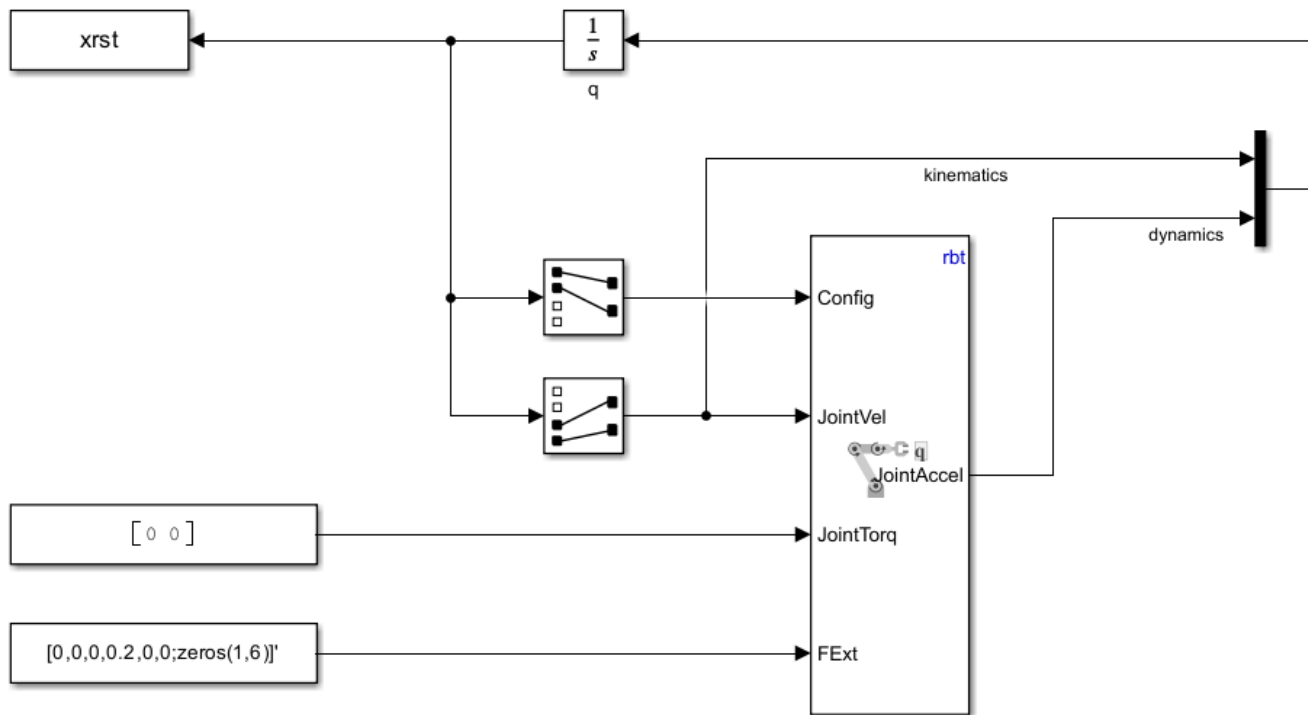
```
Qsol = ode45(@(t,q) fd(t,q,rbt),[0 10],zeros(1,4),odeset(MaxStep=0.1,AbsTol=1e-12,RelTol=1e-12))
```

Plot the simulation results on the figure from the Find Open-Loop Solution in MATLAB on page 3-286 section. The solution for the angular velocity perfectly overlaps with the solution of the equations of motion derived using Symbolic Math Toolbox.

```
figure(fig1)
hold on
plot(Qsol.x,Qsol.y(4,:), "r:")
legend("Sym", "RST")
hold off
```



As an alternative to `ode45`, you can also use the Forward Dynamics (Robotics System Toolbox) block to simulate the system in Simulink. This block uses the rigid body tree `rbt` as an input parameter, takes the current positions and velocities of the joint as inputs, takes the external torques and forces acting on the bodies as inputs, and returns the joint accelerations as the output. You can also simulate a closed-loop system by replacing the constant force with a force that is a function of the joint positions and velocities.



### Find Closed-Loop Solution of Cart-Pole System

A closed-loop control system uses feedback to control the states or outputs of a dynamical system. The closed-loop solution exists when the external forces acting on the system are functions of the state variables, which include the linear and angular positions and their velocities. In this section, you use Simulink Control Design™ to linearize the cart-pole system around the analysis points and use Control System Toolbox™ to compute the optimal gain matrix. Then you rewrite the external force as functions of the state variables and find the closed-loop solution of the cart-pole system by using `ode45`.

### Find Operating Points, Linearization, and Control Synthesis

If your dynamical system is represented by a Simulink model, you can use an array of tools to find specific operating points, linearize the system around these points, and design controllers that stabilize and guide the systems from one operating point to another. For more information, see “About Operating Points” (Simulink Control Design), `findopOptions` (Simulink Control Design), and `trim` (Simulink).

In the `cp_sim` model, set the force acting on the cart to zero.

```
set_param("cp_sim/Force", "Value", "0")
```

Define input and output points of the system for linearization. Create linear analysis points using `linio` (Simulink Control Design). Add an input corresponding to the force on the cart (the output of the Force block), and add an output corresponding to the current state vector (the output of the integrator block).

```
io(1) = linio("cp_sim/Force", 1, "input");
io(2) = linio("cp_sim/Integrator", 1, "output");
```

Use `linearize` (Simulink Control Design) to linearize the model from the selected input to the selected output around the analysis points defined by the initial values of the state variables, which are zero, and the input force, which is also zero.

```
lsys = linearize("cp_sim",io);
```

Display the state and input matrices.

```
lsys.A
```

```
ans = 4×4
```

```

      0      1.0000      0      0
15.7917      0      0      0
      0      0      0      1.0000
 0.7178      0      0      0

```

```
lsys.B
```

```
ans = 4×1
```

```

      0
 1.4634
      0
 0.9756

```

For more information about linearizing dynamical systems, see “Linearize Nonlinear Models” (Simulink Control Design).

Calculate the eigenvalues of the state matrix.

```
eig(lsys.A)
```

```
ans = 4×1
```

```

      0
      0
 3.9739
-3.9739

```

One of these eigenvalues is positive, which means that the linearized system is unstable. In other words, the cart-pole system is unstable around the selected operating point. When the pole is in the upright position, a small disturbance leads to the pole falling and the cart moving.

Find the optimal gain matrix or state feedback by using the `lqr` (Control System Toolbox) function. Specify the weight matrices for the states and inputs as the third and fourth input arguments of `lqr`, respectively.

```
K = lqr(lsys.A,lsys.B,diag([10,10,0.1,1]),0.1)
```

```
K = 1×4
```

```

49.0837   15.8752   -1.0000   -4.2198

```

For more information about designing control for linear and nonlinear systems, see “Control System Modeling with Model Objects” (Control System Toolbox), “What Is Model Predictive Control?” (Model Predictive Control Toolbox), and “Reinforcement Learning for Control Systems Applications” (Reinforcement Learning Toolbox).

### Find Closed-Loop Solution Using ode45

Now, find the closed-loop solution of the cart-pole system using `ode45`, incorporating the optimal gain matrix you previously calculated. Replace the force `F` with the scalar inner product between `-K` and the vector containing the four state variables in the two main equations. Then compute the state-space equations for the closed-loop system.

```
[ssEqsCl,statesCl] = odeToVectorField(subs([mainEq1; mainEq2],F, ...
    -K*[theta(t); diff(theta(t),t); x(t); diff(x(t),t)]));
```

Display the state variables of the system in terms of the degrees of freedom, `theta(t)` and `x(t)`.

`statesCl`

```
statesCl =
    (
      x
      Dx
      θ
      Dtheta
    )
```

Note that the order of the returned state variables is different from the order of state variables returned in the open-loop state-space equations.

Substitute the value of `dF` with 0 and other constant parameters in the state-space equations with the values defined previously.

```
ssEqsCl = subs(ssEqsCl,[dF,I,M,b,g,l,m],[0,Ival,Mval,bval,gval,lval,mval]);
```

Generate a MATLAB function that calculates the time-derivative of the state vector given the current time and state vector.

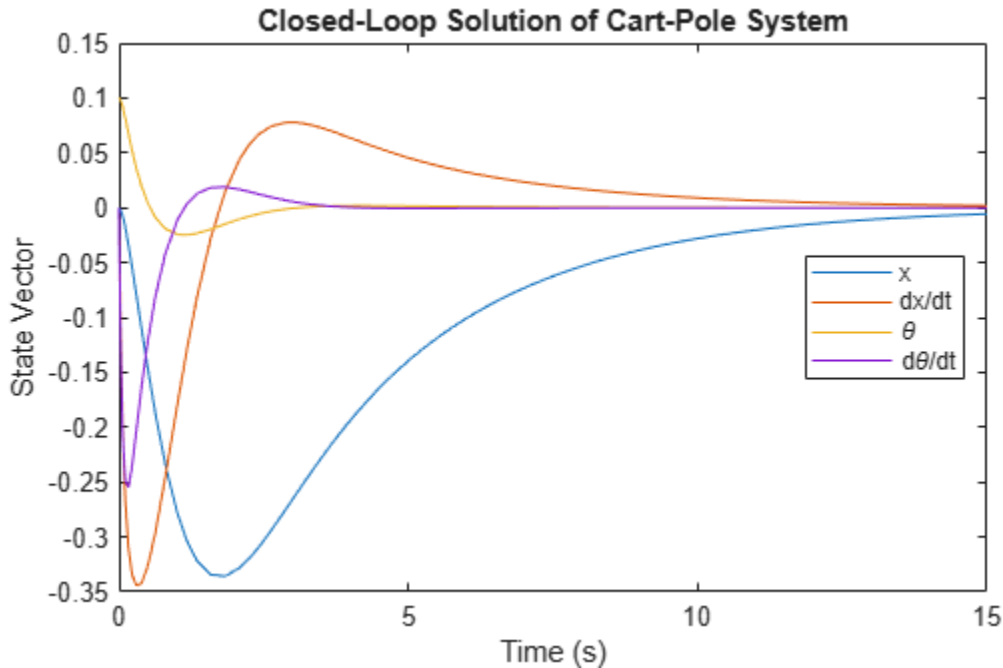
```
YdotCl = matlabFunction(ssEqsCl,vars=["t","Y"]);
```

Simulate the system for 15 seconds, starting from an initial `theta` value of 0.1.

```
YsolCl = ode45(@(t,Y) YdotCl(t,Y),[0 15],[0 0 0.1 0]);
```

Plot the closed-loop solution of the system, which is the time evolution of the state variables.

```
figure;
plot(YsolCl.x,YsolCl.y)
xlabel("Time (s)")
ylabel("State Vector")
title("Closed-Loop Solution of Cart-Pole System")
legend("x","dx/dt","θ","dθ/dt",Location="east")
```



Even though the initial pole angle is 0.1, the force applied to the cart is able to bring the system back into the upright position. In other words, the upright position for this closed-loop system is a stable equilibrium.

### Convert Equations of Motion to Mass-Matrix Representation

In addition to the state-space representation, you can rewrite the two main equations that describe the dynamics of the cart-pole system using mass-matrix representation. In this representation, you isolate the second-order derivatives of the horizontal and the angular positions of the system on the left side of the equations of motion. In other words, you express the cart-pole system equations in a form similar to Newton's second law, which is  $M(q) \ddot{q} = F(q, \dot{q}, t)$ , where  $q(t) = [x(t), \theta(t)]^T$  and  $M(q)$  is a 2-by-2 mass matrix that is a function of  $q$ .

For a system of ordinary differential equations, Symbolic Math Toolbox provides the `massMatrixForm` function to convert the system to first-order differential algebraic equations (DAEs). However, this function converts the system into the form  $M(t, y)\dot{y} = F(t, y)$ . For this reason, to use `massMatrixForm` to convert the system into the form with second-order derivatives, define the symbolic functions `Dx(t)` and `Dtheta(t)` to represent  $\dot{x}(t)$  and  $\dot{\theta}(t)$ , respectively. Replace the first derivative terms in the main equations with these symbolic functions.

```
syms Dx(t) Dtheta(t)
mainEq1subs = subs(mainEq1,[diff(x,t) diff(theta,t)],[Dx Dtheta]);
mainEq2subs = subs(mainEq2,[diff(x,t) diff(theta,t)],[Dx Dtheta]);
```

Convert the main equations into the form  $M(t, y)\dot{y} = F(t, y)$ , where  $y(t) = [\dot{x}(t), \dot{\theta}(t)]^T$ . Without loss of generality, consider the negative of the two main equations and convert them to the mass-matrix representation.

```
[matrixM,matrixF] = massMatrixForm([-mainEq1subs -mainEq2subs],[Dx Dtheta])
matrixM =
```

$$\begin{pmatrix} M + m & -l m \cos(\theta(t)) \\ -l m \cos(\theta(t)) & m l^2 + I \end{pmatrix}$$

$$\text{matrixF} = \begin{pmatrix} -l m \sin(\theta(t)) \text{Dtheta}(t)^2 + F - dF - b \text{Dx}(t) \\ dF l \cos(\theta(t)) + g l m \sin(\theta(t)) \end{pmatrix}$$

Next, in the matrix  $F$ , replace the symbolic functions  $\text{Dx}(t)$  and  $\text{Dtheta}(t)$  with the first-order derivatives that they represent, namely,  $\dot{x}(t)$  and  $\dot{\theta}(t)$ .

$$\text{matrixF} = \text{subs}(\text{matrixF}, [\text{Dx Dtheta}], [\text{diff}(x, t) \text{ diff}(\theta, t)])$$

$$\text{matrixF} = \begin{pmatrix} -l m \sin(\theta(t)) \left( \frac{\partial}{\partial t} \theta(t) \right)^2 - b \frac{\partial}{\partial t} x(t) + F - dF \\ dF l \cos(\theta(t)) + g l m \sin(\theta(t)) \end{pmatrix}$$

Using the mass-matrix representation, you can derive the equations for the linear acceleration of the two degrees of freedom as functions of their velocities, positions, and all other parameters. In other words, you can represent the equations of motion as  $\ddot{q} = M^{-1}(q) F(q, \dot{q}, t)$ .

$$\text{D2q\_eqn} = [\text{diff}(x(t), t, 2); \text{diff}(\theta(t), t, 2)] == \text{inv}(\text{matrixM}) * \text{matrixF}$$

$$\text{D2q\_eqn} = \begin{pmatrix} \frac{\partial^2}{\partial t^2} x(t) = \frac{l m \cos(\theta(t)) \sigma_3}{\sigma_1} - \frac{(m l^2 + I) \sigma_2}{\sigma_1} \\ \frac{\partial^2}{\partial t^2} \theta(t) = \frac{(M + m) \sigma_3}{\sigma_1} - \frac{l m \cos(\theta(t)) \sigma_2}{\sigma_1} \end{pmatrix}$$

where

$$\sigma_1 = -l^2 m^2 \cos(\theta(t))^2 + l^2 m^2 + M l^2 m + I m + I M$$

$$\sigma_2 = l m \sin(\theta(t)) \left( \frac{\partial}{\partial t} \theta(t) \right)^2 + b \frac{\partial}{\partial t} x(t) - F + dF$$

$$\sigma_3 = dF l \cos(\theta(t)) + g l m \sin(\theta(t))$$

Define the variable  $\text{q\_acc}$  to represent the linear acceleration, which is equal to the right side of the previous equation. You can use this variable to formulate the time evolution of the two degrees of freedom across other engineering applications.

$$\text{q\_acc} = \text{rhs}(\text{D2q\_eqn});$$

### Cart-Pole System in Other Examples

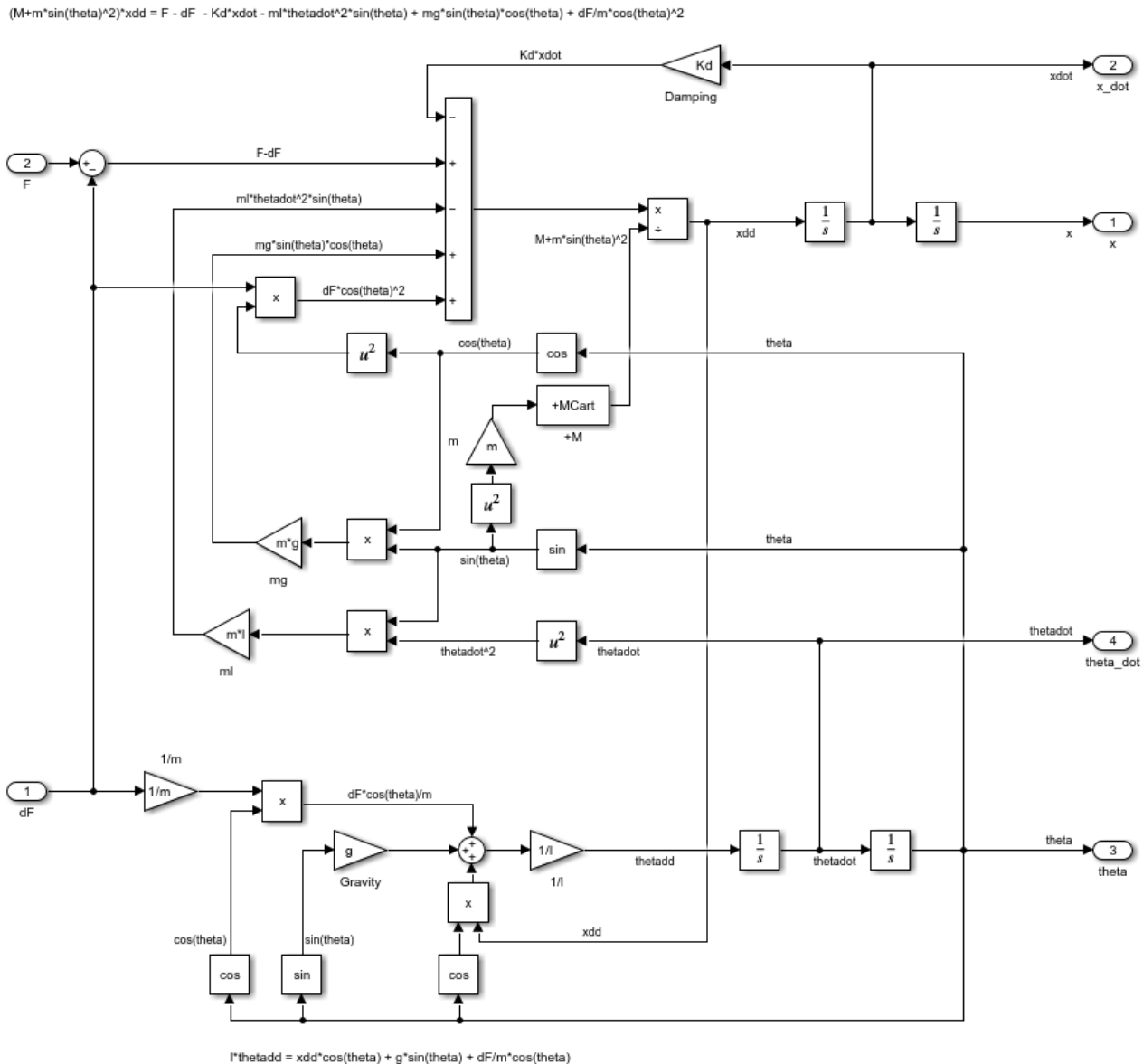
#### Examples of Cart-Pole System in Model Predictive Control Toolbox

Several examples in the Model Predictive Control Toolbox™ documentation show how to control a cart-pole system using linear and nonlinear model predictive controls. For more details, see “MPC Control of an Inverted Pendulum on a Cart” (Model Predictive Control Toolbox), “Explicit MPC Control of an Inverted Pendulum on a Cart” (Model Predictive Control Toolbox), “Time-Varying MPC Control of an Inverted Pendulum on a Cart” (Model Predictive Control Toolbox), and “Gain-Scheduled MPC Control of Inverted Pendulum on Cart” (Model Predictive Control Toolbox).



For these examples, the pole is represented by a point mass with a moment of inertia of zero with respect to the center of mass. The quantities involved in the system equations are represented with Simulink blocks. The two inputs are the external forces acting on the pole and cart, and the four outputs are the cart position and velocity, as well as the pole's angular position and velocity.

This Simulink model represents the system.



Derive the acceleration of the cart, indicated by the  $\ddot{x}$  label in the model, by using the first equation of the mass-matrix representation defined previously. The blocks leading up to  $\ddot{x}$  represent the terms on the right side of this equation.

```
xdd_eqn = diff(x(t),t,2) == simplify(subs(q_acc(1),I,0),Steps=100)
```

```
xdd_eqn =
```

$$\frac{\partial^2}{\partial t^2} x(t) = \frac{F - b \frac{\partial}{\partial t} x(t) + \frac{M dF}{m} + \frac{g m \sin(2 \theta(t))}{2} - l m \sin(\theta(t)) \left( \frac{\partial}{\partial t} \theta(t) \right)^2}{m \sin(\theta(t))^2 + M} - \frac{dF}{m}$$

Derive the angular acceleration of the pole, indicated by the `thetadd` label in the model, by using the second main equation defined at the beginning of this example.

```
thetadd_eqn = subs(isolate(mainEq2,diff(theta(t),t,2)),I,0)
```

```
thetadd_eqn =
```

$$\frac{\partial^2}{\partial t^2} \theta(t) = \frac{l m \cos(\theta(t)) \frac{\partial^2}{\partial t^2} x(t) + dF l \cos(\theta(t)) + g l m \sin(\theta(t))}{l^2 m}$$

In other examples, such as “Swing-Up Control of Pendulum Using Nonlinear Model Predictive Control” (Model Predictive Control Toolbox) and “Swing-Up Control of Pendulum Using Multistage Nonlinear MPC with Nonlinear Grey-Box Model” (Model Predictive Control Toolbox), the nonlinear plant model is defined in a MATLAB file in which the force on the pole is not considered ( $dF = 0$ ), the horizontal axis is indicated as  $z$  instead of  $x$ , and the friction coefficient is indicated as  $Kd$  instead of  $b$ . Derive the acceleration of the cart from the previous equation `xdd_eqn` by setting  $dF$  to zero and replacing the variables  $x$  and  $b$  with  $z$  and  $Kd$ , respectively.

```
syms z(t) Kd
```

```
xdd_eqn = subs(xdd_eqn,[dF x(t) b],[0 z(t) Kd])
```

```
xdd_eqn =
```

$$\frac{\partial^2}{\partial t^2} z(t) = \frac{-l m \sin(\theta(t)) \left( \frac{\partial}{\partial t} \theta(t) \right)^2 - Kd \frac{\partial}{\partial t} z(t) + F + \frac{g m \sin(2 \theta(t))}{2}}{m \sin(\theta(t))^2 + M}$$

Derive the angular acceleration of the pole by using the second mass-matrix equation defined previously.

```
thetadd_eqn = diff(theta(t),t,2) == subs(q_acc(2),[I dF x(t) b],[0 0 z(t) Kd]);
thetadd_eqn = collect(isolate(thetadd_eqn,g*sin(theta(t))),diff(theta(t),t,2))
```

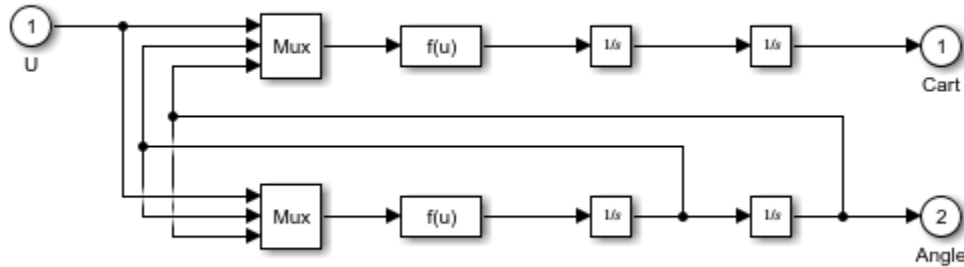
```
thetadd_eqn =
```

$$g \sin(\theta(t)) = \frac{-l m \cos(\theta(t))^2 + M l + l m}{M + m} \frac{\partial^2}{\partial t^2} \theta(t) + \frac{l m \cos(\theta(t)) \sin(\theta(t)) \left( \frac{\partial}{\partial t} \theta(t) \right)^2 + Kd \cos(\theta(t)) \frac{\partial}{\partial t} z(t) - F \cos(\theta(t))}{M + m}$$

For these equations, you can also apply the trigonometry identities  $\sin^2(\theta) + \cos^2(\theta) = 1$  and  $2\sin(\theta)\cos(\theta) = \sin(2\theta)$  to obtain the form used in the Model Predictive Control examples.

### Examples of Cart-Pole System in Simulink

For examples of the cart-pole system in the Simulink product family, see “Inverted Pendulum with Animation” (Simulink), “Inverted Pendulum Controller Tuning” (Simulink Design Optimization), and “Add App Designer App to Inverted Pendulum Model” (Simulink Real-Time). These examples represent the equations of motion using a Pendulum block. To create this Pendulum block, you can use two Fcn (Simulink) blocks to apply specific mathematical operations or user-defined functions to its input.



Here, the two Fcn blocks, with the mask label  $f(u)$ , calculate the cart's linear acceleration and the pole's angular acceleration, respectively. The user-defined functions are derived from the mass-matrix equations previously discussed, where  $I$ ,  $dF$ , and  $b$  are set to zero.

Derive the acceleration of the cart by using the mass-matrix representation.

$$D2x = \text{diff}(x(t), t, 2) == \text{simplify}(\text{subs}(q\_acc(1), [I \ dF \ b], [0 \ 0 \ 0]))$$

$D2x =$

$$\frac{\partial^2}{\partial t^2} x(t) = \frac{-l m \sin(\theta(t)) \left( \frac{\partial}{\partial t} \theta(t) \right)^2 + F + g m \cos(\theta(t)) \sin(\theta(t))}{-m \cos(\theta(t))^2 + M + m}$$

Derive the angular acceleration of the pole by using the mass-matrix representation.

$$D2\theta = \text{diff}(\theta(t), t, 2) == \text{simplify}(\text{expand}(\text{subs}(q\_acc(2), [I \ dF \ b], [0 \ 0 \ 0])))$$

$D2\theta =$

$$\frac{\partial^2}{\partial t^2} \theta(t) = \frac{-l m \cos(\theta(t)) \sin(\theta(t)) \left( \frac{\partial}{\partial t} \theta(t) \right)^2 + F \cos(\theta(t)) + M g \sin(\theta(t)) + g m \sin(\theta(t))}{l (-m \cos(\theta(t))^2 + M + m)}$$

### Examples of Cart-Pole System in Reinforcement Learning Toolbox

Reinforcement Learning Toolbox™ provides several cart-pole system environments, as described in "Use Predefined Control System Environments" (Reinforcement Learning Toolbox). The "CartPole-Discrete" and "CartPole-Continuous" environments are both available when you use the `rlPredefinedEnv` (Reinforcement Learning Toolbox) function. In these predefined environments, no force is applied to the pole ( $dF = 0$ ), the convention for  $\theta(t)$  is reversed (the angle is positive for clockwise rotation instead of counterclockwise), and there is no friction ( $b = 0$ ). The pole is a uniform rod with a total length of  $2l$ , so the distance from the cart attachment to the pole's center of mass is  $l$ , and the pole's moment of inertia is  $\frac{1}{3}ml^2$ .

To derive the equations used in these environments, first use the first main equation in this example to obtain the acceleration of the cart by applying substitutions.

$$D2x = \text{subs}(\text{isolate}(\text{mainEq1}, \text{diff}(x(t), t, 2)), [\theta(t) \ dF \ b], [-\theta(t) \ 0 \ 0])$$

$D2x =$

$$\frac{\partial^2}{\partial t^2} x(t) = \frac{F + l m \left( \sin(\theta(t)) \left( \frac{\partial}{\partial t} \theta(t) \right)^2 - \cos(\theta(t)) \frac{\partial^2}{\partial t^2} \theta(t) \right)}{M + m}$$

Next, substitute the known parameter values into the second main equation.

```
mainEq2subs = subs(mainEq2,[theta(t) dF b I],[-theta(t) 0 0 1/3*m*l^2])
```

```
mainEq2subs(t) =
```

$$\frac{4 m l^2 \frac{\partial^2}{\partial t^2} \theta(t)}{3} + m \cos(\theta(t)) l \frac{\partial^2}{\partial t^2} x(t) - g m \sin(\theta(t)) l = 0$$

Substituting the expression of the cart acceleration results in this equation in terms of the angular acceleration of the pole.

```
thdd = collect(subs(mainEq2subs,lhs(D2x),rhs(D2x)),diff(theta(t),t,2))
```

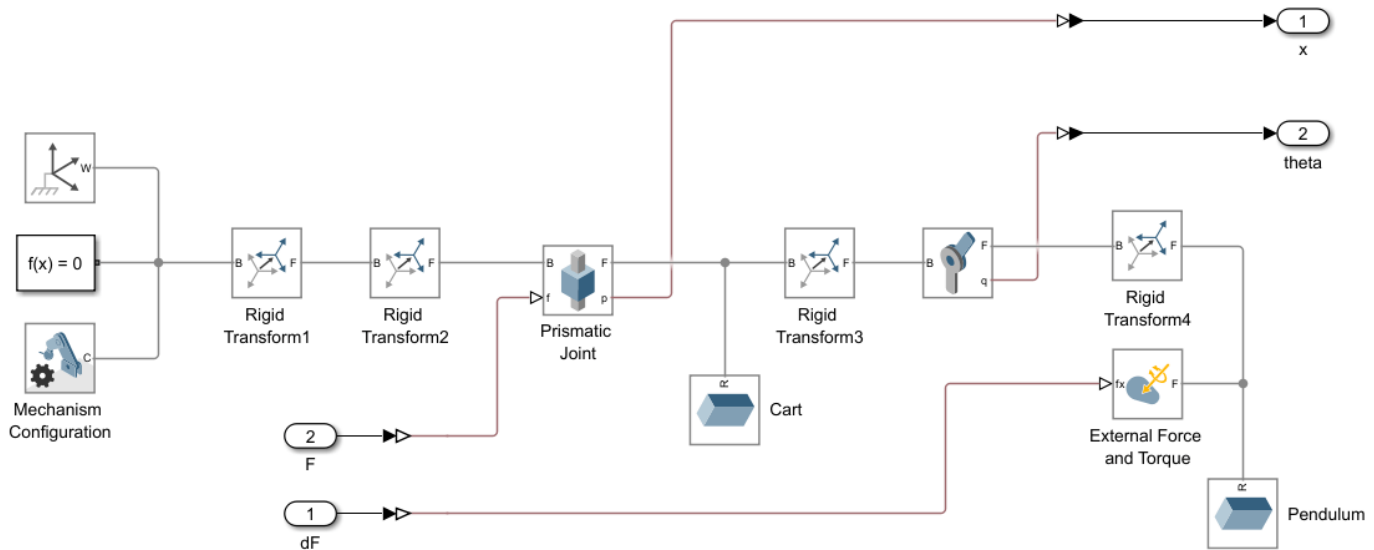
```
thdd(t) =
```

$$\left( \frac{4 l^2 m}{3} - \frac{l^2 m^2 \cos(\theta(t))^2}{M + m} \right) \frac{\partial^2}{\partial t^2} \theta(t) - g l m \sin(\theta(t)) + \frac{l m \cos(\theta(t)) \left( l m \sin(\theta(t)) \left( \frac{\partial}{\partial t} \theta(t) \right)^2 + F \right)}{M + m} = 0$$

Dividing all terms of the previous equation by  $l m$  yields the equation used in the code of the predefined environments. An alternative derivation of the system equations in this form is described in [2] on page 3-301 and [3] on page 3-301.

### Examples of Cart-Pole System in Simscape Multibody

The Simulink-based reinforcement learning environments are implemented in Simscape Multibody within a Simulink model, using the architecture in the following diagram. This diagram differs from the block diagrams used in control systems, where the connecting lines represent physical connections instead of signals, and the blocks represent bodies, joints, sensors, and coordinate transformations instead of operations on the signals. Note that the force acting on the pole,  $dF$ , is positive in the  $x$ -direction, contrary to the convention that has been used throughout this example.



This Simscape Multibody model is used in the example “Control of Inverted Pendulum on Cart” (Simulink Control Design).

## References

- [1] Lunberg, Kent H., and Taylor W. Barton. "History of Inverted-Pendulum Systems". *IFAC Proceedings Volumes*, 8th IFAC Symposium on Advances in Control Education, 42, no. 24 (January 2010): 131-135. <https://www.sciencedirect.com/science/article/pii/S1474667015316049>.
- [2] Florian, Răzvan V. "Correct equations for the dynamics of the cart-pole system". July 11, 2005; updated February 10, 2007. [https://coneural.org/florian/papers/05\\_cart\\_pole.pdf](https://coneural.org/florian/papers/05_cart_pole.pdf).
- [3] Green, Colin D. "Equations of Motion for the Cart and Pole Control Task". January 4, 2020. <https://sharpneat.sourceforge.io/research/cart-pole/cart-pole-equations.html>.
- [4] "Inverted Pendulum". In *Wikipedia*, February 24, 2024. [https://en.wikipedia.org/wiki/Inverted\\_pendulum](https://en.wikipedia.org/wiki/Inverted_pendulum).

## See Also

### Functions

`matlabFunction` | `odeToVectorField` | `massMatrixForm` | `ode45`

## Related Examples

- “Simulate the Motion of the Periodic Swing of a Pendulum” on page 4-35
- “Animation and Solution of Double Pendulum Motion” on page 4-47

- “Derive Quadrotor Dynamics for Nonlinear Model Predictive Control” on page 5-24
- “Derive and Apply Inverse Kinematics to Two-Link Robot Arm” on page 3-94
- “Analytical Solutions of the Inverse Kinematics of a Humanoid Robot” on page 3-170

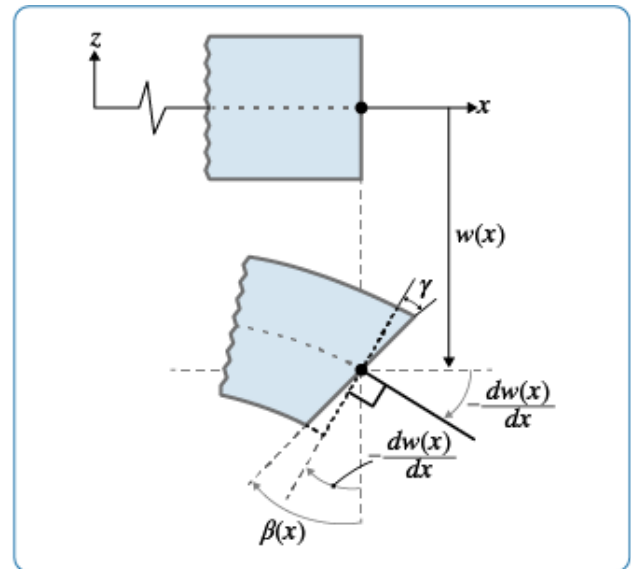
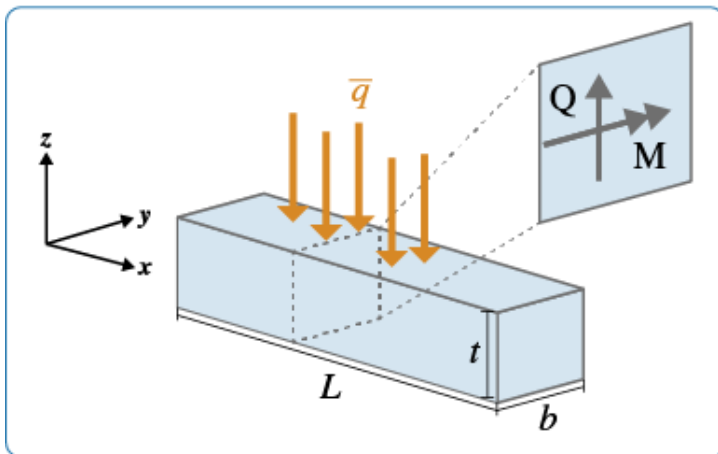
## Finite Element Formulation for Timoshenko Beam Problem

This example shows how to apply the finite element method (FEM) to solve a Timoshenko beam problem, using both linear and quadratic basis functions for analysis. The Timoshenko beam theory is a 1-D problem that reduces the complex 3-D problem of beam deformation to a set of 1-D differential equations along the length of the beam. In contrast to the Euler-Bernoulli beam theory, which does not consider shear deformation, the Timoshenko beam theory accounts for both shear deformation and rotational bending effects. The Timoshenko beam theory is generally more accurate for short, thick beams where shear deformation cannot be neglected. Using a cantilever beam and a beam fixed at both ends, this example discusses the analytical solutions of the Timoshenko beam problem and compares them with the FEM solutions.

In the thin beam limit of the standard variational formulation, the Timoshenko beam model becomes susceptible to the *transverse shear locking* effect. The term *locking* refers to an artificial stiffening, as if the beam's cross sections are "locked" together and unable to shear properly relative to each other. This issue is more pronounced when using lower-order basis functions in the FEM analysis. This example compares the results derived from linear basis functions and from quadratic basis functions to illustrate how using higher-order functions reduces the locking effect in the Timoshenko beam analysis.

### Define Beam Parameters

In this example, the Timoshenko beam extends along the  $x$ -axis, with vertical displacement  $w(x)$  in the transversal direction along the  $z$ -axis and bending rotation angle  $\beta(x)$  with rotation around the  $y$ -axis.



First, define the geometrical parameters of the beam and set their values in SI units.

- $t$  is thickness of the beam, in m.
- $b$  is the width of the beam, in m.
- $A$  is the cross-sectional area of the beam, in  $\text{m}^2$ .
- $L$  is the length of the beam, in m. The left-end  $x$ -coordinate of the beam is at 0, so  $L$  also corresponds to the right-end  $x$ -coordinate of the beam.

```
beamParams.t = 10^-2;
beamParams.b = 0.1;
beamParams.A = beamParams.b*beamParams.t;
beamParams.L = 4;
```

Next, define the load and material parameters of the beam and set their values in SI units.

- $\bar{q}$  is the external transversal distributed load along the beam's length, in N/m. The load value is set to be proportional to the thickness of the beam to the power of 3 and constant along the  $x$ -axis.
- $\bar{m}$  is the external distributed bending moment along the beam's length, in N. The moment is set to 0.
- $E$  is Young's modulus, in N/m<sup>2</sup>.
- $\nu$  is Poisson's ratio.
- $I$  is the moment of inertia of the beam's rectangular cross section in the  $yz$ -plane, in m<sup>4</sup>. Here, assume you scale the model to represent a uniform mass distribution across the beam, where the mass per area equals 1 and the moment of inertia is  $I = bt^3/12$ .
- $G$  is the shear modulus, in N/m<sup>2</sup>.
- $\alpha$  is the shear correction factor. This parameter accounts for the cross section of the beam not always being planar in the  $yz$ -plane nor remaining orthogonal to the tangent of the beam axis due to bending.

```
beamParams.q_bar = -(beamParams.t)^3;
beamParams.m_bar = 0;
beamParams.E = 1e7;
beamParams.nu = 0.3;
beamParams.I = beamParams.b*(beamParams.t^3)/12;
beamParams.G = beamParams.E/(2*(1 + beamParams.nu));
beamParams.alpha = 5/6;
```

Unlike the Euler-Bernoulli beam theory, which assumes that plane sections remain plane and perpendicular to the neutral axis, the Timoshenko beam theory accounts for shear deformation. This means that the cross-sections can warp, and the bending rotation angle  $\beta(x)$  is not necessarily the same as the slope of the deflection curve. For small angle approximations and an external load applied downward in the  $-z$  direction, the angle of the deflection curve is  $-dw(x)/dx$ . The shear angle  $\gamma(x)$  can be obtained as the difference between the bending rotation angle and the angle of the deflection curve, that is

$$\gamma(x) = \beta(x) - \left(-\frac{dw(x)}{dx}\right) = \beta(x) + \frac{dw(x)}{dx}.$$

### Derive Equilibrium Equations for Timoshenko Beam

The Timoshenko beam theory introduces two variables: the transverse deflection  $w(x)$  along the  $z$ -axis and the bending rotation angle  $\beta(x)$  with respect to the  $y$ -axis. These variables as a function of  $x$ , or the beam axial direction, describe the displacements of the beam under external load and bending moment. The theory further relates the internal load and moment through constitutive laws based on the material properties, leading to a set of equilibrium equations for the beam.

From the two displacement fields  $w(x)$  and  $\beta(x)$ , define the shear angle  $\gamma(x)$  and the bending curvature  $\kappa(x)$ .



$$\gamma(x) = \beta(x) + \frac{dw(x)}{dx},$$

$$\kappa(x) = \frac{d\beta(x)}{dx}.$$

The constitutive laws, according to Hooke's laws that relate the stresses to strains, lead to these equations for the internal load (or the internal shear force)  $Q(x)$  and the internal moment  $M(x)$ .

$$Q(x) = \alpha GA \gamma(x),$$

$$M(x) = EI \kappa(x).$$

With the presence of an external distributed moment  $m(x)$  and external distributed force  $q(x)$ , the equilibrium equations of the Timoshenko beam, according to Newton's laws, are:

$$-\frac{dM}{dx} + Q = m(x),$$

$$-\frac{dQ}{dx} = q(x).$$

In terms of the displacement variables  $w$  and  $\beta$ , the equilibrium equations become:

$$-\frac{d}{dx} \left( EI \frac{d\beta}{dx} \right) + \alpha GA \left( \beta + \frac{dw}{dx} \right) = m(x),$$

$$-\frac{d}{dx} \left( \alpha GA \left( \beta + \frac{dw}{dx} \right) \right) = q(x).$$

### Find Analytical Solution for Cantilever Beam Fixed at One End

Consider a cantilever beam that is fixed at one end and free at the other. When an external uniformly distributed load  $\bar{q}$  is applied to the cantilever beam, the beam displacements  $w(x)$  and  $\beta(x)$  have analytical solutions. You can use Symbolic Math Toolbox™ to derive these analytical solutions from the equilibrium equations by applying the necessary boundary conditions.

Create the symbolic variables for the displacement variables and the beam parameters.

```
syms w(x) beta(x) A L q_bar m_bar E I G alpha
```

Define the equilibrium equations by setting the external load and the external moment as  $q(x) = \bar{q}$  and  $m(x) = 0$ , respectively.

```
Q = alpha*G*A*(beta + diff(w,x));
M = E*I*diff(beta,x);
eqns = [-diff(M,x) + Q == 0;
        -diff(Q,x) == q_bar]
```

$$\text{eqns}(x) = \begin{pmatrix} A G \alpha \left( \frac{\partial}{\partial x} w(x) + \beta(x) \right) - E I \frac{\partial^2}{\partial x^2} \beta(x) = 0 \\ -A G \alpha \left( \frac{\partial^2}{\partial x^2} w(x) + \frac{\partial}{\partial x} \beta(x) \right) = \bar{q} \end{pmatrix}$$

The boundary conditions for a fixed left end of the cantilever beam are  $w(0) = 0$  and  $\beta(0) = 0$ . Meanwhile, the boundary conditions for the free right end of the beam are  $\gamma(L) = 0$  and  $\kappa(L) = 0$ , or equivalently  $Q(L) = M(L) = 0$ . Define these boundary conditions by introducing the intermediary variables for  $dw/dx$  and  $d\beta/dx$  that follow the definition of  $\gamma$  and  $\kappa$ .

```
Dw(x) = diff(w,x);
Dbeta(x) = diff(beta,x);
cond = [w(0)==0; beta(0)==0; beta(L)==-Dw(L); Dbeta(L)==0];
```

Find the analytical solutions for  $w(x)$  and  $\beta(x)$  by using `dsolve` on the equilibrium equations and the specified boundary conditions. Simplify the solutions by using `simplify`.

```
[beta_sol,w_sol] = dsolve(eqns,cond);
w_sol = simplify(w_sol)

w_sol =

$$\frac{\bar{q}x \left( 6AG\alpha L^2x - 4AG\alpha Lx^2 + 24EIL + AG\alpha x^3 - 12EIx \right)}{24AEGI\alpha}$$

```

```
beta_sol = simplify(beta_sol)
```

```
beta_sol =

$$-\frac{\bar{q}x \left( 3L^2 - 3Lx + x^2 \right)}{6EI}$$

```

Most textbooks on structural analysis include solutions for cantilever beam displacements. For comparison, define the solutions from references [1] on page 3-323 and [2] on page 3-323. Compare the textbook solutions with the analytical solutions obtained using Symbolic Math Toolbox by using `isAlways`. The results are logical 1, meaning that the textbook solutions agree with the solutions returned by `dsolve`.

```
w_ref(x) = 1/(alpha*G*A)*(q_bar*L*x - q_bar*x^2/2) - ...
1/(E*I)*(-q_bar*x^4/24 + q_bar*L*x^3/6 - q_bar*L^2*x^2/4)
```

```
w_ref(x) =

$$\frac{\frac{\bar{q}L^2x^2}{4} - \frac{\bar{q}Lx^3}{6} + \frac{\bar{q}x^4}{24}}{EI} - \frac{\frac{\bar{q}x^2}{2} - L\bar{q}x}{AG\alpha}$$

```

```
syms x_a
beta_ref(x) = int(-q_bar/(alpha*G*A) - diff(w_ref(x_a),2),x_a,0,x)
```

```
beta_ref(x) =

$$-\frac{\bar{q}x \left( 3L^2 - 3Lx + x^2 \right)}{6EI}$$

```

```
tf_w = isAlways(w_ref == w_sol)
```

```
tf_w = logical
1
```

```
tf_beta = isAlways(beta_ref == beta_sol)
```

```
tf_beta = logical
1
```

Next, modify the analytical solutions by substituting the values of all known parameters of the beam. In the next sections, you will plot these analytical solutions and compare them with the FEM solutions.

```
w_eval = subs(w_sol,beamParams);
beta_eval = subs(beta_sol,beamParams);
```

### Implement Basis Interpolation Functions for FEM

The FEM is a numerical technique used to solve complex engineering and mathematical problems by breaking them down into smaller, simpler parts called finite elements. These elements are interconnected at points known as nodes, forming a mesh that approximates the geometry of the original system. By discretizing the domain into a finite number of elements, FEM transforms a complex problem into a set of algebraic equations that can be solved computationally. Each element is governed by simplified equations that approximate the system's behavior, and these local equations are assembled into a global system.

In the FEM formulation of the Timoshenko beam, the displacement fields  $w^i$  and  $\beta^i$  on each mesh element, indexed by  $i$ , are approximated using basis interpolation functions. These basis functions are defined in terms of the discretized nodal values and the shape functions. One method to define the shape functions is through the use of Lagrange polynomials. To incorporate an *isoparametric* representation of the problem, which accommodates both constant and linear strain states of the mesh elements, the same shape functions are used to define both the element's geometric shape and the displacement fields. Consequently, this example defines the displacements  $w^i$ ,  $\beta^i$ , and the global coordinate  $x^i$  of each element as functions of the local coordinate  $\xi$  using the same shape functions, where:

$$w^i(\xi) \approx \sum_{j=1}^m N_j(\xi) w_j^i,$$

$$\beta^i(\xi) \approx \sum_{j=1}^m N_j(\xi) \beta_j^i,$$

$$x^i(\xi) = \sum_{j=1}^m N_j(\xi) x_j^i$$

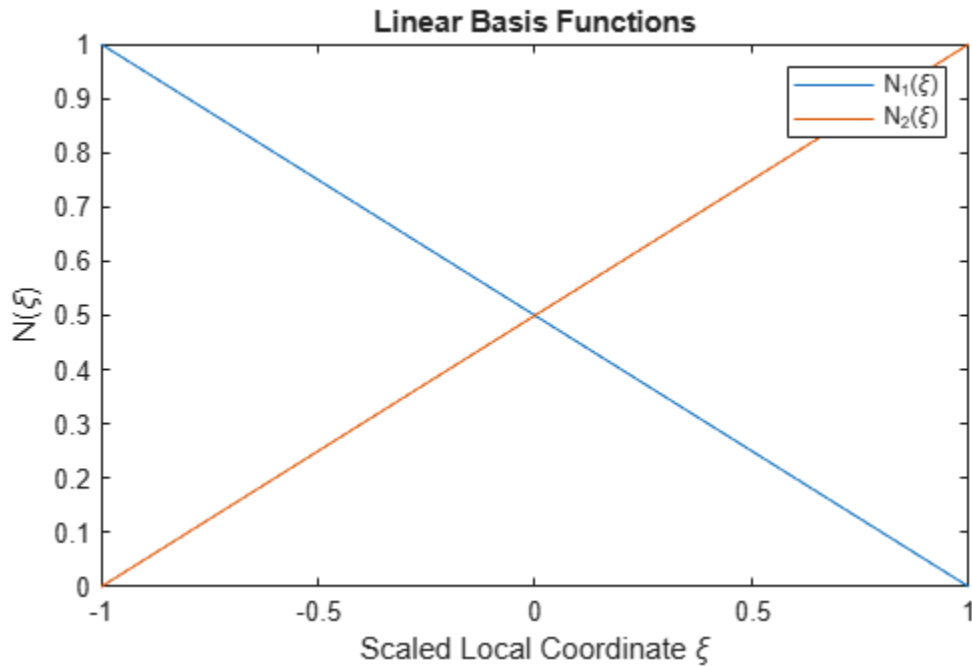
Here, the local coordinate system  $\xi$  is scaled to range from -1 to 1, the discrete variables  $w_j^i$ ,  $\beta_j^i$ , and  $x_j^i$  are the nodal values, and  $N_j(\xi)$  are the shape functions. In this example of FEM analysis, consider two-noded elements ( $m = 2$ ) and three-noded elements ( $m = 3$ ) when discretizing each element, where linear and quadratic basis functions are used for these elements, respectively.

To illustrate the linear and quadratic basis functions, define the local coordinate as a symbolic variable `xi`. Define the linear basis functions and plot them using `fplot`.

```
syms xi
N1lin(xi) = (1 - xi)/2;
N2lin(xi) = (1 + xi)/2;
Nlin(xi) = [N1lin; N2lin]
```

```
Nlin(xi) =
    (1 - xi)/2
    (1 + xi)/2
```

```
fplot(Nlin,[-1 1])
title("Linear Basis Functions")
xlabel("Scaled Local Coordinate \xi")
ylabel("N(\xi)")
legend("N_1(\xi)","N_2(\xi)")
```

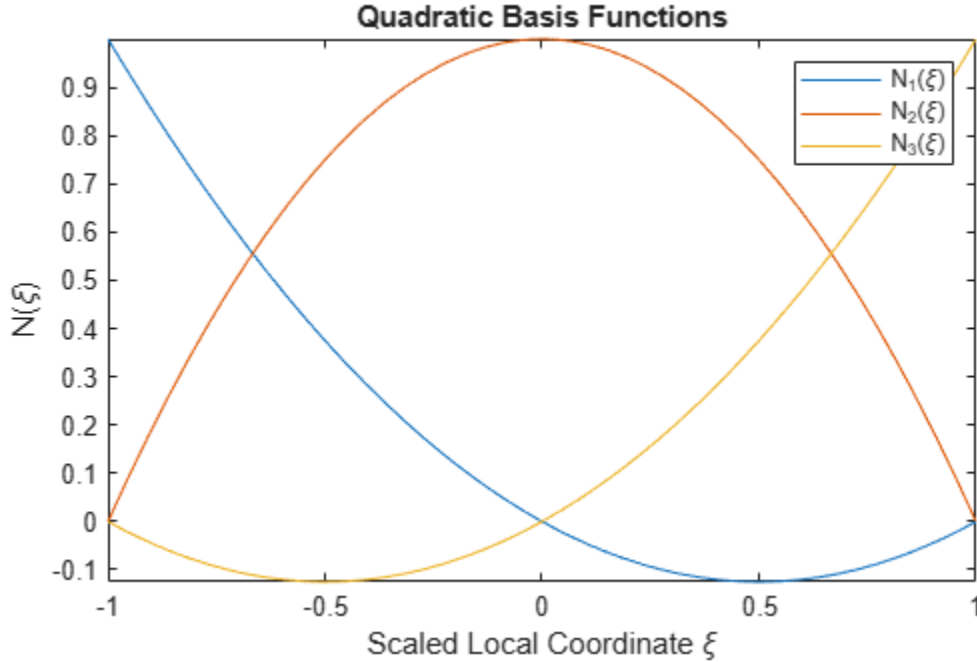


Next, define the quadratic basis functions and plot them using fplot.

```
N1quad(xi) = xi*(xi - 1)/2;
N2quad(xi) = -(xi - 1)*(xi + 1);
N3quad(xi) = xi*(xi + 1)/2;
Nquad(xi) = [N1quad; N2quad; N3quad]
```

$$N_{quad}(\xi) = \begin{pmatrix} \frac{\xi(\xi-1)}{2} \\ -(\xi-1)(\xi+1) \\ \frac{\xi(\xi+1)}{2} \end{pmatrix}$$

```
figure
fplot(Nquad,[-1 1])
title("Quadratic Basis Functions")
xlabel("Scaled Local Coordinate \xi")
ylabel("N(\xi)")
legend("N_1(\xi)","N_2(\xi)","N_3(\xi)")
```



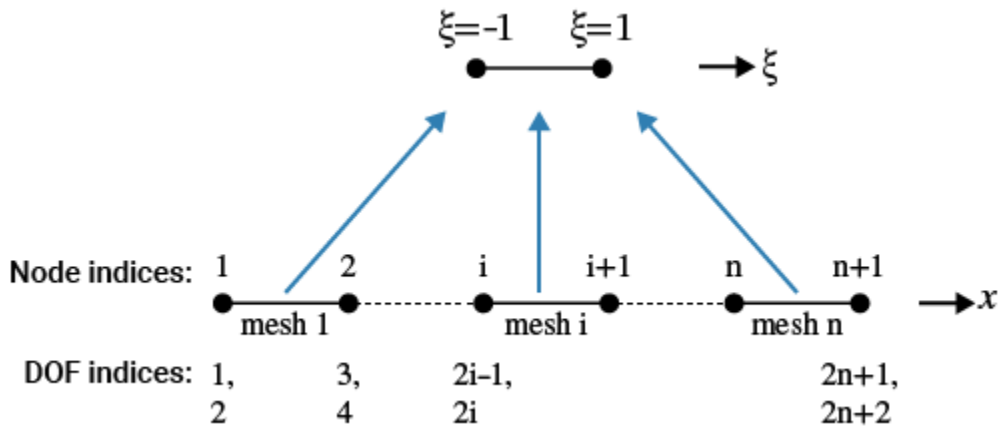
This Timoshenko beam model can suffer from *transverse shear locking*. In the thin beam limit, the Timoshenko beam model must satisfy the Kirchhoff constraint  $\gamma = 0$  and the constraint  $\kappa = 0$ . These constraints yield the trivial solution of the nodal displacements as  $w_j^i = \beta_j^i = 0$ . In other words, the element locks. The lower-order basis functions cause an erroneous stiffening to occur, which restricts the transverse shear deformation and leads to an overestimation of the beam's stiffness. The term *locking* refers to this artificial stiffening, as if the beam's cross sections are locked together and unable to shear properly relative to each other. To alleviate this problem, you can use higher-order basis functions for smaller errors in the FEM analysis to solve the Timoshenko beam problem.

### Generate Two-Noded and Three-Noded Mesh Elements

Define the functions that generate two-noded and three-noded mesh elements. These functions return the meshes as a structure with fields that represent the number of elements, the global node coordinates, the node indices of each element, and the total number of degrees of freedom (DOF).

The following figures illustrate the convention used to define the meshes. In this example, the beam is divided into equally spaced elements with the total number of mesh elements defined by  $n$ . As a result, the nodal coordinates are equally spaced. The node indices are sequentially numbered from the left end to the right end, starting from 1 up to  $n+1$  for two-noded mesh elements (or  $2*n+1$  for three-noded mesh elements). Each node has two degrees of freedom, corresponding to the transverse displacement and the bending angle. The indices for these degrees of freedom are labeled 1 and 2 for the first node, respectively, 3 and 4 for the second node, and so on until the final node. Consequently, the total number of degrees of freedom is twice the total number of nodes.

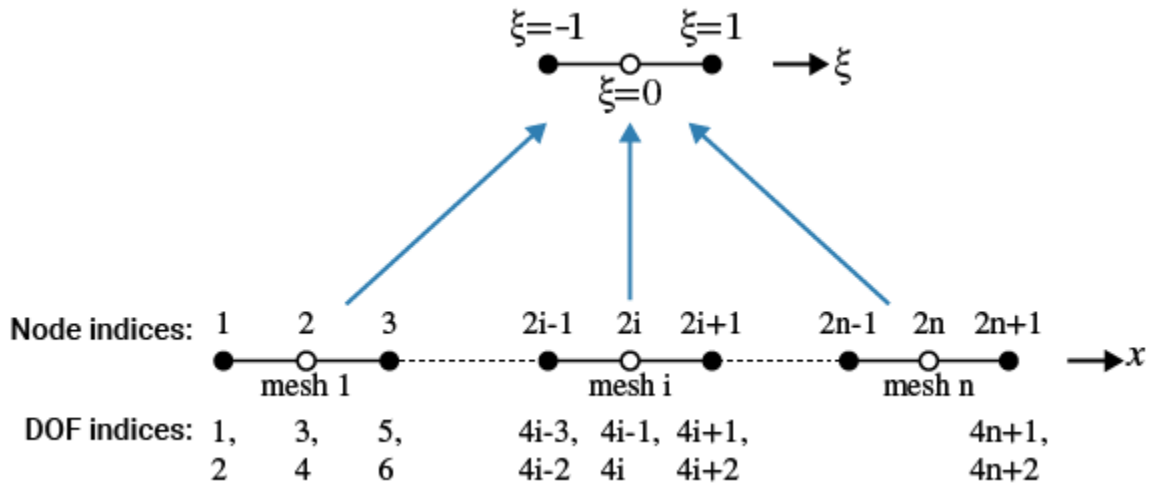
This figure illustrates a linear mesh with two-noded mesh elements, where  $n$  is the number of mesh elements. Node indices range from 1 to  $n+1$ . DOF indices range from 1 to  $2*n+2$ .



Define a function that generates two-noded mesh elements.

```
function mshLin = generateLinearMesh(L,n)
mshLin.n = n;
mshLin.nodes = linspace(0,L,n + 1)';
mshLin.elements = (1:n)' + [0 1];
mshLin.numDOFs = 2*length(mshLin.nodes);
end
```

This figure illustrates a quadratic mesh with three-noded mesh elements, where  $n$  is the number of mesh elements. Node indices range from 1 to  $2*n+1$ . DOF indices range from 1 to  $4*n+2$ .



Define a function that generates three-noded mesh elements.

```
function mshQuad = generateQuadraticMesh(xend,n)
mshQuad.n = n;
mshQuad.nodes = linspace(0,xend,2*n + 1)';
mshQuad.elements = 2*(1:n)' + [-1 0 1];
mshQuad.numDOFs = 2*length(mshQuad.nodes);
end
```

Using these functions, generate a linear mesh with 30 two-noded mesh elements and a quadratic mesh with 30 three-noded mesh elements.

```

n = 30;
mshLin = generateLinearMesh(beamParams.L,n)

mshLin = struct with fields:
    n: 30
    nodes: [31x1 double]
    elements: [30x2 double]
    numDOFs: 62

mshQuad = generateQuadraticMesh(beamParams.L,n)

mshQuad = struct with fields:
    n: 30
    nodes: [61x1 double]
    elements: [30x3 double]
    numDOFs: 122

```

### Assemble Global Stiffness Matrix and Load Vector

The FEM discretizes the beam into a finite number of elements and solves for the displacement fields. In this analysis, the global stiffness matrix connects the vector of nodal displacements with the vector of forces. In other words, the FEM solves the matrix equation of the form  $\mathbf{F} = \mathbf{K} \mathbf{d}$ , where  $\mathbf{F}$  is the load vector,  $\mathbf{K}$  is the global stiffness matrix, and  $\mathbf{d}$  is the nodal displacement vector. This section discusses the steps to derive the global stiffness matrix and how to reformulate the equilibrium equations into the matrix equation of the form  $\mathbf{F} = \mathbf{K} \mathbf{d}$ .

To derive this matrix equation, first apply the virtual work principle to the Timoshenko beam model. This principle states that the sum of the internal and external virtual work is zero with respect to infinitesimal virtual displacements, which are small variational displacements satisfying the boundary conditions of the system. Multiply the two equilibrium equations (Equations 1-2) by arbitrary variations  $\delta w$  and  $\delta \beta$ , respectively. Then, perform integration by parts and apply the corresponding Dirichlet and Neumann boundary conditions.

$$\begin{aligned}
 & \int_0^L \left( \frac{dQ}{dx} + q \right) \delta w \, dx = 0 \Rightarrow \\
 & - \int_0^L Q \frac{d\delta w}{dx} \, dx + Q(L) \delta w(L) - Q(0) \delta w(0) + \int_0^L q \delta w \, dx = 0, \\
 & \int_0^L \left( \frac{dM}{dx} - Q + m \right) \delta \beta \, dx = 0 \Rightarrow \\
 & - \int_0^L M \frac{d\delta \beta}{dx} \, dx - \int_0^L Q \delta \beta \, dx + M(L) \delta \beta(L) - M(0) \delta \beta(0) + \int_0^L m \delta \beta \, dx = 0.
 \end{aligned}$$

Add the two equations to obtain the final form of the total weak formulation of the Timoshenko beam problem.

$$\begin{aligned}
 & \underbrace{\int_0^L Q \left( \frac{d\delta w}{dx} + \delta \beta \right) dx}_{\text{boundary terms}} - \underbrace{\int_0^L M \frac{d\delta \beta}{dx} dx}_{\text{boundary terms}} + \int_0^L q \delta w \, dx + \int_0^L m \delta \beta \, dx \\
 & + Q(0) \delta w(0) + Q(L) \delta w(L) + M(0) \delta \beta(0) + M(L) \delta \beta(L) = 0.
 \end{aligned}$$

Next, cast this formulation into matrix form by defining these vectors and matrices.

- $\mathbf{u} = \begin{bmatrix} w \\ \beta \end{bmatrix}$  is the displacements.
- $\varepsilon = \begin{bmatrix} \gamma \\ \kappa \end{bmatrix} = \mathbf{L} \cdot \mathbf{u}$  is the strains, where  $\mathbf{L} = \begin{bmatrix} \frac{\partial}{\partial x} & 1 \\ 0 & \frac{\partial}{\partial x} \end{bmatrix}$ .
- $\sigma = \begin{bmatrix} Q \\ M \end{bmatrix} = \mathbf{C} \cdot \varepsilon$  is the internal loads, where  $\mathbf{C} = \begin{bmatrix} \alpha GA & 0 \\ 0 & EI \end{bmatrix}$ .
- $\mathbf{p} = \begin{bmatrix} q \\ m \end{bmatrix}$  is the external loads.
- $\mathbf{P}_0 = \begin{bmatrix} Q(0) \\ M(0) \end{bmatrix}$  and  $\mathbf{P}_L = \begin{bmatrix} Q(L) \\ M(L) \end{bmatrix}$  are forces on the boundaries.

The matrix formulation of the Timoshenko beam problem becomes:

$$\int_0^L (\sigma^T \cdot \delta \varepsilon - \mathbf{p}^T \cdot \delta \mathbf{u}) dx - \mathbf{P}_0^T \cdot \delta \mathbf{u}(0) - \mathbf{P}_L^T \cdot \delta \mathbf{u}(L) = 0.$$

Next, discretize the beam model by specifying the elements and selecting the appropriate basis functions for the given number of nodes within each mesh element. Here,  $x_1^i$  and  $x_m^i$  are the global coordinates of the end locations of the element with index  $i$ . For an element with  $m$  nodes, the approximated displacements of each element in terms of the nodal displacements are

$$\mathbf{u}^i = \mathbf{N} \cdot \mathbf{d}^i,$$

where

$$\mathbf{N} = \begin{bmatrix} N_1 & 0 & \dots & N_m & 0 \\ 0 & N_1 & \dots & 0 & N_m \end{bmatrix},$$

$$\mathbf{d}^i = \begin{bmatrix} w_1^i \\ \beta_1^i \\ \vdots \\ w_m^i \\ \beta_m^i \end{bmatrix}.$$

The approximated strains of each element are

$$\varepsilon^i = \mathbf{L} \cdot \mathbf{u}^i = \mathbf{B} \cdot \mathbf{d}^i,$$

where

$$\mathbf{B} = \mathbf{L} \mathbf{N} = \begin{bmatrix} \frac{\partial N_1}{\partial x} & N_1 & \dots & \frac{\partial N_m}{\partial x} & N_m \\ 0 & \frac{\partial N_1}{\partial x} & \dots & 0 & \frac{\partial N_m}{\partial x} \end{bmatrix}.$$

The formulation of the virtual work principle for each element, which must hold for any virtual displacements  $\delta \mathbf{d}^i$ , becomes:



$$\int_{x_1^i}^{x_m^i} (\mathbf{d}^{iT} \mathbf{B}^T \mathbf{C}^T \mathbf{B} \cdot \delta \mathbf{d}^i - \mathbf{p}^T \mathbf{N} \cdot \delta \mathbf{d}^i) dx - \begin{bmatrix} \mathbf{P}_{x_1^i}^i \\ \mathbf{P}_{x_m^i}^i \end{bmatrix}^T \cdot \delta \mathbf{d}^i = 0. \quad (3)$$

For elements  $i$  and  $i + 1$  that are connected to each other, the nodal displacements and the global coordinates of the right end of element  $i$  and the left end of the adjacent element  $i + 1$  are related to each other. For the basis functions in this example, the nodal displacements and global coordinates are continuous between the adjacent elements. In other words, this formulation yields the relations  $w_m^i = w_1^{i+1}$ ,  $\beta_m^i = \beta_1^{i+1}$ , and  $x_m^i = x_1^{i+1}$ .

Next, rewrite the virtual work formulation in Equation 3 by transposing the matrix multiplications, which results in the local stiffness matrix representation of each element.

$$\int_{x_1^i}^{x_m^i} (\mathbf{B}^T \mathbf{C} \mathbf{B} dx) \mathbf{d}^i - \int_{x_1^i}^{x_m^i} \mathbf{N}^T \mathbf{p} dx - \begin{bmatrix} \mathbf{P}_{x_1^i}^i \\ \mathbf{P}_{x_m^i}^i \end{bmatrix} = 0,$$

$$\mathbf{K}^i \cdot \mathbf{d}^i = \mathbf{F}^i.$$

Here, the local stiffness matrix is  $\mathbf{K}^i = \int_{x_1^i}^{x_m^i} \mathbf{B}^T \mathbf{C} \mathbf{B} dx$  and the local load vector is

$$\mathbf{F}^i = \int_{x_1^i}^{x_m^i} \mathbf{N}^T \mathbf{p} dx - \begin{bmatrix} \mathbf{P}_{x_1^i}^i \\ \mathbf{P}_{x_m^i}^i \end{bmatrix}.$$

Next, assemble the global stiffness matrix  $\mathbf{K}$  and load vector  $\mathbf{F}$  by following these steps:

- 1 Initialize a global stiffness matrix and a global load vector, both filled with zeros. The dimensions of the matrix and vector depend on the total number of nodes in the system and the degrees of freedom per node.
- 2 Compute the local stiffness matrix and load vector of each mesh element. Because the basis functions are defined in terms of the local coordinates  $\xi$  rather than the global coordinates  $x$ , you must apply the required transformations when performing the integration that involves these basis functions and their derivatives.
- 3 Determine the global node indices associated with the local nodes of the element. For each degree of freedom in the element, add the element's local stiffness matrix values to the corresponding positions in the global stiffness matrix. These positions are determined by the global node indices.
- 4 Similarly, add the element's local load vector values to the corresponding positions in the global load vector. In this step, you do not need to account for the  $\begin{bmatrix} \mathbf{P}_{x_1^i}^i \\ \mathbf{P}_{x_m^i}^i \end{bmatrix}$  terms in the local load vectors.

According to Newton's third law, which states that for every action, there is an equal and opposite reaction, the boundary forces between connected elements  $i$  and  $i + 1$  cancel each other out, or  $\mathbf{P}_{x_m^i}^i = -\mathbf{P}_{x_1^{i+1}}^{i+1}$ .

- 5 Loop over all elements and add their contributions to the global stiffness matrix and load vector by combining the local stiffness matrices and load vectors into a single global stiffness matrix and load vector.

Because this example describes a 1-D beam model, the transformation from local coordinates to the global coordinate involves only the derivative function  $\partial x / \partial \xi$  and its inverse. For a model with a

higher dimension, such as the 2-D Reissner–Mindlin plate, the coordinate transformation might involve the Jacobian matrix of the global coordinate vector and its determinant.

Define the `assembleGlobalKF` function to construct the global stiffness matrix and load vector. The inputs to this function are the mesh structure, basis functions, and beam parameters. In this example, consider the external loads as constants, where  $q(x) = \bar{q}$  and  $m(x) = \bar{m}$ .

```
function [K,F] = assembleGlobalKF(msh,N_basis,beamParams)

xi = sym("xi");
nNodesPerElement = length(N_basis(xi));
nDOFsPerElement = 2*nNodesPerElement;

x_basis = sym("x_basis",[nNodesPerElement 1]);
x = N_basis'*x_basis;
dN(xi) = diff(N_basis,xi)*inv(diff(x,xi));

alpha = sym("alpha");
G = sym("G");
A = sym("A");
E = sym("E");
I = sym("I");
q_bar = sym("q_bar");
m_bar = sym("m_bar");

C = [alpha*G*A  0;
      0         E*I];

Nmtx = sym(zeros(2,nDOFsPerElement));
Bmtx = sym(zeros(2,nDOFsPerElement));
Nformula = formula(N_basis);
dNformula = formula(dN);

for ii = 1:nNodesPerElement
    Nmtx(1, 2*ii - 1) = Nformula(ii);
    Nmtx(2, 2*ii) = Nformula(ii);
    Bmtx(1, 2*ii - 1) = dNformula(ii);
    Bmtx(1, 2*ii) = Nformula(ii);
    Bmtx(2, 2*ii) = dNformula(ii);
end

K = zeros(msh.numDOFs);
F = zeros(msh.numDOFs,1);

for ii = 1:msh.n
    id = msh.elements(ii,:);
    coords = msh.nodes(id);

    KEintegrand = Bmtx.'*C*Bmtx*diff(x,xi);
    KEintegrand = subs(KEintegrand,beamParams);
    KEintegrand = subs(KEintegrand,x_basis,coords);

    FEintegrand = Nmtx.'*[q_bar; m_bar]*diff(x,xi);
    FEintegrand = subs(FEintegrand,beamParams);
    FEintegrand = subs(FEintegrand,x_basis,coords);

    KE = int(KEintegrand,xi,-1,1);
    FE = int(FEintegrand,xi,-1,1);
end
```

```

    idFill = vertcat(2*id - 1, 2*id);
    idFill = idFill(:);
    K(idFill,idFill) = K(idFill,idFill) + KE;
    F(idFill) = F(idFill) + FE;
end
end

```

After assembling the global stiffness matrix and load vector, you can apply the necessary boundary conditions for the fixed displacements. This application typically involves modifying rows and columns of the global stiffness matrix and adjusting the global load vector accordingly. Finally, you can solve the global system of equations in the form of  $\mathbf{F} = \mathbf{K} \mathbf{d}$  for  $\mathbf{d}$ , which represents the unknown nodal displacements. From these displacements, you can compute other quantities of interest, such as the strains and stresses in the mesh elements.

### Find FEM Solution Using Linear Basis for Two-Noded Elements

Find the FEM solution for the cantilever beam by using the approximation of a linear basis with two-noded mesh elements. Construct the global stiffness matrix and load vector using the `assembleGlobalKF` function.

```
[K,F] = assembleGlobalKF(mshLin,Nlin,beamParams);
```

Define the fixed degrees of freedom for the cantilever beam. The indices for these degrees of freedom are 1 and 2, corresponding to the fixed leftmost node, without any transverse displacement and bending rotation. Define the free degrees of freedom of the mesh elements by excluding the fixed degrees of freedom.

```

fixedDOFs = 1:2;
freeDOFs = 1:mshLin.numDOFs;
freeDOFs = setdiff(freeDOFs,fixedDOFs);

```

Compute the nodal displacements of the cantilever beam from the global stiffness matrix and load vector.

```

u = zeros(mshLin.numDOFs,1);
u(freeDOFs) = K(freeDOFs,freeDOFs)\F(freeDOFs);

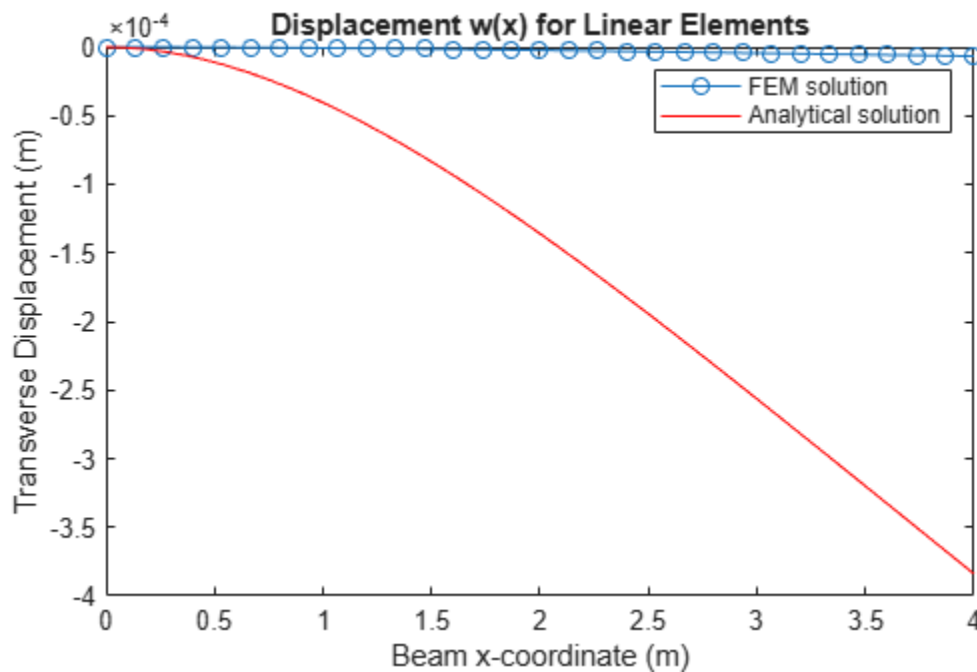
```

Plot the FEM solution of the transverse displacement  $w(x)$  of the beam. Compare this plot to the analytical solution. Due to the transverse shear locking effect in a thin beam, where the beam thickness in this example is 0.01 m and the beam length is 4 m, the FEM analysis yields the trivial solution of  $w(x) \approx 0$ . For comparison, the next section discusses the analysis using a quadratic basis with three-noded mesh elements, which do not suffer from the transverse shear locking effect.

```

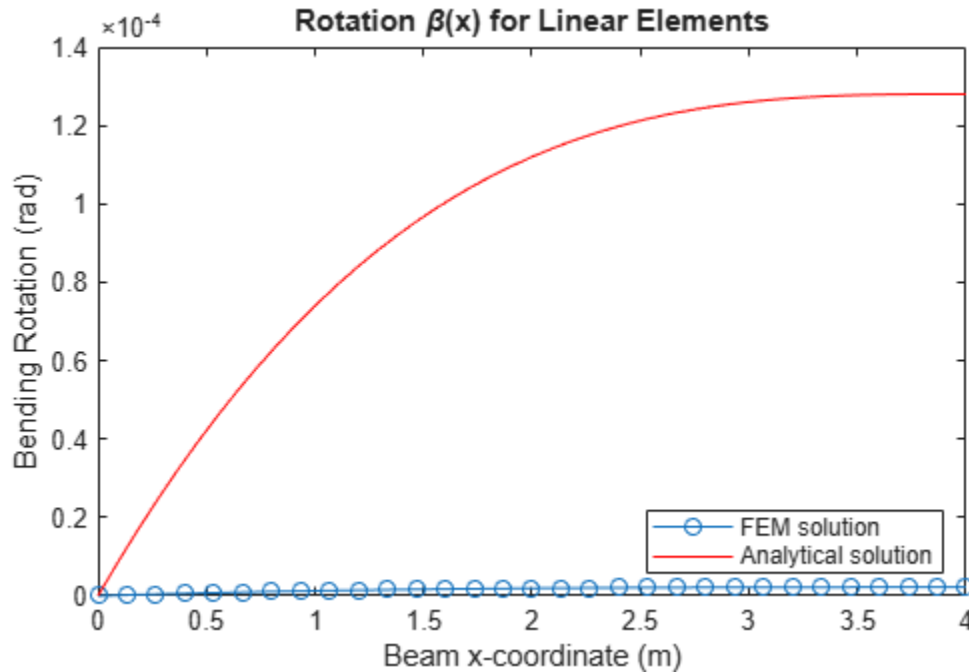
figure
plot(mshLin.nodes,u(1:2:end,1),'-o')
hold on
fplot(w_eval,[0 beamParams.L],'r-')
hold off
title("Displacement w(x) for Linear Elements")
xlabel("Beam x-coordinate (m)")
ylabel("Transverse Displacement (m)")
legend("FEM solution","Analytical solution")

```



Plot the FEM solution of the bending rotation angle  $\beta(x)$  of the beam. Due to transverse shear locking, you also obtain the trivial solution  $\beta(x) \approx 0$  from the FEM analysis, which does not match the analytical solution.

```
figure
plot(mshLin.nodes,u(2:2:end,1),"-o")
hold on
fplot(beta_eval,[0 beamParams.L],"r-")
hold off
title("Rotation \beta(x) for Linear Elements")
xlabel("Beam x-coordinate (m)")
ylabel("Bending Rotation (rad)")
legend("FEM solution","Analytical solution",Location="southeast")
```



### Find FEM Solution Using Quadratic Basis for Three-Noded Elements

Find the FEM solution for the cantilever beam by using the approximation of a quadratic basis with three-noded mesh elements. Construct the global stiffness matrix and load vector using the `assembleGlobalKF` function.

```
[K,F] = assembleGlobalKF(mshQuad,Nquad,beamParams);
```

Define the fixed degrees of freedom for the cantilever beam. Define the free degrees of freedom of the mesh elements by excluding the fixed degrees of freedom.

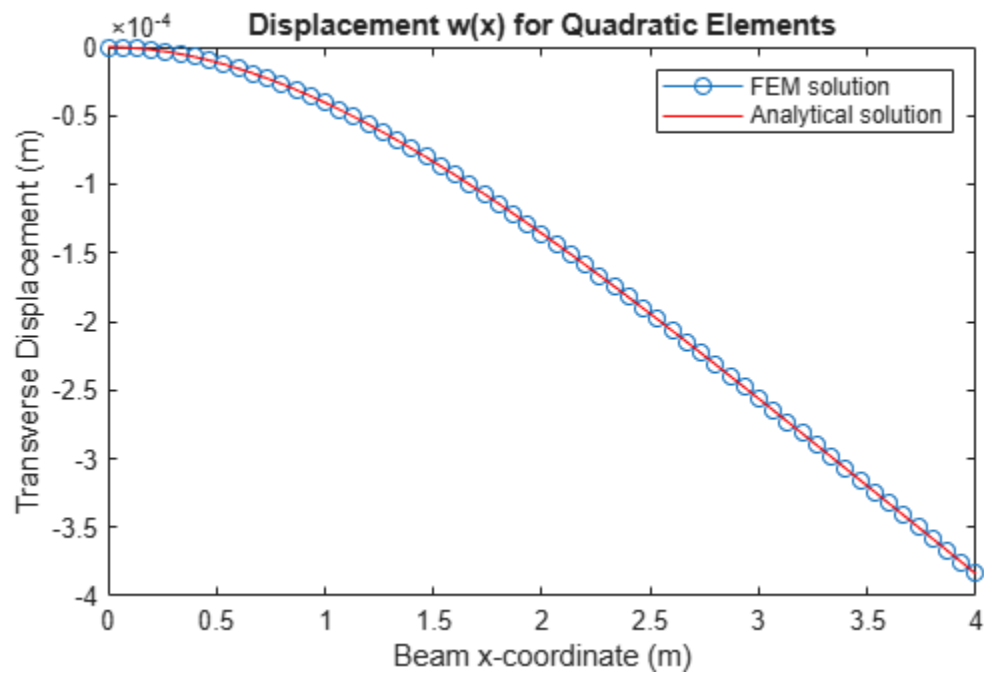
```
fixedDOFs = 1:2;
freeDOFs = 1:mshQuad.numDOFs;
freeDOFs = setdiff(freeDOFs,fixedDOFs);
```

Compute the nodal displacements of the cantilever beam from the global stiffness matrix and load vector.

```
u = zeros(mshQuad.numDOFs,1);
u(freeDOFs) = K(freeDOFs,freeDOFs)\F(freeDOFs);
```

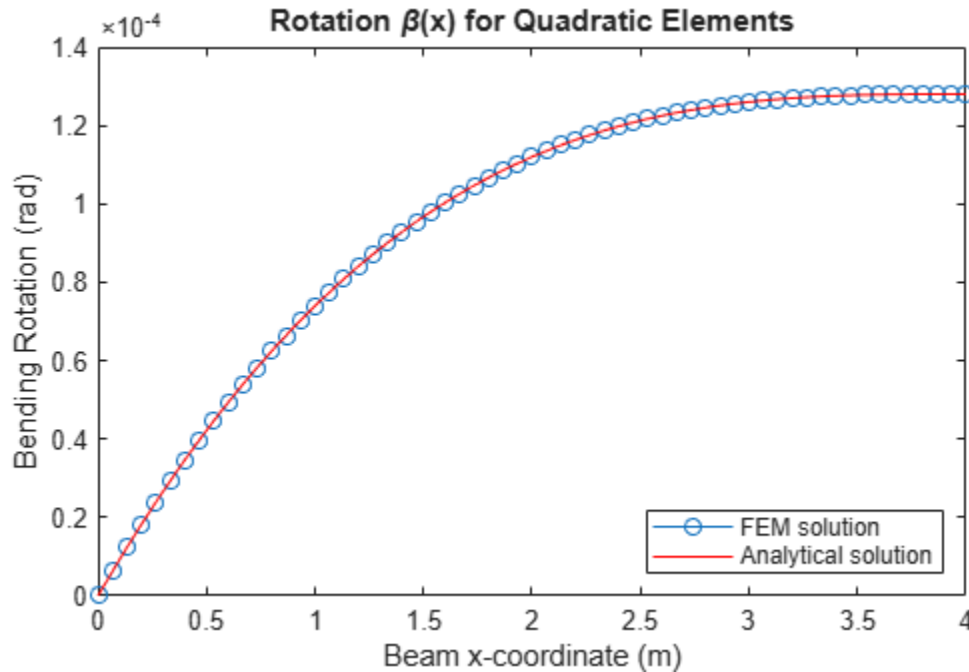
Plot the FEM solution of the transverse displacement  $w(x)$  of the beam. Compare this plot to the analytical solution. Here, the use of higher-order polynomials in the basis functions reduces the transverse shear locking effect.. The FEM solution matches the analytical solution.

```
figure
plot(mshQuad.nodes,u(1:2:end,1),"-o")
hold on
fplot(w_eval,[0 beamParams.L],"r-")
hold off
title("Displacement w(x) for Quadratic Elements")
xlabel("Beam x-coordinate (m)")
ylabel("Transverse Displacement (m)")
legend("FEM solution","Analytical solution")
```



Plot the FEM solution of the bending rotation angle  $\beta(x)$  of the beam, which matches the analytical solution.

```
figure
plot(mshQuad.nodes,u(2:2:end,1),"-o")
hold on
fplot(beta_eval,[0 beamParams.L],"r-")
hold off
title("Rotation \beta(x) for Quadratic Elements")
xlabel("Beam x-coordinate (m)")
ylabel("Bending Rotation (rad)")
legend("FEM solution","Analytical solution",Location="southeast")
```



### Find Analytical and FEM Solutions for Beam Fixed at Both Ends

Previous sections discussed the analytical and FEM solutions of a cantilever beam that is fixed at one end and free at the other. This section focuses on a beam that is fixed at both ends. This section applies the previously discussed steps to find the analytical and FEM solutions of the beam displacement.

For a beam that is fixed at both ends, the boundary conditions are  $w(0) = 0$ ,  $\beta(0) = 0$ ,  $w(L) = 0$ , and  $\beta(L) = 0$ .

```
cond = [w(0)==0; beta(0)==0; w(L)==0; beta(L)==0];
```

Find the analytical solutions for the beam displacements by using `dsolve` on the equilibrium equations and the specified boundary conditions.

```
[beta_sol,w_sol] = dsolve(eqns,cond);
w_sol = simplify(w_sol)
```

```
w_sol =

$$\frac{\bar{q} x (L - x) (-A G \alpha x^2 + A G L \alpha x + 12 E I)}{24 A E G I \alpha}$$

```

```
beta_sol = simplify(beta_sol)
```

```
beta_sol =

$$-\frac{\bar{q} x (L^2 - 3 L x + 2 x^2)}{12 E I}$$

```

Next, evaluate these solutions for the known values of the beam parameters.

```
w_eval = subs(w_sol,beamParams);
beta_eval = subs(beta_sol,beamParams);
```

To obtain the FEM solutions, construct the global stiffness matrix and load vector for the two-noded mesh elements with linear basis functions using the previously generated linear meshes.

```
[K,F] = assembleGlobalKF(mshLin,Nlin,beamParams);
```

Define the fixed degrees of freedom for the beam that is fixed at both ends according to the boundary conditions. The indices for these degrees of freedom are 1 and 2, corresponding to the fixed leftmost node, and numDOFs - 1 and numDOFs, corresponding to the fixed rightmost end, where numDOFs is the total number of degrees of freedom. Define the free degrees of freedom of the mesh elements by excluding the fixed degrees of freedom.

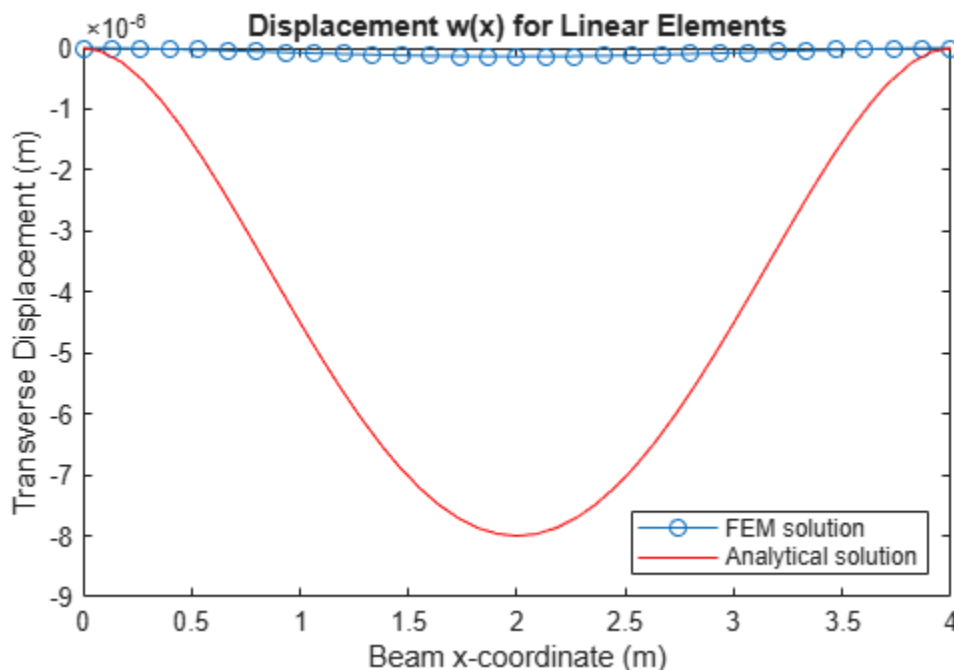
```
fixedDOFs = [1:2 mshLin.numDOFs-1:mshLin.numDOFs];
freeDOFs = 1:mshLin.numDOFs;
freeDOFs = setdiff(freeDOFs,fixedDOFs);
```

Compute the nodal displacements of the beam that is fixed at both ends from the global stiffness matrix and load vector.

```
u = zeros(mshLin.numDOFs,1);
u(freeDOFs) = K(freeDOFs,freeDOFs)\F(freeDOFs);
```

Plot the analytical and FEM solutions of the transverse displacement  $w(x)$  of the beam. Due to the transverse shear locking effect in a thin beam, the FEM analysis using linear basis functions yields the trivial solution  $w(x) \approx 0$ , which does not match the analytical solution.

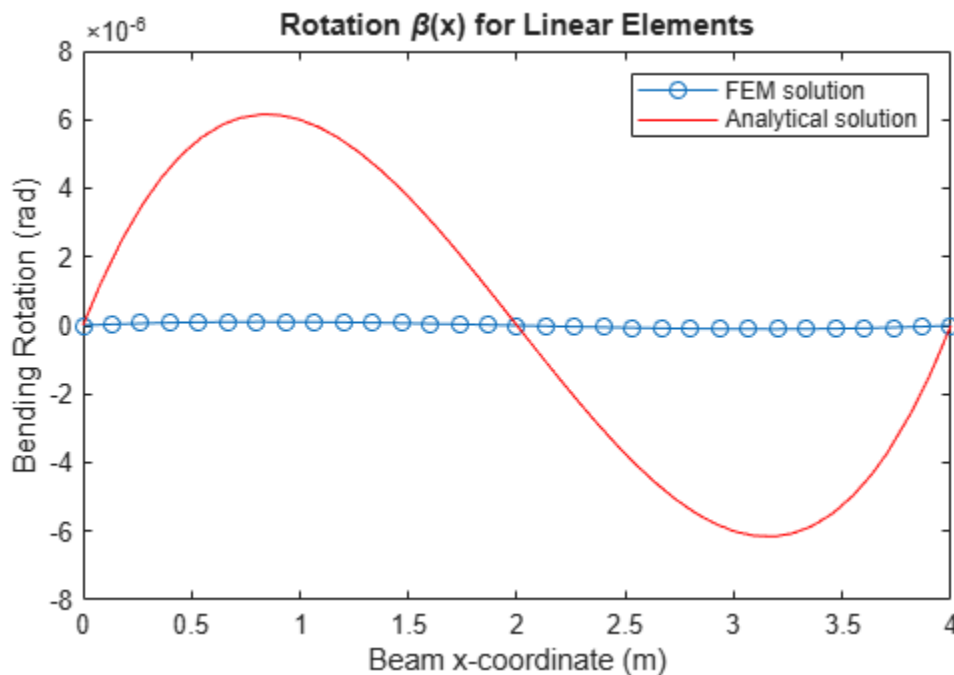
```
figure
plot(mshLin.nodes,u(1:2:end,1),"-o")
hold on
fplot(w_eval,[0 beamParams.L],"r-")
hold off
title("Displacement  $w(x)$  for Linear Elements")
xlabel("Beam x-coordinate (m)")
ylabel("Transverse Displacement (m)")
legend("FEM solution","Analytical solution",Location="southeast")
```





Plot the analytical and FEM solutions of the bending rotation angle  $\beta(x)$  of the beam. The FEM solution of the bending rotation angle is also the trivial solution  $\beta(x) \approx 0$ , which does not match the analytical solution.

```
figure
plot(mshLin.nodes,u(2:2:end,1),"-o")
hold on
fplot(beta_eval,[0 beamParams.L],"r-")
hold off
title("Rotation \beta(x) for Linear Elements")
xlabel("Beam x-coordinate (m)")
ylabel("Bending Rotation (rad)")
legend("FEM solution","Analytical solution")
```



To obtain better FEM solutions, construct the global stiffness matrix and load vector for the three-noded mesh elements with quadratic basis functions using the previously generated quadratic mesh.

```
[K,F] = assembleGlobalKF(mshQuad,Nquad,beamParams);
```

Define the fixed degrees of freedom for the beam that is fixed at both ends according to the boundary conditions. Define the free degrees of freedom of the mesh elements by excluding the fixed degrees of freedom.

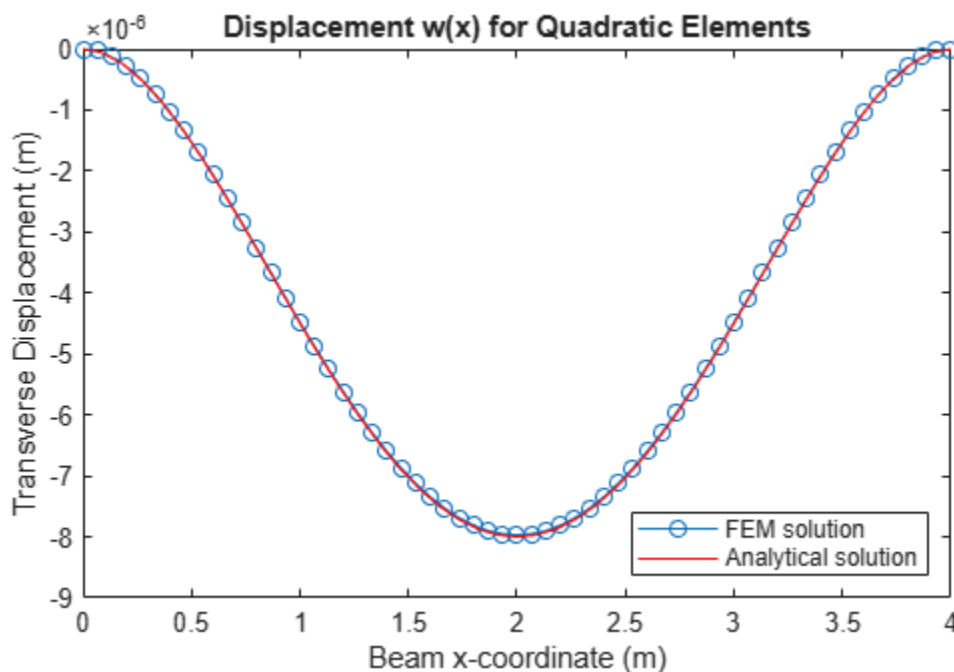
```
fixedDOFs = [1:2 mshQuad.numDOFs-1:mshQuad.numDOFs];
freeDOFs = 1:mshQuad.numDOFs;
freeDOFs = setdiff(freeDOFs,fixedDOFs);
```

Compute the nodal displacements of the beam from the global stiffness matrix and load vector.

```
u = zeros(mshQuad.numDOFs,1);
u(freeDOFs) = K(freeDOFs,freeDOFs)\F(freeDOFs);
```

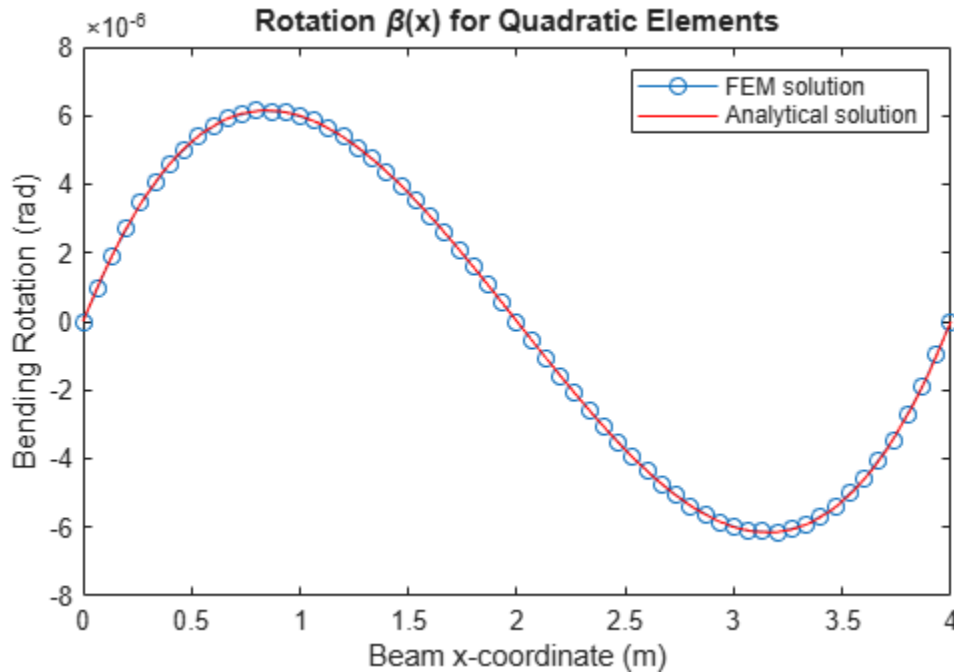
Plot the analytical and FEM solutions of the transverse displacement  $w(x)$  of the beam. The use of higher-order polynomials in the basis functions reduces the transverse shear locking effect, and the FEM solution matches the analytical solution.

```
figure
plot(mshQuad.nodes,u(1:2:end,1),"-o")
hold on
fplot(w_eval,[0 beamParams.L],"r-")
hold off
title("Displacement w(x) for Quadratic Elements")
xlabel("Beam x-coordinate (m)")
ylabel("Transverse Displacement (m)")
legend("FEM solution","Analytical solution",Location="southeast")
```



Plot the FEM solution of the bending rotation angle  $\beta(x)$  of the beam, which also matches the analytical solution.

```
figure
plot(mshQuad.nodes,u(2:2:end,1),"-o")
hold on
fplot(beta_eval,[0 beamParams.L],"r-")
hold off
title("Rotation \beta(x) for Quadratic Elements")
xlabel("Beam x-coordinate (m)")
ylabel("Bending Rotation (rad)")
legend("FEM solution","Analytical solution")
```



## References

- [1] Öchsner, Andreas. *Classical Beam Theories of Structural Mechanics*. Cham: Springer International Publishing, 2021. <https://doi.org/10.1007/978-3-030-76035-9>.
- [2] Hughes, Thomas JR. *The Finite Element Method: Linear Static and Dynamic Finite Element Analysis*. Dover Publications, 1st edition, 2000.
- [3] Bittencourt, Marco L. *Computational Solid Mechanics: Variational Formulation and High Order Approximation*. Boca Raton, FL: CRC Press, 2014. <https://doi.org/10.1201/b16392>.
- [4] Reddy, J N. "On the Dynamic Behaviour of the Timoshenko Beam Finite Elements." *Sadhana* 24, no. 3 (June 1999): 175-98. <https://doi.org/10.1007/BF02745800>.

## See Also

### Functions

`diff` | `subs` | `dsolve`

## Related Examples

- "Analytical Model of Cantilever Truss Structure for Simscape" on page 5-57
- "Solve Partial Differential Equation of Nonlinear Heat Transfer" on page 3-275

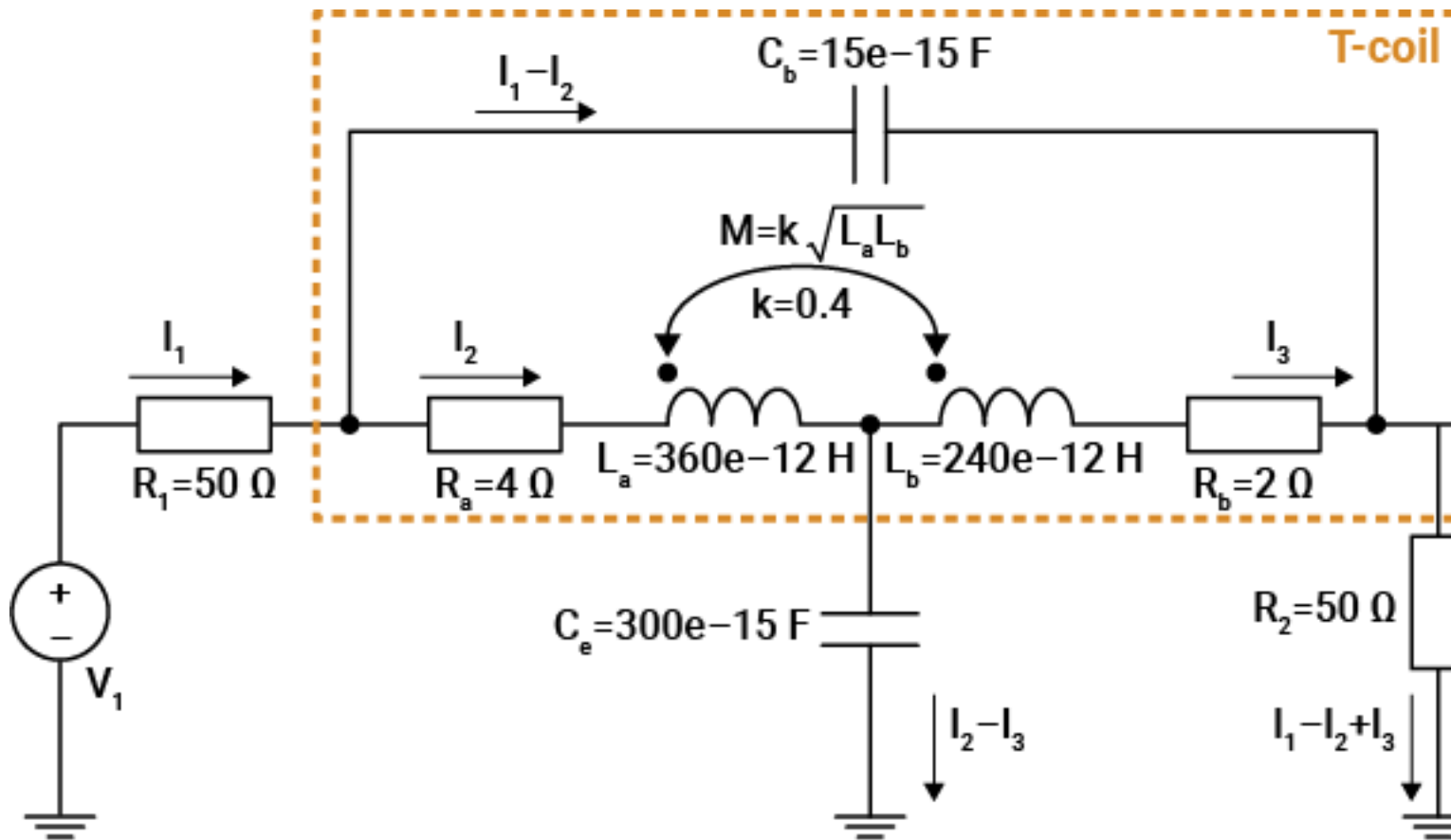
## More About

- Courseware on Finite Element Methods (MathWorks Teaching Resources)

## Analyze Transfer Function of T-Coil Circuit

This example shows how to analyze the transfer function of a T-coil circuit using Symbolic Math Toolbox™ and Control System Toolbox™. In this example, you define symbolic equations, solve for the transfer function, and analyze the stability, frequency response, and step response of the circuit.

The T-coil (or bridged T-coil) is a circuit topology that extends the signal bandwidth by a greater factor than inductive peaking does. The T-coil consists of two mutually coupled inductors and a bridge capacitor. This figure illustrates the T-coil circuit that you analyze in this example.



These variables describe the circuit:

- V1 is the input voltage.
- V2 is the output voltage.
- I1, I2, and I3 are the currents that flow on specific paths of the circuit.
- R1 is the resistor connected to the input voltage.
- R2 is the resistor connected to the output voltage.
- Ra is the parasitic resistance of the inductor La.
- Rb is the parasitic resistance of the inductor Lb.
- La is the first inductor in the T-coil.

- $L_b$  is the second inductor in the T-coil.
- $M$  is the mutual inductance between the inductors  $L_a$  and  $L_b$ , where  $k$  is the coefficient of coupling.
- $C_b$  is the bridge capacitor in the T-coil.
- $C_e$  is the capacitor that connects one of the T-coil terminals to the ground.

### Define Symbolic Variables and Equations

Define the symbolic variables for voltages, currents, resistances, inductances, mutual inductance, and capacitances. Additionally, define the Laplace variable  $s$ , which represents the complex frequency that describes the circuit in the Laplace domain.

```
syms V1 V2 I1 I2 I3
syms R1 R2 Ra Rb La Lb M Cb Ce s
```

Create a system of equations that represents the T-coil circuit. Apply Kirchhoff's voltage and current laws in the Laplace domain to obtain the system of equations.

```
eqn1 = V1 == R1*I1 + 1/(Cb*s)*(I1-I2) + V2;
eqn2 = V1 == R1*I1 + Ra*I2 + M*s*I3 + La*s*I2 + 1/(Ce*s)*(I2-I3);
eqn3 = V2 == R2*(I1-I2+I3);
eqn4 = V2 == -Rb*I3 - M*s*I2 - Lb*s*I3 + 1/(Ce*s)*(I2-I3);
eqns = [eqn1; eqn2; eqn3; eqn4]
```

$$\text{eqns} = \begin{pmatrix} V_1 = V_2 + I_1 R_1 + \frac{I_1 - I_2}{C_b s} \\ V_1 = I_1 R_1 + I_2 R_a + \frac{I_2 - I_3}{C_e s} + I_2 L_a s + I_3 M s \\ V_2 = R_2 (I_1 - I_2 + I_3) \\ V_2 = \frac{I_2 - I_3}{C_e s} - I_3 R_b - I_3 L_b s - I_2 M s \end{pmatrix}$$

### Analyze Transfer Function of Input Voltage to Output Voltage

The system of equations consists of four linearly independent equations, so you can solve for four unknowns from the equations. For this reason, explicitly specify the four variables you want to solve for when using `solve`. Solve the system of equations for the variables  $I_1$ ,  $I_2$ ,  $V_1$ , and  $V_2$ . The `solve` function returns the solutions as a structure in `result`.

```
result = solve(eqns,I1,I2,V1,V2);
```

Next, find the transfer function by calculating the ratio of the output voltage to the input voltage.

```
V2_over_V1 = result.V2/result.V1;
```

Define numerical values for the circuit parameters in SI units.

```
params.R1 = 50;
params.R2 = 50;
params.Ra = 4;
params.Rb = 2;
params.La = 360e-12;
params.Lb = 240e-12;
params.M = 0.4*sqrt(params.La*params.Lb);
```

```
params.Cb = 15e-15;
params.Ce = 300e-15;
```

Substitute the values for the circuit parameters into the transfer function. Use `vpa` to compute the transfer function to five digits of accuracy. Here, the transfer function is in symbolic form.

```
h_tcoil = vpa(subs(V2_over_V1,params),5)
h_tcoil =
    50.0 (3.2659e-46 s^4 + 7.56e-36 s^3 - 2.2709e-23 s^2 + 9.0e-14 s + 1.0)
    3.2659e-44 s^4 + 3.1924e-32 s^3 + 1.0828e-20 s^2 + 1.6866e-9 s + 106.0
```

To convert the transfer function to a numeric form, first extract the numerator and denominator of the transfer function by using `numden`.

```
[n,d] = numden(h_tcoil);
```

Convert the numerator and denominator to numeric vectors of polynomial coefficients by using `sym2poly`. Based on these coefficients, construct the transfer function model using Control System Toolbox. Use `minreal` to eliminate the uncontrollable state of the transfer function, or in other words, to cancel the pole-zero pairs in the transfer function.

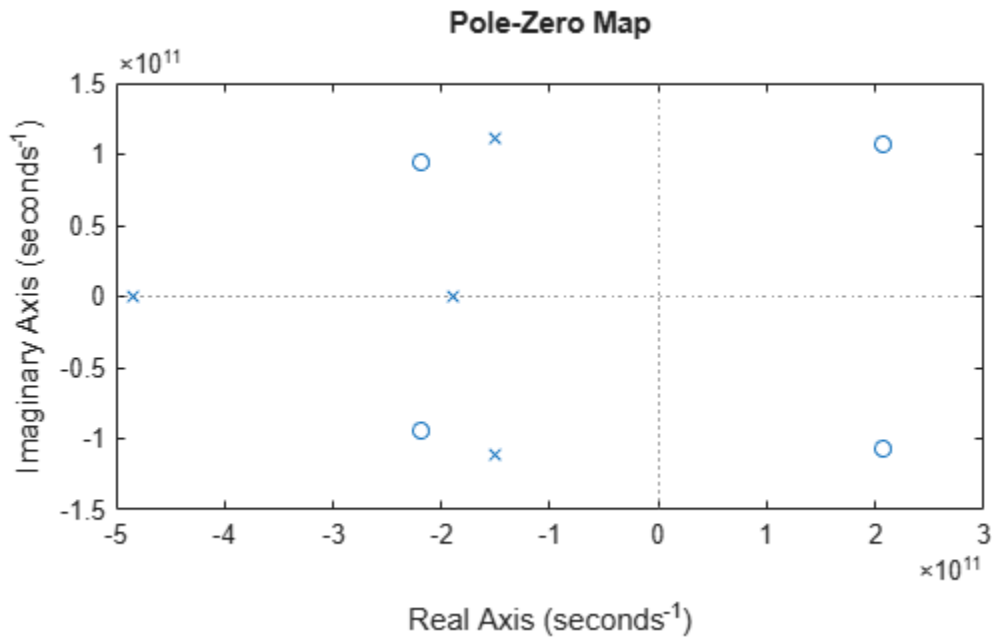
```
h_tcoil = tf(sym2poly(n),sym2poly(d));
h_tcoil = minreal(h_tcoil)
h_tcoil =
    0.5 s^4 + 1.157e10 s^3 - 3.477e22 s^2 + 1.378e32 s + 1.531e45
    -----
    s^4 + 9.775e11 s^3 + 3.315e23 s^2 + 5.164e34 s + 3.246e45
```

Continuous-time transfer function.  
Model Properties

### Pole-Zero Map

Plot the pole-zero map of the transfer function by using `pzmap`. This system has four complex zeros, marked by `o` on the plot. The system also has a pair of complex poles and two real poles, marked by `x`.

```
figure(1)
pzmap(h_tcoil)
```



To obtain the values of these poles and zeros, use `pzmap` with two output arguments.

```
[p,z] = pzmap(h_tcoil)
```

```
p = 4×1 complex  
1011 ×
```

```
-4.8499 + 0.0000i  
-1.9025 + 0.0000i  
-1.5113 + 1.1107i  
-1.5113 - 1.1107i
```

```
z = 4×1 complex  
1011 ×
```

```
2.0656 + 1.0722i  
2.0656 - 1.0722i  
-2.1813 + 0.9460i  
-2.1813 - 0.9460i
```

### Bode Plot

Create a Bode plot of the frequency response of the transfer function. The plot displays the magnitude (in dB) and phase (in degrees) of the system response as a function of frequency.

To obtain the values of the magnitude and the phase of the system response at specific frequencies, use the `bode` function with three output arguments. Because `bode` uses radians per second (rad/s) as the frequency units, define the frequencies where you want to compute the magnitude and phase in rad/s.

```
w = linspace(1e11,1e15,4000)*pi/180;  
[mag,phz,w] = bode(h_tcoil,w);
```

Convert the output frequencies from rad/s to Hz.

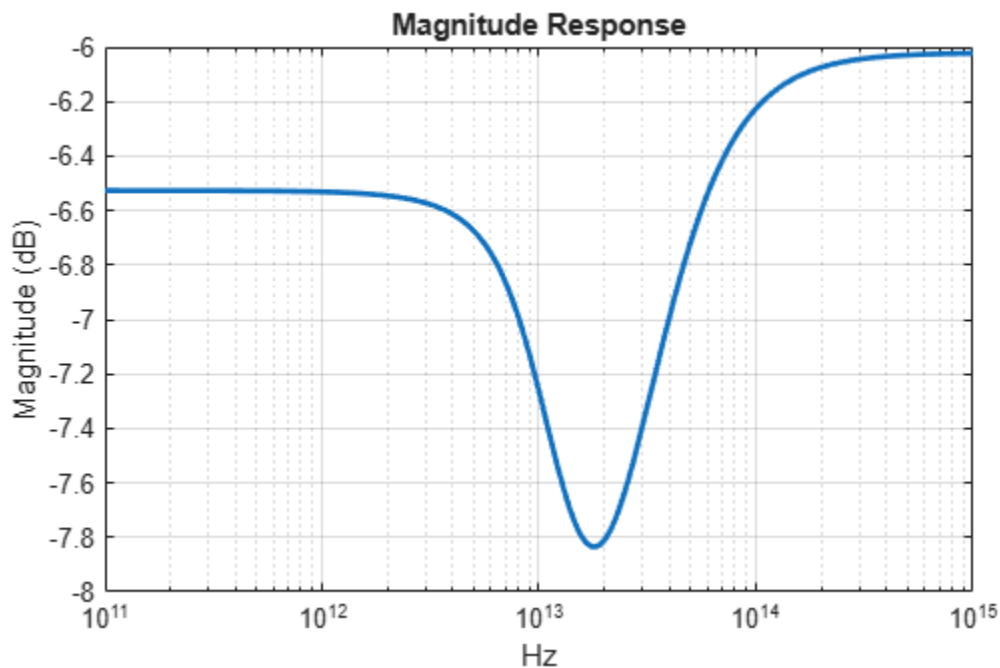
```
f = w*180/pi;
```

Convert the magnitude from absolute units to decibels.

```
magdb = 20*log10(mag(:));
```

Next, plot the magnitude response by using `semilogx`.

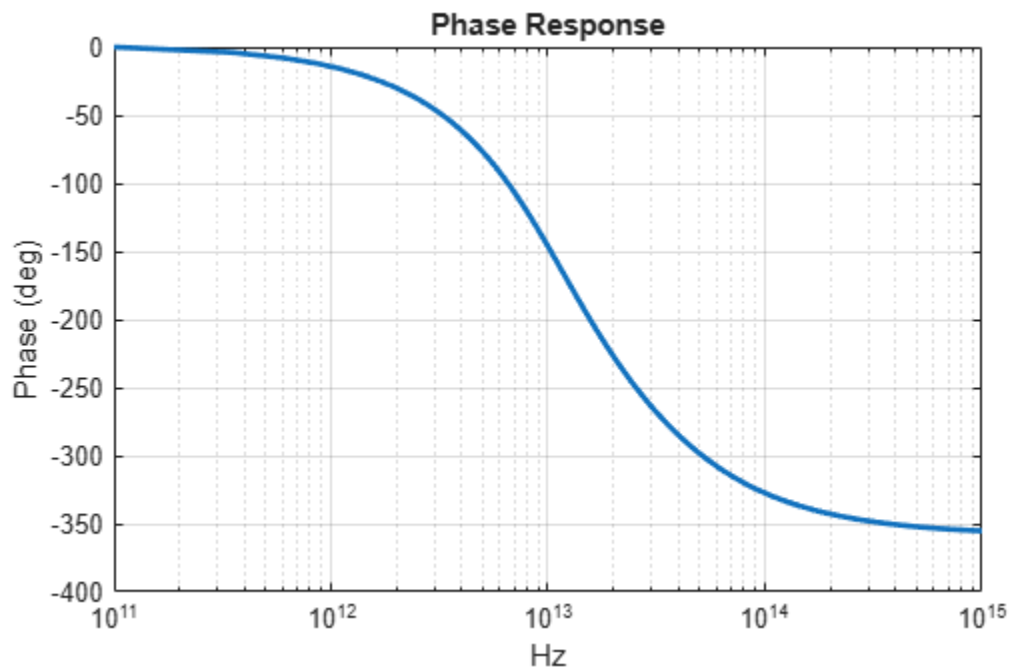
```
figure(2)
semilogx(f,magdb,LineWidth=2)
grid on
xlabel("Hz")
ylabel("Magnitude (dB)")
title("Magnitude Response")
```



Then, ensure the phase response is 0 degrees at low frequencies (DC) instead of a multiple of 360 degrees. Introducing an offset, `phz(1)`, to the phase response vector does not change the effective phase response, but it makes the plot more intuitive.

```
phz = phz(:) - phz(1);
figure(3)
semilogx(f,phz,LineWidth=2)
grid on
xlabel("Hz")
ylabel("Phase (deg)")
title("Phase Response")
```

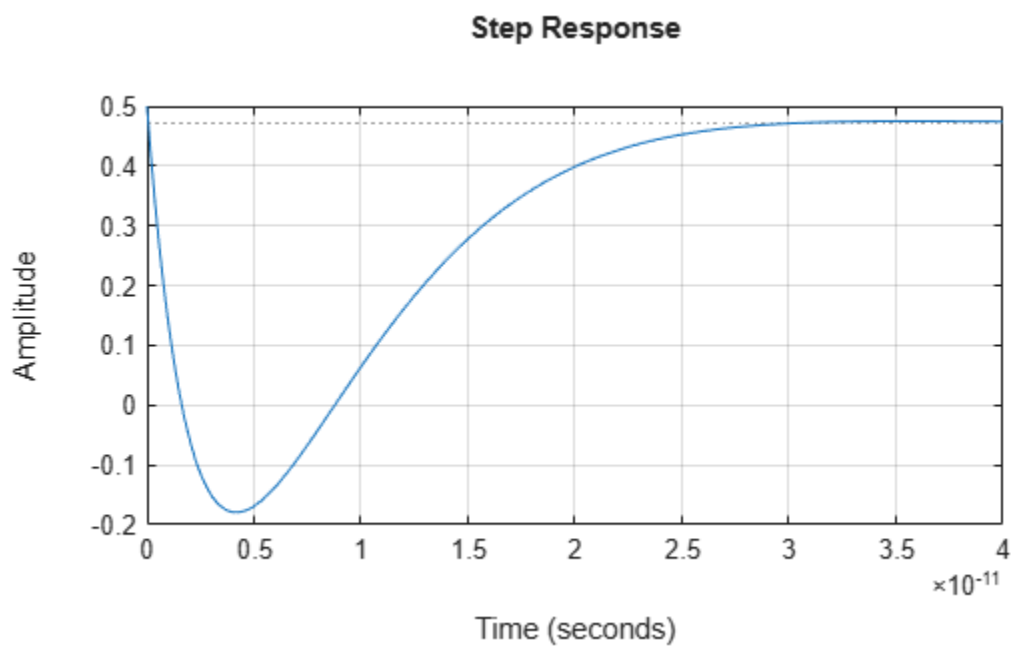




### Step Response

Plot the step response of the transfer function by using `step`.

```
figure(4)
step(h_tcoil)
grid on
```



### Compare Transfer Functions with and without Parasitic Resistances

In the previous sections, the parasitic resistances of the two inductors are assumed to be 4 ohms and 2 ohms. To compare the transfer function responses for an ideal case where the parasitic resistances are zero, you can repeat a similar process but substitute the values of Ra and Rb with 0 ohms.

```
params.Ra = 0;
params.Rb = 0;
```

Find the transfer function without parasitic resistances.

```
h_tcoil_ideal = vpa(subs(V2_over_V1,params),5)

h_tcoil_ideal =

$$\frac{50.0(3.2659e-46s^4 - 2.2745e-23s^2 + 1.0)}{3.2659e-44s^4 + 3.1168e-32s^3 + 1.0253e-20s^2 + 1.5852e-9s + 100.0}$$


[n,d] = numden(h_tcoil_ideal);
h_tcoil_ideal = tf(sym2poly(n),sym2poly(d));
h_tcoil_ideal = minreal(h_tcoil_ideal)

h_tcoil_ideal =

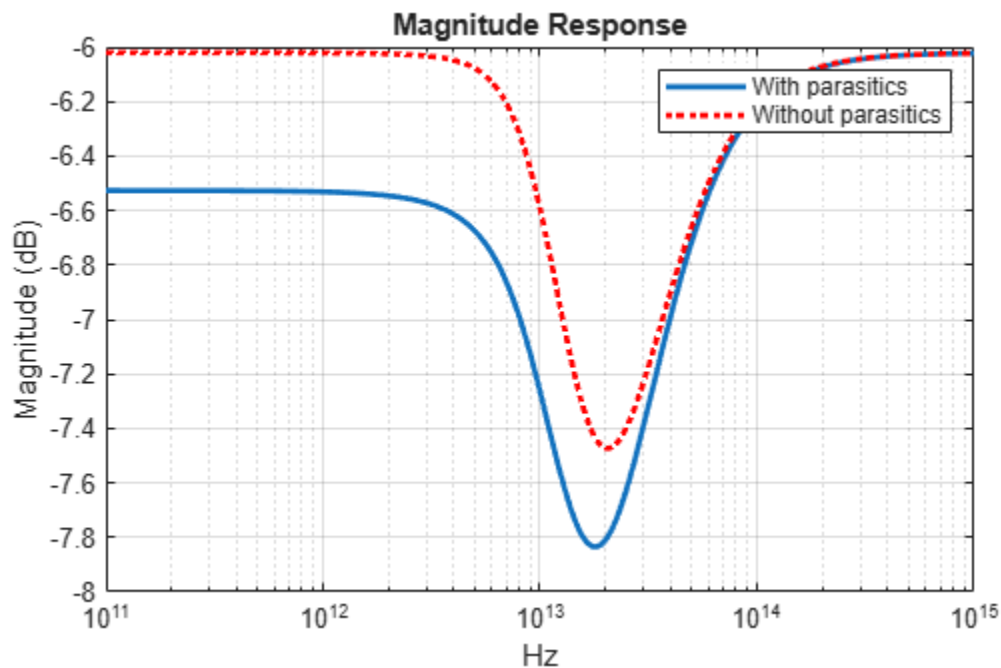
$$\frac{0.5s^4 + 0.0003357s^3 - 3.482e22s^2 - 1.729e19s + 1.531e45}{s^4 + 9.543e11s^3 + 3.139e23s^2 + 4.854e34s + 3.062e45}$$

```

Continuous-time transfer function.  
Model Properties

Next, plot the magnitude response of the transfer function without parasitic resistances.

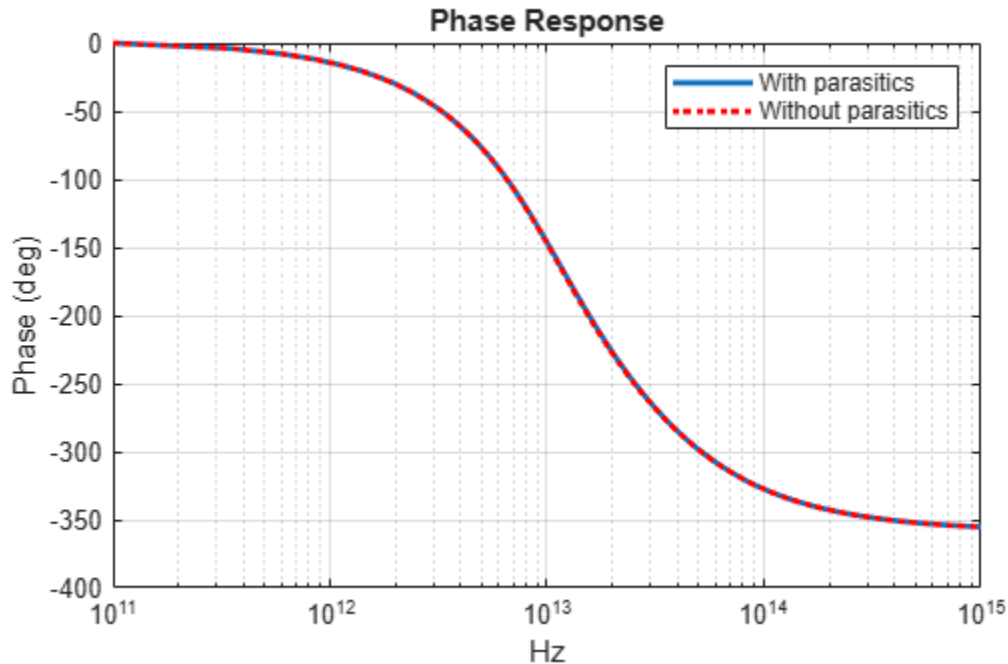
```
[mag_ideal,phz_ideal,w] = bode(h_tcoil_ideal,w);
f = w*180/pi;
magdb_ideal = 20*log10(mag_ideal(:));
figure(2)
hold on
semilogx(f,magdb_ideal,"r:",LineWidth=2)
legend("With parasitics","Without parasitics")
```



Here, you can see that the maximum attenuation of the T-coil occurs at a frequency of  $2.1 \times 10^{13}$  Hz for the case without parasitic resistances, compared to a frequency of  $1.78 \times 10^{13}$  Hz for the case with parasitic resistances. Additionally, for the case without parasitic resistances, the magnitude response at low and high frequencies is around -6 dB. Meanwhile, for the case with parasitic resistances, the magnitude response at low frequencies is around -6.53 dB.

Then, plot the phase response of the transfer function without parasitic resistances by introducing an offset to make the phase response equal to 0 at the minimum frequency range.

```
phz_ideal = phz_ideal(:) - phz_ideal(1);
figure(3)
hold on
semilogx(f,phz_ideal,"r:",LineWidth=2)
legend("With parasitics","Without parasitics")
```



The phase responses for the cases with and without parasitic resistances are very close to each other, within 1 degree.

### Analyze Transfer Function of Input Current to Output Voltage

Suppose you want to analyze the transfer function for the ratio of the output voltage V2 to the input current I1 (the transimpedance transfer function). You can compute this transfer function from the solutions of V2 and I1 that you obtained in the previous section. Calculate the ratio of `result.V2` to `result.I1`, and simplify the result. Using a similar process as in the previous sections, you can then analyze this transfer function using Control System Toolbox.

```
V2_over_I1 = simplify(result.V2/result.I1);
params.Ra = 4;
params.Rb = 2;
z_tcoil = vpa(subs(V2_over_I1,params),5)

z_tcoil =
    50.0 (3.2659e-46 s^4 + 7.56e-36 s^3 - 2.2709e-23 s^2 + 9.0e-14 s + 1.0)
    3.2659e-46 s^4 + 1.9547e-34 s^3 + 8.5913e-23 s^2 + 1.569e-11 s + 1.0

[n,d] = numden(z_tcoil);
z_tcoil = tf(sym2poly(n),sym2poly(d));
z_tcoil = minreal(z_tcoil)

z_tcoil =
    50 s^4 + 1.157e12 s^3 - 3.477e24 s^2 + 1.378e34 s + 1.531e47
    -----
    s^4 + 5.985e11 s^3 + 2.631e23 s^2 + 4.804e34 s + 3.062e45
```

Continuous-time transfer function.  
Model Properties

## References

- [1] Razavi, Behzad. "The Bridged T-Coil [A Circuit for All Seasons]." *IEEE Solid-State Circuits Magazine* 7, no. 4 (Fall 2015): 9-13. <https://doi.org/10.1109/MSSC.2015.2474258>.
- [2] Kossel, Marcel, Christian Menolfi, Jonas Weiss, Peter Buchmann, George von Bueren, Lucio Rodoni, Thomas Morf, Thomas Toifl, and Martin Schmatz. "A T-Coil-Enhanced 8.5Gb/s High-Swing Source-Series-Terminated Transmitter in 65nm Bulk CMOS." In *2008 IEEE International Solid-State Circuits Conference - Digest of Technical Papers*, 110-599. San Francisco, CA, USA: IEEE 2008. <https://doi.org/10.1109/ISSCC.2008.4523081>.

## See Also

### Functions

`solve` | `numden` | `vpa` | `tf` | `bode`

## Related Examples

- "Solve Differential Equations of RLC Circuit Using Laplace Transform" on page 3-198
- "Harmonic Analysis of Transfer Function Output" on page 3-160
- "Padé Approximant of Time-Delay Input" on page 3-261
- "Estimate Model Parameters of a Symbolically Derived Plant Model in Simulink" on page 5-74
- "Customize and Extend Simscape Libraries for a Custom DC Motor" on page 5-65

## External Websites

- Analyze T-Coil Circuit Using Symbolic Toolbox

## Symbolic Matrix Computation

This example shows how to perform simple matrix computations using Symbolic Math Toolbox™.

```
t = sym('t');
E = simplify(expm(t*A)) |
```

E =

$$\begin{pmatrix} -\frac{2t^3}{3} + \frac{11t^2}{2} - 9t + 1 & t & 7t^4 - 6 & -\frac{t(142t^3 - 1710t^2 + 5151t - 3450)}{6} & -\frac{t(1946t^3 - 128t(t^3 - 12t^2 + 36t - 24))}{3} \\ -\frac{t(7t^3 - 81t^2 + 230t - 140)}{2} & 7t^4 - 6 & -\frac{t(142t^3 - 1710t^2 + 5151t - 3450)}{6} & -\frac{t(1946t^3 - 128t(t^3 - 12t^2 + 36t - 24))}{3} & \frac{256t}{3} \end{pmatrix}$$

Generate a possibly familiar test matrix, the 5-by-5 Hilbert matrix.

```
H = sym(hilb(5))
```

H =

$$\begin{pmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} \\ \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} \\ \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} & \frac{1}{9} \end{pmatrix}$$

The determinant is very small.

```
d = det(H)
```

d =

$$\frac{1}{266716800000}$$

The elements of the inverse are integers.

```
X = inv(H)
```

$$X = \begin{pmatrix} 25 & -300 & 1050 & -1400 & 630 \\ -300 & 4800 & -18900 & 26880 & -12600 \\ 1050 & -18900 & 79380 & -117600 & 56700 \\ -1400 & 26880 & -117600 & 179200 & -88200 \\ 630 & -12600 & 56700 & -88200 & 44100 \end{pmatrix}$$

Verify that the inverse is correct.

```
I = X*H
```

$$I = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

Find the characteristic polynomial.

```
syms x; p = charpoly(H,x)
```

$$p = x^5 - \frac{563x^4}{315} + \frac{735781x^3}{2116800} - \frac{852401x^2}{222264000} + \frac{61501x}{53343360000} - \frac{1}{266716800000}$$

Try to factor the characteristic polynomial.

```
factor(p)
```

$$\text{ans} = \left( \frac{1}{266716800000} 266716800000 x^5 - 476703360000 x^4 + 92708406000 x^3 - 1022881200 x^2 + 307505 x - 1 \right)$$

The result indicates that the characteristic polynomial cannot be factored over the rational numbers.

Compute the 50 digit numerical approximations to the eigenvalues.

```
digits(50)
e = eig(vpa(H))
```

$$e = \begin{pmatrix} 1.5670506910982307955330110055207246339493152522334 \\ 0.20853421861101333590500251006882005503858202260343 \\ 0.01140749162341980655945145886658934504234843052664 \\ 0.00030589804015119172687949784069272282565614514909247 \\ 0.0000032879287721718629571150047605447313997367890230746 \end{pmatrix}$$

Create a generalized Hilbert matrix involving a free variable,  $t$ .

```
t = sym('t');
[I,J] = meshgrid(1:5);
H = 1./(I+J-t)
```

```
H =
```

$$\begin{pmatrix} -\frac{1}{t-2} & -\frac{1}{t-3} & \sigma_5 & \sigma_3 & \sigma_1 \\ -\frac{1}{t-3} & \sigma_5 & \sigma_3 & \sigma_1 & \sigma_2 \\ \sigma_5 & \sigma_3 & \sigma_1 & \sigma_2 & \sigma_4 \\ \sigma_3 & \sigma_1 & \sigma_2 & \sigma_4 & -\frac{1}{t-9} \\ \sigma_1 & \sigma_2 & \sigma_4 & -\frac{1}{t-9} & -\frac{1}{t-10} \end{pmatrix}$$

where

$$\sigma_1 = -\frac{1}{t-6}$$

$$\sigma_2 = -\frac{1}{t-7}$$

$$\sigma_3 = -\frac{1}{t-5}$$

$$\sigma_4 = -\frac{1}{t-8}$$

$$\sigma_5 = -\frac{1}{t-4}$$

Substituting  $t = 1$  retrieves the original Hilbert matrix.

`subs(H,t,1)`

$$\text{ans} = \begin{pmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} \\ \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} \\ \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} & \frac{1}{9} \end{pmatrix}$$

The reciprocal of the determinant is a polynomial in  $t$ .

`d = 1/det(H)`

$$d = -\frac{(t-2)(t-3)^2(t-4)^3(t-5)^4(t-6)^5(t-7)^4(t-8)^3(t-9)^2(t-10)}{82944}$$

`d = expand(d)`

`d =`



$$\begin{aligned}
& -\frac{t^{25}}{82944} + \frac{25t^{24}}{13824} - \frac{5375t^{23}}{41472} + \frac{40825t^{22}}{6912} - \frac{15940015t^{21}}{82944} + \frac{21896665t^{20}}{4608} - \frac{240519875t^{19}}{2592} \\
& + \frac{1268467075t^{18}}{864} - \frac{1588946776255t^{17}}{82944} + \frac{2885896606895t^{16}}{13824} - \frac{79493630114675t^{15}}{41472} \\
& + \frac{34372691161375t^{14}}{2304} - \frac{8194259295156385t^{13}}{82944} + \frac{7707965729450845t^{12}}{13824} - \frac{55608098247105175t^{11}}{20736} \\
& + \frac{37909434298793825t^{10}}{3456} - \frac{197019820623693025t^9}{5184} + \frac{10640296363350955t^8}{96} \\
& - \frac{38821472549340925t^7}{144} + \frac{12958201048605475t^6}{24} - \frac{1748754621252377t^5}{2} + 1115685328012530t^4 \\
& - 1078920141906600t^3 + 742618453752000t^2 - 323874210240000t + 67212633600000
\end{aligned}$$

The elements of the inverse are also polynomials in  $t$ .

`X = inv(H)`

`X =`

$$\begin{pmatrix}
-\frac{(t-2)(t-3)(t-4)(t-5)(t-6)\sigma_4}{576} & \frac{(t-3)(t-4)(t-5)(t-6)(t-7)(t^4-17t^3+33t^2-21t+14)}{144} \\
\frac{(t-2)(t-3)(t-4)(t-5)(t-6)\sigma_3}{144} & -\frac{(t-3)(t-4)(t-5)(t-6)(t-7)(t^4-21t^3+33t^2-21t+14)}{36} \\
-\frac{(t-2)(t-3)(t-4)(t-5)(t-6)\sigma_2}{96} & \frac{(t-3)(t-4)(t-5)(t-6)(t-7)(t^4-25t^3+33t^2-21t+14)}{24} \\
\frac{(t-2)(t-3)(t-4)(t-5)(t-6)\sigma_1}{144} & -\frac{(t-3)(t-4)(t-5)(t-6)(t-7)(t^4-29t^3+33t^2-21t+14)}{36} \\
-\frac{(t-2)(t-3)(t-4)(t-5)(t-6)(t^4-34t^3+43t^2-2414t+5040)}{576} & \frac{(t-3)(t-4)(t-5)(t-6)(t-7)(t^4-33t^3+43t^2-21t+14)}{144}
\end{pmatrix}$$

where

$$\sigma_1 = t^4 - 30t^3 + 335t^2 - 1650t + 3024$$

$$\sigma_2 = t^4 - 26t^3 + 251t^2 - 1066t + 1680$$

$$\sigma_3 = t^4 - 22t^3 + 179t^2 - 638t + 840$$

$$\sigma_4 = t^4 - 18t^3 + 119t^2 - 342t + 360$$

Substituting  $t = 1$  generates the Hilbert inverse.

`X = subs(X,t,'1')`

`X =`

$$\begin{pmatrix}
25 & -300 & 1050 & -1400 & 630 \\
-300 & 4800 & -18900 & 26880 & -12600 \\
1050 & -18900 & 79380 & -117600 & 56700 \\
-1400 & 26880 & -117600 & 179200 & -88200 \\
630 & -12600 & 56700 & -88200 & 44100
\end{pmatrix}$$

`X = double(X)`

`X = 5×5`

|       |        |         |         |        |
|-------|--------|---------|---------|--------|
| 25    | -300   | 1050    | -1400   | 630    |
| -300  | 4800   | -18900  | 26880   | -12600 |
| 1050  | -18900 | 79380   | -117600 | 56700  |
| -1400 | 26880  | -117600 | 179200  | -88200 |
| 630   | -12600 | 56700   | -88200  | 44100  |

Investigate a different example.

```
A = sym(gallery(5))
```

```
A =
```

$$\begin{pmatrix} -9 & 11 & -21 & 63 & -252 \\ 70 & -69 & 141 & -421 & 1684 \\ -575 & 575 & -1149 & 3451 & -13801 \\ 3891 & -3891 & 7782 & -23345 & 93365 \\ 1024 & -1024 & 2048 & -6144 & 24572 \end{pmatrix}$$

This matrix is "nilpotent". Its fifth power is the zero matrix.

```
A^5
```

```
ans =
```

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Because this matrix is nilpotent, its characteristic polynomial is very simple.

```
p = charpoly(A, 'lambda')
```

```
p = λ5
```

You should now be able to compute the matrix eigenvalues in your head. They are the zeros of the equation  $\lambda^5 = 0$ .

Symbolic computation can find the eigenvalues exactly.

```
lambda = eig(A)
```

```
lambda =
```

$$\begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Numeric computation involves roundoff error and finds the zeros of an equation that is something like  $\lambda^5 = \text{eps} \cdot \text{norm}(A)$ . So the computed eigenvalues are roughly  $\lambda = (\text{eps} \cdot \text{norm}(A))^{1/5}$ . Here are the eigenvalues, computed by the Symbolic Toolbox using 16 digit floating point arithmetic. It is not obvious that they should all be zero.

```
digits(16)
lambda = eig(vpa(A))
```

$$\text{lambda} = \begin{pmatrix} 0.0005617448486395847 \\ 0.0001737348850386136 - 0.0005342985684139864 i \\ 0.0001737348850386136 + 0.0005342985684139864 i \\ -0.000454607309358406 + 0.0003303865815979566 i \\ -0.000454607309358406 - 0.0003303865815979566 i \end{pmatrix}$$

This matrix is also "defective". It is not similar to a diagonal matrix. Its Jordan Canonical Form is not diagonal.

$$J = \text{jordan}(A)$$

$$J = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

The matrix exponential,  $\text{expm}(t*A)$ , is usually expressed in terms of scalar exponentials involving the eigenvalues,  $\text{exp}(\text{lambda}(i)*t)$ . But for this matrix, the elements of  $\text{expm}(t*A)$  are all polynomials in  $t$ .

$$t = \text{sym}('t'); \\ E = \text{simplify}(\text{expm}(t*A))$$

$$E = \begin{pmatrix} -\frac{2t^3}{3} + \frac{11t^2}{2} - 9t + 1 & \frac{t(4t^2 - 27t + 33)}{3} & -\frac{t(20t^2 - 117t + 126)}{6} \\ -\frac{t(7t^3 - 81t^2 + 230t - 140)}{2} & 7t^4 - 67t^3 + \frac{301t^2}{2} - 69t + 1 & -\frac{t(35t^3 - 293t^2 + 598t - 282)}{2} \\ \frac{t(142t^3 - 1710t^2 + 5151t - 3450)}{6} & -\frac{t(142t^3 - 1426t^2 + 3438t - 1725)}{3} & \frac{355t^4}{3} - \frac{3139t^3}{3} + \frac{4585t^2}{2} - 1149t \\ -\frac{t(973t^3 - 11675t^2 + 35022t - 23346)}{6} & \frac{t(1946t^3 - 19458t^2 + 46695t - 23346)}{6} & -\frac{t(4865t^3 - 42807t^2 + 93390t - 48348)}{6} \\ -\frac{128t(t^3 - 12t^2 + 36t - 24)}{3} & \frac{256t(t^3 - 10t^2 + 24t - 12)}{3} & -\frac{128t(5t^3 - 44t^2 + 96t - 48)}{3} \end{pmatrix}$$

By the way, the function "exp" computes element-by-element exponentials.

$$X = \exp(t \cdot A)$$

$$X = \begin{pmatrix} e^{-9t} & e^{11t} & e^{-21t} & e^{63t} & e^{-252t} \\ e^{70t} & e^{-69t} & e^{141t} & e^{-421t} & e^{1684t} \\ e^{-575t} & e^{575t} & e^{-1149t} & e^{3451t} & e^{-13801t} \\ e^{3891t} & e^{-3891t} & e^{7782t} & e^{-23345t} & e^{93365t} \\ e^{1024t} & e^{-1024t} & e^{2048t} & e^{-6144t} & e^{24572t} \end{pmatrix}$$

## Linear Algebraic Operations

### In this section...

“Symbolic Hilbert Matrix” on page 3-341  
 “Symbolic Linear Algebra Operations” on page 3-341  
 “Variable-Precision Arithmetic” on page 3-342  
 “Symbolic Investigation of Singular Value” on page 3-343

### Symbolic Hilbert Matrix

The following examples, which show how to perform basic linear algebraic operations, are based on a symbolic version of the 3-by-3 Hilbert matrix.

Generate the 3-by-3 Hilbert matrix. With `format short`, MATLAB prints the output shown.

```
H = hilb(3)

H =
    1.0000    0.5000    0.3333
    0.5000    0.3333    0.2500
    0.3333    0.2500    0.2000
```

The computed elements of `H` are floating-point numbers that are the ratios of small integers. `H` is a MATLAB array of class `double`.

Convert `H` to a symbolic matrix.

```
H = sym(H)

H =
[ 1, 1/2, 1/3]
[ 1/2, 1/3, 1/4]
[ 1/3, 1/4, 1/5]
```

### Symbolic Linear Algebra Operations

Symbolic operations on `H` produce results that correspond to the infinite-precision Hilbert matrix, `sym(hilb(3))`, not its floating-point approximation, `hilb(3)`.

Find the inverse of `H`.

```
inv(H)

ans =
[ 9, -36, 30]
[ -36, 192, -180]
[ 30, -180, 180]
```

Find the determinant of `H`.

```
det(H)

ans =
1/2160
```

You can use the backslash operator to solve a system of simultaneous linear equations. For example, solve  $Hx = b$ .

```
b = [1; 1; 1];  
x = H\b
```

```
x =  
 3  
-24  
 30
```

All three results—the inverse, the determinant, and the solution to the linear system—are the exact results corresponding to the infinite-precision, rational, Hilbert matrix.

## Variable-Precision Arithmetic

Contrast the preceding operations with variable-precision arithmetic using 20 digits of precision.

```
digits(20)  
V = vpa(H)
```

```
V =
```

```
[ 1.0, 0.5, 0.33333333333333333333]  
[ 0.5, 0.33333333333333333333, 0.25]  
[ 0.33333333333333333333, 0.25, 0.2]
```

The decimal points in the representation of the individual elements indicate that MATLAB is using variable-precision arithmetic. The result of each arithmetic operation is rounded to 20 significant decimal digits.

Invert the matrix and note that errors are magnified by the matrix condition number, which for `hilb(3)` is about 500.

```
cond(V)
```

```
ans =
```

```
524.0567775860608
```

Compute the difference of the inverses of the infinite-precision and variable-precision versions.

```
ih = inv(H)
```

```
ih =
```

```
[ 9, -36, 30]  
[-36, 192, -180]  
[ 30, -180, 180]
```

```
iv = inv(V)
```

```
iv =
```

```
[ 9.0, -36.0, 30.0]  
[-36.0, 192.0, -180.0]  
[ 30.0, -180.0, 180.0]
```

Although these matrices look the same, calculate the difference to see that they are not.

```
dhv = ih - iv
```

```
dhv =
```

```
[ -5.4929962552349494034e-26,  2.4556924435168009098e-25, -2.1971985020939797614e-25]
[  2.4556924435168009098e-25, -1.2666203129718236271e-24,  1.1373733422604130529e-24]
[ -2.1971985020939797614e-25,  1.1373733422604130529e-24, -1.0856745539758488233e-24]
```

Solve the equation  $V*y = b$ . The answer looks the same as the solution to  $H*x = b$ .

```
y = V\b
```

```
y =
```

```
    3.0
   -24.0
    30.0
```

Calculate the difference between  $x$  and  $y$  to see the small difference between the two solutions.

```
x-y
```

```
ans =
```

```
    8.0779356694631608874e-27
   -6.4623485355705287099e-26
    7.1085833891275815809e-26
```

Using `vpa` with `digits(16)` offers comparable precision to using standard double-precision MATLAB routines.

## Symbolic Investigation of Singular Value

Find a value  $s$  for  $H(1,1)$  that makes  $H$  singular.

```
syms s
Hs = H;
Hs(1,1) = s
Z = det(Hs)
sol = solve(Z)
```

```
Hs =
[  s, 1/2, 1/3]
[ 1/2, 1/3, 1/4]
[ 1/3, 1/4, 1/5]
```

```
Z =
s/240 - 1/270
```

```
sol =
8/9
```

Substitute the solution for  $s$  into  $Hs$ .

```
Hs = subs(Hs, s, sol)
```

```
Hs =  
[ 8/9, 1/2, 1/3]  
[ 1/2, 1/3, 1/4]  
[ 1/3, 1/4, 1/5]
```

Verify that the determinant of Hs is zero.

```
det(Hs)
```

```
ans =  
0
```

Find the null space and column space of Hs. Both spaces are nontrivial.

```
N = null(Hs)  
C = colspace(Hs)
```

```
N=  
3/10  
-6/5  
1
```

```
C =  
[ 1, 0]  
[ 0, 1]  
[ -3/10, 6/5]
```

Check that N is in the null space of Hs.

```
Hs*N
```

```
ans =  
  
0  
0  
0
```



## Basic Algebraic Operations

Basic algebraic operations on symbolic objects are the same as operations on MATLAB objects of class `double`. This is illustrated in the following example.

The Givens transformation produces a plane rotation through the angle `t`. The statements

```
syms t
G = [cos(t) sin(t); -sin(t) cos(t)]
```

create this transformation matrix.

```
G =
[ cos(t),  sin(t)]
[ -sin(t),  cos(t)]
```

Applying the Givens transformation twice should simply be a rotation through twice the angle. The corresponding matrix can be computed by multiplying `G` by itself or by raising `G` to the second power. Both

```
A = G*G
```

and

```
A = G^2
```

produce

```
A =
[ cos(t)^2 - sin(t)^2,      2*cos(t)*sin(t)]
[ -2*cos(t)*sin(t), cos(t)^2 - sin(t)^2]
```

The `simplify` function

```
A = simplify(A)
```

uses a trigonometric identity to return the expected form by trying several different identities and picking the one that produces the shortest representation.

```
A =
[ cos(2*t),  sin(2*t)]
[ -sin(2*t),  cos(2*t)]
```

The Givens rotation is an orthogonal matrix, so its transpose is its inverse. Confirming this by

```
I = G.' * G
```

which produces

```
I =
[ cos(t)^2 + sin(t)^2,      0]
[      0, cos(t)^2 + sin(t)^2]
```

and then

```
I = simplify(I)
```

$$I = \begin{bmatrix} 1, & 0 \\ 0, & 1 \end{bmatrix}$$

## Singular Value Decomposition

Singular value decomposition expresses an  $m$ -by- $n$  matrix  $A$  as  $A = U*S*V'$ . Here,  $S$  is an  $m$ -by- $n$  diagonal matrix with singular values of  $A$  on its diagonal. The columns of the  $m$ -by- $m$  matrix  $U$  are the left singular vectors for corresponding singular values. The columns of the  $n$ -by- $n$  matrix  $V$  are the right singular vectors for corresponding singular values.  $V'$  is the Hermitian transpose (the complex conjugate of the transpose) of  $V$ .

To compute the singular value decomposition of a matrix, use `svd`. This function lets you compute singular values of a matrix separately or both singular values and singular vectors in one function call. To compute singular values only, use `svd` without output arguments

```
svd(A)
```

or with one output argument

```
S = svd(A)
```

To compute singular values and singular vectors of a matrix, use three output arguments:

```
[U,S,V] = svd(A)
```

`svd` returns two unitary matrices,  $U$  and  $V$ , the columns of which are singular vectors. It also returns a diagonal matrix,  $S$ , containing singular values on its diagonal. The elements of all three matrices are floating-point numbers. The accuracy of computations is determined by the current setting of `digits`.

Create the  $n$ -by- $n$  matrix  $A$  with elements defined by  $A(i,j) = 1/(i - j + 1/2)$ . The most obvious way of generating this matrix is

```
n = 3;
for i = 1:n
    for j = 1:n
        A(i,j) = sym(1/(i-j+1/2));
    end
end
```

For  $n = 3$ , the matrix is

$A$

```
A =
[ 2, -2, -2/3]
[ 2/3, 2, -2]
[ 2/5, 2/3, 2]
```

Compute the singular values of this matrix. If you use `svd` directly, it will return exact symbolic result. For this matrix, the result is very long. If you prefer a shorter numeric result, convert the elements of  $A$  to floating-point numbers using `vpa`. Then use `svd` to compute singular values of this matrix using variable-precision arithmetic:

```
S = svd(vpa(A))
```

```
S =
3.1387302525015353960741348953506
3.0107425975027462353291981598225
1.6053456783345441725883965978052
```

Now, compute the singular values and singular vectors of A:

`[U,S,V] = svd(A)`

```
U =  
[ 0.53254331027335338470683368360204, 0.76576895948802052989304092179952,...  
  0.36054891952096214791189887728353]  
[ -0.82525689650849463222502853672224, 0.37514965283965451993171338605042,...  
  0.42215375485651489522488031917364]  
[ 0.18801243961043281839917114171742, -0.52236064041897439447429784257224,...  
  0.83173955292075192178421874331406]  
  
S =  
[ 3.1387302525015353960741348953506, 0,...  
  0]  
[ 0, 3.0107425975027462353291981598225,...  
  0]  
[ 0, 0,...  
  1.6053456783345441725883965978052]  
  
V =  
[ 0.18801243961043281839917114171742, 0.52236064041897439447429784257224,...  
  0.83173955292075192178421874331406]  
[ -0.82525689650849463222502853672224, -0.37514965283965451993171338605042,...  
  0.42215375485651489522488031917364]  
[ 0.53254331027335338470683368360204, -0.76576895948802052989304092179952,...  
  0.36054891952096214791189887728353]
```

## Eigenvalues

The symbolic eigenvalues of a square matrix  $A$  or the symbolic eigenvalues and eigenvectors of  $A$  are computed, respectively, using the commands  $E = \text{eig}(A)$  and  $[V,E] = \text{eig}(A)$ .

The variable-precision counterparts are  $E = \text{eig}(\text{vpa}(A))$  and  $[V,E] = \text{eig}(\text{vpa}(A))$ .

The eigenvalues of  $A$  are the zeros of the characteristic polynomial of  $A$ ,  $\det(A - xI)$ , which is computed by  $\text{charpoly}(A)$ .

The matrix  $H$  from the last section provides the first example:

```
H = sym([8/9 1/2 1/3; 1/2 1/3 1/4; 1/3 1/4 1/5])
```

```
H =
[ 8/9, 1/2, 1/3]
[ 1/2, 1/3, 1/4]
[ 1/3, 1/4, 1/5]
```

The matrix is singular, so one of its eigenvalues must be zero. The statement

```
[T,E] = eig(H)
```

produces the matrices  $T$  and  $E$ . The columns of  $T$  are the eigenvectors of  $H$  and the diagonal elements of  $E$  are the eigenvalues of  $H$ :

```
T =
[ 3/10, 218/285 - (4*12589^(1/2))/285, (4*12589^(1/2))/285 + 218/285]
[ -6/5, 292/285 - 12589^(1/2)/285, 12589^(1/2)/285 + 292/285]
[ 1, 1, 1]

E =
[ 0, 0, 0]
[ 0, 32/45 - 12589^(1/2)/180, 0]
[ 0, 0, 12589^(1/2)/180 + 32/45]
```

It may be easier to understand the structure of the matrices of eigenvectors,  $T$ , and eigenvalues,  $E$ , if you convert  $T$  and  $E$  to decimal notation. To do so, proceed as follows. The commands

```
Td = double(T)
Ed = double(E)
```

```
return
```

```
Td =
0.3000    -0.8098    2.3397
-1.2000    0.6309    1.4182
1.0000    1.0000    1.0000

Ed =
0         0         0
0    0.0878    0
0         0    1.3344
```

The first eigenvalue is zero. The corresponding eigenvector (the first column of  $Td$ ) is the same as the basis for the null space found in the last section. The other two eigenvalues are the result of applying the quadratic formula to  $x^2 - \frac{64}{45}x + \frac{253}{2160}$  which is the quadratic factor in  $\text{factor}(\text{charpoly}(H, x))$ :

```
syms x
g = factor(charpoly(H, x))/x
solve(g(3))

g =
[ 1/(2160*x), 1, (2160*x^2 - 3072*x + 253)/x]
ans =
32/45 - 12589^(1/2)/180
12589^(1/2)/180 + 32/45
```

Closed form symbolic expressions for the eigenvalues are possible only when the characteristic polynomial can be expressed as a product of rational polynomials of degree four or less. The Rosser matrix is a classic numerical analysis test matrix that illustrates this requirement. The statement

```
R = sym(rosser)
```

generates

```
R =
[ 611, 196, -192, 407, -8, -52, -49, 29]
[ 196, 899, 113, -192, -71, -43, -8, -44]
[ -192, 113, 899, 196, 61, 49, 8, 52]
[ 407, -192, 196, 611, 8, 44, 59, -23]
[ -8, -71, 61, 8, 411, -599, 208, 208]
[ -52, -43, 49, 44, -599, 411, 208, 208]
[ -49, -8, 8, 59, 208, 208, 99, -911]
[ 29, -44, 52, -23, 208, 208, -911, 99]
```

The commands

```
p = charpoly(R, x);
factor(p)
```

produce

```
ans =
[ x, x - 1020, x^2 - 1040500, x^2 - 1020*x + 100, x - 1000, x - 1000]
```

The characteristic polynomial (of degree 8) factors nicely into the product of two linear terms and three quadratic terms. You can see immediately that four of the eigenvalues are 0, 1020, and a double root at 1000. The other four roots are obtained from the remaining quadratics. Use

```
eig(R)
```

to find all these values

```
ans =
0
1000
1000
1020
510 - 100*26^(1/2)
100*26^(1/2) + 510
-10*10405^(1/2)
10*10405^(1/2)
```

The Rosser matrix is not a typical example; it is rare for a full 8-by-8 matrix to have a characteristic polynomial that factors into such simple form. If you change the two “corner” elements of R from 29 to 30 with the commands

```
S = R;
S(1,8) = 30;
S(8,1) = 30;
```

and then try

```
p = charpoly(S, x)
```

you find

```
p =
x^8 - 4040*x^7 + 5079941*x^6 + 82706090*x^5...
- 5327831918568*x^4 + 4287832912719760*x^3...
- 1082699388411166000*x^2 + 51264008540948000*x...
+ 40250968213600000
```

You also find that `factor(p)` is `p` itself. That is, the characteristic polynomial cannot be factored over the rationals.

For this modified Rosser matrix

```
F = eig(S)
```

returns

```
F =
-1020.053214255892
-0.17053529728769
0.2180398054830161
999.9469178604428
1000.120698293384
1019.524355263202
1019.993550129163
1020.420188201505
```

Notice that these values are close to the eigenvalues of the original Rosser matrix.

It is also possible to try to compute eigenvalues of symbolic matrices, but closed form solutions are rare. The Givens transformation is generated as the matrix exponential of the elementary matrix

$$A = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}.$$

Symbolic Math Toolbox commands

```
syms t
A = sym([0 1; -1 0]);
G = expm(t*A)
```

return

```
G =
[ exp(-t*1i)/2 + exp(t*1i)/2,
  (exp(-t*1i)*1i)/2 - (exp(t*1i)*1i)/2]
[ - (exp(-t*1i)*1i)/2 + (exp(t*1i)*1i)/2,
  exp(-t*1i)/2 + exp(t*1i)/2]
```

You can simplify this expression using `simplify`:

```
G = simplify(G)
```

```
G =  
[ cos(t), sin(t)]  
[ -sin(t), cos(t)]
```

Next, the command

```
g = eig(G)
```

produces

```
g =  
cos(t) - sin(t)*1i  
cos(t) + sin(t)*1i
```

You can rewrite g in terms of exponents:

```
g = rewrite(g, 'exp')
```

```
g =  
exp(-t*1i)  
exp(t*1i)
```



## Jordan Canonical Form

The Jordan canonical form (Jordan normal form) results from attempts to convert a matrix to its diagonal form by a similarity transformation. For a given matrix  $A$ , find a nonsingular matrix  $V$ , so that  $\text{inv}(V)*A*V$ , or, more succinctly,  $J = V \backslash A * V$ , is “as close to diagonal as possible.” For almost all matrices, the Jordan canonical form is the diagonal matrix of eigenvalues and the columns of the transformation matrix are the eigenvectors. This always happens if the matrix is symmetric or if it has distinct eigenvalues. Some nonsymmetric matrices with multiple eigenvalues cannot be converted to diagonal forms. The Jordan form has the eigenvalues on its diagonal, but some of the superdiagonal elements are one, instead of zero. The statement

```
J = jordan(A)
```

computes the Jordan canonical form of  $A$ . The statement

```
[V,J] = jordan(A)
```

also computes the similarity transformation where  $J = \text{inv}(V)*A*V$ . The columns of  $V$  are the generalized eigenvectors of  $A$ .

The Jordan form is extremely sensitive to changes. Almost any change in  $A$  causes its Jordan form to be diagonal. This implies that  $A$  must be known exactly (i.e., without round-off error, etc.) and makes it very difficult to compute the Jordan form reliably with floating-point arithmetic. Thus, computing the Jordan form with floating-point values is unreliable and not recommended.

For example, let

```
A = sym([12,32,66,116;-25,-76,-164,-294;
         21,66,143,256;-6,-19,-41,-73])
```

```
A =
[ 12, 32, 66, 116]
[ -25, -76, -164, -294]
[ 21, 66, 143, 256]
[ -6, -19, -41, -73]
```

Then

```
[V,J] = jordan(A)
```

produces

```
V =
[ 4, -2, 4, 3]
[ -6, 8, -11, -8]
[ 4, -7, 10, 7]
[ -1, 2, -3, -2]
```

```
J =
[ 1, 1, 0, 0]
[ 0, 1, 0, 0]
[ 0, 0, 2, 1]
[ 0, 0, 0, 2]
```

Show that  $J$  and  $\text{inv}(V)*A*V$  are equal by using `isequal`. The `isequal` function returns logical 1 (`true`) meaning that the inputs are equal.

```
isequal(J, inv(V)*A*V)
```

```
ans =  
    logical  
     1
```

From  $J$ , we see that  $A$  has a double eigenvalue at 1, with a single Jordan block, and a double eigenvalue at 2, also with a single Jordan block. The matrix has only two eigenvectors,  $V(:, 1)$  and  $V(:, 3)$ . They satisfy

$$\begin{aligned} A*V(:, 1) &= 1*V(:, 1) \\ A*V(:, 3) &= 2*V(:, 3) \end{aligned}$$

The other two columns of  $V$  are generalized eigenvectors of grade 2. They satisfy

$$\begin{aligned} A*V(:, 2) &= 1*V(:, 2) + V(:, 1) \\ A*V(:, 4) &= 2*V(:, 4) + V(:, 3) \end{aligned}$$

In mathematical notation, with  $v_j = v(:, j)$ , the columns of  $V$  and eigenvalues satisfy the relationships

$$(A - \lambda_1 I)v_2 = v_1$$

$$(A - \lambda_2 I)v_4 = v_3.$$

## Eigenvalues of the Laplace Operator

This example shows how to solve the eigenvalue problem of the Laplace operator on an L-shaped region.

### Membrane Problem

Consider a membrane that is fixed at the boundary  $\partial\Omega$  of a region  $\Omega$  in the plane. Its displacement  $u(x, y)$  is described by the eigenvalue problem  $\Delta u = \lambda u$ , where  $\Delta u = u_{xx} + u_{yy}$  is the Laplace operator and  $\lambda$  is a scalar parameter. The boundary condition is  $u(x, y) = 0$  for all  $(x, y) \in \partial\Omega$ .

The Laplace operator is self-adjoint and negative definite, that is, only real negative eigenvalues  $\lambda$  exist. There is a maximal (negative) discrete eigenvalue, the corresponding eigenfunction  $u$  is called the *ground state*. In this example,  $\Omega$  is an L-shaped region, and the ground state associated with this region is the L-shaped membrane that is the MATLAB® logo.

### Nine-Point Finite Difference Approximation

The simplest approach to the eigenvalue problem is to approximate the Laplacian  $\Delta u$  by a finite difference approximation (a *stencil*) on a square grid of points with distances  $h_x$  in  $x$  direction and distances  $h_y$  in  $y$  direction. In this example, approximate  $\Delta u$  with a sum  $S_h$  of nine regular grid points around the midpoint  $(x, y)$ . The unknowns are the weights  $a_{-1-1}, \dots, a_{11}$ .

```
syms u(x,y) Eps a11 a10 a1_1 a01 a00 a0_1 a_11 a_10 a_1_1
syms hx hy positive
S_h = a_11 * u(x - Eps*hx, y + Eps*hy) + ...
      a01 * u(x, y + Eps*hy) + ...
      a11 * u(x + Eps*hx, y + Eps*hy) + ...
      a_10 * u(x - Eps*hx, y) + ...
      a00 * u(x, y) + ...
      a10 * u(x + Eps*hx, y) + ...
      a_1_1 * u(x - Eps*hx, y - Eps*hy) + ...
      a0_1 * u(x, y - Eps*hy) + ...
      a1_1 * u(x + Eps*hx, y - Eps*hy);
```

Use the symbolic parameter `Eps` to sort the expansion of this expression by powers of  $h_x$  and  $h_y$ . Knowing the weights, you can approximate the Laplacian by setting `Eps = 1`.

```
t = Taylor(S_h, Eps, 'Order', 7);
```

Use the `coeffs` function to extract their coefficients of terms with the same powers of `Eps`. Each coefficient is an expression containing powers of  $h_x$ ,  $h_y$ , and derivatives of  $u$  with respect to  $x$  and  $y$ . Since  $S_h$  represents  $u_{xx} + u_{yy}$ , the coefficients of all other derivatives of  $u$  must be zero. Extract the coefficients by replacing all derivatives of  $u$ , except  $u_{xx}$  and  $u_{yy}$ , by 0. Replace  $u_{xx}$  and  $u_{yy}$  by 1. This reduces the Taylor expansion to the coefficient you want to compute, and leads to the following six linear equations.

```
C = formula(coeffs(t, Eps, 'All'));
eq0 = subs(C(7), u(x,y), 1) == 0;
eq11 = subs(C(6), [diff(u,x), diff(u,y)], [1,0]) == 0;
eq12 = subs(C(6), [diff(u,x), diff(u,y)], [0,1]) == 0;
eq21 = subs(C(5), [diff(u,x,x), diff(u,x,y), diff(u,y,y)], [1,0,0]) == 1;
eq22 = subs(C(5), [diff(u,x,x), diff(u,x,y), diff(u,y,y)], [0,1,0]) == 0;
eq23 = subs(C(5), [diff(u,x,x), diff(u,x,y), diff(u,y,y)], [0,0,1]) == 1;
```

Since there are nine unknown weights in  $S_h$ , add further equations by requiring that all third-order derivatives of  $u$  are 0.

```
eq31 = subs(C(4), [diff(u,x,x,x), diff(u,x,x,y), diff(u,x,y,y), diff(u,y,y,y)], [1,0,0,0]) == 0;
eq32 = subs(C(4), [diff(u,x,x,x), diff(u,x,x,y), diff(u,x,y,y), diff(u,y,y,y)], [0,1,0,0]) == 0;
eq33 = subs(C(4), [diff(u,x,x,x), diff(u,x,x,y), diff(u,x,y,y), diff(u,y,y,y)], [0,0,1,0]) == 0;
eq34 = subs(C(4), [diff(u,x,x,x), diff(u,x,x,y), diff(u,x,y,y), diff(u,y,y,y)], [0,0,0,1]) == 0;
```

Solve the resulting ten equations for the nine unknown weights. Use `ReturnConditions` to find all solutions including arbitrary parameters.

```
[a11,a10,a1_1,a01,a00,a0_1,a_11,a_10,a_1_1,parameters,conditions] = ...
    solve([eq0,eq11,eq12,eq21,eq22,eq23,eq31,eq32,eq33,eq34], ...
        [a11,a10,a1_1,a01,a00,a0_1,a_11,a_10,a_1_1], ...
        'ReturnConditions',true);

expand([a_11,a01,a11; ...
        a_10,a00,a01; ...
        a1_1,a0_1,a_1_1])
```

```
ans =
```

$$\begin{pmatrix} z & \frac{1}{hy^2} - 2z & z \\ \frac{1}{hx^2} - 2z & 4z - \frac{2}{hx^2} - \frac{2}{hy^2} & \frac{1}{hy^2} - 2z \\ z & \frac{1}{hy^2} - 2z & z \end{pmatrix}$$

parameters

```
parameters = z
```

Use the `subs` function to replace the weights by their computed values.

```
C = simplify(subs(C));
```

The expressions  $C(7)$ ,  $C(6)$ , and  $C(4)$  containing the 0th, 1st, and 3rd derivatives of  $u$  vanish.

```
[C(7), C(6), C(4)]
```

```
ans = (0 0 0)
```

The expression  $C(5)$  is the Laplacian of  $u$ .

```
C(5)
```

```
ans =
```

$$\frac{\partial^2}{\partial x^2} u(x,y) + \frac{\partial^2}{\partial y^2} u(x,y)$$

Thus, with the values of the weights computed above, the stencil  $S_h$  approximates the Laplacian up to order  $hx^2$ ,  $hy^2$  for any values of the arbitrary parameter  $z$ , provided that  $z$  is chosen to be of order  $O(1/hx^2, 1/hy^2)$ .

### Terms Containing Fourth and Higher Order Derivatives

Although the solution contains a free parameter  $z$ , the expression  $C(3)$  containing the fourth-order derivatives of  $u$  cannot be turned into zero by a suitable choice of  $z$ . Another option is to turn it into a multiple of the square of the Laplace operator.

```
syms d
Laplace = @(u) laplacian(u,[x,y]);
expand(d*Laplace(Laplace(u)))

ans(x, y) =
    d  $\frac{\partial^4}{\partial x^4} u(x,y)$  + 2 d  $\frac{\partial^2}{\partial y^2} \frac{\partial^2}{\partial x^2} u(x,y)$  + d  $\frac{\partial^4}{\partial y^4} u(x,y)$ 
```

Pick different derivatives of  $u$  in  $C(3)$ , and equate their coefficients with the corresponding terms.

```
subs(C(3),[diff(u,x,x,x,x),diff(u,x,x,y,y),diff(u,y,y,y,y)],[1,0,0]) == d
ans =
     $\frac{hx^2}{12} = d$ 

subs(C(3),[diff(u,x,x,x,x),diff(u,x,x,y,y),diff(u,y,y,y,y)],[0,1,0]) == 2*d
ans = hx^2 hy^2 z = 2 d

subs(C(3),[diff(u,x,x,x,x),diff(u,x,x,y,y),diff(u,y,y,y,y)],[0,0,1]) == d
ans =
     $\frac{hy^2}{12} = d$ 
```

Therefore, you can choose  $d = hx^2/12 = hy^2/12$  and  $z = 2*d/(hx^2*hy^2)$ , implying that  $hx = hy$  and  $z = 1/(6*hx*hy)$ . Hence, the stencil  $S_h$  approximates a modified Laplacian on a square grid with  $hx = hy = h$ .

$$S_h = \Delta u + \frac{h^2}{12} \Delta^2 u + O(h^3). \quad (1)$$

```
syms h
hx = h;
hy = h;
d = h^2/12;
```

Replace  $hx$  and  $hy$  by  $h$ .

```
C = subs(C);
```

Replace  $z$  by its value,  $1/(6*h^2)$ . Because  $z$  does not exist in the MATLAB workspace, you can access it only as the value stored in the `parameters` array.

```
C = subs(C,parameters,1/(6*h^2));
```

Verify the formula (1).

```
simplify(C(3) - d*Laplace(Laplace(u)))

ans(x, y) = 0
```

Now, consider the third-order terms in  $h_x$ ,  $h_y$ .

```
simplify(C(2))
```

```
ans = 0
```

Since no such terms exist in the expansion of the stencil, the term  $O(h^3)$  in (1) is in fact of order  $O(h^4)$ . Consider the fourth-order terms of the stencil.

```
factor(simplify(C(1)))
```

```
ans =
```

$$\left( \frac{1}{360} h h h h \frac{\partial^6}{\partial x^6} u(x, y) + 5 \frac{\partial^2}{\partial y^2} \frac{\partial^4}{\partial x^4} u(x, y) + 5 \frac{\partial^4}{\partial y^4} \frac{\partial^2}{\partial x^2} u(x, y) + \frac{\partial^6}{\partial y^6} u(x, y) \right)$$

Check if these terms can be identified with yet another power of the Laplace operator. However, comparison with

```
Laplace(Laplace(Laplace(u)))
```

```
ans(x, y) =
```

$$\frac{\partial^6}{\partial x^6} u(x, y) + 3 \frac{\partial^2}{\partial y^2} \frac{\partial^4}{\partial x^4} u(x, y) + 3 \frac{\partial^4}{\partial y^4} \frac{\partial^2}{\partial x^2} u(x, y) + \frac{\partial^6}{\partial y^6} u(x, y)$$

shows that the expressions of order  $O(h^4)$  cannot be identified as some multiple of the third power of the Laplace operator because the coefficients cannot be matched.

### Summary

For a square grid with distance  $h$  between neighboring grid points and the above choices of the weights, you get:

$$S_h = \Delta u + \frac{h^2}{12} \Delta^2 u + O(h^4). \quad (2)$$

Use this expansion for the numeric approach to the eigenvalue problem  $\Delta u = \lambda u$ . Add some multiple of  $\Delta^2 u = \lambda^2 u$  to the eigenvalue equation.

$$\Delta u + \frac{h^2}{12} \Delta^2 u = (\lambda + \frac{h^2}{12} \lambda^2) u.$$

The left side of this equation is well approximated by the stencil  $S_h$ . Thus, using (2), a numerical eigenvalue  $\mu$  of the stencil satisfying  $S_h u = \mu u$  must be an approximation of the eigenvalue  $\lambda$  of the Laplace operator with

$$\mu = \lambda + \frac{h^2}{12} \lambda^2 + O(h^4).$$

For given  $\mu$ , solve for  $\lambda$  to obtain a better approximation of the Laplace eigenvalue. Note that in the solution of the quadratic equation in  $\lambda$  the correct sign of the square root is given by the requirement that  $\lambda \rightarrow \mu$  for  $h \rightarrow 0$ .

$$\lambda = \frac{6}{h^2} \left( \sqrt{1 + \frac{\mu h^2}{3}} - 1 \right) = \frac{2\mu}{\sqrt{1 + \frac{\mu h^2}{3}} + 1}. \quad (3)$$

### Using Symbolic Matrices to Solve the Eigenvalue Problem

Consider an L-shaped region  $\Omega$  consisting of three unit squares.

$$\Omega = \{(x, y); -1 \leq x \leq 0, -1 \leq y \leq 0\} \cup \{(x, y); 0 \leq x \leq 1, -1 \leq y \leq 0\} \cup \{(x, y); -1 \leq x \leq 0, 0 \leq y \leq 1\}.$$

Define the coordinate values of the corners of the region.

```
xmin = -1;
xmax = 1;
ymin = -1;
ymax = 1;
```

Consider a square grid consisting of an odd number  $N_x = 2 \cdot n_x - 1$  of grid points in  $x$  direction and an odd number  $N_y = 2 \cdot n_y - 1$  of grid points in  $y$  direction.

```
nx = 6;
Nx = 2*nx-1;
hx = (xmax-xmin)/(Nx-1);
```

```
ny = 6;
Ny = 2*ny-1;
hy = (ymax-ymin)/(Ny-1);
```

Create an  $N_y$ -by- $N_x$  symbolic matrix  $u$ . Its entries  $u(i, j)$  represent the values  $u(x_{\min} + (j - 1) \cdot h_x, y_{\min} + (i - 1) \cdot h_y)$  of the solution  $u(x, y)$  of the eigenvalue problem  $\Delta u = \lambda u$ .

```
u = sym('u', [Ny, Nx]);
```

The boundaries of  $\Omega$  correspond to the following indices:

- The left boundary corresponds to  $(i = 1:N_y, j = 1)$ .
- The lower boundary corresponds to  $(i = 1, j = 1:N_x)$ .
- The right boundary corresponds to  $(i = 1:n_y, j = Nx)$  and  $(i = n_y:N_y, j = nx)$ .
- The upper boundary corresponds to  $(i = Ny, j = 1:nx)$  and  $(i = ny, j = nx:Nx)$ .

```
u(:,1) = 0;      % left boundary
u(1,:) = 0;      % lower boundary
u(1:ny,Nx) = 0;  % right boundary, upper part
u(ny:Ny,nx) = 0; % right boundary, lower part
u(Ny,1:nx) = 0;  % upper boundary, left part
u(ny,nx:Nx) = 0; % upper boundary, right part
```

The region with  $0 < x \leq 1$  and  $0 < y \leq 1$  does not belong to  $\Omega$ . Set the corresponding matrix entries  $(i = ny + 1:N_y, j = nx + 1:N_x)$  to zero. They play no further role and will be ignored.

```
u(ny + 1:Ny, nx + 1:Nx) = 0;
```

The unknowns of the problem are the following matrix entries:

```
u
u =
```

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & u_{2,2} & u_{2,3} & u_{2,4} & u_{2,5} & u_{2,6} & u_{2,7} & u_{2,8} & u_{2,9} & u_{2,10} & 0 \\ 0 & u_{3,2} & u_{3,3} & u_{3,4} & u_{3,5} & u_{3,6} & u_{3,7} & u_{3,8} & u_{3,9} & u_{3,10} & 0 \\ 0 & u_{4,2} & u_{4,3} & u_{4,4} & u_{4,5} & u_{4,6} & u_{4,7} & u_{4,8} & u_{4,9} & u_{4,10} & 0 \\ 0 & u_{5,2} & u_{5,3} & u_{5,4} & u_{5,5} & u_{5,6} & u_{5,7} & u_{5,8} & u_{5,9} & u_{5,10} & 0 \\ 0 & u_{6,2} & u_{6,3} & u_{6,4} & u_{6,5} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & u_{7,2} & u_{7,3} & u_{7,4} & u_{7,5} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & u_{8,2} & u_{8,3} & u_{8,4} & u_{8,5} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & u_{9,2} & u_{9,3} & u_{9,4} & u_{9,5} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & u_{10,2} & u_{10,3} & u_{10,4} & u_{10,5} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

The interior points of the region  $\Omega \setminus \partial\Omega$  correspond to the indices  $(i, j)$  that contain the unknown values  $u(i, j)$  of the problem. Collect these unknowns in a vector `vars`.

```
[I,J] = find(u~=0);
vars = u(u~=0);
```

Associate a symbolic expression (given by the stencil derived in the first part of this example) with each index (that is, with each unknown).

```
n = length(vars);
Lu = sym(zeros(n,1));
for k=1:n
    i = I(k);
    j = J(k);
    Lu(k) = 1/6*u(i+1,j-1) + 2/3*u(i+1,j) + 1/6*u(i+1,j+1) ...
            + 2/3*u(i,j-1) - 10/3*u(i,j) + 2/3*u(i,j+1) ...
            + 1/6*u(i-1,j-1) + 2/3*u(i-1,j) + 1/6*u(i-1,j+1);
end
Lu = Lu/hx^2;
```

Because this expression is linear in the unknown elements of  $u$  (stored in `vars`), you can treat it as a matrix acting on the vector `vars`.

```
S_h = jacobian(Lu, vars);
```

You can treat  $S_h$  as a matrix approximation of the Laplace operator. Compute its eigenvectors and eigenvalues.

```
[V,D] = eig(vpa(S_h));
```

The three maximal eigenvalues are given by the first diagonal elements of  $D$ .

```
[D(1,1), D(2,2), D(3,3)]
```

```
ans = (-9.5214641572625960021345709535953 -14.431096242107969492574666743957 -18.490392088545609858994
```

Since for this approximation you used a grid with a small number of points, only the leading digits of the eigenvalues are correct.

The third highest eigenvalue of the Laplace operator on the L-shaped region  $\Omega$  is known exactly. The exact eigenfunction of the Laplace operator is the function  $u(x, y) = \sin(\pi x) \sin(\pi y)$  associated with

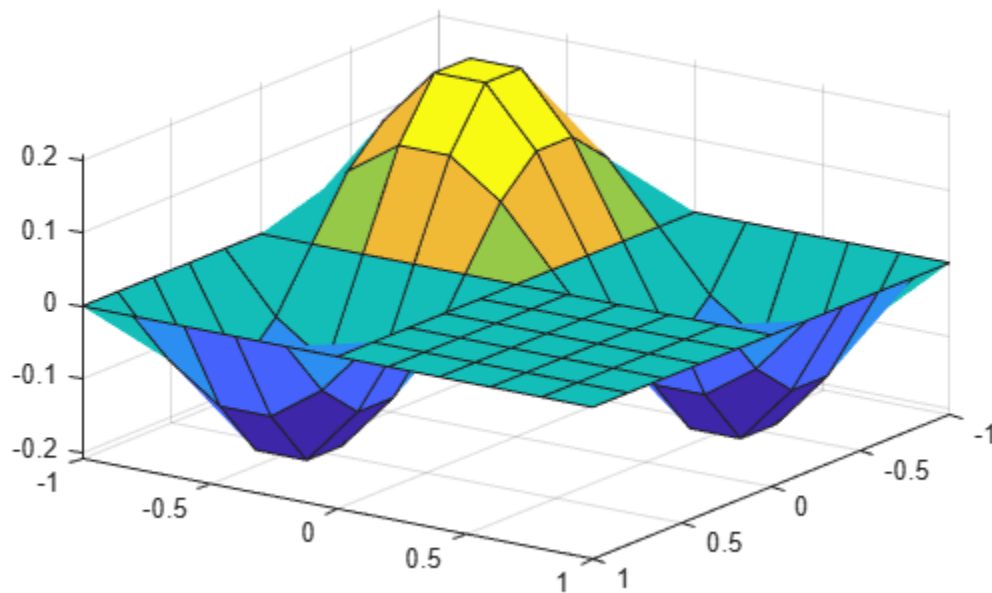


the (exact) eigenvalue  $-2\pi^2 = -19.7392\dots$ . Indeed, using equation (3) above, you can derive a better approximation of the Laplace eigenvalue  $\lambda$  from the stencil eigenvalue  $\mu$ :

```
mu = D(3,3)
mu = -18.490392088545609858994660377955
lambda = 2*mu / (sqrt(1 + mu*hx^2/3) + 1)
lambda = -19.796765119155672176257649532142
```

Plot the eigenfunction associated with the third highest eigenvalue.

```
v = V(:,3);
for k=1:n
    u(I(k),J(k)) = v(k);
end
u = double(u);
surf(xmin:hx:xmax,ymin:hy:ymax,u');
view(125, 30);
```



### Using Double-Precision Matrices to Solve the Eigenvalue Problem

When you use symbolic matrices, increasing the number of grid points drastically is not recommended because symbolic computations are significantly slower than numerical computations with MATLAB double-precision matrices. This part of the example demonstrates how to use sparse double arithmetic which allows to refine the numerical grid. The L-shaped region  $\Omega$  is set up the same way as before. Instead of denoting the interior points by symbolic unknowns, initialize the grid values  $u$  by ones and define  $\Omega$  by setting the values of boundary points and exterior points to zero. Instead of defining a symbolic expression for each interior point and computing the stencil as the Jacobian, set up the stencil matrix directly as a sparse matrix.

```
xmin = -1;
xmax = 1;
```

```

ymin = -1;
ymax = 1;

nx = 30;
Nx = 2*nx-1;
hx = (xmax-xmin)/(Nx-1);

ny = 30;
Ny = 2*ny-1;
hy = (ymax-ymin)/(Ny-1);

u = ones(Ny,Nx);
u(:,1) = 0; % left boundary
u(1:ny,Nx) = 0; % right boundary, upper part
u(ny:Ny,nx) = 0; % right boundary, lower part
u(1,:) = 0; % lower boundary
u(Ny,1:nx) = 0; % upper boundary, left part
u(ny,nx:Nx) = 0; % upper boundary, right part
u(ny + 1:Ny,nx + 1:Nx) = 0;

[I,J] = find(u ~= 0);
n = length(I);
S_h = sparse(n,n);
for k=1:n
    i = I(k);
    j = J(k);
    S_h(k,I==i+1 & J==j+1)= 1/6;
    S_h(k,I==i+1 & J== j )= 2/3;
    S_h(k,I==i+1 & J==j-1)= 1/6;
    S_h(k,I== i & J==j+1)= 2/3;
    S_h(k,I== i & J== j )=-10/3;
    S_h(k,I== i & J==j-1)= 2/3;
    S_h(k,I==i-1 & J==j+1)= 1/6;
    S_h(k,I==i-1 & J== j )= 2/3;
    S_h(k,I==i-1 & J==j-1)= 1/6;
end
S_h = S_h./hx^2;

```

Here,  $S\_h$  is the (sparse) stencil matrix. Use `eigs` that handles sparse matrices to compute the three largest eigenvalues.

```
[V,D] = eigs(S_h,3,'la');
```

The three maximal eigenvalues are the first diagonal elements of  $D$ .

```
[D(1,1), D(2,2), D(3,3)]
```

```
ans = 1×3
```

```
-9.6493 -15.1742 -19.7006
```

$D(3,3)$  approximates the exact eigenvalue  $-2\pi^2 = -19.7392088\dots$  Using the equation (3) above, derive a more accurate approximation of the Laplace eigenvalue  $\lambda$  from the stencil eigenvalue  $\mu$ .

```
mu = D(3,3)
```

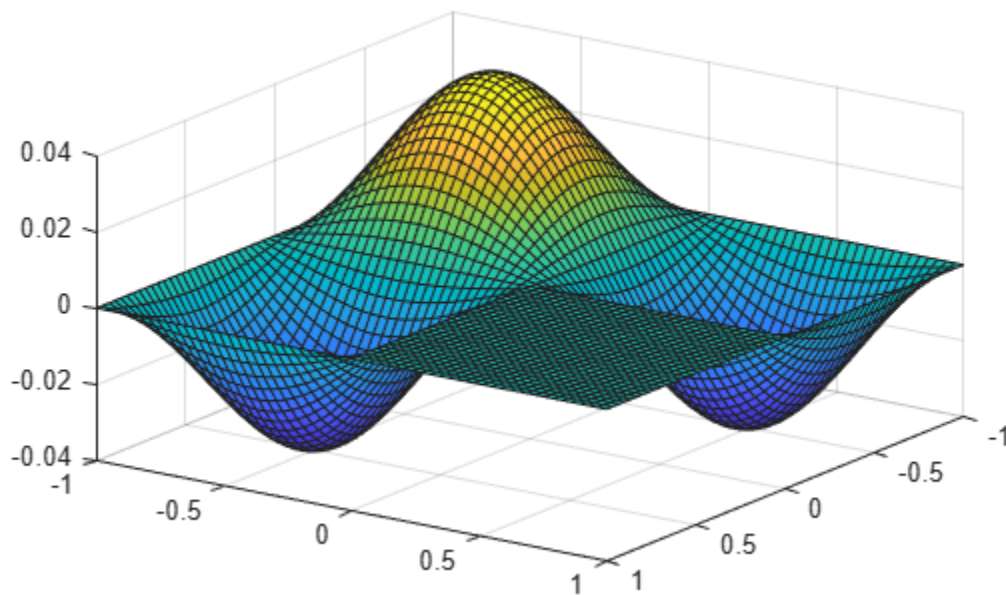
```
mu =
-19.7006
```

```
lambda = 2*mu / (sqrt(1 + mu*hx^2/3) + 1)
```

```
lambda =  
-19.7393
```

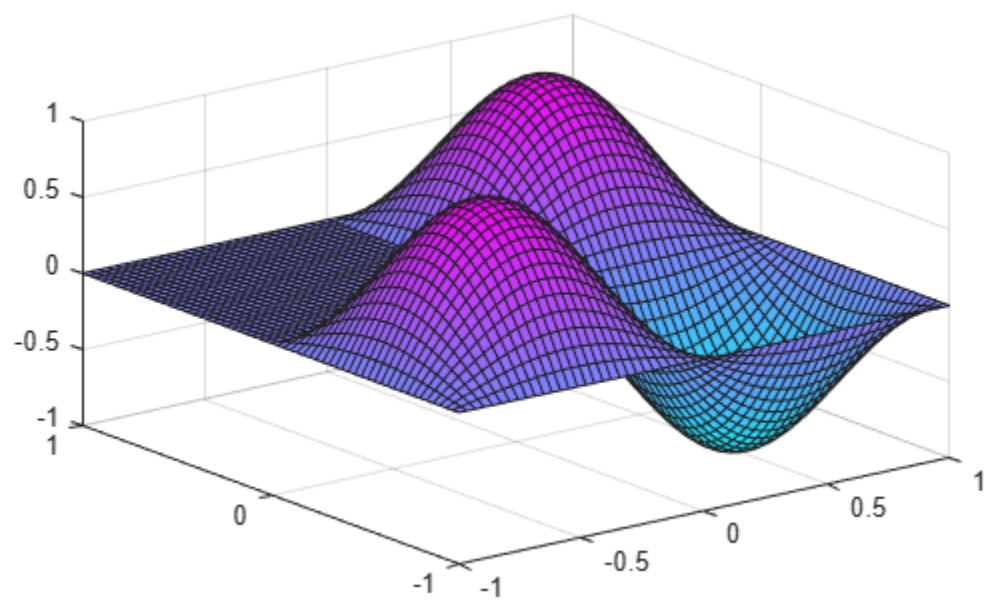
Plot the eigenfunction associated with the third highest eigenvalue.

```
v = V(:,3);  
for k=1:n  
    u(I(k),J(k)) = v(k);  
end  
surf(xmin:hx:xmax, ymin:hy:ymax,u');  
view(125, 30);
```



Note that the MATLAB `membrane` function computes the eigenfunctions of the Laplace operator by different methods.

```
membrane(3, nx - 1, 8, 8);
```



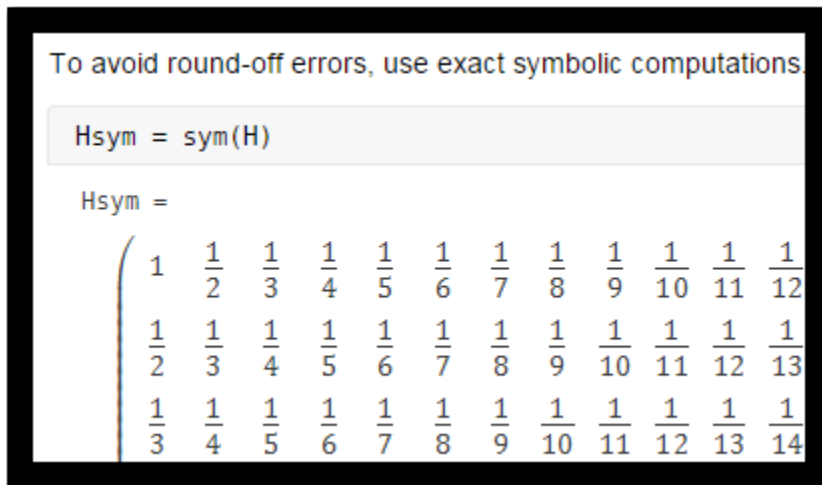
## See Also

### Functions

`diff` | `laplacian` | `solve` | `subs` | `taylor`

## Hilbert Matrices and Their Inverses

This example shows how to compute the inverse of a Hilbert matrix using Symbolic Math Toolbox™.



**Definition** : A Hilbert matrix is a square matrix with entries being the unit fraction.  $H_{ij} = \frac{1}{i+j-1}$ .

For example, the 3x3 Hilbert matrix is  $H = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \end{bmatrix}$

Symbolic computations give accurate results for these ill-conditioned matrices, while purely numerical methods fail.

Create a 20-by-20 numeric Hilbert matrix.

```
H = hilb(20);
```

Find the condition number of this matrix. Hilbert matrices are ill-conditioned, meaning that they have large condition numbers indicating that such matrices are nearly singular. Note that computing condition numbers is also prone to numeric errors.

```
cond(H)
```

```
ans =  
2.3622e+18
```

Therefore, inverting Hilbert matrices is numerically unstable. When you compute a matrix inverse,  $H \cdot \text{inv}(H)$  must return an identity matrix or a matrix close to the identity matrix within some error margin.

First, compute the inverse of H by using the `inv` function. A warning is thrown due to the numerical instability.

```
H*inv(H)
```

```
Warning: Matrix is close to singular or badly scaled. Results may be inaccurate. RCOND = 9.54235
```

```
ans = 20×20
```

```

1.0000 -0.0000 -0.0001 -0.0085 -0.0913 1.1314 1.6883 -1.0683 0.1591 -2.8
0.1861 1.0000 -0.0005 -0.0069 -0.0871 0.4017 1.7433 -1.8992 0.9020 -1.7
0.3826 -0.4950 0.9996 -0.0018 -0.0429 0.1705 0.6992 -1.8606 1.0837 -1.0
0.5041 -0.8696 0.0435 0.9959 -0.0201 -0.0490 1.0649 -1.0192 -0.0800 -0.0
0.5720 -1.0546 -0.1256 0.1861 0.9511 -0.0938 1.1348 -0.9431 1.3674 -1.0
0.6067 -1.1059 -0.4613 0.5231 -0.0094 0.7675 1.6065 -1.5955 1.5977 -0.3
0.6212 -1.0766 -0.8877 0.9672 -0.0736 -0.0672 1.9718 -1.7737 1.3339 -0.0
0.6234 -1.0026 -1.3471 1.4755 -0.2453 0.0381 0.4878 0.4395 -0.5762 0.3
0.6181 -0.9059 -1.8055 2.0007 -0.4130 0.1919 0.4012 -0.3619 0.7078 -0.0
0.6081 -0.8000 -2.2424 2.5177 -0.5738 -0.0377 1.3468 -0.6779 0.7183 0.7
0.5955 -0.6925 -2.6481 3.0289 -0.8129 0.4312 0.6145 0.2614 -0.0265 -1.0
0.5813 -0.5880 -3.0175 3.5022 -1.0473 0.3054 0.4393 -0.1110 -0.3013 -0.7
0.5664 -0.4889 -3.3501 3.9463 -1.2773 0.3750 0.3750 -0.3750 0.5000 -0.7
0.5511 -0.3963 -3.6469 4.3588 -1.4765 0.3012 1.1328 -1.6571 1.1391 0.0
0.5360 -0.3106 -3.9105 4.7517 -1.8278 0.8244 -0.0975 0.2895 -0.1999 -0.3
⋮

```

Now, use the MATLAB® `invhilb` function that offers better accuracy for Hilbert matrices. This function finds exact inverses of Hilbert matrices up to 15-by-15. For a 20-by-20 Hilbert matrix, `invhilb` finds the approximation of the matrix inverse.

```
H*invhilb(20)
```

```
ans = 20×20
```

```
1010 ×
```

```

0.0000 -0.0000 0.0000 -0.0000 0.0000 -0.0004 0.0013 -0.0037 0.0047 -0.2
-0.0000 0.0000 -0.0000 0.0000 -0.0000 -0.0001 -0.0009 0.0172 -0.0628 0.7
0.0000 0.0000 0.0000 -0.0000 -0.0000 -0.0001 0.0009 0.0042 -0.0303 0.0
0.0000 0.0000 -0.0000 -0.0000 0.0000 -0.0001 0.0004 0.0002 -0.0056 0.0
-0.0000 0.0000 -0.0000 0.0000 -0.0000 -0.0000 -0.0006 0.0068 -0.0394 0.7
0.0000 -0.0000 0.0000 -0.0000 0.0000 -0.0002 0.0014 -0.0028 0.0304 -0.7
-0.0000 0.0000 -0.0000 0.0000 -0.0000 0.0000 -0.0013 0.0101 -0.0520 0.7
0.0000 -0.0000 0.0000 -0.0000 -0.0000 -0.0002 0.0018 -0.0040 0.0231 -0.0
-0.0000 0.0000 -0.0000 0.0000 -0.0000 0.0000 0.0004 0.0144 -0.0513 0.7
0.0000 0.0000 0.0000 -0.0000 0.0000 -0.0002 0.0008 -0.0019 0.0027 -0.0
-0.0000 0.0000 -0.0000 0.0000 -0.0000 -0.0001 0.0004 0.0050 -0.0400 0.0
0.0000 0.0000 0.0000 -0.0000 0.0000 -0.0001 0.0009 -0.0015 -0.0172 0.0
-0.0000 0.0000 -0.0000 0.0000 -0.0000 -0.0000 0.0007 0.0097 -0.0384 0.0
0.0000 -0.0000 -0.0000 -0.0000 -0.0000 -0.0001 0.0007 0.0020 -0.0014 -0.0
-0.0000 0.0000 -0.0000 -0.0000 -0.0000 -0.0001 0.0000 0.0043 -0.0304 0.0
⋮

```

To avoid round-off errors, use exact symbolic computations. For this, create the symbolic Hilbert matrix.

```
Hsym = sym(H)
```

```
Hsym =
```

[illegible]

Get the value of the condition number. It has been derived by symbolic methods and is free of numerical errors.

$$\text{vpa}(\text{cond}(\text{Hsym}))$$

```
ans = 24521565858153031724608315432.509
```

Although its condition number is large, you can compute the exact inverse of the matrix.

$$Hsym * inv(Hsym)$$

ans =

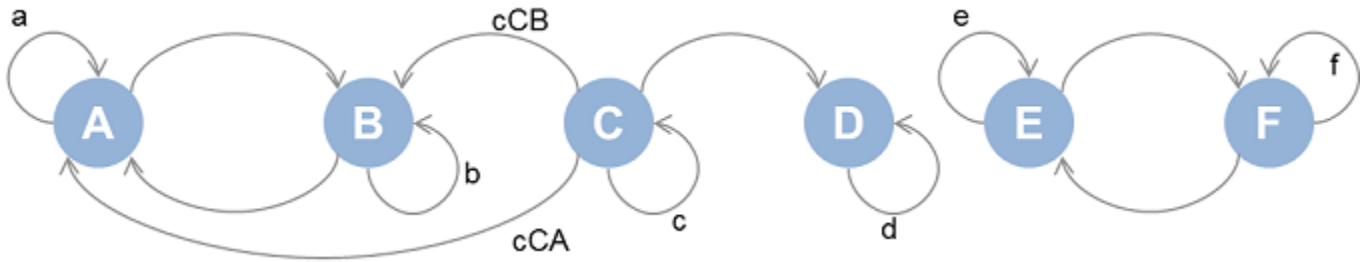




## Markov Chain Analysis and Stationary Distribution

This example shows how to derive the symbolic stationary distribution of a trivial Markov chain by computing its eigen decomposition.

The stationary distribution represents the limiting, time-independent, distribution of the states for a Markov process as the number of steps or transitions increase.



Define (positive) transition probabilities between states A through F as shown in the above image.

```
syms a b c d e f cCA cCB positive;
```

Add further assumptions bounding the transition probabilities. This will be helpful in selecting desirable stationary distributions later.

```
assumeAlso([a, b, c, e, f, cCA, cCB] < 1 & d == 1);
```

Define the transition matrix. States A through F are mapped to the columns and rows 1 through 6. Note the values in each row sum up to one.

```
P = sym(zeros(6,6));
P(1,1:2) = [a 1-a];
P(2,1:2) = [1-b b];
P(3,1:4) = [cCA cCB c (1-cCA-cCB-c)];
P(4,4) = d;
P(5,5:6) = [e 1-e];
P(6,5:6) = [1-f f];
P
```

```
P =
```

$$\begin{pmatrix} a & 1-a & 0 & 0 & 0 & 0 \\ 1-b & b & 0 & 0 & 0 & 0 \\ cCA & cCB & c & 1-cCA-cCB-c & 0 & 0 \\ 0 & 0 & 0 & d & 0 & 0 \\ 0 & 0 & 0 & 0 & e & 1-e \\ 0 & 0 & 0 & 0 & 1-f & f \end{pmatrix}$$

Compute all possible analytical stationary distributions of the states of the Markov chain. This is the problem of extracting `eig` with corresponding eigenvalues that can be equal to 1 for some value of the transition probabilities.

```
[V,D] = eig(P');
```

Analytical eigenvectors

V

$$V = \begin{pmatrix} \frac{b-1}{a-1} & 0 & 0 & -\frac{(c-d)(cCB-bcCA-bcCB+c cCA)}{\sigma_1} & \frac{b-1}{a-d} & 0 & 0 & -1 & 0 \\ 1 & 0 & 0 & -\frac{(c-d)(cCA-a cCA-a cCB+c cCB)}{\sigma_1} & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -\frac{c-d}{c+cCA+cCB-1} & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & \frac{f-1}{e-1} & 0 & 0 & 0 & -\frac{f-1}{d-e} & 0 & -1 \\ 0 & 1-d & 1 & 0 & 0 & 0 & 1 & 0 & 1 \end{pmatrix}$$

where

$$\sigma_1 = (c + cCA + cCB - 1)(a + b - ac - bc + c^2 - 1)$$

Analytical eigenvalues

diag(D)

$$\text{ans} = \begin{pmatrix} 1 \\ 1 \\ c \\ d \\ a+b-1 \\ e+f-1 \end{pmatrix}$$

Find eigenvalues that are exactly equal to 1. If there is any ambiguity in determining this condition for any eigenvalue, stop with an error - this way we are sure that below list of indices is reliable when this step is successful.

```
ix = find(isAlways(diag(D) == 1, 'Unknown', 'error'));
diag(D(ix,ix))
```

$$\text{ans} = \begin{pmatrix} 1 \\ 1 \\ d \end{pmatrix}$$

Extract the analytical stationary distributions. The eigenvectors are normalized with the 1-norm or `sum(abs(X))` prior to display.

```
for k = ix'
    V(:,k) = simplify(V(:,k)/norm(V(:,k)),1);
end
Probability = V(:,ix)

Probability =
```

$$\begin{pmatrix} \frac{b-1}{(a-1)\sigma_1} & 0 & \frac{\sigma_5}{\sigma_2} \\ \frac{1}{\sigma_1} & 0 & \frac{\sigma_6}{\sigma_2} \\ 0 & 0 & -\frac{c-1}{\sigma_3(c+cCA+cCB-1)} \\ 0 & 1 & \frac{1}{\sigma_3} \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

where

$$\sigma_1 = \sqrt{\frac{(b-1)^2}{(a-1)^2} + 1}$$

$$\sigma_2 = \sigma_3(c+cCA+cCB-1)(a+b-c-1)$$

$$\sigma_3 = \sqrt{\frac{(c-1)^2}{(c+cCA+cCB-1)^2} + \frac{\sigma_6^2}{\sigma_4} + \frac{\sigma_5^2}{\sigma_4} + 1}$$

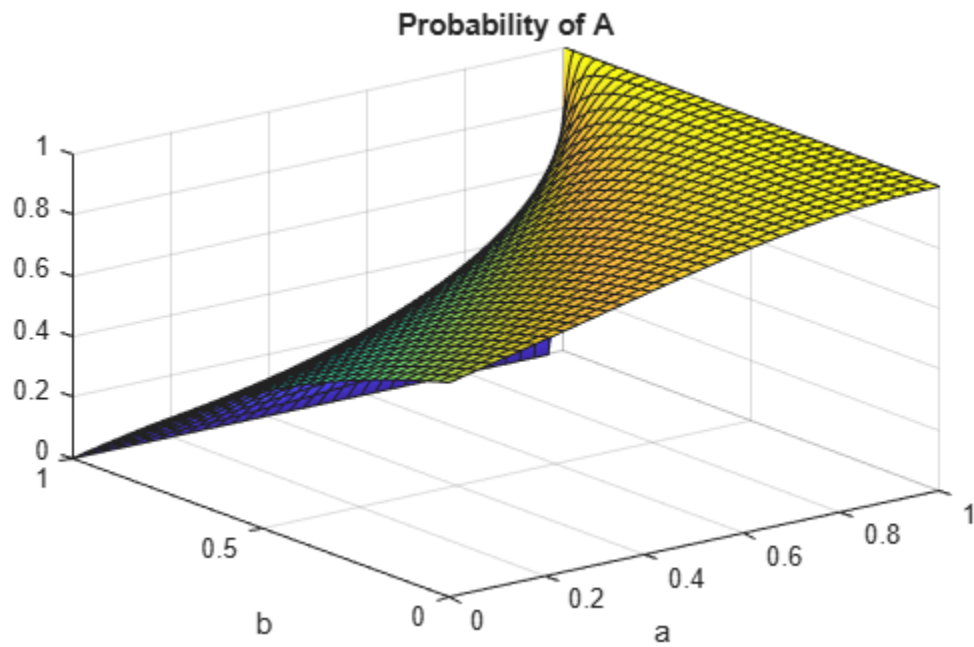
$$\sigma_4 = (c+cCA+cCB-1)^2(a+b-c-1)^2$$

$$\sigma_5 = cCB - b cCA - b cCB + c cCA$$

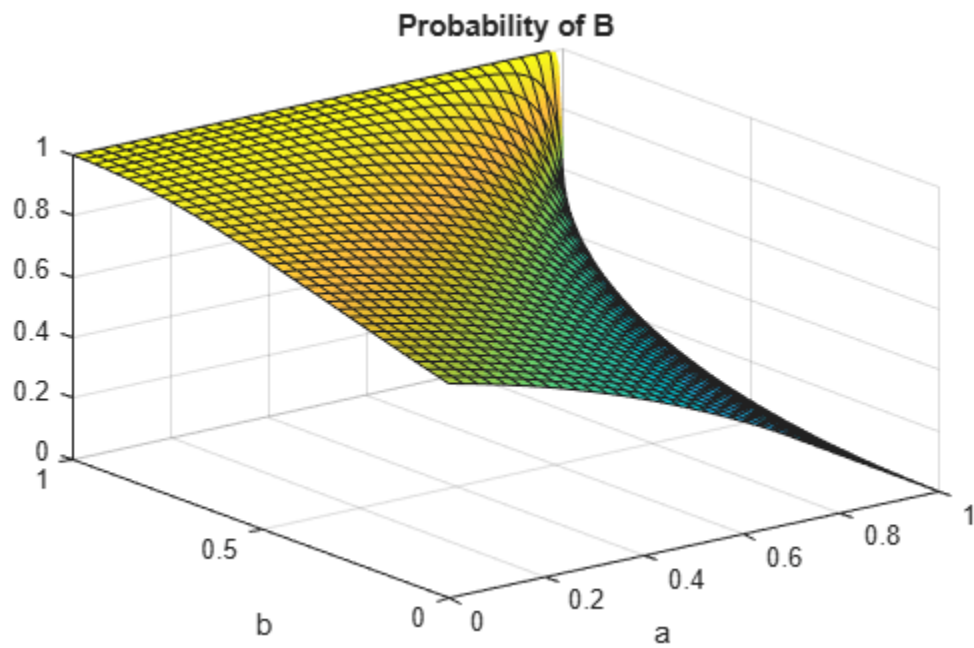
$$\sigma_6 = cCA - a cCA - a cCB + c cCB$$

The probability of the steady state being A or B in the first eigenvector case is a function of the transition probabilities a and b. Visualize this dependency.

```
fsurf(Probability(1), [0 1 0 1]);
xlabel a
ylabel b
title('Probability of A');
```



```
figure(2);
fsurf(Probability(2), [0 1 0 1]);
xlabel a
ylabel b
title('Probability of B');
```



The stationary distributions confirm the following (Recall states A through F correspond to row indices 1 through 6 ):

- State C is never reached and is therefore transient i.e. the third row is entirely zero.
- The rest of the states form three groups,  $\{A, B\}$ ,  $\{D\}$  and  $\{E, F\}$  that do not communicate with each other and are recurrent.

## Matrix Rotations and Transformations

This example shows how to do rotations and transforms in 3-D using Symbolic Math Toolbox™ and matrices.

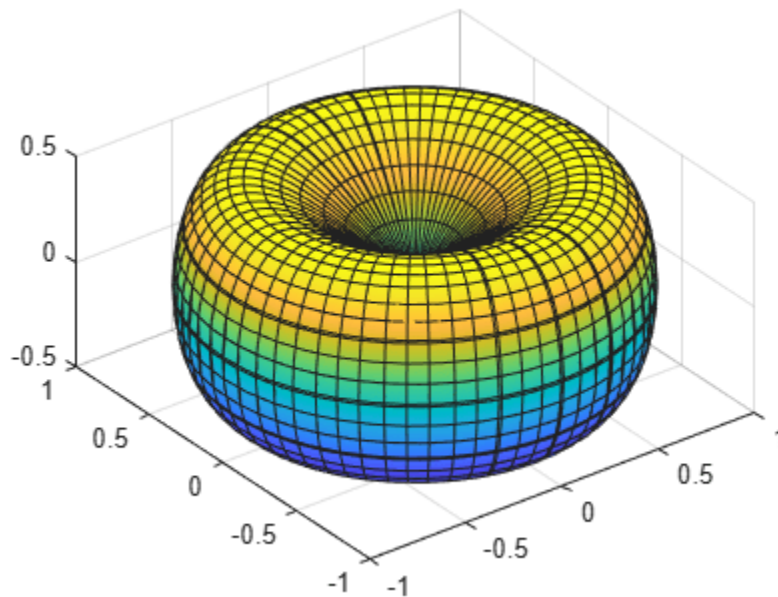
### Define and Plot Parametric Surface

Define the parametric surface  $x(u, v)$ ,  $y(u, v)$ ,  $z(u, v)$  as follows.

```
syms u v
x = cos(u)*sin(v);
y = sin(u)*sin(v);
z = cos(v)*sin(v);
```

Plot the surface using `fsurf`.

```
fsurf(x,y,z)
axis equal
```



### Create Rotation Matrices

Create 3-by-3 matrices  $R_x$ ,  $R_y$ , and  $R_z$  representing plane rotations by an angle  $t$  about the  $x$ -,  $y$ -, and  $z$ -axis, respectively.

```
syms t
Rx = [1 0 0; 0 cos(t) -sin(t); 0 sin(t) cos(t)]
Rx =
```

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(t) & -\sin(t) \\ 0 & \sin(t) & \cos(t) \end{pmatrix}$$

$$R_y = [\cos(t) \ 0 \ \sin(t); \ 0 \ 1 \ 0; \ -\sin(t) \ 0 \ \cos(t)]$$

$$R_y = \begin{pmatrix} \cos(t) & 0 & \sin(t) \\ 0 & 1 & 0 \\ -\sin(t) & 0 & \cos(t) \end{pmatrix}$$

$$R_z = [\cos(t) \ -\sin(t) \ 0; \ \sin(t) \ \cos(t) \ 0; \ 0 \ 0 \ 1]$$

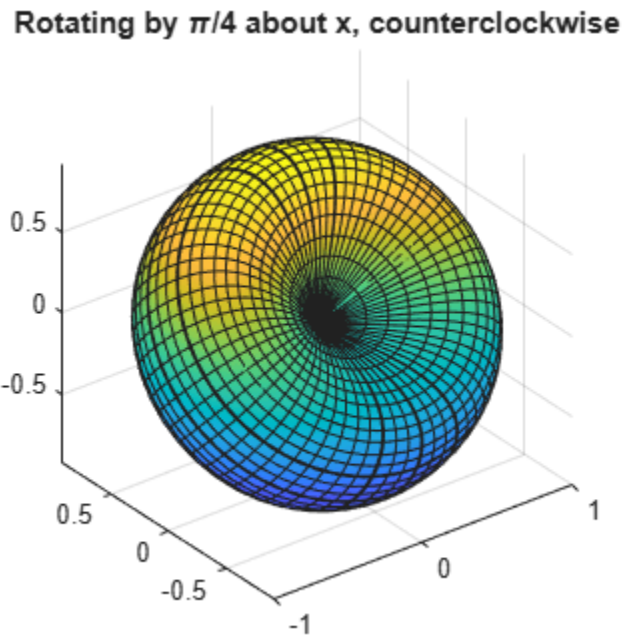
$$R_z = \begin{pmatrix} \cos(t) & -\sin(t) & 0 \\ \sin(t) & \cos(t) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

### Rotate About Each Axis in Three Dimensions

First, rotate the surface about the x-axis by 45 degrees counterclockwise.

```
xyzRx = Rx*[x;y;z];
Rx45 = subs(xyzRx, t, pi/4);

fsurf(Rx45(1), Rx45(2), Rx45(3))
title('Rotating by \pi/4 about x, counterclockwise')
axis equal
```

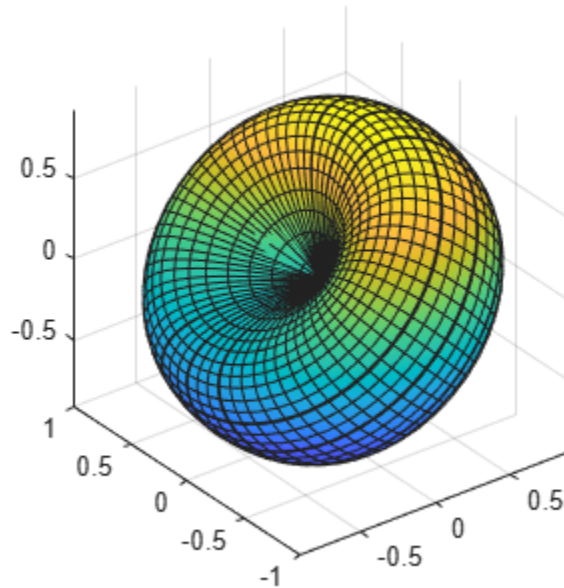


Rotate about the z-axis by 90 degrees clockwise.

```
xyzRz = Rz*Rx45;
Rx45Rz90 = subs(xyzRz, t, -pi/2);

fsurf(Rx45Rz90(1), Rx45Rz90(2), Rx45Rz90(3))
title('Rotating by \pi/2 about z, clockwise')
axis equal
```

Rotating by  $\pi/2$  about z, clockwise

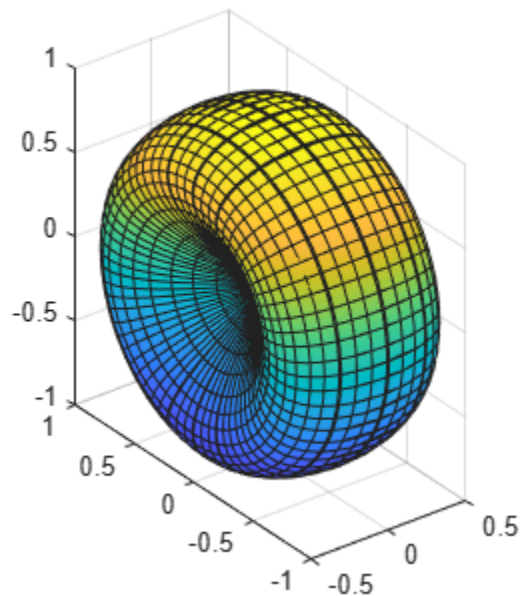


Rotate about the y-axis by 45 degrees clockwise.

```
xyzRy = Ry*Rx45Rz90;
Rx45Rz90Ry45 = subs(xyzRy, t, -pi/4);

fsurf(Rx45Rz90Ry45(1), Rx45Rz90Ry45(2), Rx45Rz90Ry45(3))
title('Rotating by \pi/4 about y, clockwise')
axis equal
```

Rotating by  $\pi/4$  about y, clockwise





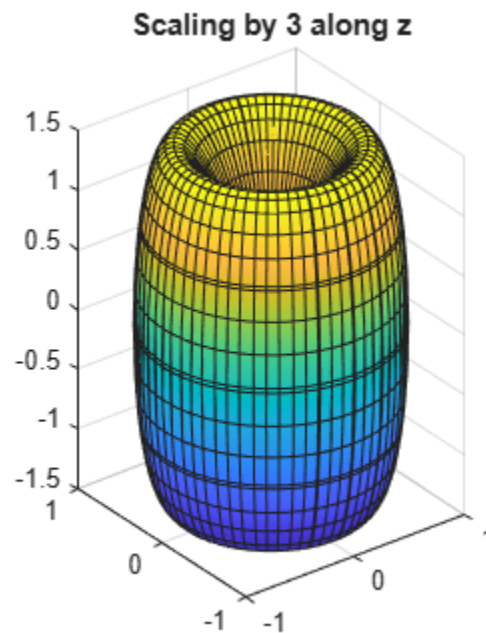
### Scale and Rotate

Scale the surface by the factor 3 along the z-axis. You can multiply the expression for z by 3,  $z = 3 \cdot z$ . The more general approach is to create a scaling matrix, and then multiply the scaling matrix by the vector of coordinates.

```
S = [1 0 0; 0 1 0; 0 0 3];
xyzScaled = S*[x; y; z]
```

```
xyzScaled =
    ( cos(u) sin(v) )
    ( sin(u) sin(v) )
    ( 3 cos(v) sin(v) )
```

```
fsurf(xyzScaled(1), xyzScaled(2), xyzScaled(3))
title('Scaling by 3 along z')
axis equal
```



Rotate the scaled surface about the x-, y-, and z-axis by 45 degrees clockwise, in order z, then y, then x. The rotation matrix for this transformation is as follows.

```
R = Rx*Ry*Rz
```

```
R =
    ( cos(t)^2      -cos(t) sin(t)      sin(t) )
    (      sigma_1  cos(t)^2 - sin(t)^3  -cos(t) sin(t) )
    ( sin(t)^2 - cos(t)^2 sin(t)      sigma_1      cos(t)^2 )
```

where

$$\sigma_1 = \cos(t) \sin(t)^2 + \cos(t) \sin(t)$$

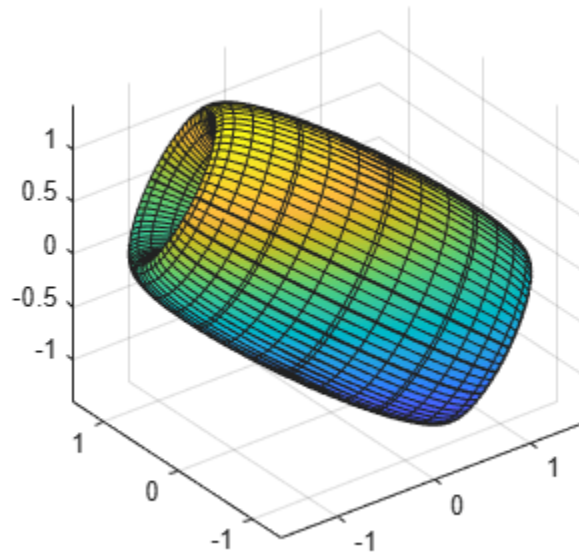
Use the rotation matrix to find the new coordinates.

```
xyzScaledRotated = R*xyzScaled;
xyzSR45 = subs(xyzScaledRotated, t, -pi/4);
```

Plot the surface.

```
fsurf(xyzSR45(1), xyzSR45(2), xyzSR45(3))
title('Rotating by \pi/4 about x, y, and z, clockwise')
axis equal
```

Rotating by  $\pi/4$  about x, y, and z, clockwise



### Check Properties of Rotation Matrix R

Rotation matrices are orthogonal matrices. Thus, the transpose of R is also its inverse, and the determinant of R is 1.

```
simplify(R.'*R)
```

```
ans =
    (1 0 0)
    (0 1 0)
    (0 0 1)
```

```
simplify(det(R))
```

```
ans = 1
```

## Clear Assumptions and Reset the Symbolic Engine

The symbolic engine workspace associated with the MATLAB workspace is usually empty. The MATLAB workspace tracks the values of symbolic variables, and passes them to the symbolic engine for evaluation as necessary. However, the symbolic engine workspace contains all assumptions you make about symbolic variables, such as whether a variable is real, positive, integer, greater or less than a certain value, and so on. These assumptions can affect solutions to equations, simplifications, and transformations, as explained in “Effects of Assumptions on Computations” on page 3-381.

For example, create a symbolic variable  $x$  and assume that it is positive.

```
syms x
assume(x > 0)
```

If you clear the variable  $x$  using the command `clear x`, the MATLAB workspace does not clear the assumption from the symbolic engine workspace.

```
clear x
assumptions
```

```
ans =
0 < x
```

To clear the assumption for the variable  $x$ , use the command `assume(x, 'clear')`.

```
syms x
assume(x > 0)
assume(x, 'clear')
assumptions
```

```
ans =
Empty sym: 1-by-0
```

Alternatively, you can create a fresh symbolic variable without assumptions using `syms`.

```
syms x
```

If you want to clear a symbolic variable and also reset the symbolic engine, use the command `clear all`.

```
syms x positive
clear all
whos
assumptions
```

```
ans =
Empty sym: 1-by-0
```

The following shows how the MATLAB workspace and symbolic engine workspace differ in their responses to a sequence of commands.

| Step | Command                                           | MATLAB Workspace | Symbolic Engine Workspace |
|------|---------------------------------------------------|------------------|---------------------------|
| 1    | syms x positive<br>or<br>syms x;<br>assume(x > 0) | x                | x > 0                     |

| Step | Command                | MATLAB Workspace | Symbolic Engine Workspace |
|------|------------------------|------------------|---------------------------|
| 2    | <code>clear x</code>   | empty            | $x > 0$                   |
| 3    | <code>syms x</code>    | x                | empty                     |
| 4    | <code>clear all</code> | empty            | empty                     |

## Check Assumptions Set on Variables

To check whether a variable, say  $x$ , has any assumptions in the symbolic engine workspace associated with the MATLAB workspace, use the `assumptions` function in the MATLAB Live Editor:

```
assumptions(x)
```

If the function returns an empty symbolic object, there are no additional assumptions on the variable. The default assumption is that  $x$  represents any complex number. Otherwise, there are additional assumptions on the value that the variable represents.

For example, while declaring the symbolic variable  $x$ , make an assumption that the value of this variable is a real number.

```
syms x real
assumptions(x)
```

```
ans =
in(x, 'real')
```

Another way to set an assumption is to use the `assume` function.

```
syms z
assume(z ~= 0);
assumptions(z)
```

```
ans =
z ~= 0
```

To see assumptions set on all variables in the MATLAB workspace, use `assumptions` without input arguments.

```
assumptions

ans =
[ in(x, 'real'), z ~= 0]
```

Clear assumptions set on  $x$  and  $z$ .

```
assume([x z], 'clear')

assumptions
```

```
ans =
Empty sym: 1-by-0
```

Equivalently, the following command also clears assumptions from  $x$  and  $z$ .

```
syms x z
```

## Effects of Assumptions on Computations

Assumptions can affect many computations, including results returned by the `solve` and `simplify` functions. For example, solve this equation without any additional assumptions on its variable.

```
syms x
solve(x^4 == 1, x)

ans =
-1
1
-1i
1i
```

Now assume that  $x$  is real and solve the same equation.

```
syms x real
solve(x^4 == 1, x)

ans =
-1
1
```

Use the `assumeAlso` function to add the assumption that  $x$  is also positive.

```
assumeAlso(x > 0)
solve(x^4 == 1, x)

ans =
1
```

Clearing  $x$  does not change the underlying assumptions that  $x$  is real and positive.

```
clear x
x = sym('x');
assumptions(x)
solve(x^4 == 1, x)

ans =
[ in(x, 'real'), 0 < x]
ans =
1
```

Clearing  $x$  with `assume(x, 'clear')` or `syms x` clears the assumptions.

```
syms x
assumptions(x)

ans =
Empty sym: 1-by-0
```

## Recognize and Avoid Round-Off Errors

When approximating a value numerically, remember that floating-point results can be sensitive to the precision used. Also, floating-point results are prone to round-off errors. The following approaches can help you recognize and avoid incorrect results.

### In this section...

“Use Symbolic Computations When Possible” on page 3-382

“Perform Calculations with Increased Precision” on page 3-382

“Compare Symbolic and Numeric Results” on page 3-384

“Plot the Function or Expression” on page 3-384

### Use Symbolic Computations When Possible

Performing computations symbolically on page 2-32 is recommended because exact symbolic computations are not prone to round-off errors. For example, standard mathematical constants have their own symbolic representations in Symbolic Math Toolbox:

```
pi
sym(pi)

ans =
    3.1416

ans =
    pi
```

Avoid unnecessary use of numeric approximations. A floating-point number approximates a constant; it is not the constant itself. Using this approximation, you can get incorrect results. For example, the `heaviside` special function returns different results for the sine of `sym(pi)` and the sine of the numeric approximation of `pi`:

```
heaviside(sin(sym(pi)))
heaviside(sin(pi))

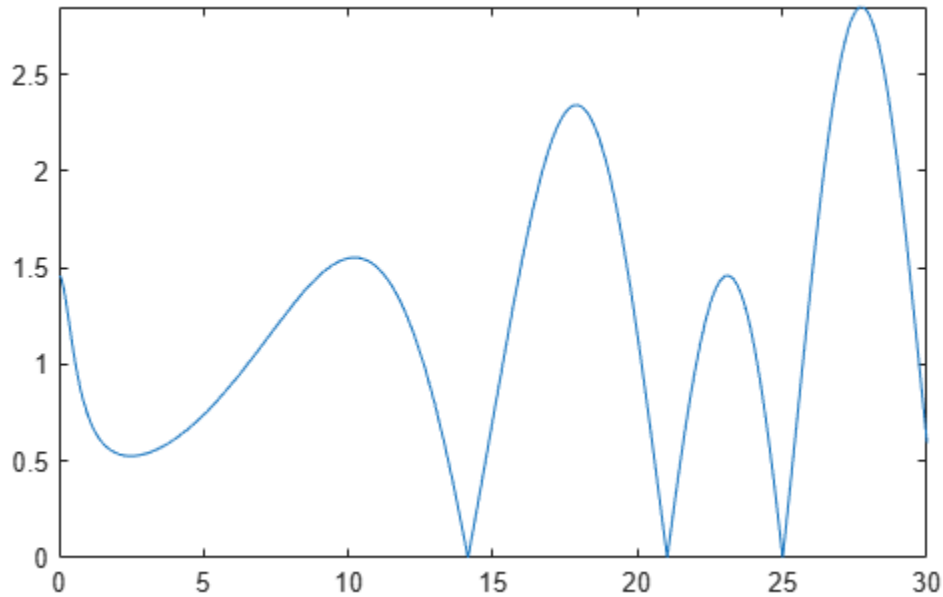
ans =
    1/2

ans =
    1
```

### Perform Calculations with Increased Precision

The Riemann hypothesis states that all nontrivial zeros of the Riemann Zeta function  $\zeta(z)$  have the same real part  $\Re(z) = 1/2$ . To locate possible zeros of the Zeta function, plot its absolute value  $|\zeta(1/2 + iy)|$ . The following plot shows the first three nontrivial roots of the Zeta function  $|\zeta(1/2 + iy)|$ .

```
syms y
fplot(abs(zeta(1/2 + i*y)), [0 30])
```



Use the numeric solver `vpasolve` to approximate the first three zeros of this Zeta function:

```
vpasolve(zeta(1/2 + i*y), y, 15)
vpasolve(zeta(1/2 + i*y), y, 20)
vpasolve(zeta(1/2 + i*y), y, 25)

ans =
14.134725141734693790457251983562

ans =
21.022039638771554992628479593897

ans =
25.010857580145688763213790992563
```

Now, consider the same function, but slightly increase the real part,  $\zeta\left(\frac{1000000001}{2000000000} + iy\right)$ . According to the Riemann hypothesis, this function does not have a zero for any real value  $y$ . If you use `vpasolve` with the 10 significant decimal digits, the solver finds the following (nonexisting) zero of the Zeta function:

```
old = digits;
digits(10)
vpasolve(zeta(1000000001/2000000000 + i*y), y, 15)

ans =
14.13472514
```

Increasing the number of digits shows that the result is incorrect. The Zeta function  $\zeta\left(\frac{1000000001}{2000000000} + iy\right)$  does not have a zero for any real value  $14 < y < 15$ :

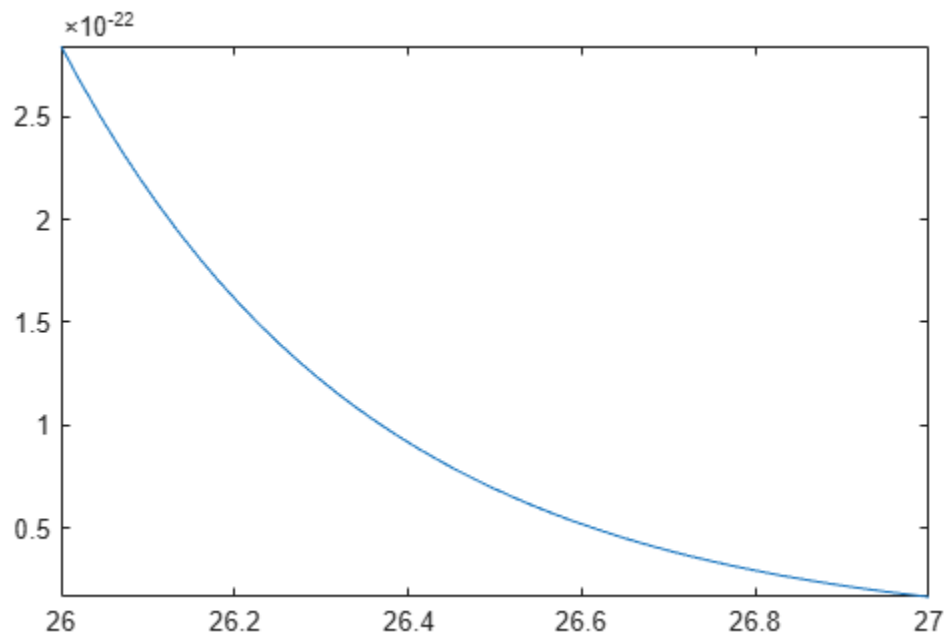




```
ans =  
-2854.225191
```

Plot this Bessel function for the values of  $x$  around  $53/2$ . The function plot shows that the approximation is incorrect:

```
syms x  
fplot(besselj(x, sym(pi)), [26 27])
```



## Increase Speed by Reducing Precision

Increase the speed of symbolic computations by reducing their precision. You can achieve this by using variable-precision arithmetic, provided by the `vpa` and `digits` functions in Symbolic Math Toolbox. When you reduce precision, you are gaining performance by reducing accuracy. For details, see “Choose Numeric or Symbolic Arithmetic” on page 2-32.

For example, finding the Riemann zeta function of the large matrix `C` using the symbolic `zeta` function takes a long time. First, initialize `C`.

```
[X,Y] = meshgrid((0:0.0025:.75),(5:-0.05:0));  
C = X + Y*i;
```

Then, find the time taken to calculate `zeta(C)`.

```
tic  
zeta(C);  
toc
```

Elapsed time is 340.204407 seconds.

Now, repeat this operation with a lower precision by using `vpa`. First, change the precision used by `vpa` to a lower precision of 10 digits by using `digits`. Then, use `vpa` to reduce the precision of `C` and find `zeta(C)` again. The operation is significantly faster.

```
digits(10)  
vpaC = vpa(C);  
tic  
zeta(vpaC);  
toc
```

Elapsed time is 113.792543 seconds.

---

**Note** `vpa` output is symbolic. To use symbolic output with a MATLAB function that does not accept symbolic values, convert symbolic values to double precision by using `double`.

---

For larger matrices, the difference in computation time can be even more significant. For example, consider the 1001-by-301 matrix `C`.

```
[X,Y] = meshgrid((0:0.0025:.75),(5:-0.005:0));  
C = X + Y*i;
```

Running `zeta(vpa(C))` with 10-digit precision takes 15 minutes, while running `zeta(C)` takes three times as long.

```
digits(10)  
vpaC = vpa(C);  
tic  
zeta(vpaC);  
toc
```

Elapsed time is 886.035806 seconds.

```
tic  
zeta(C);  
toc
```

Elapsed time is 2441.991572 seconds.

---

**Note** If you want to *increase* precision, see “Increase Precision of Numeric Calculations” on page 2-36.

---

## Prime Factorizations

This example shows how to use some elementary functions on sym objects using the Symbolic Math Toolbox™.

The built-in integer types of MATLAB® are suitable for integers smaller than  $2^{64}$ . However, we want to carry out statistical investigations on prime factorizations of larger integers. To do this, we use symbolic integers because their size is unlimited. Investigate the integers between  $N_0 + 1$  and  $N_0 + 100$ , where  $N_0 = 3 \cdot 10^{23}$ . The built-in data types cannot store such values exactly. Thus, wrap the innermost number with sym to use symbolic representation in the calculations. This avoids rounding or overflow errors:

```
N0 = 3*sym(10)^23;
disp(['Roundng error for doubles: ' char(3*10^23 - N0)]);

Roundng error for doubles: -25165824

disp(['Overflow error for integers: ' char(3*uint64(10)^23 - N0)]);

Overflow error for integers: -299981553255926290448385
```

In arithmetical operations, symbolic numbers can be combined with doubles, and the conversion takes place before the operation. Thus, the following definition cannot cause rounding errors:

```
A = N0 + (1:100);
```

Compute the prime factorizations of the elements of A using factor. The number of prime factors differs. Arrays cannot contain vectors of different lengths, but cell arrays can. To avoid memory re-allocations, initialize the cell array first, then compute the factorizations in a loop:

```
Bcell = cell(1, 100);
for i=1:100
    Bcell{i} = factor(A(i));
end
```

A more efficient approach is to use arrayfun. Setting UniformOutput to false returns the result as a cell array.

```
Bcell = arrayfun(@factor, A, 'UniformOutput', false);
```

For example, the first prime factorizations are:

```
Bcell{1:5}

ans = (13 43 233 2303316007278478583)

ans = (2 17 173 991 1223 244939 171805943)

ans = (3 11 47 139 2531 549797184491917)

ans = (2 2 330131 2953837 76910994983)

ans = (5 6271 2650266823 3610146697)
```

Obtain the largest prime factor using max. Note that if the output consists of sym objects, the option UniformOutput always has to be set to false even if the output is uniform.

```
Mcell = cellfun(@max, Bcell, 'UniformOutput', false);
```

For example, the first maximal prime factors are:

```
Mcell{1:5}
```

```
ans = 2303316007278478583
```

```
ans = 171805943
```

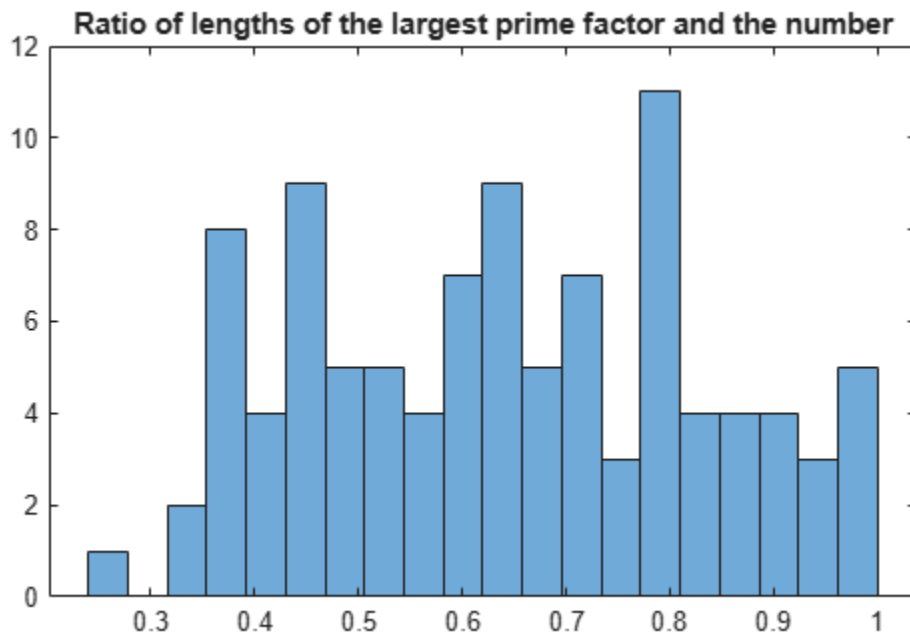
```
ans = 549797184491917
```

```
ans = 76910994983
```

```
ans = 3610146697
```

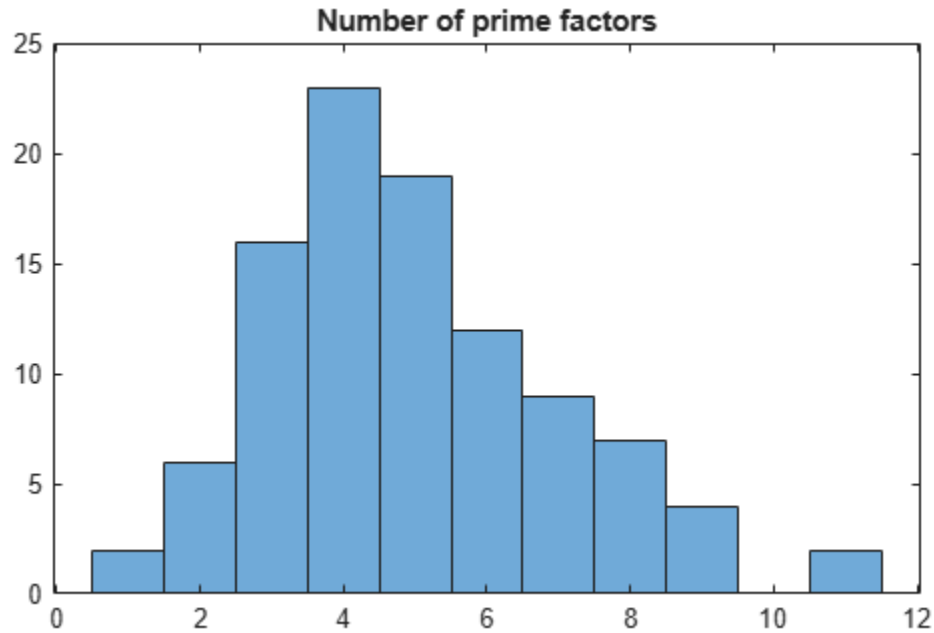
Convert the cell array to a symbolic vector, and investigate the ratio of the lengths of the largest prime factor and the number as a whole. You can apply arithmetical operations elementwise to symbolic vectors, in the same way as for doubles. Note that most statistical functions require their arguments to be double-precision numbers.

```
M = [Mcell{:}];  
histogram(double(log(M)./log(A)), 20);  
title('Ratio of lengths of the largest prime factor and the number');
```



In the same way, now investigate the distribution of the number of prime factors. Here, the result contains uniform numeric data. Therefore, you do not need to set `UniformOutput` to `false`.

```
omega = cellfun(@numel, Bcell);  
histogram(omega);  
title('Number of prime factors');
```



The interval under investigation contains two prime numbers:

```
A(omega == 1)
```

```
ans = (300000000000000000000000000037 30000000000000000000000000049)
```

We check that the maximal prime factors are about equally often in the residue classes 1 and 3 modulo 4. Note that equations of sym objects are symbolic objects themselves and not logical values; we have to convert them before we can sum them:

```
sum(logical(mod(M, 4) == 1))
```

```
ans =  
49
```

```
sum(logical(mod(M, 4) == 3))
```

```
ans =  
51
```

## Handling Large Integers to Solve the 196 Problem

This example shows how to work with large integers and their decimal representation using the Symbolic Math Toolbox™.

```
N = sym(89);
for k=0:100
    s1 = char(N);
    s2 = fliplr(s1);
    if strcmp(s1,s2)
        disp(['Finished in iteration ' num2str(k)])
        break
    end
    N = N + sym(s2);
    disp(N)
end
```

187

968

1837

9218

17347

### Palindromes

A character string is called a palindrome if it is the same when read backward. A positive integer is called a palindrome if its decimal representation is a palindrome. For example, 191, 313 and 5885 are all palindromes.

Consider the following algorithm

- Start with any positive integer  $N$  and add it to its mirror image.
- Repeat this step with the resulting number until you obtain a palindrome.

For example, choose  $N=89$ . The first 3 iterations of adding the number to its mirror image give

$$89 + 98 = 187$$

$$187 + 781 = 968$$

$$968 + 869 = 1837$$

Eventually, after 24 iterations, you would arrive at the palindrome 8813200023188.

```
N = sym(89);
for k=0:100
    s1 = char(N);
    s2 = fliplr(s1);
    if strcmp(s1, s2)
        disp(['Finished in iteration ' num2str(k)])
        break
    end
    N = N + sym(s2);
    disp(N)
end
```

187

968

1837

9218

17347

91718

173437

907808

1716517

8872688

17735476

85189247

159487405

664272356

1317544822

3602001953

7193004016

13297007933

47267087164

93445163438

176881317877

955594506548

1801200002107

8813200023188

Finished in iteration 24



### The 196-Problem

Does the algorithm terminate for every  $N$ ?

The problem is still open, and palindrome aficionados have invested many CPU years into the  $N = 196$  case which gave the problem its name. In order to play with this problem in MATLAB®, symbolic integers are useful because their size is unlimited. Use the function `sym` to convert strings of decimal digits to symbolic integers, and `char` (not `num2str`) to convert back.

Investigating the famous  $N = 196$  case produces truly huge numbers. To see how many decimal digits an integer has, simply use `log10`.

```
N = sym(196);
for k=0:1000
    s1 = char(N);
    s2 = fliplr(s1);
    N = N + sym(s2);
end
disp(['Number of digits after ' num2str(k) ' iterations: ' char(ceil(log10(N)))]);
```

Number of digits after 1000 iterations: 411



# Graphics

---

- “Create Plots of Symbolic Expressions” on page 4-2
- “Transform Spherical Coordinates to Cartesian Coordinates and Plot Analytically” on page 4-10
- “Tangent Plane and Normal Line of Implicit Surface” on page 4-17
- “Analytical Plotting with Symbolic Math Toolbox” on page 4-21
- “Simulate the Motion of the Periodic Swing of a Pendulum” on page 4-35
- “Animation and Solution of Double Pendulum Motion” on page 4-47
- “Animation and Model of Automotive Piston” on page 4-54

## Create Plots of Symbolic Expressions

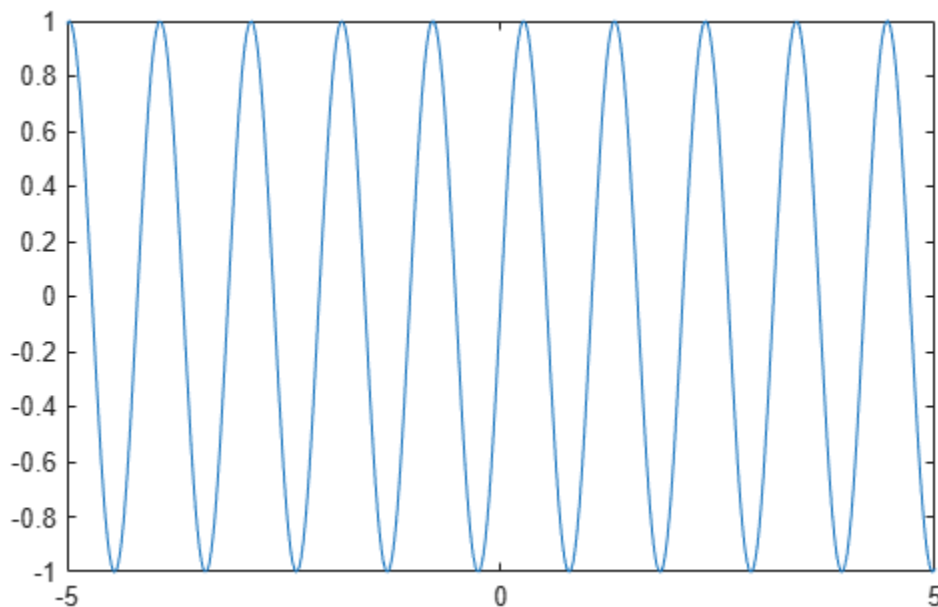
### Plot with Symbolic Plotting Functions

MATLAB® provides many techniques for plotting numerical data. Graphical capabilities of MATLAB include plotting tools, standard plotting functions, graphic manipulation and data exploration tools, and tools for printing and exporting graphics to standard formats. Symbolic Math Toolbox™ expands these graphical capabilities and lets you plot symbolic functions using:

- `fplot` to create 2-D plots of symbolic expressions, equations, or functions in Cartesian coordinates.
- `fplot3` to create 3-D parametric plots.
- `fpolarplot` to create plots in polar coordinates. (*since R2024a*)
- `fsurf` to create surface plots.
- `fcontour` to create contour plots.
- `fmesh` to create mesh plots.

Plot the symbolic expression  $\sin(6x)$  by using `fplot`. By default, `fplot` uses the range  $-5 < x < 5$ .

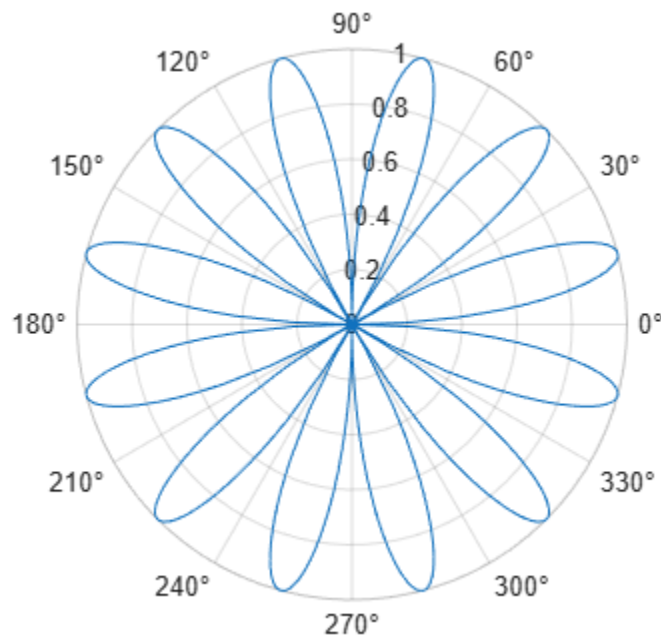
```
syms x  
fplot(sin(6*x))
```



Plot a symbolic expression or function in polar coordinates  $r$  (radius) and  $\theta$  (polar angle) by using `fpolarplot`. By default, `fpolarplot` plots a symbolic expression or function over the interval  $0 < \theta < 2\pi$ .

Plot the symbolic expression  $\sin(6t)$  in polar coordinates.

```
syms t
fpolarplot(sin(6*t))
```



### Plot Functions Numerically

As an alternative to plotting expressions symbolically, you can substitute symbolic variables with numeric values by using `subs`. Then, you can use these numeric values with plotting functions in MATLAB.

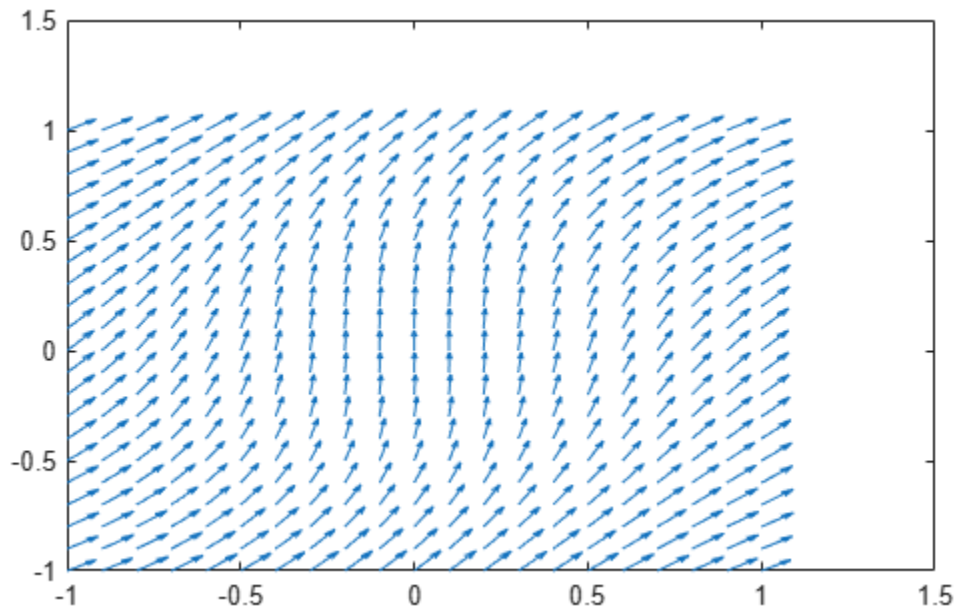
In the following expressions `u` and `v`, substitute the symbolic variables `x` and `y` with the numeric values defined by `meshgrid`.

```
syms x y
u = sin(x^2 + y^2);
v = cos(x*y);
[X,Y] = meshgrid(-1:.1:1,-1:.1:1);
U = subs(u,[x y],[X,Y]);
V = subs(v,[x y],[X,Y]);
```

Now, you can plot `U` and `V` by using standard MATLAB plotting functions.

Create a plot of the vector field defined by the functions `U(X,Y)` and `V(X,Y)` by using the MATLAB `quiver` function.

```
quiver(X,Y,U,V)
```



### Plot Multiple Symbolic Functions in One Graph

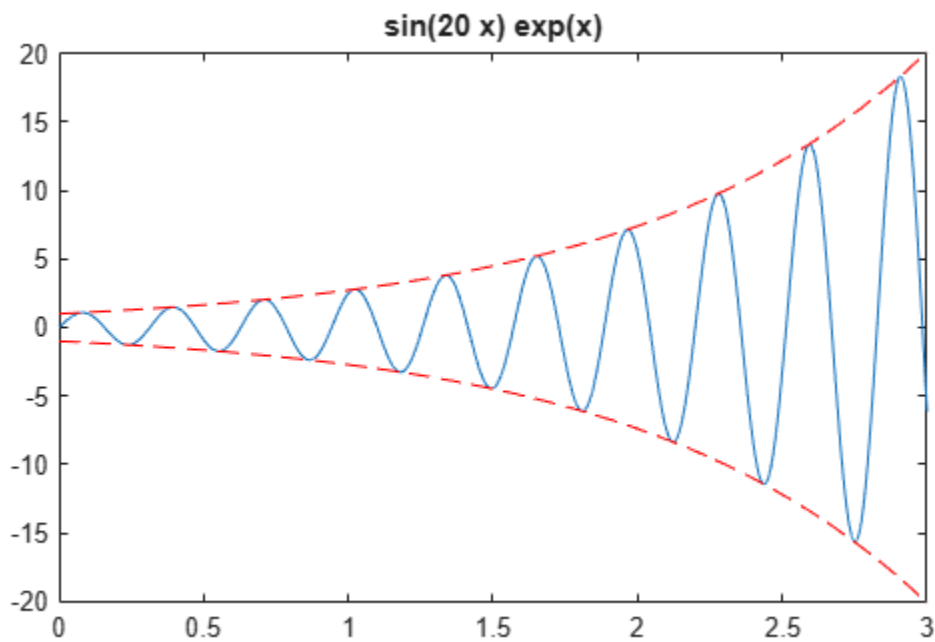
Plot several functions on one graph by adding the functions sequentially. After plotting the first function, add successive functions by using the `hold on` command. The `hold on` command keeps the existing plots. Without the `hold on` command, each new plot replaces any existing plot. After the `hold on` command, each new plot appears on top of existing plots. Switch back to the default behavior of replacing plots by using the `hold off` command.

Plot  $f = e^x \sin(20x)$  using `fplot`. Show the bounds of  $f$  by superimposing plots of  $e^x$  and  $-e^x$  as dashed red lines. Set the title by using the `DisplayName` property of the object returned by `fplot`.

```
syms x y
f = exp(x)*sin(20*x)

f = sin(20 x) e^x

obj = fplot(f,[0 3]);
hold on
fplot(exp(x),[0 3],"--r")
fplot(-exp(x),[0 3],"--r")
title(obj.DisplayName)
hold off
```



### Plot Multiple Symbolic Functions in One Figure

Display several functions side-by-side in one figure by dividing the figure window into several subplots using `subplot`. The command `subplot(m,n,p)` divides the figure into a  $m$ -by- $n$  matrix of subplots and selects the subplot  $p$ . Display multiple plots in separate subplots by selecting the subplot and using plotting commands. Plotting into multiple subplots is useful for side-by-side comparisons of plots.

Compare plots of  $\sin((x^2 + y^2)/a)$  for  $a = 10, 20, 50, 100$  by using `subplot` to create side-by-side subplots.

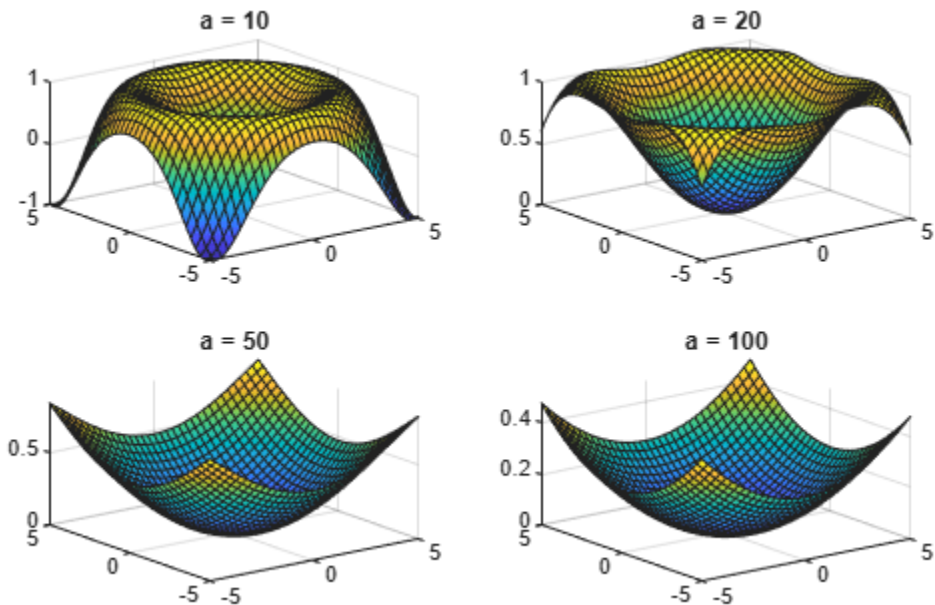
```
syms x y a
f = sin((x^2 + y^2)/a);
```

```
subplot(2,2,1)
fsurf(subs(f,a,10))
title("a = 10")
```

```
subplot(2,2,2)
fsurf(subs(f,a,20))
title("a = 20")
```

```
subplot(2,2,3)
fsurf(subs(f,a,50))
title("a = 50")
```

```
subplot(2,2,4)
fsurf(subs(f,a,100))
title("a = 100")
```



### Combine Symbolic Function Plots and Numeric Data Plots

Plot numeric and symbolic data on the same graph by using MATLAB and Symbolic Math Toolbox functions together.

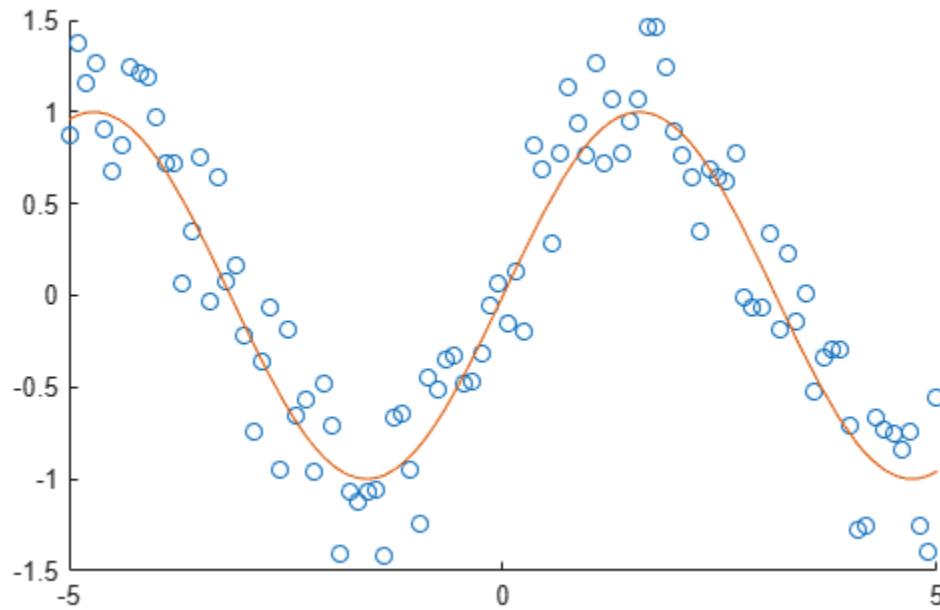
For numeric values of  $x$  between  $[-5, 5]$ , return a noisy sine curve by finding  $y = \sin(x)$  and adding random values to  $y$ . View the noisy sine curve by using `scatter` to plot the points  $(x_1, y_1)$ ,  $(x_2, y_2)$ , and so on.

```
figure
rng("default")
x = linspace(-5,5);
y = sin(x) + (-1).^randi(10,1,100).*rand(1,100)./2;
scatter(x,y)
```

Show the underlying structure in the points by superimposing a plot of the sine function. First, use `hold on` to retain the scatter plot. Then, use `fplot` to plot the sine function.

```
hold on
syms t
fplot(sin(t))
hold off
```





### Combine Numeric and Symbolic Plots in 3-D

Combine symbolic and numeric plots in 3-D by using MATLAB and Symbolic Math Toolbox plotting functions. Symbolic Math Toolbox provides these 3-D plotting functions:

- `fplot3` creates 3-D parameterized line plots.
- `fsurf` creates 3-D surface plots.
- `fmesh` creates 3-D mesh plots.

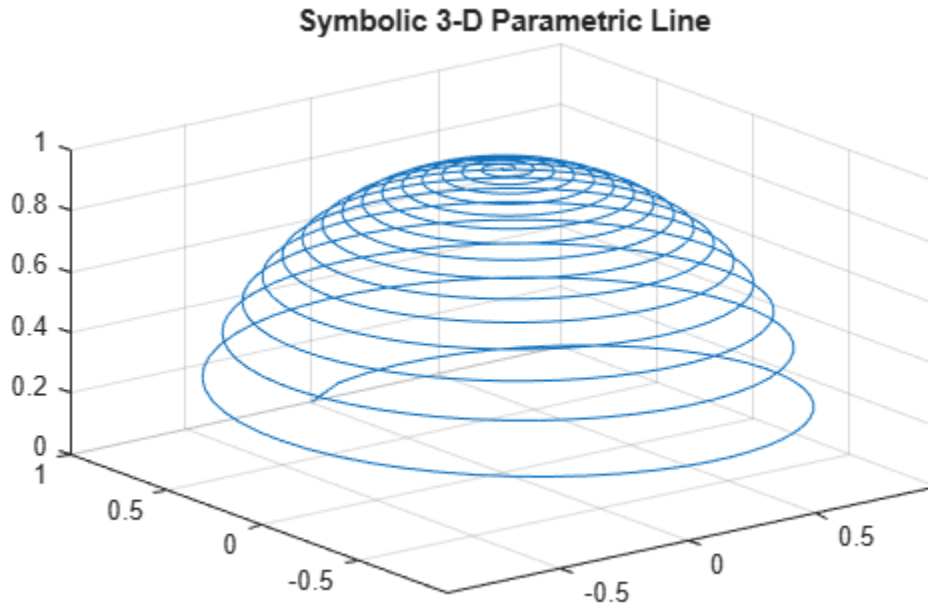
Create a spiral plot by using `fplot3` to plot the parametric line

$$x = (1 - t) \sin(100t)$$

$$y = (1 - t) \cos(100t)$$

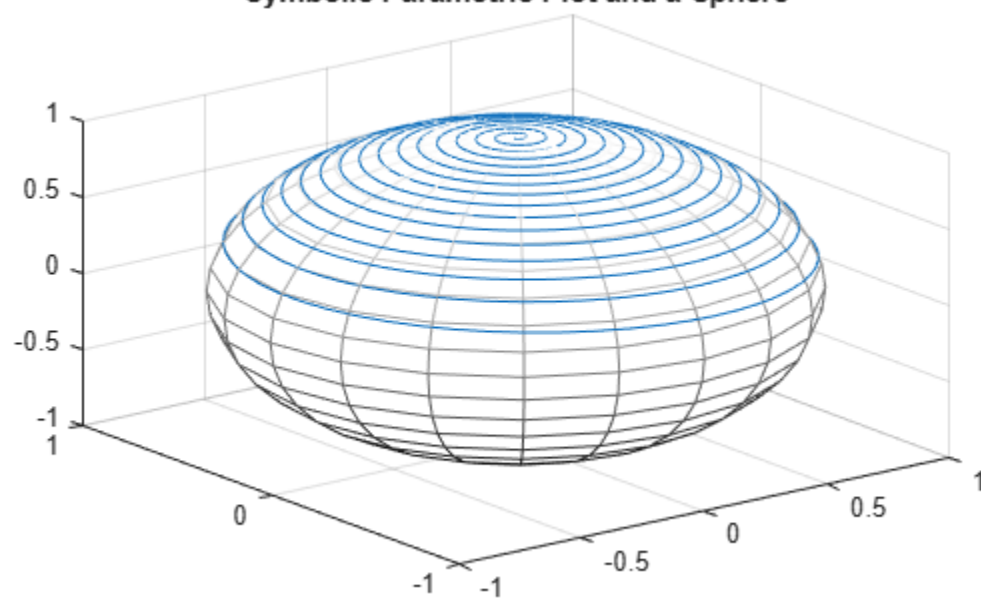
$$z = \sqrt{1 - x^2 - y^2}.$$

```
syms t
x = (1-t)*sin(100*t);
y = (1-t)*cos(100*t);
z = sqrt(1 - x^2 - y^2);
fplot3(x,y,z,[0 1])
title("Symbolic 3-D Parametric Line")
```



Superimpose a plot of a sphere with radius 1 and center at (0,0,0). Find points on the sphere numerically by using `sphere`. Plot the sphere by using `mesh`. The resulting plot shows the symbolic parametric line wrapped around the top hemisphere.

```
hold on
[X,Y,Z] = sphere;
mesh(X,Y,Z)
colormap(gray)
title("Symbolic Parametric Plot and a Sphere")
hold off
```

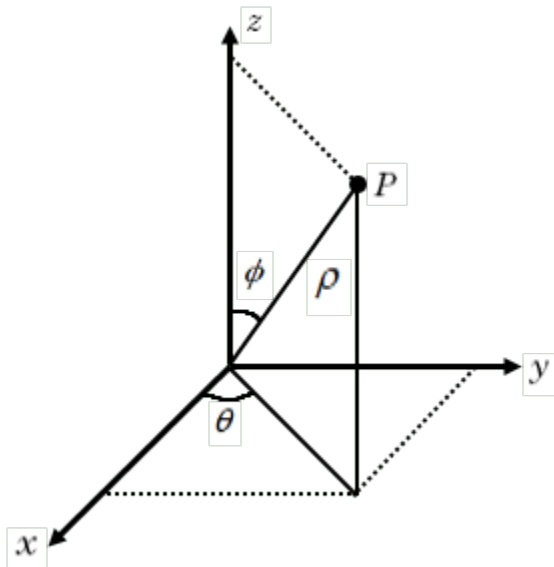
**Symbolic Parametric Plot and a Sphere**

## Transform Spherical Coordinates to Cartesian Coordinates and Plot Analytically

This example shows how to transform a symbolic expression from spherical coordinates to Cartesian coordinates, and plot the converted expression analytically without explicitly generating numerical data.

In the spherical coordinate system, the location of a point  $P$  can be characterized by three variables. Different textbooks have different conventions for the variables used to describe spherical coordinates. For these examples, this convention is used:

- The radial distance  $\rho$
- The azimuthal angle  $\theta$
- The polar angle  $\phi$



The transformation of the point  $P$  from spherical coordinates  $(\rho, \theta, \phi)$  to Cartesian coordinates  $(x, y, z)$  is given by

$$\begin{aligned}x &= \rho \sin \phi \cos \theta, \\y &= \rho \sin \phi \sin \theta, \\z &= \rho \cos \phi.\end{aligned}$$

By transforming symbolic expressions from spherical coordinates to Cartesian coordinates, you can then plot the expressions using Symbolic Math Toolbox™ graphics functions, such as `fplot3` and `fsurf`.

### Plot a Point and Its Projections

Plot the point  $P$  that is located at  $(\rho, \theta, \phi) = (1, 1.2, 0.75)$ .

Transform the spherical coordinates to Cartesian coordinates ( $x_P, y_P, z_P$ ). Because the converted coordinates contain numerical values, use `plot3` to plot the point.

```
rho = 1;
theta = 1.2;
phi = 0.75;
x_P = rho*sin(phi)*cos(theta);
y_P = rho*sin(phi)*sin(theta);
z_P = rho*cos(phi);
plot3(x_P,y_P,z_P, 'ko', 'MarkerSize',10, 'MarkerFaceColor', 'k')
hold on
```

Label each axis in the plot, change the line of sight, and set the axis scaling to use equal data units.

```
xlabel('x')
ylabel('y')
zlabel('z')
view([75 40])
axis equal;
```

Next, plot the line projection of the point  $P$  to the origin. This line projection in spherical coordinates is parameterized by  $(r, 1.2, 0.75)$ , with  $r$  ranging from 0 to 1. Specify this line parameterization as symbolic expressions, and plot it by using `fplot3`.

```
syms r
xr = r*sin(phi)*cos(theta);
yr = r*sin(phi)*sin(theta);
zr = r*cos(phi);
fplot3(xr,yr,zr,[0 rho], 'k')
```

Plot the vertical line projection to the  $xy$ -plane. This line projection in Cartesian coordinates is parameterized by  $(x_P, y_P, z)$ , with  $z$  ranging from 0 to  $z_P$ .

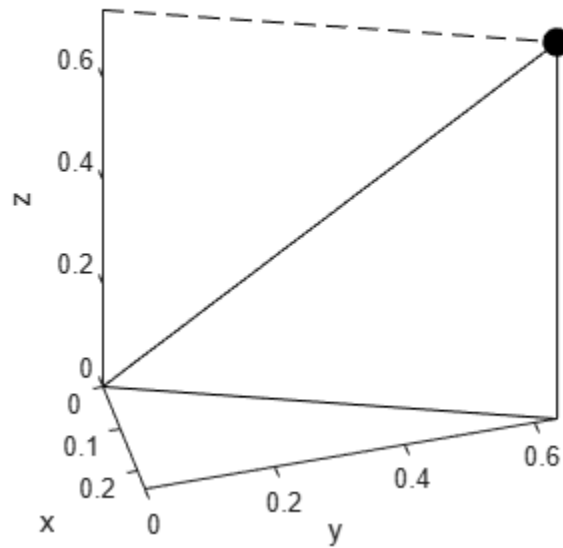
```
syms z
fplot3(sym(x_P),sym(y_P),z,[0 z_P], 'k')
```

Plot the top horizontal line projection to the  $z$ -axis. This line projection in Cartesian coordinates is parameterized by  $(r \sin\phi \cos\theta, r \sin\phi \sin\theta, z_P)$ , with  $r$  ranging from 0 to 1.

```
fplot3(xr,yr,sym(z_P),[0 rho], 'k--')
```

Plot the bottom horizontal line projection to the  $z$ -axis. This line projection in Cartesian coordinates is parameterized by  $(r \sin\phi \cos\theta, r \sin\phi \sin\theta, 0)$ , with  $r$  ranging from 0 to 1.

```
fplot3(xr,yr,sym(0),[0 rho], 'k')
```

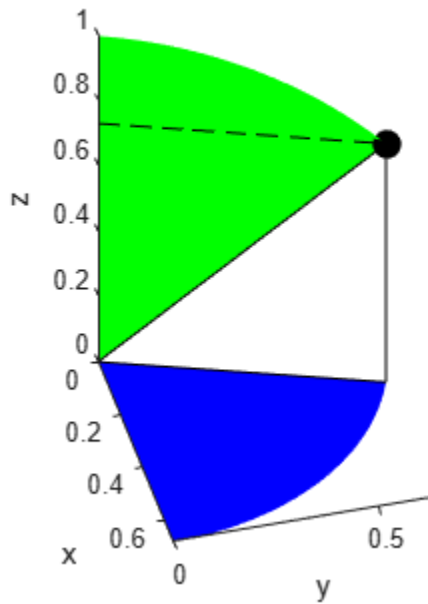


Next, plot the plane that shows the span of the azimuthal angle  $\theta$  in the  $xy$ -plane with the coordinate  $z = 0$ .

```
syms s t
x_azimuthal = s*sin(phi)*cos(t);
y_azimuthal = s*sin(phi)*sin(t);
fsurf(x_azimuthal,y_azimuthal,0,[0 rho 0 theta],'FaceColor','b','EdgeColor','none')
```

Plot the plane that shows the span of the polar angle  $\phi$ .

```
syms u v
x_polar = u*sin(v)*cos(theta);
y_polar = u*sin(v)*sin(theta);
z_polar = u*cos(v);
fsurf(x_polar,y_polar,z_polar,[0 rho 0 phi],'FaceColor','g','EdgeColor','none')
hold off
```

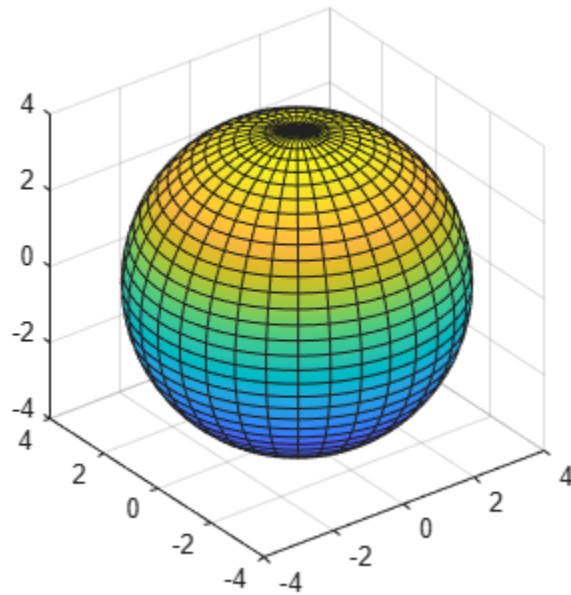


### Plot a Sphere

Plot a sphere with radius  $r = 4$ .

In spherical coordinates, the sphere is parameterized by  $(4, \theta, \phi)$ , with  $\phi$  ranging from 0 to  $\pi$  and  $\theta$  ranging from 0 to  $2\pi$ . Transform spherical coordinates to Cartesian coordinates by specifying the surface parameterization as symbolic expressions. Then plot the sphere by using `fsurf`.

```
syms phi theta
r = 4;
x = r*sin(phi)*cos(theta);
y = r*sin(phi)*sin(theta);
z = r*cos(phi);
fsurf(x,y,z,[0 pi 0 2*pi])
axis equal
```



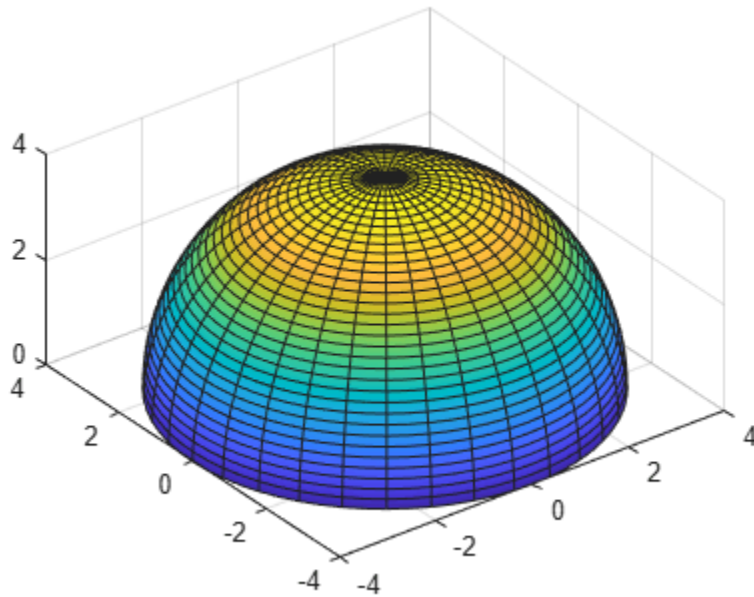
### Plot a Half Sphere

Plot a half sphere with radius  $r = 4$ .

In spherical coordinates, the sphere is parameterized by  $(4, \theta, \phi)$ , with  $\phi$  ranging from 0 to  $\pi/2$  and  $\theta$  ranging from 0 to  $2\pi$ . Transform spherical coordinates to Cartesian coordinates by specifying the surface parameterization as symbolic expressions. Then plot the half sphere by using `fsurf`.

```
syms phi theta
r = 4;
x = r*sin(phi)*cos(theta);
y = r*sin(phi)*sin(theta);
z = r*cos(phi);
fsurf(x,y,z,[0 pi/2 0 2*pi])
axis equal
```



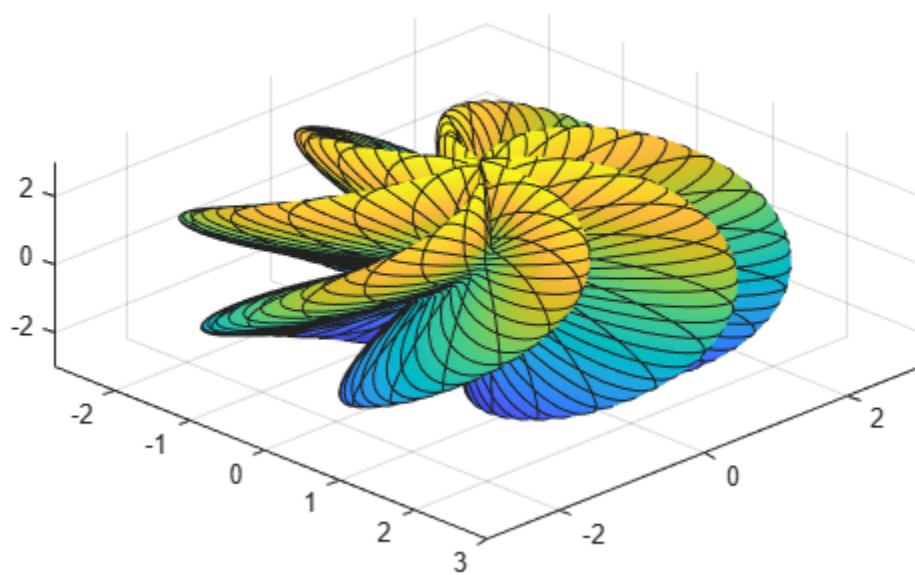


### Plot a Parameterized Surface

Plot a parameterized surface whose radial distance in spherical coordinates is related to the azimuthal and polar angles.

The surface has radial coordinates  $\rho = 2 + \sin(5\phi + 7\theta)$ , with  $\phi$  ranging from 0 to  $\pi$  and  $\theta$  ranging from 0 to  $2\pi$ . Transform spherical coordinates to Cartesian coordinates by specifying the surface parameterization as symbolic expressions. Then plot the parameterized surface by using `fsurf`.

```
syms phi theta
rho = 2 + sin(5*phi + 7*theta);
x = rho*sin(phi)*cos(theta);
y = rho*sin(phi)*sin(theta);
z = rho*cos(phi);
fsurf(x,y,z,[0 pi 0 2*pi])
view(45,50)
```



## Tangent Plane and Normal Line of Implicit Surface

Since R2021b

This example shows how to find the tangent plane and the normal line of an implicit surface. This example uses symbolic matrix variables (with the `symmatrix` data type) for compact mathematical notation.

A surface can be defined implicitly, such as the sphere  $x^2 + y^2 + z^2 = R^2$ . In general, an implicitly defined surface is expressed by the equation  $f(x, y, z) = k$ . This example finds the tangent plane and the normal line of a sphere with radius  $R = \sqrt{14}$ .

Create a symbolic matrix variable  $\mathbf{r}$  to represent the  $\langle x, y, z \rangle$  coordinates. Define the spherical function as  $f(\mathbf{r}) = \mathbf{r} \cdot \mathbf{r}$ .

```
clear; close all; clc
syms r [1 3] matrix
f = r*r.'
```

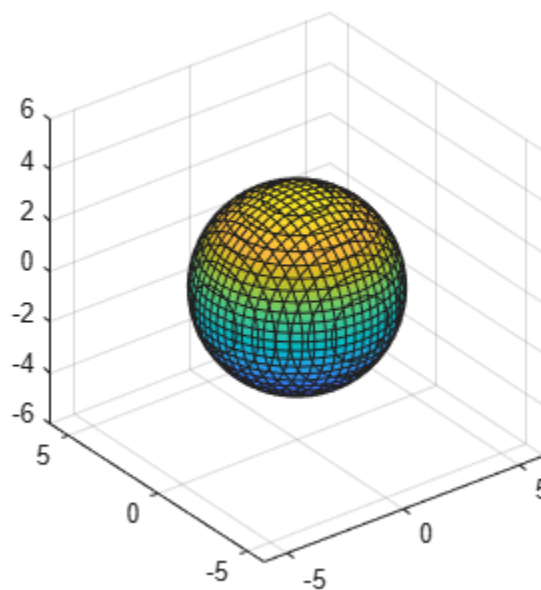
```
f = r rT
```

The implicit equation  $f(\mathbf{r}) = 14$  represents a sphere. Convert the equation to the `syms` data type using `symmatrix2sym`. Plot the equation by using the `fimplicit3` function.

```
feqn = symmatrix2sym(f == 14)
```

```
feqn = r1,12 + r1,22 + r1,32 = 14
```

```
fimplicit3(feqn)
axis equal
axis([-6 6 -6 6 -6 6])
```



Next, find the tangent plane and normal line at the point  $\mathbf{r}_0 = \langle x_0, y_0, z_0 \rangle$ .

Recall that the gradient vector of  $f$  is  $\nabla f(\mathbf{r}) = \langle f_x(\mathbf{r}), f_y(\mathbf{r}), f_z(\mathbf{r}) \rangle$ . The equation for the tangent plane at the point  $\mathbf{r}_0$  is then given by  $f_x(\mathbf{r}_0)(x - x_0) + f_y(\mathbf{r}_0)(y - y_0) + f_z(\mathbf{r}_0)(z - z_0) = 0$ . In compact mathematical notation, the tangent plane equation can be written as  $\nabla f(\mathbf{r}_0) \cdot (\mathbf{r} - \mathbf{r}_0) = 0$ .

Find the gradient of  $f(\mathbf{r})$  using the `gradient` function. Note that the result is a 3-by-1 symbolic matrix variable.

```
fgrad = gradient(f,r)
```

```
fgrad = 2 r^T
```

```
size(fgrad)
```

```
ans = 1x2
```

```
3      1
```

Define the equation for the tangent plane. Use the `subs` function to evaluate the gradient at the point  $\mathbf{r}_0 = \langle 1, -2, 3 \rangle$ .

```
r0 = [-2,1,3];
```

```
fplane = (r-r0)*subs(fgrad,r,r0)
```

```
fplane =
```

```
2 (-Σ1 + r) Σ1T
```

```
where
```

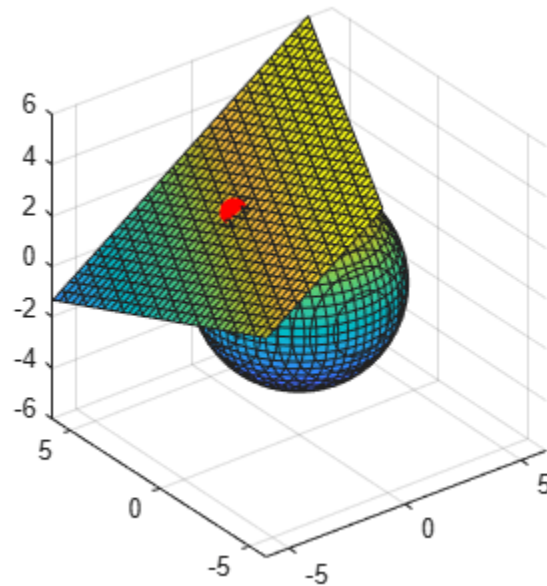
```
Σ1 = (-2 1 3)
```

Plot the point  $\mathbf{r}_0$  using `plot3`, and plot the tangent plane using `fimplicit3`.

```
hold on
```

```
plot3(r0(1),r0(2),r0(3),'ro',MarkerSize = 10,MarkerFaceColor = 'r')
```

```
fimplicit3(symmatrix2sym(fplane == 0))
```



The equation for the normal line at the point  $\mathbf{r}_0$  is given by  $\mathbf{n}(t) = \langle x_0, y_0, z_0 \rangle + t \langle f_x(\mathbf{r}_0), f_y(\mathbf{r}_0), f_z(\mathbf{r}_0) \rangle$ . In compact mathematical notation, the equation can be written as  $\mathbf{n}(t) = \mathbf{r}_0 + t \nabla f(\mathbf{r}_0)$ .

Define the equation for the normal line.

```
syms t
n = r0 + t*subs(fgrad,r,r0).'
```

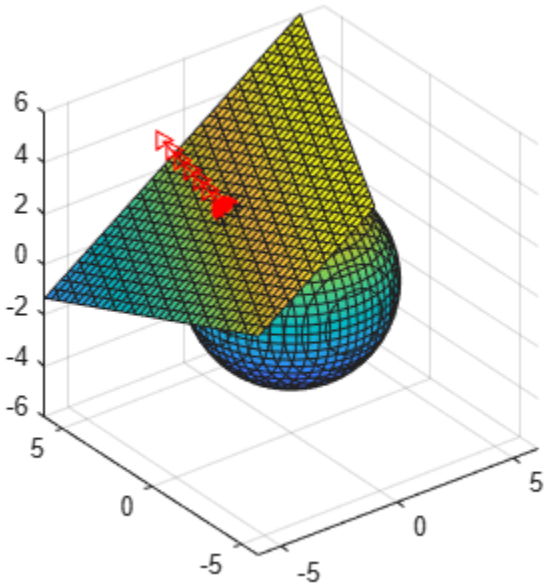
```
n =
     $\Sigma_1 + 2t\Sigma_1$ 
```

where

```
 $\Sigma_1 = (-2 \ 1 \ 3)$ 
```

Convert the normal line equation to the syms data type using `symmatrix2sym`. Extract the parametric curves  $x(t)$ ,  $y(t)$ , and  $z(t)$  for the normal line by indexing into  $n$ . Plot the normal line using `fplot3`.

```
n = symmatrix2sym(n)
n = (-4t - 2 2t + 1 6t + 3)
fplot3(n(1),n(2),n(3),[0 1], 'r->')
```



## Analytical Plotting with Symbolic Math Toolbox

Symbolic Math Toolbox™ provides analytical plotting of mathematical expressions without explicitly generating numerical data. These plots can be in 2-D or 3-D as lines, curves, contours, surfaces, or meshes.

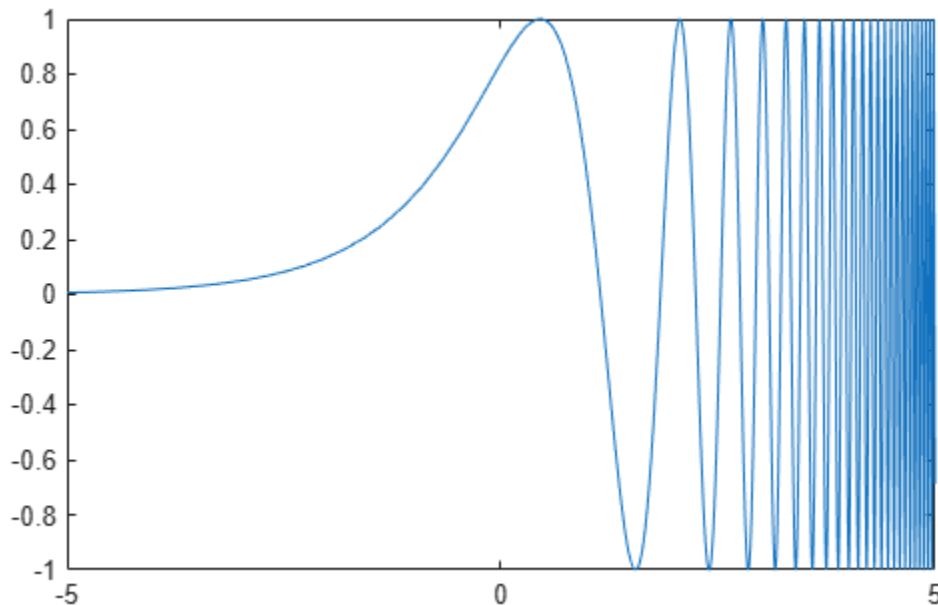
These examples feature the following graphics functions that accept symbolic functions, expressions, and equations as inputs:

- `fplot`
- `fimplicit`
- `fcontour`
- `fplot3`
- `fsurf`
- `fmesh`
- `fimplicit3`

### Plot Explicit Functions $y = f(x)$ Using `fplot`

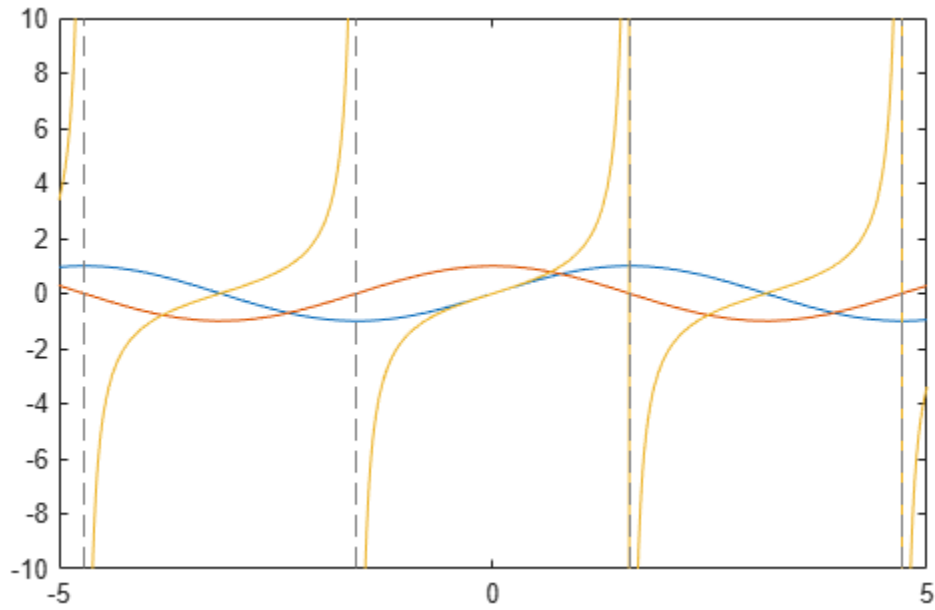
Plot the function  $\sin(\exp(x))$ .

```
syms x  
fplot(sin(exp(x)))
```



Plot the trigonometric functions  $\sin(x)$ ,  $\cos(x)$ , and  $\tan(x)$  simultaneously.

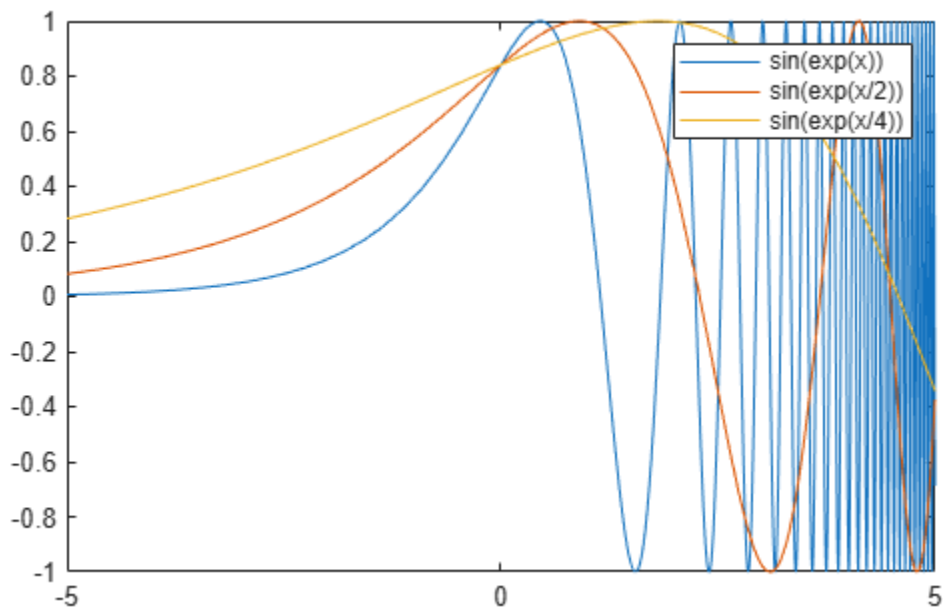
```
fplot([sin(x),cos(x),tan(x)])
```



**Plot a Function Defined by  $y = f(x, a)$  for Various Values of  $a$**

Plot the function  $\sin(\exp(x/a))$  for  $a = 1, 2$ , and  $4$ .

```
syms x a
expr = sin(exp(x/a));
fplot(subs(expr,a,[1,2,4]))
legend show
```

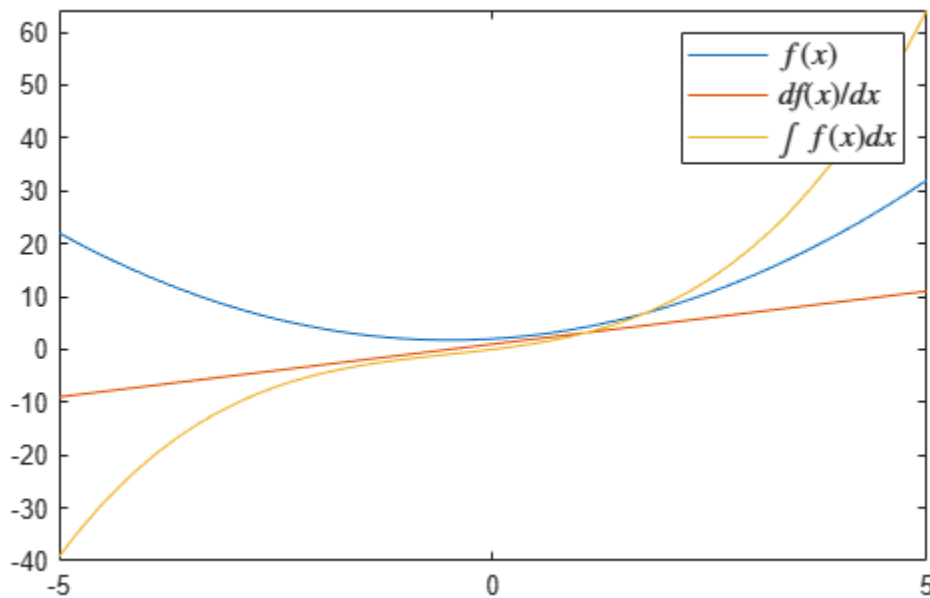




### Plot the Derivative and Integral of a Function

Plot a function  $f(x) = x(1 + x) + 2$ , its derivative  $df(x)/dx$ , and its integral  $\int f(x) dx$ .

```
syms f(x)
f(x) = x*(1 + x) + 2
f(x) = x(x+1)+2
f_diff = diff(f(x),x)
f_diff = 2x+1
f_int = int(f(x),x)
f_int =
    x(2x^2 + 3x + 12)
    -----
         6
fplot([f,f_diff,f_int])
legend({'$f(x)$','$df(x)/dx$','$\int f(x)dx$'}, 'Interpreter', 'latex', 'FontSize', 12)
```



### Plot a Function $y = g(x_0, a)$ with $a$ as the Horizontal Axis

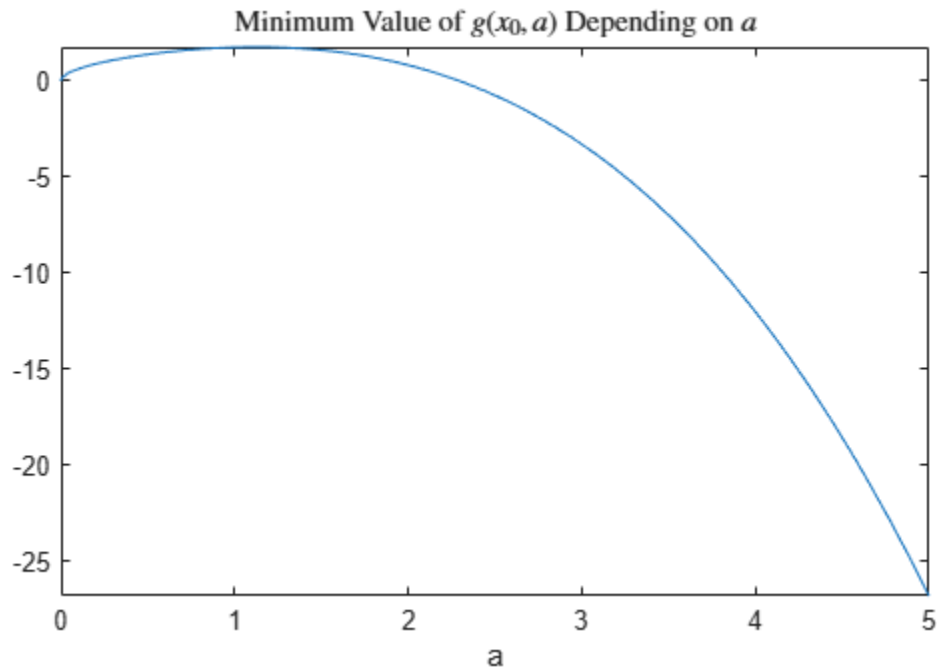
Find the  $x_0$  that minimizes a function  $g(x, a)$  by solving the differential equation  $dg(x, a)/dx = 0$ .

```
syms g(x,a);
assume(a>0);
g(x,a) = a*x*(a + x) + 2*sqrt(a)
g(x, a) = 2*sqrt(a) + a*x*(a + x)
x0 = solve(diff(g,x),x)
```

$$x_0 = -\frac{a}{2}$$

Plot the minimum value of  $g(x_0, a)$  for  $a$  from 0 to 5.

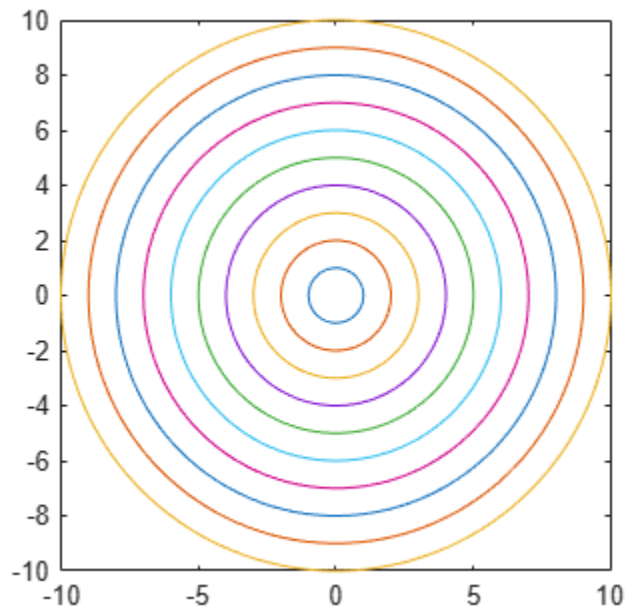
```
fplot(g(x0,a),[0 5])
xlabel('a')
title('Minimum Value of  $g(x_0,a)$  Depending on  $a$ ','interpreter','latex')
```



### Plot an Implicit Function $f(x, y) = c$ Using `fimplicit`

Plot circles defined by  $x^2 + y^2 = r^2$  with radius  $r$  as the integers from 1 to 10.

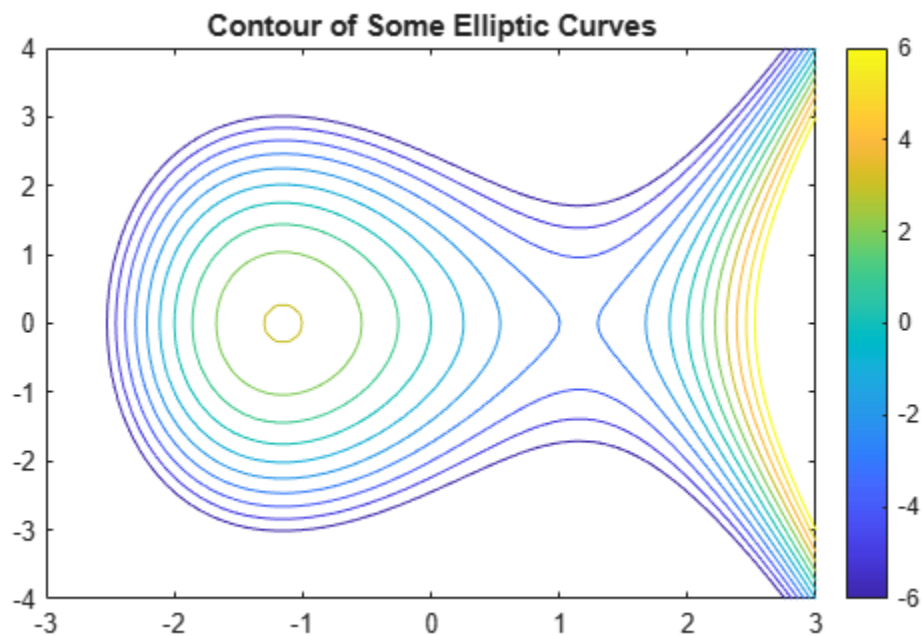
```
syms x y
r = 1:10;
fimplicit(x^2 + y^2 == r.^2,[-10 10])
axis square;
```



### Plot Contours of a Function $f(x, y)$ Using `fcontour`

Plot contours of the function  $f(x, y) = x^3 - 4x - y^2$  for contour levels from -6 to 6.

```
syms x y f(x,y)
f(x,y) = x^3 - 4*x - y^2;
fcontour(f, [-3 3 -4 4], 'LevelList', -6:6);
colorbar
title 'Contour of Some Elliptic Curves'
```



### Plot an Analytic Function and Its Approximation Using Spline Interpolant

Plot the analytic function  $f(x) = x \exp(-x) \sin(5x) - 2$ .

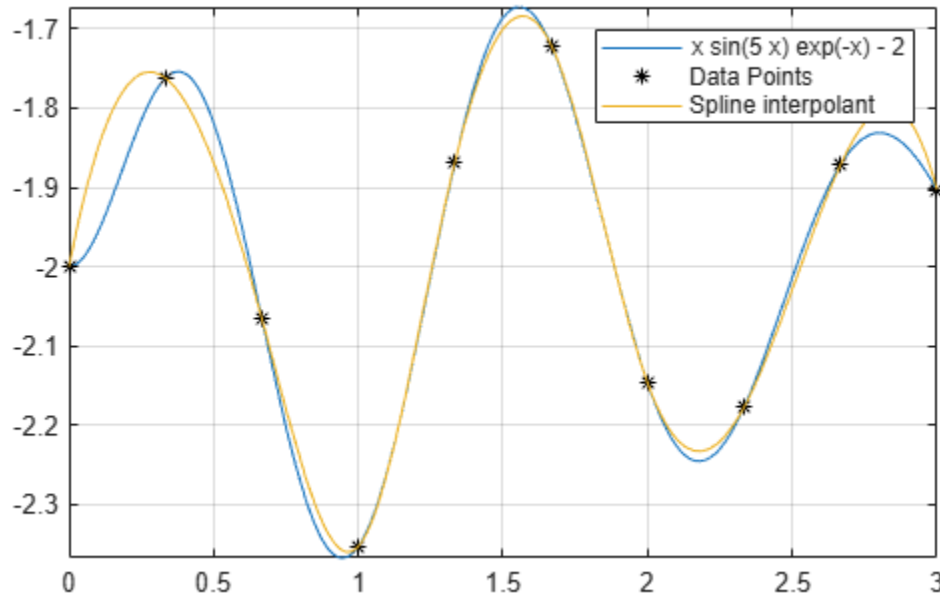
```
syms f(x)
f(x) = x*exp(-x)*sin(5*x) - 2;
fplot(f,[0,3])
```

Create a few data points from the analytic function.

```
xs = 0:1/3:3;
ys = double(subs(f,xs));
```

Plot the data points and the spline interpolant that approximates the analytic function.

```
hold on
plot(xs,ys,'*k','DisplayName','Data Points')
fplot(@(x) spline(xs,ys,x),[0 3],'DisplayName','Spline interpolant')
grid on
legend show
hold off
```



### Plot Taylor Approximations of a Function

Find the Taylor expansion of  $\cos(x)$  near  $x = 0$  up to 5th and 7th orders.

```
syms x
t5 = taylor(cos(x),x,'Order',5)
```

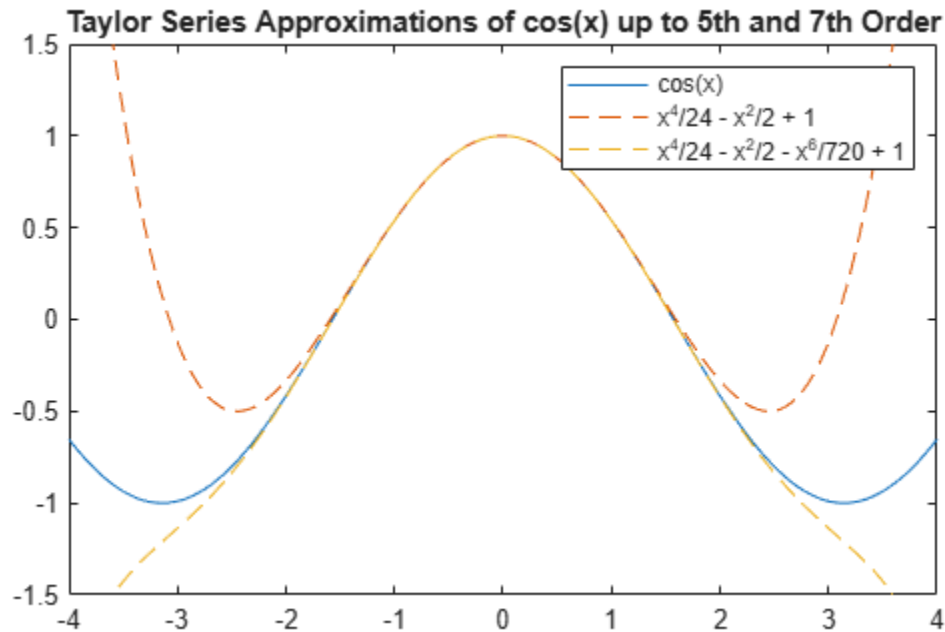
```
t5 =
      x4
    ---- - x2
     24    2
    + 1
```

```
t7 = taylor(cos(x),x,'Order',7)
```

$$t7 = -\frac{x^6}{720} + \frac{x^4}{24} - \frac{x^2}{2} + 1$$

Plot  $\cos(x)$  and its Taylor approximations.

```
fplot(cos(x))
hold on;
fplot([t5 t7], '--')
axis([-4 4 -1.5 1.5])
title('Taylor Series Approximations of cos(x) up to 5th and 7th Order')
legend show
hold off;
```



### Plot the Fourier Series Approximation of a Square Wave

A square wave of period  $2\pi$  and amplitude  $\pi/4$  can be approximated by the Fourier series expansion

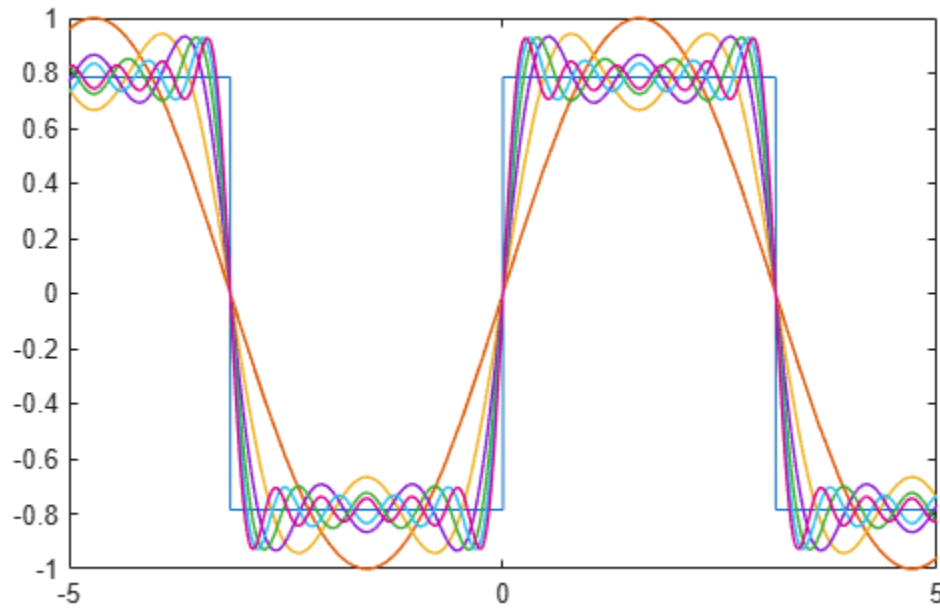
$$\sin(t) + \frac{1}{3}\sin(3t) + \frac{1}{5}\sin(5t) + \dots$$

Plot a square wave with period  $2\pi$  and amplitude  $\pi/4$ .

```
syms t y(t)
y(t) = piecewise(0 < mod(t,2*pi) <= pi, pi/4, pi < mod(t,2*pi) <= 2*pi, -pi/4);
fplot(y)
```

Plot the Fourier series approximation of the square wave.

```
hold on;
n = 6;
yFourier = cumsum(sin((1:2:2*n-1)*t)./(1:2:2*n-1));
fplot(yFourier, 'LineWidth', 1)
hold off
```

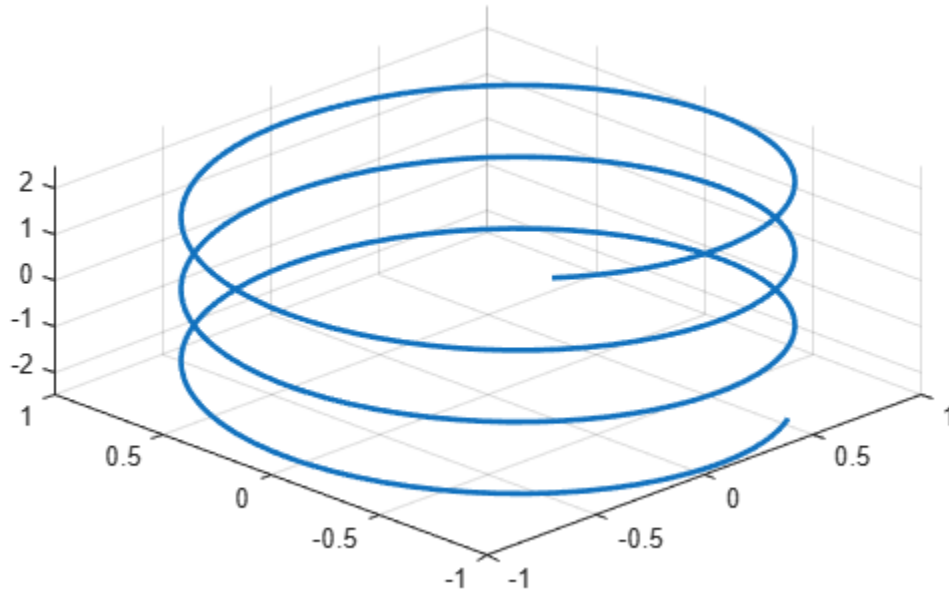


The Fourier series approximation overshoots at a jump discontinuity and the "ringing" does not die out as more terms are added to the approximation. This behavior is also known as the Gibbs phenomenon.

#### Plot a Parametric Curve $(x(t), y(t), z(t))$ Using `fplot3`

Plot a helix that is defined by  $(\sin(t), \cos(t), t/4)$  for  $t$  from -10 to 10.

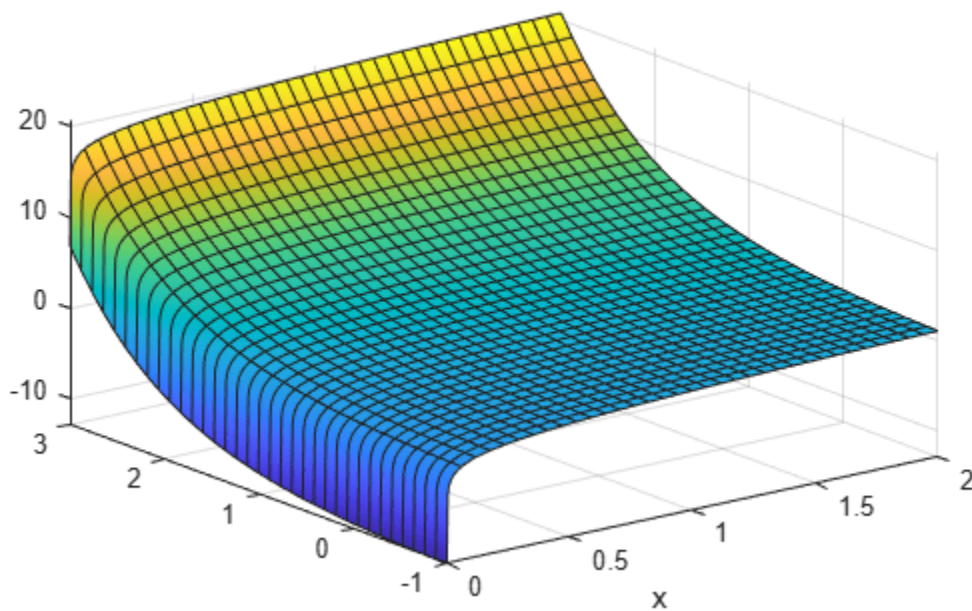
```
syms t
fplot3(sin(t),cos(t),t/4,[-10 10],'LineWidth',2)
view([-45 45])
```



### Plot a Surface Defined by $z = f(x, y)$ Using `fsurf`

Plot a surface defined by  $\log(x) + \exp(y)$ . Analytical plotting using `fsurf` (without generating numerical data) shows the curved areas and asymptotic regions near  $x = 0$ .

```
syms x y
fsurf(log(x) + exp(y), [0 2 -1 3])
xlabel('x')
```



**Plot a Multivariate Surface ( $x(u, v)$ ,  $y(u, v)$ ,  $z(u, v)$ ) Using fsurf**

Plot a multivariate surface defined by

$$x(u, v) = u$$

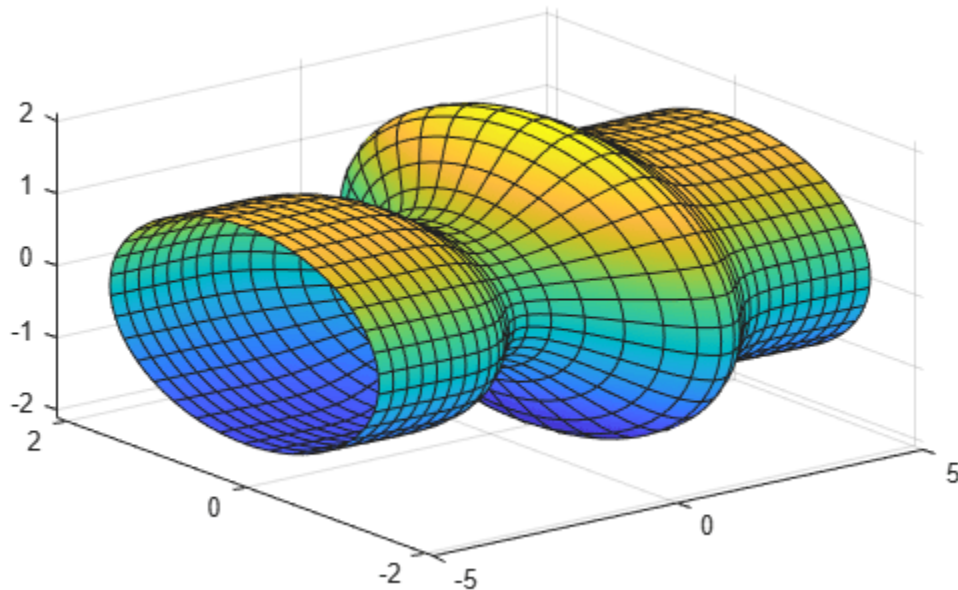
$$y(u, v) = f(u) \sin(v)$$

$$z(u, v) = f(u) \cos(v)$$

where  $f(u) = \exp(-u^2/3)\sin(u) + 3/2$ .

Set the plot interval of  $u$  from -5 to 5 and  $v$  from 0 to  $2\pi$ .

```
syms f(u) x(u,v) y(u,v) z(u,v)
f(u) = sin(u)*exp(-u^2/3)+1.5;
x(u,v) = u;
y(u,v) = f(u)*sin(v);
z(u,v) = f(u)*cos(v);
fsurf(x,y,z,[-5 5 0 2*pi])
```

**Plot a Multivariate Surface ( $x(s, t)$ ,  $y(s, t)$ ,  $z(s, t)$ ) Using fmesh**

Plot a multivariate surface defined by

$$x = r \cos(s) \sin(t)$$

$$y = r \sin(s) \sin(t)$$

$$z = r \cos(t)$$

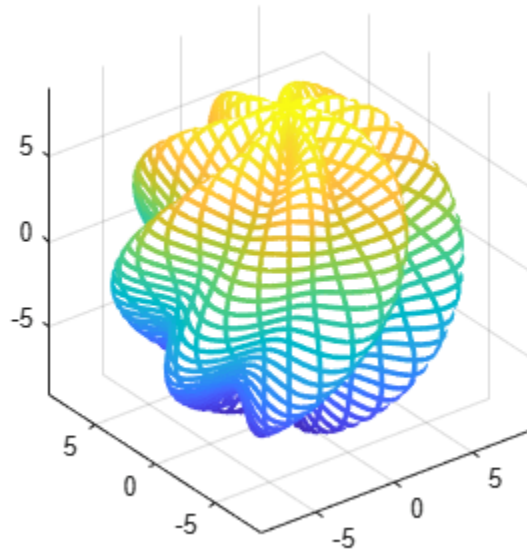
where  $r = 8 + \sin(7s + 5t)$ . Show the plotted surface as meshes by using fmesh. Set the plot interval of  $s$  from 0 to  $2\pi$  and  $t$  from 0 to  $\pi$ .



```

syms s t
r = 8 + sin(7*s + 5*t);
x = r*cos(s)*sin(t);
y = r*sin(s)*sin(t);
z = r*cos(t);
fmesh(x,y,z,[0 2*pi 0 pi], 'Linewidth',2)
axis equal

```



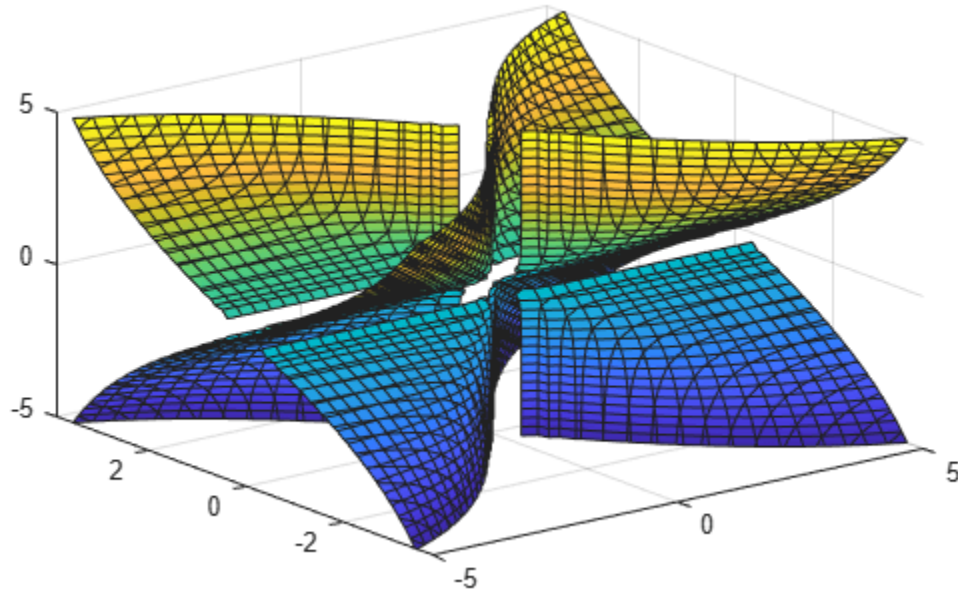
### Plot an Implicit Surface $f(x, y, z) = c$ Using `fimplicit3`

Plot the implicit surface  $1/x^2 - 1/y^2 + 1/z^2 = 0$ .

```

syms x y z
f = 1/x^2 - 1/y^2 + 1/z^2;
fimplicit3(f)

```



### Plot the Contours and Gradient of a Surface

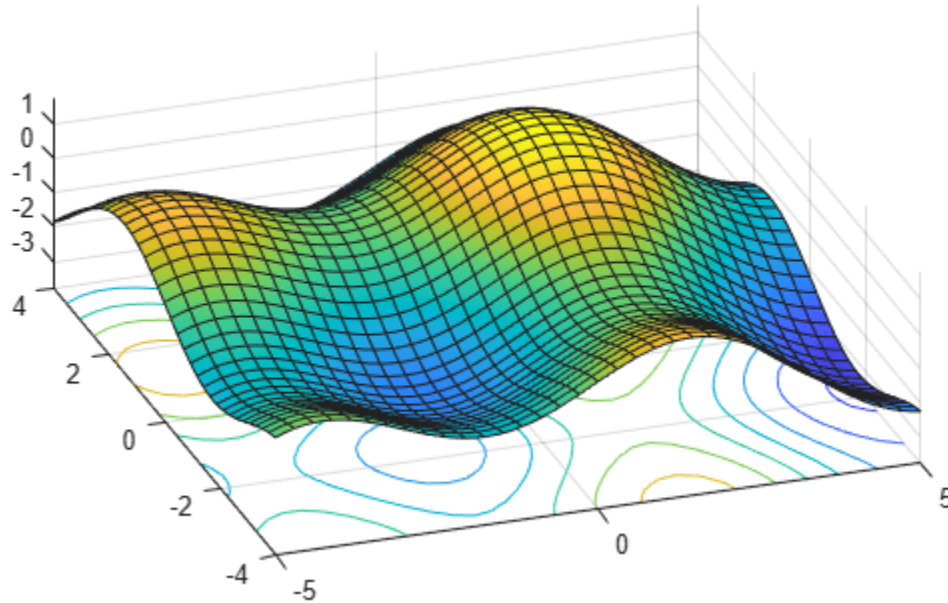
Plot the surface  $\sin(x) + \sin(y) - (x^2 + y^2)/20$  using `fsurf`. You can show the contours on the same graph by setting `'ShowContours'` to `'on'`.

```
syms x y
f = sin(x)+sin(y)-(x^2+y^2)/20
```

```
f =
```

$$\sin(x) + \sin(y) - \frac{x^2}{20} - \frac{y^2}{20}$$

```
fsurf(f,'ShowContours','on')
view(-19,56)
```



Next, plot the contours on a separate graph with finer contour lines.

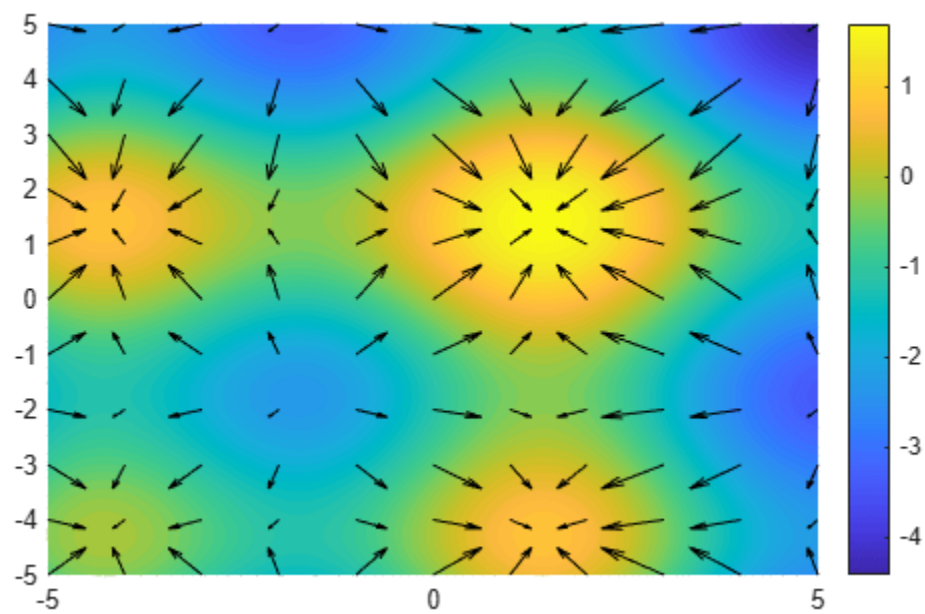
```
fcontour(f,[-5 5 -5 5], 'LevelStep',0.1, 'Fill','on')
colorbar
```

Find the gradient of the surface. Create 2-D grids using `meshgrid` and substitute the grid coordinates to evaluate the gradient numerically. Show the gradient using `quiver`.

```
hold on
Fgrad = gradient(f,[x,y])
```

```
Fgrad =
    (
    cos(x) -  $\frac{x}{10}$ 
    cos(y) -  $\frac{y}{10}$ 
    )
```

```
[xgrid,ygrid] = meshgrid(-5:5,-5:5);
Fx = subs(Fgrad(1),{x,y},{xgrid,ygrid});
Fy = subs(Fgrad(2),{x,y},{xgrid,ygrid});
quiver(xgrid,ygrid,Fx,Fy,'k')
hold off
```



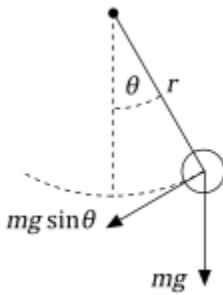
## See Also

### Functions

`fplot` | `fimplicit` | `fcontour` | `fplot3` | `fsurf` | `fmesh` | `fimplicit3`

## Simulate the Motion of the Periodic Swing of a Pendulum

This example shows how to simulate the motion of a simple pendulum using Symbolic Math Toolbox™. Derive the equation of motion of the pendulum, then solve the equation analytically for small angles and numerically for any angle.



### Step 1: Derive the Equation of Motion

The pendulum is a simple mechanical system that follows a differential equation. The pendulum is initially at rest in a vertical position. When the pendulum is displaced by an angle  $\theta$  and released, the force of gravity pulls it back towards its resting position. Its momentum causes it to overshoot and come to an angle  $-\theta$  (if there are no frictional forces), and so on. The restoring force along the motion of the pendulum due to gravity is  $-mg\sin\theta$ . Thus, according to Newton's second law, the mass times the acceleration must equal  $-mg\sin\theta$ .

```
syms m a g theta(t)
eqn = m*a == -m*g*sin(theta)

eqn(t) = a m = -g m sin(theta(t))
```

For a pendulum with length  $r$ , the acceleration of the pendulum bob is equal to the angular acceleration times  $r$ .

$$a = r \frac{d^2\theta}{dt^2}.$$

Substitute for  $a$  by using `subs`.

```
syms r
eqn = subs(eqn,a,r*diff(theta,2))

eqn(t) =
    m r \frac{\partial^2}{\partial t^2} \theta(t) = -g m \sin(\theta(t))
```

Isolate the angular acceleration in `eqn` by using `isolate`.

```
eqn = isolate(eqn,diff(theta,2))

eqn =
    \frac{\partial^2}{\partial t^2} \theta(t) = -\frac{g \sin(\theta(t))}{r}
```

Collect the constants  $g$  and  $r$  into a single parameter, which is also known as the *natural frequency*.

$$\omega_0 = \sqrt{\frac{g}{r}}.$$

```
syms omega_0
eqn = subs(eqn,g/r,omega_0^2)

eqn =

$$\frac{\partial^2}{\partial t^2} \theta(t) = -\omega_0^2 \sin(\theta(t))$$

```

### Step 2: Linearize the Equation of Motion

The equation of motion is nonlinear, so it is difficult to solve analytically. Assume the angles are small and linearize the equation by using the Taylor expansion of  $\sin\theta$ .

```
syms x
approx = taylor(sin(x),x,Order=2);
approx = subs(approx,x,theta(t))

approx = theta(t)
```

The equation of motion becomes a linear equation.

```
eqnLinear = subs(eqn,sin(theta(t)),approx)

eqnLinear =

$$\frac{\partial^2}{\partial t^2} \theta(t) = -\omega_0^2 \theta(t)$$

```

### Step 3: Solve Equation of Motion Analytically

Solve the equation `eqnLinear` by using `dsolve`. Specify initial conditions as the second argument. Simplify the solution by assuming  $\omega_0$  is real using `assume`.

```
syms theta_0 theta_t0
theta_t = diff(theta);
cond = [theta(0) == theta_0, theta_t(0) == theta_t0];
assume(omega_0,"real")
thetaSol(t) = dsolve(eqnLinear,cond)

thetaSol(t) =

$$\theta_0 \cos(\omega_0 t) + \frac{\theta_{t0} \sin(\omega_0 t)}{\omega_0}$$

```

### Step 4: Physical Significance of $\omega_0$

The term  $\omega_0 t$  is called the *phase*. The cosine and sine functions repeat every  $2\pi$ . The time needed to change  $\omega_0 t$  by  $2\pi$  is called the time period.

$$T = \frac{2\pi}{\omega_0} = 2\pi\sqrt{\frac{r}{g}}.$$

The time period  $T$  is proportional to the square root of the length of the pendulum and it does not depend on the mass. For linear equation of motion, the time period does not depend on the initial conditions.

**Step 5: Plot Pendulum Motion**

Plot the motion of the pendulum for small-angle approximation.

Define the physical parameters:

- Gravitational acceleration  $g = 9.81 \text{ m/s}^2$
- Length of pendulum  $r = 1 \text{ m}$

```
gValue = 9.81;
rValue = 1;
omega_0Value = sqrt(gValue/rValue);
T = 2*pi/omega_0Value;
```

Set initial conditions.

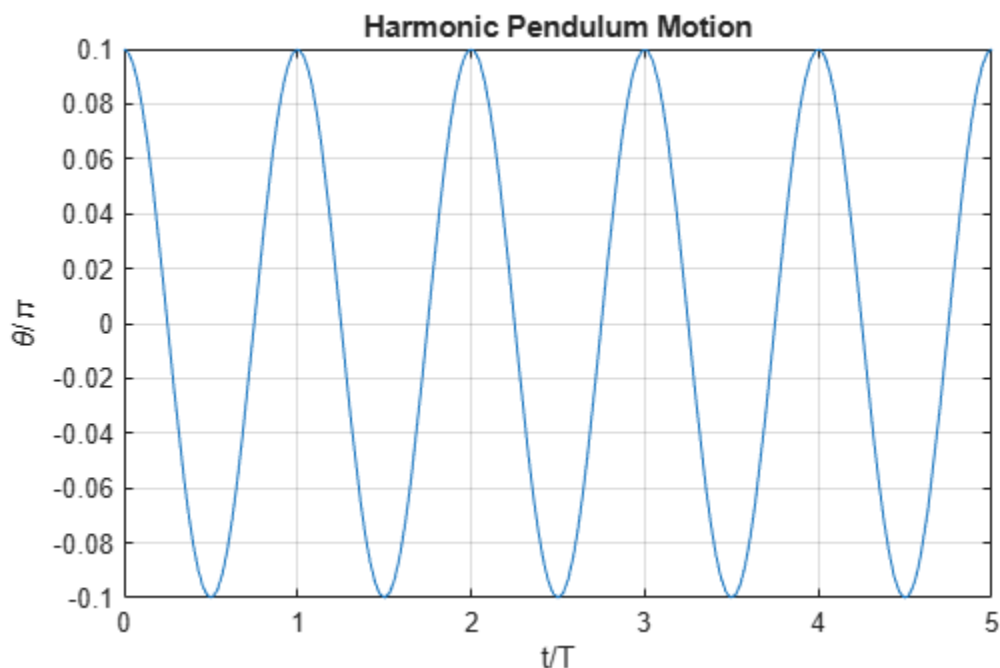
```
theta_0Value = 0.1*pi; % Solution only valid for small angles.
theta_t0Value = 0;      % Initially at rest.
```

Substitute the physical parameters and initial conditions into the general solution.

```
vars = [omega_0    theta_0    theta_t0];
values = [omega_0Value theta_0Value theta_t0Value];
thetaSolPlot = subs(thetaSol,vars,values);
```

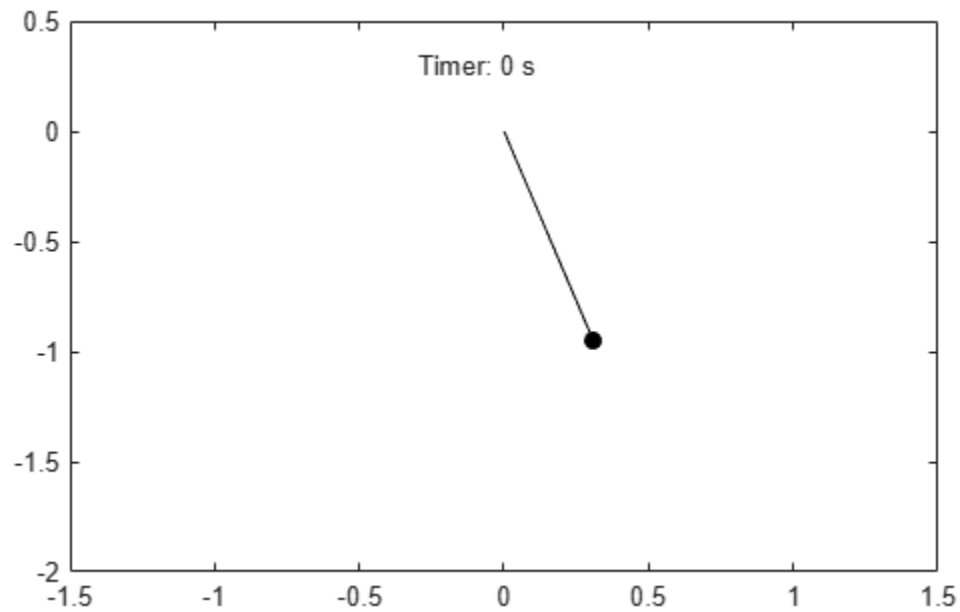
Plot the harmonic pendulum motion.

```
fplot(thetaSolPlot(t*T)/pi,[0 5]);
grid on;
title("Harmonic Pendulum Motion");
xlabel("t/T");
ylabel("\theta/\pi");
```



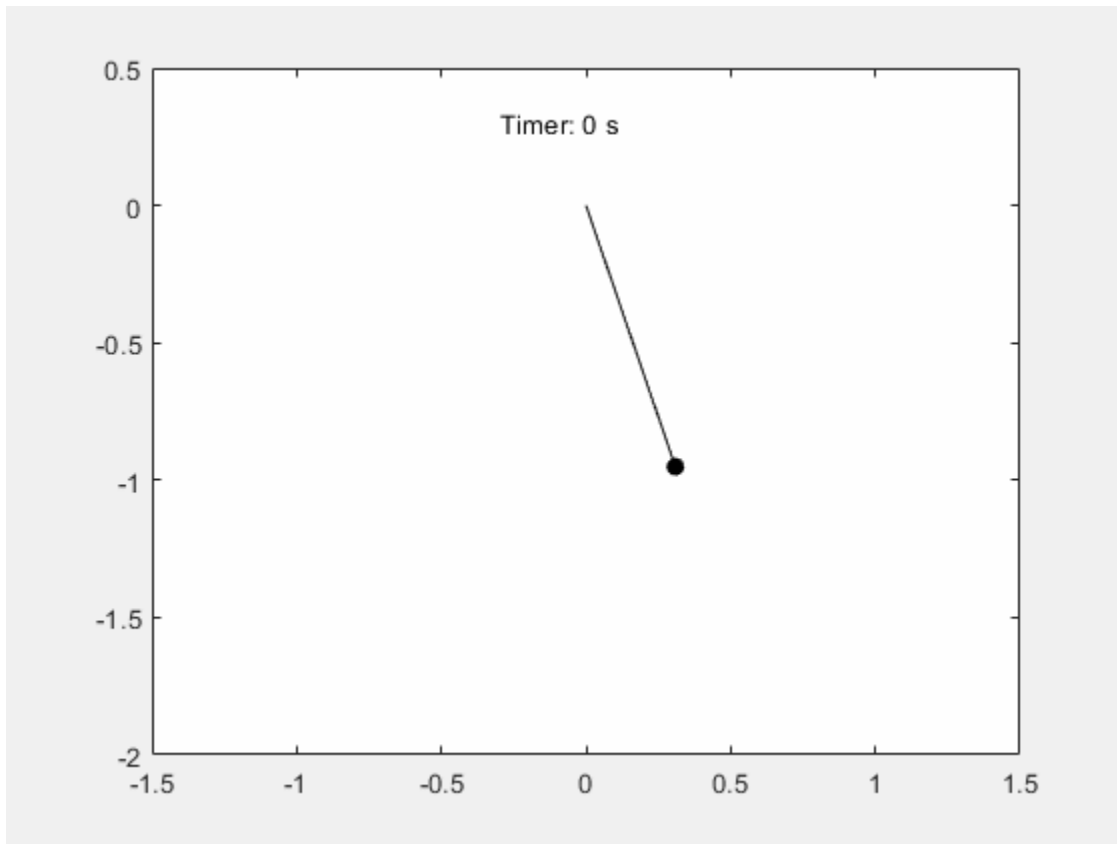
After finding the solution for  $\theta(t)$ , visualize the motion of the pendulum.

```
x_pos = sin(thetaSolPlot);  
y_pos = -cos(thetaSolPlot);  
fanimator(@fplot,x_pos,y_pos,"ko",MarkerFaceColor="k",AnimationRange=[0 5*T]);  
hold on;  
fanimator(@(t) plot([0 x_pos(t)],[0 y_pos(t)],"k-"),AnimationRange=[0 5*T]);  
fanimator(@(t) text(-0.3,0.3,"Timer: "+num2str(t,2)+" s"),AnimationRange=[0 5*T]);
```



Enter the command `playAnimation` to play the animation of the pendulum motion.





### Step 6: Determine Nonlinear Pendulum Motion Using Constant Energy Paths

To understand the nonlinear motion of the pendulum, visualize the pendulum path by using the equation for total energy. The total energy is conserved.

$$E = \frac{1}{2}mr^2\left(\frac{d\theta}{dt}\right)^2 + m g r(1 - \cos \theta)$$

Use the trigonometric identity  $1 - \cos\theta = 2\sin^2(\theta/2)$  and the relation  $\omega_0 = \sqrt{g/r}$  to rewrite the scaled energy.

$$\frac{E}{mr^2} = \frac{1}{2}\left[\left(\frac{d\theta}{dt}\right)^2 + \left(2\omega_0 \sin\frac{\theta}{2}\right)^2\right]$$

Since energy is conserved, the motion of the pendulum can be described by constant energy paths in the phase space. The phase space is an abstract space with the coordinates  $\theta$  and  $d\theta/dt$ . Visualize these paths using `fcontour`.

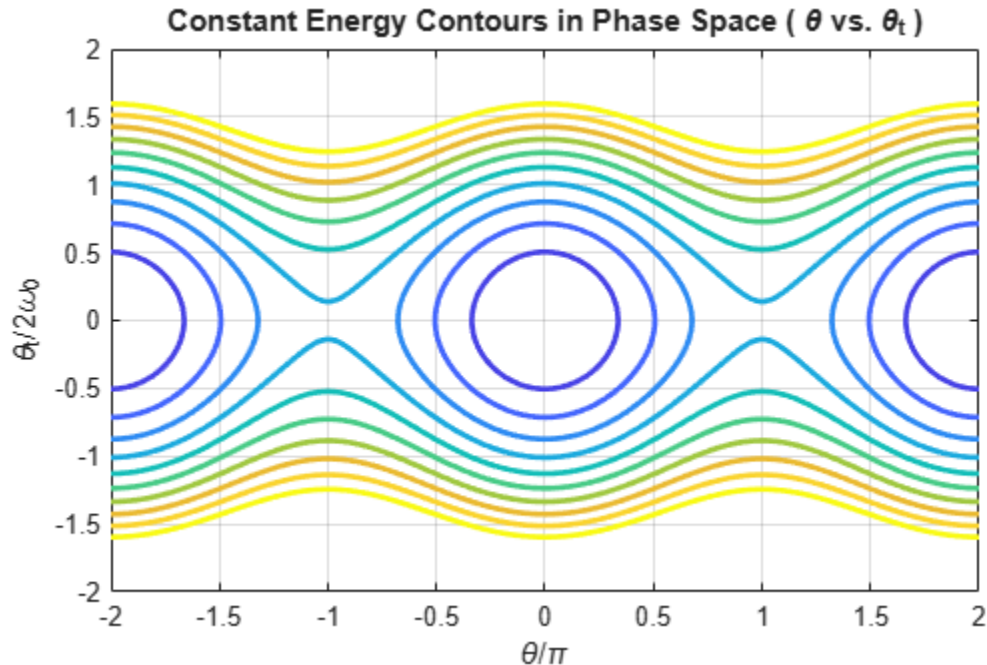
```
syms theta theta_t omega_0
E(theta, theta_t, omega_0) = (1/2)*(theta_t^2+(2*omega_0*sin(theta/2))^2);
Eplot(theta, theta_t) = subs(E,omega_0,omega_0Value);

figure;
fc = fcontour(Eplot(pi*theta, 2*omega_0Value*theta_t), 2*[-1 1 -1 1], ...
    LineWidth=2, LevelList=0:5:50, MeshDensity=1+2^8);
grid on;
```

```

title("Constant Energy Contours in Phase Space ( \theta vs. \theta_t )");
xlabel("\theta/\pi");
ylabel("\theta_t/2\omega_0");

```



The constant energy contours are symmetric about the  $\theta$  axis and  $d\theta/dt$  axis, and are periodic along the  $\theta$  axis. The figure shows two regions of distinct behavior.

The lower energies of the contour plot close upon themselves. The pendulum swings back and forth between two maximum angles and velocities. The kinetic energy of the pendulum is not enough to overcome gravitational energy and enable the pendulum to make a full loop.



The higher energies of the contour plot do not close upon themselves. The pendulum always moves in one angular direction. The kinetic energy of the pendulum is enough to overcome gravitational energy and enable the pendulum to make a full loop.



### Step 7: Solve Nonlinear Equations of Motion

The nonlinear equations of motion are second-order differential equations. Numerically solve these equations by using the `ode45` solver. Because `ode45` accepts only first-order systems, reduce the system to a first-order system. Then, generate function handles that are the input to `ode45`.

Rewrite the second-order ODE as a system of first-order ODEs.

```
syms theta(t) theta_t(t) omega_0
eqs = [diff(theta) == theta_t;
       diff(theta_t) == -omega_0^2*sin(theta)]
```

$$\text{eqs}(t) = \begin{pmatrix} \frac{\partial}{\partial t} \theta(t) = \theta_t(t) \\ \frac{\partial}{\partial t} \theta_t(t) = -\omega_0^2 \sin(\theta(t)) \end{pmatrix}$$

```
eqs = subs(eqs,omega_0,omega_0Value);
vars = [theta, theta_t];
```

Find the mass matrix  $M$  of the system and the right sides of the equations  $F$ .

```
[M,F] = massMatrixForm(eqs,vars)
```

$$M = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

$$F = \begin{pmatrix} \theta_t(t) \\ -\frac{981 \sin(\theta(t))}{100} \end{pmatrix}$$

$M$  and  $F$  refer to this form.

$$M(t, x(t)) \frac{dx}{dt} = F(t, x(t)).$$

To simplify further computations, rewrite the system in the form  $dx/dt = f(t, x(t))$ .

```
f = M\F
```

$$f = \begin{pmatrix} \theta_t(t) \\ -\frac{981 \sin(\theta(t))}{100} \end{pmatrix}$$

Convert `f` to a MATLAB function handle by using `odeFunction`. The resulting function handle is the input to the MATLAB ODE solver `ode45`.

```
f = odeFunction(f,vars)
```

```
f = function_handle with value:
    @(t,in2)[in2(2,:);sin(in2(1,:)).*(-9.81e+2./1.0e+2)]
```

### Step 8: Solve Equation of Motion for Closed Energy Contours

Solve the ODE for the closed energy contours by using `ode45`.

From the energy contour plot, closed contours satisfy the condition  $\theta_0 = 0$ ,  $\theta_{t0}/2\omega_0 \leq 1$ . Store the initial conditions of  $\theta$  and  $d\theta/dt$  in the variable `x0`.

```
x0 = [0; 1.99*omega_0Value];
```

Specify a time interval from 0 s to 10 s for finding the solution. This interval allows the pendulum to go through two full periods.

```
tInit = 0;
tFinal = 10;
```

Solve the ODE.

```
sols = ode45(f,[tInit tFinal],x0)
```

```
sols = struct with fields:
    solver: 'ode45'
    extdata: [1x1 struct]
        x: [0 3.2241e-05 1.9344e-04 9.9946e-04 0.0050 0.0252 0.1259 0.3449 0.6020 0.8591 1.116]
        y: [2x45 double]
    stats: [1x1 struct]
    idata: [1x1 struct]
```

`sols.y(1,:)` represents the angular displacement  $\theta$  and `sols.y(2,:)` represents the angular velocity  $d\theta/dt$ .

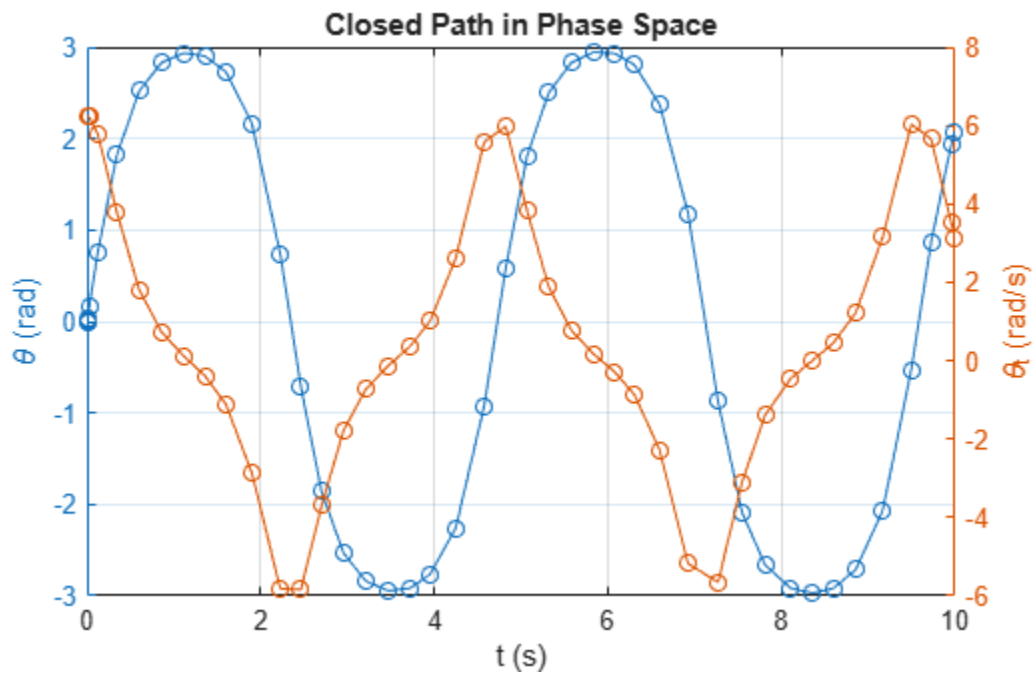
Plot the closed-path solution.

```
figure;

yyaxis left;
plot(sols.x,sols.y(1,:),"-o");
ylabel("\theta (rad)");

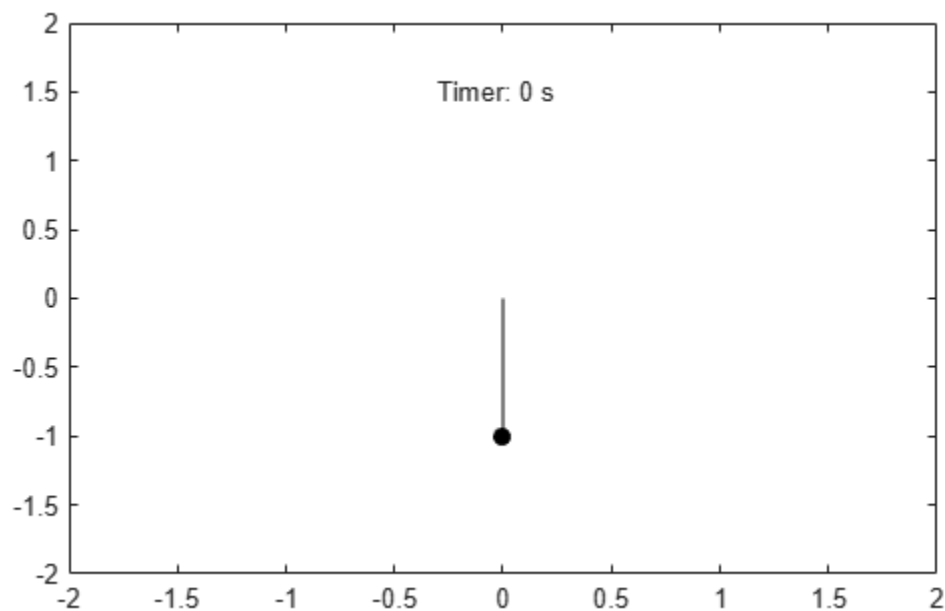
yyaxis right;
plot(sols.x,sols.y(2,:),"-o");
ylabel("\theta_t (rad/s)");

grid on;
title("Closed Path in Phase Space");
xlabel("t (s)");
```

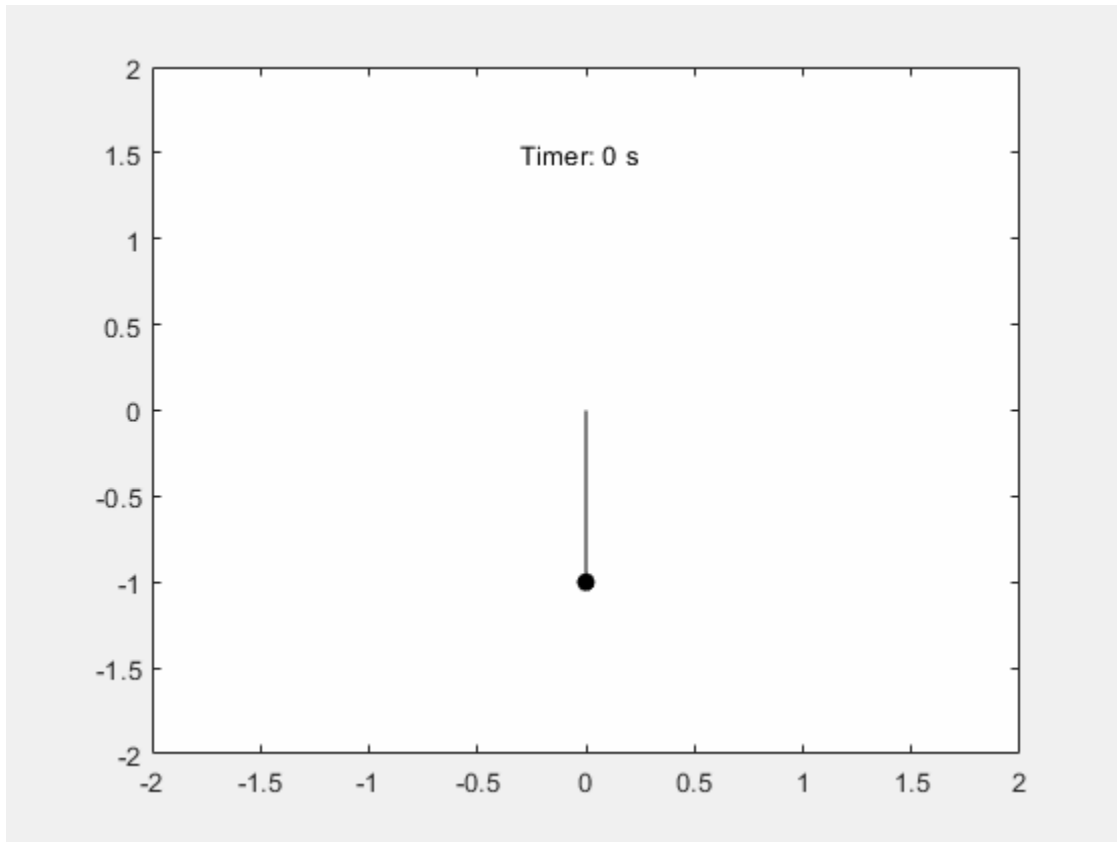


Visualize the motion of the pendulum.

```
x_pos = @(t) sin(deval(sols,t,1));
y_pos = @(t) -cos(deval(sols,t,1));
figure;
fanimator(@(t) plot(x_pos(t),y_pos(t),"ko",MarkerFaceColor="k"));
hold on;
fanimator(@(t) plot([0 x_pos(t)], [0 y_pos(t)], "k-"));
fanimator(@(t) text(-0.3,1.5, "Timer: "+num2str(t,2)+" s"));
```



Enter the command `playAnimation` to play the animation of the pendulum motion.



### Step 9: Solutions on Open Energy Contours

Solve the ODE for the open energy contours by using `ode45`. From the energy contour plot, open contours satisfy the condition  $\theta_0 = 0$ ,  $\theta_{t0}/2\omega_0 > 1$ .

```
x0 = [0; 2.01*omega_0Value];
sols = ode45(f,[tInit, tFinal],x0);
```

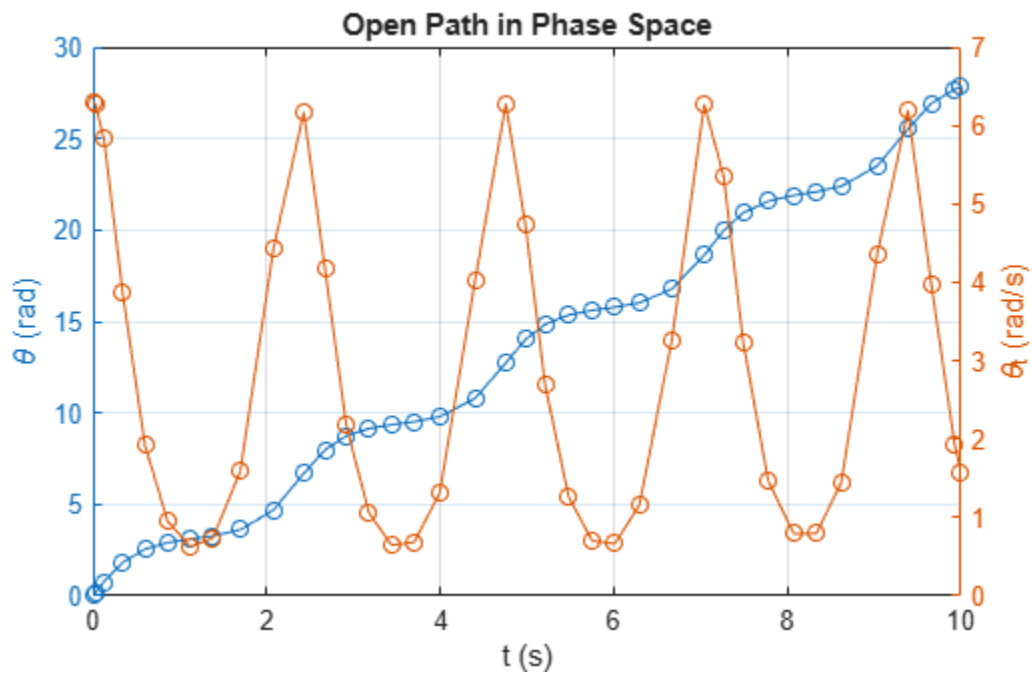
Plot the solution for the open energy contour.

```
figure;

yyaxis left;
plot(sols.x,sols.y(1,:),"-o");
ylabel("\theta (rad)");

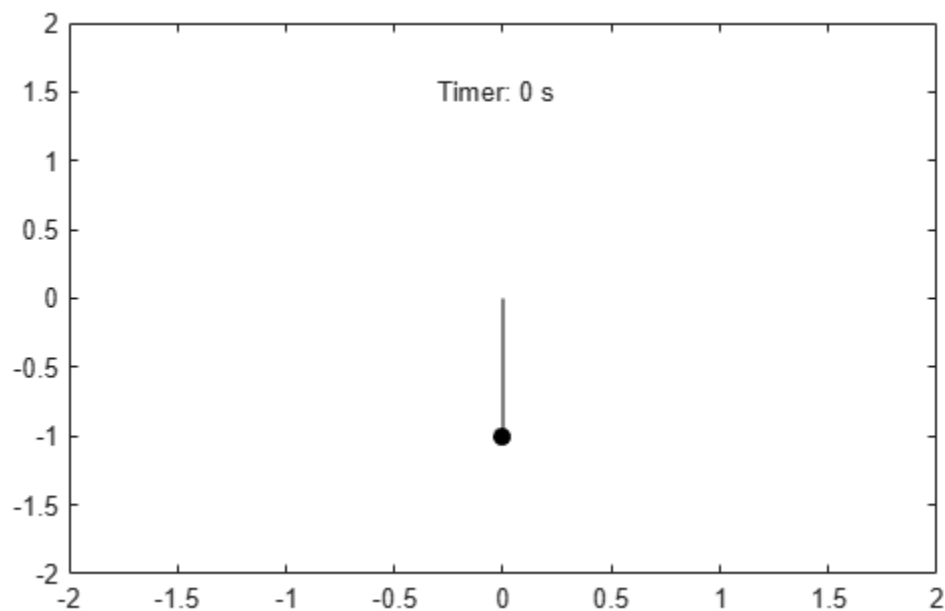
yyaxis right;
plot(sols.x,sols.y(2,:),"-o");
ylabel("\theta_t (rad/s)");

grid on;
title("Open Path in Phase Space");
xlabel("t (s)");
```

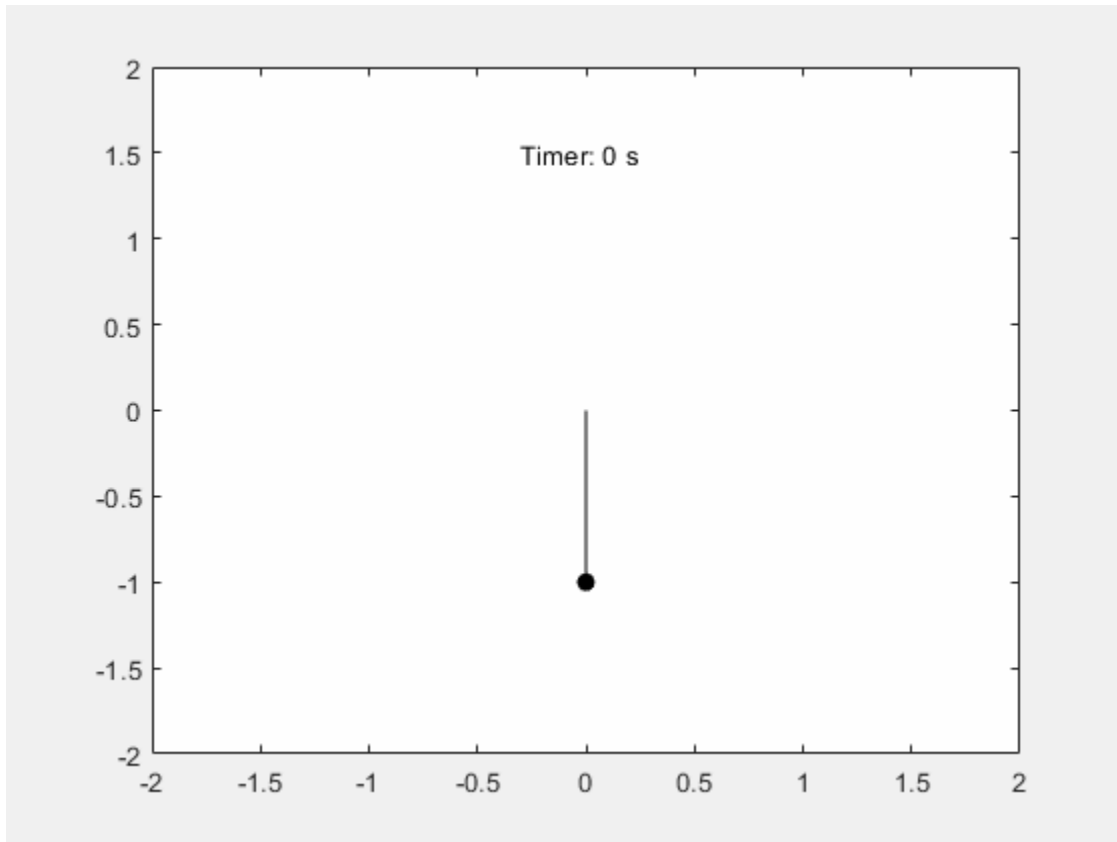


Visualize the motion of the pendulum.

```
x_pos = @(t) sin(deval(sols,t,1));
y_pos = @(t) -cos(deval(sols,t,1));
figure;
fanimator(@(t) plot(x_pos(t),y_pos(t),"ko",MarkerFaceColor="k"));
hold on;
fanimator(@(t) plot([0 x_pos(t)],[0 y_pos(t)],"k-"));
fanimator(@(t) text(-0.3,1.5,"Timer: "+num2str(t,2)+" s"));
```



Enter the command `playAnimation` to play the animation of the pendulum motion.



## See Also

### Functions

`diff` | `subs` | `dsolve` | `isolate` | `fplot` | `fanimator` | `playAnimation`

## Related Examples

- “Animation and Solution of Double Pendulum Motion” on page 4-47
- “Derive Equations of Motion and Simulate Cart-Pole System” on page 3-281



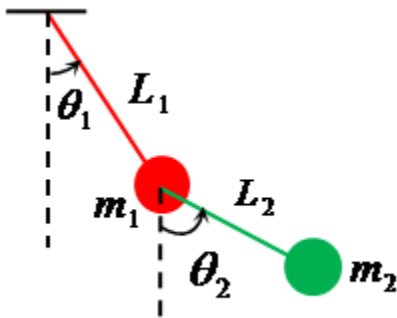
## Animation and Solution of Double Pendulum Motion

This example shows how to model the motion of a double pendulum by using MATLAB® and Symbolic Math Toolbox™.

Solve the motion equations of a double pendulum and create an animation to model the double pendulum motion.

### Step 1: Define Displacement, Velocity, and Acceleration of Double Pendulum Masses

The following figure shows the model of a double pendulum. The double pendulum consists of two pendulum bobs and two rigid rods.



Describe the motion of the double pendulum by defining the state variables:

- the angular position of the first bob  $\theta_1(t)$
- the angular position of the second bob  $\theta_2(t)$

Describe the properties of the double pendulum by defining the variables:

- the length of the first rod  $L_1$
- the length of the second rod  $L_2$
- the mass of the first bob  $m_1$
- the mass of the second bob  $m_2$
- the gravitational constant  $g$

For simplicity, ignore the masses of the two rigid rods. Specify all variables by using `syms`.

```
syms theta_1(t) theta_2(t) L_1 L_2 m_1 m_2 g
```

Define the displacements of the double pendulum in Cartesian coordinates.

```
x_1 = L_1*sin(theta_1);
y_1 = -L_1*cos(theta_1);
x_2 = x_1 + L_2*sin(theta_2);
y_2 = y_1 - L_2*cos(theta_2);
```

Find the velocities by differentiating the displacements with respect to time using the `diff` function.

```
vx_1 = diff(x_1);
vy_1 = diff(y_1);
```

```
vx_2 = diff(x_2);
vy_2 = diff(y_2);
```

Find the accelerations by differentiating the velocities with respect to time.

```
ax_1 = diff(vx_1);
ay_1 = diff(vy_1);
ax_2 = diff(vx_2);
ay_2 = diff(vy_2);
```

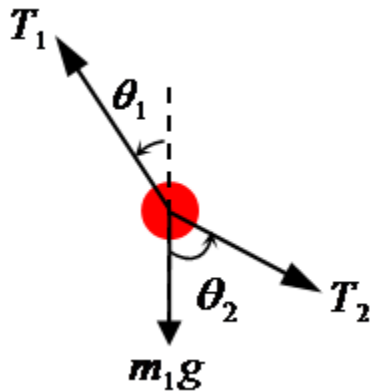
### Step 2: Define Equations of Motion

Define the equations of motion based on Newton's laws.

First, specify the tension of the first rod as  $T_1$ , and the tension of the second rod  $T_2$ .

```
syms T_1 T_2
```

Next, construct free-body diagrams of the forces that act on both masses.



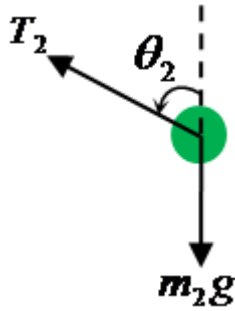
Evaluate the forces acting on  $m_1$ . Define the equations of motion of the first bob by balancing the horizontal and vertical force components. Specify these two equations as symbolic equations `eqx_1` and `eqy_1`.

```
eqx_1 = m_1*ax_1(t) == -T_1*sin(theta_1(t)) + T_2*sin(theta_2(t))
```

```
eqx_1 =
    -m_1 \left( L_1 \sin(\theta_1(t)) \left( \frac{\partial}{\partial t} \theta_1(t) \right)^2 - L_1 \cos(\theta_1(t)) \frac{\partial^2}{\partial t^2} \theta_1(t) \right) = T_2 \sin(\theta_2(t)) - T_1 \sin(\theta_1(t))
```

```
eqy_1 = m_1*ay_1(t) == T_1*cos(theta_1(t)) - T_2*cos(theta_2(t)) - m_1*g
```

```
eqy_1 =
    m_1 \left( L_1 \sin(\theta_1(t)) \frac{\partial^2}{\partial t^2} \theta_1(t) + L_1 \cos(\theta_1(t)) \left( \frac{\partial}{\partial t} \theta_1(t) \right)^2 \right) = T_1 \cos(\theta_1(t)) - g m_1 - T_2 \cos(\theta_2(t))
```



Evaluate the forces acting on  $m_2$ . Define the equations of motion of the second bob by balancing the horizontal and vertical force components. Specify these two equations as symbolic equations `eqx_2` and `eqy_2`.

$$\text{eqx\_2} = m_2 \cdot a_{x\_2}(t) == -T_2 \cdot \sin(\theta_2(t))$$

$$\begin{aligned} \text{eqx\_2} = & \\ & -m_2 \left( L_1 \sin(\theta_1(t)) \left( \frac{\partial}{\partial t} \theta_1(t) \right)^2 + L_2 \sin(\theta_2(t)) \left( \frac{\partial}{\partial t} \theta_2(t) \right)^2 - L_1 \cos(\theta_1(t)) \frac{\partial^2}{\partial t^2} \theta_1(t) - L_2 \cos(\theta_2(t)) \frac{\partial^2}{\partial t^2} \theta_2(t) \right) \\ & = -T_2 \sin(\theta_2(t)) \end{aligned}$$

$$\text{eqy\_2} = m_2 \cdot a_{y\_2}(t) == T_2 \cdot \cos(\theta_2(t)) - m_2 \cdot g$$

$$\begin{aligned} \text{eqy\_2} = & \\ & m_2 \left( L_1 \cos(\theta_1(t)) \left( \frac{\partial}{\partial t} \theta_1(t) \right)^2 + L_2 \cos(\theta_2(t)) \left( \frac{\partial}{\partial t} \theta_2(t) \right)^2 + L_1 \sin(\theta_1(t)) \frac{\partial^2}{\partial t^2} \theta_1(t) + L_2 \sin(\theta_2(t)) \frac{\partial^2}{\partial t^2} \theta_2(t) \right) \\ & = T_2 \cos(\theta_2(t)) - g m_2 \end{aligned}$$

### Step 3: Evaluate Forces and Reduce System Equations

Four equations of motion describe the kinematics of the double pendulum. Evaluate the forces acting on the rods and reduce the set of four equations to two equations.

The equations of motion have four unknowns:  $\theta_1$ ,  $\theta_2$ ,  $T_1$ , and  $T_2$ . Evaluate the two unknowns  $T_1$  and  $T_2$  from `eqx_1` and `eqy_1`. Use `solve` function to find  $T_1$  and  $T_2$ .

$$\text{Tension} = \text{solve}([\text{eqx\_1} \text{ eqy\_1}], [T_1 \ T_2]);$$

Substitute the solutions for  $T_1$  and  $T_2$  into `eqx_2` and `eqy_2`.

$$\begin{aligned} \text{eqRed\_1} &= \text{subs}(\text{eqx\_2}, [T_1 \ T_2], [\text{Tension.T\_1} \ \text{Tension.T\_2}]); \\ \text{eqRed\_2} &= \text{subs}(\text{eqy\_2}, [T_1 \ T_2], [\text{Tension.T\_1} \ \text{Tension.T\_2}]); \end{aligned}$$

The two reduced equations fully describe the pendulum motion.

### Step 4: Solve System Equations

Solve the system equations to describe the pendulum motion.

First, define the values for the masses in kg, the rod lengths in m, and the gravity in  $\text{m/s}^2$  (SI units). Substitute these values into the two reduced equations.

```

L_1 = 1;
L_2 = 1.5;
m_1 = 2;
m_2 = 1;
g = 9.8;
eqn_1 = subs(eqRed_1)

```

```
eqn_1 =
```

$$\cos(\theta_1(t)) \sigma_1 - \frac{3 \sin(\theta_2(t)) \left( \frac{\partial}{\partial t} \theta_2(t) \right)^2}{2} - \sin(\theta_1(t)) \left( \frac{\partial}{\partial t} \theta_1(t) \right)^2 + \frac{3 \cos(\theta_2(t)) \frac{\partial^2}{\partial t^2} \theta_2(t)}{2} = - \frac{2 \sin(\theta_2(t)) \left( \cos(\theta_1(t))^2 \sigma_1 + \sin(\theta_1(t)) \right)}{\cos(\theta_1(t)) \sin(\theta_2(t)) - \cos(\theta_2(t))}$$

where

$$\sigma_1 = \frac{\partial^2}{\partial t^2} \theta_1(t)$$

```
eqn_2 = subs(eqRed_2)
```

```
eqn_2 =
```

$$\cos(\theta_1(t)) \left( \frac{\partial}{\partial t} \theta_1(t) \right)^2 + \frac{3 \cos(\theta_2(t)) \left( \frac{\partial}{\partial t} \theta_2(t) \right)^2}{2} + \sin(\theta_1(t)) \sigma_1 + \frac{3 \sin(\theta_2(t)) \frac{\partial^2}{\partial t^2} \theta_2(t)}{2} = \frac{2 \cos(\theta_2(t)) \left( \cos(\theta_1(t))^2 \sigma_1 + \sin(\theta_1(t)) \right)}{\cos(\theta_1(t)) \sin(\theta_2(t)) - \cos(\theta_2(t))}$$

where

$$\sigma_1 = \frac{\partial^2}{\partial t^2} \theta_1(t)$$

The two equations are nonlinear second-order differential equations. To solve these equations, convert them to first-order differential equations by using the `odeToVectorField` function.

```
[V,S] = odeToVectorField(eqn_1,eqn_2);
```

The elements of the vector *V* represent the first-order differential equations that are equal to the time derivative of the elements of *S*. The elements of *S* are the state variables  $\theta_2$ ,  $d\theta_2/dt$ ,  $\theta_1$ , and  $d\theta_1/dt$ . The state variables describe the angular displacements and velocities of the double pendulum.

*S*

*S* =

$$\begin{pmatrix} \theta_2 \\ D\theta_2 \\ \theta_1 \\ D\theta_1 \end{pmatrix}$$

Next, convert the first order-differential equations to a MATLAB function with the handle *M*.

```
M = matlabFunction(V,Vars={'t','Y'});
```

Define the initial conditions of the state variables as  $[\pi/4 \ 0 \ \pi/6 \ 0]$ . Use the `ode45` function to solve for the state variables. The solutions are a function of time within the interval  $[0 \ 10]$ .

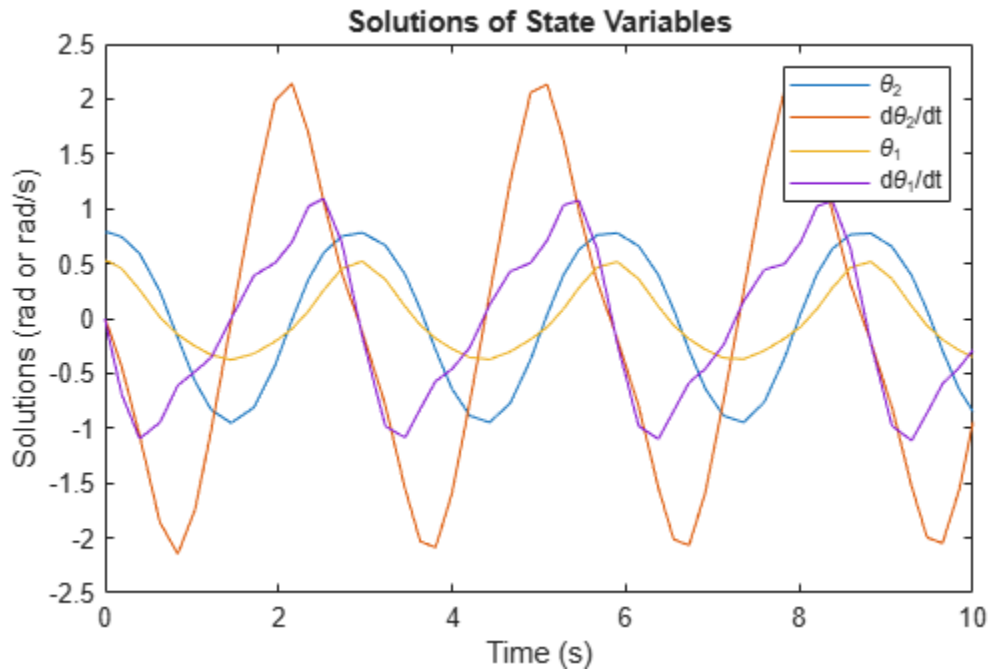
```
initCond = [pi/4 0 pi/6 0];
sols = ode45(M,[0 10],initCond);
```

Plot the solutions of the state variables.

```

plot(sols.x,sols.y)
legend("\theta_2","d\theta_2/dt","\theta_1","d\theta_1/dt")
title("Solutions of State Variables")
xlabel("Time (s)")
ylabel("Solutions (rad or rad/s)")

```



### Step 5: Create Animation of Oscillating Double Pendulum

Create the animation of the oscillating double pendulum.

First, create four functions that use `deval` to evaluate the coordinates of both pendulums from the solutions `sols`.

```

x_1 = @(t) L_1*sin(deval(sols,t,3));
y_1 = @(t) -L_1*cos(deval(sols,t,3));
x_2 = @(t) L_1*sin(deval(sols,t,3))+L_2*sin(deval(sols,t,1));
y_2 = @(t) -L_1*cos(deval(sols,t,3))-L_2*cos(deval(sols,t,1));

```

Next, create a stop-motion animation object of the first pendulum bob by using the `fanimator` function. By default, `fanimator` creates an animation object with 10 generated frames per unit time within the range of `t` from 0 to 10. Plot the coordinates by using the `plot` function. Set the x-axis and y-axis to be equal length.

```

fanimator(@(t) plot(x_1(t),y_1(t),"ro",MarkerSize=m_1*10,MarkerFaceColor="r"));
axis equal;

```

Next, add the animation objects of the first rigid rod, the second pendulum bob, and the second rigid rod.

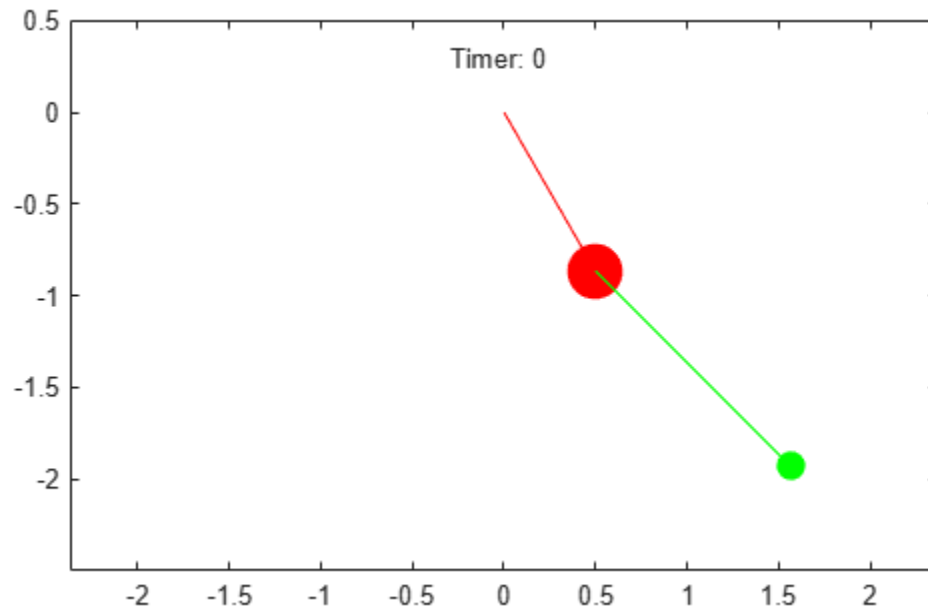
```

hold on;
fanimator(@(t) plot([0 x_1(t)],[0 y_1(t)],"r-"));
fanimator(@(t) plot(x_2(t),y_2(t),"go",MarkerSize=m_2*10,MarkerFaceColor="g"));
fanimator(@(t) plot([x_1(t) x_2(t)],[y_1(t) y_2(t)],"g-"));

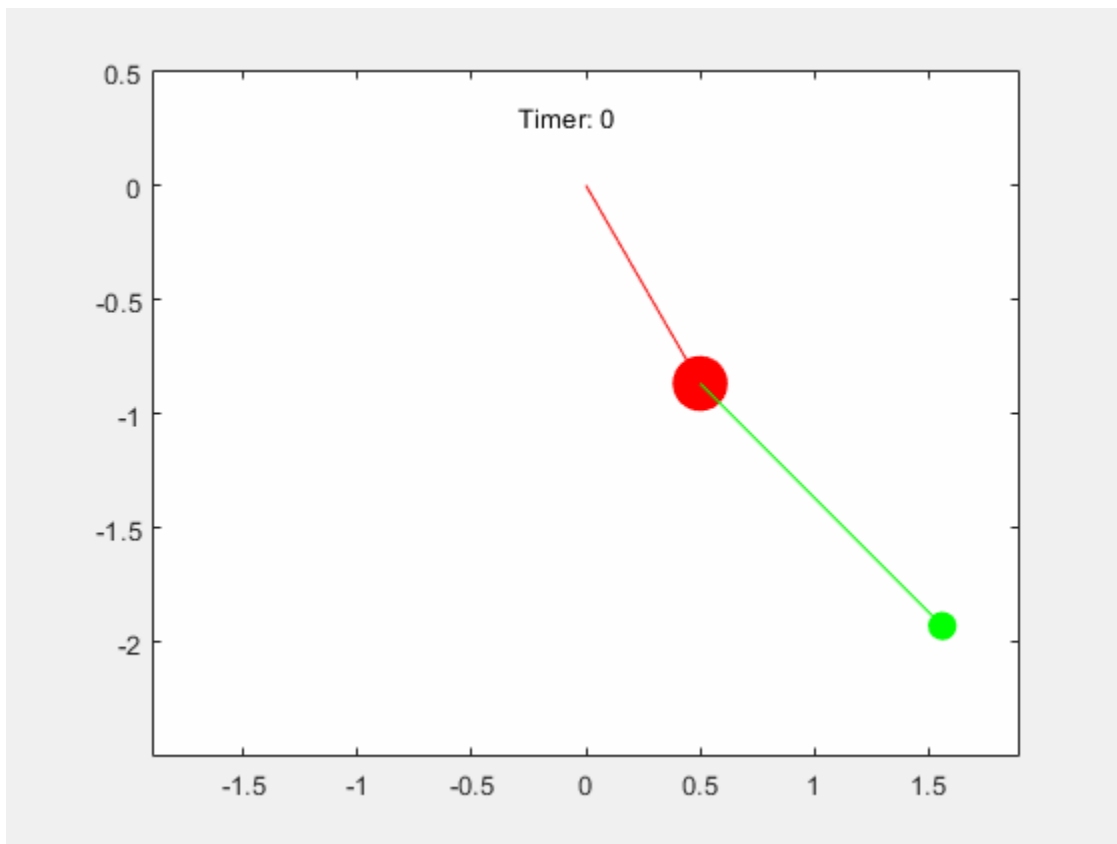
```

Add a piece of text to count the elapsed time by using the `text` function. Use `num2str` to convert the time parameter to a string.

```
fanimation(@(t) text(-0.3,0.3,"Timer: "+num2str(t,2)));  
hold off;
```



Use the command `playAnimation` to play the animation of the double pendulum.



## See Also

### Functions

`diff` | `subs` | `solve` | `matlabFunction` | `fanimator` | `playAnimation` | `ode45`

## Related Examples

- “Simulate the Motion of the Periodic Swing of a Pendulum” on page 4-35
- “Derive Equations of Motion and Simulate Cart-Pole System” on page 3-281

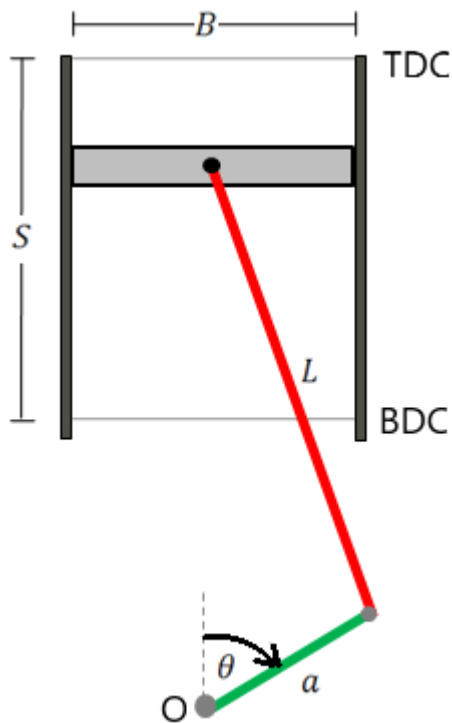
## Animation and Model of Automotive Piston

This example shows how to model the motion of an automotive piston by using MATLAB® and Symbolic Math Toolbox™.

Define the motion of an automotive piston and create an animation to model the piston motion.

### Step 1: Describe Piston Model

The following figure shows the model of an automotive piston. The moving parts of the piston consist of a connecting rod (red line), a piston crank (green line), and a piston cylinder head (gray rectangle).



Describe the properties of the piston by defining the parameters:

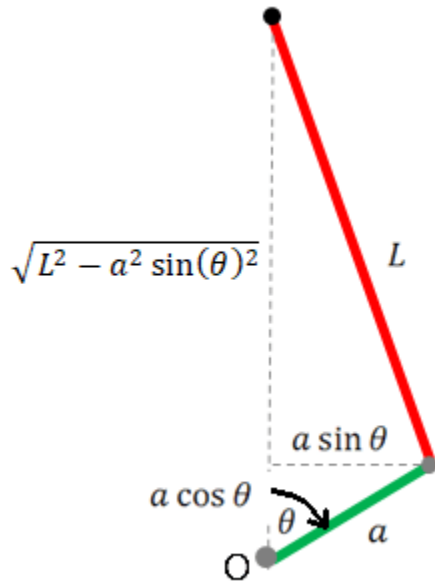
- the cylinder stroke length  $S$
- the piston bore diameter  $B$
- the length of the connecting rod  $L$
- the crank radius  $a$
- the crank angle  $\theta$

Define the origin  $O$  of the coordinate system at the crankshaft location. Label the nearest distance between the piston head and the crankshaft location as bottom dead center (BDC). The height of BDC is  $L - a$ . Label the farthest distance between the piston head and the crankshaft location as top dead center (TDC). The height of TDC is  $L + a$ .



## Step 2: Calculate and Plot Piston Height

The following figure is a schematic of the crank and connecting rod.

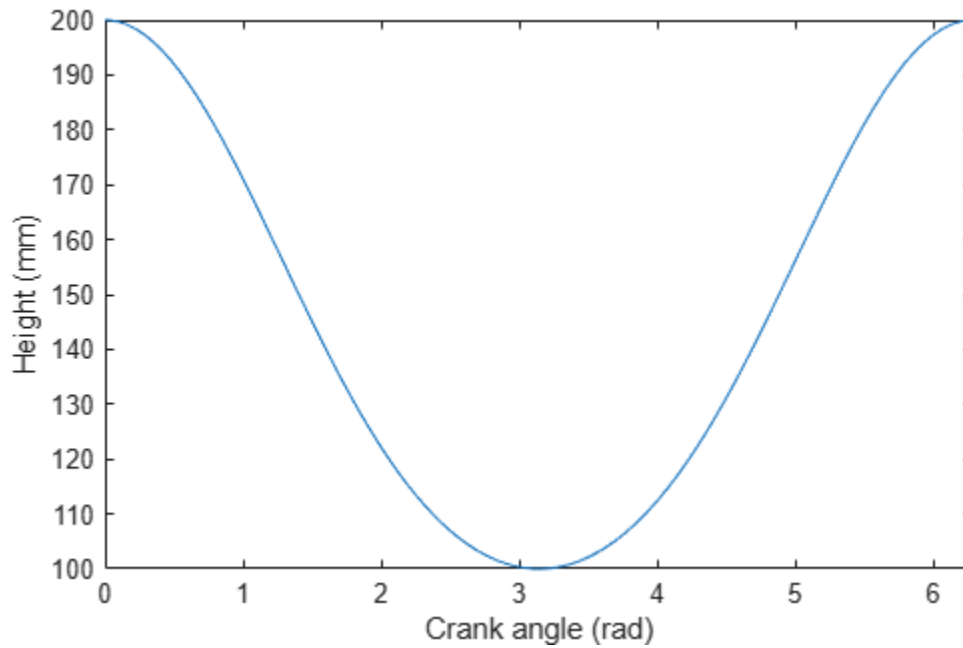


The height of the piston relative to the origin is  $H = a \cos \theta + \sqrt{L^2 - a^2 \sin(\theta)^2}$ . Define the piston height as a symbolic function by using the `syms` function.

```
syms pistHeight(L,a,theta)
pistHeight(L,a,theta) = a*cos(theta) + sqrt(L^2-a^2*sin(theta)^2);
```

Assume that the connecting rod length is  $L = 150$  mm and the crank radius is  $a = 50$  mm. Plot the piston height as a function of the crank angle for one revolution within the interval  $[0 \ 2\pi]$ .

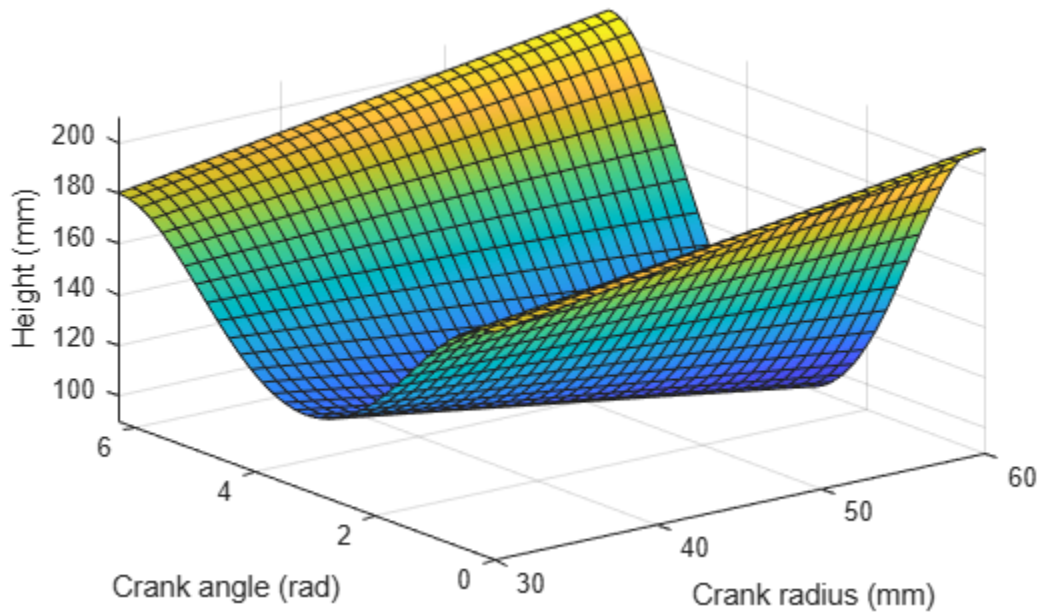
```
fplot(pistHeight(150,50,theta),[0 2*pi])
xlabel('Crank angle (rad)')
ylabel('Height (mm)')
```



The piston head is highest when the piston is at TDC and the crank angle is  $0$  or  $2\pi$ . The piston head is lowest when the piston is at BDC and the crank angle is  $\pi$ .

You can also plot the piston height for various values of  $a$  and  $\theta$ . Create a surface plot of the piston height by using the `fsurf` function. Show the piston height within the interval  $30\text{ mm} < a < 60\text{ mm}$  and  $0 < \theta < 2\pi$ .

```
fsurf(pistHeight(150,a,theta),[30 60 0 2*pi])  
xlabel('Crank radius (mm)')  
ylabel('Crank angle (rad)')  
zlabel('Height (mm)')
```



### Step 3: Calculate and Plot Volume of Piston Cylinder

The length of the combustion chamber is equal to the difference between the TDC location and the piston height. The volume of the piston cylinder can be expressed as  $V = \pi \left(\frac{B}{2}\right)^2 (L + a - H)$ .

Define the piston volume as a symbolic function and substitute the expression for  $H$  with `pistHeight`.

```
syms pistVol(L,a,theta,B)
pistVol(L,a,theta,B) = pi*(B/2)^2*(L+a-pistHeight)

pistVol(L, a, theta, B) =

$$\frac{\pi B^2 \left( L + a - a \cos(\theta) - \sqrt{L^2 - a^2 \sin^2(\theta)} \right)}{4}$$

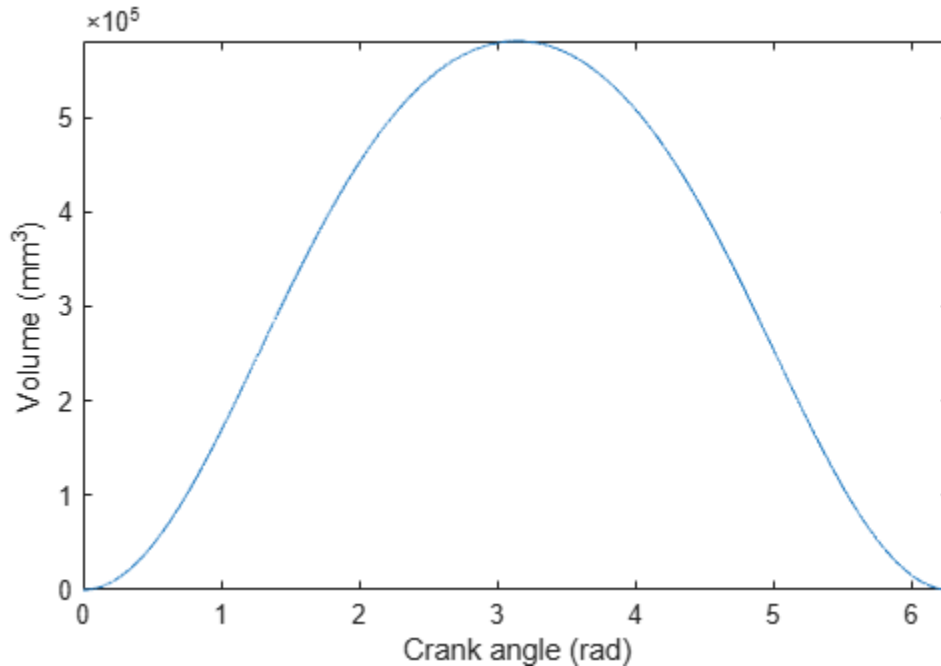
```

Next, define the values for the following parameters:

- the length of the connecting rod  $L = 150$  mm
- the crank radius  $a = 50$  mm
- the bore diameter  $B = 86$  mm

Plot the piston volume as a function of the crank angle for one revolution within the interval  $[0, 2\pi]$ .

```
fplot(pistVol(150,50,theta,86),[0 2*pi])
xlabel('Crank angle (rad)')
ylabel('Volume (mm^3)')
```



The piston volume is smallest when the piston is at TDC and the crank angle is 0 or  $2\pi$ . The piston volume is largest when the piston is at BDC and the crank angle is  $\pi$ .

#### Step 4: Evaluate Piston Motion for Changing Angular Speed

Assume the crank rotates at 30 rpm for the first 3 seconds, then steadily increases from 30 to 80 rpm for the next 4 seconds, and then remains at 80 rpm.

Define the angular speed as a function of time by using the `piecewise` function. Multiply the angular speed by  $2\pi/60$  to convert the rotational speed from rpm to rad/sec.

```
syms t0 t
rpmConv = 2*pi/60;
angVel(t0) = piecewise(t0 <= 3, 30, t0 > 3 & t0 <= 7, 30 + 50/4*(t0-3), t0 > 7, 80)*rpmConv
```

$$\text{angVel}(t_0) = \begin{cases} \pi & \text{if } t_0 \leq 3 \\ \frac{\pi \left( \frac{25}{2} t_0 - \frac{15}{2} \right)}{30} & \text{if } t_0 \in (3, 7] \\ \frac{8\pi}{3} & \text{if } 7 < t_0 \end{cases}$$

Calculate the crank angle by integrating the angular speed using the `int` function. Assume an initial crank angle of  $\theta = 0$ . Compute the integral of the angular speed from 0 to  $t$ .

```
angPos(t) = int(angVel,t0,0,t);
```

Find the piston height as a function of time by substituting the expression `angPos` for the crank angle.

```
H(t) = pistHeight(150,50,angPos)
```

$$H(t) = \begin{cases} 200 & \text{if } t = 0 \\ 100 & \text{if } t = 3 \\ \sqrt{20625} + 25 & \text{if } t = 7 \\ 50 \cos(\sigma_1) + \sqrt{22500 - 2500 \sin(\sigma_1)^2} & \text{if } 7 < t \\ \sqrt{22500 - 2500 \sin(\sigma_2)^2} - 50 \cos(\sigma_2) & \text{if } t \in (3, 7] \\ 50 \cos(\pi t) + \sqrt{22500 - 2500 \sin(\pi t)^2} & \text{if } t < 0 \vee t \in (0, 3] \end{cases}$$

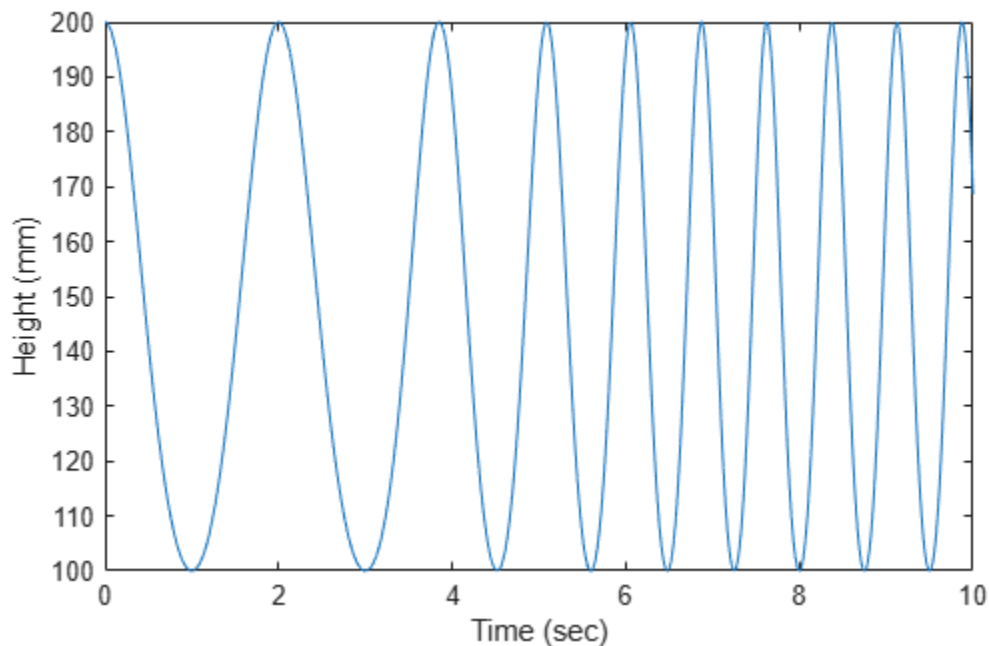
where

$$\sigma_1 = \frac{31\pi}{3} + \frac{8\pi(t-7)}{3}$$

$$\sigma_2 = \frac{\pi(5t+9)(t-3)}{24}$$

Plot the piston height as a function of time. Notice that the oscillation of the piston height becomes faster between 3 and 7 seconds.

```
fplot(H(t), [0 10])
xlabel('Time (sec)')
ylabel('Height (mm)')
```



### Step 5: Create Animation of Moving Piston

Create an animation of the moving piston given a changing angular speed.

First, create a new figure. Plot the cylinder walls that have fixed locations. Set the x-axis and y-axis to be equal length.

```
figure;
plot([-43 -43],[50 210], 'k', 'LineWidth', 3)
```

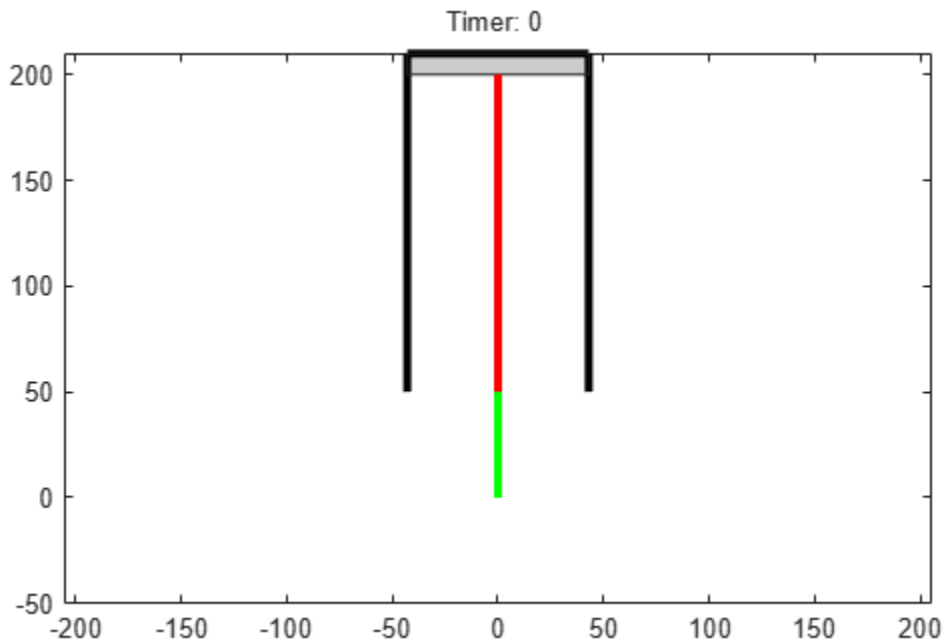
```
hold on;
plot([43 43],[50 210],'k','LineWidth',3)
plot([-43 43],[210 210],'k','LineWidth',3)
axis equal;
```

Next, create a stop-motion animation object of the piston head by using the `fanimator` function. By default, `fanimator` creates an animation object by generating 10 frames per unit time within the range of  $t$  from 0 to 10. Model the piston head as a rectangle with a thickness of 10 mm and variable height  $H(t)$ . Plot the piston head by using the `rectangle` function.

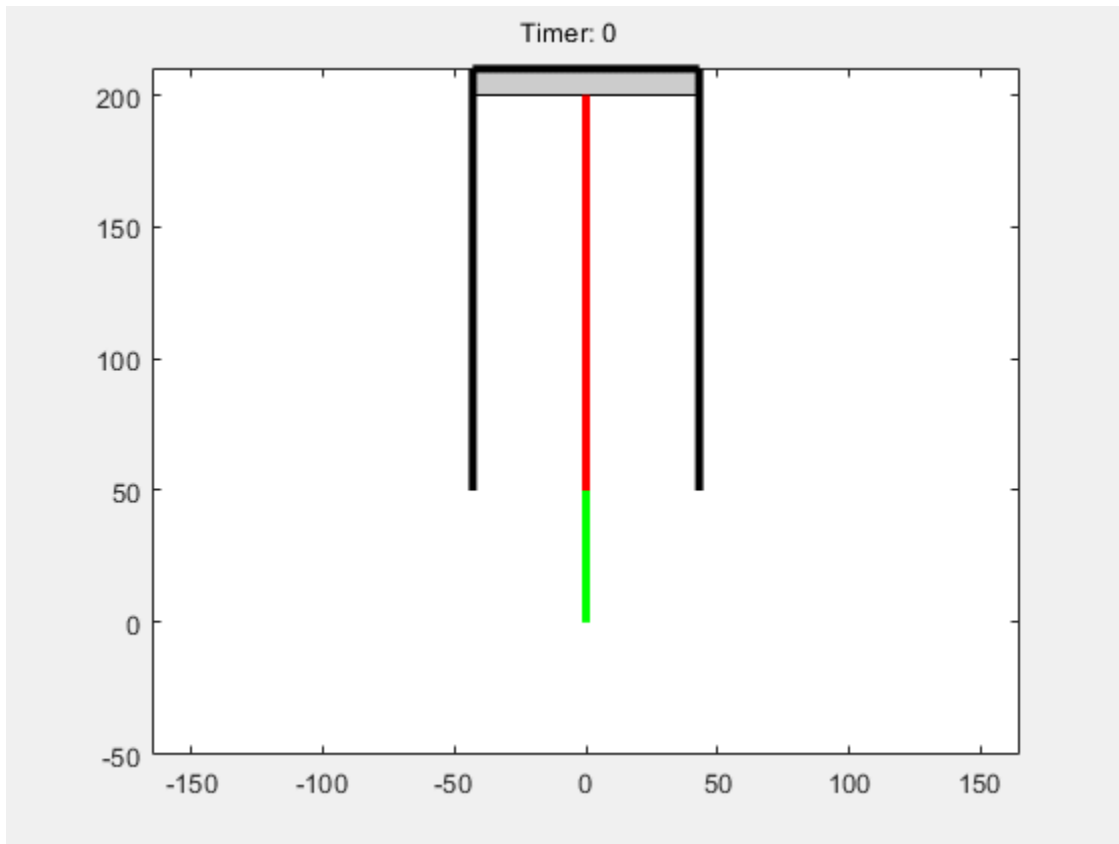
```
fanimator(@rectangle,'Position',[-43 H(t) 86 10],'FaceColor',[0.8 0.8 0.8])
```

Add the animation objects of the connecting rod and the piston crank. Add a piece of text to count the elapsed time.

```
fanimator(@t plot([0 50*sin(angPos(t))],[H(t) 50*cos(angPos(t))],'r-','LineWidth',3))
fanimator(@t plot([0 50*sin(angPos(t))],[0 50*cos(angPos(t))],'g-','LineWidth',3))
fanimator(@t text(-25,225,"Timer: "+num2str(t,2)));
hold off;
```



Use the command `playAnimation` to play the animation of the moving piston.







# Code Generation

---

- “Generate C or Fortran Code from Symbolic Expressions” on page 5-2
- “Generate MATLAB Functions from Symbolic Expressions” on page 5-3
- “Generate MATLAB Function Blocks from Symbolic Expressions” on page 5-6
- “Generate Simscape Equations from Symbolic Expressions” on page 5-8
- “Deploy Generated MATLAB Functions from Symbolic Expressions with MATLAB Compiler” on page 5-10
- “Generate C Code from Symbolic Expressions Using the MATLAB Coder App” on page 5-15
- “Derive Quadrotor Dynamics for Nonlinear Model Predictive Control” on page 5-24
- “Autonomous Underwater Vehicle Pose Estimation Using Analytical Jacobians in Custom Sensor Models” on page 5-31
- “Using Symbolic Mathematics with Optimization Toolbox Solvers” on page 5-43
- “Improving Accuracy and Performance in Optimization” on page 5-52
- “Analytical Model of Cantilever Truss Structure for Simscape” on page 5-57
- “Customize and Extend Simscape Libraries for a Custom DC Motor” on page 5-65
- “Estimate Model Parameters of a Symbolically Derived Plant Model in Simulink” on page 5-74

## Generate C or Fortran Code from Symbolic Expressions

You can generate C or Fortran code fragments from a symbolic expression, or generate files containing code fragments, using the `ccode` and `fortran` functions. These code fragments calculate numerical values as if substituting numbers for variables in the symbolic expression.

To generate code from a symbolic expression `g`, enter either `ccode(g)` or `fortran(g)`.

For example:

```
syms x y
z = 30*x^4/(x*y^2 + 10) - x^3*(y^2 + 1)^2;
fortran(z)
```

```
ans =
'      t0 = (x**4*3.0D+1)/(x*y**2+1.0D+1)-x**3*(y**2+1.0D0)**2'
```

```
ccode(z)
```

```
ans =
'      t0 = ((x*x*x*x)*3.0E+1)/(x*(y*y)+1.0E+1)-(x*x*x)*pow(y*y+1.0,2.0);'
```

To generate a file containing code, either enter `ccode(g,'file','filename')` or `fortran(g,'file','filename')`. For the example above,

```
fortran(z, 'file', 'fortrantest')
```

generates a file named `fortrantest` in the current folder. `fortrantest` consists of the following:

```
t2 = y**2
t0 = (x**4*3.0D+1)/(t2*x+1.0D+1)-x**3*(t2+1.0D0)**2
```

Similarly, the command

```
ccode(z,'file','ccodetest')
```

generates a file named `ccodetest` that consists of the lines

```
t2 = y*y;
t0 = ((x*x*x*x)*3.0E+1)/(t2*x+1.0E+1)-(x*x*x)*pow(t2+1.0,2.0);
```

`ccode` and `fortran` generate many intermediate variables. This is called *optimized* code. MATLAB generates intermediate variables as a lowercase letter `t` followed by an automatically generated number, such as `t2`. Intermediate variables can make the resulting code more efficient by reusing intermediate expressions (such as `t2` in `fortrantest` and `ccodetest`). They can also make the code easier to read by keeping expressions short.

## Generate MATLAB Functions from Symbolic Expressions

You can use `matlabFunction` to generate a MATLAB function handle that calculates numerical values as if you were substituting numbers for variables in a symbolic expression. Also, `matlabFunction` can create a file that accepts numeric arguments and evaluates the symbolic expression applied to the arguments. The generated file is available for use in any MATLAB calculation, whether or not the computer running the file has a license for Symbolic Math Toolbox functions.

### Generating a Function Handle

`matlabFunction` can generate a function handle from any symbolic expression. For example:

```
syms x y
r = sqrt(x^2 + y^2);
ht = matlabFunction(tanh(r))

ht =
    function_handle with value:
    @(x,y)tanh(sqrt(x.^2+y.^2))
```

You can use this function handle to calculate numerically:

```
ht(.5,.5)

ans =
    0.6089
```

You can pass the usual MATLAB double-precision numbers or matrices to the function handle. For example:

```
cc = [.5,3];
dd = [-.5,.5];
ht(cc, dd)

ans =
    0.6089    0.9954
```

---

**Tip** Some symbolic expressions cannot be represented using MATLAB functions. `matlabFunction` cannot convert these symbolic expressions, but issues a warning. Since these expressions might result in undefined function calls, always check conversion results and verify the results by executing the resulting function.

---

### Control the Order of Variables

`matlabFunction` generates input variables in alphabetical order from a symbolic expression. That is why the function handle in “Generating a Function Handle” on page 5-3 has `x` before `y`:

```
ht = @(x,y)tanh((x.^2 + y.^2).^(1./2))
```

You can specify the order of input variables in the function handle using the `vars` option. You specify the order by passing a cell array of character vectors or symbolic arrays, or a vector of symbolic variables. For example:

```
syms x y z
r = sqrt(x^2 + 3*y^2 + 5*z^2);
ht1 = matlabFunction(tanh(r), 'vars', [y x z])

ht1 =
    function_handle with value:
        @(y,x,z)tanh(sqrt(x.^2+y.^2.*3.0+z.^2.*5.0))

ht2 = matlabFunction(tanh(r), 'vars', {'x', 'y', 'z'})

ht2 =
    function_handle with value:
        @(x,y,z)tanh(sqrt(x.^2+y.^2.*3.0+z.^2.*5.0))

ht3 = matlabFunction(tanh(r), 'vars', {'x', [y z]})

ht3 =
    function_handle with value:
        @(x,in2)tanh(sqrt(x.^2+in2(:,1).^2.*3.0+in2(:,2).^2.*5.0))
```

## Generate a File

You can generate a file from a symbolic expression, in addition to a function handle. Specify the file name using the `file` option. Pass a character vector containing the file name or the path to the file. If you do not specify the path to the file, `matlabFunction` creates this file in the current folder.

This example generates a file that calculates the value of the symbolic matrix `F` for double-precision inputs `t`, `x`, and `y`:

```
syms x y t
z = (x^3 - tan(y))/(x^3 + tan(y));
w = z/(1 + t^2);
F = [w, (1 + t^2)*x/y; (1 + t^2)*x/y, 3*z - 1];
matlabFunction(F, 'file', 'testMatrix.m')
```

The file `testMatrix.m` contains the following code:

```
function F = testMatrix(t,x,y)
%TESTMATRIX
%    F = TESTMATRIX(T,X,Y)

t2 = x.^2;
t3 = tan(y);
t4 = t2.*x;
t5 = t.^2;
t6 = t5 + 1;
t7 = 1./y;
t8 = t6.*t7.*x;
t9 = t3 + t4;
t10 = 1./t9;
F = [-(t10.*(t3 - t4))./t6,t8; t8,- t10.*(3.*t3 - 3.*t2.*x) - 1];
```

`matlabFunction` generates many intermediate variables. This is called *optimized* code. MATLAB generates intermediate variables as a lowercase letter `t` followed by an automatically generated number, for example `t32`. Intermediate variables can make the resulting code more efficient by reusing intermediate expressions (such as `t4`, `t6`, `t8`, `t9`, and `t10` in the calculation of `F`). Using intermediate variables can make the code easier to read by keeping expressions short.

If you don't want the default alphabetical order of input variables, use the `vars` option to control the order. Continuing the example,

```
matlabFunction(F, 'file', 'testMatrix.m', 'vars', [x y t])
```

generates a file equivalent to the previous one, with a different order of inputs:

```
function F = testMatrix(x,y,t)
...
```

## Name Output Variables

By default, the names of the output variables coincide with the names you use calling `matlabFunction`. For example, if you call `matlabFunction` with the variable  $F$

```
syms x y t
z = (x^3 - tan(y))/(x^3 + tan(y));
w = z/(1 + t^2);
F = [w, (1 + t^2)*x/y; (1 + t^2)*x/y, 3*z - 1];
matlabFunction(F, 'file', 'testMatrix.m', 'vars', [x y t])
```

the generated name of an output variable is also  $F$ :

```
function F = testMatrix(x,y,t)
...
```

If you call `matlabFunction` using an expression instead of individual variables

```
syms x y t
z = (x^3 - tan(y))/(x^3 + tan(y));
w = z/(1 + t^2);
F = [w, (1 + t^2)*x/y; (1 + t^2)*x/y, 3*z - 1];
matlabFunction(w + z + F, 'file', 'testMatrix.m', ...
'vars', [x y t])
```

the default names of output variables consist of the word `out` followed by the number, for example:

```
function out1 = testMatrix(x,y,t)
...
```

To customize the names of output variables, use the `output` option:

```
syms x y z
r = x^2 + y^2 + z^2;
q = x^2 - y^2 - z^2;
f = matlabFunction(r, q, 'file', 'new_function', ...
'outputs', {'name1', 'name2'})
```

The generated function returns *name1* and *name2* as results:

```
function [name1,name2] = new_function(x,y,z)
...
```

## Generate MATLAB Function Blocks from Symbolic Expressions

Using `matlabFunctionBlock`, you can generate a MATLAB Function block. The generated block is available for use in Simulink models, whether or not the computer running the simulations has a license for Symbolic Math Toolbox.

### Generate and Edit a Block

Suppose, you want to create a model involving the symbolic expression  $r = \sqrt{x^2 + y^2}$ . Before you can convert a symbolic expression to a MATLAB Function block, create an empty model or open an existing one:

```
new_system('my_system')
open_system('my_system')
```

Create a symbolic expression and pass it to the `matlabFunctionBlock` command. Also specify the block name:

```
syms x y
r = sqrt(x^2 + y^2);
matlabFunctionBlock('my_system/my_block', r)
```

If you use the name of an existing block, the `matlabFunctionBlock` command replaces the definition of an existing block with the converted symbolic expression.

You can open and edit the generated block. To open a block, double-click it.

```
function r = my_block(x,y)
%#codegen

% x = complex(x); % required to enforce complex operations on x
% y = complex(y); % required to enforce complex operations on y
r = sqrt(x.^2+y.^2);
```

---

**Tip** Some symbolic expressions cannot be represented using MATLAB functions. `matlabFunctionBlock` cannot convert these symbolic expressions, but issues a warning. Since these expressions might result in undefined function calls, always check conversion results and verify results by running the simulation containing the resulting block.

---

### Control the Order of Input Ports

`matlabFunctionBlock` generates input variables and the corresponding input ports in alphabetical order from a symbolic expression. To change the order of input variables, use the `vars` option:

```
syms x y
mu = sym('mu');
dydt = -x - mu*y*(x^2 - 1);
matlabFunctionBlock('my_system/my_block', dydt, 'vars', [y mu x])
```

## Name the Output Ports

By default, `matlabFunctionBlock` generates the names of the output ports as the word `out` followed by the output port number, for example, `out3`. The `output` option allows you to use the custom names of the output ports:

```
syms x y
mu = sym('mu');
dydt = -x - mu*y*(x^2 - 1);
matlabFunctionBlock('my_system/my_block', dydt, 'outputs', {'name1'})
```

## Generate Simscape Equations from Symbolic Expressions

Simscape software extends the Simulink product line with tools for modeling and simulating multidomain physical systems, such as those with mechanical, hydraulic, pneumatic, thermal, and electrical components. Unlike other Simulink blocks, which represent mathematical operations or operate on signals, Simscape blocks represent physical components or relationships directly. With Simscape blocks, you build a model of a system just as you would assemble a physical system. For more information about Simscape software see “Simscape”.

You can extend the Simscape modeling environment by creating custom components. When you define a component, use the equation section of the component file to establish the mathematical relationships among a component's variables, parameters, inputs, outputs, time, and the time derivatives of each of these entities. The Symbolic Math Toolbox and Simscape software let you perform symbolic computations and use the results of these computations in the equation section. The `simscapeEquation` function translates the results of symbolic computations to Simscape language equations.

### Convert Algebraic and Differential Equations

Suppose, you want to generate a Simscape equation from the solution of the following ordinary differential equation. As a first step, use the `dsolve` function to solve the equation:

```
syms a y(t)
Dy = diff(y);
s = dsolve(diff(y, 2) == -a^2*y, y(0) == 1, Dy(pi/a) == 0);
s = simplify(s)
```

The solution is:

```
s =
cos(a*t)
```

Then, use the `simscapeEquation` function to rewrite the solution in the Simscape language:

```
simscapeEquation(s)
```

`simscapeEquation` generates the following code:

```
ans =
's == cos(a*time);'
```

The variable *time* replaces all instances of the variable *t* except for derivatives with respect to *t*. To use the generated equation, copy the equation and paste it to the equation section of the Simscape component file. Do not copy the automatically generated variable `ans` and the equal sign that follows it.

`simscapeEquation` converts any derivative with respect to the variable *t* to the Simscape notation, `X.der`, where *X* is the time-dependent variable. For example, convert the following differential equation to a Simscape equation. Also, here you explicitly specify the left and the right sides of the equation by using the syntax `simscapeEquation(LHS, RHS)`:

```
syms a x(t)
simscapeEquation(diff(x), -a^2*x)

ans =
'x.der == -a^2*x;'
```



`simscapeEquation` also translates piecewise expressions to the Simscape language. For example, the result of the following Fourier transform is a piecewise function:

```
syms v u x
assume(x, 'real')
f = exp(-x^2*abs(v))*sin(v)/v;
s = fourier(f, v, u)

s =
piecewise(x ~= 0, atan((u + 1)/x^2) - atan((u - 1)/x^2))
```

From this symbolic piecewise equation, `simscapeEquation` generates valid code for the equation section of a Simscape component file:

```
simscapeEquation(s)

ans =
    'if (x ~= 0.0)
      s == -atan(1.0/x^2*(u-1.0))+atan(1.0/x^2*(u+1.0));
    else
      s == NaN;
    end'
```

Clear the assumption that  $x$  is real by recreating it using `syms`:

```
syms x
```

## Limitations

The equation section of a Simscape component file supports a limited number of functions. For details and the list of supported functions, see [Simscape equations \(Simscape\)](#). If a symbolic expression contains functions that are not supported by Simscape, then `simscapeEquation` cannot represent the symbolic expression as a Simscape equation and issues a warning instead. Always verify the conversion result. Expressions with infinities are prone to invalid conversion.

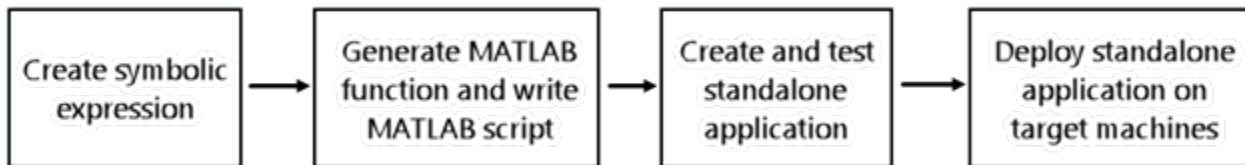
## See Also

### Functions

`simscapeEquation` | `symWriteSSC` | `symReadSSCParameters` | `symReadSSCVariables`

## Deploy Generated MATLAB Functions from Symbolic Expressions with MATLAB Compiler

This example shows how to generate a MATLAB® function from a symbolic expression and use the function to create a standalone application with MATLAB Compiler™. The target system to run the application only requires a MATLAB Runtime installation and does not require a licensed copy of MATLAB.



This example follows the steps described in “Create Standalone Application from MATLAB” (MATLAB Compiler) and updates the steps to generate a MATLAB function from a symbolic expression.

### Generate Deployable Function from Symbolic Expression

First, create a system of differential equations with state variables  $x(t)$  and  $y(t)$ , and input parameters  $\alpha$  and  $\beta$ .

```
syms x(t) y(t) alpha beta
eqs = [x - diff(x,t) == alpha*x*y; ...
      y + diff(y,t) == beta*x*y]
```

$$\text{eqs}(t) = \begin{pmatrix} x(t) - \frac{\partial}{\partial t} x(t) = \alpha x(t) y(t) \\ \frac{\partial}{\partial t} y(t) + y(t) = \beta x(t) y(t) \end{pmatrix}$$

To solve the system of differential equations, convert it to first-order differential equations by using the `odeToVectorField` function.

```
V = odeToVectorField(eqs);
```

Next, convert the symbolic expression  $V$  to a MATLAB function file by using `matlabFunction`. The converted function in the file `myODE.m` can be used without Symbolic Math Toolbox™.

```
matlabFunction(V,File="myODE")
```

```
ans = function_handle with value:
      @myODE
```

### Write Function in MATLAB

Write a MATLAB function named `plotODESols` that solves the system of differential equations using `ode45`. This function accepts the input parameters `alpha` and `beta`. Because these parameters might be interpreted as character vectors in your system, convert them to the `double` data type.

Finally, this function plots the solutions of the differential equations. Save it in the same directory as `myODE.m` function.

type `plotODESols`

```
function plotODESols(alpha,beta)
    if ischar(alpha)
        alpha = double(string(alpha));
    end
    if ischar(beta)
        beta = double(string(beta));
    end
    tspan = linspace(0,20,200);
    [t,y] = ode45(@(t,Y) myODE(Y,alpha,beta),tspan,[50 100]);
    plot(t,y,"o-")
    xlabel("Time")
    ylabel("Population")
end
```

You can use this function to create and deploy standalone application using MATLAB Compiler.

### Create Standalone Application Using `compiler.build.standaloneApplication`

Build a standalone application using a programmatic approach. Alternatively, if you want to create a standalone application package using a graphical interface, see “Create Standalone Application Using Standalone Application Compiler App” (MATLAB Compiler).

Build the standalone application using the `compiler.build.standaloneApplication` (MATLAB Compiler) function.

```
buildResults = compiler.build.standaloneApplication("plotODESols.m");
```

The `buildResults` object contains information on the build type, generated files, included support packages, and build options. For details, see `compiler.build.Results` (MATLAB Compiler).

The function generates the following files within a folder named `plotODESolsstandaloneApplication` in your current working directory:

- `includedSupportPackages.txt` — Text file that lists all support files included in the application.
- `plotODESols.exe` or `plotODESols` — Executable file that has the `.exe` extension if compiled on a Windows system, or no extension if compiled on Linux or macOS systems.
- `run_plotODESols.sh` — Shell script file that sets the library path and executes the application. This file is only generated on Linux and macOS systems.
- `mccExcludedFiles.log` — Log file that contains a list of any toolbox functions that were not included in the application. For information on non-supported functions, see “Limitations for MATLAB Compiler” (MATLAB Compiler).
- `readme.txt` — Text file that contains information on deployment prerequisites and the list of files to package for deployment.
- `requiredMCRProducts.txt` — Text file that contains product IDs of products required by MATLAB Runtime to run the application.
- `unresolvedSymbols.txt` — Text file that contains information on unresolved symbols.

To test `plotODESols` from MATLAB with the input parameters `alpha = 0.01` and `beta = 0.02`, navigate to the `plotODESolsstandaloneApplication` folder from within the MATLAB desktop environment and execute one of the following commands based on your operating system:

| Operating System | Test in MATLAB Command Window                                           |
|------------------|-------------------------------------------------------------------------|
| Windows          | <code>!plotODESols 0.01 0.02</code>                                     |
| macOS            | <code>system(['./run_plotODESols.sh ',matlabroot,' 0.01 0.02']);</code> |
| Linux            | <code>!./plotODESols 0.01 0.02</code>                                   |

### Create Standalone Application Installer Using `compiler.package.installer`

Create an installer using the `buildResults` object as an input argument to the `compiler.package.installer` function. By default, the installer is configured to download MATLAB Runtime from the web. You can modify this and specify additional options by using name-value arguments. For details, see `compiler.package.installer` (MATLAB Compiler).

For example, include MATLAB Runtime in the installer. The function creates a new folder named `plotODESolsinstaller` that contains the standalone application installer.

```
compiler.package.installer(buildResults,RuntimeDelivery="installer");
```

To install your application using an installer created by the `compiler.package.installer` function, see “Install Deployed Application” (MATLAB Compiler). For example, in Windows® platform, you can query into the `plotODESolsinstaller` folder location and double-click the file `MyAppInstaller.exe`. Complete the installation by following the instructions in the installation wizard.

### Run Standalone Application

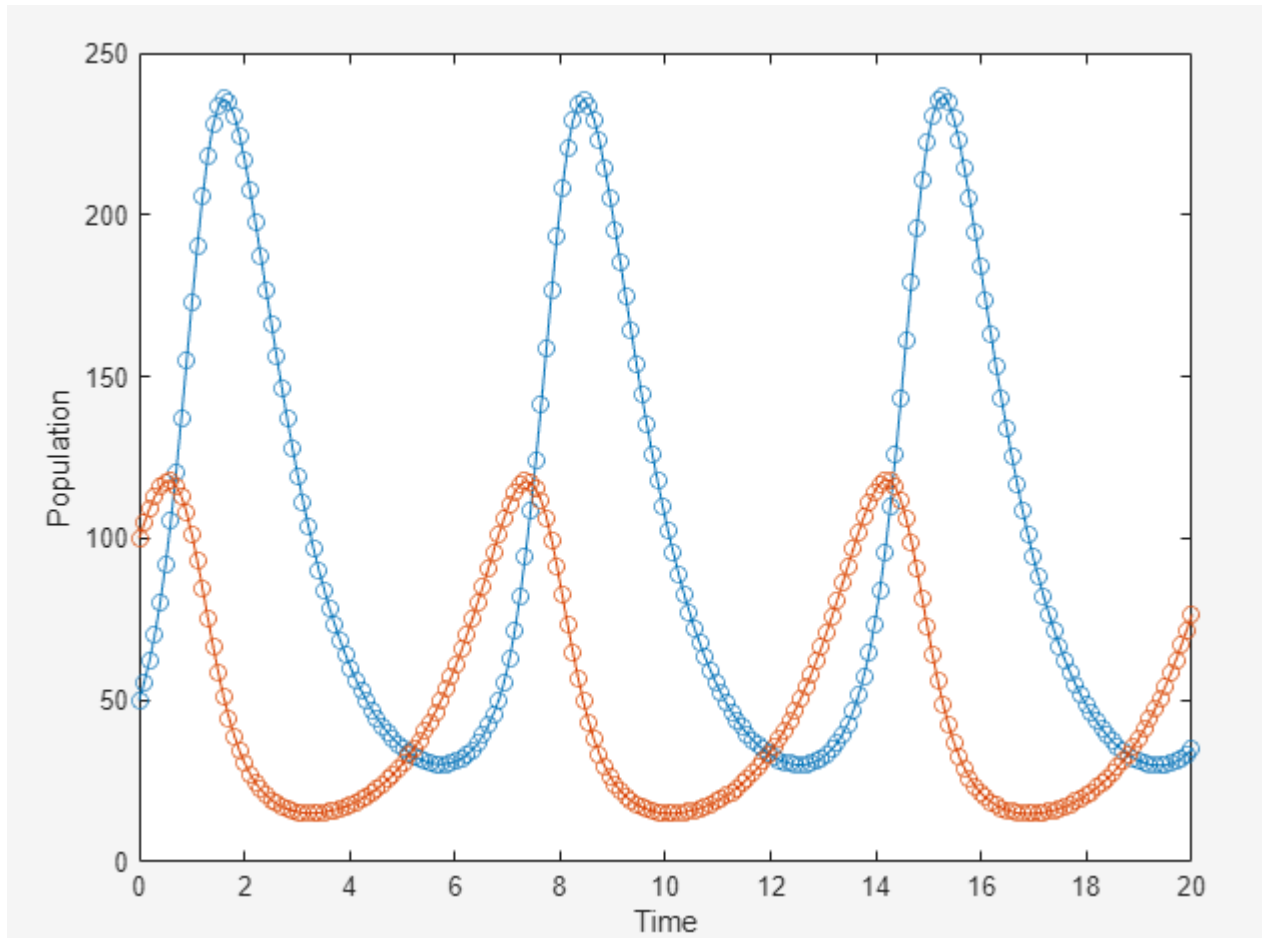
In your system command prompt, navigate to the folder containing your standalone executable.

Run `plotODESols` with the input arguments `0.01` and `0.02` by using one of the following commands based on your operating system:

| Operating System | Command                                                                                                                                                                                                               |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Windows          | <code>plotODESols 0.01 0.02</code>                                                                                                                                                                                    |
| macOS            | Using the shell script:<br><br><code>./run_plotODESols.sh &lt;MATLAB_RUNTIME_INSTALL_DIR&gt; 0.01 0.02</code><br><br>Using the executable:<br><br><code>./plotODESols.app/Contents/macOS/plotODESols 0.01 0.02</code> |
| Linux            | Using the shell script:<br><br><code>./run_plotODESols.sh &lt;MATLAB_RUNTIME_INSTALL_DIR&gt; 0.01 0.02</code><br><br>Using the executable:<br><br><code>./plotODESols 0.01 0.02</code>                                |

**NOTE:** To run the application without using the shell script on Linux and macOS, you must first add MATLAB Runtime to the library path. For details, see “Set MATLAB Runtime Library Paths for Deployment” (MATLAB Compiler).

The application plots the solutions of the system of differential equations.



### Deploy Standalone Application on Target Machines

Copy the `plotODESolsstandaloneApplication` folder to a file location on all target machines where MATLAB Runtime is installed. Run the MATLAB generated standalone application on all target machines by using the executable in the `plotODESolsstandaloneApplication` folder.

Alternatively, you can distribute the installer located in the `plotODESolsinstaller` folder. This installer will install the application along with MATLAB Runtime, which allows you to run the application outside of MATLAB.

### Tips

- Instead of using the `compiler.build.standaloneApplication` (MATLAB Compiler) function to create a standalone application, you can also use the Standalone Application Compiler (MATLAB Compiler) app to create a standalone application.
- To produce a standalone application that does not launch a Windows command shell, use `compiler.build.standaloneWindowsApplication` (MATLAB Compiler).

- To specify additional compilation options, you can use the `mcc` (MATLAB Compiler) command to create a standalone application that does not include MATLAB Runtime or an installer.

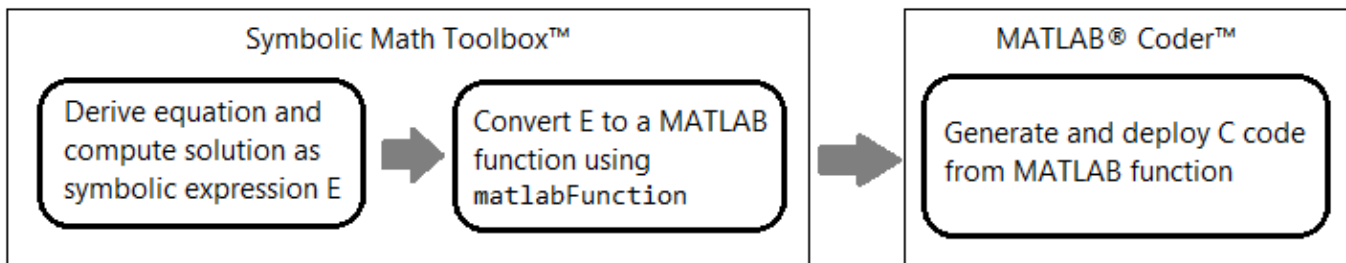
## **See Also**

### **Related Examples**

- “Download and Install MATLAB Runtime” (MATLAB Compiler)
- “Create Standalone Application from MATLAB” (MATLAB Compiler)
- “Create Standalone Application Using Standalone Application Compiler App” (MATLAB Compiler)
- “Set MATLAB Runtime Library Paths for Deployment” (MATLAB Compiler)
- “Install Deployed Application” (MATLAB Compiler)

## Generate C Code from Symbolic Expressions Using the MATLAB Coder App

This example shows how to use the MATLAB® Coder™ app to generate a static C library from symbolic expressions. First, you work with symbolic expressions in Symbolic Math Toolbox™, and convert the symbolic expressions into a deployable MATLAB function using `matlabFunction`. Next, you generate C code from the MATLAB function. The generated C code accepts inputs that have a fixed, preassigned size, but you can also specify variable-size inputs during code generation.



This example follows the steps described in “Generate Deployable Standalone Code by Using the MATLAB Coder App” (MATLAB Coder), but updates the steps to generate a MATLAB function from a symbolic expression. Alternatively, you can generate C code from a MATLAB function at the MATLAB command line by using the `codegen` (MATLAB Coder) command. For a tutorial on this workflow, see “Generate Standalone C Static Library at the Command Line” (MATLAB Coder).

### Generate Deployable MATLAB Function from Symbolic Expression

This example solves for the eigenvalues of the model Hamiltonian:

$$H = \begin{pmatrix} (q-1)^2 - \frac{\delta}{2} & \Omega \\ \Omega & \frac{\delta}{2} + (q+1)^2 \end{pmatrix},$$

where  $q$ ,  $\Omega$ , and  $\delta$  are the parameters of the Hamiltonian.

Create the symbolic variables `q`, `Omega`, and `delta` to represent the parameters of the Hamiltonian. Create a symbolic matrix for the Hamiltonian.

```
syms q Omega delta
H = [(q-1)^2 - delta/2, Omega; Omega, (q+1)^2 + delta/2]
```

$$H = \begin{pmatrix} (q-1)^2 - \frac{\delta}{2} & \Omega \\ \Omega & \frac{\delta}{2} + (q+1)^2 \end{pmatrix}$$

Find the two eigenvalues of the Hamiltonian.

```
E = eig(H)
```

```
E =
```

$$\begin{pmatrix} q^2 - \frac{\sqrt{4\Omega^2 + \delta^2 + 8\delta q + 16q^2}}{2} + 1 \\ q^2 + \frac{\sqrt{4\Omega^2 + \delta^2 + 8\delta q + 16q^2}}{2} + 1 \end{pmatrix}$$

Next, convert the two eigenvalues  $E(1)$  and  $E(2)$  to a MATLAB function file by using `matlabFunction`. Write the resulting function, which returns two elements  $E1$  and  $E2$ , to the file `myEigenvalues.m`. Specify the order of input arguments as `[q Omega delta]`.

```
matlabFunction(E(1),E(2),'File','myEigenvalues', ...
    'Vars',[q Omega delta],'Outputs',{'E1','E2'});
```

The converted function in the file `myEigenvalues.m` can be used without Symbolic Math Toolbox. The MATLAB file `myEigenvalues.m` contains the function `myEigenvalues` that implements the core algorithm in this example. The function takes `q`, `Omega`, and `delta` as inputs, all of which must be either the same size or a scalar. It then calculates the two eigenvalues as a function of these inputs.

type `myEigenvalues`

```
function [E1,E2] = myEigenvalues(q,Omega,delta)
%myEigenvalues
%    [E1,E2] = myEigenvalues(Q,Omaga,DELTA)

%    This function was generated by the Symbolic Math Toolbox version 26.1.
%    20-Apr-2026 02:19:01

t2 = Omega.^2;
t3 = delta.^2;
t4 = q.^2;
t6 = delta.*q.*8.0;
t5 = t2.*4.0;
t7 = t4.*1.6e+1;
t8 = t3+t5+t6+t7;
t9 = sqrt(t8);
t10 = t9./2.0;
E1 = t4-t10+1.0;
if nargin > 1
    E2 = t4+t10+1.0;
end
end
```

### Run MATLAB Test Script

To calculate the eigenvalues for a set of inputs, create and run the test script `myTest.m` in MATLAB. The test script specifies the inputs with the following sizes:

- `qGrid` is a 128-by-256 matrix that represents points in the two-dimensional  $(q,\Omega)$  space.
- `OmegaGrid` is a 128-by-256 matrix that represents points in the two-dimensional  $(q,\Omega)$  space.
- `delta` is a scalar.

The script then calls the function `myEigenvalues.m` to compute the eigenvalues. The output displays a plot of the eigenvalues for these input values. Below is the content of the script `myTest.m`.

```
q = linspace(-2,2,256);
Omega = linspace(0,2,128);
```



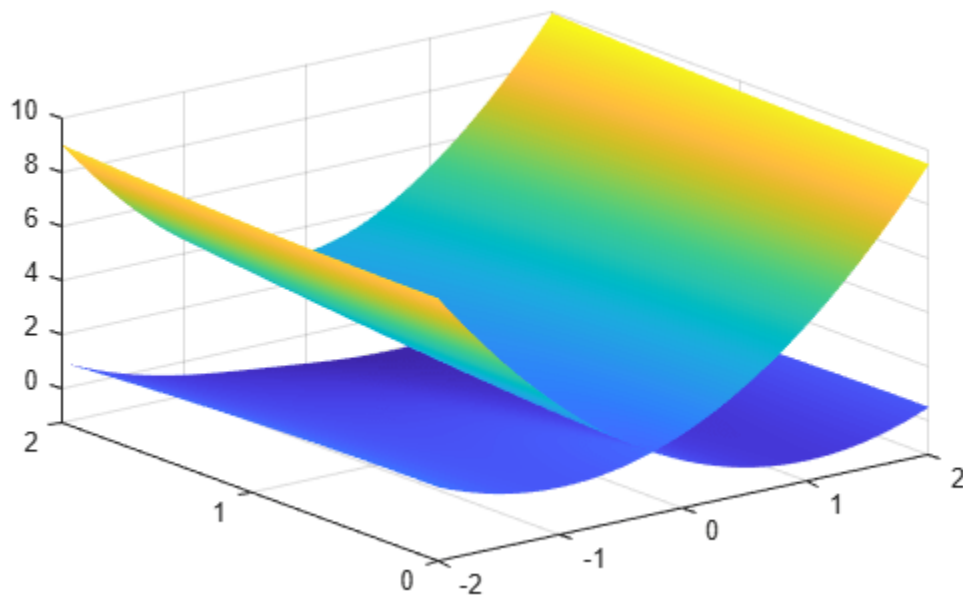
```

delta = 1;

[qGrid,OmegaGrid] = meshgrid(q,Omega);
[E1,E2] = myEigenvalues(qGrid,OmegaGrid,delta);

surf(q,Omega,E1)
hold on;
surf(q,Omega,E2)
shading interp

```



## Generate C Code from MATLAB Function

### Make MATLAB Code Suitable for Code Generation

The Code Analyzer in the MATLAB Editor continuously checks your code as you enter it. It reports issues and recommends modifications to maximize performance and maintainability.

- Open `myEigenvalues.m` in the MATLAB Editor. After the function declaration, add the `%#codegen` directive:

```

1 function [E1,E2] = myEigenvalues(q,Omega,delta) %#codegen
2 %myEigenvalues
3 % [E1,E2] = myEigenvalues(Q,Omega,DELTA)
4
5 % This function was generated by the Symbolic Math Toolbox version 25.1.
6 % 26-Dec-2024 10:31:19
7
8 t2 = Omega.^2;
9 t3 = delta.^2;
10 t4 = q.^2;
11 t6 = delta.*q.*8.0;
12 t5 = t2.*4.0;
13 t7 = t4.*1.6e+1;
14 t8 = t3+t5+t6+t7;
15 t9 = sqrt(t8);
16 t10 = t9./2.0;
17 E1 = t4-t10+1.0;
18 if nargin > 1
19     E2 = t4+t10+1.0;
20 end
21 end
22

```

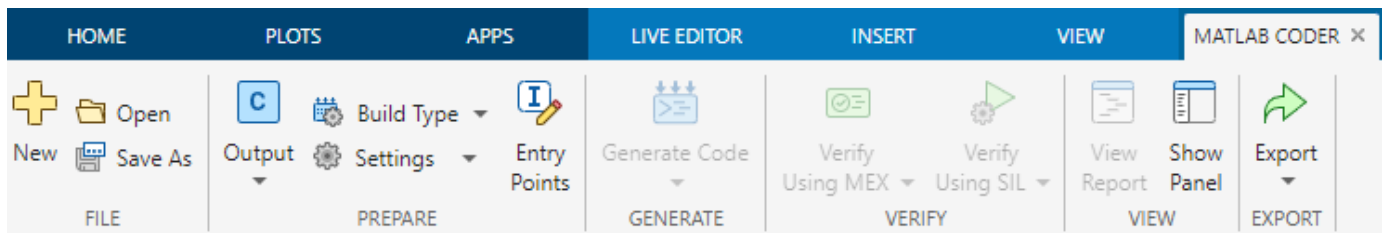
- The Code Analyzer message indicator in the top right corner of the MATLAB Editor is green. The analyzer did not detect errors, warnings, or opportunities for improvement in the code. For more information about using the Code Analyzer, see “Check Code for Errors and Warnings Using the Code Analyzer”.
- Save the file. You are now ready to compile your code by using the MATLAB Coder app. Here, *compilation* refers to the generation of C/C++ code from your MATLAB code.

### Open MATLAB Coder App and Create Project File

To generate code using the MATLAB Coder app, you must first create a MATLAB Coder project file (that has the extension `.coderprj`). This file contains the information that you provide to the code generator, including paths of MATLAB Coder entry-point functions, their input types, global variables, and the code generation configuration parameters.

- On the MATLAB toolstrip **Apps** tab, under **Code Generation**, click the **MATLAB Coder** app icon. The app opens the **Create MATLAB Coder Project** dialog box.
- Provide the name of the project file and the folder in which you want to place the file. For this example, create a file named `myEigenvalues.coderprj` in your current working folder.

A **MATLAB Coder** toolstrip opens that contains sections that correspond to the primary actions you perform when generating code, including preparing your MATLAB code for C/C++ code generation, generating code, performing verification on generated code, viewing the code generation report, and exporting the generated code.

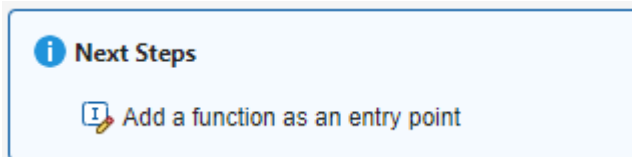


A code generation panel also opens that contains a **Next Steps** section which displays messages to guide you through the code generation steps, an **Input** section, and an **Output** section.

### Specify MATLAB Entry-Point Functions

An *entry-point function* is a top-level MATLAB function from which you generate code. For this example, your entry-point function is `myEigenvalues`.

In this state of the MATLAB Coder app, the **Next Steps** section of the code generation panel indicates that you must add an entry-point to the empty project you just created.



To add an entry-point function:

- Either click the **Add Entry Points** button in the **Inputs** section of the code generation panel or the **Entry Points** button in the toolbar.
- The **Entry Points** pane opens. Enter the name of your entry-point function `myEigenvalues`.

The app runs the Code Generation Readiness Tool on the entry-point function. This tool screens the MATLAB code for features and functions that are not supported for code generation. If the app identifies issues with an entry-point function or one of its dependencies, the **Entry Points** pane shows a warning message. Click the link next to the message to open the Code Generation Readiness Tool in a separate window where you can review and fix the issues. In this example, the app does not detect code generation readiness issues. For more information, see “Code Generation Readiness Tool” (MATLAB Coder).

Note that the Code Analyzer and the Code Generation Readiness Tool might not detect all code generation issues. After eliminating the errors or warnings that these two tools detect, generate code with MATLAB Coder to determine if your MATLAB code has other compliance issues.

Certain MATLAB functions and objects that are supported for C/C++ code generation have specific code generation limitations. These limitations and related usage notes are listed in the **Extended Capabilities** sections of their corresponding reference pages. For more information, see “Functions and Objects Supported for C/C++ Code Generation” (MATLAB Coder).

### Define Input Types

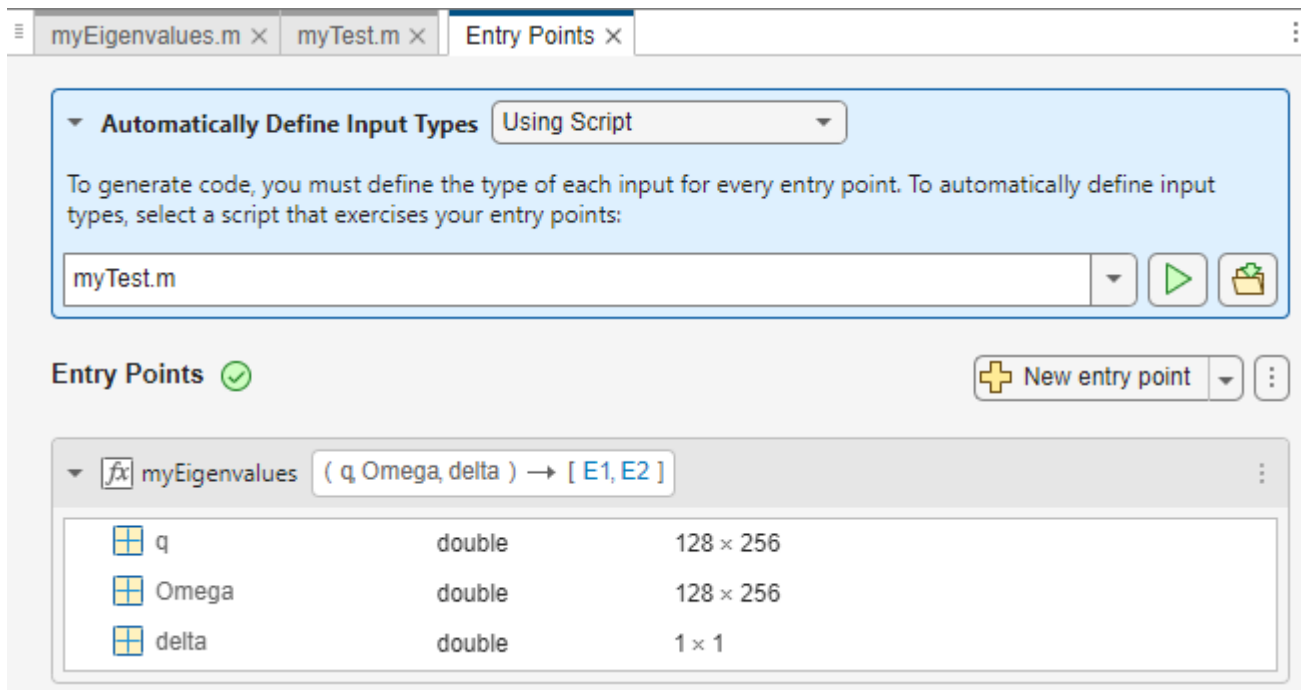
Because C/C++ uses static typing, the code generator must determine the class, size, and other properties (such as complexity) of all variables in the MATLAB files at code generation time, also known as *compile time*. Therefore, you must specify the properties of all entry-point function inputs. To specify input properties, you can:

- Instruct the app to automatically determine input properties by providing a script that calls the entry-point functions with sample inputs.
- Specify properties directly.

In this example, to define the properties of the inputs `q`, `Omega`, and `Delta`, specify the test file `myTest.m` that the code generator can use to define types automatically:

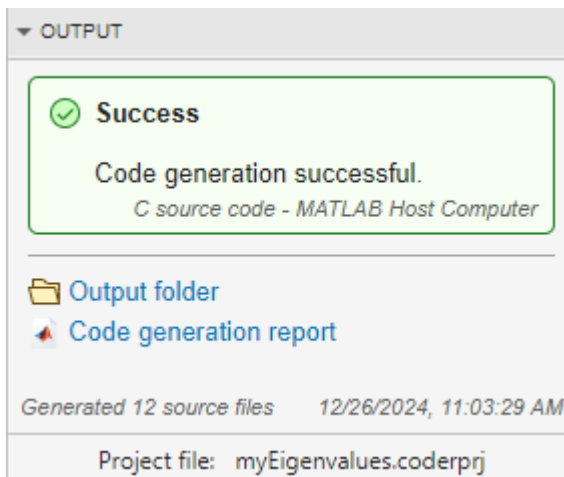
- In the **Entry Points** pane, set the **Automatically Define Input Types** parameter to **Using Script**.
- Enter or select the test file `myTest.m` and click the run button.

The test file, `myTest.m`, calls the entry-point function, `myEigenvalues`, with the expected input types. The app determines that the input `q` is a double 128-by-256 array, the input `Omega` is a double 128-by-256 array, and the input `delta` is a double 1-by-1 scalar.



### Generate C Source Code

- In the **Prepare** section of the MATLAB Coder toolstrip, make sure that **Output** is set to **C** and **Build type** is set to **Static Library (.lib)** (these are the default settings). Use the default values for the other code configuration settings as well.
- Click the **Generate Code** button either in the toolstrip or in the **Output** section of the code generation panel. MATLAB Coder generates C source files for your project in the `work\codegen\lib\myEigenvalues` folder, where `work` is the folder that contains your tutorial files. The **Output** section of the code generation panel indicates that code generation succeeded. This section also contains links to the generated output folder and the code generation report.



- Click the *Code Generation Report* link to view the report in the Report Viewer. If the code generator detects errors or warnings during code generation, the report describes the issues and provides links to the problematic MATLAB code. For more information, see “Code Generation Reports” (MATLAB Coder).

Clicking the **Generate Code** button generates source code only and does not build the source code to create binaries. To both generate source code and build binaries in a single step, open the **Generate Code** drop-down menu and select the **Generate Code and Build** option. Also, instead of generating a C static library, you can choose to generate a MEX function or other C/C++ build types. Different code configuration settings are available for the MEX and C/C++ build types. When you switch between MEX and C/C++ code generation, verify the settings that you choose.

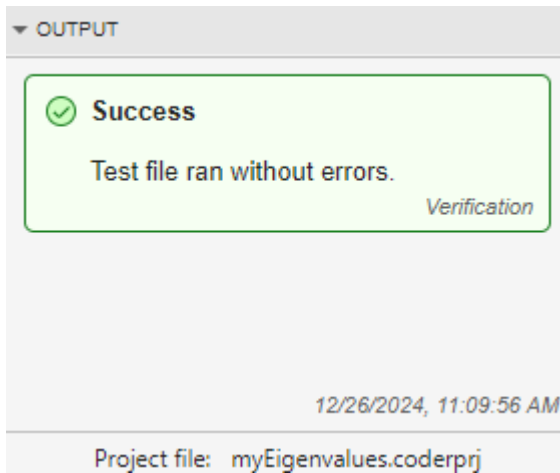
To convert MATLAB code to efficient C/C++ code, the code generator introduces optimizations that, in certain situations, cause the generated code to behave differently than the original MATLAB code. See “Differences Between Generated Code and MATLAB Code” (MATLAB Coder).

### Verify Using MEX

The **Verify Using MEX** toolstrip button generates a MEX function from your entry-point functions, runs the MEX function, and reports issues. A MEX function is generated code that can be called from inside MATLAB. It is a best practice to perform this step because you can detect and fix run-time errors that are harder to diagnose in the generated standalone C/C++ code. By default, the MEX function includes memory integrity checks. These checks perform array bounds and dimension checking. The checks detect violations of memory integrity in code generated for MATLAB functions. For more information, see “Control Run-Time Checks” (MATLAB Coder).

- In the MATLAB Coder toolstrip, open the **Verify Using MEX** drop-down menu.
- Specify a test file that calls the entry-point function with example inputs. For this example, use the test file `myTest.m` that you used to define the input types.

The app generates a MEX function. It runs the test script `myTest` replacing calls to `myEigenvalues` with calls to the generated MEX. If the app detects issues during the MEX function generation or execution, it provides warning and error messages in the Command Window. Click these messages to navigate to the problematic code and fix the issue. In this example, the app does not detect issues and shows this message in the **Output** section of the code generation panel.



Note that before generating standalone C/C++ code from your MATLAB code, generate a MEX function. Run the generated MEX function and make sure it has the same run-time behavior as your MATLAB function. If the generated MEX function produces answers that are different from MATLAB, or produces an error, you must fix these issues before proceeding to standalone code generation. Otherwise, the standalone code that you generate might be unreliable and have undefined behavior.

### Compare Generated C Code to Original MATLAB Code

To compare your generated C code to the original MATLAB code, open the C file, `myEigenvalues.c`, and the `myEigenvalues.m` file in the MATLAB Editor.

Important information about the generated C code:

- The function signature is:

```
void myEigenvalues(const double q[32768], const double Omega[32768],
                  double delta, double E1[32768], double E2[32768])
```

- `const double q[32768]` and `const double Omega[32768]` corresponds to the input `q` and `Omega` in your MATLAB code. The size of `q` is 32768, which corresponds to the total size (128 x 256) of the example input that you used when you generated C/C++ code from your MATLAB code. The same applies to the input `Omega`. In this case, the generated code uses one-dimensional arrays to represent two-dimensional arrays in the MATLAB code.
- The code generator preserves your function name and comments. When possible, the code generator preserves your variable names. Note that if a variable in your MATLAB code is set to a constant value, it does not appear as a variable in the generated C code. Instead, the generated C code contains the value of the variable.

### Generate C Code for Variable-Size Inputs

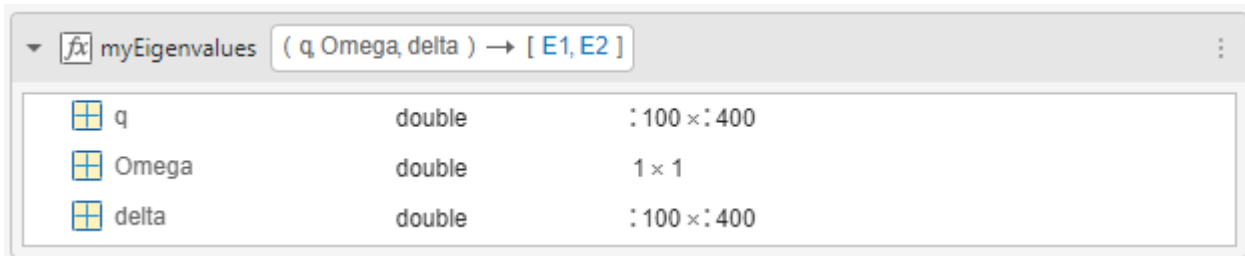
The C function that you generated for `myEigenvalues.m` can accept only inputs that have the same size as the sample inputs that you specified during code generation. However, the input arrays to the corresponding MATLAB function can be of any size. In this part of the example, you generate C code from `myEigenvalues.m` that accepts variable-size inputs.

Suppose that you want the dimensions of `q`, `Omega`, and `delta` in the generated C code to have these properties:

- The first dimension of both `q` and `delta` can vary in size up to 100.
- The second dimension of `q` and `delta` can vary in size up to 400.
- `Omega` is a scalar of size 1-by-1.

To specify these input properties:

- 1 In the **Entry Points** pane, enter the test file `myTest.m` and click **Automatically Define Input Types** as before. The test file calls the entry-point function, `myEigenvalues`, with the expected input types. The app determines that the input `q` is a double 128-by-256 array, the input `Omega` is a double 128-by-256 array, and the input `delta` is a double 1-by-1 scalar. These types specify fixed-size inputs.
- 2 Click the input type specifications to edit them. You can specify variable size, up to a specified limit, by using the `:` prefix. For example, `:100` specifies that the corresponding dimension can vary in size up to 100. Change the type for `q` to `double(:100 x :400)`, for `Omega` to `double(1 x 1)`, and for `delta` to `double(:100 x :400)`.



You can now generate code by following the same steps as before. The function signature for the generated C code in `myEigenvalues.c` now reads:

```
void myEigenvalues(const mxArray_real_T *q, double Omega,
                  const mxArray_real_T *delta, mxArray_real_T *E1,
                  mxArray_real_T *E2)
```

The arguments in the generated code correspond to these arguments in the original MATLAB function:

- `mxArray_real_T*q` — the `q` input argument
- `Omega` — the `Omega` input argument
- `mxArray_real_T*delta` — the `delta` input argument
- `mxArray_real_T*E1` — the `E1` output argument
- `mxArray_real_T*E2` — the `E2` output argument

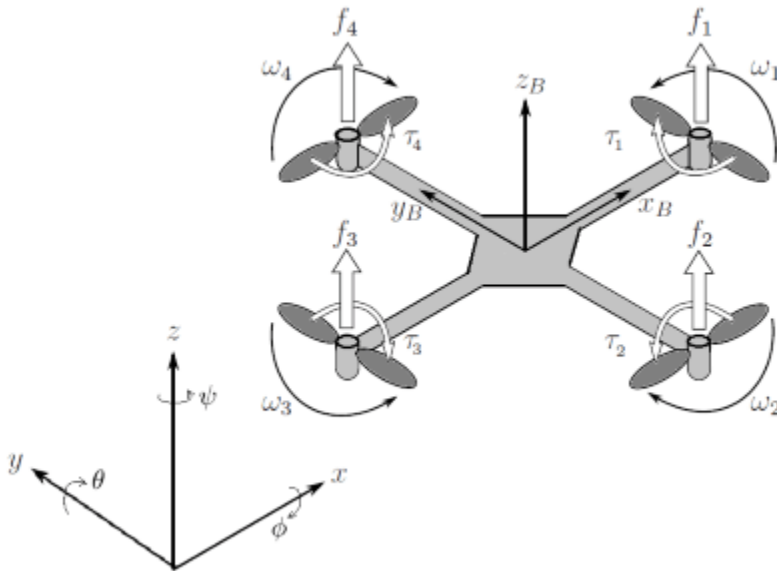
## Derive Quadrotor Dynamics for Nonlinear Model Predictive Control

This example shows how to derive a continuous-time nonlinear model of a quadrotor using Symbolic Math Toolbox™. Specifically, this example discusses the `getQuadrotorDynamicsAndJacobian` script, which generates both the quadrotor state function and its Jacobian function. These functions are used in the “Control of Quadrotor Using Nonlinear Model Predictive Control” (Model Predictive Control Toolbox) example.

A quadrotor, also known as a quadcopter, is a helicopter with four rotors. From the center of mass of the quadrotor, rotors are placed in a square formation at equal distances. The quadrotor motion is controlled by adjusting the angular velocities of the four rotors, which are spun by electric motors. The mathematical model for the quadrotor dynamics can be derived from both the Newton-Euler and the Euler-Lagrange equations [1].

### Define State Variables and Parameters

The quadrotor has six degrees of freedom (three linear coordinates and three angular coordinates) with four control inputs as shown in this figure.



The 12 states of the quadrotor are

$$(x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \dot{\phi}, \dot{\theta}, \dot{\psi})^T,$$

where:

- $\xi = (x, y, z)^T$  represents the linear positions.
- $\eta = (\phi, \theta, \psi)^T$  represents the roll, pitch, and yaw angles, respectively.



- $(\dot{x}, \dot{y}, \dot{z}, \dot{\phi}, \dot{\theta}, \dot{\psi})^T$  represents the linear and angular velocities or the time-derivative of the linear and angular positions.

For the parameters of the quadrotor motion:

- $(u_1, u_2, u_3, u_4) = (\omega_1^2, \omega_2^2, \omega_3^2, \omega_4^2)$  represents the squared angular velocities of the four rotors.
- $(I_{xx}, I_{yy}, I_{zz})$  represents the diagonal elements of the rotational inertia matrix in the body frame.
- $(k, l, m, b, g)$  represents the lift constant, rotor distance from center of mass, quadrotor mass, drag constant, and gravity.

The first four parameters  $(u_1, u_2, u_3, u_4)$  are control inputs or manipulated variables that control the quadrotor trajectory. The rest of the parameters are fixed with given values.

### Define Transformation Matrices and Coriolis Matrix

Define the matrices to transform between the inertial frame and the body frame. These transformation matrices are needed to describe the quadrotor motion in both frames. The 12 states of the quadrotor are defined in the inertial frame and the rotational inertia matrix is defined in the body frame.

Create symbolic functions to represent the time-dependent angles. This explicit time dependence is needed to evaluate the time derivatives in the mathematical derivation following [1]. Define the matrix  $\mathbf{W}_\eta$  to transform the angular velocities from the inertial frame to the body frame. Define the rotation matrix  $\mathbf{R}$  to transform the linear forces from the body frame to the inertial frame by using the `rotationMatrixEulerZYX` function defined in the Local Functions section. Create symbolic matrices to represent these transformation matrices.

```
% Create symbolic functions for time-dependent angles
% phi: roll angle
% theta: pitch angle
% psi: yaw angle
syms phi(t) theta(t) psi(t)

% Transformation matrix for angular velocities from inertial frame
% to body frame
W = [ 1, 0, -sin(theta);
      0, cos(phi), cos(theta)*sin(phi);
      0, -sin(phi), cos(theta)*cos(phi) ];

% Rotation matrix R_ZYX from body frame to inertial frame
R = rotationMatrixEulerZYX(phi, theta, psi);
```

Find the Jacobian matrix  $\mathbf{J} = \mathbf{W}_\eta^T \mathbf{I} \mathbf{W}_\eta$  that is used to express the rotational energy in the inertial frame.

```
% Create symbolic variables for diagonal elements of inertia matrix
syms Ixx Iyy Izz

% Jacobian that relates body frame to inertial frame velocities
I = [Ixx, 0, 0; 0, Iyy, 0; 0, 0, Izz];
J = W.'*I*W;
```

Next, find the Coriolis matrix  $\mathbf{C}(\eta, \dot{\eta}) = \frac{d}{dt} \mathbf{J} - \frac{1}{2} \frac{\partial}{\partial \eta} (\dot{\eta}^T \mathbf{J})$  that contains the gyroscopic and centripetal terms in the angular Euler-Lagrange equations. Use the symbolic `diff` and `jacobian` functions to

perform the differential operations. To simplify the notation for the Coriolis matrix after differentiations, substitute the explicit time-dependent terms in  $\mathbf{C}(\eta, \dot{\eta})$  with symbolic variables (that represent some values at a specific time). This simplified notation makes it easier to compare the result here with the derivation in [1].

```
% Coriolis matrix
dJ_dt = diff(J);
h_dot_J = [diff(phi,t), diff(theta,t), diff(psi,t)]*J;
grad_temp_h = transpose(jacobian(h_dot_J,[phi theta psi]));
C = dJ_dt - 1/2*grad_temp_h;
C = subsStateVars(C,t);
```

### Prove That Coriolis Matrix Definition Is Consistent

The Coriolis matrix  $\mathbf{C}(\eta, \dot{\eta})$  is easily derived using the symbolic workflow in the previous section. However, the definition of  $\mathbf{C}(\eta, \dot{\eta})$  here is different than in [1], which uses the Christoffel symbols to derive the Coriolis matrix. There can be multiple ways of defining  $\mathbf{C}(\eta, \dot{\eta})$  because it is nonunique. But, the term  $\mathbf{C}(\eta, \dot{\eta}) \dot{\eta}$  that is used in the Euler-Lagrange equations must be unique. This section proves that the symbolic definition of  $\mathbf{C}(\eta, \dot{\eta})$  is consistent with the definition in [1] by showing that the term  $\mathbf{C}(\eta, \dot{\eta}) \dot{\eta}$  is mathematically equal in both definitions.

Evaluate the term  $\mathbf{C}(\eta, \dot{\eta}) \dot{\eta}$  using the symbolic definition of  $\mathbf{C}(\eta, \dot{\eta})$  derived in the previous section.

```
% Evaluate C times etadot using symbolic definition
phidot = subsStateVars(diff(phi,t),t);
thetadot = subsStateVars(diff(theta,t),t);
psidot = subsStateVars(diff(psi,t),t);
Csym_etadot = C*[phidot; thetadot; psidot];
```

Evaluate the term  $\mathbf{C}(\eta, \dot{\eta}) \dot{\eta}$  using the definition of  $\mathbf{C}(\eta, \dot{\eta})$  as derived in [1].

```
% Evaluate C times etadot using reference paper definition
C11 = 0;
C12 = (Iyy - Izz)*(thetadot*cos(phi)*sin(phi) + psidot*sin(phi)^2*cos(theta)) ...
      + (Izz - Iyy)*(psidot*cos(phi)^2*cos(theta)) - Ixx*psidot*cos(theta);
C13 = (Izz - Iyy)*psidot*cos(phi)*sin(phi)*cos(theta)^2;
C21 = (Izz - Iyy)*(thetadot*cos(phi)*sin(phi) + psidot*sin(phi)^2*cos(theta)) ...
      + (Iyy - Izz)*psidot*cos(phi)^2*cos(theta) + Ixx*psidot*cos(theta);
C22 = (Izz - Iyy)*phidot*cos(phi)*sin(phi);
C23 = - Ixx*psidot*sin(theta)*cos(theta) + Iyy*psidot*sin(phi)^2*sin(theta)*cos(theta) ...
      + Izz*psidot*cos(phi)^2*sin(theta)*cos(theta);
C31 = (Iyy - Izz)*psidot*cos(theta)^2*sin(phi)*cos(phi) - Ixx*thetadot*cos(theta);
C32 = (Izz - Iyy)*(thetadot*cos(phi)*sin(phi)*sin(theta) + phidot*sin(phi)^2*cos(theta)) ...
      + (Iyy - Izz)*phidot*cos(phi)^2*cos(theta) + Ixx*psidot*sin(theta)*cos(theta) ...
      - Iyy*psidot*sin(phi)^2*sin(theta)*cos(theta) - Izz*psidot*cos(phi)^2*sin(theta)*cos(theta);
C33 = (Iyy - Izz)*phidot*cos(phi)*sin(phi)*cos(theta)^2 ...
      - Iyy*thetadot*sin(phi)^2*cos(theta)*sin(theta) ...
      - Izz*thetadot*cos(phi)^2*cos(theta)*sin(theta) + Ixx*thetadot*cos(theta)*sin(theta);
Cpaper = [C11, C12, C13; C21, C22, C23; C31, C32, C33];
Cpaper_etadot = Cpaper*[phidot; thetadot; psidot];
Cpaper_etadot = subsStateVars(Cpaper_etadot,t);
```

Show that both definitions give the same result for the term  $\mathbf{C}(\eta, \dot{\eta}) \dot{\eta}$ .

```
tf_Cdefinition = isAlways(Cpaper_etadot == Csym_etadot)
tf_Cdefinition = 3x1 logical array
```

1  
1  
1  
1

### Find Equations of Motion

Find the equations of motion for the 12 states.

Following [1], find the torques  $\tau_B$  in the roll, pitch, and yaw angular directions:

- The roll movement is set by decreasing the 2nd rotor velocity and increasing the 4th rotor velocity.
- The pitch movement is set by decreasing the 1st rotor velocity and increasing the 3rd rotor velocity.
- The yaw movement is set by increasing the velocities of two opposite rotors and decreasing the velocities of the other two.

Find the total thrust of the rotors  $T$  in the direction of the body  $z$ -axis.

```
% Define fixed parameters and control input
% k: lift constant
% l: distance between rotor and center of mass
% m: quadrotor mass
% b: drag constant
% g: gravity
% ui: squared angular velocity of rotor i as control input
syms k l m b g u1 u2 u3 u4

% Torques in the direction of phi, theta, psi
tau_beta = [l*k*(-u2+u4); l*k*(-u1+u3); b*(-u1+u2-u3+u4)];

% Total thrust
T = k*(u1+u2+u3+u4);
```

Next, create symbolic functions to represent the time-dependent positions. Define the 12 states  $[\xi; \eta; \ddot{\xi}; \ddot{\eta}]$  for the linear positions, angles, and their time derivatives. Once the states are defined, substitute the explicit time-dependent terms to simplify the notation.

```
% Create symbolic functions for time-dependent positions
syms x(t) y(t) z(t)

% Create state variables consisting of positions, angles,
% and their derivatives
state = [x; y; z; phi; theta; psi; diff(x,t); diff(y,t); ...
        diff(z,t); diff(phi,t); diff(theta,t); diff(psi,t)];
state = subsStateVars(state,t);
```

Create the 12 equations of motion that describe the time-derivative of the 12 states

$f = [\ddot{\xi}; \ddot{\eta}; \ddot{\xi}; \ddot{\eta}]$ . The differential equations for the linear accelerations are

$\ddot{\xi} = -(0, 0, g)^T + \mathbf{R}(0, 0, T)^T/m$ . The differential equations for the angular accelerations are  $\ddot{\eta} = \mathbf{J}^{-1}(\tau_B - \mathbf{C}(\eta, \dot{\eta}) \dot{\eta})$ . Substitute the explicit time-dependent terms to simplify the notation.

```
f = [ % Set time-derivative of the positions and angles
      state(7:12);
```

```

    % Equations for linear accelerations of the center of mass
    -g*[0;0;1] + R*[0;0;T]/m;

    % Euler-Lagrange equations for angular dynamics
    inv(J)*(tau_beta - C*state(10:12))
];

f = subsStateVars(f,t);

```

In previous steps, the fixed parameters are defined as symbolic variables to closely follow the derivation in [1]. Next, replace the fixed parameters with their given values. These values are used to design a trajectory tracking controller for a specific configuration of the quadrotor. After substituting the fixed parameters, perform algebraic simplification on the equations of motion by using `simplify`.

```

% Replace fixed parameters with given values here
IxxVal = 1.2;
IyyVal = 1.2;
IzzVal = 2.3;
kVal = 1;
lVal = 0.25;
mVal = 2;
bVal = 0.2;
gVal = 9.81;

f = subs(f, [Ixx Iyy Izz k l m b g], ...
    [IxxVal IyyVal IzzVal kVal lVal mVal bVal gVal]);
f = simplify(f);

```

### Find Jacobians and Generate Files for Nonlinear Model Predictive Control

Finally, find the analytical Jacobians of the nonlinear model functions and generate MATLAB® files using Symbolic Math Toolbox. This step is important to improve the computational efficiency when solving the nonlinear model for trajectory tracking using Model Predictive Control Toolbox™. For more information, see `nlmpc` (Model Predictive Control Toolbox) and “Specify Prediction Model for Nonlinear MPC” (Model Predictive Control Toolbox).

Find the Jacobians of the state function with respect to the state variables and control inputs.

```

% Calculate Jacobians for nonlinear prediction model
A = jacobian(f,state);
control = [u1; u2; u3; u4];
B = jacobian(f,control);

```

Generate MATLAB files for the state function and the Jacobians of the state function. These files can be used to design a trajectory tracking controller, as discussed in “Control of Quadrotor Using Nonlinear Model Predictive Control” (Model Predictive Control Toolbox).

```

% Create QuadrotorStateFcn.m with current state and control
% vectors as inputs and the state time-derivative as outputs
matlabFunction(f,'File','QuadrotorStateFcn', ...
    'Vars',{state,control});

% Create QuadrotorStateJacobianFcn.m with current state and control
% vectors as inputs and the Jacobians of the state time-derivative
% as outputs
matlabFunction(A,B,'File','QuadrotorStateJacobianFcn', ...
    'Vars',{state,control});

```

You can access the code in this script in the file `getQuadrotorDynamicsAndJacobian.m`.

### Local Functions

```
function [Rz,Ry,Rx] = rotationMatrixEulerZYX(phi,theta,psi)
% Euler ZYX angles convention
    Rx = [ 1,      0,      0;
           0,      cos(phi), -sin(phi);
           0,      sin(phi),  cos(phi) ];
    Ry = [ cos(theta), 0,      sin(theta);
           0,          1,      0;
           -sin(theta), 0,      cos(theta) ];
    Rz = [ cos(psi), -sin(psi), 0;
           sin(psi),  cos(psi), 0;
           0,         0,        1 ];
    if nargin == 3
        % Return rotation matrix per axes
        return;
    end
    % Return rotation matrix from body frame to inertial frame
    Rz = Rz*Ry*Rx;
end

function stateExpr = subsStateVars(timeExpr,var)
    if nargin == 1
        var = sym("t");
    end
    repDiff = @(ex) subsStateVarsDiff(ex,var);
    stateExpr = mapSymType(timeExpr,"diff",repDiff);
    repFun = @(ex) subsStateVarsFun(ex,var);
    stateExpr = mapSymType(stateExpr,"symfun0f",var,repFun);
    stateExpr = formula(stateExpr);
end

function newVar = subsStateVarsFun(funExpr,var)
    name = symFunType(funExpr);
    name = replace(name,"_Var","");
    stateVar = "_" + char(var);
    newVar = sym(name + stateVar);
end

function newVar = subsStateVarsDiff(diffExpr,var)
    if nargin == 1
        var = sym("t");
    end
    c = children(diffExpr);
    if ~isSymType(c{1},"symfun0f",var)
        % not f(t)
        newVar = diffExpr;
        return;
    end
    if ~any([c{2:end}] == var)
        % not derivative wrt t only
        newVar = diffExpr;
        return;
    end
    name = symFunType(c{1});
    name = replace(name,"_Var","");
```

```
extension = "_" + join(repelem("d",numel(c)-1),"") + "ot";  
stateVar = "_" + char(var);  
newVar = sym(name + extension + stateVar);  
end
```

## References

[1] Raffo, Guilherme V., Manuel G. Ortega, and Francisco R. Rubio. "An integral predictive/nonlinear  $\mathcal{H}_\infty$  control structure for a quadrotor helicopter". *Automatica* 46, no. 1 (January 2010): 29–39. <https://doi.org/10.1016/j.automatica.2009.10.018>.

[2] Tzorakoleftherakis, Emmanouil, and Todd D. Murphey. "Iterative sequential action control for stable, model-based control of nonlinear systems." *IEEE Transactions on Automatic Control* 64, no. 8 (August 2019): 3170–83. <https://doi.org/10.1109/TAC.2018.2885477>.

[3] Luukkonen, Teppo. "Modelling and control of quadcopter". Independent research project in applied mathematics, Aalto University, 2011.

## See Also

`diff` | `jacobian` | `matlabFunction`

## Related Examples

- "Control of Quadrotor Using Nonlinear Model Predictive Control" (Model Predictive Control Toolbox)
- "Specify Prediction Model for Nonlinear MPC" (Model Predictive Control Toolbox)

# Autonomous Underwater Vehicle Pose Estimation Using Analytical Jacobians in Custom Sensor Models

This example shows how to estimate the pose of an autonomous underwater vehicle (AUV) by using analytical Jacobians in custom sensor models. In this example, you define and tune an extended Kalman filter to fuse noisy sensor data, and then estimate and visualize the vehicle trajectory. You also compare pose estimates with smoothed estimates and analyze corresponding position errors.

## Define Problem Statement for AUV Pose Estimation

To estimate the position and orientation of an AUV, you use an extended Kalman filter (EKF) [1] on page 5-42 to fuse a vehicle motion model with noisy sensor measurements. The EKF is the nonlinear counterpart of the Kalman filter [2] on page 5-42. The EKF represents uncertainty about the vehicle state with a normal (Gaussian) distribution described by two quantities: a mean  $\mathbf{x}$  of the current best estimate and a covariance matrix  $\mathbf{P}$  of the associated uncertainty. The state can include position  $\mathbf{r}$ , velocity  $\mathbf{v}$ , orientation  $\mathbf{q}$ , and other navigation variables.

In the "predict" or propagate step, a nonlinear dynamics model advances the vehicle state following the formula:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, \mathbf{u}) + \text{noise},$$

where  $\mathbf{u}$  is the control input and  $\mathbf{f}$  is the state dynamics motion model. The EKF linearizes this model around the current estimate using the Jacobian  $\mathbf{F} = \partial \mathbf{f} / \partial \mathbf{x}$  to propagate the covariance matrix  $\mathbf{P}$ .

In the "correct" or update step, you compare measurements  $\mathbf{z}$  with the corresponding measurement model, which is evaluated using the current state estimate  $\mathbf{h}(\mathbf{x})$ . You compute the Kalman gain  $\mathbf{K}$  by using the linearized measurement model  $\mathbf{H} = \partial \mathbf{h} / \partial \mathbf{x}$ . To correct the state and reduce uncertainty, the Kalman gain weights the residual  $\mathbf{z} - \mathbf{h}(\mathbf{x})$ . A typical discrete update step is:

$$\hat{\mathbf{x}}^+ = \hat{\mathbf{x}}^- + \mathbf{K} (\mathbf{z} - \mathbf{h}(\hat{\mathbf{x}}^-)),$$

where  $\hat{\mathbf{x}}^-$  is the predicted state (just prior to the update) and  $\hat{\mathbf{x}}^+$  is the corrected state (just after the update). For this reason, accurate Jacobians  $\mathbf{F}$  and  $\mathbf{H}$  are crucial for stability, convergence, and computational efficiency.

When estimating the pose of a vehicle, such as a submarine, you can create custom motion and sensor models to use with the `inSEKF` (Navigation Toolbox) object in Navigation Toolbox™. You create these models by using the base classes `positioning.INSMotionModel` and `positioning.INSSensorModel`. In these custom models, you can provide analytical measurement Jacobians to improve accuracy and efficiency of the pose estimation, instead of relying on default numerical differentiation.

To complement the built-in inertial measurement unit (IMU), magnetometer, GPS, and motion models, this example defines three custom `positioning.INSSensorModel` subclasses:

- `insDepth` — A depth sensor model that measures the vehicle's down position (trivial identity Jacobian)
- `insSpeed` — A flow-relative speed sensor model that projects the vehicle's fluid-relative velocity onto the body x-axis (nontrivial Jacobian with respect to orientation, velocity, and flow)

- `insZuptVyz` — A two-axis "zero-velocity update" (ZUPT) sensor model that expects body-frame  $y$  and  $z$  velocities to be zero (nontrivial Jacobian with respect to orientation and velocity)

You provide the measurement Jacobian  $\mathbf{H}$  in these three custom sensor models for the EKF update step.

While this example enforces a two-axis ZUPT sensor model, you could add a custom motion model to directly enforce a "no side slip" constraint on the vehicle dynamics. In this case, you would provide the appropriate motion model Jacobian  $\mathbf{F}$ , which is used in the EKF predict step to propagate the state error covariance. Similarly, this approach can be extended to incorporate any other customizations you want to add to the motion model.

To compute Jacobians analytically, you can use Symbolic Math Toolbox™. In this example, you will symbolically derive expressions for the following quantities:

- The navigation-to-body-frame transformation matrix from the quaternion
- The body-frame velocity transformed from the navigation-frame (with and without fluid flow)
- The corresponding measurement Jacobians needed by `insSpeed` and `insZuptVyz`

From these symbolic expressions, you generate code for MATLAB® functions. You can use these functions in custom classes without requiring Symbolic Math Toolbox at run time. You integrate the generated Jacobian functions (`SpeedJacobian` and `ZuptVyzJacobian`) into the custom sensor classes to provide analytic measurement Jacobians to `insEKF`.

With these models, you can simulate an AUV dive-and-surface trajectory. You fuse IMU, magnetometer, and GPS readings with depth, flow-relative speed, and ZUPT measurements in the `insEKF` object. Finally, you evaluate estimation accuracy along the trajectory, and apply smoothing to further refine the pose estimates.

### Derive Quaternion, Rotation Matrix, and Jacobians

Using Symbolic Math Toolbox, you can derive a rotation matrix from a quaternion, perform velocity transformations from the navigation frame to the body frame, and compute Jacobians of the velocity measurements.

#### Define Quaternion and Rotation Matrix

First, define a unit quaternion, which consists of a scalar part  $q_0$  and a 3-by-1 vector part

$$\mathbf{q} = (q_1, q_2, q_3)^T.$$

```
syms q0 real
syms q [3 1] real
qSym = [q0; q]
```

$$qSym = \begin{pmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{pmatrix}$$

To represent the cross product  $[\mathbf{q}]_x = \mathbf{q} \times$ , generate a skew-symmetric matrix for the vector part of the quaternion.

```
qskew = cross([q q q],eye(3))
```



$$\text{qskew} = \begin{pmatrix} 0 & -q_3 & q_2 \\ q_3 & 0 & -q_1 \\ -q_2 & q_1 & 0 \end{pmatrix}$$

Compute the rotation matrix from the navigation frame to the body frame using this formula [3] on page 5-42:

$$T_{\text{nav}}^{\text{body}} = (\mathbf{q} \otimes \mathbf{q} + q_0^2 \mathbf{I} + 2q_0 [\mathbf{q}]_{\times} + [\mathbf{q}]_{\times}^2)^T$$

$$\text{Tn2bsyms} = ((\mathbf{q} * \mathbf{q}') + q_0^2 * \text{eye}(3) + 2 * q_0 * \text{qskew} + \text{qskew}^2)'$$

$$\text{Tn2bsyms} = \begin{pmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2q_0q_3 + 2q_1q_2 & 2q_1q_3 - 2q_0q_2 \\ 2q_1q_2 - 2q_0q_3 & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2q_0q_1 + 2q_2q_3 \\ 2q_0q_2 + 2q_1q_3 & 2q_2q_3 - 2q_0q_1 & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{pmatrix}$$

### Compute Jacobian for Flow-Relative Velocity in Body Frame

Next, define 3-by-1 symbolic vectors for the vehicle velocity in the navigation frame  $\mathbf{v}$  and the flow velocity  $\mathbf{v}_{\text{flow}}$ .

```
syms v v_flow [3 1] real
```

To calculate the flow-relative velocity of the vehicle, transform the vehicle velocity from the navigation frame to the body frame using this formula:

$$\mathbf{h} = T_{\text{nav}}^{\text{body}} (\mathbf{v} - \mathbf{v}_{\text{flow}})$$

$$\mathbf{vb} = \text{Tn2bsyms} * (\mathbf{v} - \mathbf{v}_{\text{flow}})$$

$$\mathbf{vb} = \begin{pmatrix} (v_1 - v_{\text{flow}1})(q_0^2 + q_1^2 - q_2^2 - q_3^2) + (v_2 - v_{\text{flow}2})(2q_0q_3 + 2q_1q_2) - (v_3 - v_{\text{flow}3})(2q_0q_2 - 2q_1q_3) \\ (v_2 - v_{\text{flow}2})(q_0^2 - q_1^2 + q_2^2 - q_3^2) - (v_1 - v_{\text{flow}1})(2q_0q_3 - 2q_1q_2) + (v_3 - v_{\text{flow}3})(2q_0q_1 + 2q_2q_3) \\ (v_3 - v_{\text{flow}3})(q_0^2 - q_1^2 - q_2^2 + q_3^2) + (v_1 - v_{\text{flow}1})(2q_0q_2 + 2q_1q_3) - (v_2 - v_{\text{flow}2})(2q_0q_1 - 2q_2q_3) \end{pmatrix}$$

Compute the Jacobians of the flow-relative velocity with respect to the quaternion, the vehicle velocity, and the flow velocity.

```
Jq = jacobian(vb,qSym);
Jv = jacobian(vb,v);
Jflow = jacobian(vb,v_flow);
```

Because the sensor model for the flow-relative velocity requires only the x-components, convert the first row of these Jacobians to a MATLAB function file using `matlabFunction`. Specify the name of the generated file as `SpeedJacobian`. Specify the `Vars` name-value argument as a cell array, such that the generated input arguments are a 4-by-1 vector variable `qSym`, a 3-by-1 vector variable `v`, and a 3-by-1 vector variable `v_flow`.

```
h = matlabFunction(Jq(1,:),Jv(1,:),Jflow(1,:),File="SpeedJacobian",Vars={qSym,v,v_flow});
```

### Compute Jacobian for Zero-Velocity Update

Finally, transform the vehicle velocity without flow and compute Jacobians for ZUPT. The velocity follows this formula:

$$h = T_{\text{nav}}^{\text{body}} \mathbf{v}$$

```
vb = Tn2bsyms*v;
Jq = jacobian(vb,qSym);
Jv = jacobian(vb,v);
```

Unlike flow-relative speed sensors, zero-velocity updates are used to correct drift in the estimated velocity when the vehicle is known to be stationary or not moving in certain directions. In this model, no fluid flow term is needed because the ZUPT is concerned only with the vehicle velocity, not the vehicle motion relative to the surrounding fluid.

Because the sensor model for ZUPT requires only the y- and z-components, convert the last two rows of these Jacobians to a MATLAB function file using `matlabFunction`. Specify the name of the generated file as `ZuptVyzJacobian`. Specify the `Vars` name-value argument as a cell array, such that the generated input arguments are a 4-by-1 vector variable `qSym` and a 3-by-1 vector variable `v`.

```
h = matlabFunction(Jq(2:3,:),Jv(2:3,:),File="ZuptVyzJacobian",Vars={qSym,v});
```

### Create Custom Sensor Models

Now that you have the analytical expressions for the Jacobians, you can implement efficient, EKF-ready sensor models that use these Jacobians directly. In this section, you define custom depth, speed, and ZUPT sensor models by using functionality in Navigation Toolbox. The `insDepth` class has a trivial Jacobian, which is an identity matrix. The `insSpeed` class uses the generated `SpeedJacobian` for its measurement Jacobian, and `insZuptVyz` uses the generated `ZuptVyzJacobian` for its measurement Jacobian. This section discusses how to expose measurement functions, provide analytic measurement Jacobians, and define simple state transitions (where applicable).

#### Create Model for Depth Sensor Measurements

To model the depth sensor measurements for sensor fusion, create the `insDepth` class as a subclass of the `positioning.INSSensorModel` class. This class implements the depth measurement function and its Jacobian, which is an identity matrix. The `measurement` method returns the depth measurement, or the z-component of the position state. The `measurementJacobian` method returns the Jacobian, or partial derivative, of the depth measurement with respect to each state variable. This Jacobian returns a row vector of zeros with a 1 at the position corresponding to the z-component.

Display the contents of the `insDepth` class.

type `insDepth.m`

```
classdef insDepth < positioning.INSSensorModel
    % INSDEPTH Model depth sensor reading for sensor fusion
    %
    % See also: insEKF, insGPS, positioning.INSSensorModel

    methods
        function h = measurement(~,filt)
            idx = stateinfo(filt);
            h = filt.State(idx.Position(3));
```

```

end

function dhdx = measurementJacobian(~,filt)
    idx = stateinfo(filt);
    dhdx = zeros(1,numel(filt.State),Like=filt.State);
    dhdx(1,idx.Position(3)) = 1;
end
end
end

```

### Create Model for Flow-Relative Speed Measurements

To model the flow-relative speed sensor measurements for sensor fusion, create the `insSpeed` class as a subclass of the `positioning.INSSensorModel` class. This class implements the speed sensor measurement, its Jacobian, and state transition functions. The `insSpeed` class introduces a state variable `FluidFlow` that represents the 3-D fluid velocity in the navigation frame. This sensor measures the vehicle velocity relative to the surrounding fluid, projected onto the body x-axis. The `measurementJacobian` method computes the partial derivatives of the measurement with respect to the state variables (orientation, velocity, and fluid flow). The Jacobian calculation is implemented using the helper function `SpeedJacobian`, which was derived in the previous section. The fluid flow is assumed to be constant, so its time derivative or state transition is zero. Therefore, the Jacobian of the state transition with respect to the state is also zero for the `FluidFlow` state.

Display the contents of the `insSpeed` class.

type `insSpeed.m`

```

classdef insSpeed < positioning.INSSensorModel
    % INSSPEED Model flow-relative speed sensor reading for sensor fusion
    %
    % See also: insEKF, insGPS, positioning.INSSensorModel

    methods
        function s = sensorstates(~,~)
            s = struct("FluidFlow",[0;0;0]);
        end

        function h = measurement(~,filt)
            q = stateparts(filt,"Orientation")'; % 4-by-1 quaternion nav to body frame
            v = stateparts(filt,"Velocity")'; % 3-by-1 velocity in nav frame
            flow = stateparts(filt,"insSpeed_FluidFlow")'; % 3-by-1 fluid in nav frame

            quat = quaternion(q(1),q(2),q(3),q(4));
            Tn2b = rotmat(quat,"frame"); % nav to body frame
            h = Tn2b(1,:)*(v - flow); % x-body axis, vehicle speed relative to fluid flow a

        end

        function dhdx = measurementJacobian(~,filt)
            q = stateparts(filt,"Orientation")'; % 4-by-1 quaternion nav to body frame
            v = stateparts(filt,"Velocity")'; % 3-by-1 velocity in nav frame
            flow = stateparts(filt,"insSpeed_FluidFlow")'; % 3-by-1 fluid in nav frame

            qidx = stateinfo(filt,"Orientation");
            vidx = stateinfo(filt,"Velocity");
            flowidx = stateinfo(filt,"insSpeed_FluidFlow");

            % Compute the Jacobian using SpeedJacobian,

```

```

        % which is generated using Symbolic Math Toolbox
        dhdx = zeros(1,numel(filt.State),Like=filt.State);
        [dhdx(1,qidx),dhdx(1,vidx),dhdx(1,flowidx)] = SpeedJacobian(q,v,flow);
    end

    function dxdt = stateTransition(~,filt,~,varargin)
        % dx/dt = f(x,t) + w(t)
        % with state x, state dynamical model f, and process noise w
        % FluidFlow assumed constant
        dxdt = struct("FluidFlow",zeros(1,3,Like=filt.State));
    end

    function dfdx = stateTransitionJacobian(~,filt,~,varargin)
        % State Dynamics Jacobian: F = df/dx, where dPhi/dt = F*Phi
        % FluidFlow is not a function of other state variables
        n = numel(filt.State);
        dfdx = struct("FluidFlow",zeros(3,n,Like=filt.State));
    end
end
end
end

```

### Create Model for Zero-Velocity Update

To model the ZUPT for sensor fusion, create the `insZuptVyz` class as a subclass of the `positioning.INSSensorModel` class. This synthetic sensor incorporates the expectation that the vehicle velocity along the y- and z-axes of the body frame are effectively zero. The `measurementJacobian` method computes the partial derivatives of the measurement with respect to the orientation and velocity state variables. The Jacobian calculation is implemented using the helper function `ZuptVyzJacobian`, which was derived in the previous section.

Display the contents of the `insZuptVyz` class.

type `insZuptVyz.m`

```

classdef insZuptVyz < positioning.INSSensorModel
    % INSZUPTVYZ Synthetic sensor for zero-velocity update along y/z body axes
    %
    % See also: insEKF, insGPS, positioning.INSSensorModel

    methods
        function h = measurement(~,filt)
            q = stateparts(filt,"Orientation"); % 4-by-1 quaternion nav to body frame
            v = stateparts(filt,"Velocity"); % 3-by-1 velocity in nav frame

            quat = quaternion(q(1),q(2),q(3),q(4));
            Tn2b = rotmat(quat,"frame"); % 3-by-3 nav to body frame transform
            h = (Tn2b(2:3,:)*v)'; % 1-by-2 velocity along y/z-body axis
        end

        function dhdx = measurementJacobian(~,filt)
            % Measurement Model Jacobian: H = dh/dx
            q = stateparts(filt,"Orientation"); % 4-by-1 quaternion nav to body frame
            v = stateparts(filt,"Velocity"); % 3-by-1 velocity in nav frame

            qidx = stateinfo(filt,"Orientation");
            vidx = stateinfo(filt,"Velocity");

            dhdx = zeros(2,numel(filt.State),Like=filt.State);
        end
    end
end

```

```

        % Compute the Jacobian using ZuptVyzJacobian,
        % which is generated using Symbolic Math Toolbox
        [dhdx(:,qid),dhdx(:,vid)] = ZuptVyzJacobian(q,v);
    end
end
end

```

## Simulate Trajectory of Autonomous Underwater Vehicle

In this section, you use the ground-truth trajectory of an AUV to design an EKF, and then you use the EKF to estimate the AUV pose using multiple sensor inputs. To do so, you load ground-truth states, create the EKF, load prerecorded sensor data, tune the filter, and estimate and visualize the vehicle state.

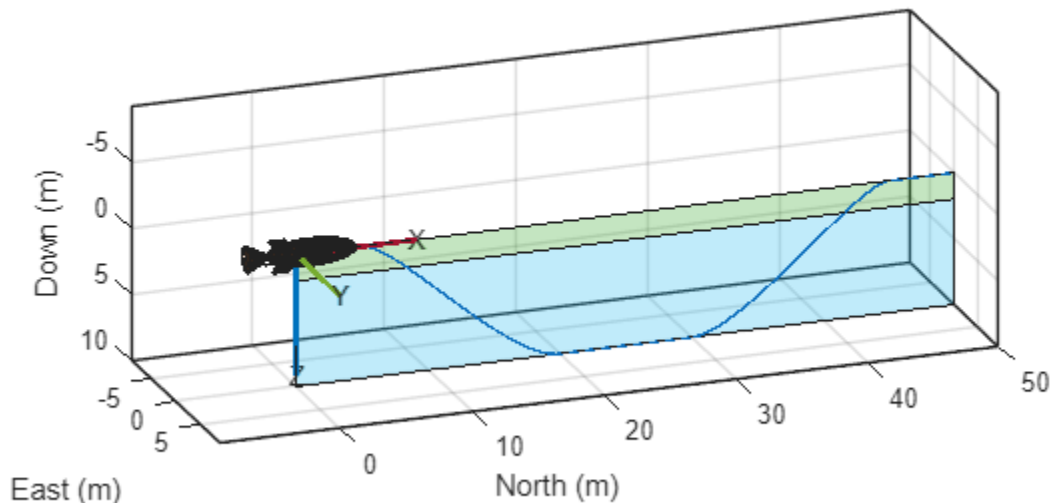
### Load and Visualize Ground-Truth Trajectory

Load the predetermined ground-truth data, which serves as the reference for pose estimation. In this example, the AUV dives to a specific depth and then resurfaces. The variable `groundTruth` provides the ground-truth data as a timetable that contains values for the AUV position, orientation, velocity, acceleration, and angular velocity, all sampled at an IMU rate of 50 Hz. The ground-truth data also includes the variable `gpsDepth`, which specifies a depth of 2 meters below the water surface, where GPS sensor readings are available.

```
load("groundTruthForAUVPoseEstimation.mat")
```

Visualize the 3-D trajectory of the vehicle using the `visualizeTrajectory` helper function. The resulting plot highlights portions of the trajectory where GPS is available in the green shaded area.

```
hpose = visualizeTrajectory(groundTruth,gpsDepth)
```



```
hpose =
  PosePatch with properties:
```

```
Orientation: [1x1 quaternion]
Position: [0 0 0]
MeshFileName: "auv.stl"
```

Use GET to show all properties

### Create Extended Kalman Filter for Sensor Fusion

Create an `insEKF` object to represent an extended Kalman filter for sensor fusion. Use the following built-in objects to model sensor measurements:

- `insAccelerometer` (Navigation Toolbox) — Accelerometer readings, including gravitational acceleration and sensor bias.
- `insGyroscope` (Navigation Toolbox) — Gyroscope readings, including the angular velocity vector and sensor bias.
- `insMagnetometer` (Navigation Toolbox) — Magnetometer readings, including the geomagnetic vector and sensor bias.
- `insGPS` (Navigation Toolbox) — GPS readings, including latitude, longitude, and altitude coordinates. This `insGPS` object enables the `insEKF` object to fuse position and velocity data.
- `insMotionPose` (Navigation Toolbox) — 3-D motion, assuming constant angular velocity and constant linear acceleration.

In addition to the built-in sensor models, define the following custom sensor models (attached as `.m` files in this example):

- `insDepth` — Depth measurements along the z-axis of the AUV body
- `insSpeed` — Flow-relative speed measurements along the x-axis of the AUV body
- `insZuptVyz` — Zero-velocity updates for the y- and z-axes of the AUV body

```
filt = insEKF(insAccelerometer,insGyroscope,insMagnetometer, ...
    insGPS,insDepth,insSpeed,insZuptVyz,insMotionPose);
```

### Load Sensor Data

Load prerecorded sensor data for the simulation. The variable `sensorData` provides the sensor data as a timetable that contains noisy readings from the accelerometer, gyroscope, magnetometer, and GPS (including velocity data in the navigation frame), as well as depth along the z-axis, flow-relative speed along the x-axis, and zero-velocity updates for the y- and z-axes of the AUV body. For the accelerometer, gyroscope, and magnetometer, the sensor data is sampled at the IMU rate of 50 Hz. The GPS readings and ZUPT data are sampled at 1 Hz. Depth and flow-relative speed data are sampled at 5 Hz.

```
load("sensorDataForAUVPoseEstimation.mat")
```

### Estimate Pose of Autonomous Underwater Vehicle

Tune the process noise and measurement noise parameters of the `insEKF` object for AUV pose estimation by using the `tuneAUVPoseFilter` helper function with the provided sensor data and ground-truth states. This step may take several minutes, depending on the data size and filter complexity.

```
tic
[tunedFilter,tunedMeasNoise] = tuneAUVPoseFilter(filt,sensorData, ...
groundTruth(:,["Position" "Orientation"]));
```

Code generation successful.

| Iteration | Parameter                 | Metric |
|-----------|---------------------------|--------|
| 1         | AdditiveProcessNoise(1)   | 2.3712 |
| 1         | AdditiveProcessNoise(33)  | 2.3712 |
| 1         | AdditiveProcessNoise(65)  | 2.3711 |
| 1         | AdditiveProcessNoise(97)  | 2.3711 |
| 1         | AdditiveProcessNoise(129) | 2.3711 |
| 1         | AdditiveProcessNoise(161) | 2.3711 |
| 1         | AdditiveProcessNoise(193) | 2.3711 |
| 1         | AdditiveProcessNoise(225) | 2.3699 |
| 1         | AdditiveProcessNoise(257) | 2.3698 |
| 1         | AdditiveProcessNoise(289) | 2.3685 |
| 1         | AdditiveProcessNoise(321) | 2.3673 |
| 1         | AdditiveProcessNoise(353) | 2.3673 |
| 1         | AdditiveProcessNoise(385) | 2.3665 |
| 1         | AdditiveProcessNoise(417) | 2.3665 |
| 1         | AdditiveProcessNoise(449) | 2.3665 |
| 1         | AdditiveProcessNoise(481) | 2.3664 |
| 1         | AdditiveProcessNoise(513) | 2.3664 |
| 1         | AdditiveProcessNoise(545) | 2.3664 |
| 1         | AdditiveProcessNoise(577) | 2.3664 |
| 1         | AdditiveProcessNoise(609) | 2.3664 |
| 1         | AdditiveProcessNoise(641) | 2.3664 |
| 1         | AdditiveProcessNoise(673) | 2.3664 |
| 1         | AdditiveProcessNoise(705) | 2.3664 |
| 1         | AdditiveProcessNoise(737) | 2.3664 |
| 1         | AdditiveProcessNoise(769) | 2.3664 |
| 1         | AdditiveProcessNoise(801) | 2.3664 |
| 1         | AdditiveProcessNoise(833) | 2.3664 |
| 1         | AdditiveProcessNoise(865) | 2.3664 |
| 1         | AdditiveProcessNoise(897) | 2.3319 |
| 1         | AdditiveProcessNoise(929) | 2.3319 |
| 1         | AdditiveProcessNoise(961) | 2.3308 |
| 1         | AccelerometerNoise        | 2.3307 |
| 1         | GyroscopeNoise            | 2.3307 |
| 1         | MagnetometerNoise         | 2.3307 |
| 1         | GPSNoise(1)               | 2.3281 |
| 1         | GPSNoise(2)               | 2.3281 |
| 1         | GPSNoise(3)               | 2.3281 |
| 1         | GPSNoise(4)               | 2.2881 |
| 1         | GPSNoise(5)               | 2.2881 |
| 1         | GPSNoise(6)               | 2.2875 |
| 1         | insDepthNoise             | 2.2875 |
| 1         | insSpeedNoise             | 2.2874 |
| 1         | insZuptVyzNoise           | 2.2874 |

```
toc
```

Elapsed time is 374.681080 seconds.

Estimate the state of the underwater vehicle using the tuned filter and sensor data. Also perform smoothing to refine the estimates.

```
tic
[stateEst,smoothEst] = estimateStates(tunedFilter,sensorData,tunedMeasNoise);
toc
```

Elapsed time is 13.250381 seconds.

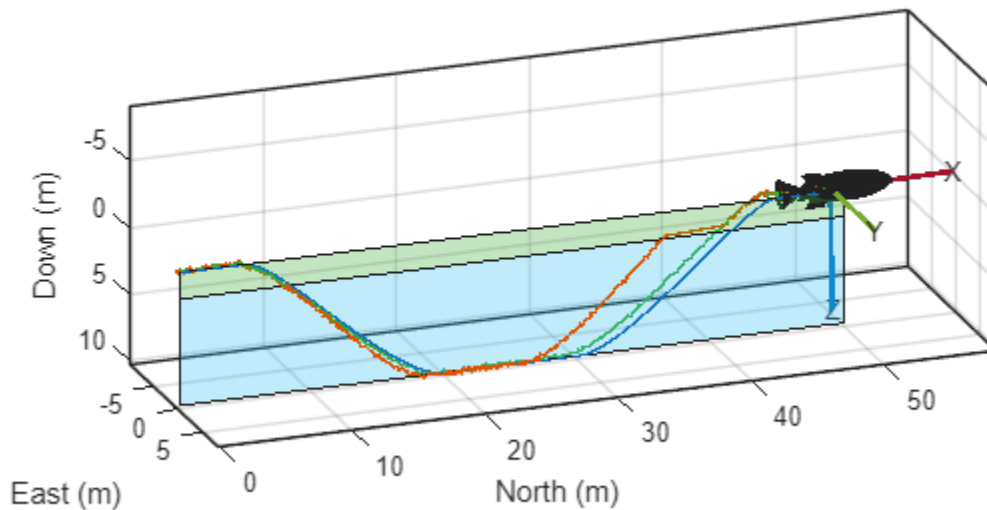
### Visualize Pose Estimations and Analyze Position Errors

Plot the estimated position trajectories and the smoother position estimates that the tuned EKF filter produced. The smoother estimates refine the trajectory by using future measurements, which typically achieve higher accuracy.

```
clr = colororder;
plot3(stateEst.Position(:,1),stateEst.Position(:,2),stateEst.Position(:,3), ...
Color=clr(2,:),LineWidth=1)
plot3(smoothEst.Position(:,1),smoothEst.Position(:,2),smoothEst.Position(:,3), ...
Color=clr(5,:),LineWidth=1)
```

Visually compare the results. Animate the estimated trajectory and overlay it on the ground-truth trajectory by using the `animateTrajectory` helper function.

```
idx = 1:50:size(stateEst,1);
animateTrajectory(hpose,stateEst(idx,:))
```



Compute the position errors by subtracting the ground-truth positions from the estimated positions at each time step. For each axis (North, East, Down), calculate the root mean squared error (RMSE), which summarizes the overall estimation accuracy.

```
posErr = stateEst.Position - groundTruth.Position;
posRmse = sqrt(mean(posErr.^2,1));
```

Plot the position errors of the estimated trajectories over time, displaying separate lines for each axis. Shade the regions where GPS is unavailable to highlight periods when the filter relies solely on inertial and other sensor data.

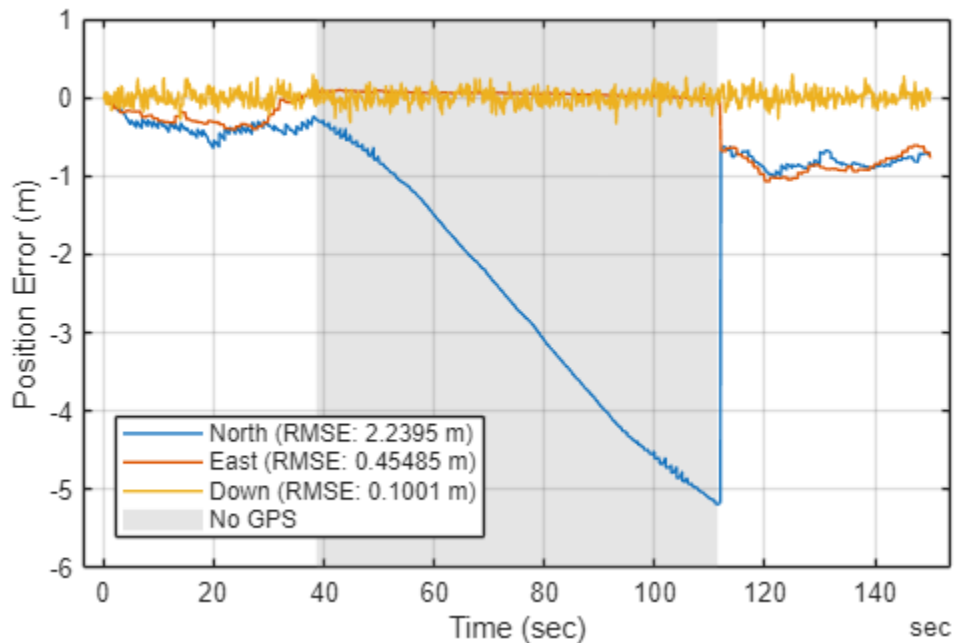


```
figure
plot(groundTruth.Time,posErr,LineWidth=1)
grid on
hold on

noGps = (groundTruth.Position(:,3) > gpsDepth);
tNoGps = groundTruth.Time(noGps);
xregion(tNoGps(1),tNoGps(end),FaceAlpha=0.2)
```

Specify the legend text and add the legend to the plot. Label the x-axis and y-axis of the plot.

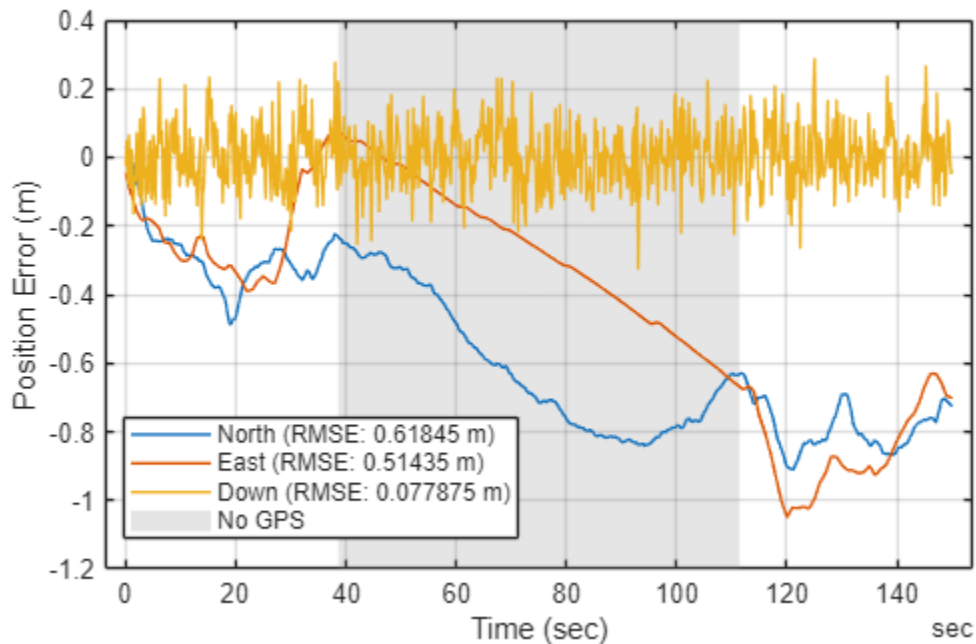
```
legendTxt = ["North" "East" "Down"] + " (RMSE: " + posRmse + " m)";
legendTxt = [legendTxt "No GPS"];
legend(legendTxt,Location="best")
xlabel("Time (sec)")
ylabel("Position Error (m)")
```



Repeat the error computation for the smoother position estimates. Plot the smoother position errors. The smoother position estimates result in lower RMSE because these estimates use information from the entire trajectory, not just past measurements.

```
posErr = smoothEst.Position - groundTruth.Position;
posRmse = sqrt(mean(posErr.^2,1));
figure
plot(groundTruth.Time,posErr,LineWidth=1)
grid on
hold on
noGps = (groundTruth.Position(:,3) > gpsDepth);
tNoGps = groundTruth.Time(noGps);
xregion(tNoGps(1),tNoGps(end),FaceAlpha=0.2)
legendTxt = ["North" "East" "Down"] + " (RMSE: " + posRmse + " m)";
legendTxt = [legendTxt "No GPS"];
legend(legendTxt,Location="best")
```

```
xlabel("Time (sec)")
ylabel("Position Error (m)")
```



## References

- [1] "Extended Kalman filter." *Wikipedia*, October 1, 2025. [https://en.wikipedia.org/wiki/Extended\\_Kalman\\_filter](https://en.wikipedia.org/wiki/Extended_Kalman_filter).
- [2] "Kalman filter." *Wikipedia*, October 4, 2025. [https://en.wikipedia.org/wiki/Kalman\\_filter](https://en.wikipedia.org/wiki/Kalman_filter).
- [3] "Quaternions and spatial rotation." *Wikipedia*, September 25, 2025. [https://en.wikipedia.org/wiki/Quaternions\\_and\\_spatial\\_rotation](https://en.wikipedia.org/wiki/Quaternions_and_spatial_rotation).

## See Also

### Functions

diff | jacobian | matlabFunction | insEKF

## Related Examples

- "Autonomous Underwater Vehicle Pose Estimation Using Inertial Sensors and Doppler Velocity Log" (Navigation Toolbox)
- "Derive Quadrotor Dynamics for Nonlinear Model Predictive Control" on page 5-24

## Using Symbolic Mathematics with Optimization Toolbox Solvers

This example shows how to use the Symbolic Math Toolbox™ functions `jacobian` and `matlabFunction` to provide analytical derivatives to optimization solvers. Optimization Toolbox™ solvers are usually more accurate and efficient when you supply gradients and Hessians of the objective and constraint functions.

Problem-based optimization can calculate and use gradients automatically; see “Automatic Differentiation in Optimization Toolbox” (Optimization Toolbox). For a problem-based example using automatic differentiation, see “Constrained Electrostatic Nonlinear Optimization Using Optimization Variables” (Optimization Toolbox).

There are several considerations in using symbolic calculations with optimization functions:

- 1 Optimization objective and constraint functions should be defined in terms of a vector, say  $\mathbf{x}$ . However, symbolic variables are scalar or complex-valued, not vector-valued. This requires you to translate between vectors and scalars.
- 2 Optimization gradients, and sometimes Hessians, are supposed to be calculated within the body of the objective or constraint functions. This means that a symbolic gradient or Hessian has to be placed in the appropriate place in the objective or constraint function file or function handle.
- 3 Calculating gradients and Hessians symbolically can be time-consuming. Therefore you should perform this calculation only once, and generate code, via `matlabFunction`, to call during execution of the solver.
- 4 Evaluating symbolic expressions with the `subs` function is time-consuming. It is much more efficient to use `matlabFunction`.
- 5 `matlabFunction` generates code that depends on the orientation of input vectors. Since `fmincon` calls the objective function with column vectors, you must be careful to call `matlabFunction` with column vectors of symbolic variables.

### First Example: Unconstrained Minimization with Hessian

The objective function to minimize is:

$$f(x_1, x_2) = \log\left(1 + 3\left(x_2 - (x_1^3 - x_1)\right)^2 + (x_1 - 4/3)^2\right).$$

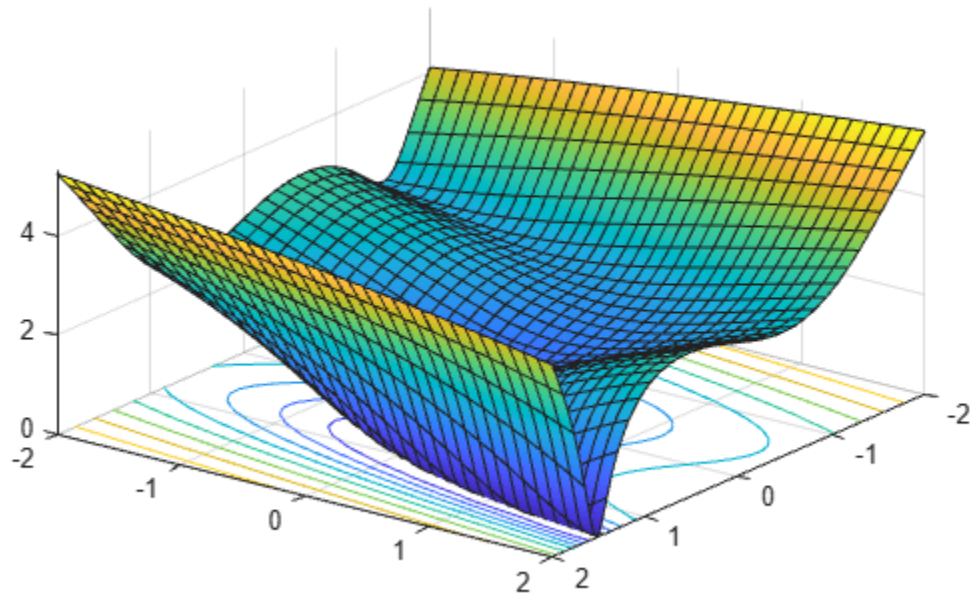
This function is positive, with a unique minimum value of zero attained at  $x_1 = 4/3$ ,  $x_2 = (4/3)^3 - 4/3 = 1.0370\dots$

We write the independent variables as  $x_1$  and  $x_2$  because in this form they can be used as symbolic variables. As components of a vector  $\mathbf{x}$  they would be written  $\mathbf{x}(1)$  and  $\mathbf{x}(2)$ . The function has a twisty valley as depicted in the plot below.

```
syms x1 x2 real
x = [x1;x2]; % column vector of symbolic variables
f = log(1 + 3*(x2 - (x1^3 - x1))^2 + (x1 - 4/3)^2)

f =
    log((x1 - 4/3)^2 + 3*(-x1^3 + x1 + x2)^2 + 1)

fsurf(f, [-2 2], ShowContours="on")
view(127, 38)
```



Compute the gradient and Hessian of f:

```
gradf = jacobian(f,x).' % column gradf
```

```
gradf =
```

$$\begin{pmatrix} -\frac{6(3x_1^2 - 1)(-x_1^3 + x_1 + x_2) - 2x_1 + \frac{8}{3}}{\sigma_1} \\ \frac{-6x_1^3 + 6x_1 + 6x_2}{\sigma_1} \end{pmatrix}$$

where

$$\sigma_1 = \left(x_1 - \frac{4}{3}\right)^2 + 3(-x_1^3 + x_1 + x_2)^2 + 1$$

```
hessf = jacobian(gradf,x)
```

```
hessf =
```

$$\begin{pmatrix} \frac{6(3x_1^2 - 1)^2 - 36x_1(-x_1^3 + x_1 + x_2) + 2}{\sigma_2} - \frac{\sigma_3^2}{\sigma_2^2} & \sigma_1 \\ \sigma_1 & \frac{6}{\sigma_2} - \frac{(-6x_1^3 + 6x_1 + 6x_2)^2}{\sigma_2^2} \end{pmatrix}$$

where

$$\sigma_1 = \frac{(-6x_1^3 + 6x_1 + 6x_2)\sigma_3}{\sigma_2^2} - \frac{18x_1^2 - 6}{\sigma_2}$$

$$\sigma_2 = \left(x_1 - \frac{4}{3}\right)^2 + 3(-x_1^3 + x_1 + x_2)^2 + 1$$

$$\sigma_3 = 6(3x_1^2 - 1)(-x_1^3 + x_1 + x_2) - 2x_1 + \frac{8}{3}$$

The `fminunc` solver expects to pass in a vector `x`, and, with the `SpecifyObjectiveGradient` option set to `true` and `HessianFcn` option set to `"objective"`, expects a list of three outputs: `[f(x), gradf(x), hessf(x)]`.

`matlabFunction` generates exactly this list of three outputs from a list of three inputs. Furthermore, using the `vars` option, `matlabFunction` accepts vector inputs.

```
fh = matlabFunction(f,gradf,hessf,vars={x});
```

Now solve the minimization problem starting at the point `[-1,2]`:

```
options = optimoptions("fminunc", ...
    SpecifyObjectiveGradient=true, ...
    HessianFcn="objective", ...
    Algorithm="trust-region", ...
    Display="final");
[xfinal,fval,exitflag,output] = fminunc(fh,[-1;2],options)
```

Local minimum possible.

`fminunc` stopped because the final change in function value relative to its initial value is less than the value of the function tolerance.

<stopping criteria details>

```
xfinal = 2×1
```

```
    1.3333
    1.0370
```

```
fval =
7.6623e-12
```

```
exitflag =
3
```

```
output = struct with fields:
    iterations: 14
    funcCount: 15
    stepsize: 0.0027
```

```
cgiterations: 11
firstorderopt: 3.4391e-05
algorithm: 'trust-region'
message: 'Local minimum possible. fminunc stopped because the final change in function value was less than the specified tolerance.'
constrviolation: []
```

Compare this with the number of iterations using no gradient or Hessian information. This requires the "quasi-newton" algorithm.

```
options = optimoptions("fminunc",Display="final",Algorithm="quasi-newton");
fh2 = matlabFunction(f,vars={x});
% fh2 = objective with no gradient or Hessian
[xfinal,fval,exitflag,output2] = fminunc(fh2,[-1;2],options)
```

Local minimum possible.

fminunc stopped because it cannot decrease the objective function along the current search direction.

<stopping criteria details>

xfinal = 2×1

```
1.3333
1.0370
```

fval =  
1.2879e-13

exitflag =  
5

```
output2 = struct with fields:
    iterations: 20
    funcCount: 96
    stepsize: 2.5300e-05
    lssteplength: 1
    firstorderopt: 3.1963e-06
    algorithm: 'quasi-newton'
    message: 'Local minimum possible. fminunc stopped because it cannot decrease the objective function along the current search direction.'
```

The number of iterations is lower when using gradients and Hessians, and there are dramatically fewer function evaluations:

```
sprintf("There are %d iterations using gradient and Hessian, but %d without them.", ...
    output.iterations,output2.iterations)
```

```
ans =
"There are 14 iterations using gradient and Hessian, but 20 without them."
```

```
sprintf("There are %d function evaluations using gradient and Hessian, but %d without them.", ...
    output.funcCount,output2.funcCount)
```

```
ans =
"There are 15 function evaluations using gradient and Hessian, but 96 without them."
```

## Second Example: Constrained Minimization Using the fmincon Interior-Point Algorithm

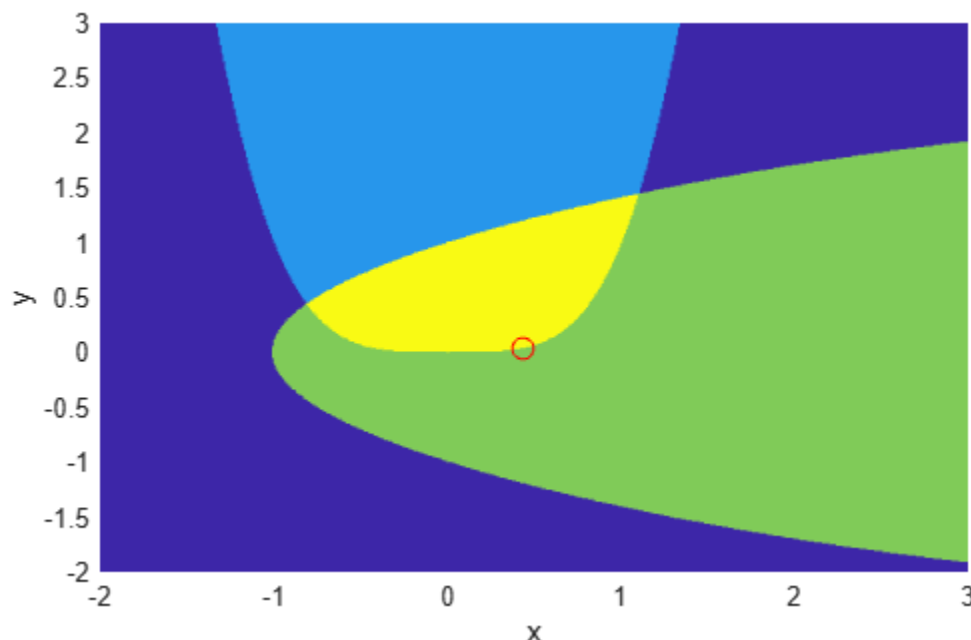
We consider the same objective function and starting point, but now have two nonlinear constraints:

$$5\sinh(x_2/5) \geq x_1^4$$

$$5\tanh(x_1/5) \geq x_2^2 - 1.$$

The constraints keep the optimization away from the global minimum point [1.333,1.037]. Visualize the two constraints:

```
[X,Y] = meshgrid(-2:.01:3);
Z = (5*sinh(Y./5) >= X.^4);
% Z=1 where the first constraint is satisfied, Z=0 otherwise
Z = Z + 2*(5*tanh(X./5) >= Y.^2 - 1);
% Z=2 where the second is satisfied, Z=3 where both are
surf(X,Y,Z,LineStyle="none");
fig = gcf;
fig.Color = "w"; % white background
view(0,90)
hold on
plot3(.4396, .0373, 4,"o",MarkerEdgeColor="r",MarkerSize=8);
% best point
xlabel("x")
ylabel("y")
hold off
```



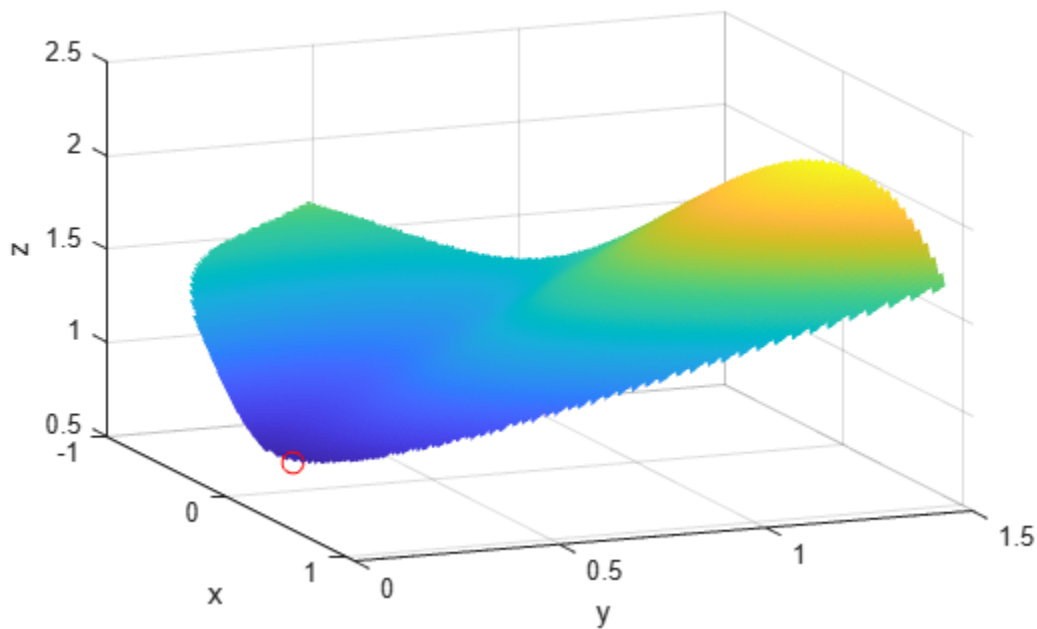
We plotted a small red circle around the optimal point.

Here is a plot of the objective function over the feasible region, the region that satisfies both constraints, pictured above in dark red, along with a small red circle around the optimal point:

```

W = log(1 + 3*(Y - (X.^3 - X)).^2 + (X - 4/3).^2);
% W = the objective function
W(Z < 3) = nan; % plot only where the constraints are satisfied
surf(X,Y,W,LineStyle="none");
view(68,20)
hold on
plot3(.4396, .0373, .8152,"o",MarkerEdgeColor="r", ...
      MarkerSize=8); % best point
xlabel("x")
ylabel("y")
zlabel("z")
hold off

```



The nonlinear constraints must be written in the form `ineqnonlin(x) <= 0`. We compute all the symbolic constraints and their derivatives, and place them in a function handle using `matlabFunction`.

The gradients of the constraints should be column vectors; they must be placed in the objective function as a matrix, with each column of the matrix representing the gradient of one constraint function. This is the transpose of the form generated by `jacobian`, so we take the transpose below.

We place the nonlinear constraints into a function handle. `fmincon` expects the nonlinear constraints and gradients to be output in the order `[ineqnonlin eqnonlin gradineqnonlin gradeqnonlin]`. Since there are no nonlinear equality constraints, we output `[]` for `eqnonlin` and `gradeqnonlin`.

```

ineqnonlin1 = x1^4 - 5*sinh(x2/5);
ineqnonlin2 = x2^2 - 5*tanh(x1/5) - 1;
ineqnonlin = [ineqnonlin1 ineqnonlin2];
gradineqnonlin = jacobian(ineqnonlin,x).'; % transpose to put in correct form
constraint = matlabFunction(ineqnonlin,[],gradineqnonlin,[],vars={x});

```



The interior-point algorithm requires its Hessian function to be written as a separate function, instead of being part of the objective function. This is because a nonlinearly constrained function needs to include those constraints in its Hessian. Its Hessian is the Hessian of the Lagrangian; see the User's Guide for more information.

The Hessian function takes two input arguments: the position vector  $x$ , and the Lagrange multiplier structure  $\lambda$ . The parts of the  $\lambda$  structure that you use for nonlinear constraints are  $\lambda.ineqnonlin$  and  $\lambda.eqnonlin$ . For the current constraint, there are no linear equalities, so we use the two multipliers  $\lambda.ineqnonlin(1)$  and  $\lambda.ineqnonlin(2)$ .

We calculated the Hessian of the objective function in the first example. Now we calculate the Hessians of the two constraint functions, and make function handle versions with `matlabFunction`.

```
hessc1 = jacobian(gradineqnonlin(:,1),x); % constraint = first ineqnonlin column
hessc2 = jacobian(gradineqnonlin(:,2),x);
```

```
hessfh = matlabFunction(hessf,vars={x});
hessc1h = matlabFunction(hessc1,vars={x});
hessc2h = matlabFunction(hessc2,vars={x});
```

To make the final Hessian, we put the three Hessians together, adding the appropriate Lagrange multipliers to the constraint functions.

```
myhess = @(x,lambda)(hessfh(x) + ...
    lambda.ineqnonlin(1)*hessc1h(x) + ...
    lambda.ineqnonlin(2)*hessc2h(x));
```

Set the options to use the interior-point algorithm, the gradient, and the Hessian, have the objective function return both the objective and the gradient, and run the solver:

```
options = optimoptions("fmincon", ...
    Algorithm="interior-point", ...
    SpecifyObjectiveGradient=true, ...
    SpecifyConstraintGradient=true, ...
    HessianFcn=myhess, ...
    Display="final");
% fh2 = objective without Hessian
fh2 = matlabFunction(f,gradf,vars={x});
[xfinal,fval,exitflag,output] = fmincon(fh2,[-1;2],...
    [],[],[],[],[],constraint,options)
```

Local minimum found that satisfies the constraints.

Optimization completed because the objective function is non-decreasing in feasible directions, to within the value of the optimality tolerance, and constraints are satisfied to within the value of the constraint tolerance.

<stopping criteria details>

```
xfinal = 2×1
```

```
    0.4396
    0.0373
```

```
fval =
    0.8152
```

```
exitflag =
1

output = struct with fields:
    iterations: 10
    funcCount: 13
    constrviolation: 0
    stepsize: 1.9160e-06
    algorithm: 'interior-point'
    firstorderopt: 1.9217e-08
    cgiterations: 0
    message: 'Local minimum found that satisfies the constraints. Optimization completed because the objective function is non-decreasing in feasible directions, to within the value of the optimality tolerance, and constraints are satisfied to within the value of the constraint tolerance.'
    bestfeasible: [1x1 struct]
```

Again, the solver makes many fewer iterations and function evaluations with gradient and Hessian supplied than when they are not:

```
options = optimoptions("fmincon",Algorithm="interior-point",...
    Display="final");
% fh3 = objective without gradient or Hessian
fh3 = matlabFunction(f,vars={x});
% constraint without gradient:
constraint = matlabFunction(ineqnonlin,[],vars={x});
[xfinal,fval,exitflag,output2] = fmincon(fh3,[-1;2],...
    [],[],[],[],[],constraint,options)
```

Local minimum found that satisfies the constraints.

Optimization completed because the objective function is non-decreasing in feasible directions, to within the value of the optimality tolerance, and constraints are satisfied to within the value of the constraint tolerance.

<stopping criteria details>

```
xfinal = 2x1
```

```
    0.4396
    0.0373
```

```
fval =
0.8152
```

```
exitflag =
1
```

```
output2 = struct with fields:
    iterations: 16
    funcCount: 51
    constrviolation: 0
    stepsize: 1.2936e-07
    algorithm: 'interior-point'
    firstorderopt: 7.8514e-07
    cgiterations: 0
    message: 'Local minimum found that satisfies the constraints. Optimization completed because the objective function is non-decreasing in feasible directions, to within the value of the optimality tolerance, and constraints are satisfied to within the value of the constraint tolerance.'
    bestfeasible: [1x1 struct]
```

```

sprintf("There are %d iterations using gradient and Hessian, but %d without them.",...
        output.iterations,output2.iterations)

ans =
"There are 10 iterations using gradient and Hessian, but 16 without them."

sprintf("There are %d function evaluations using gradient and Hessian, but %d without them.", ..
        output.funcCount,output2.funcCount)

ans =
"There are 13 function evaluations using gradient and Hessian, but 51 without them."

```

### **Cleaning Up Symbolic Variables**

The symbolic variables used in this example were assumed to be real. To clear this assumption from the symbolic engine workspace, it is not sufficient to delete the variables. You must clear the assumptions of variables using the syntax

```
assume([x1,x2], "clear")
```

All assumptions are cleared when the output of the following command is empty

```
assumptions([x1,x2])
```

```
ans =
```

```
Empty sym: 1-by-0
```

### **See Also**

#### **Functions**

jacobian | matlabFunction | fsurf

## Improving Accuracy and Performance in Optimization

This example demonstrates that the Symbolic Math Toolbox™ helps minimize errors when solving a nonlinear system of equations.

This example uses the following Symbolic Math Toolbox capabilities:

- Computing the analytical gradient using `gradient`
- Computing the analytical jacobian using `jacobian`
- Converting symbolic results into numeric functions using `matlabFunction`
- Analytical visualization with `fplot`

The goal is to find the minimum of the Rosenbrock function, or the so-called "banana" function.

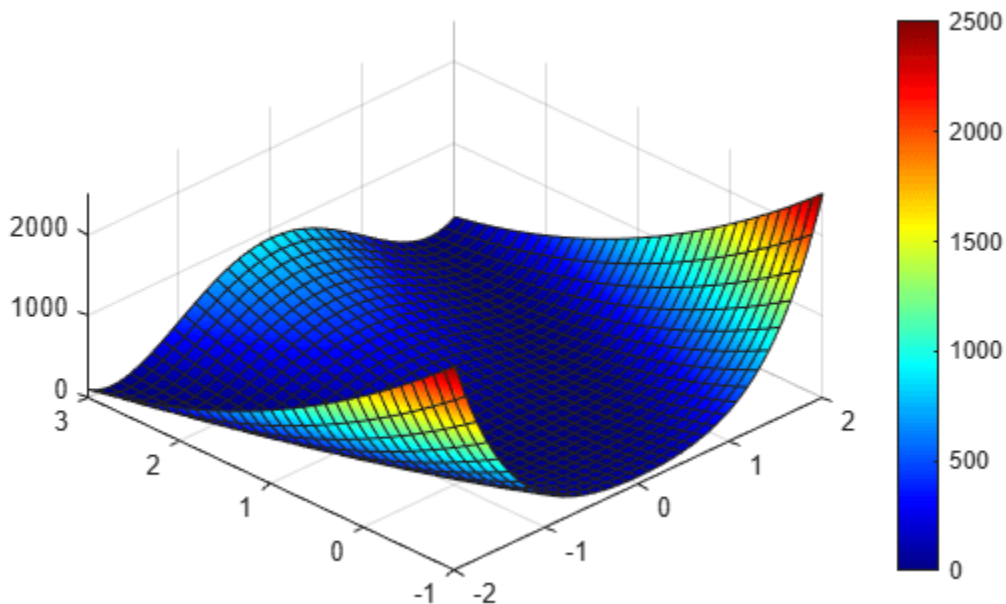
$$f(\mathbf{x}) = \sum_{i=1}^{n/2} 100(x_{2i} - x_{2i-1}^2)^2 + (1 - x_{2i-1})^2$$

Use `fplot` to create a quick visualization of a Rosenbrock function of two variables.

```
syms x y
a=1; b=100;
f(x,y)=(a-x)^2+b*(y-x^2)^2
```

$$f(x, y) = (x - 1)^2 + 100(y - x^2)^2$$

```
fsurf(f,[-2 2 -1 3])
view([-45 50])
colormap jet
colorbar
```



## Motivation

The “Solve Nonlinear System Without and Including Jacobian” (Optimization Toolbox) example in Optimization Toolbox™ solves the same problem by using the `fsolve` (Optimization Toolbox) function. The workflow shown in that example has two potential sources of errors. You would need to

- 1 Translate the equations for the Rosenbrock function and Jacobian from the math form in the text to numeric code.
- 2 Explicitly calculate the Jacobian. For complicated equations, this task is prone to errors.

This example shows that using symbolic math to describe the problem statement and subsequent steps minimizes or even eliminates such errors.

## Rewrite Rosenbrock Function as a System of Nonlinear Equations

First, convert the Rosenbrock function  $f$  to a system of nonlinear equations  $F$ , where  $F$  is a gradient of  $f$ .

```
clear
n = 64;
x = sym('x',[n,1]);
f = sum(100*(x(2:2:n)-x(1:2:n).^2).^2 + (1-x(1:2:n)).^2);
F = gradient(f,x);
```

Show the first 10 equations:

```
F(1:10)
```

```
ans =
```

$$\begin{pmatrix} 2x_1 - 400x_1(x_2 - x_1^2) - 2 \\ 200x_2 - 200x_1^2 \\ 2x_3 - 400x_3(x_4 - x_3^2) - 2 \\ 200x_4 - 200x_3^2 \\ 2x_5 - 400x_5(x_6 - x_5^2) - 2 \\ 200x_6 - 200x_5^2 \\ 2x_7 - 400x_7(x_8 - x_7^2) - 2 \\ 200x_8 - 200x_7^2 \\ 2x_9 - 400x_9(x_{10} - x_9^2) - 2 \\ 200x_{10} - 200x_9^2 \end{pmatrix}$$

To match the intermediate results shown in the “Solve Nonlinear System Without and Including Jacobian” (Optimization Toolbox), use the same form of  $F$ .

```
F(1:2:n) = simplify(F(1:2:n) + 2*x(1:2:n).*F(2:2:n));
F(1:2:n) = -F(1:2:n)/2;
F(2:2:n) = F(2:2:n)/20;
```

Now the systems of equations are identical:

```
F(1:10)
```

$$\text{ans} = \begin{pmatrix} 1 - x_1 \\ 10 x_2 - 10 x_1^2 \\ 1 - x_3 \\ 10 x_4 - 10 x_3^2 \\ 1 - x_5 \\ 10 x_6 - 10 x_5^2 \\ 1 - x_7 \\ 10 x_8 - 10 x_7^2 \\ 1 - x_9 \\ 10 x_{10} - 10 x_9^2 \end{pmatrix}$$

### Calculate Jacobian of Matrix Representing the System of Equations

Use `jacobian` to calculate the Jacobian of `F`. This function calculates the Jacobian symbolically, thus avoiding errors associated with numerical approximations of derivatives.

```
JF = jacobian(F,x);
```

Show the first 10 rows and columns of the Jacobian matrix.

```
JF(1:10,1:10)
```

$$\text{ans} = \begin{pmatrix} -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -20 x_1 & 10 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -20 x_3 & 10 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -20 x_5 & 10 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -20 x_7 & 10 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -20 x_9 & 10 \end{pmatrix}$$

Most of the elements of the Jacobian matrix `JF` are zeros. Nevertheless, when you convert this matrix to a `matlabFunction`, the result is a dense numeric matrix. Often, operations on sparse matrices are more efficient than the same operations on dense matrices.

Therefore, to create efficient code, convert only the nonzero elements of `JF` to a symbolic vector before invoking `matlabFunction`. `is` and `js` represent the sparsity pattern of `JF`.

```
[is,js] = find(JF);  
JF = JF(JF~=0);
```

### Convert System of Equations and Jacobian to a MATLAB Function

The system of equations `F`, representing the Rosenbrock function, is a symbolic matrix that consists of symbolic expressions. To be able to solve it with the `fsolve` function, convert this system to a `matlabFunction`. At this step, it is convenient to convert both `F` and its Jacobian, `JF`, to a single file-

based MATLAB function, `FJFfun`. In principle, this allows `F` and `JF` to reuse variables. Alternatively, you can convert them to individual MATLAB functions.

```
matlabFunction([F;JF], 'var', x, 'file', 'FJFfun');
```

`FJFfun` accepts variables as a list, but `fsolve` expects a vector of variables. The function `genericObj` converts the vector to a list. Anonymous function `bananaObj` is defined to fix argument values for `genericObj`. Note the use of the sparsity pattern in the function `genericObj`. Further note that this approach of using `FJFfun` and `genericObj` together is not tied to the particular expression represented by `F` and can be used as is for any `F`.

```
bananaobj = @(y) genericObj(y,@FJFfun,is,js)
```

```
bananaobj = function_handle with value:  
@(y)genericObj(y,@FJFfun,is,js)
```

### Use `fsolve` to Numerically Solve the System of Nonlinear Equations

Use `fsolve` for the system of equations, converted to MATLAB function. Use the starting point  $x(i) = -1.9$  for the odd indices, and  $x(i) = 2$  for the even indices. Set `'Display'` to `'iter'` to see the solver's progress. Set `'Jacobian'` to `'on'` to use the Jacobian defined in `bananaobj`.

```
x0(1:n,1) = -1.9;  
x0(2:2:n,1) = 2;  
options = optimoptions(@fsolve, 'Display', 'iter', 'Jacobian', 'on');  
[sol1,Fsol,exitflag,output,JAC] = fsolve(bananaobj,x0,options);
```

| Iteration | Func-count | $  f(x)  ^2$ | Norm of step | First-order optimality | Trust-region radius |
|-----------|------------|--------------|--------------|------------------------|---------------------|
| 0         | 1          | 8563.84      |              | 615                    | 1                   |
| 1         | 2          | 3093.71      | 1            | 329                    | 1                   |
| 2         | 3          | 225.104      | 2.5          | 34.8                   | 2.5                 |
| 3         | 4          | 212.48       | 6.25         | 34.1                   | 6.25                |
| 4         | 5          | 212.48       | 6.25         | 34.1                   | 6.25                |
| 5         | 6          | 212.48       | 1.5625       | 34.1                   | 1.56                |
| 6         | 7          | 116.793      | 0.390625     | 5.79                   | 0.391               |
| 7         | 8          | 109.947      | 0.390625     | 0.753                  | 0.391               |
| 8         | 9          | 99.4696      | 0.976563     | 1.2                    | 0.977               |
| 9         | 10         | 83.6416      | 2.44141      | 7.13                   | 2.44                |
| 10        | 11         | 77.7663      | 2.44141      | 9.94                   | 2.44                |
| 11        | 12         | 77.7663      | 2.44141      | 9.94                   | 2.44                |
| 12        | 13         | 43.013       | 0.610352     | 1.38                   | 0.61                |
| 13        | 14         | 36.4334      | 0.610352     | 1.58                   | 0.61                |
| 14        | 15         | 34.1448      | 1.52588      | 6.71                   | 1.53                |
| 15        | 16         | 18.0108      | 1.52588      | 4.91                   | 1.53                |
| 16        | 17         | 8.48336      | 1.52588      | 3.74                   | 1.53                |
| 17        | 18         | 3.74566      | 1.52588      | 3.58                   | 1.53                |
| 18        | 19         | 1.46166      | 1.52588      | 3.32                   | 1.53                |
| 19        | 20         | 0.29997      | 1.24265      | 1.94                   | 1.53                |
| 20        | 21         | 0            | 0.0547695    | 0                      | 1.53                |

Equation solved.

`fsolve` completed because the vector of function values is near zero as measured by the value of the function tolerance, and the problem appears regular as measured by the gradient.

<stopping criteria details>

### Use `vpasolve` to Numerically Solve the System of Nonlinear Equations

The system of nonlinear equations, `F`, can be alternatively solved with the `vpasolve` function, which is a numeric solver available in Symbolic Math Toolbox. A key difference between `vpasolve` and `fsolve` is that `fsolve` uses double-precision arithmetic to solve the equations, whereas `vpasolve` uses variable-precision arithmetic and therefore, lets you control the accuracy of computations.

```
sol2 = vpasolve(F);
```

If you assign the solution of a system of equations to one variable, `sol2`, then `vpasolve` returns the solutions in form of a structure array. You can access each solution (each field of the structure array):

```
sol2.x1
```

```
ans = 1.0
```

You also can convert `sol2` to a symbolic vector, and then access a range of the solutions. For example, display the 10th to 20th solutions:

```
for k=(1:64)
    sol2_array(k) = sol2.(char(x(k)));
end
sol2_array(10:20)

ans = (1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0)
```

### Helper Functions

```
function [F,J] = genericObj(x,FJFfun,is,js)
% genericObj takes as input, vector x, Function and Jacobian evaluation
% function handle FJFfun, and sparsity pattern is,js and returns as output,
% the numeric values of the Function and Jacobian: F and Jfunction
% FJs(1:length(x)) is the numeric vector for the Function
% FJs(length(x)+1:end) is the numeric vector for the non-zero entries of the Jacobian

xcell = num2cell(x);
FJs = FJFfun(xcell{:});
F = FJs(1:length(x));
J = sparse(is,js,FJs(length(x)+1:end));

end
```



## Analytical Model of Cantilever Truss Structure for Simscape

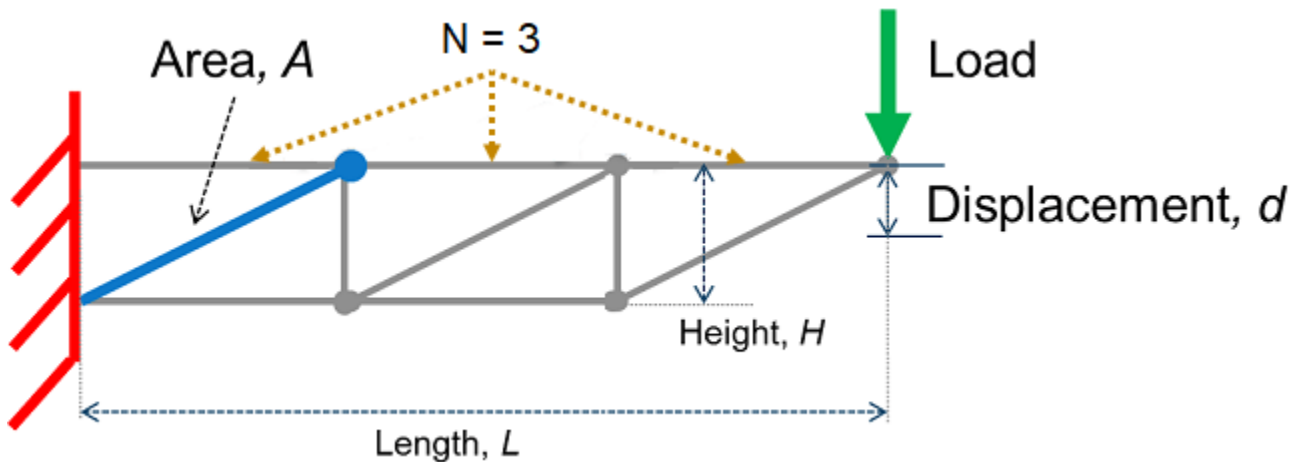
This example shows how to find parameterized analytical expressions for the displacement of a joint of a trivial cantilever truss structure in both the static and frequency domains. The analytical expression for the static case is exact. The expression for the frequency response function is an approximate reduced-order version of the actual frequency response.

This example uses the following Symbolic Math Toolbox™ capabilities:

- Simscape™ code generation using `simscapeEquation`
- LU decomposition using `lu`
- MATLAB® function generation using `matlabFunction`

### Define Model Parameters

The goal of this example is to analytically relate the displacement  $d$  to the uniform cross-section area parameter  $A$  for a particular bar in the cantilever truss structure. The governing equation is represented by  $M \frac{d^2 x}{dt^2} + Kx = F$ . Here,  $d$  results from a load at the upper-right joint of the cantilever. The truss is attached to the wall at the leftmost joints.



The truss bars are made of a linear elastic material with uniform density. The cross-section area  $A$  of the bar highlighted in blue (see figure) can vary. All other parameters, including the uniform cross-section areas of the other bars, are fixed. Obtain the displacement of the tip by using small and linear displacement assumptions.

First, define the fixed parameters of the truss.

Specify the length and height of the truss and the number of top horizontal truss bars.

```
L = 1.5;
H = 0.25;
N = 3;
```

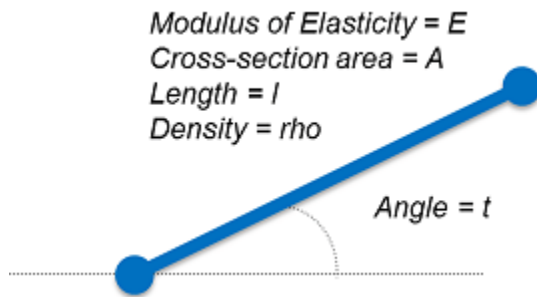
Specify the density and modulus of elasticity of the truss bar material.

```
rhoval = 1e2;
Eval = 1e7;
```

Specify the cross-section area of other truss bars.

```
AFixed = 1;
```

Next, define the local stiffness matrix of the specific truss element.



Compute the local stiffness matrix  $k$  and express it as a symbolic function. The forces and displacements of the truss element are related through the local stiffness matrix. Each node of the truss element has two degrees of freedom, horizontal and vertical displacement. The displacement of each truss element is a column matrix that corresponds to  $[x\_node1, y\_node1, x\_node2, y\_node2]$ . The resulting stiffness matrix is a 4-by-4 matrix.

```
syms A E l t real
G = [cos(t) sin(t) 0 0; 0 0 cos(t) sin(t)];
kk = A*E/l*[1 -1;-1 1];
k = simplify(G'*kk*G);
localStiffnessFn = symfun(k,[t,l,E])
```

```
localStiffnessFn(t, l, E) =
```

$$\begin{pmatrix} \sigma_2 & \sigma_1 & -\sigma_2 & -\sigma_1 \\ \sigma_1 & \sigma_3 & -\sigma_1 & -\sigma_3 \\ -\sigma_2 & -\sigma_1 & \sigma_2 & \sigma_1 \\ -\sigma_1 & -\sigma_3 & \sigma_1 & \sigma_3 \end{pmatrix}$$

where

$$\sigma_1 = \frac{A E \sin(2 t)}{2 l}$$

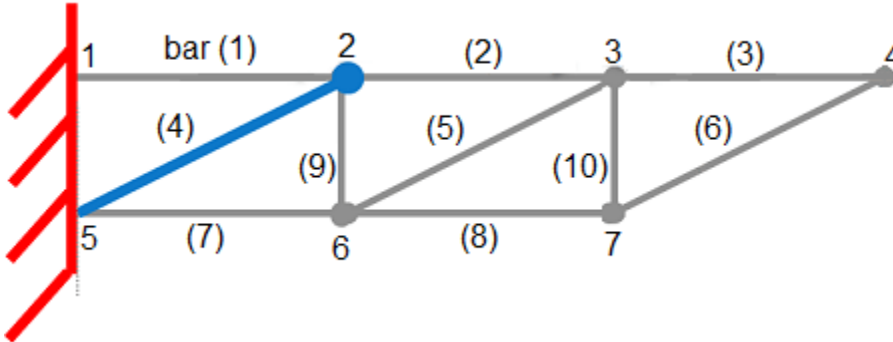
$$\sigma_2 = \frac{A E \cos(t)^2}{l}$$

$$\sigma_3 = \frac{A E \sin(t)^2}{l}$$

Compute the mass matrix in a similar way.

```
syms rho
mm = A*rho*l/6*[2 1;1 2];
m = simplify(G'*mm*G);
localMassFn = symfun(m,[t,l,rho]);
```

Now, define the bars of truss as a matrix `bars`. The indices for the bars, starting joints, and end joints are shown in the following figure.



The number of rows of the matrix `bars` is the number of bars of the truss. The columns of `bars` have four elements, which represent the following parameters:

- Length ( $l$ )
- Angle with respect to the horizontal axis ( $t$ )
- Index of starting joint
- Index of ending joint

```
bars = zeros(2*N+2*(N-1),4);
```

Define the upper and diagonal bars. Note that the bar of interest is the  $(N+1)$ th bar or bar number 4 which is the first diagonal bar from the left.

```
for n = 1:N
    lelem = L/N;
    bars(n,:) = [lelem,0,n,n+1];
    lelem = sqrt((L/N)^2+H^2);
    bars(N+n,:) = [lelem,atan2(H,L/N),N+1+n,n+1];
end
```

Define the lower and vertical bars.

```
for n = 1:N-1
    lelem = L/N;
    bars(2*N+n,:) = [lelem,0,N+1+n,N+1+n+1];
    lelem = H;
    bars(2*N+N-1+n,:) = [lelem,pi/2,N+1+n+1,n+1];
end
```

### Assemble Bars into Symbolic Global Matrices

The truss structure has seven nodes. Each node has two degrees of freedom (horizontal and vertical displacements). The total number of degrees of freedom in this system is 14.

```
numDofs = 2*2*(N+1)-2
```

```
numDofs =
14
```

The system mass matrix  $M$  and system stiffness matrix  $K$  are symbolic matrices. Assemble local element matrices from each bar into the corresponding global matrix.

```
K = sym(zeros(numDofs,numDofs));
M = sym(zeros(numDofs,numDofs));
for j = 1:size(bars,1)
    % Construct stiffness and mass matrices for bar j.
    lelem = bars(j,1); telem = bars(j,2);
    kelem = localStiffnessFn(telem,lelem,Eval);
    melem = localMassFn(telem,lelem,rhoval);
    n1 = bars(j,3); n2 = bars(j,4);
    % For bars that are not within the parameterized area A, set the stiffness
    % and mass matrices to numeric values. Note that although the values
    % kelem and melem do not have symbolic parameters, they are still
    % symbolic objects (symbolic numbers).
    if j ~= N+1
        kelem = subs(kelem,A,AFixed);
        melem = subs(melem,A,AFixed);
    end
    % Arrange the indices.
    ix = [n1*2-1,n1*2,n2*2-1,n2*2];
    % Embed local elements in system global matrices.
    K(ix,ix) = K(ix,ix) + kelem;
    M(ix,ix) = M(ix,ix) + melem;
end
```

Eliminate degrees of freedom attached to the wall.

```
wallDofs = [1,2,2*(N+1)+1,2*(N+1)+2]; % DoFs to eliminate
K(wallDofs,:) = [];
K(:,wallDofs) = [];
M(wallDofs,:) = [];
M(:,wallDofs) = [];
```

$F$  is the load vector with a load specified in the negative  $Y$  direction at the upper rightmost joint.

```
F = zeros(size(K,1),1);
F(2*N) = -1;
```

To extract the  $Y$  displacement at the upper rightmost joint, create a selection vector.

```
c = zeros(1,size(K,1));
c(2*N) = 1;
```

### Create Simscape Equations from Exact Symbolic Solution for Static Case

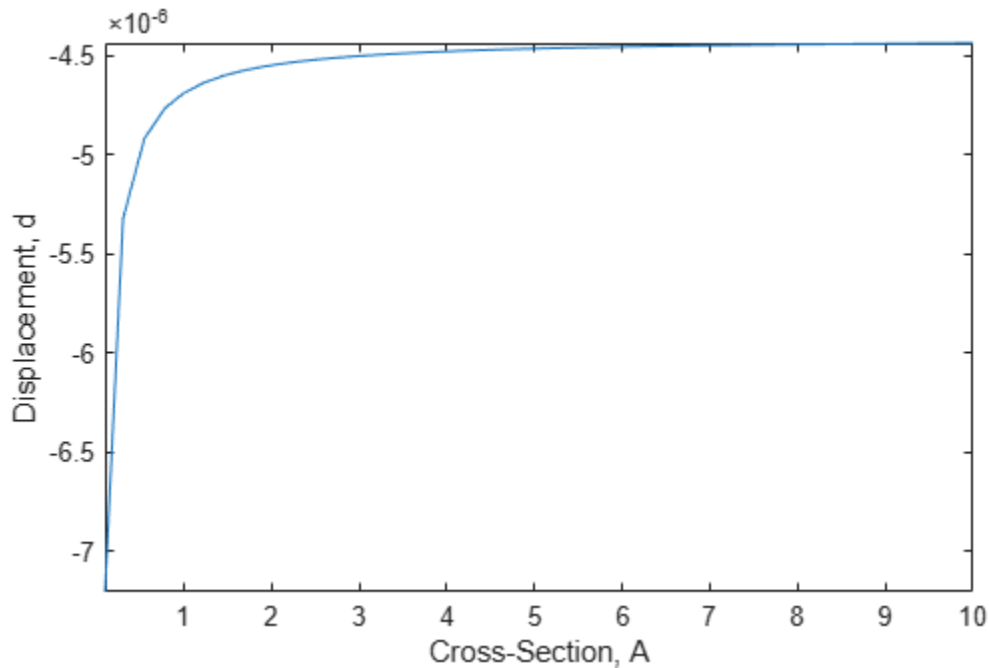
Find and plot the analytical solution for the displacement  $d$  as a function of  $A$ . Here,  $K \backslash F$  represents the displacements at all joints and  $c$  selects the particular displacements. First, show the symbolic solution representing numeric values using 16 digits for compactness.

```
d_Static = simplify(c*(K\F));
vpa(d_Static,16)
```

```
ans =
-
0.00000001118033988749895 (504.7737197247586 A2 + 384.7213595499958 A + 22.3606797749979)
-----
A (1.28 A + 0.8944271909999158)
```

```
fplot(d_Static,[AFixed/10,10*AFixed]);
```

```
hold on;  
xlabel('Cross-Section, A');  
ylabel('Displacement, d');  
title('')
```



Convert the result to a Simscape equation `ssEq` to use inside a Simscape block. To display the resulting Simscape equation, remove the semicolon.

```
syms d
ssEq = SimscapeEquation(d,d_Static);
```

Display the first 120 characters of the expression for `ssEq`.

```
strcat(ssEq(1:120), '...')
```

```
ans =
'd == (sqrt(5.0)*(A^2.2e+2+A*cos(9.272952180016122e-1)*2.0e+2+sqrt(5.0)*A^2*1.16e+2+sqrt(5.0)*1.0
```

## Compare Numeric and Symbolic Static Solutions

Compare the exact analytical solution and numeric solution over a range of A parameter values. The values form a sequence from AFixed to 5\*AFixed in increments of 1.

```
AParamValues = AFixed*(1:5)';
d_NumericArray = zeros(size(AParamValues));
for k=1:numel(AParamValues)
    beginDim = (k-1)*size(K,1)+1;
    endDim = k*size(K,1);
    % Create a numeric stiffness matrix and extract the numeric solution.
    KNumeric = double(subs(K,A,AParamValues(k)));
    d_NumericArray(k) = c*(KNumeric\F);
end
```

Compute the symbolic values over `AParamValues`. To do so, call the `subs` function, and then convert the result to double-precision numbers using `double`.

```
d_SymArray = double(subs(d_Static,A,AParamValues));
```

Visualize the data in a table.

```
T = table(AParamValues,d_SymArray,d_NumericArray)
```

```
T=5×3 table
    AParamValues    d_SymArray    d_NumericArray
    _____    _____    _____
         1         -4.6885e-06    -4.6885e-06
         2         -4.5488e-06    -4.5488e-06
         3         -4.5022e-06    -4.5022e-06
         4         -4.4789e-06    -4.4789e-06
         5         -4.4649e-06    -4.4649e-06
```

### Approximate Parametric Symbolic Solution for Frequency Response

A parametric representation for the frequency response  $H(s, A)$  can be efficient for quickly examining the effects of parameter  $A$  without resorting to potentially expensive numeric calculations for each new value of  $A$ . However, finding an exact closed-form solution for a large system with additional parameters is often impossible. Therefore, you typically approximate a solution for such systems. This example approximates the parametric frequency response near the zero frequency as follows:

- Speed up computations by using variable-precision arithmetic (`vpa`).
- Find the Padé approximation of the transfer function  $H(s, A) = c(s^2M(A) + K(A))^{-1}F$  around the frequency  $s = 0$  using the first three moments of the series. The idea is that given a transfer function, you can compute the Padé approximation moments as  $c(-K(A))^{-1}M(A)^jK(A)^{-1}F$ , where  $j \in \{0, 2, 4, 6, \dots\}$  correspond to the power series terms  $\{1, s^2, s^4, s^6, \dots\}$ . For this example, compute `HApprox(s, A)` as the sum of the first three terms. This approach is a very basic technique for reducing the order of the transfer function.
- To further speed up calculations, approximate the inner term of each moment term in terms of a Taylor series expansion of  $A$  around `AFixed`.

Use `vpa` to speed up calculations.

```
digits(32);
K = vpa(K);
M = vpa(M);
```

Compute the LU decomposition of  $K$ .

```
[LK,UK] = lu(K);
```

Create helper variables and functions.

```
iK = @(x) UK\ (LK\x);
iK_M = @(x) -iK(M*x);
momentPre = iK(F);
```

Define a frequency series corresponding to the first three moments. Here,  $s$  is the frequency.

```
syms s
sPowers = [1 s^2 s^4];
```

Set the first moment, which is the same as `d_Static` in the previous computation.

```
moments = d_Static;
```

Compute the remaining moments.

```
for p=2:numel(sPowers)
    momentPre = iK_M(momentPre);
    for pp=1:numel(momentPre)
        momentPre(pp) = taylor(momentPre(pp),A,AFixed,'Order',3);
    end
    moment = c*momentPre;
    moments = [moments;moment];
end
```

Combine the moment terms to create the approximate analytical frequency response `Happrox`.

```
Happrox = sPowers*moments;
```

Display the first three moments. Because the expressions are large, you can display them only partially.

```
momentString = arrayfun(@char,moments,'UniformOutput',false);
freqString = arrayfun(@char,sPowers.','UniformOutput',false);
table(freqString,momentString,'VariableNames',{'FreqPowers','Moments'})
```

```
ans=3×2 table
    FreqPowers
```

|          |                                                                                 |
|----------|---------------------------------------------------------------------------------|
| {'1' }   | {'-(5^(1/2)*(220*A + 200*A*cos(8352332796509007/9007199254740992) + 116*5^(1/2  |
| {'s^2' } | {'0.0000000000084654421099019119162064316556812*(A - 1)^2 - 0.00000000007900124 |
| {'s^4' } | {'0.000000000000012982169746567833536274593916742*A - 0.000000000000150071410   |

Convert the frequency response to a MATLAB function containing numeric values for `A` and `s`. You can use the resulting MATLAB function without Symbolic Math Toolbox.

```
HapproxFun = matlabFunction(Happrox,'vars',[A,s]);
```

### Compare Purely Numeric and Symbolically Derived Frequency Responses

Vary the frequency from 0 to 1 in logspace, and then convert the range to radians.

```
freq = 2*pi*logspace(0,1);
```

Compute the numeric values of the frequency response for `A = AFixed*perturbFactor`, that is, for a small perturbation around `A`.

```
perturbFactor = 1 + 0.25;
KFixed = double(subs(K,A,AFixed*perturbFactor));
MFixed = double(subs(M,A,AFixed*perturbFactor));
H_Numeric = zeros(size(freq));
for k=1:numel(freq)
    sVal = 1i*freq(k);
```

```

    H_Numeric(k) = c*((sVal^2*MFixed + KFixed)\F);
end

```

Compute the symbolic values of the frequency response over the frequency range and  $A = \text{perturbFactor} * A_{\text{Fixed}}$ .

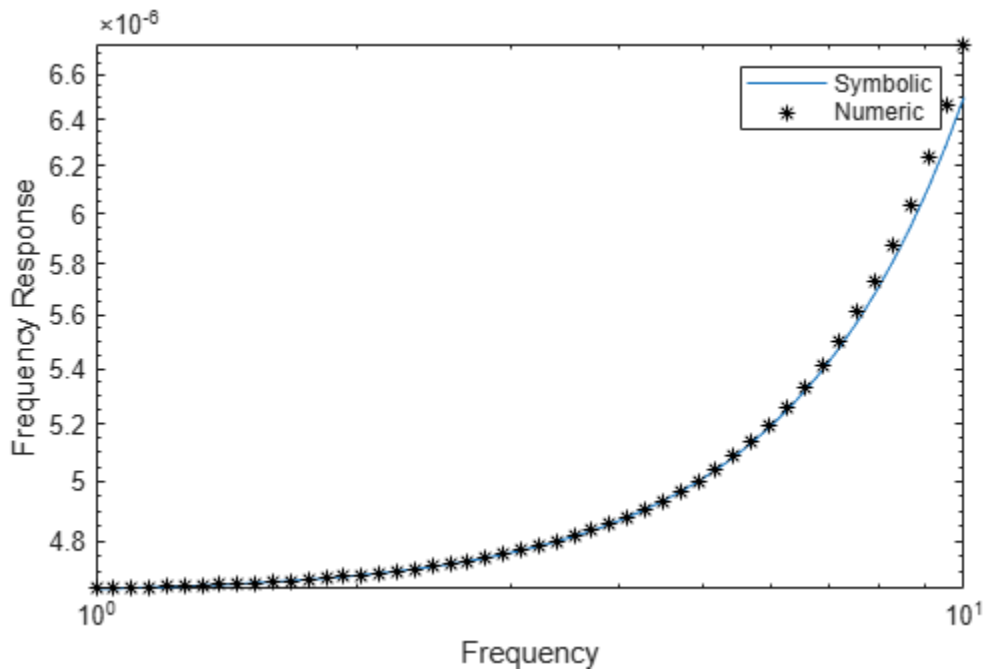
```
H_Symbolic = HApproxFun(AFixed*perturbFactor,freq*1i);
```

Plot the results.

```

figure
loglog(freq/(2*pi),abs(H_Symbolic),freq/(2*pi),abs(H_Numeric),'k*');
xlabel('Frequency');
ylabel('Frequency Response');
legend('Symbolic','Numeric');

```



The analytical and numeric curves are close in the chosen interval, but, in general, the curves diverge for frequencies outside the neighborhood of  $s = 0$ . Also, for values of  $A$  far from  $A_{\text{Fixed}}$ , the Taylor approximation introduces larger errors.



# Customize and Extend Simscape Libraries for a Custom DC Motor

Create custom-equation-based components for the Simscape™ library using the Symbolic Math Toolbox™.

## Introduction

The Symbolic Math Toolbox provides a flexible way to develop models from first engineering principles in any spatial dimension. You can develop mathematical models for steady state or transient physics.

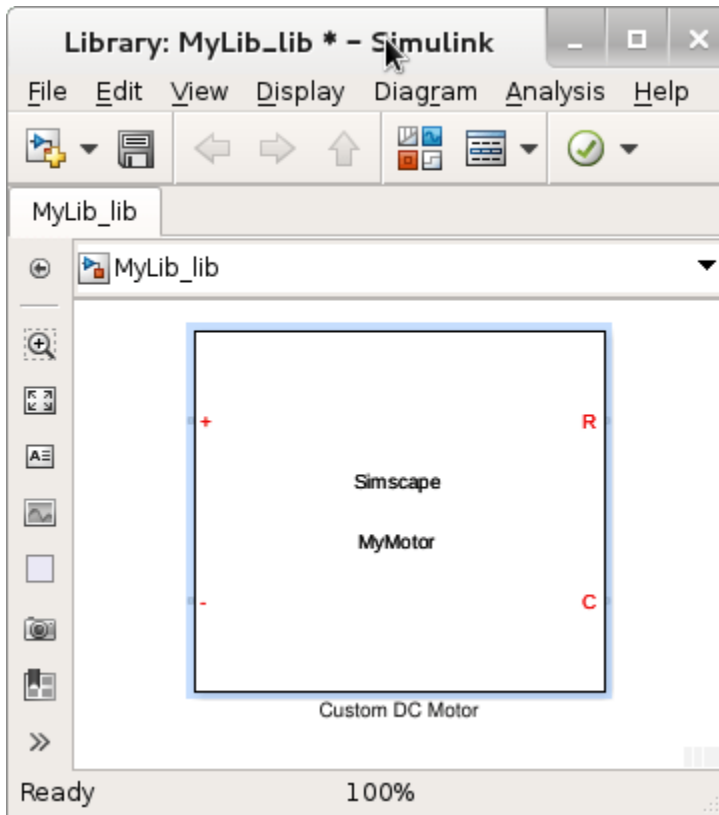
You can develop and solve the equations required to represent the physics necessary for your component; and perform your own reduced-order model mapping between the inputs  $\mathbf{x}$  and a quantity of interest  $\mathbf{f}(\mathbf{x})$ .

Here  $\mathbf{f}$  is the custom component which can represent governing equations in the form of:

- Mathematical formulas
- Numerical simulations of ODEs and PDEs

The steps in this examples are

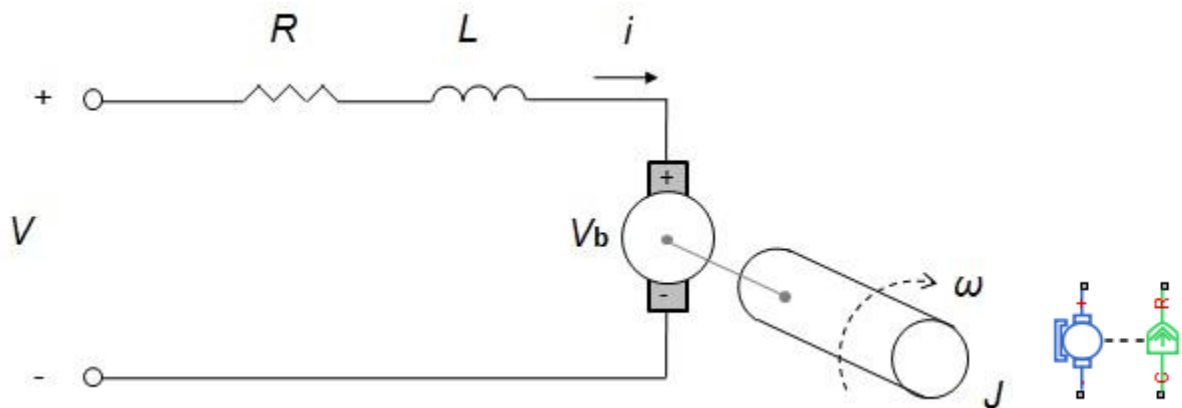
- Parameterizing a Simscape™ component using `symReadSSCVariables`
- Define custom equations for your Simscape component using `diff`
- Solve steady-state equations analytically using `solve` and `subs`
- Solve time-dependent equations numerically in MATLAB® by using `matlabFunction` and `ode45`
- Create a Simscape component using `symWriteSSC`



To run this example, you must have licenses for Simscape and Symbolic Math Toolbox.

### DC Motor Model

A DC motor is a device for converting electrical energy into mechanical energy and vice versa. A schematic of a DC motor is shown below (left figure). Blocks that simulate DC motors are provided in “Simscape Electrical”™ (right figure), which is an optional product for Simscape.



In this example, we will derive a reduced-order model representation of a DC Motor using the governing Ordinary Differential Equations (ODEs). For a DC motor, the electrical voltage and current is derived from Kirchhoff's laws and the formula of the mechanical torque is derived from Newton's laws. Using these equations we can implement a custom and parametric Simscape component.

$$\begin{pmatrix} J \frac{\partial}{\partial t} w(t) = K_i I(t) - D_r w(t) \\ L \frac{\partial}{\partial t} I(t) + R I(t) = V(t) - K_b w(t) \end{pmatrix}$$

### Import Parameters and Variables of Template Component

Suppose you have a Simscape component `MyMotorTemplate.ssc` in your current folder or on the default MATLAB path. This component has no equations yet. The template records the parameters and variables which will be used to develop our motor. You can use `type` to provide a preview of that template.

`type MyMotorTemplate.ssc`

```
component MyMotorTemplate
% Custom DC Motor
% This block implements a custom DC motor
nodes
    p = foundation.electrical.electrical; % +:left
    n = foundation.electrical.electrical; % -:left
    r = foundation.mechanical.rotational.rotational; % R:right
    c = foundation.mechanical.rotational.rotational; % C:right
end
parameters
    R = {3.9, 'Ohm'}; %Armature resistance
    L = {0.000012, 'H'}; %Armature inductance
    J = {0.000001, 'kg*m^2'}; %Inertia
    Dr = {0.000003, '(N*m*s)/rad'}; %Rotor damping
    Ki = {0.000072, '(N*m)/A'}; %Torque constant
    Kb = {0.000072, '(V*s)/rad'}; %Back-emf constant
end
variables
    torque = {0, 'N*m'}; %Total Torque
    tau = {0, 'N*m'}; %Electric Torque
    w = {0, 'rad/s'}; %Angular Velocity
    I = {0, 'A'}; %Current
    V = {0, 'V'}; %Applied voltage
    Vb = {0, 'V'}; %Counter electromotive force
end
branches
    torque : r.t -> c.t; % Through variable tau from r to c
    I : p.i -> n.i; % Through variable i from p to n
end
equations
    assert(R>0);
    w == r.w - c.w; % Across variable w from r to c
    V == p.v - n.v; % Across variable v from p to n
end
end
```

Read the names, values, and units of the parameters from the template component.

```
[parNames, parValues, parUnits] = symReadSSCParameters('MyMotorTemplate');
```

Display the parameters, their values, and the corresponding units as vectors.

```
vpa([parNames; parValues; parUnits],10)
```

```
ans =
```

$$\begin{pmatrix} Dr & J & Kb & Ki & L & R \\ 0.000003 & 0.000001 & 0.000072 & 0.000072 & 0.000012 & 3.9 \\ \frac{\text{N m s}}{\text{rad}} & \text{kg m}^2 & \frac{\text{V s}}{\text{rad}} & \frac{\text{N m}}{\text{A}} & \text{H} & \Omega \end{pmatrix}$$

Add the parameter names to the MATLAB Workspace by using the `syms` function. Parameters appear as symbolic variables in the workspace. You can use `who` to list variables in the workspace.

```
syms(parNames)
syms
```

Your symbolic variables are:

```
Dr    J    Kb    Ki    L    R    ans
```

Read and display the names of the component variables. Use `ReturnFunction` to simultaneously convert these variables to functions of the variable `t`.

```
[varFuns, varValues, varUnits] = symReadSSCVariables('MyMotorTemplate', 'ReturnFunction', true);
vpa([varFuns; varValues; varUnits],10)
```

```
ans =
```

$$\begin{pmatrix} I(t) & V(t) & Vb(t) & \tau(t) & \text{torque}(t) & w(t) \\ 0 & 0 & 0 & 0 & 0 & 0 \\ A & V & V & \text{Nm} & \text{Nm} & \frac{\text{rad}}{\text{s}} \end{pmatrix}$$

Add the variable names to the MATLAB Workspace by using the `syms` function. Variables appear as symbolic functions in the workspace. Verify that you have declared all required symbolic variables and functions using `syms`.

```
syms(varFuns)
syms
```

Your symbolic variables are:

```
Dr    I    J    Kb    Ki    L    R    V    Vb    ans    t    tau
```

### Define System of Equations for Modeling DC Motor

The differential equations for the mechanical torque are defined as `eq1` and `eq2`.  $I(t)$  represents the current and  $w(t)$  the angular velocity.

```
eq1 = torque + J*diff(w(t)) == -Dr*w(t) + tau(t)
```

```
eq1(t) =
```

$$J \frac{\partial}{\partial t} w(t) + \text{torque}(t) = \tau(t) - Dr w(t)$$

```
eq2 = tau(t) == Ki*I(t)
```

```
eq2 = \tau(t) = K_i I(t)
```

The equations for the electrical voltage and current are `eq3` and `eq4`.  $V(t)$  and  $Vb(t)$  represent the applied voltage and counter electromotive force respectively.

```
eq3 = L*diff(I(t)) + R*I(t) == V(t) - Vb(t)
```

```
eq3 =
```

$$L \frac{\partial}{\partial t} I(t) + R I(t) = V(t) - V_b(t)$$

$$\text{eq4} = V_b(t) == K_b \cdot w(t)$$

$$\text{eq4} = V_b(t) = K_b w(t)$$

We can list them together. Here, the motor's torque is assumed to be proportional to the current.

$$\text{eqs} = \text{formula}([\text{eq1}; \text{eq2}; \text{eq3}; \text{eq4}])$$

$$\text{eqs} = \begin{pmatrix} J \frac{\partial}{\partial t} w(t) + \text{torque}(t) = \tau(t) - D_r w(t) \\ \tau(t) = K_i I(t) \\ L \frac{\partial}{\partial t} I(t) + R I(t) = V(t) - V_b(t) \\ V_b(t) = K_b w(t) \end{pmatrix}$$

Extract the left and right sides of the equations.

$$\begin{aligned} \text{operands} &= \text{children}(\text{eqs}); \\ \text{operList} &= [\text{operands}\{:\}] ; \\ \text{lhs} &= \text{operList}(1:2:\text{end}) \end{aligned}$$

$$\text{lhs} = 1 \times 4 \text{ cell array} \\ \{[J * \text{diff}(w(t), t) + \text{torque}(t)]\} \quad \{[\tau(t)]\} \quad \{[L * \text{diff}(I(t), t) + R * I(t)]\} \quad \{[V_b(t)]\}$$

$$\text{rhs} = \text{operList}(2:2:\text{end})$$

$$\text{rhs} = 1 \times 4 \text{ cell array} \\ \{[\tau(t) - D_r * w(t)]\} \quad \{[K_i * I(t)]\} \quad \{[V(t) - V_b(t)]\} \quad \{[K_b * w(t)]\}$$

The second and fourth equations define the values  $\tau(t)$  and  $V_b(t)$ . To simplify the system of four equations to a system of two equations, substitute these values into the first and third equation.

$$\text{equations}(1) = \text{subs}(\text{eqs}(1), \text{lhs}(2), \text{rhs}(2))$$

$$\begin{aligned} \text{equations} &= \\ J \frac{\partial}{\partial t} w(t) + \text{torque}(t) &= K_i I(t) - D_r w(t) \end{aligned}$$

$$\text{equations}(2) = \text{subs}(\text{eqs}(3), \text{lhs}(4), \text{rhs}(4))$$

$$\begin{aligned} \text{equations} &= \\ \left( J \frac{\partial}{\partial t} w(t) + \text{torque}(t) = K_i I(t) - D_r w(t) \right. & \quad \left. L \frac{\partial}{\partial t} I(t) + R I(t) = V(t) - K_b w(t) \right) \end{aligned}$$

$$\text{equations.}'$$

$$\begin{aligned} \text{ans} &= \\ \left( J \frac{\partial}{\partial t} w(t) + \text{torque}(t) = K_i I(t) - D_r w(t) \right. & \quad \left. L \frac{\partial}{\partial t} I(t) + R I(t) = V(t) - K_b w(t) \right) \end{aligned}$$

Before solving the equations, substitute parameters with their numeric values. Also, use  $V(t) = 1$ .

```
equations = subs(equations, [parNames,V(t)], [parValues,1]);
equations = subs(equations, torque, 0);
vpa(equations.',10)
```

```
ans =

$$\begin{pmatrix} 0.000001 \frac{\partial}{\partial t} w(t) = 0.000072 I(t) - 0.000003 w(t) \\ 0.000012 \frac{\partial}{\partial t} I(t) + 3.9 I(t) = 1.0 - 0.000072 w(t) \end{pmatrix}$$

```

### Solve Equations for Steady State

For this, remove the time dependency for the functions  $w(t)$  and  $I(t)$ . For example, substitute them with symbolic variables  $ww$  and  $ii$ .

```
syms ww ii
equations_steady = subs(equations, [w(t),I(t)], [ww,ii]);
```

```
result = solve(equations_steady,ww,ii);
steadyStateW = vpa(result.ww,10)
```

```
steadyStateW = 6.151120734
```

```
steadyStateI = vpa(result.ii,10)
```

```
steadyStateI = 0.2562966973
```

### Numerically Simulate Symbolic Expression in MATLAB by Using matlabFunction and ode45

Create a valid input for `ode45` from the symbolic equations. Use `odeToVectorField` to create the MATLAB procedure which represents the dynamical system  $\frac{dY}{dt} = f(t, Y)$  with the initial condition  $Y(t_0) = Y_0$ .

```
[vfEquations, tVals] = odeToVectorField(equations)
```

```
vfEquations =

$$\begin{pmatrix} \frac{147573952589676412928}{1770887431076117} - 6 Y_2 - \frac{2877692075498690052096 Y_1}{8854437155380585} \\ \frac{83010348331692984375 Y_1}{1152921504606846976} - \frac{27670116110564328125 Y_2}{9223372036854775808} \end{pmatrix}$$

```

```
tVals =

$$\begin{pmatrix} I \\ w \end{pmatrix}$$

```

```
M = matlabFunction(vfEquations, 'Vars', {'t', 'Y'})
```

```
M = function_handle with value:
@ (t,Y) [Y(1).*(-3.25e+5)-Y(2).*6.0+8.333333333333333e+4;Y(1).*7.2e+1-Y(2).*3.0]
```

Solve the differential equation using the initial conditions  $w(0) = 0$  and  $I(0) = 0$ .

```
solution = ode45(M,[0 3],[0 0])
```

```
solution = struct with fields:
    solver: 'ode45'
    extdata: [1x1 struct]
```

```

x: [0 2.4114e-09 1.4468e-08 7.4754e-08 3.7618e-07 1.8833e-06 4.0507e-06 6.6351e-06 9.6
y: [2×293775 double]
stats: [1×1 struct]
idata: [1×1 struct]

```

Evaluate the solution at the following points in time  $t=[0.5,0.75,1]$ . The first value is the current  $I(t)$  and the second value is the angular velocity  $w(t)$ . We see that the solution for the angular velocity is starting to approach a steady state `steadyStateW`.

```
deval(solution,0.5), deval(solution,.75), deval(solution,1)
```

```
ans = 2×1
```

```

0.2563
4.7795

```

```
ans = 2×1
```

```

0.2563
5.5034

```

```
ans = 2×1
```

```

0.2563
5.8453

```

```
steadyStateW
```

```
steadyStateW = 6.151120734
```

Plot the solution.

```

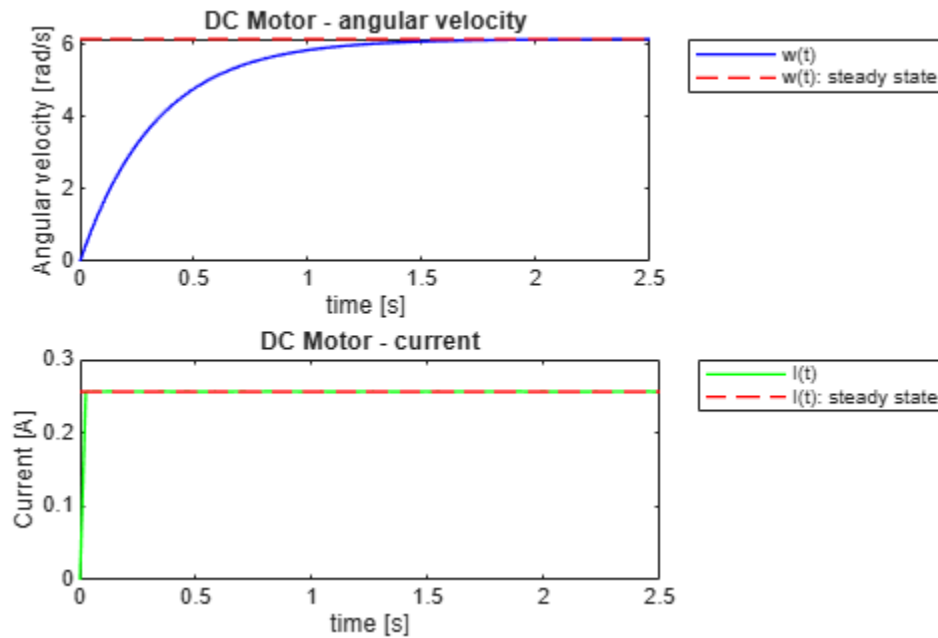
time = linspace(0,2.5);
iValues = deval(solution, time, 1);
wValues = deval(solution, time, 2);
steadyStateValuesI = vpa(steadyStateI*ones(1,100),10);
steadyStateValuesW = vpa(steadyStateW*ones(1,100),10);

figure;
plot1 = subplot(2,1,1);
plot2 = subplot(2,1,2);

plot(plot1, time, wValues, 'blue', time, steadyStateValuesW, '--red', 'LineWidth', 1)
plot(plot2, time, iValues, 'green', time, steadyStateValuesI, '--red', 'LineWidth', 1)

title(plot1, 'DC Motor - angular velocity')
title(plot2, 'DC Motor - current')
ylabel(plot1, 'Angular velocity [rad/s]')
ylabel(plot2, 'Current [A]')
xlabel(plot1, 'time [s]')
xlabel(plot2, 'time [s]')
legend(plot1, 'w(t)', 'w(t): steady state', 'Location', 'northeastoutside')
legend(plot2, 'I(t)', 'I(t): steady state', 'Location', 'northeastoutside')

```



### Save Your Mathematical Model for Use in Simscape

Generate Simscape code using the original equations eqs.

```
symWriteSSC('MyMotor.ssc', 'MyMotorTemplate.ssc', eqs, ...
    'H1Header', '% Custom DC Motor', ...
    'HelpText', {'% This block implements a custom DC motor'})
```

Display the generated component by using the type command.

```
type MyMotor.ssc
```

```
component MyMotor
% Custom DC Motor
% This block implements a custom DC motor
nodes
    p = foundation.electrical.electrical; % +:left
    n = foundation.electrical.electrical; % -:left
    r = foundation.mechanical.rotational.rotational; % R:right
    c = foundation.mechanical.rotational.rotational; % C:right
end
parameters
    R = {3.9, 'Ohm'}; %Armature resistance
    L = {0.000012, 'H'}; %Armature inductance
    J = {0.000001, 'kg*m^2'}; %Inertia
    Dr = {0.000003, '(N*m*s)/rad'}; %Rotor damping
    Ki = {0.000072, '(N*m)/A'}; %Torque constant
    Kb = {0.000072, '(V*s)/rad'}; %Back-emf constant
end
variables
    torque = {0, 'N*m'}; %Total Torque
    tau = {0, 'N*m'}; %Electric Torque
    w = {0, 'rad/s'}; %Angular Velocity
    I = {0, 'A'}; %Current
```



```

        V = {0, 'V'};           %Applied voltage
        Vb = {0, 'V'};         %Counter electromotive force
    end
    branches
        torque : r.t -> c.t; % Through variable tau from r to c
        I      : p.i -> n.i; % Through variable i from p to n
    end
    equations
        assert(R>0);
        w == r.w -c.w; % Across variable w from r to c
        V == p.v -n.v; % Across variable v from p to n
        torque+J*w.der == tau-Dr*w;
        tau == Ki*I;
        L*I.der+R*I == V-Vb;
        Vb == Kb*w;
    end
end
end

```

Build a Simscape library from the generated component.

```

if ~isfolder('+MyLib')
    mkdir +MyLib;
end
copyfile MyMotor.ssc +MyLib;
sscbuild MyLib;

```

Generating Simulink library 'MyLib\_lib' in the current directory 'C:\TEMP\tp2d6d0fee\symbolic-ex'.  
Product info can only be assigned for libraries under matlab\toolbox folder.

## See Also

### Functions

[diff](#) | [subs](#) | [solve](#) | [matlabFunction](#) | [ode45](#) | [symReadSSCParameters](#) | [symReadSSCVariables](#) | [symWriteSSC](#)

## Related Examples

- “Solve Differential Equations of RLC Circuit Using Laplace Transform” on page 3-198
- “Analyze Transfer Function of T-Coil Circuit” on page 3-324
- “Harmonic Analysis of Transfer Function Output” on page 3-160
- “Padé Approximant of Time-Delay Input” on page 3-261
- “Estimate Model Parameters of a Symbolically Derived Plant Model in Simulink” on page 5-74

## Estimate Model Parameters of a Symbolically Derived Plant Model in Simulink

This example uses Simulink Design Optimization to estimate the unknown capacitance and initial voltage of a symbolically derived algebraic model of a simple resistor-capacitor (RC) circuit. The example solves the same problem and uses the same experimental data as “Estimate Model Parameters and Initial States (Code)” (Simulink Design Optimization), but uses the closed-form solution for the RC circuit instead of the differential form.

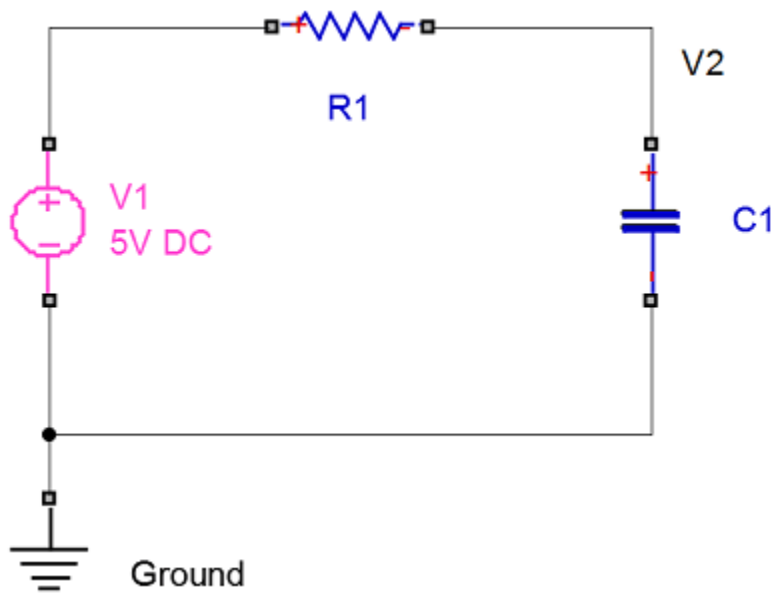
This example uses Symbolic Math Toolbox™ capabilities to:

- Solve ordinary differential equations (ODE) using `dsolve`
- Convert an analytical result into a Simulink® block using `matlabFunctionBlock`

You perform design optimization to estimate the capacitance and initial voltage values of the analytical RC circuit. In particular, you match the experimental output voltage values with the simulated values.

### Solve Equation for RC Circuit

Define and solve the following differential equation for the RC circuit.



Here,  $v_2(t)$  is the output voltage across capacitor C1,  $v_1$  is the constant voltage across resistor R1, and  $v_{20}$  is the initial voltage across the capacitor. Use `dsolve` to solve the equation.

```
syms C1 R1 v1 v20 real
syms v2(t)
deq = (v1 - v2)/R1 - C1*diff(v2,t);
v2sol = dsolve(deq, v2(0) == v20)

v2sol =
```

$$v_1 - e^{-\frac{t}{C_1 R_1}} (v_1 - v_{20})$$

Use `subs` to evaluate the solution for an  $R_1$  value of 10 kOhm and  $v_1$  value of 5 V.

```
v2sol = vpa(subs(v2sol,[R1,v1],[10e3,5]))

v2sol =
    e- $\frac{0.0001 t}{C_1}$  (v20 - 5.0) + 5.0
```

### Create Model with Block Representing RC Circuit

First, create a new Simulink model.

```
myModel = 'rcSymbolic';
new_system(myModel);
load_system(myModel);
```

Use `matlabFunctionBlock` to convert the symbolic result for the output voltage to a Simulink block representing the RC plant model. `matlabFunctionBlock` adds this new block to the model.

```
blockName = 'closedFormRC_block';
rcBlock = strcat(myModel,'/',blockName);
myVars = [C1,v20,t];
matlabFunctionBlock(rcBlock,v2sol,...
    'vars',myVars,...
    'functionName','myRC',...
    'outputs',{'v2'});
```

### Add More Blocks

Add and arrange other blocks with positions and sizes relative to the RC block.

```
rcBlockPosition = get_param(rcBlock,'position');
rcBlockWidth = rcBlockPosition(3)-rcBlockPosition(1);
rcBlockHeight = rcBlockPosition(4)-rcBlockPosition(2);
constantBlock = 'built-in/Constant';
timeBlock = 'simulink/Sources/Ramp';
outputBlock = 'built-in/Outport';
```

$C_1$  and  $v_{20}$  are the parameters to estimate. First, introduce and initialize these parameters in the MATLAB® workspace, using the initial values of 460 μF and 1 V, respectively. Then create constant blocks for both parameters.

```
C1val = 460e-6;
v20val = 1.0;
posX = rcBlockPosition(1)-rcBlockWidth*2;
posY = rcBlockPosition(2)-rcBlockHeight*3/4;
pos = [posX,posY,posX+rcBlockWidth/2,posY+rcBlockHeight/2];
add_block(constantBlock,strcat(myModel,'/C1'),'Value','C1val',...
    'Position',pos);
pos = pos + [0 rcBlockHeight 0 rcBlockHeight];
add_block(constantBlock,strcat(myModel,'/v20'),'Value','v20val',...
    'Position',pos);
```

Add a ramp for time.

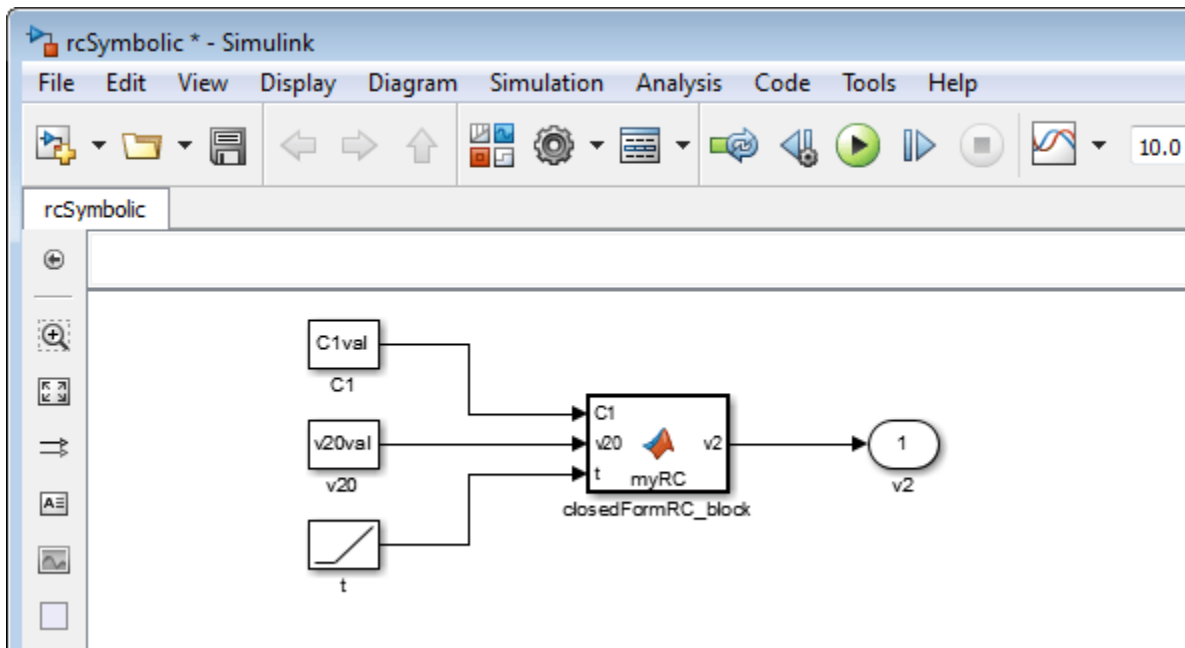
```
pos = pos + [0 rcBlockHeight 0 rcBlockHeight];
add_block(timeBlock, strcat(myModel, '/t'), 'Slope', '1', 'Position', pos);
```

Add an output port.

```
pos = [rcBlockPosition(1)+2*rcBlockWidth, ...
       rcBlockPosition(2)+rcBlockHeight/4, ...
       rcBlockPosition(1)+2*rcBlockWidth+rcBlockWidth/2, ...
       rcBlockPosition(2)+rcBlockHeight/4+rcBlockHeight/2];
add_block(outputBlock, strcat(myModel, '/v2'), 'Port', '1', 'Position', pos);
```

Now, wire blocks in the model. The model is ready for Simulink Design Optimization.

```
myAddLine = @(k) add_line(myModel, ...
    strcat(char(myVars(k)), '/1'), ...
    strcat(blockName, '/', num2str(k)), ...
    'autorouting', 'on');
arrayfun(myAddLine, (1:numel(myVars)));
add_line(myModel, strcat(blockName, '/1'), 'v2/1', 'autorouting', 'on');
open_system(myModel);
```



### Estimate Parameters

Get the measured data.

```
load sdoRCCircuit_DataFromExperiment
```

The variables `time` and `data` are loaded into the workspace. `data` is the measured capacitor voltage for times `time`.

Create an `sdo.Experiment` object to store the experimental voltage data.

```
Exp = sdo.Experiment(myModel);
```

Create an object to store the measured capacitor voltage output.

```

Voltage = Simulink.SimulationData.Signal;
Voltage.Name = 'Voltage';
Voltage.BlockPath = rcBlock;
Voltage.PortType = 'outport';
Voltage.PortIndex = 1;
Voltage.Values = timeseries(data,time);

```

Add the measured capacitor data to the experiment as the expected output data.

```
Exp.OutputData = Voltage;
```

Get the parameters. Set a minimum value for C1. Note that you already specified the initial guesses.

```

clparam = sdo.getParameterFromModel(myModel, 'C1val');
clparam.Minimum = 0;
v20param = sdo.getParameterFromModel(myModel, 'v20val');

```

Define the objective function for estimation. The code for `sdoRCSymbolic_Objective` used in this example is given in the Helper Functions section at the end of the example.

```
estFcn = @(v) sdoRCSymbolic_Objective(v,Exp,myModel);
```

Collect the model parameters to be estimated.

```
v = [clparam;v20param];
```

Because the model is entirely algebraic, turn off warning messages that instruct you to select a discrete solver.

```
set_param(myModel, 'SolverPrmCheckMsg', 'none');
```

Estimate the parameters.

```

opt = sdo.OptimizeOptions;
opt.Method = 'lsqnonlin';
v0pt = sdo.optimize(estFcn,v,opt);

```

```
Optimization started 2026-Apr-20, 23:58:38
```

| Iter | F-count | f(x)    | Step-size | First-order optimality |
|------|---------|---------|-----------|------------------------|
| 0    | 5       | 27.7093 | 1         |                        |
| 1    | 10      | 2.86889 | 1.919     | 2.94                   |
| 2    | 15      | 1.53851 | 0.3832    | 0.523                  |
| 3    | 20      | 1.35137 | 0.3347    | 0.505                  |
| 4    | 25      | 1.34473 | 0.01374   | 0.00842                |
| 5    | 30      | 1.34472 | 0.002686  | 0.00141                |

Local minimum possible.

`lsqnonlin` stopped because the final change in the sum of squares relative to its initial value is less than the value of the function tolerance.

Show the estimated values.

```
fprintf('C1 = %e v20 = %e\n',v0pt(1).Value, v0pt(2).Value);
```

```
C1 = 2.261442e-04 v20 = 2.359446e+00
```

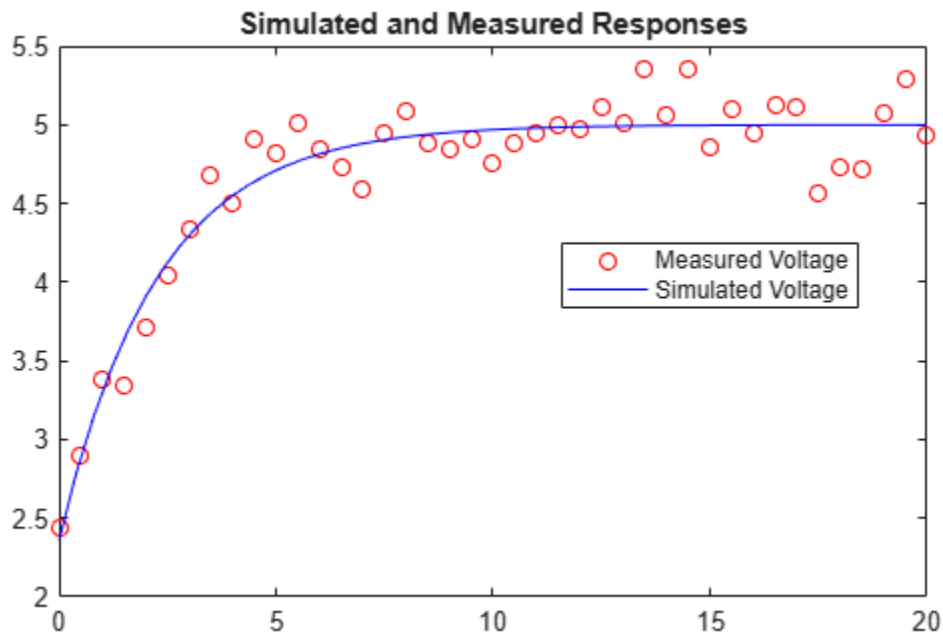
## Compare Simulated and Experimental Data

Update the experiment with the estimated capacitance and capacitor initial voltage values.

```
Exp = setEstimatedValues(Exp,v0pt);
```

Simulate the model with the estimated parameter values and compare the simulated output with the experimental data.

```
Simulator = createSimulator(Exp);
Simulator = sim(Simulator);
SimLog = find(Simulator.LoggedData,get_param(myModel,'SignalLoggingName'));
Voltage = find(SimLog,'Voltage');
plot(time,data,'ro',Voltage.Values.Time,Voltage.Values.Data,'b')
title('Simulated and Measured Responses')
legend('Measured Voltage','Simulated Voltage','Location','Best')
```



```
close_system(myModel,0);
```

## Helper Functions

```
function vals = sdoRCSymbolic_Objective(v,Exp,myModel)
    r = sdo.requirements.SignalTracking;
    r.Type      = '==';
    r.Method    = 'Residuals';
    r.Normalize = 'off';
    Exp = setEstimatedValues(Exp,v);
    Simulator = createSimulator(Exp);
    Simulator = sim(Simulator);
    SimLog = find(Simulator.LoggedData,get_param(myModel,'SignalLoggingName'));
    Voltage = find(SimLog,'Voltage');
    VoltageError = evalRequirement(r,Voltage.Values,Exp.OutputData(1).Values);
```

```
    vals.F = VoltageError(:);  
end
```

## See Also

### Related Examples

- “Solve Differential Equations of RLC Circuit Using Laplace Transform” on page 3-198
- “Analyze Transfer Function of T-Coil Circuit” on page 3-324
- “Harmonic Analysis of Transfer Function Output” on page 3-160
- “Padé Approximant of Time-Delay Input” on page 3-261
- “Customize and Extend Simscape Libraries for a Custom DC Motor” on page 5-65





# Functions

---

## abs

Symbolic absolute value (complex modulus or magnitude)

### Syntax

```
abs(z)
```

### Description

`abs(z)` returns the absolute value (or complex modulus) of  $z$ . Because symbolic variables are assumed to be complex by default, `abs` returns the complex modulus (magnitude) by default. If  $z$  is an array, `abs` acts element-wise on each element of  $z$ .

### Examples

#### Compute Absolute Values of Symbolic Numbers

```
[abs(sym(1/2)), abs(sym(0)), abs(sym(pi) - 4)]
```

```
ans =  
[ 1/2, 0, 4 - pi]
```

#### Compute Absolute Value of Complex Numbers

Compute `abs(x)^2` and simplify the result. Because symbolic variables are assumed to be complex by default, the result does not simplify to  $x^2$ .

```
syms x  
simplify(abs(x)^2)
```

```
ans =  
abs(x)^2
```

Assume  $x$  is real, and repeat the calculation. Now, the result is simplified to  $x^2$ .

```
assume(x, 'real')  
simplify(abs(x)^2)
```

```
ans =  
x^2
```

Remove assumptions on  $x$  for further calculations. For details, see “Use Assumptions on Symbolic Variables” on page 1-41.

```
assume(x, 'clear')
```

## Absolute Values of Elements of Array

Compute the absolute values of each element of matrix A.

```
A = sym([1/2+i  -25;
         i      pi/2]);
abs(A)

ans =
[ 5^(1/2)/2,  25]
[          1, pi/2]
```

## Effect of Assumptions on Absolute Value

Compute the absolute value of this expression assuming that the value of x is negative.

```
syms x
assume(x < 0)
abs(5*x^3)

ans =
-5*x^3
```

For further computations, clear the assumption on x by recreating it using syms:

```
syms x
```

## Input Arguments

### z — Input

number | vector | matrix | multidimensional array | symbolic number | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, vector, matrix, or array, variable, function, or expression.

## More About

### Complex Modulus

The absolute value of a complex number  $z = x + y*i$  is the value  $|z| = \sqrt{x^2 + y^2}$ . Here, x and y are real numbers. The absolute value of a complex number is also called a complex modulus.

## Tips

- Calling abs for a number that is not a symbolic object invokes the MATLAB abs function.

## Version History

Introduced before R2006a

### See Also

`angle` | `imag` | `real` | `sign` | `signIm`

## acos

Symbolic inverse cosine function

### Syntax

`acos(X)`

### Description

`acos(X)` returns the inverse cosine function (arccosine function) of  $X$ . All angles are in radians.

- For real values of  $X$  in the interval  $[-1, 1]$ , `acos(x)` returns the values in the interval  $[0, \pi]$ .
- For real values of  $X$  outside the interval  $[-1, 1]$  and for complex values of  $X$ , `acos(X)` returns complex values with the real parts in the interval  $[0, \pi]$ .

### Examples

#### Inverse Cosine Function for Numeric and Symbolic Arguments

Depending on its arguments, `acos` returns floating-point or exact symbolic results.

Compute the inverse cosine function for these numbers. Because these numbers are not symbolic objects, `acos` returns floating-point results.

```
A = acos([-1, -1/3, -1/2, 1/4, 1/2, sqrt(3)/2, 1])
```

```
A =
    3.1416    1.9106    2.0944    1.3181    1.0472    0.5236    0
```

Compute the inverse cosine function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `acos` returns unresolved symbolic calls.

```
symA = acos(sym([-1, -1/3, -1/2, 1/4, 1/2, sqrt(3)/2, 1]))
```

```
symA =
[ pi, pi - acos(1/3), (2*pi)/3, acos(1/4), pi/3, pi/6, 0]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

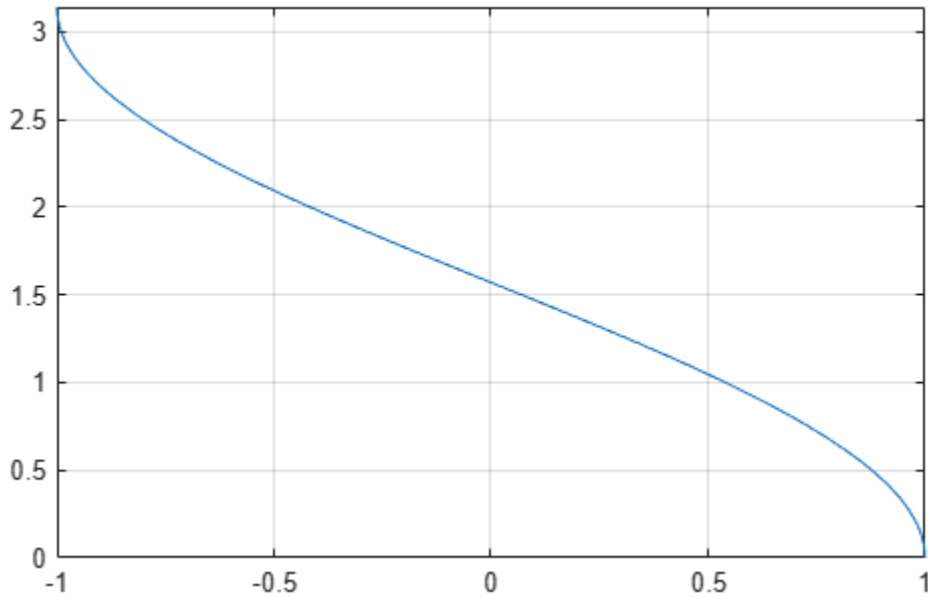
```
vpa(symA)

ans =
[ 3.1415926535897932384626433832795,...
 1.9106332362490185563277142050315,...
 2.0943951023931954923084289221863,...
 1.318116071652817965745664254646,...
 1.0471975511965977461542144610932,...
 0.52359877559829887307710723054658,...
 0]
```

#### Plot Inverse Cosine Function

Plot the inverse cosine function on the interval from -1 to 1.

```
syms x
fplot(acos(x), [-1 1])
grid on
```



### Handle Expressions Containing Inverse Cosine Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `acos`.

Find the first and second derivatives of the inverse cosine function:

```
syms x
diff(acos(x), x)
diff(acos(x), x, x)
```

```
ans =
-1/(1 - x^2)^(1/2)
```

```
ans =
-x/(1 - x^2)^(3/2)
```

Find the indefinite integral of the inverse cosine function:

```
int(acos(x), x)
```

```
ans =
x*acos(x) - (1 - x^2)^(1/2)
```

Find the Taylor series expansion of `acos(x)`:

```
taylor(acos(x), x)
```

```
ans =  
- (3*x^5)/40 - x^3/6 - x + pi/2
```

Rewrite the inverse cosine function in terms of the natural logarithm:

```
rewrite(acos(x), 'log')
```

```
ans =  
-log(x + (1 - x^2)^(1/2)*1i)*1i
```

## Input Arguments

### **X — Input**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Version History

**Introduced before R2006a**

### **See Also**

asin | atan | acsc | asec | acot | sin | cos | tan | csc | sec | cot

## acosh

Symbolic inverse hyperbolic cosine function

### Syntax

`acosh(X)`

### Description

`acosh(X)` returns the inverse hyperbolic cosine function of  $X$ .

### Examples

#### Inverse Hyperbolic Cosine Function for Numeric and Symbolic Arguments

Depending on its arguments, `acosh` returns floating-point or exact symbolic results.

Compute the inverse hyperbolic cosine function for these numbers. Because these numbers are not symbolic objects, `acosh` returns floating-point results.

```
A = acosh([-1, 0, 1/6, 1/2, 1, 2])
```

```
A =
    0.0000 + 3.1416i    0.0000 + 1.5708i    0.0000 + 1.4033i...
    0.0000 + 1.0472i    0.0000 + 0.0000i    1.3170 + 0.0000i
```

Compute the inverse hyperbolic cosine function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `acosh` returns unresolved symbolic calls.

```
symA = acosh(sym([-1, 0, 1/6, 1/2, 1, 2]))
```

```
symA =
[ pi*I, (pi*I)/2, acosh(1/6), (pi*I)/3, 0, acosh(2)]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

```
vpa(symA)
```

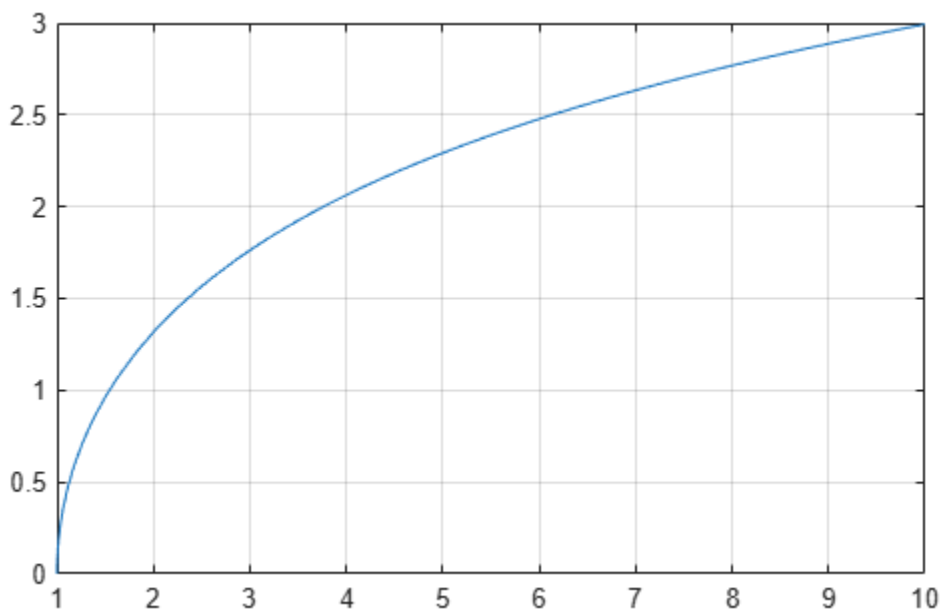
```
ans =
[ 3.1415926535897932384626433832795i,...
 1.5707963267948966192313216916398i,...
 1.4033482475752072886780470855961i,...
 1.0471975511965977461542144610932i,...
 0,...
 1.316957896924816708625046347308]
```

#### Plot Inverse Hyperbolic Cosine Function

Plot the inverse hyperbolic cosine function on the interval from 1 to 10.



```
syms x
fplot(acosh(x), [1 10])
grid on
```



### Handle Expressions Containing Inverse Hyperbolic Cosine Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `acosh`.

Find the first and second derivatives of the inverse hyperbolic cosine function. Simplify the second derivative by using `simplify`.

```
syms x
diff(acosh(x), x)
simplify(diff(acosh(x), x, x))

ans =
1/((x - 1)^(1/2)*(x + 1)^(1/2))

ans =
-x/((x - 1)^(3/2)*(x + 1)^(3/2))
```

Find the indefinite integral of the inverse hyperbolic cosine function. Simplify the result by using `simplify`.

```
int(acosh(x), x)

ans =
x*acosh(x) - (x - 1)^(1/2)*(x + 1)^(1/2)
```

Find the Taylor series expansion of `acosh(x)` for  $x > 1$ :

```
assume(x > 1)
taylor(acosh(x), x)

ans =
(x^5*3i)/40 + (x^3*i)/6 + x*i - (pi*i)/2
```

For further computations, clear the assumption on  $x$  by recreating it using `syms`:

```
syms x
```

Rewrite the inverse hyperbolic cosine function in terms of the natural logarithm:

```
rewrite(acosh(x), 'log')

ans =
log(x + (x - 1)^(1/2)*(x + 1)^(1/2))
```

## Input Arguments

### **X — Input**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Version History

**Introduced before R2006a**

### **See Also**

`asinh` | `atanh` | `acsch` | `asech` | `acoth` | `sinh` | `cosh` | `tanh` | `csch` | `sech` | `coth`

# acot

Symbolic inverse cotangent function

## Syntax

`acot(X)`

## Description

`acot(X)` returns the inverse cotangent function (arccotangent function) of  $X$ . All angles are in radians.

- For real values of  $X$ , `acot(X)` returns values in the interval  $[-\pi/2, \pi/2]$ .
- For complex values of  $X$ , `acot(X)` returns complex values with the real parts in the interval  $[-\pi/2, \pi/2]$ .

## Examples

### Inverse Cotangent Function for Numeric and Symbolic Arguments

Depending on its arguments, `acot` returns floating-point or exact symbolic results.

Compute the inverse cotangent function for these numbers. Because these numbers are not symbolic objects, `acot` returns floating-point results.

```
A = acot([-1, -1/3, -1/sqrt(3), 1/2, 1, sqrt(3)])
```

```
A =  
-0.7854    -1.2490    -1.0472    1.1071    0.7854    0.5236
```

Compute the inverse cotangent function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `acot` returns unresolved symbolic calls.

```
symA = acot(sym([-1, -1/3, -1/sqrt(3), 1/2, 1, sqrt(3)]))
```

```
symA =  
[-pi/4, -acot(1/3), -pi/3, acot(1/2), pi/4, pi/6]
```

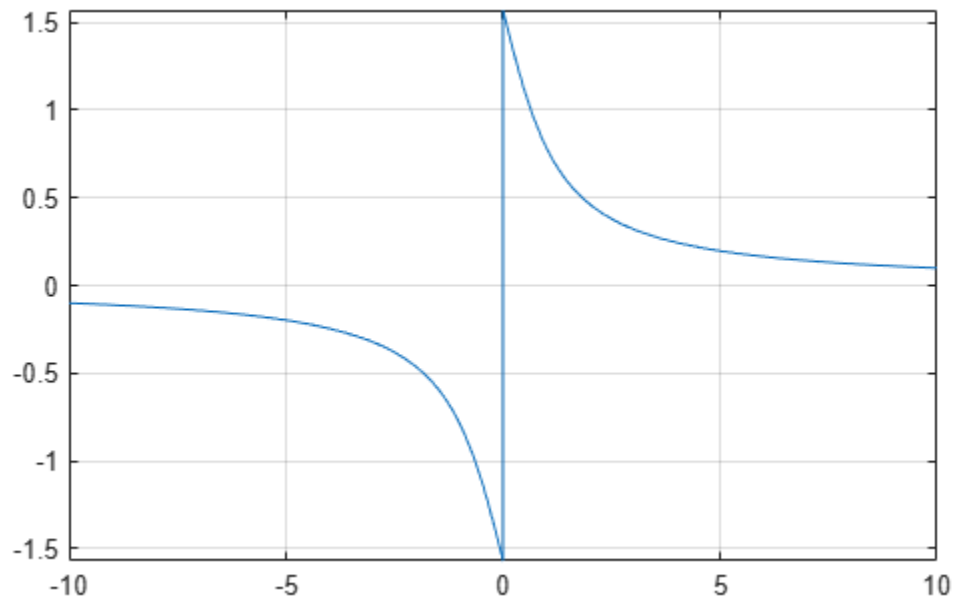
Use `vpa` to approximate symbolic results with floating-point numbers:

```
vpa(symA)  
  
ans =  
[-0.78539816339744830961566084581988,...  
-1.2490457723982544258299170772811,...  
-1.0471975511965977461542144610932,...  
1.1071487177940905030170654601785,...  
0.78539816339744830961566084581988,...  
0.52359877559829887307710723054658]
```

### Plot Inverse Cotangent Function

Plot the inverse cotangent function on the interval from -10 to 10.

```
syms x
fplot(acot(x), [-10 10])
grid on
```



### Handle Expressions Containing Inverse Cotangent Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `acot`.

Find the first and second derivatives of the inverse cotangent function:

```
syms x
diff(acot(x), x)
diff(acot(x), x, x)
```

```
ans =
-1/(x^2 + 1)
```

```
ans =
(2*x)/(x^2 + 1)^2
```

Find the indefinite integral of the inverse cotangent function:

```
int(acot(x), x)

ans =
log(x^2 + 1)/2 + x*acot(x)
```

Find the Taylor series expansion of `acot(x)` for  $x > 0$ :

```
assume(x > 0)
taylor(acot(x), x)
```

```
ans =
- x^5/5 + x^3/3 - x + pi/2
```

For further computations, clear the assumption on  $x$  by recreating it using `syms`:

```
syms x
```

Rewrite the inverse cotangent function in terms of the natural logarithm:

```
rewrite(acot(x), 'log')
```

```
ans =
(log(1 - 1i/x)*1i)/2 - (log(1i/x + 1)*1i)/2
```

## Input Arguments

### X — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Version History

Introduced before R2006a

### See Also

`acos` | `asin` | `atan` | `acsc` | `asec` | `sin` | `cos` | `tan` | `csc` | `sec` | `cot`

## acoth

Symbolic inverse hyperbolic cotangent function

### Syntax

`acoth(X)`

### Description

`acoth(X)` returns the inverse hyperbolic cotangent function of  $X$ .

### Examples

#### Inverse Hyperbolic Cotangent Function for Numeric and Symbolic Arguments

Depending on its arguments, `acoth` returns floating-point or exact symbolic results.

Compute the inverse hyperbolic cotangent function for these numbers. Because these numbers are not symbolic objects, `acoth` returns floating-point results.

```
A = acoth([-pi/2, -1, 0, 1/2, 1, pi/2])
```

```
A =
-0.7525 + 0.0000i    -Inf + 0.0000i    0.0000 + 1.5708i...
 0.5493 + 1.5708i    Inf + 0.0000i    0.7525 + 0.0000i
```

Compute the inverse hyperbolic cotangent function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `acoth` returns unresolved symbolic calls.

```
symA = acoth(sym([-pi/2, -1, 0, 1/2, 1, pi/2]))
```

```
symA =
[ -acoth(pi/2), Inf, -(pi*1i)/2, acoth(1/2), Inf, acoth(pi/2)]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

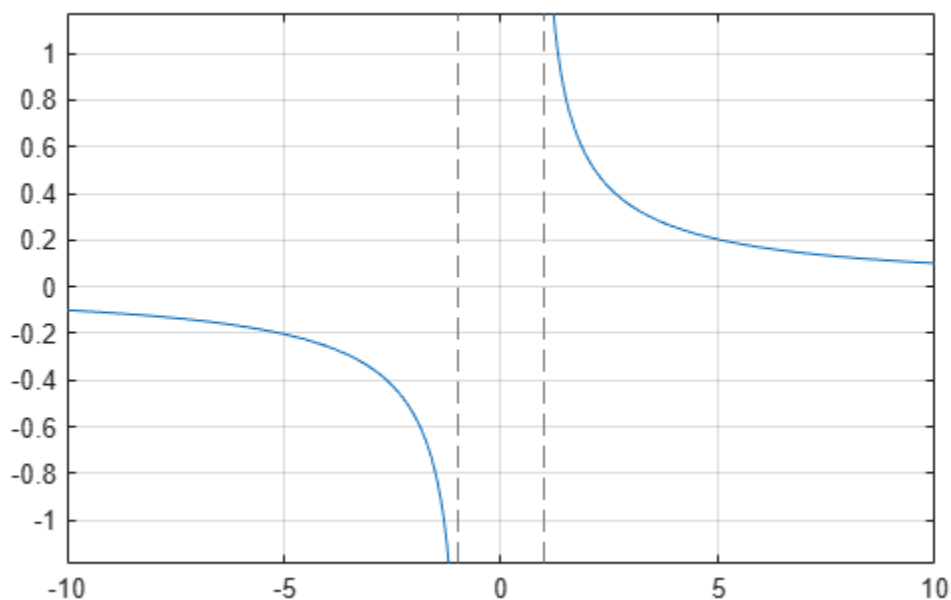
```
vpa(symA)

ans =
[ -0.75246926714192715916204347800251,...
  Inf,...
 -1.5707963267948966192313216916398i,...
 0.54930614433405484569762261846126...
 - 1.5707963267948966192313216916398i,...
  Inf,...
 0.75246926714192715916204347800251]
```

#### Plot Inverse Hyperbolic Cotangent Function

Plot the inverse hyperbolic cotangent function on the interval from -10 to 10.

```
syms x
fplot(acoth(x), [-10 10])
grid on
```



### Handle Expressions Containing Inverse Hyperbolic Cotangent Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `acoth`.

Find the first and second derivatives of the inverse hyperbolic cotangent function:

```
syms x
diff(acoth(x), x)
diff(acoth(x), x, x)
```

```
ans =
-1/(x^2 - 1)
```

```
ans =
(2*x)/(x^2 - 1)^2
```

Find the indefinite integral of the inverse hyperbolic cotangent function:

```
int(acoth(x), x)
```

```
ans =
log(x^2 - 1)/2 + x*acoth(x)
```

Find the Taylor series expansion of `acoth(x)` for  $x > 0$ :

```
assume(x > 0)
taylor(acoth(x), x)
```

```
ans =  
x^5/5 + x^3/3 + x - (pi*I)/2
```

For further computations, clear the assumption on  $x$  by recreating it using `syms`:

```
syms x
```

Rewrite the inverse hyperbolic cotangent function in terms of the natural logarithm:

```
rewrite(acoth(x), 'log')  
  
ans =  
log(1/x + 1)/2 - log(1 - 1/x)/2
```

## Input Arguments

### **X — Input**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Version History

**Introduced before R2006a**

### **See Also**

`asinh` | `acosh` | `atanh` | `acsch` | `asech` | `sinh` | `cosh` | `tanh` | `csch` | `sech` | `coth`



## acsc

Symbolic inverse cosecant function

### Syntax

`acsc(X)`

### Description

`acsc(X)` returns the inverse cosecant function (arccosecant function) of  $X$ . All angles are in radians.

- For real values of  $X$  in intervals  $[-\text{Inf}, -1]$  and  $[1, \text{Inf}]$ , `acsc` returns real values in the interval  $[-\pi/2, \pi/2]$ .
- For real values of  $X$  in the interval  $[-1, 1]$  and for complex values of  $X$ , `acsc` returns complex values with the real parts in the interval  $[-\pi/2, \pi/2]$ .

### Examples

#### Inverse Cosecant Function for Numeric and Symbolic Arguments

Depending on its arguments, `acsc` returns floating-point or exact symbolic results.

Compute the inverse cosecant function for these numbers. Because these numbers are not symbolic objects, `acsc` returns floating-point results.

```
A = acsc([-2, 0, 2/sqrt(3), 1/2, 1, 5])
```

```
A =
-0.5236 + 0.0000i    1.5708 -      Inf    1.0472 + 0.0000i    1.5708...
- 1.3170i    1.5708 + 0.0000i    0.2014 + 0.0000i
```

Compute the inverse cosecant function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `acsc` returns unresolved symbolic calls.

```
symA = acsc(sym([-2, 0, 2/sqrt(3), 1/2, 1, 5]))
```

```
symA =
[ -pi/6, Inf, pi/3, asin(2), pi/2, asin(1/5)]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

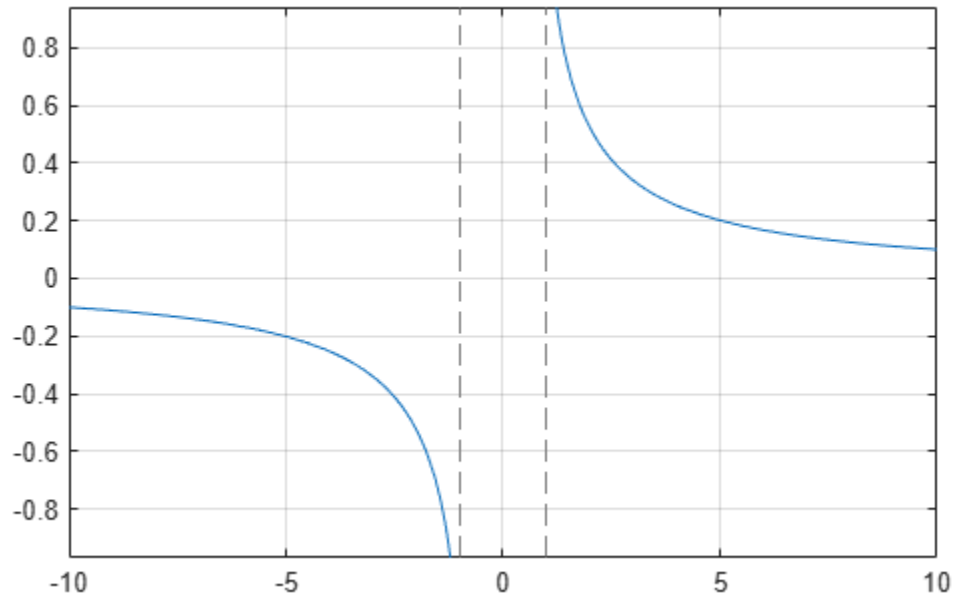
```
vpa(symA)
```

```
ans =
[ -0.52359877559829887307710723054658,...
  Inf,...
  1.0471975511965977461542144610932,...
  1.5707963267948966192313216916398...
  - 1.3169578969248165734029498707969i,...
  1.5707963267948966192313216916398,...
  0.20135792079033079660099758712022]
```

### Plot Inverse Cosecant Function

Plot the inverse cosecant function on the interval from -10 to 10.

```
syms x
fplot(acsc(x), [-10 10])
grid on
```



### Handle Expressions Containing Inverse Cosecant Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `acsc`.

Find the first and second derivatives of the inverse cosecant function:

```
syms x
diff(acsc(x), x)
diff(acsc(x), x, x)

ans =
-1/(x^2*(1 - 1/x^2)^(1/2))

ans =
2/(x^3*(1 - 1/x^2)^(1/2)) + 1/(x^5*(1 - 1/x^2)^(3/2))
```

Find the indefinite integral of the inverse cosecant function:

```
int(acsc(x), x)

ans =
x*asin(1/x) + log(x + (x^2 - 1)^(1/2))*sign(x)
```

Find the Taylor series expansion of `acsc(x)` around  $x = \text{Inf}$ :

```
taylor(acsc(x), x, Inf)
```

```
ans =  
1/x + 1/(6*x^3) + 3/(40*x^5)
```

Rewrite the inverse cosecant function in terms of the natural logarithm:

```
rewrite(acsc(x), 'log')
```

```
ans =  
-log(1i/x + (1 - 1/x^2)^(1/2))*1i
```

## Input Arguments

### X — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Version History

**Introduced before R2006a**

### See Also

acos | asin | atan | asec | acot | sin | cos | tan | csc | sec | cot

## acsch

Symbolic inverse hyperbolic cosecant function

### Syntax

`acsch(X)`

### Description

`acsch(X)` returns the inverse hyperbolic cosecant function of  $X$ .

### Examples

#### Inverse Hyperbolic Cosecant Function for Numeric and Symbolic Arguments

Depending on its arguments, `acsch` returns floating-point or exact symbolic results.

Compute the inverse hyperbolic cosecant function for these numbers. Because these numbers are not symbolic objects, `acsch` returns floating-point results.

```
A = acsch([-2*i, 0, 2*i/sqrt(3), 1/2, i, 3])
```

```
A =
    0.0000 + 0.5236i      Inf + 0.0000i      0.0000 - 1.0472i...
    1.4436 + 0.0000i      0.0000 - 1.5708i      0.3275 + 0.0000i
```

Compute the inverse hyperbolic cosecant function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `acsch` returns unresolved symbolic calls.

```
symA = acsch(sym([-2*i, 0, 2*i/sqrt(3), 1/2, i, 3]))
```

```
symA =
[ (pi*1i)/6, Inf, -(pi*1i)/3, asinh(2), -(pi*1i)/2, asinh(1/3)]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

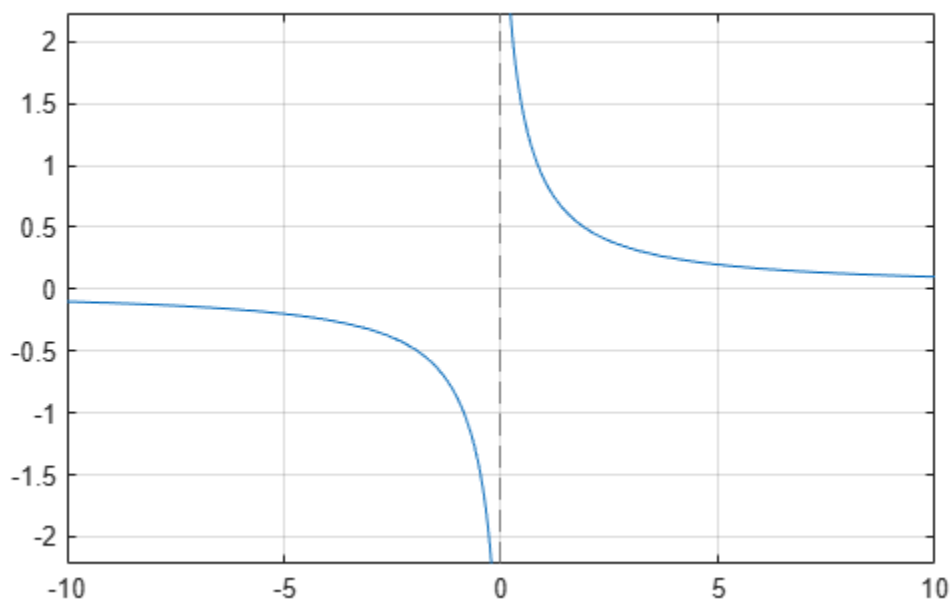
```
vpa(symA)

ans =
[ 0.52359877559829887307710723054658i,...
 Inf,...
 -1.0471975511965977461542144610932i,...
 1.4436354751788103424932767402731,...
 -1.5707963267948966192313216916398i,...
 0.32745015023725844332253525998826]
```

#### Plot Inverse Hyperbolic Cosecant Function

Plot the inverse hyperbolic cosecant function on the interval from -10 to 10.

```
syms x
fplot(acsch(x),[-10 10])
grid on
```



### Handle Expressions Containing Inverse Hyperbolic Cosecant Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `acsch`.

Find the first and second derivatives of the inverse hyperbolic cosecant function:

```
syms x
diff(acsch(x), x)
diff(acsch(x), x, x)

ans =
-1/(x^2*(1/x^2 + 1)^(1/2))

ans =
2/(x^3*(1/x^2 + 1)^(1/2)) - 1/(x^5*(1/x^2 + 1)^(3/2))
```

Find the indefinite integral of the inverse hyperbolic cosecant function:

```
int(acsch(x), x)

ans =
x*asinh(1/x) + asinh(x)*sign(x)
```

Find the Taylor series expansion of `acsch(x)` around `x = Inf`:

```
taylor(acsch(x), x, Inf)

ans =
1/x - 1/(6*x^3) + 3/(40*x^5)
```

Rewrite the inverse hyperbolic cosecant function in terms of the natural logarithm:

```
rewrite(acsch(x), 'log')  
  
ans =  
log((1/x^2 + 1)^(1/2) + 1/x)
```

## Input Arguments

### **X — Input**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Version History

**Introduced before R2006a**

### **See Also**

asinh | acosh | atanh | asech | acoth | sinh | cosh | tanh | csch | sech | coth

# adjoint

Classical adjoint (adjugate) of square matrix

## Syntax

```
X = adjoint(A)
```

## Description

`X = adjoint(A)` returns the “Classical Adjoint (Adjugate) Matrix” on page 6-24 X of A, such that  $A*X = \det(A)*\text{eye}(n) = X*A$ , where n is the number of rows in A.

## Examples

### Classical Adjoint (Adjugate) of Matrix

Find the classical adjoint of a numeric matrix.

```
A = magic(3);
X = adjoint(A)
```

*X = 3×3*

```
-53.0000    52.0000   -23.0000
 22.0000    -8.0000   -38.0000
  7.0000   -68.0000    37.0000
```

Find the classical adjoint of a symbolic matrix.

```
syms x y z
A = sym([x y z; 2 1 0; 1 0 2]);
X = adjoint(A)
```

*X =*

$$\begin{pmatrix} 2 & -2y & -z \\ -4 & 2x-z & 2z \\ -1 & y & x-2y \end{pmatrix}$$

Verify that  $\det(A)*\text{eye}(3) = X*A$  by using `isAlways`.

```
cond = det(A)*eye(3) == X*A;
isAlways(cond)
```

*ans = 3×3 logical array*

```
1 1 1
1 1 1
1 1 1
```

## Compute Inverse Using Classical Adjoint and Determinant

Compute the inverse of this matrix by computing its classical adjoint and determinant.

```
syms a b c d
A = [a b; c d];
invA = adjoint(A)/det(A)
```

```
invA =

$$\begin{pmatrix} \frac{d}{ad-bc} & -\frac{b}{ad-bc} \\ -\frac{c}{ad-bc} & \frac{a}{ad-bc} \end{pmatrix}$$

```

Verify that invA is the inverse of A.

```
isAlways(invA == inv(A))
ans = 2x2 logical array
    1     1
    1     1
```

## Input Arguments

### A — Square matrix

numeric matrix | matrix of symbolic scalar variables | symbolic function | symbolic matrix variable | symbolic matrix function | symbolic expression

Square matrix, specified as a numeric matrix, matrix of symbolic scalar variables, symbolic matrix variable, symbolic function, symbolic matrix function, or symbolic expression.

Data Types: single | double | sym | symfun | symmatrix | symfunmatrix

## More About

### Classical Adjoint (Adjugate) Matrix

The classical adjoint, or adjugate, of a square matrix  $A$  is the square matrix  $X$ , such that the  $(i,j)$ -th entry of  $X$  is the  $(j,i)$ -th cofactor of  $A$ .

The  $(j,i)$ -th cofactor of  $A$  is defined as follows.

$$a_{ji}' = (-1)^{i+j} \det(A_{ij})$$

$A_{ij}$  is the submatrix of  $A$  obtained from  $A$  by removing the  $i$ -th row and  $j$ -th column.

The classical adjoint matrix should not be confused with the adjoint matrix. The adjoint is the conjugate transpose of a matrix while the classical adjoint is another name for the adjugate matrix or cofactor transpose of a matrix.



## Version History

Introduced in R2013a

### **R2022a: Compute classical adjoint of symbolic matrix functions**

The `adjoint` function accepts an input argument of type `symfunmatrix`.

### **R2021a: Compute classical adjoint of symbolic matrix variables**

The `adjoint` function accepts an input argument of type `symmatrix`.

### **R2018b: Compute classical adjoint of numeric matrices**

The `adjoint` function accepts a numeric matrix as an input argument.

The `adjoint` function supports numeric matrices of type `double` and `single`, as well as symbolic matrices of type `sym` and `symfun`.

## See Also

`ctranspose` | `det` | `inv` | `rank`

## airy

Airy function

### Syntax

```
airy(x)
airy(0,x)
airy(1,x)
airy(2,x)
airy(3,x)

airy(n,x)

airy( __ ,1)
```

### Description

`airy(x)` returns the Airy function on page 6-31 of the first kind,  $Ai(x)$ , for each element of  $x$ .

`airy(0,x)` is the same as `airy(x)`.

`airy(1,x)` returns the derivative of  $Ai(x)$ .

`airy(2,x)` returns the Airy function on page 6-31 of the second kind,  $Bi(x)$ .

`airy(3,x)` returns the derivative of  $Bi(x)$ .

`airy(n,x)` uses the values in vector  $n$  to return the corresponding Airy functions of elements of vector  $x$ . Both  $n$  and  $x$  must have the same size.

`airy( __ ,1)` returns the “Scaled Airy Functions” on page 6-31 following the syntax for the MATLAB `airy` function.

### Examples

#### Find the Airy Function of the First Kind

Find the Airy function of the first kind,  $Ai(x)$ , for numeric or symbolic inputs using `airy`. Approximate exact symbolic outputs using `vpa`.

Find the Airy function of the first kind,  $Ai(x)$ , at 1.5. Because the input is double and not symbolic, you get a double result.

```
airy(1.5)

ans =
    0.0717
```

Find the Airy function of the values of vector  $v$  symbolically, by converting  $v$  to symbolic form using `sym`. Because the input is symbolic, `airy` returns exact symbolic results. The exact symbolic results for most symbolic inputs are unresolved function calls.

```
v = sym([-1 0 25.1 1+1i]);
vAiry = airy(v)

vAiry =
[ airy(0, -1), 3^(1/3)/(3*gamma(2/3)), airy(0, 251/10), airy(0, 1 + 1i)]
```

Numerically approximate the exact symbolic result using `vpa`.

```
vpa(vAiry)

ans =
[ 0.53556088329235211879951656563887, 0.35502805388781723926006318600418,...
 4.9152763177499054787371976959487e-38,...
 0.060458308371838149196532978116646 - 0.15188956587718140235494791259223i]
```

Find the Airy function,  $Ai(x)$ , of the symbolic input  $x^2$ . For symbolic expressions, `airy` returns an unresolved call.

```
syms x
airy(x^2)

ans =
airy(0, x^2)
```

### Find the Airy Function of the Second Kind

Find the Airy function of the second kind,  $Bi(x)$ , of the symbolic input  $[-3 \ 4 \ 1+1i \ x^2]$  by specifying the first argument as 2. Because the input is symbolic, `airy` returns exact symbolic results. The exact symbolic results for most symbolic inputs are unresolved function calls.

```
v = sym([-3 4 1+1i x^2]);
vAiry = airy(2, v)

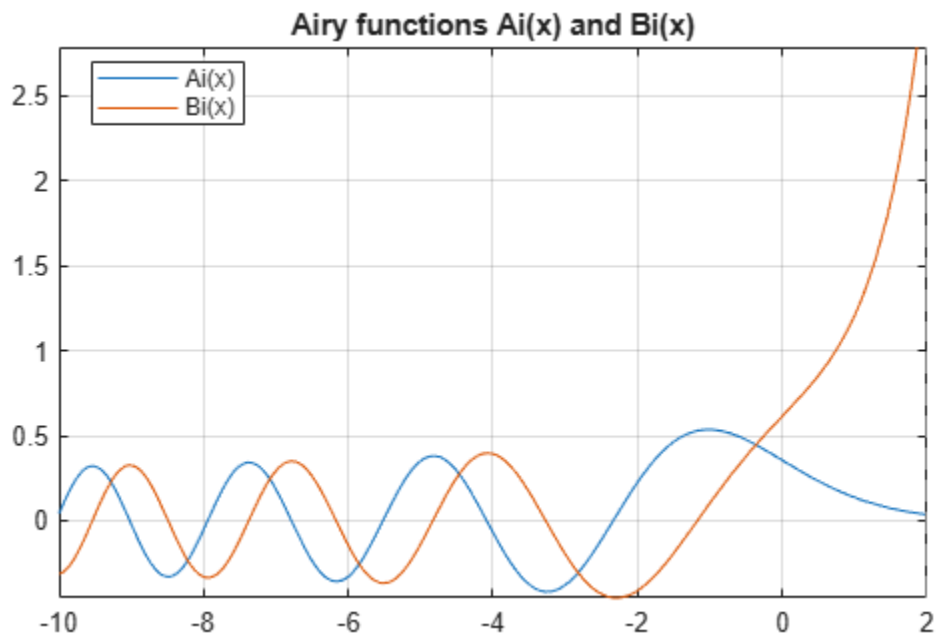
vAiry =
[ airy(2, -3), airy(2, 4), airy(2, 1 + 1i), airy(2, x^2)]
```

Use the syntax `airy(2,x)` like `airy(x)`, as described in the example “Find the Airy Function of the First Kind” on page 6-26.

### Plot Airy Functions

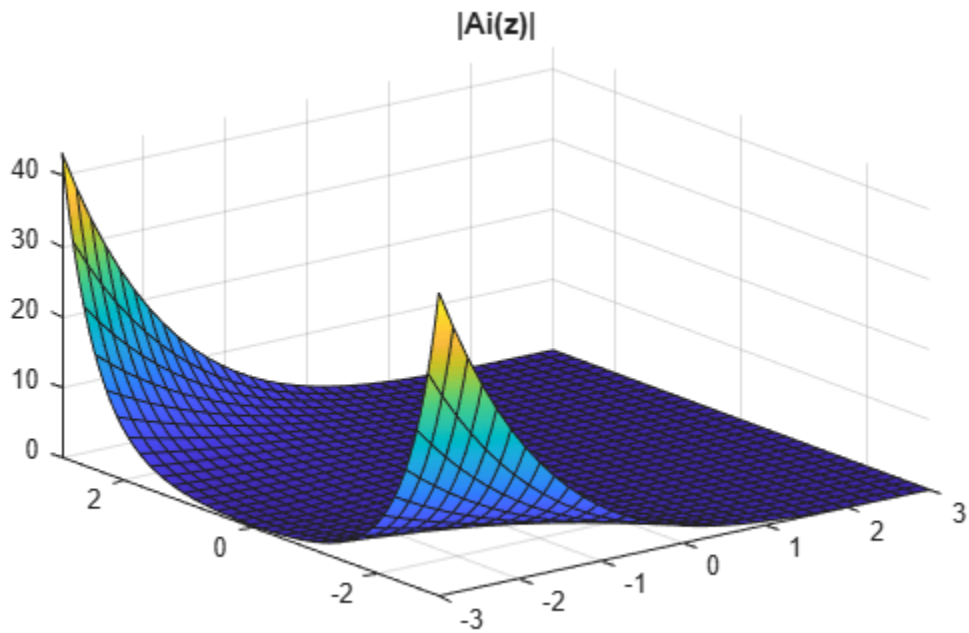
Plot the Airy Functions,  $Ai(x)$  and  $Bi(x)$ , over the interval  $[-10 \ 2]$  using `fplot`.

```
syms x
fplot(airy(x), [-10 2])
hold on
fplot(airy(2,x), [-10 2])
legend('Ai(x)', 'Bi(x)', 'Location', 'Best')
title('Airy functions Ai(x) and Bi(x)')
grid on
hold off
```



Plot the absolute value of  $Ai(z)$  over the complex plane.

```
syms y
z = x + 1i*y;
fsurf(abs(airy(z)),-3 3 -3 3)
title('|Ai(z)|')
```



### Find Derivatives of Airy Functions

Find the derivative of the Airy function of the first kind,  $Ai'(x)$ , at 0 by specifying the first argument of `airy` as 1. Then, numerically approximate the derivative using `vpa`.

```
dAi = airy(1, sym(0))
dAi_vpa = vpa(dAi)

dAi =
- (3^(1/6)*gamma(2/3))/(2*pi)
dAi_vpa =
-0.2588194037928067984051835601892
```

Find the derivative of the Airy function of the second kind,  $Bi'(x)$ , at  $x$  by specifying the first argument as 3. Then, find the derivative at  $x = 5$  by substituting for  $x$  using `subs` and calling `vpa`.

```
syms x
dBi = airy(3, x)
dBi_vpa = vpa(subs(dBi, x, 5))

dBi =
airy(3, x)
dBi_vpa =
1435.8190802179825186717212380046
```

### Solve Airy Differential Equation for Airy Functions

Show that the Airy functions  $Ai(x)$  and  $Bi(x)$  are the solutions of the differential equation

$$\frac{\partial^2 y}{\partial x^2} - xy = 0.$$

```
syms y(x)
dsolve(diff(y, 2) - x*y == 0)

ans =
C1*airy(0, x) + C2*airy(2, x)
```

### Differentiate Airy Functions

Differentiate expressions containing `airy`.

```
syms x y
diff(airy(x^2))
diff(diff(airy(3, x^2 + x*y - y^2), x), y)

ans =
2*x*airy(1, x^2)

ans =
airy(2, x^2 + x*y - y^2)*(x^2 + x*y - y^2) +...
airy(2, x^2 + x*y - y^2)*(x - 2*y)*(2*x + y) +...
airy(3, x^2 + x*y - y^2)*(x - 2*y)*(2*x + y)*(x^2 + x*y - y^2)
```

### Expand Airy Function using Taylor Series

Find the Taylor series expansion of the Airy functions,  $Ai(x)$  and  $Bi(x)$ , using `taylor`.

```

aiTaylor = taylor(airy(x))
biTaylor = taylor(airy(2, x))

aiTaylor =
- (3^(1/6)*gamma(2/3)*x^4)/(24*pi) + (3^(1/3)*x^3)/(18*gamma(2/3))...
- (3^(1/6)*gamma(2/3)*x)/(2*pi) + 3^(1/3)/(3*gamma(2/3))
biTaylor =
(3^(2/3)*gamma(2/3)*x^4)/(24*pi) + (3^(5/6)*x^3)/(18*gamma(2/3))...
+ (3^(2/3)*gamma(2/3)*x)/(2*pi) + 3^(5/6)/(3*gamma(2/3))

```

### Fourier Transform of Airy Function

Find the Fourier transform of the Airy function  $Ai(x)$  using `fourier`.

```

syms x
aiFourier = fourier(airy(x))

aiFourier =
exp((w^3*1i)/3)

```

### Numeric Roots of Airy Function

Find a root of the Airy function  $Ai(x)$  numerically using `vpasolve`.

```

syms x
vpasolve(airy(x) == 0, x)

ans =
-226.99630507523600716771890962744

```

Find a root in the interval  $[-5 \ -3]$ .

```

vpasolve(airy(x) == 0, x, [-5 -3])

ans =
-4.0879494441309706166369887014574

```

## Input Arguments

### **x** — Input

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **n** — Type of Airy function

0 (default) | number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array

Type of Airy function, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, or multidimensional array. The values of the input must be 0, 1, 2, or 3, which specify the Airy function as follows.

| <b>n</b>    | <b>Returns</b>                                                       |
|-------------|----------------------------------------------------------------------|
| 0 (default) | Airy function, $Ai(x)$ , which is the same as <code>airy(x)</code> . |

| n | Returns                                                 |
|---|---------------------------------------------------------|
| 1 | Derivative of Airy function, Ai'(x).                    |
| 2 | Airy function of the second kind, Bi(x).                |
| 3 | Derivative of Airy function of the second kind, Bi'(x). |

## More About

### Airy Functions

The Airy functions Ai(x) and Bi(x) are the two linearly independent solutions of the differential equation

$$\frac{\partial^2 y}{\partial x^2} - xy = 0.$$

Ai(x) is called the Airy function of the first kind. Bi(x) is called the Airy function of the second kind.

### Scaled Airy Functions

The Airy function of the first kind, Ai(x), is scaled as

$$e^{\left(\frac{2}{3}x^{3/2}\right)}\text{Ai}(x).$$

The derivative, Ai'(x), is scaled by the same factor.

The Airy function of the second kind, Bi(x), is scaled as

$$e^{-\left|\frac{2}{3}\text{Re}\left(x^{3/2}\right)\right|}\text{Bi}(x).$$

The derivative, Bi'(x), is scaled by the same factor.

## Tips

- When you call `airy` for inputs that are not symbolic objects, you call the MATLAB `airy` function.
- When you call `airy(n, x)`, at least one argument must be a scalar or both arguments must be vectors or matrices of the same size. If one argument is a scalar and the other is a vector or matrix, `airy(n, x)` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to the scalar.
- `airy` returns special exact values at 0.

## Version History

Introduced in R2012a

## See Also

`besseli` | `besselj` | `besselk` | `bessely`

## all

Test whether all equations and inequalities represented as elements of symbolic array are valid

### Syntax

```
all(A)
all(A,dim)
```

### Description

`all(A)` tests whether all elements of `A` return logical 1 (true). If `A` is a matrix, `all` tests all elements of each column. If `A` is a multidimensional array, `all` tests all elements along one dimension.

`all(A,dim)` tests along the dimension of `A` specified by `dim`.

### Examples

#### Test All Elements of Symbolic Vector

Create vector `V` that contains the symbolic equation and inequalities as its elements:

```
syms x
V = [x ~= x + 1, abs(x) >= 0, x == x];
```

Use `all` to test whether all of them are valid for all values of `x`:

```
all(V)

ans =
    logical
     1
```

#### Test All Elements of Symbolic Matrix

Create this matrix of symbolic equations and inequalities:

```
syms x
M = [x == x, x == abs(x); abs(x) >= 0, x ~= 2*x]

M =
[      x == x, x == abs(x)]
[ 0 <= abs(x), x ~= 2*x]
```

Use `all` to test equations and inequalities of this matrix. By default, `all` tests whether all elements of each column are valid for all possible values of variables. If all equations and inequalities in the column are valid (return logical 1), then `all` returns logical 1 for that column. Otherwise, it returns logical 0 for the column. Thus, it returns 1 for the first column and 0 for the second column:

```
all(M)

ans =
    1x2 logical array
     1     0
```



## Specify Dimension to Test Along

Create this matrix of symbolic equations and inequalities:

```
syms x
M = [x == x, x == abs(x); abs(x) >= 0, x ~= 2*x]

M =
[      x == x, x == abs(x)]
[ 0 <= abs(x),   x ~= 2*x]
```

For matrices and multidimensional arrays, `all` can test all elements along the specified dimension. To specify the dimension, use the second argument of `all`. For example, to test all elements of each column of a matrix, use the value 1 as the second argument:

```
all(M, 1)

ans =
  1x2 logical array
   1   0
```

To test all elements of each row, use the value 2 as the second argument:

```
all(M, 2)

ans =
  2x1 logical array
   0
   1
```

## Test Arrays with Numeric Values

Test whether all elements of this vector return logical 1s. Note that `all` also converts all numeric values outside equations and inequalities to logical 1s and 0s. The numeric value 0 becomes logical 0:

```
syms x
all([0, x == x])

ans =
  logical
   0
```

All nonzero numeric values, including negative and complex values, become logical 1s:

```
all([1, 2, -3, 4 + i, x == x])

ans =
  logical
   1
```

## Input Arguments

### A — Input

symbolic array

Input, specified as a symbolic array. For example, it can be an array of symbolic equations, inequalities, or logical expressions with symbolic subexpressions.

**dim — Dimension**

first non-singleton dimension (default) | integer

Dimension, specified as an integer. For example, if **A** is a matrix, **any(A,1)** tests elements of each column and returns a row vector of logical 1s and 0s. **any(A,2)** tests elements of each row and returns a column vector of logical 1s and 0s.

**Tips**

- If **A** is an empty symbolic array, **all(A)** returns logical 1.
- If some elements of **A** are just numeric values (not equations or inequalities), **all** converts these values as follows. All numeric values except 0 become logical 1. The value 0 becomes logical 0.
- If **A** is a vector and all its elements return logical 1, **all(A)** returns logical 1. If one or more elements are zero, **all(A)** returns logical 0.
- If **A** is a multidimensional array, **all(A)** treats the values along the first dimension that is not equal to 1 (nonsingleton dimension) as vectors, returning logical 1 or 0 for each vector.

**Version History**

Introduced in R2012a

**See Also**

**and** | **any** | **isAlways** | **not** | **or** | **xor**

# and

Logical AND for symbolic expressions

## Syntax

$A \ \& \ B$   
`and(A,B)`

## Description

$A \ \& \ B$  represents the logical AND.  $A \ \& \ B$  is true only when both  $A$  and  $B$  are true.

`and(A,B)` is equivalent to  $A \ \& \ B$ .

## Examples

### Construct and Set Assumptions Using AND

Combine symbolic inequalities into one condition by using `&`.

```
syms x y
cond = x>=0 & y>=0;
```

Set the assumptions represented by the condition using `assume`.

```
assume(cond)
```

Verify that the assumptions are set.

```
assumptions

ans =
[ 0 <= x, 0 <= y]
```

### Evaluate Inequalities or Conditions

Define a range for a variable by combining two inequalities into a logical condition using `&`.

```
syms x
range = 0 < x & x < 1;
```

Return the condition at  $1/2$  and  $10$  by substituting for  $x$  using `subs`. The `subs` function does not evaluate the conditions automatically.

```
x1 = subs(range,x,1/2)
x2 = subs(range,x,10)

x1 =
0 < 1/2 & 1/2 < 1
```

```
x2 =  
0 < 10 & 10 < 1
```

Evaluate the inequalities to logical 1 or 0 by using `isAlways`.

```
isAlways(x1)  
isAlways(x2)
```

```
ans =  
    logical  
     1  
ans =  
    logical  
     0
```

## Input Arguments

### A, B — Operands

symbolic equations | symbolic inequalities | symbolic expressions | symbolic arrays

Operands, specified as symbolic equations, inequalities, expressions, or arrays. Inputs A and B must either be the same size or have sizes that are compatible (for example, A is an M-by-N matrix and B is a scalar or 1-by-N row vector). For more information, see “Compatible Array Sizes for Basic Operations”.

## Tips

- If you call `simplify` for a logical expression containing symbolic subexpressions, you can get the symbolic constants `symtrue` and `symfalse`. These two constants are not the same as logical 1 (`true`) and logical 0 (`false`). To convert symbolic `symtrue` and `symfalse` to logical values, use `logical`.

## Version History

**Introduced in R2012a**

### R2016b: Implicit expansion change affects arguments for operators

*Behavior changed in R2016b*

Starting in R2016b with the addition of implicit expansion, some combinations of arguments for basic operations that previously returned errors now produce results. For example, you previously could not add a row and a column vector, but those operands are now valid for addition. In other words, an expression like `[1 2] + [1; 2]` previously returned a size mismatch error, but now it executes.

If your code uses element-wise operators and relies on the errors that MATLAB previously returned for mismatched sizes, particularly within a `try/catch` block, then your code might no longer catch those errors.

For more information on the required input sizes for basic array operations, see “Compatible Array Sizes for Basic Operations”.

## **See Also**

all | any | isAlways | not | or | piecewise | xor

## angle

Symbolic polar angle

### Syntax

`angle(Z)`

### Description

`angle(Z)` computes the polar angle of the complex value  $Z$ .

### Examples

#### Compute Polar Angle of Numeric Inputs

Compute the polar angles of these complex numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
[angle(1 + i), angle(4 + pi*i), angle(Inf + Inf*i)]
```

```
ans =  
    0.7854    0.6658    0.7854
```

#### Compute Polar Angle of Symbolic Inputs

Compute the polar angles of these complex numbers which are converted to symbolic objects:

```
[angle(sym(1) + i), angle(sym(4) + sym(pi)*i), angle(Inf + sym(Inf)*i)]
```

```
ans =  
[ pi/4, atan(pi/4), pi/4]
```

#### Compute Polar Angle of Symbolic Expressions

Compute the limits of these symbolic expressions:

```
syms x  
limit(angle(x + x^2*i/(1 + x)), x, -Inf)  
limit(angle(x + x^2*i/(1 + x)), x, Inf)
```

```
ans =  
-(3*pi)/4
```

```
ans =  
pi/4
```

#### Compute Polar Angle of Array

Compute the polar angles of the elements of matrix  $Z$ :

```
Z = sym([sqrt(3) + 3*i, 3 + sqrt(3)*i; 1 + i, i]);  
angle(Z)
```

```
ans =  
[ pi/3, pi/6]  
[ pi/4, pi/2]
```

## Input Arguments

### Z — Input

number | vector | matrix | array | symbolic number | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, array, or a symbolic number, variable, expression, function.

## Tips

- Calling `angle` for numbers (or vectors or matrices of numbers) that are not symbolic objects invokes the MATLAB `angle` function.
- If  $Z = 0$ , then `angle(Z)` returns  $0$ .

## Alternatives

For real  $X$  and  $Y$  such that  $Z = X + Y*i$ , the call `angle(Z)` is equivalent to `atan2(Y,X)`.

## Version History

Introduced in R2013a

### See Also

`atan2` | `conj` | `imag` | `real` | `sign` | `signIm`

## animationToFrame

Return structure of frames from animation objects

### Syntax

```
frames = animationToFrame
frames = animationToFrame(fig)
frames = animationToFrame( ____,Name,Value)
```

### Description

`frames = animationToFrame` returns a structure array of frames from animation objects. The animation objects must be created using the `fanimator` function.

`frames = animationToFrame(fig)` returns a structure array of frames from animation objects in the figure `fig`.

`frames = animationToFrame( ____,Name,Value)` uses the specified `Name,Value` pair arguments. Use this option with any of the input argument combinations in the previous syntaxes.

### Examples

#### Return Animation Frames

Create an animation of a moving circle, and return specific frames of the animation.

First, create two symbolic variables, `t` and `x`. The variable `t` defines the time parameter of the animation. Use `t` to set the center of the circle at  $(t, 1)$  and `x` to parameterize the perimeter of the circle within the range  $[-\pi, \pi]$ . Create the circle animation object using `fanimator`. Set the `x`-axis and `y`-axis to be equal length.

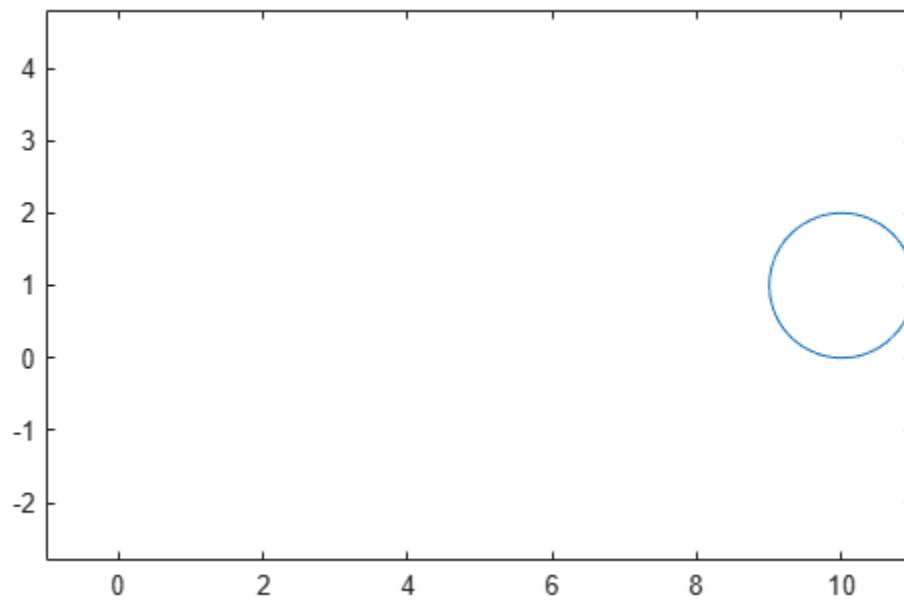
```
syms t x
fanimator(@fplot,cos(x)+t,sin(x)+1,[-pi pi])
axis equal
```

By default, `fanimator` generates an animation object with 10 frames per unit time within the range of `t` from 0 to 10. The default animation object contains a total of 101 frames. Use the command `playAnimation` to play the animation.

Next, return a structure array of frames from the animation object by using `animationToFrame`.

```
frames = animationToFrame
```



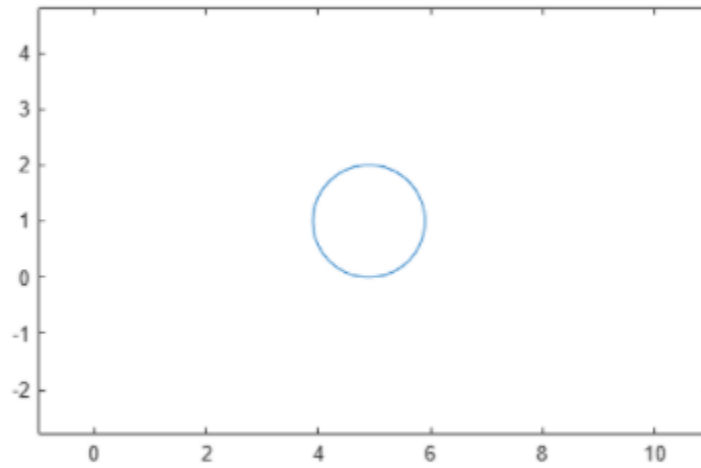


```
frames=1x101 struct array with fields:  
    cdata  
    colormap
```

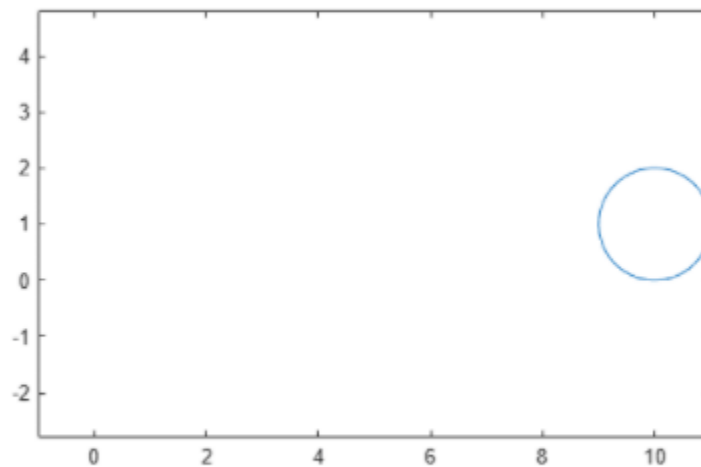
The structure `frames` contains two fields. The `cdata` field stores the image data as an array of `uint8` values.

Reconstruct the animation frames by using the `imshow` function. For example, display the 50th frame and the last frame of the animation.

```
imshow(frames(50).cdata,Interpolation="bilinear")
```



```
imshow(frames(101).cdata,Interpolation="bilinear")
```



### **Return Animation Frames in Reverse Order**

Create a moving circle animation object and a timer animation object. Return the generated animation frames in reverse order.

First, create two symbolic variables, `t` and `x`. The variable `t` defines the time parameter of the animation. Create a figure window for the animation.

```
syms t x
fig1 = figure;
```

Create a circle animation object using `f animator`. Use `t` to set the center of the circle at  $(t, 1)$  and `x` to parameterize the perimeter of the circle within the range  $[-\pi, \pi]$ . Set the x-axis and y-axis to be equal length.

```
f animator(@fplot,cos(x)+t,sin(x)+1,[-pi pi])
axis equal
```

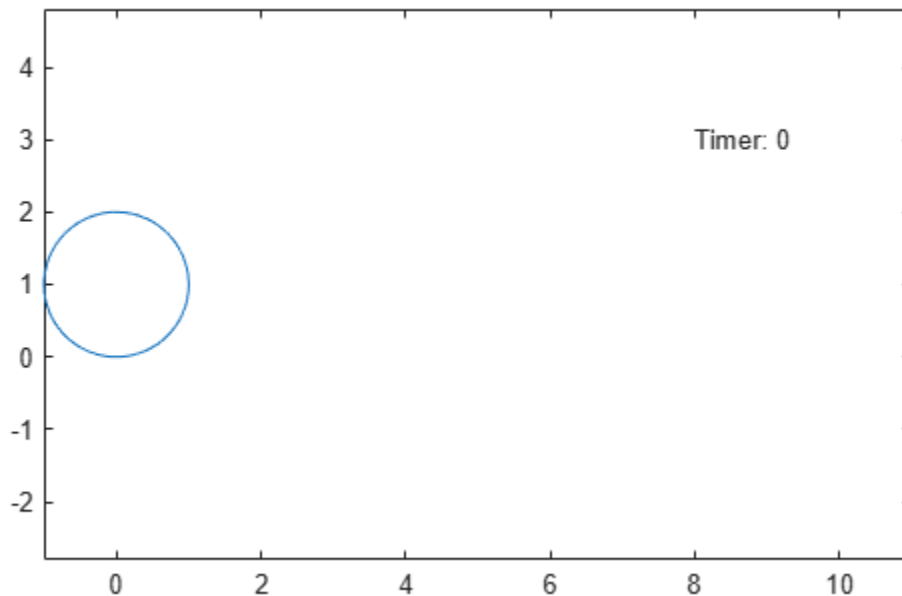
Next, use the `text` function to add a piece of text to count the elapsed time. Use `num2str` to convert the time parameter to a string.

```
hold on
f animator(@(t) text(8,3,"Timer: "+num2str(t,2)))
hold off
```

By default, `f animator` creates stop-motion frames with 10 frames per unit time within the range of `t` from 0 to 10. The default animation object contains a total of 101 frames. Use the command `playAnimation` to play the animation.

Next, return a structure array of frames from the animation in figure `fig` by using `animationToFrame`. Return the animation frames in reverse order by setting the `Backwards` argument to `true`. Set the frame rate per unit time to 2 to return a total of 21 frames.

```
frames = animationToFrame(fig1,Backwards=true,FrameRate=2)
```

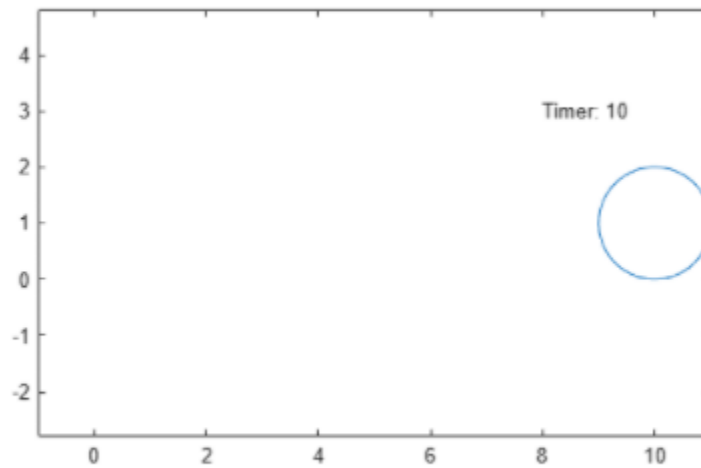


```
frames=1x21 struct array with fields:
    cdata
    colormap
```

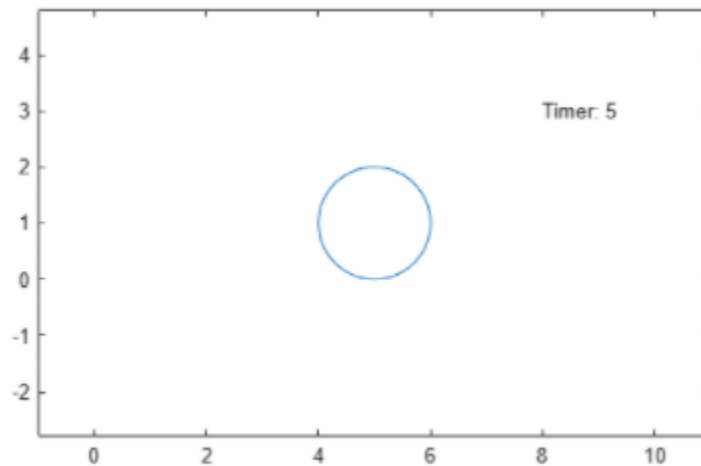
The structure `frames` contains two fields. The `cdata` field stores the image data as an array of `uint8` values.

Reconstruct the animation frames by using the `imshow` function. For example, display the first frame and the 11th frame of the animation in a new figure window.

```
fig2 = figure;  
imshow(frames(1).cdata,Interpolation="bilinear")
```



```
imshow(frames(11).cdata,Interpolation="bilinear")
```



## Input Arguments

### **fig** — Target figure

Figure object

Target figure, specified as a Figure object. For more information about Figure objects, see `figure`.

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `'Backwards',true,'AnimationRange',[-2 5]`

### **AnimationRange** — Range of animation time parameter

`[0 10]` (default) | two-element row vector

Range of the animation time parameter, specified as a two-element row vector. The two elements must be real values that are increasing.

Example: `[-2 4.5]`

### **FrameRate** — Frame rate

`10` (default) | positive value

Frame rate, specified as a positive value. The frame rate defines the number of frames per unit time when you returning animation frames as a structure array.

Example: 20

**Backwards — Backward option**

logical 0 (`false`) (default) | logical value

Backward option, specified as a logical value (boolean). If you specify `true`, then the function returns the animation frames backward or in reverse order.

Example: `true`

## Output Arguments

**frames — Animation frames**

structure array

Animation frames, returned as a structure array with two fields:

- `cdata` — The image data stored as an array of `uint8` values. The size of the image data array depends on your screen resolution.
- `colormap` — The colormap. On true color systems, this field is empty.

The `animationToFrame` function returns a structure of animation frames in the same format as the output returned by the `getframe` function.

## Version History

**Introduced in R2019a**

**See Also**

`playAnimation` | `f Animator` | `writeAnimation` | `rewindAnimation` | `getframe`

# Animator Properties

Animator appearance and behavior

---

**Note** `UIContextMenu` property is not recommended. Use `ContextMenu` instead. For more information, see “Version History”.

---

## Description

Animator properties control the appearance and behavior of an Animator object. By changing property values, you can modify certain aspects of the Animator object. You can use dot notation to refer to a particular object and property:

```
fp = f animator(@ (x) plot(x,sin(x),'bo'))
ls = fp.Visible
fp.Visible = 'off'
```

## Properties

### Frames

#### AnimationRange — Range of animation time parameter

[0 10] (default) | two-element row vector

This property is read-only.

Range of the animation time parameter, specified as a two-element row vector. The two elements must be real values that are increasing.

#### FrameRate — Frame rate

10 (default) | positive value

This property is read-only.

Frame rate, specified as a positive value. The frame rate defines the number of frames per unit time interval of an animation object.

### Interactivity

#### Visible — State of visibility

'on' (default) | on/off logical value

State of visibility, specified as 'on' or 'off', or as numeric or logical 1 (true) or 0 (false). A value of 'on' is equivalent to true, and 'off' is equivalent to false. Thus, you can use the value of this property as a logical value. The value is stored as an on/off logical value of type `matlab.lang.OnOffSwitchState`.

- 'on' — Display the object.
- 'off' — Hide the object without deleting it. You still can access the properties of an invisible object.

**ContextMenu — Context menu**

empty GraphicsPlaceholder array (default) | ContextMenu object

Context menu, specified as a ContextMenu object. Use this property to display a context menu when you right-click the object. Create the context menu using the `uicontextmenu` function.

**Callbacks****ButtonDownFcn — Mouse-click callback**

' ' (default) | function handle | cell array | character vector

Mouse-click callback, specified as one of these values:

- Function handle.
- Cell array containing a function handle and additional arguments.
- Character vector that is a valid MATLAB command or function, which is evaluated in the base workspace (not recommended).

Use this property to execute code when you click the object. If you specify this property using a function handle, then MATLAB passes two arguments to the callback function when executing the callback:

- Clicked object — Access properties of the clicked object from within the callback function.
- Event data — Empty argument. Replace it with the tilde character (~) in the function definition to indicate that this argument is not used.

For more information on how to use function handles to define callback functions, see “Create Callbacks for Graphics Objects”.

**CreateFcn — Creation function**

" " (default) | function handle | cell array | string scalar | character vector

Object creation function, specified as one of these values:

- Function handle
- Cell array in which the first element is a function handle and subsequent elements are the arguments to pass to the callback function
- String scalar or character vector containing a valid MATLAB command or function, which is evaluated in the base workspace (not recommended)

For more information about specifying a callback as a function handle, cell array, string scalar, or character vector, see “Create Callbacks for Graphics Objects”.

This property specifies a callback function to execute when MATLAB creates the object. MATLAB initializes all property values before executing the `CreateFcn` callback. If you do not specify the `CreateFcn` property, then MATLAB executes a default creation function.

Setting the `CreateFcn` property on an existing component has no effect.

If you specify this property as a function handle or cell array, you can access the object that is being created using the first argument of the callback function. Otherwise, use the `gcbo` function to access the object.



**DeleteFcn — Deletion function**

" " (default) | function handle | cell array | string scalar | character vector

Object deletion function, specified as one of these values:

- Function handle
- Cell array in which the first element is a function handle and subsequent elements are the arguments to pass to the callback function
- String scalar or character vector containing a valid MATLAB command or function, which is evaluated in the base workspace (not recommended)

For more information about specifying a callback as a function handle, cell array, string scalar, or character vector, see “Create Callbacks for Graphics Objects”.

This property specifies a callback function to execute when MATLAB deletes the object. MATLAB executes the `DeleteFcn` callback before destroying the properties of the object. If you do not specify the `DeleteFcn` property, then MATLAB executes a default deletion function.

If you specify this property as a function handle or cell array, you can access the object that is being deleted using the first argument of the callback function. Otherwise, use the `gcbo` function to access the object.

**Callback Execution Control****Interruptible — Callback interruption**

"on" (default) | on/off logical value

Callback interruption, specified as "on" or "off", or as numeric or logical 1 (true) or 0 (false). A value of "on" is equivalent to `true`, and "off" is equivalent to `false`. Thus, you can use the value of this property as a logical value. The value is stored as an on/off logical value of type `matlab.lang.OnOffSwitchState`.

This property determines if a running callback can be interrupted. There are two callback states to consider:

- The running callback is the currently executing callback.
- The interrupting callback is a callback that tries to interrupt the running callback.

MATLAB determines callback interruption behavior whenever it executes a command that processes the callback queue. These commands include `drawnow`, `figure`, `uifigure`, `getframe`, `waitfor`, and `pause`.

If the running callback does not contain one of these commands, then no interruption occurs. MATLAB first finishes executing the running callback, and later executes the interrupting callback.

If the running callback does contain one of these commands, then the `Interruptible` property of the object that owns the running callback determines if the interruption occurs:

- If the value of `Interruptible` is set to "off", then no interruption occurs. Instead, the `BusyAction` property of the object that owns the interrupting callback determines if the interrupting callback is discarded or added to the callback queue.
- If the value of `Interruptible` is set to "on", then the interruption occurs. The next time MATLAB processes the callback queue, it stops the execution of the running callback and executes

the interrupting callback. After the interrupting callback completes, MATLAB then resumes executing the running callback.

---

**Note** Callback interruption and execution behave differently in these situations:

- If the interrupting callback is a `DeleteFcn`, `CloseRequestFcn`, or `SizeChangedFcn` callback, then the interruption occurs regardless of the `Interruptible` property value.
  - If the running callback is currently executing the `waitfor` function, then the interruption occurs regardless of the `Interruptible` property value.
  - If the interrupting callback is owned by a `Timer` object, then the callback executes according to schedule regardless of the `Interruptible` property value.
- 

---

**Note** When an interruption occurs, MATLAB does not save the state of properties or the display. For example, the object returned by the `gca` or `gcf` command might change when another callback executes.

---

### **BusyAction — Callback queuing**

"queue" (default) | "cancel"

Callback queuing, specified as "queue" or "cancel". The `BusyAction` property determines how MATLAB handles the execution of interrupting callbacks. There are two callback states to consider:

- The running callback is the currently executing callback.
- The interrupting callback is a callback that tries to interrupt the running callback.

The `BusyAction` property determines callback queuing behavior only when both of these conditions are met:

- The running callback contains a command that processes the callback queue, such as `drawnow`, `figure`, `uifigure`, `getframe`, `waitfor`, or `pause`.
- The value of the `Interruptible` property of the object that owns the running callback is set to "off".

Under these conditions, the `BusyAction` property of the object that owns the interrupting callback determines how MATLAB handles the interrupting callback. Specify the `BusyAction` property as one of these values:

- "queue" — Put the interrupting callback in a queue to be processed after the running callback finishes execution.
- "cancel" — Do not execute the interrupting callback.

### **BeingDeleted — Deletion status**

on/off logical value

This property is read-only.

Deletion status, returned as an on/off logical value of type `matlab.lang.OnOffSwitchState`.

MATLAB sets the `BeingDeleted` property to 'on' when the `DeleteFcn` callback begins execution. The `BeingDeleted` property remains set to 'on' until the component object no longer exists.

Check the value of the `BeingDeleted` property to verify that the object is not about to be deleted before querying or modifying it.

### Parent/Child

#### Parent — Parent

Axes object

Parent, specified as an Axes object.

#### Children — Children

graphics object

Children, returned as a graphics object. Use this property to view the property values of the graphics object.

You cannot add or remove children using the Children property. You can only set Children to a permutation of itself.

#### HandleVisibility — Visibility of object handle

'on' (default) | 'off' | 'callback'

Visibility of the object handle in the Children property of the parent, specified as one of these values:

- 'on' — Object handle is always visible.
- 'off' — Object handle is invisible at all times. This option is useful for preventing unintended changes to the UI by another function. Set the HandleVisibility to 'off' to temporarily hide the handle during the execution of that function.
- 'callback' — Object handle is visible from within callbacks or functions invoked by callbacks, but not from within functions invoked from the command line. This option blocks access to the object at the command line, but permits callback functions to access it.

If the object is not listed in the Children property of the parent, then functions that obtain object handles by searching the object hierarchy or querying handle properties cannot return it. Examples of such functions include the `get`, `findobj`, `gca`, `gcf`, `gco`, `newplot`, `cla`, `clf`, and `close` functions.

Hidden object handles are still valid. Set the root `ShowHiddenHandles` property to 'on' to list all object handles regardless of their HandleVisibility property setting.

### Identifier

#### Type — Type of graphics object

'animator'

This property is read-only.

Type of graphics object, returned as 'animator'. Use this property to find all objects of a given type within a plotting hierarchy. For example, you can use the `findobj` function to find graphics objects of type 'animator'.

#### Tag — Object identifier

"" (default) | string scalar | character vector

Object identifier, specified as a string scalar or character vector. You can specify a unique **Tag** value to serve as an identifier for an object. When you need access to the object elsewhere in your code, you can use the `findobj` function to search for the object based on the **Tag** value.

### **UserData — User data**

`[]` (default) | array

User data, specified as any MATLAB array. For example, you can specify a scalar, vector, matrix, cell array, character array, table, or structure. Use this property to store arbitrary data on an object.

If you are working in App Designer, create public or private properties in the app to share data instead of using the **UserData** property. For more information, see “Share Data Within a Single App Designer App”.

## **Version History**

### **Introduced in R2019a**

### **R2020a: UIContextMenu property is not recommended**

*Not recommended starting in R2020a*

Starting in R2020a, using the **UIContextMenu** property to assign a context menu to a graphics object or UI component is not recommended. Use the **ContextMenu** property instead. The property values are the same.

There are no plans to remove support for the **UIContextMenu** property at this time. However, the **UIContextMenu** property no longer appears in the list returned by calling the `get` function on a graphics object or UI component.

## any

Test whether at least one of equations and inequalities represented as elements of symbolic array is valid

### Syntax

```
any(A)
any(A,dim)
```

### Description

`any(A)` tests whether at least one element of `A` returns logical 1 (true). If `A` is a matrix, `any` tests elements of each column. If `A` is a multidimensional array, `any` tests elements along one dimension.

`any(A,dim)` tests along the dimension of `A` specified by `dim`.

### Examples

#### Test Vector of Symbolic Conditions

Create vector `V` that contains the symbolic equation and inequalities as its elements:

```
syms x real
V = [x ~= x + 1, abs(x) >= 0, x == x];
```

Use `any` to test whether at least one of them is valid for all values of `x`:

```
any(V)

ans =
    logical
     1
```

#### Test Matrix of Symbolic Conditions

Create this matrix of symbolic equations and inequalities:

```
syms x real
M = [x == 2*x, x == abs(x); abs(x) >= 0, x == 2*x]

M =
[ x == 2*x, x == abs(x) ]
[ 0 <= abs(x), x == 2*x ]
```

Use `any` to test equations and inequalities of this matrix. By default, `any` tests whether any element of each column is valid for all possible values of variables. If at least one equation or inequality in the column is valid (returns logical 1), then `any` returns logical 1 for that column. Otherwise, it returns logical 0 for the column. Thus, it returns 1 for the first column and 0 for the second column:

```
any(M)
```

```
ans =  
    1×2 logical array  
    1    0
```

### Specify Dimension to Test Along

Create this matrix of symbolic equations and inequalities:

```
syms x real  
M = [x == 2*x, x == abs(x); abs(x) >= 0, x == 2*x]  
  
M =  
[ x == 2*x, x == abs(x)]  
[ 0 <= abs(x), x == 2*x]
```

For matrices and multidimensional arrays, `any` can test elements along the specified dimension. To specify the dimension, use the second argument of `any`. For example, to test elements of each column of a matrix, use the value 1 as the second argument:

```
any(M, 1)  
  
ans =  
    1×2 logical array  
    1    0
```

To test elements of each row, use the value 2 as the second argument:

```
any(M, 2)  
  
ans =  
    2×1 logical array  
    0  
    1
```

### Test Arrays with Numeric Values

Test whether any element of this vector returns logical 1. Note that `any` also converts all numeric values outside equations and inequalities to logical 1s and 0s. The numeric value 0 becomes logical 0:

```
syms x  
any([0, x == x + 1])  
  
ans =  
    logical  
    0
```

All nonzero numeric values, including negative and complex values, become logical 1s:

```
any([-4 + i, x == x + 1])  
  
ans =  
    logical  
    1
```

## Input Arguments

### A — Input

symbolic array

Input, specified as a symbolic array. For example, it can be an array of symbolic equations, inequalities, or logical expressions with symbolic subexpressions.

**dim — Dimension**

first non-singleton dimension (default) | integer

Dimension, specified as an integer. For example, if  $A$  is a matrix, `any(A,1)` tests elements of each column and returns a row vector of logical 1s and 0s. `any(A,2)` tests elements of each row and returns a column vector of logical 1s and 0s.

**Tips**

- If  $A$  is an empty symbolic array, `any(A)` returns logical 0.
- If some elements of  $A$  are just numeric values (not equations or inequalities), `any` converts these values as follows. All nonzero numeric values become logical 1. The value 0 becomes logical 0.
- If  $A$  is a vector and any of its elements returns logical 1, `any(A)` returns logical 1. If all elements are zero, `any(A)` returns logical 0.
- If  $A$  is a multidimensional array, `any(A)` treats the values along the first dimension that is not equal to 1 (non-singleton dimension) as vectors, returning logical 1 or 0 for each vector.

**Version History**

Introduced in R2012a

**See Also**

`all` | `and` | `isAlways` | `not` | `or` | `xor`

## argnames

Input variables of symbolic function or matrix function

### Syntax

```
args = argnames(f)
```

### Description

`args = argnames(f)` returns the input variables of the symbolic function or matrix function `f`.

### Examples

#### Find Input Variables of Symbolic Function

Create a symbolic function.

```
syms f(x,y)
f(x,y) = x + y;
```

Find the input variables of `f` by using `argnames`.

```
args = argnames(f)

args = (x y)
```

Create another symbolic function.

```
syms f(a,b,x,y)
f(x,b,y,a) = a*x + b*y;
```

Find the input variables of `f`. When returning variables, `argnames` uses the same order as you used when you defined the function.

```
args = argnames(f)

args = (x b y a)
```

#### Find Input Variables of Symbolic Matrix Function

Create a symbolic matrix function.

```
syms A B 2 matrix
syms f(A,B) 2 matrix keepargs
f(A,B) = A*B - 3*A + 2*eye(2);
```

Find the input variables of `f` by using `argnames`. The result is a cell array containing symbolic matrix variables.



```
args = argnames(f)

args=1x2 cell array
      {2x2 symmatrix}      {2x2 symmatrix}
```

Access the contents of the cell array by indexing into it.

```
arg1 = args{1}
```

```
arg1 = A
```

```
arg2 = args{2}
```

```
arg2 = B
```

## Input Arguments

### **f** — Input function

symbolic function | symbolic matrix function

Input function, specified as a symbolic function or symbolic matrix function.

Data Types: `symfun` | `symfunmatrix`

## Output Arguments

### **args** — Input variables of symbolic function or matrix function

symbolic variable | vector of symbolic variables | cell array of symbolic matrix variables

Input variables of a symbolic function or matrix function.

- If `f` is a symbolic function of type `symfun`, then `args` is returned as a symbolic variable or a vector of symbolic variables of type `sym`.
- If `f` is a symbolic matrix function of type `symfunmatrix`, then `args` is returned as a cell array of symbolic matrix variables of type `symmatrix`.

Data Types: `sym` | `cell`

## Version History

Introduced in R2012a

### **R2022a: Find input variables of symbolic matrix functions**

The `argnames` function accepts an input argument of type `symfunmatrix`. For an example, see “Find Input Variables of Symbolic Matrix Function” on page 6-56.

## See Also

`formula` | `syms` | `symvar` | `symfun` | `symfunmatrix`

## asec

Symbolic inverse secant function

### Syntax

`asec(X)`

### Description

`asec(X)` returns the inverse secant function (arcsecant function) of  $X$ . All angles are in radians.

- For real elements of  $X$  in the interval  $[-\text{Inf}, -1]$  and  $[1, \text{Inf}]$ , `asec` returns values in the interval  $[0, \pi]$ .
- For real values of  $X$  in the interval  $[-1, 1]$  and for complex values of  $X$ , `asec` returns complex values with the real parts in the interval  $[0, \pi]$ .

### Examples

#### Inverse Secant Function for Numeric and Symbolic Arguments

Depending on its arguments, `asec` returns floating-point or exact symbolic results.

Compute the inverse secant function for these numbers. Because these numbers are not symbolic objects, `asec` returns floating-point results.

```
A = asec([-2, 0, 2/sqrt(3), 1/2, 1, 5])

A =
    2.0944 + 0.0000i    0.0000 +    Inf    0.5236 + 0.0000i...
    0.0000 + 1.3170i    0.0000 + 0.0000i    1.3694 + 0.0000i
```

Compute the inverse secant function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `asec` returns unresolved symbolic calls.

```
symA = asec(sym([-2, 0, 2/sqrt(3), 1/2, 1, 5]))

symA =
[ (2*pi)/3, Inf, pi/6, acos(2), 0, acos(1/5)]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

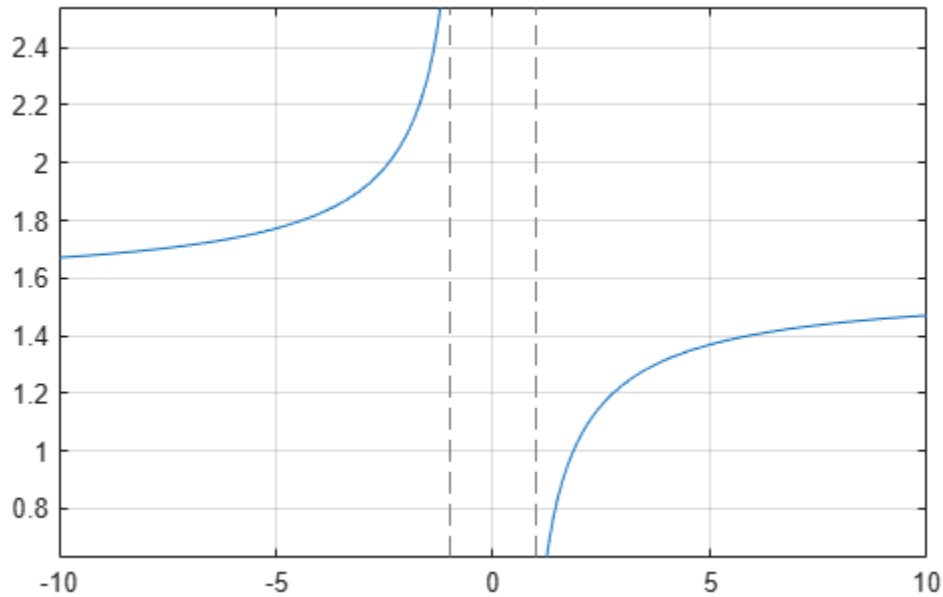
```
vpa(symA)

ans =
[ 2.0943951023931954923084289221863,...
 Inf,...
 0.52359877559829887307710723054658,...
 1.3169578969248165734029498707969i,...
 0,...
 1.3694384060045659001758622252964]
```

#### Plot Inverse Secant Function

Plot the inverse secant function on the interval from -10 to 10.

```
syms x
fplot(asec(x), [-10 10])
grid on
```



### Handle Expressions Containing Inverse Secant Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `asec`.

Find the first and second derivatives of the inverse secant function:

```
syms x
diff(asec(x), x)
diff(asec(x), x, x)

ans =
1/(x^2*(1 - 1/x^2)^(1/2))

ans =
- 2/(x^3*(1 - 1/x^2)^(1/2)) - 1/(x^5*(1 - 1/x^2)^(3/2))
```

Find the indefinite integral of the inverse secant function:

```
int(asec(x), x)

ans =
x*acos(1/x) - log(x + (x^2 - 1)^(1/2))*sign(x)
```

Find the Taylor series expansion of `asec(x)` around `x = Inf`:

```
taylor(asec(x), x, Inf)
```

```
ans =  
pi/2 - 1/x - 1/(6*x^3) - 3/(40*x^5)
```

Rewrite the inverse secant function in terms of the natural logarithm:

```
rewrite(asec(x), 'log')
```

```
ans =  
-log(1/x + (1 - 1/x^2)^(1/2)*1i)*1i
```

## Input Arguments

### **X — Input**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Version History

**Introduced before R2006a**

### **See Also**

acos | asin | atan | acsc | acot | sin | cos | tan | csc | sec | cot

# asech

Symbolic inverse hyperbolic secant function

## Syntax

`asech(X)`

## Description

`asech(X)` returns the inverse hyperbolic secant function of  $X$ .

## Examples

### Inverse Hyperbolic Secant Function for Numeric and Symbolic Arguments

Depending on its arguments, `asech` returns floating-point or exact symbolic results.

Compute the inverse hyperbolic secant function for these numbers. Because these numbers are not symbolic objects, `asech` returns floating-point results.

```
A = asech([-2, 0, 2/sqrt(3), 1/2, 1, 3])

A =
    0.0000 + 2.0944i      Inf + 0.0000i      0.0000 + 0.5236i...
    1.3170 + 0.0000i      0.0000 + 0.0000i      0.0000 + 1.2310i
```

Compute the inverse hyperbolic secant function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `asech` returns unresolved symbolic calls.

```
symA = asech(sym([-2, 0, 2/sqrt(3), 1/2, 1, 3]))

symA =
[ (pi*2i)/3, Inf, (pi*1i)/6, acosh(2), 0, acosh(1/3)]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

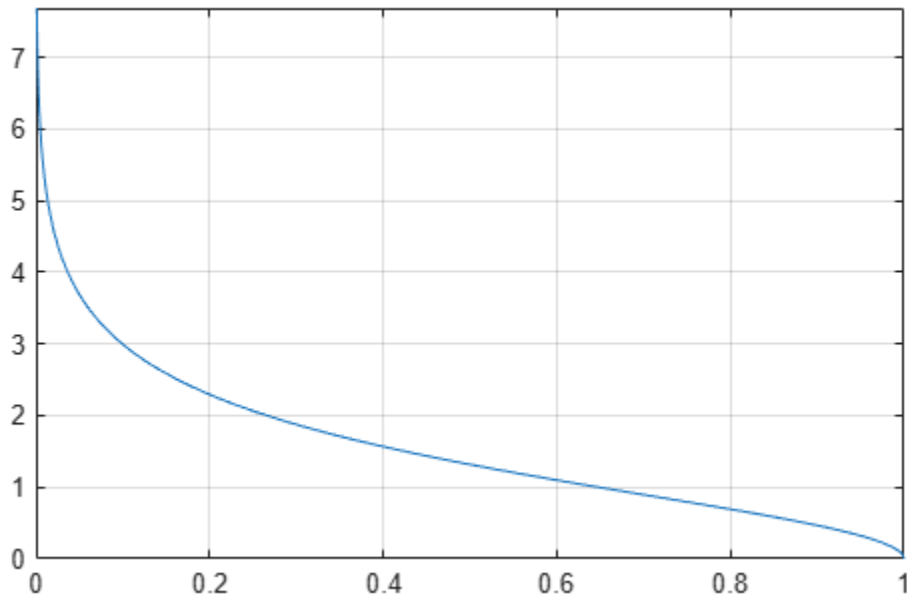
```
vpa(symA)

ans =
[ 2.0943951023931954923084289221863i,...
  Inf,...
  0.52359877559829887307710723054658i,...
  1.316957896924816708625046347308,...
  0,...
  1.230959417340774682134929178248i]
```

### Plot Inverse Hyperbolic Secant Function

Plot the inverse hyperbolic secant function on the interval from 0 to 1.

```
syms x
fplot(asech(x), [0 1])
grid on
```



### Handle Expressions Containing Inverse Hyperbolic Secant Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `asech`.

Find the first and second derivatives of the inverse hyperbolic secant function. Simplify the second derivative by using `simplify`.

```
syms x
diff(asech(x), x)
simplify(diff(asech(x), x, x))

ans =
-1/(x^2*(1/x - 1)^(1/2)*(1/x + 1)^(1/2))

ans =
-(2*x^2 - 1)/(x^5*(1/x - 1)^(3/2)*(1/x + 1)^(3/2))
```

Find the indefinite integral of the inverse hyperbolic secant function:

```
int(asech(x), x)

ans =
atan(1/((1/x - 1)^(1/2)*(1/x + 1)^(1/2))) + x*acosh(1/x)
```

Find the Taylor series expansion of `asech(x)` around  $x = \text{Inf}$ :

```
taylor(asech(x), x, Inf)
```

```
ans =
(pi*i)/2 - 1/x - 1/(6*x^3) - 3i/(40*x^5)
```

Rewrite the inverse hyperbolic secant function in terms of the natural logarithm:

```
rewrite(asech(x), 'log')
```

```
ans =
log((1/x - 1)^(1/2)*(1/x + 1)^(1/2) + 1/x)
```

## Input Arguments

### X — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Version History

Introduced before R2006a

### See Also

asinh | acosh | atanh | acsch | acoth | sinh | cosh | tanh | csch | sech | coth

## asin

Symbolic inverse sine function

### Syntax

`asin(X)`

### Description

`asin(X)` returns the inverse sine function (arcsine function) of  $X$ . All angles are in radians.

- For real values of  $X$  in the interval  $[-1, 1]$ , `asin(X)` returns the values in the interval  $[-\pi/2, \pi/2]$ .
- For real values of  $X$  outside the interval  $[-1, 1]$  and for complex values of  $X$ , `asin(X)` returns complex values with the real parts in the interval  $[-\pi/2, \pi/2]$ .

### Examples

#### Inverse Sine Function for Numeric and Symbolic Arguments

Depending on its arguments, `asin` returns floating-point or exact symbolic results.

Compute the inverse sine function for these numbers. Because these numbers are not symbolic objects, `asin` returns floating-point results.

```
A = asin([-1, -1/3, -1/2, 1/4, 1/2, sqrt(3)/2, 1])
```

```
A =  
    -1.5708    -0.3398    -0.5236     0.2527     0.5236     1.0472     1.5708
```

Compute the inverse sine function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `asin` returns unresolved symbolic calls.

```
symA = asin(sym([-1, -1/3, -1/2, 1/4, 1/2, sqrt(3)/2, 1]))
```

```
symA =  
[ -pi/2, -asin(1/3), -pi/6, asin(1/4), pi/6, pi/3, pi/2]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

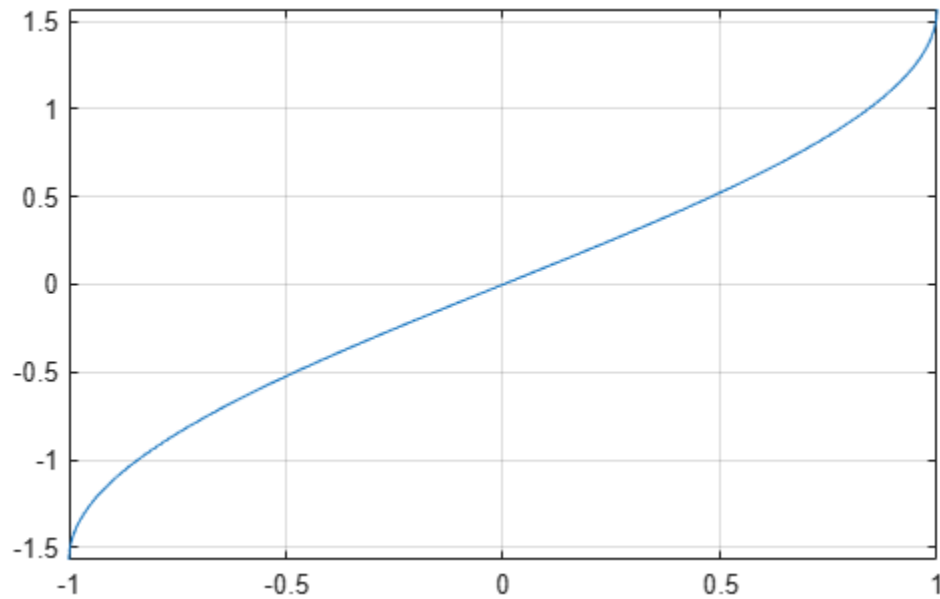
```
vpa(symA)  
  
ans =  
[ -1.5707963267948966192313216916398,...  
 -0.33983690945412193709639251339176,...  
 -0.52359877559829887307710723054658,...  
 0.25268025514207865348565743699371,...  
 0.52359877559829887307710723054658,...  
 1.0471975511965977461542144610932,...  
 1.5707963267948966192313216916398]
```

#### Plot Inverse Sine Function



Plot the inverse sine function on the interval from -1 to 1.

```
syms x
fplot(asin(x), [-1 1])
grid on
```



### Handle Expressions Containing Inverse Sine Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `asin`.

Find the first and second derivatives of the inverse sine function:

```
syms x
diff(asin(x), x)
diff(asin(x), x, x)
```

```
ans =
1/(1 - x^2)^(1/2)
```

```
ans =
x/(1 - x^2)^(3/2)
```

Find the indefinite integral of the inverse sine function:

```
int(asin(x), x)
```

```
ans =
x*asin(x) + (1 - x^2)^(1/2)
```

Find the Taylor series expansion of `asin(x)`:

```
taylor(asin(x), x)
```

```
ans =  
(3*x^5)/40 + x^3/6 + x
```

Rewrite the inverse sine function in terms of the natural logarithm:

```
rewrite(asin(x), 'log')
```

```
ans =  
-log((1 - x^2)^(1/2) + x*1i)*1i
```

## Input Arguments

### **X — Input**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Version History

**Introduced before R2006a**

### **See Also**

`acos` | `atan` | `acsc` | `asec` | `acot` | `sin` | `cos` | `tan` | `csc` | `sec` | `cot`

# asinh

Symbolic inverse hyperbolic sine function

## Syntax

`asinh(X)`

## Description

`asinh(X)` returns the inverse hyperbolic sine function of X.

## Examples

### Inverse Hyperbolic Sine Function for Numeric and Symbolic Arguments

Depending on its arguments, `asinh` returns floating-point or exact symbolic results.

Compute the inverse hyperbolic sine function for these numbers. Because these numbers are not symbolic objects, `asinh` returns floating-point results.

```
A = asinh([-i, 0, 1/6, i/2, i, 2])

A =
    0.0000 - 1.5708i    0.0000 + 0.0000i    0.1659 + 0.0000i...
    0.0000 + 0.5236i    0.0000 + 1.5708i    1.4436 + 0.0000i
```

Compute the inverse hyperbolic sine function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `asinh` returns unresolved symbolic calls.

```
symA = asinh(sym([-i, 0, 1/6, i/2, i, 2]))

symA =
[ -(pi*1i)/2, 0, asinh(1/6), (pi*1i)/6, (pi*1i)/2, asinh(2)]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

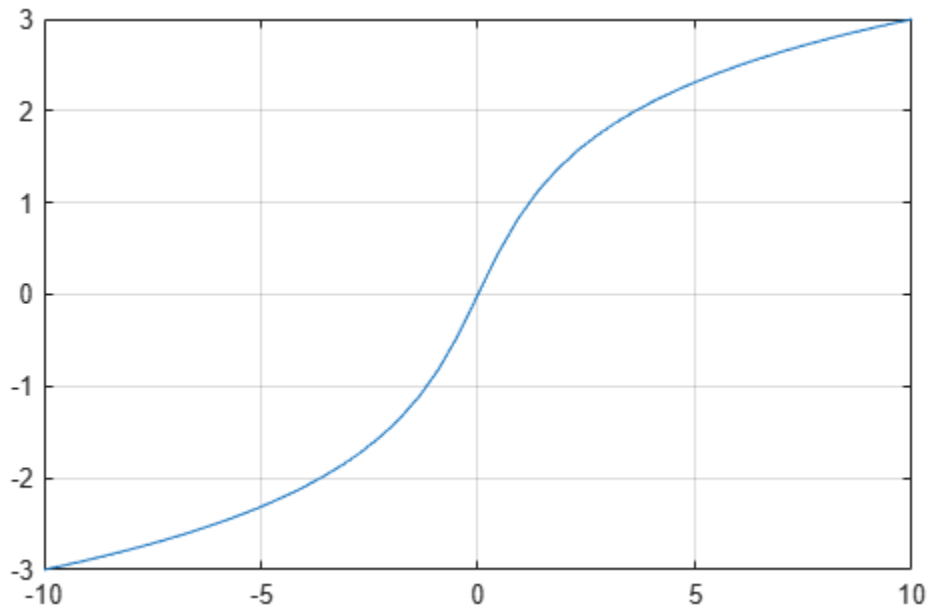
```
vpa(symA)

ans =
[ -1.5707963267948966192313216916398i,...
 0,...
 0.16590455026930117643502171631553,...
 0.52359877559829887307710723054658i,...
 1.5707963267948966192313216916398i,...
 1.4436354751788103012444253181457]
```

### Plot Inverse Hyperbolic Sine Function

Plot the inverse hyperbolic sine function on the interval from -10 to 10.

```
syms x
fplot(asinh(x),[-10 10])
grid on
```



### Handle Expressions Containing Inverse Hyperbolic Sine Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `asinh`.

Find the first and second derivatives of the inverse hyperbolic sine function:

```
syms x
diff(asinh(x), x)
diff(asinh(x), x, x)
```

```
ans =
1/(x^2 + 1)^(1/2)
```

```
ans =
-x/(x^2 + 1)^(3/2)
```

Find the indefinite integral of the inverse hyperbolic sine function:

```
int(asinh(x), x)
```

```
ans =
x*asinh(x) - (x^2 + 1)^(1/2)
```

Find the Taylor series expansion of `asinh(x)`:

```
taylor(asinh(x), x)
```

```
ans =
(3*x^5)/40 - x^3/6 + x
```

Rewrite the inverse hyperbolic sine function in terms of the natural logarithm:

```
rewrite(asinh(x), 'log')
```

```
ans =  
log(x + (x^2 + 1)^(1/2))
```

## Input Arguments

### **X — Input**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Version History

**Introduced before R2006a**

### **See Also**

acosh | atanh | acsch | asech | acoth | sinh | cosh | tanh | csch | sech | coth

## assume

Set assumption on symbolic object

### Syntax

```
assume(condition)
assume(expr, set)
assume(expr, "clear")
```

### Description

`assume(condition)` sets mathematical assumptions or conditions on symbolic variables as defined by `condition`. `assume` is not additive. Instead, it automatically deletes all previous assumptions on the variables in `condition`.

`assume(expr, set)` states that `expr` belongs to `set`. `assume` deletes previous assumptions on the variables in `expr`.

`assume(expr, "clear")` clears all assumptions on all variables in `expr`.

### Examples

#### Common Assumptions

Set an assumption using the associated syntax.

| Assumption for Symbolic Variable $x$ | Syntax                                                               |
|--------------------------------------|----------------------------------------------------------------------|
| real                                 | <code>assume(x, "real")</code>                                       |
| rational                             | <code>assume(x, "rational")</code>                                   |
| positive                             | <code>assume(x, "positive")</code>                                   |
| positive integer                     | <code>assume(x, {'positive', 'integer'})</code>                      |
| less than $-1$ or greater than $1$   | <code>assume(x &lt; -1   x &gt; 1)</code>                            |
| an integer from $2$ through $10$     | <code>assume(in(x, "integer") &amp; x &gt; 2 &amp; x &lt; 10)</code> |
| not an integer                       | <code>assume(~in(z, "integer"))</code>                               |
| not equal to $0$                     | <code>assume(x ~= 0)</code>                                          |
| even                                 | <code>assume(x/2, "integer")</code>                                  |
| odd                                  | <code>assume((x-1)/2, "integer")</code>                              |
| from $0$ through $2\pi$              | <code>assume(x &gt; 0 &amp; x &lt; 2*pi)</code>                      |
| a multiple of $\pi$                  | <code>assume(x/pi, "integer")</code>                                 |

#### Assume Variable Is Even or Odd

Assume  $x$  is even by assuming that  $x/2$  is an integer. Assume  $x$  is odd by assuming that  $(x-1)/2$  is an integer.

Assume  $x$  is even.

```
syms x
assume(x/2,"integer")
```

Find all even numbers between 0 and 10 using solve.

```
solve(x>0,x<10,x)
```

```
ans =
 2
 4
 6
 8
```

Assume  $x$  is odd. `assume` is not additive, but instead automatically deletes the previous assumption `in(x/2, "integer")`.

```
assume((x-1)/2,"integer")
solve(x>0,x<10,x)
```

```
ans =
 1
 3
 5
 7
 9
```

Clear the assumptions on  $x$  for further computations.

```
assume(x,"clear")
```

### Multiple Assumptions

Successive `assume` commands do not set multiple assumptions. Instead, each `assume` command deletes previous assumptions and sets new assumptions. Set multiple assumptions by using `assumeAlso` or the `&` operator.

Assume  $x > 5$  and then  $x < 10$  by using `assume`. Use assumptions to check that only the second assumption exists because `assume` deleted the first assumption when setting the second.

```
syms x
assume(x > 5)
assume(x < 10)
assumptions
```

```
ans =
x < 10
```

Assume the first assumption in addition to the second by using `assumeAlso`. Check that both assumptions exist.

```
assumeAlso(x > 5)
assumptions
```

```
ans =
[ 5 < x, x < 10]
```

Clear the assumptions on  $x$ .

```
assume(x, "clear")
```

Assume both conditions using the & operator. Check that both assumptions exist.

```
assume(x>5 & x<10)
assumptions
```

```
ans =
[ 5 < x, x < 10]
```

Clear the assumptions on x for future calculations.

```
assume(x, "clear")
```

### Assumptions on Integrand

Compute an indefinite integral with and without the assumption on the symbolic parameter a.

Use `assume` to set an assumption that a does not equal -1.

```
syms x a
assume(a ~= -1)
```

Compute this integral.

```
int(x^a, x)

ans =
x^(a + 1)/(a + 1)
```

Now, clear the assumption and compute the same integral. Without assumptions, `int` returns this piecewise result.

```
assume(a, "clear")
int(x^a, x)

ans =
piecewise(a == -1, log(x), a ~= -1, x^(a + 1)/(a + 1))
```

### Assumptions on Parameters and Variables of Equation

Use assumptions to restrict the returned solutions of an equation to a particular interval.

Solve this equation.

```
syms x
eqn = x^5 - (565*x^4)/6 - (1159*x^3)/2 - (2311*x^2)/6 + (365*x)/2 + 250/3;
solve(eqn, x)

ans =
-5
-1
-1/3
1/2
100
```

Use `assume` to restrict the solutions to the interval  $-1 \leq x \leq 1$ .

```
assume(-1 <= x <= 1)
solve(eqn, x)
```



```
ans =
-1
-1/3
1/2
```

Set several assumptions simultaneously by using the logical operators `and`, `or`, `xor`, `not`, or their shortcuts. For example, all negative solutions less than -1 and all positive solutions greater than 1.

```
assume(x < -1 | x > 1)
solve(eqn, x)
```

```
ans =
-5
100
```

For further computations, clear the assumptions.

```
assume(x, "clear")
```

### Use Assumptions for Simplification

Setting appropriate assumptions can result in simpler expressions.

Try to simplify the expression  $\sin(2\pi n)$  using `simplify`. The `simplify` function cannot simplify the input and returns the input as it is.

```
syms n
simplify(sin(2*n*pi))
```

```
ans =
sin(2*pi*n)
```

Assume  $n$  is an integer. `simplify` now simplifies the expression.

```
assume(n, "integer")
simplify(sin(2*n*pi))
```

```
ans =
0
```

For further computations, clear the assumption.

```
assume(n, "clear")
```

### Assumptions on Expressions

Set an assumption on a symbolic expression.

You can set assumptions not only on variables, but also on expressions. For example, compute this integral.

```
syms x
f = 1/abs(x^2 - 1);
int(f, x)
```

```
ans =
-atanh(x)/sign(x^2 - 1)
```

Set the assumption  $x^2 - 1 > 0$  to produce a simpler result.

```
assume(x^2 - 1 > 0)
int(f,x)
```

```
ans =
-atanh(x)
```

For further computations, clear the assumption.

```
assume(x,"clear")
```

### Assumptions to Prove Relations

Prove relations that hold under certain conditions by first assuming the conditions and then using `isAlways`.

Prove that  $\sin(\pi x)$  is never equal to 0 when  $x$  is not an integer. The `isAlways` function returns logical 1 (true), which means the condition holds for all values of  $x$  under the set assumptions.

```
syms x
assume(~in(x,"integer"))
isAlways(sin(pi*x) ~= 0)

ans =
    logical
     1
```

### Assumptions on Matrix Elements

Set assumptions on all elements of a matrix using `sym`.

Create the 2-by-2 symbolic matrix  $A$  with autogenerated elements. Specify set as `rational`.

```
A = sym("A",[2 2],"rational")

A =
[ A1_1, A1_2]
[ A2_1, A2_2]
```

Return the assumptions on the elements of  $A$  using `assumptions`.

```
assumptions(A)

ans =
[ in(A1_1, 'rational'), in(A1_2, 'rational'),...
  in(A2_1, 'rational'), in(A2_2, 'rational')]
```

You can also use `assume` to set assumptions on all elements of a matrix. Now, assume all elements of  $A$  have positive rational values. Set the assumptions as a cell array of character vectors `{'positive','rational'}`.

```
assume(A,{'positive','rational'})
```

Return the assumptions on the elements of  $A$  using `assumptions`.

```
assumptions(A)

ans =
[ 0 < A1_1, 0 < A1_2, 0 < A2_1, 0 < A2_2,...
  in(A1_1, 'rational'), in(A1_2, 'rational'),...
  in(A2_1, 'rational'), in(A2_2, 'rational')]
```

For further computations, clear the assumptions.

```
assume(A,"clear")
```

## Input Arguments

### **condition** — Assumption statement

symbolic expression | symbolic equation | symbolic relation | vector or matrix of symbolic expressions, equations, or relations

Assumption statement, specified as a symbolic expression, equation, relation, or vector or matrix of symbolic expressions, equations, or relations. You also can combine several assumptions by using the logical operators `and`, `or`, `xor`, `not`, or their shortcuts.

### **expr** — Expression to set assumption on

symbolic variable | symbolic expression | vector or matrix of symbolic variables or expressions

Expression to set assumption on, specified as a symbolic variable, expression, vector, or matrix. If `expr` is a vector or matrix, then `assume(expr, set)` sets an assumption that each element of `expr` belongs to `set`.

### **set** — Set of assumptions

character vector | string scalar | string array | cell array

Set of assumptions, specified as a character vector, string scalar, string array, or cell array. The available assumptions are "integer", "rational", "real", or "positive".

You can combine multiple assumptions by specifying a string array or cell array of character vectors. For example, assume a positive rational value by specifying `set` as `["positive" "rational"]` or `{'positive','rational'}`.

## Tips

- `assume` removes any assumptions previously set on the symbolic variables. To retain previous assumptions while adding an assumption, use `assumeAlso`.
- When you delete a symbolic variable from the MATLAB workspace using `clear`, all assumptions that you set on that variable remain in the symbolic engine. If you later declare a new symbolic variable with the same name, it inherits these assumptions.
- To clear all assumptions set on a symbolic variable `var`, use this command.

```
assume(var,"clear")
```

- To delete all objects in the MATLAB workspace and close the Symbolic Math Toolbox engine associated with the MATLAB workspace clearing all assumptions, use this command:

```
clear all
```

- MATLAB projects complex numbers in inequalities to the real axis. If `condition` is an inequality, then both sides of the inequality must represent real values. Inequalities with complex numbers are invalid because the field of complex numbers is not an ordered field. (It is impossible to tell whether  $5 + i$  is greater or less than  $2 + 3i$ .) For example,  $x > i$  becomes  $x > 0$ , and  $x \leq 3 + 2i$  becomes  $x \leq 3$ .
- The toolbox does not support assumptions on symbolic functions. Set assumptions on symbolic variables and expressions instead.

- When you create a new symbolic variable using `sym` and `syms`, you also can set an assumption that the variable is real, positive, integer, or rational.

```
a = sym("a", "real");  
b = sym("b", "rational");  
c = sym("c", "positive");  
d = sym("d", "positive");  
e = sym("e", {'positive', 'integer'});
```

Alternatively, you can use these commands, which are more efficient.

```
syms a real  
syms b rational  
syms c d positive  
syms e positive integer
```

## Version History

Introduced in R2012a

### See Also

#### Functions

`and` | `assumeAlso` | `assumptions` | `isAlways` | `in` | `not` | `or` | `piecewise` | `sym` | `syms`

#### Topics

“Use Assumptions on Symbolic Variables” on page 1-41

# assumeAlso

Add assumption on symbolic object

## Syntax

```
assumeAlso(condition)
assumeAlso(expr, set)
```

## Description

`assumeAlso(condition)` states that `condition` is valid for all symbolic variables in `condition`. It retains all assumptions previously set on these symbolic variables.

`assumeAlso(expr, set)` states that `expr` belongs to `set`, in addition to all previously made assumptions.

## Examples

### Assumptions Specified as Relations

Set assumptions using `assume`. Then add more assumptions using `assumeAlso`.

Solve this equation assuming that both `x` and `y` are nonnegative.

```
syms x y
assume(x >= 0 & y >= 0)
s = solve(x^2 + y^2 == 1, y)
```

```
Warning: Solutions are valid under the following
conditions: x <= 1;
x == 1.
```

```
To include parameters and conditions in the
solution, specify the 'ReturnConditions' value as
'true'.
```

```
> In solve>warnIfParams (line 482)
In solve (line 357)
```

```
s =
(1 - x)^(1/2)*(x + 1)^(1/2)
-(1 - x)^(1/2)*(x + 1)^(1/2)
```

The solver warns that both solutions hold only under certain conditions.

Add the assumption that `x < 1`. To add a new assumption without removing the previous one, use `assumeAlso`.

```
assumeAlso(x < 1)
```

Solve the same equation under the expanded set of assumptions.

```
s = solve(x^2 + y^2 == 1, y)
```

```
s =
(1 - x)^(1/2)*(x + 1)^(1/2)
```

For further computations, clear the assumptions.

```
assume([x y], 'clear')
```

### Assumptions Specified as Sets

Set assumptions using `syms`. Then add more assumptions using `assumeAlso`.

When declaring the symbolic variable `n`, set an assumption that `n` is positive.

```
syms n positive
```

Using `assumeAlso`, add more assumptions on the same variable `n`. For example, assume also that `n` is an integer.

```
assumeAlso(n, 'integer')
```

Return all assumptions affecting variable `n` using `assumptions`. In this case, `n` is a positive integer.

```
assumptions(n)

ans =
[ 0 < n, in(n, 'integer')]
```

For further computations, clear the assumptions.

```
assume(n, 'clear')
```

### Assumptions on Matrix Elements

Use the assumption on a matrix as a shortcut for setting the same assumption on each matrix element.

Create the 3-by-3 symbolic matrix `A` with auto-generated elements. To assume every element of `A` is rational, specify `set` as `'rational'`.

```
A = sym('A',[3 3],'rational')

A =
[ A1_1, A1_2, A1_3]
[ A2_1, A2_2, A2_3]
[ A3_1, A3_2, A3_3]
```

Now, add the assumption that each element of `A` is greater than 1.

```
assumeAlso(A > 1)
```

Return assumptions affecting elements of `A` using `assumptions`:

```
assumptions(A)

ans =
[ 1 < A1_1, 1 < A1_2, 1 < A1_3, 1 < A2_1, 1 < A2_2, 1 < A2_3,...
  1 < A3_1, 1 < A3_2, 1 < A3_3,...
  in(A1_1, 'rational'), in(A1_2, 'rational'), in(A1_3, 'rational'),...
  in(A2_1, 'rational'), in(A2_2, 'rational'), in(A2_3, 'rational'),...
  in(A3_1, 'rational'), in(A3_2, 'rational'), in(A3_3, 'rational')]
```

For further computations, clear the assumptions.

```
assume(A, 'clear')
```

## Contradicting Assumptions

When you add assumptions, ensure that the new assumptions do not contradict the previous assumptions. Contradicting assumptions can lead to inconsistent and unpredictable results. In some cases, `assumeAlso` detects conflicting assumptions and issues an error.

Try to set contradicting assumptions. `assumeAlso` returns an error.

```
syms y
assume(y, 'real')
assumeAlso(y == i)

Error using mupadengine/feval (line 172)
Inconsistent assumptions.
Error in sym/assumeAlso (line 627)
    feval(symengine, 'assumeAlso', cond);
```

`assumeAlso` does not guarantee to detect contradicting assumptions. For example, assume that  $y$  is nonzero, and both  $y$  and  $y*i$  are real values.

```
syms y
assume(y ~= 0)
assumeAlso(y, 'real')
assumeAlso(y*i, 'real')
```

Return all assumptions affecting variable  $y$  using `assumptions`:

```
assumptions(y)

ans =
[ in(y, 'real'), in(y*i, 'real'), y ~= 0]
```

For further computations, clear the assumptions.

```
assume(y, 'clear')
```

## Input Arguments

### condition — Assumption statement

symbolic expression | symbolic equation | relation | vector or matrix of symbolic expressions, equations, or relations

Assumption statement, specified as a symbolic expression, equation, relation, or vector or matrix of symbolic expressions, equations, or relations. You also can combine several assumptions by using the logical operators `and`, `or`, `xor`, `not`, or their shortcuts.

### expr — Expression to set assumption on

symbolic variable | symbolic expression | vector or matrix of symbolic variables or expressions

Expression to set assumption on, specified as a symbolic variable, expression, or a vector or matrix of symbolic variables or expressions. If `expr` is a vector or matrix, then `assumeAlso(expr, set)` sets an assumption that each element of `expr` belongs to `set`.

### set — Set of assumptions

character vector | string array | cell array

Set of assumptions, specified as a character vector, string array, or cell array. The available assumptions are 'integer', 'rational', 'real', or 'positive'.

You can combine multiple assumptions by specifying a string array or cell array of character vectors. For example, assume a positive rational value by specifying `set` as `["positive" "rational"]` or `{'positive','rational'}`.

## Tips

- `assumeAlso` keeps all assumptions previously set on the symbolic variables. To replace previous assumptions with the new one, use `assume`.
- When adding assumptions, always check that a new assumption does not contradict the existing assumptions. To see existing assumptions, use `assumptions`. Symbolic Math Toolbox does not guarantee to detect conflicting assumptions. Conflicting assumptions can lead to unpredictable and inconsistent results.
- When you delete a symbolic variable from the MATLAB workspace using `clear`, all assumptions that you set on that variable remain in the symbolic engine. If later you declare a new symbolic variable with the same name, it inherits these assumptions.
- To clear all assumptions set on a symbolic variable `var` use this command.

```
assume(var, 'clear')
```

- To clear all objects in the MATLAB workspace and close the Symbolic Math Toolbox engine associated with the MATLAB workspace resetting all its assumptions, use this command.

```
clear all
```

- MATLAB projects complex numbers in inequalities to the real axis. If `condition` is an inequality, then both sides of the inequality must represent real values. Inequalities with complex numbers are invalid because the field of complex numbers is not an ordered field. (It is impossible to tell whether  $5 + i$  is greater or less than  $2 + 3i$ .) For example,  $x > i$  becomes  $x > 0$ , and  $x \leq 3 + 2i$  becomes  $x \leq 3$ .
- The toolbox does not support assumptions on symbolic functions. Make assumptions on symbolic variables and expressions instead.
- Instead of adding assumptions one by one, you can set several assumptions in one function call. To set several assumptions, use `assume` and combine these assumptions by using the logical operators `and`, `or`, `xor`, `not`, `all`, `any`, or their shortcuts.

## Version History

Introduced in R2012a

## See Also

`and` | `assume` | `assumptions` | `in` | `isAlways` | `not` | `or` | `piecewise` | `sym` | `syms`

## Topics

“Use Assumptions on Symbolic Variables” on page 1-41



# assumptions

Show assumptions affecting symbolic variable, expression, or function

## Syntax

```
assumptions(var)
assumptions
```

## Description

`assumptions(var)` returns all assumptions that affect variable `var`. If `var` is an expression or function, `assumptions` returns all assumptions that affect all variables in `var`.

`assumptions` returns all assumptions that affect all variables in MATLAB Workspace.

## Examples

### Assumptions on Variables

In Symbolic Math Toolbox™, you can set mathematical assumptions or conditions when creating symbolic variables. For example, create a symbolic variable `n` and assume that the variable is an integer. Return the assumption using `assumptions`.

```
syms n integer
assumptions
```

```
ans = n ∈ ℤ
```

You can also use the `assume` function to set assumptions. For example, assume that `n` is less than `x` and that `x < 42`. The `assume` function replaces old assumptions on input with the new assumptions. Return all assumptions that affect `n`.

```
syms x
assume(n < x & x < 42)
assumptions(n)
```

```
ans = (n < x & x < 42)
```

`assumptions` returns the assumption `x < 42` because it affects `n` through the assumption `n < x`. Thus, `assumptions` returns the transitive closure of assumptions, which is all assumptions that mathematically affect the input.

Set the assumption on variable `m` that `1 < m < 3`. Return all assumptions on `m` and `x` using `assumptions`.

```
syms m
assume(1 < m < 3)
assumptions([m x])
```

```
ans = (n < x & 1 < m < 3 & x < 42)
```

To see the assumptions that affect all variables, use `assumptions` without any arguments.

```
assumptions
```

```
ans = (n < x 1 < m m < 3 x < 42)
```

For further computations, clear the assumptions.

```
assume([m n x], "clear")
```

### Multiple Assumptions on One Variable

You cannot set an additional assumption on a variable using `assume` because `assume` clears all previous assumptions on that variable. To set an additional assumption on a variable, using `assumeAlso`.

Set an assumption on  $x$  using `assume`. Set an additional assumption on  $x$  use `assumeAlso`. Use `assumptions` to return the multiple assumptions on  $x$ .

```
syms x
assume(x, "real")
assumeAlso(x < 0)
assumptions(x)
```

```
ans = (x ∈ ℝ x < 0)
```

For further computations, clear the assumptions.

```
assume(x, "clear")
```

### Assumptions Affecting Expressions and Functions

`assumptions` accepts symbolic expressions and functions as input and returns all assumptions that affect all variables in the symbolic expressions or functions.

Set assumptions on variables in a symbolic expression and function. Find all assumptions that affect all variables in the symbolic expression using `assumptions`.

```
syms a b c f(a,b,c)
expr = a*exp(b)*sin(c);
assume(a+b > 3 & in(a, "integer") & in(c, "real"))
assumptions(expr)
```

```
ans = (a ∈ ℤ c ∈ ℝ 3 < a + b)
```

Find all assumptions that affect all variables that are inputs to a symbolic function.

```
assumptions(f)
```

```
ans = (a ∈ ℤ c ∈ ℝ 3 < a + b)
```

Note that if you use `syms` to create another symbolic function after setting assumptions on the function input arguments, `syms` clears all previously set assumptions on the symbolic variables. Instead, create the symbolic function first, and then set the assumptions on the symbolic variables.

```
syms g(a,b,c)
assumptions(g)
```

```
ans =
```

```
Empty sym: 1-by-0
```

```
assume(a+b > 3 & in(a,"integer") & in(c,"real"))
assumptions(g)
```

```
ans = (a ∈ ℤ c ∈ ℝ 3 < a + b)
```

Clear the assumptions for further computations.

```
assume([a b c],"clear")
```

## Restore Old Assumptions

To restore old assumptions, first store the assumptions returned by `assumptions`. Then you can restore these assumptions at any point by calling `assume` or `assumeAlso`.

Solve the equation for a spring system using `dsolve` under the assumptions that the mass and spring constant are positive.

```
syms m k positive
syms x(t)
xsol = dsolve(m*diff(x,t,t) == -k*x, x(0) == 0)
```

```
xsol =
```

$$-C_1 \sin\left(\frac{\sqrt{k} t}{\sqrt{m}}\right)$$

Suppose you want to explore solutions unconstrained by assumptions, but want to restore the assumptions afterwards. First store the assumptions using `assumptions`, then clear the assumptions and solve the equation. `dsolve` returns unconstrained solutions.

```
tmp = assumptions;
assume([m k],"clear")
xsol = dsolve(m*diff(x,t,t) == -k*x, x(0) == 0)
```

```
xsol =
```

$$-C_1 e^{-\frac{t\sqrt{-k m}}{m}} \left( e^{\frac{2 t\sqrt{-k m}}{m}} - 1 \right)$$

Restore the original assumptions using `assume`.

```
assume(tmp)
```

After computations are complete, clear assumptions using `assume`.

```
assume([m k], "clear")
```

## Input Arguments

### **var** — Symbolic input to check for assumptions

symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix | symbolic multidimensional array

Symbolic input for which to show assumptions, specified as a symbolic variable, expression, or function, or a vector, matrix, or multidimensional array of symbolic variables, expressions, or functions.

## Tips

- When you delete a symbolic object from the MATLAB workspace by using `clear`, all assumptions that you set on that object remain in the symbolic engine. If you declare a new symbolic variable with the same name, it inherits these assumptions.
- To clear all assumptions set on a symbolic variable `var` use this command.

```
assume(var, "clear")
```

- To clear all objects in the MATLAB workspace and close the Symbolic Math Toolbox engine associated with the MATLAB workspace resetting all its assumptions, use this command.

```
clear all
```

## Version History

**Introduced in R2012a**

## See Also

`and` | `assume` | `assumeAlso` | `clear` | `in` | `isAlways` | `not` | `or` | `piecewise` | `sym` | `syms`

## Topics

“Use Assumptions on Symbolic Variables” on page 1-41

# atan

Symbolic inverse tangent

## Syntax

$P = \text{atan}(Z)$   
 $P = \text{atan}(Y,X)$

## Description

$P = \text{atan}(Z)$  returns the inverse tangent on page 6-88 (arctangent) of the elements of  $Z$ . All angles are in radians.

- For real values of  $Z$ ,  $\text{atan}(Z)$  returns values in the interval  $[-\pi/2, \pi/2]$ .
- For complex values of  $Z$ ,  $\text{atan}(Z)$  returns complex values with the real parts in the interval  $[-\pi/2, \pi/2]$ .

$P = \text{atan}(Y,X)$  returns the four-quadrant inverse tangent on page 6-88 of the elements of  $Y$  and  $X$ . This syntax with two input arguments is the same as  $\text{atan2}(Y,X)$ .

Symbolic arguments  $X$  and  $Y$  are assumed to be real, and  $\text{atan}(Y,X)$  returns values in the interval  $[-\pi, \pi]$ .

## Examples

### Inverse Tangent Function for Numeric and Symbolic Arguments

Depending on its arguments, `atan` returns floating-point or exact symbolic results.

Compute the inverse tangent function for these numbers. Because these numbers are not symbolic objects, `atan` returns floating-point results.

```
P = atan([-1, -1/3, -1/sqrt(3), 1/2, 1, sqrt(3)])
```

```
P = 1×6
```

```
-0.7854    -0.3218    -0.5236     0.4636     0.7854     1.0472
```

Compute the inverse tangent function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `atan` returns unresolved symbolic calls.

```
symP = atan(sym([-1, -1/3, -1/sqrt(3), 1/2, 1, sqrt(3)]))
```

```
symP =
```

$$\left(-\frac{\pi}{4} - \text{atan}\left(\frac{1}{3}\right) - \frac{\pi}{6} \text{atan}\left(\frac{1}{2}\right) \frac{\pi}{4} \frac{\pi}{3}\right)$$

Use `vpa` to approximate symbolic results with floating-point numbers.

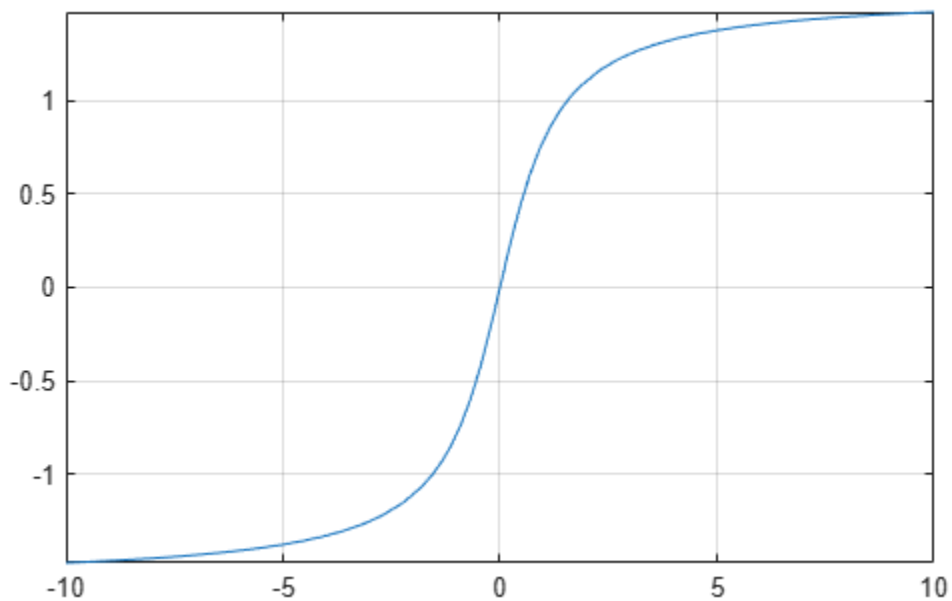
```
vpaP = vpa(symP)
```

```
vpaP = (-0.78539816339744830961566084581988 -0.32175055439664219340140461435866 -0.5235987755982988730)
```

### Plot Inverse Tangent Function

Plot the inverse tangent function on the interval from -10 to 10.

```
syms x
fplot(atan(x), [-10 10])
grid on
```



### Handle Expressions Containing Inverse Tangent Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `atan`.

Find the first and second derivatives of the inverse tangent function.

```
syms z
D1 = diff(atan(z), z)
```

$$D1 = \frac{1}{z^2 + 1}$$

```
D2 = diff(atan(z), z, z)
```

$$D2 = -\frac{2z}{(z^2 + 1)^2}$$

Find the indefinite integral of the inverse tangent function.

```
I = int(atan(z), z)
```

$$I = z \operatorname{atan}(z) - \frac{\log(z^2 + 1)}{2}$$

Find the Taylor series expansion of  $\operatorname{atan}(z)$ .

```
T = taylor(atan(z), z)
```

$$T = \frac{z^5}{5} - \frac{z^3}{3} + z$$

Rewrite the inverse tangent function in terms of the natural logarithm.

```
R = rewrite(atan(z), 'log')
```

$$R = \frac{\log(1 - z i) i}{2} - \frac{\log(1 + z i) i}{2}$$

## Input Arguments

### Z — Tangent of angle

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix | symbolic array

Tangent of angle, specified as a symbolic number, variable, expression, or function, or as a vector, matrix, or array of symbolic numbers, variables, expressions, or functions.

### Y — y-coordinates

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix | symbolic array

y-coordinates, specified as a symbolic number, variable, expression, or function, or as a vector, matrix, or array of symbolic numbers, variables, expressions, or functions. All numerical elements of Y must be real.

Inputs Y and X must either be the same size or have sizes that are compatible (for example, Y is an M-by-N matrix and X is a scalar or 1-by-N row vector). For more information, see “Compatible Array Sizes for Basic Operations”.

### X — x-coordinates

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix | symbolic array

x-coordinates, specified as a symbolic number, variable, expression, or function, or as a vector, matrix, or array of symbolic numbers, variables, expressions, or functions. All numerical elements of X must be real.

Inputs  $Y$  and  $X$  must either be the same size or have sizes that are compatible (for example,  $Y$  is an  $M$ -by- $N$  matrix and  $X$  is a scalar or 1-by- $N$  row vector). For more information, see “Compatible Array Sizes for Basic Operations”.

## More About

### Inverse Tangent

The inverse tangent is defined as

$$\operatorname{atan}(Z) = \frac{i}{2} \log\left(\frac{1 - iZ}{1 + iZ}\right).$$

### Four-Quadrant Inverse Tangent

If  $X \neq 0$  and  $Y \neq 0$ , then

$$\operatorname{atan}(Y, X) = \operatorname{atan}\left(\frac{Y}{X}\right) + \frac{\pi}{2} \operatorname{sign}(Y)(1 - \operatorname{sign}(X)).$$

Results returned by `atan(Y,X)` belong to the closed interval  $[-\pi, \pi]$ . Meanwhile, results returned by `atan(Y/X)` belong to the closed interval  $[-\pi/2, \pi/2]$ .

## Version History

Introduced before R2006a

### See Also

`acos` | `asin` | `acsc` | `asec` | `acot` | `atan2` | `sin` | `cos` | `tan` | `csc` | `sec` | `cot`



# atan2

Symbolic four-quadrant inverse tangent

## Syntax

$P = \text{atan2}(Y,X)$

## Description

$P = \text{atan2}(Y,X)$  computes the four-quadrant inverse tangent on page 6-90 (arctangent) of  $Y$  and  $X$ .

Symbolic arguments  $X$  and  $Y$  are assumed to be real, and  $\text{atan2}(Y,X)$  returns values in the interval  $[-\pi, \pi]$ .

## Examples

### Four-Quadrant Inverse Tangent for Numeric and Symbolic Arguments

Compute the arctangents of these parameters. Because these numbers are not symbolic objects, you get floating-point results.

$P = [\text{atan2}(1,1), \text{atan2}(\pi,4), \text{atan2}(\text{Inf},\text{Inf})]$

$P = 1 \times 3$

0.7854      0.6658      0.7854

Compute the arctangents of these parameters which are converted to symbolic objects.

$P = [\text{atan2}(\text{sym}(1),1), \text{atan2}(\text{sym}(\pi),\text{sym}(4)), \text{atan2}(\text{Inf},\text{sym}(\text{Inf}))]$

$P =$

$\left(\frac{\pi}{4} \text{atan}\left(\frac{\pi}{4}\right) \frac{\pi}{4}\right)$

### Limit of Four-Quadrant Inverse Tangent

Compute the limits of this symbolic expression.

syms  $x$

$P_{\text{minusinf}} = \text{limit}(\text{atan2}(x^2/(1+x),x),x,-\text{Inf})$

$P_{\text{minusinf}} =$

$-\frac{3\pi}{4}$

$P_{\text{plusinf}} = \text{limit}(\text{atan2}(x^2/(1+x),x),x,\text{Inf})$

$$P_{\text{plusinf}} = \frac{\pi}{4}$$

### Four-Quadrant Inverse Tangent of Array Input

Compute the arctangents of the elements of matrices Y and X.

```
Y = sym([3 sqrt(3); 1 1]);
X = sym([sqrt(3) 3; 1 0]);
P = atan2(Y,X)
```

$$P = \begin{pmatrix} \frac{\pi}{3} & \frac{\pi}{6} \\ \frac{\pi}{4} & \frac{\pi}{2} \end{pmatrix}$$

## Input Arguments

### Y — y-coordinates

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix | symbolic array

y-coordinates, specified as a symbolic number, variable, expression, or function, or as a vector, matrix, or array of symbolic numbers, variables, expressions, or functions. All numerical elements of Y must be real.

Inputs Y and X must either be the same size or have sizes that are compatible (for example, Y is an M-by-N matrix and X is a scalar or 1-by-N row vector). For more information, see “Compatible Array Sizes for Basic Operations”.

### X — x-coordinates

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix | symbolic array

x-coordinates, specified as a symbolic number, variable, expression, or function, or as a vector, matrix, or array of symbolic numbers, variables, expressions, or functions. All numerical elements of X must be real.

Inputs Y and X must either be the same size or have sizes that are compatible (for example, Y is an M-by-N matrix and X is a scalar or 1-by-N row vector). For more information, see “Compatible Array Sizes for Basic Operations”.

## More About

### Four-Quadrant Inverse Tangent

If  $X \neq 0$  and  $Y \neq 0$ , then

$$\text{atan2}(Y, X) = \text{atan}\left(\frac{Y}{X}\right) + \frac{\pi}{2} \text{sign}(Y)(1 - \text{sign}(X))$$

Results returned by `atan2(Y,X)` belong to the closed interval  $[-\pi, \pi]$ . Meanwhile, results returned by `atan(Y/X)` belong to the closed interval  $[-\pi/2, \pi/2]$ .

## Tips

- Calling `atan2` for numbers (or vectors or matrices of numbers) that are not symbolic objects invokes the MATLAB `atan2` function.
- If one of the arguments  $X$  and  $Y$  is a vector or a matrix, and another one is a scalar, then `atan2` expands the scalar into a vector or a matrix of the same length with all elements equal to that scalar.
- If  $X = 0$  and  $Y > 0$ , then `atan2(Y,X)` returns  $\pi/2$ .

If  $X = 0$  and  $Y < 0$ , then `atan2(Y,X)` returns  $-\pi/2$ .

If  $X = Y = 0$ , then `atan2(Y,X)` returns  $0$ .

## Alternatives

For complex  $Z = X + Y*i$ , the call `atan2(Y,X)` is equivalent to `angle(Z)`.

## Version History

Introduced in R2013a

## See Also

`angle` | `atan` | `conj` | `imag` | `real`

## atanh

Symbolic inverse hyperbolic tangent function

### Syntax

`atanh(X)`

### Description

`atanh(X)` returns the inverse hyperbolic tangent function of  $X$ .

### Examples

#### Inverse Hyperbolic Tangent Function for Numeric and Symbolic Arguments

Depending on its arguments, `atanh` returns floating-point or exact symbolic results.

Compute the inverse hyperbolic tangent function for these numbers. Because these numbers are not symbolic objects, `atanh` returns floating-point results.

```
A = atanh([-i, 0, 1/6, i/2, i, 2])
```

```
A =
    0.0000 - 0.7854i    0.0000 + 0.0000i    0.1682 + 0.0000i...
    0.0000 + 0.4636i    0.0000 + 0.7854i    0.5493 + 1.5708i
```

Compute the inverse hyperbolic tangent function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `atanh` returns unresolved symbolic calls.

```
symA = atanh(sym([-i, 0, 1/6, i/2, i, 2]))
```

```
symA =
[ -(pi*1i)/4, 0, atanh(1/6), atanh(1i/2), (pi*1i)/4, atanh(2)]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

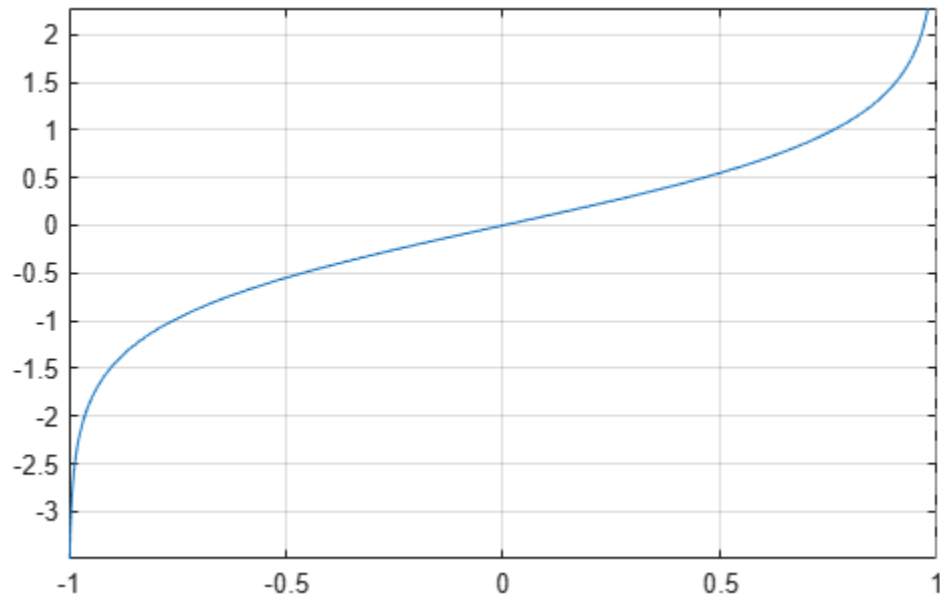
```
vpa(symA)
```

```
ans =
[ -0.78539816339744830961566084581988i,...
 0,...
 0.1682361183106064652522967051085,...
 0.46364760900080611621425623146121i,...
 0.78539816339744830961566084581988i,...
 0.54930614433405484569762261846126 - 1.5707963267948966192313216916398i]
```

#### Plot Inverse Hyperbolic Tangent Function

Plot the inverse hyperbolic tangent function on the interval from -1 to 1.

```
syms x
fplot(atanh(x), [-1 1])
grid on
```



### Handle Expressions Containing Inverse Hyperbolic Tangent Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `atanh`.

Find the first and second derivatives of the inverse hyperbolic tangent function:

```
syms x
diff(atanh(x), x)
diff(atanh(x), x, x)
```

```
ans =
-1/(x^2 - 1)
```

```
ans =
(2*x)/(x^2 - 1)^2
```

Find the indefinite integral of the inverse hyperbolic tangent function:

```
int(atanh(x), x)

ans =
log(x^2 - 1)/2 + x*atanh(x)
```

Find the Taylor series expansion of `atanh(x)`:

```
taylor(atanh(x), x)

ans =
x^5/5 + x^3/3 + x
```

Rewrite the inverse hyperbolic tangent function in terms of the natural logarithm:

```
rewrite(atanh(x), 'log')  
  
ans =  
log(x + 1)/2 - log(1 - x)/2
```

## Input Arguments

### **X — Input**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Version History

**Introduced before R2006a**

### **See Also**

acosh | asinh | acsch | asech | acoth | sinh | cosh | tanh | csch | sech | coth

# baseUnits

Base units of unit system

## Syntax

`baseUnits(unitSystem)`

## Description

`baseUnits(unitSystem)` returns the base units of the unit system `unitSystem` as a vector of symbolic units. You can use the returned units to create new unit systems by using `newUnitSystem`.

## Examples

### Base Units of Unit System

Get the base units of a unit system by using `baseUnits`. Then, modify the base units and create a new unit system using the modified base units. Available unit systems include SI, CGS, and US. For all unit systems, see “Unit Systems List” on page 2-71.

Get the base units of the SI unit system.

```
SIUnits = baseUnits('SI')

SIUnits =
[ [kg], [s], [m], [A], [cd], [mol], [K]]
```

---

**Note** Do not define a variable called `baseUnits` because the variable will prevent access to the `baseUnits` function.

---

Define base units that use kilometer for length and hour for time by modifying `SIUnits` using `subs`.

```
u = symunit;
newUnits = subs(SIUnits,[u.m u.s],[u.km u.hr])

newUnits =
[ [kg], [h], [km], [A], [cd], [mol], [K]]
```

Define the new unit system by using `newUnitSystem`.

```
newUnitSystem('SI_km_hr',newUnits)

ans =
"SI_km_hr"
```

To convert units between unit systems, see “Unit Conversions and Unit Systems” on page 2-53.

## **Input Arguments**

### **unitSystem — Name of unit system**

string | character vector

Name of the unit system, specified as a string or character vector.

## **Version History**

**Introduced in R2017b**

### **See Also**

derivedUnits | newUnitSystem | removeUnitSystem | rewrite | symunit

### **Topics**

“Units of Measurement Tutorial” on page 2-47

“Unit Conversions and Unit Systems” on page 2-53

“Units and Unit Systems List” on page 2-60

### **External Websites**

The International System of Units (SI)



# bernoulli

Bernoulli numbers and polynomials

## Syntax

```
bernoulli(n)
bernoulli(n,x)
```

## Description

`bernoulli(n)` returns the  $n$ th Bernoulli number on page 6-100.

`bernoulli(n,x)` returns the  $n$ th Bernoulli polynomial on page 6-100.

## Examples

### Bernoulli Numbers with Odd and Even Indices

The 0th Bernoulli number is 1. The next Bernoulli number can be  $-1/2$  or  $1/2$ , depending on the definition. The `bernoulli` function uses  $-1/2$ . The Bernoulli numbers with even indices  $n > 1$  alternate the signs. Any Bernoulli number with an odd index  $n > 2$  is 0.

Compute the even-indexed Bernoulli numbers with the indices from 0 to 10. Because these indices are not symbolic objects, `bernoulli` returns floating-point results.

```
bernoulli(0:2:10)

ans =
    1.0000    0.1667   -0.0333    0.0238   -0.0333    0.0758
```

Compute the same Bernoulli numbers for the indices converted to symbolic objects:

```
bernoulli(sym(0:2:10))

ans =
[ 1, 1/6, -1/30, 1/42, -1/30, 5/66]
```

Compute the odd-indexed Bernoulli numbers with the indices from 1 to 11:

```
bernoulli(sym(1:2:11))

ans =
[ -1/2, 0, 0, 0, 0, 0]
```

### Bernoulli Polynomials

For the Bernoulli polynomials, use `bernoulli` with two input arguments.

Compute the first, second, and third Bernoulli polynomials in variables  $x$ ,  $y$ , and  $z$ , respectively:

```
syms x y z
bernoulli(1, x)
```

```
bernoulli(2, y)
bernoulli(3, z)
```

```
ans =
x - 1/2
```

```
ans =
y^2 - y + 1/6
```

```
ans =
z^3 - (3*z^2)/2 + z/2
```

If the second argument is a number, `bernoulli` evaluates the polynomial at that number. Here, the result is a floating-point number because the input arguments are not symbolic numbers:

```
bernoulli(2, 1/3)
```

```
ans =
-0.0556
```

To get the exact symbolic result, convert at least one of the numbers to a symbolic object:

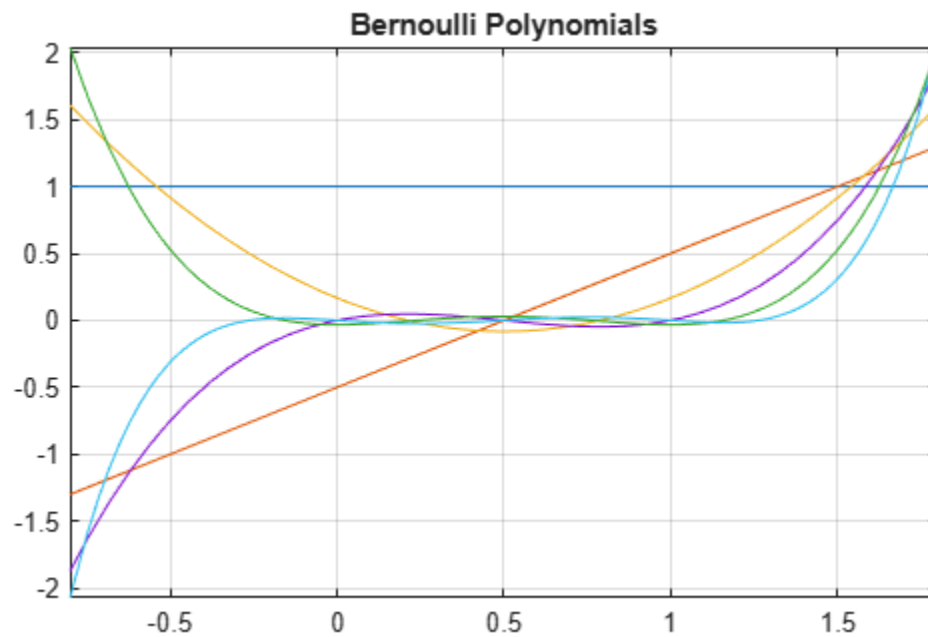
```
bernoulli(2, sym(1/3))
```

```
ans =
-1/18
```

### **Plot Bernoulli Polynomials**

Plot the first six Bernoulli polynomials.

```
syms x
fplot(bernoulli(0:5, x), [-0.8 1.8])
title('Bernoulli Polynomials')
grid on
```



### Handle Expressions Containing Bernoulli Polynomials

Many functions, such as `diff` and `expand`, handles expressions containing `bernoulli`.

Find the first and second derivatives of the Bernoulli polynomial:

```
syms n x
diff(bernoulli(n,x^2), x)

ans =
2*n*x*bernoulli(n - 1, x^2)

diff(bernoulli(n,x^2), x, x)

ans =
2*n*bernoulli(n - 1, x^2) +...
4*n*x^2*bernoulli(n - 2, x^2)*(n - 1)
```

Expand these expressions containing the Bernoulli polynomials:

```
expand(bernoulli(n, x + 3))

ans =
bernoulli(n, x) + (n*(x + 1)^n)/(x + 1) +...
(n*(x + 2)^n)/(x + 2) + (n*x^n)/x

expand(bernoulli(n, 3*x))

ans =
(3^n*bernoulli(n, x))/3 + (3^n*bernoulli(n, x + 1/3))/3 +...
(3^n*bernoulli(n, x + 2/3))/3
```

## Input Arguments

### **n — Index of the Bernoulli number or polynomial**

nonnegative integer | symbolic nonnegative integer | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Index of the Bernoulli number or polynomial, specified as a nonnegative integer, symbolic nonnegative integer, variable, expression, function, vector, or matrix. If **n** is a vector or matrix, `bernoulli` returns Bernoulli numbers or polynomials for each element of **n**. If one input argument is a scalar and the other one is a vector or a matrix, `bernoulli(n,x)` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

### **x — Polynomial variable**

symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Polynomial variable, specified as a symbolic variable, expression, function, vector, or matrix. If **x** is a vector or matrix, `bernoulli` returns Bernoulli numbers or polynomials for each element of **x**. When you use the `bernoulli` function to find Bernoulli polynomials, at least one argument must be a scalar or both arguments must be vectors or matrices of the same size. If one input argument is a scalar and the other one is a vector or a matrix, `bernoulli(n,x)` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

## More About

### **Bernoulli Polynomials**

The Bernoulli polynomials are defined as follows:

$$\frac{te^{xt}}{e^t - 1} = \sum_{n=0}^{\infty} \text{bernoulli}(n, x) \frac{t^n}{n!}$$

### **Bernoulli Numbers**

The Bernoulli numbers are defined as follows:

$$\text{bernoulli}(n) = \text{bernoulli}(n, 0)$$

## Version History

Introduced in R2014a

### **See Also**

`euler`

# bernstein

Bernstein polynomials

## Syntax

```
bernstein(f,n,t)
bernstein(g,n,t)
bernstein(g,var,n,t)
```

## Description

`bernstein(f,n,t)` with a function handle `f` returns the  $n$ th-order Bernstein polynomial on page 6-105  $\text{symsum}(\text{nchoosek}(n,k)*t^k*(1-t)^{(n-k)}*f(k/n),k,0,n)$ , evaluated at the point `t`. This polynomial approximates the function `f` over the interval  $[0,1]$ .

`bernstein(g,n,t)` with a symbolic expression or function `g` returns the  $n$ th-order Bernstein polynomial, evaluated at the point `t`. This syntax regards `g` as a univariate function of the variable determined by `symvar(g,1)`.

If any argument is symbolic, `bernstein` converts all arguments except a function handle to symbolic, and converts a function handle's results to symbolic.

`bernstein(g,var,n,t)` with a symbolic expression or function `g` returns the approximating  $n$ th-order Bernstein polynomial, regarding `g` as a univariate function of the variable `var`.

## Examples

### Approximation of Sine Function Specified as Function Handle

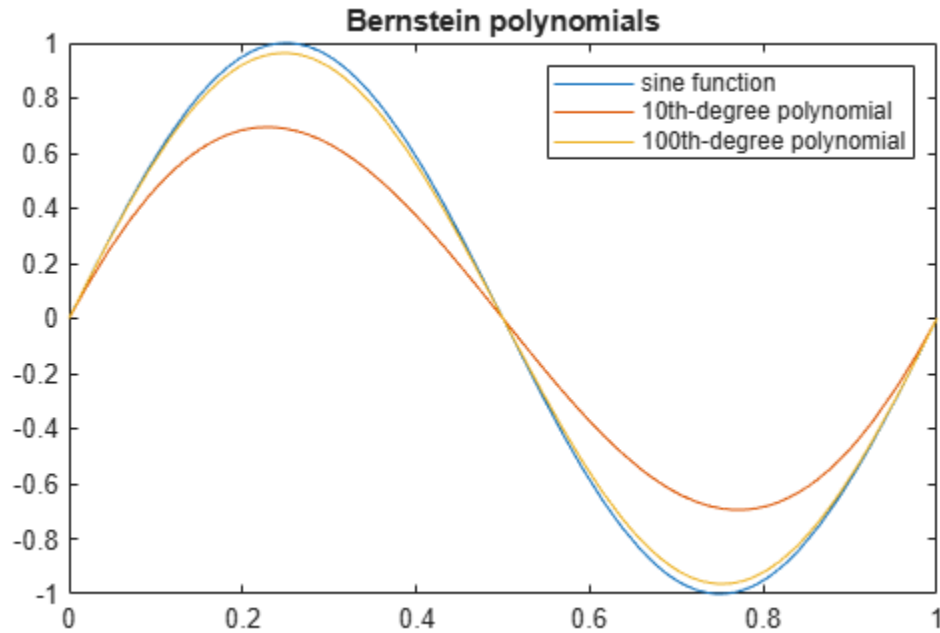
Approximate the sine function by the 10th- and 100th-degree Bernstein polynomials.

```
syms t
b10 = bernstein(@(t) sin(2*pi*t), 10, t);
b100 = bernstein(@(t) sin(2*pi*t), 100, t);

Plot sin(2*pi*t) and its approximations.

fplot(sin(2*pi*t),[0,1])
hold on
fplot(b10,[0,1])
fplot(b100,[0,1])

legend('sine function','10th-degree polynomial',...
       '100th-degree polynomial')
title('Bernstein polynomials')
hold off
```



### Approximation of Exponential Function Specified as Symbolic Expression

Approximate the exponential function by the second-order Bernstein polynomial in the variable  $t$ :

```
syms x t
bernstein(exp(x), 2, t)

ans =
(t - 1)^2 + t^2*exp(1) - 2*t*exp(1/2)*(t - 1)
```

Approximate the multivariate exponential function. When you approximate a multivariate function, `bernstein` regards it as a univariate function of the default variable determined by `symvar`. The default variable for the expression  $y \cdot \exp(x \cdot y)$  is  $x$ :

```
syms x y t
symvar(y*exp(x*y), 1)

ans =
x
```

`bernstein` treats this expression as a univariate function of  $x$ :

```
bernstein(y*exp(x*y), 2, t)

ans =
y*(t - 1)^2 + t^2*y*exp(y) - 2*t*y*exp(y/2)*(t - 1)
```

To treat  $y \cdot \exp(x \cdot y)$  as a function of the variable  $y$ , specify the variable explicitly:

```
bernstein(y*exp(x*y), y, 2, t)

ans =
t^2*exp(x) - t*exp(x/2)*(t - 1)
```

### Approximation of Linear Ramp Specified as Symbolic Function

Approximate function  $f$  representing a linear ramp by the fifth-order Bernstein polynomials in the variable  $t$ :

```
syms f(t)
f(t) = triangularPulse(1/4, 3/4, Inf, t);
p = bernstein(f, 5, t)

p =
7*t^3*(t - 1)^2 - 3*t^2*(t - 1)^3 - 5*t^4*(t - 1) + t^5
```

Simplify the result:

```
simplify(p)

ans =
-t^2*(2*t - 3)
```

### Numerical Stability of Simplified Bernstein Polynomials

When you simplify a high-order symbolic Bernstein polynomial, the result often cannot be evaluated in a numerically stable way.

Approximate this rectangular pulse function by the 100th-degree Bernstein polynomial, and then simplify the result.

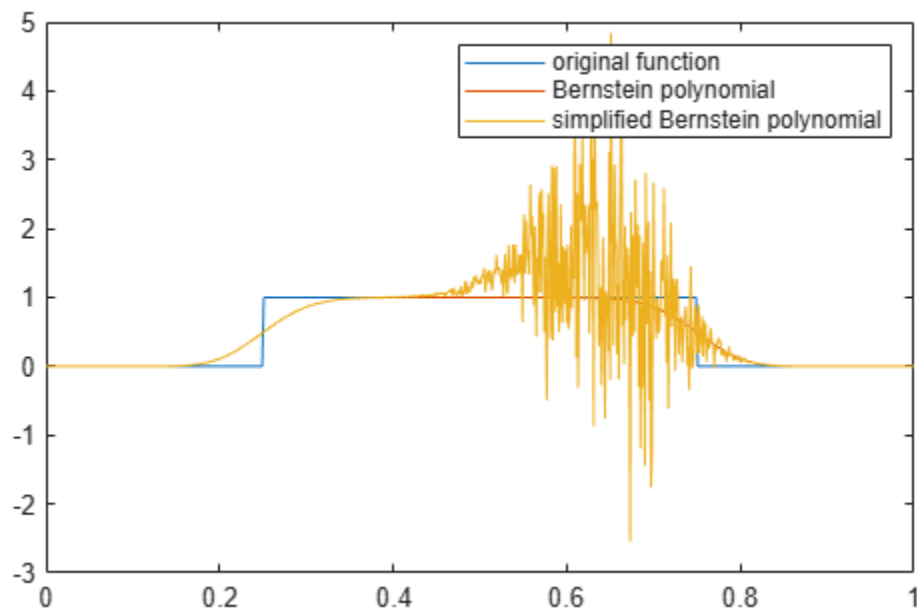
```
f = @(x)rectangularPulse(1/4,3/4,x);
b1 = bernstein(f, 100, sym('t'));
b2 = simplify(b1);
```

Convert the polynomial  $b1$  and the simplified polynomial  $b2$  to MATLAB® functions.

```
f1 = matlabFunction(b1);
f2 = matlabFunction(b2);
```

Compare the plot of the original rectangular pulse function, its numerically stable Bernstein representation  $f1$ , and its simplified version  $f2$ . The simplified version is not numerically stable.

```
t = 0:0.001:1;
plot(t, f(t), t, f1(t), t, f2(t))
hold on
legend('original function','Bernstein polynomial',...
       'simplified Bernstein polynomial')
hold off
```



## Input Arguments

### **f** — Function to be approximated by a polynomial

function handle

Function to be approximated by a polynomial, specified as a function handle. **f** must accept one scalar input argument and return a scalar value.

### **g** — Function to be approximated by a polynomial

symbolic expression | symbolic function

Function to be approximated by a polynomial, specified as a symbolic expression or function.

### **n** — Bernstein polynomial order

nonnegative integer

Bernstein polynomial order, specified as a nonnegative number.

### **t** — Evaluation point

number | symbolic number | symbolic variable | symbolic expression | symbolic function

Evaluation point, specified as a number, symbolic number, variable, expression, or function. If **t** is a symbolic function, the evaluation point is the mathematical expression that defines **t**. To extract the mathematical expression defining **t**, `bernstein` uses `formula(t)`.

### **var** — Free variable

symbolic variable

Free variable, specified as a symbolic variable.



## More About

### Bernstein Polynomials

A Bernstein polynomial is a linear combination of Bernstein basis polynomials.

A Bernstein polynomial of degree  $n$  is defined as follows:

$$B(t) = \sum_{k=0}^n \beta_k b_{k,n}(t).$$

Here,

$$b_{k,n}(t) = \binom{n}{k} t^k (1-t)^{n-k}, \quad k = 0, \dots, n$$

are the Bernstein basis polynomials, and  $\binom{n}{k}$  is a binomial coefficient.

The coefficients  $\beta_k$  are called Bernstein coefficients or Bezier coefficients.

If  $f$  is a continuous function on the interval  $[0, 1]$  and

$$B_n(f)(t) = \sum_{k=0}^n f\left(\frac{k}{n}\right) b_{k,n}(t)$$

is the approximating Bernstein polynomial, then

$$\lim_{n \rightarrow \infty} B_n(f)(t) = f(t)$$

uniformly in  $t$  on the interval  $[0, 1]$ .

## Tips

- Symbolic polynomials returned for symbolic  $t$  are numerically stable when substituting numerical values between 0 and 1 for  $t$ .
- If you simplify a symbolic Bernstein polynomial, the result can be unstable when substituting numerical values for the curve parameter  $t$ .

## Version History

Introduced in R2013b

## See Also

bernsteinMatrix | formula | nchoosek | symvar | symsum

## bernsteinMatrix

Bernstein matrix

### Syntax

```
B = bernsteinMatrix(n,t)
```

### Description

`B = bernsteinMatrix(n,t)`, where `t` is a vector, returns the `length(t)`-by-`(n+1)` Bernstein matrix `B`, such that  $B(i,k+1) = \text{nchoosek}(n,k) * t(i)^k * (1-t(i))^{(n-k)}$ . Here, the index `i` runs from 1 to `length(t)`, and the index `k` runs from 0 to `n`.

The Bernstein matrix is also called the Bezier matrix.

Use Bernstein matrices to construct Bezier curves:

```
bezierCurve = bernsteinMatrix(n, t)*P
```

Here, the `n+1` rows of the matrix `P` specify the control points of the Bezier curve. For example, to construct the second-order 3-D Bezier curve, specify the control points as:

```
P = [p0x, p0y, p0z; p1x, p1y, p1z; p2x, p2y, p2z]
```

### Examples

#### 2-D Bezier Curve

Plot the fourth-order Bezier curve specified by the control points `p0 = [0 1]`, `p1 = [4 3]`, `p2 = [6 2]`, `p3 = [3 0]`, `p4 = [2 4]`. Create a matrix with each row representing a control point.

```
P = [0 1; 4 3; 6 2; 3 0; 2 4];
```

Compute the fourth-order Bernstein matrix `B`.

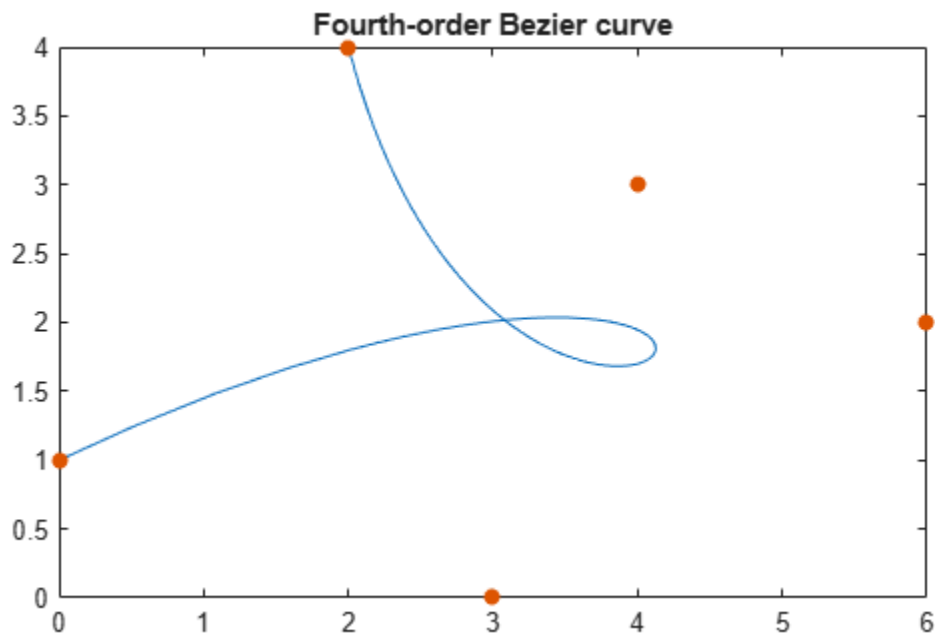
```
syms t
B = bernsteinMatrix(4,t);
```

Construct the Bezier curve.

```
bezierCurve = simplify(B*P);
```

Plot the curve adding the control points to the plot.

```
fplot(bezierCurve(1), bezierCurve(2), [0, 1])
hold on
scatter(P(:,1), P(:,2), 'filled')
title('Fourth-order Bezier curve')
hold off
```



### 3-D Bezier Curve

Construct the third-order Bezier curve specified by the 4-by-3 matrix  $P$  of control points. Each control point corresponds to a row of the matrix  $P$ .

```
P = [0 0 0; 2 2 2; 2 -1 1; 6 1 3];
```

Compute the third-order Bernstein matrix.

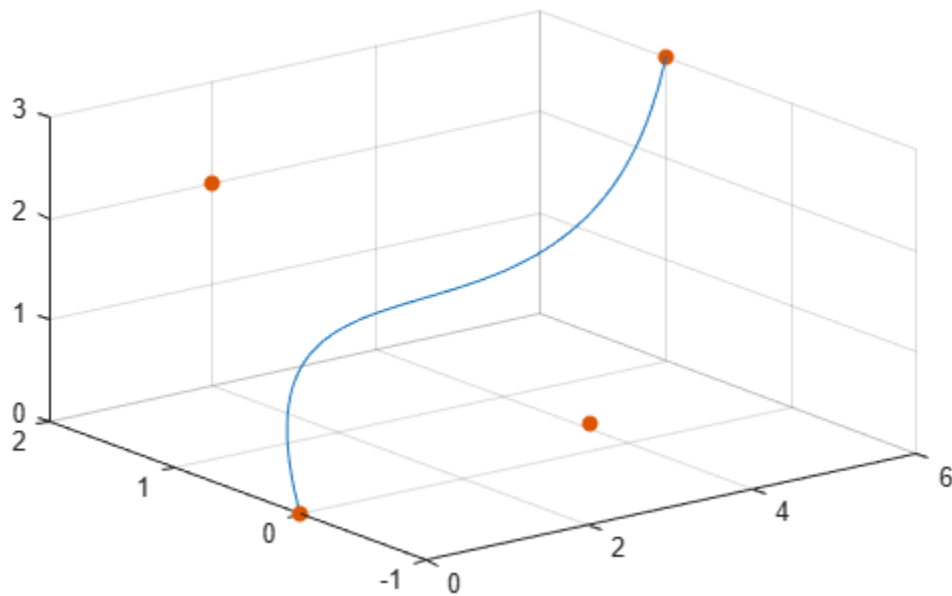
```
syms t
B = bernsteinMatrix(3,t);
```

Construct the Bezier curve.

```
bezierCurve = simplify(B*P);
```

Plot the curve adding the control points to the plot.

```
fplot3(bezierCurve(1), bezierCurve(2), bezierCurve(3), [0, 1])
hold on
scatter3(P(:,1), P(:,2), P(:,3), 'filled')
hold off
```



### 3-D Bezier Curve with the Evaluation Point Specified as a Vector

Construct the third-order Bezier curve with the evaluation point specified by the following 1-by-101 vector  $t$ .

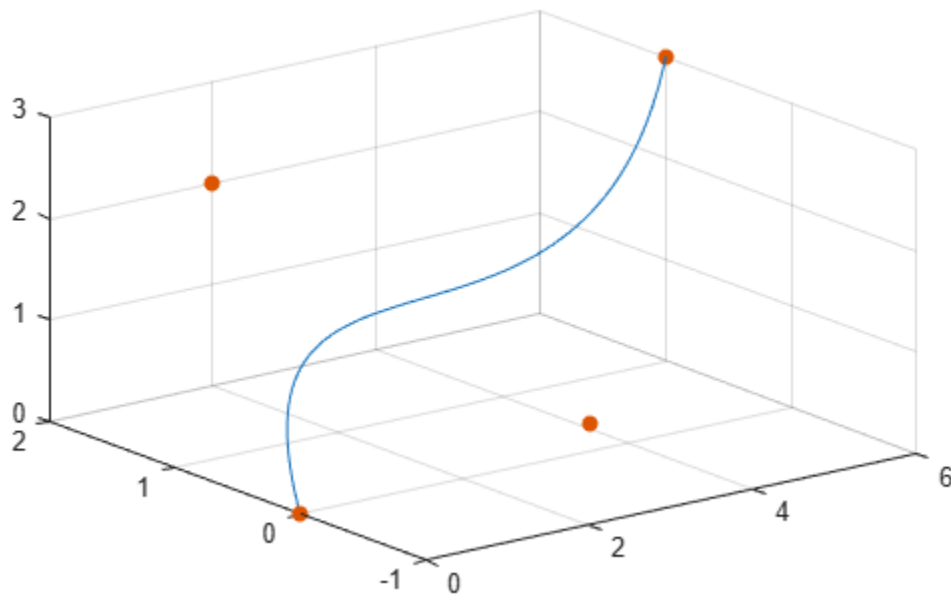
```
t = 0:1/100:1;
```

Compute the third-order 101-by-4 Bernstein matrix and specify the control points.

```
B = bernsteinMatrix(3,t);
P = [0 0 0; 2 2 2; 2 -1 1; 6 1 3];
```

Construct and plot the Bezier curve. Add grid lines and control points to the plot.

```
bezierCurve = B*P;
plot3(bezierCurve(:,1), bezierCurve(:,2), bezierCurve(:,3))
hold on
grid
scatter3(P(:,1), P(:,2), P(:,3), 'filled')
hold off
```



## Input Arguments

**n** — Approximation order

nonnegative integer

Approximation order, specified as a nonnegative integer.

**t** — Evaluation point

number | vector | symbolic number | symbolic variable | symbolic expression | symbolic vector

Evaluation point, specified as a number, symbolic number, variable, expression, or vector.

## Output Arguments

**B** — Bernstein matrix

matrix

Bernstein matrix, returned as a `length(t)`-by-`n+1` matrix.

## Version History

Introduced in R2013b

## See Also

`bernstein` | `nchoosek` | `symvar` | `symsum`

## besselh

Bessel function of third kind (Hankel function) for symbolic expressions

### Syntax

```
H = besselh(nu,K,z)
H = besselh(nu,z)
H = besselh(nu,K,z,1)
```

### Description

`H = besselh(nu,K,z)` computes the Hankel function  $H_\nu^{(K)}(z)$ , where  $K = 1$  or  $2$ , for each element of the complex array `z`. The output `H` has the symbolic data type if any input argument is symbolic. See “Bessel’s Equation” on page 6-112.

`H = besselh(nu,z)` uses  $K = 1$ .

`H = besselh(nu,K,z,1)` scales  $H_\nu^{(K)}(z)$  by  $\exp(-i*z)$  if  $K = 1$ , and by  $\exp(+i*z)$  if  $K = 2$ .

### Examples

#### Compute Hankel Function

Specify the Hankel function for a symbolic variable.

```
syms z
H = besselh(3/2,1,z)
```

$$H = -\frac{\sqrt{2} e^{zi} \left(1 + \frac{i}{z}\right)}{\sqrt{z} \sqrt{\pi}}$$

Evaluate the function symbolically and numerically at the point  $z = 1 + 2i$ .

```
Hval = subs(H,z,1+2i)
```

$$Hval = \frac{\sqrt{2} e^{-2+i} \left(-\frac{7}{5} - \frac{1}{5} i\right)}{\sqrt{1+2i} \sqrt{\pi}}$$

```
vpa(Hval)
```

```
ans = -0.084953341280586443678471523210602 - 0.056674847869835575940327724800155 i
```

Specify the function without the second argument,  $K = 1$ .

```
H2 = besselh(3/2,z)
```

```
H2 =
```

$$-\frac{\sqrt{2} e^{zi} \left(1 + \frac{i}{z}\right)}{\sqrt{z} \sqrt{\pi}}$$

Notice that the functions H and H2 are identical.

Scale the function by  $e^{-iz}$  by using the four-argument syntax.

```
Hnew = besselh(3/2,1,z,1)
```

```
Hnew =
```

$$-\frac{\sqrt{2} \left(1 + \frac{i}{z}\right)}{\sqrt{z} \sqrt{\pi}}$$

Find the derivative of H.

```
diffH = diff(H)
```

```
diffH =
```

$$\frac{\sqrt{2} e^{zi} i}{z^{5/2} \sqrt{\pi}} - \frac{\sqrt{2} e^{zi} \left(1 + \frac{i}{z}\right) i}{\sqrt{z} \sqrt{\pi}} + \frac{\sqrt{2} e^{zi} \left(1 + \frac{i}{z}\right)}{2 z^{3/2} \sqrt{\pi}}$$

## Input Arguments

### nu — Hankel function order

symbolic array | double array

Hankel function order, specified as a symbolic array or double array. If nu and z are arrays of the same size, the result is also that size. If either input is a scalar, `besselh` expands it to the other input size.

Example: `nu = 3*sym(pi)/2`

### K — Kind of Hankel function

symbolic 1 or 2 | double 1 or 2

Kind of Hankel function, specified as a symbolic or double 1 or 2. K identifies the sign of the added Bessel function Y:

$$H_\nu^{(1)}(z) = J_\nu(z) + iY_\nu(z)$$

$$H_\nu^{(2)}(z) = J_\nu(z) - iY_\nu(z).$$

Example: `K = sym(2)`

### z — Hankel function argument

symbolic array | double array

Hankel function argument, specified as a symbolic array or double array. If nu and z are arrays of the same size, the result is also that size. If either input is a scalar, `besselh` expands it to the other input size.

Example: `z = sym(1+1i)`

## More About

### Bessel's Equation

The differential equation

$$z^2 \frac{d^2 w}{dz^2} + z \frac{dw}{dz} + (z^2 - \nu^2)w = 0,$$

where  $\nu$  is a real constant, is called *Bessel's equation*, and its solutions are known as *Bessel functions*.

$J_\nu(z)$  and  $J_{-\nu}(z)$  form a fundamental set of solutions of Bessel's equation for noninteger  $\nu$ .  $Y_\nu(z)$  is a second solution of Bessel's equation—linearly independent of  $J_\nu(z)$ —defined by

$$Y_\nu(z) = \frac{J_\nu(z)\cos(\nu\pi) - J_{-\nu}(z)}{\sin(\nu\pi)}.$$

The relationship between the Hankel and Bessel functions is

$$H_\nu^{(1)}(z) = J_\nu(z) + iY_\nu(z)$$

$$H_\nu^{(2)}(z) = J_\nu(z) - iY_\nu(z).$$

Here,  $J_\nu(z)$  is `besselj`, and  $Y_\nu(z)$  is `bessely`.

## Version History

Introduced in R2018b

## References

[1] Abramowitz, M., and I. A. Stegun. *Handbook of Mathematical Functions*. National Bureau of Standards, Applied Math. Series #55, Dover Publications, 1965.

## See Also

`besseli` | `besselj` | `besselk` | `bessely`



# besseli

Modified Bessel function of the first kind for symbolic expressions

## Syntax

`besseli(nu, z)`

## Description

`besseli(nu, z)` returns the modified Bessel function of the first kind on page 6-116,  $I_\nu(z)$ .

## Examples

### Find Modified Bessel Function of First Kind

Compute the modified Bessel functions of the first kind for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
[besseli(0, 5), besseli(-1, 2), besseli(1/3, 7/4), besseli(1, 3/2 + 2*i)]
```

```
ans =
 27.2399 + 0.0000i  1.5906 + 0.0000i  1.7951 + 0.0000i  -0.1523 + 1.0992i
```

Compute the modified Bessel functions of the first kind for the numbers converted to symbolic objects. For most symbolic (exact) numbers, `besseli` returns unresolved symbolic calls.

```
[besseli(sym(0), 5), besseli(sym(-1), 2), ...
besseli(1/3, sym(7/4)), besseli(sym(1), 3/2 + 2*i)]
```

```
ans =
[ besseli(0, 5), besseli(1, 2), besseli(1/3, 7/4), besseli(1, 3/2 + 2i)]
```

For symbolic variables and expressions, `besseli` also returns unresolved symbolic calls:

```
syms x y
[besseli(x, y), besseli(1, x^2), besseli(2, x - y), besseli(x^2, x*y)]
```

```
ans =
[ besseli(x, y), besseli(1, x^2), besseli(2, x - y), besseli(x^2, x*y)]
```

### Solve Bessel Differential Equation for Modified Bessel Functions

Solve this second-order differential equation. The solutions are the modified Bessel functions of the first and the second kind.

```
syms nu w(z)
dsolve(z^2*diff(w, 2) + z*diff(w) -(z^2 + nu^2)*w == 0)
```

```
ans =
C2*besseli(nu, z) + C3*besselk(nu, z)
```

Verify that the modified Bessel function of the first kind is a valid solution of the modified Bessel differential equation.

```

syms nu z
isAlways(z^2*diff(besseli(nu, z), z, 2) + z*diff(besseli(nu, z), z)...
- (z^2 + nu^2)*besseli(nu, z) == 0)

ans =
    logical
    1

```

### Special Values of Modified Bessel Function of First Kind

If the first parameter is an odd integer multiplied by 1/2, `besseli` rewrites the Bessel functions in terms of elementary functions:

```

syms x
besseli(1/2, x)

ans =
(2^(1/2)*sinh(x))/(x^(1/2)*pi^(1/2))

besseli(-1/2, x)

ans =
(2^(1/2)*cosh(x))/(x^(1/2)*pi^(1/2))

besseli(-3/2, x)

ans =
(2^(1/2)*(sinh(x) - cosh(x)/x))/(x^(1/2)*pi^(1/2))

besseli(5/2, x)

ans =
-(2^(1/2)*((3*cosh(x))/x - sinh(x)*(3/x^2 + 1)))/(x^(1/2)*pi^(1/2))

```

### Differentiate Modified Bessel Function of First Kind

Differentiate the expressions involving the modified Bessel functions of the first kind:

```

syms x y
diff(besseli(1, x))
diff(diff(besseli(0, x^2 + x*y - y^2), x), y)

ans =
besseli(0, x) - besseli(1, x)/x

ans =
besseli(1, x^2 + x*y - y^2) +...
(2*x + y)*(besseli(0, x^2 + x*y - y^2)*(x - 2*y) - ...
(besseli(1, x^2 + x*y - y^2)*(x - 2*y))/(x^2 + x*y - y^2))

```

### Bessel Function for Matrix Input

Call `besseli` for the matrix `A` and the value 1/2. The result is a matrix of the modified Bessel functions `besseli(1/2, A(i,j))`.

```

syms x
A = [-1, pi; x, 0];
besseli(1/2, A)

```

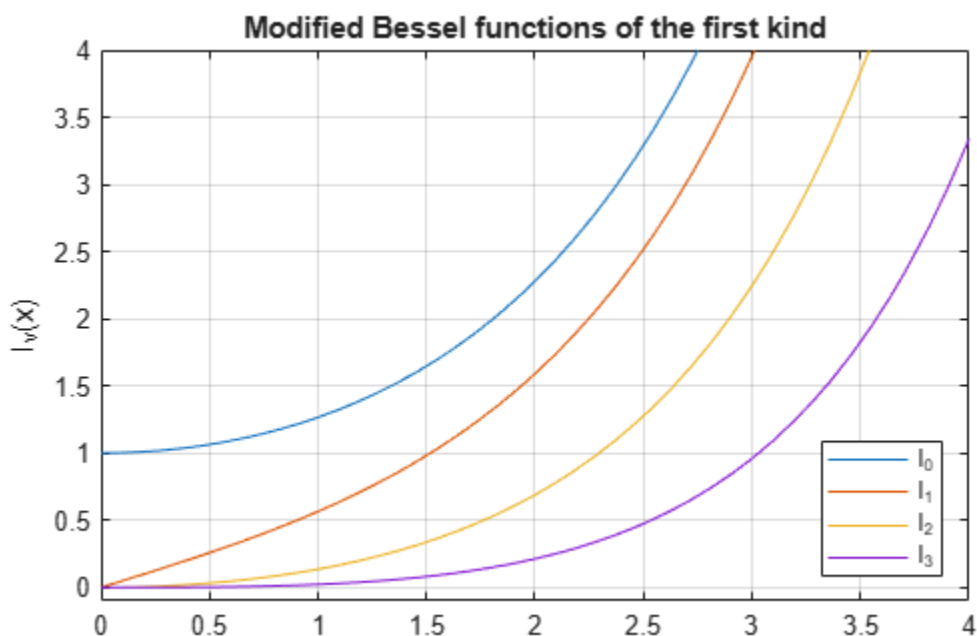
```
ans =
[ (2^(1/2)*sinh(1)*1i)/pi^(1/2), (2^(1/2)*sinh(pi))/pi]
[ (2^(1/2)*sinh(x))/(x^(1/2)*pi^(1/2)), 0]
```

### Plot the Modified Bessel Functions of the First Kind

Plot the modified Bessel functions of the first kind for  $\nu = 0, 1, 2, 3$ .

```
syms x y
fplot(besseli(0:3, x))
axis([0 4 -0.1 4])
grid on

ylabel('I_v(x)')
legend('I_0','I_1','I_2','I_3', 'Location','Best')
title('Modified Bessel functions of the first kind')
```



## Input Arguments

### nu — Input

number | vector | matrix | multidimensional array | symbolic number | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, array, or a symbolic number, variable, expression, function, or array. If `nu` is a vector or matrix, `besseli` returns the modified Bessel function of the first kind for each element of `nu`.

### z — Input

number | vector | matrix | multidimensional array | symbolic number | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, array, or a symbolic number, variable, expression, function, or array. If `nu` is a vector or matrix, `besseli` returns the modified Bessel function of the first kind for each element of `nu`.

## More About

### Modified Bessel Functions of the First Kind

The modified Bessel differential equation

$$z^2 \frac{d^2 w}{dz^2} + z \frac{dw}{dz} - (z^2 + \nu^2)w = 0$$

has two linearly independent solutions. These solutions are represented by the modified Bessel functions of the first kind,  $I_\nu(z)$ , and the modified Bessel functions of the second kind,  $K_\nu(z)$ :

$$w(z) = C_1 I_\nu(z) + C_2 K_\nu(z)$$

This formula is the integral representation of the modified Bessel functions of the first kind:

$$I_\nu(z) = \frac{(z/2)^\nu}{\sqrt{\pi} \Gamma(\nu + 1/2)} \int_0^\pi e^{z \cos(t)} \sin(t)^{2\nu} dt$$

## Tips

- Calling `besseli` for a number that is not a symbolic object invokes the MATLAB `besseli` function.
- At least one input argument must be a scalar or both arguments must be vectors or matrices of the same size. If one input argument is a scalar and the other one is a vector or a matrix, `besseli(nu, z)` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

## Version History

Introduced in R2014a

## References

- [1] Olver, F. W. J. "Bessel Functions of Integer Order." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.
- [2] Antosiewicz, H. A. "Bessel Functions of Fractional Order." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

`airy` | `besselh` | `besselj` | `besselk` | `bessely`

# besselj

Bessel function of the first kind for symbolic expressions

## Syntax

`besselj(nu, z)`

## Description

`besselj(nu, z)` returns the Bessel function of the first kind on page 6-120,  $J_\nu(z)$ .

## Examples

### Find Bessel Function of First Kind

Compute the Bessel functions of the first kind for these numbers. Because these numbers are floating point, you get floating-point results.

```
[besselj(0,5) besselj(-1,2) besselj(1/3,7/4) besselj(1,3/2+2*i)]
```

```
ans =
-0.1776 + 0.0000i -0.5767 + 0.0000i 0.5496 + 0.0000i 1.6113 + 0.3982i
```

Compute the Bessel functions of the first kind for the numbers converted to symbolic form. For most symbolic (exact) numbers, `besselj` returns unresolved symbolic calls.

```
[besselj(sym(0),5) besselj(sym(-1),2)...
 besselj(1/3,sym(7/4)) besselj(sym(1),3/2+2*i)]
```

```
ans =
[ besselj(0, 5), -besselj(1, 2), besselj(1/3, 7/4), besselj(1, 3/2 + 2i)]
```

For symbolic variables and expressions, `besselj` also returns unresolved symbolic calls.

```
syms x y
[besselj(x,y) besselj(1,x^2) besselj(2,x-y) besselj(x^2,x*y)]
```

```
ans =
[ besselj(x, y), besselj(1, x^2), besselj(2, x - y), besselj(x^2, x*y)]
```

### Solve Bessel Differential Equation for Bessel Functions

Solve this second-order differential equation. The solutions are the Bessel functions of the first and the second kind.

```
syms nu w(z)
ode = z^2*diff(w,2) + z*diff(w) +(z^2-nu^2)*w == 0;
dsolve(ode)
```

```
ans =
C2*besselj(nu, z) + C3*bessely(nu, z)
```

Verify that the Bessel function of the first kind is a valid solution of the Bessel differential equation.

```
cond = subs(ode,w,besselj(nu,z));
isAlways(cond)
```

```
ans =
    logical
     1
```

### Special Values of Bessel Function of First Kind

Show that if the first parameter is an odd integer multiplied by 1/2, `besselj` rewrites the Bessel functions in terms of elementary functions.

```
syms x
besselj(1/2,x)
```

```
ans =
(2^(1/2)*sin(x))/(x^(1/2)*pi^(1/2))
```

```
besselj(-1/2,x)
```

```
ans =
(2^(1/2)*cos(x))/(x^(1/2)*pi^(1/2))
```

```
besselj(-3/2,x)
```

```
ans =
-(2^(1/2)*(sin(x) + cos(x)/x))/(x^(1/2)*pi^(1/2))
```

```
besselj(5/2,x)
```

```
ans =
-(2^(1/2)*((3*cos(x))/x - sin(x)*(3/x^2 - 1)))/(x^(1/2)*pi^(1/2))
```

### Differentiate Bessel Function of First Kind

Differentiate expressions involving the Bessel functions of the first kind.

```
syms x y
diff(besselj(1,x))
```

```
ans =
besselj(0, x) - besselj(1, x)/x
```

```
diff(diff(besselj(0,x^2+x*y-y^2), x), y)
```

```
ans =
- besselj(1, x^2 + x*y - y^2) - ...
(2*x + y)*(besselj(0, x^2 + x*y - y^2)*(x - 2*y) - ...
(besselj(1, x^2 + x*y - y^2)*(x - 2*y))/(x^2 + x*y - y^2))
```

### Find Bessel Function for Matrix Input

Call `besselj` for the matrix `A` and the value 1/2. `besselj` acts element-wise to return matrix of Bessel functions.

```
syms x
A = [-1, pi; x, 0];
besselj(1/2, A)
```

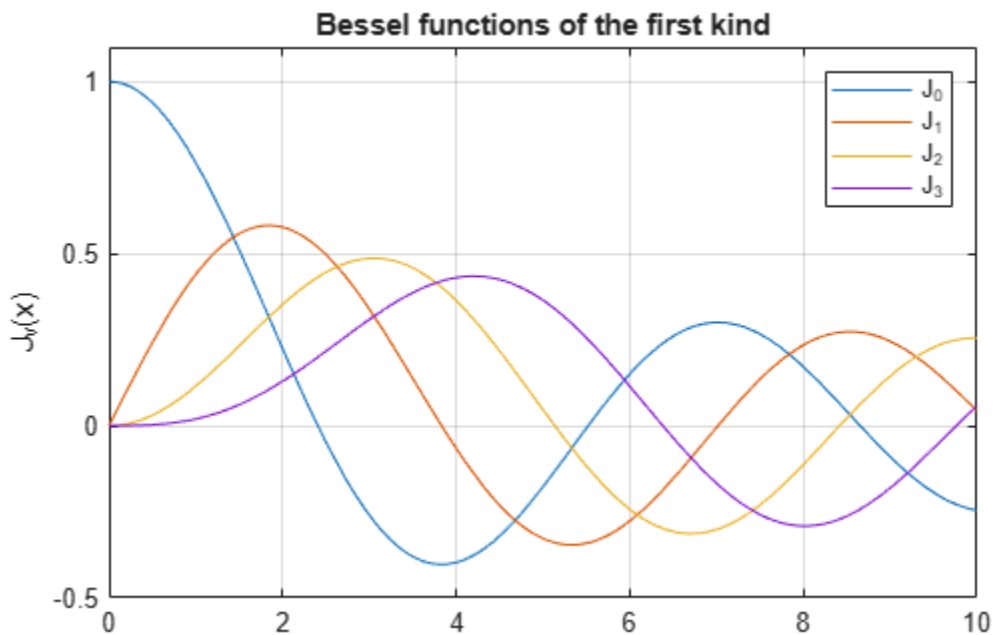
```
ans =
[ (2^(1/2)*sin(1)*1i)/pi^(1/2), 0]
[ (2^(1/2)*sin(x))/(x^(1/2)*pi^(1/2)), 0]
```

## Plot Bessel Functions of First Kind

Plot the Bessel functions of the first kind for 0, 1, 2, 3.

```
syms x y
fplot(besselj(0:3, x))
axis([0 10 -0.5 1.1])
grid on

ylabel('J_v(x)')
legend('J_0','J_1','J_2','J_3', 'Location','Best')
title('Bessel functions of the first kind')
```



## Input Arguments

### nu — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

If nu is a vector or matrix, `besselj` returns the modified Bessel function of the first kind for each element of nu.

**z — Input**

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

If `nu` is a vector or matrix, `besselj` returns the modified Bessel function of the first kind for each element of `nu`.

**More About****Bessel Functions of the First Kind**

The Bessel functions are solutions of the Bessel differential equation.

$$z^2 \frac{d^2 w}{dz^2} + z \frac{dw}{dz} + (z^2 - \nu^2)w = 0$$

These solutions are the Bessel functions of the first kind,  $J_\nu(z)$ , and the Bessel functions of the second kind,  $Y_\nu(z)$ .

$$w(z) = C_1 J_\nu(z) + C_2 Y_\nu(z)$$

This formula is the integral representation of the Bessel functions of the first kind.

$$J_\nu(z) = \frac{(z/2)^\nu}{\sqrt{\pi} \Gamma(\nu + 1/2)} \int_0^\pi \cos(z \cos(t)) \sin(t)^{2\nu} dt$$

**Tips**

- Calling `besselj` for a number that is not a symbolic object invokes the MATLAB `besselj` function.
- At least one input argument must be a scalar or both arguments must be vectors or matrices of the same size. If one input argument is a scalar and the other one is a vector or a matrix, `besselj(nu, z)` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

**Version History**

Introduced in R2014a

**References**

- [1] Olver, F. W. J. "Bessel Functions of Integer Order." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.
- [2] Antosiewicz, H. A. "Bessel Functions of Fractional Order." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.



**See Also**

[airy](#) | [besselh](#) | [besseli](#) | [besselk](#) | [bessely](#)

## besselk

Modified Bessel function of the second kind for symbolic expressions

### Syntax

`besselk(nu, z)`

### Description

`besselk(nu, z)` returns the modified Bessel function of the second kind on page 6-125,  $K_\nu(z)$ .

### Examples

#### Find Modified Bessel Function of Second Kind

Compute the modified Bessel functions of the second kind for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
[besselk(0, 5), besselk(-1, 2), besselk(1/3, 7/4), ...
    besselk(1, 3/2 + 2*i)]
```

```
ans =
    0.0037 + 0.0000i    0.1399 + 0.0000i    0.1594 + 0.0000i   -0.1620 - 0.1066i
```

Compute the modified Bessel functions of the second kind for the numbers converted to symbolic objects. For most symbolic (exact) numbers, `besselk` returns unresolved symbolic calls.

```
[besselk(sym(0), 5), besselk(sym(-1), 2), ...
    besselk(1/3, sym(7/4)), besselk(sym(1), 3/2 + 2*i)]
```

```
ans =
[ besselk(0, 5), besselk(1, 2), besselk(1/3, 7/4), besselk(1, 3/2 + 2i)]
```

For symbolic variables and expressions, `besselk` also returns unresolved symbolic calls:

```
syms x y
[besselk(x, y), besselk(1, x^2), besselk(2, x - y), besselk(x^2, x*y)]
```

```
ans =
[ besselk(x, y), besselk(1, x^2), besselk(2, x - y), besselk(x^2, x*y)]
```

#### Special Values of Modified Bessel Function of Second Kind

If the first parameter is an odd integer multiplied by  $1/2$ , `besselk` rewrites the Bessel functions in terms of elementary functions:

```
syms x
besselk(1/2, x)
```

```
ans =
(2^(1/2)*pi^(1/2)*exp(-x))/(2*x^(1/2))
```

```
besselk(-1/2, x)
```

```

ans =
(2^(1/2)*pi^(1/2)*exp(-x))/(2*x^(1/2))

besselk(-3/2, x)

ans =
(2^(1/2)*pi^(1/2)*exp(-x)*(1/x + 1))/(2*x^(1/2))

besselk(5/2, x)

ans =
(2^(1/2)*pi^(1/2)*exp(-x)*(3/x + 3/x^2 + 1))/(2*x^(1/2))

```

### Solve Bessel Differential Equation for Bessel Functions

Solve this second-order differential equation. The solutions are the modified Bessel functions of the first and the second kind.

```

syms nu w(z)
dsolve(z^2*diff(w, 2) + z*diff(w) - (z^2 + nu^2)*w == 0)

ans =
C2*besseli(nu, z) + C3*besselk(nu, z)

```

Verify that the modified Bessel function of the second kind is a valid solution of the modified Bessel differential equation:

```

syms nu z
isAlways(z^2*diff(besselk(nu, z), z, 2) + z*diff(besselk(nu, z), z)...
- (z^2 + nu^2)*besselk(nu, z) == 0)

ans =
    logical
     1

```

### Differentiate Modified Bessel Function of Second Kind

Differentiate the expressions involving the modified Bessel functions of the second kind:

```

syms x y
diff(besselk(1, x))
diff(diff(besselk(0, x^2 + x*y - y^2), x), y)

ans =
- besselk(1, x)/x - besselk(0, x)

ans =
(2*x + y)*(besselk(0, x^2 + x*y - y^2)*(x - 2*y) + ...
(besselk(1, x^2 + x*y - y^2)*(x - 2*y))/(x^2 + x*y - y^2)) - ...
besselk(1, x^2 + x*y - y^2)

```

### Find Bessel Function for Matrix Input

Call `besselk` for the matrix `A` and the value `1/2`. The result is a matrix of the modified Bessel functions `besselk(1/2, A(i,j))`.

```

syms x
A = [-1, pi; x, 0];
besselk(1/2, A)

```

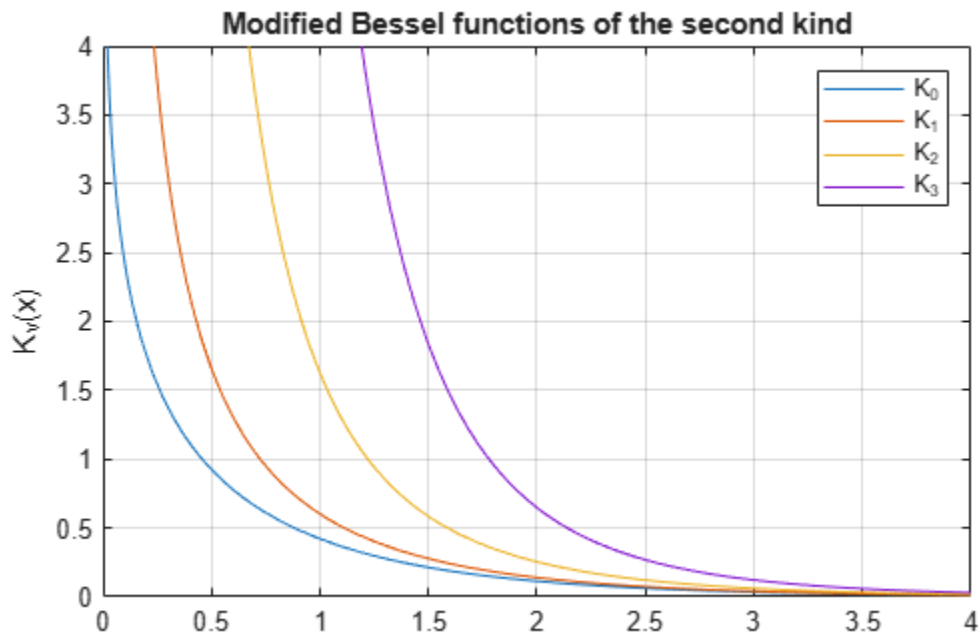
```
ans =
[ -(2^(1/2)*pi^(1/2)*exp(1)*1i)/2, (2^(1/2)*exp(-pi))/2]
[ (2^(1/2)*pi^(1/2)*exp(-x))/(2*x^(1/2)), Inf]
```

### Plot Modified Bessel Functions of Second Kind

Plot the modified Bessel functions of the second kind for  $\nu = 0, 1, 2, 3$ .

```
syms x y
fplot(besselk(0:3, x))
axis([0 4 0 4])
grid on

ylabel('K_v(x)')
legend('K_0','K_1','K_2','K_3', 'Location','Best')
title('Modified Bessel functions of the second kind')
```



### Input Arguments

#### nu — Input

number | vector | matrix | multidimensional array | symbolic number | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, array, or a symbolic number, variable, expression, function, or array. If `nu` is a vector or matrix, `besselk` returns the modified Bessel function of the first kind for each element of `nu`.

#### z — Input

number | vector | matrix | multidimensional array | symbolic number | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, array, or a symbolic number, variable, expression, function, or array. If `nu` is a vector or matrix, `besselk` returns the modified Bessel function of the first kind for each element of `nu`.

## More About

### Modified Bessel Functions of the Second Kind

The modified Bessel differential equation

$$z^2 \frac{d^2 w}{dz^2} + z \frac{dw}{dz} - (z^2 + \nu^2)w = 0$$

has two linearly independent solutions. These solutions are represented by the modified Bessel functions of the first kind,  $I_\nu(z)$ , and the modified Bessel functions of the second kind,  $K_\nu(z)$ :

$$w(z) = C_1 I_\nu(z) + C_2 K_\nu(z)$$

The modified Bessel functions of the second kind are defined via the modified Bessel functions of the first kind:

$$K_\nu(z) = \frac{\pi/2}{\sin(\nu\pi)} (I_{-\nu}(z) - I_\nu(z))$$

Here  $I_\nu(z)$  are the modified Bessel functions of the first kind:

$$I_\nu(z) = \frac{(z/2)^\nu}{\sqrt{\pi}\Gamma(\nu + 1/2)} \int_0^\pi e^{z\cos(t)} \sin(t)^{2\nu} dt$$

## Tips

- Calling `besselk` for a number that is not a symbolic object invokes the MATLAB `besselk` function.
- At least one input argument must be a scalar or both arguments must be vectors or matrices of the same size. If one input argument is a scalar and the other one is a vector or a matrix, `besselk(nu, z)` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

## Version History

Introduced in R2014a

## References

- [1] Olver, F. W. J. "Bessel Functions of Integer Order." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.
- [2] Antosiewicz, H. A. "Bessel Functions of Fractional Order." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

**See Also**

`airy` | `besselh` | `besseli` | `besselj` | `bessely`

# bessely

Bessel function of the second kind for symbolic expressions

## Syntax

`bessely(nu, z)`

## Description

`bessely(nu, z)` returns the Bessel function of the second kind on page 6-130,  $Y_\nu(z)$ .

## Examples

### Find Bessel Function of Second Kind

Compute the Bessel functions of the second kind for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
[bessely(0, 5), bessely(-1, 2), bessely(1/3, 7/4), bessely(1, 3/2 + 2*i)]
```

```
ans =  
-0.3085 + 0.0000i  0.1070 + 0.0000i  0.2358 + 0.0000i  -0.4706 + 1.5873i
```

Compute the Bessel functions of the second kind for the numbers converted to symbolic objects. For most symbolic (exact) numbers, `bessely` returns unresolved symbolic calls.

```
[bessely(sym(0), 5), bessely(sym(-1), 2), ...  
bessely(1/3, sym(7/4)), bessely(sym(1), 3/2 + 2*i)]
```

```
ans =  
[ bessely(0, 5), -bessely(1, 2), bessely(1/3, 7/4), bessely(1, 3/2 + 2i)]
```

For symbolic variables and expressions, `bessely` also returns unresolved symbolic calls:

```
syms x y  
[bessely(x, y), bessely(1, x^2), bessely(2, x - y), bessely(x^2, x*y)]
```

```
ans =  
[ bessely(x, y), bessely(1, x^2), bessely(2, x - y), bessely(x^2, x*y)]
```

### Solve Bessel Differential Equation for Bessel Functions

Solve this second-order differential equation. The solutions are the Bessel functions of the first and the second kind.

```
syms nu w(z)  
dsolve(z^2*diff(w, 2) + z*diff(w) + (z^2 - nu^2)*w == 0)
```

```
ans =  
C2*besselj(nu, z) + C3*bessely(nu, z)
```

Verify that the Bessel function of the second kind is a valid solution of the Bessel differential equation:

```
syms nu z
isAlways(z^2*diff(bessely(nu, z), z, 2) + z*diff(bessely(nu, z), z)...
+ (z^2 - nu^2)*bessely(nu, z) == 0)

ans =
    logical
     1
```

### Special Values of Bessel Function of Second Kind

If the first parameter is an odd integer multiplied by 1/2, `bessely` rewrites the Bessel functions in terms of elementary functions:

```
syms x
bessely(1/2, x)

ans =
-(2^(1/2)*cos(x))/(x^(1/2)*pi^(1/2))

bessely(-1/2, x)

ans =
(2^(1/2)*sin(x))/(x^(1/2)*pi^(1/2))

bessely(-3/2, x)

ans =
(2^(1/2)*(cos(x) - sin(x)/x))/(x^(1/2)*pi^(1/2))

bessely(5/2, x)

ans =
-(2^(1/2)*((3*sin(x))/x + cos(x)*(3/x^2 - 1)))/(x^(1/2)*pi^(1/2))
```

### Differentiate Bessel Functions of Second Kind

Differentiate the expressions involving the Bessel functions of the second kind:

```
syms x y
diff(bessely(1, x))
diff(diff(bessely(0, x^2 + x*y - y^2), x), y)

ans =
bessely(0, x) - bessely(1, x)/x

ans =
- bessely(1, x^2 + x*y - y^2) - ...
(2*x + y)*(bessely(0, x^2 + x*y - y^2)*(x - 2*y) - ...
(bessely(1, x^2 + x*y - y^2)*(x - 2*y))/(x^2 + x*y - y^2))
```

### Find Bessel Function for Matrix Input

Call `bessely` for the matrix `A` and the value 1/2. The result is a matrix of the Bessel functions `bessely(1/2, A(i,j))`.

```
syms x
A = [-1, pi; x, 0];
bessely(1/2, A)
```



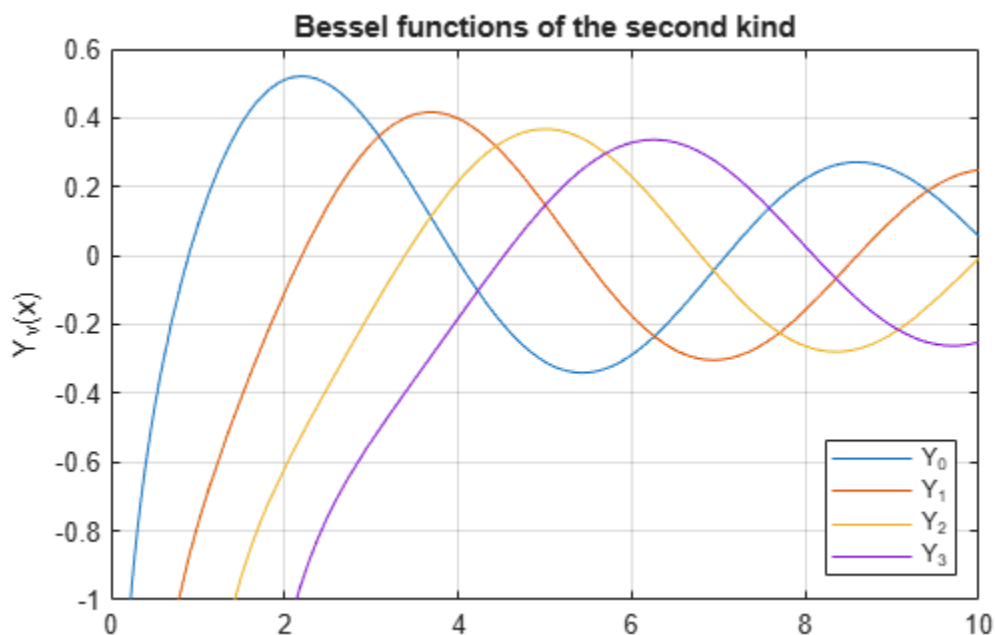
```
ans =
[ (2^(1/2)*cos(1)*1i)/pi^(1/2), 2^(1/2)/pi]
[ -(2^(1/2)*cos(x))/(x^(1/2)*pi^(1/2)),      Inf]
```

## Plot Bessel Functions of Second Kind

Plot the Bessel functions of the second kind for  $\nu = 0, 1, 2, 3$ .

```
syms x y
fplot(bessely(0:3,x))
axis([0 10 -1 0.6])
grid on

ylabel('Y_v(x)')
legend('Y_0','Y_1','Y_2','Y_3', 'Location','Best')
title('Bessel functions of the second kind')
```



## Input Arguments

### nu — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

If  $\nu$  is a vector or matrix, `bessely` returns the Bessel function of the second kind for each element of  $\nu$ .

**z — Input**

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

If  $z$  is a vector or matrix, `bessely` returns the Bessel function of the second kind for each element of  $z$ .

**More About****Bessel Function of the Second Kind**

The Bessel differential equation

$$z^2 \frac{d^2 w}{dz^2} + z \frac{dw}{dz} + (z^2 - \nu^2)w = 0$$

has two linearly independent solutions. These solutions are represented by the Bessel functions of the first kind,  $J_\nu(z)$ , and the Bessel functions of the second kind,  $Y_\nu(z)$ :

$$w(z) = C_1 J_\nu(z) + C_2 Y_\nu(z)$$

The Bessel functions of the second kind are defined via the Bessel functions of the first kind:

$$Y_\nu(z) = \frac{J_\nu(z)\cos(\nu\pi) - J_{-\nu}(z)}{\sin(\nu\pi)}$$

Here  $J_\nu(z)$  are the Bessel function of the first kind:

$$J_\nu(z) = \frac{(z/2)^\nu}{\sqrt{\pi}\Gamma(\nu + 1/2)} \int_0^\pi \cos(z\cos(t))\sin(t)^{2\nu} dt$$

**Tips**

- Calling `bessely` for a number that is not a symbolic object invokes the MATLAB `bessely` function.

At least one input argument must be a scalar or both arguments must be vectors or matrices of the same size. If one input argument is a scalar and the other one is a vector or a matrix, `bessely(nu, z)` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

**Version History**

Introduced in R2014a

## References

- [1] Olver, F. W. J. "Bessel Functions of Integer Order." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.
- [2] Antosiewicz, H. A. "Bessel Functions of Fractional Order." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

airy | `besselh` | `besseli` | `besselj` | `besselk`

## beta

Beta function

### Syntax

`beta(x,y)`

### Description

`beta(x,y)` returns the beta function on page 6-134 of  $x$  and  $y$ .

### Examples

#### Compute Beta Function for Numeric Inputs

Compute the beta function for these numbers. Because these numbers are not symbolic objects, you get floating-point results:

```
[beta(1, 5), beta(3, sqrt(2)), beta(pi, exp(1)), beta(0, 1)]
```

```
ans =  
    0.2000    0.1716    0.0379    Inf
```

#### Compute Beta Function for Symbolic Inputs

Compute the beta function for the numbers converted to symbolic objects:

```
[beta(sym(1), 5), beta(3, sym(2)), beta(sym(4), sym(4))]
```

```
ans =  
[ 1/5, 1/12, 1/140]
```

If one or both parameters are complex numbers, convert these numbers to symbolic objects:

```
[beta(sym(i), 3/2), beta(sym(i), i), beta(sym(i + 2), 1 - i)]
```

```
ans =  
[ (pi^(1/2)*gamma(1i))/(2*gamma(3/2 + 1i)), gamma(1i)^2/gamma(2i),...  
  (pi*(1/2 + 1i/2))/sinh(pi)]
```

#### Compute Beta Function for Negative Parameters

Compute the beta function for negative parameters. If one or both arguments are negative numbers, convert these numbers to symbolic objects:

```
[beta(sym(-3), 2), beta(sym(-1/3), 2), beta(sym(-3), 4), beta(sym(-3), -2)]
```

```
ans =  
[ 1/6, -9/2, Inf, Inf]
```

#### Compute Beta Function for Matrix Inputs

Call `beta` for the matrix `A` and the value 1. The result is a matrix of the beta functions `beta(A(i,j),1)`:

```
A = sym([1 2; 3 4]);
beta(A,1)
```

```
ans =
[ 1, 1/2]
[ 1/3, 1/4]
```

### Differentiate Beta Function

Differentiate the beta function, then substitute the variable  $t$  with the value  $2/3$  and approximate the result using `vpa`:

```
syms t
u = diff(beta(t^2 + 1, t))
vpa(subs(u, t, 2/3), 10)

u =
beta(t, t^2 + 1)*(psi(t) + 2*t*psi(t^2 + 1) - ...
psi(t^2 + t + 1)*(2*t + 1))

ans =
-2.836889094
```

### Expand Beta Function

Expand these beta functions:

```
syms x y
expand(beta(x, y))
expand(beta(x + 1, y - 1))

ans =
(gamma(x)*gamma(y))/gamma(x + y)

ans =
-(x*gamma(x)*gamma(y))/(gamma(x + y) - y*gamma(x + y))
```

## Input Arguments

### **x** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

If  $x$  is a vector or matrix, `beta` returns the beta function for each element of  $x$ .

### **y** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

If  $y$  is a vector or matrix, `beta` returns the beta function for each element of  $y$ .

## More About

### Beta Function

This integral defines the beta function:

$$B(x, y) = \int_0^1 t^{x-1} (1-t)^{y-1} dt = \frac{\Gamma(x)\Gamma(y)}{\Gamma(x+y)}$$

### Tips

- The beta function is uniquely defined for positive numbers and complex numbers with positive real parts. It is approximated for other numbers.
- Calling `beta` for numbers that are not symbolic objects invokes the MATLAB `beta` function. This function accepts real arguments only. If you want to compute the beta function for complex numbers, use `sym` to convert the numbers to symbolic objects, and then call `beta` for those symbolic objects.
- If one or both parameters are negative numbers, convert these numbers to symbolic objects using `sym`, and then call `beta` for those symbolic objects.
- If the beta function has a singularity, `beta` returns the positive infinity `Inf`.
- `beta(sym(0), 0)`, `beta(0, sym(0))`, and `beta(sym(0), sym(0))` return `NaN`.
- `beta(x, y) = beta(y, x)` and `beta(x, A) = beta(A, x)`.
- At least one input argument must be a scalar or both arguments must be vectors or matrices of the same size. If one input argument is a scalar and the other one is a vector or a matrix, `beta(x, y)` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

## Version History

Introduced in R2014a

### References

- [1] Zelen, M. and N. C. Severo. "Probability Functions." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

### See Also

`gamma` | `factorial` | `nchoosek` | `psi`

# cat

Concatenate symbolic arrays along specified dimension

## Syntax

```
cat(dim,A1,...,AN)
```

## Description

`cat(dim,A1,...,AN)` concatenates the arrays  $A_1, \dots, A_N$  along dimension `dim`. The remaining dimensions must be the same size.

## Examples

### Concatenate Two Vectors into Matrix

Create vectors  $A$  and  $B$ .

```
A = sym('a%d',[1 4])
B = sym('b%d',[1 4])
```

```
A =
[ a1, a2, a3, a4]
B =
[ b1, b2, b3, b4]
```

To concatenate  $A$  and  $B$  into a matrix, specify dimension `dim` as 1.

```
cat(1,A,B)
```

```
ans =
[ a1, a2, a3, a4]
[ b1, b2, b3, b4]
```

Alternatively, use the syntax `[A;B]`.

```
[A;B]
```

```
ans =
[ a1, a2, a3, a4]
[ b1, b2, b3, b4]
```

### Concatenate Two Vectors into One Vector

To concatenate two vectors into one vector, specify dimension `dim` as 2.

```
A = sym('a%d',[1 4]);
B = sym('b%d',[1 4]);
cat(2,A,B)
```

```
ans =
[ a1, a2, a3, a4, b1, b2, b3, b4]
```

Alternatively, use the syntax `[A B]`.

```
[A B]
```

```
ans =  
[ a1, a2, a3, a4, b1, b2, b3, b4]
```

### Concatenate Multidimensional Arrays Along Their Third Dimension

Create arrays A and B.

```
A = sym('a%d%d',[2 2]);  
A(:,:,2) = -A  
B = sym('b%d%d',[2 2]);  
B(:,:,2) = -B
```

```
A(:,:,1) =  
[ a11, a12]  
[ a21, a22]  
A(:,:,2) =  
[ -a11, -a12]  
[ -a21, -a22]
```

```
B(:,:,1) =  
[ b11, b12]  
[ b21, b22]  
B(:,:,2) =  
[ -b11, -b12]  
[ -b21, -b22]
```

Concatenate A and B by specifying dimension `dim` as 3.

```
cat(3,A,B)
```

```
ans(:,:,1) =  
[ a11, a12]  
[ a21, a22]  
ans(:,:,2) =  
[ -a11, -a12]  
[ -a21, -a22]  
ans(:,:,3) =  
[ b11, b12]  
[ b21, b22]  
ans(:,:,4) =  
[ -b11, -b12]  
[ -b21, -b22]
```

## Input Arguments

### **dim** — Dimension to concatenate arrays along

positive integer

Dimension to concatenate arrays along, specified as a positive integer.

### **A1, ..., AN** — Input arrays

symbolic variables | symbolic vectors | symbolic matrices | symbolic multidimensional arrays

Input arrays, specified as symbolic variables, vectors, matrices, or multidimensional arrays.



## Version History

Introduced in R2010b

### **R2022a: Concatenate symbolic matrix functions**

The `cat` function accepts input arguments of type `symfunmatrix`.

### **R2021a: Concatenate symbolic matrix variables**

The `cat` function accepts input arguments of type `symmatrix`.

## See Also

`horzcat` | `reshape` | `vertcat`

## catalan

Catalan constant

### Syntax

`catalan`

### Description

`catalan` represents the Catalan constant on page 6-138. To get a floating-point approximation with the current precision set by `digits`, use `vpa(catalan)`.

### Examples

#### Approximate Catalan Constant

Find a floating-point approximation of the Catalan constant with the default number of digits and with the 10-digit precision.

Use `vpa` to approximate the Catalan constant with the default 32-digit precision:

```
vpa(catalan)

ans =
0.91596559417721901505460351493238
```

Set the number of digits to 10 and approximate the Catalan constant:

```
old = digits(10);
vpa(catalan)

ans =
0.9159655942
```

Restore the default number of digits:

```
digits(old)
```

### More About

#### Catalan Constant

The Catalan constant is defined as follows:

$$\text{catalan} = \sum_{i=0}^{\infty} \frac{(-1)^i}{(2i+1)^2} = \frac{1}{1^2} - \frac{1}{3^2} + \frac{1}{5^2} - \frac{1}{7^2} + \dots$$

## Version History

Introduced in R2014a

**See Also**

dilog | eulergamma

## ccode

C code representation of symbolic expression

### Syntax

```
ccode(f)
ccode(f, Name, Value)
```

### Description

`ccode(f)` returns C code for the symbolic expression `f`.

`ccode(f, Name, Value)` uses additional options specified by one or more `Name, Value` pair arguments.

### Examples

#### Generate C Code from Symbolic Expression

Generate C code from the symbolic expression  $\log(1+x)$ .

```
syms x
f = log(1+x);
ccode(f)

ans =
'    t0 = log(x+1.0);'
```

Generate C code for the 3-by-3 Hilbert matrix.

```
H = sym(hilb(3));
ccode(H)

ans =
'    H[0][0] = 1.0;
    H[0][1] = 1.0/2.0;
    H[0][2] = 1.0/3.0;
    H[1][0] = 1.0/2.0;
    H[1][1] = 1.0/3.0;
    H[1][2] = 1.0/4.0;
    H[2][0] = 1.0/3.0;
    H[2][1] = 1.0/4.0;
    H[2][2] = 1.0/5.0;'
```

#### Initialize Arrays Efficiently

Because generated C code initializes only non-zero elements, you can efficiently initialize arrays by setting all elements to 0 directly in your C code. Then, use the generated C code to initialize only nonzero elements. This approach enables efficient initialization of matrices, especially sparse matrices.

Initialize the 3-by-3 identity matrix. First initialize the matrix with all elements set to 0 in your C code. Then use the generated C code to initialize the nonzero values.

```
I3 = sym(eye(3));
I3code = ccode(I3)

I3code =
    '  I3[0][0] = 1.0;
      I3[1][1] = 1.0;
      I3[2][2] = 1.0;'
```

## Write Optimized C Code to File with Comments

Write C code to the file `ccodetest.c` by specifying the `File` option. When writing to a file, `ccode` optimizes the code by using intermediate variables named `t0`, `t1`, and so on.

```
syms x
f = diff(tan(x));
ccode(f,'File','ccodetest.c')

    t0 = pow(tan(x),2.0)+1.0;
```

Include the comment `Version: 1.1` in the file by using the `Comments` option. `ccode` uses block comments.

```
ccode(f,'File','ccodetest.c','Comments','Version: 1.1')

/*
Version: 1.1
*/
    t0 = pow(tan(x),2.0)+1.0;
```

## Input Arguments

### f — Symbolic input

symbolic expression

Symbolic input, specified as a symbolic expression.

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `ccode(x^2,'File','ccode.c','Comments','V1.2')`

### File — File to write to

character vector | string

File to write to, specified as a character vector or string. When writing to a file, `ccode` optimizes the code by using intermediate variables named `t0`, `t1`, and so on.

**Comments — Comments to include in file header**

character vector | cell array of character vectors | string vector

Comments to include in the file header, specified as a character vector, cell array of character vectors, or string vector. Because `ccode` uses block comments, the comments must not contain `/*` or `*/`.

**Tips**

- To generate optimized C or C++ code from a symbolic expression, especially for a large expression, you can use the MATLAB Coder app instead of using the `ccode` function. This way, the generated code is better integrated into the MATLAB ecosystem. First, convert the symbolic expression to a deployable MATLAB function using `matlabFunction`. Then, generate C or C++ code from the MATLAB function using the MATLAB Coder app. For an example, see “Generate C Code from Symbolic Expressions Using the MATLAB Coder App” on page 5-15.

**Version History****Introduced before R2006a****See Also**

fortran | latex | matlabFunction

# cell2sym

Convert cell array to symbolic array

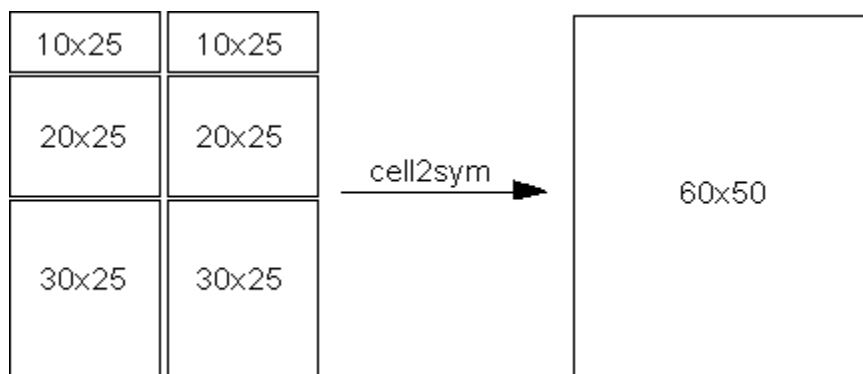
## Syntax

```
S = cell2sym(C)
S = cell2sym(C,flag)
```

## Description

`S = cell2sym(C)` converts a cell array `C` to a symbolic array `S`. The elements of `C` must be convertible to symbolic objects.

If each element of the input cell array `C` is a scalar, then `size(S) = size(C)`, and `S(k) = sym(C(k))` for all indices `k`. If the cell array `C` contains nonscalar elements, then the contents of `C` must support concatenation into an N-dimensional rectangle. Otherwise, the results are undefined. For example, the contents of cells in the same column must have the same number of columns. However, they do not need to have the same number of rows. See figure.



`S = cell2sym(C,flag)` uses the technique specified by `flag` for converting floating-point numbers to symbolic numbers.

## Examples

### Convert Cell Array of Scalars

Convert a cell array of only scalar elements to a symbolic array.

Create a cell array of scalar elements.

```
C = {'x', 'y', 'z'; 1 2 3}
```

```
C=2x3 cell array
    {'x'}    {'y'}    {'z'}
    {[1]}    {[2]}    {[3]}
```

Convert this cell array to a symbolic array.

```
S = cell2sym(C)
```

```
S =  
 $\begin{pmatrix} x & y & z \\ 1 & 2 & 3 \end{pmatrix}$ 
```

`cell2sym` does not create symbolic variables `x`, `y`, and `z` in the MATLAB® workspace. To access an element of `S`, use parentheses.

```
S(1,1)
```

```
ans = x
```

### Convert Cell Array Containing Nonscalar Elements

Convert a cell array whose elements are scalars, vectors, and matrices into a symbolic array. Such conversion is possible only if the contents of the cell array can be concatenated into an N-dimensional rectangle.

Create a cell array, the elements of which are a scalar, a row vector, a column vector, and a matrix.

```
C = {'x' [2 3 4]; ['y'; sym(9)] [6 7 8; 10 11 12]}
```

```
C=2x2 cell array  
{'x'      }      {[ 2 3 4]}  
{2x1 sym}      {2x3 double}
```

Convert this cell array to a symbolic array.

```
S = cell2sym(C)
```

```
S =  
 $\begin{pmatrix} x & 2 & 3 & 4 \\ y & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{pmatrix}$ 
```

### Choose Conversion Technique for Floating-Point Values

When converting a cell array containing floating-point numbers, you can explicitly specify the conversion technique.

Create a cell array `pi` with two elements: the double-precision value of the constant `pi` and the exact value `pi`.

```
C = {pi,sym(pi)}
```

```
C=1x2 cell array  
{[3.1416]}      {[pi]}
```



Convert this cell array to a symbolic array. By default, `cell2sym` uses the rational conversion mode. Thus, results returned by `cell2sym` without a flag are the same as results returned by `cell2sym` with the flag `'r'`.

```
S = cell2sym(C)
```

```
S = ( $\pi$   $\pi$ )
```

```
S = cell2sym(C, 'r')
```

```
S = ( $\pi$   $\pi$ )
```

Convert the same cell array to a symbolic array using the flags `'d'`, `'e'`, and `'f'`. See the “Input Arguments” on page 6-145 section for the details about conversion techniques.

```
S = cell2sym(C, 'd')
```

```
S = (3.1415926535897931159979634685442  $\pi$ )
```

```
S = cell2sym(C, 'e')
```

```
S =
```

$$\left( \pi - \frac{198 \text{ eps}}{359} \pi \right)$$

```
S = cell2sym(C, 'f')
```

```
S =
```

$$\left( \frac{884279719003555}{281474976710656} \pi \right)$$

## Input Arguments

### C — Input cell array

cell array

Input cell array, specified as a cell array. The elements of `C` must be convertible to symbolic objects.

### flag — Conversion technique

`'r'` (default) | `'d'` | `'e'` | `'f'`

Conversion technique, specified as one of the characters listed in this table.

|                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>'r'</code> | In the <i>rational</i> mode, <code>cell2sym</code> converts floating-point numbers obtained by evaluating expressions of the form $p/q$ , $p*\pi/q$ , $\text{sqrt}(p)$ , $2^q$ , and $10^q$ for modest sized integers $p$ and $q$ to the corresponding symbolic form. This approach effectively compensates for the round-off error involved in the original evaluation, but might not represent the floating-point value precisely. If <code>cell2sym</code> cannot find simple rational approximation, then it uses the same technique as it would use with the flag <code>'f'</code> . |
| <code>'d'</code> | In the <i>decimal</i> mode, <code>cell2sym</code> takes the number of digits from the current setting of <code>digits</code> . Conversions with fewer than 16 digits lose some accuracy, while more than 16 digits might not be warranted. For example, <code>cell2sym({4/3}, 'd')</code> with the 10-digit accuracy returns 1.333333333, while with the 20-digit accuracy it returns 1.333333333333332593. The latter does not end in 3s, but it is an accurate decimal representation of the floating-point number nearest to $4/3$ .                                                   |

|     |                                                                                                                                                                                                                                                                                                                                                                                       |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 'e' | In the <i>estimate error</i> mode, <code>cell2sym</code> supplements a result obtained in the rational mode by a term involving the variable <code>eps</code> . This term estimates the difference between the theoretical rational expression and its actual floating-point value. For example, <code>cell2sym({3*pi/4}, 'e')</code> returns $(3\pi)/4 - (103\text{eps})/249$ .      |
| 'f' | In the <i>floating-point</i> mode, <code>cell2sym</code> represents all values in the form $N \cdot 2^e$ or $-N \cdot 2^e$ , where $N \geq 0$ and $e$ are integers. For example, <code>cell2sym({1/10}, 'f')</code> returns $3602879701896397/36028797018963968$ . The returned rational value is the exact value of the floating-point number that you convert to a symbolic number. |

**Output Arguments**

**S — Resulting symbolic array**  
symbolic array

Resulting symbolic array, returned as a symbolic array.

**Version History**  
Introduced in R2016a

**See Also**  
`sym2cell` | `cell2mat` | `mat2cell` | `num2cell`

# changeIntegrationVariable

Integration by substitution

## Syntax

`G = changeIntegrationVariable(F,old,new)`

## Description

`G = changeIntegrationVariable(F,old,new)` applies integration by substitution to the integrals in `F`, in which `old` is replaced by `new`. `old` must depend on the previous integration variable of the integrals in `F` and `new` must depend on the new integration variable. For more information, see “Integration by Substitution” on page 6-150.

When specifying the integrals in `F`, you can return the unevaluated form of the integrals by using the `int` function with the 'Hold' option set to `true`. You can then use `changeIntegrationVariable` to show the steps of integration by substitution.

## Examples

### Change of Variable

Apply a change of variable to the definite integral  $\int_a^b f(x+c) dx$ .

Define the integral.

```
syms f(x) y a b c
F = int(f(x+c),a,b)
```

$$F = \int_a^b f(c+x) dx$$

Change the variable  $x+c$  in the integral to  $y$ .

```
G = changeIntegrationVariable(F,x+c,y)
```

$$G = \int_{a+c}^{b+c} f(y) dy$$

### Integration by Substitution

Find the integral of  $\int \cos(\log(x))dx$  using integration by substitution.

Define the integral without evaluating it by setting the 'Hold' option to `true`.

```
syms x t
F = int(cos(log(x)), 'Hold', true)
```

$$F = \int \cos(\log(x)) \, dx$$

Substitute the expression  $\log(x)$  with  $t$ .

```
G = changeIntegrationVariable(F, log(x), t)
```

$$G = \int e^t \cos(t) \, dt$$

To evaluate the integral in  $G$ , use the `release` function to ignore the 'Hold' option.

```
H = release(G)
```

$$H = \frac{e^t (\cos(t) + \sin(t))}{2}$$

Restore  $\log(x)$  in place of  $t$ .

```
H = simplify(subs(H, t, log(x)))
```

$$H = \frac{\sqrt{2} x \sin\left(\frac{\pi}{4} + \log(x)\right)}{2}$$

Compare the result to the integration result returned by `int` without setting the 'Hold' option to `true`.

```
Fcalc = int(cos(log(x)))
```

$$Fcalc = \frac{\sqrt{2} x \sin\left(\frac{\pi}{4} + \log(x)\right)}{2}$$

### Find Closed-Form Solution

Find the closed-form solution of the integral  $\int x \tan(\log(x)) dx$ .

Define the integral using the `int` function.

```
syms x
F = int(x*tan(log(x)), x)
```

$$F = \int x \tan(\log(x)) \, dx$$

The `int` function cannot find the closed-form solution of the integral.

Substitute the expression  $\log(x)$  with  $t$ . Apply integration by substitution.

```
syms t
G = changeIntegrationVariable(F, log(x), t)
```

$$G = \frac{e^{2t} {}_2F_1(1, -i; 1-i; -e^{2ti}) i}{2} + e^{t(2+2i)} {}_2F_1(1, 1-i; 2-i; -e^{2ti}) \left(-\frac{1}{4} - \frac{1}{4} i\right)$$

The closed-form solution is expressed in terms of hypergeometric functions. For more details on hypergeometric functions, see `hypergeom`.

### Numerical Integration with High Precision

Compute the integral  $\int_0^1 e^{\sqrt{\sin(x)}} dx$  numerically with high precision.

Define the integral  $\int_0^1 e^{\sqrt{\sin(x)}} dx$ . A closed-form solution to the integral does not exist.

```
syms x
F = int(exp(sqrt(sin(x))), x, 0, 1)
```

$$F = \int_0^1 e^{\sqrt{\sin(x)}} dx$$

You can use `vpa` to compute the integral numerically to 10 significant digits.

```
F10 = vpa(F, 10)
F10 = 1.944268879
```

Alternatively, you can use the `vpaintegral` function and specify the relative error tolerance.

```
Fvpa = vpaintegral(exp(sqrt(sin(x))), x, 0, 1, 'RelTol', 1e-10)
Fvpa = 1.944268879
```

The `vpa` function cannot find the numerical integration to 70 significant digits, and it returns the unevaluated integral in the form of `vpaintegral`.

```
F70 = vpa(F, 70)
F70 = 1.944268879138581167466225761060083173280747314051712224507065962575967
```

To find the numerical integration with high precision, you can perform a change of variable. Substitute the expression  $\sqrt{\sin(x)}$  with  $t$ . Compute the integral numerically to 70 significant digits.

```
syms t;
G = changeIntegrationVariable(F, sqrt(sin(x)), t)
```

$$G = \int_0^{\sqrt{\sin(1)}} \frac{2t e^t}{\sqrt{1-t^4}} dt$$

```
G70 = vpa(G, 70)
G70 = 1.944268879138581167466225761060083173280747314051712224507065962575967
```

## Input Arguments

### **F** — Expression containing integrals

symbolic expression | symbolic function | symbolic vector | symbolic matrix

Expression containing integrals, specified as a symbolic expression, function, vector, or matrix.

### **old** — Subexpression to be substituted

symbolic scalar variable | symbolic expression | symbolic function

Subexpression to be substituted, specified as a symbolic scalar variable, expression, or function. **old** must depend on the previous integration variable of the integrals in **F**.

### **new** — New subexpression

symbolic scalar variable | symbolic expression | symbolic function

New subexpression, specified as a symbolic scalar variable, expression, or function. **new** must depend on the new integration variable.

## More About

### Integration by Substitution

Mathematically, the substitution rule is formally defined for indefinite integrals as

$$\int f(g(x)) g'(x) dx = \left( \int f(t) dt \right) \Big|_{t=g(x)}$$

and for definite integrals as

$$\int_a^b f(g(x)) g'(x) dx = \int_{g(a)}^{g(b)} f(t) dt.$$

## Version History

Introduced in R2019b

### See Also

`integrateByParts` | `release` | `int` | `diff` | `vpaintegral`

# charpoly

Characteristic polynomial of matrix

## Syntax

```
charpoly(A)  
charpoly(A,var)
```

## Description

`charpoly(A)` returns a vector of coefficients of the characteristic polynomial on page 6-152 of A. If A is a symbolic matrix, `charpoly` returns a symbolic vector. Otherwise, it returns a vector of double-precision values.

`charpoly(A,var)` returns the characteristic polynomial of A in terms of `var`.

## Examples

### Compute Coefficients of Characteristic Polynomial of Matrix

Compute the coefficients of the characteristic polynomial of A by using `charpoly`.

```
A = [1 1 0; 0 1 0; 0 0 1];  
charpoly(A)
```

```
ans =  
     1     -3      3     -1
```

For symbolic input, `charpoly` returns a symbolic vector instead of double. Repeat the calculation for symbolic input.

```
A = sym(A);  
charpoly(A)
```

```
ans =  
[ 1, -3, 3, -1]
```

### Compute Characteristic Polynomial of Matrix

Compute the characteristic polynomial of the matrix A in terms of `x`.

```
syms x  
A = sym([1 1 0; 0 1 0; 0 0 1]);  
polyA = charpoly(A,x)
```

```
polyA =  
x^3 - 3*x^2 + 3*x - 1
```

Solve the characteristic polynomial for the eigenvalues of A.

```
eigenA = solve(polyA)
eigenA =
1
1
1
```

## Input Arguments

### **A — Input**

numeric matrix | symbolic matrix

Input, specified as a numeric or symbolic matrix.

### **var — Polynomial variable**

symbolic variable

Polynomial variable, specified as a symbolic variable.

## More About

### **Characteristic Polynomial of Matrix**

The characteristic polynomial of an  $n$ -by- $n$  matrix  $A$  is the polynomial  $p_A(x)$ , defined as follows.

$$p_A(x) = \det(xI_n - A)$$

Here,  $I_n$  is the  $n$ -by- $n$  identity matrix.

## Version History

Introduced in R2012b

## References

- [1] Cohen, H. "A Course in Computational Algebraic Number Theory." *Graduate Texts in Mathematics* (Axler, Sheldon and Ribet, Kenneth A., eds.). Vol. 138, Springer, 1993.
- [2] Abdeljaoued, J. "The Berkowitz Algorithm, Maple and Computing the Characteristic Polynomial in an Arbitrary Commutative Ring." *MapleTech*, Vol. 4, Number 3, pp 21-32, Birkhauser, 1997.

## See Also

det | eig | jordan | minpoly | poly2sym | sym2poly



# chebyshevT

Chebyshev polynomials of the first kind

## Syntax

`chebyshevT(n,x)`

## Description

`chebyshevT(n,x)` represents the  $n$ th degree Chebyshev polynomial of the first kind on page 6-155 at the point  $x$ .

## Examples

### First Five Chebyshev Polynomials of the First Kind

Find the first five Chebyshev polynomials of the first kind for the variable  $x$ .

```
syms x
chebyshevT([0, 1, 2, 3, 4], x)

ans =
[ 1, x, 2*x^2 - 1, 4*x^3 - 3*x, 8*x^4 - 8*x^2 + 1]
```

### Chebyshev Polynomials for Numeric and Symbolic Arguments

Depending on its arguments, `chebyshevT` returns floating-point or exact symbolic results.

Find the value of the fifth-degree Chebyshev polynomial of the first kind at these points. Because these numbers are not symbolic objects, `chebyshevT` returns floating-point results.

```
chebyshevT(5, [1/6, 1/4, 1/3, 1/2, 2/3, 3/4])

ans =
    0.7428    0.9531    0.9918    0.5000   -0.4856   -0.8906
```

Find the value of the fifth-degree Chebyshev polynomial of the first kind for the same numbers converted to symbolic objects. For symbolic numbers, `chebyshevT` returns exact symbolic results.

```
chebyshevT(5, sym([1/6, 1/4, 1/3, 1/2, 2/3, 3/4]))

ans =
[ 361/486, 61/64, 241/243, 1/2, -118/243, -57/64]
```

### Evaluate Chebyshev Polynomials with Floating-Point Numbers

Floating-point evaluation of Chebyshev polynomials by direct calls of `chebyshevT` is numerically stable. However, first computing the polynomial using a symbolic variable, and then substituting variable-precision values into this expression can be numerically unstable.

Find the value of the 500th-degree Chebyshev polynomial of the first kind at  $1/3$  and `vpa(1/3)`. Floating-point evaluation is numerically stable.

```
chebyshevT(500, 1/3)
chebyshevT(500, vpa(1/3))
```

```
ans =
    0.9631
```

```
ans =
0.963114126817085233778571286718
```

Now, find the symbolic polynomial  $T_{500} = \text{chebyshevT}(500, x)$ , and substitute  $x = \text{vpa}(1/3)$  into the result. This approach is numerically unstable.

```
syms x
T500 = chebyshevT(500, x);
subs(T500, x, vpa(1/3))
```

```
ans =
-3293905791337500897482813472768.0
```

Approximate the polynomial coefficients by using `vpa`, and then substitute  $x = \text{sym}(1/3)$  into the result. This approach is also numerically unstable.

```
subs(vpa(T500), x, sym(1/3))
```

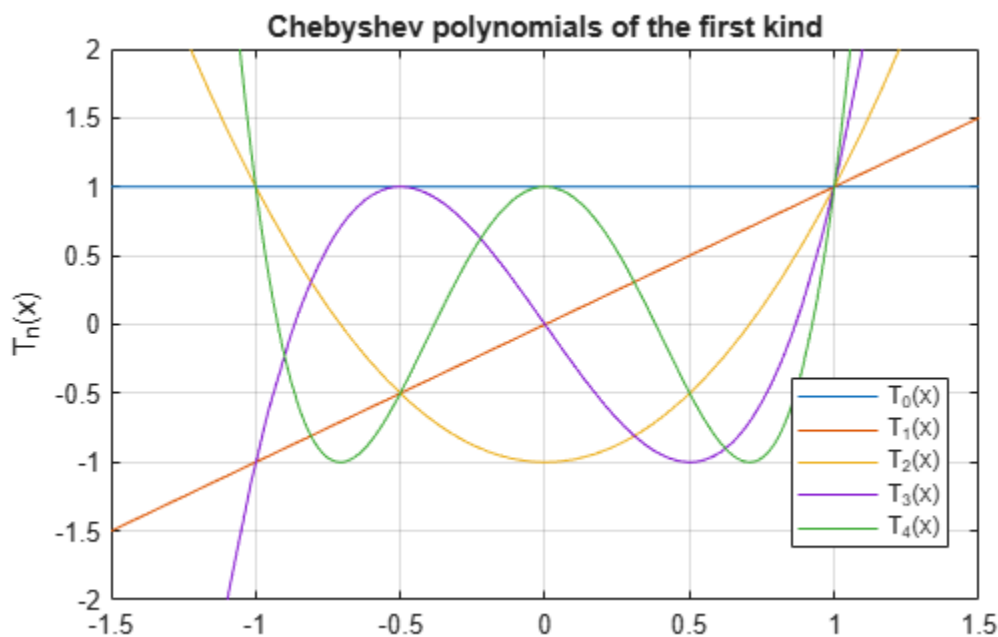
```
ans =
1202292431349342132757038366720.0
```

### Plot Chebyshev Polynomials of the First Kind

Plot the first five Chebyshev polynomials of the first kind.

```
syms x y
fplot(chebyshevT(0:4,x))
axis([-1.5 1.5 -2 2])
grid on

ylabel('T_n(x)')
legend('T_0(x)', 'T_1(x)', 'T_2(x)', 'T_3(x)', 'T_4(x)', 'Location', 'Best')
title('Chebyshev polynomials of the first kind')
```



## Input Arguments

### **n** — Degree of polynomial

nonnegative integer | symbolic variable | symbolic expression | symbolic function | vector | matrix

Degree of the polynomial, specified as a nonnegative integer, symbolic variable, expression, or function, or as a vector or matrix of numbers, symbolic numbers, variables, expressions, or functions.

### **x** — Evaluation point

number | symbolic number | symbolic variable | symbolic expression | symbolic function | vector | matrix

Evaluation point, specified as a number, symbolic number, variable, expression, or function, or as a vector or matrix of numbers, symbolic numbers, variables, expressions, or functions.

## More About

### Chebyshev Polynomials of the First Kind

- Chebyshev polynomials of the first kind are defined as  $T_n(x) = \cos(n \cdot \arccos(x))$ .

These polynomials satisfy the recursion formula

$$T(0, x) = 1, \quad T(1, x) = x, \quad T(n, x) = 2xT(n-1, x) - T(n-2, x)$$

- Chebyshev polynomials of the first kind are orthogonal on the interval  $-1 \leq x \leq 1$  with respect to the weight function  $w(x) = \frac{1}{\sqrt{1-x^2}}$ .

$$\int_{-1}^1 \frac{T(n, x)T(m, x)}{\sqrt{1-x^2}} dx = \begin{cases} 0 & \text{if } n \neq m \\ \pi & \text{if } n = m = 0 \\ \frac{\pi}{2} & \text{if } n = m \neq 0. \end{cases}$$

- Chebyshev polynomials of the first kind are special cases of the Jacobi polynomials

$$T(n, x) = \frac{2^{2n}(n!)^2}{(2n)!} P\left(n, -\frac{1}{2}, -\frac{1}{2}, x\right)$$

and Gegenbauer polynomials

$$T(n, x) = \begin{cases} \frac{1}{2} \lim_{a \rightarrow 0} \frac{n+a}{a} G(n, a, x) & \text{if } n \neq 0 \\ \lim_{a \rightarrow 0} G(0, a, x) = 1 & \text{if } n = 0 \end{cases}$$

## Tips

- `chebyshevT` returns floating-point results for numeric arguments that are not symbolic objects.
- `chebyshevT` acts element-wise on nonscalar inputs.
- At least one input argument must be a scalar or both arguments must be vectors or matrices of the same size. If one input argument is a scalar and the other one is a vector or a matrix, then `chebyshevT` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

## Version History

Introduced in R2014b

## References

- [1] Hochstrasser, U. W. "Orthogonal Polynomials." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.
- [2] Cohl, Howard S., and Connor MacKenzie. "Generalizations and Specializations of Generating Functions for Jacobi, Gegenbauer, Chebyshev and Legendre Polynomials with Definite Integrals." *Journal of Classical Analysis*, no. 1 (2013): 17-33. <https://doi.org/10.7153/jca-03-02>.

## See Also

`chebyshevU` | `gegenbauerC` | `hermiteH` | `jacobiP` | `laguerreL` | `legendreP`

# chebyshevU

Chebyshev polynomials of the second kind

## Syntax

`chebyshevU(n,x)`

## Description

`chebyshevU(n,x)` represents the  $n$ th degree Chebyshev polynomial of the second kind on page 6-159 at the point  $x$ .

## Examples

### First Five Chebyshev Polynomials of the Second Kind

Find the first five Chebyshev polynomials of the second kind for the variable  $x$ .

```
syms x
chebyshevU([0, 1, 2, 3, 4], x)

ans =
[ 1, 2*x, 4*x^2 - 1, 8*x^3 - 4*x, 16*x^4 - 12*x^2 + 1]
```

### Chebyshev Polynomials for Numeric and Symbolic Arguments

Depending on its arguments, `chebyshevU` returns floating-point or exact symbolic results.

Find the value of the fifth-degree Chebyshev polynomial of the second kind at these points. Because these numbers are not symbolic objects, `chebyshevU` returns floating-point results.

```
chebyshevU(5, [1/6, 1/3, 1/2, 2/3, 4/5])

ans =
    0.8560    0.9465    0.0000   -1.2675   -1.0982
```

Find the value of the fifth-degree Chebyshev polynomial of the second kind for the same numbers converted to symbolic objects. For symbolic numbers, `chebyshevU` returns exact symbolic results.

```
chebyshevU(5, sym([1/6, 1/4, 1/3, 1/2, 2/3, 4/5]))

ans =
[ 208/243, 33/32, 230/243, 0, -308/243, -3432/3125]
```

### Evaluate Chebyshev Polynomials with Floating-Point Numbers

Floating-point evaluation of Chebyshev polynomials by direct calls of `chebyshevU` is numerically stable. However, first computing the polynomial using a symbolic variable, and then substituting variable-precision values into this expression can be numerically unstable.

Find the value of the 500th-degree Chebyshev polynomial of the second kind at  $1/3$  and `vpa(1/3)`. Floating-point evaluation is numerically stable.

```
chebyshevU(500, 1/3)
chebyshevU(500, vpa(1/3))
```

```
ans =
    0.8680
```

```
ans =
0.86797529488884242798157148968078
```

Now, find the symbolic polynomial  $U_{500} = \text{chebyshevU}(500, x)$ , and substitute  $x = \text{vpa}(1/3)$  into the result. This approach is numerically unstable.

```
syms x
U500 = chebyshevU(500, x);
subs(U500, x, vpa(1/3))
```

```
ans =
63080680195950160912110845952.0
```

Approximate the polynomial coefficients by using `vpa`, and then substitute  $x = \text{sym}(1/3)$  into the result. This approach is also numerically unstable.

```
subs(vpa(U500), x, sym(1/3))
```

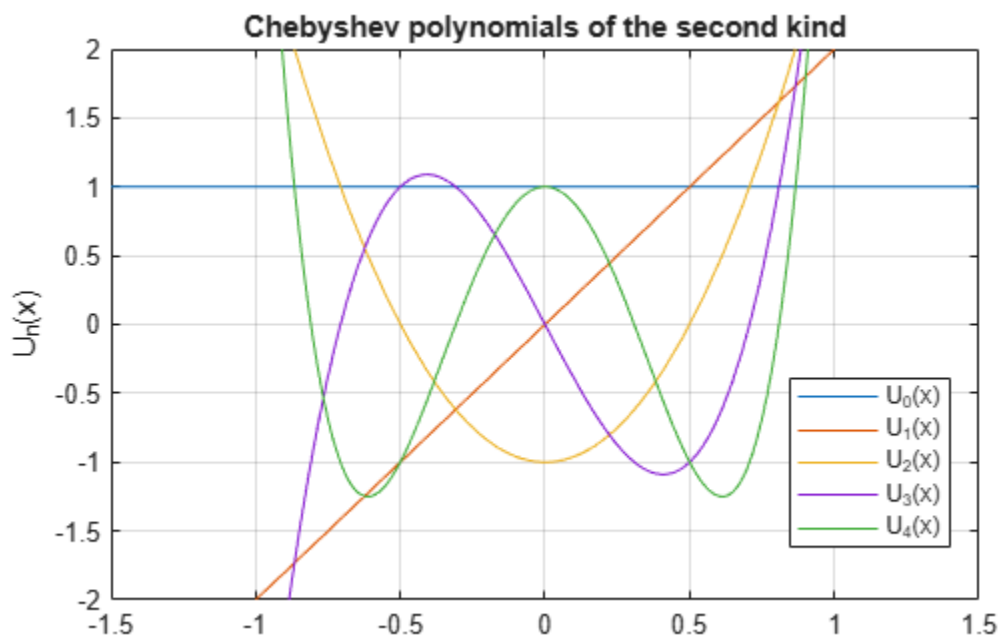
```
ans =
-1878009301399851172833781612544.0
```

### Plot Chebyshev Polynomials of the Second Kind

Plot the first five Chebyshev polynomials of the second kind.

```
syms x y
fplot(chebyshevU(0:4, x))
axis([-1.5 1.5 -2 2])
grid on

ylabel('U_n(x)')
legend('U_0(x)', 'U_1(x)', 'U_2(x)', 'U_3(x)', 'U_4(x)', 'Location', 'Best')
title('Chebyshev polynomials of the second kind')
```



## Input Arguments

### **n** — Degree of polynomial

nonnegative integer | symbolic variable | symbolic expression | symbolic function | vector | matrix

Degree of the polynomial, specified as a nonnegative integer, symbolic variable, expression, or function, or as a vector or matrix of numbers, symbolic numbers, variables, expressions, or functions.

### **x** — Evaluation point

number | symbolic number | symbolic variable | symbolic expression | symbolic function | vector | matrix

Evaluation point, specified as a number, symbolic number, variable, expression, or function, or as a vector or matrix of numbers, symbolic numbers, variables, expressions, or functions.

## More About

### Chebyshev Polynomials of the Second Kind

- Chebyshev polynomials of the second kind are defined as follows:

$$U(n, x) = \frac{\sin((n+1)\arccos(x))}{\sin(\arccos(x))}$$

These polynomials satisfy the recursion formula

$$U(0, x) = 1, \quad U(1, x) = 2x, \quad U(n, x) = 2xU(n-1, x) - U(n-2, x)$$

- Chebyshev polynomials of the second kind are orthogonal on the interval  $-1 \leq x \leq 1$  with respect to the weight function  $w(x) = \sqrt{1-x^2}$ .

$$\int_{-1}^1 U(n, x)U(m, x)\sqrt{1-x^2} dx = \begin{cases} 0 & \text{if } n \neq m \\ \frac{\pi}{2} & \text{if } n = m. \end{cases}$$

- Chebyshev polynomials of the second kind are a special case of the Jacobi polynomials

$$U(n, x) = \frac{2^{2n}n!(n+1)!}{(2n+1)!}P\left(n, \frac{1}{2}, \frac{1}{2}, x\right)$$

and Gegenbauer polynomials

$$U(n, x) = G(n, 1, x)$$

## Tips

- `chebyshevU` returns floating-point results for numeric arguments that are not symbolic objects.
- `chebyshevU` acts element-wise on nonscalar inputs.
- At least one input argument must be a scalar or both arguments must be vectors or matrices of the same size. If one input argument is a scalar and the other one is a vector or a matrix, then `chebyshevU` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

## Version History

Introduced in R2014b

## References

- [1] Hochstrasser, U. W. "Orthogonal Polynomials." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.
- [2] Cohl, Howard S., and Connor MacKenzie. "Generalizations and Specializations of Generating Functions for Jacobi, Gegenbauer, Chebyshev and Legendre Polynomials with Definite Integrals." *Journal of Classical Analysis*, no. 1 (2013): 17-33. <https://doi.org/10.7153/jca-03-02>.

## See Also

`chebyshevT` | `gegenbauerC` | `hermiteH` | `jacobiP` | `laguerreL` | `legendreP`



# checkUnits

Check for compatible dimensions and consistent units

## Syntax

```
C = checkUnits(expr)
C = checkUnits(expr, 'Compatible')
C = checkUnits(expr, 'Consistent')
```

## Description

`C = checkUnits(expr)` checks `expr` for compatible dimensions and consistent units and returns a structure containing the fields `Consistent` and `Compatible`. The fields contain logical 0 (false) or logical 1 (true) depending on the check results.

`expr` has compatible dimensions if all terms have the same dimensions, such as length or time. `expr` has consistent units if all units of the same dimension can be converted to each other with a conversion factor of 1.

`C = checkUnits(expr, 'Compatible')` only checks `expr` for compatible dimensions.

`C = checkUnits(expr, 'Consistent')` only checks `expr` for consistent units.

## Examples

### Check Dimensions of Units

Check the dimensions of an equation or expression. The dimensions are checked to confirm that the equation or expression is valid.

Verify the dimensions of the equation

$$A \frac{\text{m}}{\text{s}} = B \frac{\text{kg}}{\text{s}}$$

by using `checkUnits` with the option `'Compatible'`. MATLAB® assumes that symbolic variables are dimensionless. The `checkUnits` function returns logical 0 (false) because the dimensions of the equation are not compatible.

```
u = symunit;
syms A B
eqn = A*u.m/u.s == B*u.kg/u.s;
checkUnits(eqn, 'Compatible')
```

```
ans = logical
      0
```

Replace `u.kg` with `u.m` by using `subs` and repeat the check. Because the dimensions are now compatible, `checkUnits` returns logical 1 (true).

```
eqn = subs(eqn,u.kg,u.m);  
checkUnits(eqn,'Compatible')
```

```
ans = logical  
     1
```

### Check Consistency of Units

Checking units for consistency is a stronger check than compatibility. Units are consistent when all units of the same dimension can be converted to each other with a conversion factor of 1. For example, 1 Newton is consistent with 1 kg m/s<sup>2</sup> but not with 1 kg cm/s<sup>2</sup>.

Show that 1 Newton is consistent with 1 kg m/s<sup>2</sup> by checking `expr1` but not with 1 kg cm/s<sup>2</sup> by checking `expr2`.

```
u = symunit;  
expr1 = 1*u.N + 1*u.kg*u.m/u.s^2;  
expr2 = 1*u.N + 1*u.kg*u.cm/u.s^2;  
checkUnits(expr1,'Consistent')
```

```
ans = logical  
     1
```

```
checkUnits(expr2,'Consistent')
```

```
ans = logical  
     0
```

Show the difference between compatibility and consistency by showing that `expr2` has compatible dimensions but not consistent units.

```
checkUnits(expr2,'Compatible')
```

```
ans = logical  
     1
```

### Check Multiple Equations or Expressions

Check multiple equations or expressions by placing them in an array. `checkUnits` returns an array whose elements correspond to the elements of the input.

Check multiple equations for compatible dimensions. `checkUnits` returns `[1 0]`, meaning that the first equation has compatible dimensions while the second equation does not.

```
u = symunit;  
syms x y z  
eqn1 = x*u.m == y*u.m^2/(z*u.m);  
eqn2 = x*u.m + y*u.s == z*u.m;
```

```
eqns = [eqn1 eqn2];
compatible = checkUnits(eqns, 'Compatible')

compatible = 1x2 logical array

     1     0
```

## Check Dimensions and Consistency of Units

Check for both compatible dimensions and consistent units of the equation or expression by using `checkUnits`.

Define the equations for x- and y-displacement of a moving projectile. Check their units for compatibility and consistency.

```
u = symunit;
syms theta x y ts
g = 9.81*u.cm/u.s^2;
v = 10*u.m/u.s;
t = ts*u.s;
x = v*cos(theta)*t;
y = v*sin(theta)*t + (-g*t^2)/2;
S = checkUnits([x y])

S = struct with fields:
    Consistent: [1 0]
    Compatible: [1 1]
```

The second equation has compatible dimensions but inconsistent units. This inconsistency is because `g` incorrectly uses `cm` instead of `m`. Redefine `g` and check the equations again. The second equation now has consistent units.

```
g = 9.81*u.m/u.s^2;
y = v*sin(theta)*t + (-g*t^2)/2;
S = checkUnits([x y])

S = struct with fields:
    Consistent: [1 1]
    Compatible: [1 1]
```

## Input Arguments

### expr — Input expression

symbolic expression | symbolic equation | symbolic function | symbolic vector | symbolic matrix | symbolic multidimensional array

Input expression, specified as a symbolic expression, equation, function, vector, matrix, or multidimensional array.

## **Version History**

**Introduced in R2017a**

### **See Also**

`findUnits` | `isUnit` | `newUnit` | `separateUnits` | `symunit` | `str2symunit` | `symunit2str` | `unitConversionFactor`

### **Topics**

“Units of Measurement Tutorial” on page 2-47

“Unit Conversions and Unit Systems” on page 2-53

“Units and Unit Systems List” on page 2-60

# children

Subexpressions or terms of symbolic expression

---

**Note** Starting in R2020b, the syntax `subexpr = children(expr)` for a scalar input `expr` returns `subexpr` as a *nonnested cell array* instead of a vector. You can use `subexpr = children(expr,ind)` to index into the returned cell array of subexpressions. For more information, see “Version History”.

---

## Syntax

```
subexpr = children(expr)
subexpr = children(A)
subexpr = children( ___,ind)
```

## Description

`subexpr = children(expr)` returns a nonnested cell array containing the child subexpressions of the symbolic expression `expr`. For example, the child subexpressions of a sum are its terms.

`subexpr = children(A)` returns a nested cell array containing the child subexpressions of each expression in the symbolic matrix `A`.

`subexpr = children( ___,ind)` returns the child subexpressions of a symbolic expression `expr` or a symbolic matrix `A` as a cell array indexed by `ind`.

## Examples

### Find Child Subexpressions of Symbolic Expression

Find the child subexpressions of the symbolic expression  $x^2 + xy + y^2$ . The subexpressions are returned in a nonnested cell array. `children` uses internal sorting rules when returning the subexpressions. You can index into each element of the cell array by using `subexpr{i}`, where `i` is the cell index. The child subexpressions of a sum are its terms.

```
syms x y
subexpr = children(x^2 + x*y + y^2)
```

```
subexpr=1×3 cell array
    {[x*y]}    {[x^2]}    {[y^2]}
```

```
s1 = subexpr{1}
```

```
s1 = x y
```

```
s2 = subexpr{2}
```

```
s2 = x^2
```

```
s3 = subexpr{3}
```

```
s3 =  $y^2$ 
```

You can also index into each element of the subexpressions by specifying the index `ind` in the `children` function.

```
s1 = children(x^2 + x*y + y^2,1)
```

```
s1 =  $xy$ 
```

```
s2 = children(x^2 + x*y + y^2,2)
```

```
s2 =  $x^2$ 
```

```
s3 = children(x^2 + x*y + y^2,3)
```

```
s3 =  $y^2$ 
```

To convert the cell array of subexpressions into a vector, you can use the command `[subexpr{:}]`.

```
V = [subexpr{:}]
```

```
V = ( $xy \ x^2 \ y^2$ )
```

### Find Child Subexpressions of Equation

Find the child subexpressions of the equation  $x^2 + xy = y^2 + 1$ . The child subexpressions of the equation are returned in a 1-by-2 cell array. Index into all elements of the cell array. The subexpressions of an equation are the left and right sides of that equation.

```
syms x y
```

```
subexpr = children(x^2 + x*y == y^2 + 1)
```

```
subexpr=1x2 cell array
    {[x^2 + y*x]}    {[y^2 + 1]}
```

```
subexpr{:}
```

```
ans =  $x^2 + yx$ 
```

```
ans =  $y^2 + 1$ 
```

Next, find the child subexpressions of the inequality  $\sin(x) < \cos(x)$ . Index into all elements of the returned cell array. The child subexpressions of an inequality are the left and right sides of that inequality.

```
subexpr = children(sin(x) < cos(x))
```

```
subexpr=1x2 cell array
    {[sin(x)]}    {[cos(x)]}
```

```
subexpr{:}
```

```
ans = sin(x)
```

```
ans = cos(x)
```

### Find Child Subexpressions of Integral

Find the child subexpressions of an integral  $\int_a^b f(x) dx$ . The child subexpressions are returned as a cell array of symbolic expressions.

```
syms f(x) a b
subexpr = children(int(f(x),a,b))

subexpr=1x4 cell array
    {[f(x)]}    {[x]}    {[a]}    {[b]}
```

```
V = [subexpr{:}]
```

```
V = (f(x) x a b)
```

### Plot Taylor Approximation of Expression

Find the Taylor approximation of the  $\cos(x)$  function near  $x = 2$ .

```
syms x
t = taylor(cos(x),x,2)

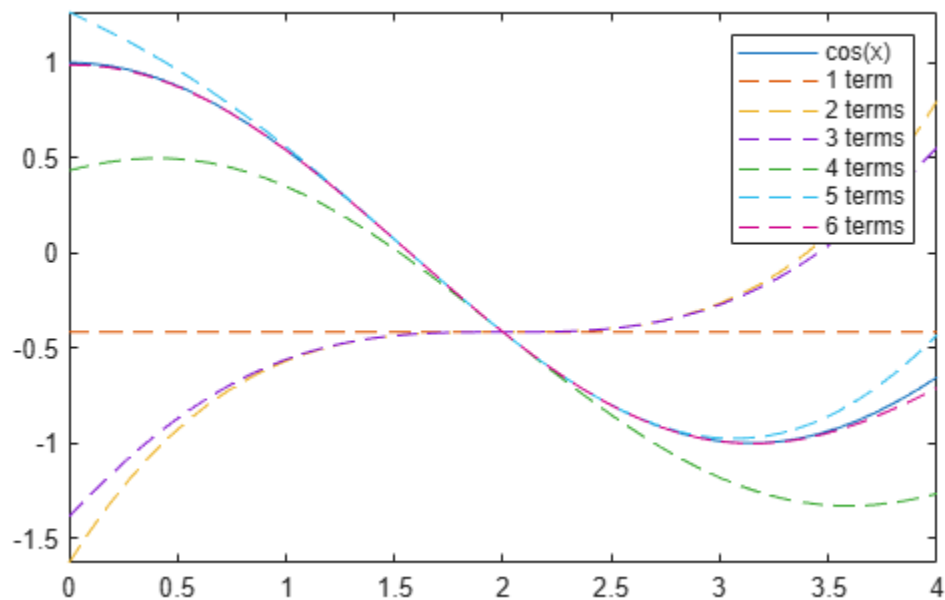
t =
```

$$\cos(2) + \frac{\sin(2)(x-2)^3}{6} - \frac{\sin(2)(x-2)^5}{120} - \sin(2)(x-2) - \frac{\cos(2)(x-2)^2}{2} + \frac{\cos(2)(x-2)^4}{24}$$

The Taylor expansion has six terms that are separated by + or - sign.

Plot the  $\cos(x)$  function. Use `children` to separate out the terms of the expansion. Show that the Taylor expansion approximates the function more closely as more terms are included.

```
fplot(cos(x),[0 4])
hold on
s = 0;
for i = 1:6
    s = s + children(t,i);
    fplot(s,[0 4], '--')
end
legend({'cos(x)', '1 term', '2 terms', '3 terms', '4 terms', '5 terms', '6 terms'})
```



### Find Child Subexpressions of Elements of Matrix

Call the `children` function to find the child subexpressions of the following symbolic matrix input. The result is a 2-by-2 nested cell array containing the child subexpressions of each element of the matrix.

```
syms x y
symM = [x + y, sin(x)*cos(y); x^3 - y^3, exp(x*y^2) + 3]
symM =

$$\begin{pmatrix} x + y & \cos(y) \sin(x) \\ x^3 - y^3 & e^{xy^2} + 3 \end{pmatrix}$$

s = children(symM)
s=2x2 cell array
    {1x2 cell}    {1x2 cell}
    {1x2 cell}    {1x2 cell}
```

To unnest or access the elements of the nested cell array `s`, use braces. For example, the `{1,1}`-element of `s` is a 1-by-2 cell array of symbolic expressions.

```
s11 = s{1,1}
s11=1x2 cell array
    {[x]}    {[y]}
```

Unnest each element of `s` using braces. Convert the nonnested cell arrays to vectors using square brackets.



```
s11vec = [s{1,1}{:}]
```

```
s11vec = (x y)
```

```
s21vec = [s{2,1}{:}]
```

```
s21vec = (x3 -y3)
```

```
s12vec = [s{1,2}{:}]
```

```
s12vec = (cos(y) sin(x))
```

```
s22vec = [s{2,2}{:}]
```

```
s22vec = (exy2 3)
```

If each element of the nested cell array `s` contains a nonnested cell array of the same size, then you can also use the `ind` input argument to access the elements of the nested cell array. The index `ind` allows children to access each column of subexpressions of the symbolic matrix input `symM`.

```
scol1 = children(symM,1)
```

```
scol1=2×2 cell array
    {[x  ]}    {[cos(y)  ]}
    {[x^3]}    {[exp(x*y^2)]}
```

```
[scol1{:}].'
```

```
ans =
    (
      x
      x3
      cos(y)
      exy2
    )
```

```
scol2 = children(symM,2)
```

```
scol2=2×2 cell array
    {[y  ]}    {[sin(x)]}
    {[ -y^3]}    {[3    ]}
```

```
[scol2{:}].'
```

```
ans =
    (
      y
      -y3
      sin(x)
      3
    )
```

## Input Arguments

### **expr** — Input expression

symbolic number | symbolic variable | symbolic function | symbolic expression

Input expression, specified as a symbolic number, variable, function, or expression.

**A — Input matrix**

symbolic matrix

Input matrix, specified as a symbolic matrix.

**ind — Index of child subexpressions to return**

positive integer

Index of child subexpressions to return, specified as a positive integer.

- If `children(expr)` returns a nonnested cell array of child subexpressions, then indexing with `children(expr,ind)` returns the `ind`-th element of the cell array.
- If `children(A)` returns a nested cell array of child subexpressions, where each cell element has the same size, then indexing with `children(A,ind)` returns the `ind`-th column of the nonnested cell array.

## Version History

**Introduced in R2012a****R2020b: children returns cell arrays***Behavior changed in R2020b*

In versions before R2020b, the syntax `subexpr = children(expr)` returns a vector `subexpr` that contains the child subexpressions of the scalar symbolic expression `expr`. The syntax `subexpr = children(A)` returns a nonnested cell array `subexpr` that contains the child subexpressions of the symbolic matrix `A`.

Starting in R2020b, the syntax `subexpr = children(expr)` returns `subexpr` as a cell array instead of a vector, and the syntax `subexpr = children(A)` returns `subexpr` as a nested cell array instead of a nonnested cell array. You can use `subexpr = children(expr,ind)` to index into the returned cell arrays of subexpressions. For example, see “Plot Taylor Approximation of Expression” on page 6-167. You can also unnest and access the elements of a cell array by indexing into the cell array using curly braces. To convert `subexpr` from a nonnested cell array to a vector, you can use the command `[subexpr{:}]`.

## See Also

`coeffs` | `lhs` | `numden` | `rhs` | `subs`

## Topics

“Create Symbolic Numbers, Variables, and Expressions” on page 1-3

# chol

Cholesky factorization

## Syntax

```
T = chol(A)
[T,p] = chol(A)
[T,p,S] = chol(A)
[T,p,s] = chol(A,'vector')
_____ = chol(A,'lower')
_____ = chol(A,'nocheck')
_____ = chol(A,'real')
_____ = chol(A,'lower','nocheck','real')
[T,p,s] = chol(A,'lower','vector','nocheck','real')
```

## Description

`T = chol(A)` returns an upper triangular matrix  $T$ , such that  $T' * T = A$ .  $A$  must be a Hermitian positive definite matrix on page 6-176. Otherwise, this syntax throws an error.

`[T,p] = chol(A)` computes the Cholesky factorization on page 6-176 of  $A$ . This syntax does not error if  $A$  is not a Hermitian positive definite matrix. If  $A$  is a Hermitian positive definite matrix, then  $p$  is 0. Otherwise,  $T$  is `sym([])`, and  $p$  is a positive integer (typically,  $p = 1$ ).

`[T,p,S] = chol(A)` returns a permutation matrix  $S$ , such that  $T' * T = S' * A * S$ , and the value  $p = 0$  if matrix  $A$  is Hermitian positive definite. Otherwise, it returns a positive integer  $p$  and an empty object  $S = \text{sym}([])$ .

`[T,p,s] = chol(A,'vector')` returns the permutation information as a vector  $s$ , such that  $A(s,s) = T' * T$ . If  $A$  is not recognized as a Hermitian positive definite matrix, then  $p$  is a positive integer and  $s = \text{sym}([])$ .

\_\_\_\_\_ = `chol(A,'lower')` returns a lower triangular matrix  $T$ , such that  $T * T' = A$ .

\_\_\_\_\_ = `chol(A,'nocheck')` skips checking whether matrix  $A$  is Hermitian positive definite. 'nocheck' lets you compute Cholesky factorization of a matrix that contains symbolic parameters without setting additional assumptions on those parameters.

\_\_\_\_\_ = `chol(A,'real')` computes the Cholesky factorization of  $A$  using real arithmetic. In this case, `chol` computes a symmetric factorization  $A = T' * T$  instead of a Hermitian factorization  $A = T' * T$ . This approach is based on the fact that if  $A$  is real and symmetric, then  $T' * T = T' * T$ . Use 'real' to avoid complex conjugates in the result.

\_\_\_\_\_ = `chol(A,'lower','nocheck','real')` computes the Cholesky factorization of  $A$  with one or more of these optional arguments: 'lower', 'nocheck', and 'real'. These optional arguments can appear in any order.

`[T,p,s] = chol(A,'lower','vector','nocheck','real')` computes the Cholesky factorization of  $A$  and returns the permutation information as a vector  $s$ . You can use one or more of these optional arguments: 'lower', 'nocheck', and 'real'. These optional arguments can appear in any order.

## Examples

### Compute Cholesky Factorization of Numeric and Symbolic Matrices

Compute the Cholesky factorization of the 3-by-3 Hilbert matrix. Because these numbers are not symbolic objects, you get floating-point results.

```
chol(hilb(3))

ans =
    1.0000    0.5000    0.3333
         0    0.2887    0.2887
         0         0    0.0745
```

Now convert this matrix to a symbolic object, and compute the Cholesky factorization:

```
chol(sym(hilb(3)))

ans =
[ 1,          1/2,          1/3]
[ 0, 3^(1/2)/6, 3^(1/2)/6]
[ 0,          0, 5^(1/2)/30]
```

### Return Lower Triangular Matrix

Compute the Cholesky factorization of the 3-by-3 Pascal matrix returning a lower triangular matrix as a result:

```
chol(sym(pascal(3)), 'lower')

ans =
[ 1, 0, 0]
[ 1, 1, 0]
[ 1, 2, 1]
```

### If Input is not Hermitian Positive Definite

Try to compute the Cholesky factorization of this matrix. Because this matrix is not Hermitian positive definite, `chol` used without output arguments or with one output argument throws an error:

```
A = sym([1 1 1; 1 2 3; 1 3 5]);
T = chol(A)
```

```
Error using sym/chol (line 132)
Cannot prove that input matrix is Hermitian positive definite.
Define a Hermitian positive definite matrix by setting
appropriate assumptions on matrix components, or use 'nocheck'
to skip checking whether the matrix is Hermitian positive definite.
```

To suppress the error, use two output arguments, `T` and `p`. If the matrix is not recognized as Hermitian positive definite, then this syntax assigns an empty symbolic object to `T` and the value 1 to `p`:

```
[T,p] = chol(A)

T =
[ empty sym ]
```

```
p =
     1
```

For a Hermitian positive definite matrix, `p` is 0:

```
[T,p] = chol(sym(pascal(3)))
```

```
T =
[ 1, 1, 1]
[ 0, 1, 2]
[ 0, 0, 1]
p =
     0
```

Alternatively, `'nocheck'` lets you skip checking whether `A` is a Hermitian positive definite matrix. Thus, this flag lets you compute the Cholesky factorization of a symbolic matrix without setting additional assumptions on its components to make it Hermitian positive definite:

```
syms a
A = [a 0; 0 a];
chol(A,'nocheck')

ans =
[ a^(1/2),      0]
[      0, a^(1/2)]
```

If you use `'nocheck'` for computing the Cholesky factorization of a matrix that is not Hermitian positive definite, `chol` can return a matrix `T` for which the identity  $T' * T = A$  does not hold. To make `isAlways` return logical 0 (false) for undecidable conditions, set `Unknown` to false.

```
T = chol(sym([1 1; 2 1]), 'nocheck')

T =
[ 1,      2]
[ 0, 3^(1/2)*1i]

isAlways(A == T'*T,'Unknown','false')

ans =
    2x2 logical array
     0     0
     0     0
```

### Return Permutation Matrix

Compute the Cholesky factorization of the 3-by-3 inverse Hilbert matrix returning the permutation matrix:

```
A = sym(invhilb(3));
[T, p, S] = chol(A)

T =
[ 3,      -12,      10]
[ 0, 4*3^(1/2), -5*3^(1/2)]
[ 0,      0,      5^(1/2)]

p =
     0
```

```
S =
    1     0     0
    0     1     0
    0     0     1
```

### Return Permutation Information as Vector

Compute the Cholesky factorization of the 3-by-3 inverse Hilbert matrix returning the permutation information as a vector:

```
A = sym(invhilb(3));
[T, p, S] = chol(A, 'vector')
```

```
T =
[ 3,      -12,      10]
[ 0, 4*3^(1/2), -5*3^(1/2)]
[ 0,      0,      5^(1/2)]
p =
    0
S =
    1     2     3
```

### Use Assumptions to Make Matrix Hermitian Positive Definite

Compute the Cholesky factorization of matrix A containing symbolic parameters. Without additional assumptions on the parameter a, this matrix is not Hermitian. To make `isAlways` return logical 0 (false) for undecidable conditions, set `Unknown` to false.

```
syms a
A = [a 0; 0 a];
isAlways(A == A', 'Unknown', 'false')
```

```
ans =
    2x2 logical array
    0     1
    1     0
```

By setting assumptions on a and b, you can define A to be Hermitian positive definite. Therefore, you can compute the Cholesky factorization of A:

```
assume(a > 0)
chol(A)
```

```
ans =
[ a^(1/2),      0]
[      0, a^(1/2)]
```

For further computations, remove the assumptions on a by recreating it using `syms`:

```
syms a
```

### Return Real Result Without Complex Conjugates

Compute the Cholesky factorization of this matrix. To skip checking whether it is Hermitian positive definite, use `'nocheck'`. By default, `chol` computes a Hermitian factorization  $A = T' * T$ . Thus, the result contains complex conjugates.

```
syms a b
A = [a b; b a];
T = chol(A, 'nocheck')

T =
[ a^(1/2), conj(b)/conj(a^(1/2))]
[ 0, (a*abs(a) - abs(b)^2)^(1/2)/abs(a)^(1/2)]
```

To avoid complex conjugates in the result, use 'real':

```
T = chol(A, 'nocheck', 'real')

T =
[ a^(1/2), b/a^(1/2)]
[ 0, ((a^2 - b^2)/a)^(1/2)]
```

When you use this flag, `chol` computes a symmetric factorization  $A = T.'*T$  instead of a Hermitian factorization  $A = T'*T$ . To make `isAlways` return logical 0 (false) for undecidable conditions, set `Unknown` to false.

```
isAlways(A == T.'*T)

ans =
    2x2 logical array
     1     1
     1     1

isAlways(A == T'*T, 'Unknown', 'false')

ans =
    2x2 logical array
     0     0
     0     0
```

## Input Arguments

### A — Input

number | vector | matrix | multidimensional array | symbolic number | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## Output Arguments

### T — Upper triangular matrix

symbolic matrix

Upper triangular matrix, returned as a symbolic matrix such that  $T.'*T = A$ . If  $T$  is a lower triangular matrix, then  $T*T' = A$ .

### p — Output

symbolic number

Flag, returned as a symbolic number. Value 0 if  $A$  is Hermitian positive definite or if you use 'nocheck'.

If `chol` does not identify  $A$  as a Hermitian positive definite matrix, then  $p$  is a positive integer.  $R$  is an upper triangular matrix of order  $q = p - 1$ , such that  $R' * R = A(1:q, 1:q)$ .

### **S — Permutation matrix**

symbolic matrix

Permutation matrix returned as a symbolic matrix.

### **s — Permutation vector**

symbolic vector

Permutation vector returned as a symbolic vector.

## **Limitations**

Matrix computations involving many symbolic variables can be slow. To increase the computational speed, reduce the number of symbolic variables by substituting the given values for some variables.

## **More About**

### **Hermitian Positive Definite Matrix**

A square complex matrix  $A$  is Hermitian positive definite if  $v' * A * v$  is real and positive for all nonzero complex vectors  $v$ , where  $v'$  is the conjugate transpose (Hermitian transpose) of  $v$ .

### **Cholesky Factorization of a Matrix**

The Cholesky factorization of a Hermitian positive definite  $n$ -by- $n$  matrix  $A$  is defined by an upper or lower triangular matrix with positive entries on the main diagonal. The Cholesky factorization of matrix  $A$  can be defined as  $T' * T = A$ , where  $T$  is an upper triangular matrix. Here  $T'$  is the conjugate transpose of  $T$ . The Cholesky factorization also can be defined as  $T * T' = A$ , where  $T$  is a lower triangular matrix.  $T$  is called the Cholesky factor of  $A$ .

## **Tips**

- Calling `chol` for numeric arguments that are not symbolic objects invokes the MATLAB `chol` function.
- If you use `'nocheck'`, then the identities  $T' * T = A$  (for an upper triangular matrix  $T$ ) and  $T * T' = A$  (for a lower triangular matrix  $T$ ) are not guaranteed to hold.
- If you use `'real'`, then the identities  $T' * T = A$  (for an upper triangular matrix  $T$ ) and  $T * T' = A$  (for a lower triangular matrix  $T$ ) are only guaranteed to hold for a real symmetric positive definite  $A$ .
- To use `'vector'`, you must specify three output arguments. Other flags do not require a particular number of output arguments.
- If you use `'matrix'` instead of `'vector'`, then `chol` returns permutation matrices, as it does by default.
- If you use `'upper'` instead of `'lower'`, then `chol` returns an upper triangular matrix, as it does by default.
- If  $A$  is not a Hermitian positive definite matrix, then the syntaxes containing the argument  $p$  typically return  $p = 1$  and an empty symbolic object  $T$ .



- To check whether a matrix is Hermitian, use the operator `'` (or its functional form `ctranspose`). Matrix `A` is Hermitian if and only if  $A' = A$ , where  $A'$  is the conjugate transpose of `A`.

## Version History

Introduced in R2013a

## See Also

`chol` | `ctranspose` | `eig` | `isAlways` | `lu` | `qr` | `svd` | `transpose` | `vpa`

## coeffs

Coefficients of polynomial

### Syntax

```
C = coeffs(p)
C = coeffs(p,var)
C = coeffs(p,vars)
[C,T] = coeffs(____)
____ = coeffs(____,'All')
```

### Description

`C = coeffs(p)` returns coefficients of the polynomial `p` with respect to all variables determined in `p` by `symvar`.

`C = coeffs(p,var)` returns coefficients of the polynomial `p` with respect to the variable `var`.

`C = coeffs(p,vars)` returns coefficients of the multivariate polynomial `p` with respect to the variables `vars`.

`[C,T] = coeffs(____)` returns the coefficient `C` and the corresponding terms `T` of the polynomial `p`.

`____ = coeffs(____,'All')` returns all coefficients, including coefficients that are 0. For example, `coeffs(2*x^2,'All')` returns `[ 2, 0, 0]` instead of `2`.

### Examples

#### Coefficients of Univariate Polynomial

Find the coefficients of this univariate polynomial. The coefficients are ordered from the lowest degree to the highest degree.

```
syms x
c = coeffs(16*x^2 + 19*x + 11)
```

```
c =
[ 11, 19, 16]
```

Reverse the ordering of coefficients by using `fliplr`.

```
c = fliplr(c)
```

```
c =
[ 16, 19, 11]
```

#### Coefficients of Multivariate Polynomial with Respect to Particular Variable

Find the coefficients of this polynomial with respect to variable `x` and variable `y`.

```
syms x y
cx = coeffs(x^3 + 2*x^2*y + 3*x*y^2 + 4*y^3, x)
cy = coeffs(x^3 + 2*x^2*y + 3*x*y^2 + 4*y^3, y)

cx =
[ 4*y^3, 3*y^2, 2*y, 1]

cy =
[ x^3, 2*x^2, 3*x, 4]
```

### Coefficients of Multivariate Polynomial with Respect to Two Variables

Find the coefficients of this polynomial with respect to both variables x and y.

```
syms x y
cxy = coeffs(x^3 + 2*x^2*y + 3*x*y^2 + 4*y^3, [x y])
cyx = coeffs(x^3 + 2*x^2*y + 3*x*y^2 + 4*y^3, [y x])

cxy =
[ 4, 3, 2, 1]

cyx =
[ 1, 2, 3, 4]
```

### Coefficients and Corresponding Terms of Univariate Polynomial

Find the coefficients and the corresponding terms of this univariate polynomial. When two outputs are provided, the coefficients are ordered from the highest degree to the lowest degree.

```
syms x
[c,t] = coeffs(16*x^2 + 19*x + 11)

c =
[ 16, 19, 11]

t =
[ x^2, x, 1]
```

### Coefficients and Corresponding Terms of Multivariate Polynomial

Find the coefficients and the corresponding terms of this polynomial with respect to variable x and variable y.

```
syms x y
[cx,tx] = coeffs(x^3 + 2*x^2*y + 3*x*y^2 + 4*y^3, x)
[cy,ty] = coeffs(x^3 + 2*x^2*y + 3*x*y^2 + 4*y^3, y)

cx =
[ 1, 2*y, 3*y^2, 4*y^3]

tx =
[ x^3, x^2, x, 1]

cy =
[ 4, 3*x, 2*x^2, x^3]

ty =
[ y^3, y^2, y, 1]
```

Find the coefficients of this polynomial with respect to both variables  $x$  and  $y$ .

```
syms x y
[cxy, txy] = coeffs(x^3 + 2*x^2*y + 3*x*y^2 + 4*y^3, [x,y])
[cyx, tyx] = coeffs(x^3 + 2*x^2*y + 3*x*y^2 + 4*y^3, [y,x])

cxy =
[ 1, 2, 3, 4]

txy =
[ x^3, x^2*y, x*y^2, y^3]

cyx =
[ 4, 3, 2, 1]

tyx =
[ y^3, x*y^2, x^2*y, x^3]
```

### All Coefficients of Polynomial

Find all coefficients of a polynomial, including coefficients that are 0, by specifying the option 'All'. The returned coefficients are ordered from the highest degree to the lowest degree.

Find all coefficients of  $3x^2$ .

```
syms x
c = coeffs(3*x^2, 'All')

c =
[ 3, 0, 0]
```

If you find coefficients with respect to multiple variables and specify 'All', then `coeffs` returns coefficients for all combinations of the variables.

Find all coefficients and corresponding terms of  $ax^2 + by$ .

```
syms a b y
[cxy, txy] = coeffs(a*x^2 + b*y, [y x], 'All')

cxy =
[ 0, 0, b]
[ a, 0, 0]
txy =
[ x^2*y, x*y, y]
[ x^2, x, 1]
```

## Input Arguments

### p — Polynomial

symbolic expression | symbolic function

Polynomial, specified as a symbolic expression or function.

### var — Polynomial variable

symbolic variable

Polynomial variable, specified as a symbolic variable.

**vars — Polynomial variables**

vector of symbolic variables

Polynomial variables, specified as a vector of symbolic variables.

**Output Arguments****C — Coefficients of polynomial**

symbolic number | symbolic variable | symbolic expression | symbolic vector | symbolic matrix |  
symbolic multidimensional array

Coefficients of polynomial, returned as a symbolic number, variable, expression, vector, matrix, or multidimensional array. If there is only one coefficient and one corresponding term, then C is returned as a scalar.

**T — Terms of polynomial**

symbolic number | symbolic variable | symbolic expression | symbolic vector | symbolic matrix |  
symbolic multidimensional array

Terms of polynomial, returned as a symbolic number, variable, expression, vector, matrix, or multidimensional array. If there is only one coefficient and one corresponding term, then T is returned as a scalar.

**Version History**

Introduced before R2006a

**See Also**

poly2sym | sym2poly

## collect

Collect coefficients of identical powers

### Syntax

```
C = collect(P)
C = collect(P,expr)
```

### Description

`C = collect(P)` collects the coefficients of identical powers of the default variable in the symbolic input `P`. The default variable in `P` is determined by `symvar`.

`C = collect(P,expr)` collects the coefficients of identical powers in terms of the specified expression. If `P` is a vector or matrix, then `collect` acts element-wise on `P`. If `expr` is a vector, then `collect` finds coefficients in terms of all expressions in `expr`.

### Examples

#### Collect Coefficients of Identical Powers of Default Variable

Collect the coefficients of identical powers of the default variable in a symbolic expression. Here, `collect` returns the expression as a polynomial in terms of `x` by collecting the coefficients of `x^2` and `x`.

```
syms x
P = (exp(x) + x)*(x + 2)

P = (x + e^x)(x + 2)

C = collect(P)

C = x^2 + (e^x + 2)x + 2 e^x
```

Because you did not specify the variable, `collect` uses the default variable determined by `symvar`. For this expression, the default variable is `x`.

```
defaultvar = symvar(P)

defaultvar = x
```

#### Collect Coefficients of Identical Powers of Specified Variable

Collect the coefficients of identical powers of a specified variable in a symbolic expression by specifying the variable as the second argument to `collect`.

Create a symbolic expression in terms of the variables `x` and `y`.

```
syms x y
P = x^2*y + y*x - x^2 - 2*x
```

$$P = xy - 2x + x^2y - x^2$$

Collect the coefficients of identical powers of  $x$ . Here, `collect` returns the expression as a polynomial in terms of  $x$  by collecting the coefficients of  $x^2$  and  $x$ .

```
Cx = collect(P,x)
```

$$Cx = (y - 1)x^2 + (y - 2)x$$

Collect the coefficients of identical powers of  $y$ . Here, `collect` returns the expression as a polynomial in terms of  $y$  by collecting the coefficients of  $y$ .

```
Cy = collect(P,y)
```

$$Cy = (x^2 + x)y - x^2 - 2x$$

You can also collect coefficients in terms of multiple variables by specifying the second argument as a vector of variables. Create another symbolic expression in terms of the variables  $x$  and  $y$ . Then collect the coefficients of identical powers of  $x$  and  $y$ . Here, `collect` returns the expression as a polynomial in terms of two variables,  $x$  and  $y$ , by collecting the coefficients of  $x^2$  and  $x*y$ .

```
syms a b
Q = a^2*x*y + a*b*x^2 + a*x*y + x^2
```

$$Q = ya^2x + bax^2 + yax + x^2$$

```
Cxy = collect(Q,[x y])
```

$$Cxy = (ab + 1)x^2 + (a^2 + a)xy$$

### Collect Coefficients in Terms of $i$ and $\pi$

Collect the coefficients of a symbolic expression in terms of  $i$ , and then in terms of  $\pi$ .

```
syms x y
P = y*pi*(pi - li) + x*(pi + li) + 3*pi
```

$$P = 3\pi + x(\pi + i) + \pi y(\pi - i)$$

```
Ci = collect(P,li)
```

$$Ci = (x - \pi y)i + 3\pi + \pi x + y\pi^2$$

```
Cpi = collect(P,pi)
```

$$Cpi = y\pi^2 + (x + 3 - y i)\pi + x i$$

### Collect Coefficients in Terms of Symbolic Expressions and Functions

You can specify the symbolic expression or function in terms of which you collect coefficients as the second argument of `collect`.

Expand the expression  $\sin(x + 3y)$  by using `expand`. Then collect the coefficients in terms of  $\sin(x)$ .

```
syms x y
P = expand(sin(x + 3*y))

P = 4 sin(x) cos(y)^3 + 4 cos(x) sin(y) cos(y)^2 - 3 sin(x) cos(y) - cos(x) sin(y)

C = collect(P, sin(x))

C = (4 cos(y)^3 - 3 cos(y)) sin(x) + 4 cos(x) cos(y)^2 sin(y) - cos(x) sin(y)
```

Collect the coefficients in terms of both  $\sin(x)$  and  $\sin(y)$  by specifying a vector input.

```
Cxy = collect(P, [sin(x) sin(y)])

Cxy = (4 cos(y)^3 - 3 cos(y)) sin(x) + (4 cos(x) cos(y)^2 - cos(x)) sin(y)
```

You can also collect coefficients in terms of the symbolic function  $y(x)$  in a symbolic expression.

```
syms y(x)
P = y^2*x + y*x^2 + y*sin(x) + x*y

P(x) = x y(x)^2 + x^2 y(x) + sin(x) y(x) + x y(x)

Cy = collect(P, y)

Cy(x) = x y(x)^2 + (x + sin(x) + x^2) y(x)
```

### Collect Coefficients for Each Element of Matrix

If you specify a matrix of symbolic inputs, `collect` acts element-wise on the matrix.

```
syms x y
P = [(x + 1)*(y + 1), x^2 + x*(x - y);
     2*x*y - x, x*y + x/y]

P =

$$\begin{pmatrix} (x+1)(y+1) & x(x-y) + x^2 \\ 2xy - x & xy + \frac{x}{y} \end{pmatrix}$$


C = collect(P, x)

C =

$$\begin{pmatrix} (y+1)x + y + 1 & 2x^2 + (-y)x \\ (2y-1)x & \left(y + \frac{1}{y}\right)x \end{pmatrix}$$

```



### Collect Coefficients in Terms of Function Calls

Collect the coefficients in terms of calls to a particular function by specifying the function name as a string scalar in the second argument. Collect the coefficients of function calls with respect to multiple functions by specifying the multiple functions as a string array.

Collect the coefficients in terms of calls to the `sin` function in `P`, where `P` is a symbolic expression that contains multiple calls to different functions.

```
syms a b c d e f x
P = a*sin(2*x) + b*sin(2*x) + c*cos(x) + ...
    d*cos(x) + e*sin(3*x) + f*sin(3*x)

P = a sin(2 x) + b sin(2 x) + e sin(3 x) + f sin(3 x) + c cos(x) + d cos(x)
```

```
C1 = collect(P,"sin")
```

```
C1 = (a + b) sin(2 x) + (e + f) sin(3 x) + c cos(x) + d cos(x)
```

Collect the coefficients in terms of calls to both the `sin` and `cos` functions in `P`.

```
C2 = collect(P,["sin" "cos"])

C2 = (a + b) sin(2 x) + (e + f) sin(3 x) + (c + d) cos(x)
```

### Collect Coefficients of Rational Function

Create a symbolic expression in terms of `x` and `y`.

```
syms x y
P = -12/((x - 2)*(x + 2)) + 8/(y + 2)

P =
```

$$\frac{8}{y+2} - \frac{12}{(x-2)(x+2)}$$

Collect the coefficients of identical powers of `x` and then `y` in the symbolic expression `P`. Here, the `collect` function returns a rational function in the form of the division of two polynomials. It collects the coefficients of identical terms with positive integer powers in the numerator and denominator separately.

```
Cx = collect(P,x)
```

```
Cx =
```

$$\frac{8x^2 - 12y - 56}{(y+2)x^2 - 4y - 8}$$

```
Cy = collect(P,y)
```

```
Cy =
```

$$\frac{-12y + 8x^2 - 56}{(x^2 - 4)y + 2x^2 - 8}$$

If the expression cannot be expressed as a rational function that is the division of two polynomials with positive integer powers on the indeterminates, then `collect` might not collect the coefficients of identical powers.

For example, create a symbolic expression that involves the square root of  $x$ . Here, `collect` does not collect identical powers of  $x$ .

```
Q = -12/(sqrt(x)*(x + 2)) + 8/(y + 2)
```

```
Q =
```

$$\frac{8}{y+2} - \frac{12}{\sqrt{x}(x+2)}$$

```
C = collect(Q,x)
```

```
C =
```

$$\frac{16\sqrt{x} - 12y + 8x^{3/2} - 24}{2\sqrt{x}y + x^{3/2}y + 4\sqrt{x} + 2x^{3/2}}$$

## Input Arguments

### P — Input expression

symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input expression, specified as a symbolic expression, symbolic function, symbolic vector, or symbolic matrix.

### expr — Expression in terms of which you collect coefficients

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | string array | character vector | cell array of character vectors

Expression in terms of which you collect the coefficients, specified as a symbolic number, symbolic variable, symbolic expression, symbolic function, symbolic vector, string array, character vector, or cell array of character vectors.

Example: `sin(x)`

Example: `[sin(x) cos(y)]`

Example: `["sin" "cos"]`

## Tips

- `collect` returns an output that is syntactically different from the input expression (although the input and output expressions might look the same). For this reason, functions like `isequal` might not return `true` when checking for equality. Instead, use `isAlways` to prove equivalence between the input and output expressions.

## Version History

Introduced before R2006a

## See Also

### Functions

combine | expand | factor | horner | numden | rewrite | simplify | simplifyFraction | symvar

### Topics

“Choose Function to Rearrange Expression” on page 3-123

## colon, :

Create symbolic vectors, array subscripting, and for-loop iterators

### Syntax

```
m:n  
m:d:n  
x:x+r  
x:d:x+r
```

### Description

$m:n$  returns a symbolic vector of values  $[m, m+1, \dots, n]$ , where  $m$  and  $n$  are symbolic constants. If  $n$  is not an increment of  $m$ , then the last value of the vector stops before  $n$ . This behavior holds for all syntaxes.

$m:d:n$  returns a symbolic vector of values  $[m, m+d, \dots, n]$ , where  $d$  is a rational number.

$x:x+r$  returns a symbolic vector of values  $[x, x+1, \dots, x+r]$ , where  $x$  is a symbolic variable and  $r$  is a rational number.

$x:d:x+r$  returns a symbolic vector of values  $[x, x+d, \dots, x+r]$ , where  $d$  and  $r$  are rational numbers.

### Examples

#### Create Numeric and Symbolic Arrays

Use the colon operator to create numeric and symbolic arrays. Because these inputs are not symbolic objects, you receive floating-point results.

```
1/2:7/2
```

```
ans =  
    0.5000    1.5000    2.5000    3.5000
```

To obtain symbolic results, convert the inputs to symbolic objects.

```
sym(1/2):sym(7/2)
```

```
ans =  
[ 1/2, 3/2, 5/2, 7/2]
```

Specify the increment used.

```
sym(1/2):2/3:sym(7/2)
```

```
ans =  
[ 1/2, 7/6, 11/6, 5/2, 19/6]
```

**Obtain Increments of Symbolic Variable**

```
syms x
x:x+2

ans =
[ x, x + 1, x + 2]
```

Specify the increment used.

```
syms x
x:3/7:x+2

ans =
[ x, x + 3/7, x + 6/7, x + 9/7, x + 12/7]
```

Obtain increments between  $x$  and  $2*x$  in intervals of  $x/3$ .

```
syms x
x:x/3:2*x

ans =
[ x, (4*x)/3, (5*x)/3, 2*x]
```

**Find Product of Harmonic Series**

Find the product of the first four terms of the harmonic series.

```
syms x
p = sym(1);
for i = x:x+3
    p = p*1/i;
end
p

p =
1/(x*(x + 1)*(x + 2)*(x + 3))
```

Use `expand` to obtain the full polynomial.

```
expand(p)

ans =
1/(x^4 + 6*x^3 + 11*x^2 + 6*x)
```

Use `subs` to replace  $x$  with 1 and find the product in fractions.

```
p = subs(p,x,1)

p =
1/24
```

Use `vpa` to return the result as a floating-point value.

```
vpa(p)

ans =
0.04166666666666666666666666666667
```

You can also perform the described operations in a single line of code.

```
vpa(subs( expand(prod(1./(x:x+3))) ,x,1))  
ans =  
0.041666666666666666666666666666667
```

## Input Arguments

### **m — Input**

symbolic constant

Input, specified as a symbolic constant.

### **n — Input**

symbolic constant

Input, specified as a symbolic constant.

### **x — Input**

symbolic variable

Input, specified as a symbolic variable.

### **r — Upper bound on vector values**

symbolic rational

Upper bound on vector values, specified as a symbolic rational. For example,  $x:x+2$  returns  $\begin{bmatrix} x, & x + 1, & x + 2 \end{bmatrix}$ .

### **d — Increment in vector values**

symbolic rational

Increment in vector values, specified as a symbolic rational. For example,  $x:1/2:x+2$  returns  $\begin{bmatrix} x, & x + 1/2, & x + 1, & x + 3/2, & x + 2 \end{bmatrix}$ .

## Version History

**Introduced before R2006a**

## See Also

reshape

# colspace

Basis for column space of matrix

## Syntax

```
colspace(A)
```

## Description

`colspace(A)` returns a symbolic matrix whose columns form a basis for the column space of the symbolic matrix `A`.

## Examples

### Compute Basis for Column Space of Symbolic Matrix

Compute the basis for the column space of a symbolic matrix.

```
A = sym([2 0;3 4;0 5]);  
B = colspace(A)
```

```
B =  
[      1,      0]  
[      0,      1]  
[ -15/8, 5/4]
```

## Input Arguments

### A — Input

symbolic matrix

Input, specified as a symbolic matrix.

## Version History

Introduced before R2006a

## See Also

`null`

## combine

Combine terms of identical algebraic structure

### Syntax

```
Y = combine(S)
Y = combine(S,T)
Y = combine( ___, 'IgnoreAnalyticConstraints', true)
```

### Description

`Y = combine(S)` rewrites products of powers in the expression `S` as a single power.

`Y = combine(S,T)` combines multiple calls to the target function `T` in the expression `S`. Use `combine` to implement the inverse functionality of `expand` with respect to the majority of the applied rules.

`Y = combine( ___, 'IgnoreAnalyticConstraints', true)` simplifies the output by applying common mathematical identities, such as  $\log(a) + \log(b) = \log(a*b)$ . These identities might not be valid for all values of the variables, but applying them can return simpler results.

### Examples

#### Powers of the Same Base

Combine powers of the same base.

```
syms x y z
combine(x^y*x^z)
```

```
ans =
x^(y + z)
```

Combine powers of numeric arguments. To prevent MATLAB from evaluating the expression, use `sym` to convert at least one numeric argument into a symbolic value.

```
syms x y
combine(x^(3)*x^y*x^exp(sym(1)))
```

```
ans =
x^(y + exp(1) + 3)
```

Here, `sym` converts 1 into a symbolic value, preventing MATLAB from evaluating the expression  $e^1$ .

#### Powers of the Same Exponent

Combine powers with the same exponents in certain cases.

```
combine(sqrt(sym(2))*sqrt(3))
```

```
ans =
6^(1/2)
```



`combine` does not usually combine the powers because the internal simplifier applies the same rules in the opposite direction to expand the result.

```
syms x y
combine(y^5*x^5)

ans =
x^5*y^5
```

### Terms with Logarithms

Combine terms with logarithms by specifying the target argument as `log`. For real positive numbers, the logarithm of a product equals the sum of the logarithms of its factors.

```
S = log(sym(2)) + log(sym(3));
combine(S, 'log')

ans =
log(6)
```

Try combining  $\log(a) + \log(b)$ . Because  $a$  and  $b$  are assumed to be complex numbers by default, the rule does not hold and `combine` does not combine the terms.

```
syms a b
S = log(a) + log(b);
combine(S, 'log')

ans =
log(a) + log(b)
```

Apply the rule by setting assumptions such that  $a$  and  $b$  satisfy the conditions for the rule.

```
assume(a > 0)
assume(b > 0)
S = log(a) + log(b);
combine(S, 'log')

ans =
log(a*b)
```

For future computations, clear the assumptions set on variables  $a$  and  $b$  by recreating them using `syms`.

```
syms a b
```

Alternatively, apply the rule by ignoring analytic constraints using `'IgnoreAnalyticConstraints'`.

```
syms a b
S = log(a) + log(b);
combine(S, 'log', 'IgnoreAnalyticConstraints', true)

ans =
log(a*b)
```

### Terms with Sine and Cosine Function Calls

Rewrite products of sine and cosine functions as a sum of the functions by setting the target argument to `sincos`.

```
syms a b
combine(sin(a)*cos(b) + sin(b)^2, 'sincos')
```

```
ans =
sin(a + b)/2 - cos(2*b)/2 + sin(a - b)/2 + 1/2
```

Rewrite sums of sine and cosine functions by setting the target argument to `sincos`.

```
combine(cos(a) + sin(a), 'sincos')
```

```
ans =
2^(1/2)*cos(a - pi/4)
```

Rewrite a cosine squared function by setting the target argument to `sincos`.

```
combine(cos(a)^2, 'sincos')
```

```
ans =
cos(2*a)/2 + 1/2
```

`combine` does not rewrite powers of sine or cosine functions with negative integer exponents.

```
syms a b
combine(sin(b)^(-2)*cos(b)^(-2), 'sincos')
```

```
ans =
1/(cos(b)^2*sin(b)^2)
```

### Exponential Terms

Combine terms with exponents by specifying the target argument as `exp`.

```
combine(exp(sym(3))*exp(sym(2)), 'exp')
```

```
ans =
exp(5)
```

```
syms a
combine(exp(a)^3, 'exp')
```

```
ans =
exp(3*a)
```

### Terms with Integrals

Combine terms with integrals by specifying the target argument as `int`.

```
syms a f(x) g(x)
combine(int(f(x),x)+int(g(x),x), 'int')
combine(a*int(f(x),x), 'int')
```

```
ans =
int(f(x) + g(x), x)
ans =
int(a*f(x), x)
```

Combine integrals with the same limits.

```
syms a b h(z)
combine(int(f(x),x,a,b)+int(h(z),z,a,b), 'int')
```

```
ans =
int(f(x) + h(x), x, a, b)
```

### Terms with Inverse Tangent Function Calls

Combine two calls to the inverse tangent function by specifying the target argument as `atan`.

```
syms a b
assume(-1 < a < 1)
assume(-1 < b < 1)
combine(atan(a) + atan(b), 'atan')
```

```
ans =
-atan((a + b)/(a*b - 1))
```

Combine two calls to the inverse tangent function. `combine` simplifies the expression to a symbolic value if possible.

```
assume(a > 0)
combine(atan(a) + atan(1/a), 'atan')
```

```
ans =
pi/2
```

For further computations, clear the assumptions:

```
syms a b
```

### Terms with Calls to Gamma Function

Combine multiple gamma functions by specifying the target as `gamma`.

```
syms x
combine(gamma(x)*gamma(1-x), 'gamma')
```

```
ans =
-pi/sin(pi*(x - 1))
```

`combine` simplifies quotients of gamma functions to rational expressions.

### Multiple Input Expressions in One Call

Evaluate multiple expressions in one function call by using a symbolic matrix as the input parameter.

```
S = [sqrt(sym(2))*sqrt(5), sqrt(2)*sqrt(sym(11))];
combine(S)
```

```
ans =
[ 10^(1/2), 22^(1/2)]
```

## Input Arguments

### S — Input expression

symbolic expression | symbolic vector | symbolic matrix | symbolic function

Input expression, specified as a symbolic expression, function, or as a vector or matrix of symbolic expressions or functions.

`combine` works recursively on subexpressions of `S`.

If `S` is a symbolic matrix, `combine` is applied to all elements of the matrix.

### T — Target function

`'atan' | 'exp' | 'gamma' | 'int' | 'log' | 'sincos' | 'sinhcosh'`

Target function, specified as `'atan'`, `'exp'`, `'gamma'`, `'int'`, `'log'`, `'sincos'`, or `'sinhcosh'`. The rewriting rules apply only to calls to the target function.

## Output Arguments

### Y — Expression with combined functions

symbolic variable | symbolic number | symbolic expression | symbolic vector | symbolic matrix

Expression with the combined functions, returned as a symbolic variable, number, expression, or as a vector or matrix of symbolic variables, numbers, or expressions.

## Algorithms

`combine` applies the following rewriting rules to the input expression `S`, depending on the value of the target argument `T`.

- When `T = 'exp'`, `combine` applies these rewriting rules where valid,

$$e^a e^b = e^{a+b}$$

$$(e^a)^b = e^{ab}.$$

- When `T = 'log'`,

$$\log(a) + \log(b) = \log(ab).$$

If  $b < 1000$ ,

$$b \log(a) = \log(a^b).$$

When  $b \geq 1000$ , `combine` does not apply this second rule.

The rules applied to rewrite logarithms do not hold for arbitrary complex values of `a` and `b`. Specify appropriate properties for `a` or `b` to enable these rewriting rules.

- When `T = 'int'`,

$$a \int f(x) dx = \int af(x) dx$$

$$\int f(x) dx + \int g(x) dx = \int f(x) + g(x) dx$$

$$\int_a^b f(x) dx + \int_a^b g(x) dx = \int_a^b f(x) + g(x) dx$$

$$\int_a^b f(x) dx + \int_a^b g(y) dy = \int_a^b f(y) + g(y) dy$$

$$\int_a^b yf(x)dx + \int_a^b xg(y)dy = \int_a^b yf(c) + xg(c)dc.$$

- When  $T = 'sincos'$ ,

$$\sin(x)\sin(y) = \frac{\cos(x-y)}{2} - \frac{\cos(x+y)}{2}.$$

`combine` applies similar rules for  $\sin(x)\cos(y)$  and  $\cos(x)\cos(y)$ .

$$A\cos(x) + B\sin(x) = A\sqrt{1 + \frac{B^2}{A^2}}\cos\left(x + \tan^{-1}\left(\frac{-B}{A}\right)\right).$$

- When  $T = 'atan'$  and  $-1 < x < 1$ ,  $-1 < y < 1$ ,

$$\operatorname{atan}(x) + \operatorname{atan}(y) = \operatorname{atan}\left(\frac{x+y}{1-xy}\right).$$

- When  $T = 'sinhcosh'$ ,

$$\sinh(x)\sinh(y) = \frac{\cosh(x+y)}{2} - \frac{\cosh(x-y)}{2}.$$

`combine` applies similar rules for  $\sinh(x)\cosh(y)$  and  $\cosh(x)\cosh(y)$ .

`combine` applies the previous rules recursively to powers of  $\sinh$  and  $\cosh$  with positive integral exponents.

- When  $T = 'gamma'$ ,

$$a\Gamma(a) = \Gamma(a+1).$$

and,

$$\frac{\Gamma(a+1)}{\Gamma(a)} = a.$$

For positive integers  $n$ ,

$$\Gamma(-a)\Gamma(a) = -\frac{\pi}{\sin(\pi a)}.$$

## Version History

Introduced in R2014a

## See Also

### Functions

`collect` | `expand` | `factor` | `horner` | `numden` | `rewrite` | `simplifyFraction` | `simplify`

### Live Editor Tasks

Simplify Symbolic Expression

### Topics

"Choose Function to Rearrange Expression" on page 3-123

“Simplify Symbolic Expressions” on page 3-133

“Simplify Symbolic Expressions Using Live Editor Task” on page 3-138

# compose

Functional composition

## Syntax

```
compose(f,g)
compose(f,g,z)
compose(f,g,x,z)
compose(f,g,x,y,z)
```

## Description

`compose(f,g)` returns  $f(g(y))$  where  $f = f(x)$  and  $g = g(y)$ . Here  $x$  is the symbolic variable of  $f$  as defined by `symvar` and  $y$  is the symbolic variable of  $g$  as defined by `symvar`.

`compose(f,g,z)` returns  $f(g(z))$  where  $f = f(x)$ ,  $g = g(y)$ , and  $x$  and  $y$  are the symbolic variables of  $f$  and  $g$  as defined by `symvar`.

`compose(f,g,x,z)` returns  $f(g(z))$  and makes  $x$  the independent variable for  $f$ . That is, if  $f = \cos(x/t)$ , then `compose(f,g,x,z)` returns  $\cos(g(z)/t)$  whereas `compose(f,g,t,z)` returns  $\cos(x/g(z))$ .

`compose(f,g,x,y,z)` returns  $f(g(z))$  and makes  $x$  the independent variable for  $f$  and  $y$  the independent variable for  $g$ . For  $f = \cos(x/t)$  and  $g = \sin(y/u)$ , `compose(f,g,x,y,z)` returns  $\cos(\sin(z/u)/t)$  whereas `compose(f,g,x,u,z)` returns  $\cos(\sin(y/z)/t)$ .

## Examples

### Compose Functions From Expressions

Show functional composition by creating functions from existing functions.

Declare functions.

```
syms x y z t u
f = 1/(1 + x^2);
g = sin(y);
h = x^t;
p = exp(-y/u);
```

Compose functions with different functions and variables as inputs.

```
a = compose(f,g)

a =
1/(sin(y)^2 + 1)

b = compose(f,g,t)

b =
1/(sin(t)^2 + 1)
```

```
c = compose(h,g,x,z)

c =
sin(z)^t

d = compose(h,g,t,z)

d =
x^sin(z)

e = compose(h,p,x,y,z)

e =
exp(-z/u)^t
```

## Input Arguments

### **f — Input**

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **g — Input**

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **x — Symbolic variable**

symbolic variable

Symbolic variable, specified as a symbolic variable.

### **y — Symbolic variable**

symbolic variable

Symbolic variable, specified as a symbolic variable.

### **z — Symbolic variable**

symbolic variable

Symbolic variable, specified as a symbolic variable.

## Version History

**Introduced before R2006a**

### **See Also**

finverse | subs | syms



# cond

Condition number of matrix

## Syntax

```
cond(A)
cond(A,P)
```

## Description

`cond(A)` returns the 2-norm condition number of matrix A.

`cond(A,P)` returns the P-norm condition number of matrix A.

## Examples

### Compute 2-Norm Condition Number of Matrix

Compute the 2-norm condition number of the inverse of the 3-by-3 magic square A.

```
A = inv(sym(magic(3)));
condN2 = cond(A)
```

```
condN2 =
(5*3^(1/2))/2
```

Use `vpa` to approximate the result.

```
vpa(condN2, 20)
```

```
ans =
4.3301270189221932338186158537647
```

### Compute Different Condition Numbers of Matrix

Compute the 1-norm condition number, the Frobenius condition number, and the infinity condition number of the inverse of the 3-by-3 magic square A.

```
A = inv(sym(magic(3)));
condN1 = cond(A, 1)
condNf = cond(A, 'fro')
condNi = cond(A, inf)
```

```
condN1 =
16/3
```

```
condNf =
(285^(1/2)*391^(1/2))/60
```



- `cond(A,2)` or `cond(A)` returns the 2-norm condition number.
- `cond(A,inf)` returns the infinity norm condition number.
- `cond(A,'fro')` returns the Frobenius norm condition number.

## More About

### Condition Number of a Matrix

Condition number of a matrix is the ratio of the largest singular value of that matrix to the smallest singular value. The P-norm condition number of the matrix A is defined as  $\text{norm}(A,P) \cdot \text{norm}(\text{inv}(A),P)$ .

## Tips

- Calling `cond` for a numeric matrix that is not a symbolic object invokes the MATLAB `cond` function.

## Version History

Introduced in R2012b

## See Also

`equationsToMatrix` | `inv` | `linsolve` | `norm` | `rank`

## conj

Complex conjugate of symbolic input

### Syntax

`conj(x)`

### Description

`conj(x)` returns the complex conjugate of  $x$ . Because symbolic scalar variables are complex by default, unresolved calls, such as `conj(x)`, can appear in the output of `norm`, `mtimes`, and other functions. For details, see “Use Assumptions on Symbolic Variables” on page 1-41.

For complex  $x$ ,  $\text{conj}(x) = \text{real}(x) - i*\text{imag}(x)$ .

### Examples

#### Conjugate of Numeric and Symbolic Input

Compute the conjugate of numeric input.

```
conj(1+3i)

ans =
    1.0000 - 3.0000i
```

Compute the conjugate of symbolic input.

```
syms x
f = x^2;
fConj = conj(f)

fConj =
conj(x)^2
```

Convert symbolic output to double by substituting for  $x$  with a number by using `subs`, and then using `double`.

```
fConj = subs(fConj,x,1+2i);           % x is 1+2i
fConj = double(fConj)

fConj =
    -3.0000 - 4.0000i
```

#### Conjugate of Real Inputs Using Assumptions

If the input is real, `conj` returns the input instead of an unresolved call. Assume  $x$  is real and find its conjugate. `conj` returns  $x$  instead of `conj(x)`, as expected.

```
syms x
assume(x,'real')
conj(x)
```

```
ans =  
x
```

Clear the assumption for further computations.

```
assume(x, 'clear')
```

## Input Arguments

### **x — Input**

number | vector | matrix | array | symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic array | symbolic function | symbolic matrix function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, scalar variable, matrix variable, array, function, matrix function, or expression.

## Version History

**Introduced before R2006a**

### **R2022a: Complex conjugate of symbolic matrix functions**

The `conj` function accepts an input argument of type `symfunmatrix`.

### **R2021a: Complex conjugate of symbolic matrix variables**

The `conj` function accepts an input argument of type `symmatrix`.

## See Also

`real` | `imag`

## COS

Symbolic cosine function

### Syntax

`cos(X)`

### Description

`cos(X)` returns the cosine function on page 6-208 of *X*.

### Examples

#### Cosine Function for Numeric and Symbolic Arguments

Depending on its arguments, `cos` returns floating-point or exact symbolic results.

Compute the cosine function for these numbers. Because these numbers are not symbolic objects, `cos` returns floating-point results.

```
A = cos([-2, -pi, pi/6, 5*pi/7, 11])  
  
A =  
   -0.4161   -1.0000    0.8660   -0.6235    0.0044
```

Compute the cosine function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `cos` returns unresolved symbolic calls.

```
symA = cos(sym([-2, -pi, pi/6, 5*pi/7, 11]))  
  
symA =  
[ cos(2), -1, 3^(1/2)/2, -cos((2*pi)/7), cos(11)]
```

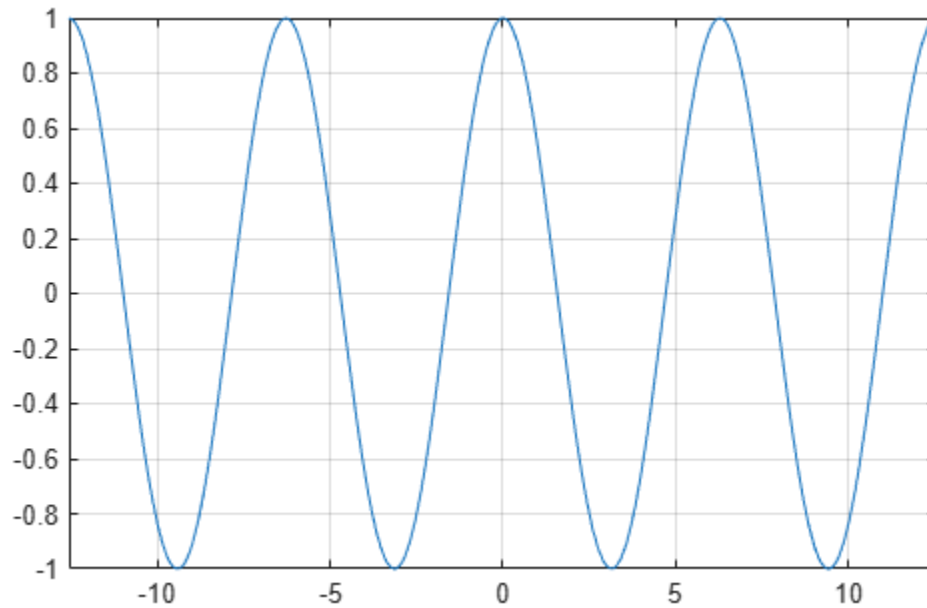
Use `vpa` to approximate symbolic results with floating-point numbers:

```
vpa(symA)  
  
ans =  
[ -0.41614683654714238699756822950076,...  
  -1.0,...  
  0.86602540378443864676372317075294,...  
  -0.62348980185873353052500488400424,...  
  0.0044256979880507857483550247239416]
```

#### Plot Cosine Function

Plot the cosine function on the interval from  $-4\pi$  to  $4\pi$ .

```
syms x  
fplot(cos(x),[-4*pi 4*pi])  
grid on
```



### Handle Expressions Containing Cosine Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `cos`.

Find the first and second derivatives of the cosine function:

```
syms x
diff(cos(x), x)
diff(cos(x), x, x)
```

```
ans =
-sin(x)
```

```
ans =
-cos(x)
```

Find the indefinite integral of the cosine function:

```
int(cos(x), x)
```

```
ans =
sin(x)
```

Find the Taylor series expansion of `cos(x)`:

```
taylor(cos(x), x)
```

```
ans =
x^4/24 - x^2/2 + 1
```

Rewrite the cosine function in terms of the exponential function:

```
rewrite(cos(x), 'exp')
```

```
ans =  
exp(-x*1i)/2 + exp(x*1i)/2
```

### Evaluate Units with cos Function

cos numerically evaluates these units automatically: radian, degree, arcmin, arcsec, and revolution.

Show this behavior by finding the cosine of x degrees and 2 radians.

```
u = symunit;  
syms x  
f = [x*u.degree 2*u.radian];  
cosinf = cos(f)  
  
cosinf =  
[ cos((pi*x)/180), cos(2)]
```

You can calculate cosinf by substituting for x using subs and then using double or vpa.

## Input Arguments

### X — Input

symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic expression |  
symbolic function | symbolic matrix function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, scalar variable, matrix variable, expression, function, matrix function, or as a vector or matrix of symbolic numbers, scalar variables, expressions, or functions.

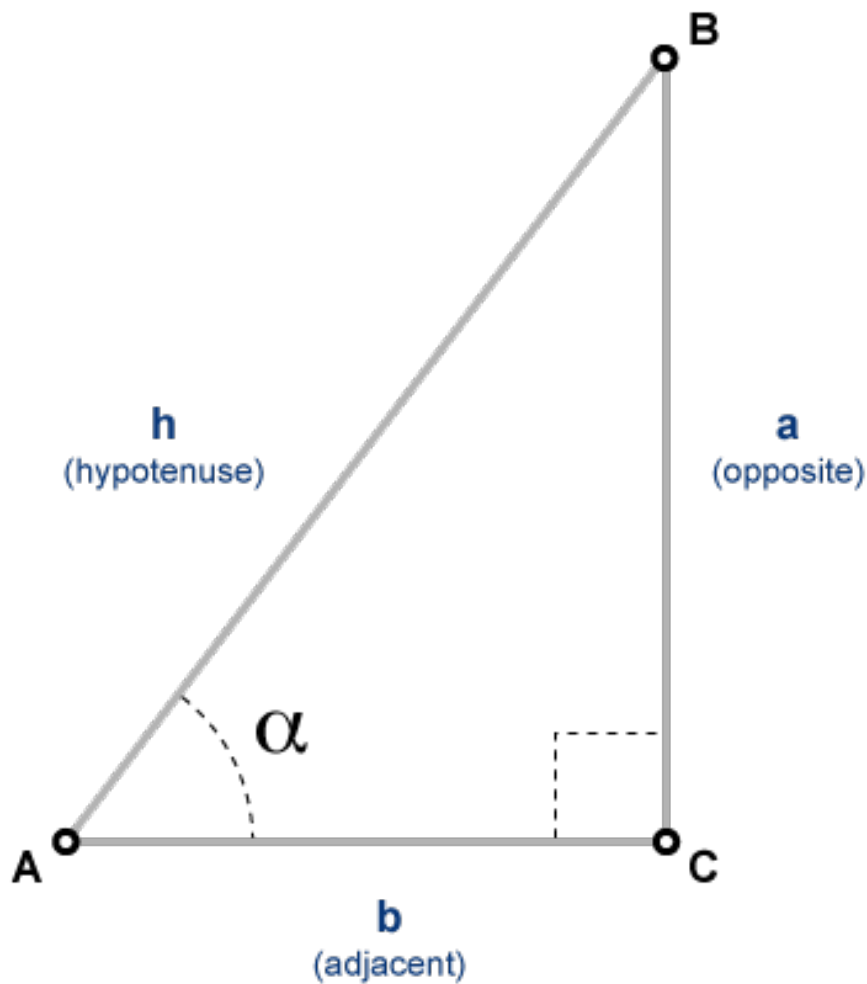
## More About

### Cosine Function

The cosine of an angle,  $\alpha$ , defined with reference to a right triangle is

$$\cos(\alpha) = \frac{\text{adjacent side}}{\text{hypotenuse}} = \frac{b}{h}.$$





The cosine of a complex argument,  $\alpha$ , is

$$\cos(\alpha) = \frac{e^{i\alpha} + e^{-i\alpha}}{2}.$$

## Version History

Introduced before R2006a

**R2022a: Compute cosine of symbolic matrix functions**

The `cos` function accepts an input argument of type `symfunmatrix`.

**R2021a: Compute cosine of symbolic matrix variables**

The `cos` function accepts an input argument of type `symmatrix`.

**See Also**

`acos` | `asin` | `atan` | `acsc` | `asec` | `acot` | `sin` | `tan` | `csc` | `sec` | `cot`

# cosh

Symbolic hyperbolic cosine function

## Syntax

`cosh(X)`

## Description

`cosh(X)` returns the hyperbolic cosine function of  $X$ .

## Examples

### Hyperbolic Cosine Function for Numeric and Symbolic Arguments

Depending on its arguments, `cosh` returns floating-point or exact symbolic results.

Compute the hyperbolic cosine function for these numbers. Because these numbers are not symbolic objects, `cosh` returns floating-point results.

```
A = cosh([-2, -pi*i, pi*i/6, 5*pi*i/7, 3*pi*i/2])
A =
    3.7622    -1.0000    0.8660   -0.6235   -0.0000
```

Compute the hyperbolic cosine function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `cosh` returns unresolved symbolic calls.

```
symA = cosh(sym([-2, -pi*i, pi*i/6, 5*pi*i/7, 3*pi*i/2]))
symA =
[ cosh(2), -1, 3^(1/2)/2, -cosh((pi*2i)/7), 0]
```

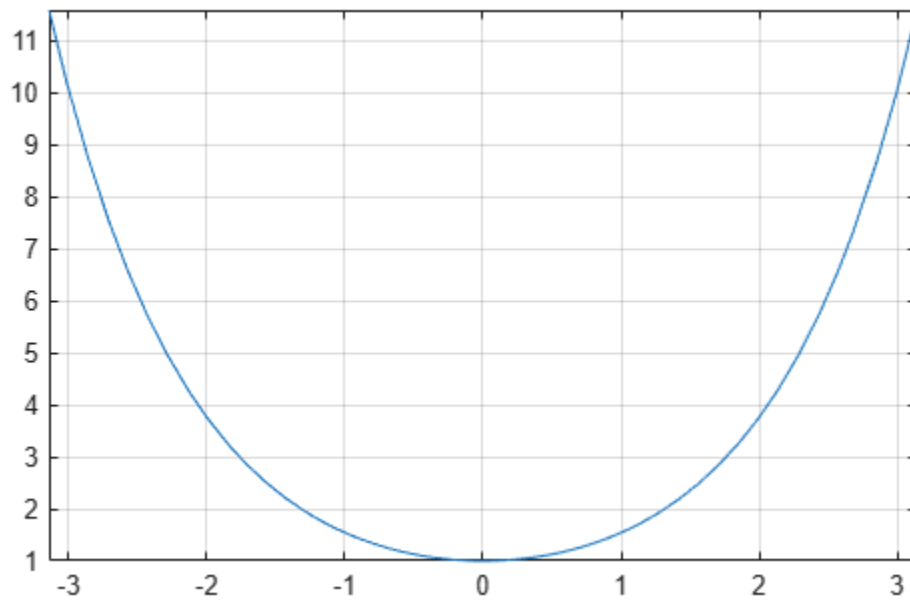
Use `vpa` to approximate symbolic results with floating-point numbers:

```
vpa(symA)
ans =
[ 3.7621956910836314595622134777737,...
 -1.0,...
 0.86602540378443864676372317075294,...
 -0.62348980185873353052500488400424,...
 0]
```

### Plot Hyperbolic Cosine Function

Plot the hyperbolic cosine function on the interval from  $-\pi$  to  $\pi$ .

```
syms x
fplot(cosh(x),[-pi pi])
grid on
```



### Handle Expressions Containing Hyperbolic Cosine Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `cosh`.

Find the first and second derivatives of the hyperbolic cosine function:

```
syms x
diff(cosh(x), x)
diff(cosh(x), x, x)
```

```
ans =
sinh(x)
```

```
ans =
cosh(x)
```

Find the indefinite integral of the hyperbolic cosine function:

```
int(cosh(x), x)
```

```
ans =
sinh(x)
```

Find the Taylor series expansion of  $\cosh(x)$ :

```
taylor(cosh(x), x)
```

```
ans =
x^4/24 + x^2/2 + 1
```

Rewrite the hyperbolic cosine function in terms of the exponential function:

```
rewrite(cosh(x), 'exp')
```

```
ans =  
exp(-x)/2 + exp(x)/2
```

## Input Arguments

### **X — Input**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Version History

**Introduced before R2006a**

### **See Also**

acosh | asinh | atanh | acsch | asech | acoth | sinh | tanh | csch | sech | coth

## coshint

Hyperbolic cosine integral function

### Syntax

`coshint(X)`

### Description

`coshint(X)` returns the hyperbolic cosine integral function on page 6-216 of X.

### Examples

#### Hyperbolic Cosine Integral Function for Numeric and Symbolic Arguments

Depending on its arguments, `coshint` returns floating-point or exact symbolic results.

Compute the hyperbolic cosine integral function for these numbers. Because these numbers are not symbolic objects, `coshint` returns floating-point results.

```
A = coshint([-1, 0, 1/2, 1, pi/2, pi])
```

```
A =
    0.8379 + 3.1416i    -Inf + 0.0000i    -0.0528 + 0.0000i    0.8379...
    + 0.0000i    1.7127 + 0.0000i    5.4587 + 0.0000i
```

Compute the hyperbolic cosine integral function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `coshint` returns unresolved symbolic calls.

```
symA = coshint(sym([-1, 0, 1/2, 1, pi/2, pi]))
```

```
symA =
[ coshint(1) + pi*1i, -Inf, coshint(1/2), coshint(1), coshint(pi/2), coshint(pi)]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

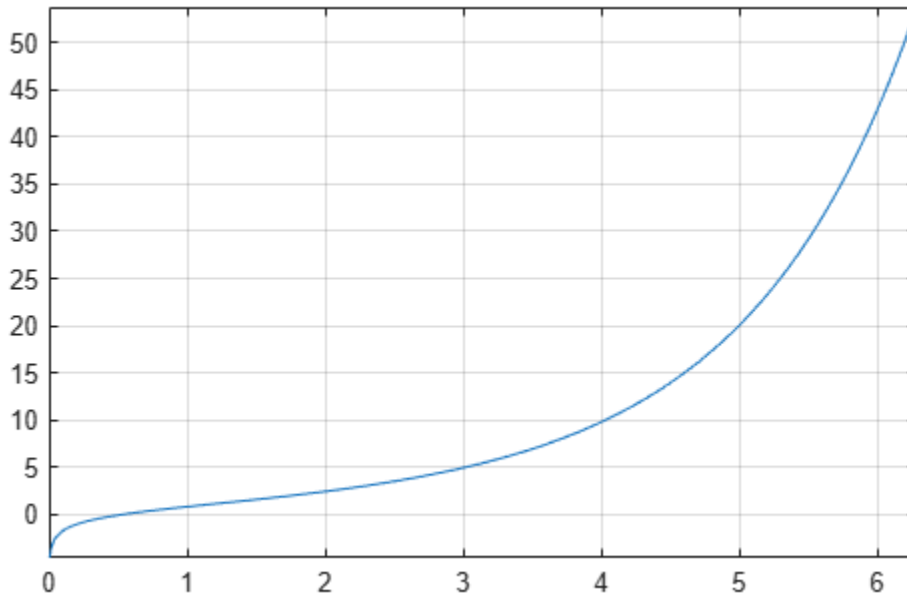
```
vpa(symA)
```

```
ans =
[ 0.83786694098020824089467857943576...
  + 3.1415926535897932384626433832795i,...
 -Inf,...
 -0.052776844956493615913136063326141,...
 0.83786694098020824089467857943576,...
 1.7126607364844281079951569897796,...
 5.4587340442160681980014878977798]
```

#### Plot Hyperbolic Cosine Integral Function

Plot the hyperbolic cosine integral function on the interval from 0 to  $2\pi$ .

```
syms x
fplot(coshint(x),[0 2*pi])
grid on
```



### Handle Expressions Containing Hyperbolic Cosine Integral Function

Many functions, such as `diff` and `int`, can handle expressions containing `coshint`.

Find the first and second derivatives of the hyperbolic cosine integral function:

```
syms x
diff(coshint(x), x)
diff(coshint(x), x, x)
```

```
ans =
cosh(x)/x
```

```
ans =
sinh(x)/x - cosh(x)/x^2
```

Find the indefinite integral of the hyperbolic cosine integral function:

```
int(coshint(x), x)

ans =
x*coshint(x) - sinh(x)
```

### Input Arguments

#### X — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## More About

### Hyperbolic Cosine Integral Function

The hyperbolic cosine integral function is defined as follows:

$$\text{Chi}(x) = \gamma + \log(x) + \int_0^x \frac{\cosh(t) - 1}{t} dt$$

Here,  $\gamma$  is the Euler-Mascheroni constant:

$$\gamma = \lim_{n \rightarrow \infty} \left( \left( \sum_{k=1}^n \frac{1}{k} \right) - \ln(n) \right)$$

## Version History

Introduced in R2014a

## References

- [1] Cautschi, W. and W. F. Cahill. "Exponential Integral and Related Functions." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

`cos` | `cosint` | `eulergamma` | `int` | `sinhint` | `sinint` | `ssinint`



# cosint

Cosine integral function

## Syntax

`cosint(X)`

## Description

`cosint(X)` returns the cosine integral function on page 6-219 of X.

## Examples

### Cosine Integral Function for Numeric and Symbolic Arguments

Depending on its arguments, `cosint` returns floating-point or exact symbolic results.

Compute the cosine integral function for these numbers. Because these numbers are not symbolic objects, `cosint` returns floating-point results.

```
A = cosint([- 1, 0, pi/2, pi, 1])
```

```
A =
    0.3374 + 3.1416i    -Inf + 0.0000i    0.4720 + 0.0000i...
    0.0737 + 0.0000i    0.3374 + 0.0000i
```

Compute the cosine integral function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `cosint` returns unresolved symbolic calls.

```
symA = cosint(sym([- 1, 0, pi/2, pi, 1]))
```

```
symA =
[ cosint(1) + pi*1i, -Inf, cosint(pi/2), cosint(pi), cosint(1)]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

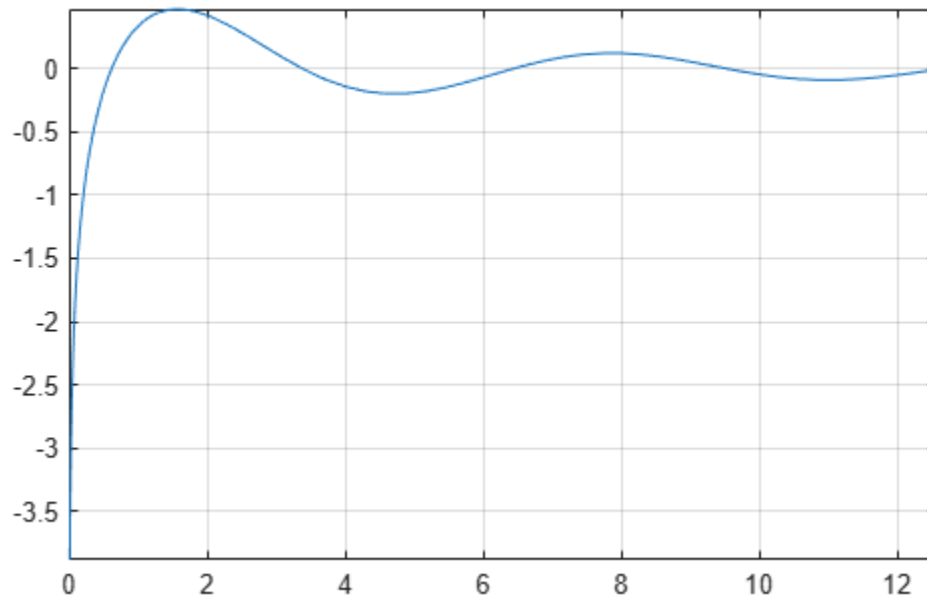
```
vpa(symA)
```

```
ans =
[ 0.33740392290096813466264620388915...
+ 3.1415926535897932384626433832795i,...
-Inf,...
0.47200065143956865077760610761413,...
0.07366791204642548599010096523015,...
0.33740392290096813466264620388915]
```

### Plot Cosine Integral Function

Plot the cosine integral function on the interval from 0 to  $4\pi$ .

```
syms x
fplot(cosint(x),[0 4*pi])
grid on
```



### Handle Expressions Containing Cosine Integral Function

Many functions, such as `diff` and `int`, can handle expressions containing `cosint`.

Find the first and second derivatives of the cosine integral function:

```
syms x
diff(cosint(x), x)
diff(cosint(x), x, x)
```

```
ans =
cos(x)/x
```

```
ans =
- cos(x)/x^2 - sin(x)/x
```

Find the indefinite integral of the cosine integral function:

```
int(cosint(x), x)

ans =
x*cosint(x) - sin(x)
```

### Input Arguments

#### X — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## More About

### Cosine Integral Function

The cosine integral function is defined as follows:

$$\text{Ci}(x) = \gamma + \log(x) + \int_0^x \frac{\cos(t) - 1}{t} dt$$

Here,  $\gamma$  is the Euler-Mascheroni constant:

$$\gamma = \lim_{n \rightarrow \infty} \left( \left( \sum_{k=1}^n \frac{1}{k} \right) - \ln(n) \right)$$

## Version History

Introduced before R2006a

## References

- [1] Gautschi, W. and W. F. Cahill. "Exponential Integral and Related Functions." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

cos | coshint | eulergamma | int | sinhint | sinint | ssinint

## cot

Symbolic cotangent function

### Syntax

`cot(X)`

### Description

`cot(X)` returns the cotangent function on page 6-222 of X.

### Examples

#### Cotangent Function for Numeric and Symbolic Arguments

Depending on its arguments, `cot` returns floating-point or exact symbolic results.

Compute the cotangent function for these numbers. Because these numbers are not symbolic objects, `cot` returns floating-point results.

```
A = cot([-2, -pi/2, pi/6, 5*pi/7, 11])  
  
A =  
    0.4577    -0.0000    1.7321    -0.7975    -0.0044
```

Compute the cotangent function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `cot` returns unresolved symbolic calls.

```
symA = cot(sym([-2, -pi/2, pi/6, 5*pi/7, 11]))  
  
symA =  
[ -cot(2), 0, 3^(1/2), -cot((2*pi)/7), cot(11)]
```

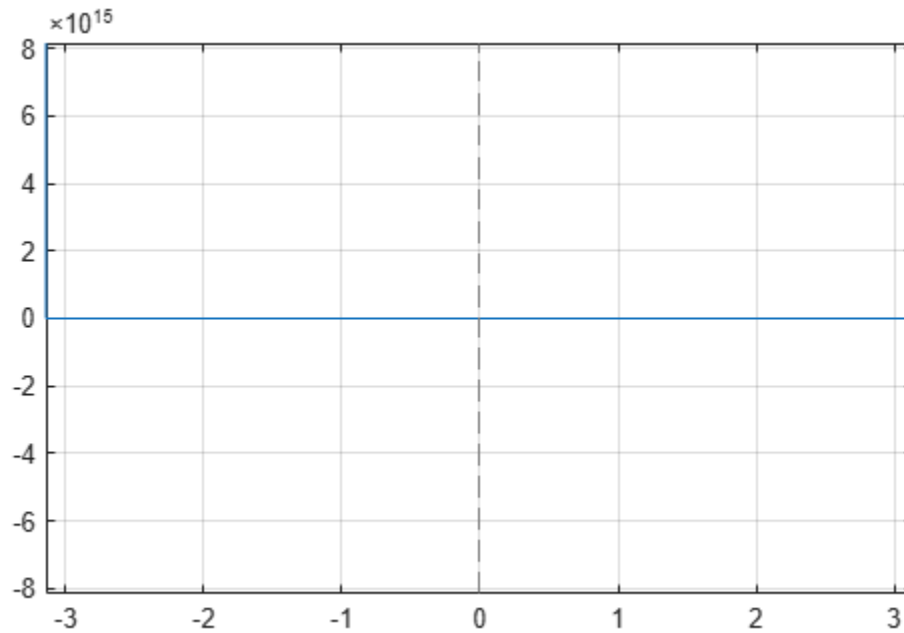
Use `vpa` to approximate symbolic results with floating-point numbers:

```
vpa(symA)  
  
ans =  
[ 0.45765755436028576375027741043205,...  
 0,...  
 1.7320508075688772935274463415059,...  
 -0.79747338888240396141568825421443,...  
 -0.0044257413313241136855482762848043]
```

#### Plot Cotangent Function

Plot the cotangent function on the interval from  $-\pi$  to  $\pi$ .

```
syms x  
fplot(cot(x),[-pi pi])  
grid on
```



### Handle Expressions Containing Cotangent Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `cot`.

Find the first and second derivatives of the cotangent function:

```
syms x
diff(cot(x), x)
diff(cot(x), x, x)

ans =
- cot(x)^2 - 1

ans =
2*cot(x)*(cot(x)^2 + 1)
```

Find the indefinite integral of the cotangent function:

```
int(cot(x), x)

ans =
log(sin(x))
```

Find the Taylor series expansion of `cot(x)` around  $x = \pi/2$ :

```
taylor(cot(x), x, pi/2)

ans =
pi/2 - x - (x - pi/2)^3/3 - (2*(x - pi/2)^5)/15
```

Rewrite the cotangent function in terms of the sine and cosine functions:

```
rewrite(cot(x), 'sincos')
```

```
ans =  
cos(x)/sin(x)
```

Rewrite the cotangent function in terms of the exponential function:

```
rewrite(cot(x), 'exp')  
  
ans =  
(exp(x*2i)*1i + 1i)/(exp(x*2i) - 1)
```

### Evaluate Units with cot Function

`cot` numerically evaluates these units automatically: radian, degree, arcmin, arcsec, and revolution.

Show this behavior by finding the cotangent of  $x$  degrees and 2 radians.

```
u = symunit;  
syms x  
f = [x*u.degree 2*u.radian];  
cotf = cot(f)  
  
cotf =  
[ cot((pi*x)/180), cot(2)]
```

You can calculate `cotf` by substituting for  $x$  using `subs` and then using `double` or `vpa`.

## Input Arguments

### X — Input

symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic expression |  
symbolic function | symbolic matrix function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, scalar variable, matrix variable, expression, function, matrix function, or as a vector or matrix of symbolic numbers, scalar variables, expressions, or functions.

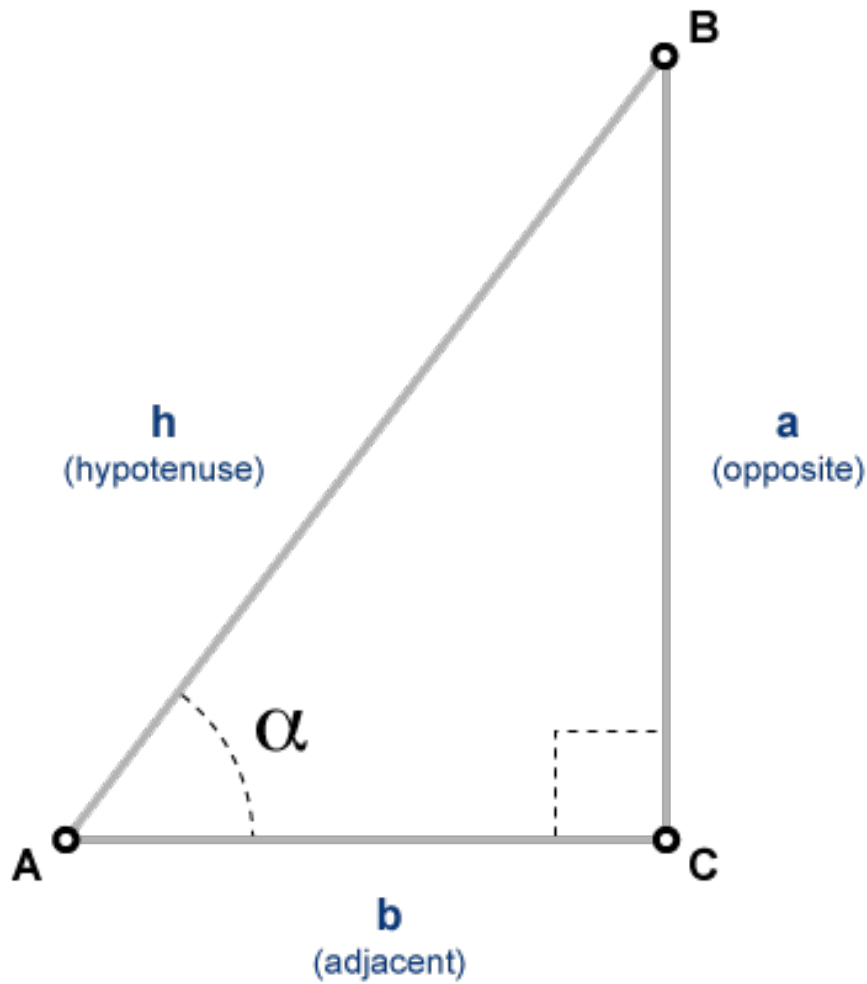
## More About

### Cotangent Function

The cotangent of an angle,  $\alpha$ , defined with reference to a right angled triangle is

$$\cot(\alpha) = \frac{1}{\tan(\alpha)} = \frac{\text{adjacent side}}{\text{opposite side}} = \frac{b}{a} .$$

.



The cotangent of a complex argument  $\alpha$  is

$$\cot(\alpha) = \frac{i(e^{i\alpha} + e^{-i\alpha})}{(e^{i\alpha} - e^{-i\alpha})}.$$

## Version History

Introduced before R2006a

**R2022a: Compute cotangent of symbolic matrix functions**

The `cot` function accepts an input argument of type `symfunmatrix`.

**R2021b: Compute cotangent of symbolic matrix variables**

The `cot` function accepts an input argument of type `symmatrix`.

**See Also**

`acos` | `asin` | `atan` | `acsc` | `asec` | `acot` | `sin` | `cos` | `tan` | `csc` | `sec`



# coth

Symbolic hyperbolic cotangent function

## Syntax

`coth(X)`

## Description

`coth(X)` returns the hyperbolic cotangent function of  $X$

## Examples

### Hyperbolic Cotangent Function for Numeric and Symbolic Arguments

Depending on its arguments, `coth` returns floating-point or exact symbolic results.

Compute the hyperbolic cotangent function for these numbers. Because these numbers are not symbolic objects, `coth` returns floating-point results.

```
A = coth([-2, -pi*i/3, pi*i/6, 5*pi*i/7, 3*pi*i/2])
A =
-1.0373 + 0.0000i    0.0000 + 0.5774i    0.0000 - 1.7321i...
0.0000 + 0.7975i    0.0000 - 0.0000i
```

Compute the hyperbolic cotangent function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `coth` returns unresolved symbolic calls.

```
symA = coth(sym([-2, -pi*i/3, pi*i/6, 5*pi*i/7, 3*pi*i/2]))
symA =
[ -coth(2), (3^(1/2)*1i)/3, -3^(1/2)*1i, -coth((pi*2i)/7), 0]
```

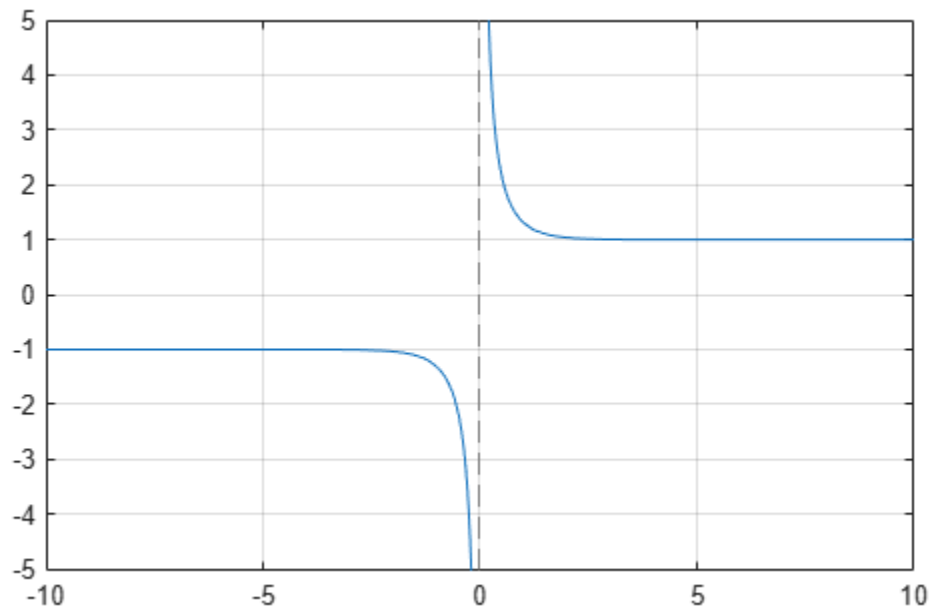
Use `vpa` to approximate symbolic results with floating-point numbers:

```
vpa(symA)
ans =
[ -1.0373147207275480958778097647678,...
0.57735026918962576450914878050196i,...
-1.7320508075688772935274463415059i,...
0.79747338888240396141568825421443i,...
0]
```

### Plot Hyperbolic Cotangent Function

Plot the hyperbolic cotangent function on the interval from -10 to 10.

```
syms x
fplot(coth(x),[-10 10])
grid on
```



### Handle Expressions Containing Hyperbolic Cotangent Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `coth`.

Find the first and second derivatives of the hyperbolic cotangent function:

```
syms x
diff(coth(x), x)
diff(coth(x), x, x)
```

```
ans =
1 - coth(x)^2
```

```
ans =
2*coth(x)*(coth(x)^2 - 1)
```

Find the indefinite integral of the hyperbolic cotangent function:

```
int(coth(x), x)
```

```
ans =
log(sinh(x))
```

Find the Taylor series expansion of `coth(x)` around  $x = \pi i/2$ :

```
taylor(coth(x), x, pi*i/2)
```

```
ans =
x - (pi*1i)/2 - (x - (pi*1i)/2)^3/3 + (2*(x - (pi*1i)/2)^5)/15
```

Rewrite the hyperbolic cotangent function in terms of the exponential function:

```
rewrite(coth(x), 'exp')
```

```
ans =  
(exp(2*x) + 1)/(exp(2*x) - 1)
```

## Input Arguments

### **X — Input**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Version History

**Introduced before R2006a**

### **See Also**

acosh | asinh | atanh | acsch | asech | acoth | sinh | cosh | tanh | csch | sech

## CSC

Symbolic cosecant function

### Syntax

`csc(X)`

### Description

`csc(X)` returns the cosecant function on page 6-230 of *X*.

### Examples

#### Cosecant Function for Numeric and Symbolic Arguments

Depending on its arguments, `csc` returns floating-point or exact symbolic results.

Compute the cosecant function for these numbers. Because these numbers are not symbolic objects, `csc` returns floating-point results.

```
A = csc([-2, -pi/2, pi/6, 5*pi/7, 11])  
A =  
   -1.0998   -1.0000    2.0000    1.2790   -1.0000
```

Compute the cosecant function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `csc` returns unresolved symbolic calls.

```
symA = csc(sym([-2, -pi/2, pi/6, 5*pi/7, 11]))  
symA =  
[ -1/sin(2), -1, 2, 1/sin((2*pi)/7), 1/sin(11)]
```

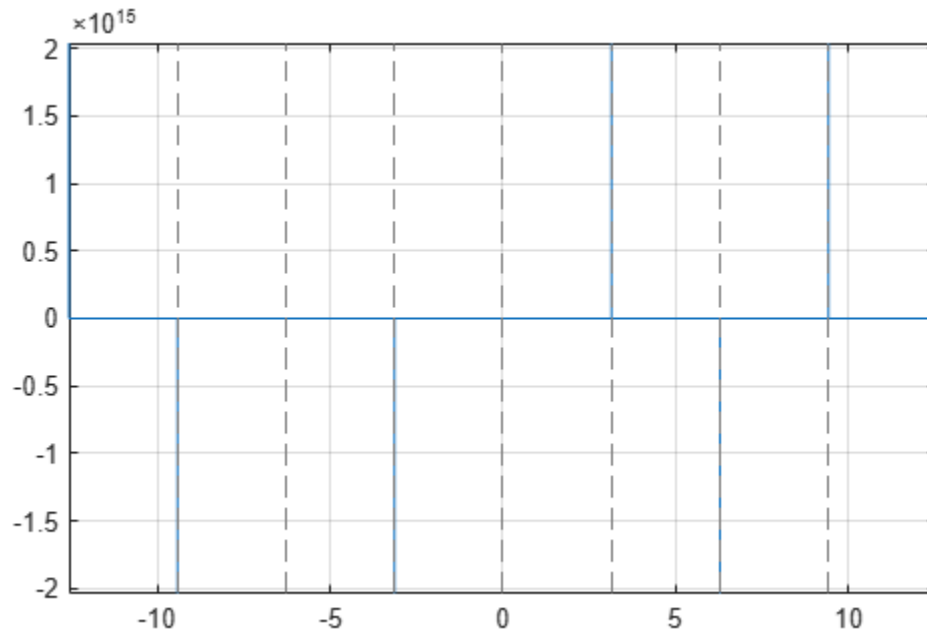
Use `vpa` to approximate symbolic results with floating-point numbers:

```
vpa(symA)  
ans =  
[ -1.0997501702946164667566973970263,...  
  -1.0,...  
   2.0,...  
  1.2790480076899326057478506072714,...  
 -1.0000097935452091313874644503551]
```

#### Plot Cosecant Function

Plot the cosecant function on the interval from  $-4\pi$  to  $4\pi$ .

```
syms x  
fplot(csc(x),[-4*pi 4*pi])  
grid on
```



### Handle Expressions Containing Cosecant Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `csc`.

Find the first and second derivatives of the cosecant function:

```
syms x
diff(csc(x), x)
diff(csc(x), x, x)

ans =
-cos(x)/sin(x)^2

ans =
1/sin(x) + (2*cos(x)^2)/sin(x)^3
```

Find the indefinite integral of the cosecant function:

```
int(csc(x), x)

ans =
log(tan(x/2))
```

Find the Taylor series expansion of `csc(x)` around  $x = \pi/2$ :

```
taylor(csc(x), x, pi/2)

ans =
(x - pi/2)^2/2 + (5*(x - pi/2)^4)/24 + 1
```

Rewrite the cosecant function in terms of the exponential function:

```
rewrite(csc(x), 'exp')
```

```
ans =  
1/((exp(-x*1i)*1i)/2 - (exp(x*1i)*1i)/2)
```

### Evaluate Units with csc Function

csc numerically evaluates these units automatically: radian, degree, arcmin, arcsec, and revolution.

Show this behavior by finding the cosecant of x degrees and 2 radians.

```
u = symunit;  
syms x  
f = [x*u.degree 2*u.radian];  
cosecf = csc(f)  
  
cosecf =  
[ 1/sin((pi*x)/180), 1/sin(2)]
```

You can calculate cosecf by substituting for x using subs and then using double or vpa.

## Input Arguments

### X — Input

symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic expression |  
symbolic function | symbolic matrix function | symbolic vector | symbolic matrix

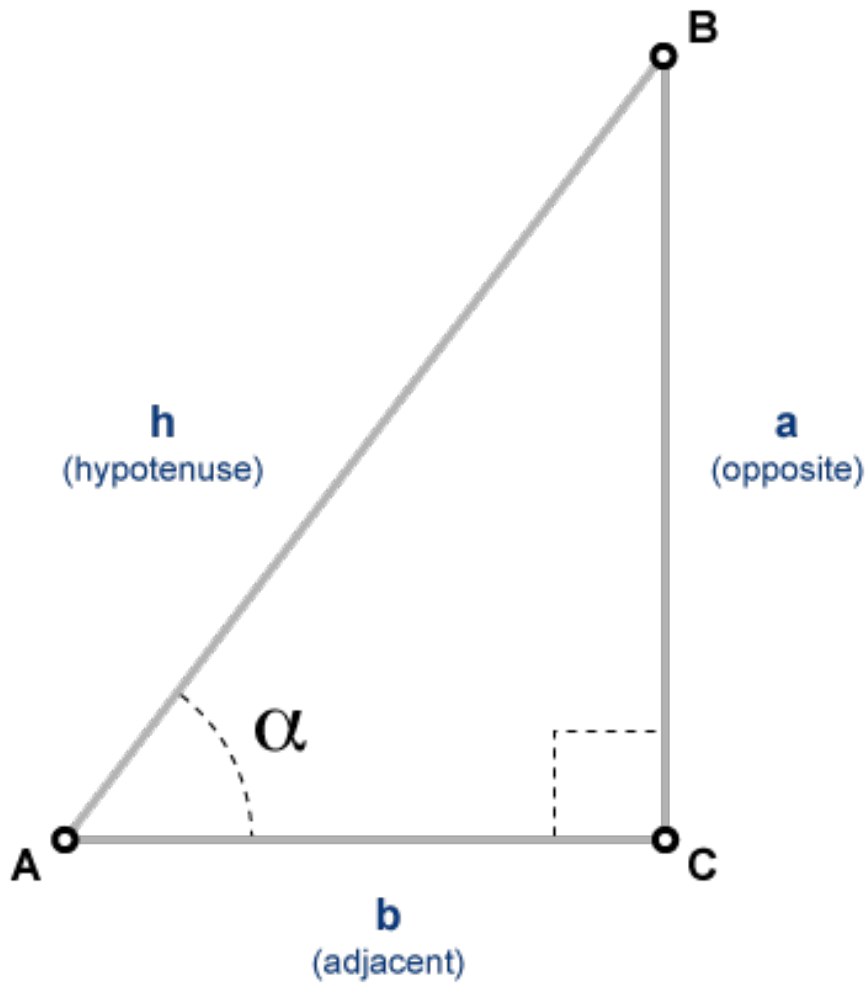
Input, specified as a symbolic number, scalar variable, matrix variable, expression, function, matrix function, or as a vector or matrix of symbolic numbers, scalar variables, expressions, or functions.

## More About

### Cosecant Function

The cosecant of an angle,  $\alpha$ , defined with reference to a right angled triangle is

$$\csc(\alpha) = \frac{1}{\sin(\alpha)} = \frac{\text{hypotenuse}}{\text{opposite side}} = \frac{h}{a} .$$



The cosecant of a complex argument,  $\alpha$ , is

$$\csc(\alpha) = \frac{2i}{e^{i\alpha} - e^{-i\alpha}}.$$

## Version History

Introduced before R2006a

**R2022a: Compute cosecant of symbolic matrix functions**

The `csc` function accepts an input argument of type `symfunmatrix`.

**R2021b: Compute cosecant of symbolic matrix variables**

The `csc` function accepts an input argument of type `symmatrix`.

**See Also**

`acos` | `asin` | `atan` | `acsc` | `asec` | `acot` | `sin` | `cos` | `tan` | `cot`



# csch

Symbolic hyperbolic cosecant function

## Syntax

`csch(X)`

## Description

`csch(X)` returns the hyperbolic cosecant function of  $X$ .

## Examples

### Hyperbolic Cosecant Function for Numeric and Symbolic Arguments

Depending on its arguments, `csch` returns floating-point or exact symbolic results.

Compute the hyperbolic cosecant function for these numbers. Because these numbers are not symbolic objects, `csch` returns floating-point results.

```
A = csch([-2, -pi*i/2, 0, pi*i/3, 5*pi*i/7, pi*i/2])
A =
-0.2757 + 0.0000i    0.0000 + 1.0000i    Inf + 0.0000i...
0.0000 - 1.1547i    0.0000 - 1.2790i    0.0000 - 1.0000i
```

Compute the hyperbolic cosecant function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `csch` returns unresolved symbolic calls.

```
symA = csch(sym([-2, -pi*i/2, 0, pi*i/3, 5*pi*i/7, pi*i/2]))
symA =
[-1/sinh(2), 1i, Inf, -(3^(1/2)*2i)/3, 1/sinh((pi*2i)/7), -1i]
```

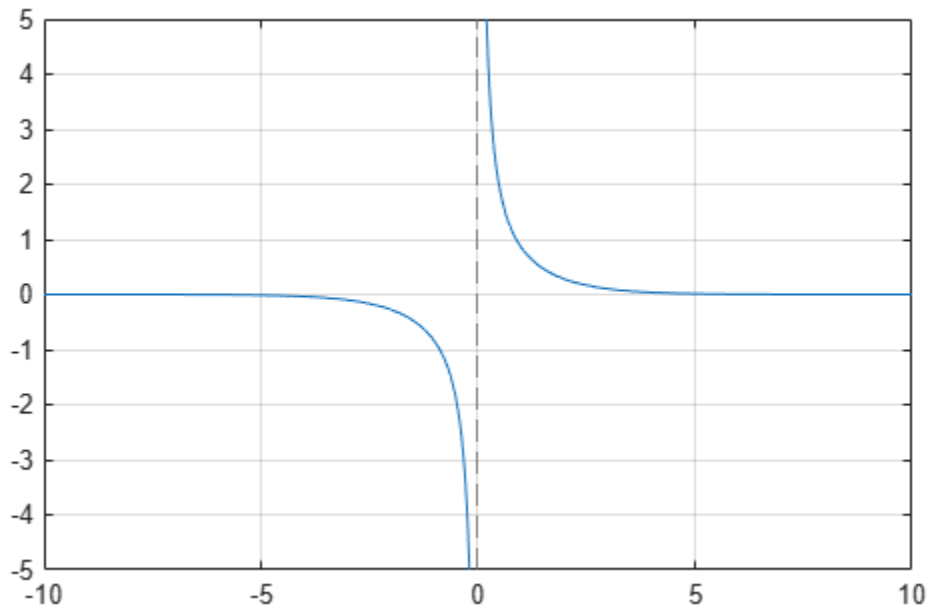
Use `vpa` to approximate symbolic results with floating-point numbers:

```
vpa(symA)
ans =
[-0.27572056477178320775835148216303,...
1.0i,...
Inf,...
-1.1547005383792515290182975610039i,...
-1.2790480076899326057478506072714i,...
-1.0i]
```

### Plot Hyperbolic Cosecant Function

Plot the hyperbolic cosecant function on the interval from -10 to 10.

```
syms x
fplot(csch(x), [-10 10])
grid on
```



### Handle Expressions Containing Hyperbolic Cosecant Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `csch`.

Find the first and second derivatives of the hyperbolic cosecant function:

```
syms x
diff(csch(x), x)
diff(csch(x), x, x)

ans =
-cosh(x)/sinh(x)^2

ans =
(2*cosh(x)^2)/sinh(x)^3 - 1/sinh(x)
```

Find the indefinite integral of the hyperbolic cosecant function:

```
int(csch(x), x)

ans =
log(tanh(x/2))
```

Find the Taylor series expansion of `csch(x)` around  $x = \pi i/2$ :

```
taylor(csch(x), x, pi*i/2)

ans =
((x - (pi*1i)/2)^2*1i)/2 - ((x - (pi*1i)/2)^4*5i)/24 - 1i
```

Rewrite the hyperbolic cosecant function in terms of the exponential function:

```
rewrite(csch(x), 'exp')
```

```
ans =  
-1/(exp(-x)/2 - exp(x)/2)
```

## Input Arguments

### **X — Input**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Version History

**Introduced before R2006a**

### **See Also**

acosh | asinh | atanh | acsch | asech | acoth | sinh | cosh | tanh | sech | coth

## ctranspose, '

Symbolic matrix complex conjugate transpose

### Syntax

```
A'  
ctranspose(A)
```

### Description

A' computes the complex conjugate transpose on page 6-237 of A.

ctranspose(A) is equivalent to A'.

### Examples

#### Conjugate Transpose of Real Matrix

Create a 2-by-3 matrix, the elements of which represent real numbers.

```
syms x y real  
A = [x x x; y y y]  
  
A =  
[ x, x, x]  
[ y, y, y]
```

Find the complex conjugate transpose of this matrix.

```
A'  
  
ans =  
[ x, y]  
[ x, y]  
[ x, y]
```

If all elements of a matrix represent real numbers, then its complex conjugate transform equals to its nonconjugate transform.

```
isAlways(A' == A.)
```

```
ans =  
 3x2 logical array  
 1     1  
 1     1  
 1     1
```

#### Conjugate Transpose of Complex Matrix

Create a 2-by-2 matrix, the elements of which represent complex numbers.

```
syms x y real  
A = [x + y*i x - y*i; y + x*i y - x*i]
```

```
A =
[ x + y*1i, x - y*1i]
[ y + x*1i, y - x*1i]
```

Find the conjugate transpose of this matrix. The complex conjugate transpose operator,  $A'$ , performs a transpose and negates the sign of the imaginary portion of the complex elements in  $A$ .

$A'$

```
ans =
[ x - y*1i, y - x*1i]
[ x + y*1i, y + x*1i]
```

For a matrix of complex numbers with nonzero imaginary parts, the complex conjugate transform is not equal to the nonconjugate transform.

```
isAlways(A' == A.', 'Unknown', 'false')
```

```
ans =
    2x2 logical array
     0     0
     0     0
```

## Input Arguments

### A — Input

number | symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix | symbolic array

Input, specified as a number, or a symbolic number, scalar variable, matrix variable, function, matrix function, expression, or vector, matrix, or array of symbolic scalar variables.

## More About

### Complex Conjugate Transpose

The complex conjugate transpose of a matrix interchanges the row and column index for each element, reflecting the elements across the main diagonal. The operation also negates the imaginary part of any complex numbers.

For example, if  $B = A'$  and  $A(1,2)$  is  $1+1i$ , then the element  $B(2,1)$  is  $1-1i$ .

## Version History

Introduced before R2006a

### R2022a: Complex conjugate transpose of symbolic matrix functions

The `ctranspose` function accepts an input argument of type `symfunmatrix`.

### R2021a: Complex conjugate transpose of symbolic matrix variables

The `ctranspose` function accepts an input argument of type `symmatrix`.

**See Also**

`ldivide` | `minus` | `mldivide` | `mpower` | `mrdivide` | `mtimes` | `plus` | `power` | `rdivide` | `times` | `transpose`

# cumprod

Symbolic cumulative product

## Syntax

```
B = cumprod(A)
B = cumprod(A,dim)
B = cumprod( ___,direction)
B = cumprod( ___,nanflag)
```

## Description

**B = cumprod(A)** returns the cumulative product of **A** starting at the beginning of the first array dimension in **A** whose size does not equal 1. The output **B** has the same size as **A**.

- If **A** is a vector, then **cumprod(A)** returns a vector containing the cumulative product of the elements of **A**.
- If **A** is a matrix, then **cumprod(A)** returns a matrix containing the cumulative products of each column of **A**.
- If **A** is a multidimensional array, then **cumprod(A)** acts along the first nonsingleton dimension on page 6-243.

**B = cumprod(A,dim)** returns the cumulative product along dimension **dim**. For example, if **A** is a matrix, then **cumprod(A,2)** returns the cumulative product of each row.

**B = cumprod( \_\_\_,direction)** specifies the direction using any of the previous syntaxes. For instance, **cumprod(A,2,'reverse')** returns the cumulative product within the rows of **A** by working from end to beginning of the second dimension.

**B = cumprod( \_\_\_,nanflag)** specifies whether to include or omit NaN values from the calculation for any of the previous syntaxes. **cumprod(A,'includenan')** includes all NaN values in the calculation while **cumprod(A,'omitnan')** ignores them.

## Examples

### Cumulative Product of Symbolic Vector

Create a symbolic vector. Find the cumulative product of its elements.

```
syms x
A = (1:5)*x
```

```
A = (x 2 x 3 x 4 x 5 x)
```

In the vector of cumulative products, element **B(2)** is the product of **A(1)** and **A(2)**, while **B(5)** is the product of elements **A(1)** through **A(5)**.

```
B = cumprod(A)
```

$$B = (x \ 2x^2 \ 6x^3 \ 24x^4 \ 120x^5)$$

### Cumulative Product of Each Column and Row in Symbolic Matrix

Create a 3-by-3 symbolic matrix A whose all elements are x.

```
syms x
A = ones(3)*x
```

$$A = \begin{pmatrix} x & x & x \\ x & x & x \\ x & x & x \end{pmatrix}$$

Compute the cumulative product of elements of A. By default, `cumprod` returns the cumulative product of each column.

```
B = cumprod(A)
```

$$B = \begin{pmatrix} x & x & x \\ x^2 & x^2 & x^2 \\ x^3 & x^3 & x^3 \end{pmatrix}$$

To compute the cumulative product of each row, set the value of the `dim` option to 2.

```
B = cumprod(A,2)
```

$$B = \begin{pmatrix} x & x^2 & x^3 \\ x & x^2 & x^3 \\ x & x^2 & x^3 \end{pmatrix}$$

### Reverse Cumulative Product of 3-D Symbolic Array

Create a 3-by-3-by-2 symbolic array.

```
syms x y
A(:,:,1) = [x y 0; x 3 x*y; x 1/3 y];
A(:,:,2) = [x y 3; 3 x y; y 3 x];
A
```

$$A(:,:,1) = \begin{pmatrix} x & y & 0 \\ x & 3 & xy \\ x & \frac{1}{3} & y \end{pmatrix}$$

$$A(:,:,2) =$$



$$\begin{pmatrix} x & y & 3 \\ 3 & x & y \\ y & 3 & x \end{pmatrix}$$

Compute the cumulative product along the rows by specifying `dim` as 2. Specify the `'reverse'` option to work from right to left in each row. The result is the same size as `A`.

```
B = cumprod(A,2,'reverse')
```

$$B(:, :, 1) = \begin{pmatrix} 0 & 0 & 0 \\ 3x^2y & 3xy & xy \\ \frac{xy}{3} & \frac{y}{3} & y \end{pmatrix}$$

$$B(:, :, 2) = \begin{pmatrix} 3xy & 3y & 3 \\ 3xy & xy & y \\ 3xy & 3x & x \end{pmatrix}$$

To compute the cumulative product along the third (page) dimension, specify `dim` as 3. Specify the `'reverse'` option to work from the largest page index to the smallest page index.

```
B = cumprod(A,3,'reverse')
```

$$B(:, :, 1) = \begin{pmatrix} x^2 & y^2 & 0 \\ 3x & 3x & xy^2 \\ xy & 1 & xy \end{pmatrix}$$

$$B(:, :, 2) = \begin{pmatrix} x & y & 3 \\ 3 & x & y \\ y & 3 & x \end{pmatrix}$$

### Symbolic Vector with NaN Values

Create a symbolic vector containing NaN values. Compute the cumulative products.

```
A = [sym('a') sym('b') 1 NaN 2]
```

```
A = (a b 1 NaN 2)
```

```
B = cumprod(A)
```

```
B = (a a b a b NaN NaN)
```

You can ignore NaN values in the cumulative product calculation using the `'omitnan'` option.

```
B = cumprod(A,'omitnan')
```

```
B = (a a b a b a b 2 a b)
```

## Input Arguments

### **A — Input array**

symbolic vector | symbolic matrix | symbolic multidimensional array

Input array, specified as a symbolic vector, matrix, or multidimensional array.

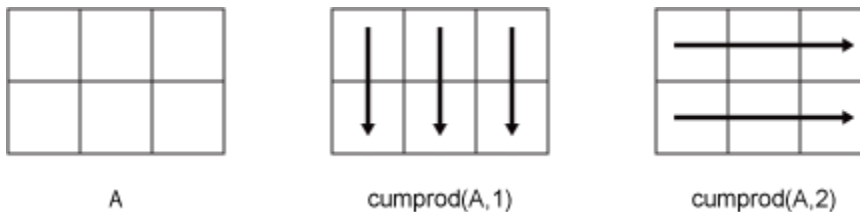
### **dim — Dimension to operate along**

positive integer scalar

Dimension to operate along, specified as a positive integer scalar. If no value is specified, then the default is the first array dimension whose size does not equal 1.

Consider a two-dimensional input array, *A*.

- `cumprod(A,1)` works on successive elements in the columns of *A* and returns the cumulative product of each column.
- `cumprod(A,2)` works on successive elements in the rows of *A* and returns the cumulative product of each row.



`cumprod` returns *A* if `dim` is greater than `ndims(A)`.

### **direction — Direction of cumulation**

'forward' (default) | 'reverse'

Direction of cumulation, specified as 'forward' (default) or 'reverse'.

- 'forward' works from 1 to end of the active dimension.
- 'reverse' works from end to 1 of the active dimension.

Data Types: char

### **nanflag — NaN condition**

'includenan' (default) | 'omitnan'

NaN condition, specified as:

- 'includenan' — Include NaN values from the input when computing the cumulative products, resulting in NaN values in the output.
- 'omitnan' — Ignore all NaN values in the input. The product of elements containing NaN values is the product of all non-NaN elements. If all elements are NaN, then `cumprod` returns 1.

Data Types: char

## Output Arguments

### **B — Cumulative product array**

vector | matrix | multidimensional array

Cumulative product array, returned as a vector, matrix, or multidimensional array of the same size as the input A.

## More About

### **First Nonsingleton Dimension**

The first nonsingleton dimension is the first dimension of an array whose size is not equal to 1.

For example:

- If X is a 1-by-n row vector, then the second dimension is the first nonsingleton dimension of X.
- If X is a 1-by-0-by-n empty array, then the second dimension is the first nonsingleton dimension of X.
- If X is a 1-by-1-by-3 array, then the third dimension is the first nonsingleton dimension of X.

## Version History

**Introduced in R2013b**

### **R2020b: Ignore NaNs using 'omitnan'**

cumprod can exclude NaNs when evaluating the cumulative product of an array that contains NaNs. You can use the option 'omitnan' to exclude NaNs and 'includenan' to include NaNs.

### **R2020b: Support for multidimensional arrays of symbolic expressions**

cumprod accepts multidimensional arrays of symbolic expressions as their input arguments.

## See Also

cumsum | fold | int | symprod | symsum

## cumsum

Symbolic cumulative sum

### Syntax

```
B = cumsum(A)
B = cumsum(A,dim)
B = cumsum( ___,direction)
B = cumsum( ___,nanflag)
```

### Description

`B = cumsum(A)` returns the cumulative sum of `A` starting at the beginning of the first array dimension in `A` whose size does not equal 1. The output `B` has the same size as `A`.

- If `A` is a vector, then `cumsum(A)` returns a vector containing the cumulative sum of the elements of `A`.
- If `A` is a matrix, then `cumsum(A)` returns a matrix containing the cumulative sums of each column of `A`.
- If `A` is a multidimensional array, then `cumsum(A)` acts along the first nonsingleton dimension on page 6-248.

`B = cumsum(A,dim)` returns the cumulative sum along dimension `dim`. For example, if `A` is a matrix, then `cumsum(A,2)` returns the cumulative sum of each row.

`B = cumsum( ___,direction)` specifies the direction using any of the previous syntaxes. For instance, `cumsum(A,2,'reverse')` returns the cumulative sum within the rows of `A` by working from end to beginning of the second dimension.

`B = cumsum( ___,nanflag)` specifies whether to include or omit NaN values from the calculation for any of the previous syntaxes. `cumsum(A,'includenan')` includes all NaN values in the calculation while `cumsum(A,'omitnan')` ignores them.

### Examples

#### Cumulative Sum of Symbolic Vector

Create a symbolic vector. Find the cumulative sum of its elements.

```
syms x
A = (1:5)*x
A = (x 2 x 3 x 4 x 5 x)
```

In the vector of cumulative sums, element `B(2)` is the sum of `A(1)` and `A(2)`, while `B(5)` is the sum of elements `A(1)` through `A(5)`.

```
B = cumsum(A)
B = (x 3 x 6 x 10 x 15 x)
```

### Cumulative Sum of Each Column and Row in Symbolic Matrix

Create a 3-by-3 symbolic matrix A whose elements are all equal to 1.

```
A = sym(ones(3))
```

$$A = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Compute the cumulative sum of elements of A. By default, cumsum returns the cumulative sum of each column.

```
B = cumsum(A)
```

$$B = \begin{pmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{pmatrix}$$

To compute the cumulative sum of each row, set the value of the dim option to 2.

```
B = cumsum(A,2)
```

$$B = \begin{pmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{pmatrix}$$

### Reverse Cumulative Sum of 3-D Symbolic Array

Create a 3-by-3-by-2 symbolic array.

```
syms x y
A(:,:,1) = [x y 3; 3 x y; y 2 x];
A(:,:,2) = [x y 1/3; 1 y x; 1/3 x 2];
A
```

$$A(:,:,1) = \begin{pmatrix} x & y & 3 \\ 3 & x & y \\ y & 2 & x \end{pmatrix}$$

$$A(:,:,2) = \begin{pmatrix} x & y & \frac{1}{3} \\ 1 & y & x \\ \frac{1}{3} & x & 2 \end{pmatrix}$$

Compute the cumulative sum along the rows by specifying `dim` as 2. Specify the `'reverse'` option to work from right to left in each row. The result is the same size as `A`.

```
B = cumsum(A,2,'reverse')
```

$$B(:,:,1) = \begin{pmatrix} x+y+3 & y+3 & 3 \\ x+y+3 & x+y & y \\ x+y+2 & x+2 & x \end{pmatrix}$$

$$B(:,:,2) = \begin{pmatrix} x+y+\frac{1}{3} & y+\frac{1}{3} & \frac{1}{3} \\ x+y+1 & x+y & x \\ x+\frac{7}{3} & x+2 & 2 \end{pmatrix}$$

To compute the cumulative sum along the third (page) dimension, specify `dim` as 3. Specify the `'reverse'` option to work from largest page index to smallest page index.

```
B = cumsum(A,3,'reverse')
```

$$B(:,:,1) = \begin{pmatrix} 2x & 2y & \frac{10}{3} \\ 4 & x+y & x+y \\ y+\frac{1}{3} & x+2 & x+2 \end{pmatrix}$$

$$B(:,:,2) = \begin{pmatrix} x & y & \frac{1}{3} \\ 1 & y & x \\ \frac{1}{3} & x & 2 \end{pmatrix}$$

### Symbolic Vector with NaN Values

Create a symbolic vector containing NaN values. Compute the cumulative sums.

```
A = [sym('a') sym('b') 1 NaN 2]
```

```
A = (a b 1 NaN 2)
```

```
B = cumsum(A)
```

```
B = (a a+b a+b+1 NaN NaN)
```

You can ignore NaN values in the cumulative sum calculation using the `'omitnan'` option.

```
B = cumsum(A,'omitnan')
```

```
B = (a a+b a+b+1 a+b+1 a+b+3)
```

## Input Arguments

### A — Input array

symbolic vector | symbolic matrix | symbolic multidimensional array

Input array, specified as a symbolic vector, matrix, or multidimensional array.

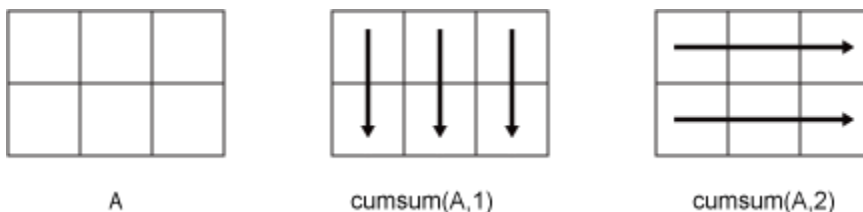
### dim — Dimension to operate along

positive integer scalar

Dimension to operate along, specified as a positive integer scalar. If no value is specified, then the default is the first array dimension whose size does not equal 1.

Consider a two-dimensional input array, A:

- `cumsum(A,1)` works on successive elements in the columns of A and returns the cumulative sum of each column.
- `cumsum(A,2)` works on successive elements in the rows of A and returns the cumulative sum of each row.



`cumsum` returns A if `dim` is greater than `ndims(A)`.

### direction — Direction of cumulation

'forward' (default) | 'reverse'

Direction of cumulation, specified as 'forward' (default) or 'reverse'.

- 'forward' works from 1 to end of the active dimension.
- 'reverse' works from end to 1 of the active dimension.

Data Types: char

### nanflag — NaN condition

'includenan' (default) | 'omitnan'

NaN condition, specified as:

- 'includenan' — Include NaN values from the input when computing the cumulative sums, resulting in NaN values in the output.
- 'omitnan' — Ignore all NaN values in the input. The sum of elements containing NaN values is the sum of all non-NaN elements. If all elements are NaN, then `cumsum` returns 0.

Data Types: char

## Output Arguments

### **B — Cumulative sum array**

vector | matrix | multidimensional array

Cumulative sum array, returned as a vector, matrix, or multidimensional array of the same size as the input A.

## More About

### **First Nonsingleton Dimension**

The first nonsingleton dimension is the first dimension of an array whose size is not equal to 1.

For example:

- If X is a 1-by-n row vector, then the second dimension is the first nonsingleton dimension of X.
- If X is a 1-by-0-by-n empty array, then the second dimension is the first nonsingleton dimension of X.
- If X is a 1-by-1-by-3 array, then the third dimension is the first nonsingleton dimension of X.

## Version History

**Introduced in R2013b**

### **R2020b: Ignore NaNs using 'omitnan'**

`cumsum` can exclude NaNs when evaluating the cumulative sum of an array that contains NaNs. You can use the option `'omitnan'` to exclude NaNs and `'includenan'` to include NaNs.

### **R2020b: Support for multidimensional arrays of symbolic expressions**

`cumsum` accepts multidimensional arrays of symbolic expressions as their input arguments.

## See Also

`cumprod` | `fold` | `int` | `symprod` | `symsum`



# curl

Curl of symbolic vector field

## Syntax

```
c = curl(V,X)
c = curl(V)
```

## Description

`c = curl(V,X)` returns the curl on page 6-253 of symbolic vector field  $V$  with respect to vector  $X$  in three-dimensional Cartesian coordinates. Both the vector field  $V$  and the vector  $X$  must be vectors with three components.

`c = curl(V)` returns the curl of the vector field  $V$  with respect to a default vector constructed from the symbolic variables in  $V$ .

## Examples

### Find Curl of Vector Field

Find the curl of the vector field  $V(x, y, z) = (x^3y^2z, y^3z^2x, z^3x^2y)$  with respect to vector  $X = (x, y, z)$  in Cartesian coordinates.

```
syms x y z
V = [x^3*y^2*z, y^3*z^2*x, z^3*x^2*y];
X = [x y z];
c = curl(V,X)
```

```
c =

$$\begin{pmatrix} x^2z^3 - 2xy^3z \\ x^3y^2 - 2xyz^3 \\ y^3z^2 - 2x^3yz \end{pmatrix}$$

```

### Plot Curl of 2-D Vector Field

Find the curl of a 2-D vector field  $F(x, y) = (\cos(x + y), \sin(x - y), 0)$ . Plot the vector field as a quiver (velocity) plot and the  $z$ -component of its curl as a contour plot.

Create the 2-D vector field  $F(x, y)$  and find its curl. The curl is a vector with only the  $z$ -component.

```
syms x y z
F = [cos(x+y) sin(x-y) 0];
c = curl(F,[x,y,z])
```

```
c =
```

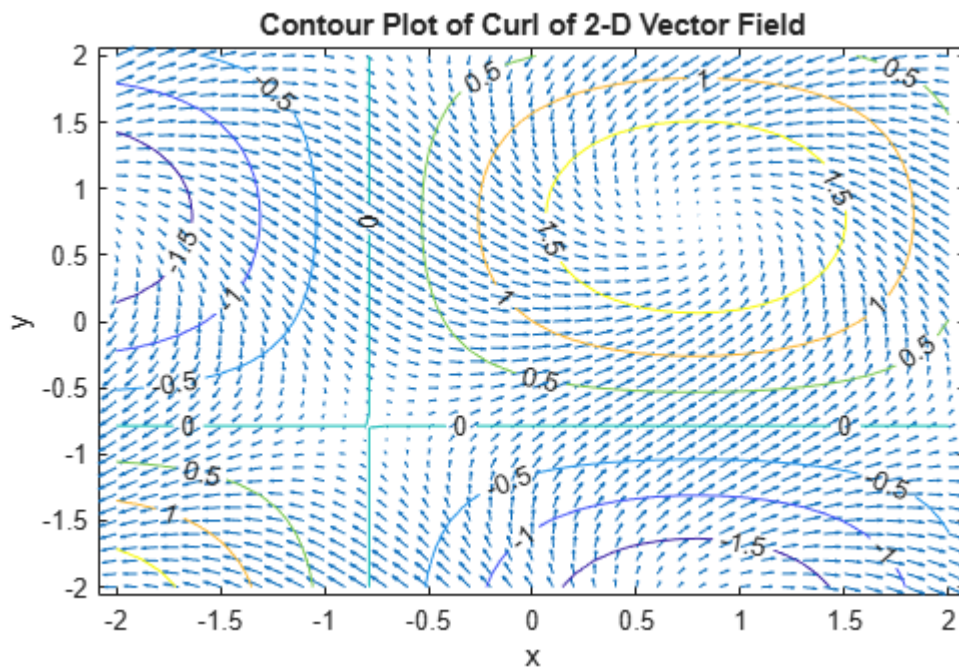
$$\begin{pmatrix} 0 \\ 0 \\ \cos(x-y) + \sin(x+y) \end{pmatrix}$$

Plot the 2-D vector field  $F(x, y)$  for the region  $-2 < x < 2$  and  $-2 < y < 2$ . MATLAB® provides the `quiver` plotting function for this task. The function does not accept symbolic arguments. First, replace symbolic variables in expressions for components of  $F$  with numeric values. Then use `quiver`.

```
v = -2:0.1:2;
[xPlot,yPlot] = meshgrid(v);
Fx = subs(F(1),{x,y},{xPlot,yPlot});
Fy = subs(F(2),{x,y},{xPlot,yPlot});
quiver(xPlot,yPlot,Fx,Fy)
hold on
```

Next, plot the contour of the z-component of the curl using `contour`.

```
cPlot = subs(c(3),{x,y},{xPlot,yPlot});
contour(xPlot,yPlot,cPlot,"ShowText","on")
title("Contour Plot of Curl of 2-D Vector Field")
xlabel("x")
ylabel("y")
```



### Derive Electromagnetic Wave Equation

Derive the electromagnetic wave equation in free space without charge and without current sources from Maxwell's equations.

First, create symbolic scalar variables to represent the vacuum permeability and permittivity. Create a symbolic matrix variable to represent the Cartesian coordinates. Create two symbolic matrix functions to represent the electric and magnetic fields as functions of space and time.

```
syms mu_0 epsilon_0
syms X [3 1] matrix
syms E(X,t) B(X,t) [3 1] matrix keepargs
```

Next, create four equations to represent Maxwell's equations.

```
Maxwell1 = divergence(E,X) == 0
```

```
Maxwell1(X, t) =  $\nabla_X \cdot E(X,t) = 0_{1,1}$ 
```

```
Maxwell2 = curl(E,X) == -diff(B,t)
```

```
Maxwell2(X, t) =  $\nabla_X \times E(X,t) = -\frac{\partial}{\partial t} B(X,t)$ 
```

```
Maxwell3 = divergence(B,X) == 0
```

```
Maxwell3(X, t) =  $\nabla_X \cdot B(X,t) = 0_{1,1}$ 
```

```
Maxwell4 = curl(B,X) == mu_0*epsilon_0*diff(E,t)
```

```
Maxwell4(X, t) =  $\nabla_X \times B(X,t) = (\epsilon_0 \mu_0) \frac{\partial}{\partial t} E(X,t)$ 
```

Then, find the wave equation for the electric field. Compute the curl of the second Maxwell equation.

```
wave_E = curl(Maxwell2,X)
```

```
wave_E(X, t) =  $\nabla_X (\nabla_X \cdot E(X,t)) - \Delta_X E(X,t) = -\nabla_X \times \left( \frac{\partial}{\partial t} B(X,t) \right)$ 
```

Substitute the first Maxwell equation in the electric field wave equation. Use lhs and rhs to obtain the left and right sides of the first Maxwell equation.

```
wave_E = subs(wave_E, lhs(Maxwell1), rhs(Maxwell1))
```

```
wave_E(X, t) =  $-\Delta_X E(X,t) = -\nabla_X \times \left( \frac{\partial}{\partial t} B(X,t) \right)$ 
```

Compute the time derivative of the fourth Maxwell equation.

```
dMaxwell4 = diff(Maxwell4,t)
```

```
dMaxwell4(X, t) =  $\nabla_X \times \left( \frac{\partial}{\partial t} B(X,t) \right) = (\epsilon_0 \mu_0) \frac{\partial}{\partial t} \frac{\partial}{\partial t} E(X,t)$ 
```

Substitute the term that involves the magnetic field  $\nabla_X \times \frac{\partial}{\partial t} \mathbf{B}(\mathbf{X}, t)$  in wave\_E with the right side of dMaxwell4. Use lhs and rhs to obtain these terms from dMaxwell4.

```
wave_E = subs(wave_E, lhs(dMaxwell4), rhs(dMaxwell4))
```

$$\begin{aligned} \text{wave\_E}(X, t) = \\ -\Delta_X E(X, t) = -(\epsilon_0 \mu_0) \frac{\partial}{\partial t} \frac{\partial}{\partial t} E(X, t) \end{aligned}$$

Using similar steps, you can also find the wave equation for the magnetic field.

$$\text{wave\_B} = \text{curl}(\text{Maxwell4}, X)$$

$$\begin{aligned} \text{wave\_B}(X, t) = \\ \nabla_X (\nabla_X \cdot B(X, t)) - \Delta_X B(X, t) = (\epsilon_0 \mu_0) \left( \nabla_X \times \left( \frac{\partial}{\partial t} E(X, t) \right) \right) \end{aligned}$$

$$\text{wave\_B} = \text{subs}(\text{wave\_B}, \text{lhs}(\text{Maxwell3}), \text{rhs}(\text{Maxwell3}))$$

$$\begin{aligned} \text{wave\_B}(X, t) = \\ -\Delta_X B(X, t) = (\epsilon_0 \mu_0) \left( \nabla_X \times \left( \frac{\partial}{\partial t} E(X, t) \right) \right) \end{aligned}$$

$$\text{dMaxwell2} = \text{diff}(\text{Maxwell2}, t)$$

$$\begin{aligned} \text{dMaxwell2}(X, t) = \\ \nabla_X \times \left( \frac{\partial}{\partial t} E(X, t) \right) = -\frac{\partial}{\partial t} \frac{\partial}{\partial t} B(X, t) \end{aligned}$$

$$\text{wave\_B} = \text{subs}(\text{wave\_B}, \text{lhs}(\text{dMaxwell2}), \text{rhs}(\text{dMaxwell2}))$$

$$\begin{aligned} \text{wave\_B}(X, t) = \\ -\Delta_X B(X, t) = -(\epsilon_0 \mu_0) \frac{\partial}{\partial t} \frac{\partial}{\partial t} B(X, t) \end{aligned}$$

## Input Arguments

### V — Three-dimensional symbolic vector field

vector of symbolic scalar variables | symbolic function | symbolic matrix variable | symbolic matrix function

Three-dimensional symbolic vector field, specified as a vector of symbolic scalar variables, symbolic function, symbolic matrix variable, or symbolic matrix function. V must have a size of 1-by-3 or 3-by-1.

- If V is a function of symbolic scalar variables, where V is of type `sym` or `symfun`, then the vector X must be of type `sym` or `symfun`.
- If V is a function of symbolic matrix variables, where V is of type `symmatrix` or `symfunmatrix`, then the vector X must be of type `symmatrix` or `symfunmatrix`.

Data Types: `sym` | `symfun` | `symmatrix` | `symfunmatrix`

### X — Three-dimensional vector with respect to which you find the curl

vector of symbolic scalar variables | symbolic function | symbolic matrix variable | symbolic matrix function

Three-dimensional vector with respect to which you find the curl, specified as a vector of symbolic scalar variables, symbolic function, symbolic matrix variable, or symbolic matrix function. X must have a size of 1-by-3 or 3-by-1.

If you do not specify  $X$  and  $V$  is a function of symbolic scalar variables, then, by default, `curl` constructs vector  $X$  from the symbolic scalar variables in  $V$  with the order of variables as defined by `symvar(V)`.

Data Types: `sym` | `symfun` | `symmatrix` | `symfunmatrix`

## Limitations

- The `curl` function does not support tensor derivatives. If the input  $V$  is a tensor field or a matrix rather than a vector, then the `curl` function returns an error.
- Symbolic Math Toolbox currently does not support the `dot` or `cross` functions for symbolic matrix variables and functions of type `symmatrix` and `symfunmatrix`. If vector calculus identities involve dot or cross products, then the toolbox displays those identities in terms of other supported functions instead. To see a list of all the functions that support symbolic matrix variables and functions, use the commands `methods symmatrix` and `methods symfunmatrix`.

## More About

### Curl of Symbolic Vector Field

The curl of the symbolic vector field  $V = (V_1, V_2, V_3)$  with respect to the vector  $X = (X_1, X_2, X_3)$  in Cartesian coordinates is this vector.

$$\text{curl}(V) = \nabla_X \times V = \begin{pmatrix} \frac{\partial V_3}{\partial X_2} - \frac{\partial V_2}{\partial X_3} \\ \frac{\partial V_1}{\partial X_3} - \frac{\partial V_3}{\partial X_1} \\ \frac{\partial V_2}{\partial X_1} - \frac{\partial V_1}{\partial X_2} \end{pmatrix}$$

## Version History

Introduced in R2012a

### R2023a: Compute curl of symbolic matrix variables and functions

The `curl` function accepts symbolic matrix variables and functions of type `symmatrix` and `symfunmatrix` as input arguments. For example, see “Derive Electromagnetic Wave Equation” on page 6-250.

## See Also

`diff` | `divergence` | `gradient` | `jacobian` | `hessian` | `laplacian` | `potential` | `vectorPotential`

## daeFunction

Convert system of differential algebraic equations to MATLAB function handle suitable for `ode15i`

### Syntax

```
f = daeFunction(eqs,vars)
f = daeFunction(eqs,vars,p1,...,pN)
f = daeFunction(____,Name=Value)
```

### Description

`f = daeFunction(eqs,vars)` converts a system of symbolic first-order differential algebraic equations (DAEs) to a MATLAB function handle acceptable as an input argument to the numerical MATLAB DAE solver `ode15i`.

`f = daeFunction(eqs,vars,p1,...,pN)` lets you specify the symbolic parameters of the system as `p1,...,pN`.

`f = daeFunction(____,Name=Value)` uses additional options specified by one or more `Name=Value` arguments.

### Examples

#### Convert DAE System to Function Handle

Create the system of differential algebraic equations. Here, the symbolic functions `x1(t)` and `x2(t)` represent the state variables of the system. The system also contains constant symbolic parameters `a`, `b`, and the parameter function `r(t)`. These parameters do not represent state variables. Specify the equations and state variables as two symbolic vectors: equations as a vector of symbolic equations, and variables as a vector of symbolic function calls.

```
syms x1(t) x2(t) a b r(t)
eqs = [diff(x1(t),t) == a*x1(t) + b*x2(t)^2,...
       x1(t)^2 + x2(t)^2 == r(t)^2];
vars = [x1(t),x2(t)];
```

Use `daeFunction` to generate a MATLAB® function handle `f` depending on the variables `x1(t)`, `x2(t)` and on the parameters `a`, `b`, `r(t)`.

```
f = daeFunction(eqs,vars,a,b,r(t))
```

```
f = function_handle with value:
    @(t,in2,in3,param1,param2,param3)[in3(1,:)-param1.*in2(1,:)-param2.*in2(2,:).^2;-param3.^2+in3(2,:)-param1.*in2(1,:)-param2.*in2(2,:).^2;-param3.^2+in3(2,:)-param1.*in2(1,:)-param2.*in2(2,:).^2]
```

Specify the parameter values, and create the reduced function handle `F` as follows.

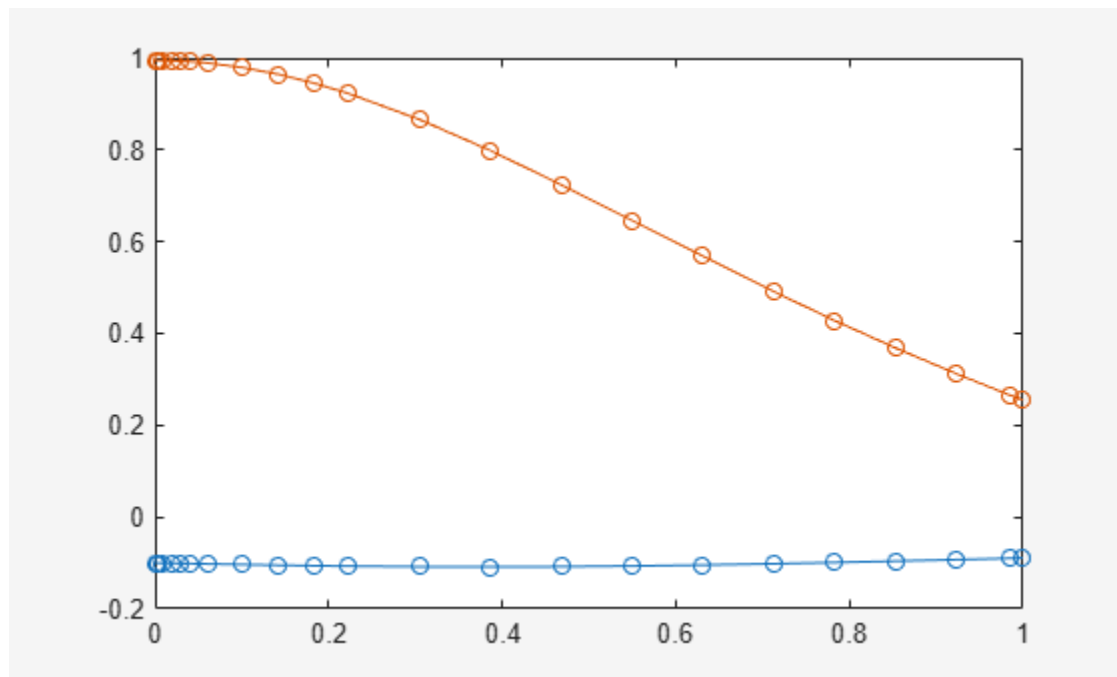
```
aVal = -0.6;
bVal = -0.1;
rFunc = @(t) cos(t)/(1 + t^2);
F = @(t,Y,YP) f(t,Y,YP,aVal,bVal,rFunc(t));
```

Specify consistent initial conditions for the DAE system.

```
t0 = 0;
y0 = [-rFunc(t0)*sin(0.1); rFunc(t0)*cos(0.1)];
yp0 = [aVal*y0(1) + bVal*y0(2)^2; 1.234];
```

Now, use `ode15i` to solve the system of equations.

```
ode15i(F,[t0,1],y0,yp0)
```



### Write Function to File with Comments

Write the generated function handle to a file by specifying the `File` option. When writing to a file, `daeFunction` optimizes the code using intermediate variables named `t0`, `t1`, .... Include comments in the file using the `Comments` option.

Write the generated function handle to the file `myfile`.

```
syms x1(t) x2(t) a b r(t)
eqs = [diff(x1(t),t) == a*x1(t) + b*x2(t)^2,...
       x1(t)^2 + x2(t)^2 == r(t)^2];
vars = [x1(t),x2(t)];
daeFunction(eqs,vars,a,b,r(t),File="myfile")

function eqs = myfile(t,in2,in3,param1,param2,param3)
%MYFILE
%   EQS = MYFILE(T,IN2,IN3,PARAM1,PARAM2,PARAM3)

%   This function was generated by the Symbolic Math Toolbox version 7.3.
%   01-Jan-2017 00:00:00

YP1 = in3(1,:);
x1 = in2(1,:);
```

```
x2 = in2(2,:);  
t2 = x2.^2;  
eqs = [YP1-param2.*t2-param1.*x1;t2-param3.^2+x1.^2];
```

Include the comment `Version: 1.1`.

```
daeFunction(eqs,vars,a,b,r(t),File="myfile", ...  
            Comments="Version: 1.1");  
  
function eqs = myfile(t,in2,in3,param4,param5,param6)  
...  
%Version: 1.1  
YP3 = in3(1,:);  
...
```

## Input Arguments

### **eqs — System of first-order DAEs**

vector of symbolic equations | vector of symbolic expressions

System of first-order DAEs, specified as a vector of symbolic equations or expressions. Here, expressions represent equations with zero right side.

### **vars — State variables**

vector of symbolic functions | vector of symbolic function calls

State variables, specified as a vector of symbolic functions or function calls, such as  $x(t)$ .

Example:  $[x(t), y(t)]$  or  $[x(t); y(t)]$

### **p1, ..., pN — Parameters of system**

symbolic variables | symbolic functions | symbolic function calls | symbolic vector | symbolic matrix

Parameters of the system, specified as symbolic variables, functions, or function calls, such as  $f(t)$ . You can also specify parameters of the system as a vector or matrix of symbolic variables, functions, or function calls. If `eqs` contains symbolic parameters other than the variables specified in `vars`, you must specify these additional parameters as  $p1, \dots, pN$ .

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `daeFunction(eqs,vars,File="myfile")`

### **Comments — Comments to include in file header**

character vector | cell array of character vectors | string vector

Comments to include in the file header, specified as a character vector, cell array of character vectors, or string vector.

### **File — Path to file containing generated code**

string | character vector



Path to the file containing generated code, specified as a string or character vector. The generated file accepts arguments of type `double`, and can be used without Symbolic Math Toolbox. If the value is an empty character vector, `odeFunction` generates an anonymous function. If the character vector does not end in `.m`, the function appends `.m`.

By default, `daeFunction` with the `File` argument generates a file containing optimized code. Optimized means intermediate variables are automatically generated to simplify or speed up the code. MATLAB generates intermediate variables as a lowercase letter `t` followed by an automatically generated number, for example `t32`. To disable code optimization, use the `Optimize` argument.

### **Optimize — Flag preventing optimization of code written to function file**

`true` (default) | `false`

Flag preventing optimization of code written to a function file, specified as `false` or `true`.

By default, `daeFunction` with the `File` argument generates a file containing optimized code. Optimized means intermediate variables are automatically generated to simplify or speed up the code. MATLAB generates intermediate variables as a lowercase letter `t` followed by an automatically generated number, for example `t32`.

`daeFunction` without the `File` argument (or with a file path specified by an empty character vector) creates a function handle. In this case, the code is not optimized. If you try to enforce code optimization by setting `Optimize` to `true`, then `daeFunction` throws an error.

### **Sparse — Flag that switches between sparse and dense matrix generation**

`false` (default) | `true`

Flag that switches between sparse and dense matrix generation, specified as `true` or `false`. When you specify `Sparse=true`, the generated function represents symbolic matrices by sparse numeric matrices. Use `Sparse=true` when you convert symbolic matrices containing many zero elements. Often, operations on sparse matrices are more efficient than the same operations on dense matrices.

## **Output Arguments**

### **f — Function handle that can serve as input argument to `ode15i`**

MATLAB function handle

Function handle that can serve as input argument to `ode15i`, returned as a MATLAB function handle.

## **Version History**

**Introduced in R2014b**

### **See Also**

`findDecoupledBlocks` | `decic` | `incidenceMatrix` | `isLowIndexDAE` | `massMatrixForm` | `matlabFunction` | `ode15i` | `odeFunction` | `reduceDAEIndex` | `reduceDAETODE` | `reduceDifferentialOrder` | `reduceRedundancies`

### **Topics**

“Solve Differential Algebraic Equations (DAEs)” on page 3-66

## dawson

Dawson integral

### Syntax

`dawson(X)`

### Description

`dawson(X)` represents the Dawson integral on page 6-260.

### Examples

#### Dawson Integral for Numeric and Symbolic Arguments

Depending on its arguments, `dawson` returns floating-point or exact symbolic results.

Compute the Dawson integrals for these numbers. Because these numbers are not symbolic objects, `dawson` returns floating-point results.

```
A = dawson([-Inf, -3/2, -1, 0, 2, Inf])
```

```
A =  
      0      -0.4282      -0.5381      0      0.3013      0
```

Compute the Dawson integrals for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `dawson` returns unresolved symbolic calls.

```
symA = dawson(sym([-Inf, -3/2, -1, 0, 2, Inf]))
```

```
symA =  
[ 0, -dawson(3/2), -dawson(1), 0, dawson(2), 0]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

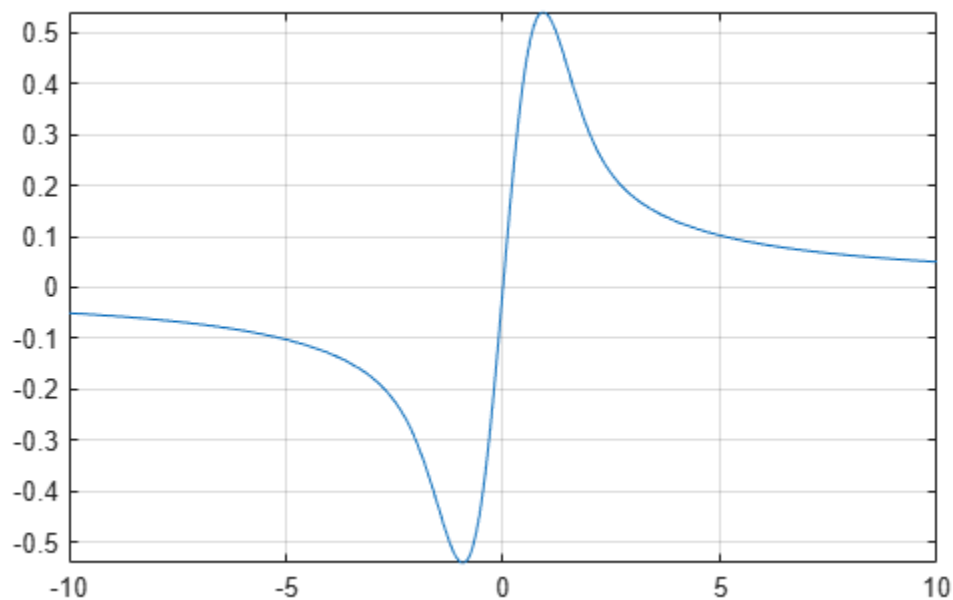
```
vpa(symA)
```

```
ans =  
[ 0,...  
-0.42824907108539862547719010515175,...  
-0.53807950691276841913638742040756,...  
0,...  
0.30134038892379196603466443928642,...  
0]
```

#### Plot the Dawson Integral

Plot the Dawson integral on the interval from -10 to 10.

```
syms x  
fplot(dawson(x),[-10 10])  
grid on
```



## Handle Expressions Containing Dawson Integral

Many functions, such as `diff` and `limit`, can handle expressions containing `dawson`.

Find the first and second derivatives of the Dawson integral:

```
syms x
diff(dawson(x), x)
diff(dawson(x), x, x)

ans =
1 - 2*x*dawson(x)

ans =
2*x*(2*x*dawson(x) - 1) - 2*dawson(x)
```

Find the limit of this expression involving `dawson`:

```
limit(x*dawson(x), Inf)

ans =
1/2
```

## Input Arguments

### X — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## More About

### Dawson Integral

The Dawson integral, also called the Dawson function, is defined as follows:

$$\text{dawson}(x) = D(x) = e^{-x^2} \int_0^x e^{t^2} dt$$

Symbolic Math Toolbox uses this definition to implement `dawson`.

The alternative definition of the Dawson integral is

$$D(x) = e^{x^2} \int_0^x e^{-t^2} dt$$

### Tips

- `dawson(0)` returns 0.
- `dawson(Inf)` returns 0.
- `dawson(-Inf)` returns 0.

## Version History

Introduced in R2014a

### See Also

`erf` | `erfc`



```
str = 4×7 char array
'1000000'
'0110110'
'1111011'
'0001011'
```

Return a binary representation with at least 8 digits by specifying the number of digits.

```
str = dec2bin(d,8)

str = 4×8 char array
'01000000'
'00110110'
'01111011'
'00001011'
```

## Input Arguments

### **d — Integers in decimal representation**

symbolic number | symbolic vector | symbolic matrix | symbolic array

Integers in decimal representation, specified as a symbolic number, vector, matrix, or array.

d can include negative integers. The function converts negative integers using their two's complement binary values.

Example: `sym([2 4])`

### **n — Number of bits**

scalar positive integer

Number of bits, specified as a scalar positive integer.

Example: 8

## Version History

Introduced in R2019a

## See Also

`dec2hex`

## External Websites

Two's complement

# dec2hex

Convert symbolic integer in decimal to hexadecimal representation

## Syntax

```
str = dec2hex(d)
str = dec2hex(d,n)
```

## Description

`str = dec2hex(d)` returns the hexadecimal representation of symbolic integer `d` as a character vector.

If `d` is a matrix or multidimensional array of symbolic integers with `N` elements, `dec2hex` returns a character array with `N` rows. Each row of the output `str` corresponds to an element of `d` accessed with linear indexing.

`str = dec2hex(d,n)` returns a hexadecimal representation with at least `n` digits.

## Examples

### Convert Integer to Hexadecimal Representation

Define a large integer  $2^{60} - 1$  as a symbolic number.

```
d = sym(2)^60 - 1
d = 1152921504606846975
```

Convert the decimal number to hexadecimal representation.

```
str = dec2hex(d)

str =
'FFFFFFFFFFFFFFF'
```

### Convert Array of Integers to Hexadecimal Representation

Create a 2-by-2 symbolic matrix that contains integers in decimal representation.

```
d = [sym(2)^6 123; 54 11]
d =
    (64 123)
    (54 11)
```

Convert the integers to hexadecimal representation using `dec2hex`. `dec2hex` returns 4 rows of character vectors. Each row contains a 2-digit hexadecimal number.

```
str = dec2hex(d)

str = 4×2 char array
    '40'
    '36'
    '7B'
    '0B'
```

Return a hexadecimal representation with at least 4 digits by specifying the number of digits.

```
str = dec2hex(d,4)

str = 4×4 char array
    '0040'
    '0036'
    '007B'
    '000B'
```

## Input Arguments

### **d — Integers in decimal representation**

symbolic number | symbolic vector | symbolic matrix | symbolic array

Integers in decimal representation, specified as a symbolic number, vector, matrix, or array.

d can include negative integers. The function converts negative integers using their two's complement binary values.

Example: `sym([2 4])`

### **n — Number of hexadecimal digits**

scalar positive integer

Number of hexadecimal digits, specified as a scalar positive integer.

Example: 8

## Version History

**Introduced in R2019a**

## See Also

`dec2bin`

## External Websites

Two's complement



# decic

Find consistent initial conditions for first-order implicit ODE system with algebraic constraints

## Syntax

```
[y0,yp0] = decic(eqs,vars,constraintEqs,t0,y0_est,fixedVars,yp0_est,options)
```

## Description

`[y0,yp0] = decic(eqs,vars,constraintEqs,t0,y0_est,fixedVars,yp0_est,options)` finds consistent initial conditions for the system of first-order implicit ordinary differential equations with algebraic constraints returned by the `reduceDAEToODE` function.

The call `[eqs,constraintEqs] = reduceDAEToODE(DA_eqs,vars)` reduces the system of differential algebraic equations `DA_eqs` to the system of implicit ODEs `eqs`. It also returns constraint equations encountered during system reduction. For the variables of this ODE system and their derivatives, `decic` finds consistent initial conditions `y0`, `yp0` at the time `t0`.

Substituting the numerical values `y0`, `yp0` into the differential equations `subs(eqs, [t; vars(t); diff(vars(t))], [t0; y0; yp0])` and the constraint equations `subs(constr, [t; vars(t); diff(vars(t))], [t0; y0; yp0])` produces zero vectors. Here, `vars` must be a column vector.

`y0_est` specifies numerical estimates for the values of the variables `vars` at the time `t0`, and `fixedVars` indicates the values in `y0_est` that must not change during the numerical search. The optional argument `yp0_est` lets you specify numerical estimates for the values of the derivatives of the variables `vars` at the time `t0`.

## Examples

### Find Consistent Initial Conditions for ODE System

Reduce the DAE system to a system of implicit ODEs. Then, find consistent initial conditions for the variables of the resulting ODE system and their first derivatives.

Create the following differential algebraic system.

```
syms x(t) y(t)
DA_eqs = [diff(x(t),t) == cos(t) + y(t);
          x(t)^2 + y(t)^2 == 1];
vars = [x(t); y(t)];
```

Use `reduceDAEToODE` to convert this system to a system of implicit ODEs.

```
[eqs,constraintEqs] = reduceDAEToODE(DA_eqs,vars)
```

```
eqs =

$$\begin{pmatrix} \frac{\partial}{\partial t} x(t) - \cos(t) - y(t) \\ -2x(t) \frac{\partial}{\partial t} x(t) - 2y(t) \frac{\partial}{\partial t} y(t) \end{pmatrix}$$

```

```
constraintEqs = -x(t)^2 - y(t)^2 + 1
```

Create an option set that specifies numerical tolerances for the numerical search.

```
options = odeset(RelTol=1e-7,AbsTol=1e-7);
```

Fix values  $t_0 = 0$  for the time and numerical estimates for consistent values of the variables and their derivatives.

```
t0 = 0;
y0_est = [0.1 0.9];
yp0_est = [0.0 0.0];
```

You can treat the constraint as an algebraic equation for the variable  $x$  with the fixed parameter  $y$ . For this, set `fixedVars = [0 1]`. Alternatively, you can treat it as an algebraic equation for the variable  $y$  with the fixed parameter  $x$ . For this, set `fixedVars = [1 0]`.

First, set the initial value  $x(t_0) = y0\_est(1) = 0.1$ .

```
fixedVars = [1 0];
[y0,yp0] = decic(eqs,vars,constraintEqs,t0,y0_est,fixedVars,yp0_est,options)
```

```
y0 = 2×1
```

```
0.1000
0.9950
```

```
yp0 = 2×1
```

```
1.9950
-0.2005
```

Now, change `fixedVars` to `[0 1]`. This fixes  $y(t_0) = y0\_est(2) = 0.9$ .

```
fixedVars = [0 1];
[y0,yp0] = decic(eqs,vars,constraintEqs,t0,y0_est,fixedVars,yp0_est,options)
```

```
y0 = 2×1
```

```
-0.4359
0.9000
```

```
yp0 = 2×1
```

```
1.9000
0.9202
```

Verify that these initial values are consistent initial values satisfying the equations and the constraints.

```
subs(eqs,[diff(vars,t);vars;t],[yp0;y0;t0])
```

```
ans =
    (0)
    (0)
```

```
subs(constraintEqs,[diff(vars,t);vars;t0],[yp0;y0;t0])
ans = 0
```

## Input Arguments

### **eqs — System of implicit ordinary differential equations**

vector of symbolic equations | vector of symbolic expressions

System of implicit ordinary differential equations, specified as a vector of symbolic equations or expressions. Here, expressions represent equations with zero right side.

Typically, you use expressions returned by `reduceDAEToODE`.

### **vars — State variables of original DAE system**

vector of symbolic functions | vector of symbolic function calls

State variables of original DAE system, specified as a vector of symbolic functions or function calls, such as `x(t)`.

Example: `[x(t),y(t)]` or `[x(t);y(t)]`

### **constraintEqs — Constraint equations found by reduceDAEToODE during system reduction**

vector of symbolic equations | vector of symbolic expressions

Constraint equations encountered during system reduction, specified as a vector of symbolic equations or expressions. These expressions or equations depend on the variables `vars`, but not on their derivatives.

Typically, you use constraint equations returned by `reduceDAEToODE`.

### **t0 — Initial time**

number

Initial time, specified as a number.

### **y0\_est — Estimates for values of variables vars at initial time t0**

numeric vector

Estimates for the values of the variables `vars` at the initial time `t0`, specified as a numeric vector.

### **fixedVars — Input vector indicating which elements of y0\_est are fixed values**

vector with elements 0 or 1

Input vector indicating which elements of `y0_est` are fixed values, specified as a vector with 0s or 1s. Fixed values of `y0_est` correspond to values 1 in `fixedVars`. These values are not modified during the numerical search. The zero entries in `fixedVars` correspond to those variables in `y0_est` for which `decic` solves the constraint equations. The number of 0s must coincide with the number of constraint equations. The Jacobian matrix of the constraints with respect to the variables `vars(fixedVars == 0)` must be invertible.

### **yp0\_est — Estimates for values of first derivatives of variables vars at initial time t0**

numeric vector

Estimates for the values of the first derivatives of the variables `vars` at the initial time `t0`, specified as a numeric vector.

**options — Options for numerical search**

options structure, returned by `odeset`

Options for numerical search, specified as an options structure, returned by `odeset`. For example, you can specify tolerances for the numerical search here.

**Output Arguments****y0 — Consistent initial values for variables**

numeric column vector

Consistent initial values for variables, returned as a numeric column vector.

**yp0 — Consistent initial values for first derivatives of variables**

numeric column vector

Consistent initial values for first derivatives of variables, returned as a numeric column vector.

**Version History**

**Introduced in R2014b**

**See Also**

`findDecoupledBlocks` | `daeFunction` | `incidenceMatrix` | `isLowIndexDAE` | `massMatrixForm` | `odeFunction` | `reduceDAEIndex` | `reduceDAETooDE` | `reduceDifferentialOrder` | `reduceRedundancies`

**Topics**

“Solve Differential Algebraic Equations (DAEs)” on page 3-66

# derivedUnits

Derived units of unit system

## Syntax

```
derivedUnits(unitSystem)
```

## Description

`derivedUnits(unitSystem)` returns the derived units of the unit system `unitSystem` as a vector of symbolic units. You can use the returned units to create new unit systems by using `newUnitSystem`.

## Examples

### Derived Units of Unit System

Get the derived units of a unit system by using `derivedUnits`. Then, modify the derived units and create a new unit system using the modified derived units. Available unit systems include SI, CGS, and US. For all unit systems, see “Unit Systems List” on page 2-71.

Get the derived units of the SI unit system.

```
dunits = derivedUnits('SI')

dunits =
[ [F], [C], [S], [H], [V], [J], [N], [lx], [lm], [Wb], [W], [Pa],...
  [Ohm], [T], [Gy], [Bq], [Sv], [Hz], [kat], [rad], [sr], [Celsius]]
```

---

**Note** Do not define a variable called `derivedUnits` because the variable will prevent access to the `derivedUnits` function.

---

Define derived units that use kilonewton for force and millibar for pressure by modifying `dunits` using `subs`.

```
u = symunit;
newUnits = subs(dunits,[u.N u.Pa],[u.kN u.mbar])

newUnits =
[ [F], [C], [S], [H], [V], [J], [kN], [lx], [lm], [Wb], [W], [mbar],...
  [Ohm], [T], [Gy], [Bq], [Sv], [Hz], [kat], [rad], [sr], [Celsius]]
```

Define the new unit system by using `newUnitSystem`. Keep the SI base units.

```
bunits = baseUnits('SI');
newUnitSystem('SI_kN_mbar',bunits,newUnits)

ans =
    "SI_kN_mbar"
```

To convert between unit systems, see “Unit Conversions and Unit Systems” on page 2-53.

## **Input Arguments**

### **unitSystem — Name of unit system**

string | character vector

Name of the unit system, specified as a string or character vector.

## **Version History**

**Introduced in R2017b**

## **See Also**

baseUnits | newUnitSystem | removeUnitSystem | rewrite | symunit | unitSystems

## **Topics**

“Units of Measurement Tutorial” on page 2-47

“Unit Conversions and Unit Systems” on page 2-53

“Units and Unit Systems List” on page 2-60

## **External Websites**

The International System of Units (SI)

# det

Determinant of symbolic matrix

## Syntax

```
B = det(A)
B = det(A, 'Algorithm', 'minor-expansion')
B = det(M)
```

## Description

`B = det(A)` returns the determinant of the square matrix of symbolic numbers, scalar variables, or functions `A`.

`B = det(A, 'Algorithm', 'minor-expansion')` uses the minor expansion algorithm to evaluate the determinant of `A`.

`B = det(M)` returns the determinant of the square symbolic matrix variable or matrix function `M`.

## Examples

### Compute Determinant of Matrix with Symbolic Scalar Variables

Compute the determinant of a matrix that contains symbolic scalar variables.

```
syms a b c d
A = [a b; c d];
B = det(A)

B = a*d - b*c
```

### Compute Determinant of Matrix with Symbolic Numbers

Compute the determinant of a matrix that contains symbolic numbers.

```
A = sym([2/3 1/3; 1 1]);
B = det(A)

B =
    1/3
```

### Compute Determinant Using Minor Expansion

Create a symbolic matrix that contains polynomial entries.

```
syms a x
A = [1, a*x^2+x, x;
     0, a*x, 2;
     3*x+2, a*x^2-1, 0]
```

$$A = \begin{pmatrix} 1 & ax^2+x & x \\ 0 & ax & 2 \\ 3x+2 & ax^2-1 & 0 \end{pmatrix}$$

Compute the determinant of the matrix using minor expansion.

```
B = det(A, 'Algorithm', 'minor-expansion')
```

$$B = 3ax^3 + 6x^2 + 4x + 2$$

### Compute Determinant of Block Matrix

Compute the determinant of a 4-by-4 block matrix

$$M = \begin{bmatrix} A & 0_{2,2} \\ C & B \end{bmatrix}$$

where  $A$ ,  $B$ , and  $C$  are 2-by-2 submatrices. The notation  $0_{2,2}$  represents a 2-by-2 submatrix of zeros.

Use symbolic matrix variables to represent the submatrices in the block matrix.

```
syms A B C [2 2] matrix
Z = symmatrix(zeros(2))
```

$$Z = 0_{2,2}$$

$$M = [A \ Z; \ C \ B]$$

$$M = \begin{pmatrix} A & 0_{2,2} \\ C & B \end{pmatrix}$$

Find the determinant of the matrix  $M$ .

```
det(M)
```

$$\text{ans} = \det \begin{pmatrix} A & 0_{2,2} \\ C & B \end{pmatrix}$$

Convert the result from symbolic matrix variables to symbolic scalar variables using `symmatrix2sym`.

```
D1 = simplify(symmatrix2sym(det(M)))
```

$$D1 = (A_{1,1}A_{2,2} - A_{1,2}A_{2,1})(B_{1,1}B_{2,2} - B_{1,2}B_{2,1})$$

Check if the determinant of matrix  $M$  is equal to the determinant of  $A$  times the determinant of  $B$ .



```
D2 = symmatrix2sym(det(A)*det(B))
D2 = (A1,1 A2,2 - A1,2 A2,1) (B1,1 B2,2 - B1,2 B2,1)
isequal(D1,D2)
ans = logical
      1
```

### Compute Determinant of Matrix Polynomial

Compute the determinant of the matrix polynomial  $a_0 \mathbf{I}_2 + \mathbf{A}$ , where  $\mathbf{A}$  is a 2-by-2 matrix.

Create the matrix  $\mathbf{A}$  as a symbolic matrix variable and the coefficient  $a_0$  as a symbolic scalar variable. Create the matrix polynomial as a symbolic matrix function  $f$  with  $a_0$  and  $\mathbf{A}$  as its parameters.

```
syms A [2 2] matrix
syms a0
syms f(a0,A) [2 2] matrix keepargs
f(a0,A) = a0*eye(2) + A

f(a0, A) = a0 I2 + A
```

Find the determinant of  $f$  using `det`. The result is a symbolic matrix function of type `symfunmatrix` that accepts scalars, vectors, and matrices as its input arguments.

```
fInv = det(f)

fInv(a0, A) = det(a0 I2 + A)
```

Convert the result from the `symfunmatrix` data type to the `symfun` data type using `symfunmatrix2symfun`. The result is a symbolic function that accepts scalars as its input arguments.

```
gInv = symfunmatrix2symfun(fInv)

gInv(a0, A1_1, A1_2, A2_1, A2_2) = A1,1 a0 + A2,2 a0 + a02 + A1,1 A2,2 - A1,2 A2,1
```

## Input Arguments

### A — Input matrix

square matrix of symbolic numbers | square matrix of symbolic scalar variables | square matrix of symbolic functions

Input matrix, specified as a square matrix of symbolic numbers, square matrix of symbolic scalar variables, or square matrix of symbolic functions.

Data Types: `single` | `double` | `sym` | `symfun`

### M — Input matrix

square symbolic matrix variable | square symbolic matrix function

Input matrix, specified as a square symbolic matrix variable or square symbolic matrix function.

Data Types: `symmatrix` | `symfunmatrix`

## Tips

- Matrix computations involving many symbolic variables can be slow. To increase the computational speed, reduce the number of symbolic variables by substituting the given values for some variables.
- The minor expansion method is generally useful to evaluate the determinant of a matrix that contains many symbolic scalar variables. This method is often suited to matrices that contain polynomial entries with multivariate coefficients.

## Version History

Introduced before R2006a

### R2022a: Compute determinant of symbolic matrix functions

The `det` function accepts an input argument of type `symfunmatrix`. For an example, see “Compute Determinant of Matrix Polynomial” on page 6-273.

### R2021a: Compute determinant of symbolic matrix variables

The `det` function accepts an input argument of type `symmatrix`. For an example, see “Compute Determinant of Block Matrix” on page 6-272.

## References

- [1] Khovanova, T. and Z. Scully. "Efficient Calculation of Determinants of Symbolic Matrices with Many Variables." arXiv preprint arXiv:1304.4691 (2013).

## See Also

`rank` | `eig`

# diag

Create diagonal matrix or get diagonals from symbolic matrices

## Syntax

```
D = diag(v)
D = diag(v,k)

x = diag(A)
x = diag(A,k)
```

## Description

`D = diag(v)` returns a square diagonal matrix with vector `v` as the main diagonal.

`D = diag(v,k)` places vector `v` on the `k`th diagonal. `k = 0` represents the main diagonal, `k > 0` is above the main diagonal, and `k < 0` is below the main diagonal.

`x = diag(A)` returns the main diagonal of `A`.

`x = diag(A,k)` returns the `k`th diagonal of `A`.

## Examples

### Create Matrix with Diagonal as Vector

Create a symbolic matrix with the main diagonal specified by the vector `v`.

```
syms a b c
v = [a b c];
diag(v)

ans =
[ a, 0, 0]
[ 0, b, 0]
[ 0, 0, c]
```

### Create Matrix with Subdiagonal as Vector

Create a symbolic matrix with the second diagonal below the main diagonal specified by the vector `v`.

```
syms a b c
v = [a b c];
diag(v,-2)

ans =
[ 0, 0, 0, 0, 0]
[ 0, 0, 0, 0, 0]
[ a, 0, 0, 0, 0]
```

```
[ 0, b, 0, 0, 0]
[ 0, 0, c, 0, 0]
```

### Extract Diagonal from Matrix

Extract the main diagonal from a square matrix.

```
syms x y z
A = magic(3).*[x, y, z];
diag(A)
```

```
ans =
 8*x
 5*y
 2*z
```

### Extract Superdiagonal from Matrix

Extract the first diagonal above the main diagonal.

```
syms x y z
A = magic(3).*[x, y, z];
diag(A,1)
```

```
ans =
  y
 7*z
```

## Input Arguments

### **v — Diagonal elements**

symbolic vector

Diagonal elements, specified as a symbolic vector. If  $v$  is a vector with  $N$  elements, then `diag(v,k)` is a square matrix of order  $N + \text{abs}(k)$ .

### **A — Input matrix**

symbolic matrix

Input matrix, specified as a symbolic matrix.

### **k — Diagonal number**

integer

Diagonal number, specified as an integer.  $k = 0$  represents the main diagonal,  $k > 0$  is above the main diagonal, and  $k < 0$  is below the main diagonal.

## Tips

- The trace of a matrix is equal to `sum(diag(A))`.

## **Version History**

Introduced before R2006a

## **See Also**

tril | triu

## diff

Differentiate symbolic expression or function

### Syntax

```
Df = diff(f)
Df = diff(f,n)
Df = diff(f,var)
Df = diff(f,var,n)
Df = diff(f,var1,...,varN)

Df = diff(f,mvar)
```

### Description

`Df = diff(f)` differentiates `f` with respect to the symbolic scalar variable in the definition of `f`. The variable of differentiation is determined by `symvar(f,1)`.

`Df = diff(f,n)` computes the `n`th derivative of `f` with respect to the symbolic scalar variable determined by `symvar`.

`Df = diff(f,var)` differentiates `f` with respect to the differentiation parameter `var`. `var` can be a symbolic scalar variable, such as `x`, a symbolic function, such as `f(x)`, or a derivative function, such as `diff(f(t),t)`.

`Df = diff(f,var,n)` computes the `n`th derivative of `f` with respect to `var`.

`Df = diff(f,var1,...,varN)` differentiates `f` with respect to the parameters `var1,...,varN`.

`Df = diff(f,mvar)` differentiates `f` with respect to the symbolic matrix variable or symbolic matrix function.

### Examples

#### Differentiate Function

Find the derivative of the function  $f(x) = \sin(x^2)$ .

```
syms f(x)
f(x) = sin(x^2);
Df = diff(f,x)
```

$$Df(x) = 2x \cos(x^2)$$

Find the value of the derivative at  $x = 2$ . Convert the value to double.

```
Df2 = Df(2)

Df2 = 4 cos(4)
```

```
double(Df2)
```

```
ans =  
-2.6146
```

### Differentiate with Respect to Specified Variable

Find the first derivative of the expression  $\sin(x*t^2)$ .

```
syms x t  
Df = diff(sin(x*t^2))
```

$$Df = t^2 \cos(t^2 x)$$

Because you did not specify the differentiation variable, `diff` uses the default variable determined by `symvar`. For this expression, the default variable is  $x$ .

```
var = symvar(sin(x*t^2),1)
```

```
var = x
```

Now, find the derivative of this expression with respect to the variable  $t$ .

```
Df = diff(sin(x*t^2),t)
```

$$Df = 2tx \cos(t^2 x)$$

### Higher-Order Derivatives of Univariate Expression

Find the fourth, fifth, and sixth derivatives of  $t^6$ .

```
syms t  
D4 = diff(t^6,4)
```

$$D4 = 360t^2$$

```
D5 = diff(t^6,5)
```

$$D5 = 720t$$

```
D6 = diff(t^6,6)
```

$$D6 = 720$$

### Higher-Order Derivatives of Multivariate Expression with Respect to Specified Variable

Find the second derivative of the expression  $x*\cos(x*y)$  with respect to the variable  $y$ .

```
syms x y  
Df = diff(x*cos(x*y),y,2)
```

$$Df = -x^3 \cos(xy)$$

### Higher-Order Derivatives of Multivariate Expression with Respect to Default Variable

Find the second derivative of the expression  $x*y$ . If you do not specify the differentiation variable, `diff` uses the variable determined by `symvar`. Because `symvar( $x*y$ , 1)` returns  $x$ , `diff` computes the second derivative of  $x*y$  with respect to  $x$ .

```
syms x y
Df = diff(x*y, 2)

Df = 0
```

If you use nested `diff` calls and do not specify the differentiation variable, `diff` determines the differentiation variable for each call. For example, find the second derivative of the expression  $x*y$  by calling the `diff` function twice.

```
Df = diff(diff(x*y))

Df = 1
```

In the first call, `diff` differentiates  $x*y$  with respect to  $x$  and returns  $y$ . In the second call, `diff` differentiates  $y$  with respect to  $y$  and returns 1.

So, `diff( $x*y$ , 2)` is equivalent to `diff( $x*y$ ,  $x$ ,  $x$ )`, and `diff(diff( $x*y$ ))` is equivalent to `diff( $x*y$ ,  $x$ ,  $y$ )`.

### Mixed Derivatives

Differentiate the expression  $x*\sin(x*y)$  with respect to the variables  $x$  and  $y$ .

```
syms x y
Df = diff(x*sin(x*y), x, y)

Df = 2*x*cos(x*y) - x^2*y*sin(x*y)
```

You also can compute mixed higher-order derivatives by specifying all differentiation variables. Find the mixed fourth derivative of the expression with respect to the variables  $x$ ,  $x$ ,  $x$ , and then  $y$ .

```
syms x y
Df = diff(x*sin(x*y), x, x, x, y)

Df = x^2*y^3*sin(x*y) - 6*x*y^2*cos(x*y) - 6*y*sin(x*y)
```

### Differentiate with Respect to Function and Derivative

Find the derivative of the function  $y = f(x)^2 \frac{df}{dx}$  with respect to  $f(x)$ .



```
syms f(x) y
y = f(x)^2*diff(f(x),x);
Dy = diff(y,f(x))
```

$$Dy = 2 f(x) \frac{\partial}{\partial x} f(x)$$

Find the second derivative of the function  $y = f(x)^2 \frac{df}{dx}$  with respect to  $f(x)$ .

```
Dy2 = diff(y,f(x),2)
```

$$Dy2 = 2 \frac{\partial}{\partial x} f(x)$$

Find the mixed derivative of the function  $y = f(x)^2 \frac{df}{dx}$  with respect to  $f(x)$  and  $\frac{df}{dx}$ .

```
Dy3 = diff(y,f(x),diff(f(x)))
```

$$Dy3 = 2 f(x)$$

### Euler-Lagrange Equation

Find the Euler-Lagrange equation that describes the motion of a mass-spring system. Define the kinetic and potential energy of the system.

```
syms x(t) m k
T = m/2*diff(x(t),t)^2;
V = k/2*x(t)^2;
```

Define the Lagrangian.

$$L = T - V$$

$$L = \frac{m \left( \frac{\partial}{\partial t} x(t) \right)^2}{2} - \frac{k x(t)^2}{2}$$

The Euler-Lagrange equation is given by

$$0 = \frac{d}{dt} \frac{\partial L(t, x, \dot{x})}{\partial \dot{x}} - \frac{\partial L(t, x, \dot{x})}{\partial x}.$$

Evaluate the term  $\partial L / \partial \dot{x}$ .

```
D1 = diff(L,diff(x(t),t))
```

$$D1 = m \frac{\partial}{\partial t} x(t)$$

Evaluate the second term  $\partial L / \partial x$ .

```
D2 = diff(L,x)
```

```
D2(t) = -k x(t)
```

Find the Euler-Lagrange equation of motion of the mass-spring system.

```
diff(D1,t) - D2 == 0
```

```
ans(t) =
```

$$m \frac{\partial^2}{\partial t^2} x(t) + k x(t) = 0$$

### Differentiate with Respect to Vectors

To evaluate derivatives with respect to vectors, you can use symbolic matrix variables. For example, find the derivatives  $\partial\alpha/\partial x$  and  $\partial\alpha/\partial y$  for the expression  $\alpha = y^T A x$ , where  $y$  is a 3-by-1 vector,  $A$  is a 3-by-4 matrix, and  $x$  is a 4-by-1 vector.

Create three symbolic matrix variables  $x$ ,  $y$ , and  $A$ , of the appropriate sizes, and use them to define  $\alpha$ .

```
syms x [4 1] matrix
syms y [3 1] matrix
syms A [3 4] matrix
alpha = y.'*A*x
```

```
alpha = y^T A x
```

Find the derivative of  $\alpha$  with respect to the vectors  $x$  and  $y$ .

```
Dx = diff(alpha,x)
```

```
Dx = y^T A
```

```
Dy = diff(alpha,y)
```

```
Dy = x^T A^T
```

### Differentiate with Respect to Matrix

To evaluate derivatives with respect to matrices, you can use symbolic matrix variables. For example, find the derivative  $\partial Y/\partial A$  for the expression  $Y = X^T A X$ , where  $X$  is a 3-by-1 vector, and  $A$  is a 3-by-3 matrix. Here,  $Y$  is a scalar that is a function of the vector  $X$  and the matrix  $A$ .

Create two symbolic matrix variables to represent  $X$  and  $A$ . Define  $Y$ .

```
syms X [3 1] matrix
syms A [3 3] matrix
Y = X.'*A*X
```

```
Y = X^T A X
```

Find the derivative of  $Y$  with respect to the matrix  $A$ .

```
D = diff(Y,A)
```

```
D = XT ⊗ X
```

The result is a Kronecker tensor product of  $X^T$  and  $X$ , which is a 3-by-3 matrix.

```
size(D)
```

```
ans = 1×2
```

```
3      3
```

### Differentiate Symbolic Matrix Function

Differentiate a symbolic matrix function with respect to its matrix argument.

Find the derivative of the function  $t(\mathbf{X}) = \mathbf{A} \cdot \sin(\mathbf{B} \cdot \mathbf{X})$ , where  $\mathbf{A}$  is a 1-by-3 matrix,  $\mathbf{B}$  is a 3-by-2 matrix, and  $\mathbf{X}$  is a 2-by-1 matrix. Create symbolic matrix variables to represent  $\mathbf{A}$ ,  $\mathbf{B}$ , and  $\mathbf{X}$ , and create a symbolic matrix function to represent  $t(\mathbf{X})$ .

```
syms A [1 3] matrix
syms B [3 2] matrix
syms X [2 1] matrix
syms t(X) [1 1] matrix keepargs
t(X) = A*sin(B*X)
```

```
t(X) = A sin(B X)
```

Differentiate the function with respect to  $\mathbf{X}$ .

```
Dt = diff(t,X)
```

```
Dt(X) = A (cos(B X) ⊙ B)
```

### Find Gradient with Respect to Vector

To find the gradient of a scalar expression with respect to a vector, you can use a symbolic matrix variable as the differentiation parameter.

Create a symbolic matrix variable  $X$  to represent a vector with three components. To see how these components are stored in Symbolic Math Toolbox™, use `symmatrix2sym` to display the elements of the symbolic matrix variable.

```
syms X [1 3] matrix
symmatrix2sym(X)
```

```
ans = (X1,1 X1,2 X1,3)
```

The components of the symbolic matrix variable are `X1_1`, `X1_2`, and `X1_3`. Create three symbolic scalar variables for these components. Create a scalar symbolic expression `expr` using these scalar variables.

```
syms X1_1 X1_2 X1_3
expr = 2*X1_2*sin(X1_1) + 3*sin(X1_3)*cos(X1_2);
```

Find the gradient of the scalar expression `expr` with respect to `X`. The `diff` function finds the first partial derivatives of `expr` with respect to each component of `X`.

```
g = diff(expr,X)
```

```
g =
    Σ1
where
```

$$\Sigma_1 = \begin{pmatrix} 2 X_{1,2} \cos(X_{1,1}) \\ 2 \sin(X_{1,1}) - 3 \sin(X_{1,2}) \sin(X_{1,3}) \\ 3 \cos(X_{1,2}) \cos(X_{1,3}) \end{pmatrix}$$

## Input Arguments

### **f** — Expression or function to differentiate

symbolic expression | symbolic function | symbolic vector | symbolic matrix | symbolic matrix variable  
| symbolic matrix function | numeric expression

Expression or function to differentiate, specified as one of these values:

- Symbolic expression
- Symbolic function
- Symbolic vector or symbolic matrix (a vector or a matrix of symbolic expressions or functions)
- Symbolic matrix variable
- Symbolic matrix function
- Numeric expression

If `f` is a symbolic vector or matrix, `diff` differentiates each element of `f` and returns a vector or a matrix of the same size as `f`.

Data Types: `sym` | `symfun` | `symmatrix` | `symfunmatrix` | `double` | `single`

### **n** — Order of derivative

nonnegative integer

Order of derivative, specified as a nonnegative integer.

### **var** — Differentiation parameter

symbolic scalar variable | symbolic function | derivative function

Differentiation parameter, specified as a symbolic scalar variable, symbolic function, or a derivative function created using the `diff` function.

If you specify differentiation with respect to the symbolic function `var = f(x)` or the derivative function `var = diff(f(x),x)`, then the first argument `f` must not contain any of these values:

- Integral transforms, such as `fourier`, `ifourier`, `laplace`, `ilaplace`, `htrans`, `ihtrans`, `ztrans`, and `iztrans`
- Unevaluated symbolic expressions that include `limit` or `int`
- Symbolic functions evaluated at a specific point, such as `f(3)` or `g(0)`

Data Types: `sym` | `symfun`

### **var1,...,varN — Differentiation parameters**

symbolic scalar variables | symbolic functions | derivative functions

Differentiation parameters, specified as symbolic scalar variables, symbolic functions, or derivative functions created using the `diff` function.

Data Types: `sym` | `symfun`

### **mvar — Differentiation parameter in the form of matrix**

symbolic matrix variable | symbolic matrix function

Differentiation parameter in the form of a matrix, specified as a symbolic matrix variable or symbolic matrix function.

When using a symbolic matrix variable or function as the differentiation parameter, `f` must be a differentiable scalar function or expression, where `mvar` can represent a scalar, vector, or matrix. The derivative of `f` cannot be a tensor or a matrix in terms of tensors. For examples, see “Differentiate with Respect to Vectors” on page 6-282 and “Differentiate with Respect to Matrix” on page 6-282.

When differentiating a scalar function `f` with respect to a vector or matrix `mvar`, `diff` uses the convention of returning an output size that is the transpose of the input size `mvar`. For example, if `f` is a 1-by-1 scalar and `mvar` is a 1-by-3 row vector, then `diff(f,mvar)` finds the derivative of `f` with respect to each element of the transposed `mvar` and returns the result as a 3-by-1 column vector.

Data Types: `symmatrix` | `symfunmatrix`

## **Limitations**

- The `diff` function does not support tensor derivatives when using a symbolic matrix variable as the differentiation parameter. If the derivative is a tensor, or the derivative is a matrix in terms of tensors, then the `diff` function generates an error.

## **Tips**

- When computing mixed higher-order derivatives with more than one variable, do not use `n` to specify the order of the derivative. Instead, specify all differentiation variables explicitly.
- To improve performance, `diff` assumes that all mixed derivatives commute. For example,

$$\frac{\partial}{\partial x} \frac{\partial}{\partial y} f(x, y) = \frac{\partial}{\partial y} \frac{\partial}{\partial x} f(x, y)$$

This assumption suffices for most engineering and scientific problems.

- If you differentiate a multivariate expression or function `f` without specifying the differentiation variable, then a nested call to `diff` and `diff(f,n)` can return different results. The reason is

that in a nested call, each differentiation step determines and uses its own differentiation variable. In calls like `diff(f,n)`, the differentiation variable is determined once by `symvar(f,1)` and used for all differentiation steps.

- If you differentiate an expression or function containing `abs` or `sign`, the arguments must be real values. For complex arguments of `abs` and `sign`, the `diff` function formally computes the derivative, but this result is not generally valid because `abs` and `sign` are not differentiable over complex numbers.

## Version History

Introduced before R2006a

### **R2023b: Differentiate expressions or functions with respect to symbolic matrix variables and symbolic matrix functions**

The `diff` function can differentiate expressions or functions of type `sym`, `symfun`, `symmatrix`, `symfunmatrix`, `double`, and `single` with respect to differentiation parameters of type `symmatrix` and `symfunmatrix`. For an example, see “Find Gradient with Respect to Vector” on page 6-283.

### **R2022a: Differentiate symbolic matrix functions**

The `diff` function accepts an input argument of type `symfunmatrix`. You can differentiate symbolic matrix functions with respect to differentiation parameters of type `sym`, `symfun`, `symmatrix`, `symfunmatrix`, `double`, and `single`. For an example, see “Differentiate Symbolic Matrix Function” on page 6-283.

### **R2021a: Differentiate symbolic matrix variables**

The `diff` function accepts an input argument of type `symmatrix`. You can differentiate symbolic matrix variables with respect to differentiation parameters of type `sym`, `symfun`, `symmatrix`, `double`, and `single`. For examples, see “Differentiate with Respect to Vectors” on page 6-282 and “Differentiate with Respect to Matrix” on page 6-282.

## See Also

### Functions

`curl` | `divergence` | `functionalDerivative` | `gradient` | `hessian` | `int` | `jacobian` | `laplacian` | `symvar`

### Topics

“Differentiation” on page 3-181

“Find Asymptotes, Critical, and Inflection Points” on page 3-222

### Teaching Resources

Calculus Derivatives (MathWorks Teaching Resources)

Beam Bending and Deflection (MathWorks Teaching Resources)

# digits

Change variable precision used

## Syntax

```
digits(d)
d1 = digits
d1 = digits(d)
```

## Description

`digits(d)` sets the precision of symbolic computations involving variable-precision arithmetic, such as `vpa`, to `d` significant decimal digits. The default is 32 digits.

`d1 = digits` returns the current precision used by `vpa`.

`d1 = digits(d)` sets the new precision `d` and returns the old precision in `d1`.

## Examples

### Increase Precision of Results

By default, MATLAB® uses 16 digits of precision. For higher precision, use `vpa`. The default precision for `vpa` is 32 digits. Increase precision beyond 32 digits by using `digits`.

Find `pi` using `vpa`, which uses the default 32 digits of precision. Confirm that the current precision is 32 by using `digits`.

```
pi32 = vpa(pi)
pi32 = 3.1415926535897932384626433832795
currentPrecision = digits
currentPrecision =
32
```

Save the current value of `digits` in `d1` and set the new precision to 70 digits. Find `pi` using `vpa`. The result has 70 digits.

```
d1 = digits(70);
pi100 = vpa(pi)
pi100 = 3.141592653589793238462643383279502884197169399375105820974944592307816
```

Note that `vpa` returns a symbolic output. To use symbolic output with a MATLAB function that does not accept symbolic values, convert symbolic values to double precision by using `double`.

Lastly, restore the old value of `digits` for further calculations.

```
digits(d1)
```

For more information, see “Increase Precision of Numeric Calculations” on page 2-36.

### Increase Speed by Decreasing Precision

Increase the speed of MATLAB calculations by using `vpa` with a lower precision. Set the lower precision by using `digits`.

First, find the time taken to perform an operation on a large input.

```
input = 1:0.01:500;  
tic  
zeta(input);  
toc
```

Elapsed time is 36.946036 seconds.

Now, repeat the operation with a lower precision by using `vpa`. Lower the precision to 8 digits by using `digits`. Then, use `vpa` to reduce the precision of `input` and perform the same operation. The time taken decreases significantly.

```
d1 = digits(8);  
vpaInput = vpa(input);  
tic  
zeta(vpaInput);  
toc
```

Elapsed time is 19.651537 seconds.

Lastly, restore the old value of `digits` for further calculations.

```
digits(d1)
```

For more information, see “Increase Speed by Reducing Precision” on page 3-386.

### Guard Digits

The number of digits that you specify using the `vpa` function or the `digits` function is the guaranteed number of digits. Internally, the toolbox can use a few more digits than you specify. These additional digits are called *guard digits*. For example, set the number of digits to 4, and then display the floating-point approximation of  $1/3$  using four digits.

```
d1 = digits(4);  
a = vpa(1/3)  
  
a = 0.3333
```

Now, display `a` using 20 digits. The result shows that the toolbox internally used more than four digits when computing `a`. The last digits in the following result are incorrect because of the round-off error.

```
d2 = digits(20);  
vpa(a)  
  
ans = 0.333333333333333303016843
```



```
digits(d1)
```

Hidden round-off errors can cause unexpected results. For example, compute the number  $1/10$  with the default 32-digit accuracy and with 10-digit accuracy.

$$a = 0.1$$

```
b = vpa(1/10)
```

```
digits(d1)
```

$$c = a - b$$

The difference  $a - b$  is not equal to zero because the toolbox internally boosts the 10-digit number  $b = 0.1$  to 32-digit accuracy. This process implies round-off errors. The toolbox actually computes the difference  $a - b$  as follows.

[illegible]

c = 0.00000000000000000000867361737988403547206008158444403

Suppose you convert a double number to a symbolic object, and then apply the `vpa` function on that object. The result can depend on the conversion technique that you used to convert a floating-point number to a symbolic object. The `sym` function lets you choose the conversion technique by specifying the optional second argument, which can be "r", "f", "d", or "e". The default is "r". For example, convert the constant  $\pi = 3.141592653589793\dots$  to a symbolic object.

$$r = \Pi$$
$$f =$$

**6-289**

```
d = sym(pi, "d")
```

```
d = 3.1415926535897931159979634685442
```

```
e = sym(pi, "e")
```

```
e =
```

$$\pi - \frac{198 \text{ eps}}{359}$$

Although the toolbox displays these numbers differently on the screen, they are rational approximations of `pi`. Use `vpa` to convert these rational approximations of `pi` back to floating-point values.

Set the number of digits to 4. Three of the four approximations give the same result.

```
d1 = digits(4);
```

```
rvpa = vpa(r)
```

```
rvpa = 3.142
```

```
fvpa = vpa(f)
```

```
fvpa = 3.142
```

```
dvpa = vpa(d)
```

```
dvpa = 3.142
```

```
evpa = vpa(e)
```

```
evpa = 3.142 - 0.5515 eps
```

Now, set the number of digits to 40. The differences between the symbolic approximations of `pi` become more visible.

```
d2 = digits(40);
```

```
rvpa = vpa(r)
```

```
rvpa = 3.141592653589793238462643383279502884197
```

```
fvpa = vpa(f)
```

```
fvpa = 3.141592653589793115997963468544185161591
```

```
dvpa = vpa(d)
```

```
dvpa = 3.1415926535897931159979634685442
```

```
evpa = vpa(e)
```

```
evpa = 3.141592653589793238462643383279502884197 - 0.5515320334261838440111420612813370473538 eps
```

Restore the old value of `digits` for further calculations.

```
digits(d1)
```

## Input Arguments

### **d — New accuracy setting**

positive integer scalar

New accuracy setting, specified as a positive integer scalar. The setting specifies the number of significant decimal digits to be used for variable-precision calculations. **d** must be greater than 1 and less than  $10^8 + 1$ . If the value **d** is not an integer, **digits** rounds it to the nearest integer.

## Output Arguments

### **d1 — Current accuracy setting**

double-precision number

Current accuracy setting, returned as a double-precision number. The setting specifies the number of significant decimal digits currently used for variable-precision calculations.

## Version History

Introduced before R2006a

## See Also

`double` | `vpa`

### Topics

"Increase Precision of Numeric Calculations" on page 2-36

"Recognize and Avoid Round-Off Errors" on page 3-382

"Increase Speed by Reducing Precision" on page 3-386

"Change Output Format of Symbolic and Variable-Precision Arithmetic" on page 2-8

## dilog

Dilogarithm function

### Syntax

`dilog(X)`

### Description

`dilog(X)` returns the dilogarithm function.

### Examples

#### Dilogarithm Function for Numeric and Symbolic Arguments

Depending on its arguments, `dilog` returns floating-point or exact symbolic results.

Compute the dilogarithm function for these numbers. Because these numbers are not symbolic objects, `dilog` returns floating-point results.

```
A = dilog([-1, 0, 1/4, 1/2, 1, 2])
```

```
A =  
    2.4674 - 2.1776i    1.6449 + 0.0000i    0.9785 + 0.0000i...  
    0.5822 + 0.0000i    0.0000 + 0.0000i   -0.8225 + 0.0000i
```

Compute the dilogarithm function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `dilog` returns unresolved symbolic calls.

```
symA = dilog(sym([-1, 0, 1/4, 1/2, 1, 2]))
```

```
symA =  
[ pi^2/4 - pi*log(2)*1i, pi^2/6, dilog(1/4), pi^2/12 - log(2)^2/2, 0, -pi^2/12]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

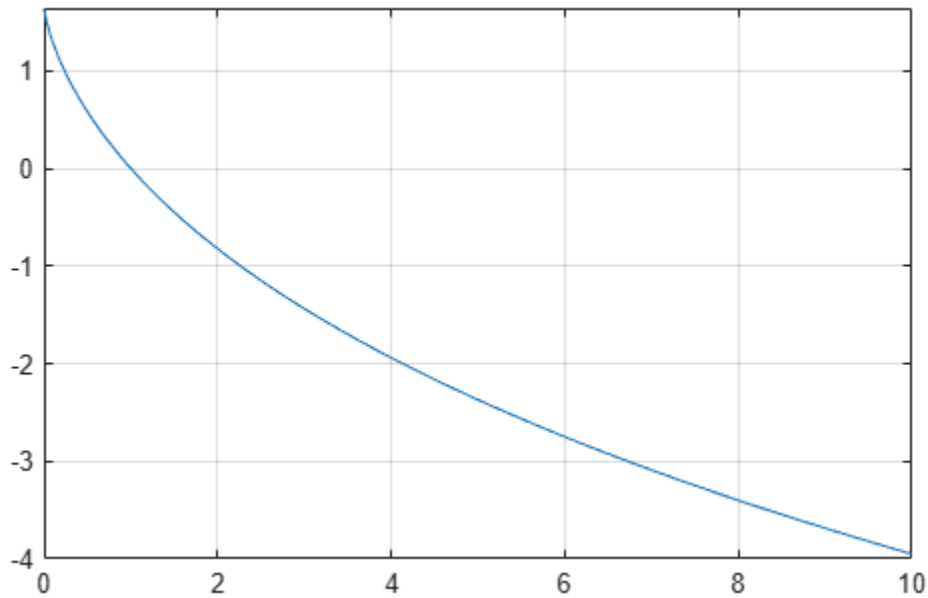
```
vpa(symA)
```

```
ans =  
[ 2.467401100272339654708622749969 - 2.1775860903036021305006888982376i,...  
 1.644934066848226436472415166646,...  
 0.97846939293030610374306666652456,...  
 0.58224052646501250590265632015968,...  
 0,...  
 -0.82246703342411321823620758332301]
```

#### Plot Dilogarithm Function

Plot the dilogarithm function on the interval from 0 to 10.

```
syms x
fplot(dilog(x),[0 10])
grid on
```



### Handle Expressions Containing Dilogarithm Function

Many functions, such as `diff`, `int`, and `limit`, can handle expressions containing `dilog`.

Find the first and second derivatives of the dilogarithm function:

```
syms x
diff(dilog(x), x)
diff(dilog(x), x, x)

ans =
-log(x)/(x - 1)

ans =
log(x)/(x - 1)^2 - 1/(x*(x - 1))
```

Find the indefinite integral of the dilogarithm function:

```
int(dilog(x), x)

ans =
x*(dilog(x) + log(x) - 1) - dilog(x)
```

Find the limit of this expression involving `dilog`:

```
limit(dilog(x)/x, Inf)

ans =
0
```

## Input Arguments

### X — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## More About

### Dilogarithm Function

There are two common definitions of the dilogarithm function.

The implementation of the `dilog` function uses the following definition:

$$\text{dilog}(x) = \int_1^x \frac{\ln(t)}{1-t} dt$$

Another common definition of the dilogarithm function is

$$\text{Li}_2(x) = \int_x^0 \frac{\ln(1-t)}{t} dt$$

Thus,  $\text{dilog}(x) = \text{Li}_2(1-x)$ .

## Tips

- `dilog(sym(-1))` returns  $\pi^2/4 - \pi \cdot \log(2) \cdot i$ .
- `dilog(sym(0))` returns  $\pi^2/6$ .
- `dilog(sym(1/2))` returns  $\pi^2/12 - \log(2)^2/2$ .
- `dilog(sym(1))` returns 0.
- `dilog(sym(2))` returns  $-\pi^2/12$ .
- `dilog(sym(i))` returns  $\pi^2/16 - (\pi \cdot \log(2) \cdot i)/4 - \text{catalan} \cdot i$ .
- `dilog(sym(-i))` returns  $\text{catalan} \cdot i + (\pi \cdot \log(2) \cdot i)/4 + \pi^2/16$ .
- `dilog(sym(1 + i))` returns  $-\text{catalan} \cdot i - \pi^2/48$ .
- `dilog(sym(1 - i))` returns  $\text{catalan} \cdot i - \pi^2/48$ .
- `dilog(sym(Inf))` returns  $-\text{Inf}$ .

## Version History

Introduced in R2014a

## References

- [1] Stegun, I. A. "Miscellaneous Functions." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

log | zeta

## dirac

Dirac delta function

### Syntax

```
d = dirac(x)
d = dirac(n,x)
```

### Description

`d = dirac(x)` represents the Dirac delta function on page 6-299 of `x`.

`d = dirac(n,x)` represents the  $n$ th derivative of the Dirac delta function at  $x$ .

### Examples

#### Handle Expressions Involving Dirac and Heaviside Functions

Compute derivatives and integrals of expressions involving the Dirac delta and Heaviside functions.

Find the first and second derivatives of the Heaviside function. The result is the Dirac delta function and its first derivative.

```
syms x
Dx = diff(heaviside(x), x)

Dx =  $\delta(x)$ 

D2x = diff(heaviside(x), x, x)

D2x =  $\delta'(x)$ 
```

Find the indefinite integral of the Dirac delta function. The results returned by `int` do not include integration constants.

```
f = int(dirac(x), x)

f = heaviside(x)
```

Find the integral of the sine function involving the Dirac delta function.

```
syms a
f = int(dirac(x - a)*sin(x), x, -Inf, Inf)

f = sin(a)
```

#### Use Assumptions on Variables

`dirac` takes into account assumptions on variables.



```
syms x real
assumeAlso(x ~= 0)
d = dirac(x)
```

```
d = 0
```

For further computations, clear the assumptions on  $x$  by recreating it using `syms`.

```
syms x
```

### Evaluate Dirac Delta Function for Symbolic Matrix

Compute the Dirac delta function of  $x$  and its first three derivatives.

Use a vector  $n = [0, 1, 2, 3]$  to specify the order of derivatives. The `dirac` function expands the scalar into a vector of the same size as  $n$  and computes the result.

```
syms x
n = [0, 1, 2, 3];
d = dirac(n, x)

d = (delta(x) delta'(x) delta''(x) delta'''(x))
```

Substitute  $x$  with  $0$ .

```
d0 = subs(d, x, 0)

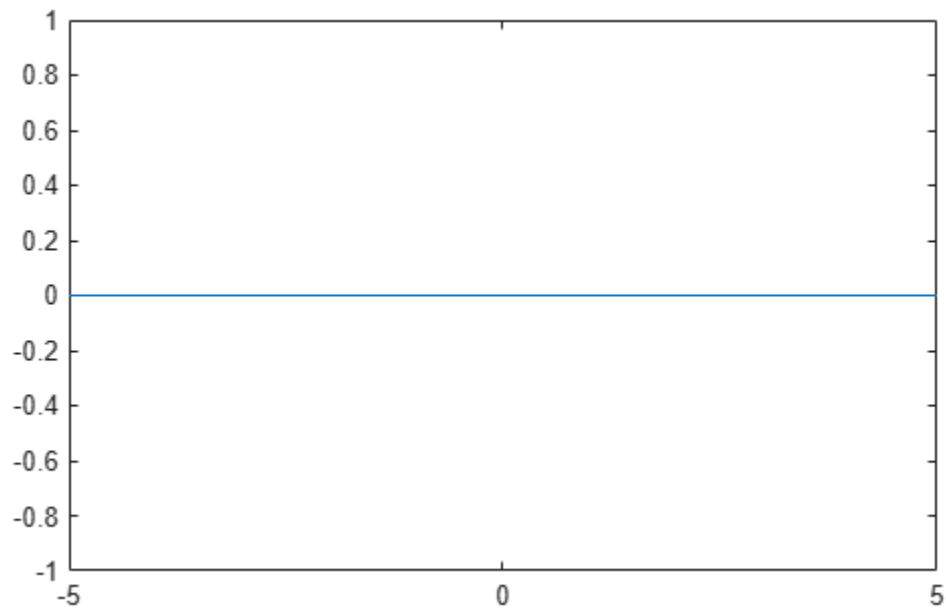
d0 = (inf - inf inf - inf)
```

### Plot Dirac Delta Function

You can use `fplot` to plot the Dirac delta function over the default interval  $[-5, 5]$ . However, `dirac(x)` returns `Inf` at  $x$  equal to  $0$ , and `fplot` does not plot the infinity.

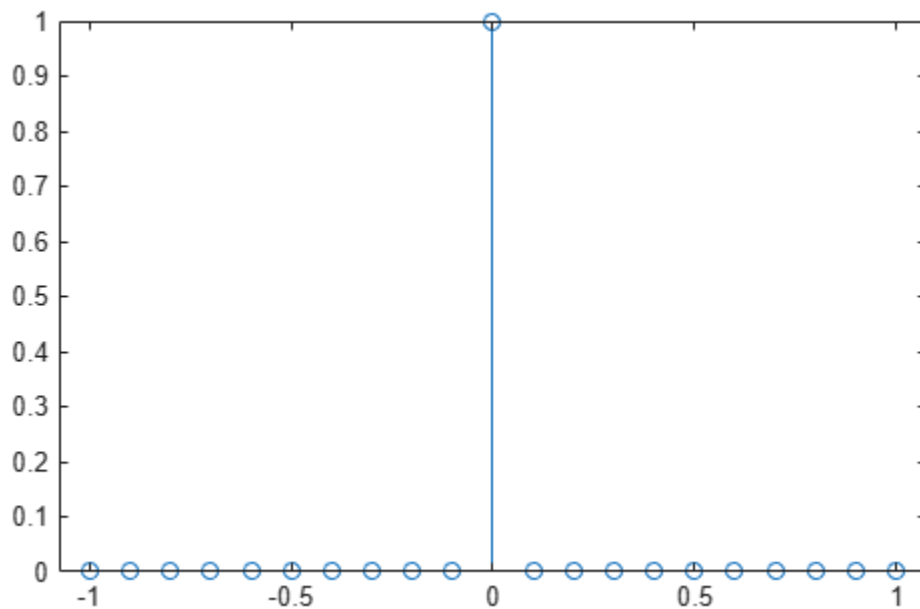
Declare a symbolic variable  $x$  and plot the symbolic expression `dirac(x)` by using `fplot`.

```
syms x
fplot(dirac(x))
```



To handle the infinity at  $x$  equal to 0, use numeric values instead of symbolic values. Set the Inf value to 1 and plot the Dirac delta function by using `stem`.

```
x = -1:0.1:1;  
y = dirac(x);  
idx = y == Inf; % find Inf  
y(idx) = 1;    % set Inf to finite value  
stem(x,y)
```



## Input Arguments

### **x — Input**

number | symbolic number | symbolic variable | symbolic expression | symbolic function | vector | matrix | multidimensional array

Input, specified as a number, symbolic number, variable, expression, or function, representing a real number. This input can also be a vector, matrix, or multidimensional array of numbers, symbolic numbers, variables, expressions, or functions.

### **n — Order of derivative**

nonnegative number | symbolic variable | symbolic expression | symbolic function | vector | matrix | multidimensional array

Order of derivative, specified as a nonnegative number, or symbolic variable, expression, or function representing a nonnegative number. This input can also be a vector, matrix, or multidimensional array of nonnegative numbers, symbolic numbers, variables, expressions, or functions.

## More About

### **Dirac Delta Function**

The Dirac delta function,  $\delta(x)$ , has the value 0 for all  $x \neq 0$ , and  $\infty$  for  $x = 0$ . The Dirac delta function satisfies the identity

$$\int_{-\infty}^{\infty} \delta(x) dx = 1 .$$

This is a heuristic definition of the Dirac delta function. A rigorous definition of the Dirac delta function requires the theory of distributions or measure theory.

For any smooth function  $f$  and a real number  $a$ , the Dirac delta function has the property

$$\int_{-\infty}^{\infty} \delta(x - a) f(x) = f(a) .$$

## Tips

- For complex values  $x$  with nonzero imaginary parts, `dirac` returns `NaN`.
- `dirac` returns floating-point results for numeric arguments that are not symbolic objects.
- `dirac` acts element-wise on nonscalar inputs.
- The input arguments  $x$  and  $n$  must be vectors or matrices of the same size, or else one of them must be a scalar. If one input argument is a scalar and the other one is a vector or a matrix, then `dirac` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

## Version History

Introduced before R2006a

**See Also**

heaviside | kroneckerDelta

# displayFormula

Display symbolic formula from string

## Syntax

```
displayFormula(symstr)
displayFormula(symstr,old,new)
```

## Description

`displayFormula(symstr)` displays the symbolic formula from the string `symstr` without evaluating the operations. All workspace variables that are specified in `symstr` are replaced by their values.

`displayFormula(symstr,old,new)` replaces only the expression or variable `old` with `new`. Expressions or variables other than `old` are not replaced by their values.

## Examples

### Multiplication Formula of Matrix and Scalar

Create a 3-by-3 matrix. Multiply the matrix by the scalar coefficient  $K^2$ .

```
syms K A
A = [-1, 0, 1; 1, 2, 0; 1, 1, 0];
B = K^2*A
```

$$B = \begin{pmatrix} -K^2 & 0 & K^2 \\ K^2 & 2K^2 & 0 \\ K^2 & K^2 & 0 \end{pmatrix}$$

The result automatically shows the multiplication being carried out element-wise.

Show the multiplication formula without evaluating the operations by using `displayFormula`. Input the formula as a string. The variable `A` in the string is replaced by its values.

```
displayFormula("F = K^2*A")
```

$$F = K^2 \begin{pmatrix} -1 & 0 & 1 \\ 1 & 2 & 0 \\ 1 & 1 & 0 \end{pmatrix}$$

### Multiplication Formula of Matrix and Vector

Create a 3-by-3 matrix and a 3-by-1 vector. Create a symbolic equation that multiplies the matrix and the vector.

```
syms A [3 3]
syms v B [3 1]
eqn = B == A*v
```

```
eqn =

$$\begin{pmatrix} B_1 = A_{1,1} v_1 + A_{1,2} v_2 + A_{1,3} v_3 \\ B_2 = A_{2,1} v_1 + A_{2,2} v_2 + A_{2,3} v_3 \\ B_3 = A_{3,1} v_1 + A_{3,2} v_2 + A_{3,3} v_3 \end{pmatrix}$$

```

The result shows the multiplication being carried out where the elements of the matrix and the vector are combined.

Show the multiplication formula without combining the elements by using `displayFormula`. Input the formula as a string.

```
displayFormula("B == A*v")
```

$$\begin{pmatrix} B_1 \\ B_2 \\ B_3 \end{pmatrix} = \begin{pmatrix} A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,1} & A_{3,2} & A_{3,3} \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix}$$

### Display Differential Equation

Define a string that describes a differential equation.

```
S = "m*diff(y,t,t) == m*g-k*y";
```

Create a string array that combines the differential equation and additional text. Display the formula along with the text.

```
symstr = ["The equation of motion is"; S; "where k is the elastic coefficient."];
displayFormula(symstr)
```

The equation of motion is

$$m \frac{\partial^2}{\partial t^2} y = m g - k y$$

where k is the elastic coefficient.

### Display and Evaluate Symbolic Expression

Create a string S representing a symbolic expression.

```
S = "exp(2*pi*i)";
```

Create another string `symstr` that contains S.

```
symstr = "1 + S + S^2 + cos(S)"
```

```
symstr =
"1 + S + S^2 + cos(S) "
```

Display `symstr` as a formula without evaluating the operations by using `displayFormula`. `S` in `symstr` is replaced by its value.

```
displayFormula(symstr)
```

$$1 + e^{2\pi i} + (e^{2\pi i})^2 + \cos(e^{2\pi i})$$

To evaluate the strings `S` and `symstr` as symbolic expressions, use `str2sym`.

```
S = str2sym(S)
```

```
S = 1
```

```
expr = str2sym(symstr)
```

```
expr = S + cos(S) + S^2 + 1
```

Substitute the variable `S` with its value by using `subs`. Evaluate the result in double precision using `double`.

```
double(subs(expr))
```

```
ans =
3.5403
```

## Display and Solve Quadratic Equation

Define a string that represents a quadratic formula with the coefficients `a`, `b`, and `c`.

```
syms a b c k
symstr = "a*x^2 + b*x + c";
```

Display the quadratic formula, replacing `a` with `k`.

```
displayFormula(symstr,a,k)
```

$$kx^2 + bx + c$$

Display the quadratic formula again, replacing `a`, `b`, and `c` with 2, 3, and -1, respectively.

```
displayFormula(symstr,[a b c],[2 3 -1])
```

$$2x^2 + 3x - 1$$

To solve the quadratic equation, convert the string into a symbolic expression using `str2sym`. Use `solve` to find the zeros of the quadratic equation.

```
f = str2sym(symstr);
sol = solve(f)
```

```
sol =
```

$$\begin{pmatrix} -\frac{b + \sqrt{b^2 - 4ac}}{2a} \\ -\frac{b - \sqrt{b^2 - 4ac}}{2a} \end{pmatrix}$$

Use `subs` to replace `a`, `b`, and `c` in the solution with 2, 3, and -1, respectively.

```
solValues = subs(sol,[a b c],[2 3 -1])
```

```
solValues =  

$$\begin{pmatrix} -\frac{\sqrt{17}}{4} - \frac{3}{4} \\ \frac{\sqrt{17}}{4} - \frac{3}{4} \end{pmatrix}$$

```

## Input Arguments

### **symstr** — String representing symbolic formula

character vector | string scalar | cell array of character vectors | string array

String representing a symbolic formula, specified as a character vector, string scalar, cell array of character vectors, or string array.

You can also combine a string that represents a symbolic formula with regular text (enclosed in single quotation marks) as a string array. For an example, see “Display Differential Equation” on page 6-302.

### **old** — Expression or variable to be replaced

character vector | string scalar | cell array of character vectors | string array | symbolic variable | symbolic function | symbolic expression | symbolic array

Expression or variable to be replaced, specified as a character vector, string scalar, cell array of character vectors, string array, symbolic variable, function, expression, or array.

### **new** — New value

number | character vector | string scalar | cell array of character vectors | string array | symbolic number | symbolic variable | symbolic expression | symbolic array

New value, specified as a number, character vector, string scalar, cell array of character vectors, string array, symbolic number, variable, expression, or array.

## Version History

Introduced in R2019b

## See Also

`str2sym` | `subs` | `syms` | `sym` | `solve`



# divergence

Divergence of symbolic vector field

## Syntax

```
d = divergence(V,X)
d = divergence(V)
```

## Description

`d = divergence(V,X)` returns the divergence on page 6-309 of symbolic vector field  $V$  with respect to vector  $X$  in Cartesian coordinates. Vectors  $V$  and  $X$  must have the same length.

`d = divergence(V)` returns the divergence of the vector field  $V$  with respect to a default vector constructed from the symbolic variables in  $V$ .

## Examples

### Find Divergence of Vector Field

Find the divergence of the vector field  $V(x, y, z) = (x, 2y^2, 3z^3)$  with respect to vector  $X = (x, y, z)$ .

```
syms x y z
V = [x 2*y^2 3*z^3];
X = [x y z];
div = divergence(V,X)
```

```
div = 9z^2 + 4y + 1
```

Show that the divergence of the curl of the vector field is 0.

```
divCurl = divergence(curl(V,X),X)
```

```
divCurl = 0
```

Find the divergence of the gradient of the scalar field  $f(x, y, z) = x^2 + y^2 + z^2$ . The result is the Laplacian of the scalar field.

```
syms x y z
f = x^2 + y^2 + z^2;
divGrad = divergence(gradient(f,X),X)
```

```
divGrad = 6
```

### Find Electric Charge Density from Electric Field

Gauss's Law in differential form states that the divergence of an electric field is proportional to the electric charge density.

$$\vec{\nabla} \cdot \vec{E}(\vec{r}) = \frac{\rho(\vec{r})}{\epsilon_0}$$

Find the electric charge density for the electric field  $\vec{E} = x^2\hat{i} + y^2\hat{j}$ .

```
syms x y ep0
E = [x^2 y^2];
rho = divergence(E,[x y])*ep0

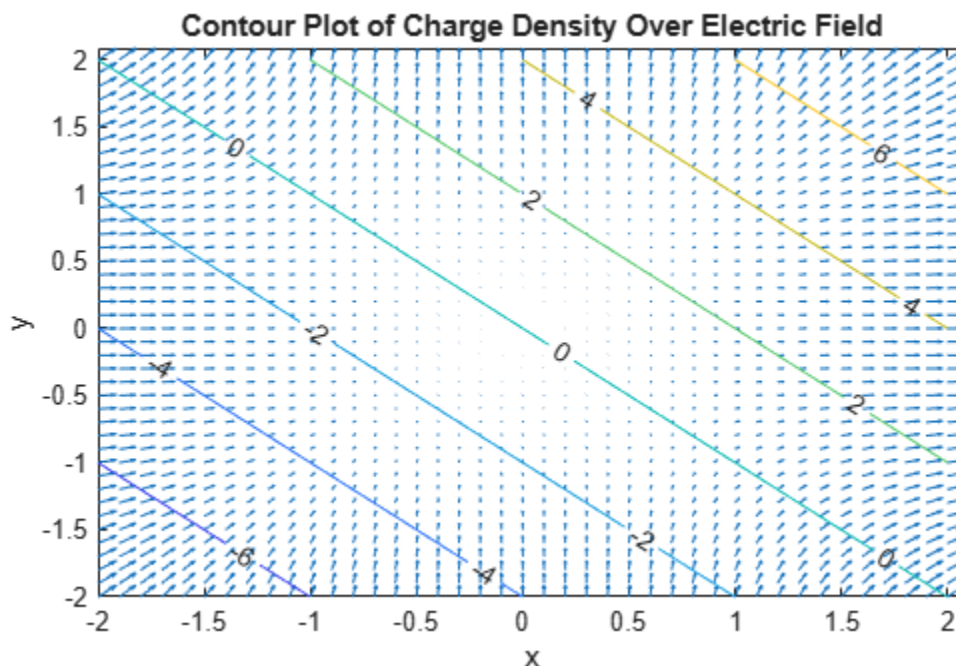
rho = ep0 (2 x + 2 y)
```

Visualize the electric field and electric charge density for  $-2 < x < 2$  and  $-2 < y < 2$  with  $\epsilon_0 = 1$ . Create a grid of values of  $x$  and  $y$  using `meshgrid`. Find the values of the electric field and charge density by substituting grid values using `subs`. Simultaneously substitute the grid values `xPlot` and `yPlot` into the charge density `rho` by using cell arrays as inputs to `subs`.

```
rho = subs(rho,ep0,1);
v = -2:0.1:2;
[xPlot,yPlot] = meshgrid(v);
Ex = subs(E(1),x,xPlot);
Ey = subs(E(2),y,yPlot);
rhoPlot = double(subs(rho,{x,y},{xPlot,yPlot}));
```

Plot the electric field using `quiver`. Overlay the charge density using `contour`. The contour lines indicate the values of the charge density.

```
quiver(xPlot,yPlot,Ex,Ey)
hold on
contour(xPlot,yPlot,rhoPlot,"ShowText","on")
title("Contour Plot of Charge Density Over Electric Field")
xlabel("x")
ylabel("y")
```



## Derive Electromagnetic Wave Equation

Derive the electromagnetic wave equation in free space without charge and without current sources from Maxwell's equations.

First, create symbolic scalar variables to represent the vacuum permeability and permittivity. Create a symbolic matrix variable to represent the Cartesian coordinates. Create two symbolic matrix functions to represent the electric and magnetic fields as functions of space and time.

```
syms mu_0 epsilon_0
syms X [3 1] matrix
syms E(X,t) B(X,t) [3 1] matrix keepargs
```

Next, create four equations to represent Maxwell's equations.

```
Maxwell1 = divergence(E,X) == 0
```

```
Maxwell1(X, t) =  $\nabla_X \cdot E(X,t) = 0_{1,1}$ 
```

```
Maxwell2 = curl(E,X) == -diff(B,t)
```

```
Maxwell2(X, t) =  $\nabla_X \times E(X,t) = -\frac{\partial}{\partial t} B(X,t)$ 
```

```
Maxwell3 = divergence(B,X) == 0
```

```
Maxwell3(X, t) =  $\nabla_X \cdot B(X,t) = 0_{1,1}$ 
```

```
Maxwell4 = curl(B,X) == mu_0*epsilon_0*diff(E,t)
```

```
Maxwell4(X, t) =  $\nabla_X \times B(X,t) = (\epsilon_0 \mu_0) \frac{\partial}{\partial t} E(X,t)$ 
```

Then, find the wave equation for the electric field. Compute the curl of the second Maxwell equation.

```
wave_E = curl(Maxwell2,X)
```

```
wave_E(X, t) =  $\nabla_X (\nabla_X \cdot E(X,t)) - \Delta_X E(X,t) = -\nabla_X \times \left( \frac{\partial}{\partial t} B(X,t) \right)$ 
```

Substitute the first Maxwell equation in the electric field wave equation. Use lhs and rhs to obtain the left and right sides of the first Maxwell equation.

```
wave_E = subs(wave_E, lhs(Maxwell1), rhs(Maxwell1))
```

```
wave_E(X, t) =  $-\Delta_X E(X,t) = -\nabla_X \times \left( \frac{\partial}{\partial t} B(X,t) \right)$ 
```

Compute the time derivative of the fourth Maxwell equation.

```
dMaxwell4 = diff(Maxwell4,t)
```

```
dMaxwell4(X, t) =
```

$$\nabla_X \times \left( \frac{\partial}{\partial t} B(X, t) \right) = (\epsilon_0 \mu_0) \frac{\partial}{\partial t} \frac{\partial}{\partial t} E(X, t)$$

Substitute the term that involves the magnetic field  $\nabla_X \times \frac{\partial}{\partial t} \mathbf{B}(\mathbf{X}, t)$  in `wave_E` with the right side of `dMaxwell4`. Use `lhs` and `rhs` to obtain these terms from `dMaxwell4`.

```
wave_E = subs(wave_E, lhs(dMaxwell4), rhs(dMaxwell4))
```

$$\begin{aligned} \text{wave\_E}(X, t) = \\ -\Delta_X E(X, t) = -(\epsilon_0 \mu_0) \frac{\partial}{\partial t} \frac{\partial}{\partial t} E(X, t) \end{aligned}$$

Using similar steps, you can also find the wave equation for the magnetic field.

```
wave_B = curl(Maxwell4, X)
```

$$\begin{aligned} \text{wave\_B}(X, t) = \\ \nabla_X (\nabla_X \cdot B(X, t)) - \Delta_X B(X, t) = (\epsilon_0 \mu_0) \left( \nabla_X \times \left( \frac{\partial}{\partial t} E(X, t) \right) \right) \end{aligned}$$

```
wave_B = subs(wave_B, lhs(Maxwell3), rhs(Maxwell3))
```

$$\begin{aligned} \text{wave\_B}(X, t) = \\ -\Delta_X B(X, t) = (\epsilon_0 \mu_0) \left( \nabla_X \times \left( \frac{\partial}{\partial t} E(X, t) \right) \right) \end{aligned}$$

```
dMaxwell2 = diff(Maxwell2, t)
```

$$\begin{aligned} \text{dMaxwell2}(X, t) = \\ \nabla_X \times \left( \frac{\partial}{\partial t} E(X, t) \right) = -\frac{\partial}{\partial t} \frac{\partial}{\partial t} B(X, t) \end{aligned}$$

```
wave_B = subs(wave_B, lhs(dMaxwell2), rhs(dMaxwell2))
```

$$\begin{aligned} \text{wave\_B}(X, t) = \\ -\Delta_X B(X, t) = -(\epsilon_0 \mu_0) \frac{\partial}{\partial t} \frac{\partial}{\partial t} B(X, t) \end{aligned}$$

## Input Arguments

### V — Symbolic vector field

vector of symbolic scalar variables | symbolic function | symbolic matrix variable | symbolic matrix function

Symbolic vector field, specified as a vector of symbolic scalar variables, symbolic function, symbolic matrix variable, or symbolic matrix function. `V` must have the same length as `X`.

- If `V` is a function of symbolic scalar variables, where `V` is of type `sym` or `symfun`, then the vector `X` must be of type `sym` or `symfun`.
- If `V` is a function of symbolic matrix variables, where `V` is of type `symmatrix` or `symfunmatrix`, then the vector `X` must be of type `symmatrix` or `symfunmatrix`.

Data Types: `sym` | `symfun` | `symmatrix` | `symfunmatrix`

### X — Vector with respect to which you find the divergence

vector of symbolic scalar variables | symbolic function | symbolic matrix variable | symbolic matrix function

Vector with respect to which you find the divergence, specified as a vector of symbolic scalar variables, symbolic function, symbolic matrix variable, or symbolic matrix function.  $X$  must have the same length as  $V$ .

- If you do not specify  $X$  and  $V$  is a function of symbolic scalar variables, then, by default, `divergence` constructs vector  $X$  from the symbolic scalar variables in  $V$  with the order of variables as defined by `symvar(V)`.
- If  $X$  is a symbolic matrix variable of type `symmatrix`, then  $X$  must have a size of 1-by- $N$  or  $N$ -by-1.
- If  $V$  and  $X$  are scalars, then `divergence(V,X) = diff(V,X)`.

Data Types: `sym` | `symfun` | `symmatrix` | `symfunmatrix`

## Limitations

- The `divergence` function does not support tensor derivatives. If the input  $V$  is a tensor field or a matrix rather than a vector, then the `divergence` function returns an error.
- Symbolic Math Toolbox currently does not support the `dot` or `cross` functions for symbolic matrix variables and functions of type `symmatrix` and `symfunmatrix`. If vector calculus identities involve dot or cross products, then the toolbox displays those identities in terms of other supported functions instead. To see a list of all the functions that support symbolic matrix variables and functions, use the commands `methods symmatrix` and `methods symfunmatrix`.

## More About

### Divergence of Symbolic Vector Field

The divergence of the symbolic vector field  $V = (V_1, \dots, V_n)$  with respect to the vector  $X = (X_1, \dots, X_n)$  in Cartesian coordinates is the sum of partial derivatives of  $V$  with respect to  $X_1, \dots, X_n$ .

$$\text{div}(\vec{V}) = \nabla_X \cdot \vec{V} = \sum_{i=1}^n \frac{\partial V_i}{\partial X_i}$$

## Version History

Introduced in R2012a

### R2023a: Compute divergence of symbolic matrix variables and functions

The `divergence` function accepts symbolic matrix variables and functions of type `symmatrix` and `symfunmatrix` as input arguments. For example, see “Derive Electromagnetic Wave Equation” on page 6-307.

## See Also

`curl` | `diff` | `gradient` | `jacobian` | `hessian` | `laplacian` | `potential` | `vectorPotential`

## divisors

Divisors of integer or expression

### Syntax

```
divisors(n)
divisors(expr, vars)
```

### Description

`divisors(n)` finds all nonnegative divisors of an integer `n`.

`divisors(expr, vars)` finds the divisors of a polynomial expression `expr`. Here, `vars` are polynomial variables.

### Examples

#### Divisors of Integers

Find all nonnegative divisors of these integers.

Find the divisors of integers. You can use double precision numbers or numbers converted to symbolic objects. If you call `divisors` for a double-precision number, then it returns a vector of double-precision numbers.

```
divisors(42)

ans =
     1     2     3     6     7    14    21    42
```

Find the divisors of negative integers. `divisors` returns nonnegative divisors for negative integers.

```
divisors(-42)

ans =
     1     2     3     6     7    14    21    42
```

If you call `divisors` for a symbolic number, it returns a symbolic vector.

```
divisors(sym(42))

ans =
[ 1, 2, 3, 6, 7, 14, 21, 42]
```

The only divisor of 0 is 0.

```
divisors(0)

ans =
     0
```

## Divisors of Univariate Polynomials

Find the divisors of univariate polynomial expressions.

Find the divisors of this univariate polynomial. You can specify the polynomial as a symbolic expression.

```
syms x
divisors(x^4 - 1, x)

ans =
[ 1, x - 1, x + 1, (x - 1)*(x + 1), x^2 + 1, (x^2 + 1)*(x - 1), ...
(x^2 + 1)*(x + 1), (x^2 + 1)*(x - 1)*(x + 1)]
```

You also can use a symbolic function to specify the polynomial.

```
syms f(t)
f(t) = t^5;
divisors(f,t)

ans(t) =
[ 1, t, t^2, t^3, t^4, t^5]
```

When finding the divisors of a polynomial, `divisors` does not return the divisors of the constant factor.

```
f(t) = 9*t^5;
divisors(f,t)

ans(t) =
[ 1, t, t^2, t^3, t^4, t^5]
```

## Divisors of Multivariate Polynomials

Find the divisors of multivariate polynomial expressions.

Find the divisors of the multivariate polynomial expression. Suppose that  $u$  and  $v$  are variables, and  $a$  is a symbolic parameter. Specify the variables as a symbolic vector.

```
syms a u v
divisors(a*u^2*v^3, [u,v])

ans =
[ 1, u, u^2, v, u*v, u^2*v, v^2, u*v^2, u^2*v^2, v^3, u*v^3, u^2*v^3]
```

Now, suppose that this expression contains only one variable (for example,  $v$ ), while  $a$  and  $u$  are symbolic parameters. Here, `divisors` treats the expression  $a*u^2$  as a constant and ignores it, returning only the divisors of  $v^3$ .

```
divisors(a*u^2*v^3, v)

ans =
[ 1, v, v^2, v^3]
```

## Input Arguments

**n** — Number for which to find divisors

number | symbolic number

Number for which to find the divisors, specified as a number or symbolic number.

**expr — Polynomial expression for which to find divisors**

symbolic expression | symbolic function

Polynomial expression for which to find divisors, specified as a symbolic expression or symbolic function.

**vars — Polynomial variables**

symbolic variable | vector of symbolic variables

Polynomial variables, specified as a symbolic variable or a vector of symbolic variables.

## Tips

- `divisors(0)` returns 0.
- `divisors(expr,vars)` does not return the divisors of the constant factor when finding the divisors of a polynomial.
- If you do not specify polynomial variables, `divisors` returns as many divisors as it can find, including the divisors of constant symbolic expressions. For example, `divisors(sym(pi)^2*x^2)` returns [ 1, pi, pi^2, x, pi\*x, pi^2\*x, x^2, pi\*x^2, pi^2\*x^2] while `divisors(sym(pi)^2*x^2, x)` returns [ 1, x, x^2].
- For rational numbers, `divisors` returns all divisors of the numerator divided by all divisors of the denominator. For example, `divisors(sym(9/8))` returns [ 1, 3, 9, 1/2, 3/2, 9/2, 1/4, 3/4, 9/4, 1/8, 3/8, 9/8].

## Version History

Introduced in R2014b

## See Also

`coeffs` | `factor` | `numden`



# double

Convert symbolic values to MATLAB double precision

## Syntax

```
d = double(s)
```

## Description

`d = double(s)` converts the symbolic values `s` to double precision. Converting symbolic values to double precision is useful when a MATLAB function does not accept symbolic values. For differences between symbolic and double-precision numbers, see “Choose Numeric or Symbolic Arithmetic” on page 2-32.

## Examples

### Convert Symbolic Number to Double Precision

Convert symbolic numbers to double precision by using `double`. Symbolic numbers are exact, while double-precision numbers have round-off errors.

Convert  $\pi$  and  $\frac{1}{3}$  from symbolic form to double precision.

```
symN = sym([pi 1/3])
```

```
symN =  
      ( $\pi \quad \frac{1}{3}$ )
```

```
doubleN = double(symN)
```

```
doubleN = 1×2
```

```
    3.1416    0.3333
```

For information on round-off errors, see “Recognize and Avoid Round-Off Errors” on page 3-382.

### Convert Variable Precision to Double Precision

Variable-precision numbers created by `vpa` are symbolic values. When a MATLAB function does not accept symbolic values, convert variable precision to double precision by using `double`.

Convert  $\pi$  and  $\frac{1}{3}$  from variable-precision form to double precision.

```
vpaN = vpa([pi 1/3])
```

```
vpaN = (3.1415926535897932384626433832795 0.33333333333333333333333333333333)
doubleN = double(vpaN)
doubleN = 1×2

    3.1416    0.3333
```

### Convert Symbolic Matrix to Double-Precision Matrix

Convert the symbolic numbers in matrix `symM` to double-precision numbers by using `double`.

```
a = sym(sqrt(2));
b = sym(2/3);
symM = [a b; a*b b/a]
```

```
symM =

$$\begin{pmatrix} \sqrt{2} & \frac{2}{3} \\ \frac{2\sqrt{2}}{3} & \frac{\sqrt{2}}{3} \end{pmatrix}$$

```

```
doubleM = double(symM)
```

```
doubleM = 2×2

    1.4142    0.6667
    0.9428    0.4714
```

### High-Precision Conversion

When converting symbolic expressions that suffer from internal cancellation or round-off errors, increase the working precision by using `digits` before converting the number.

Convert a numerically unstable expression `Y` with `double`. Then, increase precision to 100 digits by using `digits` and convert `Y` again. This high-precision conversion is accurate, while the low-precision conversion is not.

```
Y = ((exp(sym(200)) + 1)/(exp(sym(200)) - 1)) - 1;
lowPrecisionY = double(Y)
```

```
lowPrecisionY =
0
```

```
digitsOld = digits(100);
highPrecisionY = double(Y)
```

```
highPrecisionY =
2.7678e-87
```

Restore the old precision used by `digits` for further calculations.

```
digits(digits0ld)
```

### Convert Symbolic Results to Double Precision

Solve the trigonometric equation  $\sin(2x) + \cos(x) = 0$  by using `solve`. Set the `ReturnConditions` option to `true` to return the complete solution, parameters used in the solution, and conditions on those parameters.

```
syms x
eqn = sin(2*x) + cos(x) == 0;
[solx,params,conds] = solve(eqn,x,ReturnConditions=true)
```

```
solx =
```

$$\begin{pmatrix} \frac{\pi}{2} + \pi k \\ 2\pi k - \frac{\pi}{6} \\ \frac{7\pi}{6} + 2\pi k \end{pmatrix}$$

```
params = k
```

```
conds =
```

$$\begin{pmatrix} k \in \mathbb{Z} \\ k \in \mathbb{Z} \\ k \in \mathbb{Z} \end{pmatrix}$$

The solver does not create the variable  $k$  for the parameters in the MATLAB® workspace. Create this variable. Find the solutions for  $k = 2$  by using `subs`.

```
syms k
sols_k2 = subs(solx,k,2)
```

```
sols_k2 =
```

$$\begin{pmatrix} \frac{5\pi}{2} \\ \frac{23\pi}{6} \\ \frac{31\pi}{6} \end{pmatrix}$$

The solutions are exact symbolic numbers. Convert these numbers to double-precision numbers.

```
doublesols_k2 = double(sols_k2)
```

```
doublesols_k2 = 3×1
```

```
7.8540
12.0428
16.2316
```

### Convert Symbolic Matrix Variable of Numbers to Double Precision

Create a symbolic expression  $S$  that represents  $\mathbf{A}^2 - 2\mathbf{A} + \mathbf{I}_2$ , where  $\mathbf{A}$  is a 2-by-2 symbolic matrix variable.

```
syms A 2 matrix
S = A*A - 2*A + eye(2)
```

$$S = \mathbf{I}_2 - 2\mathbf{A} + \mathbf{A}^2$$

Substitute  $\mathbf{A}$  with symbolic numbers  $\begin{bmatrix} \cos(\frac{\pi}{5}) & \sin(\frac{\pi}{4}) \\ -1 & 0 \end{bmatrix}$ .

```
Aval = [cos(sym(pi)/5) sin(pi/4); -1 0];
symS = subs(S,A,Aval)
```

$$\text{symS} = -2\Sigma_1 + \Sigma_1^2 + \mathbf{I}_2$$

where

$$\Sigma_1 = \begin{pmatrix} \frac{\sqrt{5}}{4} + \frac{1}{4} & \frac{\sqrt{2}}{2} \\ -1 & 0 \end{pmatrix}$$

Convert the result to a double-precision matrix.

```
doubleS = double(symS)
```

```
doubleS = 2x2
```

$$\begin{pmatrix} -0.6706 & -0.8422 \\ 1.1910 & 0.2929 \end{pmatrix}$$

## Input Arguments

### s — Symbolic input

symbolic number | array of symbolic numbers | symbolic matrix variable of numbers

Symbolic input, specified as a symbolic number, array of symbolic numbers, or symbolic matrix variable of numbers. If the input contains only numeric values (does not contain symbolic numbers), then the MATLAB `double` function is called.

Data Types: `sym` | `symmatrix`

## Version History

Introduced before R2006a

**R2022b: Convert symbolic matrix variables of numbers to double precision**

You can convert a symbolic matrix variable of numbers of type `symmatrix` to a double-precision matrix of type `double`. For an example, see “Convert Symbolic Matrix Variable of Numbers to Double Precision” on page 6-315.

## See Also

`sym` | `vpa` | `subs`

## Topics

“Choose Numeric or Symbolic Arithmetic” on page 2-32

“Increase Precision of Numeric Calculations” on page 2-36

“Recognize and Avoid Round-Off Errors” on page 3-382

“Increase Speed by Reducing Precision” on page 3-386

“Change Output Format of Symbolic and Variable-Precision Arithmetic” on page 2-8

## dsolve

Solve system of differential equations

### Syntax

```
S = dsolve(eqn)
S = dsolve(eqn,cond)
S = dsolve( ___,Name=Value)

[y1,...,yN] = dsolve( ___)
```

### Description

`S = dsolve(eqn)` solves the differential equation `eqn`, where `eqn` is a symbolic equation. Use `diff` and `==` to represent differential equations. For example, `diff(y,x) == y` represents the equation  $dy/dx = y$ . Solve a system of differential equations by specifying `eqn` as a vector of those equations.

`S = dsolve(eqn,cond)` solves `eqn` with the initial or boundary condition `cond`.

`S = dsolve( ___,Name=Value)` uses additional options specified by one or more `Name=Value` arguments.

`[y1,...,yN] = dsolve( ___)` assigns the solutions to the variables `y1,...,yN`.

### Examples

#### Solve Differential Equation

Solve the first-order differential equation  $\frac{dy}{dt} = ay$ .

Specify the first-order derivative by using `diff` and the equation by using `==`. Then, solve the equation by using `dsolve`.

```
syms y(t) a
eqn = diff(y,t) == a*y;
S = dsolve(eqn)
```

$S = C_1 e^{at}$

The solution includes a constant. To eliminate constants, see “Solve Differential Equations with Conditions” on page 6-319. For a full workflow, see “Solve Partial Differential Equation of Tsunami Model” on page 3-59.

#### Solve Second-Order Differential Equation

Solve the second-order differential equation  $\frac{d^2y}{dt^2} = ay$ .

Specify the second-order derivative of  $y$  by using `diff(y,t,2)` and the equation by using `==`. Then, solve the equation by using `dsolve`.

```
syms y(t) a
eqn = diff(y,t,2) == a*y;
ySol(t) = dsolve(eqn)
```

$$ySol(t) = C_1 e^{-\sqrt{a}t} + C_2 e^{\sqrt{a}t}$$

### Solve Differential Equations with Conditions

Solve the first-order differential equation  $\frac{dy}{dt} = ay$  with the initial condition  $y(0) = 5$ .

Specify the initial condition as the second input to `dsolve` by using the `==` operator. Specifying condition eliminates arbitrary constants, such as  $C_1$ ,  $C_2$ , . . . , from the solution.

```
syms y(t) a
eqn = diff(y,t) == a*y;
cond = y(0) == 5;
ySol(t) = dsolve(eqn,cond)
```

$$ySol(t) = 5 e^{at}$$

Next, solve the second-order differential equation  $\frac{d^2y}{dt^2} = a^2y$  with the initial conditions  $y(0) = b$  and  $y'(0) = 1$ .

Specify the second initial condition by assigning `diff(y,t)` to `Dy` and then using `Dy(0) == 1`.

```
syms y(t) a b
eqn = diff(y,t,2) == a^2*y;
Dy = diff(y,t);
cond = [y(0)==b, Dy(0)==1];
ySol(t) = dsolve(eqn,cond)
```

$$ySol(t) = \frac{e^{at} (ab + 1)}{2a} + \frac{e^{-at} (ab - 1)}{2a}$$

This second-order differential equation has two specified conditions, so constants are eliminated from the solution. In general, to eliminate constants from the solution, the number of conditions must equal the order of the equation.

### Solve System of Differential Equations

Solve the system of differential equations

$$\begin{aligned}\frac{dy}{dt} &= z \\ \frac{dz}{dt} &= -y.\end{aligned}$$

Specify the system of equations as a vector. `dsolve` returns a structure containing the solutions.

```
syms y(t) z(t)
eqns = [diff(y,t) == z, diff(z,t) == -y];
S = dsolve(eqns)

S = struct with fields:
    z: C2*cos(t) - C1*sin(t)
    y: C1*cos(t) + C2*sin(t)
```

Access the solutions by addressing the elements of the structure.

```
ySol(t) = S.y
ySol(t) = C1*cos(t) + C2*sin(t)

zSol(t) = S.z
zSol(t) = C2*cos(t) - C1*sin(t)
```

### Assign Outputs to Functions or Variables

When solving for multiple functions, `dsolve` returns a structure by default. Alternatively, you can assign solutions to functions or variables directly by explicitly specifying the outputs as a vector. `dsolve` sorts the outputs in alphabetical order using `symvar`.

Solve a system of differential equations and assign the outputs to functions.

```
syms y(t) z(t)
eqns = [diff(y,t)==z, diff(z,t)==-y];
[ySol(t),zSol(t)] = dsolve(eqns)

ySol(t) = C1*cos(t) + C2*sin(t)
zSol(t) = C2*cos(t) - C1*sin(t)
```

### Find Explicit and Implicit Solutions of Differential Equation

Solve the differential equation  $\frac{\partial}{\partial t}y(t) = e^{-y(t)} + y(t)$ . `dsolve` returns an explicit solution in terms of a Lambert W function that has a constant value.

```
syms y(t)
eqn = diff(y) == y+exp(-y)

eqn(t) =
     $\frac{\partial}{\partial t} y(t) = e^{-y(t)} + y(t)$ 

sol = dsolve(eqn)

sol = W0(-1)
```



To return implicit solutions of the differential equation, set the `Implicit` option to `true`. An implicit solution has the form  $F(y(t)) = g(t)$ .

```
sol = dsolve(eqn,Implicit=true)
```

```
sol =
```

$$\left( \left( \int \frac{e^y}{y e^y + 1} dy \right) \Big|_{y=y(t)} = C_1 + t \right)$$

$$e^{-y(t)} (e^{y(t)} y(t) + 1) = 0$$

### Find Implicit Solution When No Explicit Solution Is Found

If `dsolve` cannot find an explicit solution of a differential equation analytically, then it returns an empty symbolic array. You can solve the differential equation by using MATLAB® numerical solver, such as `ode45`. For more information, see “Solve a Second-Order Differential Equation Numerically” on page 3-57.

```
syms y(x)
eqn = diff(y) == (x-exp(-x))/(y(x)+exp(y(x)));
S = dsolve(eqn)
```

Warning: Unable to find symbolic solution.

```
S =
```

```
[ empty sym ]
```

Alternatively, you can try finding an implicit solution of the differential equation by specifying the `Implicit` option to `true`. An implicit solution has the form  $F(y(x)) = g(x)$ .

```
S = dsolve(eqn,Implicit=true)
```

```
S =
```

$$e^{y(x)} + \frac{y(x)^2}{2} = C_1 + e^{-x} + \frac{x^2}{2}$$

### Include Special Cases by Turning Off Internal Simplifications

Solve the differential equation  $\frac{dy}{dt} = \frac{a}{\sqrt{y}} + y$  with condition  $y(a) = 1$ . By default, `dsolve` applies simplifications that are not generally correct, but produce simpler solutions. For more details, see “Algorithms” on page 6-326.

```
syms a y(t)
eqn = diff(y) == a/sqrt(y) + y;
cond = y(a) == 1;
ySimplified = dsolve(eqn,cond)
```

```
ySimplified =
```

$$\left( e^{\frac{3t}{2} - \frac{3a}{2} + \log(a+1)} - a \right)^{2/3}$$

To return the solutions that include all possible values of the parameter  $a$ , turn off simplifications by setting `IgnoreAnalyticConstraints` to `false`.

`yNotSimplified = dsolve(eqn, cond, IgnoreAnalyticConstraints=false)`

`yNotSimplified =`

$$\left\{ \begin{array}{ll} \{\sigma_1\} & \text{if } -\frac{\pi}{2} < \sigma_2 \\ \left\{ \left\{ \sigma_1, -\left( -a + e^{\frac{3t}{2} - \frac{3a}{2} + \log\left(a + \left(-\frac{1}{2} + \sigma_3\right)^{3/2}\right) + 2\pi C_2 i} \right)^{2/3} \left(\frac{1}{2} + \sigma_3\right) \right\} \right\} & \text{if } \sigma_2 \leq -\frac{\pi}{2} \text{ if } C_2 \in \mathbb{Z} \\ \emptyset & \text{if } C_2 \notin \mathbb{Z} \end{array} \right.$$

where

$$\sigma_1 = \left( -a + e^{\frac{3t}{2} - \frac{3a}{2} + \log(a+1) + 2\pi C_2 i} \right)^{2/3}$$

$$\sigma_2 = \text{angle} \left( e^{\frac{3C_1}{2} + \frac{3t}{2}} - a \right)$$

$$\sigma_3 = \frac{\sqrt{3} i}{2}$$

### Find Series Solution of Differential Equation

Solve the second-order differential equation  $(x^2 - 1)^2 \frac{\partial^2}{\partial x^2} y(x) + (x + 1) \frac{\partial}{\partial x} y(x) - y(x) = 0$ . `dsolve` returns a solution that contains a term with unevaluated integral.

`syms y(x)`  
`eqn = (x^2-1)^2*diff(y,2) + (x+1)*diff(y) - y == 0;`  
`S = dsolve(eqn)`

`S =`

$$C_2 (x + 1) + C_1 (x + 1) \int \frac{e^{\frac{1}{2}(x-1)} (1-x)^{1/4}}{(x+1)^{9/4}} dx$$

To return series solutions of the differential equation around  $x = -1$ , set the `ExpansionPoint` to `-1`. `dsolve` returns two linearly independent solutions in terms of a Puiseux series expansion.

`S = dsolve(eqn, ExpansionPoint=-1)`

`S =`

$$\left( \frac{1}{(x+1)^{1/4}} - \frac{5(x+1)^{3/4}}{4} + \frac{5(x+1)^{7/4}}{48} + \frac{5(x+1)^{11/4}}{336} + \frac{115(x+1)^{15/4}}{33792} + \frac{169(x+1)^{19/4}}{184320} \right) x + 1$$

Find other series solutions around the expansion point  $\infty$  by setting `ExpansionPoint` to `Inf`.

```
S = dsolve(eqn,ExpansionPoint=Inf)
```

$$S = \begin{pmatrix} \frac{1}{6x^2} + \frac{1}{8x^4} + \frac{1}{90x^5} + 1 \\ x - \frac{1}{6x^2} - \frac{1}{8x^4} \end{pmatrix}$$

The default truncation order of the series expansion is 6. To obtain more terms in the Puiseux series solutions, set `Order` to 8.

```
S = dsolve(eqn,ExpansionPoint=Inf,Order=8)
```

$$S = \begin{pmatrix} \frac{1}{6x^2} + \frac{1}{8x^4} + \frac{1}{90x^5} + \frac{37}{336x^6} + \frac{37}{1680x^7} + 1 \\ x - \frac{1}{6x^2} - \frac{1}{8x^4} - \frac{1}{90x^5} - \frac{37}{336x^6} \end{pmatrix}$$

### Specify Initial Condition for Function with Different One-Sided Limits

Solve the differential equation  $\frac{dy}{dx} = \frac{1}{x^2}e^{-\frac{1}{x}}$  without specifying the initial condition.

```
syms y(x)
eqn = diff(y) == exp(-1/x)/x^2;
ySol(x) = dsolve(eqn)
```

$$ySol(x) = C_1 + e^{-\frac{1}{x}}$$

To eliminate constants from the solution, specify the initial condition  $y(0) = 1$ .

```
cond = y(0) == 1;
S = dsolve(eqn,cond)
```

$$S = e^{-\frac{1}{x}} + 1$$

The function  $e^{-\frac{1}{x}}$  in the solution  $ySol(x)$  has different one-sided limits at  $x = 0$ . The function has a right-side limit,  $\lim_{x \rightarrow 0^+} e^{-\frac{1}{x}} = 0$ , but it has undefined left-side limit,  $\lim_{x \rightarrow 0^-} e^{-\frac{1}{x}} = \infty$ .

When you specify the condition  $y(x_0)$  for a function with different one-sided limits at  $x_0$ , `dsolve` treats the condition as a limit from the right,  $\lim_{x \rightarrow x_0^+}$ .

## Input Arguments

### eqn — Differential equation or system of equations

symbolic equation | vector of symbolic equations

Differential equation or system of equations, specified as a symbolic equation or a vector of symbolic equations. Specify a differential equation by using the `==` operator. If `eqn` is a symbolic expression (without the right side), the solver assumes that the right side is 0, and solves the equation `eqn == 0`.

In the equation, represent differentiation by using `diff`. For example, `diff(y,x)` differentiates the symbolic function  $y(x)$  with respect to  $x$ . Create the symbolic function  $y(x)$  by using `syms` and solve the equation  $d^2y(x)/dx^2 = x*y(x)$  using `dsolve`.

```
syms y(x)
S = dsolve(diff(y,x,2) == x*y)
```

Specify a system of differential equations by using a vector of equations, as in `syms y(t) z(t); S = dsolve([diff(y,t) == z, diff(z,t) == -y])`. Here,  $y$  and  $z$  must be symbolic functions that depend on symbolic variables, which are  $t$  in this case. The right side must be symbolic expressions that depend on  $t$ ,  $y$  and  $z$ . Note that Symbolic Math Toolbox currently does not support composite symbolic functions, that is, symbolic functions that depend on another symbolic functions.

### cond — Initial or boundary condition

symbolic equation | vector of symbolic equations

Initial or boundary condition, specified as a symbolic equation or vector of symbolic equations.

When a condition contains a derivative, represent the derivative with `diff`. Assign the `diff` call to a variable and use the variable to specify the condition. For example, see “Solve Differential Equations with Conditions” on page 6-319.

Specify multiple conditions by using a vector of equations. If the number of conditions is less than the number of dependent variables, the solutions contain the arbitrary constants  $C_1, C_2, \dots$ .

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `IgnoreAnalyticConstraints=false` does not apply internal simplifications.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

### ExpansionPoint — Expansion point of Puiseux series solution

0 (default) | number | symbolic number | symbolic variable | symbolic function | symbolic expression

Expansion point of a Puiseux series solution, specified as a number, or a symbolic number, variable, function, or expression. Specifying this option returns the solution of a differential equation in terms of a Puiseux series (a power series that allows negative and fractional exponents). The expansion point cannot depend on the series variable. For example, see “Find Series Solution of Differential Equation” on page 6-322.

Data Types: `single` | `double` | `sym`

Complex Number Support: Yes

### **IgnoreAnalyticConstraints — Option to use internal simplifications**

true (default) | false

Option to use internal simplifications, specified as true or false.

By default, the solver applies simplifications while solving the differential equation, which could lead to results not generally valid. In other words, this option applies mathematical identities that are convenient, but the results might not hold for all possible values of the variables. Therefore, by default, the solver does not guarantee the completeness of results. If `IgnoreAnalyticConstraints` is true, always verify results returned by the `dsolve` function. For more details, see “Algorithms” on page 6-326.

To solve ordinary differential equations without these simplifications, set `IgnoreAnalyticConstraints` to false. Results obtained with `IgnoreAnalyticConstraints` set to false are correct for all values of the arguments. For certain equations, `dsolve` might not return an explicit solution if you set `IgnoreAnalyticConstraints` to false.

### **Implicit — Option to return implicit solution**

false (default) | true

Option to return an implicit solution, specified as false or true. For a differential equation with variables  $x$  and  $y(x)$ , an implicit solution has the form  $F(y(x)) = g(x)$ .

By default, the solver tries to find an explicit solution  $y(x) = f(x)$  analytically when solving a differential equation. If `dsolve` cannot find an explicit solution, then you can try finding a solution in implicit form by specifying the `Implicit` option to true.

### **MaxDegree — Maximum degree of polynomial equation for which solver uses explicit formulas**

2 (default) | positive integer smaller than 5

Maximum degree of polynomial equations for which the solver uses explicit formulas, specified as a positive integer smaller than 5. `dsolve` does not use explicit formulas when solving polynomial equations of degrees larger than `MaxDegree`.

### **Order — Truncation order of Puiseux series solution**

0 (default) | positive integer | symbolic positive integer

Truncation order of a Puiseux series solution, specified as a positive integer or a symbolic positive integer. Specifying this option returns the solution of a differential equation in terms of a Puiseux series (a power series that allow negative and fractional exponents). The truncation order  $n$  is the exponent in the  $O$ -term:  $O(\text{var}^n)$  or  $O(\text{var}^{-n})$ .

## **Output Arguments**

### **S — Solutions of differential equation**

symbolic expression | vector of symbolic expressions

Solutions of differential equation, returned as a symbolic expression or a vector of symbolic expressions. The size of `S` is the number of solutions.

**y1, ..., yN — Variables storing solutions of differential equations**

vector of symbolic variables

Variables storing solutions of differential equations, returned as a vector of symbolic variables. The number of output variables must equal the number of dependent variables in a system of equations. `dsolve` sorts the dependent variables alphabetically, and then assigns the solutions for the variables to output variables or symbolic arrays.

**Tips**

- If `dsolve` cannot find an explicit or implicit solution, then it issues a warning and returns the empty `sym`. In this case, try to find a numeric solution using the MATLAB `ode23` or `ode45` function. Sometimes, the output is an equivalent lower-order differential equation or an integral.
- `dsolve` does not always return complete solutions even if `IgnoreAnalyticConstraints` is `false`.
- If `dsolve` returns a function that has different one-sided limits at  $x_0$  and you specify the condition  $y(x_0)$ , then `dsolve` treats the condition as a limit from the right,  $\lim_{x \rightarrow x_0^+}$ .

**Algorithms**

If you do not set `IgnoreAnalyticConstraints` to `false`, then `dsolve` applies some of these rules while solving the equation:

- $\log(a) + \log(b) = \log(a \cdot b)$  for all values of  $a$  and  $b$ . In particular, the following equality is applied for all values of  $a$ ,  $b$ , and  $c$ :

$$(a \cdot b)^c = a^c \cdot b^c.$$

- $\log(a^b) = b \cdot \log(a)$  for all values of  $a$  and  $b$ . In particular, the following equality is applied for all values of  $a$ ,  $b$ , and  $c$ :

$$(a^b)^c = a^{b \cdot c}.$$

- If  $f$  and  $g$  are standard mathematical functions and  $f(g(x)) = x$  for all small positive numbers,  $f(g(x)) = x$  is assumed to be valid for all complex  $x$ . In particular:

- $\log(e^x) = x$
- $\text{asin}(\sin(x)) = x$ ,  $\text{acos}(\cos(x)) = x$ ,  $\text{atan}(\tan(x)) = x$
- $\text{asinh}(\sinh(x)) = x$ ,  $\text{acosh}(\cosh(x)) = x$ ,  $\text{atanh}(\tanh(x)) = x$
- $W_k(x \cdot e^x) = x$  for all branch indices  $k$  of the Lambert W function.
- The solver can multiply both sides of an equation by any expression except 0.
- The solutions of polynomial equations must be complete.

**Version History**

Introduced before R2006a

**R2020a: Return solutions of differential equations in implicit form or truncated series expansion**

With `dsolve`:

- Use the `Implicit` name-value argument to return implicit solutions of differential equations.
- Use the `ExpansionPoint` and `Order` name-value arguments to specify the expansion point and truncation order of Puiseux series solutions of differential equations.

**R2019b: `dsolve` will no longer support character vector or string inputs**

*Warns starting in R2019b*

`dsolve` will not accept equations as strings or character vectors in a future release. Instead, use symbolic expressions or `sym` objects to define differential equations. For example, replace inputs such as `dsolve('Dy = exp(y)')` with `syms y(t); dsolve(diff(y,t) == exp(y))`.

**See Also**

`functionalDerivative` | `isolate` | `linsolve` | `ode23` | `ode45` | `odeToVectorField` | `solve` | `syms` | `vpasolve`

**Topics**

“Solve Differential Equation” on page 3-49

“Solve a System of Differential Equations” on page 3-53

**Teaching Resources**

Applied Ordinary Differential Equations (MathWorks Teaching Resources)

Beam Bending and Deflection (MathWorks Teaching Resources)

## ei

One-argument exponential integral function

### Syntax

`ei(x)`

### Description

`ei(x)` returns the one-argument exponential integral defined as

$$\text{ei}(x) = \int_{-\infty}^x \frac{e^t}{t} dt.$$

### Examples

#### Exponential Integral for Floating-Point and Symbolic Numbers

Compute exponential integrals for numeric inputs. Because these numbers are not symbolic objects, you get floating-point results.

```
s = [ei(-2), ei(-1/2), ei(1), ei(sqrt(2))]
```

```
s =
```

```
-0.0489    -0.5598    1.8951    3.0485
```

Compute exponential integrals for the same numbers converted to symbolic objects. For most symbolic (exact) numbers, `ei` returns unresolved symbolic calls.

```
s = [ei(sym(-2)), ei(sym(-1/2)), ei(sym(1)), ei(sqrt(sym(2)))]
```

```
s =
```

```
[ ei(-2), ei(-1/2), ei(1), ei(2^(1/2))]
```

Use `vpa` to approximate this result with 10-digit accuracy.

```
vpa(s, 10)
```

```
ans =
```

```
[ -0.04890051071, -0.5597735948, 1.895117816, 3.048462479]
```

#### Branch Cut at Negative Real Axis

The negative real axis is a branch cut. The exponential integral has a jump of height  $2\pi i$  when crossing this cut. Compute the exponential integrals at  $-1$ , above  $-1$ , and below  $-1$  to demonstrate this.

```
[ei(-1), ei(-1 + 10^(-10)*i), ei(-1 - 10^(-10)*i)]
```

```
ans =
```

```
-0.2194 + 0.0000i    -0.2194 + 3.1416i    -0.2194 - 3.1416i
```



## Derivatives of Exponential Integral

Compute the first, second, and third derivatives of a one-argument exponential integral.

```
syms x
diff(ei(x), x)
diff(ei(x), x, 2)
diff(ei(x), x, 3)

ans =
exp(x)/x

ans =
exp(x)/x - exp(x)/x^2

ans =
exp(x)/x - (2*exp(x))/x^2 + (2*exp(x))/x^3
```

## Limits of Exponential Integral

Compute the limits of a one-argument exponential integral.

```
syms x
limit(ei(2*x^2/(1+x)), x, -Inf)
limit(ei(2*x^2/(1+x)), x, 0)
limit(ei(2*x^2/(1+x)), x, Inf)

ans =
0

ans =
-Inf

ans =
Inf
```

## Input Arguments

### x — Input

floating-point number | symbolic number | symbolic variable | symbolic expression | symbolic function  
| symbolic vector | symbolic matrix

Input specified as a floating-point number or symbolic number, variable, expression, function, vector, or matrix.

## Tips

- The one-argument exponential integral is singular at  $x = 0$ . The toolbox uses this special value:  $ei(0) = -\text{Inf}$ .

## Algorithms

The relation between `ei` and `expint` is

$$ei(x) = -\text{expint}(1, -x) + (\ln(x) - \ln(1/x))/2 - \ln(-x)$$

Both functions `ei(x)` and `expint(1,x)` have a logarithmic singularity at the origin and a branch cut along the negative real axis. The `ei` function is not continuous when approached from above or below this branch cut.

## Version History

Introduced in R2013a

## References

- [1] Gautschi, W., and W. F. Gahill “Exponential Integral and Related Functions.” *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

`expint` | `int` | `vpa`

# eig

Eigenvalues and eigenvectors of symbolic matrix

## Syntax

```
lambda = eig(A)
[V,D] = eig(A)
[V,D,p] = eig(A)
```

## Description

`lambda = eig(A)` returns the eigenvalues of the square symbolic matrix `A` as a symbolic vector.

`[V,D] = eig(A)` returns the eigenvectors and eigenvalues of `A` as symbolic matrices `V` and `D`. The columns of `V` present eigenvectors of `A`. The main diagonal of `D` present eigenvalues of `A`.

- If `V` is the same size as `A`, then the matrix `A` has a full set of linearly independent eigenvectors that satisfy  $A*V = V*D$ .
- If `V` has fewer columns than `A`, then the matrix `A` is defective. In this case, at least one of the eigenvalues  $\lambda$  has an algebraic multiplicity  $m > 1$  with fewer than  $m$  linearly independent eigenvectors associated with  $\lambda$ .

`[V,D,p] = eig(A)` also returns a vector of indices `p`. The length of `p` is equal to the number of linearly independent eigenvectors, so  $A*V = V*D(p,p)$ .

---

**Note** The symbolic `eig` function computes eigenvalues and eigenvectors symbolically, with different syntaxes and functionalities compared to the MATLAB `eig` function. For example, the symbolic `eig` function does not support solving the generalized eigenvalue problem (with two input arguments). To solve this problem, convert the input matrices to a MATLAB numeric type and use the MATLAB `eig` function instead.

---

## Examples

### Compute Eigenvalues

Compute eigenvalues for the magic square of order 5.

```
A = sym(magic(5));
lambda = eig(A)

lambda =
```

$$\begin{pmatrix} 65 \\ \sqrt{\frac{625}{2} - \frac{5\sqrt{3145}}{2}} \\ \sqrt{\frac{5\sqrt{3145}}{2} + \frac{625}{2}} \\ -\sqrt{\frac{625}{2} - \frac{5\sqrt{3145}}{2}} \\ -\sqrt{\frac{5\sqrt{3145}}{2} + \frac{625}{2}} \end{pmatrix}$$

### Compute Numeric Eigenvalues to High Precision

Compute numeric eigenvalues for the magic square of order 5 using variable-precision arithmetic.

```
A = magic(5);
lambda = eig(vpa(A))

lambda =
```

$$\begin{pmatrix} 65.0 \\ 21.276765471473795530626426697974 \\ 13.126280930709218802525643085949 \\ -13.126280930709218802525643085949 \\ -21.276765471473795530626426697974 \end{pmatrix}$$

### Convert Symbolic Eigenvalues to Floating Point

Create a 5-by-5 symbolic matrix from the magic square of order 6. Compute the eigenvalues of the matrix using `eig`.

```
M = magic(6);
A = sym(M(1:5,1:5));
lambda = eig(A)
```

```
lambda =
```

$$\begin{pmatrix} \text{root}(\sigma_1, z, 1) \\ \text{root}(\sigma_1, z, 2) \\ \text{root}(\sigma_1, z, 3) \\ \text{root}(\sigma_1, z, 4) \\ \text{root}(\sigma_1, z, 5) \end{pmatrix}$$

where

$$\sigma_1 = z^5 - 100z^4 + 134z^3 + 66537z^2 - 450198z - 1294704$$

The `eig` function cannot find the exact eigenvalues in terms of symbolic numbers. Instead, it returns them in terms of the `root` function.

Use `vpa` to numerically approximate the eigenvalues.

```
lambdaVpa = vpa(lambda)
```

```
lambdaVpa =  


$$\begin{pmatrix} -2.181032364984695108354692701065 \\ 9.8395828502812312578803604206392 \\ -25.131641669799891607267584639192 \\ 26.341617610275869035465716505806 \\ 91.131473574227486422276200413812 \end{pmatrix}$$

```

### Compute Eigenvalues and Eigenvectors of Defective Matrix

Compute the exact eigenvalues and eigenvectors for one of the MATLAB® test matrices that is defective.

```
A = sym(gallery(5))
```

```
A =  


$$\begin{pmatrix} -9 & 11 & -21 & 63 & -252 \\ 70 & -69 & 141 & -421 & 1684 \\ -575 & 575 & -1149 & 3451 & -13801 \\ 3891 & -3891 & 7782 & -23345 & 93365 \\ 1024 & -1024 & 2048 & -6144 & 24572 \end{pmatrix}$$

```

```
[V,D] = eig(A)
```

```
V =  


$$\begin{pmatrix} 0 \\ \frac{21}{256} \\ \frac{71}{128} \\ \frac{973}{256} \\ 1 \end{pmatrix}$$

```

```
D =  


$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

```

The output V is a 5-by-1 column vector with five-fold eigenvalues of 0. This result means that the matrix A has an eigenvalue of 0 with algebraic multiplicity 5 and geometric multiplicity 1.

### Find Linearly Independent Eigenvectors and Their Eigenvalues

Compute the exact eigenvalues and eigenvectors of a 4-by-4 symbolic matrix. Return a vector of indices that relate the eigenvalues to their linearly independent eigenvectors.

```
syms c
A = [c 1 0 0; 0 c 0 0; 0 0 3*c 0; 0 0 0 3*c];
[V,D,p] = eig(A)
```

```
V =
    (1 0 0)
    (0 0 0)
    (0 1 0)
    (0 0 1)
```

```
D =
    (c 0 0 0)
    (0 c 0 0)
    (0 0 3 c 0)
    (0 0 0 3 c)
```

```
p = 1×3
     1     3     4
```

The matrix  $A$  has two eigenvalues,  $c$  and  $3c$ , where each eigenvalue occurs twice. Meanwhile, there are three linearly independent eigenvectors. The vector of indices  $p$  shows that:

- $p(1) = 1$ , so the first eigenvector (the first column of  $V$ ) corresponds to the first diagonal element of  $D$  with eigenvalue  $c$ .
- $p(2) = 3$ , so the second eigenvector (the second column of  $V$ ) corresponds to the third diagonal element of  $D$  with eigenvalue  $3c$ .
- $p(3) = 4$ , so the third eigenvector (the third column of  $V$ ) corresponds to the fourth diagonal element of  $D$  with eigenvalue  $3c$ .

This result means the eigenvalue  $c$  that occurs twice has only one linearly independent eigenvector (the eigenvalue  $c$  has algebraic multiplicity 2 and geometric multiplicity 1). The eigenvalue  $3c$  that occurs twice has two linearly independent eigenvectors (the eigenvalue  $3c$  has algebraic multiplicity 2 and geometric multiplicity 2).

Show that the matrix  $A*V$  is equal to  $V*D(p,p)$ .

```
A*V
```

```
ans =
    (c 0 0)
    (0 0 0)
    (0 3 c 0)
    (0 0 3 c)
```

```
V*D(p,p)
```

```
ans =
    (c 0 0)
    (0 0 0)
    (0 3 c 0)
    (0 0 3 c)
```

```
tf = isequal(A*V,V*D(p,p))
```

```
tf = logical
    1
```

## Input Arguments

### **A — Square matrix**

symbolic matrix

Square matrix, specified as a symbolic matrix.

## Output Arguments

### **lambda — Eigenvalues (returned as vector)**

symbolic column vector | column vector of symbolic numbers

Eigenvalues, returned as a symbolic column vector or column vector of symbolic numbers.

### **V — Right eigenvectors**

square symbolic matrix

Right eigenvectors, returned as a square symbolic matrix. The columns of V are the right eigenvectors of A.

### **D — Eigenvalues (returned as matrix)**

symbolic diagonal matrix

Eigenvalues, returned as a symbolic diagonal matrix. The eigenvalues of A are on the main diagonal of D.

### **p — Vector of indices**

symbolic row vector

Vector of indices, returned as a symbolic row vector. The length of p is the total number of linearly independent eigenvectors of A.

## Tips

- Matrix computations involving many symbolic variables can be slow. To increase the computational speed, reduce the number of symbolic variables by substituting the given values for some variables.
- Calling `eig` for numeric matrices that are not symbolic objects (not created by `sym`, `syms`, or `vpa`) invokes the MATLAB `eig` function.

## Version History

**Introduced before R2006a**

**R2021b: `eig(A)` returns eigenvalues in terms of the root function**

*Behavior changed in R2021b*

If `eig(A)` cannot find the exact eigenvalues in terms of symbolic numbers, it now returns the exact eigenvalues in terms of the `root` function instead. In previous releases, `eig(A)` returns the eigenvalues as floating-point numbers.

For example, compute the eigenvalues of a 5-by-5 symbolic matrix. The `eig` function returns the exact eigenvalues in terms of the `root` function. This output is consistent with the results returned by the `solve` or `root` function when solving for the roots of a polynomial.

```
M = magic(6);
A = sym(M(1:5,1:5));
lambda = eig(A)

lambda =
root(z^5 - 100*z^4 + 134*z^3 + 66537*z^2 - 450198*z - 1294704, z, 1)
root(z^5 - 100*z^4 + 134*z^3 + 66537*z^2 - 450198*z - 1294704, z, 2)
root(z^5 - 100*z^4 + 134*z^3 + 66537*z^2 - 450198*z - 1294704, z, 3)
root(z^5 - 100*z^4 + 134*z^3 + 66537*z^2 - 450198*z - 1294704, z, 4)
root(z^5 - 100*z^4 + 134*z^3 + 66537*z^2 - 450198*z - 1294704, z, 5)
```

Use `vpa` to numerically approximate the eigenvalues.

```
lambdaVpa = vpa(lambda)

lambdaVpa =
-2.181032364984695108354692701065
9.8395828502812312578803604206392
-25.131641669799891607267584639192
26.341617610275869035465716505806
91.131473574227486422276200413812
```

## See Also

`charpoly` | `jordan` | `svd` | `vpa`

## Topics

“Eigenvalues” on page 3-349



# eliminate

Eliminate variables from rational equations

## Syntax

```
expr = eliminate(eqns, vars)
```

## Description

`expr = eliminate(eqns, vars)` eliminates the variables `vars` from the rational equations `eqns`. The result is a vector of symbolic expressions that is equal to zero.

## Examples

### Eliminate Variables from Rational Equations

Create two rational equations that contain the variables `x` and `y`.

```
syms x y
eqns = [x*y/(x-2) + y == 5/(y - x), y-x == 1/(x-1)]

eqns =
```

$$\left( y + \frac{xy}{x-2} = -\frac{5}{x-y} \quad y-x = \frac{1}{x-1} \right)$$

Eliminate the variable `x`. The result is a symbolic expression that is equal to zero.

```
expr = eliminate(eqns, x)

expr = [6 y^2 - 5 y - 75]
```

### Eliminate Variables from Polynomial Equations

Create two polynomial equations that contain the variables `x` and `y`.

```
syms x y
eqns = [2*x+y == 5; y-x == 1]

eqns =
```

$$\left( \begin{array}{l} 2x + y = 5 \\ y - x = 1 \end{array} \right)$$

Eliminate the variable `x` from the equations. The result is a symbolic expression that is equal to zero.

```
expr = eliminate(eqns, x)

expr = [3 y - 7]
```

Now, create three polynomial equations that contain the variables  $x$ ,  $y$ , and  $z$ . Eliminate the variable  $x$ . The result is a vector of symbolic expressions that is equal to zero.

```
syms z
eqns = [x^2 + y - z^2 == 2;
        x - z == y;
        x^2 + y^2 - z == 4];
expr = eliminate(eqns,x)

expr = [5z^3 - 5z^2 - 8z + 4y - 8, 5z^4 - 11z^2 - 18z - 8]
```

To eliminate both  $x$  and  $y$ , use the `eliminate` function and specify the two variables as the vector `[x y]`.

```
expr = eliminate(eqns,[x y])

expr = [5z^4 - 11z^2 - 18z - 8]
```

## Input Arguments

### **eqns** — Rational equations

vector of symbolic equations | vector of symbolic expressions

Rational equations, specified as a vector of symbolic equations or symbolic expressions. A rational equation is an equation that contains at least one fraction in which the numerator and the denominator are polynomials.

The relation operator `==` defines symbolic equations. If a symbolic expression `eqn` in `eqns` has no right side, then a symbolic equation with a right side equal to `0` is assumed.

### **vars** — Variables to eliminate

vector of symbolic variables

Variables to eliminate, specified as a vector of symbolic variables.

## Version History

Introduced in R2018a

### See Also

`gbasis` | `solve`

# ellipj

Jacobi elliptic functions

## Syntax

`[SN,CN,DN] = ellipj(u,m)`

## Description

`[SN,CN,DN] = ellipj(u,m)` returns the Jacobi sn, cn, and dn elliptic functions evaluated for the corresponding elements of `u` and `m`. Inputs `u` and `m` must be the same size, or either `u` or `m` must be scalar.

## Examples

### Compute Jacobi Elliptic Functions

Compute the Jacobi elliptic functions for `u = 0.75` and `m = 0.5`. Because these numbers are not symbolic objects, you get floating-point results.

```
[SN,CN,DN] = ellipj(0.75,0.5)
```

```
SN =  
0.6585
```

```
CN =  
0.7526
```

```
DN =  
0.8850
```

Compute the Jacobi elliptic functions for the same numbers converted to symbolic objects. For most symbolic (exact) numbers, `ellipj` returns results using the `jacobiSN`, `jacobiCN`, and `jacobiDN` functions.

```
[SN,CN,DN] = ellipj(sym(3/4),sym(1/2))
```

```
SN =  
sn(3/4 | 1/2)
```

```
CN =  
cn(3/4 | 1/2)
```

```
DN =  
dn(3/4 | 1/2)
```

Use `vpa` to approximate the symbolic results with floating-point numbers.

```
vpa([SN,CN,DN],10)
```

```
ans = (0.6585147441 0.7525678254 0.8849741046)
```

### Compute Jacobi Elliptic Functions When $m$ Is Not Between 0 and 1

If the argument  $m$  is not in  $[0\ 1]$ , then convert that argument to a symbolic object before using `ellipj`.

```
[SN,CN,DN] = ellipj(1,sym(pi/2))
```

```
SN =  
    sn(1 |  $\frac{\pi}{2}$ )
```

```
CN =  
    cn(1 |  $\frac{\pi}{2}$ )
```

```
DN =  
    dn(1 |  $\frac{\pi}{2}$ )
```

Alternatively, use `jacobiSN`, `jacobiCN`, and `jacobiDN` to compute the Jacobi elliptic functions separately.

```
SN = jacobiSN(1,sym(pi/2))
```

```
SN =  
    sn(1 |  $\frac{\pi}{2}$ )
```

```
CN = jacobiCN(1,sym(pi/2))
```

```
CN =  
    cn(1 |  $\frac{\pi}{2}$ )
```

```
DN = jacobiDN(1,sym(pi/2))
```

```
DN =  
    dn(1 |  $\frac{\pi}{2}$ )
```

### Compute Jacobi Elliptic Functions for Matrix Input

Call `ellipj` for symbolic matrix input. When the input arguments are matrices with the same size, `ellipj` computes the Jacobi elliptic functions for each element.

```
[SN,CN,DN] = ellipj(sym([-1 0; 1 1/2]),sym([1 pi/2; -1 0]))
```

```
SN =  
     $\begin{pmatrix} -\tanh(1) & 0 \\ \operatorname{sn}(1|-1) \sin\left(\frac{1}{2}\right) \end{pmatrix}$ 
```

$$\text{CN} = \begin{pmatrix} \frac{1}{\cosh(1)} & 1 \\ \text{cn}(1|-1) \cos\left(\frac{1}{2}\right) \end{pmatrix}$$

$$\text{DN} = \begin{pmatrix} \frac{1}{\cosh(1)} & 1 \\ \text{dn}(1|-1) & 1 \end{pmatrix}$$

## Input Arguments

### **u** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

### **m** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## Output Arguments

### **SN** — Jacobi sn elliptic function

symbolic expression

Jacobi sn elliptic function, returned as a symbolic expression.

### **CN** — Jacobi cn elliptic function

symbolic expression

Jacobi cn elliptic function, returned as a symbolic expression.

### **DN** — Jacobi dn elliptic function

symbolic expression

Jacobi dn elliptic function, returned as a symbolic expression.

## More About

### Jacobi Elliptic Functions

The Jacobi elliptic functions are defined as

$$\begin{aligned} \text{sn}(u, m) &= \sin\phi \\ \text{cn}(u, m) &= \cos\phi \\ \text{dn}(u, m) &= \sqrt{1 - m\sin^2\phi} \end{aligned}$$

where  $\phi$  satisfies the incomplete elliptic integral of the first kind

$$u = \int^{\phi} \frac{d\theta}{\sqrt{1 - m\sin^2\theta}}.$$

## Tips

- Calling `ellipj` for numbers that are not symbolic objects invokes the MATLAB `ellipj` function. This function accepts only  $0 \leq m \leq 1$ . To compute the Jacobi elliptic functions for values out of this range, use `sym` or `vpa` to convert the numbers to symbolic objects, and then call `ellipj` for those symbolic objects. Alternatively, use the `jacobiSN`, `jacobiCN`, and `jacobiDN` functions to compute the elliptic functions separately.
- For most symbolic (exact) numbers, `ellipj` returns results using the `jacobiSN`, `jacobiCN`, and `jacobiDN` functions. You can approximate such results with floating-point numbers using `vpa`.

## Version History

Introduced in R2017b

## References

- [1] Abramowitz, M. and I. A. Stegun, *Handbook of Mathematical Functions*, Dover Publications (1965), 17.6.

## See Also

`ellipticF` | `jacobiAM` | `jacobiSN` | `jacobiCN` | `jacobiDN`

# ellipke

Complete elliptic integrals of the first and second kinds

## Syntax

```
[K,E] = ellipke(m)
```

## Description

`[K,E] = ellipke(m)` returns the complete elliptic integrals of the first on page 6-345 and second kinds on page 6-345.

## Examples

### Compute Complete Elliptic Integrals of First and Second Kind

Compute the complete elliptic integrals of the first and second kinds for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
[K0, E0] = ellipke(0)
[K05, E05] = ellipke(1/2)
```

```
K0 =
    1.5708
```

```
E0 =
    1.5708
```

```
K05 =
    1.8541
```

```
E05 =
    1.3506
```

Compute the complete elliptic integrals for the same numbers converted to symbolic objects. For most symbolic (exact) numbers, `ellipke` returns results using the `ellipticK` and `ellipticE` functions.

```
[K0, E0] = ellipke(sym(0))
[K05, E05] = ellipke(sym(1/2))
```

```
K0 =
pi/2
```

```
E0 =
pi/2
```

```
K05 =
ellipticK(1/2)
```

```
E05 =
ellipticE(1/2)
```

Use `vpa` to approximate `K05` and `E05` with floating-point numbers:

```
vpa([K05, E05], 10)

ans =
[ 1.854074677, 1.350643881]
```

### Compute Integrals When Input is Not Between 0 and 1

If the argument does not belong to the range from 0 to 1, then convert that argument to a symbolic object before using `ellipke`:

```
[K, E] = ellipke(sym(pi/2))

K =
ellipticK(pi/2)

E =
ellipticE(pi/2)
```

Alternatively, use `ellipticK` and `ellipticE` to compute the integrals of the first and the second kinds separately:

```
K = ellipticK(sym(pi/2))
E = ellipticE(sym(pi/2))

K =
ellipticK(pi/2)

E =
ellipticE(pi/2)
```

### Compute Integrals for Matrix Input

Call `ellipke` for this symbolic matrix. When the input argument is a matrix, `ellipke` computes the complete elliptic integrals of the first and second kinds for each element.

```
[K, E] = ellipke(sym([-1 0; 1/2 1]))

K =
[ ellipticK(-1), pi/2]
[ ellipticK(1/2), Inf]

E =
[ ellipticE(-1), pi/2]
[ ellipticE(1/2), 1]
```

## Input Arguments

### **m** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.



## Output Arguments

### K — Complete elliptic integral of the first kind

symbolic expression

Complete elliptic integral of the first kind, returned as a symbolic expression.

### E — Complete elliptic integral of the second kind

symbolic expression

Complete elliptic integral of the second kind, returned as a symbolic expression.

## More About

### Complete Elliptic Integral of the First Kind

The complete elliptic integral of the first kind is defined as follows:

$$K(m) = F\left(\frac{\pi}{2} \middle| m\right) = \int_0^{\pi/2} \frac{1}{\sqrt{1 - m\sin^2\theta}} d\theta$$

Note that some definitions use the elliptical modulus  $k$  or the modular angle  $\alpha$  instead of the parameter  $m$ . They are related as  $m = k^2 = \sin^2\alpha$ .

### Complete Elliptic Integral of the Second Kind

The complete elliptic integral of the second kind is defined as follows:

$$E(m) = E\left(\frac{\pi}{2} \middle| m\right) = \int_0^{\pi/2} \sqrt{1 - m\sin^2\theta} d\theta$$

Note that some definitions use the elliptical modulus  $k$  or the modular angle  $\alpha$  instead of the parameter  $m$ . They are related as  $m = k^2 = \sin^2\alpha$ .

## Tips

- Calling `ellipke` for numbers that are not symbolic objects invokes the MATLAB `ellipke` function. This function accepts only  $0 \leq m \leq 1$ . To compute the complete elliptic integrals of the first and second kinds for the values out of this range, use `sym` to convert the numbers to symbolic objects, and then call `ellipke` for those symbolic objects. Alternatively, use the `ellipticK` and `ellipticE` functions to compute the integrals separately.
- For most symbolic (exact) numbers, `ellipke` returns results using the `ellipticK` and `ellipticE` functions. You can approximate such results with floating-point numbers using `vpa`.
- If  $m$  is a vector or a matrix, then `[K,E] = ellipke(m)` returns the complete elliptic integrals of the first and second kinds, evaluated for each element of  $m$ .

## Alternatives

You can use `ellipticK` and `ellipticE` to compute elliptic integrals of the first and second kinds separately.

## Version History

Introduced in R2013a

## References

- [1] Milne-Thomson, L. M. "Elliptic Integrals." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

`ellipke` | `ellipticE` | `ellipticK` | `vpa`

# ellipticCE

Complementary complete elliptic integral of the second kind

## Syntax

`ellipticCE(m)`

## Description

`ellipticCE(m)` returns the complementary complete elliptic integral of the second kind on page 6-349.

## Examples

### Find Complementary Complete Elliptic Integral of the Second Kind

Compute the complementary complete elliptic integrals of the second kind for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
s = [ellipticCE(0), ellipticCE(pi/4),...
    ellipticCE(1), ellipticCE(pi/2)]
```

```
s =
    1.0000    1.4828    1.5708    1.7753
```

Compute the complementary complete elliptic integrals of the second kind for the same numbers converted to symbolic objects. For most symbolic (exact) numbers, `ellipticCE` returns unresolved symbolic calls.

```
s = [ellipticCE(sym(0)), ellipticCE(sym(pi/4)),...
    ellipticCE(sym(1)), ellipticCE(sym(pi/2))]
```

```
s =
[ 1, ellipticCE(pi/4), pi/2, ellipticCE(pi/2)]
```

Use `vpa` to approximate this result with floating-point numbers:

```
vpa(s, 10)
```

```
ans =
[ 1.0, 1.482786927, 1.570796327, 1.775344699]
```

### Find Elliptic Integral for Matrix Input

Call `ellipticCE` for this symbolic matrix. When the input argument is a matrix, `ellipticCE` computes the complementary complete elliptic integral of the second kind for each element.

```
ellipticCE(sym([pi/6 pi/4; pi/3 pi/2]))
```

```
ans =
[ ellipticCE(pi/6), ellipticCE(pi/4)]
[ ellipticCE(pi/3), ellipticCE(pi/2)]
```

### Differentiate Complementary Complete Elliptic Integral of the Second Kind

Differentiate these expressions involving the complementary complete elliptic integral of the second kind:

```
syms m
diff(ellipticCE(m))
diff(ellipticCE(m^2), m, 2)

ans =
ellipticCE(m)/(2*m - 2) - ellipticCK(m)/(2*m - 2)

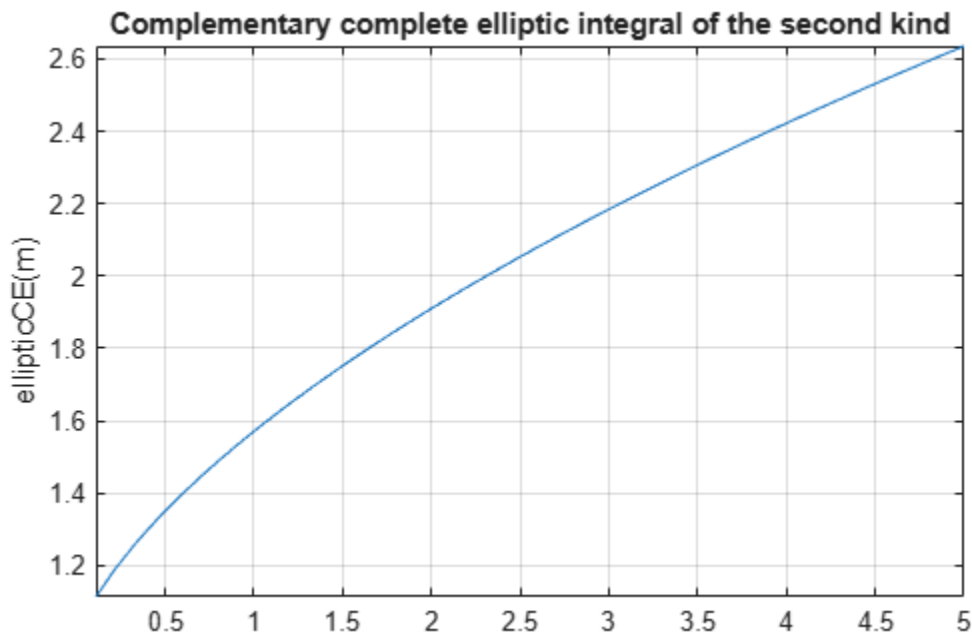
ans =
(2*ellipticCE(m^2))/(2*m^2 - 2) -...
(2*ellipticCK(m^2))/(2*m^2 - 2) +...
2*m*((2*m*ellipticCK(m^2))/(2*m^2 - 2) -...
ellipticCE(m^2)/(m*(m^2 - 1)))/(2*m^2 - 2) +...
(2*m*(ellipticCE(m^2)/(2*m^2 - 2) -...
ellipticCK(m^2)/(2*m^2 - 2)))/(2*m^2 - 2) -...
(4*m*ellipticCE(m^2))/(2*m^2 - 2)^2 +...
(4*m*ellipticCK(m^2))/(2*m^2 - 2)^2)
```

Here, `ellipticCK` represents the complementary complete elliptic integral of the first kind.

### Plot Complementary Complete Elliptic Integral of Second Kind

Plot the complementary complete elliptic integral of the second kind.

```
syms m
fplot(ellipticCE(m))
title('Complementary complete elliptic integral of the second kind')
ylabel('ellipticCE(m)')
grid on
```



## Input Arguments

### **m** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## More About

### Complementary Complete Elliptic Integral of the Second Kind

The complementary complete elliptic integral of the second kind is defined as  $E'(m) = E(1-m)$ , where  $E(m)$  is the complete elliptic integral of the second kind:

$$E(m) = E\left(\frac{\pi}{2} \middle| m\right) = \int_0^{\pi/2} \sqrt{1 - m \sin^2 \theta} d\theta$$

Note that some definitions use the elliptical modulus  $k$  or the modular angle  $\alpha$  instead of the parameter  $m$ . They are related as  $m = k^2 = \sin^2 \alpha$ .

## Tips

- `ellipticCE` returns floating-point results for numeric arguments that are not symbolic objects.
- For most symbolic (exact) numbers, `ellipticCE` returns unresolved symbolic calls. You can approximate such results with floating-point numbers using `vpa`.
- If  $m$  is a vector or a matrix, then `ellipticCE(m)` returns the complementary complete elliptic integral of the second kind, evaluated for each element of  $m$ .

## Version History

Introduced in R2013a

## References

- [1] Milne-Thomson, L. M. "Elliptic Integrals." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

`ellipke` | `ellipticCK` | `ellipticCPi` | `ellipticE` | `ellipticK` | `ellipticF` | `ellipticPi` | `vpa`

## ellipticCK

Complementary complete elliptic integral of the first kind

### Syntax

`ellipticCK(m)`

### Description

`ellipticCK(m)` returns the complementary complete elliptic integral of the first kind on page 6-352.

### Examples

#### Find Complementary Complete Elliptic Integral of First Kind

Compute the complementary complete elliptic integrals of the first kind for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
s = [ellipticCK(1/2), ellipticCK(pi/4), ellipticCK(1), ellipticCK(inf)]
```

```
s =  
    1.8541    1.6671    1.5708    NaN
```

Compute the complete elliptic integrals of the first kind for the same numbers converted to symbolic objects. For most symbolic (exact) numbers, `ellipticCK` returns unresolved symbolic calls.

```
s = [ellipticCK(sym(1/2)), ellipticCK(sym(pi/4)), ...  
    ellipticCK(sym(1)), ellipticCK(sym(inf))]
```

```
s =  
[ ellipticCK(1/2), ellipticCK(pi/4), pi/2, ellipticCK(Inf)]
```

Use `vpa` to approximate this result with floating-point numbers:

```
vpa(s, 10)
```

```
ans =  
[ 1.854074677, 1.667061338, 1.570796327, NaN]
```

#### Differentiate Complementary Complete Elliptic Integral of First Kind

Differentiate these expressions involving the complementary complete elliptic integral of the first kind:

```
syms m  
diff(ellipticCK(m))  
diff(ellipticCK(m^2), m, 2)
```

```
ans =  
ellipticCE(m)/(2*m*(m - 1)) - ellipticCK(m)/(2*m - 2)
```

```
ans =  
(2*(ellipticCE(m^2)/(2*m^2 - 2) - ...
```

```

ellipticCK(m^2)/(2*m^2 - 2))/(m^2 - 1) -...
(2*ellipticCE(m^2))/(m^2 - 1)^2 -...
(2*ellipticCK(m^2))/(2*m^2 - 2) +...
(8*m^2*ellipticCK(m^2))/(2*m^2 - 2)^2 +...
(2*m*((2*m*ellipticCK(m^2))/(2*m^2 - 2) -...
ellipticCE(m^2)/(m*(m^2 - 1)))/(2*m^2 - 2) -...
ellipticCE(m^2)/(m^2*(m^2 - 1))

```

Here, `ellipticCE` represents the complementary complete elliptic integral of the second kind.

### Find Elliptic Integral for Matrix Input

Call `ellipticCK` for this symbolic matrix. When the input argument is a matrix, `ellipticCK` computes the complementary complete elliptic integral of the first kind for each element.

```
ellipticCK(sym([pi/6 pi/4; pi/3 pi/2]))
```

```

ans =
[ ellipticCK(pi/6), ellipticCK(pi/4)]
[ ellipticCK(pi/3), ellipticCK(pi/2)]

```

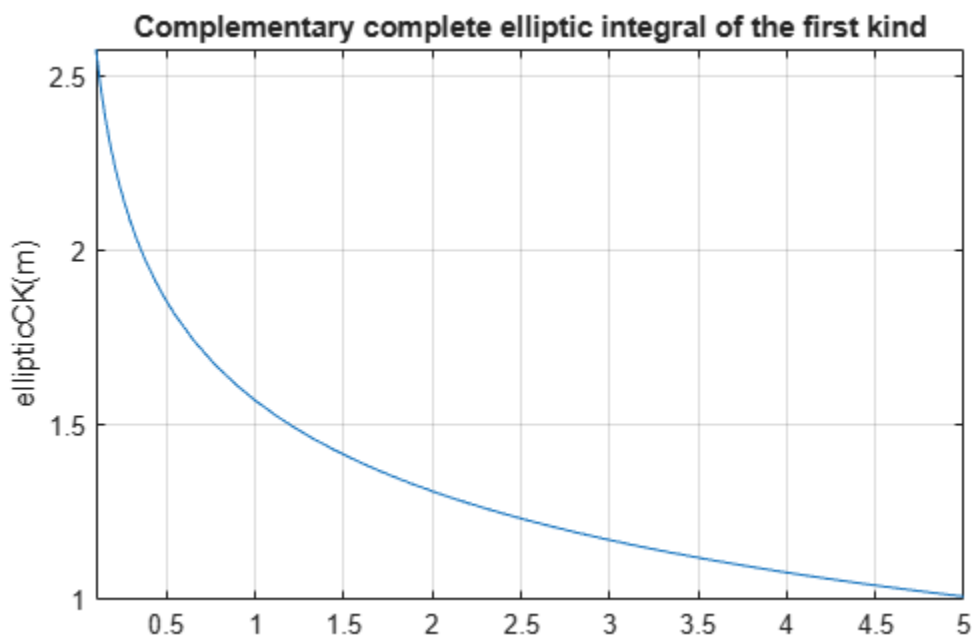
### Plot Complementary Complete Elliptic Integral of First Kind

Plot complementary complete elliptic integral of first kind.

```

syms m
fplot(ellipticCK(m),[0.1 5])
title('Complementary complete elliptic integral of the first kind')
ylabel('ellipticCK(m)')
grid on
hold off

```



## Input Arguments

### **m** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## More About

### Complementary Complete Elliptic Integral of the First Kind

The complementary complete elliptic integral of the first kind is defined as  $K'(m) = K(1-m)$ , where  $K(m)$  is the complete elliptic integral of the first kind:

$$K(m) = F\left(\frac{\pi}{2} \middle| m\right) = \int_0^{\pi/2} \frac{1}{\sqrt{1 - m \sin^2 \theta}} d\theta$$

Note that some definitions use the elliptical modulus  $k$  or the modular angle  $\alpha$  instead of the parameter  $m$ . They are related as  $m = k^2 = \sin^2 \alpha$ .

## Tips

- `ellipticK` returns floating-point results for numeric arguments that are not symbolic objects.
- For most symbolic (exact) numbers, `ellipticCK` returns unresolved symbolic calls. You can approximate such results with floating-point numbers using the `vpa` function.
- If  $m$  is a vector or a matrix, then `ellipticCK(m)` returns the complementary complete elliptic integral of the first kind, evaluated for each element of  $m$ .

## Version History

Introduced in R2013a

## References

- [1] Milne-Thomson, L. M. "Elliptic Integrals." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

`ellipke` | `ellipticCE` | `ellipticCPi` | `ellipticE` | `ellipticK` | `ellipticF` | `ellipticPi` | `vpa`



# ellipticCPi

Complementary complete elliptic integral of the third kind

## Syntax

`ellipticCPi(n,m)`

## Description

`ellipticCPi(n,m)` returns the complementary complete elliptic integral of the third kind on page 6-354.

## Examples

### Compute Complementary Complete Elliptic Integrals of Third Kind

Compute the complementary complete elliptic integrals of the third kind for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
s = [ellipticCPi(-1, 1/3), ellipticCPi(0, 1/2), ...
    ellipticCPi(9/10, 1), ellipticCPi(-1, 0)]
```

```
s =
    1.3703    1.8541    4.9673    Inf
```

Compute the complementary complete elliptic integrals of the third kind for the same numbers converted to symbolic objects. For most symbolic (exact) numbers, `ellipticCPi` returns unresolved symbolic calls.

```
s = [ellipticCPi(-1, sym(1/3)), ellipticCPi(sym(0), 1/2), ...
    ellipticCPi(sym(9/10), 1), ellipticCPi(-1, sym(0))]
```

```
s =
[ ellipticCPi(-1, 1/3), ellipticCK(1/2), (pi*10^(1/2))/2, Inf]
```

Here, `ellipticCK` represents the complementary complete elliptic integrals of the first kind.

Use `vpa` to approximate this result with floating-point numbers:

```
vpa(s, 10)

ans =
[ 1.370337322, 1.854074677, 4.967294133, Inf]
```

### Differentiate Complementary Complete Elliptic Integrals of Third Kind

Differentiate these expressions involving the complementary complete elliptic integral of the third kind:

```
syms n m
diff(ellipticCPi(n, m), n)
diff(ellipticCPi(n, m), m)
```

```

ans =
ellipticCK(m)/(2*n*(n - 1)) -...
ellipticCE(m)/(2*(n - 1)*(m + n - 1)) -...
(ellipticCPi(n, m)*(n^2 + m - 1))/(2*n*(n - 1)*(m + n - 1))

ans =
ellipticCE(m)/(2*m*(m + n - 1)) - ellipticCPi(n, m)/(2*(m + n - 1))

```

Here, `ellipticCK` and `ellipticCE` represent the complementary complete elliptic integrals of the first and second kinds.

## Input Arguments

### **n** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

### **m** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## More About

### Complementary Complete Elliptic Integral of the Third Kind

The complementary complete elliptic integral of the third kind is defined as  $\Pi'(m) = \Pi(n, 1-m)$ , where  $\Pi(n, m)$  is the complete elliptic integral of the third kind:

$$\Pi(n, m) = \Pi\left(n; \frac{\pi}{2} \middle| m\right) = \int_0^{\pi/2} \frac{1}{(1 - n \sin^2 \theta) \sqrt{1 - m \sin^2 \theta}} d\theta$$

Note that some definitions use the elliptical modulus  $k$  or the modular angle  $\alpha$  instead of the parameter  $m$ . They are related as  $m = k^2 = \sin^2 \alpha$ .

## Tips

- `ellipticCPi` returns floating-point results for numeric arguments that are not symbolic objects.
- For most symbolic (exact) numbers, `ellipticCPi` returns unresolved symbolic calls. You can approximate such results with floating-point numbers using `vpa`.
- At least one input argument must be a scalar or both arguments must be vectors or matrices of the same size. If one input argument is a scalar and the other one is a vector or a matrix, then `ellipticCPi` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

## Version History

Introduced in R2013a

## References

- [1] Milne-Thomson, L. M. "Elliptic Integrals." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

ellipke | ellipticCE | ellipticCK | ellipticE | ellipticK | ellipticF | ellipticPi | vpa

## ellipticE

Complete and incomplete elliptic integrals of the second kind

### Syntax

```
ellipticE(m)
ellipticE(phi,m)
```

### Description

`ellipticE(m)` returns the complete elliptic integral of the second kind on page 6-358.

`ellipticE(phi,m)` returns the incomplete elliptic integral of the second kind on page 6-358.

### Examples

#### Find Complete Elliptic Integrals of Second Kind

Compute the complete elliptic integrals of the second kind for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
s = [ellipticE(-10.5), ellipticE(-pi/4),...
     ellipticE(0), ellipticE(1)]
```

```
s =
    3.7096    1.8443    1.5708    1.0000
```

Compute the complete elliptic integral of the second kind for the same numbers converted to symbolic objects. For most symbolic (exact) numbers, `ellipticE` returns unresolved symbolic calls.

```
s = [ellipticE(sym(-10.5)), ellipticE(sym(-pi/4)),...
     ellipticE(sym(0)), ellipticE(sym(1))]
```

```
s =
[ ellipticE(-21/2), ellipticE(-pi/4), pi/2, 1]
```

Use `vpa` to approximate this result with floating-point numbers:

```
vpa(s, 10)

ans =
[ 3.70961391, 1.844349247, 1.570796327, 1.0]
```

#### Differentiate Elliptic Integrals of Second Kind

Differentiate these expressions involving elliptic integrals of the second kind. `ellipticK` and `ellipticF` represent the complete and incomplete elliptic integrals of the first kind, respectively.

```
syms m
diff(ellipticE(pi/3, m))
diff(ellipticE(m^2), m, 2)
```

```
ans =
ellipticE(pi/3, m)/(2*m) - ellipticF(pi/3, m)/(2*m)
```

```
ans =
2*m*((ellipticE(m^2)/(2*m^2) - ...
ellipticK(m^2)/(2*m^2))/m - ellipticE(m^2)/m^3 + ...
ellipticK(m^2)/m^3 + (ellipticK(m^2)/m + ...
ellipticE(m^2)/(m*(m^2 - 1)))/(2*m^2)) + ...
ellipticE(m^2)/m^2 - ellipticK(m^2)/m^2
```

### Elliptic Integral for Matrix Input

Call `ellipticE` for this symbolic matrix. When the input argument is a matrix, `ellipticE` computes the complete elliptic integral of the second kind for each element.

```
ellipticE(sym([1/3 1; 1/2 0]))
```

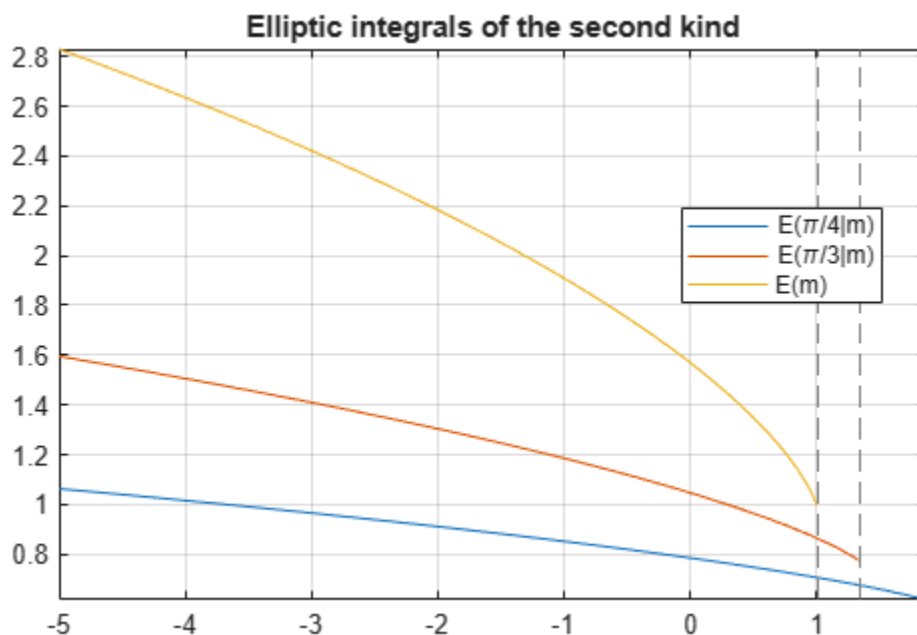
```
ans =
[ ellipticE(1/3),    1]
[ ellipticE(1/2), pi/2]
```

### Plot Complete and Incomplete Elliptic Integrals of Second Kind

Plot the incomplete elliptic integrals `ellipticE(phi,m)` for  $\phi = \pi/4$  and  $\phi = \pi/3$ . Also plot the complete elliptic integral `ellipticE(m)`.

```
syms m
fplot([ellipticE(pi/4,m) ellipticE(pi/3,m) ellipticE(m)])

title('Elliptic integrals of the second kind')
legend('E(\pi/4|m)', 'E(\pi/3|m)', 'E(m)', 'Location', 'Best')
grid on
```



## Input Arguments

### **m** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

### **phi** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## More About

### Incomplete Elliptic Integral of the Second Kind

The incomplete elliptic integral of the second kind is defined as follows:

$$E(\varphi|m) = \int_0^{\varphi} \sqrt{1 - m \sin^2 \theta} d\theta$$

Note that some definitions use the elliptical modulus  $k$  or the modular angle  $\alpha$  instead of the parameter  $m$ . They are related as  $m = k^2 = \sin^2 \alpha$ .

### Complete Elliptic Integral of the Second Kind

The complete elliptic integral of the second kind is defined as follows:

$$E(m) = E\left(\frac{\pi}{2} \middle| m\right) = \int_0^{\pi/2} \sqrt{1 - m \sin^2 \theta} d\theta$$

Note that some definitions use the elliptical modulus  $k$  or the modular angle  $\alpha$  instead of the parameter  $m$ . They are related as  $m = k^2 = \sin^2 \alpha$ .

## Tips

- `ellipticE` returns floating-point results for numeric arguments that are not symbolic objects.
- For most symbolic (exact) numbers, `ellipticE` returns unresolved symbolic calls. You can approximate such results with floating-point numbers using `vpa`.
- If `m` is a vector or a matrix, then `ellipticE(m)` returns the complete elliptic integral of the second kind, evaluated for each element of `m`.
- At least one input argument must be a scalar or both arguments must be vectors or matrices of the same size. If one input argument is a scalar and the other one is a vector or a matrix, then `ellipticE` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

- `ellipticE(pi/2, m) = ellipticE(m)`.

## Alternatives

You can use `ellipke` to compute elliptic integrals of the first and second kinds in one function call.

## Version History

Introduced in R2013a

## References

- [1] Milne-Thomson, L. M. "Elliptic Integrals." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

`ellipke` | `ellipticCE` | `ellipticCK` | `ellipticCPi` | `ellipticF` | `ellipticK` | `ellipticPi` | `vpa`

## ellipticF

Incomplete elliptic integral of the first kind

### Syntax

`ellipticF(phi,m)`

### Description

`ellipticF(phi,m)` returns the incomplete elliptic integral of the first kind on page 6-362.

### Examples

#### Find Incomplete Elliptic Integrals of First Kind

Compute the incomplete elliptic integrals of the first kind for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
s = [ellipticF(pi/3, -10.5), ellipticF(pi/4, -pi),...
    ellipticF(1, -1), ellipticF(pi/2, 0)]
```

```
s =
    0.6184    0.6486    0.8964    1.5708
```

Compute the incomplete elliptic integrals of the first kind for the same numbers converted to symbolic objects. For most symbolic (exact) numbers, `ellipticF` returns unresolved symbolic calls.

```
s = [ellipticF(sym(pi/3), -10.5), ellipticF(sym(pi/4), -pi),...
    ellipticF(sym(1), -1), ellipticF(pi/6, sym(0))]
```

```
s =
[ ellipticF(pi/3, -21/2), ellipticF(pi/4, -pi), ellipticF(1, -1), pi/6]
```

Use `vpa` to approximate this result with floating-point numbers:

```
vpa(s, 10)
```

```
ans =
[ 0.6184459461, 0.6485970495, 0.8963937895, 0.5235987756]
```

#### Differentiate Incomplete Elliptic Integrals of First Kind

Differentiate this expression involving the incomplete elliptic integral of the first kind. `ellipticE` represents the incomplete elliptic integral of the second kind.

```
syms m
diff(ellipticF(pi/4, m))
```

```
ans =
1/(4*(1 - m/2)^(1/2)*(m - 1)) - ellipticF(pi/4, m)/(2*m) - ...
ellipticE(pi/4, m)/(2*m*(m - 1))
```

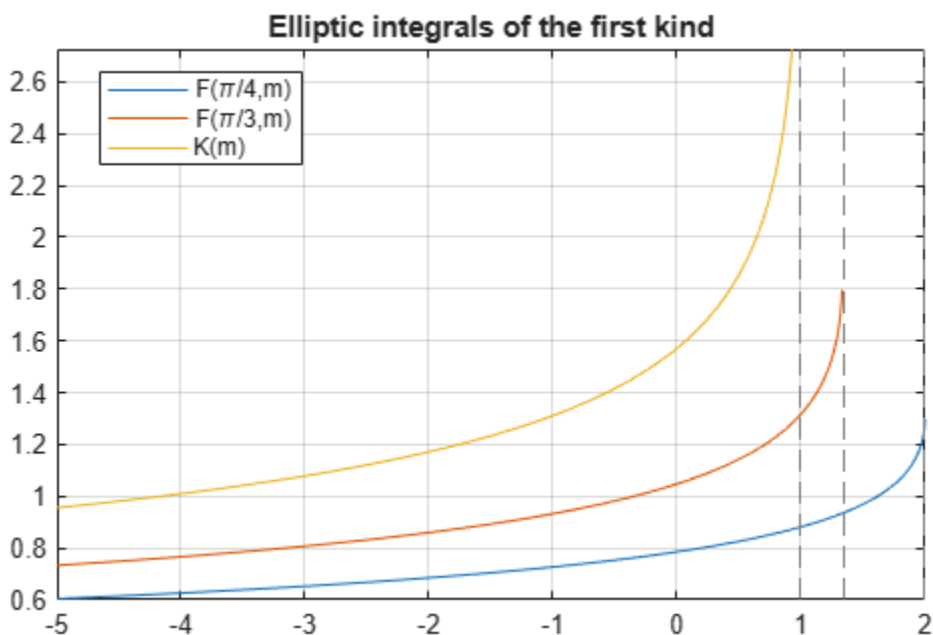


## Plot Incomplete and Complete Elliptic Integrals

Plot the incomplete elliptic integrals `ellipticF(phi,m)` for  $\phi = \pi/4$  and  $\phi = \pi/3$ . Also plot the complete elliptic integral `ellipticK(m)`.

```
syms m
fplot([ellipticF(pi/4, m) ellipticF(pi/3, m) ellipticK(m)])
grid on

title('Elliptic integrals of the first kind')
legend('F(\pi/4,m)', 'F(\pi/3,m)', 'K(m)', 'Location', 'Best')
```



## Input Arguments

### **m** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

### **phi** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## More About

### Incomplete Elliptic Integral of the First Kind

The complete elliptic integral of the first kind is defined as follows:

$$F(\varphi|m) = \int_0^\varphi \frac{1}{\sqrt{1 - m\sin^2\theta}} d\theta$$

Note that some definitions use the elliptical modulus  $k$  or the modular angle  $\alpha$  instead of the parameter  $m$ . They are related as  $m = k^2 = \sin^2\alpha$ .

## Tips

- `ellipticF` returns floating-point results for numeric arguments that are not symbolic objects.
- For most symbolic (exact) numbers, `ellipticF` returns unresolved symbolic calls. You can approximate such results with floating-point numbers using `vpa`.
- At least one input argument must be a scalar or both arguments must be vectors or matrices of the same size. If one input argument is a scalar and the other one is a vector or a matrix, `ellipticF` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.
- `ellipticF(pi/2, m) = ellipticK(m)`.

## Version History

Introduced in R2013a

## References

- [1] Milne-Thomson, L. M. "Elliptic Integrals." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

`ellipke` | `ellipticCE` | `ellipticCK` | `ellipticCPi` | `ellipticE` | `ellipticK` | `ellipticPi` | `vpa`

# ellipticK

Complete elliptic integral of the first kind

## Syntax

`ellipticK(m)`

## Description

`ellipticK(m)` returns the complete elliptic integral of the first kind on page 6-365.

## Examples

### Find Complete Elliptic Integrals of First Kind

Compute the complete elliptic integrals of the first kind for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
s = [ellipticK(1/2), ellipticK(pi/4), ellipticK(1), ellipticK(-5.5)]
```

```
s =  
    1.8541    2.2253    Inf    0.9325
```

Compute the complete elliptic integrals of the first kind for the same numbers converted to symbolic objects. For most symbolic (exact) numbers, `ellipticK` returns unresolved symbolic calls.

```
s = [ellipticK(sym(1/2)), ellipticK(sym(pi/4)), ...  
    ellipticK(sym(1)), ellipticK(sym(-5.5))]
```

```
s =  
[ ellipticK(1/2), ellipticK(pi/4), Inf, ellipticK(-11/2)]
```

Use `vpa` to approximate this result with floating-point numbers:

```
vpa(s, 10)  
  
ans =  
[ 1.854074677, 2.225253684, Inf, 0.9324665884]
```

### Differentiate Complete Elliptic Integral of First Kind

Differentiate these expressions involving the complete elliptic integral of the first kind. `ellipticE` represents the complete elliptic integral of the second kind.

```
syms m  
diff(ellipticK(m))  
diff(ellipticK(m^2), m, 2)  
  
ans =  
- ellipticK(m)/(2*m) - ellipticE(m)/(2*m*(m - 1))  
  
ans =  
(2*ellipticE(m^2))/(m^2 - 1)^2 - (2*(ellipticE(m^2)/(2*m^2) - ...
```

```

ellipticK(m^2)/(2*m^2))/(m^2 - 1) + ellipticK(m^2)/m^2 +...
(ellipticK(m^2)/m + ellipticE(m^2)/(m*(m^2 - 1)))/m +...
ellipticE(m^2)/(m^2*(m^2 - 1))

```

### Elliptic Integral for Matrix Input

Call `ellipticK` for this symbolic matrix. When the input argument is a matrix, `ellipticK` computes the complete elliptic integral of the first kind for each element.

```

ellipticK(sym([-2*pi -4; 0 1]))

ans =
[ ellipticK(-2*pi), ellipticK(-4)]
[          pi/2,          Inf]

```

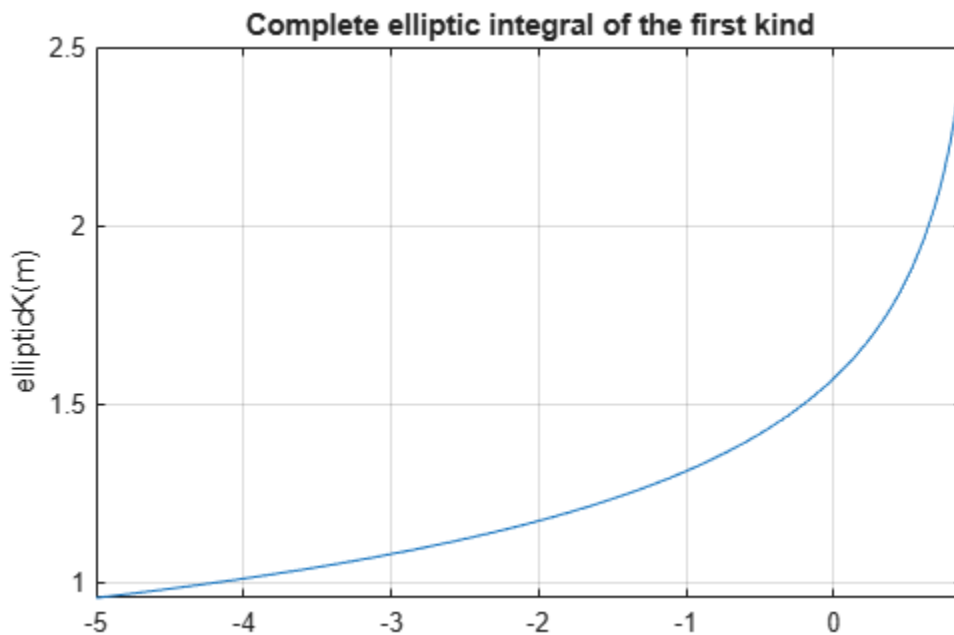
### Plot Complete Elliptic Integral of First Kind

Plot the complete elliptic integral of the first kind.

```

syms m
fplot(ellipticK(m))
title('Complete elliptic integral of the first kind')
ylabel('ellipticK(m)')
grid on

```



## Input Arguments

### m — Input

number | vector | matrix | multidimensional array | symbolic number | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## More About

### Complete Elliptic Integral of the First Kind

The complete elliptic integral of the first kind is defined as follows:

$$K(m) = F\left(\frac{\pi}{2} \middle| m\right) = \int_0^{\pi/2} \frac{1}{\sqrt{1 - m \sin^2 \theta}} d\theta$$

Note that some definitions use the elliptical modulus  $k$  or the modular angle  $\alpha$  instead of the parameter  $m$ . They are related as  $m = k^2 = \sin^2 \alpha$ .

## Tips

- `ellipticK` returns floating-point results for numeric arguments that are not symbolic objects.
- For most symbolic (exact) numbers, `ellipticK` returns unresolved symbolic calls. You can approximate such results with floating-point numbers using `vpa`.
- If  $m$  is a vector or a matrix, then `ellipticK(m)` returns the complete elliptic integral of the first kind, evaluated for each element of  $m$ .

## Alternatives

You can use `ellipke` to compute elliptic integrals of the first and second kinds in one function call.

## Version History

Introduced in R2013a

## References

- [1] Milne-Thomson, L. M. "Elliptic Integrals." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

`ellipke` | `ellipticCE` | `ellipticCK` | `ellipticCPi` | `ellipticE` | `ellipticF` | `ellipticPi` | `vpa`

## ellipticNome

Elliptic nome function

### Syntax

`ellipticNome(m)`

### Description

`ellipticNome(m)` returns the “Elliptic Nome” on page 6-368 of *m*. If *m* is an array, then `ellipticNome` acts element-wise.

### Examples

#### Calculate Elliptic Nome for Numeric Inputs

```
ellipticNome(1.3)
```

```
ans =  
    0.0881 - 0.2012i
```

Call `ellipticNome` on array inputs. `ellipticNome` acts element-wise when *m* is an array.

```
ellipticNome([2 1 -3/2])
```

```
ans =  
    0.0000 - 0.2079i    1.0000 + 0.0000i   -0.0570 + 0.0000i
```

#### Calculate Elliptic Nome for Symbolic Numbers

Convert numeric input to symbolic form using `sym`, and find the elliptic nome. For symbolic input where  $m = 0$ ,  $1/2$ , or  $1$ , `ellipticNome` returns exact symbolic output.

```
ellipticNome([0 1/2 1])
```

```
ans =  
      0      0.0432      1.0000
```

Show that for any other symbolic values of *m*, `ellipticNome` returns an unevaluated function call.

```
ellipticNome(sym(2))
```

```
ans =  
ellipticNome(2)
```

#### Find Elliptic Nome for Symbolic Variables or Expressions

For symbolic variables or expressions, `ellipticNome` returns the unevaluated function call.

```
syms x
f = ellipticNome(x)
```

```
f =
ellipticNome(x)
```

Substitute values for the variables by using `subs`, and convert values to double by using `double`.

```
f = subs(f, x, 5)
```

```
f =
ellipticNome(5)
```

```
fVal = double(f)
```

```
fVal =
-0.1008 - 0.1992i
```

Calculate `f` to higher precision using `vpa`.

```
fVal = vpa(f)
```

```
fVal =
- 0.10080189716733475056506021415746 - 0.19922973618609837873340100821425i
```

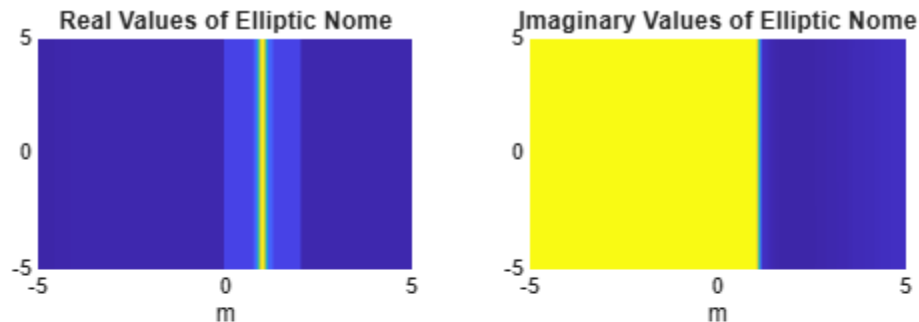
### Plot Elliptic Nome

Plot the real and imaginary values of the elliptic nome using `fcontour`. Fill plot contours by setting `Fill` to `on`.

```
syms m
f = ellipticNome(m);
```

```
subplot(2,2,1)
fcontour(real(f),'Fill','on')
title('Real Values of Elliptic Nome')
xlabel('m')
```

```
subplot(2,2,2)
fcontour(imag(f),'Fill','on')
title('Imaginary Values of Elliptic Nome')
xlabel('m')
```



## Input Arguments

### **m** — Input

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

## More About

### Elliptic Nome

The elliptic nome is

$$q(m) = e^{-\frac{\pi K'(m)}{K(m)}}$$

where  $K$  is the complete elliptic integral of the first kind, implemented as `ellipticK`.

$|q(m)| \leq 1$  holds for all  $m \in \mathbb{C}$ .

## Version History

Introduced in R2017b



**See Also**

ellipticK | jacobiAM | jacobiCD | jacobiCN | jacobiCS | jacobiDC | jacobiDN | jacobiDS |  
jacobiNC | jacobiND | jacobiNS | jacobiSC | jacobiSD | jacobiSN

## ellipticPi

Complete and incomplete elliptic integrals of the third kind

### Syntax

```
ellipticPi(n,m)
ellipticPi(n,phi,m)
```

### Description

`ellipticPi(n,m)` returns the complete elliptic integral of the third kind on page 6-372.

`ellipticPi(n,phi,m)` returns the incomplete elliptic integral of the third kind on page 6-371.

### Examples

#### Compute the Incomplete Elliptic Integrals of Third Kind

Compute the incomplete elliptic integrals of the third kind for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
s = [ellipticPi(-2.3, pi/4, 0), ellipticPi(1/3, pi/3, 1/2),...
    ellipticPi(-1, 0, 1), ellipticPi(2, pi/6, 2)]

s =
    0.5877    1.2850         0    0.7507
```

Compute the incomplete elliptic integrals of the third kind for the same numbers converted to symbolic objects. For most symbolic (exact) numbers, `ellipticPi` returns unresolved symbolic calls.

```
s = [ellipticPi(-2.3, sym(pi/4), 0), ellipticPi(sym(1/3), pi/3, 1/2),...
    ellipticPi(-1, sym(0), 1), ellipticPi(2, pi/6, sym(2))]
```

s =

```
[ ellipticPi(-23/10, pi/4, 0), ellipticPi(1/3, pi/3, 1/2),...
0, (2^(1/2)*3^(1/2))/2 - ellipticE(pi/6, 2)]
```

Here, `ellipticE` represents the incomplete elliptic integral of the second kind.

Use `vpa` to approximate this result with floating-point numbers:

```
vpa(s, 10)

ans =
[ 0.5876852228, 1.285032276, 0, 0.7507322117]
```

#### Differentiate Incomplete Elliptic Integrals of Third Kind

Differentiate these expressions involving the complete elliptic integral of the third kind:

```
syms n m
diff(ellipticPi(n, m), n)
diff(ellipticPi(n, m), m)
```

```
ans =
ellipticK(m)/(2*n*(n - 1)) + ellipticE(m)/(2*(m - n)*(n - 1)) - ...
(ellipticPi(n, m)*(- n^2 + m))/(2*n*(m - n)*(n - 1))
```

```
ans =
- ellipticPi(n, m)/(2*(m - n)) - ellipticE(m)/(2*(m - n)*(m - 1))
```

Here, `ellipticK` and `ellipticE` represent the complete elliptic integrals of the first and second kinds.

### Compute Integrals for Matrix Input

Call `ellipticPi` for the scalar and the matrix. When one input argument is a matrix, `ellipticPi` expands the scalar argument to a matrix of the same size with all its elements equal to the scalar.

```
ellipticPi(sym(0), sym([1/3 1; 1/2 0]))
```

```
ans =
[ ellipticK(1/3), Inf]
[ ellipticK(1/2), pi/2]
```

Here, `ellipticK` represents the complete elliptic integral of the first kind.

## Input Arguments

### **n** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

### **m** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

### **phi** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## More About

### Incomplete Elliptic Integral of the Third Kind

The incomplete elliptic integral of the third kind is defined as follows:

$$\Pi(n; \varphi | m) = \int_0^{\varphi} \frac{1}{(1 - n \sin^2 \theta) \sqrt{1 - m \sin^2 \theta}} d\theta$$

Note that some definitions use the elliptical modulus  $k$  or the modular angle  $\alpha$  instead of the parameter  $m$ . They are related as  $m = k^2 = \sin^2\alpha$ .

### Complete Elliptic Integral of the Third Kind

The complete elliptic integral of the third kind is defined as follows:

$$\Pi(n, m) = \Pi\left(n; \frac{\pi}{2} \middle| m\right) = \int_0^{\pi/2} \frac{1}{(1 - n\sin^2\theta)\sqrt{1 - m\sin^2\theta}} d\theta$$

Note that some definitions use the elliptical modulus  $k$  or the modular angle  $\alpha$  instead of the parameter  $m$ . They are related as  $m = k^2 = \sin^2\alpha$ .

### Tips

- `ellipticPi` returns floating-point results for numeric arguments that are not symbolic objects.
- For most symbolic (exact) numbers, `ellipticPi` returns unresolved symbolic calls. You can approximate such results with floating-point numbers using `vpa`.
- All non-scalar arguments must have the same size. If one or two input arguments are non-scalar, then `ellipticPi` expands the scalars into vectors or matrices of the same size as the non-scalar arguments, with all elements equal to the corresponding scalar.
- `ellipticPi(n, pi/2, m) = ellipticPi(n, m)`.

## Version History

Introduced in R2013a

### References

- [1] Milne-Thomson, L. M. "Elliptic Integrals." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

### See Also

`ellipke` | `ellipticCE` | `ellipticCK` | `ellipticCPi` | `ellipticE` | `ellipticF` | `ellipticK` | `vpa`

## eq

Define symbolic equation

### Syntax

```
A == B
eq(A,B)
```

### Description

`A == B` defines a symbolic equation. Use the equation as input to functions such as `solve`, `assume`, `fcontour`, and `subs`.

`eq(A,B)` is equivalent to `A == B`.

### Examples

#### Define and Solve Equation

Solve this trigonometric equation. Define the equation by using the `==` operator.

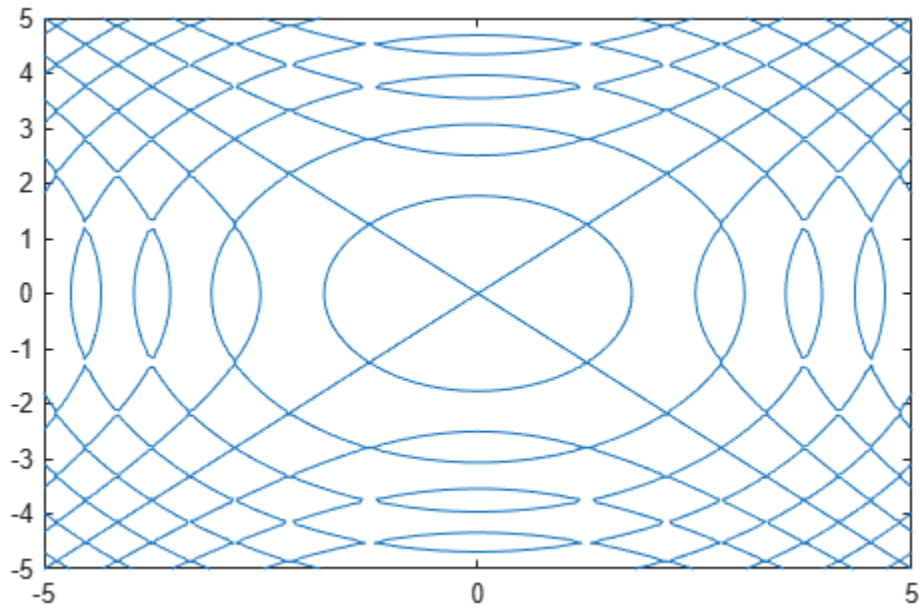
```
syms x
eqn = sin(x) == cos(x);
xsol = solve(eqn,x)
```

```
xsol =
    pi/4
```

#### Plot Symbolic Equation

Plot the equation  $\sin(x^2) = \sin(y^2)$  by using `fimplicit`. Define the equation by using the `==` operator.

```
syms x y
eqn = sin(x^2) == sin(y^2);
fimplicit(eqn)
```



### Test Equality of Symbolic Expressions

Test the equality of two symbolic expressions by using `isAlways`.

```
syms x
eqn = x + 1 == x + 1;
tf = isAlways(eqn)

tf = logical
     1

eqn = sin(x)/cos(x) == tan(x);
tf = isAlways(eqn)

tf = logical
     1
```

### Test Equality of Symbolic Matrices

Check the equality of two symbolic matrices by using `isAlways`.

```
A = sym(hilb(3));
B = sym([1 1/2 5; 1/2 2 1/4; 1/3 1/8 1/5]);
tf = isAlways(A == B)

tf = 3x3 logical array
```

```

1  1  0
1  0  1
1  0  1

```

Compare a matrix and a scalar. The `==` operator expands the scalar into a matrix of the same dimensions as the input matrix.

```

A = sym(hilb(3));
B = sym(1/2);
tf = isAlways(A == B)

tf = 3×3 logical array

```

```

0  1  0
1  0  0
0  0  0

```

## Input Arguments

### A — Input

number | vector | matrix | array | symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic array | symbolic function | symbolic matrix function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, scalar variable, matrix variable, array, function, matrix function, or expression.

### B — Input

number | vector | matrix | array | symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic array | symbolic function | symbolic matrix function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, scalar variable, matrix variable, array, function, matrix function, or expression.

## Tips

- Calling `==` or `eq` for nonsymbolic A and B invokes the MATLAB `eq` function. This MATLAB `eq` function returns a logical array with elements set to logical 1 (`true`) where A and B are equal; otherwise, elements of the returned array are logical 0 (`false`).
- If both A and B are arrays, then they must have the same dimensions. `A == B` returns an array of equations `A(i,j,...) == B(i,j,...)`.

## Version History

Introduced in R2012a

### R2026a: Symbolic functions must have matching arguments in equations

*Behavior changed in R2026a*

When you define a symbolic equation involving symbolic functions using the `eq (==)` operator, the symbolic functions on both sides of the equation must have the same function arguments. For

example, this code now returns an error because the left side of the symbolic equation is a symbolic function with `t` as its argument, while the right side contains a symbolic function with `a` and `t` as its arguments.

```
syms x(t) k(a,t)
eq = diff(x,t) == -k*t
```

```
Error using symfun/privResolveArgs (line 247)
Symbolic function input arguments must match.
```

```
Error in sym/privComparison (line 889)
      args = privResolveArgs(A, B);
            ~~~~~
Error in == (line 350)
      X = privComparison(A, B, '_equal', 'FALSE', 'FALSE');
          ~~~~~
```

In previous releases, the same code returned the equation `0 == 1`, which is always invalid and represents a contradiction.

```
syms x(t) k(a,t)
eq = diff(x,t) == -k*t
```

```
eq =
```

```
0 == 1
```

### **R2022a: Define equation of symbolic matrix functions**

The `eq` function accepts input arguments of type `symfunmatrix`.

### **R2021a: Define equation of symbolic matrix variables**

The `eq` function accepts input arguments of type `symmatrix`.

## **See Also**

`ge` | `gt` | `isAlways` | `le` | `lt` | `ne` | `solve`

## **Topics**

“Solve Equations” on page 1-31

“Use Assumptions on Symbolic Variables” on page 1-41



# equationsToMatrix

Convert linear equations to matrix form

## Syntax

```
[A,b] = equationsToMatrix(eqns)
[A,b] = equationsToMatrix(eqns,vars)
A = equationsToMatrix( ____ )
```

## Description

`[A,b] = equationsToMatrix(eqns)` converts equations `eqns` to matrix form. `eqns` must be a linear system of equations in all variables that `symvar` finds in `eqns`.

`[A,b] = equationsToMatrix(eqns,vars)` converts `eqns` to matrix form, where `eqns` must be linear in `vars`.

`A = equationsToMatrix( ____ )` returns only the coefficient matrix of the system of equations.

## Examples

### Convert Linear Equations to Matrix Form

Convert a system of linear equations to matrix form. `equationsToMatrix` automatically detects the variables in the equations by using `symvar`. The returned coefficient matrix follows the variable order determined by `symvar`.

```
syms x y z
eqns = [x+y-2*z == 0,
        x+y+z == 1,
        2*y-z == -5];
[A,b] = equationsToMatrix(eqns)
```

`A =`

$$\begin{pmatrix} 1 & 1 & -2 \\ 1 & 1 & 1 \\ 0 & 2 & -1 \end{pmatrix}$$

`b =`

$$\begin{pmatrix} 0 \\ 1 \\ -5 \end{pmatrix}$$

```
vars = symvar(eqns)
```

```
vars = (x y z)
```

You can change the arrangement of the coefficient matrix by specifying other variable order.

```
vars = [x,z,y];
[A,b] = equationsToMatrix(eqns,vars)
```

$$A = \begin{pmatrix} 1 & -2 & 1 \\ 1 & 1 & 1 \\ 0 & -1 & 2 \end{pmatrix}$$

$$b = \begin{pmatrix} 0 \\ 1 \\ -5 \end{pmatrix}$$

### Specify Variables in Equations

Convert a linear system of equations to the matrix form by specifying independent variables. This is useful when the equations are only linear in some variables.

For this system, specify the variables as `[s t]` because the system is not linear in `r`.

```
syms r s t
eqns = [s-2*t+r^2 == -1
        3*s-t == 10];
vars = [s t];
[A,b] = equationsToMatrix(eqns,vars)
```

$$A = \begin{pmatrix} 1 & -2 \\ 3 & -1 \end{pmatrix}$$

$$b = \begin{pmatrix} -r^2 - 1 \\ 10 \end{pmatrix}$$

### Return Only Coefficient Matrix of Equations

Return only the coefficient matrix of the equations by specifying a single output argument.

```
syms x y z
eqns = [x+y-2*z == 0,
        x+y+z == 1,
        2*y-z == -5];
vars = [x y z];
A = equationsToMatrix(eqns,vars)
```

$$A = \begin{pmatrix} 1 & 1 & -2 \\ 1 & 1 & 1 \\ 0 & 2 & -1 \end{pmatrix}$$

## Solve System of Equations That Are Functions of Time

Consider the following system of linear equations that are functions of time:

$$\begin{aligned} 2x(t) + y(t) + z(t) &= 2u(t) \\ -x(t) + y(t) - z(t) &= v(t) \\ x(t) + 2y(t) + 3z(t) &= -10 \end{aligned}$$

Declare the system of equations.

```
syms x(t) y(t) z(t) u(t) v(t)
eqn1 = 2*x + y + z == 2*u;
eqn2 = -x + y - z == v;
eqn3 = x + 2*y + 3*z == -10;
eqn = [eqn1; eqn2; eqn3]
```

$$\text{eqn}(t) = \begin{pmatrix} 2x(t) + y(t) + z(t) = 2u(t) \\ y(t) - x(t) - z(t) = v(t) \\ x(t) + 2y(t) + 3z(t) = -10 \end{pmatrix}$$

Specify the independent variables  $x(t)$ ,  $y(t)$ , and  $z(t)$  in the equations as a symbolic vector `vars`. Use the `equationsToMatrix` function to convert the system of equations into the matrix form.

```
vars = [x(t); y(t); z(t)];
[A,b] = equationsToMatrix(eqn,vars)
```

$$A = \begin{pmatrix} 2 & 1 & 1 \\ -1 & 1 & -1 \\ 1 & 2 & 3 \end{pmatrix}$$

$$b = \begin{pmatrix} 2u(t) \\ v(t) \\ -10 \end{pmatrix}$$

Solve the matrix form of the equations using the `linsolve` function.

```
X = linsolve(A,b)
```

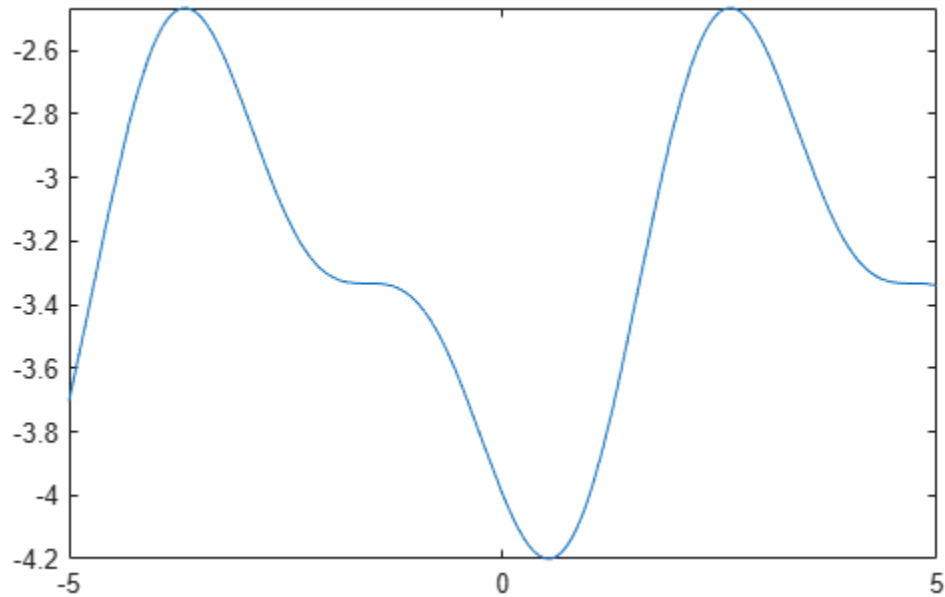
$$X = \begin{pmatrix} \frac{10u(t)}{9} - \frac{v(t)}{9} + \frac{20}{9} \\ \frac{4u(t)}{9} + \frac{5v(t)}{9} - \frac{10}{9} \\ -\frac{2u(t)}{3} - \frac{v(t)}{3} - \frac{10}{3} \end{pmatrix}$$

Evaluate the  $z(t)$  solution for the functions  $u(t) = \cos(t)$  and  $v(t) = \sin(2t)$ . Plot the  $z(t)$  solution.

```
zSol = subs(X(3),[u(t) v(t)],[cos(t) sin(2*t)])
```

$$zSol = -\frac{\sin(2t)}{3} - \frac{2\cos(t)}{3} - \frac{10}{3}$$

fplot(zSol)



## Input Arguments

### eqns — Linear equations

vector of symbolic equations or expressions

Linear equations, specified as a vector of symbolic equations or expressions. Symbolic equations are defined by using the `==` operator, such as  $x + y == 1$ . For symbolic expressions, `equationsToMatrix` assumes that the right side is 0.

Equations must be linear in terms of `vars`.

### vars — Independent variables

vector of symbolic variables (default) | vector of symbolic functions

Independent variables in `eqns`, specified as a vector of symbolic variables or symbolic functions.

## Output Arguments

### A — Coefficient matrix

symbolic matrix

Coefficient matrix of the system of linear equations, specified as a symbolic matrix.

### b — Right sides of equations

symbolic matrix

Vector containing the right sides of equations, specified as a symbolic matrix.

## More About

### Matrix Representation of System of Linear Equations

A system of linear equations

$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1$$

$$a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2$$

...

$$a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n = b_m$$

can be represented as the matrix equation  $A \cdot \vec{x} = \vec{b}$ . Here,  $A$  is the coefficient matrix.

$$A = \begin{pmatrix} a_{11} & \dots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \dots & a_{mn} \end{pmatrix}$$

$\vec{b}$  is the vector containing the right sides of equations.

$$\vec{b} = \begin{pmatrix} b_1 \\ \vdots \\ b_m \end{pmatrix}$$

## Version History

Introduced in R2012b

### See Also

`linsolve` | `odeToVectorField` | `solve` | `symvar`

### Topics

"Solve System of Linear Equations" on page 3-36

## erf

Error function

### Syntax

`erf(X)`

### Description

`erf(X)` represents the error function on page 6-385 of X. If X is a vector or a matrix, `erf(X)` computes the error function of each element of X.

### Examples

#### Error Function for Floating-Point and Symbolic Numbers

Depending on its arguments, `erf` can return floating-point or exact symbolic results.

Compute the error function for these numbers. Because these numbers are not symbolic objects, you get the floating-point results:

```
A = [erf(1/2), erf(1.41), erf(sqrt(2))]
```

```
A =  
    0.5205    0.9539    0.9545
```

Compute the error function for the same numbers converted to symbolic objects. For most symbolic (exact) numbers, `erf` returns unresolved symbolic calls:

```
symA = [erf(sym(1/2)), erf(sym(1.41)), erf(sqrt(sym(2)))]
```

```
symA =  
[ erf(1/2), erf(141/100), erf(2^(1/2))]
```

Use `vpa` to approximate symbolic results with the required number of digits:

```
d = digits(10);  
vpa(symA)  
digits(d)  
  
ans =  
[ 0.5204998778, 0.9538524394, 0.9544997361]
```

#### Error Function for Variables and Expressions

For most symbolic variables and expressions, `erf` returns unresolved symbolic calls.

Compute the error function for  $x$  and  $\sin(x) + x \cdot \exp(x)$ :

```
syms x  
f = sin(x) + x*exp(x);  
erf(x)  
erf(f)
```

```
ans =
erf(x)

ans =
erf(sin(x) + x*exp(x))
```

### Error Function for Vectors and Matrices

If the input argument is a vector or a matrix, `erf` returns the error function for each element of that vector or matrix.

Compute the error function for elements of matrix `M` and vector `V`:

```
M = sym([0 inf; 1/3 -inf]);
V = sym([1; -i*inf]);
erf(M)
erf(V)
```

```
ans =
[ 0, 1]
[ erf(1/3), -1]
```

```
ans =
erf(1)
-Inf*1i
```

### Special Values of Error Function

`erf` returns special values for particular parameters.

Compute the error function for  $x = 0$ ,  $x = \infty$ , and  $x = -\infty$ . Use `sym` to convert `0` and infinities to symbolic objects. The error function has special values for these parameters:

```
[erf(sym(0)), erf(sym(Inf)), erf(sym(-Inf))]
```

```
ans =
[ 0, 1, -1]
```

Compute the error function for complex infinities. Use `sym` to convert complex infinities to symbolic objects:

```
[erf(sym(i*Inf)), erf(sym(-i*Inf))]
```

```
ans =
[ Inf*1i, -Inf*1i]
```

### Handling Expressions That Contain Error Function

Many functions, such as `diff` and `int`, can handle expressions containing `erf`.

Compute the first and second derivatives of the error function:

```
syms x
diff(erf(x), x)
diff(erf(x), x, 2)

ans =
(2*exp(-x^2))/pi^(1/2)
```

```
ans =
-(4*x*exp(-x^2))/pi^(1/2)
```

Compute the integrals of these expressions:

```
int(erf(x), x)
int(erf(log(x)), x)
```

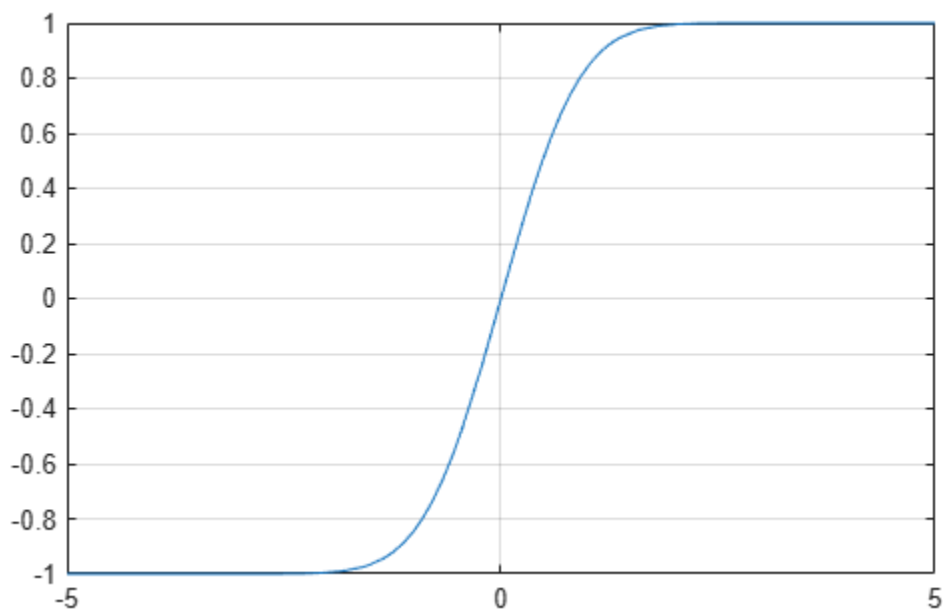
```
ans =
exp(-x^2)/pi^(1/2) + x*erf(x)
```

```
ans =
x*erf(log(x)) - int((2*exp(-log(x)^2))/pi^(1/2), x)
```

### Plot Error Function

Plot the error function on the interval from -5 to 5.

```
syms x
fplot(erf(x), [-5 5])
grid on
```



### Input Arguments

#### X — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.



## More About

### Error Function

The following integral defines the error function:

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$$

### Tips

- Calling `erf` for a number that is not a symbolic object invokes the MATLAB `erf` function. This function accepts real arguments only. If you want to compute the error function for a complex number, use `sym` to convert that number to a symbolic object, and then call `erf` for that symbolic object.
- For most symbolic (exact) numbers, `erf` returns unresolved symbolic calls. You can approximate such results with floating-point numbers using `vpa`.

### Algorithms

The toolbox can simplify expressions that contain error functions and their inverses. For real values  $x$ , the toolbox applies these simplification rules:

- $\operatorname{erfinv}(\operatorname{erf}(x)) = \operatorname{erfinv}(1 - \operatorname{erfc}(x)) = \operatorname{erfcinv}(1 - \operatorname{erf}(x)) = \operatorname{erfcinv}(\operatorname{erfc}(x)) = x$
- $\operatorname{erfinv}(-\operatorname{erf}(x)) = \operatorname{erfinv}(\operatorname{erfc}(x) - 1) = \operatorname{erfcinv}(1 + \operatorname{erf}(x)) = \operatorname{erfcinv}(2 - \operatorname{erfc}(x)) = -x$

For any value  $x$ , the system applies these simplification rules:

- $\operatorname{erfcinv}(x) = \operatorname{erfinv}(1 - x)$
- $\operatorname{erfinv}(-x) = -\operatorname{erfinv}(x)$
- $\operatorname{erfcinv}(2 - x) = -\operatorname{erfcinv}(x)$
- $\operatorname{erf}(\operatorname{erfinv}(x)) = \operatorname{erfc}(\operatorname{erfcinv}(x)) = x$
- $\operatorname{erf}(\operatorname{erfcinv}(x)) = \operatorname{erfc}(\operatorname{erfinv}(x)) = 1 - x$

## Version History

Introduced before R2006a

### References

- [1] Gautschi, W. "Error Function and Fresnel Integrals." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

**See Also**

`erfc` | `erfcinv` | `erfi` | `erfinv`

# erfc

Complementary error function

## Syntax

```
erfc(X)
erfc(K,X)
```

## Description

`erfc(X)` represents the complementary error function on page 6-390 of X, that is,  $\text{erfc}(X) = 1 - \text{erf}(X)$ .

`erfc(K,X)` represents the iterated integral of the complementary error function on page 6-391 of X, that is,  $\text{erfc}(K, X) = \int(\text{erfc}(K - 1, y), y, X, \infty)$ .

## Examples

### Complementary Error Function for Floating-Point and Symbolic Numbers

Depending on its arguments, `erfc` can return floating-point or exact symbolic results.

Compute the complementary error function for these numbers. Because these numbers are not symbolic objects, you get the floating-point results:

```
A = [erfc(1/2), erfc(1.41), erfc(sqrt(2))]
```

A =

```
    0.4795    0.0461    0.0455
```

Compute the complementary error function for the same numbers converted to symbolic objects. For most symbolic (exact) numbers, `erfc` returns unresolved symbolic calls:

```
symA = [erfc(sym(1/2)), erfc(sym(1.41)), erfc(sqrt(sym(2)))]
```

symA =

```
[ erfc(1/2), erfc(141/100), erfc(2^(1/2))]
```

Use `vpa` to approximate symbolic results with the required number of digits:

```
d = digits(10);
vpa(symA)
digits(d)
```

ans =

```
[ 0.4795001222, 0.04614756064, 0.0455002639]
```

### Error Function for Variables and Expressions

For most symbolic variables and expressions, `erfc` returns unresolved symbolic calls.

Compute the complementary error function for `x` and  $\sin(x) + x \cdot \exp(x)$ :

```
syms x
f = sin(x) + x*exp(x);
erfc(x)
erfc(f)

ans =
erfc(x)

ans =
erfc(sin(x) + x*exp(x))
```

### Complementary Error Function for Vectors and Matrices

If the input argument is a vector or a matrix, `erfc` returns the complementary error function for each element of that vector or matrix.

Compute the complementary error function for elements of matrix `M` and vector `V`:

```
M = sym([0 inf; 1/3 -inf]);
V = sym([1; -i*inf]);
erfc(M)
erfc(V)

ans =
[ 1, 0]
[ erfc(1/3), 2]

ans =
erfc(1)
1 + Inf*Ii
```

Compute the iterated integral of the complementary error function for the elements of `V` and `M`, and the integer `-1`:

```
erfc(-1, M)
erfc(-1, V)

ans =
[ 2/pi^(1/2), 0]
[ (2*exp(-1/9))/pi^(1/2), 0]

ans =
(2*exp(-1))/pi^(1/2)
Inf
```

### Special Values of Complementary Error Function

`erfc` returns special values for particular parameters.

Compute the complementary error function for  $x = 0$ ,  $x = \infty$ , and  $x = -\infty$ . The complementary error function has special values for these parameters:

```
[erfc(0), erfc(Inf), erfc(-Inf)]

ans =
1 0 2
```

Compute the complementary error function for complex infinities. Use `sym` to convert complex infinities to symbolic objects:

```
[erfc(sym(i*Inf)), erfc(sym(-i*Inf))]
```

```
ans =  
[ 1 - Inf*1i, 1 + Inf*1i]
```

### Handling Expressions That Contain Complementary Error Function

Many functions, such as `diff` and `int`, can handle expressions containing `erfc`.

Compute the first and second derivatives of the complementary error function:

```
syms x  
diff(erfc(x), x)  
diff(erfc(x), x, 2)  
  
ans =  
-(2*exp(-x^2))/pi^(1/2)
```

```
ans =  
(4*x*exp(-x^2))/pi^(1/2)
```

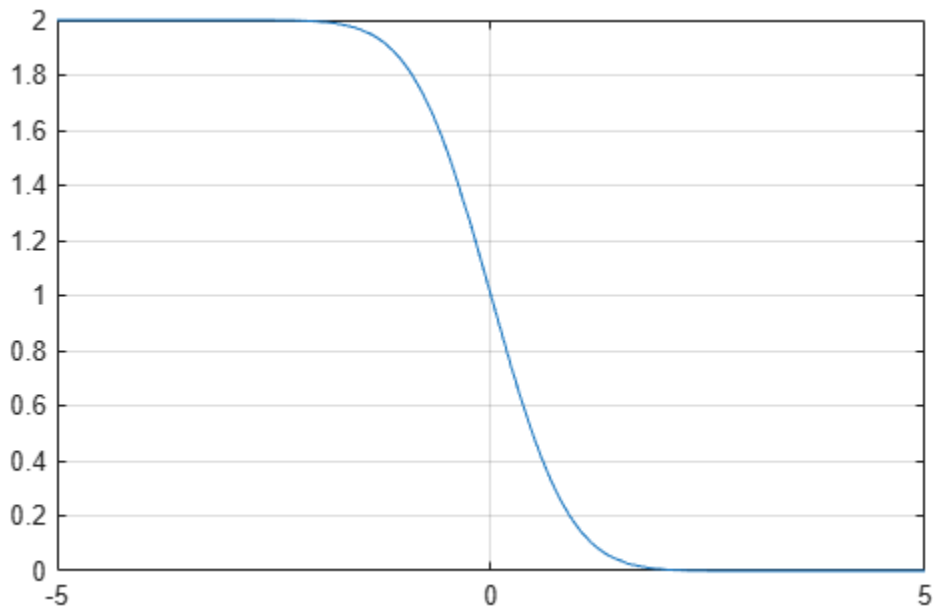
Compute the integrals of these expressions:

```
syms x  
int(erfc(-1, x), x)  
  
ans =  
erf(x)  
  
int(erfc(x), x)  
  
ans =  
x*erfc(x) - exp(-x^2)/pi^(1/2)  
  
int(erfc(2, x), x)  
  
ans =  
(x^3*erfc(x))/6 - exp(-x^2)/(6*pi^(1/2)) + ...  
(x*erfc(x))/4 - (x^2*exp(-x^2))/(6*pi^(1/2))
```

### Plot Complementary Error Function

Plot the complementary error function on the interval from -5 to 5.

```
syms x  
fplot(erfc(x), [-5 5])  
grid on
```



## Input Arguments

### X — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

### K — Input representing an integer larger than -2

number | symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input representing an integer larger than -2, specified as a number, symbolic number, variable, expression, or function. This arguments can also be a vector or matrix of numbers, symbolic numbers, variables, expressions, or functions.

## More About

### Complementary Error Function

The following integral defines the complementary error function:

$$\text{erfc}(x) = \frac{2}{\sqrt{\pi}} \int_x^{\infty} e^{-t^2} dt = 1 - \text{erf}(x)$$

Here  $\text{erf}(x)$  is the error function.

## Iterated Integral of Complementary Error Function

The following integral is the iterated integral of the complementary error function:

$$\operatorname{erfc}(k, x) = \int_x^{\infty} \operatorname{erfc}(k - 1, y) dy$$

Here,  $\operatorname{erfc}(0, x) = \operatorname{erfc}(x)$ .

## Tips

- Calling `erfc` for a number that is not a symbolic object invokes the MATLAB `erfc` function. This function accepts real arguments only. If you want to compute the complementary error function for a complex number, use `sym` to convert that number to a symbolic object, and then call `erfc` for that symbolic object.
- For most symbolic (exact) numbers, `erfc` returns unresolved symbolic calls. You can approximate such results with floating-point numbers using `vpa`.
- At least one input argument must be a scalar or both arguments must be vectors or matrices of the same size. If one input argument is a scalar and the other one is a vector or a matrix, then `erfc` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

## Algorithms

The toolbox can simplify expressions that contain error functions and their inverses. For real values  $x$ , the toolbox applies these simplification rules:

- $\operatorname{erfinv}(\operatorname{erf}(x)) = \operatorname{erfinv}(1 - \operatorname{erfc}(x)) = \operatorname{erfcinv}(1 - \operatorname{erf}(x)) = \operatorname{erfcinv}(\operatorname{erfc}(x)) = x$
- $\operatorname{erfinv}(-\operatorname{erf}(x)) = \operatorname{erfinv}(\operatorname{erfc}(x) - 1) = \operatorname{erfcinv}(1 + \operatorname{erf}(x)) = \operatorname{erfcinv}(2 - \operatorname{erfc}(x)) = -x$

For any value  $x$ , the system applies these simplification rules:

- $\operatorname{erfcinv}(x) = \operatorname{erfinv}(1 - x)$
- $\operatorname{erfinv}(-x) = -\operatorname{erfinv}(x)$
- $\operatorname{erfcinv}(2 - x) = -\operatorname{erfcinv}(x)$
- $\operatorname{erf}(\operatorname{erfinv}(x)) = \operatorname{erfc}(\operatorname{erfcinv}(x)) = x$
- $\operatorname{erf}(\operatorname{erfcinv}(x)) = \operatorname{erfc}(\operatorname{erfinv}(x)) = 1 - x$

## Version History

Introduced in R2011b

## References

- [1] Gautschi, W. "Error Function and Fresnel Integrals." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

**See Also**

erf | erfcinv | erfi | erfinv



# erfcinv

Inverse complementary error function

## Syntax

`erfcinv(X)`

## Description

`erfcinv(X)` computes the inverse complementary error function on page 6-395 of X. If X is a vector or a matrix, `erfcinv(X)` computes the inverse complementary error function of each element of X.

## Examples

### Inverse Complementary Error Function for Floating-Point and Symbolic Numbers

Depending on its arguments, `erfcinv` can return floating-point or exact symbolic results.

Compute the inverse complementary error function for these numbers. Because these numbers are not symbolic objects, you get floating-point results:

```
A = [erfcinv(1/2), erfcinv(1.33), erfcinv(3/2)]
```

```
A =
    0.4769    -0.3013    -0.4769
```

Compute the inverse complementary error function for the same numbers converted to symbolic objects. For most symbolic (exact) numbers, `erfcinv` returns unresolved symbolic calls:

```
symA = [erfcinv(sym(1/2)), erfcinv(sym(1.33)), erfcinv(sym(3/2))]
```

```
symA =
[ -erfcinv(3/2), erfcinv(133/100), erfcinv(3/2)]
```

Use `vpa` to approximate symbolic results with the required number of digits:

```
d = digits(10);
vpa(symA)
digits(d)
```

```
ans =
[ 0.4769362762, -0.3013321461, -0.4769362762]
```

### Inverse Complementary Error Function for Variables and Expressions

For most symbolic variables and expressions, `erfcinv` returns unresolved symbolic calls.

Compute the inverse complementary error function for  $x$  and  $\sin(x) + x \cdot \exp(x)$ . For most symbolic variables and expressions, `erfcinv` returns unresolved symbolic calls:

```
syms x
f = sin(x) + x*exp(x);
```

```
erfcinv(x)
erfcinv(f)

ans =
erfcinv(x)

ans =
erfcinv(sin(x) + x*exp(x))
```

### Inverse Complementary Error Function for Vectors and Matrices

If the input argument is a vector or a matrix, `erfcinv` returns the inverse complementary error function for each element of that vector or matrix.

Compute the inverse complementary error function for elements of matrix `M` and vector `V`:

```
M = sym([0 1 + i; 1/3 1]);
V = sym([2; inf]);
erfcinv(M)
erfcinv(V)

ans =
[          Inf, NaN]
[ -erfcinv(5/3),  0]

ans =
-Inf
NaN
```

### Special Values of Inverse Complementary Error Function

`erfcinv` returns special values for particular parameters.

Compute the inverse complementary error function for  $x = 0$ ,  $x = 1$ , and  $x = 2$ . The inverse complementary error function has special values for these parameters:

```
[erfcinv(0), erfcinv(1), erfcinv(2)]

ans =
Inf      0  -Inf
```

### Handling Expressions That Contain Inverse Complementary Error Function

Many functions, such as `diff` and `int`, can handle expressions containing `erfcinv`.

Compute the first and second derivatives of the inverse complementary error function:

```
syms x
diff(erfcinv(x), x)
diff(erfcinv(x), x, 2)

ans =
-(pi^(1/2)*exp(erfcinv(x)^2))/2

ans =
(pi*exp(2*erfcinv(x)^2)*erfcinv(x))/2
```

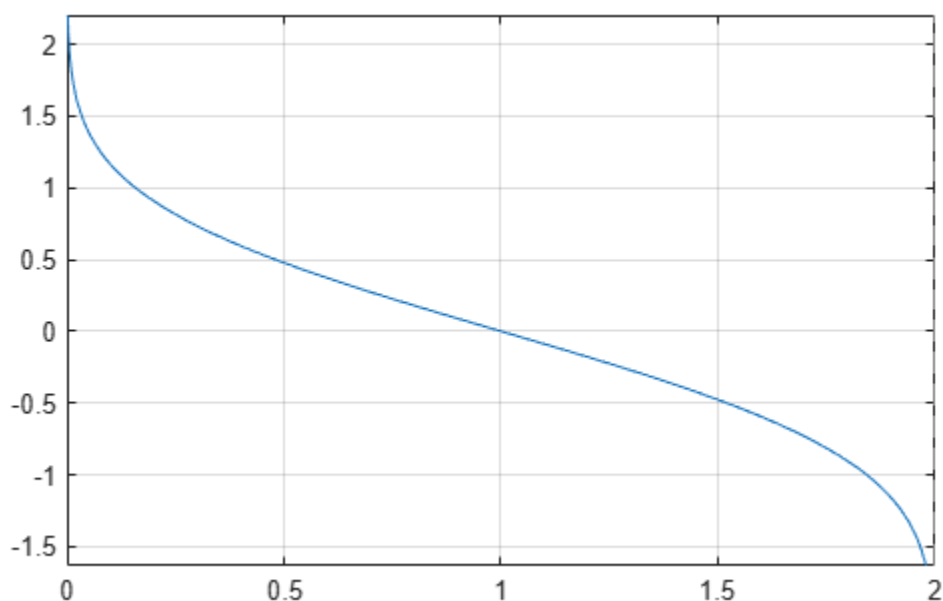
Compute the integral of the inverse complementary error function:

```
int(erfcinv(x), x)
ans =
exp(-erfcinv(x)^2)/pi^(1/2)
```

### Plot Inverse Complementary Error Function

Plot the inverse complementary error function on the interval from 0 to 2.

```
syms x
fplot(erfcinv(x),[0 2])
grid on
```



## Input Arguments

### X — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## More About

### Inverse Complementary Error Function

The inverse complementary error function is defined as  $\text{erfc}^{-1}(x)$ , such that  $\text{erfc}(\text{erfc}^{-1}(x)) = x$ . Here

$$\text{erfc}(x) = \frac{2}{\sqrt{\pi}} \int_x^{\infty} e^{-t^2} dt = 1 - \text{erf}(x)$$

is the complementary error function.

## Tips

- Calling `erfcinv` for a number that is not a symbolic object invokes the MATLAB `erfcinv` function. This function accepts real arguments only. If you want to compute the inverse complementary error function for a complex number, use `sym` to convert that number to a symbolic object, and then call `erfcinv` for that symbolic object.
- If  $x < 0$  or  $x > 2$ , or if  $x$  is complex, then `erfcinv(x)` returns NaN.

## Algorithms

The toolbox can simplify expressions that contain error functions and their inverses. For real values  $x$ , the toolbox applies these simplification rules:

- $\text{erfinv}(\text{erf}(x)) = \text{erfinv}(1 - \text{erfc}(x)) = \text{erfcinv}(1 - \text{erf}(x)) = \text{erfcinv}(\text{erfc}(x)) = x$
- $\text{erfinv}(-\text{erf}(x)) = \text{erfinv}(\text{erfc}(x) - 1) = \text{erfcinv}(1 + \text{erf}(x)) = \text{erfcinv}(2 - \text{erfc}(x)) = -x$

For any value  $x$ , the toolbox applies these simplification rules:

- $\text{erfcinv}(x) = \text{erfinv}(1 - x)$
- $\text{erfinv}(-x) = -\text{erfinv}(x)$
- $\text{erfcinv}(2 - x) = -\text{erfcinv}(x)$
- $\text{erf}(\text{erfinv}(x)) = \text{erfc}(\text{erfcinv}(x)) = x$
- $\text{erf}(\text{erfcinv}(x)) = \text{erfc}(\text{erfinv}(x)) = 1 - x$

## Version History

Introduced in R2012a

## References

- [1] Gautschi, W. "Error Function and Fresnel Integrals." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

`erf` | `erfc` | `erfi` | `erfinv`

# erfi

Imaginary error function

## Syntax

`erfi(x)`

## Description

`erfi(x)` returns the imaginary error function on page 6-400 of  $x$ . If  $x$  is a vector or a matrix, `erfi(x)` returns the imaginary error function of each element of  $x$ .

## Examples

### Imaginary Error Function for Floating-Point and Symbolic Numbers

Depending on its arguments, `erfi` can return floating-point or exact symbolic results.

Compute the imaginary error function for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
s = [erfi(1/2), erfi(1.41), erfi(sqrt(2))]  
  
s =  
    0.6150    3.7382    3.7731
```

Compute the imaginary error function for the same numbers converted to symbolic objects. For most symbolic (exact) numbers, `erfi` returns unresolved symbolic calls.

```
s = [erfi(sym(1/2)), erfi(sym(1.41)), erfi(sqrt(sym(2)))]  
  
s =  
[ erfi(1/2), erfi(141/100), erfi(2^(1/2))]
```

Use `vpa` to approximate this result with the 10-digit accuracy:

```
vpa(s, 10)  
  
ans =  
[ 0.6149520947, 3.738199581, 3.773122512]
```

### Imaginary Error Function for Variables and Expressions

Compute the imaginary error function for  $x$  and  $\sin(x) + x \cdot \exp(x)$ . For most symbolic variables and expressions, `erfi` returns unresolved symbolic calls.

```
syms x  
f = sin(x) + x*exp(x);  
erfi(x)  
erfi(f)  
  
ans =  
erfi(x)
```

```
ans =  
erfi(sin(x) + x*exp(x))
```

### Imaginary Error Function for Vectors and Matrices

If the input argument is a vector or a matrix, `erfi` returns the imaginary error function for each element of that vector or matrix.

Compute the imaginary error function for elements of matrix `M` and vector `V`:

```
M = sym([0 inf; 1/3 -inf]);  
V = sym([1; -i*inf]);  
erfi(M)  
erfi(V)
```

```
ans =  
[ 0, Inf]  
[ erfi(1/3), -Inf]
```

```
ans =  
erfi(1)  
-1i
```

### Special Values of Imaginary Error Function

Compute the imaginary error function for  $x = 0$ ,  $x = \infty$ , and  $x = -\infty$ . Use `sym` to convert `0` and infinities to symbolic objects. The imaginary error function has special values for these parameters:

```
[erfi(sym(0)), erfi(sym(inf)), erfi(sym(-inf))]
```

```
ans =  
[ 0, Inf, -Inf]
```

Compute the imaginary error function for complex infinities. Use `sym` to convert complex infinities to symbolic objects:

```
[erfi(sym(i*inf)), erfi(sym(-i*inf))]
```

```
ans =  
[ 1i, -1i]
```

### Handling Expressions That Contain Imaginary Error Function

Many functions, such as `diff` and `int`, can handle expressions containing `erfi`.

Compute the first and second derivatives of the imaginary error function:

```
syms x  
diff(erfi(x), x)  
diff(erfi(x), x, 2)
```

```
ans =  
(2*exp(x^2))/pi^(1/2)
```

```
ans =  
(4*x*exp(x^2))/pi^(1/2)
```

Compute the integrals of these expressions:

```

int(erfi(x), x)
int(erfi(log(x)), x)

ans =
x*erfi(x) - exp(x^2)/pi^(1/2)

ans =
x*erfi(log(x)) - int((2*exp(log(x)^2))/pi^(1/2), x)

```

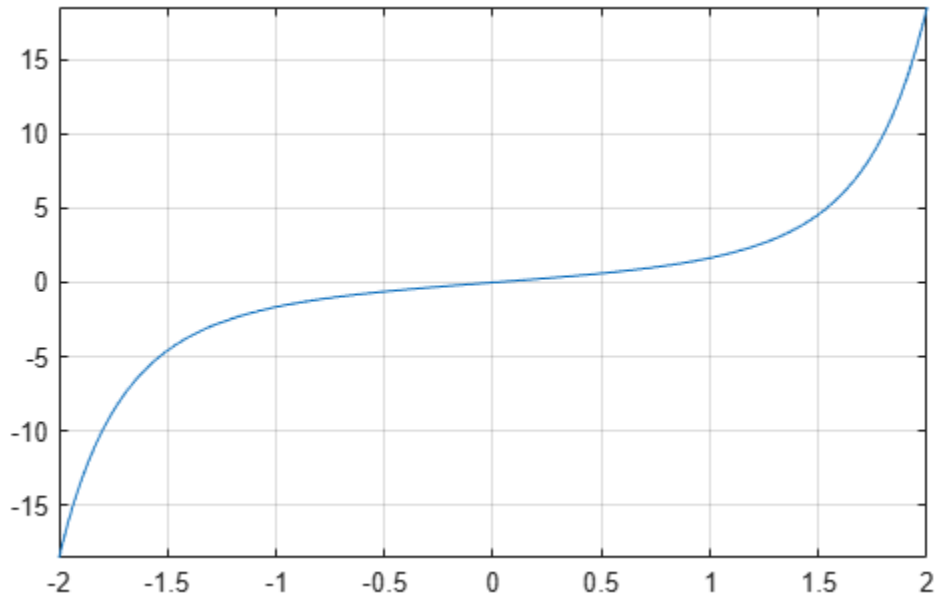
### Plot Imaginary Error Function

Plot the imaginary error function on the interval from -2 to 2.

```

syms x
fplot(erfi(x), [-2,2])
grid on

```



### Input Arguments

#### **x — Input**

floating-point number | symbolic number | symbolic variable | symbolic expression | symbolic function  
| symbolic vector | symbolic matrix

Input, specified as a floating-point or symbolic number, variable, expression, function, vector, or matrix.

## More About

### Imaginary Error Function

The imaginary error function is defined as:

$$\operatorname{erfi}(x) = -i \operatorname{erf}(ix) = \frac{2}{\sqrt{\pi}} \int_0^x e^{t^2} dt$$

### Tips

- `erfi` returns special values for these parameters:
  - `erfi(0) = 0`
  - `erfi(Inf) = Inf`
  - `erfi(-Inf) = -Inf`
  - `erfi(i*Inf) = i`
  - `erfi(-i*Inf) = -i`

## Version History

Introduced in R2013a

### See Also

`erf` | `erfc` | `erfcinv` | `erfinv` | `vpa`



# erfinv

Inverse error function

## Syntax

`erfinv(X)`

## Description

`erfinv(X)` computes the inverse error function on page 6-403 of *X*. If *X* is a vector or a matrix, `erfinv(X)` computes the inverse error function of each element of *X*.

## Examples

### Inverse Error Function for Floating-Point and Symbolic Numbers

Depending on its arguments, `erfinv` can return floating-point or exact symbolic results.

Compute the inverse error function for these numbers. Because these numbers are not symbolic objects, you get floating-point results:

```
A = [erfinv(1/2), erfinv(0.33), erfinv(-1/3)]
```

```
A =
    0.4769    0.3013   -0.3046
```

Compute the inverse error function for the same numbers converted to symbolic objects. For most symbolic (exact) numbers, `erfinv` returns unresolved symbolic calls:

```
symA = [erfinv(sym(1)/2), erfinv(sym(0.33)), erfinv(sym(-1)/3)]
```

```
symA =
[ erfinv(1/2), erfinv(33/100), -erfinv(1/3)]
```

Use `vpa` to approximate symbolic results with the required number of digits:

```
d = digits(10);
vpa(symA)
digits(d)

ans =
[ 0.4769362762, 0.3013321461, -0.3045701942]
```

### Inverse Error Function for Variables and Expressions

For most symbolic variables and expressions, `erfinv` returns unresolved symbolic calls.

Compute the inverse error function for *x* and  $\sin(x) + x \cdot \exp(x)$ . For most symbolic variables and expressions, `erfinv` returns unresolved symbolic calls:

```
syms x
f = sin(x) + x*exp(x);
```

```
erfinv(x)
erfinv(f)

ans =
erfinv(x)

ans =
erfinv(sin(x) + x*exp(x))
```

### Inverse Error Function for Vectors and Matrices

If the input argument is a vector or a matrix, `erfinv` returns the inverse error function for each element of that vector or matrix.

Compute the inverse error function for elements of matrix `M` and vector `V`:

```
M = sym([0 1 + i; 1/3 1]);
V = sym([-1; inf]);
erfinv(M)
erfinv(V)

ans =
[ 0, NaN]
[ erfinv(1/3), Inf]

ans =
-Inf
NaN
```

### Special Values of Inverse Complementary Error Function

`erfinv` returns special values for particular parameters.

Compute the inverse error function for  $x = -1$ ,  $x = 0$ , and  $x = 1$ . The inverse error function has special values for these parameters:

```
[erfinv(-1), erfinv(0), erfinv(1)]

ans =
-Inf 0 Inf
```

### Handling Expressions That Contain Inverse Complementary Error Function

Many functions, such as `diff` and `int`, can handle expressions containing `erfinv`.

Compute the first and second derivatives of the inverse error function:

```
syms x
diff(erfinv(x), x)
diff(erfinv(x), x, 2)

ans =
(pi^(1/2)*exp(erfinv(x)^2))/2

ans =
(pi*exp(2*erfinv(x)^2)*erfinv(x))/2
```

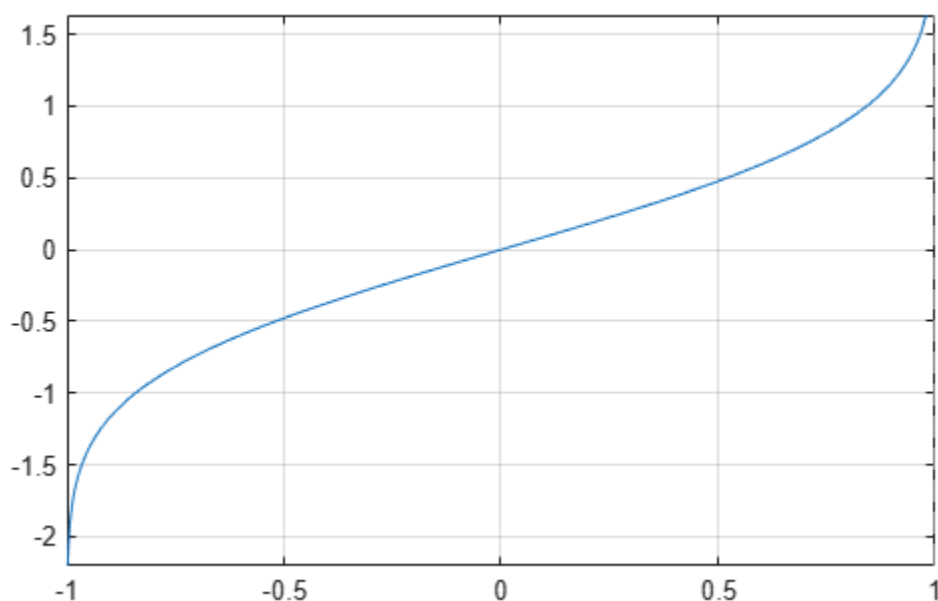
Compute the integral of the inverse error function:

```
int(erfinv(x), x)
ans =
-exp(-erfinv(x)^2)/pi^(1/2)
```

### Plot Inverse Error Function

Plot the inverse error function on the interval from -1 to 1.

```
syms x
fplot(erfinv(x),[-1,1])
grid on
```



## Input Arguments

### X — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## More About

### Inverse Error Function

The inverse error function is defined as  $\text{erf}^{-1}(x)$ , such that  $\text{erf}(\text{erf}^{-1}(x)) = \text{erf}^{-1}(\text{erf}(x)) = x$ . Here

$$\text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$$

is the error function.

## Tips

- Calling `erfinv` for a number that is not a symbolic object invokes the MATLAB `erfinv` function. This function accepts real arguments only. If you want to compute the inverse error function for a complex number, use `sym` to convert that number to a symbolic object, and then call `erfinv` for that symbolic object.
- If  $x < -1$  or  $x > 1$ , or if  $x$  is complex, then `erfinv(x)` returns NaN.

## Algorithms

The toolbox can simplify expressions that contain error functions and their inverses. For real values  $x$ , the toolbox applies these simplification rules:

- $\text{erfinv}(\text{erf}(x)) = \text{erfinv}(1 - \text{erfc}(x)) = \text{erfcinv}(1 - \text{erf}(x)) = \text{erfcinv}(\text{erfc}(x)) = x$
- $\text{erfinv}(-\text{erf}(x)) = \text{erfinv}(\text{erfc}(x) - 1) = \text{erfcinv}(1 + \text{erf}(x)) = \text{erfcinv}(2 - \text{erfc}(x)) = -x$

For any value  $x$ , the toolbox applies these simplification rules:

- $\text{erfcinv}(x) = \text{erfinv}(1 - x)$
- $\text{erfinv}(-x) = -\text{erfinv}(x)$
- $\text{erfcinv}(2 - x) = -\text{erfcinv}(x)$
- $\text{erf}(\text{erfinv}(x)) = \text{erfc}(\text{erfcinv}(x)) = x$
- $\text{erf}(\text{erfcinv}(x)) = \text{erfc}(\text{erfinv}(x)) = 1 - x$

## Version History

Introduced in R2012a

## References

- [1] Gautschi, W. "Error Function and Fresnel Integrals." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

`erf` | `erfc` | `erfcinv` | `erfi`

# euler

Euler numbers and polynomials

## Syntax

```
euler(n)
euler(n,x)
```

## Description

`euler(n)` returns the  $n$ th Euler number on page 6-408.

`euler(n,x)` returns the  $n$ th Euler polynomial on page 6-407.

## Examples

### Euler Numbers with Odd and Even Indices

The Euler numbers with even indices alternate the signs. Any Euler number with an odd index is 0.

Compute the even-indexed Euler numbers with the indices from 0 to 10:

```
euler(0:2:10)
```

```
ans =
      1      -1      5     -61...
    1385   -50521
```

Compute the odd-indexed Euler numbers with the indices from 1 to 11:

```
euler(1:2:11)
```

```
ans =
      0      0      0      0      0      0
```

### Euler Polynomials

For the Euler polynomials, use `euler` with two input arguments.

Compute the first, second, and third Euler polynomials in variables  $x$ ,  $y$ , and  $z$ , respectively:

```
syms x y z
euler(1, x)
euler(2, y)
euler(3, z)
```

```
ans =
x - 1/2
```

```
ans =
y^2 - y
```

```
ans =
z^3 - (3*z^2)/2 + 1/4
```

If the second argument is a number, `euler` evaluates the polynomial at that number. Here, the result is a floating-point number because the input arguments are not symbolic numbers:

```
euler(2, 1/3)
```

```
ans =
-0.2222
```

To get the exact symbolic result, convert at least one number to a symbolic object:

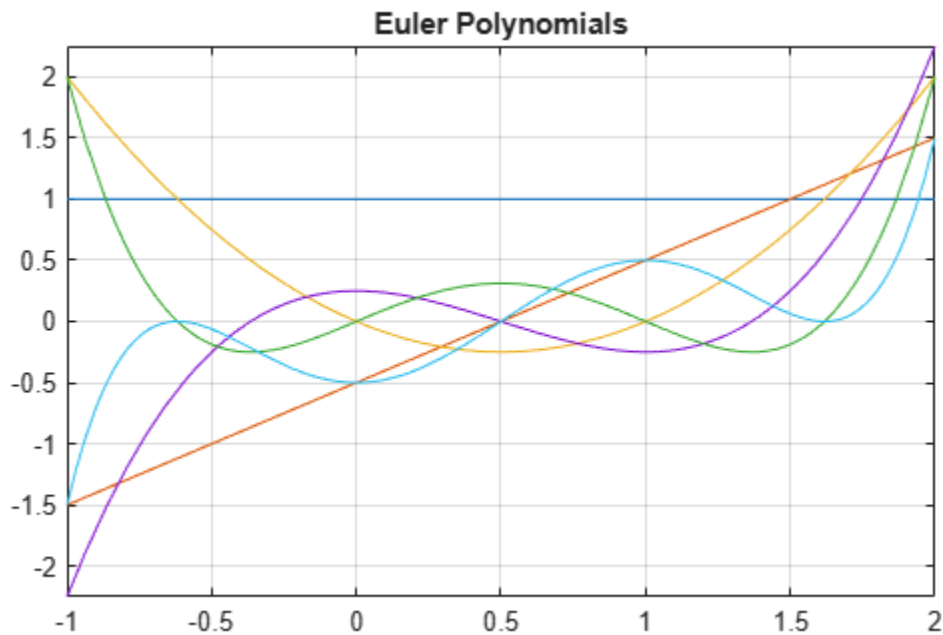
```
euler(2, sym(1/3))
```

```
ans =
-2/9
```

### Plot Euler Polynomials

Plot the first six Euler polynomials.

```
syms x
fplot(euler(0:5, x), [-1 2])
title('Euler Polynomials')
grid on
```



### Handle Expressions Containing Euler Polynomials

Many functions, such as `diff` and `expand`, can handle expressions containing `euler`.

Find the first and second derivatives of the Euler polynomial:

```

syms n x
diff(euler(n,x^2), x)

ans =
2*n*x*euler(n - 1, x^2)

diff(euler(n,x^2), x, x)

ans =
2*n*euler(n - 1, x^2) + 4*n*x^2*euler(n - 2, x^2)*(n - 1)

Expand these expressions containing the Euler polynomials:

expand(euler(n, 2 - x))

ans =
2*(1 - x)^n - (-1)^n*euler(n, x)

expand(euler(n, 2*x))

ans =
(2*2^n*bernoulli(n + 1, x + 1/2))/(n + 1) - ...
(2*2^n*bernoulli(n + 1, x))/(n + 1)

```

## Input Arguments

### **n** — Index of the Euler number or polynomial

nonnegative integer | symbolic nonnegative integer | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Index of the Euler number or polynomial, specified as a nonnegative integer, symbolic nonnegative integer, variable, expression, function, vector, or matrix. If **n** is a vector or matrix, `euler` returns Euler numbers or polynomials for each element of **n**. If one input argument is a scalar and the other one is a vector or a matrix, `euler(n,x)` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

### **x** — Polynomial variable

symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Polynomial variable, specified as a symbolic variable, expression, function, vector, or matrix. If **x** is a vector or matrix, `euler` returns Euler numbers or polynomials for each element of **x**. When you use the `euler` function to find Euler polynomials, at least one argument must be a scalar or both arguments must be vectors or matrices of the same size. If one input argument is a scalar and the other one is a vector or a matrix, `euler(n,x)` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

## More About

### Euler Polynomials

The Euler polynomials are defined as follows:

$$\frac{2e^{xt}}{e^t + 1} = \sum_{n=0}^{\infty} \text{euler}(n, x) \frac{t^n}{n!}$$

### **Euler Numbers**

The Euler numbers are defined in terms of Euler polynomials as follows:

$$\text{euler}(n) = 2^n \text{euler}\left(n, \frac{1}{2}\right)$$

### **Tips**

- For the other meaning of Euler's number,  $e = 2.71828\dots$ , call `exp(1)` to return the double-precision representation. For the exact representation of Euler's number  $e$ , call `exp(sym(1))`.
- For the Euler-Mascheroni constant, see `eulergamma`.

## **Version History**

**Introduced in R2014a**

### **See Also**

`bernoulli` | `eulergamma`



# eulergamma

Euler-Mascheroni constant

## Syntax

eulergamma

## Description

eulergamma represents the Euler-Mascheroni constant on page 6-410. To get a floating-point approximation with the current precision set by `digits`, use `vpa(eulergamma)`.

## Examples

### Represent and Numerically Approximate the Euler-Mascheroni Constant

Represent the Euler-Mascheroni constant using `eulergamma`, which returns the symbolic form `eulergamma`.

```
eulergamma
```

```
ans =  
eulergamma
```

Use `eulergamma` in symbolic calculations. Numerically approximate your result with `vpa`.

```
a = eulergamma;  
g = a^2 + log(a)  
gVpa = vpa(g)  
  
g =  
log(eulergamma) + eulergamma^2  
gVpa =  
-0.21636138917392614801928563244766
```

Find the double-precision approximation of the Euler-Mascheroni constant using `double`.

```
double(eulergamma)  
  
ans =  
0.5772
```

### Show Relation of Euler-Mascheroni Constant to Gamma Functions

Show the relations between the Euler-Mascheroni constant  $\gamma$ , digamma function  $\Psi$ , and gamma function  $\Gamma$ .

Show that  $\gamma = -\Psi(1)$ .

```
-psi(sym(1))  
  
ans =  
eulergamma
```

Show that  $\gamma = -\Gamma'(x)|_{x=1}$ .

```
syms x
-subs(diff(gamma(x)),x,1)

ans =
eulergamma
```

## More About

### Euler-Mascheroni Constant

The Euler-Mascheroni constant is defined as follows:

$$\gamma = \lim_{n \rightarrow \infty} \left( \left( \sum_{k=1}^n \frac{1}{k} \right) - \ln(n) \right)$$

## Tips

- For the value  $e = 2.71828\dots$ , called Euler's number, use `exp(1)` to return the double-precision representation. For the exact representation of Euler's number  $e$ , call `exp(sym(1))`.
- For the other meaning of Euler's numbers and for Euler's polynomials, see `euler`.

## Version History

Introduced in R2014a

## See Also

`coshint` | `euler`

# eulerPhi

Euler phi function

## Syntax

```
p = eulerPhi(n)
```

## Description

`p = eulerPhi(n)` evaluates the Euler phi function on page 6-413 or (also known as the totient function) for a positive integer  $n$ .

## Examples

### Multiplicative Property of Euler Phi Function

Compute the Euler phi function  $\phi(n)$  for the integer  $n = 35$ .

```
p = eulerPhi(35)
```

```
p =  
24
```

The Euler phi function satisfies the multiplicative property  $\phi(xy) = \phi(x) \phi(y)$  if the two integers  $x$  and  $y$  are relatively prime (also known as coprime). The integer factorization of 35 is 7 and 5, which are relatively prime. Show that  $\phi(35)$  satisfies the multiplicative property.

Calculate  $\phi(x)$  and  $\phi(y)$  for the two factors.

```
px = eulerPhi(7)
```

```
px =  
6
```

```
py = eulerPhi(5)
```

```
py =  
4
```

Verify that  $px$  and  $py$  satisfy the multiplicative property.

```
p = px*py
```

```
p =  
24
```

### Euler's Product Formula

If a positive integer  $n$  has prime factorization  $n = p_1^{k_1} p_2^{k_2} \dots p_r^{k_r}$  with distinct prime factors  $p_1, p_2, \dots, p_r$ , then the Euler phi function satisfies the product formula

$$\phi(n) = n \left(1 - \frac{1}{p_1}\right) \left(1 - \frac{1}{p_2}\right) \dots \left(1 - \frac{1}{p_r}\right).$$

The integer  $n = 36$  has distinct prime factors 2 and 3. Show that  $\phi(36)$  satisfies the Euler's product formula.

Declare 36 as a symbolic number and evaluate  $\phi(36)$ .

```
n = sym(36)
```

```
n = 36
```

```
p = eulerPhi(n)
```

```
p = 12
```

List the prime factors of 36.

```
f_n = factor(n)
```

```
f_n = (2 2 3 3)
```

Substitute the prime factors 2 and 3 into the product formula.

```
p_product = n*(1-1/2)*(1-1/3)
```

```
p_product = 12
```

### Euler's Theorem

Euler's theorem states that  $a^{\phi(n)} \equiv 1 \pmod{n}$  if and only if the two positive integers  $a$  and  $n$  are relatively prime. Show that the Euler phi function  $\phi(n)$  satisfies Euler's theorem for the integers  $a = 15$  and  $n = 4$ .

```
a = 15;
```

```
n = 4;
```

```
isCongruent = powermod(a,eulerPhi(n),n) == mod(1,n)
```

```
isCongruent = logical
1
```

Confirm that  $a$  and  $n$  are relatively prime.

```
g = gcd(a,n)
```

```
g =
1
```

## Mean Value of Euler Phi Function Results

Calculate the Euler phi function  $\phi(n)$  for the integers  $n$  from 1 to 1000.

```
P = eulerPhi(1:1000);
```

Find the mean value of  $\phi(n)/n$ .

```
Pave = mean(P./(1:1000))
```

```
Pave =  
0.6082
```

## Input Arguments

### **n** — Input

number | vector | matrix | array | symbolic number | symbolic array

Input, specified as a number, vector, matrix, array, symbolic number, or symbolic array. The elements of  $n$  must be positive integers.

Data Types: `single` | `double` | `sym`

## More About

### Euler Phi Function

The Euler phi function  $\phi(n)$  computes the number of integers between 1 and  $n$  that are relatively prime (also known as coprime) to  $n$ . Two integers are relatively prime if there is no integer greater than one that divides them both. In other words, their greatest common divisor is one.

## Version History

Introduced in R2020a

## References

- [1] Redmond, D. *Number Theory: An Introduction to Pure and Applied Mathematics*. New York: Marcel Dekker, 1996.

## See Also

`isprime` | `primes`

## expand

Expand expressions and simplify inputs of functions by using identities

### Syntax

```
expand(S)
expand(S, Name, Value)
```

### Description

`expand(S)` multiplies all parentheses in `S`, and simplifies inputs to functions such as `cos(x + y)` by applying standard identities.

`expand(S, Name, Value)` uses additional options specified by one or more name-value pair arguments. For example, specifying `'IgnoreAnalyticConstraints'` as `true` uses convenient identities to simplify the input.

### Examples

#### Expand Symbolic Expression

```
syms x
p = (x - 2)*(x - 4);
expand(p)

ans =
x^2 - 6*x + 8
```

#### Expand Trigonometric Expression

Expand the trigonometric expression `cos(x + y)`. Simplify the `cos` function input `x + y` to `x` or `y` by applying standard identities.

```
syms x y
expand(cos(x + y))

ans =
cos(x)*cos(y) - sin(x)*sin(y)
```

#### Expand Exponential Expression

Expand  $e^{(a+b)^2}$ . Simplify the `exp` function input,  $(a + b)^2$ , by applying standard identities.

```
syms a b
f = exp((a + b)^2);
expand(f)
```

```
ans =
exp(a^2)*exp(b^2)*exp(2*a*b)
```

### Expand Vector of Expressions

Expand expressions in a vector. Simplify the inputs to functions in the expressions by applying identities.

```
syms t
V = [sin(2*t), cos(2*t)];
expand(V)

ans =
[ 2*cos(t)*sin(t), 2*cos(t)^2 - 1]
```

### Expand Only Arithmetic and Suppress Expansion of Functions

By default, `expand` both expands terms raised to powers and expands functions by applying identities that simplify inputs to the functions. Expand only terms raised to powers and suppress expansion of functions by using `'ArithmeticOnly'`.

Expand  $(\sin(3x) - 1)^2$ . By default, `expand` will expand the power  $^2$  and simplify the `sin` input  $3x$  to  $x$ .

```
syms x
f = (sin(3*x) - 1)^2;
expand(f)

ans =
2*sin(x) + sin(x)^2 - 8*cos(x)^2*sin(x) - 8*cos(x)^2*sin(x)^2...
+ 16*cos(x)^4*sin(x)^2 + 1
```

Suppress expansion of functions, such as  $\sin(3x)$ , by setting `ArithmeticOnly` to `true`.

```
expand(f, 'ArithmeticOnly', true)

ans =
sin(3*x)^2 - 2*sin(3*x) + 1
```

### Simplify Log Input by Removing Constraints

Simplify the input of `log` function calls. By default, `expand` does not simplify logarithm input because the identities used are not valid for complex values of variables.

```
syms a b c
f = log((a*b/c)^2);
expand(f)

ans =
log((a^2*b^2)/c^2)
```

Apply identities to simplify the input of logarithms by setting `'IgnoreAnalyticConstraints'` to `true`.

```
expand(f, 'IgnoreAnalyticConstraints', true)
```

```
ans =  
2*log(a) + 2*log(b) - 2*log(c)
```

## Input Arguments

### S — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.*

Example: `expand(S, 'ArithmeticOnly', true)`

### ArithmeticOnly — Expand only algebraic expressions

false (default) | true

Expand only algebraic expressions, specified as the comma-separated pair consisting of 'ArithmeticOnly' and true or false. If the value is true, the function expands the arithmetic part of an expression without expanding trigonometric, hyperbolic, logarithmic, and special functions. This option does not prevent the expansion of powers and roots.

### IgnoreAnalyticConstraints — Use convenient identities for simplification

false (default) | true

Use convenient identities for simplification, specified as the comma-separated pair consisting of 'IgnoreAnalyticConstraints' and true or false.

Setting 'IgnoreAnalyticConstraints' to true can give you simpler solutions, which could lead to results not generally valid. In other words, this option applies mathematical identities that are convenient, but the results might not hold for all values of the variables. In some cases, this option can let `expand` return simpler results that might not be equivalent to the initial expression. See “Algorithms” on page 6-416.

## Algorithms

When you use 'IgnoreAnalyticConstraints', `expand` applies some of these rules.

- $\log(a) + \log(b) = \log(a \cdot b)$  for all values of  $a$  and  $b$ . In particular, the following equality is valid for all values of  $a$ ,  $b$ , and  $c$ :  

$$(a \cdot b)^c = a^c \cdot b^c.$$
- $\log(a^b) = b \cdot \log(a)$  for all values of  $a$  and  $b$ . In particular, the following equality is valid for all values of  $a$ ,  $b$ , and  $c$ :



$$(a^b)^c = a^{b \cdot c}.$$

- If  $f$  and  $g$  are standard mathematical functions and  $f(g(x)) = x$  for all small positive numbers,  $f(g(x)) = x$  is assumed to be valid for all complex  $x$ .
  - $\log(e^x) = x$
  - $\text{asin}(\sin(x)) = x$ ,  $\text{acos}(\cos(x)) = x$ ,  $\text{atan}(\tan(x)) = x$
  - $\text{asinh}(\sinh(x)) = x$ ,  $\text{acosh}(\cosh(x)) = x$ ,  $\text{atanh}(\tanh(x)) = x$
  - $W_k(x \cdot e^x) = x$  for all values of  $k$

## Version History

Introduced before R2006a

## See Also

### Functions

`collect` | `combine` | `factor` | `horner` | `numden` | `rewrite` | `simplify` | `simplifyFraction`

### Live Editor Tasks

Simplify Symbolic Expression

### Topics

“Choose Function to Rearrange Expression” on page 3-123

“Simplify Symbolic Expressions” on page 3-133

“Simplify Symbolic Expressions Using Live Editor Task” on page 3-138

## expint

Exponential integral function

### Syntax

```
expint(x)
expint(n,x)
```

### Description

`expint(x)` returns the one-argument exponential integral function defined as

$$\text{expint}(x) = \int_x^{\infty} \frac{e^{-t}}{t} dt.$$

`expint(n,x)` returns the two-argument exponential integral function defined as

$$\text{expint}(n,x) = \int_1^{\infty} \frac{e^{-xt}}{t^n} dt.$$

### Examples

#### One-Argument Exponential Integral for Floating-Point and Symbolic Numbers

Compute the exponential integrals for floating-point numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
s = [expint(1/3), expint(1), expint(-2)]

s =
    0.8289 + 0.0000i    0.2194 + 0.0000i   -4.9542 - 3.1416i
```

Compute the exponential integrals for the same numbers converted to symbolic objects. For positive values  $x$ , `expint(x)` returns `-ei(-x)`. For negative values  $x$ , it returns `-pi*i - ei(-x)`.

```
s = [expint(sym(1)/3), expint(sym(1)), expint(sym(-2))]

s =
[ -ei(-1/3), -ei(-1), - ei(2) - pi*1i]
```

Use `vpa` to approximate this result with 10-digit accuracy.

```
vpa(s, 10)

ans =
[ 0.8288877453, 0.2193839344, - 4.954234356 - 3.141592654i]
```

#### Two-Argument Exponential Integral for Floating-Point and Symbolic Numbers

When computing two-argument exponential integrals, convert the numbers to symbolic objects.

```
s = [expint(2, sym(1)/3), expint(sym(1), Inf), expint(-1, sym(-2))]
s =
[ expint(2, 1/3), 0, -exp(2)/4]
```

Use `vpa` to approximate this result with 25-digit accuracy.

```
vpa(s, 25)

ans =
[ 0.4402353954575937050522018, 0, -1.847264024732662556807607]
```

### Two-Argument Exponential Integral with Nonpositive First Argument

Compute two-argument exponential integrals. If  $n$  is a nonpositive integer, then `expint( $n$ ,  $x$ )` returns an explicit expression in the form  $\exp(-x) \cdot p(1/x)$ , where  $p$  is a polynomial of degree  $1 - n$ .

```
syms x
expint(0, x)
expint(-1, x)
expint(-2, x)

ans =
exp(-x)/x

ans =
exp(-x)*(1/x + 1/x^2)

ans =
exp(-x)*(1/x + 2/x^2 + 2/x^3)
```

### Derivatives of Exponential Integral

Compute the first, second, and third derivatives of a one-argument exponential integral.

```
syms x
diff(expint(x), x)
diff(expint(x), x, 2)
diff(expint(x), x, 3)

ans =
-exp(-x)/x

ans =
exp(-x)/x + exp(-x)/x^2

ans =
- exp(-x)/x - (2*exp(-x))/x^2 - (2*exp(-x))/x^3
```

Compute the first derivatives of a two-argument exponential integral.

```
syms n x
diff(expint(n, x), x)
diff(expint(n, x), n)

ans =
-expint(n - 1, x)
```

```
ans =
- hypergeom([1 - n, 1 - n], [2 - n, 2 - n],...
    -x)/(n - 1)^2 - (x^(n - 1)*pi*(psi(n) - ...
    log(x) + pi*cot(pi*n)))/(sin(pi*n)*gamma(n))
```

## Input Arguments

### x — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input specified as a symbolic number, variable, expression, function, vector, or matrix.

### n — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input specified as a symbolic number, variable, expression, function, vector, or matrix. When you compute the two-argument exponential integral function, at least one argument must be a scalar.

## Tips

- Calling `expint` for numbers that are not symbolic objects invokes the MATLAB `expint` function. This function accepts one argument only. To compute the two-argument exponential integral, use `sym` to convert the numbers to symbolic objects, and then call `expint` for those symbolic objects. You can approximate the results with floating-point numbers using `vpa`.
- The following values of the exponential integral differ from those returned by the MATLAB `expint` function: `expint(sym(Inf)) = 0`, `expint(-sym(Inf)) = -Inf`, `expint(sym(NaN)) = NaN`.
- For positive real  $x$ ,  $\text{expint}(x) = -\text{ei}(-x)$ . For negative real  $x$ ,  $\text{expint}(x) = -\pi i - \text{ei}(-x)$ .
- If one input argument is a scalar and the other argument is a vector or a matrix, then `expint(n,x)` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

## Algorithms

The relation between `expint` and `ei` is

$$\text{expint}(1,-x) = \text{ei}(x) + (\ln(x) - \ln(1/x))/2 - \ln(-x)$$

Both functions `ei(x)` and `expint(1,x)` have a logarithmic singularity at the origin and a branch cut along the negative real axis. The `ei` function is not continuous when approached from above or below this branch cut.

The `expint` function is related to the upper incomplete gamma function `igamma` as

$$\text{expint}(n,x) = (x^{(n-1)}) * \text{igamma}(1-n,x)$$

## Version History

Introduced in R2013a

## See Also

ei | expint | vpa

## expm

Matrix exponential of symbolic matrices

### Syntax

`R = expm(A)`

### Description

`R = expm(A)` computes the matrix exponential on page 6-422 of the square symbolic matrix `A`.

### Examples

#### Matrix Exponential

Compute the matrix exponential for the 2-by-2 matrix and simplify the result.

```
syms x
A = [0 x; -x 0];
simplify(expm(A))

ans =
[ cos(x), sin(x)]
[ -sin(x), cos(x)]
```

### Input Arguments

#### A — Input matrix

square matrix

Input matrix, specified as a square symbolic matrix.

### Output Arguments

#### R — Resulting matrix

symbolic matrix

Resulting function, returned as a symbolic matrix.

### More About

#### Matrix Exponential

The matrix exponential  $e^A$  of matrix  $A$  is

$$e^A = \sum_{k=0}^{\infty} \frac{1}{k!} A^k = 1 + A + \frac{A^2}{2} + \dots$$

## Version History

Introduced before R2006a

## See Also

`eig` | `funm` | `jordan` | `logm` | `sqrtn`

## ezcontour

(Not recommended) Contour plotter

---

**Note** `ezcontour` is not recommended. Use `fcontour` instead.

---

### Syntax

```
ezcontour(f)
ezcontour(f,domain)
ezcontour( ___,n)
```

### Description

`ezcontour(f)` plots the contour lines of  $f(x,y)$ , where  $f$  is a symbolic expression that represents a mathematical function of two variables, such as  $x$  and  $y$ . `ezcontour` automatically adds a title and axis labels.

The function  $f$  is plotted over the default domain  $-2\pi < x < 2\pi$ ,  $-2\pi < y < 2\pi$ . MATLAB software chooses the computational grid according to the amount of variation that occurs; if the function  $f$  is not defined (singular) for points on the grid, then these points are not plotted.

`ezcontour(f,domain)` plots  $f(x,y)$  over the specified domain. domain can be either a 4-by-1 vector  $[xmin, xmax, ymin, ymax]$  or a 2-by-1 vector  $[min, max]$  (where,  $min < x < max$ ,  $min < y < max$ ). If  $f$  is a function of the variables  $u$  and  $v$  (rather than  $x$  and  $y$ ), then the domain endpoints  $umin, umax, vmin$ , and  $vmax$  are sorted alphabetically. Thus, `ezcontour(u^2 - v^3, [0,1], [3,6])` plots the contour lines for  $u^2 - v^3$  over  $0 < u < 1$ ,  $3 < v < 6$ .

`ezcontour( ___, n)` plots  $f$  over the default domain using an  $n$ -by- $n$  grid. The default value for  $n$  is 60.

### Examples

#### Plot Contour Lines of Symbolic Expression

The following mathematical expression defines a function of two variables,  $x$  and  $y$ .

$$f(x,y) = 3(1-x)^2 e^{-x^2 - (y+1)^2} - 10\left(\frac{x}{5} - x^3 - y^5\right) e^{-x^2 - y^2} - \frac{1}{3} e^{-(x+1)^2 - y^2}.$$

`ezcontour` requires a `sym` argument that expresses this function using MATLAB syntax to represent exponents, natural logs, etc. This function is represented by the symbolic expression

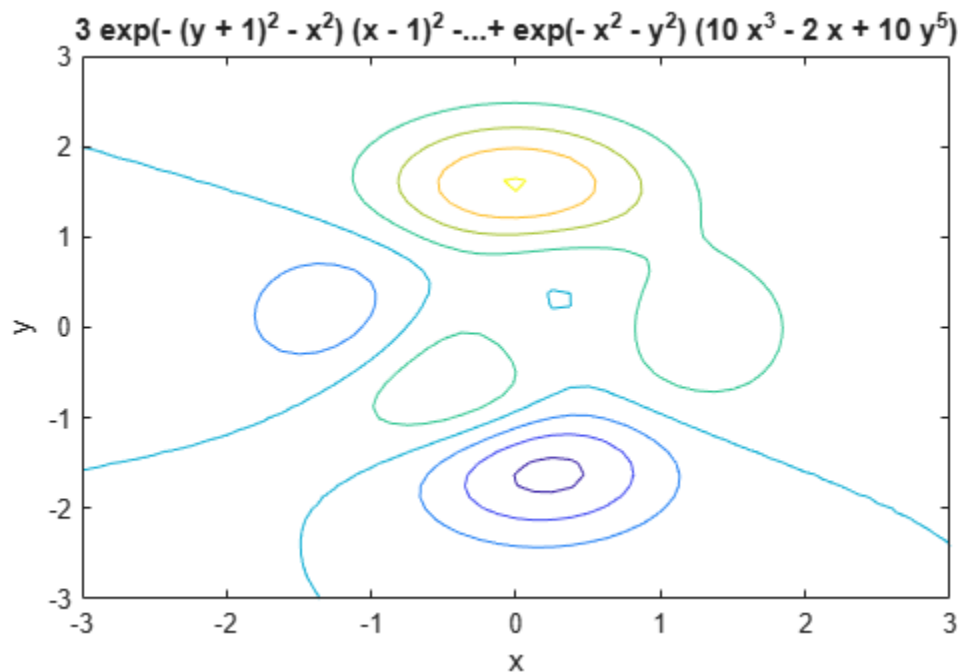
```
syms x y
f = 3*(1-x)^2*exp(-(x^2)-(y+1)^2)...
    - 10*(x/5 - x^3 - y^5)*exp(-x^2-y^2)...
    - 1/3*exp(-(x+1)^2 - y^2);
```

For convenience, this expression is written on three lines.



Pass the sym `f` to `ezcontour` along with a domain ranging from -3 to 3 and specify a computational grid of 49-by-49.

```
ezcontour(f, [-3,3], 49)
```



In this particular case, the title is too long to fit at the top of the graph so MATLAB abbreviates the title.

## Input Arguments

### **f** — Input

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **domain** — Domain to plot over

symbolic vector

Domain to plot over, specified as a symbolic vector. `domain` is a 4-by-1 vector  $[xmin, xmax, ymin, ymax]$  or a 2-by-1 vector  $[min, max]$  (where,  $min < x < max$ ,  $min < y < max$ ). If  $f$  is a function of the variables  $u$  and  $v$  (rather than  $x$  and  $y$ ), then the domain endpoints  $umin$ ,  $umax$ ,  $vmin$ , and  $vmax$  are sorted alphabetically

### **n** — grid points

number | symbolic number

Grid points, specified as a number or a symbolic number.

## Version History

Introduced before R2006a

**See Also**

`contour` | `fcontour` | `fmesh` | `fplot` | `fplot3` | `fsurf`

# ezcontourf

(Not recommended) Filled contour plotter

---

**Note** ezcontourf is not recommended. Use fcontour instead.

---

## Syntax

```
ezcontourf(f)
ezcontourf(f,domain)
ezcontourf(____,n)
```

## Description

ezcontourf(f) plots the contour lines of  $f(x,y)$ , where  $f$  is a sym that represents a mathematical function of two variables, such as  $x$  and  $y$ .

The function  $f$  is plotted over the default domain  $-2\pi < x < 2\pi$ ,  $-2\pi < y < 2\pi$ . MATLAB software chooses the computational grid according to the amount of variation that occurs; if the function  $f$  is not defined (singular) for points on the grid, then these points are not plotted.

ezcontourf automatically adds a title and axis labels.

ezcontourf(f,domain) plots  $f(x,y)$  over the specified domain. domain can be either a 4-by-1 vector  $[xmin, xmax, ymin, ymax]$  or a 2-by-1 vector  $[min, max]$  (where,  $min < x < max$ ,  $min < y < max$ ). If  $f$  is a function of the variables  $u$  and  $v$  (rather than  $x$  and  $y$ ), then the domain endpoints  $umin$ ,  $umax$ ,  $vmin$ , and  $vmax$  are sorted alphabetically. Thus, ezcontourf( $u^2 - v^3$ , [0,1], [3,6]) plots the contour lines for  $u^2 - v^3$  over  $0 < u < 1$ ,  $3 < v < 6$ .

ezcontourf(\_\_\_\_,n) plots  $f$  over the default domain using an  $n$ -by- $n$  grid. The default value for  $n$  is 60.

## Examples

### Plot Filled Contours

The following mathematical expression defines a function of two variables,  $x$  and  $y$ .

$$f(x,y) = 3(1-x)^2 e^{-x^2 - (y+1)^2} - 10\left(\frac{x}{5} - x^3 - y^5\right) e^{-x^2 - y^2} - \frac{1}{3} e^{-(x+1)^2 - y^2}.$$

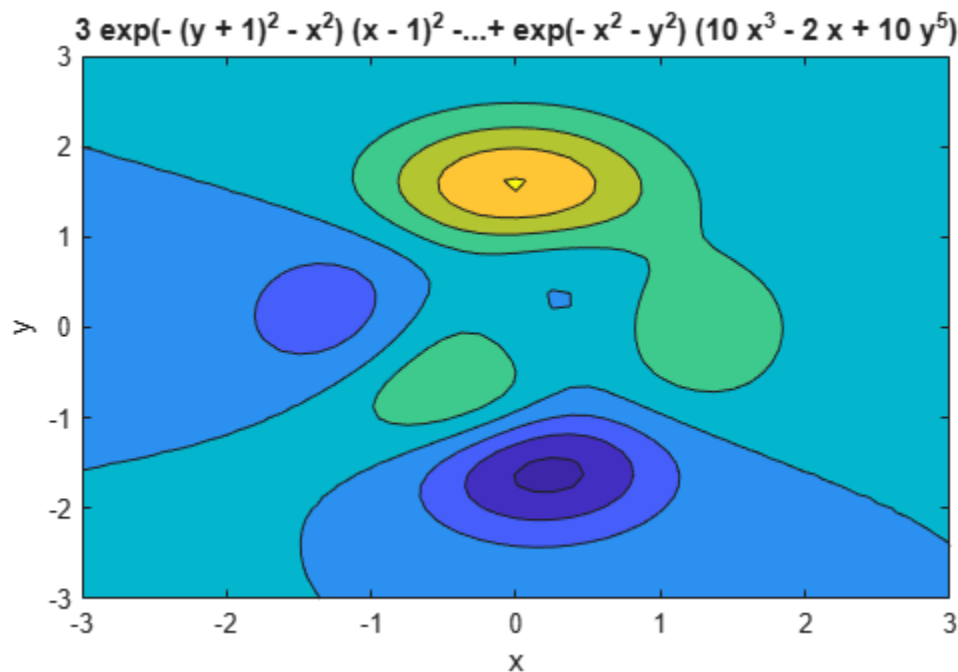
ezcontourf requires a sym argument that expresses this function using MATLAB syntax to represent exponents, natural logs, etc. This function is represented by the symbolic expression

```
syms x y
f = 3*(1-x)^2*exp(-(x^2)-(y+1)^2)...
    - 10*(x/5 - x^3 - y^5)*exp(-x^2-y^2)...
    - 1/3*exp(-(x+1)^2 - y^2);
```

For convenience, this expression is written on three lines.

Pass the sym `f` to `ezcontourf` along with a domain ranging from -3 to 3 and specify a grid of 49-by-49.

```
ezcontourf(f,[-3,3],49)
```



In this particular case, the title is too long to fit at the top of the graph so MATLAB abbreviates the title.

## Input Arguments

### **f** — Input

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **domain** — Domain to plot over

symbolic vector

Domain to plot over, specified as a symbolic vector. `domain` is a 4-by-1 vector `[xmin, xmax, ymin, ymax]` or a 2-by-1 vector `[min, max]` (where,  $\min < x < \max$ ,  $\min < y < \max$ ). If  $f$  is a function of the variables  $u$  and  $v$  (rather than  $x$  and  $y$ ), then the domain endpoints  $umin$ ,  $umax$ ,  $vmin$ , and  $vmax$  are sorted alphabetically

### **n** — grid points

number | symbolic number

Grid points, specified as a number or a symbolic number.

## Version History

Introduced before R2006a

**See Also**

`contour` | `fcontour` | `fmesh` | `fplot` | `fplot3` | `fsurf`

## ezmesh

(Not recommended) 3-D mesh plotter

---

**Note** ezmesh is not recommended. Use fmesh instead.

---

### Syntax

```
ezmesh(f)
ezmesh(f,domain)
ezmesh(x,y,z)
ezmesh(x,y,z,domain)
ezmesh( __ ,n)
ezmesh( __ , 'circ')
```

### Description

ezmesh(f) creates a graph of  $f(x,y)$ , where  $f$  is a symbolic expression that represents a mathematical function of two variables, such as  $x$  and  $y$ .

The function  $f$  is plotted over the default domain  $-2\pi < x < 2\pi$ ,  $-2\pi < y < 2\pi$ . MATLAB software chooses the computational grid according to the amount of variation that occurs; if the function  $f$  is not defined (singular) for points on the grid, then these points are not plotted.

ezmesh(f,domain) plots  $f$  over the specified domain. domain can be either a 4-by-1 vector [ $xmin$ ,  $xmax$ ,  $ymin$ ,  $ymax$ ] or a 2-by-1 vector [ $min$ ,  $max$ ] (where,  $min < x < max$ ,  $min < y < max$ ).

If  $f$  is a function of the variables  $u$  and  $v$  (rather than  $x$  and  $y$ ), then the domain endpoints  $umin$ ,  $umax$ ,  $vmin$ , and  $vmax$  are sorted alphabetically. Thus, ezmesh( $u^2 - v^3$ , [0,1], [3,6]) plots  $u^2 - v^3$  over  $0 < u < 1$ ,  $3 < v < 6$ .

ezmesh(x,y,z) plots the parametric surface  $x = x(s,t)$ ,  $y = y(s,t)$ , and  $z = z(s,t)$  over the square  $-2\pi < s < 2\pi$ ,  $-2\pi < t < 2\pi$ .

ezmesh(x,y,z,domain) plots the parametric surface using the specified domain.

ezmesh( \_\_ ,n) plots  $f$  over the default domain using an  $n$ -by- $n$  grid. The default value for  $n$  is 60.

ezmesh( \_\_ , 'circ') plots  $f$  over a disk centered on the domain.

### Examples

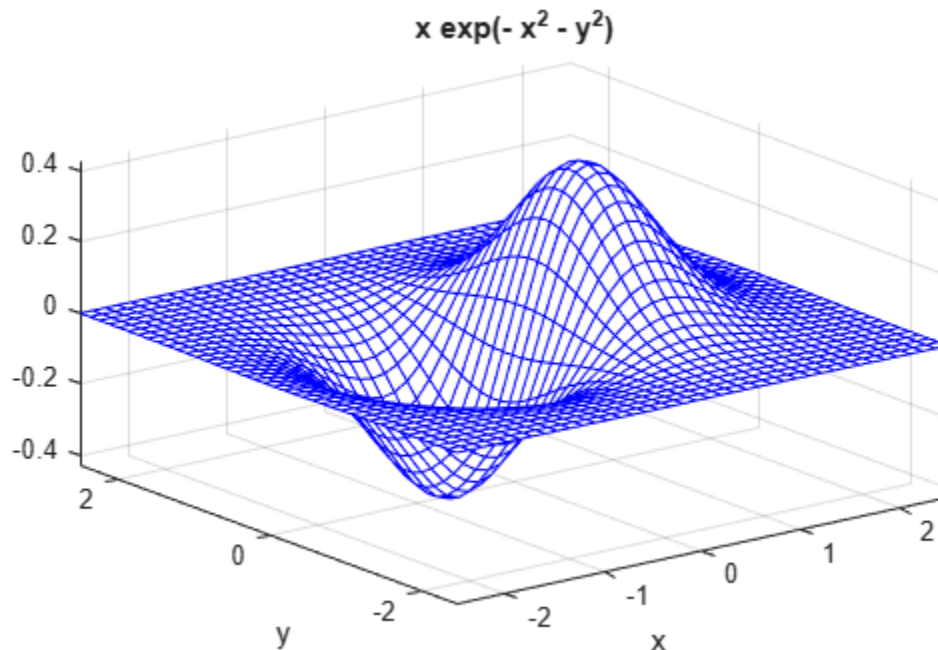
#### 3-D Mesh Plot of Symbolic Expression

This example visualizes the function,

$$f(x,y) = xe^{-x^2 - y^2},$$

with a mesh plot drawn on a 40-by-40 grid. The mesh lines are set to a uniform blue color by setting the colormap to a single color.

```
syms x y
ezmesh(x*exp(-x^2-y^2), [-2.5, 2.5], 40)
colormap([0 0 1])
```



## Input Arguments

### **f** — Input

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **x** — Input

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **y** — Input

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **z** — Input

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **domain** — Domain to plot over

symbolic vector

Domain to plot over, specified as a symbolic vector. **domain** is a 4-by-1 vector  $[xmin, xmax, ymin, ymax]$  or a 2-by-1 vector  $[min, max]$  (where,  $min < x < max$ ,  $min < y < max$ ). If  $f$  is a function of the

variables  $u$  and  $v$  (rather than  $x$  and  $y$ ), then the domain endpoints  $umin$ ,  $umax$ ,  $vmin$ , and  $vmax$  are sorted alphabetically.

**n — grid points**

number | symbolic number

Grid points, specified as a number or a symbolic number.

## Version History

Introduced before R2006a

**See Also**

fcontour | fmesh | fplot | fplot3 | fsurf | mesh



# ezmeshc

(Not recommended) Combined mesh and contour plotter

---

**Note** ezmeshc is not recommended. Use fmesh instead.

---

## Syntax

```
ezmeshc(f)
ezmeshc(f, domain)
ezmeshc(x, y, z)
ezmeshc(x, y, z, domain)
ezmeshc( ___, n)
ezmeshc( ___, 'circ')
```

## Description

ezmeshc(f) creates a graph of  $f(x,y)$ , where  $f$  is a symbolic expression that represents a mathematical function of two variables, such as  $x$  and  $y$ .

The function  $f$  is plotted over the default domain  $-2\pi < x < 2\pi$ ,  $-2\pi < y < 2\pi$ . MATLAB software chooses the computational grid according to the amount of variation that occurs; if the function  $f$  is not defined (singular) for points on the grid, then these points are not plotted.

ezmeshc(f, domain) plots  $f$  over the specified domain. domain can be either a 4-by-1 vector [ $xmin$ ,  $xmax$ ,  $ymin$ ,  $ymax$ ] or a 2-by-1 vector [ $min$ ,  $max$ ] (where,  $min < x < max$ ,  $min < y < max$ ).

If  $f$  is a function of the variables  $u$  and  $v$  (rather than  $x$  and  $y$ ), then the domain endpoints  $umin$ ,  $umax$ ,  $vmin$ , and  $vmax$  are sorted alphabetically. Thus, ezmeshc( $u^2 - v^3$ , [0,1], [3,6]) plots  $u^2 - v^3$  over  $0 < u < 1$ ,  $3 < v < 6$ .

ezmeshc(x, y, z) plots the parametric surface  $x = x(s,t)$ ,  $y = y(s,t)$ , and  $z = z(s,t)$  over the square  $-2\pi < s < 2\pi$ ,  $-2\pi < t < 2\pi$ .

ezmeshc(x, y, z, domain) plots the parametric surface using the specified domain.

ezmeshc( \_\_\_, n) plots  $f$  over the default domain using an  $n$ -by- $n$  grid. The default value for  $n$  is 60.

ezmeshc( \_\_\_, 'circ') plots  $f$  over a disk centered on the domain.

## Examples

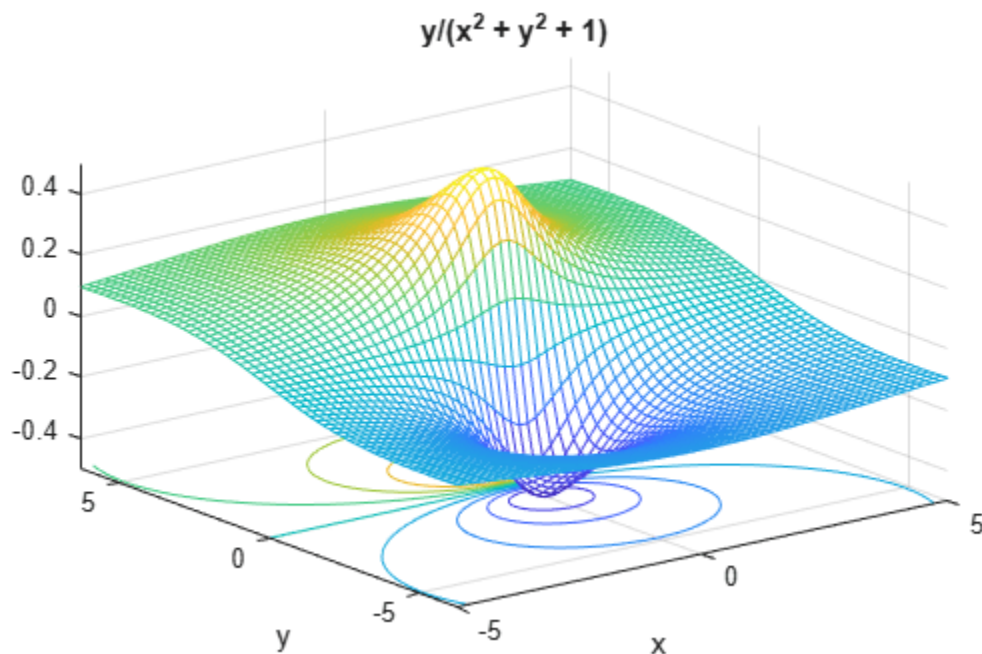
### 3-D Mesh Plot with Contours

Create a mesh/contour graph of the expression,

$$f(x, y) = \frac{y}{1 + x^2 + y^2},$$

over the domain  $-5 < x < 5$ ,  $-2\pi < y < 2\pi$ . Use the mouse to rotate the axes to better observe the contour lines (this picture uses a view of azimuth = -65 and elevation = 26).

```
syms x y
ezmeshc(y/(1 + x^2 + y^2),[-5,5,-2*pi,2*pi])
```



## Input Arguments

### **f — Input**

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **x — Input**

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **y — Input**

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **z — Input**

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **domain — Domain to plot over**

symbolic vector

Domain to plot over, specified as a symbolic vector. **domain** is a 4-by-1 vector  $[xmin, xmax, ymin, ymax]$  or a 2-by-1 vector  $[min, max]$  (where,  $min < x < max$ ,  $min < y < max$ ). If  $f$  is a function of the

variables  $u$  and  $v$  (rather than  $x$  and  $y$ ), then the domain endpoints  $umin$ ,  $umax$ ,  $vmin$ , and  $vmax$  are sorted alphabetically.

**n — grid points**

number | symbolic number

Grid points, specified as a number or a symbolic number.

## Version History

Introduced before R2006a

**See Also**

fcontour | fmesh | fplot | fplot3 | fsurf | meshc

## ezplot

(Not recommended) Plot symbolic expression, equation, or function

---

**Note** `ezplot` is not recommended. Use `fplot` instead. For implicit plots, use `fimplicit`.

---

### Syntax

```
ezplot(f)
ezplot(f,[min,max])
ezplot(f,[xmin,xmax,ymin,ymax])

ezplot(x,y)
ezplot(x,y,[tmin,tmax])

ezplot(f,[min,max],fig)
ezplot(f,[xmin,xmax,ymin,ymax],fig)
ezplot(x,y,[tmin,tmax],fig)

h = ezplot( __ )
```

### Description

`ezplot(f)` plots a symbolic expression, equation, or function  $f$ . By default, `ezplot` plots a univariate expression or function over the range  $[-2\pi\ 2\pi]$  or over a subinterval of this range. If  $f$  is an equation or function of two variables, the default range for both variables is  $[-2\pi\ 2\pi]$  or over a subinterval of this range.

`ezplot(f,[min,max])` plots  $f$  over the specified range. If  $f$  is a univariate expression or function, then  $[min,max]$  specifies the range for that variable. This is the range along the abscissa (horizontal axis). If  $f$  is an equation or function of two variables, then  $[min,max]$  specifies the range for both variables, that is the ranges along both the abscissa and the ordinate.

`ezplot(f,[xmin,xmax,ymin,ymax])` plots  $f$  over the specified ranges along the abscissa and the ordinate.

`ezplot(x,y)` plots the parametrically defined planar curve  $x = x(t)$  and  $y = y(t)$  over the default range  $0 \leq t \leq 2\pi$  or over a subinterval of this range.

`ezplot(x,y,[tmin,tmax])` plots  $x = x(t)$  and  $y = y(t)$  over the specified range  $tmin \leq t \leq tmax$ .

`ezplot(f,[min,max],fig)` plots  $f$  over the specified range in the figure with the figure number or figure handle `fig`. The title of each plot window contains the word Figure and the number, for example, **Figure 1**, **Figure 2**, and so on. If `fig` is already open, `ezplot` overwrites the content of that figure with the new plot.

`ezplot(f,[xmin,xmax,ymin,ymax],fig)` plots  $f$  over the specified ranges along the abscissa and the ordinate in `fig`.

`ezplot(x,y,[tmin,tmax],fig)` plots  $x = x(t)$  and  $y = y(t)$  over the specified range in `fig`.

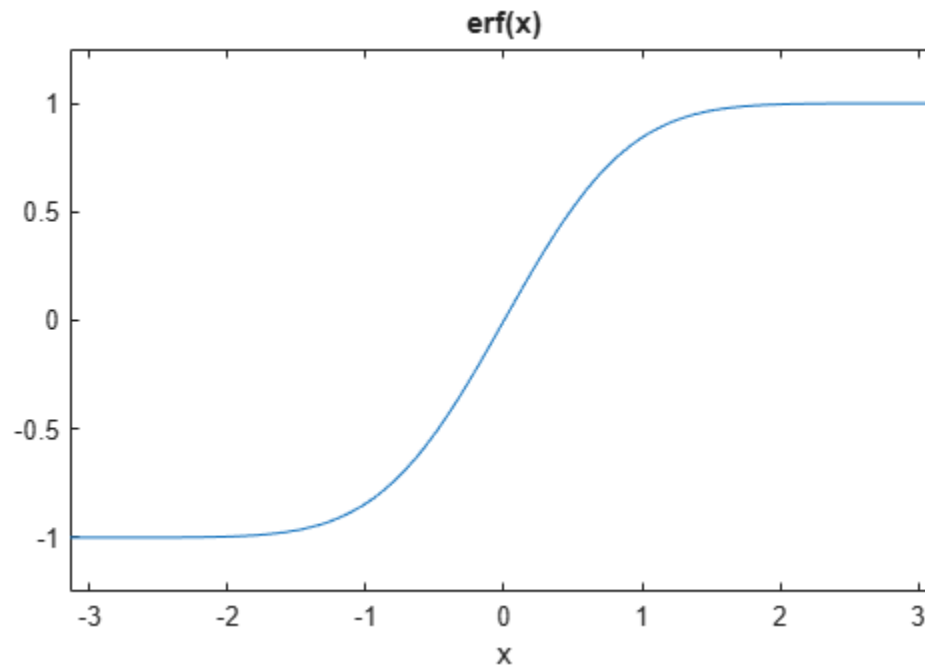
`h = ezplot( __ )` returns the plot handle as either a chart line or contour object.

## Examples

### Plot Over Particular Range

Plot the expression  $\text{erf}(x) \cdot \sin(x)$  over the range  $[-\pi, \pi]$ :

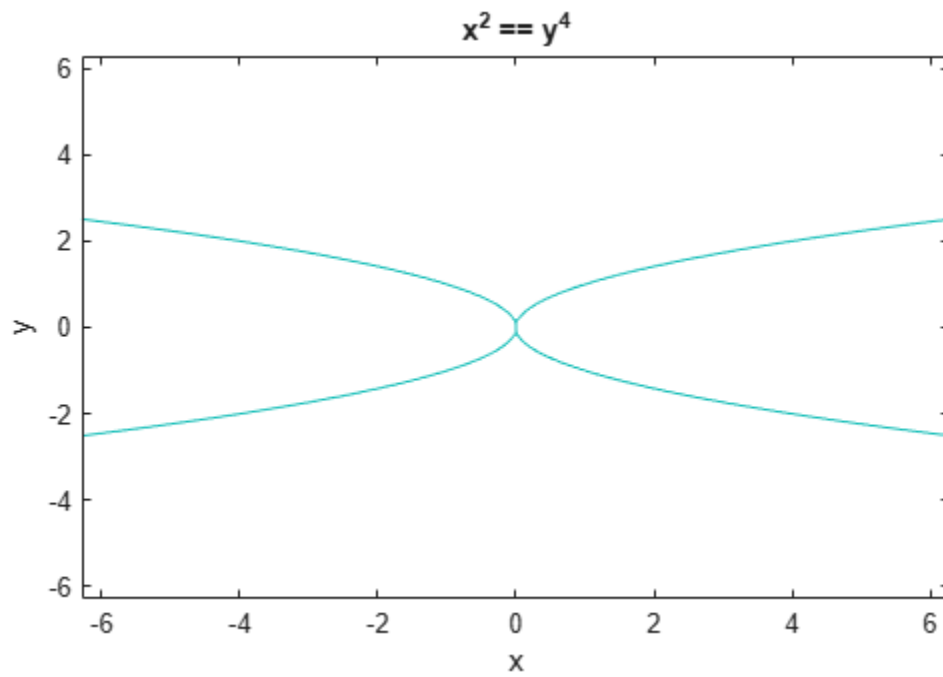
```
syms x
ezplot(erf(x), [-pi, pi])
```



### Plot Over Default Range

Plot this equation over the default range.

```
syms x y
ezplot(x^2 == y^4)
```



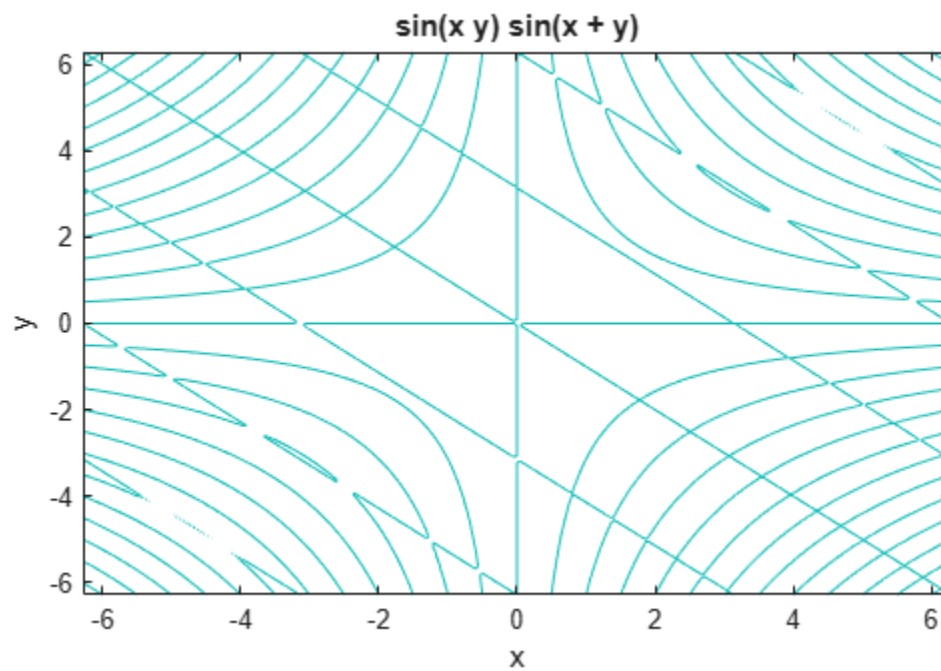
### Plot Symbolic Function

Create this symbolic function  $f(x, y)$ :

```
syms x y
f(x, y) = sin(x + y)*sin(x*y);
```

Plot this function over the default range:

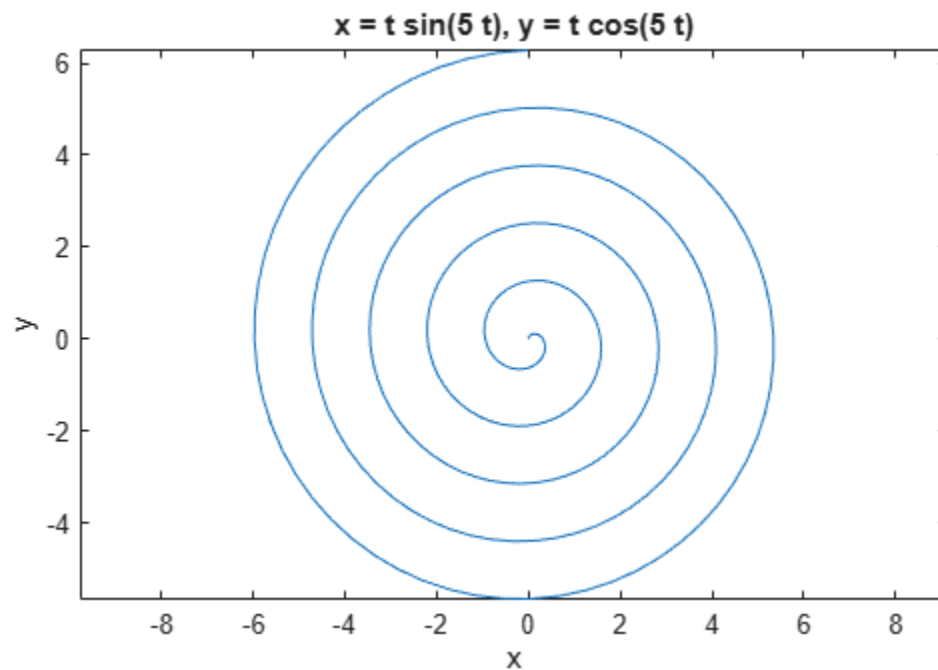
```
ezplot(f)
```



### Plot Parametric Curve

Plot this parametric curve:

```
syms t
x = t*sin(5*t);
y = t*cos(5*t);
ezplot(x, y)
```



## Input Arguments

### **f — Input**

symbolic expression | symbolic equation | symbolic function

Symbolic expression, equation, or function.

### **[min,max] — Numbers specifying the plotting range**

$[-2\pi, 2\pi]$  (default) | vector | symbolic vector

Numbers specifying the plotting range, specified as a vector of length 2. For a univariate expression or function, the plotting range applies to that variable. For an equation or function of two variables, the plotting range applies to both variables. In this case, the range is the same for the abscissa and the ordinate.

### **[xmin,xmax,ymin,ymax] — Numbers specifying the plotting range**

$[-2\pi, 2\pi, -2\pi, 2\pi]$  (default) | vector | symbolic vector

Numbers specifying the plotting range, specified as a numeric or symbolic vector along the abscissa (first two numbers) and the ordinate (last two numbers).

### **fig — Figure**

figure handle | figure window

Figure specified as a figure handle or figure window where you want to display a plot. For figure handle, the current figure handle returned by `gcf`. For figure number, if no plot windows are open, then 1. If one plot window is open, then the number in the title of that window. If more than one plot window is open, then the highest number in the titles of open windows.

### **x,y — Parametric curve**

symbolic expression | symbolic function

Parametric curve specified as symbolic expressions or functions  $x = x(t)$  and  $y = y(t)$ .

### **[tmin,tmax] — Plotting range for a parametric curve**

$[0, 2\pi]$  (default) | vector | symbolic vector

Plotting range for a parametric curve specified as a numeric or symbolic vector.

## Output Arguments

### **h — Chart line or contour line object**

scalar

Chart line or contour line object, returned as a scalar. For details, see [Chart Line and Contour](#).

## Tips

- If you do not specify a plot range, `ezplot` uses the interval  $[-2\pi, 2\pi]$  as a starting point. Then it can choose to display a part of the plot over a subinterval of  $[-2\pi, 2\pi]$  where the plot has significant variation. Also, when selecting the plotting range, `ezplot` omits extreme values associated with singularities.



- `ezplot` open a plot window and displays a plot there. If any plot windows are already open, `ezplot` does not create a new window. Instead, it displays the new plot in the currently active window. (Typically, it is the window with the highest number.) To display the new plot in a new plot window or in an existing window other than that with highest number, use `fig`.
- If `f` is an equation or function of two variables, then the alphabetically first variable defines the abscissa (horizontal axis) and the other variable defines the ordinate (vertical axis). Thus, `ezplot(x^2 == a^2, [-3,3,-2,2])` creates the plot of the equation  $x^2 = a^2$  with  $-3 \leq a \leq 3$  along the horizontal axis, and  $-2 \leq x \leq 2$  along the vertical axis.

## Version History

Introduced before R2006a

### See Also

`fcontour` | `fmesh` | `fplot` | `fplot3` | `fsurf` | `plot`

### Topics

“Create Plots of Symbolic Expressions” on page 4-2

## ezplot3

(Not recommended) 3-D parametric curve plotter

---

**Note** ezplot3 is not recommended. Use fplot3 instead.

---

### Syntax

```
ezplot3(x,y,z)
ezplot3(x,y,z,[tmin,tmax])
ezplot3(____,'animate')
```

### Description

ezplot3( $x,y,z$ ) plots the spatial curve  $x = x(t)$ ,  $y = y(t)$ , and  $z = z(t)$  over the default domain  $0 < t < 2\pi$ .

ezplot3( $x,y,z,[tmin,tmax]$ ) plots the curve  $x = x(t)$ ,  $y = y(t)$ , and  $z = z(t)$  over the domain  $tmin < t < tmax$ .

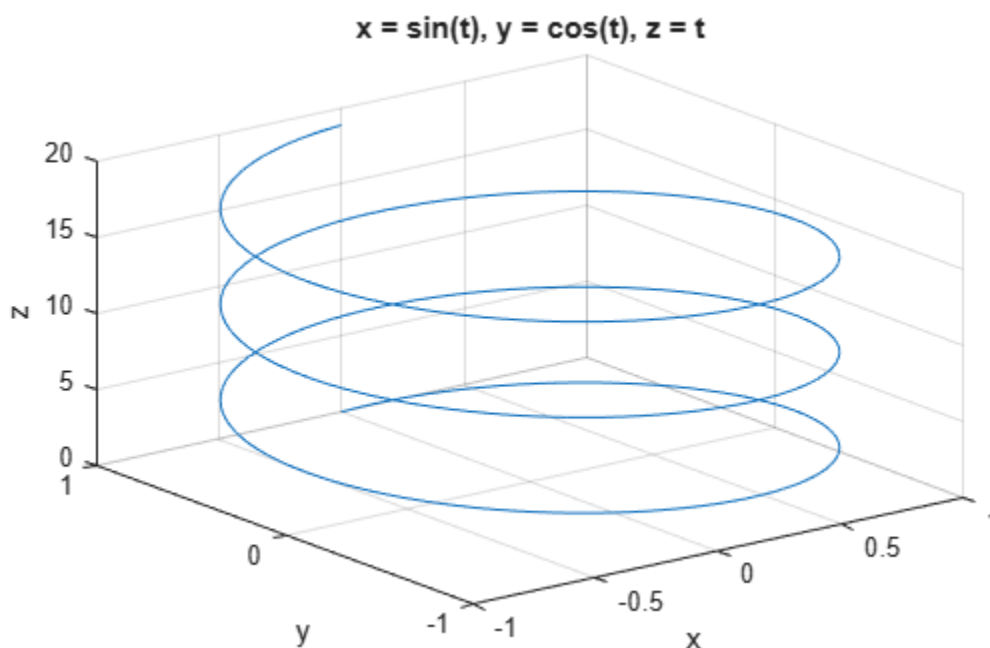
ezplot3(\_\_\_\_,'animate') produces an animated trace of the spatial curve.

### Examples

#### 3-D Parametric Curve

Plot the parametric curve  $x = \sin(t)$ ,  $y = \cos(t)$ ,  $z = t$  over the domain  $[0, 6\pi]$ .

```
syms t
ezplot3(sin(t), cos(t), t,[0,6*pi])
```



## Input Arguments

### **x — Input**

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **y — Input**

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **z — Input**

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **[tmin,tmax] — Domain to plot over**

symbolic vector

Domain to plot over, specified as a symbolic vector. `ezplot3` plots over the domain  $tmin < t < tmax$ .

## Version History

Introduced before R2006a

## See Also

`fcontour` | `fmesh` | `fplot` | `fplot3` | `fsurf` | `plot3`

## ezpolar

(Not recommended) Polar coordinate plotter

---

**Note** `ezpolar` is not recommended. Use `fpolarplot` instead.

---

### Syntax

```
ezpolar(f)
ezpolar(f, [a b])
```

### Description

`ezpolar(f)` plots the polar curve  $r = f(\theta)$  over the default domain  $0 < \theta < 2\pi$ .

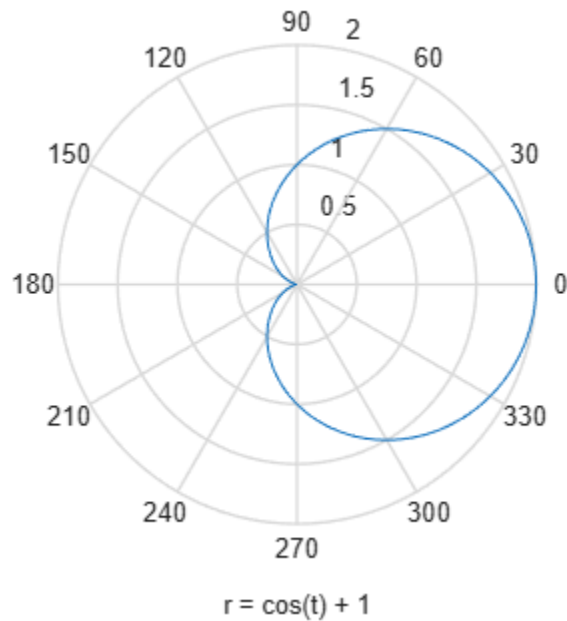
`ezpolar(f, [a b])` plots  $f$  for  $a < \theta < b$ .

### Examples

#### Polar Plot of Symbolic Expression

Create a polar plot of the function  $r(t) = 1 + \cos(t)$  over the domain  $0 < t < 2\pi$ .

```
syms t
ezpolar(1 + cos(t))
```



## Input Arguments

### **f** — Input

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **[a b]** — Angle to plot over

vector | symbolic vector

Angle to plot over, specified as a vector, or a symbolic vector.

## Version History

**Introduced before R2006a**

### **R2024a: Not recommended**

*Not recommended starting in R2024a*

The `ezpolar` function is not recommended. However, there are no plans to remove the function. To plot a symbolic expression or function in polar coordinates, use `fpolarplot` instead.

The `fpolarplot` function has these advantages over `ezpolar`:

- Unlike `ezpolar`, `fpolarplot` uses additional evaluation points when needed for more accurate function plotting.
- With `fpolarplot`, you can specify an argument to customize the line style, marker symbol, and color.
- `fpolarplot` uses a `PolarAxes` object, which you can customize by setting the `PolarAxes` Properties. For example, you can change the range of angle values or choose whether to display angles in radians or degrees.

This table shows some typical usages of `ezpolar` and how to update your code to use `fpolarplot` instead. To display the plotted expression or function as `ezpolar` does, use the `title` function.

| Not Recommended                            | Recommended                                                       |
|--------------------------------------------|-------------------------------------------------------------------|
| <pre>syms t ezpolar(1 + cos(t))</pre>      | <pre>syms t fpolarplot(1 + cos(t)) title("r = 1 + cos(t)")</pre>  |
| <pre>syms t ezpolar(cos(t),[0 pi/4])</pre> | <pre>syms t fpolarplot(cos(t),[0 pi/4]) title("r = cos(t)")</pre> |

## ezsurf

(Not recommended) Plot 3-D surface

---

**Note** ezsurf is not recommended. Use fsurf instead.

---

### Syntax

```
ezsurf(f)
ezsurf(f,[xmin,xmax])
ezsurf(f,[xmin,xmax,ymin,ymax])

ezsurf(x,y,z)
ezsurf(x,y,z,[smin,smax])
ezsurf(x,y,z,[smin,smax,tmin,tmax])

ezsurf(____,n)
ezsurf(____,'circ')

h = ezsurf(____)
```

### Description

ezsurf(f) plots a two-variable symbolic expression or function  $f(x,y)$  over the range  $-2\pi < x < 2\pi$ ,  $-2\pi < y < 2\pi$ .

ezsurf(f,[xmin,xmax]) plots  $f(x,y)$  over the specified range  $x_{\min} < x < x_{\max}$ . This is the range along the abscissa (horizontal axis).

ezsurf(f,[xmin,xmax,ymin,ymax]) plots  $f(x,y)$  over the specified ranges along the abscissa,  $x_{\min} < x < x_{\max}$ , and the ordinate,  $y_{\min} < y < y_{\max}$ .

When determining the range values, ezsurf sorts variables alphabetically. For example, ezsurf( $x^2 - a^3$ , [0,1,3,6]) plots  $x^2 - a^3$  over  $0 < a < 1$ ,  $3 < x < 6$ .

ezsurf(x,y,z) plots the parametric surface  $x = x(s,t)$ ,  $y = y(s,t)$ ,  $z = z(s,t)$  over the range  $-2\pi < s < 2\pi$ ,  $-2\pi < t < 2\pi$ .

ezsurf(x,y,z,[smin,smax]) plots the parametric surface  $x = x(s,t)$ ,  $y = y(s,t)$ ,  $z = z(s,t)$  over the specified range  $s_{\min} < s < s_{\max}$ .

ezsurf(x,y,z,[smin,smax,tmin,tmax]) plots the parametric surface  $x = x(s,t)$ ,  $y = y(s,t)$ ,  $z = z(s,t)$  over the specified ranges  $s_{\min} < s < s_{\max}$  and  $t_{\min} < t < t_{\max}$ .

ezsurf(\_\_\_\_,n) specifies the grid. You can specify  $n$  after the input arguments in any of the previous syntaxes. By default,  $n = 60$ .

ezsurf(\_\_\_\_,'circ') creates the surface plot over a disk centered on the range. You can specify 'circ' after the input arguments in any of the previous syntaxes.

`h = ezsurf( __ )` returns a handle `h` to the surface plot object. You can use the output argument `h` with any of the previous syntaxes.

## Examples

### Plot Function Over Default Range

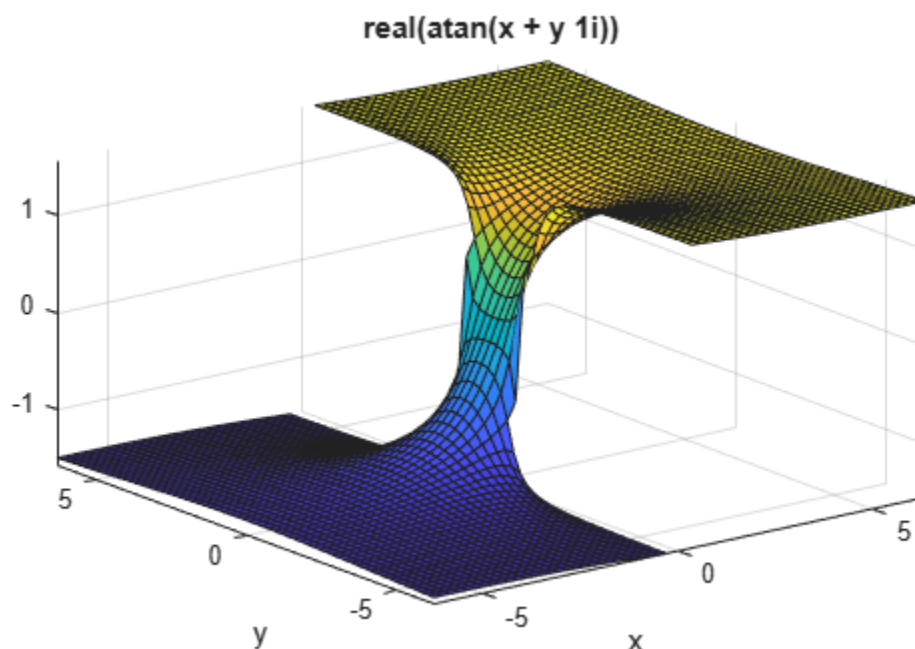
Plot the symbolic function  $f(x,y) = \text{real}(\text{atan}(x + i*y))$  over the default range  $-2\pi < x < 2\pi$ ,  $-2\pi < y < 2\pi$ .

Create the symbolic function.

```
syms f(x,y)
f(x,y) = real(atan(x + i*y));
```

Plot this function using `ezsurf`.

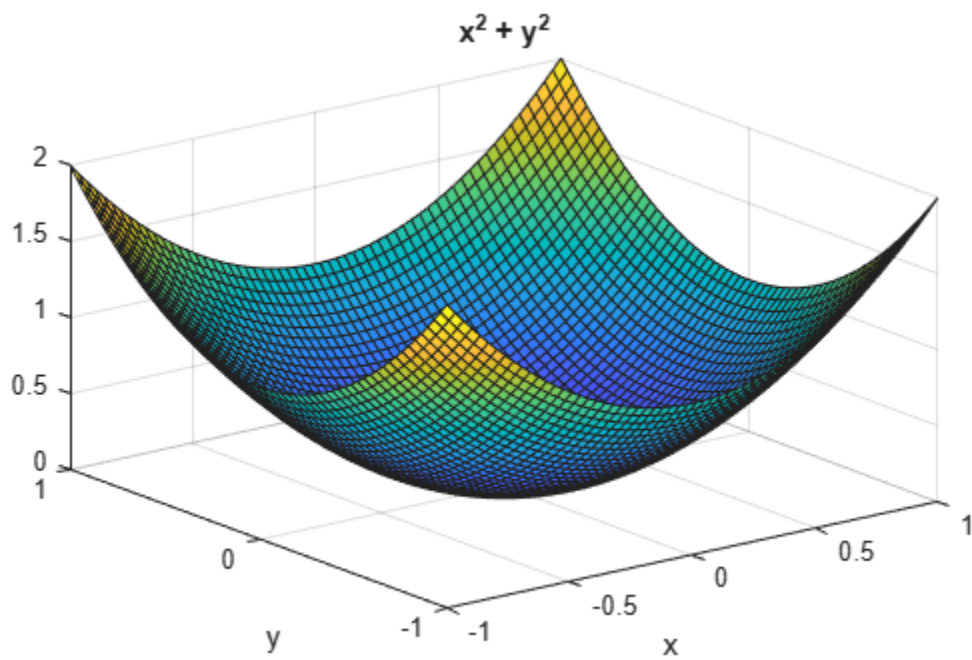
```
ezsurf(f)
```



### Specify Plotting Ranges

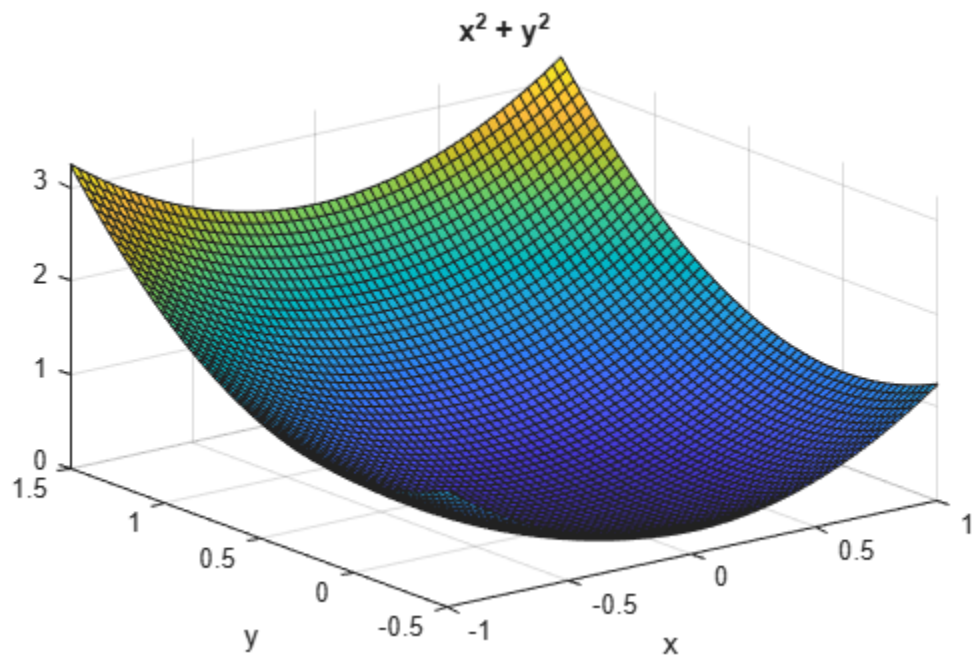
Plot the symbolic expression  $x^2 + y^2$  over the range  $-1 < x < 1$ . Because you do not specify the range for the  $y$ -axis, `ezsurf` chooses it automatically.

```
syms x y
ezsurf(x^2 + y^2, [-1, 1])
```



Specify the range for both axes.

```
ezsurf(x^2 + y^2, [-1, 1, -0.5, 1.5])
```





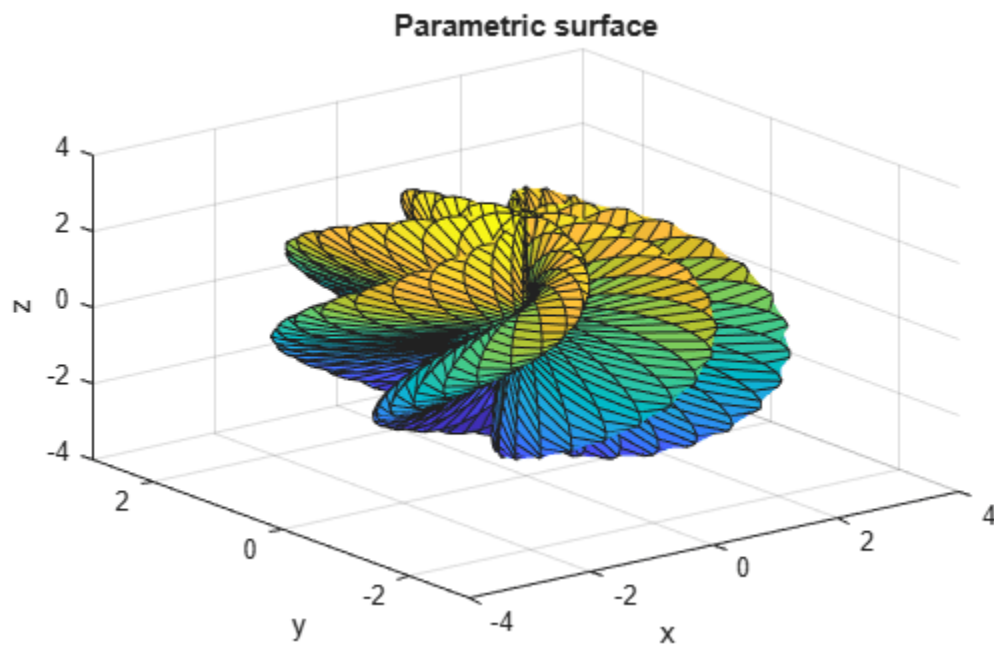
### Plot Parameterized Surface

Define the parametric surface  $x(s,t)$ ,  $y(s,t)$ ,  $z(s,t)$  as follows.

```
syms s t
r = 2 + sin(7*s + 5*t);
x = r*cos(s)*sin(t);
y = r*sin(s)*sin(t);
z = r*cos(t);
```

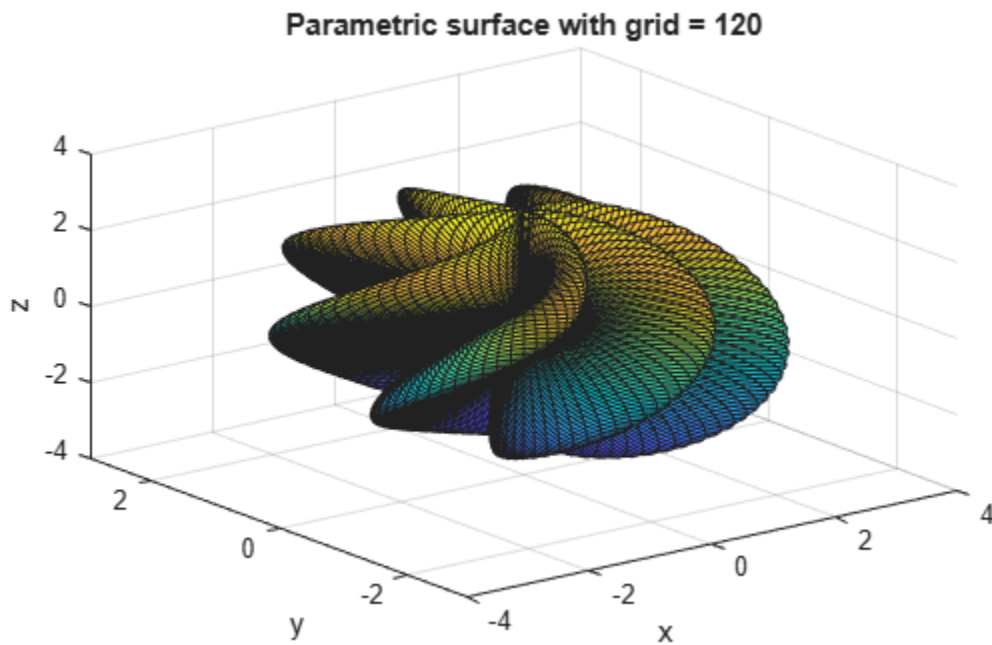
Plot the function using ezsurf.

```
ezsurf(x, y, z, [0, 2*pi, 0, pi])
title('Parametric surface')
```



To create a smoother plot, increase the number of mesh points.

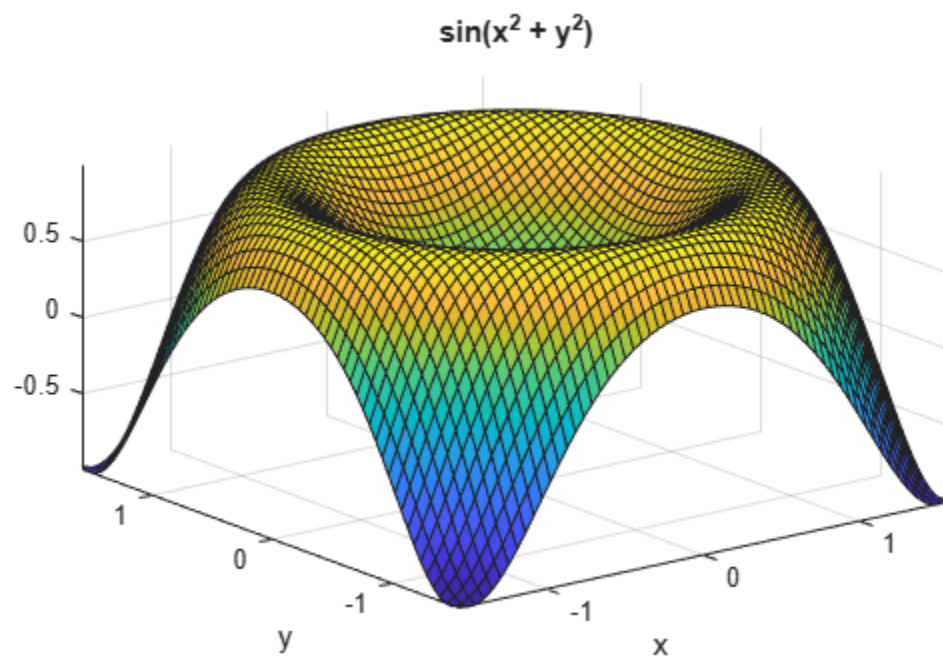
```
ezsurf(x, y, z, [0, 2*pi, 0, pi], 120)
title('Parametric surface with grid = 120')
```



### Specify Disk Plotting Range

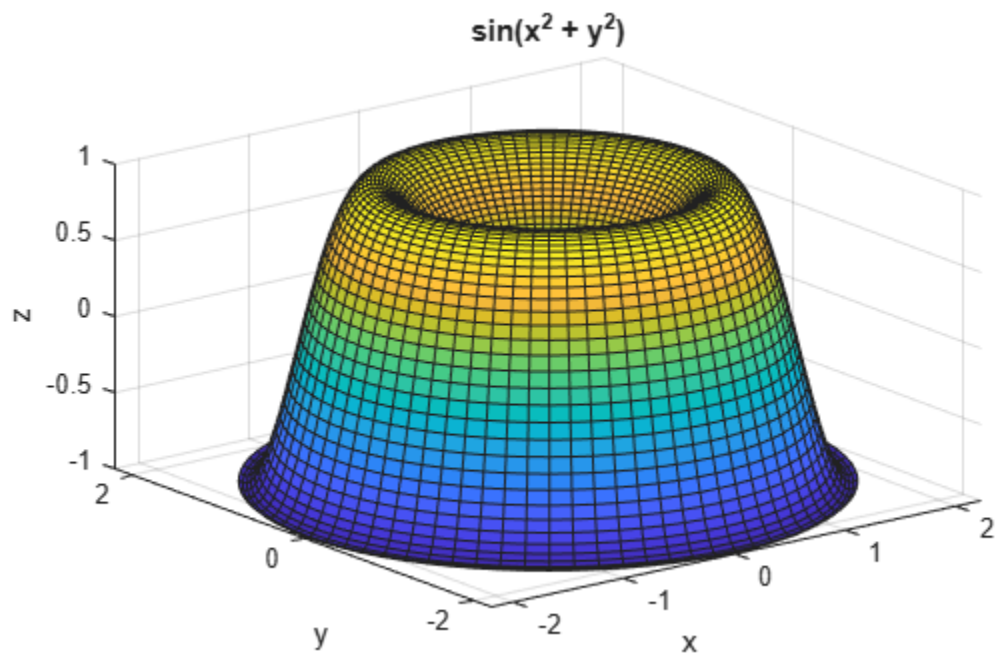
First, plot the expression  $\sin(x^2 + y^2)$  over the square range  $-\pi/2 < x < \pi/2$ ,  $-\pi/2 < y < \pi/2$ .

```
syms x y
ezsurf(sin(x^2 + y^2), [-pi/2, pi/2, -pi/2, pi/2])
```



Now, plot the same expression over the disk range.

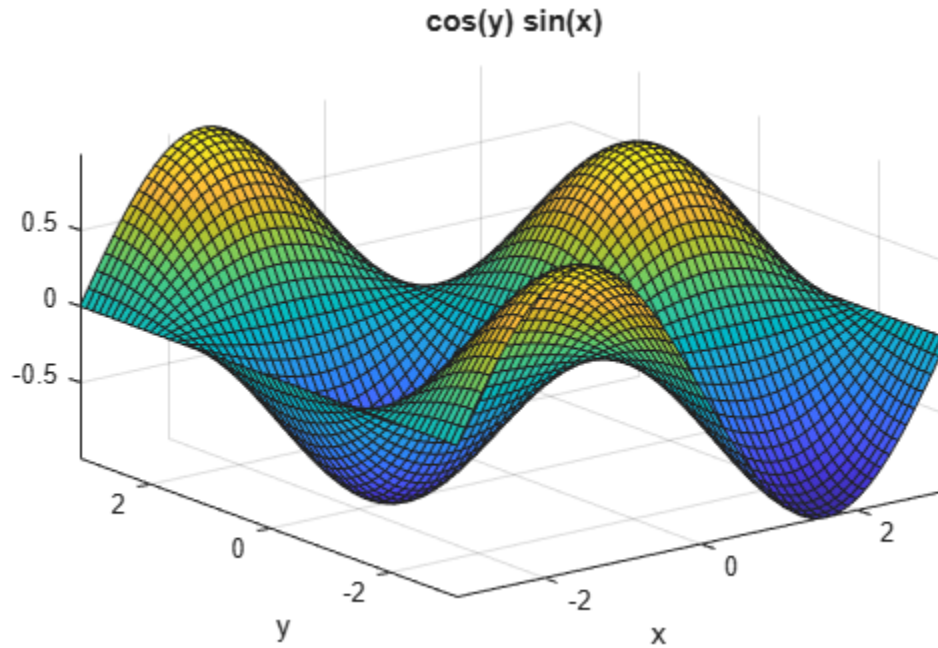
```
ezsurf(sin(x^2 + y^2), [-pi/2, pi/2, -pi/2, pi/2], 'circ')
```



### Use Handle to Surface Plot

Plot the symbolic expression  $\sin(x) \cos(y)$ , and assign the result to the handle `h`.

```
syms x y
h = ezsurf(sin(x)*cos(y), [-pi, pi])
```



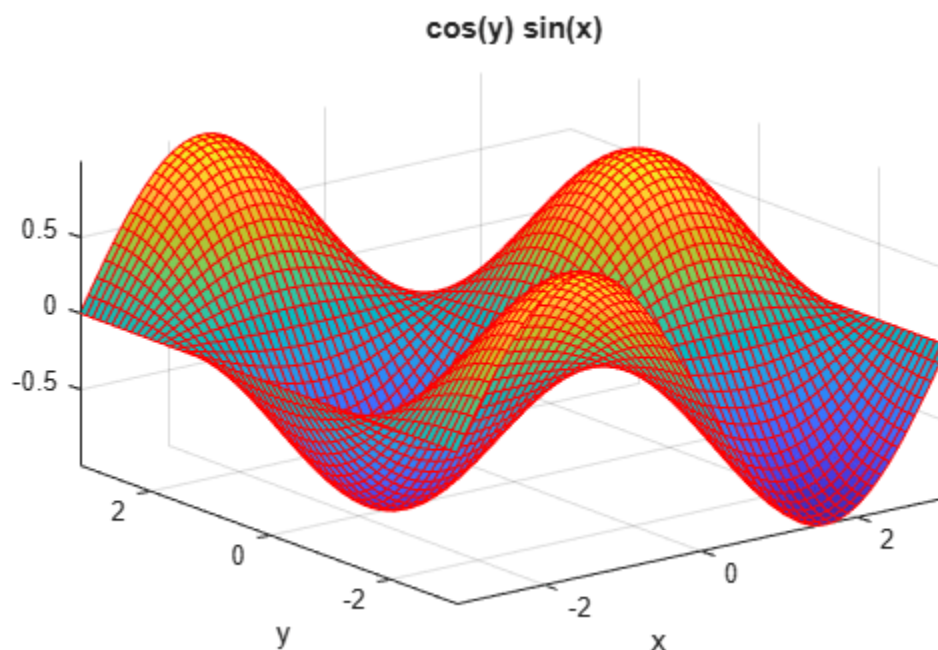
```
h =
Surface with properties:

    EdgeColor: [0.1294 0.1294 0.1294]
    LineStyle: '-'
    FaceColor: 'flat'
    FaceLighting: 'flat'
    FaceAlpha: 1
        XData: [60x60 double]
        YData: [60x60 double]
        ZData: [60x60 double]
        CData: [60x60 double]
```

Show all properties

You can use this handle to change properties of the plot. For example, change the color of the area outline.

```
h.EdgeColor = 'red'
```



```
h =
  Surface with properties:
    EdgeColor: [1 0 0]
    LineStyle: '-'
    FaceColor: 'flat'
    FaceLighting: 'flat'
    FaceAlpha: 1
    XData: [60x60 double]
    YData: [60x60 double]
    ZData: [60x60 double]
    CData: [60x60 double]
```

Show all properties

## Input Arguments

### **f — Function to plot**

symbolic expression with two variables | symbolic function of two variables

Function to plot, specified as a symbolic expression or function of two variables.

Example: `ezsurf(x^2 + y^2)`

### **x,y,z — Parametric function to plot**

three symbolic expressions with two variables | three symbolic functions of two variables

Parametric function to plot, specified as three symbolic expressions or functions of two variables.

Example: `ezsurf(s*cos(t), s*sin(t), t)`

**n — Grid value**

integer

Grid value, specified as an integer. The default grid value is 60.

**Output Arguments****h — Surface plot handle**

scalar

Surface plot handle, returned as a scalar. It is a unique identifier, which you can use to query and modify properties of the surface plot.

**Tips**

- `ezsurf` chooses the computational grid according to the amount of variation that occurs. If `f` is singular for some points on the grid, then `ezsurf` omits these points. The value at these points is set to NaN.

**Version History**

Introduced before R2006a

**See Also**

`fcontour` | `fmesh` | `fplot` | `fplot3` | `fsurf` | `surf`

**Topics**

“Create Plots of Symbolic Expressions” on page 4-2

# ezsurf

(Not recommended) Combined surface and contour plotter

---

**Note** ezsurf is not recommended. Use fsurf instead.

---

## Syntax

```
ezsurf(f)
ezsurf(f, domain)
ezsurf(x, y, z)
ezsurf(x, y, z, domain)
ezsurf( __, n)
ezsurf( __, 'circ')
```

## Description

`ezsurf(f)` creates a graph of  $f(x,y)$ , where  $f$  is a symbolic expression that represents a mathematical function of two variables, such as  $x$  and  $y$ . The function  $f$  is plotted over the default domain  $-2\pi < x < 2\pi$ ,  $-2\pi < y < 2\pi$ . MATLAB software chooses the computational grid according to the amount of variation that occurs; if the function  $f$  is not defined (singular) for points on the grid, then these points are not plotted.

`ezsurf(f, domain)` plots  $f$  over the specified domain. `domain` can be either a 4-by-1 vector  $[xmin, xmax, ymin, ymax]$  or a 2-by-1 vector  $[min, max]$  (where,  $min < x < max$ ,  $min < y < max$ ).

If  $f$  is a function of the variables  $u$  and  $v$  (rather than  $x$  and  $y$ ), then the domain endpoints  $umin$ ,  $umax$ ,  $vmin$ , and  $vmax$  are sorted alphabetically. Thus, `ezsurf(u^2 - v^3, [0,1], [3,6])` plots  $u^2 - v^3$  over  $0 < u < 1$ ,  $3 < v < 6$ .

`ezsurf(x, y, z)` plots the parametric surface  $x = x(s,t)$ ,  $y = y(s,t)$ , and  $z = z(s,t)$  over the square  $-2\pi < s < 2\pi$ ,  $-2\pi < t < 2\pi$ .

`ezsurf(x, y, z, domain)` plots the parametric surface using the specified domain.

`ezsurf( __, n)` plots  $f$  over the default domain using an  $n$ -by- $n$  grid. The default value for  $n$  is 60.

`ezsurf( __, 'circ')` plots  $f$  over a disk centered on the domain.

## Examples

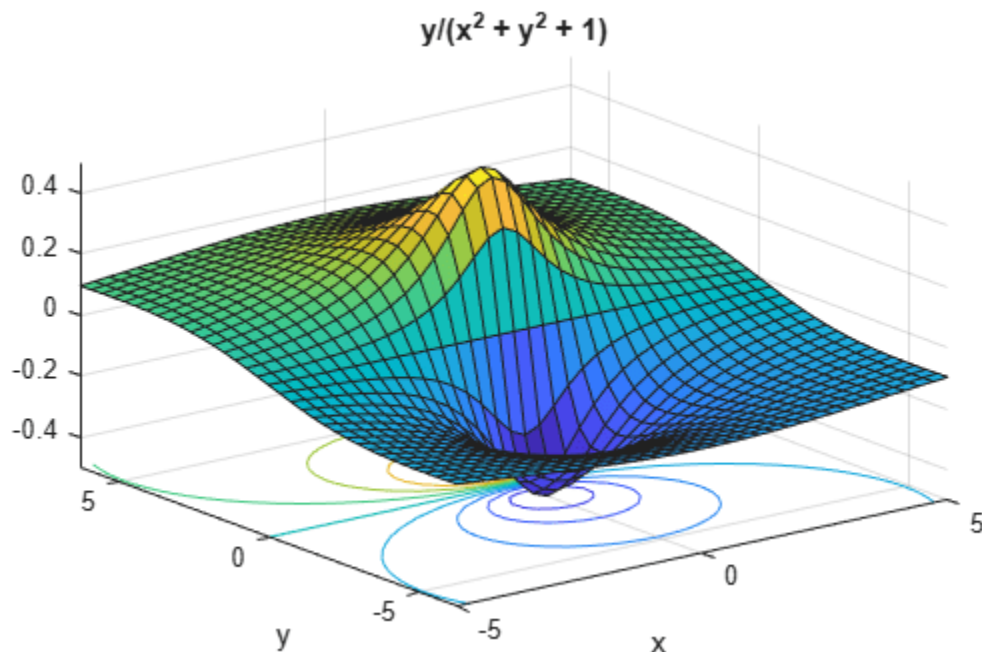
### 3-D Surface Plot with Contours

Create a surface/contour plot of the expression,

$$f(x, y) = \frac{y}{1 + x^2 + y^2},$$

over the domain  $-5 < x < 5$ ,  $-2\pi < y < 2\pi$ , with a computational grid of size 35-by-35. Use the mouse to rotate the axes to better observe the contour lines (this picture uses a view of azimuth = -65 and elevation = 26).

```
syms x y
ezsurf(y/(1 + x^2 + y^2), [-5,5, -2*pi,2*pi], 35)
```



## Input Arguments

### **f** — Input

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **x** — Input

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **y** — Input

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **z** — Input

symbolic function | symbolic expression

Input, specified as a symbolic function or expression.

### **domain** — Domain to plot over

symbolic vector



Domain to plot over, specified as a symbolic vector. `domain` is a 4-by-1 vector  $[xmin, xmax, ymin, ymax]$  or a 2-by-1 vector  $[min, max]$  (where,  $min < x < max$ ,  $min < y < max$ ). If  $f$  is a function of the variables  $u$  and  $v$  (rather than  $x$  and  $y$ ), then the domain endpoints  $umin$ ,  $umax$ ,  $vmin$ , and  $vmax$  are sorted alphabetically.

**n — grid points**

number | symbolic number

Grid points, specified as a number or a symbolic number.

## Version History

Introduced before R2006a

**See Also**

`fcontour` | `fmesh` | `fplot` | `fplot3` | `fsurf` | `surf`

## factor

Factorization

### Syntax

```
F = factor(x)
F = factor(x,vars)
F = factor( ____,Name,Value)
```

### Description

`F = factor(x)` returns all irreducible factors of `x` in vector `F`. If `x` is an integer, `factor` returns the prime factorization of `x`. If `x` is a symbolic expression, `factor` returns the subexpressions that are factors of `x`.

`F = factor(x,vars)` returns an array of factors `F`, where `vars` specifies the variables of interest. All factors not containing a variable in `vars` are separated into the first entry `F(1)`. The other entries are irreducible factors of `x` that contain one or more variables from `vars`.

`F = factor( ____,Name,Value)` uses additional options specified by one or more `Name,Value` pair arguments. This syntax can use any of the input arguments from the previous syntaxes.

### Examples

#### Factor Integer Numbers

```
F = factor(823429252)

F =
      2      2      59      283      12329
```

To factor integers greater than `flintmax`, convert the integer to a symbolic object using `sym`. Then place the number in quotation marks to represent it accurately.

```
F = factor(sym('82342925225632328'))

F =
[ 2, 2, 2, 251, 401, 18311, 5584781]
```

To factor a negative integer, convert it to a symbolic object using `sym`.

```
F = factor(sym(-92465))

F =
[ -1, 5, 18493]
```

#### Perform Prime Factorization of Large Numbers

Perform prime factorization for 41758540882408627201. Since the integer is greater than `flintmax`, convert it to a symbolic object using `sym`, and place the number in quotation marks to represent it accurately.

```
n = sym('41758540882408627201');
factor(n)
```

```
ans =
[ 479001599, 87178291199]
```

### Factor Symbolic Fractions

Factor the fraction  $112/81$  by converting it into a symbolic object using `sym`.

```
F = factor(sym(112/81))

F =
[ 2, 2, 2, 2, 7, 1/3, 1/3, 1/3, 1/3]
```

### Factor Polynomials

Factor the polynomial  $x^6 - 1$ .

```
syms x
F = factor(x^6-1)

F =
[ x - 1, x + 1, x^2 + x + 1, x^2 - x + 1]
```

Factor the polynomial  $y^6 - x^6$ .

```
syms y
F = factor(y^6-x^6)

F =
[ -1, x - y, x + y, x^2 + x*y + y^2, x^2 - x*y + y^2]
```

### Separate Factors Containing Specified Variables

Factor  $y^2x^2$  for factors containing  $x$ .

```
syms x y
F = factor(y^2*x^2,x)

F =
[ y^2, x, x]
```

`factor` combines all factors without  $x$  into the first element. The remaining elements of  $F$  contain irreducible factors that contain  $x$ .

Factor the polynomial  $y$  for factors containing symbolic variables  $b$  and  $c$ .

```
syms a b c d
y = -a*b^5*c*d*(a^2 - 1)*(a*d - b*c);
F = factor(y,[b c])

F =
[ -a*d*(a - 1)*(a + 1), b, b, b, b, b, c, a*d - b*c]
```

`factor` combines all factors without  $b$  or  $c$  into the first element of  $F$ . The remaining elements of  $F$  contain irreducible factors of  $y$  that contain either  $b$  or  $c$ .

### Choose Factorization Modes

Use the `FactorMode` argument to choose a particular factorization mode.

Factor an expression without specifying the factorization mode. By default, `factor` uses factorization over rational numbers. In this mode, `factor` keeps rational numbers in their exact symbolic form.

```
syms x
factor(x^3 + 2, x)

ans =
x^3 + 2
```

Factor the same expression, but this time use numeric factorization over real numbers. This mode factors the expression into linear and quadratic irreducible polynomials with real coefficients and converts all numeric values to floating-point numbers.

```
factor(x^3 + 2, x, 'FactorMode', 'real')

ans =
[ x + 1.2599210498948731647672106072782, ...
  x^2 - 1.2599210498948731647672106072782*x + 1.5874010519681994747517056392723]
```

Factor this expression using factorization over complex numbers. In this mode, `factor` reduces quadratic polynomials to linear expressions with complex coefficients. This mode converts all numeric values to floating-point numbers.

```
factor(x^3 + 2, x, 'FactorMode', 'complex')

ans =
[ x + 1.2599210498948731647672106072782, ...
  x - 0.62996052494743658238360530363911 + 1.0911236359717214035600726141898i, ...
  x - 0.62996052494743658238360530363911 - 1.0911236359717214035600726141898i]
```

Factor this expression using the full factorization mode. This mode factors the expression into linear expressions, reducing quadratic polynomials to linear expressions with complex coefficients. This mode keeps rational numbers in their exact symbolic form.

```
factor(x^3 + 2, x, 'FactorMode', 'full')

ans =
[ x + 2^(1/3), ...
  x - 2^(1/3)*((3^(1/2)*1i)/2 + 1/2), ...
  x + 2^(1/3)*((3^(1/2)*1i)/2 - 1/2)]
```

Approximate the result with floating-point numbers by using `vpa`. Because the expression does not contain any symbolic parameters besides the variable `x`, the result is the same as in complex factorization mode.

```
vpa(ans)

ans =
[ x + 1.2599210498948731647672106072782, ...
  x - 0.62996052494743658238360530363911 - 1.0911236359717214035600726141898i, ...
  x - 0.62996052494743658238360530363911 + 1.0911236359717214035600726141898i]
```

## Approximate Results Containing RootOf

In the full factorization mode, `factor` also can return results as a symbolic sums over polynomial roots expressed as `RootOf`.

Factor this expression.

```
syms x
s = factor(x^3 + x - 3, x, 'FactorMode','full')

s =
[ x - root(z^3 + z - 3, z, 1),...
  x - root(z^3 + z - 3, z, 2),...
  x - root(z^3 + z - 3, z, 3)]
```

Approximate the result with floating-point numbers by using `vpa`.

```
vpa(s)

ans =
[ x - 1.2134116627622296341321313773815,...
  x + 0.60670583138111481706606568869074 + 1.450612249188441526515442203395i,...
  x + 0.60670583138111481706606568869074 - 1.450612249188441526515442203395i]
```

## Input Arguments

### x — Input to factor

number | symbolic number | symbolic expression | symbolic function

Input to factor, specified as a number, or a symbolic number, expression, or function.

### vars — Variables of interest

symbolic variable | vector of symbolic variables

Variables of interest, specified as a symbolic variable or a vector of symbolic variables. Factors that do not contain a variable specified in `vars` are grouped into the first element of `F`. The remaining elements of `F` contain irreducible factors of `x` that contain a variable in `vars`.

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1,...,NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `factor(x^3 - 2,x,'FactorMode','real')`

### FactorMode — Factorization mode

'rational' (default) | 'real' | 'complex' | 'full'

Factorization mode, specified as the comma-separated pair consisting of 'FactorMode' and one of these character vectors.

|            |                                      |
|------------|--------------------------------------|
| 'rational' | Factorization over rational numbers. |
|------------|--------------------------------------|

|           |                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 'real'    | Factorization over real numbers. A real numeric factorization is a factorization into linear and quadratic irreducible polynomials with real coefficients. This factorization mode requires the coefficients of the input to be convertible to real floating-point numbers. All other inputs (for example, those inputs containing symbolic or complex coefficients) are treated as irreducible. |
| 'complex' | Factorization over complex numbers. A complex numeric factorization is a factorization into linear factors whose coefficients are floating-point numbers. Such factorization is only available if the coefficients of the input are convertible to floating-point numbers, that is, if the roots can be determined numerically. Symbolic inputs are treated as irreducible.                      |
| 'full'    | Full factorization. A full factorization is a symbolic factorization into linear factors. The result shows these factors using radicals or as a symsum ranging over a RootOf.                                                                                                                                                                                                                    |

Output Arguments

F — Factors of input

symbolic vector

Factors of input, returned as a symbolic vector.

Tips

- To factor an integer greater than `flintmax`, wrap the integer with `sym`. Then place the integer in quotation marks to represent it accurately, for example, `sym('465971235659856452')`.
- To factor a negative integer, wrap the integer with `sym`, for example, `sym(-3)`.

Version History

Introduced before R2006a

See Also

`collect` | `combine` | `divisors` | `expand` | `horner` | `numden` | `rewrite` | `simplify` | `simplifyFraction`

Topics

“Choose Function to Rearrange Expression” on page 3-123

# factorial

Factorial of symbolic input

## Syntax

```
f = factorial(n)
```

## Description

`f = factorial(n)` returns the factorial on page 6-465 of `n`. If `n` is an array, `factorial` acts element-wise on `n`.

## Examples

### Factorial for Symbolic Number

Compute the factorial for a symbolic number.

```
f = factorial(sym(20))
f = 2432902008176640000
```

### Compute Factorial Function for Symbolic Expression

Compute the factorial function for a symbolic expression. `factorial` returns exact symbolic output as the function call.

```
syms n
expr = n^2 + 1;
f = factorial(expr)
```

```
f = (n^2 + 1)!
```

Calculate the factorial for a value of `n = 3`. Substitute the value of `n` by using `subs`.

```
fVal = subs(f,n,3)
fVal = 3628800
```

### Differentiate Factorial Function

Differentiate an expression containing the factorial function  $(n^2 + n + 1)!$

```
syms n
f = factorial(n^2 + n + 1)
```

$$f = (n^2 + n + 1)!$$

```
df = diff(f)
```

$$df = (n^2 + n + 1)! \psi(n^2 + n + 2) (2n + 1)$$

The derivative of the factorial function is expressed in terms of the `psi` function.

### Expand Factorial Function

Expand an expression containing the factorial function.

```
syms n
f = factorial(n^2 + n + 1);
f1 = expand(f)
```

$$f1 = (n^2 + n)! (n^2 + n + 1)$$

### Limit of Factorial Function

Compute the limit at infinity for an expression containing the factorial function.

```
syms n
f = factorial(n)/exp(n);
fLim = limit(f,n,Inf)
```

```
fLim = ∞
```

### Compute Factorial for Array Input

Compute factorial for array input. `factorial` acts element-wise on array input.

```
A = sym([1 3; 4 5]);
f = factorial(A)
```

$$f = \begin{pmatrix} 1 & 6 \\ 24 & 120 \end{pmatrix}$$

## Input Arguments

### n — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.



## More About

### Factorial Function

The factorial of a number  $n$  is defined as follows.

$$n! = \prod_{k=1}^n k$$

The factorial of 0 is 1.

### Tips

- Calling `factorial` for a number that is not a symbolic object invokes the MATLAB `factorial` function.

## Version History

**Introduced in R2012a**

### See Also

`beta` | `gamma` | `nchoosek` | `psi`

## factorIntegerPower

Perfect power factoring

### Syntax

```
x = factorIntegerPower(n)
[x,k] = factorIntegerPower(n)
```

### Description

`x = factorIntegerPower(n)` factors the number `n` into its perfect power  $x^k$  and returns the base `x`. If several perfect powers exist, `x` is returned for maximum `k`. The function `factorIntegerPower` acts element-wise on array input.

`[x,k] = factorIntegerPower(n)` returns both the base `x` and power `k`.

### Examples

#### Factor Integer into Perfect Power

Factor 64 into its perfect power. If several perfect powers exist for a number, the maximum `k` is returned.

```
n = 64;
[x,k] = factorIntegerPower(n)
```

```
x =
    2
k =
    6
```

Find perfect powers of 7, 841, and 2541865828329.

```
n = [7 841 2541865828329];
[x,k] = factorIntegerPower(n)
```

```
x =
    7    29    3
k =
    1     2   26
```

Reconstruct the numbers. Return exact symbolic integers instead of floating point by converting `x` to symbolic form.

```
sym(x).^k
```

```
ans =
[ 7, 841, 2541865828329]
```

### Test if Number Is Perfect Power

If a number is not a perfect power, `factorIntegerPower` returns the number itself as the base with exponent 1. So, a number is a perfect power if it does not equal its base.

Check if 125 is a perfect power. `isequal` returns logical 0 (false), meaning 125 is not equal to the returned base. Therefore, 125 is a perfect power.

```
n = 125;
isequal(n, factorIntegerPower(n))

ans =
    logical
     0
```

### Input Arguments

#### **n — Input**

number | vector | matrix | array | symbolic number | symbolic array

Input, specified as a number, vector, matrix, array, or a symbolic number or array. `n` must be a positive integer.

### Output Arguments

#### **x — Base in perfect power**

number | vector | matrix | array | symbolic number | symbolic array

Base in perfect power, returned as a number, vector, matrix, array, or a symbolic number or array.

#### **k — Power in perfect power**

number | vector | matrix | array | symbolic number | symbolic array

Power in perfect power, returned as a number, vector, matrix, array, or a symbolic number or array.

## Version History

Introduced in R2018a

### See Also

`factor`

## fanimator

Create stop-motion animation object

### Syntax

```
fanimator(f)
fanimator(f, args)

fanimator( ____, Name, Value)
fanimator(ax, ____)
fp = fanimator( ____)
```

### Description

`fanimator(f)` creates a stop-motion animation object from the function `f`. The function `f` must return graphics objects that depend on only one variable. This variable defines the time parameter of the animation.

By default, `fanimator` creates stop-motion frames of `f(t0)`, generating 10 frames per unit interval of `t0` within the range of `t0` from 0 to 10.

`fanimator(f, args)` allows the function `f` to depend on multiple variables. `args` specifies the input arguments of `f`.

By default, the variable `t = sym('t')` is the time parameter of the animation. This syntax creates stop-motion frames of `f(subs(args, t, t0))` within the range of `t0` from 0 to 10. You can animate a specific property of the graphics objects by setting its value to depend on `t` in the input argument `args`.

`fanimator( ____, Name, Value)` specifies the animation properties using one or more `Name, Value` pair arguments. Use this option with any of the input argument combinations in the previous syntaxes. Name-value pair settings apply to the animation object created.

`fanimator(ax, ____)` creates a stop-motion animation object in the axis specified by `ax` instead of in the current axis (`gca`). The option `ax` can precede any input argument combinations in the previous syntaxes.

`fp = fanimator( ____)` returns an `Animator` object. Use `fp` to query and modify the properties of a specific animation object. For a list of properties, see [Animator Properties](#).

### Examples

#### Create Animation of Moving Point and Circle

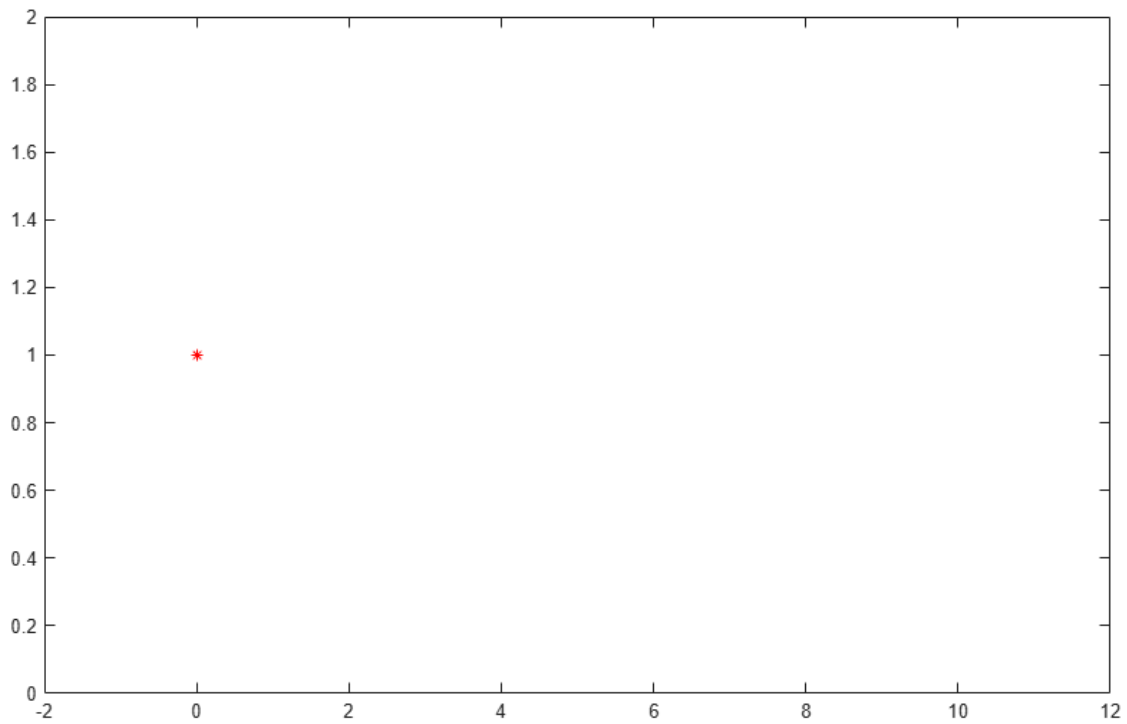
Animate a point and a circle that move along a straight line.

First, create a function to plot a point at  $(t, 1)$ . The variable `t` defines the time parameter of the animation.

```
f = @(t) plot(t,1,'r*');
```

Create a stop-motion animation object defined by f.

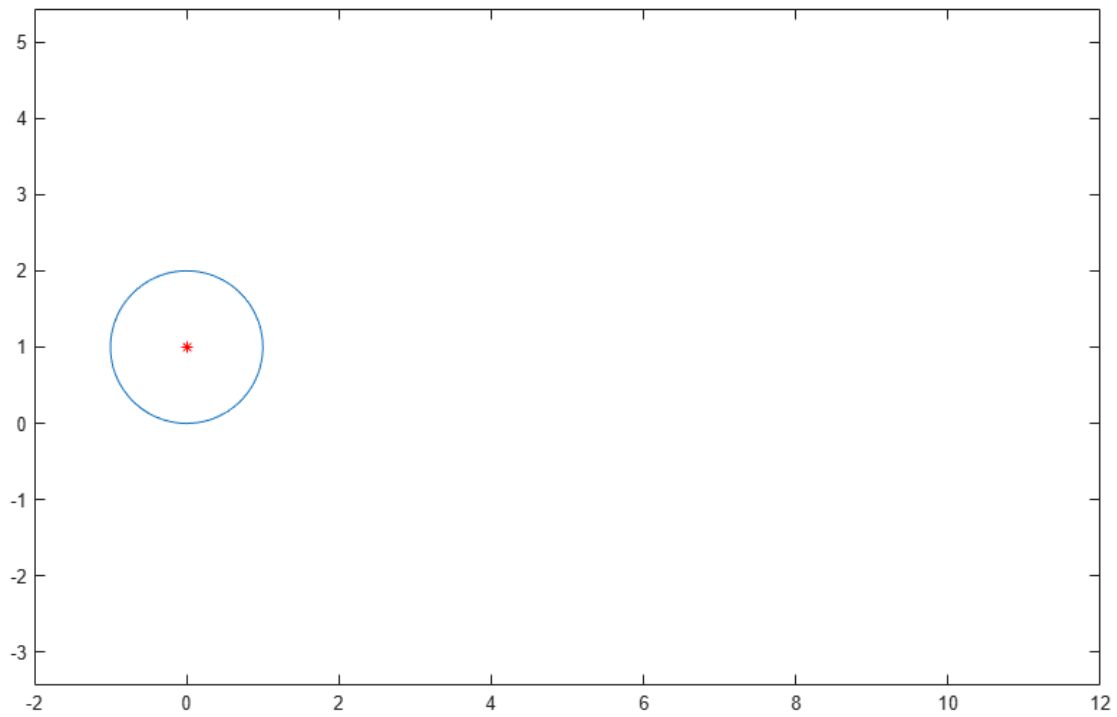
```
fanimator(f)
```



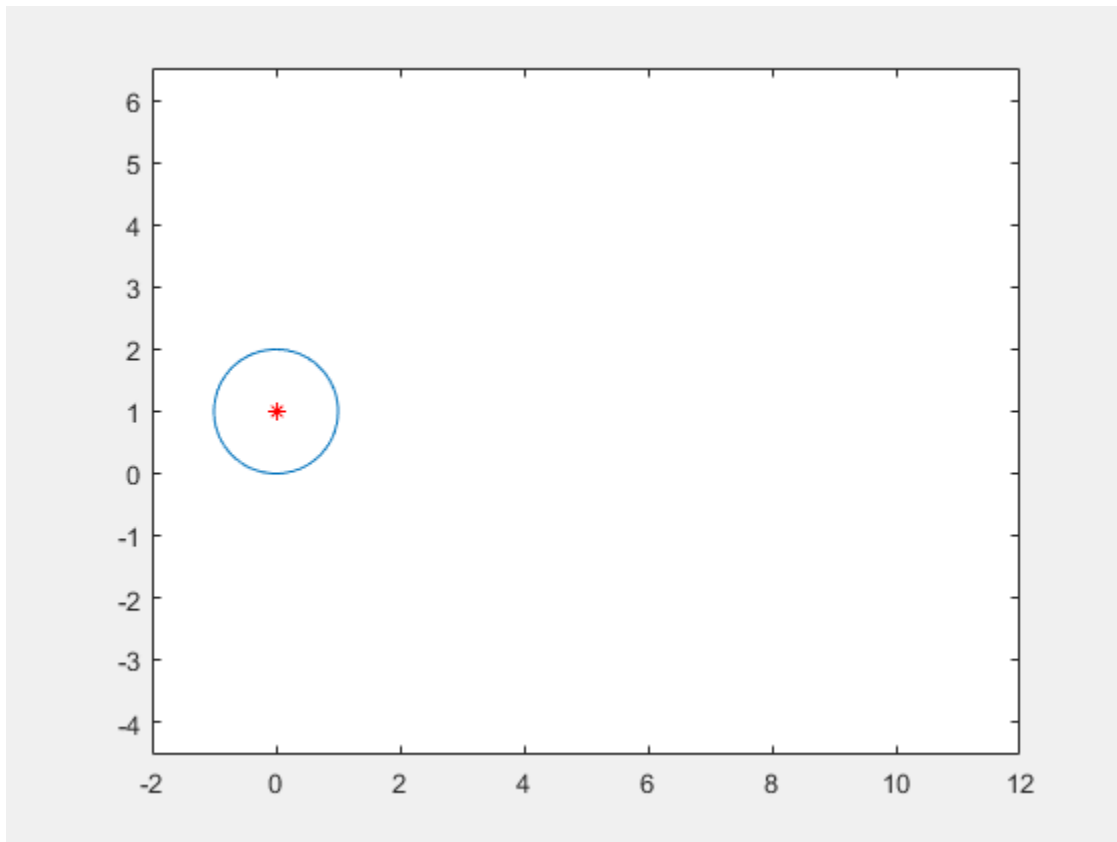
Next, create a function handle by using `fplot` to plot a unit circle. The circle is a function of two variables.

Create two symbolic variables `t` and `x`. Use `t` to set the center of the circle at  $(t, 1)$  and `x` to parameterize the perimeter of the circle within the range  $[-\pi, \pi]$ . Add the circle animation object to the existing plot. Set the x-axis and y-axis to be equal length.

```
syms t x
hold on
fanimator(@fplot,cos(x)+t,sin(x)+1,[-pi pi])
axis equal
hold off
```



Enter the command `playAnimation` to play the animation. By default, `f Animator` creates an animation object, generating 10 frames per unit time within the range of  $t$  from 0 to 10.

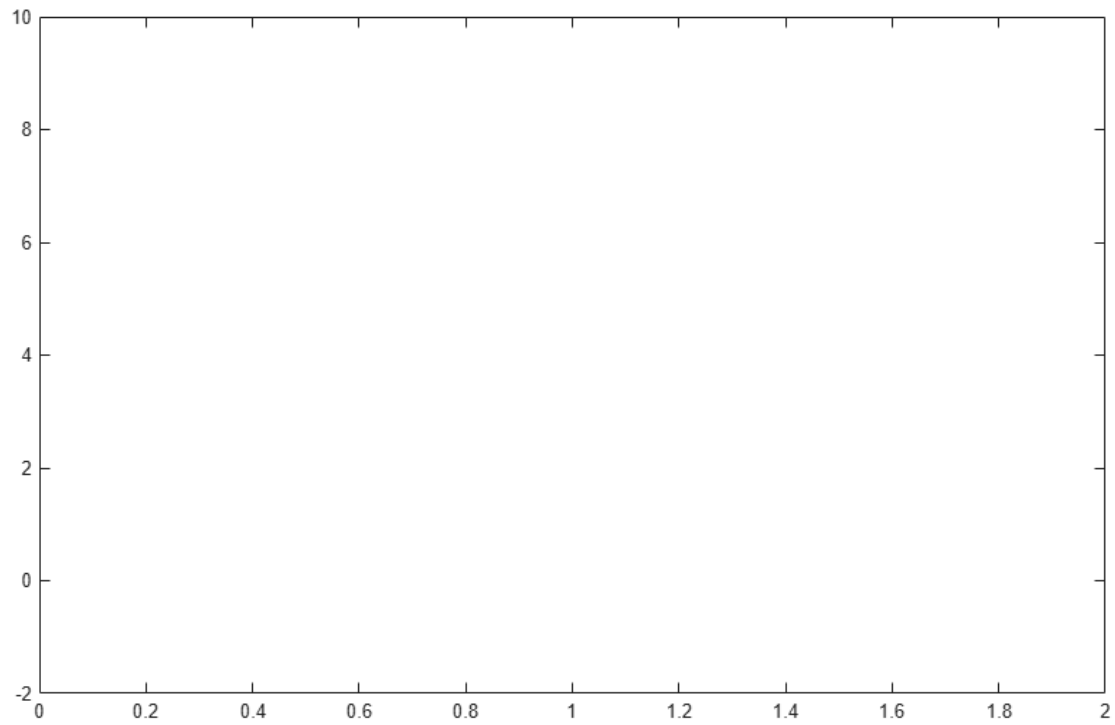


### Create Animation of Changing Line

Animate a line that changes vertical length and line width. You can animate a specific graphics property by setting its value to depend on the animation time parameter. By default, the variable  $t$  is the time parameter of the animation.

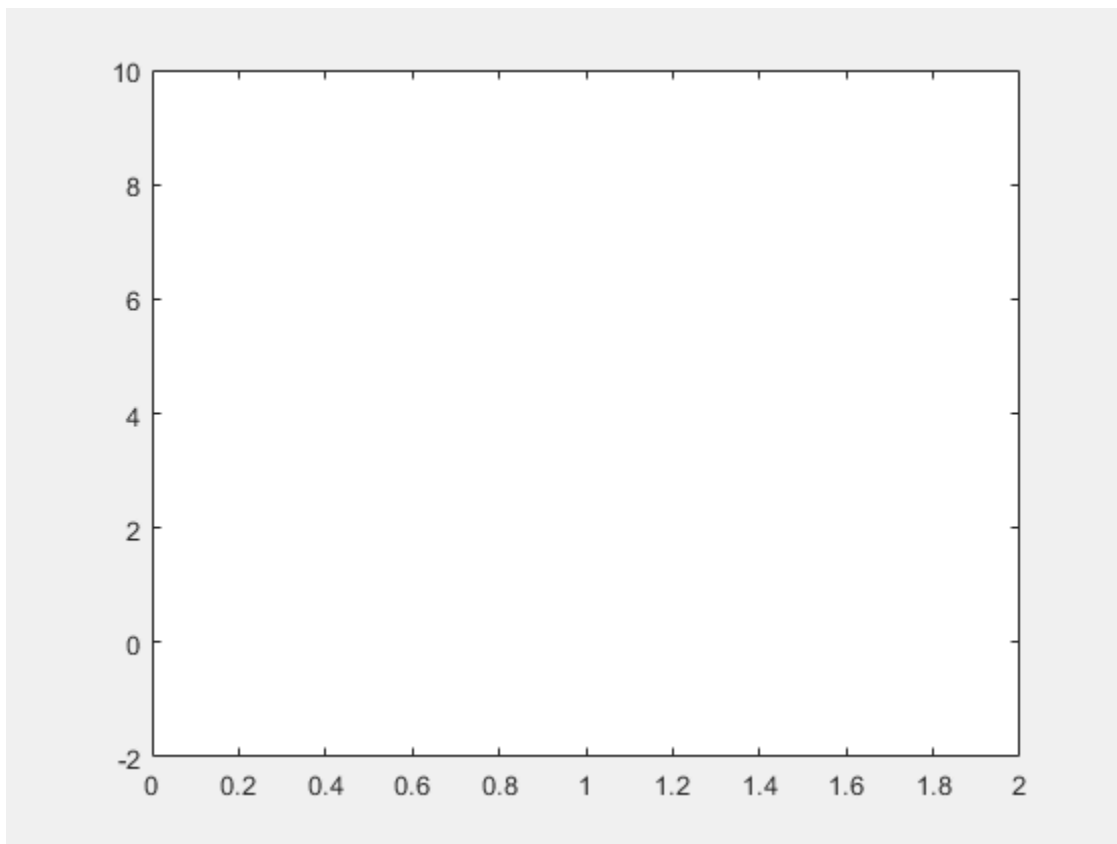
Create two symbolic variables,  $y$  and  $t$ . Plot a line with  $y$  coordinates within the interval  $[0 \ t]$  by using `fplot`. Use the `fanimator` function to create the line animation object. `fanimator` changes the line vertical length by increasing the value of  $t$  from 0 to 10.

```
syms y t
fanimator(@fplot,1,y,[0 t])
```



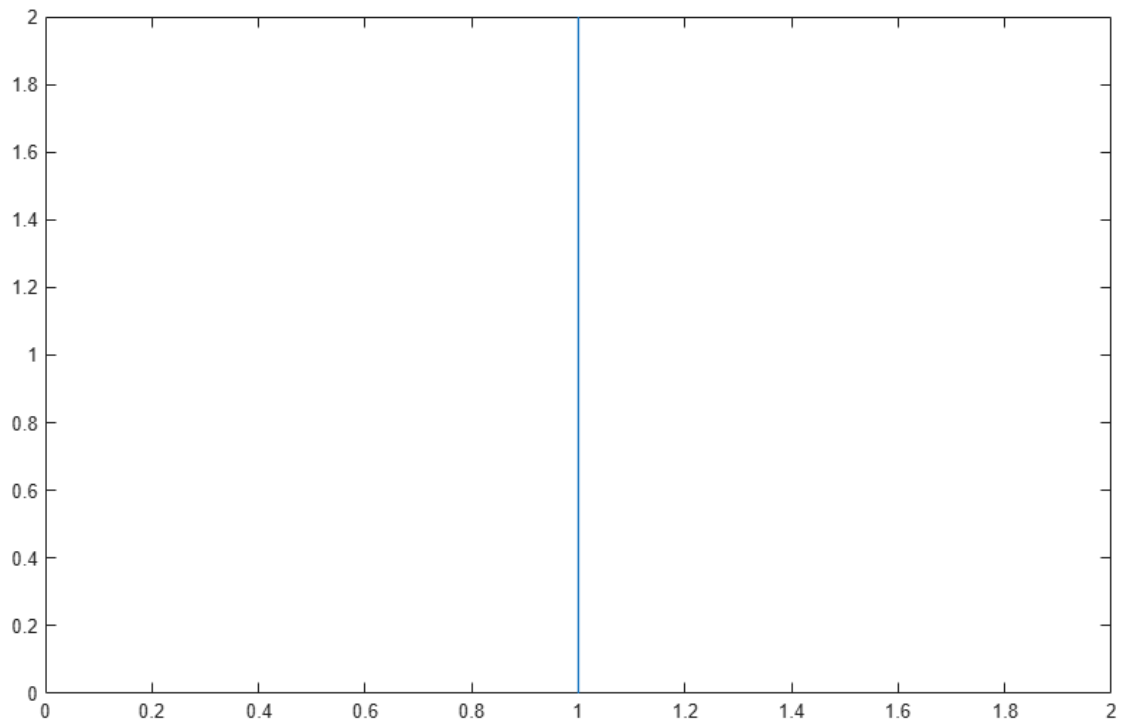
Enter the command `playAnimation` to play the animation.



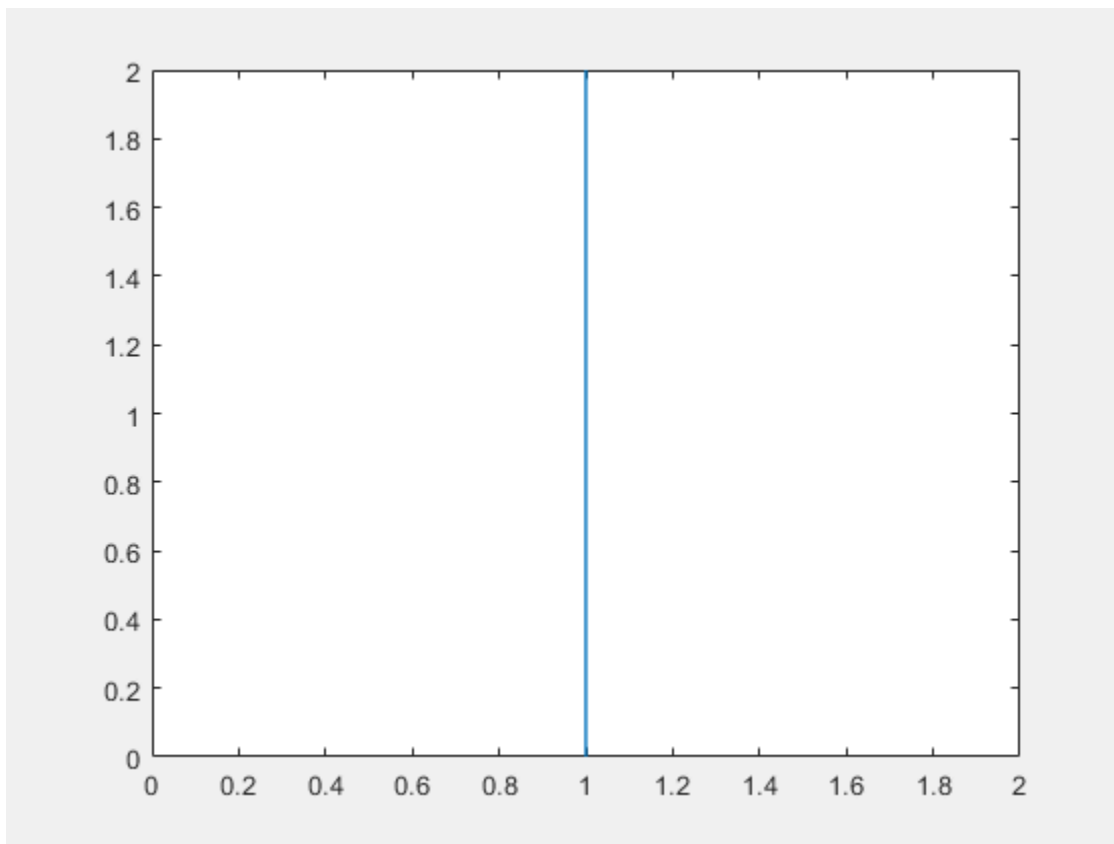


Now plot a line with y coordinates within the interval  $[0, 2]$  by using `fplot`. Set the `'LineWidth'` property value to  $t+1$ . Use the `fanimator` function to create the line animation object. `fanimator` changes the line width by increasing the value of  $t$  from 0 to 10.

```
fanimator(@fplot,1,y,[0 2],'LineWidth',t+1)
```



Enter the command `playAnimation` to play the animation.



### Create Animation of Circle with Timer

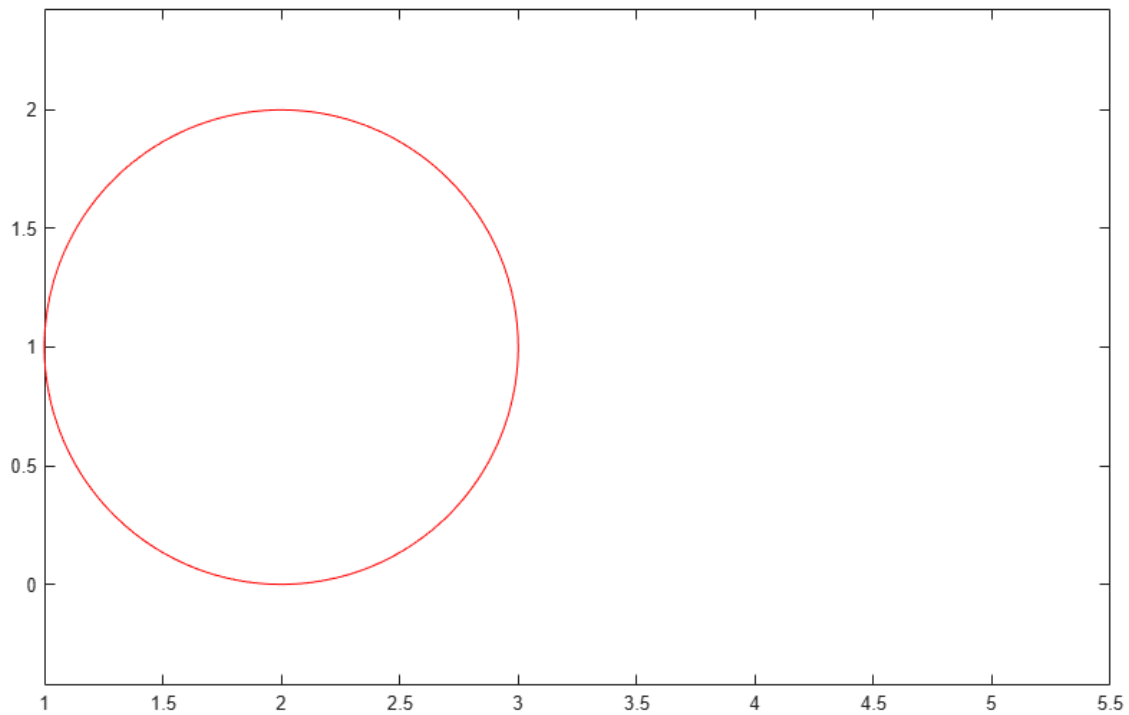
Animate a circle with a timer.

First, create a function that plots a unit circle and save it in a file named `circ.m`. The function uses `fplot` to plot a unit circle centered at  $(t, 1)$ , and the local symbolic variable  $x$  to parameterize the perimeter of the circle.

```
function C = circ(t)
    x = sym('x');
    C = fplot(cos(x)+t, sin(x)+1, [0 2*pi], 'Color', 'red');
end
```

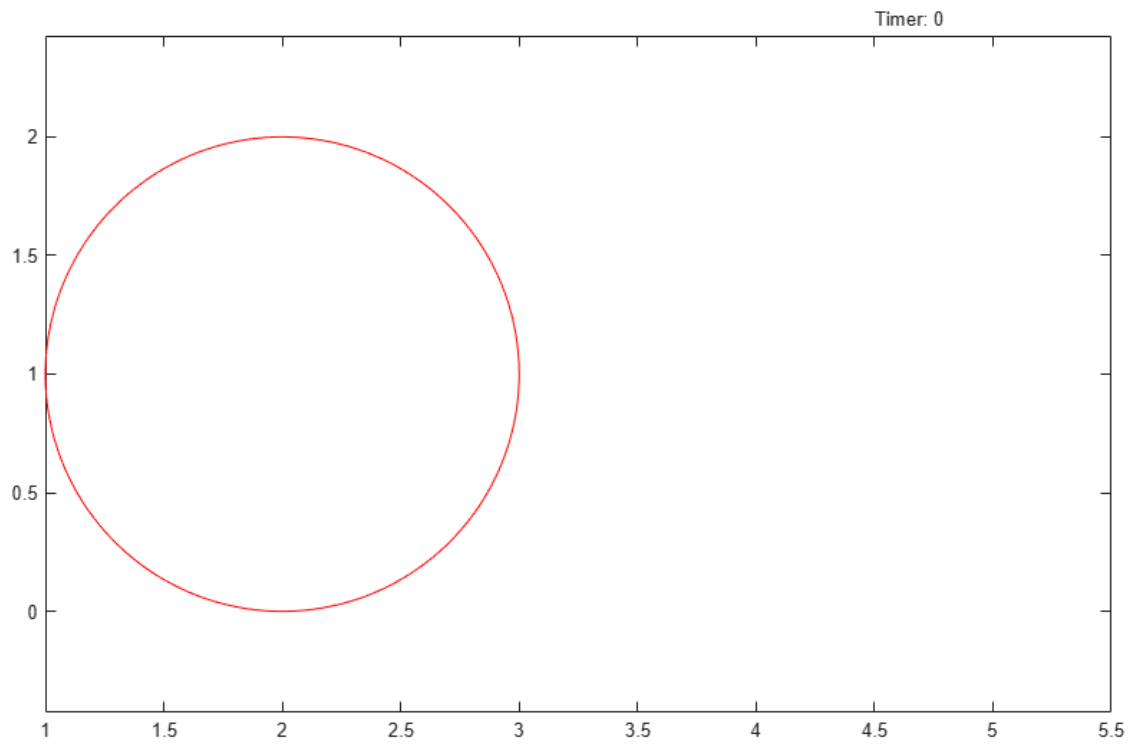
Use `fanimator` to create a unit circle animation object. Set the animation range of the time parameter to `[2 4.5]` and the frame rate per unit time to 4. Set the x-axis and y-axis to be equal length.

```
fanimator(@circ, 'AnimationRange', [2 4.5], 'FrameRate', 4)
axis equal
```

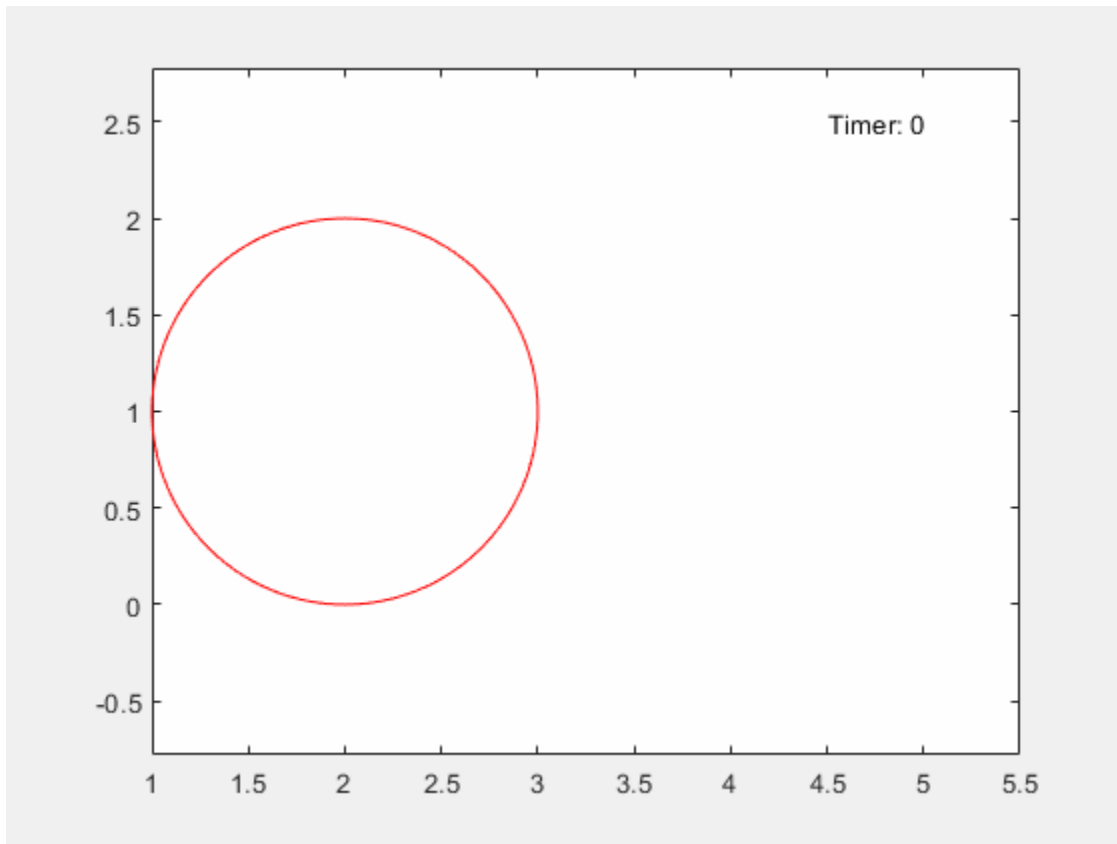


Next, add a timer animation object. Create a piece of text to count the elapsed time by using the `text` function. Use `num2str` to convert the time parameter to a string. Set the animation range of the timer to `[0 4.5]`.

```
hold on
f animator(@(t) text(4.5,2.5,"Timer: "+num2str(t,2)), 'AnimationRange',[0 4.5])
hold off
```



Enter the command `playAnimation` to play the animation. The timer counts the elapsed time from 0 to 4.5 seconds. The moving circle starts at 2 seconds and stops at 4.5 seconds.

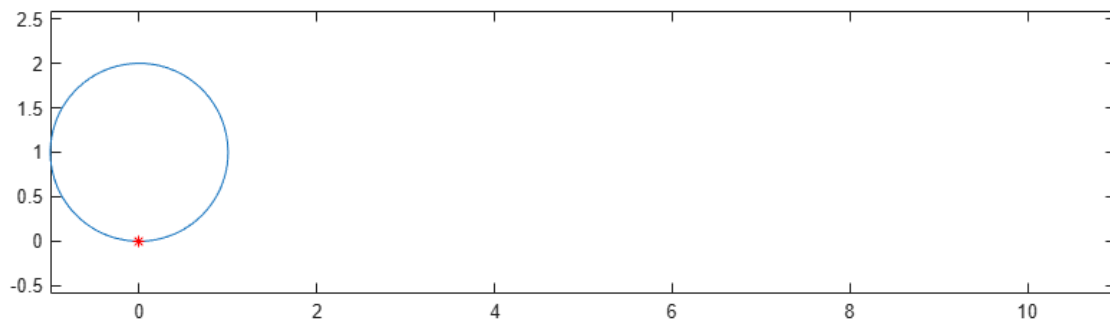


### Create Animation of Cycloids

Animate two cycloids in separate axes. A cycloid is the curve traced by a fixed point on a circle as the circle moves along a straight line without slipping.

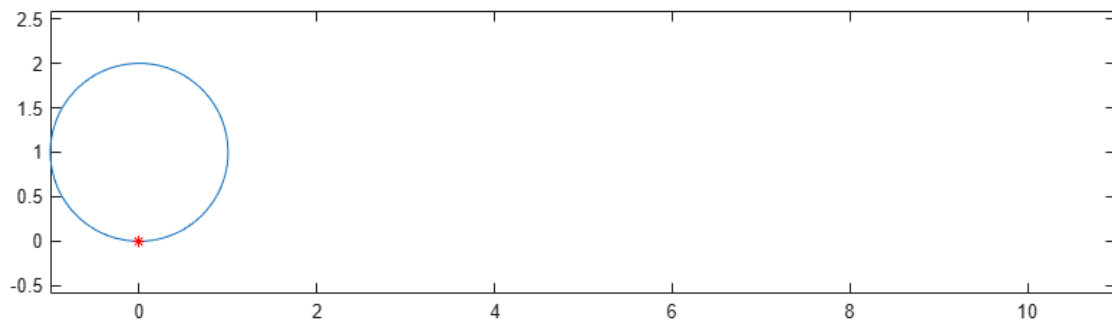
First, create two symbolic variables  $x$  and  $t$ . Create a figure with two subplots and return the first axes object as `ax1`. Create a moving circle animation object in `ax1` and add a fixed point on the rim of the circle. Set the x-axis and y-axis to be equal length.

```
syms x t
ax1 = subplot(2,1,1);
f animator(ax1, @fplot, cos(x)+t, sin(x)+1, [-pi pi])
axis equal
hold on
f animator(ax1, @(t) plot(t-sin(t), 1-cos(t), 'r*'))
```



To trace the cycloid, use a time variable in the plotting interval. The `fplot` function plots a curve within the interval  $[0 \ t]$ . Create the cycloid animation object. By default, `fanimator` creates stop-motion frames within the range of  $t$  from 0 to 10 seconds. `fanimator` plots the first frame at  $t$  equal to 0.

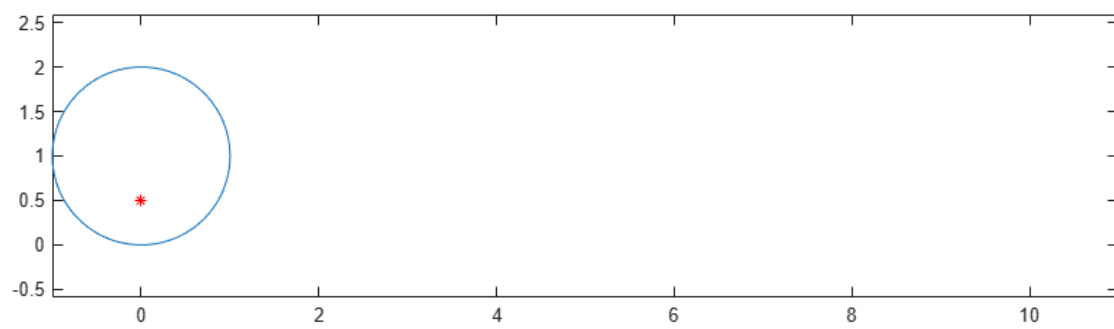
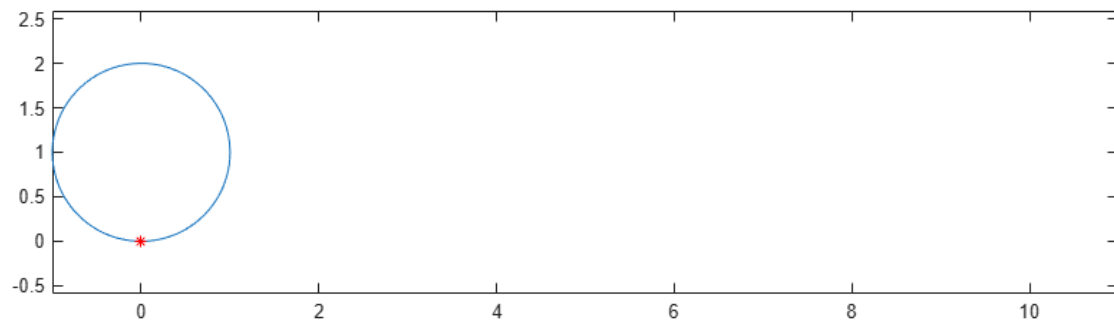
```
fanimator(ax1, @fplot, x-sin(x), 1-cos(x), [0 t], 'k')
hold off
```



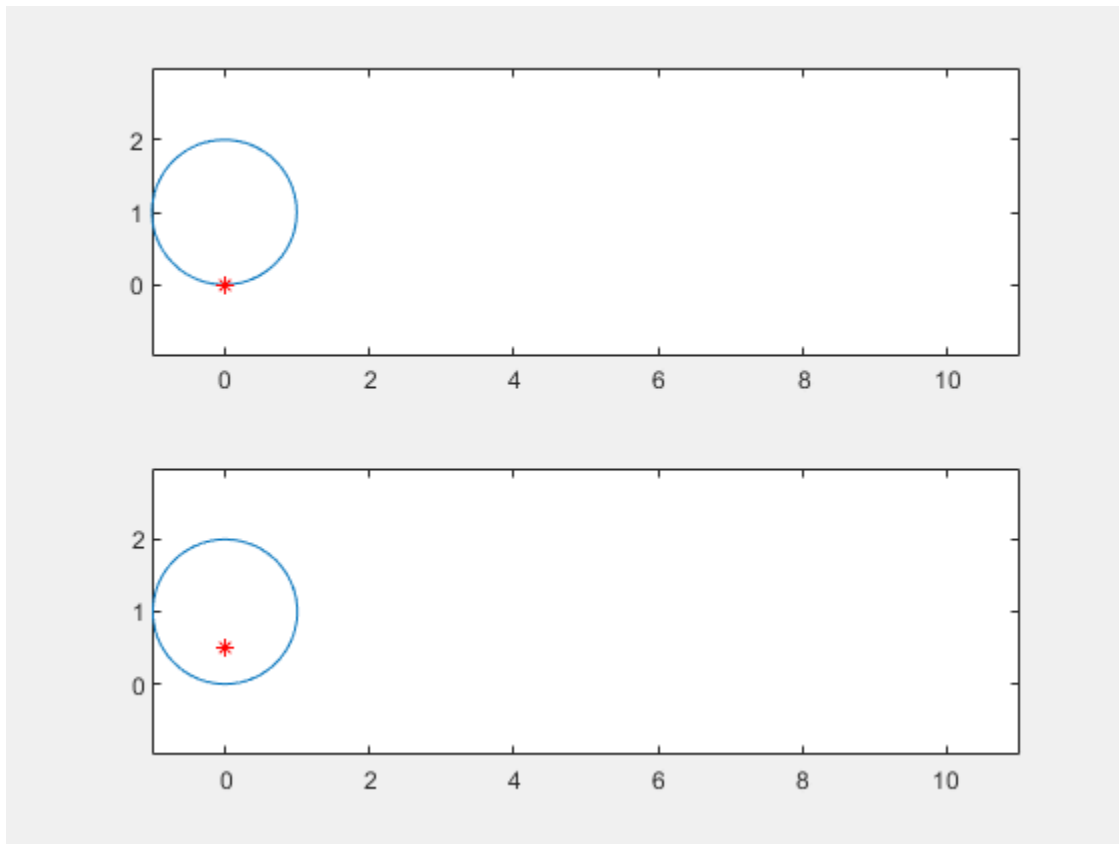
Next, create another cycloid on the second axes object `ax2`. Trace the curve created by a fixed point at a distance of  $1/2$  from the center of the circle. Set the x-axis and y-axis to be equal length.

```
ax2 = subplot(2,1,2);
f animator(ax2, @fplot, cos(x)+t, sin(x)+1, [-pi pi])
axis equal
hold on
f animator(ax2, @(t) plot(t-sin(t)/2, 1-cos(t)/2, 'r*'))
f animator(ax2, @fplot, x-sin(x)/2, 1-cos(x)/2, [0 t], 'k')
hold off
```





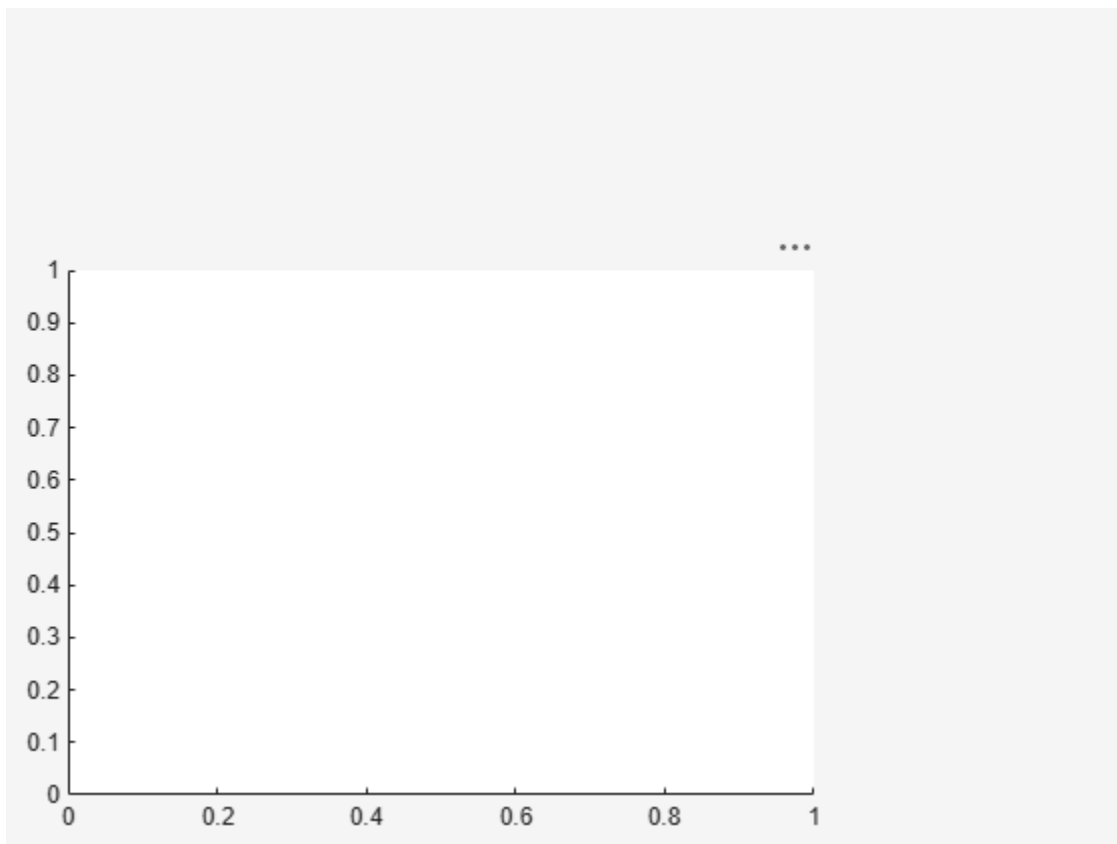
Enter the command `playAnimation` to play the animation.



### Create Animation in UI Figure

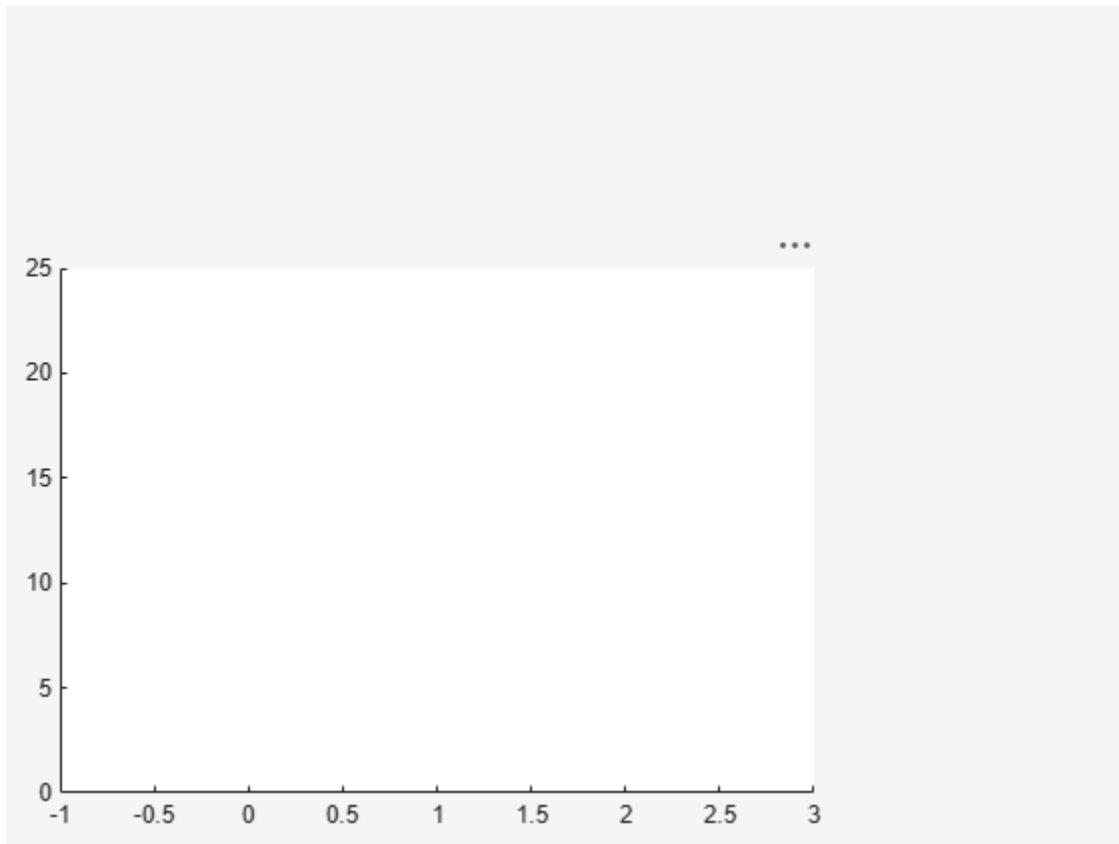
Create a UI figure. Specify the UI axes of the figure.

```
fig = uifigure;  
ax = uiaxes(fig);
```

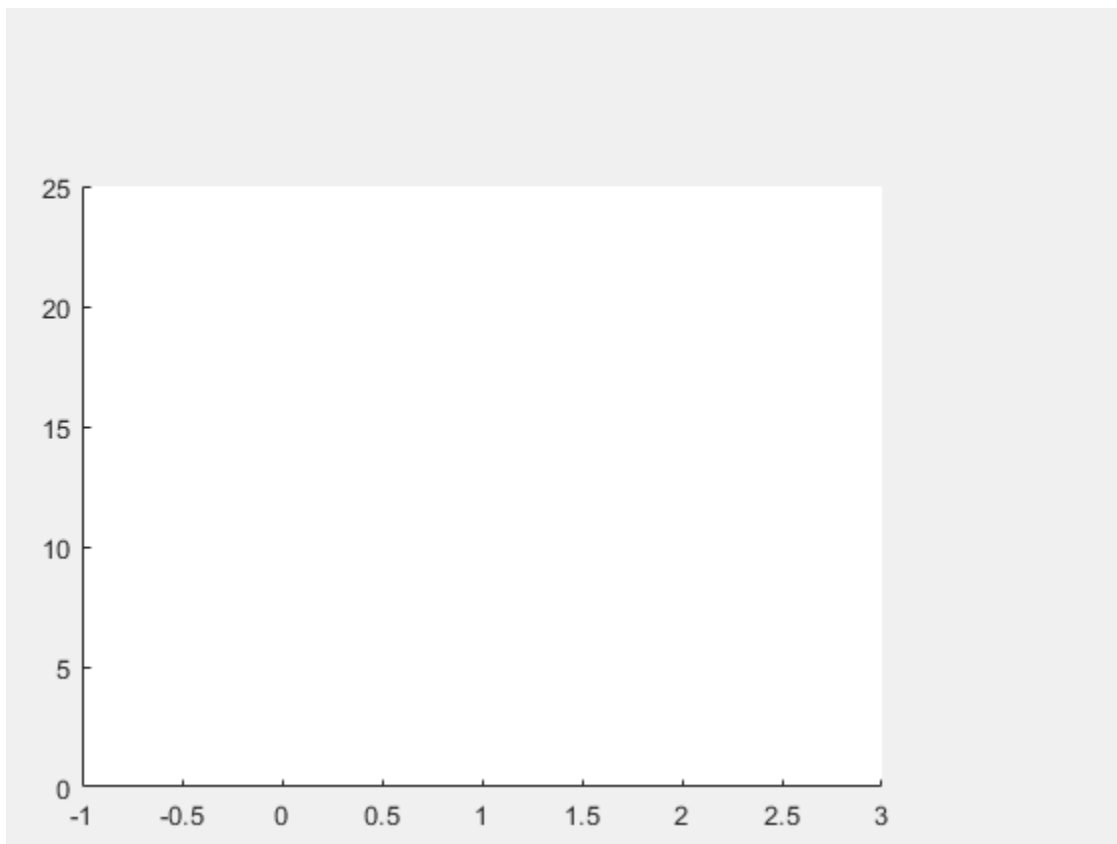


Add an animation object to the UI axes using `fanimator`. Create two symbolic variables, `x` and `t`. Plot a curve that grows exponentially as a function of time `t` within the interval `[0 3]`.

```
syms x t;
fanimator(ax,@fplot,exp(x),[0 t], 'r', 'AnimationRange',[0 3])
```



Play the animation in the UI figure `fig` by entering the command `playAnimation(fig)`. Alternatively, you can also use the command `playAnimation(ax.Parent)`.



## Input Arguments

### **f** — Function returning graphics objects

function handle

Function returning graphics objects, specified as a function handle. For more information about function handles, see “Create Function Handle”.

### **args** — Further arguments

input arguments

Further arguments, specified as input arguments of a function handle that returns the graphics objects.

### **ax** — Target axes

Axes object

Target axes, specified as an Axes object. For more information about Axes objects, see `axes`.

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: 'AnimationRange',[2 8],'FrameRate',30

### **AnimationParameter — Animation time parameter**

`sym('t')` (default) | symbolic variable

Animation time parameter, specified as a symbolic variable.

Example: `sym('y')`

### **AnimationRange — Range of animation time parameter**

`[0 10]` (default) | two-element row vector

Range of the animation time parameter, specified as a two-element row vector. The two elements must be real values that are increasing.

Example: `[-2 4.5]`

### **FrameRate — Frame rate**

`10` (default) | positive value

Frame rate, specified as a positive value. The frame rate defines the number of frames per unit time for the animation object.

Example: `30`

## **Output Arguments**

### **fp — Animation object**

scalar

Animation object, returned as a scalar. You can use this object to query and modify the properties of the generated animation frames. For a list of properties, see [Animator Properties](#).

## **Tips**

- When you create a graph by using a plotting function, such as `fplot`, MATLAB creates a series of graphics objects. You can then animate a specific property of the graphics objects by using the `fanimate` and the `playAnimation` functions. Note that some functions, such as `title` and `xlabel`, create text objects that cannot be animated. Instead, use the `text` function to create text objects that can be animated.

## **Version History**

**Introduced in R2019a**

## **See Also**

`animationToFrame` | `playAnimation` | `rewindAnimation` | `writeAnimation`

# fcontour

Plot contours of symbolic expression

## Syntax

```
fcontour(f)
fcontour(f,[min max])
fcontour(f,[xmin xmax ymin ymax])

fcontour( ___,LineStyle)
fcontour( ___,Name,Value)
fcontour(ax, ___)
fc = fcontour( ___)

```

## Description

`fcontour(f)` plots the contour lines of symbolic expression  $f(x,y)$  over the default interval of  $x$  and  $y$ , which is  $[-5\ 5]$ .

`fcontour(f,[min max])` plots  $f$  over the interval  $\min < x < \max$  and  $\min < y < \max$ .

`fcontour(f,[xmin xmax ymin ymax])` plots  $f$  over the interval  $x_{\min} < x < x_{\max}$  and  $y_{\min} < y < y_{\max}$ . The `fcontour` function uses `symvar` to order the variables and assign intervals.

`fcontour( ___,LineStyle)` uses `LineStyle` to set the line style and color. `fcontour` doesn't support markers.

`fcontour( ___,Name,Value)` specifies line properties using one or more `Name,Value` pair arguments. Use this option with any of the input argument combinations in the previous syntaxes. `Name,Value` pair settings apply to all the lines plotted. To set options for individual plots, use the objects returned by `fcontour`.

`fcontour(ax, ___)` plots into the axes object `ax` instead of the current axes object `gca`.

`fc = fcontour( ___)` returns a function contour object. Use the object to query and modify properties of a specific contour plot. For details, see [Function Contour](#).

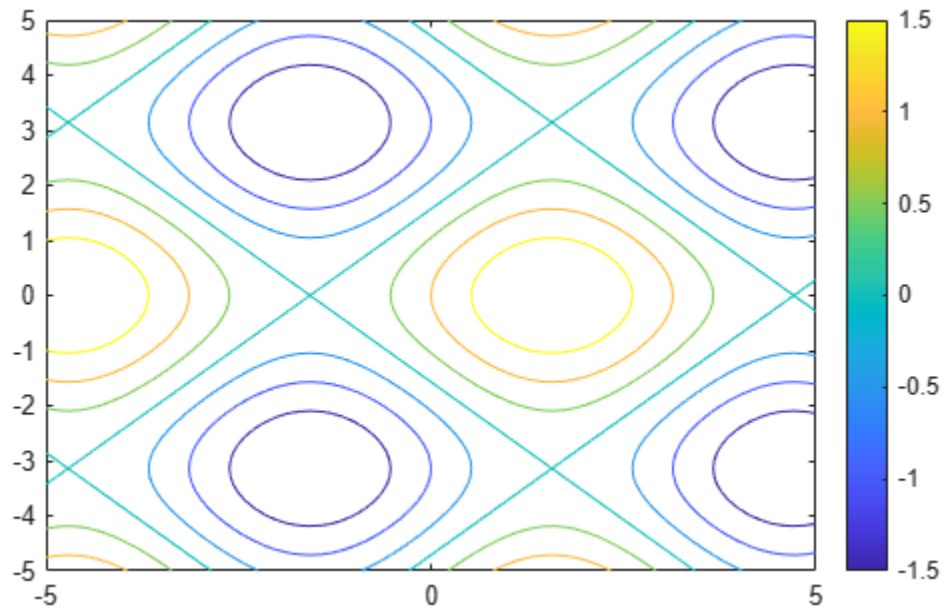
## Examples

### Plot Contours of Symbolic Expression

Plot the contours of  $\sin(x) + \cos(y)$  over the default range of  $-5 < x < 5$  and  $-5 < y < 5$ . Show the colorbar. Find a contour's level by matching the contour's color with the colorbar value.

```
syms x y
fcontour(sin(x) + cos(y))
colorbar

```

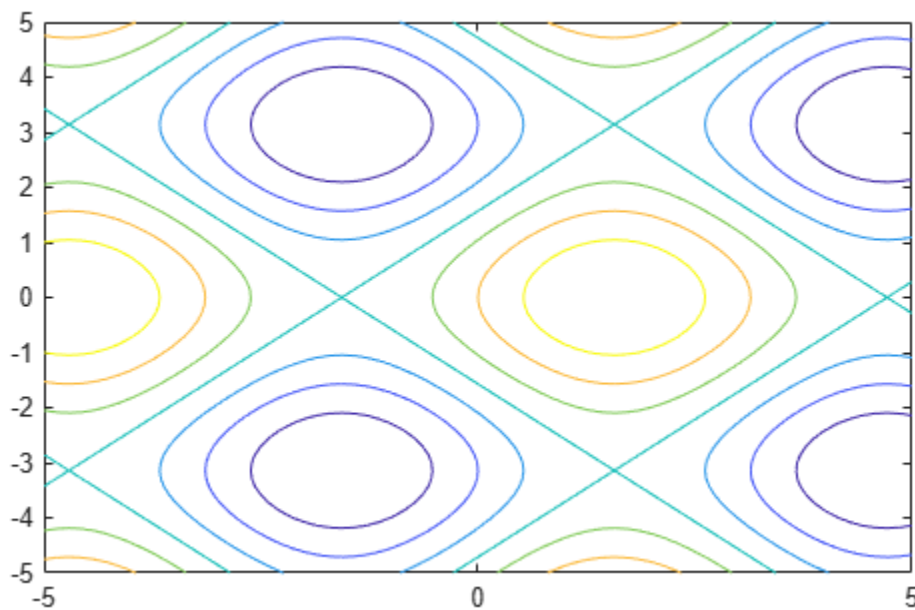


### Plot Contours of Symbolic Function

Plot the contours of  $f(x, y) = \sin(x) + \cos(y)$  over the default range of  $-5 < x < 5$  and  $-5 < y < 5$ .

```
syms f(x,y)
f(x,y) = sin(x) + cos(y);
fcontour(f)
```

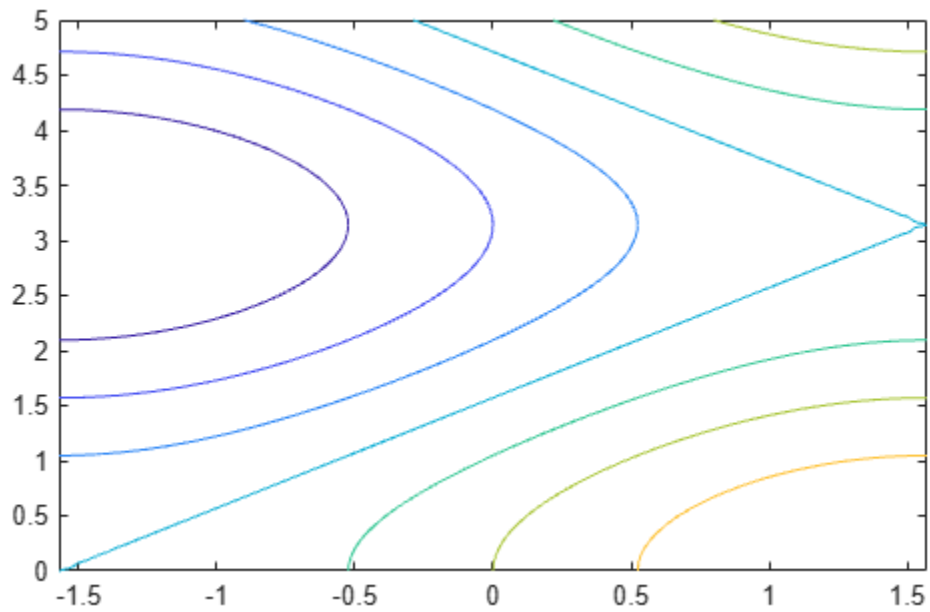




### Specify Plotting Interval

Plot  $\sin(x) + \cos(y)$  over  $-\pi/2 < x < \pi/2$  and  $0 < y < 5$  by specifying the plotting interval as the second argument of `fcontour`.

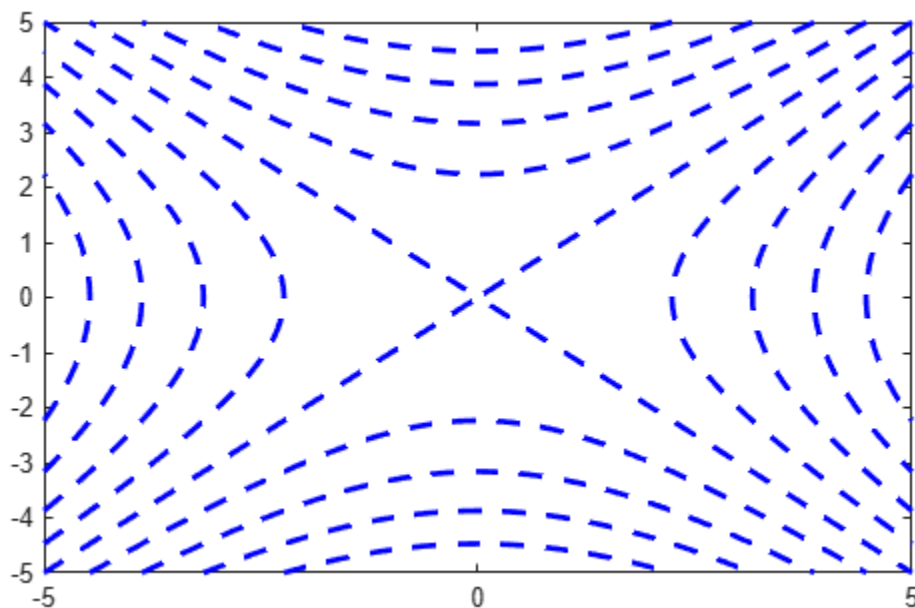
```
syms x y
f = sin(x) + cos(y);
fcontour(f, [-pi/2 pi/2 0 5])
```



### Change Line Style, Color and Width

Plot the contours of  $x^2 - y^2$  as blue, dashed lines by specifying the `LineStyle` input. Specify a `LineWidth` of 2. Markers are not supported by `fcontour`.

```
syms x y
fcontour(x^2 - y^2, '--b', 'LineWidth', 2)
```



### Plot Multiple Contour Plots on Same Figure

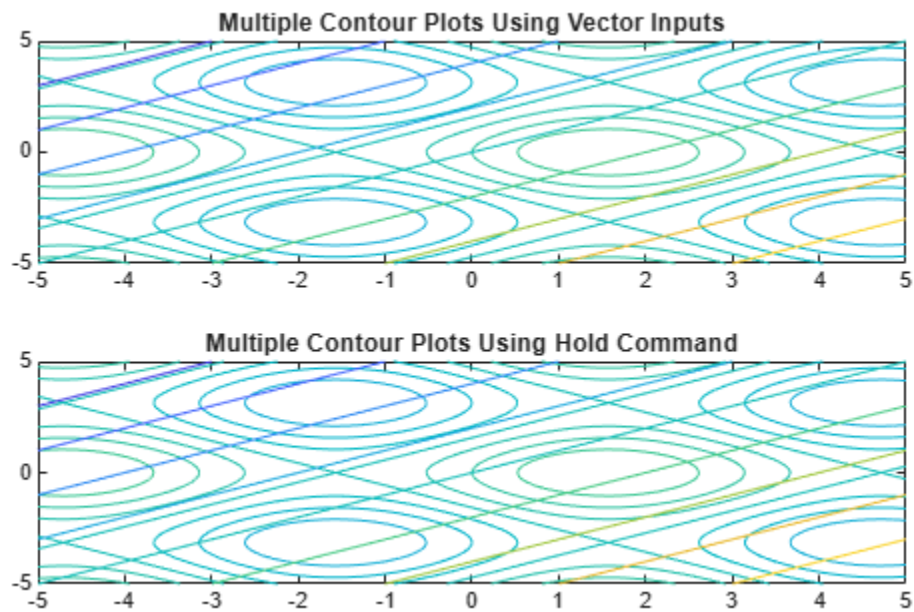
Plot multiple contour plots either by passing the inputs as a vector or by using `hold on` to successively plot on the same figure. If you specify `LineStyle` and Name-Value arguments, they apply to all contour plots. You cannot specify individual `LineStyle` and Name-Value pair arguments for each plot.

Divide a figure into two subplots by using `subplot`. On the first subplot, plot  $\sin(x) + \cos(y)$  and  $x - y$  by using vector input. On the second subplot, plot the same expressions by using `hold on`.

```
syms x y
subplot(2,1,1)
fcontour([sin(x)+cos(y) x-y])
title('Multiple Contour Plots Using Vector Inputs')

subplot(2,1,2)
fcontour(sin(x)+cos(y))
hold on
fcontour(x-y)
title('Multiple Contour Plots Using Hold Command')

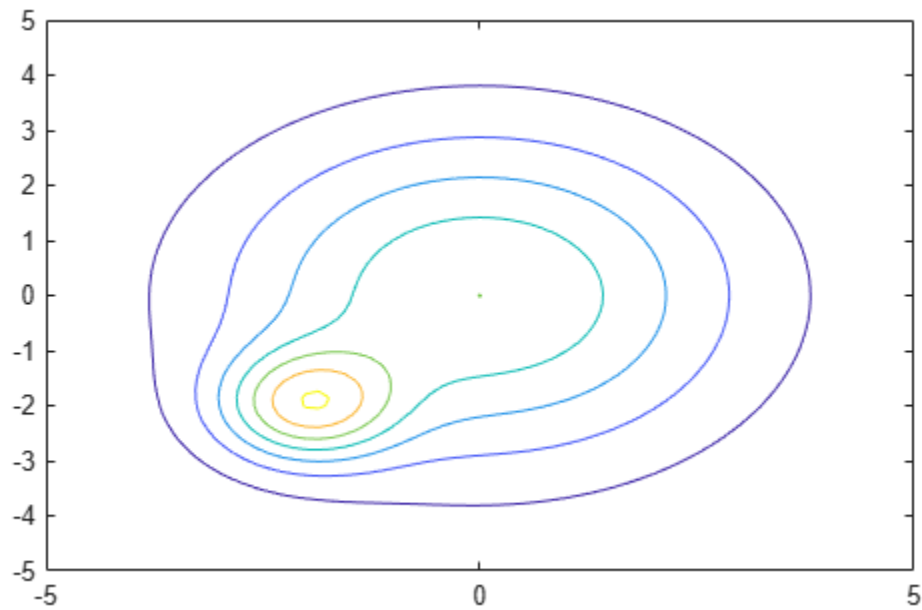
hold off
```



### Modify Contour Plot After Creation

Plot the contours of  $e^{-(x/3)^2 - (y/3)^2} + e^{-(x+2)^2 - (y+2)^2}$ . Specify an output to make `fcontour` return the plot object.

```
syms x y
f = exp(-(x/3)^2-(y/3)^2) + exp(-(x+2)^2-(y+2)^2);
fc = fcontour(f)
```



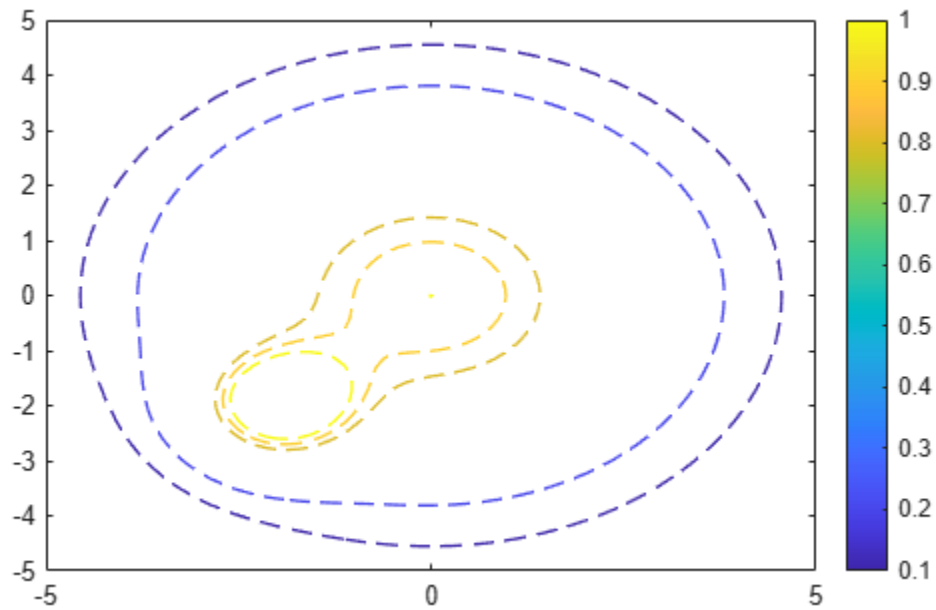
```
fc =
  FunctionContour with properties:

    Function: exp(- x^2/9 - y^2/9) + exp(- (x + 2)^2 - (y + 2)^2)
    LineColor: 'flat'
    LineStyle: '-'
    LineWidth: 0.5000
    Fill: off
    LevelList: [0.2000 0.4000 0.6000 0.8000 1 1.2000 1.4000]
```

Show all properties

Change the `LineWidth` to 1 and the `LineStyle` to a dashed line by using dot notation to set properties of the object `fc`. Visualize contours close to 0 and 1 by setting `LevelList` to `[1 0.9 0.8 0.2 0.1]`.

```
fc.LineStyle = '--';
fc.LineWidth = 1;
fc.LevelList = [1 0.9 0.8 0.2 0.1];
colorbar
```



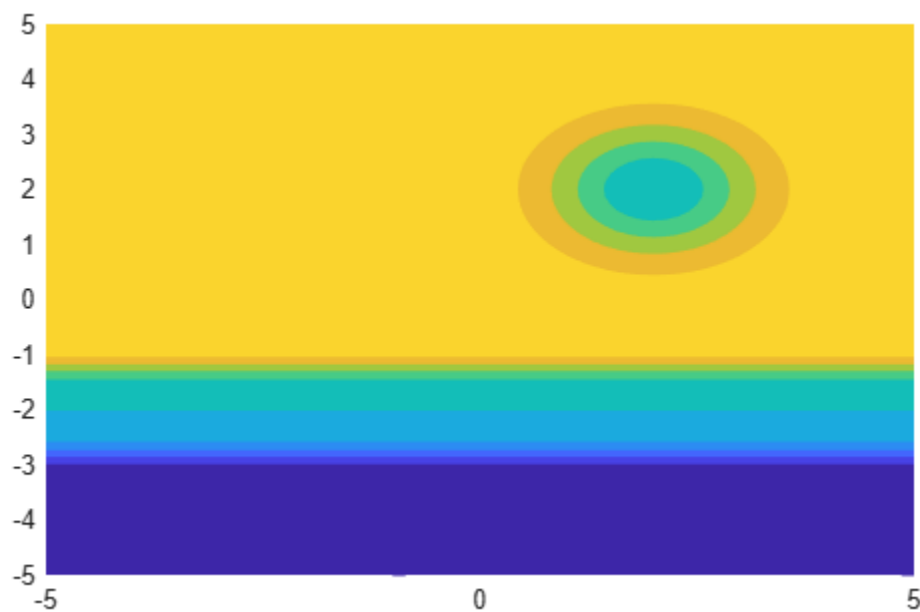
### Fill Area Between Contours

Fill the area between contours by setting the `Fill` input of `fcontour` to `'on'`. If you want interpolated shading instead, use the `fsurf` function with its option `'EdgeColor'` set to `'none'` followed by the command `view(0,90)`.

Create a plot that looks like a sunset by filling the contours of

$$\text{erf}((y+2)^3) - e^{(-0.65((x-2)^2 + (y-2)^2))}.$$

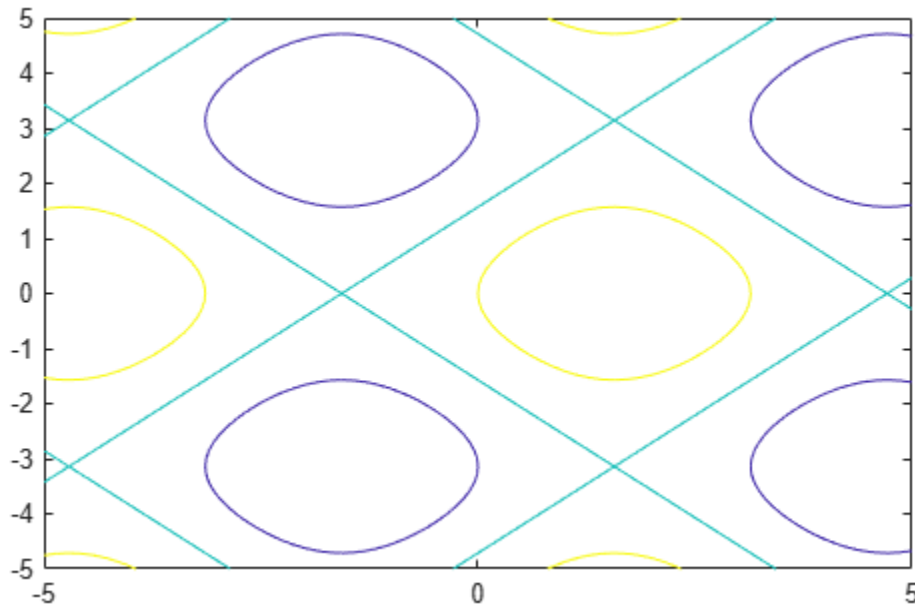
```
syms x y
f = erf((y+2)^3) - exp(-0.65*((x-2)^2+(y-2)^2));
fcontour(f, 'Fill', 'on')
```



### Specify Levels for Contour Lines

Set the values at which `fcontour` draws contours by using the `'LevelList'` option.

```
syms x y
f = sin(x) + cos(y);
fcontour(f, 'LevelList', [-1 0 1])
```



### Control Resolution of Contour Lines

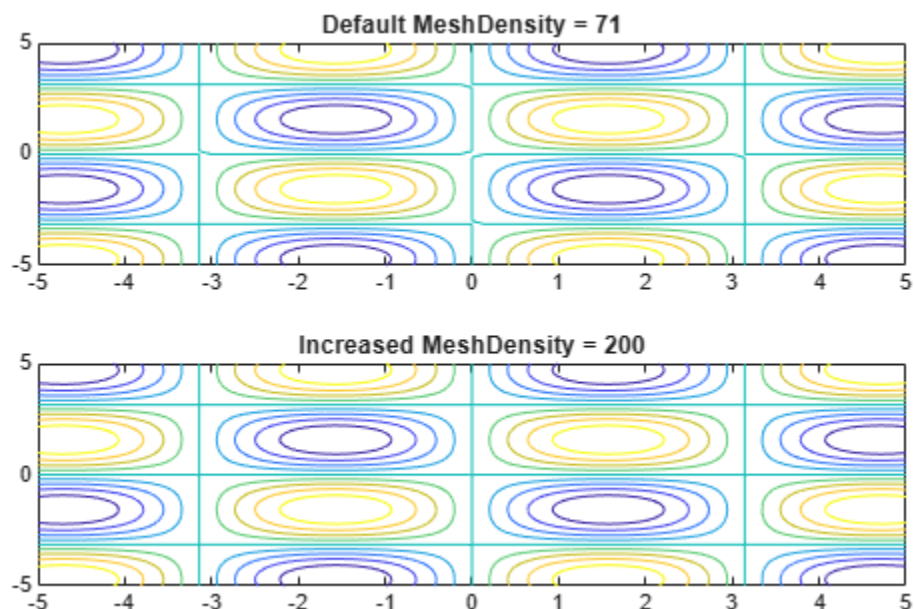
Control the resolution of contour lines by using the 'MeshDensity' option. Increasing 'MeshDensity' can make smoother, more accurate plots while decreasing it can increase plotting speed.

Divide a figure into two using `subplot`. In the first subplot, plot the contours of  $\sin(x)\sin(y)$ . The corners of the squares do not meet. To fix this issue, increase 'MeshDensity' to 200 in the second subplot. The corners now meet, showing that by increasing 'MeshDensity' you increase the plot's resolution.

```
syms x y
subplot(2,1,1)
fcontour(sin(x).*sin(y))
title('Default MeshDensity = 71')

subplot(2,1,2)
fcontour(sin(x).*sin(y), 'MeshDensity', 200)
title('Increased MeshDensity = 200')
```





### Add Title and Axis Labels and Format Ticks

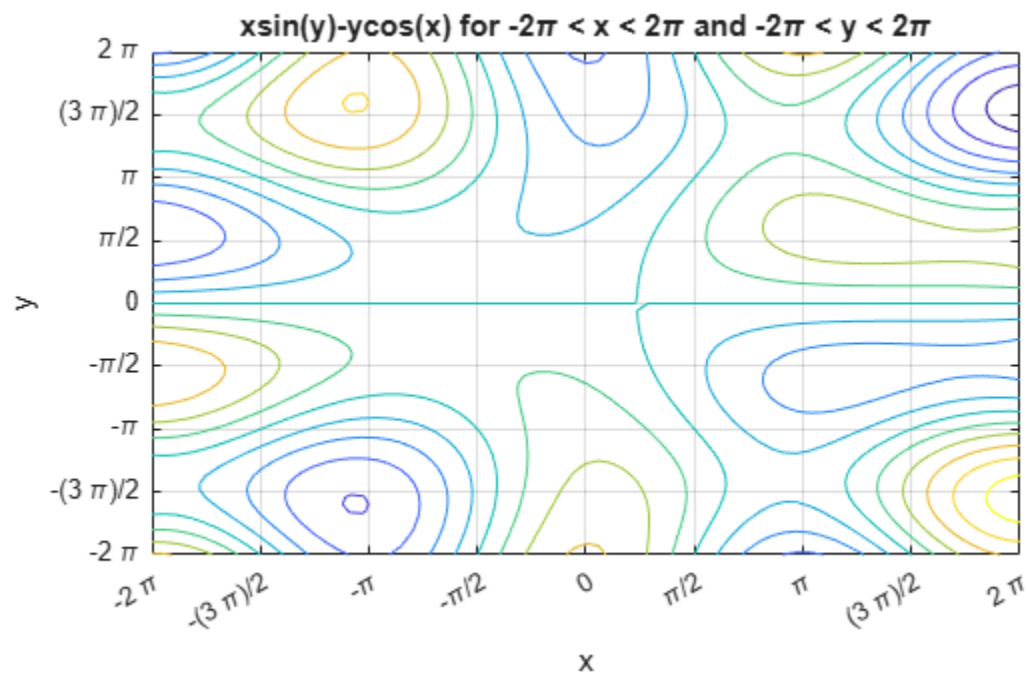
Plot  $x\sin(y) - y\cos(x)$ . Add a title and axis labels. Create the x-axis ticks by spanning the x-axis limits at intervals of  $\pi/2$ . Display these ticks by using the `XTick` property. Create x-axis labels by using `arrayfun` to apply `texlabel` to `S`. Display these labels by using the `XTickLabel` property. Repeat these steps for the y-axis.

To use LaTeX in plots, see `latex`.

```
syms x y
fcontour(x*sin(y)-y*cos(x), [-2*pi 2*pi])
grid on
title('xsin(y)-ycos(x) for -2\pi < x < 2\pi and -2\pi < y < 2\pi')
xlabel('x')
ylabel('y')
ax = gca;

S = sym(ax.XLim(1):pi/2:ax.XLim(2));
ax.XTick = double(S);
ax.XTickLabel = arrayfun(@texlabel, S, 'UniformOutput', false);

S = sym(ax.YLim(1):pi/2:ax.YLim(2));
ax.YTick = double(S);
ax.YTickLabel = arrayfun(@texlabel, S, 'UniformOutput', false);
```

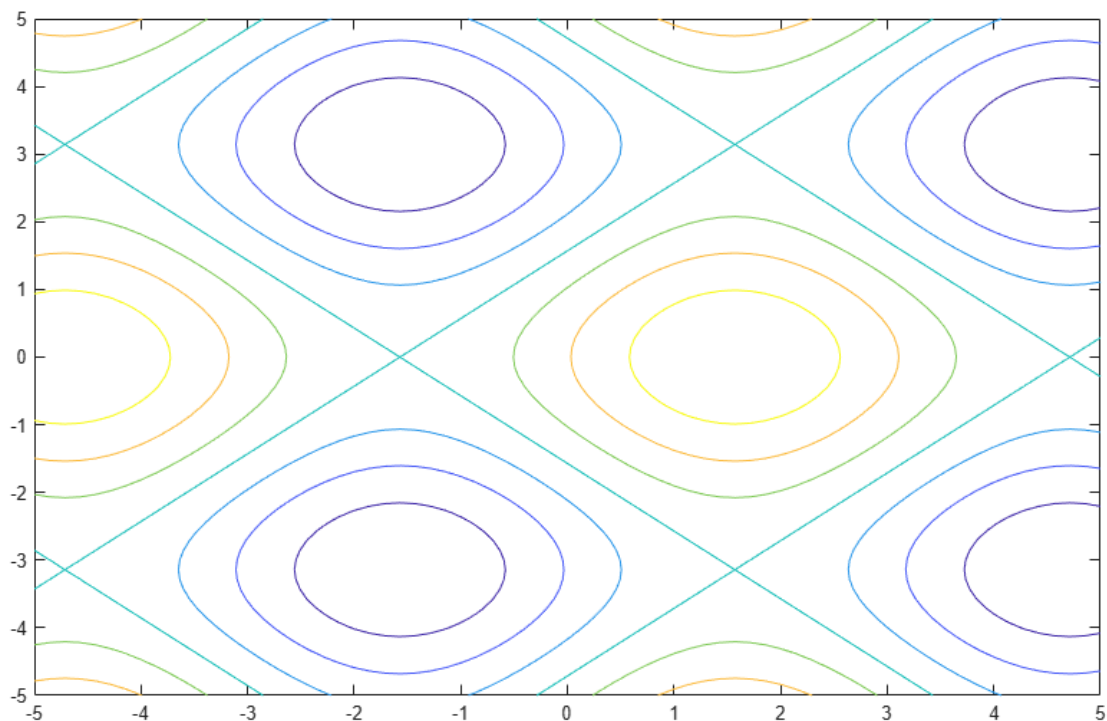


### Create Animation of Contours

Create animation of contours by changing the displayed expression using the `Function` property of the function handle, and then using `drawnow` to update the plot. To export to GIF, see `imwrite`.

By varying the variable  $k$  from  $-\pi/8$  to  $\pi/8$ , animate the parametric curve  $k \sin(x) + k \cos(y)$ .

```
syms x y
fc = fcontour(-pi/8.*sin(x)-pi/8.*cos(y));
for k = -pi/8:0.01:pi/8
    fc.Function = k.*sin(x)+k.*cos(y);
    drawnow
    pause(0.05)
end
```



## Input Arguments

### **f** — Expression or function to be plotted

symbolic expression | symbolic function

Expression or function to be plotted, specified as a symbolic expression or function.

### **[min max]** — Plotting range for x and y

[-5 5] (default) | vector of two numbers

Plotting range for x and y, specified as a vector of two numbers. The default range is [-5 5].

### **[xmin xmax ymin ymax]** — Plotting range for x and y

[-5 5 -5 5] (default) | vector of four numbers

Plotting range for x and y, specified as a vector of four numbers. The default range is [-5 5 -5 5].

### **ax** — Axes object

axes object

Axes object. If you do not specify an axes object, then the plot function uses the current axes.

### **LineStyle** — Line style and color

character vector | string

Line style and color, specified as a character vector or string containing a line style specifier, a color specifier, or both.

Example: ' - - r ' specifies red dashed lines

These two tables list the line style and color options.

| Line Style Specifier | Description          |
|----------------------|----------------------|
| -                    | Solid line (default) |
| - -                  | Dashed line          |
| :                    | Dotted line          |
| - .                  | Dash-dot line        |

| Color Specifier | Description |
|-----------------|-------------|
| y               | yellow      |
| m               | magenta     |
| c               | cyan        |
| r               | red         |
| g               | green       |
| b               | blue        |
| w               | white       |
| k               | black       |

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: 'MeshDensity',30

The properties listed here are only a subset. For a complete list, see `Function Contour`.

### MeshDensity — Number of evaluation points per direction

71 (default) | number

Number of evaluation points per direction, specified as a number. The default is 71. Because `fcontour` uses adaptive evaluation, the actual number of evaluation points is greater.

Example: 30

### Fill — Fill between contour lines

'off' (default) | on/off logical value

Fill between contour lines, specified as 'on' or 'off', or as numeric or logical 1 (true) or 0 (false). A value of 'on' is equivalent to true, and 'off' is equivalent to false. Thus, you can use the value of this property as a logical value. The value is stored as an on/off logical value of type `matlab.lang.OnOffSwitchState`.

- A value of 'on' fill the spaces between contour lines with color.

- A value of 'off' leaves the spaces between the contour lines unfilled.

### LevelList — Contour levels

vector of z values

Contour levels, specified as a vector of z values. By default, the `fcontour` function chooses values that span the range of values in the `ZData` property.

Setting this property sets the associated mode property to manual.

Data Types: `single` | `double` | `int8` | `int16` | `int32` | `int64` | `uint8` | `uint16` | `uint32` | `uint64`

### LevelListMode — Selection mode for LevelList

'auto' (default) | 'manual'

Selection mode for the `LevelList`, specified as one of these values:

- 'auto' — Determine the values based on the `ZData` values.
- 'manual' — Use manually specified values. To specify the values, set the `LevelList` property. When the mode is 'manual', the `LevelList` values do not change if you change the `Function` property or the limits.

### LevelStep — Spacing between contour lines

0 (default) | scalar numeric value

Spacing between contour lines, specified as a scalar numeric value. For example, specify a value of 2 to draw contour lines at increments of 2. By default, `LevelStep` is determined by using the `ZData` values.

Setting this property sets the associated mode property to 'manual'.

Example: 3.4

Data Types: `single` | `double` | `int8` | `int16` | `int32` | `int64` | `uint8` | `uint16` | `uint32` | `uint64`

### LevelStepMode — Selection mode for LevelStep

'auto' (default) | 'manual'

Selection mode for the `LevelStep`, specified as one of these values:

- 'auto' — Determine the value based on the `ZData` values.
- 'manual' — Use a manually specified value. To specify the value, set the `LevelStep` property. When the mode is 'manual', the value of `LevelStepMode` does not change when the `Function` property or the limits change.

### LineColor — Color of contour lines

'flat' (default) | RGB triplet | hexadecimal color code | 'r' | 'g' | 'b' | ...

Color of contour lines, specified as 'flat', an RGB triplet, a hexadecimal color code, a color name, or a short name. To use a different color for each contour line, specify 'flat'. The color is determined by the contour value of the line, the colormap, and the scaling of data values into the colormap. For more information on color scaling, see “Control Colormap Limits”.

To use the same color for all the contour lines, specify an RGB triplet, a hexadecimal color code, a color name, or a short name.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range  $[0, 1]$ , for example,  $[0.4 \ 0.6 \ 0.7]$ .
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name     | RGB Triplet    | Hexadecimal Color Code | Appearance                                                                            |
|------------|----------------|----------------|------------------------|---------------------------------------------------------------------------------------|
| "red"      | "r"            | [1 0 0]        | "#FF0000"              |    |
| "green"    | "g"            | [0 1 0]        | "#00FF00"              |    |
| "blue"     | "b"            | [0 0 1]        | "#0000FF"              |    |
| "cyan"     | "c"            | [0 1 1]        | "#00FFFF"              |    |
| "magenta"  | "m"            | [1 0 1]        | "#FF00FF"              |    |
| "yellow"   | "y"            | [1 1 0]        | "#FFFF00"              |  |
| "black"    | "k"            | [0 0 0]        | "#000000"              |  |
| "white"    | "w"            | [1 1 1]        | "#FFFFFF"              |  |
| "none"     | Not applicable | Not applicable | Not applicable         | No color                                                                              |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                       |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

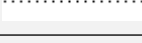
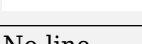
Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

**LineStyle — Line style**

" - " (default) | " - - " | " : " | " - . " | "none"

Line style, specified as one of the options listed in this table.

| Line Style | Description      | Resulting Line                                                                      |
|------------|------------------|-------------------------------------------------------------------------------------|
| " - "      | Solid line       |  |
| " - - "    | Dashed line      |  |
| " : "      | Dotted line      |  |
| " - . "    | Dash-dotted line |  |
| "none"     | No line          | No line                                                                             |

**LineWidth — Line width**

0.5 (default) | positive value

Line width, specified as a positive value in points, where 1 point = 1/72 of an inch. If the line has markers, then the line width also affects the marker edges.

The line width cannot be thinner than the width of a pixel. If you set the line width to a value that is less than the width of a pixel on your system, the line is displayed as one pixel wide.

**Output Arguments****fc — One or more function contour objects**

scalar | vector

One or more function contour objects, returned as a scalar or a vector. These objects are unique identifiers, which you can use to query and modify the properties of a specific contour plot. For details, see Function Contour.

**Algorithms**

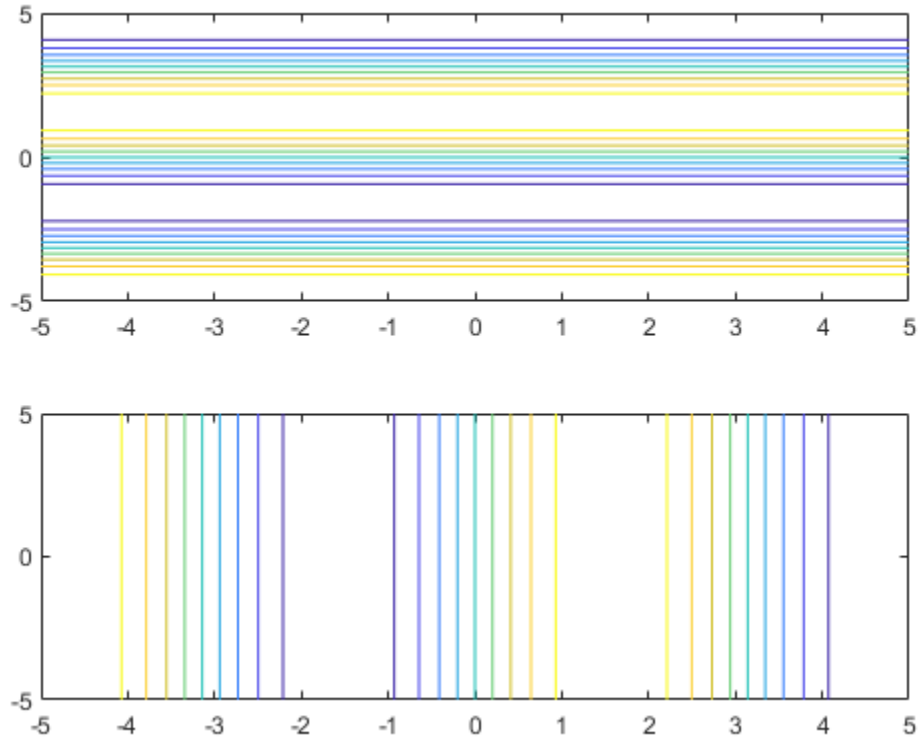
fcontour assigns the symbolic variables in **f** to the x-axis, then the y-axis, and **symvar** determines the order of the variables to be assigned. Therefore, variable and axis names might not correspond. To force fcontour to assign x or y to its corresponding axis, create the symbolic function to plot, then pass the symbolic function to fcontour.

For example, the following code plots the contour of the surface  $f(x,y) = \sin(y)$  in two ways. The first way forces the waves to oscillate with respect to the y-axis. In other words, the first plot assigns the y variable to the corresponding y-axis. The second plot assigns y to the x-axis because it is the first (and only) variable in the symbolic function.

```
syms x y;
f(x,y) = sin(y);

figure;
subplot(2,1,1)
fcontour(f);
```

```
subplot(2,1,2)  
fcontour(f(x,y)); % Or fcontour(sin(y));
```



## Version History

Introduced in R2016a

## See Also

### Functions

`fimplicit` | `fimplicit3` | `fmesh` | `fplot` | `fplot3` | `fpolarplot` | `fsurf`

### Properties

Function Contour

### Topics

“Create Plots of Symbolic Expressions” on page 4-2



# fibonacci

Fibonacci numbers

## Syntax

```
f = fibonacci(n)
```

## Description

`f = fibonacci(n)` returns the  $n^{\text{th}}$  “Fibonacci Number” on page 6-508.

## Examples

### Find Fibonacci Numbers

Find the sixth Fibonacci number by using `fibonacci`.

```
f = fibonacci(6)
```

```
f =  
8
```

Find the first 10 Fibonacci numbers.

```
n = 1:10;  
f = fibonacci(n)
```

```
f = 1×10
```

```
1      1      2      3      5      8     13     21     34     55
```

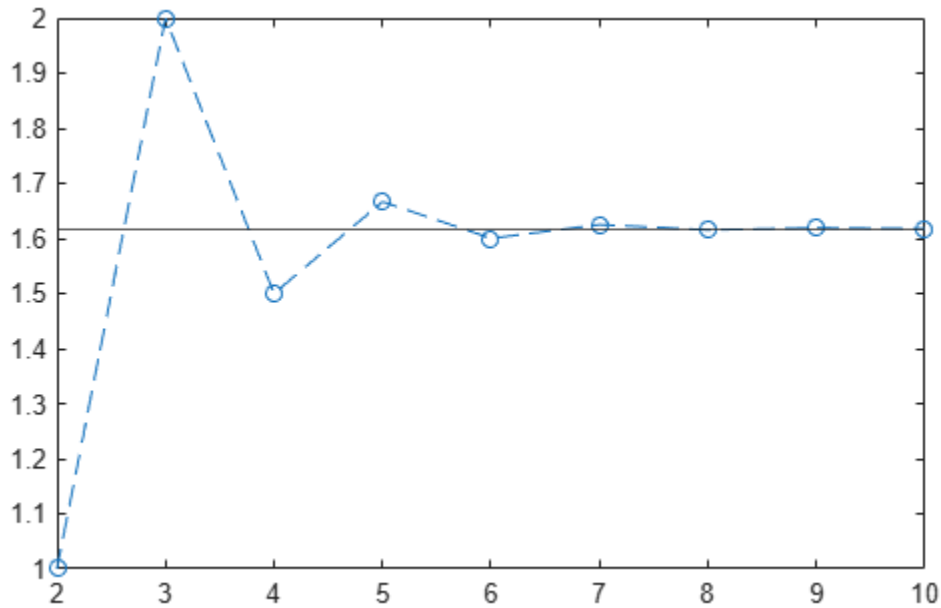
### Fibonacci Sequence Approximates Golden Ratio

The ratio of successive Fibonacci numbers converges to the golden ratio 1.61803. . . . Show this convergence by plotting this ratio against the golden ratio for the first 10 Fibonacci numbers.

```
n = 2:10;  
ratio = fibonacci(n)./fibonacci(n-1);
```

```
plot(n,ratio,'--o')  
hold on
```

```
yline(1.61803)  
hold off
```



### Symbolically Represent Fibonacci Numbers

Use Fibonacci numbers in symbolic calculations by representing them with symbolic input. `fibonacci` returns the input.

Represent the  $n^{\text{th}}$  Fibonacci number.

```
syms n
f = fibonacci(n)

f = fibonacci(n)
```

### Find Large Fibonacci Numbers

Find large Fibonacci numbers by specifying the input symbolically using `sym`. Symbolic input returns exact symbolic output instead of double output. Convert symbolic numbers to double by using the `double` function.

Find the 300th Fibonacci number.

```
num = sym(300);
f300sym = fibonacci(num)

f300sym = 222232244629420445529739893461909967206666939096499764990979600
```

Convert `fib300` to double. The result is a floating-point approximation.

```
f300double = double(f300sym)
```

```
f300double =
2.2223e+62
```

For more information on symbolic and double arithmetic, see “Choose Numeric or Symbolic Arithmetic” on page 2-32.

## Golden Spiral Using Fibonacci Numbers

The Fibonacci numbers are commonly visualized by plotting the Fibonacci spiral. The Fibonacci spiral approximates the golden spiral.

Approximate the golden spiral for the first 8 Fibonacci numbers. Define the four cases for the right, top, left, and bottom squares in the plot by using a `switch` statement. Form the spiral by defining the equations of arcs through the squares in `eqnArc`. Draw the squares and arcs by using `rectangle` and `fimplicit` respectively.

```
x = 0;
y = 1;
syms v u

axis off
hold on

for n = 1:8

    a = fibonacci(n);

    % Define squares and arcs
    switch mod(n,4)
        case 0
            y = y - fibonacci(n-2);
            x = x - a;
            eqnArc = (u-(x+a))^2 + (v-y)^2 == a^2;
        case 1
            y = y - a;
            eqnArc = (u-(x+a))^2 + (v-(y+a))^2 == a^2;
        case 2
            x = x + fibonacci(n-1);
            eqnArc = (u-x)^2 + (v-(y+a))^2 == a^2;
        case 3
            x = x - fibonacci(n-2);
            y = y + fibonacci(n-1);
            eqnArc = (u-x)^2 + (v-y)^2 == a^2;
    end

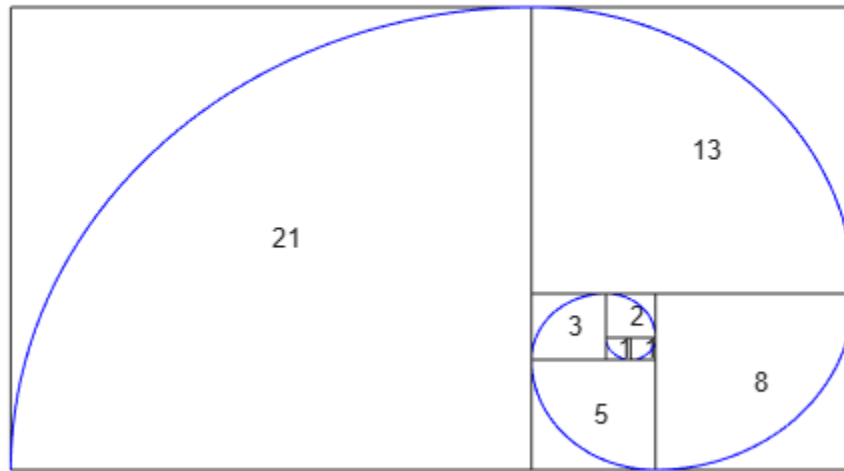
    % Draw square
    pos = [x y a a];
    rectangle('Position', pos)

    % Add Fibonacci number
    xText = (x+x+a)/2;
    yText = (y+y+a)/2;
    text(xText, yText, num2str(a))

    % Draw arc
```

```
interval = [x x+a y y+a];
fimplicit(eqnArc, interval, 'b')
```

```
end
```



## Input Arguments

### **n** — Input

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

## More About

### **Fibonacci Number**

The Fibonacci numbers are the sequence 0, 1, 1, 2, 3, 5, 8, 13, 21....

Given that the first two numbers are  $F_0 = 0$  and  $F_1 = 1$ , the  $n^{\text{th}}$  Fibonacci number is

$$F_n = F_{n-1} + F_{n-2}.$$

Applying this formula repeatedly generates the Fibonacci numbers.

## Version History

Introduced in R2017a

# fimplicit

Plot implicit symbolic equation or function

## Syntax

```
fimplicit(f)
fimplicit(f,[min max])
fimplicit(f,[xmin xmax ymin ymax])

fimplicit(___,LineStyle)
fimplicit(___,Name,Value)
fimplicit(ax,___)
fi = fimPLICIT(___)
```

## Description

`fimplicit(f)` plots the implicit symbolic equation or function `f` over the default interval `[-5 5]` for `x` and `y`.

`fimplicit(f,[min max])` plots `f` over the interval `min < x < max` and `min < y < max`.

`fimplicit(f,[xmin xmax ymin ymax])` plots `f` over the interval `xmin < x < xmax` and `ymin < y < ymax`. The `fimplicit` function uses `symvar` to order the variables and assign intervals.

`fimplicit(___,LineStyle)` uses `LineStyle` to set the line style, marker symbol, and line color.

`fimplicit(___,Name,Value)` specifies line properties using one or more `Name,Value` pair arguments. Use this option with any of the input argument combinations in the previous syntaxes. `Name,Value` pair settings apply to all the lines plotted. To set options for individual lines, use the objects returned by `fimplicit`.

`fimplicit(ax,___)` plots into the axes specified by `ax` instead of the current axes `gca`.

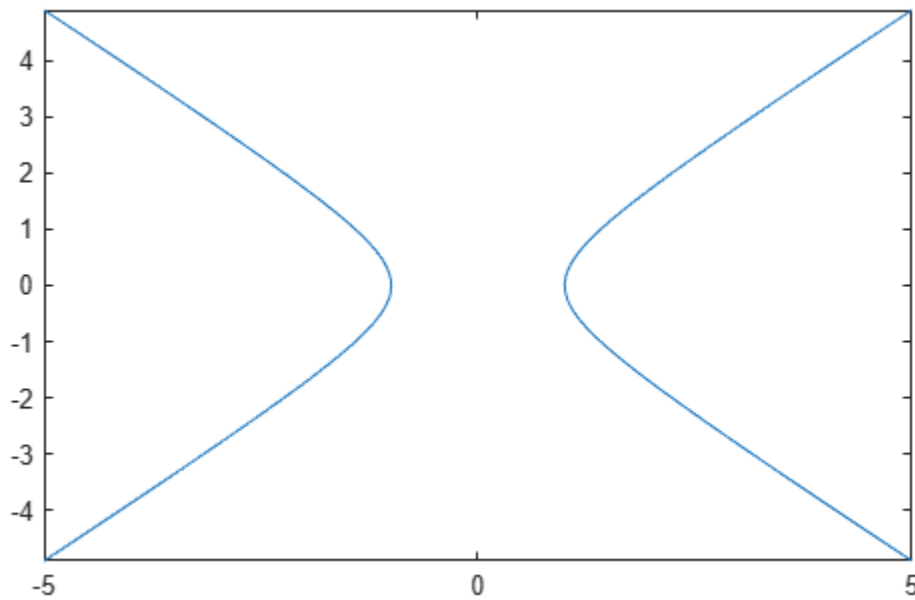
`fi = fimPLICIT(___)` returns an implicit function line object. Use the object to query and modify properties of a specific line. For details, see [Implicit Function Line](#).

## Examples

### Plot Implicit Symbolic Equation

Plot the hyperbola  $x^2 - y^2 = 1$  by using `fimplicit`. The `fimplicit` function uses the default interval of `[-5,5]` for `x` and `y`.

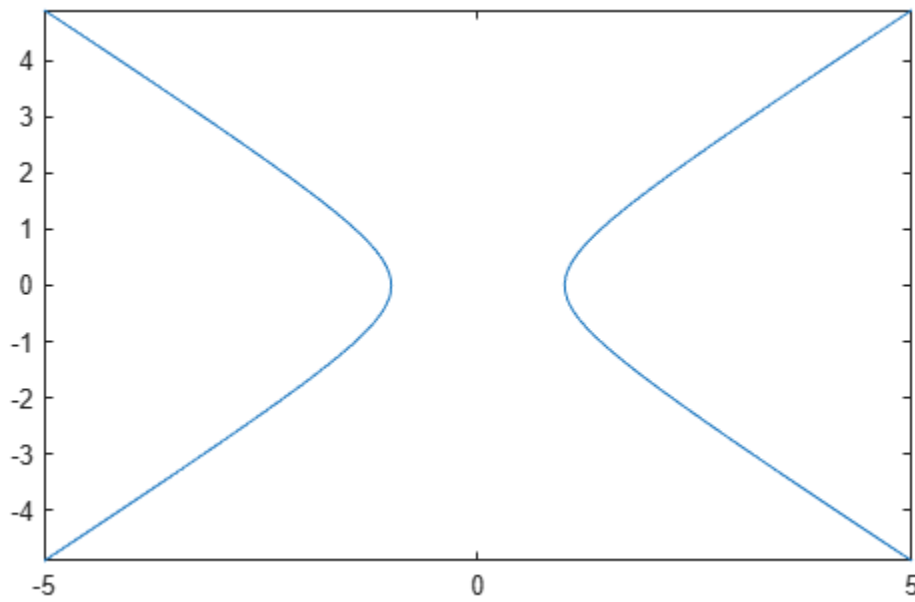
```
syms x y
fimplicit(x^2 - y^2 == 1)
```



### Plot Implicit Symbolic Function

Plot the hyperbola described by the function  $f(x, y) = x^2 - y^2 - 1$  by first declaring the symbolic function  $f(x, y)$  using `syms`. The `fimplicit` function uses the default interval of  $[-5, 5]$  for  $x$  and  $y$ .

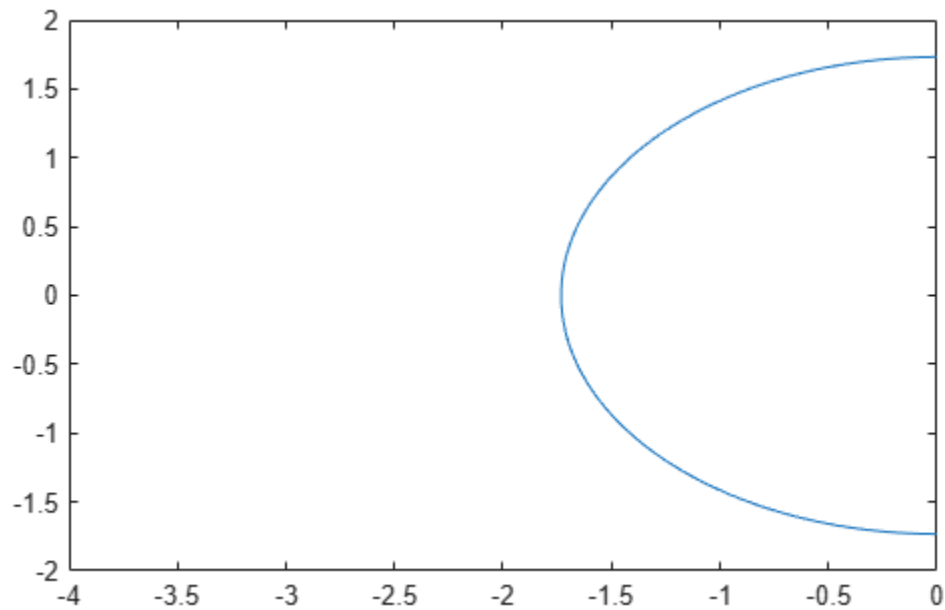
```
syms f(x,y)
f(x,y) = x^2 - y^2 - 1;
fimplicit(f)
```



### Specify Plotting Interval

Plot half of the circle  $x^2 + y^2 = 3$  by using the intervals  $-4 < x < 0$  and  $-2 < y < 2$ . Specify the plotting interval as the second argument of `fimplicit`.

```
syms x y
circle = x^2 + y^2 == 3;
fimplicit(circle, [-4 0 -2 2])
```



### Plot Multiple Implicit Equations

You can plot multiple equations either by passing the inputs as a vector or by using `hold on` to successively plot on the same figure. If you specify `LineStyle` and Name-Value arguments, they apply to all lines. To set options for individual plots, use the function handles returned by `fimplicit`.

Divide a figure into two subplots by using `subplot`. On the first subplot, plot  $x^2 + y^2 = 1$  and  $x^2 + y^2 = 3$  using vector input. On the second subplot, plot the same inputs by using `hold on`.

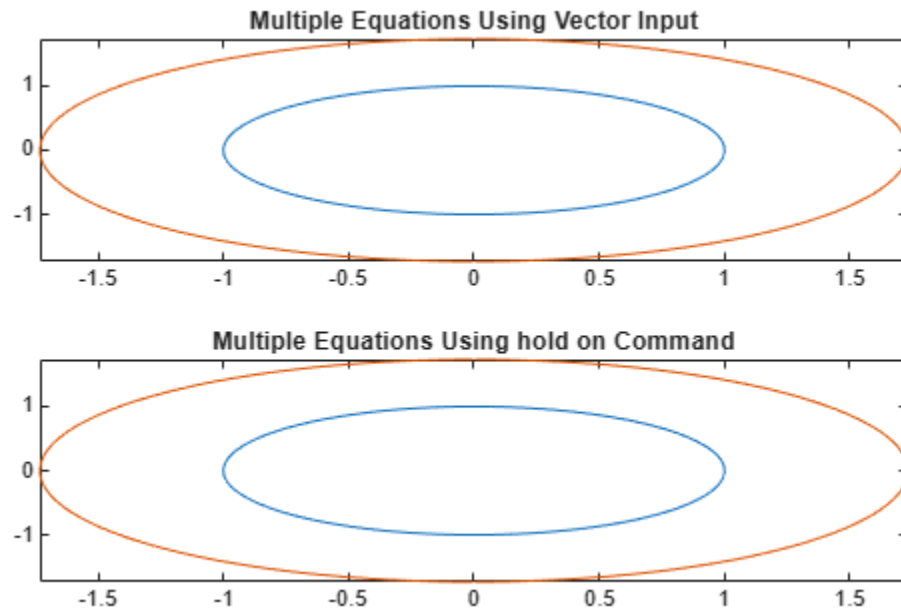
```
syms x y
circle1 = x^2 + y^2 == 1;
circle2 = x^2 + y^2 == 3;

subplot(2,1,1)
fimplicit([circle1 circle2])
title('Multiple Equations Using Vector Input')

subplot(2,1,2)
fimplicit(circle1)
hold on
fimplicit(circle2)
title('Multiple Equations Using hold on Command')

hold off
```

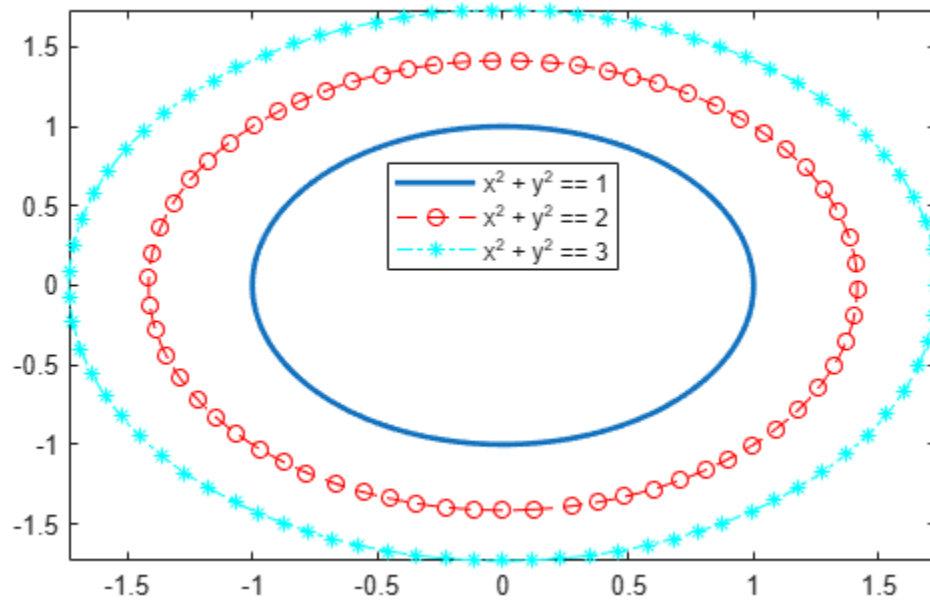




### Change Line Properties and Display Markers

Plot three concentric circles of increasing diameter. For the first line, use a linewidth of 2. For the second, specify a dashed red line style with circle markers. For the third, specify a cyan, dash-dot line style with asterisk markers. Display the legend.

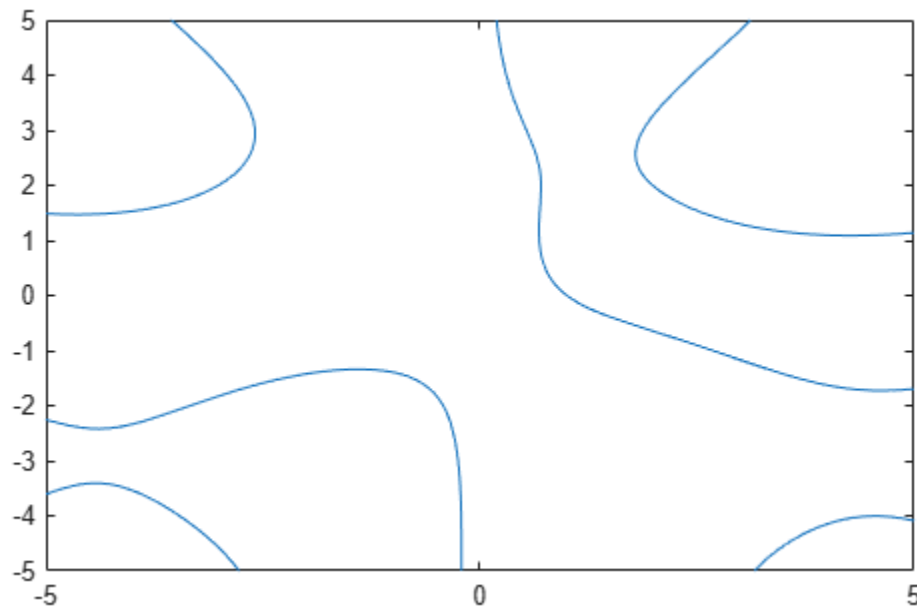
```
syms x y
circle = x^2 + y^2;
fimplicit(circle == 1, Linewidth = 2)
hold on
fimplicit(circle == 2, "--or")
fimplicit(circle == 3, "-.*c")
legend(Location="best")
hold off
```



### Modify Implicit Plot After Creation

Plot  $y\sin(x) + x\cos(y) = 1$ . Specify an output to make `fimplicit` return the plot object.

```
syms x y
eqn = y*sin(x) + x*cos(y) == 1;
fi = fimplot(eqn)
```



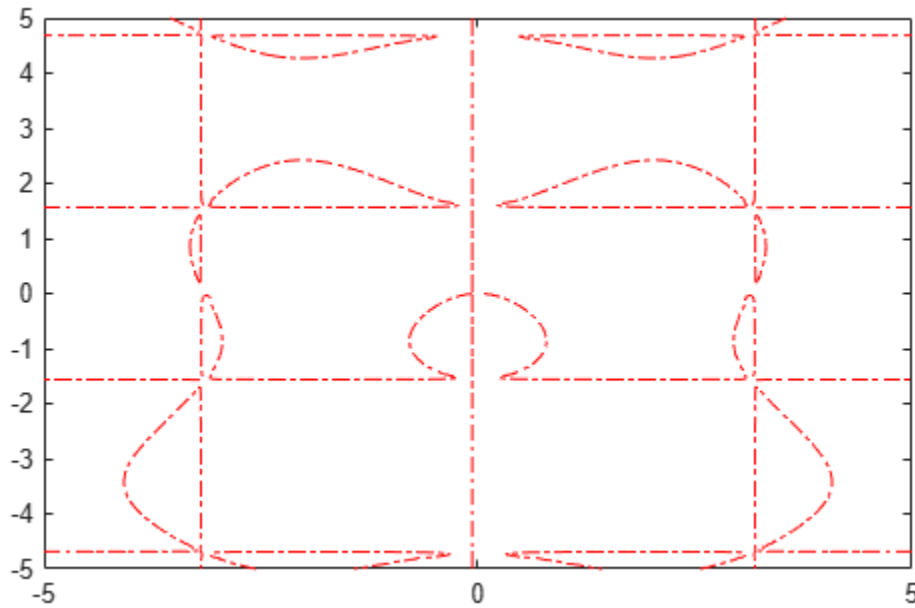
```
fi =
  ImplicitFunctionLine with properties:

    Function: x*cos(y) + y*sin(x) == 1
    Color: [0.0660 0.4430 0.7450]
    LineStyle: '-'
    LineWidth: 0.5000
```

Show all properties

Change the plotted equation to  $x\cos(y) + y\sin(x) = 0$  by using dot notation to set properties. Similarly, change the line color to red and line style to a dash-dot line. The horizontal and vertical lines in the output are artifacts that should be ignored.

```
fi.Function = x/cos(y) + y/sin(x) == 0;
fi.Color = 'r';
fi.LineStyle = '-.';
```



### Add Title and Axis Labels and Format Ticks

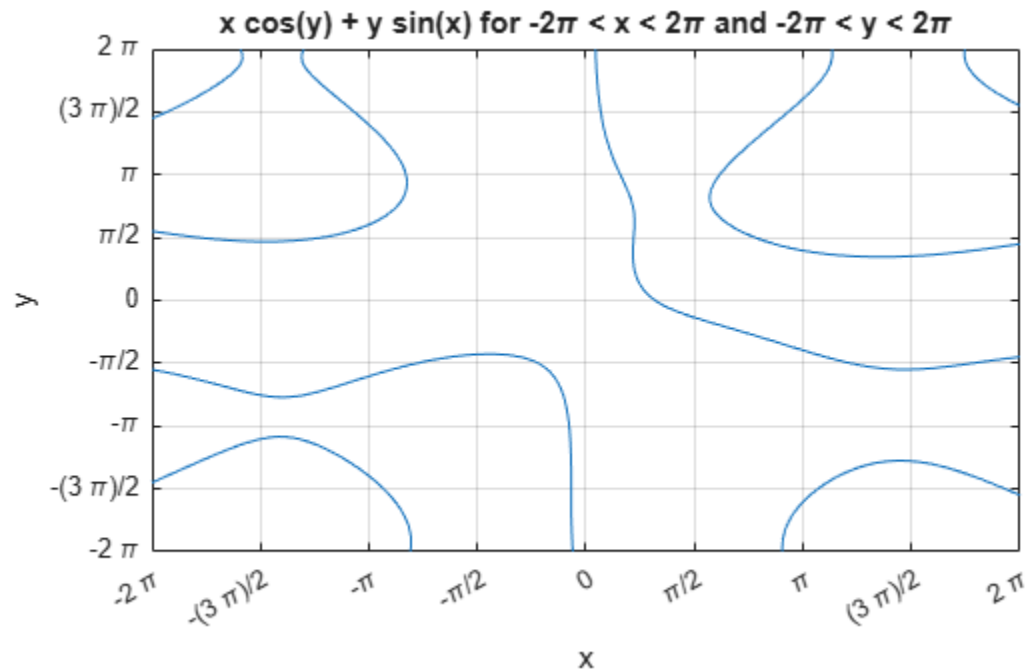
Plot  $x\cos(y) + y\sin(x) = 1$  over the interval  $-2\pi < x < 2\pi$  and  $-2\pi < y < 2\pi$ . Add a title and axis labels. Create the x-axis ticks by spanning the x-axis limits at intervals of  $\pi/2$ . Display these ticks by using the `XTick` property. Create x-axis labels by using `arrayfun` to apply `texlabel` to `S`. Display these labels by using the `XTickLabel` property. Repeat these steps for the y-axis.

To use LaTeX in plots, see `latex`.

```
syms x y
eqn = x*cos(y) + y*sin(x) == 1;
fimplicit(eqn, [-2*pi 2*pi])
grid on
title('x cos(y) + y sin(x) for -2\pi < x < 2\pi and -2\pi < y < 2\pi')
xlabel('x')
ylabel('y')
ax = gca;

S = sym(ax.XLim(1):pi/2:ax.XLim(2));
ax.XTick = double(S);
ax.XTickLabel = arrayfun(@texlabel, S, 'UniformOutput', false);

S = sym(ax.YLim(1):pi/2:ax.YLim(2));
ax.YTick = double(S);
ax.YTickLabel = arrayfun(@texlabel, S, 'UniformOutput', false);
```



### Re-Evaluation on Zoom

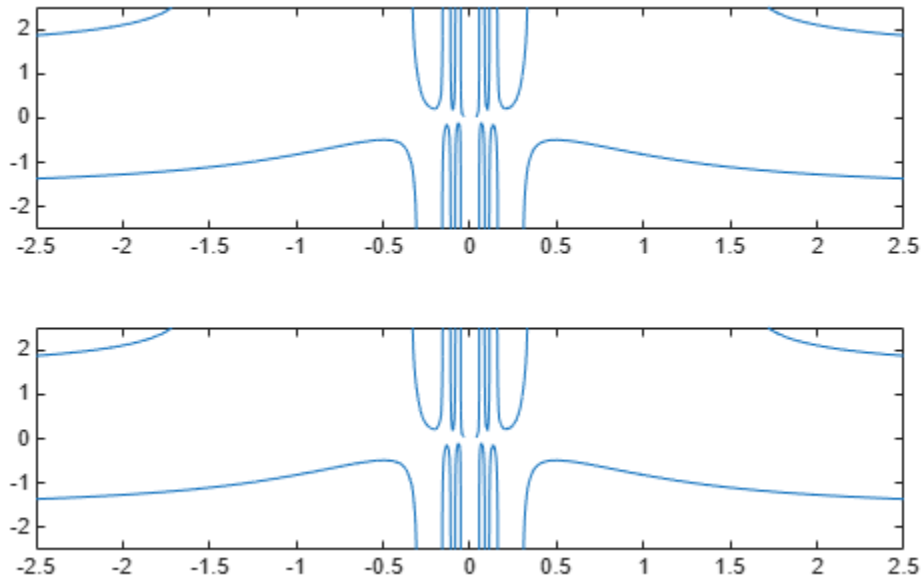
When you zoom into a plot, `fimplicit` re-evaluates the plot automatically. This re-evaluation on zoom can reveal hidden detail at smaller scales.

Divide a figure into two by using `subplot`. Plot  $x\cos(y) + y\sin(1/x) = 0$  in both the first and second subplots. Zoom into the second subplot by using `zoom`. The zoomed subplot shows detail that is not visible in the first subplot.

```
syms x y
eqn = x*cos(y) + y*sin(1/x) == 0;

subplot(2,1,1)
fimplicit(eqn)

subplot(2,1,2)
fimplicit(eqn)
zoom(2)
```



## Input Arguments

### **f** — Implicit equation or function to plot

symbolic equation | symbolic expression | symbolic function

Implicit equation or function to plot, specified as a symbolic equation, expression, or function. If the right-hand side is not specified, then it is assumed to be 0.

### **[min max]** — Plotting range for x and y

[-5 5] (default) | vector of two numbers

Plotting range for x and y, specified as a vector of two numbers. The default range is [-5 5].

### **[xmin xmax ymin ymax]** — Plotting range for x and y

[-5 5 -5 5] (default) | vector of four numbers

Plotting range for x and y, specified as a vector of four numbers. The default range is [-5 5 -5 5].

### **ax** — Axes object

axes object

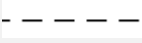
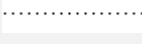
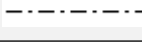
Axes object. If you do not specify an axes object, then `fimplicit` uses the current axes `gca`.

### **LineStyle** — Line style, marker, and color

string scalar | character vector

Line style, marker, and color, specified as a string scalar or character vector containing symbols. The symbols can appear in any order. You do not need to specify all three characteristics (line style, marker, and color). For example, if you omit the line style and specify the marker, then the plot shows only the marker and no line.

Example: "--or" is a red dashed line with circle markers.

| Line Style | Description      | Resulting Line                                                                      |
|------------|------------------|-------------------------------------------------------------------------------------|
| "_"        | Solid line       |  |
| "_ _"      | Dashed line      |  |
| "_:"       | Dotted line      |  |
| "_ ."      | Dash-dotted line |  |

| Marker    | Description                | Resulting Marker                                                                      |
|-----------|----------------------------|---------------------------------------------------------------------------------------|
| "o"       | Circle                     |    |
| "+"       | Plus sign                  |    |
| "*"       | Asterisk                   |    |
| "."       | Point                      |  |
| "x"       | Cross                      |  |
| "_"       | Horizontal line            |  |
| " "       | Vertical line              |  |
| "square"  | Square                     |  |
| "diamond" | Diamond                    |  |
| "^"       | Upward-pointing triangle   |  |
| "v"       | Downward-pointing triangle |  |

| Marker      | Description             | Resulting Marker                                                                    |
|-------------|-------------------------|-------------------------------------------------------------------------------------|
| ">"         | Right-pointing triangle |  |
| "<"         | Left-pointing triangle  |  |
| "pentagram" | Pentagram               |  |
| "hexagram"  | Hexagram                |  |

| Color Name | Short Name | RGB Triplet | Appearance                                                                            |
|------------|------------|-------------|---------------------------------------------------------------------------------------|
| "red"      | "r"        | [1 0 0]     |    |
| "green"    | "g"        | [0 1 0]     |    |
| "blue"     | "b"        | [0 0 1]     |    |
| "cyan"     | "c"        | [0 1 1]     |  |
| "magenta"  | "m"        | [1 0 1]     |  |
| "yellow"   | "y"        | [1 1 0]     |  |
| "black"    | "k"        | [0 0 0]     |  |
| "white"    | "w"        | [1 1 1]     |  |

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

The function line properties listed here are only a subset. For a complete list, see [Implicit Function Line](#).

Example: `'Marker','o','MarkerFaceColor','red'`

### MeshDensity — Number of evaluation points per direction

151 (default) | number

Number of evaluation points per direction, specified as a number. The default is 151.



**Color — Line color**

[0 0.4470 0.7410] (default) | RGB triplet | hexadecimal color code | "r" | "g" | "b" | ...

Line color, specified as an RGB triplet, a hexadecimal color code, a color name, or a short name.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range [0, 1], for example, [0.4 0.6 0.7].
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name | RGB Triplet | Hexadecimal Color Code | Appearance                                                                            |
|------------|------------|-------------|------------------------|---------------------------------------------------------------------------------------|
| "red"      | "r"        | [1 0 0]     | "#FF0000"              |    |
| "green"    | "g"        | [0 1 0]     | "#00FF00"              |    |
| "blue"     | "b"        | [0 0 1]     | "#0000FF"              |  |
| "cyan"     | "c"        | [0 1 1]     | "#00FFFF"              |  |
| "magenta"  | "m"        | [1 0 1]     | "#FF00FF"              |  |
| "yellow"   | "y"        | [1 1 0]     | "#FFFF00"              |  |
| "black"    | "k"        | [0 0 0]     | "#000000"              |  |
| "white"    | "w"        | [1 1 1]     | "#FFFFFF"              |  |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                       |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose( "%02X%02X%02X", round( RGB*255 ) )`.

Example: "blue"

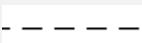
Example: [0 0 1]

Example: "#0000FF"

### LineStyle — Line style

" - " (default) | " - - " | " : " | " - . " | "none"

Line style, specified as one of the options listed in this table.

| Line Style | Description      | Resulting Line                                                                      |
|------------|------------------|-------------------------------------------------------------------------------------|
| " - "      | Solid line       |  |
| " - - "    | Dashed line      |  |
| " : "      | Dotted line      |  |
| " - . "    | Dash-dotted line |  |
| "none"     | No line          | No line                                                                             |

### LineWidth — Line width

0.5 (default) | positive value

Line width, specified as a positive value in points, where 1 point = 1/72 of an inch. If the line has markers, then the line width also affects the marker edges.

The line width cannot be thinner than the width of a pixel. If you set the line width to a value that is less than the width of a pixel on your system, the line is displayed as one pixel wide.

### Marker — Marker symbol

"none" (default) | "o" | "+" | "\*" | "." | ...

Marker symbol, specified as one of the values listed in this table. By default, the object does not display markers. Specifying a marker symbol adds markers at each data point or vertex.

| Marker | Description | Resulting Marker                                                                      |
|--------|-------------|---------------------------------------------------------------------------------------|
| "o"    | Circle      |  |
| "+"    | Plus sign   |  |
| "*"    | Asterisk    |  |
| "."    | Point       |  |

| Marker      | Description                | Resulting Marker                                                                      |
|-------------|----------------------------|---------------------------------------------------------------------------------------|
| "x"         | Cross                      |    |
| "_"         | Horizontal line            |    |
| " "         | Vertical line              |    |
| "square"    | Square                     |    |
| "diamond"   | Diamond                    |    |
| "^"         | Upward-pointing triangle   |    |
| "v"         | Downward-pointing triangle |   |
| ">"         | Right-pointing triangle    |  |
| "<"         | Left-pointing triangle     |  |
| "pentagram" | Pentagram                  |  |
| "hexagram"  | Hexagram                   |  |
| "none"      | No markers                 | Not applicable                                                                        |

### MarkerEdgeColor — Marker outline color

"auto" (default) | RGB triplet | hexadecimal color code | "r" | "g" | "b" | ...

Marker outline color, specified as "auto", an RGB triplet, a hexadecimal color code, a color name, or a short name. The default value of "auto" uses the same color as the Color property.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range  $[0, 1]$ , for example,  $[0.4 \ 0.6 \ 0.7]$ .

- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name     | RGB Triplet    | Hexadecimal Color Code | Appearance                                                                          |
|------------|----------------|----------------|------------------------|-------------------------------------------------------------------------------------|
| "red"      | "r"            | [1 0 0]        | "#FF0000"              |  |
| "green"    | "g"            | [0 1 0]        | "#00FF00"              |  |
| "blue"     | "b"            | [0 0 1]        | "#0000FF"              |  |
| "cyan"     | "c"            | [0 1 1]        | "#00FFFF"              |  |
| "magenta"  | "m"            | [1 0 1]        | "#FF00FF"              |  |
| "yellow"   | "y"            | [1 1 0]        | "#FFFF00"              |  |
| "black"    | "k"            | [0 0 0]        | "#000000"              |  |
| "white"    | "w"            | [1 1 1]        | "#FFFFFF"              |  |
| "none"     | Not applicable | Not applicable | Not applicable         | No color                                                                            |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                       |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

### MarkerFaceColor — Marker fill color

"none" (default) | "auto" | RGB triplet | hexadecimal color code | "r" | "g" | "b" | ...

Marker fill color, specified as "auto", an RGB triplet, a hexadecimal color code, a color name, or a short name. The "auto" value uses the same color as the `MarkerEdgeColor` property.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range  $[0, 1]$ , for example,  $[0.4 \ 0.6 \ 0.7]$ .
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name     | RGB Triplet    | Hexadecimal Color Code | Appearance                                                                            |
|------------|----------------|----------------|------------------------|---------------------------------------------------------------------------------------|
| "red"      | "r"            | [1 0 0]        | "#FF0000"              |    |
| "green"    | "g"            | [0 1 0]        | "#00FF00"              |    |
| "blue"     | "b"            | [0 0 1]        | "#0000FF"              |    |
| "cyan"     | "c"            | [0 1 1]        | "#00FFFF"              |    |
| "magenta"  | "m"            | [1 0 1]        | "#FF00FF"              |    |
| "yellow"   | "y"            | [1 1 0]        | "#FFFF00"              |   |
| "black"    | "k"            | [0 0 0]        | "#000000"              |  |
| "white"    | "w"            | [1 1 1]        | "#FFFFFF"              |  |
| "none"     | Not applicable | Not applicable | Not applicable         | No color                                                                              |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                       |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

Example:  $[0.3 \ 0.2 \ 0.1]$

Example: "green"

Example: "#D2F9A7"

**MarkerSize — Marker size**

6 (default) | positive value

Marker size, specified as a positive value in points, where 1 point = 1/72 of an inch.

**Output Arguments****fi — One or more implicit function line objects**

scalar | vector

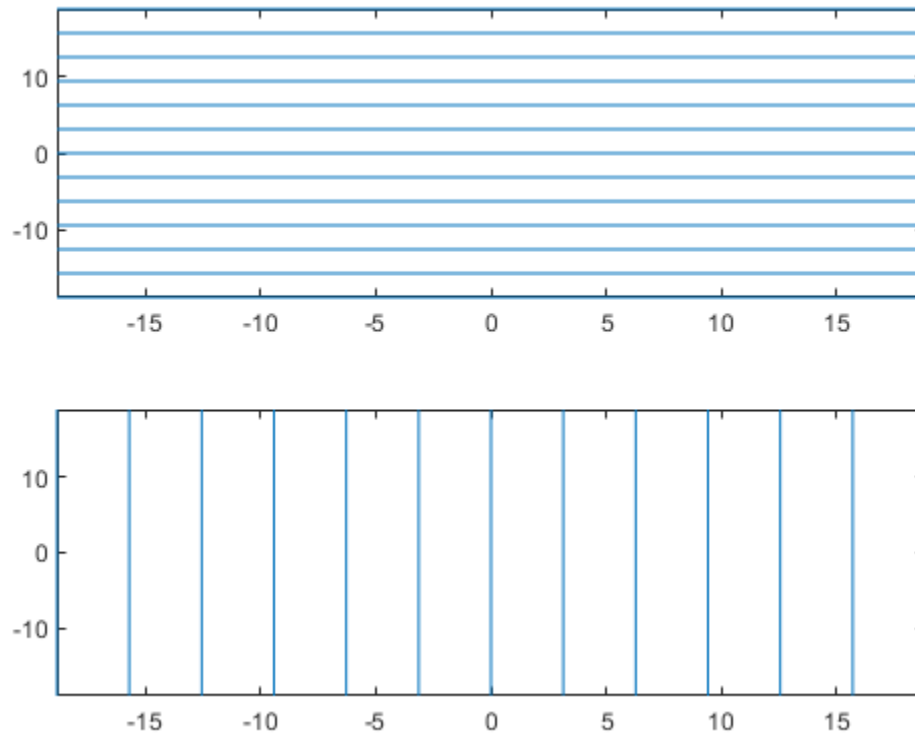
One or more implicit function line objects, returned as a scalar or a vector. You can use these objects to query and modify properties of a specific line. For a list of properties, see [Implicit Function Line](#).

**Algorithms**

`fimplicit` assigns the symbolic variables in `f` to the x-axis, then the y-axis, and `symvar` determines the order of the variables to be assigned. Therefore, variable and axis names might not correspond. To force `fimplicit` to assign `x` or `y` to its corresponding axis, create the symbolic function to plot, then pass the symbolic function to `fimplicit`.

For example, the following code plots the roots of the implicit function  $f(x,y) = \sin(y)$  in two ways. The first way forces the waves to oscillate with respect to the y-axis. In other words, the first plot assigns the `y` variable to the corresponding y-axis. The second plot assigns `y` to the x-axis because it is the first (and only) variable in the symbolic function.

```
syms x y;  
f(x,y) = sin(y);  
intvl = [-6 6]*pi;  
  
figure;  
subplot(2,1,1)  
fimplicit(f,intvl);  
subplot(2,1,2)  
fimplicit(f(x,y),intvl); % Or fimplicit(sin(y) == 0,intvl);
```



## Version History

Introduced in R2016b

## See Also

### Functions

`fcontour` | `fimplicit3` | `fmesh` | `fplot` | `fplot3` | `fpolarplot` | `fsurf`

### Properties

Implicit Function Line

### Topics

“Create Plots of Symbolic Expressions” on page 4-2

## fimplicit3

Plot 3-D implicit equation or function

### Syntax

```
fimplicit3(f)
fimplicit3(f,[min max])
fimplicit3(f,[xmin xmax ymin ymax zmin zmax])

fimplicit3(___,LineSpec)
fimplicit3(___,Name,Value)
fimplicit3(ax,___)
fi = fimplicit3(___)
```

### Description

`fimplicit3(f)` plots the 3-D implicit equation or function  $f(x,y,z)$  over the default interval  $[-5, 5]$  for  $x$ ,  $y$ , and  $z$ .

`fimplicit3(f,[min max])` plots  $f(x,y,z)$  over the interval  $[min\ max]$  for  $x$ ,  $y$ , and  $z$ .

`fimplicit3(f,[xmin xmax ymin ymax zmin zmax])` plots  $f(x,y,z)$  over the interval  $[xmin\ xmax]$  for  $x$ ,  $[ymin\ ymax]$  for  $y$ , and  $[zmin\ zmax]$  for  $z$ . The `fimplicit3` function uses `symvar` to order the variables and assign intervals.

`fimplicit3(___,LineSpec)` uses `LineSpec` to set the line style, marker symbol, and face color.

`fimplicit3(___,Name,Value)` specifies line properties using one or more `Name,Value` pair arguments. Use this option with any of the input argument combinations in the previous syntaxes.

`fimplicit3(ax,___)` plots into the axes with the object `ax` instead of the current axes object `gca`.

`fi = fimplicit3(___)` returns an implicit function surface object. Use the object to query and modify properties of a specific surface. For details, see [Implicit Function Surface](#).

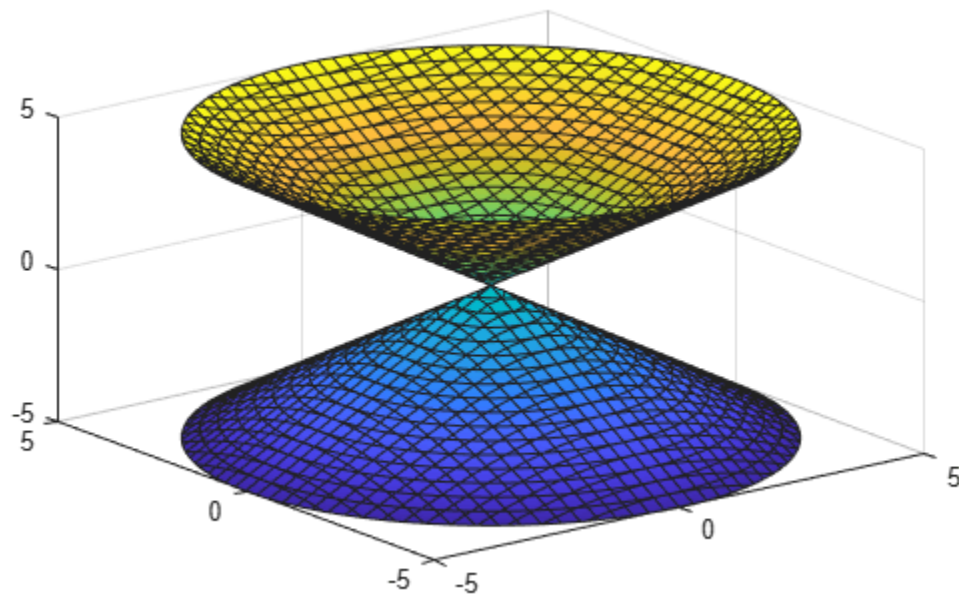
### Examples

#### Plot 3-D Implicit Symbolic Equation

Plot the hyperboloid  $x^2 + y^2 - z^2 = 0$  by using `fimplicit3`. The `fimplicit3` function plots over the default interval of  $[-5, 5]$  for  $x$ ,  $y$ , and  $z$ .

```
syms x y z
fimplicit3(x^2 + y^2 - z^2)
```

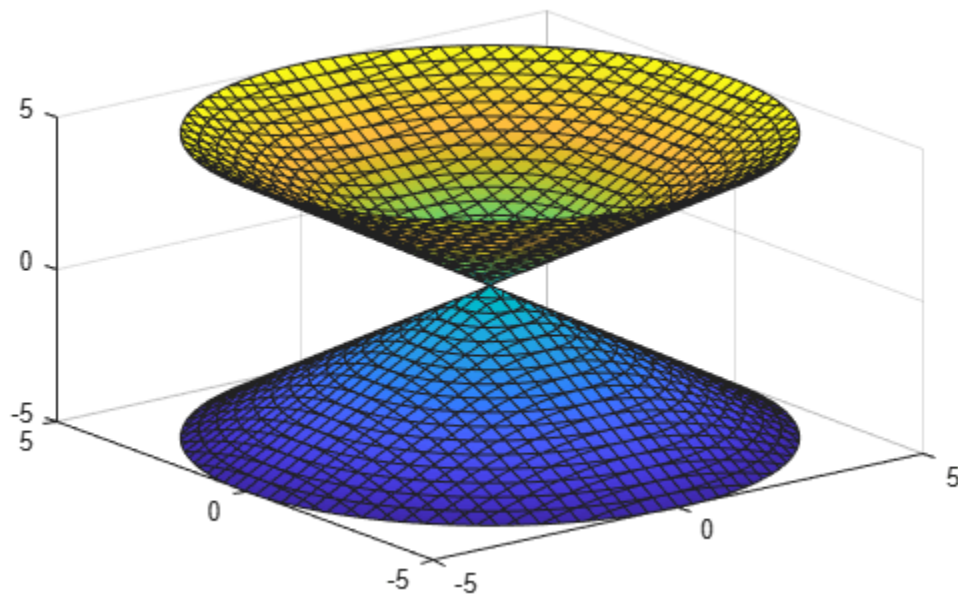




### Plot 3-D Implicit Symbolic Function

Plot the hyperboloid specified by the function  $f(x, y, z) = x^2 + y^2 - z^2$ . The `fimplicit3` function plots over the default interval of  $[-5, 5]$  for  $x$ ,  $y$ , and  $z$ .

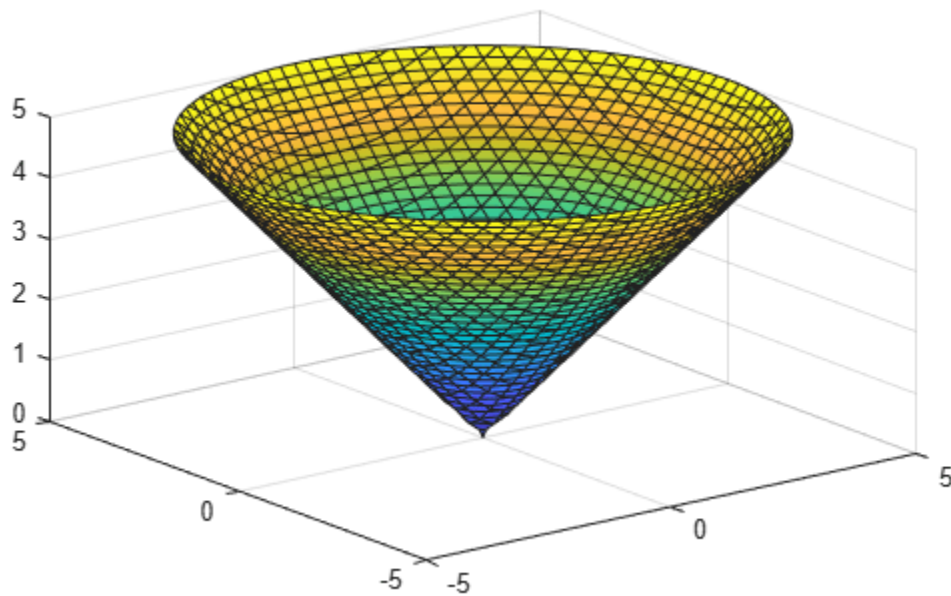
```
syms f(x,y,z)
f(x,y,z) = x^2 + y^2 - z^2;
fimplicit3(f)
```



### Specify Plotting Interval

Specify the plotting interval by specifying the second argument to `fimplicit3`. Plot the upper half of the hyperboloid  $x^2 + y^2 - z^2 = 0$  by specifying the interval  $0 < z < 5$ . For  $x$  and  $y$ , use the default interval  $[-5, 5]$ .

```
syms x y z
f = x^2 + y^2 - z^2;
interval = [-5 5 -5 5 0 5];
fimplicit3(f, interval)
```



### Add Title and Axis Labels and Format Ticks

Plot the implicit equation  $x\sin(y) + z\cos(x) = 0$  over the interval  $(-2\pi, 2\pi)$  for all axes.

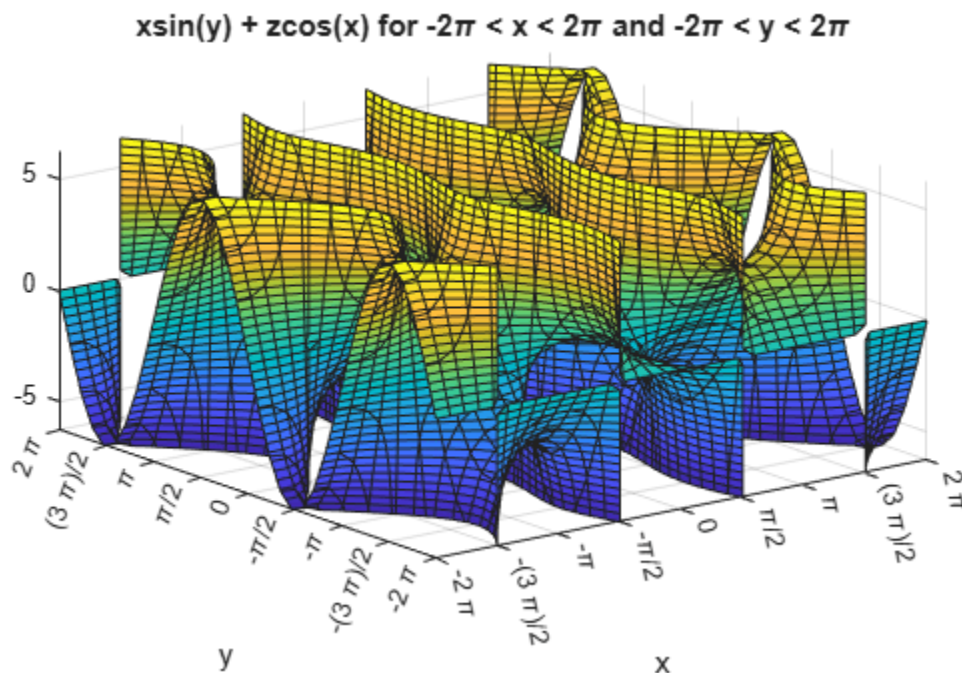
Create the x-axis ticks by spanning the x-axis limits at intervals of  $\pi/2$ . Convert the axis limits to precise multiples of  $\pi/2$  by using `round` and get the symbolic tick values in `S`. Display these ticks by using the `XTick` property. Create x-axis labels by using `arrayfun` to apply `texlabel` to `S`. Display these labels by using the `XTickLabel` property. Repeat these steps for the y-axis.

To use LaTeX in plots, see `latex`.

```
syms x y z
eqn = x*sin(y) + z*cos(x);
fimplicit3(eqn, [-2*pi 2*pi])
title('xsin(y) + zcos(x) for -2\pi < x < 2\pi and -2\pi < y < 2\pi')
xlabel('x')
ylabel('y')
ax = gca;

S = sym(ax.XLim(1):pi/2:ax.XLim(2));
S = sym(round(vpa(S/pi*2))*pi/2);
ax.XTick = double(S);
ax.XTickLabel = arrayfun(@texlabel, S, 'UniformOutput', false);

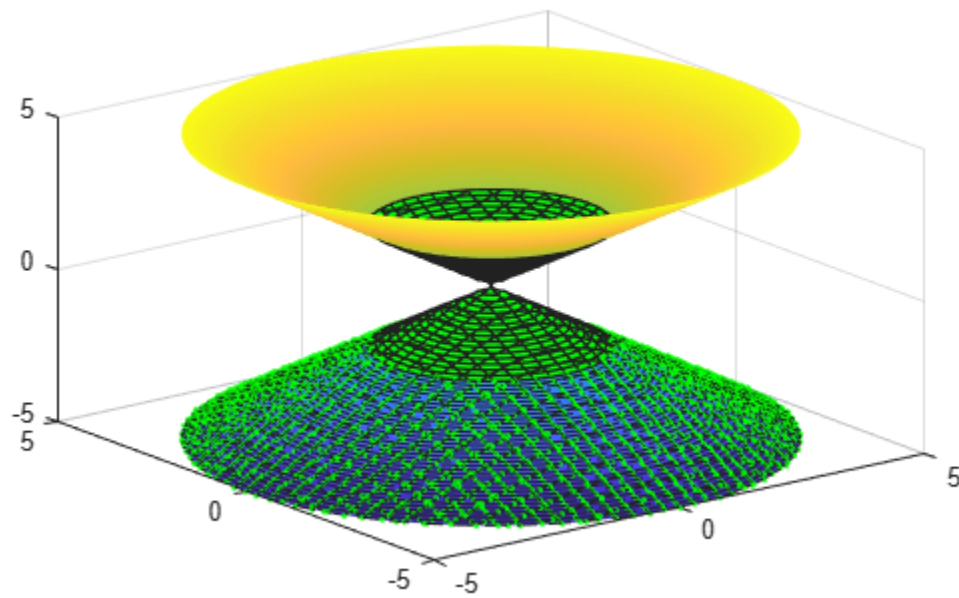
S = sym(ax.YLim(1):pi/2:ax.YLim(2));
S = sym(round(vpa(S/pi*2))*pi/2);
ax.YTick = double(S);
ax.YTickLabel = arrayfun(@texlabel, S, 'UniformOutput', false);
```



### Line Style and Width for Implicit Surface Plot

Plot the implicit surface  $x^2 + y^2 - z^2 = 0$  with different line styles for different values of  $z$ . For  $-5 < z < -2$ , use a dashed line with green dot markers. For  $-2 < z < 2$ , use a `LineWidth` of 1 and a green face color. For  $2 < z < 5$ , turn off the lines by setting `EdgeColor` to `none`.

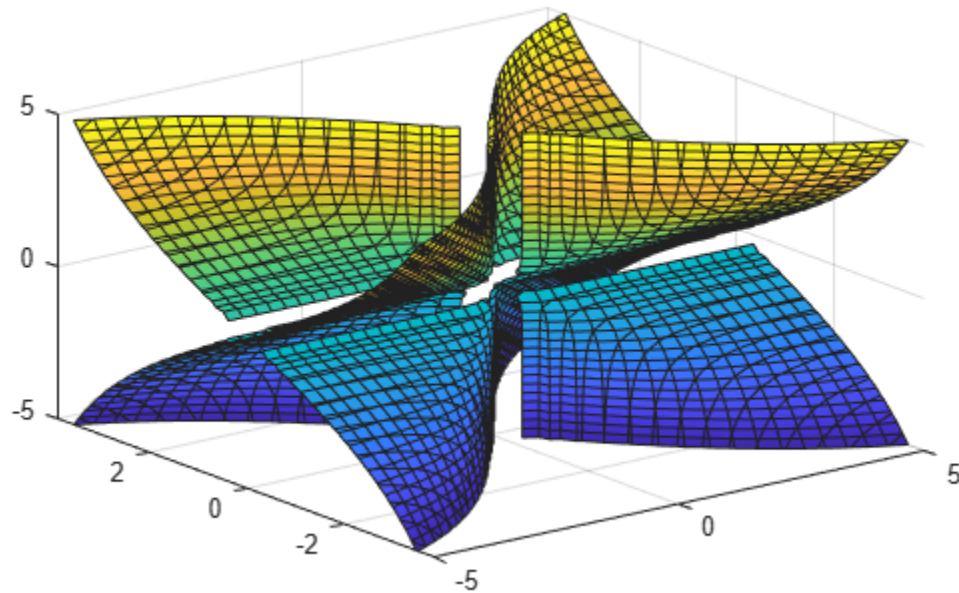
```
syms x y z
f = x^2 + y^2 - z^2;
fimplicit3(f, [-5 5 -5 5 -5 -2], '--.', 'MarkerEdgeColor', 'g')
hold on
fimplicit3(f, [-5 5 -5 5 -2 2], 'LineWidth', 1, 'FaceColor', 'g')
fimplicit3(f, [-5 5 -5 5 2 5], 'EdgeColor', 'none')
```



### Modify Implicit Surface After Creation

Plot the implicit surface  $1/x^2 - 1/y^2 + 1/z^2 = 0$ . Specify an output to make `fimplicit3` return the plot object.

```
syms x y z
f = 1/x^2 - 1/y^2 + 1/z^2;
fi = fimplicit3(f)
```

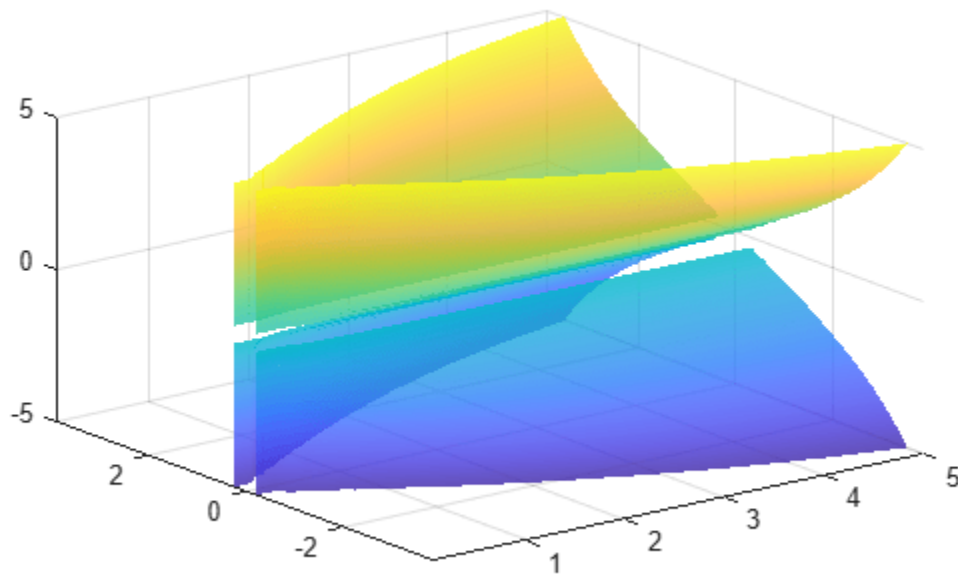


```
fi =
  ImplicitFunctionSurface with properties:
    Function: 1/x^2 - 1/y^2 + 1/z^2
    EdgeColor: [0.1294 0.1294 0.1294]
    LineStyle: '-'
    FaceColor: 'interp'

  Show all properties
```

Show only the positive x-axis by setting the `XRange` property of `fi` to `[0 5]`. Remove the lines by setting the `EdgeColor` property to `'none'`. Visualize the hidden surfaces by making the plot transparent by setting the `FaceAlpha` property to `0.8`.

```
fi.XRange = [0 5];
fi.EdgeColor = 'none';
```



```
fi.FaceAlpha = 0.8;
```

### Control Resolution of Implicit Surface Plot

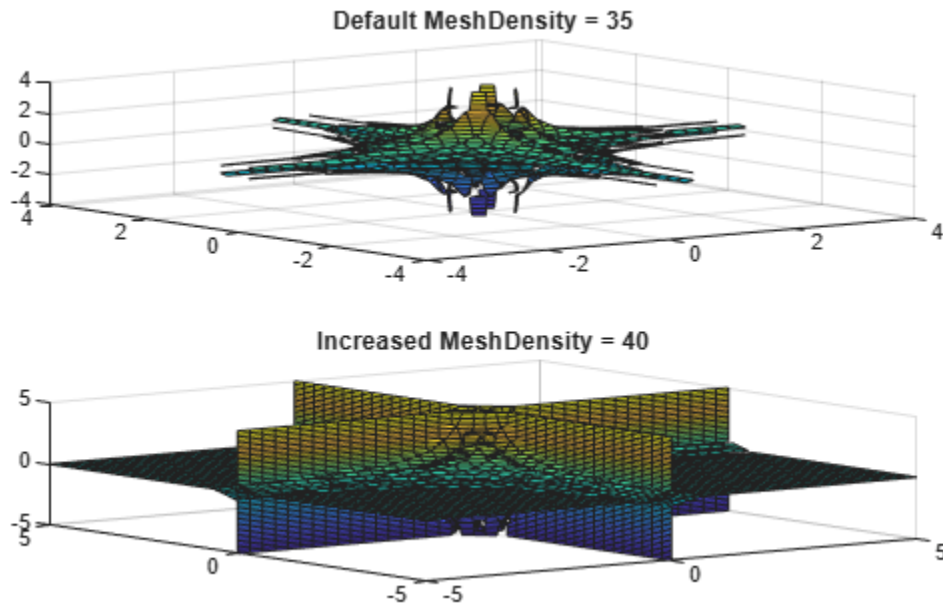
Control the resolution of an implicit surface plot by using the 'MeshDensity' option. Increasing 'MeshDensity' can make smoother, more accurate plots while decreasing 'MeshDensity' can increase plotting speed.

Divide a figure into two by using `subplot`. In the first subplot, plot the implicit surface  $\sin(1/(xyz))$ . The surface has large gaps. Fix this issue by increasing the 'MeshDensity' to 40 in the second subplot. `fimplicit3` fills the gaps showing that by increasing 'MeshDensity' you increased the resolution of the plot.

```
syms x y z
f = sin(1/(x*y*z));

subplot(2,1,1)
fimplicit3(f)
title('Default MeshDensity = 35')

subplot(2,1,2)
fimplicit3(f,'MeshDensity',40)
title('Increased MeshDensity = 40')
```



### Apply Rotation and Translation to Implicit Surface Plot

Apply rotation and translation to the implicit surface plot of a torus.

A torus can be defined by an implicit equation in Cartesian coordinates as

$$f(x, y, z) = (x^2 + y^2 + z^2 + R^2 - a^2)^2 - 4R^2(x^2 + y^2)$$

where

- $a$  is the radius of the tube
- $R$  is the distance from the center of the tube to the center of the torus

Define the values for  $a$  and  $R$  as 1 and 5, respectively. Plot the torus using `fimplicit3`.

```
syms x y z
a = 1;
R = 4;
f(x,y,z) = (x^2+y^2+z^2+R^2-a^2)^2 - 4*R^2*(x^2+y^2);
fimplicit3(f)
hold on
```

Apply rotation to the torus around the  $x$ -axis. Define the rotation matrix. Rotate the torus by 90 degrees or  $\pi/2$  radians. Shift the center of the torus by 5 along the  $x$ -axis.

```
alpha = pi/2;
Rx = [1 0 0;
      0 cos(alpha) sin(alpha);
      0 -sin(alpha) cos(alpha)];
```



```

r = [x; y; z];
r_90 = Rx*r;
g = subs(f,[x,y,z],[r_90(1)-5,r_90(2),r_90(3)]);

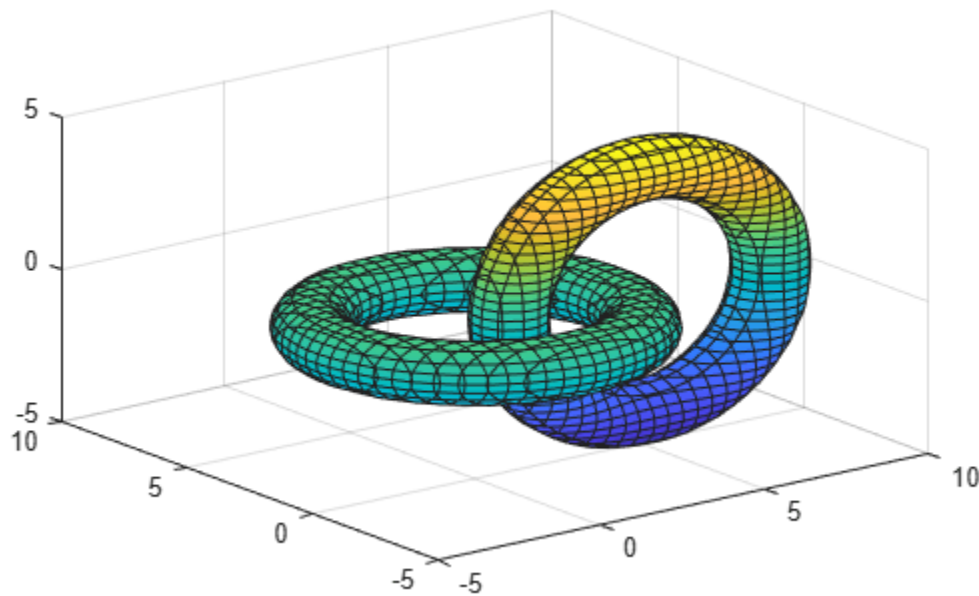
```

Add a second plot of the rotated and translated torus to the existing graph.

```

fimplicit3(g)
axis([-5 10 -5 10 -5 5])
hold off

```



## Input Arguments

### **f** — 3-D implicit equation or function to plot

symbolic equation | symbolic expression | symbolic function

3-D implicit equation or function to plot, specified as a symbolic equation, expression, or function. If an expression or function is specified, then `fimplicit3` assumes the right-hand side to be 0.

### **[min max]** — Plotting interval for x-, y- and z- axes

[-5 5] (default) | vector of two numbers

Plotting interval for x-, y- and z- axes, specified as a vector of two numbers. The default is [-5 5].

### **[xmin xmax ymin ymax zmin zmax]** — Plotting interval for x-, y- and z- axes

[-5 5 -5 5 -5 5] (default) | vector of six numbers

Plotting interval for x-, y- and z- axes, specified as a vector of six numbers. The default is [-5 5 -5 5 -5 5].

### **ax** — Axes object

axes object

Axes object. If you do not specify an axes object, then `fimplicit3` uses the current axes.

### LineStyle — Line style, marker, and color

string scalar | character vector

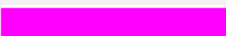
Line style, marker, and color, specified as a string scalar or character vector containing symbols. The symbols can appear in any order. You do not need to specify all three characteristics (line style, marker, and color). For example, if you omit the line style and specify the marker, then the plot shows only the marker and no line.

Example: `"- -or"` is a red dashed line with circle markers.

| Line Style | Description      | Resulting Line                                                                      |
|------------|------------------|-------------------------------------------------------------------------------------|
| " - "      | Solid line       |  |
| " - - "    | Dashed line      |  |
| " : "      | Dotted line      |  |
| " - . "    | Dash-dotted line |  |

| Marker   | Description     | Resulting Marker                                                                      |
|----------|-----------------|---------------------------------------------------------------------------------------|
| "o"      | Circle          |   |
| "+"      | Plus sign       |  |
| "*"      | Asterisk        |  |
| ". "     | Point           |  |
| "x"      | Cross           |  |
| " _ "    | Horizontal line |  |
| "   "    | Vertical line   |  |
| "square" | Square          |  |

| Marker      | Description                | Resulting Marker                                                                     |
|-------------|----------------------------|--------------------------------------------------------------------------------------|
| "diamond"   | Diamond                    |   |
| "^"         | Upward-pointing triangle   |   |
| "v"         | Downward-pointing triangle |   |
| ">"         | Right-pointing triangle    |   |
| "<"         | Left-pointing triangle     |   |
| "pentagram" | Pentagram                  |   |
| "hexagram"  | Hexagram                   |  |

| Color Name | Short Name | RGB Triplet | Appearance                                                                            |
|------------|------------|-------------|---------------------------------------------------------------------------------------|
| "red"      | "r"        | [1 0 0]     |  |
| "green"    | "g"        | [0 1 0]     |  |
| "blue"     | "b"        | [0 0 1]     |  |
| "cyan"     | "c"        | [0 1 1]     |  |
| "magenta"  | "m"        | [1 0 1]     |  |
| "yellow"   | "y"        | [1 1 0]     |  |
| "black"    | "k"        | [0 0 0]     |  |
| "white"    | "w"        | [1 1 1]     |  |

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: 'Marker', 'o', 'MarkerFaceColor', 'red'

The properties listed here are only a subset. For a complete list, see Implicit Function Surface.

### MeshDensity — Number of evaluation points per direction

35 (default) | number

Number of evaluation points per direction, specified as a number. The default is 35.

Example: 100

### EdgeColor — Line color

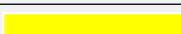
[0 0 0] (default) | 'interp' | RGB triplet | hexadecimal color code | 'r' | 'g' | 'b' | ...

Line color, specified as 'interp', an RGB triplet, a hexadecimal color code, a color name, or a short name. The default RGB triplet value of [0 0 0] corresponds to black. The 'interp' value colors the edges based on the ZData values.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range [0, 1], for example, [0.4 0.6 0.7].
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name     | RGB Triplet    | Hexadecimal Color Code | Appearance                                                                            |
|------------|----------------|----------------|------------------------|---------------------------------------------------------------------------------------|
| "red"      | "r"            | [1 0 0]        | "#FF0000"              |  |
| "green"    | "g"            | [0 1 0]        | "#00FF00"              |  |
| "blue"     | "b"            | [0 0 1]        | "#0000FF"              |  |
| "cyan"     | "c"            | [0 1 1]        | "#00FFFF"              |  |
| "magenta"  | "m"            | [1 0 1]        | "#FF00FF"              |  |
| "yellow"   | "y"            | [1 1 0]        | "#FFFF00"              |  |
| "black"    | "k"            | [0 0 0]        | "#000000"              |  |
| "white"    | "w"            | [1 1 1]        | "#FFFFFF"              |  |
| "none"     | Not applicable | Not applicable | Not applicable         | No color                                                                              |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                     |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

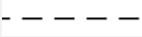
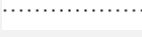
Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

### LineStyle — Line style

"-" (default) | "--" | ":" | "-." | "none"

Line style, specified as one of the options listed in this table.

| Line Style | Description      | Resulting Line                                                                        |
|------------|------------------|---------------------------------------------------------------------------------------|
| " - "      | Solid line       |  |
| " - - "    | Dashed line      |  |
| " : "      | Dotted line      |  |
| " - . "    | Dash-dotted line |  |
| "none"     | No line          | No line                                                                               |

### LineWidth — Line width

0.5 (default) | positive value

Line width, specified as a positive value in points, where 1 point = 1/72 of an inch. If the line has markers, then the line width also affects the marker edges.

The line width cannot be thinner than the width of a pixel. If you set the line width to a value that is less than the width of a pixel on your system, the line is displayed as one pixel wide.

### Marker — Marker symbol

"none" (default) | "o" | "+" | "\*" | "." | ...

Marker symbol, specified as one of the values listed in this table. By default, the object does not display markers. Specifying a marker symbol adds markers at each data point or vertex.

| Marker      | Description                | Resulting Marker                                                                      |
|-------------|----------------------------|---------------------------------------------------------------------------------------|
| "o"         | Circle                     |    |
| "+"         | Plus sign                  |    |
| "*"         | Asterisk                   |    |
| ". "        | Point                      |    |
| "x"         | Cross                      |    |
| "_ "        | Horizontal line            |    |
| "  "        | Vertical line              |   |
| "square"    | Square                     |  |
| "diamond"   | Diamond                    |  |
| "^"         | Upward-pointing triangle   |  |
| "v"         | Downward-pointing triangle |  |
| ">"         | Right-pointing triangle    |  |
| "<"         | Left-pointing triangle     |  |
| "pentagram" | Pentagram                  |  |

| Marker     | Description | Resulting Marker                                                                    |
|------------|-------------|-------------------------------------------------------------------------------------|
| "hexagram" | Hexagram    |  |
| "none"     | No markers  | Not applicable                                                                      |

### MarkerEdgeColor — Marker outline color

'auto' (default) | RGB triplet | hexadecimal color code | 'r' | 'g' | 'b' | ...

Marker outline color, specified as 'auto', an RGB triplet, a hexadecimal color code, a color name, or a short name. The default value of 'auto' uses the same color as the EdgeColor property.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range  $[0, 1]$ , for example,  $[0.4 \ 0.6 \ 0.7]$ .
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name     | RGB Triplet    | Hexadecimal Color Code | Appearance                                                                            |
|------------|----------------|----------------|------------------------|---------------------------------------------------------------------------------------|
| "red"      | "r"            | [1 0 0]        | "#FF0000"              |  |
| "green"    | "g"            | [0 1 0]        | "#00FF00"              |  |
| "blue"     | "b"            | [0 0 1]        | "#0000FF"              |  |
| "cyan"     | "c"            | [0 1 1]        | "#00FFFF"              |  |
| "magenta"  | "m"            | [1 0 1]        | "#FF00FF"              |  |
| "yellow"   | "y"            | [1 1 0]        | "#FFFF00"              |  |
| "black"    | "k"            | [0 0 0]        | "#000000"              |  |
| "white"    | "w"            | [1 1 1]        | "#FFFFFF"              |  |
| "none"     | Not applicable | Not applicable | Not applicable         | No color                                                                              |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                       |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |

| Palette                     | Palette Colors                                                                     |
|-----------------------------|------------------------------------------------------------------------------------|
| "glow" — Dark theme default |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

Example: `[0.5 0.5 0.5]`

Example: `'blue'`

Example: `'#D2F9A7'`

### MarkerFaceColor — Marker fill color

"none" (default) | "auto" | RGB triplet | hexadecimal color code | "r" | "g" | "b" | ...

Marker fill color, specified as "auto", an RGB triplet, a hexadecimal color code, a color name, or a short name. The "auto" value uses the same color as the `MarkerEdgeColor` property.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range `[0, 1]`, for example, `[0.4 0.6 0.7]`.
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes `"#FF8800"`, `"#ff8800"`, `"#F80"`, and `"#f80"` are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name | RGB Triplet | Hexadecimal Color Code | Appearance                                                                            |
|------------|------------|-------------|------------------------|---------------------------------------------------------------------------------------|
| "red"      | "r"        | [1 0 0]     | "#FF0000"              |  |
| "green"    | "g"        | [0 1 0]     | "#00FF00"              |  |
| "blue"     | "b"        | [0 0 1]     | "#0000FF"              |  |
| "cyan"     | "c"        | [0 1 1]     | "#00FFFF"              |  |
| "magenta"  | "m"        | [1 0 1]     | "#FF00FF"              |  |
| "yellow"   | "y"        | [1 1 0]     | "#FFFF00"              |  |



| Color Name | Short Name     | RGB Triplet    | Hexadecimal Color Code | Appearance                                                                          |
|------------|----------------|----------------|------------------------|-------------------------------------------------------------------------------------|
| "black"    | "k"            | [0 0 0]        | "#000000"              |  |
| "white"    | "w"            | [1 1 1]        | "#FFFFFF"              |  |
| "none"     | Not applicable | Not applicable | Not applicable         | No color                                                                            |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                     |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot,"FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

Example: [0.3 0.2 0.1]

Example: "green"

Example: "#D2F9A7"

### MarkerSize — Marker size

6 (default) | positive value

Marker size, specified as a positive value in points, where 1 point = 1/72 of an inch.

## Output Arguments

### fi — One or more objects

scalar | vector

One or more objects, returned as a scalar or a vector. The object is an implicit function surface object. You can use these objects to query and modify properties of a specific line. For details, see Implicit Function Surface.

## Algorithms

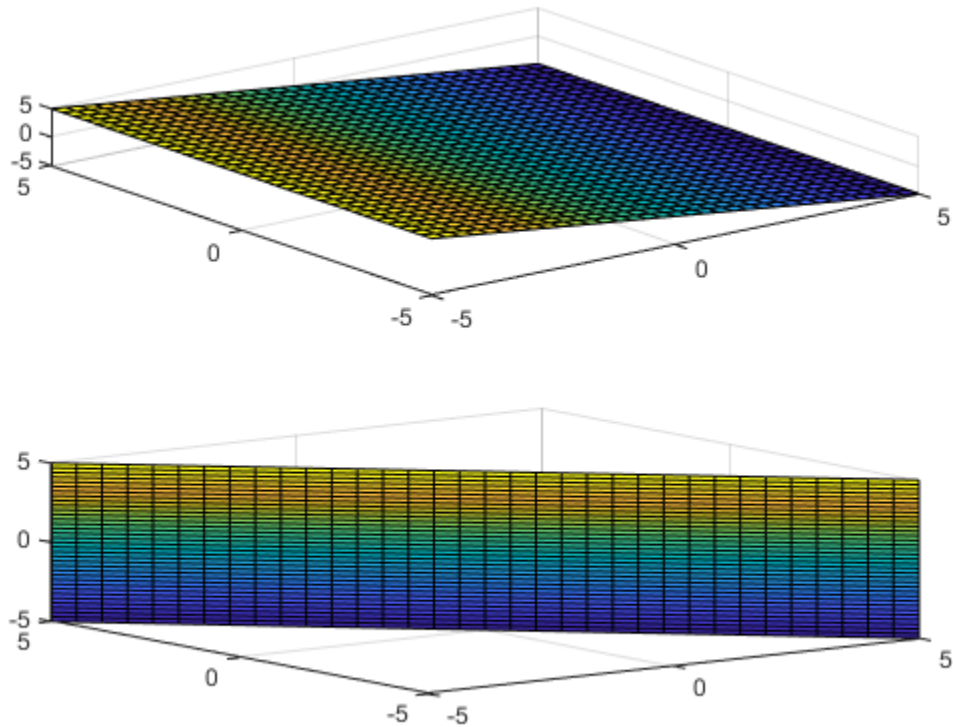
`fimplicit3` assigns the symbolic variables in `f` to the `x` axis, the `y` axis, then the `z` axis, and `symvar` determines the order of the variables to be assigned. Therefore, variable and axis names might not

correspond. To force `fimplicit3` to assign  $x$ ,  $y$ , or  $z$  to its corresponding axis, create the symbolic function to plot, then pass the symbolic function to `fimplicit3`.

For example, the following code plots the roots of the implicit function  $f(x,y,z) = x + z$  in two ways. The first way forces `fimplicit3` to assign  $x$  and  $z$  to their corresponding axes. In the second way, `fimplicit3` defers to `symvar` to determine variable order and axis assignment: `fimplicit3` assigns  $x$  and  $z$  to the  $x$ - and  $y$ -axes, respectively.

```
syms x y z;
f(x,y,z) = x + z;

figure;
subplot(2,1,1)
fimplicit3(f);
view(-38,71);
subplot(2,1,2)
fimplicit3(f(x,y,z)); % Or fimplicit3(x + z);
```



## Version History

Introduced in R2016b

## See Also

### Functions

`fcontour` | `fimplicit` | `fmesh` | `fplot` | `fplot3` | `fpolarplot` | `fsurf`

**Properties**

Implicit Function Surface

**Topics**

“Create Plots of Symbolic Expressions” on page 4-2

## findDecoupledBlocks

Search for decoupled blocks in systems of equations

### Syntax

```
[eqsBlocks, varsBlocks] = findDecoupledBlocks(eqs, vars)
```

### Description

`[eqsBlocks, varsBlocks] = findDecoupledBlocks(eqs, vars)` identifies subsets (blocks) of equations that can be used to define subsets of variables. The number of variables `vars` must coincide with the number of equations `eqs`.

The  $i$ th block is the set of equations determining the variables in `vars(varsBlocks{i})`. The variables in `vars([varsBlocks{1}, ..., varsBlocks{i-1}])` are determined recursively by the previous blocks of equations. After you solve the first block of equations for the first block of variables, the second block of equations, given by `eqs(eqsBlocks{2})`, defines a decoupled subset of equations containing only the subset of variables given by the second block of variables, `vars(varsBlock{2})`, plus the variables from the first block (these variables are known at this time). Thus, if a nontrivial block decomposition is possible, you can split the solution process for a large system of equations involving many variables into several steps, where each step involves a smaller subsystem.

The number of blocks `length(eqsBlocks)` coincides with `length(varsBlocks)`. If `length(eqsBlocks) = length(varsBlocks) = 1`, then a nontrivial block decomposition of the equations is not possible.

### Examples

#### Block Lower Triangular Decomposition of DAE System

Compute a block lower triangular decomposition (BLT decomposition) of a symbolic system of differential algebraic equations (DAEs).

Create the following system of four differential algebraic equations. Here, the symbolic function calls `x1(t)`, `x2(t)`, `x3(t)`, and `x4(t)` represent the state variables of the system. The system also contains symbolic parameters `c1`, `c2`, `c3`, `c4`, and functions `f(t,x,y)` and `g(t,x,y)`.

```
syms x1(t) x2(t) x3(t) x4(t)
syms c1 c2 c3 c4
syms f(t,x,y) g(t,x,y)

eqs = [c1*diff(x1(t),t)+c2*diff(x3(t),t)==c3*f(t,x1(t),x3(t));...
       c2*diff(x1(t),t)+c1*diff(x3(t),t)==c4*g(t,x3(t),x4(t));...
       x1(t)==g(t,x1(t),x3(t));...
       x2(t)==f(t,x3(t),x4(t))];

vars = [x1(t),x2(t),x3(t),x4(t)];
```

Use `findDecoupledBlocks` to find the block structure of the system.

```
[eqsBlocks,varsBlocks] = findDecoupledBlocks(eqs,vars)
```

```
eqsBlocks=1×3 cell array
    {[1 3]}    {[2]}    {[4]}
```

```
varsBlocks=1×3 cell array
    {[1 3]}    {[4]}    {[2]}
```

The first block contains two equations in two variables.

```
eqs(eqsBlocks{1})
```

```
ans =
    (
        c1  $\frac{\partial}{\partial t}$  x1(t) + c2  $\frac{\partial}{\partial t}$  x3(t) = c3 f(t, x1(t), x3(t))
        x1(t) = g(t, x1(t), x3(t))
    )
```

```
vars(varsBlocks{1})
```

```
ans = (x1(t) x3(t))
```

After you solve this block for the variables  $x_1(t)$ ,  $x_3(t)$ , you can solve the next block of equations. This block consists of one equation.

```
eqs(eqsBlocks{2})
```

```
ans =
    c2  $\frac{\partial}{\partial t}$  x1(t) + c1  $\frac{\partial}{\partial t}$  x3(t) = c4 g(t, x3(t), x4(t))
```

The block involves one variable.

```
vars(varsBlocks{2})
```

```
ans = x4(t)
```

After you solve the equation from block 2 for the variable  $x_4(t)$ , the remaining block of equations, `eqs(eqsBlocks{3})`, defines the remaining variable, `vars(varsBlocks{3})`.

```
eqs(eqsBlocks{3})
```

```
ans = x2(t) = f(t, x3(t), x4(t))
```

```
vars(varsBlocks{3})
```

```
ans = x2(t)
```

Find the permutations that convert the system to a block lower triangular form.

```
eqsPerm = [eqsBlocks{:}]
```

```
eqsPerm = 1×4
```

```
    1    3    2    4
```

```
varsPerm = [varsBlocks{:}]
```

```
varsPerm = 1×4
```

```
1      3      4      2
```

Convert the system to a block lower triangular system of equations.

```
eqs = eqs(eqsPerm)
```

```
eqs =
```

$$\begin{pmatrix} c_1 \frac{\partial}{\partial t} x_1(t) + c_2 \frac{\partial}{\partial t} x_3(t) = c_3 f(t, x_1(t), x_3(t)) \\ x_1(t) = g(t, x_1(t), x_3(t)) \\ c_2 \frac{\partial}{\partial t} x_1(t) + c_1 \frac{\partial}{\partial t} x_3(t) = c_4 g(t, x_3(t), x_4(t)) \\ x_2(t) = f(t, x_3(t), x_4(t)) \end{pmatrix}$$

```
vars = vars(varsPerm)
```

```
vars = (x1(t) x3(t) x4(t) x2(t))
```

Find the incidence matrix of the resulting system. The incidence matrix shows that the system of permuted equations has three diagonal blocks of size 2-by-2, 1-by-1, and 1-by-1.

```
incidenceMatrix(eqs, vars)
```

```
ans = 4×4
```

```
1      1      0      0
1      1      0      0
1      1      1      0
0      1      1      1
```

### BLT Decomposition and Solution of Linear System

Find blocks of equations in a linear algebraic system, and then solve the system by sequentially solving each block of equations starting from the first one.

Create the following system of linear algebraic equations.

```
syms x1 x2 x3 x4 x5 x6 c1 c2 c3
```

```
eqs = [c1*x1 + x3 + x5 == c1 + c2 + 1;...
       x1 + x3 + x4 + 2*x6 == 4 + c2;...
       x1 + 2*x3 + c3*x5 == 1 + 2*c2 + c3;...
       x2 + x3 + x4 + x5 == 2 + c2;...
       x1 - c2*x3 + x5 == 2 - c2^2;...
       x1 - x3 + x4 - x6 == 1 - c2];
```

```
vars = [x1,x2,x3,x4,x5,x6];
```

Use `findDecoupledBlocks` to convert the system to a lower triangular form. For this system, `findDecoupledBlocks` identifies three blocks of equations and corresponding variables.

```
[eqsBlocks,varsBlocks] = findDecoupledBlocks(eqs,vars)
```

```
eqsBlocks=1×3 cell array
    {[1 3 5]}    {[2 6]}    {[4]}
```

```
varsBlocks=1×3 cell array
    {[1 3 5]}    {[4 6]}    {[2]}
```

Identify the variables in the first block. This block consists of three equations in three variables.

```
vars(varsBlocks{1})
```

```
ans = (x1 x3 x5)
```

Solve the first block of equations for the first block of variables assigning the solutions to the corresponding variables.

```
[x1,x3,x5] = solve(eqs(eqsBlocks{1}),vars(varsBlocks{1}))
```

```
x1 = 1
```

```
x3 = c2
```

```
x5 = 1
```

Identify the variables in the second block. This block consists of two equations in two variables.

```
vars(varsBlocks{2})
```

```
ans = (x4 x6)
```

Solve this block of equations assigning the solutions to the corresponding variables.

```
[x4,x6] = solve(eqs(eqsBlocks{2}),vars(varsBlocks{2}))
```

```
x4 =
```

$$\frac{x_3}{3} - x_1 - \frac{c_2}{3} + 2$$

```
x6 =
```

$$\frac{2c_2}{3} - \frac{2x_3}{3} + 1$$

Use `subs` to evaluate the result for the already known values of variables `x1`, `x3`, and `x5`.

```
x4 = subs(x4)
```

```
x4 = 1
```

```
x6 = subs(x6)
```

```
x6 = 1
```

Identify the variables in the third block. This block consists of one equation in one variable.

```
vars(varsBlocks{3})
```

```
ans = x2
```

Solve this equation assigning the solution to `x2`.

```
x2 = solve(eqs(eqsBlocks{3}),vars(varsBlocks{3}))
```

```
x2 = c2 - x3 - x4 - x5 + 2
```

Use `subs` to evaluate the result for the already known values of all other variables of this system.

```
x2 = subs(x2)
```

```
x2 = 0
```

Alternatively, you can rewrite this example using the `for`-loop. This approach lets you use the example for larger systems of equations.

```
syms x1 x2 x3 x4 x5 x6 c1 c2 c3
```

```
eqs = [c1*x1 + x3 + x5 == c1 + c2 + 1;...
       x1 + x3 + x4 + 2*x6 == 4 + c2;...
       x1 + 2*x3 + c3*x5 == 1 + 2*c2 + c3;...
       x2 + x3 + x4 + x5 == 2 + c2;...
       x1 - c2*x3 + x5 == 2 - c2^2
       x1 - x3 + x4 - x6 == 1 - c2];
```

```
vars = [x1,x2,x3,x4,x5,x6];
```

```
[eqsBlocks,varsBlocks] = findDecoupledBlocks(eqs,vars);
```

```
vars_sol = vars;
```

```
for i = 1:numel(eqsBlocks)
    sol = solve(eqs(eqsBlocks{i}),vars(varsBlocks{i}));
    vars_sol_per_block = subs(vars(varsBlocks{i}),sol);
    for k=1:i-1
        vars_sol_per_block = subs(vars_sol_per_block,vars(varsBlocks{k}),...
                                   vars_sol(varsBlocks{k}));
    end
    vars_sol(varsBlocks{i}) = vars_sol_per_block
end
```

```
vars_sol = (1 x2 c2 x4 1 x6)
```

```
vars_sol = (1 x2 c2 1 1 1)
```

```
vars_sol = (1 0 c2 1 1 1)
```

## Input Arguments

### **eqs** — System of equations

vector of symbolic equations | vector of symbolic expressions

System of equations, specified as a vector of symbolic equations or expressions.

### **vars** — Variables

vector of symbolic variables | vector of symbolic functions | vector of symbolic function calls

Variables, specified as a vector of symbolic variables, functions, or function calls, such as `x(t)`.



Example: `[x(t),y(t)]` or `[x(t);y(t)]`

## Output Arguments

### **eqsBlocks — Indices defining blocks of equations**

cell array

Indices defining blocks of equations, returned as a cell array. Each block of indices is a row vector of double-precision integer numbers. The  $i$ th block of equations consists of the equations `eqs(eqsBlocks{i})` and involves only the variables in `vars(varsBlocks{1:i})`.

### **varsBlocks — Indices defining blocks of variables**

cell array

Indices defining blocks of variables, returned as a cell array. Each block of indices is a row vector of double-precision integer numbers. The  $i$ th block of equations consists of the equations `eqs(eqsBlocks{i})` and involves only the variables in `vars(varsBlocks{1:i})`.

## Tips

- The implemented algorithm requires that for each variable in `vars` there must be at least one matching equation in `eqs` involving this variable. The same equation cannot also be matched to another variable. If the system does not satisfy this condition, then `findDecoupledBlocks` throws an error. In particular, `findDecoupledBlocks` requires that `length(eqs) = length(vars)`.
- Applying the permutations `e = [eqsBlocks{:}]` to the vector `eqs` and `v = [varsBlocks{:}]` to the vector `vars` produces an incidence matrix `incidenceMatrix(eqs(e), vars(v))` that has a block lower triangular sparsity pattern.

## Version History

Introduced in R2014b

## See Also

`daeFunction` | `decic` | `diag` | `incidenceMatrix` | `isLowIndexDAE` | `massMatrixForm` | `odeFunction` | `reduceDAEIndex` | `reduceDAEToODE` | `reduceDifferentialOrder` | `reduceRedundancies` | `tril` | `triu`

## findSymType

Find symbolic subobjects of specific type

### Syntax

```
X = findSymType(symObj,type)
X = findSymType(symObj,funType,vars)
```

### Description

`X = findSymType(symObj,type)` returns a vector of symbolic subobjects of type `type` from the symbolic object `symObj`. The input `type` must be a case-sensitive string scalar or character vector, and it can include a logical expression.

- If `symObj` does not contain a symbolic subobject of type `type`, then `findSymType` returns an empty scalar.
- If `symObj` contains several subexpressions of type `type`, then `findSymType` returns the largest matching subexpression.

`X = findSymType(symObj,funType,vars)` returns a vector of unassigned symbolic functions that depend on the variables `vars` from the symbolic input `symObj`.

You can set the function type `funType` to `'symfunOf'` or `'symfunDependingOn'`. For example, `syms f(x); findSymType(f,'symfunOf',x)` returns `f(x)`.

### Examples

#### Symbolic Number and Constant

Find and return specific symbolic numbers and constants in a symbolic expression.

Create a symbolic expression.

```
expr = sym('1/2')*pi + vpa(pi)
```

```
expr =  
 $\frac{\pi}{2} + 3.141592653589793$ 
```

Find symbolic numbers of type `'integer'`.

```
X = findSymType(expr,'integer')
```

```
X = (1 2)
```

Find symbolic numbers of type `'integer | real'`.

```
X = findSymType(expr,'integer | real')
```

```
X =
```

$$\left(\frac{1}{2} 3.141592653589793\right)$$

Find symbolic numbers of type 'vpareal'.

```
X = findSymType(expr, 'vpareal')
```

```
X = 3.141592653589793
```

Find symbolic numbers of type 'complex'.

```
X = findSymType(expr, 'complex')
```

```
X =
```

```
Empty sym: 1-by-0
```

The `findSymType` function returns an empty scalar since the expression `expr` does not contain any complex numbers.

Now find symbolic constant of type 'constant'.

```
X = findSymType(expr, 'constant')
```

```
X =
```

$$\frac{\pi}{2} + 3.141592653589793$$

When there are several subexpressions of type 'constant', `findSymType` returns the largest matching subexpression.

## Symbolic Variable and Function

Find and return symbolic variables and functions in a symbolic equation.

Create a symbolic equation.

```
syms y(t) k
eq = diff(y) + k*y == sin(y)
```

```
eq(t) =
```

$$\frac{\partial}{\partial t} y(t) + k y(t) = \sin(y(t))$$

Find symbolic variables of type 'variable' in the equation.

```
X = findSymType(eq, 'variable')
```

```
X = (k t)
```

Find an unassigned symbolic function of type 'symfun' in the equation.

```
X = findSymType(eq, 'symfun')
```

```
X = y(t)
```

Find a symbolic math function of type 'diff' in the equation.

```
X = findSymType(eq, 'diff')
```

```
X =  
 $\frac{\partial}{\partial t} y(t)$ 
```

### Symbolic Function of Specific Variables

Find and return symbolic functions with specific variable dependencies in an expression.

Create a symbolic expression.

```
syms n f(x) g(x) y(x,t)  
expr = x + f(x^n) + g(x) + y(x,t)
```

```
expr = x + f(x^n) + g(x) + y(x,t)
```

Find unassigned symbolic functions of type 'symfun' in the expression.

```
X = findSymType(expr, 'symfun')
```

```
X = (f(x^n) g(x) y(x,t))
```

Find symbolic functions that depend on the exact sequence of variables [x t] using 'symfunOf'.

```
X = findSymType(expr, 'symfunOf', [x t])
```

```
X = y(x,t)
```

Find symbolic functions that have a dependency on the variable x using 'symfunDependingOn'.

```
X = findSymType(expr, 'symfunDependingOn', x)
```

```
X = (g(x) y(x,t))
```

## Input Arguments

### symObj — Symbolic objects

symbolic expressions | symbolic functions | symbolic variables | symbolic numbers | symbolic units

Symbolic objects, specified as symbolic expressions, symbolic functions, symbolic variables, symbolic numbers, or symbolic units.

### type — Symbolic types

scalar string | character vector

Symbolic types, specified as a case-sensitive scalar string or character vector. The input type can contain a logical expression. The value options follow.

| Symbolic Type Category        | String Values                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| numbers                       | <ul style="list-style-type: none"> <li>'integer' — integer numbers</li> <li>'rational' — rational numbers</li> <li>'vpareal' — variable-precision floating-point real numbers</li> <li>'complex' — complex numbers</li> <li>'real' — real numbers, including 'integer', 'rational', and 'vpareal'</li> <li>'number' — numbers, including 'integer', 'rational', 'vpareal', 'complex', and 'real'</li> </ul>      |
| constants                     | 'constant' — symbolic mathematical constants, including 'number'                                                                                                                                                                                                                                                                                                                                                 |
| symbolic math functions       | 'vpa', 'sin', 'exp', and so on — symbolic math functions in symbolic expressions                                                                                                                                                                                                                                                                                                                                 |
| unassigned symbolic functions | <ul style="list-style-type: none"> <li>'F', 'g', and so on — function name of an unassigned symbolic function</li> <li>'symfun' — unassigned symbolic functions</li> </ul>                                                                                                                                                                                                                                       |
| arithmetic operators          | <ul style="list-style-type: none"> <li>'plus' — addition operator + and subtraction operator -</li> <li>'times' — multiplication operator * and division operator /</li> <li>'power' — power or exponentiation operator ^ and square root operator sqrt</li> </ul>                                                                                                                                               |
| variables                     | 'variable' — symbolic variables                                                                                                                                                                                                                                                                                                                                                                                  |
| units                         | 'unit' — symbolic units                                                                                                                                                                                                                                                                                                                                                                                          |
| expressions                   | 'expression' — symbolic expressions, including all of the preceding symbolic types                                                                                                                                                                                                                                                                                                                               |
| logical expressions           | <ul style="list-style-type: none"> <li>'or' — logical OR operator  </li> <li>'and' — logical AND operator &amp;</li> <li>'not' — logical NOT operator ~</li> <li>'xor' — logical exclusive-OR operator xor</li> <li>'logicalconstant' — symbolic logical constants symtrue and symfalse</li> <li>'logicalexpression' — logical expressions, including 'or', 'and', 'not', 'xor', symtrue and symfalse</li> </ul> |
| equations and inequalities    | <ul style="list-style-type: none"> <li>'eq' — equality operator ==</li> <li>'ne' — inequality operator ~=</li> <li>'lt' — less-than operator &lt; or greater-than operator &gt;</li> <li>'le' — less-than-or-equal-to operator &lt;= or greater-than-or-equal-to operator &gt;=</li> <li>'equation' — symbolic equations and inequalities, including 'eq', 'ne', 'lt', and 'le'</li> </ul>                       |
| unsupported symbolic types    | 'unsupported' — unsupported symbolic types                                                                                                                                                                                                                                                                                                                                                                       |

If `symObj` contains several subexpressions of type `type`, then `findSymType` returns the largest matching subexpression (topmost matching node in a tree data structure).

**funType — Function type**

'symfunOf' | 'symfunDependingOn'

Function type, specified as 'symfunOf' or 'symfunDependingOn'.

- 'symfunOf' finds and returns the unassigned symbolic functions that depend on the exact sequence of variables specified by the array `vars`. For example, `syms f(x,y); findSymType(f,'symfunOf',[x y])` returns `f(x,y)`.
- 'symfunDependingOn' finds and returns the unassigned symbolic functions that have a dependency on the variables specified by the array `vars`. For example, `syms f(x,y); findSymType(f,'symfunDependingOn',x)` returns `f(x,y)`.

**vars — Input variables**

symbolic variables | symbolic array

Input variables, specified as symbolic variables or a symbolic array.

## Version History

Introduced in R2019a

**See Also**

`symFunType` | `hasSymType` | `symType` | `sym` | `syms` | `isSymType` | `mapSymType`

# findUnits

Find units in input

## Syntax

`U = findUnits(expr)`

## Description

`U = findUnits(expr)` returns a row vector of units in the symbolic expression `expr`.

## Examples

### Find Units in Expression

Find the units in an expression by using `findUnits`.

```
u = symunit;
syms x
units = findUnits(x*u.m + 2*u.N)

units = (N m)
```

### Find Units in Array of Equations or Expressions

Find the units in an array of equations or expressions by using `findUnits`. The `findUnits` function concatenates all units found in the input to return a row vector of units. `findUnits` returns only base units.

```
u = symunit;
array = [2*u.m + 3*u.K, 1*u.N == 1*u.kg/(u.m*u.s^2)];
units = findUnits(array)

units = (K N kg m s)
```

## Input Arguments

### `expr` — Input

symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a symbolic number, variable, vector, matrix, multidimensional array, function, or a symbolic expression.

## **Version History**

**Introduced in R2017a**

### **See Also**

`checkUnits` | `isUnit` | `newUnit` | `separateUnits` | `symunit` | `str2symunit` | `symunit2str` | `unitConversionFactor`

### **Topics**

“Units of Measurement Tutorial” on page 2-47

“Unit Conversions and Unit Systems” on page 2-53

“Units and Unit Systems List” on page 2-60

### **External Websites**

The International System of Units (SI)



# finverse

Functional inverse

## Syntax

```
g = finverse(f)
g = finverse(f,var)
```

## Description

`g = finverse(f)` returns the inverse of function `f`, such that  $f(g(x)) = x$ . If `f` contains more than one variable, use the next syntax to specify the independent variable.

`g = finverse(f,var)` uses the symbolic variable `var` as the independent variable, such that  $f(g(var)) = var$ .

## Examples

### Compute Functional Inverse

Compute functional inverse for this trigonometric function.

```
syms x
f(x) = 1/tan(x);
g = finverse(f)
```

```
g(x) =
atan(1/x)
```

Compute functional inverse for this exponential function by specifying the independent variable.

```
syms u v
finverse(exp(u-2*v), u)
```

```
ans =
2*v + log(u)
```

## Input Arguments

### **f** — Input

symbolic expression | symbolic function

Input, specified as a symbolic expression or function.

### **var** — Independent variable

symbolic variable

Independent variable, specified as a symbolic variable.

## Limitations

- `finverse` does not issue a warning when the functional inverse is not unique.

## Version History

Introduced before R2006a

## See Also

`compose` | `syms`

# fmesh

Plot 3-D mesh

## Syntax

```
fmesh(f)
fmesh(f,[min max])
fmesh(f,[xmin xmax ymin ymax])

fmesh(funx,funy,funz)
fmesh(funx,funy,funz,[uvmin uvmax])
fmesh(funx,funy,funz,[umin umax vmin vmax])

fmesh( ___,LineSpec)
fmesh( ___,Name,Value)
fmesh(ax, ___)
obj = fmesh( ___)
```

## Description

`fmesh(f)` creates a mesh plot of the symbolic expression  $f(x,y)$  over the default interval  $[-5\ 5]$  for  $x$  and  $y$ .

`fmesh(f,[min max])` plots  $f(x,y)$  over the interval  $[min\ max]$  for  $x$  and  $y$ .

`fmesh(f,[xmin xmax ymin ymax])` plots  $f(x,y)$  over the interval  $[xmin\ xmax]$  for  $x$  and  $[ymin\ ymax]$  for  $y$ . The `fmesh` function uses `symvar` to order the variables and assign intervals.

`fmesh(funx,funy,funz)` plots the parametric mesh  $x = x(u,v)$ ,  $y = y(u,v)$ ,  $z = z(u,v)$  over the interval  $[-5\ 5]$  for  $u$  and  $v$ .

`fmesh(funx,funy,funz,[uvmin uvmax])` plots the parametric mesh  $x = x(u,v)$ ,  $y = y(u,v)$ ,  $z = z(u,v)$  over the interval  $[uvmin\ uvmax]$  for  $u$  and  $v$ .

`fmesh(funx,funy,funz,[umin umax vmin vmax])` plots the parametric mesh  $x = x(u,v)$ ,  $y = y(u,v)$ ,  $z = z(u,v)$  over the interval  $[umin\ umax]$  for  $u$  and  $[vmin\ vmax]$  for  $v$ . The `fmesh` function uses `symvar` to order the parametric variables and assign intervals.

`fmesh( ___,LineSpec)` uses the `LineSpec` to set the line style, marker symbol, and plot color.

`fmesh( ___,Name,Value)` specifies surface properties using one or more `Name,Value` pair arguments. Use this option with any of the input argument combinations in the previous syntaxes.

`fmesh(ax, ___)` plots into the axes with the object `ax` instead of the current axes object `gca`.

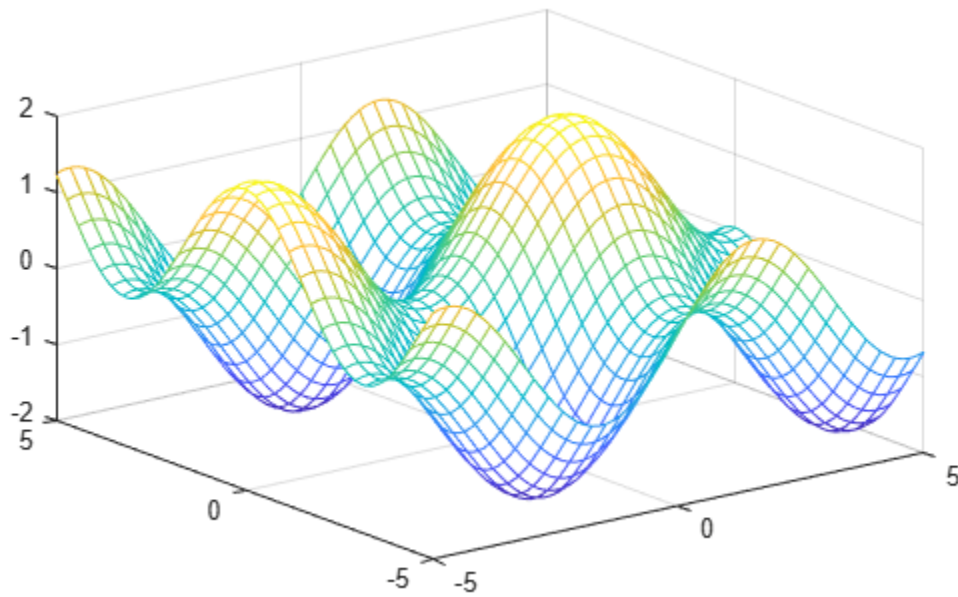
`obj = fmesh( ___)` returns a function surface object or a parameterized function surface object. Use the object to query and modify properties of a specific mesh.

## Examples

### 3-D Mesh Plot of Symbolic Expression

Plot a mesh of the input  $\sin(x) + \cos(y)$  over the default range  $-5 < x < 5$  and  $-5 < y < 5$ .

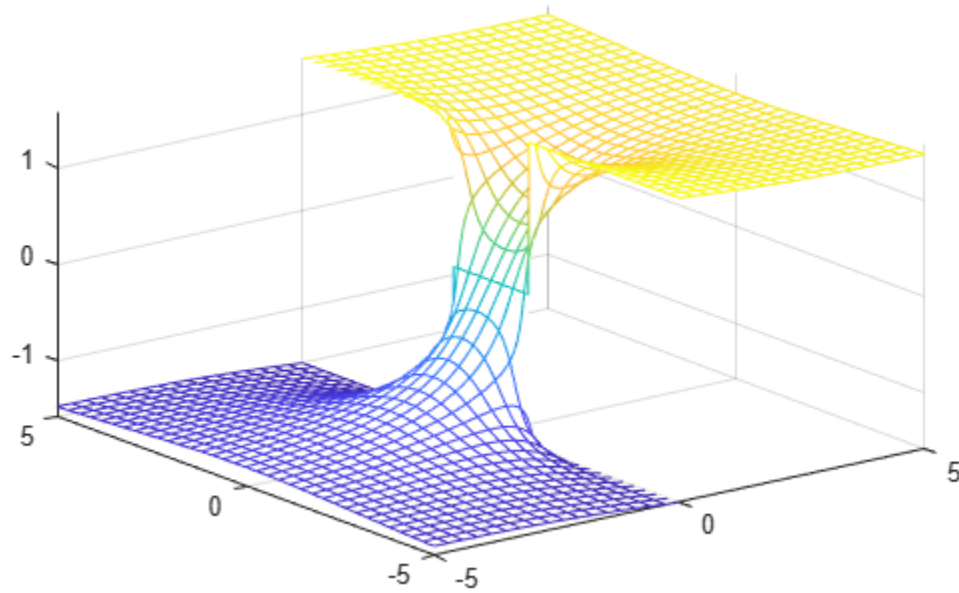
```
syms x y
fmesh(sin(x)+cos(y))
```



### 3-D Mesh Plot of Symbolic Function

Plot a 3-D mesh of the real part of  $\tan^{-1}(x + iy)$  over the default range  $-5 < x < 5$  and  $-5 < y < 5$ .

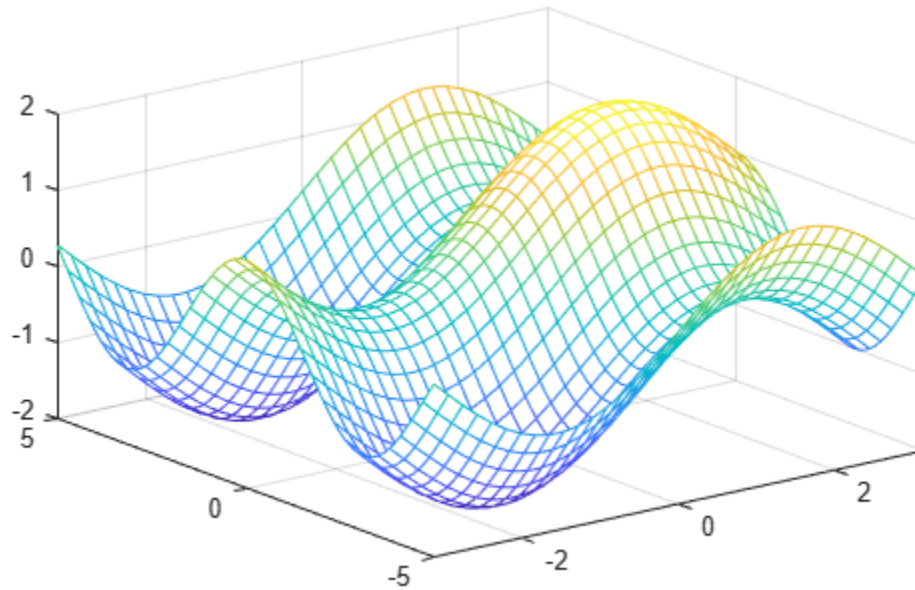
```
syms f(x,y)
f(x,y) = real(atan(x + i*y));
fmesh(f)
```



### Specify Plotting Interval of Mesh Plot

Plot  $\sin(x) + \cos(y)$  over  $-\pi < x < \pi$  and  $-5 < y < 5$  by specifying the plotting interval as the second argument of `fmesh`.

```
syms x y
f = sin(x) + cos(y);
fmesh(f, [-pi pi -5 5])
```



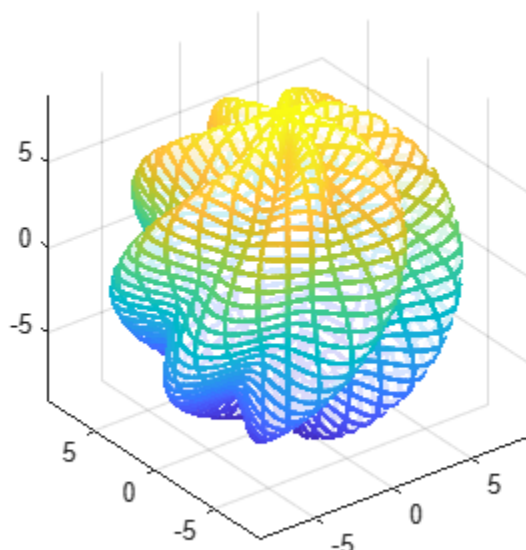
### Parameterized Mesh Plot

Plot the parameterized mesh

$$\begin{aligned}x &= r\cos(s)\sin(t) \\y &= r\sin(s)\sin(t) \\z &= r\cos(t) \\ \text{where } r &= 8 + \sin(7s + 5t)\end{aligned}$$

for  $0 < s < 2\pi$  and  $0 < t < \pi$ . Make the aspect ratio of the axes equal using `axis equal`. See the entire mesh by making the mesh partially transparent using `alpha`.

```
syms s t
r = 8 + sin(7*s + 5*t);
x = r*cos(s)*sin(t);
y = r*sin(s)*sin(t);
z = r*cos(t);
fmesh(x, y, z, [0 2*pi 0 pi], 'Linewidth', 2)
axis equal
alpha(0.8)
```



## Input Arguments

### **f** — 3-D expression or function to be plotted

symbolic expression | symbolic function

Expression or function to be plotted, specified as a symbolic expression or function.

### **[min max]** — Plotting interval for x- and y-axes

[-5 5] (default) | vector of two numbers

Plotting interval for x- and y-axes, specified as a vector of two numbers. The default is [-5 5].

### **[xmin xmax ymin ymax]** — Plotting interval for x- and y-axes

[-5 5 -5 5] (default) | vector of four numbers

Plotting interval for x- and y-axes, specified as a vector of four numbers. The default is [-5 5 -5 5].

### **funx, funy, funz** — Parametric functions of u and v

symbolic expressions | symbolic functions

Parametric functions of u and v, specified as a symbolic expression or function.

### **[uvmín uvmax]** — Plotting interval for u and v

[-5 5] (default) | vector of two numbers

Plotting interval for u and v axes, specified as a vector of two numbers. The default is [-5 5].

### **[umín umax vmín vmax]** — Plotting interval for u and v

[-5 5 -5 5] (default) | vector of four numbers

Plotting interval for u and v, specified as a vector of four numbers. The default is [-5 5 -5 5].

**ax — Axes object**

axes object

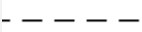
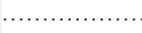
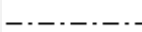
Axes object. If you do not specify an axes object, then `fmesh` uses the current axes.

**LineStyle — Line style, marker, and color**

string scalar | character vector

Line style, marker, and color, specified as a string scalar or character vector containing symbols. The symbols can appear in any order. You do not need to specify all three characteristics (line style, marker, and color). For example, if you omit the line style and specify the marker, then the plot shows only the marker and no line.

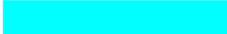
Example: `--or` is a red dashed line with circle markers.

| Line Style | Description      | Resulting Line                                                                      |
|------------|------------------|-------------------------------------------------------------------------------------|
| "_"        | Solid line       |  |
| "_ _"      | Dashed line      |  |
| ": "       | Dotted line      |  |
| "- ."      | Dash-dotted line |  |

| Marker | Description     | Resulting Marker                                                                      |
|--------|-----------------|---------------------------------------------------------------------------------------|
| "o"    | Circle          |  |
| "+"    | Plus sign       |  |
| "*"    | Asterisk        |  |
| "."    | Point           |  |
| "x"    | Cross           |  |
| "_ "   | Horizontal line |  |
| "  "   | Vertical line   |  |



| Marker      | Description                | Resulting Marker                                                                      |
|-------------|----------------------------|---------------------------------------------------------------------------------------|
| "square"    | Square                     |    |
| "diamond"   | Diamond                    |    |
| "^"         | Upward-pointing triangle   |    |
| "v"         | Downward-pointing triangle |    |
| ">"         | Right-pointing triangle    |    |
| "<"         | Left-pointing triangle     |    |
| "pentagram" | Pentagram                  |   |
| "hexagram"  | Hexagram                   |  |

| Color Name | Short Name | RGB Triplet | Appearance                                                                            |
|------------|------------|-------------|---------------------------------------------------------------------------------------|
| "red"      | "r"        | [1 0 0]     |  |
| "green"    | "g"        | [0 1 0]     |  |
| "blue"     | "b"        | [0 0 1]     |  |
| "cyan"     | "c"        | [0 1 1]     |  |
| "magenta"  | "m"        | [1 0 1]     |  |
| "yellow"   | "y"        | [1 1 0]     |  |
| "black"    | "k"        | [0 0 0]     |  |
| "white"    | "w"        | [1 1 1]     |  |

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.*

Example: `'Marker','o','MarkerFaceColor','red'`

### MeshDensity — Number of evaluation points per direction

35 (default) | number

Number of evaluation points per direction, specified as a number. The default is 35. Because `fmesh` objects use adaptive evaluation, the actual number of evaluation points is greater.

Example: 100

### ShowContours — Display contour plot under plot

'off' (default) | on/off logical value

Display contour plot under plot, specified as 'on' or 'off', or as numeric or logical 1 (true) or 0 (false). A value of 'on' is equivalent to true, and 'off' is equivalent to false. Thus, you can use the value of this property as a logical value. The value is stored as an on/off logical value of type `matlab.lang.OnOffSwitchState`.

### EdgeColor — Line color

'interp' (default) | RGB triplet | hexadecimal color code | 'r' | 'g' | 'b' | ...

Line color, specified as 'interp', an RGB triplet, a hexadecimal color code, a color name, or a short name. The default value of 'interp' colors the edges based on the `ZData` property values.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range `[0,1]`, for example, `[0.4 0.6 0.7]`.
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes `"#FF8800"`, `"#ff8800"`, `"#F80"`, and `"#f80"` are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name | RGB Triplet | Hexadecimal Color Code | Appearance                                                                            |
|------------|------------|-------------|------------------------|---------------------------------------------------------------------------------------|
| "red"      | "r"        | [1 0 0]     | "#FF0000"              |  |
| "green"    | "g"        | [0 1 0]     | "#00FF00"              |  |
| "blue"     | "b"        | [0 0 1]     | "#0000FF"              |  |
| "cyan"     | "c"        | [0 1 1]     | "#00FFFF"              |  |

| Color Name | Short Name     | RGB Triplet    | Hexadecimal Color Code | Appearance                                                                          |
|------------|----------------|----------------|------------------------|-------------------------------------------------------------------------------------|
| "magenta"  | "m"            | [1 0 1]        | "#FF00FF"              |  |
| "yellow"   | "y"            | [1 1 0]        | "#FFFF00"              |  |
| "black"    | "k"            | [0 0 0]        | "#000000"              |  |
| "white"    | "w"            | [1 1 1]        | "#FFFFFF"              |  |
| "none"     | Not applicable | Not applicable | Not applicable         | No color                                                                            |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                     |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

Example: 'blue'

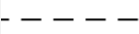
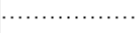
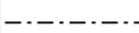
Example: [0 0 1]

Example: '#0000FF'

### LineStyle — Line style

"-" (default) | "--" | ":" | "-." | "none"

Line style, specified as one of the options listed in this table.

| Line Style | Description      | Resulting Line                                                                        |
|------------|------------------|---------------------------------------------------------------------------------------|
| " - "      | Solid line       |  |
| " - - "    | Dashed line      |  |
| " : "      | Dotted line      |  |
| " - . "    | Dash-dotted line |  |

| Line Style | Description | Resulting Line |
|------------|-------------|----------------|
| "none"     | No line     | No line        |

**LineWidth — Line width**

0.5 (default) | positive value

Line width, specified as a positive value in points, where 1 point = 1/72 of an inch. If the line has markers, then the line width also affects the marker edges.

The line width cannot be thinner than the width of a pixel. If you set the line width to a value that is less than the width of a pixel on your system, the line is displayed as one pixel wide.

**Marker — Marker symbol**

"none" (default) | "o" | "+" | "\*" | "." | ...

Marker symbol, specified as one of the values listed in this table. By default, the object does not display markers. Specifying a marker symbol adds markers at each data point or vertex.

| Marker    | Description     | Resulting Marker                                                                      |
|-----------|-----------------|---------------------------------------------------------------------------------------|
| "o"       | Circle          |    |
| "+"       | Plus sign       |  |
| "*"       | Asterisk        |  |
| "."       | Point           |  |
| "x"       | Cross           |  |
| "_"       | Horizontal line |  |
| " "       | Vertical line   |  |
| "square"  | Square          |  |
| "diamond" | Diamond         |  |

| Marker      | Description                | Resulting Marker                                                                    |
|-------------|----------------------------|-------------------------------------------------------------------------------------|
| "^"         | Upward-pointing triangle   |  |
| "v"         | Downward-pointing triangle |  |
| ">"         | Right-pointing triangle    |  |
| "<"         | Left-pointing triangle     |  |
| "pentagram" | Pentagram                  |  |
| "hexagram"  | Hexagram                   |  |
| "none"      | No markers                 | Not applicable                                                                      |

### MarkerEdgeColor — Marker outline color

'auto' (default) | RGB triplet | hexadecimal color code | 'r' | 'g' | 'b' | ...

Marker outline color, specified as 'auto', an RGB triplet, a hexadecimal color code, a color name, or a short name. The default value of 'auto' uses the same color as the EdgeColor property.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range  $[0, 1]$ , for example,  $[0.4 \ 0.6 \ 0.7]$ .
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name | RGB Triplet | Hexadecimal Color Code | Appearance                                                                            |
|------------|------------|-------------|------------------------|---------------------------------------------------------------------------------------|
| "red"      | "r"        | [1 0 0]     | "#FF0000"              |  |
| "green"    | "g"        | [0 1 0]     | "#00FF00"              |  |
| "blue"     | "b"        | [0 0 1]     | "#0000FF"              |  |
| "cyan"     | "c"        | [0 1 1]     | "#00FFFF"              |  |

| Color Name | Short Name     | RGB Triplet    | Hexadecimal Color Code | Appearance                                                                          |
|------------|----------------|----------------|------------------------|-------------------------------------------------------------------------------------|
| "magenta"  | "m"            | [1 0 1]        | "#FF00FF"              |  |
| "yellow"   | "y"            | [1 1 0]        | "#FFFF00"              |  |
| "black"    | "k"            | [0 0 0]        | "#000000"              |  |
| "white"    | "w"            | [1 1 1]        | "#FFFFFF"              |  |
| "none"     | Not applicable | Not applicable | Not applicable         | No color                                                                            |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                     |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

Example: [0.5 0.5 0.5]

Example: 'blue'

Example: '#D2F9A7'

### MarkerFaceColor — Marker fill color

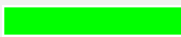
"none" (default) | "auto" | RGB triplet | hexadecimal color code | "r" | "g" | "b" | ...

Marker fill color, specified as "auto", an RGB triplet, a hexadecimal color code, a color name, or a short name. The "auto" value uses the same color as the `MarkerEdgeColor` property.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range [0, 1], for example, [0.4 0.6 0.7].
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name     | RGB Triplet    | Hexadecimal Color Code | Appearance                                                                          |
|------------|----------------|----------------|------------------------|-------------------------------------------------------------------------------------|
| "red"      | "r"            | [1 0 0]        | "#FF0000"              |  |
| "green"    | "g"            | [0 1 0]        | "#00FF00"              |  |
| "blue"     | "b"            | [0 0 1]        | "#0000FF"              |  |
| "cyan"     | "c"            | [0 1 1]        | "#00FFFF"              |  |
| "magenta"  | "m"            | [1 0 1]        | "#FF00FF"              |  |
| "yellow"   | "y"            | [1 1 0]        | "#FFFF00"              |  |
| "black"    | "k"            | [0 0 0]        | "#000000"              |  |
| "white"    | "w"            | [1 1 1]        | "#FFFFFF"              |  |
| "none"     | Not applicable | Not applicable | Not applicable         | No color                                                                            |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                       |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

Example: [0.3 0.2 0.1]

Example: "green"

Example: "#D2F9A7"

### MarkerSize — Marker size

6 (default) | positive value

Marker size, specified as a positive value in points, where 1 point = 1/72 of an inch.

## Output Arguments

### **obj** — One or more objects

scalar | vector

One or more objects, returned as a scalar or a vector. The object is either a function surface object or parameterized mesh object, depending on the type of plot. You can use these objects to query and modify properties of a specific line. For details, see [Function Surface](#) and [Parameterized Function Surface](#).

## Tips

- For additional examples, follow the `fsurf` page because `fmesh` and `fsurf` share the same syntax. All examples on the `fsurf` page apply to `fmesh`.

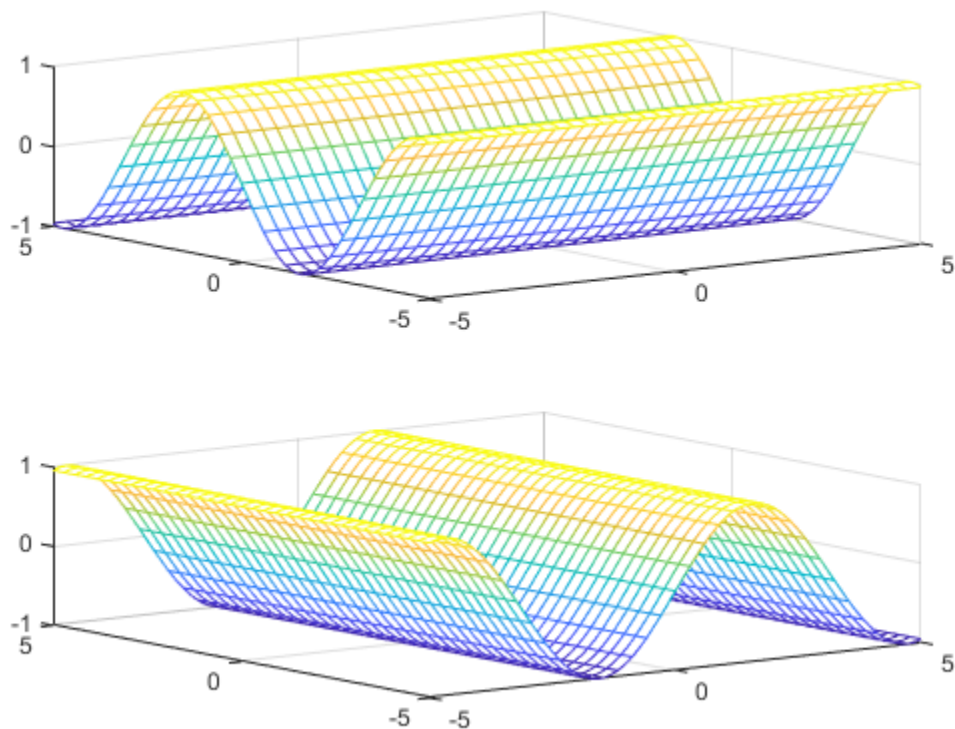
## Algorithms

`fmesh` assigns the symbolic variables in `f` to the x-axis, then the y-axis, and `symvar` determines the order of the variables to be assigned. Therefore, variable and axis names might not correspond. To force `fmesh` to assign `x` or `y` to its corresponding axis, create the symbolic function to plot, then pass the symbolic function to `fmesh`.

For example, the following code plots the mesh of  $f(x,y) = \sin(y)$  in two ways. The first way forces the waves to oscillate with respect to the y-axis. In other words, the first plot assigns the `y` variable to the corresponding y-axis. The second plot assigns `y` to the x-axis because it is the first (and only) variable in the symbolic function.

```
syms x y;  
f(x,y) = sin(y);  
  
figure;  
subplot(2,1,1)  
fmesh(f);  
subplot(2,1,2)  
fmesh(f(x,y)); % Or fmesh(sin(y));
```





## Version History

Introduced in R2016a

## See Also

### Functions

`fcontour` | `fimplicit` | `fimplicit3` | `fplot` | `fplot3` | `fsurf`

### Properties

Function Surface | Parameterized Function Surface

### Topics

“Create Plots of Symbolic Expressions” on page 4-2

## fold

Combine (fold) vector elements using function

### Syntax

```
y = fold(fun,x)
y = fold(fun,x,initVal)
```

### Description

`y = fold(fun,x)` folds the elements of `x` by using `fun`. That is, `fold` applies the function `fun` on the first two elements of `x`, and then recursively applies `fun` on the result and the next element until the last element is combined. Programmatically, this syntax is equivalent to `fold(fun,x) = fun(fold(fun,x(1:end-1)),x(end))`.

`y = fold(fun,x,initVal)` folds the elements of `x` starting from the initial value `initVal` by using `fun`.

### Examples

#### Fold Vector Using Function

Fold a vector of symbolic variables using the `power` function. The output shows how `fold` combines elements of the vector from left to right by applying the specified function.

```
syms a b c d
x = [a b c d];
y = fold(@power,x)
```

$$y = \left((a^b)^c\right)^d$$

#### Assume Variable Belongs to Set of Values

Set an assumption that the variable `x` belongs to the set of values 1, 2, ..., 10. Applying the `or` function to the conditions `x == 1, ..., x == 10` using `fold`. Check that the assumption is set by using `assumptions`.

```
syms x
cond = fold(@or, x == 1:10);
assume(cond)
assumptions
```

```
ans = x = 1 v x = 2 v x = 3 v x = 4 v x = 5 v x = 6 v x = 7 v x = 8 v x = 9 v x = 10
```

## Specify Initial Value

You can specify an initial value for the fold operation by using the third input argument. For example, apply the `power` function recursively using the initial value of 100 and the elements [a 2 3]. The result is  $100^{(6*a)}$ , which is equal to  $((100^a)^2)^3$ .

```
syms a
x = [a 2 3];
y = fold(@power,x,100)

y = 1006 a
```

## Input Arguments

### fun — Function used to fold vector

function handle

Function used to fold the vector, specified as a function handle.

Example: `@or`

Example: `@power`

### x — Vector to fold

vector | symbolic vector | cell vector

Vector to fold, specified as a vector, symbolic vector, or cell vector. If an element of `x` is a symbolic function, then the formula or definition of the symbolic function is used by applying `formula(x(i))`.

### initVal — Initial value of fold operation

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Initial value of the fold operation, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

Specify the default value using the third argument. The `fold` function returns this default value if the input vector to `fold` is empty.

## Output Arguments

### y — Result of fold operation

number | symbolic number | symbolic expression

Result of fold operation, returned as a number, symbolic number, or symbolic expression.

## Version History

**Introduced in R2016b**

**R2026a: Specify third input argument as initial value instead of default value**

*Behavior changed in R2026a*

When you specify the third input argument of the `fold` function, specify the initial value instead of the default value for the fold operation. For example, this code returns  $100^{(6*a)}$ , which is equal to  $((100^a)^2)^3$ .

```
syms a
x = [a 2 3];
y = fold(@power,x,100)

y =
100^(6*a)
```

If you specify the second argument as an empty array, then `fold` returns the initial value as the output.

```
y = fold(@power,[],100)

y =
100
```

Before R2026a, the third input argument of `fold` specified the default value. If the second argument was not empty, then `fold` applied the specified operation, ignoring the default value. For example, the same code returned  $a^6$ , which is equal to  $(a^2)^3$ .

```
syms a
x = [a 2 3];
y = fold(@power,x,100)

y =
a^6
```

If you specified the second argument as an empty array, then `fold` returned the default value as the output.

```
y = fold(@power,[],100)

y =
100
```

## See Also

`prod` | `sum`

# formula

Return body of symbolic function or matrix function

## Syntax

```
formula(f)
```

## Description

`formula(f)` returns the body, or definition, of the symbolic function or matrix function `f`.

## Examples

### Return Body of Symbolic Function

Return the body, or definition, of a symbolic function by using `formula`.

```
syms x y
f(x,y) = x + y;
formula(f)
```

```
ans = x + y
```

If the symbolic function does not have a definition, `formula` returns the symbolic function.

```
syms g(x,y)
formula(g)
```

```
ans = g(x,y)
```

### Return Body of Symbolic Matrix Function

Return the body, or definition, of a symbolic matrix function by using `formula`.

```
syms A 2 matrix
syms f(A) 2 matrix keepargs
f(A) = A*A - 3*A + 2*eye(2);
formula(f)
```

```
ans = 2I2 - 3A + A2
```

If the symbolic matrix function does not have a definition, `formula` returns the symbolic matrix function.

```
syms g(A) [1 2] matrix keepargs
formula(g)
```

```
ans = g(A)
```

## Input Arguments

### **f — Input function**

symbolic function | symbolic matrix function

Input function, specified as a symbolic function or symbolic matrix function.

Data Types: `symfun` | `symfunmatrix`

## Version History

**Introduced in R2012a**

### **R2022a: Return body of symbolic matrix functions**

The `formula` function accepts an input argument of type `symfunmatrix`. For an example, see “Return Body of Symbolic Matrix Function” on page 6-581.

### **See Also**

`argnames` | `syms` | `symvar` | `symfun` | `symfunmatrix`

# fortran

Fortran representation of symbolic expression

## Syntax

```
fortran(f)
fortran(f,Name,Value)
```

## Description

`fortran(f)` returns Fortran code for the symbolic expression `f`.

`fortran(f,Name,Value)` uses additional options specified by one or more `Name,Value` pair arguments.

## Examples

### Generate Fortran Code from Symbolic Expression

Generate Fortran code from the symbolic expression `log(1+x)`.

```
syms x
f = log(1+x);
fortran(f)

ans =
    '          t0 = log(x+1.0D0)'
```

Generate Fortran code for the 3-by-3 Hilbert matrix.

```
H = sym(hilb(3));
fortran(H)

ans =
    '          H(1,1) = 1.0D0
          H(1,2) = 1.0D0/2.0D0
          H(1,3) = 1.0D0/3.0D0
          H(2,1) = 1.0D0/2.0D0
          H(2,2) = 1.0D0/3.0D0
          H(2,3) = 1.0D0/4.0D0
          H(3,1) = 1.0D0/3.0D0
          H(3,2) = 1.0D0/4.0D0
          H(3,3) = 1.0D0/5.0D0'
```

### Write Fortran Code to File with Comments

Write generated Fortran code to a file by specifying the `File` option. When writing to a file, `fortran` optimizes the code using intermediate variables named `t0`, `t1`, .... Include comments in the file by using the `Comments` option.

```
syms x
f = diff(tan(x));
fortran(f,'File','fortrantest')

t0 = tan(x)**2+1.0D0
```

Include the comment **Version: 1.1**. Comment lines must be shorter than 71 characters to conform with Fortran 77.

```
fortran(f,'File','fortrantest','Comments','Version: 1.1')

*Version: 1.1
t0 = tan(x)**2+1.0D0
```

## Input Arguments

### **f** — Symbolic input

symbolic expression

Symbolic input, specified as a symbolic expression.

## Name-Value Pair Arguments

Specify optional pairs of arguments as **Name1=Value1, ..., NameN=ValueN**, where **Name** is the argument name and **Value** is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `fortran(x^2,'File','fortrancode','Comments','V1.2')`

### **File** — File to write to

character vector | string

File to write to, specified as a character vector or string. When writing to a file, `fortran` optimizes the code using intermediate variables named `t0`, `t1`, ....

### **Comments** — Comments to include in file header

character vector | cell array of character vectors | string vector

Comments to include in the file header, specified as a character vector, cell array of character vectors, or string vector. Comment lines must be shorter than 71 characters to conform with Fortran 77.

## Tips

- MATLAB is left-associative while Fortran is right-associative. If ambiguity exists in an expression, the `fortran` function must follow MATLAB to create an equivalent representation. For example, `fortran` represents  $a^b^c$  in MATLAB as  $(a^{**b})^{**c}$  in Fortran.

## Version History

**Introduced before R2006a**



**See Also**

`ccode` | `latex` | `matlabFunction`

## fourier

Fourier transform of symbolic expression or function

### Syntax

```
FT = fourier(f)
FT = fourier(f,transVar)
FT = fourier(f,var,transVar)
```

### Description

`FT = fourier(f)` returns the Fourier transform on page 6-595 of `f`. By default, the function `symvar` determines the independent variable, and `w` is the transformation variable.

`FT = fourier(f,transVar)` uses the transformation variable `transVar` instead of `w`.

`FT = fourier(f,var,transVar)` uses the independent variable `var` and the transformation variable `transVar` instead of `symvar(f,1)` and `w`, respectively.

### Examples

#### Fourier Transform of Common Functions

Compute the Fourier transform of common functions. By default, the transform is in terms of `w`.

Find the Fourier transform of a rectangular pulse function.

```
syms a b t
f = rectangularPulse(a,b,t);
FT = fourier(f)

FT =
```

$$-\frac{\sin(a w) + \cos(a w) i}{w} + \frac{\sin(b w) + \cos(b w) i}{w}$$

Find the Fourier transform of a unit impulse (Dirac delta function).

```
f = dirac(t);
FT = fourier(f)
```

```
FT = 1
```

Find the Fourier transform of the absolute value function.

```
f = a*abs(t);
FT = fourier(f)
```

```
FT =
```

$$-\frac{2 a}{w^2}$$

Find the Fourier transform of the step function (Heaviside function).

```
f = heaviside(t);
FT = fourier(f)
```

$$FT = \pi \delta(w) - \frac{i}{w}$$

Find the Fourier transform of a constant.

```
f = a;
FT = fourier(f)
```

$$FT = 2\pi \delta'(w) i$$

Find the Fourier transform of the cosine function.

```
f = a*cos(b*t);
FT = fourier(f)
```

$$FT = \pi a (\delta(b+w) + \delta(b-w))$$

Find the Fourier transform of the sine function.

```
f = a*sin(b*t);
FT = fourier(f)
```

$$FT = \pi a (\delta(b+w) - \delta(b-w)) i$$

Find the Fourier transform of the sign function.

```
f = sign(t);
FT = fourier(f)
```

$$FT = -\frac{2i}{w}$$

Find the Fourier transform of the right-sided exponential function without any assumptions.

```
f = exp(-t*abs(a))*heaviside(t);
FT = fourier(f)
```

$$FT = \frac{1}{|a| + w i} - \left( \frac{\text{sign}(|a|)}{2} - \frac{1}{2} \right) \text{fourier}(e^{-t|a|}, t, w)$$

Find the Fourier transform of the right-sided exponential function, assuming that  $a > 0$ . Clear the assumption after computing the Fourier transform.

```
assume(a > 0)
FTwithAssumption = fourier(f)
```

$$FTwithAssumption = \frac{1}{a + w i}$$

```
assume(a, "clear")
```

Find the Fourier transform of the double-sided exponential function, assuming that  $a > 0$ . Clear the assumption after computing the Fourier transform.

```
assume(a > 0)
f = exp(-a*t^2);
FTwithAssumption = fourier(f)
```

```
FTwithAssumption =
```

$$\frac{\sqrt{\pi} e^{-\frac{w^2}{4a}}}{\sqrt{a}}$$

```
assume(a, "clear")
```

Find the Fourier transform of a Gaussian function, assuming that  $b$  and  $c$  are real. Simplify the result and clear the assumptions after computing the Fourier transform.

```
syms c
assume([b c], "real")
f = a*exp(-(t-b)^2/(2*c^2));
FT = fourier(f);
FTsimplified = simplify(FT)
```

```
FTsimplified =
```

$$\sqrt{2} a \sqrt{\pi} e^{-\frac{w(wc^2 + 2bi)}{2}} |c|$$

```
assume([b c], "clear")
```

Find the Fourier transform of the Bessel function of the first kind with  $\nu = 1$ . Simplify the result.

```
syms x
f = besselj(1,x);
FT = fourier(f);
FTsimplified = simplify(FT)
```

```
FTsimplified =
```

$$-\frac{2 w \text{rectangularPulse}(-1, 1, -w) i}{\sqrt{1 - w^2}}$$

### Specify Independent and Transformation Variables

Find the Fourier transform of  $\exp(-t^2 - x^2)$ . By default, `symvar` determines the independent variable, and  $w$  is the transformation variable. Here, `symvar` chooses  $x$ .

```
syms t x
f = exp(-t^2-x^2);
FT = fourier(f)
```

```
FT =
```

$$\sqrt{\pi} e^{-t^2 - \frac{w^2}{4}}$$

Specify the transformation variable as  $y$ . If you specify only one variable, that variable is the transformation variable. `symvar` still determines the independent variable.

```
syms y
FT = fourier(f,y)
```

FT =

$$\sqrt{\pi} e^{-t^2 - \frac{y^2}{4}}$$

Specify the independent and transformation variables as t and y, respectively.

```
FT = fourier(f,t,y)
```

FT =

$$\sqrt{\pi} e^{-x^2 - \frac{y^2}{4}}$$

### Plot Rectangular Pulse and Its Fourier Transform

Create a rectangular pulse with a starting point at -1/2 and an endpoint at 1/2.

```
syms t
f1 = rectangularPulse(-1/2,1/2,t);
```

Find its Fourier transform. Simplify the result. Here, the Fourier transform contains only the real part.

```
FT1 = fourier(f1);
FT1 = simplify(FT1)
```

FT1 =

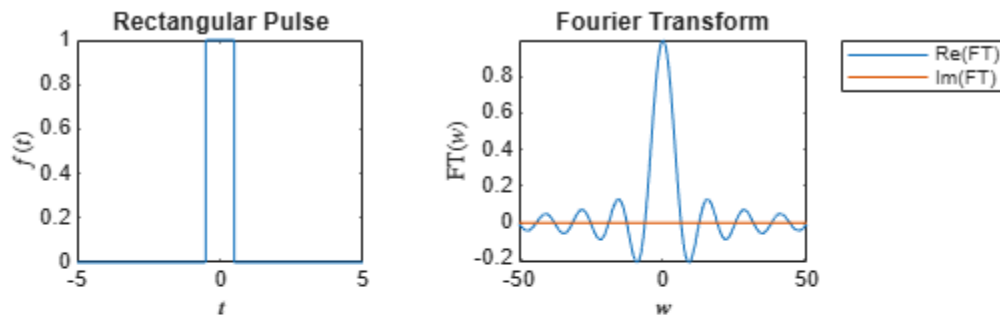
$$\frac{2 \sin\left(\frac{w}{2}\right)}{w}$$

Plot the rectangular pulse in a 2-by-2 tiled chart layout by using `fplot`. Plot the real and imaginary parts of its Fourier transform.

```
tilayout(2,2)

nexttile
fplot(f1)
xlabel("$t$",Interpreter="latex")
ylabel("$f(t)$",Interpreter="latex")
title("Rectangular Pulse")

nexttile
fplot(real(FT1),[-50 50])
hold on
fplot(imag(FT1),[-50 50])
xlabel("$w$",Interpreter="latex")
ylabel("FT($w$)",Interpreter="latex")
title("Fourier Transform")
legend("Re(FT)","Im(FT)",Location="northeastoutside")
```



Next, create another rectangular pulse with a starting point at 0 and an endpoint at 1.

```
f2 = rectangularPulse(0,1,t);
```

Find its Fourier transform. Here, the Fourier transform contains both real and imaginary parts.

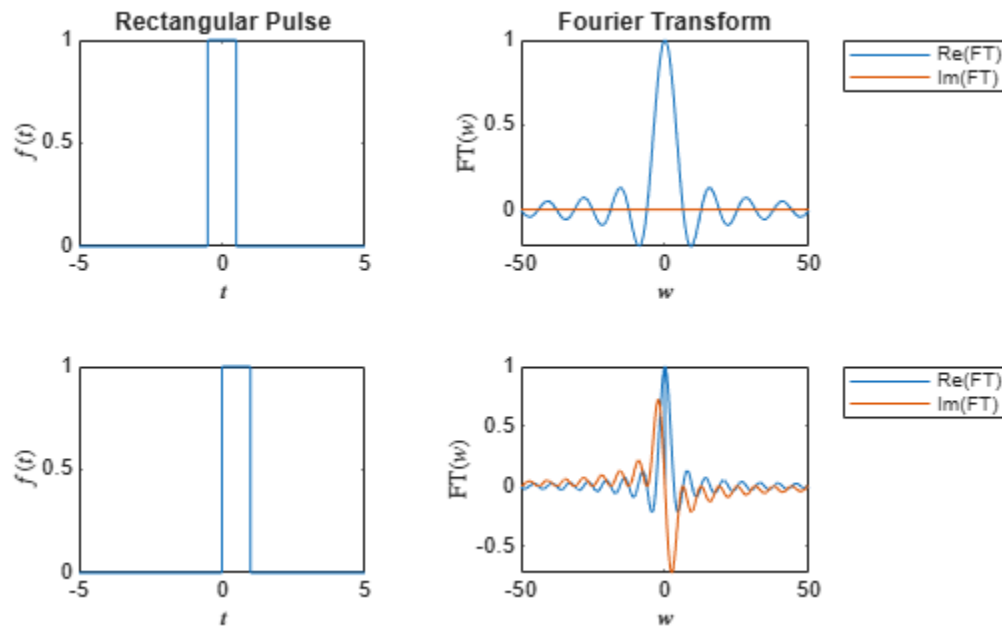
```
FT2 = fourier(f2)
```

```
FT2 =
    sin(w) + cos(w) i - i
      w          w
```

Plot the rectangular pulse. Plot the real and imaginary parts of its Fourier transform.

```
nexttile
fplot(f2)
xlabel("$t$",Interpreter="latex")
ylabel("$f(t)$",Interpreter="latex")

nexttile
fplot(real(FT2),[-50 50])
hold on
fplot(imag(FT2),[-50 50])
xlabel("$w$",Interpreter="latex")
ylabel("FT($w$)",Interpreter="latex")
legend("Re(FT)", "Im(FT)",Location="northeastoutside")
```



### Fourier Transforms Involving Dirac and Heaviside Functions

Find the Fourier transform of  $t^3$ . The result is in terms of the third derivative of the Dirac delta function.

```
syms t w
FT = fourier(t^3,t,w)

FT = -2  $\pi \delta'''(w)$  i
```

Find the Fourier transform of the Heaviside function with a discontinuity at  $t_0$ .

```
syms t0
FT = fourier(heaviside(t - t0),t,w)

FT = e^{-t_0 w i} \left( \pi \delta(w) - \frac{i}{w} \right)
```

### Specify Fourier Transform Parameters

As described in the “More About” on page 6-595 section,  $c$  and  $s$  are parameters of the Fourier transform. By default, the `fourier` function uses  $c = 1$  and  $s = -1$ . However, you can change those parameter values by using `sympref`.

Compute the Fourier transform of  $t \cdot \exp(-t^2)$  using the default values of the Fourier parameters.

```
syms t w
f = t*exp(-t^2);
FT = fourier(f,t,w)
```

FT =

$$-\frac{w\sqrt{\pi} e^{-\frac{w^2}{4}} i}{2}$$

Change the Fourier parameters to  $c = 1$  and  $s = 1$  by using `sympref`, and compute the transform again. The result changes.

```
sympref("FourierParameters",[1 1]);
FT = fourier(f,t,w)
```

FT =

$$\frac{w\sqrt{\pi} e^{-\frac{w^2}{4}} i}{2}$$

Change the Fourier parameters to  $c = 1/(2\pi)$  and  $s = 1$ , and compute the transform again. The result changes.

```
sympref("FourierParameters",[1/(2*sym(pi)) 1]);
FT = fourier(f,t,w)
```

FT =

$$\frac{w e^{-\frac{w^2}{4}} i}{4\sqrt{\pi}}$$

Symbolic settings that you set using `sympref` persist through your current and future MATLAB® sessions. Restore the default values of  $c$  and  $s$  by setting `FourierParameters` to "default".

```
sympref("FourierParameters","default");
```

## Fourier Transform Using Array Inputs

Find the Fourier transform of the matrix  $M$ . Specify the independent and transformation variables for each matrix element by using matrices of the same size. When the arguments are nonscalar, `fourier` acts on them element-wise.

```
syms a b c d w x y z
M = [exp(x) 1; sin(y) 1i*z];
vars = [w x; y z];
transVars = [a b; c d];
FT = fourier(M,vars,transVars)
```

FT =

$$\begin{pmatrix} 2\pi e^x \delta(a) & 2\pi \delta(b) \\ -\pi(\delta(c-1) - \delta(c+1)) i & -2\pi \delta'(d) \end{pmatrix}$$

If you call `fourier` with both scalar and nonscalar arguments, then it expands the scalar arguments to match the nonscalar arguments by using scalar expansion. Nonscalar arguments must be the same size.



```
FT = fourier(x,vars,transVars)
```

```
FT =

$$\begin{pmatrix} 2\pi x \delta(a) & 2\pi \delta'(b) i \\ 2\pi x \delta(c) & 2\pi x \delta(d) \end{pmatrix}$$

```

### Uncomputable Fourier Transform

If `fourier` cannot transform the input, then it returns an unevaluated call.

```
syms f(t) w
FT = fourier(f,t,w)
```

```
FT = fourier(f(t), t, w)
```

Return the original expression by using `ifourier`.

```
f = ifourier(FT,w,t)
```

```
f = f(t)
```

### Two-Dimensional Fourier Transform

Find the 2-D Fourier transform by independently applying a sequence of 1-D Fourier transforms to each of the independent variables.

In this example, set the Fourier parameters to  $c = 1$  and  $s = 2\pi$  such that the formula for the 2-D Fourier transform is

$$F(u, v) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) e^{i(2\pi ux + 2\pi vy)} dx dy.$$

You can specify these parameters by using `sympref`.

```
sympref("FourierParameters",[1 2*sym(pi)]);
```

Find the 2-D Fourier transform of a 2-D unit rectangle located at the origin with width  $a$  and height  $b$ . Perform the 1-D Fourier transforms on the variable  $x$ , followed by the variable  $y$ . Simplify the result.

```
syms x y a b real
f(x,y) = rectangularPulse(-a/2,a/2,x)*rectangularPulse(-b/2,b/2,y);
syms u v
FT = fourier(f,x,u);
FT = fourier(FT,y,v);
FT = simplify(FT)
```

```
FT =

$$\frac{\sin(\pi a u) \sin(\pi b v)}{u v \pi^2}$$

```

Find the Fourier transform of a 2-D Gaussian function with standard deviation  $\sigma$  in both the  $x$ - and  $y$ -directions.

```
syms sigma real
f(x,y) = 1/2/sym(pi)/sigma^2*exp((-x^2-y^2)/2/sigma^2);
syms u v
FT = fourier(f,x,u);
FT = fourier(FT,y,v);
FT = simplify(FT)

FT = e^{-2 \sigma^2 \pi^2 (u^2 + v^2)}
```

Find the Fourier transform of two point sources located at  $(x, y) = (0, a)$  and  $(x, y) = (0, -a)$ .

```
f(x,y) = 1/2*dirac(x)*(dirac(y-a)+dirac(y+a));
FT = fourier(f,x,u);
FT = fourier(FT,y,v);
FT = simplify(FT)

FT = cos(2 \pi a v)
```

Find the Fourier transform of a unit circle aperture. Define the unit circle by using the `heaviside` function.

```
f(x,y) = heaviside(x^2+y^2-1);
FT = fourier(f,x,u);
FT = fourier(FT,y,v)

FT = fourier(fourier(heaviside(x^2 + y^2 - 1), x, u), y, v)
```

Here, the `fourier` function cannot transform the input function and returns an unevaluated call. As an alternative, you can find the 2-D Fourier transform numerically by using the `fft2` function. For an example of how to find the 2-D Fourier transform of a circular aperture, see “2-D Fourier Transforms”.

Symbolic settings that you set using `sympref` persist through your current and future MATLAB sessions. Restore the default settings.

```
sympref("default");
```

## Input Arguments

### **f** — Input

symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic expression, function, vector, or matrix.

### **transVar** — Transformation variable

w (default) | v | symbolic variable | symbolic expression | symbolic vector | symbolic matrix

Transformation variable, specified as a symbolic variable, expression, vector, or matrix. This variable is often referred to as the “frequency variable.” By default, `fourier` uses `w`. If `w` is the independent variable of `f`, then `fourier` uses `v`.

### **var** — Independent variable

`symvar` of `f` (default) | symbolic variable | symbolic vector | symbolic matrix

Independent variable, specified as a symbolic variable, vector, or matrix. This variable is often referred to as the "time variable" or the "space variable." If you do not specify this argument, then `fourier` uses the function `symvar(f,1)` to determine the independent variable.

## More About

### Fourier Transform

The Fourier transform of the expression  $f = f(x)$  with respect to the variable  $x$  at the point  $w$  is

$$F(w) = c \int_{-\infty}^{\infty} f(x) e^{iswx} dx,$$

where  $c$  and  $s$  are parameters of the Fourier transform. The `fourier` function uses  $c = 1$  and  $s = -1$ .

## Tips

- If any argument is an array, then `fourier` acts element-wise on all elements of the array.
- If the first argument contains a symbolic function, then the second argument must be a scalar.
- To compute the inverse Fourier transform, use `ifourier`.
- `fourier` does not transform piecewise. Instead, try to rewrite piecewise by using the functions `heaviside`, `rectangularPulse`, or `triangularPulse`.

## Version History

Introduced before R2006a

## References

- [1] Oberhettinger, Fritz. *Tables of Fourier Transforms and Fourier Transforms of Distributions*. Berlin, Heidelberg: Springer Berlin Heidelberg, 1990.

## See Also

### Functions

`ifourier` | `laplace` | `ilaplace` | `ztrans` | `iztrans` | `sympref` | `symvar`

### Topics

"Fourier and Inverse Fourier Transforms" on page 3-194

### Teaching Resources

Fourier Analysis (MathWorks Teaching Resources)

## fplot

Plot symbolic expression or function

### Syntax

```
fplot(f)
fplot(f,[xmin xmax])

fplot(xt,yt)
fplot(xt,yt,[tmin tmax])

fplot( ___,LineStyle)
fplot( ___,Name=Value)
fplot(ax, ___)
fp = fplot( ___)
```

### Description

`fplot(f)` plots symbolic input `f` over the default interval `[-5 5]`.

`fplot(f,[xmin xmax])` plots `f` over the interval `[xmin xmax]`.

`fplot(xt,yt)` plots `xt = x(t)` and `yt = y(t)` over the default range of `t`, which is `[-5 5]`.

`fplot(xt,yt,[tmin tmax])` plots `xt = x(t)` and `yt = y(t)` over the specified range `[tmin tmax]`.

`fplot( ___,LineStyle)` uses `LineStyle` to set the line style, marker symbol, and line color.

`fplot( ___,Name=Value)` specifies line properties using one or more `Name=Value` arguments. Use this option with any of the input argument combinations in the previous syntaxes. `Name=Value` settings apply to all the lines plotted. To set options for individual lines, use the objects returned by `fplot`.

`fplot(ax, ___)` plots into the axes specified by `ax` instead of the current axes `gca`.

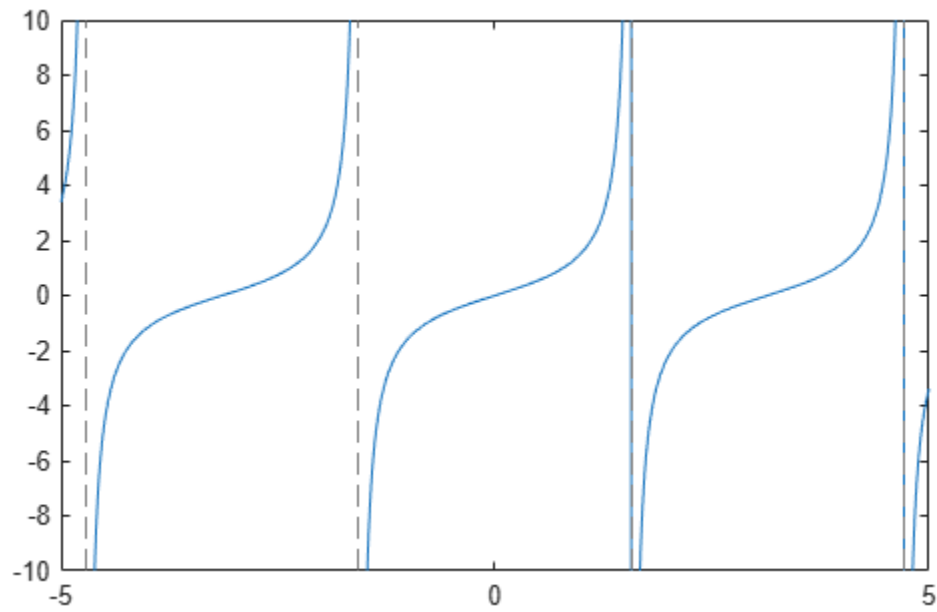
`fp = fplot( ___)` returns a function line object or parameterized line object, depending on the type of plot. Use the object to query and modify properties of a specific line. For details, see [Function Line](#) and [Parameterized Function Line](#).

### Examples

#### Plot Symbolic Expression

Plot `tan(x)` over the default range `[-5 5]`. `fplot` shows poles by default. For details, see the `ShowPoles` argument in “[Name-Value Arguments](#)” on page 6-611.

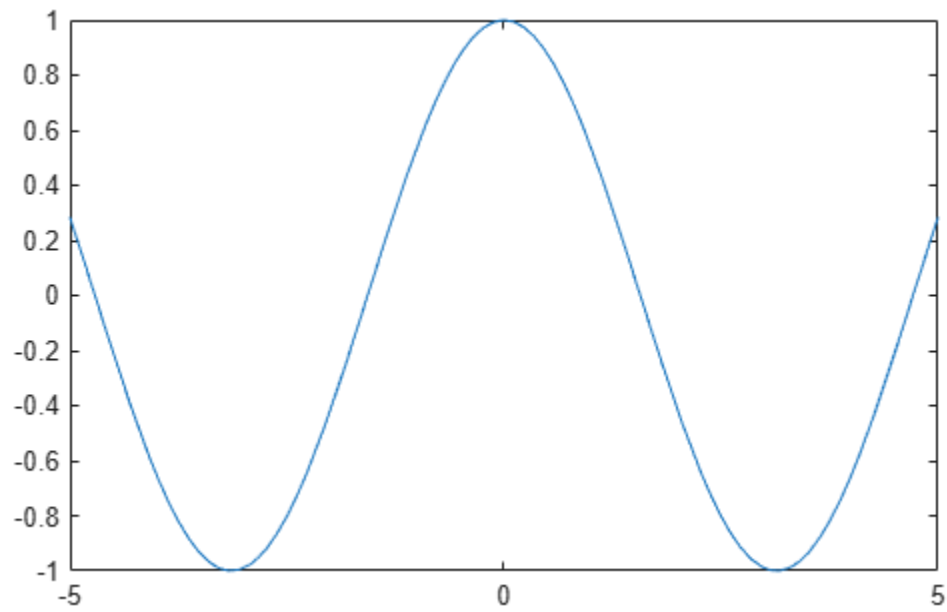
```
syms x
fplot(tan(x))
```



### Plot Symbolic Function

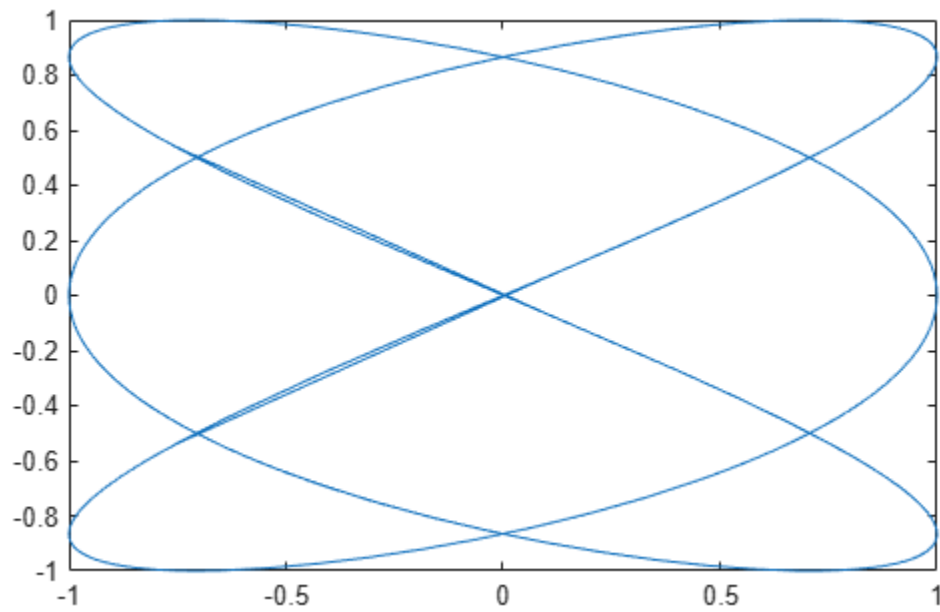
Plot the symbolic function  $f(x) = \cos(x)$  over the default range  $[-5, 5]$ .

```
syms f(x)  
f(x) = cos(x);  
fplot(f)
```

**Plot Parametric Curve**

Plot the parametric curve  $x = \cos(3*t)$  and  $y = \sin(2*t)$ .

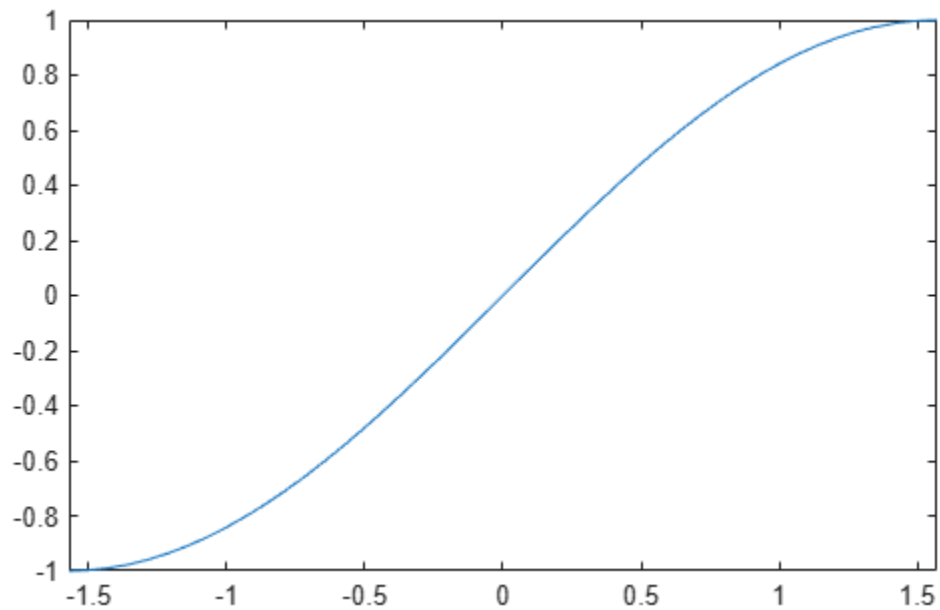
```
syms t
x = cos(3*t);
y = sin(2*t);
fplot(x,y)
```



### Specify Plotting Interval

Plot  $\sin(x)$  over  $[-\pi/2, \pi/2]$  by specifying the plotting interval as the second input to `fplot`.

```
syms x  
fplot(sin(x), [-pi/2 pi/2])
```



### Plot Multiple Lines on Same Figure

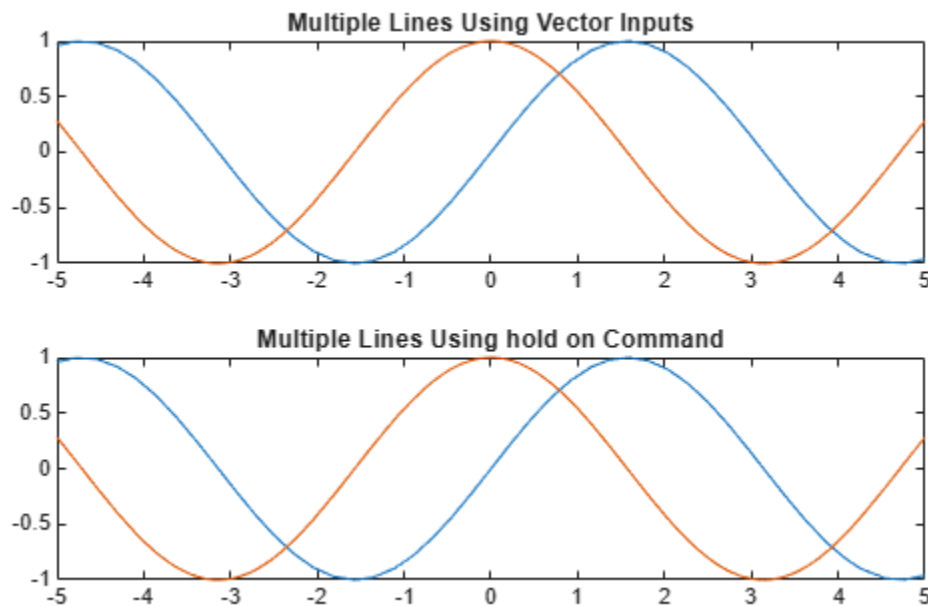
You can plot multiple lines either by passing the inputs as a vector or by using `hold on` to successively plot on the same figure. If you specify `LineStyle` and Name-Value arguments, they apply to all lines. To set options for individual plots, use the function handles returned by `fplot`.

Divide a figure into two subplots using `subplot`. On the first subplot, plot  $\sin(x)$  and  $\cos(x)$  using vector input. On the second subplot, plot  $\sin(x)$  and  $\cos(x)$  using `hold on`.

```
syms x
tiledlayout(2,1)
nexttile
fplot([sin(x) cos(x)])
title("Multiple Lines Using Vector Inputs")

nexttile
fplot(sin(x))
hold on
fplot(cos(x))
title("Multiple Lines Using hold on Command")

hold off
```



### Change Line Properties and Display Markers

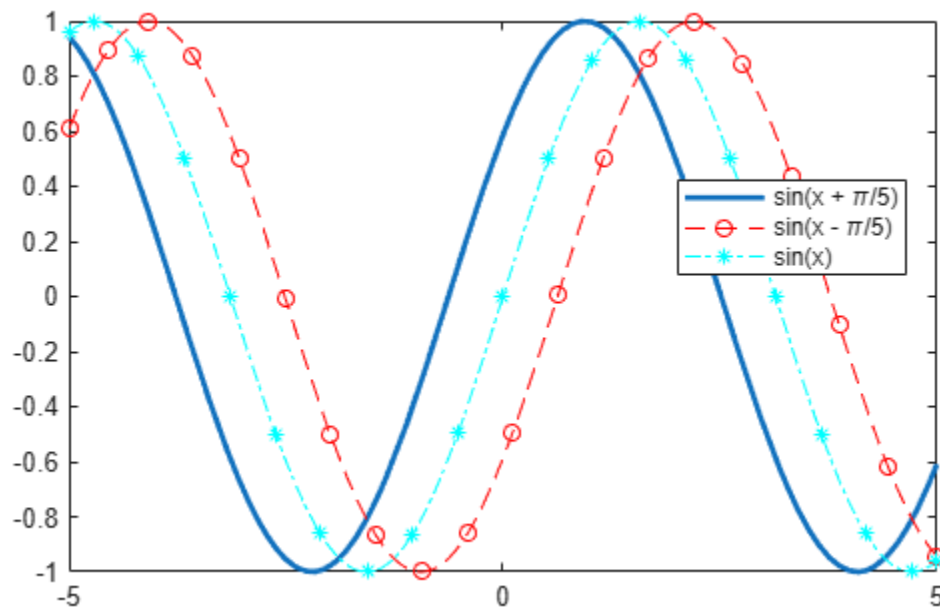
Plot three sine curves with a phase shift between each line. For the first line, use a linewidth of 2. For the second, specify a dashed red line style with circle markers. For the third, specify a cyan, dash-dot line style with asterisk markers. Display the legend.



```

syms x
fplot(sin(x+pi/5),Linewidth=2)
hold on
fplot(sin(x-pi/5),"--or")
fplot(sin(x),"-.*c")
legend(Location="best")
hold off

```



### Control Resolution of Plot

Control the resolution of a plot by using the `MeshDensity` option. Increasing `MeshDensity` can make smoother, more accurate plots, while decreasing it can increase plotting speed.

Divide a figure into two by using `subplot`. In the first subplot, plot a step function from  $x = 2.1$  to  $x = 2.15$ . The plot's resolution is too low to detect the step function. Fix this issue by increasing `MeshDensity` to 39 in the second subplot. The plot now detects the step function and shows that by increasing `MeshDensity` you increased the plot's resolution.

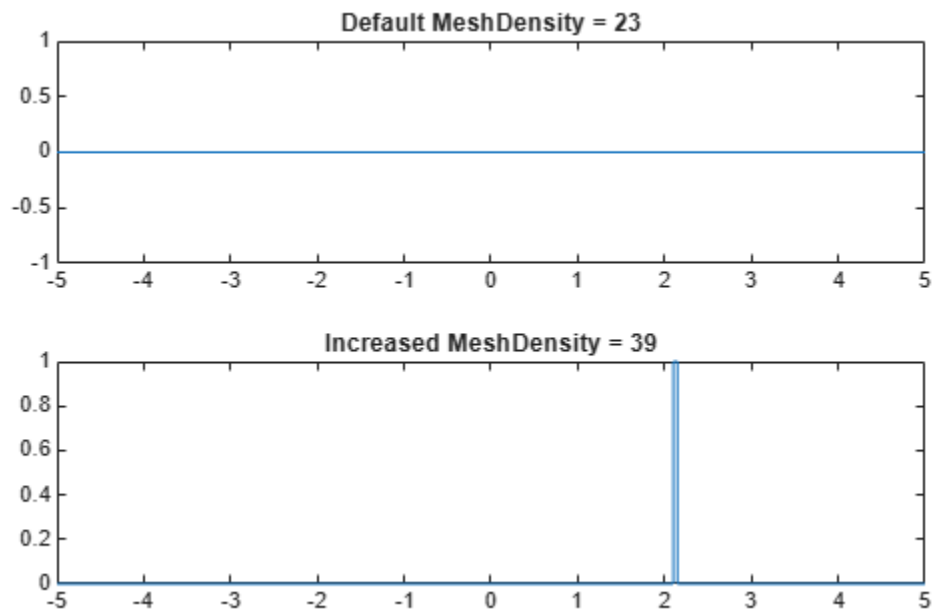
```

syms x
stepFn = rectangularPulse(2.1, 2.15, x);

subplot(2,1,1)
fplot(stepFn);
title("Default MeshDensity = 23")

subplot(2,1,2)
fplot(stepFn,MeshDensity=39);
title("Increased MeshDensity = 39")

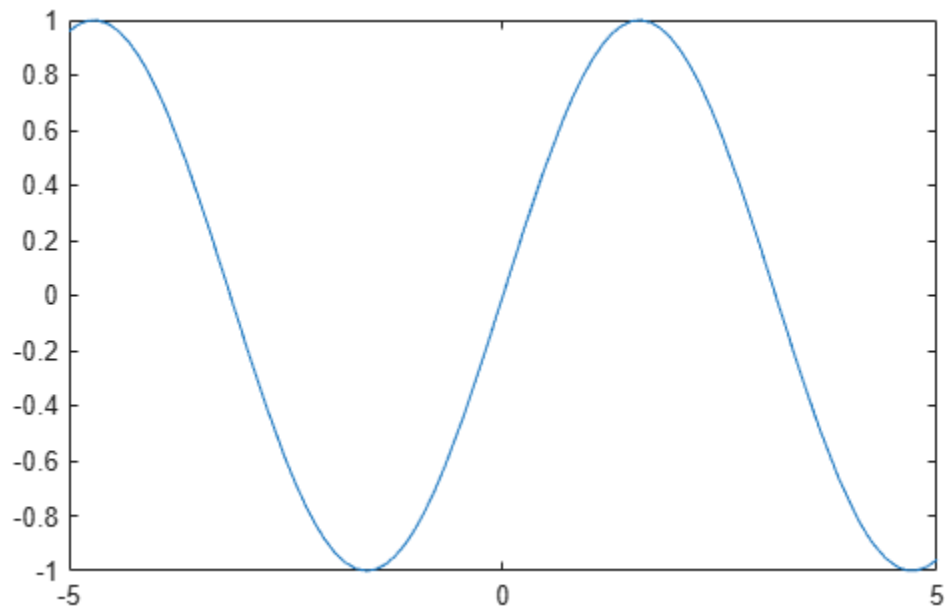
```



### Modify Plot After Creation

Plot  $\sin(x)$ . Specify an output to make `fplot` return the plot object.

```
syms x  
h = fplot(sin(x))
```



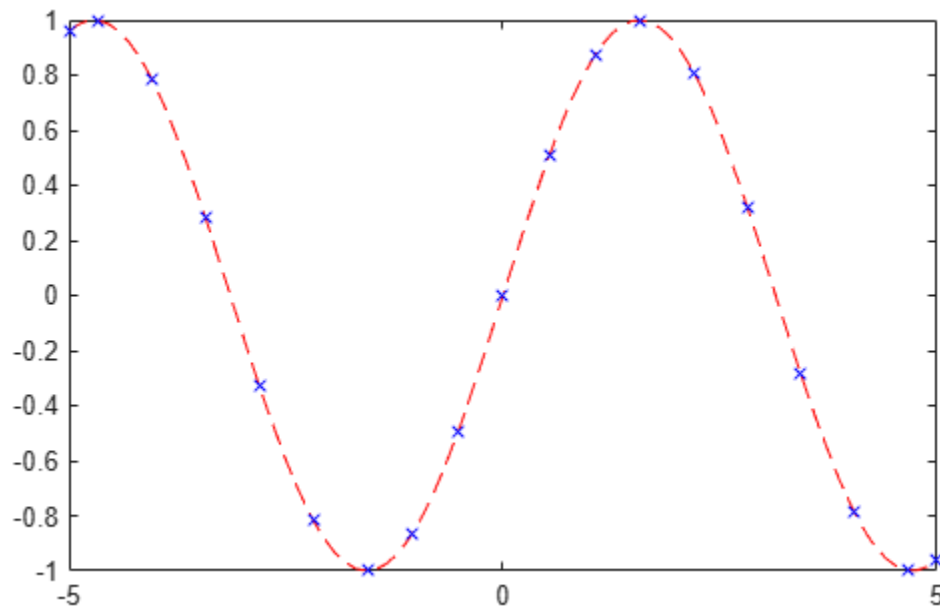
```
h =
FunctionLine with properties:

    Function: sin(x)
      Color: [0.0660 0.4430 0.7450]
LineStyle: '-'
LineWidth: 0.5000

Show all properties
```

Change the default blue line to a dashed red line by using dot notation to set properties. Similarly, add "x" marker and set the marker color to blue.

```
h.LineStyle = "--";
h.Color = "r";
h.Marker = "x";
h.MarkerEdgeColor = "b";
```



### Add Title and Axis Labels and Format Ticks

Plot  $\sin(x)$  over the interval  $[-2\pi, 2\pi]$ . Add a title and axis labels. Create the x-axis ticks by spanning the x-axis limits at intervals of  $\pi/2$ . Display these ticks by using the `XTick` property. Create x-axis labels by using `arrayfun` to apply `texlabel` to `S`. Display these labels by using the `XTickLabel` property.

To use LaTeX in plots, see `latex`.

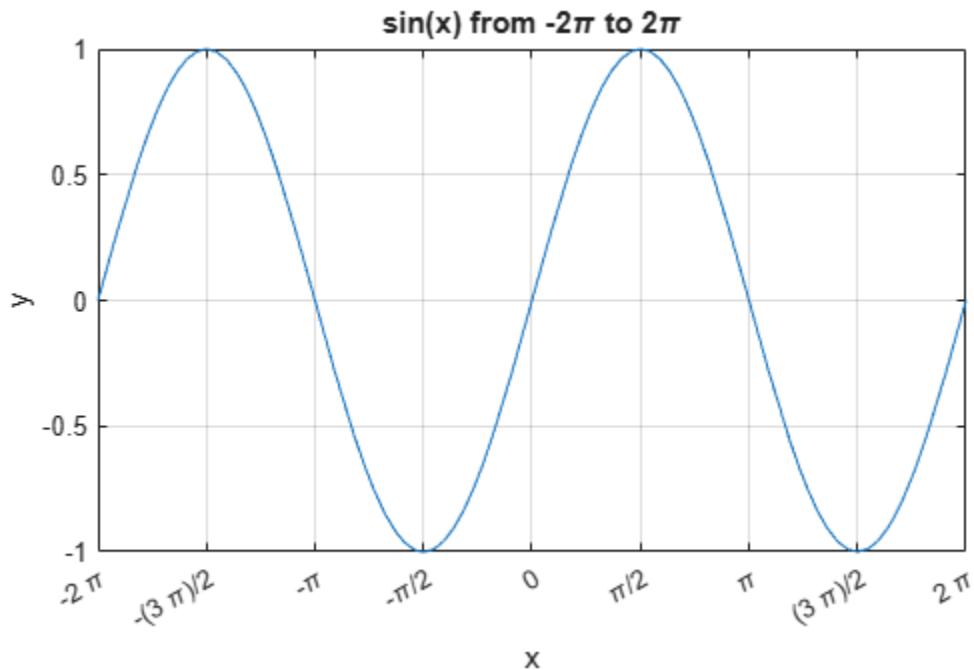
```
syms x
fplot(sin(x),[-2*pi 2*pi])
```

```

grid on
title("sin(x) from -2\pi to 2\pi")
xlabel("x")
ylabel("y")

ax = gca;
S = sym(ax.XLim(1):pi/2:ax.XLim(2));
ax.XTick = double(S);
ax.XTickLabel = arrayfun(@texlabel,S,UniformOutput=false);

```



### Zoom into Plot

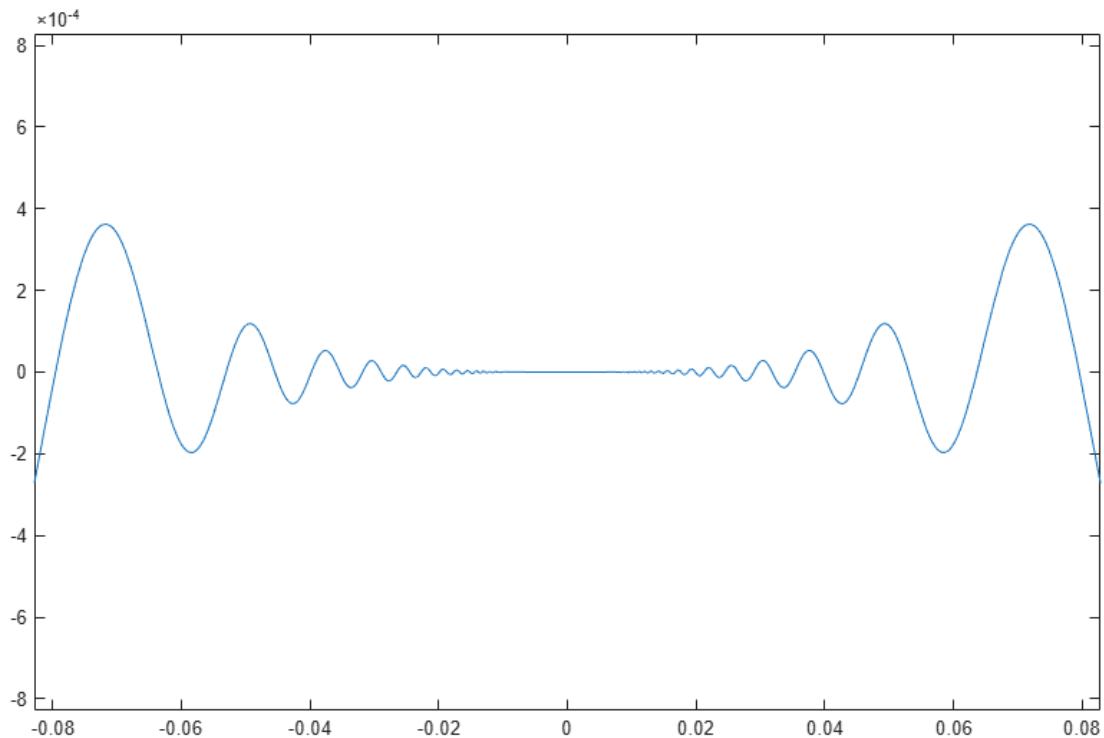
When you zoom into a plot, `fplot` reevaluates the plot automatically. The reevaluation when zooming reveals more details at smaller scales.

Plot the curve  $y = x^3 \sin(1/x)$  for  $-2 < x < 2$  and  $-0.02 < y < 0.02$ . Zoom into the plot using `zoom` and redraw the plot using `drawnow`. Because of reevaluation on zoom, `fplot` reveals more details. Repeat the zoom 6 times to view smaller-scale details.

```

syms x
fplot(x^3*sin(1/x));
axis([-2 2 -0.02 0.02]);
for i=1:6
    zoom(1.7)
    pause(0.5)
end

```



### Create Animation of Parametric Curve

Create an animation of a parametric curve where one of the parameters changes with time. The  $x$ - and  $y$ -coordinates of the parametric curve are given by

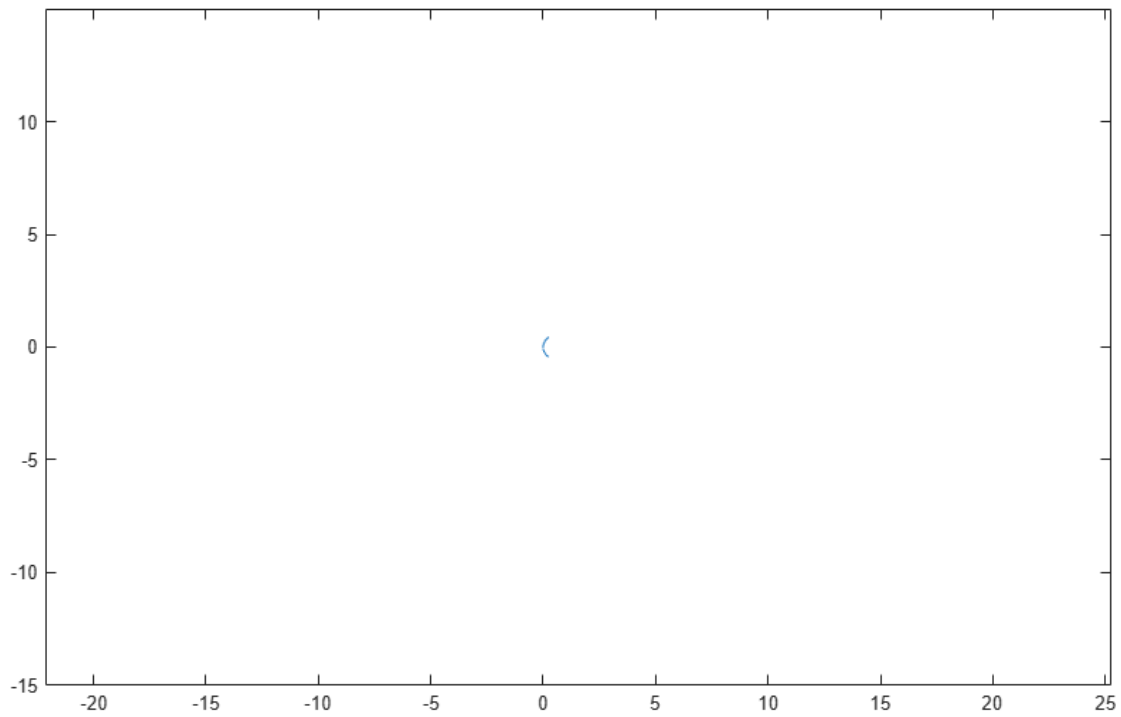
$$x = kt \sin(kt)$$

$$y = kt \cos(kt).$$

By varying the variable  $t$  from 0.1 to 3, animate the parametric curve.

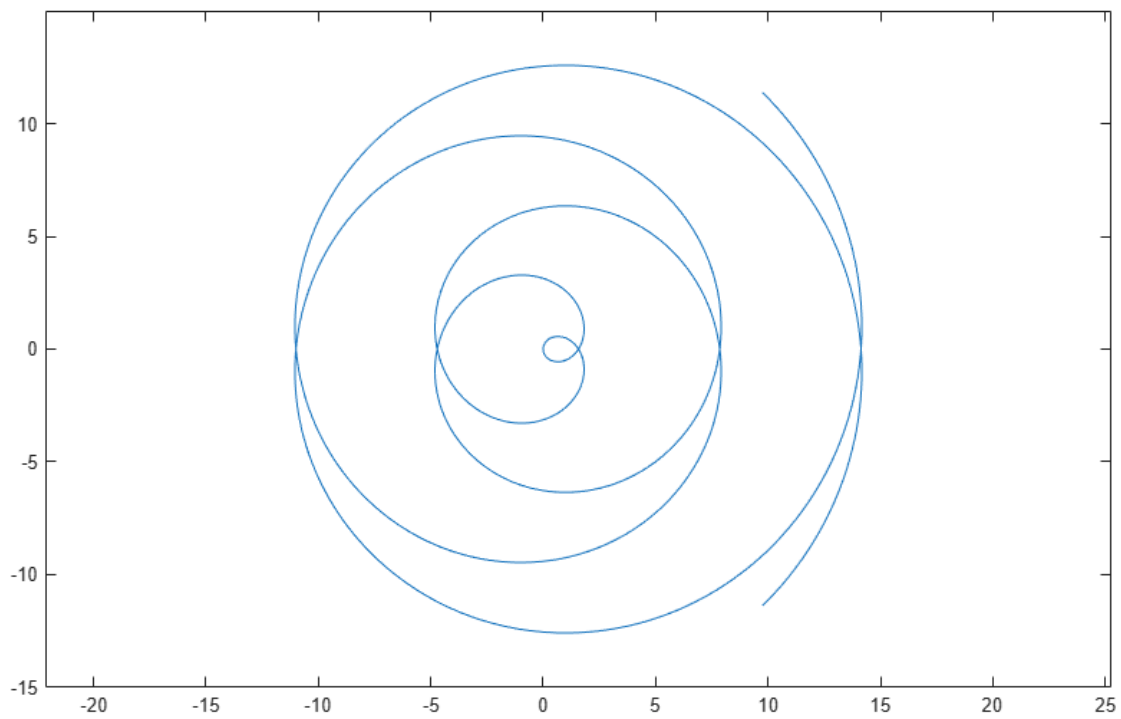
Create two symbolic variables  $k$  and  $t$ . Use the variable  $k$  to parameterize the curve within the range  $[-5 \ 5]$  and use the variable  $t$  to animate the curve as the time proceeds from 0.1 to 3. Create a stop-motion animation object of the time snapshots by using `f animator`. Set the  $x$ -axis and  $y$ -axis to be equal length.

```
syms k t
f animator(@fplot,k*t*sin(k*t),k*t*cos(k*t),[-5 5],AnimationRange = [0.1 3])
axis equal
```



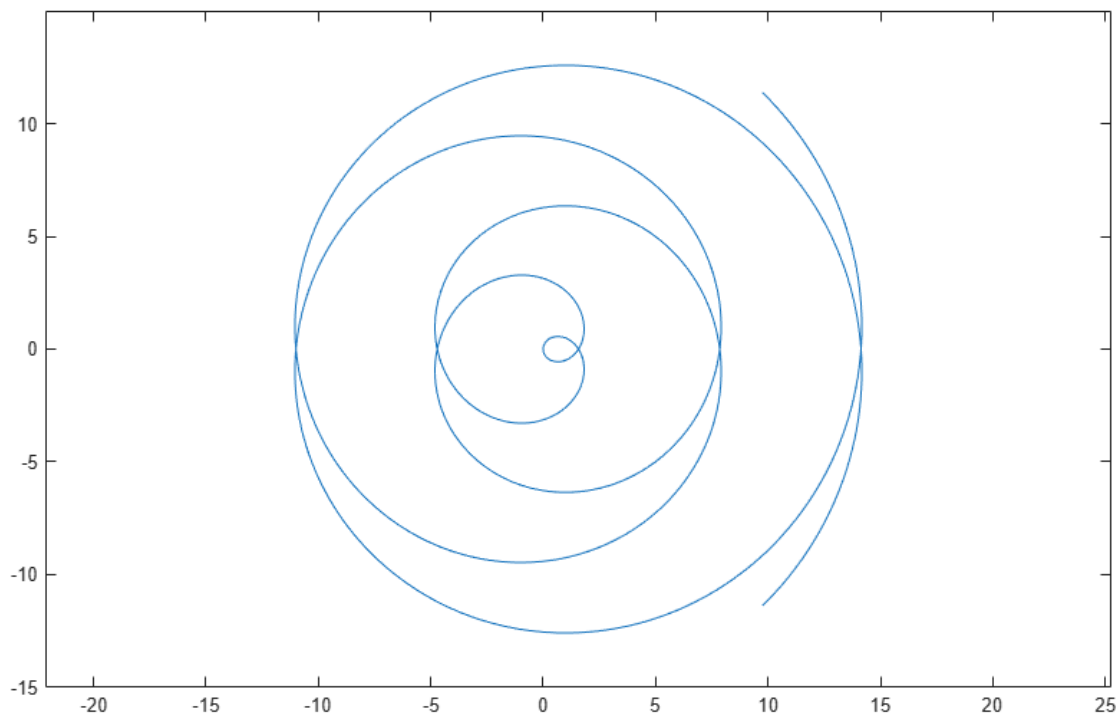
Play the animation by using `playAnimation`.

`playAnimation`



You can also save the animation as a GIF file by using `writeAnimation`.

```
writeAnimation("parametricCurve.gif")
```



## Input Arguments

### **f** — Expression or function to plot

symbolic expression | symbolic function

Expression or function to plot, specified as a symbolic expression or function.

### **[xmin xmax]** — Plotting interval for x-coordinates

[-5 5] (default) | vector of two numbers

Plotting interval for x-coordinates, specified as a vector of two numbers. The default range is [-5 5]. However, if `fplot` detects a finite number of discontinuities in `f`, then `fplot` expands the range to show them.

### **xt** — Parametric input for x-coordinates

symbolic expression | symbolic function

Parametric input for x-coordinates, specified as a symbolic expression or function. `fplot` uses `symvar` to find the parameter.

### **yt** — Parametric input for y-axis

symbolic expression | symbolic function

Parametric input for y-axis, specified as a symbolic expression or function. `fplot` uses `symvar` to find the parameter.



**[tmin tmax] — Range of values of parameter t**

[-5 5] (default) | vector of two numbers

Range of values of parameter `t`, specified as a vector of two numbers. The default range is [-5 5].

**ax — Axes object**

axes object

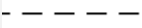
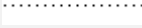
Axes object. If you do not specify an axes object, then `fplot` uses the current axes `gca`.

**LineStyle — Line style, marker, and color**

string scalar | character vector

Line style, marker, and color, specified as a string scalar or character vector containing symbols. The symbols can appear in any order. You do not need to specify all three characteristics (line style, marker, and color). For example, if you omit the line style and specify the marker, then the plot shows only the marker and no line.

Example: `"- -or"` is a red dashed line with circle markers.

| Line Style | Description      | Resulting Line                                                                        |
|------------|------------------|---------------------------------------------------------------------------------------|
| "_"        | Solid line       |    |
| "_ _"      | Dashed line      |   |
| ": "       | Dotted line      |  |
| "_ ."      | Dash-dotted line |  |

| Marker | Description     | Resulting Marker                                                                      |
|--------|-----------------|---------------------------------------------------------------------------------------|
| "o"    | Circle          |  |
| "+"    | Plus sign       |  |
| "*"    | Asterisk        |  |
| ". "   | Point           |  |
| "x"    | Cross           |  |
| "_ "   | Horizontal line |  |

| Marker      | Description                | Resulting Marker                                                                      |
|-------------|----------------------------|---------------------------------------------------------------------------------------|
| " "         | Vertical line              |    |
| "square"    | Square                     |    |
| "diamond"   | Diamond                    |    |
| "^"         | Upward-pointing triangle   |    |
| "v"         | Downward-pointing triangle |    |
| ">"         | Right-pointing triangle    |    |
| "<"         | Left-pointing triangle     |   |
| "pentagram" | Pentagram                  |  |
| "hexagram"  | Hexagram                   |  |

| Color Name | Short Name | RGB Triplet | Appearance                                                                            |
|------------|------------|-------------|---------------------------------------------------------------------------------------|
| "red"      | "r"        | [1 0 0]     |  |
| "green"    | "g"        | [0 1 0]     |  |
| "blue"     | "b"        | [0 0 1]     |  |
| "cyan"     | "c"        | [0 1 1]     |  |
| "magenta"  | "m"        | [1 0 1]     |  |
| "yellow"   | "y"        | [1 1 0]     |  |
| "black"    | "k"        | [0 0 0]     |  |
| "white"    | "w"        | [1 1 1]     |  |

## Name-Value Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.*

The function line properties listed here are only a subset. For a complete list, see [Function Line](#).

Example: `Marker="o", MarkerFaceColor="red"`

### MeshDensity — Number of evaluation points

23 (default) | number

Number of evaluation points, specified as a number. The default is 23. Because `fplot` uses adaptive evaluation, the actual number of evaluation points is greater.

### ShowPoles — Display asymptotes at poles

"on" (default) | on/off logical value

Display asymptotes at poles, specified as "on" or "off", or as numeric or logical 1 (true) or 0 (false). A value of "on" is equivalent to true, and "off" is equivalent to false. Thus, you can use the value of this property as a logical value. The value is stored as an on/off logical value of type `matlab.lang.OnOffSwitchState`.

The asymptotes display as gray, dashed vertical lines. `fplot` displays asymptotes only with the `fplot(f)` syntax or variants, and not with the `fplot(xt,yt)` syntax.

### Color — Line color

[0 0.4470 0.7410] (default) | RGB triplet | hexadecimal color code | "r" | "g" | "b" | ...

Line color, specified as an RGB triplet, a hexadecimal color code, a color name, or a short name.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range [0, 1], for example, [0.4 0.6 0.7].
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name | RGB Triplet | Hexadecimal Color Code | Appearance                                                                            |
|------------|------------|-------------|------------------------|---------------------------------------------------------------------------------------|
| "red"      | "r"        | [1 0 0]     | "#FF0000"              |  |
| "green"    | "g"        | [0 1 0]     | "#00FF00"              |  |
| "blue"     | "b"        | [0 0 1]     | "#0000FF"              |  |

| Color Name | Short Name | RGB Triplet | Hexadecimal Color Code | Appearance                                                                          |
|------------|------------|-------------|------------------------|-------------------------------------------------------------------------------------|
| "cyan"     | "c"        | [0 1 1]     | "#00FFFF"              |  |
| "magenta"  | "m"        | [1 0 1]     | "#FF00FF"              |  |
| "yellow"   | "y"        | [1 1 0]     | "#FFFF00"              |  |
| "black"    | "k"        | [0 0 0]     | "#000000"              |  |
| "white"    | "w"        | [1 1 1]     | "#FFFFFF"              |  |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                     |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

Example: "blue"

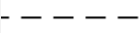
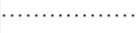
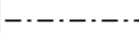
Example: [0 0 1]

Example: "#0000FF"

### LineStyle — Line style

"-" (default) | "--" | ":" | "-." | "none"

Line style, specified as one of the options listed in this table.

| Line Style | Description      | Resulting Line                                                                        |
|------------|------------------|---------------------------------------------------------------------------------------|
| "-"        | Solid line       |  |
| --"        | Dashed line      |  |
| :"         | Dotted line      |  |
| "-."       | Dash-dotted line |  |
| "none"     | No line          | No line                                                                               |

**LineWidth — Line width**

0.5 (default) | positive value

Line width, specified as a positive value in points, where 1 point = 1/72 of an inch. If the line has markers, then the line width also affects the marker edges.

The line width cannot be thinner than the width of a pixel. If you set the line width to a value that is less than the width of a pixel on your system, the line is displayed as one pixel wide.

**Marker — Marker symbol**

"none" (default) | "o" | "+" | "\*" | "." | ...

Marker symbol, specified as one of the values listed in this table. By default, the object does not display markers. Specifying a marker symbol adds markers at each data point or vertex.

| Marker    | Description              | Resulting Marker                                                                      |
|-----------|--------------------------|---------------------------------------------------------------------------------------|
| "o"       | Circle                   |    |
| "+"       | Plus sign                |    |
| "*"       | Asterisk                 |   |
| ","       | Point                    |  |
| "x"       | Cross                    |  |
| "_"       | Horizontal line          |  |
| " "       | Vertical line            |  |
| "square"  | Square                   |  |
| "diamond" | Diamond                  |  |
| "^"       | Upward-pointing triangle |  |

| Marker      | Description                | Resulting Marker                                                                    |
|-------------|----------------------------|-------------------------------------------------------------------------------------|
| "v"         | Downward-pointing triangle |  |
| ">"         | Right-pointing triangle    |  |
| "<"         | Left-pointing triangle     |  |
| "pentagram" | Pentagram                  |  |
| "hexagram"  | Hexagram                   |  |
| "none"      | No markers                 | Not applicable                                                                      |

### MarkerEdgeColor — Marker outline color

"auto" (default) | RGB triplet | hexadecimal color code | "r" | "g" | "b" | ...

Marker outline color, specified as "auto", an RGB triplet, a hexadecimal color code, a color name, or a short name. The default value of "auto" uses the same color as the `Color` property.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range  $[0, 1]$ , for example,  $[0.4 \ 0.6 \ 0.7]$ .
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name | RGB Triplet | Hexadecimal Color Code | Appearance                                                                            |
|------------|------------|-------------|------------------------|---------------------------------------------------------------------------------------|
| "red"      | "r"        | [1 0 0]     | "#FF0000"              |  |
| "green"    | "g"        | [0 1 0]     | "#00FF00"              |  |
| "blue"     | "b"        | [0 0 1]     | "#0000FF"              |  |
| "cyan"     | "c"        | [0 1 1]     | "#00FFFF"              |  |
| "magenta"  | "m"        | [1 0 1]     | "#FF00FF"              |  |
| "yellow"   | "y"        | [1 1 0]     | "#FFFF00"              |  |

| Color Name | Short Name     | RGB Triplet    | Hexadecimal Color Code | Appearance                                                                          |
|------------|----------------|----------------|------------------------|-------------------------------------------------------------------------------------|
| "black"    | "k"            | [0 0 0]        | "#000000"              |  |
| "white"    | "w"            | [1 1 1]        | "#FFFFFF"              |  |
| "none"     | Not applicable | Not applicable | Not applicable         | No color                                                                            |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                     |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot,"FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X",round(RGB*255))`.

### MarkerFaceColor — Marker fill color

"none" (default) | "auto" | RGB triplet | hexadecimal color code | "r" | "g" | "b" | ...

Marker fill color, specified as "auto", an RGB triplet, a hexadecimal color code, a color name, or a short name. The "auto" value uses the same color as the `MarkerEdgeColor` property.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range  $[0, 1]$ , for example, `[0.4 0.6 0.7]`.
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes `"#FF8800"`, `"#ff8800"`, `"#F80"`, and `"#f80"` are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name | RGB Triplet | Hexadecimal Color Code | Appearance                                                                            |
|------------|------------|-------------|------------------------|---------------------------------------------------------------------------------------|
| "red"      | "r"        | [1 0 0]     | "#FF0000"              |  |

| Color Name | Short Name     | RGB Triplet    | Hexadecimal Color Code | Appearance                                                                          |
|------------|----------------|----------------|------------------------|-------------------------------------------------------------------------------------|
| "green"    | "g"            | [0 1 0]        | "#00FF00"              |  |
| "blue"     | "b"            | [0 0 1]        | "#0000FF"              |  |
| "cyan"     | "c"            | [0 1 1]        | "#00FFFF"              |  |
| "magenta"  | "m"            | [1 0 1]        | "#FF00FF"              |  |
| "yellow"   | "y"            | [1 1 0]        | "#FFFF00"              |  |
| "black"    | "k"            | [0 0 0]        | "#000000"              |  |
| "white"    | "w"            | [1 1 1]        | "#FFFFFF"              |  |
| "none"     | Not applicable | Not applicable | Not applicable         | No color                                                                            |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                       |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |    |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

Example: [0.3 0.2 0.1]

Example: "green"

Example: "#D2F9A7"

### MarkerSize — Marker size

6 (default) | positive value

Marker size, specified as a positive value in points, where 1 point = 1/72 of an inch.

## Output Arguments

**fp** — One or more function or parameterized line objects

scalar | vector



One or more function or parameterized function line objects, returned as a scalar or a vector.

- If you use the `fplot(f)` syntax or a variation of this syntax, then `fplot` returns function line objects.
- If you use the `fplot(xt, yt)` syntax or a variation of this syntax, then `fplot` returns parameterized line objects.

You can use these objects to query and modify properties of a specific line. For a list of properties, see [Function Line](#) and [Parameterized Function Line](#).

## Tips

- If `fplot` detects a finite number of discontinuities in `f`, then `fplot` expands the range to show them.
- If `fplot` is used with a function handle to a named or anonymous function (that is not a symbolic expression or function), then the MATLAB `fplot` function is called. In this case, the function handle must accept a vector input argument and return a vector output argument of the same size. Use array operators instead of matrix operators for the best performance. For example, use `.*` (times) instead of `*` (mtimes) in the code `fplot(@(x) x.*sin(2*pi*x))`.

## Version History

Introduced in R2016a

## See Also

### Functions

`fcontour` | `fimplicit` | `fimplicit3` | `fmesh` | `fplot3` | `fpolarplot` | `fsurf`

### Properties

[Function Line](#) | [Parameterized Function Line](#)

### Topics

[“Create Plots of Symbolic Expressions” on page 4-2](#)

## fplot3

Plot 3-D parametric curve

### Syntax

```
fplot3(xt,yt,zt)
fplot3(xt,yt,zt,[tmin tmax])

fplot3( ___,LineSpec)
fplot3( ___,Name,Value)
fplot3(ax, ___)
fp = fplot3( ___)

```

### Description

`fplot3(xt,yt,zt)` plots the parametric curve  $xt = x(t)$ ,  $yt = y(t)$ , and  $zt = z(t)$  over the default interval  $-5 < t < 5$ .

`fplot3(xt,yt,zt,[tmin tmax])` plots  $xt = x(t)$ ,  $yt = y(t)$ , and  $zt = z(t)$  over the interval  $tmin < t < tmax$ .

`fplot3( ___,LineSpec)` uses `LineSpec` to set the line style, marker symbol, and line color.

`fplot3( ___,Name,Value)` specifies line properties using one or more `Name,Value` pair arguments. Use this option with any of the input argument combinations in the previous syntaxes. `Name,Value` pair settings apply to all the lines plotted. To set options for individual lines, use the objects returned by `fplot3`.

`fplot3(ax, ___)` plots into the axes object `ax` instead of the current axes `gca`.

`fp = fplot3( ___)` returns a parameterized function line object. Use the object to query and modify properties of a specific parameterized line. For details, see [Parameterized Function Line](#).

### Examples

#### Plot 3-D Parametric Line

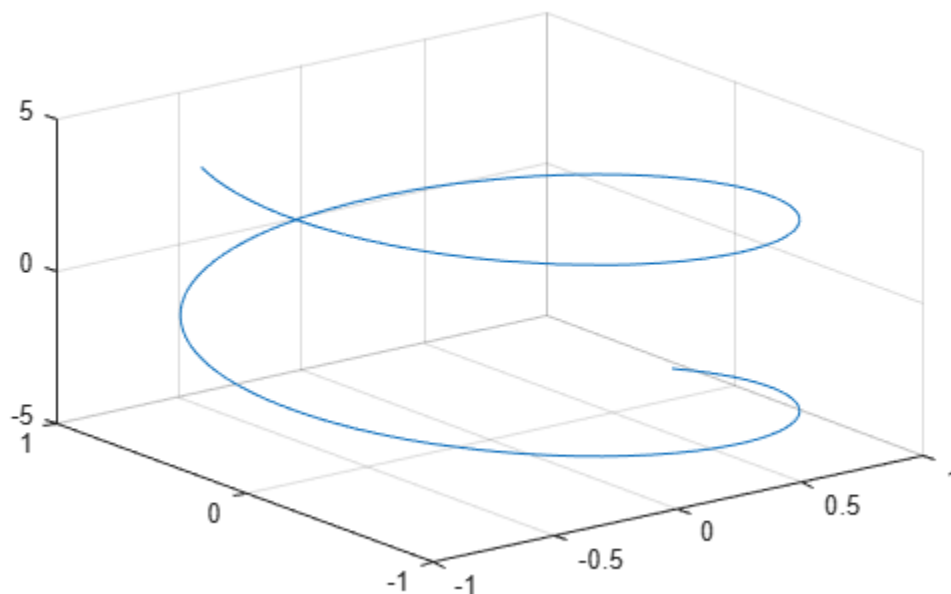
Plot the 3-D parametric line

$$\begin{aligned}x &= \sin(t) \\ y &= \cos(t) \\ z &= t\end{aligned}$$

over the default parameter range `[-5 5]`.

```
syms t
xt = sin(t);
yt = cos(t);
```

```
zt = t;
fplot3(xt,yt,zt)
```



### Specify Parameter Range

Plot the parametric line

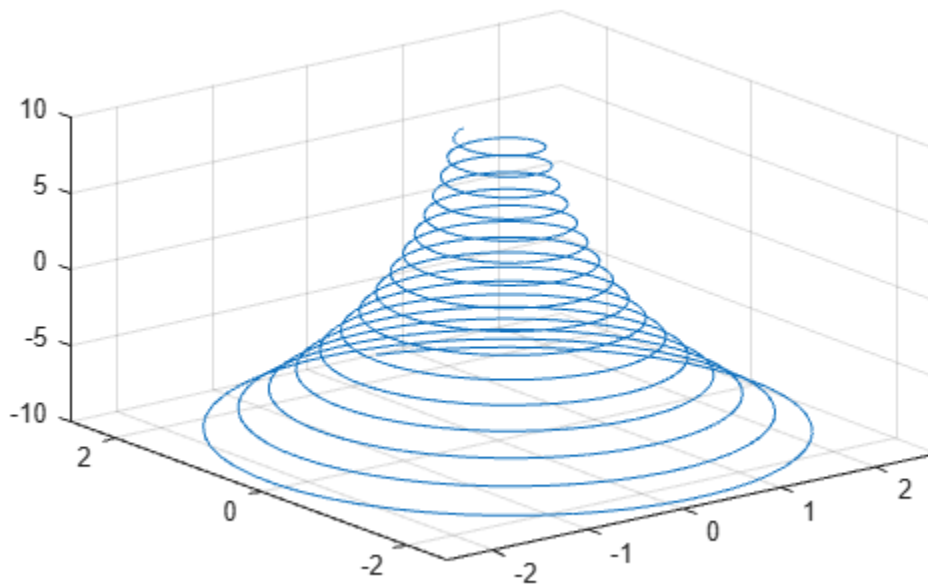
$$x = e^{-t/10} \sin(5t)$$

$$y = e^{-t/10} \cos(5t)$$

$$z = t$$

over the parameter range  $[-10 \ 10]$  by specifying the fourth argument of `fplot3`.

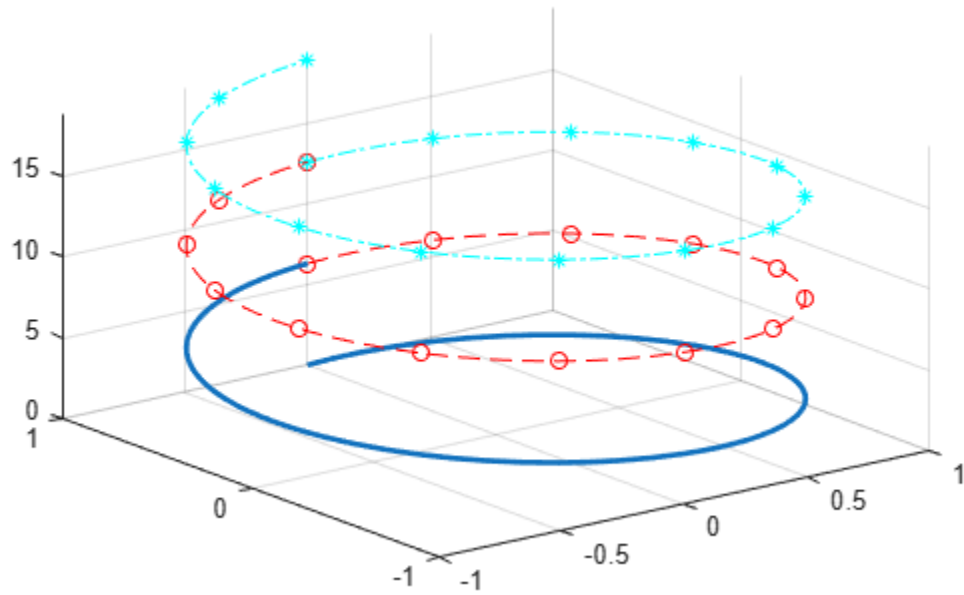
```
syms t
xt = exp(-t/10).*sin(5*t);
yt = exp(-t/10).*cos(5*t);
zt = t;
fplot3(xt,yt,zt,[-10 10])
```



### Change Line Properties and Display Markers

Plot the same 3-D parametric curve three times over different intervals of the parameter. For the first curve, use a linewidth of 2. For the second, specify a dashed red line style with circle markers. For the third, specify a cyan, dash-dot line style with asterisk markers.

```
syms t
fplot3(sin(t), cos(t), t, [0 2*pi], 'LineWidth', 2)
hold on
fplot3(sin(t), cos(t), t, [2*pi 4*pi], '--or')
fplot3(sin(t), cos(t), t, [4*pi 6*pi], '-.*c')
```

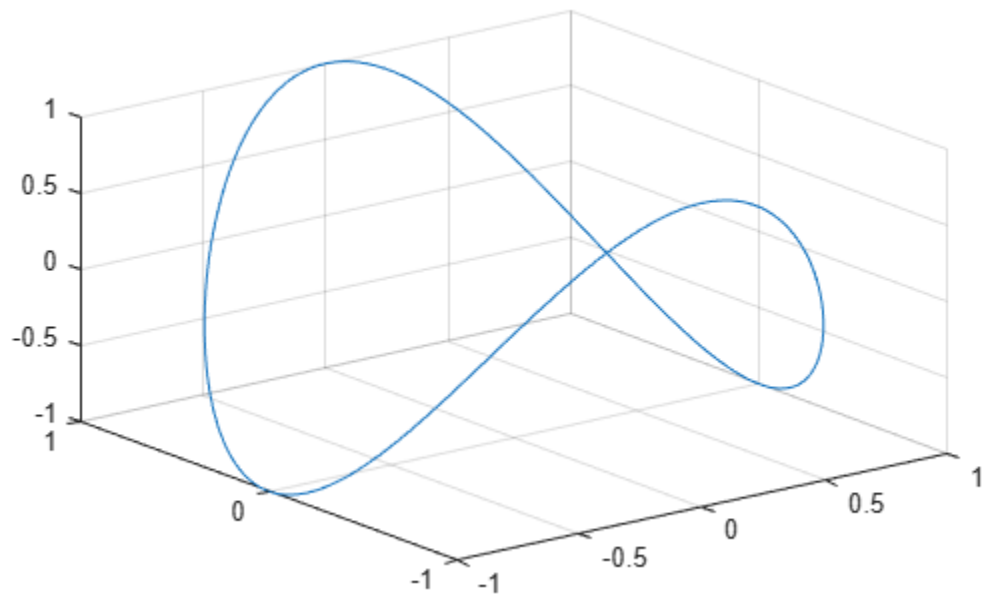


### Plot 3-D Parametric Line Using Symbolic Functions

Plot the 3-D parametric line

$$\begin{aligned}x(t) &= \sin(t) \\ y(t) &= \cos(t) \\ z(t) &= \cos(2t).\end{aligned}$$

```
syms x(t) y(t) z(t)
x(t) = sin(t);
y(t) = cos(t);
z(t) = cos(2*t);
fplot3(x,y,z)
```



### Plot Multiple Lines on Same Figure

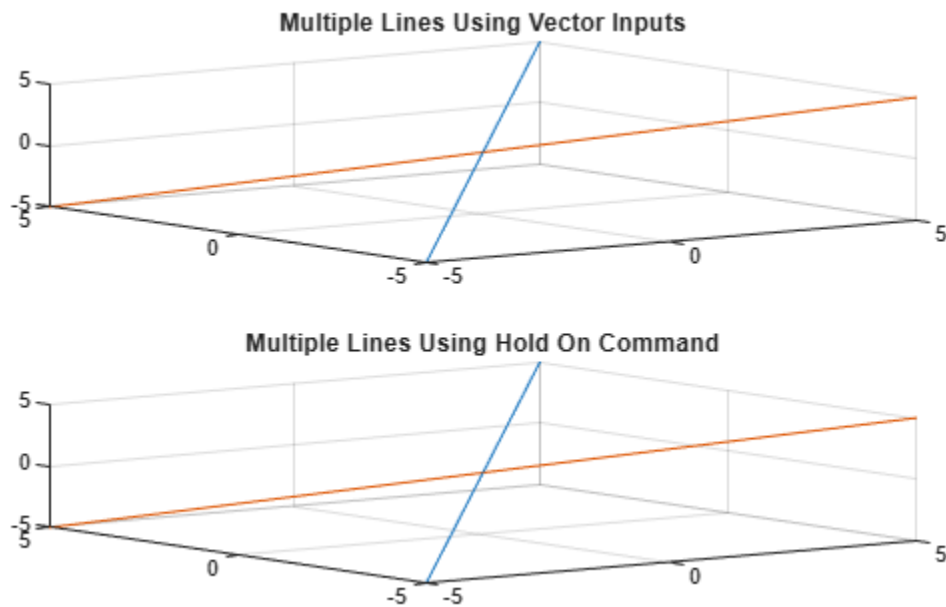
Plot multiple lines either by passing the inputs as a vector or by using `hold on` to successively plot on the same figure. If you specify `LineStyle` and Name-Value arguments, they apply to all lines. To set options for individual lines, use the function handles returned by `fplot3`.

Divide a figure into two subplots using `subplot`. On the first subplot, plot two parameterized lines using vector input. On the second subplot, plot the same lines using `hold on`.

```
syms t
subplot(2,1,1)
fplot3([t -t], t, [t -t])
title('Multiple Lines Using Vector Inputs')

subplot(2,1,2)
fplot3(t, t, t)
hold on
fplot3(-t, t, -t)
title('Multiple Lines Using Hold On Command')

hold off
```



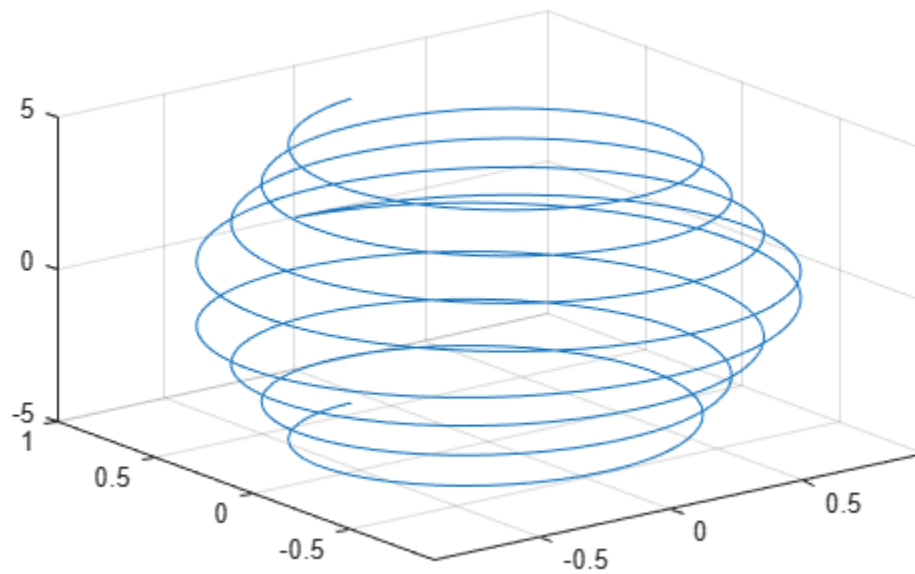
### Modify 3-D Parametric Line After Creation

Plot the parametric line

$$\begin{aligned}x &= e^{-|t|/10} \sin(5|t|) \\ y &= e^{-|t|/10} \cos(5|t|) \\ z &= t.\end{aligned}$$

Provide an output to make `fplot` return the plot object.

```
syms t
xt = exp(-abs(t)/10).*sin(5*abs(t));
yt = exp(-abs(t)/10).*cos(5*abs(t));
zt = t;
fp = fplot3(xt,yt,zt)
```



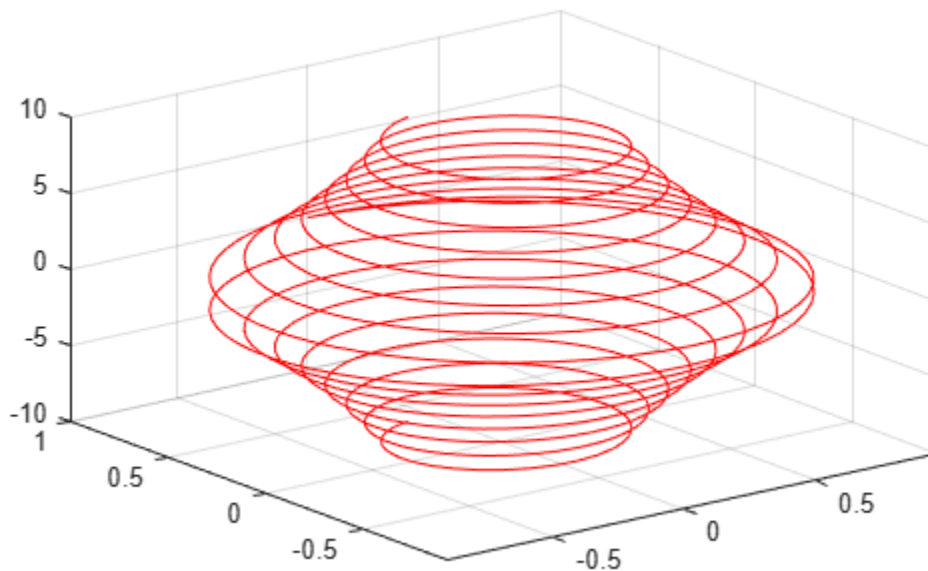
```
fp =  
ParameterizedFunctionLine with properties:  
  
XFunction: exp(-abs(t)/10)*sin(5*abs(t))  
YFunction: exp(-abs(t)/10)*cos(5*abs(t))  
ZFunction: t  
Color: [0.0660 0.4430 0.7450]  
LineStyle: '-'  
LineWidth: 0.5000
```

Show all properties

Change the range of parameter values to `[-10 10]` and the line color to red by using the `TRange` and `Color` properties of `fp` respectively.

```
fp.TRange = [-10 10];  
fp.Color = 'r';
```





### Add Title and Axis Labels and Format Ticks

For  $t$  values in the range  $-2\pi$  to  $2\pi$ , plot the parametric line

$$\begin{aligned}x &= t \\ y &= t/2 \\ z &= \sin(6t).\end{aligned}$$

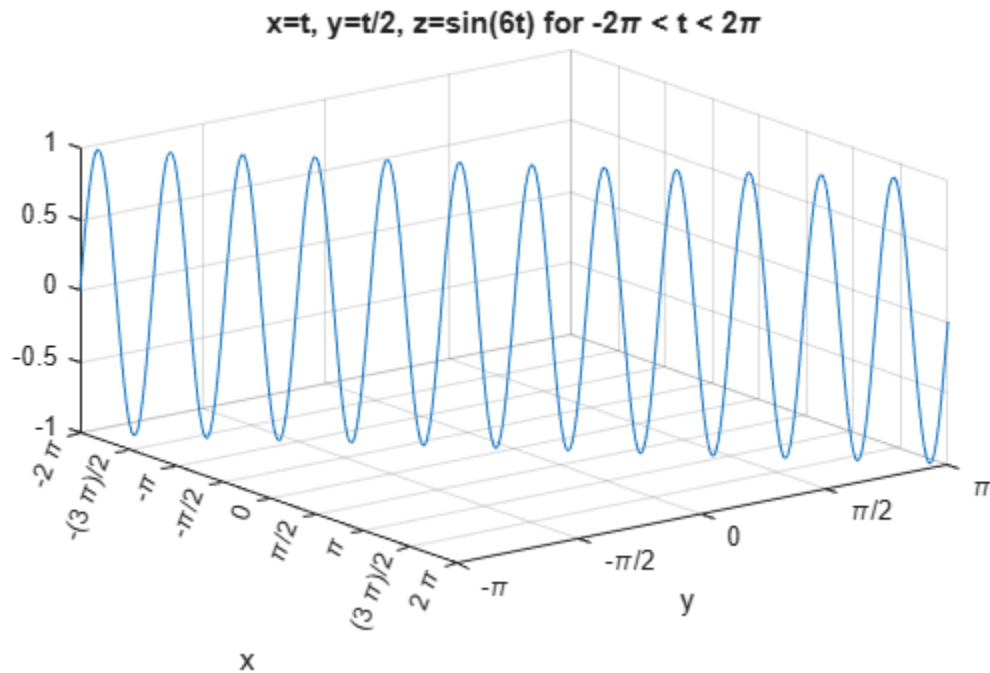
Add a title and axis labels. Create the x-axis ticks by spanning the x-axis limits at intervals of  $\pi/2$ . Display these ticks by using the `XTick` property. Create x-axis labels by using `arrayfun` to apply `texlabel` to `S`. Display these labels by using the `XTickLabel` property. Repeat these steps for the y-axis.

To use LaTeX in plots, see `latex`.

```
syms t
xt = t;
yt = t/2;
zt = sin(6*t);
fplot3(xt,yt,zt,[-2*pi 2*pi], 'MeshDensity',30)
view(52.5,30)
xlabel('x')
ylabel('y')
title('x=t, y=t/2, z=sin(6t) for -2\pi < t < 2\pi')
ax = gca;

S = sym(ax.XLim(1):pi/2:ax.XLim(2));
ax.XTick = double(S);
ax.XTickLabel = arrayfun(@texlabel, S, 'UniformOutput', false);
```

```
S = sym(ax.YLim(1):pi/2:ax.YLim(2));
ax.YTick = double(S);
ax.YTickLabel = arrayfun(@texlabel, S, 'UniformOutput', false);
```



### Create Animation of Parametric Curve

Create an animation by changing the displayed expression using the `XFunction`, `YFunction`, and `ZFunction` properties, and then use `drawnow` to update the plot. To export to GIF, see `imwrite`.

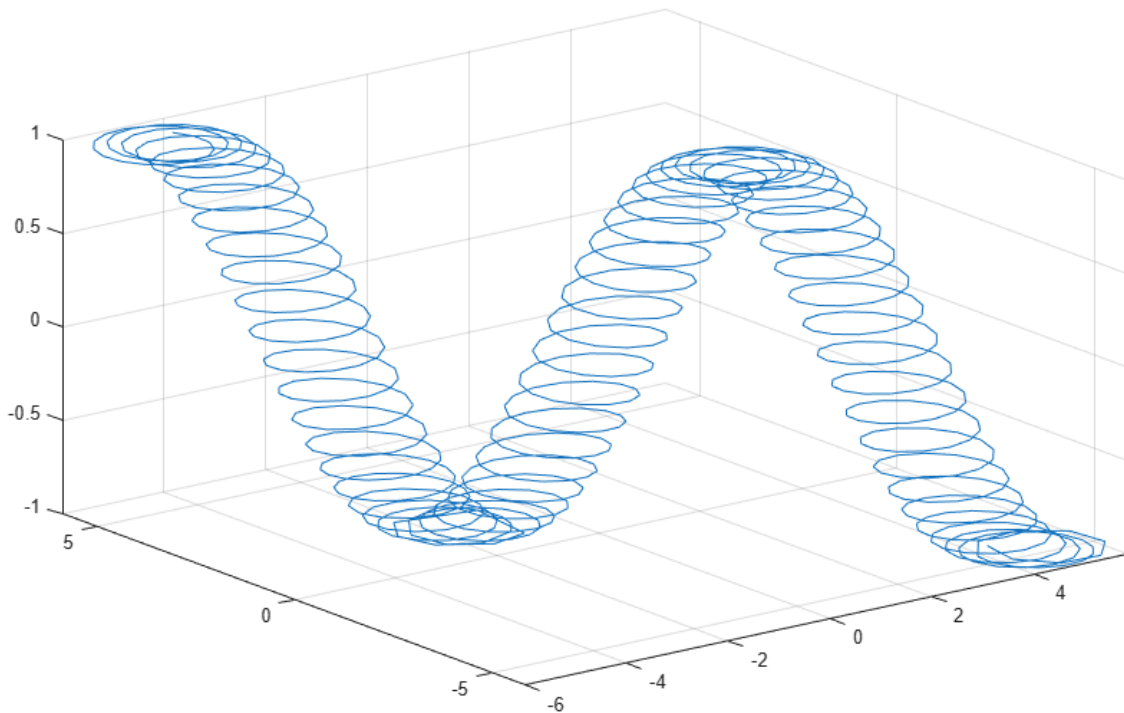
By varying the variable  $k$  from  $0$  to  $4\pi$ , animate the parametric curve

$$x = t + \sin(40t)$$

$$y = -t + \cos(40t)$$

$$z = \sin(t + k)$$

```
syms t
fp = fplot3(t+sin(40*t), -t+cos(40*t), sin(t));
for i=0:pi/10:4*pi
    fp.ZFunction = sin(t+i);
    drawnow
end
```



## Input Arguments

### **xt — Parametric input for x-axis**

symbolic expression | symbolic function

Parametric input for x-axis, specified as a symbolic expression or function. `fplot3` uses `symvar` to find the parameter.

### **yt — Parametric input for y-axis**

symbolic expression | symbolic function

Parametric input for y-axis, specified as a symbolic expression or function. `fplot3` uses `symvar` to find the parameter.

### **zt — Parametric input for z-axis**

symbolic expression | symbolic function

Parametric input for z-axis, specified as a symbolic expression or function. `fplot3` uses `symvar` to find the parameter.

### **[tmin tmax] — Range of values of parameter**

[-5 5] (default) | vector of two numbers

Range of values of parameter, specified as a vector of two numbers. The default range is [-5 5].

**ax — Axes object**

axes object

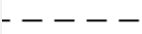
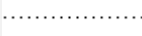
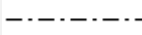
Axes object. If you do not specify an axes object, then `fplot3` uses the current axes.

**LineStyle — Line style, marker, and color**

string scalar | character vector

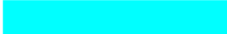
Line style, marker, and color, specified as a string scalar or character vector containing symbols. The symbols can appear in any order. You do not need to specify all three characteristics (line style, marker, and color). For example, if you omit the line style and specify the marker, then the plot shows only the marker and no line.

Example: `--or` is a red dashed line with circle markers.

| Line Style | Description      | Resulting Line                                                                      |
|------------|------------------|-------------------------------------------------------------------------------------|
| "_"        | Solid line       |  |
| "_ _"      | Dashed line      |  |
| ": "       | Dotted line      |  |
| "- ."      | Dash-dotted line |  |

| Marker | Description     | Resulting Marker                                                                      |
|--------|-----------------|---------------------------------------------------------------------------------------|
| "o"    | Circle          |  |
| "+"    | Plus sign       |  |
| "*"    | Asterisk        |  |
| "."    | Point           |  |
| "x"    | Cross           |  |
| "_ "   | Horizontal line |  |
| "  "   | Vertical line   |  |

| Marker      | Description                | Resulting Marker                                                                      |
|-------------|----------------------------|---------------------------------------------------------------------------------------|
| "square"    | Square                     |    |
| "diamond"   | Diamond                    |    |
| "^"         | Upward-pointing triangle   |    |
| "v"         | Downward-pointing triangle |    |
| ">"         | Right-pointing triangle    |    |
| "<"         | Left-pointing triangle     |    |
| "pentagram" | Pentagram                  |   |
| "hexagram"  | Hexagram                   |  |

| Color Name | Short Name | RGB Triplet | Appearance                                                                            |
|------------|------------|-------------|---------------------------------------------------------------------------------------|
| "red"      | "r"        | [1 0 0]     |  |
| "green"    | "g"        | [0 1 0]     |  |
| "blue"     | "b"        | [0 0 1]     |  |
| "cyan"     | "c"        | [0 1 1]     |  |
| "magenta"  | "m"        | [1 0 1]     |  |
| "yellow"   | "y"        | [1 1 0]     |  |
| "black"    | "k"        | [0 0 0]     |  |
| "white"    | "w"        | [1 1 1]     |  |

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `'Marker', 'o', 'MarkerFaceColor', 'red'`

The properties listed here are only a subset. For a complete list, see [Parameterized Function Line](#).

### MeshDensity — Number of evaluation points

23 (default) | number

Number of evaluation points, specified as a number. The default is 23. Because `fplot3` uses adaptive evaluation, the actual number of evaluation points is greater.

### Color — Line color

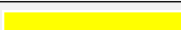
[0 0.4470 0.7410] (default) | RGB triplet | hexadecimal color code | "r" | "g" | "b" | ...

Line color, specified as an RGB triplet, a hexadecimal color code, a color name, or a short name.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range `[0, 1]`, for example, `[0.4 0.6 0.7]`.
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (`#`) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes `"#FF8800"`, `"#ff8800"`, `"#F80"`, and `"#f80"` are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name | RGB Triplet | Hexadecimal Color Code | Appearance                                                                            |
|------------|------------|-------------|------------------------|---------------------------------------------------------------------------------------|
| "red"      | "r"        | [1 0 0]     | "#FF0000"              |  |
| "green"    | "g"        | [0 1 0]     | "#00FF00"              |  |
| "blue"     | "b"        | [0 0 1]     | "#0000FF"              |  |
| "cyan"     | "c"        | [0 1 1]     | "#00FFFF"              |  |
| "magenta"  | "m"        | [1 0 1]     | "#FF00FF"              |  |
| "yellow"   | "y"        | [1 1 0]     | "#FFFF00"              |  |
| "black"    | "k"        | [0 0 0]     | "#000000"              |  |
| "white"    | "w"        | [1 1 1]     | "#FFFFFF"              |  |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                     |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

Example: "blue"

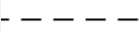
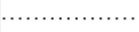
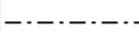
Example: [0 0 1]

Example: "#0000FF"

### LineStyle — Line style

"-" (default) | "--" | ":" | "-." | "none"

Line style, specified as one of the options listed in this table.

| Line Style | Description      | Resulting Line                                                                        |
|------------|------------------|---------------------------------------------------------------------------------------|
| " - "      | Solid line       |  |
| " - - "    | Dashed line      |  |
| " : "      | Dotted line      |  |
| " - . "    | Dash-dotted line |  |
| "none"     | No line          | No line                                                                               |

### LineWidth — Line width

0.5 (default) | positive value

Line width, specified as a positive value in points, where 1 point = 1/72 of an inch. If the line has markers, then the line width also affects the marker edges.

The line width cannot be thinner than the width of a pixel. If you set the line width to a value that is less than the width of a pixel on your system, the line is displayed as one pixel wide.

### Marker — Marker symbol

"none" (default) | "o" | "+" | "\*" | "." | ...

Marker symbol, specified as one of the values listed in this table. By default, the object does not display markers. Specifying a marker symbol adds markers at each data point or vertex.

| Marker      | Description                | Resulting Marker                                                                      |
|-------------|----------------------------|---------------------------------------------------------------------------------------|
| "o"         | Circle                     |    |
| "+"         | Plus sign                  |    |
| "*"         | Asterisk                   |    |
| ". "        | Point                      |    |
| "x"         | Cross                      |    |
| "_ "        | Horizontal line            |    |
| "  "        | Vertical line              |   |
| "square"    | Square                     |  |
| "diamond"   | Diamond                    |  |
| "^"         | Upward-pointing triangle   |  |
| "v"         | Downward-pointing triangle |  |
| ">"         | Right-pointing triangle    |  |
| "<"         | Left-pointing triangle     |  |
| "pentagram" | Pentagram                  |  |



| Marker     | Description | Resulting Marker                                                                    |
|------------|-------------|-------------------------------------------------------------------------------------|
| "hexagram" | Hexagram    |  |
| "none"     | No markers  | Not applicable                                                                      |

### MarkerEdgeColor — Marker outline color

"auto" (default) | RGB triplet | hexadecimal color code | "r" | "g" | "b" | ...

Marker outline color, specified as "auto", an RGB triplet, a hexadecimal color code, a color name, or a short name. The default value of "auto" uses the same color as the Color property.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range  $[0, 1]$ , for example,  $[0.4 \ 0.6 \ 0.7]$ .
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name     | RGB Triplet    | Hexadecimal Color Code | Appearance                                                                            |
|------------|----------------|----------------|------------------------|---------------------------------------------------------------------------------------|
| "red"      | "r"            | [1 0 0]        | "#FF0000"              |  |
| "green"    | "g"            | [0 1 0]        | "#00FF00"              |  |
| "blue"     | "b"            | [0 0 1]        | "#0000FF"              |  |
| "cyan"     | "c"            | [0 1 1]        | "#00FFFF"              |  |
| "magenta"  | "m"            | [1 0 1]        | "#FF00FF"              |  |
| "yellow"   | "y"            | [1 1 0]        | "#FFFF00"              |  |
| "black"    | "k"            | [0 0 0]        | "#000000"              |  |
| "white"    | "w"            | [1 1 1]        | "#FFFFFF"              |  |
| "none"     | Not applicable | Not applicable | Not applicable         | No color                                                                              |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                       |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |

| Palette                     | Palette Colors                                                                     |
|-----------------------------|------------------------------------------------------------------------------------|
| "glow" — Dark theme default |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

### MarkerFaceColor — Marker fill color

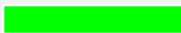
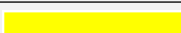
"none" (default) | "auto" | RGB triplet | hexadecimal color code | "r" | "g" | "b" | ...

Marker fill color, specified as "auto", an RGB triplet, a hexadecimal color code, a color name, or a short name. The "auto" value uses the same color as the `MarkerEdgeColor` property.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range  $[0, 1]$ , for example,  $[0.4 \ 0.6 \ 0.7]$ .
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name     | RGB Triplet    | Hexadecimal Color Code | Appearance                                                                            |
|------------|----------------|----------------|------------------------|---------------------------------------------------------------------------------------|
| "red"      | "r"            | [1 0 0]        | "#FF0000"              |  |
| "green"    | "g"            | [0 1 0]        | "#00FF00"              |  |
| "blue"     | "b"            | [0 0 1]        | "#0000FF"              |  |
| "cyan"     | "c"            | [0 1 1]        | "#00FFFF"              |  |
| "magenta"  | "m"            | [1 0 1]        | "#FF00FF"              |  |
| "yellow"   | "y"            | [1 1 0]        | "#FFFF00"              |  |
| "black"    | "k"            | [0 0 0]        | "#000000"              |  |
| "white"    | "w"            | [1 1 1]        | "#FFFFFF"              |  |
| "none"     | Not applicable | Not applicable | Not applicable         | No color                                                                              |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                     |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

Example: `[0.3 0.2 0.1]`

Example: `"green"`

Example: `"#D2F9A7"`

### MarkerSize — Marker size

6 (default) | positive value

Marker size, specified as a positive value in points, where 1 point = 1/72 of an inch.

## Output Arguments

### fp — One or more parameterized function line objects

scalar | vector

One or more parameterized line objects, returned as a scalar or a vector. You can use these objects to query and modify properties of a specific parameterized line. For details, see Parameterized Function Line.

## Version History

Introduced in R2016a

## See Also

### Functions

`fcontour` | `fimplicit` | `fimplicit3` | `fmesh` | `fplot` | `fpolarplot` | `fsurf`

### Properties

Parameterized Function Line

**Topics**

“Create Plots of Symbolic Expressions” on page 4-2

# fpolarplot

Plot symbolic expression or function in polar coordinates

## Syntax

```
fpolarplot(f)
fpolarplot(f,[a b])
fpolarplot( ___,LineSpec)
fpolarplot( ___,Name=Value)
fpolarplot(pax, ___)
fp = fpolarplot( ___)

```

## Description

`fpolarplot(f)` plots the curve  $r = f(\theta)$  in polar coordinates over the default interval  $0 \leq \theta \leq 2\pi$ .

`fpolarplot(f,[a b])` plots  $r = f(\theta)$  over the specified interval  $a \leq \theta \leq b$ .

`fpolarplot( ___,LineSpec)` specifies the line style, marker symbol, and line color. For example, `"-r"` plots a solid red line. Use this option after any of the input argument combinations in the previous syntaxes.

`fpolarplot( ___,Name=Value)` specifies line properties using one or more name-value arguments. For example, `LineWidth=2` specifies a line width of 2 points.

`fpolarplot(pax, ___)` plots into the `PolarAxes` object specified by `pax` instead of the current axes (`gca`).

`fp = fpolarplot( ___)` returns a `FunctionLine` object. Use the object to query and modify properties of the plotted line. For a list of properties, see `Function Line`.

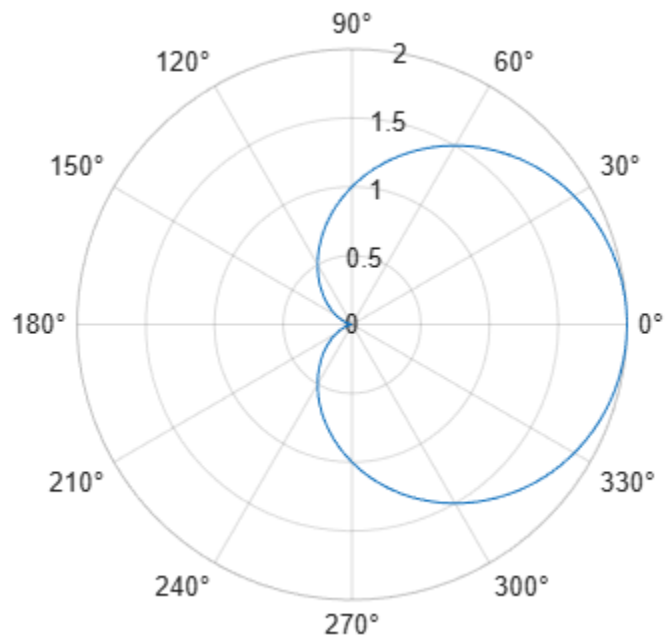
## Examples

### Polar Plot of Symbolic Expression

Create a polar plot of the expression  $1 + \cos(t)$  over the interval  $0 \leq \theta \leq 2\pi$ .

```
syms t
expr = 1 + cos(t);
fpolarplot(expr)

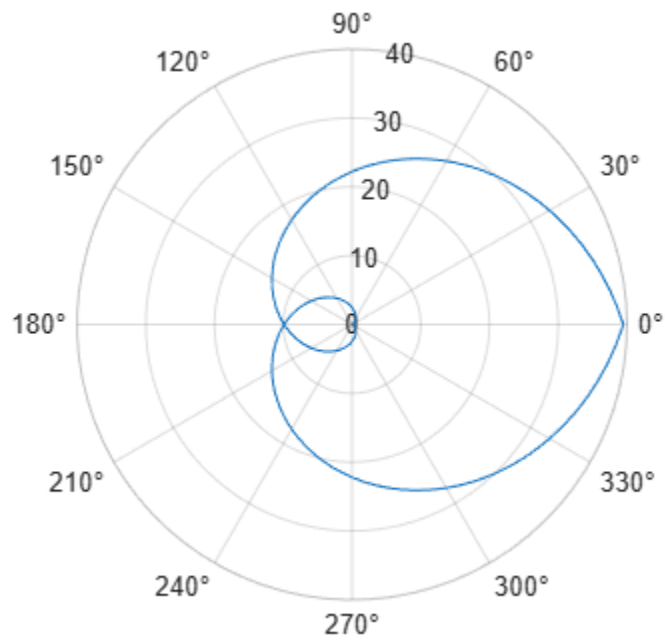
```



### Specify Plotting Interval

Plot  $f(t) = t^2$  over the interval  $-2\pi \leq t \leq 2\pi$  by specifying the plotting interval as the second input to `fpolarplot`.

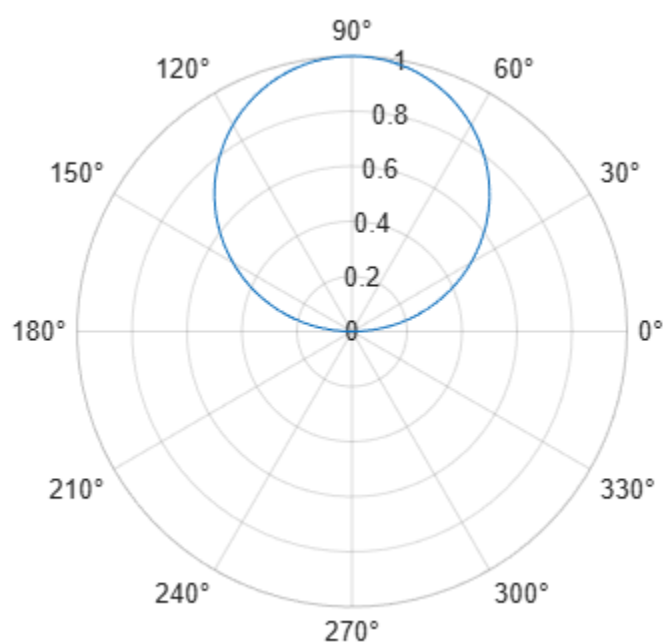
```
syms  $f(t)$   
 $f(t) = t^2$ ;  
fpolarplot(f(t), [-2*pi 2*pi])
```



### Plot Negative Radius Values

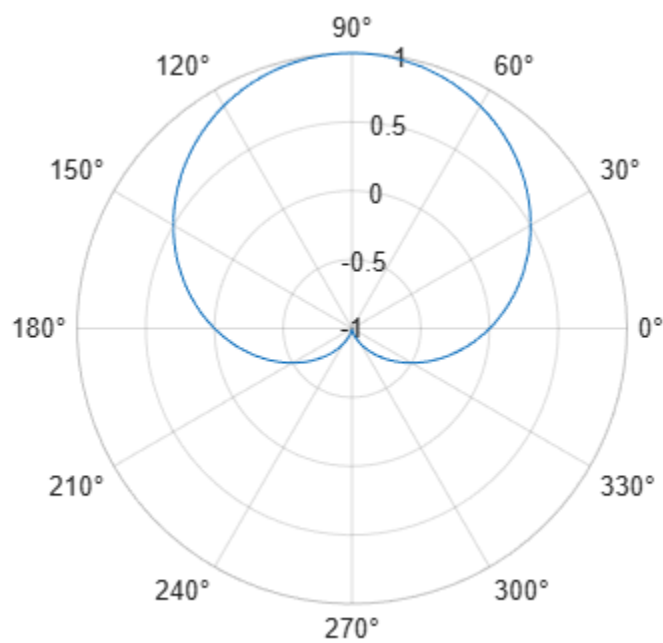
Create a polar plot using negative radius values. By default, `fpolarplot` reflects negative values through the origin.

```
syms t
f = sin(t);
fpolarplot(f)
```



Change the limits of the  $r$ -axis so it ranges from -1 to 1.

```
rlim([-1 1])
```

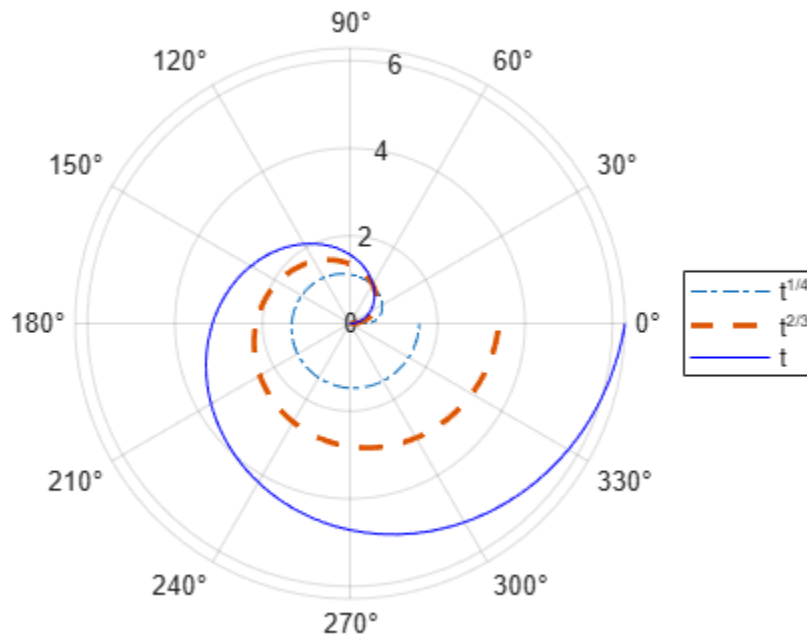




### Change Line Properties

Plot three curves in polar coordinates. Use a dash-dotted line for the first curve, a dashed line with a line width of 2 for the second curve, and a solid blue line for the third curve.

```
syms t
fpolarplot(t^(1/4), "-.")
hold on
fpolarplot(t^(2/3), "--", LineWidth=2)
fpolarplot(t, "-b")
legend
hold off
```



Alternatively, you can plot a vector of symbolic expressions by using a single `fpolarplot` command, and return an array of function line object handles.

```
syms t
figure
fp = fpolarplot([t^(1/4), t^(2/3), t])

fp =
    3×1 FunctionLine array:

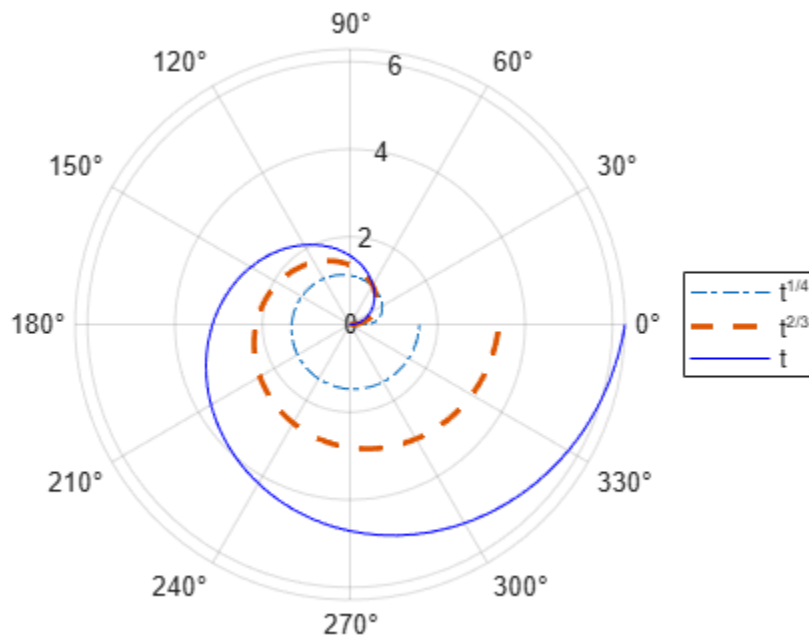
    FunctionLine
    FunctionLine
    FunctionLine
```

legend

You can access and modify properties of the function line object by using dot notation. For example, change the color, line style, and line width of each curve.

```
fp(1).LineStyle = "-.";
fp(2).LineStyle = "--";
```

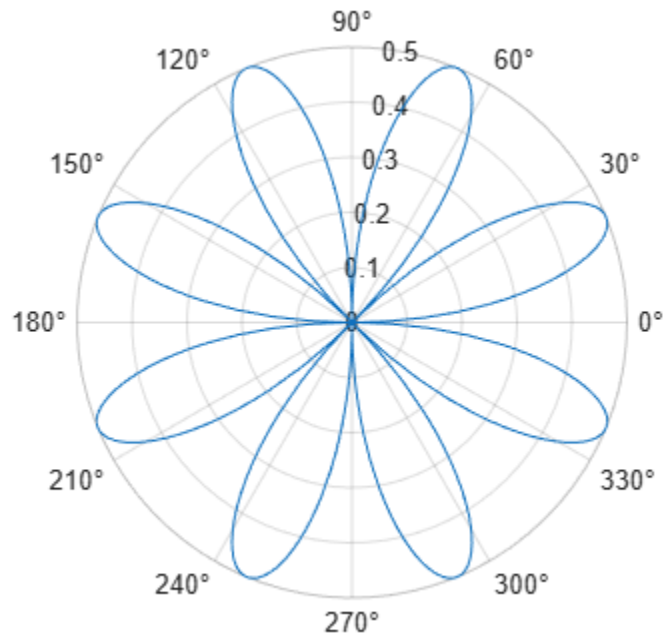
```
fp(2).LineWidth = 2;
fp(3).Color = "blue";
```



### Modify Polar Plot After Creation

Plot the curve  $r = \sin(2\theta) \cos(2\theta)$  in polar coordinates, and return the function line object.

```
syms theta
r = sin(2*theta)*cos(2*theta);
fp = fpolarplot(r)
```

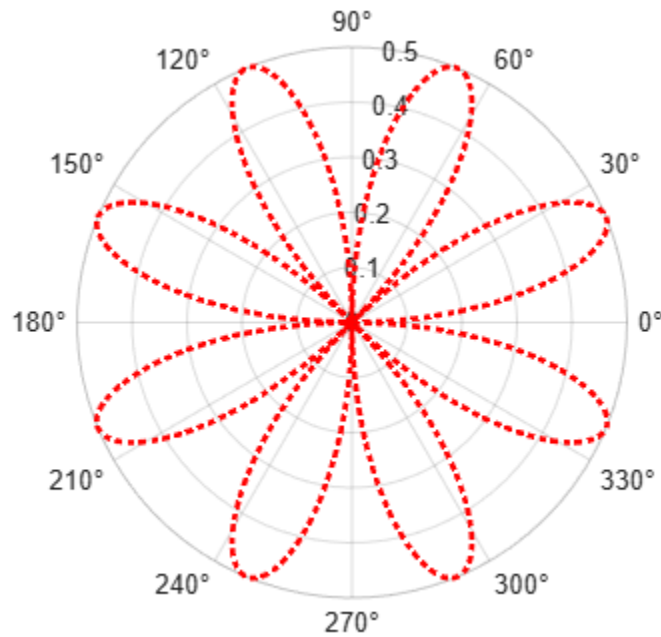


```
fp =  
  FunctionLine with properties:  
    Function: cos(2*theta)*sin(2*theta)  
      Color: [0.0660 0.4430 0.7450]  
    LineStyle: '-'  
    LineWidth: 0.5000
```

Show all properties

You can access and modify properties of the function line object by using dot notation. For example, change the color, line style, and line width.

```
fp.Color = "r";  
fp.LineStyle = ":";  
fp.LineWidth = 2;
```



## Input Arguments

### **f** — Expression or function to plot

symbolic expression | symbolic function | vector of symbolic expressions | vector of symbolic functions

Expression or function to plot, specified as a symbolic expression, symbolic function, vector of symbolic expressions, or vector of symbolic functions.

Example: `syms t; f(t) = [t^(1/4); t^(2/3); t]`

### **[a b]** — Interval for polar angle

`[0 2*pi]` (default) | two-element vector | two-element symbolic vector

Interval for the polar angle  $\theta$ , specified as a two-element vector or symbolic vector of the form `[a b]`. `fpolarplot` plots over the specified interval  $a \leq \theta \leq b$ .

### **pax** — PolarAxes object

PolarAxes object

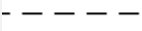
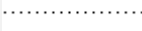
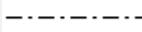
PolarAxes object. You can modify the appearance and behavior of a PolarAxes object by setting its properties. For a list of properties, see PolarAxes.

### **LineStyle** — Line style, marker, and color

string scalar | character vector

Line style, marker, and color, specified as a string scalar or character vector containing symbols. The symbols can appear in any order. You do not need to specify all three characteristics (line style, marker, and color). For example, if you omit the line style and specify the marker, then the plot shows only the marker and no line.

Example: `--or` is a red dashed line with circle markers.

| Line Style | Description      | Resulting Line                                                                      |
|------------|------------------|-------------------------------------------------------------------------------------|
| " - "      | Solid line       |  |
| " - - "    | Dashed line      |  |
| " : "      | Dotted line      |  |
| " - . "    | Dash-dotted line |  |

| Marker    | Description                | Resulting Marker                                                                      |
|-----------|----------------------------|---------------------------------------------------------------------------------------|
| "o"       | Circle                     |    |
| "+"       | Plus sign                  |    |
| "*"       | Asterisk                   |    |
| ". "      | Point                      |   |
| "x"       | Cross                      |  |
| " - "     | Horizontal line            |  |
| "   "     | Vertical line              |  |
| "square"  | Square                     |  |
| "diamond" | Diamond                    |  |
| "^"       | Upward-pointing triangle   |  |
| "v"       | Downward-pointing triangle |  |

| Marker      | Description             | Resulting Marker                                                                    |
|-------------|-------------------------|-------------------------------------------------------------------------------------|
| ">"         | Right-pointing triangle |  |
| "<"         | Left-pointing triangle  |  |
| "pentagram" | Pentagram               |  |
| "hexagram"  | Hexagram                |  |

| Color Name | Short Name | RGB Triplet | Appearance                                                                            |
|------------|------------|-------------|---------------------------------------------------------------------------------------|
| "red"      | "r"        | [1 0 0]     |    |
| "green"    | "g"        | [0 1 0]     |    |
| "blue"     | "b"        | [0 0 1]     |    |
| "cyan"     | "c"        | [0 1 1]     |  |
| "magenta"  | "m"        | [1 0 1]     |  |
| "yellow"   | "y"        | [1 1 0]     |  |
| "black"    | "k"        | [0 0 0]     |  |
| "white"    | "w"        | [1 1 1]     |  |

## Name-Value Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

The properties listed here are only a subset. For a full list, see `FunctionLine` Properties.

Example: `fpolarplot(f,Marker="o",MarkerFaceColor="red")`

### Color — Line color

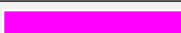
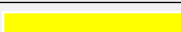
[0 0.4470 0.7410] (default) | RGB triplet | hexadecimal color code | "r" | "g" | "b" | ...

Line color, specified as an RGB triplet, a hexadecimal color code, a color name, or a short name.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range  $[0, 1]$ , for example,  $[0.4 \ 0.6 \ 0.7]$ .
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name | RGB Triplet | Hexadecimal Color Code | Appearance                                                                            |
|------------|------------|-------------|------------------------|---------------------------------------------------------------------------------------|
| "red"      | "r"        | [1 0 0]     | "#FF0000"              |    |
| "green"    | "g"        | [0 1 0]     | "#00FF00"              |    |
| "blue"     | "b"        | [0 0 1]     | "#0000FF"              |    |
| "cyan"     | "c"        | [0 1 1]     | "#00FFFF"              |    |
| "magenta"  | "m"        | [1 0 1]     | "#FF00FF"              |    |
| "yellow"   | "y"        | [1 1 0]     | "#FFFF00"              |    |
| "black"    | "k"        | [0 0 0]     | "#000000"              |   |
| "white"    | "w"        | [1 1 1]     | "#FFFFFF"              |  |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                       |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

Example: "blue"

Example: [0 0 1]

Example: "#0000FF"

**LineStyle — Line style**

" - " (default) | " - - " | " : " | " - . " | "none"

Line style, specified as one of the options listed in this table.

| Line Style | Description      | Resulting Line                                                                      |
|------------|------------------|-------------------------------------------------------------------------------------|
| " - "      | Solid line       |  |
| " - - "    | Dashed line      |  |
| " : "      | Dotted line      |  |
| " - . "    | Dash-dotted line |  |
| "none"     | No line          | No line                                                                             |

**LineWidth — Line width**

0.5 (default) | positive value

Line width, specified as a positive value in points, where 1 point = 1/72 of an inch. If the line has markers, then the line width also affects the marker edges.

The line width cannot be thinner than the width of a pixel. If you set the line width to a value that is less than the width of a pixel on your system, the line is displayed as one pixel wide.

**Marker — Marker symbol**

"none" (default) | "o" | "+" | "\*" | "." | ...

Marker symbol, specified as one of the values listed in this table. By default, the object does not display markers. Specifying a marker symbol adds markers at each data point or vertex.

| Marker | Description | Resulting Marker                                                                      |
|--------|-------------|---------------------------------------------------------------------------------------|
| "o"    | Circle      |  |
| "+"    | Plus sign   |  |
| "*"    | Asterisk    |  |
| "."    | Point       |  |
| "x"    | Cross       |  |



| Marker      | Description                | Resulting Marker                                                                      |
|-------------|----------------------------|---------------------------------------------------------------------------------------|
| " _ "       | Horizontal line            |    |
| "   "       | Vertical line              |    |
| "square"    | Square                     |    |
| "diamond"   | Diamond                    |    |
| "^"         | Upward-pointing triangle   |    |
| "v"         | Downward-pointing triangle |    |
| ">"         | Right-pointing triangle    |   |
| "<"         | Left-pointing triangle     |  |
| "pentagram" | Pentagram                  |  |
| "hexagram"  | Hexagram                   |  |
| "none"      | No markers                 | Not applicable                                                                        |

### MarkerEdgeColor — Marker outline color

"auto" (default) | RGB triplet | hexadecimal color code | "r" | "g" | "b" | ...

Marker outline color, specified as "auto", an RGB triplet, a hexadecimal color code, a color name, or a short name. The default value of "auto" uses the same color as the Color property.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range [0,1], for example, [0.4 0.6 0.7].
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case

sensitive. Therefore, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name     | RGB Triplet    | Hexadecimal Color Code | Appearance                                                                          |
|------------|----------------|----------------|------------------------|-------------------------------------------------------------------------------------|
| "red"      | "r"            | [1 0 0]        | "#FF0000"              |  |
| "green"    | "g"            | [0 1 0]        | "#00FF00"              |  |
| "blue"     | "b"            | [0 0 1]        | "#0000FF"              |  |
| "cyan"     | "c"            | [0 1 1]        | "#00FFFF"              |  |
| "magenta"  | "m"            | [1 0 1]        | "#FF00FF"              |  |
| "yellow"   | "y"            | [1 1 0]        | "#FFFF00"              |  |
| "black"    | "k"            | [0 0 0]        | "#000000"              |  |
| "white"    | "w"            | [1 1 1]        | "#FFFFFF"              |  |
| "none"     | Not applicable | Not applicable | Not applicable         | No color                                                                            |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                       |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

### MarkerFaceColor — Marker fill color

"none" (default) | "auto" | RGB triplet | hexadecimal color code | "r" | "g" | "b" | ...

Marker fill color, specified as "auto", an RGB triplet, a hexadecimal color code, a color name, or a short name. The "auto" value uses the same color as the `MarkerEdgeColor` property.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range  $[0, 1]$ , for example,  $[0.4 \ 0.6 \ 0.7]$ .
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name     | RGB Triplet    | Hexadecimal Color Code | Appearance                                                                            |
|------------|----------------|----------------|------------------------|---------------------------------------------------------------------------------------|
| "red"      | "r"            | [1 0 0]        | "#FF0000"              |    |
| "green"    | "g"            | [0 1 0]        | "#00FF00"              |    |
| "blue"     | "b"            | [0 0 1]        | "#0000FF"              |    |
| "cyan"     | "c"            | [0 1 1]        | "#00FFFF"              |    |
| "magenta"  | "m"            | [1 0 1]        | "#FF00FF"              |    |
| "yellow"   | "y"            | [1 1 0]        | "#FFFF00"              |    |
| "black"    | "k"            | [0 0 0]        | "#000000"              |  |
| "white"    | "w"            | [1 1 1]        | "#FFFFFF"              |  |
| "none"     | Not applicable | Not applicable | Not applicable         | No color                                                                              |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                       |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

Example: `[0.3 0.2 0.1]`

Example: `"green"`

Example: "#D2F9A7"

**MarkerSize — Marker size**

6 (default) | positive value

Marker size, specified as a positive value in points, where 1 point = 1/72 of an inch.

## Version History

Introduced in R2024a

### See Also

**Functions**

fcontour | fimplicit | fimplicit3 | fplot | fplot3

**Properties**

Function Line

**Topics**

"Create Plots of Symbolic Expressions" on page 4-2

# frac

Find fractional part

## Syntax

$Y = \text{frac}(X)$

## Description

$Y = \text{frac}(X)$  returns the fractional part of the specified symbolic number. If  $X$  contains a mixed number,  $\text{frac}(X)$  returns its proper fraction. If  $X$  contains an improper fraction,  $\text{frac}(X)$  divides the numerator of the fraction by its denominator and returns the remainder. This function is equivalent to  $X - \text{fix}(X)$ .

## Examples

### Find Fractional Part of Symbolic Number

Find the fractional part of the improper fraction  $\frac{8}{6}$ . Define the improper fraction as a symbolic number. This improper fraction is equal to  $1\frac{1}{3}$  in mixed number notation. The `frac` function returns  $\frac{1}{3}$  as the fractional part.

```
X = sym(8)/6;
Y = frac(X)
```

```
Y =
    1/3
```

### Find Fractional Part of Decimal Approximation of Irrational Number

Find the fractional part of  $\pi$ , approximated to 10 digits of precision. Set the number of significant digits to 10 when using `vpa` to approximate  $\pi$ .

```
digits(10)
X = vpa(sym(pi))
```

```
X = 3.141592654
```

Use `frac` to find the fractional part of this approximated number.

```
Y = frac(X)
```

```
Y = 0.1415926536
```

Restore the significant digits used by `vpa` to the default 32 digits.

```
digits(32)
```

### Find Fractional Part of Symbolic Expression

Define a symbolic expression `expr`. Find the fractional part by using `frac`. Because the expression contains a symbolic variable `n` with an indeterminate value, `frac` returns a result that is equivalent to `expr - fix(expr)`.

```
syms n
expr = 2*n - 2/3;
Y = frac(expr)
```

Y =  
$$2n - \text{fix}\left(2n - \frac{2}{3}\right) - \frac{2}{3}$$

### Input Arguments

#### X — Input array

symbolic number | symbolic variable | symbolic function | symbolic expression

Input array, specified as a symbolic number, variable, function, or expression.

Data Types: `sym` | `symfun`

### Output Arguments

#### Y — Fractional part

symbolic number | symbolic variable | symbolic expression

Fractional part, returned as a symbolic number, variable, or expression.

Data Types: `sym`

## Version History

Introduced in R2009b

### See Also

`partfrac` | `numden` | `simplifyFraction` | `round` | `fix` | `ceil` | `floor`

# fresnelc

Fresnel cosine integral function

## Syntax

`fresnelc(z)`

## Description

`fresnelc(z)` returns the Fresnel cosine integral on page 6-657 of *z*.

## Examples

### Fresnel Cosine Integral Function for Numeric and Symbolic Input Arguments

Find the Fresnel cosine integral function for these numbers. Since these are not symbolic objects, you receive floating-point results.

```
fresnelc([-2 0.001 1.22+0.31i])

ans =
-0.4883 + 0.0000i    0.0010 + 0.0000i    0.8617 - 0.2524i
```

Find the Fresnel cosine integral function symbolically by converting the numbers to symbolic objects:

```
y = fresnelc(sym([-2 0.001 1.22+0.31i]))

y =
[ -fresnelc(2), fresnelc(1/1000), fresnelc(61/50 + 31i/100)]
```

Use `vpa` to approximate results:

```
vpa(y)

ans =
[ -0.48825340607534075450022350335726, 0.00099999999999975325988997279422003,...
 0.86166573430841730950055370401908 - 0.25236540291386150167658349493972i]
```

### Fresnel Cosine Integral Function for Special Values

Find the Fresnel cosine integral function for special values:

```
fresnelc([0 Inf -Inf i*Inf -i*Inf])

ans =
0.0000 + 0.0000i    0.5000 + 0.0000i   -0.5000 + 0.0000i...
 0.0000 + 0.5000i    0.0000 - 0.5000i
```

### Fresnel Cosine Integral for Symbolic Functions

Find the Fresnel cosine integral for the function  $\exp(x) + 2x$ :

```
syms f(x)
f = exp(x)+2*x;
fresnelc(f)

ans =
fresnelc(2*x + exp(x))
```

### Fresnel Cosine Integral for Symbolic Vectors and Arrays

Find the Fresnel cosine integral for elements of vector V and matrix M:

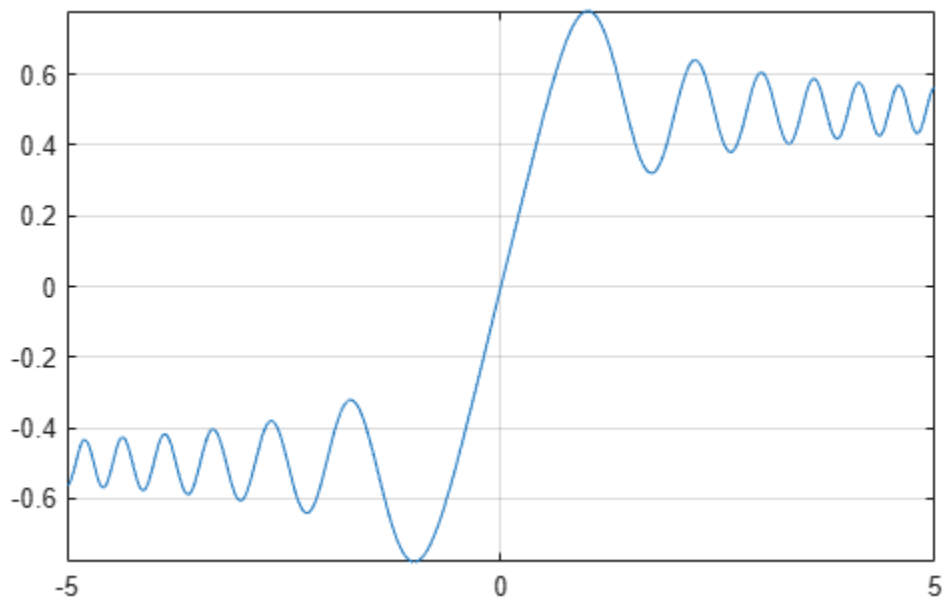
```
syms x
V = [sin(x) 2i -7];
M = [0 2; i exp(x)];
fresnelc(V)
fresnelc(M)

ans =
[ fresnelc(sin(x)), fresnelc(2i), -fresnelc(7)]
ans =
[      0,      fresnelc(2)]
[ fresnelc(1i), fresnelc(exp(x))]
```

### Plot Fresnel Cosine Integral Function

Plot the Fresnel cosine integral function from  $x = -5$  to  $x = 5$ .

```
syms x
fplot(fresnelc(x),[-5 5])
grid on
```





## Differentiate and Find Limits of Fresnel Cosine Integral

The functions `diff` and `limit` handle expressions containing `fresnelc`.

Find the third derivative of the Fresnel cosine integral function:

```
syms x
diff(fresnelc(x),x,3)

ans =
- pi*sin((pi*x^2)/2) - x^2*pi^2*cos((pi*x^2)/2)
```

Find the limit of the Fresnel cosine integral function as  $x$  tends to infinity:

```
syms x
limit(fresnelc(x),Inf)

ans =
1/2
```

## Taylor Series Expansion of Fresnel Cosine Integral

Use `taylor` to expand the Fresnel cosine integral in terms of the Taylor series:

```
syms x
taylor(fresnelc(x))

ans =
x - (x^5*pi^2)/40
```

## Simplify Expressions Containing fresnelc

Use `simplify` to simplify expressions:

```
syms x
simplify(3*fresnelc(x)+2*fresnelc(-x))

ans =
fresnelc(x)
```

## Input Arguments

### **z** — Upper limit on Fresnel cosine integral

numeric value | vector | matrix | multidimensional array | symbolic variable | symbolic expression | symbolic vector | symbolic matrix | symbolic function

Upper limit on the Fresnel cosine integral, specified as a numeric value, vector, matrix, or as a multidimensional array, or a symbolic variable, expression, vector, matrix, or function.

## More About

### Fresnel Cosine Integral

The Fresnel cosine integral of  $z$  is

$$\text{fresnelc}(z) = \int_0^z \cos\left(\frac{\pi t^2}{2}\right) dt.$$

## Algorithms

`fresnelc` is analytic throughout the complex plane. It satisfies  $\text{fresnelc}(-z) = -\text{fresnelc}(z)$ ,  $\text{conj}(\text{fresnelc}(z)) = \text{fresnelc}(\text{conj}(z))$ , and  $\text{fresnelc}(i*z) = i*\text{fresnelc}(z)$  for all complex values of  $z$ .

`fresnelc` returns special values for  $z = 0$ ,  $z = \pm\infty$ , and  $z = \pm i\infty$  which are 0,  $\pm 5$ , and  $\pm 0.5i$ .  
`fresnelc(z)` returns symbolic function calls for all other symbolic values of  $z$ .

## Version History

Introduced in R2014a

### See Also

`fresnels` | `erf`

# fresnels

Fresnel sine integral function

## Syntax

fresnels(z)

## Description

fresnels(z) returns the Fresnel sine integral on page 6-661 of z.

## Examples

### Fresnel Sine Integral Function for Numeric and Symbolic Arguments

Find the Fresnel sine integral function for these numbers. Since these are not symbolic objects, you receive floating-point results.

```
fresnels([-2 0.001 1.22+0.31i])

ans =
-0.3434 + 0.0000i    0.0000 + 0.0000i    0.7697 + 0.2945i
```

Find the Fresnel sine integral function symbolically by converting the numbers to symbolic objects:

```
y = fresnels(sym([-2 0.001 1.22+0.31i]))

y =
[ -fresnels(2), fresnels(1/1000), fresnels(61/50 + 31i/100)]
```

Use vpa to approximate the results:

```
vpa(y)

ans =
[ -0.34341567836369824219530081595807, 0.00000000052359877559820659249174920261227,...
 0.76969209233306959998384249252902 + 0.29449530344285433030167256417637i]
```

### Fresnel Sine Integral for Special Values

Find the Fresnel sine integral function for special values:

```
fresnels([0 Inf -Inf i*Inf -i*Inf])

ans =
0.0000 + 0.0000i    0.5000 + 0.0000i   -0.5000 + 0.0000i    0.0000 - 0.5000i...
0.0000 + 0.5000i
```

### Fresnel Sine Integral for Symbolic Functions

Find the Fresnel sine integral for the function  $\exp(x) + 2x$ :

```
syms x
f = symfun(exp(x)+2*x,x);
fresnels(f)
```

```
ans(x) =
fresnels(2*x + exp(x))
```

### Fresnel Sine Integral for Symbolic Vectors and Arrays

Find the Fresnel sine integral for elements of vector  $V$  and matrix  $M$ :

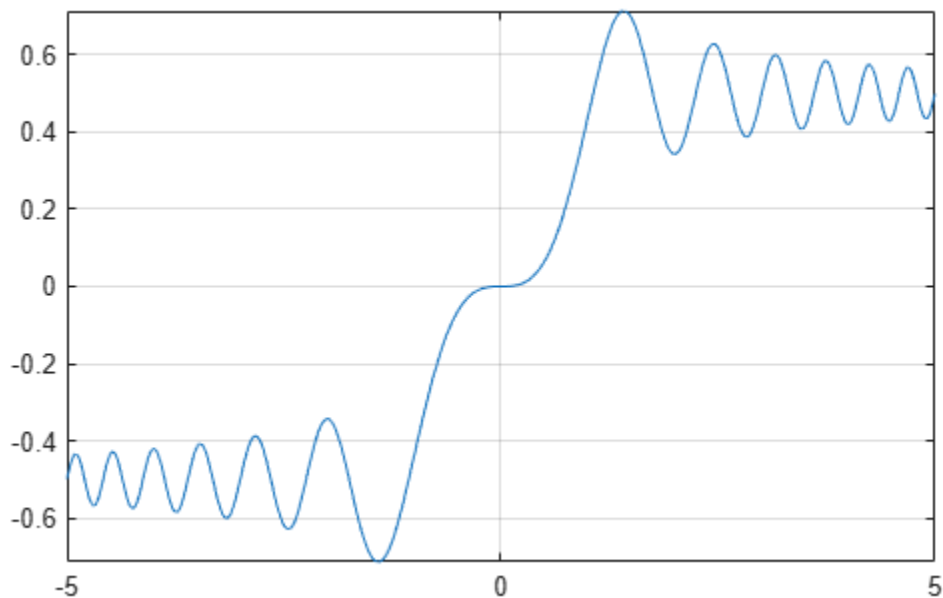
```
syms x
V = [sin(x) 2i -7];
M = [0 2; i exp(x)];
fresnels(V)
fresnels(M)

ans =
[ fresnels(sin(x)), fresnels(2i), -fresnels(7)]
ans =
[      0,      fresnels(2)]
[ fresnels(1i), fresnels(exp(x))]
```

### Plot Fresnel Sine Integral Function

Plot the Fresnel sine integral function from  $x = -5$  to  $x = 5$ .

```
syms x
fplot(fresnels(x),[-5 5])
grid on
```



### Differentiate and Find Limits of Fresnel Sine Integral

The functions `diff` and `limit` handle expressions containing `fresnels`.

Find the fourth derivative of the Fresnel sine integral function:

```
syms x
diff(fresnels(x),x,4)
```

```
ans =
- 3*x*pi^2*sin((pi*x^2)/2) - x^3*pi^3*cos((pi*x^2)/2)
```

Find the limit of the Fresnel sine integral function as  $x$  tends to infinity:

```
syms x
limit(fresnels(x),Inf)
```

```
ans =
1/2
```

### Taylor Series Expansion of Fresnel Sine Integral

Use `taylor` to expand the Fresnel sine integral in terms of the Taylor series:

```
syms x
taylor(fresnels(x))
```

```
ans =
(pi*x^3)/6
```

### Simplify Expressions Containing fresnels

Use `simplify` to simplify expressions:

```
syms x
simplify(3*fresnels(x)+2*fresnels(-x))
```

```
ans =
fresnels(x)
```

## Input Arguments

### $z$ — Upper limit on the Fresnel sine integral

numeric value | vector | matrix | multidimensional array | symbolic variable | symbolic expression | symbolic vector | symbolic matrix | symbolic function

Upper limit on the Fresnel sine integral, specified as a numeric value, vector, matrix, or a multidimensional array or as a symbolic variable, expression, vector, matrix, or function.

## More About

### Fresnel Sine Integral

The Fresnel sine integral of  $z$  is

$$\text{fresnels}(z) = \int^z \sin\left(\frac{\pi t^2}{2}\right) dt$$

.

## Algorithms

The `fresnels(z)` function is analytic throughout the complex plane. It satisfies  $\text{fresnels}(-z) = -\text{fresnels}(z)$ ,  $\text{conj}(\text{fresnels}(z)) = \text{fresnels}(\text{conj}(z))$ , and  $\text{fresnels}(i*z) = -i*\text{fresnels}(z)$  for all complex values of  $z$ .

`fresnels(z)` returns special values for  $z = 0$ ,  $z = \pm\infty$ , and  $z = \pm i\infty$  which are 0,  $\pm 5$ , and  $\mp 0.5i$ .  
`fresnels(z)` returns symbolic function calls for all other symbolic values of  $z$ .

## Version History

Introduced in R2014a

### See Also

`fresnelc` | `erf`

# fsurf

Plot 3-D surface

## Syntax

```
fsurf(f)
fsurf(f,[min max])
fsurf(f,[xmin xmax ymin ymax])

fsurf(funx,funy,funz)
fsurf(funx,funy,funz,[uvmin uvmax])
fsurf(funx,funy,funz,[umin umax vmin vmax])

fsurf( ___,LineSpec)
fsurf( ___,Name,Value)
fsurf(ax, ___)
fs = fsurf( ___)

```

## Description

`fsurf(f)` creates a surface plot of the symbolic expression  $f(x,y)$  over the default interval  $[-5\ 5]$  for  $x$  and  $y$ .

`fsurf(f,[min max])` plots  $f(x,y)$  over the interval  $[min\ max]$  for  $x$  and  $y$ .

`fsurf(f,[xmin xmax ymin ymax])` plots  $f(x,y)$  over the interval  $[xmin\ xmax]$  for  $x$  and  $[ymin\ ymax]$  for  $y$ . The `fsurf` function uses `symvar` to order the variables and assign intervals.

`fsurf(funx,funy,funz)` plots the parametric surface  $x = x(u,v)$ ,  $y = y(u,v)$ ,  $z = z(u,v)$  over the interval  $[-5\ 5]$  for  $u$  and  $v$ .

`fsurf(funx,funy,funz,[uvmin uvmax])` plots the parametric surface  $x = x(u,v)$ ,  $y = y(u,v)$ ,  $z = z(u,v)$  over the interval  $[uvmin\ uvmax]$  for  $u$  and  $v$ .

`fsurf(funx,funy,funz,[umin umax vmin vmax])` plots the parametric surface  $x = x(u,v)$ ,  $y = y(u,v)$ ,  $z = z(u,v)$  over the interval  $[umin\ umax]$  for  $u$  and  $[vmin\ vmax]$  for  $v$ . The `fsurf` function uses `symvar` to order the parametric variables and assign intervals.

`fsurf( ___,LineSpec)` uses `LineSpec` to set the line style, marker symbol, and face color. Use this option after any of the previous input argument combinations.

`fsurf( ___,Name,Value)` specifies line properties using one or more `Name,Value` pair arguments. Use this option after any of the input argument combinations in the previous syntaxes.

`fsurf(ax, ___)` plots into the axes with the object `ax` instead of the current axes object `gca`.

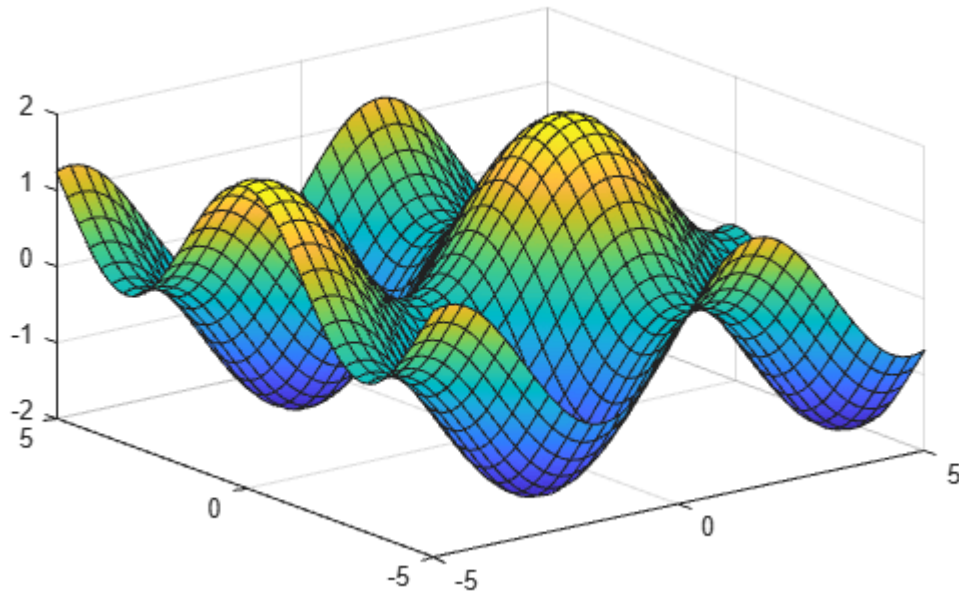
`fs = fsurf( ___)` returns a function surface object or parameterized function surface object, depending on the type of surface. Use the object to query and modify properties of a specific surface. For details, see [Function Surface](#) and [Parameterized Function Surface](#).

## Examples

### 3-D Surface Plot of Symbolic Expression

Plot the input  $\sin(x) + \cos(y)$  over the default range  $-5 < x < 5$  and  $-5 < y < 5$ .

```
syms x y
fsurf(sin(x)+cos(y))
```

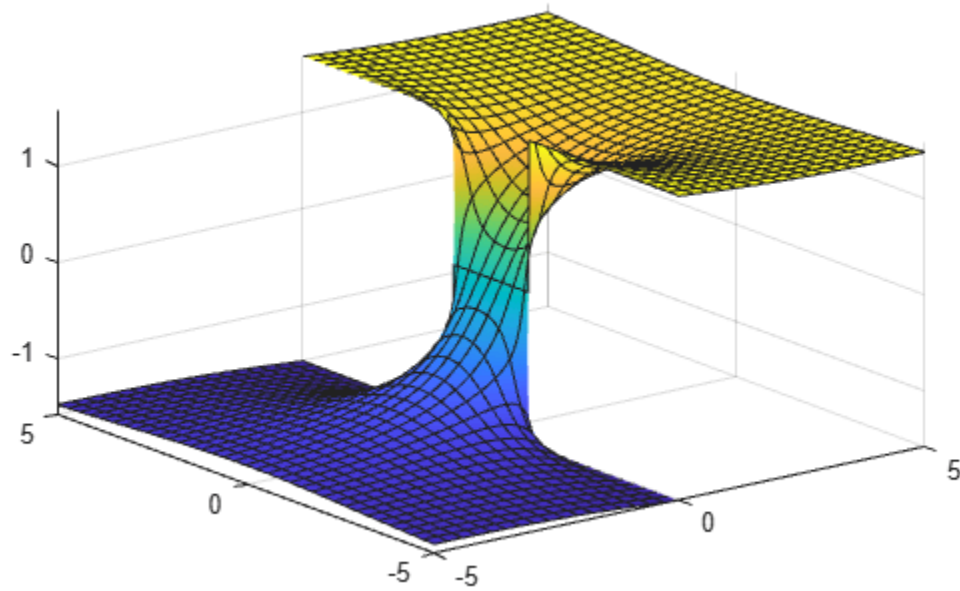


### 3-D Surface Plot of Symbolic Function

Plot the real part of  $\tan^{-1}(x + iy)$  over the default range  $-5 < x < 5$  and  $-5 < y < 5$ .

```
syms f(x,y)
f(x,y) = real(atan(x + i*y));
fsurf(f)
```

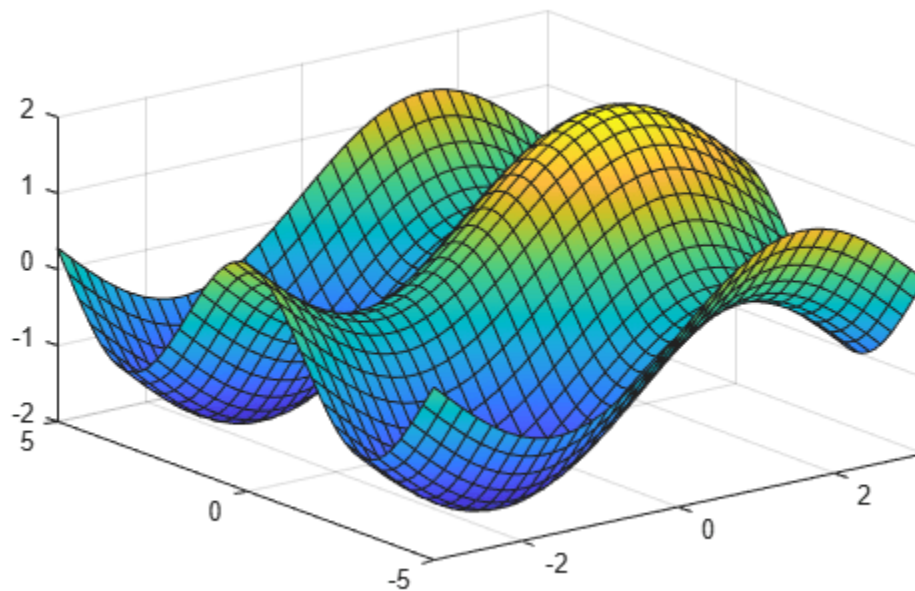




### Specify Plotting Interval of Surface Plot

Plot  $\sin(x) + \cos(y)$  over  $-\pi < x < \pi$  and  $-5 < y < 5$  by specifying the plotting interval as the second argument of `fsurf`.

```
syms x y  
f = sin(x) + cos(y);  
fsurf(f, [-pi pi -5 5])
```



### Parameterized Surface Plot

Plot the parameterized surface

$$x = r\cos(s)\sin(t)$$

$$y = r\sin(s)\sin(t)$$

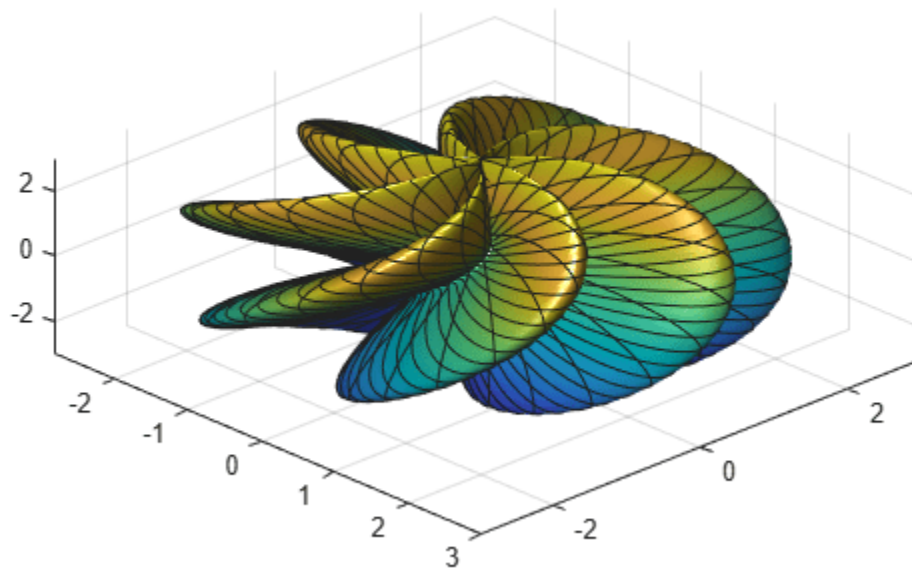
$$z = r\cos(t)$$

$$\text{where } r = 2 + \sin(7s + 5t)$$

for  $0 < s < 2\pi$  and  $0 < t < \pi$ .

Improve the plot's appearance by using `camlight`.

```
syms s t
r = 2 + sin(7*s + 5*t);
x = r*cos(s)*sin(t);
y = r*sin(s)*sin(t);
z = r*cos(t);
fsurf(x, y, z, [0 2*pi 0 pi])
camlight
view(46,52)
```



### Surface Plot of Piecewise Expression

Plot the piecewise expression of the Klein bottle

$$\begin{aligned}
 x(u, v) &= \begin{cases} -4 \cos(u) [1 + \sin(u)] - r(u) \cos(u) \cos(v) & 0 < u \leq \pi \\ -4 \cos(u) [1 + \sin(u)] + r(u) \cos(v) & \pi < u < 2\pi \end{cases} \\
 y(u, v) &= r(u) \sin(v) \\
 z(u, v) &= \begin{cases} -14 \sin(u) - r(u) \sin(u) \cos(v) & 0 < u \leq \pi \\ -14 \sin(u) & \pi < u < 2\pi \end{cases} \\
 \text{where } r(u) &= 4 - 2 \cos(u)
 \end{aligned}$$

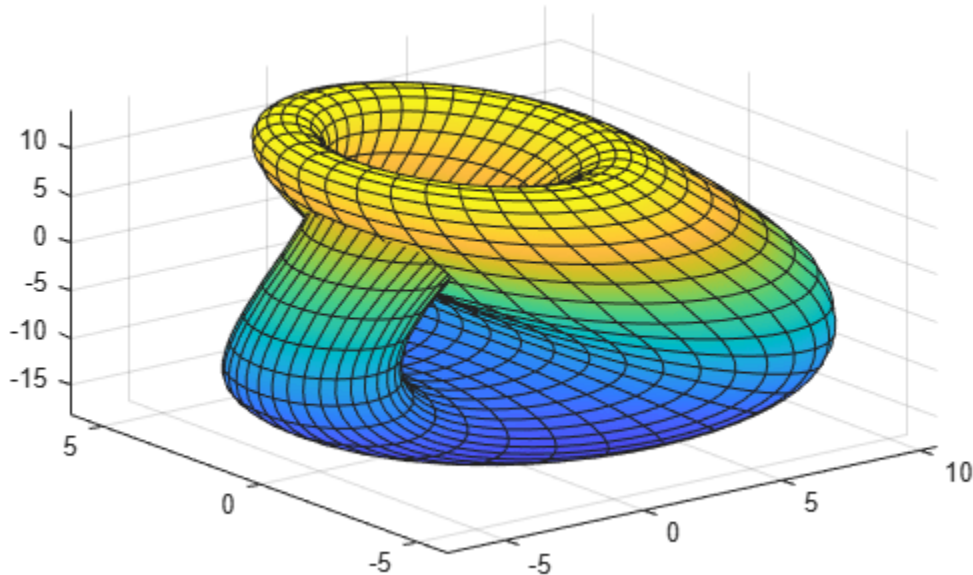
for  $0 < u < 2\pi$  and  $0 < v < 2\pi$ .

Show that the Klein bottle has only a one-sided surface.

```

syms u v;
r = @(u) 4 - 2*cos(u);
x = piecewise(u <= pi, -4*cos(u)*(1+sin(u)) - r(u)*cos(u)*cos(v), ...
              u > pi, -4*cos(u)*(1+sin(u)) + r(u)*cos(v));
y = r(u)*sin(v);
z = piecewise(u <= pi, -14*sin(u) - r(u)*sin(u)*cos(v), ...
              u > pi, -14*sin(u));
h = fsurf(x,y,z, [0 2*pi 0 2*pi]);

```



### Add Title and Axis Labels and Format Ticks

For  $x$  and  $y$  from  $-2\pi$  to  $2\pi$ , plot the 3-D surface  $y\sin(x) - x\cos(y)$ . Add a title and axis labels.

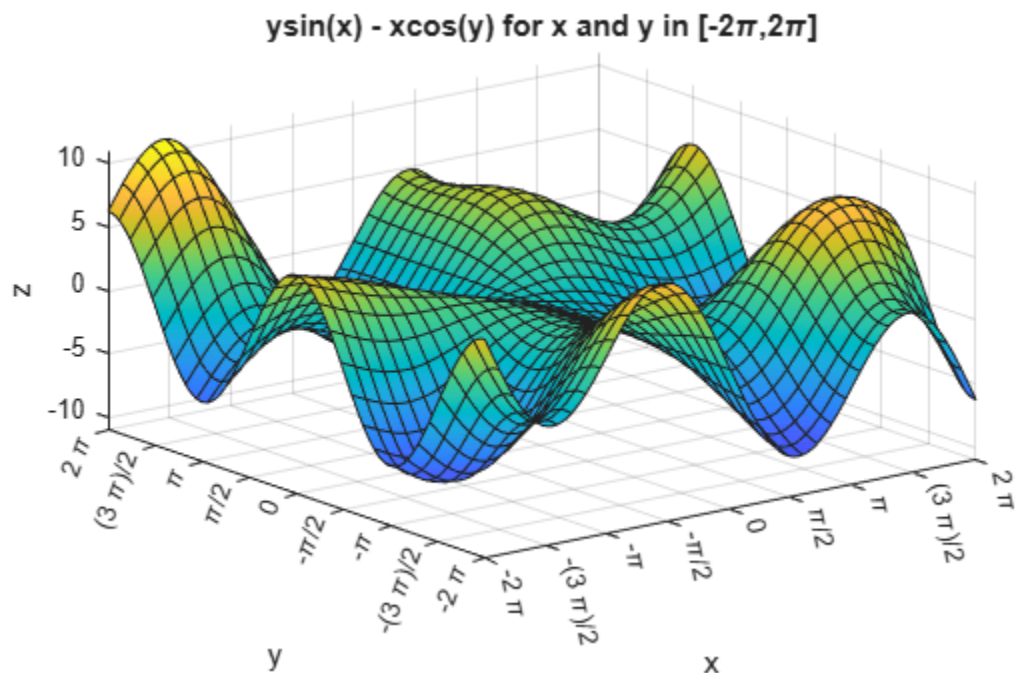
Create the x-axis ticks by spanning the x-axis limits at intervals of  $\pi/2$ . Convert the axis limits to precise multiples of  $\pi/2$  by using `round` and get the symbolic tick values in `S`. Display these ticks by using the `XTick` property. Create x-axis labels by using `arrayfun` to apply `texlabel` to `S`. Display these labels by using the `XTickLabel` property. Repeat these steps for the y-axis.

To use LaTeX in plots, see `latex`.

```
syms x y
fsurf(y.*sin(x)-x.*cos(y), [-2*pi 2*pi])
title('ysin(x) - xcos(y) for x and y in [-2\pi,2\pi]')
xlabel('x')
ylabel('y')
zlabel('z')

ax = gca;
S = sym(ax.XLim(1):pi/2:ax.XLim(2));
S = sym(round(vpa(S/pi*2))*pi/2);
ax.XTick = double(S);
ax.XTickLabel = arrayfun(@texlabel,S,'UniformOutput',false);

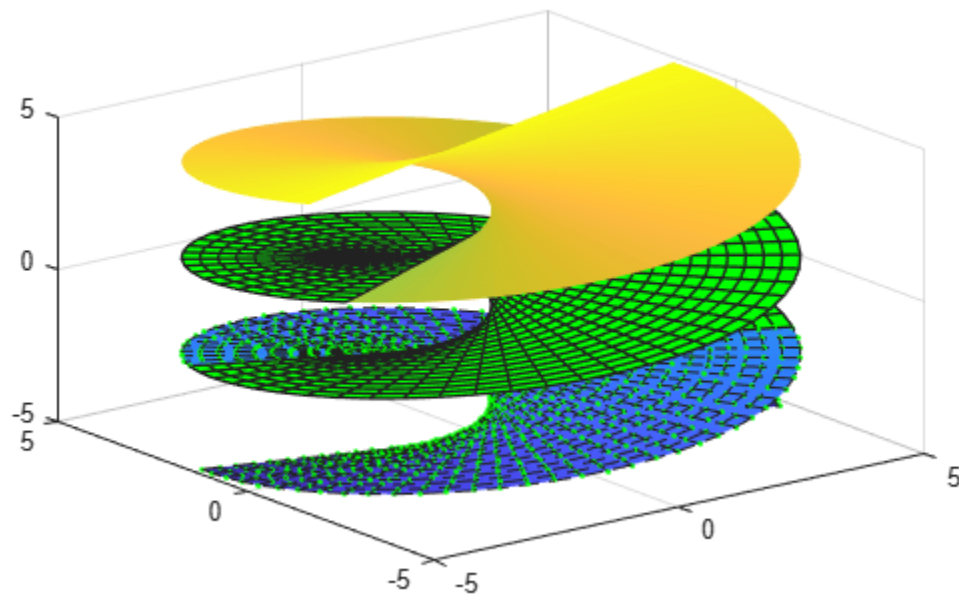
S = sym(ax.YLim(1):pi/2:ax.YLim(2));
S = sym(round(vpa(S/pi*2))*pi/2);
ax.YTick = double(S);
ax.YTickLabel = arrayfun(@texlabel,S,'UniformOutput',false);
```



### Line Style and Width for Surface Plot

Plot the parametric surface  $x = s\sin(t)$ ,  $y = -s\cos(t)$ ,  $z = t$  with different line styles for different values of  $t$ . For  $-5 < t < -2$ , use a dashed line with green dot markers. For  $-2 < t < 2$ , use a LineWidth of 1 and a green face color. For  $2 < t < 5$ , turn off the lines by setting EdgeColor to none.

```
syms s t
fsurf(s*sin(t),-s*cos(t),t,[-5 5 -5 -2], '--.', 'MarkerEdgeColor','g')
hold on
fsurf(s*sin(t),-s*cos(t),t,[-5 5 -2 2], 'LineWidth',1, 'FaceColor','g')
fsurf(s*sin(t),-s*cos(t),t,[-5 5 2 5], 'EdgeColor','none')
```



### Modify Surface After Creation

Plot the parametric surface

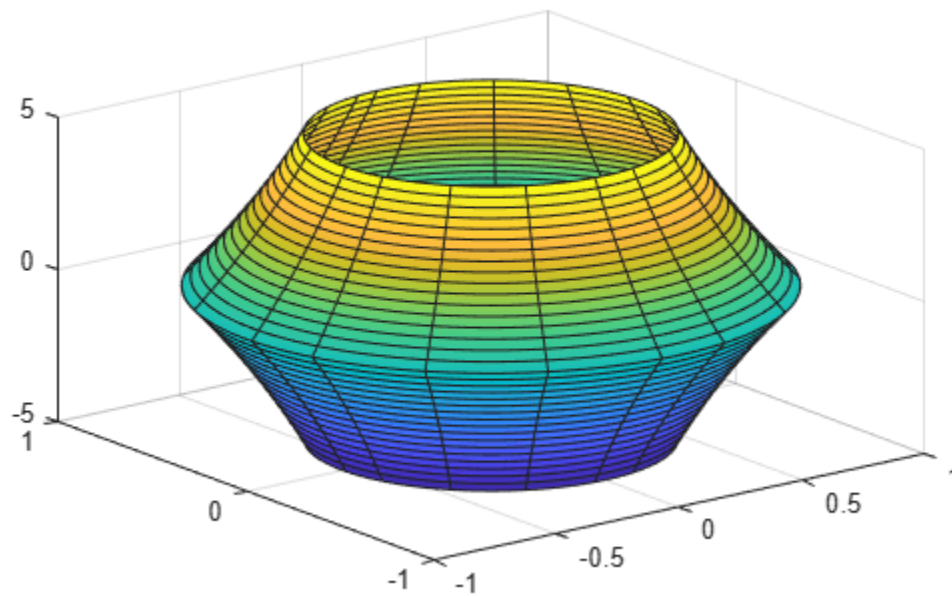
$$x = e^{-|u|/10} \sin(5|v|)$$

$$y = e^{-|u|/10} \cos(5|v|)$$

$$z = u.$$

Specify an output to make `fcontour` return the plot object.

```
syms u v
x = exp(-abs(u)/10).*sin(5*abs(v));
y = exp(-abs(u)/10).*cos(5*abs(v));
z = u;
fs = fsurf(x,y,z)
```



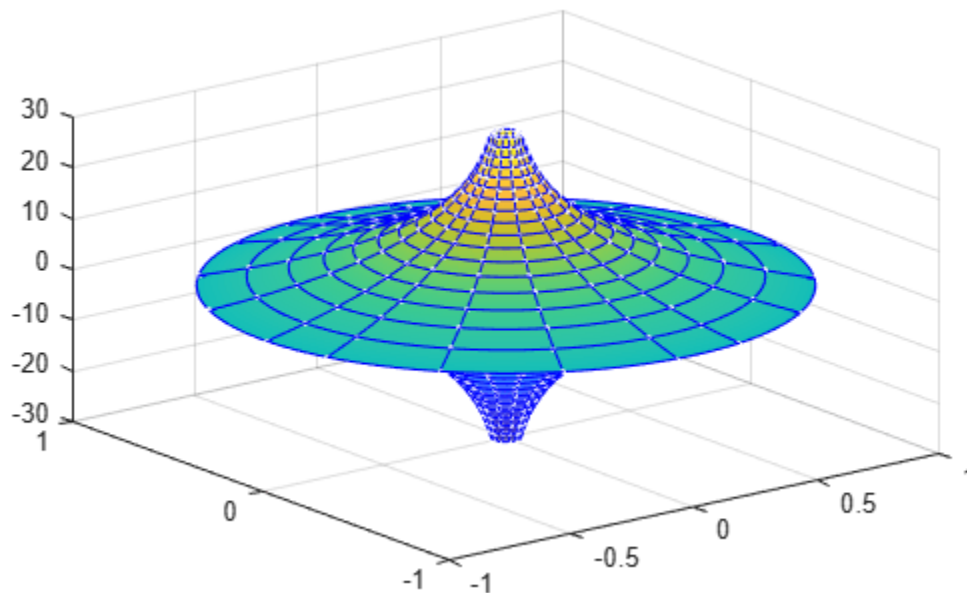
```
fs =
  ParameterizedFunctionSurface with properties:

    XFunction: exp(-abs(u)/10)*sin(5*abs(v))
    YFunction: exp(-abs(u)/10)*cos(5*abs(v))
    ZFunction: u
    EdgeColor: [0.1294 0.1294 0.1294]
    LineStyle: '-'
    FaceColor: 'interp'
```

Show all properties

Change the range of `u` to `[-30 30]` by using the `URange` property of `fs`. Set the line color to blue by using the `EdgeColor` property and specify white, dot markers by using the `Marker` and `MarkerEdgeColor` properties.

```
fs.URange = [-30 30];
fs.EdgeColor = 'b';
fs.Marker = '.';
fs.MarkerEdgeColor = 'w';
```



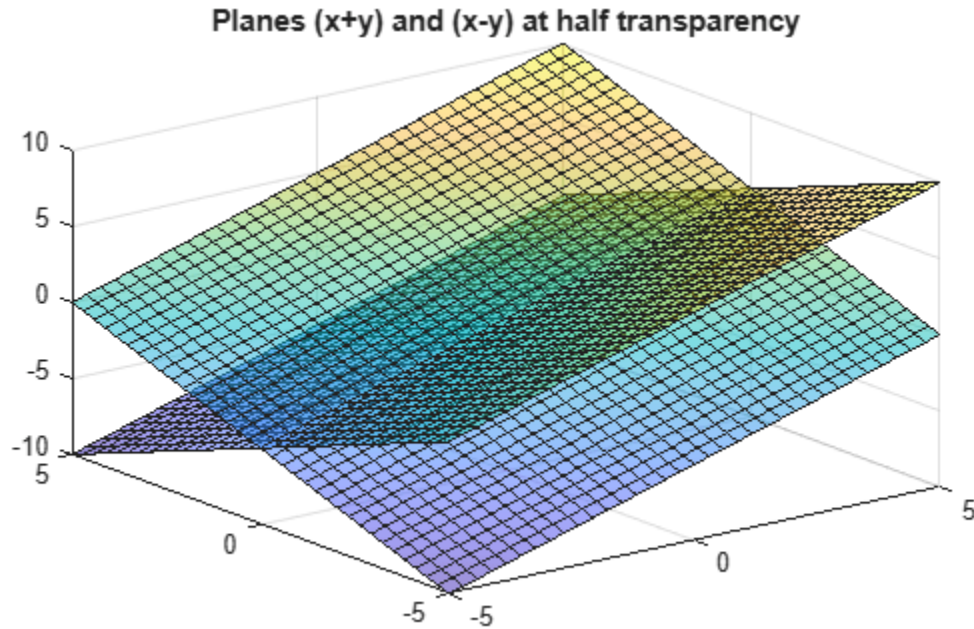
### Multiple Surface Plots and Transparent Surfaces

Plot multiple surfaces using vector input to `fsurf`. Alternatively, use `hold on` to plot successively on the same figure. When displaying multiple surfaces on the same figure, transparency is useful. Adjust the transparency of surface plots by using the `FaceAlpha` property. `FaceAlpha` varies from 0 to 1, where 0 is full transparency and 1 is no transparency.

Plot the planes  $x + y$  and  $x - y$  using vector input to `fsurf`. Show both planes by making them half transparent using `FaceAlpha`.

```
syms x y
h = fsurf([x+y x-y]);
h(1).FaceAlpha = 0.5;
h(2).FaceAlpha = 0.5;
title('Planes (x+y) and (x-y) at half transparency')
```





### Control Resolution of Surface Plot

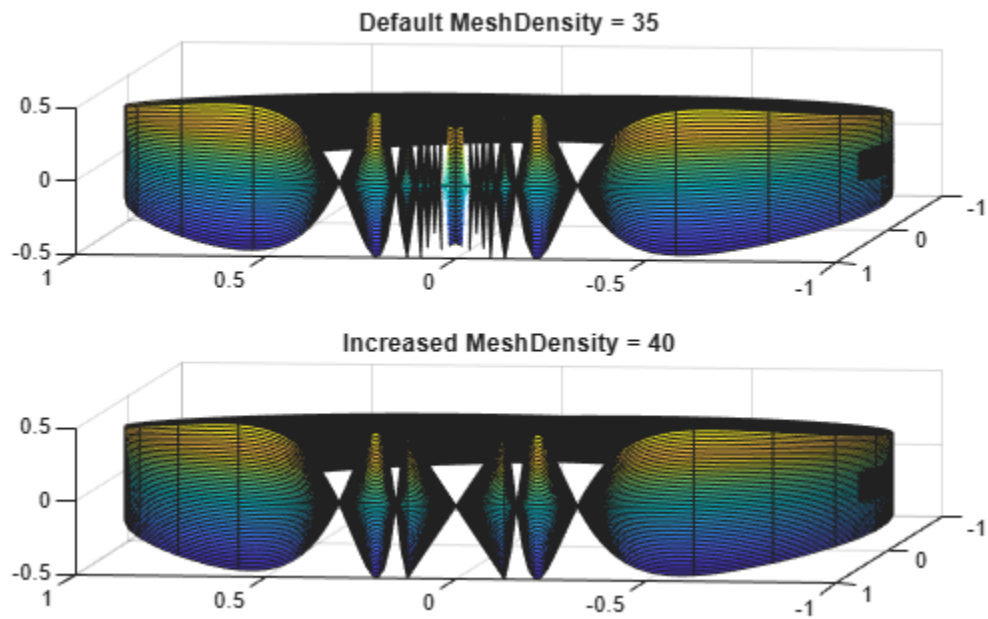
Control the resolution of a surface plot using the 'MeshDensity' option. Increasing 'MeshDensity' can make smoother, more accurate plots while decreasing it can increase plotting speed.

Divide a figure into two using `subplot`. In the first subplot, plot the parametric surface  $x = \sin(s)$ ,  $y = \cos(s)$ , and  $z = (t/10)\sin(1/s)$ . The surface has a large gap. Fix this issue by increasing the 'MeshDensity' to 40 in the second subplot. `fsurf` fills the gap showing that by increasing 'MeshDensity' you increased the plot's resolution.

`syms s t`

```
subplot(2,1,1)
fsurf(sin(s), cos(s), t/10.*sin(1./s))
view(-172,25)
title('Default MeshDensity = 35')
```

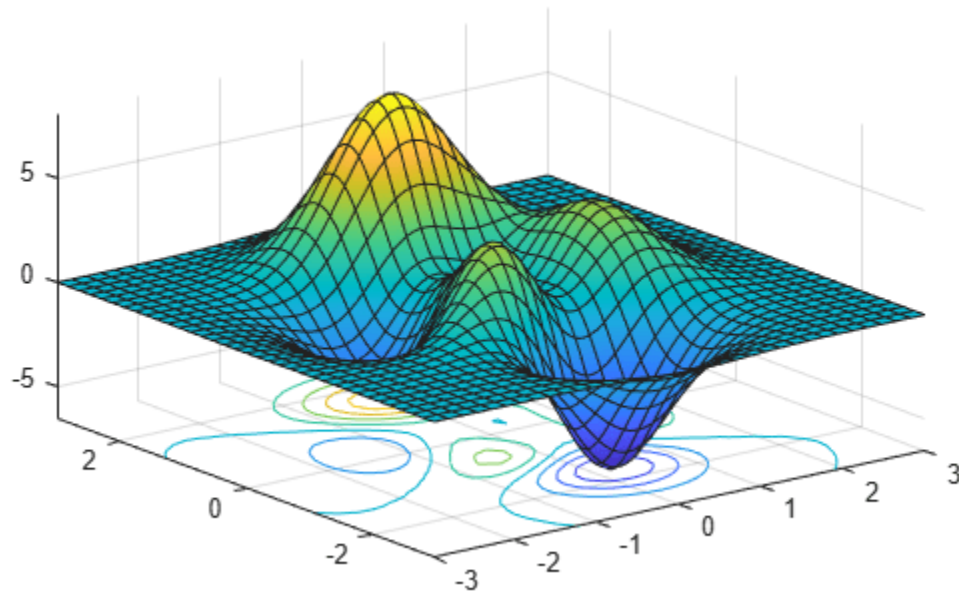
```
subplot(2,1,2)
fsurf(sin(s), cos(s), t/10.*sin(1./s), 'MeshDensity', 40)
view(-172,25)
title('Increased MeshDensity = 40')
```



### Show Contours Below Surface Plot

Show contours for the surface plot of the expression  $f$  by setting the 'ShowContours' option to 'on'.

```
syms x y
f = 3*(1-x)^2*exp(-(x^2)-(y+1)^2)...
- 10*(x/5 - x^3 - y^5)*exp(-x^2-y^2)...
- 1/3*exp(-(x+1)^2 - y^2);
fsurf(f,[-3 3], 'ShowContours', 'on')
```



### Create Animation of Surface Plots

Create an animation of surface plots by changing the displayed expression using the `Function`, `XFunction`, `YFunction`, and `ZFunction` properties, and then use `drawnow` to update the plot. To export to GIF, see `imwrite`.

By varying the variable  $k$  from 1 to 3, animate the parametric surface

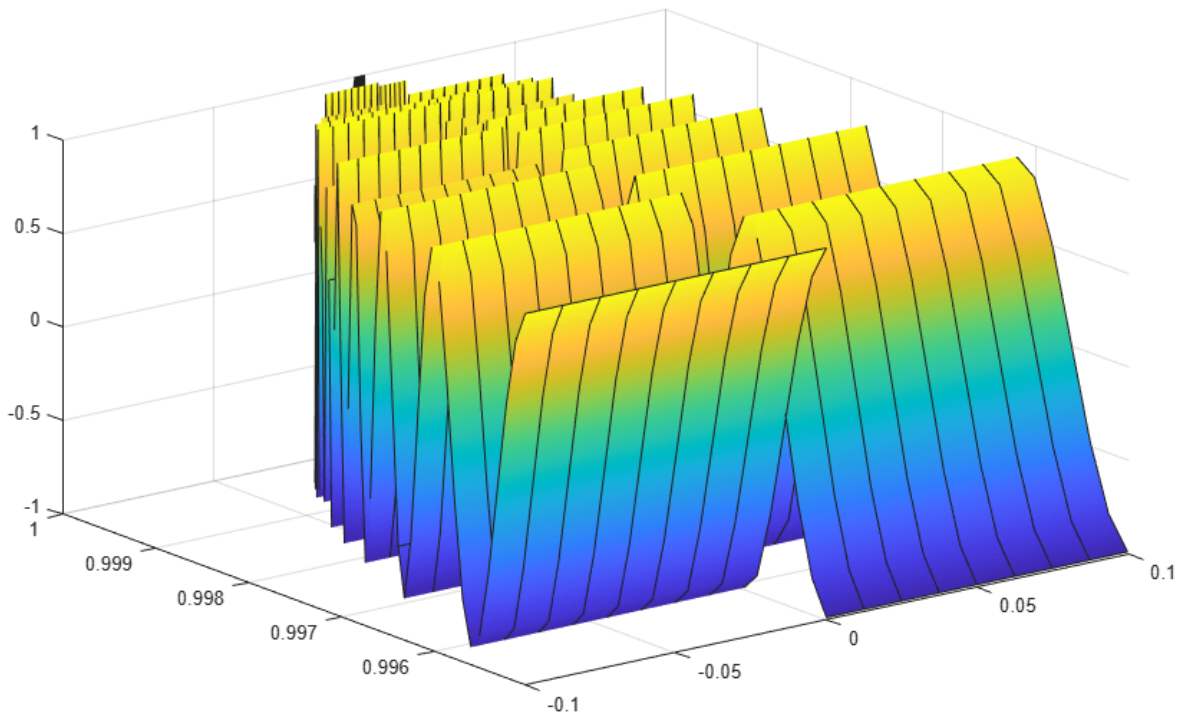
$$x = t \sin(s)$$

$$y = t \cos(s)$$

$$z = \sin(k/s)$$

for  $-0.1 < s < 0.1$  and  $0 < t < 1$ . Increase plotting speed by reducing `MeshDensity` to 9.

```
syms s t
h = fsurf(t.*sin(s), cos(s), sin(1./s), [-0.1 0.1 0 1]);
h.MeshDensity = 9;
for i=1:0.1:3
    h.ZFunction = sin(i./s);
    drawnow
end
```



### Improve Appearance of Surface Plot

Create a symbolic expression  $f$  for the function

$$f = 3(1-x)^2 \exp(-(x^2) - (y+1)^2) - 10(x/5 - x^3 - y^5) \exp(-x^2 - y^2) - 1/3 \exp(-(x+1)^2 - y^2).$$

Plot the expression  $f$  as a surface. Improve the appearance of the surface plot by using the properties of the handle returned by `fsurf`, the lighting properties, and the `colormap`.

Create a light by using `camlight`. Increase brightness by using `brighten`. Remove the lines by setting `EdgeColor` to `'none'`. Increase the ambient light using `AmbientStrength`. For details, see “Lighting, Transparency, and Shading”. Turn the axes box on. For the title, convert  $f$  to LaTeX using `latex`. Finally, to improve the appearance of the axes ticks, axes labels, and title, set `'Interpreter'` to `'latex'`.

```
syms x y
f = 3*(1-x)^2*exp(-(x^2)-(y+1)^2)...
    - 10*(x/5 - x^3 - y^5)*exp(-x^2-y^2)...
    - 1/3*exp(-(x+1)^2 - y^2);
h = fsurf(f,[-3 3]);

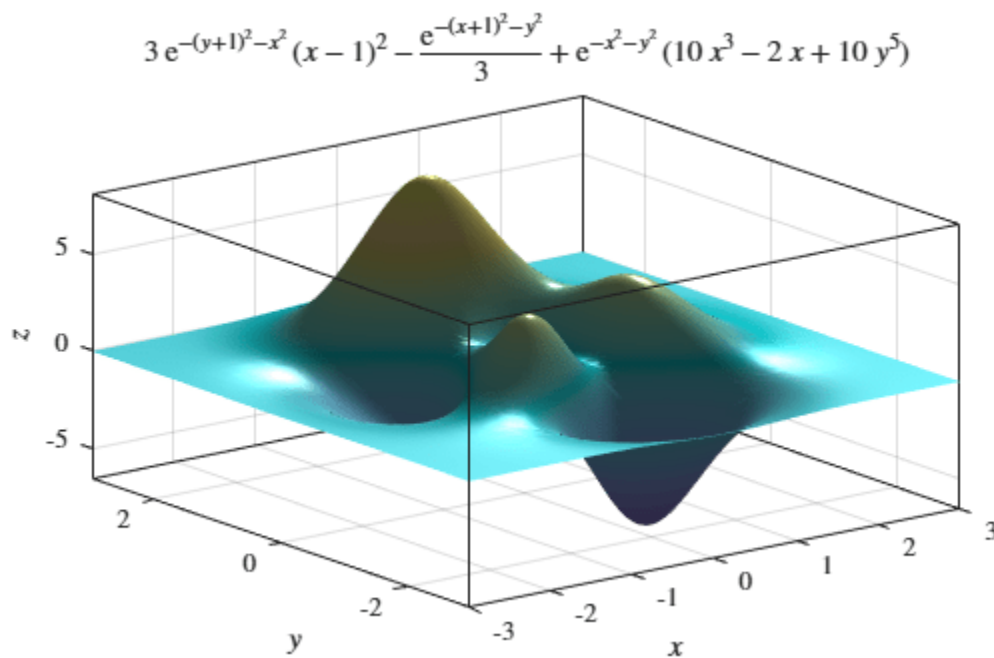
camlight(110,70)
brighten(0.6)
h.EdgeColor = 'none';
h.AmbientStrength = 0.4;
```

```

a = gca;
a.TickLabelInterpreter = 'latex';
a.Box = 'on';
a.BoxStyle = 'full';

xlabel('$x$', 'Interpreter', 'latex')
ylabel('$y$', 'Interpreter', 'latex')
zlabel('$z$', 'Interpreter', 'latex')
title_latex = ['$' latex(f) '$'];
title(title_latex, 'Interpreter', 'latex')

```



### Surface Plot with Bounded Plane

Plot a cylindrical shell bounded below by the  $x - y$  plane and above by the plane  $z = x + 2$ .

```

syms r t u
fsurf(cos(t), sin(t), u*(cos(t)+2), [0 2*pi 0 1])
hold on;

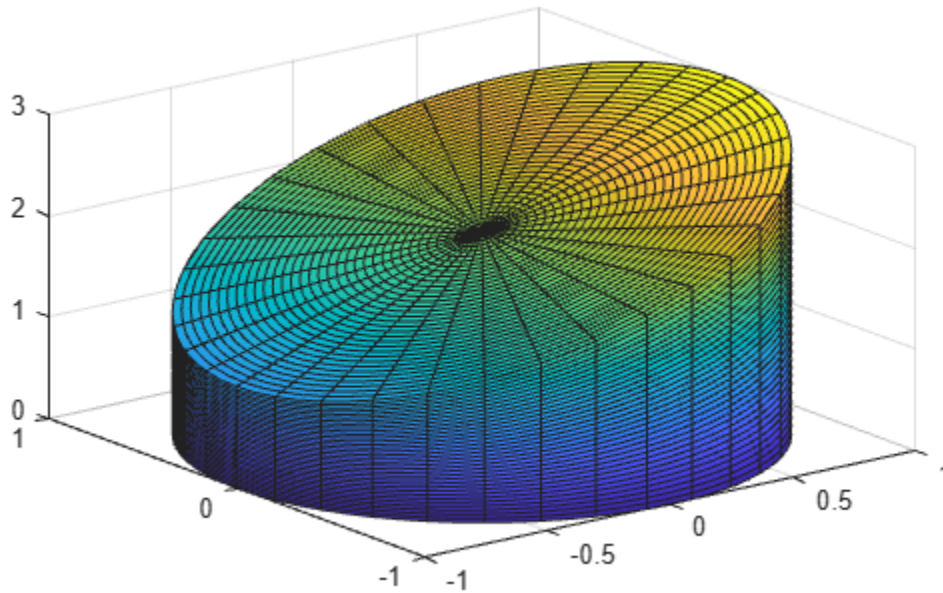
```

Add a surface plot of the plane  $z = x + 2$ .

```

fsurf(r*cos(t), r*sin(t), r*cos(t)+2, [0 1 0 2*pi])

```



### Apply Rotation and Translation to Surface Plot

Apply rotation and translation to the surface plot of a torus.

A torus can be defined parametrically by

$$\begin{aligned}x(\theta, \varphi) &= (R + a \cos \theta) \cos \varphi \\y(\theta, \varphi) &= (R + a \cos \theta) \sin \varphi \\z(\theta, \varphi) &= a \sin \theta\end{aligned}$$

where

- $\theta$  is the polar angle and  $\varphi$  is the azimuthal angle
- $a$  is the radius of the tube
- $R$  is the distance from the center of the tube to the center of the torus

Define the values for  $a$  and  $R$  as 1 and 5, respectively. Plot the torus using `fsurf`.

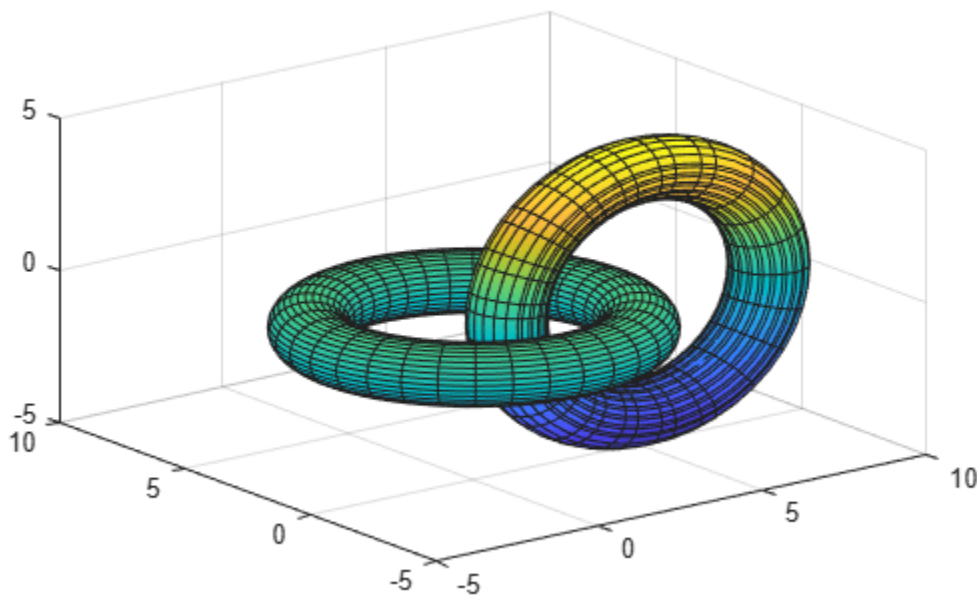
```
syms theta phi
a = 1;
R = 4;
x = (R + a*cos(theta))*cos(phi);
y = (R + a*cos(theta))*sin(phi);
z = a*sin(theta);
fsurf(x,y,z,[0 2*pi 0 2*pi])
hold on
```

Apply rotation to the torus around the x-axis. Define the rotation matrix. Rotate the torus by 90 degrees or  $\pi/2$  radians.

```
alpha = pi/2;
Rx = [1 0 0;
      0 cos(alpha) -sin(alpha);
      0 sin(alpha) cos(alpha)];
r = [x; y; z];
r_90 = Rx*r;
```

Shift the center of the torus by 5 along the x-axis. Add a second plot of the rotated and translated torus to the existing graph.

```
fsurf(r_90(1)+5,r_90(2),r_90(3))
axis([-5 10 -5 10 -5 5])
hold off
```



## Input Arguments

**f** — 3-D expression or function to be plotted

symbolic expression | symbolic function

Expression or function to be plotted, specified as a symbolic expression or function.

**[min max]** — Plotting interval for x- and y-axes

[-5 5] (default) | vector of two numbers

Plotting interval for x- and y-axes, specified as a vector of two numbers. The default is [-5 5].

**[xmin xmax ymin ymax]** — Plotting interval for x- and y-axes

[-5 5 -5 5] (default) | vector of four numbers

Plotting interval for x- and y-axes, specified as a vector of four numbers. The default is [-5 5 -5 5].

**funx, funy, funz — Parametric functions of u and v**

symbolic expressions | symbolic functions

Parametric functions of u and v, specified as a symbolic expression or function.

**[uvmín uvmax] — Plotting interval for u and v**

[-5 5] (default) | vector of two numbers

Plotting interval for u and v axes, specified as a vector of two numbers. The default is [-5 5].

**[umín umax vmín vmax] — Plotting interval for u and v**

[-5 5 -5 5] (default) | vector of four numbers

Plotting interval for u and v, specified as a vector of four numbers. The default is [-5 5 -5 5].

**ax — Axes object**

axes object

Axes object. If you do not specify an axes object, then fsurf uses the current axes.

**LineStyle — Line style, marker, and color**

string scalar | character vector

Line style, marker, and color, specified as a string scalar or character vector containing symbols. The symbols can appear in any order. You do not need to specify all three characteristics (line style, marker, and color). For example, if you omit the line style and specify the marker, then the plot shows only the marker and no line.

Example: "--or" is a red dashed line with circle markers.

| Line Style | Description      | Resulting Line                                                                        |
|------------|------------------|---------------------------------------------------------------------------------------|
| "_"        | Solid line       |  |
| "_ _"      | Dashed line      |  |
| " : "      | Dotted line      |  |
| "_ ."      | Dash-dotted line |  |

| Marker | Description | Resulting Marker                                                                      |
|--------|-------------|---------------------------------------------------------------------------------------|
| "o"    | Circle      |  |
| "+"    | Plus sign   |  |
| "*"    | Asterisk    |  |



| Marker      | Description                | Resulting Marker                                                                      |
|-------------|----------------------------|---------------------------------------------------------------------------------------|
| "."         | Point                      |    |
| "x"         | Cross                      |    |
| "_"         | Horizontal line            |    |
| " "         | Vertical line              |    |
| "square"    | Square                     |    |
| "diamond"   | Diamond                    |    |
| "^"         | Upward-pointing triangle   |   |
| "v"         | Downward-pointing triangle |  |
| ">"         | Right-pointing triangle    |  |
| "<"         | Left-pointing triangle     |  |
| "pentagram" | Pentagram                  |  |
| "hexagram"  | Hexagram                   |  |

| Color Name | Short Name | RGB Triplet | Appearance                                                                            |
|------------|------------|-------------|---------------------------------------------------------------------------------------|
| "red"      | "r"        | [1 0 0]     |  |
| "green"    | "g"        | [0 1 0]     |  |
| "blue"     | "b"        | [0 0 1]     |  |

| Color Name | Short Name | RGB Triplet | Appearance                                                                          |
|------------|------------|-------------|-------------------------------------------------------------------------------------|
| "cyan"     | "c"        | [0 1 1]     |  |
| "magenta"  | "m"        | [1 0 1]     |  |
| "yellow"   | "y"        | [1 1 0]     |  |
| "black"    | "k"        | [0 0 0]     |  |
| "white"    | "w"        | [1 1 1]     |  |

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `'Marker','o','MarkerFaceColor','red'`

The properties listed here are only a subset. For a complete list, see `Function Surface`.

### MeshDensity — Number of evaluation points per direction

35 (default) | number

Number of evaluation points per direction, specified as a number. The default is 35. Because `fsurf` objects use adaptive evaluation, the actual number of evaluation points is greater.

Example: 100

### ShowContours — Display contour plot under plot

'off' (default) | on/off logical value

Display contour plot under plot, specified as 'on' or 'off', or as numeric or logical 1 (true) or 0 (false). A value of 'on' is equivalent to true, and 'off' is equivalent to false. Thus, you can use the value of this property as a logical value. The value is stored as an on/off logical value of type `matlab.lang.OnOffSwitchState`.

### EdgeColor — Line color

[0 0 0] (default) | 'interp' | RGB triplet | hexadecimal color code | 'r' | 'g' | 'b' | ...

Line color, specified as 'interp', an RGB triplet, a hexadecimal color code, a color name, or a short name. The default RGB triplet value of [0 0 0] corresponds to black. The 'interp' value colors the edges based on the `ZData` values.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range [0, 1], for example, [0.4 0.6 0.7].
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case

sensitive. Therefore, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name     | RGB Triplet    | Hexadecimal Color Code | Appearance                                                                          |
|------------|----------------|----------------|------------------------|-------------------------------------------------------------------------------------|
| "red"      | "r"            | [1 0 0]        | "#FF0000"              |  |
| "green"    | "g"            | [0 1 0]        | "#00FF00"              |  |
| "blue"     | "b"            | [0 0 1]        | "#0000FF"              |  |
| "cyan"     | "c"            | [0 1 1]        | "#00FFFF"              |  |
| "magenta"  | "m"            | [1 0 1]        | "#FF00FF"              |  |
| "yellow"   | "y"            | [1 1 0]        | "#FFFF00"              |  |
| "black"    | "k"            | [0 0 0]        | "#000000"              |  |
| "white"    | "w"            | [1 1 1]        | "#FFFFFF"              |  |
| "none"     | Not applicable | Not applicable | Not applicable         | No color                                                                            |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                       |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

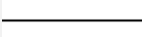
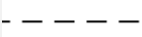
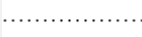
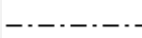
Get the RGB triplets using `RGB = get(groot,"FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X",round(RGB*255))`.

### LineStyle — Line style

" - " (default) | " - - " | " : " | " - . " | " none "

Line style, specified as one of the options listed in this table.

| Line Style | Description      | Resulting Line                                                                      |
|------------|------------------|-------------------------------------------------------------------------------------|
| " - "      | Solid line       |  |
| " - - "    | Dashed line      |  |
| " : "      | Dotted line      |  |
| " - . "    | Dash-dotted line |  |
| "none"     | No line          | No line                                                                             |

**LineWidth — Line width**

0.5 (default) | positive value

Line width, specified as a positive value in points, where 1 point = 1/72 of an inch. If the line has markers, then the line width also affects the marker edges.

The line width cannot be thinner than the width of a pixel. If you set the line width to a value that is less than the width of a pixel on your system, the line is displayed as one pixel wide.

**Marker — Marker symbol**

"none" (default) | "o" | "+" | "\*" | "." | ...

Marker symbol, specified as one of the values listed in this table. By default, the object does not display markers. Specifying a marker symbol adds markers at each data point or vertex.

| Marker | Description     | Resulting Marker                                                                      |
|--------|-----------------|---------------------------------------------------------------------------------------|
| "o"    | Circle          |  |
| "+"    | Plus sign       |  |
| "*"    | Asterisk        |  |
| "."    | Point           |  |
| "x"    | Cross           |  |
| " - "  | Horizontal line |  |
| "   "  | Vertical line   |  |

| Marker      | Description                | Resulting Marker                                                                      |
|-------------|----------------------------|---------------------------------------------------------------------------------------|
| "square"    | Square                     |    |
| "diamond"   | Diamond                    |    |
| "^"         | Upward-pointing triangle   |    |
| "v"         | Downward-pointing triangle |    |
| ">"         | Right-pointing triangle    |    |
| "<"         | Left-pointing triangle     |    |
| "pentagram" | Pentagram                  |   |
| "hexagram"  | Hexagram                   |  |
| "none"      | No markers                 | Not applicable                                                                        |

### MarkerEdgeColor — Marker outline color

'auto' (default) | RGB triplet | hexadecimal color code | 'r' | 'g' | 'b' | ...

Marker outline color, specified as 'auto', an RGB triplet, a hexadecimal color code, a color name, or a short name. The default value of 'auto' uses the same color as the EdgeColor property.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range [0,1], for example, [0.4 0.6 0.7].
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name     | RGB Triplet    | Hexadecimal Color Code | Appearance                                                                          |
|------------|----------------|----------------|------------------------|-------------------------------------------------------------------------------------|
| "red"      | "r"            | [1 0 0]        | "#FF0000"              |  |
| "green"    | "g"            | [0 1 0]        | "#00FF00"              |  |
| "blue"     | "b"            | [0 0 1]        | "#0000FF"              |  |
| "cyan"     | "c"            | [0 1 1]        | "#00FFFF"              |  |
| "magenta"  | "m"            | [1 0 1]        | "#FF00FF"              |  |
| "yellow"   | "y"            | [1 1 0]        | "#FFFF00"              |  |
| "black"    | "k"            | [0 0 0]        | "#000000"              |  |
| "white"    | "w"            | [1 1 1]        | "#FFFFFF"              |  |
| "none"     | Not applicable | Not applicable | Not applicable         | No color                                                                            |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                       |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |   |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

Example: [0.5 0.5 0.5]

Example: 'blue'

Example: '#D2F9A7'

### MarkerFaceColor — Marker fill color

"none" (default) | "auto" | RGB triplet | hexadecimal color code | "r" | "g" | "b" | ...

Marker fill color, specified as "auto", an RGB triplet, a hexadecimal color code, a color name, or a short name. The "auto" value uses the same color as the `MarkerEdgeColor` property.

For a custom color, specify an RGB triplet or a hexadecimal color code.

- An RGB triplet is a three-element row vector whose elements specify the intensities of the red, green, and blue components of the color. The intensities must be in the range  $[0, 1]$ , for example,  $[0.4 \ 0.6 \ 0.7]$ .
- A hexadecimal color code is a string scalar or character vector that starts with a hash symbol (#) followed by three or six hexadecimal digits, which can range from 0 to F. The values are not case sensitive. Therefore, the color codes "#FF8800", "#ff8800", "#F80", and "#f80" are equivalent.

Alternatively, you can specify some common colors by name. This table lists the named color options, the equivalent RGB triplets, and the hexadecimal color codes.

| Color Name | Short Name     | RGB Triplet    | Hexadecimal Color Code | Appearance                                                                            |
|------------|----------------|----------------|------------------------|---------------------------------------------------------------------------------------|
| "red"      | "r"            | [1 0 0]        | "#FF0000"              |    |
| "green"    | "g"            | [0 1 0]        | "#00FF00"              |    |
| "blue"     | "b"            | [0 0 1]        | "#0000FF"              |    |
| "cyan"     | "c"            | [0 1 1]        | "#00FFFF"              |    |
| "magenta"  | "m"            | [1 0 1]        | "#FF00FF"              |    |
| "yellow"   | "y"            | [1 1 0]        | "#FFFF00"              |    |
| "black"    | "k"            | [0 0 0]        | "#000000"              |  |
| "white"    | "w"            | [1 1 1]        | "#FFFFFF"              |  |
| "none"     | Not applicable | Not applicable | Not applicable         | No color                                                                              |

This table lists the default color palettes for plots in the light and dark themes.

| Palette                                                                | Palette Colors                                                                       |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| "gem" — Light theme default<br>Most plots use these colors by default. |  |
| "glow" — Dark theme default                                            |  |

You can get the RGB triplets and hexadecimal color codes for these palettes using the `orderedcolors` and `rgb2hex` functions. For example, get the RGB triplets for the "gem" palette and convert them to hexadecimal color codes.

```
RGB = orderedcolors("gem");
H = rgb2hex(RGB);
```

Get the RGB triplets using `RGB = get(groot, "FactoryAxesColorOrder")`.

Get the hexadecimal color codes using `H = compose("#%02X%02X%02X", round(RGB*255))`.

Example:  $[0.3 \ 0.2 \ 0.1]$

Example: "green"

Example: "#D2F9A7"

**MarkerSize — Marker size**

6 (default) | positive value

Marker size, specified as a positive value in points, where 1 point = 1/72 of an inch.

**Output Arguments****fs — One or more objects**

scalar | vector

One or more objects, returned as a scalar or a vector. The object is either a function surface object or parameterized surface object, depending on the type of plot. You can use these objects to query and modify properties of a specific line. For details, see [Function Surface](#) and [Parameterized Function Surface](#).

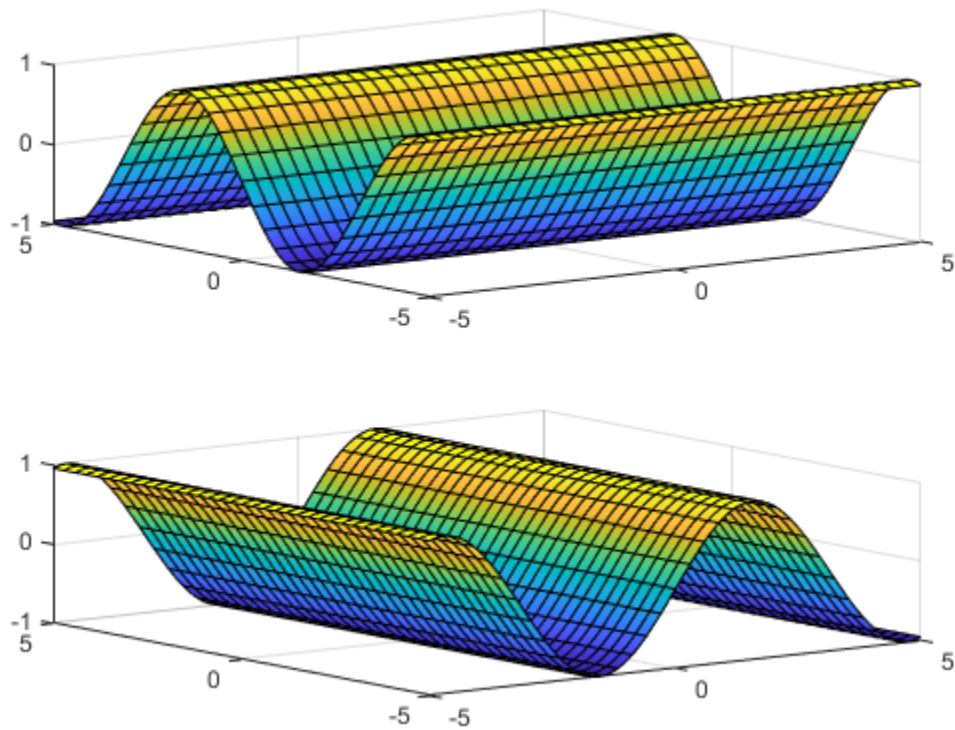
**Algorithms**

`fsurf` assigns the symbolic variables in `f` to the x-axis, then the y-axis, and `symvar` determines the order of the variables to be assigned. Therefore, variable and axis names might not correspond. To force `fsurf` to assign `x` or `y` to its corresponding axis, create the symbolic function to plot, then pass the symbolic function to `fsurf`.

For example, the following code plots  $f(x,y) = \sin(y)$  in two ways. The first way forces the waves to oscillate with respect to the y-axis. In other words, the first plot assigns the `y` variable to the corresponding y-axis. The second plot assigns `y` to the x-axis because it is the first (and only) variable in the symbolic function.

```
syms x y;  
f(x,y) = sin(y);  
  
figure;  
subplot(2,1,1)  
fsurf(f);  
subplot(2,1,2)  
fsurf(f(x,y)); % Or fsurf(sin(y));
```





## Version History

Introduced in R2016a

## See Also

### Functions

`fcontour` | `fimplicit` | `fimplicit3` | `fmesh` | `fplot` | `fplot3`

### Properties

Function Surface | Parameterized Function Surface

### Topics

“Create Plots of Symbolic Expressions” on page 4-2

## functionalDerivative

Functional derivative (variational derivative)

### Syntax

```
G = functionalDerivative(f,y)
```

### Description

`G = functionalDerivative(f,y)` returns the functional derivative on page 6-695  $\frac{\delta S}{\delta y}(x)$  of the functional  $S[y] = \int_a^b f[x, y(x), y'(x), \dots] dx$  with respect to the function  $y = y(x)$ , where  $x$  represents one or more independent variables. The functional derivative relates the change in the functional  $S[y]$  with respect to a small variation in  $y(x)$ . The functional derivative is also known as the variational derivative.

If  $y$  is a vector of symbolic functions, `functionalDerivative` returns a vector of functional derivatives with respect to the functions in  $y$ , where all functions in  $y$  must depend on the same independent variables.

### Examples

#### Functional Derivative with Respect to Single Function

Find the functional derivative of the functional  $S[y] = \int_a^b y(x) \sin(y(x)) dx$  with respect to the function  $y$ , where the integrand is  $f[y(x)] = y(x) \sin(y(x))$ .

Declare  $y(x)$  as a symbolic function and define  $f$  as the integrand of  $S$ . Use  $f$  and  $y$  as the parameters of `functionalDerivative`.

```
syms y(x)
f = y*sin(y);
G = functionalDerivative(f,y)
```

```
G(x) = sin(y(x)) + cos(y(x)) y(x)
```

#### Functional Derivative with Respect to Two Functions

Find the functional derivative of the functional  $S[u, v] = \int_a^b \left( u^2(x) \frac{dv(x)}{dx} + v(x) \frac{d^2 u(x)}{dx^2} \right) dx$  with respect to the functions  $u$  and  $v$ , where the integrand is  $f[u(x), v(x), u''(x), v'(x)] = u^2 \frac{dv}{dx} + v \frac{d^2 u}{dx^2}$ .

Declare  $u(x)$  and  $v(x)$  as symbolic functions, and define  $f$  as the integrand of  $S$ .

```
syms u(x) v(x)
f = u^2*diff(v,x) + v*diff(u,x,x);
```

Specify a vector of symbolic functions [u v] as the second input argument in `functionalDerivative`.

```
G = functionalDerivative(f,[u v])
```

$$G(x) = \begin{pmatrix} \frac{\partial^2}{\partial x^2} v(x) + 2 u(x) \frac{\partial}{\partial x} v(x) \\ \frac{\partial^2}{\partial x^2} u(x) - 2 u(x) \frac{\partial}{\partial x} u(x) \end{pmatrix}$$

`functionalDerivative` returns a vector of symbolic functions containing the functional derivatives of the integrand `f` with respect to `u` and `v`, respectively.

### Euler-Lagrange Equation of Simple Mass-Spring System

Find the Euler-Lagrange equation of a mass  $m$  that is connected to a spring with spring constant  $k$ .

Define the kinetic energy  $T$ , potential energy  $V$ , and Lagrangian  $L$  of the system. The Lagrangian is the difference between the kinetic and potential energy.

```
syms m k x(t)
T = 1/2*m*diff(x,t)^2;
V = 1/2*k*x^2;
L = T - V
```

$$L(t) = \frac{m \left( \frac{\partial}{\partial t} x(t) \right)^2}{2} - \frac{k x(t)^2}{2}$$

In Lagrangian mechanics, the action functional of the system is equal to the integral of the Lagrangian over time, or  $S[x] = \int_1^{t_2} L[t, x(t), \dot{x}(t)] dt$ . The Euler-Lagrange equation describes the motion of the system for which  $S[x(t)]$  is stationary.

Find the Euler-Lagrange equation by taking the functional derivative of the integrand  $L$  and setting it equal to 0.

```
eqn = functionalDerivative(L,x) == 0
```

$$\text{eqn}(t) = -m \frac{\partial^2}{\partial t^2} x(t) - k x(t) = 0$$

`eqn` is the differential equation that describes mass-spring oscillation.

Solve `eqn` using `dsolve`. Assume the mass  $m$  and spring constant  $k$  are positive. Set the initial conditions for the oscillation amplitude as  $x(0) = 10$  and the initial velocity of the mass as  $\dot{x}(0) = 0$ .

```

assume(m, 'positive')
assume(k, 'positive')
Dx(t) = diff(x(t), t);
xSol = dsolve(eqn, [x(0) == 10, Dx(0) == 0])

```

```

xSol =
    10 cos( $\frac{\sqrt{k} t}{\sqrt{m}}$ )

```

Clear assumptions for further calculations.

```

assume([k m], 'clear')

```

### Differential Equation of Brachistochrone Problem

The Brachistochrone problem is to find the quickest path of descent of a particle under gravity without friction. The motion is confined to a vertical plane. The time for a body to move along a curve  $y(x)$  from point  $a$  to  $b$  under gravity  $g$  is given by

$$t = \int_a^b \sqrt{\frac{1 + y'^2}{2gy}} dx.$$

Find the quickest path by minimizing the change in  $t$  with respect to small variations in the path  $y$ .

The condition for a minimum is  $\frac{\delta t}{\delta y}(x) = 0$ .

Compute the functional derivative to obtain the differential equation that describes the Brachistochrone problem. Use `simplify` to simplify the equation to its expected form.

```

syms g y(x)
assume(g, 'positive')
f = sqrt((1 + diff(y)^2)/(2*g*y));
eqn = functionalDerivative(f, y) == 0;
eqn = simplify(eqn)

```

```

eqn(x) =
    2 y(x)  $\frac{\partial^2}{\partial x^2} y(x) + \left(\frac{\partial}{\partial x} y(x)\right)^2 = -1$ 

```

This equation is the standard differential equation of the Brachistochrone problem. To find the solutions of the differential equation, use `dsolve`. Specify the 'Implicit' option to `true` to return implicit solutions, which have the form  $F(y(x)) = g(x)$ .

```

sols = dsolve(eqn, 'Implicit', true)

```

```

sols =

```

$$\begin{pmatrix} y(x) = C_5 + x \, i \\ y(x) = C_4 - x \, i \\ \sigma_1 = C_3 - x \\ \frac{C_1 + y(x)}{y(x)} = 0 \\ \sigma_1 = C_2 + x \end{pmatrix}$$

where

$$\sigma_1 = C_1 \operatorname{atan}\left(\sqrt{-\frac{C_1}{y(x)} - 1}\right) - y(x) \sqrt{-\frac{C_1}{y(x)} - 1}$$

The symbolic solver `dsolve` returns general solutions in the complex space. Symbolic Math Toolbox™ does not accept the assumption that the symbolic function  $y(x)$  is real.

Depending on the boundary conditions, there are two real-space solutions to the Brachistochrone problem. One of the two solutions below describes a cycloid curve in real space.

```
solCycloid1 = sols(3)
```

$$\text{solCycloid1} = C_1 \operatorname{atan}\left(\sqrt{-\frac{C_1}{y(x)} - 1}\right) - y(x) \sqrt{-\frac{C_1}{y(x)} - 1} = C_3 - x$$

```
solCycloid2 = sols(5)
```

$$\text{solCycloid2} = C_1 \operatorname{atan}\left(\sqrt{-\frac{C_1}{y(x)} - 1}\right) - y(x) \sqrt{-\frac{C_1}{y(x)} - 1} = C_2 + x$$

Another solution in real space is a horizontal straight line, where  $y$  is a constant.

```
solStraight = simplify(sols(4))
```

$$\text{solStraight} = C_1 + y(x) = 0$$

To illustrate the cycloid solution, consider an example with boundary conditions  $y(0) = 5$  and  $y(4) = 1$ . In this case, the equation that can satisfy the given boundary conditions is `solCycloid1`. Substitute the two boundary conditions into `solCycloid1`.

```
eq1 = subs(solCycloid1,[x y(x)],[0 5]);
eq2 = subs(solCycloid1,[x y(x)],[4 1]);
```

The two equations, `eq1` and `eq2`, have two unknown coefficients,  $C_1$  and  $C_3$ . Use `vpasolve` to find numerical solutions for the coefficients. Substitute these solutions into `solCycloid1`.

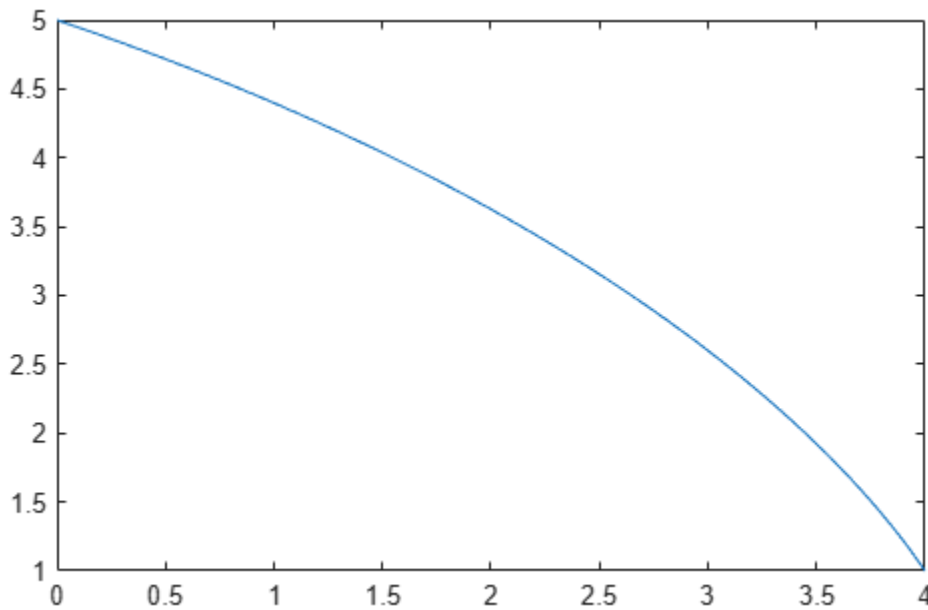
```
coeffs = vpasolve([eq1 eq2]);
eqCycloid = subs(solCycloid1,{'C1','C3'},{coeffs.C1,coeffs.C3})
```

$$\begin{aligned} \text{eqCycloid} = & -6.4199192418473511250705556729108 \operatorname{atan}\left(\sqrt{\frac{6.4199192418473511250705556729108}{y(x)} - 1}\right) \\ & - y(x) \sqrt{\frac{6.4199192418473511250705556729108}{y(x)} - 1} = -x - 5.8078336827583088482183433150164 \end{aligned}$$

The implicit equation `eqCycloid` describes the cycloid solution of the Brachistochrone problem in terms of  $x$  and  $y(x)$ .

You can then use `fimplicit` to plot `eqCycloid`. Since `fimplicit` only accepts implicit symbolic equations that contain symbolic variables  $x$  and  $y$ , convert the symbolic function  $y(x)$  to a symbolic variable  $y$ . Use `mapSymType` to convert  $y(x)$  to  $y$ . Plot the cycloid solution within the boundary conditions  $0 < x < 4$  and  $1 < y < 5$ .

```
funToVar = @(obj) sym('y');
eqPlot = mapSymType(eqCycloid, 'symfun', funToVar);
fimplicit(eqPlot,[0 4 1 5])
```



### Minimal Surface Equation in 3-D Space

For a function  $u(x, y)$  that describes a surface in 3-D space, the surface area can be determined by the functional

$$F[u] = \int_1^{y_2} \int_1^{x_2} f[x, y(x), u(x, y), u_x, u_y] dx dy = \int_1^{y_2} \int_1^{x_2} \sqrt{1 + u_x^2 + u_y^2} dx dy$$

where  $u_x$  and  $u_y$  are the partial derivatives of  $u$  with respect to  $x$  and  $y$ .

Find the functional derivative of the integrand  $f$  with respect to  $u$ .

```
syms u(x,y)
f = sqrt(1 + diff(u,x)^2 + diff(u,y)^2);
G = functionalDerivative(f,u)
```

$G(x, y) =$

$$-\frac{\left(\frac{\partial}{\partial y} u(x, y)\right)^2 \frac{\partial^2}{\partial x^2} u(x, y) + \frac{\partial^2}{\partial x^2} u(x, y) + \sigma_1^2 \frac{\partial^2}{\partial y^2} u(x, y) - 2 \frac{\partial}{\partial y} \sigma_1 \frac{\partial}{\partial y} u(x, y) \sigma_1 + \frac{\partial^2}{\partial y^2} u(x, y)}{\left(\sigma_1^2 + \left(\frac{\partial}{\partial y} u(x, y)\right)^2 + 1\right)^{3/2}}$$

where

$$\sigma_1 = \frac{\partial}{\partial x} u(x, y)$$

The result is the equation  $G$  that describes the minimal surface of a 3-D surface defined by  $u(x, y)$ . The solutions to this equation describe minimal surfaces in 3-D space, such as soap bubbles.

## Input Arguments

### **f** — Integrand of functional

symbolic variable | symbolic function | symbolic expression

Integrand of a functional, specified as a symbolic variable, function, or expression. The argument  $f$  represents the density of the functional.

### **y** — Differentiation function

unassigned symbolic function | vector of unassigned symbolic functions | matrix of unassigned symbolic functions | multidimensional array of unassigned symbolic functions

Differentiation function, specified as an unassigned symbolic function or a vector, matrix, or multidimensional array of unassigned symbolic functions. The argument  $y$  can be a function of one or more independent variables. If  $y$  is a vector of symbolic functions, `functionalDerivative` returns a vector of functional derivatives with respect to the functions in  $y$ , where all functions in  $y$  must depend on the same independent variables.

Example: `syms y(x); G = functionalDerivative(y*sin(y),y)`

## Output Arguments

### **G** — Functional derivative

symbolic function | vector of symbolic functions

Functional derivative, returned as a symbolic function or a vector of symbolic functions. If input  $y$  is a vector, matrix, or an array, then  $G$  is a vector.

## More About

### Functional Derivative

Consider a functional

$$S[y] = \int_a^b f[x, y(x), y'(x), \dots, y^{(n)}(x)] dx,$$

which can take any path from  $a$  to  $b$  in the  $x$ -space.

For a small variation in the path  $y(x)$ , define the change as  $\delta y(x) = \varepsilon \phi(x)$  in which  $\phi(x)$  is an arbitrary test function. The change in the functional  $S$  is

$$DS[y] = \lim_{\varepsilon \rightarrow 0} \frac{S[y + \varepsilon\phi] - S[y]}{\varepsilon} = \int_a^b \frac{\delta S}{\delta y}(x) \phi(x) dx.$$

The expression  $\frac{\delta S}{\delta y}(x)$  is the functional derivative of  $S$  with respect to  $y$ . The linear functional  $DS[y]$  is also known as the first variation or the Gateaux differential of the functional  $S$ .

One method to calculate the functional derivative is to apply Taylor expansion to the expression  $S[y + \varepsilon\phi]$  with respect to  $\varepsilon$ . By keeping the first order terms in  $\varepsilon$ , performing integration by parts, and choosing the boundary conditions  $\phi(a) = \phi(b) = \phi'(a) = \phi'(b) = \dots = 0$ , the functional derivative becomes

$$\begin{aligned} \frac{\delta S}{\delta y}(x) &= \frac{\partial f}{\partial y} - \frac{d}{dx} \frac{\partial f}{\partial y'} + \frac{d^2}{dx^2} \frac{\partial f}{\partial y''} - \dots + (-1)^n \frac{d^n}{dx^n} \left( \frac{\partial f}{\partial y^{(n)}} \right) \\ &= \sum_{i=0}^n (-1)^i \frac{d^i}{dx^i} \left( \frac{\partial f}{\partial y^{(i)}} \right). \end{aligned}$$

## Version History

Introduced in R2015a

### See Also

`diff` | `dsolve` | `int`

### Topics

“Functional Derivatives Tutorial” on page 3-226



# funm

General matrix function

## Syntax

```
F = funm(A,f)
```

## Description

`F = funm(A,f)` computes the function  $f(A)$  for the square matrix  $A$ . For details, see “Matrix Function” on page 6-700.

## Examples

### Matrix Cube Root

Find matrix  $B$ , such that  $B^3 = A$ , where  $A$  is a 3-by-3 identity matrix.

To solve  $B^3 = A$ , compute the cube root of the matrix  $A$  using the `funm` function. Create the symbolic function  $f(x) = x^{1/3}$  and use it as the second argument for `funm`. The cube root of an identity matrix is the identity matrix itself.

```
A = sym(eye(3))
```

```
syms f(x)
f(x) = x^(1/3);
```

```
B = funm(A,f)
```

```
A =
[ 1, 0, 0]
[ 0, 1, 0]
[ 0, 0, 1]
```

```
B =
[ 1, 0, 0]
[ 0, 1, 0]
[ 0, 0, 1]
```

Replace one of the 0 elements of matrix  $A$  with 1 and compute the matrix cube root again.

```
A(1,2) = 1
B = funm(A,f)
```

```
A =
[ 1, 1, 0]
[ 0, 1, 0]
[ 0, 0, 1]
```

```
B =
[ 1, 1/3, 0]
[ 0, 1, 0]
[ 0, 0, 1]
```

Now, compute the cube root of the upper triangular matrix.

```
A(1:2,2:3) = 1
B = funm(A,f)
```

```
A =
[ 1, 1, 1]
[ 0, 1, 1]
[ 0, 0, 1]
```

```
B =
[ 1, 1/3, 2/9]
[ 0, 1, 1/3]
[ 0, 0, 1]
```

Verify that  $B^3 = A$ .

```
B^3
```

```
ans =
[ 1, 1, 1]
[ 0, 1, 1]
[ 0, 0, 1]
```

### Matrix Lambert W Function

Find the matrix Lambert W function.

First, create a 3-by-3 matrix A using variable-precision arithmetic with five digit accuracy. In this example, using variable-precision arithmetic instead of exact symbolic numbers lets you speed up computations and decrease memory usage. Using only five digits helps the result to fit on screen.

```
savedefault = digits(5);
A = vpa(magic(3))
```

```
A =
[ 8.0, 1.0, 6.0]
[ 3.0, 5.0, 7.0]
[ 4.0, 9.0, 2.0]
```

Create the symbolic function  $f(x) = \text{lambertw}(x)$ .

```
syms f(x)
f(x) = lambertw(x);
```

To find the Lambert W function ( $W_0$  branch) in a matrix sense, call `funm` using  $f(x)$  as its second argument.

```
W0 = funm(A,f)
```

```
W0 =
[ 1.5335 + 0.053465i, 0.11432 + 0.47579i, 0.36208 - 0.52925i]
[ 0.21343 + 0.073771i, 1.3849 + 0.65649i, 0.41164 - 0.73026i]
[ 0.26298 - 0.12724i, 0.51074 - 1.1323i, 1.2362 + 1.2595i]
```

Verify that this result is a solution of the matrix equation  $A = W_0 \cdot e^{W_0}$  within the specified accuracy.

```
W0*expm(W0)
```

```
ans =
[ 8.0, 1.0 - 5.6843e-13i, 6.0 + 1.1369e-13i]
[ 3.0 - 2.2737e-13i, 5.0 - 2.8422e-14i, 7.0 - 4.1211e-13i]
[ 4.0 - 2.2737e-13i, 9.0 - 9.9476e-14i, 2.0 + 1.4779e-12i]
```

Now, create the symbolic function  $f(x)$  representing the branch  $W_{-1}$  of the Lambert W function.

```
f(x) = lambertw(-1,x);
```

Find the  $W_{-1}$  branch for the matrix A.

```
Wm1 = funm(A,f)
```

```
Wm1 =
[ 0.40925 - 4.7154i, 0.54204 + 0.5947i, 0.13764 - 0.80906i]
[ 0.38028 + 0.033194i, 0.65189 - 3.8732i, 0.056763 - 1.0898i]
[ 0.2994 - 0.24756i, - 0.105 - 1.6513i, 0.89453 - 3.0309i]
```

Verify that this result is the solution of the matrix equation  $A = Wm1 \cdot e^{Wm1}$  within the specified accuracy.

```
Wm1*expm(Wm1)
```

```
ans =
[ 8.0 - 8.3844e-13i, 1.0 - 3.979e-13i, 6.0 - 9.0949e-13i]
[ 3.0 - 9.6634e-13i, 5.0 + 1.684e-12i, 7.0 + 4.5475e-13i]
[ 4.0 - 1.3642e-12i, 9.0 + 1.6698e-12i, 2.0 + 1.7053e-13i]
```

## Matrix Exponential, Logarithm, and Square Root

You can use `funm` with appropriate second arguments to find matrix exponential, logarithm, and square root. However, the more efficient approach is to use the functions `expm`, `logm`, and `sqrtn` for this task.

Create this square matrix and find its exponential, logarithm, and square root.

```
syms x
A = [1 -1; 0 x]
expA = expm(A)
logA = logm(A)
sqrtA = sqrtn(A)
```

```
A =
[ 1, -1]
[ 0, x]
```

```
expA =
[ exp(1), (exp(1) - exp(x))/(x - 1)]
[ 0, exp(x)]
```

```
logA =
[ 0, -log(x)/(x - 1)]
[ 0, log(x)]
```

```
sqrtA =
[ 1, 1/(x - 1) - x^(1/2)/(x - 1)]
[ 0, x^(1/2)]
```

Find the matrix exponential, logarithm, and square root of  $A$  using `funm`. Use the symbolic expressions `exp(x)`, `log(x)`, and `sqrt(x)` as the second argument of `funm`. The results are identical.

```
expA = funm(A,exp(x))
logA = funm(A,log(x))
sqrtA = funm(A,sqrt(x))

expA =
[ exp(1), exp(1)/(x - 1) - exp(x)/(x - 1)]
[      0,                               exp(x)]

logA =
[ 0, -log(x)/(x - 1)]
[ 0,      log(x)]

sqrtA =
[ 1, 1/(x - 1) - x^(1/2)/(x - 1)]
[ 0,                x^(1/2)]
```

## Input Arguments

### A — Input matrix

square matrix

Input matrix, specified as a square symbolic or numeric matrix.

### f — Function

symbolic function | symbolic expression

Function, specified as a symbolic function or expression.

## Output Arguments

### F — Resulting matrix

symbolic matrix

Resulting function, returned as a symbolic matrix.

## More About

### Matrix Function

Matrix function is a scalar function that maps one matrix to another.

Suppose,  $f(x)$ , where  $x$  is a scalar, has a Taylor series expansion. Then the matrix function  $f(A)$ , where  $A$  is a matrix, is defined by the Taylor series of  $f(A)$ , with addition and multiplication performed in the matrix sense.

If  $A$  can be represented as  $A = P \cdot D \cdot P^{-1}$ , where  $D$  is a diagonal matrix, such that

$$D = \begin{pmatrix} d_1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & d_n \end{pmatrix}$$

then the matrix function  $f(A)$  can be computed as follows:

$$f(A) = P \cdot \begin{pmatrix} f(d_1) & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & f(d_n) \end{pmatrix} \cdot P^{-1}$$

Non-diagonalizable matrices can be represented as  $A = P \cdot J \cdot P^{-1}$ , where  $J$  is a Jordan form of the matrix  $A$ . Then, the matrix function  $f(A)$  can be computed by using the following definition on each Jordan block:

$$f \left( \begin{pmatrix} \lambda & 1 & 0 & \cdots & 0 \\ 0 & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & 1 \\ 0 & \cdots & \cdots & 0 & \lambda \end{pmatrix} \right) = \begin{pmatrix} \frac{f(\lambda)}{0!} & \frac{f'(\lambda)}{1!} & \frac{f''(\lambda)}{2!} & \cdots & \frac{f^{(n-1)}(\lambda)}{(n-1)!} \\ 0 & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \frac{f''(\lambda)}{2!} \\ \vdots & \ddots & \ddots & \ddots & \frac{f'(\lambda)}{1!} \\ 0 & \cdots & \cdots & 0 & \frac{f(\lambda)}{0!} \end{pmatrix}$$

## Tips

- For compatibility with the MATLAB `funm` function, `funm` accepts the following arguments:
  - Function handles such as `@exp` and `@sin`, as its second input argument.
  - The `options` input argument, such as `funm(A,f,options)`.
  - Additional input arguments of the function `f`, such as `funm(A,f,options,p1,p2,...)`
  - The `exitflag` output argument, such as `[F,exitflag] = funm(A,f)`. Here, `exitflag` is 1 only if the `funm` function call errors, for example, if it encounters a division by zero. Otherwise, `exitflag` is 0.

For more details and a list of all acceptable function handles, see the MATLAB `funm` function.

- If the input matrix `A` is numeric (not a symbolic object) and the second argument `f` is a function handle, then the `funm` call invokes the MATLAB `funm` function.

## Version History

Introduced in R2014b

## See Also

`eig` | `expm` | `jordan` | `logm` | `sqrtn`

## funtool

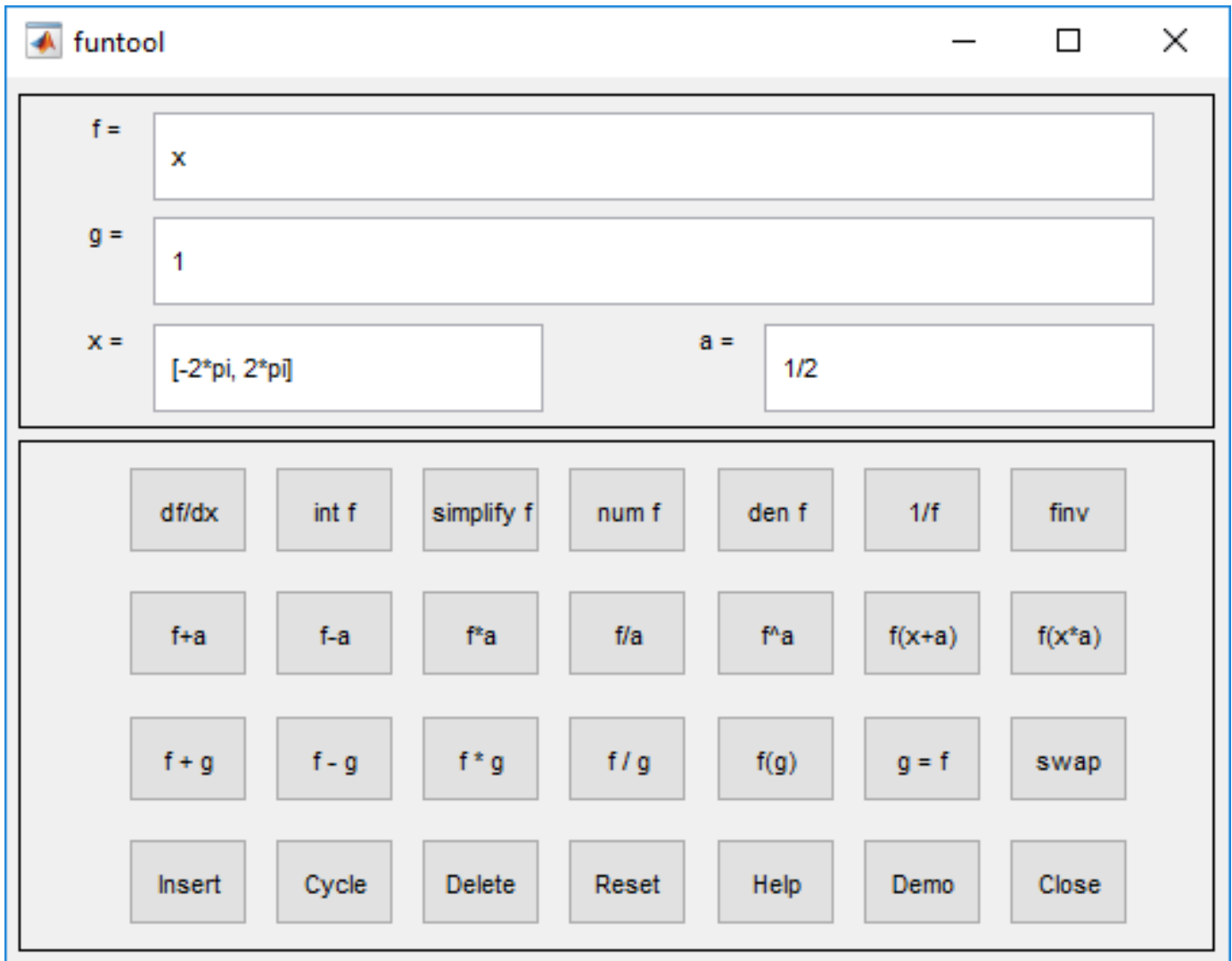
Function calculator

### Description

The `funtool` app is a visual function calculator that manipulates and displays functions of one variable. At the click of a button, for example, `funtool` draws a graph representing the sum, product, difference, or ratio of two functions that you specify. `funtool` includes a function memory that allows you to store functions for later retrieval.

### Open the funtool App

At startup, `funtool` displays graphs of a pair of functions,  $f(x) = x$  and  $g(x) = 1$ . The graphs plot the functions over the domain  $[-2\pi, 2\pi]$ . `funtool` also displays a control panel that lets you save, retrieve, redefine, combine, and transform  $f$  and  $g$ .



The top of the control panel contains a group of editable text fields.

|           |                                                                                                                                                          |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>f=</b> | Displays a symbolic expression representing $f$ . Edit this field to redefine $f$ .                                                                      |
| <b>g=</b> | Displays a symbolic expression representing $g$ . Edit this field to redefine $g$ .                                                                      |
| <b>x=</b> | Displays the domain used to plot $f$ and $g$ . Edit this field to specify a different domain.                                                            |
| <b>a=</b> | Displays a constant factor used to modify $f$ (see button descriptions in the next section). Edit this field to change the value of the constant factor. |

`funtool` redraws  $f$  and  $g$  to reflect any changes you make to the contents of the control panel's text fields.

The bottom part of the control panel contains an array of buttons that transform  $f$  and perform other operations.

The first row of control buttons replaces  $f$  with various transformations of  $f$ .

|                                |                                      |
|--------------------------------|--------------------------------------|
| <b>df/dx</b>                   | Derivative of $f$                    |
| <b>int <math>f</math></b>      | Integral of $f$                      |
| <b>simplify <math>f</math></b> | Simplified form of $f$ , if possible |
| <b>num <math>f</math></b>      | Numerator of $f$                     |
| <b>den <math>f</math></b>      | Denominator of $f$                   |
| <b>1/<math>f</math></b>        | Reciprocal of $f$                    |



|             |                                                                                                                                                                                                                           |                                                       |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| <b>finv</b> | Inverse of $f$                                                                                                                                                                                                            |                                                       |
|             | The operators <b>int f</b> and <b>finv</b> can fail if the corresponding symbolic expressions do not exist in closed form.                                                                                                |                                                       |
|             | The second row of buttons translates and scales $f$ and the domain of $f$ by a constant factor. To specify the factor, enter its value in the field labeled <b>a=</b> on the calculator control panel. The operations are |                                                       |
|             | <b>f+a</b>                                                                                                                                                                                                                | Replaces $f(x)$ by $f(x) + a$ .                       |
|             | <b>f-a</b>                                                                                                                                                                                                                | Replaces $f(x)$ by $f(x) - a$ .                       |
|             | <b>f*a</b>                                                                                                                                                                                                                | Replaces $f(x)$ by $f(x) * a$ .                       |
|             | <b>f/a</b>                                                                                                                                                                                                                | Replaces $f(x)$ by $f(x) / a$ .                       |
|             | <b>f^a</b>                                                                                                                                                                                                                | Replaces $f(x)$ by $f(x) ^ a$ .                       |
|             | <b>f(x+a)</b>                                                                                                                                                                                                             | Replaces $f(x)$ by $f(x + a)$ .                       |
|             | <b>f(x*a)</b>                                                                                                                                                                                                             | Replaces $f(x)$ by $f(x * a)$ .                       |
|             | The first four buttons of the third row replace $f$ with a combination of $f$ and $g$ .                                                                                                                                   |                                                       |
|             | <b>f+g</b>                                                                                                                                                                                                                | Replaces $f(x)$ by $f(x) + g(x)$ .                    |
|             | <b>f-g</b>                                                                                                                                                                                                                | Replaces $f(x)$ by $f(x) - g(x)$ .                    |
|             | <b>f*g</b>                                                                                                                                                                                                                | Replaces $f(x)$ by $f(x) * g(x)$ .                    |
|             | <b>f/g</b>                                                                                                                                                                                                                | Replaces $f(x)$ by $f(x) / g(x)$ .                    |
|             | The remaining buttons on the third row interchange $f$ and $g$ .                                                                                                                                                          |                                                       |
|             | <b>g=f</b>                                                                                                                                                                                                                | Replaces $g$ with $f$ .                               |
|             | <b>swap</b>                                                                                                                                                                                                               | Replaces $f$ with $g$ and $g$ with $f$ .              |
|             | The first three buttons in the fourth row allow you to store and retrieve functions from the calculator's function memory.                                                                                                |                                                       |
|             | <b>Insert</b>                                                                                                                                                                                                             | Adds $f$ to the end of the list of stored functions.  |
|             | <b>Cycle</b>                                                                                                                                                                                                              | Replaces $f$ with the next item on the function list. |
|             | <b>Delete</b>                                                                                                                                                                                                             | Deletes $f$ from the list of stored functions.        |
|             | The other four buttons on the fourth row perform miscellaneous functions:                                                                                                                                                 |                                                       |
|             | <b>Reset</b>                                                                                                                                                                                                              | Resets the calculator to its initial state.           |
|             | <b>Help</b>                                                                                                                                                                                                               | Displays the online help for the calculator.          |
|             | <b>Demo</b>                                                                                                                                                                                                               | Runs a short demo of the calculator.                  |
|             | <b>Close</b>                                                                                                                                                                                                              | Closes the calculator's windows.                      |

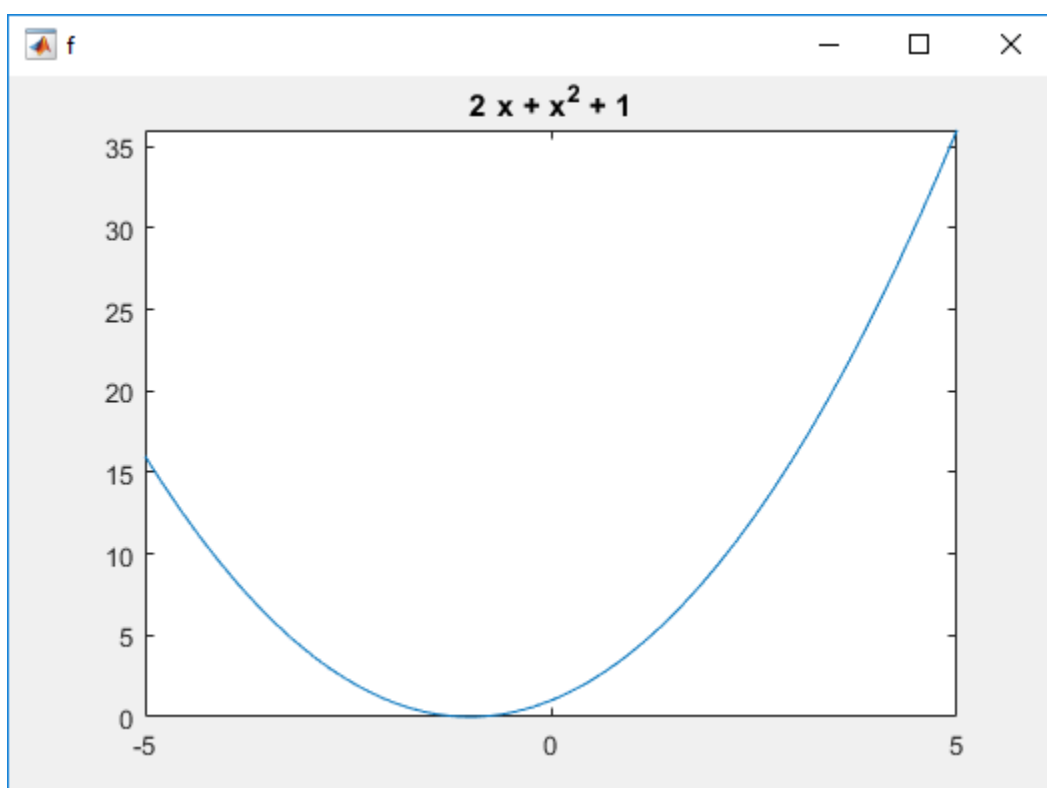
## Examples

**Display Quadratic Function and Its Derivative**

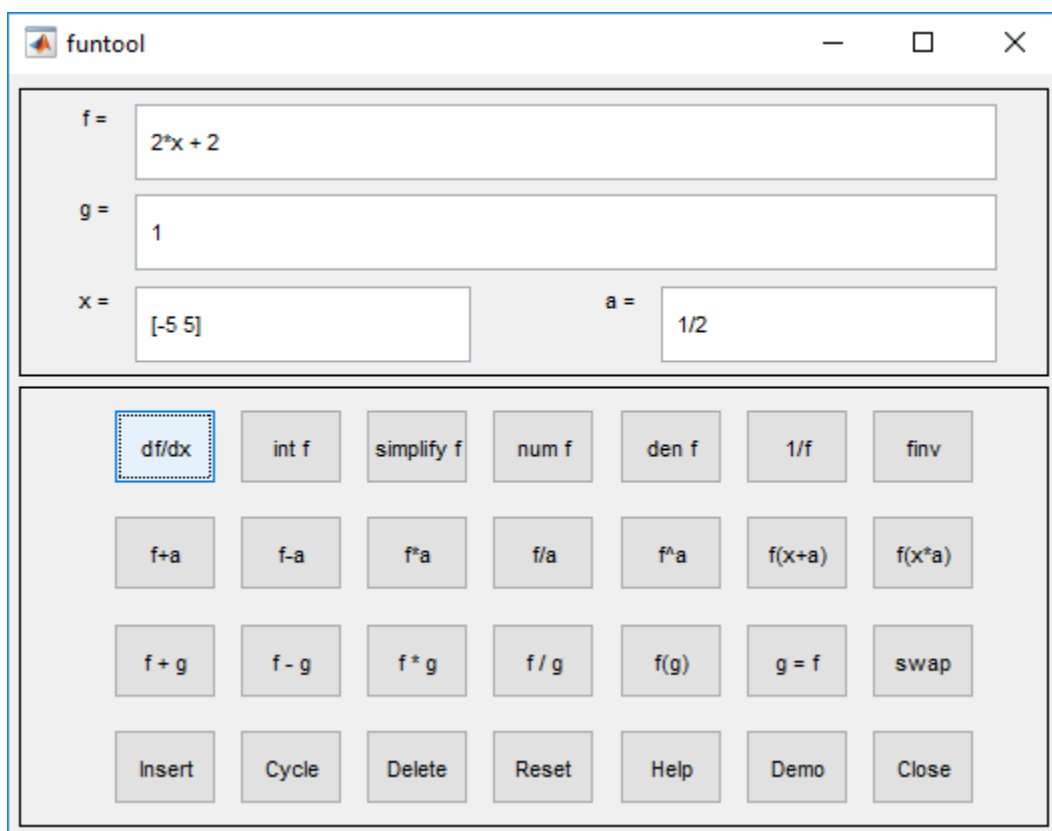
Display a quadratic function by editing the text field  $f = x^2 + 2x + 1$ . Plot the quadratic function within the interval  $x = [-5 \ 5]$ .

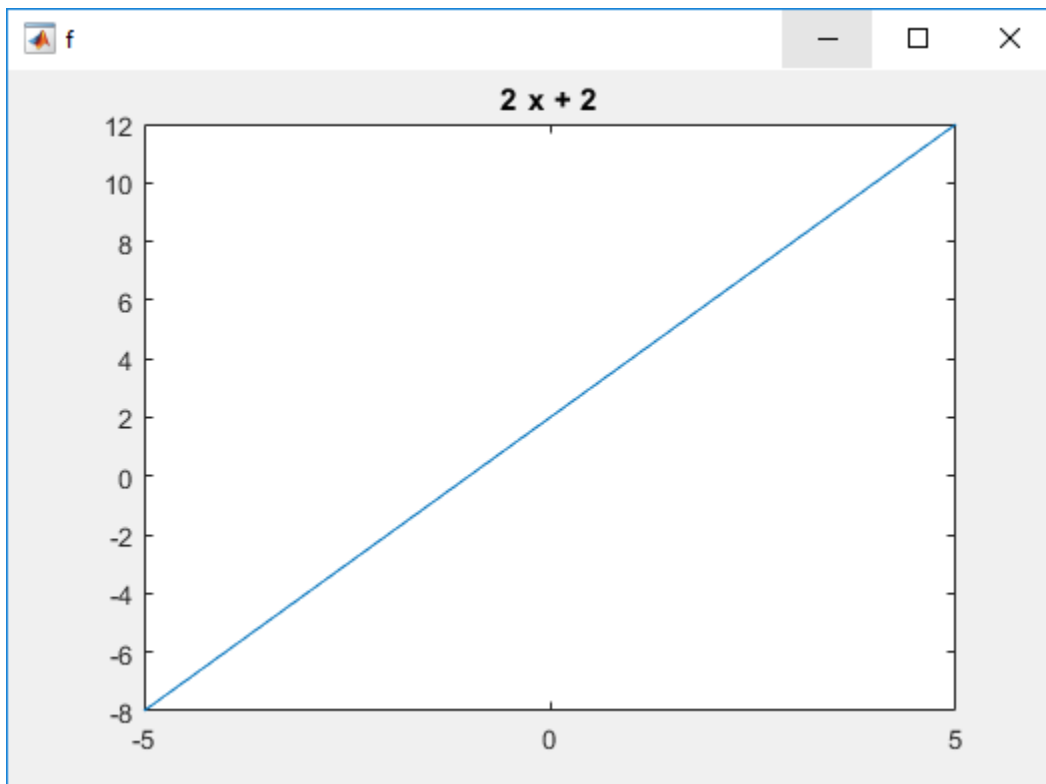
The screenshot shows the 'funtool' software window. It has a title bar with a small icon, the text 'funtool', and standard window controls (minimize, maximize, close). The main area is divided into two sections. The top section contains three text input fields: 'f =' with the value  $x^2 + 2x + 1$ , 'g =' with the value 1, and 'x =' with the value  $[-5 \ 5]$ . To the right of the 'x =' field is an 'a =' field with the value  $1/2$ . The bottom section is a grid of buttons for various mathematical operations. The buttons are arranged in four rows and seven columns. The first row contains: df/dx, int f, simplify f, num f, den f, 1/f, and finv. The second row contains: f+a, f-a, f\*a, f/a, f^a, f(x+a), and f(x\*a). The third row contains: f + g, f - g, f \* g, f / g, f(g), g = f, and swap. The fourth row contains: Insert, Cycle, Delete, Reset, Help, Demo, and Close.

|                                                         |                |  |     |     |  |  |
|---------------------------------------------------------|----------------|--|-----|-----|--|--|
| f =                                                     | $x^2 + 2x + 1$ |  |     |     |  |  |
| g =                                                     | 1              |  |     |     |  |  |
| x =                                                     | [-5 5]         |  | a = | 1/2 |  |  |
| df/dx   int f   simplify f   num f   den f   1/f   finv |                |  |     |     |  |  |
| f+a   f-a   f*a   f/a   f^a   f(x+a)   f(x*a)           |                |  |     |     |  |  |
| f + g   f - g   f * g   f / g   f(g)   g = f   swap     |                |  |     |     |  |  |
| Insert   Cycle   Delete   Reset   Help   Demo   Close   |                |  |     |     |  |  |



Click **df/dx** to find the derivative of the quadratic function.





## Version History

Introduced before R2006a

## See Also

fplot | syms | taylor tool | rsums

## gamma

Gamma function

### Syntax

`gamma(X)`

### Description

`gamma(X)` returns the gamma function on page 6-712 of a symbolic variable or symbolic expression `X`.

### Examples

#### Gamma Function for Numeric and Symbolic Arguments

Depending on its arguments, `gamma` returns floating-point or exact symbolic results.

Compute the gamma function for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
A = gamma([-11/3, -7/5, -1/2, 1/3, 1, 4])  
  
A =  
    0.2466    2.6593   -3.5449    2.6789    1.0000    6.0000
```

Compute the gamma function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `gamma` returns unresolved symbolic calls.

```
symA = gamma(sym([-11/3, -7/5, -1/2, 1/3, 1, 4]))  
  
symA =  
[ (27*pi*3^(1/2))/(440*gamma(2/3)), gamma(-7/5), ...  
 -2*pi^(1/2), (2*pi*3^(1/2))/(3*gamma(2/3)), 1, 6]
```

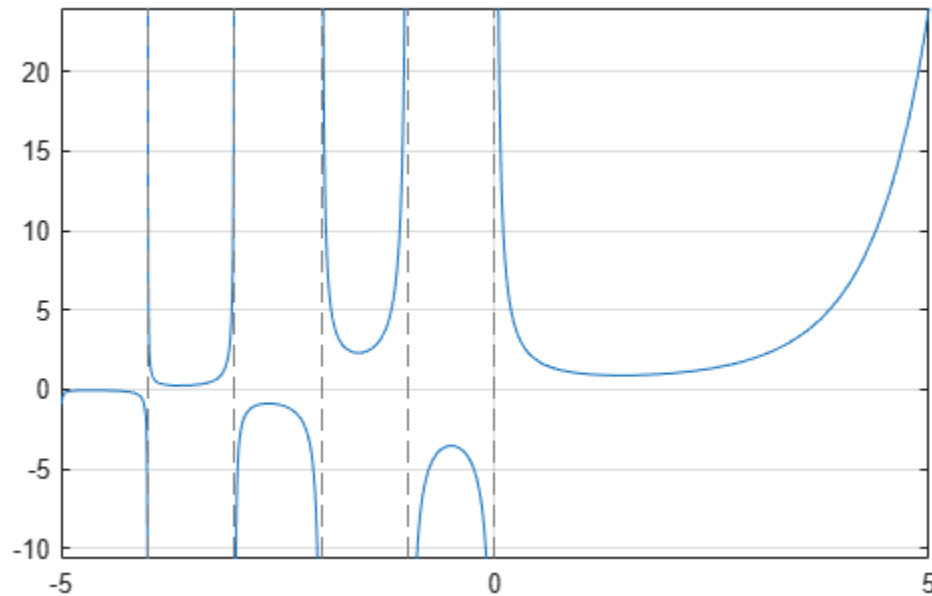
Use `vpa` to approximate symbolic results with floating-point numbers:

```
vpa(symA)  
  
ans =  
[ 0.24658411512650858900694446388517, ...  
 2.6592718728800305399898810505738, ...  
 -3.5449077018110320545963349666823, ...  
 2.6789385347077476336556929409747, ...  
 1.0, 6.0]
```

#### Plot Gamma Function

Plot the gamma function and add grid lines.

```
syms x
fplot(gamma(x))
grid on
```



### Handle Expressions Containing Gamma Function

Many functions, such as `diff`, `limit`, and `simplify`, can handle expressions containing gamma.

Differentiate the gamma function, and then substitute the variable  $t$  with the value 1:

```
syms t
u = diff(gamma(t^3 + 1))
u1 = subs(u, t, 1)

u =
3*t^2*gamma(t^3 + 1)*psi(t^3 + 1)

u1 =
3 - 3*eulergamma
```

Approximate the result using `vpa`:

```
vpa(u1)

ans =
1.2683530052954014181804637297528
```

Compute the limit of the following expression that involves the gamma function:

```
syms x
limit(x/gamma(x), x, inf)

ans =
0
```

Simplify the following expression:

```
syms x
simplify(gamma(x)*gamma(1 - x))

ans =
pi/sin(pi*x)
```

## Input Arguments

### X — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as symbolic number, variable, expression, function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## More About

### Gamma Function

The following integral defines the gamma function:

$$\Gamma(z) = \int_0^{\infty} t^{z-1} e^{-t} dt.$$

## Version History

Introduced before R2006a

### See Also

beta | factorial | igamma | gamma1n | nchoosek | pochhammer | psi



# gammaln

Logarithmic gamma function

## Syntax

`gammaln(X)`

## Description

`gammaln(X)` returns the logarithmic gamma function for each element of X.

## Examples

### Logarithmic Gamma Function for Numeric and Symbolic Arguments

Depending on its arguments, `gammaln` returns floating-point or exact symbolic results.

Compute the logarithmic gamma function for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
A = gammaln([1/5, 1/2, 2/3, 8/7, 3])
```

```
A =  
    1.5241    0.5724    0.3032   -0.0667    0.6931
```

Compute the logarithmic gamma function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `gammaln` returns results in terms of the `gammaln`, `log`, and `gamma` functions.

```
symA = gammaln(sym([1/5, 1/2, 2/3, 8/7, 3]))
```

```
symA =  
[ gammaln(1/5), log(pi^(1/2)), gammaln(2/3),...  
log(gamma(1/7)/7), log(2)]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

```
vpa(symA)
```

```
ans =  
[ 1.5240638224307845248810564939263,...  
0.57236494292470008707171367567653,...  
0.30315027514752356867586281737201,...  
-0.066740877459477468649396334098109,...  
0.69314718055994530941723212145818]
```

### Definition of the Logarithmic Gamma Function on Complex Plane

`gammaln` is defined for all complex arguments, except negative infinity.

Compute the logarithmic gamma function for positive integer arguments. For such arguments, the logarithmic gamma function is defined as the natural logarithm of the gamma function, `gammaln(x) = log(gamma(x))`.

```
pos = gammaln(sym([1/4, 1/3, 1, 5, Inf]))
```

```
pos =  
[ log((pi*2^(1/2))/gamma(3/4)), log((2*pi*3^(1/2))/(3*gamma(2/3))), 0, log(24), Inf]
```

Compute the logarithmic gamma function for nonpositive integer arguments. For nonpositive integers, `gammaln` returns `Inf`.

```
nonposint = gammaln(sym([0, -1, -2, -5, -10]))
```

```
nonposint =  
[ Inf, Inf, Inf, Inf, Inf]
```

Compute the logarithmic gamma function for complex and negative rational arguments. For these arguments, `gammaln` returns unresolved symbolic calls.

```
complex = gammaln(sym([i, -1 + 2*i, -2/3, -10/3]))
```

```
complex =  
[ gammaln(1i), gammaln(- 1 + 2i), gammaln(-2/3), gammaln(-10/3)]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

```
vpa(complex)
```

```
ans =  
[ - 0.65092319930185633888521683150395 - 1.8724366472624298171188533494366i,...  
 - 3.3739449232079248379476073664725 - 3.4755939462808110432931921583558i,...  
 1.3908857550359314511651871524423 - 3.1415926535897932384626433832795i,...  
 - 0.93719017334928727370096467598178 - 12.566370614359172953850573533118i]
```

Compute the logarithmic gamma function of negative infinity:

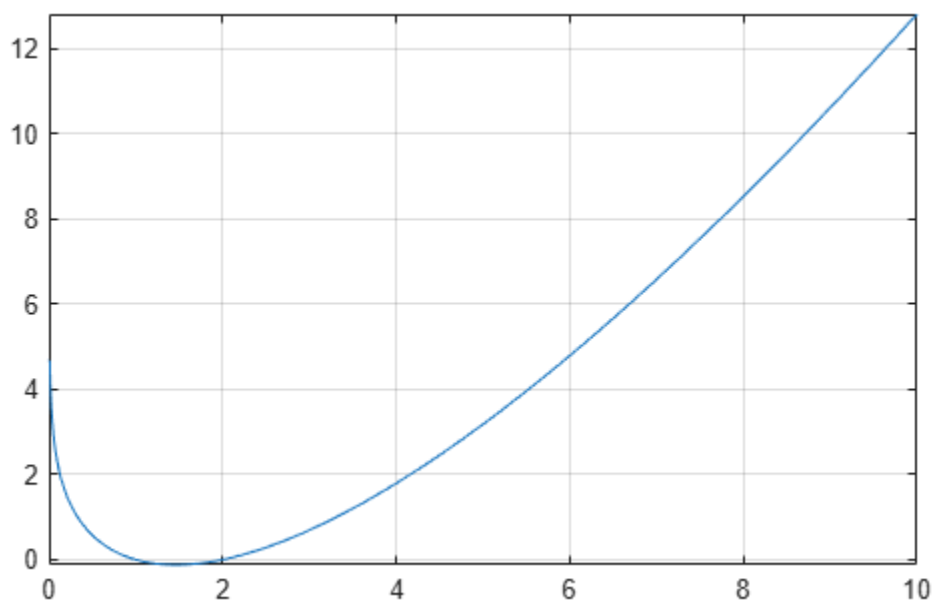
```
gammaln(sym(-Inf))
```

```
ans =  
NaN
```

### Plot Logarithmic Gamma Function

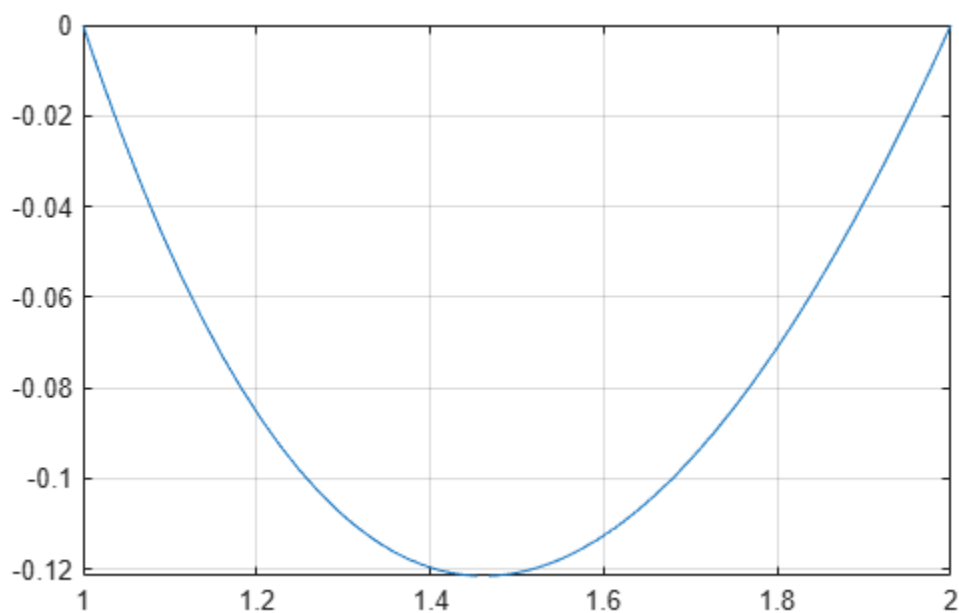
Plot the logarithmic gamma function on the interval from 0 to 10.

```
syms x  
fplot(gammaln(x),[0 10])  
grid on
```



To see the negative values better, plot the same function on the interval from 1 to 2.

```
fplot(gammaln(x), [1 2])
grid on
```



### Handle Expressions Containing Logarithmic Gamma Function

Many functions, such as `diff` and `limit`, can handle expressions containing `lngamma`.

Differentiate the logarithmic gamma function:

```
syms x
diff(gammain(x), x)
```

```
ans =
psi(x)
```

Compute the limits of these expressions containing the logarithmic gamma function:

```
syms x
limit(1/gammain(x), x, Inf)
```

```
ans =
0
```

```
limit(gammain(x - 1) - gammain(x - 2), x, 0)
```

```
ans =
log(2) + pi*1i
```

## Input Arguments

### X — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as symbolic number, variable, expression, function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Algorithms

For single or double input to `gammain(x)`,  $x$  must be real and positive.

For symbolic input,

- `gammain(x)` is defined for all complex  $x$  except the singular points  $0, -1, -2, \dots$ .
- For positive real  $x$ , `gammain(x)` represents the logarithmic gamma function  $\log(\text{gamma}(x))$ .
- For negative real  $x$  or for complex  $x$ ,  $\text{gammain}(x) = \log(\text{gamma}(x)) + f(x)2\pi i$  where  $f(x)$  is some integer valued function. The integer multiples of  $2\pi i$  are chosen such that `gammain(x)` is analytic throughout the complex plane with a branch cut along the negative real semi axis.
- For negative real  $x$ , `gammain(x)` is equal to the limit of  $\log(\text{gamma}(x))$  from 'above'.

## Version History

Introduced in R2014a

### See Also

`beta` | `gamma` | `log` | `nchoosek` | `psi`

# gcd

Greatest common divisor of symbolic numbers and polynomials

## Syntax

```
G = gcd(A)
G = gcd(A,B)
[G,M] = gcd(A)
[G,U,V] = gcd(A,B,X)
```

## Description

$G = \text{gcd}(A)$  finds the greatest common divisor (GCD) of all symbolic elements in  $A$ .

$G = \text{gcd}(A,B)$  finds the greatest common divisor of  $A$  and  $B$ .

$[G,M] = \text{gcd}(A)$  also returns the coefficients  $M$  for the linear combination of elements in  $A$  that results in  $G$ .

$[G,U,V] = \text{gcd}(A,B,X)$  also returns the Bézout coefficients,  $U$  and  $V$ , such that  $G = A*U + B*V$ . For multivariate polynomials, specify the polynomial variable  $X$  such that it does not appear in the denominator of  $U$  and  $V$ . If you do not specify  $X$ , then `gcd` uses the default variable determined by `symvar`.

## Examples

### Greatest Common Divisor of Symbolic Integers

Find the greatest common divisor of four integers, specified as elements of a symbolic vector.

```
A = sym([4420 -128 8984 -488]);
G = gcd(A)

G = 4
```

Alternatively, specify these values as elements of a symbolic matrix.

```
A = sym([4420 -128; 8984 -488]);
G = gcd(A)

G = 4
```

### Greatest Common Divisor of Rational Numbers

The greatest common divisor of rational numbers  $a_1, a_2, \dots, a_n$  is a number  $g$ , such that  $a_1/g, a_2/g, \dots, a_n/g$  are integers, and  $\text{gcd}(a_1/g, a_2/g, \dots, a_n/g) = 1$ .

Find the greatest common divisor of five rational numbers.

```
A = sym([1/4 1/3 1/2 2/3 3/4])
```

```
A =  
 $\begin{pmatrix} \frac{1}{4} & \frac{1}{3} & \frac{1}{2} & \frac{2}{3} & \frac{3}{4} \end{pmatrix}$ 
```

```
G = gcd(A)
```

```
G =  
 $\frac{1}{12}$ 
```

### Greatest Common Divisor of Complex Numbers

`gcd` computes the greatest common divisor of complex numbers over the Gaussian integers (complex numbers with real and imaginary parts that are integers). It returns a complex number with a positive real part and a nonnegative imaginary part.

Find the greatest common divisor of three complex numbers.

```
A = sym([3 + 1i, 4 - 2i, -5 - 5i]);  
G = gcd(A)
```

```
G = 3 + i
```

### Greatest Common Divisor of Elements of Matrices

If you specify two input arguments and at least one of them is nonscalar, `gcd` finds the greatest common divisors element-wise.

Find the greatest common divisors for the elements of two matrices. When you specify two nonscalar input arguments, they must be the same size.

```
A = sym([309 186; 486 224]);  
B = sym([558 444; 1024 1984]);  
G = gcd(A,B)
```

```
G =  
 $\begin{pmatrix} 3 & 6 \\ 2 & 32 \end{pmatrix}$ 
```

Find the greatest common divisors for the elements of matrix `A` and the value `200`. Here, `gcd` expands `200` into a 2-by-2 matrix with all elements equal to `200`.

```
G = gcd(A,200)
```

```
G =  
 $\begin{pmatrix} 1 & 2 \\ 2 & 8 \end{pmatrix}$ 
```

## Greatest Common Divisor of Polynomials

Find the greatest common divisor of two univariate polynomials.

```
syms x
A = x^3 - 3*x^2 + 3*x - 1;
B = x^2 - 5*x + 4;
G = gcd(A,B)

G = x - 1
```

Find the greatest common divisor of three multivariate polynomials. Because there are more than two polynomials, specify them as elements of a symbolic vector.

```
syms x y
A = [x^2*y + x^3, (x + y)^2, x^2 + x*y^2 + x*y + x + y^3 + y];
G = gcd(A)

G = x + y
```

## Linear Combination of Inputs

A theorem in number theory states that the GCD of two numbers is the smallest positive linear combination of those numbers. Show that the GCD of 64 and 44 is a positive linear combination of those numbers.

```
A = sym([64 44]);
[G,M] = gcd(A)

G = 4

M = (-2 3)

tf = isequal(G,sum(M.*A))

tf = logical
    1
```

## Bézout Coefficients

Find the greatest common divisor and the Bézout coefficients of two polynomials with respect to variable  $x$ . When computing Bézout coefficients, `gcd` ensures that the polynomial variable does not appear in the denominators of these coefficients.

```
syms x y
[G,U,V] = gcd(x^2*y + x^3, (x + y)^2, x)

G = x + y

U =
    1
   y^2
```

$$V = \frac{1}{y} - \frac{x}{y^2}$$

Find the greatest common divisor and the Bézout coefficients of the same polynomials with respect to variable  $y$ .

$$[G,U,V] = \text{gcd}(x^2y + x^3, (x + y)^2, y)$$

$$G = x + y$$

$$U = \frac{1}{x^2}$$

$$V = 0$$

If you do not specify the polynomial variable, then the toolbox uses `symvar` to determine the variable.

$$[G,U,V] = \text{gcd}(x^2y + x^3, (x + y)^2)$$

$$G = x + y$$

$$U = \frac{1}{y^2}$$

$$V = \frac{1}{y} - \frac{x}{y^2}$$

### Solve Diophantine Equation

Solve the Diophantine equation  $(1 - z)x + z(1 - 2z)y = 3$  for  $x$  and  $y$ .

Start by defining the coefficients for  $x$  and  $y$ , and the right side of this Diophantine equation.

```
syms z
A = 1 - z;
B = z*(1 - 2*z);
R = 3;
```

Use the syntax  $[G,U,V] = \text{gcd}(A,B)$  to find the greatest common divisor  $G$  of the polynomials  $A$  and  $B$  and the Bézout coefficients  $U$  and  $V$ , such that  $G = A*U + B*V$ .

$$[G,U,V] = \text{gcd}(A,B)$$

$$G = 1$$

$$U = 2z + 1$$

$$V = -1$$

If the GCD divides the right side of the equation, or in other words,  $R/G$  is an integer, then the Diophantine equation has a solution. Use the Bézout coefficients  $U$  and  $V$  to compute one particular solution and the general solution for the Diophantine equation.



```

if mod(R,G) == 0
    S = R/G;
    disp("Particular solution:")
    x1 = U*S
    y1 = V*S
    disp("General solution:")
    syms k integer
    x = x1 + B/G*k
    y = y1 - A/G*k
    params = assumptions(k)
else
    disp("This Diophantine equation does not have a solution.")
end

```

Particular solution:

$$x_1 = 6z + 3$$

$$y_1 = -3$$

General solution:

$$x = 6z - kz(2z - 1) + 3$$

$$y = k(z - 1) - 3$$

$$\text{params} = k \in \mathbb{Z}$$

Finally, confirm that the general solution satisfies the Diophantine equation.

$$\text{eqCheck} = \text{simplify}(A*x + B*y) == R$$

$$\text{eqCheck} = 3 = 3$$

## Input Arguments

### A — Input value

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input value, specified as a symbolic number, variable, expression, function, or a vector or matrix of symbolic numbers, variables, expressions, or functions.

### B — Input value

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input value, specified as a symbolic number, variable, expression, function, or a vector or matrix of symbolic numbers, variables, expressions, or functions.

If A and B are nonscalar, they must have the same size. If either A or B is scalar, then gcd expands the scalar into a vector or matrix of the same size as the nonscalar argument, with all elements equal to the corresponding scalar.

### X — Polynomial variable

symbolic variable

Polynomial variable, specified as a symbolic variable. By default, `gcd` uses the variable determined by `symvar`.

## Output Arguments

### **G — Greatest common divisor**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Greatest common divisor, returned as a symbolic number, variable, expression, function, or a vector or matrix of symbolic numbers, variables, expressions, or functions.

### **M — Coefficients for linear combination of elements in input A**

symbolic vector | symbolic matrix | symbolic array

Coefficients for the linear combination of elements in input A that equals GCD of input, returned as a symbolic vector, matrix, or array.

### **U, V — Bézout coefficients**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Bézout coefficients, returned as symbolic numbers, variables, expressions, functions, or vectors or matrices of symbolic numbers, variables, expressions, or functions.

## Tips

- Calling `gcd` with numbers that are not symbolic objects invokes the MATLAB `gcd` function.
- The MATLAB `gcd` function does not accept rational or complex arguments. To find the greatest common divisor of rational or complex numbers, convert these numbers to symbolic objects by using `sym`, and then use `gcd`.

## Version History

**Introduced in R2014b**

## See Also

`lcm`

## ge

Define greater than or equal to condition

### Syntax

```
A >= B
ge(A,B)
```

### Description

$A \geq B$  creates the condition greater than or equal.

`ge(A,B)` is equivalent to  $A \geq B$ .

### Examples

#### Set and Use Assumption Using Greater Than Equal To

Set the assumption that  $x$  is greater than or equal to 3 by using `assume`.

```
syms x
assume(x >= 3)
```

Solve this equation involving  $x$ . The solver only returns solutions that are valid under the assumption on  $x$ .

```
eqn = (x-1)*(x-2)*(x-3)*(x-4) == 0;
solve(eqn,x)
```

```
ans =
     3
     4
```

#### Find Values that Satisfy Condition

Set the condition `abs(sin(x)) >= 1/2`.

```
syms x
cond = abs(sin(x)) >= 1/2;
```

Find multiples of  $\pi/24$  that satisfy the condition by using a `for` loop from 0 to  $\pi$ .

```
for i = 0:sym(pi)/12:sym(pi)
    if subs(cond,x,i)
        disp(i)
    end
end
```

```
pi/6
pi/4
pi/3
(5*pi)/12
```

```
pi/2  
(7*pi)/12  
(2*pi)/3  
(3*pi)/4  
(5*pi)/6
```

## Input Arguments

### A — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

### B — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## Tips

- Calling `>=` or `ge` for non-symbolic A and B invokes the MATLAB `ge` function. This function returns a logical array with elements set to logical 1 (`true`) where A is greater than or equal to B; otherwise, it returns logical 0 (`false`).
- If both A and B are arrays, then these arrays must have the same dimensions. `A >= B` returns an array of relations `A(i,j,...) >= B(i,j,...)`
- If one input is scalar and the other an array, then the scalar input is expanded into an array of the same dimensions as the other array.
- The field of complex numbers is not an ordered field. MATLAB projects complex numbers in relations to a real axis. For example, `x >= i` becomes `x >= 0`, and `x >= 3+2*i` becomes `x >= 3`.

## Version History

Introduced in R2012a

### See Also

`eq` | `gt` | `isAlways` | `le` | `lt` | `ne`

### Topics

“Use Assumptions on Symbolic Variables” on page 1-41

# gegenbauerC

Gegenbauer polynomials

## Syntax

`gegenbauerC(n,a,x)`

## Description

`gegenbauerC(n,a,x)` represents the  $n$ th-degree Gegenbauer (ultraspherical) polynomial on page 6-727 with parameter  $a$  at the point  $x$ .

## Examples

### First Four Gegenbauer Polynomials

Find the first four Gegenbauer polynomials for the parameter  $a$  and variable  $x$ .

```
syms a x
gegenbauerC([0, 1, 2, 3], a, x)

ans =
[ 1, 2*a*x, (2*a^2 + 2*a)*x^2 - a, ...
((4*a^3)/3 + 4*a^2 + (8*a)/3)*x^3 + (- 2*a^2 - 2*a)*x]
```

### Gegenbauer Polynomials for Numeric and Symbolic Arguments

Depending on its arguments, `gegenbauerC` returns floating-point or exact symbolic results.

Find the value of the fifth-degree Gegenbauer polynomial for the parameter  $a = 1/3$  at these points. Because these numbers are not symbolic objects, `gegenbauerC` returns floating-point results.

```
gegenbauerC(5, 1/3, [1/6, 1/4, 1/3, 1/2, 2/3, 3/4])

ans =
    0.1520    0.1911    0.1914    0.0672   -0.1483   -0.2188
```

Find the value of the fifth-degree Gegenbauer polynomial for the same numbers converted to symbolic objects. For symbolic numbers, `gegenbauerC` returns exact symbolic results.

```
gegenbauerC(5, 1/3, sym([1/6, 1/4, 1/3, 1/2, 2/3, 3/4]))

ans =
[ 26929/177147, 4459/23328, 33908/177147, 49/729, -26264/177147, -7/32]
```

### Evaluate Chebyshev Polynomials with Floating-Point Numbers

Floating-point evaluation of Gegenbauer polynomials by direct calls of `gegenbauerC` is numerically stable. However, first computing the polynomial using a symbolic variable, and then substituting variable-precision values into this expression can be numerically unstable.

Find the value of the 500th-degree Gegenbauer polynomial for the parameter 4 at  $1/3$  and `vpa(1/3)`. Floating-point evaluation is numerically stable.

```
gegenbauerC(500, 4, 1/3)
gegenbauerC(500, 4, vpa(1/3))
```

```
ans =
    -1.9161e+05
```

```
ans =
-191609.10250897532784888518393655
```

Now, find the symbolic polynomial  $C500 = \text{gegenbauerC}(500, 4, x)$ , and substitute  $x = \text{vpa}(1/3)$  into the result. This approach is numerically unstable.

```
syms x
C500 = gegenbauerC(500, 4, x);
subs(C500, x, vpa(1/3))
```

```
ans =
-8.0178726380235741521208852037291e+35
```

Approximate the polynomial coefficients by using `vpa`, and then substitute  $x = \text{sym}(1/3)$  into the result. This approach is also numerically unstable.

```
subs(vpa(C500), x, sym(1/3))
```

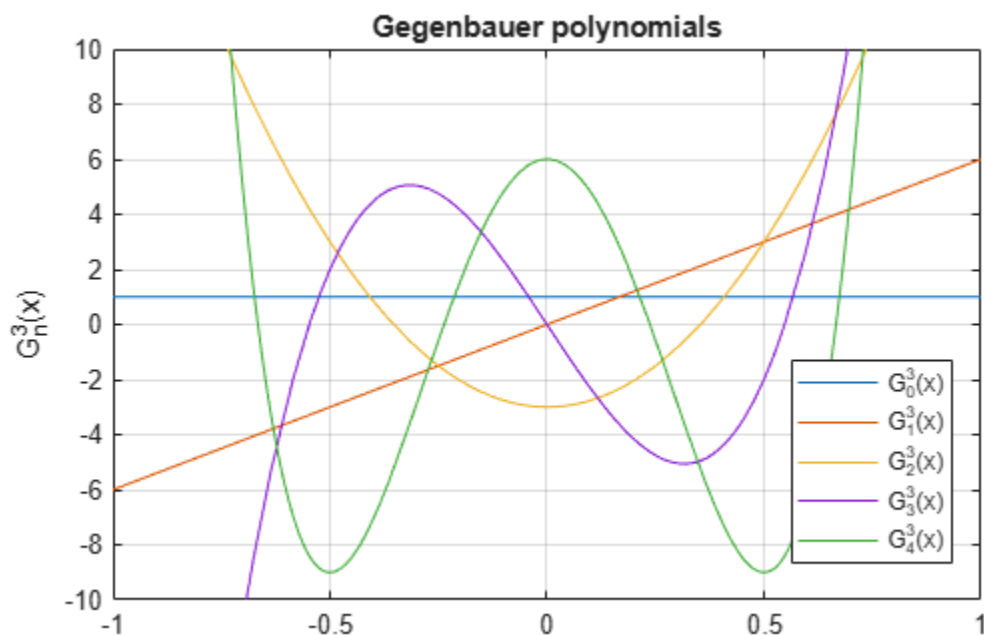
```
ans =
-8.1125412405858470246887213923167e+36
```

### Plot Gegenbauer Polynomials

Plot the first five Gegenbauer polynomials for the parameter  $a = 3$ .

```
syms x y
fplot(gegenbauerC(0:4,3,x))
axis([-1 1 -10 10])
grid on

ylabel('G_n^3(x)')
title('Gegenbauer polynomials')
legend('G_0^3(x)', 'G_1^3(x)', 'G_2^3(x)', 'G_3^3(x)', 'G_4^3(x)', ...
       'Location', 'Best')
```



## Input Arguments

### **n** — Degree of polynomial

nonnegative integer | symbolic variable | symbolic expression | symbolic function | vector | matrix

Degree of the polynomial, specified as a nonnegative integer, symbolic variable, expression, or function, or as a vector or matrix of numbers, symbolic numbers, variables, expressions, or functions.

### **a** — Parameter

number | symbolic number | symbolic variable | symbolic expression | symbolic function | vector | matrix

Parameter, specified as a nonnegative integer, symbolic variable, expression, or function, or as a vector or matrix of numbers, symbolic numbers, variables, expressions, or functions.

### **x** — Evaluation point

number | symbolic number | symbolic variable | symbolic expression | symbolic function | vector | matrix

Evaluation point, specified as a number, symbolic number, variable, expression, or function, or as a vector or matrix of numbers, symbolic numbers, variables, expressions, or functions.

## More About

### Gegenbauer Polynomials

- Gegenbauer polynomials are defined by this recursion formula.

$$G(0, a, x) = 1, \quad G(1, a, x) = 2ax, \quad G(n, a, x) = \frac{2x(n+a-1)}{n}G(n-1, a, x) - \frac{n+2a-2}{n}G(n-2, a, x)$$

- For all real  $a > -1/2$ , Gegenbauer polynomials are orthogonal on the interval  $-1 \leq x \leq 1$  with respect to the weight function  $w(x) = (1 - x^2)^{a - 1/2}$ .

$$\int_{-1}^1 G(n, a, x) G(m, a, x) (1 - x^2)^{a - 1/2} dx = \begin{cases} 0 & \text{if } n \neq m \\ \frac{\pi 2^{1-2a} \Gamma(n+2a)}{n!(n+a)(\Gamma(a))^2} & \text{if } n = m. \end{cases}$$

- Chebyshev polynomials of the first and second kinds are special cases of the Gegenbauer polynomials.

$$T(n, x) = \begin{cases} \frac{1}{2} \lim_{a \rightarrow 0} \frac{n+a}{a} G(n, a, x) & \text{if } n \neq 0 \\ \lim_{a \rightarrow 0} G(0, a, x) = 1 & \text{if } n = 0 \end{cases}$$

$$U(n, x) = G(n, 1, x)$$

- Legendre polynomials are also a special case of the Gegenbauer polynomials.

$$P(n, x) = G\left(n, \frac{1}{2}, x\right)$$

## Tips

- `gegenbauerC` returns floating-point results for numeric arguments that are not symbolic objects.
- `gegenbauerC` acts element-wise on nonscalar inputs.
- All nonscalar arguments must have the same size. If one or two input arguments are nonscalar, then `gegenbauerC` expands the scalars into vectors or matrices of the same size as the nonscalar arguments, with all elements equal to the corresponding scalar.

## Version History

Introduced in R2014b

## References

- [1] Hochstrasser, U. W. "Orthogonal Polynomials." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.
- [2] Cohl, Howard S., and Connor MacKenzie. "Generalizations and Specializations of Generating Functions for Jacobi, Gegenbauer, Chebyshev and Legendre Polynomials with Definite Integrals." *Journal of Classical Analysis*, no. 1 (2013): 17-33. <https://doi.org/10.7153/jca-03-02>.

## See Also

`chebyshevT` | `chebyshevU` | `hermiteH` | `jacobiP` | `laguerreL` | `legendreP`



# gradient

Gradient vector of symbolic scalar field

## Syntax

```
g = gradient(f,v)
g = gradient(f)
```

## Description

`g = gradient(f,v)` returns the gradient vector on page 6-733 of symbolic scalar field `f` with respect to vector `v` in Cartesian coordinates.

`g = gradient(f)` returns the gradient vector of the scalar field `f` with respect to a default vector constructed from the symbolic variables in `f`.

## Examples

### Find Gradient of Function

The gradient of a scalar function `f` with respect to the vector `v` is the vector of the first partial derivatives of `f` with respect to each element of `v`.

Find the gradient vector of `f(x,y,z)` with respect to vector `[x,y,z]`. The gradient is a vector with these components.

```
syms x y z
f(x,y,z) = 2*y*z*sin(x) + 3*x*sin(z)*cos(y);
v = [x,y,z];
gradient(f,v)
```

```
ans(x, y, z) =

$$\begin{pmatrix} 3 \cos(y) \sin(z) + 2 y z \cos(x) \\ 2 z \sin(x) - 3 x \sin(y) \sin(z) \\ 2 y \sin(x) + 3 x \cos(y) \cos(z) \end{pmatrix}$$

```

### Plot Gradient of Function

Find the gradient of a function `f(x,y)`, and plot it as a quiver (velocity) plot.

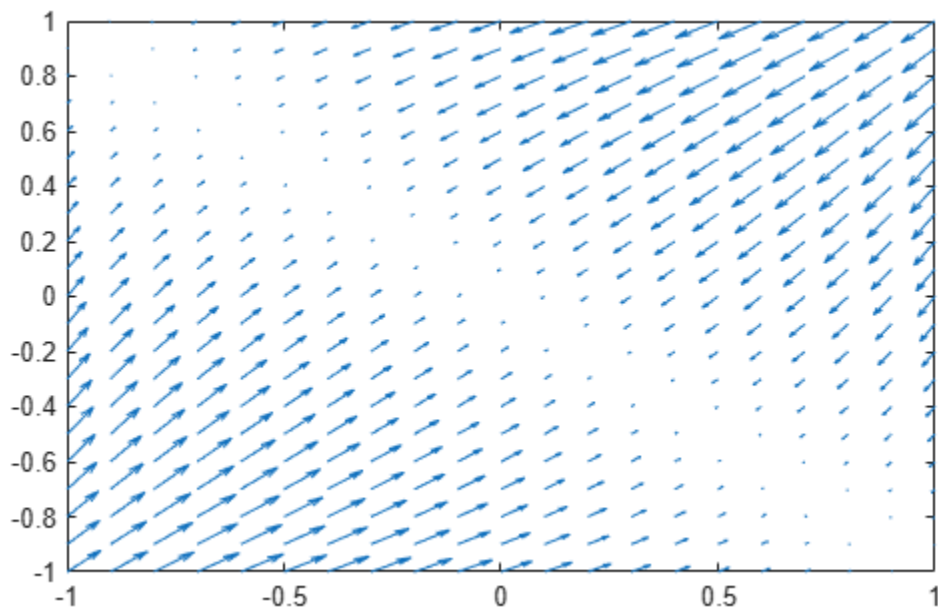
Find the gradient vector of `f(x,y)` with respect to vector `[x,y]`. The gradient is vector `g` with these components.

```
syms x y
f = -(sin(x) + sin(y))^2;
v = [x y];
g = gradient(f,v)
```

$$g = \begin{pmatrix} -2 \cos(x) (\sin(x) + \sin(y)) \\ -2 \cos(y) (\sin(x) + \sin(y)) \end{pmatrix}$$

Now plot the vector field defined by these components. MATLAB® provides the `quiver` plotting function for this task. The function does not accept symbolic arguments. First, replace symbolic variables in expressions for components of `g` with numeric values. Then use `quiver`.

```
[X,Y] = meshgrid(-1:.1:1,-1:.1:1);
G1 = subs(g(1),v,{X,Y});
G2 = subs(g(2),v,{X,Y});
quiver(X,Y,G1,G2)
```



### Gradient of Matrix Multiplication

Use symbolic matrix variables to define a matrix multiplication that returns a scalar.

```
syms X Y [3 1] matrix
A = Y.'*X
```

$$A = Y^T X$$

Find the gradient of the matrix multiplication with respect to `X`.

```
gX = gradient(A,X)
```

$$gX = Y$$

Find the gradient of the matrix multiplication with respect to `Y`.

```
gY = gradient(A,Y)
```

$$gY = X$$

### Gradient of Multivariable Function

Find the gradient of the multivariable function

$$f(x) = \sin^2(x_{1,1}) + \sin^2(x_{1,2}) + \sin^2(x_{1,3})$$

with respect to the vector  $x = [x_{1,1}, x_{1,2}, x_{1,3}]$ .

Use a symbolic matrix variable to express the function  $f$  and its gradient in terms of the vector  $x$ .

```
syms x [1 3] matrix
f = sin(x)*sin(x).'
```

$$f = \sin(x) \sin(x)^T$$

$$g = \text{gradient}(f, x)$$

$$g = 2 (\cos(x) \odot I_3) \sin(x)^T$$

To express the gradient in terms of the elements of  $x$ , convert the result to a vector of symbolic scalar variables using `symmatrix2sym`.

$$g = \text{symmatrix2sym}(g)$$

$$g = \begin{pmatrix} 2 \cos(x_{1,1}) \sin(x_{1,1}) \\ 2 \cos(x_{1,2}) \sin(x_{1,2}) \\ 2 \cos(x_{1,3}) \sin(x_{1,3}) \end{pmatrix}$$

Alternatively, you can convert  $f$  and  $x$  to symbolic expressions of scalar variables and use them as inputs to the `gradient` function.

$$g = \text{gradient}(\text{symmatrix2sym}(f), \text{symmatrix2sym}(x))$$

$$g = \begin{pmatrix} 2 \cos(x_{1,1}) \sin(x_{1,1}) \\ 2 \cos(x_{1,2}) \sin(x_{1,2}) \\ 2 \cos(x_{1,3}) \sin(x_{1,3}) \end{pmatrix}$$

### Find Vector Calculus Identities Involving Gradients

Create a 3-by-1 vector as a symbolic matrix variable  $\mathbf{X}$ . Create a scalar field that is a function of  $\mathbf{X}$  as a symbolic matrix function  $A(\mathbf{X})$ , keeping the existing definition of  $\mathbf{X}$ .

```
syms X [3 1] matrix
syms A(X) [1 1] matrix keepargs
```

Find the gradient of  $A(\mathbf{X})$  with respect to  $\mathbf{X}$ . The `gradient` function returns an unevaluated formula.

```
gradA = gradient(A,X)
```

```
gradA(X) =  $\nabla_X A(X)$ 
```

Show that the divergence of the gradient of  $A(\mathbf{X})$  is equal to the Laplacian of  $A(\mathbf{X})$ , that is  $\nabla_{\mathbf{X}} \cdot \nabla_{\mathbf{X}} A(\mathbf{X}) = \Delta_{\mathbf{X}} A(\mathbf{X})$ .

```
divOfGradA = divergence(gradA,X)
```

```
divOfGradA(X) =  $\Delta_X A(X)$ 
```

```
lapA = laplacian(A,X)
```

```
lapA(X) =  $\Delta_X A(X)$ 
```

Show that the curl of the gradient of  $A(\mathbf{X})$  is zero, that is  $\nabla_{\mathbf{X}} \times \nabla_{\mathbf{X}} A(\mathbf{X}) = \mathbf{0}_{3,1}$ .

```
curlOfGradA = curl(gradA,X)
```

```
curlOfGradA(X) =  $\mathbf{0}_{3,1}$ 
```

## Input Arguments

### **f** — Symbolic scalar field

symbolic expression | symbolic function | symbolic matrix variable | symbolic matrix function

Symbolic scalar field, specified as a symbolic expression, symbolic function, symbolic matrix variable, or symbolic matrix function.

- If **f** is a function of symbolic scalar variables, where **f** is of type `sym` or `symfun`, then the vector **v** must be of type `sym` or `symfun`.
- If **f** is a function of symbolic matrix variables, where **f** is of type `symmatrix` or `symfunmatrix`, then the vector **v** must be of type `symmatrix` or `symfunmatrix`.

Data Types: `sym` | `symfun` | `symmatrix` | `symfunmatrix`

### **v** — Vector with respect to which you find the gradient

vector of symbolic scalar variables | symbolic function | symbolic matrix variable | symbolic matrix function

Vector with respect to which you find the gradient, specified as a vector of symbolic scalar variables, symbolic function, symbolic matrix variable, or symbolic matrix function.

- If you do not specify **v** and **f** is a function of symbolic scalar variables, then, by default, `gradient` constructs vector **v** from the symbolic scalar variables in **f** with the order of variables as defined by `symvar(f)`.
- If **v** is a symbolic matrix variable of type `symmatrix`, then **v** must have a size of 1-by-N or N-by-1.
- If **v** is scalar, then `gradient(f,v) = diff(f,v)`.
- If **v** is an empty symbolic object, such as `sym([ ])`, then `gradient` returns an empty symbolic object.

When finding the gradient of a scalar function **f** with respect to a row or column vector **v**, `gradient` uses the convention of always returning the output as a column vector. For example, if **f** is a 1-by-1

scalar and  $v$  is a 1-by-3 row vector, then `gradient(f,v)` finds the derivative of  $f$  with respect to each element of  $v$  and returns the result as a 3-by-1 column vector.

Data Types: `sym` | `symfun` | `symmatrix` | `symfunmatrix`

## Limitations

- The `gradient` function does not support tensor derivatives. If the gradient is a tensor field or a matrix rather than a vector, then the `gradient` function returns an error.
- Symbolic Math Toolbox currently does not support the `dot` or `cross` functions for symbolic matrix variables and functions of type `symmatrix` and `symfunmatrix`. If vector calculus identities involve dot or cross products, then the toolbox displays those identities in terms of other supported functions instead. To see a list of all the functions that support symbolic matrix variables and functions, use the commands `methods symmatrix` and `methods symfunmatrix`.

## More About

### Gradient Vector

The gradient vector of  $f(\mathbf{x})$  with respect to the vector  $\mathbf{x} = (x_1, x_2, \dots, x_n)$  is the vector of the first partial derivatives of  $f$ .

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \left( \frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$$

## Version History

### Introduced in R2011b

#### R2023a: Compute gradient of symbolic matrix functions

The `gradient` function accepts symbolic matrix functions of type `symfunmatrix` as input arguments. For example, see “Find Vector Calculus Identities Involving Gradients” on page 6-731.

#### R2021b: Compute gradient of symbolic matrix variables

The `gradient` function accepts symbolic matrix variables of type `symmatrix` as input arguments. For examples, see “Gradient of Matrix Multiplication” on page 6-730 and “Gradient of Multivariable Function” on page 6-731.

## See Also

`curl` | `divergence` | `diff` | `hessian` | `jacobian` | `laplacian` | `potential` | `quiver` | `vectorPotential`

## gbasis

Reduced Groebner basis

### Syntax

```
gbasis(poly)
gbasis(poly,vars)
gbasis( ___, 'MonomialOrder', MonomialOrder)
```

### Description

`gbasis(poly)` returns the Groebner basis of the vector of polynomials `poly`. By default, `gbasis` finds independent variables in `poly` by using `symvar`, and uses the monomial ordering `degreeInverseLexicographic`.

`gbasis(poly,vars)` also uses the independent variables `vars`.

`gbasis( ___, 'MonomialOrder', MonomialOrder)` also uses the specified monomial order in addition to the input arguments in previous syntaxes. Options are `'degreeInverseLexicographic'`, `'degreeLexicographic'`, or `'lexicographic'`. By default, `gbasis` uses `'degreeInverseLexicographic'`.

### Examples

#### Groebner Basis of Polynomials

Calculate the Groebner basis of the polynomials  $x^2 - y^2$  and  $x^2 + y$ . By default, `gbasis` finds the independent variables by using `symvar`.

```
syms x y
p = [x^2-y^2, x^2+y];
gbasis(p)
```

```
ans =
[ x^2 + y, y^2 + y]
```

#### Specify Independent Variables

Specify the independent variables as the second argument of `gbasis`.

Compute the Groebner basis of the polynomials  $a*y + x^2*y + a$  and  $a*x^2 + y$  with the independent variables `[x y]`.

```
syms x y a
p = [a*y + x^2*y + a, a*x^2 + y];
vars = [x y];
grobnerBasis = gbasis(p,vars)
```

```
grobnerBasis =
[ a*x^2 + y, - y^2/a + a*y + a]
```

## Groebner Basis with Monomial Order

By default, `gbasis` uses the monomial order `degreeInverseLexicographic`. Change the monomial order by using the `'MonomialOrder'` name-value pair argument.

Find the Groebner basis of the polynomials  $y*z^2+1$  and  $y^2*x^2-y-z^3$  with lexicographic monomial order.

```
syms x y z
p = [y*z^2 + 1, y^2*x^2 - y - z^3];
grobnerBasis = gbasis(p,'MonomialOrder','lexicographic')
```

```
grobnerBasis =
[ x^2 - z^7 + z^2, y*z^2 + 1]
```

Use the variables `[z y]` with `degreeLexicographic` monomial order.

```
grobnerBasis = gbasis(p,[z y],'MonomialOrder','degreeLexicographic')

grobnerBasis =
[ x^2*y^2 - y - z^3, y*z^2 + 1, x^2*y^3 - y^2 + z]
```

## Input Arguments

### **poly** — Polynomials

vector of symbolic expressions

Polynomials, specified as a vector of symbolic expressions.

### **vars** — Independent variables

vector of symbolic variables

Independent variables, specified as a vector of symbolic variables.

### **MonomialOrder** — Monomial order

`'degreeInverseLexicographic'` (default) | `'degreeLexicographic'` | `'lexicographic'`

Monomial order, specified as the comma-separated pair of `'MonomialOrder'` and one of the values `'degreeInverseLexicographic'`, `'degreeLexicographic'`, or `'lexicographic'`. If `vars` is specified, then monomials are sorted with respect to the order of variables in `vars`.

- `lexicographic` sorts the terms of the polynomial using lexicographic ordering.
- `degreeLexicographic` sorts the terms of a polynomial according to the total degree of each term. If terms have equal total degrees, `polynomialReduce` sorts them using lexicographic ordering.
- `degreeInverseLexicographic` sorts the terms of a polynomial according to the total degree of each term. If terms have equal total degrees, `polynomialReduce` sorts them using inverse lexicographic ordering.

## **Version History**

**Introduced in R2018a**

## **See Also**

`eliminate` | `solve`

## **Topics**

"Solve Algebraic Equations" on page 3-3

"Solve System of Linear Equations" on page 3-36



# gt

Define greater than relation

## Syntax

```
A > B
gt(A,B)
```

## Description

$A > B$  creates a greater than relation.

`gt(A,B)` is equivalent to  $A > B$ .

## Examples

### Set and Use Assumption Using Greater Than

Use `assume` and the relational operator `>` to set the assumption that  $x$  is greater than 3:

```
syms x
assume(x > 3)
```

Solve this equation. The solver takes into account the assumption on variable  $x$ , and therefore returns this solution.

```
solve((x - 1)*(x - 2)*(x - 3)*(x - 4) == 0, x)
```

```
ans =
4
```

### Find Values that Satisfy Condition

Use the relational operator `>` to set this condition on variable  $x$ :

```
syms x
cond = abs(sin(x)) + abs(cos(x)) > 7/5;

for i = 0:sym(pi/24):sym(pi)
    if subs(cond, x, i)
        disp(i)
    end
end
```

Use the `for` loop with step  $\pi/24$  to find angles from 0 to  $\pi$  that satisfy that condition:

```
(5*pi)/24
pi/4
(7*pi)/24
(17*pi)/24
(3*pi)/4
(19*pi)/24
```

## Input Arguments

### A — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

### B — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## Tips

- Calling `>` or `gt` for non-symbolic `A` and `B` invokes the MATLAB `gt` function. This function returns a logical array with elements set to logical 1 (`true`) where `A` is greater than `B`; otherwise, it returns logical 0 (`false`).
- If both `A` and `B` are arrays, then these arrays must have the same dimensions. `A > B` returns an array of relations `A(i,j,...) > B(i,j,...)`
- If one input is scalar and the other an array, then the scalar input is expanded into an array of the same dimensions as the other array. In other words, if `A` is a variable (for example, `x`), and `B` is an  $m$ -by- $n$  matrix, then `A` is expanded into  $m$ -by- $n$  matrix of elements, each set to `x`.
- The field of complex numbers is not an ordered field. MATLAB projects complex numbers in relations to a real axis. For example, `x > i` becomes `x > 0`, and `x > 3 + 2*i` becomes `x > 3`.

## Version History

Introduced in R2012a

### See Also

`eq` | `ge` | `isAlways` | `le` | `lt` | `ne`

### Topics

“Use Assumptions on Symbolic Variables” on page 1-41

# harmonic

Harmonic function (harmonic number)

## Syntax

`harmonic(x)`

## Description

`harmonic(x)` returns the harmonic function on page 6-742 of `x`. For integer values of `x`, `harmonic(x)` generates harmonic numbers.

## Examples

### Generate Harmonic Numbers

Generate the first 10 harmonic numbers.

```
harmonic(sym(1:10))

ans =
[ 1, 3/2, 11/6, 25/12, 137/60, 49/20, 363/140, 761/280, 7129/2520, 7381/2520]
```

### Harmonic Function for Numeric and Symbolic Arguments

Find the harmonic function for these numbers. Since these are not symbolic objects, you get floating-point results.

```
harmonic([2 i 13/3])

ans =
1.5000 + 0.0000i 0.6719 + 1.0767i 2.1545 + 0.0000i
```

Find the harmonic function symbolically by converting the numbers to symbolic objects.

```
y = harmonic(sym([2 i 13/3]))

y =
[ 3/2, harmonic(1i), 8571/1820 - (pi*3^(1/2))/6 - (3*log(3))/2]
```

If the denominator of `x` is 2, 3, 4, or 6, and  $|x| < 500$ , then the result is expressed in terms of `pi` and `log`.

Use `vpa` to approximate the results obtained.

```
vpa(y)

ans =
[ 1.5, 0.67186598552400983787839057280431...
+ 1.07667404746858117413405079475i,...
2.1545225442213858782694336751358]
```

For  $|x| > 1000$ , `harmonic` returns the function call as it is. Use `vpa` to force `harmonic` to evaluate the function call.

```
harmonic(sym(1001))
vpa(harmonic(sym(1001)))

ans =
harmonic(1001)
ans =
7.4864698615493459116575172053329
```

### Harmonic Function for Special Values

Find the harmonic function for special values.

```
harmonic([0 1 -1 Inf -Inf])

ans =
     0     1    Inf    Inf    NaN
```

### Harmonic Function for Symbolic Functions

Find the harmonic function for the symbolic function  $f$ .

```
syms f(x)
f(x) = exp(x) + tan(x);
y = harmonic(f)

y(x) =
harmonic(exp(x) + tan(x))
```

### Harmonic Function for Symbolic Vectors and Matrices

Find the harmonic function for elements of vector  $V$  and matrix  $M$ .

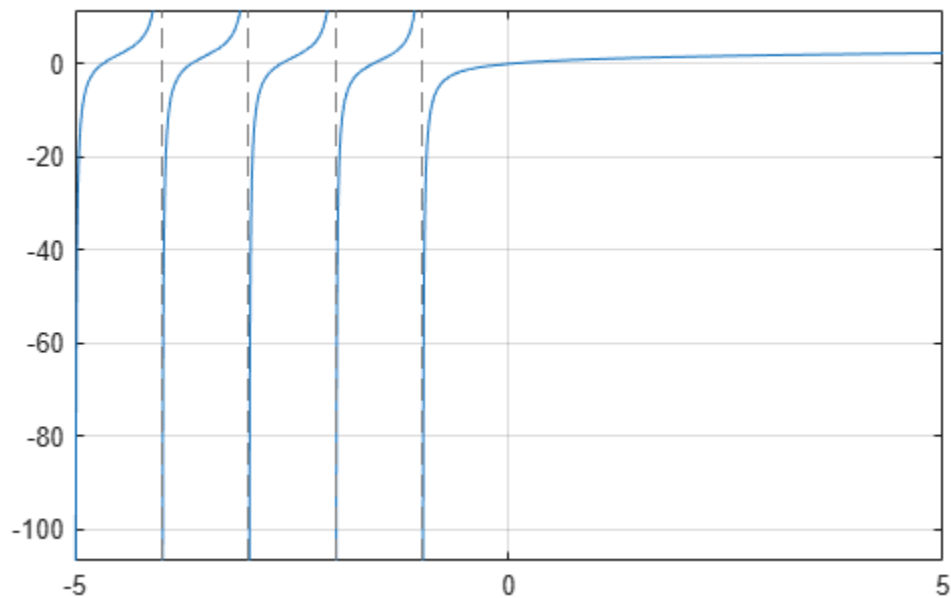
```
syms x
V = [x sin(x) 3*i];
M = [exp(i*x) 2; -6 Inf];
harmonic(V)
harmonic(M)

ans =
[ harmonic(x), harmonic(sin(x)), harmonic(3i)]
ans =
[ harmonic(exp(x*1i)), 3/2]
[      Inf, Inf]
```

### Plot Harmonic Function

Plot the harmonic function from  $x = -5$  to  $x = 5$ .

```
syms x
fplot(harmonic(x), [-5 5])
grid on
```



### Differentiate and Find Limit of Harmonic Function

The functions `diff` and `limit` handle expressions containing `harmonic`.

Find the second derivative of `harmonic(x^2+1)`.

```
syms x
diff(harmonic(x^2+1),x,2)

ans =
2*psi(1, x^2 + 2) + 4*x^2*psi(2, x^2 + 2)
```

Find the limit of `harmonic(x)` as  $x$  tends to  $\infty$  and of  $(x+1)*\text{harmonic}(x)$  as  $x$  tends to  $-1$ .

```
syms x
limit(harmonic(x),Inf)
limit((x+1)*harmonic(x),-1)

ans =
Inf
ans =
-1
```

### Taylor Series Expansion of Harmonic Function

Use `taylor` to expand the harmonic function in terms of the Taylor series.

```
syms x
taylor(harmonic(x))

ans =
(pi^6*x^5)/945 - zeta(5)*x^4 + (pi^4*x^3)/90...
- zeta(3)*x^2 + (pi^2*x)/6
```

## Expand Harmonic Function

Use `expand` to expand the harmonic function.

```
syms x
expand(harmonic(2*x+3))

ans =
harmonic(x + 1/2)/2 + log(2) + harmonic(x)/2 - 1/(2*(x + 1/2))...
+ 1/(2*x + 1) + 1/(2*x + 2) + 1/(2*x + 3)
```

## Input Arguments

### x — Input

number | vector | matrix | multidimensional array | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix | symbolic N-D array

Input, specified as number, vector, matrix, or as a multidimensional array or symbolic variable, expression, function, vector, matrix, or multidimensional array.

## More About

### Harmonic Function

The harmonic function for  $x$  is defined as

$$\text{harmonic}(x) = \sum_{k=1}^x \frac{1}{k}$$

It is also defined as

$$\text{harmonic}(x) = \Psi(x+1) + \gamma$$

where  $\Psi(x)$  is the polygamma function and  $\gamma$  is the Euler-Mascheroni constant.

## Algorithms

The harmonic function is defined for all complex arguments  $z$  except for negative integers  $-1, -2, \dots$  where a singularity occurs.

If  $x$  has denominator 1, 2, 3, 4, or 6, then an explicit result is computed and returned. For other rational numbers, `harmonic` uses the functional equation  $\text{harmonic}(x+1) = \text{harmonic}(x) + \frac{1}{x}$  to obtain a result with an argument  $x$  from the interval  $[0, 1]$ .

`expand` expands `harmonic` using the equations  $\text{harmonic}(x+1) = \text{harmonic}(x) + \frac{1}{x}$ ,

$\text{harmonic}(-x) = \text{harmonic}(x) - \frac{1}{x} + \pi \cot(\pi x)$ , and the Gauss multiplication formula for  $\text{harmonic}(kx)$ , where  $k$  is an integer.

`harmonic` implements the following explicit formulae:

$$\text{harmonic}\left(\frac{-1}{2}\right) = -2\ln(2)$$

$$\text{harmonic}\left(-\frac{2}{3}\right) = -\frac{3}{2}\ln(3) - \frac{\sqrt{3}}{6}\pi$$

$$\text{harmonic}\left(-\frac{1}{3}\right) = -\frac{3}{2}\ln(3) + \frac{\sqrt{3}}{6}\pi$$

$$\text{harmonic}\left(\frac{-3}{4}\right) = -3\ln(2) - \frac{\pi}{2}$$

$$\text{harmonic}\left(\frac{-1}{4}\right) = -3\ln(2) + \frac{\pi}{2}$$

$$\text{harmonic}\left(\frac{-5}{6}\right) = -2\ln(2) - \frac{3}{2}\ln(3) - \frac{\sqrt{3}}{2}\pi$$

$$\text{harmonic}\left(\frac{-1}{6}\right) = -2\ln(2) - \frac{3}{2}\ln(3) + \frac{\sqrt{3}}{2}\pi$$

$$\text{harmonic}(0) = 0$$

$$\text{harmonic}\left(\frac{1}{2}\right) = 2 - 2\ln(2)$$

$$\text{harmonic}\left(\frac{1}{3}\right) = 3 - \frac{3}{2}\ln(3) - \frac{\sqrt{3}}{6}\pi$$

$$\text{harmonic}\left(\frac{2}{3}\right) = \frac{3}{2} - \frac{3}{2}\ln(3) + \frac{\sqrt{3}}{6}\pi$$

$$\text{harmonic}\left(\frac{1}{4}\right) = 4 - 3\ln(2) - \frac{\pi}{2}$$

$$\text{harmonic}\left(\frac{3}{4}\right) = \frac{4}{3} - 3\ln(2) + \frac{\pi}{2}$$

$$\text{harmonic}\left(\frac{1}{6}\right) = 6 - 2\ln(2) - \frac{3}{2}\ln(3) - \frac{\sqrt{3}}{2}\pi$$

$$\text{harmonic}\left(\frac{5}{6}\right) = \frac{6}{5} - 2\ln(2) - \frac{3}{2}\ln(3) + \frac{\sqrt{3}}{2}\pi$$

$$\text{harmonic}(1) = 1$$

$$\text{harmonic}(\infty) = \infty$$

$$\text{harmonic}(-\infty) = \text{NaN}$$

## Version History

Introduced in R2014a

**See Also**

`beta` | `nchoosek` | `factorial` | `gamma` | `gamma1n` | `zeta`



# has

Check if expression contains particular subexpression

## Syntax

`has(expr, subexpr)`

## Description

`has(expr, subexpr)` returns logical 1 (true) if `expr` contains `subexpr`. Otherwise, it returns logical 0 (false).

- If `expr` is an array, `has(expr, subexpr)` returns an array of the same size as `expr`. The returned array contains logical 1s (true) where the elements of `expr` contain `subexpr`, and logical 0s (false) where they do not.
- If `subexpr` is an array, `has(expr, subexpr)` checks if `expr` contains any element of `subexpr`.

## Examples

### Check If Expression Contains Particular Subexpression

Use the `has` function to check if an expression contains a particular variable or subexpression.

Check if these expressions contain variable `z`.

```
syms x y z
has(x + y + z, z)
```

```
ans =
    logical
     1
```

```
has(x + y, z)
```

```
ans =
    logical
     0
```

Check if `x + y + z` contains the following subexpressions. Note that `has` finds the subexpression `x + z` even though the terms `x` and `z` do not appear next to each other in the expression.

```
has(x + y + z, x + y)
has(x + y + z, y + z)
has(x + y + z, x + z)
```

```
ans =
    logical
     1
ans =
    logical
     1
ans =
```

```
logical
1
```

Check if the expression  $(x + 1)^2$  contains  $x^2$ . Although  $(x + 1)^2$  is mathematically equivalent to the expression  $x^2 + 2*x + 1$ , the result is a logical 0 because `has` typically does not transform expressions to different forms when testing for subexpressions.

```
has((x + 1)^2, x^2)
```

```
ans =
logical
0
```

Expand the expression and then call `has` to check if the result contains  $x^2$ . Because `expand((x + 1)^2)` transforms the original expression to  $x^2 + 2*x + 1$ , the `has` function finds the subexpression  $x^2$  and returns logical 1.

```
has(expand((x + 1)^2), x^2)
```

```
ans =
logical
1
```

### Check If Expression Contains Any of Specified Subexpressions

Check if a symbolic expression contains any of subexpressions specified as elements of a vector.

If an expression contains one or more of the specified subexpressions, `has` returns logical 1.

```
syms x
has(sin(x) + cos(x) + x^2, [tan(x), cot(x), sin(x), exp(x)])
```

```
ans =
logical
1
```

If an expression does not contain any of the specified subexpressions, `has` returns logical 0.

```
syms x
has(sin(x) + cos(x) + x^2, [tan(x), cot(x), exp(x)])
```

```
ans =
logical
0
```

### Find Matrix Elements Containing Particular Subexpression

Using `has`, find those elements of a symbolic matrix that contain a particular subexpression.

First, create a matrix.

```
syms x y
M = [sin(x)*sin(y), cos(x*y) + 1; cos(x)*tan(x), 2*sin(x)^2]
```

```
M =
[ sin(x)*sin(y), cos(x*y) + 1]
[ cos(x)*tan(x), 2*sin(x)^2]
```

Use `has` to check which elements of `M` contain `sin(x)`. The result is a matrix of the same size as `M`, with `1s` and `0s` as its elements. For the elements of `M` containing the specified expression, `has` returns logical `1s`. For the elements that do not contain that subexpression, `has` returns logical `0s`.

```
T = has(M, sin(x))
```

```
T =
    2x2 logical array
     1     0
     0     1
```

Return only the elements that contain `sin(x)` and replace all other elements with `0` by multiplying `M` by `T` elementwise.

```
M.*T
```

```
ans =
 [ sin(x)*sin(y),      0]
 [      0, 2*sin(x)^2]
```

To check if any of matrix elements contain a particular subexpression, use `any`.

```
any(has(M(:), sin(x)))
```

```
ans =
    logical
     1
```

```
any(has(M(:), cos(y)))
```

```
ans =
    logical
     0
```

### Find Vector Elements Containing Any of Specified Subexpressions

Using `has`, find those elements of a symbolic vector that contain any of the specified subexpressions.

```
syms x y z
T = has([x + 1, cos(y) + 1, y + z, 2*x*cos(y)], [x, cos(y)])
```

```
T =
    1x4 logical array
     1     1     0     1
```

Return only the elements of the original vector that contain `x` or `cos(y)` or both, and replace all other elements with `0` by multiplying the original vector by `T` elementwise.

```
[x + 1, cos(y) + 1, y + z, 2*x*cos(y)].*T
```

```
ans =
 [ x + 1, cos(y) + 1, 0, 2*x*cos(y)]
```

### Use has for Symbolic Functions

If `expr` or `subexpr` is a symbolic function, `has` uses `formula(expr)` or `formula(subexpr)`. This approach lets the `has` function check if an expression defining the symbolic function `expr` contains an expression defining the symbolic function `subexpr`.

Create a symbolic function.

```
syms x
f(x) = sin(x) + cos(x);
```

Here,  $\sin(x) + \cos(x)$  is an expression defining the symbolic function  $f$ .

```
formula(f)
```

```
ans =
cos(x) + sin(x)
```

Check if  $f$  and  $f(x)$  contain  $\sin(x)$ . In both cases `has` checks if the expression  $\sin(x) + \cos(x)$  contains  $\sin(x)$ .

```
has(f, sin(x))
has(f(x), sin(x))
```

```
ans =
    logical
     1
ans =
    logical
     1
```

Check if  $f(x^2)$  contains  $f$ . For these arguments, `has` returns logical 0 (false) because it does not check if the expression  $f(x^2)$  contains the letter  $f$ . This call is equivalent to `has(f(x^2), formula(f))`, which, in turn, resolves to `has(cos(x^2) + sin(x^2), cos(x) + sin(x))`.

```
has(f(x^2), f)
```

```
ans =
    logical
     0
```

### Check for Calls to Particular Function

Check for calls to a particular function by specifying the function name as the second argument. Check for calls to any one of multiple functions by specifying the multiple functions as a cell array of character vectors.

Integrate  $\tan(x^7)$ . Determine if the integration is successful by checking the result for calls to `int`. Because `has` finds the `int` function and returns logical 1 (true), the integration is not successful.

```
syms x
f = int(tan(x^7), x);
has(f, 'int')
```

```
ans =
    logical
     1
```

Check if the solution to a differential equation contains calls to either `sin` or `cos` by specifying the second argument as `{'sin', 'cos'}`. The `has` function returns logical 0 (false), which means the solution does not contain calls to either `sin` or `cos`.

```
syms y(x) a
sol = dsolve(diff(y,x) == a*y);
has(sol, {'sin' 'cos'})
```

```
ans =
    logical
     0
```

## Input Arguments

### **expr — Expression to test**

symbolic expression | symbolic function | symbolic equation | symbolic inequality | symbolic vector | symbolic matrix | symbolic array

Expression to test, specified as a symbolic expression, function, equation, or inequality. Also it can be a vector, matrix, or array of symbolic expressions, functions, equations, and inequalities.

### **subexpr — Subexpression to check for**

symbolic variable | symbolic expression | symbolic function | symbolic equation | symbolic inequality | symbolic vector | symbolic matrix | symbolic array | character vector | cell array of character vectors

Subexpression to test for, specified as a symbolic variable, expression, function, equation, or inequality, or a character vector, or a cell array of character vectors. **subexpr** can also be a vector, matrix, or array of symbolic variables, expressions, functions, equations, and inequalities.

## Tips

- **has** does not transform or simplify expressions. This is why it does not find subexpressions like  $x^2$  in expressions like  $(x + 1)^2$ . However, in some cases **has** might find that an expression or subexpression can be represented in a form other than its original form. For example, **has** finds that the expression  $-x - 1$  can be represented as  $-(x + 1)$ . Thus, the call `has(-x - 1, x + 1)` returns 1.
- If **expr** is an empty symbolic array, **has** returns an empty logical array of the same size as **expr**.

## Version History

**Introduced in R2015b**

## See Also

`subs` | `subexpr` | `times`

## hasSymType

Determine whether symbolic object contains specific type

### Syntax

```
TF = hasSymType(symObj,type)
TF = hasSymType(symObj,funType,vars)
```

### Description

`TF = hasSymType(symObj,type)` returns logical 1 (true) if the symbolic object `symObj` contains a subobject of type `type`, and logical 0 (false) otherwise. The input `type` must be a case-sensitive string scalar or character vector, and it can include a logical expression.

`TF = hasSymType(symObj,funType,vars)` checks whether `symObj` contains an unassigned symbolic function that depends on the variables `vars`.

You can set the function type `funType` to `'symfunOf'` or `'symfunDependingOn'`. For example, `syms f(x); hasSymType(f,'symfunOf',x)` returns logical 1.

### Examples

#### Symbolic Variable, Constant, or Number

Determine whether a symbolic expression contains a symbolic variable, constant, or number of a specific type.

Create a symbolic expression.

```
syms x;
expr = sym('1/2') + 2*pi + x

expr =
    x + 2 pi +  $\frac{1}{2}$ 
```

Check whether `expr` contains a symbolic variable of type `'variable'`.

```
TF = hasSymType(expr,'variable')

TF = logical
    1
```

Check whether `expr` contains a symbolic constant of type `'constant'`.

```
TF = hasSymType(expr,'constant')

TF = logical
    1
```

Check whether `expr` contains a symbolic number of type 'integer'.

```
TF = hasSymType(expr, 'integer')
```

$$TF = \frac{logical}{1}$$

Check whether `expr` contains a symbolic number of type `'integer | real'`.

```
TF = hasSymType(expr, 'integer | real')
```

$$TF = \frac{logical}{1}$$

Check whether `expr` contains a symbolic number of type 'complex'.

```
TF = hasSymType(expr, 'complex')
```

$$TF = \frac{logical}{0}$$

### Symbolic Function or Operator

Determine whether a symbolic equation contains a symbolic function or operator of a specific type.

Create a symbolic equation.

```
syms f(x) n
```

```
eq = f(x^n) + int(f(x),x) + vpa(2.7) == 1i
```

eq =

$$f(x^n) + \int f(x) \, dx$$

[illegible]
$$=$$

i

Check whether eq contains the symbolic function 'f'.

```
TF = hasSymType(eq, 'f')
```

$$TF = \frac{logical}{1}$$

Check whether eq contains an unassigned symbolic function of type 'symfun'.

```
TF = hasSymType(eq, 'symfun')
```

$$TF = \frac{logical}{1}$$

Check whether eq contains a symbolic math function of type 'int'.

```
TF = hasSymType(eq, 'int')
```

```
TF = logical
1
```

Check whether eq contains an operator of type 'power'.

```
TF = hasSymType(eq, 'power')
```

```
TF = logical
1
```

## Function of Multiple Variables

Create a symbolic function of multiple variables using `syms`.

```
syms f(x,y,z)
g = f + x*y + pi
```

```
g(x, y, z) = pi + x*y + f(x,y,z)
```

Check whether g depends on the exact variable x using 'symfunOf'.

```
TF = hasSymType(g, 'symfunOf', x)
```

```
TF = logical
0
```

Check whether g depends on the exact sequence of variables [x y z] using 'symfunOf'.

```
TF = hasSymType(g, 'symfunOf', [x y z])
```

```
TF = logical
1
```

Check whether g has any dependency on the variables [y x] using 'symfunDependingOn'.

```
TF = hasSymType(g, 'symfunDependingOn', [y x])
```

```
TF = logical
1
```

## Input Arguments

### **symObj** — Symbolic objects

symbolic expressions | symbolic functions | symbolic variables | symbolic numbers | symbolic units



Symbolic objects, specified as symbolic expressions, symbolic functions, symbolic variables, symbolic numbers, or symbolic units.

### type — Symbolic types

scalar string | character vector

Symbolic types, specified as a case-sensitive scalar string or character vector. The input type can contain a logical expression. The value options follow.

| Symbolic Type Category        | String Values                                                                                                                                                                                                                                                                                                                                                                                               | Examples Returning Logical 1                                                                                                                                                                                                                                                                                  |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| numbers                       | <ul style="list-style-type: none"> <li>'integer' — integer numbers</li> <li>'rational' — rational numbers</li> <li>'vpareal' — variable-precision floating-point real numbers</li> <li>'complex' — complex numbers</li> <li>'real' — real numbers, including 'integer', 'rational', and 'vpareal'</li> <li>'number' — numbers, including 'integer', 'rational', 'vpareal', 'complex', and 'real'</li> </ul> | <ul style="list-style-type: none"> <li>hasSymType(sym(2), 'integer')</li> <li>hasSymType(sym(1/2), 'rational')</li> <li>hasSymType(vpa(0.5), 'vpareal')</li> <li>hasSymType(vpa(1i), 'complex')</li> <li>hasSymType([sym(1/2) vpa(0.5)], 'real')</li> <li>hasSymType([vpa(1i) sym(1/2)], 'number')</li> </ul> |
| constants                     | 'constant' — symbolic mathematical constants, including 'number'                                                                                                                                                                                                                                                                                                                                            | hasSymType([sym(pi) vpa(1i)], 'constant')                                                                                                                                                                                                                                                                     |
| symbolic math functions       | 'vpa', 'sin', 'exp', and so on — symbolic math functions in symbolic expressions                                                                                                                                                                                                                                                                                                                            | hasSymType(vpa(sym(pi)), 'vpa')                                                                                                                                                                                                                                                                               |
| unassigned symbolic functions | <ul style="list-style-type: none"> <li>'F', 'g', and so on — function name of an unassigned symbolic function</li> <li>'symfun' — unassigned symbolic functions</li> </ul>                                                                                                                                                                                                                                  | <ul style="list-style-type: none"> <li>syms F(x);<br/>hasSymType(F(x+2), 'F')</li> <li>syms g(x);<br/>hasSymType(g(x), 'symfun')</li> </ul>                                                                                                                                                                   |
| arithmetic operators          | <ul style="list-style-type: none"> <li>'plus' — addition operator + and subtraction operator -</li> <li>'times' — multiplication operator * and division operator /</li> <li>'power' — power or exponentiation operator ^ and square root operator sqrt</li> </ul>                                                                                                                                          | <ul style="list-style-type: none"> <li>syms x y;<br/>hasSymType(2*x + y, 'plus')</li> <li>syms x y;<br/>hasSymType(x*y, 'times')</li> <li>syms x y;<br/>hasSymType(x^(y+2), 'power')</li> </ul>                                                                                                               |
| variables                     | 'variable' — symbolic variables                                                                                                                                                                                                                                                                                                                                                                             | hasSymType(sym('x'), 'variable')                                                                                                                                                                                                                                                                              |
| units                         | 'unit' — symbolic units                                                                                                                                                                                                                                                                                                                                                                                     | hasSymType(symunit('m'), 'unit')                                                                                                                                                                                                                                                                              |
| expressions                   | 'expression' — symbolic expressions, including all of the preceding symbolic types                                                                                                                                                                                                                                                                                                                          | hasSymType(sym('x') + 1, 'expression')                                                                                                                                                                                                                                                                        |

| Symbolic Type Category     | String Values                                                                                                                                                                                                                                                                                                                                                                                                    | Examples Returning Logical 1                                                                                                                                                                                                                                                                                                                       |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| logical expressions        | <ul style="list-style-type: none"> <li>'or' — logical OR operator  </li> <li>'and' — logical AND operator &amp;</li> <li>'not' — logical NOT operator ~</li> <li>'xor' — logical exclusive-OR operator xor</li> <li>'logicalconstant' — symbolic logical constants symtrue and symfalse</li> <li>'logicalexpression' — logical expressions, including 'or', 'and', 'not', 'xor', symtrue and symfalse</li> </ul> | <ul style="list-style-type: none"> <li>syms x y;<br/>hasSymType(x y, 'or')</li> <li>syms x y;<br/>hasSymType(x&amp;y, 'and')</li> <li>syms x;<br/>hasSymType(~x, 'not')</li> <li>syms x y;<br/>hasSymType(xor(x,y), 'xor')</li> <li>hasSymType(symtrue, 'logicalconstant')</li> <li>syms x y;<br/>hasSymType(~x y, 'logicalexpression')</li> </ul> |
| equations and inequalities | <ul style="list-style-type: none"> <li>'eq' — equality operator ==</li> <li>'ne' — inequality operator ~=</li> <li>'lt' — less-than operator &lt; or greater-than operator &gt;</li> <li>'le' — less-than-or-equal-to operator &lt;= or greater-than-or-equal-to operator &gt;=</li> <li>'equation' — symbolic equations and inequalities, including 'eq', 'ne', 'lt', and 'le'</li> </ul>                       | <ul style="list-style-type: none"> <li>syms x;<br/>hasSymType(x==2, 'eq')</li> <li>syms x;<br/>hasSymType(x~=1, 'ne')</li> <li>syms x;<br/>hasSymType(x&gt;0, 'lt')</li> <li>syms x;<br/>hasSymType(x&lt;=2, 'le')</li> <li>syms x;<br/>hasSymType([x&gt;0<br/>x~=1], 'equation')</li> </ul>                                                       |
| unsupported symbolic types | 'unsupported' — unsupported symbolic types                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                    |

### funType — Function type

'symfunOf' | 'symfunDependingOn'

Function type, specified as 'symfunOf' or 'symfunDependingOn'.

- 'symfunOf' checks whether symObj contains an unassigned symbolic function that depends on the exact sequence of variables specified by the array vars. For example, syms f(x,y); hasSymType(f, 'symfunOf', [x y]) returns logical 1.
- 'symfunDependingOn' checks whether symObj contains an unassigned symbolic function that has a dependency on the variables specified by the array vars. For example, syms f(x,y); hasSymType(f, 'symfunDependingOn', [y x]) returns logical 1.

### vars — Input variables

symbolic variables | symbolic array

Input variables, specified as symbolic variables or a symbolic array.

## Tips

- To check whether a symbolic expression contains a particular subexpression, use the `has` function.

## Version History

Introduced in R2019a

## See Also

`has` | `symFunType` | `isSymType` | `symType` | `sym` | `syms` | `mapSymType` | `findSymType`

## heaviside

Heaviside step function

### Syntax

```
H = heaviside(x)
```

### Description

`H = heaviside(x)` evaluates the Heaviside step function (also known as the unit step function) at `x`; the Heaviside step function returns 0 for  $x < 0$ ,  $1/2$  for  $x = 0$ , and 1 for  $x > 0$ .

### Examples

#### Evaluate Heaviside Function for Symbolic and Numeric Arguments

The `heaviside` function returns 0,  $1/2$ , or 1 depending on the argument value. If the argument is a floating-point number (not a symbolic object), then `heaviside` returns floating-point results.

Evaluate the Heaviside step function for a symbolic input `sym(-3)`. The function `heaviside(x)` returns 0 for  $x < 0$ .

```
H = heaviside(sym(-3))
```

```
H = 0
```

Evaluate the Heaviside step function for a symbolic input `sym(3)`. The function `heaviside(x)` returns 1 for  $x > 0$ .

```
H = heaviside(sym(3))
```

```
H = 1
```

Evaluate the Heaviside step function for a symbolic input `sym(0)`. The function `heaviside(x)` returns  $1/2$  for  $x = 0$ .

```
H = heaviside(sym(0))
```

```
H =  
    1  
    2
```

For a numeric input  $x = 0$ , the function `heaviside(x)` returns floating-point results.

```
H = heaviside(0)
```

```
H =  
0.5000
```

### Use Assumptions on Symbolic Variables

heaviside takes into account assumptions on variables.

Create a symbolic variable  $x$  and assume that it is less than 0.

```
syms x
assume(x < 0)
```

Evaluate the Heaviside step function for the symbolic input  $x$ .

```
H = heaviside(x)
```

```
H = 0
```

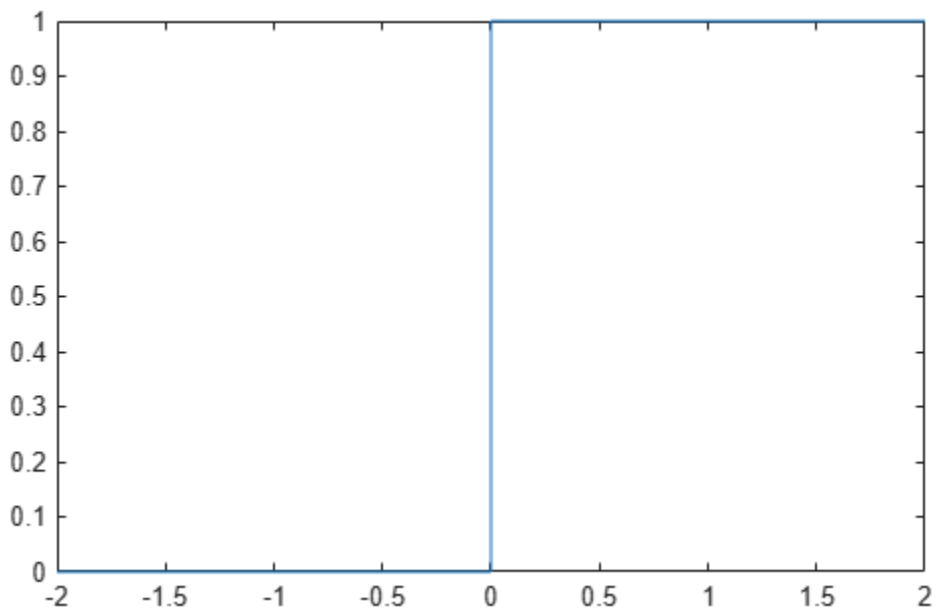
For further computations, clear the assumptions on  $x$  by recreating it using `syms`.

```
syms x
```

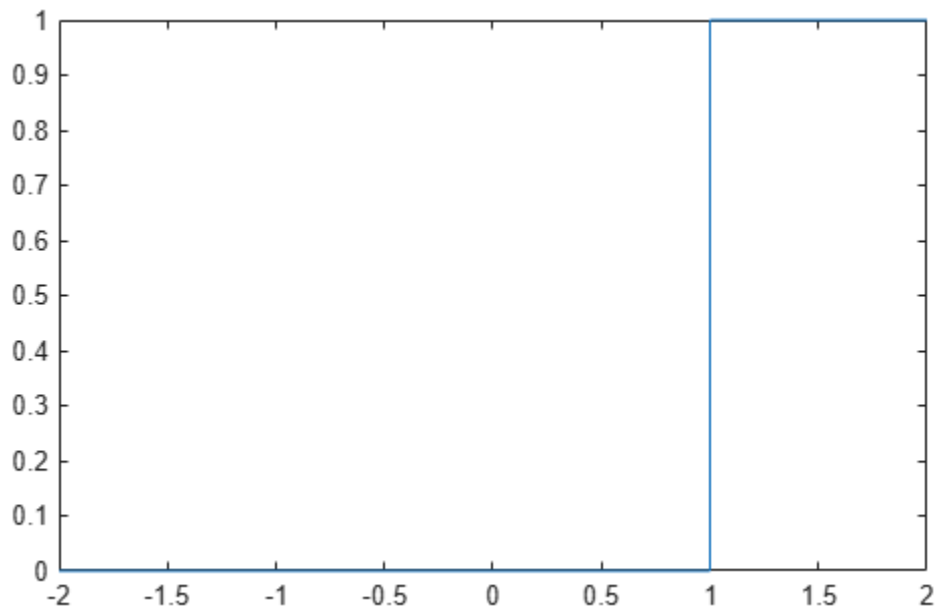
### Plot Heaviside Function

Plot the Heaviside step function for  $x$  and  $x - 1$ .

```
syms x
fplot(heaviside(x), [-2, 2])
```



```
fplot(heaviside(x - 1), [-2, 2])
```



### Evaluate Heaviside Function for Symbolic Matrix

Evaluate the Heaviside function for a symbolic matrix. When the input argument is a matrix, `heaviside` computes the Heaviside function for each element.

```
syms x
H = heaviside(sym([-1 0; 1/2 x]))
```

H =

$$\begin{pmatrix} 0 & \frac{1}{2} \\ 1 & \text{heaviside}(x) \end{pmatrix}$$

### Differentiate and Integrate Expressions Involving Heaviside Function

Compute derivatives and integrals of expressions involving the Heaviside function.

Find the first derivative of the Heaviside function. The first derivative of the Heaviside function is the Dirac delta function.

```
syms x
diff_H = diff(heaviside(x),x)
```

diff\_H =  $\delta(x)$

Evaluate the integral  $\int_{-\infty}^{\infty} e^{-x} H(x) dx$ .

```
syms x
int_H = int(exp(-x)*heaviside(x),x,-Inf,Inf)

int_H = 1
```

### Find Fourier and Laplace Transforms of Heaviside Function

Find the fourier transform of the Heaviside function.

```
syms x
F = fourier(heaviside(x))

F =
```

$$\pi \delta(w) - \frac{i}{w}$$

Find the laplace transform of the Heaviside function.

```
syms x
L = laplace(heaviside(x))

L =
```

$$\frac{1}{s}$$

### Change Value of Heaviside Function at Origin

The default value of the Heaviside function at the origin is 1/2.

```
H = heaviside(sym(0))

H =
```

$$\frac{1}{2}$$

Other common values for the Heaviside function at the origin are 0 and 1. To change the value of `heaviside` at the origin, use `sympref` and specify the value of the "HeavisideAtOrigin" setting. Store the previous parameter value returned by `sympref`, so that you can restore it later.

```
oldparam = sympref("HeavisideAtOrigin",1);
```

Check the new value of `heaviside` at 0.

```
H = heaviside(sym(0))

H = 1
```

The symbolic settings that you set using `sympref` persist throughout your current and future MATLAB® sessions. To restore the previous value of `heaviside` at the origin, use the value stored in `oldparam`.

```
sympref("HeavisideAtOrigin",oldparam);
```

Alternatively, you can restore the default value of "HeavisideAtOrigin" by using the "default" setting.

```
sympref("HeavisideAtOrigin","default");
```

## Input Arguments

### **x — Input**

number | symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a number, symbolic number, variable, expression, function, vector, or matrix.

## Version History

**Introduced before R2006a**

### **See Also**

dirac | laplace | sympref



# hermiteForm

Hermite form of matrix

## Syntax

```
H = hermiteForm(A)
[U,H] = hermiteForm(A)
___ = hermiteForm(A,var)
```

## Description

`H = hermiteForm(A)` returns the Hermite normal form on page 6-765 of a matrix `A`. The elements of `A` must be integers or polynomials in a variable determined by `symvar(A,1)`. The Hermite form `H` is an upper triangular matrix.

`[U,H] = hermiteForm(A)` returns the Hermite normal form of `A` and a unimodular transformation matrix `U`, such that  $H = U \cdot A$ .

`___ = hermiteForm(A,var)` assumes that the elements of `A` are univariate polynomials in the specified variable `var`. If `A` contains other variables, `hermiteForm` treats those variables as symbolic parameters.

You can use the input argument `var` in any of the previous syntaxes.

If `A` does not contain `var`, then `hermiteForm(A)` and `hermiteForm(A,var)` return different results.

## Examples

### Hermite Form for Matrix of Integers

Find the Hermite form of an inverse Hilbert matrix.

```
A = sym(invhilb(5))
```

A =

$$\begin{pmatrix} 25 & -300 & 1050 & -1400 & 630 \\ -300 & 4800 & -18900 & 26880 & -12600 \\ 1050 & -18900 & 79380 & -117600 & 56700 \\ -1400 & 26880 & -117600 & 179200 & -88200 \\ 630 & -12600 & 56700 & -88200 & 44100 \end{pmatrix}$$

```
H = hermiteForm(A)
```

H =

$$\begin{pmatrix} 5 & 0 & -210 & -280 & 630 \\ 0 & 60 & 0 & 0 & 0 \\ 0 & 0 & 420 & 0 & 0 \\ 0 & 0 & 0 & 840 & 0 \\ 0 & 0 & 0 & 0 & 2520 \end{pmatrix}$$

### Hermite Form for Matrix of Univariate Polynomials

Create a 2-by-2 matrix, the elements of which are polynomials in the variable  $x$ .

```
syms x
A = [x^2 + 3, (2*x - 1)^2; (x + 2)^2, 3*x^2 + 5]
```

$$A = \begin{pmatrix} x^2 + 3 & (2x - 1)^2 \\ (x + 2)^2 & 3x^2 + 5 \end{pmatrix}$$

Find the Hermite form of this matrix.

```
H = hermiteForm(A)
```

$$H = \begin{pmatrix} 1 & \frac{4x^3}{49} + \frac{47x^2}{49} - \frac{76x}{49} + \frac{20}{49} \\ 0 & x^4 + 12x^3 - 13x^2 - 12x - 11 \end{pmatrix}$$

### Hermite Form for Matrix of Multivariate Polynomials

Create a 2-by-2 matrix that contains two variables:  $x$  and  $y$ .

```
syms x y
A = [2/x + y, x^2 - y^2; 3*sin(x) + y, x]
```

$$A = \begin{pmatrix} y + \frac{2}{x} & x^2 - y^2 \\ y + 3\sin(x) & x \end{pmatrix}$$

Find the Hermite form of this matrix. If you do not specify the polynomial variable, `hermiteForm` uses `symvar(A, 1)` and thus determines that the polynomial variable is  $x$ . Because  $3\sin(x) + y$  is not a polynomial in  $x$ , `hermiteForm(A)` throws an error.

Find the Hermite form of  $A$  specifying that all elements of  $A$  are polynomials in the variable  $y$ .

```
H = hermiteForm(A, y)
```

$$H = \begin{pmatrix} 1 & \frac{xy^2}{3x\sin(x) - 2} + \frac{x(x - x^2)}{3x\sin(x) - 2} \\ 0 & 3y^2\sin(x) - 3x^2\sin(x) + y^3 + y(x - x^2) + 2 \end{pmatrix}$$

### Hermite Form and Transformation Matrix

Find the Hermite form and the corresponding transformation matrix for an inverse Hilbert matrix.

```
A = sym(invhilb(3));
[U,H] = hermiteForm(A)
```

$$U = \begin{pmatrix} 13 & 9 & 7 \\ 6 & 4 & 3 \\ 20 & 15 & 12 \end{pmatrix}$$

$$H = \begin{pmatrix} 3 & 0 & 30 \\ 0 & 12 & 0 \\ 0 & 0 & 60 \end{pmatrix}$$

Verify that  $H = U*A$ .

```
isAlways(H == U*A)
```

```
ans = 3×3 logical array
```

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Find the Hermite form and the corresponding transformation matrix for a matrix of polynomials.

```
syms x y
A = [2*(x - y), 3*(x^2 - y^2);
     4*(x^3 - y^3), 5*(x^4 - y^4)];
[U,H] = hermiteForm(A,x)
```

$$U = \begin{pmatrix} \frac{1}{2} & 0 \\ 2x^2 + 2xy + 2y^2 - 1 \end{pmatrix}$$

$$H = \begin{pmatrix} x - y & \frac{3x^2}{2} - \frac{3y^2}{2} \\ 0 & x^4 + 6x^3y - 6xy^3 - y^4 \end{pmatrix}$$

Verify that  $H = U*A$ .

```
isAlways(H == U*A)
```

```
ans = 2×2 logical array
```

$$\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

### If You Specify Variable for Integer Matrix

If a matrix does not contain a particular variable, and you call `hermiteForm` specifying that variable as the second argument, then the result differs from what you get without specifying that variable. For example, create a matrix that does not contain any variables.

```
A = [9 -36 30; -36 192 -180; 30 -180 180]
```

```
A = 3×3
```

```
     9    -36     30
    -36    192    -180
     30   -180    180
```

Call `hermiteForm` specifying variable `x` as the second argument. In this case, `hermiteForm` assumes that the elements of `A` are univariate polynomials in `x`.

```
syms x
hermiteForm(A,x)
```

```
ans = 3×3
```

```
     1     0     0
     0     1     0
     0     0     1
```

Call `hermiteForm` without specifying variables. In this case, `hermiteForm` treats `A` as a matrix of integers.

```
hermiteForm(A)
```

```
ans = 3×3
```

```
     3     0     30
     0    12     0
     0     0    60
```

## Input Arguments

### A — Input matrix

symbolic matrix

Input matrix, specified as a symbolic matrix, the elements of which are integers or univariate polynomials. If the elements of `A` contain more than one variable, use the `var` argument to specify a polynomial variable, and treat all other variables as symbolic parameters. If `A` is multivariate, and you do not specify `var`, then `hermiteForm` uses `symvar(A,1)` to determine a polynomial variable.

### var — Polynomial variable

symbolic variable

Polynomial variable, specified as a symbolic variable.

## Output Arguments

### **H — Hermite normal form of input matrix**

symbolic matrix

Hermite normal form of input matrix, returned as a symbolic matrix. The Hermite form of a matrix is an upper triangular matrix.

### **U — Transformation matrix**

unimodular symbolic matrix

Transformation matrix, returned as a unimodular symbolic matrix. If elements of  $A$  are integers, then elements of  $U$  are also integers, and  $\det(U) = 1$  or  $\det(U) = -1$ . If elements of  $A$  are polynomials, then elements of  $U$  are univariate polynomials, and  $\det(U)$  is a constant.

## More About

### **Hermite Normal Form**

For any square  $n$ -by- $n$  matrix  $A$  with integer coefficients, there exists an  $n$ -by- $n$  matrix  $H$  and an  $n$ -by- $n$  unimodular matrix  $U$ , such that  $U*A = H$ , where  $H$  is the Hermite normal form of  $A$ . A unimodular matrix is a real square matrix, such that its determinant equals 1 or -1. If  $A$  is a matrix of polynomials, then the determinant of  $U$  is a constant.

hermiteForm returns the Hermite normal form of a nonsingular integer square matrix  $A$  as an upper triangular matrix  $H$ , such that  $H_{jj} \geq 0$  and  $-\frac{H_{jj}}{2} < H_{ij} \leq \frac{H_{jj}}{2}$  for  $j > i$ . If  $A$  is not a square matrix or a singular matrix, the matrix  $H$  is simply an upper triangular matrix.

## Version History

Introduced in R2015b

### **See Also**

jordan | smithForm

## hermiteH

Hermite polynomials

### Syntax

`hermiteH(n,x)`

### Description

`hermiteH(n,x)` represents the  $n$ th-degree Hermite polynomial at the point  $x$ .

### Examples

#### First Five Hermite Polynomials

Find the first five Hermite polynomials for the variable  $x$ .

```
syms x
hermiteH([0 1 2 3 4], x)

ans =
[ 1, 2*x, 4*x^2 - 2, 8*x^3 - 12*x, 16*x^4 - 48*x^2 + 12]
```

#### Hermite Polynomials for Numeric and Symbolic Arguments

Depending on whether the input is numeric or symbolic, `hermiteH` returns numeric or exact symbolic results.

Find the value of the fifth-degree Hermite polynomial at  $1/3$ . Because the input is numeric, `hermiteH` returns numeric results.

```
hermiteH(5,1/3)
```

```
ans =
    34.2058
```

Find the same result for exact symbolic input. `hermiteH` returns an exact symbolic result.

```
hermiteH(5,sym(1/3))
```

```
ans =
8312/243
```

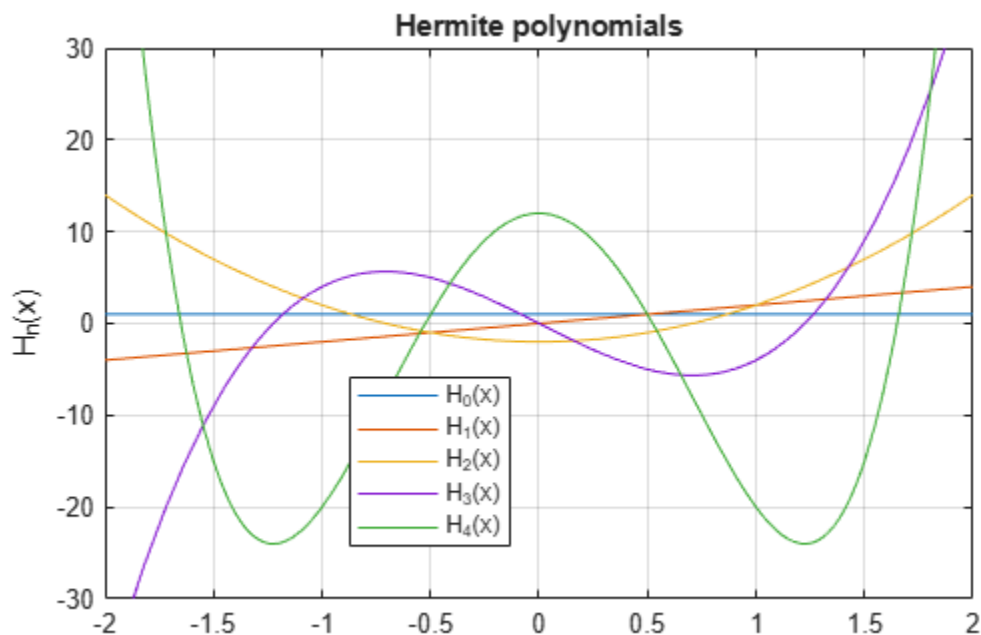
#### Plot Hermite Polynomials

Plot the first five Hermite polynomials.

```
syms x y
fplot(hermiteH(0:4,x))
```

```
axis([-2 2 -30 30])
grid on

ylabel('H_n(x)')
legend('H_0(x)', 'H_1(x)', 'H_2(x)', 'H_3(x)', 'H_4(x)', 'Location', 'Best')
title('Hermite polynomials')
```



## Input Arguments

### **n** — Degree of polynomial

nonnegative integer | symbolic variable | symbolic expression | symbolic function | vector | matrix

Degree of the polynomial, specified as a nonnegative integer, symbolic variable, expression, or function, or as a vector or matrix of numbers, symbolic numbers, variables, expressions, or functions.

### **x** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## More About

### Hermite Polynomials

Hermite polynomials are defined by this recursion formula.

$$H(0, x) = 1, \quad H(1, x) = 2x, \quad H(n, x) = 2xH(n-1, x) - 2(n-1)H(n-2, x)$$

Hermite polynomials in MATLAB satisfy this normalization.

$$\int_{-\infty}^{\infty} (H_n(x))^2 e^{-x^2} dx = 2^n \sqrt{\pi} n!$$

## Tips

- `hermiteH` returns floating-point results for numeric arguments that are not symbolic objects.
- `hermiteH` acts element-wise on nonscalar inputs.
- At least one input argument must be a scalar or both arguments must be vectors or matrices of the same size. If one input argument is a scalar and the other one is a vector or a matrix, then `hermiteH` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

## Version History

Introduced in R2014b

## References

- [1] Hochstrasser, U. W. "Orthogonal Polynomials." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

`chebyshevT` | `chebyshevU` | `gegenbauerC` | `jacobiP` | `laguerreL` | `legendreP`



# hessian

Hessian matrix of symbolic scalar function

## Syntax

`hessian(f,v)`

## Description

`hessian(f,v)` finds the Hessian matrix on page 6-770 of the symbolic scalar function `f` with respect to vector `v` in Cartesian coordinates.

If you do not specify `v`, then `hessian(f)` finds the Hessian matrix of the scalar function `f` with respect to a vector constructed from all symbolic variables found in `f`. The order of variables in this vector is defined by `symvar`.

## Examples

### Find Hessian Matrix of Scalar Function

Find the Hessian matrix of a function by using `hessian`. Then find the Hessian matrix of the same function as the Jacobian of the gradient of the function.

Find the Hessian matrix of this function of three variables:

```
syms x y z
f = x*y + 2*z*x;
hessian(f,[x,y,z])
```

```
ans =
[ 0, 1, 2]
[ 1, 0, 0]
[ 2, 0, 0]
```

Alternatively, compute the Hessian matrix of this function as the Jacobian of the gradient of that function:

```
jacobian(gradient(f))

ans =
[ 0, 1, 2]
[ 1, 0, 0]
[ 2, 0, 0]
```

## Input Arguments

### **f** — Scalar function

symbolic expression | symbolic function

Scalar function, specified as symbolic expression or symbolic function.

**v — Vector with respect to which you find Hessian matrix**

symbolic vector

Vector with respect to which you find Hessian matrix, specified as a symbolic vector. By default, `v` is a vector constructed from all symbolic variables found in `f`. The order of variables in this vector is defined by `symvar`.

If `v` is an empty symbolic object, such as `sym([ ])`, then `hessian` returns an empty symbolic object.

**More About****Hessian Matrix**

The Hessian matrix of  $f(x)$  is the square matrix of the second partial derivatives of  $f(x)$ .

$$H(f) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

**Version History**

Introduced in R2011b

**See Also**

`curl` | `divergence` | `diff` | `gradient` | `jacobian` | `laplacian` | `potential` | `vectorPotential`

# horner

Horner nested polynomial representation

## Syntax

```
horner(p)
horner(p,var)
```

## Description

`horner(p)` returns the Horner form of the polynomial `p`.

`horner(p,var)` uses the variable in `var`.

## Examples

### Horner Form of Polynomial

Find the Horner representation of a polynomial.

```
syms x
p = x^3 - 6*x^2 + 11*x - 6;
horner(p)
```

```
ans =
x*(x*(x - 6) + 11) - 6
```

Specify the variable in the polynomial by using the second argument.

```
syms a b y
p = a*y*x^3 - y*x^2 - 11*b*y*x + 2;
horner(p,x)
```

```
ans =
2 - x*(11*b*y + x*(y - a*x*y))
```

```
horner(p,y)
```

```
ans =
2 - y*(- a*x^3 + x^2 + 11*b*x)
```

## Input Arguments

### **p** — Polynomial

symbolic expression | symbolic function | array of symbolic expressions | array of symbolic functions

Polynomial, specified as a symbolic expression or function, or an array of symbolic expressions or functions.

### **var** — Variable

symbolic variable | array of symbolic variables

Variable, specified as a symbolic variable or an array of symbolic variables.

## **Version History**

**Introduced before R2006a**

## **See Also**

`collect` | `combine` | `expand` | `factor` | `numden` | `rewrite` | `simplify` | `simplifyFraction`

# horzcat

Concatenate symbolic arrays horizontally

## Syntax

```
horzcat(A1,...,AN)
[A1 ... AN]
```

## Description

`horzcat(A1,...,AN)` horizontally concatenates the symbolic arrays `A1,...,AN`. For vectors and matrices, all inputs must have the same number of rows. For multidimensional arrays, `horzcat` concatenates inputs along the second dimension. The remaining dimensions must match.

`[A1 ... AN]` is a shortcut for `horzcat(A1,...,AN)`.

## Examples

### Concatenate Two Symbolic Matrices Horizontally

Create matrices A and B.

```
A = sym('a%d%d',[2 2])
B = sym('b%d%d',[2 2])
```

```
A =
[ a11, a12]
[ a21, a22]
B =
[ b11, b12]
[ b21, b22]
```

Concatenate A and B.

```
horzcat(A,B)
```

```
ans =
[ a11, a12, b11, b12]
[ a21, a22, b21, b22]
```

Alternatively, use the shortcut `[A B]`.

```
[A B]
```

```
ans =
[ a11, a12, b11, b12]
[ a21, a22, b21, b22]
```

### Concatenate Multiple Symbolic Arrays Horizontally

```
A = sym('a%d',[3 1]);
B = sym('b%d%d',[3 3]);
```

```
C = sym('c%d%d',[3 2]);  
horzcat(C,A,B)
```

```
ans =  
[ c11, c12, a1, b11, b12, b13]  
[ c21, c22, a2, b21, b22, b23]  
[ c31, c32, a3, b31, b32, b33]
```

Alternatively, use the shortcut `[C A B]`.

```
[C A B]
```

```
ans =  
[ c11, c12, a1, b11, b12, b13]  
[ c21, c22, a2, b21, b22, b23]  
[ c31, c32, a3, b31, b32, b33]
```

### Concatenate Multidimensional Arrays Horizontally

Create the 3-D symbolic arrays A and B.

```
A = sym('a%d%d',[2 3]);  
A(:,:,2) = -A  
B = sym('b%d%d',[2 2]);  
B(:,:,2) = -B
```

```
A(:,:,1) =  
[ a11, a12, a13]  
[ a21, a22, a23]  
A(:,:,2) =  
[ -a11, -a12, -a13]  
[ -a21, -a22, -a23]
```

```
B(:,:,1) =  
[ b11, b12]  
[ b21, b22]  
B(:,:,2) =  
[ -b11, -b12]  
[ -b21, -b22]
```

Use `horzcat` to concatenate A and B.

```
horzcat(A,B)
```

```
ans(:,:,1) =  
[ a11, a12, a13, b11, b12]  
[ a21, a22, a23, b21, b22]  
ans(:,:,2) =  
[ -a11, -a12, -a13, -b11, -b12]  
[ -a21, -a22, -a23, -b21, -b22]
```

Alternatively, use the shortcut `[A B]`.

```
[A B]
```

```
ans(:,:,1) =  
[ a11, a12, a13, b11, b12]  
[ a21, a22, a23, b21, b22]  
ans(:,:,2) =
```

```
[ -a11, -a12, -a13, -b11, -b12]  
[ -a21, -a22, -a23, -b21, -b22]
```

## Input Arguments

### **A1, ..., AN — Input arrays**

symbolic scalar variable | symbolic matrix variable | symbolic matrix function | symbolic vector | symbolic matrix | symbolic multidimensional array

Input arrays, specified as symbolic scalar variables, matrix variables, matrix functions, or vectors, matrices, or multidimensional arrays of symbolic scalar variables.

## Version History

Introduced before R2006a

### **R2022a: Concatenate symbolic matrix functions horizontally**

The horzcat function accepts input arguments of type `symfunmatrix`.

### **R2021a: Concatenate symbolic matrix variables horizontally**

The horzcat function accepts input arguments of type `symmatrix`.

## See Also

`cat` | `vertcat`

## htrans

Hilbert transform

### Syntax

```
H = htrans(f)
H = htrans(f,transVar)
H = htrans(f,var,transVar)
```

### Description

`H = htrans(f)` returns the Hilbert transform of symbolic function `f`. By default, the independent variable is `t` and the transformation variable is `x`.

`H = htrans(f,transVar)` uses the transformation variable `transVar` instead of `x`.

`H = htrans(f,var,transVar)` uses the independent variable `var` and the transformation variable `transVar` instead of `t` and `x`, respectively.

- If all input arguments are arrays of the same size, then `htrans` acts element-wise.
- If one input is a scalar and the others are arrays of the same size, then `htrans` expands the scalar into an array of the same size.
- If `f` is an array of symbolic expressions with different independent variables, then `var` must be a symbolic array with elements corresponding to the independent variables.

### Examples

#### Transform Symbolic Expression

Compute the Hilbert transform of `sin(t)`. By default, the transform returns a function of `x`.

```
syms t;
f = sin(t);
H = htrans(f)

H = -cos(x)
```

#### Transform Sinc Function

Compute the Hilbert transform of the `sinc(x)` function, which is equal to  $\sin(\pi x)/(\pi x)$ . Express the result as a function of `u`.

```
syms f(x) H(u);
f(x) = sinc(x);
H(u) = htrans(f,u)

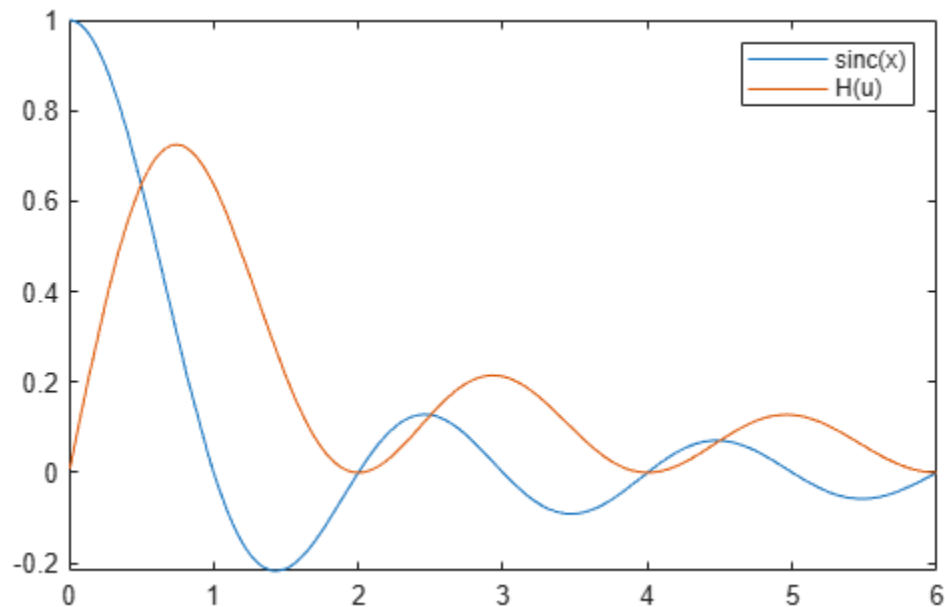
H(u) =
```



$$-\frac{\frac{\cos(\pi u)}{u} - \frac{1}{u}}{\pi}$$

Plot the sinc function and its Hilbert transform.

```
fplot(f(x),[0 6])
hold on
fplot(H(u),[0 6])
legend('sinc(x)','H(u)')
```



### Apply Phase Shifts

Create a sine wave with a positive frequency in real space.

```
syms A x t u;
assume([x t], 'real')
y = A*sin(2*pi*10*t + 5*x)
```

$$y = A \sin(5x + 20\pi t)$$

Apply a -90-degree phase shift to the positive frequency component using the Hilbert transform. Specify the independent variable as  $t$  and the transformation variable as  $u$ .

```
H = htrans(y,t,u)
```

$$H = -A \cos(5x + 20\pi u)$$

Now create a complex signal with negative frequency. Apply a 90-degree phase shift to the negative frequency component using the Hilbert transform.

```
z = A*exp(-1i*10*t)
```

```

z = A e-10 t i
H = htrans(z)
H = A e-10 x i i

```

### Calculate Instantaneous Frequency

Create a real-valued signal  $f(t)$  with two frequency components, 60 Hz and 90 Hz.

```

syms t f(t) F(s)
f(t) = sin(2*pi*60*t) + sin(2*pi*90*t)

f(t) = sin(120 pi t) + sin(180 pi t)

```

Calculate the corresponding analytic signal  $F(s)$  using the Hilbert transform.

```

F(s) = f(s) + 1i*htrans(f(t),s)

F(s) = sin(120 pi s) + sin(180 pi s) - cos(120 pi s) i - cos(180 pi s) i

```

Calculate the instantaneous frequency of  $F(s)$  using

$$f_{\text{instant}}(s) = \frac{1}{2\pi} \frac{d\phi(s)}{ds},$$

where  $\phi(s) = \arg[F(s)]$  is the instantaneous phase of the analytic signal.

```

InstantFreq(s) = diff(angle(F(s)),s)/(2*pi);
assume(s,'real')
simplify(InstantFreq(s))

ans = 75

```

## Input Arguments

### **f** — Input

symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic expression, symbolic function, symbolic vector, or symbolic matrix.

### **var** — Independent variable

t (default) | symbolic variable | symbolic vector | symbolic matrix

Independent variable, specified as a symbolic variable, symbolic vector, or symbolic matrix. This variable is usually in the time domain. If you do not specify the variable, then `htrans` uses `t` by default. If `f` does not contain `t`, then `htrans` uses the function `symvar` to determine the independent variable.

### **transVar** — Transformation variable

x (default) | v | symbolic variable | symbolic vector | symbolic matrix

Transformation variable, specified as a symbolic variable, symbolic vector, or symbolic matrix. This variable is in the same domain as `var`. If you do not specify the variable, then `htrans` uses `x` by default. If `x` is the independent variable of `f`, then `htrans` uses the transformation variable `v`.

## Output Arguments

### H — Hilbert transform of `f`

symbolic expression | symbolic function | symbolic vector | symbolic matrix

Hilbert transform or harmonic conjugate of the input function `f`. The output `H` is a function of the variable specified by `transVar`.

When `htrans` cannot transform the input function, it returns an unevaluated call. To return the original expression, apply the inverse Hilbert transform to the output by using `ihtrans`.

## More About

### Hilbert Transform

The Hilbert transform  $H = H(x)$  of the expression  $f = f(t)$  with respect to the variable  $t$  at point  $x$  is

$$H(x) = \frac{1}{i\pi} \text{p.v.} \int_{-\infty}^{\infty} \frac{f(t)}{x-t} dt.$$

Here, p.v. represents the Cauchy principal value of the integral. The function  $f(t)$  can be complex, but  $t$  and  $x$  must be real.

## Tips

- To compute the inverse Hilbert transform, use `ihtrans`. The Hilbert transform of a function is equal to the negative of its inverse Hilbert transform.
- For a signal in the time domain, the Hilbert transform applies a -90-degree phase shift to positive frequencies of the corresponding Fourier components. It also applies a 90-degree phase shift to negative frequencies.
- For a real-valued signal `a`, the Hilbert transform `b = htrans(a)` returns its harmonic conjugate `b`. The real signal `a = real(z)` and its Hilbert transform `b = imag(z)` form the analytic signal `z = a + 1i*b`.

## Version History

Introduced in R2019a

## See Also

`ihtrans` | `fourier` | `ifourier` | `laplace` | `ilaplace`

## hurwitzZeta

Hurwitz zeta function

### Syntax

```
Z = hurwitzZeta(s,a)
Z = hurwitzZeta(n,s,a)
```

### Description

`Z = hurwitzZeta(s,a)` evaluates the Hurwitz zeta function on page 6-783 for the numeric or symbolic inputs `s` and `a`. The Hurwitz zeta function is defined only if `s` is not 1 and `a` is neither 0 nor a negative integer.

`Z = hurwitzZeta(n,s,a)` returns the `n`th derivative of `hurwitzZeta(s,a)` with respect to the variable `s`.

### Examples

#### Numeric and Symbolic Inputs

Evaluate the Hurwitz zeta function with numeric input arguments.

```
Z = hurwitzZeta(0,1)
```

```
Z =  
-0.5000
```

Compute the symbolic output of `hurwitzZeta` by converting the inputs to symbolic numbers using `sym`.

```
symZ = hurwitzZeta(sym([0 2]),1)
```

```
symZ =  
 $\left(-\frac{1}{2} \frac{\pi^2}{6}\right)$ 
```

Use the `vpa` function to approximate symbolic results with the default 32 digits of precision.

```
valZ = vpa(symZ)
```

```
valZ = (-0.5 1.644934066848226436472415166646)
```

#### Special Values

For certain parameter values, symbolic evaluation of the Hurwitz zeta function returns special values that are related to other symbolic functions.

For `a = 1`, the Hurwitz zeta function returns the Riemann zeta function `zeta`.

```
syms s a;
Z = hurwitzZeta(s,1)
```

$$Z = \zeta(s)$$

For  $s = 2$ , the Hurwitz zeta function returns the first derivative of the digamma function  $\psi$ .

```
Z = hurwitzZeta(2,a)
```

$$Z = \psi'(a)$$

For nonpositive integers  $s$ , the Hurwitz zeta function returns polynomials in terms of  $a$ .

```
Z = hurwitzZeta(0,a)
```

$$Z = \frac{1}{2} - a$$

```
Z = hurwitzZeta(-1,a)
```

$$Z = -\frac{a^2}{2} + \frac{a}{2} - \frac{1}{12}$$

```
Z = hurwitzZeta(-2,a)
```

$$Z = -\frac{a^3}{3} + \frac{a^2}{2} - \frac{a}{6}$$

### Differentiate Hurwitz Zeta Function

Find the first derivative of the Hurwitz zeta function with respect to the variable  $s$ .

```
syms s a
Z = hurwitzZeta(1,s,a)
```

$$Z = \zeta'(s, a)$$

Evaluate the first derivative at  $s = 0$  and  $a = 1$  by using the `subs` function.

```
symZ = subs(Z,[s a],[0 1])
```

$$\text{symZ} = -\frac{\log(2)}{2} - \frac{\log(\pi)}{2}$$

Use the `diff` function to find the first derivative of the Hurwitz zeta function with respect to  $a$ .

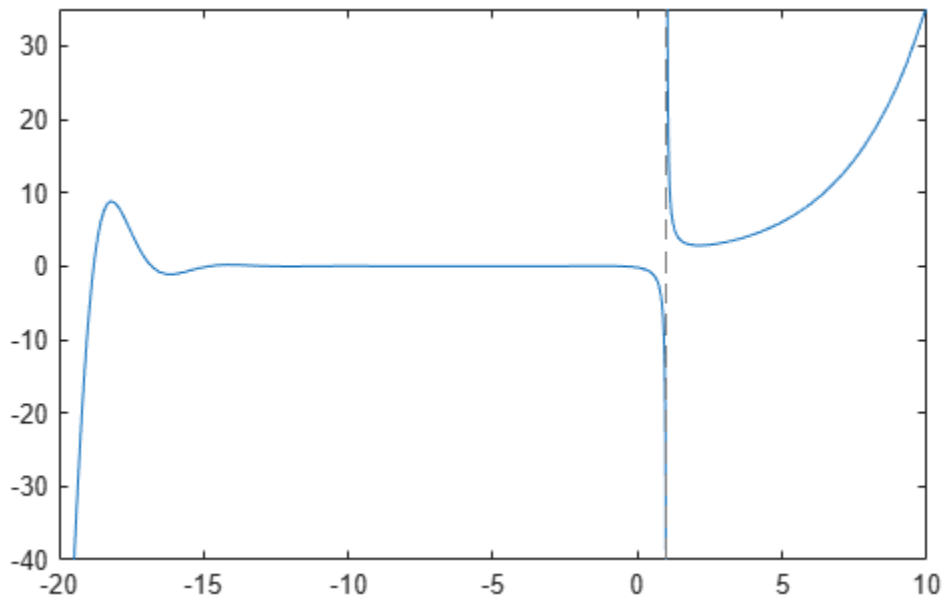
```
Z = diff(hurwitzZeta(s,a),a)
```

$$Z = -s\zeta(s+1, a)$$

### Plot Hurwitz Zeta Function

Plot the Hurwitz zeta function for  $s$  within the interval  $[-20 \ 10]$ , given  $a = 0.7$ .

```
fplot(@(s) hurwitzZeta(s,0.7),[-20 10])
axis([-20 10 -40 35]);
```



### Input Arguments

#### **s — Input**

number | array | symbolic number | symbolic variable | symbolic function | symbolic expression | symbolic array

Input, specified as a number, array, symbolic number, symbolic variable, symbolic function, symbolic expression, or symbolic array. The Hurwitz zeta function is defined only for values of  $s$  not equal to 1.

Data Types: single | double | sym | symfun  
Complex Number Support: Yes

#### **a — Input**

number | array | symbolic number | symbolic variable | symbolic function | symbolic expression | symbolic array

Input, specified as a number, array, symbolic number, symbolic variable, symbolic function, symbolic expression, or symbolic array. The Hurwitz zeta function is defined only for values of  $a$  not equal to 0 or a negative integer.

Data Types: single | double | sym | symfun  
Complex Number Support: Yes

#### **n — Order of derivative**

nonnegative integer

Order of derivative, specified as a nonnegative integer.

## More About

### Hurwitz Zeta Function

The Hurwitz zeta function is defined by the formula

$$\zeta(s, a) = \sum_{k=0}^{\infty} \frac{1}{(k+a)^s}.$$

The summation series converges only when  $\text{Re}(s) > 1$  and  $a$  is neither 0 nor a negative integer. Analytic continuation extends the definition of the function to the entire complex plane, except for a simple pole at  $s = 1$ .

## Tips

- Floating-point evaluation of the Hurwitz zeta function can be slow for complex arguments or high-precision numbers. To increase the computational speed, you can reduce the floating-point precision by using the `vpa` and `digits` functions. For more information, see “Increase Speed by Reducing Precision” on page 3-386.
- The Hurwitz zeta function is related to other special functions. For example, it can be expressed in terms of the polylogarithm  $\text{Li}_s(z)$  and the gamma function  $\Gamma(z)$ :

$$\zeta(1-s, a) = \frac{\Gamma(s)}{(2\pi)^s} \left[ e^{-ins/2} \text{Li}_s(e^{2\pi ia}) + e^{ins/2} \text{Li}_s(e^{-2\pi ia}) \right].$$

Here,  $\text{Re}(s) > 0$  and  $\text{Im}(a) > 0$ , or  $\text{Re}(s) > 1$  and  $\text{Im}(a) = 0$ .

## Version History

Introduced in R2019a

## References

- [1] Olver, F. W. J., A. B. Olde Daalhuis, D. W. Lozier, B. I. Schneider, R. F. Boisvert, C. W. Clark, B. R. Miller, and B. V. Saunders, eds., Chapter 25. Zeta and Related Functions, *NIST Digital Library of Mathematical Functions*, Release 1.0.20, Sept. 15, 2018.

## See Also

`bernoulli` | `gamma` | `psi` | `polylog` | `zeta`

# hypergeom

Hypergeometric function

## Syntax

`hypergeom(a,b,z)`

## Description

`hypergeom(a,b,z)` represents the generalized hypergeometric function on page 6-786.

## Examples

### Hypergeometric Function for Numeric and Symbolic Arguments

Depending on whether the input is floating point or symbolic, `hypergeom` returns floating point or symbolic results.

Compute the hypergeometric function for these numbers. Because these numbers are floating point, `hypergeom` returns floating-point results.

```
A = [hypergeom([1 2], 2.5, 2),...
      hypergeom(1/3, [2 3], pi),...
      hypergeom([1 1/2], 1/3, 3*i)]

A =
    -1.2174 - 0.8330i    1.2091 + 0.0000i    -0.2028 + 0.2405i
```

Return exact symbolic results by converting at least one of the inputs to symbolic form by using `sym`. For most symbolic (exact) inputs, `hypergeom` returns unresolved symbolic calls.

```
symA = [hypergeom([1 2], 2.5, sym(2)),...
        hypergeom(1/3, [2 3], sym(pi)),...
        hypergeom([1 1/2], sym(1/3), 3*i)]

symA =
[ hypergeom([1, 2], 5/2, 2), hypergeom(1/3, [2, 3], pi), hypergeom([1/2, 1], 1/3, 3i)]
```

Convert the symbolic result to high-precision floating-point by using `vpa`.

```
vpa(symA)

ans =
[ - 1.2174189301051728850455150601879 - 0.83304055090469367131547768563638i,...
  1.2090631887094273193917339575087,...
  - 0.20275169745081962937527290365593 + 0.24050134226872040357481317881983i]
```

### Special Values of Hypergeometric Function

Show that `hypergeom` returns special values for certain input values.



```
syms a b c d x
hypergeom([], [], x)
```

```
ans =
exp(x)
```

```
hypergeom([a b c d], [a b c d], x)
```

```
ans =
exp(x)
```

```
hypergeom(a, [], x)
```

```
ans =
1/(1 - x)^a
```

Show that the hypergeometric function is always 1 at 0.

```
syms a b c d
hypergeom([a b], [c d], 0)
```

```
ans =
1
```

If, after canceling identical parameters in the first two arguments, the list of upper parameters contains 0, the resulting hypergeometric function is constant with the value 1. For details, see “Algorithms” on page 6-787.

```
hypergeom([0 0 2 3], [a 0 4], x)
```

```
ans =
1
```

If, after canceling identical parameters in the first two arguments, the upper parameters contain a negative integer larger than the largest negative integer in the lower parameters, the hypergeometric function is a polynomial.

```
hypergeom([-4 -2 3], [-3 1 4], x)
```

```
ans =
(3*x^2)/5 - 2*x + 1
```

Hypergeometric functions reduce to other special functions for certain input values.

```
hypergeom([1], [a], x)
hypergeom([a], [a, b], x)
```

```
ans =
(exp(x/2)*whittakerM(1 - a/2, a/2 - 1/2, -x))/(-x)^(a/2)
```

```
ans =
x^(1/2 - b/2)*gamma(b)*besseli(b - 1, 2*x^(1/2))
```

## Handling Expressions That Contain Hypergeometric Functions

Many symbolic functions, such as `diff` and `taylor`, handle expressions containing `hypergeom`.

Differentiate this expression containing the hypergeometric function.

```
syms a b c d x
diff(1/x*hypergeom([a b],[c d],x), x)

ans =
(a*b*hypergeom([a + 1, b + 1], [c + 1, d + 1], x))/(c*d*x)...
- hypergeom([a, b], [c, d], x)/x^2
```

Compute the Taylor series of this hypergeometric function.

```
taylor(hypergeom([1 2],3,x), x)

ans =
(2*x^5)/7 + x^4/3 + (2*x^3)/5 + x^2/2 + (2*x)/3 + 1
```

## Input Arguments

### **a** — Upper parameters of hypergeometric function

number | vector | symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector

Upper parameters of hypergeometric function, specified as a number, variable, symbolic expression, symbolic function, or vector.

### **b** — Lower parameters of hypergeometric function

number | vector | symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector

Lower parameters of hypergeometric function, specified as a number, variable, symbolic expression, symbolic function, or vector.

### **z** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## More About

### Generalized Hypergeometric Function

The generalized hypergeometric function of order  $p$ ,  $q$  is defined as follows.

$${}_pF_q(a; b; z) = {}_pF_q(a_1, \dots, a_j, \dots, a_p; b_1, \dots, b_k, \dots, b_q; z) = \sum_{n=0}^{\infty} \left( \frac{(a_1)_n \dots (a_j)_n \dots (a_p)_n}{(b_1)_n \dots (b_k)_n \dots (b_q)_n} \right) \left( \frac{z^n}{n!} \right).$$

Here  $a = [a_1, a_2, \dots, a_p]$  and  $b = [b_1, b_2, \dots, b_q]$  are vectors of lengths  $p$  and  $q$ , respectively.

$(a)_k$  and  $(b)_k$  are Pochhammer symbols.

For empty vectors  $a$  and  $b$ , `hypergeom` is defined as follows.

$${}_0F_q(; b; z) = \sum_{k=0}^{\infty} \frac{1}{(b_1)_k (b_2)_k \dots (b_q)_k} \left( \frac{z^k}{k!} \right)$$

$${}_pF_0(a; ; z) = \sum_{k=0}^{\infty} (a_1)_k (a_2)_k \dots (a_p)_k \left( \frac{z^k}{k!} \right)$$

$${}_0F_0(; ; z) = \sum_{k=0}^{\infty} \left( \frac{z^k}{k!} \right) = e^z.$$

### Pochhammer Symbol

$$(x)_n = \frac{\Gamma(x+n)}{\Gamma(x)}.$$

If  $n$  is a positive integer, then  $(x)_n = x(x+1)\dots(x+n-1)$ .

### Algorithms

The hypergeometric function is

$${}_pF_q(a; b; z) = {}_pF_q(a_1, \dots, a_p; b_1, \dots, b_q; z) = \sum_{n=0}^{\infty} \left( \frac{(a_1)_n \dots (a_p)_n}{(b_1)_n \dots (b_q)_n} \right) \left( \frac{z^n}{n!} \right).$$

- The hypergeometric function has convergence criteria:
  - Converges if  $p \leq q$  and  $|z| < \infty$ .
  - Converges if  $p = q + 1$  and  $|z| < 1$ . For  $|z| \geq 1$ , the series diverges, and is defined by analytic continuation.
  - Diverges if  $p > q + 1$  and  $z \neq 0$ . Here, the series is defined by an asymptotic expansion of  ${}_pF_q(a; b; z)$  around  $z = 0$ . The branch cut is the positive real axis.
- The function is a polynomial, called the hypergeometric polynomial, if any  $a_j$  is a nonpositive integer.
- The function is undefined:
  - If any  $b_k$  is a nonpositive integer such that  $b_k > a_j$  where  $a_j$  is also a nonpositive integer, because division by 0 occurs
  - If any  $b_k$  is a nonpositive integer and no  $a_j$  is a nonpositive integer
- The function has reduced order when upper and lower parameter values are equal and cancel. If  $r$  values of the upper and lower parameters are equal (that is,  $a = [a_1, \dots, a_{p-r}, c_1, \dots, c_r]$ ,  $b = [b_1, \dots, b_{q-r}, c_1, \dots, c_r]$ ), then the order  $(p, q)$  of  ${}_pF_q(a; b; z)$  is reduced to  $(p-r, q-r)$ :

$${}_pF_q(a_1, \dots, a_{p-r}, c_1, \dots, c_r; b_1, \dots, b_{q-r}, c_1, \dots, c_r; z) = {}_{p-r}F_{q-r}(a_1, \dots, a_{p-r}; b_1, \dots, b_{q-r}; z)$$

This rule applies even if any  $c_i$  is zero or a negative integer [2].

- ${}_pF_q(a; b; z)$  is symmetric. That is, it does not depend on the order  $a_1, a_2, \dots$  in  $a$  or  $b_1, b_2, \dots$  in  $b$ .
- $U(z) = {}_pF_q(a; b; z)$  satisfies the differential equation in  $z$

$$[\delta(\delta + b - 1) - z(\delta + a)]U(z) = 0, \quad \delta = z \frac{\partial}{\partial z}.$$

Here,  $(\delta + a)$  represents

$$\prod_{i=1}^p (\delta + a_i).$$

And  $(\delta + b)$  represents

$$\prod_{j=1}^q (\delta + b_j).$$

Thus, the order of this differential equation is  $\max(p, q + 1)$ , and the hypergeometric function is only one of its solutions. If  $p < q + 1$ , this differential equation has a regular singularity at  $z = 0$  and an irregular singularity at  $z = \infty$ . If  $p = q + 1$ , the points  $z = 0$ ,  $z = 1$ , and  $z = \infty$  are regular singularities, which explains the convergence properties of the hypergeometric series.

- The hypergeometric function has these special values:
  - ${}_pF_p(a; a; z) = {}_0F_0(;; z) = e^z$ .
  - ${}_pF_q(a; b; z) = 1$  if the list of upper parameters  $a$  contains more 0s than the list of lower parameters  $b$ .
  - ${}_pF_q(a; b; 0) = 1$ .

## Version History

Introduced before R2006a

## References

- [1] Oberhettinger, F. "Hypergeometric Functions." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.
- [2] Luke, Y.L. "The Special Functions and Their Approximations", Vol. 1, Academic Press, New York, 1969.
- [3] Prudnikov, A.P., Yu.A. Brychkov, and O.I. Marichev, "Integrals and Series", Vol. 3: More Special Functions, Gordon and Breach, 1990.

## See Also

kummerU | meijerG | whittakerM | whittakerW

# ifourier

Inverse Fourier transform

## Syntax

```
ifourier(F)
ifourier(F,transVar)
ifourier(F,var,transVar)
```

## Description

`ifourier(F)` returns the “Inverse Fourier Transform” on page 6-792 of *F*. By default, the independent variable is *w* and the transformation variable is *x*. If *F* does not contain *w*, `ifourier` uses the function `symvar`.

`ifourier(F,transVar)` uses the transformation variable `transVar` instead of *x*.

`ifourier(F,var,transVar)` uses the independent variable `var` and the transformation variable `transVar` instead of *w* and *x*, respectively.

## Examples

### Inverse Fourier Transform of Symbolic Expression

Compute the inverse Fourier transform of  $\exp(-w^2/4)$ . By default, the inverse transform is in terms of *x*.

```
syms w
F = exp(-w^2/4);
ifourier(F)

ans =
exp(-x^2)/pi^(1/2)
```

### Default Independent Variable and Transformation Variable

Compute the inverse Fourier transform of  $\exp(-w^2-a^2)$ . By default, the independent and transformation variables are *w* and *x*, respectively.

```
syms a w t
F = exp(-w^2-a^2);
ifourier(F)

ans =
exp(- a^2 - x^2/4)/(2*pi^(1/2))
```

Specify the transformation variable as *t*. If you specify only one variable, that variable is the transformation variable. The independent variable is still *w*.

```
ifourier(F,t)

ans =
exp(- a^2 - t^2/4)/(2*pi^(1/2))
```

### Inverse Fourier Transforms Involving Dirac and Heaviside Functions

Compute the inverse Fourier transform of expressions in terms of Dirac and Heaviside functions.

```
syms t w
ifourier(dirac(w), w, t)

ans =
1/(2*pi)

f = 2*exp(-abs(w))-1;
ifourier(f,w,t)

ans =
-(2*pi*dirac(t) - 4/(t^2 + 1))/(2*pi)

f = exp(-w)*heaviside(w);
ifourier(f,w,t)

ans =
-1/(2*pi*(- 1 + t*1i))
```

### Specify Parameters of Inverse Fourier Transform

Specify parameters of the inverse Fourier transform.

Compute the inverse Fourier transform of this expression using the default values of the Fourier parameters  $c = 1$ ,  $s = -1$ . For details, see “Inverse Fourier Transform” on page 6-792.

```
syms t w
f = -(sqrt(sym(pi))*w*exp(-w^2/4)*i)/2;
ifourier(f,w,t)

ans =
t*exp(-t^2)
```

Change the Fourier parameters to  $c = 1$ ,  $s = 1$  by using `sympref`, and compute the transform again. The sign of the result changes.

```
sympref('FourierParameters',[1 1]);
ifourier(f,w,t)

ans =
-t*exp(-t^2)
```

Change the Fourier parameters to  $c = 1/(2\pi)$ ,  $s = 1$ . The result changes.

```
sympref('FourierParameters',[1/(2*sym(pi)) 1]);
ifourier(f,w,t)
```

```
ans =
-2*pi*t*exp(-t^2)
```

Symbolic settings that you set using `sympref` persist through your current and future MATLAB sessions. Restore the default values of `c` and `s` by setting `FourierParameters` to `'default'`.

```
sympref('FourierParameters','default');
```

## Inverse Fourier Transform of Array Inputs

Find the inverse Fourier transform of the matrix `M`. Specify the independent and transformation variables for each matrix entry by using matrices of the same size. When the arguments are nonscalars, `ifourier` acts on them element-wise.

```
syms a b c d w x y z
M = [exp(x), 1; sin(y), i*z];
vars = [w, x; y, z];
transVars = [a, b; c, d];
ifourier(M,vars,transVars)

ans =
[      exp(x)*dirac(a),      dirac(b)]
[ (dirac(c - 1)*1i)/2 - (dirac(c + 1)*1i)/2, dirac(1, d)]
```

If `ifourier` is called with both scalar and nonscalar arguments, then it expands the scalars to match the nonscalars by using scalar expansion. Nonscalar arguments must be the same size.

```
ifourier(x,vars,transVars)

ans =
[ x*dirac(a), -dirac(1, b)*1i]
[ x*dirac(c),      x*dirac(d)]
```

## If Inverse Fourier Transform Cannot Be Found

If `ifourier` cannot transform the input, then it returns an unevaluated call to `fourier`.

```
syms F(w) t
f = ifourier(F,w,t)

f =
fourier(F(w), w, -t)/(2*pi)
```

## Input Arguments

### F — Input

symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic expression, function, vector, or matrix.

### var — Independent variable

x (default) | symbolic variable

Independent variable, specified as a symbolic variable. This variable is often called the "frequency variable." If you do not specify the variable, then `ifourier` uses `w`. If `F` does not contain `w`, then `ifourier` uses the function `symvar` to determine the independent variable.

### **transVar — Transformation variable**

`x` (default) | `t` | symbolic variable | symbolic expression | symbolic vector | symbolic matrix

Transformation variable, specified as a symbolic variable, expression, vector, or matrix. It is often called the "time variable" or "space variable." By default, `ifourier` uses `x`. If `x` is the independent variable of `F`, then `ifourier` uses `t`.

## **More About**

### **Inverse Fourier Transform**

The inverse Fourier transform of the expression  $F = F(w)$  with respect to the variable `w` at the point `x` is

$$f(x) = \frac{|s|}{2\pi c} \int_{-\infty}^{\infty} F(w) e^{-iswx} dw.$$

`c` and `s` are parameters of the inverse Fourier transform. The `ifourier` function uses `c = 1`, `s = -1`.

## **Tips**

- If any argument is an array, then `ifourier` acts element-wise on all elements of the array.
- If the first argument contains a symbolic function, then the second argument must be a scalar.
- The toolbox computes the inverse Fourier transform via the Fourier transform:

$$\text{ifourier}(F, w, t) = \frac{1}{2\pi} \text{fourier}(F, w, -t).$$

If `ifourier` cannot find an explicit representation of the inverse Fourier transform, then it returns results in terms of the Fourier transform.

- To compute the Fourier transform, use `fourier`.

## **Version History**

Introduced before R2006a

## **References**

- [1] Oberhettinger, F. "Tables of Fourier Transforms and Fourier Transforms of Distributions." Springer, 1990.

## **See Also**

`fourier` | `ilaplace` | `iztrans` | `laplace` | `sympref` | `ztrans`



**Topics**

"Fourier and Inverse Fourier Transforms" on page 3-194

# igamma

Incomplete gamma function

## Syntax

```
g = igamma(nu,z)
```

## Description

`g = igamma(nu,z)` returns the upper incomplete gamma function.

`igamma` uses the definition of the upper incomplete gamma function on page 6-796. However, the default MATLAB `gammainc` function uses the definition of the regularized lower incomplete gamma function on page 6-796, where `gammainc(z,nu) = 1 - igamma(nu,z)/gamma(nu)`. The order of input arguments differs between these functions.

## Examples

### Compute Incomplete Gamma Function for Numeric and Symbolic Arguments

Depending on its arguments, `igamma` returns floating-point or exact symbolic results.

Compute the upper incomplete gamma function for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
g = [igamma(0,1), igamma(3,sqrt(2)), igamma(pi,exp(1)), igamma(3,Inf)]
```

```
g = 1×4
```

```
0.2194    1.6601    1.1979    0
```

Compute the upper incomplete gamma function for the numbers converted to symbolic objects.

```
symg = [igamma(sym(0),1), igamma(3,sqrt(sym(2))), ...  
igamma(sym(pi),exp(sym(1))), igamma(3,sym(Inf))]
```

```
symg = (-ei(-1) e-√2 (2√2 + 4) Γ(π, e) 0)
```

Use `vpa` to approximate symbolic results with floating-point numbers.

```
vpag = vpa(symg)
```

```
vpag = (0.21938393439552027367716377546012 1.6601049038903044104826564373576 1.1979302081330828196865548
```

## Compute Lower Incomplete Gamma Function

`igamma` is implemented according to the definition of the upper incomplete gamma function. If you want to compute the lower incomplete gamma function, convert results returned by `igamma` as follows.

Compute the upper incomplete gamma function for these arguments using `igamma`.

```
z = [-5/3, -1/2, 0, 1/3];
gupper = igamma(1/3,z)

gupper = 1×4 complex

-0.3900 - 5.3156i    1.3156 - 2.3614i    2.6789 + 0.0000i    0.7569 + 0.0000i
```

Subtract these results out of the gamma function to obtain the lower incomplete gamma function.

```
glower = gamma(1/3) - gupper

glower = 1×4 complex

3.0689 + 5.3156i    1.3634 + 2.3614i    0.0000 + 0.0000i    1.9220 + 0.0000i
```

For comparison, you can also calculate the regularized lower incomplete gamma function by using the MATLAB `gammainc` function. The regularized definition differs by the gamma function factor.

```
glower2 = gammainc(z,1/3)*gamma(1/3)

glower2 = 1×4 complex

3.0689 + 5.3156i    1.3634 + 2.3614i    0.0000 + 0.0000i    1.9220 + 0.0000i
```

Because `gammainc` does not accept complex arguments, if one or both arguments are complex numbers, use `igamma` to compute the regularized lower incomplete gamma function.

```
gcomplex = 1 - igamma(1/2,1i)/gamma(1/2)

gcomplex =
0.9693 + 0.4741i
```

## Input Arguments

### **nu** — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

### **z** — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## More About

### Upper Incomplete Gamma Function

The following integral defines the upper incomplete gamma function:

$$\Gamma(\eta, z) = \int_z^{\infty} t^{\eta-1} e^{-t} dt$$

### Regularized Lower Incomplete Gamma Function

The following integral defines the regularized lower incomplete gamma function:

$$\gamma(z, \eta) = \frac{1}{\Gamma(\eta)} \int_0^z t^{\eta-1} e^{-t} dt,$$

where the gamma function  $\Gamma(\eta)$  is defined by

$$\Gamma(\eta) = \int_0^{\infty} t^{\eta-1} e^{-t} dt.$$

## Tips

- The MATLAB `gammainc` function does not accept complex arguments. For complex arguments, use `igamma`.
- `gammainc(z, nu) = 1 - igamma(nu, z)/gamma(nu)` represents the regularized lower incomplete gamma function in terms of the upper incomplete gamma function.
- `igamma(nu, z) = gamma(nu)*(1 - gammainc(z, nu))` represents the upper incomplete gamma function in terms of the regularized lower incomplete gamma function.
- `gammainc(z, nu, "upper") = igamma(nu, z)/gamma(nu)`.

## Version History

Introduced in R2014a

## See Also

`gammainc` | `gamma` | `ei` | `erfc` | `factorial` | `int`

# ihtrans

Inverse Hilbert transform

## Syntax

```
f = ihtrans(H)
f = ihtrans(H,transVar)
f = ihtrans(H,var,transVar)
```

## Description

`f = ihtrans(H)` returns the inverse Hilbert transform of symbolic function `H`. By default, the independent variable is `x` and the transformation variable is `t`.

`f = ihtrans(H,transVar)` uses the transformation variable `transVar` instead of `t`.

`f = ihtrans(H,var,transVar)` uses the independent variable `var` and the transformation variable `transVar` instead of `x` and `t`, respectively.

- If all input arguments are arrays of the same size, then `ihtrans` acts element-wise.
- If one input is a scalar and the others are arrays of the same size, then `ihtrans` expands the scalar into an array of the same size.
- If `f` is an array of symbolic expressions with different independent variables, then `var` must be a symbolic array with elements corresponding to the independent variables.

## Examples

### Inverse Hilbert Transform of Symbolic Expression

Compute the inverse Hilbert transform of  $\cos(x)$ . By default, the inverse transform returns a function of `t`.

```
syms x;
f = cos(x);
H = ihtrans(f)

H = -sin(t)
```

### Inverse Hilbert Transform of Sinc Function

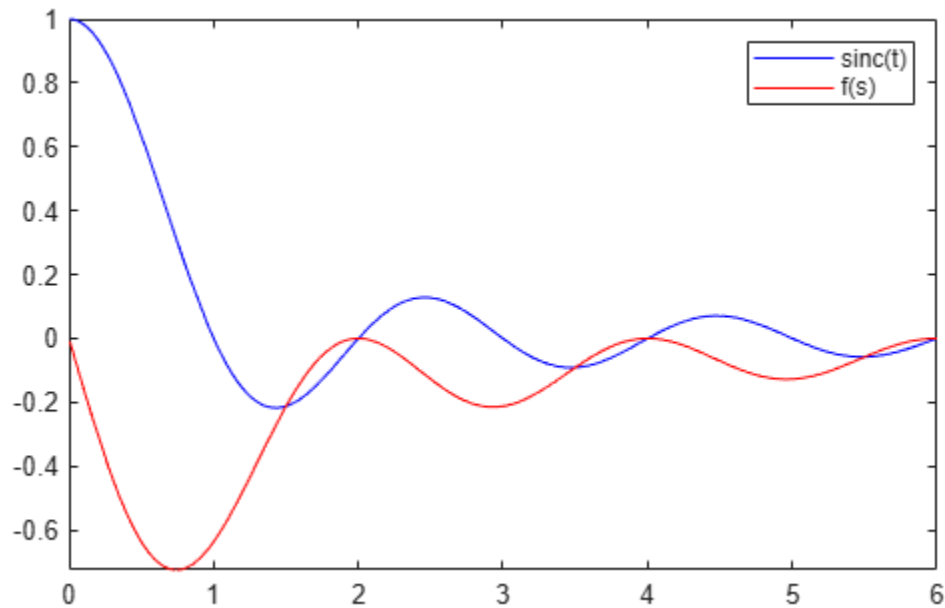
Compute the inverse Hilbert transform of the `sinc(t)` function, which is equal to  $\sin(\pi t)/(\pi t)$ . Express the result as a function of `s`.

```
syms H(t) f(s);
H(t) = sinc(t);
f(s) = ihtrans(H,s)
```

$$f(s) = \frac{\frac{\cos(\pi s)}{s} - \frac{1}{s}}{\pi}$$

Plot the sinc function and its inverse Hilbert transform.

```
fplot(H(t),[0 6],'b')
hold on
fplot(f(s),[0 6],'r')
legend('sinc(t)','f(s)')
```



### Apply Phase Shifts

Create a sine wave with a positive frequency in real space.

```
syms A x t u;
assume([x t],'real')
H = A*sin(2*pi*10*t + 5*x)
```

$$H = A \sin(5x + 20\pi t)$$

Apply a 90-degree phase shift to the positive frequency component using the inverse Hilbert transform. Specify the independent variable as  $x$  and the transformation variable as  $u$ , respectively.

```
f = ihtrans(H,x,u)
```

$$f = A \cos(5u + 20\pi t)$$

Now create a complex signal with negative frequency. Apply a -90-degree phase shift to the negative frequency component using the inverse Hilbert transform.

```
Z = A*exp(-1i*10*t)
```

```
Z = A e-10ti
```

```
f = ihtrans(Z)
```

```
f = -A e-10ui i
```

## Calculate Instantaneous Frequency

Create a real-valued signal  $f(s)$  with two frequency components, 60 Hz and 90 Hz.

```
syms s f(x) F(t)
f(s) = sin(2*pi*60*s) + sin(2*pi*90*s)
```

```
f(s) = sin(120 π s) + sin(180 π s)
```

Calculate the corresponding analytic signal  $F(t)$  using the inverse Hilbert transform.

```
F(t) = ihtrans(f(s),t) + 1i*f(t)
```

```
F(t) = cos(120 π t) + cos(180 π t) + sin(120 π t) i + sin(180 π t) i
```

Calculate the instantaneous frequency of  $F(t)$  using

$$f_{\text{instant}}(t) = \frac{1}{2\pi} \frac{d\phi(t)}{dt},$$

where  $\phi(t) = \arg[F(t)]$  is the instantaneous phase of the analytic signal.

```
InstantFreq(t) = diff(angle(F(t)),t)/(2*pi);
assume(t,'real')
simplify(InstantFreq(t))
```

```
ans = 75
```

## Input Arguments

### H — Input

symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic expression, symbolic function, symbolic vector, or symbolic matrix.

### var — Independent variable

x (default) | symbolic variable | symbolic vector | symbolic matrix

Independent variable, specified as a symbolic variable, symbolic vector, or symbolic matrix. This variable is usually in the time domain. If you do not specify the variable, then `ihtrans` uses `x` by default. If `H` does not contain `x`, then `ihtrans` uses the function `symvar` to determine the independent variable.

### transVar — Transformation variable

t (default) | u | symbolic variable | symbolic vector | symbolic matrix

Transformation variable, specified as a symbolic variable, symbolic vector, or symbolic matrix. This variable is in the same domain as `var`. If you do not specify the variable, then `ihtrans` uses `t` by default. If `t` is the independent variable of `H`, then `ihtrans` uses the transformation variable `u`.

## Output Arguments

### **f** — Inverse Hilbert transform of **H**

symbolic expression | symbolic function | symbolic vector | symbolic matrix

Inverse Hilbert transform of the input function `H`. The output `f` is a function of the variable specified by `transVar`.

When `ihtrans` cannot transform the input function, it returns an unevaluated call. To return the original expression, apply the Hilbert transform to the output by using `htrans`.

## More About

### Inverse Hilbert Transform

The inverse Hilbert transform  $f = f(t)$  of the expression  $H = H(x)$  with respect to the variable  $x$  at point  $t$  is

$$f(t) = \frac{1}{\pi} \text{p.v.} \int_{-\infty}^{\infty} \frac{H(x)}{x - t} dx.$$

Here, p.v. represents the Cauchy principal value of the integral. The function  $H(x)$  can be complex, but  $x$  and  $t$  must be real.

## Tips

- To compute the Hilbert transform, use `htrans`. The inverse Hilbert transform of a function is equal to the negative of its Hilbert transform.
- For a signal in the time domain, the inverse Hilbert transform applies a 90-degree phase shift to negative frequencies of the corresponding Fourier components. It also applies a -90-degree phase shift to positive frequencies.
- A real-valued signal `b` is the harmonic conjugate of its inverse Hilbert transform `a = ihtrans(b)`. The inverse Hilbert transform `a = real(z)` and the signal `b = imag(z)` form the analytic signal `z = a + 1i*b`.

## Version History

Introduced in R2019a

## See Also

`htrans` | `fourier` | `ifourier` | `laplace` | `ilaplace`



# ilaplace

Inverse Laplace transform

## Syntax

```
f = ilaplace(F)
f = ilaplace(F,transVar)
f = ilaplace(F,var,transVar)
```

## Description

`f = ilaplace(F)` returns the “Inverse Laplace Transform” on page 6-804 of F. By default, the independent variable is `s` and the transformation variable is `t`. If `F` does not contain `s`, `ilaplace` uses the function `symvar`.

`f = ilaplace(F,transVar)` uses the transformation variable `transVar` instead of `t`.

`f = ilaplace(F,var,transVar)` uses the independent variable `var` and the transformation variable `transVar` instead of `s` and `t`, respectively.

## Examples

### Inverse Laplace Transform of Symbolic Expression

Compute the inverse Laplace transform of  $1/s^2$ . By default, the inverse transform is in terms of `t`.

```
syms s
F = 1/s^2;
f = ilaplace(F)

f = t
```

### Default Independent Variable and Transformation Variable

Compute the inverse Laplace transform of  $1/(s-a)^2$ . By default, the independent and transformation variables are `s` and `t`, respectively.

```
syms a s
F = 1/(s-a)^2;
f = ilaplace(F)

f = t e^{at}
```

Specify the transformation variable as `x`. If you specify only one variable, that variable is the transformation variable. The independent variable is still `s`.

```
syms x
f = ilaplace(F,x)
```

$$f = x e^{ax}$$

Specify both the independent and transformation variables as  $a$  and  $x$  in the second and third arguments, respectively.

$$f = \text{ilaplace}(F, a, x)$$

$$f = x e^{sx}$$

### Inverse Laplace Transforms Involving Dirac and Heaviside Functions

Compute the following inverse Laplace transforms that involve the Dirac and Heaviside functions.

```
syms s t
f1 = ilaplace(1,s,t)

f1 =  $\delta(t)$ 

F = exp(-2*s)/(s^2+1);
f2 = ilaplace(F,s,t)

f2 = heaviside(t-2) sin(t-2)
```

### Inverse Laplace Transform as Convolution

Create two functions  $f(t) = \text{heaviside}(t)$  and  $g(t) = \exp(-t)$ . Find the Laplace transforms of the two functions by using `laplace`. Because the Laplace transform is defined as a unilateral or one-sided transform, it only applies to the signals in the region  $t \geq 0$ .

```
syms t positive
f(t) = heaviside(t);
g(t) = exp(-t);
F = laplace(f);
G = laplace(g);
```

Find the inverse Laplace transform of the product of the Laplace transforms of the two functions.

$$h = \text{ilaplace}(F*G)$$

$$h = 1 - e^{-t}$$

According to the convolution theorem for causal signals, the inverse Laplace transform of this product is equal to the convolution of the two functions, which is the integral  $\int_0^t f(\tau) g(t - \tau) d\tau$  with  $t \geq 0$ . Find this integral.

```
syms tau
conv_fg = int(f(tau)*g(t-tau),tau,0,t)

conv_fg = 1 - e^{-t}
```

Show that the inverse Laplace transform of the product of the Laplace transforms is equal to the convolution, where  $h$  is equal to `conv_fg`.

```
isAlways(h == conv_fg)

ans = logical
      1
```

### Inverse Laplace Transform of Array Inputs

Find the inverse Laplace transform of the matrix M. Specify the independent and transformation variables for each matrix entry by using matrices of the same size. When the arguments are nonscalars, `ilaplace` acts on them element-wise.

```
syms a b c d w x y z
M = [exp(x) 1; sin(y) 1i*z];
vars = [w x; y z];
transVars = [a b; c d];
f = ilaplace(M,vars,transVars)
```

```
f =
      (
          ex δ(a)      δ(b)
      ilaplace(sin(y),y,c) δ'(d) i)
```

If `ilaplace` is called with both scalar and nonscalar arguments, then it expands the scalars to match the nonscalars by using scalar expansion. Nonscalar arguments must be the same size.

```
syms w x y z a b c d
f = ilaplace(x,vars,transVars)
```

```
f =
      (
          x δ(a)  δ'(b)
          x δ(c)  x δ(d))
```

### Inverse Laplace Transform of Symbolic Function

Compute the Inverse Laplace transform of symbolic functions. When the first argument contains symbolic functions, then the second argument must be a scalar.

```
syms F1(x) F2(x) a b
F1(x) = exp(x);
F2(x) = x;
f = ilaplace([F1 F2],x,[a b])
```

```
f = (ilaplace(ex,x,a) δ'(b))
```

### If Inverse Laplace Transform Cannot Be Found

If `ilaplace` cannot compute the inverse transform, then it returns an unevaluated call to `ilaplace`.

```
syms F(s) t
F(s) = exp(s);
f(t) = ilaplace(F,s,t)
```

```
f(t) = ilaplace(es,s,t)
```

Return the original expression by using `laplace`.

```
F(s) = laplace(f,t,s)
```

```
F(s) = es
```

## Input Arguments

### F — Input

symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic expression, function, vector, or matrix.

### var — Independent variable

s (default) | symbolic variable | symbolic expression | symbolic vector | symbolic matrix

Independent variable, specified as a symbolic variable, expression, vector, or matrix. This variable is often called the "complex frequency variable." If you do not specify the variable, then `ilaplace` uses `s`. If `F` does not contain `s`, then `ilaplace` uses the function `symvar` to determine the independent variable.

### transVar — Transformation variable

t (default) | x | symbolic variable | symbolic expression | symbolic vector | symbolic matrix

Transformation variable, specified as a symbolic variable, expression, vector, or matrix. It is often called the "time variable" or "space variable." By default, `ilaplace` uses `t`. If `t` is the independent variable of `F`, then `ilaplace` uses `x`.

## More About

### Inverse Laplace Transform

The inverse Laplace transform of  $F(s)$  is the signal  $f(t)$  such that `laplace(f(t),t,s)` is  $F(s)$ . The inverse Laplace transform `ilaplace(F(s),s,t)` may only match the original signal  $f(t)$  for  $t \geq 0$ .

## Tips

- If any argument is an array, then `ilaplace` acts element-wise on all elements of the array.
- If the first argument contains a symbolic function, then the second argument must be a scalar.
- To compute the direct Laplace transform, use `laplace`.
- For a signal  $f(t)$ , computing the Laplace transform (`laplace`) and then the inverse Laplace transform (`ilaplace`) of the result may not return the original signal for  $t < 0$ . This is because the definition of `laplace` uses the unilateral transform on page 6-949. This definition assumes that the signal  $f(t)$  is only defined for all real numbers  $t \geq 0$ . Therefore, the inverse result is not

unique for  $t < 0$  and it may not match the original signal for negative  $t$ . One way to retrieve the original signal is to multiply the result of `ilaplace` by a Heaviside step function. For example, both of these code blocks:

```
syms t;  
laplace(sin(t))
```

and

```
syms t;  
laplace(sin(t)*heaviside(t))
```

return  $1/(s^2 + 1)$ . However, the inverse Laplace transform

```
syms s;  
ilaplace(1/(s^2 + 1))
```

returns  $\sin(t)$ , not  $\sin(t)*\text{heaviside}(t)$ .

## Version History

Introduced before R2006a

## See Also

[fourier](#) | [ifourier](#) | [iztrans](#) | [laplace](#) | [ztrans](#) | [rewrite](#)

## Topics

“Solve Differential Equations of RLC Circuit Using Laplace Transform” on page 3-198

## imag

Imaginary part of complex number

### Syntax

```
imag(z)
```

### Description

`imag(z)` returns the imaginary part of `z`. If `z` is a matrix, `imag` acts elementwise on `z`.

### Examples

#### Compute Imaginary Part of Numeric Inputs

Find the imaginary parts of these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
[imag(2 + 3/2*i), imag(sin(5*i)), imag(2*exp(1 + i))]  
  
ans =  
    1.5000    74.2032    4.5747
```

#### Compute Imaginary Part of Symbolic Inputs

Compute the imaginary parts of the numbers converted to symbolic objects:

```
[imag(sym(2) + 3/2*i), imag(4/(sym(1) + 3*i)), imag(sin(sym(5)*i))]  
  
ans =  
[ 3/2, -6/5, sinh(5)]
```

Compute the imaginary part of this symbolic expression:

```
imag(2*exp(1 + sym(i)))  
  
ans =  
2*exp(1)*sin(1)
```

#### Compute Imaginary Part of Symbolic Expressions

In general, `imag` cannot extract the entire imaginary parts from symbolic expressions containing variables. However, `imag` can rewrite and sometimes simplify the input expression:

```
syms a x y  
imag(a + 2)  
imag(x + y*i)  
  
ans =  
imag(a)  
  
ans =  
imag(x) + real(y)
```

If you assign numeric values to these variables or if you specify that these variables are real, `imag` can extract the imaginary part of the expression:

```
syms a
a = 5 + 3*i;
imag(a + 2)
```

```
ans =
     3
```

```
syms x y real
imag(x + y*i)
```

```
ans =
     y
```

Clear the assumption that `x` and `y` are real by recreating them using `syms`:

```
syms x y
```

### Compute Imaginary Part for Matrix Input

Find the imaginary parts of the elements of matrix `A`:

```
syms x
A = [-1 + sym(i), sinh(x); exp(10 + sym(7)*i), exp(sym(pi)*i)];
imag(A)
```

```
ans =
[      1, imag(sinh(x))]
[ exp(10)*sin(7),      0]
```

## Input Arguments

### **z** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## Tips

- Calling `imag` for a number that is not a symbolic object invokes the MATLAB `imag` function.

## Alternatives

You can compute the imaginary part of `z` via the conjugate:  $\text{imag}(z) = (z - \text{conj}(z))/2i$ .

## Version History

Introduced before R2006a

**See Also**

`conj` | `in` | `real` | `sign` | `signIm`



# in

Numeric type of symbolic input

## Syntax

`in(x,type)`

## Description

`in(x,type)` expresses the logical condition that `x` is of the specified `type`.

## Examples

### Express Condition on Symbolic Variable or Expression

The syntax `in(x,type)` expresses the condition that `x` is of the specified `type`. Express the condition that `x` is of type `Real`.

```
syms x
cond = in(x,'real')
```

```
cond =
in(x, 'real')
```

Evaluate the condition using `isAlways`. Because `isAlways` cannot determine the condition, it issues a warning and returns logical 0 (false).

```
isAlways(cond)
```

```
Warning: Unable to prove 'in(x, 'real')'.
```

```
ans =
    logical
     0
```

Assume the condition `cond` is true using `assume`, and evaluate the condition again. The `isAlways` function returns logical 1 (true) indicating that the condition is true.

```
assume(cond)
isAlways(cond)
```

```
ans =
    logical
     1
```

To use `x` in further computations, clear its assumption recreating it using `syms`.

```
syms x
```

### Express Conditions in Output

Functions such as `solve` use `in` in output to express conditions.

Solve the equation  $\sin(x) == 0$  using `solve`. Set the option `ReturnConditions` to `true` to return conditions on the solution. The `solve` function uses `in` to express the conditions.

```
syms x
[solx, params, conds] = solve(sin(x) == 0, 'ReturnConditions', true)

solx =
pi*k

params =
k

conds =
in(k, 'integer')
```

The solution is  $\pi k$  with parameter  $k$  under the condition `in(k, 'integer')`. You can use this condition to set an assumption for further computations. Under the assumption, `solve` returns only integer values of  $k$ .

```
assume(conds)
k = solve(solx > 0, solx < 5*pi, params)

k =
1
2
3
4
```

To find the solutions corresponding to these values of  $k$ , use `subs` to substitute for  $k$  in `solx`.

```
subs(solx, k)

ans =
pi
2*pi
3*pi
4*pi
```

Clear the assumption on  $k$  to use it in further computations.

```
assume(params, 'clear')
```

### Test if Elements of Symbolic Matrix Are Rational

Create symbolic matrix  $M$ .

```
syms x y z
M = sym([1.22 i x; sin(y) 3*x 0; Inf sqrt(3) sym(22/7)])

M =
[ 61/50,      1i,      x]
[ sin(y),    3*x,     0]
[      Inf,  3^(1/2), 22/7]
```

Use `isAlways` to test if the elements of  $M$  are rational numbers. The `in` function acts on  $M$  element-by-element. Note that `isAlways` returns logical `0` (false) for statements that cannot be decided and issues a warning for those statements.

```
in(M, 'rational')
```

```
ans =
[ in(61/50, 'rational'),      in(1i, 'rational'),      in(x, 'rational')]
[ in(sin(y), 'rational'),    in(3*x, 'rational'),    in(0, 'rational')]
[ in(Inf, 'rational'), in(3^(1/2), 'rational'), in(22/7, 'rational)']
```

```
isAlways(in(M,'rational'))
```

```
Warning: Unable to prove 'in(sin(y), 'rational')'.
```

```
Warning: Unable to prove 'in(3*x, 'rational')'.
```

```
Warning: Unable to prove 'in(x, 'rational')'.
```

```
ans =
    3×3 logical array
     1     0     0
     0     0     1
     0     0     1
```

## Input Arguments

### x — Input

symbolic number | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic expression | symbolic function

Input, specified as a symbolic number, vector, matrix, multidimensional array, expression, or function.

### type — Type of input

'real' | 'positive' | 'integer' | 'rational'

Type of input, specified as 'real', 'positive', 'integer', or 'rational'.

## Version History

**Introduced in R2014b**

### See Also

assume | assumeAlso | false | imag | isalways | isequal | isequaln | isfinite | isinf | piecewise | real | true

# incidenceMatrix

Find incidence matrix of system of equations

## Syntax

```
A = incidenceMatrix(eqs,vars)
```

## Description

`A = incidenceMatrix(eqs,vars)` for  $m$  equations `eqs` and  $n$  variables `vars` returns an  $m$ -by- $n$  matrix `A`. Here,  $A(i,j) = 1$  if `eqs(i)` contains `vars(j)` or any derivative of `vars(j)`. All other elements of `A` are 0s.

## Examples

### Incidence Matrix

Find the incidence matrix of a system of five equations in five variables.

Create the following symbolic vector `eqs` containing five symbolic differential equations.

```
syms y1(t) y2(t) y3(t) y4(t) y5(t) c1 c3
eqs = [diff(y1(t),t) == y2(t),...
       diff(y2(t),t) == c1*y1(t) + c3*y3(t),...
       diff(y3(t),t) == y2(t) + y4(t),...
       diff(y4(t),t) == y3(t) + y5(t),...
       diff(y5(t),t) == y4(t)];
```

Create the vector of variables. Here, `c1` and `c3` are symbolic parameters (not variables) of the system.

```
vars = [y1(t), y2(t), y3(t), y4(t), y5(t)];
```

Find the incidence matrix `A` for the equations `eqs` and with respect to the variables `vars`.

```
A = incidenceMatrix(eqs, vars)
```

```
A =
     1     1     0     0     0
     1     1     1     0     0
     0     1     1     1     0
     0     0     1     1     1
     0     0     0     1     1
```

## Input Arguments

### eqs — Equations

vector of symbolic equations | vector of symbolic expressions

Equations, specified as a vector of symbolic equations or expressions.

**vars — Variables**

vector of symbolic variables | vector of symbolic functions | vector of symbolic function calls

Variables, specified as a vector of symbolic variables, symbolic functions, or function calls, such as  $x(t)$ .

**Output Arguments****A — Incidence matrix**

matrix of double-precision values

Incidence matrix, returned as a matrix of double-precision values.

**Version History**

**Introduced in R2014b**

**See Also**

`daeFunction` | `decic` | `findDecoupledBlocks` | `isLowIndexDAE` | `massMatrixForm` | `odeFunction` | `reduceDAEIndex` | `reduceDAEToODE` | `reduceDifferentialOrder` | `reduceRedundancies` | `spy`

## int

Definite and indefinite integrals

### Syntax

```
F = int(expr)
F = int(expr,var)

F = int(expr,a,b)
F = int(expr,var,a,b)

F = int( ____,Name=Value)
```

### Description

`F = int(expr)` computes the integral of the symbolic expression `expr`. Because you do not specify bounds for the integral, this syntax computes the indefinite integral. `int` uses the default integration variable determined by `symvar(expr,1)`. If `expr` is a constant, then the default integration variable is `x`.

`F = int(expr,var)` computes the indefinite integral of `expr` with respect to the symbolic scalar variable `var`.

`F = int(expr,a,b)` computes the definite integral of `expr` from `a` to `b`. `int` uses the default integration variable determined by `symvar(expr,1)`. If `expr` is a constant, then the default integration variable is `x`.

`int(expr,[a b])` is equivalent to `int(expr,a,b)`.

`F = int(expr,var,a,b)` computes the definite integral of `expr` with respect to the symbolic scalar variable `var` from `a` to `b`.

`int(expr,var,[a b])` is equivalent to `int(expr,var,a,b)`.

`F = int( ____,Name=Value)` specifies additional options using one or more `Name=Value` arguments. For example, `IgnoreAnalyticConstraints=true` specifies that `int` applies additional simplifications to the integrand.

---

**Note** The `int` function computes integral symbolically, and it is not related to integer data types in MATLAB. For more information about integers, see “Integers”.

---

### Examples

#### Indefinite Integral of Univariate Expression

Define a univariate expression.

```
syms x
expr = -2*x/(1+x^2)^2;
```

Find the indefinite integral of the univariate expression.

```
F = int(expr)
```

$$F = \frac{1}{x^2 + 1}$$

### Indefinite Integrals of Multivariate Function

Define a multivariate function with variables x and z.

```
syms x z
f(x,z) = x/(1+z^2);
```

Find the indefinite integrals of the multivariate expression with respect to the variables x and z.

```
Fx = int(f,x)
```

$$F_x(x, z) = \frac{x^2}{2(z^2 + 1)}$$

```
Fz = int(f,z)
```

$$F_z(x, z) = x \operatorname{atan}(z)$$

If you do not specify the integration variable, then `int` uses the first variable returned by `symvar` as the integration variable.

```
var = symvar(f,1)
```

```
var = x
```

```
F = int(f)
```

$$F(x, z) = \frac{x^2}{2(z^2 + 1)}$$

### Definite Integrals of Symbolic Expressions

Integrate a symbolic expression from 0 to 1.

```
syms x
expr = x*log(1+x);
F = int(expr,[0 1])
```

$$F = \frac{1}{4}$$

Integrate another expression from  $\sin(t)$  to 1.

```
syms t
F = int(2*x,[sin(t) 1])
```

```
F = cos(t)^2
```

When `int` cannot compute the value of a definite integral, numerically approximate the integral by using `vpa`.

```
syms x
f = cos(x)/sqrt(1 + x^2);
Fint = int(f,x,[0 10])
```

```
Fint =
```

$$\int_0^{10} \frac{\cos(x)}{\sqrt{x^2 + 1}} dx$$

```
Fvpa = vpa(Fint)
```

```
Fvpa = 0.37570628299079723478493405557162
```

To approximate integrals directly, use `vpaintegral` instead of `vpa`. The `vpaintegral` function is faster and provides control over integration tolerances.

```
Fvpaint = vpaintegral(f,x,[0 10])
```

```
Fvpaint = 0.375706
```

### Integrals of Matrix Elements

Define a symbolic matrix containing four expressions as its elements.

```
syms a t
M = [exp(t) exp(a*t); sin(t) cos(t)]
```

```
M =
```

$$\begin{pmatrix} e^t & e^{at} \\ \sin(t) & \cos(t) \end{pmatrix}$$

Find indefinite integrals of the matrix element-wise.

```
F = int(M,t)
```

```
F =
```

$$\begin{pmatrix} e^t & \frac{e^{at}}{a} \\ -\cos(t) & \sin(t) \end{pmatrix}$$

### Apply IgnoreAnalyticConstraints

Define a symbolic function and compute its indefinite integral.



```
syms f(x)
f(x) = acos(cos(x));
F = int(f,x)
```

$$F(x) = x \operatorname{acos}(\cos(x)) - \frac{x^2}{2 \operatorname{sign}(\sin(x))}$$

By default, `int` uses strict mathematical rules. These rules do not let `int` rewrite `acos(cos(x))` as `x`.

If you want a simple practical solution, set `IgnoreAnalyticConstraints` to `true`.

```
F = int(f,x,IgnoreAnalyticConstraints=true)
```

$$F(x) = \frac{x^2}{2}$$

### Ignore Special Cases

Define a symbolic expression  $x^t$  and compute its indefinite integral with respect to the variable  $x$ .

```
syms x t
F = int(x^t,x)
```

$$F = \begin{cases} \log(x) & \text{if } t = -1 \\ \frac{x^{t+1}}{t+1} & \text{if } t \neq -1 \end{cases}$$

By default, `int` returns the general results for all values of the other symbolic parameter  $t$ . In this example, `int` returns two integral results for the case  $t = -1$  and  $t \neq -1$ .

To ignore special cases of parameter values, set `IgnoreSpecialCases` to `true`. With this option, `int` ignores the special case  $t = -1$  and returns the solution for  $t \neq -1$ .

```
F = int(x^t,x,IgnoreSpecialCases=true)
```

$$F = \frac{x^{t+1}}{t+1}$$

### Find Cauchy Principal Value

Define a symbolic function  $f(x) = 1/(x-1)$  that has a pole at  $x = 1$ .

```
syms x
f(x) = 1/(x-1)
```

$$f(x) =$$

$$\frac{1}{x-1}$$

Compute the definite integral of this function from  $x = 0$  to  $x = 2$ . Since the integration interval includes the pole, the result is not defined.

```
F = int(f,[0 2])
```

```
F = NaN
```

However, the Cauchy principal value of the integral exists. To compute the Cauchy principal value of the integral, set `PrincipalValue` to `true`.

```
F = int(f,[0 2],PrincipalValue=true)
```

```
F = 0
```

### Unevaluated Integral and Integration by Parts

Find the integral of  $\int x e^x dx$ .

Define the integral without evaluating it by setting the `Hold` option to `true`.

```
syms x g(y)
```

```
F = int(x*exp(x),Hold=true)
```

```
F =
```

$$\int x e^x dx$$

You can apply integration by parts to `F` by using the `integrateByParts` function. Use `exp(x)` as the differential to be integrated.

```
G = integrateByParts(F,exp(x))
```

```
G =
```

$$x e^x - \int e^x dx$$

To evaluate the integral in `G`, use the `release` function to ignore the `Hold` option.

```
Gcalc = release(G)
```

```
Gcalc = x e^x - e^x
```

Compare the result to the integration result returned by `int` without setting the `Hold` option.

```
Fcalc = int(x*exp(x))
```

```
Fcalc = e^x (x - 1)
```

### Approximate Indefinite Integrals

If `int` cannot compute a closed form of an integral, then it returns an unresolved integral.

```
syms f(x)
f(x) = sin(sinh(x));
F = int(f,x)
```

$$F(x) = \int \sin(\sinh(x)) \, dx$$

You can approximate the integrand function  $f(x)$  as polynomials by using the Taylor expansion. Apply `taylor` to expand the integrand function  $f(x)$  as polynomials around  $x = 0$ . Compute the integral of the approximated polynomials.

```
fTaylor = taylor(f,x,ExpansionPoint=0,Order=10)
```

$$fTaylor(x) = \frac{x^9}{5670} - \frac{x^7}{90} - \frac{x^5}{15} + x$$

```
Fapprox = int(fTaylor,x)
```

$$Fapprox(x) = \frac{x^{10}}{56700} - \frac{x^8}{720} - \frac{x^6}{90} + \frac{x^2}{2}$$

### Line Integral of Vector Field

Define a vector field in 3D space.

```
syms x y z
F(x,y,z) = [x^2*y*z; x*y; 2*y*z]
```

$$F(x, y, z) = \begin{pmatrix} x^2 y z \\ x y \\ 2 y z \end{pmatrix}$$

Next, define a parametric curve.

```
syms t real
rx(t) = sin(t);
ry(t) = cos(t);
rz(t) = t;
r(t) = [rx; ry; rz]
```

$$r(t) = \begin{pmatrix} \sin(t) \\ \cos(t) \\ t \end{pmatrix}$$

The line integral of a vector field  $\mathbf{F}$  along the curve  $C$  that is parameterized by  $\mathbf{r}(t)$  is defined as

$$\int_C \mathbf{F} \cdot d\mathbf{r} = \int_a^b \mathbf{F}(\mathbf{r}(t)) \cdot \frac{d\mathbf{r}(t)}{dt} dt ,$$

where the operator  $\cdot$  denotes a scalar product.

Use this definition to compute the line integral of the vector field along the parameterized curve  $\mathbf{r}(t)$  from  $t = -5$  to  $t = 5$ .

```
drdt = diff(r(t),t);
integrand = dot(F(rx,ry,rz),drdt);
W = int(integrand,t,[-5,5])
```

W =

$$-\frac{2 \sin(5)^3}{3}$$

## Input Arguments

### expr — Integrand

symbolic expression | symbolic function | symbolic vector | symbolic matrix | symbolic number

Integrand, specified as a symbolic expression, function, vector, matrix, or number.

### var — Integration variable

symbolic variable

Integration variable, specified as a symbolic variable. If you do not specify this variable, `int` uses the default variable determined by `symvar(expr,1)`. If `expr` is a constant, then the default variable is `x`.

### a — Lower bound

number | symbolic number | symbolic variable | symbolic expression | symbolic function

Lower bound, specified as a number, symbolic number, variable, expression, or function (including expressions and functions with infinities).

### b — Upper bound

number | symbolic number | symbolic variable | symbolic expression | symbolic function

Upper bound, specified as a number, symbolic number, variable, expression, or function (including expressions and functions with infinities).

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `IgnoreAnalyticConstraints=true` specifies that `int` applies purely algebraic simplifications to the integrand.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

### IgnoreAnalyticConstraints — Indicator for applying purely algebraic simplifications to integrand

false (default) | true

Indicator for applying purely algebraic simplifications to the integrand, specified as `true` or `false`. If the value is `true`, apply purely algebraic simplifications to the integrand. This option can provide

simpler results for expressions, for which the direct use of the integrator returns complicated results. In some cases, it also enables `int` to compute integrals that cannot be computed otherwise.

Using this option can lead to results not generally valid. This option applies mathematical identities that are convenient, but the results do not always hold for all values of variables. For details, see “Algorithms” on page 6-822.

### **IgnoreSpecialCases — Indicator for ignoring special cases**

`false` (default) | `true`

Indicator for ignoring special cases, specified as `true` or `false`. This ignores cases that require one or more parameters to be elements of a comparatively small set, such as a fixed finite set or a set of integers.

### **PrincipalValue — Indicator for returning principal value**

`false` (default) | `true`

Indicator for returning the principal value, specified as `true` or `false`. If the value is `true`, `int` computes the Cauchy principal value of the integral. In live scripts, the Cauchy principal value of the unevaluated integral appears as the  $\int$  symbol.

### **Hold — Indicator for unevaluated integration**

`false` (default) | `true`

Indicator for unevaluated integration, specified as `true` or `false`. If the value is `true`, `int` returns integrals without evaluating them.

## **Tips**

- In contrast to differentiation, symbolic integration is a more complicated task. If `int` cannot compute an integral of an expression, check for these reasons:
  - The antiderivative does not exist in a closed form.
  - The antiderivative exists, but `int` cannot find it.

If `int` cannot compute a closed form of an integral, it returns an unresolved integral.

For some integrals that have closed form solutions, where these solutions are complicated and `int` returns unresolved integrals, you can use `simplify` to obtain the closed form solutions. For example, the following code finds the closed form solution of the integral of  $f(x)$ :

```
syms x
f(x) = x*log(x/2+sqrt(x^2+1));
F = int(f,x)
simplify(F,Steps=10)
```

Otherwise, you can try approximating unresolved integrals by using one of these methods:

- For indefinite integrals, use series expansions. Use this method to approximate an integral around a particular value of the variable.
- For definite integrals, use numeric approximations.
- For indefinite integrals, `int` does not return a constant of integration in the result. The results of integrating mathematically equivalent expressions may be different. For example, `syms x;`

`int((x+1)^2)` returns  $(x+1)^3/3$ , while `syms x; int(x^2+2*x+1)` returns  $(x*(x^2+3*x+3))/3$ , which differs from the first result by  $1/3$ .

- For indefinite integrals, `int` implicitly assumes that the integration variable `var` is real. For definite integrals, `int` restricts the integration variable `var` to the specified integration interval, unless the interval is incompatible with an existing assumption. If one or both integration bounds `a` and `b` are not numeric, `int` assumes that  $a \leq b$  unless you explicitly specify otherwise.

For example, using the default assumption  $a \leq b$ , this code returns 0 as the result of evaluating the definite integral at  $(a,b)=(3,2)$ .

```
syms x a b
F(a,b) = int heaviside(x-1)*heaviside(4-x), a,b;
F(3,2)

ans =
0
```

If you explicitly specify the assumption  $a > b$ , then this code returns -1 as the integral result.

```
assume(a > b)
F(a,b) = int heaviside(x-1)*heaviside(4-x), a,b;
F(3,2)

ans =
-1
```

## Algorithms

When you use `IgnoreAnalyticConstraints`, `int` applies some of these rules:

- $\log(a) + \log(b) = \log(a \cdot b)$  for all values of  $a$  and  $b$ . In particular, the following equality is valid for all values of  $a$ ,  $b$ , and  $c$ :

$$(a \cdot b)^c = a^c \cdot b^c.$$

- $\log(a^b) = b \cdot \log(a)$  for all values of  $a$  and  $b$ . In particular, the following equality is valid for all values of  $a$ ,  $b$ , and  $c$ :

$$(a^b)^c = a^{b \cdot c}.$$

- If  $f$  and  $g$  are standard mathematical functions and  $f(g(x)) = x$  for all small positive numbers, then  $f(g(x)) = x$  is assumed to be valid for all complex values  $x$ . In particular:
  - $\log(e^x) = x$
  - $\text{asin}(\sin(x)) = x$ ,  $\text{acos}(\cos(x)) = x$ ,  $\text{atan}(\tan(x)) = x$
  - $\text{asinh}(\sinh(x)) = x$ ,  $\text{acosh}(\cosh(x)) = x$ ,  $\text{atanh}(\tanh(x)) = x$
  - $W_k(x \cdot e^x) = x$  for all branch indices  $k$  of the Lambert W function.

## Version History

Introduced before R2006a

**R2019b: Return unevaluated integral**

`int(___, 'Hold', true)` returns integrals without evaluating them. Use `release` to return the evaluated integrals by ignoring the 'Hold' option in the `int` function.

## See Also

`diff` | `dsolve` | `functionalDerivative` | `symvar` | `vpaintegral` | `integrateByParts` | `changeIntegrationVariable` | `release` | `rewrite`

## Topics

"Integration" on page 3-186

## Teaching Resources

Calculus Integrals (MathWorks Teaching Resources)

Beam Bending and Deflection (MathWorks Teaching Resources)

# integrateByParts

Integration by parts

## Syntax

```
G = integrateByParts(F,du)
```

## Description

`G = integrateByParts(F,du)` applies integration by parts to the integrals in `F`, in which the differential `du` is integrated. For more information, see “Integration by Parts” on page 6-826.

When specifying the integrals in `F`, you can return the unevaluated form of the integrals by using the `int` function with the 'Hold' option set to true. You can then use `integrateByParts` to show the steps of integration by parts.

## Examples

### Product of Functions

Create a symbolic expression `F` that is the integral of a product of functions.

```
syms u(x) v(x)
F = int(u*diff(v))
```

```
F(x) =

$$\int u(x) \frac{\partial}{\partial x} v(x) dx$$

```

Apply integration by parts to `F`.

```
g = integrateByParts(F,diff(u))
```

```
g =

$$u(x) v(x) - \int v(x) \frac{\partial}{\partial x} u(x) dx$$

```

### Exponential Function

Apply integration by parts to the integral  $\int x^2 e^x dx$ .

Define the integral using the `int` function. Show the result without evaluating the integral by setting the 'Hold' option to true.

```
syms x
F = int(x^2*exp(x),'Hold',true)
```

```
F =
```



$$\int x^2 e^x dx$$

To show the steps of integration, apply integration by parts to F and use  $\exp(x)$  as the differential to be integrated.

```
G = integrateByParts(F,exp(x))
```

$$G = x^2 e^x - \int 2x e^x dx$$

```
H = integrateByParts(G,exp(x))
```

$$H = x^2 e^x - 2x e^x + \int 2 e^x dx$$

Evaluate the integral in H by using the `release` function to ignore the 'Hold' option.

```
F1 = release(H)
```

$$F1 = 2 e^x + x^2 e^x - 2x e^x$$

Compare the result to the integration result returned by the `int` function without setting the 'Hold' option to `true`.

```
F2 = int(x^2*exp(x))
```

$$F2 = e^x (x^2 - 2x + 2)$$

## Exponential and Trigonometric Functions

Apply integration by parts to the integral  $\int e^{ax} \sin(bx) dx$ .

Define the integral using the `int` function. Show the integral without evaluating it by setting the 'Hold' option to `true`.

```
syms x a b
F = int(exp(a*x)*sin(b*x),'Hold',true)
```

$$F = \int e^{ax} \sin(bx) dx$$

To show the steps of integration, apply integration by parts to F and use  $u'(x) = e^{ax}$  as the differential to be integrated.

```
G = integrateByParts(F,exp(a*x))
```

$$G = \frac{e^{ax} \sin(bx)}{a} - \int \frac{b e^{ax} \cos(bx)}{a} dx$$

Evaluate the integral in G by using the `release` function to ignore the 'Hold' option.

```
F1 = release(G)
```

```
F1 =
```

$$\frac{e^{ax} \sin(bx)}{a} - \frac{b e^{ax} (a \cos(bx) + b \sin(bx))}{a(a^2 + b^2)}$$

Simplify the result.

```
F2 = simplify(F1)
```

```
F2 =
```

$$-\frac{e^{ax} (b \cos(bx) - a \sin(bx))}{a^2 + b^2}$$

## Input Arguments

### F — Expression containing integrals

symbolic expression | symbolic function | symbolic vector | symbolic matrix

Expression containing integrals, specified as a symbolic expression, function, vector, or matrix.

Example: `int(u*diff(v))`

### du — Differential to be integrated

symbolic variable | symbolic expression | symbolic function

Differential to be integrated, specified as a symbolic variable, expression, or function.

Example: `diff(u)`

## More About

### Integration by Parts

Mathematically, the rule of integration by parts is formally defined for indefinite integrals as

$$\int u'(x) v(x) dx = u(x) v(x) - \int u(x) v'(x) dx$$

and for definite integrals as

$$\int_a^b u'(x) v(x) dx = u(b) v(b) - u(a) v(a) - \int_a^b u(x) v'(x) dx.$$

## Version History

Introduced in R2019b

### See Also

`changeIntegrationVariable` | `release` | `int` | `diff` | `vpaintegral`

# inv

Inverse of symbolic matrix

## Syntax

`D = inv(A)`

## Description

`D = inv(A)` returns the inverse of a symbolic matrix A.

## Examples

### Compute Inverse of Symbolic Matrix

Compute the inverse of a matrix of symbolic numbers.

```
A = sym([2 -1 0; -1 2 -1; 0 -1 2]);
D = inv(A)
```

$$D = \begin{pmatrix} \frac{3}{4} & \frac{1}{2} & \frac{1}{4} \\ \frac{1}{2} & 1 & \frac{1}{2} \\ \frac{1}{4} & \frac{1}{2} & \frac{3}{4} \end{pmatrix}$$

Compute the inverse of a matrix of symbolic scalar variables.

```
syms a b c d
A = [a b; c d];
D = inv(A)
```

$$D = \begin{pmatrix} \frac{d}{ad-bc} & -\frac{b}{ad-bc} \\ -\frac{c}{ad-bc} & \frac{a}{ad-bc} \end{pmatrix}$$

### Compute Inverse of Hilbert Matrix with Symbolic Numbers

Compute the inverse of the Hilbert matrix that contains symbolic numbers.

```
D = inv(sym(hilb(4)))
```

$$D = \begin{pmatrix} 16 & -120 & 240 & -140 \\ -120 & 1200 & -2700 & 1680 \\ 240 & -2700 & 6480 & -4200 \\ -140 & 1680 & -4200 & 2800 \end{pmatrix}$$

**Compute Inverse of Block Matrix**

Find the inverse of a 4-by-4 block matrix

$$\mathbf{C} = \begin{bmatrix} \mathbf{A} & \mathbf{0}_{2,2} \\ \mathbf{0}_{2,2} & \mathbf{B} \end{bmatrix}$$

where  $\mathbf{A}$  and  $\mathbf{B}$  are 2-by-2 submatrices. The notation  $\mathbf{0}_{2,2}$  represents a 2-by-2 submatrix of zeros.

Use symbolic matrix variables to represent the submatrices in the block matrix.

```
syms A B [2 2] matrix
Z = symmatrix(zeros(2))
```

```
Z = 0_{2,2}
```

```
C = [A Z; Z B]
```

```
C =
```

$$\begin{pmatrix} A & \mathbf{0}_{2,2} \\ \mathbf{0}_{2,2} & B \end{pmatrix}$$

Find the inverse of the matrix  $\mathbf{C}$ .

```
D = inv(C)
```

```
D =
```

$$\begin{pmatrix} A & \mathbf{0}_{2,2} \\ \mathbf{0}_{2,2} & B \end{pmatrix}^{-1}$$

To show the elements of the inverse matrix, convert the result from a symbolic matrix variable to symbolic scalar variables using `symmatrix2sym`.

```
D1 = symmatrix2sym(D)
```

```
D1 =
```

$$\begin{pmatrix} \frac{A_{2,2}}{\sigma_2} & -\frac{A_{1,2}}{\sigma_2} & 0 & 0 \\ -\frac{A_{2,1}}{\sigma_2} & \frac{A_{1,1}}{\sigma_2} & 0 & 0 \\ 0 & 0 & \frac{B_{2,2}}{\sigma_1} & -\frac{B_{1,2}}{\sigma_1} \\ 0 & 0 & -\frac{B_{2,1}}{\sigma_1} & \frac{B_{1,1}}{\sigma_1} \end{pmatrix}$$

where

$$\sigma_1 = B_{1,1} B_{2,2} - B_{1,2} B_{2,1}$$

$$\sigma_2 = A_{1,1} A_{2,2} - A_{1,2} A_{2,1}$$

## Compute Inverse of Matrix Polynomial

Compute the inverse of the matrix polynomial  $a_0 \mathbf{I}_2 + \mathbf{A}$ , where  $\mathbf{A}$  is a 2-by-2 matrix.

Create the matrix  $\mathbf{A}$  as a symbolic matrix variable and the coefficient  $a_0$  as a symbolic scalar variable. Create the matrix polynomial as a symbolic matrix function  $f$  with  $a_0$  and  $\mathbf{A}$  as its parameters.

```
syms A [2 2] matrix
syms a0
syms f(a0,A) [2 2] matrix keepargs
f(a0,A) = a0*eye(2) + A
```

```
f(a0, A) = a0*I2 + A
```

Find the inverse of  $f$  using `inv`. The result is a symbolic matrix function of type `symfunmatrix` that accepts scalars, vectors, and matrices as its input arguments.

```
fInv = inv(f)
```

```
fInv(a0, A) = (a0*I2 + A)^-1
```

Convert the result from the `symfunmatrix` data type to the `symfun` data type using `symfunmatrix2symfun`. The result is a symbolic function that accepts scalars as its input arguments.

```
gInv = symfunmatrix2symfun(fInv)
```

```
gInv(a0, A1_1, A1_2, A2_1, A2_2) =
```

$$\begin{pmatrix} \frac{A_{2,2} + a_0}{\sigma_1} & -\frac{A_{1,2}}{\sigma_1} \\ -\frac{A_{2,1}}{\sigma_1} & \frac{A_{1,1} + a_0}{\sigma_1} \end{pmatrix}$$

where

$$\sigma_1 = A_{1,1} a_0 + A_{2,2} a_0 + a_0^2 + A_{1,1} A_{2,2} - A_{1,2} A_{2,1}$$

## Input Arguments

### A — Input matrix

square numeric matrix | square matrix of symbolic scalar variables | square symbolic matrix variable | square symbolic matrix function | symbolic expression with square size

Input matrix, specified as a square numeric matrix, square matrix of symbolic scalar variables, square symbolic matrix variable, square symbolic matrix function, or symbolic expression with square size.

Data Types: `single` | `double` | `sym` | `symmatrix` | `symfunmatrix`

## Tips

- Matrix computations involving many symbolic variables can be slow. To increase the computational speed, reduce the number of symbolic variables by substituting the given values for some variables.

## Version History

Introduced before R2006a

### **R2022a: Compute inverse of symbolic matrix functions**

The `inv` function accepts an input argument of type `symfunmatrix`. For an example, see “Compute Inverse of Matrix Polynomial” on page 6-828.

### **R2021b: Compute inverse of symbolic matrix variables**

The `inv` function accepts an input argument of type `symmatrix`. For an example, see “Compute Inverse of Block Matrix” on page 6-828.

### **See Also**

`eig` | `det` | `rank`

# isAlways

Determine if symbolic conditions are true for all values of variables

## Syntax

```
tf = isAlways(cond)
tf = isAlways(cond,Unknown=option)
```

## Description

`tf = isAlways(cond)` checks if the conditions in `cond` are always mathematically true and returns an array of logical values. `isAlways` checks if `cond` holds true for all possible values of the symbolic variables in `cond` including all assumptions on the variables. If an element in `cond` is always true, then the corresponding element in `tf` is a logical 1 (true). Otherwise, the corresponding element in `tf` is a logical 0 (false).

`tf = isAlways(cond,Unknown=option)` returns the output of undecidable conditions as specified by `option`.

## Examples

### Test Symbolic Conditions

Check if an inequality is true for all values of its variable.

```
syms x
tf = isAlways(abs(x) >= 0)

tf = logical
    1
```

`isAlways` returns logical 1 (true), indicating that the inequality `abs(x) >= 0` is true for all values of `x`.

Check if an equation is true for all values of its variable.

```
tf = isAlways(sin(x)^2 + cos(x)^2 == 1)

tf = logical
    1
```

`isAlways` returns logical 1 (true), indicating that the equation `sin(x)^2 + cos(x)^2 == 1` is true for all values of `x`.

### Test If One of Several Conditions Holds True

Check if at least one of two conditions holds true. To check if at least one of several conditions is true, combine them using the logical operator `or` or its shortcut `|`.

```
syms x
tf = isAlways(sin(x)^2 + cos(x)^2 == 1 | x^2 > 0)

tf = logical
    1
```

Check if both conditions hold true. To check if several conditions are true, combine them using the logical operator `and` or its shortcut `&`.

```
tf = isAlways(sin(x)^2 + cos(x)^2 == 1 & abs(x) > 2*abs(x))

tf = logical
    0
```

For multiple conditions, you can also represent the conditions as a symbolic array.

```
cond = [sin(x)^2 + cos(x)^2 == 1; abs(x) > 2*abs(x)]

cond =

$$\begin{pmatrix} \cos(x)^2 + \sin(x)^2 = 1 \\ 2|x| < |x| \end{pmatrix}$$


tf = isAlways(cond)

tf = 2x1 logical array

    1
    0
```

### Handle Output for Undecidable Conditions

When `isAlways` cannot determine if a condition is true, it returns logical 0 (false) and issues a warning by default.

```
syms x
tf = isAlways(2*x >= x)

Warning: Unable to prove 'x <= 2*x'.

tf = logical
    0
```

To change this default behavior, use the `Unknown` name-value argument. For example, specify `Unknown` as `"false"` to suppress the warning and make `isAlways` return logical 0 (false) if it cannot determine whether the condition is true.

```
tf = isAlways(2*x >= x,Unknown="false")
```



```
tf = logical
    0
```

Instead of "false", you can also specify Unknown as "error" to return an error, and as "true" to return logical 1 (true).

## Test Conditions with Assumptions

Check an inequality under the assumption that  $x$  is negative. When `isAlways` tests an equation or inequality, it takes into account assumptions on variables in that equation or inequality.

```
syms x
assume(x < 0)
tf = isAlways(2*x < x)

tf = logical
    1
```

For further computations, clear the assumption on  $x$  by recreating it using `syms`.

```
syms x
```

## Input Arguments

### cond — Condition to check

symbolic condition | vector of symbolic conditions | matrix of symbolic conditions | multidimensional array of symbolic conditions

Condition to check, specified as a symbolic condition, or a vector, matrix, or multidimensional array of symbolic conditions.

### option — Return option for undecidable condition

"falseWithWarning" (default) | "false" | "true" | "error"

Return option for an undecidable condition, specified as one of these values:

|                              |                                                                                                    |
|------------------------------|----------------------------------------------------------------------------------------------------|
| "falseWithWarning" (default) | On undecidable inputs, return logical 0 (false) and a warning that the condition cannot be proven. |
| "false"                      | On undecidable inputs, return logical 0 (false).                                                   |
| "true"                       | On undecidable inputs, return logical 1 (true).                                                    |
| "error"                      | On undecidable inputs, return an error.                                                            |

## Version History

Introduced in R2012a

**R2014b: isAlways warns when returning false for undecidable inputs***Warns starting in R2014b*

`isAlways` issues a warning when returning `false` (or logical 0) for undecidable inputs. To suppress the warning, set the `Unknown` option to `false` by using `isAlways(cond,Unknown="false")`. For details, see “Handle Output for Undecidable Conditions” on page 6-832.

**See Also**

`assume` | `assumeAlso` | `assumptions` | `in` | `isequal` | `isequaln` | `isfinite` | `isinf` | `isnan` | `logical` | `sym` | `syms`

**Topics**

“Use Assumptions on Symbolic Variables” on page 1-41

“Clear Assumptions and Reset the Symbolic Engine” on page 3-379

“Check Symbolic Equations, Inequalities, and Conditional Statements” on page 2-23

# isequal

Determine if symbolic inputs are equal

## Syntax

```
tf = isequal(A,B)
tf = isequal(A1,A2,...,An)
```

## Description

`tf = isequal(A,B)` returns logical 1 (true) if A and B are the same size and their contents are of equal value (from a coding perspective). Otherwise, `isequal` returns logical 0 (false). `isequal` does not consider NaN (Not a Number) values to be equal. `isequal` recursively compares the contents of symbolic data structures and the properties of objects. If all contents in the corresponding locations are equal, `isequal` returns logical 1 (true).

`tf = isequal(A1,A2,...,An)` returns logical 1 (true) if all the inputs `A1,A2,...,An` are equal.

## Examples

### Test Numbers for Equality

Test numeric or symbolic inputs for equality using `isequal`. If you compare numeric inputs against symbolic inputs, `isequal` returns 0 (false) because `double` and `sym` are distinct data types.

Test if 2 and 5 are equal. Because you are comparing doubles, the MATLAB® `isequal` function is called. `isequal` returns 0 (false) as expected.

```
tf = isequal(2,5)

tf = logical
    0
```

Test if the solution of the equation  $\cos(x) == -1$  is  $\pi$ . The `isequal` function returns 1 (true), meaning the solution is equal to  $\pi$ .

```
syms x
sol = solve(cos(x) == -1,x);
tf = isequal(sol,sym(pi))

tf = logical
    1
```

Compare the double and symbolic representations of  $\pi$ . `isequal` returns 0 (false) because `double` and `sym` are distinct data types. To return 1 (true) in this case, use `logical` instead.

```
usingIsEqual = isequal(pi,sym(pi))
```

```
usingIsEqual = logical
0

usingLogical = logical(pi == sym(pi))

usingLogical = logical
1
```

### Test Symbolic Expressions for Equality

Test if the expressions  $\tan(x)$  and  $\sin(x)/\cos(x)$  are equal (from a coding perspective). The `isequal` function returns 0 (false) because the expressions are different. `isequal` does not mathematically compare the expressions.

```
syms x
tf = isequal(tan(x), sin(x)/cos(x))

tf = logical
0
```

Rewrite the expression  $\tan(x)$  in terms of  $\sin(x)$  and  $\cos(x)$ . Test if `rewrite` correctly rewrites  $\tan(x)$  as  $\sin(x)/\cos(x)$ . The `isequal` function returns 1 (true).

```
f = rewrite(tan(x), "sincos");
testf = sin(x)/cos(x);
tf = isequal(f, testf)

tf = logical
1
```

To check if the mathematical comparison  $\tan(x) == \sin(x)/\cos(x)$  holds true for all values of  $x$ , use `isAlways`.

```
tf = isAlways(tan(x) == sin(x)/cos(x))

tf = logical
1
```

### Test Symbolic Vectors and Matrices for Equality

Test if solutions of the quadratic equation found by `solve` are equal to the expected solutions. The `isequal` function returns 1 (true), meaning the symbolic vectors are equal.

```
syms a b c x
eqn = a*x^2 + b*x + c;
Sol = solve(eqn, x);
testSol = [-(b+(b^2-4*a*c)^(1/2))/(2*a); -(b-(b^2-4*a*c)^(1/2))/(2*a)];
tf = isequal(Sol, testSol)
```

```
tf = logical
    1
```

The Hilbert matrix is a special matrix that is difficult to invert accurately. If the inverse is accurately computed, then multiplying the inverse by the original Hilbert matrix returns the identity matrix.

Use this condition to test if the inverse of `hilb(20)` is correctly calculated. `isequal` returns `1` (`true`), meaning that the product of the inverse and the original Hilbert matrix is equal to the identity matrix.

```
H = sym(hilb(20));
prod = H*inv(H);
eye20 = sym(eye(20));
tf = isequal(prod,eye20)
```

```
tf = logical
    1
```

## Compare Inputs Containing NaN

Compare three vectors containing NaN (not a number). `isequal` returns logical `0` (`false`) because `isequal` does not treat NaN values as equal.

```
syms x
A1 = [x NaN NaN];
A2 = [x NaN NaN];
A3 = [x NaN NaN];
tf = isequal(A1,A2,A3)
```

```
tf = logical
    0
```

## Input Arguments

### A,B — Inputs to compare (as separate arguments)

numbers | vectors | matrices | arrays | symbolic numbers | symbolic scalar variables | symbolic matrix variables | symbolic arrays | symbolic functions | symbolic matrix functions | symbolic expressions

Inputs to compare, specified as numbers, vectors, matrices, or arrays, or symbolic numbers, scalar variables, matrix variables, arrays, functions, matrix functions, or expressions.

### A1,A2,...,An — Series of inputs to compare (as separate arguments)

numbers | vectors | matrices | arrays | symbolic numbers | symbolic scalar variables | symbolic matrix variables | symbolic arrays | symbolic functions | symbolic matrix functions | symbolic expressions

Series of inputs to compare, specified as numbers, vectors, matrices, or arrays, or symbolic numbers, scalar variables, matrix variables, arrays, functions, matrix functions, or expressions.

## Tips

- `isequal(A,B)` checks if A and B are the same size and their contents are equal (from a coding perspective). To check whether the condition `A == B` is always mathematically true for all values of variables in A and B, use `isAlways(A == B)`.
- If one of the input arguments is a symbolic type and the other input is a MATLAB numeric type with the same value, then `isequal` returns logical 0 (false) because the inputs do not have the same data type. For example, `tf = isequal(1,sym(1))` returns 0 (false).

## Version History

Introduced before R2006a

### R2022a: Check equality of symbolic matrix functions

The `isequal` function accepts input arguments of type `symfunmatrix`.

### R2021a: Check equality of symbolic matrix variables

The `isequal` function accepts input arguments of type `symmatrix`.

## See Also

`in` | `isAlways` | `isequaln` | `isfinite` | `isinf` | `isnan` | `logical`

## Topics

“Check Symbolic Equations, Inequalities, and Conditional Statements” on page 2-23

# isequaln

Test symbolic objects for equality, treating NaN values as equal

## Syntax

```
tf = isequaln(A,B)
tf = isequaln(A1,A2,...,An)
```

## Description

`tf = isequaln(A,B)` returns logical 1 (true) if A and B are the same size and their contents are of equal value. Otherwise, `isequaln` returns logical 0 (false). All NaN (not a number) values are considered to be equal to each other. `isequaln` recursively compares the contents of symbolic data structures and the properties of objects. If all contents in the respective locations are equal, `isequaln` returns logical 1 (true).

`tf = isequaln(A1,A2,...,An)` returns logical 1 (true) if all the inputs are equal.

## Examples

### Compare Two Expressions

Use `isequaln` to compare these two expressions.

```
syms x
tf = isequaln(abs(x),x)

tf = logical
    0
```

For positive x, these expressions are identical.

```
assume(x > 0)
tf = isequaln(abs(x),x)

tf = logical
    1
```

For further computations, remove the assumption on x by recreating it using `syms`.

```
syms x
```

### Compare Two Matrices

Use `isequaln` to compare these two matrices.

```
A = hilb(3);
B = sym(A);
tf = isequaln(A,B)

tf = logical
    0
```

## Compare Vectors Containing NaN Values

Use `isequaln` to compare these vectors.

```
syms x
A1 = [x NaN NaN];
A2 = [x NaN NaN];
A3 = [x NaN NaN];
tf = isequaln(A1,A2,A3)

tf = logical
    1
```

## Input Arguments

### A,B — Inputs to compare

symbolic numbers | symbolic scalar variables | symbolic matrix variables | symbolic expressions | symbolic functions | symbolic matrix functions | symbolic vectors | symbolic matrices

Inputs to compare, specified as symbolic numbers, scalar variables, matrix variables, expressions, functions, matrix functions, vectors, or matrices. If one of the arguments is a symbolic object and the other one is numeric, the toolbox converts the numeric object to symbolic before comparing them.

### A1,A2,...,An — Series of inputs to compare

symbolic numbers | symbolic scalar variables | symbolic matrix variables | symbolic expressions | symbolic functions | symbolic matrix functions | symbolic vectors | symbolic matrices

Series of inputs to compare, specified as symbolic numbers, scalar variables, matrix variables, expressions, functions, matrix functions, vectors, or matrices. If at least one of the arguments is a symbolic object, the toolbox converts all other arguments to symbolic objects before comparing them.

## Tips

- `isequaln(A,B)` checks if A and B are the same size and their contents are equal (from coding perspective), treating NaN values as equal. To check whether the mathematical comparison  $A == B$  holds for all values of variables in A and B, use `isAlways(A == B)`.
- If one of the input arguments is a symbolic type and the other input is a MATLAB numeric type with the same value, then `isequaln` returns logical 0 (false) because the inputs are not equal (from coding perspective). For example, `tf = isequaln(1,sym(1))` returns 0 (false).



## Version History

Introduced in R2013a

### **R2022a: Check equality of symbolic matrix functions**

The `isequaln` function accepts input arguments of type `symfunmatrix`.

### **R2021a: Check equality of symbolic matrix variables**

The `isequaln` function accepts input arguments of type `symmatrix`.

## See Also

`in` | `isAlways` | `isequal` | `isequaln` | `isfinite` | `isinf` | `isnan`

## Topics

“Check Symbolic Equations, Inequalities, and Conditional Statements” on page 2-23

## isfinite

Check whether symbolic array elements are finite

### Syntax

```
isfinite(A)
```

### Description

`isfinite(A)` returns an array of the same size as `A` containing logical 1s (true) where the elements of `A` are finite, and logical 0s (false) where they are not. For a complex number, `isfinite` returns 1 if both the real and imaginary parts of that number are finite. Otherwise, it returns 0.

### Examples

#### Determine Which Elements of Symbolic Array Are Finite Values

Using `isfinite`, determine which elements of this symbolic matrix are finite values:

```
isfinite(sym([pi NaN Inf; 1 + i Inf + i NaN + i]))
```

```
ans =  
    2×3 logical array  
    1     0     0  
    1     0     0
```

#### Determine if Exact and Approximated Values Are Finite

Approximate these symbolic values with the 50-digit accuracy:

```
V = sym([pi, 2*pi, 3*pi, 4*pi]);  
V_approx = vpa(V, 50);
```

The cotangents of the exact values are not finite:

```
cot(V)  
isfinite(cot(V))  
  
ans =  
[ Inf, Inf, Inf, Inf]
```

```
ans =  
    1×4 logical array  
    0     0     0     0
```

Nevertheless, the cotangents of the approximated values are finite due to the round-off errors:

```
isfinite(cot(V_approx))  
  
ans =  
    1×4 logical array  
    1     1     1     1
```

## Input Arguments

### A — Input value

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic array | symbolic vector | symbolic matrix

Input value, specified as a symbolic number, variable, expression, or function, or as an array, vector, or matrix of symbolic numbers, variables, expressions, functions.

## Tips

- For any A, exactly one of the three quantities `isfinite(A)`, `isinf(A)`, or `isnan(A)` is 1 for each element.
- Elements of A are recognized as finite if they are
  - Not symbolic NaN
  - Not symbolic Inf or -Inf
  - Not sums or products containing symbolic infinities Inf or -Inf

## Version History

Introduced in R2013b

### See Also

`in` | `isAlways` | `isequal` | `isequaln` | `isinf` | `isnan`

## isinf

Check whether symbolic array elements are infinite

### Syntax

```
isinf(A)
```

### Description

`isinf(A)` returns an array of the same size as `A` containing logical 1s (true) where the elements of `A` are infinite, and logical 0s (false) where they are not. For a complex number, `isinf` returns 1 if the real or imaginary part of that number is infinite or both real and imaginary parts are infinite. Otherwise, it returns 0.

### Examples

#### Determine Which Elements of Symbolic Array Are Infinite

Using `isinf`, determine which elements of this symbolic matrix are infinities:

```
isinf(sym([pi NaN Inf; 1 + i Inf + i NaN + i]))  
  
ans =  
    2×3 logical array  
    0     0     1  
    0     1     0
```

#### Determine if Exact and Approximated Values Are Infinite

Approximate these symbolic values with the 50-digit accuracy:

```
V = sym([pi, 2*pi, 3*pi, 4*pi]);  
V_approx = vpa(V, 50);
```

The cotangents of the exact values are infinite:

```
cot(V)  
isinf(cot(V))  
  
ans =  
[ Inf, Inf, Inf, Inf]  
  
ans =  
    1×4 logical array  
    1     1     1     1
```

Nevertheless, the cotangents of the approximated values are not infinite due to the round-off errors:

```
isinf(cot(V_approx))  
  
ans =  
    1×4 logical array  
    0     0     0     0
```

## Input Arguments

### A — Input value

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic array | symbolic vector | symbolic matrix

Input value, specified as a symbolic number, variable, expression, or function, or as an array, vector, or matrix of symbolic numbers, variables, expressions, functions.

## Tips

- For any A, exactly one of the three quantities `isfinite(A)`, `isinf(A)`, or `isnan(A)` is 1 for each element.
- The elements of A are recognized as infinite if they are
  - Symbolic Inf or -Inf
  - Sums or products containing symbolic Inf or -Inf and not containing the value NaN.

## Version History

**Introduced in R2013b**

### See Also

`in` | `isAlways` | `isequal` | `isequaln` | `isfinite` | `isnan`

## isLowIndexDAE

Check if differential index of system of equations is lower than 2

### Syntax

```
tf = isLowIndexDAE(eqs,vars)
```

### Description

`tf = isLowIndexDAE(eqs,vars)` checks if the system `eqs` of first-order semilinear differential algebraic equations (DAEs) has a low differential index. If the differential index of the system is 0 or 1, then `isLowIndexDAE` returns logical 1 (true). If the differential index of `eqs` is higher than 1, then `isLowIndexDAE` returns logical 0 (false).

The number of equations `eqs` must match the number of variables `vars`.

### Examples

#### Check Differential Index of DAE System

Check if a system of first-order semilinear DAEs has a low differential index (0 or 1).

Create the following system of differential algebraic equations. Here,  $x(t)$  and  $y(t)$  are the state variables of the system. Specify the equations as a vector of symbolic equations and the variables as a vector of symbolic function calls.

```
syms x(t) y(t)
eqs = [diff(x(t),t) == x(t) + y(t); x(t)^2 + y(t)^2 == 1]
```

```
eqs =

$$\begin{pmatrix} \frac{\partial}{\partial t} x(t) = x(t) + y(t) \\ x(t)^2 + y(t)^2 = 1 \end{pmatrix}$$

```

```
vars = [x(t), y(t)]
```

```
vars = (x(t) y(t))
```

Use `isLowIndexDAE` to check the differential order of the system. The differential order of this system is 1. For systems of index 0 and 1, `isLowIndexDAE` returns 1 (true).

```
tf = isLowIndexDAE(eqs,vars)
```

```
tf = logical
     1
```

## Reduce Differential Index of DAE System

Check if the following DAE system has a low or high differential index. If the index is higher than 1, then use `reduceDAEIndex` to reduce it.

Create the following system of differential algebraic equations. Here,  $x(t)$ ,  $y(t)$ , and  $z(t)$  are the state variables of the system. Specify the equations as a vector of symbolic equations and the variables as a vector of symbolic function calls.

```
syms x(t) y(t) z(t) f(t)
eqs = [diff(x(t),t) == x(t) + z(t);
       diff(y(t),t) == f(t);
       x(t) == y(t)];
vars = [x(t), y(t), z(t)];
```

Use `isLowIndexDAE` to check the differential index of the system. For this system `isLowIndexDAE` returns 0 (false). This means that the differential index of the system is 2 or higher.

```
tf = isLowIndexDAE(eqs,vars)
```

```
tf = logical
    0
```

Use `reduceDAEIndex` to rewrite the system so that the differential index is 1. Calling this function with four output arguments also shows the differential index of the original system. The new system has one additional state variable,  $Dy(t)$ .

```
[newEqs,newVars,~,oldIndex] = reduceDAEIndex(eqs,vars)
```

```
newEqs =

$$\begin{pmatrix} \frac{\partial}{\partial t} x(t) - x(t) - z(t) \\ Dy(t) - f(t) \\ x(t) - y(t) \\ \frac{\partial}{\partial t} x(t) - Dy(t) \end{pmatrix}$$

```

```
newVars =

$$\begin{pmatrix} x(t) \\ y(t) \\ z(t) \\ Dy(t) \end{pmatrix}$$

```

```
oldIndex =
2
```

Check if the differential order of the new system is lower than 2.

```
tf = isLowIndexDAE(newEqs,newVars)
```

```
tf = logical
    1
```

## Input Arguments

### **eqs — System of first-order semilinear differential algebraic equations**

vector of symbolic equations | vector of symbolic expressions

System of first-order semilinear differential algebraic equations, specified as a vector of symbolic equations or expressions.

### **vars — State variables**

vector of symbolic functions | vector of symbolic function calls

State variables, specified as a vector of symbolic functions or function calls, such as  $x(t)$ .

Example:  $[x(t), y(t)]$

## Version History

**Introduced in R2014b**

### **See Also**

`daeFunction` | `decic` | `findDecoupledBlocks` | `incidenceMatrix` | `massMatrixForm` | `odeFunction` | `reduceDAEIndex` | `reduceDAEToODE` | `reduceDifferentialOrder` | `reduceRedundancies`

### **Topics**

“Solve Differential Algebraic Equations (DAEs)” on page 3-66



# isnan

Check whether symbolic array elements are NaNs

## Syntax

```
isnan(A)
```

## Description

`isnan(A)` returns an array of the same size as `A` containing logical 1s (true) where the elements of `A` are symbolic NaNs, and logical 0s (false) where they are not.

## Examples

### Determine Which Elements of Symbolic Array Are NaNs

Using `isnan`, determine which elements of this symbolic matrix are NaNs:

```
isnan(sym([pi NaN Inf; 1 + i Inf + i NaN + i]))
```

```
ans =
    2×3 logical array
    0     1     0
    0     0     1
```

## Input Arguments

### A — Input value

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic array | symbolic vector | symbolic matrix

Input value, specified as a symbolic number, variable, expression, or function, or as an array, vector, or matrix of symbolic numbers, variables, expressions, functions.

## Tips

- For any `A`, exactly one of the three quantities `isfinite(A)`, `isinf(A)`, or `isnan(A)` is 1 for each element.
- Symbolic expressions and functions containing NaN evaluate to NaN. For example, `sym(NaN + i)` returns symbolic NaN.

## Version History

Introduced in R2013b

## See Also

`isAlways` | `isequal` | `isequaln` | `isfinite` | `isinf`

## isolate

Isolate variable or expression in equation

### Syntax

```
isolate(eqn, expr)
```

### Description

`isolate(eqn, expr)` rearranges the equation `eqn` so that the expression `expr` appears on the left side. The result is similar to solving `eqn` for `expr`. If `isolate` cannot isolate `expr`, it moves all terms containing `expr` to the left side. The output of `isolate` lets you eliminate `expr` from `eqn` by using `subs`.

### Examples

#### Isolate Variable in Equation

Isolate  $x$  in the equation  $a*x^2 + b*x + c == 0$ .

```
syms x a b c
eqn = a*x^2 + b*x + c == 0;
xSol = isolate(eqn, x)
```

```
xSol =
x == -(b + (b^2 - 4*a*c)^(1/2))/(2*a)
```

You can use the output of `isolate` to eliminate the variable from the equation using `subs`.

Eliminate  $x$  from `eqn` by substituting `lhs(xSol)` for `rhs(xSol)`.

```
eqn2 = subs(eqn, lhs(xSol), rhs(xSol))

eqn2 =
c + (b + (b^2 - 4*a*c)^(1/2))^2/(4*a) - (b*(b + (b^2 - 4*a*c)^(1/2)))/(2*a) == 0
```

#### Isolate Expression in Equation

Isolate  $y(t)$  in the following equation.

```
syms y(t)
eqn = a*y(t)^2 + b*c == 0;
isolate(eqn, y(t))

ans =
y(t) == ((-b)^(1/2)*c^(1/2))/a^(1/2)
```

Isolate  $a*y(t)$  in the same equation.

```
isolate(eqn, a*y(t))

ans =
a*y(t) == -(b*c)/y(t)
```

## isolate Returns Simplest Solution

For equations with multiple solutions, `isolate` returns the simplest solution.

Demonstrate this behavior by isolating  $x$  in  $\sin(x) == 0$ , which has multiple solutions at  $0$ ,  $\pi$ ,  $3\pi/2$ , and so on.

```
isolate(sin(x) == 0, x)
```

```
ans =  
x == 0
```

`isolate` does not consider special cases when returning the solution. Instead, `isolate` returns a general solution that is not guaranteed to hold for all values of the variables in the equation.

Isolate  $x$  in the equation  $a*x^2/(x-a) == 1$ . The returned value of  $x$  does not hold in the special case  $a = 0$ .

```
syms a x  
isolate(a*x^2/(x-a) == 1, x)  
  
ans =  
x == ((-(2*a - 1)*(2*a + 1))^(1/2) + 1)/(2*a)
```

## isolate Follows Assumptions on Variables

`isolate` returns only results that are consistent with the assumptions on the variables in the equation.

First, assume  $x$  is negative, and then isolate  $x$  in the equation  $x^4 == 1$ .

```
syms x  
assume(x < 0)  
eqn = x^4 == 1;  
isolate(x^4 == 1, x)  
  
ans =  
x == -1
```

Remove the assumption. `isolate` chooses a different solution to return.

```
assume(x, 'clear')  
isolate(x^4 == 1, x)  
  
ans =  
x == 1
```

## Tips

- If `eqn` has no solution, `isolate` errors. `isolate` also ignores special cases. If the only solutions to `eqn` are special cases, then `isolate` ignores those special cases and errors.
- The returned solution is not guaranteed to hold for all values of the variables in the solution.
- `expr` cannot be a mathematical constant such as `pi`.

## Input Arguments

### **eqn** — Input equation

symbolic equation

Input equation, specified as a symbolic equation.

Example:  $a*x^2 + b*x + c == 0$

### **expr** — Variable or expression to isolate

symbolic variable | symbolic expression

Variable or expression to isolate, specified as a symbolic variable or expression.

## Version History

**Introduced in R2017a**

### See Also

`dsolve` | `lhs` | `linsolve` | `rhs` | `solve` | `subs` | `vpasolve`

### Topics

"Solve Algebraic Equations" on page 3-3

"Solve System of Algebraic Equations" on page 3-7

"Solve Equations Numerically" on page 3-28

"Solve System of Linear Equations" on page 3-36

# isPrimitiveRoot

Determine which array elements are primitive roots

## Syntax

```
TF = isPrimitiveRoot(G,N)
```

## Description

`TF = isPrimitiveRoot(G,N)` returns a logical array containing 1 (`true`) for the corresponding elements of `G` that are primitive roots on page 6-854 modulo `N`, and 0 (`false`) for the corresponding elements that are not primitive roots. The elements of `G` must be integers, and the elements of `N` must be positive integers.

## Examples

### Find Primitive Roots Modulo 11

Create a row vector containing positive integers from 1 to 11. Determine if they are primitive roots modulo 11.

```
G = 1:11;
TF = isPrimitiveRoot(G,11)
```

```
TF = 1×11 logical array
```

```
    0    1    0    0    0    1    1    1    0    0    0
```

Find the smallest positive integer that is a primitive root modulo 11.

```
Z1 = find(TF,1)
```

```
Z1 =
     2
```

Show all positive integers (less than or equal to 11) that are primitive roots modulo 11.

```
Z = G(TF)
```

```
Z = 1×4
```

```
     2     6     7     8
```

### Find Primitive Roots Modulo 15

Create a row vector containing integers from -15 to 15. Find the integers that are primitive roots modulo 15.

```
G = -15:15;  
Z = G(isPrimitiveRoot(G,15))  
  
Z =  
  
1×0 empty double row vector
```

The integer 15 has no primitive roots.

## Input Arguments

### **G — Base**

number | vector | matrix | array | symbolic number | symbolic array

Base, specified as a number, vector, matrix, array, symbolic number, or symbolic array. The elements of G must be integers. G and N must be the same size, or else one of them must be a scalar.

Data Types: single | double | sym

### **N — Divisor**

number | vector | matrix | array | symbolic number | symbolic array

Divisor, specified as a number, vector, matrix, array, symbolic number, or symbolic array. The elements of N must be positive integers. G and N must be the same size, or else one of them must be a scalar.

Data Types: single | double | sym

## More About

### **Primitive Root**

A number  $g$  is a primitive root modulo  $n$  if every number  $a$  coprime to  $n$  (or  $\gcd(a, n) = 1$ ) is congruent to a power of  $g$  modulo  $n$ . That is,  $g$  is a primitive root modulo  $n$  if for every integer  $a$  coprime to  $n$ , there is an integer  $k$  such that  $g^k \equiv a \pmod{n}$ . The primitive roots modulo  $n$  exist if and only if  $n = 1, 2, 4, p^k$ , or  $2p^k$ , where  $p$  is an odd prime and  $k$  is a positive integer.

For example, the integer 2 is a primitive root modulo 5 because  $2^k \equiv a \pmod{5}$  is satisfied for every integer  $a$  that is coprime to 5.

$$\begin{aligned}2^1 &= 2 \equiv 2 \pmod{5} \\2^2 &= 4 \equiv 4 \pmod{5} \\2^3 &= 8 \equiv 3 \pmod{5} \\2^4 &= 16 \equiv 1 \pmod{5} \\&\vdots\end{aligned}$$

## Version History

Introduced in R2020a

**See Also**

eulerPhi | prevprime | nextprime | nthprime

## isSymType

Determine whether symbolic object is specific type

### Syntax

```
TF = isSymType(symObj,type)
TF = isSymType(symObj,funType,vars)
```

### Description

`TF = isSymType(symObj,type)` returns logical 1 (true) if the symbolic object `symObj` is of type `type`, and logical 0 (false) otherwise. The input `type` must be a case-sensitive string scalar or character vector, and it can include a logical expression. For example, `isSymType(sym('3'),'real & integer')` returns logical 1.

If `symObj` is a symbolic expression with a topmost operator of type `type`, then `isSymType(symObj,type)` also returns logical 1.

`TF = isSymType(symObj,funType,vars)` checks whether `symObj` is an unassigned symbolic function that depends on the symbolic variables `vars`.

You can set the function type `funType` to `'symfunOf'` or `'symfunDependingOn'`. For example, `syms f(x); isSymType(f,'symfunOf',x)` returns logical 1.

### Examples

#### Symbolic Number and Constant

Create a symbolic number. Check whether the symbolic number is of type `'rational'`.

```
a = sym('1/2');
TF = isSymType(a,'rational')
```

```
TF = logical
    1
```

Now construct a symbolic array by including symbolic numbers or constants in the array elements.

```
N = [sym('1/2'), vpa(0.5), pi, vpa(pi), 1i]
```

```
N =
    (1/2 0.5 pi 3.1415926535897932384626433832795 i)
```

Check whether each array element is of type `'real'`.

```
TF = isSymType(N,'real')
```

```
TF = 1x5 logical array
```



```
1  1  0  1  0
```

Check whether each array element is of type 'integer | real'.

```
TF = isSymType(N, 'integer | real')
```

```
TF = 1×5 logical array
```

```
1  1  0  1  0
```

Check whether each array element is of type 'number'.

```
TF = isSymType(N, 'number')
```

```
TF = 1×5 logical array
```

```
1  1  0  1  1
```

Check whether each array element is of type 'constant'.

```
TF = isSymType(N, 'constant')
```

```
TF = 1×5 logical array
```

```
1  1  1  1  1
```

### Topmost Operator of Symbolic Expression

Determine whether the topmost operator of a symbolic expression is of a specific type, such as 'plus' or 'power'.

Create a symbolic expression.

```
syms x
expr = x^2 + 2*x - 1
```

```
expr = x2 + 2x - 1
```

Check whether the topmost operator of expr is of type 'plus'.

```
TF = isSymType(expr, 'plus')
```

```
TF = logical
1
```

Check whether the topmost operator of expr is of type 'power'.

```
TF = isSymType(expr, 'power')
```

```
TF = logical
0
```

Now perform a symbolic square root operation in the expression.

```
expr = sqrt(x^2 + 2*x - 1)
```

```
expr =  $\sqrt{x^2 + 2x - 1}$ 
```

Check whether the topmost operator of `expr` is of type 'power'.

```
TF = isSymType(expr, 'power')
```

```
TF = logical  
    1
```

### Select Specific Equations

Select specific equations that are constant on the right side.

Create an array of three symbolic equations.

```
syms r(t) x(t) y(t)  
eq1 = [x(t) == r(t)*cos(t), y(t) == r(t)*sin(t), r(t) == 5]  
  
eq1 = (x(t) = cos(t) r(t) y(t) = r(t) sin(t) r(t) = 5)
```

Select the right side of each equation using the `rhs` function. Check whether the right side of each equation is of type 'constant'.

```
TF = isSymType(rhs(eq1), 'constant')
```

```
TF = 1x3 logical array  
  
    0    0    1
```

Return the reduced equation that is constant on the right side.

```
eq2 = eq1(TF)  
  
eq2 = r(t) = 5
```

### Symbolic Function of Multiple Variables

Create a symbolic function of multiple variables  $f(x, y)$  using `syms`. Check whether the unassigned symbolic function `f` is of type 'symfun'.

```
syms f(x, y)  
TF = isSymType(f, 'symfun')  
  
TF = logical  
    1
```

Check whether `f` depends on the exact variable `x`.

```
TF = isSymType(f, 'symfunOf', x)
```

```
TF = logical
    0
```

Check whether  $f$  depends on the exact sequence of variables  $[x \ y]$ .

```
TF = isSymType(f, 'symfunOf', [x y])
```

```
TF = logical
    1
```

Check whether  $f$  depends on the variable  $x$ .

```
TF = isSymType(f, 'symfunDependingOn', x)
```

```
TF = logical
    1
```

## Input Arguments

### symObj — Symbolic objects

symbolic expressions | symbolic functions | symbolic variables | symbolic numbers | symbolic units

Symbolic objects, specified as symbolic expressions, symbolic functions, symbolic variables, symbolic numbers, or symbolic units.

### type — Symbolic types

scalar string | character vector

Symbolic types, specified as a case-sensitive scalar string or character vector. The input type can contain a logical expression. The value options follow.

| Symbolic Type Category | String Values                                                                                                                                                                                                                                                                                                                                                                                               | Examples Returning Logical 1                                                                                                                                                                                                                                                                            |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| numbers                | <ul style="list-style-type: none"> <li>'integer' — integer numbers</li> <li>'rational' — rational numbers</li> <li>'vpareal' — variable-precision floating-point real numbers</li> <li>'complex' — complex numbers</li> <li>'real' — real numbers, including 'integer', 'rational', and 'vpareal'</li> <li>'number' — numbers, including 'integer', 'rational', 'vpareal', 'complex', and 'real'</li> </ul> | <ul style="list-style-type: none"> <li>isSymType(sym(2), 'integer')</li> <li>isSymType(sym(1/2), 'rational')</li> <li>isSymType(vpa(0.5), 'vpareal')</li> <li>isSymType(vpa(1i), 'complex')</li> <li>isSymType([sym(1/2) vpa(0.5)], 'real')</li> <li>isSymType([vpa(1i) sym(1/2)], 'number')</li> </ul> |

| Symbolic Type Category        | String Values                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Examples Returning Logical 1                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| constants                     | 'constant' — symbolic mathematical constants, including 'number'                                                                                                                                                                                                                                                                                                                                                                                                     | <code>isSymType([sym(pi) vpa(1i)], 'constant')</code>                                                                                                                                                                                                                                                                                                                                                  |
| symbolic math functions       | 'vpa', 'sin', 'exp', and so on — topmost symbolic math functions in symbolic expressions                                                                                                                                                                                                                                                                                                                                                                             | <code>isSymType(vpa(sym(pi)), 'vpa')</code>                                                                                                                                                                                                                                                                                                                                                            |
| unassigned symbolic functions | <ul style="list-style-type: none"> <li>'F', 'g', and so on — function name of an unassigned symbolic function</li> <li>'symfun' — unassigned symbolic functions</li> </ul>                                                                                                                                                                                                                                                                                           | <ul style="list-style-type: none"> <li><code>syms F(x); isSymType(F(x+2), 'F')</code></li> <li><code>syms g(x); isSymType(g(x), 'symfun')</code></li> </ul>                                                                                                                                                                                                                                            |
| arithmetic operators          | <ul style="list-style-type: none"> <li>'plus' — addition operator + and subtraction operator -</li> <li>'times' — multiplication operator * and division operator /</li> <li>'power' — power or exponentiation operator ^ and square root operator sqrt</li> </ul>                                                                                                                                                                                                   | <ul style="list-style-type: none"> <li><code>syms x y; isSymType(2*x + y, 'plus')</code></li> <li><code>syms x y; isSymType(x*y, 'times')</code></li> <li><code>syms x y; isSymType(x^(y+2), 'power')</code></li> </ul>                                                                                                                                                                                |
| variables                     | 'variable' — symbolic variables                                                                                                                                                                                                                                                                                                                                                                                                                                      | <code>isSymType(sym('x'), 'variable')</code>                                                                                                                                                                                                                                                                                                                                                           |
| units                         | 'unit' — symbolic units                                                                                                                                                                                                                                                                                                                                                                                                                                              | <code>isSymType(symunit('m'), 'unit')</code>                                                                                                                                                                                                                                                                                                                                                           |
| expressions                   | 'expression' — symbolic expressions, including all of the preceding symbolic types                                                                                                                                                                                                                                                                                                                                                                                   | <code>isSymType(sym('x') + 1, 'expression')</code>                                                                                                                                                                                                                                                                                                                                                     |
| logical expressions           | <ul style="list-style-type: none"> <li>'or' — logical OR operator  </li> <li>'and' — logical AND operator &amp;</li> <li>'not' — logical NOT operator ~</li> <li>'xor' — logical exclusive-OR operator xor</li> <li>'logicalconstant' — symbolic logical constants <code>symtrue</code> and <code>symfalse</code></li> <li>'logicalexpression' — logical expressions, including 'or', 'and', 'not', 'xor', <code>symtrue</code> and <code>symfalse</code></li> </ul> | <ul style="list-style-type: none"> <li><code>syms x y; isSymType(x y, 'or')</code></li> <li><code>syms x y; isSymType(x&amp;y, 'and')</code></li> <li><code>syms x; isSymType(~x, 'not')</code></li> <li><code>syms x y; isSymType(xor(x,y), 'xor')</code></li> <li><code>isSymType(symtrue, 'logicalconstant')</code></li> <li><code>syms x y; isSymType(~x y, 'logicalexpression')</code></li> </ul> |

| Symbolic Type Category     | String Values                                                                                                                                                                                                                                                                                                                                                                              | Examples Returning Logical 1                                                                                                                                                                                                                                                   |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| equations and inequalities | <ul style="list-style-type: none"> <li>'eq' — equality operator ==</li> <li>'ne' — inequality operator ~=</li> <li>'lt' — less-than operator &lt; or greater-than operator &gt;</li> <li>'le' — less-than-or-equal-to operator &lt;= or greater-than-or-equal-to operator &gt;=</li> <li>'equation' — symbolic equations and inequalities, including 'eq', 'ne', 'lt', and 'le'</li> </ul> | <ul style="list-style-type: none"> <li>syms x;<br/>isSymType(x==2,'eq')</li> <li>syms x;<br/>isSymType(x~=1,'ne')</li> <li>syms x;<br/>isSymType(x&gt;0,'lt')</li> <li>syms x;<br/>isSymType(x&lt;=2,'le')</li> <li>syms x; isSymType([x&gt;0<br/>x~=1],'equation')</li> </ul> |
| unsupported symbolic types | 'unsupported' — unsupported symbolic types                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                |

### funType — Function type

'symfunOf' | 'symfunDependingOn'

Function type, specified as 'symfunOf' or 'symfunDependingOn'.

- 'symfunOf' checks whether symObj is an unassigned symbolic function that depends on the exact sequence of variables specified by the array vars. For example, syms f(x,y); isSymType(f,'symfunOf',[x y]) returns logical 1.
- 'symfunDependingOn' checks whether symObj is an unassigned symbolic function that depends on the variables specified by the array vars. For example, syms f(x,y); isSymType(f,'symfunDependingOn',x) returns logical 1.

### vars — Input variables

symbolic variables | symbolic array

Input variables, specified as symbolic variables or a symbolic array.

## Version History

Introduced in R2019a

### See Also

symFunType | hasSymType | symType | sym | syms

## isUnit

Determine if input is a symbolic unit

### Syntax

```
tf = isUnit(expr)
```

### Description

`tf = isUnit(expr)` returns logical 1 (true) if `expr` is a unit, or a product of powers of units, and logical 0 (false) if it is not.

### Examples

#### Determine if Input is a Unit

Determine if an expression is a symbolic unit by using `isUnit`.

Test if `3*u.m` is a symbolic unit, where `u = symunit`. The `isUnit` function returns logical 0 (false) because `3*u.m` contains the symbolic number 3.

```
u = symunit;  
isUnit(3*u.m)
```

```
ans =  
    logical  
         0
```

Check if `u.m`, `u.mW`, and `x*u.Hz` are units, where `u = symunit`. The `isUnit` function returns the array `[1 1 0]`, meaning that the first two expressions are units but the third expression is not.

```
syms x  
units = [u.m u.mW x*u.Hz];  
isUnit(units)
```

```
ans =  
    1x3 logical array  
         1     1     0
```

### Input Arguments

#### **expr** — Input

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

## Tips

- 1 represents a dimensionless unit. Hence, `isUnit(sym(1))` returns logical 1 (true).

## Version History

Introduced in R2017a

## See Also

`checkUnits` | `findUnits` | `newUnit` | `separateUnits` | `symunit` | `str2symunit` | `symunit2str` | `unitConversionFactor`

## Topics

“Units of Measurement Tutorial” on page 2-47

“Unit Conversions and Unit Systems” on page 2-53

“Units and Unit Systems List” on page 2-60

## External Websites

The International System of Units (SI)

## iztrans

Inverse Z-transform

### Syntax

```
iztrans(F)
iztrans(F,transVar)
iztrans(F,var,transVar)
```

### Description

`iztrans(F)` returns the “Inverse Z-Transform” on page 6-866 of F. By default, the independent variable is  $z$  and the transformation variable is  $n$ . If F does not contain  $z$ , `iztrans` uses the function `symvar`.

`iztrans(F,transVar)` uses the transformation variable `transVar` instead of  $n$ .

`iztrans(F,var,transVar)` uses the independent variable `var` and transformation variable `transVar` instead of  $z$  and  $n$  respectively.

### Examples

#### Inverse Z-Transform of Symbolic Expression

Compute the inverse Z-transform of  $2*z/(z-2)^2$ . By default, the inverse transform is in terms of  $n$ .

```
syms z
F = 2*z/(z-2)^2;
iztrans(F)
```

```
ans =
2^n + 2^n*(n - 1)
```

#### Specify Independent Variable and Transformation Variable

Compute the inverse Z-transform of  $1/(a*z)$ . By default, the independent and transformation variables are  $z$  and  $n$ , respectively.

```
syms z a
F = 1/(a*z);
iztrans(F)
```

```
ans =
kroneckerDelta(n - 1, 0)/a
```

Specify the transformation variable as  $m$ . If you specify only one variable, that variable is the transformation variable. The independent variable is still  $z$ .

```
syms m
iztrans(F,m)
```

```
ans =
kroneckerDelta(m - 1, 0)/a
```



Specify both the independent and transformation variables as `a` and `m` in the second and third arguments, respectively.

```
iztrans(F,a,m)

ans =
kroneckerDelta(m - 1, 0)/z
```

### Inverse Z-Transforms Involving Kronecker Delta Function

Compute the inverse Z-transforms of these expressions. The results involve the Kronecker Delta function.

```
syms n z
iztrans(1/z,z,n)

ans =
kroneckerDelta(n - 1, 0)

f = (z^3 + 3*z^2)/z^5;
iztrans(f,z,n)

ans =
kroneckerDelta(n - 2, 0) + 3*kroneckerDelta(n - 3, 0)
```

### Inverse Z-Transform of Array Inputs

Find the inverse Z-transform of the matrix `M`. Specify the independent and transformation variables for each matrix entry by using matrices of the same size. When the arguments are nonscalars, `iztrans` acts on them element-wise.

```
syms a b c d w x y z
M = [exp(x) 1; sin(y) i*z];
vars = [w x; y z];
transVars = [a b; c d];
iztrans(M,vars,transVars)

ans =
[ exp(x)*kroneckerDelta(a, 0), kroneckerDelta(b, 0)]
[      iztrans(sin(y), y, c),   iztrans(z, z, d)*1i]
```

If `iztrans` is called with both scalar and nonscalar arguments, then it expands the scalars to match the nonscalars by using scalar expansion. Nonscalar arguments must be the same size.

```
syms w x y z a b c d
iztrans(x,vars,transVars)

ans =
[ x*kroneckerDelta(a, 0),      iztrans(x, x, b)]
[ x*kroneckerDelta(c, 0), x*kroneckerDelta(d, 0)]
```

### Inverse Z-Transform of Symbolic Function

Compute the Inverse Z-transform of symbolic functions. When the first argument contains symbolic functions, then the second argument must be a scalar.

```
syms f1(x) f2(x) a b
f1(x) = exp(x);
```

```
f2(x) = x;
iztrans([f1, f2], x, [a, b])

ans =
[ iztrans(exp(x), x, a), iztrans(x, x, b)]
```

### If Inverse Z-Transform Cannot Be Found

If `iztrans` cannot compute the inverse transform, it returns an unevaluated call.

```
syms F(z) n
F(z) = exp(z);
f = iztrans(F, z, n)

f =
iztrans(exp(z), z, n)
```

Return the original expression by using `ztrans`.

```
ztrans(f, n, z)

ans =
exp(z)
```

## Input Arguments

### F — Input

symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic expression, function, vector, or matrix.

### var — Independent variable

x (default) | symbolic variable | symbolic expression | symbolic vector | symbolic matrix

Independent variable, specified as a symbolic variable, expression, vector, or matrix. This variable is often called the "complex frequency variable." If you do not specify the variable, then `iztrans` uses `z`. If `F` does not contain `z`, then `iztrans` uses the function `symvar`.

### transVar — Transformation variable

x (default) | t | symbolic variable | symbolic expression | symbolic vector | symbolic matrix

Transformation variable, specified as a symbolic variable, expression, vector, or matrix. It is often called the "time variable" or "space variable." By default, `iztrans` uses `n`. If `n` is the independent variable of `F`, then `iztrans` uses `k`.

## More About

### Inverse Z-Transform

Where  $R$  is a positive number, such that the function  $F = F(z)$  is analytic on and outside the circle  $|z| = R$ , the inverse Z-transform is

$$f(n) = \frac{1}{2\pi i} \oint_{|z|=R} F(z) z^{n-1} dz, \quad n = 0, 1, 2, \dots$$

## Tips

- If any argument is an array, then `iztrans` acts element-wise on all elements of the array.
- If the first argument contains a symbolic function, then the second argument must be a scalar.
- To compute the direct Z-transform, use `ztrans`.

## Version History

Introduced before R2006a

## See Also

`fourier` | `ifourier` | `ilaplace` | `kroneckerDelta` | `laplace` | `ztrans`

## Topics

“Solve Difference Equations Using Z-Transform” on page 3-205

## **jacobiAM**

Jacobi amplitude function

### **Syntax**

```
jacobiAM(u,m)
```

### **Description**

`jacobiAM(u,m)` returns the “Jacobi Amplitude Function” on page 6-870 of `u` and `m`. If `u` or `m` is an array, then `jacobiAM` acts element-wise.

### **Examples**

#### **Calculate Jacobi Amplitude Function for Numeric Inputs**

```
jacobiAM(2,1)
```

```
ans =  
    1.3018
```

Call `jacobiAM` on array inputs. `jacobiAM` acts element-wise when `u` or `m` is an array.

```
jacobiAM([2 1 -3],[1 2 3])
```

```
ans =  
    1.3018    0.7370    0.6155
```

#### **Calculate Jacobi Amplitude Function for Symbolic Numbers**

Convert numeric input to symbolic form using `sym`, and find the Jacobi amplitude function. For symbolic input where `u = 0` or `m = 0` or `1`, `jacobiAM` returns exact symbolic output.

```
jacobiAM(sym(2),sym(1))
```

```
ans =  
2*atan(exp(2)) - pi/2
```

Show that for other values of `u` or `m`, `jacobiAM` returns an unevaluated function call.

```
jacobiAM(sym(2),sym(3))
```

```
ans =  
jacobiAM(2, 3)
```

#### **Find Jacobi Amplitude Function for Symbolic Variables or Expressions**

For symbolic variables or expressions, `jacobiAM` returns the unevaluated function call.

```
syms x y
f = jacobiAM(x,y)
```

```
f =
jacobiAM(x, y)
```

Substitute values for the variables by using `subs`, and convert values to double by using `double`.

```
f = subs(f, [x y], [3 5])
```

```
f =
jacobiAM(3, 5)
```

```
fVal = double(f)
```

```
fVal =
    0.0311
```

Calculate `f` to higher precision using `vpa`.

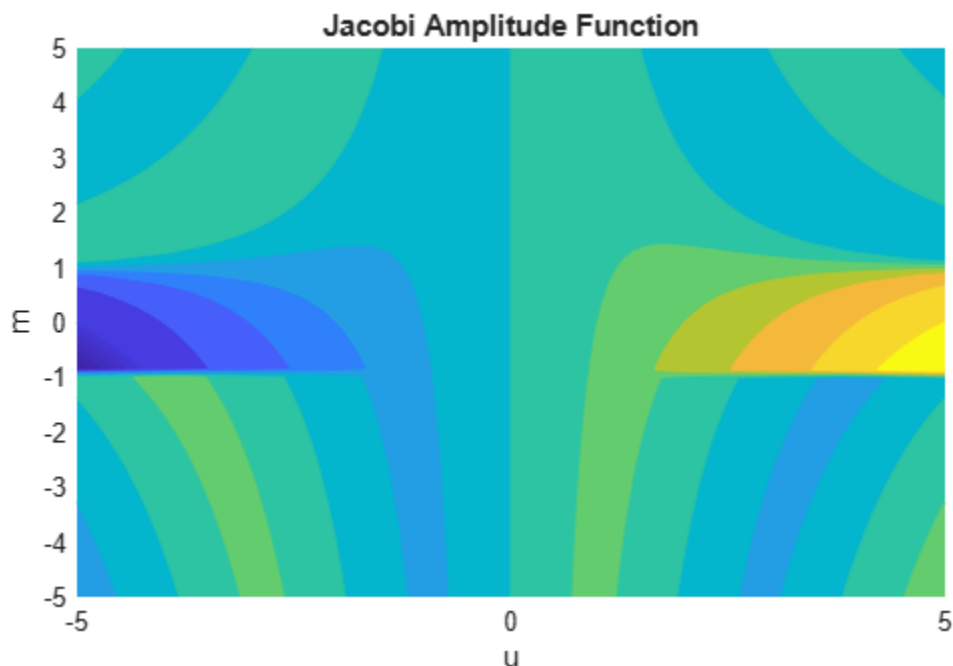
```
fVal = vpa(f)
```

```
fVal =
0.031149815412430844987208470634926
```

### Plot Jacobi Amplitude Function

Plot the Jacobi amplitude function using `fcontours`. Set `u` on the x-axis and `m` on the y-axis by using the symbolic function `f` with the variable order `(u,m)`. Fill plot contours by setting `Fill` to on.

```
syms f(u,m)
f(u,m) = jacobiAM(u,m);
fcontour(f, 'Fill', 'on')
title('Jacobi Amplitude Function')
xlabel('u')
ylabel('m')
```



## Input Arguments

### **u — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **m — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

## More About

### **Jacobi Amplitude Function**

The Jacobi amplitude function  $\text{am}(u,m)$  is defined by  $\text{am}(u,m) = \varphi$  where  $F(\varphi,m) = u$  and  $F$  represents the incomplete elliptic integral of the first kind.  $F$  is implemented as `ellipticF`.

## Version History

**Introduced in R2017b**

### **See Also**

`ellipticF` | `jacobiCD` | `jacobiCN` | `jacobiCS` | `jacobiDC` | `jacobiDN` | `jacobiDS` | `jacobiNC` | `jacobiND` | `jacobiNS` | `jacobiSC` | `jacobiSD` | `jacobiSN` | `jacobiZeta`

# jacobian

Jacobian matrix of symbolic function

## Syntax

`jacobian(f,v)`

## Description

`jacobian(f,v)` computes the Jacobian matrix on page 6-873 of symbolic function `f` with respect to `v`. The  $(i,j)$  element of the result is  $\frac{\partial f(i)}{\partial v(j)}$ .

## Examples

### Jacobian of Vector Function

The Jacobian of a vector function is a matrix of the partial derivatives of that function.

Compute the Jacobian matrix of  $[x*y*z, y^2, x + z]$  with respect to  $[x, y, z]$ .

```
syms x y z
jacobian([x*y*z,y^2,x + z],[x,y,z])
```

```
ans =
      yz  xz  xy
      0  2y  0
      1   0  1
```

Now, compute the Jacobian of  $[x*y*z, y^2, x + z]$  with respect to  $[x;y;z]$ .

```
jacobian([x*y*z,y^2,x + z],[x;y;z])
```

```
ans =
      yz  xz  xy
      0  2y  0
      1   0  1
```

The Jacobian matrix is invariant to the orientation of the vector in the second input position.

### Jacobian of Scalar Function

The Jacobian of a scalar function is the transpose of its gradient.

Compute the Jacobian of  $2*x + 3*y + 4*z$  with respect to  $[x, y, z]$ .

```
syms x y z
jacobian(2*x + 3*y + 4*z,[x,y,z])
```

```
ans = (2 3 4)
```

Now, compute the gradient of the same expression.

```
gradient(2*x + 3*y + 4*z, [x, y, z])
```

```
ans =

$$\begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix}$$

```

### Jacobian with Respect to Scalar

The Jacobian of a function with respect to a scalar is the first derivative of that function. For a vector function, the Jacobian with respect to a scalar is a vector of the first derivatives.

Compute the Jacobian of  $[x^2y, x\sin(y)]$  with respect to  $x$ .

```
syms x y
jacobian([x^2*y, x*sin(y)], x)
```

```
ans =

$$\begin{pmatrix} 2xy \\ \sin(y) \end{pmatrix}$$

```

Now, compute the derivatives.

```
diff([x^2*y, x*sin(y)], x)
```

```
ans = (2 x y sin(y))
```

### Jacobian of Coordinate Change

Specify polar coordinates  $r(t)$ ,  $\phi(t)$ , and  $\theta(t)$  that are functions of time.

```
syms r(t) phi(t) theta(t)
```

Define the coordinate transformation from spherical coordinates to Cartesian coordinates.

```
R = [r*sin(phi)*cos(theta), r*sin(phi)*sin(theta), r*cos(phi)]
```

```
R(t) = (cos(theta(t)) sin(phi(t)) r(t) sin(phi(t)) sin(theta(t)) r(t) cos(phi(t)) r(t))
```

Find the Jacobian of the coordinate change from spherical coordinates to Cartesian coordinates.

```
jacobian(R, [r, phi, theta])
```

```
ans(t) =

$$\begin{pmatrix} \cos(\theta(t)) \sin(\phi(t)) & \cos(\phi(t)) \cos(\theta(t)) r(t) & -\sin(\phi(t)) \sin(\theta(t)) r(t) \\ \sin(\phi(t)) \sin(\theta(t)) & \cos(\phi(t)) \sin(\theta(t)) r(t) & \cos(\theta(t)) \sin(\phi(t)) r(t) \\ \cos(\phi(t)) & -\sin(\phi(t)) r(t) & 0 \end{pmatrix}$$

```



## Input Arguments

### **f** — Scalar or vector function

symbolic expression | symbolic function | symbolic vector

Scalar or vector function, specified as a symbolic expression, function, or vector. If **f** is a scalar, then the Jacobian matrix of **f** is the transposed gradient of **f**.

### **v** — Vector of variables or functions with respect to which you compute Jacobian

symbolic variable | symbolic function | symbolic vector

Vector of variables or functions with respect to which you compute Jacobian, specified as a symbolic variable, symbolic function, or vector of symbolic variables. If **v** is a scalar, then the result is equal to the transpose of `diff(f,v)`. If **v** is an empty symbolic object, such as `sym([])`, then `jacobian` returns an empty symbolic object.

## More About

### Jacobian Matrix

The Jacobian matrix of the vector function  $f = (f_1(x_1, \dots, x_n), \dots, f_n(x_1, \dots, x_n))$  is the matrix of the derivatives of **f**:

$$J(x_1, \dots, x_n) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \dots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}$$

## Version History

Introduced before R2006a

### See Also

`curl` | `divergence` | `diff` | `gradient` | `hessian` | `laplacian` | `potential` | `vectorPotential`

## jacobiCD

Jacobi CD elliptic function

### Syntax

```
jacobiCD(u,m)
```

### Description

`jacobiCD(u,m)` returns the “Jacobi CD Elliptic Function” on page 6-876 of `u` and `m`. If `u` or `m` is an array, then `jacobiCD` acts element-wise.

### Examples

#### Calculate Jacobi CD Elliptic Function for Numeric Inputs

```
jacobiCD(2,1)
```

```
ans =  
1
```

Call `jacobiCD` on array inputs. `jacobiCD` acts element-wise when `u` or `m` is an array.

```
jacobiCD([2 1 -3],[1 2 3])
```

```
ans =  
1.0000    2.3829 -178.6290
```

#### Calculate Jacobi CD Elliptic Function for Symbolic Numbers

Convert numeric input to symbolic form using `sym`, and find the Jacobi CD elliptic function. For symbolic input where `u = 0` or `m = 0` or `1`, `jacobiCD` returns exact symbolic output.

```
jacobiCD(sym(2),sym(1))
```

```
ans =  
1
```

Show that for other values of `u` or `m`, `jacobiCD` returns an unevaluated function call.

```
jacobiCD(sym(2),sym(3))
```

```
ans =  
jacobiCD(2, 3)
```

#### Find Jacobi CD Elliptic Function for Symbolic Variables or Expressions

For symbolic variables or expressions, `jacobiCD` returns the unevaluated function call.

```
syms x y
f = jacobiCD(x,y)
```

```
f =
jacobiCD(x, y)
```

Substitute values for the variables by using `subs`, and convert values to double by using `double`.

```
f = subs(f, [x y], [3 5])
```

```
f =
jacobiCD(3, 5)
```

```
fVal = double(f)
```

```
fVal =
    1.0019
```

Calculate `f` to higher precision using `vpa`.

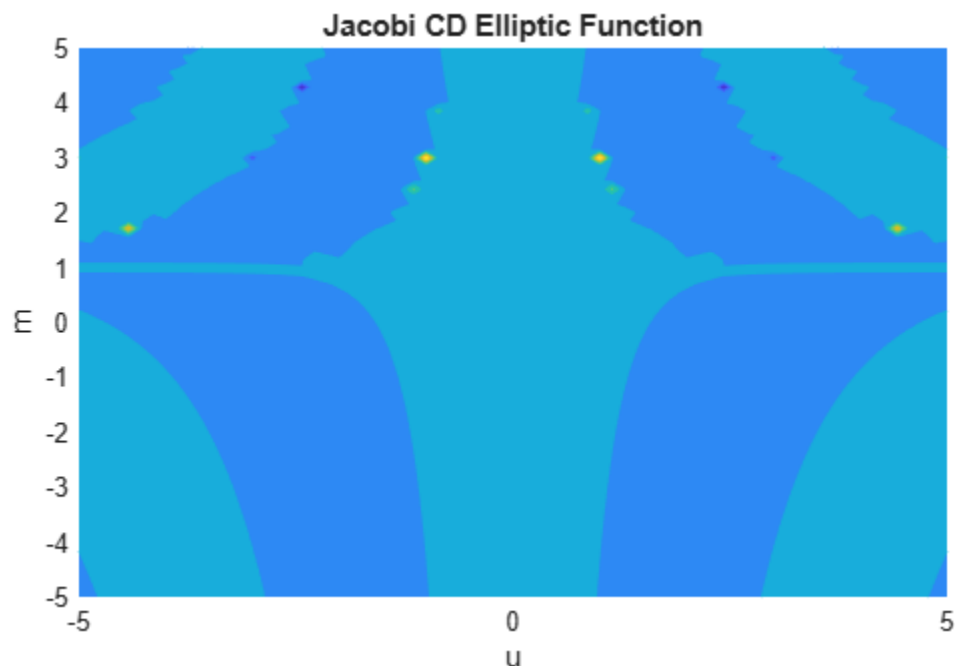
```
fVal = vpa(f)
```

```
fVal =
1.0019475527333315357888731083364
```

### Plot Jacobi CD Elliptic Function

Plot the Jacobi CD elliptic function using `fcontour`. Set `u` on the x-axis and `m` on the y-axis by using the symbolic function `f` with the variable order `(u,m)`. Fill plot contours by setting `Fill` to on.

```
syms f(u,m)
f(u,m) = jacobiCD(u,m);
fcontour(f, 'Fill', 'on')
title('Jacobi CD Elliptic Function')
xlabel('u')
ylabel('m')
```



## Input Arguments

### **u — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **m — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

## More About

### **Jacobi CD Elliptic Function**

The Jacobi CD elliptic function is

$$\text{cd}(u,m) = \text{cn}(u,m)/\text{dn}(u,m)$$

where cn and dn are the respective Jacobi elliptic functions.

The Jacobi elliptic functions are meromorphic and doubly periodic in their first argument with periods  $4K(m)$  and  $4iK'(m)$ , where  $K$  is the complete elliptic integral of the first kind, implemented as `ellipticK`.

## Version History

**Introduced in R2017b**

### **See Also**

`jacobiAM` | `jacobiCN` | `jacobiCS` | `jacobiDC` | `jacobiDN` | `jacobiDS` | `jacobiNC` | `jacobiND` | `jacobiNS` | `jacobiSC` | `jacobiSD` | `jacobiSN` | `jacobiZeta` | `ellipticK`

# **jacobiCN**

Jacobi CN elliptic function

## **Syntax**

```
jacobiCN(u,m)
```

## **Description**

`jacobiCN(u,m)` returns the “Jacobi CN Elliptic Function” on page 6-879 of `u` and `m`. If `u` or `m` is an array, then `jacobiCN` acts element-wise.

## **Examples**

### **Calculate Jacobi CN Elliptic Function for Numeric Inputs**

```
jacobiCN(2,1)
```

```
ans =  
    0.2658
```

Call `jacobiCN` on array inputs. `jacobiCN` acts element-wise when `u` or `m` is an array.

```
jacobiCN([2 1 -3],[1 2 3])
```

```
ans =  
    0.2658    0.7405    0.8165
```

### **Calculate Jacobi CN Elliptic Function for Symbolic Numbers**

Convert numeric input to symbolic form using `sym`, and find the Jacobi CN elliptic function. For symbolic input where `u = 0` or `m = 0` or `1`, `jacobiCN` returns exact symbolic output.

```
jacobiCN(sym(2),sym(1))
```

```
ans =  
1/cosh(2)
```

Show that for other values of `u` or `m`, `jacobiCN` returns an unevaluated function call.

```
jacobiCN(sym(2),sym(3))
```

```
ans =  
jacobiCN(2, 3)
```

### **Find Jacobi CN Elliptic Function for Symbolic Variables or Expressions**

For symbolic variables or expressions, `jacobiCN` returns the unevaluated function call.

```
syms x y
f = jacobiCN(x,y)

f =
jacobiCN(x, y)
```

Substitute values for the variables by using `subs`, and convert values to double by using `double`.

```
f = subs(f, [x y], [3 5])

f =
jacobiCN(3, 5)

fVal = double(f)

fVal =
    0.9995
```

Calculate `f` to higher precision using `vpa`.

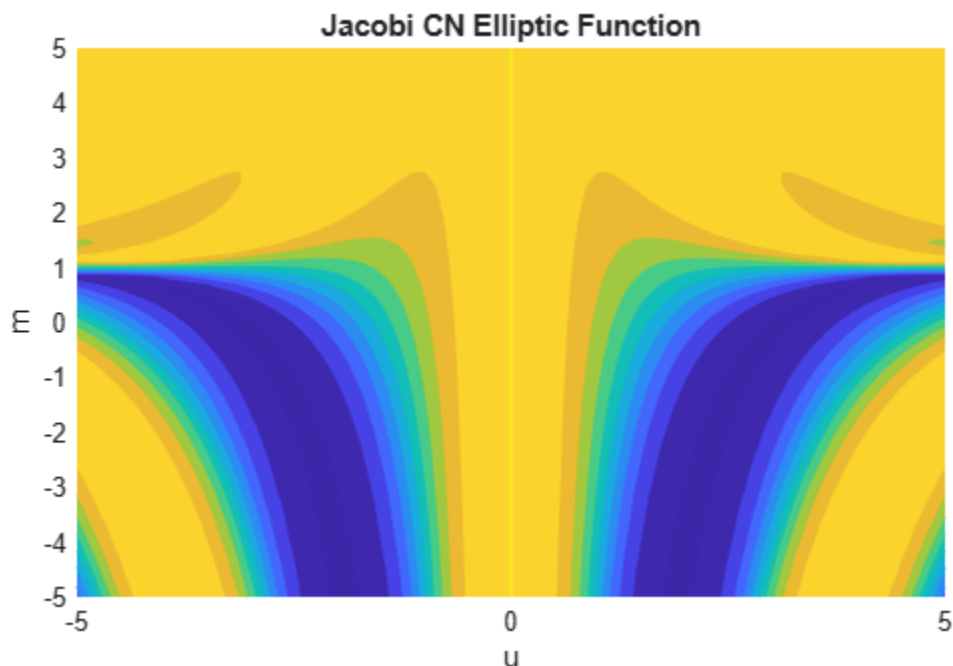
```
fVal = vpa(f)

fVal =
0.9995148837279268257000709197021
```

### Plot Jacobi CN Elliptic Function

Plot the Jacobi CN elliptic function using `fcontour`. Set `u` on the x-axis and `m` on the y-axis by using the symbolic function `f` with the variable order `(u,m)`. Fill plot contours by setting `Fill` to `on`.

```
syms f(u,m)
f(u,m) = jacobiCN(u,m);
fcontour(f, 'Fill', 'on')
title('Jacobi CN Elliptic Function')
xlabel('u')
ylabel('m')
```



## Input Arguments

### **u — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **m — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

## More About

### **Jacobi CN Elliptic Function**

The Jacobi CN elliptic function is  $\text{cn}(u,m) = \cos(\text{am}(u,m))$  where  $\text{am}$  is the Jacobi amplitude function.

The Jacobi elliptic functions are meromorphic and doubly periodic in their first argument with periods  $4K(m)$  and  $4iK'(m)$ , where  $K$  is the complete elliptic integral of the first kind, implemented as `ellipticK`.

## Version History

Introduced in R2017b

### **See Also**

`jacobiAM` | `jacobiCD` | `jacobiCS` | `jacobiDC` | `jacobiDN` | `jacobiDS` | `jacobiNC` | `jacobiND` | `jacobiNS` | `jacobiSC` | `jacobiSD` | `jacobiSN` | `jacobiZeta` | `ellipticK`

## **jacobiCS**

Jacobi CS elliptic function

### **Syntax**

```
jacobiCS(u,m)
```

### **Description**

`jacobiCS(u,m)` returns the “Jacobi CS Elliptic Function” on page 6-882 of `u` and `m`. If `u` or `m` is an array, then `jacobiCS` acts element-wise.

### **Examples**

#### **Calculate Jacobi CS Elliptic Function for Numeric Inputs**

```
jacobiCS(2,1)
```

```
ans =  
    0.2757
```

Call `jacobiCS` on array inputs. `jacobiCS` acts element-wise when `u` or `m` is an array.

```
jacobiCS([2 1 -3],[1 2 3])
```

```
ans =  
    0.2757    1.1017    1.4142
```

#### **Calculate Jacobi CS Elliptic Function for Symbolic Numbers**

Convert numeric input to symbolic form using `sym`, and find the Jacobi CS elliptic function. For symbolic input where `u = 0` or `m = 0` or `1`, `jacobiCS` returns exact symbolic output.

```
jacobiCS(sym(2),sym(1))
```

```
ans =  
1/sinh(2)
```

Show that for other values of `u` or `m`, `jacobiCS` returns an unevaluated function call.

```
jacobiCS(sym(2),sym(3))
```

```
ans =  
jacobiCS(2, 3)
```

#### **Find Jacobi CS Elliptic Function for Symbolic Variables or Expressions**

For symbolic variables or expressions, `jacobiCS` returns the unevaluated function call.



```
syms x y
f = jacobiCS(x,y)
```

```
f =
jacobiCS(x, y)
```

Substitute values for the variables by using `subs`, and convert values to double by using `double`.

```
f = subs(f, [x y], [3 5])
```

```
f =
jacobiCS(3, 5)
```

```
fVal = double(f)
```

```
fVal =
    32.0925
```

Calculate `f` to higher precision using `vpa`.

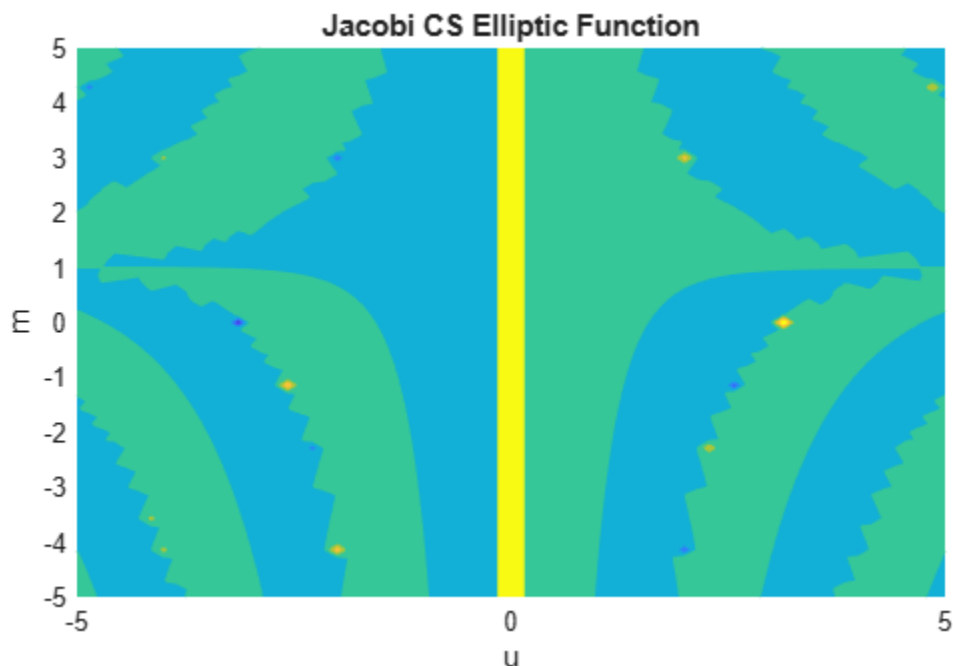
```
fVal = vpa(f)
```

```
fVal =
32.092535022751828816106562829547
```

### Plot Jacobi CS Elliptic Function

Plot the Jacobi CS elliptic function using `fcontour`. Set `u` on the x-axis and `m` on the y-axis by using the symbolic function `f` with the variable order `(u,m)`. Fill plot contours by setting `Fill` to on.

```
syms f(u,m)
f(u,m) = jacobiCS(u,m);
fcontour(f, 'Fill', 'on')
title('Jacobi CS Elliptic Function')
xlabel('u')
ylabel('m')
```



## Input Arguments

### **u — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **m — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

## More About

### **Jacobi CS Elliptic Function**

The Jacobi CS elliptic function is

$$cs(u,m) = cn(u,m)/sn(u,m)$$

where cn and sn are the respective Jacobi elliptic functions.

The Jacobi elliptic functions are meromorphic and doubly periodic in their first argument with periods  $4K(m)$  and  $4iK'(m)$ , where  $K$  is the complete elliptic integral of the first kind, implemented as `ellipticK`.

## Version History

**Introduced in R2017b**

### **See Also**

`jacobiAM` | `jacobiCD` | `jacobiCN` | `jacobiDC` | `jacobiDN` | `jacobiDS` | `jacobiNC` | `jacobiND` | `jacobiNS` | `jacobiSC` | `jacobiSD` | `jacobiSN` | `jacobiZeta` | `ellipticK`

# **jacobiDC**

Jacobi DC elliptic function

## **Syntax**

```
jacobiDC(u,m)
```

## **Description**

`jacobiDC(u,m)` returns the “Jacobi DC Elliptic Function” on page 6-885 of `u` and `m`. If `u` or `m` is an array, then `jacobiDC` acts element-wise.

## **Examples**

### **Calculate Jacobi DC Elliptic Function for Numeric Inputs**

```
jacobiDC(2,1)
```

```
ans =  
    1
```

Call `jacobiDC` on array inputs. `jacobiDC` acts element-wise when `u` or `m` is an array.

```
jacobiDC([2 1 -3],[1 2 3])
```

```
ans =  
    1.0000    0.4197   -0.0056
```

### **Calculate Jacobi DC Elliptic Function for Symbolic Numbers**

Convert numeric input to symbolic form using `sym`, and find the Jacobi DC elliptic function. For symbolic input where `u = 0` or `m = 0` or `1`, `jacobiDC` returns exact symbolic output.

```
jacobiDC(sym(2),sym(1))
```

```
ans =  
    1
```

Show that for other values of `u` or `m`, `jacobiDC` returns an unevaluated function call.

```
jacobiDC(sym(2),sym(3))
```

```
ans =  
jacobiDC(2, 3)
```

### **Find Jacobi DC Elliptic Function for Symbolic Variables or Expressions**

For symbolic variables or expressions, `jacobiDC` returns the unevaluated function call.

```
syms x y
f = jacobiDC(x,y)

f =
jacobiDC(x, y)
```

Substitute values for the variables by using `subs`, and convert values to double by using `double`.

```
f = subs(f, [x y], [3 5])

f =
jacobiDC(3, 5)

fVal = double(f)

fVal =
    0.9981
```

Calculate `f` to higher precision using `vpa`.

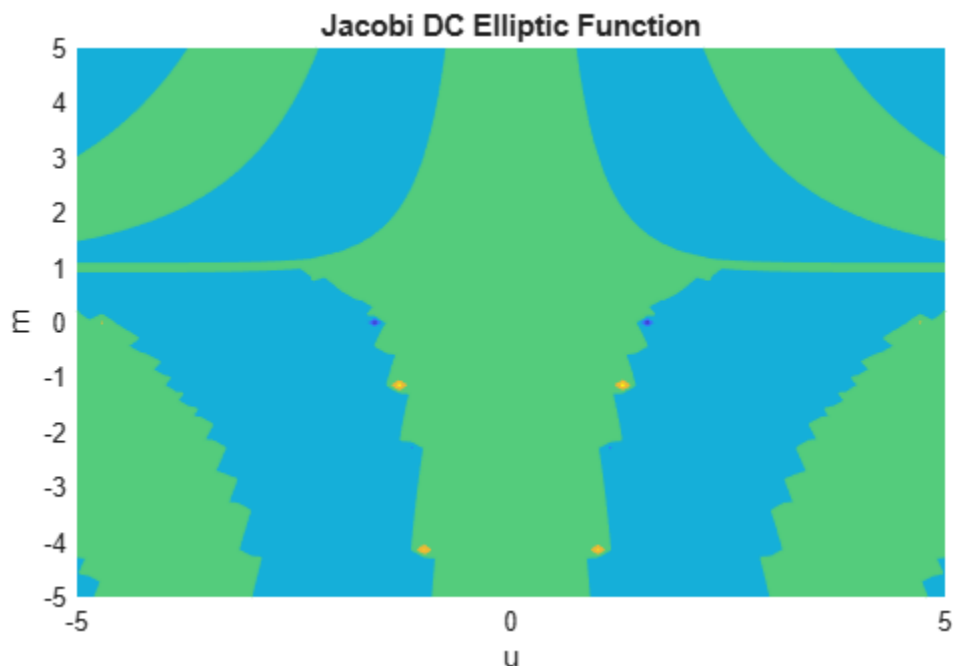
```
fVal = vpa(f)

fVal =
0.99805623285568333815968501058428
```

### Plot Jacobi DC Elliptic Function

Plot the Jacobi DC elliptic function using `fcontour`. Set `u` on the x-axis and `m` on the y-axis by using the symbolic function `f` with the variable order `(u,m)`. Fill plot contours by setting `Fill` to `on`.

```
syms f(u,m)
f(u,m) = jacobiDC(u,m);
fcontour(f, 'Fill', 'on')
title('Jacobi DC Elliptic Function')
xlabel('u')
ylabel('m')
```



## Input Arguments

### **u — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **m — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

## More About

### **Jacobi DC Elliptic Function**

The Jacobi DC elliptic function is

$$dc(u,m) = dn(u,m)/cn(u,m)$$

where dn and cn are the respective Jacobi elliptic functions.

The Jacobi elliptic functions are meromorphic and doubly periodic in their first argument with periods  $4K(m)$  and  $4iK'(m)$ , where  $K$  is the complete elliptic integral of the first kind, implemented as `ellipticK`.

## Version History

**Introduced in R2017b**

### **See Also**

`jacobiAM` | `jacobiCD` | `jacobiCN` | `jacobiCS` | `jacobiDN` | `jacobiDS` | `jacobiNC` | `jacobiND` | `jacobiNS` | `jacobiSC` | `jacobiSD` | `jacobiSN` | `jacobiZeta` | `ellipticK`

## jacobiDN

Jacobi DN elliptic function

### Syntax

```
jacobiDN(u,m)
```

### Description

`jacobiDN(u,m)` returns the “Jacobi DN Elliptic Function” on page 6-888 of `u` and `m`. If `u` or `m` is an array, then `jacobiDN` acts element-wise.

### Examples

#### Calculate Jacobi DN Elliptic Function for Numeric Inputs

```
jacobiDN(2,1)
```

```
ans =  
    0.2658
```

Call `jacobiDN` on array inputs. `jacobiDN` acts element-wise when `u` or `m` is an array.

```
jacobiDN([2 1 -3],[1 2 3])
```

```
ans =  
    0.2658    0.3107   -0.0046
```

#### Calculate Jacobi DN Elliptic Function for Symbolic Numbers

Convert numeric input to symbolic form using `sym`, and find the Jacobi DN elliptic function. For symbolic input where `u = 0` or `m = 0` or `1`, `jacobiDN` returns exact symbolic output.

```
jacobiDN(sym(2),sym(1))
```

```
ans =  
1/cosh(2)
```

Show that for other values of `u` or `m`, `jacobiDN` returns an unevaluated function call.

```
jacobiDN(sym(2),sym(3))
```

```
ans =  
jacobiDN(2, 3)
```

#### Find Jacobi DN Elliptic Function for Symbolic Variables or Expressions

For symbolic variables or expressions, `jacobiDN` returns the unevaluated function call.

```
syms x y
f = jacobiDN(x,y)
```

```
f =
jacobiDN(x, y)
```

Substitute values for the variables by using `subs`, and convert values to double by using `double`.

```
f = subs(f, [x y], [3 5])
```

```
f =
jacobiDN(3, 5)
```

```
fVal = double(f)
```

```
fVal =
    0.9976
```

Calculate `f` to higher precision using `vpa`.

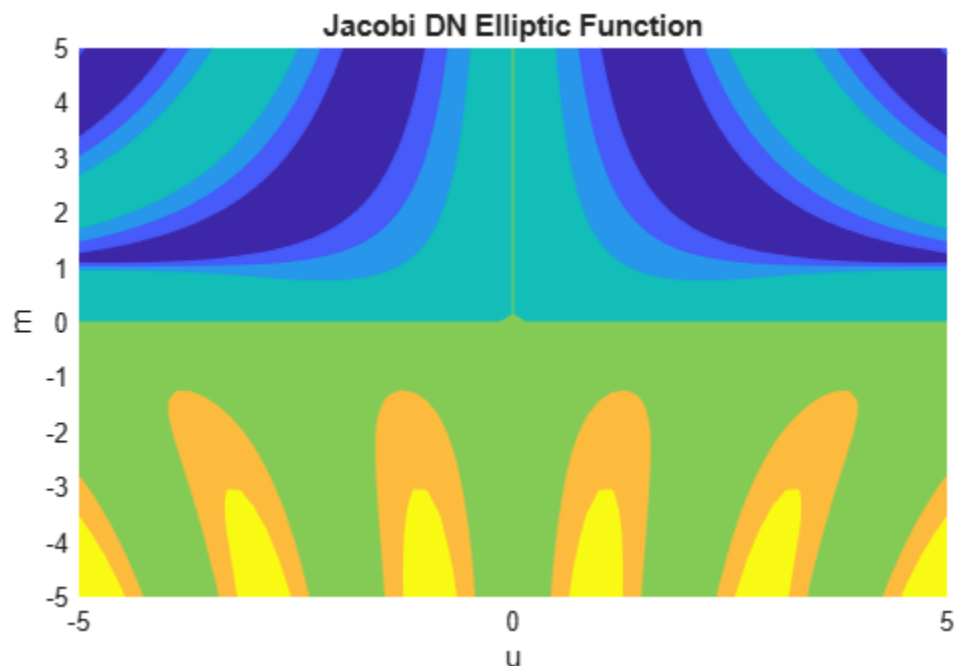
```
fVal = vpa(f)
```

```
fVal =
0.99757205953668099307853539907267
```

### Plot Jacobi DN Elliptic Function

Plot the Jacobi DN elliptic function using `fcontour`. Set `u` on the x-axis and `m` on the y-axis by using the symbolic function `f` with the variable order `(u,m)`. Fill plot contours by setting `Fill` to on.

```
syms f(u,m)
f(u,m) = jacobiDN(u,m);
fcontour(f, 'Fill', 'on')
title('Jacobi DN Elliptic Function')
xlabel('u')
ylabel('m')
```



## Input Arguments

### **u — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **m — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

## More About

### **Jacobi DN Elliptic Function**

The Jacobi DN elliptic function is

$$\operatorname{dn}(u, m) = \sqrt{1 - m \sin(\phi)^2}$$

where  $\phi$  is such that  $F(\phi, m) = u$  and  $F$  represents the incomplete elliptic integral of the first kind.  $F$  is implemented as `ellipticF`.

The Jacobi elliptic functions are meromorphic and doubly periodic in their first argument with periods  $4K(m)$  and  $4iK'(m)$ , where  $K$  is the complete elliptic integral of the first kind, implemented as `ellipticK`.

## Version History

Introduced in R2017b

### **See Also**

`jacobiAM` | `jacobiCD` | `jacobiCN` | `jacobiCS` | `jacobiDC` | `jacobiDS` | `jacobiNC` | `jacobiND` | `jacobiNS` | `jacobiSC` | `jacobiSD` | `jacobiSN` | `jacobiZeta` | `ellipticK`



# **jacobiDS**

Jacobi DS elliptic function

## **Syntax**

```
jacobiDS(u,m)
```

## **Description**

`jacobiDS(u,m)` returns the “Jacobi DS Elliptic Function” on page 6-891 of `u` and `m`. If `u` or `m` is an array, then `jacobiDS` acts element-wise.

## **Examples**

### **Calculate Jacobi DS Elliptic Function for Numeric Inputs**

```
jacobiDS(2,1)
```

```
ans =  
    0.2757
```

Call `jacobiDS` on array inputs. `jacobiDS` acts element-wise when `u` or `m` is an array.

```
jacobiDS([2 1 -3],[1 2 3])
```

```
ans =  
    0.2757    0.4623   -0.0079
```

### **Calculate Jacobi DS Elliptic Function for Symbolic Numbers**

Convert numeric input to symbolic form using `sym`, and find the Jacobi DS elliptic function. For symbolic input where `u = 0` or `m = 0` or `1`, `jacobiDS` returns exact symbolic output.

```
jacobiDS(sym(2),sym(1))
```

```
ans =  
1/sinh(2)
```

Show that for other values of `u` or `m`, `jacobiDS` returns an unevaluated function call.

```
jacobiDS(sym(2),sym(3))
```

```
ans =  
jacobiDS(2, 3)
```

### **Find Jacobi DS Elliptic Function for Symbolic Variables or Expressions**

For symbolic variables or expressions, `jacobiDS` returns the unevaluated function call.

```
syms x y
f = jacobiDS(x,y)

f =
jacobiDS(x, y)
```

Substitute values for the variables by using `subs`, and convert values to double by using `double`.

```
f = subs(f, [x y], [3 5])

f =
jacobiDS(3, 5)

fVal = double(f)

fVal =
    32.0302
```

Calculate `f` to higher precision using `vpa`.

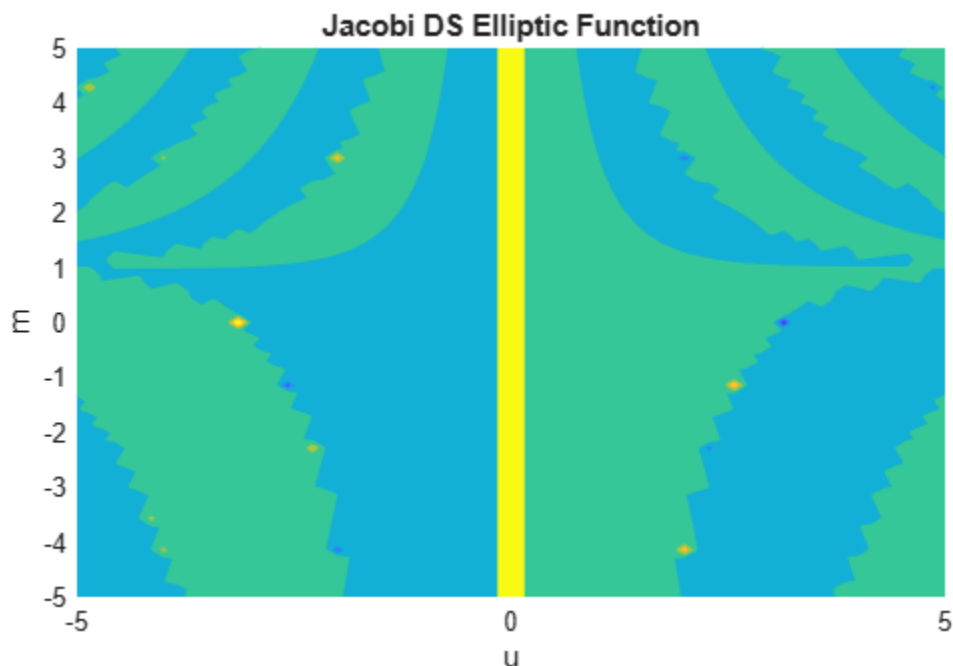
```
fVal = vpa(f)

fVal =
32.030154607596772037587224629884
```

### Plot Jacobi DS Elliptic Function

Plot the Jacobi DS elliptic function using `fcontour`. Set `u` on the x-axis and `m` on the y-axis by using the symbolic function `f` with the variable order `(u,m)`. Fill plot contours by setting `Fill` to `on`.

```
syms f(u,m)
f(u,m) = jacobiDS(u,m);
fcontour(f, 'Fill', 'on')
title('Jacobi DS Elliptic Function')
xlabel('u')
ylabel('m')
```



## Input Arguments

### **u — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **m — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

## More About

### **Jacobi DS Elliptic Function**

The Jacobi DS elliptic function is  

$$ds(u,m) = dn(u,m)/sn(u,m)$$

where dn and sn are the respective Jacobi elliptic functions.

The Jacobi elliptic functions are meromorphic and doubly periodic in their first argument with periods  $4K(m)$  and  $4iK'(m)$ , where  $K$  is the complete elliptic integral of the first kind, implemented as `ellipticK`.

## Version History

**Introduced in R2017b**

### **See Also**

`jacobiAM` | `jacobiCD` | `jacobiCN` | `jacobiCS` | `jacobiDC` | `jacobiDN` | `jacobiNC` | `jacobiND` | `jacobiNS` | `jacobiSC` | `jacobiSD` | `jacobiSN` | `jacobiZeta` | `ellipticK`

## jacobiNC

Jacobi NC elliptic function

### Syntax

```
jacobiNC(u,m)
```

### Description

`jacobiNC(u,m)` returns the “Jacobi NC Elliptic Function” on page 6-894 of `u` and `m`. If `u` or `m` is an array, then `jacobiNC` acts element-wise.

### Examples

#### Calculate Jacobi NC Elliptic Function for Numeric Inputs

```
jacobiNC(2,1)
```

```
ans =  
    3.7622
```

Call `jacobiNC` on array inputs. `jacobiNC` acts element-wise when `u` or `m` is an array.

```
jacobiNC([2 1 -3],[1 2 3])
```

```
ans =  
    3.7622    1.3505    1.2247
```

#### Calculate Jacobi NC Elliptic Function for Symbolic Numbers

Convert numeric input to symbolic form using `sym`, and find the Jacobi NC elliptic function. For symbolic input where `u = 0` or `m = 0` or `1`, `jacobiNC` returns exact symbolic output.

```
jacobiNC(sym(2),sym(1))
```

```
ans =  
cosh(2)
```

Show that for other values of `u` or `m`, `jacobiNC` returns an unevaluated function call.

```
jacobiNC(sym(2),sym(3))
```

```
ans =  
jacobiNC(2, 3)
```

#### Find Jacobi NC Elliptic Function for Symbolic Variables or Expressions

For symbolic variables or expressions, `jacobiNC` returns the unevaluated function call.

```
syms x y
f = jacobiNC(x,y)
```

```
f =
jacobiNC(x, y)
```

Substitute values for the variables by using `subs`, and convert values to double by using `double`.

```
f = subs(f, [x y], [3 5])
```

```
f =
jacobiNC(3, 5)
```

```
fVal = double(f)
```

```
fVal =
    1.0005
```

Calculate `f` to higher precision using `vpa`.

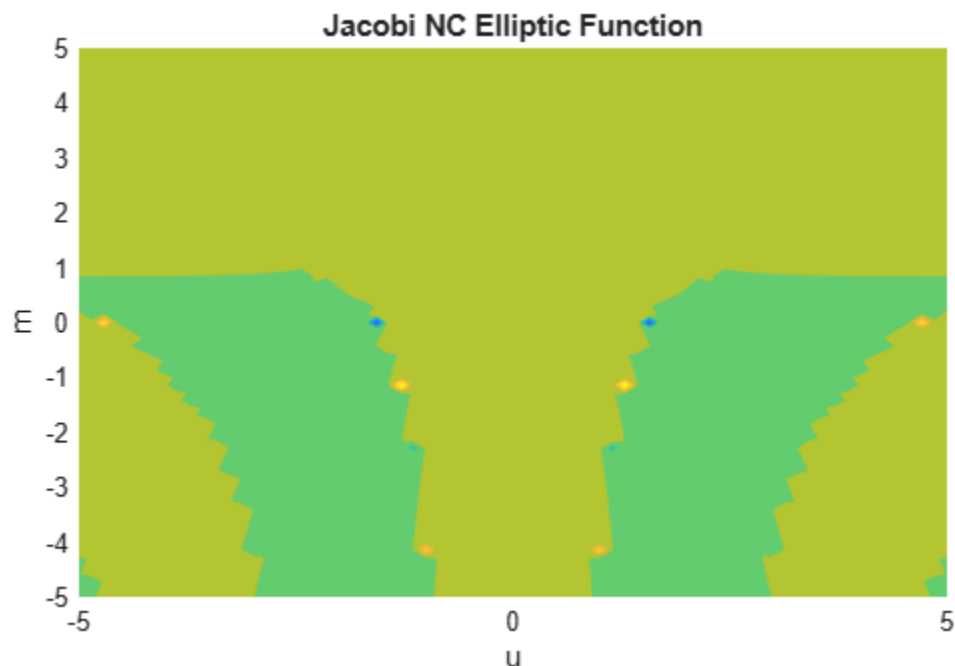
```
fVal = vpa(f)
```

```
fVal =
1.0004853517240922102007985618873
```

### Plot Jacobi NC Elliptic Function

Plot the Jacobi NC elliptic function using `fcontour`. Set `u` on the x-axis and `m` on the y-axis by using the symbolic function `f` with the variable order `(u,m)`. Fill plot contours by setting `Fill` to on.

```
syms f(u,m)
f(u,m) = jacobiNC(u,m);
fcontour(f, 'Fill', 'on')
title('Jacobi NC Elliptic Function')
xlabel('u')
ylabel('m')
```



## Input Arguments

### **u — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **m — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

## More About

### **Jacobi NC Elliptic Function**

The Jacobi NC elliptic function is  
$$nc(u,m) = 1/cn(u,m)$$

where cn is the respective Jacobi elliptic function.

The Jacobi elliptic functions are meromorphic and doubly periodic in their first argument with periods  $4K(m)$  and  $4iK'(m)$ , where  $K$  is the complete elliptic integral of the first kind, implemented as `ellipticK`.

## Version History

**Introduced in R2017b**

### **See Also**

`jacobiAM` | `jacobiCD` | `jacobiCN` | `jacobiCS` | `jacobiDC` | `jacobiDN` | `jacobiDS` | `jacobiND` | `jacobiNS` | `jacobiSC` | `jacobiSD` | `jacobiSN` | `jacobiZeta` | `ellipticK`

# **jacobiND**

Jacobi ND elliptic function

## **Syntax**

```
jacobiND(u,m)
```

## **Description**

`jacobiND(u,m)` returns the “Jacobi ND Elliptic Function” on page 6-897 of `u` and `m`. If `u` or `m` is an array, then `jacobiND` acts element-wise.

## **Examples**

### **Calculate Jacobi ND Elliptic Function for Numeric Inputs**

```
jacobiND(2,1)
```

```
ans =  
    3.7622
```

Call `jacobiND` on array inputs. `jacobiND` acts element-wise when `u` or `m` is an array.

```
jacobiND([2 1 -3],[1 2 3])
```

```
ans =  
    3.7622    3.2181 -218.7739
```

### **Calculate Jacobi ND Elliptic Function for Symbolic Numbers**

Convert numeric input to symbolic form using `sym`, and find the Jacobi ND elliptic function. For symbolic input where `u = 0` or `m = 0` or `1`, `jacobiND` returns exact symbolic output.

```
jacobiND(sym(2),sym(1))
```

```
ans =  
cosh(2)
```

Show that for other values of `u` or `m`, `jacobiND` returns an unevaluated function call.

```
jacobiND(sym(2),sym(3))
```

```
ans =  
jacobiND(2, 3)
```

### **Find Jacobi ND Elliptic Function for Symbolic Variables or Expressions**

For symbolic variables or expressions, `jacobiND` returns the unevaluated function call.

```
syms x y
f = jacobiND(x,y)

f =
jacobiND(x, y)
```

Substitute values for the variables by using `subs`, and convert values to double by using `double`.

```
f = subs(f, [x y], [3 5])

f =
jacobiND(3, 5)

fVal = double(f)

fVal =
    1.0024
```

Calculate `f` to higher precision using `vpa`.

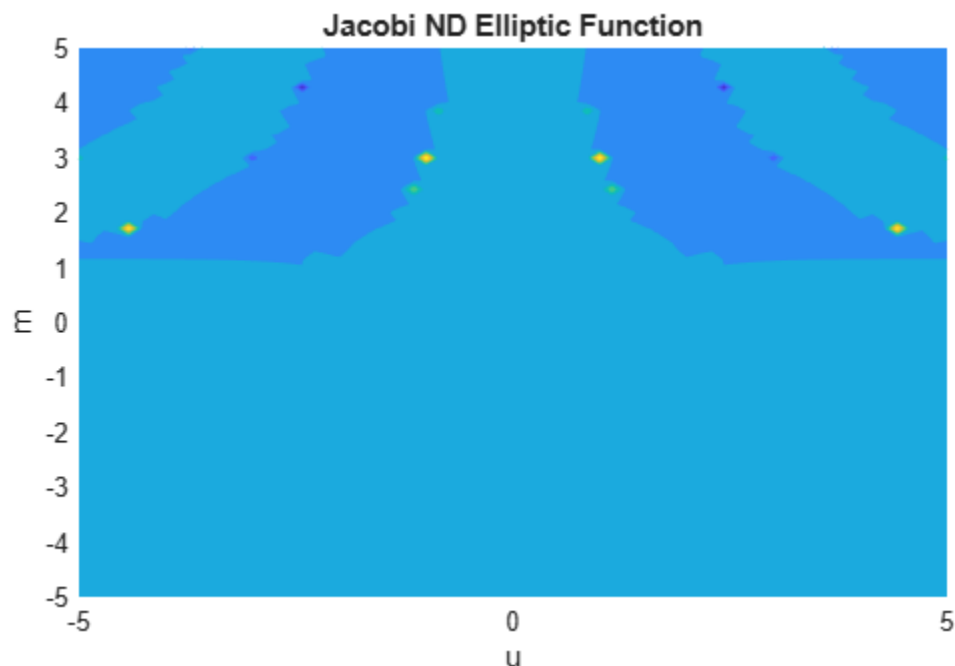
```
fVal = vpa(f)

fVal =
1.0024338497055006289470589737758
```

### Plot Jacobi ND Elliptic Function

Plot the Jacobi ND elliptic function using `fcontour`. Set `u` on the x-axis and `m` on the y-axis by using the symbolic function `f` with the variable order `(u,m)`. Fill plot contours by setting `Fill` to `on`.

```
syms f(u,m)
f(u,m) = jacobiND(u,m);
fcontour(f, 'Fill', 'on')
title('Jacobi ND Elliptic Function')
xlabel('u')
ylabel('m')
```





## Input Arguments

### **u — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **m — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

## More About

### **Jacobi ND Elliptic Function**

The Jacobi ND elliptic function is  
$$\text{nd}(u,m) = 1/\text{dn}(u,m)$$

where dn is the respective Jacobi elliptic function.

The Jacobi elliptic functions are meromorphic and doubly periodic in their first argument with periods  $4K(m)$  and  $4iK'(m)$ , where  $K$  is the complete elliptic integral of the first kind, implemented as `ellipticK`.

## Version History

**Introduced in R2017b**

### **See Also**

`jacobiAM` | `jacobiCD` | `jacobiCN` | `jacobiCS` | `jacobiDC` | `jacobiDN` | `jacobiDS` | `jacobiNC` | `jacobiNS` | `jacobiSC` | `jacobiSD` | `jacobiSN` | `jacobiZeta` | `ellipticK`

## **jacobiNS**

Jacobi NS elliptic function

### **Syntax**

```
jacobiNS(u,m)
```

### **Description**

`jacobiNS(u,m)` returns the “Jacobi NS Elliptic Function” on page 6-900 of `u` and `m`. If `u` or `m` is an array, then `jacobiNS` acts element-wise.

### **Examples**

#### **Calculate Jacobi NS Elliptic Function for Numeric Inputs**

```
jacobiNS(2,1)
```

```
ans =  
    1.0373
```

Call `jacobiNS` on array inputs. `jacobiNS` acts element-wise when `u` or `m` is an array.

```
jacobiNS([2 1 -3],[1 2 3])
```

```
ans =  
    1.0373    1.4879    1.7321
```

#### **Calculate Jacobi NS Elliptic Function for Symbolic Numbers**

Convert numeric input to symbolic form using `sym`, and find the Jacobi NS elliptic function. For symbolic input where `u = 0` or `m = 0` or `1`, `jacobiNS` returns exact symbolic output.

```
jacobiNS(sym(2),sym(1))
```

```
ans =  
coth(2)
```

Show that for other values of `u` or `m`, `jacobiNS` returns an unevaluated function call.

```
jacobiNS(sym(2),sym(3))
```

```
ans =  
jacobiNS(2, 3)
```

#### **Find Jacobi NS Elliptic Function for Symbolic Variables or Expressions**

For symbolic variables or expressions, `jacobiNS` returns the unevaluated function call.

```
syms x y
f = jacobiNS(x,y)
```

```
f =
jacobiNS(x, y)
```

Substitute values for the variables by using `subs`, and convert values to double by using `double`.

```
f = subs(f, [x y], [3 5])
```

```
f =
jacobiNS(3, 5)
```

```
fVal = double(f)
```

```
fVal =
    32.1081
```

Calculate `f` to higher precision using `vpa`.

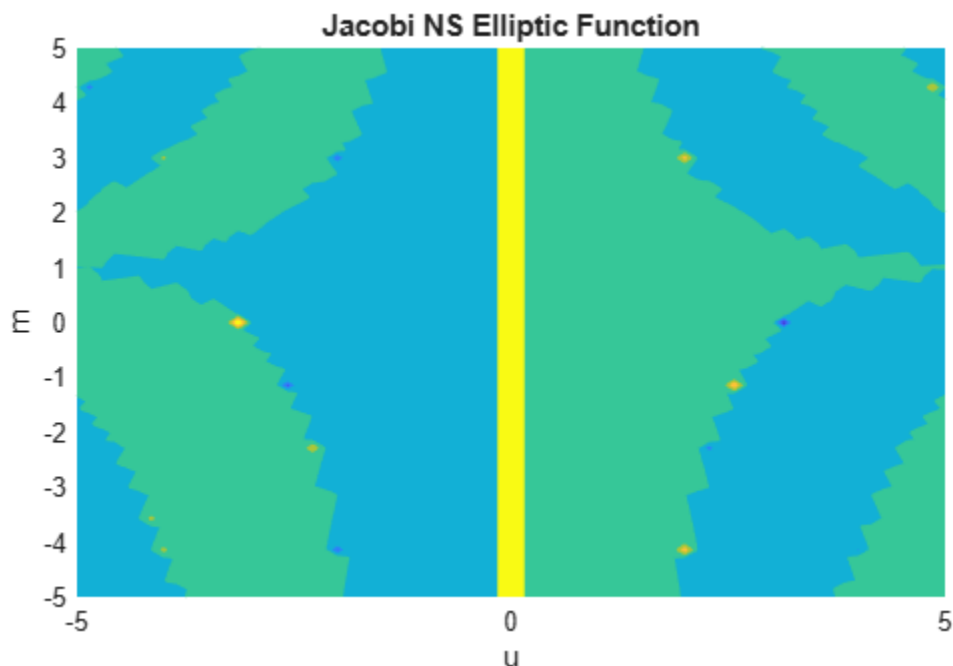
```
fVal = vpa(f)
```

```
fVal =
32.1081111189955611054545195854805
```

### Plot Jacobi NS Elliptic Function

Plot the Jacobi NS elliptic function using `fcontour`. Set `u` on the x-axis and `m` on the y-axis by using the symbolic function `f` with the variable order `(u,m)`. Fill plot contours by setting `Fill` to on.

```
syms f(u,m)
f(u,m) = jacobiNS(u,m);
fcontour(f, 'Fill', 'on')
title('Jacobi NS Elliptic Function')
xlabel('u')
ylabel('m')
```



## Input Arguments

### **u — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **m — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

## More About

### **Jacobi NS Elliptic Function**

The Jacobi NS elliptic function is  
$$ns(u,m) = 1/sn(u,m)$$

where sn is the respective Jacobi elliptic function.

The Jacobi elliptic functions are meromorphic and doubly periodic in their first argument with periods  $4K(m)$  and  $4iK'(m)$ , where  $K$  is the complete elliptic integral of the first kind, implemented as `ellipticK`.

## Version History

**Introduced in R2017b**

### **See Also**

`jacobiAM` | `jacobiCD` | `jacobiCN` | `jacobiCS` | `jacobiDC` | `jacobiDN` | `jacobiDS` | `jacobiNC` | `jacobiND` | `jacobiSC` | `jacobiSD` | `jacobiSN` | `jacobiZeta` | `ellipticK`

# **jacobiP**

Jacobi polynomials

## **Syntax**

```
jacobiP(n,a,b,x)
```

## **Description**

`jacobiP(n,a,b,x)` returns the  $n$ th degree Jacobi polynomial on page 6-904 with parameters  $a$  and  $b$  at  $x$ .

## **Examples**

### **Find Jacobi Polynomials for Numeric and Symbolic Inputs**

Find the Jacobi polynomial of degree 2 for numeric inputs.

```
jacobiP(2,0.5,-3,6)
```

```
ans =  
    7.3438
```

Find the Jacobi polynomial for symbolic inputs.

```
syms n a b x  
jacobiP(n,a,b,x)
```

```
ans =  
jacobiP(n, a, b, x)
```

If the degree of the Jacobi polynomial is not specified, `jacobiP` cannot find the polynomial and returns the function call.

Specify the degree of the Jacobi polynomial as 1 to return the form of the polynomial.

```
J = jacobiP(1,a,b,x)
```

```
J =  
a/2 - b/2 + x*(a/2 + b/2 + 1)
```

To find the numeric value of a Jacobi polynomial, call `jacobiP` with the numeric values directly. Do not substitute into the symbolic polynomial because the result can be inaccurate due to round-off. Test this by using `subs` to substitute into the symbolic polynomial, and compare the result with a numeric call.

```
J = jacobiP(300, -1/2, -1/2, x);  
subs(J,x,vpa(1/2))  
jacobiP(300, -1/2, -1/2, vpa(1/2))
```

```
ans =  
101573673381249394050.64541318209
```

```
ans =
0.032559931334979678350422392588404
```

When `subs` is used to substitute into the symbolic polynomial, the numeric result is subject to round-off error. The direct numerical call to `jacobiP` is accurate.

### Find Jacobi Polynomial with Vector and Matrix Inputs

Find the Jacobi polynomials of degrees 1 and 2 by setting  $n = [1 \ 2]$  for  $a = 3$  and  $b = 1$ .

```
syms x
jacobiP([1 2],3,1,x)

ans =
[ 3*x + 1, 7*x^2 + (7*x)/2 - 1/2]
```

`jacobiP` acts on  $n$  element-wise to return a vector with two entries.

If multiple inputs are specified as a vector, matrix, or multidimensional array, these inputs must be the same size. Find the Jacobi polynomials for  $a = [1 \ 2; 3 \ 1]$ ,  $b = [2 \ 2; 1 \ 3]$ ,  $n = 1$  and  $x$ .

```
a = [1 2; 3 1];
b = [2 2; 1 3];
J = jacobiP(1,a,b,x)

J =
[ (5*x)/2 - 1/2, 3*x]
[ 3*x + 1, 3*x - 1]
```

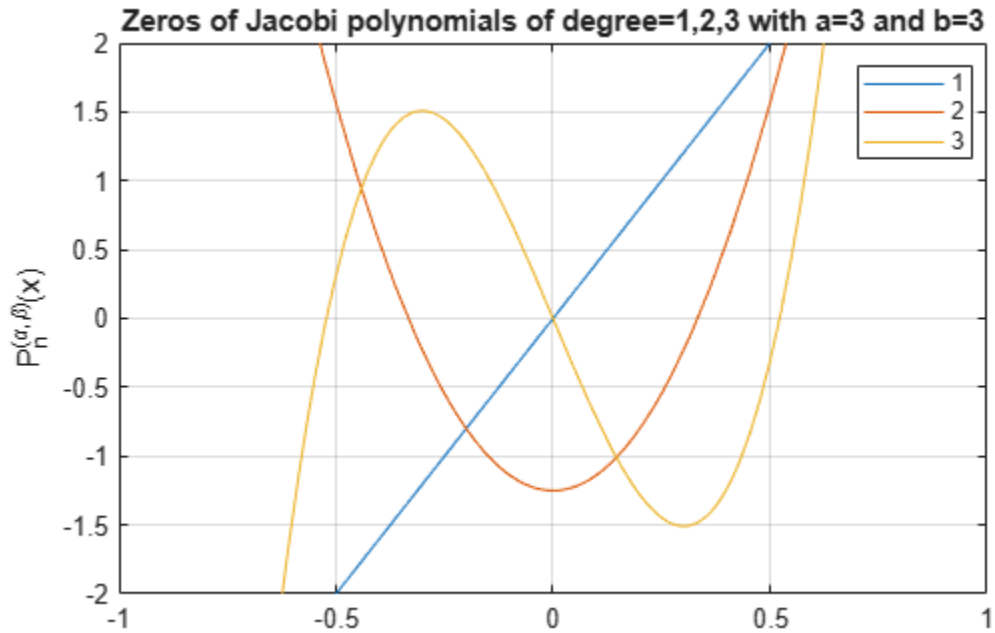
`jacobiP` acts element-wise on  $a$  and  $b$  to return a matrix of the same size as  $a$  and  $b$ .

### Visualize Zeros of Jacobi Polynomials

Plot Jacobi polynomials of degree 1, 2, and 3 for  $a = 3$ ,  $b = 3$ , and  $-1 < x < 1$ . To better view the plot, set axis limits by using `axis`.

```
syms x
fplot(jacobiP(1:3,3,3,x))
axis([-1 1 -2 2])
grid on

ylabel('P_n^{(\alpha,\beta)}(x)')
title('Zeros of Jacobi polynomials of degree=1,2,3 with a=3 and b=3');
legend('1','2','3','Location','best')
```



### Prove Orthogonality of Jacobi Polynomials with Respect to Weight Function

The Jacobi polynomials  $P(n, a, b, x)$  are orthogonal with respect to the weight function  $(1 - x)^a(1 + x)^b$  on the interval  $[-1, 1]$ .

Prove  $P(3, a, b, x)$  and  $P(5, a, b, x)$  are orthogonal with respect to the weight function  $(1 - x)^a(1 + x)^b$  by integrating their product over the interval  $[-1, 1]$ , where  $a = 3.5$  and  $b = 7.2$ .

```
syms x
a = 3.5;
b = 7.2;
P3 = jacobiP(3, a, b, x);
P5 = jacobiP(5, a, b, x);
w = (1-x)^a*(1+x)^b;
int(P3*P5*w, x, -1, 1)
```

```
ans =
0
```

### Input Arguments

#### **n** — Degree of Jacobi polynomial

nonnegative integer | vector of nonnegative integers | matrix of nonnegative integers | multidimensional array of nonnegative integers | symbolic nonnegative integer | symbolic variable | symbolic vector | symbolic matrix | symbolic function | symbolic expression | symbolic multidimensional array

Degree of Jacobi polynomial, specified as a nonnegative integer, or a vector, matrix, or multidimensional array of nonnegative integers, or a symbolic nonnegative integer, variable, vector, matrix, function, expression, or multidimensional array.

**a — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic vector | symbolic matrix | symbolic function | symbolic expression | symbolic multidimensional array

Input, specified as a number, vector, matrix, multidimensional array, or a symbolic number, vector, matrix, function, expression, or multidimensional array.

**b — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic vector | symbolic matrix | symbolic function | symbolic expression | symbolic multidimensional array

Input, specified as a number, vector, matrix, multidimensional array, or a symbolic number, vector, matrix, function, expression, or multidimensional array.

**x — Evaluation point**

number | vector | matrix | multidimensional array | symbolic number | symbolic vector | symbolic matrix | symbolic function | symbolic expression | symbolic multidimensional array

Evaluation point, specified as a number, vector, matrix, multidimensional array, or a symbolic number, vector, matrix, function, expression, or multidimensional array.

## More About

### Jacobi Polynomials

- The Jacobi polynomials are given by the recursion formula

$$2nc_n c_{2n-2} P(n, a, b, x) = c_{2n-1} (c_{2n-2} c_{2n} x + a^2 - b^2) P(n-1, a, b, x) - 2(n-1+a)(n-1+b) c_{2n} P(n-2, a, b, x),$$

where

$$c_n = n + a + b$$

$$P(0, a, b, x) = 1$$

$$P(1, a, b, x) = \frac{a-b}{2} + \left(1 + \frac{a+b}{2}\right)x.$$

- For fixed real  $a > -1$  and  $b > -1$ , the Jacobi polynomials are orthogonal on the interval  $[-1, 1]$  with respect to the weight function  $w(x) = (1-x)^a(1+x)^b$ .

$$\int_{-1}^1 P(n, a, b, x) P(m, a, b, x) (1-x)^a (1+x)^b dx = \begin{cases} 0 & \text{if } n \neq m \\ \frac{2^{a+b+1}}{2n+a+b+1} \frac{\Gamma(n+a+1)\Gamma(n+b+1)}{\Gamma(n+a+b+1)n!} & \text{if } n = m. \end{cases}$$

- For  $a = 0$  and  $b = 0$ , the Jacobi polynomials  $P(n, 0, 0, x)$  reduce to the Legendre polynomials  $P(n, x)$ .
- The relation between Jacobi polynomials  $P(n, a, b, x)$  and Chebyshev polynomials of the first kind  $T(n, x)$  is

$$T(n, x) = \frac{2^{2n}(n!)^2}{(2n)!} P\left(n, -\frac{1}{2}, -\frac{1}{2}, x\right).$$

- The relation between Jacobi polynomials  $P(n, a, b, x)$  and Chebyshev polynomials of the second kind  $U(n, x)$  is



$$U(n, x) = \frac{2^{2n} n! (n+1)!}{(2n+1)!} P\left(n, \frac{1}{2}, \frac{1}{2}, x\right).$$

- The relation between Jacobi polynomials  $P(n, a, b, x)$  and Gegenbauer polynomials  $G(n, a, x)$  is

$$G(n, a, x) = \frac{\Gamma\left(a + \frac{1}{2}\right) \Gamma(n + 2a)}{\Gamma(2a) \Gamma\left(n + a + \frac{1}{2}\right)} P\left(n, a - \frac{1}{2}, a - \frac{1}{2}, x\right).$$

## Version History

Introduced in R2014b

## See Also

chebyshevT | chebyshevU | gegenbauerC | hermiteH | hypergeom | laguerreL | legendreP

## jacobiSC

Jacobi SC elliptic function

### Syntax

```
jacobiSC(u,m)
```

### Description

`jacobiSC(u,m)` returns the “Jacobi SC Elliptic Function” on page 6-908 of `u` and `m`. If `u` or `m` is an array, then `jacobiSC` acts element-wise.

### Examples

#### Calculate Jacobi SC Elliptic Function for Numeric Inputs

```
jacobiSC(2,1)
```

```
ans =  
    3.6269
```

Call `jacobiSC` on array inputs. `jacobiSC` acts element-wise when `u` or `m` is an array.

```
jacobiSC([2 1 -3],[1 2 3])
```

```
ans =  
    3.6269    0.9077    0.7071
```

#### Calculate Jacobi SC Elliptic Function for Symbolic Numbers

Convert numeric input to symbolic form using `sym`, and find the Jacobi SC elliptic function. For symbolic input where `u = 0` or `m = 0` or `1`, `jacobiSC` returns exact symbolic output.

```
jacobiSC(sym(2),sym(1))
```

```
ans =  
sinh(2)
```

Show that for other values of `u` or `m`, `jacobiSC` returns an unevaluated function call.

```
jacobiSC(sym(2),sym(3))
```

```
ans =  
jacobiSC(2, 3)
```

#### Find Jacobi SC Elliptic Function for Symbolic Variables or Expressions

For symbolic variables or expressions, `jacobiSC` returns the unevaluated function call.

```
syms x y
f = jacobiSC(x,y)
```

```
f =
jacobiSC(x, y)
```

Substitute values for the variables by using `subs`, and convert values to double by using `double`.

```
f = subs(f, [x y], [3 5])
```

```
f =
jacobiSC(3, 5)
```

```
fVal = double(f)
```

```
fVal =
    0.0312
```

Calculate `f` to higher precision using `vpa`.

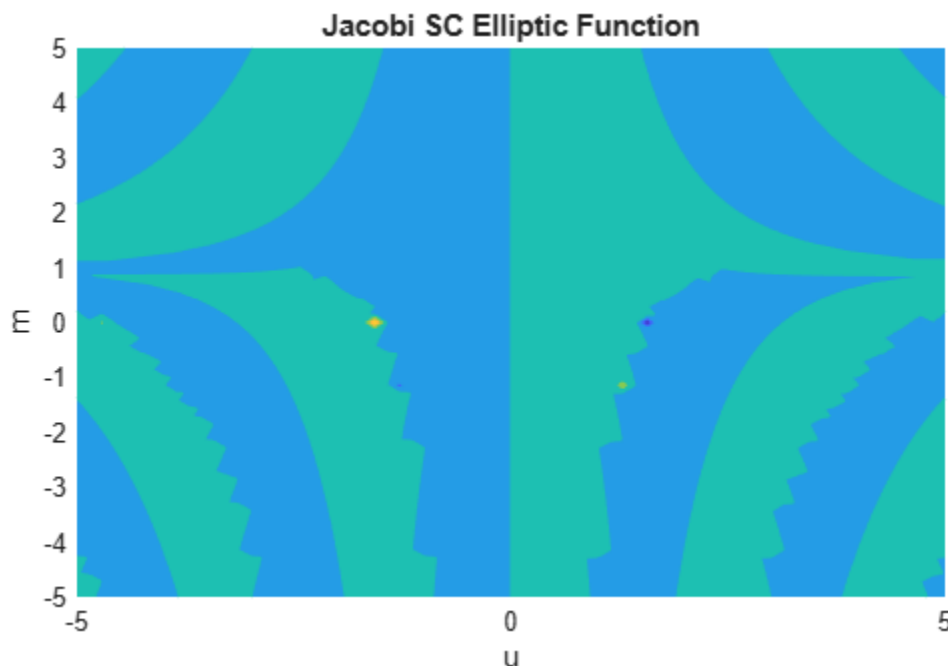
```
fVal = vpa(f)
```

```
fVal =
0.031159894327171581127518352857409
```

### Plot Jacobi SC Elliptic Function

Plot the Jacobi SC elliptic function using `fcontour`. Set `u` on the x-axis and `m` on the y-axis by using the symbolic function `f` with the variable order `(u,m)`. Fill plot contours by setting `Fill` to on.

```
syms f(u,m)
f(u,m) = jacobiSC(u,m);
fcontour(f, 'Fill', 'on')
title('Jacobi SC Elliptic Function')
xlabel('u')
ylabel('m')
```



## Input Arguments

### **u — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **m — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

## More About

### **Jacobi SC Elliptic Function**

The Jacobi SC elliptic function is

$$sc(u,m) = sn(u,m)/cn(u,m)$$

where sn and cn are the respective Jacobi elliptic functions.

The Jacobi elliptic functions are meromorphic and doubly periodic in their first argument with periods  $4K(m)$  and  $4iK'(m)$ , where  $K$  is the complete elliptic integral of the first kind, implemented as `ellipticK`.

## Version History

**Introduced in R2017b**

### **See Also**

`jacobiAM` | `jacobiCD` | `jacobiCN` | `jacobiCS` | `jacobiDC` | `jacobiDN` | `jacobiDS` | `jacobiNC` | `jacobiND` | `jacobiNS` | `jacobiSD` | `jacobiSN` | `jacobiZeta` | `ellipticK`

# **jacobiSD**

Jacobi SD elliptic function

## **Syntax**

```
jacobiSD(u,m)
```

## **Description**

`jacobiSD(u,m)` returns the “Jacobi SD Elliptic Function” on page 6-911 of `u` and `m`. If `u` or `m` is an array, then `jacobiSD` acts element-wise.

## **Examples**

### **Calculate Jacobi SD Elliptic Function for Numeric Inputs**

```
jacobiSD(2,1)
```

```
ans =  
    3.6269
```

Call `jacobiSD` on array inputs. `jacobiSD` acts element-wise when `u` or `m` is an array.

```
jacobiSD([2 1 -3],[1 2 3])
```

```
ans =  
    3.6269    2.1629 -126.3078
```

### **Calculate Jacobi SD Elliptic Function for Symbolic Numbers**

Convert numeric input to symbolic form using `sym`, and find the Jacobi SD elliptic function. For symbolic input where `u = 0` or `m = 0` or `1`, `jacobiSD` returns exact symbolic output.

```
jacobiSD(sym(2),sym(1))
```

```
ans =  
sinh(2)
```

Show that for other values of `u` or `m`, `jacobiSD` returns an unevaluated function call.

```
jacobiSD(sym(2),sym(3))
```

```
ans =  
jacobiSD(2, 3)
```

### **Find Jacobi SD Elliptic Function for Symbolic Variables or Expressions**

For symbolic variables or expressions, `jacobiSD` returns the unevaluated function call.

```
syms x y
f = jacobiSD(x,y)

f =
jacobiSD(x, y)
```

Substitute values for the variables by using `subs`, and convert values to double by using `double`.

```
f = subs(f, [x y], [3 5])

f =
jacobiSD(3, 5)

fVal = double(f)

fVal =
    0.0312
```

Calculate `f` to higher precision using `vpa`.

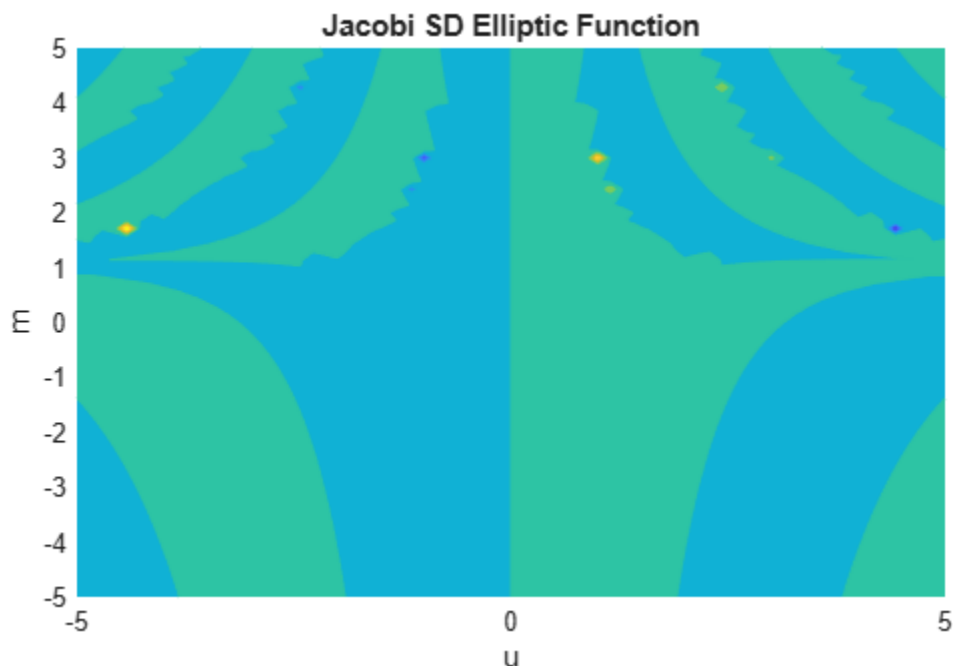
```
fVal = vpa(f)

fVal =
0.0312220579864538785956650143970485
```

### Plot Jacobi SD Elliptic Function

Plot the Jacobi SD elliptic function using `fcontour`. Set `u` on the x-axis and `m` on the y-axis by using the symbolic function `f` with the variable order `(u,m)`. Fill plot contours by setting `Fill` to on.

```
syms f(u,m)
f(u,m) = jacobiSD(u,m);
fcontour(f, 'Fill', 'on')
title('Jacobi SD Elliptic Function')
xlabel('u')
ylabel('m')
```



## Input Arguments

### **u — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **m — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

## More About

### **Jacobi SD Elliptic Function**

The Jacobi SD elliptic function is

$$sd(u,m) = sn(u,m)/dn(u,m)$$

where sn and dn are the respective Jacobi elliptic functions.

The Jacobi elliptic functions are meromorphic and doubly periodic in their first argument with periods  $4K(m)$  and  $4iK'(m)$ , where  $K$  is the complete elliptic integral of the first kind, implemented as `ellipticK`.

## Version History

**Introduced in R2017b**

### **See Also**

`jacobiAM` | `jacobiCD` | `jacobiCN` | `jacobiCS` | `jacobiDC` | `jacobiDN` | `jacobiDS` | `jacobiNC` | `jacobiND` | `jacobiNS` | `jacobiSC` | `jacobiSN` | `jacobiZeta` | `ellipticK`

## jacobiSN

Jacobi SN elliptic function

### Syntax

```
jacobiSN(u,m)
```

### Description

`jacobiSN(u,m)` returns the “Jacobi SN Elliptic Function” on page 6-914 of `u` and `m`. If `u` or `m` is an array, then `jacobiSN` acts element-wise.

### Examples

#### Calculate Jacobi SN Elliptic Function for Numeric Inputs

```
jacobiSN(2,1)
```

```
ans =  
    0.9640
```

Call `jacobiSN` on array inputs. `jacobiSN` acts element-wise when `u` or `m` is an array.

```
jacobiSN([2 1 -3],[1 2 3])
```

```
ans =  
    0.9640    0.6721    0.5773
```

#### Calculate Jacobi SN Elliptic Function for Symbolic Numbers

Convert numeric input to symbolic form using `sym`, and find the Jacobi SN elliptic function. For symbolic input where `u = 0` or `m = 0` or `1`, `jacobiSN` returns exact symbolic output.

```
jacobiSN(sym(2),sym(1))
```

```
ans =  
tanh(2)
```

Show that for other values of `u` or `m`, `jacobiSN` returns an unevaluated function call.

```
jacobiSN(sym(2),sym(3))
```

```
ans =  
jacobiSN(2, 3)
```

#### Find Jacobi SN Elliptic Function for Symbolic Variables or Expressions

For symbolic variables or expressions, `jacobiSN` returns the unevaluated function call.



```
syms x y
f = jacobiSN(x,y)
```

```
f =
jacobiSN(x, y)
```

Substitute values for the variables by using `subs`, and convert values to double by using `double`.

```
f = subs(f, [x y], [3 5])
```

```
f =
jacobiSN(3, 5)
```

```
fVal = double(f)
```

```
fVal =
    0.0311
```

Calculate `f` to higher precision using `vpa`.

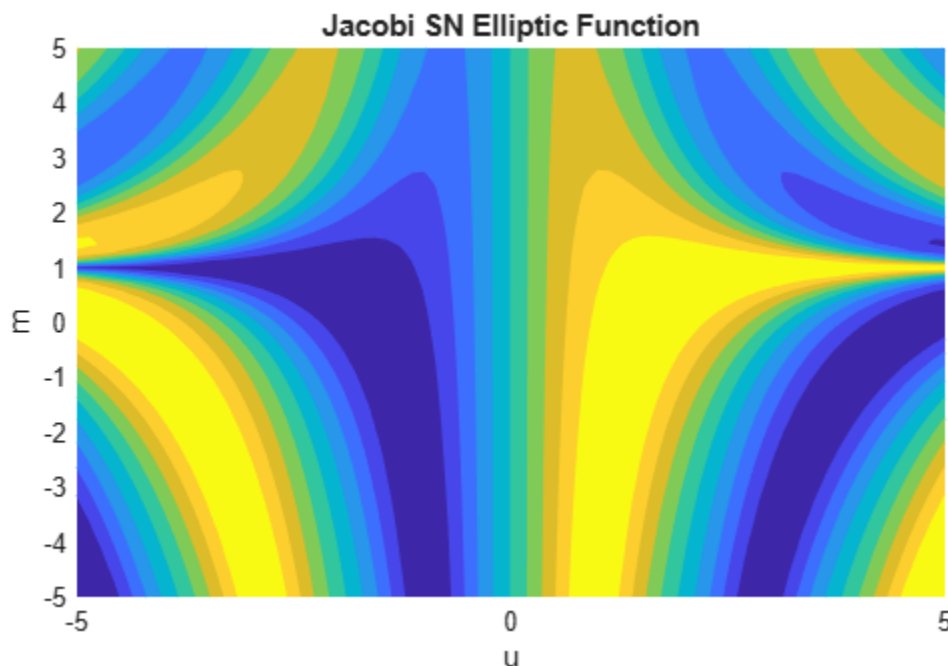
```
fVal = vpa(f)
```

```
fVal =
0.031144778155397389598324170696454
```

### Plot Jacobi SN Elliptic Function

Plot the Jacobi SN elliptic function using `fcontour`. Set `u` on the x-axis and `m` on the y-axis by using the symbolic function `f` with the variable order `(u,m)`. Fill plot contours by setting `Fill` to on.

```
syms f(u,m)
f(u,m) = jacobiSN(u,m);
fcontour(f, 'Fill', 'on')
title('Jacobi SN Elliptic Function')
xlabel('u')
ylabel('m')
```



## Input Arguments

### **u — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **m — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

## More About

### **Jacobi SN Elliptic Function**

The Jacobi SN elliptic function is  $\text{sn}(u,m) = \sin(\text{am}(u,m))$  where  $\text{am}$  is the Jacobi amplitude function.

The Jacobi elliptic functions are meromorphic and doubly periodic in their first argument with periods  $4K(m)$  and  $4iK'(m)$ , where  $K$  is the complete elliptic integral of the first kind, implemented as `ellipticK`.

## Version History

Introduced in R2017b

### **See Also**

`jacobiAM` | `jacobiCD` | `jacobiCN` | `jacobiCS` | `jacobiDC` | `jacobiDN` | `jacobiDS` | `jacobiNC` | `jacobiND` | `jacobiNS` | `jacobiSC` | `jacobiSD` | `jacobiZeta` | `ellipticK`

# jacobiSymbol

Jacobi symbol

## Syntax

`J = jacobiSymbol(a,n)`

## Description

`J = jacobiSymbol(a,n)` returns the value of the Jacobi symbol on page 6-918 for integer `a` and positive odd integer `n`.

## Examples

### Periodicity of Jacobi Symbol

Find the Jacobi symbol for  $a = 1, 2, \dots, 9$  and  $n = 3$ .

```
a = 1:9;
n = 3;
J = jacobiSymbol(a,n)
```

```
J = 1×9
```

```
     1     -1      0      1     -1      0      1     -1      0
```

The Jacobi symbol is periodic with respect to its first argument, where  $\left(\frac{a}{n}\right) = \left(\frac{b}{n}\right)$  if  $a \equiv b \pmod{n}$ .

### Completely Multiplicative Property of Jacobi Symbol

Find the Jacobi symbol for  $a = 28$  and  $n = 9$ .

```
J = jacobiSymbol(28,9)
```

```
J =
1
```

The Jacobi symbol is a completely multiplicative function, where the Jacobi symbol satisfies the relation  $\left(\frac{a}{n}\right) = \left(\frac{a_1}{n}\right) \times \left(\frac{a_2}{n}\right) \times \dots \times \left(\frac{a_r}{n}\right)$  for  $a = a_1 \times a_2 \times \dots \times a_r$ . Show that the Jacobi symbol follows this relation for  $a = 28 = 2 \times 2 \times 7$ .

```
Ja = jacobiSymbol(2,9)*jacobiSymbol(2,9)*jacobiSymbol(7,9)
```

```
Ja =
1
```

The Jacobi symbol also satisfies the relation  $\left(\frac{a}{n}\right) = \left(\frac{a}{n_1}\right) \times \left(\frac{a}{n_2}\right) \times \dots \left(\frac{a}{n_r}\right)$  for  $n = n_1 \times n_2 \times \dots n_r$ . Show that the Jacobi symbol follows this relation for  $n = 9 = 3 \times 3$ .

```
Jb = jacobiSymbol(28,3)*jacobiSymbol(28,3)
```

```
Jb =  
1
```

### Primality Test Using Jacobi Symbol

You can use the Jacobi symbol  $\left(\frac{a}{n}\right)$  for the Solovay-Strassen primality test. If an odd integer  $n > 1$  is prime, then the congruence

$$\left(\frac{a}{n}\right) \equiv a^{(n-1)/2} \pmod{n}$$

must hold for all integers  $a$ . If there is an integer  $a$  in  $\{1, 2, \dots, n-1\}$  such that the congruence relation is not satisfied, then  $n$  is not prime.

Check if  $n = 561$  is prime or not. Choose  $a = 19$  and perform the primality test. Compare the two values in the congruence relation.

First calculate the left side of the congruence relation using the Jacobi symbol.

```
n = 561;  
a = 14;  
J = jacobiSymbol(a,n)
```

```
J =  
1
```

Calculate the right side of the congruence relation.

```
m = powermod(a,(n-1)/2,n)
```

```
m =  
67
```

The integer  $n = 561$  is not prime since it does not satisfy the congruence relation for  $a = 19$ .

```
checkPrime = mod(J,n) == m
```

```
checkPrime = logical  
0
```

Next, check if  $n = 557$  is prime or not. Choose  $a = 19$  and perform the primality test.

```
n = 557;  
a = 19;  
J = jacobiSymbol(a,n);  
m = powermod(a,(n-1)/2,n);
```

The integer  $n = 557$  is probably prime since it satisfies the congruence relation for  $a = 19$ .

```
checkPrime = mod(J,n) == m
```

```
checkPrime = logical
1
```

Perform the primality test for all  $a$  in  $\{1, 2, \dots, 556\}$  to confirm that the integer  $n = 557$  is indeed prime.

```
a = 1:n-1;
J = jacobiSymbol(a,n);
m = powermod(a,(n-1)/2,n);
checkPrime = all(mod(J,n) == m)
```

```
checkPrime = logical
1
```

Verify the result with `isprime`.

```
isprime(n)
```

```
ans = logical
1
```

## Input Arguments

### **a** — Input

number | vector | matrix | array | symbolic number | symbolic array

Input, specified as a number, vector, matrix, array, symbolic number, or symbolic array. The elements of **a** must be integers. **a** and **n** must be the same size, or else one of them must be a scalar.

Data Types: single | double | sym

### **n** — Input

number | vector | matrix | array | symbolic number | symbolic array

Input, specified as a number, vector, matrix, array, symbolic number, or symbolic array. The elements of **n** must be positive odd integers. **a** and **n** must be the same size, or else one of them must be a scalar.

Data Types: single | double | sym

## Output Arguments

### **J** — Output value

-1 | 0 | 1

Output value, returned as -1, 0, or 1.

Data Types: single | double | sym

## More About

### Jacobi Symbol

The Jacobi symbol  $\left(\frac{a}{n}\right)$  is defined as the product of the Legendre symbols

$$\left(\frac{a}{n}\right) = \left(\frac{a}{p_1}\right)^{k_1} \left(\frac{a}{p_2}\right)^{k_2} \cdots \left(\frac{a}{p_r}\right)^{k_r}$$

for an integer  $a$  and a positive odd integer  $n$  with prime factorization

$$n = p_1^{k_1} p_2^{k_2} \cdots p_r^{k_r}.$$

The Legendre symbol  $\left(\frac{a}{p}\right)$  for an integer  $a$  and an odd prime  $p$  is defined as

$$\left(\frac{a}{p}\right) = \begin{cases} 0 & \text{if } a \equiv 0 \pmod{p} \\ 1 & \text{if } a \text{ is a quadratic residue modulo } p \text{ or } x^2 \equiv a \pmod{p} \text{ has a nonzero solution for } x \\ -1 & \text{if } a \text{ is a quadratic nonresidue modulo } p \text{ or } x^2 \equiv a \pmod{p} \text{ has no solutions for } x. \end{cases}$$

When  $n$  is an odd prime, the Jacobi symbol is equal to the Legendre symbol.

## Version History

Introduced in R2020a

### References

- [1] Dietzfelbinger, M. *Primality Testing in Polynomial Time: From Randomized Algorithms to "PRIMES Is in P"*. Springer, 2004.

### See Also

`mod` | `powermod` | `isprime`

# **jacobiZeta**

Jacobi zeta function

## **Syntax**

```
jacobiZeta(u,m)
```

## **Description**

`jacobiZeta(u,m)` returns the “Jacobi Zeta Function” on page 6-921 of `u` and `m`. If `u` or `m` is an array, then `jacobiZeta` acts element-wise.

## **Examples**

### **Calculate Jacobi Zeta Function for Numeric Inputs**

```
jacobiZeta(2,1)
```

```
ans =  
    0.9640
```

Call `jacobiZeta` on array inputs. `jacobiZeta` acts element-wise when `u` or `m` is an array.

```
jacobiZeta([2 1 -3],[1 2 3])
```

```
ans =  
    0.9640 + 0.0000i    0.5890 - 0.4569i   -2.3239 + 1.9847i
```

### **Calculate Jacobi Zeta Function for Symbolic Numbers**

Convert numeric input to symbolic form using `sym`, and find the Jacobi zeta function. For symbolic input where `u = 0` or `m = 0` or `1`, `jacobiZeta` returns exact symbolic output.

```
jacobiZeta(sym(2),sym(1))
```

```
ans =  
tanh(2)
```

Show that for other values of `u` or `m`, `jacobiZeta` returns an unevaluated function call.

```
jacobiZeta(sym(2),sym(3))
```

```
ans =  
jacobiZeta(2, 3)
```

### **Find Jacobi Zeta Function for Symbolic Variables or Expressions**

For symbolic variables or expressions, `jacobiZeta` returns the unevaluated function call.

```
syms x y
f = jacobiZeta(x,y)
```

```
f =
jacobiZeta(x, y)
```

Substitute values for the variables by using `subs`, and convert values to double by using `double`.

```
f = subs(f, [x y], [3 5])
```

```
f =
jacobiZeta(3, 5)
```

```
fVal = double(f)
```

```
fVal =
    4.0986 - 3.0018i
```

Calculate `f` to arbitrary precision using `vpa`.

```
fVal = vpa(f)
```

```
fVal =
4.0986033838332279126523721581432 - 3.0017792319714320747021938869936i
```

### Plot Jacobi Zeta Function

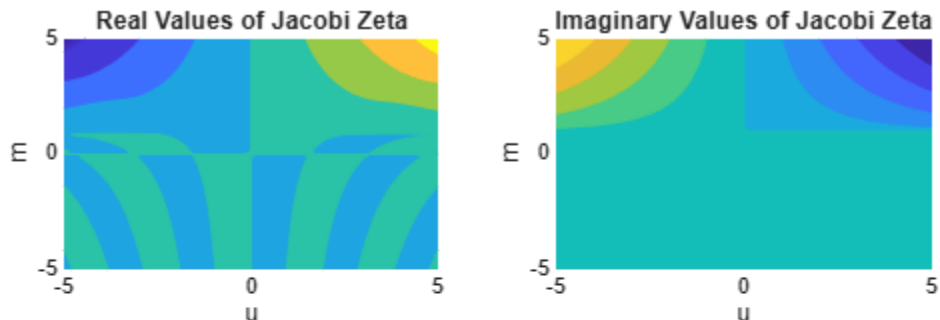
Plot real and imaginary values of the Jacobi zeta function using `fcontour`. Set `u` on the x-axis and `m` on the y-axis by using the symbolic function `f` with the variable order `(u,m)`. Fill plot contours by setting `Fill` to `on`.

```
syms f(u,m)
f(u,m) = jacobiZeta(u,m);
```

```
subplot(2,2,1)
fcontour(real(f), 'Fill', 'on')
title('Real Values of Jacobi Zeta')
xlabel('u')
ylabel('m')
```

```
subplot(2,2,2)
fcontour(imag(f), 'Fill', 'on')
title('Imaginary Values of Jacobi Zeta')
xlabel('u')
ylabel('m')
```





## Input Arguments

### **u — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **m — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

## More About

### **Jacobi Zeta Function**

The Jacobi zeta function `jacobiZeta(u,m)` is defined as

$$Z(u, m) = E(\varphi, m) - \frac{E(m)}{K(m)} F(\varphi, m).$$

The definitions of the terms in above equation are:

- $E(\varphi | m)$  and  $E(m)$  are the incomplete and complete elliptic integrals of the second kind, respectively, implemented as `ellipticE`.

- $K(m)$  is the complete elliptic integral of the first kind, implemented as `ellipticK`.
- $F(\varphi \mid m)$  is the incomplete elliptic integral of the first kind, implemented as `ellipticF`.
- $\operatorname{am}(u, m)$  is the Jacobi's amplitude function, implemented as `jacobiAM`.

The argument  $u$  is related to  $\varphi$  by the relations  $u = F(\varphi \mid m)$  and  $\operatorname{am}(u, m) = \varphi$ , where  $\operatorname{am}(u, m)$  is the Jacobi's amplitude function.

## Version History

Introduced in R2017b

## References

- [1] Olver, F. W. J., A. B. Olde Daalhuis, D. W. Lozier, B. I. Schneider, R. F. Boisvert, C. W. Clark, B. R. Miller, B. V. Saunders, H. S. Cohl, and M. A. McClain, eds., Chapter 22. Jacobian Elliptic Functions, *NIST Digital Library of Mathematical Functions*, Release 1.0.26 of 2020-03-15.

## See Also

`jacobiAM` | `jacobiCD` | `jacobiCN` | `jacobiCS` | `jacobiDC` | `jacobiDN` | `jacobiDS` | `jacobiNC` | `jacobiND` | `jacobiNS` | `jacobiSC` | `jacobiSD` | `jacobiSN`

# jordan

Jordan normal form (Jordan canonical form)

## Syntax

```
J = jordan(A)
[V,J] = jordan(A)
```

## Description

`J = jordan(A)` computes the Jordan normal form of the matrix `A`. Because the Jordan form of a numeric matrix is sensitive to numerical errors, prefer converting numeric input to exact symbolic form.

`[V,J] = jordan(A)` computes the Jordan form `J` and the similarity transform `V`. The matrix `V` contains the generalized eigenvectors of `A` as columns, such that  $V \backslash A * V = J$ .

## Examples

### Compute Jordan Form and Similarity Transform

Compute the Jordan form and the similarity transform for a matrix. Because the Jordan form of a numeric matrix is sensitive to numerical errors, first convert the matrix to symbolic form by using `sym`.

```
A = [1 -3 -2;
     -1 1 -1;
      2 4 5];
A = sym(A);
[V,J] = jordan(A)
```

```
V =
    (-1 1 -1)
    (-1 0 0)
    (2 0 1)
```

```
J =
    (2 1 0)
    (0 2 0)
    (0 0 3)
```

Verify that `V` satisfies the condition  $V \backslash A * V = J$  by using `isAlways`.

```
cond = J == V \ A * V;
tf = isAlways(cond)
```

```
tf = 3×3 logical array
```

```
1 1 1
1 1 1
```

1 1 1

## **Version History**

Introduced before R2006a

## **See Also**

`charpoly` | `eig` | `hermiteForm` | `inv` | `smithForm`

# kroneckerDelta

Kronecker delta function

## Syntax

```
kroneckerDelta(m)
kroneckerDelta(m,n)
```

## Description

`kroneckerDelta(m)` returns 1 if  $m == 0$  and 0 if  $m \neq 0$ .

`kroneckerDelta(m,n)` returns 1 if  $m == n$  and 0 if  $m \neq n$ .

## Examples

### Compare Two Symbolic Variables

---

**Note** For `kroneckerDelta` with numeric inputs, use the `eq` function instead.

---

Set symbolic variable `m` equal to symbolic variable `n` and test their equality using `kroneckerDelta`.

```
syms m n
m = n;
kroneckerDelta(m,n)
```

```
ans =
1
```

`kroneckerDelta` returns 1 indicating that the inputs are equal.

Compare symbolic variables `p` and `q`.

```
syms p q
kroneckerDelta(p,q)
```

```
ans =
kroneckerDelta(p - q, 0)
```

`kroneckerDelta` cannot decide if  $p == q$  and returns the function call with the undecidable input. Note that `kroneckerDelta(p, q)` is equal to `kroneckerDelta(p - q, 0)`.

To force a logical result for undecidable inputs, use `isAlways`. The `isAlways` function issues a warning and returns logical 0 (false) for undecidable inputs. Set the `Unknown` option to `false` to suppress the warning.

```
isAlways(kroneckerDelta(p, q), 'Unknown', 'false')
```

```
ans =
logical
0
```

### Compare Symbolic Variable with Zero

Set symbolic variable  $m$  to 0 and test  $m$  for equality with 0. The `kroneckerDelta` function errors because it does not accept numeric inputs of type double.

```
m = 0;
kroneckerDelta(m)
```

Undefined function 'kroneckerDelta' for input arguments of type 'double'.

Use `sym` to convert 0 to a symbolic object before assigning it to  $m$ . This is because `kroneckerDelta` only accepts symbolic inputs.

```
syms m
m = sym(0);
kroneckerDelta(m)
```

```
ans =
     1
```

`kroneckerDelta` returns 1 indicating that  $m$  is equal to 0. Note that `kroneckerDelta(m)` is equal to `kroneckerDelta(m, 0)`.

### Compare Vector of Numbers with Symbolic Variable

Compare a vector of numbers [1 2 3 4] with symbolic variable  $m$ . Set  $m$  to 3.

```
V = 1:4
syms m
m = sym(3)
sol = kroneckerDelta(V,m)
```

```
V =
     1     2     3     4
m =
     3
sol =
[ 0, 0, 1, 0]
```

`kroneckerDelta` acts on  $V$  element-wise to return a vector, `sol`, which is the same size as  $V$ . The third element of `sol` is 1 indicating that the third element of  $V$  equals  $m$ .

### Compare Two Matrices

Compare matrices  $A$  and  $B$ .

Declare matrices  $A$  and  $B$ .

```
syms m
A = [m m+1 m+2;m-2 m-1 m]
B = [m m+3 m+2;m-1 m-1 m+1]
```

```
A =
[ m, m + 1, m + 2]
[ m - 2, m - 1, m]
B =
[ m, m + 3, m + 2]
[ m - 1, m - 1, m + 1]
```

Compare A and B using `kroneckerDelta`.

```
sol = kroneckerDelta(A,B)
```

```
sol =  
[ 1, 0, 1]  
[ 0, 1, 0]
```

`kroneckerDelta` acts on A and B element-wise to return the matrix `sol` which is the same size as A and B. The elements of `sol` that are 1 indicate that the corresponding elements of A and B are equal. The elements of `sol` that are 0 indicate that the corresponding elements of A and B are not equal.

### Use `kroneckerDelta` in Inputs to Other Functions

`kroneckerDelta` appears in the output of `iztrans`.

```
syms z n  
sol = iztrans(1/(z-1), z, n)
```

```
sol =  
1 - kroneckerDelta(n, 0)
```

Use this output as input to `ztrans` to return the initial input expression.

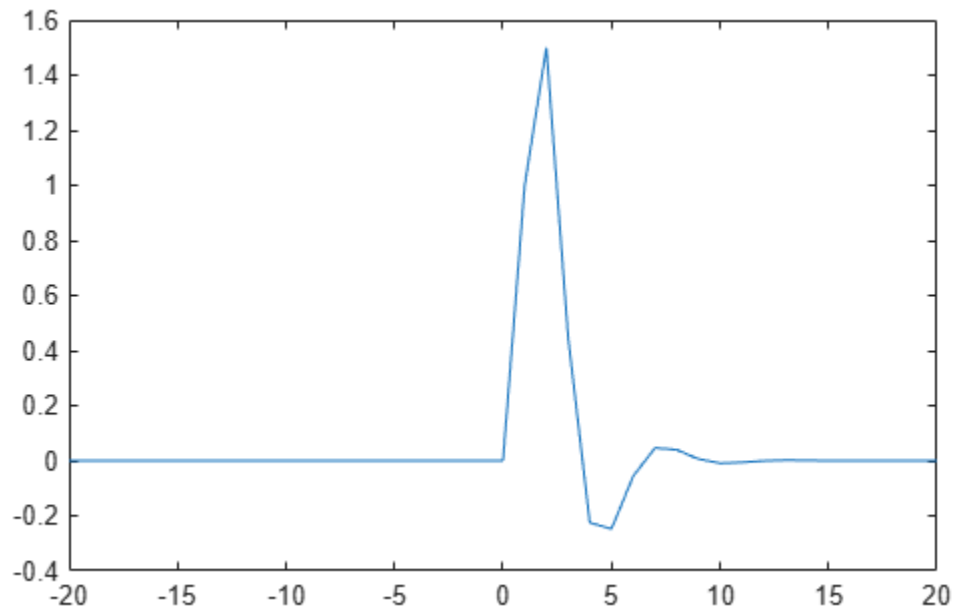
```
ztrans(sol, n, z)
```

```
ans =  
z/(z - 1) - 1
```

### Filter Response to Kronecker Delta Input

Use `filter` to find the response of a filter when the input is the Kronecker Delta function. Convert `k` to a symbolic vector using `sym` because `kroneckerDelta` only accepts symbolic inputs, and convert it back to double using `double`. Provide arbitrary filter coefficients `a` and `b` for simplicity.

```
b = [0 1 1];  
a = [1 -0.5 0.3];  
k = -20:20;  
x = double(kroneckerDelta(sym(k)));  
y = filter(b,a,x);  
plot(k,y)
```



## Input Arguments

### **m** — Input

number | vector | matrix | multidimensional array | symbolic number | symbolic vector | symbolic matrix | symbolic function | symbolic multidimensional array

Input, specified as a number, vector, matrix, multidimensional array, or a symbolic number, vector, matrix, function, or multidimensional array. At least one of the inputs, *m* or *n*, must be symbolic.

### **n** — Input

number | vector | matrix | multidimensional array | symbolic number | symbolic vector | symbolic matrix | symbolic function | symbolic multidimensional array

Input, specified as a number, vector, matrix, multidimensional array, or a symbolic number, vector, matrix, function, or multidimensional array. At least one of the inputs, *m* or *n*, must be symbolic.

## More About

### Kronecker Delta Function

The Kronecker delta function is defined as

$$\delta(m, n) = \begin{cases} 0 & \text{if } m \neq n \\ 1 & \text{if } m = n \end{cases}$$

## Tips

- When *m* or *n* is NaN, the `kroneckerDelta` function returns NaN.



## Version History

Introduced in R2014b

## See Also

iztrans | ztrans

## kummerU

Confluent hypergeometric Kummer U function

### Syntax

`kummerU(a, b, z)`

### Description

`kummerU(a, b, z)` computes the value of confluent hypergeometric function,  $U(a, b, z)$ . If the real parts of  $z$  and  $a$  are positive values, then the integral representations of the Kummer U function is as follows:

$$U(a, b, z) = \frac{1}{\Gamma(a)} \int_0^{\infty} e^{-zt} t^{a-1} (1+t)^{b-a-1} dt$$

### Examples

#### Equation Returning the Kummer U Function as Its Solution

`dsolve` can return solutions of second-order ordinary differential equations in terms of the Kummer U function.

Solve this equation. The solver returns the results in terms of the Kummer U function and another hypergeometric function.

```
syms t z y(z)
dsolve(z^3*diff(y,2) + (z^2 + t)*diff(y) + z*y)

ans =
(C4*hypergeom(1i/2, 1 + 1i, t/(2*z^2)))/z^1i +...
(C3*kummerU(1i/2, 1 + 1i, t/(2*z^2)))/z^1i
```

#### Kummer U Function for Numeric and Symbolic Arguments

Depending on its arguments, `kummerU` can return floating-point or exact symbolic results.

Compute the Kummer U function for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
A = [kummerU(-1/3, 2.5, 2)
     kummerU(1/3, 2, pi)
     kummerU(1/2, 1/3, 3*i)]
```

```
A =
    0.8234 + 0.0000i
    0.7284 + 0.0000i
    0.4434 - 0.3204i
```

Compute the Kummer U function for the numbers converted to symbolic objects. For most symbolic (exact) numbers, `kummerU` returns unresolved symbolic calls.

```
symA = [kummerU(-1/3, 2.5, sym(2))
kummerU(1/3, 2, sym(pi))
kummerU(1/2, sym(1/3), 3*i)]
```

```
symA =
    kummerU(-1/3, 5/2, 2)
    kummerU(1/3, 2, pi)
    kummerU(1/2, 1/3, 3i)
```

Use `vpa` to approximate symbolic results with the required number of digits.

```
vpa(symA,10)
```

```
ans =
    0.8233667846
    0.7284037305
    0.4434362538 - 0.3204327531i
```

### Some Special Values of Kummer U

The Kummer U function has special values for some parameters.

If  $a$  is a negative integer, the Kummer U function reduces to a polynomial.

```
syms a b z
[kummerU(-1, b, z)
kummerU(-2, b, z)
kummerU(-3, b, z)]
```

```
ans =
    6*z*(b^2/2 + (3*b)/2 + 1) - 2*b - 6*z^2*(b/2 + 1) - 3*b^2 - b^3 + z^3
```

If  $b = 2*a$ , the Kummer U function reduces to an expression involving the modified Bessel function of the second kind.

```
kummerU(a, 2*a, z)
```

```
ans =
(z^(1/2 - a)*exp(z/2)*besselk(a - 1/2, z/2))/pi^(1/2)
```

If  $a = 1$  or  $a = b$ , the Kummer U function reduces to an expression involving the incomplete gamma function.

```
kummerU(1, b, z)
```

```
ans =
z^(1 - b)*exp(z)*igamma(b - 1, z)
```

```
kummerU(a, a, z)
```

```
ans =
exp(z)*igamma(1 - a, z)
```

If  $a = 0$ , the Kummer U function is 1.

```
kummerU(0, a, z)
```

```
ans =
1
```

### Handle Expressions Containing the Kummer U Function

Many functions, such as `diff`, `int`, and `limit`, can handle expressions containing `kummerU`.

Find the first derivative of the Kummer U function with respect to  $z$ .

```
syms a b z
diff(kummerU(a, b, z), z)

ans =
(a*kummerU(a + 1, b, z)*(a - b + 1))/z - (a*kummerU(a, b, z))/z
```

Find the indefinite integral of the Kummer U function with respect to  $z$ .

```
int(kummerU(a, b, z), z)

ans =
((b - 2)/(a - 1) - 1)*kummerU(a, b, z) + ...
(kummerU(a + 1, b, z)*(a - a*b + a^2))/(a - 1) - ...
(z*kummerU(a, b, z))/(a - 1)
```

Find the limit of this Kummer U function.

```
limit(kummerU(1/2, -1, z), z, 0)

ans =
4/(3*pi^(1/2))
```

## Input Arguments

### **a** — Parameter of Kummer U function

number | vector | symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector

Parameter of Kummer U function, specified as a number, variable, symbolic expression, symbolic function, or vector.

### **b** — Parameter of Kummer U function

number | vector | symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector

Parameter of Kummer U function, specified as a number, variable, symbolic expression, symbolic function, or vector.

### **z** — Argument of Kummer U function

number | vector | symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector

Argument of Kummer U function, specified as a number, variable, symbolic expression, symbolic function, or vector. If  $z$  is a vector, `kummerU(a,b,z)` is evaluated element-wise.

## More About

### Confluent Hypergeometric Function (Kummer U Function)

The confluent hypergeometric function (Kummer U function) is one of the solutions of the differential equation

$$z \frac{\partial^2}{\partial z^2} y + (b - z) \frac{\partial}{\partial z} y - ay = 0$$

The other solution is the hypergeometric function  ${}_1F_1(a, b, z)$ .

The Whittaker W function can be expressed in terms of the Kummer U function:

$$W_{a,b}(z) = e^{-z/2} z^{b+1/2} U\left(b - a + \frac{1}{2}, 2b + 1, z\right)$$

## Tips

- kummerU returns floating-point results for numeric arguments that are not symbolic objects.
- kummerU acts element-wise on nonscalar inputs.
- All nonscalar arguments must have the same size. If one or two input arguments are nonscalar, then kummerU expands the scalars into vectors or matrices of the same size as the nonscalar arguments, with all elements equal to the corresponding scalar.

## Version History

Introduced in R2014b

## References

- [1] Slater, L. J. "Confluent Hypergeometric Functions." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

hypergeom | whittakerM | whittakerW

# laguerreL

Generalized Laguerre Function and Laguerre Polynomials

## Syntax

```
laguerreL(n,x)
laguerreL(n,a,x)
```

## Description

`laguerreL(n,x)` returns the Laguerre polynomial of degree  $n$  if  $n$  is a nonnegative integer. When  $n$  is not a nonnegative integer, `laguerreL` returns the Laguerre function. For details, see “Generalized Laguerre Function” on page 6-937.

`laguerreL(n,a,x)` returns the generalized Laguerre polynomial of degree  $n$  if  $n$  is a nonnegative integer. When  $n$  is not a nonnegative integer, `laguerreL` returns the generalized Laguerre function.

## Examples

### Find Laguerre Polynomials for Numeric and Symbolic Inputs

Find the Laguerre polynomial of degree 3 for input 4.3.

```
laguerreL(3,4.3)
```

```
ans =
    2.5838
```

Find the Laguerre polynomial for symbolic inputs. Specify degree  $n$  as 3 to return the explicit form of the polynomial.

```
syms x
laguerreL(3,x)
```

```
ans =
- x^3/6 + (3*x^2)/2 - 3*x + 1
```

If the degree of the Laguerre polynomial  $n$  is not specified, `laguerreL` cannot find the polynomial. When `laguerreL` cannot find the polynomial, it returns the function call.

```
syms n x
laguerreL(n,x)
```

```
ans =
laguerreL(n, x)
```

### Find Generalized Laguerre Polynomial

Find the explicit form of the generalized Laguerre polynomial  $L(n,a,x)$  of degree  $n = 2$ .

```
syms a x
laguerreL(2,a,x)
```

```
ans =
(3*a)/2 - x*(a + 2) + a^2/2 + x^2/2 + 1
```

### Return Generalized Laguerre Function

When  $n$  is not a nonnegative integer, `laguerreL(n,a,x)` returns the generalized Laguerre function.

```
laguerreL(-2.7,3,2)
```

```
ans =
0.2488
```

`laguerreL` is not defined for certain inputs and returns an error.

```
syms x
laguerreL(-5/2, -3/2, x)
```

```
Error using symengine
Function 'laguerreL' not supported for parameter values '-5/2' and '-3/2'.
```

### Find Laguerre Polynomial with Vector and Matrix Inputs

Find the Laguerre polynomials of degrees 1 and 2 by setting  $n = [1 \ 2]$ .

```
syms x
laguerreL([1 2],x)

ans =
[ 1 - x, x^2/2 - 2*x + 1]
```

`laguerreL` acts element-wise on  $n$  to return a vector with two elements.

If multiple inputs are specified as a vector, matrix, or multidimensional array, the inputs must be the same size. Find the generalized Laguerre polynomials where input arguments  $n$  and  $x$  are matrices.

```
syms a
n = [2 3; 1 2];
xM = [x^2 11/7; -3.2 -x];
laguerreL(n,a,xM)

ans =
[ a^2/2 - a*x^2 + (3*a)/2 + x^4/2 - 2*x^2 + 1,...
  a^3/6 + (3*a^2)/14 - (253*a)/294 - 676/1029]
[
  a + 21/5,...
  a^2/2 + a*x + (3*a)/2 + x^2/2 + 2*x + 1]
```

`laguerreL` acts element-wise on  $n$  and  $x$  to return a matrix of the same size as  $n$  and  $x$ .

### Differentiate and Find Limits of Laguerre Polynomials

Use `limit` to find the limit of a generalized Laguerre polynomial of degree 3 as  $x$  tends to  $\infty$ .

```
syms x
expr = laguerreL(3,2,x);
limit(expr,x,Inf)
```

```
ans =
-Inf
```

Use `diff` to find the third derivative of the generalized Laguerre polynomial `laguerreL(n,a,x)`.

```
syms n a
expr = laguerreL(n,a,x);
diff(expr,x,3)

ans =
-laguerreL(n - 3, a + 3, x)
```

### Find Taylor Series Expansion of Laguerre Polynomials

Use `taylor` to find the Taylor series expansion of the generalized Laguerre polynomial of degree 2 at  $x = 0$ .

```
syms a x
expr = laguerreL(2,a,x);
taylor(expr,x)

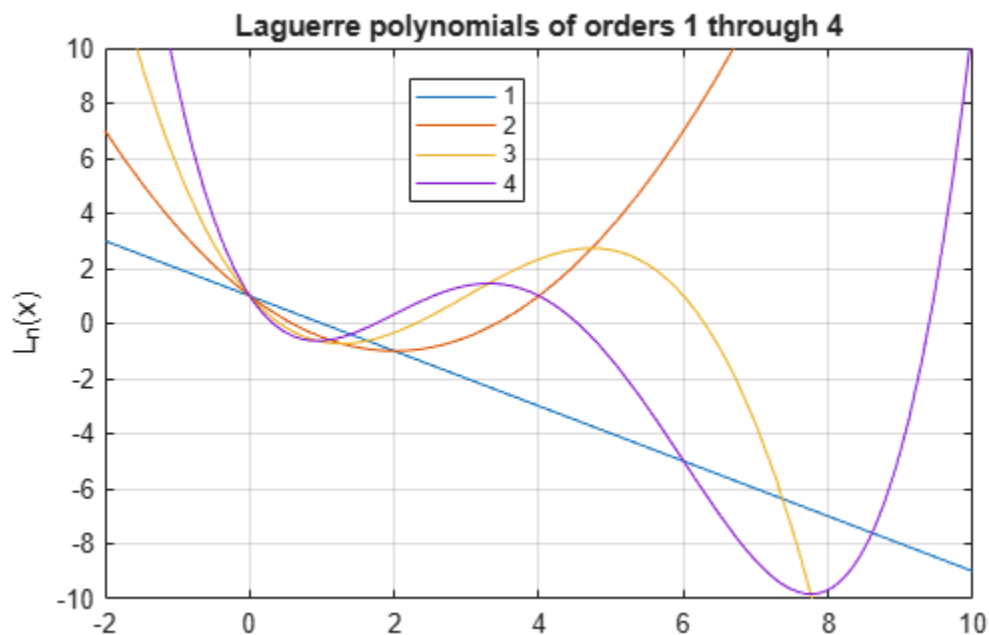
ans =
(3*a)/2 - x*(a + 2) + a^2/2 + x^2/2 + 1
```

### Plot Laguerre Polynomials

Plot the Laguerre polynomials of orders 1 through 4.

```
syms x
fplot(laguerreL(1:4,x))
axis([-2 10 -10 10])
grid on

ylabel('L_n(x)')
title('Laguerre polynomials of orders 1 through 4')
legend('1','2','3','4','Location','best')
```





## Input Arguments

### **n — Degree of polynomial**

number | vector | matrix | multidimensional array | symbolic number | symbolic vector | symbolic matrix | symbolic function | symbolic multidimensional array

Degree of polynomial, specified as a number, vector, matrix, multidimensional array, or a symbolic number, vector, matrix, function, or multidimensional array.

### **x — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic vector | symbolic matrix | symbolic function | symbolic multidimensional array

Input, specified as a number, vector, matrix, multidimensional array, or a symbolic number, vector, matrix, function, or multidimensional array.

### **a — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic vector | symbolic matrix | symbolic function | symbolic multidimensional array

Input, specified as a number, vector, matrix, multidimensional array, or a symbolic number, vector, matrix, function, or multidimensional array.

## More About

### Generalized Laguerre Function

The generalized Laguerre function is defined in terms of the hypergeometric function as

$$\text{laguerreL}(n, a, x) = \binom{n+a}{a} {}_1F_1(-n; a+1; x).$$

For nonnegative integer values of  $n$ , the function returns the generalized Laguerre polynomials that are orthogonal with respect to the scalar product

$$\langle f1, f2 \rangle = \int_0^{\infty} e^{-x} x^a f1(x) f2(x) dx.$$

In particular, the generalized Laguerre polynomials satisfy this normalization.

$$\langle \text{laguerreL}(n, a, x), \text{laguerreL}(m, a, x) \rangle = \begin{cases} 0 & \text{if } n \neq m \\ \frac{\Gamma(a+n+1)}{n!} & \text{if } n = m. \end{cases}$$

## Algorithms

- The generalized Laguerre function is not defined for all values of parameters  $n$  and  $a$  because certain restrictions on the parameters exist in the definition of the hypergeometric functions. If the generalized Laguerre function is not defined for a particular pair of  $n$  and  $a$ , the `laguerreL` function returns an error message. See “Return Generalized Laguerre Function” on page 6-935.
- The calls `laguerreL(n, x)` and `laguerreL(n, 0, x)` are equivalent.

- If  $n$  is a nonnegative integer, the `laguerreL` function returns the explicit form of the corresponding Laguerre polynomial.
- The special values  $\text{laguerreL}(n, a, 0) = \binom{n+a}{a}$  are implemented for arbitrary values of  $n$  and  $a$ .
- If  $n$  is a negative integer and  $a$  is a numerical noninteger value satisfying  $a \geq -n$ , then `laguerreL` returns 0.
- If  $n$  is a negative integer and  $a$  is an integer satisfying  $a < -n$ , the function returns an explicit expression defined by the reflection rule

$$\text{laguerreL}(n, a, x) = (-1)^a e^x \text{laguerreL}(-n - a - 1, a, -x)$$

- If all arguments are numerical and at least one argument is a floating-point number, then `laguerreL(x)` returns a floating-point number. For all other arguments, `laguerreL(n, a, x)` returns a symbolic function call.

## Version History

Introduced in R2014b

## See Also

`chebyshevT` | `chebyshevU` | `gegenbauerC` | `hermiteH` | `hypergeom` | `jacobiP` | `legendreP`

# lambertw

Lambert W function

## Syntax

```
lambertw(x)
lambertw(k,x)
```

## Description

`lambertw(x)` returns the principal branch of the Lambert W function on page 6-944. This syntax is equivalent to `lambertw(0,x)`.

`lambertw(k,x)` is the  $k$ th branch of the Lambert W function. This syntax returns real values only if  $k = 0$  or  $k = -1$ .

## Examples

### Return Equation with Lambert W Function as Its Solution

The Lambert W function  $W(x)$  is a set of solutions of the equation  $x = W(x)e^{W(x)}$ .

Solve this equation. The solution is the Lambert W function.

```
syms x W
eqn = x == W*exp(W);
solve(eqn,W)
```

```
ans =
lambertw(0, x)
```

Verify that branches of the Lambert W function are valid solutions of the equation  $x = W e^W$ :

```
k = -2:2;
eqn = subs(eqn,W,lambertw(k,x));
isAlways(eqn)
```

```
ans =
1x5 logical array
     1     1     1     1     1
```

### Lambert W Function for Numeric and Symbolic Arguments

Depending on its arguments, `lambertw` can return floating-point or exact symbolic results.

Compute the Lambert W functions for these numbers. Because the numbers are not symbolic objects, you get floating-point results.

```
A = [0 -1/exp(1); pi i];
lambertw(A)

ans =
    0.0000 + 0.0000i   -1.0000 + 0.0000i
    1.0737 + 0.0000i    0.3747 + 0.5764i

lambertw(-1,A)
```

```
ans =
   -Inf + 0.0000i   -1.0000 + 0.0000i
  -0.3910 - 4.6281i  -1.0896 - 2.7664i
```

Compute the Lambert W functions for the numbers converted to symbolic objects. For most symbolic (exact) numbers, `lambertw` returns unresolved symbolic calls.

```
A = [0 -1/exp(sym(1)); pi i];
W0 = lambertw(A)

W0 =
[
    0,          -1]
[ lambertw(0, pi), lambertw(0, 1i)]

Wmin1 = lambertw(-1,A)
```

```
Wmin1 =
[
   -Inf,          -1]
[ lambertw(-1, pi), lambertw(-1, 1i)]
```

Convert symbolic results to double by using `double`.

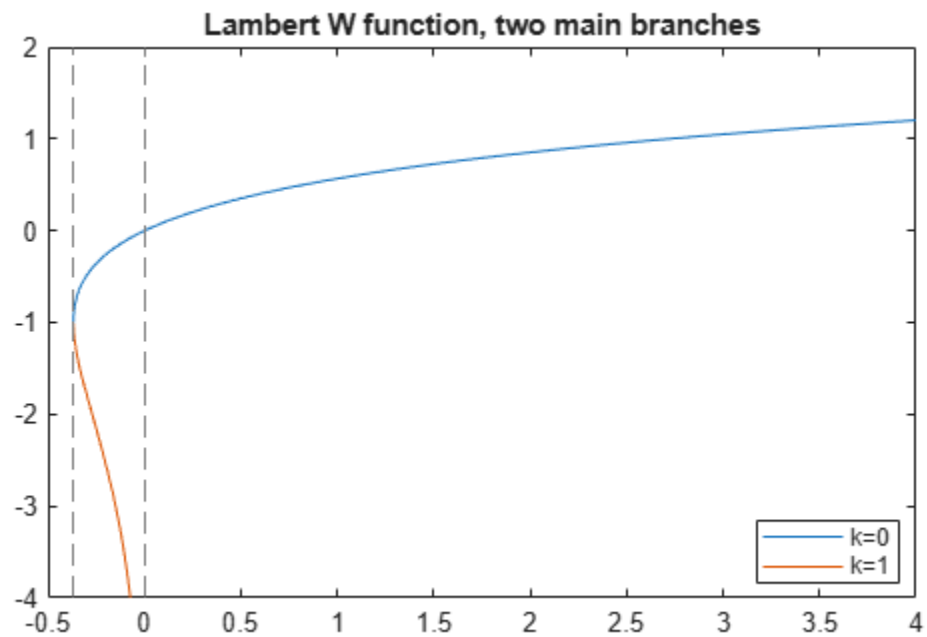
```
double(W0)

ans =
    0.0000 + 0.0000i   -1.0000 + 0.0000i
    1.0737 + 0.0000i    0.3747 + 0.5764i
```

### Plot Two Main Branches of Lambert W Function

Plot the two main branches,  $W_0(x)$  and  $W_{-1}(x)$ , of the Lambert W function.

```
syms x
fplot(lambertw(x))
hold on
fplot(lambertw(-1,x))
hold off
axis([-0.5 4 -4 2])
title('Lambert W function, two main branches')
legend('k=0','k=1','Location','best')
```

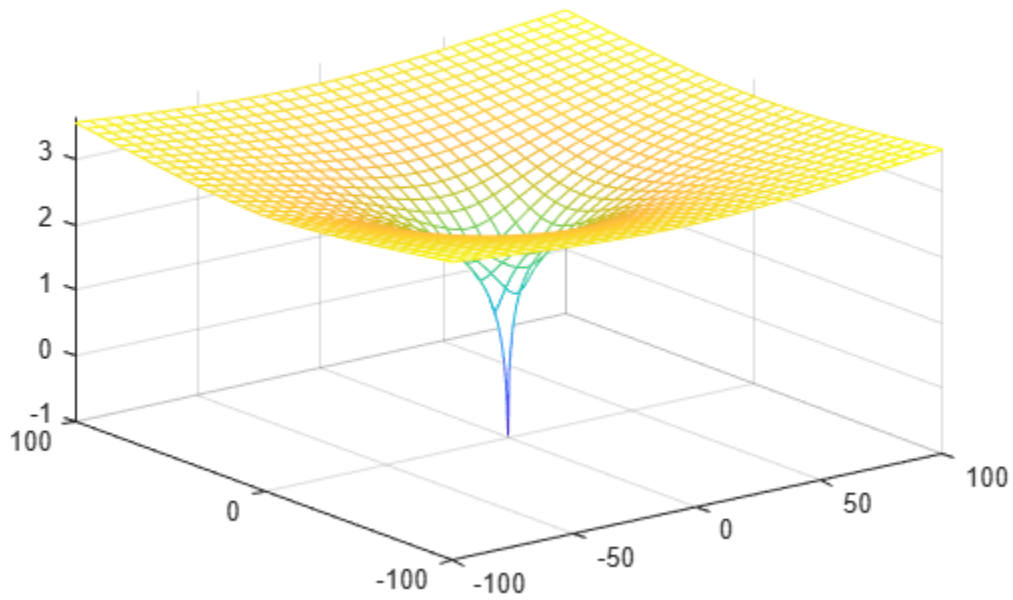


### Lambert W Function Plot on Complex Plane

Plot the principal branch of the Lambert W function on the complex plane.

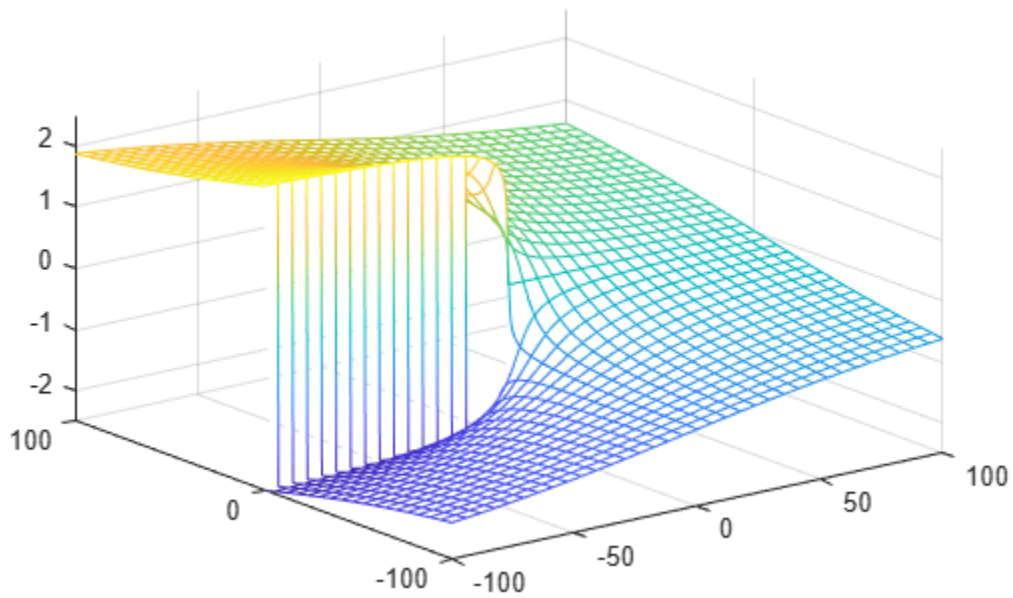
Plot the real value of the Lambert W function by using `fmesh`. Simultaneously plot the contours by setting 'ShowContours' to 'On'.

```
syms x y
f = lambertw(x + 1i*y);
interval = [-100 100 -100 100];
fmesh(real(f),interval,'ShowContours','On')
```

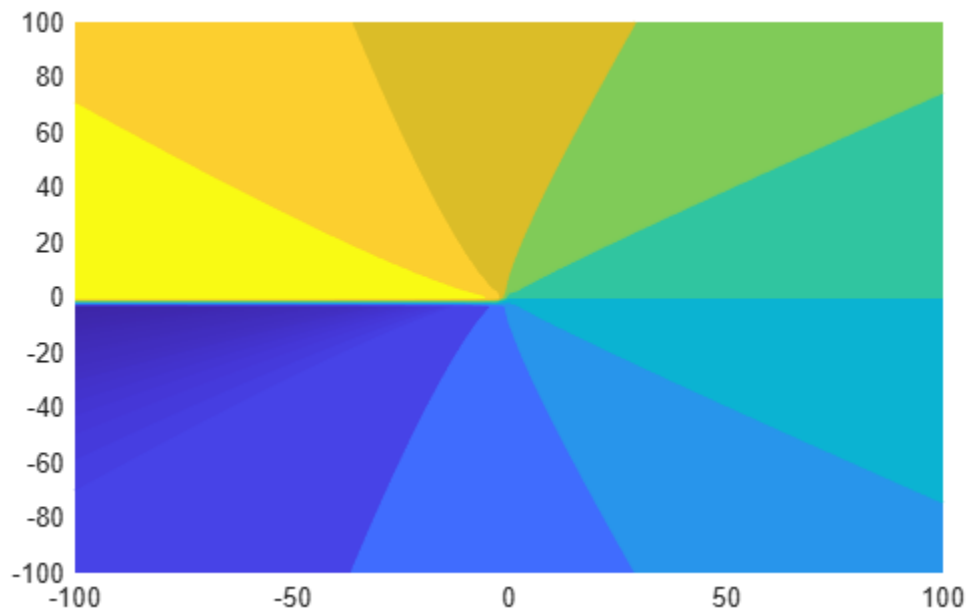


Plot the imaginary value of the Lambert W function. The plot has a branch cut along the negative real axis. Plot the contours separately.

```
fmesh(imag(f),interval)
```

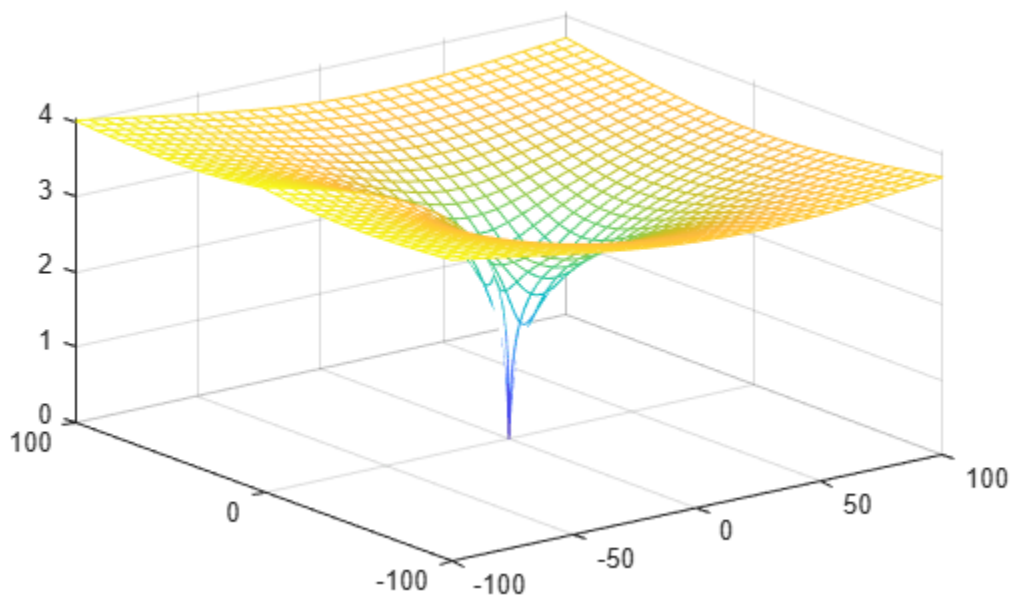


```
fcontour(imag(f),interval,'Fill','on')
```



Plot the absolute value of the Lambert W function.

```
fmesh(abs(f),interval,'ShowContours','On')
```



## Input Arguments

### **x — Input**

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

At least one input argument must be a scalar, or both arguments must be vectors or matrices of the same size. If one input argument is a scalar and the other is a vector or matrix, `lambertw` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

### **k — Branch of Lambert W function**

integer | vector or matrix of integers | symbolic integer | symbolic vector or matrix of integers

Branch of Lambert W function, specified as an integer, a vector or matrix of integers, a symbolic integer, or a symbolic vector or matrix of integers.

At least one input argument must be a scalar, or both arguments must be vectors or matrices of the same size. If one input argument is a scalar and the other is a vector or matrix, `lambertw` expands the scalar into a vector or matrix of the same size as the other argument with all elements equal to that scalar.

## More About

### **Lambert W Function**

The Lambert W function  $W(x)$  represents the solutions  $y$  of the equation  $ye^y = x$  for any complex number  $x$ .

- For complex  $x$ , the equation has an infinite number of solutions  $y = \text{lambertW}(k,x)$  where  $k$  ranges over all integers.
- For all real  $x \geq 0$ , the equation has exactly one real solution  $y = \text{lambertW}(x) = \text{lambertW}(0,x)$ .
- For real  $x$  where  $-e^{-1} < x < 0$ , the equation has exactly two real solutions. The larger solution is represented by  $y = \text{lambertW}(x)$  and the smaller solution by  $y = \text{lambertW}(-1,x)$ .
- For  $x = -e^{-1}$ , the equation has exactly one real solution  $y = -1 = \text{lambertW}(0, -\exp(-1)) = \text{lambertW}(-1, -\exp(-1))$ .

## Version History

Introduced before R2006a

## References

- [1] Corless, R.M., G.H. Gonnet, D.E.G. Hare, D.J. Jeffrey, and D.E. Knuth. "On the Lambert W Function." *Advances in Computational Mathematics*, Vol. 5, pp. 329–359, 1996.



## See Also

### Functions

wrightOmega

# laplace

Laplace transform

## Syntax

```
F = laplace(f)
F = laplace(f,transVar)
F = laplace(f,var,transVar)
```

## Description

`F = laplace(f)` returns the “Laplace Transform” on page 6-949 of `f`. By default, the independent variable is `t` and the transformation variable is `s`. If `f` does not contain `t`, then `laplace` uses the function `symvar` to determine the independent variable.

`F = laplace(f,transVar)` uses the transformation variable `transVar` instead of `s`.

`F = laplace(f,var,transVar)` uses the independent variable `var` and the transformation variable `transVar` instead of `t` and `s`, respectively.

## Examples

### Laplace Transform of Symbolic Expression

Compute the Laplace transform of  $1/\sqrt{x}$ . By default, the transform is in terms of `s`.

```
syms x y
f = 1/sqrt(x);
F = laplace(f)
```

`F =`

$$\frac{\sqrt{\pi}}{\sqrt{s}}$$

### Specify Independent Variable and Transformation Variable

Compute the Laplace transform of  $\exp(-a*t)$ . By default, the independent variable is `t`, and the transformation variable is `s`.

```
syms a t y
f = exp(-a*t);
F = laplace(f)
```

`F =`

$$\frac{1}{a + s}$$

Specify the transformation variable as  $y$ . If you specify only one variable, that variable is the transformation variable. The independent variable is still  $t$ .

```
F = laplace(f,y)
```

$$F = \frac{1}{a + y}$$

Specify both the independent and transformation variables as  $a$  and  $y$  in the second and third arguments, respectively.

```
F = laplace(f,a,y)
```

$$F = \frac{1}{t + y}$$

### Laplace Transforms of Dirac and Heaviside Functions

Compute the Laplace transforms of the Dirac and Heaviside functions.

```
syms t s
syms a positive
F = laplace(dirac(t-a),t,s)
```

$$F = e^{-as}$$

```
F = laplace(heaviside(t-a),t,s)
```

$$F = \frac{e^{-as}}{s}$$

### Relation Between Laplace Transform of Function and Its Derivative

Show that the Laplace transform of the derivative of a function is expressed in terms of the Laplace transform of the function itself.

```
syms f(t) s
Df = diff(f(t),t);
F = laplace(Df,t,s)

F = slaplace(f(t),t,s) - f(0)
```

### Laplace Transform of Array Inputs

Find the Laplace transform of the matrix  $M$ . Specify the independent and transformation variables for each matrix entry by using matrices of the same size. When the arguments are nonscalars, `laplace` acts on them element-wise.

```
syms a b c d w x y z
M = [exp(x) 1; sin(y) 1i*z];
vars = [w x; y z];
transVars = [a b; c d];
F = laplace(M,vars,transVars)
```

$$F = \begin{pmatrix} \frac{e^x}{a} & \frac{1}{b} \\ \frac{1}{c^2 + 1} & \frac{i}{d^2} \end{pmatrix}$$

If `laplace` is called with both scalar and nonscalar arguments, then it expands the scalars to match the nonscalars by using scalar expansion. Nonscalar arguments must be the same size.

```
F = laplace(x,vars,transVars)
```

$$F = \begin{pmatrix} \frac{x}{a} & \frac{1}{b^2} \\ \frac{x}{c} & \frac{x}{d} \end{pmatrix}$$

### Laplace Transform of Symbolic Function

Compute the Laplace transform of symbolic functions. When the first argument contains symbolic functions, then the second argument must be a scalar.

```
syms f1(x) f2(x) a b
f1(x) = exp(x);
f2(x) = x;
F = laplace([f1 f2],x,[a b])
```

$$F = \begin{pmatrix} \frac{1}{a-1} & \frac{1}{b^2} \end{pmatrix}$$

### If Laplace Transform Cannot Be Found

If `laplace` cannot transform the input then it returns an unevaluated call.

```
syms f(t) s
f(t) = 1/t;
F(s) = laplace(f,t,s)
```

$$F(s) = \text{laplace}\left(\frac{1}{t}, t, s\right)$$

Return the original expression by using `ilaplace`.

```
f(t) = ilaplace(F,s,t)
```

$$f(t) = \frac{1}{t}$$

## Input Arguments

### **f** — Input

symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic expression, function, vector, or matrix.

### **var** — Independent variable

$t$  (default) | symbolic variable

Independent variable, specified as a symbolic variable. This variable is often called the "time variable" or the "space variable." If you do not specify the variable then, by default, `laplace` uses  $t$ . If  $f$  does not contain  $t$ , then `laplace` uses the function `symvar` to determine the independent variable.

### **transVar** — Transformation variable

$s$  (default) |  $z$  | symbolic variable | symbolic expression | symbolic vector | symbolic matrix

Transformation variable, specified as a symbolic variable, expression, vector, or matrix. This variable is often called the "complex frequency variable." If you do not specify the variable then, by default, `laplace` uses  $s$ . If  $s$  is the independent variable of  $f$ , then `laplace` uses  $z$ .

## More About

### Laplace Transform

The Laplace transform  $F(s)$  of the expression  $f(t)$  with respect to the variable  $t$  at the point  $s$  is a unilateral transform defined by

$$F(s) = \int_{0^-}^{\infty} f(t) e^{-st} dt.$$

## Tips

- If any argument is an array, then `laplace` acts element-wise on all elements of the array.
- If the first argument contains a symbolic function, then the second argument must be a scalar.
- To compute the inverse Laplace transform, use `ilaplace`.

## Algorithms

The Laplace transform is defined as a unilateral or one-sided transform. This definition assumes that the signal  $f(t)$  is only defined for all real numbers  $t \geq 0$ , or  $f(t) = 0$  for  $t < 0$ . Therefore, for a generalized signal with  $f(t) \neq 0$  for  $t < 0$ , the Laplace transform of  $f(t)$  gives the same result as if  $f(t)$  is multiplied by a Heaviside step function.

For example, both of these code blocks:

```
syms t;  
laplace(sin(t))  
  
and  
  
syms t;  
laplace(sin(t)*heaviside(t))  
  
return 1/(s^2 + 1).
```

## Version History

Introduced before R2006a

## See Also

[fourier](#) | [ifourier](#) | [ilaplace](#) | [iztrans](#) | [ztrans](#)

## Topics

“Solve Differential Equations of RLC Circuit Using Laplace Transform” on page 3-198

# laplacian

Laplacian of symbolic field

## Syntax

```
l = laplacian(f,v)
l = laplacian(f)
```

## Description

`l = laplacian(f,v)` returns the “Laplacian” on page 6-953 of the symbolic field `f` with respect to the vector `v` in Cartesian coordinates. If `f` is an array, then the function computes the Laplacian for each element of `f` and returns the output `l` that is the same size as `f`.

`l = laplacian(f)` returns the Laplacian of the symbolic field `f` with respect to a default vector constructed from the symbolic variables in `f`.

## Examples

### Laplacian of Scalar Field

Create a symbolic function for the scalar field  $f(x, y, z) = \sin(x) + y^2 + z^3$ . Compute the Laplacian of this function with respect to the vector  $(x, y, z)$ .

```
syms x y z
f(x,y,z) = sin(x) + y^2 + z^3;
v = [x y z];
lf = laplacian(f,v)

lf(x, y, z) = 6z - sin(x) + 2
```

Create another scalar field  $g(x, y, z) = x^2y + z$ . Find the Laplacian without specifying the vector to differentiate with. Because `g` is a function of symbolic scalar variables, `laplacian` finds the Laplacian of `g` with respect to the default variables as defined by `symvar(g)`.

```
g(x,y,z) = x^2*y + z;
lg = laplacian(g)

lg(x, y, z) = 2y
```

### Laplacian of Vector Field

Create a symbolic vector field  $F = \begin{bmatrix} \sin(x) + y^2 + z^3 \\ x^2y + z \end{bmatrix}$ . Find the Laplacian of this vector field with respect to  $(x, y, z)$ .

```
syms x y z
F = [sin(x) + y^2 + z^3; x^2*y + z];
```

```
v = [x y z];
l = laplacian(F,v)

l =
    (6 z - sin(x) + 2)
      2 y
```

### Find Vector Calculus Identities Involving Laplacian

Create a 3-by-1 vector as a symbolic matrix variable  $\mathbf{X}$ . Create a scalar field that is a function of  $\mathbf{X}$  as a symbolic matrix function  $\psi(\mathbf{X})$ , keeping the existing definition of  $\mathbf{X}$ .

```
syms X [3 1] matrix
syms psi(X) [1 1] matrix keepargs
```

Show that the divergence of the gradient of  $\psi(\mathbf{X})$  is equal to the Laplacian of  $\psi(\mathbf{X})$ , that is  $\nabla_{\mathbf{X}} \cdot \nabla_{\mathbf{X}} \psi(\mathbf{X}) = \Delta_{\mathbf{X}} \psi(\mathbf{X})$ .

```
divOfGradPsi = divergence(gradient(psi,X),X)
divOfGradPsi(X) = ΔX ψ(X)

lapPsi = laplacian(psi,X)
lapPsi(X) = ΔX ψ(X)
```

Create a vector field that is a function of  $\mathbf{X}$  as a symbolic matrix function  $\mathbf{A}(\mathbf{X})$ .

```
syms A(X) [3 1] matrix keepargs
```

Show that the gradient of the divergence of  $\mathbf{A}(\mathbf{X})$  minus the curl of the curl of  $\mathbf{A}(\mathbf{X})$  is equal to the Laplacian of  $\mathbf{A}(\mathbf{X})$ , that is  $\nabla_{\mathbf{X}} \nabla_{\mathbf{X}} \cdot \mathbf{A}(\mathbf{X}) - \nabla_{\mathbf{X}} \times \nabla_{\mathbf{X}} \times \mathbf{A}(\mathbf{X}) = \Delta_{\mathbf{X}} \mathbf{A}(\mathbf{X})$ .

```
identityA = gradient(divergence(A,X),X) - curl(curl(A,X),X)
identityA(X) = ΔX A(X)
```

## Input Arguments

### f — Symbolic field

symbolic expression | symbolic function | symbolic matrix variable | symbolic matrix function

Symbolic field, specified as a symbolic expression, symbolic function, symbolic matrix variable, or symbolic matrix function. The input field can be a scalar, vector, matrix, or a multidimensional array, where the Laplacian is computed for each element in the input.

- If  $f$  is a function of symbolic scalar variables, where  $f$  is of type `sym` or `symfun`, then the vector  $v$  must be of type `sym` or `symfun`.
- If  $f$  is a function of symbolic matrix variables, where  $f$  is of type `symmatrix` or `symfunmatrix`, then the vector  $v$  must be of type `symmatrix` or `symfunmatrix`.

Data Types: `sym` | `symfun` | `symmatrix` | `symfunmatrix`



**v — Vector with respect to which you find the Laplacian**

vector of symbolic scalar variables | symbolic function | symbolic matrix variable | symbolic matrix function

Vector with respect to which you find the Laplacian, specified as a vector of symbolic scalar variables, symbolic function, symbolic matrix variable, or a symbolic matrix function.

- If you do not specify **v** and **f** is a function of symbolic scalar variables, then, by default, `laplacian` constructs vector **v** from the symbolic scalar variables in **f** with the order of variables as defined by `symvar(f)`.
- If **v** is a symbolic matrix variable of type `symmatrix`, then **v** must have a size of 1-by-N or N-by-1.
- If **v** is scalar, then `laplacian(f,v) = diff(f,2,v)`.

Data Types: `sym` | `symfun` | `symmatrix` | `symfunmatrix`

**Limitations**

- Symbolic Math Toolbox currently does not support the `dot` or `cross` functions for symbolic matrix variables and functions of type `symmatrix` and `symfunmatrix`. If vector calculus identities involve dot or cross products, then the toolbox displays those identities in terms of other supported functions instead. To see a list of all the functions that support symbolic matrix variables and functions, use the commands `methods symmatrix` and `methods symfunmatrix`.
- If the input data type of the symbolic field **f** is `symmatrix` or `symfunmatrix`, then `laplacian` does not evaluate the partial derivatives of **f**. Instead, it returns an unevaluated formula for symbolic manipulation and formula rearrangement.

**More About****Laplacian**

The Laplacian or Laplace's differential operator of the scalar field  $f$  with respect to the vector  $x = (x_1, \dots, x_n)$  is the sum of the second derivatives of  $f$  with respect to  $x_1, \dots, x_n$ .

$$\Delta_x f = \sum_{i=1}^n \frac{\partial^2 f}{\partial x_i^2}$$

If  $f$  is a vector field or a tensor field (multidimensional array), then the Laplacian operator is applied to each element in  $f$ .

**Alternatives**

The Laplacian of a scalar function or functional expression is the divergence of the gradient of that function or expression.

$$\Delta f = \nabla \cdot (\nabla f)$$

For a symbolic scalar field **f**, you can also compute the Laplacian using the `divergence` and `gradient` functions.

```
syms f(x,y)
divergence(gradient(f(x,y)), [x y])
```

## Version History

Introduced in R2012a

**R2023a: Compute Laplacian of symbolic matrix variables, functions, and multidimensional arrays**

The `laplacian` function accepts symbolic matrix variables and functions of type `symmatrix` and `symfunmatrix` as input arguments. For example, see “Find Vector Calculus Identities Involving Laplacian” on page 6-952.

You can also compute the Laplacian of a multidimensional array `f`. The `laplacian` function computes the Laplacian for each element of `f` and returns the output `l` that is the same size as `f`. In previous releases, `f` must be scalar. For example, see “Laplacian of Vector Field” on page 6-951.

### See Also

`curl` | `diff` | `divergence` | `gradient` | `hessian` | `jacobian` | `potential` | `vectorPotential`

# latex

LaTeX form of symbolic expression

## Syntax

```
chr = latex(S)
```

## Description

`chr = latex(S)` returns the LaTeX form of the symbolic expression `S`.

## Examples

### LaTeX Form of Symbolic Expressions

Find the LaTeX form of the symbolic expressions  $x^2 + 1/x$  and  $\sin(\pi x) + \phi$ .

```
syms x phi
chr = latex(x^2 + 1/x)

chr =
'\frac{1}{x}+x^2'

chr = latex(sin(pi*x) + phi)

chr =
'\phi +\sin\left(\pi \,x\right)'
```

### LaTeX Form of Symbolic Array

Find the LaTeX form of the symbolic array `S`.

```
syms x
S = [sym(1)/3 x; exp(x) x^2]

S =

$$\begin{pmatrix} \frac{1}{3} & x \\ e^x & x^2 \end{pmatrix}$$


chr = latex(S)

chr =
'\left(\begin{array}{cc} \frac{1}{3} & x \\ {\mathrm{e}}^x & x^2 \end{array}\right)'
```

### LaTeX Form of Symbolic Matrix Variables

Perform computation using several symbolic matrix variables, and then find their LaTeX forms.

Create 3-by-3 and 3-by-1 symbolic matrix variables.

```
syms A 3 matrix
syms X [3 1] matrix
```

Find the Hessian matrix of  $X^T A X$ . Derived equations involving symbolic matrix variables appear in typeset as they would be in textbooks.

```
f = X.'*A*X
```

```
f = XTA X
```

```
H = diff(f,X,X.')
```

```
H = AT + A
```

Generate the LaTeX forms of the symbolic matrix variables  $f$  and  $H$ .

```
chrf = latex(f)
```

```
chrf =
'\textbf{X}^{\mathrm{T}}\textbf{A}\textbf{X}'
```

```
chrH = latex(H)
```

```
chrH =
'\textbf{A}^{\mathrm{T}}+\textbf{A}'
```

### LaTeX Form of Symbolic Matrix Functions

Perform computation using symbolic matrix functions, and then find their LaTeX forms.

Create a 3-by-1 symbolic matrix variable.

```
syms X [3 1] matrix
```

Create a symbolic matrix function that represents the formula  $f(\mathbf{X}) = \mathbf{X}^T \mathbf{X}$ .

```
syms f(X) [1 1] matrix keepargs
f(X) = X.'*X
```

```
f(X) = XTX
```

Find the derivative of  $f(\mathbf{X})$  with respect to  $\mathbf{X}$ .

```
Df = diff(f,X)
```

```
Df(X) = 2 XT
```

Generate the LaTeX forms of the symbolic matrix functions  $f$  and  $Df$ .

```
chrf = latex(f)
```

```
chrf =
'\textbf{X}}^{\mathrm{T}}\,\,\textbf{X}'

chrDf = latex(Df)

chrDf =
'2\,\,\textbf{X}}^{\mathrm{T}}'
```

## Modify Generated LaTeX with Symbolic Settings

Modify generated LaTeX by changing symbolic settings using the `sympref` function.

Generate the LaTeX form of the expression  $\pi$  with the default symbolic setting.

```
sympref("default");
chr = latex(sym(pi))

chr =
'\pi '
```

Set "FloatingPointOutput" to `true` to return symbolic output in floating-point format. Generate the LaTeX form of  $\pi$  in floating-point format.

```
sympref("FloatingPointOutput",true);
chr = latex(sym(pi))

chr =
'3.1416'
```

Now change the output order of a symbolic polynomial. Create a symbolic polynomial and set "PolynomialDisplayStyle" to "ascend". Generate the LaTeX form of the polynomial sorted in ascending order.

```
syms x;
poly = x^2 - 2*x + 1;
sympref("PolynomialDisplayStyle","ascend");
chr = latex(poly)

chr =
'1-2\,x+x^2'
```

The symbolic settings that you set using `sympref` persist through your current and future MATLAB® sessions. Restore the default values by specifying the "default" option.

```
sympref("default");
```

## Use LaTeX to Format Title, Axis Labels, and Ticks

For  $x$  and  $y$  from  $-2\pi$  to  $2\pi$ , plot the 3-D surface  $y\sin(x) - x\cos(y)$ . Store the axes object in `a` by using `gca`. Use `latex` interpreter for the tick labels.

Create the x-axis ticks by spanning the x-axis limits at intervals of  $\pi/2$ . Convert the axis limits to precise multiples of  $\pi/2$  using `round` and get the symbolic tick values in `S`. Set the locations of the

x-axis ticks by using the `xticks` function. Create the LaTeX labels for the x-axis by using `arrayfun` to apply `latex` to `S` and then concatenating `$`. Display the labels by using the `xticklabels` function.

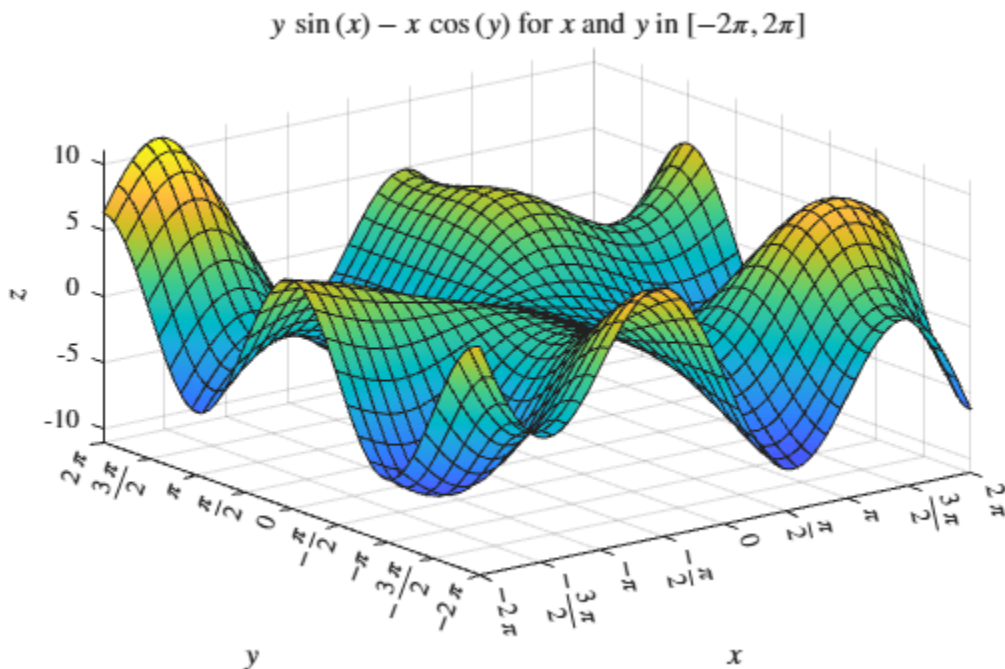
Repeat these steps for the y-axis. Set the x- and y-axes labels and the title using the `latex` interpreter.

```
syms x y
f = y*sin(x)-x*cos(y);
fsurf(f,[-2*pi 2*pi])
a = gca;
a.TickLabelInterpreter = "latex";

S = sym(a.XLim(1):pi/2:a.XLim(2));
S = sym(round(S/pi*2)*pi/2);
xticks(double(S));
labels = "$" + arrayfun(@latex,S,UniformOutput=false) + "$";
xticklabels(labels);

S = sym(a.YLim(1):pi/2:a.YLim(2));
S = sym(round(S/pi*2)*pi/2);
yticks(double(S));
labels = "$" + arrayfun(@latex,S,UniformOutput=false) + "$";
yticklabels(labels);

xlabel("$x$",Interpreter="latex");
ylabel("$y$",Interpreter="latex");
zlabel("$z$",Interpreter="latex");
titletext = "$" + latex(f) + "$ for $x$ and $y$ in $[-2\pi,2\pi]$";
title(titletext,Interpreter="latex")
```



## Input Arguments

### S — Input

symbolic number | symbolic variable | symbolic vector | symbolic array | symbolic function | symbolic expression | symbolic matrix variable | symbolic matrix function

Input, specified as a symbolic number, variable, vector, array, function, expression, matrix variable, or symbolic matrix function.

Data Types: `sym` | `symfun` | `symmatrix` | `symfunmatrix`

## Version History

Introduced before R2006a

### R2022a: Generate LaTeX form of symbolic matrix functions

The `latex` function accepts an input argument of type `symfunmatrix`. For an example, see “LaTeX Form of Symbolic Matrix Functions” on page 6-956.

### R2021b: Generate LaTeX form of symbolic matrix variables

The `latex` function accepts an input argument of type `symmatrix`. For an example, see “LaTeX Form of Symbolic Matrix Variables” on page 6-955.

### See Also

`ccode` | `displayFormula` | `fortran` | `texlabel` | `mathml` | `sympref`

### Topics

“Copy and Paste Symbolic Output in Live Editor” on page 2-18

## lcm

Least common multiple of symbolic numbers and polynomials

### Syntax

```
L = lcm(A)
L = lcm(A,B)
```

### Description

$L = \text{lcm}(A)$  finds the least common multiple (LCM) of all symbolic elements in  $A$ .

$L = \text{lcm}(A,B)$  finds the least common multiple of  $A$  and  $B$ .

### Examples

#### Least Common Multiple of Symbolic Integers

Find the least common multiple of four integers, specified as elements of a symbolic vector.

```
A = sym([4420 -128 8984 -488]);
L = lcm(A)
```

```
L = 9689064320
```

Alternatively, specify these values as elements of a symbolic matrix.

```
A = sym([4420 -128; 8984 -488]);
L = lcm(A)
```

```
L = 9689064320
```

#### Least Common Multiple of Rational Numbers

Find the least common multiple of five symbolic rational numbers.

```
A = sym([3/4 7/3 11/2 12/3 33/4])
```

```
A =
    (3 7 11 33)
    (4 3 2 4)
```

```
L = lcm(A)
```

```
L = 924
```



### Least Common Multiple of Complex Numbers

Find the least common multiple of three symbolic complex numbers.

```
A = sym([3 + 1i, -4 - 2i, -5 - 5i]);
L = lcm(A)

L = 10
```

### Least Common Multiple of Elements of Matrices

If you specify two input arguments and at least one of them is nonscalar, `lcm` finds the least common multiples element-wise.

Find the least common multiples for the elements of two matrices. When you specify two nonscalar input arguments, they must be the same size.

```
A = sym([309 186; 486 224]);
B = sym([558 444; 1024 1984]);
L = lcm(A,B)
```

```
L =
    ( 57474 13764 )
    (248832 13888)
```

Find the least common multiples for the elements of matrix A and the value 99. Here, `lcm` expands 99 into a 2-by-2 matrix with all elements equal to 99.

```
L = lcm(A,99)
```

```
L =
    (10197 6138 )
    ( 5346 22176)
```

### Least Common Multiple of Polynomials

Find the least common multiple of two univariate polynomials.

```
syms x
A = x^3 - 3*x^2 + 3*x - 1;
B = x^2 - 5*x + 4;
L = lcm(A,B)
```

```
L = (x - 4)(x^3 - 3x^2 + 3x - 1)
```

Find the least common multiple of three multivariate polynomials. Because there are more than two polynomials, specify them as elements of a symbolic vector.

```
syms x y
A = [x^2*y + x^3, (x + y)^2, x^2 + x*y^2 + x*y + x + y^3 + y];
L = lcm(A)
```

$$L = (x^3 + yx^2)(x^2 + xy^2 + xy + x + y^3 + y)$$

## Input Arguments

### A — Input value

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input value, specified as a symbolic number, variable, expression, function, or a vector or matrix of symbolic numbers, variables, expressions, or functions.

### B — Input value

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input value, specified as a symbolic number, variable, expression, function, or a vector or matrix of symbolic numbers, variables, expressions, or functions.

If A and B are nonscalar, they must have the same size. If either A or B is scalar, then `lcm` expands the scalar into a vector or matrix of the same size as the nonscalar argument, with all elements equal to the corresponding scalar.

## Output Arguments

### L — Least common multiple

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Least common multiple, returned as a symbolic number, variable, expression, function, or a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Tips

- Calling `lcm` with numbers that are not symbolic objects invokes the MATLAB `lcm` function.
- The MATLAB `lcm` function does not accept rational or complex arguments. To find the least common multiple of rational or complex numbers, convert these numbers to symbolic objects by using `sym`, and then use `lcm`.

## Version History

**Introduced in R2014b**

### R2026a: `lcm` returns real positive integers for inputs that are real integers

*Behavior changed in R2026a*

The `lcm` function now always returns a real positive integer for an input array that contains real positive and negative integers. For example, this code returns 256.

```
L = lcm(sym([256 -128]))
```

```
L =  
256
```

In previous releases, if the input array contains an odd number of real negative integers, then `lcm` returns a real negative integer. For example, the same code returns -256 in previous releases.

```
L = lcm(sym([256 -128]))
```

```
L =  
-256
```

## See Also

`gcd`

## ldivide, ./

Symbolic array left division

### Syntax

```
B.\A  
ldivide(B,A)
```

### Description

`B.\A` divides `A` by `B`.

`ldivide(B,A)` is equivalent to `B.\A`.

### Examples

#### Divide Scalar by Matrix

Create a 2-by-3 matrix.

```
B = sym('b', [2 3])  
  
B =  
[ b1_1, b1_2, b1_3]  
[ b2_1, b2_2, b2_3]
```

Divide the symbolic expression `sin(a)` by each element of the matrix `B`.

```
syms a  
B.\sin(a)  
  
ans =  
[ sin(a)/b1_1, sin(a)/b1_2, sin(a)/b1_3]  
[ sin(a)/b2_1, sin(a)/b2_2, sin(a)/b2_3]
```

#### Divide Matrix by Matrix

Create a 3-by-3 symbolic Hilbert matrix and a 3-by-3 diagonal matrix.

```
H = sym(hilb(3))  
d = diag(sym([1 2 3]))  
  
H =  
[ 1, 1/2, 1/3]  
[ 1/2, 1/3, 1/4]  
[ 1/3, 1/4, 1/5]  
  
d =  
[ 1, 0, 0]  
[ 0, 2, 0]  
[ 0, 0, 3]
```

Divide  $d$  by  $H$  by using the elementwise left division operator `.\`. This operator divides each element of the first matrix by the corresponding element of the second matrix. The dimensions of the matrices must be the same.

```
H.\d
```

```
ans =  
[ 1, 0, 0]  
[ 0, 6, 0]  
[ 0, 0, 15]
```

### Divide Expression by Symbolic Function

Divide a symbolic expression by a symbolic function. The result is a symbolic function.

```
syms f(x)  
f(x) = x^2;  
f1 = f.\(x^2 + 5*x + 6)
```

```
f1(x) =  
(x^2 + 5*x + 6)/x^2
```

## Input Arguments

### A — Input

symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix | symbolic array

Input, specified as a symbolic scalar variable, matrix variable, function, matrix function, expression, or vector, matrix, or array of symbolic scalar variables. Inputs  $A$  and  $B$  must be the same size unless one is a scalar. A scalar value expands into an array of the same size as the other input.

### B — Input

symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix | symbolic array

Input, specified as a symbolic scalar variable, matrix variable, function, matrix function, expression, or vector, matrix, or array of symbolic scalar variables. Inputs  $A$  and  $B$  must be the same size unless one is a scalar. A scalar value expands into an array of the same size as the other input.

## Version History

Introduced before R2006a

### R2022a: Array division of symbolic matrix functions

The `ldivide` function accepts input arguments of type `symfunmatrix`.

### R2021a: Array division of symbolic matrix variables

The `ldivide` function accepts input arguments of type `symmatrix`.

**See Also**

`ctranspose` | `minus` | `mldivide` | `mpower` | `mrdivide` | `mtimes` | `plus` | `power` | `rdivide` | `times`  
| `transpose`

# le

Define less than or equal to condition

## Syntax

```
A <= B
le(A,B)
```

## Description

$A \leq B$  defines the condition less than or equal to.

`le(A,B)` is equivalent to  $A \leq B$ .

## Examples

### Set and Use Assumption Using Less Than or Equal To

Set the assumption that  $x$  is less than or equal to 3 by using `assume`.

```
syms x
cond = x <= 3;
assume(cond)
```

Solve an equation for  $x$ . The solver only returns solutions that are valid under the assumption on  $x$ .

```
eqn = (x-1)*(x-2)*(x-3)*(x-4) == 0;
solve(eqn,x)
```

```
ans =
     1
     2
     3
```

### Find Values that Satisfy Condition

Set the condition `abs(sin(x)) <= 1/2`.

```
syms x
cond = abs(sin(x)) <= 1/2;
```

Find multiples of  $\pi/24$  that satisfy the condition by using a `for` loop from 0 to  $\pi$ .

```
for i = 0:sym(pi/12):sym(pi)
    if subs(cond, x, i)
        disp(i)
    end
end

0
pi/12
pi/6
```

```
(5*pi)/6  
(11*pi)/12  
pi
```

## Input Arguments

### A — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

### B — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## Tips

- Calling `<=` or `le` for non-symbolic A and B invokes the MATLAB `le` function. This function returns a logical array with elements set to logical 1 (`true`) where A is less than or equal to B; otherwise, it returns logical 0 (`false`).
- If both A and B are arrays, then these arrays must have the same dimensions. `A <= B` returns an array of relations `A(i,j,...) <= B(i,j,...)`.
- If one input is scalar and the other an array, then the scalar input is expanded into an array of the same dimensions as the other array.
- The field of complex numbers is not an ordered field. MATLAB projects complex numbers in relations to the real axis. For example, `x <= i` becomes `x <= 0`, and `x <= 3 + 2*i` becomes `x <= 3`.

## Version History

Introduced in R2012a

### See Also

`eq` | `ge` | `gt` | `isAlways` | `lt` | `ne`

### Topics

“Use Assumptions on Symbolic Variables” on page 1-41



# legendreP

Legendre polynomials

## Syntax

`y = legendreP(n,x)`

## Description

`y = legendreP(n,x)` returns the nth degree Legendre polynomial on page 6-972 at x.

## Examples

### Find Legendre Polynomials for Numeric and Symbolic Inputs

Find the Legendre polynomial of degree 3 at 5.6.

`y = legendreP(3,5.6)`

`y =`  
430.6400

Find the Legendre polynomial of degree 2 at x.

`syms x`  
`y = legendreP(2,x)`

`y =`  
$$\frac{3x^2}{2} - \frac{1}{2}$$

If you do not specify a numerical value for the degree n, the `legendreP` function cannot find the explicit form of the polynomial and returns the function call.

`syms n`  
`y = legendreP(n,x)`

`y = legendreP(n,x)`

### Find Legendre Polynomial with Vector and Matrix Inputs

Find the Legendre polynomials of degrees 1 and 2 by setting `n = [1 2]`.

`syms x`  
`y = legendreP([1 2],x)`

`y =`  
$$\left( x \frac{3x^2}{2} - \frac{1}{2} \right)$$

`legendreP` acts element-wise on `n` to return a vector with two elements.

If multiple inputs are specified as a vector, matrix, or multidimensional array, the inputs must be the same size. Find the Legendre polynomials where input arguments `n` and `x` are matrices.

```
n = [2 3; 1 2];
xM = [x^2 11/7; -3.2 -x];
yM = legendreP(n,xM)
```

```
yM =
    (
        3 x^4 - 1/2      2519
         2              343
    )
    (
        -16      3 x^2 - 1
         5        2      2
    )
```

`legendreP` acts element-wise on `n` and `x` to return a matrix of the same size as `n` and `x`.

### Differentiate and Find Limits of Legendre Polynomials

Use `limit` to find the limit of a Legendre polynomial of degree 3 as `x` tends to  $-\infty$ .

```
syms x
P3 = legendreP(3,x);
P3 = limit(P3,x,-Inf)
```

```
P3 = -∞
```

Use `diff` to find the third derivative of the Legendre polynomial of degree 5.

```
P5 = legendreP(5,x);
D3P5 = diff(P5,x,3)
```

```
D3P5 =
    945 x^2 - 105
      2        2
```

### Find Taylor Series Expansion of Legendre Polynomial

Use `taylor` to find the Taylor series expansion of the Legendre polynomial of degree 2 at `x = 0`.

```
syms x
y = legendreP(2,x)
```

```
y =
    3 x^2 - 1
      2      2
```

```
t = taylor(y,x)
```

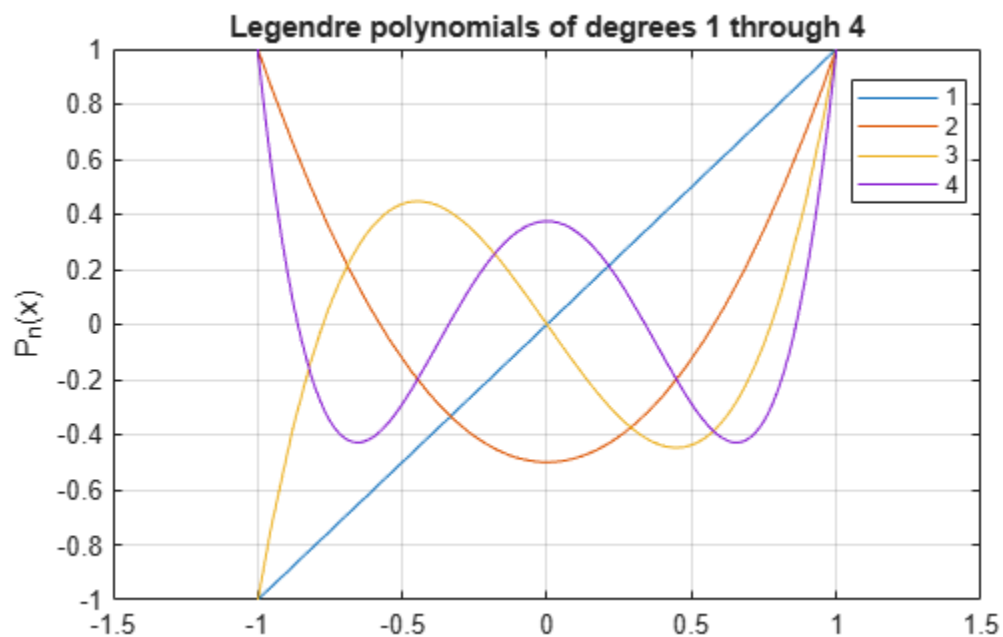
```
t =
    3 x^2 - 1
      2      2
```

## Plot Legendre Polynomials

Plot Legendre polynomials of orders 1 through 4.

```
syms x y
fplot(legendreP(1:4, x))
axis([-1.5 1.5 -1 1])
grid on

ylabel('P_n(x)')
title('Legendre polynomials of degrees 1 through 4')
legend('1','2','3','4','Location','best')
```



## Find Roots of Legendre Polynomial

Use `vpasolve` to find the roots of the Legendre polynomial of degree 7.

```
syms x
roots = vpasolve(legendreP(7,x) == 0)

roots =
    [-0.94910791234275852452618968404785
     -0.74153118559939443986386477328079
     -0.40584515137739716690660641207696
      0
      0.40584515137739716690660641207696
      0.74153118559939443986386477328079
      0.94910791234275852452618968404785]
```

## Input Arguments

### **n — Degree of polynomial**

nonnegative number | vector | matrix | multidimensional array | symbolic number | symbolic vector | symbolic matrix | symbolic function | symbolic multidimensional array

Degree of polynomial, specified as a nonnegative number, vector, matrix, multidimensional array, or a symbolic number, vector, matrix, function, or multidimensional array. All elements of nonscalar inputs should be nonnegative integers or symbols.

### **x — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic vector | symbolic matrix | symbolic function | symbolic multidimensional array

Input, specified as a number, vector, matrix, multidimensional array, or a symbolic number, vector, matrix, function, or multidimensional array.

## Output Arguments

### **y — Legendre polynomial**

number | vector | matrix | multidimensional array | symbolic number | symbolic vector | symbolic matrix | symbolic function | symbolic multidimensional array

Legendre polynomial, returned as a number, vector, matrix, multidimensional array, or a symbolic number, vector, matrix, function, or multidimensional array.

## More About

### Legendre Polynomial

- The Legendre polynomials are defined as

$$P(n, x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} (x^2 - 1)^n.$$

- The Legendre polynomials satisfy the recursion formula

$$P(n, x) = \frac{2n-1}{n} x P(n-1, x) - \frac{n-1}{n} P(n-2, x),$$

where

$$P(0, x) = 1$$

$$P(1, x) = x.$$

- The Legendre polynomials are orthogonal on the interval  $[-1, 1]$  with respect to the weight function  $w(x) = 1$ , where

$$\int_{x=-1}^{x=1} P(n, x) P(m, x) dx = \begin{cases} 0 & \text{if } n \neq m \\ \frac{1}{n+1/2} & \text{if } n = m. \end{cases}$$

- The relation with Gegenbauer polynomials  $G(n, a, x)$  is

$$P(n, x) = G\left(n, \frac{1}{2}, x\right).$$

- The relation with Jacobi polynomials  $P(n, a, b, x)$  is

$$P(n, x) = P(n, 0, 0, x).$$

## Version History

Introduced in R2014b

### See Also

[chebyshevT](#) | [chebyshevU](#) | [gegenbauerC](#) | [hermiteH](#) | [hypergeom](#) | [jacobiP](#) | [laguerreL](#)

## lhs

Left side (LHS) of equation

### Syntax

```
lhsEqn = lhs(eqn)
```

### Description

`lhsEqn = lhs(eqn)` returns the left side of the symbolic equation `eqn`. The value of `eqn` also can be a symbolic condition, such as  $x > 0$ . If `eqn` is an array, then `lhs` returns an array of the left sides of the equations in `eqn`.

Conditions that use the  $>$  or  $>=$  operator are internally rewritten using the  $<$  or  $<=$  operator. Therefore, `lhs` returns the original right side. For example, `lhs(x > 0)` returns `0`.

### Examples

#### Find Left Side of Equation

Find the left side of the equation  $2y = x^2$  by using `lhs`.

First, declare the equation.

```
syms x y
eqn = 2*y == x^2
```

```
eqn = 2*y = x^2
```

Find the left side of `eqn` by using `lhs`.

```
lhsEqn = lhs(eqn)
```

```
lhsEqn = 2*y
```

*Copyright 2017-2025s The MathWorks, Inc.*

#### Find Left Side of Condition

Find the left side of the condition  $x + y < 1$  by using `lhs`.

First, declare the condition.

```
syms x y
cond = x + y < 1
```

```
cond = x + y < 1
```

Find the left side of `cond` by using `lhs`.

```
lhsCond = lhs(cond)
```

```
lhsCond = x + y
```

### Find Left Side of Equations in Array

For an array that contains equations and conditions, `lhs` returns an array of the left sides of those equations or conditions. The output array is the same size as the input array.

Find the left side of the equations and conditions in the vector `V`.

```
syms x y
V = [y^2 == x^2, x ~= 0, x*y >= 1]
```

```
V = (y^2 = x^2 x ≠ 0 1 ≤ x y)
```

```
lhsV = lhs(V)
```

```
lhsV = (y^2 x 1)
```

Because any condition using the `>=` operator is internally rewritten using the `<=` operator, the sides of the last condition in `V` are exchanged.

### Find Left Side of Equation Involving Symbolic Matrix Variables

Find the left side of a symbolic equation that involves symbolic matrix variables.

Create the symbolic matrix variables and the symbolic equation.

```
syms A [2 2] matrix
syms B [2 1] matrix
syms C [1 2] matrix
eqn = B*C == A*A - 2*A + eye(2)
```

```
eqn = B C = I2 - 2 A + A2
```

Find the left side of the equation by using `lhs`.

```
lhsEqn = lhs(eqn)
```

```
lhsEqn = B C
```

## Input Arguments

### eqn — Equation or condition

symbolic equation | symbolic condition | vector of symbolic equations or conditions | matrix of symbolic equations or conditions | multidimensional array of symbolic equations or conditions

Equation or condition, specified as a symbolic equation or condition, or a vector, matrix, or multidimensional array of symbolic equations or conditions.

Data Types: `sym` | `symfun` | `symmatrix` | `symfunmatrix`

## Version History

Introduced in R2017a

### **R2025a: Get left and right sides of symbolic equation that includes piecewise expressions**

You can use the `lhs` and `rhs` functions to get the left and right sides of a symbolic equation that includes piecewise expressions. For an example, see “Solve System of Equations with Piecewise Expressions” on page 6-1177.

### **R2022b: Find left side of symbolic equation or condition involving symbolic matrix variables and matrix functions**

The `lhs` function accepts a symbolic equation or condition involving symbolic matrix variables and matrix functions as an input argument. The input data type can be `symmatrix` or `symfunmatrix`. For an example, see “Find Left Side of Equation Involving Symbolic Matrix Variables” on page 6-975.

## See Also

`assume` | `children` | `rhs` | `subs`



# limit

Limit of symbolic expression

## Syntax

```
limit(f,var,a)
limit(f,a)
limit(f)

limit(f,var,a,"left")
limit(f,var,a,"right")
```

## Description

`limit(f,var,a)` returns the bidirectional limit on page 6-979 of the symbolic expression `f` when `var` approaches `a`.

`limit(f,a)` uses the default variable found by `symvar`.

`limit(f)` returns the limit at 0.

`limit(f,var,a,"left")` returns the left-side limit on page 6-979 of `f` as `var` approaches `a`.

`limit(f,var,a,"right")` returns the right-side limit on page 6-979 of `f` as `var` approaches `a`.

## Examples

### Limit of Symbolic Expression

Calculate the bidirectional limit of this symbolic expression as `x` approaches 0.

```
syms x h
f = sin(x)/x;
limit(f,x,0)
```

```
ans = 1
```

Calculate the limit of this expression as `h` approaches 0.

```
f = (sin(x+h)-sin(x))/h;
limit(f,h,0)
```

```
ans = cos(x)
```

### Right and Left Limits of Symbolic Expression

Calculate the right- and left-side limits of symbolic expressions.

```
syms x
f = 1/x;
limit(f,x,0,"right")
```

```
ans = ∞
```

```
limit(f,x,0,"left")
```

```
ans = - ∞
```

Because the limit from the left is not equal to the limit from the right, the two-sided limit does not exist. In this case, `limit` returns NaN (Not a Number).

```
limit(f,x,0)
```

```
ans = NaN
```

### Limit of Expressions in Symbolic Vector

Calculate the limit of expressions in a symbolic vector. `limit` acts element-wise on the vector.

```
syms x a
V = [(1+a/x)^x exp(-x)];
limit(V,x,Inf)
```

```
ans = (ea 0)
```

### Set Assumption When Calculating Limit

Calculate the right-side limit of  $x^n$  as  $x \rightarrow 0^+$  for real  $n$  and then for positive  $n$ .

Create the symbolic variables  $x$  and  $n$ . When you create symbolic variables, they are assumed to be complex by default. Set the assumption that  $n$  is real.

```
syms x n
assume(n,"real")
```

Find the limit using this assumption.

```
limit(x^n,x,0,"right")
```

```
ans =

$$\begin{cases} 1 & \text{if } n = 0 \\ 0 & \text{if } 0 < n \\ \infty & \text{if } n < 0 \end{cases}$$

```

Next, assume that  $n$  is positive. Find the limit using this assumption.

```
assume(n>0)
limit(x^n,x,0,"right")
```

```
ans = 0
```

## Input Arguments

### **f — Input**

symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic expression, function, vector, or matrix.

### **var — Independent variable**

x | symbolic variable

Independent variable, specified as a symbolic variable. If you do not specify var, then symvar determines the independent variable.

### **a — Limit point**

number | symbolic number | symbolic variable | symbolic expression

Limit point, specified as a number or a symbolic number, variable, or expression.

## More About

### **Bidirectional Limit**

$$L = \lim_{x \rightarrow a} f(x), x - a \in \mathbb{R} \setminus \{0\}.$$

### **Left-Side Limit**

$$L = \lim_{x \rightarrow a^-} f(x), x - a < 0.$$

### **Right-Side Limit**

$$L = \lim_{x \rightarrow a^+} f(x), x - a > 0.$$

## Version History

Introduced before R2006a

### **See Also**

diff | poles | taylor

## linsolve

Solve symbolic linear equations in matrix form

### Syntax

```
X = linsolve(A,B)
[X,R] = linsolve(A,B)
```

### Description

`X = linsolve(A,B)` solves the matrix equation  $AX = B$ , where  $A$  is a symbolic matrix and  $B$  is a symbolic column vector.

`[X,R] = linsolve(A,B)` also returns the reciprocal of the condition number of  $A$  if  $A$  is a square matrix. Otherwise, `linsolve` returns the rank of  $A$ .

### Examples

#### Solve Linear Equations in Matrix Form

Solve this system of linear equations in matrix form by using `linsolve`.

$$\begin{bmatrix} 2 & 1 & 1 \\ -1 & 1 & -1 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \\ -10 \end{bmatrix}$$

```
A = [ 2 1 1;
      -1 1 -1;
      1 2 3];
B = [2; 3; -10];
X = linsolve(A,B)
```

```
X = 3×1
```

```
 3
 1
-5
```

The solutions are  $x = 3$ ,  $y = 1$ , and  $z = -5$ .

#### Compute Condition Number of Square Matrix

Compute the reciprocal of the condition number of the square coefficient matrix by using two output arguments.

```
syms a x y z
A = [a 0 0; 0 a 0; 0 0 1];
```

```
B = [x; y; z];
[X, R] = linsolve(A, B)
```

X =

$$\begin{pmatrix} \frac{x}{a} \\ \frac{y}{a} \\ z \end{pmatrix}$$

R =

$$\frac{1}{\max(|a|, 1) \max\left(\frac{1}{|a|}, 1\right)}$$

### Compute Rank of Nonsquare Matrix

If the coefficient matrix is rectangular, `linsolve` returns the rank of the coefficient matrix as the second output argument. Show this behavior.

```
syms a b x y
A = [a 0 1; 1 b 0];
B = [x; y];
[X,R] = linsolve(A,B)
```

Warning: Solution is not unique because the system is rank-deficient.

X =

$$\begin{pmatrix} \frac{x}{a} \\ -\frac{x - a y}{a b} \\ 0 \end{pmatrix}$$

R = 2

## Input Arguments

### A — Coefficient matrix

symbolic matrix

Coefficient matrix, specified as a symbolic matrix.

### B — Right side of equations

symbolic vector | symbolic matrix

Right side of equations, specified as a symbolic vector or matrix.

## Output Arguments

### X — Solution

symbolic vector | symbolic matrix

Solution, returned as a symbolic vector or matrix.

**R — Reciprocal condition number or rank**

symbolic number | symbolic expression

Reciprocal condition number or rank, returned as a symbolic number or expression. If  $A$  is a square matrix, `linsolve` returns the condition number of  $A$ . Otherwise, `linsolve` returns the rank of  $A$ .

**More About****Matrix Representation of System of Linear Equations**

A system of linear equations is as follows.

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n &= b_2 \\ &\dots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n &= b_m \end{aligned}$$

This system can be represented as the matrix equation  $A \cdot \vec{x} = \vec{b}$ , where  $A$  is the coefficient matrix.

$$A = \begin{pmatrix} a_{11} & \dots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \dots & a_{mn} \end{pmatrix}$$

$\vec{b}$  is the vector containing the right sides of equations.

$$\vec{b} = \begin{pmatrix} b_1 \\ \vdots \\ b_m \end{pmatrix}$$

**Tips**

- If the solution is not unique, `linsolve` issues a warning, chooses one solution, and returns it.
- If the system does not have a solution, `linsolve` issues a warning and returns  $X$  with all elements set to `Inf`.
- Calling `linsolve` for numeric matrices that are not symbolic objects invokes the MATLAB `linsolve` function. This function accepts real arguments only. If your system of equations uses complex numbers, use `sym` to convert at least one matrix to a symbolic matrix, and then call `linsolve`.

**Version History**

Introduced in R2012b

**See Also**

`cond` | `dsolve` | `equationsToMatrix` | `inv` | `norm` | `odeToVectorField` | `rank` | `solve` | `symvar` | `vpasolve`

**Topics**

"Solve System of Algebraic Equations" on page 3-7

# log

Natural logarithm of entries of symbolic matrix

## Syntax

$Y = \log(X)$

## Description

$Y = \log(X)$  returns the natural logarithm of  $X$ .

## Examples

### Compute Natural Logarithm

Compute the natural logarithm of each entry of this symbolic matrix:

```
syms x
M = x*hilb(2);
log(M)

ans =
[ log(x), log(x/2)]
[ log(x/2), log(x/3)]
```

### Differentiate Symbolic Expression

Differentiate this symbolic expression:

```
syms x
diff(log(x^3), x)

ans =
3/x
```

## Input Arguments

### X — Input

number | vector | matrix | array | symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic array | symbolic function | symbolic matrix function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, scalar variable, matrix variable, array, function, matrix function, or expression.

## Version History

Introduced before R2006a

**R2022a: Compute natural logarithm of symbolic matrix functions**



The `log` function accepts an input argument of type `symfunmatrix`.

**R2021a: Compute natural logarithm of symbolic matrix variables**

The `log` function accepts an input argument of type `symmatrix`.

**See Also**

`log2` | `log10`

## log10

Log base 10 of symbolic input

### Syntax

`log10(x)`

### Description

`log10(x)` returns the logarithm to the base 10 of  $x$ . If  $x$  is an array, `log10` acts element-wise on  $x$ .

### Examples

#### Log Base 10 of Numeric and Symbolic Input

Compute the log base 10 of numeric input.

```
log10(20)
```

```
ans =  
    1.3010
```

Compute the log base 10 of symbolic input. The result is in terms of `log`.

```
syms x  
f = x^2;  
fLog10 = log10(f)
```

```
fLog10 =  
log(x^2)/log(10)
```

Convert symbolic output to double by substituting for  $x$  with a number using `subs`, and then using `double`.

```
fLog10 = subs(fLog10,x,5);      % x is 5  
fLog10 = double(fLog10)
```

```
fLog10 =  
    1.3979
```

### Input Arguments

#### **x** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## Version History

Introduced before R2006a

## See Also

log | log2

## log2

Base-2 logarithm of symbolic input

### Syntax

```
Y = log2(X)
[F,E] = log2(X)
```

### Description

$Y = \log_2(X)$  returns the logarithm to the base 2 of  $X$  such that  $2^Y = X$ . If  $X$  is an array, then  $\log_2$  acts element-wise on  $X$ .

$[F,E] = \log_2(X)$  returns arrays of mantissas and exponents,  $F$  and  $E$ , such that  $X = F \cdot 2^E$ . The values returned in  $F$  are in the range  $0.5 \leq \text{abs}(F) < 1$ . Any zeros in  $X$  return  $F = 0$  and  $E = 0$ .

### Examples

#### Base-2 Logarithm of Numeric and Symbolic Input

Compute the base-2 logarithm of a numeric input.

```
y = log2(4^(1/3))

y =
    0.6667
```

Compute the base-2 logarithm of a symbolic input. The result is in terms of the natural logarithm  $\log$  function.

```
syms x
ySym = log2(x^(1/3))

ySym =
    
$$\frac{\log(x^{1/3})}{\log(2)}$$

```

Substitute the symbolic variable  $x$  with a number by using `subs`. Simplify the result by using `simplify`.

```
yVal = subs(ySym,x,4)

yVal =
    
$$\frac{\log(4^{1/3})}{\log(2)}$$


simplify(yVal)

ans =
    
$$\frac{2}{3}$$

```

### Find Mantissa and Exponent of Base-2 Logarithm

Find the mantissa and exponent of a base-2 logarithm of an input  $X$ . The mantissa  $F$  and the exponent  $E$  satisfy the relation  $X = F \cdot 2^E$ .

Create a symbolic variable  $a$  and assume that it is real. Create a symbolic vector  $X$  that contains symbolic numbers and expressions. Find the exponent and mantissa for each element of  $X$ .

```
syms a real;
X = [1 0.5*2^a 5/7]
```

$$X = \begin{pmatrix} 1 & 2^a & \frac{5}{7} \end{pmatrix}$$

```
[F,E] = log2(X)
```

$$F = \begin{pmatrix} \frac{1}{2^{\left\lfloor \frac{\log\left(\frac{2^a}{2}\right)}{\log(2)} + 1 \right\rfloor}} & 2^a & \frac{5}{7} \end{pmatrix}$$

$$E = \begin{pmatrix} \left\lfloor \frac{\log\left(\frac{2^a}{2}\right)}{\log(2)} + 1 \right\rfloor & 1 & 0 \end{pmatrix}$$

The values returned in  $F$  have magnitudes in the range  $0.5 \leq \text{abs}(F) < 1$ .

Simplify the results using `simplify`.

```
F = simplify(F)
```

$$F = \begin{pmatrix} \frac{1}{2} & 2^{a - \lfloor a \rfloor - 1} & \frac{5}{7} \end{pmatrix}$$

```
E = simplify(E)
```

$$E = \begin{pmatrix} 1 & \lfloor a \rfloor & 0 \end{pmatrix}$$

### Input Arguments

#### **X** — Input array

symbolic number | symbolic array | symbolic variable | symbolic function | symbolic expression

Input array, specified as a symbolic number, array, variable, function, or expression.

- When computing the base-2 logarithms of complex elements in  $X$ , `log2` ignores their imaginary parts.

- For the syntax  $[F, E] = \log_2(X)$ , any zeros in  $X$  produce  $F = 0$  and  $E = 0$ . Input values of  $\text{Inf}$ ,  $-\text{Inf}$ , or  $\text{NaN}$  are returned unchanged in  $F$  with a corresponding exponent of  $E = 0$ .

## Output Arguments

### **Y — Base-2 logarithm values**

symbolic number | symbolic vector | symbolic matrix | symbolic array

Base-2 logarithm values, returned as a symbolic number, vector, matrix, or array of the same size as  $X$ .

### **F — Mantissa values**

symbolic number | symbolic vector | symbolic matrix | symbolic array

Mantissa values, returned as a symbolic scalar, vector, matrix, or array of the same size as  $X$ . The values in  $F$  and  $E$  satisfy  $X = F \cdot 2.^E$ .

### **E — Exponent values**

symbolic number | symbolic vector | symbolic matrix | symbolic array

Exponent values, returned as a symbolic scalar, vector, matrix, or array of the same size as  $X$ . The values in  $F$  and  $E$  satisfy  $X = F \cdot 2.^E$ .

## Tips

- For floating-point input, the syntax  $[F, E] = \log_2(X)$  corresponds to the ANSI<sup>®</sup> C function `frexp()` and the IEEE<sup>®</sup> standard function `logb()`. Any zeros in  $X$  produce  $F = 0$  and  $E = 0$ .

## Version History

Introduced before R2006a

## See Also

`log` | `log10` | `power`

# logical

Determine if symbolic equation, inequality, or condition is true

## Syntax

```
tf = logical(cond)
```

## Description

`tf = logical(cond)` checks if the conditions in `cond` are true and returns an array of logical values. To test conditions that require assumptions or mathematical transformations, use `isAlways` instead.

## Examples

### Test Symbolic Conditions

Test if  $3/5$  is less than  $2/3$ .

```
tf = logical(sym(3)/5 < sym(2)/3)

tf = logical
    1
```

To check if several conditions are true at the same time, combine them by using logical operators. For example, check if 1 is less than 2 and if  $\exp(\log(x)) == x$ . Note that when you define a condition that uses other functions, such as `exp` and `log`, these functions are evaluated when defining the condition.

```
syms x
cond1 = 1 < 2 & exp(log(x)) == x

cond1 = x = x

tf = logical(cond1)

tf = logical
    1
```

For multiple conditions, you can represent the conditions as a symbolic array.

```
cond2 = [1 < 2; exp(log(x)) == x]

cond2 =
    ( 1 )
    ( x = x )

tf = logical(cond2)

tf = 2×1 logical array
```

```
1  
1
```

### Test Condition Involving Different Data Types

Test an inequality that has a symbolic type on the left side and a numeric type on the right side. The expressions on both sides have compatible data types, and the inequality is true.

```
tf = logical(sym(11)/4 - sym(1)/2 > 2)  
  
tf = logical  
1
```

`logical` also checks the validity of equations and inequalities involving other functions, such as `int`. For example, the left side of this equation that contains an integral evaluates to `sym(1)`. Both sides of the equation are equal, and `logical` returns 1 (true).

```
syms x  
tf = logical(int(x,x,0,2) - 1 == 1)  
  
tf = logical  
1
```

### Test Equation with Simplification

`logical` does not simplify or apply mathematical transformations when checking if a condition is true. For example, check the equality of  $(x + 1)^2$  and  $x^2 + 2x + 1$  using `logical`.

```
syms x  
tf = logical((x+1)^2 == x^2+2*x+1)  
  
tf = logical  
0
```

Simplify the condition represented by the symbolic equation using `simplify`. The `simplify` function returns the symbolic logical constant `symtrue` because the equation is always true for all values of  $x$ .

```
cond = simplify((x+1)^2 == x^2+2*x+1)  
  
cond = symtrue
```

Using `logical` on `symtrue` converts the symbolic logical constant to logical 1 (true).

```
tf = logical(cond)  
  
tf = logical  
1
```



To check an equation that requires simplifications, you can use `isAlways` instead.

```
tf = isAlways((x+1)^2 == x^2+2*x+1)

tf = logical
    1
```

### Test Equation with Assumption

Check if an equation that involves a function, such as `sqrt`, is true. Without an additional assumption that `x` is nonnegative, `sqrt(x^2)` does not evaluate to `x`.

```
syms x
tf = logical(x == sqrt(x^2))

tf = logical
    0
```

Use `assume` to set an assumption that `x` is nonnegative. Now the expression `sqrt(x^2)` evaluates to `x`, and `logical` returns 1 because `x == x` is true.

```
assume(x >= 0)
tf = logical(x == sqrt(x^2))

tf = logical
    1
```

Note that the `logical` function itself ignores assumptions on symbolic variables.

```
syms x
assume(x == 5)
tf = logical(x == 5)

tf = logical
    0
```

To compare expressions taking into account assumptions on their variables, use `isAlways`.

```
tf = isAlways(x == 5)

tf = logical
    1
```

For further computations, clear the assumption on `x` by recreating it using `syms`.

```
syms x
```

### Compare `logical` and `isAlways`

The `logical` function does not simplify or apply mathematical transformations when checking if a condition is true. For example, `logical` does not recognize the mathematical equivalence of this equation.

```
syms x
tf = logical(sin(x)/cos(x) == tan(x))

tf = logical
0
```

`logical` also considers this inequality to be true.

```
tf = logical(sin(x)/cos(x) ~= tan(x))

tf = logical
1
```

To test the validity of equations and inequalities that require simplification or mathematical transformations, use `isAlways` instead. `isAlways` issues a warning when returning `false` for undecidable inputs.

```
tf = isAlways(sin(x)/cos(x) == tan(x))

tf = logical
1
```

```
tf = isAlways(sin(x)/cos(x) ~= tan(x))
```

Warning: Unable to prove 'sin(x)/cos(x) ~= tan(x)'.

```
tf = logical
0
```

## Input Arguments

### **cond** — Input

symbolic equation | symbolic inequality | symbolic array of equations or inequalities

Input, specified as a symbolic equation, inequality, or a symbolic array of equations or inequalities. You also can combine several conditions by using the logical operators `and`, `or`, `xor`, `not`, or their shortcuts.

## Tips

- For symbolic equations, `logical` returns logical 1 (`true`) only if the left and right sides are equal. Otherwise, it returns logical 0 (`false`).
- For symbolic inequalities constructed with `~=`, `logical` returns logical 0 (`false`) only if the left and right sides are equal. Otherwise, it returns logical 1 (`true`).
- For all other inequalities (constructed with `<`, `<=`, `>`, or `>=`), `logical` returns logical 1 if it can prove that the inequality is true and logical 0 if it can prove that the inequality is false. If `logical` cannot determine whether an inequality is true or false, it returns an error.
- `logical` does not simplify or mathematically transform a conditional statement. To compare a conditional statement applying mathematical transformations and simplifications, use `isAlways`.

- If you use `logical` to check a conditional statement that involves a symbolic type, then the data types of the compared expressions must be compatible. For example, `logical(1==sym(1))` returns `1` (`true`). If the expressions do not have compatible data types, then `logical` returns an error. For example, `syms f(x) g(y); tf = logical(f~=g)` returns an error.
- `logical` ignores assumptions on symbolic variables.

## Version History

Introduced in R2012a

### See Also

`assume` | `assumeAlso` | `assumptions` | `in` | `isAlways` | `isequal` | `isequaln` | `isfinite` | `isinf` | `isnan` | `sym` | `syms`

### Topics

“Use Assumptions on Symbolic Variables” on page 1-41

“Clear Assumptions and Reset the Symbolic Engine” on page 3-379

“Check Symbolic Equations, Inequalities, and Conditional Statements” on page 2-23

## logint

Logarithmic integral function

### Syntax

```
A = logint(x)
```

### Description

`A = logint(x)` evaluates the logarithmic integral function on page 6-998 (integral logarithm).

### Examples

#### Integral Logarithm for Numeric and Symbolic Arguments

`logint` returns floating-point or exact symbolic results depending on the arguments you use.

Compute integral logarithms for these numbers. Because these numbers are not symbolic objects, `logint` returns floating-point results.

```
A = logint([-1, 0, 1/4, 1/2, 1, 2, 10])
```

```
A =  
    0.0737 + 3.4227i    0.0000 + 0.0000i   -0.1187 + 0.0000i   -0.3787 + 0.0000i...  
   -Inf + 0.0000i    1.0452 + 0.0000i    6.1656 + 0.0000i
```

Compute integral logarithms for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `logint` returns unresolved symbolic calls.

```
symA = logint(sym([-1, 0, 1/4, 1/2, 1, 2, 10]))
```

```
symA =  
[ logint(-1), 0, logint(1/4), logint(1/2), -Inf, logint(2), logint(10)]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

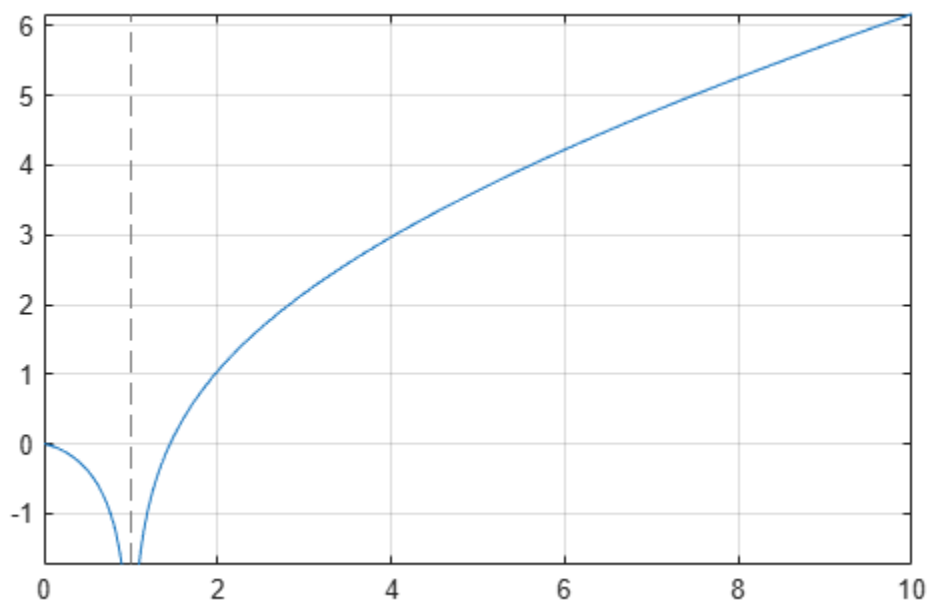
```
A = vpa(symA)
```

```
A =  
[ 0.07366791204642548599010096523015...  
 + 3.4227333787773627895923750617977i,...  
 0,...  
 -0.11866205644712310530509570647204,...  
 -0.37867104306108797672720718463656,...  
 -Inf,...  
 1.0451637801174927848445888891946,...  
 6.1655995047872979375229817526695]
```

#### Plot Integral Logarithm

Plot the integral logarithm function on the interval from 0 to 10.

```
syms x
fplot(logint(x),[0 10])
grid on
```



### Handle Expressions Containing Integral Logarithm

Many functions, such as `diff` and `limit`, can handle expressions containing `logint`.

Find the first and second derivatives of the integral logarithm:

```
syms x
dA = diff(logint(x), x)
dA = diff(logint(x), x, x)
```

```
dA =
1/log(x)
```

```
dA =
-1/(x*log(x)^2)
```

Find the right and left limits of this expression involving `logint`:

```
A_r = limit(exp(1/x)/logint(x + 1), x, 0, 'right')
```

```
A_r =
Inf
```

```
A_l = limit(exp(1/x)/logint(x + 1), x, 0, 'left')
```

```
A_l =
0
```

## Input Arguments

### **x — Input**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## More About

### **Logarithmic Integral Function**

The logarithmic integral function, also called the integral logarithm, is defined as follows:

$$\text{logint}(x) = \text{li}(x) = \int_0^x \frac{1}{\ln(t)} dt$$

## Tips

- `logint(sym(0))` returns 1.
- `logint(sym(1))` returns `-Inf`.
- `logint(z) = ei(log(z))` for all complex  $z$ .

## Version History

Introduced in R2014a

## References

- [1] Gautschi, W., and W. F. Cahill. "Exponential Integral and Related Functions." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

`coshint` | `cosint` | `ei` | `expint` | `int` | `log` | `sinhint` | `sinint` | `ssinint`

# logm

Matrix logarithm

## Syntax

$R = \text{logm}(A)$

## Description

$R = \text{logm}(A)$  computes the matrix logarithm of the square matrix  $A$ .

## Examples

### Matrix Logarithm

Compute the matrix logarithm for the 2-by-2 matrix.

```
syms x
A = [x 1; 0 -x];
logm(A)

ans =
[ log(x), log(x)/(2*x) - log(-x)/(2*x) ]
[      0,                  log(-x) ]
```

## Input Arguments

### A — Input matrix

square matrix

Input matrix, specified as a square symbolic matrix.

## Output Arguments

### R — Resulting matrix

symbolic matrix

Resulting function, returned as a symbolic matrix.

## Version History

Introduced in R2014b

## See Also

eig | expm | funm | jordan | sqrtm

# lt

Define less than relation

## Syntax

```
A < B  
lt(A,B)
```

## Description

$A < B$  creates a less than relation.

`lt(A,B)` is equivalent to  $A < B$ .

## Examples

### Set and Use Assumption Using Less

Use `assume` and the relational operator `<` to set the assumption that  $x$  is less than 3:

```
syms x  
assume(x < 3)
```

Solve this equation. The solver takes into account the assumption on variable  $x$ , and therefore returns these two solutions.

```
solve((x - 1)*(x - 2)*(x - 3)*(x - 4) == 0, x)
```

```
ans =  
 1  
 2
```

### Find Values that Satisfy Condition

Use the relational operator `<` to set this condition on variable  $x$ :

```
syms x  
cond = abs(sin(x)) + abs(cos(x)) < 6/5;
```

Use the `for` loop with step  $\pi/24$  to find angles from 0 to  $\pi$  that satisfy that condition:

```
for i = 0:sym(pi/24):sym(pi)  
    if subs(cond, x, i)  
        disp(i)  
    end  
end  
  
0  
pi/24  
(11*pi)/24  
pi/2  
(13*pi)/24
```



```
(23*pi)/24
pi
```

## Input Arguments

### A — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

### B — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## Tips

- Calling `<` or `lt` for non-symbolic `A` and `B` invokes the MATLAB `lt` function. This function returns a logical array with elements set to logical 1 (`true`) where `A` is less than `B`; otherwise, it returns logical 0 (`false`).
- If both `A` and `B` are arrays, then these arrays must have the same dimensions. `A < B` returns an array of relations `A(i,j,...) < B(i,j,...)`
- If one input is scalar and the other an array, then the scalar input is expanded into an array of the same dimensions as the other array. In other words, if `A` is a variable (for example, `x`), and `B` is an  $m$ -by- $n$  matrix, then `A` is expanded into  $m$ -by- $n$  matrix of elements, each set to `x`.
- The field of complex numbers is not an ordered field. MATLAB projects complex numbers in relations to a real axis. For example, `x < i` becomes `x < 0`, and `x < 3 + 2*i` becomes `x < 3`.

## Version History

Introduced in R2012a

## See Also

`eq` | `ge` | `gt` | `isAlways` | `le` | `ne`

## Topics

“Use Assumptions on Symbolic Variables” on page 1-41

# lu

LU factorization

## Syntax

```
[L,U] = lu(A)
[L,U,P] = lu(A)
[L,U,p] = lu(A,'vector')
[L,U,p,q] = lu(A,'vector')
[L,U,P,Q,R] = lu(A)
[L,U,p,q,R] = lu(A,'vector')
lu(A)
```

## Description

`[L,U] = lu(A)` returns an upper triangular matrix `U` and a matrix `L`, such that  $A = L*U$ . Here, `L` is a product of the inverse of the permutation matrix and a lower triangular matrix.

`[L,U,P] = lu(A)` returns an upper triangular matrix `U`, a lower triangular matrix `L`, and a permutation matrix `P`, such that  $P*A = L*U$ . The syntax `lu(A,'matrix')` is identical.

`[L,U,p] = lu(A,'vector')` returns the permutation information as a vector `p`, such that  $A(p,:) = L*U$ .

`[L,U,p,q] = lu(A,'vector')` returns the permutation information as two row vectors `p` and `q`, such that  $A(p,q) = L*U$ .

`[L,U,P,Q,R] = lu(A)` returns an upper triangular matrix `U`, a lower triangular matrix `L`, permutation matrices `P` and `Q`, and a scaling matrix `R`, such that  $P*(R\backslash A)*Q = L*U$ . The syntax `lu(A,'matrix')` is identical.

`[L,U,p,q,R] = lu(A,'vector')` returns the permutation information in two row vectors `p` and `q`, such that  $R(:,p)\backslash A(:,q) = L*U$ .

`lu(A)` returns the matrix that contains the strictly lower triangular matrix `L` (the matrix without its unit diagonal) and the upper triangular matrix `U` as submatrices. Thus, `lu(A)` returns the matrix  $U + L - \text{eye}(\text{size}(A))$ , where `L` and `U` are defined as `[L,U,P] = lu(A)`. The matrix `A` must be square.

## Examples

### Compute LU Factorization of Matrix

Compute the LU factorization of this matrix. Because the numbers are not symbolic objects, you get floating-point results.

```
M = [2 -3 -1; 1/2 1 -1; 0 1 -1];
[L, U] = lu(M)
```

```
L =
    1.0000    0    0
    0.2500    1.0000    0
    0    0.5714    1.0000
```

```
U =
    2.0000   -3.0000   -1.0000
    0    1.7500   -0.7500
    0    0   -0.5714
```

Now convert this matrix to a symbolic object, and compute the LU factorization.

```
M = sym(M);
[L, U] = lu(M)
```

```
L =
[ 1, 0, 0]
[ 1/4, 1, 0]
[ 0, 4/7, 1]
```

```
U =
[ 2, -3, -1]
[ 0, 7/4, -3/4]
[ 0, 0, -4/7]
```

### Compute Lower Triangular, Upper Triangular, and Permutation Matrices

Return the lower and upper triangular matrices and the permutation matrix by providing three output arguments.

```
syms a
[L, U, P] = lu(sym([0 0 a; a 2 3; 0 a 2]))
```

```
L =
[ 1, 0, 0]
[ 0, 1, 0]
[ 0, 0, 1]
```

```
U =
[ a, 2, 3]
[ 0, a, 2]
[ 0, 0, a]
```

```
P =
    0    1    0
    0    0    1
    1    0    0
```

### Return Permutation as Vector

Return the permutation information as a vector by using the 'vector' flag.

```
syms a
A = [0 0 a; a 2 3; 0 a 2];
[L, U, p] = lu(A, 'vector')
```

```
L =
[ 1, 0, 0]
```

```

[ 0, 1, 0]
[ 0, 0, 1]
U =
[ a, 2, 3]
[ 0, a, 2]
[ 0, 0, a]
p =
     2     3     1

```

Check that  $A(p,:) = L*U$  by using `isAlways`.

```
isAlways(A(p,:) == L*U)
```

```

ans =
    3×3 logical array
     1     1     1
     1     1     1
     1     1     1

```

Restore the permutation matrix  $P$  from the vector  $p$ .

```

P = zeros(3, 3);
for i = 1:3
    P(i, p(i)) = 1;
end
P

```

```

P =
     0     1     0
     0     0     1
     1     0     0

```

### Return Permutation as Two Vectors

Return the permutation information as two vectors  $p$  and  $q$ .

```

syms a
A = [a, 2, 3*a; 2*a, 3, 4*a; 4*a, 5, 6*a];
[L, U, p, q] = lu(A, 'vector')

```

```

L =
[ 1, 0, 0]
[ 2, 1, 0]
[ 4, 3, 1]
U =
[ a, 2, 3*a]
[ 0, -1, -2*a]
[ 0, 0, 0]
p =
     1     2     3
q =
     1     2     3

```

Check that  $A(p, q) = L*U$  by using `isAlways`.

```
isAlways(A(p, q) == L*U)
```

```
ans =
  3x3 logical array
   1     1     1
   1     1     1
   1     1     1
```

### Compute Scaling Matrix with Other Matrices

Return the lower and upper triangular matrices, permutation matrices, and scaling matrix.

```
syms a
A = [0, a; 1/a, 0; 0, 1/5; 0, -1];
[L, U, P, Q, R] = lu(A)
```

```
L =
[ 1,      0, 0, 0]
[ 0,      1, 0, 0]
[ 0, 1/(5*a), 1, 0]
[ 0,    -1/a, 0, 1]
```

```
U =
[ 1/a, 0]
[  0, a]
[  0, 0]
[  0, 0]
```

```
P =
   0     1     0     0
   1     0     0     0
   0     0     1     0
   0     0     0     1
```

```
Q =
   1     0
   0     1
```

```
R =
[ 1, 0, 0, 0]
[ 0, 1, 0, 0]
[ 0, 0, 1, 0]
[ 0, 0, 0, 1]
```

Check that  $P*(R\backslash A)*Q = L*U$  by using `isAlways`.

```
isAlways(P*(R\A)*Q == L*U)
```

```
ans =
  4x2 logical array
   1     1
   1     1
   1     1
   1     1
```

### Return Permutation a Two Vectors with Scaling Matrix

Return the permutation information as vectors `p` and `q` by using the `'vector'` flag. Also, compute the scaling matrix `R`.

```
syms a
A = [0, a; 1/a, 0; 0, 1/5; 0, -1];
[L, U, p, q, R] = lu(A, 'vector')
```

```
L =
[ 1,      0, 0, 0]
[ 0,      1, 0, 0]
[ 0, 1/(5*a), 1, 0]
[ 0,    -1/a, 0, 1]
U =
[ 1/a, 0]
[ 0, a]
[ 0, 0]
[ 0, 0]
p =
     2     1     3     4
q =
     1     2
R =
[ 1, 0, 0, 0]
[ 0, 1, 0, 0]
[ 0, 0, 1, 0]
[ 0, 0, 0, 1]
```

Check that  $R(:,p) \backslash A(:,q) = L*U$  by using `isAlways`.

```
isAlways(R(:,p)\A(:,q) == L*U)
```

```
ans =
    4x2 logical array
     1     1
     1     1
     1     1
     1     1
```

### Return Triangular Matrices as Submatrices

Return triangular matrices as submatrices by specifying one or no output arguments.

```
syms a
A = [0 0 a; a 2 3; 0 a 2];
lu(A)
```

```
ans =
[ a, 2, 3]
[ 0, a, 2]
[ 0, 0, a]
```

Verify that the resulting matrix is equal to  $U + L - \text{eye}(\text{size}(A))$ , where  $L$  and  $U$  are defined as  $[L, U, P] = \text{lu}(A)$ .

```
[L,U,P] = lu(A);
U + L - eye(size(A))
```

```
ans =
[ a, 2, 3]
```

```
[ 0, a, 2]
[ 0, 0, a]
```

## Input Arguments

### A — Input

numeric matrix | symbolic matrix

Input, specified as a numeric or symbolic matrix.

## More About

### LU Factorization of a Matrix

LU factorization expresses an  $m$ -by- $n$  matrix  $A$  as  $P^*A = L*U$ . Here,  $L$  is an  $m$ -by- $m$  lower triangular matrix,  $U$  is an  $m$ -by- $n$  upper triangular matrix, and  $P$  is a permutation matrix.

### Permutation Vector

The permutation vector  $p$  contains numbers corresponding to row exchanges in the matrix  $A$ . For an  $m$ -by- $m$  matrix,  $p$  represents the following permutation matrix with indices  $i$  and  $j$  ranging from 1 to  $m$ .

$$P_{ij} = \delta_{p_i, j} = \begin{cases} 1 & \text{if } j = p_i \\ 0 & \text{if } j \neq p_i \end{cases}$$

## Tips

- Calling `lu` for numeric arguments that are not symbolic objects invokes the MATLAB `lu` function.
- The `thresh` option supported by the MATLAB `lu` function does not affect symbolic inputs.
- If you use `'matrix'` instead of `'vector'`, then `lu` returns permutation matrices, as it does by default.
- $L$  and  $U$  are nonsingular if and only if  $A$  is nonsingular. `lu` also can compute the LU factorization of a singular matrix  $A$ . In this case,  $L$  or  $U$  is a singular matrix.
- Most algorithms for computing LU factorization are variants of Gaussian elimination.

## Version History

Introduced in R2013a

### See Also

`chol` | `eig` | `isAlways` | `lu` | `qr` | `svd` | `vpa`

## mapSymType

Apply function to symbolic subobjects of specific type

### Syntax

```
X = mapSymType(symObj,type,func)
X = mapSymType(symObj,funType,vars,func)
```

### Description

`X = mapSymType(symObj,type,func)` applies the function `func` to the symbolic subobjects of type `type` in the symbolic object `symObj`. The input `type` must be a case-sensitive string scalar or character vector, and it can include a logical expression.

- `func` must be a function handle or a symbolic function of type `'symfun'`.
- `func` must return a scalar that can be converted to a symbolic object using the `sym` or `str2sym` function.

If `symObj` contains several subexpressions of type `type`, then `mapSymType` applies the function `func` to the largest subexpression.

`X = mapSymType(symObj,funType,vars,func)` applies the function `func` to the unassigned symbolic functions that depend on the variables `vars` in the symbolic object `symObj`.

You can set the function type `funType` to `'symfunOf'` or `'symfunDependingOn'`. For example, `syms f(x); mapSymType(f,'symfunOf',x,@(u)cos(u))` returns `cos(f(x))`.

### Examples

#### Apply Function to Symbolic Numbers in Expression

Create a symbolic expression that contains symbolic numbers using `sym`.

```
expr = sym('2') + 1i*pi
expr = 2 +  $\pi i$ 
```

Construct a function handle that computes the square of a number.

```
sq = @(y) y^2;
```

Apply the function `sq` to the symbolic subobject of type `'integer'` in the expression `expr`.

```
X = mapSymType(expr,'integer',sq)
X = 4 +  $\pi i$ 
```

You can also apply an existing MATLAB® function, such as `exp`. Apply the `exp` function to the symbolic subobject of type `'complex'` in the expression `expr`.

```
X = mapSymType(expr,'complex',@exp)
```



$$X = \pi e^i + 2$$

### Apply Symbolic Function to Symbolic Subobjects in Equation

Apply a symbolic function to specific subobjects in a symbolic equation.

Create a symbolic equation.

```
syms x t
eq = 0.5*x + sin(x) == t/4

eq =
```

$$\frac{x}{2} + \sin(x) = \frac{t}{4}$$

Construct a symbolic function that multiplies an input by 2.

```
syms f(u)
f(u) = 2*u;
```

Apply the symbolic function f to the symbolic subobjects of type 'variable' in the equation eq.

```
X = mapSymType(eq, 'variable', f)

X =
```

$$x + \sin(2x) = \frac{t}{2}$$

The symbolic variables x and t in the equation are multiplied by 2.

You can also apply the same symbolic function that is created using symfun.

```
X = mapSymType(eq, 'variable', symfun(2*u,u))

X =
```

$$x + \sin(2x) = \frac{t}{2}$$

Now create an unassigned symbolic function. Apply the unassigned function to the symbolic subobjects of type 'sin' in the equation eq.

```
syms g(u)
X = mapSymType(eq, 'sin', g)

X =
```

$$\frac{x}{2} + g(\sin(x)) = \frac{t}{4}$$

### Apply Function to Largest Subexpression of Specific Type

Convert the largest symbolic subexpression of specific type in an expression.

Create a symbolic expression.

```
syms f(x) y
expr = sin(x) + f(x) - 2*y

expr =  $f(x) - 2y + \sin(x)$ 
```

Apply the `log` function to the symbolic subobject of type 'expression' in the expression `expr`.

```
X = mapSymType(expr, 'expression', @log)

X =  $\log(f(x) - 2y + \sin(x))$ 
```

When there are several subexpressions of type 'expression', `mapSymType` applies the `log` function to the largest subexpression.

### Symbolic Functions of Specific Variables

Convert unassigned symbolic functions with specific variable dependencies in an expression.

Create a symbolic expression.

```
syms f(x) g(t) h(x,t)
expr = f(x) + 2*g(t) + h(x,t)*sin(x)

expr =  $2g(t) + f(x) + \sin(x)h(x,t)$ 
```

Construct a function handle that converts an input to a symbolic variable with name 'z'.

```
func = @(obj) sym('z');
```

Apply the conversion function `func` to the unassigned symbolic functions in the expression `expr`.

Convert the functions that depend on the exact sequence of variables `[x t]` using 'symfunOf'.

```
X = mapSymType(expr, 'symfunOf', [x t], func)

X =  $2g(t) + f(x) + z\sin(x)$ 
```

Convert the functions that have a dependency on the variable `t` using 'symfunDependingOn'.

```
X = mapSymType(expr, 'symfunDependingOn', x, func)

X =  $z + 2g(t) + z\sin(x)$ 
```

### Remove Variable Dependency of Symbolic Functions

Remove variable dependency of unassigned symbolic functions in a symbolic array.

Create a symbolic array consisting of multiple equations.

```
syms f1(t) f2(t) g1(t) g2(t)
eq = [f1(t) + f2(t) == 0, f1(t) == 2*g1(t), g1(t) == diff(g2(t))]

eq =
```

$$\left( f_1(t) + f_2(t) = 0 \quad f_1(t) = 2 g_1(t) \quad g_1(t) = \frac{\partial}{\partial t} g_2(t) \right)$$

Apply the `symFunType` function to replace an unassigned symbolic function with a variable of the same name.

Find all functions that have a dependency on the variable `t` using `'symfunOf'` and convert them using `symFunType`.

```
X = mapSymType(eq, 'symfunOf', t, @symFunType)
```

$$X = (f_1 + f_2 = 0 \quad f_1 = 2 g_1 \quad g_1 = 0)$$

## Rewrite Symbolic Expression

Create a symbolic expression. Find its inverse Laplace transform.

```
syms s;
G = (s+10)/(s^2+2*s+4)/(s^2-4*s+1);
expr = ilaplace(G)
```

$$\text{expr} = \frac{19 e^{-t} \left( \cos(\sqrt{3} t) + \frac{\sqrt{3} \sin(\sqrt{3} t)}{19} \right)}{39} - \frac{19 e^{2t} \left( \cosh(\sqrt{3} t) - \frac{18 \sqrt{3} \sinh(\sqrt{3} t)}{19} \right)}{39}$$

The result is in terms of the `exp`, `sin`, `cos`, `sinh`, and `cosh` functions.

Rewrite `sinh` and `cosh` in the result as `exp`. Use `mapSymType` to apply the `rewrite` function to subexpressions that contain `sinh` or `cosh`.

```
expr = mapSymType(expr, "sinh|cosh", @(subexpr) rewrite(subexpr, "exp"))
```

$$\text{expr} = \frac{19 e^{-t} \left( \cos(\sqrt{3} t) + \frac{\sqrt{3} \sin(\sqrt{3} t)}{19} \right)}{39} - \frac{19 e^{2t} \left( \frac{e^{\sqrt{3} t}}{2} + \frac{e^{-\sqrt{3} t}}{2} - \frac{18 \sqrt{3} \left( \frac{e^{\sqrt{3} t}}{2} - \frac{e^{-\sqrt{3} t}}{2} \right)}{19} \right)}{39}$$

## Input Arguments

### symObj — Symbolic objects

symbolic expressions | symbolic functions | symbolic variables | symbolic numbers | symbolic units

Symbolic objects, specified as symbolic expressions, symbolic functions, symbolic variables, symbolic numbers, or symbolic units.

### type — Symbolic types

scalar string | character vector

Symbolic types, specified as a case-sensitive scalar string or character vector. The input type can contain a logical expression. The value options follow.

| Symbolic Type Category        | String Values                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| numbers                       | <ul style="list-style-type: none"> <li>'integer' — integer numbers</li> <li>'rational' — rational numbers</li> <li>'vpareal' — variable-precision floating-point real numbers</li> <li>'complex' — complex numbers</li> <li>'real' — real numbers, including 'integer', 'rational', and 'vpareal'</li> <li>'number' — numbers, including 'integer', 'rational', 'vpareal', 'complex', and 'real'</li> </ul>      |
| constants                     | 'constant' — symbolic mathematical constants, including 'number'                                                                                                                                                                                                                                                                                                                                                 |
| symbolic math functions       | 'vpa', 'sin', 'exp', and so on — symbolic math functions in symbolic expressions                                                                                                                                                                                                                                                                                                                                 |
| unassigned symbolic functions | <ul style="list-style-type: none"> <li>'F', 'g', and so on — function name of an unassigned symbolic function</li> <li>'symfun' — unassigned symbolic functions</li> </ul>                                                                                                                                                                                                                                       |
| arithmetic operators          | <ul style="list-style-type: none"> <li>'plus' — addition operator + and subtraction operator -</li> <li>'times' — multiplication operator * and division operator /</li> <li>'power' — power or exponentiation operator ^ and square root operator sqrt</li> </ul>                                                                                                                                               |
| variables                     | 'variable' — symbolic variables                                                                                                                                                                                                                                                                                                                                                                                  |
| units                         | 'unit' — symbolic units                                                                                                                                                                                                                                                                                                                                                                                          |
| expressions                   | 'expression' — symbolic expressions, including all of the preceding symbolic types                                                                                                                                                                                                                                                                                                                               |
| logical expressions           | <ul style="list-style-type: none"> <li>'or' — logical OR operator  </li> <li>'and' — logical AND operator &amp;</li> <li>'not' — logical NOT operator ~</li> <li>'xor' — logical exclusive-OR operator xor</li> <li>'logicalconstant' — symbolic logical constants symtrue and symfalse</li> <li>'logicalexpression' — logical expressions, including 'or', 'and', 'not', 'xor', symtrue and symfalse</li> </ul> |
| equations and inequalities    | <ul style="list-style-type: none"> <li>'eq' — equality operator ==</li> <li>'ne' — inequality operator ~=</li> <li>'lt' — less-than operator &lt; or greater-than operator &gt;</li> <li>'le' — less-than-or-equal-to operator &lt;= or greater-than-or-equal-to operator &gt;=</li> <li>'equation' — symbolic equations and inequalities, including 'eq', 'ne', 'lt', and 'le'</li> </ul>                       |
| unsupported symbolic types    | 'unsupported' — unsupported symbolic types                                                                                                                                                                                                                                                                                                                                                                       |

**func — Input function**

function handle | symbolic function

Input function, specified as a function handle or symbolic function. For more information about function handles and symbolic function, see “Create Function Handle” and `symfun`, respectively.

If `symObj` contains several subexpressions of type `type`, then `mapSymType` applies the function `func` to the largest subexpression (topmost matching node in a tree data structure).

**funType — Function type**

'symfunOf' | 'symfunDependingOn'

Function type, specified as 'symfunOf' or 'symfunDependingOn'.

- 'symfunOf' applies `func` to the unassigned symbolic functions that depend on the exact sequence of variables specified by the array `vars`. For example, `syms f(x,y); mapSymType(f,'symfunOf',[x y],@(g)g^2)` returns  $f(x,y)^2$ .
- 'symfunDependingOn' applies `func` to the unassigned symbolic functions that have a dependency on the variables specified by the array `vars`. For example, `syms f(x,y); mapSymType(f,'symfunDependingOn',x, @(g)g/2)` returns  $f(x,y)/2$ .

**vars — Input variables**

symbolic variables | symbolic array

Input variables, specified as symbolic variables or a symbolic array.

## Version History

Introduced in R2019a

**See Also**

`symFunType` | `isSymType` | `symType` | `sym` | `syms` | `findSymType` | `hasSymType` | `str2sym` | `symfun`

## massMatrixForm

Extract mass matrix and right side of semilinear system of differential algebraic equations

### Syntax

```
[M,F] = massMatrixForm(eqs,vars)
```

### Description

`[M,F] = massMatrixForm(eqs,vars)` returns the mass matrix  $M$  and the right side of equations  $F$  of a semilinear system of first-order differential algebraic equations (DAEs). Algebraic equations in `eqs` that do not contain any derivatives of the variables in `vars` correspond to empty rows of the mass matrix  $M$ .

The mass matrix  $M$  and the right side of equations  $F$  refer to this form.

$$M(t, x(t))\dot{x}(t) = F(t, x(t)).$$

### Examples

#### Convert DAE System to Mass Matrix Form

Convert a semilinear system of differential algebraic equations to mass matrix form.

Create the following system of differential algebraic equations. Here, the functions  $x(t)$  and  $y(t)$  represent state variables of the system. The system also contains symbolic parameters `alpha` and `beta`, and the function  $f(t, x, y)$ . Specify the equations and variables as two symbolic vectors: equations as a vector of symbolic equations, and variables as a vector of symbolic function calls.

```
syms x(t) y(t) f(t,x,y) alpha beta
eqs = [x(t) - diff(x(t),t) == alpha*f(t,x(t),y(t)); ...
       y(t) + diff(y(t),t) == beta*f(t,x(t),y(t))]
```

$$\text{eqs} = \begin{pmatrix} x(t) - \frac{\partial}{\partial t} x(t) = \alpha f(t, x(t), y(t)) \\ \frac{\partial}{\partial t} y(t) + y(t) = \beta f(t, x(t), y(t)) \end{pmatrix}$$

```
vars = [x(t),y(t)];
```

Find the mass matrix form of this system.

```
[M,F] = massMatrixForm(eqs,vars)
```

$$M = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}$$

```
F =
```

$$\begin{pmatrix} \alpha f(t, x(t), y(t)) - x(t) \\ \beta f(t, x(t), y(t)) - y(t) \end{pmatrix}$$

Solve this system using the numerical solver `ode15s`. Before you use `ode15s`, assign the following values to symbolic parameters of the system:  $\alpha = 0.01$ ,  $\beta = 0.02$ ,  $f(t, x, y) = x*y$ . Also, replace the state variables  $x(t)$  and  $y(t)$  by variables  $Y1$  and  $Y2$ , that are acceptable by `matlabFunction`.

`syms Y1 Y2`

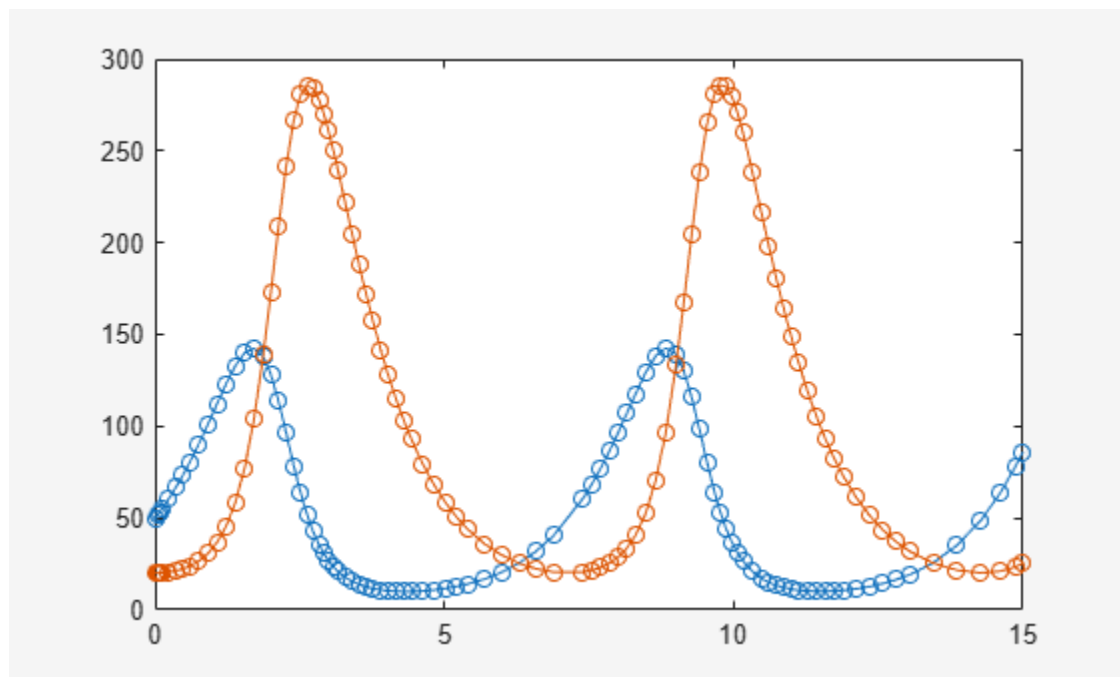
```
M = subs(M,[vars,alpha,beta,f],[Y1,Y2,0.01,0.02,@(t,x,y) x*y]);
F = subs(F,[vars,alpha,beta,f],[Y1,Y2,0.01,0.02,@(t,x,y) x*y]);
```

Create the following function handles `MM` and `FF`. You can use these function handles as input arguments for `odeset` and `ode15s`. These functions require state variables to be specified as column vectors.

```
MM = matlabFunction(M,Vars={t,[Y1; Y2]});
FF = matlabFunction(F,Vars={t,[Y1; Y2]});
```

Solve the system using `ode15s`. Specify the initial values for the variables  $Y1$  and  $Y2$  as 20 and 10. Plot the solutions over the time interval  $[0 \ 10]$ .

```
opt = odeset(Mass=MM);
ode15s(FF,[0 15],[50; 20],opt)
```



### Determine and Generate State Variables Programmatically

Create the following system of differential algebraic equations. Here, the symbolic functions  $x1(t)$ ,  $x2(t)$ , and  $x3(t)$  represent state variables of the system. The system also contains symbolic parameters  $a$  and  $b$ .

```
syms x1(t) x2(t) x3(t) a b
eqs = [a*diff(x1,t) == x1*x3- x2;
       diff(x2,t) == b*x1 - 1;
       0 == x1 + x2 + x3]
```

$$\text{eqs}(t) = \begin{pmatrix} a \frac{\partial}{\partial t} x_1(t) = x_1(t) x_3(t) - x_2(t) \\ \frac{\partial}{\partial t} x_2(t) = b x_1(t) - 1 \\ 0 = x_1(t) + x_2(t) + x_3(t) \end{pmatrix}$$

Find the state variables of the system of equations. Often, these variables are functions of time only. To programmatically determine the state variables, use `findSymType` by specifying `symfunOf` as `t`.

```
vars = findSymType(eqs,"symfunOf",t)
```

```
vars = (x1(t) x2(t) x3(t))
```

Find the mass matrix form of this system.

```
[M,F] = massMatrixForm(eqs,vars)
```

$$M = \begin{pmatrix} a & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

$$F = \begin{pmatrix} x_1(t) x_3(t) - x_2(t) \\ b x_1(t) - 1 \\ x_1(t) + x_2(t) + x_3(t) \end{pmatrix}$$

Next, assign the values of `a` and `b` as 2 and 1, respectively. Also, replace the state variables by variables `Y1`, `Y2`, and so on, that are acceptable by `matlabFunction`. You can use `sym` to programmatically generate the variables `Y1`, `Y2`, and so on.

```
numVars = numel(vars);
vecY = sym("Y",[1,numVars]);
M = subs(M,[a b vars],[2 1 vecY]);
F = subs(F,[a b vars],[2 1 vecY]);
```

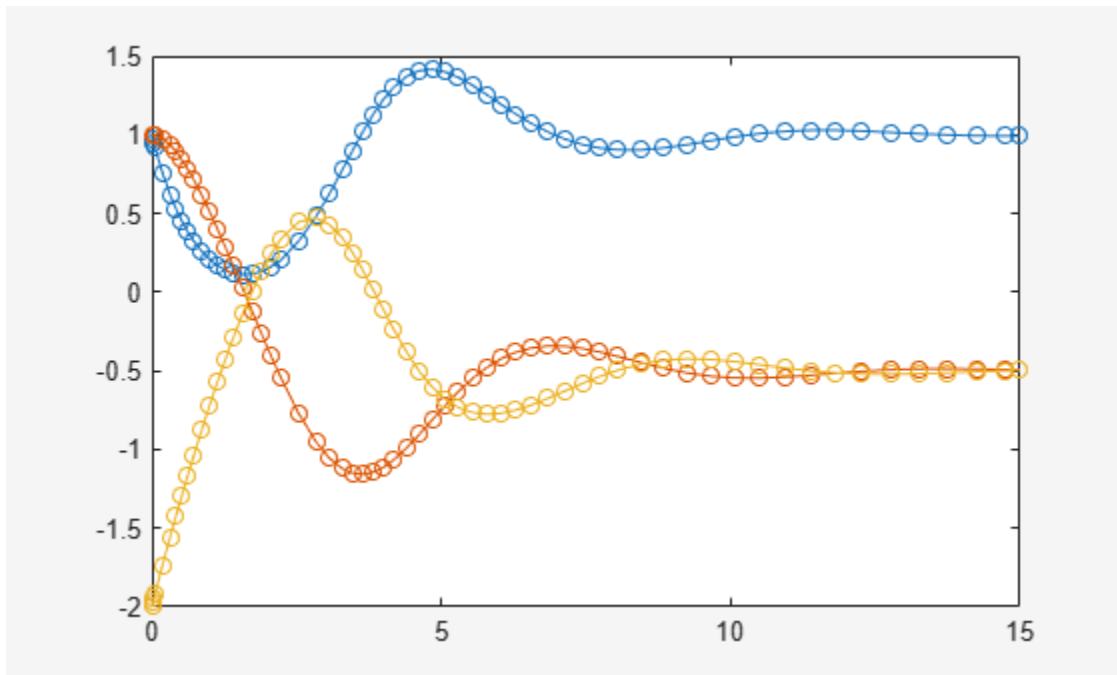
Create the following function handles `MM` and `FF`. You use these function handles as input arguments for `odeset` and `ode15s`. These functions require state variables to be specified as column vectors. Because the symbolic variables in `vecY` are complex, use the nonconjugate transpose to convert `vecY` to a column vector.

```
MM = matlabFunction(M,Vars={t,vecY.'});
FF = matlabFunction(F,Vars={t,vecY.'});
```

Solve the system using `ode15s`. Specify the initial values for the variables `Y1`, `Y2`, and `Y3` as 1, 1, and -2, respectively. Plot the solutions over the time interval `[0 15]`.

```
opt = odeset(Mass=MM);
ode15s(FF,[0 15],[1; 1; -2],opt)
```





## Input Arguments

### **eqs — System of semilinear first-order DAEs**

vector of symbolic equations | vector of symbolic expressions

System of semilinear first-order DAEs, specified as a vector of symbolic equations or expressions.

### **vars — State variables**

vector of symbolic functions | vector of symbolic function calls

State variables, specified as a vector of symbolic functions or function calls, such as  $x(t)$ .

Example:  $[x(t), y(t)]$  or  $[x(t); y(t)]$

## Output Arguments

### **M — Mass matrix**

symbolic matrix

Mass matrix of the system, returned as a symbolic matrix. The number of rows is the number of equations in `eqs`, and the number of columns is the number of variables in `vars`.

### **F — Right sides of equations**

symbolic column vector of symbolic expressions

Right sides of equations, returned as a column vector of symbolic expressions. The number of elements in this vector is equal to the number of equations in `eqs`.

## **Version History**

**Introduced in R2014b**

### **See Also**

`findDecoupledBlocks` | `daeFunction` | `decic` | `incidenceMatrix` | `isLowIndexDAE` | `matlabFunction` | `odeFunction` | `ode15s` | `odeset` | `reduceDAEIndex` | `reduceDAETooDE` | `reduceDifferentialOrder` | `reduceRedundancies`

### **Topics**

“Solve DAEs Using Mass Matrix Solvers” on page 3-82

# mathml

Generate MathML from symbolic expression

## Syntax

```
chr = mathml(f)
chr = mathml(f, Name, Value)
```

## Description

`chr = mathml(f)` returns the generated MathML from the symbolic expression `f`.

`chr = mathml(f, Name, Value)` uses additional options specified by one or more name-value pair arguments. For example, generate MathML for inline display by specifying `DisplayInline` as `true`.

## Examples

### MathML from Symbolic Expression

Generate MathML from a symbolic expression.

```
syms x
f = 1/exp(x^2);
chr = mathml(f)

chr =
'<math xmlns='http://www.w3.org/1998/Math/MathML' display='block'>
  <msup>
    <mo>&ee;</mo>
    <mrow>
      <mo>-</mo>
      <msup>
        <mi>x</mi>
        <mn>2</mn>
      </msup>
    </mrow>
  </msup>
</math>
'
```

### Inline Display of MathML

Generate MathML for inline display by specifying `DisplayInline` as `true`.

```
syms x
f = 1/exp(x^2);
chr = mathml(f, 'DisplayInline', true)

chr =
'<math xmlns='http://www.w3.org/1998/Math/MathML'>
```

```

<msup>
  <mo>&ee;</mo>
  <mrow>
    <mo>-</mo>
    <msup>
      <mi>x</mi>
      <mn>2</mn>
    </msup>
  </mrow>
</msup>
</math>

```

### Add Tooltips for Symbols to Generated MathML

Use MathML tooltips for units and some special functions to provide more information. Generate tooltips by specifying `Tooltips` as `true`.

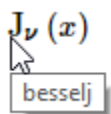
```

syms nu x
f = besselj(nu,x);
chr = mathml(f,'Tooltips',true)

chr =
'<math xmlns='http://www.w3.org/1998/Math/MathML' display='block'>
  <mrow>
    <msub>
      <maction actiontype='tooltip'>
        <mo>J</mo>
        <mtext>besselj</mtext>
      </maction>
      <mi>&nu;</mi>
    </msub>
    <mrow>
      <mo form='prefix'>(</mo>
      <mi>x</mi>
      <mo form='postfix'>)</mo>
    </mrow>
  </mrow>
</math>

```

When you use MathML in a web page, then hovering your mouse on  $J$  displays a tooltip containing *besselj*.



### Modify Generated MathML with Symbolic Settings

Modify generated MathML by using the `sympref` function.

Generate the MathML form of the expression  $\pi$  with the default symbolic setting.

```
sympref('default');
chr = mathml(sym(pi))

chr =
'<math xmlns='http://www.w3.org/1998/Math/MathML' display='block'>
  <mi>&pi;</mi>
</math>
'
```

Set 'FloatingPointOutput' to true to return symbolic output in floating-point format. Generate the MathML form of  $\pi$  in floating-point format.

```
sympref('FloatingPointOutput',true);
chr = mathml(sym(pi))

chr =
'<math xmlns='http://www.w3.org/1998/Math/MathML' display='block'>
  <mn>3.1416</mn>
</math>
'
```

Now change the output order of a symbolic polynomial. Create a symbolic polynomial and set 'PolynomialDisplayStyle' to 'ascend'. Generate MathML form of the polynomial sorted in ascending order.

```
syms x;
poly = x^2 - 2*x + 1;
sympref('PolynomialDisplayStyle','ascend');
chr = mathml(poly)

chr =
'<math xmlns='http://www.w3.org/1998/Math/MathML' display='block'>
  <mrow>
    <mn>1</mn>
    <mo>-</mo>
    <mrow>
      <mn>2</mn>
      <mo form='infix'>&InvisibleTimes;</mo>
      <mi>x</mi>
    </mrow>
    <mo>+</mo>
    <msup>
      <mi>x</mi>
      <mn>2</mn>
    </msup>
  </mrow>
</math>
'
```

The settings that you set using `sympref` persist through your current and future MATLAB sessions. Restore the default values by specifying the 'default' option.

```
sympref('default');
```

## Input Arguments

### f — Input

symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a symbolic number, variable, array, function, or expression.

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `mathml(f, 'Tooltips', true)`

### **DisplayInline** — Inline MathML display

`false` (default) | `true`

Inline MathML display, specified as the comma-separated pair consisting of `'DisplayInline'` and either `true` or `false` (default).

### **Tooltips** — tooltips

`false` (default) | `true`

Tooltips in MathML output, specified as the comma-separated pair consisting of `'Tooltips'` and either `true` or `false` (default). `mathml` adds tooltips for units and some special functions.

## Version History

**Introduced in R2018b**

### **See Also**

`latex` | `texlabel` | `ccode` | `fortran` | `sympref`

### **Topics**

“Copy and Paste Symbolic Output in Live Editor” on page 2-18

# matlabFunction

Convert symbolic expression to function handle or file

## Syntax

```
ht = matlabFunction(f)
ht = matlabFunction(f1,...,fN)
ht = matlabFunction( ___,Name,Value)
```

## Description

`ht = matlabFunction(f)` converts the symbolic expression or function `f` to a MATLAB function with handle `ht`. If there is an equivalent MATLAB function operating on the `double` data type for the symbolic expression or function, then the converted function can be used without Symbolic Math Toolbox.

`ht = matlabFunction(f1,...,fN)` converts `f1,...,fN` to a MATLAB function with `N` outputs. The function handle is `ht`. Each element of `f1,...,fN` can be a symbolic expression, function, or a vector or matrix of symbolic expressions or functions.

`ht = matlabFunction( ___,Name,Value)` specifies options using one or more name-value arguments in addition to any of the input argument combinations in the previous syntaxes.

For example, you can specify the `File` name-value argument to write the generated MATLAB function to a file. You can also specify the `Vars` name-value argument to generate a MATLAB function with input arguments that are a combination of scalar and vector variables.

## Examples

### Convert Symbolic Expression to Anonymous Function

Convert the symbolic expression `r` to a MATLAB function with the handle `ht`. The converted function can be used without Symbolic Math Toolbox.

```
syms x y
r = sqrt(x^2 + y^2);
ht = matlabFunction(r)

ht =
function_handle with value:
 @(x,y)sqrt(x.^2+y.^2)
```

Convert multiple symbolic expressions using comma-separated input.

```
ht = matlabFunction(r,r^2)

ht =
function_handle with value:
 @(x,y)deal(sqrt(x.^2+y.^2),x.^2+y.^2)
```

### Convert Symbolic Function to Anonymous Function

Create a symbolic function and convert it to a MATLAB function with the handle `ht`.

```
syms x y
f(x,y) = x^3 + y^3;
ht = matlabFunction(f)

ht =
    function_handle with value:
        @(x,y)x.^3+y.^3
```

### Write MATLAB Function to File with Comments

Write the generated MATLAB function to a file by specifying the `File` name-value argument. Existing files are overwritten. When writing to a file, `matlabFunction` optimizes the code using intermediate variables named `t0`, `t1`, and so on. Include comments in the file by using the `Comments` name-value argument.

Write the MATLAB function generated from `f` to the file `myfile`.

```
syms x
f = x^2 + log(x^2);
matlabFunction(f,"File","myfile");

function f = myfile(x)
%MYFILE
%    F = MYFILE(X)

%    This function was generated by the Symbolic Math Toolbox version 8.4.
%    01-Sep-2019 00:00:00

t2 = x.^2;
f = t2+log(t2);
```

Include the comment `Version: 1.1` in the file.

```
matlabFunction(f,"File","myfile","Comments","Version: 1.1");

function f = myfile(x)
...
%Version: 1.1
t2 = x.^2;
...
```

### Specify Input Arguments for Generated Function

When converting a symbolic expression to a MATLAB function, you can specify the order of the input arguments of the resulting function. You can also specify that some input arguments are vectors instead of scalar variables.

Create a symbolic expression.

```
syms x y z
r = x + y/2 + z/3;
```



Convert  $r$  to a MATLAB function and write this function to the file `myfile`. By default, `matlabFunction` uses alphabetical order for the input arguments when converting symbolic expressions that contain only lowercase letters for the variable names. The generated input arguments are scalar variables  $x$ ,  $y$ , and  $z$ .

```
matlabFunction(r,"File","myfile");
```

```
function r = myfile(x,y,z)
%MYFILE
%    R = MYFILE(X,Y,Z)
r = x+y./2.0+z./3.0;
```

Specify the `Vars` name-value argument as the vector `[y z x]` to modify the order of the input arguments for the generated MATLAB function. The generated input arguments are scalar variables  $y$ ,  $z$ , and  $x$ .

```
matlabFunction(r,"File","myfile","Vars",[y z x]);
```

```
function r = myfile(y,z,x)
%MYFILE
%    R = MYFILE(Y,Z,X)
r = x+y./2.0+z./3.0;
```

Now, convert a symbolic expression  $v$  to a MATLAB function whose input arguments are a scalar and a vector. Specify the `Vars` name-value argument as the cell array `{t,[x y z]}`. The generated input arguments are a scalar variable  $t$  and a 1-by-3 vector variable `in2`.

```
syms x y z t
v = (x + y/2 + z/3)*exp(-t);
matlabFunction(v,"File","myfile","Vars",{t,[x y z]});
```

```
function v = myfile(t,in2)
%MYFILE
%    R = MYFILE(T,IN2)
x = in2(:,1);
y = in2(:,2);
z = in2(:,3);
v = exp(-t).*(x+y./2.0+z./3.0);
```

### Generate Function with Vector Input Arguments

To generate a MATLAB function with input arguments that are vector variables, specify the `Vars` name-value argument as a cell array.

Create a symbolic expression that finds the dot product of two 1-by-3 vectors.

```
syms x y [1 3] real
f = dot(x,y);
```

Convert the expression  $f$  to a MATLAB function. Specify `Vars` as a cell array `{x,y}`. The generated input arguments are two 1-by-3 vector variables, `in1` and `in2`, that correspond to  $x$  and  $y$ , respectively.

```
matlabFunction(f,"File","myfile","Vars",{x,y});
```

```
function f = myfile(in1,in2)
%MYFILE
```

```
% F = MYFILE(IN1,IN2)
x1 = in1(:,1);
x2 = in1(:,2);
x3 = in1(:,3);
y1 = in2(:,1);
y2 = in2(:,2);
y3 = in2(:,3);
f = x1.*y1+x2.*y2+x3.*y3;
```

Now create a symbolic function that is a function of four variables.

```
syms g(x,y,z,t)
g(x,y,z,t) = x^2 + y^2 + z^2 - t^2;
```

Convert the function `g` to a MATLAB function. To specify the generated input arguments as a 4-by-1 column vector from the input variables of `g`, specify `Vars` as a cell array that contains the 4-by-1 vector. You can use `argnames` to obtain the input variables `x`, `y`, `z`, and `t` from `g`.

```
matlabFunction(g,"File","myfunction","Vars",{argnames(g).'});

function g = myfunction(in1)
%MYFUNCTION
% G = MYFUNCTION(IN1)
x = in1(1,:);
y = in1(2,:);
z = in1(3,:);
t = in1(4,:);
g = -t.^2+x.^2+y.^2+z.^2;
```

### Disable Code Optimization

When you convert a symbolic expression to a MATLAB function and write the resulting function to a file, `matlabFunction` optimizes the code by default. This approach can help simplify and speed up further computations that use the file. However, generating the optimized code from some symbolic expressions and functions can be time-consuming. Use the `Optimize` name-value argument to disable code optimization.

Create a symbolic expression.

```
syms x
r = x^2*(x^2 + 1);
```

Convert `r` to a MATLAB function and write the function to the file `myfile`. By default, `matlabFunction` creates a file containing the optimized code.

```
f = matlabFunction(r,"File","myfile");

function r = myfile(x)
%MYFILE
% R = MYFILE(X)
t2 = x.^2;
r = t2.*(t2+1.0);
```

Disable the code optimization by setting `Optimize` to `false`.

```
f = matlabFunction(r,"File","myfile","Optimize",false);
```

```
function r = myfile(x)
%MYFILE
%    R = MYFILE(X)
r = x.^2.*(x.^2+1.0);
```

## Generate Sparse Matrices

When you convert a symbolic matrix to a MATLAB function, `matlabFunction` represents the symbolic matrix with a dense matrix by default. If most of the elements of the input symbolic matrix are zeros, the more efficient approach is to represent the symbolic matrix with a sparse matrix.

Create a 3-by-3 symbolic diagonal matrix.

```
syms x
A = diag(x*ones(1,3))

A =
[ x, 0, 0]
[ 0, x, 0]
[ 0, 0, x]
```

Convert `A` to a MATLAB function representing a numeric matrix, and write the result to the file `myfile1`. By default, the generated MATLAB function creates a dense numeric matrix specifying each element of the matrix, including all zero elements.

```
f1 = matlabFunction(A,"File","myfile1");

function A = myfile1(x)
%MYFILE1
%    A = MYFILE1(X)
A = reshape([x,0.0,0.0,0.0,x,0.0,0.0,0.0,x],[3,3]);
```

Convert `A` to a MATLAB function by setting `Sparse` name-value argument to `true`. Now, the generated MATLAB function creates a sparse numeric matrix specifying only nonzero elements and assuming that all other elements are zeros.

```
f2 = matlabFunction(A,"File","myfile2","Sparse",true);

function A = myfile2(x)
%MYFILE2
%    A = MYFILE2(X)
A = sparse([1,2,3],[1,2,3],[x,x,x],3,3);
```

## Specify Output Variables

When converting a symbolic expression to a MATLAB function, you can specify the names of the output variables. Note that `matlabFunction` without the `File` name-value argument (or with a file path specified as an empty character vector) creates a function handle and ignores the `Outputs` name-value argument.

Create symbolic expressions `r` and `q`.

```
syms x y z
r = x^2 + y^2 + z^2;
q = x^2 - y^2 - z^2;
```

Convert  $r$  and  $q$  to a single MATLAB function and write the resulting function to a file `myfile`, which returns a vector of two elements, `name1` and `name2`.

```
f = matlabFunction(r,q,"File","myfile", ...
    "Outputs",{ 'name1', 'name2' });
```

```
function [name1,name2] = myfile(x,y,z)
%MYFILE
% [NAME1,NAME2] = MYFILE(X,Y,Z)
t2 = x.^2;
t3 = y.^2;
t4 = z.^2;
name1 = t2+t3+t4;
if nargin > 1
    name2 = t2-t3-t4;
end
```

### Speed Up Evaluation by Converting Symbolic Function to Anonymous Function

You can speed up the evaluation of a symbolic function at given coordinates by converting the symbolic function to an anonymous MATLAB function. Use `matlabFunction` to perform the conversion. Evaluation of a symbolic function returns symbolic numbers that are exact, while evaluation of a MATLAB function returns double-precision numbers.

Create a symbolic function  $f(x, y, z)$  that is a function of  $x$ ,  $y$ , and  $z$ .

```
syms f(x,y,z)
f(x,y,z) = y*z*sin(x) + x*sin(z)*cos(y) - z^3;
```

Create 3-D grid coordinates at the specified intervals.

```
[xDouble,yDouble,zDouble] = meshgrid(1:20,1:50,1:20);
```

Evaluate the symbolic function at these coordinates. Measure the elapsed time using a pair of `tic` and `toc` calls.

```
tic
fResult = f(xDouble,yDouble,zDouble);
toc
```

Elapsed time is 3.670466 seconds.

Here, evaluation is slow, but it returns symbolic numbers that are exact. Show a sample of the results.

```
fResult(1:2,1:2,20)
```

```
ans =
(20 sin(1) + cos(1) sin(20) - 8000 20 sin(2) + 2 cos(1) sin(20) - 8000)
(40 sin(1) + cos(2) sin(20) - 8000 40 sin(2) + 2 cos(2) sin(20) - 8000)
```

To speed up the evaluation of the function, convert the symbolic function to a MATLAB function using `matlabFunction`. Evaluate the MATLAB function at the same coordinates.

```
f1 = matlabFunction(f);
tic
fResult = f1(xDouble,yDouble,zDouble);
toc
```

Elapsed time is 0.058740 seconds.

Here, evaluation is faster. The evaluated MATLAB function returns double-precision numbers. Show a sample of the results.

```
fResult(1:2,1:2,20)
```

```
ans = 2×2
103 ×
    -7.9827    -7.9808
    -7.9667    -7.9644
```

## Input Arguments

### **f** — Symbolic input to be converted to MATLAB function

symbolic expression | symbolic function | symbolic vector | symbolic matrix

Symbolic input to be converted to a MATLAB function, specified as a symbolic expression, function, vector, or matrix. When converting sparse symbolic vectors or matrices, specify the `Sparse` name-value argument as `true`.

### **f1, ..., fN** — Symbolic input to be converted to MATLAB function with N outputs

symbolic expressions | symbolic functions | symbolic vectors | symbolic matrices

Symbolic input to be converted to a MATLAB function with N outputs, specified as symbolic expressions, functions, vectors, or matrices, separated by commas.

`matlabFunction` does not create a separate output argument for each element of a symbolic vector or matrix. For example, `ht = matlabFunction([x + 1, y + 1])` creates a MATLAB function with one output argument, while `ht = matlabFunction(x + 1, y + 1)` creates a MATLAB function with two output arguments.

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `matlabFunction(f,File="myfile",Optimize=false)`

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `matlabFunction(f,"File","myfile","Optimize",false)`

### **Comments** — Comments to include in file header

character vector | cell array of character vectors | string array

Comments to include in the file header, specified as a character vector, cell array of character vectors, or string array.

**File — Path to file containing generated MATLAB function**

character vector | string scalar

Path to the file containing the generated MATLAB function, specified as a character vector or string scalar. The generated function accepts arguments of type `double` and can be used without Symbolic Math Toolbox. If `File` is empty, `matlabFunction` generates an anonymous function. If `File` does not end in `.m`, the function appends `.m`.

When writing to a file, `matlabFunction` optimizes the code using intermediate variables named `t0`, `t1`, and so on. To disable code optimization, use the `Optimize` name-value argument.

See “Write MATLAB Function to File with Comments” on page 6-1024.

**Optimize — Whether to optimize code written to function file**

`true` or `1` (default) | `false` or `0`

Whether to optimize code written to a function file, specified as logical `1` (`true`) or `0` (`false`).

When writing to a file, `matlabFunction` optimizes the code by default using intermediate variables named `t0`, `t1`, and so on.

Using `matlabFunction` without the `File` name-value argument (or with a file path specified as an empty character vector) creates a function handle. In this case, the code is not optimized by default. If you try to enforce code optimization by setting `Optimize` to `true`, then `matlabFunction` throws an error.

See “Disable Code Optimization” on page 6-1026.

**Sparse — Whether to use sparse matrix generation**

`false` or `0` (default) | `true` or `1`

Whether to use sparse matrix generation, specified as logical `0` (`false`) or `1` (`true`). By default, `Sparse` is `false`, so the generated MATLAB function represents symbolic matrices with dense numeric matrices. When you specify `Sparse` as `true`, the generated MATLAB function represents symbolic matrices with sparse numeric matrices. Specify `Sparse` as `true` when you convert symbolic matrices containing many zero elements. Often, operations on sparse matrices are more efficient than the same operations on dense matrices.

See “Generate Sparse Matrices” on page 6-1027.

**Vars — Order of input variables or vectors in generated MATLAB function**

character vector | vector of symbolic variables | one-dimensional cell array of character vectors | one-dimensional cell array of symbolic variables | one-dimensional cell array of vectors of symbolic variables

Order of input variables or vectors in a generated MATLAB function, specified as a character vector, a vector of symbolic variables, or a one-dimensional cell array of character vectors, symbolic variables, or vectors of symbolic variables.

The number of specified input variables must equal or exceed the number of symbolic variables in `f`. Do not use the same names for the input variables specified by `Vars` and the output variables specified by `Outputs`.

By default, when you convert symbolic expressions that contain only lowercase letters for the variable names, the order of the input variables is alphabetical. When you convert symbolic functions, their input arguments appear in front of other variables, and all other variables are ordered alphabetically.

Specify `Vars` as a vector to generate a MATLAB function with input arguments that are scalar variables. Specify `Vars` as a cell array to generate a MATLAB function with input arguments that are a combination of scalar and vector variables.

See “Specify Input Arguments for Generated Function” on page 6-1024.

### **Outputs — Names of output variables**

one-dimensional cell array of character vectors

Names of output variables, specified as a one-dimensional cell array of character vectors.

If you do not specify the output variable names, then they coincide with the names you use when calling `matlabFunction`. If you call `matlabFunction` using an expression instead of individual variables, the default names of output variables consist of the word `out` followed by a number, for example, `out3`.

Do not use the same names for the input variables specified by `Vars` and the output variables specified by `Outputs`.

`matlabFunction` without the `File` name-value argument (or with a file path specified as an empty character vector) creates a function handle. In this case, `matlabFunction` ignores the `Outputs` name-value argument.

See “Specify Output Variables” on page 6-1027.

## **Output Arguments**

### **ht — Function handle**

MATLAB function handle

MATLAB function handle. You can use this function handle to return numerical results to be used without Symbolic Math Toolbox.

## **Limitations**

- Some symbolic functions that have no corresponding MATLAB functions operating on the `double` data type, such as `simplify` and `solve`, are kept as symbolic functions in the converted MATLAB function handle or file. The converted file that consists of these functions cannot be deployed using MATLAB Coder or MATLAB Compiler. You need to create your own functions with the `double` data type to replace these symbolic functions. If you are interested in a symbolic function that cannot be deployed, please contact MathWorks Technical Support.

## **Tips**

- When you use the `File` name-value argument, use `rehash` to make the generated function available immediately. `rehash` updates the MATLAB list of known files for directories on the search path.
- If the `File` name-value argument is empty, then MATLAB function returns an anonymous function.

- If a symbolic expression involves a conditional statement, such as a piecewise condition, `matlabFunction` can convert the expression to a MATLAB file, but not an anonymous function. The function in the generated file can accept only scalar inputs. For example, convert a piecewise condition to a MATLAB file named `piecewiseFunc`.

```
syms x
p = piecewise(x<0, x^2-8, x>=0, -x)
matlabFunction(p,"File","piecewiseFunc")
y1 = piecewiseFunc(0)
y2 = piecewiseFunc(-2)
```

- Use `matlabFunction` to convert one or more symbolic expressions to a MATLAB function and write the resulting function to an M-file. You can then use the generated M-file to create standalone applications and web apps using MATLAB Compiler. For example, see “Deploy Generated MATLAB Functions from Symbolic Expressions with MATLAB Compiler” on page 5-10.

You can also use the generated M-file to create C or C++ code using the MATLAB Coder app. For example, see “Generate C Code from Symbolic Expressions Using the MATLAB Coder App” on page 5-15.

- To generate a MATLAB function with input arguments that are a combination of scalar and vector variables, specify the `Vars` name-value argument as a cell array. For examples, see “Specify Input Arguments for Generated Function” on page 6-1024 and “Generate Function with Vector Input Arguments” on page 6-1025.

## Version History

Introduced in R2008b

### See Also

`ccode` | `daeFunction` | `fortran` | `rehash` | `matlabFunctionBlock` | `odeFunction` | `simscapeEquation` | `subs` | `sym2poly`

### Topics

“Generate MATLAB Functions from Symbolic Expressions” on page 5-3

“Generate C Code from Symbolic Expressions Using the MATLAB Coder App” on page 5-15

“Deploy Generated MATLAB Functions from Symbolic Expressions with MATLAB Compiler” on page 5-10

“Using Symbolic Mathematics with Optimization Toolbox Solvers” on page 5-43



# matlabFunctionBlock

Convert symbolic expression to MATLAB function block

## Syntax

```
matlabFunctionBlock(block,f)
matlabFunctionBlock(block,f1,...,fN)
matlabFunctionBlock( ___,Name,Value)
```

## Description

`matlabFunctionBlock(block,f)` converts `f` to a MATLAB function block that you can use in Simulink models. Here, `f` can be a symbolic expression, function, or a vector of symbolic expressions or functions.

`block` specifies the name of the block that you create or modify.

`matlabFunctionBlock(block,f1,...,fN)` converts symbolic expressions or functions `f1,...,fN` to a MATLAB function block with `N` outputs. Each element of `f1,...,fN` can be a symbolic expression, function, or a vector of symbolic expressions or functions.

`matlabFunctionBlock( ___,Name,Value)` converts a symbolic expression, function, or a vector of symbolic expressions or functions to a MATLAB function block using additional options specified by one or more name-value arguments with any of the input argument combinations in the previous syntaxes.

## Examples

### Convert Symbolic Expression to MATLAB Function Block

Create a new model and convert a symbolic expression to a MATLAB function block. Include comments in the block by specifying the `Comments` option.

Create a new model and open it.

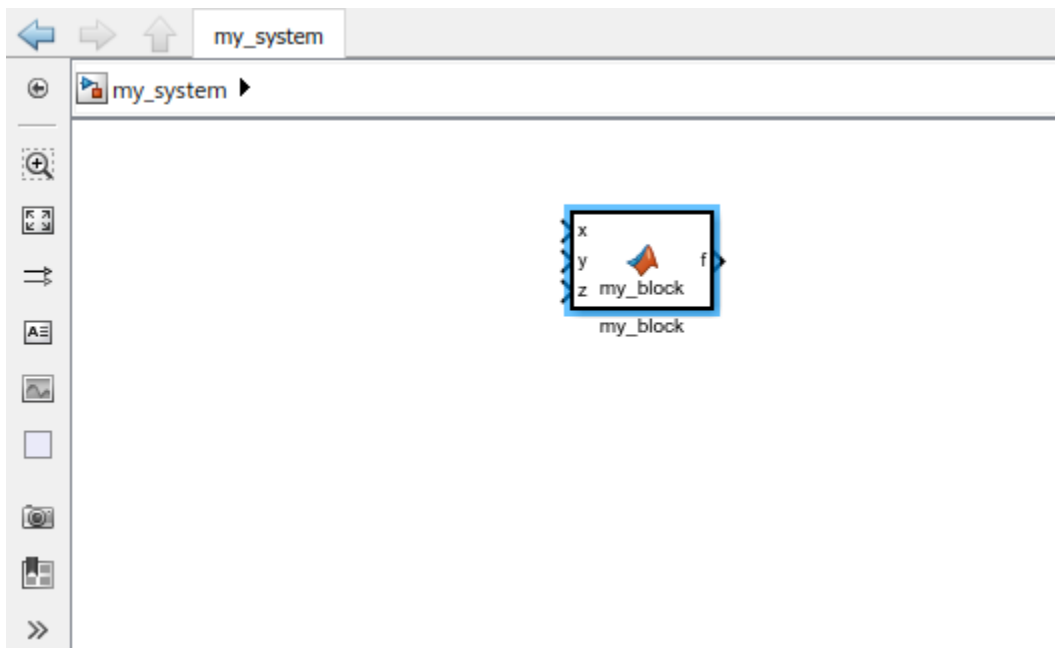
```
new_system('my_system')
open_system('my_system')
```

Create a symbolic expression.

```
syms x y z
f = x^2 + y^2 + z^2;
```

Use `matlabFunctionBlock` to create the block `my_block` containing the symbolic expression. `matlabFunctionBlock` overwrites existing blocks.

```
matlabFunctionBlock('my_system/my_block',f)
```



Double-click the generated block to open and edit the function defining the block.

```
function f = my_block(x,y,z)
%#codegen

% This function was generated by the Symbolic Math Toolbox version 7.3.
% 01-Jan-2017 00:00:00

f = x.^2+y.^2+z.^2;
```

Include the comment `Version 1.1` in the block.

```
matlabFunctionBlock('my_system/my_block',f,'Comments','Version: 1.1')

function f = my_block(x,y,z)
...
%Version: 1.1
f = x.^2+y.^2+z.^2;
```

Save and close `my_system`.

```
save_system('my_system')
close_system('my_system')
```

### Convert Symbolic Function to MATLAB Function Block

Create a new model and convert a symbolic function to a MATLAB function block.

Create a new empty model and open it.

```
new_system('my_system')
open_system('my_system')
```

Create a symbolic function.

```
syms x y z
f(x,y,z) = x^2 + y^2 + z^2;
```

Convert `f` to a MATLAB function block. Double-click the block to see the function.

```
matlabFunctionBlock('my_system/my_block',f)

function f = my_block(x,y,z)
%#codegen
f = x.^2+y.^2+z.^2;
```

### Create Blocks with Multiple Outputs

Convert several symbolic expressions to a MATLAB function block with multiple output ports.

Create a new empty model and open it.

```
new_system('my_system')
open_system('my_system')
```

Create three symbolic expressions.

```
syms x y z
f = x^2;
g = y^2;
h = z^2;
```

Convert them to a MATLAB function block. `matlabFunctionBlock` creates a block with three output ports. Double-click the block to see the function.

```
matlabFunctionBlock('my_system/my_block',f,g,h)

function [f,g,h] = my_block(x,y,z)
%#codegen
f = x.^2;
if nargout > 1
    g = y.^2;
end
if nargout > 2
    h = z.^2;
end
```

### Specify Function Name for Generated Function

Specify the name of the function defining the generated MATLAB function block.

Create a new empty model and open it.

```
new_system('my_system')
open_system('my_system')
```

Create a symbolic expression.

```
syms x y z
f = x^2 + y^2 + z^2;
```

Generate a block and set the function name to `my_function`. Double-click the block to see the function.

```
matlabFunctionBlock('my_system/my_block',f,...
    'FunctionName','my_function')

function f = my_function(x,y,z)
%#codegen
f = x.^2+y.^2+z.^2;
```

### Disable Code Optimization

When you convert a symbolic expression to a MATLAB function block, `matlabFunctionBlock` optimizes the code by default. This approach can help simplify and speed up further computations that use the file. Nevertheless, generating the optimized code from some symbolic expressions and functions can be very time-consuming. Use `Optimize` to disable code optimization.

Create a new empty model and open it.

```
new_system('my_system')
open_system('my_system')
```

Create a symbolic expression.

```
syms x
r = x^2*(x^2 + 1);
```

Use `matlabFunctionBlock` to create the block `my_block` containing the symbolic expression. Double-click the block to see the function defining the block. By default, `matlabFunctionBlock` creates a file containing the optimized code.

```
matlabFunctionBlock('my_system/my_block',r)

function r = my_block(x)
%#codegen
t2 = x.^2;
r = t2.*(t2+1.0);
```

Disable the code optimization by setting the value of `Optimize` to `false`.

```
matlabFunctionBlock('my_system/my_block',r,...
    'Optimize',false)

function r = my_block(x)
%#codegen
r = x.^2.*(x.^2+1.0);
```

### Specify Input Ports for Generated Block

Specify the order of the input variables that form the input ports in a generated block.

Create a new empty model and open it.

```
new_system('my_system')
open_system('my_system')
```

Create a symbolic expression.

```
syms x y z
f = x^2 + y^2 + z^2;
```

Convert the expression to a MATLAB function block. By default, `matlabFunctionBlock` uses alphabetical order of input arguments when converting symbolic expressions.

```
matlabFunctionBlock('my_system/my_block',f)
```

```
function f = my_block(x,y,z)
%#codegen
f = x.^2+y.^2+z.^2;
```

Use the `Vars` argument to specify the order of the input ports.

```
matlabFunctionBlock('my_system/my_block',f,...
    'Vars',[y z x])
```

```
function f = my_block(y,z,x)
%#codegen
f = x.^2+y.^2+z.^2;
```

## Specify Input Ports for Automatically Generated Elements

Specify the input variables explicitly for automatically generated elements.

Create a new empty model and open it.

```
new_system('my_system')
open_system('my_system')
```

Create a symbolic variable `u` and a 3-by-1 vector of symbolic variables `x`. The following command also automatically generates the symbolic variables `x1`, `x2`, and `x3` in the MATLAB workspace.

```
syms u
syms x [3 1]
whos
```

| Name | Size | Bytes | Class | Attributes |
|------|------|-------|-------|------------|
| u    | 1x1  | 8     | sym   |            |
| x    | 3x1  | 8     | sym   |            |
| x1   | 1x1  | 8     | sym   |            |
| x2   | 1x1  | 8     | sym   |            |
| x3   | 1x1  | 8     | sym   |            |

Create a symbolic expression that uses the symbolic variables `u` and `x`.

```
A = [0 1 0; 0 0 1; -1 -1 -2];
b = [0; 0; 3];
xdot = A*x+b*u

xdot =
```

$$3*u - x1 - \overset{x2}{x3} - 2*x3$$

Convert the `xdot` expression to a MATLAB function block. Because the elements of `xdot` contain the variable `u` and the automatically generated variables `x1`, `x2`, and `x3`, specify the `Vars` argument in terms of these variables explicitly instead of using `u` and `x`.

```
matlabFunctionBlock('my_system/my_block',xdot,'Vars',{u,x1,x2,x3})

function xdot = my_block(u,x1,x2,x3)
%#codegen
xdot = [x2;x3;u.*3.0-x1-x2-x3.*2.0];
```

### Specify Output Ports

When generating a block, rename the output variables and the corresponding ports.

Create a new empty model and open it.

```
new_system('my_system')
open_system('my_system')
```

Create a symbolic expression.

```
syms x y z
f = x^2 + y^2 + z^2;
```

Convert the expression to a MATLAB function block and specify the names of the output variables and ports. Double-click the block to see the function defining the block.

```
matlabFunctionBlock('my_system/my_block',f,f + 1,f + 2,...
    'Outputs',{'name1','name2','name3'})

function [name1,name2,name3] = my_block(x,y,z)
%#codegen
t2 = x.^2;
t3 = y.^2;
t4 = z.^2;
name1 = t2+t3+t4;
if nargout > 1
    name2 = t2+t3+t4+1.0;
end
if nargout > 2
    name3 = t2+t3+t4+2.0;
end
```

### Specify Function Name, Input and Output Ports

Call `matlabFunctionBlock` using several name-value pair arguments simultaneously.

Create a new empty model and open it.

```
new_system('my_system')
open_system('my_system')
```

Create a symbolic expression.

```
syms x y z
f = x^2 + y^2 + z^2;
```

Call `matlabFunctionBlock` using the name-value pair arguments to specify the function name, the order of the input ports, and the names of the output ports. Double-click the block to see the function defining the block.

```
matlabFunctionBlock('my_system/my_block',f,f + 1,f + 2,...
    'FunctionName','my_function','Vars',[y z x],...
    'Outputs',{'name1','name2','name3'})

function [name1,name2,name3] = my_function(y,z,x)
%#codegen
t2 = x.^2;
t3 = y.^2;
t4 = z.^2;
name1 = t2+t3+t4;
if nargin > 1
    name2 = t2+t3+t4+1.0;
end
if nargin > 2
    name3 = t2+t3+t4+2.0;
end
```

## Specify Complex Input

Specify a complex input in the generated MATLAB function block.

Create a new empty model and open it.

```
new_system('my_system')
open_system('my_system')
```

Create a symbolic function that finds the square root of an input.

```
syms x
f = sqrt(x);
```

Convert `f` to a MATLAB function block.

```
matlabFunctionBlock('my_system/my_block',f,'FunctionName','sqrt_block')
```

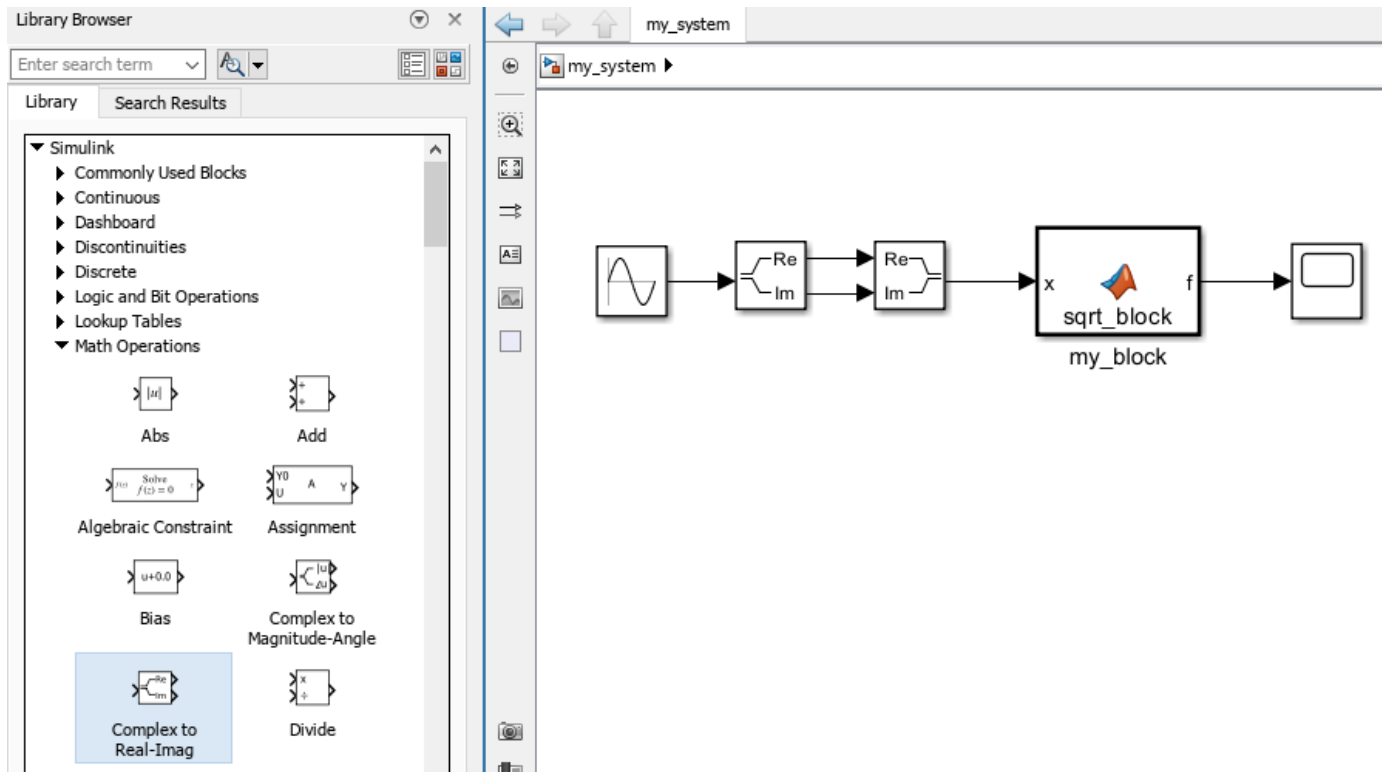
Because the square root function generates a complex result when the input `x` is negative, you need to manually specify a complex input in the generated block. Otherwise, you can encounter an error when running the function block with a negative input.

Double-click the block to see the function defining the block. Add the line `x = complex(x);` to specify a complex input. Starting in R2023a, `matlabFunctionBlock` provides this line and you can uncomment it. The generated function block now accepts a complex input and returns a complex output.

```
function f = sqrt_block(x)
%#codegen
x = complex(x); % required to enforce complex operations on x
```

```
f = sqrt(x);
end
```

As an alternative, you can also insert the Complex to Real-Imag and Real-Imag to Complex blocks from the **Simulink > Math Operations** library. Add these blocks to convert the input signal that goes into `x` to a complex signal.



## Input Arguments

### **block** — Block to create or modify

character vector

Block to create or modify, specified as a character vector.

### **f** — Symbolic input to be converted to MATLAB function block

symbolic expression | symbolic function | symbolic vector | symbolic matrix

Symbolic input to be converted to a MATLAB function block, specified as a symbolic expression, function, vector, or matrix.

### **f1, ..., fN** — Symbolic input to be converted to MATLAB function block with N outputs

several symbolic expressions | several symbolic functions | several symbolic vectors | several symbolic matrices

Symbolic input to be converted to a MATLAB function block with N outputs, specified as several symbolic expressions, functions, vectors, or matrices, separated by commas.



## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `matlabFunctionBlock('my_system/my_block',f,'FunctionName','myfun')`

### Comments — Comments to include in file header

character vector | cell array of character vectors | string vector

Comments to include in the file header, specified as a character vector, cell array of character vectors, or string vector.

### FunctionName — Name of function

coincides with the input argument `block` (default) | character vector

Name of the function, specified as a character vector. By default, `matlabFunction(block,...)` uses `block` as the function name.

See “Specify Function Name for Generated Function” on page 6-1035.

### Optimize — Flag for code optimization

true (default) | false

Flag for code optimization, specified as `true` or `false`.

When writing to a file, `matlabFunctionBlock` optimizes the code using intermediate variables named `t0`, `t1`, ...

See “Disable Code Optimization” on page 6-1036.

### Vars — Order of input variables and corresponding input ports of generated block

character vector | vector of symbolic variables | one-dimensional cell array of character vectors | one-dimensional cell array of symbolic variables | one-dimensional cell array of vectors of symbolic variables

Order of input variables and corresponding input ports of the generated block, specified as a character vector, a vector of symbolic variables, or a one-dimensional cell array of character vectors, symbolic variables, or vectors of symbolic variables.

The number of specified input ports must equal or exceed the number of free variables in `f`. Do not use the same names for the input ports specified by `Vars` and the output ports specified by `Outputs`.

By default, when you convert symbolic expressions, the order is alphabetical. When you convert symbolic functions, their input arguments appear in front of other variables, and all other variables are sorted alphabetically.

See “Specify Input Ports for Generated Block” on page 6-1036.

### Outputs — Names of output ports

`out` followed by output port numbers (default) | one-dimensional cell array of character vectors

Names of output ports, specified as a one-dimensional cell array of character vectors. If you do not specify the output port names, `matlabFunctionBlock` uses names that consist of the word `out` followed by output port numbers, for example, `out3`.

Do not use the same names for the input ports specified by `Vars` and the output ports specified by `Outputs`. See “Specify Output Ports” on page 6-1038.

## Limitations

- Some symbolic functions that have no corresponding MATLAB functions operating on the `double` data type, such as `simplify` and `solve`, are kept as symbolic functions in the generated MATLAB function block. The converted function block that consists of these functions cannot be used in Simulink models. You need to create your own functions with the `double` data type to replace these symbolic functions. If you are interested in a symbolic function that cannot be used in Simulink models, contact MathWorks Technical Support.

## Version History

**Introduced in R2009a**

## See Also

`ccode` | `fortran` | `matlabFunction` | `simscapeEquation` | `subs` | `sym2poly`

## Topics

“Implement MATLAB Functions in Simulink with MATLAB Function Blocks” (Simulink)

“Generate MATLAB Function Blocks from Symbolic Expressions” on page 5-6

# max

Maximum elements of symbolic input

## Syntax

```
M = max(A)
M = max(A,[],"all")
M = max(A,[],dim)
M = max(A,[], ___, nanflag)
[M,I] = max( ___)

C = max(A,B)
C = max(A,B,nanflag)
```

## Description

`M = max(A)` returns the maximum elements of a symbolic input.

- If `A` is a vector, then `max(A)` returns the maximum of `A`.
- If `A` is a matrix, then `max(A)` is a row vector containing the maximum value of each column of `A`.

For an input `A` that contains a symbolic expression, the symbolic `max` function returns an unevaluated expression that excludes elements in `A` that do not represent maximum values. The output might have another argument that represents the condition for the symbolic variable. For example, `syms x; max([1 x])` returns the output `max([1, x], [], 2, 'omitnan', ~in(x, 'real'))` in the Command Window because `x` is complex.

`M = max(A,[],"all")` returns the maximum over all elements of `A`.

`M = max(A,[],dim)` returns the maximum element along dimension `dim`. For example, if `A` is a matrix, then `max(A,[],2)` returns a column vector containing the maximum value of each row.

`M = max(A,[], ___, nanflag)` specifies whether to include or omit NaN values in `A` for any of the previous syntaxes. For example, `max(A,[],"includenan")` includes all NaN values when computing the maximum, while `max(A,[],"omitnan")` ignores all NaN values.

`[M,I] = max( ___)` also returns the index into the operating dimension that corresponds to the first occurrence of the maximum value of `A`.

`C = max(A,B)` returns an array containing the largest elements taken from `A` or `B`.

`C = max(A,B,nanflag)` also specifies how to treat NaN values.

## Examples

### Largest Real Element

Create a symbolic vector of real elements. Find the largest real element using the symbolic `max` function.

```
syms x real
A = [23 42 37 18 x];
M = max(A)

M = max([42, x], [], 2, "omitnan", symfalse)
```

The symbolic `max` function returns an unevaluated expression. The expression excludes elements in `A` that do not represent maximum values.

### Largest Element Involving NaN

Create a symbolic vector and compute its maximum, excluding NaN values.

```
syms x positive
A = [1.75 x 3.25 -2.5 NaN 0.5 NaN 0.2 -4*x];
M = max(A, [], "omitnan")

M =
    max( $\left[\frac{13}{4}, x\right]$ , [], 2, "omitnan", symfalse)
```

`max(A)` also produces this result because "omitnan" is the default option.

Use the "includenan" option to return NaN.

```
M = max(A, [], "includenan")

M = NaN
```

### Largest Element in Each Matrix Column

Create a symbolic matrix and find the largest element in each column.

```
syms x real
A = [1 x -0.5; -2 2 x]

A =

$$\begin{pmatrix} 1 & x & -\frac{1}{2} \\ -2 & 2 & x \end{pmatrix}$$


M = max(A)

M =

$$\left(1 \max([2, x], [], 2, "omitnan", symfalse) \max\left[-\frac{1}{2}, x\right], [], 2, "omitnan", symfalse\right)$$

```

### Largest Element in Each Matrix Row

Create a symbolic matrix and find the largest element in each row.

```
syms x real
A = [1 x -0.5; -2 2 x]

A =

$$\begin{pmatrix} 1 & x & -\frac{1}{2} \\ -2 & 2 & x \end{pmatrix}$$


M = max(A,[],2)

M =

$$\begin{pmatrix} \max([1, x], [], 2, \text{"omitnan"}, \text{symfalse}) \\ \max([2, x], [], 2, \text{"omitnan"}, \text{symfalse}) \end{pmatrix}$$

```

### Largest Element Indices

Create a symbolic matrix A. Find the largest elements in each row of A and return the column indices corresponding to the first occurrence of the maximum value in each row. Because A contains a symbolic variable x with an undetermined value, max cannot explicitly evaluate the maximum value in each row and returns the indices as -1.

```
syms x real
A = [1 x -0.5; -2 2 x]

A =

$$\begin{pmatrix} 1 & x & -\frac{1}{2} \\ -2 & 2 & x \end{pmatrix}$$


[M,I] = max(A,[],2)

M =

$$\begin{pmatrix} \max([1, x], [], 2, \text{"omitnan"}, \text{symfalse}) \\ \max([2, x], [], 2, \text{"omitnan"}, \text{symfalse}) \end{pmatrix}$$


I = 2×1

    -1
    -1
```

Substitute a symbolic number for the variable x in the symbolic matrix A. Find the largest elements in each row again. Here, max is able to evaluate the maximum values and returns the column indices corresponding to the first occurrence of the maximum value in each row.

```
A = subs(A,x,sqrt(2))

A =

$$\begin{pmatrix} 1 & \sqrt{2} & -\frac{1}{2} \\ -2 & 2 & \sqrt{2} \end{pmatrix}$$


[M,I] = max(A,[],2)

M =
```

$$\begin{pmatrix} \sqrt{2} \\ 2 \end{pmatrix}$$

$$I = 2 \times 1$$

$$\begin{pmatrix} 2 \\ 2 \end{pmatrix}$$

### Maximum of Matrix

Create a symbolic matrix. Find the maximum over all dimensions of the matrix by using the `max` function with the "all" option.

```
syms x real
A = [1 x -0.5; -2 2 x]

A =

$$\begin{pmatrix} 1 & x & -\frac{1}{2} \\ -2 & 2 & x \end{pmatrix}$$


M = max(A, [], "all")

M = max([2, x], [], 2, "omitnan", symfalse)
```

### Largest Element Comparison

Create two symbolic matrices with complex elements. Find the largest elements taken from the two matrices. The largest elements are the complex values with the largest magnitude.

```
syms x y
A = [x 2+1i; 3 4i; -5 y]

A =

$$\begin{pmatrix} x & 2 + i \\ 3 & 4 i \\ -5 & y \end{pmatrix}$$


B = [1 y; 2i 1+1i; -3 x]

B =

$$\begin{pmatrix} 1 & y \\ 2 i & 1 + i \\ -3 & x \end{pmatrix}$$


C = max(A,B)

C =

$$\begin{pmatrix} \max([1, x], [], 2, "omitnan", x \notin \mathbb{R}) & \max([2 + i, y], [], 2, "omitnan", \text{symtrue}) \\ 3 & 4 i \\ -5 & \max([x, y], [], 2, "omitnan", x \notin \mathbb{R} \vee y \notin \mathbb{R}) \end{pmatrix}$$

```

## Plot Expression with Maximum Function

Define the following expression by using the symbolic `max` function. Assume that the variable `x` is real.

$$f(x) = \begin{cases} \sqrt{x-1} & x > 1 \\ 0 & x < 1 \end{cases}$$

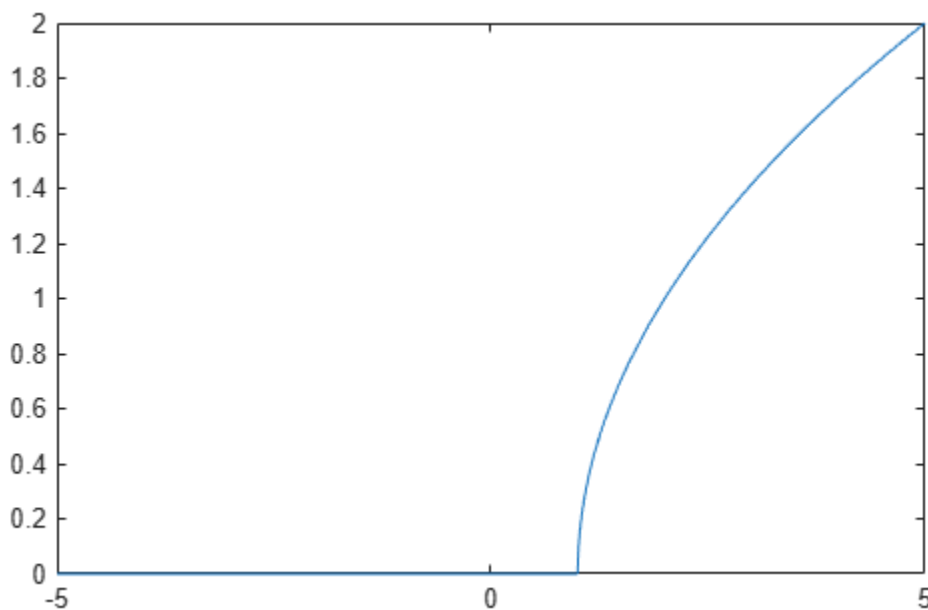
```
syms x real
```

```
f(x) = sqrt(max(x,1) - 1)
```

```
f(x) = sqrt(max([1,x],[1,2,"omitnan",symfalse)] - 1)
```

Plot the expression by using `fplot`.

```
fplot(f,[-5 5])
```



## Input Arguments

### A — Input array

symbolic expression | vector of symbolic expressions | matrix of symbolic expressions

Input array, specified as a symbolic expression, vector of symbolic expressions, or matrix of symbolic expressions.

- If `A` is complex, then `max(A)` returns the complex value with the largest magnitude. If magnitudes are equal, then `max(A)` returns the value with the largest magnitude and the largest phase angle.

- If  $A$  is a scalar, then  $\max(A)$  returns  $A$ .
- If  $A$  is a 0-by-0 empty array, then  $\max(A)$  is an empty array as well.

Data Types: `sym` | `single` | `double`  
 Complex Number Support: Yes

### **dim — Dimension to operate along**

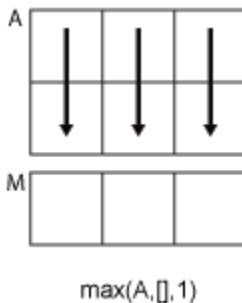
positive integer scalar

Dimension to operate along, specified as a positive integer scalar. If you do not specify a value, then the default is the first array dimension whose size does not equal 1.

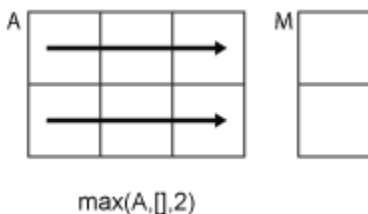
Dimension  $\text{dim}$  indicates the dimension whose length reduces to 1. So,  $\text{size}(M, \text{dim})$  is 1, while the sizes of all other dimensions remain the same, unless  $\text{size}(A, \text{dim})$  is 0. If  $\text{size}(A, \text{dim})$  is 0, then  $\max(A, \text{dim})$  returns an empty array with the same size as  $A$ .

Consider a two-dimensional input array,  $A$ :

- If  $\text{dim} = 1$ , then  $\max(A, [], 1)$  returns a row vector containing the largest element in each column.



- If  $\text{dim} = 2$ , then  $\max(A, [], 2)$  returns a column vector containing the largest element in each row.



$\max$  returns  $A$  if  $\text{dim}$  is greater than  $\text{ndims}(A)$ .

### **nanflag — NaN condition**

"omitnan" (default) | "includenan"

NaN condition, specified as one of these values:

- "omitnan" — Ignore all NaN values in the input. If all elements are NaN, then  $\max$  returns the first one.
- "includenan" — Include the NaN values in the input for the calculation.

### **B — Additional input array**

symbolic expression | vector of symbolic expressions | matrix of symbolic expressions



Additional input array, specified as a symbolic expression, vector of symbolic expressions, or matrix of symbolic expressions. Inputs A and B must either be the same size or have sizes that are compatible (for example, A is an M-by-N matrix and B is a scalar or 1-by-N row vector). For more information, see “Compatible Array Sizes for Basic Operations”.

Data Types: `sym` | `single` | `double`  
Complex Number Support: Yes

## Output Arguments

### M — Maximum values

symbolic expression | vector of symbolic expressions | matrix of symbolic expressions

Maximum values, returned as a symbolic expression, vector of symbolic expressions, or matrix of symbolic expressions. `size(M,dim)` is 1, while the sizes of all other dimensions match the size of the corresponding dimension in A, unless `size(A,dim)` is 0. If `size(A,dim)` is 0, then M is an empty array with the same size as A.

### I — Index

scalar | vector | matrix

Index, returned as a scalar, vector, or matrix. I is the same size as the output M.

If the largest element occurs more than once, then I contains the index to the first occurrence of the value.

Data Types: `double`

### C — Maximum elements from A or B

symbolic expression | vector of symbolic expressions | matrix of symbolic expressions

Maximum elements from A or B, returned as a symbolic expression, vector, or matrix of symbolic expressions. The size of C is determined by implicit expansion of the dimensions of A and B. For more information, see “Compatible Array Sizes for Basic Operations”.

## Version History

Introduced in R2021a

### R2021a: Find maximum elements of symbolic expressions

The `max` function can now operate on symbolic expressions involving symbolic variables. In previous releases, such symbolic expressions resulted in an error.

## See Also

### Functions

`sort` | `min`

### Topics

“Array Indexing”

## meijerG

Meijer G-function

### Syntax

`meijerG(a,b,c,d,z)`

### Description

`meijerG(a,b,c,d,z)` returns the “Meijer G-Function” on page 6-1053. `meijerG` is element-wise in `z`. The input parameters `a`, `b`, `c`, and `d` are vectors that can be empty, as in `meijerG([], [], 3.2, [], 1)`.

### Examples

#### Calculate Meijer G-Function for Numeric Inputs

```
syms x
meijerG(3, [], [], 2, 5)
```

```
ans =
    25
```

Call `meijerG` when `z` is an array. `meijerG` acts element-wise.

```
a = 2;
z = [1 2 3];
meijerG(a, [], [], [], z)
```

```
ans =
    0.3679    1.2131    2.1496
```

#### Calculate Meijer G-Function for Symbolic Numbers

Convert numeric input to symbolic form using `sym`, and find the Meijer G-function. For certain symbolic inputs, `meijerG` returns exact symbolic output using other functions.

```
meijerG(sym(2), [], [], [], sym(3))
```

```
ans =
3*exp(-1/3)
```

```
meijerG(sym(2/5), [], sym(1/2), [], sym(3))
```

```
ans =
(2^(4/5)*3^(1/2)*gamma(1/10))/80
```

### Find Meijer G-Function for Symbolic Variables or Expressions

For symbolic variables or expressions, `meijerG` returns an output in terms of simple or special functions.

```
syms a b c d z
f = meijerG(a,b,c,d,z)

f =
(gamma(c - a + 1)*(1/z)^(1 - a)*hypergeom([c - a + 1, d - a + 1],...
b - a + 1, 1/z))/(gamma(b - a + 1)*gamma(a - d))
```

Substitute values for the variables by using `subs`, and convert values to double by using `double`.

```
fVal = subs(f, [a b c d z], [1.2 3 5 7 9])

fVal =
(266*9^(1/5)*hypergeom([24/5, 34/5], 14/5, 1/9))/(25*gamma(-29/5))

double(fVal)

ans =
5.7586e+03
```

Calculate `fVal` to higher precision using `vpa`.

```
vpa(fVal)

ans =
5758.5946416377834597597497022199
```

### Relations Between Meijer G-Function and Other Functions

Show relations between `meijerG` and simpler functions for given parameter values.

Show that when `a`, `b`, and `d` are empty, and `c = 0`, then `meijerG` reduces to `exp(-z)`.

```
syms z
meijerG([], [], 0, [], z)

ans =
exp(-z)
```

Show that when `a`, `b`, and `d` are empty, and `c = [1/2 -1/2]`, then `meijerG` reduces to  $2K_v(1, 2z^{1/2})$ .

```
meijerG([], [], [1/2 -1/2], [], z)

ans =
2*besselk(1, 2*z^(1/2))
```

### Plot Meijer G-Function

Plot the real and imaginary values of the Meijer G-function for values of `b` and `z`, where `a = [-2 2]` and `c` and `d` are empty. Fill the contours by setting `Fill` to `on`.

```
syms b z
f = meijerG([-2 2], b, [], [], z);

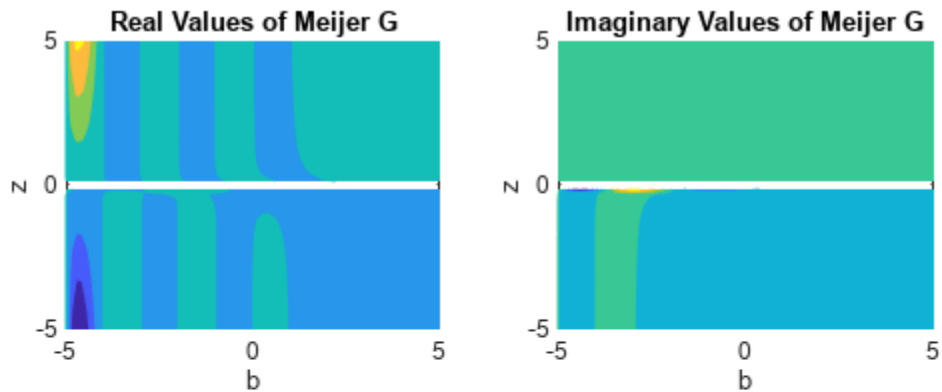
subplot(2,2,1)
fcontour(real(f), 'Fill', 'on')
```

```

title('Real Values of Meijer G')
xlabel('b')
ylabel('z')

subplot(2,2,2)
fcontour(imag(f),'Fill','on')
title('Imaginary Values of Meijer G')
xlabel('b')
ylabel('z')

```



## Input Arguments

### **a** — Input

number | vector | symbolic number | symbolic variable | symbolic vector | symbolic function | symbolic expression

Input, specified as a number or vector, or a symbolic number, variable, vector, function, or expression.

### **b** — Input

number | vector | symbolic number | symbolic variable | symbolic vector | symbolic function | symbolic expression

Input, specified as a number or vector, or a symbolic number, variable, vector, function, or expression.

**c — Input**

number | vector | symbolic number | symbolic variable | symbolic vector | symbolic function | symbolic expression

Input, specified as a number or vector, or a symbolic number, variable, vector, function, or expression.

**d — Input**

number | vector | symbolic number | symbolic variable | symbolic vector | symbolic function | symbolic expression

Input, specified as a number or vector, or a symbolic number, variable, vector, function, or expression.

**z — Input**

number | vector | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number or vector, or a symbolic number, variable, vector, function, or expression.

**More About****Meijer G-Function**

The Meijer G-function `meijerG([a1, ..., an], [an+1, ..., ap], [b1, ..., bm], [bm+1, ..., bq], z)` is a general function that includes other special functions as particular cases, and is defined as

$$G_{p,q}^{m,n} \left( \begin{matrix} a_1, \dots, a_p \\ b_1, \dots, b_q \end{matrix} \middle| z \right) = \frac{1}{2\pi i} \int \frac{\left( \prod_{j=1}^m \Gamma(b_j - s) \right) \left( \prod_{j=1}^n \Gamma(1 - a_j + s) \right)}{\left( \prod_{j=m+1}^q \Gamma(1 - b_j + s) \right) \left( \prod_{j=n+1}^p \Gamma(a_j - s) \right)} z^s ds.$$

**Algorithms**

For the Meijer G-function `meijerG([a1, ..., an], [an+1, ..., ap], [b1, ..., bm], [bm+1, ..., bq], z)`, for  $a_i \in (a_1, \dots, a_n)$  and  $b_j \in (b_1, \dots, b_m)$ , no pair of parameters  $a_i - b_j$  should differ by a positive integer.

The Meijer G-function involves a complex contour integral with one of the following types of integration paths:

- The contour goes from  $-i\infty$  to  $i\infty$  so that all poles of  $\Gamma(b_j - s)$ ,  $j = 1, \dots, m$  lie to the right of the path, and all poles of  $\Gamma(1 - a_k + s)$ ,  $k = 1, \dots, n$  lie to the left of the path. The integral converges if  $c = m + n - \frac{p+q}{2} > 0$ ,  $|\arg(z)| < c\pi$ . If  $|\arg(z)| = c\pi$ ,  $c \geq 0$ , the integral converges absolutely

when  $p = q$  and  $\Re(\psi) < -1$ , where  $\Psi = \left( \sum_{j=1}^q b_j \right) - \left( \sum_{i=1}^p a_i \right)$ . When  $p \neq q$ , the integral converges if

you choose the contour so that the contour points near  $i\infty$  and  $-i\infty$  have a real part  $\sigma$  satisfying  $(q - p)\sigma > \Re(\psi) + 1 - \frac{q-p}{2}$ .

- The contour is a loop beginning and ending at  $\infty$  and encircling all poles of  $\Gamma(b_j - s)$ ,  $j = 1, \dots, m$  moving in the negative direction, but none of the poles of  $\Gamma(1 - a_k + s)$ ,  $k = 1, \dots, n$ . The integral converges if  $q \geq 1$  and either  $p < q$  or  $p = q$  and  $|z| < 1$ .
- The contour is a loop beginning and ending at  $-\infty$  and encircling all poles of  $\Gamma(1 - a_k + s)$ ,  $k = 1, \dots, n$  moving in the positive direction, but none of the poles of  $\Gamma(b_j + s)$ ,  $j = 1, \dots, m$ . The integral converges if  $p \geq 1$  and either  $p > q$  or  $p = q$  and  $|z| > 1$ .

The integral represents an inverse Laplace transform or, more specifically, a Mellin-Barnes type of integral.

For a given set of parameters, the contour chosen in the definition of the Meijer G-function is the one for which the integral converges. If the integral converges for several contours, all contours lead to the same function.

The Meijer G-function satisfies a differential equation of order  $\max(p, q)$  with respect to a variable  $z$ :

$$\left( (-1)^{m+n-p} z \left( \prod_{i=1}^p \left( z \frac{d}{dz} - a_i - 1 \right) \right) - \prod_{j=1}^q \left( z \frac{d}{dz} - b_j \right) \right) G_{p,q}^{m,n} \left( \begin{matrix} a_1, \dots, a_p \\ b_1, \dots, b_q \end{matrix} \middle| z \right) = 0.$$

If  $p < q$ , this differential equation has a regular singularity at  $z = 0$  and an irregular singularity at  $z = \infty$ . If  $p = q$ , the points  $z = 0$  and  $z = \infty$  are regular singularities, and there is an additional regular singularity at  $z = (-1)^{m+n-p}$ .

The Meijer G-function represents an analytic continuation of the hypergeometric function [1]. For particular choices of parameters, you can express the Meijer G-function through the hypergeometric function. For example, if no two of the  $b_h$  terms,  $h = 1, \dots, m$ , differ by an integer or zero and all poles are simple, then

$$G_{p,q}^{m,n} \left( \begin{matrix} a_1, \dots, a_p \\ b_1, \dots, b_q \end{matrix} \middle| z \right) = \sum_{h=1}^m \frac{\left( \prod_{j=1 \dots m, j \neq h} \Gamma(b_j - b_h) \right) \left( \prod_{j=1}^n \Gamma(1 + b_h - a_j) \right)}{\left( \prod_{j=m+1}^q \Gamma(1 + b_h - b_j) \right) \left( \prod_{j=n+1}^p \Gamma(a_j - b_h) \right)} z^{b_h} {}_pF_{q-1} \left( \begin{matrix} A_h; B_h; (-1)^{p-m-n} z \end{matrix} \right).$$

Here  $p < q$  or  $p = q$  and  $|z| < 1$ .  $A_h$  denotes

$$A_h = 1 + b_h - a_1, \dots, 1 + b_h - a_p.$$

$B_h$  denotes

$$B_h = 1 + b_h - b_1, \dots, 1 + b_h - b_{(h-1)}, 1 + b_h - b_{h+1}, \dots, 1 + b_h - b_q.$$

Meijer G-functions with different parameters can represent the same function.

- The Meijer G-function is symmetric with respect to the parameters. Changing the order inside each of the following lists of vectors does not change the resulting Meijer G-function:  $[a_1, \dots, a_n]$ ,  $[a_{n+1}, \dots, a_p]$ ,  $[b_1, \dots, b_m]$ ,  $[b_{m+1}, \dots, b_q]$ .
- If  $z$  is not a negative real number and  $z \neq 0$ , the function satisfies the following identity:

$$G_{p,q}^{m,n} \left( \begin{matrix} a_1, \dots, a_p \\ b_1, \dots, b_q \end{matrix} \middle| z \right) = G_{q,p}^{n,m} \left( \begin{matrix} 1-b_1, \dots, 1-b_p \\ 1-a_1, \dots, 1-a_p \end{matrix} \middle| \frac{1}{z} \right).$$

- If  $0 < n < p$  and  $r = a_1 - a_p$  is an integer, the function satisfies the following identity:

$$G_{p,q}^{m,n} \left( \begin{matrix} a_1, a_2, \dots, a_{p-1}, a_p \\ b_1, b_2, \dots, b_{q-1}, b_q \end{matrix} \middle| z \right) = G_{p,q}^{m,n} \left( \begin{matrix} a_p, a_2, \dots, a_{p-1}, a_1 \\ b_1, b_2, \dots, b_{q-1}, b_q \end{matrix} \middle| z \right).$$

- If  $0 < m < q$  and  $r = b_1 - b_q$  is an integer, the function satisfies the following identity:

$$G_{p,q}^{m,n} \left( \begin{matrix} a_1, a_2, \dots, a_{p-1}, a_p \\ b_1, b_2, \dots, b_{q-1}, b_q \end{matrix} \middle| z \right) = (-1)^r G_{p,q}^{m,n} \left( \begin{matrix} a_1, a_2, \dots, a_{p-1}, a_p \\ b_q, b_2, \dots, b_{q-1}, b_1 \end{matrix} \middle| z \right).$$

According to these rules, the `meijerG` function call can return `meijerG` with modified input parameters.

## Version History

Introduced in R2017b

## References

- [1] Luke, Y. L., *The Special Functions and Their Approximations*. Vol. 1. New York: Academic Press, 1969.
- [2] Prudnikov, A. P., Yu. A. Brychkov, and O. I. Marichev, *Integrals and Series*. Vol 3: More Special Functions. Gordon and Breach, 1990.
- [3] Abramowitz, M., I. A. Stegun, *Handbook of Mathematical Functions*. 9th printing. New York: Dover Publications, 1970.

## See Also

`hypergeom`

## min

Minimum elements of symbolic input

### Syntax

```
M = min(A)
M = min(A,[],"all")
M = min(A,[],dim)
M = min(A,[], ___, nanflag)
[M,I] = min( ___)

C = min(A,B)
C = min(A,B,nanflag)
```

### Description

`M = min(A)` returns the minimum elements of a symbolic input.

- If `A` is a vector, then `min(A)` returns the minimum of `A`.
- If `A` is a matrix, then `min(A)` is a row vector containing the minimum value of each column of `A`.

For an input `A` that contains a symbolic expression, the symbolic `min` function returns an unevaluated expression that excludes elements in `A` that do not represent minimum values. The output might have another argument that represents the condition for the symbolic variable. For example, `syms x; min([1 x])` returns the output `min([1, x], [], 2, 'omitnan', ~in(x, 'real'))` in the Command Window because `x` is complex.

`M = min(A,[],"all")` returns the minimum over all elements of `A`.

`M = min(A,[],dim)` returns the minimum element along dimension `dim`. For example, if `A` is a matrix, then `min(A,[],2)` returns a column vector containing the minimum value of each row.

`M = min(A,[], ___, nanflag)` specifies whether to include or omit NaN values in `A` for any of the previous syntaxes. For example, `min(A,[],"includenan")` includes all NaN values when computing the minimum, while `min(A,[],"omitnan")` ignores all NaN values.

`[M,I] = min( ___)` also returns the index into the operating dimension that corresponds to the first occurrence of the minimum value of `A`.

`C = min(A,B)` returns an array containing the smallest elements taken from `A` or `B`.

`C = min(A,B,nanflag)` also specifies how to treat NaN values.

### Examples

#### Smallest Real Element

Create a symbolic vector of real elements. Find the smallest real element using the symbolic `min` function.



```
syms x real
A = [23 42 37 18 x];
M = min(A)

M = min([18, x], [], 2, "omitnan", symfalse)
```

The symbolic `min` function returns an unevaluated expression. The expression excludes elements in `A` that do not represent minimum values.

### Smallest Element Involving NaN

Create a symbolic vector and compute its minimum, excluding NaN values.

```
syms x positive
A = [1.75 x 3.25 -2.5 NaN 0.5 NaN 0.2 -4*x];
M = min(A, [], "omitnan")

M =
    min( $\left[-\frac{5}{2}, -4x\right]$ , [], 2, "omitnan", symfalse)
```

`min(A)` also produces this result because "omitnan" is the default option.

Use the "includenan" option to return NaN.

```
M = min(A, [], "includenan")

M = NaN
```

### Smallest Element in Each Matrix Column

Create a symbolic matrix and find the smallest element in each column.

```
syms x real
A = [1 x -0.5; -2 1 x]

A =

$$\begin{pmatrix} 1 & x & -\frac{1}{2} \\ -2 & 1 & x \end{pmatrix}$$


M = min(A)

M =

$$\left(-2 \min([1, x], [], 2, "omitnan", symfalse) \min\left(\left[-\frac{1}{2}, x\right], [], 2, "omitnan", symfalse\right)\right)$$

```

### Smallest Element in Each Matrix Row

Create a symbolic matrix and find the smallest element in each row.

```
syms x real
A = [1 x -0.5; -2 1 x]

A =

$$\begin{pmatrix} 1 & x & -\frac{1}{2} \\ -2 & 1 & x \end{pmatrix}$$


M = min(A,[],2)

M =

$$\begin{pmatrix} \min\left(-\frac{1}{2}, x\right), [], 2, \text{"omitnan"}, \text{symfalse} \\ \min(-2, x), [], 2, \text{"omitnan"}, \text{symfalse} \end{pmatrix}$$

```

### Smallest Element Indices

Create a symbolic matrix **A**. Find the smallest elements in each row of **A** and return the column indices corresponding to the first occurrence of the minimum value in each row. Because **A** contains a symbolic variable **x** with an undetermined value, **min** cannot explicitly evaluate the minimum value in each row and returns the indices as **-1**.

```
syms x real
A = [1 x -0.5; -2 2 x]

A =

$$\begin{pmatrix} 1 & x & -\frac{1}{2} \\ -2 & 2 & x \end{pmatrix}$$


[M,I] = min(A,[],2)

M =

$$\begin{pmatrix} \min\left(-\frac{1}{2}, x\right), [], 2, \text{"omitnan"}, \text{symfalse} \\ \min(-2, x), [], 2, \text{"omitnan"}, \text{symfalse} \end{pmatrix}$$


I = 2×1

-1
-1
```

Substitute a symbolic number for the variable **x** in the symbolic matrix **A**. Find the smallest elements in each row again. Here, **min** is able to evaluate the minimum values and returns the column indices corresponding to the first occurrence of the minimum value in each row.

```
A = subs(A,x,sqrt(2))

A =

$$\begin{pmatrix} 1 & \sqrt{2} & -\frac{1}{2} \\ -2 & 2 & \sqrt{2} \end{pmatrix}$$


[M,I] = min(A,[],2)
```

$$M = \begin{pmatrix} -\frac{1}{2} \\ -2 \end{pmatrix}$$

$$I = 2 \times 1$$

$$\begin{pmatrix} 3 \\ 1 \end{pmatrix}$$

### Minimum of Matrix

Create a symbolic matrix. Find the minimum over all dimensions of the matrix by using the `min` function with the "all" option.

```
syms x real
A = [1 x -0.5; -2 1 x]
```

$$A = \begin{pmatrix} 1 & x & -\frac{1}{2} \\ -2 & 1 & x \end{pmatrix}$$

```
M = min(A,[],"all")
```

```
M = min([-2,x],[],2,"omitnan",symfalse)
```

### Smallest Element Comparison

Create two symbolic matrices with complex elements. Find the smallest elements taken from the two matrices. The largest elements are the complex values with the smallest magnitude.

```
syms x y
A = [x 2+1i; 3 4i; -5 y]
```

$$A = \begin{pmatrix} x & 2 + i \\ 3 & 4i \\ -5 & y \end{pmatrix}$$

```
B = [1 y; 2i 1+1i; -3 x]
```

$$B = \begin{pmatrix} 1 & y \\ 2i & 1 + i \\ -3 & x \end{pmatrix}$$

```
C = min(A,B)
```

```
C =
```

$$\left( \begin{array}{cc} \min([1, x], [], 2, \text{"omitnan"}, x \notin \mathbb{R}) & \min([2 + i, y], [], 2, \text{"omitnan"}, \text{symtrue}) \\ 2i & 1 + i \\ -3 & \min([x, y], [], 2, \text{"omitnan"}, x \notin \mathbb{R} \vee y \notin \mathbb{R}) \end{array} \right)$$

### Plot Expression with Minimum Function

Define the following expression by using the symbolic `min` function. Assume that the variable  $x$  is real.

$$f(x) = \begin{cases} 0 & x > 1 \\ \sqrt{1-x} & x < 1 \end{cases}$$

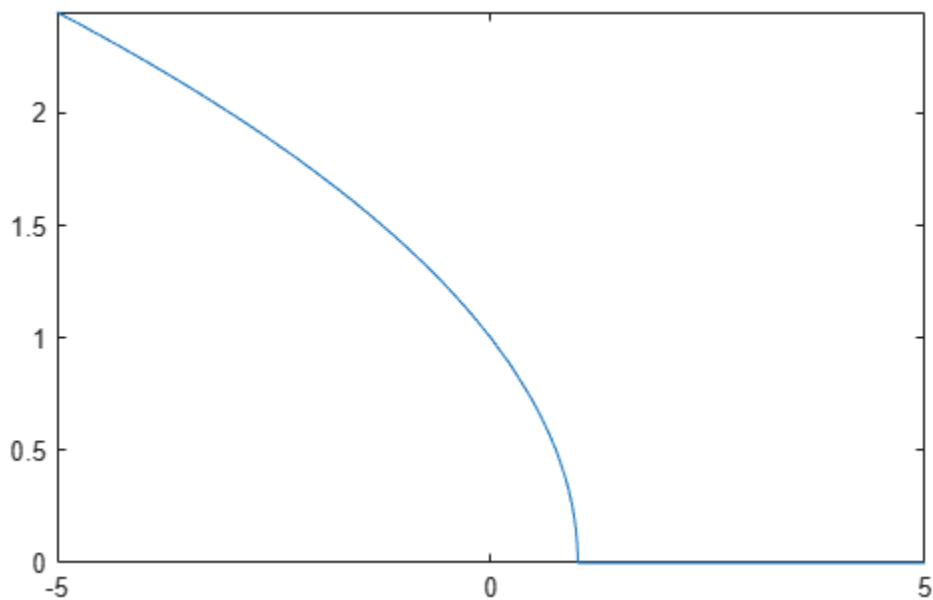
`syms x real`

`f(x) = sqrt(1 - min(x, 1))`

`f(x) = sqrt(1 - min([1, x], [], 2, "omitnan", symfalse))`

Plot the expression by using `fplot`.

`fplot(f, [-5 5])`



### Input Arguments

#### A — Input array

symbolic expression | vector of symbolic expressions | matrix of symbolic expressions

Input array, specified as a symbolic expression, vector of symbolic expressions, or matrix of symbolic expressions.

- If  $A$  is complex, then  $\min(A)$  returns the complex number with the smallest magnitude. If magnitudes are equal, then  $\min(A)$  returns the value with the smallest magnitude and the smallest phase angle.
- If  $A$  is a scalar, then  $\min(A)$  returns  $A$ .
- If  $A$  is a 0-by-0 empty array, then  $\min(A)$  is an empty array as well.

Data Types: `sym` | `single` | `double`

Complex Number Support: Yes

### **dim — Dimension to operate along**

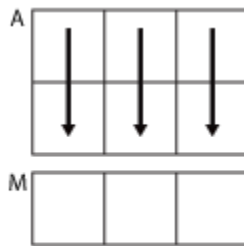
positive integer scalar

Dimension to operate along, specified as a positive integer scalar. If you do not specify a value, then the default is the first array dimension whose size does not equal 1.

Dimension  $\text{dim}$  indicates the dimension whose length reduces to 1. So,  $\text{size}(M, \text{dim})$  is 1, while the sizes of all other dimensions remain the same, unless  $\text{size}(A, \text{dim})$  is 0. If  $\text{size}(A, \text{dim})$  is 0, then  $\min(A, \text{dim})$  returns an empty array with the same size as  $A$ .

Consider a two-dimensional input array,  $A$ :

- If  $\text{dim} = 1$ , then  $\min(A, [], 1)$  returns a row vector containing the smallest element in each column.



$\min(A, [], 1)$

- If  $\text{dim} = 2$ , then  $\min(A, [], 2)$  returns a column vector containing the smallest element in each row.



$\min(A, [], 2)$

$\min$  returns  $A$  if  $\text{dim}$  is greater than  $\text{ndims}(A)$ .

### **nanflag — NaN condition**

"omitnan" (default) | "includenan"

NaN condition, specified as one of these values:

- "omitnan" — Ignore all NaN values in the input. If all elements are NaN, then  $\min$  returns the first one.

- "includenan" — Include the NaN values in the input for the calculation.

**B — Additional input array**

symbolic expression | vector of symbolic expressions | matrix of symbolic expressions

Additional input array, specified as a symbolic expression, vector of symbolic expressions, or matrix of symbolic expressions. Inputs A and B must either be the same size or have sizes that are compatible (for example, A is an M-by-N matrix and B is a scalar or 1-by-N row vector). For more information, see "Compatible Array Sizes for Basic Operations".

Data Types: sym | single | double

Complex Number Support: Yes

## Output Arguments

**M — Minimum values**

symbolic expression | vector of symbolic expressions | matrix of symbolic expressions

Minimum values, returned as a symbolic expression, vector of symbolic expressions, or matrix of symbolic expressions. `size(M,dim)` is 1, while the sizes of all other dimensions match the size of the corresponding dimension in A, unless `size(A,dim)` is 0. If `size(A,dim)` is 0, then M is an empty array with the same size as A.

**I — Index**

scalar | vector | matrix

Index, returned as a scalar, vector, or matrix. I is the same size as the output M.

If the smallest element occurs more than once, then I contains the index to the first occurrence of the value.

Data Types: double

**C — Minimum elements from A or B**

symbolic expression | vector of symbolic expressions | matrix of symbolic expressions

Minimum elements from A or B, returned as a symbolic expression, vector of symbolic expressions, or matrix of symbolic expressions. The size of C is determined by implicit expansion of the dimensions of A and B. For more information, see "Compatible Array Sizes for Basic Operations".

## Version History

Introduced in R2021a

**R2021a: Find minimum elements of symbolic expressions**

The `min` function can now operate on symbolic expressions involving symbolic variables. In previous releases, such symbolic expressions resulted in an error.

## See Also

**Functions**

sort | max

**Topics**

"Array Indexing"

## minpoly

Minimal polynomial of matrix

### Syntax

```
minpoly(A)
minpoly(A,var)
```

### Description

`minpoly(A)` returns a vector of the coefficients of the minimal polynomial on page 6-1065 of A. If A is a symbolic matrix, `minpoly` returns a symbolic vector. Otherwise, it returns a vector with elements of type double.

`minpoly(A,var)` returns the minimal polynomial of A in terms of var.

### Examples

#### Compute Minimal Polynomial of Matrix

Compute the minimal polynomial of the matrix A in terms of the variable x:

```
syms x
A = sym([1 1 0; 0 1 0; 0 0 1]);
minpoly(A, x)
```

```
ans =
x^2 - 2*x + 1
```

#### Compute Coefficients of Minimal Polynomial

To find the coefficients of the minimal polynomial of A, call `minpoly` with one argument. Since A is numeric, `minpoly` returns coefficients as double-precision values:

```
A = sym([1 1 0; 0 1 0; 0 0 1]);
minpoly(A)
```

```
ans =
[ 1, -2, 1]
```

Find the coefficients of the minimal polynomial of the symbolic matrix A. For this matrix, `minpoly` returns the symbolic vector of coefficients:

```
A = sym([0 2 0; 0 0 2; 2 0 0]);
P = minpoly(A)
```

```
P =
[ 1, 0, 0, -8]
```



## Input Arguments

### **A — Input**

numeric matrix | symbolic matrix

Input, specified as a numeric or symbolic matrix.

### **var — Input**

symbolic variable

Input, specified as a symbolic variable. If you do not specify `var`, `minpoly` returns a vector of coefficients of the minimal polynomial instead of returning the polynomial itself.

## More About

### **Minimal Polynomial of a Matrix**

The minimal polynomial of a square matrix  $A$  is the monic polynomial  $p(x)$  of the least degree, such that  $p(A) = 0$ .

## Version History

Introduced in R2012b

### **See Also**

`charpoly` | `eig` | `jordan` | `poly2sym` | `sym2poly`

## minus, -

Symbolic subtraction

### Syntax

```
-A  
A - B  
minus(A,B)
```

### Description

-A returns the negation of A.

A - B subtracts B from A and returns the result.

minus(A,B) is an alternate way to execute A - B.

### Examples

#### Subtract Scalar from Array

Subtract 2 from array A.

```
syms x  
A = [x 1; -2 sin(x)];  
A - 2
```

```
ans =  
[ x - 2,          -1]  
[    -4, sin(x) - 2]
```

minus subtracts 2 from each element of A.

Subtract the identity matrix from matrix M:

```
syms x y z  
M = [0 x; y z];  
M - eye(2)
```

```
ans =  
[ -1,    x]  
[  y, z - 1]
```

#### Subtract Numeric and Symbolic Arguments

Subtract one number from another. Because these are not symbolic objects, you receive floating-point results.

```
11/6 - 5/4
```

```
ans =  
0.5833
```

Perform subtraction symbolically by converting the numbers to symbolic objects.

```
sym(11/6) - sym(5/4)
```

```
ans =  
7/12
```

Alternatively, call `minus` to perform subtraction.

```
minus(sym(11/6),sym(5/4))
```

```
ans =  
7/12
```

## Subtract Matrices

Subtract matrices B and C from A.

```
A = sym([3 4; 2 1]);  
B = sym([8 1; 5 2]);  
C = sym([6 3; 4 9]);  
Y = A - B - C
```

```
Y =  
[ -11,  0]  
[  -7, -10]
```

Use syntax `-Y` to negate the elements of Y.

```
-Y
```

```
ans =  
[ 11,  0]  
[  7, 10]
```

## Subtract Functions

Subtract function g from function f.

```
syms f(x) g(x)  
f = sin(x) + 2*x;  
y = f - g
```

```
y(x) =  
2*x - g(x) + sin(x)
```

## Input Arguments

### A — Input

symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix | symbolic array

Input, specified as a symbolic scalar variable, matrix variable, function, matrix function, expression, or vector, matrix, or array of symbolic scalar variables.

### B — Input

symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix | symbolic array

Input, specified as a symbolic scalar variable, matrix variable, function, matrix function, expression, or vector, matrix, or array of symbolic scalar variables.

### **Tips**

- All nonscalar arguments must have the same size. If one input argument is nonscalar, then `minus` expands the scalar into an array of the same size as the nonscalar argument, with all elements equal to the corresponding scalar.

## **Version History**

**Introduced before R2006a**

### **R2022a: Compute subtraction of symbolic matrix functions**

The `minus` function accepts an input argument of type `symfunmatrix`.

### **R2021a: Compute subtraction of symbolic matrix variables**

The `minus` function accepts an input argument of type `symmatrix`.

### **See Also**

`ctranspose` | `ldivide` | `mldivide` | `mpower` | `mrdivide` | `mtimes` | `plus` | `power` | `rdivide` | `times` | `transpose`

# mixedUnits

Split unit into sum of units

## Syntax

```
mixedUnits(quantity,units)
```

## Description

`mixedUnits(quantity,units)` splits the physical quantity `quantity` into a linear combination of the units in `units`.

- Units in `units` must be in descending order of magnitude.
- Units in `quantity` and `units` must be compatible.
- `quantity` must not contain symbolic variables.

## Examples

### Split Quantity into Combination of Units

Split 8000 seconds into hours, minutes, and seconds by using `mixedUnits`. The result is 2 hours, 13 minutes, and 20 seconds.

```
u = symunit;
t = 8000*u.s;
tunits = [u.hour u.minute u.second];
tSplit = mixedUnits(t,tunits)

tSplit =
[ 2, 13, 20]
```

Customize the displayed output by using `compose`.

```
compose("%d hours + %d minutes + %.1f seconds", double(tSplit))

ans =
    "2 hours + 13 minutes + 20.0 seconds"
```

Convert the geographic coordinate  $15.352^\circ$  into degrees ( $^\circ$ ), arcminutes ( $'$ ), and arcseconds ( $''$ ). The result is  $15^\circ 21' 36/5''$ .

```
gCoord = 15.352*u.degree;
gUnits = [u.degree u.arcmin u.arcsec];
gCoordSplit = mixedUnits(gCoord,gUnits)

gCoordSplit =
[ 15, 21, 36/5]
```

Convert the result from symbolic to floating point by using `double`.

```
gCoordDb1 = double(gCoordSplit)
```

```
gCoordDbl =  
    15.0000    21.0000    7.2000
```

Reconstruct the original coordinate by summing the split units and rewriting the result to degrees. `mixedUnits` returns an exact symbolic result instead of a numeric approximation. For details, see “Choose Numeric or Symbolic Arithmetic” on page 2-32.

```
gOrig = sum(gCoordSplit.*gUnits);  
gOrig = rewrite(gOrig,u.degree)
```

```
gOrig =  
(1919/125)*[deg]
```

## Input Arguments

### **quantity** — Input

symbolic expression with units

Input, specified as a symbolic expression with units. `quantity` must not contain symbolic variables. Units in `quantity` and `units` must be compatible.

### **units** — Units for representing input

vector of symbolic units

Units to represent input as, specified as a vector of symbolic units. Units must be in descending order of magnitude. Units in `quantity` and `units` must be compatible.

## Version History

**Introduced in R2018a**

## See Also

`symunit` | `unitConversionFactor`

### **Topics**

“Units of Measurement Tutorial” on page 2-47

“Unit Conversions and Unit Systems” on page 2-53

“Units and Unit Systems List” on page 2-60

# mldivide, \

Symbolic matrix left division

## Syntax

```
X = A\B
X = mldivide(A,B)
```

## Description

$X = A \setminus B$  solves the symbolic system of linear equations in matrix form,  $A * X = B$  for  $X$ .

If the solution does not exist or if it is not unique, the `\` operator issues a warning.

$A$  can be a rectangular matrix, but the equations must be consistent. The symbolic operator `\` does not compute least-squares solutions.

$X = \text{mldivide}(A,B)$  is equivalent to  $x = A \setminus B$ .

## Examples

### System of Equations in Matrix Form

Solve a system of linear equations specified by a square matrix of coefficients and a vector of right sides of equations.

Create a matrix containing the coefficient of equation terms, and a vector containing the right sides of equations.

```
A = sym(pascal(4))
b = sym([4; 3; 2; 1])
```

```
A =
[ 1, 1, 1, 1]
[ 1, 2, 3, 4]
[ 1, 3, 6, 10]
[ 1, 4, 10, 20]
```

```
b =
4
3
2
1
```

Use the operator `\` to solve this system.

```
X = A\b
```

```
X =
5
-1
0
0
```

### Rank-Deficient System

Create a matrix containing the coefficients of equation terms, and a vector containing the right sides of equations.

```
A = sym(magic(4))
b = sym([0; 1; 1; 0])
```

```
A =
[ 16,  2,  3, 13]
[  5, 11, 10,  8]
[  9,  7,  6, 12]
[  4, 14, 15,  1]
```

```
b =
 0
 1
 1
 0
```

Find the rank of the system. This system contains four equations, but its rank is 3. Therefore, the system is rank-deficient. This means that one variable of the system is not independent and can be expressed in terms of other variables.

```
rank(horzcat(A,b))
```

```
ans =
 3
```

Try to solve this system using the symbolic `\` operator. Because the system is rank-deficient, the returned solution is not unique.

```
A\b
```

```
Warning: Solution is not unique because the system is rank-deficient.
```

```
ans =
 1/34
19/34
-9/17
  0
```

### Inconsistent System

Create a matrix containing the coefficient of equation terms, and a vector containing the right sides of equations.

```
A = sym(magic(4))
b = sym([0; 1; 2; 3])
```

```
A =
[ 16,  2,  3, 13]
[  5, 11, 10,  8]
[  9,  7,  6, 12]
[  4, 14, 15,  1]
```

```
b =
 0
```



```
1
2
3
```

Try to solve this system using the symbolic `\` operator. The operator issues a warning and returns a vector with all elements set to `Inf` because the system of equations is inconsistent, and therefore, no solution exists. The number of elements in the resulting vector equals the number of equations (rows in the coefficient matrix).

```
A\b
```

```
Warning: Solution does not exist because the system is inconsistent.
```

```
ans =
    Inf
    Inf
    Inf
    Inf
```

Find the reduced row echelon form of this system. The last row shows that one of the equations reduced to  $0 = 1$ , which means that the system of equations is inconsistent.

```
rref(horzcat(A,b))
```

```
ans =
[ 1, 0, 0, 1, 0]
[ 0, 1, 0, 3, 0]
[ 0, 0, 1, -3, 0]
[ 0, 0, 0, 0, 1]
```

## Input Arguments

### A — Coefficient matrix

symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix

Coefficient matrix, specified as a symbolic number, scalar variable, matrix variable, function, matrix function, expression, or vector or matrix of symbolic scalar variables.

### B — Right side

symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix

Right side, specified as a symbolic number, scalar variable, matrix variable, function, matrix function, expression, or vector or matrix of symbolic scalar variables.

## Output Arguments

### X — Solution

symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix

Solution, returned as a symbolic number, scalar variable, matrix variable, function, matrix function, expression, or vector or matrix of symbolic scalar variables.

## Tips

- Matrix computations involving many symbolic variables can be slow. To increase the computational speed, reduce the number of symbolic variables by substituting the given values for some variables.
- When dividing by zero, `mldivide` considers the numerator's sign and returns `Inf` or `-Inf` accordingly.

```
syms x
[sym(0)\sym(1), sym(0)\sym(-1), sym(0)\x]

ans =
[ Inf, -Inf, Inf*x]
```

## Version History

**Introduced before R2006a**

### **R2022a: Solve system of linear equations involving symbolic matrix functions**

The `mldivide` function accepts input arguments of type `symfunmatrix`.

### **R2021a: Solve system of linear equations involving symbolic matrix variables**

The `mldivide` function accepts input arguments of type `symmatrix`.

## See Also

`ctranspose` | `ldivide` | `minus` | `mpower` | `mrdivide` | `mtimes` | `plus` | `power` | `rdivide` | `times` | `transpose`

# mod

Symbolic modulus after division

---

**Note** Starting in R2020b, `mod` no longer finds the modulus for each coefficient of a symbolic polynomial. For more information, see “Version History”.

---

## Syntax

`m = mod(a,b)`

## Description

`m = mod(a,b)` finds the modulus on page 6-1078 after division. To find the remainder, use `rem`.

If `a` is a polynomial expression, then `mod(a,b)` returns the unevaluated modulus of the polynomial.

## Examples

### Find Modulus of Integers Divided by Integers

Find the modulus after division when both the dividend and divisor are integers.

Find the modulus after division for these numbers.

```
m = [mod(sym(27),4), mod(sym(27),-4), mod(sym(-27),4), mod(sym(-27),-4)]
```

```
m = (3 -1 1 -3)
```

### Find Modulus of Rationals Divided by Integers

Find the modulus after division when the dividend is a rational number, and the divisor is an integer.

Find the modulus after division for these numbers.

```
m = [mod(sym(22/3),5), mod(sym(1/2),7), mod(sym(27/6),-11)]
```

```
m =
```

$$\left(\frac{7}{3} \frac{1}{2} - \frac{13}{2}\right)$$

### Find Modulus of Polynomial Expression Divided by Integer

Find the modulus after division when the dividend is a polynomial expression, and the divisor is an integer. If the dividend is a polynomial expression, then `mod` returns a symbolic expression without evaluating the modulus.

Find the modulus after division by 10 for the polynomial  $x^3 - 2x + 999$ .

```
syms x
a = x^3 - 2*x + 999;
mUneval = mod(a,10)

mUneval = x^3 - 2x + 999 mod 10
```

To evaluate the modulus for each polynomial coefficient, first extract the coefficients of each term using `coeffs`.

```
[c,t] = coeffs(a)

c = (1 -2 999)

t = (x^3 x 1)
```

Next, find the modulus of each coefficient in `c` divided by 10. Reconstruct a new polynomial using the evaluated coefficients.

```
cMod10 = mod(c,10);
mEval = sum(cMod10.*t)

mEval = x^3 + 8x + 9
```

### Find Modulus of Symbolic Matrices

For vectors and matrices, `mod` finds the modulus after division element-wise. When both arguments are nonscalar, they must have the same size. If one argument is a scalar, the `mod` function expands the scalar input into an array of the same size as the other input.

Find the modulus after division for the elements of two matrices.

```
A = sym([27,28; 29,30]);
B = sym([2,3; 4,5]);
M = mod(A,B)
```

```
M =
    (1 1)
    (1 0)
```

Find the modulus after division for the elements of matrix `A` and the value 9. Here, `mod` expands 9 into the 2-by-2 matrix with all elements equal to 9.

```
M = mod(A,9)

M =
    (0 1)
    (2 3)
```

### Create Periodic Sawtooth Waves

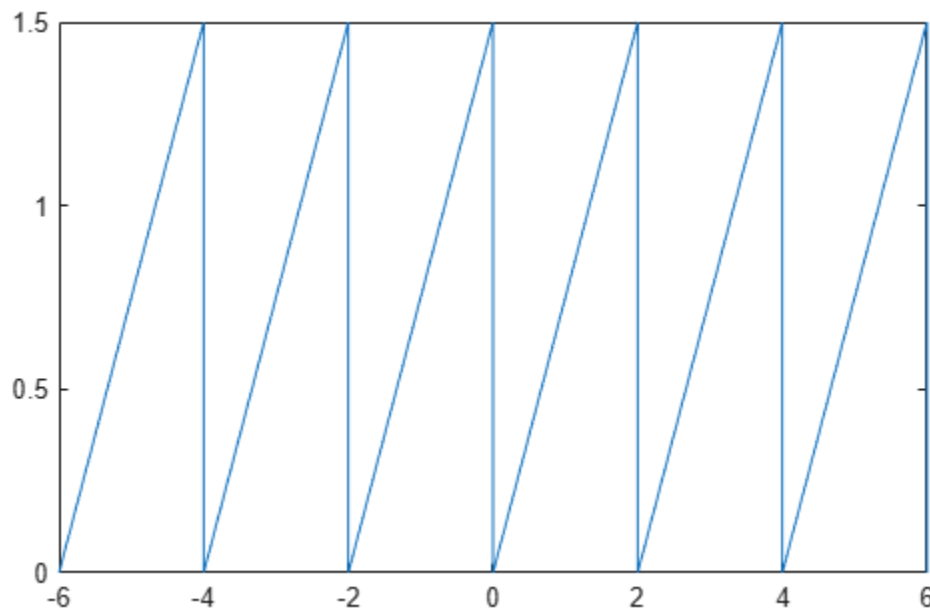
Create two periodic functions that represents sawtooth waves.

Define the sawtooth wave with period  $T = 2$  and amplitude  $A = 1.5$ . Create a symbolic function  $y(x)$ . Use `mod` functions to define the sawtooth wave for each period. The sawtooth wave increases linearly for a full period, and it drops back to zero at the start of another period.

```
T = 2;
A = 1.5;
syms y(x);
y(x) = A*mod(x,T)/T;
```

Plot this sawtooth wave for the interval  $[-6 \ 6]$ .

```
fplot(y,[-6 6])
```

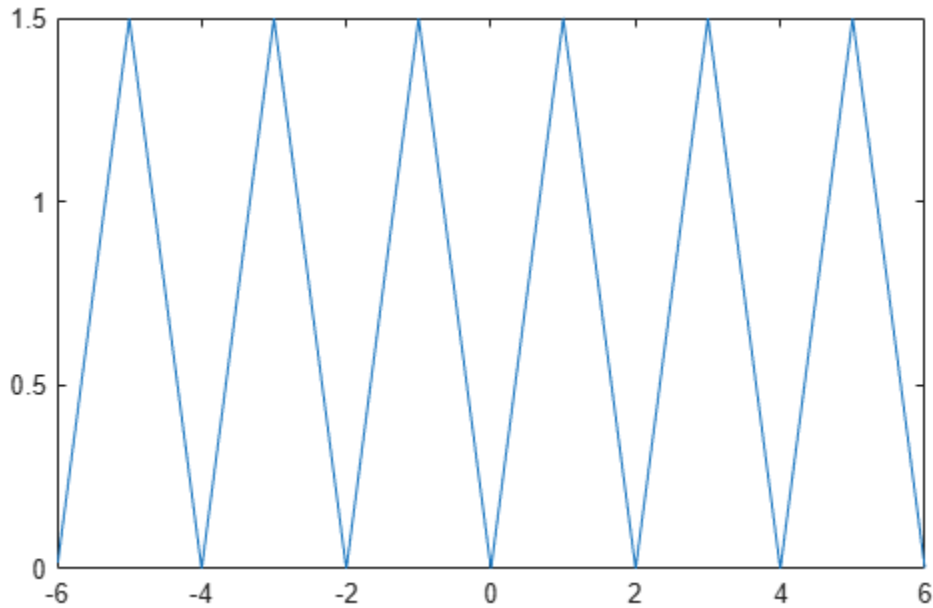


Next, create another sawtooth wave that is symmetrical within a single period. Use `piecewise` to define the sawtooth wave that is increasing linearly for the first half of a period, and then decreasing linearly for the second half of a period.

```
y(x) = piecewise(0 < mod(x,T) <= (T/2), 2*A*mod(x,T)/T, ...
(T/2) < mod(x,T) <= T, 2*A - 2*A*mod(x,T)/T);
```

Plot this sawtooth wave for the interval  $[-6 \ 6]$ .

```
fplot(y,[-6 6])
```



## Input Arguments

### **a — Dividend (numerator)**

number | symbolic number | symbolic variable | polynomial expression | vector | matrix

Dividend (numerator), specified as a number, symbolic number, variable, polynomial expression, or a vector or matrix of numbers, symbolic numbers, variables, or polynomial expressions. Inputs **a** and **b** must be the same size unless one is a scalar. The function expands a scalar input into an array of the same size as the other input.

### **b — Divisor (denominator)**

number | symbolic number | vector | matrix

Divisor (denominator), specified as a number, symbolic number, or a vector or matrix of numbers or symbolic numbers. Inputs **a** and **b** must be the same size unless one is a scalar. The function expands a scalar input into an array of the same size as the other input.

## More About

### **Modulus**

The modulus of  $a$  and  $b$  is

$$\text{mod}(a, b) = a - b \cdot \text{floor}\left(\frac{a}{b}\right),$$

where `floor` rounds  $(a / b)$  toward negative infinity. For example, the modulus of -8 and -3 is -2, but the modulus of -8 and 3 is 1.

If  $b = 0$ , then  $\text{mod}(a, b) = \text{mod}(a, 0) = 0$ .

## Tips

- Calling `mod` for numbers that are not symbolic objects invokes the MATLAB `mod` function.

## Version History

Introduced before R2006a

### **R2020b: `mod` no longer finds the modulus for each coefficient of a symbolic polynomial**

*Behavior changed in R2020b*

Starting in R2020b, `mod` no longer finds the modulus for each coefficient of a symbolic polynomial. Instead, `mod(a,b)` returns an unevaluated symbolic expression if `a` is a symbolic polynomial and `b` is a real number. To find the modulus for each coefficient of the polynomial `a`, use `[c,t] = coeffs(a); sum(mod(c,b).*t)`. You can now create periodic symbolic functions by defining the periodicity using `mod`. For example, see “Create Periodic Sawtooth Waves” on page 6-1076.

## See Also

`powermod` | `quorem` | `rem` | `coeffs`

## mpower, ^

Symbolic matrix power

### Syntax

$A^B$   
`mpower(A,B)`

### Description

$A^B$  computes  $A$  to the  $B$  power.

`mpower(A,B)` is equivalent to  $A^B$ .

### Examples

#### Matrix Base and Scalar Exponent

Create a 2-by-2 matrix.

```
A = sym('a%d%d', [2 2])
```

```
A =  
[ a11, a12]  
[ a21, a22]
```

Find  $A^2$ .

```
A^2  
  
ans =  
[ a11^2 + a12*a21, a11*a12 + a12*a22]  
[ a11*a21 + a21*a22, a22^2 + a12*a21]
```

#### Scalar Base and Matrix Exponent

Create a 2-by-2 symbolic magic square.

```
A = sym(magic(2))
```

```
A =  
[ 1, 3]  
[ 4, 2]
```

Find  $\pi^A$ .

```
sym(pi)^A  
  
ans =  
[ (3*pi^7 + 4)/(7*pi^2), (3*(pi^7 - 1))/(7*pi^2)]  
[ (4*(pi^7 - 1))/(7*pi^2), (4*pi^7 + 3)/(7*pi^2)]
```



## Input Arguments

### A — Base

number | symbolic number | symbolic scalar variable | symbolic function | symbolic matrix function | symbolic expression | square symbolic matrix variable | square matrix of symbolic scalar variables

Base, specified as a number or a symbolic number, scalar variable, function, matrix function, expression, square symbolic matrix variable, or square matrix of symbolic scalar variables. A and B must be one of the following:

- Both are scalars.
- A is a square matrix, and B is a scalar.
- B is a square matrix, and A is a scalar.

### B — Exponent

number | symbolic number | symbolic scalar variable | symbolic function | symbolic expression | square matrix of symbolic scalar variables

Exponent, specified as a number or a symbolic number, scalar variable, function, expression, or square matrix of symbolic scalar variables. A and B must be one of the following:

- Both are scalars.
- A is a square matrix, and B is a scalar.
- B is a square matrix, and A is a scalar.

## Version History

**Introduced before R2006a**

### R2022a: Compute matrix power of symbolic matrix functions

The `mpower` function accepts an input argument of type `symfunmatrix`.

### R2021a: Compute matrix power of symbolic matrix variables

The `mpower` function accepts an input argument of type `symmatrix`.

### See Also

`ctranspose` | `ldivide` | `minus` | `mldivide` | `mrdivide` | `mtimes` | `plus` | `power` | `rdivide` | `times` | `transpose`

## mrdivide, /

Symbolic matrix right division

### Syntax

```
X = B/A  
X = mrdivide(B,A)
```

### Description

$X = B/A$  solves the symbolic system of linear equations in matrix form,  $X*A = B$  for  $X$ . The matrices  $A$  and  $B$  must contain the same number of columns. The right division of matrices  $B/A$  is equivalent to  $(A'\backslash B')'$ .

If the solution does not exist or if it is not unique, the `/` operator issues a warning.

$A$  can be a rectangular matrix, but the equations must be consistent. The symbolic operator `/` does not compute least-squares solutions.

$X = \text{mrdivide}(B,A)$  is equivalent to  $x = B/A$ .

### Examples

#### System of Equations in Matrix Form

Solve a system of linear equations specified by a square matrix of coefficients and a vector of right sides of equations.

Create a matrix containing the coefficient of equation terms, and a vector containing the right sides of equations.

```
A = sym(pascal(4))  
b = sym([4 3 2 1])
```

```
A =  
[ 1, 1, 1, 1]  
[ 1, 2, 3, 4]  
[ 1, 3, 6, 10]  
[ 1, 4, 10, 20]
```

```
b =  
[ 4, 3, 2, 1]
```

Use the operator `/` to solve this system.

```
X = b/A  
  
X =  
[ 5, -1, 0, 0]
```

## Rank-Deficient System

Create a matrix containing the coefficient of equation terms, and a vector containing the right sides of equations.

```
A = sym(magic(4))'
b = sym([0 1 1 0])
```

```
A =
[ 16,  5,  9,  4]
[  2, 11,  7, 14]
[  3, 10,  6, 15]
[ 13,  8, 12,  1]
```

```
b =
[ 0, 1, 1, 0]
```

Find the rank of the system. This system contains four equations, but its rank is 3. Therefore, the system is rank-deficient. This means that one variable of the system is not independent and can be expressed in terms of other variables.

```
rank(vercat(A,b))
```

```
ans =
3
```

Try to solve this system using the symbolic / operator. Because the system is rank-deficient, the returned solution is not unique.

```
b/A
```

```
Warning: Solution is not unique because the system is rank-deficient.
```

```
ans =
[ 1/34, 19/34, -9/17, 0]
```

## Inconsistent System

Create a matrix containing the coefficient of equation terms, and a vector containing the right sides of equations.

```
A = sym(magic(4))'
b = sym([0 1 2 3])
```

```
A =
[ 16,  5,  9,  4]
[  2, 11,  7, 14]
[  3, 10,  6, 15]
[ 13,  8, 12,  1]
```

```
b =
[ 0, 1, 2, 3]
```

Try to solve this system using the symbolic / operator. The operator issues a warning and returns a vector with all elements set to `Inf` because the system of equations is inconsistent, and therefore, no solution exists. The number of elements equals the number of equations (rows in the coefficient matrix).

```
b/A
```

Warning: Solution does not exist because the system is inconsistent.

```
ans =  
[ Inf, Inf, Inf, Inf]
```

Find the reduced row echelon form of this system. The last row shows that one of the equations reduced to  $0 = 1$ , which means that the system of equations is inconsistent.

```
rref(vertcat(A,b)')
```

```
ans =  
[ 1, 0, 0, 1, 0]  
[ 0, 1, 0, 3, 0]  
[ 0, 0, 1, -3, 0]  
[ 0, 0, 0, 0, 1]
```

## Input Arguments

### A — Coefficient matrix

symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix

Coefficient matrix, specified as a symbolic number, scalar variable, matrix variable, function, matrix function, expression, or vector or matrix of symbolic scalar variables.

### B — Right side

symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix

Right side, specified as a symbolic number, scalar variable, matrix variable, function, matrix function, expression, or vector or matrix of symbolic scalar variables.

## Output Arguments

### X — Solution

symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix

Solution, returned as a symbolic number, scalar variable, matrix variable, function, matrix function, expression, or vector or matrix of symbolic scalar variables.

## Tips

- Matrix computations involving many symbolic variables can be slow. To increase the computational speed, reduce the number of symbolic variables by substituting the given values for some variables.
- When dividing by zero, `mrdivide` considers the numerator's sign and returns `Inf` or `-Inf` accordingly.

```
syms x  
[sym(1)/sym(0), sym(-1)/sym(0), x/sym(0)]  
  
ans =  
[ Inf, -Inf, Inf*x]
```

## Version History

Introduced before R2006a

### **R2022a: Solve system of linear equations involving symbolic matrix functions**

The `mrdivide` function accepts input arguments of type `symfunmatrix`.

### **R2021a: Solve system of linear equations involving symbolic matrix variables**

The `mrdivide` function accepts input arguments of type `symmatrix`.

### **See Also**

`ctranspose` | `ldivide` | `minus` | `mldivide` | `mpower` | `mtimes` | `plus` | `power` | `rdivide` | `times` | `transpose`

## mtimes, \*

Symbolic matrix multiplication

### Syntax

$A*B$   
`mtimes(A,B)`

### Description

$A*B$  is the matrix product of  $A$  and  $B$ . If  $A$  is an  $m$ -by- $p$  and  $B$  is a  $p$ -by- $n$  matrix, then the result is an  $m$ -by- $n$  matrix  $C$  defined as

$$C(i, j) = \sum_{k=1}^p A(i, k)B(k, j)$$

For nonscalar  $A$  and  $B$ , the number of columns of  $A$  must equal the number of rows of  $B$ . Matrix multiplication is not universally commutative for nonscalar inputs. That is, typically  $A*B$  is not equal to  $B*A$ . If at least one input is scalar, then  $A*B$  is equivalent to  $A.*B$  and is commutative.

`mtimes(A,B)` is equivalent to  $A*B$ .

### Examples

#### Multiply Two Vectors

Create a 1-by-5 row vector and a 5-by-1 column vector.

```
syms x
A = [x, 2*x^2, 3*x^3, 4*x^4]
B = [1/x; 2/x^2; 3/x^3; 4/x^4]
```

```
A =
[ x, 2*x^2, 3*x^3, 4*x^4]
```

```
B =
 1/x
2/x^2
3/x^3
4/x^4
```

Find the matrix product of these two vectors.

$A*B$

```
ans =
30
```

#### Multiply Two Matrices

Create a 4-by-3 matrix and a 3-by-2 matrix.

```
A = sym('a%d%d', [4 3])
B = sym('b%d%d', [3 2])
```

```
A =
[ a11, a12, a13]
[ a21, a22, a23]
[ a31, a32, a33]
[ a41, a42, a43]
```

```
B =
[ b11, b12]
[ b21, b22]
[ b31, b32]
```

Multiply A by B.

A\*B

```
ans =
[ a11*b11 + a12*b21 + a13*b31, a11*b12 + a12*b22 + a13*b32]
[ a21*b11 + a22*b21 + a23*b31, a21*b12 + a22*b22 + a23*b32]
[ a31*b11 + a32*b21 + a33*b31, a31*b12 + a32*b22 + a33*b32]
[ a41*b11 + a42*b21 + a43*b31, a41*b12 + a42*b22 + a43*b32]
```

### Multiply Matrix by Scalar

Create a 4-by-4 Hilbert matrix H.

```
H = sym(hilb(4))
```

```
H =
[ 1, 1/2, 1/3, 1/4]
[ 1/2, 1/3, 1/4, 1/5]
[ 1/3, 1/4, 1/5, 1/6]
[ 1/4, 1/5, 1/6, 1/7]
```

Multiply H by  $e^\pi$ .

```
C = H*exp(sym(pi))
```

```
C =
[ exp(pi), exp(pi)/2, exp(pi)/3, exp(pi)/4]
[ exp(pi)/2, exp(pi)/3, exp(pi)/4, exp(pi)/5]
[ exp(pi)/3, exp(pi)/4, exp(pi)/5, exp(pi)/6]
[ exp(pi)/4, exp(pi)/5, exp(pi)/6, exp(pi)/7]
```

Use `vpa` and `digits` to approximate symbolic results with the required number of digits. For example, approximate it with five-digit accuracy.

```
old = digits(5);
vpa(C)
digits(old)
```

```
ans =
[ 23.141, 11.57, 7.7136, 5.7852]
[ 11.57, 7.7136, 5.7852, 4.6281]
[ 7.7136, 5.7852, 4.6281, 3.8568]
[ 5.7852, 4.6281, 3.8568, 3.3058]
```

## Input Arguments

### A — Input

symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix

Input, specified as a symbolic number, scalar variable, matrix variable, function, matrix function, expression, or vector or matrix of symbolic scalar variables. Inputs A and B must be the same size unless one is a scalar. A scalar value expands into an array of the same size as the other input.

### B — Input

symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix

Input, specified as a symbolic number, scalar variable, matrix variable, function, matrix function, expression, or vector or matrix of symbolic scalar variables. Inputs A and B must be the same size unless one is a scalar. A scalar value expands into an array of the same size as the other input.

## Version History

**Introduced before R2006a**

### **R2022a: Compute matrix multiplication of symbolic matrix functions**

The `mtimes` function accepts an input argument of type `symfunmatrix`.

### **R2021a: Compute matrix multiplication of symbolic matrix variables**

The `mtimes` function accepts an input argument of type `symmatrix`.

### See Also

`ctranspose` | `ldivide` | `minus` | `mldivide` | `mpower` | `mrdivide` | `plus` | `power` | `rdivide` | `times` | `transpose`



# nchoosek

Binomial coefficient

## Syntax

```
b = nchoosek(n,k)
C = nchoosek(v,k)
```

## Description

`b = nchoosek(n,k)` returns the binomial coefficient of  $n$  and  $k$ , defined as  $n!/(k!(n-k)!)$ . This is the number of combinations of  $n$  items taken  $k$  at a time.

`C = nchoosek(v,k)` returns a matrix containing all possible combinations of the elements of vector  $v$  taken  $k$  at a time. Matrix  $C$  has  $k$  columns and  $n!/(k!(n-k)!)$  rows, where  $n$  is `length(v)`. In this syntax,  $k$  must be a nonnegative integer.

## Examples

### Binomial Coefficients for Numeric and Symbolic Arguments

Compute the binomial coefficients for these expressions.

```
syms n
[nchoosek(n, n), nchoosek(n, n + 1), nchoosek(n, n - 1)]

ans =
[ 1, 0, n]
```

If one or both parameters are negative numbers, convert these numbers to symbolic objects.

```
[nchoosek(sym(-1), 3), nchoosek(sym(-7), 2), nchoosek(sym(-5), -5)]

ans =
[ -1, 28, 1]
```

If one or both parameters are complex numbers, convert these numbers to symbolic objects.

```
[nchoosek(sym(i), 3), nchoosek(sym(i), i), nchoosek(sym(i), i + 1)]

ans =
[ 1/2 + 1i/6, 1, 0]
```

### Handle Expressions Containing Binomial Coefficients

Many functions, such as `diff` and `expand`, can handle expressions containing `nchoosek`.

Differentiate the binomial coefficient.

```
syms n k
diff(nchoosek(n, 2))
```

```
ans =  
-(psi(n - 1) - psi(n + 1))*nchoosek(n, 2)
```

Expand the binomial coefficient.

```
expand(nchoosek(n, k))
```

```
ans =  
-(n*gamma(n))/(k^2*gamma(k)*gamma(n - k) - k*n*gamma(k)*gamma(n - k))
```

### Pascal Triangle

Use `nchoosek` to build the Pascal triangle.

```
m = 5;  
for n = 0:m  
    C = sym([]);  
    for k = 0:n  
        C = horzcat(C, nchoosek(n, k));  
    end  
    disp(C)  
end
```

```
1  
[ 1, 1]  
[ 1, 2, 1]  
[ 1, 3, 3, 1]  
[ 1, 4, 6, 4, 1]  
[ 1, 5, 10, 10, 5, 1]
```

### All Combinations of Vector Elements

Find all combinations of elements of a 1-by-5 symbolic row vector taken three and four at a time.

Create a 1-by-5 symbolic vector with the elements `x1`, `x2`, `x3`, `x4`, and `x5`.

```
v = sym('x', [1, 5])
```

```
v =  
[ x1, x2, x3, x4, x5]
```

Find all combinations of the elements of `v` taken three at a time.

```
C = nchoosek(v, 3)
```

```
C =  
[ x1, x2, x3]  
[ x1, x2, x4]  
[ x1, x3, x4]  
[ x2, x3, x4]  
[ x1, x2, x5]  
[ x1, x3, x5]  
[ x2, x3, x5]  
[ x1, x4, x5]  
[ x2, x4, x5]  
[ x3, x4, x5]
```

```
C = nchoosek(v, 4)
```

```

C =
[ x1, x2, x3, x4]
[ x1, x2, x3, x5]
[ x1, x2, x4, x5]
[ x1, x3, x4, x5]
[ x2, x3, x4, x5]

```

## Input Arguments

### **n — Number of possible choices**

symbolic number | symbolic variable | symbolic expression | symbolic function

Number of possible choices, specified as a symbolic number, variable, expression, or function.

### **k — Number of selected choices**

symbolic number | symbolic variable | symbolic expression | symbolic function

Number of selected choices, specified as a symbolic number, variable, expression, or function. If the first argument is a symbolic vector  $v$ , then  $k$  must be a nonnegative integer.

### **v — Set of all choices**

symbolic vector

Set of all choices, specified as a vector of symbolic numbers, variables, expressions, or functions.

## Output Arguments

### **b — Binomial coefficient**

nonnegative scalar value

Binomial coefficient, returned as a nonnegative scalar value.

### **C — All combinations of v**

matrix

All combinations of  $v$ , returned as a matrix of the same type as  $v$ .

## More About

### **Binomial Coefficient**

If  $n$  and  $k$  are integers and  $0 \leq k \leq n$ , the binomial coefficient is defined as:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

For complex numbers, the binomial coefficient is defined via the gamma function:

$$\binom{n}{k} = \frac{\Gamma(n+1)}{\Gamma(k+1)\Gamma(n-k+1)}$$

## Tips

- Calling `nchoosek` for numbers that are not symbolic objects invokes the MATLAB `nchoosek` function.
- If one or both parameters are complex or negative numbers, convert these numbers to symbolic objects using `sym`, and then call `nchoosek` for those symbolic objects.

## Algorithms

If  $k < 0$  or  $n - k < 0$ , `nchoosek(n,k)` returns 0.

If one or both arguments are complex, `nchoosek` uses the formula representing the binomial coefficient via the `gamma` function.

## Version History

Introduced in R2012a

## See Also

`beta` | `gamma` | `factorial` | `psi`

# ne

Define inequality

## Syntax

```
A ~= B
ne(A,B)
```

## Description

$A \neq B$  creates a symbolic inequality.

`ne(A,B)` is equivalent to  $A \neq B$ .

## Examples

### Set and Use Assumption Using Not Equal

Use `assume` and the relational operator `~=` to set the assumption that `x` does not equal to 5:

```
syms x
assume(x ~= 5)
```

Solve this equation. The solver takes into account the assumption on variable `x`, and therefore returns only one solution.

```
solve((x - 5)*(x - 6) == 0, x)

ans =
6
```

## Input Arguments

### A — Input

number | vector | matrix | array | symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic array | symbolic function | symbolic matrix function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, scalar variable, matrix variable, array, function, matrix function, or expression.

### B — Input

number | vector | matrix | array | symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic array | symbolic function | symbolic matrix function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, scalar variable, matrix variable, array, function, matrix function, or expression.

## Tips

- Calling `~=` or `ne` for non-symbolic `A` and `B` invokes the MATLAB `ne` function. This function returns a logical array with elements set to logical 1 (`true`) where `A` is not equal to `B`; otherwise, it returns logical 0 (`false`).
- If both `A` and `B` are arrays, then these arrays must have the same dimensions. `A ~= B` returns an array of inequalities `A(i,j,...) ~= B(i,j,...)`
- If one input is scalar and the other an array, then the scalar input is expanded into an array of the same dimensions as the other array. In other words, if `A` is a variable (for example, `x`), and `B` is an  $m$ -by- $n$  matrix, then `A` is expanded into  $m$ -by- $n$  matrix of elements, each set to `x`.

## Alternatives

You can also define inequality using `eq` (or its shortcut `==`) and the logical negation `not` (or `~`). Thus, `A ~= B` is equivalent to `~(A == B)`.

## Version History

Introduced in R2012a

**R2022a: Define inequality of symbolic matrix functions and matrix variables**

The `ne` function accepts input arguments of type `symfunmatrix` and `symmatrix`.

## See Also

`eq` | `ge` | `gt` | `isAlways` | `le` | `lt`

## Topics

“Use Assumptions on Symbolic Variables” on page 1-41

# newUnit

Define new unit

## Syntax

```
c = newUnit(name,definition)
```

## Description

`c = newUnit(name,definition)` defines the new unit name using the expression `definition`. The definition must be in terms of existing symbolic units. You cannot redefine a predefined unit or any of its alternate names.

## Examples

### Define New Unit and Rewrite Unit

Load the collection of symbolic units by using `symunit`. Find information on the predefined unit `u.c_0` for the speed of light.

```
u = symunit;
unitInfo(u.c_0)
```

speed of light in vacuum - a physical unit of velocity.

Get all units for measuring 'Velocity' by calling `unitInfo('Velocity')`.

Show that the exact value of the speed of light in SI is 299792458 metres per second.

```
c = unitConvert(u.c_0,'SI')
```

```
c =
299792458*([m]/[s])
```

Define the new unit `speedOfLightApprox` for the approximate value of the speed of light as  $3e8$  meters per second.

```
u = symunit;
c = newUnit('speedOfLightApprox',3e8*u.m/u.s)
```

```
c =
[speedOfLightApprox]
```

Alternatively, you can specify the unit by using `u.speedOfLightApprox`.

Define the equation  $E = mc^2$  using the new unit.

```
syms mass
m = mass*u.kg;
E = m*c^2
```

```
E =
mass*[kg]*[speedOfLightApprox]^2
```

Rewrite E in terms of meters per second.

```
E = rewrite(E,u.m/u.s)
```

```
E =  
9000000000000000000*mass*([kg]*[m]^2)/[s]^2)
```

Since the standard unit of energy is the joule, rewrite E in terms of Joule.

```
E = rewrite(E,u.Joule)
```

```
E =  
9000000000000000000*mass*[J]
```

## Input Arguments

**name — Name of new unit**

character vector | string

Name of the new unit, specified as a character vector or string. You cannot redefine a predefined unit or any of its alternate names.

**definition — Definition of new unit**

symbolic expression of units

Definition of the new unit, specified as a symbolic expression of units. The new unit must be defined in terms of existing symbolic units. For example, `newUnit('workday',8*u.hour)`, where `u = symunit`, defines `workday` as a unit representing 8 hours.

## Version History

Introduced in R2017a

## See Also

`checkUnits` | `isUnit` | `removeUnit` | `separateUnits` | `symunit` | `str2symunit` | `symunit2str` | `unitConversionFactor`

## Topics

“Units of Measurement Tutorial” on page 2-47

“Unit Conversions and Unit Systems” on page 2-53

“Units and Unit Systems List” on page 2-60

## External Websites

The International System of Units (SI)



# newUnitSystem

Define unit system

## Syntax

```
newUnitSystem(name,baseUnits)
newUnitSystem(name,baseUnits,derivedUnits)
```

## Description

`newUnitSystem(name,baseUnits)` defines a new “Unit System” on page 6-1099 with the name `name` and the base units `baseUnits`. Now, you can convert units into the new unit system by using `rewrite`. By default, available unit systems include SI, CGS, and US. For all unit systems, see “Unit Systems List” on page 2-71.

`newUnitSystem(name,baseUnits,derivedUnits)` additionally specifies the derived units `derivedUnits`.

## Examples

### Define New Unit System from Existing System

A unit system is a collection of units to express quantities. The easiest way to define a new unit system is to modify a default unit system, such as SI, CGS, or US.

Modify SI to use kilometer for length and hour for time by getting the base units using `baseunits` and modifying them by using `subs`.

```
u = symunit;
SIUnits = baseUnits('SI')

SIUnits =
[ [kg], [s], [m], [A], [cd], [mol], [K]]

newUnits = subs(SIUnits,[u.m u.s],[u.km u.hr])

newUnits =
[ [kg], [h], [km], [A], [cd], [mol], [K]]
```

---

**Note** Do not define a variable called `baseUnits` because the variable will prevent access to the `baseUnits` function.

---

Define the new unit system `SI_km_hr` using the new base units.

```
newUnitSystem('SI_km_hr',newUnits)

ans =
"SI_km_hr"
```

Rewrite 5 meter/second to the SI\_km\_hr unit system. As expected, the result is in terms of kilometers and hours.

```
rewrite(5*u.m/u.s, 'SI_km_hr')

ans =
18*([km]/[h])
```

### Specify Base and Derived Units Directly

Specify a new unit system by specifying the base and derived units directly. A unit system has up to 7 base units. For details, see “Unit System” on page 6-1099.

Define a new unit system with these base units: gram, hour, meter, ampere, candela, mol, and celsius. Specify these derived units: kilowatt, newton, and volt.

```
u = symunit;
sysName = 'myUnitSystem';
bunits = [u.g u.hr u.m u.A u.cd u.mol u.Celsius];
dunits = [u.kW u.N u.V];
newUnitSystem(sysName,bunits,dunits)

ans =
    "myUnitSystem"
```

Rewrite 2000 Watts to the new system. By default, `rewrite` uses base units, which can be hard to read.

```
rewrite(2000*u.W,sysName)

ans =
9331200000000000000*(([g]*[m]^2)/[h]^3)
```

Instead, for readability, rewrite 2000 Watts to *derived* units of `myUnitSystem` by specifying 'Derived' as the third argument. Converting to the derived units of a unit system attempts to select convenient units. The result uses the derived unit, kilowatt, instead of base units. For more information, see “Unit Conversions and Unit Systems” on page 2-53.

```
rewrite(2000*u.W,sysName,'Derived')

ans =
2*[kW]
```

## Input Arguments

### name — Name of unit system

string | character vector

Name of unit system, specified as a string or character vector.

### baseUnits — Base units of unit system

vector of symbolic units

Base units of unit system, specified as a vector of symbolic units. The base units must be independent in terms of the dimensions mass, time, length, electric current, luminous intensity, amount of substance, and temperature. Thus, in a unit system, there are up to 7 base units.

### **derivedUnits — Derived units of unit system**

vector of symbolic units

Derived units of unit system, specified as a vector of symbolic units. Derived units are optional and added for convenience of representation.

## **More About**

### **Unit System**

A unit system is a collection of base units and derived units that follows these rules:

- Base units must be independent in terms of the dimensions mass, time, length, electric current, luminous intensity, amount of substance, and temperature. Therefore, a unit system has up to 7 base units. As long as the independence is satisfied, any unit can be a base unit, including units such as newton or watt.
- A unit system can have less than 7 base units. For example, mechanical systems need base units only for the dimensions length, mass, and time.
- Derived units in a unit system must have a representation in terms of the products of powers of the base units for that system. Unlike base units, derived units do not have to be independent.
- Derived units are optional and added for convenience of representation. For example,  $\text{kg m/s}^2$  is abbreviated by newton.
- An example of a unit system is the SI unit system, which has 7 base units: kilogram, second, meter, ampere, candela, mol, and kelvin. There are 22 derived units found by calling `derivedUnits('SI')`.

## **Version History**

**Introduced in R2017b**

### **See Also**

`baseUnits` | `derivedUnits` | `newUnit` | `removeUnitSystem` | `rewrite` | `symunit` | `unitSystems`

### **Topics**

“Units of Measurement Tutorial” on page 2-47

“Unit Conversions and Unit Systems” on page 2-53

“Units and Unit Systems List” on page 2-60

### **External Websites**

The International System of Units (SI)

## nextprime

Next prime number

### Syntax

```
nextprime(n)
```

### Description

`nextprime(n)` returns the next prime number greater than or equal to `n`. If `n` is a vector or matrix, then `nextprime` acts element-wise on `n`.

### Examples

#### Find Next Prime Number

Find the next prime number greater than 100.

```
nextprime(100)
```

```
ans =  
101
```

Find the next prime numbers greater than 1000, 10000, and 100000 by specifying the input as a vector.

```
v = [1000 10000 100000];  
nextprime(v)
```

```
ans =  
      1009      10007     100003
```

#### Find Large Prime Number

When finding large prime numbers, return exact symbolic integers by using symbolic input. Further, if your input has 15 or more digits, then use quotation marks and wrap the number in `sym` to represent the number accurately. For more information, see “Numeric to Symbolic Conversion” on page 2-29.

Find a large prime number by using `10^sym(18)`.

```
nextprime(10^sym(18))
```

```
ans =  
10000000000000000003
```

Find the next prime number to 823572345728582545 by using quotation marks.

```
nextprime(sym('823572345728582545'))
```

```
ans =  
823572345728582623
```

## Input Arguments

### **n — Input**

number | vector | matrix | array | symbolic number | symbolic array

Input, specified as a number, vector, matrix, array, or a symbolic number or array.

## Version History

**Introduced in R2016b**

### **See Also**

isprime | nthprime | prevprime | primes

## norm

Norm of symbolic vector or matrix

### Syntax

```
n = norm(v)
n = norm(v,p)

n = norm(A)
n = norm(A,P)

n = norm(X,"fro")
```

### Description

`n = norm(v)` returns the 2-norm or the magnitude of symbolic vector `v`.

`n = norm(v,p)` returns the `p`-norm of symbolic vector `v`.

`n = norm(A)` returns the 2-norm of symbolic matrix `A`. Because symbolic variables are assumed to be complex by default, the norm can contain unresolved calls to `conj` and `abs`.

`n = norm(A,P)` returns the `P`-norm of symbolic matrix `A`.

`n = norm(X,"fro")` returns the Frobenius norm of symbolic multidimensional array `X`.

### Examples

#### Magnitude of Symbolic Vector

Create a symbolic vector and calculate the magnitude.

```
syms x y z
r = [x y z]

r = (x y z)

n = norm(r)

n =  $\sqrt{|x|^2 + |y|^2 + |z|^2}$ 
```

#### Compute 2-Norm of Matrix

Compute the 2-norm of the inverse of the 3-by-3 magic square `A`.

```
A = inv(sym(magic(3)))

A =
```

$$\begin{pmatrix} \frac{53}{360} & -\frac{13}{90} & \frac{23}{360} \\ -\frac{11}{180} & \frac{1}{45} & \frac{19}{180} \\ -\frac{7}{360} & \frac{17}{90} & -\frac{37}{360} \end{pmatrix}$$

```
norm2 = norm(A)
```

```
norm2 =  
 $\frac{\sqrt{3}}{6}$ 
```

Use `vpa` to approximate the result with 20-digit accuracy.

```
norm2_vpa = vpa(norm2,20)
```

```
norm2_vpa = 0.28867513459481288225
```

### Effect of Assumptions on Norm

Compute the norm of  $[x \ y]$  and simplify the result. Because symbolic scalar variables are assumed to be complex by default, the calls to `abs` do not simplify.

```
syms x y  
n = simplify(norm([x y]))
```

```
n =  $\sqrt{|x|^2 + |y|^2}$ 
```

Assume  $x$  and  $y$  are real, and repeat the calculation. Now, the result is simplified.

```
assume([x y], "real")  
n = simplify(norm([x y]))
```

```
n =  $\sqrt{x^2 + y^2}$ 
```

Remove assumptions on  $x$  for further calculations. For details, see “Use Assumptions on Symbolic Variables” on page 1-41.

```
assume(x, "clear")
```

### Compute Different Types of Norms of Matrix

Compute the 1-norm, Frobenius norm, and infinity norm of the inverse of the 3-by-3 magic square  $A$ .

```
A = inv(sym(magic(3)))
```

```
A =  
 $\begin{pmatrix} \frac{53}{360} & -\frac{13}{90} & \frac{23}{360} \\ -\frac{11}{180} & \frac{1}{45} & \frac{19}{180} \\ -\frac{7}{360} & \frac{17}{90} & -\frac{37}{360} \end{pmatrix}$ 
```

```
norm1 = norm(A,1)

norm1 =
    16
   45

normf = norm(A,"fro")

normf =
    sqrt(391)
    60

normi = norm(A,Inf)

normi =
    16
   45
```

Use `vpa` to approximate these results to 20-digit accuracy.

```
norm1_vpa = vpa(norm1,20)
norm1_vpa = 0.35555555555555555555555556
normf_vpa = vpa(normf,20)
normf_vpa = 0.32956199888808647519
normi_vpa = vpa(normi,20)
normi_vpa = 0.35555555555555555555555556
```

### Compute Different Types of Norms of Vector

Compute the 1-norm, 2-norm, and 3-norm of the column vector  $V = [V_x; V_y; V_z]$ .

```
syms Vx Vy Vz
V = [Vx; Vy; Vz];
norm1 = norm(V,1)

norm1 = |Vx| + |Vy| + |Vz|

norm2 = norm(V)

norm2 = sqrt(|Vx|^2 + |Vy|^2 + |Vz|^2)

norm3 = norm(V,3)

norm3 = (|Vx|^3 + |Vy|^3 + |Vz|^3)^(1/3)
```

Compute the infinity norm, negative infinity norm, and Frobenius norm of  $V$ .

```
normi = norm(V,Inf)

normi = max(|Vx|, |Vy|, |Vz|)

normni = norm(V,-Inf)
```



$$\text{norm}_1 = \min(|V_x|, |V_y|, |V_z|)$$

$$\text{norm}_f = \text{norm}(V, \text{"fro"})$$

$$\text{norm}_f = \sqrt{|V_x|^2 + |V_y|^2 + |V_z|^2}$$

## Input Arguments

### **v — Input vector**

vector of symbolic scalar variables | symbolic matrix variable | symbolic function | symbolic matrix function

Input vector, specified as a vector of symbolic scalar variables, symbolic matrix variable, function, or matrix function that represents a vector.

### **p — Input**

2 (default) | 1 | positive integer scalar | Inf | -Inf | "fro"

- $\text{norm}(v, p)$  is computed as  $\sum(\text{abs}(v) .^p)^{(1/p)}$  for  $1 \leq p < \text{Inf}$ .
- $\text{norm}(v)$  computes the 2-norm of  $V$ .
- $\text{norm}(v, \text{Inf})$  is computed as  $\max(\text{abs}(V))$ .
- $\text{norm}(v, -\text{Inf})$  is computed as  $\min(\text{abs}(V))$ .

### **A — Input matrix**

matrix of symbolic scalar variables | symbolic matrix variable | symbolic function | symbolic matrix function

Input matrix, specified as a matrix of symbolic scalar variables, symbolic matrix variable, function, or matrix function that represents a matrix.

### **P — Input**

2 (default) | 1 | Inf | "fro"

One of these values 1, 2, Inf, or "fro".

- $\text{norm}(A, 1)$  returns the 1-norm of  $A$ .
- $\text{norm}(A, 2)$  or  $\text{norm}(A)$  returns the 2-norm of  $A$ .
- $\text{norm}(A, \text{Inf})$  returns the infinity norm of  $A$ .
- $\text{norm}(A, \text{"fro"})$  returns the Frobenius norm of  $A$ .

### **X — Input array**

multidimensional array of symbolic scalar variables

Input array, specified as a multidimensional array of symbolic scalar variables.

## More About

### **1-Norm of a Matrix**

The 1-norm of an  $m$ -by- $n$  matrix  $A$  is defined as follows:

$$\|A\|_1 = \max_j \left( \sum_{i=1}^m |A_{ij}| \right), \text{ where } j = 1 \dots n$$

### 2-Norm of a Matrix

The 2-norm of an  $m$ -by- $n$  matrix  $A$  is defined as follows:

$$\|A\|_2 = \sqrt{\max \text{ eigenvalue of } A^H A}$$

The 2-norm is also called the spectral norm of a matrix.

### Infinity Norm of a Matrix

The infinity norm of an  $m$ -by- $n$  matrix  $A$  is defined as follows:

$$\|A\|_\infty = \max \left( \sum_{j=1}^n |A_{1j}|, \sum_{j=1}^n |A_{2j}|, \dots, \sum_{j=1}^n |A_{mj}| \right)$$

### Frobenius Norm of a Matrix and Multidimensional Array

The Frobenius norm of an  $m$ -by- $n$  matrix  $A$  is defined as follows:

$$\|A\|_F = \sqrt{\sum_{i=1}^m \left( \sum_{j=1}^n |A_{ij}|^2 \right)}$$

The Frobenius norm of an  $l$ -by- $m$ -by- $n$  multidimensional array  $X$  is defined as follows:

$$\|X\|_F = \sqrt{\sum_{i=1}^l \left( \sum_{j=1}^m \left( \sum_{k=1}^n |X_{ijk}|^2 \right) \right)}$$

### P-Norm of a Vector

The P-norm of a 1-by- $n$  or  $n$ -by-1 vector  $V$  is defined as follows:

$$\|V\|_P = \left( \sum_{i=1}^n |V_i|^P \right)^{1/P}$$

Here  $n$  must be an integer greater than 1.

### Frobenius Norm of a Vector

The Frobenius norm of a 1-by- $n$  or  $n$ -by-1 vector  $V$  is defined as follows:

$$\|V\|_F = \sqrt{\sum_{i=1}^n |V_i|^2}$$

The Frobenius norm of a vector coincides with its 2-norm.

### Infinity and Negative Infinity Norm of a Vector

The infinity norm of a 1-by- $n$  or  $n$ -by-1 vector  $V$  is defined as follows:

$$\|V\|_{\infty} = \max(|V_i|), \text{ where } i = 1 \dots n$$

The negative infinity norm of a 1-by- $n$  or  $n$ -by-1 vector  $V$  is defined as follows:

$$\|V\|_{-\infty} = \min(|V_i|), \text{ where } i = 1 \dots n$$

## Tips

- Calling `norm` for a numeric matrix that is not a symbolic object invokes the MATLAB `norm` function.

## Version History

**Introduced in R2012b**

### **R2022a: Compute Frobenius norm of symbolic array**

The `norm` function accepts a symbolic multidimensional array as its input argument. Use the syntax `norm(X, "fro")` to return the Frobenius norm of a symbolic array  $X$ .

### **R2022a: Compute norm of symbolic matrix functions**

The `norm` function accepts an input argument of type `symfunmatrix`.

### **R2021a: Compute norm of symbolic matrix variables**

The `norm` function accepts an input argument of type `symmatrix`.

## See Also

`cond` | `equationsToMatrix` | `inv` | `linsolve` | `rank`

## not

Logical NOT for symbolic expressions

### Syntax

```
~A  
not(A)
```

### Description

$\sim A$  represents the logical NOT.  $\sim A$  is true when  $A$  is false and false when  $A$  is true.

`not(A)` is equivalent to  $\sim A$ .

### Examples

#### Set Assumption Using NOT

Create a logical condition by using `~`.

```
syms x y  
cond = ~(x > y);
```

Set the assumption represented by the condition using `assume`.

```
assume(cond)
```

Verify that the assumption is set.

```
assumptions
```

```
ans =  
~y < x
```

#### Evaluate Logical Expressions

Specify a range for  $x$  by creating a condition using the logical operators `~` and `&`.

```
syms x  
range = abs(x) < 1 & ~(abs(x)<1/3);
```

Return the conditions at  $0$  and  $2/3$  by substituting for  $x$  using `subs`. The `subs` function does not evaluate the conditions automatically.

```
x1 = subs(range,x,0)  
x2 = subs(range,x,2/3)
```

```
x1 =  
0 < 1 & ~0 < 1/3  
x2 =  
2/3 < 1 & ~2/3 < 1/3
```

Evaluate the inequalities to logical 1 or 0 by using `isAlways`.

```
isAlways(x1)
isAlways(x2)
```

```
ans =
    logical
     0
```

```
ans =
    logical
     1
```

## Input Arguments

### A — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## Tips

- If you call `simplify` for a logical expression containing symbolic subexpressions, you can get the symbolic constants `symtrue` and `symfalse`. These two constants are not the same as logical 1 (`true`) and logical 0 (`false`). To convert symbolic `symtrue` and `symfalse` to logical values, use `logical`.

## Version History

Introduced in R2012a

## See Also

`all` | `and` | `any` | `isAlways` | `or` | `piecewise` | `xor`

## nthprime

nth prime number

### Syntax

```
nthprime(n)
```

### Description

`nthprime(n)` returns the `nth` prime number. `nthprime` acts element-wise on array inputs.

### Examples

#### Find Nth Prime Number

Find the 223rd prime number.

```
nthprime(223)
```

```
ans =  
    1409
```

For large prime numbers, return exact symbolic integers by using symbolic input.

```
n = sym(223222222);  
nthprime(n)
```

```
ans =  
4738278383
```

Find the 10th, 100th, and 1000th prime numbers.

```
n = [10 100 1000];  
nthprime(n)
```

```
ans =  
    29    541   7919
```

#### Generate Random Prime Number

Generate a random prime number between the 100,000th and 200,000th prime numbers.

```
rng default % for reproducibility  
range = [100000 200000];  
nthprime(randi(range))
```

```
ans =  
    2476423
```

## Input Arguments

### **n** — Input

number | vector | matrix | array | symbolic number | symbolic array

Input, specified as a number, vector, matrix, array, or a symbolic number or array. n must be a positive integer.

## Version History

Introduced in R2018a

### See Also

nextprime | prevprime

## nthroot

Nth root of symbolic numbers

### Syntax

```
y = nthroot(x,n)
```

### Description

`y = nthroot(x,n)` returns the  $n$ th root of  $x$  with the phase angle closest to the phase of  $x$ . The output  $y$  has symbolic data type if any input argument is symbolic. The variables satisfy  $y.^n = x$ .

### Examples

#### Calculate Nth Roots

Calculate the  $n$ th root of a negative number.

```
x = sym(-27);  
n = -3;  
y = nthroot(x,n)
```

```
y =  
    - $\frac{1}{3}$ 
```

Check that the answer solves the equation  $y^n = x$ .

```
y^n  
ans = -27
```

Calculate the  $n$ th root of a complex number.

```
x = sym(1 + 1i);  
y = nthroot(x,4)
```

```
y = (1 + i)1/4
```

Find a numeric equivalent of the root.

```
vpa(y)  
ans = 1.0695539323639858023756790408254 + 0.2127475047267430357507130792184 i
```

Check that the answer solves the equation  $y^n = x$ .

```
y^4  
ans = 1 + i
```

Calculate the  $n$ th roots of an array.



```
x = sym([-27, -8, -4
         27, 64, -12])
```

```
x =

$$\begin{pmatrix} -27 & -8 & -4 \\ 27 & 64 & -12 \end{pmatrix}$$

```

```
n = sym([3, 3, 4
         3, 2, -2])
```

```
n =

$$\begin{pmatrix} 3 & 3 & 4 \\ 3 & 2 & -2 \end{pmatrix}$$

```

```
y = nthroot(x, n)
```

```
y =

$$\begin{pmatrix} -3 & -2 & (-1)^{3/4} 4^{1/4} \\ 3 & 8 & -\frac{\sqrt{12} i}{12} \end{pmatrix}$$

```

Check that the answer solves the equation  $y^n = x$ .

```
y.^n
```

```
ans =

$$\begin{pmatrix} -27 & -8 & -4 \\ 27 & 64 & -12 \end{pmatrix}$$

```

Use `nthroot` in further symbolic calculations.

```
syms x
y = solve(nthroot(x, -3) == -3, x)
```

```
y =

$$-\frac{1}{27}$$

```

```
syms x n
y = diff(nthroot(x, n), x)
```

```
y =

$$\frac{\frac{n}{\sqrt[n]{x}}}{n x}$$

```

## Input Arguments

### **x** — Input array for taking root

symbolic array | numeric array

Input array for taking root, specified as a symbolic or numeric array. When taking the root, the function acts element-wise.

If both `x` and `n` are nonscalar arrays, they must have the same size. If any element of `x` or `n` is symbolic and some elements are numeric, `nthroot` converts numeric arguments to symbolic before processing.

Example: `[sym(-8),sym(8);sym(-27),sym(27)]`

**n — Input array for order of root**

symbolic array | real array

Input array for order of root, specified as a symbolic array or real array.

- If an element of  $x$  is not real and positive, meaning it is either negative or has a nonzero imaginary part, then the corresponding element of  $n$  must be a nonzero integer.
- If an element of  $x$  is real and positive, then the corresponding element of  $n$  can have any nonzero real value.

If both  $x$  and  $n$  are nonscalar arrays, they must have the same size. If any element of  $x$  or  $n$  are symbolic and some elements are numeric, `nthroot` converts numeric arguments to symbolic before processing.

Example: `sym(-3)`

## Version History

Introduced in R2018b

## See Also

`power`

# null

Form basis for null space of matrix

## Syntax

```
Z = null(A)
```

## Description

`Z = null(A)` returns a list of vectors that form the basis for the null space of a matrix `A`. The product `A*Z` is zero. `size(Z, 2)` is the nullity of `A`. If `A` has full rank, `Z` is empty.

## Examples

### Form Basis for Null Space of Matrix

Find the basis for the null space and the nullity of the magic square of symbolic numbers. Verify that `A*Z` is zero.

```
A = sym(magic(4));
Z = null(A)
nullityOfA = size(Z, 2)
A*Z
```

```
Z =
-1
-3
 3
 1
```

```
nullityOfA =
1
```

```
ans =
0
0
0
0
```

### Form Basis for Null Space of Matrix of Full Rank

Find the basis for the null space of the matrix `B` that has full rank.

```
B = sym(hilb(3))
Z = null(B)
```

```
B =
[ 1, 1/2, 1/3]
[ 1/2, 1/3, 1/4]
[ 1/3, 1/4, 1/5]
```

$Z =$   
Empty sym: 1-by-0

## Input Arguments

### **A — Input**

numeric matrix | symbolic matrix

Input, specified as a numeric or symbolic matrix.

## Version History

Introduced before R2006a

### **See Also**

rank | rref | svd

# numden

Extract numerator and denominator

## Syntax

`[N,D] = numden(A)`

## Description

`[N,D] = numden(A)` converts `A` to a rational form where the numerator and denominator are relatively prime polynomials with integer coefficients. The function returns the numerator and denominator of the rational form of an expression.

If `A` is a symbolic or a numeric matrix, then `N` is the symbolic matrix of numerators, and `D` is the symbolic matrix of denominators. Both `N` and `D` are matrices of the same size as `A`.

## Examples

### Numerators and Denominators of Symbolic Numbers

Find the numerator and denominator of a symbolic number.

```
[n, d] = numden(sym(4/5))
```

```
n =  
4
```

```
d =  
5
```

### Numerators and Denominators of Symbolic Expressions

Find the numerator and denominator of the symbolic expression.

```
syms x y  
[n,d] = numden(x/y + y/x)
```

```
n =  
x^2 + y^2
```

```
d =  
x*y
```

### Numerators and Denominators of Matrix Elements

Find the numerator and denominator of each element of a symbolic matrix.

```
syms a b  
[n,d] = numden([a/b, 1/b; 1/a, 1/(a*b)])
```

```
n =  
[ a, 1]  
[ 1, 1]
```

```
d =  
[ b,  b]  
[ a, a*b]
```

## Input Arguments

### **A — Input**

symbolic number | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, expression, function, vector, or matrix.

## Output Arguments

### **N — Numerator**

symbolic number | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Numerator, returned as a symbolic number, expression, function, vector, or matrix.

### **D — Denominator**

symbolic number | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Denominator, returned as a symbolic number, expression, function, vector, or matrix.

## Version History

**Introduced before R2006a**

### **See Also**

`divisors` | `partfrac` | `simplifyFraction`

### **Topics**

“Extract Numerators and Denominators of Rational Expressions” on page 3-131

# odeFunction

Convert symbolic expressions to function handle for ODE solvers

## Syntax

```
f = odeFunction(expr,vars)
f = odeFunction(expr,vars,p1,...,pN)
f = odeFunction( ___,Name=Value)
```

## Description

`f = odeFunction(expr,vars)` converts a system of symbolic algebraic expressions to a MATLAB function handle. This function handle can be used as input to the numerical MATLAB ODE solvers, except for `ode15i`. The argument `vars` specifies the state variables of the system.

`f = odeFunction(expr,vars,p1,...,pN)` specifies the symbolic parameters of the system as `p1,...,pN`.

`f = odeFunction( ___,Name=Value)` uses additional options specified by one or more `Name=Value` pair arguments.

## Examples

### Create Function Handle for ODE Solvers and Solve DAEs

Convert a system of symbolic differential algebraic equations to a function handle suitable for the MATLAB ODE solvers. Then solve the system by using the `ode15s` solver.

Create the following second-order differential algebraic equation.

```
syms y(t);
eqn = diff(y(t),t,2) == (1-y(t)^2)*diff(y(t),t) - y(t);
```

Use `reduceDifferentialOrder` to rewrite that equation as a system of two first-order differential equations. Here, `vars` is a vector of state variables of the system. The new variable `Dy(t)` represents the first derivative of `y(t)` with respect to `t`.

```
[eqs,vars] = reduceDifferentialOrder(eqn,y(t))
```

```
eqs =

$$\begin{pmatrix} \frac{\partial}{\partial t} \text{Dyt}(t) + y(t) + \text{Dyt}(t) \left( y(t)^2 - 1 \right) \\ \text{Dyt}(t) - \frac{\partial}{\partial t} y(t) \end{pmatrix}$$

```

```
vars =

$$\begin{pmatrix} y(t) \\ \text{Dyt}(t) \end{pmatrix}$$

```

Set initial conditions for `y(t)` and its derivative `Dy(t)` to 2 and 0 respectively.

```
initConditions = [2 0];
```

Find the mass matrix  $M$  of the system and the right sides of the equations  $F$ .

```
[M,F] = massMatrixForm(eqs,vars)
```

$M =$

$$\begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$$

$F =$

$$\begin{pmatrix} -y(t) - \text{Dyt}(t) (y(t)^2 - 1) \\ -\text{Dyt}(t) \end{pmatrix}$$

$M$  and  $F$  refer to the form  $M(t, x(t)) \dot{x}(t) = F(t, x(t))$ . To simplify further computations, rewrite the system in the form  $\dot{x}(t) = f(t, x(t))$ .

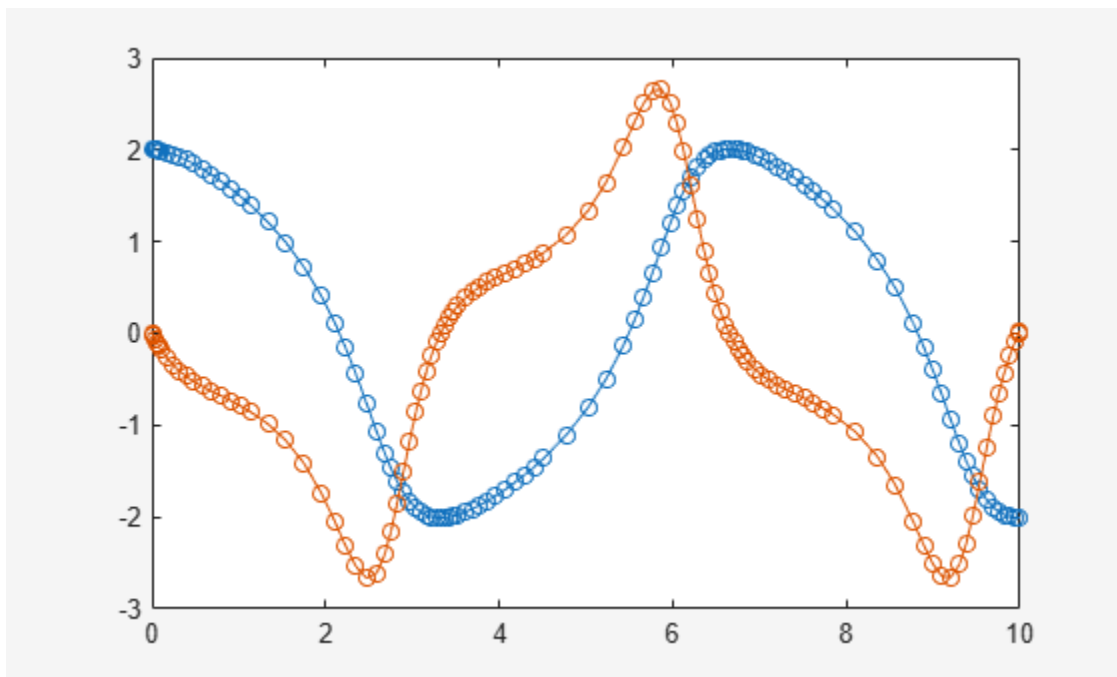
```
f = M\F
```

$f =$

$$\begin{pmatrix} \text{Dyt}(t) \\ -\text{Dyt}(t) y(t)^2 - y(t) + \text{Dyt}(t) \end{pmatrix}$$

Convert  $f$  to a MATLAB function handle by using `odeFunction`. The resulting function handle is input to the MATLAB ODE solver `ode15s`.

```
odefun = odeFunction(f,vars);  
ode15s(odefun,[0 10],initConditions)
```





## Function Handles for System Containing Symbolic Parameters

Convert a system of symbolic differential equations containing both state variables and symbolic parameters to a function handle suitable for the MATLAB ODE solvers.

Create the system of differential algebraic equations. Here, the symbolic functions  $x_1(t)$  and  $x_2(t)$  represent the state variables of the system. The system also contains constant symbolic parameters  $a$ ,  $b$ , and the parameter function  $r(t)$ . These parameters do not represent state variables. Specify the equations and state variables as two symbolic vectors: equations as a vector of symbolic equations, and variables as a vector of symbolic function calls.

```
syms x1(t) x2(t) a b r(t)
eqs = [diff(x1(t),t) == a*x1(t) + b*x2(t)^2, ...
       x1(t)^2 + x2(t)^2 == r(t)^2];
vars = [x1(t) x2(t)];
```

Find the mass matrix  $M$  and vector of the right side  $F$  for this system.  $M$  and  $F$  refer to the form  $M(t, x(t)) \dot{x}(t) = F(t, x(t))$ .

```
[M,F] = massMatrixForm(eqs,vars)
```

$M =$

$$\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$$

$F =$

$$\begin{pmatrix} b x_2(t)^2 + a x_1(t) \\ r(t)^2 - x_1(t)^2 - x_2(t)^2 \end{pmatrix}$$

Use `odeFunction` to generate MATLAB function handles from  $M$  and  $F$ . The function handle  $F$  contains symbolic parameters.

```
M = odeFunction(M,vars)
```

```
M = function_handle with value:
@(t,in2)reshape([1.0,0.0,0.0,0.0],[2,2])
```

```
F = odeFunction(F,vars,a,b,r(t))
```

```
F = function_handle with value:
@(t,in2,param1,param2,param3)[param1.*in2(1,:)+param2.*in2(2,:).^2;param3.^2-in2(1,:).^2-in2
```

Specify the parameter values.

```
aVal = -0.6;
bVal = -0.1;
rFunc = @(t) cos(t)/(1+t^2);
```

Create the reduced function handle  $F$ .

```
F = @(t,Y) F(t,Y,aVal,bVal,rFunc(t));
```

Specify consistent initial conditions for the DAE system.

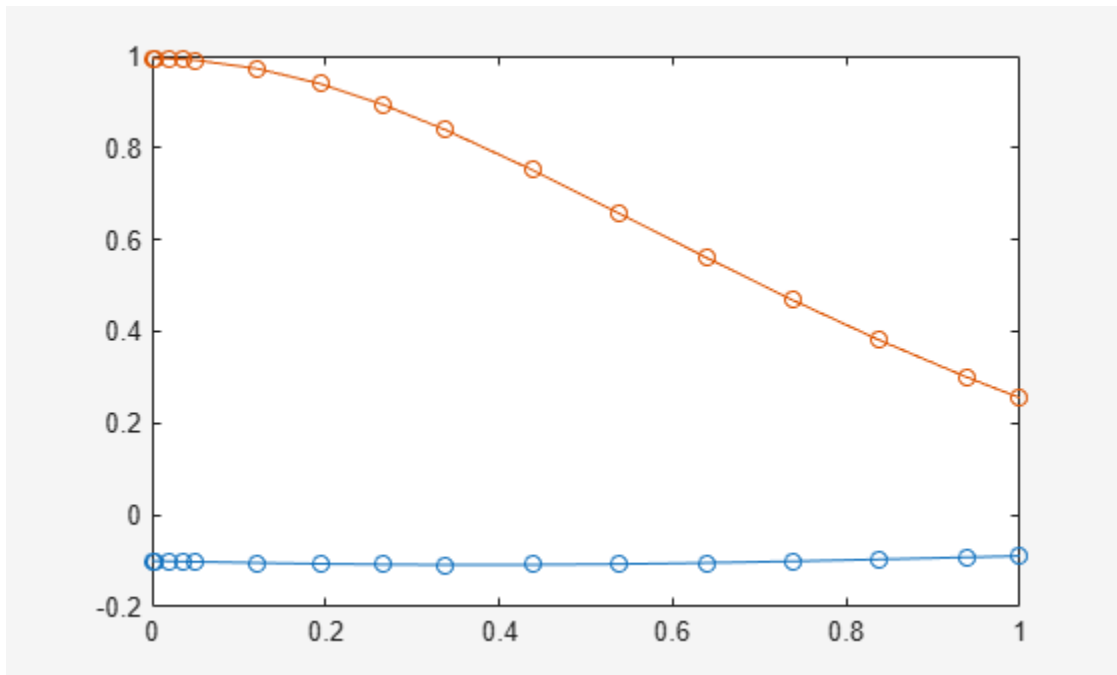
```
t0 = 0;
y0 = [-rFunc(t0)*sin(0.1); rFunc(t0)*cos(0.1)];
yp0 = [aVal*y0(1) + bVal*y0(2)^2; 1.234];
```

Create an option set that contains the mass matrix  $M$  of the system and vector  $yp0$  of initial conditions for the derivatives.

```
opt = odeset(mass=M,InitialSlope=yp0);
```

Now, use `ode15s` to solve the system of equations.

```
ode15s(F,[t0 1],y0,opt)
```



### Write Function Handles to File with Comments

Write the generated function handles to files by using the `File` option. When writing to files, `odeFunction` optimizes the code using intermediate variables named `t0`, `t1`, .... Include comments the files by specifying the `Comments` option.

Define the system of differential equations. Find the mass matrix  $M$  and the right side  $F$ .

```
syms x(t) y(t)
eqs = [diff(x(t),t)+2*diff(y(t),t) == 0.1*y(t), ...
        x(t)-y(t) == cos(t)-0.2*t*sin(x(t))];
vars = [x(t) y(t)];
[M,F] = massMatrixForm(eqs,vars);
```

Write the MATLAB code for  $M$  and  $F$  to the files `myfileM` and `myfileF`. `odeFunction` overwrites existing files. Include the comment `Version: 1.1` in the files. You can open and edit the output files.

```

M = odeFunction(M,vars,File="myfileM",Comments="Version: 1.1");
type myfileM

function expr = myfileM(t,in2)
%myfileM
%   EXPR = myfileM(T,IN2)

%   This function was generated by the Symbolic Math Toolbox version 26.1.
%   20-Apr-2026 01:22:30

%Version: 1.1
expr = reshape([1.0,0.0,2.0,0.0],[2,2]);
end

F = odeFunction(F,vars,File="myfileF",Comments="Version: 1.1");
type myfileF

function expr = myfileF(t,in2)
%myfileF
%   EXPR = myfileF(T,IN2)

%   This function was generated by the Symbolic Math Toolbox version 26.1.
%   20-Apr-2026 01:22:32

%Version: 1.1
x = in2(1,:);
y = in2(2,:);
expr = [y./1.0e+1;-x+y+cos(t)-(t.*sin(x))./5.0];
end

```

Specify consistent initial values for  $x(t)$  and  $y(t)$  and their first derivatives.

```

xy0 = [2; 1];           % x(t) and y(t)
xyp0 = [0; 0.05*xy0(2)]; % derivatives of x(t) and y(t)

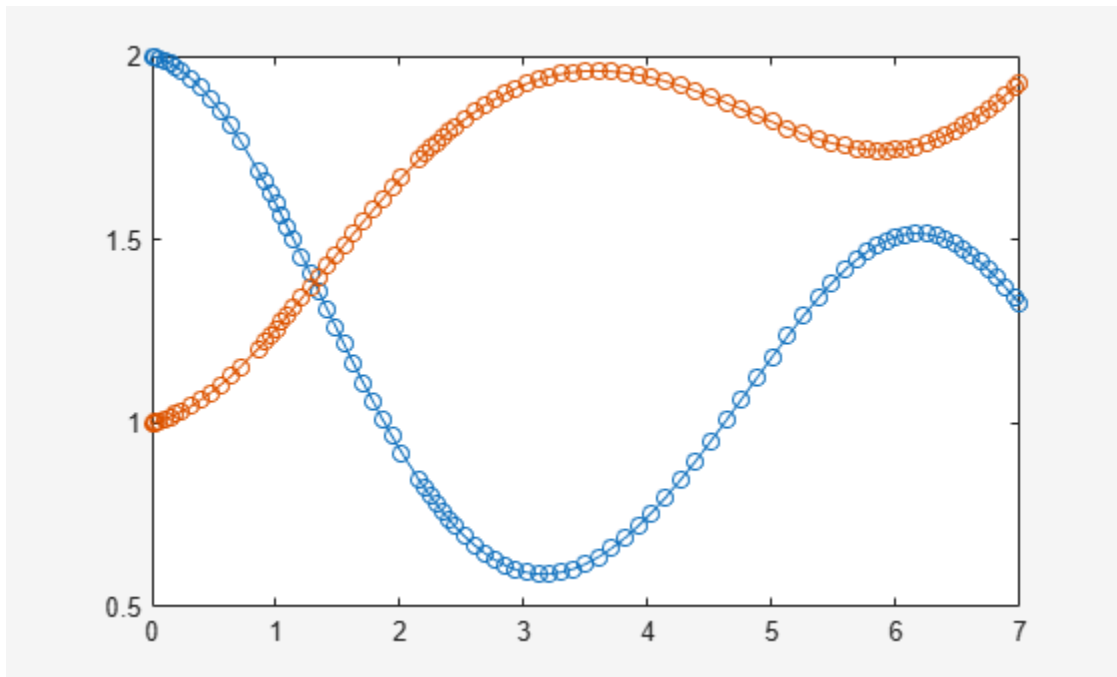
```

Create an option set that contains the mass matrix  $M$ , initial conditions  $xyp0$ , and numerical tolerances for the numerical search.

```
opt = odeset(mass=M,RelTol=1e-6,AbsTol=1e-6,InitialSlope=xyp0);
```

Solve the system of equations by using `ode15s`.

```
ode15s(F,[0 7],xy0,opt)
```



### Convert Differential Equations to Function Handles with Sparse Matrices

Specify the name-value argument `Sparse` as `true` when converting symbolic differential equations to MATLAB function handles with sparse matrices.

Create the system of differential algebraic equations. Here, the symbolic functions  $x_1(t)$  and  $x_2(t)$  represent the state variables of the system. Specify the equations and state variables as two symbolic vectors: equations as a vector of symbolic equations, and variables as a vector of symbolic function calls.

```
syms x1(t) x2(t) a b r(t)
eqs = [diff(x1(t),t) == a*x1(t) + b*x2(t)^2, ...
       x1(t)^2 + x2(t)^2 == r(t)^2];
vars = [x1(t) x2(t)];
```

Find the mass matrix  $M$  and vector of the right side  $F$  for this system.  $M$  and  $F$  refer to the form  $M(t, x(t)) \dot{x}(t) = F(t, x(t))$ .

```
[M,F] = massMatrixForm(eqs,vars)
```

$M =$

$$\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$$

$F =$

$$\begin{pmatrix} b x_2(t)^2 + a x_1(t) \\ r(t)^2 - x_1(t)^2 - x_2(t)^2 \end{pmatrix}$$

Generate MATLAB function handles from M and F. Because most of the elements of the mass matrix M are zeros, use the Sparse argument when converting M.

```
M = odeFunction(M,vars,Sparse=true)
```

```
M = function_handle with value:  
    @(t,in2)sparse([1],[1],[1.0],2,2)
```

```
F = odeFunction(F,vars,a,b,r(t))
```

```
F = function_handle with value:  
    @(t,in2,param1,param2,param3)[param1.*in2(1,:)+param2.*in2(2,:).^2;param3.^2-in2(1,:).^2-in2
```

Specify the parameter values.

```
aVal = -0.6;  
bVal = -0.1;  
rFunc = @(t) cos(t)/(1 + t^2);
```

Create the reduced function handle F.

```
F = @(t,Y) F(t,Y,aVal,bVal,rFunc(t));
```

Specify consistent initial conditions for the DAE system.

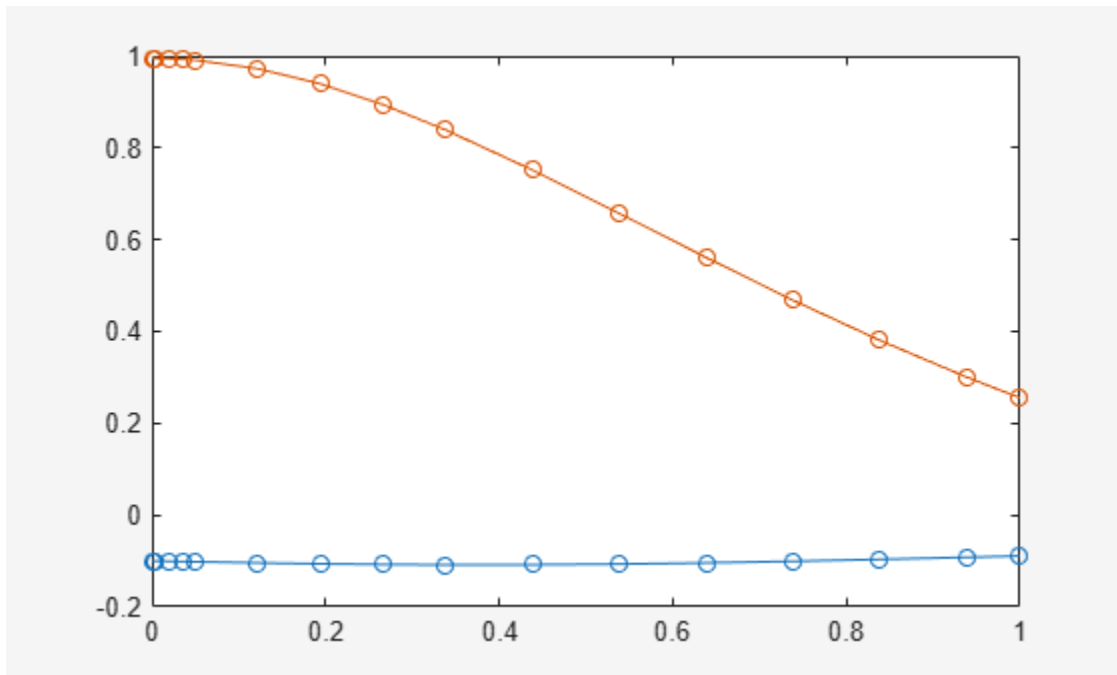
```
t0 = 0;  
y0 = [-rFunc(t0)*sin(0.1); rFunc(t0)*cos(0.1)];  
yp0 = [aVal*y0(1) + bVal*y0(2)^2; 1.234];
```

Create an option set that contains the mass matrix M of the system and vector yp0 of initial conditions for the derivatives.

```
opt = odeset(mass=M,InitialSlope=yp0);
```

Solve the system of equations using ode15s.

```
ode15s(F,[t0 1],y0,opt)
```



## Input Arguments

### **expr** — System of algebraic expressions

vector of symbolic expressions

System of algebraic expressions, specified as a vector of symbolic expressions.

### **vars** — State variables

vector of symbolic functions | vector of symbolic function calls

State variables, specified as a vector of symbolic functions or function calls, such as  $x(t)$ .

Example:  $[x(t), y(t)]$  or  $[x(t); y(t)]$

### **p1, ..., pN** — Parameters of system

symbolic variables | symbolic functions | symbolic function calls | symbolic vector | symbolic matrix

Parameters of the system, specified as symbolic variables, functions, or function calls, such as  $f(t)$ . You can also specify parameters of the system as a vector or matrix of symbolic variables, functions, or function calls. If **expr** contains symbolic parameters other than the variables specified in **vars**, you must specify these additional parameters as  $p1, \dots, pN$ .

## Name-Value Pair Arguments

Specify optional pairs of arguments as **Name1=Value1, ..., NameN=ValueN**, where **Name** is the argument name and **Value** is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `odeFunction(expr,vars,File="myfile")`

### **Comments — Comments to include in file header**

character vector | cell array of character vectors | string vector

Comments to include in the file header, specified as a character vector, cell array of character vectors, or string vector.

### **File — Path to file containing generated code**

character vector

Path to the file containing generated code, specified as a character vector. The generated file accepts arguments of type `double`, and can be used without Symbolic Math Toolbox. If the value is empty, `odeFunction` generates an anonymous function. If the character vector does not end in `.m`, the function appends `.m`.

By default, `odeFunction` with the `File` argument generates a file containing optimized code. Optimized means intermediate variables are automatically generated to simplify or speed up the code. MATLAB generates intermediate variables as a lowercase letter `t` followed by an automatically generated number, for example `t32`. To disable code optimization, use the `Optimize` argument.

### **Optimize — Flag preventing optimization of code written to function file**

`true` (default) | `false`

Flag preventing optimization of code written to a function file, specified as `false` or `true`.

By default, `odeFunction` with the `File` argument generates a file containing optimized code. Optimized means intermediate variables are automatically generated to simplify or speed up the code. MATLAB generates intermediate variables as a lowercase letter `t` followed by an automatically generated number, for example `t32`.

`odeFunction` without the `File` argument (or with a file path specified by an empty character vector) creates a function handle. In this case, the code is not optimized. If you try to enforce code optimization by setting `Optimize` to `true`, then `odeFunction` throws an error.

### **Sparse — Flag that switches between sparse and dense matrix generation**

`false` (default) | `true`

Flag that switches between sparse and dense matrix generation, specified as `true` or `false`. When you specify `Sparse=true`, the generated function represents symbolic matrices by sparse numeric matrices. Use `Sparse=true` when you convert symbolic matrices containing many zero elements. Often, operations on sparse matrices are more efficient than the same operations on dense matrices. See “Convert Differential Equations to Function Handles with Sparse Matrices” on page 6-1124.

## **Output Arguments**

### **f — Function handle that is input to numerical MATLAB ODE solvers, except `ode15i`**

MATLAB function handle

Function handle that can serve as input argument to all numerical MATLAB ODE solvers, except for `ode15i`, returned as a MATLAB function handle.

`odeFunction` returns a function handle suitable for the ODE solvers such as `ode45`, `ode15s`, `ode23t`, and others. The only ODE solver that does not accept this function handle is the solver for

fully implicit differential equations, `ode15i`. To convert the system of equations to a function handle suitable for `ode15i`, use `daeFunction`.

## **Version History**

**Introduced in R2015a**

### **See Also**

`findDecoupledBlocks` | `daeFunction` | `decic` | `incidenceMatrix` | `isLowIndexDAE` | `massMatrixForm` | `matlabFunction` | `ode15i` | `ode15s` | `ode45` | `ode23t` | `reduceDAEIndex` | `reduceDAEToODE` | `reduceDifferentialOrder` | `reduceRedundancies`

### **Topics**

“Solve DAEs Using Mass Matrix Solvers” on page 3-82



# odeToVectorField

Reduce order of differential equations to first-order

## Syntax

```
V = odeToVectorField(eqn1,...,eqnN)
[V,S] = odeToVectorField(eqn1,...,eqnN)
```

## Description

`V = odeToVectorField(eqn1,...,eqnN)` converts higher-order differential equations `eqn1,...,eqnN` to a system of first-order differential equations, returned as a symbolic vector.

`[V,S] = odeToVectorField(eqn1,...,eqnN)` converts `eqn1,...,eqnN` and returns two symbolic vectors. The first vector `V` is the same as the output of the previous syntax. The second vector `S` shows the substitutions made to obtain `V`.

## Examples

### Convert Second-Order Differential Equation to First-Order System

Define a second-order differential equation:

$$\frac{d^2y}{dt^2} + y^2t = 3t.$$

Convert the second-order differential equation to a system of first-order differential equations.

```
syms y(t)
eqn = diff(y,2) + y^2*t == 3*t;
V = odeToVectorField(eqn)
```

$$V = \begin{pmatrix} Y_2 \\ 3t - tY_1^2 \end{pmatrix}$$

The elements of `V` represent the system of first-order differential equations, where  $V[i] = Y_i'$  and  $Y_1 = y$ . Here, the output `V` represents these equations:

$$\frac{dY_1}{dt} = Y_2$$

$$\frac{dY_2}{dt} = 3t - tY_1^2.$$

For details on the relation between the input and output, see “Algorithms” on page 6-1132.

### Return Substitutions Made When Reducing Order of Differential Equations

When reducing the order of differential equations, return the substitutions that `odeToVectorField` makes by specifying a second output argument.

```
syms f(t) g(t)
eqn1 = diff(g) == g-f;
eqn2 = diff(f,2) == g+f;
eqns = [eqn1 eqn2];
[V,S] = odeToVectorField(eqns)
```

$$V = \begin{pmatrix} Y_2 \\ Y_1 + Y_3 \\ Y_3 - Y_1 \end{pmatrix}$$

$$S = \begin{pmatrix} f \\ Df \\ g \end{pmatrix}$$

The elements of  $V$  represent the system of first-order differential equations, where  $V[i] = Y_i'$ . The output  $S$  shows the substitutions being made,  $S[1] = Y_1 = f$ ,  $S[2] = Y_2 = \text{diff}(f)$ , and  $S[3] = Y_3 = g$ .

### Solve Higher-Order Differential Equation Numerically

Solve a higher-order differential equation numerically by reducing the order of the equation, generating a MATLAB® function handle, and then finding the numerical solution using the `ode45` function.

Convert the following second-order differential equation to a system of first-order differential equations by using `odeToVectorField`.

$$\frac{d^2 y}{dt^2} = (1 - y^2) \frac{dy}{dt} - y.$$

```
syms y(t)
eqn = diff(y,2) == (1-y^2)*diff(y)-y;
V = odeToVectorField(eqn)
```

$$V = \begin{pmatrix} Y_2 \\ -(Y_1^2 - 1)Y_2 - Y_1 \end{pmatrix}$$

Generate a MATLAB function handle from  $V$  by using `matlabFunction`.

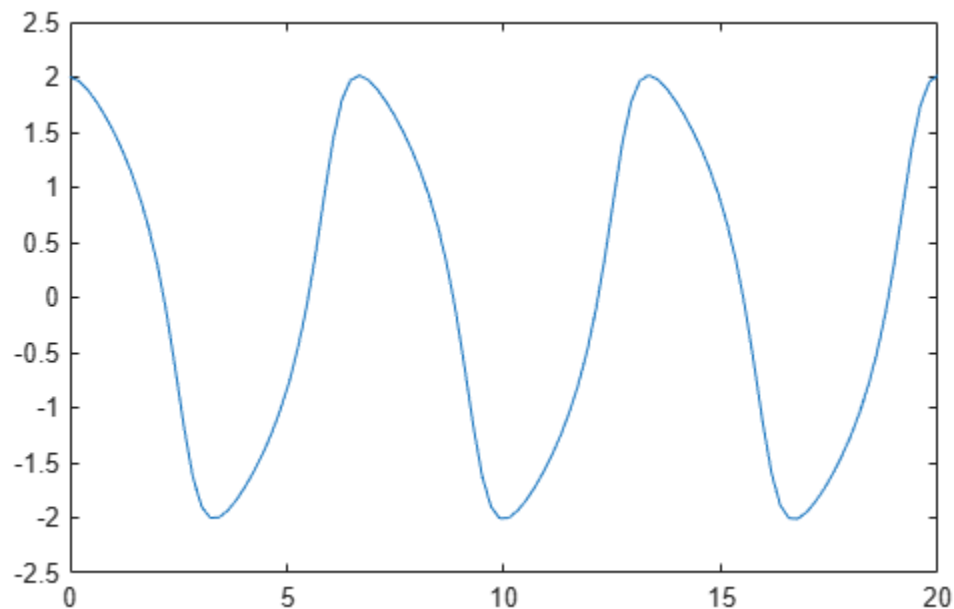
```
M = matlabFunction(V, 'vars', {'t', 'Y'})
M = function_handle with value:
@(t,Y)[Y(2); -(Y(1).^2-1.0).*Y(2)-Y(1)]
```

Specify the solution interval to be  $[0 \ 20]$  and the initial conditions to be  $y'(0) = 2$  and  $y''(0) = 0$ . Solve the system of first-order differential equations by using `ode45`.

```
interval = [0 20];
yInit = [2 0];
ySol = ode45(M,interval,yInit);
```

Next, plot the solution  $y(t)$  within the interval  $t = [0 \ 20]$ . Generate the values of  $t$  by using `linspace`. Evaluate the solution for  $y(t)$ , which is the first index in `ySol`, by calling the `deval` function with an index of 1. Plot the solution using `plot`.

```
tValues = linspace(0,20,100);
yValues = deval(ySol,tValues,1);
plot(tValues,yValues)
```



### Convert Second-Order Differential Equation with Initial Condition

Convert the second-order differential equation  $y''(x) = x$  with the initial condition  $y(0) = a$  to a first-order system.

```
syms y(x) a
eqn = diff(y,x,2) == x;
cond = y(0) == a;
V = odeToVectorField(eqn,cond)
```

$V =$

$$\begin{pmatrix} Y_2 \\ x \end{pmatrix}$$

## Input Arguments

### eqn1, ..., eqnN — Higher-order differential equations

symbolic differential equation | array of symbolic differential equations

Higher-order differential equations, specified as a symbolic differential equation or an array of symbolic differential equations. Use the `==` operator to create an equation. Use the `diff` function to indicate differentiation. For example, represent  $d^2y(t)/dt^2 = t y(t)$  by entering the following command.

```
syms y(t)
eqn = diff(y,2) == t*y;
```

## Output Arguments

### V — First-order differential equations

symbolic expression | vector of symbolic expressions

First-order differential equations, returned as a symbolic expression or a vector of symbolic expressions. Each element of this vector is the right side of the first-order differential equation  $Y[i]' = V[i]$ .

### S — Substitutions in first-order equations

vector of symbolic expressions

Substitutions in first-order equations, returned as a vector of symbolic expressions. The elements of the vector represent the substitutions, such that  $S(1) = Y[1]$ ,  $S(2) = Y[2]$ , ...

## Tips

- To solve the resulting system of first-order differential equations, generate a MATLAB function handle using `matlabFunction` with `V` as an input. Then, use the generated MATLAB function handle as an input for the MATLAB numerical solver `ode23` or `ode45`.
- `odeToVectorField` can convert only quasi-linear differential equations. That is, the highest-order derivatives must appear linearly. For example, `odeToVectorField` can convert  $y*y''(t) = -t^2$  because it can be rewritten as  $y''(t) = -t^2/y$ . However, it cannot convert  $y''(t)^2 = -t^2$  or  $\sin(y''(t)) = -t^2$ .

## Algorithms

To convert an  $n$ th-order differential equation

$$a_n(t)y^{(n)} + a_{n-1}(t)y^{(n-1)} + \dots + a_1(t)y' + a_0(t)y + r(t) = 0$$

into a system of first-order differential equations, `odetovectorfield` makes these substitutions.

$$\begin{aligned} Y_1 &= y \\ Y_2 &= y' \\ Y_3 &= y'' \\ &\dots \\ Y_{n-1} &= y^{(n-2)} \\ Y_n &= y^{(n-1)} \end{aligned}$$

Using the new variables, it rewrites the equation as a system of  $n$  first-order differential equations:

$$Y_1' = y' = Y_2$$

$$Y_2' = y'' = Y_3$$

...

$$Y_{n-1}' = y^{(n-1)} = Y_n$$

$$Y_n' = -\frac{a_{n-1}(t)}{a_n(t)}Y_n - \frac{a_{n-2}(t)}{a_n(t)}Y_{n-1} - \dots - \frac{a_1(t)}{a_n(t)}Y_2 - \frac{a_0(t)}{a_n(t)}Y_1 + \frac{r(t)}{a_n(t)}$$

`odeToVectorField` returns the right sides of these equations as the elements of vector  $V$  and the substitutions made as the second output  $S$ .

## Version History

**Introduced in R2012a**

**R2019b: `odeToVectorField` will no longer support character vector or string inputs**

*Warns starting in R2019b*

`odeToVectorField` will not accept equations as strings or character vectors in a future release. Instead, use symbolic expressions or `sym` objects to define differential equations. For example, replace inputs such as `odeToVectorField('D2y=x')` with `syms y(x)`, `odeToVectorField(diff(y,x,2)==x)`.

## See Also

`dsolve` | `matlabFunction` | `odeFunction` | `ode23` | `ode45`

## or

Logical OR for symbolic expressions

### Syntax

```
A | B  
or(A,B)
```

### Description

$A \mid B$  represents the logical OR.  $A \mid B$  is true when either A or B is true, or when both A and B are true.

`or(A,B)` is equivalent to  $A \mid B$ .

### Examples

#### Set Assumption Using OR

Combine these symbolic inequalities into a logical condition by using `|`.

```
syms x y  
xy = x>=0 | y>=0;
```

Set the assumption represented by the condition using `assume`.

```
assume(xy)
```

Verify that the assumptions are set.

```
assumptions
```

```
ans =  
0 <= x | 0 <= y
```

#### Set and Evaluate Condition

Combine two symbolic inequalities into a logical expression by using `|`.

```
range = x < -1 | x > 1;
```

Substitute  $x$  with 0 and 10. Although the inequalities have values, `subs` does not evaluate them to logical 1 or 0.

```
x1 = subs(range,x,10)  
x2 = subs(range,x,0)
```

```
x1 =  
1 < 10 | 10 < -1  
x2 =  
0 < -1 | 1 < 0
```

Evaluate the inequalities by using `isAlways`.

```
isAlways(x1)
```

```
ans =
    logical
     1
```

```
isAlways(x2)
```

```
ans =
    logical
     0
```

### Combine Multiple Conditions

Combine multiple conditions by applying `or` to the conditions using the `fold` function.

Set the condition that `x` equals an integer between 1 and 10.

```
syms x
cond = fold(@or, x == 1:10);
assume(cond)
assumptions

ans =
x == 1 | x == 2 | x == 3 | x == 4 | x == 5 | ...
x == 6 | x == 7 | x == 8 | x == 9 | x == 10
```

## Input Arguments

### A, B — Operands

symbolic equations | symbolic inequalities | symbolic expressions | symbolic arrays

Operands, specified as symbolic equations, inequalities, expressions, or arrays. Inputs `A` and `B` must either be the same size or have sizes that are compatible (for example, `A` is an `M`-by-`N` matrix and `B` is a scalar or 1-by-`N` row vector). For more information, see “Compatible Array Sizes for Basic Operations”.

## Tips

- If you call `simplify` for a logical expression containing symbolic subexpressions, you can get the symbolic constants `symtrue` and `symfalse`. These two constants are not the same as logical 1 (`true`) and logical 0 (`false`). To convert symbolic `symtrue` and `symfalse` to logical values, use `logical`.

## Version History

**Introduced in R2012a**

### R2016b: Implicit expansion change affects arguments for operators

*Behavior changed in R2016b*

Starting in R2016b with the addition of implicit expansion, some combinations of arguments for basic operations that previously returned errors now produce results. For example, you previously could

not add a row and a column vector, but those operands are now valid for addition. In other words, an expression like `[1 2] + [1; 2]` previously returned a size mismatch error, but now it executes.

If your code uses element-wise operators and relies on the errors that MATLAB previously returned for mismatched sizes, particularly within a `try/catch` block, then your code might no longer catch those errors.

For more information on the required input sizes for basic array operations, see “Compatible Array Sizes for Basic Operations”.

### **See Also**

`all` | `and` | `any` | `isAlways` | `not` | `piecewise` | `xor`



# orth

Orthonormal basis for range of symbolic matrix

## Syntax

```
orth(A)
orth(A,'real')
orth(A,'skipnormalization')
orth(A,'real','skipnormalization')
```

## Description

`orth(A)` computes an orthonormal basis on page 6-1139 for the range of A.

`orth(A,'real')` computes an orthonormal basis using a real scalar product in the orthogonalization process.

`orth(A,'skipnormalization')` computes a non-normalized orthogonal basis. In this case, the vectors forming the columns of B do not necessarily have length 1.

`orth(A,'real','skipnormalization')` computes a non-normalized orthogonal basis using a real scalar product in the orthogonalization process.

## Examples

### Compute Orthonormal Basis

Compute an orthonormal basis of the range of this matrix. Because these numbers are not symbolic objects, you get floating-point results.

```
A = [2 -3 -1; 1 1 -1; 0 1 -1];
B = orth(A)
```

```
B =
   -0.9859   -0.1195    0.1168
    0.0290   -0.8108   -0.5846
    0.1646   -0.5729    0.8029
```

Now, convert this matrix to a symbolic object, and compute an orthonormal basis:

```
A = sym([2 -3 -1; 1 1 -1; 0 1 -1]);
B = orth(A)
```

```
B =
[ (2*5^(1/2))/5, -6^(1/2)/6, -(2^(1/2)*15^(1/2))/30]
[ 5^(1/2)/5, 6^(1/2)/3, (2^(1/2)*15^(1/2))/15]
[ 0, 6^(1/2)/6, -(2^(1/2)*15^(1/2))/6]
```

You can use `double` to convert this result to the double-precision numeric form. The resulting matrix differs from the matrix returned by the MATLAB `orth` function because these functions use different versions of the Gram-Schmidt orthogonalization algorithm:

```
double(B)
```

```
ans =
    0.8944    -0.4082    -0.1826
    0.4472     0.8165     0.3651
         0     0.4082    -0.9129
```

Verify that  $B' * B = I$ , where  $I$  is the identity matrix:

```
B'*B
```

```
ans =
[ 1, 0, 0]
[ 0, 1, 0]
[ 0, 0, 1]
```

Now, verify that the 2-norm of each column of  $B$  is 1:

```
norm(B(:, 1))
norm(B(:, 2))
norm(B(:, 3))
```

```
ans =
1
```

```
ans =
1
```

```
ans =
1
```

### Compute Real Orthonormal Basis

Compute an orthonormal basis of this matrix using 'real' to avoid complex conjugates:

```
syms a
A = [a 1; 1 a];
B = orth(A, 'real')

B =
[ a/(a^2 + 1)^(1/2),      -(a^2 - 1)/((a^2 + 1)*((a^2 - ...
1)^2/(a^2 + 1)^2 + (a^2*(a^2 - 1)^2)/(a^2 + 1)^2)^(1/2))]
[ 1/(a^2 + 1)^(1/2), (a*(a^2 - 1))/((a^2 + 1)*((a^2 - ...
1)^2/(a^2 + 1)^2 + (a^2*(a^2 - 1)^2)/(a^2 + 1)^2)^(1/2)]]
```

### Compute Orthogonal Basis by Skipping Normalization

Compute an orthogonal basis of this matrix using 'skipnormalization'. The lengths of the resulting vectors (the columns of matrix  $B$ ) are not required to be 1

```
syms a
A = [a 1; 1 a];
B = orth(A, 'skipnormalization')

B =
[ a,      -(a^2 - 1)/(a*conj(a) + 1)]
[ 1, -(conj(a) - a^2*conj(a))/(a*conj(a) + 1)]
```

## Compute Real Orthogonal Basis

Compute an orthogonal basis of this matrix using 'skipnormalization' and 'real':

```
syms a
A = [a 1; 1 a];
B = orth(A,'skipnormalization','real')

B =
[ a,      -(a^2 - 1)/(a^2 + 1)]
[ 1, (a*(a^2 - 1))/(a^2 + 1)]
```

## Input Arguments

### A — Input

symbolic matrix

Input, specified as a symbolic matrix.

## More About

### Orthonormal Basis

An orthonormal basis for the range of matrix A is matrix B, such that:

- $B' * B = I$ , where I is the identity matrix.
- The columns of B span the same space as the columns of A.
- The number of columns of B is the rank of A.

## Tips

- Calling `orth` for numeric arguments that are not symbolic objects invokes the MATLAB `orth` function. Results returned by MATLAB `orth` can differ from results returned by `orth` because these two functions use different algorithms to compute an orthonormal basis. The Symbolic Math Toolbox `orth` function uses the classic Gram-Schmidt orthogonalization algorithm. The MATLAB `orth` function uses the modified Gram-Schmidt algorithm because the classic algorithm is numerically unstable.
- Using 'skipnormalization' to compute an orthogonal basis instead of an orthonormal basis can speed up your computations.

## Algorithms

`orth` uses the classic Gram-Schmidt orthogonalization algorithm.

## Version History

Introduced in R2013a

## See Also

`norm` | `null` | `orth` | `rank` | `svd`

## pade

Padé approximant

### Syntax

```
pade(f,var)
pade(f,var,a)
pade( ____,Name,Value)
```

### Description

`pade(f,var)` returns the third-order Padé approximant of the expression `f` at `var = 0`. For details, see “Padé Approximant” on page 6-1144.

If you do not specify `var`, then `pade` uses the default variable determined by `symvar(f,1)`.

`pade(f,var,a)` returns the third-order Padé approximant of expression `f` at the point `var = a`.

`pade( ____,Name,Value)` uses additional options specified by one or more `Name,Value` pair arguments. You can specify `Name,Value` after the input arguments in any of the previous syntaxes.

### Examples

#### Find Padé Approximant for Symbolic Expressions

Find the Padé approximant of  $\sin(x)$ . By default, `pade` returns a third-order Padé approximant.

```
syms x
pade(sin(x))

ans =
-(x*(7*x^2 - 60))/(3*(x^2 + 20))
```

#### Specify Expansion Variable

If you do not specify the expansion variable, `symvar` selects it. Find the Padé approximant of  $\sin(x) + \cos(y)$ . The `symvar` function chooses `x` as the expansion variable.

```
syms x y
pade(sin(x) + cos(y))

ans =
(- 7*x^3 + 3*cos(y)*x^2 + 60*x + 60*cos(y))/(3*(x^2 + 20))
```

Specify the expansion variable as `y`. The `pade` function returns the Padé approximant with respect to `y`.

```
pade(sin(x) + cos(y),y)

ans =
(12*sin(x) + y^2*sin(x) - 5*y^2 + 12)/(y^2 + 12)
```

### Approximate Value of Function at Particular Point

Find the value of  $\tan(3\pi/4)$ . Use `pade` to find the Padé approximant for  $\tan(x)$  and substitute into it using `subs` to find  $\tan(3\pi/4)$ .

```
syms x
f = tan(x);
P = pade(f);
y = subs(P,x,3*pi/4)

y =
(pi*((9*pi^2)/16 - 15))/(4*((9*pi^2)/8 - 5))
```

Use `vpa` to convert `y` into a numeric value.

```
vpa(y)

ans =
-1.2158518789569086447244881326842
```

### Increase Accuracy of Padé Approximant

You can increase the accuracy of the Padé approximant by increasing the order. If the expansion point is a pole or a zero, the accuracy can also be increased by setting `OrderMode` to `relative`. The `OrderMode` option has no effect if the expansion point is not a pole or zero.

Find the Padé approximant of  $\tan(x)$  using `pade` with an expansion point of 0 and Order of [1 1]. Find the value of  $\tan(1/5)$  by substituting into the Padé approximant using `subs`, and use `vpa` to convert  $1/5$  into a numeric value.

```
syms x
p11 = pade(tan(x),x,0,'Order',[1 1])
p11 = subs(p11,x,vpa(1/5))

p11 =
x
p11 =
0.2
```

Find the approximation error by subtracting `p11` from the actual value of  $\tan(1/5)$ .

```
y = tan(vpa(1/5));
error = y - p11

error =
0.0027100355086724833213582716475345
```

Increase the accuracy of the Padé approximant by increasing the order using `Order`. Set `Order` to [2 2], and find the error.

```
p22 = pade(tan(x),x,0,'Order',[2 2])
p22 = subs(p22,x,vpa(1/5));
error = y - p22

p22 =
-(3*x)/(x^2 - 3)
error =
0.0000073328059697806186555689448317799
```

The accuracy increases with increasing order.

If the expansion point is a pole or zero, the accuracy of the Padé approximant decreases. Setting the `OrderMode` option to `relative` compensates for the decreased accuracy. For details, see “Padé Approximant” on page 6-1144. Because the `tan` function has a zero at 0, setting `OrderMode` to `relative` increases accuracy. This option has no effect if the expansion point is not a pole or zero.

```
p22Rel = pade(tan(x),x,0,'Order',[2 2],'OrderMode','relative')
p22Rel = subs(p22Rel,x,vpa(1/5));
error = y - p22Rel
```

```
p22Rel =
(x*(x^2 - 15))/(3*(2*x^2 - 5))
error =
0.00000000084084014806113311713765317725998
```

The accuracy increases if the expansion point is a pole or zero and `OrderMode` is set to `relative`.

### Plot Accuracy of Padé Approximant

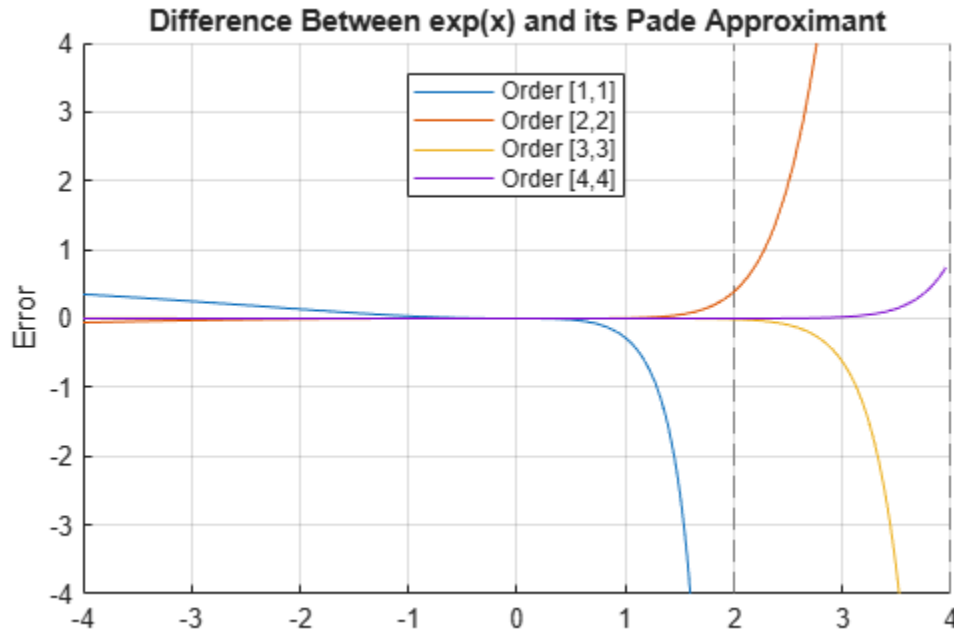
Plot the difference between  $\exp(x)$  and its Padé approximants of orders `[1 1]` through `[4 4]`. Use `axis` to focus on the region of interest. The plot shows that accuracy increases with increasing order of the Padé approximant.

```
syms x
expr = exp(x);

hold on
grid on

for i = 1:4
    fplot(expr - pade(expr,'Order',i))
end

axis([-4 4 -4 4])
legend('Order [1,1]', 'Order [2,2]', 'Order [3,3]', 'Order [4,4]', ...
       'Location','Best')
title('Difference Between exp(x) and its Pade Approximant')
ylabel('Error')
```



## Input Arguments

### **f** — Input to approximate

symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input to approximate, specified as a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **var** — Expansion variable

symbolic variable

Expansion variable, specified as a symbolic variable. If you do not specify `var`, then `pade` uses the default variable determined by `symvar(f,1)`.

### **a** — Expansion point

number | symbolic number | symbolic variable | symbolic function | symbolic expression

Expansion point, specified as a number, or a symbolic number, variable, function, or expression. The expansion point cannot depend on the expansion variable. You also can specify the expansion point as a `Name,Value` pair argument. If you specify the expansion point both ways, then the `Name,Value` pair argument takes precedence.

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.*

Example: `pade(f, 'Order', [2 2])` returns the Padé approximant of `f` of order  $m = 2$  and  $n = 2$ .

### ExpansionPoint — Expansion point

number | symbolic number | symbolic variable | symbolic function | symbolic expression

Expansion point, specified as a number, or a symbolic number, variable, function, or expression. The expansion point cannot depend on the expansion variable. You can also specify the expansion point using the input argument `a`. If you specify the expansion point both ways, then the `Name, Value` pair argument takes precedence.

### Order — Order of Padé approximant

integer | vector of two integers | symbolic integer | symbolic vector of two integers

Order of the Padé approximant, specified as an integer, a vector of two integers, or a symbolic integer, or vector of two integers. If you specify a single integer, then the integer specifies both the numerator order  $m$  and denominator order  $n$  producing a Padé approximant with  $m = n$ . If you specify a vector of two integers, then the first integer specifies  $m$  and the second integer specifies  $n$ . By default, `pade` returns a Padé approximant with  $m = n = 3$ .

### OrderMode — Flag that selects absolute or relative order for Padé approximant

'absolute' (default) | 'relative'

Flag that selects absolute or relative order for Padé approximant, specified as 'absolute' or 'relative'. The default value of 'absolute' uses the standard definition of the Padé approximant. If you set 'OrderMode' to 'relative', it only has an effect when there is a pole or a zero at the expansion point `a`. In this case, to increase accuracy, `pade` multiplies the numerator by  $(\text{var} - a)^p$  where  $p$  is the multiplicity of the zero or pole at the expansion point. For details, see "Padé Approximant" on page 6-1144.

## More About

### Padé Approximant

By default, `pade` approximates the function  $f(x)$  using the standard form of the Padé approximant of order  $[m, n]$  around  $x = x_0$  which is

$$\frac{a_0 + a_1(x - x_0) + \dots + a_m(x - x_0)^m}{1 + b_1(x - x_0) + \dots + b_n(x - x_0)^n}.$$

When `OrderMode` is `relative`, and a pole or zero exists at the expansion point  $x = x_0$ , the `pade` function uses this form of the Padé approximant

$$\frac{(x - x_0)^p(a_0 + a_1(x - x_0) + \dots + a_m(x - x_0)^m)}{1 + b_1(x - x_0) + \dots + b_n(x - x_0)^n}.$$

The parameters  $p$  and  $a_0$  are given by the leading order term  $f = a_0(x - x_0)^p + O((x - x_0)^{p+1})$  of the series expansion of  $f$  around  $x = x_0$ . Thus,  $p$  is the multiplicity of the pole or zero at  $x_0$ .

## Tips

- If you use both the third argument `a` and `ExpansionPoint` to specify the expansion point, the value specified via `ExpansionPoint` prevails.



## Algorithms

- The parameters  $a_1, \dots, b_n$  are chosen such that the series expansion of the Padé approximant coincides with the series expansion of  $f$  to the maximal possible order.
- The expansion points  $\pm\infty$  and  $\pm i\infty$  are not allowed.
- When `pade` cannot find the Padé approximant, it returns the function call.
- For `pade` to return the Padé approximant, a Taylor or Laurent series expansion of  $f$  must exist at the expansion point.

## Version History

Introduced in R2014b

## See Also

`series` | `taylor`

## Topics

“Padé Approximant” on page 3-212

## partfrac

Partial fraction decomposition

### Syntax

```
partfrac(expr,var)
partfrac(expr,var,Name,Value)
```

### Description

`partfrac(expr,var)` finds the partial fraction decomposition of `expr` with respect to `var`. If you do not specify `var`, then `partfrac` uses the variable determined by `symvar`.

`partfrac(expr,var,Name,Value)` finds the partial fraction decomposition using additional options specified by one or more `Name,Value` pair arguments.

### Examples

#### Partial Fraction Decomposition of Symbolic Expressions

Find partial fraction decomposition of univariate and multivariate expressions.

First, find partial fraction decomposition of univariate expressions. For expressions with one variable, you can omit specifying the variable.

```
syms x
partfrac(x^2/(x^3 - 3*x + 2))

ans =
5/(9*(x - 1)) + 1/(3*(x - 1)^2) + 4/(9*(x + 2))
```

Find partial fraction decomposition of a multivariate expression with respect to a particular variable.

```
syms a b
partfrac(a^2/(a^2 - b^2),a)

ans =
b/(2*(a - b)) - b/(2*(a + b)) + 1

partfrac(a^2/(a^2 - b^2),b)

ans =
a/(2*(a + b)) + a/(2*(a - b))
```

If you do not specify the variable, then `partfrac` computes partial fraction decomposition with respect to a variable determined by `symvar`.

```
symvar(a^2/(a^2 - b^2),1)
partfrac(a^2/(a^2 - b^2))

ans =
b
```

```
ans =
a/(2*(a + b)) + a/(2*(a - b))
```

### Factorization Modes

Choose a particular factorization mode by using the `FactorMode` input.

Find the partial fraction decomposition without specifying the factorization mode. By default, `partfrac` uses factorization over rational numbers. In this mode, `partfrac` keeps numbers in their exact symbolic form.

```
syms x
f = 1/(x^3 + 2);
partfrac(f,x)
```

```
ans =
1/(x^3 + 2)
```

Repeat the decomposition with numeric factorization over real numbers. In this mode, `partfrac` factors the denominator into linear and quadratic irreducible polynomials with real coefficients. This mode converts all numeric values to floating-point numbers.

```
partfrac(f,x,'FactorMode','real')
```

```
ans =
0.2099868416491455274612017678797/(x + 1.2599210498948731647672106072782) - ...
(0.2099868416491455274612017678797*x - 0.52913368398939982491723521309077)/(x^2 - ...
1.2599210498948731647672106072782*x + 1.5874010519681994747517056392723)
```

Repeat the decomposition with factorization over complex numbers. In this mode, `partfrac` reduces quadratic polynomials in the denominator to linear expressions with complex coefficients. This mode converts all numbers to floating point.

```
partfrac(f,x,'FactorMode','complex')
```

```
ans =
0.2099868416491455274612017678797/(x + 1.2599210498948731647672106072782) + ...
(- 0.10499342082457276373060088393985 - 0.1818539332862023392667876903163i)/...
(x - 0.62996052494743658238360530363911 - 1.0911236359717214035600726141898i) + ...
(- 0.10499342082457276373060088393985 + 0.1818539332862023392667876903163i)/...
(x - 0.62996052494743658238360530363911 + 1.0911236359717214035600726141898i)
```

Find the partial fraction decomposition of this expression using the full factorization mode. In this mode, `partfrac` factors the denominator into linear expressions, reducing quadratic polynomials to linear expressions with complex coefficients. This mode keeps numbers in their exact symbolic form.

```
pfFull = partfrac(f,x,'FactorMode','full')
```

```
pfFull =
2^(1/3)/(6*(x + 2^(1/3))) + ...
(2^(1/3)*((3^(1/2)*1i)/2 - 1/2))/(6*(x + 2^(1/3)*((3^(1/2)*1i)/2 - 1/2))) - ...
(2^(1/3)*((3^(1/2)*1i)/2 + 1/2))/(6*(x - 2^(1/3)*((3^(1/2)*1i)/2 + 1/2)))
```

Approximate the result with floating-point numbers by using `vpa`. Because the expression does not contain any symbolic parameters besides the variable `x`, the result is the same as in complex factorization mode.

```
vpa(pfFull)
```

```
ans =
0.2099868416491455274612017678797/(x + 1.2599210498948731647672106072782) + ...
```

```
(- 0.10499342082457276373060088393985 - 0.18185393932862023392667876903163i)/...
(x - 0.62996052494743658238360530363911 - 1.0911236359717214035600726141898i) +...
(- 0.10499342082457276373060088393985 + 0.18185393932862023392667876903163i)/...
(x - 0.62996052494743658238360530363911 + 1.0911236359717214035600726141898i)
```

In the complex mode, `partfrac` factors only those expressions in the denominator whose coefficients can be converted to floating-point numbers. Show this by replacing 2 in `f` with a symbolic variable and find the partial fraction decomposition in complex mode. `partfrac` returns the expression unchanged.

```
syms a
f = subs(f,2,a);
partfrac(f,x,'FactorMode','complex')
```

```
ans =
1/(x^3 + a)
```

When you use the full factorization mode, `partfrac` factors expressions in the denominator symbolically. Thus, `partfrac` in the full factorization mode factors the expression.

```
partfrac(1/(x^3 + a), x, 'FactorMode', 'full')

ans =
1/(3*(-a)^(2/3)*(x - (-a)^(1/3))) -...
((3^(1/2)*1i)/2 + 1/2)/(3*(-a)^(2/3)*(x + (-a)^(1/3)*((3^(1/2)*1i)/2 + 1/2))) +...
((3^(1/2)*1i)/2 - 1/2)/(3*(-a)^(2/3)*(x - (-a)^(1/3)*((3^(1/2)*1i)/2 - 1/2)))
```

### Full Factorization Mode Returns root

In full factorization mode, `partfrac` represents coefficients using `root` when it is not mathematically possible to find the coefficients as exact symbolic numbers. Show this behavior.

```
syms x
s = partfrac(1/(x^3 + x - 3), x, 'FactorMode','full')

s =
symsum(-(6*root(z^3 + z - 3, z, k)^2)/247 +...
(27*root(z^3 + z - 3, z, k))/247 +...
4/247)/(root(z^3 + z - 3, z, k) - x), k, 1, 3)
```

Approximate the result with floating-point numbers by using `vpa`.

```
vpa(s)

ans =
0.1846004942289254798185772017286/(x - 1.2134116627622296341321313773815) +...
(- 0.092300247114462739909288600864302 + 0.11581130283490645120989658654914i)/...
(x + 0.60670583138111481706606568869074 - 1.450612249188441526515442203395i) +...
(- 0.092300247114462739909288600864302 - 0.11581130283490645120989658654914i)/...
(x + 0.60670583138111481706606568869074 + 1.450612249188441526515442203395i)
```

### Numerators and Denominators of Partial Fraction Decomposition

Return a vector of numerators and a vector of denominators of the partial fraction decomposition.

First, find the partial fraction decomposition of the expression.

```
syms x
P = partfrac(x^2/(x^3 - 3*x + 2), x)

P =
5/(9*(x - 1)) + 1/(3*(x - 1)^2) + 4/(9*(x + 2))
```

Partial fraction decomposition is a sum of fractions. Use the `children` function to return a vector containing the terms of that sum. Then, use `numden` to extract the numerators and denominators of the terms.

```
C = children(P);
C = [C{:}];
[N,D] = numden(C)
```

```
N =
[ 5, 1, 4]
```

```
D =
[ 9*x - 9, 3*(x - 1)^2, 9*x + 18]
```

Reconstruct the partial fraction decomposition from the vectors of numerators and denominators.

```
P1 = sum(N./D)
```

```
P1 =
1/(3*(x - 1)^2) + 5/(9*x - 9) + 4/(9*x + 18)
```

Verify that the reconstructed expression, `P1`, is equivalent to the original partial fraction decomposition, `P`.

```
isAlways(P1 == P)
```

```
ans =
    logical
     1
```

## Input Arguments

### **expr** — Rational expression

symbolic expression | symbolic function

Rational expression, specified as a symbolic expression or function.

### **var** — Variable of interest

symbolic variable

Variable of interest, specified as a symbolic variable.

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `partfrac(1/(x^3 - 2),x,'FactorMode','real')`

### **FactorMode** — Factorization mode

'rational' (default) | 'real' | 'complex' | 'full'

Factorization mode, specified as the comma-separated pair consisting of 'FactorMode' and one of these character vectors.

|            |                                                                                                                                                                                                                                                                              |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 'rational' | Factorization over rational numbers.                                                                                                                                                                                                                                         |
| 'real'     | Factorization into linear and quadratic polynomials with real coefficients. The coefficients of the input must be convertible to real floating-point numbers.                                                                                                                |
| 'complex'  | Factorization into linear polynomials whose coefficients are floating-point numbers. The coefficients of the input must be convertible to floating-point numbers.                                                                                                            |
| 'full'     | Factorization into linear polynomials with exact symbolic coefficients. If <code>partfrac</code> cannot calculate coefficients as exact symbolic numbers, then <code>partfrac</code> represents coefficients by using <code>symsum</code> ranging over a <code>root</code> . |

## More About

### Partial Fraction Decomposition

Partial fraction decomposition is an operation on rational expressions.

$$f(x) = g(x) + \frac{p(x)}{q(x)},$$

Where the denominator of the expression can be written as  $q(x) = q_1(x)q_2(x)\dots$ , the partial fraction decomposition is an expression of this form.

$$f(x) = g(x) + \sum_j \frac{p_j(x)}{q_j(x)}$$

Here, the denominators  $q_j(x)$  are irreducible polynomials or powers of irreducible polynomials. The numerators  $p_j(x)$  are polynomials of smaller degrees than the corresponding denominators  $q_j(x)$ .

Partial fraction decomposition can simplify integration by integrating each term of the returned expression separately.

## Version History

Introduced in R2015a

### See Also

`children` | `coeffs` | `collect` | `combine` | `compose` | `divisors` | `expand` | `factor` | `horner` | `numden` | `rewrite` | `simplify` | `simplifyFraction`

### Topics

"Choose Function to Rearrange Expression" on page 3-123

# pdeCoefficients

Extract coefficients of partial differential equation

## Syntax

```
coeffs = pdeCoefficients(pdeeq,u)
symCoeffs = pdeCoefficients(pdeeq,u,'Symbolic',true)
```

## Description

`coeffs = pdeCoefficients(pdeeq,u)` extracts the coefficients of a partial differential equation (PDE) as a structure of double-precision numbers and function handles, which can be used as input of the `specifyCoefficients` function in Partial Differential Equation Toolbox™.

`pdeeq` is a scalar PDE or a PDE system in symbolic form that is a function of `u`. The `pdeCoefficients` function converts `pdeeq` into a system of equations of the form

$$\mathbf{m} \frac{\partial^2 \mathbf{u}}{\partial t^2} + \mathbf{d} \frac{\partial \mathbf{u}}{\partial t} - \nabla \cdot (\mathbf{c} \otimes \nabla \mathbf{u}) + \mathbf{a} \mathbf{u} = \mathbf{f}$$

and returns the structure `coeffs` that contains the coefficients `m`, `d`, `c`, `a`, and `f`. For more information, see “Equations You Can Solve Using Partial Differential Equation Toolbox” (Partial Differential Equation Toolbox).

If `pdeCoefficients` cannot convert a PDE into the divergence form above, then it issues a warning message and writes all remaining gradients to the `f` coefficient. Partial Differential Equation Toolbox will be unable to solve this type of PDE.

`symCoeffs = pdeCoefficients(pdeeq,u,'Symbolic',true)` returns the PDE coefficients as a structure of symbolic expressions.

## Examples

### Extract Coefficients From a Symbolic PDE

Create a symbolic PDE that represents the Laplacian of the variable `u(x,y)`.

```
syms u(x,y)
pdeeq = laplacian(u,[x y]) == -3
```

```
pdeeq(x, y) =
    \frac{\partial^2}{\partial x^2} u(x,y) + \frac{\partial^2}{\partial y^2} u(x,y) = -3
```

Extract the coefficients of the PDE.

```
coeffs = pdeCoefficients(pdeeq,u)

coeffs = struct with fields:
    a: 0
```

```
c: [4×1 double]
m: 0
d: 0
f: -3
```

`pdeCoefficients` converts the symbolic PDE into a scalar PDE equation of the form

$$m \frac{\partial^2 u}{\partial t^2} + d \frac{\partial u}{\partial t} - \nabla \cdot (c \nabla u) + au = f$$

and extracts the coefficients `m`, `d`, `c`, `a`, and `f` into the structure `coeffs`. For 2-D systems of  $N$  equations, `c` is a tensor with  $4N^2$  elements. For more information, see “`c` Coefficient for `specifyCoefficients`” (Partial Differential Equation Toolbox). In this case,  $N = 1$ , so the coefficient `c` is a 4-by-1 column vector that represents  $c_{xx}$ ,  $c_{xy}$ ,  $c_{yx}$ , and  $c_{yy}$ .

```
coeffs.c
```

```
ans = 4×1

    -1
     0
     0
    -1
```

### Solve Homogenous Laplace's Equation

Solve a 2-D homogenous Laplace's equation in the domain bounded by a unit circle. Use the `pdeCoefficients` function to extract the coefficients of the Laplace's equation.

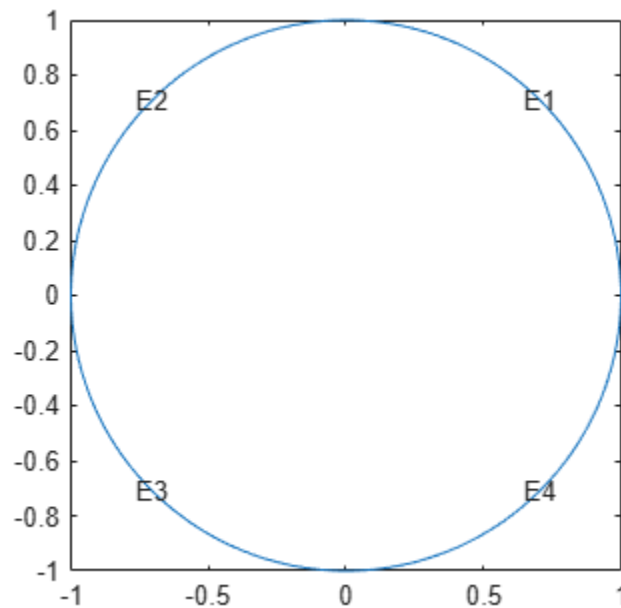
Create the PDE model and include the geometry.

```
model = createpde;
geometryFromEdges(model,@circleg);
```

Plot the geometry and display the edge labels for use in the boundary condition definition.

```
figure
pdegplot(model,'EdgeLabels','on')
axis equal
```





Create a symbolic expression `pdeeq` that represents the Laplace's equation.

```
syms u(x,y)
pdeeq = laplacian(u,[x y])
```

```
pdeeq(x, y) =

$$\frac{\partial^2}{\partial x^2} u(x, y) + \frac{\partial^2}{\partial y^2} u(x, y)$$

```

Extract the coefficients of the Laplace's equation.

```
coeffs = pdeCoefficients(pdeeq,u)
```

```
coeffs = struct with fields:
    a: 0
    c: [4x1 double]
    m: 0
    d: 0
    f: 0
```

Specify the coefficients of the PDE model.

```
specifyCoefficients(model,'m',coeffs.m,'d',coeffs.d, ...
    'c',coeffs.c,'a',coeffs.a,'f',coeffs.f);
```

Apply the Dirichlet boundary condition  $u(x, y) = x^4 + y^4$  at all 4 edges that form the circle.

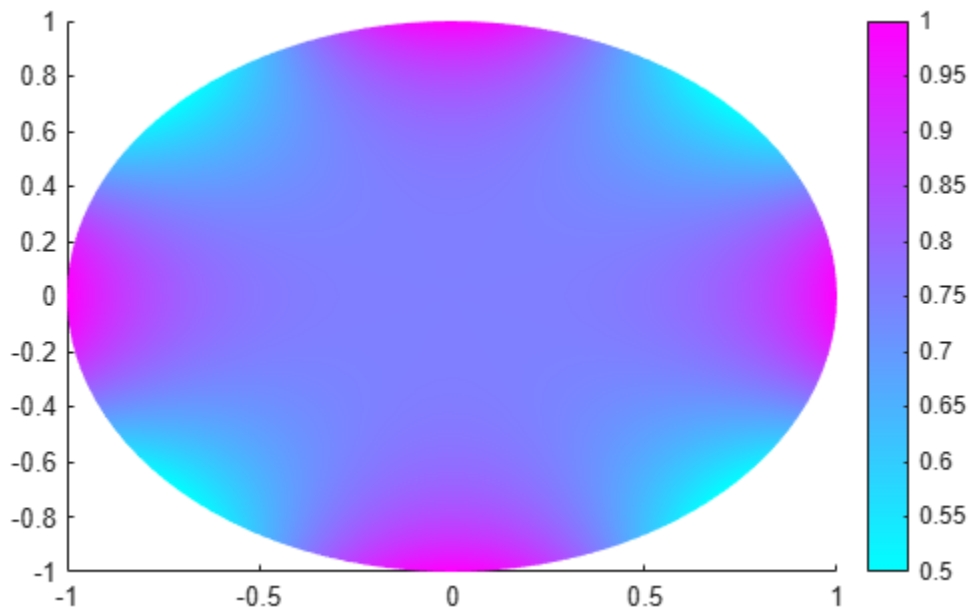
```
applyBoundaryCondition(model,'dirichlet','Edge',1:4,'u',@(region,state) region.x.^4 + region.y.^4)
```

Generate the default mesh for the geometry.

```
generateMesh(model);
```

Solve the PDE and plot the solution.

```
results = solvepde(model);
pdeplot(model, 'XYData', results.NodalSolution)
```



### Solve Poisson's Equation with Nonconstant f Coefficient

Solve a 2-D Poisson's equation in the domain bounded by a unit circle. The divergence form of the PDE has nonconstant f coefficient. You can solve the PDE by extracting the coefficients using `pdeCoefficients` and specifying the coefficients in the PDE model using `specifyCoefficients`.

Create the PDE model and include the geometry.

```
model = createpde;
geometryFromEdges(model,@circleg);
```

Create a symbolic expression `pdeeq` that represents Poisson's equation.

```
syms u(x,y)
pdeeq = diff(u,x,x) + diff(u,y,y) - 1/(x^2 + y^2)
```

```
pdeeq(x, y) =
    
$$\frac{\partial^2}{\partial x^2} u(x, y) - \frac{1}{x^2 + y^2} + \frac{\partial^2}{\partial y^2} u(x, y)$$

```

Extract the coefficients of the equation.

```
coeffs = pdeCoefficients(pdeeq,u)

coeffs = struct with fields:
    a: 0
```

```

c: [4×1 double]
m: 0
d: 0
f: @makeCoefficient/coefficientFunction

```

The `f` coefficient is not a constant. It is displayed as `@makeCoefficient/coefficientFunction`, which indicates that it has been returned from the workspace of some internal function. To show the formula behind the function handle, use `coeffs.f('show')`.

```
coeffs.f('show')
```

```
ans =
```

$$\frac{1}{x^2 + y^2}$$

Specify the coefficients of the PDE model.

```
specifyCoefficients(model,'m',coeffs.m,'d',coeffs.d, ...
    'c',coeffs.c,'a',coeffs.a,'f',coeffs.f);
```

Apply the Dirichlet boundary condition  $u(x, y) = 0$  at all 4 edges that form the circle.

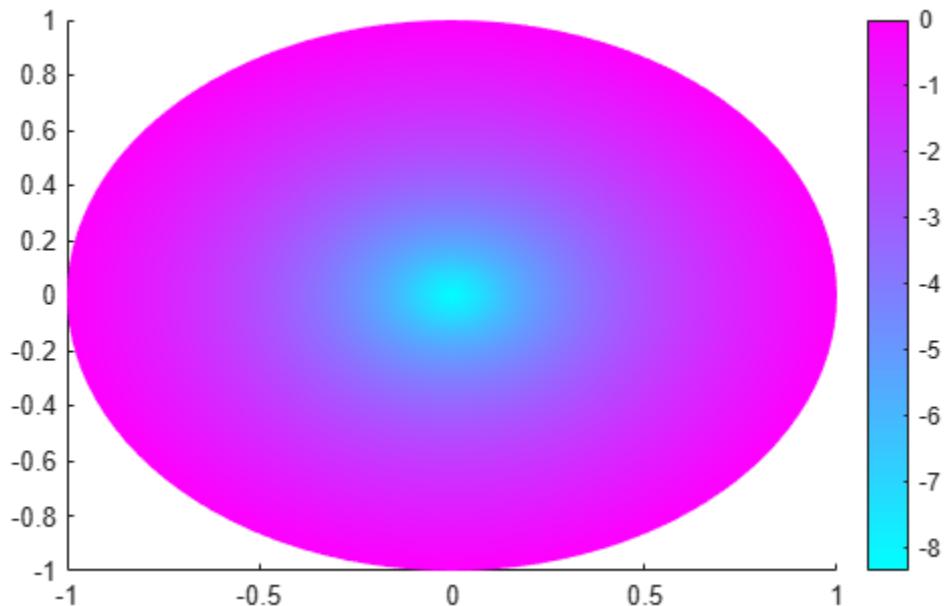
```
applyBoundaryCondition(model,'dirichlet','Edge',1:4,'u',0);
```

Generate the default mesh for the geometry.

```
generateMesh(model);
```

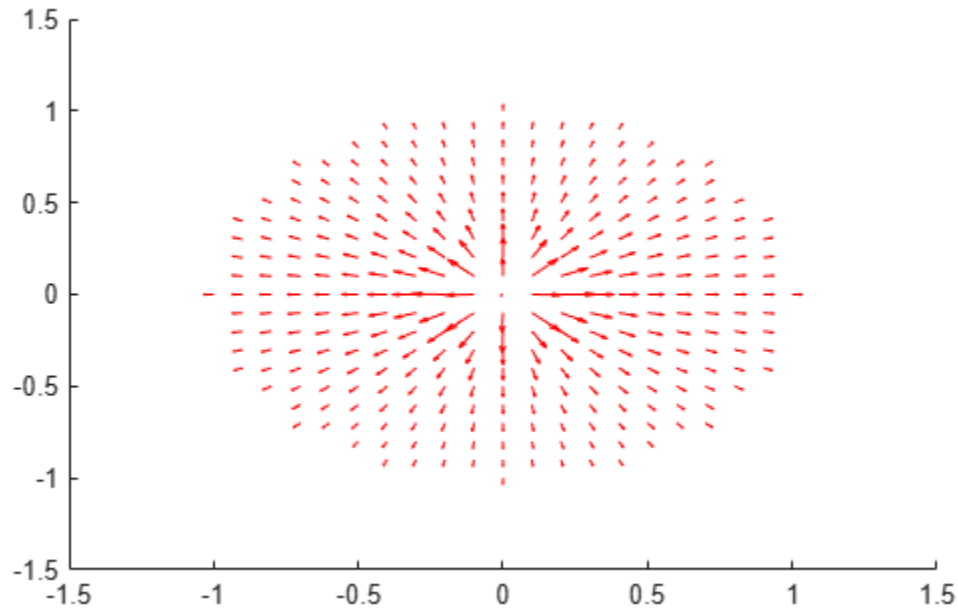
Solve the PDE. Plot the nodal solution using the option 'XYData' in the `pdeplot` function.

```
results = solvepde(model);
pdeplot(model,'XYData',results.NodalSolution)
```



Plot the gradient of the solution at the nodal locations using the option 'FlowData'.

```
pdeplot(model, 'FlowData', [results.XGradients results.YGradients])
```



### PDE With No Divergence Form

Construct a PDE with no divergence form.

```
syms u(x,y)
pdeeq = diff(u,x,x) + cos(x+y)/4*diff(u,x,y) + 1/2*diff(u,y,y)
pdeeq(x, y) =
```

$$\frac{\partial^2}{\partial x^2} u(x, y) + \frac{\cos(x+y)}{4} \frac{\partial}{\partial y} \frac{\partial}{\partial x} u(x, y) + \frac{\partial^2}{\partial y^2} u(x, y)$$

Extract its coefficients. When `pdeCoefficients` cannot convert a PDE into the divergence form

$$m \frac{\partial^2 u}{\partial t^2} + d \frac{\partial u}{\partial t} - \nabla \cdot (c \otimes \nabla u) + au = f,$$

it issues a warning message, but it still produces output.

```
coeffs = pdeCoefficients(pdeeq,u)
```

Warning: After extracting m, d, and c, some gradients remain. Writing all remaining terms to f.

```
coeffs = struct with fields:
  a: 0
  c: @makeCoefficient/coefficientFunction
```

```

m: 0
d: 0
f: @makeCoefficient/coefficientFunction

```

To show the function handles in the extracted coefficients **c** and **f**, use the option 'show'. All remaining gradients in the PDE are written to the **f** coefficient.

```
coeffs.c('show')
```

```
ans =
```

$$\begin{pmatrix} -1 \\ \frac{1}{8} - \frac{\cos(x+y)}{4} \\ \frac{1}{8} - \frac{\cos(x+y)}{4} \\ -\frac{1}{2} \end{pmatrix}$$

```
coeffs.f('show')
```

```
ans =
```

$$\frac{\cos(x+y) \frac{\partial}{\partial y} \frac{\partial}{\partial x} u(x,y)}{4} - \frac{\sin(x+y) \frac{\partial}{\partial y} u(x,y)}{4} - \frac{\sin(x+y) \frac{\partial}{\partial x} u(x,y)}{4} - \frac{\frac{\partial}{\partial y} \frac{\partial}{\partial x} u(x,y)}{4}$$

Since the PDE has no divergence form required by the PDE Toolbox, the toolbox will be unable to solve this PDE.

## Solve Time-Dependent Wave Equation

Solve a time-dependent wave equation in the domain bounded by a unit circle. The wave equation is a PDE with a scalar function  $u(t, x, y)$  that depends on time  $t$  and coordinates  $x$  and  $y$ .

Create the PDE model and include the geometry.

```

model = createpde(1);
geometryFromEdges(model,@circleg);

```

Create a symbolic PDE that represents the wave equation.

```

syms u(t,x,y)
pdeeq = diff(u,t,t) - laplacian(u,[x y])

```

```
pdeeq(t, x, y) =
```

$$\frac{\partial^2}{\partial t^2} u(t, x, y) - \frac{\partial^2}{\partial x^2} u(t, x, y) - \frac{\partial^2}{\partial y^2} u(t, x, y)$$

Extract the coefficients of the PDE.

```
coeffs = pdeCoefficients(pdeeq,u)
```

```

coeffs = struct with fields:
    a: 0
    c: [4x1 double]

```

```

m: 1
d: 0
f: 0

```

Specify the coefficients of the PDE model.

```

specifyCoefficients(model,'m',coeffs.m,'d',coeffs.d, ...
    'c',coeffs.c,'a',coeffs.a,'f',coeffs.f);

```

Set the initial conditions of the time-dependent problem on the entire geometry.

```

setInitialConditions(model,0,1);

```

Apply the Dirichlet boundary condition  $u(x, y) = 0$  at all 4 edges that form the circle.

```

applyBoundaryCondition(model,'dirichlet','Edge',1:4,'u',0);

```

Generate the default mesh for the geometry.

```

generateMesh(model);

```

Find the solutions of the time-dependent PDE at a time range from 0s to 50s with a 2s interval.

```

results = solvepde(model,linspace(0,2,50));

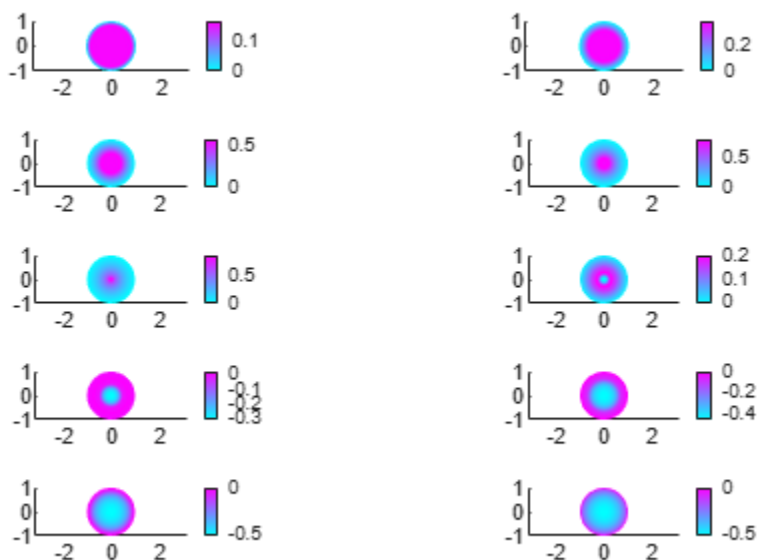
```

Plot the solution of the wave equation for each 5s interval.

```

figure;
for k = 1:10
    subplot(5,2,k);
    pdeplot(model,'XYData',results.NodalSolution(:,k*5))
    axis equal
end

```



## System of Several PDEs

Solve a system of two second-order PDEs. You can solve the PDE system by extracting the PDE coefficients symbolically using `pdeCoefficients`, converting the coefficients to double-precision numbers using `pdeCoefficientsToDouble`, and specifying the coefficients in the PDE model using `specifyCoefficients`.

The system of PDEs represents the deflection of a clamped structural plate under a uniform pressure load. The system of PDEs with the dependent variables  $u_1$  and  $u_2$  is given by

$$-\nabla^2 u_1 + u_2 = 0,$$

$$-D\nabla^2 u_2 = p,$$

where  $D$  is the bending stiffness of the plate given by

$$D = \frac{Eh^3}{12(1-\nu^2)},$$

and  $E$  is the modulus of elasticity,  $\nu$  is Poisson's ratio,  $h$  is the plate thickness,  $u_1$  is the transverse deflection of the plate, and  $p$  is the pressure load.

Create a PDE model for the system of two equations.

```
model = createpde(2);
```

Create a square geometry. Specify the side length of the square. Then include the geometry in the PDE model.

```
len = 10.0;
gdm = [3 4 0 len len 0 0 0 len len]';
g = decsg(gdm,'S1',('S1'))';
geometryFromEdges(model,g);
```

Specify the values of the physical parameters of the system. Let the external pressure  $p$  be a symbolic variable `pres` that can take any value.

```
E = 1.0e6;
h_thick = 0.1;
nu = 0.3;
D = E*h_thick^3/(12*(1 - nu^2));
syms pres
```

Declare the PDE system as a system symbolic equations. Extract the coefficients of the PDE and return them in symbolic form.

```
syms u1(x,y) u2(x,y)
pdeeq = [-laplacian(u1) + u2; -D*laplacian(u2) - pres];
symCoeffs = pdeCoefficients(pdeeq,[u1 u2],'Symbolic',true)

symCoeffs = struct with fields:
    m: 0
    a: [2x2 sym]
```

```

c: [4×4 sym]
f: [2×1 sym]
d: 0

```

Display the coefficients  $m$ ,  $a$ ,  $c$ ,  $f$ , and  $d$ .

```
structfun(@disp,symCoeffs)
```

```
0
```

$$\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{25000}{273} & 0 \\ 0 & 0 & 0 & \frac{25000}{273} \end{pmatrix}$$

$$\begin{pmatrix} 0 \\ \text{pres} \end{pmatrix}$$

```
0
```

Substitute a value for `pres` using the `subs` function. Since the outputs of `subs` are symbolic objects, use the `pdeCoefficientsToDouble` function to convert the coefficients to the `double` data type, which makes them valid inputs for the Partial Differential Equation Toolbox.

```

symCoeffs = subs(symCoeffs,pres,2);
coeffs = pdeCoefficientsToDouble(symCoeffs)

```

```
coeffs = struct with fields:
```

```

a: [4×1 double]
c: [16×1 double]
m: 0
d: 0
f: [2×1 double]

```

Specify the PDE coefficients for the PDE model.

```

specifyCoefficients(model,'m',coeffs.m,'d',coeffs.d, ...
    'c',coeffs.c,'a',coeffs.a,'f',coeffs.f);

```

Specify spring stiffness. Specify boundary conditions by defining distributed springs on all four edges.

```

k = 1e7;
bOuter = applyBoundaryCondition(model,'neumann','Edge',(1:4), ...
    'g',[0 0],'q',[0 0; k 0]);

```

Specify the mesh size of the geometry and generate a mesh for the PDE model.

```

hmax = len/20;
generateMesh(model,'Hmax',hmax);

```

Solve the model.



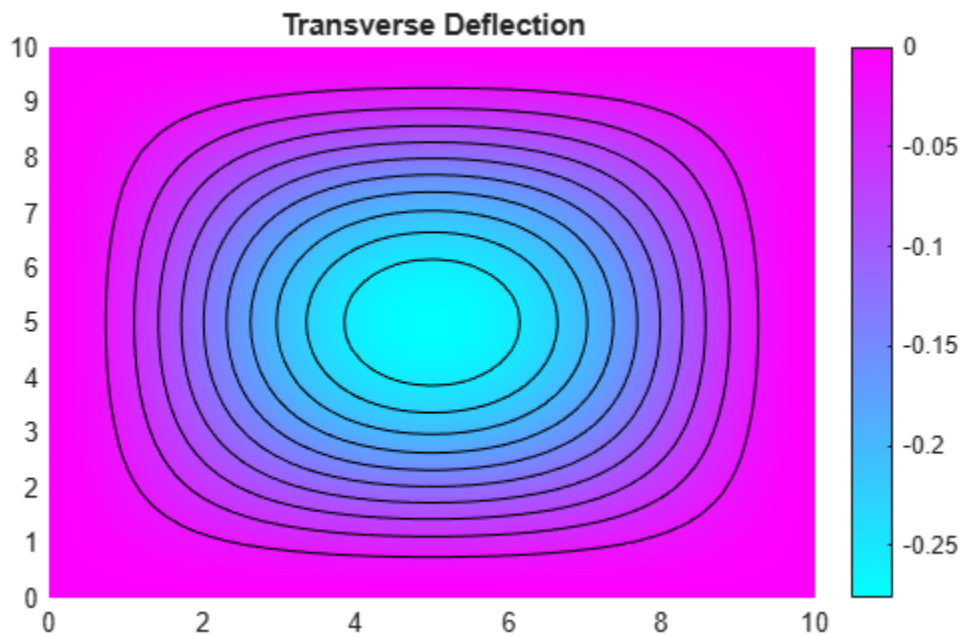
```
res = solvepde(model);
```

Access the solution at the nodal locations.

```
u = res.NodalSolution;
```

Plot the transverse deflection of the plate.

```
numNodes = size(model.Mesh.Nodes,2);
figure;
pdeplot(model,'XYData',u(1:numNodes),'contour','on')
title 'Transverse Deflection'
```



Find the transverse deflection at the plate center.

```
wMax = min(u(1:numNodes,1))
```

```
wMax =  
-0.2763
```

Compare the result with the deflection at the plate center computed analytically.

```
pres = 2;  
wMax = -.0138*pres*len^4/(E*h_thick^3)
```

```
wMax =  
-0.2760
```

## Input Arguments

### pdeeq — PDE in symbolic form

symbolic equation | symbolic expression | symbolic vector

PDE in symbolic form, specified as a symbolic equation, expression, or a symbolic vector.

### **u — Dependent variables of PDE**

symbolic function

Dependent variables of PDE, specified as a symbolic function. `u` must contain stationary or time-dependent variables in two or three dimensions. For example, create the variable `u` using one of these statements:

- `syms u(x,y)`
- `syms u(t,x,y)`
- `syms u(x,y,z)`
- `syms u(t,x,y,z)`

## **Output Arguments**

### **coeffs — Coefficients of PDE operating on doubles**

structure of doubles and function handles

Coefficients of PDE operating on doubles, returned as a structure of double-precision numbers and function handles as required by the `specifyCoefficients` function. The fields of the structure are `a`, `c`, `m`, `d`, and `f`. For details on interpreting the coefficients in the format required by Partial Differential Equation Toolbox, see:

- “c Coefficient for `specifyCoefficients`” (Partial Differential Equation Toolbox)
- “m, d, or a Coefficient for `specifyCoefficients`” (Partial Differential Equation Toolbox)
- “f Coefficient for `specifyCoefficients`” (Partial Differential Equation Toolbox)

When `pdeCoefficients` returns a coefficient as a function handle, the function handle takes two structures as input arguments, `location` and `state`, and returns double-precision output. The function handle is displayed as `@makeCoefficient/coefficientFunction` in the Command Window. To display the formula of the function handle in `coeffs.f` in symbolic form, use `coeffs.f('show')`.

In some cases, not all generated coefficients can be used by `specifyCoefficients`. For example, the `d` coefficient must take a special matrix form when `m` is nonzero. For more details, see “d Coefficient When m Is Nonzero” (Partial Differential Equation Toolbox).

### **symCoeffs — Coefficients of PDE in symbolic form**

structure of symbolic expressions

Coefficients of PDE in symbolic form, returned as a structure of symbolic expressions.

## **Version History**

**Introduced in R2021a**

### **See Also**

`syms` | `diff` | `laplacian` | `pdeCoefficientsToDouble` | `specifyCoefficients`

**Topics**

"Solve Partial Differential Equation of Nonlinear Heat Transfer" on page 3-275

## pdeCoefficientsToDouble

Convert symbolic PDE coefficients to double format

### Syntax

```
coeffs = pdeCoefficientsToDouble(symCoeffs)
coeffs = pdeCoefficientsToDouble(symCoeffs,u)
```

### Description

`coeffs = pdeCoefficientsToDouble(symCoeffs)` converts the symbolic objects of the structure `symCoeffs` to double-precision numbers or function handles. The output is a structure `coeffs` that can then be used to define the coefficients of a PDE model by calling `specifyCoefficients` in Partial Differential Equation Toolbox.

The structure `coeffs` contains the coefficients `m`, `d`, `c`, `a`, and `f` for the PDE system with the form

$$\mathbf{m} \frac{\partial^2 \mathbf{u}}{\partial t^2} + \mathbf{d} \frac{\partial \mathbf{u}}{\partial t} - \nabla \cdot (\mathbf{c} \otimes \nabla \mathbf{u}) + \mathbf{a} \mathbf{u} = \mathbf{f}$$

that can be solved in Partial Differential Equation Toolbox. For more information, see “Equations You Can Solve Using Partial Differential Equation Toolbox” (Partial Differential Equation Toolbox).

`coeffs = pdeCoefficientsToDouble(symCoeffs,u)` converts the symbolic objects of the structure `symCoeffs` to double-precision numbers or function handles for the coefficients of a PDE system with the dependent variables `u`.

### Examples

#### Extract Coefficients from a Symbolic PDE

Create a symbolic PDE that represents the Laplacian of the variable  $u(x,y)$ .

```
syms u(x,y) f
pdeeq = laplacian(u,[x y]) == f
```

```
pdeeq(x, y) =
    \frac{\partial^2}{\partial x^2} u(x,y) + \frac{\partial^2}{\partial y^2} u(x,y) = f
```

Extract the coefficients of the PDE as symbolic expressions and display their values.

```
symCoeffs = pdeCoefficients(pdeeq,u,'Symbolic',true);
structfun(@disp,symCoeffs)
```

```
0
```

```
0
```

$$\begin{pmatrix} -1 & 0 \\ 0 & -1 \end{pmatrix}$$

$$f$$

$$0$$

`pdeCoefficients` converts the symbolic PDE into a scalar PDE equation of the form

$$m \frac{\partial^2 u}{\partial t^2} + d \frac{\partial u}{\partial t} - \nabla \cdot (c \nabla u) + au = f,$$

and extract the coefficients `a`, `c`, `m`, `d`, and `f` into the structure `symCoeffs`.

Choose a value for `f`. Since `symCoeffs` are symbolic objects, use `pdeCoefficientsToDouble` to convert the coefficients to double data type. The coefficients with double data type are valid inputs for the `specifyCoefficients` function in the Partial Differential Equation Toolbox.

```
symCoeffs = subs(symCoeffs,f,-3);
coeffs = pdeCoefficientsToDouble(symCoeffs)
```

```
coeffs = struct with fields:
  a: 0
  c: [4×1 double]
  m: 0
  d: 0
  f: -3
```

## System of Several PDEs

Solve a system of two second-order PDEs. You can solve the PDE system by extracting the PDE coefficients symbolically using `pdeCoefficients`, converting the coefficients to double-precision numbers using `pdeCoefficientsToDouble`, and specifying the coefficients in the PDE model using `specifyCoefficients`.

The system of PDEs represents the deflection of a clamped structural plate under a uniform pressure load. The system of PDEs with the dependent variables  $u_1$  and  $u_2$  is given by

$$-\nabla^2 u_1 + u_2 = 0,$$

$$-D \nabla^2 u_2 = p,$$

where  $D$  is the bending stiffness of the plate given by

$$D = \frac{Eh^3}{12(1-\nu^2)},$$

and  $E$  is the modulus of elasticity,  $\nu$  is Poisson's ratio,  $h$  is the plate thickness,  $u_1$  is the transverse deflection of the plate, and  $p$  is the pressure load.

Create a PDE model for the system of two equations.

```
model = createpde(2);
```

Create a square geometry. Specify the side length of the square. Then include the geometry in the PDE model.

```
len = 10.0;
gdm = [3 4 0 len len 0 0 0 len len]';
g = decsg(gdm,'S1','S1');
geometryFromEdges(model,g);
```

Specify the values of the physical parameters of the system. Let the external pressure  $p$  be a symbolic variable `pres` that can take any value.

```
E = 1.0e6;
h_thick = 0.1;
nu = 0.3;
D = E*h_thick^3/(12*(1 - nu^2));
syms pres
```

Declare the PDE system as a system symbolic equations. Extract the coefficients of the PDE and return them in symbolic form.

```
syms u1(x,y) u2(x,y)
pdeeq = [-laplacian(u1) + u2; -D*laplacian(u2) - pres];
symCoeffs = pdeCoefficients(pdeeq,[u1 u2],'Symbolic',true)
```

```
symCoeffs = struct with fields:
    m: 0
    a: [2x2 sym]
    c: [4x4 sym]
    f: [2x1 sym]
    d: 0
```

Display the coefficients `m`, `a`, `c`, `f`, and `d`.

```
structfun(@disp,symCoeffs)
```

```
0
```

$$\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{25000}{273} & 0 \\ 0 & 0 & 0 & \frac{25000}{273} \end{pmatrix}$$

$$\begin{pmatrix} 0 \\ pres \end{pmatrix}$$

```
0
```

Substitute a value for `pres` using the `subs` function. Since the outputs of `subs` are symbolic objects, use the `pdeCoefficientsToDouble` function to convert the coefficients to the `double` data type, which makes them valid inputs for the Partial Differential Equation Toolbox.

```
symCoeffs = subs(symCoeffs,pres,2);
coeffs = pdeCoefficientsToDouble(symCoeffs)
```

```
coeffs = struct with fields:
    a: [4×1 double]
    c: [16×1 double]
    m: 0
    d: 0
    f: [2×1 double]
```

Specify the PDE coefficients for the PDE model.

```
specifyCoefficients(model,'m',coeffs.m,'d',coeffs.d, ...
    'c',coeffs.c,'a',coeffs.a,'f',coeffs.f);
```

Specify spring stiffness. Specify boundary conditions by defining distributed springs on all four edges.

```
k = 1e7;
bOuter = applyBoundaryCondition(model,'neumann','Edge',(1:4), ...
    'g',[0 0],'q',[0 0; k 0]);
```

Specify the mesh size of the geometry and generate a mesh for the PDE model.

```
hmax = len/20;
generateMesh(model,'Hmax',hmax);
```

Solve the model.

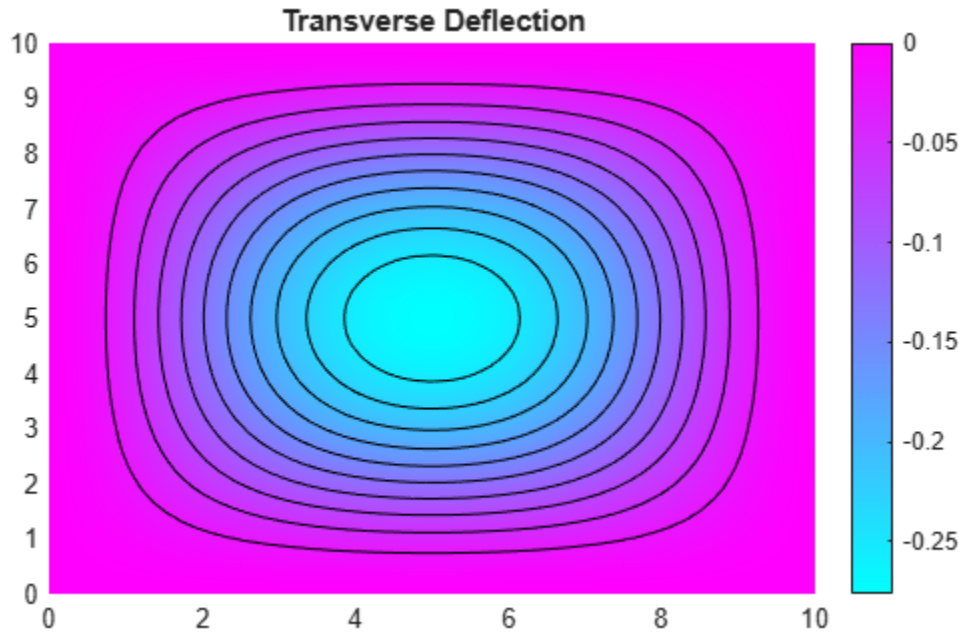
```
res = solvepde(model);
```

Access the solution at the nodal locations.

```
u = res.NodalSolution;
```

Plot the transverse deflection of the plate.

```
numNodes = size(model.Mesh.Nodes,2);
figure;
pdeplot(model,'XYData',u(1:numNodes),'contour','on')
title 'Transverse Deflection'
```



Find the transverse deflection at the plate center.

```
wMax = min(u(1:numNodes,1))
```

```
wMax =  
-0.2763
```

Compare the result with the deflection at the plate center computed analytically.

```
pres = 2;  
wMax = -.0138*pres*len^4/(E*h_thick^3)
```

```
wMax =  
-0.2760
```

### Specify Dependent Variables of PDE System

Create a PDE system of four equations with four dependent variables,  $A$ ,  $E$ ,  $P$ , and  $T$ .

```
syms A(x,y,z,t) E(x,y,z,t) P(x,y,z,t) T(x,y,z,t)  
eqn1 = diff(A,t) == -A*E*(-exp(T));  
eqn2 = diff(P,t) == -P*E*(-exp(T));  
eqn3 = diff(E,t) == -A*E*(-exp(T)) - P*E*(-exp(T));  
eqn4 = diff(T,t) == laplacian(T,[x y z]) + A*E*(-exp(T)) + P*E*(-exp(T));  
pdeeq = [eqn1; eqn2; eqn3; eqn4]  
  
pdeeq(x, y, z, t) =
```



$$\begin{pmatrix} \frac{\partial}{\partial t} A(x, y, z, t) = \sigma_2 \\ \frac{\partial}{\partial t} P(x, y, z, t) = \sigma_1 \\ \frac{\partial}{\partial t} E(x, y, z, t) = \sigma_2 + \sigma_1 \\ \frac{\partial}{\partial t} T(x, y, z, t) = \frac{\partial^2}{\partial x^2} T(x, y, z, t) + \frac{\partial^2}{\partial y^2} T(x, y, z, t) + \frac{\partial^2}{\partial z^2} T(x, y, z, t) - \sigma_2 - \sigma_1 \end{pmatrix}$$

where

$$\sigma_1 = e^{T(x, y, z, t)} E(x, y, z, t) P(x, y, z, t)$$

$$\sigma_2 = e^{T(x, y, z, t)} A(x, y, z, t) E(x, y, z, t)$$

Extract the coefficients of the PDE system as symbolic expressions. Specify the variable `u` to represent the dependent variables.

```
u = [A E P T];
symCoeffs = pdeCoefficients(pdeeq,u,'Symbolic',true)
```

`symCoeffs` = struct with fields:

```
m: [4×4 sym]
a: [4×4 sym]
c: [12×12 sym]
f: [4×1 sym]
d: [4×4 sym]
```

Display the coefficients `m`, `a`, `c`, `f`, and `d`.

```
structfun(@disp,symCoeffs)
```

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$$\begin{pmatrix} -\sigma_2 & 0 & 0 & 0 \\ 0 & \sigma_1 & 0 & 0 \\ -\sigma_2 & \sigma_1 & 0 & 0 \\ \sigma_2 e^{T(x, y, z, t)} P(x, y, z, t) & 0 & 0 & 0 \end{pmatrix}$$

where

$$\sigma_1 = -e^{T(x, y, z, t)} P(x, y, z, t)$$

$$\sigma_2 = e^{T(x, y, z, t)} E(x, y, z, t)$$

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$
$$\begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$
$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Convert the coefficients to the `double` data type so that they are valid inputs for the `specifyCoefficients` function in Partial Differential Equation Toolbox. Because the `a` coefficient in `symCoeffs` contains the dependent variables and is not constant, calling `pdeCoefficientsToDouble(symCoeffs)` without the second input argument can return an error. Instead, specify the second argument as the dependent variables `u` when using `pdeCoefficientsToDouble`.

```
coeffs = pdeCoefficientsToDouble(symCoeffs,u)
```

```
coeffs = struct with fields:
  a: @makeCoefficient/coefficientFunction
  c: [144x1 double]
  m: 0
  d: [16x1 double]
  f: 0
```

## Input Arguments

### **symCoeffs** — Coefficients of PDE in symbolic form

structure of symbolic expressions

Coefficients of PDE in symbolic form, specified as a structure of symbolic expressions.

### **u** — Dependent variables of PDE

symbolic function

Dependent variables of PDE, specified as a symbolic function. The argument `u` must contain stationary or time-dependent variables in two or three dimensions.

Example: `syms E(x,y,z,t) B(x,y,z,t); u = [E B];`

## Output Arguments

### **coeffs** — Coefficients of PDE operating on doubles

structure of doubles and function handles

Coefficients of PDE operating on doubles, returned as a structure of double-precision numbers and function handles as required by the `specifyCoefficients` function. The fields of the structure are `a`, `c`, `m`, `d`, and `f`. For details on interpreting the coefficients in the format required by Partial Differential Equation Toolbox, see:

- “c Coefficient for `specifyCoefficients`” (Partial Differential Equation Toolbox)
- “m, d, or a Coefficient for `specifyCoefficients`” (Partial Differential Equation Toolbox)
- “f Coefficient for `specifyCoefficients`” (Partial Differential Equation Toolbox)

When `pdeCoefficientsToDouble` returns a coefficient as a function handle, the function handle takes two structures as input arguments, `location` and `state`, and returns double-precision output. The function handle is displayed as `@makeCoefficient/coefficientFunction` in the Command Window. To display the formula of the function handle in `coeffs.f` in symbolic form, use `coeffs.f('show')`.

In some cases, not all generated coefficients can be used by `specifyCoefficients`. For example, the `d` coefficient must take a special matrix form when `m` is nonzero. For more details, see “d Coefficient When m Is Nonzero” (Partial Differential Equation Toolbox).

## Version History

Introduced in R2021a

### **R2023a: Specify dependent variables when converting PDE coefficients**

You can specify the dependent variables `u` of a PDE system as the second input argument of `pdeCoefficientsToDouble` when converting symbolic coefficients of the PDE system to double-precision numbers or function handles. For example, see “Specify Dependent Variables of PDE System” on page 6-1168.

## See Also

`syms` | `diff` | `laplacian` | `pdeCoefficients` | `specifyCoefficients`

## Topics

“Solve Partial Differential Equation of Nonlinear Heat Transfer” on page 3-275

## piecewise

Conditionally defined expression or function

### Syntax

```
pw = piecewise(cond1, val1, cond2, val2, ...)  
pw = piecewise(cond1, val1, cond2, val2, ..., otherwiseVal)
```

### Description

`pw = piecewise(cond1, val1, cond2, val2, ...)` returns the piecewise expression or function `pw` whose value is `val1` when condition `cond1` is true, is `val2` when `cond2` is true, and so on. If no condition is true, the value of `pw` is `NaN`.

`pw = piecewise(cond1, val1, cond2, val2, ..., otherwiseVal)` returns the piecewise expression or function `pw` that has the value `otherwiseVal` if no condition is true.

### Examples

#### Define and Evaluate Piecewise Expression

Define the following piecewise expression by using `piecewise`.

$$y = \begin{cases} -1 & x < 0 \\ 1 & x > 0 \end{cases}$$

```
syms x  
y = piecewise(x < 0, -1, x > 0, 1)
```

$$y = \begin{cases} -1 & \text{if } x < 0 \\ 1 & \text{if } 0 < x \end{cases}$$

Evaluate `y` at `-2`, `0`, and `2` by using `subs` to substitute for `x`. Because `y` is undefined at `x = 0`, the value is `NaN`.

```
subs(y, x, [-2 0 2])  
ans = (-1 NaN 1)
```

#### Define Piecewise Function

Define the following function symbolically.

$$y(x) = \begin{cases} -1 & x < 0 \\ 1 & x > 0 \end{cases}$$

```
syms y(x)
y(x) = piecewise(x < 0, -1, x > 0, 1)
```

$$y(x) = \begin{cases} -1 & \text{if } x < 0 \\ 1 & \text{if } 0 < x \end{cases}$$

Because  $y(x)$  is a symbolic function, you can directly evaluate it for values of  $x$ . Evaluate  $y(x)$  at -2, 0, and 2. Because  $y(x)$  is undefined at  $x = 0$ , the value is NaN. For details, see “Create Symbolic Functions” on page 1-9.

```
y([-2 0 2])
ans = (-1 NaN 1)
```

### Set Value When No Condition Is True

Set the value of a piecewise function when no condition is true (called *otherwise value*) by specifying an additional input argument. If an additional argument is not specified, the default otherwise value of the function is NaN.

Define the piecewise function

$$y = \begin{cases} -2 & x < -2 \\ 0 & -2 < x < 0 \\ 1 & \text{otherwise} \end{cases}$$

```
syms y(x)
y(x) = piecewise(x < -2, -2, (-2 < x) & (x < 0), 0, 1)
```

$$y(x) = \begin{cases} -2 & \text{if } x < -2 \\ 0 & \text{if } x \in (-2, 0) \\ 1 & \text{otherwise} \end{cases}$$

Evaluate  $y(x)$  between -3 and 1 by generating values of  $x$  using `linspace`. At -2 and 0,  $y(x)$  evaluates to 1 because the other conditions are not true.

```
xvalues = linspace(-3,1,5)
xvalues = 1×5
    -3    -2    -1     0     1

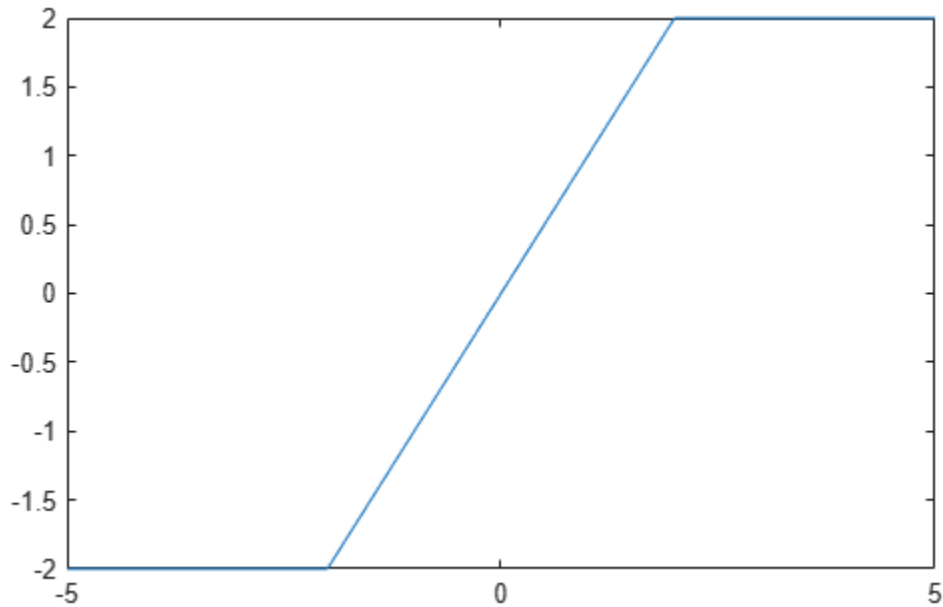
yvalues = y(xvalues)
yvalues = (-2 1 0 1 1)
```

### Plot Piecewise Expression

Plot the following piecewise expression by using `fplot`.

$$y = \begin{cases} -2 & x < -2 \\ x & -2 < x < 2 \\ 2 & x > 2 \end{cases}$$

```
syms x
y = piecewise(x < -2, -2, -2 < x < 2, x, x > 2, 2);
fplot(y)
```



### Assumptions and Piecewise Expressions

On creation, a piecewise expression applies existing assumptions. Apply assumptions set after creating the piecewise expression by using `simplify` on the expression.

Assume  $x > 0$ . Then define a piecewise expression with the same condition  $x > 0$ . `piecewise` automatically applies the assumption to simplify the condition.

```
syms x
assume(x > 0)
pw = piecewise(x < 0, -1, x > 0, 1)
```

```
pw = 1
```

Clear the assumption on  $x$  for further computations.

```
assume(x, "clear")
```

Create a piecewise expression `pw` with the condition  $x > 0$ . Then set the assumption that  $x > 0$ . Apply the assumption to `pw` by using `simplify`.

```
pw = piecewise(x < 0, -1, x > 0, 1);
assume(x > 0)
pw = simplify(pw)

pw = 1
```

Clear the assumption on x for further computations.

```
assume(x, "clear")
```

### Differentiate, Integrate, and Find Limits of Piecewise Expression

Differentiate, integrate, and find limits of a piecewise expression by using `diff`, `int`, and `limit` respectively.

Differentiate the following piecewise expression by using `diff`.

$$y = \begin{cases} 1/x & x < -1 \\ \sin(x)/x & x \geq -1 \end{cases}$$

```
syms x
y = piecewise(x < -1, 1/x, x >= -1, sin(x)/x);
diffy = diff(y, x)
```

$$\text{diffy} = \begin{cases} -\frac{1}{x^2} & \text{if } x < -1 \\ \frac{\cos(x)}{x} - \frac{\sin(x)}{x^2} & \text{if } -1 < x \end{cases}$$

Integrate y by using `int`.

```
inty = int(y, x)

inty = \begin{cases} \log(x) & \text{if } x < -1 \\ \sinint(x) & \text{if } -1 \leq x \end{cases}
```

Find the limits of y at 0 by using `limit`.

```
limit(y, x, 0)
```

```
ans = 1
```

Find the right- and left-sided limits of y at -1. For details, see `limit`.

```
limit(y, x, -1, "right")
```

```
ans = sin(1)
```

```
limit(y, x, -1, "left")
```

```
ans = -1
```

### Elementary Operations on Piecewise Expressions

Add, subtract, divide, and multiply two piecewise expressions. The resulting piecewise expression is only defined where the initial piecewise expressions are defined.

```
syms x
pw1 = piecewise(x < -1, -1, x >= -1, 1);
pw2 = piecewise(x < 0, -2, x >= 0, 2);
add = pw1+pw2

add =
```

$$\begin{cases} -3 & \text{if } x < -1 \\ -1 & \text{if } x \in [-1, 0) \\ 3 & \text{if } 0 \leq x \end{cases}$$

```
sub = pw1-pw2
```

```
sub =
```

$$\begin{cases} 1 & \text{if } x < -1 \\ 3 & \text{if } x \in [-1, 0) \\ -1 & \text{if } 0 \leq x \end{cases}$$

```
mul = pw1*pw2
```

```
mul =
```

$$\begin{cases} 2 & \text{if } x < -1 \\ -2 & \text{if } x \in [-1, 0) \\ 2 & \text{if } 0 \leq x \end{cases}$$

```
div = pw1/pw2
```

```
div =
```

$$\begin{cases} \frac{1}{2} & \text{if } x < -1 \\ -\frac{1}{2} & \text{if } x \in [-1, 0) \\ \frac{1}{2} & \text{if } 0 \leq x \end{cases}$$

### Extract Expressions and Conditions from Piecewise

Extract conditions and values from a piecewise expression by using the `children` function.

```
syms x
pw = piecewise(x < -2, sin(x), (-2 < x) & (x < 2), x, 2 < x, exp(-x))
```

```
pw =
```

$$\begin{cases} \sin(x) & \text{if } x < -2 \\ x & \text{if } x \in (-2, 2) \\ e^{-x} & \text{if } 2 < x \end{cases}$$

```
c = children(pw);
pwExpr = [c{:}, 1]
```

```
pwExpr = (sin(x) x e-x)
```



```
pwCond = [c{:,2}]
pwCond = (x < -2 -2 < x ∧ x < 2 2 < x)
```

## Modify or Extend Piecewise Expression

Modify a piecewise expression by replacing part of the expression using `subs`. Extend a piecewise expression by specifying the expression as the otherwise value of a new piecewise expression. This action combines the two piecewise expressions. `piecewise` does not check for overlapping or conflicting conditions. Instead, like an if-else ladder, `piecewise` returns the value for the first true condition.

Change the condition  $x < 2$  in a piecewise expression to  $x < 0$  by using `subs`.

```
syms x
pw = piecewise(x < 2, -1, x > 0, 1);
pw = subs(pw, x < 2, x < 0)

pw =

$$\begin{cases} -1 & \text{if } x < 0 \\ 1 & \text{if } 0 < x \end{cases}$$

```

Add the condition  $x > 5$  with the value  $1/x$  to `pw` by creating a new piecewise expression with `pw` as the otherwise value.

```
pw = piecewise(x > 5, 1/x, pw)

pw =

$$\begin{cases} \frac{1}{x} & \text{if } 5 < x \\ -1 & \text{if } x < 0 \\ 1 & \text{if } 0 < x \end{cases}$$

```

## Solve System of Equations with Piecewise Expressions

Define a system of equations that includes piecewise expressions. The first equation defines the variable  $y$  for the two conditions  $0 < x < 2$  and  $2 < x < 4$ . The second equation defines  $x \cdot y$  for the same two conditions.

```
syms x y
eq1 = y == piecewise((0 < x) & (x < 2), x^2 + x - 2, (2 < x) & (x < 4), 3/2*x)

eq1 =

$$\begin{cases} y = x^2 + x - 2 & \text{if } x \in (0, 2) \\ y = \frac{3x}{2} & \text{if } x \in (2, 4) \end{cases}$$


eq2 = y*x == piecewise((0 < x) & (x < 2), x - 1, (2 < x) & (x < 4), 9)

eq2 =

$$\begin{cases} xy = x - 1 & \text{if } x \in (0, 2) \\ xy = 9 & \text{if } x \in (2, 4) \end{cases}$$

```

To solve this system of equations, first substitute the variable  $y$  in the second equation with the expression for  $y$  from the first equation. You can use the `lhs` and `rhs` functions to get the left and right sides of the first equation.

```
eq2 = subs(eq2, lhs(eq1), rhs(eq1))
```

$$\text{eq2} = \begin{cases} x(x^2 + x - 2) = x - 1 & \text{if } x \in (0, 2) \\ \frac{3x^2}{2} = 9 & \text{if } x \in (2, 4) \end{cases}$$

Then, solve the second equation for the variable  $x$ .

```
sols = solve(eq2, x)
```

$$\text{sols} = \begin{pmatrix} 1 \\ \sqrt{6} \\ \sqrt{2} - 1 \end{pmatrix}$$

Here,  $x = 1$  and  $x = \sqrt{2} - 1$  are solutions that satisfy the condition  $0 < x < 2$ , while  $x = \sqrt{6}$  is a solution that satisfies the condition  $2 < x < 4$ .

## Input Arguments

### **cond** — Condition

symbolic condition | symbolic variable

Condition, specified as a symbolic condition or variable. A symbolic variable represents an unknown condition.

Example:  $x > 2$

### **val** — Value when condition is satisfied

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Value when condition is satisfied, specified as a number, vector, matrix, or multidimensional array, or as a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **otherwiseVal** — Value if no conditions are true

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Value if no conditions are true, specified as a number, vector, matrix, or multidimensional array, or as a symbolic number, variable, vector, matrix, multidimensional array, function, or expression. If `otherwiseVal` is not specified, its value is `NaN`.

## Output Arguments

### **pw** — Piecewise expression or function

symbolic expression | symbolic function

Piecewise expression or function, returned as a symbolic expression or function. The value of `pw` is the value `val` of the first condition `cond` that is true. To find the value of `pw`, use `subs` to substitute for variables in `pw`.

## Tips

- `piecewise` does not check for overlapping or conflicting conditions. A piecewise expression returns the value of the first true condition and disregards any following overlapping true expressions. Thus, `piecewise` follows the behavior of an if-else statement.
- You can use the `lhs` and `rhs` functions to get the left and right sides of a symbolic equation that includes a piecewise expression.

## Version History

Introduced in R2016b

## See Also

`and` | `assume` | `assumeAlso` | `assumptions` | `if` | `in` | `isAlways` | `not` | `or`

## pinv

Moore-Penrose inverse (pseudoinverse) of symbolic matrix

### Syntax

```
X = pinv(A)
```

### Description

`X = pinv(A)` returns the pseudoinverse of `A`. Pseudoinverse is also called the Moore-Penrose inverse.

### Examples

#### Compute Pseudoinverse of Matrix

Compute the pseudoinverse of this matrix. Because these numbers are not symbolic objects, you get floating-point results.

```
A = [1 1i 3; 1 3 2];  
X = pinv(A)
```

```
X =  
    0.0729 + 0.0312i    0.0417 - 0.0312i  
   -0.2187 - 0.0521i    0.3125 + 0.0729i  
    0.2917 + 0.0625i    0.0104 - 0.0938i
```

Now, convert this matrix to a symbolic object, and compute the pseudoinverse.

```
A = sym([1 1i 3; 1 3 2]);  
X = pinv(A)
```

```
X =  
[ 7/96 + 1i/32, 1/24 - 1i/32]  
[ - 7/32 - 5i/96, 5/16 + 7i/96]  
[ 7/24 + 1i/16, 1/96 - 3i/32]
```

Check that  $A*X*A = A$  and  $X*A*X = X$ .

```
isAlways(A*X*A == A)
```

```
ans =  
2×3 logical array  
    1    1    1  
    1    1    1
```

```
isAlways(X*A*X == X)
```

```
ans =  
3×2 logical array  
    1    1  
    1    1  
    1    1
```

Now, verify that  $A^*X$  and  $X^*A$  are Hermitian matrices.

```
isAlways(A*X == (A*X)')
```

```
ans =
    2x2 logical array
     1     1
     1     1
```

```
isAlways(X*A == (X*A)')
```

```
ans =
    3x3 logical array
     1     1     1
     1     1     1
     1     1     1
```

## Compute Pseudoinverse of Matrix

Compute the pseudoinverse of this matrix.

```
syms a
A = [1 a; -a 1];
X = pinv(A)

X =
[ (a*conj(a) + 1)/(a^2*conj(a)^2 + a^2 + conj(a)^2 + 1) -...
 (conj(a)*(a - conj(a)))/(a^2*conj(a)^2 + a^2 + conj(a)^2 + 1),
 - (a - conj(a))/(a^2*conj(a)^2 + a^2 + conj(a)^2 + 1) -...
 (conj(a)*(a*conj(a) + 1))/(a^2*conj(a)^2 + a^2 + conj(a)^2 + 1)]
[ (a - conj(a))/(a^2*conj(a)^2 + a^2 + conj(a)^2 + 1) +...
 (conj(a)*(a*conj(a) + 1))/(a^2*conj(a)^2 + a^2 + conj(a)^2 + 1),
 (a*conj(a) + 1)/(a^2*conj(a)^2 + a^2 + conj(a)^2 + 1) -...
 (conj(a)*(a - conj(a)))/(a^2*conj(a)^2 + a^2 + conj(a)^2 + 1)]
```

Now, compute the pseudoinverse of  $A$  assuming that  $a$  is real.

```
assume(a, 'real')
A = [1 a; -a 1];
X = pinv(A)

X =
[ 1/(a^2 + 1), -a/(a^2 + 1)]
[ a/(a^2 + 1), 1/(a^2 + 1)]
```

For further computations, remove the assumption on  $a$  by recreating it using `syms`.

```
syms a
```

## Input Arguments

### A — Input

symbolic matrix

Input, specified as a symbolic matrix.

## Output Arguments

### **X — Pseudoinverse of matrix**

symbolic matrix

Pseudoinverse of matrix, returned as a symbolic matrix, such that  $A*X*A = A$  and  $X*A*X = X$ .

## More About

### **Moore-Penrose Pseudoinverse**

The pseudoinverse of an  $m$ -by- $n$  matrix  $A$  is an  $n$ -by- $m$  matrix  $X$ , such that  $A*X*A = A$  and  $X*A*X = X$ . The matrices  $A*X$  and  $X*A$  must be Hermitian.

## Tips

- Calling `pinv` for numeric arguments that are not symbolic objects invokes the MATLAB `pinv` function.
- For an invertible matrix  $A$ , the Moore-Penrose inverse  $X$  of  $A$  coincides with the inverse of  $A$ .

## Version History

Introduced in R2013a

## See Also

`inv` | `rank` | `pinv` | `svd`

# playAnimation

Play animation objects in a MATLAB figure window

## Syntax

```
playAnimation
playAnimation(fig)
playAnimation( ___,Name,Value)
```

## Description

`playAnimation` plays animation objects in a MATLAB figure window. The animation objects must be created using the `fanimator` function.

By default, the variable `t = sym('t')` is the time parameter of the animation objects. `playAnimation` plays the animation with 10 frames per unit interval of `t` within the range of `t` from 0 to 10.

`playAnimation(fig)` plays animation objects in the figure `fig`.

`playAnimation( ___,Name,Value)` plays the animation objects with the specified `Name,Value` pair arguments. Use this option with any of the input argument combinations in the previous syntaxes.

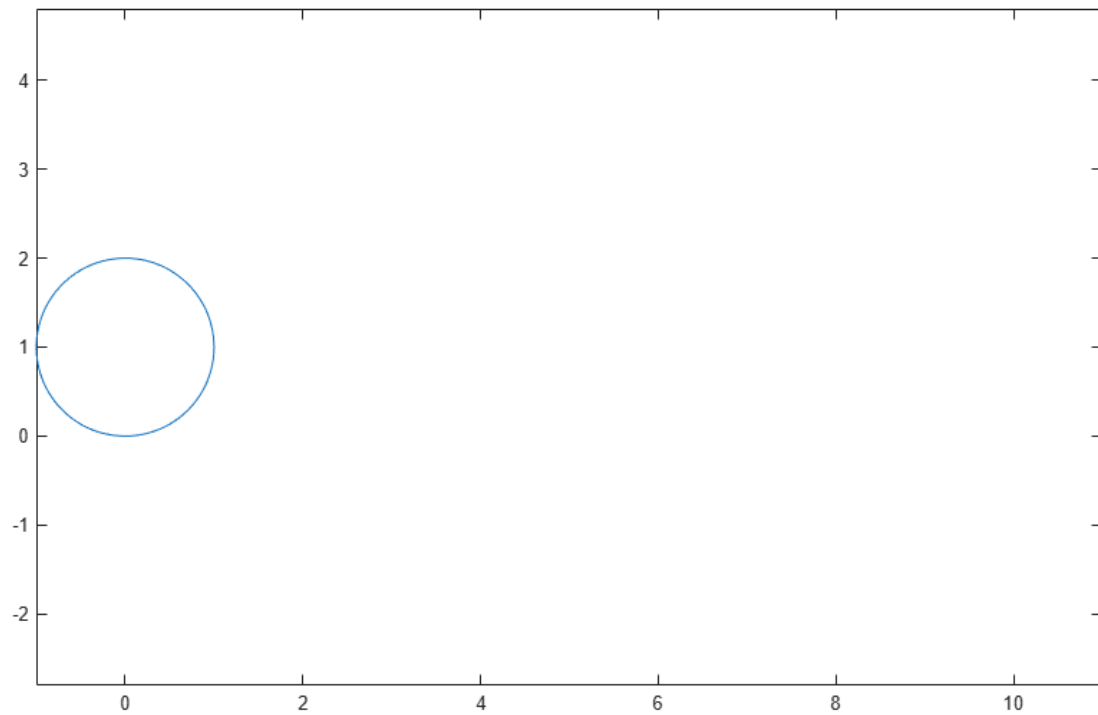
## Examples

### Animate Moving Circle

First, create an animation object of a moving circle using `fanimator`.

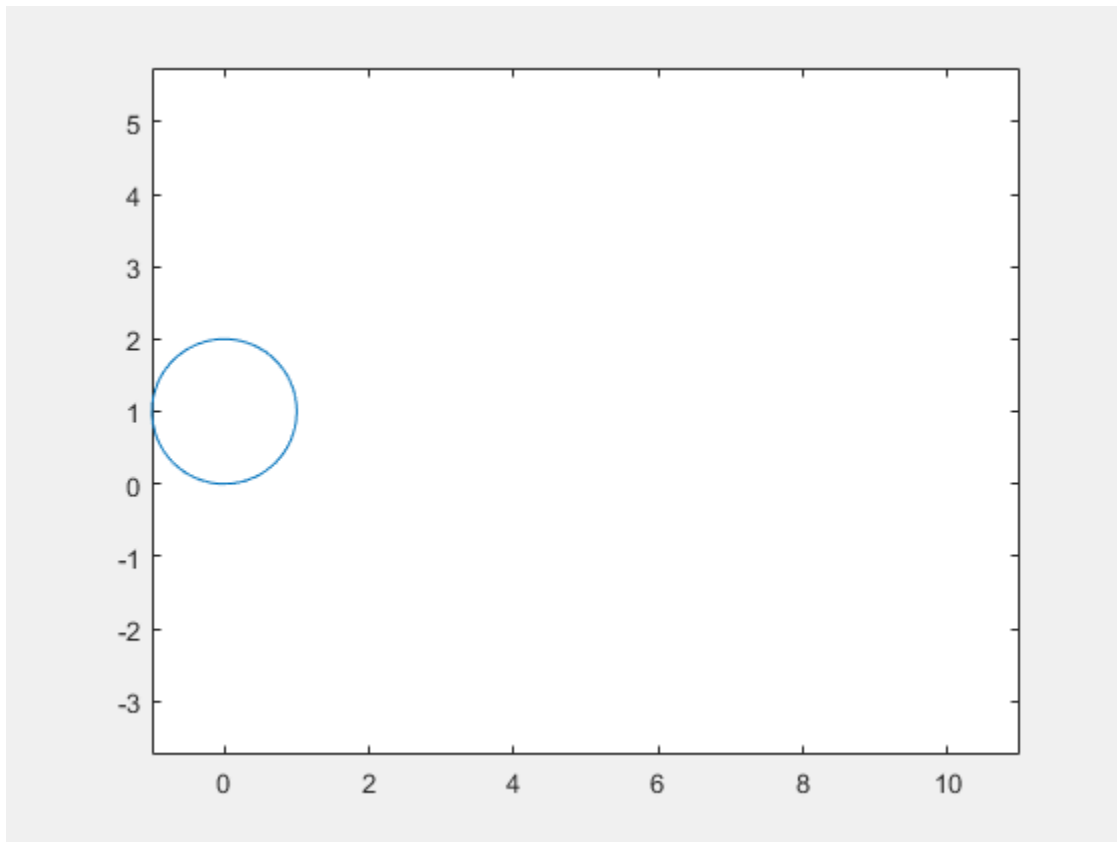
Create two symbolic variables, `t` and `x`. The variable `t` defines the time parameter of the animation. Use `t` to set the center of the circle at  $(t, 1)$  and `x` to parameterize the perimeter of the circle within the range  $[-\pi, \pi]$ . Set the x-axis and y-axis to be equal length.

```
syms t x
fanimator(@fplot,cos(x)+t,sin(x)+1,[-pi pi])
axis equal
```



Next, enter the command `playAnimation` to play the animation.





By default, `playAnimation` plays an animation with 10 generated frames per unit time within the range of  $t$  from 0 to 10.

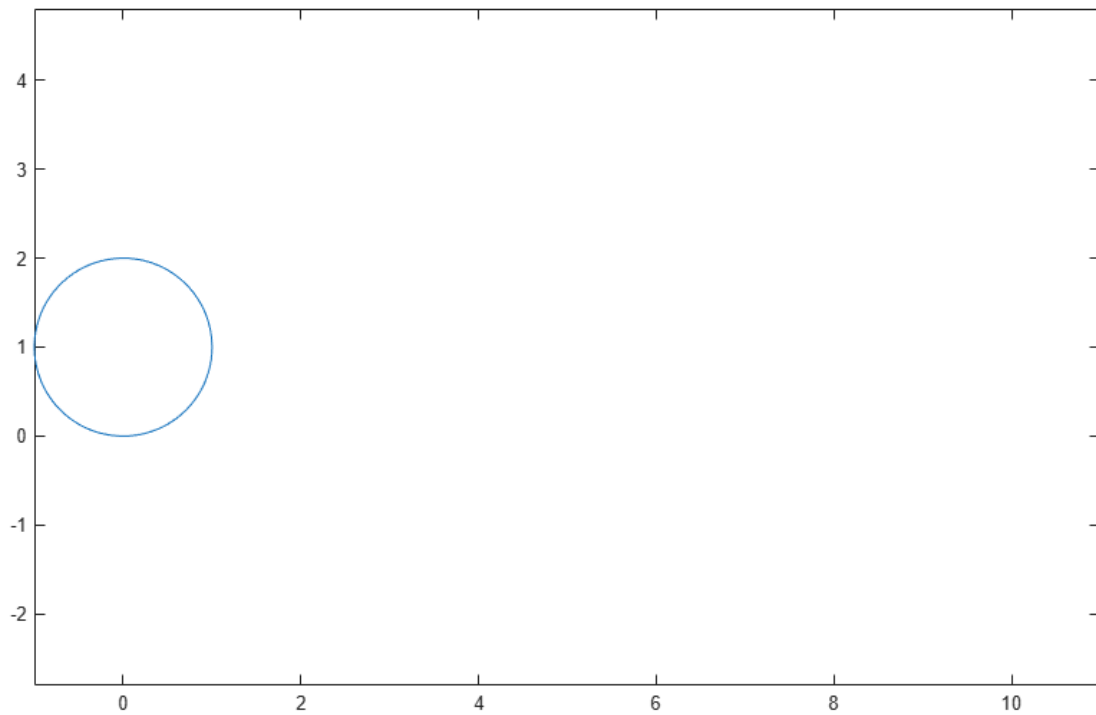
### Animate Moving Circle with Timer

Create two symbolic variables,  $t$  and  $x$ . The variable  $t$  defines the time parameter of the animation.

```
syms t x
```

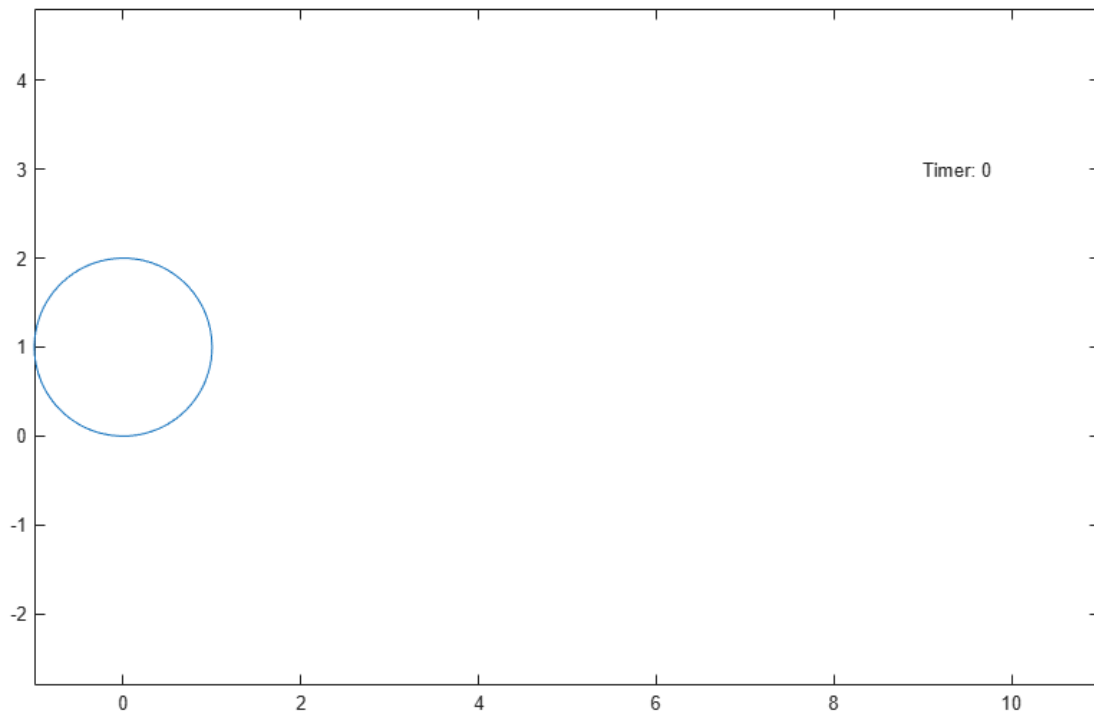
Create a circle animation object using `f animator`. Use  $t$  to set the center of the circle at  $(t, 1)$  and  $x$  to parameterize the perimeter of the circle within the range  $[-\pi, \pi]$ . Set the x-axis and y-axis to be equal length.

```
f animator(@fplot,cos(x)+t,sin(x)+1,[-pi pi])
axis equal
```



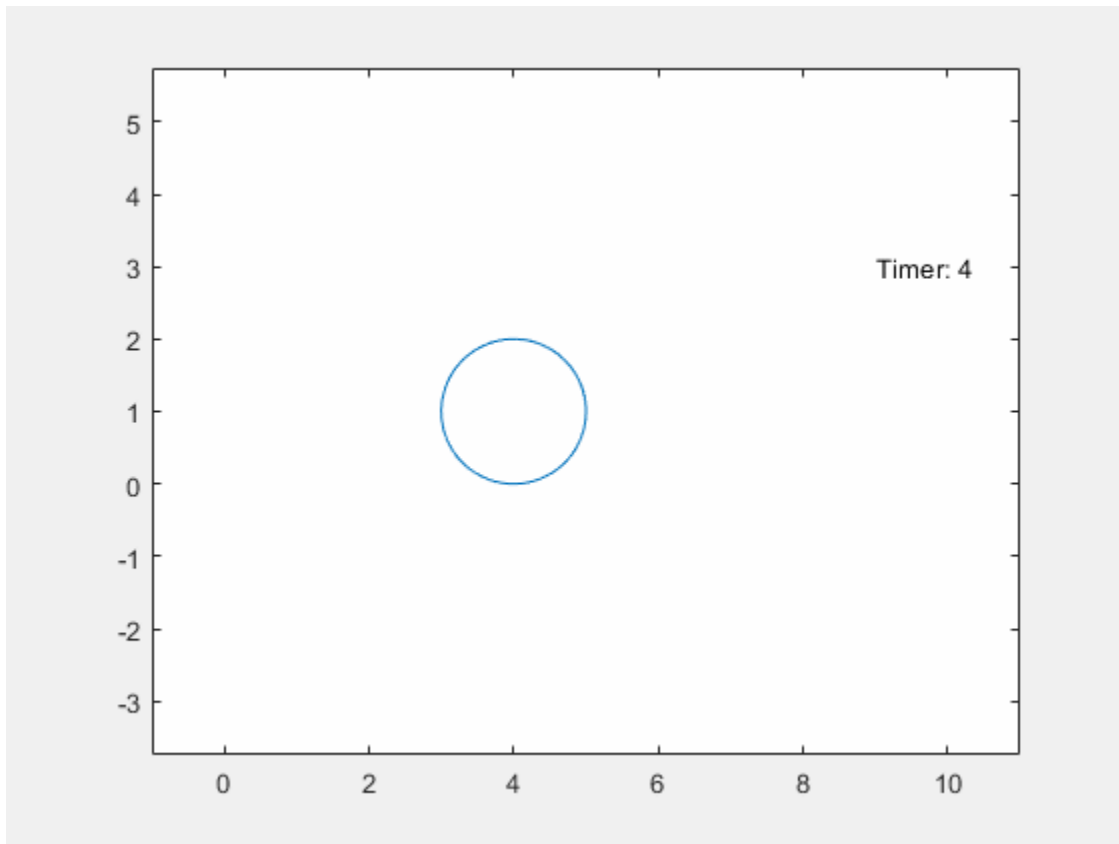
Add a piece of text to count the elapsed time by using the `text` function. Use `num2str` to convert the time parameter to a string.

```
hold on
fanimation(@(t) text(9,3,"Timer: "+num2str(t,2)))
hold off
```



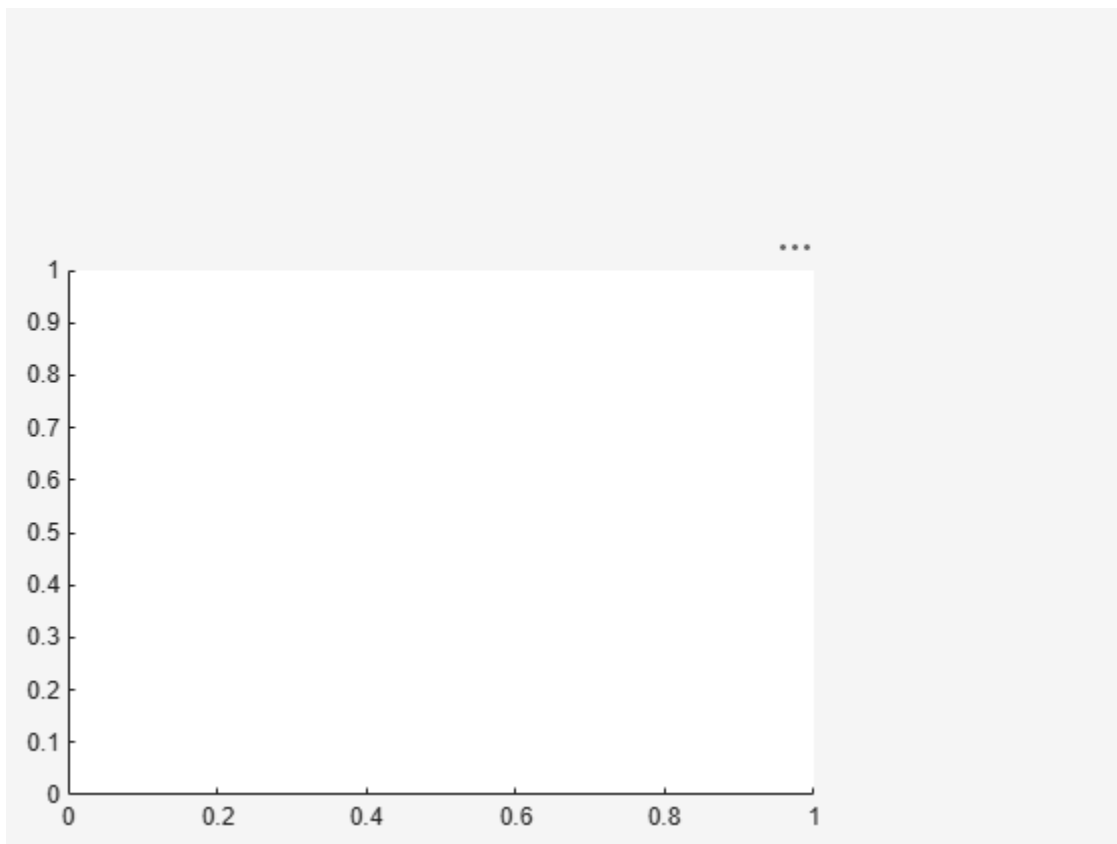
By default, `playAnimation` plays the animation with 10 generated frames per unit time within the range of `t` from 0 to 10. Change the range of the time parameter to `[4 8]` using the `'AnimationRange'` property. Change the frame rate per unit time to 4 using the `'FrameRate'` property. Play the animation in the current figure by entering the following command.

```
playAnimation(gcf,'AnimationRange',[4 8],'FrameRate',4)
```

**Create Animation in UI Figure**

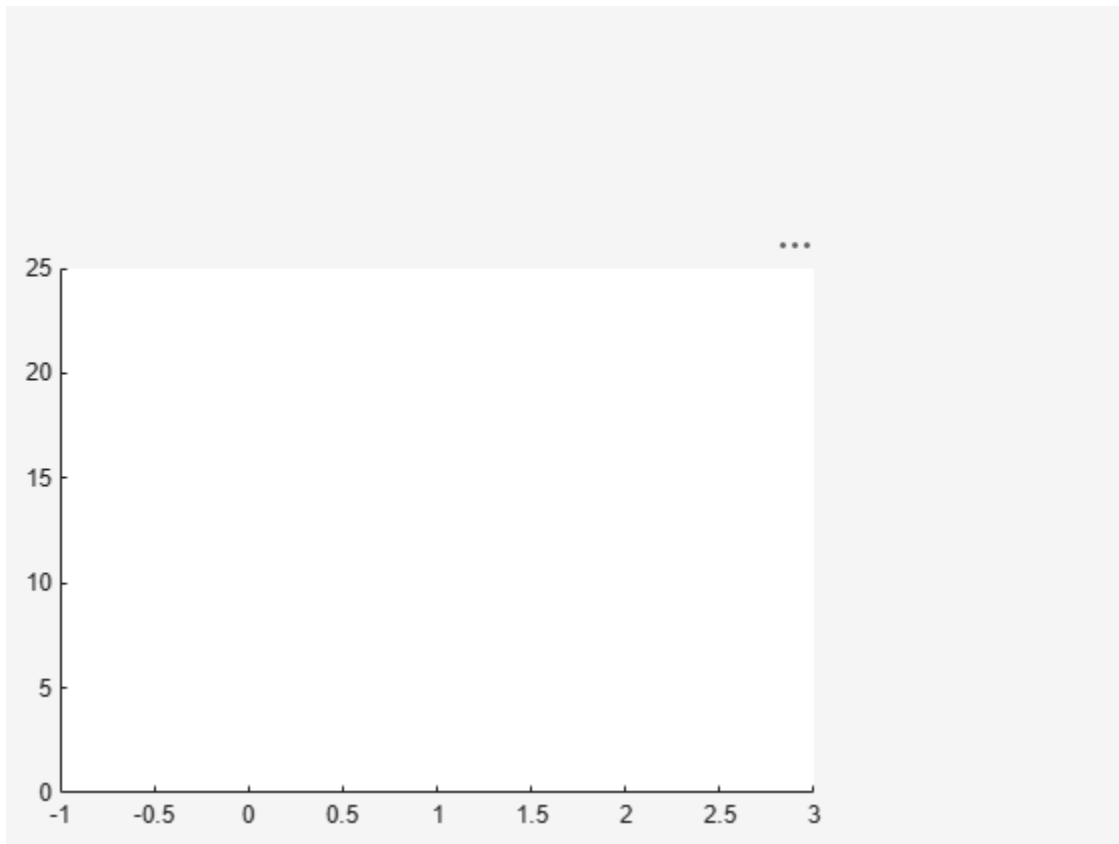
Create a UI figure. Specify the UI axes of the figure.

```
fig = uifigure;  
ax = uiaxes(fig);
```

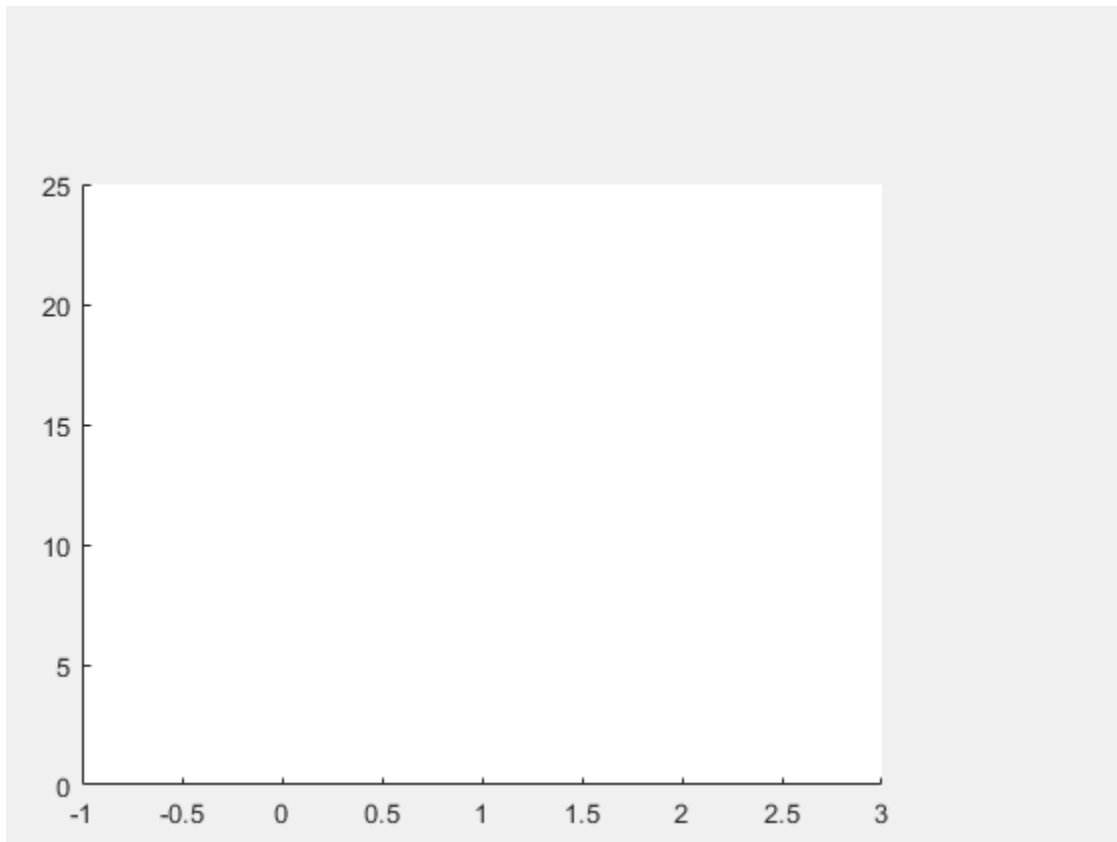


Add an animation object to the UI axes using `fanimate`. Create two symbolic variables, `x` and `t`. Plot a curve that grows exponentially as a function of time `t` within the interval `[0 3]`.

```
syms x t;  
fanimate(ax,@fplot,exp(x),[0 t], 'r', 'AnimationRange',[0 3])
```



Play the animation in the UI figure `fig` by entering the command `playAnimation(fig)`. Alternatively, you can also use the command `playAnimation(ax.Parent)`.



## Input Arguments

### **fig** — Target figure

Figure object

Target figure, specified as a Figure object. For more information about Figure objects, see [figure](#).

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `'Backwards',true,'FrameRate',25`

### **AnimationRange** — Range of animation time parameter

`[0 10]` (default) | two-element row vector

Range of the animation time parameter, specified as a two-element row vector. The two elements must be real values that are increasing.

Example: `[-2 4.5]`

**FrameRate — Frame rate**

10 (default) | positive value

Frame rate, specified as a positive value. The frame rate defines the number of frames per unit time when playing the animation objects.

Example: 30

**Backwards — Backward option**

logical 0 (false) (default) | logical value (boolean)

Backward option, specified as a logical value (boolean). If you specify the option `true`, then the function plays the animation backward.

Example: `true`

**SpeedFactor — Speed factor**

1 (default) | real nonzero value

Speed factor, specified as a real nonzero value. The speed factor sets the ratio of one unit interval of the animation time parameter to one second of clock time.

- If you specify a negative value for 'SpeedFactor' and keep the default value 0 (false) for 'Backwards' option, then the function plays the animation backward with the specified speed factor. For example, `playAnimation('SpeedFactor', -1)` launches the same animation as `playAnimation('Backwards', true)`.
- If you specify a zero value for 'SpeedFactor', then `playAnimation('SpeedFactor', 0)` launches a still frame indefinitely and does not play any animation.

Example: 2

**Tips**

- When you create a graph by using a plotting function, such as `fplot`, MATLAB creates a series of graphics objects. You can then animate a specific property of the graphics objects by using the `fanimator` and the `playAnimation` functions. Note that some functions, such as `title` and `xlabel`, create text objects that cannot be animated. Instead, use the `text` function to create text objects that can be animated.

**Version History**

Introduced in R2019a

**See Also**

`animationToFrame` | `fanimator` | `rewindAnimation` | `writeAnimation`



# plus, +

Symbolic addition

## Syntax

```
A + B
plus(A,B)
```

## Description

$A + B$  adds A and B.

`plus(A,B)` is equivalent to  $A + B$ .

## Examples

### Add Scalar to Array

`plus` adds  $x$  to each element of the array.

```
syms x
A = [x sin(x) 3];
A + x

ans =

[ 2*x, x + sin(x), x + 3]
```

### Add Two Matrices

Add the identity matrix to matrix M.

```
syms x
M = [x x^2; Inf 0];
M + eye(2)

ans =

[ x + 1, x^2]
[  Inf,  1]
```

Alternatively, use `plus(M,eye(2))`.

```
plus(M,eye(2))

ans =

[ x + 1, x^2]
[  Inf,  1]
```

### Add Symbolic Functions

```
syms f(x) g(x)
f(x) = x^2 + 5*x + 6;
```

```
g(x) = 3*x - 2;  
h = f + g
```

```
h(x) =  
x^2 + 8*x + 4
```

### Add Expression to Symbolic Function

Add expression `expr` to function `f`.

```
syms f(x)  
f(x) = x^2 + 3*x + 2;  
expr = x^2 - 2;  
f(x) = f(x) + expr  
  
f(x) =  
2*x^2 + 3*x
```

## Input Arguments

### A — Input

symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix | symbolic array

Input, specified as a symbolic scalar variable, matrix variable, function, matrix function, expression, or vector, matrix, or array of symbolic scalar variables.

### B — Input

symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix | symbolic array

Input, specified as a symbolic scalar variable, matrix variable, function, matrix function, expression, or vector, matrix, or array of symbolic scalar variables.

## Tips

- All nonscalar arguments must be the same size. If one input argument is nonscalar, then `plus` expands the scalar into an array of the same size as the nonscalar argument, with all elements equal to the scalar.

## Version History

Introduced before R2006a

### R2022a: Compute addition of symbolic matrix functions

The `plus` function accepts an input argument of type `symfunmatrix`.

### R2021a: Compute addition of symbolic matrix variables

The `plus` function accepts an input argument of type `symmatrix`.

## See Also

`ctranspose` | `ldivide` | `minus` | `mldivide` | `mpower` | `mrdivide` | `mtimes` | `power` | `rdivide` | `times` | `transpose`

## pochhammer

Pochhammer symbol

### Syntax

`pochhammer(x,n)`

### Description

`pochhammer(x,n)` returns the “Pochhammer Symbol” on page 6-1198  $(x)_n$ .

### Examples

#### Find Pochhammer Symbol for Numeric and Symbolic Inputs

Find the Pochhammer symbol for the numeric inputs  $x = 3$  at  $n = 2$ .

```
pochhammer(3,2)
```

```
ans =  
    12
```

Find the Pochhammer symbol for the symbolic input  $x$  at  $n = 3$ . The `pochhammer` function does not automatically return the expanded form of the expression. Use `expand` to force `pochhammer` to return the form of the expanded expression.

```
syms x  
P = pochhammer(x, 3)  
P = expand(P)
```

```
P =  
pochhammer(x, 3)  
P =  
x^3 + 3*x^2 + 2*x
```

#### Rewrite and Factor Outputs of Pochhammer

If conditions are satisfied, `expand` rewrites the solution using `gamma`.

```
syms n x  
assume(x>0)  
assume(n>0)  
P = pochhammer(x, n);  
P = expand(P)
```

```
P =  
gamma(n + x)/gamma(x)
```

To use the variables in further computations, clear their assumptions by recreating them using `syms`.

```
syms n x
```

To convert expanded output of `pochhammer` into its factors, use `factor`.

```
P = expand(pochhammer(x, 4));
P = factor(P)
```

```
P =
[ x, x + 3, x + 2, x + 1]
```

### Differentiate Pochhammer Symbol

Differentiate `pochhammer` once with respect to `x`.

```
syms n x
diff(pochhammer(x,n),x)
```

```
ans =
pochhammer(x, n)*(psi(n + x) - psi(x))
```

Differentiate `pochhammer` twice with respect to `n`.

```
diff(pochhammer(x,n),n,2)
```

```
ans =
pochhammer(x, n)*psi(n + x)^2 + pochhammer(x, n)*psi(1, n + x)
```

### Taylor Series Expansion of Pochhammer Symbol

Use `taylor` to find the Taylor series expansion of `pochhammer` with `n = 3` around the expansion point `x = 2`.

```
syms x
taylor(pochhammer(x,3),x,2)
```

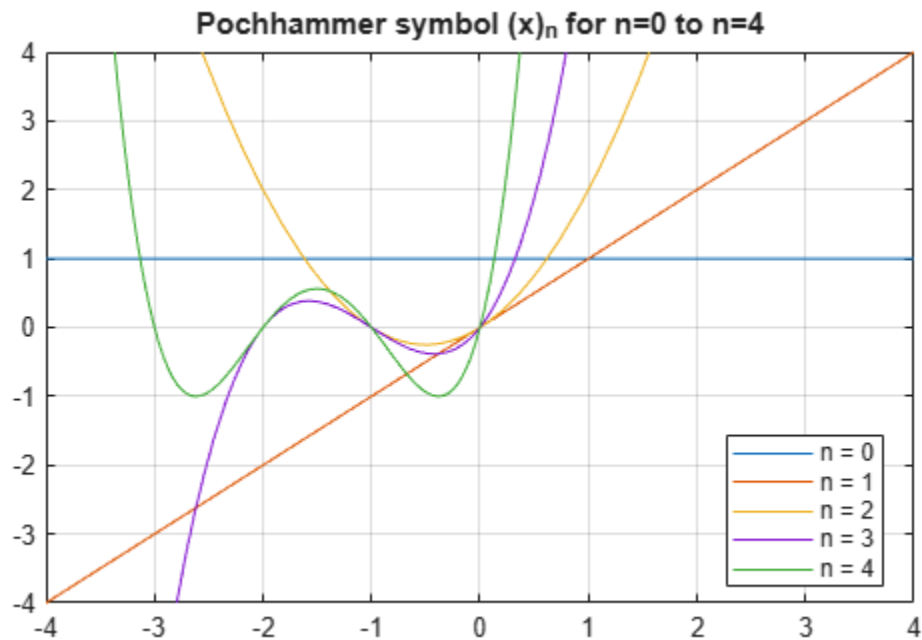
```
ans =
26*x + 9*(x - 2)^2 + (x - 2)^3 - 28
```

### Plot Pochhammer Symbol

Plot the Pochhammer symbol from `n = 0` to `n = 4` for `x`. Use `axis` to display the region of interest.

```
syms x
fplot(pochhammer(x,0:4))
axis([-4 4 -4 4])
```

```
grid on
legend('n = 0', 'n = 1', 'n = 2', 'n = 3', 'n = 4', 'Location', 'Best')
title('Pochhammer symbol (x)_n for n=0 to n=4')
```



## Input Arguments

### **x — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **n — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

## More About

### **Pochhammer Symbol**

Pochhammer's symbol is defined as

$$(x)_n = \frac{\Gamma(x+n)}{\Gamma(x)},$$

where  $\Gamma$  is the Gamma function.

If  $n$  is a positive integer, Pochhammer's symbol is

$$(x)_n = x(x+1)\dots(x+n-1)$$

## Algorithms

- If  $x$  and  $n$  are numerical values, then an explicit numerical result is returned. Otherwise, a symbolic function call is returned.
- If both  $x$  and  $x + n$  are nonpositive integers, then

$$(x)_n = (-1)^n \frac{\Gamma(1-x)}{\Gamma(1-x-n)}.$$

- The following special cases are implemented.

$$(x)_0 = 1$$

$$(x)_1 = x$$

$$(x)_{-1} = \frac{1}{x-1}$$

$$(1)_n = \Gamma(n+1)$$

$$(2)_n = \Gamma(n+2)$$

- If  $n$  is a positive integer, then `expand(pochhammer(x,n))` returns the expanded polynomial  $x(x+1)\dots(x+n-1)$ .
- If  $n$  is not an integer, then `expand(pochhammer(x,n))` returns a representation in terms of `gamma`.

## Version History

Introduced in R2014b

### See Also

`factorial` | `gamma`

## poles

Poles of expression or function

### Syntax

```
P = poles(f,var)
P = poles(f,var,a,b)
[P,N] = poles(____)
[P,N,R] = poles(____)
```

### Description

`P = poles(f,var)` finds the poles of `f` with respect to variable `var`.

`P = poles(f,var,a,b)` returns poles in the interval  $(a,b)$ .

`[P,N] = poles(____)` returns the poles of `f` and their orders in `N`.

`[P,N,R] = poles(____)` returns the poles of `f`, their orders, and residues in `R`.

### Examples

#### Find Poles of Symbolic Expressions

```
syms x
poles(1/(x-1i))
```

```
ans =
1i
```

```
poles(sin(x)/(x-1))
```

```
ans =
1
```

#### Specify Independent Variable

Find the poles of this expression. If you do not specify a variable, `poles` uses the default variable determined by `symvar`.

```
syms x a
f = 1/((x-1)*(a-2));
poles(f)
```

```
ans =
1
```

Find the poles with respect to `a` by specifying the second argument.



```
syms x a
poles(f,a)
```

```
ans =
2
```

### Find Poles in Interval

Find the poles of the tangent function in the interval  $(-\pi, \pi)$ .

```
syms x
poles(tan(x), x, -pi, pi)
```

```
ans =
-pi/2
pi/2
```

The tangent function has an infinite number of poles. If you do not specify the interval, `poles` cannot find all of them. It issues a warning and returns an empty symbolic object.

```
syms x
poles(tan(x))
```

```
Warning: Unable to determine poles.
ans =
Empty sym: 0-by-1
```

If `poles` can prove that the input does not have poles in the interval, it returns empty without issuing a warning.

```
syms x
poles(tan(x), x, -1, 1)
```

```
ans =
Empty sym: 0-by-1
```

### Return Order of Poles

Return orders along with poles by using two output arguments. Restrict the search interval to  $(-\pi, \pi)$ .

```
syms x
[Poles, Orders] = poles(tan(x)/(x-1)^3, x, -pi, pi)
```

```
Poles =
-pi/2
pi/2
1
```

```
Orders =
1
1
3
```

### Return Order and Residue of Poles

Return the residues and orders along with the poles by specifying three output arguments.

```
syms x a
[Poles, Orders, Residues] = poles(a/(x^2*(x-1)), x)

Poles =
     1
     0
Orders =
     1
     2
Residues =
     a
    -a
```

## Input Arguments

### **f** — Input

symbolic expression | symbolic function.

Input, specified as a symbolic expression or function.

### **var** — Independent variable

symbolic variable

Independent variable, specified as a symbolic variable.

### **a, b** — Search interval for poles

vector of two real numbers | vector of two real symbolic numbers

Search interval for poles, specified as a vector of two real numeric or symbolic numbers (including infinities).

## Tips

- If `poles` cannot find all nonremovable singularities and cannot prove that they do not exist, it issues a warning and returns an empty symbolic object.
- If `poles` can prove that the input does not have poles (in the specified interval or complex plane), it returns empty without issuing a warning.
- `a` and `b` must be real numbers or infinities. If you provide complex numbers, `poles` uses an empty interval and returns an empty symbolic object.

## Version History

Introduced in R2012b

## See Also

`limit` | `solve` | `symvar` | `vpasolve`

# poly2sym

Create symbolic polynomial from vector of coefficients

## Syntax

```
p = poly2sym(c)
p = poly2sym(c,var)
```

## Description

`p = poly2sym(c)` creates the symbolic polynomial expression `p` from the vector of coefficients `c`. The polynomial variable is `x`. If `c = [c1,c2,...,cn]`, then `p = poly2sym(c)` returns  $c_1x^{n-1} + c_2x^{n-2} + \dots + c_n$ .

This syntax does not create the symbolic variable `x` in the MATLAB Workspace.

`p = poly2sym(c,var)` uses `var` as a polynomial variable when creating the symbolic polynomial expression `p` from the vector of coefficients `c`.

## Examples

### Create Polynomial Expression

Create a polynomial expression from a symbolic vector of coefficients. If you do not specify a polynomial variable, `poly2sym` uses `x`.

```
syms a b c d
p = poly2sym([a,b,c,d])
```

$$p = ax^3 + bx^2 + cx + d$$

Create a polynomial expression from a symbolic vector of rational coefficients.

```
p = poly2sym(sym([1/2,-1/3,1/4]))
```

$$p = \frac{x^2}{2} - \frac{x}{3} + \frac{1}{4}$$

Create a polynomial expression from a numeric vector of floating-point coefficients. The toolbox converts floating-point coefficients to rational numbers before creating a polynomial expression.

```
p = poly2sym([0.75,-0.5,0.25])
```

$$p = \frac{3x^2}{4} - \frac{x}{2} + \frac{1}{4}$$

### Specify Polynomial Variable

Create a polynomial expression from a symbolic vector of coefficients. Use  $t$  as a polynomial variable.

```
syms a b c d t
p = poly2sym([a,b,c,d],t)
```

$$p = at^3 + bt^2 + ct + d$$

To use a symbolic expression, such as  $t^2 + 1$  or  $\exp(t)$ , instead of a polynomial variable, substitute the variable using `subs`.

```
p1 = subs(p,t,t^2 + 1)
```

$$p1 = d + a(t^2 + 1)^3 + b(t^2 + 1)^2 + c(t^2 + 1)$$

```
p2 = subs(p,t,exp(t))
```

$$p2 = d + c e^t + a e^{3t} + b e^{2t}$$

### Find Roots of Polynomial and Check Solution

Create a polynomial expression from a numeric vector of integer coefficients.

```
p_coeffs = [1 4 5 4 4];
p = poly2sym(p_coeffs)
```

$$p = x^4 + 4x^3 + 5x^2 + 4x + 4$$

Because `poly2sym` does not create the symbolic variable  $x$  in the workspace, create this variable by using `syms`. Find the roots of the polynomial by using `solve`.

```
syms x
p_roots = solve(p,x)
```

$$p\_roots = \begin{pmatrix} -2 \\ -2 \\ -i \\ i \end{pmatrix}$$

The polynomial has 4 roots. To check if these roots are indeed the correct solution, you can reconstruct the original polynomial from the roots.

Find the factored form of the polynomial by subtracting each root from  $x$ .

```
p_elem = x-p_roots
```

$$p\_elem = \begin{pmatrix} x + 2 \\ x + 2 \\ x + i \\ x - i \end{pmatrix}$$

Take the product of the factored form of the polynomial.

```
p_new = prod(p_elem)
```

```
p_new = (x + 2)^2 (x - i) (x + i)
```

Expand the polynomial and confirm that the result is the same as the original expression.

```
p_new = expand(p_new)
```

```
p_new = x^4 + 4 x^3 + 5 x^2 + 4 x + 4
```

## Input Arguments

### **c — Polynomial coefficients**

numeric vector | symbolic vector

Polynomial coefficients, specified as a numeric or symbolic vector. Argument *c* can be a column or row vector.

### **var — Polynomial variable**

symbolic variable

Polynomial variable, specified as a symbolic variable.

## Output Arguments

### **p — Polynomial**

symbolic expression

Polynomial, returned as a symbolic expression.

## Tips

- When you call `poly2sym` for a numeric vector *c*, the toolbox converts the numeric vector to a vector of symbolic numbers using the default (rational) conversion mode of `sym`.

## Version History

**Introduced before R2006a**

## See Also

`coeffs` | `sym` | `sym2poly`

## polylog

Polylogarithm

### Syntax

```
Li = polylog(n,x)
```

### Description

`Li = polylog(n,x)` returns the polylogarithm of the order `n` and the argument `x`.

### Examples

#### Polylogarithms of Numeric and Symbolic Arguments

`polylog` returns floating-point numbers or exact symbolic results depending on the arguments you use.

Compute the polylogarithms of numeric input arguments. The `polylog` function returns floating-point numbers.

```
Li = [polylog(3,-1/2), polylog(4,1/3), polylog(5,3/4)]
```

```
Li =  
    -0.4726    0.3408    0.7697
```

Compute the polylogarithms of the same input arguments by converting them to symbolic objects. For most symbolic (exact) numbers, `polylog` returns unresolved symbolic calls.

```
symA = [polylog(3,sym(-1/2)), polylog(sym(4),1/3), polylog(5,sym(3/4))]
```

```
symA =  
[ polylog(3, -1/2), polylog(4, 1/3), polylog(5, 3/4)]
```

Approximate the symbolic results with the default number of 32 significant digits by using `vpa`.

```
Li = vpa(symA)
```

```
Li =  
[ -0.47259784465889687461862319312655,...  
  0.3407911308562507524776409440122,...  
  0.76973541059975738097269173152535]
```

The `polylog` function also accepts noninteger values of the order `n`. Compute `polylog` for complex arguments.

```
Li = polylog(-0.2i,2.5)
```

```
Li =  
    -2.5030 + 0.3958i
```

## Explicit Expressions for Polylogarithms

If the order of the polylogarithm is 0, 1, or a negative integer, then `polylog` returns an explicit expression.

The polylogarithm of  $n = 1$  is a logarithmic function.

```
syms x
Li = polylog(1,x)
```

```
Li =
-log(1 - x)
```

The polylogarithms of  $n < 1$  are rational expressions.

```
Li = polylog(0,x)
```

```
Li =
-x/(x - 1)
```

```
Li = polylog(-1,x)
```

```
Li =
x/(x - 1)^2
```

```
Li = polylog(-2,x)
```

```
Li =
-(x^2 + x)/(x - 1)^3
```

```
Li = polylog(-3,x)
```

```
Li =
(x^3 + 4*x^2 + x)/(x - 1)^4
```

```
Li = polylog(-10,x)
```

```
Li =
-(x^10 + 1013*x^9 + 47840*x^8 + 455192*x^7 + ...
1310354*x^6 + 1310354*x^5 + 455192*x^4 + ...
47840*x^3 + 1013*x^2 + x)/(x - 1)^11
```

## Special Values

The `polylog` function has special values for some parameters.

If the second argument is 0, then the polylogarithm is equal to 0 for any integer value of the first argument. If the second argument is 1, then the polylogarithm is the Riemann zeta function of the first argument.

```
syms n
Li = [polylog(n,0), polylog(n,1)]
```

```
Li =
[ 0, zeta(n)]
```

If the second argument is -1, then the polylogarithm has a special value for any integer value of the first argument except 1.

```
assume(n ~= 1)
Li = polylog(n, -1)

Li =
zeta(n)*(2^(1 - n) - 1)
```

To do other computations, clear the assumption on  $n$  by recreating it using `syms`.

```
syms n
```

Compute other special values of the polylogarithm function.

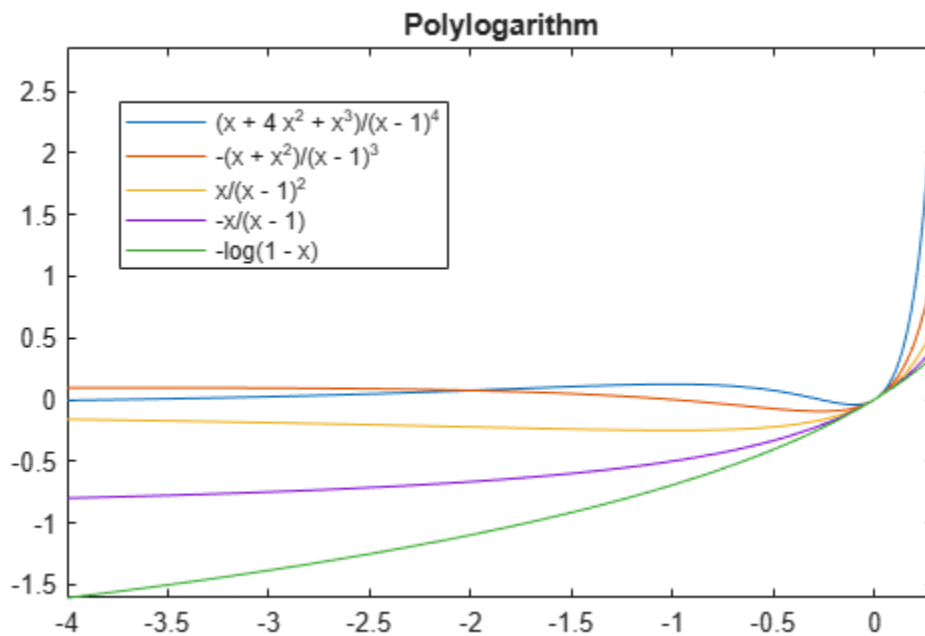
```
Li = [polylog(4, sym(1)), polylog(sym(5), -1), polylog(2, sym(i))]

Li =
[ pi^4/90, -(15*zeta(5))/16, catalan*1i - pi^2/48]
```

### Plot Polylogarithms

Plot the polylogarithms of the integer orders  $n$  from -3 to 1 within the interval  $x = [-4 \ 0.3]$ .

```
syms x
for n = -3:1
    fplot(polylog(n,x), [-4 0.3])
    hold on
end
title("Polylogarithm")
legend(Location="best")
hold off
```



### Handle Expressions Containing Polylogarithms

Many functions, such as `diff` and `int`, can handle expressions containing `polylog`.



Differentiate these expressions containing polylogarithms.

```
syms n x
dLi = diff(polylog(n, x), x)
dLi = diff(x*polylog(n, x), x)

dLi =
polylog(n - 1, x)/x

dLi =
polylog(n, x) + polylog(n - 1, x)
```

Compute the integrals of these expressions containing polylogarithms.

```
intLi = int(polylog(n, x)/x, x)
intLi = int(polylog(n, x) + polylog(n - 1, x), x)

intLi =
polylog(n + 1, x)

intLi =
x*polylog(n, x)
```

## Input Arguments

### **n — Order of polylogarithm**

number | array | symbolic number | symbolic variable | symbolic function | symbolic expression | symbolic array

Order of the polylogarithm, specified as a number, array, symbolic number, symbolic variable, symbolic function, symbolic expression, or symbolic array.

Data Types: single | double | sym | symfun

### **x — Argument of polylogarithm**

number | array | symbolic number | symbolic variable | symbolic function | symbolic expression | symbolic array

Argument of the polylogarithm, specified as a number, array, symbolic number, symbolic variable, symbolic function, symbolic expression, or symbolic array.

Data Types: single | double | sym | symfun

## More About

### **Polylogarithm**

For a complex number  $z$  of modulus  $|z| < 1$ , the polylogarithm of order  $n$  is defined as:

$$\text{Li}_n(z) = \sum_{k=1}^{\infty} \frac{z^k}{k^n}.$$

Analytic continuation extends this function the whole complex plane, with a branch cut along the real interval  $[1, \infty)$  for  $n \geq 1$ .

## Tips

- `polylog(2,x)` is equivalent to `dilog(1 - x)`.
- The logarithmic integral function (the integral logarithm) uses the same notation, `li(x)`, but without an index. The toolbox provides the `logint` function to compute the logarithmic integral function.
- Floating-point evaluation of the polylogarithm function can be slow for complex arguments or high-precision numbers. To increase the computational speed, you can reduce the floating-point precision by using the `vpa` and `digits` functions. For more information, see “Increase Speed by Reducing Precision” on page 3-386.
- The polylogarithm function is related to other special functions. For example, it can be expressed in terms of the Hurwitz zeta function  $\zeta(s,a)$  and the gamma function  $\Gamma(z)$ :

$$\text{Li}_n(z) = \frac{\Gamma(1-n)}{(2\pi)^{1-n}} \left[ i^{1-n} \zeta\left(1-n, \frac{1}{2} + \frac{\ln(-z)}{2\pi i}\right) + i^{n-1} \zeta\left(1-n, \frac{1}{2} - \frac{\ln(-z)}{2\pi i}\right) \right].$$

Here,  $n \neq 0, 1, 2, \dots$

## Version History

Introduced in R2014b

## References

- [1] Olver, F. W. J., A. B. Olde Daalhuis, D. W. Lozier, B. I. Schneider, R. F. Boisvert, C. W. Clark, B. R. Miller, and B. V. Saunders, eds., Chapter 25. Zeta and Related Functions, *NIST Digital Library of Mathematical Functions*, Release 1.0.20, Sept. 15, 2018.

## See Also

`dilog` | `log` | `logint` | `hurwitzZeta` | `zeta`

# polynomialDegree

Degree of polynomial

## Syntax

```
polynomialDegree(p)
polynomialDegree(p,vars)
```

## Description

`polynomialDegree(p)` returns the degree of polynomial `p` with respect to all variables found in `p` by `symvar`.

`polynomialDegree(p,vars)` returns the degree of `p` with respect to the variables in `vars`.

## Examples

### Degree of Polynomial

Find the degree of the polynomial  $x^3 + x + 1$ .

```
syms x
p = x^3 + x + 1;
deg = polynomialDegree(p)

deg =
     3
```

### Degree of Multivariate Polynomial with Respect to Variable

Specify variables as the second argument of `polynomialDegree`. Find the degree of the polynomial  $a^2x^3 + b^6x$  with the default independent variables found by `symvar`, the variable `x`, and the variables `[a x]`.

When using the default variables, the degree is 7 because, by default, `a` and `b` are variables. So the total degree of  $b^6x$  is 7.

```
syms a b x
p = a^2*x^3 + b^6*x;
deg = polynomialDegree(p)    % uses symvar

deg =
     7

deg = polynomialDegree(p,x)

deg =
     3
```

```
vars = [a x];  
deg = polynomialDegree(p,vars)  
  
deg =  
    5
```

## Input Arguments

### **p — Polynomial**

symbolic expression | symbolic function

Polynomial, specified as a symbolic expression or function.

### **vars — Polynomial variables**

vector of symbolic variables

Polynomial variables, specified as a vector of symbolic variables.

## Version History

**Introduced in R2018a**

### **See Also**

`coeffs` | `polynomialReduce`

# polynomialReduce

Reduce polynomials by division

## Syntax

```
r = polynomialReduce(p,d)
r = polynomialReduce(p,d,vars)
r = polynomialReduce( ___, 'MonomialOrder', MonomialOrder)

[r,q] = polynomialReduce( ___)
```

## Description

`r = polynomialReduce(p,d)` returns the “Polynomial Reduction” on page 6-1216 of `p` by `d` with respect to all variables in `p` determined by `symvar`. The input `d` can be a vector of polynomials.

`r = polynomialReduce(p,d,vars)` uses the polynomial variables in `vars`.

`r = polynomialReduce( ___, 'MonomialOrder', MonomialOrder)` also uses the specified monomial order in addition to the input arguments in previous syntaxes. Options are 'degreeInverseLexicographic', 'degreeLexicographic', or 'lexicographic'. By default, `polynomialReduce` uses 'degreeInverseLexicographic'.

`[r,q] = polynomialReduce( ___)` also returns the quotient in `q`.

## Examples

### Divide Two Polynomials

Find the quotient and remainder when  $x^3 - x*y^2 + 1$  is divided by  $x + y$ .

```
syms x y
p = x^3 - x*y^2 + 1;
d = x + y;
[r,q] = polynomialReduce(p,d)

r =
1
q =
x^2 - y*x
```

Reconstruct the original polynomial from the quotient and remainder. Check that the reconstructed polynomial equals `p` by using `isAlways`.

```
pOrig = expand(sum(q.*d) + r);
isAlways(p == pOrig)
```

```
ans =  
    logical  
    1
```

### Specify Polynomial Variables

Specify the polynomial variables as the second argument of `polynomialReduce`.

Divide  $\exp(a)x^2 + 2xy + 1$  by  $x - y$  with the polynomial variables  $[x, y]$ , treating  $a$  as a symbolic parameter.

```
syms a x y  
p = exp(a)*x^2 + 2*x*y + 1;  
d = x - y;  
vars = [x y];  
r = polynomialReduce(p,d,vars)  
  
r =  
(exp(a) + 2)*y^2 + 1
```

### Reduce Polynomial by Several Polynomials

Reduce  $x^5 - x^2y^6 - xy$  by  $x^2 + y$  and  $x^2 - y^3$ .

```
syms x y  
p = x^5 - x*y^6 - x*y;  
d = [x^2 + y, x^2 - y^3];  
[r,q] = polynomialReduce(p,d)  
  
r =  
-x*y  
q =  
[ x^3 - x*y^3, x*y^3 - x*y]
```

Reconstruct the original polynomial from the quotient and remainder. Check that the reconstructed polynomial equals  $p$  by using `isAlways`.

```
pOrig = expand(q*d.' + r);  
isAlways(p == pOrig)
```

```
ans =  
    logical  
    1
```

### Specify Term Order of Polynomials

By default, `polynomialReduce` orders the terms in the polynomials with the term order `degreeInverseLexicographic`. Change the term order to `lexicographic` or `degreeLexicographic` by using the `'MonomialOrder'` name-value pair argument.

Divide two polynomials by using the `lexicographic` term order.

```
syms x y  
p = x^2 + y^3 + 1;
```

```
d = x - y^2;
r = polynomialReduce(p,d,'MonomialOrder','lexicographic')

r =
y^4 + y^3 + 1
```

Divide the same polynomials by using the `degreeLexicographic` term order.

```
r = polynomialReduce(p,d,'MonomialOrder','degreeLexicographic')

r =
x^2 + y*x + 1
```

## Input Arguments

### **p — Polynomial to divide**

symbolic expression | symbolic function

Polynomial to divide, specified as a symbolic expression or function.

### **d — Polynomials to divide by**

symbolic expression or function | vector of symbolic expressions or functions

Polynomials to divide by, specified as a symbolic expression or function or a vector of symbolic expressions or functions.

### **vars — Polynomial variables**

vector of symbolic variables

Polynomial variables, specified as a vector of symbolic variables.

### **MonomialOrder — Monomial order of divisors**

'degreeInverseLexicographic' (default) | 'degreeLexicographic' | 'lexicographic'

Monomial order of divisors, specified as 'degreeInverseLexicographic', 'degreeLexicographic', or 'lexicographic'. If you specify `vars`, then `polynomialReduce` sorts variables based on the order of variables in `vars`.

- `lexicographic` sorts the terms of a polynomial using lexicographic ordering.
- `degreeLexicographic` sorts the terms of a polynomial according to the total degree of each term. If terms have equal total degrees, `polynomialReduce` sorts the terms using lexicographic ordering.
- `degreeInverseLexicographic` sorts the terms of a polynomial according to the total degree of each term. If terms have equal total degrees, `polynomialReduce` sorts the terms using inverse lexicographic ordering.

## Output Arguments

### **r — Remainder of polynomial division**

symbolic polynomial

Remainder of polynomial division, returned as a symbolic polynomial.

### **q — Quotient of polynomial division**

symbolic polynomial | vector of symbolic polynomials

Quotient of polynomial division, returned as a symbolic polynomial or a vector of symbolic polynomials.

## More About

### Polynomial Reduction

Polynomial reduction is the division of the polynomial  $p$  by the divisor polynomials  $d_1, d_2, \dots, d_n$ . The terms of the divisor polynomials are ordered according to a certain term order. The quotients  $q_1, q_2, \dots, q_n$  and the remainder  $r$  satisfy this equation.

$$p = q_1 d_1 + q_2 d_2 + \dots + q_n d_n + r.$$

No term in  $r$  can be divided by the leading terms of any of the divisors  $d_1, d_2, \dots, d_n$ .

## Version History

Introduced in R2018a

### See Also

`eliminate` | `gbasis` | `polynomialDegree`



# potential

Potential of vector field

## Syntax

```
potential(V,X)
potential(V,X,Y)
```

## Description

`potential(V,X)` computes the potential of the vector field  $V$  with respect to the vector  $X$  in Cartesian coordinates. The vector field  $V$  must be a gradient field.

`potential(V,X,Y)` computes the potential of vector field  $V$  with respect to  $X$  using  $Y$  as base point for the integration.

## Examples

### Compute Potential of Vector Field

Compute the potential of this vector field with respect to the vector  $[x, y, z]$ :

```
syms x y z
P = potential([x, y, z*exp(z)], [x y z])

P =
x^2/2 + y^2/2 + exp(z)*(z - 1)
```

Use the `gradient` function to verify the result:

```
simplify(gradient(P, [x y z]))

ans =
      x
      y
z*exp(z)
```

### Specify Integration Base Point

Compute the potential of this vector field specifying the integration base point as  $[0 \ 0 \ 0]$ :

```
syms x y z
P = potential([x, y, z*exp(z)], [x y z], [0 0 0])

P =
x^2/2 + y^2/2 + exp(z)*(z - 1) + 1
```

Verify that  $P([0 \ 0 \ 0]) = 0$ :

```
subs(P, [x y z], [0 0 0])

ans =
0
```

### Test Potential for Field Without Gradient

If a vector field is not gradient, `potential` returns NaN:

```
potential([x*y, y], [x y])
```

```
ans =  
NaN
```

## Input Arguments

### V — Vector field

3-D symbolic vector of symbolic expressions or functions (default)

Vector field, specified as a 3-D vector of symbolic expressions or functions.

### X — Input

vector of three symbolic variables

Input, specified as a vector of three symbolic variables with respect to which you compute the potential.

### Y — Input

symbolic vector

Input, specified as a symbolic vector of variables, expressions, or numbers that you want to use as a base point for the integration. If you use this argument, `potential` returns  $P(X)$  such that  $P(Y) = 0$ . Otherwise, the potential is only defined up to some additive constant.

## More About

### Scalar Potential of Gradient Vector Field

The potential of a gradient vector field  $V(X) = [v_1(x_1, x_2, \dots), v_2(x_1, x_2, \dots), \dots]$  is the scalar  $P(X)$  such that  $V(X) = \nabla P(X)$ .

The vector field is gradient if and only if the corresponding Jacobian is symmetrical:

$$\left( \frac{\partial v_i}{\partial x_j} \right) = \left( \frac{\partial v_j}{\partial x_i} \right)$$

The potential function represents the potential in its integral form:

$$P(X) = \int_0^1 (X - Y) \cdot V(Y + \lambda(X - Y)) d\lambda$$

## Tips

- If `potential` cannot verify that  $V$  is a gradient field, it returns NaN.
- Returning NaN does not prove that  $V$  is not a gradient field. For performance reasons, `potential` sometimes does not sufficiently simplify partial derivatives, and therefore, it cannot verify that the field is gradient.

- If `Y` is a scalar, then `potential` expands it into a vector of the same length as `X` with all elements equal to `Y`.

## Version History

Introduced in R2012a

## See Also

`curl` | `diff` | `divergence` | `gradient` | `jacobian` | `hessian` | `laplacian` | `vectorPotential`

## power, .^

Symbolic array power

### Syntax

```
A.^B  
power(A,B)
```

### Description

$A.^B$  computes  $A$  to the  $B$  power and is an element-wise operation.

`power(A,B)` is equivalent to  $A.^B$ .

### Examples

#### Square Each Matrix Element

Create a 2-by-3 matrix.

```
A = sym('a', [2 3])
```

```
A =  
[ a1_1, a1_2, a1_3]  
[ a2_1, a2_2, a2_3]
```

Square each element of the matrix.

```
A.^2
```

```
ans =  
[ a1_1^2, a1_2^2, a1_3^2]  
[ a2_1^2, a2_2^2, a2_3^2]
```

#### Use Matrices for Base and Exponent

Create a 3-by-3 symbolic Hilbert matrix and a 3-by-3 diagonal matrix.

```
H = sym(hilb(3))  
d = diag(sym([1 2 3]))
```

```
H =  
[ 1, 1/2, 1/3]  
[ 1/2, 1/3, 1/4]  
[ 1/3, 1/4, 1/5]
```

```
d =  
[ 1, 0, 0]  
[ 0, 2, 0]  
[ 0, 0, 3]
```

Raise the elements of the Hilbert matrix to the powers of the diagonal matrix. The base and the exponent must be matrices of the same size.

```
H.^d
```

```
ans =
[ 1, 1, 1]
[ 1, 1/9, 1]
[ 1, 1, 1/125]
```

## Input Arguments

### A — Input

number | symbolic number | symbolic scalar variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix | symbolic array

Input, specified as a number or a symbolic number, scalar variable, matrix variable, function, matrix function, expression, or vector, matrix, or array of symbolic scalar variables. Inputs A and B must be the same size unless one is a scalar. A scalar value expands into an array of the same size as the other input.

### B — Input

number | symbolic number | symbolic scalar variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix | symbolic array

Input, specified as a number or a symbolic number, scalar variable, matrix variable, function, matrix function, expression, or vector, matrix, or array of symbolic scalar variables. Inputs A and B must be the same size unless one is a scalar. A scalar value expands into an array of the same size as the other input.

## Version History

**Introduced before R2006a**

### R2022a: Compute element-wise power of symbolic matrix functions

The power function accepts an input argument of type `symfunmatrix`.

### R2021a: Compute element-wise power of symbolic matrix variables

The power function accepts an input argument of type `symmatrix`.

## See Also

`ctranspose` | `ldivide` | `minus` | `mldivide` | `mpower` | `mrdivide` | `mtimes` | `nthroot` | `plus` | `rdivide` | `times` | `transpose`

## powermod

Modular exponentiation

### Syntax

```
c = powermod(a,b,m)
```

### Description

`c = powermod(a,b,m)` returns the modular exponentiation  $a^b \bmod m$ . The input `a`, `b` must be integers, and `m` must be a nonnegative integer. For more information, see “Modular Exponentiation” on page 6-1223.

### Examples

#### Compute Modular Exponentiation

Compute the modular exponentiation  $a^b \bmod m$  by using `powermod`. The `powermod` function is efficient because it does not calculate the exponential  $a^b$ .

```
c = powermod(3,5,7)
```

```
c =  
    5
```

#### Prove Fermat's Little Theorem

Fermat's little theorem states that if  $p$  is prime and  $a$  is not divisible by  $p$ , then  $a^{(p-1)} \bmod p$  is 1.

Test Fermat's little theorem for  $p = 5$ ,  $a = 3$ . As expected, `powermod` returns 1.

```
p = 5;  
a = 3;  
c = powermod(a,p-1,p)
```

```
c =  
    1
```

Test the same case for all values of  $a$  less than  $p$ . The function `powermod` acts element-wise to return a vector of ones.

```
p = 5;  
a = 1:p-1;  
c = powermod(a,p-1,p)
```

```
c =  
    1    1    1    1
```

## Compute Fermat Primes Using Fermat Primality Test

Fermat's little theorem states that if  $p$  is a prime number and  $a$  is not divisible by  $p$ , then  $a^{(p-1)} \bmod p$  is 1. On the contrary, if  $a^{(p-1)} \bmod p$  is 1 and  $a$  is not divisible by  $p$ , then  $p$  is not always a prime number ( $p$  can be a pseudoprime).

Test numbers from 300 to 400 for primality by using Fermat's little theorem with base 2.

```
p = 300:400;
remainder = powermod(2,p-1,p);
primesFermat = p(remainder == 1)

primesFermat =
    307    311    313    317    331    337    341    347    349    353...
    359    367    373    379    383    389    397
```

Find Fermat pseudoprimes by comparing the results with `isprime`. 341 is a Fermat pseudoprime.

```
primeNumbers = p(isprime(p));
setdiff(primesFermat,primeNumbers)

ans =
    341
```

## Input Arguments

### **a — Base**

number | vector | matrix | array | symbolic number | symbolic array

Base, specified as a number, vector, matrix, array, or a symbolic number or array. **a** must be an integer.

### **b — Exponent or power**

number | vector | matrix | array | symbolic number | symbolic array

Exponent or power, specified as a number, vector, matrix, array, or a symbolic number or array. **b** must be an integer.

### **m — Divisor**

number | vector | matrix | array | symbolic number | symbolic array

Divisor, specified as a number, vector, matrix, array, or a symbolic number or array. **m** must be a nonnegative integer.

## More About

### Modular Exponentiation

For a positive exponent  $b$ , the modular exponentiation  $c$  is defined as

$$c = a^b \bmod m.$$

For a negative exponent  $b$ , the definition can be extended by finding the modular multiplicative inverse  $d$  of  $a$  modulo  $m$ , that is

$$c = d^{-b} \bmod m.$$

where  $d$  satisfies the relation

$$ad \bmod m = 1.$$

## **Version History**

**Introduced in R2018a**

## **See Also**

`mod` | `nextprime` | `nthprime` | `prevprime`



# pretty

Pretty-print symbolic expressions

---

**Note** pretty is not recommended. Use live scripts instead for full math rendering. For more information, see “What Is a Live Script or Function?”

---

## Syntax

pretty(X)

## Description

pretty(X) prints X in a plain-text format that resembles typeset mathematics in the Command Window. For full math rendering and a more interactive workflow, use live scripts instead. See “What Is a Live Script or Function?”

## Examples

### Pretty-Print Symbolic Expressions

Pretty-print symbolic expressions in the Command Window.

```
A = sym(pascal(2))
B = eig(A)
pretty(B)
```

A =

```
[1, 1]
[1, 2]
```

B =

```
3/2 - 5^(1/2)/2
5^(1/2)/2 + 3/2
```

```
/ 3    sqrt(5) \
| - - - - - |
| 2      2    |
|          |
| sqrt(5)  3  |
| - - - - - + - |
\      2      2 /
```

## Pretty-Print Long Expressions

Solve an equation in the Command Window, and then use `pretty` to represent the solutions in a format similar to typeset mathematics. For better readability, `pretty` uses abbreviations when representing long expressions.

```
syms x
s = solve(x^4 + 2*x + 1, x, MaxDegree=3);
pretty(s)
```

$$\begin{array}{c} \left( \begin{array}{c} -1 \\ \sqrt[3]{\frac{1}{9} \sqrt{2} + \frac{1}{3}} \\ \frac{1}{9} \sqrt{2} - \frac{\sqrt{2}}{2} + \frac{1}{3} \\ \sqrt{2} + \frac{1}{9} \sqrt{2} - \frac{\sqrt{2}}{2} + \frac{1}{3} \end{array} \right) \end{array}$$

where

$$\begin{array}{l} \#1 = \frac{\sqrt{3} \left( \sqrt[3]{\frac{1}{9} \sqrt{2} + \frac{1}{3}} \right)}{2} \\ \#2 = \left( \frac{\sqrt{11} \sqrt{27}}{27} - \frac{17 \sqrt[3]{1}}{27} \right) \end{array}$$

## Input Arguments

### X — Input

symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic array | symbolic function | symbolic matrix function | symbolic expression

Input, specified as a symbolic number, scalar variable, matrix variable, array, function, matrix function, or expression.

## Version History

Introduced before R2006a

### R2026a: pretty function displays fully typeset mathematics in live scripts

*Behavior changed in R2026a*

When you use the `pretty` function on a symbolic expression within live scripts, the function no longer displays plain-text output. Instead, in live scripts, the `pretty` function displays its input as fully typeset mathematics. Because live scripts automatically display symbolic expressions in typeset mathematical notation with full math rendering, you do not need to use `pretty`.

### R2022a: Pretty-print symbolic matrix functions

The `pretty` function accepts an input argument of type `symfunmatrix`.

### **R2021a: Pretty-print symbolic matrix variables**

The `pretty` function accepts an input argument of type `symmatrix`.

## **See Also**

### **Topics**

"What Is a Live Script or Function?"

## prevprime

Previous prime number

### Syntax

```
prevprime(n)
```

### Description

`prevprime(n)` returns the largest prime number smaller than or equal to `n`. If `n` is a vector or matrix, then `prevprime` acts element-wise on `n`.

### Examples

#### Find Previous Prime Number

Find the largest prime number smaller than 100.

```
prevprime(100)
```

```
ans =  
97
```

Find the largest prime numbers smaller than 1000, 10000, and 100000 by specifying the input as a vector.

```
v = [1000 10000 100000];  
prevprime(v)
```

```
ans =  
          997          9973          99991
```

#### Find Large Prime Number

When finding large prime numbers, return exact symbolic integers by using symbolic input. Further, if your input has 15 or more digits, then use quotation marks and wrap the number in `sym` to represent the number accurately. For more information, see “Numeric to Symbolic Conversion” on page 2-29.

Find a large prime number by using `10^sym(18)`.

```
prevprime(10^sym(18))
```

```
ans =  
99999999999999989
```

Find the prime number previous to 823572345728582545 by using quotation marks.

```
prevprime(sym('823572345728582545'))
```

```
ans =  
823572345728582543
```

## Input Arguments

### **n — Input**

number | vector | matrix | array | symbolic number | symbolic array

Input, specified as a number, vector, matrix, array, or a symbolic number or array.

## Version History

**Introduced in R2016b**

### **See Also**

isprime | nextprime | nthprime | primes

## psi

Digamma function

### Syntax

```
psi(x)
psi(k,x)
```

### Description

`psi(x)` computes the digamma function on page 6-1232 of  $x$ .

`psi(k,x)` computes the polygamma function on page 6-1232 of  $x$ , which is the  $k$ th derivative of the digamma function at  $x$ .

### Examples

#### Compute Digamma and Polygamma for Numeric Inputs

Compute the digamma and polygamma functions for these numbers. Because these numbers are not symbolic objects, you get the floating-point results.

```
[psi(1/2) psi(2, 1/2) psi(1.34) psi(1, sin(pi/3))]  
  
ans =  
   -1.9635   -16.8288   -0.1248    2.0372
```

#### Compute Digamma and Polygamma for Symbolic Inputs

Compute the digamma and polygamma functions for the numbers converted to symbolic objects.

```
[psi(sym(1/2)), psi(1, sym(1/2)), psi(sym(1/4))]  
  
ans =  
[ - eulergamma - 2*log(2), pi^2/2, - eulergamma - pi/2 - 3*log(2)]
```

For some symbolic (exact) numbers, `psi` returns unresolved symbolic calls.

```
psi(sym(sqrt(2)))  
  
ans =  
psi(2^(1/2))
```

#### Compute Derivatives of Digamma and Polygamma Functions

Compute the derivatives of these expressions containing the digamma and polygamma functions.

```
syms x  
diff(psi(1, x^3 + 1), x)  
diff(psi(sin(x)), x)  
  
ans =  
3*x^2*psi(2, x^3 + 1)
```

```
ans =
cos(x)*psi(1, sin(x))
```

### Expand Digamma and Polygamma Functions

Expand the expressions containing the digamma functions.

```
syms x
expand(psi(2*x + 3))
expand(psi(x + 2)*psi(x))

ans =
psi(x + 1/2)/2 + log(2) + psi(x)/2 +...
1/(2*x + 1) + 1/(2*x + 2) + 1/(2*x)

ans =
psi(x)/x + psi(x)^2 + psi(x)/(x + 1)
```

### Limit of Digamma and Polygamma Functions

Compute the limits for expressions containing the digamma and polygamma functions.

```
syms x
limit(x*psi(x), x, 0)
limit(psi(3, x), x, inf)

ans =
-1

ans =
0
```

### Compute Digamma for Matrix Input

Compute the digamma function for elements of matrix **M** and vector **V**.

```
M = sym([0 inf; 1/3 1/2]);
V = sym([1, inf]);
psi(M)
psi(V)

ans =
[
      Inf,      Inf]
[ -eulergamma - (3*log(3))/2 - (pi*3^(1/2))/6, -eulergamma - 2*log(2)]

ans =
[ -eulergamma, Inf]
```

### Compute Polygamma for Matrix Input

Compute the polygamma function for elements of matrix **M** and vector **V**. The **psi** function acts elementwise on nonscalar inputs.

```
M = sym([0 inf; 1/3 1/2]);
polyGammaM = [1 3; 2 2];
V = sym([1, inf]);
polyGammaV = [6 6];
psi(polyGammaM,M)
psi(polyGammaV,V)
```

```
ans =  
[  
    Inf, 0]  
[ -26*zeta(3) - (4*3^(1/2)*pi^3)/9, -14*zeta(3)]
```

```
ans =  
[ -720*zeta(7), 0]
```

Because all elements of `polyGammaV` have the same value, you can replace `polyGammaV` by a scalar of that value. `psi` expands the scalar into a nonscalar of the same size as `V` and computes the result.

```
V = sym([1, inf]);  
psi(6,V)
```

```
ans =  
[ -720*zeta(7), 0]
```

## Input Arguments

### **x** — Input

symbolic number | symbolic variable | symbolic expression | symbolic array

Input, specified as a symbolic number, variable, expression, or array, or expression.

### **k** — Input

nonnegative integer | nonnegative integer or vector, matrix or multidimensional array of nonnegative integers.

Input, specified as a nonnegative integer or vector, matrix or multidimensional array of nonnegative integers. If `x` is nonscalar and `k` is scalar, then `k` is expanded into a nonscalar of the same dimensions as `x` with each element being equal to `k`. If both `x` and `k` are nonscalars, they must have the same dimensions.

## More About

### Digamma Function

The digamma function is the first derivative of the logarithm of the gamma function:

$$\psi(x) = \frac{d}{dx} \ln \Gamma(x) = \frac{\Gamma'(x)}{\Gamma(x)}$$

### Polygamma Function

The polygamma function of the order  $k$  is the  $(k + 1)$ th derivative of the logarithm of the gamma function:

$$\psi^{(k)}(x) = \frac{d^{k+1}}{dx^{k+1}} \ln \Gamma(x) = \frac{d^k}{dx^k} \psi(x)$$

## Tips

- Calling `psi` for a number that is not a symbolic object invokes the MATLAB `psi` function. This function accepts real nonnegative arguments `x`. If you want to compute the polygamma function for a complex number, use `sym` to convert that number to a symbolic object, and then call `psi` for that symbolic object.



- `psi(0, x)` is equivalent to `psi(x)`.

## Version History

Introduced in R2011b

## See Also

`beta` | `gamma` | `nchoosek` | `factorial`

## qr

QR decomposition of symbolic matrix

### Syntax

```
R = qr(A)
[Q,R] = qr(A)
[Q,R,P] = qr(A)
[___] = qr(A,"econ")
[Q,R,P] = qr( ___,outputForm)
[___] = qr(A,0)

[C,R] = qr(A,B)
[C,R,P] = qr(A,B)
[___] = qr(A,B,"econ")
[C,R,P] = qr( ___,outputForm)
[___] = qr(A,B,0)

___ = qr( ___, "real")
```

### Description

$R = \text{qr}(A)$  returns the R factor of the QR decomposition on page 6-1244  $A = Q \cdot R$ . Here,  $A$  is an  $m$ -by- $n$  symbolic matrix,  $R$  is an  $m$ -by- $n$  upper triangular symbolic matrix, and  $Q$  is an  $m$ -by- $m$  unitary symbolic matrix.

$[Q,R] = \text{qr}(A)$  returns an upper triangular matrix  $R$  and a unitary matrix  $Q$  such that  $A = Q \cdot R$ .

$[Q,R,P] = \text{qr}(A)$  also returns a permutation matrix  $P$  such that  $A \cdot P = Q \cdot R$ . If all elements of  $A$  can be approximated by floating-point numbers, then this syntax chooses the permutation matrix  $P$  so that  $\text{abs}(\text{diag}(R))$  is decreasing. Otherwise, it returns  $P = \text{eye}(n)$ .

$[___] = \text{qr}(A, \text{"econ"})$  returns the economy-size decomposition using any of the previous output argument combinations. The sizes of the outputs depend on the size of  $m$ -by- $n$  matrix  $A$ :

- If  $m > n$ , then  $\text{qr}$  computes only the first  $n$  columns of  $Q$  and the first  $n$  rows of  $R$ .
- If  $m \leq n$ , then the economy-size decomposition is the same as the QR decomposition without the "econ" option.

By default, when you use "econ",  $\text{qr}$  returns the permutation information as a matrix  $P$ .

$[Q,R,P] = \text{qr}( ___, \text{outputForm})$  specifies whether to return the permutation information  $P$  as a matrix or a vector. For example, if  $\text{outputForm}$  is "vector", then  $A(:,P) = Q \cdot R$ . The default value of  $\text{outputForm}$  is "matrix" such that  $A \cdot P = Q \cdot R$ .

$[___] = \text{qr}(A, 0)$  is equivalent to  $\text{qr}(A, \text{"econ"}, \text{"vector"})$ . This syntax is not recommended. Use the "econ" option instead.

$[C,R] = \text{qr}(A,B)$  returns an upper triangular matrix  $R$  and a matrix  $C$  such that  $C = Q' \cdot B$  and  $A = Q \cdot R$ . Here,  $A$  and  $B$  must have the same number of rows.

$C$  and  $R$  represent the solution of the matrix equation  $A*X = B$  as  $X = R \backslash C$ .

$[C,R,P] = \text{qr}(A,B)$  also returns a permutation matrix  $P$  such that  $A*P = Q*R$ . If all elements of  $A$  can be approximated by floating-point numbers, then this syntax chooses the permutation matrix  $P$  so that  $\text{abs}(\text{diag}(R))$  is decreasing. Otherwise, it returns  $P = \text{eye}(n)$ .

$C$ ,  $R$ , and  $P$  represent the solution of the matrix equation  $A*X = B$  as  $X = P*(R \backslash C)$ .

$[\text{___}] = \text{qr}(A,B,\text{"econ"})$  returns the economy-size decomposition using any of the previous output argument combinations. The sizes of the outputs depend on the size of  $m$ -by- $n$  matrix  $A$ :

- If  $m > n$ , then  $\text{qr}$  computes only the first  $n$  rows of  $C$  and  $R$ .
- If  $m \leq n$ , then the economy-size decomposition is the same as the QR decomposition without the "econ" option.

By default, when you use "econ",  $\text{qr}$  returns the permutation information as a matrix  $P$ .

$[C,R,P] = \text{qr}(\text{___},\text{outputForm})$  specifies whether to return the permutation information  $P$  as a matrix or a vector. For example, if  $\text{outputForm}$  is "vector", then the solution of the matrix equation  $A*X = B$  is  $X(P,:) = R \backslash C$ . The default value of  $\text{outputForm}$  is "matrix" such that the solution to  $A*X = B$  is  $X = P*(R \backslash C)$ .

$[\text{___}] = \text{qr}(A,B,0)$  is equivalent to  $\text{qr}(A,B,\text{"econ"},\text{"vector"})$ . This syntax is not recommended. Use the "econ" option instead.

$\text{___} = \text{qr}(\text{___},\text{"real"})$  assumes that the input arguments and intermediate results are real, and therefore, suppresses calls to  $\text{abs}$  and  $\text{conj}$ . When you use this option,  $\text{qr}$  assumes that all symbolic variables represent real numbers. All numeric arguments should be real numbers for this syntax.

Use "real" to avoid complex conjugates in the result.

## Examples

### Upper Triangular Factor of QR Decomposition

Compute the upper triangular  $R$  factor of the QR decomposition of the 4-by-4 Wilkinson's eigenvalue test matrix.

Create the 4-by-4 Wilkinson's eigenvalue test matrix.

```
A = sym(wilkinson(4))
```

$A =$

$$\begin{pmatrix} \frac{3}{2} & 1 & 0 & 0 \\ 1 & \frac{1}{2} & 1 & 0 \\ 0 & 1 & \frac{1}{2} & 1 \\ 0 & 0 & 1 & \frac{3}{2} \end{pmatrix}$$

Return the  $R$  factor of the QR decomposition without returning the  $Q$  factor.

$$R = \text{qr}(A)$$

$$R = \begin{pmatrix} \frac{\sqrt{13}}{2} & \frac{4\sqrt{13}}{13} & \frac{2\sqrt{13}}{13} & 0 \\ 0 & \frac{\sqrt{13}\sqrt{53}}{26} & \frac{10\sqrt{13}\sqrt{53}}{689} & \frac{2\sqrt{13}\sqrt{53}}{53} \\ 0 & 0 & \frac{\sqrt{53}\sqrt{381}}{106} & \frac{172\sqrt{53}\sqrt{381}}{20193} \\ 0 & 0 & 0 & \frac{35\sqrt{381}}{762} \end{pmatrix}$$

### QR Decomposition of Pascal Matrix

Compute the QR decomposition of the 3-by-3 Pascal matrix.

Create the 3-by-3 Pascal matrix.

$$A = \text{sym}(\text{pascal}(3))$$

$$A = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 1 & 3 & 6 \end{pmatrix}$$

Find the Q and R matrices representing the QR decomposition of A.

$$[Q,R] = \text{qr}(A)$$

$$Q = \begin{pmatrix} \frac{\sqrt{3}}{3} & -\frac{\sqrt{2}}{2} & \frac{\sqrt{6}}{6} \\ \frac{\sqrt{3}}{3} & 0 & -\frac{\sqrt{6}}{3} \\ \frac{\sqrt{3}}{3} & \frac{\sqrt{2}}{2} & \frac{\sqrt{6}}{6} \end{pmatrix}$$

$$R = \begin{pmatrix} \sqrt{3} & 2\sqrt{3} & \frac{10\sqrt{3}}{3} \\ 0 & \sqrt{2} & \frac{5\sqrt{2}}{2} \\ 0 & 0 & \frac{\sqrt{6}}{6} \end{pmatrix}$$

Verify that  $A = Q \cdot R$  by using `isAlways`.

$$TF = \text{isAlways}(A == Q \cdot R)$$

$TF = 3 \times 3$  logical array

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

1 1 1

### Permuted QR Decomposition

Use permutations to improve the numerical stability of the QR decomposition for floating-point matrices. The `qr` function returns permutation information either as a matrix or as a vector.

Set the number of significant decimal digits, used for variable-precision arithmetic, to 10. Approximate the 3-by-3 symbolic Hilbert matrix with floating-point numbers.

```
previoussetting = digits(10);
A = vpa(hilb(3))
```

```
A =
    (
      1.0      0.5      0.3333333333
      0.5      0.3333333333      0.25
      0.3333333333      0.25      0.2
    )
```

First, compute the QR decomposition of `A` without permutations.

```
[Q,R] = qr(A)
```

```
Q =
    (0.8571428571 -0.5016049166 0.1170411472
     0.4285714286 0.5684855721 -0.7022468832
     0.2857142857 0.6520863915 0.7022468832
    )
```

```
R =
    (1.166666667 0.6428571429 0.45
     0 0.1017143303 0.1053370325
     0 0 0.003901371573
    )
```

Compute the difference between `A` and `Q*R`. The computed `Q` and `R` matrices do not strictly satisfy the equality `A = Q*R` because of round-off errors.

```
E = A - Q*R
```

```
E =
    (-3.469446952e-18 -6.938893904e-18 -4.33680869e-18
     0 -1.734723476e-18 -8.67361738e-19
     0 -8.67361738e-19 -4.33680869e-19
    )
```

To increase numerical stability of the QR decomposition, use permutations by specifying the syntax with three output arguments. For matrices that do not contain symbolic variables, expressions, or functions, this syntax triggers pivoting so that `abs(diag(R))` in the returned matrix `R` is decreasing.

```
[Q,R,P] = qr(A)
```

```
Q =
    (0.8571428571 -0.4969293466 -0.1355261854
     0.4285714286 0.5421047417 0.7228063223
     0.2857142857 0.6776309272 -0.6776309272
    )
```

$$R = \begin{pmatrix} 1.166666667 & 0.45 & 0.6428571429 \\ 0 & 0.1054092553 & 0.1016446391 \\ 0 & 0 & 0.003764616262 \end{pmatrix}$$

$$P = 3 \times 3$$

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

Check the equality  $A \cdot P = Q \cdot R$ . QR decomposition with permutations results in smaller round-off errors.

$$E = A \cdot P - Q \cdot R$$

$$E = \begin{pmatrix} -3.469446952e-18 & -4.33680869e-18 & -6.938893904e-18 \\ 0 & -8.67361738e-19 & -1.734723476e-18 \\ 0 & -4.33680869e-19 & -1.734723476e-18 \end{pmatrix}$$

Now, return the permutation information as a vector by using the "vector" argument.

$$[Q, R, p] = \text{qr}(A, \text{"vector"})$$

$$Q = \begin{pmatrix} 0.8571428571 & -0.4969293466 & -0.1355261854 \\ 0.4285714286 & 0.5421047417 & 0.7228063223 \\ 0.2857142857 & 0.6776309272 & -0.6776309272 \end{pmatrix}$$

$$R = \begin{pmatrix} 1.166666667 & 0.45 & 0.6428571429 \\ 0 & 0.1054092553 & 0.1016446391 \\ 0 & 0 & 0.003764616262 \end{pmatrix}$$

$$p = 1 \times 3$$

$$\begin{pmatrix} 1 & 3 & 2 \end{pmatrix}$$

Verify that  $A(:, p) = Q \cdot R$ .

$$E = A(:, p) - Q \cdot R$$

$$E = \begin{pmatrix} -3.469446952e-18 & -4.33680869e-18 & -6.938893904e-18 \\ 0 & -8.67361738e-19 & -1.734723476e-18 \\ 0 & -4.33680869e-19 & -1.734723476e-18 \end{pmatrix}$$

Exact symbolic computations do not result in round-off errors.

$$\begin{aligned} A &= \text{sym}(\text{hilb}(3)); \\ [Q, R] &= \text{qr}(A); \\ E &= A - Q \cdot R \end{aligned}$$

$$E =$$

$$\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

Restore the number of significant decimal digits to its previous setting.

`digits(previoussetting)`

### Solve Matrix Equation with QR Decomposition

You can solve systems of equations in matrix form by using `qr`.

Solve the system of equations  $A*X = b$ , where  $A$  is a 5-by-5 symbolic matrix and  $b$  is a 5-by-1 symbolic vector.

`A = sym(invhilb(5))`

$$A = \begin{pmatrix} 25 & -300 & 1050 & -1400 & 630 \\ -300 & 4800 & -18900 & 26880 & -12600 \\ 1050 & -18900 & 79380 & -117600 & 56700 \\ -1400 & 26880 & -117600 & 179200 & -88200 \\ 630 & -12600 & 56700 & -88200 & 44100 \end{pmatrix}$$

`b = sym([1:5]')`

$$b = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{pmatrix}$$

Find matrices  $C$  and  $R$  such that  $C = Q'*b$  and  $A = Q*R$ .

`[C,R] = qr(A,b);`

Compute the solution  $X$ .

`X = R\C`

$$X = \begin{pmatrix} 5 \\ \frac{71}{20} \\ \frac{197}{70} \\ \frac{657}{280} \\ \frac{1271}{630} \end{pmatrix}$$

Verify that  $X$  is the solution of the system  $A*X = b$  by using `isAlways`.

```
tf = isAlways(A*X == b)
```

```
tf = 5×1 logical array
```

```
1
1
1
1
1
```

### Solve Matrix Equation with Permuted QR Decomposition

When solving systems of equations that contain floating-point numbers, use QR decomposition with a permutation matrix or vector.

Solve the system of equations  $A \cdot X = b$ , where  $A$  is a 3-by-3 symbolic matrix with variable precision and  $b$  is a 3-by-1 symbolic vector with variable precision. Find matrices  $C$  and  $R$  such that  $C = Q' \cdot b$  and  $A = Q \cdot R$ .

```
previoussetting = digits(10);
A = vpa([2 -3 -1; 1 1 -1; 0 1 -1]);
b = vpa([2; 0; -1]);
[C,R,P] = qr(A,b)
```

```
C =
```

$$\begin{pmatrix} -2.110579412 \\ -0.2132007164 \\ 0.7071067812 \end{pmatrix}$$

```
R =
```

$$\begin{pmatrix} 3.31662479 & 0.3015113446 & -1.507556723 \\ 0 & 1.705605731 & -1.492405014 \\ 0 & 0 & 0.7071067812 \end{pmatrix}$$

```
P = 3×3
```

$$\begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

Compute the solution  $X$ .

```
X = P*(R\C)
```

```
X =
```

$$\begin{pmatrix} 1.0 \\ -0.25 \\ 0.75 \end{pmatrix}$$

Alternatively, return the permutation information as a vector.

```
[C,R,p] = qr(A,b,"vector")
```



```

C =
    (-2.110579412)
    (-0.2132007164)
    (0.7071067812)

R =
    (3.31662479 0.3015113446 -1.507556723)
    (0 1.705605731 -1.492405014)
    (0 0 0.7071067812)

p = 1×3
    2    3    1

```

Compute the solution X with permutation vector p.

```
X(p,:) = R\C
```

```

X =
    ( 1.0 )
    (-0.25)
    ( 0.75 )

```

Restore the number of significant decimal digits to its previous setting.

```
digits(previoussetting)
```

### Economy-Size QR Decomposition

Compute the economy-size QR decomposition by using the "econ" option.

Create a matrix that consists of the first two columns of the 4-by-4 Pascal matrix.

```

A = sym(pascal(4));
A = A(:,1:2)

```

```

A =
    (1 1)
    (1 2)
    (1 3)
    (1 4)

```

Compute the QR decomposition for this matrix.

```
[Q,R] = qr(A)
```

```
Q =
```

$$\begin{pmatrix} \frac{1}{2} & -\sigma_1 & \frac{\sqrt{3}\sqrt{10}}{10} & 0 \\ \frac{1}{2} & -\frac{\sqrt{5}}{10} & -\frac{2\sqrt{3}\sqrt{10}}{15} & \frac{\sqrt{6}}{6} \\ \frac{1}{2} & \frac{\sqrt{5}}{10} & -\frac{\sqrt{3}\sqrt{10}}{30} & -\frac{\sqrt{6}}{3} \\ \frac{1}{2} & \sigma_1 & \frac{\sqrt{3}\sqrt{10}}{15} & \frac{\sqrt{6}}{6} \end{pmatrix}$$

where

$$\sigma_1 = \frac{3\sqrt{5}}{10}$$

$$R = \begin{pmatrix} 2 & 5 \\ 0 & \sqrt{5} \\ 0 & 0 \\ 0 & 0 \end{pmatrix}$$

Now, compute the economy-size QR decomposition for this matrix. Because the number of rows is greater than the number of columns in A, `qr` computes only the first 2 columns of Q and the first 2 rows of R.

`[Q,R] = qr(A, "econ")`

$$Q = \begin{pmatrix} \frac{1}{2} & -\frac{3\sqrt{5}}{10} \\ \frac{1}{2} & -\frac{\sqrt{5}}{10} \\ \frac{1}{2} & \frac{\sqrt{5}}{10} \\ \frac{1}{2} & \frac{3\sqrt{5}}{10} \end{pmatrix}$$

$$R = \begin{pmatrix} 2 & 5 \\ 0 & \sqrt{5} \end{pmatrix}$$

### Avoid Complex Conjugates

You can avoid complex conjugates in the result of QR decomposition by using the `"real"` option.

Create a matrix that includes a symbolic variable as an element.

`syms x`  
`A = [1 2; 3 x]`

$$A = \begin{pmatrix} 1 & 2 \\ 3 & x \end{pmatrix}$$

Compute the QR decomposition of this matrix. By default, `qr` assumes that `x` represents a complex number, and therefore, the result contains expressions with the `abs` function.

`[Q,R] = qr(A)`

`Q =`

$$\begin{pmatrix} \frac{\sqrt{10}}{10} & -\frac{\frac{3x}{10} - \frac{9}{5}}{\sigma_1} \\ \frac{3\sqrt{10}}{10} & \frac{\frac{x}{10} - \frac{3}{5}}{\sigma_1} \end{pmatrix}$$

where

$$\sigma_1 = \sqrt{\left|\frac{x}{10} - \frac{3}{5}\right|^2 + \left|\frac{3x}{10} - \frac{9}{5}\right|^2}$$

`R =`

$$\begin{pmatrix} \sqrt{10} & \frac{\sqrt{10}(3x+2)}{10} \\ 0 & \sqrt{\left|\frac{x}{10} - \frac{3}{5}\right|^2 + \left|\frac{3x}{10} - \frac{9}{5}\right|^2} \end{pmatrix}$$

When you use the `"real"` option, `qr` assumes that all symbolic variables represent real numbers and can return shorter results.

`[Q,R] = qr(A,"real")`

`Q =`

$$\begin{pmatrix} \frac{\sqrt{10}}{10} & -\frac{\frac{3x}{10} - \frac{9}{5}}{\sqrt{\frac{x^2}{10} - \frac{6x}{5} + \frac{18}{5}}} \\ \frac{3\sqrt{10}}{10} & \frac{\frac{x}{10} - \frac{3}{5}}{\sqrt{\frac{x^2}{10} - \frac{6x}{5} + \frac{18}{5}}} \end{pmatrix}$$

`R =`

$$\begin{pmatrix} \sqrt{10} & \frac{\sqrt{10}(3x+2)}{10} \\ 0 & \sqrt{\frac{x^2}{10} - \frac{6x}{5} + \frac{18}{5}} \end{pmatrix}$$

## Input Arguments

### A — Symbolic input

*m*-by-*n* symbolic matrix

Symbolic input, specified as an *m*-by-*n* symbolic matrix.

### B — Symbolic input

symbolic vector | symbolic matrix

Symbolic input, specified as a symbolic vector or matrix. B must have the same number of rows as A.

**outputForm — Shape of permutation output**`"matrix" (default) | "vector"`

Shape of the permutation output, specified as `"matrix"` or `"vector"`. This option controls whether the permutation output  $P$  is returned as a permutation matrix or vector. You must specify three output arguments to `qr` to use this option.

- If `outputForm` is `"matrix"`, then  $P$  is a permutation matrix that satisfies  $A^*P = Q^*R$ .
- If `outputForm` is `"vector"`, then  $P$  is a permutation vector that satisfies  $A(:,P) = Q^*R$ .

## Output Arguments

**R — R factor of QR decomposition**`upper triangular symbolic matrix`

R factor of the QR decomposition, returned as an upper triangular symbolic matrix that satisfies  $A = Q^*R$  for an  $m$ -by- $n$  matrix  $A$ .

**Q — Q factor of QR decomposition**`unitary symbolic matrix`

Q factor of the QR decomposition, returned as a unitary symbolic matrix that satisfies  $A = Q^*R$  for an  $m$ -by- $n$  matrix  $A$ .

- For full decomposition, `qr(A)` returns  $Q$  as an  $m$ -by- $m$  unitary matrix.
- For rectangular matrix  $A$  with  $m > n$ , the economy-size decomposition `qr(A, "econ")` computes only the first  $n$  columns of  $Q$  and first  $n$  rows of  $R$ .

**P — Permutation information**`matrix | vector`

Permutation information, returned as a matrix or vector of double-precision values. The shape of  $P$  depends on the value of `outputForm`.

**C — Linear system factor**`symbolic matrix`

Linear system factor, returned as a symbolic matrix representing the solution of the matrix equation  $A^*X = B$ , such that  $C = Q^*B$  and  $X = R \setminus C$ . If you specify the permutation output  $P$ , then the solution is either  $X = P^*(R \setminus C)$  when `outputForm` is `"matrix"` or  $X(P,:) = R \setminus C$  when `outputForm` is `"vector"`.

## More About

**QR Decomposition of Matrix**

The QR decomposition, also known as the QR factorization, expresses an  $m$ -by- $n$  matrix  $A$  as  $A = Q^*R$ . For the full decomposition,  $Q$  is an  $m$ -by- $m$  unitary matrix, and  $R$  is an  $m$ -by- $n$  upper triangular matrix. If the components of  $A$  are real numbers, then  $Q$  is an orthogonal matrix.

## Tips

- The upper triangular matrix  $R$  satisfies the condition  $R = \text{chol}(A'*A)$ .
- The arguments "econ" and 0 affect only the shape of the returned matrices.
- Calling `qr` for numeric matrices that are not symbolic objects (not created by `sym`, `syms`, or `vpa`) invokes the MATLAB `qr` function.
- Matrix computations involving many symbolic variables can be slow. To increase the computational speed, reduce the number of symbolic variables by substituting the given values for some variables.

## Version History

### Introduced in R2014a

#### **R2023a: Economy-size decomposition returns permutation information as matrix**

*Behavior changed in R2023a*

The syntaxes  $[Q,R,P] = \text{qr}(A, \text{"econ"})$  and  $[C,R,P] = \text{qr}(A,B, \text{"econ"})$  for economy-size decomposition of symbolic inputs now return the permutation information as a matrix  $P$ . In previous releases, these syntaxes returned the permutation information as a vector  $P$ .

- Starting in R2023a, the syntaxes `qr(__, "econ")` and `qr(__, "econ", "matrix")` are equivalent. They both return the permutation information as a matrix  $P$ . Furthermore, the syntaxes `qr(__, 0)` and `qr(__, "econ", "vector")` are equivalent. They both return the permutation information as a vector  $P$ .
- Before R2023a, the syntaxes `qr(__, "econ")`, `qr(__, 0)`, and `qr(__, "econ", "vector")` were equivalent. They all returned the permutation information as a vector  $P$ . The syntax `qr(__, "econ", "matrix")` previously returned an error.

## See Also

`chol` | `eig` | `lu` | `svd`

## quorem

Quotient and remainder

### Syntax

```
[Q,R] = quorem(A,B,var)
[Q,R] = quorem(A,B)
```

### Description

`[Q,R] = quorem(A,B,var)` divides `A` by `B` and returns the quotient `Q` and remainder `R` of the division, such that  $A = Q*B + R$ . This syntax regards `A` and `B` as polynomials in the variable `var`.

If `A` and `B` are matrices, `quorem` performs elements-wise division, using `var` as a variable. It returns the quotient `Q` and remainder `R` of the division, such that  $A = Q.*B + R$ .

`[Q,R] = quorem(A,B)` uses the variable determined by `symvar(A,1)`. If `symvar(A,1)` returns an empty symbolic object `sym([ ])`, then `quorem` uses the variable determined by `symvar(B,1)`.

If both `symvar(A,1)` and `symvar(B,1)` are empty, then `A` and `B` must both be integers or matrices with integer elements. In this case, `quorem(A,B)` returns symbolic integers `Q` and `R`, such that  $A = Q*B + R$ . If `A` and `B` are matrices, then `Q` and `R` are symbolic matrices with integer elements, such that  $A = Q.*B + R$ , and each element of `R` is smaller in absolute value than the corresponding element of `B`.

### Examples

#### Divide Multivariate Polynomials

Compute the quotient and remainder of the division of these multivariate polynomials with respect to the variable `y`:

```
syms x y
p1 = x^3*y^4 - 2*x*y + 5*x + 1;
p2 = x*y;
[q, r] = quorem(p1, p2, y)
```

```
q =
x^2*y^3 - 2
```

```
r =
5*x + 1
```

#### Divide Univariate Polynomials

Compute the quotient and remainder of the division of these univariate polynomials:

```
syms x
p = x^3 - 2*x + 5;
[q, r] = quorem(x^5, p)
```

```
q =
x^2 + 2
```

```
r =
- 5*x^2 + 4*x - 10
```

### Divide Integers

Compute the quotient and remainder of the division of these integers:

```
[q, r] = quorem(sym(10)^5, sym(985))
```

```
q =
101
```

```
r =
515
```

## Input Arguments

### A — Dividend (numerator)

symbolic integer | polynomial | symbolic vector | symbolic matrix

Dividend (numerator), specified as a symbolic integer, polynomial, or a vector or matrix of symbolic integers or polynomials.

### B — Divisor (denominator)

symbolic integer | polynomial | symbolic vector | symbolic matrix

Divisor (denominator), specified as a symbolic integer, polynomial, or a vector or matrix of symbolic integers or polynomials.

### var — Polynomial variable

symbolic variable

Polynomial variable, specified as a symbolic variable.

## Output Arguments

### Q — Quotient of the division

symbolic integer | symbolic expression | symbolic vector | symbolic matrix

Quotient of the division, returned as a symbolic integer, expression, or a vector or matrix of symbolic integers or expressions.

### R — Remainder of the division

symbolic integer | symbolic expression | symbolic vector | symbolic matrix

Remainder of the division, returned as a symbolic integer, expression, or a vector or matrix of symbolic integers or expressions.

## Version History

Introduced before R2006a

**See Also**

deconv | mod



# rank

Rank of symbolic matrix

## Syntax

`k = rank(A)`

## Description

`k = rank(A)` returns the rank of symbolic matrix `A`.

## Examples

### Rank of Symbolic Matrix

Create a symbolic matrix.

```
syms a b c d
A = [a b; c d]
```

```
A =
    (a b)
    (c d)
```

Determine the rank of the matrix.

```
k = rank(A)
```

```
k =
2
```

### Rank of Symbolic Matrix Without Round-Off Errors

The symbolic `rank` function returns the exact rank of a symbolic matrix when the rank calculation does not require any further simplification and does not have applied conditions or assumptions. In contrast, numeric rank calculations performed on floating-point matrices can be susceptible to round-off errors, which can yield inaccurate results. The exact symbolic calculation is useful for ill-conditioned matrices, such as the Hilbert matrix. The rank of a Hilbert matrix of order  $n$  is  $n$ .

Find the rank of the Hilbert matrix of order 15 numerically. Then convert the numeric matrix to a symbolic matrix using `sym` and find the rank symbolically.

```
H = hilb(15);
k1 = rank(H)
```

```
k1 =
12
```

```
k2 = rank(sym(H))
```

```
k2 =
15
```

The symbolic calculation returns the correct rank of 15. However, the numeric calculation returns an inaccurate rank of 12 due to round-off errors.

### Inaccurate Rank Calculation That Requires Simplification

The rank function can overestimate or underestimate the rank of a symbolic matrix if the calculation requires further simplification.

Find the rank of the following square matrix and its determinant.

```
syms x
A = [1 - sin(x)  cos(x);
     cos(x)      1 + sin(x)]
```

```
A =

$$\begin{pmatrix} 1 - \sin(x) & \cos(x) \\ \cos(x) & \sin(x) + 1 \end{pmatrix}$$

```

```
k = rank(A)
```

```
k =
2
```

```
detA = simplify(det(A))
```

```
detA = 0
```

A square matrix has a full rank if and only if its determinant is nonzero. In this case, `rank` overestimates the rank of the matrix and returns the full rank 2.

Although the matrix elements are already simplified, `rank` returns an overestimated result because it does not apply the required trigonometric identity  $\cos^2(x) + \sin^2(x) = 1$  to calculate the exact rank of the matrix. For this reason, `rank` returns an overestimated result even after substituting a value for the variable in the symbolic matrix.

```
A2 = subs(A,x,3)
```

```
A2 =

$$\begin{pmatrix} 1 - \sin(3) & \cos(3) \\ \cos(3) & \sin(3) + 1 \end{pmatrix}$$

```

```
k2 = rank(A2)
```

```
k2 =
2
```

You can further convert this matrix to variable precision and find its rank.

```
A3 = vpa(A2)
```

```
A3 =

$$\begin{pmatrix} 0.85887999194013277789925519719189 & -0.98999249660044545727157279473126 \\ -0.98999249660044545727157279473126 & 1.1411200080598672221007448028081 \end{pmatrix}$$

```

```
k3 = rank(A3)
```

```
k3 =  
1
```

Here, `rank` returns the correct result for the variable-precision matrix.

Next, find the rank of the following square matrix and its determinant.

```
syms phi  
B = [cos(phi) cos(phi - 2*pi/3) cos(phi - 4*pi/3);  
     sin(phi) sin(phi - 2*pi/3) sin(phi - 4*pi/3);  
     1         1                 1]
```

```
B =  

$$\begin{pmatrix} \cos(\phi) & \cos\left(\phi - \frac{2\pi}{3}\right) & \cos\left(\phi - \frac{4\pi}{3}\right) \\ \sin(\phi) & \sin\left(\phi - \frac{2\pi}{3}\right) & \sin\left(\phi - \frac{4\pi}{3}\right) \\ 1 & 1 & 1 \end{pmatrix}$$

```

```
k = rank(B)
```

```
k =  
2
```

```
detB = simplify(det(B))
```

```
detB =  

$$-\frac{3\sqrt{3}}{2}$$

```

The determinant of the matrix is nonzero, so the matrix has a full rank of 3. In this case, `rank` underestimates the rank of the matrix and returns 2.

Rewrite the matrix using `simplify` and find its rank.

```
B2 = simplify(B)
```

```
B2 =  

$$\begin{pmatrix} \cos(\phi) & -\cos\left(\phi + \frac{\pi}{3}\right) & -\sin\left(\phi + \frac{\pi}{6}\right) \\ \sin(\phi) & -\sin\left(\phi + \frac{\pi}{3}\right) & \cos\left(\phi + \frac{\pi}{6}\right) \\ 1 & 1 & 1 \end{pmatrix}$$

```

```
k2 = rank(B2)
```

```
k2 =  
3
```

Here, `rank` returns the correct result for the simplified matrix.

### Solve System of Equations to Determine Rank of Matrix

The `rank` function can return an inaccurate result if the rank calculation requires further simplification or has applied conditions or assumptions. In this case, you can investigate the solutions

of a system of equations defined by the input matrix to better understand the rank of the matrix. For example, create a matrix and calculate its rank.

```
syms s
A = [1 s 0 2+s^2;
     s 1 2*s 0;
     2 0 2 1]
```

```
A =

$$\begin{pmatrix} 1 & s & 0 & s^2 + 2 \\ s & 1 & 2s & 0 \\ 2 & 0 & 2 & 1 \end{pmatrix}$$

```

```
k = rank(A)
```

```
k =
3
```

Here, `rank` returns the full rank, which is 3. However, this result may not apply to all values of `s`.

To better understand the rank of the matrix, solve the system of equations  $\mathbf{Ax} = 0$  by using `solve` and setting the `ReturnConditions` name-value argument to `true` to return any parameters and conditions of the solutions. According to the Rouché–Capelli theorem, a system of equations  $\mathbf{Ax} = \mathbf{b}$  with  $n$  variables in  $\mathbf{x}$  has at least one solution if and only if the rank of the coefficient matrix  $\mathbf{A}$  is equal to the rank of the augmented matrix  $[\mathbf{A}|\mathbf{b}]$ . If the system has solutions, they form an affine subspace of  $\mathbb{R}^n$  of dimension  $n - \text{rank}(\mathbf{A})$ . In other words, the solution is unique if and only if  $n = \text{rank}(\mathbf{A})$ . Otherwise, the system has infinitely many solutions (the system is *underdetermined*) and the solutions have  $k = n - \text{rank}(\mathbf{A})$  free parameters.

```
syms x [4 1]
sols = solve(A*x == 0,x,"ReturnConditions",true)

sols = struct with fields:
    x1: -2*z
    x2: -s*z
    x3: (3*z)/2
    x4: z
    parameters: z
    conditions: s ~= -1 & s ~= 1 & s ~= -1i & s ~= 1i
```

Here, `solve` returns nonunique solutions for the four variables in `x` with one free parameter, where specific conditions must be satisfied by the solutions of `x`. Therefore, if all the conditions for the variable `s` in the solutions are satisfied, the rank of the matrix is  $4 - 1 = 3$ .

Next, investigate the condition where `s = -1`. Substitute this value in the matrix `A` and solve another system of equations defined by the substituted matrix.

```
A2 = subs(A,s,-1)
```

```
A2 =

$$\begin{pmatrix} 1 & -1 & 0 & 3 \\ -1 & 1 & -2 & 0 \\ 2 & 0 & 2 & 1 \end{pmatrix}$$

```

```
sols2 = solve(A2*x == 0,x,"ReturnConditions",true)
```

```
sols2 = struct with fields:
    x1: -2*z
    x2: z
    x3: (3*z)/2
    x4: z
    parameters: z
    conditions: symtrue
```

Here, `solve` returns different solutions for the substituted matrix. The solutions have one free parameter, which is  $z$ , and has no additional conditions. Therefore, the rank of the substituted matrix where  $s = -1$  is  $4 - 1 = 3$ .

Next, investigate the condition where  $s = -1i$ . Substitute this value in the matrix  $A$  and solve another system of equations defined by the substituted matrix.

```
A3 = subs(A,s,-1i)
```

```
A3 =

$$\begin{pmatrix} 1 & -i & 0 & 1 \\ -i & 1 & -2i & 0 \\ 2 & 0 & 2 & 1 \end{pmatrix}$$

```

```
sols3 = solve(A3*x == 0,x,"ReturnConditions",true)
```

```
sols3 = struct with fields:
    x1: - z - z1/2
    x2: z*1i - (z1*1i)/2
    x3: z
    x4: z1
    parameters: [z z1]
    conditions: symtrue
```

Here, `solve` returns different solutions with two free parameters, which are  $z$  and  $z1$ , and no additional conditions. Therefore, the rank of this substituted matrix where  $s = -1i$  is  $4 - 2 = 2$ .

## Input Arguments

### A — Input

number | vector | matrix | symbolic number | symbolic vector | symbolic matrix

Input, specified as a number, vector, or matrix or a symbolic number, vector, or matrix.

## Limitations

- The `rank` function returns the exact rank of a symbolic matrix only if the rank calculation does not require any further simplification and does not have applied conditions or assumptions. If further simplification, conditions, or assumptions are needed, then `rank` can return an inaccurate result. In this case, to better understand the rank of the input matrix, you can investigate the solutions of a system of equations defined by the matrix using `solve`. For example, see “Solve System of Equations to Determine Rank of Matrix” on page 6-1251.

## Version History

Introduced before R2006a

## References

[1] Shafarevich, Igor R., and Alexey O. Remizov. *Linear Algebra and Geometry*. Berlin, Heidelberg: Springer, 2013.

## See Also

`eig` | `null` | `rref` | `solve`

# rat

Rational fraction approximation (continued fraction)

## Syntax

```
R = rat(X)
R = rat(X,tol)

[N,D] = rat(____)

____ = rat(____,Name,Value)
```

## Description

`R = rat(X)` returns the rational fraction approximation of `X` to within the default tolerance, `1.e-6*norm(X(:),1)`. The approximation is a character array containing the simple continued fraction on page 6-1260 with finite terms.

`R = rat(X,tol)` approximates `X` to within the tolerance, `tol`.

`[N,D] = rat(____)` returns two arrays, `N` and `D`, such that `N./D` approximates `X`. You can use this output syntax with any of the previous input syntaxes.

`____ = rat(____,Name,Value)` uses additional options specified by one or more `Name,Value` pair arguments to approximate `X`.

## Examples

### Find Rational Approximation of Irrational Number

Declare the irrational number  $\sqrt{3}$  as a symbolic number.

```
X = sqrt(sym(3))
```

```
X =  $\sqrt{3}$ 
```

Find the rational fraction approximation (truncated continued fraction) of this number. The resulting expression is a character vector.

```
R = rat(X)
```

```
R =
'2 + 1/(-4 + 1/(4 + 1/(-4 + 1/(4 + 1/(-4)))))'
```

Display the symbolic formula from the character vector `R`.

```
displayFormula(['A rational approximation of X is'; R])
```

```
A rational approximation of X is
```

$$2 + \frac{1}{-4 + \frac{1}{4 + \frac{1}{-4 + \frac{1}{4 + \frac{1}{-4}}}}}$$

### Approximate Value of $\pi$ with Different Precision

Represent the mathematical quantity  $\pi$  as a symbolic constant. The constant  $\pi$  is an irrational number.

```
X = sym(pi)
```

```
X =  $\pi$ 
```

Use `vpa` to show the decimal representation of  $\pi$  with 12 significant digits.

```
Xdec = vpa(X,12)
```

```
Xdec = 3.14159265359
```

Find the rational fraction approximation of  $\pi$  using the `rat` function with default tolerance. The resulting expression is a character vector.

```
R = rat(sym(pi))
```

```
R =  
'3 + 1/(7 + 1/(16))'
```

Use `str2sym` to turn the character vector into a single fractional number.

```
Q = str2sym(R)
```

```
Q =  
 $\frac{355}{113}$ 
```

Show the decimal representation of the fractional number 355/113. This approximation agrees with  $\pi$  to 6 decimal places.

```
Qdec = vpa(Q,12)
```

```
Qdec = 3.14159292035
```

You can specify a tolerance for additional accuracy in the approximation.

```
R = rat(sym(pi),1e-8)
```

```
R =  
'3 + 1/(7 + 1/(16 + 1/(-294)))'
```

```
Q = str2sym(R)
```

```
Q =  
 $\frac{104348}{33215}$ 
```



The resulting approximation, 104348/33215, agrees with  $\pi$  to 9 decimal places.

```
Qdec = vpa(Q,12)
Qdec = 3.14159265392
```

### Approximate Solution of Equation

Solve the equation  $\cos(x) + x^2 + x = 42$  using `vpasolve`. The solution is returned in decimal representation.

```
syms x
sol = vpasolve(cos(x) + x^2 + x == 42)
sol = 5.9274875551262136192212919837749
```

Approximate the solution as a continued fraction.

```
R = rat(sol)
R =
'6 + 1/(-14 + 1/(5 + 1/(-5)))'
```

To extract the coefficients in the denominator of the continued fraction, you can use the `regexp` function and convert them to a character array.

```
S = char(regexp(R, '(-*\d+', 'match'))
S = 3x4 char array
    '(-14'
    '(5  '
    '(-5  '
```

Return the result as a symbolic array.

```
coeffs = sym(S(:,2:end))
coeffs =
    (-14)
     5
    (-5)
```

Use `str2sym` to turn the continued fraction `R` into a single fractional number.

```
Q = str2sym(R)
Q =
    1962
    331
```

You can also return the numerator and denominator of the rational approximation by specifying two output arguments for the `rat` function.

```
[N,D] = rat(sol)
N = 1962
```

```
D = 331
```

### Find Rational Approximation of Golden Ratio

Define the golden ratio  $X = (1 + \sqrt{5})/2$  as a symbolic number.

```
X = (sym(1) + sqrt(5))/ 2
```

```
X =
```

$$\frac{\sqrt{5}}{2} + \frac{1}{2}$$

Find the rational approximation of  $X$  within a tolerance of  $1e-4$ .

```
R = rat(X,1e-4)
```

```
R =
```

$$'2 + 1/(-3 + 1/(3 + 1/(-3 + 1/(3 + 1/(-3))))))'$$

To return the rational approximation with 10 coefficients, set the 'Length' option to 10. This option ignores the specified tolerance in the approximation.

```
R10 = rat(X,1e-4, 'Length', 10)
```

```
R10 =
```

$$'2 + 1/(-3 + 1/(3 + 1/(-3 + 1/(3 + 1/(-3 + 1/(3 + 1/(-3 + 1/(3 + 1/(-3))))))))))'$$

To return the rational approximation with all positive coefficients, set the 'Positive' option to true.

```
Rpos = rat(X,1e-4, 'Positive', true)
```

```
Rpos =
```

$$'1 + 1/(1 + 1/(1 + 1/(1 + 1/(1 + 1/(1 + 1/(1 + 1/(1 + 1/(1 + 1/(1))))))))))'$$

## Input Arguments

### X — Input

number | vector | matrix | array | symbolic number | symbolic array

Input, specified as a number, vector, matrix, array, symbolic number, or symbolic array.

Data Types: single | double | sym

Complex Number Support: Yes

### tol — Tolerance

scalar

Tolerance, specified as a scalar. N and D approximate X, such that  $N./D - X < tol$ . The default tolerance is  $1e-6 \cdot \text{norm}(X(:), 1)$ .

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `'Length',5,'Positive',true`

### Length — Number of coefficients

positive integer

Number of coefficients or terms of the continued fraction, specified as a positive integer. Specifying this option overrides the tolerance argument `tol`.

Example: 5

### Positive — Option to return positive coefficients

logical 0 (false) (default) | logical value

Option to return positive coefficients, specified as a logical value (boolean). If you specify `true`, then `rat` returns a regular continued fraction expansion with all positive integers in the denominator.

Example: `true`

## Output Arguments

### R — Continued fraction

character array

Continued fraction, returned as a character array.

- If `X` is an array of  $m$  elements and all elements are real numbers, then `R` is returned as a character array with  $m$  rows.
- If `X` is an array of  $m$  elements that contains a complex number, then `R` is returned as a character array with  $2m+1$  rows. The first  $m$  rows of `R` represent the continued fraction expansion of the real parts of `X`, followed by `' +i* ... '` in the  $(m+1)$ -th row, and the last  $m$  rows represent the continued fraction expansions of the imaginary parts of `X`.

### N — Numerator

number | vector | matrix | array | symbolic number | symbolic array

Numerator, returned as a number, vector, matrix, array, symbolic number, or symbolic array. `N./D` approximates `X`.

### D — Denominator

number | vector | matrix | array | symbolic number | symbolic array

Denominator, returned as a number, vector, matrix, array, symbolic number, or symbolic array. `N./D` approximates `X`.

## Limitations

- You can only specify the `Name`, `Value` arguments, such as `'Length',5`, `'Positive',true`, if the array `X` contains a symbolic number or the data type of `X` is `sym`.

## More About

### Simple Continued Fraction

The `rat` function approximates each element of `X` by a simple continued fraction of the form

$$R = \frac{N}{D} = a_1 + \frac{1}{a_2 + \frac{1}{\ddots + \frac{1}{a_k}}}$$

with a finite number of integer terms  $a_1, a_2, \dots, a_k$ . The accuracy of the rational approximation increases with the number of terms.

## Version History

Introduced in R2020a

## See Also

`format` | `vpa` | `sym`

# rdivide, ./

Symbolic array right division

## Syntax

```
A./B
rdivide(A,B)
```

## Description

A./B divides A by B.

rdivide(A,B) is equivalent to A./B.

## Examples

### Divide Scalar by Matrix

Create a 2-by-3 matrix.

```
B = sym('b', [2 3])

B =
[ b1_1, b1_2, b1_3]
[ b2_1, b2_2, b2_3]
```

Divide the symbolic expression `sin(a)` by each element of the matrix B.

```
syms a
sin(a)./B

ans =
[ sin(a)/b1_1, sin(a)/b1_2, sin(a)/b1_3]
[ sin(a)/b2_1, sin(a)/b2_2, sin(a)/b2_3]
```

### Divide Matrix by Matrix

Create a 3-by-3 symbolic Hilbert matrix and a 3-by-3 diagonal matrix.

```
H = sym(hilb(3))
d = diag(sym([1 2 3]))

H =
[ 1, 1/2, 1/3]
[ 1/2, 1/3, 1/4]
[ 1/3, 1/4, 1/5]

d =
[ 1, 0, 0]
[ 0, 2, 0]
[ 0, 0, 3]
```

Divide `d` by `H` by using the elementwise right division operator `./`. This operator divides each element of the first matrix by the corresponding element of the second matrix. The dimensions of the matrices must be the same.

```
d./H
```

```
ans =  
[ 1, 0, 0]  
[ 0, 6, 0]  
[ 0, 0, 15]
```

### Divide Expression by Symbolic Function

Divide a symbolic expression by a symbolic function. The result is a symbolic function.

```
syms f(x)  
f(x) = x^2;  
f1 = (x^2 + 5*x + 6)./f
```

```
f1(x) =  
(x^2 + 5*x + 6)/x^2
```

## Input Arguments

### A — Input

symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix | symbolic array

Input, specified as a symbolic scalar variable, matrix variable, function, matrix function, expression, or vector, matrix, or array of symbolic scalar variables. Inputs `A` and `B` must be the same size unless one is a scalar. A scalar value expands into an array of the same size as the other input.

### B — Input

symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix | symbolic array

Input, specified as a symbolic scalar variable, matrix variable, function, matrix function, expression, or vector, matrix, or array of symbolic scalar variables. Inputs `A` and `B` must be the same size unless one is a scalar. A scalar value expands into an array of the same size as the other input.

## Version History

Introduced before R2006a

### R2022a: Array division of symbolic matrix functions

The `rdivide` function accepts input arguments of type `symfunmatrix`.

### R2021a: Array division of symbolic matrix variables

The `rdivide` function accepts input arguments of type `symmatrix`.

**See Also**

ctranspose | ldivide | minus | mldivide | mpower | mrdivide | mtimes | plus | power | times  
| transpose

## real

Real part of complex number

### Syntax

```
real(z)
```

### Description

`real(z)` returns the real part of `z`. If `z` is a matrix, `real` acts elementwise on `z`.

### Examples

#### Compute Real Part of Numeric Inputs

Find the real parts of these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
[real(2 + 3/2*i), real(sin(5*i)), real(2*exp(1 + i))]  
  
ans =  
    2.0000         0    2.9374
```

#### Compute Real Part of Symbolic Inputs

Compute the real parts of the numbers converted to symbolic objects:

```
[real(sym(2) + 3/2*i), real(4/(sym(1) + 3*i)), real(sin(sym(5)*i))]  
  
ans =  
[ 2, 2/5, 0]
```

Compute the real part of this symbolic expression:

```
real(2*exp(1 + sym(i)))  
  
ans =  
2*cos(1)*exp(1)
```

#### Compute Real Part of Symbolic Expressions

In general, `real` cannot extract the entire real parts from symbolic expressions containing variables. However, `real` can rewrite and sometimes simplify the input expression:

```
syms a x y  
real(a + 2)  
real(x + y*i)  
  
ans =  
real(a) + 2  
  
ans =  
real(x) - imag(y)
```



If you assign numeric values to these variables or specify that these variables are real, `real` can extract the real part of the expression:

```
syms a
a = 5 + 3*i;
real(a + 2)
```

```
ans =
    7
```

```
syms x y real
real(x + y*i)
```

```
ans =
    x
```

Clear the assumption that `x` and `y` are real by recreating them using `syms`:

```
syms x y
```

### Compute Real Part for Matrix Input

Find the real parts of the elements of matrix `A`:

```
syms x
A = [-1 + sym(i), sinh(x); exp(10 + sym(7)*i), exp(sym(pi)*i)];
real(A)
```

```
ans =
[ -1, real(sinh(x))]
[ cos(7)*exp(10), -1]
```

## Input Arguments

### **z** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## Tips

- Calling `real` for a number that is not a symbolic object invokes the MATLAB `real` function.

## Alternatives

You can compute the real part of `z` via the conjugate: `real(z) = (z + conj(z))/2`.

## Version History

Introduced before R2006a

**See Also**

`conj` | `imag` | `in` | `sign` | `signIm`

# rectangularPulse

Rectangular pulse function

## Syntax

```
r = rectangularPulse(a,b,x)
r = rectangularPulse(x)
```

## Description

`r = rectangularPulse(a,b,x)` returns the “Rectangular Pulse Function” on page 6-1270.

`r = rectangularPulse(x)` is the same as `rectangularPulse(-1/2,1/2,x)`.

## Examples

### Compute Rectangular Pulse Function

Compute the rectangular pulse function at coordinates -2, -1, 0, 1, and 2 with specified rising and falling edge at -1 and 1. Because these inputs are not symbolic objects, you get floating-point results.

```
r = rectangularPulse(-1,1,-2:2)

r = 1x5

    0    0.5000    1.0000    0.5000    0
```

Compute the rectangular pulse function for the same inputs in symbolic form.

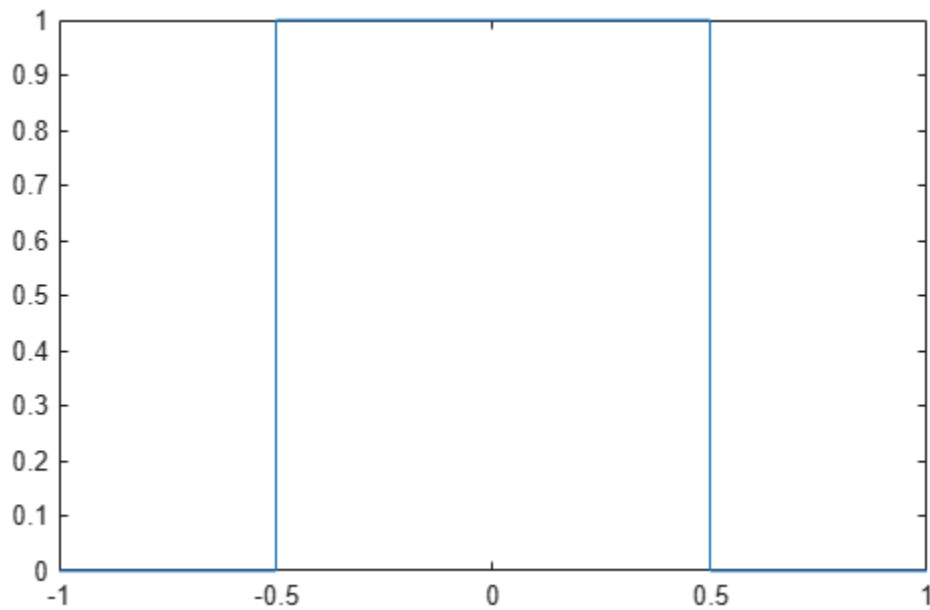
```
r = rectangularPulse(sym(-1),1,-2:2)

r =
    (0 1/2 1 1/2 0)
```

### Plot Rectangular Pulse Function

Plot the rectangular pulse function using `fplot`.

```
syms x
fplot(rectangularPulse(x), [-1 1])
```



### Edge Values of Rectangular Pulse

Show that if  $a < b$ , the rectangular pulse function for  $x = a$  and  $x = b$  equals  $1/2$ .

```
syms a b x
assume(a < b)
r = rectangularPulse(a,b,a)
```

```
r =
    1/2
```

```
r = rectangularPulse(a,b,b)
```

```
r =
    1/2
```

For further computations, remove the assumptions on the variables  $a$  and  $b$  by recreating them using `syms`.

```
syms a b
```

For  $a = b$ , the rectangular pulse function returns 0.

```
r = rectangularPulse(a,a,x)
```

```
r = 0
```

## Change Values of Rectangular Pulse at Rising and Falling Edges

The default value of the `rectangularPulse` function at the rising and falling edges is  $1/2$ .

```
syms a b
assume(a < b)
r = rectangularPulse(a,b,a)
```

```
r =
    1/2
```

```
r = rectangularPulse(a,b,b)
```

```
r =
    1/2
```

Another common value at these edges is 1. To change the value of `rectangularPulse` at the edges, use `sympref` to set the value of the `'HeavisideAtOrigin'` setting. Store the previous parameter value returned by `sympref`, so that you can restore it later.

```
oldparam = sympref('HeavisideAtOrigin',1);
```

Check the new value of `rectangularPulse` at the rising and falling edges.

```
r = rectangularPulse(a,b,a)
```

```
r = 1
```

```
r = rectangularPulse(a,b,b)
```

```
r = 1
```

The settings you set using `sympref` persist throughout your current and future MATLAB® sessions. To restore the previous value of `rectangularPulse` at the edges, use the value stored in `oldparam`.

```
sympref('HeavisideAtOrigin',oldparam);
```

Alternatively, you can restore the default value of `'HeavisideAtOrigin'` by using the `'default'` setting.

```
sympref('HeavisideAtOrigin','default');
```

## Relation Between Rectangular Pulse and Heaviside Function

When the rising or falling edge of `rectangularPulse` is at the infinity, then this function is the same as the Heaviside step function (the unit step function).

```
syms x
r = rectangularPulse(-Inf,0,x)
```

```
r = heaviside(-x)
```

```
r = rectangularPulse(0,Inf,x)
```

```
r = heaviside(x)
```

## Input Arguments

### **a — Starting position of rectangular pulse**

- 1/2 (default) | number | symbolic scalar variable

Starting position of rectangular pulse, specified as a number or a symbolic scalar variable. This argument specifies the rising edge of the rectangular pulse function.

### **b — Ending position of rectangular pulse**

1/2 (default) | number | symbolic scalar variable

Ending position of rectangular pulse, specified as a number or a symbolic scalar variable. This argument specifies the falling edge of the rectangular pulse function.

### **x — Input coordinates**

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input coordinates, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## More About

### **Rectangular Pulse Function**

- The rectangular pulse function returns 1 if the input coordinates  $x$  are within the specified interval range  $a < x < b$ .
- The rectangular pulse function, by default, returns  $\frac{1}{2}$  at the edges of the specified interval where  $x = a$  or  $x = b$  ( $a$  must not equal to  $b$ ). To change the default value of the rectangular pulse at the edges, you can use `sympref`. For more details, see “Change Values of Rectangular Pulse at Rising and Falling Edges” on page 6-1268.
- Otherwise, the rectangular pulse function returns 0.

The rectangular pulse function is also known as the rectangular function, normalized boxcar function, Heaviside Pi function, or gate function.

## Tips

- If  $a$  and  $b$  are variables or expressions with variables, `rectangularPulse` assumes that  $a < b$ . If  $a$  and  $b$  are numerical values, such that  $a > b$ , `rectangularPulse` throws an error.
- If  $a = b$ , `rectangularPulse` returns 0.

## Version History

Introduced in R2012b

## See Also

`dirac` | `heaviside` | `triangularPulse` | `sympref`

# reduceDAEIndex

Convert system of first-order differential algebraic equations to equivalent system of differential index 1

## Syntax

```
[newEqs,newVars] = reduceDAEIndex(eqs,vars)
[newEqs,newVars,R] = reduceDAEIndex(eqs,vars)
[newEqs,newVars,R,oldIndex] = reduceDAEIndex(eqs,vars)
```

## Description

`[newEqs,newVars] = reduceDAEIndex(eqs,vars)` converts a high-index system of first-order differential algebraic equations `eqs` to an equivalent system `newEqs` of differential index 1.

`reduceDAEIndex` keeps the original equations and variables and introduces new variables and equations. After conversion, `reduceDAEIndex` checks the differential index of the new system by calling `isLowIndexDAE`. If the index of `newEqs` is 2 or higher, then `reduceDAEIndex` issues a warning.

`[newEqs,newVars,R] = reduceDAEIndex(eqs,vars)` returns matrix `R` that expresses the new variables in `newVars` as derivatives of the original variables `vars`.

`[newEqs,newVars,R,oldIndex] = reduceDAEIndex(eqs,vars)` returns the differential index, `oldIndex`, of the original system of DAEs, `eqs`.

## Examples

### Reduce Differential Index of DAE System

Check if the following DAE system has a low (0 or 1) or high (>1) differential index. If the index is higher than 1, then use `reduceDAEIndex` to reduce it.

Create the following system of differential algebraic equations. Here, the symbolic functions  $x(t)$ ,  $y(t)$ , and  $z(t)$  represent the state variables of the system. Specify the equations as a vector of symbolic equations and the variables as a vector of symbolic function calls.

```
syms x(t) y(t) z(t) f(t)
eqs = [diff(x) == x + z;
       diff(y) == f(t);
       x == y];
vars = [x(t), y(t), z(t)];
```

Use `isLowIndexDAE` to check the differential index of the system. For this system, `isLowIndexDAE` returns 0 (false). This means that the differential index of the system is 2 or higher.

```
tf = isLowIndexDAE(eqs,vars)
```

```
tf = logical
    0
```

Use `reduceDAEIndex` to rewrite the system so that the differential index is 1. The new system has one additional state variable, `Dyt(t)`.

```
[newEqs,newVars] = reduceDAEIndex(eqs,vars)
```

```
newEqs =
    (
        \frac{\partial}{\partial t} x(t) - x(t) - z(t)
        Dyt(t) - f(t)
        x(t) - y(t)
        \frac{\partial}{\partial t} x(t) - Dyt(t)
    )
```

```
newVars =
    (
        x(t)
        y(t)
        z(t)
        Dyt(t)
    )
```

Check if the differential order of the new system is lower than 2.

```
tf = isLowIndexDAE(newEqs, newVars)
```

```
tf = logical
    1
```

### Reduce Differential Index and Return More Details

Reduce the differential index of a system that contains two second-order differential algebraic equation. Because the equations are second-order equations, first use `reduceDifferentialOrder` to rewrite the system to a system of first-order DAEs.

Create the following system of second-order DAEs. Here,  $x(t)$ ,  $y(t)$ , and  $F(t)$  are the state variables of the system. Specify the equations as a vector of symbolic equations and the variables as a vector of symbolic function calls.

```
syms t x(t) y(t) F(t) r g
eqs = [diff(x(t), t, t) == -F(t)*x(t);
       diff(y(t), t, t) == -F(t)*y(t) - g;
       x(t)^2 + y(t)^2 == r^2];
vars = [x(t), y(t), F(t)];
```

Rewrite this system so that all equations become first-order differential equations. The `reduceDifferentialOrder` function replaces the second-order DAE by two first-order expressions by introducing the new variables `Dxt(t)` and `Dyt(t)`. It also replaces the first-order equations by symbolic expressions.

```
[eqs,vars] = reduceDifferentialOrder(eqs,vars)
```



$$\text{eqs} = \begin{pmatrix} \frac{\partial}{\partial t} \text{Dxt}(t) + F(t) x(t) \\ \frac{\partial}{\partial t} \text{Dyt}(t) + g + F(t) y(t) \\ -r^2 + x(t)^2 + y(t)^2 \\ \text{Dxt}(t) - \frac{\partial}{\partial t} x(t) \\ \text{Dyt}(t) - \frac{\partial}{\partial t} y(t) \end{pmatrix}$$

$$\text{vars} = \begin{pmatrix} x(t) \\ y(t) \\ F(t) \\ \text{Dxt}(t) \\ \text{Dyt}(t) \end{pmatrix}$$

Use `reduceDAEIndex` to rewrite the system so that the differential index is 1.

`[eqs,vars,R,originalIndex] = reduceDAEIndex(eqs,vars)`

$$\text{eqs} = \begin{pmatrix} \text{Dxtt}(t) + F(t) x(t) \\ g + \text{Dytt}(t) + F(t) y(t) \\ -r^2 + x(t)^2 + y(t)^2 \\ \text{Dxt}(t) - \text{Dxt}_1(t) \\ \text{Dyt}(t) - \text{Dyt}_1(t) \\ 2 \text{Dxt}_1(t) x(t) + 2 \text{Dyt}_1(t) y(t) \\ 2 y(t) \frac{\partial}{\partial t} \text{Dyt}_1(t) + 2 \text{Dxt}_1(t)^2 + 2 \text{Dyt}_1(t)^2 + 2 \text{Dxt}_1(t) x(t) \\ \text{Dxtt}(t) - \text{Dxt}_1t(t) \\ \text{Dytt}(t) - \frac{\partial}{\partial t} \text{Dyt}_1(t) \\ \text{Dyt}_1(t) - \frac{\partial}{\partial t} y(t) \end{pmatrix}$$

$$\text{vars} = \begin{pmatrix} x(t) \\ y(t) \\ F(t) \\ \text{Dxt}(t) \\ \text{Dyt}(t) \\ \text{Dytt}(t) \\ \text{Dxtt}(t) \\ \text{Dxt}_1(t) \\ \text{Dyt}_1(t) \\ \text{Dxt}_1t(t) \end{pmatrix}$$

`R =`

$$\begin{pmatrix} \text{Dy}t(t) & \frac{\partial}{\partial t} \text{Dy}t(t) \\ \text{Dx}t(t) & \frac{\partial}{\partial t} \text{Dx}t(t) \\ \text{Dx}t_1(t) & \frac{\partial}{\partial t} x(t) \\ \text{Dy}t_1(t) & \frac{\partial}{\partial t} y(t) \\ \text{Dx}t1t(t) & \frac{\partial^2}{\partial t^2} x(t) \end{pmatrix}$$

```
originalIndex =
3
```

Use `reduceRedundancies` to shorten the system.

```
[eqs,vars] = reduceRedundancies(eqs,vars)
```

```
eqs =
```

$$\begin{pmatrix} \text{Dx}t(t) + F(t) x(t) \\ g + \text{Dy}t(t) + F(t) y(t) \\ -r^2 + x(t)^2 + y(t)^2 \\ 2 \text{Dx}t(t) x(t) + 2 \text{Dy}t(t) y(t) \\ 2 \text{Dx}t(t)^2 + 2 \text{Dy}t(t)^2 + 2 \text{Dx}t(t) x(t) + 2 \text{Dy}t(t) y(t) \\ \text{Dy}t(t) - \frac{\partial}{\partial t} \text{Dy}t(t) \\ \text{Dy}t(t) - \frac{\partial}{\partial t} y(t) \end{pmatrix}$$

```
vars =
```

$$\begin{pmatrix} x(t) \\ y(t) \\ F(t) \\ \text{Dx}t(t) \\ \text{Dy}t(t) \\ \text{Dy}t(t) \\ \text{Dx}t(t) \end{pmatrix}$$

## Input Arguments

### **eqs** — System of first-order DAEs

vector of symbolic equations | vector of symbolic expressions

System of first-order DAEs, specified as a vector of symbolic equations or expressions.

### **vars** — State variables

vector of symbolic functions | vector of symbolic function calls

State variables, specified as a vector of symbolic functions or function calls, such as `x(t)`.

Example: `[x(t), y(t)]`

## Output Arguments

### **newEqs — System of first-order DAEs of differential index 1**

column vector of symbolic expressions

System of first-order DAEs of differential index 1, returned as a column vector of symbolic expressions.

### **newVars — Extended set of variables**

column vector of symbolic function calls

Extended set of variables, returned as a column vector of symbolic function calls. This vector includes the original state variables `vars` followed by the generated variables that replace the second- and higher-order derivatives in `eqs`.

### **R — Relations between new and original variables**

symbolic matrix

Relations between new and original variables, returned as a symbolic matrix with two columns. The first column contains the new variables. The second column contains their definitions as derivatives of the original variables `vars`.

### **oldIndex — Differential index of original DAE system**

integer

Differential index of original DAE system, returned as an integer or NaN.

## Algorithms

The implementation of `reduceDAEIndex` uses the Pantelides algorithm. This algorithm reduces higher-index systems to lower-index systems by selectively adding differentiated forms of the original equations. The Pantelides algorithm can underestimate the differential index of a new system, and therefore, can fail to reduce the differential index to 1. In this case, `reduceDAEIndex` issues a warning and, for the syntax with four output arguments, returns the value of `oldIndex` as NaN. The `reduceDAEToODE` function uses more reliable, but slower Gaussian elimination. Note that `reduceDAEToODE` requires the DAE system to be semilinear.

## Version History

Introduced in R2014b

## See Also

`daeFunction` | `decic` | `findDecoupledBlocks` | `incidenceMatrix` | `isLowIndexDAE` | `massMatrixForm` | `odeFunction` | `reduceDAEToODE` | `reduceDifferentialOrder` | `reduceRedundancies`

## Topics

“Solve Differential Algebraic Equations (DAEs)” on page 3-66

## reduceDAEToODE

Convert system of first-order semilinear differential algebraic equations to equivalent system of differential index 0

### Syntax

```
newEqs = reduceDAEToODE(eqs,vars)
[newEqs,constraintEqs] = reduceDAEToODE(eqs,vars)
[newEqs,constraintEqs,oldIndex] = reduceDAEToODE(eqs,vars)
```

### Description

`newEqs = reduceDAEToODE(eqs,vars)` converts a high-index system of first-order semilinear algebraic equations `eqs` to an equivalent system of ordinary differential equations, `newEqs`. The differential index of the new system is 0, that is, the Jacobian of `newEqs` with respect to the derivatives of the variables in `vars` is invertible.

`[newEqs,constraintEqs] = reduceDAEToODE(eqs,vars)` returns a vector of constraint equations.

`[newEqs,constraintEqs,oldIndex] = reduceDAEToODE(eqs,vars)` returns the differential index `oldIndex` of the original system of semilinear DAEs, `eqs`.

### Examples

#### Convert DAE System to Implicit ODE System

Convert a system of differential algebraic equations (DAEs) to a system of implicit ordinary differential equations (ODEs).

Create the following system of differential algebraic equations. Here, the symbolic functions  $x(t)$ ,  $y(t)$ , and  $z(t)$  represent the state variables of the system. Specify the equations as a vector of symbolic equations and the variables as a vector of symbolic function calls.

```
syms x(t) y(t) z(t)
eqs = [diff(x,t) + x*diff(y,t) == y;
       x*diff(x,t) + x^2*diff(y,t) == sin(x);
       x^2 + y^2 == t*z]
```

```
eqs(t) =

$$\begin{pmatrix} x(t) \frac{\partial}{\partial t} y(t) + \frac{\partial}{\partial t} x(t) = y(t) \\ x(t)^2 \frac{\partial}{\partial t} y(t) + x(t) \frac{\partial}{\partial t} x(t) = \sin(x(t)) \\ x(t)^2 + y(t)^2 = t z(t) \end{pmatrix}$$

```

```
vars = [x(t), y(t), z(t)]
```

```
vars = (x(t) y(t) z(t))
```

Use `reduceDAEToODE` to rewrite the system so that the differential index is 0.

```
newEqs = reduceDAEToODE(eqs,vars)
```

```
newEqs =
```

$$\begin{pmatrix} \sigma_1 - y(t) + \frac{\partial}{\partial t} x(t) \\ (\cos(x(t)) - y(t)) \frac{\partial}{\partial t} x(t) - \sigma_1 \\ z(t) - 2x(t) \frac{\partial}{\partial t} x(t) - 2y(t) \frac{\partial}{\partial t} y(t) + t \frac{\partial}{\partial t} z(t) \end{pmatrix}$$

where

$$\sigma_1 = x(t) \frac{\partial}{\partial t} y(t)$$

### Reduce System and Return More Details

Check if the following DAE system has a low (0 or 1) or high (>1) differential index. If the index is higher than 1, first try to reduce the index by using `reduceDAEIndex` and then by using `reduceDAEToODE`.

Create the system of differential algebraic equations. Here, the functions `x1(t)`, `x2(t)`, and `x3(t)` represent the state variables of the system. The system also contains the functions `q1(t)`, `q2(t)`, and `q3(t)`. These functions do not represent state variables. Specify the equations as a vector of symbolic equations and the variables as a vector of symbolic function calls.

```
syms x1(t) x2(t) x3(t) q1(t) q2(t) q3(t)
eqs = [diff(x2) == q1 - x1;
       diff(x3) == q2 - 2*x2 - t*(q1-x1);
       q3 - t*x2 - x3];
vars = [x1(t), x2(t), x3(t)];
```

Use `isLowIndexDAE` to check the differential index of the system. For this system, `isLowIndexDAE` returns 0 (false). This means that the differential index of the system is 2 or higher.

```
tf = isLowIndexDAE(eqs,vars)
```

```
tf = logical
    0
```

Use `reduceDAEIndex` as your first attempt to rewrite the system so that the differential index is 1. For this system, `reduceDAEIndex` issues a warning because it cannot reduce the differential index of the system to 0 or 1.

```
[newEqs,newVars] = reduceDAEIndex(eqs,vars)
```

```
Warning: Index of reduced DAEs is larger than 1.
```

```
newEqs =
```

$$\begin{pmatrix} \frac{\partial}{\partial t} x_2(t) - q_1(t) + x_1(t) \\ Dx3t(t) - q_2(t) + 2 x_2(t) + t (q_1(t) - x_1(t)) \\ q_3(t) - x_3(t) - t x_2(t) \\ \frac{\partial}{\partial t} q_3(t) - x_2(t) - t \frac{\partial}{\partial t} x_2(t) - Dx3t(t) \end{pmatrix}$$

$$\text{newVars} = \begin{pmatrix} x_1(t) \\ x_2(t) \\ x_3(t) \\ Dx3t(t) \end{pmatrix}$$

If `reduceDAEIndex` cannot reduce the semilinear system so that the index is 0 or 1, try using `reduceDAEToODE`. This function can be much slower, therefore it is not recommended as a first choice. Use the syntax with two output arguments to also return the constraint equations.

$$[\text{newEqs}, \text{constraintEqs}] = \text{reduceDAEToODE}(\text{eqs}, \text{vars})$$

$$\text{newEqs} = \begin{pmatrix} \frac{\partial}{\partial t} x_2(t) - q_1(t) + x_1(t) \\ \frac{\partial}{\partial t} x_3(t) - q_2(t) + 2 x_2(t) + t q_1(t) - t x_1(t) \\ \frac{\partial}{\partial t} x_1(t) - \frac{\partial}{\partial t} q_1(t) + \frac{\partial^2}{\partial t^2} q_2(t) - \frac{\partial^3}{\partial t^3} q_3(t) \end{pmatrix}$$

$$\text{constraintEqs} = \begin{pmatrix} \frac{\partial}{\partial t} q_3(t) - q_2(t) + x_2(t) \\ x_3(t) - q_3(t) + t x_2(t) \\ -\frac{\partial^2}{\partial t^2} q_3(t) + \frac{\partial}{\partial t} q_2(t) - q_1(t) + x_1(t) \end{pmatrix}$$

Use the syntax with three output arguments to return the new equations, constraint equations, and the differential index of the original system, `eqs`.

$$[\text{newEqs}, \text{constraintEqs}, \text{oldIndex}] = \text{reduceDAEToODE}(\text{eqs}, \text{vars})$$

$$\text{newEqs} = \begin{pmatrix} \frac{\partial}{\partial t} x_2(t) - q_1(t) + x_1(t) \\ \frac{\partial}{\partial t} x_3(t) - q_2(t) + 2 x_2(t) + t q_1(t) - t x_1(t) \\ \frac{\partial}{\partial t} x_1(t) - \frac{\partial}{\partial t} q_1(t) + \frac{\partial^2}{\partial t^2} q_2(t) - \frac{\partial^3}{\partial t^3} q_3(t) \end{pmatrix}$$

$$\text{constraintEqs} = \begin{pmatrix} \frac{\partial}{\partial t} q_3(t) - q_2(t) + x_2(t) \\ x_3(t) - q_3(t) + t x_2(t) \\ -\frac{\partial^2}{\partial t^2} q_3(t) + \frac{\partial}{\partial t} q_2(t) - q_1(t) + x_1(t) \end{pmatrix}$$

```
oldIndex =
3
```

## Input Arguments

### **eqs — System of first-order semilinear DAEs**

vector of symbolic equations | vector of symbolic expressions

System of first-order semilinear DAEs, specified as a vector of symbolic equations or expressions.

### **vars — State variables**

vector of symbolic functions | vector of symbolic function calls

State variables, specified as a vector of symbolic functions or function calls, such as  $x(t)$ .

Example:  $[x(t), y(t)]$  or  $[x(t); y(t)]$

## Output Arguments

### **newEqs — System of implicit ordinary differential equations**

column vector of symbolic expressions

System of implicit ordinary differential equations, returned as a column vector of symbolic expressions. The differential index of this system is 0.

### **constraintEqs — Constraint equations encountered during system reduction**

column vector of symbolic expressions

Constraint equations encountered during system reduction, returned as a column vector of symbolic expressions. These expressions depend on the variables `vars`, but not on their derivatives. The constraints are conserved quantities of the differential equations in `newEqs`, meaning that the time derivative of each constraint vanishes modulo the equations in `newEqs`.

You can use these equations to determine consistent initial conditions for the DAE system.

### **oldIndex — Differential index of original DAE system eqs**

integer

Differential index of original DAE system `eqs`, returned as an integer.

## Algorithms

The implementation of `reduceDAEToODE` is based on Gaussian elimination. This algorithm is more reliable than the Pantelides algorithm used by `reduceDAEIndex`, but it can be much slower.

## Version History

**Introduced in R2014b**

**See Also**

`daeFunction` | `decic` | `findDecoupledBlocks` | `incidenceMatrix` | `isLowIndexDAE` |  
`massMatrixForm` | `odeFunction` | `reduceDAEIndex` | `reduceDifferentialOrder` |  
`reduceRedundancies`

**Topics**

“Solve Semilinear DAE System” on page 3-75



# reduceDifferentialOrder

Reduce system of higher-order differential equations to equivalent system of first-order differential equations

## Syntax

```
[newEqs,newVars] = reduceDifferentialOrder(eqs,vars)
[newEqs,newVars,R] = reduceDifferentialOrder(eqs,vars)
```

## Description

`[newEqs,newVars] = reduceDifferentialOrder(eqs,vars)` rewrites a system of higher-order differential equations `eqs` as a system of first-order differential equations `newEqs` by substituting derivatives in `eqs` with new variables. Here, `newVars` consists of the original variables `vars` augmented with these new variables.

`[newEqs,newVars,R] = reduceDifferentialOrder(eqs,vars)` returns the matrix `R` that expresses the new variables in `newVars` as derivatives of the original variables `vars`.

## Examples

### Reduce Differential Order of DAE System

Reduce a system containing higher-order DAEs to a system containing only first-order DAEs.

Create the system of differential equations, which includes a second-order expression. Here,  $x(t)$  and  $y(t)$  are the state variables of the system, and  $c1$  and  $c2$  are parameters. Specify the equations and variables as two symbolic vectors: equations as a vector of symbolic equations, and variables as a vector of symbolic function calls.

```
syms x(t) y(t) c1 c2
eqs = [diff(x(t),t,t) + sin(x(t)) + y(t) == c1*cos(t),...
       diff(y(t),t) == c2*x(t)];
vars = [x(t),y(t)];
```

Rewrite this system so that all equations become first-order differential equations. The `reduceDifferentialOrder` function replaces the higher-order DAE by first-order expressions by introducing the new variable  $Dxt(t)$ . It also represents all equations as symbolic expressions.

```
[newEqs,newVars] = reduceDifferentialOrder(eqs,vars)
```

```
newEqs =

$$\begin{pmatrix} \frac{\partial}{\partial t} Dxt(t) + \sin(x(t)) + y(t) - c_1 \cos(t) \\ \frac{\partial}{\partial t} y(t) - c_2 x(t) \\ Dxt(t) - \frac{\partial}{\partial t} x(t) \end{pmatrix}$$

```

```
newVars =
```

$$\begin{pmatrix} x(t) \\ y(t) \\ Dxt(t) \end{pmatrix}$$

### Show Relations Between Generated and Original Variables

Reduce a system containing a second- and a third-order expression to a system containing only first-order DAEs. In addition, return a matrix that expresses the variables generated by `reduceDifferentialOrder` via the original variables of this system.

Create a system of differential equations, which includes a second- and a third-order expression. Here,  $x(t)$  and  $y(t)$  are the state variables of the system. Specify the equations and variables as two symbolic vectors: equations as a vector of symbolic equations, and variables as a vector of symbolic function calls.

```
syms x(t) y(t) f(t)
eqs = [diff(x(t),t,t) == diff(f(t),t,t,t), diff(y(t),t,t,t) == diff(f(t),t,t)];
vars = [x(t),y(t)];
```

Call `reduceDifferentialOrder` with three output arguments. This syntax returns matrix  $R$  with two columns: the first column contains the new variables, and the second column expresses the new variables as derivatives of the original variables,  $x(t)$  and  $y(t)$ .

```
[newEqs,newVars,R] = reduceDifferentialOrder(eqs,vars)
```

```
newEqs =
```

$$\begin{pmatrix} \frac{\partial}{\partial t} Dxt(t) - \frac{\partial^3}{\partial t^3} f(t) \\ \frac{\partial}{\partial t} Dytt(t) - \frac{\partial^2}{\partial t^2} f(t) \\ Dxt(t) - \frac{\partial}{\partial t} x(t) \\ Dyt(t) - \frac{\partial}{\partial t} y(t) \\ Dytt(t) - \frac{\partial}{\partial t} Dyt(t) \end{pmatrix}$$

```
newVars =
```

$$\begin{pmatrix} x(t) \\ y(t) \\ Dxt(t) \\ Dyt(t) \\ Dytt(t) \end{pmatrix}$$

```
R =
```

$$\begin{pmatrix} Dxt(t) & \frac{\partial}{\partial t} x(t) \\ Dyt(t) & \frac{\partial}{\partial t} y(t) \\ Dytt(t) & \frac{\partial^2}{\partial t^2} y(t) \end{pmatrix}$$

## Input Arguments

### **eqs — System containing higher-order differential equations**

vector of symbolic equations | vector of symbolic expressions

System containing higher-order differential equations, specified as a vector of symbolic equations or expressions.

### **vars — Variables of original differential equations**

vector of symbolic functions | vector of symbolic function calls

Variables of original differential equations, specified as a vector of symbolic functions, or function calls, such as  $x(t)$ .

Example:  $[x(t), y(t)]$

## Output Arguments

### **newEqs — System of first-order differential equations**

column vector of symbolic expressions

System of first-order differential equations, returned as a column vector of symbolic expressions.

### **newVars — Extended set of variables**

column vector of symbolic function calls

Extended set of variables, returned as a column vector of symbolic function calls. This vector includes the original state variables `vars` followed by the generated variables that replace the higher-order derivatives in `eqs`.

### **R — Relations between new and original variables**

symbolic matrix

Relations between new and original variables, returned as a symbolic matrix with two columns. The first column contains the new variables `newVars`. The second column contains their definition as derivatives of the original variables `vars`.

## Version History

**Introduced in R2014b**

## See Also

`findDecoupledBlocks` | `daeFunction` | `decic` | `incidenceMatrix` | `isLowIndexDAE` | `massMatrixForm` | `odeFunction` | `reduceDAEIndex` | `reduceDAETooDE` | `reduceRedundancies`

## Topics

“Solve Differential Algebraic Equations (DAEs)” on page 3-66

## reduceRedundancies

Simplify system of first-order differential algebraic equations by eliminating redundant equations and variables

### Syntax

```
[newEqs,newVars] = reduceRedundancies(eqs,vars)
[newEqs,newVars,R] = reduceRedundancies(eqs,vars)
```

### Description

[newEqs,newVars] = reduceRedundancies(eqs,vars) eliminates redundant equations and variables from the system of first-order differential algebraic equations (DAEs) eqs. The input argument vars specifies the state variables of the system.

reduceRedundancies returns the new DAE system as a column vector newEqs and the reduced state variables as a column vector newVars. Each element of newEqs represents an equation with right side equal to zero.

[newEqs,newVars,R] = reduceRedundancies(eqs,vars) returns a structure array R containing information on the eliminated equations and variables.

### Examples

#### Reduce DAE System by Removing Redundant Equations

Simplify a system of five differential algebraic equations (DAEs) in four state variables to a system of two equations in two state variables.

Create the following system of five DAEs in four state variables  $x_1(t)$ ,  $x_2(t)$ ,  $x_3(t)$ , and  $x_4(t)$ . The system also contains symbolic parameters  $a_1$ ,  $a_2$ ,  $a_3$ ,  $a_4$ ,  $b$ ,  $c$ , and the function  $f(t)$  that are not state variables.

```
syms x1(t) x2(t) x3(t) x4(t) a1 a2 a3 a4 b c f(t)
eqs = [a1*diff(x1(t),t)+a2*diff(x2(t),t) == b*x4(t),
       a3*diff(x2(t),t)+a4*diff(x3(t),t) == c*x4(t),
       x1(t) == 2*x2(t),
       x4(t) == f(t),
       f(t) == sin(t)];
vars = [x1(t),x2(t),x3(t),x4(t)];
```

Use reduceRedundancies to eliminate redundant equations and corresponding state variables.

```
[newEqs,newVars] = reduceRedundancies(eqs,vars)
```

```
newEqs =
```

$$\begin{pmatrix} a_1 \frac{\partial}{\partial t} x_1(t) + \frac{a_2}{2} \frac{\partial}{\partial t} x_1(t) - b f(t) \\ \frac{a_3}{2} \frac{\partial}{\partial t} x_1(t) + a_4 \frac{\partial}{\partial t} x_3(t) - c f(t) \end{pmatrix}$$

```
newVars =

$$\begin{pmatrix} x_1(t) \\ x_3(t) \end{pmatrix}$$

```

### Specify Input Order of State Variables

Specify input order of the state variables to choose which variables are being returned when eliminating DAEs.

Create a system of four DAEs in four state variables  $V_{ac}(t)$ ,  $V_1(t)$ ,  $V_2(t)$ , and  $I(t)$ . The system also contains symbolic parameters  $L$ ,  $R$ , and  $V_0$ .

```
syms V_ac(t) V1(t) V2(t) I(t) L R V0
eqs = [V_ac(t) == V1(t) + V2(t),
       V1(t) == I(t)*R,
       V2(t) == L*diff(I(t),t),
       V_ac(t) == V0*cos(t)]
```

```
eqs =

$$\begin{pmatrix} V_{ac}(t) = V_1(t) + V_2(t) \\ V_1(t) = R I(t) \\ V_2(t) = L \frac{\partial}{\partial t} I(t) \\ V_{ac}(t) = V_0 \cos(t) \end{pmatrix}$$

```

```
vars = [V_ac(t),I(t),V1(t),V2(t)]
```

```
vars = (V_ac(t) I(t) V1(t) V2(t))
```

Use `reduceRedundancies` to eliminate redundant equations and variables. `reduceRedundancies` prioritizes to keep the state variables in the vector `vars` starting from the first element.

```
[newEqs,newVars] = reduceRedundancies(eqs,vars)
```

```
newEqs =

$$-L \frac{\partial}{\partial t} I(t) - R I(t) + V_0 \cos(t)$$

```

```
newVars = I(t)
```

Here, `reduceRedundancies` returns a reduced equation in term of the variable  $I(t)$ .

When multiple ways of reducing the DAEs exist, specify a different input order of the state variables to choose which variables are being returned. Specify another vector that contains a different order of the state variables. Eliminate the DAEs again.

```
vars2 = [V_ac(t),V1(t),V2(t),I(t)]
```

```
vars2 = (V_ac(t) V1(t) V2(t) I(t))
```

```
[newEqs,newVars] = reduceRedundancies(eqs,vars2)
```

```
newEqs =
```

$$-\frac{L \frac{\partial}{\partial t} V_1(t) + R V_1(t) - R V_0 \cos(t)}{R}$$

`newVars = V1(t)`

Here, `reduceRedundancies` returns a reduced equation in term of the state variable `V1(t)`.

### Obtain Information About Eliminated Equations

Declare three output arguments when calling `reduceRedundancies` to simplify a system of equations and return information about the eliminated equations.

Create the following system of five differential algebraic equations (DAEs) in four state variables `x1(t)`, `x2(t)`, `x3(t)`, and `x4(t)`. The system also contains symbolic parameters `a1`, `a2`, `a3`, `a4`, `b`, `c`, and the function `f(t)` that are not state variables.

```
syms x1(t) x2(t) x3(t) x4(t) a1 a2 a3 a4 b c f(t)
eqs = [a1*diff(x1(t),t)+a2*diff(x2(t),t) == b*x4(t),
       a3*diff(x2(t),t)+a4*diff(x3(t),t) == c*x4(t),
       x1(t) == 2*x2(t),
       x4(t) == f(t),
       f(t) == sin(t)];
vars = [x1(t),x2(t),x3(t),x4(t)];
```

Call `reduceRedundancies` with three output arguments.

```
[newEqs,newVars,R] = reduceRedundancies(eqs,vars)
```

```
newEqs =

$$\begin{pmatrix} a_1 \frac{\partial}{\partial t} x_1(t) + \frac{a_2}{2} \frac{\partial}{\partial t} x_1(t) - b f(t) \\ \frac{a_3}{2} \frac{\partial}{\partial t} x_1(t) + a_4 \frac{\partial}{\partial t} x_3(t) - c f(t) \end{pmatrix}$$

```

```
newVars =

$$\begin{pmatrix} x_1(t) \\ x_3(t) \end{pmatrix}$$

```

```
R = struct with fields:
    solvedEquations: [2x1 sym]
    constantVariables: [x4(t) f(t)]
    replacedVariables: [x2(t) x1(t)/2]
    otherEquations: f(t) - sin(t)
```

The function `reduceRedundancies` returns information about eliminated equations to `R`. Here, `R` is a structure array with four fields.

The `solvedEquations` field contains the equations that are eliminated by `reduceRedundancies`. The eliminated equations contain those state variables from `vars` that do not appear in `newEqs`. The right side of each eliminated equation is equal to zero.

```
R1 = R.solvedEquations
```

$$R1 = \begin{pmatrix} x_4(t) - f(t) \\ x_1(t) - 2 x_2(t) \end{pmatrix}$$

The `constantVariables` field contains a matrix with two columns. The first column contains those state variables from `vars` that `reduceRedundancies` replaced by constant values. The second column contains the corresponding constant values.

$$R2 = R.constantVariables$$

$$R2 = \begin{pmatrix} x_4(t) & f(t) \end{pmatrix}$$

The `replacedVariables` field contains a matrix with two columns. The first column contains those state variables from `vars` that `reduceRedundancies` replaced by expressions in terms of other variables. The second column contains the corresponding values of the eliminated variables.

$$R3 = R.replacedVariables$$

$$R3 = \begin{pmatrix} x_2(t) & \frac{x_1(t)}{2} \end{pmatrix}$$

The `otherEquations` field contains those equations from `eqs` that do not contain any of the state variables `vars`.

$$R4 = R.otherEquations$$

$$R4 = f(t) - \sin(t)$$

## Input Arguments

### **eqs — System of first-order DAEs**

vector of symbolic equations | vector of symbolic expressions

System of first-order DAEs, specified as a vector of symbolic equations or expressions.

The relation operator `==` defines symbolic equations. If you specify the element of `eqs` as a symbolic expression without a right side, then a symbolic equation with right side equal to zero is assumed.

### **vars — State variables**

vector of symbolic functions | vector of symbolic function calls

State variables, specified as a vector of symbolic functions or function calls, such as `x(t)`.

The input order of the state variables determines which reduced variables are being returned. If multiple ways of reducing the DAEs exist, then `reduceRedundancies` prioritizes to keep the state variables in `vars` starting from the first element.

Example: `[x(t), z(t), y(t)]`

## Output Arguments

### **newEqs — System of first-order DAEs**

column vector of symbolic expressions

System of first-order DAEs, returned as a column vector of symbolic expressions. Each element of `newEqs` represents an equation with right side equal to zero.

**newVars — Reduced set of variables**

column vector of symbolic function calls

Reduced set of variables, returned as a column vector of symbolic function calls.

**R — Information about eliminated variables**

structure array

Information about eliminated variables, returned as a structure array containing four fields. To access this information, use:

- `R.solvedEquations` to return a symbolic column vector of all equations that `reduceRedundancies` used to replace those state variables that do not appear in `newEqs`.
- `R.constantVariables` to return a matrix with the following two columns. The first column contains those original state variables of the vector `vars` that were eliminated and replaced by constant values. The second column contains the corresponding constant values.
- `R.replacedVariables` to return a matrix with the following two columns. The first column contains those original state variables of the vector `vars` that were eliminated and replaced in terms of other variables. The second column contains the corresponding values of the eliminated variables.
- `R.otherEquations` to return a column vector containing all original equations `eqs` that do not contain any of the input variables `vars`.

## Version History

Introduced in R2014b

### See Also

`daeFunction` | `decic` | `findDecoupledBlocks` | `incidenceMatrix` | `isLowIndexDAE` | `massMatrixForm` | `odeFunction` | `reduceDAEIndex` | `reduceDAEToODE` | `reduceDifferentialOrder`

### Topics

“Solve Differential Algebraic Equations (DAEs)” on page 3-66



# release

Evaluate integrals

## Syntax

`release(expr)`

## Description

`release(expr)` evaluates the integrals in the expression `expr`. The `release` function ignores the 'Hold' option in the `int` function when the integrals are defined.

## Examples

### Unevaluated Integral

Define a symbolic call to an integral  $\int \cos(x) dx$  without evaluating it. Set the 'Hold' option to true when defining the integral using the `int` function.

```
syms x
F = int(cos(x), 'Hold', true)
```

F =

$$\int \cos(x) dx$$

Use `release` to evaluate the integral by ignoring the 'Hold' option.

```
G = release(F)
```

```
G = sin(x)
```

### Unevaluated Integral and Integration by Parts

Find the integral of  $\int x e^x dx$ .

Define the integral without evaluating it by setting the `Hold` option to `true`.

```
syms x g(y)
F = int(x*exp(x), Hold=true)
```

F =

$$\int x e^x dx$$

You can apply integration by parts to `F` by using the `integrateByParts` function. Use `exp(x)` as the differential to be integrated.

```
G = integrateByParts(F, exp(x))
```

$$G = x e^x - \int e^x dx$$

To evaluate the integral in G, use the `release` function to ignore the `Hold` option.

```
Gcalc = release(G)
```

$$Gcalc = x e^x - e^x$$

Compare the result to the integration result returned by `int` without setting the `Hold` option.

```
Fcalc = int(x*exp(x))
```

$$Fcalc = e^x (x - 1)$$

### Integration by Substitution

Find the integral of  $\int \cos(\log(x)) dx$  using integration by substitution.

Define the integral without evaluating it by setting the '`Hold`' option to `true`.

```
syms x t
F = int(cos(log(x)), 'Hold', true)
```

$$F = \int \cos(\log(x)) dx$$

Substitute the expression  $\log(x)$  with `t`.

```
G = changeIntegrationVariable(F, log(x), t)
```

$$G = \int e^t \cos(t) dt$$

To evaluate the integral in G, use the `release` function to ignore the '`Hold`' option.

```
H = release(G)
```

$$H = \frac{e^t (\cos(t) + \sin(t))}{2}$$

Restore  $\log(x)$  in place of `t`.

```
H = simplify(subs(H, t, log(x)))
```

$$H = \frac{\sqrt{2} x \sin\left(\frac{\pi}{4} + \log(x)\right)}{2}$$

Compare the result to the integration result returned by `int` without setting the '`Hold`' option to `true`.

```
Fcalc = int(cos(log(x)))
```

```
Fcalc =
```

$$\frac{\sqrt{2} \times \sin\left(\frac{\pi}{4} + \log(x)\right)}{2}$$

## Input Arguments

**expr — Expression containing integrals**

symbolic expression | symbolic function | symbolic vector | symbolic matrix

Expression containing integrals, specified as a symbolic expression, function, vector, or matrix.

## Version History

**Introduced in R2019b**

## See Also

integrateByParts | int | diff | changeIntegrationVariable

## rem

Remainder after division

### Syntax

`rem(a,b)`

### Description

`rem(a,b)` finds the remainder after division. If  $b \neq 0$ , then  $\text{rem}(a,b) = a - \text{fix}(a/b)*b$ . If  $b = 0$  or  $b = \text{Inf}$  or  $b = -\text{Inf}$ , then `rem` returns `NaN`.

The `rem` function does not support complex numbers: all values must be real numbers.

To find the remainder after division of polynomials, use `quoem`.

### Examples

#### Divide Integers by Integers

Find the remainder after division in case both the dividend and divisor are integers.

Find the modulus after division for these numbers.

```
[rem(sym(27), 4), rem(sym(27), -4), rem(sym(-27), 4), rem(sym(-27), -4)]
```

```
ans =  
[ 3, 3, -3, -3]
```

#### Divide Rationals by Integers

Find the remainder after division in case the dividend is a rational number, and the divisor is an integer.

Find the remainder after division for these numbers.

```
[rem(sym(22/3), 5), rem(sym(1/2), -7), rem(sym(27/6), -11)]
```

```
ans =  
[ 7/3, 1/2, 9/2]
```

#### Divide Elements of Matrices

For vectors and matrices, `rem` finds the remainder after division element-wise. Nonscalar arguments must be the same size.

Find the remainder after division for the elements of these two matrices.

```
A = sym([27, 28; 29, 30]);  
B = sym([2, 3; 4, 5]);  
rem(A,B)
```

```
ans =  
[ 1, 1]  
[ 1, 0]
```

Find the remainder after division for the elements of matrix A and the value 9. Here, `rem` expands 9 into the 2-by-2 matrix with all elements equal to 9.

```
rem(A,9)
```

```
ans =  
[ 0, 1]  
[ 2, 3]
```

## Input Arguments

### **a — Dividend (numerator)**

number | symbolic number | vector | matrix

Dividend (numerator), specified as a number, symbolic number, or a vector or matrix of numbers or symbolic numbers.

### **b — Divisor (denominator)**

number | symbolic number | vector | matrix

Divisor (denominator), specified as a number, symbolic number, or a vector or matrix of numbers or symbolic numbers.

## Tips

- Calling `rem` for numbers that are not symbolic objects invokes the MATLAB `rem` function.
- All nonscalar arguments must be the same size. If one input arguments is nonscalar, then `mod` expands the scalar into a vector or matrix of the same size as the nonscalar argument, with all elements equal to the corresponding scalar.

## Version History

**Introduced before R2006a**

## See Also

`mod` | `quorem`

## removeUnit

Remove unit

### Syntax

```
removeUnit(unit)
```

### Description

`removeUnit(unit)` removes the symbolic unit `unit`. You can remove only user-defined units created with `newUnit`. You cannot remove predefined units. If `unit` is a vector, `removeUnit` removes all units in `unit`.

### Examples

#### Remove Unit

Remove units you define by using `removeUnit`. Create the unit `warp3`, use the unit in calculations, and then remove the unit.

Define the unit `warp3` as 3 times the speed of light.

```
u = symunit;  
warp3 = newUnit('warp3',3*u.c_0)
```

```
warp3 =  
[warp3]
```

Convert  $1e10$  meter per second to `u.warp3`.

```
speed = rewrite(1e10*u.m/u.s,u.warp3)
```

```
speed =  
(5000000000/449688687)*[warp3]
```

After calculations, remove the unit `u.warp3` by using `removeUnit`.

```
removeUnit(u.warp3)
```

Conversion to `u.warp3` now throws an error.

### Input Arguments

#### **unit** — Unit name

symbolic unit | vector of symbolic units

Unit name, specified as a symbolic unit or a vector of symbolic units.

## Version History

Introduced in R2017b

### See Also

`checkUnits` | `isUnit` | `newUnit` | `newUnitSystem` | `symunit`

### Topics

"Units of Measurement Tutorial" on page 2-47

"Unit Conversions and Unit Systems" on page 2-53

"Units and Unit Systems List" on page 2-60

### External Websites

The International System of Units (SI)

## reshape

Reshape symbolic array

### Syntax

```
reshape(A,n1,n2)
reshape(A,n1,...,nM)
reshape(A,...,[],...)
reshape(A,sz)
```

### Description

`reshape(A,n1,n2)` returns the  $n1$ -by- $n2$  matrix, which has the same elements as  $A$ . The elements are taken column-wise from  $A$  to fill in the elements of the  $n1$ -by- $n2$  matrix.

`reshape(A,n1,...,nM)` returns the  $n1$ -by-...-by- $nM$  array, which has the same elements as  $A$ . The elements are taken column-wise from  $A$  to fill in the elements of the  $n1$ -by-...-by- $nM$  array.

`reshape(A,...,[],...)` lets you represent a size value with the placeholder `[]` while calculating the magnitude of that size value automatically. For example, if  $A$  has size 2-by-6, then `reshape(A,4,[],...)` returns a 4-by-3 array.

`reshape(A,sz)` reshapes  $A$  into an array with size specified by  $sz$ , where  $sz$  is a vector.

### Examples

#### Reshape Symbolic Row Vector into Column Vector

Reshape  $V$ , which is a 1-by-4 row vector, into the 4-by-1 column vector  $Y$ . Here,  $V$  and  $Y$  must have the same number of elements.

Create the vector  $V$ .

```
syms f(x) y
V = [3 f(x) -4 y]

V =
[ 3, f(x), -4, y]
```

Reshape  $V$  into  $Y$ .

```
Y = reshape(V,4,1)

Y =
     3
f(x)
    -4
     y
```

Alternatively, use  $Y = V.'$  where  $.$  is the nonconjugate transpose.



## Reshape Symbolic Matrix

Reshape the 2-by-6 symbolic matrix **M** into a 4-by-3 matrix.

```
M = sym([1 9 4 3 0 1; 3 9 5 1 9 2])
N = reshape(M,4,3)
```

```
M =
[ 1, 9, 4, 3, 0, 1]
[ 3, 9, 5, 1, 9, 2]
```

```
N =
[ 1, 4, 0]
[ 3, 5, 9]
[ 9, 3, 1]
[ 9, 1, 2]
```

**M** and **N** must have the same number of elements. `reshape` reads **M** column-wise to fill in the elements of **N** column-wise.

Alternatively, use a size vector to specify the dimensions of the reshaped matrix.

```
sz = [4 3];
N = reshape(M,sz)
```

```
N =
[ 1, 4, 0]
[ 3, 5, 9]
[ 9, 3, 1]
[ 9, 1, 2]
```

## Automatically Set Dimension of Reshaped Matrix

When you replace a dimension with the placeholder `[]`, `reshape` calculates the required magnitude of that dimension to reshape the matrix.

Create the matrix **M**.

```
M = sym([1 9 4 3 0 1; 3 9 5 1 9 2])
```

```
M =
[ 1, 9, 4, 3, 0, 1]
[ 3, 9, 5, 1, 9, 2]
```

Reshape **M** into a matrix with three columns.

```
reshape(M,[],3)
```

```
ans =
[ 1, 4, 0]
[ 3, 5, 9]
[ 9, 3, 1]
[ 9, 1, 2]
```

`reshape` calculates that a reshaped matrix of three columns needs four rows.

## Reshape Matrix Row-wise

Reshape a matrix row-wise by transposing the result.

Create matrix M.

```
syms x
M = sym([1 9 0 sin(x) 2 2; NaN x 5 1 4 7])

M =
[ 1, 9, 0, sin(x), 2, 2]
[ NaN, x, 5, 1, 4, 7]
```

Reshape M row-wise by transposing the result.

```
reshape(M,4,3).'

ans =
[ 1, NaN, 9, x]
[ 0, 5, sin(x), 1]
[ 2, 4, 2, 7]
```

Note that `.'` returns the non-conjugate transpose while `'` returns the conjugate transpose.

### Reshape 3-D Array into 2-D Matrix

Reshape the 3-by-3-by-2 array M into a 9-by-2 matrix.

M has 18 elements. Because a 9-by-2 matrix also has 18 elements, M can be reshaped into it. Construct M.

```
syms x
M = [sin(x) x 4; 3 2 9; 8 x x];
M(:,:,2) = M'

M(:,:,1) =
[ sin(x), x, 4]
[ 3, 2, 9]
[ 8, x, x]
M(:,:,2) =
[ sin(conj(x)), 3, 8]
[ conj(x), 2, conj(x)]
[ 4, 9, conj(x)]
```

Reshape M into a 9-by-2 matrix.

```
N = reshape(M,9,2)

N =
[ sin(x), sin(conj(x))]
[ 3, conj(x)]
[ 8, 4]
[ x, 3]
[ 2, 2]
[ x, 9]
[ 4, 8]
[ 9, conj(x)]
[ x, conj(x)]
```

### Use reshape to Break Up Arrays

Use `reshape` instead of loops to break up arrays for further computation. Use `reshape` to break up the vector V to find the product of every three elements.

Create vector V.

```
syms x
V = [exp(x) 1 3 9 x 2 7 7 1 8 x^2 3 4 sin(x) x]

V =
[ exp(x), 1, 3, 9, x, 2, 7, 7, 1, 8, x^2, 3, 4, sin(x), x]
```

Specify 3 for the number of rows. Use the placeholder [ ] for the number of columns. This lets reshape automatically calculate the number of columns required for three rows.

```
M = prod( reshape(V,3,[ ]) )

M =
[ 3*exp(x), 18*x, 49, 24*x^2, 4*x*sin(x)]
```

reshape calculates that five columns are required for a matrix of three rows. prod then multiplies the elements of each column to return the result.

## Input Arguments

### A — Input array

symbolic vector | symbolic matrix | symbolic multidimensional array

Input array, specified as a symbolic vector, matrix, or multidimensional array.

### n1,n2 — Dimensions of reshaped matrix

comma-separated scalars

Dimensions of reshaped matrix, specified as comma-separated scalars. For example, reshape(A,3,2) returns a 3-by-2 matrix. The number of elements in the output array specified by n1,n2 must be equal to numel(A).

### n1,...,nM — Dimensions of reshaped array

comma-separated scalars

Dimensions of reshaped array, specified as comma-separated scalars. For example, reshape(A,3,2,2) returns a 3-by-2-by-2 matrix. The number of elements in the output array specified by n1,...,nM must be equal to numel(A).

### sz — Size of reshaped array

numeric vector

Size of reshaped array, specified as a numeric vector. For example, reshape(A,[3 2]) returns a 3-by-2 matrix. The number of elements in the output array specified by sz must be equal to numel(A).

## Version History

Introduced before R2006a

## See Also

colon | transpose

## resultant

Resultant of two polynomials

### Syntax

```
resultant(p,q)
resultant(p,q,var)
```

### Description

`resultant(p,q)` returns the resultant of the polynomials `p` and `q` with respect to the variable found by `symvar`.

`resultant(p,q,var)` returns the resultant with respect to the variable `var`.

### Examples

#### Resultant of Two Polynomials

Find the resultant of two polynomials.

```
syms x y
p = x^2+y;
q = x-2*y;
resultant(p,q)
```

```
ans =
4*y^2 + y
```

Find the resultant with respect to a specific variable by using the third argument.

```
resultant(p,q,y)
```

```
ans =
2*x^2 + x
```

#### Solve Polynomial Equations in Two Variables

If two polynomials have a common root, then the resultant must be 0 at that root. Solve polynomial equations in two variables by calculating the resultant with respect to one variable, and solving the resultant for the other variable.

First, calculate the resultant of two polynomials with respect to `x` to return a polynomial in `y`.

```
syms x y
p = y^3 - 2*x^2 + 3*x*y;
q = x^3 + 2*y^2 - 5*x^2*y;
res = resultant(p,q,x)

res =
y^9 - 35*y^8 + 44*y^6 + 126*y^5 - 32*y^4
```

Solve the resultant for y values of the roots. Avoid numerical roundoff errors by solving equations symbolically using the `solve` function. `solve` represents the solutions symbolically by using `root`.

```
yRoots = solve(res)
```

```
yRoots =
      0
      0
      0
      0
root(z^5 - 35*z^4 + 44*z^2 + 126*z - 32, z, 1)
root(z^5 - 35*z^4 + 44*z^2 + 126*z - 32, z, 2)
root(z^5 - 35*z^4 + 44*z^2 + 126*z - 32, z, 3)
root(z^5 - 35*z^4 + 44*z^2 + 126*z - 32, z, 4)
root(z^5 - 35*z^4 + 44*z^2 + 126*z - 32, z, 5)
```

Calculate numeric values by using `vpa`.

```
vpa(yRoots)
```

```
ans =
      0
      0
      0
      0
      0.23545637976581197505601615070637
- 0.98628744767074109264070992415511 - 1.1027291033304653904984097788422i
- 0.98628744767074109264070992415511 + 1.1027291033304653904984097788422i
      1.7760440932430169904041045113342
      34.96107442233265321982129918627
```

Assume that you want to investigate the fifth root. For the fifth root, calculate the x value by substituting the y value into p and q. Then simultaneously solve the polynomials for x. Avoid numerical roundoff errors by solving equations symbolically using `solve`.

```
eqns = subs([p q], y, yRoots(5));
xRoot5 = solve(eqns,x);
```

Calculate the numeric value of the fifth root by using `vpa`.

```
root5 = vpa([xRoot5 yRoots(5)])

root5 =
[ 0.37078716473998365045397220797284, 0.23545637976581197505601615070637]
```

Verify that the root is correct by substituting `root5` into p and q. The result is 0 within roundoff error.

```
subs([p q],[x y],root5)

ans =
[ -6.313690360861895794753956010471e-41, -9.1835496157991211560057541970488e-41]
```

## Input Arguments

### p — Polynomial

symbolic expression | symbolic function

Polynomial, specified as a symbolic expression or function.

**q — Polynomial**

symbolic expression | symbolic function

Polynomial, specified as a symbolic expression or function.

**var — Variable**

symbolic variable

Variable, specified as a symbolic variable.

## Version History

**Introduced in R2018a**

**See Also**

`eliminate` | `gcd` | `solve`

# removeUnitSystem

Remove unit system

## Syntax

```
removeUnitSystem(unitSystem)
```

## Description

`removeUnitSystem(unitSystem)` removes the unit system `unitSystem`. You can remove only user-defined unit systems created with `newUnitSystem`. You cannot remove predefined unit systems listed in “Unit Systems List” on page 2-71.

## Examples

### Remove Unit System

Define a unit system, use the unit system to rewrite units, and then remove the unit system by using `removeUnitSystem`.

Define the unit system `mySystem` with SI base units and the derived unit kilowatt hour.

```
u = symunit;
bunits = baseUnits('SI');
dunits = [u.kWh];
mySystem = newUnitSystem('mySystem',bunits,dunits)

mySystem =
    "mySystem"
```

Convert 50,000 Joules to derived units of `mySystem` by using `rewrite` with the third argument `'Derived'`. As expected, the result is in kilowatt hour.

```
rewrite(50000*u.J,mySystem,'Derived')

ans =
    (1/72)*[kWh]
```

Remove the unit system `mySystem` by using `removeUnitSystem`.

```
removeUnitSystem(mySystem)
```

Converting units to `mySystem` now throws an error.

## Input Arguments

### `unitSystem` — Name of unit system

string | character vector

Name of the unit system, specified as a string or character vector.

## **Version History**

**Introduced in R2017b**

### **See Also**

`baseUnits` | `derivedUnits` | `newUnitSystem` | `removeUnit` | `rewrite` | `symunit` | `unitSystems`

### **Topics**

"Units of Measurement Tutorial" on page 2-47

"Unit Conversions and Unit Systems" on page 2-53

"Units and Unit Systems List" on page 2-60

### **External Websites**

The International System of Units (SI)



# rewindAnimation

Rewind previously played animation objects

## Syntax

```
rewindAnimation
rewindAnimation(fig)
```

## Description

`rewindAnimation` rewinds previously played animation objects by restoring the animation time parameter to its initial value. The animation objects must be created using the `fanimate` function.

`rewindAnimation(fig)` rewinds animation objects in the figure `fig`.

## Examples

### Rewind Animation of Moving Circle

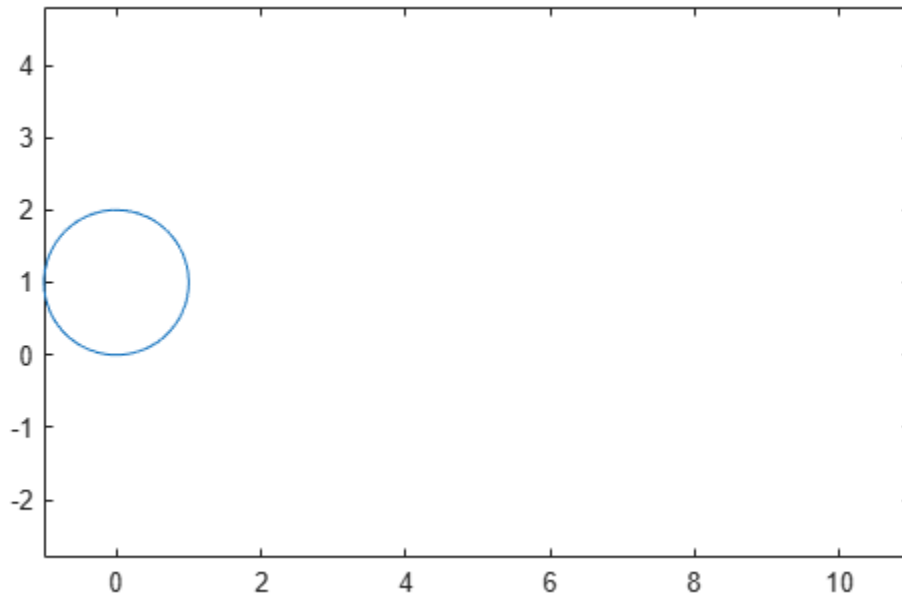
Create an animation of a moving circle and rewind it using `rewindAnimation`.

First, create two symbolic variables, `t` and `x`. The variable `t` defines the time parameter of the animation. Use `t` to set the center of the circle at  $(t, 1)$  and `x` to parameterize the perimeter of the circle within the range  $[-\pi, \pi]$ . Create the circle animation object using `fanimate`. Set the x-axis and y-axis to be equal length.

```
syms t x
fanimate(@fplot,cos(x)+t,sin(x)+1,[-pi pi])
axis equal
```

Play the animation by entering the command `playAnimation`. By default, `playAnimation` plays the animation within the range of `t` from 0 to 10. You can rewind the animation by using `rewindAnimation`. `rewindAnimation` restores the animation time parameter to its initial value at  $t = 0$  and shows the starting animation frame.

```
rewindAnimation
```



### Rewind Animation of Moving Circle with Timer

Create an animation of a moving circle with a timer, and rewind the animation using `rewindAnimation`.

First, create two symbolic variables, `t` and `x`. The variable `t` defines the time parameter of the animation. Create a figure window for the animation.

```
syms t x
fig = figure;
```

Create the circle animation object using `fanimator`. Use `t` to set the center of the circle at  $(t, 1)$  and `x` to parameterize the perimeter of the circle within the range  $[-\pi, \pi]$ . Set the range of the animation time parameter to  $[4, 8]$ . Set the x-axis and y-axis to be equal length.

```
fanimator(@fplot,cos(x)+t,sin(x)+1,[-pi pi],'AnimationRange',[4 8])
axis equal
```

Next, add a timer animation object. Use the `text` function to create a piece of text to count the elapsed time. Use `num2str` to convert the time parameter to a string.

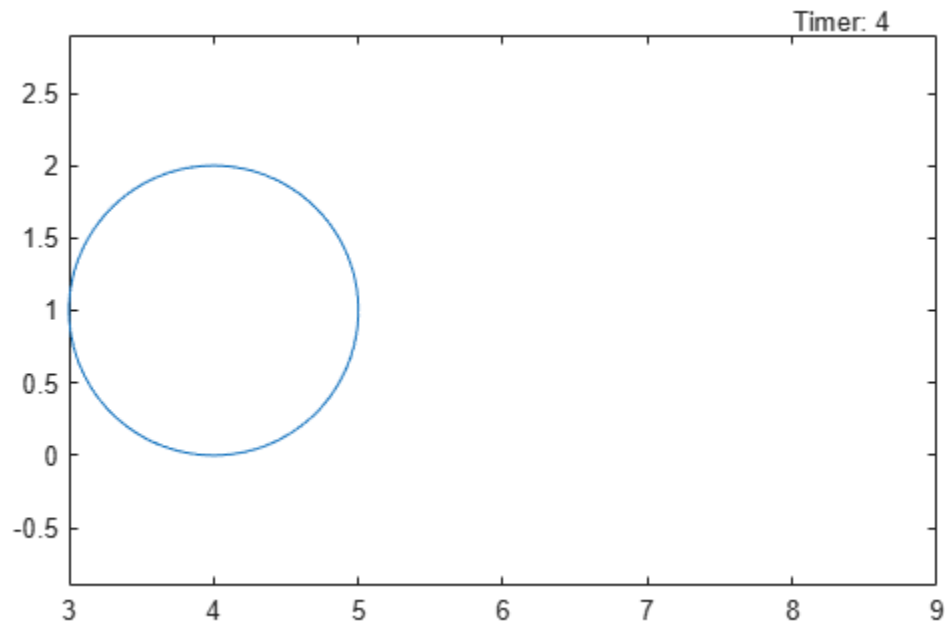
```
hold on
fanimator(@(t) text(8,3,"Timer: "+num2str(t,2)), 'AnimationRange',[4 8])
hold off
```

Play the animation in figure `fig` between 4 and 8 seconds by entering the `playAnimation` command.

```
playAnimation(fig,'AnimationRange',[4 8])
```

You can rewind a previously played animation by using `rewindAnimation`. `rewindAnimation` restores the animation time parameter to its initial value at  $t = 4$  and shows the starting animation frame.

```
rewindAnimation(fig)
```



## Input Arguments

**fig** — Target figure

Figure object

Target figure, specified as a Figure object. For more information about Figure objects, see `figure`.

## Version History

Introduced in R2019a

## See Also

`animationToFrame` | `playAnimation` | `f Animator` | `writeAnimation`

## rewrite

Rewrite expression in terms of another function

### Syntax

`R = rewrite(expr,target)`

### Description

`R = rewrite(expr,target)` rewrites the symbolic expression `expr` in terms of another function as specified by `target`. The rewritten expression is mathematically equivalent to the original expression. If `expr` is a vector or matrix, `rewrite` acts element-wise on `expr`.

### Examples

#### Rewrite Between Trigonometric and Exponential Functions

Rewrite any trigonometric function in terms of the exponential function by specifying the target "exp".

```
syms x
sin2exp = rewrite(sin(x),"exp")
```

$$\sin 2\exp = \frac{e^{-xi} - i}{2} - \frac{e^{xi} - i}{2}$$

```
tan2exp = rewrite(tan(x),"exp")
```

$$\tan 2\exp = -\frac{e^{2xi} - i - i}{e^{2xi} + 1}$$

Rewrite the exponential function in terms of any trigonometric function by specifying the trigonometric function as the target. For a full list of target options, see "target" on page 6-0 .

```
syms x
exp2sin = rewrite(exp(x),"sin")
```

$$\exp 2\sin = -2 \sin\left(\frac{x}{2}\right)^2 - \sin(x) - i + 1$$

```
exp2tan = rewrite(-(exp(x*2i)*1i - 1i)/(exp(x*2i) + 1),"tan")
```

$$\exp 2\tan = -\frac{\frac{(\tan(x) - i) - i}{\tan(x) + i} + i}{\frac{\tan(x) - i}{\tan(x) + i} - 1}$$

Simplify `exp2tan` to the expected form by using `simplify`.

```
exp2tan = simplify(exp2tan)
exp2tan = tan(x)
```

### Rewrite Between Trigonometric Functions

Rewrite any trigonometric function in terms of any other trigonometric function by specifying the target. For a full list of target options, see “target” on page 6-0 .

Rewrite  $\tan(x)$  in terms of the sine function by specifying the target "sin".

```
syms x
tan2sin = rewrite(tan(x), "sin")

tan2sin =
```

$$-\frac{\sin(x)}{2 \sin\left(\frac{x}{2}\right)^2 - 1}$$

### Rewrite Between Inverse Trigonometric and Logarithm Functions

Rewrite any inverse trigonometric function in terms of the logarithm function by specifying the target "log". For a full list of target options, see “target” on page 6-0 .

Rewrite  $\arccos(x)$  and  $\operatorname{arccot}(x)$  in terms of the log function.

```
syms x
acos2log = rewrite(acos(x), "log")

acos2log = -log(x + sqrt(1 - x^2) i) i

acot2log = rewrite(acot(x), "log")

acot2log =
```

$$\frac{\log\left(1 - \frac{i}{x}\right) i}{2} - \frac{\log\left(1 + \frac{i}{x}\right) i}{2}$$

Similarly, rewrite the logarithm function in terms of any inverse trigonometric function by specifying the inverse trigonometric function as the target.

### Rewrite Elements of Matrix

Rewrite each element of a matrix by calling `rewrite` on the matrix.

Rewrite all elements of a matrix in terms of the `exp` function.

```
syms x
matrix = [sin(x) cos(x); sinh(x) cosh(x)];
R = rewrite(matrix, "exp")
```

$$R = \begin{pmatrix} \frac{e^{-xi}}{2} - \frac{e^{xi}}{2} & \frac{e^{-xi}}{2} + \frac{e^{xi}}{2} \\ \frac{e^x}{2} - \frac{e^{-x}}{2} & \frac{e^{-x}}{2} + \frac{e^x}{2} \end{pmatrix}$$

### Rewrite Between Sine and Cosine Functions

Rewrite the cosine function in terms of the sine function. Here, the `rewrite` function rewrites the cosine function using the identity  $\cos(x) = 1 - 2 \sin\left(\frac{x}{2}\right)^2$ , which is valid for any  $x$ .

```
syms x
R = rewrite(cos(x), "sin")
```

$$R = 1 - 2 \sin\left(\frac{x}{2}\right)^2$$

`rewrite` does not rewrite  $\sin(x)$  as either  $-\sqrt{1 - \cos^2(x)}$  or  $\sqrt{1 - \cos^2(x)}$  because these expressions are not valid for all  $x$ . However, using the square of these expressions to represent  $\sin(x)^2$  is valid for all  $x$ . Thus, `rewrite` can rewrite  $\sin(x)^2$ .

```
syms x
R1 = rewrite(sin(x), "cos")
R1 = sin(x)
R2 = rewrite(sin(x)^2, "cos")
R2 = 1 - cos(x)^2
```

### Rewrite root Function

Find the root of a polynomial by using `root`. The result is a column vector in terms of the root function with  $k = 1, 2, 3, 4$ , or  $5$  for the  $k$ th root of the polynomial.

```
syms x
sols = root(x^5 - x^4 - 1, x)
```

$$\text{sols} = \begin{pmatrix} \text{root}(x^5 - x^4 - 1, x, 1) \\ \text{root}(x^5 - x^4 - 1, x, 2) \\ \text{root}(x^5 - x^4 - 1, x, 3) \\ \text{root}(x^5 - x^4 - 1, x, 4) \\ \text{root}(x^5 - x^4 - 1, x, 5) \end{pmatrix}$$

Expand the root function in `sols` by using `rewrite` with the "expandroot" option. The result is in terms of arithmetic operations such as  $^$ ,  $*$ ,  $/$ ,  $+$ , and  $-$  that operate on exact symbolic numbers.

Because the expanded result can involve many terms that operate arithmetically, numerical approximation of this result can be inaccurate (due to accumulation of round-off errors).

```
R = rewrite(sols,"expandroot")
```

R =

$$\begin{pmatrix} \frac{1}{2} - \frac{\sqrt{3} i}{2} \\ \frac{1}{2} + \frac{\sqrt{3} i}{2} \\ \frac{1}{3\sigma_1} + \sigma_1 \\ -\frac{1}{6\sigma_1} - \frac{\sigma_1}{2} - \frac{\sqrt{3}\left(\frac{1}{3\sigma_1} - \sigma_1\right) i}{2} \\ -\frac{1}{6\sigma_1} - \frac{\sigma_1}{2} + \frac{\sqrt{3}\left(\frac{1}{3\sigma_1} - \sigma_1\right) i}{2} \end{pmatrix}$$

where

$$\sigma_1 = \left( \frac{\sqrt{23}\sqrt{108}}{108} + \frac{1}{2} \right)^{1/3}$$

As an alternative, you can numerically approximate `sols` directly by using `vpa` to return variable-precision symbolic numbers. The resulting numeric values have the default 32 significant digits, which are more accurate.

```
solsVpa = vpa(sols)
```

solsVpa =

$$\begin{pmatrix} -0.66235897862237301298045442723905 - 0.56227951206230124389918214490937 i \\ -0.66235897862237301298045442723905 + 0.56227951206230124389918214490937 i \\ 0.5 - 0.86602540378443864676372317075294 i \\ 0.5 + 0.86602540378443864676372317075294 i \\ 1.3247179572447460259609088544781 \end{pmatrix}$$

To use `sols` without Symbolic Math Toolbox™, you can generate code and convert `sols` to a MATLAB® function by using `matlabFunction`. The generated file uses the `roots` function that operates on the numeric double data type.

```
matlabFunction(sols,"File","myfile");
type myfile
```

```
function sols = myfile
%MYFILE
%    SOLS = MYFILE
```

```
%    This function was generated by the Symbolic Math Toolbox version 26.1.
%    20-Apr-2026 21:32:11
```

```
t0 = roots([1.0,-1.0,0.0,0.0,0.0,-1.0]);
t2 = t0(1);
t0 = roots([1.0,-1.0,0.0,0.0,0.0,-1.0]);
t3 = t0(2);
t0 = roots([1.0,-1.0,0.0,0.0,0.0,-1.0]);
```

```

t4 = t0(3);
t0 = roots([1.0,-1.0,0.0,0.0,0.0,-1.0]);
t5 = t0(4);
t0 = roots([1.0,-1.0,0.0,0.0,0.0,-1.0]);
t6 = t0(5);
sols = [t2;t3;t4;t5;t6];
end

```

## Rewrite symsum and root Functions

Find the indefinite integral of a polynomial fraction. The result is in terms of the `symsum` and `root` functions as represented by the  $\sum$  and `root` symbols.

```

syms x
F = int(1/(x^3 + x - 1),x)

```

```

F =

$$\sum_{k=1}^3 \log\left(\text{root}\left(z^3 - \frac{3z}{31} - \frac{1}{31}, z, k\right) \left(3x + \text{root}\left(z^3 - \frac{3z}{31} - \frac{1}{31}, z, k\right) (6x - 9)\right) \text{root}\left(z^3 - \frac{3z}{31} - \frac{1}{31}, z, k\right)\right)$$


```

Rewrite the result in terms of arithmetic operations. Because the symbolic summation is the outermost operation, first apply the `rewrite` function with the "expandsum" option to expand `symsum`. Then, apply `rewrite` again with the "expandroot" option to rewrite the `root` function. The resulting symbolic expression is in terms of arithmetic operations such as  $^$ ,  $*$ ,  $/$ ,  $+$ , and  $-$ .

```

R = rewrite(rewrite(F,"expandsum"),"expandroot")

```

```

R =

$$\log((3x + (6x - 9)\sigma_3)\sigma_3)\sigma_3 - \log(-(3x - (6x - 9)\sigma_2)\sigma_2)\sigma_2 - \log(-(3x - (6x - 9)\sigma_1)\sigma_1)\sigma_1$$


```

where

$$\sigma_1 = \frac{1}{62\sigma_4} + \frac{\sigma_4}{2} - \frac{\sqrt{3}\left(\frac{1}{31\sigma_4} - \sigma_4\right)i}{2}$$

$$\sigma_2 = \frac{1}{62\sigma_4} + \frac{\sigma_4}{2} + \frac{\sqrt{3}\left(\frac{1}{31\sigma_4} - \sigma_4\right)i}{2}$$

$$\sigma_3 = \frac{1}{31\sigma_4} + \sigma_4$$

$$\sigma_4 = \left(\frac{\sqrt{27}\sqrt{119164}}{119164} + \frac{1}{62}\right)^{1/3}$$

## Input Arguments

### expr — Input to rewrite

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix | symbolic multidimensional array

Input to rewrite or replace, specified as a symbolic number, variable, expression, function, vector, matrix, or multidimensional array.



**target — Target function or function to expand**

string scalar | character vector

Target function or function to expand, specified as a string scalar or character vector. This table summarizes the rewriting rules for all allowed **target** options.

| Target                             | Rewrite or Replace These Functions                                     | In Terms of These Functions                                        |
|------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------------|
| "exp"                              | All trigonometric and hyperbolic functions including inverse functions | exp, log                                                           |
| "log"                              | All inverse trigonometric and hyperbolic functions                     | log                                                                |
| "sincos"                           | tan, cot, exp, sinh, cosh, tanh, coth                                  | sin, cos                                                           |
| "sin", "cos", "tan", or "cot"      | sin, cos, exp, tan, cot, sinh, cosh, tanh, coth except the target      | Target trigonometric function                                      |
| "sinhcosh"                         | tan, cot, exp, sin, cos, tanh, coth                                    | sinh, cosh                                                         |
| "sinh", "cosh", "tanh", "coth"     | tan, cot, exp, sin, cos, sinh, cosh, tanh, coth except the target      | Target hyperbolic function                                         |
| "asin", "acos", "atan", "acot"     | log, and all inverse trigonometric and inverse hyperbolic functions    | Target inverse trigonometric function                              |
| "asinh", "acosh", "atanh", "acoth" | log, and all inverse trigonometric and inverse hyperbolic functions    | Target inverse hyperbolic function                                 |
| "sqrt"                             | $\text{abs}(x + \text{li}*y)$                                          | $\text{sqrt}(x^2 + y^2)$                                           |
| "heaviside"                        | sign, triangularPulse, rectangularPulse                                | heaviside                                                          |
| "piecewise"                        | abs, heaviside, sign, triangularPulse, rectangularPulse                | piecewise                                                          |
| "expandroot"                       | root                                                                   | Arithmetic operations such as $\wedge$ , $*$ , $/$ , $+$ , and $-$ |
| "expandsum"                        | symsum                                                                 | Arithmetic operations such as $+$ and $-$                          |

**Tips**

- **rewrite** replaces symbolic function calls in **expr** with another function as specified by **target** only if the replacement is mathematically valid. Otherwise, it keeps the original function calls. For example, see “Rewrite Between Sine and Cosine Functions” on page 6-1310.

## Version History

Introduced in R2012a

### R2023a: Expand root and symsum functions

You can rewrite or expand symbolic expressions that use the `root` and `symsum` functions in terms of arithmetic operations. For examples, see “Rewrite root Function” on page 6-1310 and “Rewrite symsum and root Functions” on page 6-1312.

## See Also

### Functions

`collect` | `combine` | `expand` | `factor` | `horner` | `numden` | `simplify` | `simplifyFraction` | `unitConvert`

### Live Editor Tasks

Simplify Symbolic Expression

### Topics

“Choose Function to Rearrange Expression” on page 3-123

“Simplify Symbolic Expressions Using Live Editor Task” on page 3-138

# rhs

Right side (RHS) of equation

## Syntax

```
rhsEqn = rhs(eqn)
```

## Description

`rhsEqn = rhs(eqn)` returns the right side of the symbolic equation `eqn`. The value of `eqn` also can be a symbolic condition, such as  $x > 0$ . If `eqn` is an array, then `rhs` returns an array of the right sides of the equations in `eqn`.

Conditions that use the  $>$  or  $>=$  operator are internally rewritten using the  $<$  or  $<=$  operator. Therefore, `rhs` returns the original left side. For example, `rhs(x > 0)` returns  $x$ .

## Examples

### Find Right Side of Equation

Find the right side of the equation  $2y = x^2$  by using `rhs`.

First, declare the equation.

```
syms x y
eqn = 2*y == x^2
```

```
eqn = 2 y = x^2
```

Find the right side of `eqn` by using `rhs`.

```
rhsEqn = rhs(eqn)
```

```
rhsEqn = x^2
```

### Find Right Side of Condition

Find the right side of the condition  $x < y + 1$  by using `rhs`.

First, declare the condition.

```
syms x y
cond = x < y + 1
```

```
cond = x < y + 1
```

Find the right side of `cond` by using `rhs`.

```
rhsCond = rhs(cond)
rhsCond = y + 1
```

### Find Right Side of Equations in Array

For an array that contains equations and conditions, `rhs` returns an array of the right sides of those equations or conditions. The output array is the same size as the input array.

Find the right side of the equations and conditions in the vector `V`.

```
syms x y
V = [y^2 == x^2, x ~= 0, x*y >= 1]
V = (y^2 = x^2 x ≠ 0 1 ≤ x y)
rhsV = rhs(V)
rhsV = (x^2 0 x y)
```

Because any condition using the `>=` operator is internally rewritten using the `<=` operator, the sides of the last condition in `V` are exchanged.

### Find Right Side of Equation Involving Symbolic Matrix Variables

Find the right side of a symbolic equation that involves symbolic matrix variables.

Create the symbolic matrix variables and the symbolic equation.

```
syms A [2 2] matrix
syms B [2 1] matrix
syms C [1 2] matrix
eqn = B*C == A*A - 2*A + eye(2)
eqn = B C = I2 - 2 A + A2
```

Find the right side of the equation by using `rhs`.

```
rhsEqn = rhs(eqn)
rhsEqn = I2 - 2 A + A2
```

## Input Arguments

### eqn — Equation or condition

symbolic equation | symbolic condition | vector of symbolic equations or conditions | matrix of symbolic equations or conditions | multidimensional array of symbolic equations or conditions

Equation or condition, specified as a symbolic equation or condition, or a vector, matrix, or multidimensional array of symbolic equations or conditions.

Data Types: `sym` | `symfun` | `symmatrix` | `symfunmatrix`

## Version History

Introduced in R2017a

### **R2025a: Get left and right sides of symbolic equation that includes piecewise expressions**

You can use the `lhs` and `rhs` functions to get the left and right sides of a symbolic equation that includes piecewise expressions. For an example, see “Solve System of Equations with Piecewise Expressions” on page 6-1177.

### **R2022b: Find right side of symbolic equation or condition involving symbolic matrix variables and matrix functions**

The `rhs` function accepts a symbolic equation or condition involving symbolic matrix variables and matrix functions as an input argument. The input data type can be `symmatrix` or `symfunmatrix`. For an example, see “Find Right Side of Equation Involving Symbolic Matrix Variables” on page 6-1316.

## See Also

`assume` | `children` | `lhs` | `subs`

## root

Represent roots of polynomial

### Syntax

```
r = root(p,x)
r = root(p,x,k)
```

### Description

`r = root(p,x)` returns a column vector of numbered roots of symbolic polynomial `p` with respect to `x`. Symbolically solving a high-degree polynomial for its roots can be complex and not all polynomials can be solved analytically. In this case, the Symbolic Math Toolbox uses the `root` function to represent the roots of the polynomial.

`r = root(p,x,k)` represents the `k`th root of symbolic polynomial `p` with respect to `x`.

### Examples

#### Represent Roots of High-Degree Polynomial

Represent the roots of the polynomial  $x^3 + 1$  using `root`. The `root` function returns a column vector. The elements of this vector represent the three roots of the polynomial.

```
syms x
p = x^3 + 1;
root(p,x)

ans =

$$\begin{pmatrix} \text{root}(x^3 + 1, x, 1) \\ \text{root}(x^3 + 1, x, 2) \\ \text{root}(x^3 + 1, x, 3) \end{pmatrix}$$

```

`root( $x^3 + 1, x, 1$ )` represents the first root of `p`, while `root( $x^3 + 1, x, 2$ )` represents the second root, and so on. Use this syntax to represent roots of high-degree polynomials.

#### Find Roots of Quadratic Polynomial

Find the roots of the quadratic polynomial  $x^2 - x - 1$ . You can use the `root` function to represent these roots.

```
syms x
p = x^2 - x - 1;
r = root(p,x)
```

$$r = \begin{pmatrix} \text{root}(x^2 - x - 1, x, 1) \\ \text{root}(x^2 - x - 1, x, 2) \end{pmatrix}$$

To convert these roots to high-precision floating point numbers, you can use `vpa`.

`rVpa = vpa(r)`

$$rVpa = \begin{pmatrix} -0.61803398874989484820458683436564 \\ 1.6180339887498948482045868343656 \end{pmatrix}$$

### Find Roots of High-Degree Polynomial

When solving a high-degree polynomial, `solve` represents the roots by using `root`. Alternatively, you can either return an explicit solution by using the `MaxDegree` option or return a numerical result by using `vpa`.

Find the roots of  $x^3 + 3x - 16$ .

```
syms x
p = x^3 + 3*x - 16;
R = solve(p,x)
```

$$R = \begin{pmatrix} \text{root}(z^3 + 3z - 16, z, 1) \\ \text{root}(z^3 + 3z - 16, z, 2) \\ \text{root}(z^3 + 3z - 16, z, 3) \end{pmatrix}$$

Find the roots explicitly by setting the `MaxDegree` option to the degree of the polynomial. Polynomials with a degree greater than 4 do not have explicit solutions.

`Rexplicit = solve(p,x,"MaxDegree",3)`

$$Rexplicit = \begin{pmatrix} \sigma_1 - \frac{1}{\sigma_1} \\ \frac{1}{2\sigma_1} - \frac{\sigma_1}{2} - \frac{\sqrt{3}\left(\frac{1}{\sigma_1} + \sigma_1\right)i}{2} \\ \frac{1}{2\sigma_1} - \frac{\sigma_1}{2} + \frac{\sqrt{3}\left(\frac{1}{\sigma_1} + \sigma_1\right)i}{2} \end{pmatrix}$$

where

$$\sigma_1 = (\sqrt{65} + 8)^{1/3}$$

Calculate the roots numerically by using `vpa` to convert `R` to high-precision floating point.

`Rnumeric = vpa(R)`

```
Rnumeric =
    2.1267693318103912337456401562601
    (-1.0633846659051956168728200781301 - 2.5283118563671914055545884653776 i)
    (-1.0633846659051956168728200781301 + 2.5283118563671914055545884653776 i)
```

If the call to `root` contains parameters, substitute the parameters with numbers by using `subs` before calling `vpa`.

### Use root in Symbolic Computations

You can use the `root` function as input to Symbolic Math Toolbox functions such as `simplify`, `subs`, and `diff`.

Simplify an expression containing `root` using the `simplify` function.

```
syms x
r = root(x^6 + x, x, 1);
simplify(sin(r)^2 + cos(r)^2)

ans = 1
```

Substitute for parameters in `root` with numbers using `subs`.

```
syms b
subs(root(x^2 + b*x, x, 1), b, 5)

ans = root(x^2 + 5 x, x, 1)
```

Substituting for parameters using `subs` is necessary before converting `root` to numeric form using `vpa`.

Differentiate an expression containing `root` with respect to a parameter using `diff`.

```
diff(root(x^2 + b*x, x, 1), b)

ans =
    root(x^2 + b x, x, 1)
    b
```

### Find Inverse Laplace Transform of Ratio of Polynomials

Find the inverse Laplace transform of a ratio of two polynomials using `ilaplace`. The inverse Laplace transform is returned in terms of `root`.

```
syms s
G = (s^3 + 1)/(s^6 + s^5 + s^2);
H = ilaplace(G)

H =
    t - \left( \sum_{k=1}^4 \frac{e^{t \operatorname{root}(z^4 + z^3 + 1, z, k)}}{4 \operatorname{root}(z^4 + z^3 + 1, z, k) + 3} \right)
```



When you get the `root` function in output, you can use the `root` function as input in subsequent symbolic calculations. However, if a numerical result is required, convert the `root` function to a high-precision numeric result using `vpa`.

Convert the inverse Laplace transform to numeric form using `vpa`.

```
H_vpa = simplify(vpa(H))
```

```
H_vpa = t + 0.30881178580997278695808136329347 e-1.0189127943851558447865795886366 t
cos(0.60256541999859902604398442197193 t) - 0.30881178580997278695808136329347
e0.5189127943851558447865795886366 t cos(0.666609844932018579153758800733 t)
- 0.6919689479355443779463355813596 e-1.0189127943851558447865795886366 t
sin(0.60256541999859902604398442197193 t) - 0.16223098826244593894459034019473
e0.5189127943851558447865795886366 t sin(0.666609844932018579153758800733 t)
```

## Input Arguments

### **p** — Symbolic polynomial

symbolic expression

Symbolic polynomial, specified as a symbolic expression.

### **x** — Variable

symbolic variable

Variable, specified as a symbolic variable.

### **k** — Number of polynomial root

number | vector | matrix | multidimensional array | symbolic number | symbolic vector | symbolic matrix | symbolic multidimensional array

Number of polynomial root, specified as a number, vector, matrix, multidimensional array, or a symbolic number, vector, matrix, or multidimensional array. When `k` is a nonscalar, `root` acts element-wise on `k`.

Example: `root(f,x,3)` represents the third root of `f`.

## Tips

- You can numerically approximate a symbolic expression involving the `root` function by using `vpa` to return variable-precision symbolic numbers. Starting in R2023a, you can convert the expression to a MATLAB function that can be used without Symbolic Math Toolbox by using `matlabFunction`. The generated file uses the `roots` function that operates on the numeric double data type.

## Version History

Introduced in R2015b

## See Also

`solve` | `vpa` | `rewrite`

## rref

Reduced row echelon form of matrix (Gauss-Jordan elimination)

### Syntax

`rref(A)`

### Description

`rref(A)` computes the reduced row echelon form of the symbolic matrix `A`. If the elements of a matrix contain free symbolic variables, `rref` regards the matrix as nonzero.

To solve a system of linear equations, use `linsolve`.

### Examples

#### Compute Reduced Row Echelon Form of Numeric Matrix

Compute the reduced row echelon form of the magic square matrix.

```
rref(sym(magic(4)))
```

```
ans =  
[ 1, 0, 0, 1]  
[ 0, 1, 0, 3]  
[ 0, 0, 1, -3]  
[ 0, 0, 0, 0]
```

#### Compute Reduced Row Echelon Form of Symbolic Matrix

Compute the reduced row echelon form of the following symbolic matrix.

```
syms a b c  
A = [a b c; b c a; a + b, b + c, c + a];  
rref(A)
```

```
ans =  
[ 1, 0, -(- c^2 + a*b)/(- b^2 + a*c)]  
[ 0, 1, -(- a^2 + b*c)/(- b^2 + a*c)]  
[ 0, 0, 0]
```

## Version History

Introduced before R2006a

### See Also

`eig` | `jordan` | `linsolve` | `rank`

## rsums

Interactive evaluation of Riemann sums

### Syntax

```
rsums(f)
rsums(f,a,b)
rsums(f,[a,b])
```

### Description

`rsums(f)` interactively approximates the integral of  $f(x)$  by middle Riemann sums for  $x$  from 0 to 1. `rsums(f)` displays a graph of  $f(x)$  using 10 terms (rectangles). You can adjust the number of terms taken in the middle Riemann sum by using the slider below the graph. The number of terms available ranges from 2 to 128.  $f$  can be a character vector or a symbolic expression. The height of each rectangle is determined by the value of the function in the middle of each interval.

`rsums(f,a,b)` and `rsums(f,[a,b])` approximates the integral for  $x$  from  $a$  to  $b$ .

### Examples

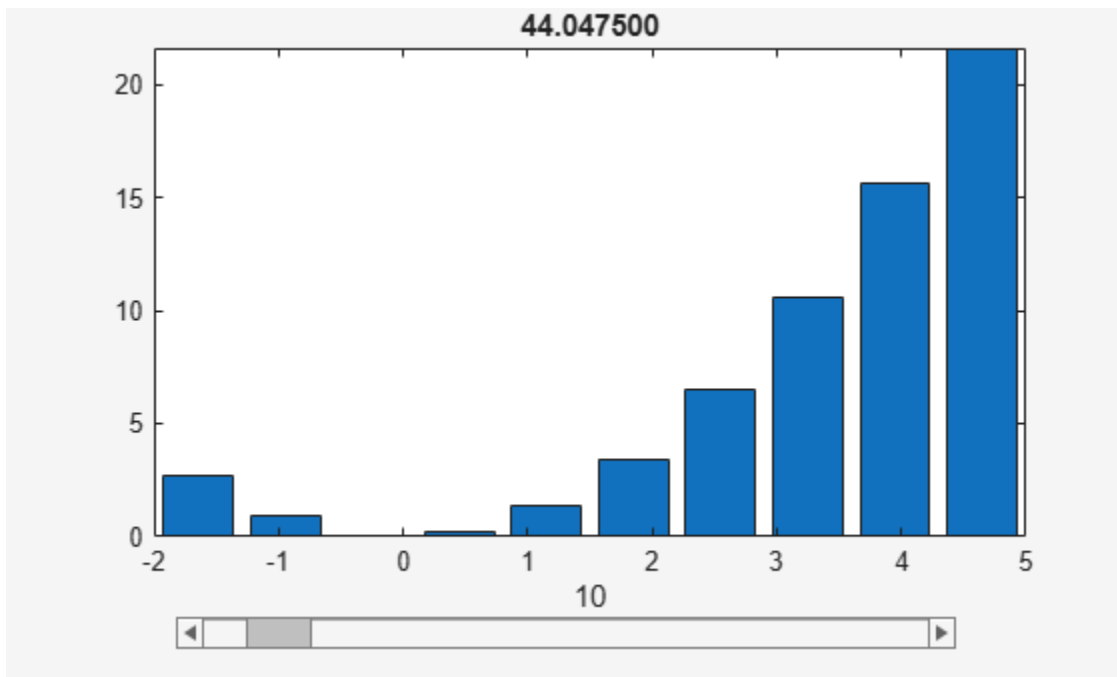
#### Approximate Integral by Riemann Sum

Approximate the integral  $\int_0^1 x^2 dx$  by middle Riemann sum. `rsums` displays a graph of  $x^2$  using 10 terms of the midpoint Riemann sum for the integration range from 0 to 1. The total sum is 0.3325.

```
syms x
rsums(x^2)
```

Change the integration range of  $x$  from -2 to 5. The total Riemann sum is 44.0475.

```
rsums(x^2,-2,5)
```



## Input Arguments

### **f — Integrand**

symbolic expression | symbolic function | symbolic number

Integrand, specified as a symbolic expression, function, or number.

### **a — Lower bound**

number | symbolic number

Lower bound, specified as a number or symbolic number.

### **b — Upper bound**

number | symbolic number

Upper bound, specified as a number or symbolic number.

## Version History

Introduced before R2006a

## See Also

taylorTool | int | funtool

## sec

Symbolic secant function

### Syntax

`sec(X)`

### Description

`sec(X)` returns the secant function on page 6-1327 of X.

### Examples

#### Secant Function for Numeric and Symbolic Arguments

Depending on its arguments, `sec` returns floating-point or exact symbolic results.

Compute the secant function for these numbers. Because these numbers are not symbolic objects, `sec` returns floating-point results.

```
A = sec([-2, -pi, pi/6, 5*pi/7, 11])

A =
    -2.4030    -1.0000     1.1547    -1.6039    225.9531
```

Compute the secant function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `sec` returns unresolved symbolic calls.

```
symA = sec(sym([-2, -pi, pi/6, 5*pi/7, 11]))

symA =
[ 1/cos(2), -1, (2*3^(1/2))/3, -1/cos((2*pi)/7), 1/cos(11)]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

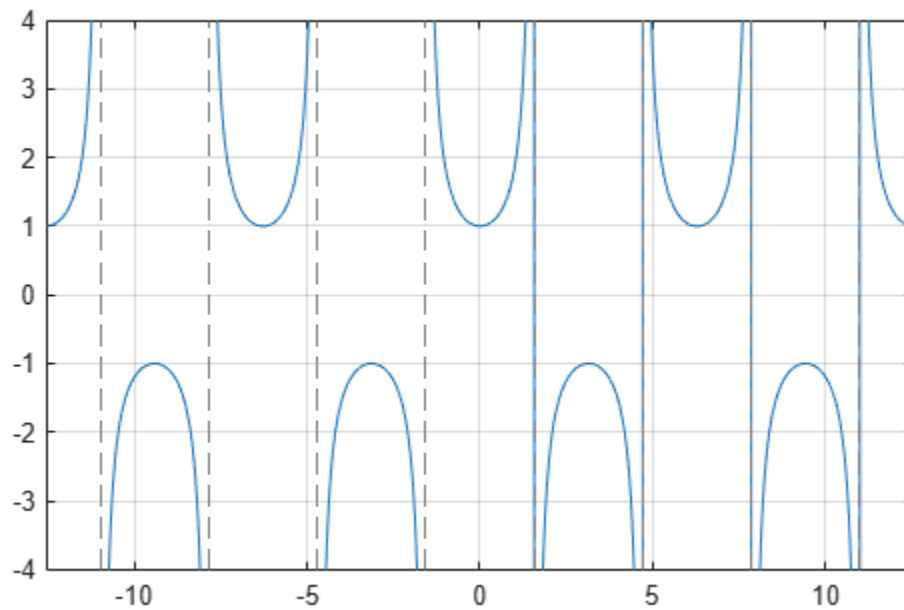
```
vpa(symA)

ans =
[ -2.4029979617223809897546004014201,...
  -1.0,...
  1.1547005383792515290182975610039,...
  -1.6038754716096765049444092780298,...
  225.95305931402493269037542703557]
```

#### Plot Secant Function

Plot the secant function on the interval from  $-4\pi$  to  $4\pi$ .

```
syms x
fplot(sec(x),[-4*pi 4*pi])
grid on
```



### Handle Expressions Containing Secant Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `sec`.

Find the first and second derivatives of the secant function:

```
syms x
diff(sec(x), x)
diff(sec(x), x, x)

ans =
sin(x)/cos(x)^2

ans =
1/cos(x) + (2*sin(x)^2)/cos(x)^3
```

Find the indefinite integral of the secant function:

```
int(sec(x), x)

ans =
log(1/cos(x)) + log(sin(x) + 1)
```

Find the Taylor series expansion of `sec(x)`:

```
taylor(sec(x), x)

ans =
(5*x^4)/24 + x^2/2 + 1
```

Rewrite the secant function in terms of the exponential function:

```
rewrite(sec(x), 'exp')
```

```
ans =
1/(exp(-x*1i)/2 + exp(x*1i)/2)
```

### Evaluate Units with sec Function

sec numerically evaluates these units automatically: radian, degree, arcmin, arcsec, and revolution.

Show this behavior by finding the secant of x degrees and 2 radians.

```
u = symunit;
syms x
f = [x*u.degree 2*u.radian];
secf = sec(f)

secf =
[ 1/cos((pi*x)/180), 1/cos(2)]
```

You can calculate secf by substituting for x using subs and then using double or vpa.

## Input Arguments

### X — Input

symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic expression | symbolic function | symbolic matrix function | symbolic vector | symbolic matrix

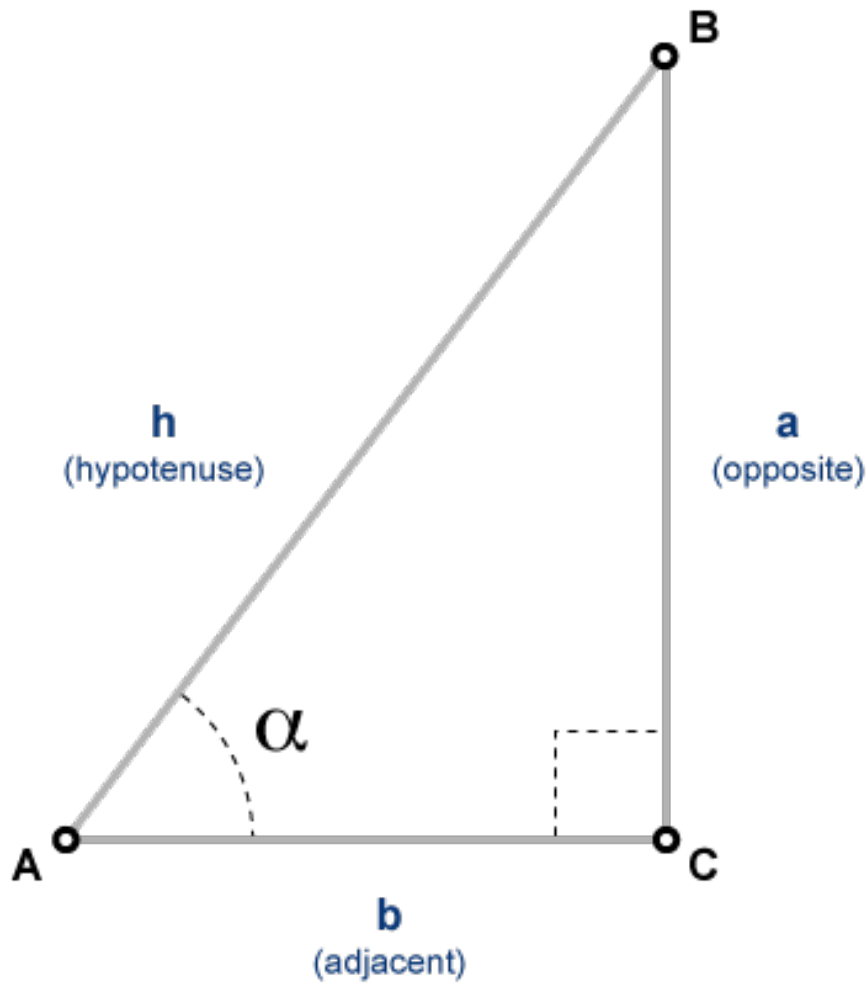
Input, specified as a symbolic number, scalar variable, matrix variable, expression, function, matrix function, or as a vector or matrix of symbolic numbers, scalar variables, expressions, or functions.

## More About

### Secant Function

The secant of an angle,  $\alpha$ , defined with reference to a right angled triangle is

$$\sec(\alpha) = \frac{1}{\cos(\alpha)} = \frac{\text{hypotenuse}}{\text{adjacent side}} = \frac{h}{b}.$$



The secant of a complex argument,  $\alpha$ , is

$$\sec(\alpha) = \frac{2}{e^{i\alpha} + e^{-i\alpha}}.$$

## Version History

Introduced before R2006a

**R2022a: Compute secant of symbolic matrix functions**

The `sec` function accepts an input argument of type `symfunmatrix`.

**R2021b: Compute secant of symbolic matrix variables**

The `sec` function accepts an input argument of type `symmatrix`.



**See Also**

`acos` | `asin` | `atan` | `acsc` | `asec` | `acot` | `sin` | `cos` | `tan` | `csc` | `cot`

## sech

Symbolic hyperbolic secant function

### Syntax

`sech(X)`

### Description

`sech(X)` returns the hyperbolic secant function of  $X$ .

### Examples

#### Hyperbolic Secant Function for Numeric and Symbolic Arguments

Depending on its arguments, `sech` returns floating-point or exact symbolic results.

Compute the hyperbolic secant function for these numbers. Because these numbers are not symbolic objects, `sech` returns floating-point results.

```
A = sech([-2, -pi*i, pi*i/6, 0, pi*i/3, 5*pi*i/7, 1])

A =
    0.2658    -1.0000    1.1547    1.0000    2.0000    -1.6039    0.6481
```

Compute the hyperbolic secant function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `sech` returns unresolved symbolic calls.

```
symA = sech(sym([-2, -pi*i, pi*i/6, 0, pi*i/3, 5*pi*i/7, 1]))

symA =
[ 1/cosh(2), -1, (2*3^(1/2))/3, 1, 2, -1/cosh((pi*2i)/7), 1/cosh(1)]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

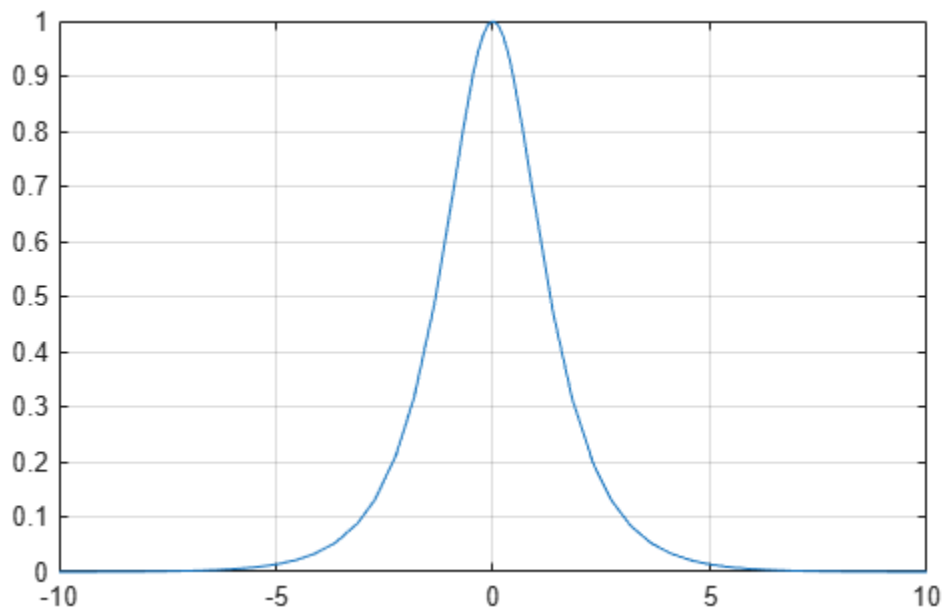
```
vpa(symA)

ans =
[ 0.26580222883407969212086273981989,...
 -1.0,...
 1.1547005383792515290182975610039,...
 1.0,...
 2.0,...
 -1.6038754716096765049444092780298,...
 0.64805427366388539957497735322615]
```

#### Plot Hyperbolic Secant Function

Plot the hyperbolic secant function on the interval from -10 to 10.

```
syms x
fplot(sech(x), [-10, 10])
grid on
```



### Handle Expressions Containing Hyperbolic Secant Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `sech`.

Find the first and second derivatives of the hyperbolic secant function:

```
syms x
diff(sech(x), x)
diff(sech(x), x, x)

ans =
-sinh(x)/cosh(x)^2

ans =
(2*sinh(x)^2)/cosh(x)^3 - 1/cosh(x)
```

Find the indefinite integral of the hyperbolic secant function:

```
int(sech(x), x)

ans =
2*atan(exp(x))
```

Find the Taylor series expansion of `sech(x)`:

```
taylor(sech(x), x)

ans =
(5*x^4)/24 - x^2/2 + 1
```

Rewrite the hyperbolic secant function in terms of the exponential function:

```
rewrite(sech(x), 'exp')
```

```
ans =  
1/(exp(-x)/2 + exp(x)/2)
```

## Input Arguments

### **X — Input**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Version History

**Introduced before R2006a**

### **See Also**

acosh | asinh | atanh | acsch | asech | acoth | sinh | cosh | tanh | csch | coth

# separateUnits

Separate units from expression

## Syntax

```
[Data,Units] = separateUnits(expr)
Data = separateUnits(expr)
```

## Description

`[Data,Units] = separateUnits(expr)` returns the symbolic units of the symbolic expression `expr` in `Units` and the rest of `expr` in `Data`.

`Data = separateUnits(expr)` removes symbolic units from `expr` and then returns the rest.

## Examples

### Separate Units and Expression

Separate the units from the expression `10*t*u.m/u.s`, where `u = symunit`, by providing two output arguments for `separateUnits`.

```
u = symunit;
syms t
speed = 10*t*u.m/u.s;
[Data,Units] = separateUnits(speed)
```

```
Data = 10 t
```

```
Units =
    m
    s
```

Return only the expression with the units removed by providing one output argument.

```
Data = separateUnits(speed)
```

```
Data = 10 t
```

### Separate Compatible Units

If the input has compatible units that can be converted to the same unit, then `separateUnits` performs the conversion and returns the separated result. Units are compatible when they have the same dimensions (although the conversion factor may not necessarily be 1).

Separate the units from `2*u.m + 30*u.cm + 12*u.in`. Even though the units differ, `separateUnits` converts them to the same unit and returns the separated result.

```
u = symunit;  
[Data,Units] = separateUnits(2*u.m + 30*u.cm + 12*u.in)  
  
Data =  
    1628  
    625  
  
Units = m
```

## Input Arguments

### **expr** — Input

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic equation | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, equation, multidimensional array, function, or expression.

When adding or subtracting quantities with different units in `expr`, the units must be compatible. Units are compatible when they have the same dimensions. For example, see “Separate Compatible Units” on page 6-1333.

## Output Arguments

### **Data** — Expression after removing units

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic equation | symbolic multidimensional array | symbolic function | symbolic expression

Expression after removing units, returned as a number, vector, matrix, or multidimensional array, or a symbolic number, variable, vector, matrix, equation, multidimensional array, function, or expression.

### **Units** — Units from input

symbolic units

Units from input, returned as symbolic units.

## Tips

- If an expression has incompatible units, then `separateUnits` errors. Units are incompatible when they do not have the same dimensions, such as length and time.

For example, this code will error.

```
u = symunit;  
[Data,Units] = separateUnits(2*u.m + 3*u.s)
```

Instead, to return the units in the input, use `findUnits`.

```
Units = findUnits(2*u.m + 3*u.s)  
  
Units =  
[[m], [s]]
```

## Version History

Introduced in R2017a

### R2022b: separateUnits returns simplified expression after separating units

*Behavior changed in R2022b*

Starting in R2022b, `separateUnits` returns a simplified expression after separating units. For example, separate the units from an expression `expr`, and return the remaining expression as `Data` and the units as `Units`.

```
u = symunit;
syms t tau;
t_s = t*u.s;
t_0 = 0.01*u.s;
tau_s = tau*u.s;
a = 2*u.V;
b = 6*u.V;
expr = a + b*exp(-(t_s-t_0)/tau_s)
[Data,Units] = separateUnits(expr)

expr =
2*[V] + 6*exp((((1/100)*[s] + (-t)*[s])/tau)*(1/[s]))*[V]

Data =
6*exp(-(100*t - 1)/(100*tau)) + 2

Units =
[V]
```

In previous releases, `separateUnits` returns an unsimplified expression: `Data = 6*exp((((1/100)*[s] - t*[s])/tau)*(1/[s])) + 2`.

## See Also

`checkUnits` | `findUnits` | `isUnit` | `newUnit` | `symunit` | `str2symunit` | `symunit2str` | `unitConversionFactor`

## Topics

“Units of Measurement Tutorial” on page 2-47  
 “Unit Conversions and Unit Systems” on page 2-53  
 “Units and Unit Systems List” on page 2-60

## External Websites

The International System of Units (SI)

## series

Puiseux series

### Syntax

```
series(f,var)
series(f,var,a)
series( ____,Name,Value)
```

### Description

`series(f,var)` approximates  $f$  with the Puiseux series expansion of  $f$  up to the fifth order at the point  $\text{var} = 0$ . If you do not specify `var`, then `series` uses the default variable determined by `symvar(f,1)`.

`series(f,var,a)` approximates  $f$  with the Puiseux series expansion of  $f$  at the point  $\text{var} = a$ .

`series( ____,Name,Value)` uses additional options specified by one or more `Name,Value` pair arguments. You can specify `Name,Value` after the input arguments in any of the previous syntaxes.

### Examples

#### Find Puiseux Series Expansion

Find the Puiseux series expansions of univariate and multivariate expressions.

Find the Puiseux series expansion of this expression at the point  $x = 0$ .

```
syms x
series(1/sin(x), x)

ans =
x/6 + 1/x + (7*x^3)/360
```

Find the Puiseux series expansion of this multivariate expression. If you do not specify the expansion variable, `series` uses the default variable determined by `symvar(f,1)`.

```
syms s t
f = sin(s)/sin(t);
symvar(f, 1)
series(f)

ans =
t

ans =
sin(s)/t + (7*t^3*sin(s))/360 + (t*sin(s))/6
```

To use another expansion variable, specify it explicitly.

```
syms s t
f = sin(s)/sin(t);
series(f, s)
```



```
ans =
s^5/(120*sin(t)) - s^3/(6*sin(t)) + s/sin(t)
```

### Specify Expansion Point

Find the Puiseux series expansion of  $\psi(x)$  around  $x = \text{Inf}$ . The default expansion point is 0. To specify a different expansion point, use the `ExpansionPoint` name-value pair.

```
series(psi(x), x, 'ExpansionPoint', Inf)
```

```
ans =
log(x) - 1/(2*x) - 1/(12*x^2) + 1/(120*x^4)
```

Alternatively, specify the expansion point as the third argument of `series`.

```
syms x
series(psi(x), x, Inf)
```

```
ans =
log(x) - 1/(2*x) - 1/(12*x^2) + 1/(120*x^4)
```

### Plot Puiseux Series Approximation

Find the Puiseux series expansion of  $\exp(x)/x$  using different truncation orders.

Find the series expansion up to the default truncation order 6.

```
syms x
f = exp(x)/x;
s6 = series(f, x)
```

```
s6 =
x/2 + 1/x + x^2/6 + x^3/24 + x^4/120 + 1
```

Use `Order` to control the truncation order. For example, approximate the same expression up to the orders 7 and 8.

```
s7 = series(f, x, 'Order', 7)
```

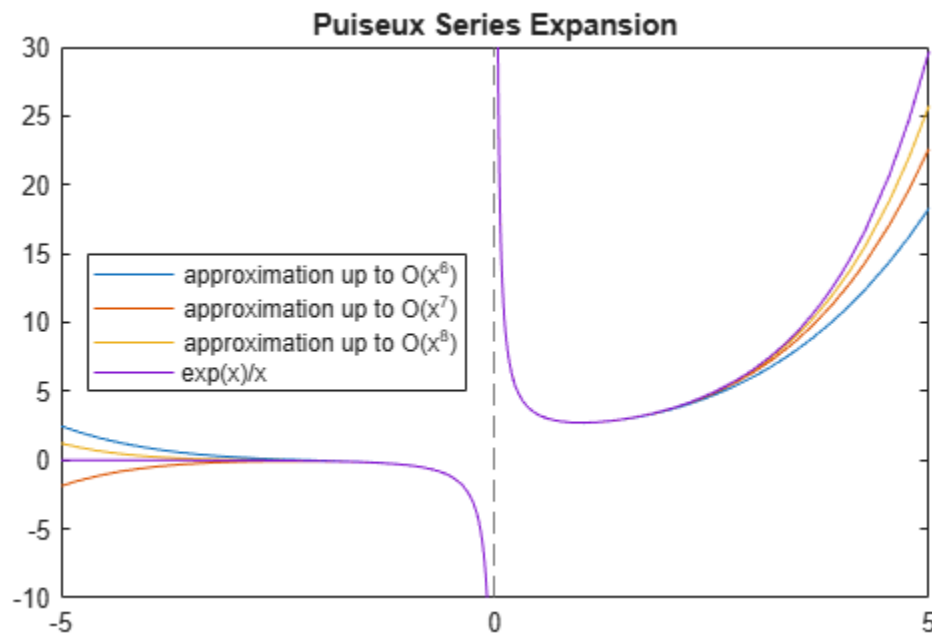
```
s7 =
x/2 + 1/x + x^2/6 + x^3/24 + x^4/120 + x^5/720 + 1
```

```
s8 = series(f, x, 'Order', 8)
```

```
s8 =
x/2 + 1/x + x^2/6 + x^3/24 + x^4/120 + x^5/720 + x^6/5040 + 1
```

Plot the original expression `f` and its approximations `s6`, `s7`, and `s8`. Note how the accuracy of the approximation depends on the truncation order.

```
fplot([s6 s7 s8 f])
legend('approximation up to 0(x^6)', 'approximation up to 0(x^7)', ...
      'approximation up to 0(x^8)', 'exp(x)/x', 'Location', 'Best')
title('Puiseux Series Expansion')
```



### Specify Direction of Expansion

Find the Puiseux series approximations using the `Direction` argument. This argument lets you change the convergence area, which is the area where `series` tries to find converging Puiseux series expansion approximating the original expression.

Find the Puiseux series approximation of this expression. By default, `series` finds the approximation that is valid in a small open circle in the complex plane around the expansion point.

```
syms x
series(sin(sqrt(-x)), x)

ans =
(-x)^(1/2) - (-x)^(3/2)/6 + (-x)^(5/2)/120
```

Find the Puiseux series approximation of the same expression that is valid in a small interval to the left of the expansion point. Then, find an approximation that is valid in a small interval to the right of the expansion point.

```
syms x
series(sin(sqrt(-x)), x)
series(sin(sqrt(-x)), x, 'Direction', 'left')
series(sin(sqrt(-x)), x, 'Direction', 'right')

ans =
(-x)^(1/2) - (-x)^(3/2)/6 + (-x)^(5/2)/120

ans =
- x^(1/2)*1i - (x^(3/2)*1i)/6 - (x^(5/2)*1i)/120

ans =
x^(1/2)*1i + (x^(3/2)*1i)/6 + (x^(5/2)*1i)/120
```

Try computing the Puiseux series approximation of this expression. By default, `series` tries to find an approximation that is valid in the complex plane around the expansion point. For this expression, such approximation does not exist.

```
series(real(sin(x)), x)
```

```
Error using sym/series>scalarSeries (line 90)
Unable to compute series expansion.
```

However, the approximation exists along the real axis, to both sides of  $x = 0$ .

```
series(real(sin(x)), x, 'Direction', 'realAxis')
```

```
ans =
x^5/120 - x^3/6 + x
```

## Input Arguments

### **f** — Input to approximate

symbolic expression | symbolic function | symbolic vector | symbolic matrix | symbolic multidimensional array

Input to approximate, specified as a symbolic expression or function. It also can be a vector, matrix, or multidimensional array of symbolic expressions or functions.

### **var** — Expansion variable

symbolic variable

Expansion variable, specified as a symbolic variable. If you do not specify `var`, then `series` uses the default variable determined by `symvar(f,1)`.

### **a** — Expansion point

0 (default) | number | symbolic number | symbolic variable | symbolic function | symbolic expression

Expansion point, specified as a number, or a symbolic number, variable, function, or expression. The expansion point cannot depend on the expansion variable.

You also can specify the expansion point as a `Name,Value` pair argument. If you specify the expansion point both ways, then the `Name,Value` pair argument takes precedence.

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `series(psi(x),x,'ExpansionPoint',Inf,'Order',9)`

### **ExpansionPoint** — Expansion point

0 (default) | number | symbolic number | symbolic variable | symbolic function | symbolic expression

Expansion point, specified as a number, or a symbolic number, variable, function, or expression. The expansion point cannot depend on the expansion variable.

You can also specify the expansion point using the input argument `a`. If you specify the expansion point both ways, then the `Name, Value` pair argument takes precedence.

### Order — Truncation order of Puiseux series expansion

6 (default) | positive integer | symbolic positive integer

Truncation order of Puiseux series expansion, specified as a positive integer or a symbolic positive integer.

`series` computes the Puiseux series approximation with the order  $n - 1$ . The truncation order  $n$  is the exponent in the  $O$ -term:  $O(\text{var}^n)$ .

### Direction — Direction for area of convergence of Puiseux series expansion

'complexPlane' (default) | 'left' | 'right' | 'realAxis'

Direction for area of convergence of Puiseux series expansion, specified as:

|                |                                                                                                                                                      |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| 'left'         | Find a Puiseux series approximation that is valid in a small interval to the left of the expansion point.                                            |
| 'right'        | Find a Puiseux series approximation that is valid in a small interval to the right of the expansion point.                                           |
| 'realAxis'     | Find a Puiseux series approximation that is valid in a small interval on the both sides of the expansion point.                                      |
| 'complexPlane' | Find a Puiseux series approximation that is valid in a small open circle in the complex plane around the expansion point. This is the default value. |

## Tips

- If you use both the third argument `a` and the `ExpansionPoint` name-value pair to specify the expansion point, the value specified via `ExpansionPoint` prevails.

## Version History

Introduced in R2015b

## See Also

`pade` | `taylor`

# sign

Sign of real or complex value

## Syntax

`sign(z)`

## Description

`sign(z)` returns the sign of real or complex value  $z$ . The sign of a complex number  $z$  is defined as  $z/|z|$ . If  $z$  is a vector or a matrix, `sign(z)` returns the sign of each element of  $z$ .

## Examples

### Signs of Real Numbers

Find the signs of these symbolic real numbers:

```
[sign(sym(1/2)), sign(sym(0)), sign(sym(pi) - 4)]
```

```
ans =  
[ 1, 0, -1]
```

### Signs of Matrix Elements

Find the signs of the real and complex elements of matrix A:

```
A = sym([(1/2 + i), -25; i*(i + 1), pi/6 - i*pi/2]);  
sign(A)
```

```
ans =  
[ 5^(1/2)*(1/5 + 2i/5), -1]  
[ 2^(1/2)*(- 1/2 + 1i/2), 5^(1/2)*18^(1/2)*(1/30 - 1i/10)]
```

### Sign of Symbolic Expression

Find the sign of this expression assuming that the value  $x$  is negative:

```
syms x  
assume(x < 0)  
sign(5*x^3)
```

```
ans =  
-1
```

For further computations, clear the assumption on  $x$  by recreating it using `syms`:

```
syms x
```

## Input Arguments

### **z — Input**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input specified as a symbolic number, variable, expression, function, vector, or matrix.

## More About

### **Sign Function**

The sign function of any number  $z$  is defined via the absolute value of  $z$ :

$$\text{sign}(z) = \frac{z}{|z|}$$

Thus, the sign function of a real number  $z$  can be defined as follows:

$$\text{sign}(z) = \begin{cases} -1 & \text{if } x < 0 \\ 0 & \text{if } x = 0 \\ 1 & \text{if } x > 0 \end{cases}$$

## Tips

- Calling `sign` for a number that is not a symbolic object invokes the MATLAB `sign` function.

## Version History

**Introduced in R2013a**

### **See Also**

`abs` | `angle` | `imag` | `real` | `signIm`

# signIm

Sign of the imaginary part of complex number

## Syntax

`signIm(z)`

## Description

`signIm(z)` returns the sign of the imaginary part of a complex number  $z$ . For all complex numbers with a nonzero imaginary part,  $\text{signIm}(z) = \text{sign}(\text{imag}(z))$ . For real numbers,  $\text{signIm}(z) = -\text{sign}(z)$ .

$$\text{signIm}(z) = \begin{cases} 1 & \text{if } \text{Im}(z) > 0 \text{ or } \text{Im}(z) = 0 \text{ and } \text{Re}(z) < 0 \\ 0 & \text{if } z = 0 \\ -1 & \text{otherwise} \end{cases}$$

## Examples

### Symbolic Results Including signIm

Results of symbolic computations, especially symbolic integration, can include the `signIm` function.

Integrate this expression. For complex values  $a$  and  $x$ , this integral includes `signIm`.

```
syms a x
f = 1/(a^2 + x^2);
F = int(f, x, -Inf, Inf)

F =
(pi*signIm(1i/a))/a
```

### Signs of Imaginary Parts of Numbers

Find the signs of imaginary parts of complex numbers with nonzero imaginary parts and of real numbers.

Use `signIm` to find the signs of imaginary parts of these numbers. For complex numbers with nonzero imaginary parts, `signIm` returns the sign of the imaginary part of the number.

```
[signIm(-18 + 3*i), signIm(-18 - 3*i),...
signIm(10 + 3*i), signIm(10 - 3*i),...
signIm(Inf*i), signIm(-Inf*i)]
```

```
ans =
     1     -1     1     -1     1     -1
```

For real positive numbers, `signIm` returns `-1`.

```
[signIm(2/3), signIm(1), signIm(100), signIm(Inf)]
```

```
ans =
    -1    -1    -1    -1
```

For real negative numbers, `signIm` returns 1.

```
[signIm(-2/3), signIm(-1), signIm(-100), signIm(-Inf)]
```

```
ans =
     1     1     1     1
```

`signIm(0)` is 0.

```
[signIm(0), signIm(0 + 0*i), signIm(0 - 0*i)]
```

```
ans =
     0     0     0
```

### Signs of Imaginary Parts of Symbolic Expressions

Find the signs of imaginary parts of symbolic expressions that represent complex numbers.

Call `signIm` for these symbolic expressions without additional assumptions. Because `signIm` cannot determine if the imaginary part of a symbolic expression is positive, negative, or zero, it returns unresolved symbolic calls.

```
syms x y z
[signIm(z), signIm(x + y*i), signIm(x - 3*i)]
```

```
ans =
[ signIm(z), signIm(x + y*I), signIm(x - 3i)]
```

Assume that `x`, `y`, and `z` are positive values. Find the signs of imaginary parts of the same symbolic expressions.

```
syms x y z positive
[signIm(z), signIm(x + y*i), signIm(x - 3*i)]
```

```
ans =
[ -1, 1, -1]
```

For further computations, clear the assumptions by recreating the variables using `syms`.

```
syms x y z
```

Find the first derivative of the `signIm` function. `signIm` is a constant function, except for the jump discontinuities along the real axis. The `diff` function ignores these discontinuities.

```
syms z
diff(signIm(z), z)
```

```
ans =
 0
```

### Signs of Imaginary Parts of Matrix Elements

`signIm` accepts vectors and matrices as its input argument. This lets you find the signs of imaginary parts of several numbers in one function call.

Find the signs of imaginary parts of the real and complex elements of matrix `A`.



```
A = sym([(1/2 + i), -25; i*(i + 1), pi/6 - i*pi/2]);  
signIm(A)
```

```
ans =  
[ 1, 1]  
[ 1, -1]
```

## Input Arguments

### **z — Input representing complex number**

number | symbolic number | symbolic variable | symbolic expression | vector | matrix

Input representing complex number, specified as a number, symbolic number, symbolic variable, expression, vector, or matrix.

## Tips

- `signIm(NaN)` returns NaN.

## Version History

**Introduced in R2014b**

## See Also

`conj` | `imag` | `real` | `sign`

# simplify

Algebraic simplification

## Syntax

```
S = simplify(expr)
S = simplify(expr, Name=Value)
```

## Description

`S = simplify(expr)` performs algebraic simplification of `expr`. If `expr` is a symbolic vector or matrix, this function simplifies each element of `expr`.

`S = simplify(expr, Name=Value)` performs algebraic simplification of `expr` using additional options specified by one or more `Name=Value` arguments.

## Examples

### Simplify Expressions

Simplify these symbolic expressions.

```
syms x a b c
S = simplify(sin(x)^2 + cos(x)^2)

S = 1

S = simplify(exp(c*log(sqrt(a+b))))

S = (a + b)c/2
```

### Simplify Matrix Elements

Call `simplify` for this symbolic matrix. When the input argument is a vector or matrix, `simplify` tries to find a simpler form of each element of the vector or matrix.

```
syms x
M = [(x^2 + 5*x + 6)/(x + 2), sin(x)*sin(2*x) + cos(x)*cos(2*x);
      (exp(-x*1i)*1i)/2 - (exp(x*1i)*1i)/2, sqrt(16)];
S = simplify(M)

S =

$$\begin{pmatrix} x + 3 & \cos(x) \\ \sin(x) & 4 \end{pmatrix}$$

```

### Get Simpler Results for Logarithms and Powers

Simplify a symbolic expression that contains logarithms and powers. By default, `simplify` does not combine powers and logarithms because combining them is not valid for generic complex values.

```
syms x
expr = (log(x^2 + 2*x + 1) - log(x + 1))*sqrt(x^2);
S = simplify(expr)
```

$$S = -\left(\log(x+1) - \log\left((x+1)^2\right)\right)\sqrt{x^2}$$

To apply the simplification rules that allow the `simplify` function to combine powers and logarithms, set `IgnoreAnalyticConstraints` to `true`.

```
S = simplify(expr,IgnoreAnalyticConstraints=true)
```

$$S = x\log(x+1)$$

### Get Simpler Results Using More Simplification Steps

Simplify this expression.

```
syms x
expr = ((exp(-x*1i)*1i) - (exp(x*1i)*1i))/(exp(-x*1i) + exp(x*1i));
S = simplify(expr)
```

$$S = -\frac{e^{2xi}i - i}{e^{2xi} + 1}$$

By default, `simplify` uses one internal simplification step. You can get different, often shorter, simplification results by increasing the number of simplification steps.

```
S10 = simplify(expr,Steps=10)
```

$$S10 = \frac{2i}{e^{2xi} + 1} - i$$

```
S30 = simplify(expr,Steps=30)
```

$$S30 = \frac{(\cos(x) - \sin(x)i)i}{\cos(x)} - i$$

```
S50 = simplify(expr,Steps=50)
```

$$S50 = \tan(x)$$

If you are unable to return the desired result, try alternate simplification functions. See “Choose Function to Rearrange Expression” on page 3-123.

### Get Equivalent Results for Symbolic Expression

Get equivalent results for a symbolic expression by setting the value of `All` to `true`.

```
syms x
expr = cos(x)^2 - sin(x)^2;
S = simplify(expr, All=true)
```

$$S = \begin{pmatrix} \cos(2x) \\ \cos(x)^2 - \sin(x)^2 \end{pmatrix}$$

Increase the number of simplification steps to 10. Find the other equivalent results for the same expression.

```
S = simplify(expr, Steps=10, All=true)
```

$$S = \begin{pmatrix} \cos(2x) \\ 1 - 2\sin(x)^2 \\ 2\cos(x)^2 - 1 \\ \cos(x)^2 - \sin(x)^2 \\ \cot(2x)\sin(2x) \\ \frac{e^{-2xi}}{2} + \frac{e^{2xi}}{2} \end{pmatrix}$$

### Separate Real and Imaginary Parts

Attempt to separate real and imaginary parts of an expression by setting the value of `Criterion` to `"preferReal"`.

```
syms x
f = (exp(x + exp(-x*1i))/2 - exp(x*1i)/2)*1i)/2 - ...
    (exp(-x - exp(-x*1i))/2 + exp(x*1i)/2)*1i)/2;
S = simplify(f, Criterion="preferReal", Steps=100)
```

$$S = \sin(\sin(x)) \cosh(x) + \cos(\sin(x)) \sinh(x) i$$

If `Criterion` is not set to `"preferReal"`, then `simplify` returns a shorter result but the real and imaginary parts are not separated.

```
S = simplify(f, Steps=100)
```

$$S = \sin(\sin(x) + xi)$$

When you set `Criterion` to `"preferReal"`, the simplifier disfavors expression forms where complex values appear inside subexpressions. In nested subexpressions, the deeper the complex value appears inside an expression, the least preference this form of an expression gets.

## Avoid Imaginary Terms in Exponents

Attempt to avoid imaginary terms in exponents by setting `Criterion` to `"preferReal"`.

Show this behavior by simplifying a complex symbolic expression with and without setting `Criterion` to `"preferReal"`. When `Criterion` is set to `"preferReal"`, then `simplify` places the imaginary term outside the exponent.

```
expr = sym(1i)^(1i+1);
withoutPreferReal = simplify(expr,Steps=100)

withoutPreferReal =

$$(-1)^{\frac{1}{2}} + \frac{1}{2}i$$


withPreferReal = simplify(expr,Criterion="preferReal",Steps=100)

withPreferReal =

$$e^{-\frac{\pi}{2}} i$$

```

## Simplify Units

Simplify expressions containing symbolic units of the same dimension by using `simplify`.

```
u = symunit;
expr = 300*u.cm + 40*u.inch + 2*u.m;
S = simplify(expr)

S =

$$\frac{752}{125} \text{ m}$$

```

`simplify` automatically chooses the unit to rewrite into. To choose a specific unit, use `rewrite`.

## Get Simpler Result by Expanding Expression

In most cases, to simplify a symbolic expression using Symbolic Math Toolbox™, you only need to use the `simplify` function. But for some large and complex expressions, you can obtain a faster and simpler result by using the `expand` function before applying `simplify`.

For instance, this workflow gives better results when finding the determinant of a matrix that represents the Kerr metric [1]. Declare the parameters of the Kerr metric.

```
syms theta real;
syms r rs a real positive;
```

Define the matrix that represents the Kerr metric.

```
rho = sqrt(r^2 + a^2*cos(theta)^2);
delta = r^2 + a^2 - r*rs;
g(1,1) = - (1 - r*rs/rho^2);
g(1,4) = - (rs*a*r*sin(theta)^2)/rho^2;
```

```

g(4,1) = - (rs*a*r*sin(theta)^2)/rho^2;
g(2,2) = rho^2/delta;
g(3,3) = rho^2;
g(4,4) = (r^2 + a^2 + rs*a^2*r*sin(theta)^2/rho^2)*sin(theta)^2;

```

Evaluate the determinant of the Kerr metric.

```
det_g = det(g)
```

```

det_g =

$$-\frac{\sin(\theta)^2 \left( a^6 \cos(\theta)^4 + a^4 r^2 \cos(\theta)^4 + 2 a^4 r^2 \cos(\theta)^2 + rs a^4 r \cos(\theta)^2 \sin(\theta)^2 - rs a^4 r \cos(\theta)^2 + 2 a^2 r^4 \cos(\theta)^2 + a^2 r^4 - rs a^2 r^2 \right)}{a^2 + r^2 - rs r}$$


```

Simplify the determinant using the `simplify` function.

```
D = simplify(det_g)
```

```

D = -sin(theta)^2 (a^2 cos(theta)^2 + r^2) (-a^2 sin(theta)^2 + a^2 + r^2)

```

Instead, flatten the expression using the `expand` function, and then apply the `simplify` function. The result is simpler with this extra step.

```
D = simplify(expand(det_g))
```

```

D = -sin(theta)^2 (-a^2 sin(theta)^2 + a^2 + r^2)^2

```

## Input Arguments

### **expr** — Input expression

symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input expression, specified as a symbolic expression, function, vector, or matrix.

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `Seconds=60` limits the simplification process to 60 seconds.

### **All** — Option to return equivalent results

false (default) | true

Option to return equivalent results, specified as one of these logical values. When you use this option, the input argument `expr` must be a scalar.

|       |                                                                        |
|-------|------------------------------------------------------------------------|
| false | Use the default option to return only the final simplification result. |
|-------|------------------------------------------------------------------------|

|                   |                                                                                                                                                                                                          |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>true</code> | Return a column vector of equivalent results for the input expression. You can use this option along with the <code>Steps</code> option to obtain alternative expressions in the simplification process. |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Criterion — Simplification criterion

`"default"` (default) | `"preferReal"`

Simplification criterion, specified as one of these character vectors.

|                           |                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>"default"</code>    | Use the default (internal) simplification criteria.                                                                                                                                                                                                                                                                                                                          |
| <code>"preferReal"</code> | Favor the forms of $S$ containing real values over the forms containing complex values. If any form of $S$ contains complex values, the simplifier disfavors the forms where complex values appear inside subexpressions. In case of nested subexpressions, the deeper the complex value appears inside an expression, the least preference this form of an expression gets. |

### IgnoreAnalyticConstraints — Simplification rules

`false` (default) | `true`

Simplification rules, specified as one of these values.

|                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>false</code> | Use strict simplification rules. <code>simplify</code> always returns results that are analytically equivalent to the initial expression.                                                                                                                                                                                                                                                                                                                                                         |
| <code>true</code>  | Apply purely algebraic simplifications to expressions. Setting <code>IgnoreAnalyticConstraints</code> to <code>true</code> can give you simpler solutions, which could lead to results not generally valid. In other words, this option applies mathematical identities that are convenient, but the results might not hold for all possible values of the variables. In some cases, the results might not be equivalent to the initial expression. For details, see “Algorithms” on page 6-1352. |

### Seconds — Time limit for the simplification process

`Inf` (default) | `positive number`

Time limit for the simplification process, specified as a positive value that denotes the maximal time in seconds.

### Steps — Number of simplification steps

`1` (default) | `positive number`

Number of simplification steps, specified as a positive value that denotes the maximal number of internal simplification steps. Note that increasing the number of simplification steps can slow down your computations.

`simplify(expr, Steps=n)` is equivalent to `simplify(expr, n)`, where  $n$  is the number of simplification steps.

## Tips

- Simplification of mathematical expression is not a clearly defined subject. There is no universal idea as to which form of an expression is simplest. The form of a mathematical expression that is simplest for one problem might be complicated or even unsuitable for another problem.

## Algorithms

When you use `IgnoreAnalyticConstraints`, then `simplify` follows some of these rules:

- $\log(a) + \log(b) = \log(a \cdot b)$  for all values of  $a$  and  $b$ . In particular, the following equality is valid for all values of  $a$ ,  $b$ , and  $c$ :

$$(a \cdot b)^c = a^c \cdot b^c.$$

- $\log(a^b) = b \cdot \log(a)$  for all values of  $a$  and  $b$ . In particular, the following equality is valid for all values of  $a$ ,  $b$ , and  $c$ :

$$(a^b)^c = a^{b \cdot c}.$$

- If  $f$  and  $g$  are standard mathematical functions and  $f(g(x)) = x$  for all small positive numbers,  $f(g(x)) = x$  is assumed to be valid for all complex values of  $x$ . In particular:

- $\log(e^x) = x$
- $\text{asin}(\sin(x)) = x$ ,  $\text{acos}(\cos(x)) = x$ ,  $\text{atan}(\tan(x)) = x$
- $\text{asinh}(\sinh(x)) = x$ ,  $\text{acosh}(\cosh(x)) = x$ ,  $\text{atanh}(\tanh(x)) = x$
- $W_k(x \cdot e^x) = x$  for all branch indices  $k$  of the Lambert W function.

## Version History

Introduced before R2006a

## References

[1] Zee, A. *Einstein Gravity in a Nutshell*. Princeton: Princeton University Press, 2013.

## See Also

### Functions

`collect` | `combine` | `expand` | `factor` | `horner` | `numden` | `rewrite` | `simplifyFraction`

### Live Editor Tasks

Simplify Symbolic Expression

### Topics

“Simplify Symbolic Expressions” on page 3-133

“Choose Function to Rearrange Expression” on page 3-123

“Simplify Symbolic Expressions Using Live Editor Task” on page 3-138



# simplifyFraction

Simplify symbolic rational expressions

## Syntax

```
simplifyFraction(expr)
simplifyFraction(expr, 'Expand', true)
```

## Description

`simplifyFraction(expr)` simplifies the rational expression `expr` such that the numerator and denominator have no divisors in common.

`simplifyFraction(expr, 'Expand', true)` expands the numerator and denominator of the resulting simplified fraction as polynomials without factorization.

## Examples

### Simplify Symbolic Rational Expressions

Simplify two rational expressions by using `simplifyFraction`.

```
syms x y
fraction = (x^2-1)/(x+1);
simplifyFraction(fraction)

ans =
x - 1

fraction = (y*(x^2-1))/((x+1)*(x-1));
simplifyFraction(fraction)

ans =
y
```

### Expand Simplified Rational Expression

Create a rational expression. Simplify the expression by using `simplifyFraction`.

```
syms x y
fraction = ((y+1)^2*(x^2-1))/((x+1)*(x-1)^2);
simplifyFraction(fraction)

ans =
(y + 1)^2/(x - 1)
```

Simplify the same rational expression again. Expand the numerator and denominator of the resulting fraction by setting 'Expand' to true.

```
simplifyFraction(fraction, 'Expand', true)
```

```
ans =  
(y^2 + 2*y + 1)/(x - 1)
```

### Simplify Rational Subexpressions of Expressions

Simplify rational expressions by using `simplifyFraction`.

```
syms x  
expr = ((x^2+2*x+1)/(x+1))^(1/2);  
simplifyFraction(expr)
```

```
ans =  
(x + 1)^(1/2)
```

Simplify rational expressions that contain irrational subexpressions instead of variables.

```
expr = (1-sin(x)^2)/(1-sin(x));  
simplifyFraction(expr)
```

```
ans =  
sin(x) + 1
```

`simplifyFraction` does not apply algebraic identities to simplify the rational expression. Show that `simplifyFraction` does not apply standard trigonometric identities.

```
expr = (1-cos(x)^2)/sin(x);  
simplifyFraction(expr)
```

```
ans =  
-(cos(x)^2 - 1)/sin(x)
```

## Input Arguments

### **expr** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## Tips

- `expr` can contain irrational subexpressions, such as `sin(x)` and `x^(-1/3)`. `simplifyFraction` simplifies such expressions as if they were variables.
- `simplifyFraction` does not apply algebraic identities.

## Alternatives

You can also simplify rational expressions using the general simplification function `simplify`. However, `simplifyFraction` is more efficient for simplifying rational expressions.

## Version History

Introduced in R2011b

## See Also

### Functions

`collect` | `combine` | `expand` | `factor` | `horner` | `numden` | `rewrite` | `simplify`

### Live Editor Tasks

Simplify Symbolic Expression

### Topics

“Simplify Symbolic Expressions” on page 3-133

“Choose Function to Rearrange Expression” on page 3-123

“Simplify Symbolic Expressions Using Live Editor Task” on page 3-138

# Simplify Symbolic Expression

Simplify symbolic expressions in Live Editor

## Description

The Simplify Symbolic Expression task enables you to interactively simplify or rearrange symbolic expressions. The task automatically generates MATLAB code for your live script. For more information about Live Editor tasks, see “Add Interactive Tasks to a Live Script”.

Using this task, you can:

- Perform algebraic simplification of symbolic expressions.
- Rewrite expressions in terms of another function.
- Expand algebraic expressions.
- Combine terms of identical algebraic structure.
- Generate the code used to simplify or rearrange expressions.

## Related Functions

The code that Simplify Symbolic Expression generates to simplify expressions uses these functions.

- `simplify`
- `simplifyFraction`
- `rewrite`
- `expand`
- `combine`

Simplify Symbolic Expression


☒ Autorun
 


simplifiedExpr

= Simplified expression **sumOfLog** using Combine

Select expression

Expression

sumOfLog ▼

Specify simplification method

Method

Combine ▼

Applied to

log ▼

☒ Ignore analytic constraints

Display result

☒ Expression
 ☒ Simplified expression

▼ Hide code

```

% Compute simplified symbolic expression
sumOfLog

sumOfLog = log(a) + log(b)

simplifiedExpr = combine(sumOfLog,"log","IgnoreAnalyticConstraints",true)

simplifiedExpr = log(a b)

```

## Description (collapsed portion)

### Related Functions

The code that Simplify Symbolic Expression generates to simplify expressions uses these functions.

- `simplify`
- `simplifyFraction`
- `rewrite`
- `expand`
- `combine`

## Open the Task

To add the Simplify Symbolic Expression task to a live script in the MATLAB Editor:

- On the **Live Editor** tab, select **Task > Simplify Symbolic Expression**.
- In a code block in your script, type a relevant keyword, such as `simplify`, `symbolic`, `rewrite`, `expand`, or `combine`. Select Simplify Symbolic Expression from the suggested command completions.

## Parameters

**Method** — Specify simplification method

`Simplify` (default) | `Simplify fraction` | `Rewrite` | `Expand` | `Combine`

Specify the simplification method from the drop-down list:

| Simplification Method          | Description                                                              |
|--------------------------------|--------------------------------------------------------------------------|
| <code>Simplify</code>          | Perform algebraic simplification.                                        |
| <code>Simplify fraction</code> | Simplify symbolic rational expressions.                                  |
| <code>Rewrite</code>           | Rewrite expressions in terms of another function.                        |
| <code>Expand</code>            | Expand expressions and simplify inputs of functions by using identities. |
| <code>Combine</code>           | Combine terms of identical algebraic structure.                          |

**Effort** — Specify computational effort used to simplify

`Minimum` (default) | `Low` | `Medium` | `High` | `Full`

Specify the computational effort used for the `Simplify` method from the drop-down list:

| Simplification Effort | Description                                                                       |
|-----------------------|-----------------------------------------------------------------------------------|
| <code>Minimum</code>  | Minimum effort with fastest computation time (can return most complicated result) |
| <code>Low</code>      | Low effort with faster computation time                                           |
| <code>Medium</code>   | Medium effort with normal computation time                                        |
| <code>High</code>     | High effort with slower computation time                                          |
| <code>Full</code>     | Full effort with slowest computation time (can return simplest result)            |

**Multiply out brackets** — Multiply out brackets when expanding expressions

`off` (default) | `on`

Select this check box to not expand special functions for the `Expand` method. This option expands the arithmetic part of an expression, such as powers and roots, without expanding trigonometric, hyperbolic, logarithmic, and special functions.

**Ignore analytic constraints** — Ignore analytic constraints when expanding expressions

`off` (default) | `on`

Select this check box to apply purely algebraic simplifications to the `Expand` method, such as  $\log(a) + \log(b) = \log(a*b)$  with the assumption that  $a$  and  $b$  are real positive numbers. Setting `Ignore analytic constraints` to `on` can give you simpler solutions, which could lead to results not generally valid. This option applies mathematical identities that are convenient for most engineering workflow, but do not always hold for all values of variables. In some cases, this option can lead to simpler results that are not equivalent to the initial expression. For details, see “Algorithms” on page 6-1359.

## Algorithms

When you use `Ignore analytic constraints`, then the simplification follows some of these rules:

- $\log(a) + \log(b) = \log(a \cdot b)$  for all values of  $a$  and  $b$ . In particular, the following equality is valid for all values of  $a$ ,  $b$ , and  $c$  :

$$(a \cdot b)^c = a^c \cdot b^c.$$

- $\log(a^b) = b \cdot \log(a)$  for all values of  $a$  and  $b$ . In particular, the following equality is valid for all values of  $a$ ,  $b$ , and  $c$  :

$$(a^b)^c = a^{b \cdot c}.$$

- If  $f$  and  $g$  are standard mathematical functions and  $f(g(x)) = x$  for all small positive numbers,  $f(g(x)) = x$  is assumed to be valid for all complex values of  $x$ . In particular:

- $\log(e^x) = x$
- $\text{asin}(\sin(x)) = x$ ,  $\text{acos}(\cos(x)) = x$ ,  $\text{atan}(\tan(x)) = x$
- $\text{asinh}(\sinh(x)) = x$ ,  $\text{acosh}(\cosh(x)) = x$ ,  $\text{atanh}(\tanh(x)) = x$
- $W_k(x \cdot e^x) = x$  for all branch indices  $k$  of the Lambert W function.

## Version History

Introduced in R2020a

## See Also

### Functions

`simplify` | `simplifyFraction` | `combine` | `expand` | `rewrite`

### Live Editor Tasks

Solve Symbolic Equation

### Topics

"Add Interactive Tasks to a Live Script"

"Simplify Symbolic Expressions Using Live Editor Task" on page 3-138

"Choose Function to Rearrange Expression" on page 3-123

"Simplify Symbolic Expressions" on page 3-133

## simscapeEquation

Convert symbolic expressions to Simscape language equations

### Syntax

```
simscapeEquation(f)
simscapeEquation(LHS,RHS)
```

### Description

`simscapeEquation(f)` converts the symbolic expression `f` to a Simscape language equation. This function call converts any derivative with respect to the variable `t` to the Simscape notation `X.der`. Here `X` is the time-dependent variable. In the resulting Simscape equation, the variable `time` replaces all instances of the variable `t` except for derivatives with respect to `t`.

`simscapeEquation` converts expressions with the second and higher-order derivatives to a system of first-order equations, introducing new variables, such as `x1`, `x2`, and so on.

`simscapeEquation(LHS,RHS)` returns a Simscape equation `LHS == RHS`.

### Examples

#### Convert Expressions to Simscape Equations

Convert the following expressions to Simscape language equations.

```
syms t x(t) y(t)
phi = diff(x) + 5*y + sin(t);
simscapeEquation(phi)
simscapeEquation(diff(y),phi)

ans =
    'phi == sin(time)+y*5.0+x.der;'

ans =
    'y.der == sin(time)+y*5.0+x.der;'
```

#### Convert ODE to Simscape Equation

Convert this expression containing the second derivative.

```
syms x(t)
eqn1 = diff(x,2) - diff(x) + sin(t);
simscapeEquation(eqn1)

ans =
    'x.der == x1;
    eqn1 == sin(time)-x1+x1.der;'
```

Convert this expression containing the fourth and second derivatives.



```
eqn2 = diff(x,4) + diff(x,2) - diff(x) + sin(t);  
simscapeEquation(eqn2)
```

```
ans =  
    'x.der == x1;  
    x1.der == x2;  
    x2.der == x3;  
    eqn2 == sin(time)-x1+x2+x3.der;'
```

## Tips

- The equation section of a Simscape component file supports a limited number of functions. For details and the list of supported functions, see [Simscape equations \(Simscape\)](#). If a symbolic equation contains functions that are not available in the equation section of a Simscape component file, `simscapeEquation` cannot convert these equations correctly to Simscape equations. Such expressions do not trigger an error message. Expressions with infinities are prone to invalid conversion.

## Version History

**Introduced in R2010a**

## See Also

[matlabFunctionBlock](#) | [matlabFunction](#) | [ccode](#) | [fortran](#) | [symWriteSSC](#)

## Topics

“Generate Simscape Equations from Symbolic Expressions” on page 5-8

## sin

Symbolic sine function

### Syntax

`sin(X)`

### Description

`sin(X)` returns the sine function on page 6-1364 of X.

### Examples

#### Sine Function for Numeric and Symbolic Arguments

Depending on its arguments, `sin` returns floating-point or exact symbolic results.

Compute the sine function for these numbers. Because these numbers are not symbolic objects, `sin` returns floating-point results.

```
A = sin([-2, -pi, pi/6, 5*pi/7, 11])  
  
A =  
   -0.9093   -0.0000    0.5000    0.7818   -1.0000
```

Compute the sine function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `sin` returns unresolved symbolic calls.

```
symA = sin(sym([-2, -pi, pi/6, 5*pi/7, 11]))  
  
symA =  
[ -sin(2), 0, 1/2, sin((2*pi)/7), sin(11)]
```

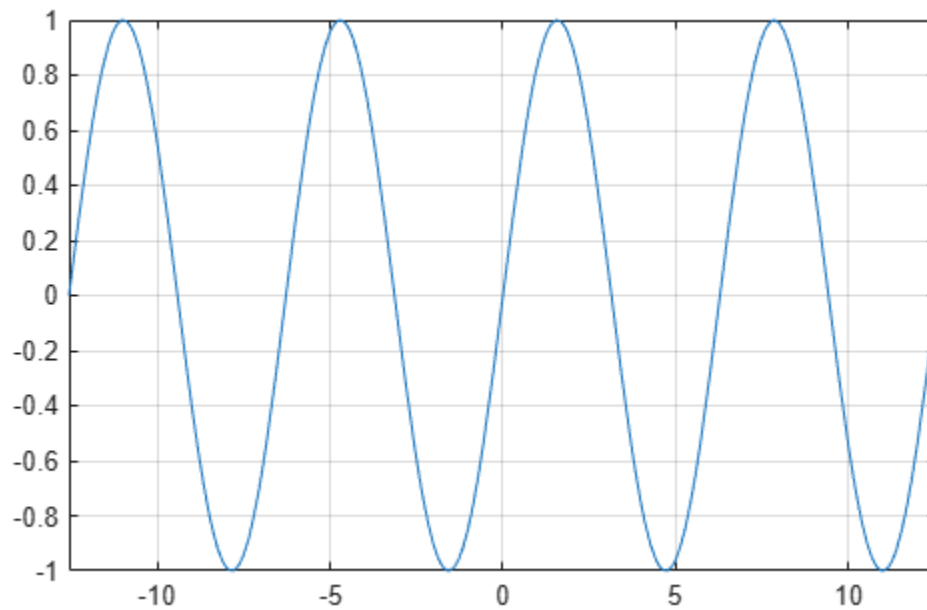
Use `vpa` to approximate symbolic results with floating-point numbers:

```
vpa(symA)  
  
ans =  
[ -0.90929742682568169539601986591174,...  
  0,...  
  0.5,...  
  0.78183148246802980870844452667406,...  
 -0.99999020655070345705156489902552]
```

#### Plot Sine Function

Plot the sine function on the interval from  $-4\pi$  to  $4\pi$ .

```
syms x  
fplot(sin(x),[-4*pi 4*pi])  
grid on
```



### Handle Expressions Containing Sine Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `sin`.

Find the first and second derivatives of the sine function:

```
syms x
diff(sin(x), x)
diff(sin(x), x, x)
```

```
ans =
cos(x)
```

```
ans =
-sin(x)
```

Find the indefinite integral of the sine function:

```
int(sin(x), x)
```

```
ans =
-cos(x)
```

Find the Taylor series expansion of `sin(x)`:

```
taylor(sin(x), x)
```

```
ans =
x^5/120 - x^3/6 + x
```

Rewrite the sine function in terms of the exponential function:

```
rewrite(sin(x), 'exp')
```

```
ans =  
(exp(-x*1i)*1i)/2 - (exp(x*1i)*1i)/2
```

### Evaluate Units with `sin` Function

`sin` numerically evaluates these units automatically: radian, degree, arcmin, arcsec, and revolution.

Show this behavior by finding the sine of `x` degrees and 2 radians.

```
u = symunit;  
syms x  
f = [x*u.degree 2*u.radian];  
sinf = sin(f)  
  
sinf =  
[ sin((pi*x)/180), sin(2)]
```

You can calculate `sinf` by substituting for `x` using `subs` and then using `double` or `vpa`.

## Input Arguments

### X — Input

symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic expression |  
symbolic function | symbolic matrix function | symbolic vector | symbolic matrix

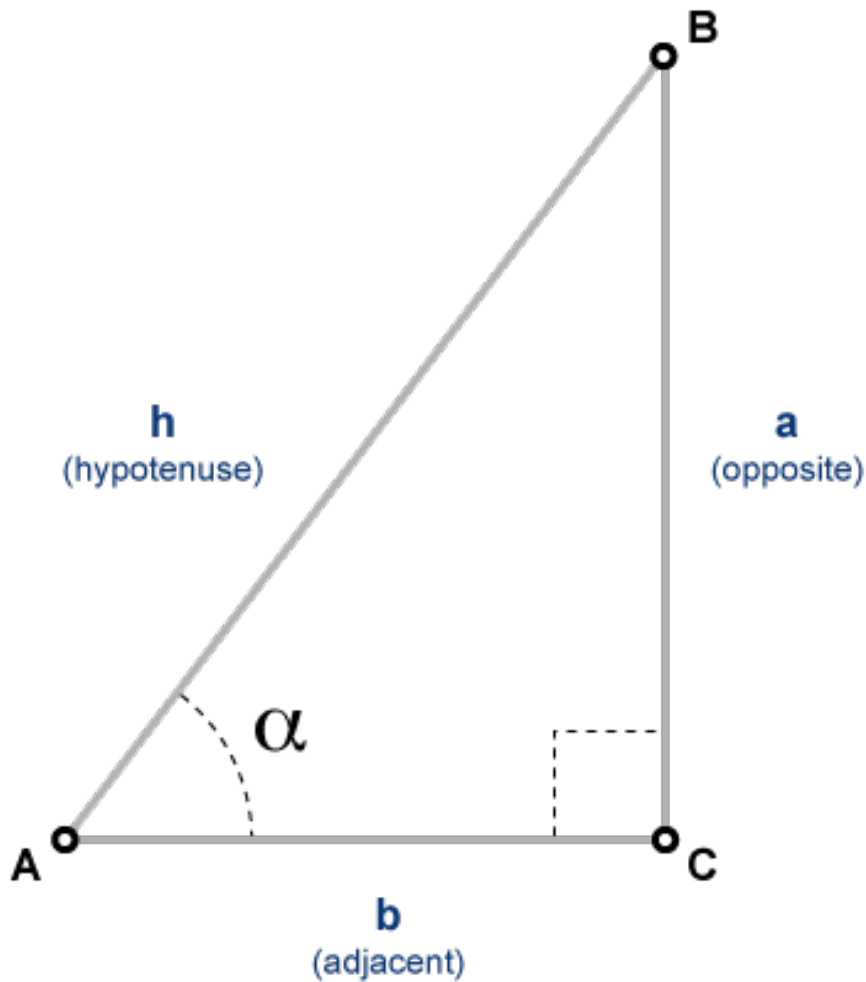
Input, specified as a symbolic number, scalar variable, matrix variable, expression, function, matrix function, or as a vector or matrix of symbolic numbers, scalar variables, expressions, or functions.

## More About

### Sine Function

The sine of an angle,  $\alpha$ , defined with reference to a right triangle is

$$\sin(\alpha) = \frac{\text{opposite side}}{\text{hypotenuse}} = \frac{a}{h} .$$



The sine of a complex argument,  $\alpha$ , is

$$\sin(\alpha) = \frac{e^{i\alpha} - e^{-i\alpha}}{2i}.$$

## Version History

Introduced before R2006a

**R2022a: Compute sine of symbolic matrix functions**

The `sin` function accepts an input argument of type `symfunmatrix`.

**R2021a: Compute sine of symbolic matrix variables**

The `sin` function accepts an input argument of type `symmatrix`.

**See Also**

`acos` | `asin` | `atan` | `acsc` | `asec` | `acot` | `cos` | `tan` | `csc` | `sec` | `sinc` | `cot`

# sinc

Normalized sinc function

## Syntax

`sinc(x)`

## Description

`sinc(x)` returns  $\sin(\pi x)/(\pi x)$ . The symbolic `sinc` function does not implement floating-point results, only symbolic results. Floating-point results are returned by the `sinc` function in Signal Processing Toolbox™.

## Examples

### Sinc Function of Symbolic Inputs

Create a `sinc` function of a symbolic variable `x`.

```
syms x
sinc(x)

ans =

$$\frac{\sin(\pi x)}{x \pi}$$

```

Show that `sinc` returns 1 at 0, 0 at other integer inputs, and exact symbolic values for other inputs.

```
V = sym([-1 0 1 3/2]);
S = sinc(V)

S =

$$\begin{pmatrix} 0 & 1 & 0 & -\frac{2}{3\pi} \end{pmatrix}$$

```

Convert the exact symbolic output to high-precision floating point by using `vpa`.

```
vpa(S)

ans = (0 1.0 0 -0.21220659078919378102517835116335)
```

### Fourier Transforms Involving Sinc Function

Although `sinc` can appear in tables of Fourier transforms, `fourier` does not return `sinc` in output.

Show that `fourier` transforms a pulse in terms of `sin` and `cos`.

```
syms x
fourier(rectangularPulse(x))
```

$$\text{ans} = \frac{\sin\left(\frac{w}{2}\right) + \cos\left(\frac{w}{2}\right) i}{w} - \frac{-\sin\left(\frac{w}{2}\right) + \cos\left(\frac{w}{2}\right) i}{w}$$

Show that fourier transforms sinc in terms of heaviside.

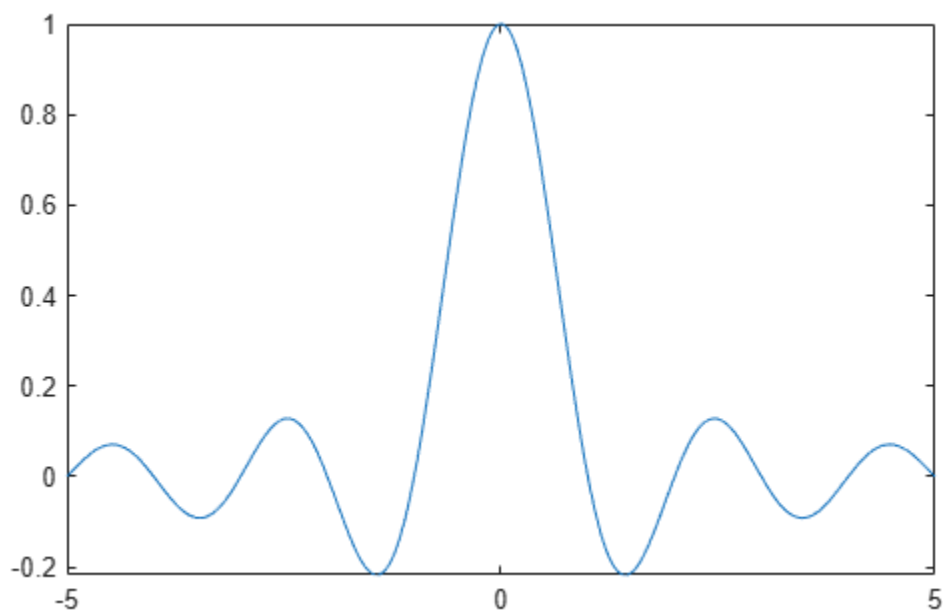
```
fourier(sinc(x))
```

$$\text{ans} = \frac{\pi \operatorname{heaviside}(\pi - w) - \pi \operatorname{heaviside}(-w - \pi)}{\pi}$$

### Plot Sinc Function

Plot the sinc function by using `fplot`.

```
syms x
fplot(sinc(x))
```



### Rewrite Sinc Function to Other Functions

Rewrite the sinc function to the exponential function `exp` by using `rewrite`.

```
syms x
rewrite(sinc(x), 'exp')
```

```
ans =
```



$$\frac{\frac{e^{-\pi x i}}{2} - \frac{e^{\pi x i}}{2}}{x \pi}$$

### Differentiate, Integrate, and Expand the Sinc Function

Differentiate, integrate, and expand sinc by using the `diff`, `int`, and `taylor` functions, respectively.

Differentiate `sinc`.

```
syms x
diff(sinc(x))

ans =

$$\frac{\cos(\pi x)}{x} - \frac{\sin(\pi x)}{x^2 \pi}$$

```

Integrate `sinc` from `-Inf` to `Inf`.

```
int(sinc(x), [-Inf Inf])
```

```
ans = 1
```

Integrate `sinc` from `-Inf` to `x`.

```
int(sinc(x), -Inf, x)
```

```
ans =

$$\frac{\text{sinint}(\pi x)}{\pi} + \frac{1}{2}$$

```

Find the Taylor expansion of `sinc`.

```
taylor(sinc(x))
```

```
ans =

$$\frac{\pi^4 x^4}{120} - \frac{\pi^2 x^2}{6} + 1$$

```

### Prove Identity Involving Sinc Function

Prove an identity by defining the identity as a condition and using the `isAlways` function to check the condition.

Prove the identity  $\text{sinc}(x) = \frac{1}{\Gamma(1+x)\Gamma(1-x)}$ .

```
syms x
cond = sinc(x) == 1/(gamma(1+x)*gamma(1-x));
isAlways(cond)
```

```
ans = logical  
    1
```

## Input Arguments

### **x — Input**

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## Version History

**Introduced in R2018b**

### **See Also**

`sin`

# sinh

Symbolic hyperbolic sine function

## Syntax

`sinh(X)`

## Description

`sinh(X)` returns the hyperbolic sine function of  $X$ .

## Examples

### Hyperbolic Sine Function for Numeric and Symbolic Arguments

Depending on its arguments, `sinh` returns floating-point or exact symbolic results.

Compute the hyperbolic sine function for these numbers. Because these numbers are not symbolic objects, `sinh` returns floating-point results.

```
A = sinh([-2, -pi*i, pi*i/6, 5*pi*i/7, 3*pi*i/2])
A =
-3.6269 + 0.0000i    0.0000 - 0.0000i    0.0000 + 0.5000i...
0.0000 + 0.7818i    0.0000 - 1.0000i
```

Compute the hyperbolic sine function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `sinh` returns unresolved symbolic calls.

```
symA = sinh(sym([-2, -pi*i, pi*i/6, 5*pi*i/7, 3*pi*i/2]))
symA =
[ -sinh(2), 0, 1i/2, sinh((pi*2i)/7), -1i]
```

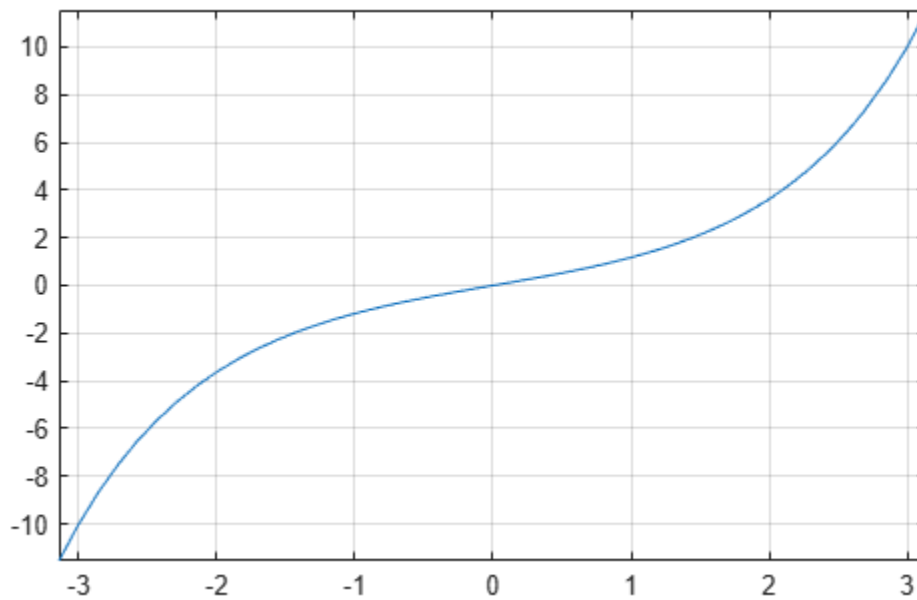
Use `vpa` to approximate symbolic results with floating-point numbers:

```
vpa(symA)
ans =
[ -3.6268604078470187676682139828013,...
0,...
0.5i,...
0.78183148246802980870844452667406i,...
-1.0i]
```

### Plot Hyperbolic Sine Function

Plot the hyperbolic sine function on the interval from  $-\pi$  to  $\pi$ .

```
syms x
fplot(sinh(x),[-pi pi])
grid on
```



### Handle Expressions Containing Hyperbolic Sine Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `sinh`.

Find the first and second derivatives of the hyperbolic sine function:

```
syms x
diff(sinh(x), x)
diff(sinh(x), x, x)
```

```
ans =
cosh(x)
```

```
ans =
sinh(x)
```

Find the indefinite integral of the hyperbolic sine function:

```
int(sinh(x), x)
```

```
ans =
cosh(x)
```

Find the Taylor series expansion of `sinh(x)`:

```
taylor(sinh(x), x)
```

```
ans =
x^5/120 + x^3/6 + x
```

Rewrite the hyperbolic sine function in terms of the exponential function:

```
rewrite(sinh(x), 'exp')
```

```
ans =  
exp(x)/2 - exp(-x)/2
```

## Input Arguments

### **X — Input**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Version History

**Introduced before R2006a**

### **See Also**

acosh | asinh | atanh | acsch | asech | acoth | cosh | tanh | csch | sech | coth

## sinhint

Hyperbolic sine integral function

### Syntax

```
sinhint(X)
```

### Description

`sinhint(X)` returns the hyperbolic sine integral function on page 6-1376 of X.

### Examples

#### Hyperbolic Sine Integral Function for Numeric and Symbolic Arguments

Depending on its arguments, `sinhint` returns floating-point or exact symbolic results.

Compute the hyperbolic sine integral function for these numbers. Because these numbers are not symbolic objects, `sinhint` returns floating-point results.

```
A = sinhint([-pi, -1, 0, pi/2, 2*pi])  
  
A =  
    -5.4696    -1.0573         0     1.8027    53.7368
```

Compute the hyperbolic sine integral function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `sinhint` returns unresolved symbolic calls.

```
symA = sinhint(sym([-pi, -1, 0, pi/2, 2*pi]))  
  
symA =  
[ -sinhint(pi), -sinhint(1), 0, sinhint(pi/2), sinhint(2*pi)]
```

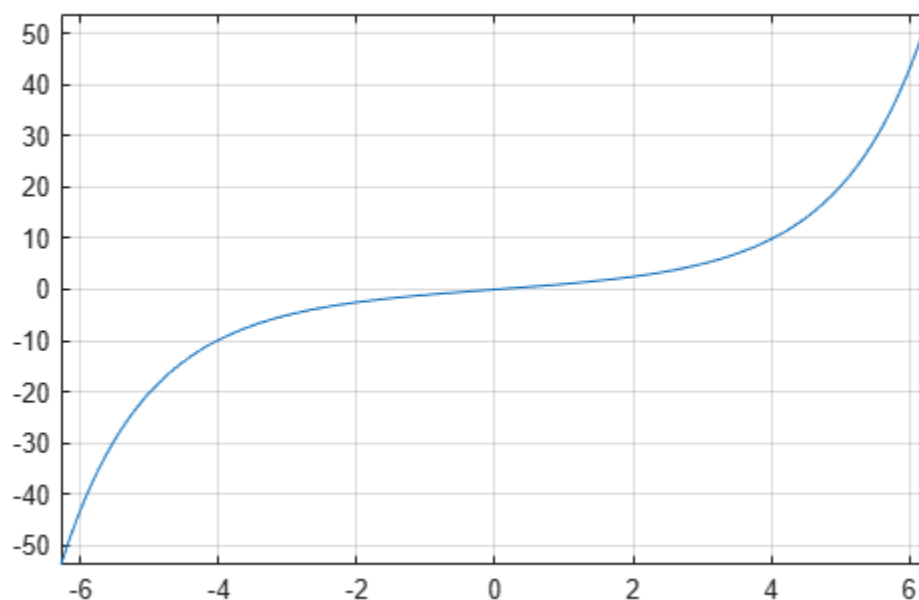
Use `vpa` to approximate symbolic results with floating-point numbers:

```
vpa(symA)  
  
ans =  
[ -5.4696403451153421506369580091277,...  
 -1.0572508753757285145718423548959,...  
 0,...  
 1.802743198288293882089794577617,...  
 53.736750620859153990408011863262]
```

#### Plot Hyperbolic Sine Integral Function

Plot the hyperbolic sine integral function on the interval from  $-2\pi$  to  $2\pi$ .

```
syms x  
fplot(sinhint(x),[-2*pi 2*pi])  
grid on
```



### Handle Expressions Containing Hyperbolic Sine Integral Function

Many functions, such as `diff`, `int`, and `taylor`, can handle expressions containing `sinhint`.

Find the first and second derivatives of the hyperbolic sine integral function:

```
syms x
diff(sinhint(x), x)
diff(sinhint(x), x, x)

ans =
sinh(x)/x

ans =
cosh(x)/x - sinh(x)/x^2
```

Find the indefinite integral of the hyperbolic sine integral function:

```
int(sinhint(x), x)

ans =
x*sinhint(x) - cosh(x)
```

Find the Taylor series expansion of `sinhint(x)`:

```
taylor(sinhint(x), x)

ans =
x^5/600 + x^3/18 + x
```

## Input Arguments

### X — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## More About

### Hyperbolic Sine Integral Function

The hyperbolic sine integral function is defined as follows:

$$\text{Shi}(x) = \int_0^x \frac{\sinh(t)}{t} dt$$

## Version History

Introduced in R2014a

## References

- [1] Gautschi, W. and W. F. Cahill. "Exponential Integral and Related Functions." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

coshint | cosint | eulergamma | int | sin | sinint | ssinint



# sinint

Sine integral function

## Syntax

`sinint(X)`

## Description

`sinint(X)` returns the sine integral function of  $X$ .

## Examples

### Sine Integral Function for Numeric and Symbolic Arguments

Depending on its arguments, `sinint` returns floating-point or exact symbolic results.

Compute the sine integral function for these numbers. Because these numbers are not symbolic objects, `sinint` returns floating-point results.

```
A = sinint([- pi, 0, pi/2, pi, 1])
```

```
A =  
-1.8519      0      1.3708      1.8519      0.9461
```

Compute the sine integral function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `sinint` returns unresolved symbolic calls.

```
symA = sinint(sym([- pi, 0, pi/2, pi, 1]))
```

```
symA =  
[ -sinint(pi), 0, sinint(pi/2), sinint(pi), sinint(1)]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

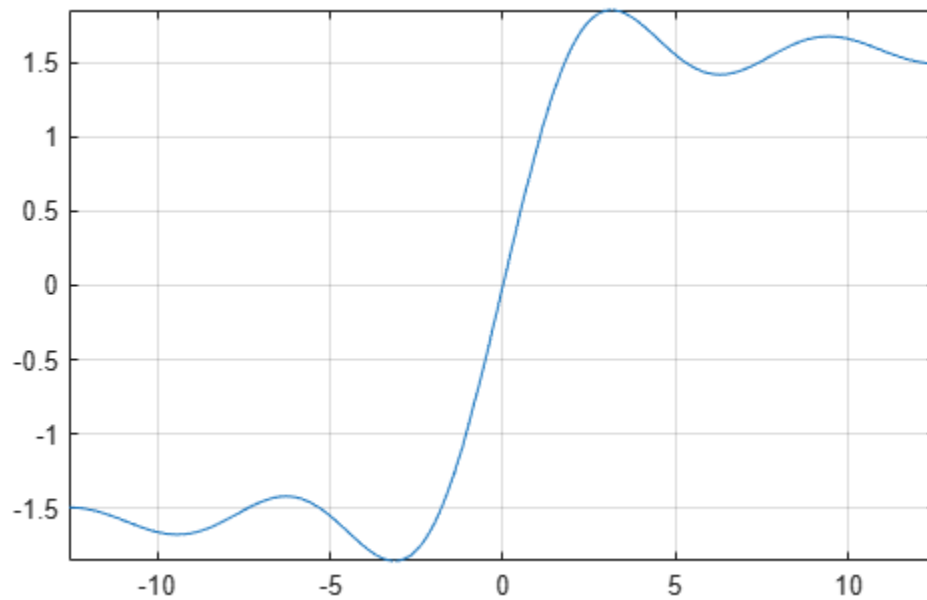
```
vpa(symA)
```

```
ans =  
[ -1.851937051982466170361053370158,...  
0,...  
1.3707621681544884800696782883816,...  
1.851937051982466170361053370158,...  
0.94608307036718301494135331382318]
```

### Plot Sine Integral Function

Plot the sine integral function on the interval from  $-4\pi$  to  $4\pi$ .

```
syms x  
fplot(sinint(x),[-4*pi 4*pi])  
grid on
```



### Handle Expressions Containing Sine Integral Function

Many functions, such as `diff`, `int`, and `taylor`, can handle expressions containing `sinint`.

Find the first and second derivatives of the sine integral function:

```
syms x
diff(sinint(x), x)
diff(sinint(x), x, x)
```

```
ans =
sin(x)/x
```

```
ans =
cos(x)/x - sin(x)/x^2
```

Find the indefinite integral of the sine integral function:

```
int(sinint(x), x)
```

```
ans =
cos(x) + x*sinint(x)
```

Find the Taylor series expansion of `sinint(x)`:

```
taylor(sinint(x), x)
```

```
ans =
x^5/600 - x^3/18 + x
```

## Input Arguments

### X — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## More About

### Sine Integral Function

The sine integral function is defined as follows:

$$\text{Si}(x) = \int_0^x \frac{\sin(t)}{t} dt$$

## Version History

Introduced before R2006a

## References

- [1] Gautschi, W. and W. F. Cahill. "Exponential Integral and Related Functions." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

coshint | cosint | eulergamma | int | sin | sinhint | ssinint

## smithForm

Smith form of matrix

### Syntax

```
S = smithForm(A)
[U,V,S] = smithForm(A)

___ = smithForm(A,var)
```

### Description

`S = smithForm(A)` returns the Smith normal form on page 6-1384 of a square invertible matrix `A`. The elements of `A` must be integers or polynomials in a variable determined by `symvar(A,1)`. The Smith form `S` is a diagonal matrix.

`[U,V,S] = smithForm(A)` returns the Smith normal form of `A` and unimodular transformation matrices `U` and `V`, such that  $S = U \cdot A \cdot V$ .

`___ = smithForm(A,var)` assumes that the elements of `A` are univariate polynomials in the specified variable `var`. If `A` contains other variables, `smithForm` treats those variables as symbolic parameters.

You can use the input argument `var` in any of the previous syntaxes.

If `A` does not contain `var`, then `smithForm(A)` and `smithForm(A,var)` return different results.

### Examples

#### Smith Form for Matrix of Integers

Find the Smith form of an inverse Hilbert matrix.

```
A = sym(invhilb(5))
S = smithForm(A)

A =
[ 25, -300, 1050, -1400, 630]
[ -300, 4800, -18900, 26880, -12600]
[ 1050, -18900, 79380, -117600, 56700]
[ -1400, 26880, -117600, 179200, -88200]
[ 630, -12600, 56700, -88200, 44100]

S =
[ 5, 0, 0, 0, 0]
[ 0, 60, 0, 0, 0]
[ 0, 0, 420, 0, 0]
[ 0, 0, 0, 840, 0]
[ 0, 0, 0, 0, 2520]
```

### Smith Form for Matrix of Univariate Polynomials

Create a 2-by-2 matrix, the elements of which are polynomials in the variable  $x$ .

```
syms x
A = [x^2 + 3, (2*x - 1)^2; (x + 2)^2, 3*x^2 + 5]

A =
[ x^2 + 3, (2*x - 1)^2]
[ (x + 2)^2, 3*x^2 + 5]
```

Find the Smith form of this matrix.

```
S = smithForm(A)

S =
[ 1, 0]
[ 0, x^4 + 12*x^3 - 13*x^2 - 12*x - 11]
```

### Smith Form for Matrix of Multivariate Polynomials

Create a 2-by-2 matrix containing two variables:  $x$  and  $y$ .

```
syms x y
A = [2/x + y, x^2 - y^2; 3*sin(x) + y, x]

A =
[ y + 2/x, x^2 - y^2]
[ y + 3*sin(x), x]
```

Find the Smith form of this matrix. If you do not specify the polynomial variable, `smithForm` uses `symvar(A,1)` and thus determines that the polynomial variable is  $x$ . Because  $3*\sin(x) + y$  is not a polynomial in  $x$ , `smithForm` throws an error.

```
S = smithForm(A)

Error using mupadengine/feval (line 163)
Cannot convert the matrix entries to integers or univariate polynomials.
```

Find the Smith form of  $A$  specifying that all elements of  $A$  are polynomials in the variable  $y$ .

```
S = smithForm(A,y)

S =
[ 1, 0]
[ 0, 3*y^2*sin(x) - 3*x^2*sin(x) + y^3 + y*(- x^2 + x) + 2]
```

### Smith Form and Transformation Matrices

Find the Smith form and transformation matrices for an inverse Hilbert matrix.

```
A = sym(invhilb(3));
[U,V,S] = smithForm(A)

U =
[ 1, 1, 1]
[ -4, -1, 0]
[ 10, 5, 3]
```

```
V =
[ 1, -2, 0]
[ 0, 1, 5]
[ 0, 1, 4]
```

```
S =
[ 3, 0, 0]
[ 0, 12, 0]
[ 0, 0, 60]
```

Verify that  $S = U \cdot A \cdot V$ .

```
isAlways(S == U*A*V)
```

```
ans =
3x3 logical array
    1     1     1
    1     1     1
    1     1     1
```

Find the Smith form and transformation matrices for a matrix of polynomials.

```
syms x y
A = [2*(x - y), 3*(x^2 - y^2);
     4*(x^3 - y^3), 5*(x^4 - y^4)];
[U,V,S] = smithForm(A,x)
```

```
U =
[ 0, 1]
[ 1, - x/(10*y^3) - 3/(5*y^2)]
```

```
V =
[ -x/(4*y^3), - (5*x*y^2)/2 - (5*x^2*y)/2 - (5*x^3)/2 - (5*y^3)/2]
[ 1/(5*y^3), 2*x^2 + 2*x*y + 2*y^2]
```

```
S =
[ x - y, 0]
[ 0, x^4 + 6*x^3*y - 6*x*y^3 - y^4]
```

Verify that  $S = U \cdot A \cdot V$ .

```
isAlways(S == U*A*V)
```

```
ans =
2x2 logical array
    1     1
    1     1
```

### If You Specify Variable for Integer Matrix

If a matrix does not contain a particular variable, and you call `smithForm` specifying that variable as the second argument, then the result differs from what you get without specifying that variable. For example, create a matrix that does not contain any variables.

```
A = [9 -36 30; -36 192 -180; 30 -180 180]
```

```
A =
    9   -36    30
```

```
-36   192  -180
 30  -180   180
```

Call `smithForm` specifying variable `x` as the second argument. In this case, `smithForm` assumes that the elements of `A` are univariate polynomials in `x`.

```
syms x
smithForm(A,x)
```

```
ans =
     1     0     0
     0     1     0
     0     0     1
```

Call `smithForm` without specifying variables. In this case, `smithForm` treats `A` as a matrix of integers.

```
smithForm(A)
```

```
ans =
     3     0     0
     0    12     0
     0     0    60
```

## Input Arguments

### **A — Input matrix**

square invertible symbolic matrix

Input matrix, specified as a square invertible symbolic matrix, the elements of which are integers or univariate polynomials. If the elements of `A` contain more than one variable, use the `var` argument to specify a polynomial variable, and treat all other variables as symbolic parameters. If `A` is multivariate, and you do not specify `var`, then `smithForm` uses `symvar(A,1)` to determine a polynomial variable.

### **var — Polynomial variable**

symbolic variable

Polynomial variable, specified as a symbolic variable.

## Output Arguments

### **S — Smith normal form of input matrix**

symbolic diagonal matrix

Smith normal form of input matrix, returned as a symbolic diagonal matrix. The first diagonal element divides the second, the second divides the third, and so on.

### **U — Transformation matrix**

unimodular symbolic matrix

Transformation matrix, returned as a unimodular symbolic matrix. If elements of `A` are integers, then elements of `U` are also integers, and  $\det(U) = 1$  or  $\det(U) = -1$ . If elements of `A` are polynomials, then elements of `U` are univariate polynomials, and  $\det(U)$  is a constant.

**V — Transformation matrix**

unimodular symbolic matrix

Transformation matrix, returned as a unimodular symbolic matrix. If elements of  $A$  are integers, then elements of  $V$  are also integers, and  $\det(V) = 1$  or  $\det(V) = -1$ . If elements of  $A$  are polynomials, then elements of  $V$  are univariate polynomials, and  $\det(V)$  is a constant.

**More About****Smith Normal Form**

Smith normal form of an  $n$ -by- $n$  matrix  $A$  is an  $n$ -by- $n$  diagonal matrix  $S$ , such that  $S_{i,i}$  divides  $S_{i+1,i+1}$  for all  $i < n$ .

**Version History**

Introduced in R2015b

**See Also**

hermiteForm | jordan



# solve

Equations and systems solver

## Syntax

```
S = solve(eqn,var)
S = solve(eqn,var,Name=Value)

Y = solve(eqns,vars)
Y = solve(eqns,vars,Name=Value)

[y1,...,yN] = solve(eqns,vars)
[y1,...,yN] = solve(eqns,vars,Name=Value)
[y1,...,yN,parameters,conditions] = solve(eqns,vars,ReturnConditions=true)
```

## Description

`S = solve(eqn,var)` solves the symbolic equation `eqn` for the variable `var`. If you do not specify `var`, the `symvar` function determines the variable to solve for. For example, `solve(x + 1 == 2, x)` solves the equation  $x + 1 = 2$  for  $x$ .

`S = solve(eqn,var,Name=Value)` uses additional options specified by one or more `Name=Value` arguments.

`Y = solve(eqns,vars)` solves the system of equations `eqns` for the variables `vars` and returns a structure that contains the solutions. If you do not specify `vars`, `solve` uses `symvar` to find the variables to solve for. In this case, the number of variables that `symvar` finds is equal to the number of equations `eqns`.

`Y = solve(eqns,vars,Name=Value)` uses additional options specified by one or more `Name=Value` arguments.

`[y1,...,yN] = solve(eqns,vars)` solves the system of equations `eqns` for the variables `vars`. The solutions are assigned to the variables `y1,...,yN`. If you do not specify the variables, `solve` uses `symvar` to find the variables to solve for. In this case, the number of variables that `symvar` finds is equal to the number of output arguments `N`.

`[y1,...,yN] = solve(eqns,vars,Name=Value)` uses additional options specified by one or more `Name=Value` arguments.

`[y1,...,yN,parameters,conditions] = solve(eqns,vars,ReturnConditions=true)` returns the additional arguments `parameters` and `conditions` that specify the parameters in the solution and the conditions on the solution.

## Examples

**Solve Quadratic Equation**

Solve the quadratic equation without specifying a variable to solve for. `solve` chooses `x` to return the solution.

```
syms a b c x
eqn = a*x^2 + b*x + c == 0
```

$$\text{eqn} = ax^2 + bx + c = 0$$

```
S = solve(eqn)
```

$$S = \begin{pmatrix} -\frac{b + \sqrt{b^2 - 4ac}}{2a} \\ -\frac{b - \sqrt{b^2 - 4ac}}{2a} \end{pmatrix}$$

Specify the variable to solve for and solve the quadratic equation for `a`.

```
Sa = solve(eqn,a)
```

$$S_a = -\frac{c + bx}{x^2}$$

**Solve Polynomial and Return Real Solutions**

Solve a fifth-degree polynomial. It has five solutions.

```
syms x
eqn = x^5 == 3125;
S = solve(eqn,x)
```

$$S = \begin{pmatrix} 5 \\ -\sigma_1 - \frac{5}{4} - \frac{5\sqrt{2}\sqrt{5-\sqrt{5}}i}{4} \\ -\sigma_1 - \frac{5}{4} + \frac{5\sqrt{2}\sqrt{5-\sqrt{5}}i}{4} \\ \sigma_1 - \frac{5}{4} - \frac{5\sqrt{2}\sqrt{\sqrt{5}+5}i}{4} \\ \sigma_1 - \frac{5}{4} + \frac{5\sqrt{2}\sqrt{\sqrt{5}+5}i}{4} \end{pmatrix}$$

where

$$\sigma_1 = \frac{5\sqrt{5}}{4}$$

Return only real solutions by setting the `Real` argument to `true`. The only real solutions of this equation is 5.

```
S = solve(eqn,x,Real=true)
```

`S = 5`

### Numerically Solve Equations

When `solve` cannot symbolically solve an equation, it tries to find a numeric solution using `vpasolve`. The `vpasolve` function returns the first solution found.

Try solving the following equation. `solve` returns a numeric solution because it cannot find a symbolic solution.

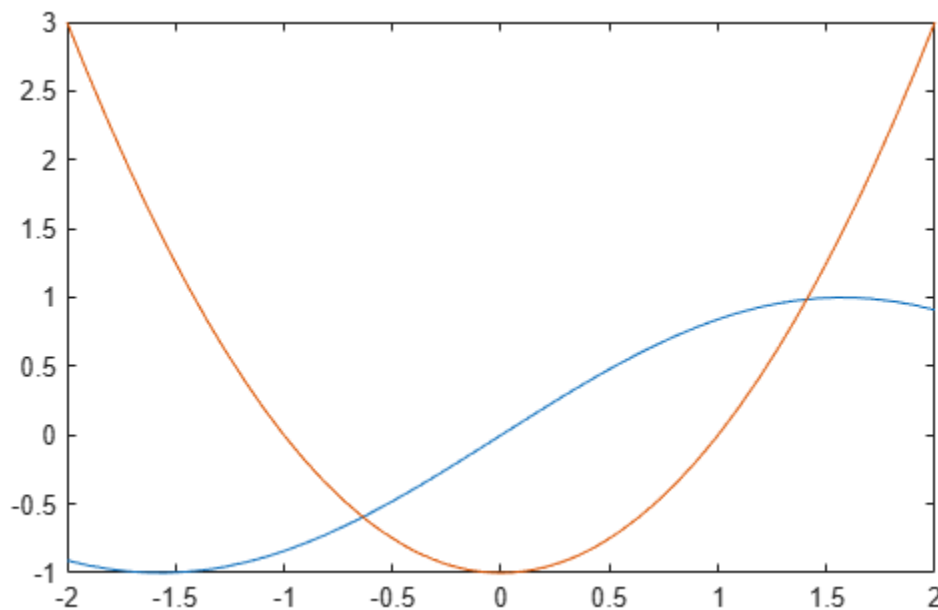
```
syms x
eqn = sin(x) == x^2 - 1;
S = solve(eqn,x)
```

Warning: Unable to solve symbolically. Returning a numeric solution using [matlab:web\(fu](matlab:web(fu)

```
S = -0.63673265080528201088799090383828
```

Plot the left and the right sides of the equation. Observe that the equation also has a positive solution.

```
fplot([lhs(eqn) rhs(eqn)], [-2 2])
```



Find the other solution by directly calling the numeric solver `vpasolve` and specifying the interval.

```
V = vpasolve(eqn,x,[0 2])
```

```
V = 1.4096240040025962492355939705895
```

### Solve Multivariate Equations and Assign Outputs to Structure

When solving for multiple variables, it can be more convenient to store the outputs in a structure array than in separate variables. The `solve` function returns a structure when you specify a single output argument and multiple outputs exist.

Solve a system of equations to return the solutions in a structure array.

```
syms u v
eqns = [2*u + v == 0, u - v == 1];
S = solve(eqns,[u v])
```

```
S = struct with fields:
  u: 1/3
  v: -2/3
```

Access the solutions by addressing the elements of the structure.

```
S.u
```

```
ans =
    1/3
```

```
S.v
```

```
ans =
   -2/3
```

Using a structure array allows you to conveniently substitute solutions into other expressions.

Use the `subs` function to substitute the solutions `S` into other expressions.

```
expr1 = u^2;
e1 = subs(expr1,S)
```

```
e1 =
    1/9
```

```
expr2 = 3*v + u;
e2 = subs(expr2,S)
```

```
e2 =
   -5/3
```

If `solve` returns an empty object, then no solutions exist.

```
eqns = [3*u+2, 3*u+1];
S = solve(eqns,u)
```

```
S =
```

```
Empty sym: 0-by-1
```

## Solve Inequalities

The `solve` function can solve inequalities and return solutions that satisfy the inequalities. Solve the following inequalities.

$$x > 0$$

$$y > 0$$

$$x^2 + y^2 + xy < 1$$

Set `ReturnConditions` to `true` to return any parameters in the solution and conditions on the solution.

```
syms x y
eqn1 = x > 0;
eqn2 = y > 0;
eqn3 = x^2 + y^2 + x*y < 1;
eqns = [eqn1 eqn2 eqn3];

S = solve(eqns,[x y],ReturnConditions=true);
S.x
```

$$\text{ans} = \frac{\sqrt{u - 3v^2}}{2} - \frac{v}{2}$$

S.y

$$\text{ans} = v$$

S.parameters

$$\text{ans} = (u \ v)$$

S.conditions

$$\text{ans} = 4v^2 < u \wedge u < 4 \wedge 0 < v$$

The parameters `u` and `v` do not exist in MATLAB® workspace and must be accessed using `S.parameters`.

Check if the values `u = 7/2` and `v = 1/2` satisfy the condition using `subs` and `isAlways`.

```
condWithValues = subs(S.conditions, S.parameters, [7/2,1/2]);
isAlways(condWithValues)
```

$$\text{ans} = \text{logical}$$

$$1$$

`isAlways` returns logical 1 (true) indicating that these values satisfy the condition. Substitute these parameter values into `S.x` and `S.y` to find a solution for `x` and `y`.

```
xSol = subs(S.x, S.parameters, [7/2,1/2])
```

$$\text{xSol} =$$

$$\frac{\sqrt{11}}{4} - \frac{1}{4}$$

```
ySol = subs(S.y, S.parameters, [7/2,1/2])
```

```
ySol =  
1  
2
```

### Solve Multivariate Equations and Assign Outputs to Variables

Solve the system of equations.

$$2u^2 + v^2 = 0$$

$$u - v = 1$$

When solving for more than one variable, the order in which you specify the variables defines the order in which the solver returns the solutions. Assign the solutions to variables `solv` and `solu` by specifying the variables explicitly. The solver returns an array of solutions for each variable.

```
syms u v  
eqns = [2*u^2 + v^2 == 0, u - v == 1];  
vars = [v u];  
[solv, solu] = solve(eqns,vars)
```

```
solv =  
⎛ -2/3 - √2 i/3 ⎞  
⎛ -2/3 + √2 i/3 ⎞
```

```
solu =  
⎛ 1/3 - √2 i/3 ⎞  
⎛ 1/3 + √2 i/3 ⎞
```

Entries with the same index form the pair of solutions.

```
solutions = [solv solu]
```

```
solutions =  
⎛ -2/3 - √2 i/3 1/3 - √2 i/3 ⎞  
⎛ -2/3 + √2 i/3 1/3 + √2 i/3 ⎞
```

### Use Parameters and Conditions to Refine Solution

Return the complete solution of an equation with parameters and conditions of the solution by specifying `ReturnConditions` as `true`.

Solve the equation  $\sin(x) = 0$ . Provide two additional output variables for output arguments `parameters` and `conditions`.

```
syms x
eqn = sin(x) == 0;
[solx,parameters,conditions] = solve(eqn,x,ReturnConditions=true)

solx =  $\pi k$ 

parameters =  $k$ 

conditions =  $k \in \mathbb{Z}$ 
```

The solution  $\pi k$  contains the parameter  $k$ , where  $k$  must be an integer. The variable  $k$  does not exist in the MATLAB® workspace and must be accessed using `parameters`.

Restrict the solution to  $0 < x < 2\pi$ . Find a valid value of  $k$  for this restriction. Assume the condition, `conditions`, and use `solve` to find  $k$ . Substitute the value of  $k$  found into the solution for  $x$ .

```
assume(conditions)
restriction = [solx > 0, solx < 2*pi];
solk = solve(restriction,parameters)

solk = 1

valx = subs(solx,parameters,solk)

valx =  $\pi$ 
```

Alternatively, determine the solution for  $x$  by choosing a value of  $k$ . Check if the value chosen satisfies the condition on  $k$  using `isAlways`.

Check if  $k = 4$  satisfies the condition on  $k$ .

```
condk4 = subs(conditions,parameters,4);
isAlways(condk4)

ans = logical
     1
```

`isAlways` returns logical 1(true), meaning that 4 is a valid value for  $k$ . Substitute  $k$  with 4 to obtain a solution for  $x$ . Use `vpa` to obtain a numeric approximation.

```
valx = subs(solx,parameters,4)

valx =  $4\pi$ 

vpa(valx)

ans = 12.566370614359172953850573533118
```

### Apply Mathematical Rules

Solve the equation  $\exp(\log(x)\log(3x)) = 4$ .

When you define the symbolic variable  $x$ , the variable represents a complex value by default. The `solve` function applies only mathematical rules or simplifications that are valid for all values of  $x$ .

The solver does not assume that  $x$  is a positive real number, so it does not apply the logarithmic identity  $\log(3x) = \log(3) + \log(x)$ . As a result, `solve` cannot solve the equation symbolically.

```
syms x
eqn = exp(log(x)*log(3*x)) == 4;
S = solve(eqn,x)
```

Warning: Unable to solve symbolically. Returning a numeric solution using `<a href="matlab:web(fu`

```
S = -14.009379055223370038369334703094 - 2.9255310052111119036668717988769 i
```

To apply mathematical rules that might allow `solve` to find a symbolic solution, specify the `IgnoreAnalyticConstraints` name-value argument as `true`. For details, see “Algorithms” on page 6-1402.

```
S = solve(eqn,x,IgnoreAnalyticConstraints=true)
```

```
S =
```

$$\begin{pmatrix} \frac{\sqrt{3} e^{-\frac{\sqrt{\log(256) + \log(3)^2}}{2}}}{3} \\ \frac{\sqrt{3} e^{\frac{\sqrt{\log(256) + \log(3)^2}}{2}}}{3} \end{pmatrix}$$

Here, `solve` applies logarithmic identities that allow the solver to find solutions by assuming that  $x$  is a real positive number. Because these identities are not always valid in general, it is a good practice to verify the solutions when the solver ignores constraints.

To verify the solutions, for example, substitute  $x$  in the original equation with the second solution. Check if the resulting equation is `true` by using `isAlways`.

```
verifypeqn = subs(eqn,x,S(2))
```

```
verifypeqn =
```

$$e^{\log\left(\frac{\sqrt{3} e^{\frac{\sqrt{\log(256) + \log(3)^2}}{2}}}{3}\right) \log\left(\frac{\sqrt{3} e^{-\frac{\sqrt{\log(256) + \log(3)^2}}{2}}}{3}\right)} = 4$$

```
tf = isAlways(verifypeqn)
```

```
tf = logical
    1
```

## Simplify Solutions

The `solve` function might not return simplified solutions. For example, solve an equation.

```
syms x
S = solve((sin(x) - 2*cos(x))/(sin(x) + 2*cos(x)) == 1/2, x)
S =
```



$$\begin{pmatrix} -\log\left(-\frac{\sqrt{-\frac{140}{37} + \frac{48}{37}i}}{2}\right) i \\ -\log\left(\frac{\sqrt{-\frac{140}{37} + \frac{48}{37}i}}{2}\right) i \end{pmatrix}$$

To simplify the solutions, use `simplify`.

```
S = simplify(S)
```

$$S = \begin{pmatrix} -\log\left(\sqrt{37}\left(-\frac{1}{37} - \frac{6}{37}i\right)\right) i \\ \log(2) i - \frac{\log\left(-\frac{140}{37} + \frac{48}{37}i\right) i}{2} \end{pmatrix}$$

Use `simplify` with more steps to further simplify the solutions.

```
S = simplify(S,Steps=50)
```

$$S = \begin{pmatrix} \operatorname{atan}(6) - \pi \\ \frac{\pi}{2} - \frac{\operatorname{atan}\left(\frac{12}{35}\right)}{2} \end{pmatrix}$$

### Ignore Assumptions on Variables

The `sym` and `syms` functions let you set assumptions for symbolic variables.

Assume that the variable `x` is positive.

```
syms x positive
```

When you solve an equation for a variable under assumptions, the solver only returns solutions consistent with the assumptions. Solve this equation for `x`.

```
eqn = x^2 + 5*x - 6 == 0;
S = solve(eqn,x)
```

```
S = 1
```

Allow solutions that do not satisfy the assumptions by setting `IgnoreProperties` to `true`.

```
S = solve(eqn,x,IgnoreProperties=true)
```

```
S =
```

$$\begin{pmatrix} -6 \\ 1 \end{pmatrix}$$

For further computations, clear the assumption that you set on the variable `x` by recreating it using `syms`.

syms [x](#)

### Solve Polynomial Equations of High Degree

When you solve a polynomial equation, the solver might use `root` to return the solutions. Solve a third-degree polynomial.

```
syms x a
eqn = x^3 + x^2 + a == 0;
solve(eqn, x)
```

```
ans =

$$\begin{pmatrix} \text{root}(z^3 + z^2 + a, z, 1) \\ \text{root}(z^3 + z^2 + a, z, 2) \\ \text{root}(z^3 + z^2 + a, z, 3) \end{pmatrix}$$

```

Try to get an explicit solution for such equations by calling the solver with `MaxDegree` argument. This argument specifies the maximum degree of polynomials for which the solver tries to return explicit solutions. The default value is 2. Increasing this value, you can get explicit solutions for higher order polynomials.

Solve the same equations for explicit solutions by increasing the value of `MaxDegree` to 3.

```
S = solve(eqn,x,MaxDegree=3)
```

```
S =
```

$$\begin{pmatrix} \frac{1}{9\sigma_1} + \sigma_1 - \frac{1}{3} \\ -\frac{1}{18\sigma_1} - \frac{\sigma_1}{2} - \frac{1}{3} - \frac{\sqrt{3}\left(\frac{1}{9\sigma_1} - \sigma_1\right)i}{2} \\ -\frac{1}{18\sigma_1} - \frac{\sigma_1}{2} - \frac{1}{3} + \frac{\sqrt{3}\left(\frac{1}{9\sigma_1} - \sigma_1\right)i}{2} \end{pmatrix}$$

where

$$\sigma_1 = \left( \sqrt{\left(\frac{a}{2} + \frac{1}{27}\right)^2 - \frac{1}{729}} - \frac{a}{2} - \frac{1}{27} \right)^{1/3}$$

### Return One Solution

Solve the equation  $\sin(x) + \cos(2x) = 1$ .

Instead of returning an infinite set of periodic solutions, the solver picks three solutions that it considers to be the most practical.

```
syms x
eqn = sin(x) + cos(2*x) == 1;
S = solve(eqn,x)
```

$$S = \begin{pmatrix} 0 \\ \frac{\pi}{6} \\ \frac{5\pi}{6} \end{pmatrix}$$

Choose only one solution by setting `PrincipalValue` to `true`.

```
S1 = solve(eqn,x,PrincipalValue=true)
```

```
S1 = 0
```

### Different Solutions from Different Variable Orders

When you solve equations with multiple variables using `solve`, the order in which you specify the variables can affect the solutions. In certain cases, a different ordering can yield different solutions that satisfy the equation or system of equations to be solved.

Solve an equation with two unknowns, `a` and `b`.

```
syms a b
eqn = a*exp(1i*b) == 1
```

```
eqn = a ebi = 1
```

First, specify the variables to solve for in the order `a` followed by `b`. Here, the solution includes one arbitrary parameter, with `a` as an exponential function and `b` as an arbitrary value.

```
[sola,solb,parameters,conditions] = solve(eqn,a,b,ReturnConditions=true)
```

```
sola = e-zi
```

```
solb = z
```

```
parameters = z
```

```
conditions = symtrue
```

Next, specify the variables to solve for in the reverse order, `b` followed by `a`. Here, the solution has one arbitrary nonzero parameter and one integer parameter. The solution for `b` is a logarithmic function plus an integer multiple of  $2\pi$ , and `a` is an arbitrary nonzero value.

```
[solb,sola,parameters,conditions] = solve(eqn,b,a,ReturnConditions=true)
```

```
solb =
```

$$2\pi k - \log\left(\frac{1}{z}\right) i$$

```
sola = z
```

```
parameters = (k z)
```

```
conditions = k ∈ ℤ ∧ z ≠ 0
```

### Solve Equations for Symbolic Functions and Derivatives

You can solve symbolic equations where the unknowns are symbolic functions or their derivatives.

For example, create a quadratic equation that involves the symbolic function  $f(x)$ . Find  $f(x)$  using `solve`.

```
syms f(x) b c
eq = f(x)*x^2 + b*x + c == 0;
fsol = solve(eq,f(x))
```

```
fsol =

$$-\frac{c + bx}{x^2}$$

```

Next, create an equation that involves a trigonometric function of the symbolic function  $\theta(t)$ . Find  $\theta(t)$  using `solve` with `ReturnConditions` set to `true`.

```
syms t theta(t)
eq = 3 == -9.8*sin(theta(t));
[thetasol,param,cond] = solve(eq,theta(t),ReturnConditions=true)
```

```
thetasol =

$$\begin{pmatrix} 2\pi k - \arcsin\left(\frac{15}{49}\right) \\ \pi + \arcsin\left(\frac{15}{49}\right) + 2\pi k \end{pmatrix}$$

```

```
param = k
```

```
cond =

$$\begin{pmatrix} k \in \mathbb{Z} \\ k \in \mathbb{Z} \end{pmatrix}$$

```

Then, solve a system of equations for the derivative of  $f(x)$  and the variable  $a$ .

```
syms f(x) a
eq1 = diff(f,x) == 3*a - 2*x;
eq2 = diff(f,x) == a + x^2;
[Df,asol] = solve([eq1 eq2],diff(f,x),a)
```

```
Df =

$$\frac{3x^2}{2} + x$$

```

```
asol =

$$\frac{x^2}{2} + x$$

```

The output `Df` represents the solution for `diff(f,x)`. From this solution, you can then use `dsolve` to solve the differential equation for  $f(x)$ .

```
fsol(x) = dsolve(diff(f,x) == Df)
```

```
fsol(x) =

$$C_1 + \frac{x^2(x+1)}{2}$$

```

## Solve Equations and Inequalities Including Nonsymbolic Expressions

Create a system of equations and inequalities that includes a logical relation.

```
syms x y
eqns = [2*x + y == 5, y - x == -1, 3 < 5]

eqns = (2 x + y = 5 y - x = -1 1)
```

Because the logical relation  $3 < 5$  does not involve any symbolic variables, MATLAB evaluates the relation to `logical(1)` to support arithmetic operations, such as  $2*(3 < 5) = 2$ . In `eqns`, the `logical(1)` is then converted to `sym(1)`.

```
class(eqns)
```

```
ans =
'sym'
```

Solve the system.

```
sols = solve(eqns,[x y])

sols = struct with fields:
  x: [0x1 sym]
  y: [0x1 sym]
```

The `solve` function returns an empty structure because, for elements without a right side, the solver automatically equates the right side to 0. In this case, the solver interprets the element `sym(1)` in the vector `eqns` as `sym(1) == sym(0)`, which means the system has a false condition with no solutions.

Instead, specify the logical relation explicitly as a symbolic object, that is, `sym(3) < 5`. Create the system of equations and inequalities such that all elements are already of type `sym`.

```
eqns = [2*x + y == 5, y - x == -1, sym(3) < 5]

eqns = (2 x + y = 5 y - x = -1 3 < 5)
```

Solve the system.

```
sols = solve(eqns,[x y])

sols = struct with fields:
  x: 2
  y: 1
```

Here, the `solve` function returns the expected solutions because the solver interprets the logical relation as a symbolic logical constant for the `true` condition.

## Input Arguments

### eqn — Equation to solve

symbolic expression | symbolic equation

Equation to solve, specified as a symbolic expression or symbolic equation. The relation operator `==` defines symbolic equations. If `eqn` is a symbolic expression (without the right side), the solver assumes that the right side is 0 and solves the equation `eqn == 0`.

**var — Variable to solve for**  
symbolic variable | symbolic function

Variable to solve for, specified as a symbolic variable or symbolic function. By default, `solve` uses the variable determined by `symvar`.

**eqns — System of equations to solve**  
symbolic vector | symbolic matrix | comma-separated list

System of equations to solve, specified as a symbolic vector, symbolic matrix, or comma-separated list of symbolic expressions and equations.

- If any elements of `eqns` are symbolic expressions without a right side, `solve` treats the right side of those expressions as 0.
- If you specify `eqns` as a vector or matrix that contains nonsymbolic elements, `solve` converts all elements to symbolic objects. To avoid the solver misinterpreting nonsymbolic elements and returning an empty solution, define all elements as type `sym`. For example, see “Solve Equations and Inequalities Including Nonsymbolic Expressions” on page 6-1397.

**vars — Variables to solve for**  
symbolic vector | symbolic matrix | comma-separated list

Variables to solve for, specified as a symbolic vector, symbolic matrix, or comma-separated list of variables. By default, `solve` uses the variables determined by `symvar`.

The order in which you specify these variables defines the order in which the solver returns the solutions. In a few cases, a different ordering can yield different solutions that satisfy the equation or system of equations to be solved.

Example: `[x y z]`

Example: `x,y,z`

**Name-Value Pair Arguments**

Specify optional pairs of arguments as `Name1=Value1,...,NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `Real=true` specifies that the solver returns real solutions.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

**Real — Return only real solutions**  
`false` (default) | `true`

Option to return only real solutions, specified as one of these values.

|                    |                       |
|--------------------|-----------------------|
| <code>false</code> | Return all solutions. |
|--------------------|-----------------------|

|                   |                                                                                                                                                                                                           |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>true</code> | Return only those solutions for which every subexpression of the original equation represents a real number. This option also assumes that all symbolic parameters of an equation represent real numbers. |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

See “Solve Polynomial and Return Real Solutions” on page 6-1386.

### **ReturnConditions — Return parameters and conditions**

`false` (default) | `true`

Option to return parameters in solution and conditions under which the solution is true, specified as one of these values.

|                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>false</code> | Do not return parameterized solutions and the conditions under which the solution holds. The <code>solve</code> function replaces parameters with appropriate values.                                                                                                                                                                                                                                                                                                                             |
| <code>true</code>  | Return the parameters in the solution and the conditions under which the solution holds. For a call with a single output variable, <code>solve</code> returns a structure with the fields <code>parameters</code> and <code>conditions</code> . For multiple output variables, <code>solve</code> assigns the parameters and conditions to the last two output variables. This behavior means that the number of output variables must be equal to the number of variables to solve for plus two. |

See “Solve Inequalities” on page 6-1389.

Example: `[v1,v2,params,conditions] = solve(sin(x) + y == 0,y^2 == 3,ReturnConditions=true)` returns the parameters in `params` and conditions in `conditions`.

### **IgnoreAnalyticConstraints — Simplification rules applied to expressions and equations**

`false` (default) | `true`

Simplification rules applied to expressions and equations, specified as one of these values.

|                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>false</code> | Use strict simplification rules.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <code>true</code>  | Apply purely algebraic simplifications to expressions and equations. Setting <code>IgnoreAnalyticConstraints</code> to <code>true</code> can give you simpler solutions, which could lead to results not generally valid. In other words, this option applies mathematical identities that are convenient, but the results might not hold for all possible values of the variables. In some cases, it also enables <code>solve</code> to solve equations and systems that cannot be solved otherwise. For details, see “Algorithms” on page 6-1402. |

See “Apply Mathematical Rules” on page 6-1391.

### **IgnoreProperties — Option to return solutions inconsistent with properties of variables**

`false` (default) | `true`

Option to return solutions inconsistent with the properties of variables, specified as one of these values.

|                    |                                                                         |
|--------------------|-------------------------------------------------------------------------|
| <code>false</code> | Do not include solutions inconsistent with the properties of variables. |
| <code>true</code>  | Include solutions inconsistent with the properties of variables.        |

See “Ignore Assumptions on Variables” on page 6-1393.

**MaxDegree — Maximum degree of polynomial equations for which solver uses explicit formulas**

2 (default) | positive integer smaller than 5

Maximum degree of polynomial equations for which solver uses explicit formulas, specified as a positive integer smaller than 5. The solver does not use explicit formulas that involve radicals when solving polynomial equations of a degree larger than the specified value.

See “Solve Polynomial Equations of High Degree” on page 6-1394.

**PrincipalValue — Option to return one solution**

false (default) | true

Option to return one solution, specified as one of these values.

|       |                                                                                                                                          |
|-------|------------------------------------------------------------------------------------------------------------------------------------------|
| false | Return all solutions.                                                                                                                    |
| true  | Return only one solution. If an equation or a system of equations does not have a solution, the solver returns an empty symbolic object. |

See “Return One Solution” on page 6-1394.

**Output Arguments****S — Solutions of equation**

symbolic array

Solutions of an equation, returned as a symbolic array. The size of a symbolic array corresponds to the number of the solutions.

**Y — Solutions of system of equations**

structure

Solutions of a system of equations, returned as a structure. The number of fields in the structure correspond to the number of independent variables in a system. If `ReturnConditions` is set to `true`, the `solve` function returns two additional fields that contain the parameters in the solution, and the conditions under which the solution is true.

**y1, ..., yN — Solutions of system of equations**

symbolic variables

Solutions of a system of equations, returned as symbolic variables. The number of output variables or symbolic arrays must be equal to the number of independent variables in a system. If you explicitly specify independent variables `vars`, then the solver uses the same order to return the solutions. If you do not specify `vars`, the toolbox sorts independent variables alphabetically, and then assigns the solutions for these variables to the output variables.

**parameters — Parameters in solution**

vector of generated parameters

Parameters in a solution, returned as a vector of generated parameters. This output argument is only returned if `ReturnConditions` is `true`. If a single output argument is provided, `parameters` is returned as a field of a structure. If multiple output arguments are provided, `parameters` is returned as the second-to-last output argument. The generated parameters do not appear in the MATLAB workspace. They must be accessed using `parameters`.



Example: `[solx, params, conditions] = solve(sin(x) == 0, ReturnConditions=true)` returns the parameter `k` in the argument `params`.

### **conditions — Conditions under which solutions are valid**

vector of symbolic expressions

Conditions under which solutions are valid, returned as a vector of symbolic expressions. This output argument is only returned if `ReturnConditions` is `true`. If a single output argument is provided, `conditions` is returned as a field of a structure. If multiple output arguments are provided, `conditions` is returned as the last output argument.

Example: `syms x; [solx, params, conditions] = solve(sin(x) == 0, ReturnConditions=true)` returns the condition `in(k, 'integer')` in `conditions`. The solution in `solx` is valid only under the condition that `k` is an integer.

## **Tips**

- If `solve` cannot find a solution and `ReturnConditions` is `false`, the `solve` function internally calls the numeric solver `vpasolve` that tries to find a numeric solution. For polynomial equations and systems without symbolic parameters, the numeric solver returns all solutions. For nonpolynomial equations and systems without symbolic parameters, the numeric solver returns only one solution (if a solution exists).
- If `solve` cannot find a solution and `ReturnConditions` is `true`, `solve` returns an empty solution with a warning. If no solutions exist, `solve` returns an empty solution without a warning.
- If the solution contains parameters and `ReturnConditions` is `true`, `solve` returns the parameters in the solution and the conditions under which the solutions are true. If `ReturnConditions` is `false`, the `solve` function either chooses values of the parameters and returns the corresponding results, or returns parameterized solutions without choosing particular values. In the latter case, `solve` also issues a warning indicating the values of parameters in the returned solutions.
- If a parameter does not appear in any condition, it means the parameter can take any complex value.
- The output of `solve` can contain parameters from the input equations in addition to parameters introduced by `solve`.
- Parameters introduced by `solve` do not appear in the MATLAB workspace. They must be accessed using the output argument that contains them. Alternatively, to use the parameters in the MATLAB workspace use `syms` to initialize the parameter. For example, if the parameter is `k`, use `syms k`.
- The variable names `parameters` and `conditions` are not allowed as inputs to `solve`.
- To solve differential equations, use the `dsolve` function.
- When solving a system of equations, always assign the result to output arguments. Output arguments let you access the values of the solutions of a system.
- `MaxDegree` only accepts positive integers smaller than 5 because, in general, there are no explicit expressions for the roots of polynomials of degrees higher than 4.
- The output variables `y1, ..., yN` do not specify the variables for which `solve` solves equations or systems. If `y1, ..., yN` are the variables that appear in `eqns`, then there is no guarantee that `solve(eqns)` will assign the solutions to `y1, ..., yN` using the correct order. Thus, when you run `[b,a] = solve(eqns)`, you might get the solutions for `a` assigned to `b` and vice versa. To ensure the order of the returned solutions, specify the variables `vars`. For example, the call `[b,a] = solve(eqns,b,a)` assigns the solutions for `a` to `a` and the solutions for `b` to `b`.

## Algorithms

When you use `IgnoreAnalyticConstraints`, the solver applies some of these rules to the expressions on both sides of an equation.

- $\log(a) + \log(b) = \log(a \cdot b)$  for all values of  $a$  and  $b$ . In particular, the following equality is valid for all values of  $a$ ,  $b$ , and  $c$ :

$$(a \cdot b)^c = a^c \cdot b^c.$$

- $\log(a^b) = b \cdot \log(a)$  for all values of  $a$  and  $b$ . In particular, the following equality is valid for all values of  $a$ ,  $b$ , and  $c$ :

$$(a^b)^c = a^{b \cdot c}.$$

- If  $f$  and  $g$  are standard mathematical functions and  $f(g(x)) = x$  for all small positive numbers,  $f(g(x)) = x$  is assumed to be valid for all complex values  $x$ . In particular:

- $\log(e^x) = x$
- $\text{asin}(\sin(x)) = x$ ,  $\text{acos}(\cos(x)) = x$ ,  $\text{atan}(\tan(x)) = x$
- $\text{asinh}(\sinh(x)) = x$ ,  $\text{acosh}(\cosh(x)) = x$ ,  $\text{atanh}(\tanh(x)) = x$
- $W_k(x \cdot e^x) = x$  for all branch indices  $k$  of the Lambert W function.

- The solver can multiply both sides of an equation by any expression except 0.
- The solutions of polynomial equations must be complete.

## Version History

Introduced before R2006a

### R2025a: Solve for symbolic functions and their derivatives

You can now solve symbolic equations where the unknowns are symbolic functions or their derivatives. For example, this code returns the solution for the symbolic function  $f(x)$ .

```
syms f(x) b c
eq = f(x)*x^2 + b*x + c == 0;
fsol = solve(eq,f(x))
```

```
fsol =
-(c + b*x)/x^2
```

In previous releases, this code returns a warning and an empty structure as the solution.

This code now returns the solution for the symbolic function derivative `diff(f,x)` and the symbolic variable `a`.

```
syms f(x) a
eq1 = diff(f,x) == 3*a - 2*x;
eq2 = diff(f,x) == a + x^2;
[Df,asol] = solve([eq1 eq2],diff(f,x),a)
```

```
Df =
(3*x^2)/2 + x
```

```
asol =  
x^2/2 + x
```

In previous releases, this code returns an error.

For more information, see “Solve Equations for Symbolic Functions and Derivatives” on page 6-1395.

### **R2018a: Support for character vectors has been removed**

*Errors starting in R2018a*

Support for character vector or string inputs has been removed. Instead, use `syms` to declare variables and replace inputs such as `solve('2*x == 1','x')` with `solve(2*x == 1,x)`.

## **See Also**

### **Functions**

`dsolve` | `isolate` | `linsolve` | `root` | `subs` | `symvar` | `vpasolve`

### **Live Editor Tasks**

Solve Symbolic Equation

### **Topics**

“Solve Algebraic Equations” on page 3-3

“Solve System of Algebraic Equations” on page 3-7

“Solve System of Linear Equations” on page 3-36

“Solve Algebraic Equation Using Live Editor Task” on page 3-42

“Select Numeric or Symbolic Solver” on page 3-39

“Choose an Approach for Solving Equations Using solve Function” on page 3-16

### **Teaching Resources**

Beam Bending and Deflection (MathWorks Teaching Resources)

## solveRecurrence

Solve recurrence relations

### Syntax

```
sol = solveRecurrence(eqn, f)
sol = solveRecurrence(eqn, f, cond)
```

### Description

`sol = solveRecurrence(eqn, f)` solves the symbolic recurrence relations on page 6-1408 in `eqn` for the functions `f`.

`sol = solveRecurrence(eqn, f, cond)` solves the recurrence relations with the initial conditions `cond`.

### Examples

#### Gamma Function

Find the solution of the recurrence relation  $f(n) = n f(n - 1)$  for the gamma function.

Create a symbolic function `f(n)` and a symbolic equation `eqn` to represent the recurrence relation.

```
syms f(n)
eqn = f(n) == n*f(n-1);
```

Find the solution. The result is the gamma function with a constant multiplier.

```
sol = solveRecurrence(eqn, f)
sol = C1 Γ(n + 1)
```

#### Fibonacci and Lucas Numbers

Create a symbolic function `f(n)` and a symbolic equation `eqn` to represent the recurrence relation for the Fibonacci and Lucas sequences.

```
syms f(n)
eqn = f(n) == f(n-1) + f(n-2)
eqn = f(n) = f(n-1) + f(n-2)
```

Specify the initial conditions for the Fibonacci sequence.

```
cond = [f(0) == 0, f(1) == 1]
cond = (f(0) = 0 f(1) = 1)
```

Solve the recurrence relation using these initial conditions.

```
sol = solveRecurrence(eqn,f,cond)
```

```
sol =
```

$$\frac{\sqrt{5}\left(\frac{\sqrt{5}}{2} + \frac{1}{2}\right)^n}{5} - \frac{\sqrt{5}\left(\frac{1}{2} - \frac{\sqrt{5}}{2}\right)^n}{5}$$

Find the 10th Fibonacci number.

```
sol10 = simplify(subs(sol,n,10))
```

```
sol10 = 55
```

Next, specify the initial conditions for the Lucas sequence.

```
cond = [f(0) == 2, f(1) == 1]
```

```
cond = (f(0) = 2 f(1) = 1)
```

Solve the recurrence relation using these initial conditions.

```
sol = solveRecurrence(eqn,f,cond)
```

```
sol =
```

$$\left(\frac{1}{2} - \frac{\sqrt{5}}{2}\right)^n + \left(\frac{\sqrt{5}}{2} + \frac{1}{2}\right)^n$$

Find the 10th Lucas number.

```
sol10 = simplify(subs(sol,n,10))
```

```
sol10 = 123
```

## System of Linear Recurrence Relations

Create a system of linear recurrence relations.

```
syms a(n) b(n)
```

```
eqn = [a(n+1) == a(n) + 2*b(n); ...  
       b(n+1) == 3*a(n) + b(n)]
```

```
eqn =
```

$$\begin{pmatrix} a(n+1) = a(n) + 2b(n) \\ b(n+1) = 3a(n) + b(n) \end{pmatrix}$$

Specify the initial conditions for the functions  $a(0)$  and  $b(0)$ .

```
cond = [a(0) == 2; b(0) == 3]
```

```
cond =
```

$$\begin{pmatrix} a(0) = 2 \\ b(0) = 3 \end{pmatrix}$$

Solve the system of recurrence relations using these initial conditions.

```
sol = solveRecurrence(eqn,[a b],cond)
```

$$\text{sol} = \begin{pmatrix} \sigma_2 + \sigma_4 - \sigma_1 + \sigma_3 \\ \sigma_2 + \frac{3\sigma_4}{2} - \sigma_1 + \frac{3\sigma_3}{2} \end{pmatrix}$$

where

$$\sigma_1 = \frac{\sqrt{6}\sigma_3}{2}$$

$$\sigma_2 = \frac{\sqrt{6}\sigma_4}{2}$$

$$\sigma_3 = (1 - \sqrt{6})^n$$

$$\sigma_4 = (\sqrt{6} + 1)^n$$

Check if the solutions for  $a(n)$  and  $b(n)$  satisfy the recurrence relations. Substitute  $[a; b]$  with  $\text{sol}$ , and use `isAlways` to determine if both sides of the recurrence relations are equal.

```
tf = isAlways(subs(eqn,[a; b],sol))
```

*tf = 2×1 logical array*

```
1
1
```

Check if the solutions also satisfy the initial conditions.

```
tf = isAlways(subs(cond,[a; b],sol))
```

*tf = 2×1 logical array*

```
1
1
```

### Substitution to Solve Noncomputable Recurrence Relation

Create a recurrence relation for the Bessel functions.

```
syms J(n) z
eqn = J(n+1) == 2*n/z*J(n) - J(n-1)
```

$$\text{eqn} = J(n+1) = \frac{2nJ(n)}{z} - J(n-1)$$

Specify the initial conditions using the Bessel function of the first kind.

```
cond = [J(0) == besselj(0,z); J(1) == besselj(1,z)]
```

```
cond =
```

$$\begin{pmatrix} J(0) = J_0(z) \\ J(1) = J_1(z) \end{pmatrix}$$

`solveRecurrence` cannot always solve recurrence relations with variable coefficients. In this case, the function returns an unevaluated call.

```
sol = solveRecurrence(eqn,J,cond)
```

```
sol =
```

$$\text{solveRecurrence}\left(J(n+1) = \frac{2nJ(n)}{z} - J(n-1), J(n), [J(0) = J_0(z), J(1) = J_1(z)]\right)$$

However, if you substitute a specific value for the indexing parameter into the solution, the solver might be able to return an analytic solution. For example, substitute  $n = 4$ . The analytic solution is in terms of the variable  $z$  and the initial conditions  $J_0(z)$  and  $J_1(z)$ .

```
soln4 = subs(sol,n,4)
```

```
soln4 =
```

$$-\frac{8z^2J_1(z) - z^3J_0(z) + 24zJ_0(z) - 48J_1(z)}{z^3}$$

Check if this solution is equal to the Bessel function of the first kind of order 4.

```
tf = isAlways(soln4 == besselj(4,z))
```

```
tf = logical
    1
```

## Input Arguments

### **eqn — Recurrence relations to solve**

symbolic equation | vector of symbolic equations

Recurrence relations to solve, specified as a symbolic equation or vector of symbolic equations. The symbolic equations must contain symbolic functions that depend on a single indexing parameter, or the function argument must have only one independent variable.

The `solveRecurrence` function consistently solves symbolic equations that define a system of linear recurrence relations with constant coefficients. In some cases, `solveRecurrence` can also solve linear recurrence relations with nonconstant coefficients or a system of nonlinear recurrence relations.

```
Example: syms f(n); eqn = f(n) == 3*f(n-1) - f(n-2)
```

```
Example: syms f(n); eqn = f(n+1)*f(n) == 2*f(n) + 1
```

### **f — Functions to solve for**

symbolic function | vector of symbolic functions

Functions to solve for, specified as a symbolic function or vector of symbolic functions.

### **cond — Initial conditions**

symbolic equation | vector of symbolic equations

Initial conditions, specified as a symbolic equation or vector of symbolic equations.

## Output Arguments

### **sol** — Solution of recurrence relations

symbolic expression | vector of symbolic expressions

Solution of the recurrence relations, returned as a symbolic expression or vector of symbolic expressions.

If the `solveRecurrence` function cannot solve the recurrence relations, then it returns an unevaluated call.

## More About

### Recurrence Relations

Recurrence relations are equations that relate the next term in a sequence  $f(n+1)$  to the previous terms in the sequence  $f(n), \dots, f(n-m)$ . These relations contain functions  $f(n)$  that depend on an indexing parameter  $n$ .

The `solveRecurrence` function solves  $f(n)$  in linear recurrence relations (or linear difference equations) with constant coefficients  $c_i$  that have the form

$$f(n) = c_0 + c_1 f(n-1) + \dots + c_k f(n-k).$$

This form is considered homogeneous if  $c_0 = 0$  and nonhomogeneous if  $c_0 \neq 0$ . In some cases, the `solveRecurrence` function can also solve linear recurrence relations with nonconstant coefficients  $c_0, \dots, c_k$  that depend on  $n$  or nonlinear recurrence relations, such as

$$f(n+1)f(n) = c_0 + c_1 f(n).$$

The `solveRecurrence` function also solves a system of linear recurrence relations that has the form

$$\begin{bmatrix} f_1(n+1) \\ f_2(n+1) \\ \vdots \\ f_k(n+1) \end{bmatrix} = \begin{bmatrix} c_{10} & c_{11} & c_{12} & \cdots & c_{1k} \\ c_{20} & c_{21} & c_{22} & \cdots & c_{2k} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ c_{k0} & c_{k1} & c_{k2} & \cdots & c_{kk} \end{bmatrix} \begin{bmatrix} 1 \\ f_1(n) \\ f_2(n) \\ \vdots \\ f_k(n) \end{bmatrix}.$$

## Version History

Introduced in R2026a

## See Also

### Functions

`solve` | `dsolve` | `linsolve` | `vpasolve` | `subs`



# Solve Symbolic Equation

Find analytic solutions of symbolic equations in Live Editor

## Description

The Solve Symbolic Equation task enables you to interactively find analytic solutions of symbolic equations. The task automatically generates MATLAB code for your live script. For more information about Live Editor tasks, see “Add Interactive Tasks to a Live Script”.

Using this task, you can:

- Find analytic solutions of symbolic equations, which include a single equation and a system of algebraic equations.
- Specify the solver options to find solutions.
- Generate the code used to solve equations.

Solve Symbolic Equation


☒ Autorun
 


solution

= Analytic solution of equation QuadraticEquation with respect to x

Select equation

Equation

QuadraticEquation

Variables

x

for example: a, b, c, x

Specify solver options

☐ Return real solutions
 ☐ Return conditions
 ☐ Ignore analytic constraints
 ☒ Return one solution
 ☐ Expand all roots
 ☐ Ignore properties

Display result

☒ Equation
 ☒ Solution

▼ Hide code

```
% Compute analytic solution of a symbolic equation
QuadraticEquation

QuadraticEquation = a x^2 + b x + c = 42

solution = solve(QuadraticEquation,x,"PrincipalValue",true);
% Display symbolic solution returned by solve
displaySymSolution(solution);

solution =
```

$$-\frac{b + \sqrt{b^2 + 168 a - 4 a c}}{2 a}$$

## Open the Task

To add the Solve Symbolic Equation task to a live script in the MATLAB Editor:

- On the **Live Editor** tab, select **Task > Solve Symbolic Equation**.
- In a code block in your script, type a relevant keyword, such as `solve`, `symbolic`, or `equation`. Select **Solve Symbolic Equation** from the suggested command completions.

## Parameters

**Return real solutions** — Return only real solutions

`off` (default) | `on`

Select this check box to return solutions for which every subexpression of the equation represents a real number. This option assumes all parameters of the equation represent real numbers.

**Return one solution** — Return one solution

`off` (default) | `on`

Select this check box to return a single solution (principal value). If an equation or a system of equations does not have any solution, the solver returns an empty symbolic object.

**Return conditions** — Return more general solution and its constraints

`off` (default) | `on`

Select this check box to return the more general solution and the analytic constraints under which the solution holds. This option returns a structure with the fields `parameters` and `conditions` that contain the parameters in the solution and the conditions under which they hold, respectively.

**Expand all roots** — Expand solutions in terms of square roots

`off` (default) | `on`

Select this check box to express the `root` function in terms of square roots in the solutions. The results can be lengthy or less accurate for floating-point approximations.

**Ignore analytic constraints** — Option to ignore analytic constraints

`off` (default) | `on`

Select this check box to apply purely algebraic simplifications, such as  $\log(a) + \log(b) = \log(a*b)$  with the assumption that `a` and `b` are real positive numbers. Setting **Ignore analytic constraints** to `on` can give you simpler solutions, which could lead to results not generally valid. In other words, this option applies mathematical identities that are convenient for most engineering workflow, but do not always hold for all values of variables. In some cases, it also enables the Solve Symbolic Equation task to solve equations and systems that cannot be solved otherwise. For details, see “Algorithms” on page 6-1411.

**Ignore properties** — Option to ignore assumptions on variables to solve for

`off` (default) | `on`

Select this check box to ignore assumptions on the variables to solve for. This option may include solutions that are inconsistent with assumptions on the variables to solve for.

## Algorithms

When you use `Ignore analytic constraints`, the solver applies some of these rules to the expressions on both sides of an equation.

- $\log(a) + \log(b) = \log(a \cdot b)$  for all values of  $a$  and  $b$ . In particular, the following equality is valid for all values of  $a$ ,  $b$ , and  $c$  :

$$(a \cdot b)^c = a^c \cdot b^c.$$

- $\log(a^b) = b \cdot \log(a)$  for all values of  $a$  and  $b$ . In particular, the following equality is valid for all values of  $a$ ,  $b$ , and  $c$  :

$$(a^b)^c = a^{b \cdot c}.$$

- If  $f$  and  $g$  are standard mathematical functions and  $f(g(x)) = x$  for all small positive numbers,  $f(g(x)) = x$  is assumed to be valid for all complex values  $x$ . In particular:
  - $\log(e^x) = x$
  - $\text{asin}(\sin(x)) = x$ ,  $\text{acos}(\cos(x)) = x$ ,  $\text{atan}(\tan(x)) = x$
  - $\text{asinh}(\sinh(x)) = x$ ,  $\text{acosh}(\cosh(x)) = x$ ,  $\text{atanh}(\tanh(x)) = x$
  - $W_k(x \cdot e^x) = x$  for all branch indices  $k$  of the Lambert W function.
- The solver can multiply both sides of an equation by any expression except 0.
- The solutions of polynomial equations must be complete.

## Version History

Introduced in R2020a

## See Also

### Functions

`solve`

### Live Editor Tasks

Simplify Symbolic Expression

### Topics

"Add Interactive Tasks to a Live Script"

"Solve Algebraic Equation Using Live Editor Task" on page 3-42

"Solve Algebraic Equations" on page 3-3

## sort

Sort elements of symbolic arrays

### Syntax

```
Y = sort(X)
Y = sort(X,dim)
Y = sort( ___,direction)
[Y,I] = sort( ___)
```

### Description

`Y = sort(X)` sorts the elements of `X` in the default ascending order.

- If `X` is a vector, then `sort(X)` sorts the vector elements of `X`.
- If `X` is a matrix, then `sort(X)` treats the columns of `X` as vectors and sorts each column independently.
- If `X` is a multidimensional array, then `sort(X)` operates along the first array dimension whose size does not equal 1, treating the elements as vectors.

`Y = sort(X,dim)` sorts the elements of `X` along the dimension `dim`. For example, if `X` is a two-dimensional matrix, then `sort(X,1)` sorts the elements of each column of `X`, and `sort(X,2)` sorts the elements of each row.

`Y = sort( ___,direction)` sorts the elements of `X` in the order specified by `direction` using any of the previous syntaxes. "ascend" indicates ascending order (the default), and "descend" indicates descending order.

`[Y,I] = sort( ___)` also returns a collection of index vectors for any of the previous syntaxes. `I` is the same size as `X` and describes the arrangement of the elements of `X` into `Y` along the sorted dimension. For example, if `X` is an `m`-by-`n` matrix and you sort the elements of each column (`dim = 1`), then each column of `I` is an index vector of the sorted column of `X`, such that

```
for j = 1:n
    Y(:,j) = X(I(:,j),j);
end
```

### Examples

#### Sort Symbolic Vector in Ascending Order

Create a symbolic row vector `X` and sort its elements in ascending order. When `X` contains symbolic numbers and variables, then `sort(X)` returns the sorted numbers followed by the sorted variables.

```
syms a b c d e
X = [7 e 1 c 5 d a b]

X = (7 e 1 c 5 d a b)
```

```
sort(X)
```

```
ans = (1 5 7 a b c d e)
```

### Sort Symbolic Matrix Along Its Columns and Rows

When sorting the elements of a matrix, `sort` can work along the columns or rows of that matrix.

Create a symbolic matrix.

```
X = magic(3)*sym("a")
```

```
X =
```

$$\begin{pmatrix} 8a & a & 6a \\ 3a & 5a & 7a \\ 4a & 9a & 2a \end{pmatrix}$$

Sort the matrix. By default, the `sort` command sorts the elements of each column.

```
Y = sort(X)
```

```
Y =
```

$$\begin{pmatrix} 3a & a & 2a \\ 4a & 5a & 6a \\ 8a & 9a & 7a \end{pmatrix}$$

To sort the elements of each row, set the value of the `dim` option to 2.

```
Y = sort(X,2)
```

```
Y =
```

$$\begin{pmatrix} a & 6a & 8a \\ 3a & 5a & 7a \\ 2a & 4a & 9a \end{pmatrix}$$

### Sort Symbolic Matrix in Descending Order

Create a symbolic matrix.

```
X = magic(3)*sym("a")
```

```
X =
```

$$\begin{pmatrix} 8a & a & 6a \\ 3a & 5a & 7a \\ 4a & 9a & 2a \end{pmatrix}$$

Sort the elements of each row in descending order.

```
Y = sort(X,2,"descend")
```

```
Y =
```

$$\begin{pmatrix} 8a & 6a & a \\ 7a & 5a & 3a \\ 9a & 4a & 2a \end{pmatrix}$$

### Find Indices of Sorted Matrix Elements in Original Matrix

To find the indices that each element of a sorted matrix  $Y$  had in the original matrix  $X$ , call `sort` with two output arguments.

Create a symbolic matrix  $X$ .

```
X = magic(3)*sym("a")
```

$$X = \begin{pmatrix} 8a & a & 6a \\ 3a & 5a & 7a \\ 4a & 9a & 2a \end{pmatrix}$$

Sort each column of  $X$  and return the indices of the sorted elements as  $I$ . Each column of  $I$  contains the presorted positions of elements in  $Y$ .

```
[Y,I] = sort(X)
```

$$Y = \begin{pmatrix} 3a & a & 2a \\ 4a & 5a & 6a \\ 8a & 9a & 7a \end{pmatrix}$$

```
I = 3x3
```

$$\begin{pmatrix} 2 & 1 & 3 \\ 3 & 2 & 1 \\ 1 & 3 & 2 \end{pmatrix}$$

### Sort 3-D Symbolic Array

Create a 2-by-2-by-2 symbolic array that contains symbolic numbers, variables, and functions.

```
syms x y f(x)
```

```
X(:,:,1) = [y 1; 1/3 x];
```

```
X(:,:,2) = [x -2; 1/4 f(x)];
```

```
X
```

$$X(:,:,1) = \begin{pmatrix} y & 1 \\ \frac{1}{3} & x \end{pmatrix}$$

$$X(:,:,2) = \begin{pmatrix} x & -2 \\ \frac{1}{4} & f(x) \end{pmatrix}$$

Sort its elements in ascending order along the third dimension.

```
Y = sort(X,3)
```

$$Y(:, :, 1) = \begin{pmatrix} x & -2 \\ \frac{1}{4} & x \end{pmatrix}$$

$$Y(:, :, 2) = \begin{pmatrix} y & 1 \\ \frac{1}{3} & f(x) \end{pmatrix}$$

Use  $X(:)$ , the column representation of  $X$ , to sort all of the elements of  $X$ .

```
Y = sort(X(:))
```

$$Y = \begin{pmatrix} -2 \\ \frac{1}{4} \\ \frac{1}{3} \\ 1 \\ x \\ x \\ y \\ f(x) \end{pmatrix}$$

### Sort Symbolic Vector of Real and Complex Numbers

Sort a symbolic vector that contains real and complex numbers. The `sort` function sorts the real and complex numbers together, considering the magnitude of the numbers, followed by their phase angles in the interval  $(-\pi, \pi]$  to break ties.

```
X = sym([6 -1/2 3+4i 5i 4+3i])
```

$$X = \left( 6 - \frac{1}{2} \quad 3 + 4i \quad 5i \quad 4 + 3i \right)$$

```
Y = sort(X)
```

$$Y = \left( -\frac{1}{2} \quad 4 + 3i \quad 3 + 4i \quad 5i \quad 6 \right)$$

### Sort Rational and Irrational Numbers

Create a vector that contains the irrational number  $\pi$  and other rational numbers that approximate  $\pi$ .

```
X = [sym(pi), sym(22)/7, 333/106, 223/71]
```

```
X =
```

$$\left( \pi \quad \frac{22}{7} \quad \frac{333}{106} \quad \frac{223}{71} \right)$$

Sort these numbers in ascending order.

```
Y = sort(X)
```

```
Y =
```

$$\left( \frac{223}{71} \quad \frac{333}{106} \quad \pi \quad \frac{22}{7} \right)$$

For reference, display the sorted result in decimal representation with a precision of six significant digits using `vpa`.

```
Yvpa = vpa(Y,6)
```

```
Yvpa = (3.14085 3.14151 3.14159 3.14286)
```

## Input Arguments

### X — Input array

symbolic vector | symbolic matrix | symbolic multidimensional array

Input array, specified as a symbolic vector, matrix, or multidimensional array. `sort` uses the following rules:

- If `X` contains only symbolic numbers, then `sort(X)` sorts the elements numerically. `sort` evaluates symbolic numbers internally and aims to sort the evaluated numbers without round-off errors. For complex numbers, `sort(X)` sorts the numbers first by their magnitude, followed by their phase angles in the interval  $(-\pi, \pi]$  to break ties.
- If `X` contains only symbolic variables, then `sort(X)` sorts the variable names first by their alphabetical components and then by their numerical components. For example, for `X = [x1, y, x10, x2]`, the vector sorted in ascending order is `[x1, x2, x10, y]`, instead of `[x1, x10, x2, y]`.
- If `X` contains symbolic numbers and variables, then `sort(X)` returns the sorted numbers followed by the sorted variables.
- If `X` contains symbolic numbers, expressions, and functions, comparing and sorting the elements can be computationally complex. Therefore, `sort` uses internal sorting rules to optimize its performance.

### dim — Dimension to operate along

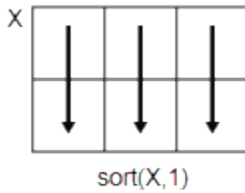
positive integer scalar

Dimension to operate along, specified as a positive integer scalar. If you do not specify a value, then the default is the first array dimension whose size does not equal 1.

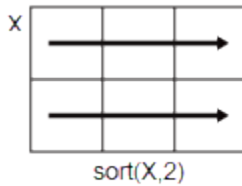
Consider a two-dimensional input array `X`:

- `sort(X,1)` sorts the elements in the columns of `X`.





- `sort(X,2)` sorts the elements in the rows of X.



`sort` returns X if `dim` is greater than `ndims(X)`.

#### direction — Sorting direction

"ascend" (default) | "descend"

Sorting direction, specified as "ascend" or "descend".

## Output Arguments

### Y — Sorted array

symbolic vector | symbolic matrix | symbolic multidimensional array

Sorted array, returned as a symbolic vector, matrix, or multidimensional array. Y is the same size and type as X.

### I — Sort index

symbolic vector | symbolic matrix | symbolic multidimensional array

Sort index, returned as a symbolic vector, matrix, or multidimensional array. I is the same size as X. The index vectors are oriented along the same dimension that `sort` operates on. For example, if X is a 2-by-3 matrix, then `[Y,I] = sort(X,2)` sorts the elements in each row of X. The output I is a collection of 1-by-3 row index vectors that contain the presorted positions of each row of Y.

## Tips

- Calling `sort` on arrays of numbers that are not symbolic objects invokes the MATLAB `sort` function.

## Version History

Introduced before R2006a

### R2024b: Different sorting rules for rational, irrational, and complex numbers

*Behavior changed in R2024b*

When sorting symbolic numbers, `sort` no longer groups rational and irrational numbers separately. Instead, `sort` evaluates symbolic numbers internally and aims to sort the evaluated numbers without

round-off errors. For complex numbers, `sort` sorts the numbers by their magnitude first, followed by their phase angles in the interval  $(-\pi, \pi]$  to break ties.

For example, this code now returns the complex number, which has the smallest magnitude, followed by real numbers with larger magnitudes, regardless of whether they are rational or irrational.

```
Y = sort([sym(22)/7, 333/106, sym(pi), 3i])  
  
Y =  
[3i, 333/106, pi, 22/7]
```

In previous releases, `sort` grouped rational real numbers first, followed by rational complex numbers, and then irrational numbers. For complex numbers, `sort` sorted the numbers first by their real parts, followed by the imaginary parts to break ties.

For example, before R2024b, the same code returned the rational numbers that are real, followed by the rational number that is imaginary, and then the irrational number.

```
Y = sort([sym(22)/7, 333/106, sym(pi), 3i])  
  
Y =  
[333/106, 22/7, 3i, pi]
```

### **R2020b: Sort multidimensional arrays of symbolic expressions**

`sort` accepts multidimensional arrays of symbolic expressions as input.

# sqrtm

Matrix square root

## Syntax

```
X = sqrtm(A)
[X,resnorm] = sqrtm(A)
```

## Description

`X = sqrtm(A)` returns a matrix  $X$ , such that  $X^2 = A$  and the eigenvalues of  $X$  are the square roots of the eigenvalues of  $A$ .

`[X,resnorm] = sqrtm(A)` returns a matrix  $X$  and the residual norm  $\text{norm}(A-X^2, 'fro') / \text{norm}(A, 'fro')$ .

## Examples

### Compute Square Root of Matrix

Compute the square root of this matrix. Because these numbers are not symbolic objects, you get floating-point results.

```
A = [2 -2 0; -1 3 0; -1/3 5/3 2];
X = sqrtm(A)
```

```
X =
    1.3333    -0.6667     0.0000
   -0.3333     1.6667    -0.0000
   -0.0572     0.5286     1.4142
```

Now, convert this matrix to a symbolic object, and compute its square root again:

```
A = sym([2 -2 0; -1 3 0; -1/3 5/3 2]);
X = sqrtm(A)

X =
[
    4/3,      -2/3,      0]
[
   -1/3,      5/3,      0]
[ (2*2^(1/2))/3 - 1, 1 - 2^(1/2)/3, 2^(1/2)]
```

Check the correctness of the result:

```
isAlways(X^2 == A)

ans =
    3×3 logical array
     1     1     1
     1     1     1
     1     1     1
```

### Return Residual of Matrix Square Root

Use the syntax with two output arguments to return the square root of a matrix and the residual:

```
A = vpa(sym([0 0; 0 5/3]), 100);  
[X,resnorm] = sqrtm(A)  
  
X =  
[ 0, 0]  
[ 0, 1.2909944487358056283930884665941]  
  
resnorm =  
2.9387358770557187699218413430556e-40
```

### Input Arguments

#### A — Input

symbolic matrix

Input, specified as a symbolic matrix.

### Output Arguments

#### X — Matrix square root

symbolic matrix

Matrix square root, returned as a symbolic matrix such that  $X^2 = A$ .

#### resnorm — Residual

symbolic expression

Residual, returned as a symbolic expression. The residual is computed as  $\text{norm}(A-X^2, 'fro') / \text{norm}(A, 'fro')$ .

### Tips

- Calling `sqrtm` for a matrix that is not a symbolic object invokes the MATLAB `sqrtm` function.

### Version History

Introduced in R2013a

### See Also

`cond` | `eig` | `expm` | `funm` | `jordan` | `logm` | `norm`

# ssinint

Shifted sine integral function

## Syntax

`ssinint(X)`

## Description

`ssinint(X)` returns the shifted sine integral function on page 6-1423  $\text{ssinint}(X) = \text{sinint}(X) - \pi/2$ .

## Examples

### Shifted Sine Integral Function for Numeric and Symbolic Arguments

Depending on its arguments, `ssinint` returns floating-point or exact symbolic results.

Compute the shifted sine integral function for these numbers. Because these numbers are not symbolic objects, `ssinint` returns floating-point results.

```
A = ssinint([- pi, 0, pi/2, pi, 1])
```

```
A =
    -3.4227    -1.5708    -0.2000     0.2811    -0.6247
```

Compute the shifted sine integral function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `ssinint` returns unresolved symbolic calls.

```
symA = ssinint(sym([- pi, 0, pi/2, pi, 1]))
```

```
symA =
[ - pi - ssinint(pi), -pi/2, ssinint(pi/2), ssinint(pi), ssinint(1)]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

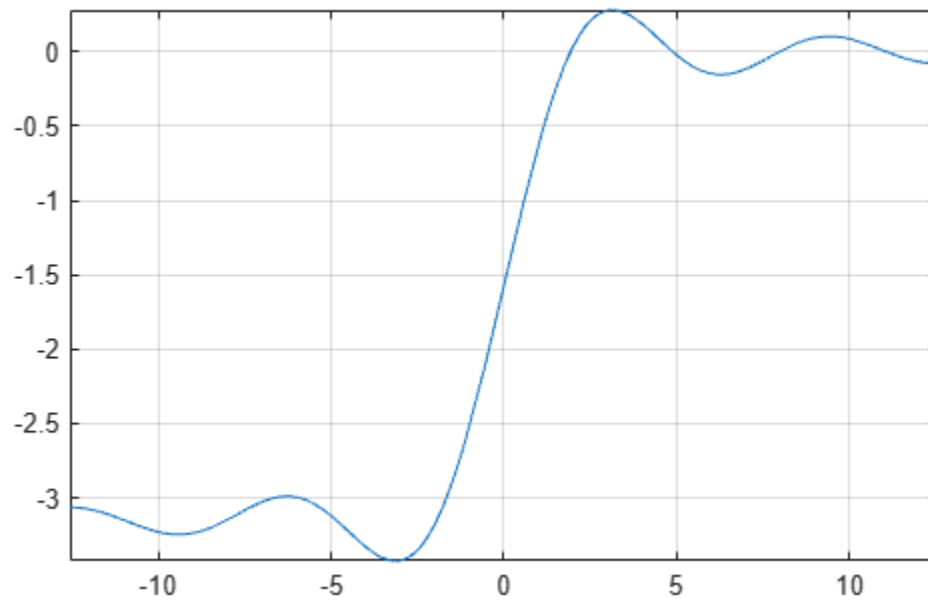
```
vpa(symA)
```

```
ans =
[ -3.4227333787773627895923750617977,...
 -1.5707963267948966192313216916398,...
 -0.20003415864040813916164340325818,...
 0.28114072518756955112973167851824,...
 -0.62471325642771360428996837781657]
```

### Plot Shifted Sine Integral Function

Plot the shifted sine integral function on the interval from  $-4\pi$  to  $4\pi$ .

```
syms x
fplot(ssinint(x),[-4*pi 4*pi])
grid on
```



### Handle Expressions Containing Shifted Sine Integral Function

Many functions, such as `diff`, `int`, and `taylor`, can handle expressions containing `ssinint`.

Find the first and second derivatives of the shifted sine integral function:

```
syms x
diff(ssinint(x), x)
diff(ssinint(x), x, x)
```

```
ans =
sin(x)/x
```

```
ans =
cos(x)/x - sin(x)/x^2
```

Find the indefinite integral of the shifted sine integral function:

```
int(ssinint(x), x)
```

```
ans =
cos(x) + x*ssinint(x)
```

Find the Taylor series expansion of `ssinint(x)`:

```
taylor(ssinint(x), x)
```

```
ans =
x^5/600 - x^3/18 + x - pi/2
```

## Input Arguments

### X — Input

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## More About

### Sine Integral Function

The sine integral function is defined as follows:

$$\text{Si}(x) = \int_0^x \frac{\sin(t)}{t} dt$$

### Shifted Sine Integral Function

The sine integral function is defined as  $\text{Ssi}(x) = \text{Si}(x) - \pi/2$ .

## Version History

Introduced in R2014a

## References

- [1] Gautschi, W. and W. F. Cahill. "Exponential Integral and Related Functions." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

coshint | cosint | eulergamma | int | sin | sinhint | sinhint | sinint

## str2symunit

Convert character vector or string to unit

### Syntax

```
str2symunit(unitStr)
str2symunit(unitStr,toolbox)
```

### Description

`str2symunit(unitStr)` converts the character vector or string `unitStr` to symbolic units.

`str2symunit(unitStr,toolbox)` converts the character vector `unitStr` assuming it represents units in the toolbox `toolbox`. The allowed values of `toolbox` are 'Aerospace', 'SimBiology', 'Simscape', or 'Simulink'.

### Examples

#### Convert Character Vector to Unit

Convert the character vector 'km/hour' to symbolic units.

```
unit = str2symunit('km/hour')

unit =
1*([km]/[h])
```

Use this unit to define a speed of 50 km/hour.

```
speed = 50*unit

speed =
50*([km]/[h])
```

#### Convert Units of Specified Toolbox

Convert units from other toolboxes to symbolic units by specifying the toolbox name as the second argument to `str2symunit`. The allowed names are 'Aerospace', 'SimBiology', 'Simscape', or 'Simulink'.

Convert 'km/h-s' from Aerospace Toolbox to symbolic units.

```
unit = str2symunit('km/h-s','Aerospace')

unit =
1*([km]/([h]*[s]))
```

Convert 'molecules/s' from SimBiology® to symbolic units.

```
unit = str2symunit('molecule/s','SimBiology')

unit =
1*([molecule]/[s])
```



Convert 'gee/km' from Simscape to symbolic units.

```
unit = str2symunit('gee/km','Simscape')
```

```
unit =  
1*([g_n]/[km])
```

Convert 'rad/second' from Simulink to symbolic units.

```
unit = str2symunit('rad/second','Simulink')
```

```
unit =  
1*([rad]/[s])
```

## Input Arguments

### **unitStr** — Input units

character vector | string

Input, specified as a character vector or string.

Example: `str2symunit('km/hour')`

### **toolbox** — Toolbox to which units belong

'Aerospace' | 'SimBiology' | 'Simscape' | 'Simulink'

Toolbox to which input belongs, specified as 'Aerospace', 'SimBiology', 'Simscape', or 'Simulink'.

Example: `str2symunit('km/h-s', 'Aerospace')`

## Version History

**Introduced in R2017a**

## See Also

`checkUnits` | `findUnits` | `isUnit` | `newUnit` | `separateUnits` | `symunit` | `symunit2str` | `unitConversionFactor`

## Topics

"Units of Measurement Tutorial" on page 2-47

"Unit Conversions and Unit Systems" on page 2-53

"Units and Unit Systems List" on page 2-60

## External Websites

The International System of Units (SI)

## str2sym

Evaluate string representing symbolic expression

### Syntax

```
str2sym(symstr)
```

### Description

`str2sym(symstr)` evaluates `symstr` where `symstr` is a string representing a symbolic expression. Enter symbolic expressions as strings only when reading expressions from text files or when specifying numbers exactly. Otherwise, do not use strings for symbolic input.

### Examples

#### Evaluate String as Symbolic Expression

Evaluate the string `'sin(pi)'`. `str2sym` returns the expected result.

```
str2sym('sin(pi)')
```

```
ans =  
0
```

`str2sym` assumes the `=` operator represents an equation, not an assignment. Also, `str2sym` does not add the variables contained in the string to the workspace.

Show this behavior by evaluating `'x^2 = 4'`. The `str2sym` function returns the equation `x^2 == 4` but `x` does not appear in the workspace.

```
eqn = str2sym('x^2 = 4')
```

```
eqn =  
x^2 == 4
```

Find the variable in `eqn` by using `symvar`. The variable `var` now refers to `x`.

```
var = symvar(eqn)
```

```
var =  
x
```

Assign values from `eqn` by solving `eqn` for `var` and assigning the result.

```
varVal = solve(eqn,var)
```

```
varVal =  
-2  
2
```

## Substitute Workspace Values into String Input

`str2sym` does not substitute values from the workspace for variables in the input. Therefore, `str2sym` has reproducible output. Instead, substitute workspace values by using `subs` on the output of `str2sym`.

Set `y` to 2. Then, evaluate `'y^2'` with and without `subs` to show how `subs` substitutes `y` with its value.

```
y = 2;
withoutSubs = str2sym('y^2')

withoutSubs =
y^2

withSubs = subs(str2sym('y^2'))

withSubs =
4
```

## Evaluate Strings from File as Symbolic Expressions

When symbolic expressions are stored as strings in a file, evaluate the strings by reading the file and using `str2sym`.

Assume the file `mySym.txt` contains this text.

```
a = 2.431
y = a*exp(t)
diff(z(t),t) = b*y*z
```

Evaluate expressions in `mySym.txt` using `str2sym`.

```
filename = 'mySym.txt';
filetext = fileread(filename);
filetext = splitlines(filetext);
str2sym(filetext)

ans =
      a == 2.431
      y == a*exp(t)
diff(z(t), t) == b*y*z
```

The output of `str2sym` is independent of workspace values, which means the output is reproducible. Show this reproducibility by assigning a value to `b` and re-evaluating the stored expressions.

```
b = 5;
str2sym(filetext)

ans =
      a == 2.431
      y == a*exp(t)
diff(z(t), t) == b*y*z
```

To use workspace values or a value from input equations, use `subs` (solve the equation first using `solve`), as described in “Evaluate String as Symbolic Expression” on page 6-1426 and “Substitute Workspace Values into String Input” on page 6-1426.

### Execute Functions in String Input

`str2sym` executes functions in input when the functions are on the path. Otherwise, `str2sym` returns the symbolic object as expected. This behavior means that the output is reproducible.

Show this behavior by reading a differential equation and initial condition from a file. Solve the equation for the condition. Because `str2sym` does not evaluate  $y(t)$  in the equation, the output is reproducible.

```
filename = 'mySym.txt';
filetext = fileread(filename);
filetext = splitlines(filetext);
eqn = str2sym(filetext(1))

eqn =
diff(y(t), t) == -y(t)

cond = str2sym(filetext(2))

cond =
y(0) == 2

ySol = dsolve(eqn,cond)

ySol =
2*exp(-t)
```

### Exactly Represent Large Numbers and High-Precision Numbers

Because the MATLAB parser automatically converts all numbers to double precision, maintain original precision by entering large numbers and high-precision numbers as strings. Instead of `str2sym`, enter integers using `sym` and floating-point numbers using `vpa` because `sym` and `vpa` are faster.

Show the error between entering a ratio of large integers directly versus the exact string representation.

```
num = sym(12230984290/38490293482)

num =
5724399718238385/18014398509481984

numExact = sym('12230984290/38490293482')

numExact =
6115492145/19245146741

error = num - numExact

error =
-7827162395/346689742765832461975814144
```

Show the error between entering a high-precision number directly versus the exact string representation.

```
num = vpa(8.023098429038490293482)

num =
8.0230984290384910195825796108693

numExact = vpa('8.023098429038490293482')

numExact =
8.023098429038490293482

error = num - numExact

error =
0.000000000000000072610057961086928844451883343504
```

For details, see “Numeric to Symbolic Conversion” on page 2-29. For full workflows, see “Find Almost Integers with High-Precision Arithmetic” on page 2-41 and “Prime Factorizations” on page 3-388.

## Convert Hexadecimal and Binary Values to Symbolic Decimal Numbers

Starting in R2019b, you can represent hexadecimal and binary values using character vectors. Hexadecimal values start with a 0x or 0X prefix, while binary values start with a 0b or 0B prefix. You can then convert the hexadecimal and binary values to symbolic decimal numbers using `str2sym`. For more information, see “Hexadecimal and Binary Values”.

Create a character vector that represents a hexadecimal value. Convert the value to symbolic decimal number.

```
H = '0x2A'
D = str2sym(H)

D =
42
```

Create a character vector that represents a binary value. Convert the value to symbolic decimal number.

```
B = '0b101010'
D = str2sym(B)

D =
42
```

## Input Arguments

### **symstr** — String representing symbolic expression

character vector | string | cell array of character vectors

String representing a symbolic expression, specified as a character vector, string, or cell array of character vectors.

## **Tips**

- `str2sym` assumes the `=` operator represents an equation, not an assignment.
- `str2sym` does not create variables contained in the input.
- `str2sym('inf')` returns infinity (`Inf`).
- `str2sym('i')` returns the imaginary number `1i`.

## **Version History**

**Introduced in R2017b**

## **See Also**

`sym` | `syms` | `subs` | `vpa`

## **Topics**

“Numeric to Symbolic Conversion” on page 2-29

“Create Symbolic Numbers, Variables, and Expressions” on page 1-3

# subexpr

Rewrite symbolic expression in terms of common subexpressions

## Syntax

```
[r,sigma] = subexpr(expr)
[r,var] = subexpr(expr,'var')
[r,var] = subexpr(expr,var)
```

## Description

`[r,sigma] = subexpr(expr)` rewrites the symbolic expression `expr` in terms of a common subexpression, substituting this common subexpression with the symbolic variable `sigma`. The input expression `expr` cannot contain the variable `sigma`.

`[r,var] = subexpr(expr,'var')` substitutes the common subexpression by `var`. The input expression `expr` cannot contain the symbolic variable `var`.

`[r,var] = subexpr(expr,var)` is equivalent to `[r,var] = subexpr(expr,'var')`, except that the symbolic variable `var` must already exist in the MATLAB workspace.

This syntax overwrites the value of the variable `var` with the common subexpression found in `expr`. To avoid overwriting the value of `var`, use another variable name as the second output argument. For example, use `[r,var1] = subexpr(expr,var)`.

## Examples

### Rewrite Expression Using Abbreviations

Solve the following equation. The solutions are very long expressions. To display the solutions, remove the semicolon at the end of the `solve` command.

```
syms a b c d x
solutions = solve(a*x^3 + b*x^2 + c*x + d == 0, x, 'MaxDegree', 3);
```

These long expressions have common subexpressions. To shorten the expressions, abbreviate the common subexpression by using `subexpr`. If you do not specify the variable to use for abbreviations as the second input argument of `subexpr`, then `subexpr` uses the variable `sigma`.

```
[r, sigma] = subexpr(solutions)
```

```
r =
```

$$\begin{pmatrix} \sigma - \frac{b}{3a} - \frac{\sigma_2}{\sigma} \\ \frac{\sigma_2}{2\sigma} - \frac{b}{3a} - \frac{\sigma}{2} - \sigma_1 \\ \frac{\sigma_2}{2\sigma} - \frac{b}{3a} - \frac{\sigma}{2} + \sigma_1 \end{pmatrix}$$

where

$$\sigma_1 = \frac{\sqrt{3}\left(\sigma + \frac{\sigma_2}{\sigma}\right) i}{2}$$

$$\sigma_2 = \frac{c}{3a} - \frac{b^2}{9a^2}$$

sigma =

$$\left( \sqrt{\left( \frac{d}{2a} + \frac{b^3}{27a^3} - \frac{bc}{6a^2} \right)^2 + \left( \frac{c}{3a} - \frac{b^2}{9a^2} \right)^3} - \frac{b^3}{27a^3} - \frac{d}{2a} + \frac{bc}{6a^2} \right)^{1/3}$$

### Customize Abbreviation Variables

Solve a quadratic equation.

`syms a b c x`

`solutions = solve(a*x^2 + b*x + c == 0, x)`

`solutions =`

$$\begin{pmatrix} -\frac{b + \sqrt{b^2 - 4ac}}{2a} \\ -\frac{b - \sqrt{b^2 - 4ac}}{2a} \end{pmatrix}$$

Use `syms` to create the symbolic variable `s`, and then replace common subexpressions in the result with this variable.

`syms s`

`[abbrSolutions,s] = subexpr(solutions,s)`

`abbrSolutions =`

$$\begin{pmatrix} -\frac{b+s}{2a} \\ -\frac{b-s}{2a} \end{pmatrix}$$

$$s = \sqrt{b^2 - 4ac}$$

Alternatively, use `'s'` to specify the abbreviation variable.

`[abbrSolutions,s] = subexpr(solutions,'s')`

`abbrSolutions =`



$$\begin{pmatrix} -\frac{b+s}{2a} \\ -\frac{b-s}{2a} \end{pmatrix}$$

$$s = \sqrt{b^2 - 4ac}$$

Both syntaxes overwrite the value of the variable `s` with the common subexpression. Therefore, you cannot, for example, substitute `s` with some value.

```
subs(abbrSolutions,s,0)
```

$$\text{ans} = \begin{pmatrix} -\frac{b+s}{2a} \\ -\frac{b-s}{2a} \end{pmatrix}$$

To avoid overwriting the value of the variable `s`, use another variable name for the second output argument.

```
syms s
[abbrSolutions,t] = subexpr(solutions,'s')
```

```
abbrSolutions =
```

$$\begin{pmatrix} -\frac{b+s}{2a} \\ -\frac{b-s}{2a} \end{pmatrix}$$

$$t = \sqrt{b^2 - 4ac}$$

```
subs(abbrSolutions,s,0)
```

```
ans =
```

$$\begin{pmatrix} -\frac{b}{2a} \\ -\frac{b}{2a} \end{pmatrix}$$

## Input Arguments

**expr** — Long expression containing common subexpressions

symbolic expression | symbolic function

Long expression containing common subexpressions, specified as a symbolic expression or function.

**var** — Variable to use for substituting common subexpressions

character vector | symbolic variable

Variable to use for substituting common subexpressions, specified as a character vector or symbolic variable.

`subexpr` throws an error if the input expression `expr` already contains `var`.

## Output Arguments

**r** — Expression with common subexpressions replaced by abbreviations

symbolic expression | symbolic function

Expression with common subexpressions replaced by abbreviations, returned as a symbolic expression or function.

**var** — Variable used for abbreviations

symbolic variable

Variable used for abbreviations, returned as a symbolic variable.

## Version History

Introduced before R2006a

## See Also

children | simplify | subs

## Topics

"Abbreviate Common Terms in Long Expressions" on page 3-157

# subs

Symbolic substitution

## Syntax

```
snew = subs(s,match,replacement)
snew = subs(s,replacement)
snew = subs(s)
```

```
sMnew = subs(sM,matchM,replacementM)
sMnew = subs(sM,replacementM)
sMnew = subs(sM)
```

## Description

### Substitute Symbolic Scalar Variables and Functions

`snew = subs(s,match,replacement)` returns a copy of `s`, replacing all occurrences of `match` with `replacement`, and then evaluates `s`. Here, `s` is an expression of symbolic scalar variables or a symbolic function, and `match` specifies the symbolic scalar variables or symbolic function to be substituted.

- If `match` and `replacement` are both vectors or cell arrays of the same size, `subs` replaces each element of `match` with the corresponding element of `replacement`.
- If `match` is a scalar, and `replacement` is a vector or matrix, then `subs(s,match,replacement)` replaces all instances of `match` in `s` with `replacement`, performing all operations element-wise. All constant terms in `s` are replaced with the constant multiplied by a vector or matrix of all ones.

`snew = subs(s,replacement)` returns a copy of `s`, replacing all occurrences of the default symbolic scalar variable in `s` with `replacement`, and then evaluates `s`. The default variable is defined by `symvar(s,1)`.

`snew = subs(s)` returns a copy of `s`, replacing symbolic scalar variables in `s` with their assigned values in the MATLAB workspace, and then evaluates `s`. Variables with no assigned values remain as variables.

### Substitute Symbolic Matrix Variables and Functions

`sMnew = subs(sM,matchM,replacementM)` returns a copy of `sM`, replacing all occurrences of `matchM` with `replacementM`, and then evaluates `sM`. Here, `sM` is an expression, equation, or condition involving symbolic matrix variables and matrix functions, and `matchM` specifies the symbolic matrix variables and matrix functions to be substituted. The substitution values `replacementM` must have the same size as `matchM`.

`sMnew = subs(sM,replacementM)` returns a copy of `sM`, replacing all occurrences of the default symbolic matrix variable in `sM` with `replacementM`, and then evaluates `sM`.

`sMnew = subs(sM)` returns a copy of `sM`, replacing symbolic matrix variables in `sM` with their assigned values in the MATLAB workspace, and then evaluates `sM`. Variables with no assigned values remain as variables.

## Examples

### Single Substitution

Replace  $a$  with 4 in this expression.

```
syms a b
expr = subs(a + b, a, 4)

expr = b + 4
```

Replace  $a*b$  with 5 in this expression.

```
expr = subs(a*b^2, a*b, 5)

expr = 5 b
```

### Default Substitution Variable

Substitute the default symbolic scalar variable in this expression with  $a$ . If you do not specify the scalar variable or expression to replace, `subs` uses `symvar` to find the default variable. For  $x + y$ , the default variable is  $x$ .

```
syms x y a
var = symvar(x + y, 1)

var = x
```

Therefore, `subs` replaces  $x$  with  $a$ .

```
expr = subs(x + y, a)

expr = a + y
```

### Evaluate Expression with New Values

When you assign a new value to a symbolic scalar variable, expressions containing the variable are not automatically evaluated. Instead, evaluate expressions by using `subs`.

Define the expression  $y = x^2$ .

```
syms x
y = x^2;
```

Assign 2 to  $x$ . The value of  $y$  is still  $x^2$  instead of 4.

```
x = 2;
y

y = x^2
```

Evaluate  $y$  with the new value of  $x$  by using `subs`.

```

yeval = subs(y)
yeval = 4

```

### Multiple Substitutions

Make multiple substitutions by specifying the variables to substitute and the new values as vectors.

```

syms a b
expr = subs(cos(a) + sin(b), [a,b], [sym("alpha"),2])
expr = sin(2) + cos(alpha)

```

Alternatively, for multiple substitutions, use cell arrays.

```

expr = subs(cos(a) + sin(b), {a,b}, {sym("alpha"),2})
expr = sin(2) + cos(alpha)

```

### Substitute Scalars with Arrays

Replace the symbolic scalar variable `a` in this expression with the 3-by-3 magic square matrix. Note that the constant 1 expands to the 3-by-3 matrix with all its elements equal to 1.

```

syms a t
expr3by3 = subs(exp(a*t) + 1, a, -magic(3))
expr3by3 =

```

$$\begin{pmatrix} e^{-8t} + 1 & e^{-t} + 1 & e^{-6t} + 1 \\ e^{-3t} + 1 & e^{-5t} + 1 & e^{-7t} + 1 \\ e^{-4t} + 1 & e^{-9t} + 1 & e^{-2t} + 1 \end{pmatrix}$$

You can also replace an element of a vector, matrix, or array with a nonscalar value. For example, create these 2-by-2 matrices.

```
A = sym("A",[2,2])
```

```
A =
```

$$\begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix}$$

```
B = sym("B",[2,2])
```

```
B =
```

$$\begin{pmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{pmatrix}$$

Replace the first element of the matrix `A` with the matrix `B`. While making this substitution, `subs` expands the 2-by-2 matrix `A` into this 4-by-4 matrix.

```
A4by4 = subs(A, A(1,1), B)
```

```
A4by4 =

$$\begin{pmatrix} B_{1,1} & B_{1,2} & A_{1,2} & A_{1,2} \\ B_{2,1} & B_{2,2} & A_{1,2} & A_{1,2} \\ A_{2,1} & A_{2,1} & A_{2,2} & A_{2,2} \\ A_{2,1} & A_{2,1} & A_{2,2} & A_{2,2} \end{pmatrix}$$

```

`subs` does not let you replace a nonscalar or matrix with a scalar that shrinks the matrix size.

### Substitute Symbolic Scalar Variables in Structure Array

Create a structure array with symbolic expressions as the field values.

```
syms x y z
S = struct("f1",x*y,"f2",y + z,"f3",y^2)

S = struct with fields:
  f1: x*y
  f2: y + z
  f3: y^2
```

Replace the symbolic scalar variables `x`, `y`, and `z` with numeric values.

```
Sval = subs(S,[x y z],[0.5 1 1.5])

Sval = struct with fields:
  f1: 1/2
  f2: 5/2
  f3: 1
```

### Substitute Multiple Scalars with Arrays

Replace the symbolic scalar variables `x` and `y` in an expression with these 2-by-2 matrices. When you make multiple substitutions involving vectors or matrices, use cell arrays to specify the variables to substitute and the new values.

```
syms x y
expr = subs(x*y, {x,y}, {[0 1; -1 0], [1 -1; -2 1]})

expr =

$$\begin{pmatrix} 0 & -1 \\ 2 & 0 \end{pmatrix}$$

```

Note that because `x` and `y` are scalars, these substitutions are element-wise.

```
expr = [0 1; -1 0].*[1 -1; -2 1]

expr = 2x2
    0    -1
```

2 0

## Substitutions in Equations

Eliminate scalar variables from an equation by using the variable's value from another equation. In the second equation, isolate the variable on the left side using `isolate`, and then substitute the right side with the variable in the first equation.

First, declare the equations `eqn1` and `eqn2`.

```
syms x y
eqn1 = sin(x)+y == x^2 + y^2;
eqn2 = y*x == cos(x);
```

Isolate `y` in `eqn2` by using `isolate`.

```
eqn2 = isolate(eqn2,y)
```

```
eqn2 =
      y =  $\frac{\cos(x)}{x}$ 
```

Eliminate `y` from `eqn1` by substituting the left side of `eqn2` with the right side of `eqn2`.

```
eqn1 = subs(eqn1,lhs(eqn2),rhs(eqn2))
```

```
eqn1 =
      sin(x) +  $\frac{\cos(x)}{x}$  =  $\frac{\cos(x)^2}{x^2}$  + x^2
```

## Substitutions in Functions

Replace `x` with `a` in this symbolic function.

```
syms x y a
syms f(x,y)
f(x,y) = x + y;
f = subs(f,x,a)
```

```
f(x, y) = a + y
```

`subs` replaces the values in the symbolic function formula, but it does not replace input arguments of the function.

```
formula = formula(f)
```

```
formula = a + y
```

```
args = argnames(f)
```

```
args = (x y)
```

Replace the arguments of a symbolic function explicitly.

```
syms x y
f(x,y) = x + y;
f(a,y) = subs(f,x,a);
f
```

```
f(a, y) = a + y
```

### Substitute Variables with Corresponding Values from Structure

Suppose you want to verify the solutions of this system of equations.

```
syms x y
eqs = [x^2 + y^2 == 1, x == y];
S = solve(eqs,[x y]);
S.x
```

```
ans =

$$\begin{pmatrix} -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \end{pmatrix}$$

```

```
S.y
```

```
ans =

$$\begin{pmatrix} -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \end{pmatrix}$$

```

Verify the solutions by substituting the solutions into the original system.

```
tf = isAlways(subs(eqs,S))
```

```
tf = 2×2 logical array
```

```
1 1
1 1
```

### Substitution Involving First-Order and Second-Order Derivatives

Create a symbolic expression that involves first-order and second-order derivatives.

```
syms r(t)
expr = diff(r,t,t) + diff(r,t)
```

```
expr(t) =

$$\frac{\partial^2}{\partial t^2} r(t) + \frac{\partial}{\partial t} r(t)$$

```



Substitute the first-order derivative with another variable  $v_0$  and the second-order derivative with another variable  $D2r$ . To do so, you must first substitute the derivative with the higher order, which is the second-order derivative.

```
syms v_0 D2r
exprnew = subs(expr,diff(r,t,t),D2r);
exprnew = subs(exprnew,diff(r,t),v_0)

exprnew(t) = D2r + v_0
```

You can also make these substitutions in a single `subs` call by specifying the variables to replace and their replacements as vectors.

```
exprnew = subs(expr,[diff(r,t,t) diff(r,t)],[D2r v_0])

exprnew(t) = D2r + v_0
```

If you want to substitute only the first-order derivative while keeping the second-order derivative intact, you can continue with the previous workflow by changing the temporary variable  $D2r$  back to  $\frac{\partial^2}{\partial t^2}r(t)$ .

```
exprnew = subs(exprnew,D2r,diff(r,t,t))

exprnew(t) =
     $\frac{\partial^2}{\partial t^2} r(t) + v_0$ 
```

For comparison, if you first substitute the first-order derivative in the original expression with  $v_0$ , then the result is  $v_0$ . Here, `subs` replaces the first-order derivative with  $v_0$  and the second-order derivative with  $\frac{\partial}{\partial t} \frac{\partial r(t)}{\partial t} = \frac{\partial}{\partial t} v_0 = 0$ . This substitution results in an expression that does not contain a second-order derivative.

```
syms v_0
exprnew = subs(expr,diff(r,t),v_0)

exprnew(t) = v_0
```

## Substitute Symbolic Matrix Variables with Arrays

Define the product of two 2-by-2 matrices. Declare the matrices as symbolic matrix variables with the `symmatrix` data type.

```
syms X Y [2 2] matrix
sM = X*Y

sM = X Y
```

Replace the matrix variables  $X$  and  $Y$  with 2-by-2 symbolic matrices. When you make multiple substitutions involving vectors or matrices, use cell arrays to specify the matrix variables to be substituted and their new values. The new values must have the same size as the matrix variables to be substituted.

```
S = subs(sM,{X,Y},{[0 sqrt(sym(2)); sqrt(sym(2)) 0], [1 -1; -2 1]})
```

```
S =
  \Sigma_1
  where
```

$$\Sigma_1 = \begin{pmatrix} -2\sqrt{2} & \sqrt{2} \\ \sqrt{2} & -\sqrt{2} \end{pmatrix}$$

Convert the expression `S` to the `sym` data type to show the result of the substituted matrix multiplication.

```
Ssym = symmatrix2sym(S)
```

```
Ssym =
  \begin{pmatrix} -2\sqrt{2} & \sqrt{2} \\ \sqrt{2} & -\sqrt{2} \end{pmatrix}
```

### Characteristic Polynomial of Matrix

Create a matrix of symbolic numbers.

```
A = sym([1 4 2; 4 1 2; 2 2 3])
```

```
A =
  \begin{pmatrix} 1 & 4 & 2 \\ 4 & 1 & 2 \\ 2 & 2 & 3 \end{pmatrix}
```

Compute the coefficients of the characteristic polynomial of `A` using the `charpoly` function.

```
c = charpoly(A)
```

```
c = (1 -5 -17 21)
```

Next, define `X` as a 3-by-3 symbolic matrix variable. Use the coefficients `c` to create the polynomial  $p(X) = c_1X^3 + c_2X^2 + c_3X + c_4I_3$ , where `X` is an indeterminate that represents a 3-by-3 matrix.

```
syms X [3 3] matrix
p = c(1)*X^3 + c(2)*X^2 + c(3)*X + c(4)*X^0
```

```
p = 21 I_3 - 17 X - 5 X^2 + X^3
```

Substitute `X` in the polynomial  $p(X)$  with `A` using the `subs` function. According to the Cayley-Hamilton theorem, this substitution results in a 3-by-3 zero matrix because the coefficients `c` are the characteristic polynomial of `A`. Use `symmatrix2sym` to convert the substituted expression to a matrix of symbolic numbers.

```
Y = subs(p,A)
```

```
Y =
```

$$-17 \Sigma_1 - 5 \Sigma_1^2 + \Sigma_1^3 + 21 I_3$$

where

$$\Sigma_1 = \begin{pmatrix} 1 & 4 & 2 \\ 4 & 1 & 2 \\ 2 & 2 & 3 \end{pmatrix}$$

`Z = symmatrix2sym(Y)`

$$Z = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

### Substitute Variables in Symbolic Matrix Function

Define the function  $f(\mathbf{A}) = \mathbf{A}^2 - 2\mathbf{A} + \mathbf{I}_2$ , where  $\mathbf{A}$  is a 2-by-2 matrix and  $\mathbf{I}_2$  is a 2-by-2 identity matrix. Substitute the variable  $\mathbf{A}$  with another expression and evaluate the new function.

Create a 2-by-2 symbolic matrix variable  $\mathbf{A}$ . Create a symbolic matrix function  $f(\mathbf{A})$ , keeping the existing definition of  $\mathbf{A}$  in the workspace. Assign the polynomial expression of  $f(\mathbf{A})$ .

```
syms A 2 matrix
syms f(A) 2 matrix keepargs
f(A) = A*A - 2*A + eye(2)
```

$$f(\mathbf{A}) = \mathbf{I}_2 - 2\mathbf{A} + \mathbf{A}^2$$

Next, create new symbolic matrix variables  $\mathbf{B}$  and  $\mathbf{C}$ . Create a new symbolic matrix function  $g(\mathbf{B}, \mathbf{C})$ , keeping the existing definitions of  $\mathbf{B}$  and  $\mathbf{C}$  in the workspace.

```
syms B C 2 matrix
syms g(B,C) 2 matrix keepargs
```

Substitute the variable  $\mathbf{A}$  in  $f(\mathbf{A})$  with  $\mathbf{B} + \mathbf{C}$ . Assign the substituted result to the new function  $g(\mathbf{B}, \mathbf{C})$ .

```
g(B,C) = subs(f,A,B+C)
```

$$g(\mathbf{B}, \mathbf{C}) = (\mathbf{B} + \mathbf{C})^2 + \mathbf{I}_2 - 2\mathbf{B} - 2\mathbf{C}$$

Evaluate  $g(\mathbf{B}, \mathbf{C})$  for the matrix values  $\mathbf{B} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$  and  $\mathbf{C} = \begin{bmatrix} 1 & -1 \\ -2 & 1 \end{bmatrix}$  using `subs`.

```
S = subs(g(B,C),{B,C},{[0 1; -1 0],[1 -1; -2 1]})
```

`S =`

$$-2 \Sigma_1 - 2 \Sigma_2 + (\Sigma_1 + \Sigma_2)^2 + I_2$$

where

$$\Sigma_1 = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$$

$$\Sigma_2 = \begin{pmatrix} 1 & -1 \\ -2 & 1 \end{pmatrix}$$

Convert the expression `S` from the `symmatrix` data type to the `sym` data type to show the result of the substituted polynomial.

```
Ssym = symmatrix2sym(S)
```

```
Ssym =  
      (0 0)  
      (0 0)
```

### Substitute Symbolic Matrix Variables and Matrix Functions in Equation

Define the equation  $\frac{\partial}{\partial \mathbf{X}^T} \frac{\partial}{\partial \mathbf{X}} f(\mathbf{X}, \mathbf{A}) = 2 \mathbf{A}$ , where  $\mathbf{A}$  is a 3-by-3 matrix and  $\mathbf{X}$  is a 3-by-1 matrix.

Substitute  $f(\mathbf{X}, \mathbf{A})$  with another symbolic expression and  $\mathbf{A}$  with symbolic values. Check if the equation is true for these values.

Create two symbolic matrix variables  $\mathbf{A}$  and  $\mathbf{X}$ . Create a symbolic matrix function  $f(\mathbf{X}, \mathbf{A})$ , keeping the existing definitions of  $\mathbf{A}$  and  $\mathbf{X}$  in the workspace. Create the equation.

```
syms A [3 3] matrix  
syms X [3 1] matrix  
syms f(X,A) [1 1] matrix keepargs  
eq = diff(diff(f,X),X.') == 2*A
```

```
eq(X, A) =  
           $\frac{\partial}{\partial \mathbf{X}^T} \frac{\partial}{\partial \mathbf{X}} f(\mathbf{X}, \mathbf{A}) = 2 \mathbf{A}$ 
```

Substitute  $f(\mathbf{X}, \mathbf{A})$  with  $\mathbf{X}^T \mathbf{A} \mathbf{X}$  and evaluate the second-order differential function in the equation for this expression.

```
eq = subs(eq,f,X.'*A*X)
```

```
eq(X, A) = AT + A = 2 A
```

Substitute  $\mathbf{A}$  with the Hilbert matrix of order 3.

```
eq = subs(eq,A,hilb(3))
```

```
eq(X, A) =
```

$$\Sigma_1 + \Sigma_1^T = 2 \Sigma_1$$

where

$$\Sigma_1 = \begin{pmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \end{pmatrix}$$

Check if the equation is true for these values by using `isAlways`. Because `isAlways` accepts only a symbolic input of type `symfun` or `sym`, convert `eq` from type `symfunmatrix` to type `symfun` before using `isAlways`.

```
tf = isAlways(symfunmatrix2symfun(eq))
```

```
tf = 3×3 logical array
```

```
1    1    1
1    1    1
1    1    1
```

### Evaluate Expression Involving Symbolic Matrix Variables

Define the expression  $\mathbf{XY}^2 - \mathbf{YX}^2$ , where  $\mathbf{X}$  and  $\mathbf{Y}$  are 3-by-3 matrices. Create the matrices as symbolic matrix variables.

```
syms X Y [3 3] matrix
C = X*Y^2 - Y*X^2
```

$$C = XY^2 - YX^2$$

Assign values to the matrices  $\mathbf{X}$  and  $\mathbf{Y}$ .

```
X = [-1 2 pi; 0 1/2 2; 2 1 0];
Y = [3 2 2; -1 2 1; 1 2 -1];
```

Evaluate the expression  $\mathbf{C}$  with the assigned values of  $\mathbf{X}$  and  $\mathbf{Y}$  by using `subs`.

```
Cnew = subs(C)
```

$$C_{\text{new}} = -\Sigma_1 \Sigma_2^2 + \Sigma_2 \Sigma_1^2$$

where

$$\Sigma_1 = \begin{pmatrix} 3 & 2 & 2 \\ -1 & 2 & 1 \\ 1 & 2 & -1 \end{pmatrix}$$

$$\Sigma_2 = \begin{pmatrix} -1 & 2 & \pi \\ 0 & \frac{1}{2} & 2 \\ 2 & 1 & 0 \end{pmatrix}$$

Convert the result from the `symmatrix` data type to the `double` data type.

```
Cnum = double(Cnew)
```

```
Cnum = 3×3
```

```
-42.8496  -13.3584  -13.4336  
-0.7168   3.1416   0.0752  
-3.2832  29.8584  16.4248
```

## Input Arguments

### **s** — Symbolic input

symbolic scalar variable | symbolic expression | symbolic equation | symbolic function | symbolic array | symbolic matrix | structure

Symbolic input, specified as a symbolic scalar variable, expression, equation, function, array, matrix, or a structure.

Data Types: `sym` | `symfun` | `struct`

### **match** — Scalar variable to substitute

symbolic scalar variable | symbolic function | symbolic expression | symbolic array | cell array

Scalar variable to substitute, specified as a symbolic scalar variable, function, expression, array, or a cell array.

Data Types: `sym` | `symfun` | `cell`

### **replacement** — New replacement value

number | symbolic number | symbolic scalar variable | symbolic function | symbolic expression | symbolic array | structure | cell array

New replacement value to substitute with, specified as a number, symbolic number, scalar variable, function, expression, array, structure, or a cell array.

Data Types: `sym` | `symfun` | `single` | `double` | `int8` | `int16` | `int32` | `int64` | `uint8` | `uint16` | `uint32` | `uint64` | `struct` | `cell`

### **sM** — Symbolic input

symbolic matrix variable | symbolic matrix function | symbolic expression | symbolic equation | symbolic condition

Symbolic input, specified as a symbolic matrix variable, matrix function, expression, equation, or condition.

Data Types: `symmatrix` | `symfunmatrix`

### **matchM** — Matrix variable or function to substitute

symbolic matrix variable | symbolic matrix function | symbolic expression | cell array

Matrix variable or function to substitute, specified as a symbolic matrix variable, matrix function, expression, or a cell array.

Data Types: `symmatrix` | `symfunmatrix` | `cell`

**replacementM — New replacement value**

number | symbolic number | symbolic matrix variable | symbolic matrix function | symbolic expression  
| symbolic array | cell array

New replacement value to substitute with, specified as a number, symbolic number, matrix variable, matrix function, expression, array, or a cell array. **replacementM** must have the same size as **matchM** or the default symbolic matrix variable in **sM**.

Data Types: sym | symmatrix | symfunmatrix | single | double | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64 | struct | cell

**Tips**

- `subs(s, __)` does not modify `s`. To modify `s`, use `s = subs(s, __)`.
- If `s` is a univariate polynomial and `replacement` is a numeric matrix, use `polyvalm(sym2poly(s), replacement)` to evaluate `s` as a matrix. All constant terms are replaced with the constant multiplied by an identity matrix.

**Version History**

Introduced before R2006a

**R2023b: Substitute symbolic matrix variables with their workspace values**

You can use the syntax `subs(sM)` to substitute the symbolic matrix variables and matrix functions in `sM` with their assigned values in the MATLAB workspace and then evaluate `sM`. Variables with no assigned values remain as variables. For an example, see “Evaluate Expression Involving Symbolic Matrix Variables” on page 6-1445.

**R2022b: Substitute symbolic matrix variables and matrix functions in symbolic equations or conditions**

The `subs` function accepts a symbolic equation or condition of type `symmatrix` or `symfunmatrix` as the first input argument. For an example, see “Substitute Symbolic Matrix Variables and Matrix Functions in Equation” on page 6-1444.

**R2022b: Symbolic substitution involving derivatives follows input order**

*Behavior changed in R2022b*

Starting in R2022b, symbolic substitution involving derivatives or the `diff` function follows the input order of the symbolic objects to be substituted. For example, this code performs symbolic substitution involving the `diff` function.

```
syms m k x(t)
syms x_t x_t_ddot
eqSHM = m*diff(x(t),t,2) == -k*x(t);
eqSHMnew = subs(eqSHM,[x(t) diff(x(t),t,2)],[x_t x_t_ddot])
```

Before R2022b, the code returns this output:

```
eqSHMnew =
m*x_t_ddot == -k*x_t
```

Starting in R2022b, the code returns this output:

```
eqSHMnew =  
0 == -k*x_t
```

The difference in output is due to `subs` now first substituting  $x(t)$  with  $x\_t$ , resulting in  $m \cdot \text{diff}(x\_t, t, 2)$  on the left side, which is equal to 0. To obtain the substitution result as in previous releases, specify the  $\text{diff}(x(t), t, 2)$  term first so that it is substituted before the  $x(t)$  term.

```
eqSHMnew = subs(eqSHM, [diff(x(t), t, 2) x(t)], [x_t_ddot x_t])  
  
eqSHMnew =  
m*x_t_ddot == -k*x_t
```

### **R2022a: Substitute variables in symbolic matrix functions**

The `subs` function accepts a symbolic matrix function of type `symfunmatrix` as the first input argument. For an example, see “Substitute Variables in Symbolic Matrix Function” on page 6-1443.

### **R2021b: Substitute symbolic matrix variables in symbolic expressions**

The `subs` function accepts a symbolic expression of type `symmatrix` as the first input argument. For examples, see “Substitute Symbolic Matrix Variables with Arrays” on page 6-1441 and “Characteristic Polynomial of Matrix” on page 6-1442.

## **See Also**

### **Functions**

`double` | `lhs` | `rhs` | `simplify` | `subexpr` | `vpa`

### **Topics**

“Substitutions in Symbolic Expressions” on page 1-34

“Substitute Parameters in Symbolic Solutions” on page 3-144

“Substitute Elements in Symbolic Matrices” on page 3-146

“Substitute Scalars with Matrices” on page 3-148

“Substitute Values into Symbolic Expressions Using `subs`” on page 3-150



## svd

Singular value decomposition of symbolic matrix

### Syntax

```
sigma = svd(A)

[U,S,V] = svd(A)
[U,S,V] = svd(A,0)
[U,S,V] = svd(A,'econ')

[___] = svd( ___,outputForm)
```

### Description

`sigma = svd(A)` returns a vector `sigma` containing the singular values of a symbolic matrix `A`.

`[U,S,V] = svd(A)` returns numeric unitary matrices `U` and `V` with the columns containing the singular vectors, and a diagonal matrix `S` containing the singular values. The matrices satisfy the condition  $A = U*S*V'$ , where  $V'$  is the Hermitian transpose (the complex conjugate transpose) of  $V$ . The singular vector computation uses variable-precision arithmetic. `svd` does not compute symbolic singular vectors. Therefore, the input matrix `A` must be convertible to floating-point numbers. For example, it can be a matrix of symbolic numbers.

`[U,S,V] = svd(A,0)` returns an economy-size decomposition of matrix `A`. If `A` is an  $m$ -by- $n$  matrix with  $m > n$ , then `svd` computes only the first  $n$  columns of `U` and `S` is an  $n$ -by- $n$  matrix. For  $m \leq n$ , this syntax is equivalent to `svd(A)`.

`[U,S,V] = svd(A,'econ')` returns a different economy-size decomposition of matrix `A`. If `A` is an  $m$ -by- $n$  matrix with  $m \geq n$ , then this syntax is equivalent to `svd(A,0)`. For  $m < n$ , `svd` computes only the first  $m$  columns of `V` and `S` is an  $m$ -by- $m$  matrix.

`[___] = svd( ___,outputForm)` returns the singular values in the form specified by `outputForm` using any of the input or output arguments in previous syntaxes. Specify `outputForm` as `'vector'` to return the singular values as a column vector or as `'matrix'` to return the singular values as a diagonal matrix.

### Examples

#### Singular Values of Symbolic Numbers

Compute the singular values of the symbolic 5-by-5 magic square. The result is a column vector.

```
A = sym(magic(5));
sigma = svd(A)

sigma =
```

$$\begin{pmatrix} 65 \\ \sqrt{5} \sqrt{\sqrt{1345} + 65} \\ \sqrt{65} \sqrt{\sqrt{5} + 5} \\ \sqrt{65} \sqrt{5 - \sqrt{5}} \\ \sqrt{5} \sqrt{65 - \sqrt{1345}} \end{pmatrix}$$

Alternatively, specify the 'matrix' option to return the singular values as a diagonal matrix.

```
S = svd(A, 'matrix')
```

$$S = \begin{pmatrix} 65 & 0 & 0 & 0 & 0 \\ 0 & \sqrt{5} \sqrt{\sqrt{1345} + 65} & 0 & 0 & 0 \\ 0 & 0 & \sqrt{65} \sqrt{\sqrt{5} + 5} & 0 & 0 \\ 0 & 0 & 0 & \sqrt{65} \sqrt{5 - \sqrt{5}} & 0 \\ 0 & 0 & 0 & 0 & \sqrt{5} \sqrt{65 - \sqrt{1345}} \end{pmatrix}$$

### Singular Values of Symbolic Expressions

Compute singular values of a matrix whose elements are symbolic expressions.

```
syms t real
A = [0 1; -1 0];
E = expm(t*A)
```

$$E = \begin{pmatrix} \cos(t) & \sin(t) \\ -\sin(t) & \cos(t) \end{pmatrix}$$

```
sigma = svd(E)
```

$$\text{sigma} = \begin{pmatrix} \sqrt{\cos(t)^2 + \sin(t)^2} \\ \sqrt{\cos(t)^2 + \sin(t)^2} \end{pmatrix}$$

Simplify the result.

```
sigma = simplify(sigma)
```

$$\text{sigma} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

For further computations, remove the assumption that  $t$  is real by recreating it using `syms`.

```
syms t
```

### Convert Symbolic Singular Values to Floating-Point Numbers

Create a 5-by-5 symbolic matrix from the magic square of order 6. Compute the singular values of the matrix using `svd`.

```
M = magic(6);
A = sym(M(1:5,1:5));
sigma = svd(A)
```

```
sigma =

$$\begin{pmatrix} \sqrt{\text{root}(\sigma_1, z, 5)} \\ \sqrt{\text{root}(\sigma_1, z, 4)} \\ \sqrt{\text{root}(\sigma_1, z, 3)} \\ \sqrt{\text{root}(\sigma_1, z, 2)} \\ \sqrt{\text{root}(\sigma_1, z, 1)} \end{pmatrix}$$

```

where

$$\sigma_1 = z^5 - 11357 z^4 + 26691022 z^3 - 17903673324 z^2 + 979310921328 z - 1676258447616$$

The `svd` function cannot find the exact singular values in terms of symbolic numbers. Instead, it returns them in terms of the `root` function.

Use `vpa` to numerically approximate the singular values.

```
sigmaVpa = vpa(sigma)

sigmaVpa =

$$\begin{pmatrix} 91.903382299388875598380645217105 \\ 41.667523645705677947038130902387 \\ 33.389311761352625550607303429805 \\ 7.6138651481371046117950798870896 \\ 1.3299296132187199146053915272808 \end{pmatrix}$$

```

### Singular Values and Singular Vectors

Compute the singular values and singular vectors of the 5-by-5 magic square.

```
old = digits(10);
A = sym(magic(5))
```

```
A =

$$\begin{pmatrix} 17 & 24 & 1 & 8 & 15 \\ 23 & 5 & 7 & 14 & 16 \\ 4 & 6 & 13 & 20 & 22 \\ 10 & 12 & 19 & 21 & 3 \\ 11 & 18 & 25 & 2 & 9 \end{pmatrix}$$

```

```
[U,S,V] = svd(A)
```

```
U =
```

$$\begin{pmatrix} 0.4472135955 & 0.5456348731 & 0.5116672736 & -0.1954395076 & -0.4497583632 \\ 0.4472135955 & 0.4497583632 & -0.1954395076 & 0.5116672736 & 0.5456348731 \\ 0.4472135955 & -1.547164189e-27 & -0.632455532 & -0.632455532 & 1.213456644e-27 \\ 0.4472135955 & -0.4497583632 & -0.1954395076 & 0.5116672736 & -0.5456348731 \\ 0.4472135955 & -0.5456348731 & 0.5116672736 & -0.1954395076 & 0.4497583632 \end{pmatrix}$$

$$S = \begin{pmatrix} 65.0 & 0 & 0 & 0 & 0 \\ 0 & 22.54708869 & 0 & 0 & 0 \\ 0 & 0 & 21.68742536 & 0 & 0 \\ 0 & 0 & 0 & 13.403566 & 0 \\ 0 & 0 & 0 & 0 & 11.90078954 \end{pmatrix}$$

$$V = \begin{pmatrix} 0.4472135955 & 0.4045164361 & 0.2465648962 & 0.6627260007 & 0.3692782866 \\ 0.4472135955 & 0.005566159714 & 0.6627260007 & -0.2465648962 & -0.5476942741 \\ 0.4472135955 & -0.8201651916 & 1.767621593e-27 & 9.706484055e-28 & 0.3568319751 \\ 0.4472135955 & 0.005566159714 & -0.6627260007 & 0.2465648962 & -0.5476942741 \\ 0.4472135955 & 0.4045164361 & -0.2465648962 & -0.6627260007 & 0.3692782866 \end{pmatrix}$$

`digits(old)`

Compute the product of `U`, `S`, and the Hermitian transpose of `V` with the 10-digit accuracy. The result is the original matrix `A` with all its elements converted to floating-point numbers.

`vpa(U*S*V',10)`

$$\text{ans} = \begin{pmatrix} 17.0 & 24.0 & 1.0 & 8.0 & 15.0 \\ 23.0 & 5.0 & 7.0 & 14.0 & 16.0 \\ 4.0 & 6.0 & 13.0 & 20.0 & 22.0 \\ 10.0 & 12.0 & 19.0 & 21.0 & 3.0 \\ 11.0 & 18.0 & 25.0 & 2.0 & 9.0 \end{pmatrix}$$

### Thin or Economy SVD

Calculate the full and economy-size decompositions of a rectangular matrix within 8-digit accuracy.

```
old = digits(8);
A = sym([1 2; 3 4; 5 6; 7 8])
```

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{pmatrix}$$

`[U,S,V] = svd(A)`

`U =`

$$\begin{pmatrix} 0.15248323 & -0.82264747 & -0.39450102 & -0.37995913 \\ 0.34991837 & -0.42137529 & 0.24279655 & 0.80065588 \\ 0.54735351 & -0.020103103 & 0.69790998 & -0.46143436 \\ 0.74478865 & 0.38116908 & -0.5462055 & 0.040737612 \end{pmatrix}$$

$$S = \begin{pmatrix} 14.269095 & 0 \\ 0 & 0.62682823 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}$$

$$V = \begin{pmatrix} 0.64142303 & 0.7671874 \\ 0.7671874 & -0.64142303 \end{pmatrix}$$

`[U,S,V] = svd(A, 'econ')`

$$U = \begin{pmatrix} 0.15248323 & -0.82264747 \\ 0.34991837 & -0.42137529 \\ 0.54735351 & -0.020103103 \\ 0.74478865 & 0.38116908 \end{pmatrix}$$

$$S = \begin{pmatrix} 14.269095 & 0 \\ 0 & 0.62682823 \end{pmatrix}$$

$$V = \begin{pmatrix} 0.64142303 & 0.7671874 \\ 0.7671874 & -0.64142303 \end{pmatrix}$$

`digits(old)`

Since  $A$  is a 4-by-2 matrix, `svd(A, 'econ')` returns fewer columns in  $U$  and fewer rows in  $S$  compared to a full decomposition. Extra rows of zeros in  $S$  are excluded, along with the corresponding columns in  $U$  that would multiply with those zeros in the expression  $A = U*S*V'$ .

## Input Arguments

### **A — Input matrix**

symbolic matrix

Input matrix, specified as a symbolic matrix. For syntaxes with one output argument, the elements of  $A$  can be symbolic numbers, variables, expressions, or functions. For syntaxes with three output arguments, the elements of  $A$  must be convertible to floating-point numbers.

### **outputForm — Output format of singular values**

'vector' | 'matrix'

*Since R2021b*

Output format of singular values, specified as 'vector' or 'matrix'. This option allows you to specify whether the singular values are returned as a column vector or diagonal matrix. The default behavior varies according to the number of outputs specified:

- If you specify one output, such as `sigma = svd(A)`, then the singular values are returned as a column vector by default.
- If you specify three outputs, such as `[U,S,V] = svd(A)`, then the singular values are returned as a diagonal matrix, `S`, by default.

## Output Arguments

### **sigma — Singular values**

symbolic vector | vector of symbolic numbers

Singular values of a matrix, returned as a vector. If `sigma` is a vector of numbers, then its elements are sorted in descending order.

### **U — Singular vectors**

matrix of symbolic numbers

Singular vectors, returned as a unitary matrix. Each column of this matrix is a singular vector.

### **S — Singular values**

matrix of symbolic numbers

Singular values, returned as a diagonal matrix. Diagonal elements of this matrix appear in descending order.

### **V — Singular vectors**

matrix of symbolic numbers

Singular vectors, returned as a unitary matrix. Each column of this matrix is a singular vector.

## Tips

- The second arguments `0` and `'econ'` only affect the shape of the returned matrices. These arguments do not affect the performance of the computations.
- Calling `svd` for numeric matrices that are not symbolic objects invokes the MATLAB `svd` function.
- Matrix computations involving many symbolic variables can be slow. To increase the computational speed, reduce the number of symbolic variables by substituting the given values for some variables.

## Version History

Introduced before R2006a

### **R2021b: Return singular values as column vector or diagonal matrix**

The `svd` function accepts the `outputForm` input argument. You can specify `outputForm` as `'vector'` or `'matrix'` to return singular values as a column vector or a diagonal matrix.

### **R2021b: `sigma = svd(A)` returns singular values in terms of the root function**

*Behavior changed in R2021b*

If `sigma = svd(A)` cannot find the exact singular values in terms of symbolic numbers, it now returns the exact singular values in terms of the `root` function instead. In previous releases, `sigma = svd(A)` returns the singular values as floating-point numbers.

For example, compute the singular values of a 5-by-5 symbolic matrix. The `svd` function returns the exact singular values in terms of the `root` function. This is consistent with the results returned by the `solve` or `root` function when solving for the roots of a polynomial.

```
M = magic(6);
A = sym(M(1:5,1:5));
sigma = svd(A)

sigma =
root(z^5 - 11357*z^4 + 26691022*z^3 - 17903673324*z^2 + 979310921328*z - 1676258447616, z, 5)^(1/2)
root(z^5 - 11357*z^4 + 26691022*z^3 - 17903673324*z^2 + 979310921328*z - 1676258447616, z, 4)^(1/2)
root(z^5 - 11357*z^4 + 26691022*z^3 - 17903673324*z^2 + 979310921328*z - 1676258447616, z, 3)^(1/2)
root(z^5 - 11357*z^4 + 26691022*z^3 - 17903673324*z^2 + 979310921328*z - 1676258447616, z, 2)^(1/2)
root(z^5 - 11357*z^4 + 26691022*z^3 - 17903673324*z^2 + 979310921328*z - 1676258447616, z, 1)^(1/2)
```

Use `vpa` to numerically approximate the singular values.

```
sigmaVpa = vpa(sigma)

sigmaVpa =
91.903382299388875598380645217105
41.667523645705677947038130902387
33.389311761352625550607303429805
7.6138651481371046117950798870896
1.3299296132187199146053915272808
```

## See Also

`chol` | `digits` | `eig` | `lu` | `inv` | `qr` | `svd` | `vpa`

## Topics

“Singular Value Decomposition” on page 3-347

## sym

Create symbolic variables, expressions, functions, matrices

### Syntax

```
x = sym("x")
A = sym("a",[n1 ... nM])
A = sym("a",n)
```

```
sym( ___,set)
sym( ___, "clear")
```

```
sym(num)
sym(num,flag)
sym(strnum)
```

```
symexpr = sym(h)
symexpr = sym(M)
```

### Description

`x = sym("x")` creates symbolic scalar variable `x`.

`A = sym("a",[n1 ... nM])` creates an `n1`-by-...-by-`nM` symbolic array filled with automatically generated elements. For example, `A = sym("a",[1 3])` creates the row vector `A = [a1 a2 a3]`. The generated elements `a1`, `a2`, and `a3` do not appear in the MATLAB workspace. For multidimensional arrays, these elements have the prefix `a` followed by the element's index using `_` as a delimiter, such as `a1_3_2`.

`A = sym("a",n)` creates an `n`-by-`n` symbolic matrix filled with automatically generated elements.

`sym( ___,set)` creates a symbolic variable or array and sets the assumption that the variable or all array elements belong to `set`. Here, `set` can be `"real"`, `"positive"`, `"integer"`, or `"rational"`. You also can combine multiple assumptions by specifying a string array or cell array of character vectors. For example, assume a positive rational value by specifying `set` as `["positive" "rational"]` or `{'positive','rational'}`.

`sym( ___, "clear")` clears assumptions set on a symbolic variable or array. You can specify `"clear"` after the input arguments in any of the previous syntaxes, except combining `"clear"` and `set`. You cannot set and clear an assumption in the same function call to `sym`.

`sym(num)` converts a number or numeric matrix specified by `num` to a symbolic number or symbolic matrix.

`sym(num,flag)` uses the technique specified by `flag` to convert floating-point numbers to symbolic numbers.

`sym(strnum)` converts the character vector or string specified by `strnum` to an accurate symbolic number without approximation.



`symexpr = sym(h)` creates a symbolic expression or matrix `symexpr` from an anonymous MATLAB function associated with the function handle `h`.

`symexpr = sym(M)` converts a symbolic matrix variable `M` of type `symmatrix` to an array of symbolic scalar variables `symexpr` of type `sym`.

## Examples

### Create Symbolic Variables

Create the symbolic variables `x` and `y`.

```
x = sym("x")
```

```
x = x
```

```
y = sym("y")
```

```
y = y
```

### Create Symbolic Vectors

Create a 1-by-4 symbolic vector `a` with automatically generated elements  $a_1, \dots, a_4$ .

```
a = sym("a",[1 4])
```

```
a = (a1 a2 a3 a4)
```

You can specify the format for the element names by using a format operator within the first argument. `sym` replaces `%d` with the index of the element to generate the element names.

```
b = sym("x_%d",[1 4])
```

```
b = (x1 x2 x3 x4)
```

However, these syntaxes do not create symbolic variables  $a_1, \dots, a_4, x_1, \dots, x_4$  in the MATLAB workspace. To access the elements of `a` and `b`, use the standard indexing methods.

```
a(1)
```

```
ans = a1
```

```
b(2:3)
```

```
ans = (x2 x3)
```

### Create Symbolic Matrices

Create a 3-by-4 symbolic matrix with automatically generated elements. The `sym` function generates matrix elements of the form  $A_{i,j}$ . Here, `sym` generates the elements  $A_{1,1}, \dots, A_{3,4}$ .

```
A = sym("A",[3 4])
```

```
A =
```

$$\begin{pmatrix} A_{1,1} & A_{1,2} & A_{1,3} & A_{1,4} \\ A_{2,1} & A_{2,2} & A_{2,3} & A_{2,4} \\ A_{3,1} & A_{3,2} & A_{3,3} & A_{3,4} \end{pmatrix}$$

Create a 4-by-4 matrix with the element names  $x_{1,1}, \dots, x_{4,4}$  by using a format operator within the first argument. `sym` replaces `%d` with the index of the element to generate the element names.

```
B = sym("x_%d_%d",4)
```

```
B =
```

$$\begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} & x_{1,4} \\ x_{2,1} & x_{2,2} & x_{2,3} & x_{2,4} \\ x_{3,1} & x_{3,2} & x_{3,3} & x_{3,4} \\ x_{4,1} & x_{4,2} & x_{4,3} & x_{4,4} \end{pmatrix}$$

These syntaxes do not create symbolic variables  $A_{1,1}, \dots, A_{3,4}, x_{1,1}, \dots, x_{4,4}$  in the MATLAB workspace. To access an element of a matrix, use parentheses.

```
A(2,3)
```

```
ans = A_{2,3}
```

```
B(4,2)
```

```
ans = x_{4,2}
```

### Create Symbolic Multidimensional Arrays

Create a 2-by-2-by-2 symbolic array with automatically generated elements  $a_{1,1,1}, \dots, a_{2,2,2}$ .

```
A = sym("a",[2 2 2])
```

```
A(:, :, 1) =
```

$$\begin{pmatrix} a_{1,1,1} & a_{1,2,1} \\ a_{2,1,1} & a_{2,2,1} \end{pmatrix}$$

```
A(:, :, 2) =
```

$$\begin{pmatrix} a_{1,1,2} & a_{1,2,2} \\ a_{2,1,2} & a_{2,2,2} \end{pmatrix}$$

### Create Symbolic Numbers

Convert numeric values to symbolic numbers or expressions. Use `sym` on subexpressions instead of the entire expression for better accuracy. Using `sym` on entire expressions is inaccurate because MATLAB® first converts the expression to a floating-point number, which loses accuracy. `sym` cannot always recover this lost accuracy.

```

inaccurate1 = sym(1/1234567)

inaccurate1 =

$$\frac{7650239286923505}{9444732965739290427392}$$


accurate1 = 1/sym(1234567)

accurate1 =

$$\frac{1}{1234567}$$


inaccurate2 = sym(sqrt(1234567))

inaccurate2 =

$$\frac{4886716562018589}{4398046511104}$$


accurate2 = sqrt(sym(1234567))

accurate2 =  $\sqrt{1234567}$ 

inaccurate3 = sym(exp(pi))

inaccurate3 =

$$\frac{6513525919879993}{281474976710656}$$


accurate3 = exp(sym(pi))

accurate3 =  $e^{\pi}$ 

```

### Create Large Symbolic Numbers

When creating symbolic numbers with 15 or more digits, use quotation marks to accurately represent the numbers.

```

inaccurateNum = sym(111111111111111111)

inaccurateNum = 11111111111111110656

accurateNum = sym("111111111111111111")

accurateNum = 111111111111111111

```

When you use quotation marks to create symbolic complex numbers, specify the imaginary part of a number as `1i`, `2i`, and so on.

```

sym("1234567 + 1i")

ans = 1234567 + i

```

### Convert Function Handles to Symbolic Expressions

Convert anonymous functions associated with MATLAB® handles to a symbolic expression and a symbolic matrix.

```
h_expr = @(x)(sin(x) + cos(x));  
sym_expr = sym(h_expr)
```

```
sym_expr = cos(x) + sin(x)
```

```
h_matrix = @(x)(x*pascal(3));  
sym_matrix = sym(h_matrix)
```

```
sym_matrix =  

$$\begin{pmatrix} x & x & x \\ x & 2x & 3x \\ x & 3x & 6x \end{pmatrix}$$

```

### Set Assumptions While Creating Variables

Create the symbolic variables  $x$ ,  $y$ ,  $z$ , and  $t$  while simultaneously setting assumptions that  $x$  is real,  $y$  is positive,  $z$  is rational, and  $t$  is a positive integer.

```
x = sym("x", "real");  
y = sym("y", "positive");  
z = sym("z", "rational");  
t = sym("t", ["positive" "integer"]);
```

Check the assumptions on  $x$ ,  $y$ ,  $z$ , and  $t$  using `assumptions`.

```
assumptions
```

```
ans = (t ∈ ℤ x ∈ ℝ z ∈ ℚ 1 ≤ t 0 < y)
```

For further computations, clear the assumptions using `assume`.

```
assume([x y z t], "clear")  
assumptions
```

```
ans =
```

```
Empty sym: 1-by-0
```

### Set Assumptions on Matrix Elements

Create a symbolic matrix and set assumptions on each element of that matrix.

```
A = sym("A%d%d", [2 2], "positive")
```

```
A =
```

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$$

Solve an equation involving the first element of A. MATLAB assumes that this element is positive.

```
solve(A(1,1)^2-1, A(1,1))
```

```
ans = 1
```

Check the assumptions on the elements of A by using `assumptions`.

```
assumptions(A)
```

```
ans = (0 < A11 0 < A12 0 < A21 0 < A22)
```

Clear all previously set assumptions on elements of the symbolic matrix by using `assume`.

```
assume(A, "clear");  
assumptions(A)
```

```
ans =
```

```
Empty sym: 1-by-0
```

Solve the same equation again.

```
solve(A(1,1)^2-1, A(1,1))
```

```
ans =  
  -1  
   1
```

## Choose Conversion Technique for Floating-Point Values

Convert `pi` to a symbolic value.

Choose the conversion technique by specifying the optional second argument, which can be "r", "f", "d", or "e". The default is "r". See the Input Arguments section for details about the conversion techniques.

```
r = sym(pi)
```

```
r =  $\pi$ 
```

```
f = sym(pi, "f")
```

```
f =  
  884279719003555  
  281474976710656
```

```
d = sym(pi, "d")
```

```
d = 3.1415926535897931159979634685442
```

```
e = sym(pi, "e")
```

$$e = \pi - \frac{198 \text{ eps}}{359}$$

### Convert Hessian Matrix

Create 3-by-3 and 3-by-1 symbolic matrix variables.

```
syms A [3 3] matrix
syms X [3 1] matrix
```

Find the Hessian matrix of  $\mathbf{X}^T \mathbf{A} \mathbf{X}$ .

```
f = X.'*A*X;
M = diff(f,X,X.')
```

$$M = A^T + A$$

Convert the result from a symbolic matrix variable to a matrix of symbolic scalar variables.

```
S = sym(M)
```

$$S = \begin{pmatrix} 2 A_{1,1} & A_{1,2} + A_{2,1} & A_{1,3} + A_{3,1} \\ A_{1,2} + A_{2,1} & 2 A_{2,2} & A_{2,3} + A_{3,2} \\ A_{1,3} + A_{3,1} & A_{2,3} + A_{3,2} & 2 A_{3,3} \end{pmatrix}$$

Alternatively, you can use `symmatrix2sym` to convert a symbolic matrix variable to an array of symbolic scalar variables.

```
S = symmatrix2sym(M)
```

$$S = \begin{pmatrix} 2 A_{1,1} & A_{1,2} + A_{2,1} & A_{1,3} + A_{3,1} \\ A_{1,2} + A_{2,1} & 2 A_{2,2} & A_{2,3} + A_{3,2} \\ A_{1,3} + A_{3,1} & A_{2,3} + A_{3,2} & 2 A_{3,3} \end{pmatrix}$$

## Input Arguments

### "x" — Variable name

string | character vector

Variable name, specified as a string or character vector. Argument `x` must be a valid variable name. That is, `x` must begin with a letter and contain only alphanumeric characters and underscores. To verify that the name is a valid variable name, use `isvarname`.

Example: "x", "y123", 'z\_1'

### "a" — Prefix for automatically generated matrix elements

character vector | string

Prefix for automatically generated matrix elements, specified as a string or character vector. Argument **a** must be a valid variable name. That is, **a** must begin with a letter and contain only alphanumeric characters and underscores. To verify that the name is a valid variable name, use `isvarname`.

If you specify the argument **a** and its vector, matrix, or array dimensions in the argument `[n1 ... nM]`, then **a** can include formatting operators such as `"a_%d_%d"`. For examples, see “Create Symbolic Vectors” on page 6-1457 and “Create Symbolic Matrices” on page 6-1457.

Example: `"a", "b", 'a_bc'`

### **[n1 ... nM] — Vector, matrix, or array dimensions**

vector of integers

Vector, matrix, or array dimensions, specified as a vector of integers. As a shortcut, you can create a square matrix by specifying only one integer. For example, `A = sym("A",3)` creates a square 3-by-3 matrix.

Example: `[2 3]`

### **set — Assumptions on symbolic variable or matrix**

string array | character vector | cell array of character vectors

Assumptions on the symbolic variable or matrix, specified as a character vector, string array, or cell array of character vectors. The available assumptions are `"integer"`, `"rational"`, `"real"`, or `"positive"`.

You can combine multiple assumptions by specifying a string array or cell array of character vectors. For example, assume a positive rational value by specifying `set` as `["positive" "rational"]` or `{'positive','rational'}`.

Example: `"integer"`

### **num — Numeric value to be converted to symbolic number or matrix**

number | symbolic constant | matrix of numbers

Numeric value to be converted to symbolic number or matrix, specified as a number, symbolic constant, or matrix of numbers.

Example: `pi`

### **flag — Conversion technique**

`"r"` (default) | `"d"` | `"e"` | `"f"`

Conversion technique, specified as one of the characters listed in this table.

|                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>"r"</code> | When <code>sym</code> uses the <i>rational</i> mode, it converts floating-point numbers obtained by evaluating expressions of the form $p/q$ , $p\pi/q$ , $\sqrt{p}$ , $2^q$ , and $10^q$ (for modest-sized integers $p$ and $q$ ) to the corresponding symbolic form. For example, <code>sym(1/10,"r")</code> returns <code>1/10</code> . This mode effectively compensates for the round-off error involved in the original evaluation but might not represent the floating-point value precisely. If <code>sym</code> cannot find a simple rational approximation, then it uses the same technique as it would use with the flag <code>"f"</code> . |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| "d" | When <code>sym</code> uses the <i>decimal</i> mode, it takes the number of digits from the current setting of <code>digits</code> . Conversions with fewer than 16 digits lose some accuracy, while more than 16 digits might not be warranted. For example, <code>sym(4/3,"d")</code> with 10-digit accuracy returns 1.333333333, while with 20-digit accuracy it returns 1.333333333333332593. The latter does not end in 3s, but it is an accurate decimal representation of the floating-point number nearest to 4/3. |
| "e" | When <code>sym</code> uses the <i>estimate error</i> mode, it supplements a result obtained in the rational mode by a term involving the variable <code>eps</code> . This term estimates the difference between the theoretical rational expression and its actual floating-point value. For example, <code>sym(3*pi/4,"e")</code> returns $(3\pi)/4 - (103\text{eps})/249$ .                                                                                                                                             |
| "f" | When <code>sym</code> uses the <i>floating-point to rational</i> mode, it returns the symbolic form for all values in the form $N \cdot 2^e$ or $-N \cdot 2^e$ , where $N \geq 0$ is a nonnegative integer and $e$ is an integer. The returned symbolic number is a precise rational number that is equal to the floating-point value. For example, <code>sym(1/10,"f")</code> returns 3602879701896397/36028797018963968.                                                                                                |

**strnum — String representing symbolic number**

string | character vector

String representing a symbolic number, specified as a string or character vector.

Example: '1/10'

**h — Anonymous function**

MATLAB function handle

Anonymous function, specified as a MATLAB function handle. For more information, see “Anonymous Functions”.

Example: `h = @(x)sin(x)`

**M — Symbolic matrix variable to convert**

symbolic matrix variable

Symbolic matrix variable to convert, specified as a symbolic matrix variable.

Alternatively, you can use `symmatrix2sym` to convert a symbolic matrix variable to an array of symbolic scalar variables.

Example: `syms A 2 matrix; M = A^2 + eye(2)`

Data Types: `symmatrix`

**Output Arguments****x — Variable**

symbolic scalar variable

Variable, returned as a symbolic scalar variable.

**A — Vector or matrix with automatically generated elements**

symbolic vector | symbolic matrix



Vector or matrix with automatically generated elements, returned as a symbolic vector or matrix of symbolic scalar variables. The elements of this vector or matrix do not appear in the MATLAB workspace.

### **symexpr — Expression or matrix converted from anonymous MATLAB function or symbolic matrix variable**

symbolic expression | symbolic matrix

Expression or matrix converted from an anonymous MATLAB function or a symbolic matrix variable, returned as a symbolic expression or matrix of symbolic scalar variables.

Data Types: sym

## **Tips**

- Statements like `pi = sym(pi)` and `delta = sym("1/10")` create symbolic numbers that avoid the floating-point approximations inherent in the values of `pi` and `1/10`. The `pi` created in this way stores the symbolic number in a workspace variable named `pi`, which temporarily replaces the built-in numeric function with the same name. Use `clear pi` to restore the floating-point representation of `pi`.
- `sym` always treats `i` in character vector input as an identifier. To input the imaginary number `i`, use `1i` instead.
- `clear x` does not clear the symbolic object of its assumptions, such as real, positive, or any assumptions set by `assume`, `sym`, or `syms`. To remove assumptions, use one of these options:
  - `assume(x,"clear")` removes all assumptions affecting `x`.
  - `clear all` clears all objects in the MATLAB workspace and resets the symbolic engine.
  - `assume` and `assumeAlso` provide more flexibility for setting assumptions on variables.
- When you replace one or more elements of a numeric vector or matrix with a symbolic number, MATLAB converts that number to a double-precision number.

```
A = eye(3);
A(1,1) = sym(pi)
```

```
A =
    3.1416         0         0
         0    1.0000         0
         0         0    1.0000
```

You cannot replace elements of a numeric vector or matrix with a symbolic variable, expression, or function because these elements cannot be converted to double-precision numbers. For example, `A(1,1) = sym("a")` throws an error.

- When you use the syntax `A = sym("a",[n1 ... nM])`, the `sym` function assigns only the symbolic array `A` to the MATLAB workspace. To also assign the automatically generated elements of `A`, use the `syms` function instead. For example, `syms a [1 3]` creates the row vector `a = [a1 a2 a3]` and the symbolic variables `a1`, `a2`, and `a3` in the MATLAB workspace.

## Alternative Functionality

### Alternative Approaches for Creating Symbolic Variables

To create several symbolic variables in one function call, use `syms`. Using `syms` also clears assumptions from the named variables.

## Version History

Introduced before R2006a

### R2022b: Convert symbolic matrix variables

You can convert a symbolic matrix variable `M` of type `symmatrix` to an array of symbolic scalar variables `symexpr` of type `sym` by using `symexpr = sym(M)`. For an example, see “Convert Hessian Matrix” on page 6-1462.

### R2020a: `sym("pi")` creates symbolic variable

*Behavior changed in R2020a*

`sym("pi")` now creates a symbolic variable named `pi` instead of a symbolic number representing the mathematical constant  $\pi$ . In previous releases, both `sym("pi")` and `sym(pi)` create symbolic numbers representing the constant  $\pi$ .

For example, the command `a = sym("pi")` creates a symbolic variable named `pi` and assigns it to the workspace variable `a`.

```
a = sym("pi")
class(a)
symType(a)
vpa(2*a)

a =
pi

ans =
'sym'

ans =
"variable"

ans =
2.0*pi
```

To create a symbolic number representing the constant  $\pi$ , use `a = sym(pi)` instead.

```
a = sym(pi)
class(a)
symType(a)
vpa(2*a)

a =
pi

ans =
```

```
'sym'

ans =
    "constant"

ans =
6.283185307179586476925286766559
```

This behavior also applies to the mathematical constants `catalan` and `eulergamma`.

### **R2018a: Support for character vectors has been removed**

*Errors starting in R2018a*

Support for character vectors that are not valid variable names and that do not define a number has been removed. To create symbolic expressions, first create symbolic variables, and then use operations on them. For example, use `syms x; x + 1` instead of `sym('x + 1')`, `exp(sym(pi))` instead of `sym('exp(pi)')`, and `syms f(var1,...,varN)` instead of `f(var1,...,varN) = sym('f(var1,...,varN)')`.

### **See Also**

`assume` | `double` | `str2sym` | `symfun` | `syms` | `symvar`

### **Topics**

“Create Symbolic Numbers, Variables, and Expressions” on page 1-3

“Create Symbolic Functions” on page 1-9

“Create Symbolic Matrices” on page 1-11

“Choose `syms` or `sym` Function” on page 2-4

“Use Assumptions on Symbolic Variables” on page 1-41

“Add Subscripts, Superscripts, and Accents to Symbolic Variables in the Live Editor” on page 2-14

“Change Output Format of Symbolic and Variable-Precision Arithmetic” on page 2-8

## sym2cell

Convert symbolic array to cell array

### Syntax

```
C = sym2cell(S)
```

### Description

`C = sym2cell(S)` converts a symbolic array `S` to a cell array `C`. The resulting cell array has the same size and dimensions as the input symbolic array.

### Examples

#### Convert Symbolic Array to Cell Array

Convert a matrix of symbolic variables and numbers to a cell array.

Create the following symbolic matrix.

```
syms x y  
S = [x 2 3 4; y 6 7 8; 9 10 11 12]
```

```
S =  

$$\begin{pmatrix} x & 2 & 3 & 4 \\ y & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{pmatrix}$$

```

Convert this matrix to a cell array by using `sym2cell`. The size of the resulting cell array corresponds to the size of the input matrix. Each cell contains an element of the symbolic matrix `S`.

```
C = sym2cell(S)
```

```
C=3x4 cell array  
 {[x]}    {[2]}    {[3]}    {[4]}  
 {[y]}    {[6]}    {[7]}    {[8]}  
 {[9]}    {[10]}   {[11]}   {[12]}
```

To access an element in each cell, use curly braces.

```
[C{1,1:4}]
```

```
ans = (x 2 3 4)
```

```
[C{1:3,1}]
```

```
ans = (x y 9)
```

## Input Arguments

### **S — Input symbolic array**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix | symbolic multidimensional array

Input symbolic array, specified as a symbolic vector, matrix, or multidimensional array. S also can be a scalar, that is, a symbolic number, variable, expression, or function.

## Output Arguments

### **C — Resulting cell array**

cell array

Resulting cell array, returned as a cell array such that `size(C) = size(S)`. Each element of the input symbolic array S is enclosed in a separate cell.

## Version History

**Introduced in R2016a**

### **See Also**

`cell2sym` | `cell2mat` | `mat2cell` | `num2cell`

## sym2poly

Extract vector of all numeric coefficients, including zeros, from symbolic polynomial

### Syntax

```
c = sym2poly(p)
```

### Description

`c = sym2poly(p)` returns the numeric vector of coefficients `c` of the symbolic polynomial `p`. The returned vector `c` includes all coefficients, including those equal 0.

`sym2poly` returns coefficients in order of descending powers of the polynomial variable. If  $c_1x^{n-1} + c_2x^{n-2} + \dots + c_n$ , then `c = sym2poly(p)` returns `c = [c1 c2 ... cn]`.

### Examples

#### Extract Numeric Coefficients of Polynomial

Create row vectors of coefficients of symbolic polynomials.

Extract integer coefficients of a symbolic polynomial into a numeric row vector.

```
syms x
c = sym2poly(x^3 - 2*x - 5)

c =
     1         0     -2     -5
```

Extract rational and integer coefficients of a symbolic polynomial into a vector. Because `sym2poly` returns numeric double-precision results, it approximates exact rational coefficients with double-precision numbers.

```
c = sym2poly(1/2*x^3 - 2/3*x - 5)

c =
    0.5000         0    -0.6667    -5.0000
```

### Input Arguments

#### **p** — Polynomial

symbolic expression

Polynomial, specified as a symbolic expression.

### Output Arguments

#### **c** — Polynomial coefficients

numeric row vector

Polynomial coefficients, returned as a numeric row vector.

## Tips

- To extract symbolic coefficients of a polynomial, use `coeffs`. This function returns a symbolic vector of coefficients and omits all zeros. For example, `syms a b x; c = coeffs(a*x^3 - 5*b,x)` returns `c = [ -5*b, a]`.

## Version History

Introduced before R2006a

## See Also

`coeffs` | `poly2sym`

## symfalse

Symbolic logical constant false

### Syntax

```
symfalse
F = symfalse(n)
F = symfalse(sz)
T = symfalse(sz1,...,szN)
```

### Description

`symfalse` is the symbolic logical constant for the false condition.

`F = symfalse(n)` returns an  $n$ -by- $n$  matrix of symbolic logical `symfalses`.

`F = symfalse(sz)` returns an array of symbolic logical `symfalses` where the size vector, `sz`, defines `size(T)`. For example, `symfalse([2 3])` returns a 2-by-3 array of symbolic logical `symfalses`.

`T = symfalse(sz1,...,szN)` returns a `sz1`-by-...-by-`szN` array of symbolic logical `symfalses` where `sz1,...,szN` indicates the size of each dimension. For example, `symfalse(2,3)` returns a 2-by-3 array of symbolic logical `symfalses`.

### Examples

#### Simplify Symbolic Inequality

Create a symbolic inequality  $x^2 > 4$ .

```
syms x
eq = x^2 > 4
```

```
eq = 4 < x^2
```

Assume that  $-2 < x < 2$ .

```
assume(-2<x & x<2)
```

Simplify the condition represented by the symbolic inequality `eq`. The `simplify` function returns the symbolic logical constant `symfalse` since the condition never holds for the assumption  $-2 < x < 2$ .

```
F = simplify(eq)
```

```
F = symfalse
```

Display the data type of `F`, which is `sym`.

```
class(F)
```



```
ans =
'sym'
```

You can also use `isAlways` to check if the inequality does not hold under the assumption being made. In this example, `isAlways` returns logical 0 (false).

```
TF = isAlways(eq)
```

```
TF = logical
    0
```

### Generate Array of Symbolic Logical symfalses

Use `symfalse` to generate a 3-by-3 square matrix of symbolic logical `symfalses`.

```
F = symfalse(3)
```

```
F =
(symfalse symfalse symfalse)
(symfalse symfalse symfalse)
(symfalse symfalse symfalse)
```

Display the data type of `T`, which is `sym`.

```
class(F)
```

```
ans =
'sym'
```

Next, use `symfalse` to generate a 3-by-2-by-2 array of symbolic logical `symfalses`.

```
F = symfalse(3,2,2)
```

```
F(:,:,1) =
(symfalse symfalse)
(symfalse symfalse)
(symfalse symfalse)
```

```
F(:,:,2) =
(symfalse symfalse)
(symfalse symfalse)
(symfalse symfalse)
```

Alternatively, you can use a size vector to specify the size of the array.

```
F = symfalse([3,2,2])
```

```
F(:,:,1) =
(symfalse symfalse)
(symfalse symfalse)
(symfalse symfalse)
```

```
F(:,:,2) =
```

```
(symfalse symfalse)
(symfalse symfalse)
(symfalse symfalse)
```

### Create Truth Table for Logical Operation

Create a truth table for the and operation applied to the two symbolic logical constants, `symtrue` and `symfalse`.

```
A = [symtrue symfalse]
```

```
A = (symtrue symfalse)
```

```
B = [symtrue; symfalse]
```

```
B =
    (symtrue)
    (symfalse)
```

```
TF = and(A,B)
```

```
TF =
    (symtrue symfalse)
    (symfalse symfalse)
```

### Combine Symbolic Logical Constants

Combine symbolic logical constants with logical operators `and`, `not`, `or`, and `xor` (or their shortcuts).

```
TF = xor(symtrue,or(symfalse,symfalse))
```

```
TF = symtrue
```

```
TF = symtrue & ~(symfalse)
```

```
TF = symtrue
```

### Convert `symfalse` to Other Data Types

Convert the symbolic logical constant `symfalse` to a logical value.

```
T1 = logical(symfalse)
```

```
T1 = logical
    0
```

Convert the symbolic logical constant `symfalse` to numeric values in double precision and variable precision.

```
T2 = double(symfalse)
```

```
T2 =
0
```

```
T3 = vpa(symfalse)
```

```
T3 = 0.0
```

Show the data types of T1, T2, and T3.

```
whos
```

| Name | Size | Bytes | Class   | Attributes |
|------|------|-------|---------|------------|
| T1   | 1x1  | 1     | logical |            |
| T2   | 1x1  | 8     | double  |            |
| T3   | 1x1  | 8     | sym     |            |

## Input Arguments

### **n** — Size of square matrix

scalar

Size of square matrix, specified as an integer. `n` sets the output array size to `n`-by-`n`. For example, `symfalse(3)` returns a 3-by-3 array of symbolic logical `symfalse`s.

- If `n` is 0, then `F` is an empty matrix.
- If `n` is negative, then it is treated as 0.

### **sz** — Size vector

row vector of integers

Size vector, specified as a row vector of integers. For example, `symfalse([2 3])` returns a 2-by-3 array of symbolic logical `symfalse`s.

- If the size of any dimension is 0, then `F` is an empty array.
- If the size of any dimension is negative, then it is treated as 0.
- If any trailing dimensions greater than 2 have a size of 1, then the output `F` does not include those dimensions. For example, `symfalse([2 2 1 1])` returns a 2-by-2 array and `symfalse([2 2 1 2 1])` returns a 2-by-2-by-1-by-2 array.

### **sz1, ..., szN** — Size inputs

comma-separated list of integers

Size inputs, specified by a comma-separated list of integers. For example, `symfalse(2,3)` returns a 2-by-3 array of symbolic logical `symfalse`s.

- If the size of any dimension is 0, then `F` is an empty array.
- If the size of any dimension is negative, then it is treated as 0.
- If any trailing dimensions greater than 2 have a size of 1, then the output `F` does not include those dimensions. For example, `symfalse([2,2,1,1])` returns a 2-by-2 array and `symfalse([2,2,1,2,1])` returns a 2-by-2-by-1-by-2 array.

## Output Arguments

### **F — Symbolic logical constant for false condition**

scalar | vector | matrix | *N*-D array

Symbolic logical constant for false condition, returned as a scalar, vector, matrix, or *N*-D array.

Data Types: sym

## Tips

- The command `sym(false)` returns a symbolic number 0, and `sym(symfalse)` returns `symfalse`.
- When you combine two arrays of symbolic logical constants with logical operations using `and`, `or`, or `xor` function, the arrays must either be the same size or have sizes that are compatible. For more information on the required input sizes for basic array operations, see “Compatible Array Sizes for Basic Operations”.

## Version History

**Introduced in R2020a**

## See Also

`isAlways` | `symtrue` | `and` | `or` | `not` | `xor`

# symfun

Create symbolic functions

## Syntax

```
f(inputs) = formula
f = symfun(formula,inputs)
f = symfun(fM)
```

## Description

`f(inputs) = formula` creates the symbolic function `f`. For example, `f(x,y) = x + y`. The symbolic variables in `inputs` are the input arguments. The symbolic expression `formula` is the body of the function `f`.

`f = symfun(formula,inputs)` is the formal way to create a symbolic function.

`f = symfun(fM)` converts a symbolic matrix function `fM` of type `symfunmatrix` to a symbolic function `f` of type `symfun`.

## Examples

### Create and Define Symbolic Functions

Define the symbolic function `f(x,y) = x + y`. First, create the function by using `syms`. Then define the function.

```
syms f(x,y)
f(x,y) = x + y
```

```
f(x, y) = x + y
```

Find the value of `f` at `x = 1` and `y = 2`.

```
f(1,2)
```

```
ans = 3
```

Define the function again by using the formal syntax.

```
syms x y
f = symfun(x+y,[x y])
```

```
f(x, y) = x + y
```

### Return Body and Arguments of Symbolic Function

Return the body of a symbolic function by using `formula`. You can use the body for operations such as indexing into the function. Return the arguments of a symbolic function by using `argnames`.

Index into the symbolic function  $[x^2, y^4]$ . Since a symbolic function is a scalar, you cannot directly index into the function. Instead, index into the body of the function.

```
syms f(x,y)
f(x,y) = [x^2, y^4];
fbody = formula(f);
fbody(1)
```

```
ans = x2
```

```
fbody(2)
```

```
ans = y4
```

Return the arguments of the function.

```
fvars = argnames(f)
```

```
fvars = (x y)
```

### Combine Two Symbolic Functions

Create two symbolic functions.

```
syms f(x) g(x)
f(x) = 2*x^2 - x;
g(x) = 3*x^2 + 2*x;
```

Combine the two symbolic functions into another symbolic function  $h(x)$  with the data type `symfun`.

```
h(x) = [f(x); g(x)]
```

```
h(x) =

$$\begin{pmatrix} 2x^2 - x \\ 3x^2 + 2x \end{pmatrix}$$

```

Evaluate the function  $h(x)$  at  $x = 1$  and  $x = 2$ .

```
h(1)
```

```
ans =

$$\begin{pmatrix} 1 \\ 5 \end{pmatrix}$$

```

```
h(2)
```

```
ans =

$$\begin{pmatrix} 6 \\ 16 \end{pmatrix}$$

```

You can also combine the two functions into an array of symbolic expressions with the data type `sym`.

```
h_expr = [f(x); g(x)]
```

```
h_expr =
```

$$\begin{pmatrix} 2x^2 - x \\ 3x^2 + 2x \end{pmatrix}$$

Index into `h_expr` to access the first and the second symbolic expressions.

```
h_expr(1)
```

```
ans = 2x^2 - x
```

```
h_expr(2)
```

```
ans = 3x^2 + 2x
```

### Convert Symbolic Matrix Function to Symbolic Function

Create 2-by-1 and 2-by-2 symbolic matrix variables to represent the matrices  $\mathbf{X}$  and  $\mathbf{A}$ .

```
syms X [2 1] matrix
syms A [2 2] matrix
```

Create two symbolic matrix functions to represent the functions  $F(\mathbf{X}, \mathbf{A})$  and  $\partial F(\mathbf{X}, \mathbf{A})/\partial \mathbf{X}^T$ . When creating the symbolic matrix functions, keep existing definitions of the symbolic matrix variables  $\mathbf{X}$  and  $\mathbf{A}$  in the workspace. The symbolic matrix functions require matrices of the same sizes as  $\mathbf{X}$  and  $\mathbf{A}$  as their input arguments.

```
syms F(X,A) [1 1] matrix keepargs
syms dF(X,A) [2 1] matrix keepargs
```

Define the function  $F(\mathbf{X}, \mathbf{A}) = \mathbf{X}^T \mathbf{A} \mathbf{X}$  and find its derivative  $\partial F(\mathbf{X}, \mathbf{A})/\partial \mathbf{X}^T$ . The resulting symbolic matrix functions are in matrix notation in terms of  $\mathbf{X}$  and  $\mathbf{A}$ .

```
F(X,A) = X.'*A*X
```

```
F(X, A) = X^T A X
```

```
dF(X,A) = diff(F,X.')
```

```
dF(X, A) = A X + A^T X
```

Convert the symbolic matrix functions from data type `symfunmatrix` to `symfun`. The resulting symbolic functions are in scalar notation in terms of the matrix elements of  $\mathbf{X}$  and  $\mathbf{A}$ . These functions accept scalars as their input arguments.

```
Fsymfun = symfun(F)
```

```
Fsymfun(X1, X2, A1_1, A1_2, A2_1, A2_2) = X1 (A1,1 X1 + A1,2 X2) + X2 (A2,1 X1 + A2,2 X2)
```

```
dFsymfun = symfun(dF)
```

```
dFsymfun(X1, X2, A1_1, A1_2, A2_1, A2_2) =
    ( 2 A1,1 X1 + A1,2 X2 + A2,1 X2 )
    ( A1,2 X1 + A2,1 X1 + 2 A2,2 X2 )
```

## Input Arguments

### **formula — Function body**

symbolic expression | vector of symbolic expressions | matrix of symbolic expressions

Function body, specified as a symbolic expression, vector of symbolic expressions, or matrix of symbolic expressions that can be converted to the `sym` data type.

Example:  $x + y$

### **inputs — Input argument or arguments of function**

symbolic variable | array of symbolic variables

Input argument or arguments of a function, specified as a symbolic variable or array of symbolic variables, respectively.

Example:  $[x, y]$

Data Types: `sym`

### **fM — Symbolic matrix function to convert**

symbolic matrix function

Symbolic matrix function to convert, specified as a symbolic matrix function.

Alternatively, you can use `symfunmatrix2symfun` to convert a symbolic matrix function to a symbolic function.

Data Types: `symfunmatrix`

## Output Arguments

### **f — Symbolic function**

`symfun` object

Symbolic function, returned as a `symfun` object.

While the data type of the function `f` is `symfun`, the data type of the evaluated function, such as `f(1,2)`, is `sym`.

## Version History

**Introduced in R2012a**

### **R2024b: Convert symbolic matrix function**

You can convert a symbolic matrix function of type `symfunmatrix` to a symbolic function of type `symfun` by using `symfun`. For an example, see “Convert Symbolic Matrix Function to Symbolic Function” on page 6-1479.

## See Also

`argnames` | `formula` | `matlabFunction` | `sym` | `syms` | `symvar`



**Topics**

"Create Symbolic Functions" on page 1-9

## symFunType

Determine functional type of symbolic object

### Syntax

```
s = symFunType(symObj)
```

### Description

`s = symFunType(symObj)` returns the functional type of a symbolic object.

- If `symObj` is a symbolic function or a symbolic expression, then `symFunType` returns the topmost function name or operator of `symObj`. For example, `syms x; symFunType(2*sin(x))` returns "times".
- If `symObj` is not a symbolic function or a symbolic expression, then `symFunType` returns the same output as `symType`. For example, `symFunType(sym('2'))` returns "integer".

### Examples

#### Symbolic Function or Expression

Create an array of symbolic functions and expressions.

```
syms f(x)
expr = [f(x) sin(x) exp(x) int(f(x)) diff(f(x))]
```

```
expr =
    (f(x) sin(x) e^x ∫ f(x) dx ∂/∂x f(x))
```

Determine the functional type of each array element.

```
s = symFunType(expr)

s = 1x5 string
    "f"      "sin"      "exp"      "int"      "diff"
```

#### Topmost Arithmetic Operators of Symbolic Expressions

Create two symbolic expressions. Determine the topmost arithmetic operators of the expressions.

```
syms x
expr1 = x/(x^2+x+2);
expr2 = x + 1/(x^2+x+2);
s1 = symFunType(expr1)
```

```
s1 =
    "times"
```

```
s2 = symFunType(expr2)
```

```
s2 =  
"plus"
```

To return the terms separated by the operators, use `children`.

```
terms1 = children(expr1)
```

```
terms1=1×2 cell array  
{[x]} { [1/(x^2 + x + 2)] }
```

```
terms2 = children(expr2)
```

```
terms2=1×2 cell array  
{[x]} { [1/(x^2 + x + 2)] }
```

## Comparison Operators in Equations and Inequalities

Create an array of symbolic equations and inequalities.

```
syms x y  
eqns = [x+y==2, x<=5, y>3]
```

```
eqns = (x + y = 2 x ≤ 5 3 < y)
```

Determine the topmost comparison operator in each array element.

```
s = symFunType(eqns)
```

```
s = 1×3 string  
"eq" "le" "lt"
```

## Input Arguments

### symObj — Symbolic objects

symbolic expressions | symbolic functions | symbolic variables | symbolic numbers | symbolic units

Symbolic objects, specified as symbolic expressions, symbolic functions, symbolic variables, symbolic numbers, or symbolic units.

## Output Arguments

### s — Symbolic functional types

string array

Symbolic functional types, returned as a string array. If `symObj` is a symbolic function or a symbolic expression, then `symFunType` returns the topmost function name or operator of `symObj`. This table shows output values for various symbolic objects.

| Symbolic Functional Types       | Returned Output                                                                                                                                                                                                                                                                                              | Input Example                                                                                                                                                                                                                                                                                                                 |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| symbolic math functions         | "sin", "exp", "fourier", and so on — name of the topmost symbolic math function in a symbolic expression                                                                                                                                                                                                     | <code>syms f(x);<br/>symFunType([sin(x),<br/>exp(x), fourier(f(x))])</code>                                                                                                                                                                                                                                                   |
| unassigned symbolic functions   | "f", "g", and so on — unassigned symbolic function                                                                                                                                                                                                                                                           | <code>syms f(x) g(x);<br/>symFunType([f, g(x+2)])</code>                                                                                                                                                                                                                                                                      |
| arithmetic operators            | <ul style="list-style-type: none"> <li>• "plus" — addition operator + and subtraction operator -</li> <li>• "times" — multiplication operator * and division operator /</li> <li>• "power" — power or exponentiation operator ^ and square root operator sqrt</li> </ul>                                     | <ul style="list-style-type: none"> <li>• <code>syms x;<br/>symFunType(x^2-x)</code></li> <li>• <code>syms x;<br/>symFunType(2*x^2)</code></li> <li>• <code>syms x;<br/>symFunType([x^2<br/>sqrt(x)])</code></li> </ul>                                                                                                        |
| equations and inequalities      | <ul style="list-style-type: none"> <li>• "eq" — equality operator ==</li> <li>• "ne" — inequality operator ~=</li> <li>• "lt" — less-than operator &lt; or greater-than operator &gt;</li> <li>• "le" — less-than-or-equal-to operator &lt;= or greater-than-or-equal-to operator &gt;=</li> </ul>           | <ul style="list-style-type: none"> <li>• <code>syms x y;<br/>symFunType(x==y)</code></li> <li>• <code>syms x y;<br/>symFunType(x~=y)</code></li> <li>• <code>syms x y;<br/>symFunType(x&lt;y)</code></li> <li>• <code>syms x y;<br/>symFunType(x&gt;=y)</code></li> </ul>                                                     |
| logical operators and constants | <ul style="list-style-type: none"> <li>• "or" — logical OR operator  </li> <li>• "and" — logical AND operator &amp;</li> <li>• "not" — logical NOT operator ~</li> <li>• "xor" — logical exclusive-OR operator xor</li> <li>• "logicalconstant" — symbolic logical constants symtrue and symfalse</li> </ul> | <ul style="list-style-type: none"> <li>• <code>syms x y;<br/>symFunType(x y)</code></li> <li>• <code>syms x y;<br/>symFunType(x&amp;y)</code></li> <li>• <code>syms x; symFunType(~x)</code></li> <li>• <code>syms x y;<br/>symFunType(xor(x,y))</code></li> <li>• <code>symFunType([symtrue<br/>symfalse])</code></li> </ul> |
| numbers                         | <ul style="list-style-type: none"> <li>• "integer" — integer number</li> <li>• "rational" — rational number</li> <li>• "vpereal" — variable-precision floating-point real number</li> <li>• "complex" — complex number</li> </ul>                                                                            | <ul style="list-style-type: none"> <li>• <code>symFunType(sym('-1'))</code></li> <li>• <code>symFunType(sym('1/2'))</code></li> <li>• <code>symFunType([sym('1.5')<br/>vpa('3/2')])</code></li> <li>• <code>symFunType(sym('1+2i'))</code></li> </ul>                                                                         |
| constants                       | "constant" — symbolic mathematical constant                                                                                                                                                                                                                                                                  | <code>symFunType(sym([pi<br/>catalan]))</code>                                                                                                                                                                                                                                                                                |
| variables                       | "variable"                                                                                                                                                                                                                                                                                                   | <code>symFunType(sym(x))</code>                                                                                                                                                                                                                                                                                               |
| units                           | "unit"                                                                                                                                                                                                                                                                                                       | <code>symFunType(symunit('m'))</code>                                                                                                                                                                                                                                                                                         |

| Symbolic<br>Functional<br>Types | Returned Output | Input Example |
|---------------------------------|-----------------|---------------|
| unsupported<br>symbolic types   | "unsupported"   |               |

## Version History

Introduced in R2019a

## See Also

sym | syms | symfun | hasSymType | isSymType | symType

## symfunmatrix

Create symbolic matrix function

### Syntax

```
f = symfunmatrix(formula,inputs)
f = symfunmatrix(formula,inputs,[nrow ncol])
```

### Description

`f = symfunmatrix(formula,inputs)` creates the symbolic matrix function `f`. The symbolic expression `formula` is the body of the function `f` that can be converted to the `symmatrix` data type. The variables in `inputs` are the input arguments of the function `f`.

- If the input arguments of function `f` are multiple variables, `inputs` must be a cell array of symbolic scalar and matrix variables.
- If the input argument of function `f` is a single variable, you can specify `inputs` as a symbolic scalar variable or a symbolic matrix variable.

`f = symfunmatrix(formula,inputs,[nrow ncol])` also explicitly specifies the size of the evaluated symbolic matrix function `f(var1,var2,...)` as `nrow-by-ncol` for the `inputs = {var1,var2,...}`.

- If `formula` represents an unassigned abstract function, then `f(var1,var2,...)` has the size of `nrow-by-ncol`. For example:

```
syms x 2 matrix;
f = symfunmatrix('g(x)',{x},[3 4]);
size(f(x))
```

```
ans =
     3     4
```

- If `formula` represents a symbolic expression or a function with definition, then the size of `f(var1,var2,...)` follows the size of `formula`. For example:

```
syms X Y 2 matrix;
f = symfunmatrix(X*Y - Y*X,{X,Y},[3 3]);
size(f(X,Y))
```

```
ans =
     2     2
```

### Examples

#### Define and Evaluate Symbolic Matrix Functions

Create two symbolic matrix variables with size 2-by-2.

```
syms X Y [2 2] matrix
```

Create a symbolic matrix function that represents the matrix operation  $\mathbf{XY} - \mathbf{YX}$  by using `symfunmatrix`.

```
f = symfunmatrix(X*Y - Y*X,{X,Y})
```

```
f(X, Y) = XY - YX
```

Evaluate the function for the matrix values  $\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 2 & 2 \end{bmatrix}$  and  $\mathbf{Y} = \begin{bmatrix} -2 & 1 \\ 0 & 4 \end{bmatrix}$ . The evaluated function is a symbolic matrix variable of data type `symmatrix`.

```
fEval = f([1 2; 2 2],[-2 1; 0 4])
```

```
fEval =  
-Σ1 + Σ2
```

where

$$\Sigma_1 = \begin{pmatrix} 0 & -2 \\ 8 & 8 \end{pmatrix}$$

$$\Sigma_2 = \begin{pmatrix} -2 & 9 \\ -4 & 10 \end{pmatrix}$$

```
class(fEval)
```

```
ans =  
'symmatrix'
```

Convert the evaluated function to the `sym` data type.

```
fSym = symmatrix2sym(fEval)
```

```
fSym =  
 $\begin{pmatrix} -2 & 11 \\ -12 & 2 \end{pmatrix}$ 
```

## Input Arguments

### formula — Function body

symbolic expression | abstract function

Function body, specified as a symbolic expression that can be converted to the `symmatrix` data type or an abstract function.

Example: `X*Y.`

### inputs — Input argument or arguments of function

cell array of symbolic scalar and matrix variables | symbolic scalar variable | symbolic matrix variable

Input argument or arguments of a function, specified as a cell array of symbolic scalar and matrix variables, a symbolic scalar variable, or a symbolic matrix variable.

Example: `{X,Y}, symmatrix('t',[2 3])`

Data Types: `cell` | `sym` | `symmatrix`

**[nrow ncol] — Dimensions of evaluated symbolic matrix function**

vector of nonnegative integers

Dimensions of the evaluated symbolic matrix function, specified as a vector of nonnegative integers. As a shortcut, you can create a square symbolic matrix function by specifying only one integer.

Example: [2 3]

**Output Arguments****f — Symbolic matrix function**

symfunmatrix object

Symbolic matrix function, returned as a symfunmatrix object.

While the data type of the function `f` is `symfunmatrix`, the data type of the evaluated function, such as `f([-2 3],[1 0])`, is `symmatrix`.

**Limitations**

- To show all the functions in Symbolic Math Toolbox that accept symbolic matrix functions as input, use the command `methods symfunmatrix`.

**Tips**

- When evaluating a symbolic matrix function, you must substitute values that have the same size as the defined input arguments. For example, see “Define and Evaluate Symbolic Matrix Functions” on page 6-1486. For comparison, this example returns an error:

```
syms X [2 2] matrix
syms f(X) [1 1] matrix keepargs
f(ones(4))
```

**Version History****Introduced in R2022a****R2023a: Specify inputs as a symbolic scalar variable or a symbolic matrix variable**

When creating a symbolic matrix function with a single variable as its input argument, you can specify the `inputs` argument as a symbolic scalar variable or a symbolic matrix variable. For example, you can create a symbolic matrix function using:

```
syms X [4 5] matrix
f = symfunmatrix("f(X)",X,[1 1])
```

In previous releases, the `inputs` argument must be a cell array of symbolic scalar and matrix variables, even if function `f` has only one input argument.

**See Also**

argnames | formula | syms | symfun | symmatrix | symfunmatrix2symfun



**Topics**

"Create Symbolic Numbers, Variables, and Expressions" on page 1-3

"Create Symbolic Functions" on page 1-9

"Create Symbolic Matrices" on page 1-11

"Create Symbolic Matrix Variables" on page 1-13

"Use Symbolic Objects to Represent Mathematical Objects" on page 1-20

## symfunmatrix2symfun

Convert symbolic matrix function to symbolic function

### Syntax

```
f = symfunmatrix2symfun(fM)
```

### Description

`f = symfunmatrix2symfun(fM)` converts a symbolic matrix function `fM` of type `symfunmatrix` to a symbolic function `f` of type `symfun`.

The output symbolic function has the same matrix dimensions as the input symbolic matrix function, and its components are filled with automatically generated elements. For example, `syms fM(x) [1 2] matrix; f = symfunmatrix2symfun(fM)` converts the symbolic matrix function `fM(x)` to the symbolic function `f(x) = [fM1_1(x), fM1_2(x)]`. The generated elements `fM1_1` and `fM1_2` do not appear in the MATLAB workspace.

### Examples

#### Differentiate Function with Respect to Matrix Argument

Create 2-by-1 and 2-by-2 symbolic matrix variables to represent the matrices **X** and **A**.

```
syms X [2 1] matrix
syms A [2 2] matrix
```

Create two symbolic matrix functions to represent the functions  $F(\mathbf{X}, \mathbf{A})$  and  $\partial F(\mathbf{X}, \mathbf{A})/\partial \mathbf{X}^T$ . When creating the symbolic matrix functions, keep existing definitions of the symbolic matrix variables **X** and **A** in the workspace. The symbolic matrix functions require matrices of the same sizes as **X** and **A** as their input arguments.

```
syms F(X,A) [1 1] matrix keepargs
syms dF(X,A) [2 1] matrix keepargs
```

Define the function  $F(\mathbf{X}, \mathbf{A}) = \mathbf{X}^T \mathbf{A} \mathbf{X}$  and find its derivative  $\partial F(\mathbf{X}, \mathbf{A})/\partial \mathbf{X}^T$ . The resulting symbolic functions are in matrix notation in terms of **X** and **A**.

```
F(X,A) = X.'*A*X
```

```
F(X, A) = XTA X
```

```
dF(X,A) = diff(F,X.')
```

```
dF(X, A) = A X + ATX
```

Convert the symbolic matrix functions from data type `symfunmatrix` to `symfun`. The resulting symbolic functions are in scalar notation in terms of the matrix elements of **X** and **A**. These functions accept scalars as their input arguments.

```
Fsymfun = symfunmatrix2symfun(F)
```

```
Fsymfun(X1, X2, A1_1, A1_2, A2_1, A2_2) = X1 (A1,1 X1 + A1,2 X2) + X2 (A2,1 X1 + A2,2 X2)
```

```
dFsymfun = symfunmatrix2symfun(dF)
```

```
dFsymfun(X1, X2, A1_1, A1_2, A2_1, A2_2) =
```

$$\begin{pmatrix} 2 A_{1,1} X_1 + A_{1,2} X_2 + A_{2,1} X_2 \\ A_{1,2} X_1 + A_{2,1} X_1 + 2 A_{2,2} X_2 \end{pmatrix}$$

## Input Arguments

### fM — Input

symbolic matrix function

Input, specified as a symbolic matrix function.

Data Types: `symfunmatrix`

## Tips

- To show all the functions in Symbolic Math Toolbox that accept symbolic matrix functions as input, use the command `methods symfunmatrix`.
- Instead of using `symfunmatrix2symfun` to convert a symbolic matrix function to a symbolic function, you can use the shortcut `symfun`. For an example, see “Convert Symbolic Matrix Function to Symbolic Function” on page 6-1479.

## Version History

Introduced in R2022a

## See Also

`syms` | `symfun` | `symfunmatrix`

## Topics

“Create Symbolic Numbers, Variables, and Expressions” on page 1-3

“Create Symbolic Functions” on page 1-9

“Create Symbolic Matrices” on page 1-11

“Create Symbolic Matrix Variables” on page 1-13

“Use Symbolic Objects to Represent Mathematical Objects” on page 1-20

## **symmatrix**

Create symbolic matrix variable

### **Syntax**

```
X = symmatrix('X',[nrow ncol])
X = symmatrix('X',n)
X = symmatrix('X')

X = symmatrix(S)
```

### **Description**

`X = symmatrix('X',[nrow ncol])` creates an `nrow`-by-`ncol` symbolic matrix variable `X`. Symbolic matrix variables represent matrices, vectors, and scalars in compact matrix notation. For more information, see “Create Symbolic Matrix Variables” on page 1-13.

`X = symmatrix('X',n)` creates an `n`-by-`n` symbolic matrix variable `X`.

`X = symmatrix('X')` creates a 1-by-1 symbolic matrix variable `X`.

`X = symmatrix(S)` converts a numeric matrix or a matrix of symbolic scalar variables specified by `S` to a symbolic matrix variable `X`.

### **Examples**

#### **Add Two Matrices**

Create two symbolic matrix variables with size 2-by-3. Nonscalar symbolic matrix variables are displayed as bold characters in the Live Editor and Command Window.

```
A = symmatrix('A',[2 3])
```

```
A = A
```

```
B = symmatrix('B',[2 3])
```

```
B = B
```

Add the two matrices. The summation of the two symbolic matrix variables is denoted by the matrix notation  $A+B$ .

```
X = A + B
```

```
X = A + B
```

#### **Commutation Relation of Symbolic Matrix Variables**

Symbolic matrix variables represent matrices, vectors, and scalars in compact matrix notation. When representing nonscalars, these variables are noncommutative. When mathematical formulas involve

matrices and vectors, writing them using symbolic matrix variables is more concise and clear than writing them componentwise.

Create two symbolic matrix variables.

```
A = symmatrix('A',[2 2]);
B = symmatrix('B',[2 2]);
```

Check the commutation relation for multiplication between two symbolic matrix variables.

```
A*B - B*A
ans =  $AB - BA$ 
isequal(A*B,B*A)
ans = logical
      0
```

Check the commutation relation for addition between two symbolic matrix variables.

```
isequal(A+B,B+A)
ans = logical
      1
```

### Evaluate Hessian Matrix

Create 3-by-3 and 3-by-1 symbolic matrix variables.

```
A = symmatrix('A',3)
A = A
X = symmatrix('X',[3 1])
X = X
```

Find the Hessian matrix of  $X^TAX$ . Derived equations involving symbolic matrix variables are displayed in typeset as they would be in textbooks.

```
f = X.'*A*X;
H = diff(f,X,X.')
H =  $A^T + A$ 
```

### Convert Numeric Matrix to Symbolic Matrix Variable

Create a Hilbert matrix of order 4. The data type of the matrix is double.

```
H = hilb(4)
```

```
H = 4×4
```

```
    1.0000    0.5000    0.3333    0.2500
    0.5000    0.3333    0.2500    0.2000
    0.3333    0.2500    0.2000    0.1667
    0.2500    0.2000    0.1667    0.1429
```

```
class(H)
```

```
ans =
'double'
```

Convert the numeric matrix to a symbolic matrix variable. The data type of the converted matrix is **symmatrix**.

```
X = symmatrix(H)
```

```
X =
```

 $\Sigma_1$ 

where

$$\Sigma_1 = \begin{pmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} \\ \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} \end{pmatrix}$$

```
class(X)
```

```
ans =
'symmatrix'
```

### Convert Symbolic Matrix Variable to Matrix of Symbolic Scalar Variables

Create two symbolic matrix variables with size 2-by-2.

```
A = symmatrix('A',2)
```

```
A = A
```

```
B = symmatrix('B',2)
```

```
B = B
```

Perform matrix multiplication between A and B. The multiplication of the two symbolic matrix variables is represented by the matrix notation  $AB$ .

```
X = A*B
```

```
X = AB
```

Convert the symbolic matrix variable  $X$  to a matrix of symbolic scalar variables  $S$ . The multiplication of two matrices of symbolic scalar variables is represented by the elements of the matrix product.

$S = \text{symmatrix2sym}(X)$

$$S = \begin{pmatrix} A_{1,1} B_{1,1} + A_{1,2} B_{2,1} & A_{1,1} B_{1,2} + A_{1,2} B_{2,2} \\ A_{2,1} B_{1,1} + A_{2,2} B_{2,1} & A_{2,1} B_{1,2} + A_{2,2} B_{2,2} \end{pmatrix}$$

## Input Arguments

### **X — Variable name**

character vector

Variable name, specified as a character vector. Argument  $X$  must be a valid variable name. That is,  $X$  must begin with a letter and can contain only alphanumeric characters and underscores. To verify that the name is a valid variable name, use `isvarname`.

Example: `x`, `y12`, `z_1`

### **[nrow ncol] — Vector or matrix dimensions**

vector of integers

Vector or matrix dimensions, specified as a vector of integers. `nrow` is the number of rows, and `ncol` is the number of columns. As a shortcut, you can create a square symbolic matrix variable by specifying only one integer. For example, `X = symmatrix('X',3)` creates a square 3-by-3 symbolic matrix variable.

Example: `[2 3]`, `[2,3]`

### **S — Numeric matrix or matrix of symbolic scalar variables to be converted**

number | numeric matrix | symbolic scalar variable | matrix of symbolic scalar variables

Numeric matrix or matrix of symbolic scalar variables to be converted to symbolic matrix variable, specified as a number, numeric matrix, symbolic scalar variable, or a matrix of symbolic scalar variables.

Example: `10`, `eye(3)`, `pi`, `hilb(3)`

## Limitations

- Differentiation functions, such as `jacobian` and `laplacian`, currently do not accept symbolic matrix variables as input. To evaluate differentiation with respect to vectors and matrices, you can use the `diff` function instead.
- To show all the functions in Symbolic Math Toolbox that accept symbolic matrix variables as input, use the command `methods symmatrix`.

## Alternative Functionality

### **Alternative Approaches for Creating Symbolic Matrix Variables**

To create several symbolic matrix variables in one function call, use `syms var1 ... varN [nrow ncol] matrix`. For more details, see `syms`.

## **Version History**

**Introduced in R2021a**

### **See Also**

`syms` | `sym` | `symfun` | `symfunmatrix` | `str2sym` | `symvar` | `symmatrix2sym`

### **Topics**

“Create Symbolic Numbers, Variables, and Expressions” on page 1-3

“Create Symbolic Functions” on page 1-9

“Create Symbolic Matrices” on page 1-11

“Create Symbolic Matrix Variables” on page 1-13

“Use Symbolic Objects to Represent Mathematical Objects” on page 1-20



# symmatrix2sym

Convert symbolic matrix variable to array of scalar variables

## Syntax

```
S = symmatrix2sym(M)
```

## Description

`S = symmatrix2sym(M)` converts a symbolic matrix variable `M` of type `symmatrix` to an array of symbolic scalar variables `S` of type `sym`.

The output array is the same size as the input symbolic matrix variable and its components are filled with automatically generated elements. For example, `syms M [1 3] matrix; S = symmatrix2sym(M)` creates the matrix `S = [M1_1, M1_2, M1_3]`. The generated elements `M1_1`, `M1_2`, and `M1_3` do not appear in the MATLAB workspace.

## Examples

### Convert Addition of Two Matrices

Create two symbolic matrix variables with size 2-by-3. Nonscalar symbolic matrix variables are displayed as bold characters in the Live Editor and Command Window.

```
syms A B [2 3] matrix
A
```

```
A = A
```

```
B
```

```
B = B
```

Add the two matrices. The result is represented by the matrix notation  $A+B$ .

```
X = A + B
```

```
X = A + B
```

The data type of `X` is `symmatrix`.

```
class(X)
```

```
ans =
'symmatrix'
```

Convert the symbolic matrix variable `X` to a matrix of symbolic scalar variables `Y`. The result is denoted by the sum of the matrix components.

```
Y = symmatrix2sym(X)
```

```
Y =
```

$$\begin{pmatrix} A_{1,1} + B_{1,1} & A_{1,2} + B_{1,2} & A_{1,3} + B_{1,3} \\ A_{2,1} + B_{2,1} & A_{2,2} + B_{2,2} & A_{2,3} + B_{2,3} \end{pmatrix}$$

The data type of Y is `sym`.

```
class(Y)
```

```
ans =  
'sym'
```

Show that the converted result in Y is equal to the sum of two matrices of symbolic scalar variables.

```
syms A B [2 3]  
Y2 = A + B
```

```
Y2 =  
      ( A1,1 + B1,1  A1,2 + B1,2  A1,3 + B1,3 )  
      ( A2,1 + B2,1  A2,2 + B2,2  A2,3 + B2,3 )
```

```
isequal(Y,Y2)
```

```
ans = logical  
      1
```

### Convert Hessian Matrix

Create 3-by-3 and 3-by-1 symbolic matrix variables.

```
syms A [3 3] matrix  
syms X [3 1] matrix
```

Find the Hessian matrix of  $X^T A X$ .

```
f = X.'*A*X;  
H = diff(f,X,X.')
```

```
H = AT + A
```

Convert the result from a symbolic matrix variable H to a matrix of symbolic scalar variables S.

```
S = symmatrix2sym(H)
```

```
S =  
      ( 2 A1,1  A1,2 + A2,1  A1,3 + A3,1 )  
      ( A1,2 + A2,1  2 A2,2  A2,3 + A3,2 )  
      ( A1,3 + A3,1  A2,3 + A3,2  2 A3,3 )
```

### Find Two-Norm of Vector

Create a 1-by-3 symbolic matrix variable that represents a vector.

```
syms A [1 3] matrix
```

Find the 2-norm of the vector A. The result is a symbolic matrix variable with `symmatrix` data type.

```
N = norm(A)
```

```
N = ||A||2
```

```
class(N)
```

```
ans =  
'symmatrix'
```

Convert `N` to a symbolic scalar variable to express the 2-norm in terms of the components of A. The result is a symbolic scalar variable with `sym` data type.

```
N = symmatrix2sym(N)
```

```
N =  $\sqrt{|A_{1,1}|^2 + |A_{1,2}|^2 + |A_{1,3}|^2}$ 
```

```
class(N)
```

```
ans =  
'sym'
```

### Find Dot Product of Two Vectors

Create two vectors of size 3-by-1 as symbolic matrix variables.

```
syms A B [3 1] matrix
```

Find the dot product of the two vectors by evaluating `transpose(A)*B`.

```
C = transpose(A)*B
```

```
C =  $A^T B$ 
```

Convert `C` to a symbolic scalar variable to express the dot product in terms of the components of A and B.

```
C = symmatrix2sym(C)
```

```
C =  $A_1 B_1 + A_2 B_2 + A_3 B_3$ 
```

### Concatenate Two Matrices

Create two 2-by-3 symbolic matrix variables.

```
syms A B [2 3] matrix
```

Concatenate the two matrices vertically using the command `vertcat(A,B)` or `[A; B]`.

```
C = [A; B]
```

$$C = \begin{pmatrix} A \\ B \end{pmatrix}$$

Convert C to a matrix of symbolic scalar variables.

```
C = symmatrix2sym(C)
```

$$C = \begin{pmatrix} A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,1} & A_{2,2} & A_{2,3} \\ B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,1} & B_{2,2} & B_{2,3} \end{pmatrix}$$

## Input Arguments

### M — Input

symbolic matrix variable

Input, specified as a symbolic matrix variable.

Data Types: `symmatrix`

## Tips

- To show all the functions in Symbolic Math Toolbox that accept symbolic matrix variables as input, use the command `methods symmatrix`.
- Instead of using `symmatrix2sym` to convert a symbolic matrix variable to an array of symbolic scalar variables, you can use the shortcut `sym`. For an example, see “Convert Hessian Matrix” on page 6-1462.

## Version History

Introduced in R2021a

## See Also

`sym` | `syms` | `symfun` | `str2sym` | `symmatrix` | `symfunmatrix2symfun`

### Topics

“Create Symbolic Numbers, Variables, and Expressions” on page 1-3

“Create Symbolic Functions” on page 1-9

“Create Symbolic Matrices” on page 1-11

“Create Symbolic Matrix Variables” on page 1-13

“Use Symbolic Objects to Represent Mathematical Objects” on page 1-20

# sympref

Set symbolic settings

## Syntax

```
oldVal = sympref(pref,value)
oldVal = sympref(pref)

oldPrefs = sympref(prefs)
oldPrefs = sympref()
```

## Description

`oldVal = sympref(pref,value)` sets the symbolic setting `pref` to `value` and returns the previous value of the setting to `oldVal`. You can set the setting to its default value using `sympref(pref,'default')`.

Symbolic settings can affect the computation of the symbolic functions `fourier`, `ifourier`, and `heaviside`, and the display format of symbolic output.

`oldVal = sympref(pref)` returns the current value of `pref`.

`oldPrefs = sympref(prefs)` sets multiple symbolic settings to the values in the structure `prefs` and returns the previous values of all settings to `oldPrefs`. You can set all symbolic settings to their default values using `sympref('default')`.

`oldPrefs = sympref()` returns the current values of all symbolic settings.

---

**Note** Symbolic settings persist through successive MATLAB sessions. Opening a new session does not restore the default settings.

---

## Examples

### Change Parameter Values of Fourier Transform

The Fourier transform  $F(w)$  of  $f = f(t)$  is

$$F(w) = c \int_{-\infty}^{\infty} f(t) e^{i s w t} dt,$$

where  $c$  and  $s$  are parameters with the default values 1 and -1, respectively. Other common values for  $c$  are  $1/(2\pi)$  and  $1/\sqrt{2\pi}$ , and other common values for  $s$  are 1,  $-2\pi$ , and  $2\pi$ .

Find the Fourier transform of  $\sin(t)$  with default  $c$  and  $s$  parameters.

```
syms t w
F = fourier(sin(t),t,w)
F = -pi (delta(w - 1) - delta(w + 1)) i
```

Find the same Fourier transform with  $c = 1/(2\pi)$  and  $s = 1$ . Set these parameter values by changing the 'FourierParameters' setting. Represent  $\pi$  exactly by using `sym`. Specify the values of  $c$  and  $s$  as the vector `[1/(2*sym(pi)) 1]`. Store the previous values returned by `sympref` so that you can restore them later.

```
oldVal = sympref('FourierParameters',[1/(2*sym(pi)) 1])
```

```
oldVal = (1 -1)
```

```
F = fourier(sin(t),t,w)
```

```
F =
```

$$\frac{\delta(w-1) i}{2} - \frac{\delta(w+1) i}{2}$$

The settings that you set using `sympref` persist through your current and future MATLAB® sessions. Restore the previous values of  $c$  and  $s$  to `oldVal`.

```
sympref('FourierParameters',oldVal);
```

Alternatively, you can restore the default values of  $c$  and  $s$  by specifying the 'default' option.

```
sympref('FourierParameters','default');
```

### Change Value of Heaviside Function at Origin

In Symbolic Math Toolbox™, the default value of the Heaviside function at the origin is  $1/2$ . Return the value of `heaviside(0)`. Find the Z-transform of `heaviside(x)` for this default value of `heaviside(0)`.

```
syms x
H = heaviside(sym(0))
```

```
H =
```

$$\frac{1}{2}$$

```
Z = ztrans(heaviside(x))
```

```
Z =
```

$$\frac{1}{z-1} + \frac{1}{2}$$

Other common values for the Heaviside function at the origin are 0 and 1. Set `heaviside(0)` to 1 by changing the 'HeavisideAtOrigin' setting. Store the previous value returned by `sympref` so that you can restore it later.

```
oldVal = sympref('HeavisideAtOrigin',1)
```

```
oldVal =
```

$$\frac{1}{2}$$

Check if the new value of `heaviside(0)` is 1. Find the Z-transform of `heaviside(x)` for this value.

```
H = heaviside(sym(0))
```

```
H = 1
```

```
Z = ztrans heaviside(x)
```

```
Z =
```

$$\frac{1}{z-1} + 1$$

The new output of `heaviside(0)` modifies the output of `ztrans`.

The settings that you set using `sympref` persist through your current and future MATLAB® sessions. Restore the previous value of `heaviside(0)` to `oldVal`.

```
sympref('HeavisideAtOrigin',oldVal);
```

Alternatively, you can restore the default value of `'HeavisideAtOrigin'` by specifying the `'default'` option.

```
sympref('HeavisideAtOrigin','default');
```

## Modify Display of Symbolic Expressions in Live Scripts

By default, symbolic expressions in live scripts are displayed in abbreviated output format, and typeset in mathematical notation. You can turn off abbreviated output format and typesetting using symbolic settings.

Create a symbolic expression and return the output, which is abbreviated by default.

```
syms a b c d x
f = a*x^3 + b*x^2 + c*x + d;
outputAbbrev = sin(f) + cos(f) + tan(f) + log(f) + 1/f

outputAbbrev =
```

$$\cos(\sigma_1) + \log(\sigma_1) + \sin(\sigma_1) + \tan(\sigma_1) + \frac{1}{\sigma_1}$$

where

$$\sigma_1 = ax^3 + bx^2 + cx + d$$

Turn off abbreviated output format by setting `'AbbreviateOutput'` to `false`. Redisplay the expression.

```
sympref('AbbreviateOutput',false);
outputLong = sin(f) + cos(f) + tan(f) + log(f) + 1/f

outputLong =
```

$$\cos(ax^3 + bx^2 + cx + d) + \log(ax^3 + bx^2 + cx + d) + \sin(ax^3 + bx^2 + cx + d) + \tan(ax^3 + bx^2 + cx + d) + \frac{1}{ax^3 + bx^2 + cx + d}$$

Create another symbolic expression and return the output, which is typeset in mathematical notation by default. Turn off rendered output and use ASCII output instead by setting `'TypesetOutput'` to `false`. First, show the typeset output.

```
syms a b c d x
f = exp(a^b)+pi
```

$$f = \pi + e^{a^b}$$

Turn off typesetting by setting 'TypesetOutput' to false. Redisplay the expression.

```
sympref('TypesetOutput',false);
f = exp(a^b)+pi
```

```
f =
```

```
pi + exp(a^b)
```

The settings that you set using `sympref` persist through your current and future MATLAB® sessions. Restore the default values of 'AbbreviateOutput' and 'TypesetOutput' by specifying the 'default' option.

```
sympref('AbbreviateOutput','default');
sympref('TypesetOutput','default');
```

### Display Symbolic Results in Floating-Point Format

Display symbolic results in floating-point output format, that is the short, fixed-decimal format with 4 digits after the decimal point.

Create a quadratic equation.

```
syms x
eq = x^2 - 2e3/sym(pi)*x + 0.5 == 0
```

$$eq = x^2 - \frac{2000x}{\pi} + \frac{1}{2} = 0$$

Find the solutions of the equation using `solve`.

```
sols = solve(eq,x)
```

$$sols = \begin{pmatrix} -\frac{\sqrt{2}\sqrt{2000000-\pi^2}-2000}{2\pi} \\ \frac{\sqrt{2}\sqrt{2000000-\pi^2}+2000}{2\pi} \end{pmatrix}$$

Set the 'FloatingPointOutput' setting to true. Display the quadratic equation and its solutions in floating-point format.

```
sympref('FloatingPointOutput',true);
eq
```

$$eq = x^2 - 636.6198x + 0.5000 = 0$$



```
sols
```

```
sols =  
      (7.8540e-04)  
      (636.6190 )
```

The floating-point format displays each symbolic number in the short, fixed-decimal format with 4 digits after the decimal point. The 'FloatingPointOutput' setting does not affect the floating-point precision in a symbolic computation. To compute symbolic numbers using floating-point arithmetic, use the `vpa` function.

Now restore the default value of 'FloatingPointOutput' by specifying the 'default' option. Compute the floating-point approximation of the solutions in 8 significant digits using `vpa`.

```
sympref('FloatingPointOutput','default');  
sols = vpa(sols,8)  
  
sols =  
      (0.00078539913)  
      (636.61899 )
```

### Modify Output Order of Symbolic Polynomial

Create a symbolic polynomial expression consisting of multiple variables. Display the polynomial in the default order.

```
syms x y a b  
p1 = b^2*x^2 + a^2*x + y^3 + 2  
  
p1 = a^2 x + b^2 x^2 + y^3 + 2
```

The default option sorts the output in alphabetical order, without distinguishing the different symbolic variables in each monomial term.

Now display the same polynomial in ascending order by setting 'PolynomialDisplayStyle' to 'ascend'.

```
sympref('PolynomialDisplayStyle','ascend');  
p1  
  
p1 = 2 + y^3 + a^2 x + b^2 x^2
```

The 'ascend' option sorts the output in ascending order based on the importance of the variables. Here, the most important variable  $x$  with the highest order in a monomial term is displayed last.

Display the polynomial in descending order by setting 'PolynomialDisplayStyle' to 'descend'.

```
sympref('PolynomialDisplayStyle','descend');  
p1  
  
p1 = b^2 x^2 + a^2 x + y^3 + 2
```

The settings that you set using `sympref` persist through your current and future MATLAB® sessions. Restore the default value of 'PolynomialDisplayStyle' by specifying the 'default' option.

```
sympref('PolynomialDisplayStyle','default');
```

### Modify Display of Symbolic Matrix in Live Scripts

By default, a symbolic matrix in live scripts is set in parentheses (round brackets). You can specify the use of square brackets instead by using `sympref`.

Create a symbolic matrix consisting of symbolic variables and numbers.

```
syms x y
A = [x*y, 2; 4, y^2]
```

```
A =
```

$$\begin{pmatrix} xy & 2 \\ 4 & y^2 \end{pmatrix}$$

Display the matrix with square brackets by setting `'MatrixWithSquareBrackets'` to `true`.

```
sympref('MatrixWithSquareBrackets',true);
A
```

```
A =
```

$$\begin{bmatrix} xy & 2 \\ 4 & y^2 \end{bmatrix}$$

The settings that you set using `sympref` persist through your current and future MATLAB® sessions. Restore the default value by specifying the `'default'` option.

```
sympref('MatrixWithSquareBrackets','default');
```

### Save and Restore All Symbolic Settings

Instead of saving and restoring individual settings one by one, you can use `sympref` to save and restore all symbolic settings simultaneously.

Return a structure containing the values of all symbolic settings by using `sympref()`.

```
oldPrefs = sympref()

oldPrefs = struct with fields:
    FourierParameters: [1 -1]
    HeavisideAtOrigin: 1/2
    AbbreviateOutput: 1
    TypesetOutput: 1
    FloatingPointOutput: 0
    PolynomialDisplayStyle: 'default'
    MatrixWithSquareBrackets: 0
```

Access the value of each symbolic setting by addressing the field of the structure. Alternatively, you can use the command `sympref(pref)`.

```

val1 = oldPrefs.FourierParameters
val1 = (1 -1)
val2 = oldPrefs.HeavisideAtOrigin
val2 =
    1
   2
val3 = sympref('FourierParameters')
val3 = (1 -1)

```

To modify multiple symbolic settings simultaneously, you can create a structure `prefs` that contains the setting values. Use the command `sympref(prefs)` to set multiple settings.

```

prefs.FourierParameters = [1/(2*sym(pi)) 1]

prefs = struct with fields:
    FourierParameters: [1/(2*pi)    1]

prefs.HeavisideAtOrigin = 1

prefs = struct with fields:
    FourierParameters: [1/(2*pi)    1]
    HeavisideAtOrigin: 1

```

```
sympref(prefs);
```

Because symbolic settings persist through your current and future MATLAB® sessions, you need to restore your previous settings. Restore the saved settings using `sympref(oldPrefs)`.

```
sympref(oldPrefs);
```

Alternatively, you can set all symbolic settings to their default values by specifying the `'default'` option.

```
sympref('default');
```

## Input Arguments

### **pref** — Symbolic setting

character vector | string

Symbolic setting, specified as a character vector or string. The value options for each symbolic setting follow.

| setting               | Value                                                                                                                                            | Description                                                                                                                                                                                                                                                                                                     |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 'FourierParameters'   | Two-element row vector $[c, s]$ . The parameters $c$ and $s$ must be numeric or symbolic numbers.<br><br>Default:<br><code>sym([1, -1])</code> . | Set the values of the parameters $c$ and $s$ in the Fourier transform:<br><br>$F(w) = c \int_{-\infty}^{\infty} f(t)e^{iswt} dt.$<br><br>See "Change Parameter Values of Fourier Transform" on page 6-1501.                                                                                                     |
| 'HeavisideAtOrigin'   | Scalar value, specified as a numeric or symbolic number.<br><br>Default: <code>sym(1/2)</code> .                                                 | Set the value of the Heaviside function <code>heaviside(0)</code> at the origin.<br><br>See "Change Value of Heaviside Function at Origin" on page 6-1502.                                                                                                                                                      |
| 'AbbreviateOutput'    | Logical value (boolean).<br><br>Default: logical 1 ( <code>true</code> ).                                                                        | Specify whether or not to use abbreviated output format of symbolic variables and expressions in Live Scripts.<br><br>See "Modify Display of Symbolic Expressions in Live Scripts" on page 6-1503.                                                                                                              |
| 'TypesetOutput'       | Logical value (boolean).<br><br>Default: logical 1 ( <code>true</code> ).                                                                        | Typeset or use ASCII characters for the output of symbolic variables and expressions in Live Scripts.<br><br>See "Modify Display of Symbolic Expressions in Live Scripts" on page 6-1503.                                                                                                                       |
| 'FloatingPointOutput' | Logical value (boolean).<br><br>Default: logical 0 ( <code>false</code> ).                                                                       | Specify whether or not to display symbolic results in floating-point output format.<br><br>The <code>true</code> value option displays symbolic results in the short fixed-decimal format with 4 digits after the decimal point.<br><br>See "Display Symbolic Results in Floating-Point Format" on page 6-1504. |

| setting                    | Value                                                                                                         | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----------------------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 'PolynomialDisplayStyle'   | Character vector or scalar string, specified as 'default', 'ascend', or 'descend'.<br><br>Default: 'default'. | Display a symbolic polynomial in default, ascending, or descending order. <ul style="list-style-type: none"> <li>The 'default' option sorts the output in alphabetical order, without distinguishing the different symbolic variables in each monomial term.</li> <li>The 'ascend' option sorts the output in ascending order based on the standard mathematical notation for polynomials. For example, the variable x with the highest order in a monomial term is displayed last, preceded by monomial terms that contain the variables y, z, t, s, and so on.</li> <li>The 'descend' option sorts the output in descending order based on the standard mathematical notation for polynomials. This option is the exact opposite of 'ascend'.</li> </ul> See “Modify Output Order of Symbolic Polynomial” on page 6-1505. |
| 'MatrixWithSquareBrackets' | Logical value (boolean).<br><br>Default: logical 0 (false).                                                   | Set matrices in round brackets or parentheses (round brackets) in Live Scripts.<br><br>See “Modify Display of Symbolic Matrix in Live Scripts” on page 6-1506.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |

**value — Value of symbolic setting**`'default'` (default) | valid value

Value of symbolic setting, specified as `'default'` or a valid value of the specified setting `pref`.

**prefs — Symbolic settings**

structure array

Symbolic settings, specified as a structure array. You can set multiple settings by declaring the field names and the valid setting values.

**Output Arguments****oldVal — Value of symbolic setting**

valid value

Value of symbolic setting, returned as a valid value. `oldVal` represents the existing value of the setting `pref` before the call to `sympref`.

**oldPrefs — All symbolic settings**

structure array

All symbolic settings, returned as a structure array. `oldPrefs` represent the existing values of all settings before the call to `sympref`.

## Tips

- The `clear` command does *not* reset or affect symbolic settings. Use `sympref` to manipulate symbolic settings.
- The symbolic settings you set using `sympref` also determine the output generated by the `latex` and `mathml` functions.
- The 'FloatingPointOutput' setting affects only the output display format of symbolic numbers. To change the output display format of numeric numbers, use the `format` function. To compute symbolic numbers using floating-point precision, use the `vpa` or `digits` functions.

## Version History

### Introduced in R2015a

### R2019a: Additional settings to change display of symbolic output

The `sympref` function accepts the following settings:

- 'FloatingPointOutput' displays symbolic numbers in short fixed-decimal format.
- 'PolynomialDisplayStyle' displays polynomials in ascending or descending order.
- 'MatrixWithSquareBrackets' displays symbolic matrices with square brackets in Live Scripts.

## See Also

`fourier` | `heaviside` | `ifourier` | `digits` | `vpa` | `taylor` | `latex` | `mathml`

## Topics

“Change Output Display Format of Symbolic Results in the Live Editor” on page 2-11

“Change Output Format of Symbolic and Variable-Precision Arithmetic” on page 2-8

# symprod

Product of series

## Syntax

```
F = symprod(f,k,a,b)
F = symprod(f,k)
```

## Description

`F = symprod(f,k,a,b)` returns the product of the series with terms that expression `f` specifies, which depend on symbolic variable `k`. The value of `k` ranges from `a` to `b`. If you do not specify `k`, `symprod` uses the variable that `symvar` determines. If `f` is a constant, then the default variable is `x`.

`F = symprod(f,k)` returns the product of the series that expression `f` specifies, which depend on symbolic variable `k`. The value of `k` starts at 1 with an unspecified upper bound. The product `F` is returned in terms of `k` where `k` represents the upper bound. This product `F` differs from the indefinite product. If you do not specify `k`, `symprod` uses the variable that `symvar` determines. If `f` is a constant, then the default variable is `x`.

## Examples

### Find Product of Series Specifying Bounds

Find the following products of series

$$P1 = \prod_{k=2}^{\infty} 1 - \frac{1}{k^2},$$

$$P2 = \prod_{k=2}^{\infty} \frac{k^2}{k^2 - 1}.$$

```
syms k
P1 = symprod(1 - 1/k^2, k, 2, Inf)
P2 = symprod(k^2/(k^2 - 1), k, 2, Inf)
```

```
P1 =
1/2
P2 =
2
```

Alternatively, specify bounds as a row or column vector.

```
syms k
P1 = symprod(1 - 1/k^2, k, [2 Inf])
P2 = symprod(k^2/(k^2 - 1), k, [2; Inf])
```

```
P1 =
1/2
P2 =
2
```

**Find Product of Series Specifying Product Index and Bounds**

Find the product of series

$$P = \prod_{k=1}^{10000} \frac{e^{kx}}{x}.$$

```
syms k x
P = symprod(exp(k*x)/x, k, 1, 10000)

P =
exp(50005000*x)/x^10000
```

**Find Product of Series with Unspecified Bounds**

When you do not specify the bounds of a series are unspecified, the variable  $k$  starts at 1. In the returned expression,  $k$  itself represents the upper bound.

Find the products of series with an unspecified upper bound

$$P1 = \prod_k k,$$

$$P2 = \prod_k \frac{2k-1}{k^2}.$$

```
syms k
P1 = symprod(k, k)
P2 = symprod((2*k - 1)/k^2, k)

P1 =
factorial(k)
P2 =
(1/2^(2*k)*2^(k + 1)*factorial(2*k))/(2*factorial(k)^3)
```

**Input Arguments****f — Expression defining terms of series**

symbolic expression | symbolic function | symbolic vector | symbolic matrix | symbolic number

Expression defining terms of series, specified as a symbolic expression, function, constant, or a vector or matrix of symbolic expressions, functions, or constants.

**k — Product index**

symbolic variable

Product index, specified as a symbolic variable. If you do not specify this variable, `symprod` uses the default variable that `symvar(expr, 1)` determines. If  $f$  is a constant, then the default variable is  $x$ .

**a — Lower bound of product index**

number | symbolic number | symbolic variable | symbolic expression | symbolic function

Lower bound of product index, specified as a number, symbolic number, variable, expression, or function (including expressions and functions with infinities).

**b — Upper bound of product index**

number | symbolic number | symbolic variable | symbolic expression | symbolic function



Upper bound of product index, specified as a number, symbolic number, variable, expression, or function (including expressions and functions with infinities).

## More About

### Definite Product

The definite product of a series is defined as

$$\prod_{i=a}^b x_i = x_a \cdot x_{a+1} \cdot \dots \cdot x_b$$

### Indefinite Product

The indefinite product of  $x_i$  over  $i$  is

$$f(i) = \prod_i x_i$$

This definition holds under the assumption that the following identity is true for all values of  $i$ .

$$\frac{f(i+1)}{f(i)} = x_i$$

---

**Note** symprod does not compute indefinite products.

---

## Version History

Introduced in R2011b

### See Also

cumprod | cumsum | int | syms | symsum | symvar

## symReadSSCParameters

Load parameters from Simscape component

### Syntax

```
[names,values,units] = symReadSSCParameters(componentName)
```

### Description

[names,values,units] = symReadSSCParameters(componentName) returns cell arrays containing the names, values, and units of all parameters from the Simscape component called componentName.

### Examples

#### Parameters of Simscape Component

Load the names, values, and units of the parameters of a Simscape component.

Suppose you have the Simscape component `friction.ssc` in your current folder.

```
type('friction.ssc');

component friction < foundation.mechanical.rotational.branch

parameters
    brkwy_trq = { 25, 'N*m' };           % Breakaway friction torque
    Col_trq = { 20, 'N*m' };             % Coulomb friction torque
    visc_coef = { 0.001, 'N*m*s/rad' }; % Viscous friction coefficient
    trans_coef = { 10, 's/rad' };        % Transition approximation coefficient
    vel_thr = { 1e-4, 'rad/s' };         % Linear region velocity threshold
end

parameters (Access=private)
    brkwy_trq_th = { 24.995, 'N*m' };    % Breakaway torque at threshold velocity
end

function setup
    % Parameter range checking
    if brkwy_trq <= 0
        pm_error('simscape:GreaterThanZero','Breakaway friction torque' )
    end
    if Col_trq <= 0
        pm_error('simscape:GreaterThanZero','Coulomb friction torque' )
    end
    if Col_trq > brkwy_trq
        pm_error('simscape:LessThanOrEqual','Coulomb friction torque',...
            'Breakaway friction torque')
    end
    if visc_coef < 0
        pm_error('simscape:GreaterThanOrEqualToZero','Viscous friction coefficient')
```

```

end
if trans_coef <= 0
    pm_error('simscape:GreaterThanZero','Transition approximation coefficient')
end
if vel_thr <= 0
    pm_error('simscape:GreaterThanZero','Linear region velocity threshold')
end

% Computing breakaway torque at threshold velocity
brkwy_trq_th = visc_coef * vel_thr + Col_trq + (brkwy_trq - Col_trq) * ...
    exp(-trans_coef * vel_thr);
end

equations
    if (abs(w) <= vel_thr)
        % Linear region
        t == brkwy_trq_th * w / vel_thr;
    elseif w > 0
        t == visc_coef * w + Col_trq + ...
            (brkwy_trq - Col_trq) * exp(-trans_coef * w);
    else
        t == visc_coef * w - Col_trq - ...
            (brkwy_trq - Col_trq) * exp(-trans_coef * abs(w));
    end
end
end

```

Load the names, values, and units of the parameters of the component `friction.ssc`.

```
[names,values,units] = symReadSSCParameters('friction.ssc');
```

In this example, all elements of the resulting cell arrays are scalars. You can convert the cell arrays to symbolic vectors.

```

names_sym = cell2sym(names)

names_sym =
[ Col_trq, brkwy_trq, brkwy_trq_th, trans_coef, vel_thr, visc_coef]

values_sym = cell2sym(values)

values_sym =
[ 20, 25, 4999/200, 10, 1/10000, 1/1000]

```

Create individual symbolic variables from the elements of the cell array `names` in the MATLAB workspace. This command creates the symbolic variables `Col_trq`, `brkwy_trq`, `brkwy_trq_th`, `trans_coef`, `vel_thr`, and `visc_coef` as sym objects in the workspace.

```
syms(names)
```

## Input Arguments

**componentName** — Simscape component name

file name enclosed in single quotes

Simscape component name, specified as a file name enclosed in single quotes. The file must have the extension `.ssc`. If you do not provide the file extension, `symReadSSCParameters` assumes it to be `.ssc`. The component must be on the MATLAB path or in the current folder.

Example: `'MyComponent.ssc'`

## Output Arguments

**names — Names of all parameters of Simscape component**

cell array

Names of all parameters of a Simscape component, returned as a cell array.

Data Types: `cell`

**values — Values of all parameters of Simscape component**

cell array

Values of all parameters of a Simscape component, returned as a cell array.

Data Types: `cell`

**units — Units of all parameters of Simscape component**

cell array

Units of all parameters of a Simscape component, returned as a cell array.

Data Types: `cell`

## Version History

Introduced in R2016a

## See Also

`symReadSSCVariables` | `symWriteSSC`

# symReadSSCVariables

Load variables from Simscape component

## Syntax

```
[names,values,units] = symReadSSCVariables(componentName)
[names,values,units] = symReadSSCVariables(
componentName,'ReturnFunctions',true)
```

## Description

[names,values,units] = symReadSSCVariables(componentName) returns cell arrays containing the names, values, and units of all variables from the Simscape component called componentName.

[names,values,units] = symReadSSCVariables(componentName,'ReturnFunctions',true) returns the names as symbolic functions of variable t.

## Examples

### Variables of Simscape Component

Load the names, values, and units of the variables of a Simscape component.

Suppose you have the Simscape component `friction.ssc` in your current folder.

```
type('friction.ssc');

component friction < foundation.mechanical.rotational.branch

parameters
    brkwy_trq = { 25, 'N*m' };           % Breakaway friction torque
    Col_trq = { 20, 'N*m' };             % Coulomb friction torque
    visc_coef = { 0.001, 'N*m*s/rad' }; % Viscous friction coefficient
    trans_coef = { 10, 's/rad' };        % Transition approximation coefficient
    vel_thr = { 1e-4, 'rad/s' };         % Linear region velocity threshold
end

parameters (Access=private)
    brkwy_trq_th = { 24.995, 'N*m' };    % Breakaway torque at threshold velocity
end

function setup
    % Parameter range checking
    if brkwy_trq <= 0
        pm_error('simscape:GreaterThanZero','Breakaway friction torque' )
    end
    if Col_trq <= 0
        pm_error('simscape:GreaterThanZero','Coulomb friction torque' )
    end
end
```

```

    if Col_trq > brkwy_trq
        pm_error('simscape:LessThanOrEqual','Coulomb friction torque',...
            'Breakaway friction torque')
    end
    if visc_coef < 0
        pm_error('simscape:GreaterThanOrEqualToZero','Viscous friction coefficient')
    end
    if trans_coef <= 0
        pm_error('simscape:GreaterThanZero','Transition approximation coefficient')
    end
    if vel_thr <= 0
        pm_error('simscape:GreaterThanZero','Linear region velocity threshold')
    end

    % Computing breakaway torque at threshold velocity
    brkwy_trq_th = visc_coef * vel_thr + Col_trq + (brkwy_trq - Col_trq) * ...
        exp(-trans_coef * vel_thr);
end

equations
    if (abs(w) <= vel_thr)
        % Linear region
        t == brkwy_trq_th * w / vel_thr;
    elseif w > 0
        t == visc_coef * w + Col_trq + ...
            (brkwy_trq - Col_trq) * exp(-trans_coef * w);
    else
        t == visc_coef * w - Col_trq - ...
            (brkwy_trq - Col_trq) * exp(-trans_coef * abs(w));
    end
end
end

```

Load the names, values, and units of the variables of the component `friction.ssc`.

```
[names,values,units] = symReadSSCVariables('friction.ssc');
```

In this example, all elements of the resulting cell arrays are scalars. You can convert the cell arrays to symbolic vectors.

```
names_sym = cell2sym(names)
```

```
names_sym =
[ t, w]
```

```
values_sym = cell2sym(values)
```

```
values_sym =
[ 0, 0]
```

Create individual symbolic variables from the elements of the cell array `names` in the MATLAB workspace. This command creates the symbolic variables `t` and `w` as `sym` objects in the workspace.

```
syms(names)
```

## Variables of Simscape Component Returned as Functions

Load the names of the variables of a Simscape component while converting them to symbolic functions of the variable `t`.

Suppose you have the Simscape component `source.ssc` in your current folder.

```
type('source.ssc');

component source
% Electrical Source
% Defines an electrical source with positive and negative external nodes.
% Also defines associated through and across variables.

nodes
    p = foundation.electrical.electrical; % :top
    n = foundation.electrical.electrical; % :bottom
end

variables(Access=protected)
    i = { 0, 'A' }; % Current
    v = { 0, 'V' }; % Voltage
end

branches
    i : p.i -> n.i;
end

equations
    v == p.v - n.v;
end

end
```

Load the names of the variables of the component `source.ssc` by setting `'ReturnFunctions'` to `true`.

```
[names,~,~] = symReadSSCVariables('source.ssc','ReturnFunctions',true);
```

In this example, all elements of the resulting cell arrays are scalars. You can convert the cell arrays to symbolic vectors.

```
names_symfun = cell2sym(names)

names_symfun =
[ i(t), v(t)]
```

Create individual symbolic functions from the elements of the cell array `names` in the MATLAB workspace. This command creates the symbolic functions `i` and `v` as `symfun` objects, and their variable `t` as a `sym` object in the workspace.

```
syms(names)
```

## Input Arguments

**componentName** — Simscape component name

file name enclosed in single quotes

Simscape component name, specified as a file name enclosed in single quotes. The file must have the extension `.ssc`. If you do not provide the file extension, `symReadSSCVariables` assumes it to be `.ssc`. The component must be on the MATLAB path or in the current folder.

Example: `'MyComponent.ssc'`

## Output Arguments

**names — Names of all variables of Simscape component**

cell array

Names of all variables of a Simscape component, returned as a cell array.

Data Types: `cell`

**values — Values of all variables of Simscape component**

cell array

Values of all variables of a Simscape component, returned as a cell array.

Data Types: `cell`

**units — Units of all variables of Simscape component**

cell array

Units of all variables of a Simscape component, returned as a cell array.

Data Types: `cell`

## Version History

**Introduced in R2016a**

## See Also

`symReadSSCParameters` | `symWriteSSC`



# syms

Create symbolic scalar variables and functions, and matrix variables and functions

## Syntax

```
syms var1 ... varN
syms var1 ... varN [n1 ... nM]
syms var1 ... varN n
syms ____ set

syms f(var1,...,varN)
syms f(var1,...,varN) [n1 ... nM]
syms f(var1,...,varN) n

syms var1 ... varN [nrow ncol] matrix
syms var1 ... varN n matrix

syms f(var1,...,varN) [nrow ncol] matrix
syms f(var1,...,varN) [nrow ncol] matrix keepargs
syms f(var1,...,varN) n matrix
syms f(var1,...,varN) n matrix keepargs

syms(symArray)

syms
S = syms
```

## Description

### Symbolic Scalar Variables

`syms var1 ... varN` creates symbolic scalar variables `var1` through `varN` of type `sym`. Separate the different variables with spaces. This syntax clears all previous definitions of `var1 ... varN`.

`syms var1 ... varN [n1 ... nM]` creates arrays of symbolic scalar variables `var1 ... varN`, where each array has the size `n1-by-...-by-nM` and contains automatically generated symbolic scalar variables as its elements. For example, `syms a [1 3]` creates the symbolic array `a = [a1 a2 a3]` and the symbolic scalar variables `a1`, `a2`, and `a3` in the MATLAB workspace. For multidimensional arrays, these elements have the prefix `a` followed by the element's index using `_` as a delimiter, such as `a1_3_2`.

`syms var1 ... varN n` creates `n`-by-`n` matrices of symbolic scalar variables filled with automatically generated elements.

`syms ____ set` sets the assumption that the created symbolic scalar variables belong to `set`, and clears other assumptions. Here, `set` can be `real`, `positive`, `integer`, or `rational`. You can also combine multiple assumptions using spaces. For example, `syms x positive rational` creates a symbolic scalar variable `x` with a positive rational value. Use this option in addition to any of the input argument combinations in previous syntaxes.

### Symbolic Scalar Functions

`syms f(var1,...,varN)` creates the symbolic function `f` of type `symfun` and the symbolic scalar variables `var1,...,varN` of type `sym`, which represent the input arguments of `f`. This syntax clears all previous definitions of `var1,...,varN` including symbolic assumptions. The evaluated symbolic function `f(var1,...,varN)` is of type `sym`.

`syms f(var1,...,varN) [n1 ... nM]` creates an `n1`-by-...-by-`nM` symbolic array with automatically generated symbolic functions as its elements. This syntax also generates the symbolic scalar variables `var1,...,varN` that represent the input arguments of `f`. For example, `syms f(x) [1 2]` creates the symbolic array `f(x) = [f1(x) f2(x)]`, the symbolic functions `f1` and `f2`, and the symbolic scalar variable `x` in the MATLAB workspace. For multidimensional arrays, these elements have the prefix `f` followed by the element's index using `_` as a delimiter, such as `f1_3_2`.

`syms f(var1,...,varN) n` creates an `n`-by-`n` matrix of symbolic functions filled with automatically generated elements.

### Symbolic Matrix Variables

`syms var1 ... varN [nrow ncol] matrix` creates symbolic matrix variables `var1 ... varN` of type `symmatrix`, where each symbolic matrix variable has the size `nrow`-by-`ncol`. (since R2021a)

`syms var1 ... varN n matrix` creates `n`-by-`n` symbolic matrix variables. (since R2021a)

### Symbolic Matrix Functions

`syms f(var1,...,varN) [nrow ncol] matrix` creates the symbolic matrix function `f` of type `symfunmatrix` and the symbolic scalar variables `var1,...,varN` of type `sym`. The evaluated symbolic matrix function `f(var1,...,varN)` is of type `symmatrix` and has the size `nrow`-by-`ncol`. This syntax clears all previous definitions of `var1,...,varN` including symbolic assumptions. (since R2022a)

`syms f(var1,...,varN) [nrow ncol] matrix keepargs` keeps existing definitions of `var1,...,varN` in the workspace. If any of the variables `var1,...,varN` does not exist in the workspace, then this syntax creates them as symbolic scalar variables of type `sym`. The size of the evaluated symbolic matrix function `f(var1,...,varN)` is `nrow`-by-`ncol`. (since R2022a)

`syms f(var1,...,varN) n matrix` creates a square symbolic matrix function, where the evaluated symbolic matrix function `f(var1,...,varN)` has the size `n`-by-`n`. This syntax clears all previous definitions of `var1,...,varN` including symbolic assumptions. (since R2022a)

`syms f(var1,...,varN) n matrix keepargs` keeps existing definitions of `var1,...,varN` in the workspace. If any of the variables `var1,...,varN` does not exist in the workspace, then this syntax creates them as symbolic scalar variables of type `sym`. (since R2022a)

### Array of Symbolic Objects

`syms(symArray)` creates the symbolic scalar variables and functions contained in `symArray`, where `symArray` is either a vector of symbolic scalar variables or a cell array of symbolic scalar variables and functions. This syntax clears all previous definitions of variables specified in `symArray` including symbolic assumptions. Use this syntax only when such an array is returned by another function, such as `solve` or `symReadSSCVariables`.

### Names of Symbolic Objects

`syms` lists the names of all symbolic scalar variables, functions, matrix variables, matrix functions, and arrays in the MATLAB workspace.

`S = syms` returns a cell array of the names of all symbolic scalar variables, functions, matrix variables, matrix functions, and arrays.

## Examples

### Create Symbolic Scalar Variables

Create symbolic scalar variables `x` and `y`.

```
syms x y
x
y
y = y
```

### Create Vector of Symbolic Scalar Variables

Create a 1-by-4 vector of symbolic scalar variables `a` with the automatically generated elements  $a_1, \dots, a_4$ . This command also creates the symbolic scalar variables `a1`, ..., `a4` in the MATLAB workspace.

```
syms a [1 4]
a
a = (a1 a2 a3 a4)
```

`whos`

| Name | Size | Bytes | Class | Attributes |
|------|------|-------|-------|------------|
| a    | 1x4  | 8     | sym   |            |
| a1   | 1x1  | 8     | sym   |            |
| a2   | 1x1  | 8     | sym   |            |
| a3   | 1x1  | 8     | sym   |            |
| a4   | 1x1  | 8     | sym   |            |

You can change the naming format of the generated elements by using a format character vector. Declare the symbolic scalar variables by enclosing each variable name in single quotes. `syms` replaces `%d` in the format character vector with the index of the element to generate the element names.

```
syms 'p_a%d' 'p_b%d' [1 4]
p_a
p_a = (pa1 pa2 pa3 pa4)
p_b
p_b = (pb1 pb2 pb3 pb4)
```

### Create Matrix of Symbolic Scalar Variables

Create a 3-by-4 matrix of symbolic scalar variables with automatically generated elements. The elements are of the form  $A_{i,j}$ , which generates the symbolic matrix variables  $A_{1,1}, \dots, A_{3,4}$ .

```
syms A [3 4]
```

```
A
```

```
A =
```

$$\begin{pmatrix} A_{1,1} & A_{1,2} & A_{1,3} & A_{1,4} \\ A_{2,1} & A_{2,2} & A_{2,3} & A_{2,4} \\ A_{3,1} & A_{3,2} & A_{3,3} & A_{3,4} \end{pmatrix}$$

### Manage Assumptions for Symbolic Scalar Variables

Create symbolic scalar variables  $x$  and  $y$ , and assume that they are integers.

```
syms x y integer
```

Create another scalar variable  $z$ , and assume that it has a positive rational value.

```
syms z positive rational
```

Check assumptions.

```
assumptions
```

```
ans = (x ∈ ℤ y ∈ ℤ z ∈ ℚ 0 < z)
```

Alternatively, check assumptions on each variable. For example, check assumptions set on the variable  $x$ .

```
assumptions(x)
```

```
ans = x ∈ ℤ
```

Clear assumptions on  $x$ ,  $y$ , and  $z$ .

```
assume([x y z], "clear")  
assumptions
```

```
ans =
```

```
Empty sym: 1-by-0
```

Create a 1-by-3 symbolic array  $a$ , and assume that the array elements have real values.

```
syms a [1 3] real  
assumptions
```

```
ans = (a1 ∈ ℝ a2 ∈ ℝ a3 ∈ ℝ)
```

### Create and Evaluate Symbolic Functions

Create symbolic functions with one and two arguments.

```
syms s(t) f(x,y)
```

Both  $s$  and  $f$  are abstract symbolic functions. They do not have symbolic expressions assigned to them, so the bodies of these functions are  $s(t)$  and  $f(x,y)$ , respectively.

Specify a formula for  $f$ .

```
f(x,y) = x + 2*y
```

```
f(x, y) = x + 2*y
```

Compute the function value at the point  $x = 1$  and  $y = 2$ .

```
f(1,2)
```

```
ans = 5
```

### Create and Evaluate Symbolic Functions with Matrices as Formulas

Create a symbolic function and specify its formula by using a matrix of symbolic scalar variables.

```
syms x
M = [x x^3; x^2 x^4];
f(x) = M
```

```
f(x) =
    (
      x  x^3
      x^2 x^4
    )
```

Compute the function value at the point  $x = 2$ .

```
f(2)
```

```
ans =
    (
      2  8
      4 16
    )
```

Compute the value of this function for  $x = [1 \ 2 \ 3; \ 4 \ 5 \ 6]$ . The result is a cell array of symbolic matrices.

```
xVal = [1 2 3; 4 5 6];
y = f(xVal)
```

```
y=2x2 cell array
    {2x3 sym}    {2x3 sym}
    {2x3 sym}    {2x3 sym}
```

Access the contents of a cell in the cell array by using braces.

```
y{1}
ans =

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$$

```

### Create and Evaluate Symbolic Matrices as Functions of Two Variables

Create a 2-by-2 symbolic matrix with automatically generated symbolic functions as its elements.

```
syms f(x,y) 2
f
f(x, y) =

$$\begin{pmatrix} f_{1,1}(x,y) & f_{1,2}(x,y) \\ f_{2,1}(x,y) & f_{2,2}(x,y) \end{pmatrix}$$

```

Assign symbolic expressions to the symbolic functions  $f_{1,1}(x,y)$  and  $f_{2,2}(x,y)$ . These functions are displayed as  $f_{1,1}(x,y)$  and  $f_{2,2}(x,y)$  in the Live Editor. When you assign these expressions, the symbolic matrix **f** still contains the initial symbolic functions in its elements.

```
f1_1(x,y) = 2*x;
f2_2(x,y) = x - y;
f
```

```
f(x, y) =

$$\begin{pmatrix} f_{1,1}(x,y) & f_{1,2}(x,y) \\ f_{2,1}(x,y) & f_{2,2}(x,y) \end{pmatrix}$$

```

Substitute the expressions assigned to  $f_{1,1}(x,y)$  and  $f_{2,2}(x,y)$  by using the `subs` function.

```
A = subs(f)
A(x, y) =

$$\begin{pmatrix} 2x & f_{1,2}(x,y) \\ f_{2,1}(x,y) & x-y \end{pmatrix}$$

```

Evaluate the value of the symbolic matrix **A**, which contains the substituted expressions, at  $x = 2$  and  $y = 3$ .

```
A(2,3)
ans =

$$\begin{pmatrix} 4 & f_{1,2}(2,3) \\ f_{2,1}(2,3) & -1 \end{pmatrix}$$

```

### Create Symbolic Matrix Variables

Create two symbolic matrix variables with size 2-by-3. Nonscalar symbolic matrix variables are displayed as bold characters in the Live Editor and Command Window.

```
syms A B [2 3] matrix
A
```

```
A = A
```

```
B
```

```
B = B
```

Add the two matrices. The result is represented by the matrix notation  $A+B$ .

```
X = A + B
```

```
X = A + B
```

The data type of X is `symmatrix`.

```
class(X)
```

```
ans =
'symmatrix'
```

Convert the symbolic matrix variable X to a matrix of symbolic scalar variables Y. The result is denoted by the sum of the matrix components.

```
Y = symmatrix2sym(X)
```

```
Y =

$$\begin{pmatrix} A_{1,1} + B_{1,1} & A_{1,2} + B_{1,2} & A_{1,3} + B_{1,3} \\ A_{2,1} + B_{2,1} & A_{2,2} + B_{2,2} & A_{2,3} + B_{2,3} \end{pmatrix}$$

```

The data type of Y is `sym`.

```
class(Y)
```

```
ans =
'sym'
```

Show that the converted result in Y is equal to the sum of two matrices of symbolic scalar variables.

```
syms A B [2 3]
Y2 = A + B
```

```
Y2 =

$$\begin{pmatrix} A_{1,1} + B_{1,1} & A_{1,2} + B_{1,2} & A_{1,3} + B_{1,3} \\ A_{2,1} + B_{2,1} & A_{2,2} + B_{2,2} & A_{2,3} + B_{2,3} \end{pmatrix}$$

```

```
isequal(Y,Y2)
```

```
ans = logical
      1
```

### Commutation Relation of Symbolic Matrix Variables

Symbolic matrix variables represent matrices, vectors, and scalars in compact matrix notation. When representing nonscalars, these variables are noncommutative. When mathematical formulas involve

matrices and vectors, writing them using symbolic matrix variables is more concise and clear than writing them componentwise.

Create two symbolic matrix variables.

```
syms A B [2 2] matrix
```

Check the commutation relation for multiplication between two symbolic matrix variables.

```
A*B - B*A
```

```
ans = A*B - B*A
```

```
isequal(A*B,B*A)
```

```
ans = logical
      0
```

Check the commutation relation for addition between two symbolic matrix variables.

```
isequal(A+B,B+A)
```

```
ans = logical
      1
```

### Find Hessian Matrix

Create 3-by-3 and 3-by-1 symbolic matrix variables.

```
syms A 3 matrix
syms X [3 1] matrix
```

Find the Hessian matrix of  $X^T A X$ . Derived equations involving symbolic matrix variables are displayed in typeset as they would be in textbooks.

```
f = X.'*A*X
```

```
f =  $X^T A X$ 
```

```
H = diff(f,X,X.')
```

```
H =  $A^T + A$ 
```

### Create Function of Vector and Scalar

Create the function  $v(\mathbf{r}, t) = \frac{\|\mathbf{r}\|_2}{t}$ , where  $\mathbf{r}$  is a 1-by-3 vector and  $t$  is a scalar.

Create  $\mathbf{r}$  as a symbolic matrix variable and  $t$  as a symbolic scalar variable. Create  $v(\mathbf{r}, t)$  as a symbolic matrix function and keep existing definitions of  $\mathbf{r}$  and  $t$  in the workspace.



```
syms r [1 3] matrix
syms t
syms v(r,t) [1 1] matrix keepargs
```

Assign the formula of  $v(\mathbf{r}, t)$ .

```
v(r,t) = norm(r)/t
```

```
v(r, t) = ||r||2 t-1
```

The symbolic matrix function accepts scalars, vectors, and matrices as its input arguments. Evaluate the function for the vector value  $\mathbf{r} = (1, 2, 2)$  and the scalar value  $t = 3$ .

```
vEval = v([1 2 2],3)
```

```
vEval =
    1/3 ||Σ1||2
```

where

```
Σ1 = (1 2 2)
```

Convert the evaluated function from the `symmatrix` data type to the `sym` data type.

```
vSym = symmatrix2sym(vEval)
```

```
vSym = 1
```

### Create Symbolic Objects from Returned Symbolic Array

Certain functions, such as `solve` and `symReadSSCVariables`, can return a vector of symbolic scalar variables or a cell array of symbolic scalar variables and functions. These variables or functions do not automatically appear in the MATLAB workspace. Create these variables or functions from the vector or cell array by using `syms`.

Solve the equation  $\sin(x) == 1$  by using `solve`. The parameter `k` in the solution does not appear in the MATLAB workspace.

```
syms x
eqn = sin(x) == 1;
[sol,parameter,condition] = solve(eqn,x,ReturnConditions=true);
parameter

parameter = k
```

Create the parameter `k` by using `syms`. The parameter `k` now appears in the MATLAB workspace.

```
syms(parameter)
```

Similarly, use `syms` to create the symbolic objects contained in a vector or cell array. Examples of functions that return a cell array of symbolic objects are `symReadSSCVariables` and `symReadSSCParameters`.

### List All Symbolic Objects

Create some symbolic scalar variables, functions, matrix variables, matrix functions, and arrays.

```
syms a f(x)
syms A [2 1]
syms B [2 2] matrix
syms g(B) [2 2] matrix keepargs
```

Display a list of all symbolic scalar variables, functions, matrix variables, matrix functions, and arrays that currently exist in the MATLAB workspace by using `syms`.

```
syms
```

```
Your symbolic variables are:
```

```
A  A1  A2  B   a   f   g   x
```

Instead of displaying a list, return a cell array by providing an output to `syms`.

```
S = syms
```

```
S = 8x1 cell
    {'A' }
    {'A1'}
    {'A2'}
    {'B' }
    {'a' }
    {'f' }
    {'g' }
    {'x' }
```

### Delete All Symbolic Objects

Create several symbolic objects.

```
syms a b c f(x)
syms D g(y) [2 2] matrix
```

Return all symbolic objects as a cell array by using the `syms` function. Use the `cellfun` function to delete all symbolic objects in the cell array `symObj`.

```
symObj = syms;
cellfun(@clear,symObj)
```

Check that you deleted all symbolic objects by calling `syms`. The output is empty, meaning no symbolic objects exist in the MATLAB workspace.

```
syms
```

### Input Arguments

**var1 ... varN** — Symbolic scalar variables, matrices, arrays, or matrix variables

valid variable names separated by spaces

Symbolic scalar variables, matrices, arrays, or matrix variables, specified as valid variable names separated by spaces. That is, each variable name must begin with a letter and can contain only alphanumeric characters and underscores. To verify that a variable name is valid, use `isvarname`.

Example: `x y123 z_1`

### **[n1 ... nM] — Dimensions of vector, matrix, or array of symbolic scalar variables**

vector of nonnegative integers

Dimensions of vector, matrix, or array of symbolic scalar variables, specified as a vector of nonnegative integers.

Example: `[2 3], [2,3]`

### **n — Dimensions of square matrix**

nonnegative scalar integer

Dimensions of the square matrix, specified as a nonnegative scalar integer. For example, `syms x 3 matrix` creates a square 3-by-3 symbolic matrix variable.

Example: `3`

### **set — Assumptions on symbolic scalar variables**

`real | positive | integer | rational`

Assumptions on symbolic scalar variables, specified as `real`, `positive`, `integer`, or `rational`.

You can combine multiple assumptions using spaces. For example, `syms x positive rational` creates a symbolic scalar variable `x` with a positive rational value.

Example: `rational`

### **f(var1, ..., varN) — Symbolic function or matrix function with its input arguments**

function and variable names with parentheses

Symbolic function or matrix function with its input arguments, specified as function and variable names with parentheses. The function name `f` and the variable names `var1...varN` must be valid variable names. That is, they must begin with a letter and can contain only alphanumeric characters and underscores. To verify that a variable name is valid, use `isvarname`.

Example: `F(r,t), f(x,y)`

### **[nrow ncol] — Dimensions of symbolic matrix variables or functions**

vector of nonnegative integers

Dimensions of symbolic matrix variables or functions, specified as a vector of nonnegative integers.

Example: `[2 3], [2,3]`

### **symArray — Symbolic scalar variables and functions to create**

vector of symbolic scalar variables | cell array of symbolic scalar variables and functions

Symbolic scalar variables and functions to create, specified as a vector of symbolic scalar variables or a cell array of symbolic scalar variables and functions. Such a vector or array is typically the output of another function, such as `solve` or `symReadSSCVariables`.

## Output Arguments

**S** — Names of all symbolic scalar variables, functions, matrix variables, matrix functions, and arrays

cell array of character vectors

Names of all symbolic scalar variables, functions, matrix variables, matrix functions, and arrays in the MATLAB workspace, returned as a cell array of character vectors.

## Limitations

- Differentiation functions, such as `divergence`, `curl`, `jacobian`, and `laplacian`, currently do not accept symbolic matrix variables and symbolic matrix functions as input. To evaluate differentiation with respect to vectors and matrices, use the `diff` function instead.
- To show all the functions in Symbolic Math Toolbox that accept symbolic matrix variables and symbolic matrix functions as input, use the command `methods symmatrix` and `methods symfunmatrix`.

## Tips

- You can create multiple symbolic objects in one call. For example, `syms f(x) g(t) y` creates two symbolic functions (`f` and `g`) and three symbolic scalar variables (`x`, `t`, and `y`).
- `syms` is a shortcut for `sym`, `symfun`, `symmatrix`, and `symfunmatrix`. This shortcut lets you create several symbolic objects with one function call. For example, you can use `sym` and create each symbolic scalar variable separately. However, when you create variables using `sym`, any existing assumptions on the created variables are retained. You can also use `symfun` to create symbolic functions.
- In functions and scripts, do not use `syms` to create symbolic scalar variables with the same names as MATLAB functions. For these names, MATLAB does not create symbolic scalar variables, but keeps the names assigned to the MATLAB functions. If you want to create a symbolic scalar variable with the same name as a MATLAB function inside a function or a script, use `sym` instead. For example, use `alpha = sym("alpha")`.
- Avoid using `syms` within functions or parallel for loops because it generates variables without direct output assignment and modifies the global state of the workspace. Using `syms` within functions can create side effects and other unexpected behaviors. Instead, use `sym` with left-side output assignment, such as `t = sym("t")`. For more details, see “Choose `syms` or `sym` Function” on page 2-4.
- These variable names are invalid with `syms`: `integer`, `real`, `rational`, `positive`, and `clear`. To create symbolic scalar variables with these names, use `sym`. For example, `real = sym("real")`.
- `clear x` does not clear the symbolic object of its assumptions, such as `real`, `positive`, or any assumptions set by `assume`, `sym`, or `syms`. To remove assumptions, use one of these options:
  - `syms x` or `syms f(x)` clears all assumptions from `x`.
  - `assume(x, "clear")` clears all assumptions from `x`.
  - `clear all` clears all objects in the MATLAB workspace and resets the symbolic engine.
  - `assume` and `assumeAlso` provide more flexibility for setting assumptions on symbolic scalar variables.

- When you replace one or more elements of a numeric vector or matrix with a symbolic number, MATLAB converts that number to a double-precision number.

```
A = eye(3);
A(1,1) = sym(pi)
```

```
A =
    3.1416         0         0
         0    1.0000         0
         0         0    1.0000
```

You cannot replace elements of a numeric vector or matrix with a symbolic scalar variable, expression, or function because these elements cannot be converted to double-precision numbers. For example, `syms a; A(1,1) = a` throws an error.

- When evaluating a symbolic matrix function, you must substitute values that have the same size as the defined input arguments. For example, see “Create Function of Vector and Scalar” on page 6-1528. For comparison, this example returns an error:

```
syms X [2 2] matrix
syms f(X) [1 1] matrix keepargs
f(ones(4))
```

## Version History

### Introduced before R2006a

#### R2022b: Return the names of all symbolic objects in the MATLAB workspace

*Behavior changed in R2022b*

You can list the names of all symbolic objects in the MATLAB workspace using `syms` and return them as a cell array using `S = syms`.

Starting in R2022b, the returned names of all symbolic objects include all symbolic matrix variables and matrix functions of type `symmatrix` and `symfunmatrix` in the MATLAB workspace. In previous releases, `syms` only returns the names of all symbolic scalar variables, functions, and arrays of type `sym` and `symfun`.

#### R2022a: Create symbolic matrix functions of type `symfunmatrix`

Use these syntaxes to create symbolic matrix functions of type `symfunmatrix`:

```
syms f(var1,...,varN) [nrow ncol] matrix
syms f(var1,...,varN) [nrow ncol] matrix keepargs
syms f(var1,...,varN) n matrix
syms f(var1,...,varN) n matrix keepargs
```

Symbolic matrix functions allow you to represent parameter-dependent functions that operate on scalars, vectors, and matrices in compact matrix notation. Using them, you can represent matrix- and vector-based expressions in Symbolic Math Toolbox and perform linear algebra calculations. For example, see “Create Function of Vector and Scalar” on page 6-1528.

#### R2021b: `syms` clears assumptions on symbolic variables when creating symbolic functions

*Behavior changed in R2021b*

When creating symbolic functions with symbolic variables as input arguments, `syms` clears all previously set assumptions on the symbolic variables. For example:

```
syms x y z real
syms t real positive
assumptions

ans =

[in(x, 'real'), in(y, 'real'), in(z, 'real'), 0 < t]

syms f(x,y,z,t)
assumptions

ans =

Empty sym: 1-by-0
```

To set assumptions on the input arguments of symbolic functions, create the symbolic functions first, and then set the assumptions on the symbolic variables (or recreate the symbolic variables with set assumptions). For example:

```
syms g(x,y,z,t)
assume(x,'real')
assume(t,'positive')
assumptions

ans =

[in(x, 'real'), 0 < t]

syms y z real
assumptions

ans =

[in(x, 'real'), in(y, 'real'), in(z, 'real'), 0 < t]
```

### **R2021a: Create symbolic matrix variables of type `symmatrix`**

Use these syntaxes to create symbolic matrix variables of type `symmatrix`:

```
syms var1 ... varN [nrow ncol] matrix
syms var1 ... varN n matrix
```

Symbolic matrix variables offer a concise typeset display and show mathematical formulas with more clarity. Using them, you can represent matrix- and vector-based expressions in Symbolic Math Toolbox and perform linear algebra calculations. For example, see “Commutation Relation of Symbolic Matrix Variables” on page 6-1527 and “Find Hessian Matrix” on page 6-1528.

### **R2018b: `syms` clears assumptions on symbolic variables**

*Behavior changed in R2018b*

`syms` now clears any assumptions on the symbolic scalar variables that it creates. For example,

```
syms x real
assume(x <= 5);
assumptions(x)
```

```
ans =

x <= 5

syms x
assumptions(x)

ans =

Empty sym: 1-by-0
```

To retain existing assumptions on a cleared variable, recreate it using `sym` instead of `syms`. For example,

```
syms x real
assume(x <= 5);
clear x
x = sym('x');
assumptions(x)

ans =

x <= 5
```

### **R2018b: syms will no longer support clear option**

*Warns starting in R2018b*

The syntaxes `syms x clear` and the equivalent `syms('x', 'clear')` now issue a warning that they will be removed in a future release.

In previous releases, both syntaxes cleared all assumptions applied to `x`. To update your code, call `syms` and specify the symbolic scalar variables whose assumptions you want to clear. For example, `syms x` clears all assumptions applied to `x`.

## **See Also**

`assume` | `assumeAlso` | `assumptions` | `displayFormula` | `sym` | `symfun` | `symmatrix` | `symfunmatrix` | `symvar`

## **Topics**

“Create Symbolic Numbers, Variables, and Expressions” on page 1-3

“Create Symbolic Functions” on page 1-9

“Create Symbolic Matrices” on page 1-11

“Create Symbolic Matrix Variables” on page 1-13

“Use Symbolic Objects to Represent Mathematical Objects” on page 1-20

“Choose `syms` or `sym` Function” on page 2-4

“Use Assumptions on Symbolic Variables” on page 1-41

“Add Subscripts, Superscripts, and Accents to Symbolic Variables in the Live Editor” on page 2-14

“Change Output Format of Symbolic and Variable-Precision Arithmetic” on page 2-8

## symsum

Symbolic sum of series

### Syntax

```
F = symsum(f,k,a,b)
F = symsum(f,k)
```

### Description

`F = symsum(f,k,a,b)` returns the symbolic definite sum on page 6-1539 of the series `f` with respect to the summation index `k` from the lower bound `a` to the upper bound `b`. If you do not specify `k`, `symsum` uses the variable determined by `symvar` as the summation index. If `f` is a constant, then the default variable is `x`.

`symsum(f,k,[a b])` or `symsum(f,k,[a; b])` is equivalent to `symsum(f,k,a,b)`.

`F = symsum(f,k)` returns the indefinite sum on page 6-1539 (antidifference) of the series `f` with respect to the summation index `k`. The `f` argument defines the series such that the indefinite sum `F` satisfies the relation  $F(k+1) - F(k) = f(k)$ . If you do not specify `k`, `symsum` uses the variable determined by `symvar` as the summation index. If `f` is a constant, then the default variable is `x`.

### Examples

#### Find Sum of Sequence of Numbers

Find the sum of the integer numbers  $1 + 2 + 3 + \dots + n = \sum_{k=1}^n k$ .

```
syms k n
F1 = symsum(k,k,1,n)
```

$$F1 = \frac{n(n+1)}{2}$$

Find the sum of the square numbers  $1^2 + 2^2 + 3^2 + \dots + n^2 = \sum_{k=1}^n k^2$ .

```
syms k n
F2 = symsum(k^2,k,1,n)
```

$$F2 = \frac{n(2n+1)(n+1)}{6}$$

Find the sum of the cubic numbers  $1^3 + 2^3 + 3^3 + \dots + n^3 = \sum_{k=1}^n k^3$ .

```
syms k n
F3 = symsum(k^3,k,1,n)
```

F3 =



$$\frac{n^2 (n+1)^2}{4}$$

### Find Sum of Series with Bounds

Find the following sums of series.

$$F1 = \sum_{k=0}^{10} k^2$$

$$F2 = \sum_{k=1}^{\infty} \frac{1}{k^2}$$

$$F3 = \sum_{k=1}^{\infty} \frac{x^k}{k!}$$

syms  $x$

```
F1 = symsum(k^2,k,0,10)
```

```
F1 = 385
```

```
F2 = symsum(1/k^2,k,1,Inf)
```

```
F2 =
```

$$\frac{\pi^2}{6}$$

```
F3 = symsum(x^k/factorial(k),k,1,Inf)
```

```
F3 = e^x - 1
```

Alternatively, you can specify summation bounds as a row or column vector.

```
F1 = symsum(k^2,k,[0 10])
```

```
F1 = 385
```

```
F2 = symsum(1/k^2,k,[1;Inf])
```

```
F2 =
```

$$\frac{\pi^2}{6}$$

```
F3 = symsum(x^k/factorial(k),k,[1 Inf])
```

```
F3 = e^x - 1
```

### Find Indefinite Sum of Series

Find the following indefinite sums of series (antidifferences).

$$F1 = \sum_k k$$

$$F2 = \sum_k 2^k$$

$$F3 = \sum_k \frac{1}{k^2}$$

```
syms k
F1 = symsum(k,k)
```

$$F1 = \frac{k^2}{2} - \frac{k}{2}$$

```
F2 = symsum(2^k,k)
```

$$F2 = 2^k$$

```
F3 = symsum(1/k^2,k)
```

$$F3 = \begin{cases} -\psi'(k) & \text{if } 0 < k \\ \psi'(1-k) & \text{if } k \leq 0 \end{cases}$$

### Summation of Polynomial Series

Find the summation of the polynomial series  $F(x) = \sum_{k=1}^8 a_k x^k$ .

If you know that the coefficient  $a_k$  is a function of some integer variable  $k$ , use the `symsum` function.

For example, find the sum  $F(x) = \sum_{k=1}^8 kx^k$ .

```
syms x k
F(x) = symsum(k*x^k,k,1,8)
```

$$F(x) = 8x^8 + 7x^7 + 6x^6 + 5x^5 + 4x^4 + 3x^3 + 2x^2 + x$$

Calculate the summation series for  $x = 2$ .

```
F(2)
```

```
ans = 3586
```

Alternatively, if you know that the coefficients  $a_k$  are a vector of values, you can use the `sum` function.

For example, the coefficients are  $a_1, \dots, a_8 = 1, \dots, 8$ . Declare the term  $x^k$  as a vector by using `subs(x^k,k,1:8)`.

```
a = 1:8;
G(x) = sum(a.*subs(x^k,k,1:8))
```

$$G(x) = 8x^8 + 7x^7 + 6x^6 + 5x^5 + 4x^4 + 3x^3 + 2x^2 + x$$

Calculate the summation series for  $x = 2$ .

$G(2)$

ans = 3586

## Input Arguments

### **f** — Expression defining terms of series

symbolic expression | symbolic function | symbolic vector | symbolic matrix | symbolic number

Expression defining terms of series, specified as a symbolic expression, function, vector, matrix, or symbolic number.

### **k** — Summation index

symbolic variable

Summation index, specified as a symbolic variable. If you do not specify this variable, `symsum` uses the default variable determined by `symvar(expr,1)`. If `f` is a constant, then the default variable is `x`.

### **a** — Lower bound of summation index

number | symbolic number | symbolic variable | symbolic expression | symbolic function

Lower bound of the summation index, specified as a number, symbolic number, variable, expression, or function (including expressions and functions with infinities).

### **b** — Upper bound of summation index

number | symbolic number | symbolic variable | symbolic expression | symbolic function

Upper bound of the summation index, specified as a number, symbolic number, variable, expression, or function (including expressions and functions with infinities).

## More About

### Definite Sum

The definite sum of a series is defined as

$$\sum_{k=a}^b f_k = f_a + f_{a+1} + \dots + f_b.$$

### Indefinite Sum

The indefinite sum (antidifference) of a series is defined as

$$F(x) = \sum_x f(x),$$

such that

$$F(x+1) - F(x) = f(x).$$

## **Version History**

**Introduced before R2006a**

### **See Also**

`cumsum` | `int` | `sum` | `symprod` | `syms` | `symvar` | `rewrite`

### **Topics**

“Symbolic Summation” on page 3-210

# symtrue

Symbolic logical constant true

## Syntax

```
symtrue
T = symtrue(n)
T = symtrue(sz)
T = symtrue(sz1,...,szN)
```

## Description

`symtrue` is the symbolic logical constant for the true condition.

`T = symtrue(n)` returns an  $n$ -by- $n$  matrix of symbolic logical `symtrue`s.

`T = symtrue(sz)` returns an array of symbolic logical `symtrue`s where the size vector, `sz`, defines `size(T)`. For example, `symtrue([2 3])` returns a 2-by-3 array of symbolic logical `symtrue`s.

`T = symtrue(sz1,...,szN)` returns a  $sz1$ -by-...-by- $szN$  array of symbolic logical `symtrue`s where  $sz1, \dots, szN$  indicates the size of each dimension. For example, `symtrue(2,3)` returns a 2-by-3 array of symbolic logical `symtrue`s.

## Examples

### Simplify Symbolic Inequality

Create a symbolic inequality  $x^2 > 4$ .

```
syms x
eq = x^2 > 4
```

```
eq = 4 < x^2
```

Assume that  $x > 2$ .

```
assume(x>2)
```

Simplify the condition represented by the symbolic inequality `eq`. The `simplify` function returns the symbolic logical constant `symtrue` since the condition always holds for the assumption  $x > 2$ .

```
T = simplify(eq)
```

```
T = symtrue
```

Display the data type of `T`, which is `sym`.

```
class(T)
```

```
ans =
'sym'
```

You can also use `isAlways` to check if the inequality holds under the assumption being made. In this example, `isAlways` returns logical 1 (`true`).

```
TF = isAlways(eq)
```

```
TF = logical  
    1
```

### Generate Array of Symbolic Logical `symtrue`s

Use `symtrue` to generate a 3-by-3 square matrix of symbolic logical `symtrue`s.

```
T = symtrue(3)
```

```
T =  
    (symtrue symtrue symtrue)  
    (symtrue symtrue symtrue)  
    (symtrue symtrue symtrue)
```

Display the data type of `T`, which is `sym`.

```
class(T)
```

```
ans =  
'sym'
```

Next, use `symtrue` to generate a 3-by-2-by-2 array of symbolic logical `symtrue`'s.

```
T = symtrue(3,2,2)
```

```
T(:,:,1) =  
    (symtrue symtrue)  
    (symtrue symtrue)  
    (symtrue symtrue)
```

```
T(:,:,2) =  
    (symtrue symtrue)  
    (symtrue symtrue)  
    (symtrue symtrue)
```

Alternatively, you can use a size vector to specify the size of the array.

```
T = symtrue([3,2,2])
```

```
T(:,:,1) =  
    (symtrue symtrue)  
    (symtrue symtrue)  
    (symtrue symtrue)
```

```
T(:,:,2) =  
    (symtrue symtrue)  
    (symtrue symtrue)  
    (symtrue symtrue)
```

### Create Truth Table for Logical Operation

Create a truth table for the and operation applied to the two symbolic logical constants, `symtrue` and `symfalse`.

```
A = [symtrue symfalse]
```

```
A = (symtrue symfalse)
```

```
B = [symtrue; symfalse]
```

```
B =  
    (symtrue  
    symfalse)
```

```
TF = and(A,B)
```

```
TF =  
    (symtrue symfalse  
    symfalse symfalse)
```

### Combine Symbolic Logical Constants

Combine symbolic logical constants with logical operators `and`, `not`, `or`, and `xor` (or their shortcuts).

```
TF = xor(symtrue,or(symfalse,symfalse))
```

```
TF = symtrue
```

```
TF = symtrue & ~(symfalse)
```

```
TF = symtrue
```

### Convert `symtrue` to Other Data Types

Convert the symbolic logical constant `symtrue` to a logical value.

```
T1 = logical(symtrue)
```

```
T1 = logical  
    1
```

Convert the symbolic logical constant `symtrue` to numeric values in double precision and variable precision.

```
T2 = double(symtrue)
```

```
T2 =  
    1
```

```
T3 = vpa(symtrue)
```

```
T3 = 1.0
```

Show the data types of T1, T2, and T3.

```
whos
```

| Name | Size | Bytes | Class   | Attributes |
|------|------|-------|---------|------------|
| T1   | 1x1  | 1     | logical |            |
| T2   | 1x1  | 8     | double  |            |
| T3   | 1x1  | 8     | sym     |            |

## Input Arguments

### **n — Size of square matrix**

scalar

Size of square matrix, specified as an integer. `n` sets the output array size to `n`-by-`n`. For example, `symtrue(3)` returns a 3-by-3 array of symbolic logical `symtrue`s.

- If `n` is 0, then `T` is an empty matrix.
- If `n` is negative, then it is treated as 0.

### **sz — Size vector**

row vector of integers

Size vector, specified as a row vector of integers. For example, `symtrue([2 3])` returns a 2-by-3 array of symbolic logical `symtrue`s.

- If the size of any dimension is 0, then `T` is an empty array.
- If the size of any dimension is negative, then it is treated as 0.
- If any trailing dimensions greater than 2 have a size of 1, then the output `T` does not include those dimensions. For example, `symtrue([2 2 1 1])` returns a 2-by-2 array and `symtrue([2 2 1 2 1])` returns a 2-by-2-by-1-by-2 array.

### **sz1, ..., szN — Size inputs**

comma-separated list of integers

Size inputs, specified by a comma-separated list of integers. For example, `symtrue(2,3)` returns a 2-by-3 array of symbolic logical `symtrue`s.

- If the size of any dimension is 0, then `T` is an empty array.
- If the size of any dimension is negative, then it is treated as 0.
- If any trailing dimensions greater than 2 have a size of 1, then the output `T` does not include those dimensions. For example, `symtrue([2,2,1,1])` returns a 2-by-2 array and `symtrue([2,2,1,2,1])` returns a 2-by-2-by-1-by-2 array.

## Output Arguments

### **T — Symbolic logical constant for true condition**

scalar | vector | matrix | *N*-D array



Symbolic logical constant for true condition, returned as a scalar, vector, matrix, or *N*-D array.

Data Types: sym

## Tips

- The command `sym(true)` returns a symbolic number 1, and `sym(symtrue)` returns `symtrue`.
- When you combine two arrays of symbolic logical constants with logical operations using `and`, `or`, or `xor` function, the arrays must either be the same size or have sizes that are compatible. For more information on the required input sizes for basic array operations, see “Compatible Array Sizes for Basic Operations”.

## Version History

Introduced in R2020a

## See Also

`isAlways` | `symfalse` | `and` | `or` | `not` | `xor`

## symType

Determine type of symbolic object

### Syntax

```
s = symType(symObj)
```

### Description

`s = symType(symObj)` returns the type of a symbolic object. For example, `symType(sym('x'))` returns "variable".

### Examples

#### Symbolic Number

Create a symbolic number and determine its type.

```
a = sym('3/9');  
s = symType(a)
```

```
s =  
"rational"
```

Now construct a symbolic array by including symbolic numbers in the array elements. Determine the symbolic type of each array element.

```
B = [-5, a, vpa(a), 1i, pi];  
s = symType(B)
```

```
s = 1x5 string      "integer"      "rational"      "vpareal"      "complex"      "constant"
```

#### Symbolic Function or Expression

Create a symbolic function  $f(x)$  using `syms`.

```
syms f(x)
```

Determine the type of the function. Because  $f(x)$  is an unassigned symbolic function, it has the symbolic type "symfun".

```
s = symType(f)
```

```
s =  
"symfun"
```

Assigning a mathematical expression to  $f(x)$  changes its symbolic type.

```
f(x) = x^2;
s = symType(f)
```

```
s =
"expression"
```

Now check the symbolic type of  $f(x) = x$  and its derivative.

```
f(x) = x;
s = symType(f)
```

```
s =
"variable"
```

```
s = symType(diff(f))
```

```
s =
"integer"
```

## Inequalities and Solutions

Determine the type of various symbolic objects when solving for inequalities.

Create a quadratic function.

```
syms y(x)
y(x) = 100 - 5*x^2
```

```
y(x) = 100 - 5x^2
```

Set two inequalities to the quadratic function. Check the symbolic type of each inequality.

```
eq1 = y(x) > 10;
eq2 = x > 2;
s = symType([eq1 eq2])
```

```
s = 1x2 string
    "equation"    "equation"
```

Solve the inequalities using `solve`. Return the solutions by setting 'ReturnConditions' to true.

```
eqSol = solve([eq1 eq2], 'ReturnConditions', true);
sols = eqSol.conditions
```

```
sols = x < sqrt(18) ^ 2 < x
```

Determine the symbolic type of the solutions.

```
s = symType(sols)
```

```
s =
"logicalexpression"
```

## Input Arguments

### **symObj** — Symbolic objects

symbolic numbers | symbolic variables | symbolic expressions | symbolic functions | symbolic units

Symbolic objects, specified as symbolic numbers, symbolic variables, symbolic expressions, symbolic functions, or symbolic units.

## Output Arguments

### **s** — Symbolic types

string array

Symbolic types, returned as a string array. This table shows output values for various symbolic objects.

| Output               | Description                                            | Input Example                                         |
|----------------------|--------------------------------------------------------|-------------------------------------------------------|
| "integer"            | symbolic integer number                                | <code>symType(sym(' -1 '))</code>                     |
| "rational"           | symbolic rational number                               | <code>symType(sym(' 1/2 '))</code>                    |
| "vpareal"            | symbolic variable-precision floating-point real number | <code>symType([sym(' 1.5 ')<br/>vpa(' 3/2 ')])</code> |
| "complex"            | symbolic complex number                                | <code>symType(sym(' 1+2i '))</code>                   |
| "constant"           | symbolic mathematical constant                         | <code>symType(sym([pi<br/>catalan]))</code>           |
| "variable"           | symbolic variable                                      | <code>syms x; symType(x)</code>                       |
| "symfun"             | unassigned symbolic function                           | <code>syms f(x); symType(f)</code>                    |
| "expression"         | symbolic expression                                    | <code>syms x; symType(sqrt(x))</code>                 |
| "equation"           | symbolic equation and inequality                       | <code>syms x; symType(x&gt;=0)</code>                 |
| "unit"               | symbolic unit                                          | <code>symType(symunit('meter'))</code>                |
| "logical expression" | symbolic logical expression                            | <code>syms x y; symType(x y)</code>                   |
| "logical constant"   | symbolic logical constant                              | <code>symType([symtrue<br/>symfalse])</code>          |
| "unsupported"        | symbolic object not supported by symType               |                                                       |

## Version History

Introduced in R2019a

### See Also

`sym` | `syms` | `symfun` | `isSymType` | `symFunType` | `hasSymType`

# symunit

Units of measurement

## Syntax

`u = symunit`

## Description

`u = symunit` returns the units collection. Then, specify any unit by using `u.unit`. For example, specify 3 meters as `3*u.m`. Common alternate names for units are supported, such as `u.meter` and `u.metre`. Plurals are not supported. For details about the list of available units, see “Units and Unit Systems List” on page 2-60.

## Examples

### Specify Units of Measurement

Before specifying units, load units by using `symunit`. Then, specify a unit by using dot notation.

Specify a length of 3 meters. You can also use aliases `u.meter` or `u.metre`.

```
u = symunit;
length = 3*u.m
```

```
length = 3 m
```

Specify the acceleration due to gravity of 9.81 meters per second squared. Because units are symbolic expressions, numeric inputs are converted to exact symbolic values. Here, 9.81 is converted to  $\frac{981}{100}$ .

```
g = 9.81*u.m/u.s^2
```

```
g =
     $\frac{981}{100} \frac{\text{m}}{\text{s}^2}$ 
```

For more information about the differences between symbolic and numeric arithmetic, see “Choose Numeric or Symbolic Arithmetic” on page 2-32.

### Operations on Units and Conversion to Double

Units behave like symbolic expressions when you perform standard operations on them. For numeric operations, separate the value from the units, substitute for any symbolic parameters, and convert the result to double.

Find the speed required to travel 5 km in 2 hours.

```
u = symunit;  
d = 5*u.km;  
t = 2*u.hr;  
s = d/t
```

```
s =  
    5 km  
    2 h
```

The value 5/2 is symbolic. You may prefer double output, or require double output for a MATLAB® function that does not accept symbolic values. Convert to double by separating the numeric value using `separateUnits` and then using `double`.

```
[sNum,sUnits] = separateUnits(s)
```

```
sNum =  
    5  
    2
```

```
sUnits =  
    km  
    h
```

```
sNum = double(sNum)
```

```
sNum =  
2.5000
```

For the complete units workflow, see “Units of Measurement Tutorial” on page 2-47.

### Convert Between Units

Use your preferred unit by rewriting units using `unitConvert`. Also, instead of specifying specific units, you can specify that the output should be in terms of SI units.

Calculate the force required to accelerate 2 kg by 5 m/s². The expression is not automatically rewritten in terms of Newtons.

```
u = symunit;  
m = 2*u.kg;  
a = 5*u.m/u.s^2;  
F = m*a
```

```
F =  
    10 kg m  
       s²
```

Convert the expression to newtons by using `unitConvert`.

```
F = unitConvert(F,u.N)
```

```
F = 10 N
```

Convert 5 cm to inches.

```
length = 5*u.cm;  
length = unitConvert(length,u.in)
```

```
length =
     $\frac{250}{127}$  in
```

Convert `length` to SI units. The result is in meters.

```
length = unitConvert(length, 'SI')
```

```
length =
     $\frac{1}{20}$  m
```

### Simplify Units of Same Dimension

Simplify expressions containing units of the same dimension by using `simplify`. Units are not automatically simplified or checked for consistency unless you call `simplify`.

```
u = symunit;
expr = 300*u.cm + 40*u.inch + 2*u.m
```

```
expr = 300 cm + 40 in + 2 m
```

```
expr = simplify(expr)
```

```
expr =
     $\frac{752}{125}$  m
```

`simplify` automatically chooses the unit to return. To convert to a specific unit, use `unitConvert`.

```
expr = unitConvert(expr, u.ft)
```

```
expr =
     $\frac{7520}{381}$  ft
```

### Convert Absolute Temperatures and Temperature Differences

By default, temperatures are assumed to represent temperature differences. For example, `5*u.Celsius` represents a temperature difference of 5 degrees Celsius. This assumption allows arithmetic operations on temperature values and conversion between temperature scales.

To represent absolute temperatures, use `kelvin` so that you do not have to distinguish an absolute temperature from a temperature difference.

Convert 23 degrees Celsius to K, treating the temperature first as a temperature difference and then as an absolute temperature.

```
u = symunit;
T = 23*u.Celsius;
diffK = unitConvert(T, u.K)
```

```
diffK = 23 K
```

```
absK = unitConvert(T,u.K,Temperature="absolute")
```

```
absK =  
    5923  
    20 K
```

## Limitations

- When using symbolic units, the value of 0 times a symbolic unit is returned as a dimensionless 0. To preserve the unit when multiplying a symbolic unit by 0, use a cell array to represent the zero measurement.

For example, you can define 0 degrees Celsius as a cell array and convert it to degrees Fahrenheit by using the `unitConvert` function.

```
u = symunit;  
tC = {0,u.Celsius};  
tF = unitConvert(tC,u.Fahrenheit,'Temperature','Absolute')  
  
tF =  
32*[Fahrenheit]
```

## Tips

- You can use tab expansion to find names of units. Type `u.`, press **Tab**, and continue typing.
- 1 represents a dimensionless unit. Hence, `isUnit(sym(1))` returns logical 1 (true).
- Certain non-linear units, such as decibels, are not implemented because arithmetic operations are not possible for these units.
- Instead of using dot notation to specify units, you can alternatively use string input for `symunit(unit)`. For example, `symunit("m")` specifies the unit meter.

## Version History

**Introduced in R2017a**

### **R2022b: New physical constants added to symbolic units**

`symunit` includes these new physical constants:

- `R_K` - von Klitzing constant
- `alpha` - fine-structure constant
- `g_e` - electron g-factor
- `g_p` - proton g-factor
- `gamma_e` - electron gyromagnetic ratio
- `gamma_p` - proton gyromagnetic ratio

For more information, see “Units and Unit Systems List” on page 2-60.

### **R2020a: New physical constants added to symbolic units**



New physical constants added to `symunit`:

- `G_0` - conductance
- `e_0` - vacuum electric permittivity or electric constant
- `F_c` - Faraday constant
- `mu_0` - vacuum magnetic permeability
- `sigma` - Stefan-Boltzmann constant
- `phi_0` - magnetic flux quantum
- `R_c` - molar gas constant
- `m_p` - proton mass
- `R_inf` - Rydberg constant

For more information, see “Units and Unit Systems List” on page 2-60.

## See Also

`checkUnits` | `isUnit` | `newUnit` | `separateUnits` | `symunit2str` | `unitConversionFactor` | `unitConvert` | `unitInfo`

## Topics

“Units of Measurement Tutorial” on page 2-47

“Unit Conversions and Unit Systems” on page 2-53

“Units and Unit Systems List” on page 2-60

## External Websites

The International System of Units (SI)

## symunit2str

Convert unit to character vector

### Syntax

```
symunit2str(unit)
symunit2str(unit,toolbox)
```

### Description

`symunit2str(unit)` converts the symbolic unit `unit` to a character vector.

`symunit2str(unit,toolbox)` converts the symbolic unit `unit` to a character vector representing units in the toolbox `toolbox`. The allowed values of `toolbox` are 'Aerospace', 'SimBiology', 'Simscape', or 'Simulink'.

### Examples

#### Convert Unit to Character Vector

Convert the symbolic unit `u.km` to a character vector, where `u = symunit`.

```
u = symunit;
unitStr = symunit2str(u.km)

unitStr =
    'km'
```

#### Convert Unit for Specified Toolbox

Convert symbolic units to character vectors representing units in other toolboxes by specifying the toolbox name as the second argument to `symunit2str`. The allowed toolboxes are 'Aerospace', 'SimBiology', 'Simscape', or 'Simulink'. The unit must exist in the target toolbox to be valid.

Where `u = symunit`, convert `u.km/(u.hour*u.s)` to a character vector representing units from Aerospace Toolbox.

```
u = symunit;
unit = symunit2str(u.km/(u.hour*u.s), 'Aerospace')

unit =
    'km/h-s'
```

Convert `u.molecule/u.s` to a character vector representing units from SimBiology.

```
unit = symunit2str(u.molecule/u.s, 'SimBiology')

unit =
    'molecule/second'
```

Convert `u.gn/u.km` to a character vector representing units from Simscape.

```
unit = symunit2str(u.gn/u.km, 'Simscape')
```

```
unit =  
    'gee/km'
```

Convert `u.rad/u.s` to a character vector representing units from Simulink.

```
unit = symunit2str(u.rad/u.s, 'Simulink')  
  
unit =  
    'rad/s'
```

## Input Arguments

### **unit** — Symbolic unit to convert

symbolic unit

Symbolic unit to convert, specified as a symbolic unit.

### **toolbox** — Toolbox to represent unit in

'Aerospace' | 'SimBiology' | 'Simscape' | 'Simulink'

Toolbox to represent unit in, specified as 'Aerospace', 'SimBiology', 'Simscape', or 'Simulink'.

Example: `symunit2str(u.km/u.h, 'Aerospace')`

## Version History

**Introduced in R2017a**

### **See Also**

`checkUnits` | `findUnits` | `isUnit` | `newUnit` | `separateUnits` | `symunit` | `str2symunit` | `unitConversionFactor`

### **Topics**

“Units of Measurement Tutorial” on page 2-47  
 “Unit Conversions and Unit Systems” on page 2-53  
 “Units and Unit Systems List” on page 2-60

### **External Websites**

The International System of Units (SI)

## symvar

Find symbolic variables in symbolic input

### Syntax

```
symvar(s)
symvar(s,n)
```

### Description

`symvar(s)` returns a vector of all symbolic variables in `s`. The variables are in alphabetical order with uppercase letters preceding lowercase letters.

`symvar(s,n)` chooses the `n` symbolic variables in `s` that are alphabetically closest to `x` and returns them in alphabetical order. If `s` is a symbolic function, `symvar(s,n)` returns the input arguments of `s` before other variables in `s`.

### Examples

#### Find Symbolic Variables in Expression

Find all symbolic variables in an expression. `symvar` returns the variables in alphabetical order.

```
syms wa wb yx ya
sum = wa + wb + ya + yx;
symvar(sum)
```

```
ans =
[ wa, wb, ya, yx]
```

Find the first three symbolic variables in an expression. `symvar` chooses variables that are alphabetically closest to `x` and returns them in alphabetical order.

```
syms a x y b
f = a*x^2/(sin(3*y-b));
symvar(f,3)
```

```
ans =
[ b, x, y]
```

#### Find Symbolic Variables in Function

Find all symbolic variables in this function. For a symbolic function, `symvar` returns the function inputs before other variables.

```
syms x y a b
f(x,y) = a*x^2/(sin(3*y-b));
symvar(f)
```

```
ans =
[ x, y, a, b]
```

```
g(x,y) = 1;
symvar(g)
```

```
ans =
[ x, y]
```

Find the first three symbolic variables in `f`.

```
symvar(f,3)
```

```
ans =
[ x, y, b]
```

### Find Default Variable of Expression

When a symbolic function, such as `solve`, needs to find the default independent variable in symbolic, the function uses `symvar`. Find the default independent variable for symbolic expressions.

```
syms v z
g = v + z;
symvar(g,1)
```

```
ans =
z
```

```
syms aaa aab
g = aaa + aab;
symvar(g,1)
```

```
ans =
aaa
```

```
syms X1 x2 xa xb
g = X1 + x2 + xa + xb;
symvar(g,1)
```

```
ans =
x2
```

When differentiating, integrating, substituting, or solving equations, MATLAB uses the variable returned by `symvar(s,1)` as a default variable. For a symbolic expression or matrix, `symvar(s,1)` returns the variable closest to `x`. For a function, `symvar(s,1)` returns the first input argument of `s`.

## Input Arguments

### **s** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

### **n** — Number of variables

integer

Number of variables, specified as an integer or `Inf`. If `n` exceeds the number of variables in `s`, then `symvar` returns the number of variables in `s`.

### **Tips**

- `symvar` treats the constants `pi`, `i`, and `j` as variables.
- If there are no symbolic variables in `s`, `symvar` returns the empty vector.

### **Algorithms**

When `symvar` sorts the symbolic variables alphabetically, all uppercase letters have precedence over lowercase: 0 1 ... 9 A B ... Z a b ... z.

## **Version History**

**Introduced in R2008b**

### **See Also**

`argnames` | `sym` | `symfun` | `syms`

### **Topics**

“Find Symbolic Variables in Expressions, Functions, and Matrices” on page 2-2

# symWriteSSC

Create new Simscape component

## Syntax

```
symWriteSSC(newComponentName,templateComponentName,eqns)
symWriteSSC(newComponentName,templateComponentName,eqns,Name,Value)
```

## Description

`symWriteSSC(newComponentName,templateComponentName,eqns)` creates a new Simscape component `newComponentName` using an existing component `templateComponentName` as a template and adding `eqns`. Thus, the new component has both the existing equations taken from the template component and the added equations.

`symWriteSSC(newComponentName,templateComponentName,eqns,Name,Value)` uses additional options specified by one or more `Name,Value` pair arguments.

## Examples

### Create Component with Additional Equation

Create a new Simscape component by using an existing component as a template and adding an equation.

Suppose you have the Simscape component `spring.ssc` in your current folder.

```
type('spring.ssc');

component spring < foundation.mechanical.rotational.branch

parameters
    spr_rate = { 10, 'N*m/rad' };
end

variables
    phi = { value = { 0, 'rad'}, priority = priority.high };
end

function setup
    if spr_rate <= 0
        pm_error('simscape:GreaterThanZero','Spring rate' )
    end
end

equations
    w == phi.der;
    t == spr_rate*phi;
end

end
```

Create symbolic variables with names of the parameters and variables of the component you are going to use when creating new equations. Also create a symbolic variable, `u`, to denote the energy of the rotational spring.

```
syms spr_rate phi u
```

Create the equation defining the energy `u`.

```
eq = u == spr_rate*phi^2/2;
```

Create the new component, `myRotationalSpring.ssc`, that is a copy of the component `spring.ssc` with an additional equation defining the energy of the rotational spring.

```
symWriteSSC('myRotationalSpring.ssc','spring.ssc',eq)
```

```
Warning: Equations contain undeclared variables 'u'.  
> In symWriteSSC (line 94)
```

`symWriteSSC` creates the component `myRotationalSpring.ssc`.

```
type('myRotationalSpring.ssc');  
  
component myRotationalSpring  
  
parameters  
    spr_rate = { 10, 'N*m/rad' };  
end  
  
variables  
    phi = { value = { 0, 'rad'}, priority = priority.high };  
end  
  
function setup  
    if spr_rate <= 0  
        pm_error('simscape:GreaterThanZero','Spring rate' )  
    end  
end  
  
equations  
    w == phi.der;  
    t == spr_rate*phi;  
    u == phi^2*spr_rate*(1.0/2.0);  
end  
  
end
```

### Add Component Title and Description

Create a Simscape component with the title and descriptive text different from those of the template component.

Suppose you have the Simscape component `spring.ssc` in your current folder. This component does not have any title or descriptive text.

```
type('spring.ssc');
```



```

component spring < foundation.mechanical.rotational.branch

parameters
    spr_rate = { 10, 'N*m/rad' };
end

variables
    phi = { value = { 0, 'rad'}, priority = priority.high };
end

function setup
    if spr_rate <= 0
        pm_error('simscape:GreaterThanZero','Spring rate' )
    end
end

equations
    w == phi.der;
    t == spr_rate*phi;
end

end

```

Create symbolic variables with names of the parameters and variables of the component you are going to use when creating new equations. Also create a symbolic variable, *u*, to denote the energy of the rotational spring.

```
syms spr_rate phi u
```

Create the equation defining the energy *u*.

```
eq = u == spr_rate*phi^2/2;
```

Create the new component, `myRotationalSpring.ssc`, based on the `spring.ssc` component. Add the equation `eq`, the title “Rotational Spring”, and a few lines of descriptive text to the new component.

```

symWriteSSC('myRotationalSpring.ssc','spring.ssc',eq,...
'H1Header','% Rotational Spring',...
'HelpText',{ '% The block represents an ideal mechanical rotational linear spring.',...
              '% Connections R and C are mechanical rotational conserving ports.'...
              '% The block positive direction is from port R to port C. This means'...
              '% that the torque is positive if it acts in the direction from R to C.'})

```

```

Warning: Equations contain undeclared variables 'u'.
> In symWriteSSC (line 94)

```

`symWriteSSC` creates the component `myRotationalSpring.ssc`.

```

type('myRotationalSpring.ssc');

component myRotationalSpring
% Rotational Spring
% The block represents an ideal mechanical rotational linear spring.
% Connections R and C are mechanical rotational conserving ports.
% The block positive direction is from port R to port C. This means
% that the torque is positive if it acts in the direction from R to C.

```

```
parameters
    spr_rate = { 10, 'N*m/rad' };
end

variables
    phi = { value = { 0, 'rad'}, priority = priority.high };
end

function setup
    if spr_rate <= 0
        pm_error('simscape:GreaterThanZero','Spring rate' )
    end
end

equations
    w == phi.der;
    t == spr_rate*phi;
    u == phi^2*spr_rate*(1.0/2.0);
end

end
```

## Input Arguments

### **newComponentName** — Name of Simscape component to create

file name enclosed in single quotes

Name of Simscape component to create, specified as a file name enclosed in single quotes. File must have the extension `.ssc`. If you do not provide file extension, `symWriteSSC` assumes it to be `.ssc`. If you do not specify the absolute path, `symWriteSSC` creates the new component in the current folder.

Example: `'MyNewComponent.ssc'`

### **templateComponentName** — Name of template Simscape component

file name enclosed in single quotes

Name of template Simscape component, specified as a file name enclosed in single quotes. File must have the extension `.ssc`. If you do not provide the file extension, `symWriteSSC` assumes it to be `.ssc`. The component must be on the MATLAB path or in the current folder.

Example: `'TemplateComponent.ssc'`

### **eqns** — Symbolic equations

row vector

Symbolic equations, specified as a row vector.

Example: `[ y(t) == diff(x(t), t), m*diff(y(t), t, t) + b*y(t) + k*x(t) == F]`

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `symWriteSSC('myComp.ssc','template.ssc',eq,'H1Header','% New title','HelpText',{'% Description of the','% new component'})`

### **H1Header — Title**

row vector of characters

Title specified as a row vector of characters (type `char`) starting with `%`. If the first character is not `%`, then `symWriteSSC` adds `%`.

If the template component has a title and you use `H1Header`, the new component will have the title specified by `H1Header`. If the template component has a title and you call `symWriteSSC` without `H1Header`, the new component will have the same title as the template component.

Example: `'H1Header','% New title'`

### **HelpText — Descriptive text**

cell array of row vectors of characters

Descriptive text, specified as a cell array of row vectors of characters. Each row vector must start with `%`. If the first character is not `%`, then `symWriteSSC` adds `%`.

If the template component has descriptive text and you use `HelpText`, the new component will have only the text specified by `HelpText`. In this case, `symWriteSSC` does not copy the descriptive text of the template component to the new component. If the template component has a title and you call `symWriteSSC` without `HelpText`, the new component will have the same descriptive text as the template component.

Example: `'HelpText',{'% Description of the','% new component'}`

## **Version History**

Introduced in R2016a

### **See Also**

`symReadSSCParameters` | `symReadSSCVariables` | `simscapeEquation`

## tan

Symbolic tangent function

### Syntax

`tan(X)`

### Description

`tan(X)` returns the tangent function on page 6-1566 of X.

### Examples

#### Tangent Function for Numeric and Symbolic Arguments

Depending on its arguments, `tan` returns floating-point or exact symbolic results.

Compute the tangent function for these numbers. Because these numbers are not symbolic objects, `tan` returns floating-point results.

```
A = tan([-2, -pi, pi/6, 5*pi/7, 11])  
  
A =  
    2.1850    0.0000    0.5774   -1.2540 -225.9508
```

Compute the tangent function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `tan` returns unresolved symbolic calls.

```
symA = tan(sym([-2, -pi, pi/6, 5*pi/7, 11]))  
  
symA =  
[ -tan(2), 0, 3^(1/2)/3, -tan((2*pi)/7), tan(11)]
```

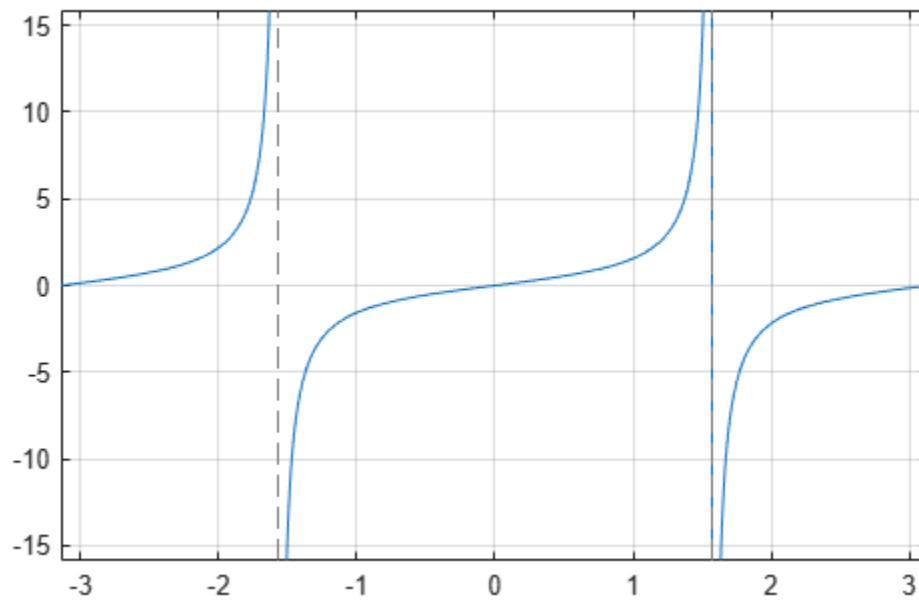
Use `vpa` to approximate symbolic results with floating-point numbers:

```
vpa(symA)  
  
ans =  
[ 2.1850398632615189916433061023137,...  
 0,...  
 0.57735026918962576450914878050196,...  
 -1.2539603376627038375709109783365,...  
 -225.95084645419514202579548320345]
```

#### Plot Tangent Function

Plot the tangent function on the interval from  $-\pi$  to  $\pi$ .

```
syms x  
fplot(tan(x),[-pi pi])  
grid on
```



### Handle Expressions Containing Tangent Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `tan`.

Find the first and second derivatives of the tangent function:

```
syms x
diff(tan(x), x)
diff(tan(x), x, x)

ans =
tan(x)^2 + 1

ans =
2*tan(x)*(tan(x)^2 + 1)
```

Find the indefinite integral of the tangent function:

```
int(tan(x), x)

ans =
-log(cos(x))
```

Find the Taylor series expansion of `tan(x)`:

```
taylor(tan(x), x)

ans =
(2*x^5)/15 + x^3/3 + x
```

Rewrite the tangent function in terms of the sine and cosine functions:

```
rewrite(tan(x), 'sincos')
```

```
ans =  
sin(x)/cos(x)
```

Rewrite the tangent function in terms of the exponential function:

```
rewrite(tan(x), 'exp')  
  
ans =  
-(exp(x*2i)*1i - 1i)/(exp(x*2i) + 1)
```

### Evaluate Units with tan Function

`tan` numerically evaluates these units automatically: radian, degree, arcmin, arcsec, and revolution.

Show this behavior by finding the tangent of  $x$  degrees and 2 radians.

```
u = symunit;  
syms x  
f = [x*u.degree 2*u.radian];  
tanf = tan(f)  
  
tanf =  
[ tan((pi*x)/180), tan(2)]
```

You can calculate `tanf` by substituting for  $x$  using `subs` and then using `double` or `vpa`.

## Input Arguments

### X — Input

symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic expression |  
symbolic function | symbolic matrix function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, scalar variable, matrix variable, expression, function, matrix function, or as a vector or matrix of symbolic numbers, scalar variables, expressions, or functions.

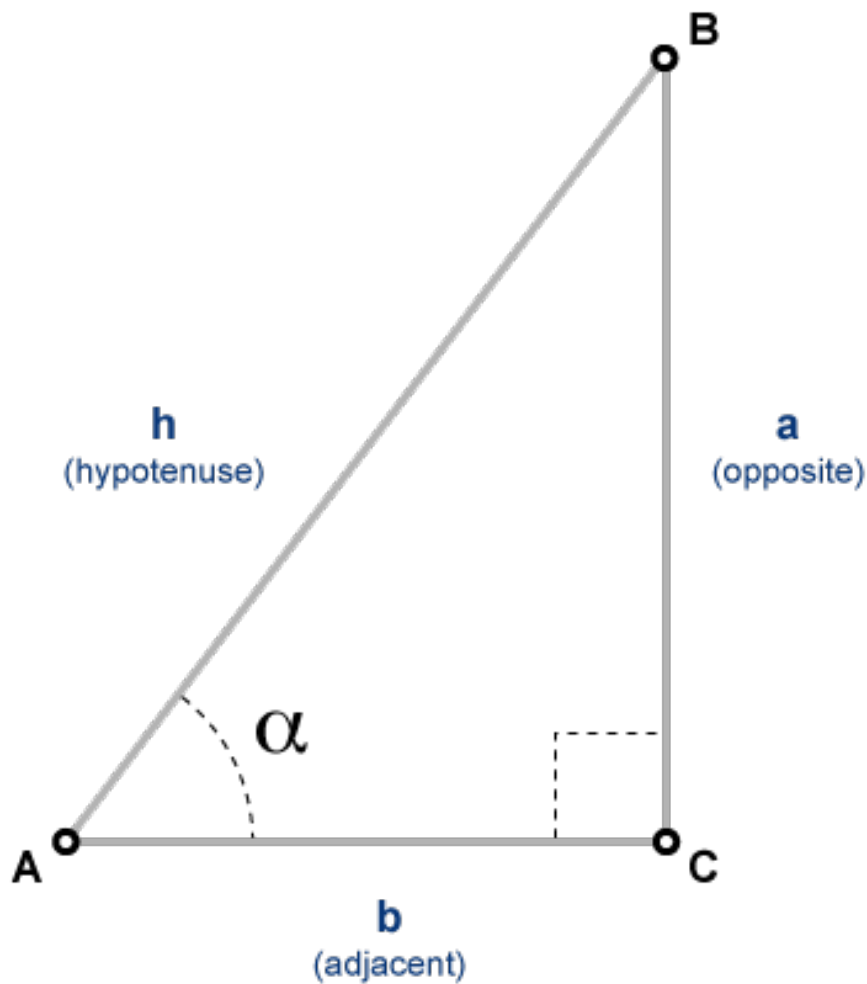
## More About

### Tangent Function

The tangent of an angle,  $\alpha$ , defined with reference to a right triangle is

$$\tan(\alpha) = \frac{\text{opposite side}}{\text{adjacent side}} = \frac{a}{b}.$$

.



The tangent of a complex argument,  $\alpha$ , is

$$\tan(\alpha) = \frac{e^{i\alpha} - e^{-i\alpha}}{i(e^{i\alpha} + e^{-i\alpha})}.$$

## Version History

Introduced before R2006a

**R2022a: Compute tangent of symbolic matrix functions**

The tan function accepts an input argument of type symfunmatrix.

**R2021a: Compute tangent of symbolic matrix variables**

The `tan` function accepts an input argument of type `symmatrix`.

**See Also**

`acos` | `asin` | `atan` | `acsc` | `asec` | `acot` | `sin` | `cos` | `csc` | `sec` | `cot`



# tanh

Symbolic hyperbolic tangent function

## Syntax

`tanh(X)`

## Description

`tanh(X)` returns the hyperbolic tangent function of  $X$ .

## Examples

### Hyperbolic Tangent Function for Numeric and Symbolic Arguments

Depending on its arguments, `tanh` returns floating-point or exact symbolic results.

Compute the hyperbolic tangent function for these numbers. Because these numbers are not symbolic objects, `tanh` returns floating-point results.

```
A = tanh([-2, -pi*i, pi*i/6, pi*i/3, 5*pi*i/7])

A =
-0.9640 + 0.0000i    0.0000 + 0.0000i    0.0000 + 0.5774i...
0.0000 + 1.7321i    0.0000 - 1.2540i
```

Compute the hyperbolic tangent function for the numbers converted to symbolic objects. For many symbolic (exact) numbers, `tanh` returns unresolved symbolic calls.

```
symA = tanh(sym([-2, -pi*i, pi*i/6, pi*i/3, 5*pi*i/7]))

symA =
[ -tanh(2), 0, (3^(1/2)*1i)/3, 3^(1/2)*1i, -tanh((pi*2i)/7)]
```

Use `vpa` to approximate symbolic results with floating-point numbers:

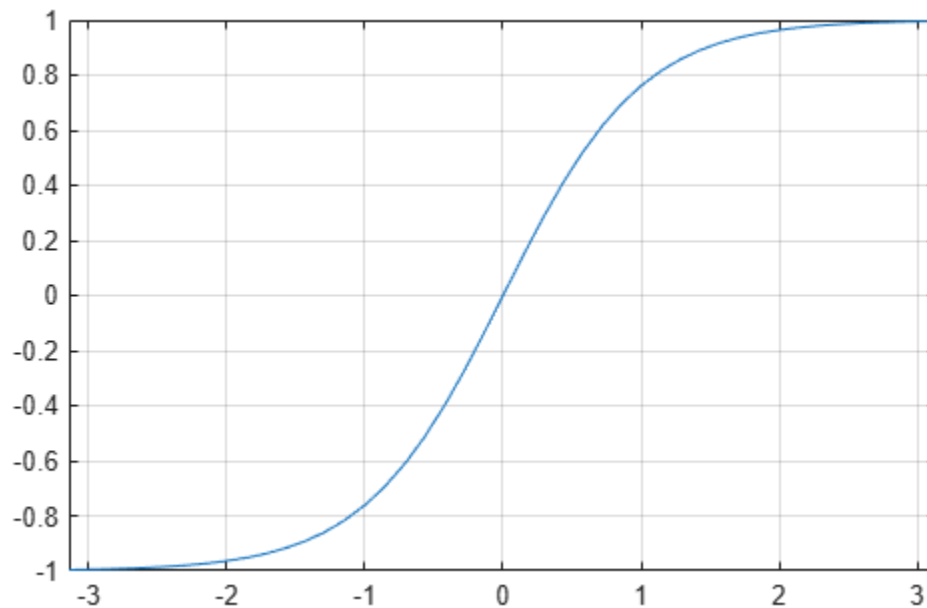
```
vpa(symA)

ans =
[ -0.96402758007581688394641372410092,...
0,...
0.57735026918962576450914878050196i,...
1.7320508075688772935274463415059i,...
-1.2539603376627038375709109783365i]
```

### Plot Hyperbolic Tangent Function

Plot the hyperbolic tangent function on the interval from  $-\pi$  to  $\pi$ .

```
syms x
fplot(tanh(x),[-pi pi])
grid on
```



### Handle Expressions Containing Hyperbolic Tangent Function

Many functions, such as `diff`, `int`, `taylor`, and `rewrite`, can handle expressions containing `tanh`.

Find the first and second derivatives of the hyperbolic tangent function:

```
syms x
diff(tanh(x), x)
diff(tanh(x), x, x)

ans =
1 - tanh(x)^2

ans =
2*tanh(x)*(tanh(x)^2 - 1)
```

Find the indefinite integral of the hyperbolic tangent function:

```
int(tanh(x), x)

ans =
log(cosh(x))
```

Find the Taylor series expansion of  $\tanh(x)$ :

```
taylor(tanh(x), x)

ans =
(2*x^5)/15 - x^3/3 + x
```

Rewrite the hyperbolic tangent function in terms of the exponential function:

```
rewrite(tanh(x), 'exp')
```

```
ans =  
(exp(2*x) - 1)/(exp(2*x) + 1)
```

## Input Arguments

### **X — Input**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic number, variable, expression, or function, or as a vector or matrix of symbolic numbers, variables, expressions, or functions.

## Version History

**Introduced before R2006a**

### **See Also**

acosh | asinh | atanh | acoth | asech | acsch | sinh | cosh | coth | sech | csch

# taylor

Taylor series

## Syntax

```
T = taylor(f,var)
T = taylor(f,var,a)
T = taylor( ___,Name=Value)
```

## Description

`T = taylor(f,var)` approximates `f` with the Taylor series expansion on page 6-1578 of `f` up to the fifth order at the point `var = 0`. If you do not specify `var`, then `taylor` uses the default variable determined by `symvar(f,1)`.

`T = taylor(f,var,a)` approximates `f` with the Taylor series expansion of `f` at the point `var = a`.

`T = taylor( ___,Name=Value)` specifies options using one or more name-value arguments in addition to any of the input argument combinations in previous syntaxes. For example, you can specify the expansion point, truncation order, or order mode of the Taylor series expansion.

## Examples

### Find Maclaurin Series of Univariate Expressions

Find the Maclaurin series expansions of the exponential, sine, and cosine functions up to the fifth order.

```
syms x
T1 = taylor(exp(x))

T1 =

$$\frac{x^5}{120} + \frac{x^4}{24} + \frac{x^3}{6} + \frac{x^2}{2} + x + 1$$

```

```
T2 = taylor(sin(x))
```

```
T2 =

$$\frac{x^5}{120} - \frac{x^3}{6} + x$$

```

```
T3 = taylor(cos(x))
```

```
T3 =

$$\frac{x^4}{24} - \frac{x^2}{2} + 1$$

```

You can use the `sympref` function to modify the output order of symbolic polynomials. Redisplay the polynomials in ascending order.

```
sympref("PolynomialDisplayStyle","ascend");
T1
```

$$T1 = 1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \frac{x^4}{24} + \frac{x^5}{120}$$

```
T2
```

$$T2 = x - \frac{x^3}{6} + \frac{x^5}{120}$$

```
T3
```

$$T3 = 1 - \frac{x^2}{2} + \frac{x^4}{24}$$

The display format you set using `sympref` persists through your current and future MATLAB® sessions. Restore the default value by specifying the "default" option.

```
sympref("default");
```

### Specify Expansion Point

Find the Taylor series expansions at  $x = 1$  for these functions. The default expansion point is 0. To specify a different expansion point, use `ExpansionPoint`.

```
syms x
T = taylor(log(x),x,ExpansionPoint=1)
```

$$T = x - \frac{(x-1)^2}{2} + \frac{(x-1)^3}{3} - \frac{(x-1)^4}{4} + \frac{(x-1)^5}{5} - 1$$

Alternatively, specify the expansion point as the third argument of `taylor`.

```
T = taylor(acot(x),x,1)
```

$$T = \frac{\pi}{4} - \frac{x}{2} + \frac{(x-1)^2}{4} - \frac{(x-1)^3}{12} + \frac{(x-1)^5}{40} + \frac{1}{2}$$

### Specify Truncation Order

Find the Maclaurin series expansion for  $f = \sin(x)/x$ . The default truncation order is 6. The Taylor series approximation of this expression does not have a fifth-degree term, so `taylor` approximates this expression with the fourth-degree polynomial.

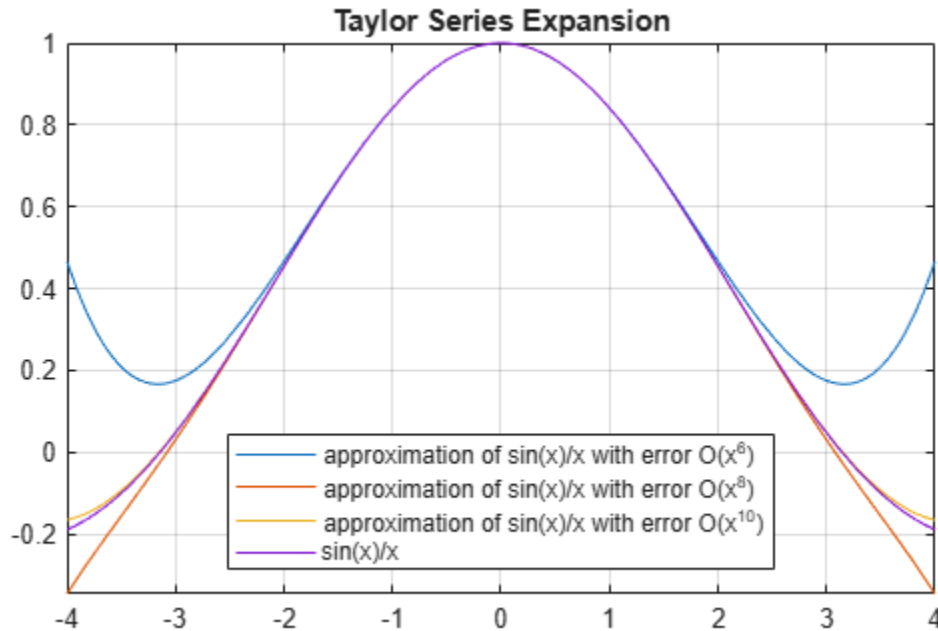
```
syms x
f = sin(x)/x;
T6 = taylor(f,x);
```

Use `Order` to control the truncation order. For example, approximate the same expression up to the orders 7 and 9.

```
T8 = taylor(f,x,Order=8);
T10 = taylor(f,x,Order=10);
```

Plot the original expression `f` and its approximations `T6`, `T8`, and `T10`. Note how the accuracy of the approximation depends on the truncation order.

```
fplot([T6 T8 T10 f])
xlim([-4 4])
grid on
legend("approximation of sin(x)/x with error O(x^6)", ...
      "approximation of sin(x)/x with error O(x^8)", ...
      "approximation of sin(x)/x with error O(x^{10})", ...
      "sin(x)/x",Location="Best")
title("Taylor Series Expansion")
```



### Specify Truncation Order as Relative or Absolute

Find the Taylor series expansion of this expression. By default, `taylor` uses an absolute order, which is the truncation order of the computed series.

```
syms x
T = taylor(1/exp(x) - exp(x) + 2*x,x,Order=5)
```

T =

$$-\frac{x^3}{3}$$

Find the Taylor series expansion with a relative truncation order by using `OrderMode`. For some expressions, a relative truncation order provides more accurate approximations.

```
T = taylor(1/exp(x) - exp(x) + 2*x,x,Order=5,OrderMode="relative")
```

$$T = -\frac{x^7}{2520} - \frac{x^5}{60} - \frac{x^3}{3}$$

### Find Maclaurin Series of Multivariate Expressions

Find the Maclaurin series expansion of this multivariate expression. If you do not specify the vector of variables, `taylor` treats `f` as a function of one independent variable.

```
syms x y z
f = sin(x) + cos(y) + exp(z);
T = taylor(f)
```

$$T = \frac{x^5}{120} - \frac{x^3}{6} + x + \cos(y) + e^z$$

Find the multivariate Maclaurin series expansion by specifying the vector of variables.

```
syms x y z
f = sin(x) + cos(y) + exp(z);
T = taylor(f,[x,y,z])
```

$$T = \frac{x^5}{120} - \frac{x^3}{6} + x + \frac{y^4}{24} - \frac{y^2}{2} + \frac{z^5}{120} + \frac{z^4}{24} + \frac{z^3}{6} + \frac{z^2}{2} + z + 2$$

You can use the `sympref` function to modify the output order of a symbolic polynomial. Redisplay the polynomial in ascending order.

```
sympref("PolynomialDisplayStyle","ascend");
T
```

$$T = 2 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24} + \frac{z^5}{120} - \frac{y^2}{2} + \frac{y^4}{24} + x - \frac{x^3}{6} + \frac{x^5}{120}$$

The display format you set using `sympref` persists through your current and future MATLAB sessions. Restore the default value by specifying the `"default"` option.

```
sympref("default");
```

### Specify Expansion Point for Multivariate Expression

Find the multivariate Taylor series expansion by specifying both the vector of variables and the vector of values defining the expansion point.

```
syms x y
f = y*exp(x - 1) - x*log(y);
T = taylor(f,[x y],[1 1],Order=3)
```

$$T = x + \frac{(x-1)^2}{2} + \frac{(y-1)^2}{2}$$

If you specify the expansion point as a scalar **a**, **taylor** transforms that scalar into a vector of the same length as the vector of variables. All elements of the expansion vector equal **a**.

```
T = taylor(f,[x y],1,Order=3)
```

$$T = x + \frac{(x-1)^2}{2} + \frac{(y-1)^2}{2}$$

### Error Estimate in Taylor Approximation

Find the error estimate when approximating a function  $f(x) = \log(x + 1)$  using the Taylor series expansion. Here, consider the Taylor approximation up to the 7th order (with the truncation order  $n = 8$ ) at the expansion point  $a = 0$ .

The error or remainder in the Taylor approximation is given by the Lagrange form:

$$R_{n-1}(x) = \frac{f^n(c)}{n!}(x-a)^n.$$

The upper bound of the error estimate can be calculated by finding a positive real number  $M$  such that  $|f^n(c)| \leq M$  for all  $c$  between  $a$  and  $x$ .

Find the Taylor series expansion of the function  $f(x) = \log(x + 1)$  up to the 7th order by specifying **Order** as 8.

```
syms x
f = log(x+1)
```

```
f = log(x + 1)
```

```
T = taylor(f,Order=8)
```

$$T = \frac{x^7}{7} - \frac{x^6}{6} + \frac{x^5}{5} - \frac{x^4}{4} + \frac{x^3}{3} - \frac{x^2}{2} + x$$

To estimate the error in the Taylor approximation, first compute the term  $f^8(c)$ .

```
syms c
fn(c) = subs(diff(f,8),x,c)
```

$$fn(c) = -\frac{5040}{(c+1)^8}$$



For positive values of  $x$ , the upper bound of the error estimate can be calculated by using the relation  $|f^8(c)| \leq 5040$  (because  $c$  must be a positive value between 0 and a positive  $x$ ). Next, find the upper bound of the error estimate  $R_{upper}(x)$  by using the Lagrange form  $R_7(x)$  and the relation  $|f^8(c)| \leq 5040$ .

```
Rupper(x) = 5040*x^8/factorial(8)
```

```
Rupper(x) =  
      x^8  
      8
```

Evaluate the Taylor series expansion at the point  $x = 0.5$ . Find the upper bound of the error estimate in the Taylor approximation.

```
Teval = subs(T,x,0.5)
```

```
Teval =  
      909  
      2240
```

```
Rmax = double(Rupper(0.5))
```

```
Rmax =  
4.8828e-04
```

For comparison, evaluate the exact function at  $x = 0.5$  and find the remainder in the Taylor approximation.

```
feval = subs(f,x,0.5)
```

```
feval =  
      log(3/2)
```

```
R = double(abs(feval-Teval))
```

```
R =  
3.3846e-04
```

## Input Arguments

### **f** — Input to approximate

symbolic expression | symbolic function | symbolic vector | symbolic matrix | symbolic multidimensional array

Input to approximate, specified as a symbolic expression or function. It also can be a vector, matrix, or multidimensional array of symbolic expressions or functions.

### **var** — Expansion variable

symbolic variable

Expansion variable, specified as a symbolic variable. If you do not specify `var`, then `taylor` uses the default variable determined by `symvar(f,1)`.

**a — Expansion point**

0 (default) | number | symbolic number | symbolic variable | symbolic function | symbolic expression

Expansion point, specified as a number, or a symbolic number, variable, function, or expression. The expansion point cannot depend on the expansion variable. You also can specify the expansion point as a name-value argument. If you specify the expansion point both ways, then the name-value argument takes precedence.

**Name-Value Pair Arguments**

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Example: `taylor(log(x),x,ExpansionPoint=1,Order=9)`

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

**ExpansionPoint — Expansion point**

0 (default) | number | symbolic number | symbolic variable | symbolic function | symbolic expression

Expansion point, specified as a number, or a symbolic number, variable, function, or expression. The expansion point cannot depend on the expansion variable. You can also specify the expansion point using the input argument `a`. If you specify the expansion point both ways, then the name-value argument takes precedence.

**Order — Truncation order of Taylor series expansion**

6 (default) | positive integer | symbolic positive integer

Truncation order of the Taylor series expansion, specified as a positive integer or a symbolic positive integer. `taylor` computes the Taylor series approximation with the order  $n - 1$ . The truncation order  $n$  is the exponent in the  $O$ -term:  $O(\text{var}^n)$ .

**OrderMode — Order mode indicator**

"absolute" (default) | "relative"

Order mode indicator, specified as "absolute" or "relative". This indicator specifies whether to use absolute or relative order when computing the Taylor polynomial approximation.

Absolute order is the truncation order of the computed series. Relative order  $n$  means that the exponents of `var` in the computed series range from the leading order  $m$  to the highest exponent  $m + n - 1$ . Here  $m + n$  is the exponent of `var` in the  $O$ -term:  $O(\text{var}^{m+n})$ .

**More About****Taylor Series Expansion**

A Taylor series expansion represents an analytic function  $f(x)$  as an infinite sum of terms around the expansion point  $x = a$ :

$$f(x) = f(a) + \frac{f'(a)}{1!}(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \dots = \sum_{m=0}^{\infty} \frac{f^{(m)}(a)}{m!} \cdot (x-a)^m$$

A Taylor series expansion requires a function to have derivatives up to an infinite order around the expansion point.

### Maclaurin Series Expansion

The Taylor series expansion around  $x = 0$  is called a Maclaurin series expansion:

$$f(x) = f(0) + \frac{f'(0)}{1!}x + \frac{f''(0)}{2!}x^2 + \dots = \sum_{m=0}^{\infty} \frac{f^{(m)}(0)}{m!}x^m$$

### Tips

- If you use both the third argument `a` and `ExpansionPoint` to specify the expansion point, then the value specified by `ExpansionPoint` prevails.
- If `var` is a vector, then the expansion point `a` must be a scalar or a vector of the same length as `var`. If `var` is a vector and `a` is a scalar, then `a` is expanded into a vector of the same length as `var` with all elements equal to `a`.
- If the expansion point is infinity or negative infinity, then `taylor` computes the Laurent series expansion, which is a power series in  $1/\text{var}$ .
- You can use the `sympref` function to modify the output order of symbolic polynomials.
- If `taylor` cannot find the Taylor series expansion, then use `series` to find the more general Puiseux series expansion.

## Version History

Introduced before R2006a

### See Also

`pade` | `series` | `symvar` | `coeffs` | `polynomialDegree` | `sympref`

### Topics

"Taylor Series" on page 3-192

## taylortool

Taylor series calculator

### Syntax

```
taylortool  
taylortool(f)  
t = taylortool(____)
```

### Description

`taylortool` initiates a GUI that computes the Taylor series expansion. The GUI that graphs a function against the Nth partial sum of its Taylor series about a base point  $x = a$ . The default function, value of N, base point, and interval of computation for `taylortool` are  $f = x \cos(x)$ ,  $N = 7$ ,  $a = 0$ , and  $[-2\pi, 2\pi]$ , respectively.

`taylortool(f)` initiates the GUI that computes the Taylor series expansion of the given expression  $f$ .

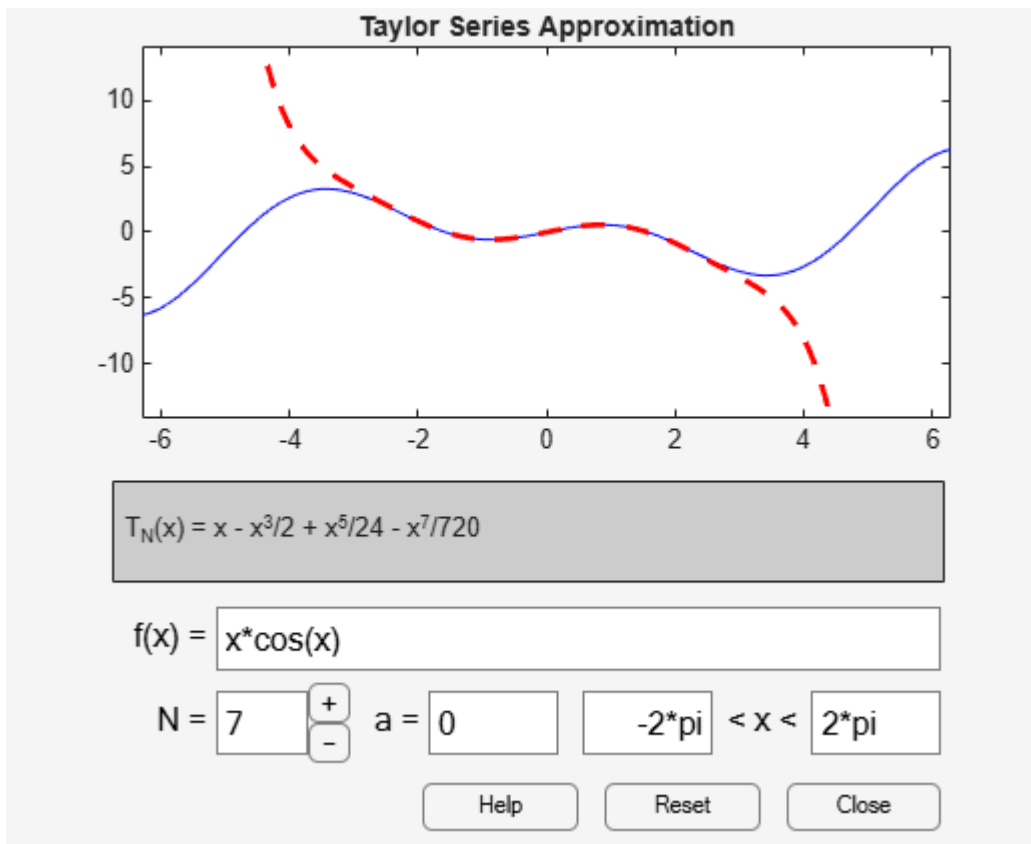
`t = taylortool(____)` returns the Figure object. Use `t` to query or modify properties of the figure after it is created.

### Examples

#### Find Taylor Series Expansion of Function

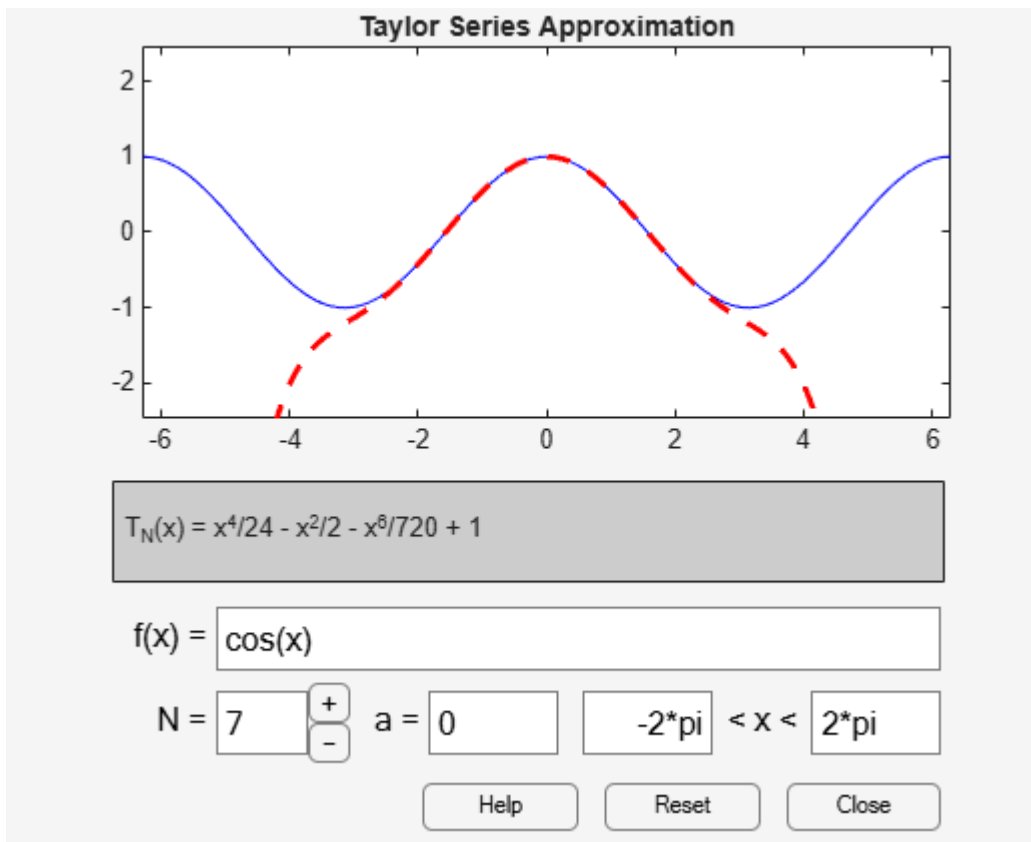
Open a GUI that computes the Taylor series expansion. By default, the GUI shows the Taylor series expansion of the function  $f(x) = x \cos(x)$ . The blue solid line shows the function  $f(x)$  and the red dashed line shows its Taylor approximation.

```
taylortool
```



Next, find the Taylor series expansion of the function  $\cos(x)$ . Assign an output variable to `taylortool` to return the Figure object.

```
syms x
t = taylortool(cos(x))
```



```
t =
Figure (Taylor Tool) with properties:
```

```
Number: []
Name: 'Taylor Tool'
Color: [0.9608 0.9608 0.9608]
Position: [112.5000 75 390 315]
Units: 'points'
```

Show all properties

Close the figure.

```
close(t)
```

## Input Arguments

### **f** — Input to approximate

symbolic expression | symbolic function

Input to approximate, specified as a symbolic expression or function.

## **Version History**

Introduced before R2006a

## **See Also**

[funtool](#) | [rsums](#) | [taylor](#)

## **Topics**

"Taylor Series" on page 3-192

## texlabel

TeX representation of symbolic expression

### Syntax

```
texlabel(expr)
texlabel(expr, 'literal')
```

### Description

`texlabel(expr)` converts the symbolic expression `expr` into the TeX equivalent for use in character vectors. `texlabel` converts Greek variable names, such as `delta`, into Greek letters. Annotation functions, such as `title`, `xlabel`, and `text` can use the TeX character vector as input. To obtain the LaTeX representation, use `latex`.

`texlabel(expr, 'literal')` interprets Greek variable names literally.

### Examples

#### Generate TeX Character Vector

Use `texlabel` to generate TeX character vectors for these symbolic expressions.

```
syms x y lambda12 delta
texlabel(sin(x) + x^3)
texlabel(3*(1-x)^2*exp(-(x^2) - (y+1)^2))
texlabel(lambda12^(3/2)/pi - pi*delta^(2/3))

ans =
    '{sin}({x}) + {x}^{3}'

ans =
    '{3} {exp}(- ({y} + {1})^{2} - {x}^{2}) ({x} - {1})^{2}'

ans =
    '{\lambda_{12}}^{\{3\}/\{2\}}/{\pi} - {\delta}^{\{2\}/\{3\}} {\pi}'
```

To make `texlabel` interpret Greek variable names literally, include the argument `'literal'`.

```
texlabel(lambda12, 'literal')

ans =
    '{lambda12}'
```

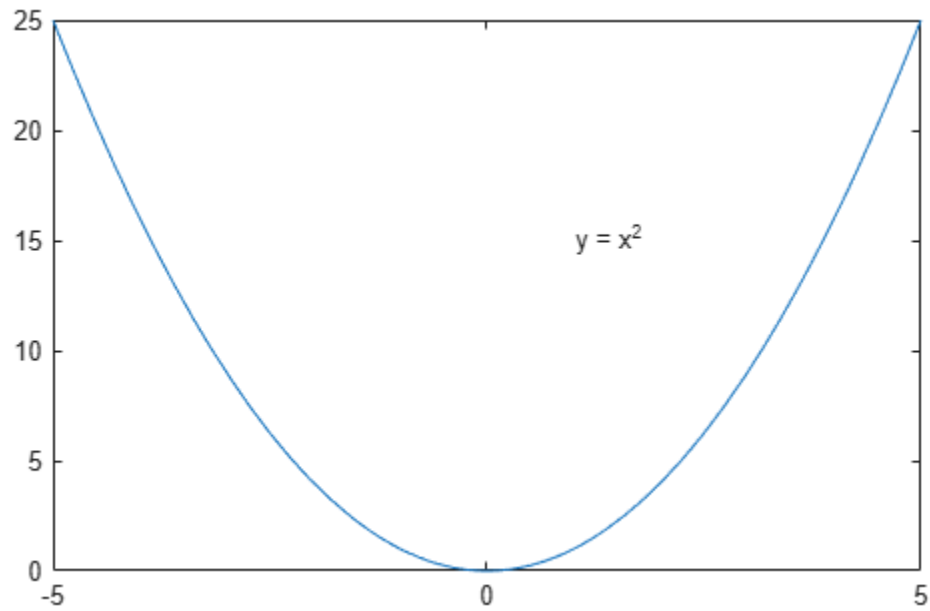
#### Insert TeX in Figure

Plot  $y = x^2$  using `fplot`. Show the plotted expression `y` by using `texlabel` to generate a TeX character vector that `text` inserts into the figure.

```
syms x
y = x^2;
```



```
fplot(y)
ylabel = texlabel(y);
text(1, 15, ['y = ' ylabel]);
```



## Input Arguments

**expr** — Expression to be converted

symbolic expression

Expression to be converted, specified as a symbolic expression.

## Version History

Introduced before R2006a

## See Also

latex | text | title | xlabel | ylabel | zlabel

## times, .\*

Symbolic array multiplication

### Syntax

```
A.*B  
times(A,B)
```

### Description

A.\*B performs element-wise multiplication of A and B.

times(A,B) is equivalent to A.\*B.

### Examples

#### Multiply Matrix by Scalar

Create a 2-by-3 matrix.

```
A = sym('a', [2 3])  
  
A =  
[ a1_1, a1_2, a1_3]  
[ a2_1, a2_2, a2_3]
```

Multiply the matrix by the symbolic expression `sin(b)`. Multiplying a matrix by a scalar means multiplying each element of the matrix by that scalar.

```
syms b  
A.*sin(b)  
  
ans =  
[ a1_1*sin(b), a1_2*sin(b), a1_3*sin(b)]  
[ a2_1*sin(b), a2_2*sin(b), a2_3*sin(b)]
```

#### Multiply Two Matrices

Create a 3-by-3 symbolic Hilbert matrix and a 3-by-3 diagonal matrix.

```
H = sym(hilb(3))  
d = diag(sym([1 2 3]))  
  
H =  
[ 1, 1/2, 1/3]  
[ 1/2, 1/3, 1/4]  
[ 1/3, 1/4, 1/5]  
  
d =  
[ 1, 0, 0]  
[ 0, 2, 0]  
[ 0, 0, 3]
```

Multiply the matrices by using the element-wise multiplication operator `.*`. This operator multiplies each element of the first matrix by the corresponding element of the second matrix. The dimensions of the matrices must be the same.

```
H.*d
```

```
ans =
[ 1,    0,    0]
[ 0, 2/3,    0]
[ 0,    0, 3/5]
```

## Multiply Expression by Symbolic Function

Multiply a symbolic expression by a symbolic function. The result is a symbolic function.

```
syms f(x)
f(x) = x^2;
f1 = (x^2 + 5*x + 6).*f
```

```
f1(x) =
x^2*(x^2 + 5*x + 6)
```

## Input Arguments

### A — Input

number | symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix | symbolic array

Input, specified as a number, or a symbolic number, scalar variable, matrix variable, function, matrix function, expression, or vector, matrix, or array of symbolic scalar variables. Inputs A and B must be the same size unless one is a scalar. A scalar value expands into an array of the same size as the other input.

### B — Input

number | symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix | symbolic array

Input, specified as a number, or a symbolic number, scalar variable, matrix variable, function, matrix function, expression, or vector, matrix, or array of symbolic scalar variables. Inputs A and B must be the same size unless one is a scalar. A scalar value expands into an array of the same size as the other input.

## Version History

Introduced before R2006a

### R2022a: Compute element-wise multiplication of symbolic matrix functions

The `times` function accepts an input argument of type `symfunmatrix`.

### R2021a: Compute element-wise multiplication of symbolic matrix variables

The `times` function accepts an input argument of type `symmatrix`.

**See Also**

`ctranspose` | `ldivide` | `minus` | `mldivide` | `mpower` | `mrdivide` | `mtimes` | `plus` | `power` | `rdivide` | `transpose`

# toeplitz

Symbolic Toeplitz matrix

## Syntax

```
toeplitz(c,r)
toeplitz(r)
```

## Description

`toeplitz(c,r)` generates a nonsymmetric Toeplitz matrix on page 6-1591 having `c` as its first column and `r` as its first row. If the first elements of `c` and `r` are different, `toeplitz` issues a warning and uses the first element of the column.

`toeplitz(r)` generates a symmetric Toeplitz matrix if `r` is real. If `r` is complex, but its first element is real, then this syntax generates the Hermitian Toeplitz matrix formed from `r`. If the first element of `r` is not real, then the resulting matrix is Hermitian off the main diagonal, meaning that  $T_{ij} = \text{conjugate}(T_{ji})$  for  $i \neq j$ .

## Examples

### Generate Toeplitz Matrix

Generate the Toeplitz matrix from these vectors. Because these vectors are not symbolic objects, you get floating-point results.

```
c = [1 2 3 4 5 6];
r = [1 3/2 3 7/2 5];
toeplitz(c,r)

ans =
    1.0000    1.5000    3.0000    3.5000    5.0000
    2.0000    1.0000    1.5000    3.0000    3.5000
    3.0000    2.0000    1.0000    1.5000    3.0000
    4.0000    3.0000    2.0000    1.0000    1.5000
    5.0000    4.0000    3.0000    2.0000    1.0000
    6.0000    5.0000    4.0000    3.0000    2.0000
```

Now, convert these vectors to a symbolic object, and generate the Toeplitz matrix:

```
c = sym([1 2 3 4 5 6]);
r = sym([1 3/2 3 7/2 5]);
toeplitz(c,r)

ans =
[ 1, 3/2, 3, 7/2, 5]
[ 2, 1, 3/2, 3, 7/2]
[ 3, 2, 1, 3/2, 3]
[ 4, 3, 2, 1, 3/2]
[ 5, 4, 3, 2, 1]
[ 6, 5, 4, 3, 2]
```

**Generate Toeplitz Matrix from Vector**

Generate the Toeplitz matrix from this vector:

```
syms a b c d
T = toeplitz([a b c d])

T =
[      a,      b,      c,      d]
[ conj(b),      a,      b,      c]
[ conj(c), conj(b),      a,      b]
[ conj(d), conj(c), conj(b),      a]
```

If you specify that all elements are real, then the resulting Toeplitz matrix is symmetric:

```
syms a b c d real
T = toeplitz([a b c d])

T =
[ a, b, c, d]
[ b, a, b, c]
[ c, b, a, b]
[ d, c, b, a]
```

For further computations, clear the assumptions by recreating the variables using `syms`:

```
syms a b c d
```

**Generate Toeplitz with Complex Values**

Generate the Toeplitz matrix from a vector containing complex numbers:

```
T = toeplitz(sym([1, 2, 1i]))

T =
[ 1, 2, 1i]
[ 2, 1, 2]
[-1i, 2, 1]
```

If the first element of the vector is real, then the resulting Toeplitz matrix is Hermitian:

```
isAlways(T == T')

ans =
3x3 logical array
1 1 1
1 1 1
1 1 1
```

If the first element is not real, then the resulting Toeplitz matrix is Hermitian off the main diagonal:

```
T = toeplitz(sym([1i, 2, 1]))

T =
[ 1i, 2, 1]
[ 2, 1i, 2]
[ 1, 2, 1i]

isAlways(T == T')
```

```
ans =
  3x3 logical array
    0     1     1
    1     0     1
    1     1     0
```

### Use Vectors with Conflicting First Element

Generate a Toeplitz matrix using these vectors to specify the first column and the first row. Because the first elements of these vectors are different, `toeplitz` issues a warning and uses the first element of the column:

```
syms a b c
toeplitz([a b c], [1 b/2 a/2])
```

Warning: First element of given column does not match first element of given row.  
Column wins diagonal conflict.

```
ans =
[ a, b/2, a/2]
[ b,  a, b/2]
[ c,  b,  a]
```

## Input Arguments

### **c** — First column of Toeplitz matrix

vector | symbolic vector

First column of Toeplitz matrix, specified as a vector or symbolic vector.

### **r** — First row of Toeplitz matrix

vector | symbolic vector

First row of Toeplitz matrix, specified as a vector or symbolic vector.

## More About

### Toeplitz Matrix

A Toeplitz matrix is a matrix that has constant values along each descending diagonal from left to right. For example, matrix  $T$  is a symmetric Toeplitz matrix:

$$T = \begin{pmatrix} t_0 & t_1 & t_2 & & t_k \\ t_{-1} & t_0 & t_1 & \cdots & \\ t_{-2} & t_{-1} & t_0 & & \\ & \vdots & & \ddots & \vdots \\ & & & t_0 & t_1 & t_2 \\ & & & \cdots & t_{-1} & t_0 & t_1 \\ t_{-k} & & & & t_{-2} & t_{-1} & t_0 \end{pmatrix}$$

## Tips

- Calling `toeplitz` for numeric arguments that are not symbolic objects invokes the MATLAB `toeplitz` function.

## **Version History**

**Introduced in R2013a**

## **See Also**

`toeplitz`



# transpose, .'

Symbolic matrix transpose

## Syntax

A. '  
transpose(A)

## Description

A. ' computes the nonconjugate transpose on page 6-1594 of A.

transpose(A) is equivalent to A. '.

## Examples

### Transpose of Real Matrix

Create a 2-by-3 matrix, the elements of which represent real numbers.

```
syms x y real
A = [x x x; y y y]

A =
[ x, x, x]
[ y, y, y]
```

Find the nonconjugate transpose of this matrix.

```
A. '

ans =
[ x, y]
[ x, y]
[ x, y]
```

If all elements of a matrix represent real numbers, then its complex conjugate transform equals its nonconjugate transform.

```
isAlways(A' == A. ')

ans =
 3x2 logical array
 1     1
 1     1
 1     1
```

### Transpose of Complex Matrix

Create a 2-by-2 matrix, the elements of which represent complex numbers.

```
syms x y real
A = [x + y*i x - y*i; y + x*i y - x*i]
```

```
A =
[ x + y*1i, x - y*1i]
[ y + x*1i, y - x*1i]
```

Find the nonconjugate transpose of this matrix. The nonconjugate transpose operator, `A.'`, performs a transpose without conjugation. That is, it does not change the sign of the imaginary parts of the elements.

```
A.'
```

```
ans =
[ x + y*1i, y + x*1i]
[ x - y*1i, y - x*1i]
```

For a matrix of complex numbers with nonzero imaginary parts, the nonconjugate transform is not equal to the complex conjugate transform.

```
isAlways(A.' == A', 'Unknown', 'false')
```

```
ans =
    2x2 logical array
    0     0
    0     0
```

## Input Arguments

### A — Input

number | symbolic number | symbolic scalar variable | symbolic matrix variable | symbolic function | symbolic matrix function | symbolic expression | symbolic vector | symbolic matrix | symbolic array

Input, specified as a number, or a symbolic number, scalar variable, matrix variable, function, matrix function, expression, or vector, matrix, or array of symbolic scalar variables.

## More About

### Nonconjugate Transpose

The nonconjugate transpose of a matrix interchanges the row and column index for each element, reflecting the elements across the main diagonal. The diagonal elements themselves remain unchanged. This operation does not affect the sign of the imaginary parts of complex elements.

For example, if `B = A.'` and `A(3,2)` is `1+1i`, then the element `B(2,3)` is `1+1i`.

## Version History

Introduced before R2006a

### R2022a: Nonconjugate transpose of symbolic matrix functions

The transpose function accepts an input argument of type `symfunmatrix`.

### R2021a: Nonconjugate transpose of symbolic matrix variables

The `transpose` function accepts an input argument of type `symmatrix`.

**See Also**

`ctranspose` | `ldivide` | `minus` | `mldivide` | `mpower` | `mrdivide` | `mtimes` | `plus` | `power` | `rdivide` | `times`

## triangularPulse

Triangular pulse function

### Syntax

```
triangularPulse(a,b,c,x)  
triangularPulse(a,c,x)  
triangularPulse(x)
```

### Description

`triangularPulse(a,b,c,x)` returns the “Triangular Pulse Function” on page 6-1599.

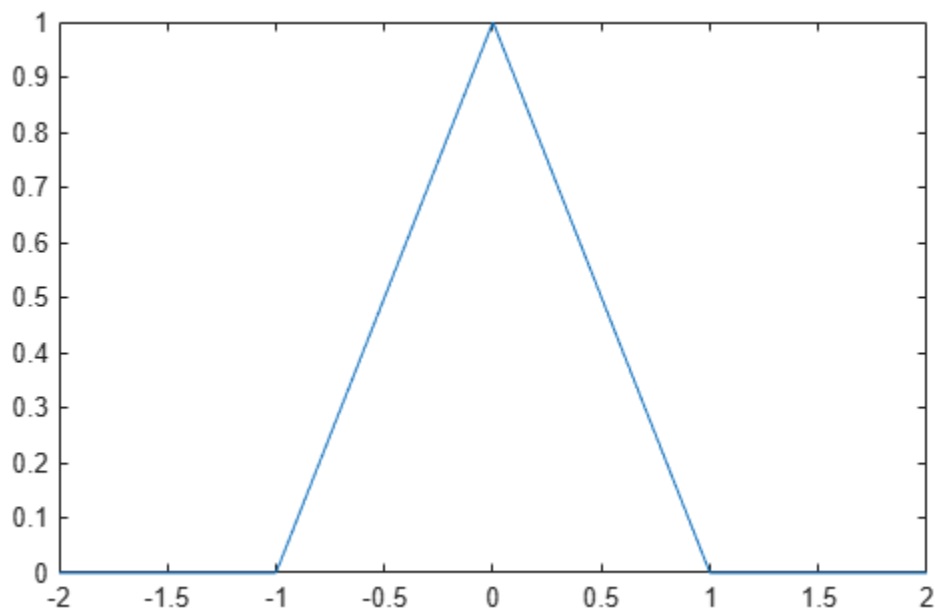
`triangularPulse(a,c,x)` is a shortcut for `triangularPulse(a, (a + c)/2, c, x)`.

`triangularPulse(x)` is a shortcut for `triangularPulse(-1, 0, 1, x)`.

### Examples

#### Plot Triangular Pulse Function

```
syms x  
fplot(triangularPulse(x), [-2 2])
```



### Compute Triangular Pulse Function

Compute the triangular pulse function for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
[triangularPulse(-2, 0, 2, -3)
triangularPulse(-2, 0, 2, -1/2)
triangularPulse(-2, 0, 2, 0)
triangularPulse(-2, 0, 2, 3/2)
triangularPulse(-2, 0, 2, 3)]]
```

```
ans =
      0
    0.7500
    1.0000
    0.2500
      0
```

Compute the same values symbolically by converting the numbers to symbolic objects.

```
[triangularPulse(sym(-2), 0, 2, -3)
triangularPulse(-2, 0, 2, sym(-1/2))
triangularPulse(-2, sym(0), 2, 0)
triangularPulse(-2, 0, 2, sym(3/2))
triangularPulse(-2, 0, sym(2), 3)]
```

```
ans =
      0
    3/4
      1
    1/4
      0
```

### Fixed-Form Triangular Pulse of Width 2

Use `triangularPulse` with one input argument as a shortcut for computing `triangularPulse(-1, 0, 1, x)`:

```
syms x
triangularPulse(x)
```

```
ans =
triangularPulse(-1, 0, 1, x)
```

### Symmetrical Triangular Pulse

Use `triangularPulse` with three input arguments as a shortcut for computing `triangularPulse(a, (a + c)/2, c, x)`:

```
syms a c x
triangularPulse(a, c, x)
```

```
ans =  
triangularPulse(a, a/2 + c/2, c, x)
```

### Special Cases of Triangular Pulse Function

Depending on the relation between inputs, the `triangularPulse` has special values.

Compute the triangular pulse function for  $a < x < b$ :

```
syms a b c x  
assume(a < x < b)  
triangularPulse(a, b, c, x)
```

```
ans =  
(a - x)/(a - b)
```

For further computations, remove the assumption by recreating the variables using `syms`:

```
syms a b x
```

Compute the triangular pulse function for  $b < x < c$ :

```
assume(b < x < c)  
triangularPulse(a, b, c, x)
```

```
ans =  
-(c - x)/(b - c)
```

For further computations, remove the assumption:

```
syms b c x
```

Compute the triangular pulse function for  $a = b$ :

```
syms a b c x  
assume(b < c)  
triangularPulse(b, b, c, x)  
  
ans =  
-((c - x)*rectangularPulse(b, c, x))/(b - c)
```

Compute the triangular pulse function for  $c = b$ :

```
assume(a < b)  
triangularPulse(a, b, b, x)  
  
ans =  
((a - x)*rectangularPulse(a, b, x))/(a - b)
```

For further computations, remove all assumptions on `a`, `b`, and `c`:

```
syms a b c
```

### Input Arguments

#### **a** — Input

-1 (default) | number | symbolic scalar

Input, specified as a number or a symbolic scalar. This argument specifies the rising edge of the triangular pulse function.

**b — Input**

number | symbolic scalar

Input, specified as a number or a symbolic scalar. This argument specifies the peak of the triangular pulse function. If you specify  $a$  and  $c$ , then  $(a + c)/2$ . Otherwise, 0.

**c — Input**

1 (default) | number | symbolic scalar

Input, specified as a number or a symbolic scalar. This argument specifies the falling edge of the triangular pulse function.

**x — Input**

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## More About

### Triangular Pulse Function

If  $a < x < b$ , then the triangular pulse function equals  $(x - a)/(b - a)$ .

If  $b < x < c$ , then the triangular pulse function equals  $(c - x)/(c - b)$ .

If  $x \leq a$  or  $x \geq c$ , then the triangular pulse function equals 0.

The triangular pulse function is also called the triangle function, hat function, tent function, or sawtooth function.

## Tips

- If  $a$ ,  $b$ , and  $c$  are variables or expressions with variables, `triangularPulse` assumes that  $a \leq b \leq c$ . If  $a$ ,  $b$ , and  $c$  are numerical values that do not satisfy this condition, `triangularPulse` throws an error.
- If  $a = b = c$ , `triangularPulse` returns 0.
- If  $a = b$  or  $b = c$ , the triangular function can be expressed in terms of the rectangular function.

## Version History

Introduced in R2012b

## See Also

`dirac` | `heaviside` | `rectangularPulse`

## tril

Return lower triangular part of symbolic matrix

### Syntax

```
tril(A)
tril(A,k)
```

### Description

`tril(A)` returns a triangular matrix that retains the lower part of the matrix `A`. The upper triangle of the resulting matrix is padded with zeros.

`tril(A,k)` returns a matrix that retains the elements of `A` on and below the `k`-th diagonal. The elements above the `k`-th diagonal equal to zero. The values `k = 0`, `k > 0`, and `k < 0` correspond to the main, superdiagonals, and subdiagonals, respectively.

### Examples

#### Lower Triangular Part of Symbolic Matrix

Display the matrix retaining only the lower triangle of the original symbolic matrix:

```
syms a b c
A = [a b c; 1 2 3; a + 1 b + 2 c + 3];
tril(A)
```

```
ans =
[      a,      0,      0]
[      1,      2,      0]
[ a + 1, b + 2, c + 3]
```

#### Triangular Matrix On and Below Specified Superdiagonal

Display the matrix that retains the elements of the original symbolic matrix on and below the first superdiagonal:

```
syms a b c
A = [a b c; 1 2 3; a + 1 b + 2 c + 3];
tril(A, 1)
```

```
ans =
[      a,      b,      0]
[      1,      2,      3]
[ a + 1, b + 2, c + 3]
```

#### Triangular Matrix On and Below Specified Subdiagonal

Display the matrix that retains the elements of the original symbolic matrix on and below the first subdiagonal:



```
syms a b c
A = [a b c; 1 2 3; a + 1 b + 2 c + 3];
tril(A, -1)
```

```
ans =
[      0,      0,  0]
[      1,      0,  0]
[ a + 1, b + 2,  0]
```

## Input Arguments

### A — Input

matrix | symbolic matrix

Input, specified as a numeric or symbolic matrix.

### k — Diagonal

number | symbolic number

Diagonal, specified as a numeric or symbolic number.

## Version History

Introduced before R2006a

## See Also

diag | triu

## triu

Return upper triangular part of symbolic matrix

### Syntax

```
triu(A)
triu(A,k)
```

### Description

`triu(A)` returns a triangular matrix that retains the upper part of the matrix `A`. The lower triangle of the resulting matrix is padded with zeros.

`triu(A,k)` returns a matrix that retains the elements of `A` on and above the  $k$ -th diagonal. The elements below the  $k$ -th diagonal equal to zero. The values  $k = 0$ ,  $k > 0$ , and  $k < 0$  correspond to the main, superdiagonals, and subdiagonals, respectively.

### Examples

#### Upper Triangular Part of Symbolic Matrix

Display the matrix retaining only the upper triangle of the original symbolic matrix:

```
syms a b c
A = [a b c; 1 2 3; a + 1 b + 2 c + 3];
triu(A)
```

```
ans =
[ a, b,      c]
[ 0, 2,      3]
[ 0, 0, c + 3]
```

#### Triangular Matrix On and Above Specified Superdiagonal

Display the matrix that retains the elements of the original symbolic matrix on and above the first superdiagonal:

```
syms a b c
A = [a b c; 1 2 3; a + 1 b + 2 c + 3];
triu(A, 1)
```

```
ans =
[ 0, b, c]
[ 0, 0, 3]
[ 0, 0, 0]
```

#### Triangular Matrix On and Above Specified Subdiagonal

Display the matrix that retains the elements of the original symbolic matrix on and above the first subdiagonal:

```
syms a b c
A = [a b c; 1 2 3; a + 1 b + 2 c + 3];
triu(A, -1)
```

```
ans =
[ a,      b,      c]
[ 1,      2,      3]
[ 0, b + 2, c + 3]
```

## Input Arguments

### A — Input

matrix | symbolic matrix

Input, specified as a numeric or symbolic matrix.

### k — Diagonal

number | symbolic number

Diagonal, specified as a numeric or symbolic number.

## Version History

Introduced before R2006a

## See Also

diag | tril

## unitConversionFactor

Conversion factor between units

### Syntax

```
C = unitConversionFactor(unit1,unit2)
C = unitConversionFactor(unit1,unit2,'Force',true)
```

### Description

`C = unitConversionFactor(unit1,unit2)` returns the conversion factor `C` between units `unit1` and `unit2` so that `unit1 = C*unit2`.

`C = unitConversionFactor(unit1,unit2,'Force',true)` forces `unitConversionFactor` to return a conversion factor even if the units are not dimensionally compatible.

### Examples

#### Conversion Factor Between Units

Find the conversion factor between inches and centimeters.

```
u = symunit;
inch2cm = unitConversionFactor(u.inch,u.cm)

inch2cm =
127/50
```

Convert the conversion factor to double.

```
inch2cm = double(inch2cm)

inch2cm =
    2.5400
```

Find the conversion factor between Newtons and  $\text{kg m/s}^2$ . The conversion factor is 1.

```
convFactor = unitConversionFactor(1*u.N, 1*u.kg*u.m/u.s^2)

convFactor =
1
```

#### Convert Between Units

Convert between units quickly by using text input as a shortcut. Convert 3 inch to cm.

```
3*double(unitConversionFactor("inch","cm"))
```

```
ans =
    7.6200
```

### Convert Between Dimensionally Incompatible Units

Convert between dimensionally incompatible units by specifying the argument 'Force' as true.

Convert between Watts and Joules. `unitConversionFactor` returns the factor 1/[s] because 1 W = 1 J/s.

```
u = symunit;
convFactor = unitConversionFactor(u.W, u.J, 'Force', true)

convFactor =
    1/[s]
```

If you do not specify 'Force' as true, then `unitConversionFactor` returns an error.

```
unitConversionFactor(u.W, u.J)

Error using unitConversionFactor (line 81)
Incompatible units.
```

### Input Arguments

#### **unit — Units**

character vector | string | symbolic units

Units, specified as a character vector, string, or symbolic units.

## Version History

Introduced in R2017a

### See Also

`checkUnits` | `findUnits` | `isUnit` | `newUnit` | `separateUnits` | `symunit` | `str2symunit` | `symunit2str`

### Topics

“Units of Measurement Tutorial” on page 2-47  
 “Unit Conversions and Unit Systems” on page 2-53  
 “Units and Unit Systems List” on page 2-60

### External Websites

The International System of Units (SI)

## unitConvert

Convert units to other units of measurement

### Syntax

```
convertedExpr = unitConvert(expr,units)
convertedExpr = unitConvert(expr,unitSystem)
convertedExpr = unitConvert(expr,unitSystem,unitMode)

convertedExpr = unitConvert( ____,Temperature=tempMode)
```

### Description

`convertedExpr = unitConvert(expr,units)` converts symbolic units in the input expression to the specified symbolic units. If the conversion is not possible, `unitConvert` returns the input expression without converting it.

`convertedExpr = unitConvert(expr,unitSystem)` converts symbolic units in the input expression to the base units of the specified unit system.

You can specify `unitSystem` as one of these predefined unit systems: "SI", "CGS", "US", "GU", "ESU", or "EMU". Alternatively, you can specify a custom unit system defined by using the `newUnitSystem` function.

`convertedExpr = unitConvert(expr,unitSystem,unitMode)` specifies whether to use the base units or derived units of the specified unit system by specifying `unitMode` as "base" or "derived". If you convert to the "SI" unit system, you can also specify `unitMode` as "constants" to convert the units in the input expression to the seven SI-defining constants.

`convertedExpr = unitConvert( ____,Temperature=tempMode)` specifies whether temperatures in the input expression represent absolute temperatures or temperature differences. Specify `tempMode` as "absolute" or "difference" after any of the input argument combinations in the previous syntaxes. By default, temperatures are assumed to represent differences during conversion.

### Examples

#### Convert Between Units

Convert 5 centimeters to inches. Because the calculation is symbolic, `unitConvert` returns a symbolic fractional result.

```
u = symunit;
length = unitConvert(5*u.cm,u.in)

length =
     $\frac{250}{127}$  in
```

Convert `length` to its numeric value in double precision by separating the value from the units using `separateUnits` and converting the value using `double`. Alternatively, keep the units by using `vpa`.

```
lengthDouble = double(separateUnits(length))
```

```
lengthDouble =  
1.9685
```

```
lengthVpa = vpa(length)
```

```
lengthVpa = 1.968503937007874015748031496063 in
```

Calculate the force required to accelerate 2 kg by 5 m/s<sup>2</sup>. The result is not automatically converted to newtons.

```
m = 2*u.kg;  
a = 5*u.m/u.s^2;  
F = m*a
```

```
F =  
10  $\frac{\text{kg m}}{\text{s}^2}$ 
```

Convert `F` to newtons by using `unitConvert`.

```
F = unitConvert(F,u.N)
```

```
F = 10 N
```

For more information, see “Unit Conversions and Unit Systems” on page 2-53.

### Specify Compound Unit for Conversion

Convert 5 kilometers per hour to meters per second by specifying meters per second as a compound unit.

```
u = symunit;  
v = unitConvert(5*u.km/u.hr,u.m/u.s)
```

```
v =  
 $\frac{25}{18} \frac{\text{m}}{\text{s}}$ 
```

### Specify Multiple Units for Conversion

Specify multiple units for conversion by specifying the second argument as a vector of units. This syntax allows you to specify the target units for each dimension of the input expression.

Convert 5 kilometers per hour to centimeters per minute.

```
u = symunit;  
v = 5*u.km/u.hr;  
units = [u.cm u.min];  
vConverted = unitConvert(v,units)
```

```
vConverted =  
 $\frac{25000}{3} \frac{\text{cm}}{\text{min}}$ 
```

### Perform Element-Wise Conversion

Create a symbolic vector for several measurements with multiple units.

```
syms x y
u = symunit;
v = 5*u.km/u.hr + x*u.cm/u.min;
measurements = [v; y*u.lb; 5*u.ft]
```

```
measurements =

$$\begin{pmatrix} 5 \frac{\text{km}}{\text{h}} + x \frac{\text{cm}}{\text{min}} \\ y \text{ lbm} \\ 5 \text{ ft} \end{pmatrix}$$

```

To convert the units in each element of this vector, you can use the `arrayfun` function and specify the target units as a vector, with each element corresponding to each measurement. For example, convert the mixed units of kilometers per hour and centimeters per minute in the first element to meters per second, convert the unit of pounds in the second element to grams, and convert the unit of feet in the third element to inches.

```
targetUnits = [u.m/u.s; u.g; u.in];
convertedMeas = arrayfun(@(vals,units) ...
    unitConvert(vals,units),measurements,targetUnits)
```

```
convertedMeas =

$$\begin{pmatrix} \frac{25}{18} \frac{\text{m}}{\text{s}} + \frac{x}{6000} \frac{\text{m}}{\text{s}} \\ \frac{45359237}{100000} y \text{ g} \\ 60 \text{ in} \end{pmatrix}$$

```

### Convert Units to Conventional Unit System

Instead of converting to specific units, you can convert the units in an input expression to the base units or derived units of another conventional unit system, such as SI, CGS, US, GU, ESU, or EMU. You can also convert the input units to the seven defining constants of the SI unit system.

Convert 5 meters to the base units of the US unit system. `unitConvert` returns the result in feet.

```
u = symunit;
length = 5*u.m
```

```
length = 5 m
```

```
lengthInUS = unitConvert(length, "US")
```

```
lengthInUS =

$$\frac{6250}{381} \text{ ft}$$

```



If you specify the new unit system as "SI", you can convert the input to the seven SI-defining constants. For example, convert 5 meters to the SI-defining constants. Here, the length is expressed in terms of the speed of light in vacuum and the caesium hyperfine frequency.

```
lengthInSIConst = unitConvert(length, "SI", "constants")
```

```
lengthInSIConst =  
     $\frac{3283082775}{21413747} \frac{c_0}{\Delta\nu_{\text{Cs}}}$ 
```

Next, convert 10 newtons to derived units in CGS by specifying the unit mode as "derived". The result is in dynes. Repeat the conversion without the unit mode "derived" to get a result in the base units.

```
F = 10*u.N;  
FInCGSDerived = unitConvert(F, "CGS", "derived")
```

```
FInCGSDerived = 1000000 dyn
```

```
FInCGSBase = unitConvert(F, "CGS")
```

```
FInCGSBase =  
     $1000000 \frac{\text{cm g}}{\text{s}^2}$ 
```

## Convert Units to Custom Unit System

Define a custom unit system for atomic units (au) to use for unit conversions. First, define the base units based on this table.

| Dimension       | Unit                                                            | Implementation                    |
|-----------------|-----------------------------------------------------------------|-----------------------------------|
| Mass            | Electron rest mass $m_e$                                        | <code>u.m_e</code>                |
| Electric charge | Electron charge $e$                                             | <code>u.e</code>                  |
| Length          | Bohr radius $a_0$                                               | <code>u.a_0</code>                |
| Time            | Reduced Planck constant $\hbar$ divided by Hartree energy $E_h$ | Define using <code>newUnit</code> |

```
u = symunit;  
newUnit("t_au", u.h_bar/u.E_h);  
baseCustom = [u.m_e u.e u.a_0 u.t_au]
```

```
baseCustom = (m_e e a_0 t_au)
```

Next, define the derived units based on this table.

| Dimension        | Unit                            | Implementation       |
|------------------|---------------------------------|----------------------|
| Angular momentum | Reduced Planck constant $\hbar$ | <code>u.h_bar</code> |
| Energy           | Hartree energy $E_h$            | <code>u.E_h</code>   |

| Dimension              | Unit                                                | Implementation       |
|------------------------|-----------------------------------------------------|----------------------|
| Electric dipole moment | Electron charge $e$ multiplied by Bohr radius $a_0$ | Define using newUnit |
| Magnetic dipole moment | Bohr magneton                                       | Define using newUnit |
| Electric potential     | Hartree energy $E_h$ divided by electron charge $e$ | Define using newUnit |

```
newUnit("p_au", u.e*u.a_0);
newUnit("m_au", u.e*u.h_bar/(2*u.me));
newUnit("V_au", u.E_h/u.e);
derivedCustom = [u.h_bar u.E_h u.p_au u.m_au u.V_au]
```

```
derivedCustom = ( $\hbar$  Eh pau mau Vau)
```

Finally, define the custom unit system for the atomic units.

```
newUnitSystem("atomicUnits", baseCustom, derivedCustom)
```

```
ans =
"atomicUnits"
```

Now, convert the properties of a proton to the base units of the atomic units.

```
proton = [1.672621925e-27*u.kg;           % mass
          1.602176634e-19*u.C;           % charge
          5.4e-24*u.e*u.cm;              % electric dipole moment
          1.410606795e-26*u.m^2*u.A;      % magnetic dipole moment
          0.5*u.h_bar                     % spin angular momentum
        ];
proton = unitConvert(proton, "atomicUnits", "base")
```

```
proton =

$$\begin{pmatrix} 1836.1526748780789486223496324943 m_e \\ 0.99999999999999993368785457223212 e \\ 0.00000000000000010204521072979158739839711133946 a_0 e \\ \frac{0.0023892317923137943916446667268902}{\pi} \frac{a_0^2 e}{t_{au}} \\ \frac{4.9348022005629884193493647544533}{\pi^2} \frac{a_0^2 m_e}{t_{au}} \end{pmatrix}$$

```

You can also convert the properties of the proton to the derived units of the atomic units.

```
proton = unitConvert(proton, "atomicUnits", "derived")
```

```
proton =

$$\begin{pmatrix} 1836.1526748780789486223496324943 m_e \\ 0.99999999999999993368785457223212 e \\ 0.00000000000000010204521072979158739839711133946 p_{au} \\ 0.00048415958638447923027051145918036 \pi m_{au} \\ 0.5 \hbar \end{pmatrix}$$

```

After completing the calculations, remove the unit system and the added units.

```
removeUnitSystem("atomicUnits")
removeUnit([u.t_au u.p_au u.m_au u.V_au])
```

## Convert Absolute Temperatures and Temperature Differences

By default, temperatures are assumed to represent temperature differences. For example, `5*u.Celsius` represents a temperature difference of 5 degrees Celsius. This assumption allows arithmetic operations on temperature values and conversion between temperature scales.

To represent absolute temperatures, use kelvin so that you do not have to distinguish an absolute temperature from a temperature difference.

Convert 23 degrees Celsius to K, treating the temperature first as a temperature difference and then as an absolute temperature.

```
u = symunit;
T = 23*u.Celsius;
diffK = unitConvert(T,u.K)

diffK = 23 K

absK = unitConvert(T,u.K,Temperature="absolute")

absK =
     $\frac{5923}{20}$  K
```

## Input Arguments

### expr — Input expression

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix | symbolic multidimensional array

Input expression, specified as a symbolic number, variable, expression, function, vector, matrix, or multidimensional array. If the conversion is not possible, `unitConvert` returns the input expression without converting it.

### units — Units to convert input to

symbolic unit | vector of symbolic units

Units to convert the input to, specified as a symbolic unit or vector of symbolic units.

A symbolic unit can be a single symbolic unit, like `u.kg`, or a compound unit, like `u.m/u.s^2`.

### unitSystem — Unit system to convert input to

"SI" | "CGS" | "US" | "GU" | "ESU" | "EMU" | symbolic unit system

Unit system to convert the input to, specified as one of these values:

- "SI" - International System of Units
- "CGS" - Centimeter-gram-second system of units

- "US" - United States customary units
- "GU" - Gaussian units
- "ESU" - Electrostatic units
- "EMU" - Electromagnetic units
- symbolic units system - Custom unit system defined by using the `newUnitSystem` function.

**unitMode — Conversion mode for new unit system**

"base" (default) | "derived" | "constants"

Conversion mode for the new unit system, specified as one of these values:

- "base" - Convert to the base units of the new unit system.
- "derived" - Convert to the derived units of the new unit system.
- "constants" - If the new unit system is "SI", then convert the units in the input expression to the seven SI-defining constants.

**tempMode — Conversion mode for temperature**

"difference" (default) | "absolute"

Conversion mode for temperature, specified as one of these values:

- "difference" - Convert input temperatures as temperature differences.
- "absolute" - Convert input temperatures as absolute temperatures.

## Output Arguments

**convertedExpr — Output expression in converted units**

symbolic number | symbolic variable | symbolic expression | symbolic function | symbolic vector | symbolic matrix | symbolic multidimensional array

Output expression in converted units, returned as a symbolic number, variable, expression, function, vector, matrix, or multidimensional array.

## Tips

- When you use symbolic units, the value of 0 multiplied by a symbolic unit, such as `0*u.Celsius`, is returned as a dimensionless 0. To preserve the unit when multiplying a symbolic unit by 0, use a cell array to represent the zero measurement.

For example, you can define 0 degrees Celsius as a cell array and convert it to degrees Fahrenheit by using the `unitConvert` function.

```
u = symunit;  
tC = {0,u.Celsius};  
tF = unitConvert(tC,u.Fahrenheit,Temperature="absolute")  
  
tF =  
32*[Fahrenheit]
```

## Version History

Introduced in R2018b

## See Also

### Functions

`baseUnits` | `derivedUnits` | `newUnit` | `newUnitSystem` | `symunit`

### Topics

“Units of Measurement Tutorial” on page 2-47

“Unit Conversions and Unit Systems” on page 2-53

“Units and Unit Systems List” on page 2-60

### External Websites

The International System of Units (SI)

## unitInfo

Information on units of measurement

### Syntax

```
unitInfo(unit)
unitInfo(dim)
unitInfo
```

```
A = unitInfo( __ )
```

### Description

`unitInfo(unit)` returns information for the symbolic unit `unit`.

`unitInfo(dim)` returns available units for the dimension `dim`.

`unitInfo` returns a list of available physical dimensions.

`A = unitInfo( __ )` returns the output in `A` using any of the input arguments in the previous syntaxes. Dimensions are returned as strings, and units are returned as symbolic units.

### Examples

#### Find Information on Units and Dimensions

Find information on unit `u.Wb` where `u = symunit`. The `unitInfo` function specifies that `Wb` is the SI unit of magnetic flux.

```
u = symunit;
unitInfo(u.Wb)
```

```
weber - a physical unit of magnetic flux. ['SI']
```

```
Get all units for measuring 'MagneticFlux' by calling unitInfo('MagneticFlux').
SI units accept all SI prefixes. For details, see SI Unit Prefixes List.
```

Find all available units for 'MagneticFlux' as described.

```
unitInfo('MagneticFlux')
```

```
All units of dimension 'MagneticFlux':
```

```
abWb - abweber
Mx - maxwell
phi_0 - magnetic flux quantum
statWb - statweber
Wb - weber ['SI']
```

```
Get the base SI units of any unit above by calling rewrite(<unit>,'SI').
SI units accept all SI prefixes. For details, see SI Unit Prefixes List.
```

`unitInfo` also returns information on compound units. Find information on `u.m/u.s^2`.

```
unitInfo(u.m/u.s^2)
```

meter per square second - a physical unit of acceleration. ['SI']

Get all units for measuring 'Acceleration' by calling `unitInfo('Acceleration')`. SI units accept all SI prefixes. For details, see SI Unit Prefixes List.

## Find All Physical Dimensions Available

Return all available dimensions using `unitInfo` without input arguments.

```
unitInfo
```

```
"AbsorbedDose"
"AbsorbedDoseOrDoseEquivalent"
"AbsorbedDoseRate"
"Acceleration"
...
...
"Length"
"Luminance"
"LuminousEfficacy"
...
...
"Torque"
"Velocity"
"Volume"
```

## Use Information on Units and Dimensions

Store information returned by `unitInfo` for use by providing an output.

Store the dimension of `u.C`.

```
u = symunit;
dimC = unitInfo(u.C)
```

```
dimC =
    "ElectricCharge"
```

Find and store all units for the dimension `dimC`.

```
unitsEC = unitInfo(dimC)
```

```
unitsEC =
    [abC]
    [C]
    [e]
    [Fr]
    [statC]
```

Find information on the third unit of `unitsEC`.

```
unitInfo(unitsEC(3))
```

elementary charge - a physical unit of electric charge.

Get all units for measuring 'ElectricCharge' by calling `unitInfo('ElectricCharge')`.

Store [e]. Then, find the number of electrons in a coulomb of electric charge.

```
electronCharge = unitsEC(3);
numElectrons = simplify(u.C/electronCharge)
```

[illegible]

Show that approximately  $6.24 \times 10^{18}$  electrons are in a coulomb by converting the high-precision symbolic result to double.

```
numElectrons = double(numElectrons)
```

```
numElectrons =  
    6.2415e+18
```

## Input Arguments

**unit** — Unit name

symbolic unit | character vector | string

Unit name, specified as a symbolic unit, character vector, or string. If `unit` is not a known base or derived unit in “Units and Unit Systems List” on page 2-60, then `A = unitInfo(unit)` returns an empty string `''`.

Example: `unitInfo(u.m)` where `u = symunit`

**dim — Dimension**

character vector | string

Dimension, specified as a character vector or string.

Example: `unitInfo( 'Length' )`

## Version History

## Introduced in R2017b

## See Also

```
rewrite | simplify | symunit
```

## Topics

“Units of Measurement Tutorial” on page 2-47

“Unit Conversions and Unit Systems” on page 2-53

"Units and Unit Systems List" on page 2-60

## External Websites

## The International System of Units (SI)



# unitSystems

List available unit systems

## Syntax

unitSystems

## Description

unitSystems returns a row vector of available unit systems.

- To add custom unit systems, see `newUnitSystem`.
- To convert between unit systems, see “Unit Conversions and Unit Systems” on page 2-53.
- For available unit systems, see “Unit Systems List” on page 2-71.

## Examples

### Get Available Unit Systems

Get available unit systems by using `unitSystems`. Add a custom unit system and check that `unitSystems` lists it as available.

Find the unit systems available by default.

```
unitSystems
```

```
ans =  
    1×6 string array  
    "CGS"    "EMU"    "ESU"    "GU"    "SI"    "US"
```

Add a custom unit system that modifies the SI base units. For details, see `newUnitSystem` and “Unit Conversions and Unit Systems” on page 2-53.

```
u = symunit;  
SIUnits = baseUnits('SI');  
newUnits = subs(SIUnits,[u.m u.s],[u.km u.hr]);  
newUnitSystem('SI_km_hr',newUnits)
```

```
ans =  
    "SI_km_hr"
```

Check that the new unit system is available by using `unitSystems`.

```
unitSystems
```

```
ans =  
    1×7 string array  
    "CGS"    "EMU"    "ESU"    "GU"    "SI"    "SI_km_hr"    "US"
```

After calculations, remove the new unit system and check that it is unavailable.

```
removeUnitSystem('SI_km_hr');  
unitSystems  
  
ans =  
    1×6 string array  
    "CGS"    "EMU"    "ESU"    "GU"    "SI"    "US"
```

## Version History

**Introduced in R2017b**

### See Also

[baseUnits](#) | [derivedUnits](#) | [newUnitSystem](#) | [removeUnitSystem](#) | [rewrite](#) | [symunit](#)

### Topics

“Units of Measurement Tutorial” on page 2-47  
“Unit Conversions and Unit Systems” on page 2-53  
“Units and Unit Systems List” on page 2-60

### External Websites

The International System of Units (SI)

# vectorPotential

Vector potential of vector field

## Syntax

```
vectorPotential(V,X)
vectorPotential(V)
```

## Description

`vectorPotential(V,X)` computes the vector potential of the vector field on page 6-1620  $V$  with respect to the vector  $X$  in Cartesian coordinates. The vector field  $V$  and the vector  $X$  are both three-dimensional.

`vectorPotential(V)` returns the vector potential  $V$  with respect to a vector constructed from the first three symbolic variables found in  $V$  by `symvar`.

## Examples

### Compute Vector Potential of Field

Compute the vector potential of this row vector field with respect to the vector  $[x, y, z]$ :

```
syms x y z
vectorPotential([x^2*y, -1/2*y^2*x, -x*y*z], [x y z])

ans =
    -(x*y^2*z)/2
    -x^2*y*z
    0
```

Compute the vector potential of this column vector field with respect to the vector  $[x, y, z]$ :

```
syms x y z
f(x,y,z) = 2*y^3 - 4*x*y;
g(x,y,z) = 2*y^2 - 16*z^2+18;
h(x,y,z) = -32*x^2 - 16*x*y^2;
A = vectorPotential([f; g; h], [x y z])

A(x, y, z) =
    z*(2*y^2 + 18) - (16*z^3)/3 + (16*x*y*(y^2 + 6*x))/3
    2*y*z*(- y^2 + 2*x)
    0
```

### Test if Vector Potential Exists for Field

To check whether the vector potential exists for a particular vector field, compute the divergence of that vector field:

```
syms x y z
V = [x^2 2*y z];
divergence(V, [x y z])
```

```
ans =  
2*x + 3
```

If the divergence is not equal to 0, the vector potential does not exist. In this case, `vectorPotential` returns the vector with all three components equal to NaN:

```
vectorPotential(V, [x y z])
```

```
ans =  
NaN  
NaN  
NaN
```

## Input Arguments

### V — Vector field

3-D symbolic vector of symbolic expressions or functions (default)

Vector field, specified as a 3-D vector of symbolic expressions or functions.

### X — Input

vector of three symbolic variables

Input, specified as a vector of three symbolic variables with respect to which you compute the vector potential.

## More About

### Vector Potential of a Vector Field

The vector potential of a vector field  $V$  is a vector field  $A$ , such that:

$$V = \nabla \times A = \text{curl}(A)$$

## Tips

- The vector potential exists if and only if the divergence of a vector field  $V$  with respect to  $X$  equals 0. If `vectorPotential` cannot verify that  $V$  has a vector potential, it returns the vector with all three components equal to NaN.

## Version History

Introduced in R2012a

## See Also

`curl` | `diff` | `divergence` | `gradient` | `jacobian` | `hessian` | `laplacian` | `potential`

# vertcat

Concatenate symbolic arrays vertically

## Syntax

```
vertcat(A1,...,AN)
[A1;...;AN]
```

## Description

`vertcat(A1,...,AN)` vertically concatenates the symbolic arrays `A1,...,AN`. For vectors and matrices, all inputs must have the same number of columns. For multidimensional arrays, `vertcat` concatenates inputs along the first dimension. The remaining dimensions must match.

`[A1;...;AN]` is a shortcut for `vertcat(A1,...,AN)`.

## Examples

### Concatenate Two Symbolic Vectors Vertically

Concatenate the two symbolic vectors `A` and `B` to form a symbolic matrix.

```
A = sym('a%d',[1 4]);
B = sym('b%d',[1 4]);
vertcat(A,B)
```

```
ans =
[ a1, a2, a3, a4]
[ b1, b2, b3, b4]
```

Alternatively, you can use the shorthand `[A;B]` to concatenate `A` and `B`.

```
[A;B]

ans =
[ a1, a2, a3, a4]
[ b1, b2, b3, b4]
```

### Concatenate Multiple Symbolic Arrays Vertically

Concatenate multiple symbolic arrays into one symbolic matrix.

```
A = sym('a%d',[1 3]);
B = sym('b%d%d',[4 3]);
C = sym('c%d%d',[2 3]);
vertcat(C,A,B)
```

```
ans =
[ c11, c12, c13]
[ c21, c22, c23]
[ a1, a2, a3]
[ b11, b12, b13]
[ b21, b22, b23]
```

```
[ b31, b32, b33]
[ b41, b42, b43]
```

### Concatenate Multidimensional Arrays Vertically

Create the 3-D symbolic arrays A and B.

```
A = [2 4; 1 7; 3 3];
A(:,:,2) = [8 9; 4 5; 6 2];
A = sym(A)
B = [8 3; 0 2];
B(:,:,2) = [6 2; 3 3];
B = sym(B)
```

```
A(:,:,1) =
[ 2, 4]
[ 1, 7]
[ 3, 3]
A(:,:,2) =
[ 8, 9]
[ 4, 5]
[ 6, 2]
```

```
B(:,:,1) =
[ 8, 3]
[ 0, 2]
B(:,:,2) =
[ 6, 2]
[ 3, 3]
```

Use `vertcat` to concatenate A and B.

```
vertcat(A,B)
```

```
ans(:,:,1) =
[ 2, 4]
[ 1, 7]
[ 3, 3]
[ 8, 3]
[ 0, 2]
```

```
ans(:,:,2) =
[ 8, 9]
[ 4, 5]
[ 6, 2]
[ 6, 2]
[ 3, 3]
```

## Input Arguments

### A1, ..., AN — Input arrays

symbolic scalar variable | symbolic matrix variable | symbolic matrix function | symbolic vector | symbolic matrix | symbolic multidimensional array

Input arrays, specified as symbolic scalar variables, matrix variables, matrix functions, or vectors, matrices, or multidimensional arrays of symbolic scalar variables.

## Version History

Introduced before R2006a

### **R2022a: Concatenate symbolic matrix functions vertically**

The `vertcat` function accepts input arguments of type `symfunmatrix`.

### **R2021a: Concatenate symbolic matrix variables vertically**

The `vertcat` function accepts input arguments of type `symmatrix`.

## See Also

`cat` | `horzcat`

## vpa

Variable-precision arithmetic (arbitrary-precision arithmetic)

### Syntax

```
yVpa = vpa(y)
yVpa = vpa(y,d)

yVpa = vpa( ___,BoostPrecision=tf)
```

### Description

`yVpa = vpa(y)` uses variable-precision arithmetic (with arbitrary-precision floating-point numbers) to evaluate each element of the input `y` to at least `d` significant digits, where `d` is the value of the `digits` function. The default value of `digits` is 32.

`yVpa = vpa(y,d)` uses at least `d` significant digits instead of the value of `digits`.

`yVpa = vpa( ___,BoostPrecision=tf)`, when `y` is double precision, specifies whether to increase the precision of `y` when converting it to a symbolic number with variable precision. By default, when you pass a double-precision input to `vpa`, the function attempts to increase the precision during conversion. To preserve the original bit pattern of the double-precision input, set `BoostPrecision` to `false`. You can use this option in addition to any of the input argument combinations in previous syntaxes.

### Examples

#### Evaluate Symbolic Inputs Using Variable-Precision Arithmetic

Evaluate symbolic inputs using variable-precision floating-point arithmetic. By default, `vpa` calculates values to 32 significant digits.

Evaluate  $\pi$  to 32 significant digits.

```
p = sym(pi);
pVpa = vpa(p)

pVpa = 3.1415926535897932384626433832795
```

Evaluate a symbolic expression.

```
syms x
a = sym(1/3);
f = a*sin(2*p*x);
fVpa = vpa(f)

fVpa = 0.33333333333333333333333333333333 sin(6.283185307179586476925286766559 x)
```

Evaluate elements of symbolic vectors or matrices using variable-precision arithmetic.



```

V = [x/p a^3];
VVpa = vpa(V)

VVpa = (0.31830988618379067153776752674503 x 0.037037037037037037037037037037)

M = [sin(p) cos(p/5); exp(p*x) x/log(p)];
MVpa = vpa(M)

MVpa =
      0      0.80901699437494742410229341718282
      e      0.87356852683023186835397746476334 x
(3.1415926535897932384626433832795 x

```

### Change Precision Used by vpa

By default, `vpa` evaluates inputs to 32 significant digits. You can change the number of significant digits by using the `digits` function.

Approximate the symbolic expression  $100001/10001$  using seven significant digits. Set the number of significant digits using `digits`, and save the original value of `digits`. The `vpa` function returns only five significant digits, which can mean the subsequent digits are zeros.

```

digitsOld = digits(7);
y = sym(100001)/10001;
yVpa = vpa(y)

```

```
yVpa = 9.9991
```

Check if the subsequent digits are zeros by using a higher precision value of 25. The result shows that the subsequent digits are in fact zeros that are part of a repeating decimal.

```

digits(25)
yVpa = vpa(y)

```

```
yVpa = 9.999100089991000899910009
```

Alternatively, to override `digits` for a single `vpa` call, change the precision by specifying the second argument.

Find  $\pi$  to 100 significant digits by specifying the second argument.

```
pVpa = vpa(sym(pi),100)
```

```
pVpa = 3.141592653589793238462643383279502884197169399375105820974944592307816406286208998628034825342
```

Restore the original precision value for further calculations.

```
digits(digitsOld)
```

### Numerically Approximate Symbolic Results

While symbolic results are exact, they might not be in a convenient form. You can use `vpa` to numerically approximate exact symbolic results.

Solve a high-degree polynomial for its roots using `solve`. The `solve` function cannot symbolically solve the high-degree polynomial and represents the roots using `root`.

```
syms x
y = solve(x^4 - x + 1, x)
```

```
y =

$$\begin{pmatrix} \text{root}(z^4 - z + 1, z, 1) \\ \text{root}(z^4 - z + 1, z, 2) \\ \text{root}(z^4 - z + 1, z, 3) \\ \text{root}(z^4 - z + 1, z, 4) \end{pmatrix}$$

```

Use `vpa` to numerically approximate the roots.

```
yVpa = vpa(y)
```

```
yVpa =

$$\begin{pmatrix} 0.72713608449119683997667565867496 - 0.43001428832971577641651985839602 i \\ 0.72713608449119683997667565867496 + 0.43001428832971577641651985839602 i \\ -0.72713608449119683997667565867496 - 0.93409928946052943963903028710582 i \\ -0.72713608449119683997667565867496 + 0.93409928946052943963903028710582 i \end{pmatrix}$$

```

### vpa Uses Guard Digits to Maintain Precision

The `digits` function specifies the minimum number of significant digits to use in variable-precision arithmetic. Alternatively, you can use the `vpa(y, d)` syntax to convert an input `y` to at least `d` significant digits. Internally, `vpa` uses more digits than specified. These additional digits are called *guard digits* because they guard against round-off errors in subsequent calculations.

Approximate  $\pi$  using four significant digits.

```
pi_4digits = vpa(sym(pi),4)
```

```
pi_4digits = 3.142
```

Next, use the previous result to approximate  $\pi$  to six significant digits. The result shows that the `vpa` function used more than four digits during the initial conversion of  $\pi$ , demonstrating the use of guard digits.

```
pi_6digits = vpa(pi_4digits,6)
```

```
pi_6digits = 3.14159
```

### Maintain Precision of Common Double-Precision Inputs

The double-precision format stores a number using 64 bits of memory, resulting in limited numerical precision. When you call `vpa` on a double-precision input, `vpa` generally cannot restore any lost precision beyond the typical 16 significant digits of the double-precision representation, even though `vpa` returns more digits than the original double-precision value. However, `vpa` uses an algorithm

that can recognize and increase the precision of operations of the form  $\frac{p}{q}$ ,  $\frac{p\pi}{q}$ ,  $\left(\frac{p}{q}\right)^{\frac{1}{2}}$ ,  $2^q$ , and  $10^q$ , where  $p$  and  $q$  are modest-sized integers.

First, demonstrate that `vpa` cannot increase the precision for a double-precision evaluation that involves the `log` function. Call `vpa` on a double-precision result and the same symbolic result.

```
yDouble = log(3);
ySym = log(sym(3));
yDoubleToVpa = vpa(yDouble)

yDoubleToVpa = 1.0986122886681097821082175869378

ySymToVpa = vpa(ySym)

ySymToVpa = 1.0986122886681096913952452369225

d = yDoubleToVpa - ySymToVpa

d = 0.000000000000000090712972350015300422166375941649
```

As expected, the double-precision result differs from the symbolic result after the 16th decimal place.

Demonstrate that `vpa` increases the precision for expressions of the form  $\frac{p}{q}$ ,  $\frac{p\pi}{q}$ ,  $\left(\frac{p}{q}\right)^{\frac{1}{2}}$ ,  $2^q$ , and  $10^q$ , where  $p$  and  $q$  are modest-sized integers, by finding the difference between the `vpa` call on the double-precision result and on the exact symbolic result. The differences are 0.0 showing that `vpa` restores lost precision for the double-precision input.

```
d = vpa(1/3) - vpa(1/sym(3))

d = 0.0

d = vpa(pi) - vpa(sym(pi))

d = 0.0

d = vpa(1/sqrt(2)) - vpa(1/sqrt(sym(2)))

d = 0.0

d = vpa(2^66) - vpa(2^sym(66))

d = 0.0

d = vpa(10^25) - vpa(10^sym(25))

d = 0.0
```

Starting in R2026a, you can choose to use `vpa` without increasing the precision for double-precision expressions of the form  $\frac{p}{q}$ ,  $\frac{p\pi}{q}$ ,  $\left(\frac{p}{q}\right)^{\frac{1}{2}}$ ,  $2^q$ , and  $10^q$  by specifying the `BoostPrecision` option as `false`. For example, convert the double-precision value of  $1/3$  to variable precision by preserving the bit pattern of the double representation, without restoring any lost precision after the 16th decimal place. The result is not equal to calling `vpa` on the exact symbolic result.

```
yDoubleToVpa = vpa(1/3,BoostPrecision=false)

yDoubleToVpa = 0.33333333333333331482961625624739
```

```
ySymToVpa = vpa(1/sym(3))
ySymToVpa = 0.33333333333333333333333333333333
d = yDoubleToVpa - ySymToVpa
d = -0.0000000000000000018503717077085942340393891746679
```

### Identify Hidden Errors in Operations with Different Precision

When you perform operations on numbers with different precision, the results can contain hidden errors.

For example, evaluate  $1/10$  with 6-digit precision and with 5-digit precision.

```
a = vpa(1/10,6)
a = 0.1
b = vpa(1/10,5)
b = 0.1
```

Although the displayed values of **a** and **b** look the same, these two variables are not exactly equal because they store different digits beyond the 6th and 5th significant digits, respectively. Check their equality by computing the difference **a - b**.

```
roundoff = a - b
roundoff = 0.00000000000000053290705182007513940334320068359
digits
Digits = 32
```

The result of **a - b** is not zero. The reason is that **a** is accurate to 6 digits and **b** is accurate to 5 digits, but the subtraction uses the default 32 digits of precision set by **digits**.

To show the different values of **a** and **b**, convert their original values to 32 significant digits.

```
a_32digits = vpa(a,32)
a_32digits = 0.099999999999999644728632119949907
b_32digits = vpa(b,32)
b_32digits = 0.0999999999999994315658113919198513
```

Compute the difference between these converted values.

```
roundoff = a_32digits - b_32digits
roundoff = 0.00000000000000053290705182007513940334320068359
```

### Use String Input to Accurately Represent Numbers

Approximate the number 1111111111111111 with 32 significant digits by using `vpa`. The result is not accurate after the 16th digit because when you enter a number in MATLAB, it is converted to the nearest representable double-precision value, which might not be exactly equal to the number you typed.

```
xVpa = vpa(1111111111111111)
```

```
xVpa = 111111111111111104.0
```

To accurately represent the number, use string input. Here, `vpa` approximates the number 1111111111111111 accurately to 32 significant digits.

```
xVpa = vpa("1111111111111111")
```

```
xVpa = 1111111111111111.0
```

### Evaluate Symbolic Matrix Variable Using Variable-Precision Arithmetic

Create a symbolic expression  $S$  that represents  $\sin\left(\begin{bmatrix} \pi & \frac{\pi}{2} \\ \frac{\pi}{2} & \frac{\pi}{3} \end{bmatrix} \mathbf{X}\right)$ , where  $\mathbf{X}$  is a 2-by-1 symbolic matrix variable.

```
syms X [2 1] matrix
S = sin(hilb(2)*pi*X)
```

```
S =
    sin(Σ1 X)
```

where

$$\Sigma_1 = \begin{pmatrix} \pi & \frac{\pi}{2} \\ \frac{\pi}{2} & \frac{\pi}{3} \end{pmatrix}$$

Evaluate the expression using variable-precision arithmetic.

```
SVpa = vpa(S)
```

```
SVpa =
    (sin(3.1415926535897932384626433832795 X1 + 1.5707963267948966192313216916398 X2))
    (sin(1.5707963267948966192313216916398 X1 + 1.0471975511965977461542144610932 X2))
```

### Preserve Bit Pattern of Double-Precision Inputs

When converting a double-precision number to a symbolic number, `vpa` by default attempts to increase the precision, which can lead to unintended results. Specifically, `vpa` might interpret the least significant digits of double-precision numbers as round-off errors.

For example, consider two adjacent double-precision numbers: 1 and the next larger double-precision number after 1.

```
x = 1;
y = 1 + eps;
```

If you convert these two numbers using `vpa`, the converted numbers are equal and their difference is zero. In this case, the least significant bit of `y` is not preserved because `vpa` treats it as a round-off error.

```
d = vpa(y) - vpa(x)
```

```
d = 0.0
```

However, in double-precision representation, these two numbers have different bit patterns, which can be observed in hexadecimal format.

```
x_hexStr = num2hex(x)
```

```
x_hexStr =
'3ff0000000000000'
```

```
y_hexStr = num2hex(y)
```

```
y_hexStr =
'3ff0000000000001'
```

To preserve the bit pattern when converting double-precision inputs with `vpa`, specify the `BoostPrecision` option as `false`. Now, the difference between the converted numbers is closer to `eps`.

```
d = vpa(y,BoostPrecision=false) - vpa(x,BoostPrecision=false)
```

```
d = 0.00000000000000022204460492503130808472633361816
```

```
format longE
eps
```

```
ans =
    2.220446049250313e-16
```

Although `vpa` preserves the bit pattern of the `double` numbers, it still uses the default 32 significant digits and attempts to fill in the additional digits beyond the significant digits of the `double` numbers. To make the difference between the converted numbers equal to `eps`, you can set the number of significant digits to 16 for the `vpa` operations.

```
digits(16)
d = vpa(y,BoostPrecision=false) - vpa(x,BoostPrecision=false)
```

```
d = 0.0000000000000002220446049250313
```

Finally, restore the precision used by `vpa` to 32 digits and the output display format to the default.

```
digits(32)
format default
```

## Input Arguments

### y — Input to evaluate

scalar | vector | matrix | multidimensional array | symbolic number | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic expression | symbolic function | symbolic character vector | symbolic matrix variable

Input to evaluate, specified as a scalar, vector, matrix, or multidimensional array of numeric values or strings, or as a symbolic number, vector, matrix, multidimensional array, expression, function, character vector, or matrix variable.

- When you enter a number in MATLAB, it is converted to the nearest representable double-precision value, which might not be exactly equal to the number you typed. To accurately represent the number, you can specify `y` as a string input.
- If you specify `y` as a single-precision number, `vpa` first converts it to double-precision before converting the number to variable-precision.

Data Types: double | single | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64 | char | string | sym | symfun | symmatrix  
Complex Number Support: Yes

### d — Number of significant digits

positive integer scalar

Number of significant digits, specified as a positive integer scalar. If you do not specify this argument, then the `digits` function determines the number of significant digits used by `vpa`.

If `d` is not an integer, `vpa` rounds it to the nearest integer using the `round` function.

### tf — Option to increase precision for double-precision input

true or 1 (default) | false or 0

Option to increase the precision for double-precision input, specified as a numeric or logical 1 (true) or 0 (false). By default, when `y` is double precision, `vpa` attempts to increase the precision of `y` when converting it to a symbolic number with variable precision. To preserve the original bit pattern of the double-precision input, specify this argument as `false`.

If `y` is not double precision, this argument has no effect.

## Output Arguments

### yVpa — Variable-precision output

symbolic number | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic expression | symbolic function

Variable-precision output, returned as a symbolic number, vector, matrix, multidimensional array, expression, or function.

- For almost all input data types (such as `sym`, `symmatrix`, `string`, `double`, `single`, `int64`, and so on), `vpa` returns the output as data type `sym`.
- If the input is a symbolic function of type `symfun`, then `vpa` returns the output as data type `symfun`. For example, `syms f(x); f(x) = pi*x; g = vpa(f)` returns the output `g` as type `symfun`.

- If the input is an evaluated symbolic function of type `sym`, such as `g = vpa(f(x))`, then `vpa` returns the output as data type `sym`.

Data Types: `sym` | `symfun`

## Tips

- `vpa` does not convert fractions in the exponent to floating point. For example, `vpa(a^sym(2/5))` returns `a^(2/5)`.
- `vpa` internally uses more digits than the number of digits specified by `digits`. These extra digits guard against round-off errors in subsequent calculations. For an example, see “`vpa` Uses Guard Digits to Maintain Precision” on page 6-1626.
- When you call `vpa` on a numeric input, such as `1/3`, `2^(-5)`, or `sin(pi/4)`, the expression is first evaluated as a double-precision number, providing 64 bits or approximately 16 digits of precision. `vpa` then increases the precision to the specified number of significant digits. For more accurate results, define symbolic inputs instead by using `sym`. For example, to approximate `exp(1)` to 32 significant digits, use `vpa(exp(sym(1)))`.
- By default, `vpa` increases the precision for numeric inputs that match the forms  $p/q$ ,  $p\pi/q$ ,  $(p/q)^{1/2}$ ,  $2^q$ , and  $10^q$ , where  $p$  and  $q$  are modest-sized integers. For more information, see “Maintain Precision of Common Double-Precision Inputs” on page 6-1626.
- Variable-precision arithmetic is different from IEEE Floating-Point Standard 754 in these ways:
  - Inside computations, division by zero throws an error.
  - The exponent range is larger than in any predefined IEEE mode. `vpa` underflows below approximately  $10^{(-323228496)}$ .
  - Denormalized numbers are not implemented.
  - Zeros are not signed.
  - The number of *binary* digits in the mantissa of a result might differ between variable-precision arithmetic and IEEE predefined types.
  - There is only one NaN representation. No distinction is made between quiet and signaling NaN.
  - No floating-point number exceptions are available.

## Version History

Introduced before R2006a

### R2026a: Preserve bit pattern when converting double-precision numbers

When you pass a double-precision number to `vpa`, by default, the function attempts to increase the precision of the number when converting it to variable precision. To preserve the original bit pattern of the double-precision number without increasing its precision, you can set the `BoostPrecision` option to `false`. For an example, see “Preserve Bit Pattern of Double-Precision Inputs” on page 6-1629.

### R2022b: Evaluate symbolic matrix variables with variable-precision arithmetic

You can evaluate a symbolic matrix variable of type `symmatrix` with variable-precision arithmetic. The result is a symbolic expression with variable-precision numbers and scalar variables of type `sym`.



For an example, see “Evaluate Symbolic Matrix Variable Using Variable-Precision Arithmetic” on page 6-1629.

### **R2018a: Support for character vectors has been removed**

*Errors starting in R2018a*

Support for character vectors that do not define a number has been removed. Instead, first create symbolic numbers and variables using `sym` and `syms`, and then use operations on them. For example, use `vpa((1 + sqrt(sym(5)))/2)` instead of `vpa('(1 + sqrt(5))/2')`.

## **See Also**

### **Functions**

`digits` | `double` | `root` | `vpaintegral`

### **Topics**

“Increase Precision of Numeric Calculations” on page 2-36

“Recognize and Avoid Round-Off Errors” on page 3-382

“Increase Speed by Reducing Precision” on page 3-386

“Choose Numeric or Symbolic Arithmetic” on page 2-32

“Change Output Format of Symbolic and Variable-Precision Arithmetic” on page 2-8

## vpaintegral

Numerical integration using variable precision

### Syntax

```
vpaintegral(f,a,b)
vpaintegral(f,x,a,b)
vpaintegral( ___,Name,Value)
```

### Description

`vpaintegral(f,a,b)` numerically approximates  $f$  from  $a$  to  $b$ . The default variable  $x$  in  $f$  is found by `symvar`.

`vpaintegral(f,[a b])` is equal to `vpaintegral(f,a,b)`.

`vpaintegral(f,x,a,b)` performs numerical integration using the integration variable  $x$ .

`vpaintegral( ___,Name,Value)` uses additional options specified by one or more `Name,Value` pair arguments.

### Examples

#### Numerically Integrate Symbolic Expression

Numerically integrate the symbolic expression  $x^2$  from 1 to 2.

```
syms x
vpaintegral(x^2, 1, 2)
```

```
ans =
2.33333
```

#### Numerically Integrate Symbolic Function

Numerically integrate the symbolic function  $y(x) = x^2$  from 1 to 2.

```
syms y(x)
y(x) = x^2;
vpaintegral(y, 1, 2)
```

```
ans =
2.33333
```

#### High-Precision Numerical Integration

`vpaintegral` uses variable-precision arithmetic while the MATLAB `integral` function uses double-precision arithmetic. Using the default values of tolerance, `vpaintegral` can handle values that cause the MATLAB `integral` function to overflow or underflow.

Integrate `besseli(5,25*u).*exp(-u*25)` by using both `integral` and `vpaintegral`. The `integral` function returns NaN and issues a warning while `vpaintegral` returns the correct result.

```

syms u x
f = besseli(5,25*x).*exp(-x*25);
fun = @(u)besseli(5,25*u).*exp(-u*25);

usingIntegral = integral(fun, 0, 30)
usingVpaintegral = vpaintegral(f, 0, 30)

Warning: Infinite or Not-a-Number value encountered.
usingIntegral =
    NaN

usingVpaintegral =
    0.688424

```

### Increase Precision Using Tolerances

The `digits` function does not affect `vpaintegral`. Instead, increase the precision of `vpaintegral` by decreasing the integration tolerances. Conversely, increase the speed of numerical integration by increasing the tolerances. Control the tolerance used by `vpaintegral` by changing the relative tolerance `RelTol` and absolute tolerance `AbsTol`, which affect the integration through the condition

$$|Q - I| \leq \max(AbsTol, |Q| \cdot RelTol)$$

where  $Q$  = Calculated Integral  
 $I$  = Exact Integral.

Numerically integrate `besselj(0,x)` from 0 to  $\pi$ , to 32 significant figures by setting `RelTol` to  $10^{(-32)}$ . Turn off `AbsTol` by setting it to 0.

```

syms x
vpaintegral(besselj(0,x), [0 pi], 'RelTol', 1e-32, 'AbsTol', 0)

ans =
1.3475263146739901712314731279612

```

Using lower tolerance values increases precision at the cost of speed.

### Complex Path Integration Using Waypoints

Integrate  $1/(2z-1)$  over the triangular path from 0 to  $1+1i$  to  $1-1i$  back to 0 by specifying waypoints.

```

syms z
vpaintegral(1/(2*z-1), [0 0], 'Waypoints', [1+1i 1-1i])

ans =
- 8.67362e-19 - 3.14159i

```

Reversing the direction of the integral, by changing the order of the waypoints and exchanging the limits, changes the sign of the result.

### Multiple Integrals

Perform multiple integration by nesting calls to `vpaintegral`. Integrate

$$\int_{-1}^2 \int_1^3 xy \, dx \, dy.$$

```
syms x y
vpaintegral(vpaintegral(x*y, x, [1 3]), y, [-1 2])

ans =
6.0
```

The limits of integration can be symbolic expressions or functions. Integrate over the triangular region  $0 \leq x \leq 1$  and  $|y| < x$  by specifying the limits of the integration over  $y$  in terms of  $x$ .

```
vpaintegral(vpaintegral(sin(x-y)/(x-y), y, [-x x]), x, [0 1])

ans =
0.89734
```

## Input Arguments

### **f** — Expression or function to integrate

symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Expression or function to integrate, specified as a symbolic number, variable, vector, matrix, multidimensional array, function, or expression.

### **a, b** — Limits of integration

list of two numbers | list of two symbolic numbers | list of two symbolic variables | list of two symbolic functions | list of two symbolic expressions

Limits of integration, specified as a list of two numbers, symbolic numbers, symbolic variables, symbolic functions, or symbolic expressions.

### **x** — Integration variable

symbolic variable

Integration variable, specified as a symbolic variable. If  $x$  is not specified, the integration variable is found by `symvar`.

## Name-Value Pair Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `'RelTol', 1e-20`

### **RelTol** — Relative error tolerance

1e-6 (default) | positive real number

Relative error tolerance, specified as a positive real number. The default is `1e-6`. The `RelTol` argument determines the accuracy of the integration only if  $RelTol \cdot |Q| > AbsTol$ , where  $Q$  is the calculated integral. In this case, `vpaintegral` satisfies the condition  $|Q - I| \leq RelTol \cdot |Q|$ , where  $I$  is the exact integral value. To use only `RelTol` and turn off `AbsTol`, set `AbsTol` to `0`.

Example: `1e-8`

**AbsTol — Absolute error tolerance**

1e-10 (default) | non-negative real number

Absolute error tolerance, specified as a non-negative real number. The default is 1e-10. `AbsTol` determines the accuracy of the integration if  $AbsTol > RelTol \cdot |Q|$ , where  $Q$  is the calculated integral. In this case, `vpaintegral` satisfies the condition  $|Q - I| \leq AbsTol$ , where  $I$  is the exact integral value. To turn off `AbsTol` and use only `RelTol`, set `AbsTol` to 0.

Example: 1e-12

**Waypoints — Integration path**

vector of numbers | vector of symbolic numbers | vector of symbolic expressions | vector of symbolic functions

Integration path, specified as a vector of numbers, or as a vector of symbolic numbers, expressions, or functions. `vpaintegral` integrates along the sequence of straight-line paths (lower limit to the first waypoint, from the first to the second waypoint, and so on) and finally from the last waypoint to the upper limit. For contour integrals, set equal lower and upper limits and define the contour using waypoints.

**MaxFunctionCalls — Maximum evaluations of input**

10^5 (default) | positive integer | positive symbolic integer

Maximum evaluations of input, specified as a positive integer or a positive symbolic integer. The default value is 10^5. If the number of evaluations of `f` is greater than `MaxFunctionCalls`, then `vpaintegral` throws an error. For unlimited evaluations, set `MaxFunctionCalls` to `Inf`.

**Tips**

- Ensure that the input is integrable. If the input is not integrable, the output of `vpaintegral` is unpredictable.
- The `digits` function does not affect `vpaintegral`. To increase precision, use the `RelTol` and `AbsTol` arguments instead.

**Version History**

Introduced in R2016b

**See Also**

diff | int | integral | vpa

**Topics**

"Integration" on page 3-186

## vpasolve

Solve symbolic equations numerically

### Syntax

```
S = vpasolve(eqn,var)
S = vpasolve(eqn,var,init_param)

Y = vpasolve(eqns,vars)
Y = vpasolve(eqns,vars,init_param)

[y1,...,yN] = vpasolve(eqns,vars)
[y1,...,yN] = vpasolve(eqns,vars,init_param)

___ = vpasolve( ___,Random=true)
```

### Description

`S = vpasolve(eqn,var)` numerically solves the equation `eqn` for the variable `var` using variable-precision arithmetic and returns the solutions with 32 significant digits by default. To change the number of significant digits, use the `digits` function.

If you do not specify `var`, `vpasolve` solves for the default variable as determined by `symvar`. For example, `vpasolve(x + 1 == 2)` numerically solves the equation  $x + 1 = 2$  for  $x$ .

`S = vpasolve(eqn,var,init_param)` numerically solves the equation `eqn` for the variable `var` using the initial guess or search range `init_param`.

`Y = vpasolve(eqns,vars)` numerically solves the system of equations `eqns` for the variables `vars`. This syntax returns a structure array `Y` that contains the solutions. The fields in the structure array correspond to the variables specified by `vars`. If you do not specify `vars`, `vpasolve` solves for the default variables determined by `symvar`.

`Y = vpasolve(eqns,vars,init_param)` numerically solves the system of equations `eqns` for the variables `vars` using the initial guess or search range `init_param`.

`[y1,...,yN] = vpasolve(eqns,vars)` numerically solves the system of equations `eqns` for the variables `vars`. This syntax assigns the solutions to the variables `y1,...,yN`. If you do not specify `vars`, `vpasolve` solves for the default variables determined by `symvar`.

`[y1,...,yN] = vpasolve(eqns,vars,init_param)` numerically solves the system of equations `eqns` for the variables `vars` using the initial guess or search range `init_param`.

`___ = vpasolve( ___,Random=true)` uses a random initial guess for finding solutions. Use this input to avoid returning the same solution repeatedly for nonpolynomial equations. If you specify initial guesses for all variables, setting `Random` to `true` has no effect.

### Examples

### Solve Polynomial and Nonpolynomial Equations

Solve a polynomial equation. For polynomial equations, `vpasolve` returns all solutions.

```
syms x
S = vpasolve(2*x^4 + 3*x^3 - 4*x^2 - 3*x + 2 == 0, x)

S =
```

$$\begin{pmatrix} -2.0 \\ -1.0 \\ 0.5 \\ 1.0 \end{pmatrix}$$

Solve a nonpolynomial equation. For nonpolynomial equations, `vpasolve` returns the first solution that it finds.

```
S = vpasolve(sin(x) == 1/2, x)

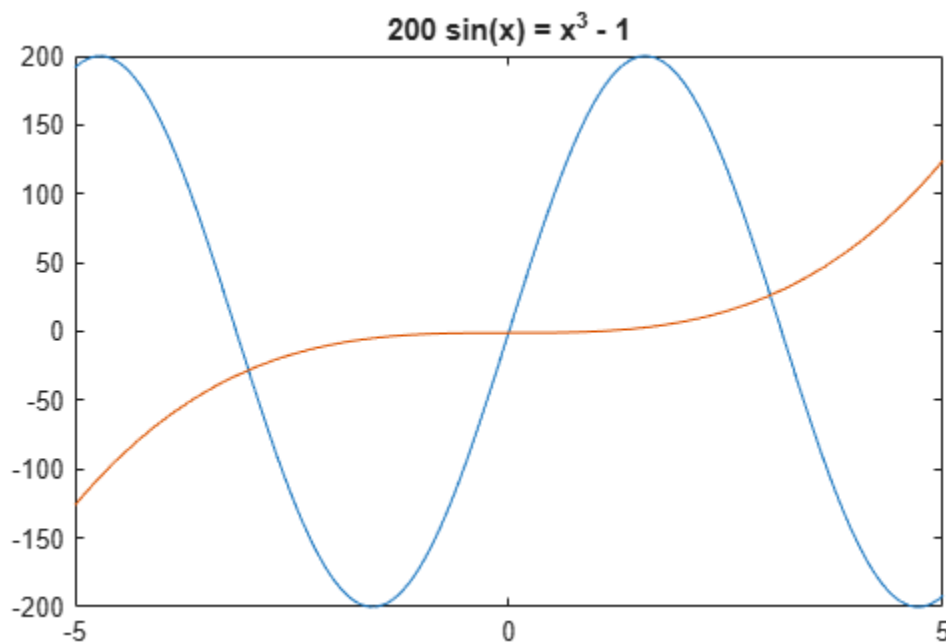
S = 0.52359877559829887307710723054658
```

### Find Multiple Solutions by Specifying Initial Guesses

Find multiple solutions of the equation  $200 \sin(x) = x^3 - 1$  by specifying the initial guesses when using `vpasolve`.

Plot the left and right sides of the equation.

```
syms x
eqnLeft = 200*sin(x);
eqnRight = x^3 - 1;
fplot([eqnLeft eqnRight],[-5 5])
title(texlabel(eqnLeft) + " = " + texlabel(eqnRight))
```



The plot shows that the equation has three solutions. If you do not specify the initial guess, `vpasolve` returns the first solution that it finds.

```
S1 = vpasolve(eqnLeft == eqnRight, x)
S1 = -0.00500000214585835715725440675982988
```

Find one of the other solutions by specifying an initial guess that is close to that solution.

```
S2 = vpasolve(eqnLeft == eqnRight, x, -3)
S2 = -3.0009954677086430679926572924945
S3 = vpasolve(eqnLeft == eqnRight, x, 4)
S3 = 3.0098746383859522384063444361906
```

### Solve System of Equations

Solve a system of equations. Use one output argument to return the solutions in the form of a structure array.

```
syms u v
Y = vpasolve([v^3 + 2*u == v, v^2 == u], [u,v])
Y = struct with fields:
    u: [3×1 sym]
    v: [3×1 sym]
```

Display the solutions by accessing the fields of the structure array `Y`.



```
uSol = Y.u
```

```
uSol =
      0
      5.8284271247461900976033774484194
      0.1715728752538099023966225515806
```

```
vSol = Y.v
```

```
vSol =
      0
     -2.4142135623730950488016887242097
      0.4142135623730950488016887242097
```

If `vpasolve` cannot find a solution, it returns an empty object.

```
syms x
eqns = [3*x+2, 3*x+1];
Y = vpasolve(eqns,x)
```

```
Y =
```

```
Empty sym: 0-by-1
```

### Assign Solutions to Variables When Solving System of Equations

When solving a system of equations, use multiple output arguments to assign the solutions directly to output variables. The order in which the solver returns the solutions follows the order in which you specify the variables.

```
syms x y
[sol_x, sol_y] = vpasolve([x*sin(10*x) == y^3, y^2 == exp(-2*x/3)], [x,y])

sol_x = 88.90707209659114864849280774681
sol_y = 0.00000000000013470479710676694388973703681918
```

### Specify Ranges of Solutions

You can specify ranges of solutions of an equation. For example, if you want to restrict your search to only real solutions, you cannot use assumptions because `vpasolve` ignores assumptions. Instead, specify a search interval. For the following equation, if you do not specify ranges, the numeric solver returns all six solutions of the equation.

```
syms x
S = vpasolve(x^6 - x^2 == 3, x)

S =
```

$$\begin{pmatrix} -1.2929423350084724369196550436382 \\ 1.2929423350084724369196550436382 \\ -0.50188125716943915856832436499602 - 1.0429452224956770037495194222175 i \\ -0.50188125716943915856832436499602 + 1.0429452224956770037495194222175 i \\ 0.50188125716943915856832436499602 - 1.0429452224956770037495194222175 i \\ 0.50188125716943915856832436499602 + 1.0429452224956770037495194222175 i \end{pmatrix}$$

Suppose you need only real solutions of this equation. You cannot use assumptions on variables because `vpasolve` ignores them.

```
assume(x, "real")
S = vpasolve(x^6 - x^2 == 3, x)
```

$$S = \begin{pmatrix} -1.2929423350084724369196550436382 \\ 1.2929423350084724369196550436382 \\ -0.50188125716943915856832436499602 - 1.0429452224956770037495194222175 i \\ -0.50188125716943915856832436499602 + 1.0429452224956770037495194222175 i \\ 0.50188125716943915856832436499602 - 1.0429452224956770037495194222175 i \\ 0.50188125716943915856832436499602 + 1.0429452224956770037495194222175 i \end{pmatrix}$$

Clear the assumptions on `x` for further computations.

```
assume(x, "clear")
```

Specify the search range to restrict the returned results to particular ranges. For example, to return only real solutions of this equation, specify the search interval as `[-Inf Inf]`.

```
S = vpasolve(x^6 - x^2 == 3, x, [-Inf Inf])
```

$$S = \begin{pmatrix} -1.2929423350084724369196550436382 \\ 1.2929423350084724369196550436382 \end{pmatrix}$$

To return nonnegative solutions, specify the search interval as `[0 Inf]`.

```
S = vpasolve(x^6 - x^2 == 3, x, [0 Inf])
```

$$S = 1.2929423350084724369196550436382$$

The search range can also contain complex numbers, such as `[-1, 1+2i]`. In this case, `vpasolve` uses a rectangular search area in the complex plane where `-1` specifies the bottom-left corner of the search area and `1+2i` specifies the top-right corner of that area.

```
S = vpasolve(x^6 - x^2 == 3, x, [-1 1+2i])
```

$$S = \begin{pmatrix} -0.50188125716943915856832436499602 + 1.0429452224956770037495194222175 i \\ 0.50188125716943915856832436499602 + 1.0429452224956770037495194222175 i \end{pmatrix}$$

### Provide Initial Guess to Find Solutions

Find the solution for the following system of equations.

```
syms x y
eqn1 = exp(-x^2-y^2)*(x-4) - exp((-x^2-y^2)/2)*(x-2) == 0
```

```
eqn1 =

$$e^{-x^2-y^2}(x-4) - e^{-\frac{x^2}{2}-\frac{y^2}{2}}(x-2) = 0$$

```

```
eqn2 = exp(-x^2-y^2)*(y-2) - exp((-x^2-y^2)/2)*(y-4) == 0
```

```
eqn2 =

$$e^{-x^2-y^2}(y-2) - e^{-\frac{x^2}{2}-\frac{y^2}{2}}(y-4) = 0$$

```

Find the solution for the variables x and y without specifying initial guess. `vpasolve` cannot find a solution and it returns an empty object.

```
[solX, solY] = vpasolve([eqn1 eqn2],[x y])
```

```
solX =
```

```
Empty sym: 0-by-1
```

```
solY =
```

```
Empty sym: 0-by-1
```

Now specify the initial guesses  $x = 2$  and  $y = 4$ . `vpasolve` returns the solutions that are close to the initial guesses.

```
[solX, solY] = vpasolve([eqn1 eqn2],[x y],[2; 4])
```

```
solX = 1.9999092125057125429174334656647
```

```
solY = 4.0000907874942874570825665343353
```

### Find Multiple Solutions for Nonpolynomial Equation

By default, `vpasolve` returns the same solution on every call. To find more than one solution for nonpolynomial equations, set `Random` to `true`. This makes `vpasolve` use a random initial guess which can lead to different solutions on successive calls.

If `Random` is not specified, `vpasolve` returns the same solution on every call.

```
syms x
f = x-tan(x);
for n = 1:3
    S = vpasolve(f,x)
end
```

```
S = 0
```

```
S = 0
```

```
S = 0
```

When `Random` is set to `true`, `vpasolve` returns a distinct solution on every call.

```
for n = 1:3
    S = vpasolve(f,x,Random=true)
end

S = -227.76107684764829218924973598808
S = 102.09196646490764333652956578441
S = 61.244730260374400372753016364097
```

The `Random` option can be used in conjunction with a search range.

```
S = vpasolve(f,x,[10 12],Random=true)
S = 10.904121659428899827148702790189
```

## Input Arguments

### **eqn** — Equation to solve

symbolic equation | symbolic expression

Equation to solve, specified as a symbolic equation or symbolic expression. A symbolic equation is defined by the relation operator `==`. If `eqn` is a symbolic expression (without the right side), the solver assumes that the right side is 0, and solves the equation `eqn == 0`.

### **var** — Variable to solve equation for

symbolic variable

Variable to solve equation for, specified as a symbolic variable. If `var` is not specified, `symvar` determines the variables.

### **eqns** — System of equations or expressions to solve

symbolic vector | symbolic matrix | symbolic array

System of equations or expressions to solve, specified as a symbolic vector, matrix, or array of equations or expressions. These equations or expressions can also be separated by commas. If an equation is a symbolic expression (without the right side), the solver assumes that the right side of the equation is 0.

### **vars** — Variables to solve system of equations for

symbolic vector | symbolic matrix

Variables to solve system of equations for, specified as a symbolic vector or symbolic matrix. These variables are specified as a vector or comma-separated list. If `vars` is not specified, `symvar` determines the variables.

### **init\_param** — Initial guess or search range for solution

numeric value | vector | matrix with two columns

Initial guess or search range for a solution, specified as a numeric value, vector, or matrix with two columns.

- If `init_param` is a number or, in the case of multivariate equations, a vector of numbers, then the numeric solver uses it as an initial guess. If `init_param` is specified as a scalar while the system

of equations is multivariate, then the numeric solver uses the scalar value as an initial guess for all variables. For an example, see “Find Multiple Solutions by Specifying Initial Guesses” on page 6-1639.

- If `init_param` is a matrix with two columns, then the two entries of the rows specify the bounds of an initial guess for the corresponding variables. To specify an initial guess in a matrix of search ranges, specify both columns as the initial guess value.
- If you specify `init_param` as a search range `[a b]` and the values `a`, `b` are complex numbers, then `vpasolve` searches for the solutions in the rectangular search area in the complex plane. Here, `a` specifies the bottom-left corner of the rectangular search area, and `b` specifies the top-right corner of that area. For an example, see “Specify Ranges of Solutions” on page 6-1641.
- To omit a search range for a variable, set the search range for that variable to `[NaN, NaN]` in `init_param`. All other uses of `NaN` in `init_param` will error.

## Output Arguments

### **S — Solutions of univariate equation**

symbolic value | symbolic array

Solutions of univariate equation, returned as symbolic value or symbolic array. The size of a symbolic array corresponds to the number of the solutions.

### **Y — Solutions of system of equations**

structure array

Solutions of system of equations, returned as a structure array. The number of fields in the structure array corresponds to the number of variables to be solved.

### **y1, ..., yN — Variables that are assigned solutions of system of equations**

array of numeric variables | array of symbolic variables

Variables that are assigned solutions of system of equations, returned as an array of numeric or symbolic variables. The number of output variables or symbolic arrays must equal the number of variables to be solved. If you explicitly specify independent variables `vars`, then the solver uses the same order to return the solutions. If you do not specify `vars`, the toolbox sorts independent variables alphabetically, and then assigns the solutions for these variables to the output variables or symbolic arrays.

## Tips

- If `vpasolve` cannot find a solution, it returns an empty object. Provide initial guess to help the solver finding a solution. For an example, see “Provide Initial Guess to Find Solutions” on page 6-1642.
- For polynomial equations, `vpasolve` returns all solutions. For nonpolynomial equations, there is no general method of finding all solutions and `vpasolve` returns only one solution by default. To find several different solutions for nonpolynomial, you can set `Random` to true and use `vpasolve` repeatedly.
- When you solve a system of equations with nonunique solutions, the behavior of `vpasolve` depends on whether the system is polynomial or nonpolynomial. If polynomial, `vpasolve` returns all solutions by introducing an arbitrary parameter. If nonpolynomial, a single numerical solution is returned, if it exists.

- When you solve a system of rational equations, `vpasolve` transforms the rational equations to polynomials by multiplying out the denominators. `vpasolve` returns all solutions of the resulting polynomial system, which also include the roots of the denominators.
- `vpasolve` ignores assumptions set on variables. You can restrict the returned results to particular ranges by specifying appropriate search ranges using the argument `init_param`. However, if the equations or expressions have other variables besides the variables to solve for as specified by `vars`, then `vpasolve` returns the solutions for `vars` in terms of the other variables that are complex in general.
- The output variables `y1, ..., yN` do not specify the variables for which `vpasolve` solves equations or systems. If `y1, ..., yN` are the variables that appear in `eqns`, that does not guarantee that `vpasolve(eqns)` will assign the solutions to `y1, ..., yN` using the correct order. Thus, for the call `[a,b] = vpasolve(eqns)`, you might get the solutions for `a` assigned to `b` and vice versa. To ensure the order of the returned solutions, specify the variables `vars`. For example, the call `[b,a] = vpasolve(eqns,[b,a])` assigns the solutions for `a` assigned to `a` and the solutions for `b` assigned to `b`.
- You can solve equations symbolically using `solve`, and then numerically approximate the results using `vpa`. Using this approach, you get numeric approximations of all solutions found by the symbolic solver. However, this can reduce computational speed since solving symbolically and postprocessing the results take more time than directly using the numeric solver `vpasolve`.

## Algorithms

- When you set `Random` to `true` and specify a search range for a variable, random initial guesses within the search range are chosen using the internal random number generator (with uniform distribution).
- When you set `Random` to `true` and do not specify a search range for a variable, random initial guesses are generated using a Cauchy distribution with a half-width of `100`. This means the initial guesses are real valued and have a large spread of values on repeated calls.

## Version History

### Introduced in R2012b

### R2021b: `vpasolve(eqns,vars)` returns all solutions for the symbolic matrix `vars`

*Behavior changed in R2021b*

If `vars` is declared as a symbolic matrix, `vpasolve(eqns,vars)` now returns all solutions for `vars`.

For example, find the solutions of four equations with four unknowns. Declare the four unknowns as a symbolic matrix. `vpasolve(eqns,vars)` now returns all solutions for the unknowns.

```
syms a b c d
eqn1 = b + c + d == 50;
eqn2 = a + c + d == 65;
eqn3 = a + b + d == 70;
eqn4 = a + b + c == 65;
vars = [a b; c d];
sol = vpasolve([eqn1 eqn2 eqn3 eqn4],vars)

sol =
  struct with fields:
    a: 33.333333333333333333333333333333
    c: 13.333333333333333333333333333333
```

[illegible]

In previous releases, if `vars` is a symbolic matrix, then `vpasolve(eqns,vars)` treats `vars` as a column vector and it only returns the solutions for the first column of `vars`. For example, find the solutions for the same equations with the unknowns declared as a symbolic matrix.

```
syms a b c d
eqn1 = b + c + d == 50;
eqn2 = a + c + d == 65;
eqn3 = a + b + d == 70;
eqn4 = a + b + c == 65;
vars = [a b; c d];
sol = vpasolve([eqn1 eqn2 eqn3 eqn4],vars)
```

```
sol =  
  struct with fields:  
    a: [1x1 sym]  
    c: [1x1 sym]
```

## See Also

dsolve | equationsToMatrix | fzero | isolate | linsolve | solve | symvar | vpa | digits

## vpasum

Numerical summation using variable precision

### Syntax

```
s = vpasum(f,a,b)
s = vpasum(f,x,a,b)
```

### Description

`s = vpasum(f,a,b)` numerically approximates the sum of a series defined by `f` from `a` to `b`. The default summation variable `x` in `f` is determined by `symvar`. The summation bounds `a` and `b` must be real.

`vpasum(f,[a,b])` is equal to `vpasum(f,a,b)`.

`s = vpasum(f,x,a,b)` performs numerical summation using the summation variable `x`.

### Examples

#### Numerically Sum Symbolic Expression

Numerically sum the symbolic expression  $\frac{1}{1.2^x}$  from 1 to 1,000.

```
syms x;
s = vpasum(1/1.2^x,1,1000)

s = 5.0
```

#### Summation of Symbolic Function

Find the summation of the symbolic function  $y(x) = x^2$  from  $a$  to  $b$ . Since the limits of summation must be real values, assume that the bounds  $a$  and  $b$  are real.

```
syms y(x)
syms a b real
y(x) = x^2;
s = vpasum(y,a,b)
```

```
s =

$$\sum_{x=a}^b x^2$$

```



## Increase Performance of Numerical Series Summation

Compare the computation time to evaluate symbolic and numerical summations.

Find the symbolic summation of the series  $\sum_{k=1}^{\infty} (-1)^k \frac{\log(k)}{k^3}$  using `symsum`. Use `vpa` to numerically evaluate the symbolic summation using 32 significant digits. Measure the time required to declare the symbolic summation and evaluate its numerical value.

```
syms k
tic
y = symsum((-1)^k*log(k)/k^3,k,1,Inf)

y =

$$\sum_{k=1}^{\infty} \frac{(-1)^k \log(k)}{k^3}$$


yVpa = vpa(y)
yVpa = 0.059705906160195358363429266287926
toc

Elapsed time is 3.785970 seconds.
```

To increase computation performance (shorten computation time), use `vpasum` to evaluate the same numerical summation without evaluating the symbolic summation.

```
tic
y = vpasum((-1)^k*log(k)/k^3,k,1,Inf)
y = 0.059705906160195358363429266287926
toc

Elapsed time is 0.204619 seconds.
```

## Input Arguments

### **f** — Expression or function to sum

symbolic number | symbolic variable | symbolic function | symbolic expression | symbolic vector | symbolic matrix | symbolic multidimensional array

Expression or function to sum, specified as a symbolic number, variable, function, expression, vector, matrix, or multidimensional array.

### **a, b** — Limits of summation

two comma-separated numbers | symbolic numbers | symbolic variables | symbolic functions | symbolic expressions

Limits of summation, specified as two comma-separated numbers, symbolic numbers, symbolic variables, symbolic functions, or symbolic expressions. Specifying the summation range from *a* to *b* can also be done using a vector with two elements. The limits of summation must be real.

### **x** — Summation variable

symbolic variable

Summation variable, specified as a symbolic variable. If  $x$  is not specified, the integration variable is determined by `symvar(f)`.

## Algorithms

Depending on whether the series is alternating or monotone, `vpasum` tries a number of strategies to calculate its limit: Levin's u-transformation, the Euler-Maclaurin formula, or van Wijngaarden's trick.

For example, the Euler-Maclaurin formula is

$$\sum_{i=a}^b f(i) = \frac{f(a) + f(b)}{2} + \int_a^b f(x) dx + \left( \sum_{m=1}^{\lfloor p/2 \rfloor} \frac{B_{2m}}{(2m)!} (f(b)^{2m-1} - f(a)^{2m-1}) \right) + R_p,$$

where  $B_{2m}$  represents the 2mth Bernoulli on page 6-97 number and  $R_p$  is an error term which depends on  $a$ ,  $b$ ,  $p$ , and  $f$ .

## Version History

Introduced in R2020b

## References

- [1] Olver, F. W. J., A. B. Olde Daalhuis, D. W. Lozier, B. I. Schneider, R. F. Boisvert, C. W. Clark, B. R. Miller, B. V. Saunders, H. S. Cohl, and M. A. McClain, eds., Chapter 2.10 Sums and Sequences, *NIST Digital Library of Mathematical Functions*, Release 1.0.26 of 2020-03-15.

## See Also

`vpa` | `vpaintegral` | `symsum`

# whittakerM

Whittaker M function

## Syntax

`whittakerM(a,b,z)`

## Description

`whittakerM(a,b,z)` returns the value of the Whittaker M function on page 6-1653.

## Examples

### Compute Whittaker M Function for Numeric Input

Compute the Whittaker M function for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
[whittakerM(1, 1, 1), whittakerM(-2, 1, 3/2 + 2*i),...
whittakerM(2, 2, 2), whittakerM(3, -0.3, 1/101)]
```

```
ans =
    0.7303          -9.2744 + 5.4705i    2.6328          0.3681
```

### Compute Whittaker M Function for Symbolic Input

Compute the Whittaker M function for the numbers converted to symbolic objects. For most symbolic (exact) numbers, `whittakerM` returns unresolved symbolic calls.

```
[whittakerM(sym(1), 1, 1), whittakerM(-2, sym(1), 3/2 + 2*i),...
whittakerM(2, 2, sym(2)), whittakerM(sym(3), -0.3, 1/101)]
```

```
ans =
[ whittakerM(1, 1, 1), whittakerM(-2, 1, 3/2 + 2i),
whittakerM(2, 2, 2), whittakerM(3, -3/10, 1/101)]
```

For symbolic variables and expressions, `whittakerM` also returns unresolved symbolic calls:

```
syms a b x y
[whittakerM(a, b, x), whittakerM(1, x, x^2),...
whittakerM(2, x, y), whittakerM(3, x + y, x*y)]
```

```
ans =
[ whittakerM(a, b, x), whittakerM(1, x, x^2),...
whittakerM(2, x, y), whittakerM(3, x + y, x*y)]
```

### Solve ODE for Whittaker Functions

Solve this second-order differential equation. The solutions are given in terms of the Whittaker functions.

```
syms a b w(z)
dsolve(diff(w, 2) + (-1/4 + a/z + (1/4 - b^2)/z^2)*w == 0)
```

```
ans =
C2*whittakerM(-a,-b,-z) + C3*whittakerW(-a,-b,-z)
```

### Verify Whittaker Functions are Solution of ODE

Verify that the Whittaker M function is a valid solution of this differential equation:

```
syms a b z
isAlways(diff(whittakerM(a,b,z), z, 2) +...
(-1/4 + a/z + (1/4 - b^2)/z^2)*whittakerM(a,b,z) == 0)
```

```
ans =
    logical
     1
```

Verify that  $\text{whittakerM}(-a, -b, -z)$  also is a valid solution of this differential equation:

```
syms a b z
isAlways(diff(whittakerM(-a,-b,-z), z, 2) +...
(-1/4 + a/z + (1/4 - b^2)/z^2)*whittakerM(-a,-b,-z) == 0)
```

```
ans =
    logical
     1
```

### Compute Special Values of Whittaker M Function

The Whittaker M function has special values for some parameters:

```
whittakerM(sym(-3/2), 1, 1)
```

```
ans =
exp(1/2)
```

```
syms a b x
whittakerM(0, b, x)
```

```
ans =
4^b*x^(1/2)*gamma(b + 1)*besseli(b, x/2)
```

```
whittakerM(a + 1/2, a, x)
```

```
ans =
x^(a + 1/2)*exp(-x/2)*whittakerM(a, a - 5/2, x)
```

```
ans =
(2*x^(a - 2)*exp(-x/2)*(2*a^2 - 7*a + x^2/2 -...
x*(2*a - 3) + 6))/pochhammer(2*a - 4, 2)
```

### Differentiate Whittaker M Function

Differentiate the expression involving the Whittaker M function:

```
syms a b z
diff(whittakerM(a,b,z), z)
```

```
ans =
(whittakerM(a + 1, b, z)*(a + b + 1/2))/z -...
(a/z - 1/2)*whittakerM(a, b, z)
```

## Compute Whittaker M Function for Matrix Input

Compute the Whittaker M function for the elements of matrix A:

```
syms x
A = [-1, x^2; 0, x];
whittakerM(-1/2, 0, A)

ans =
[ exp(-1/2)*1i, exp(x^2/2)*(x^2)^(1/2) ]
[ 0, x^(1/2)*exp(x/2) ]
```

## Input Arguments

### a — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

If **a** is a vector or matrix, `whittakerM` returns the beta function for each element of **a**.

### b — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

If **b** is a vector or matrix, `whittakerM` returns the beta function for each element of **b**.

### z — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

If **x** is a vector or matrix, `whittakerM` returns the beta function for each element of **z**.

## More About

### Whittaker M Function

The Whittaker functions  $M_{a,b}(z)$  and  $W_{a,b}(z)$  are linearly independent solutions of this differential equation:

$$\frac{d^2 w}{dz^2} + \left( -\frac{1}{4} + \frac{a}{z} + \frac{1/4 - b^2}{z^2} \right) w = 0$$

The Whittaker M function is defined via the confluent hypergeometric functions:

$$M_{a,b}(z) = e^{-z/2} z^{b+1/2} M\left(b - a + \frac{1}{2}, 1 + 2b, z\right)$$

## Tips

- All non-scalar arguments must have the same size. If one or two input arguments are non-scalar, then `whittakerM` expands the scalars into vectors or matrices of the same size as the non-scalar arguments, with all elements equal to the corresponding scalar.

## Version History

Introduced in R2012a

## References

- [1] Slater, L. J. "Confluent Hypergeometric Functions." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

`hypergeom` | `kummerU` | `whittakerW`

# whittakerW

Whittaker W function

## Syntax

`whittakerW(a,b,z)`

## Description

`whittakerW(a,b,z)` returns the value of the Whittaker W function on page 6-1657.

## Examples

### Compute Whittaker W Function for Numeric Input

Compute the Whittaker W function for these numbers. Because these numbers are not symbolic objects, you get floating-point results.

```
[whittakerW(1, 1, 1), whittakerW(-2, 1, 3/2 + 2*i),...
whittakerW(2, 2, 2), whittakerW(3, -0.3, 1/101)]
```

```
ans =
    1.1953          -0.0156 - 0.0225i    4.8616          -0.1692
```

### Compute Whittaker W Function for Symbolic Input

Compute the Whittaker W function for the numbers converted to symbolic objects. For most symbolic (exact) numbers, `whittakerW` returns unresolved symbolic calls.

```
[whittakerW(sym(1), 1, 1), whittakerW(-2, sym(1), 3/2 + 2*i),...
whittakerW(2, 2, sym(2)), whittakerW(sym(3), -0.3, 1/101)]
```

```
ans =
[ whittakerW(1, 1, 1), whittakerW(-2, 1, 3/2 + 2i),
whittakerW(2, 2, 2), whittakerW(3, -3/10, 1/101)]
```

For symbolic variables and expressions, `whittakerW` also returns unresolved symbolic calls:

```
syms a b x y
[whittakerW(a, b, x), whittakerW(1, x, x^2),...
whittakerW(2, x, y), whittakerW(3, x + y, x*y)]
```

```
ans =
[ whittakerW(a, b, x), whittakerW(1, x, x^2),
whittakerW(2, x, y), whittakerW(3, x + y, x*y)]
```

### Solve ODE for Whittaker Functions

Solve this second-order differential equation. The solutions are given in terms of the Whittaker functions.

```
syms a b w(z)
dsolve(diff(w, 2) + (-1/4 + a/z + (1/4 - b^2)/z^2)*w == 0)
```

```
ans =
C2*whittakerM(-a, -b, -z) + C3*whittakerW(-a, -b, -z)
```

### Verify Whittaker Functions are Solution of ODE

Verify that the Whittaker W function is a valid solution of this differential equation:

```
syms a b z
isAlways(diff(whittakerW(a, b, z), z, 2) +...
(-1/4 + a/z + (1/4 - b^2)/z^2)*whittakerW(a, b, z) == 0)
```

```
ans =
    logical
     1
```

Verify that  $\text{whittakerW}(-a, -b, -z)$  also is a valid solution of this differential equation:

```
syms a b z
isAlways(diff(whittakerW(-a, -b, -z), z, 2) +...
(-1/4 + a/z + (1/4 - b^2)/z^2)*whittakerW(-a, -b, -z) == 0)
```

```
ans =
    logical
     1
```

### Compute Special Values of Whittaker W Function

The Whittaker W function has special values for some parameters:

```
whittakerW(sym(-3/2), 1/2, 0)
```

```
ans =
4/(3*pi^(1/2))
```

```
syms a b x
whittakerW(0, b, x)
```

```
ans =
(x^(b + 1/2)*besselk(b, x/2))/(x^b*pi^(1/2))
```

```
whittakerW(a, -a + 1/2, x)
```

```
ans =
x^(1 - a)*x^(2*a - 1)*exp(-x/2)
```

```
whittakerW(a - 1/2, a, x)
```

```
ans =
(x^(a + 1/2)*exp(-x/2)*exp(x)*igamma(2*a, x))/x^(2*a)
```

### Differentiate Whittaker W Function

Differentiate the expression involving the Whittaker W function:

```
syms a b z
diff(whittakerW(a,b,z), z)
```

```
ans =
- (a/z - 1/2)*whittakerW(a, b, z) -...
whittakerW(a + 1, b, z)/z
```



## Compute Whittaker W Function for Matrix Input

Compute the Whittaker W function for the elements of matrix A:

```
syms x
A = [-1, x^2; 0, x];
whittakerW(-1/2, 0, A)

ans =
[ -exp(-1/2)*(ei(1) + pi*1i)*1i,...
  exp(x^2)*exp(-x^2/2)*expint(x^2)*(x^2)^(1/2)]
[ 0,...
  x^(1/2)*exp(-x/2)*exp(x)*expint(x)]
```

## Input Arguments

### a — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

If **a** is a vector or matrix, `whittakerW` returns the beta function for each element of **a**.

### b — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

If **b** is a vector or matrix, `whittakerW` returns the beta function for each element of **b**.

### z — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

If **x** is a vector or matrix, `whittakerW` returns the beta function for each element of **z**.

## More About

### Whittaker W Function

The Whittaker functions  $M_{a,b}(z)$  and  $W_{a,b}(z)$  are linearly independent solutions of this differential equation:

$$\frac{d^2 w}{dz^2} + \left( -\frac{1}{4} + \frac{a}{z} + \frac{1/4 - b^2}{z^2} \right) w = 0$$

The Whittaker W function is defined via the confluent hypergeometric functions:

$$W_{a,b}(z) = e^{-z/2} z^{b+1/2} U\left(b - a + \frac{1}{2}, 1 + 2b, z\right)$$

## Tips

- All non-scalar arguments must have the same size. If one or two input arguments are non-scalar, then `whittakerW` expands the scalars into vectors or matrices of the same size as the non-scalar arguments, with all elements equal to the corresponding scalar.

## Version History

Introduced in R2012a

## References

- [1] Slater, L. J. "Confluent Hypergeometric Functions." *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. (M. Abramowitz and I. A. Stegun, eds.). New York: Dover, 1972.

## See Also

`hypergeom` | `kummerU` | `whittakerM`

# wrightOmega

Wright omega function

## Syntax

```
wrightOmega(x)
```

## Description

`wrightOmega(x)` computes the Wright omega function on page 6-1661 of *x*. If *z* is a matrix, `wrightOmega` acts elementwise on *z*.

## Examples

### Compute Wright Omega Function of Numeric Inputs

Compute the Wright omega function for these numbers. Because these numbers are not symbolic objects, you get floating-point results:

```
wrightOmega(1/2)
```

```
ans =  
    0.7662
```

```
wrightOmega(pi)
```

```
ans =  
    2.3061wrightOmega(-1+i*pi)
```

```
ans =  
   -1.0000 + 0.0000i
```

### Compute Wright Omega Function of Symbolic Numbers

Compute the Wright omega function for the numbers converted to symbolic objects. For most symbolic (exact) numbers, `wrightOmega` returns unresolved symbolic calls:

```
wrightOmega(sym(1/2))
```

```
ans =  
wrightOmega(1/2)
```

```
wrightOmega(sym(pi))
```

```
ans =  
wrightOmega(pi)
```

For some exact numbers, `wrightOmega` has special values:

```
wrightOmega(-1+i*sym(pi))
```

```
ans =  
   -1
```

### Compute Wright Omega Function of Symbolic Expression

Compute the Wright omega function for  $x$  and  $\sin(x) + x\exp(x)$ . For symbolic variables and expressions, `wrightOmega` returns unresolved symbolic calls:

```
syms x
wrightOmega(x)
wrightOmega(sin(x) + x*exp(x))

ans =
wrightOmega(x)

ans =
wrightOmega(sin(x) + x*exp(x))
```

### Compute Derivative of Wright Omega Function

Now compute the derivatives of these expressions:

```
diff(wrightOmega(x), x, 2)
diff(wrightOmega(sin(x) + x*exp(x)), x)

ans =
wrightOmega(x)/(wrightOmega(x) + 1)^2 -...
wrightOmega(x)^2/(wrightOmega(x) + 1)^3

ans =
(wrightOmega(sin(x) + x*exp(x))*(cos(x) +...
exp(x) + x*exp(x)))/(wrightOmega(sin(x) + x*exp(x)) + 1)
```

### Compute Wright Omega Function for Matrix Input

Compute the Wright omega function for elements of matrix  $M$  and vector  $V$ :

```
M = [0 pi; 1/3 -pi];
V = sym([0; -1+i*pi]);
wrightOmega(M)
wrightOmega(V)

ans =
    0.5671    2.3061
    0.6959    0.0415

ans =
lambertw(0, 1)
-1
```

## Input Arguments

### **x** — Input

number | vector | matrix | array | symbolic number | symbolic variable | symbolic array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix, or array, or a symbolic number, variable, array, function, or expression.

## More About

### Wright omega Function

The Wright omega function is defined in terms of the Lambert W function:

$$\omega(x) = W\left[\frac{\operatorname{Im}(x) - \pi}{2\pi}\right](e^x)$$

The Wright omega function  $\omega(x)$  is a solution of the equation  $Y + \log(Y) = X$ .

## Version History

Introduced in R2011b

## References

- [1] Corless, R. M. and D. J. Jeffrey. "The Wright omega Function." *Artificial Intelligence, Automated Reasoning, and Symbolic Computation* (J. Calmet, B. Benhamou, O. Caprotti, L. Henocque, and V. Sorge, eds.). Berlin: Springer-Verlag, 2002, pp. 76-89.

## See Also

`lambertW` | `log`

## writeAnimation

Save animation as video file

### Syntax

```
writeAnimation(filename)
writeAnimation(fig,filename)
writeAnimation( ____,Name,Value)
```

```
writeAnimation(vidObj)
writeAnimation(fig,vidObj)
```

### Description

`writeAnimation(filename)` writes animation objects in the current figure to a GIF or AVI video file. The animation objects must be created using the `fanimator` function. The extension of `filename` sets the video format, and must be either `'.gif'` or `'.avi'`.

- If you do not specify the file extension, then `writeAnimation` chooses the `'.avi'` extension by default.
- If you specify any other file extension, such as `'.mp4'` or `'.mpg'`, then `writeAnimation` returns an error message.

`writeAnimation(fig,filename)` writes animation objects in the figure `fig` to a GIF or AVI video file.

`writeAnimation( ____,Name,Value)` writes animation objects with the specified `Name,Value` pair arguments. Use this option with any of the input argument combinations in the previous syntaxes. You can set the name-value pair settings to specify the properties of a GIF or AVI video file.

`writeAnimation(vidObj)` writes animation objects in the current figure to a `VideoWriter` object. This syntax provides the option to save animation objects in another video file format, such as `'MPEG-4'` or `'Uncompressed AVI'`.

`writeAnimation(fig,vidObj)` writes animation objects in the figure `fig` to a `VideoWriter` object.

If you save an animation as a `VideoWriter` object, then the properties of the output video file follow the properties of the `VideoWriter` object.

### Examples

#### Save Animation of Moving Circle as GIF File

Create a moving circle animation object and save it as a GIF file.

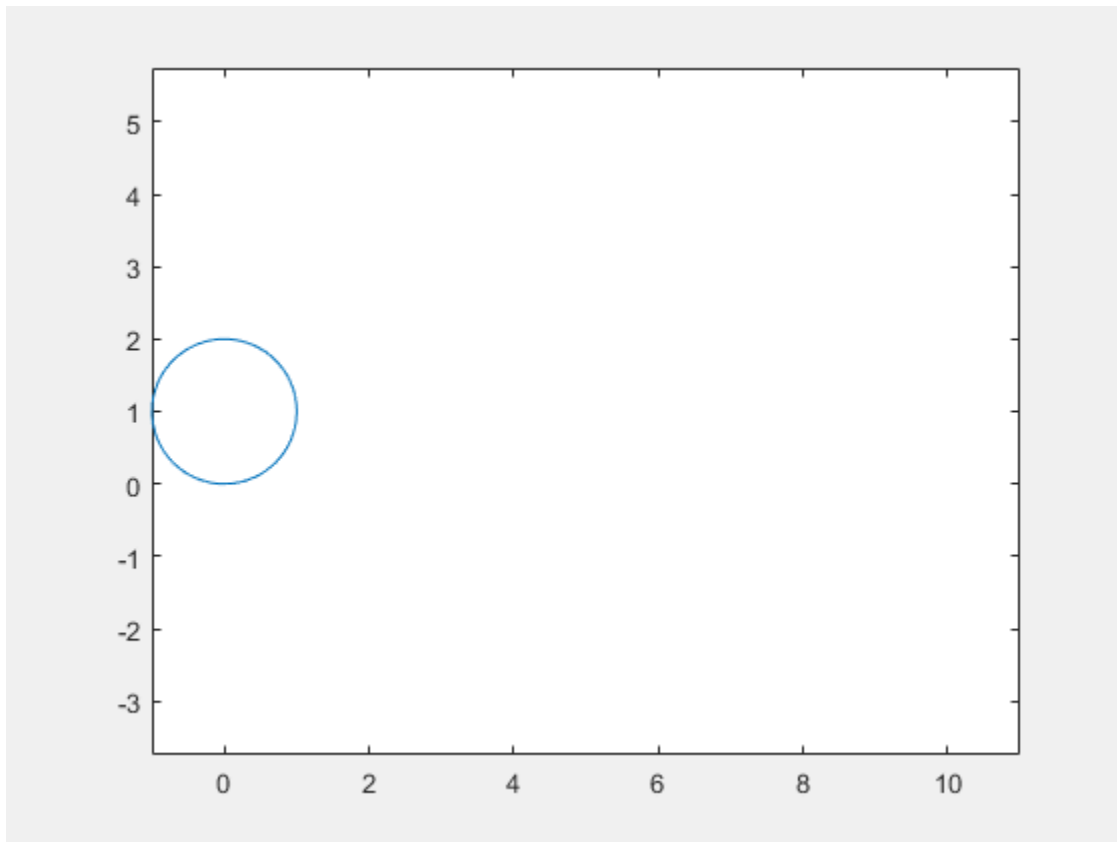
Create two symbolic variables, `t` and `x`. The variable `t` defines the time parameter of the animation. Use `t` to set the center of the circle at `(t,1)` and `x` to parameterize the perimeter of the circle within

the range  $[-\pi, \pi]$ . Create the circle animation object using `fanimator`. Set the x-axis and y-axis to be equal length.

```
syms t x
fanimator(@fplot,cos(x)+t,sin(x)+1,[-pi pi])
axis equal
```

Enter the command `playAnimation` to play the animation. Save the animation as a GIF video file named 'wheel.gif'.

```
writeAnimation('wheel.gif')
```



### Save Animation of Moving Circle as MPEG-4 File

Create a moving circle animation object and save it as an MPEG-4 file.

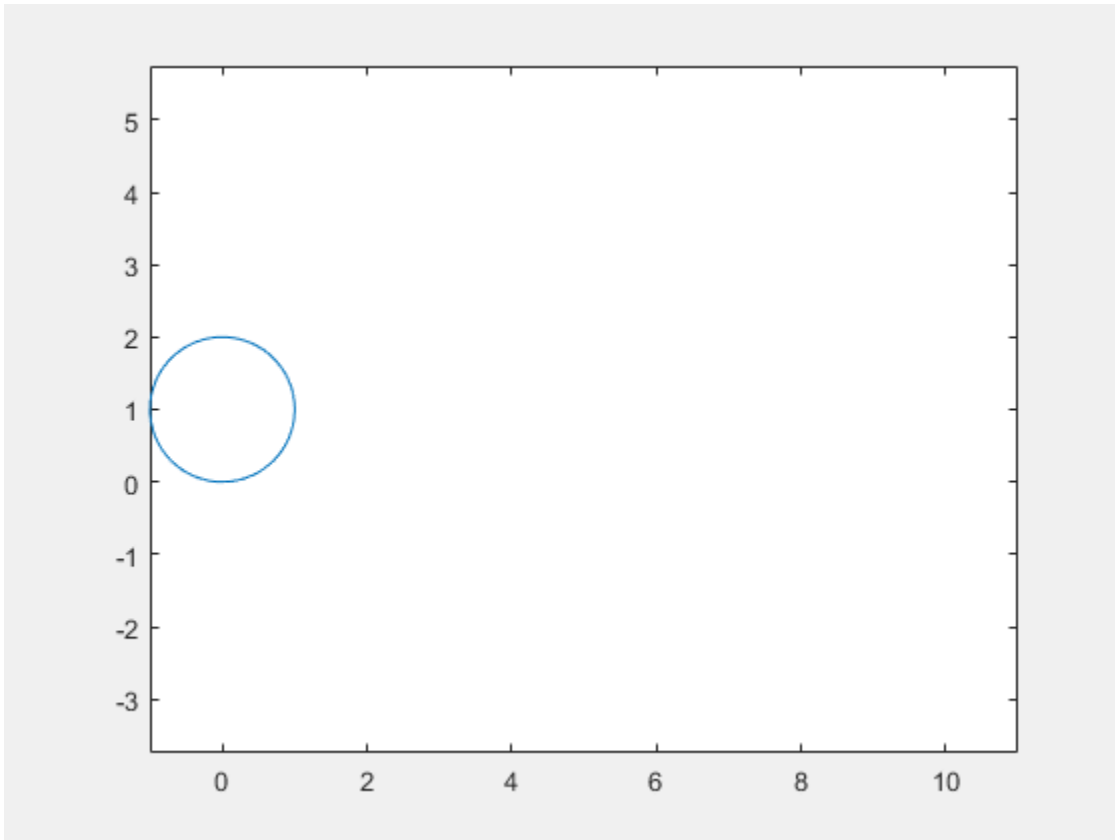
First, create two symbolic variables, `t` and `x`. The variable `t` defines the time parameter of the animation. Use `t` to set the center of the circle at  $(t, 1)$  and `x` to parameterize the perimeter of the circle within the range  $[-\pi, \pi]$ . Create the circle animation object using `fanimator`. Set the x-axis and y-axis to be equal length.

```
syms t x
fanimator(@fplot,cos(x)+t,sin(x)+1,[-pi pi])
axis equal
```

Enter the command `playAnimation` to play the animation.

Next, save the animation as an MPEG-4 file. Create a video object that saves to a file named 'myFile' by using the `VideoWriter` function. Specify the video file format as 'MPEG-4'. Open the video file, use `writeAnimation` to save the circle animation object, and close the video file.

```
vidObj = VideoWriter('myFile','MPEG-4');
open(vidObj)
writeAnimation(vidObj)
close(vidObj)
```



### Save Animation as Looping GIF File

Create a circle animation object and save it as a GIF file that plays a specified number of loops.

First, create two symbolic variables,  $t$  and  $x$ . The variable  $t$  defines the time parameter of the animation. Create a figure window for the animation.

```
syms t x
fig = figure;
```

Create the circle animation object using `f animator`. Use  $t$  to set the center of the circle at  $(t, 1)$  and  $x$  to parameterize the perimeter of the circle within the range  $[-\pi, \pi]$ . Set the x-axis and y-axis to be equal length.

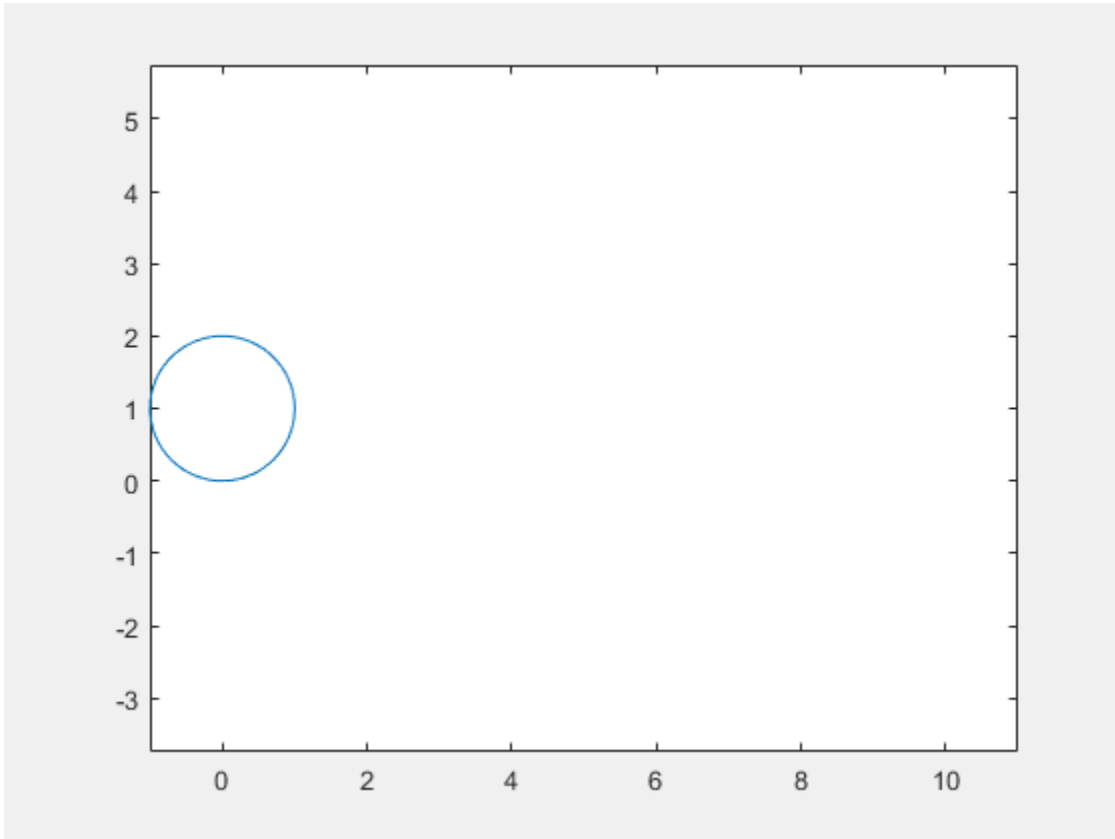
```
syms t x
f animator(@fplot,cos(x)+t,sin(x)+1,[-pi pi])
axis equal
```



Enter the command `playAnimation` to play the animation.

Next, save the animation in the figure `fig` as a GIF file named `'loop.gif'` by using the `writeAnimation` function. The `writeAnimation` function always plays the animation once in a MATLAB figure window before saving the animation. When saving the animation as a GIF file, the created GIF file plays the animation once and repeats the number of loops as specified. For this example, set `'LoopCount'` to 1. The GIF file plays the animation twice.

```
writeAnimation(fig,'loop.gif','LoopCount',1)
```



Note that to properly show the number of loops in a GIF video file, you must open the file in an application with GIF decoder.

## Input Arguments

### **filename** — Video filename

string | character vector

Video filename, specified as a string scalar or character vector. The extension of filename sets the video format, and must be either `'.gif'` or `'.avi'`. You must have permission to write the file.

- If you do not specify the file extension, then `writeAnimation` uses `'.avi'` by default.
- If filename already exists, then `writeAnimation` overwrites the file.
- If filename does not include a full path, then the function saves the animation to the current folder.

**vidObj — Video object**

VideoWriter object

Video object, specified as a VideoWriter object. The VideoWriter object provides the option to control the output video format when you save animation objects. For more information about VideoWriter object in MATLAB, see VideoWriter.

**fig — Target figure**

Figure object

Target figure, specified as a Figure object. For more information about Figure objects, see figure.

**Name-Value Pair Arguments**

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

*Before R2021a, use commas to separate each name and value, and enclose Name in quotes.*

Example: `'FrameRate',15,'LoopCount',2`

**AnimationRange — Range of animation time parameter**`[0 10]` (default) | two-element row vector

Range of the animation time parameter, specified as a two-element row vector. The two elements must be real values that are increasing.

Example: `[-2 4.5]`

**FrameRate — Frame rate**`10` (default) | positive value

Frame rate, specified as a positive value. The frame rate defines the number of frames per unit time when you write animation objects to a video file.

Example: `30`

**Backwards — Backward option**logical `0` (`false`) (default) | logical value

Backward option, specified as a logical value (boolean). If you specify `true`, then the function saves the animation backward or in reverse order.

Example: `true`

**LoopCount — Animation loop count**`0` (default) | nonnegative integer (from `0` to `65535`) | `Inf`

Animation loop count, specified as a nonnegative integer (from `0` to `65535`) or `Inf`. This value sets the number of repeated animation loops in a GIF file. Setting this value has no effect if you use a video file format other than GIF.

- If you use the default value of `0`, then the GIF file plays the animation once.
- If you set `'LoopCount'` to an integer  $n$ , then the GIF file plays the animation once plus  $n$  repeats (a total of  $n+1$  times).

- To repeat the animation infinitely, use the `Inf` value.

Example: 1

## Version History

Introduced in R2019a

### See Also

`f Animator` | `animationToFrame` | `playAnimation` | `rewindAnimation` | `VideoWriter`

## xor

Logical XOR for symbolic expressions

### Syntax

`xor(A,B)`

### Description

`xor(A,B)` represents the logical exclusive disjunction. `xor(A,B)` is true when either A or B is true. If both A and B are true or false, `xor(A,B)` is false.

### Examples

#### Set and Evaluate Condition

Combine two symbolic inequalities into a logical expression using `xor`.

```
syms x
range = xor(x > -10, x < 10);
```

Replace variable `x` with 11 and 0. If you replace `x` with 11, then inequality `x > -10` is valid and `x < 10` is invalid. If you replace `x` with 0, both inequalities are valid. Note that `subs` only substitutes the numeric values into the inequalities. It does not evaluate the inequalities to logical 1 or 0.

```
x1 = subs(range,x,11)
x2 = subs(range,x,0)
```

```
x1 =
-10 < 11 xor 11 < 10
```

```
x2 =
-10 < 0 xor 0 < 10
```

To evaluate these inequalities to logical 1 or 0, use `isAlways`. If only one inequality is valid, the expression with `xor` evaluates to logical 1. If both inequalities are valid, the expression with `xor` evaluates to logical 0.

```
isAlways(x1)
isAlways(x2)
```

```
ans =
    logical
     1
```

```
ans =
    logical
     0
```

Note that `simplify` does not simplify these logical expressions to logical 1 or 0. Instead, `simplify` returns *symbolic* constants `symtrue` or `symfalse`.

```
s1 = simplify(x1)
s2 = simplify(x2)
```

```
s1 =
symtrue
```

```
s2 =
symfalse
```

Convert symbolic `symtrue` or `symfalse` to logical values using `logical`.

```
logical(s1)
logical(s2)
```

```
ans =
    logical
         1
```

```
ans =
    logical
         0
```

## Input Arguments

### A, B — Operands

symbolic equations | symbolic inequalities | symbolic expressions | symbolic arrays

Operands, specified as symbolic equations, inequalities, expressions, or arrays. Inputs A and B must either be the same size or have sizes that are compatible (for example, A is an M-by-N matrix and B is a scalar or 1-by-N row vector). For more information, see “Compatible Array Sizes for Basic Operations”.

## Tips

- If you call `simplify` for a logical expression containing symbolic subexpressions, you can get the symbolic constants `symtrue` and `symfalse`. These two constants are not the same as logical 1 (true) and logical 0 (false). To convert symbolic `symtrue` and `symfalse` to logical values, use `logical`.
- `assume` and `assumeAlso` do not accept assumptions that contain `xor`.

## Version History

### Introduced in R2012a

### R2016b: Implicit expansion change affects arguments for operators

*Behavior changed in R2016b*

Starting in R2016b with the addition of implicit expansion, some combinations of arguments for basic operations that previously returned errors now produce results. For example, you previously could not add a row and a column vector, but those operands are now valid for addition. In other words, an expression like `[1 2] + [1; 2]` previously returned a size mismatch error, but now it executes.

If your code uses element-wise operators and relies on the errors that MATLAB previously returned for mismatched sizes, particularly within a `try/catch` block, then your code might no longer catch those errors.

For more information on the required input sizes for basic array operations, see “Compatible Array Sizes for Basic Operations”.

**See Also**

`all` | `and` | `any` | `isAlways` | `not` | `or`

# zeta

Riemann zeta function

## Syntax

```
zeta(z)
zeta(n,z)
```

## Description

`zeta(z)` evaluates the Riemann zeta function at the elements of `z`, where `z` is a numeric or symbolic input.

`zeta(n,z)` returns the `n`th derivative of `zeta(z)`.

## Examples

### Find Riemann Zeta Function for Numeric and Symbolic Inputs

Find the Riemann zeta function for numeric inputs.

```
zeta([0.7 i 4 11/3])

ans =
    -2.7784 + 0.0000i    0.0033 - 0.4182i    1.0823 + 0.0000i    1.1094 + 0.0000i
```

Find the Riemann zeta function symbolically by converting the inputs to symbolic objects using `sym`. The `zeta` function returns exact results.

```
zeta(sym([0.7 i 4 11/3]))

ans =
[ zeta(7/10), zeta(1i), pi^4/90, zeta(11/3)]
```

`zeta` returns unevaluated function calls for symbolic inputs that do not have results implemented. The implemented results are listed in “Algorithms” on page 6-1674.

Find the Riemann zeta function for a matrix of symbolic expressions.

```
syms x y
Z = zeta([x sin(x); 8*x/11 x + y])

Z =
[      zeta(x), zeta(sin(x))]
[ zeta((8*x)/11), zeta(x + y)]
```

### Find Riemann Zeta Function for Large Inputs

For values of  $|z| > 1000$ , `zeta(z)` might return an unevaluated function call. Use `expand` to force `zeta` to evaluate the function call.

```
zeta(sym(1002))
expand(zeta(sym(1002)))
```

```
ans =  
zeta(1002)  
ans =  
(1087503...312*pi^1002)/15156647...375
```

### **Differentiate Riemann Zeta Function**

Find the third derivative of the Riemann zeta function at point  $x$ .

```
syms x  
expr = zeta(3,x)
```

```
expr =  
zeta(3, x)
```

Find the third derivative at  $x = 4$  by substituting 4 for  $x$  using `subs`.

```
expr = subs(expr,x,4)
```

```
expr =  
zeta(3, 4)
```

Evaluate `expr` using `vpa`.

```
expr = vpa(expr)
```

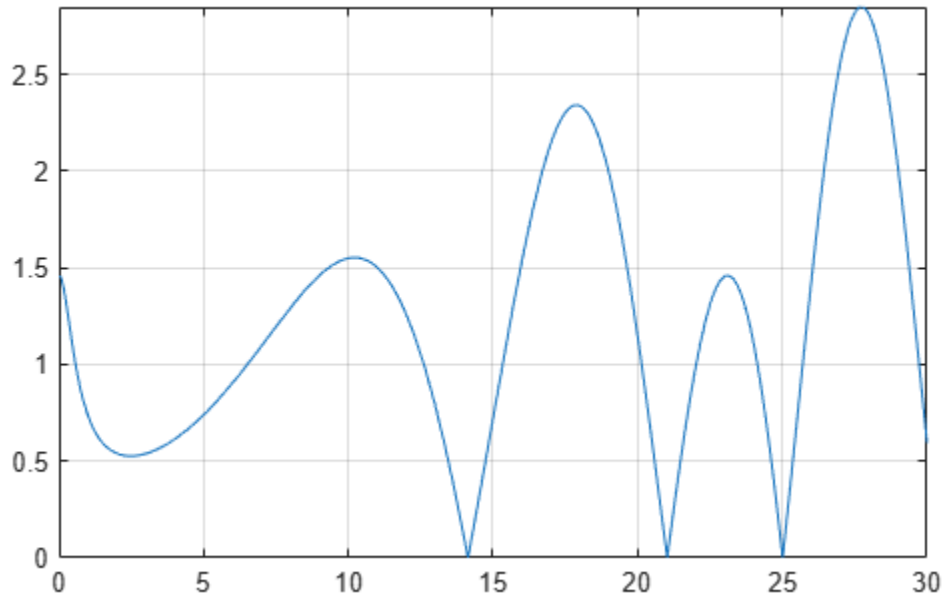
```
expr =  
-0.07264084989132137196244616781177
```

### **Plot Zeros of Riemann Zeta Function**

Zeros of the Riemann Zeta function  $\zeta(x+i*y)$  are found along the line  $x = 1/2$ . Plot the absolute value of the function along this line for  $0 < y < 30$  to view the first three zeros.

```
syms y  
fplot(abs(zeta(1/2+1i*y)),[0 30])  
grid on
```





## Input Arguments

### **z — Input**

number | vector | matrix | multidimensional array | symbolic number | symbolic variable | symbolic vector | symbolic matrix | symbolic multidimensional array | symbolic function | symbolic expression

Input, specified as a number, vector, matrix or multidimensional array, or a symbolic number, variable, vector, matrix, multidimensional array, function or expression.

### **n — Order of derivative**

nonnegative integer

Order of derivative, specified as a nonnegative integer.

## More About

### **Riemann Zeta Function**

The Riemann zeta function is defined by

$$\zeta(z) = \sum_{k=1}^{\infty} \frac{1}{k^z}$$

The series converges only if the real part of  $z$  is greater than 1. The definition of the function is extended to the entire complex plane, except for a simple pole  $z = 1$ , by analytic continuation.

## Tips

- Floating point evaluation is slow for large values of  $n$ .

## Algorithms

The following exact values are implemented.

- $\zeta(0) = -\frac{1}{2}$
- $\zeta(1, 0) = -\frac{\log(\pi)}{2} - \frac{\log(2)}{2}$
- $\zeta(\infty) = 1$
- If  $z < 0$  and  $z$  is an even integer,  $\zeta(z) = 0$ .
- If  $z < 0$  and  $z$  is an odd integer

$$\zeta(z) = -\frac{\text{bernoulli}(1-z)}{1-z}$$

For  $z < -1000$ , `zeta(z)` returns an unevaluated function call. To force evaluation, use `expand(zeta(z))`.

- If  $z > 0$  and  $z$  is an even integer

$$\zeta(z) = \frac{(2\pi)^z |\text{bernoulli}(z)|}{2z!}$$

For  $z > 1000$ , `zeta(z)` returns an unevaluated function call. To force evaluation, use `expand(zeta(z))`.

- If  $n > 0$ ,  $\zeta(n, \infty) = 0$ .
- If the argument does not evaluate to a listed special value, `zeta` returns the symbolic function call.

## Version History

Introduced before R2006a

## See Also

`bernoulli` | `hurwitzZeta` | `gamma` | `psi`

# ztrans

Z-transform

## Syntax

```
fz = ztrans(f)
fz = ztrans(f,transVar)
fz = ztrans(f,var,transVar)
```

## Description

`fz = ztrans(f)` finds the “Z-Transform” on page 6-1678 of `f`. By default, the independent variable is `n` and the transformation variable is `z`. If `f` does not contain `n`, `ztrans` uses `symvar`.

`fz = ztrans(f,transVar)` uses the transformation variable `transVar` instead of `z`.

`fz = ztrans(f,var,transVar)` uses the independent variable `var` and transformation variable `transVar` instead of `n` and `z`, respectively.

## Examples

### Z-Transform of Symbolic Expression

Compute the Z-transform of  $\sin(n)$ . By default, the transform is in terms of  $z$ .

```
syms n
f = sin(n);
fz = ztrans(f)

fz =
```

$$\frac{z \sin(1)}{z^2 - 2 \cos(1) z + 1}$$

### Specify Independent Variable and Transformation Variable

Compute the Z-transform of  $\exp(m+n)$ . By default, the independent variable is  $n$  and the transformation variable is  $z$ .

```
syms m n
f = exp(m+n);
fz = ztrans(f)

fz =
```

$$\frac{z e^m}{z - e}$$

Specify the transformation variable as `y`. If you specify only one variable, that variable is the transformation variable. The independent variable is still `n`.

```
syms y
fz = ztrans(f,y)
```

$$fz = \frac{y e^m}{y - e}$$

Specify both the independent and transformation variables as  $m$  and  $y$  in the second and third arguments, respectively.

```
fz = ztrans(f,m,y)
```

$$fz = \frac{y e^n}{y - e}$$

### Z-Transforms Involving Heaviside Function and Binomial Coefficient

Compute the Z-transforms of the Heaviside function and the binomial coefficient.

```
syms n z
fz = ztrans heaviside(n-3),n,z)
```

$$fz = \frac{\frac{1}{z-1} + \frac{1}{2}}{z^3}$$

```
fz = ztrans(nchoosek(n,2))
```

$$fz = \frac{z}{(z-1)^3}$$

### Z-Transform of Array Inputs

Find the Z-transform of the matrix  $M$ . Specify the independent and transformation variables for each matrix entry by using matrices of the same size. When the arguments are nonscalars, `ztrans` acts on them element-wise.

```
syms a b c d w x y z
M = [exp(x) 1; sin(y) i*z];
vars = [w x; y z];
transVars = [a b; c d];
fz = ztrans(M,vars,transVars)
```

$$fz = \begin{pmatrix} \frac{a e^x}{a-1} & \frac{b}{b-1} \\ \frac{c \sin(1)}{c^2 - 2 \cos(1) c + 1} & \frac{d i}{(d-1)^2} \end{pmatrix}$$

If `ztrans` is called with both scalar and nonscalar arguments, then it expands the scalars to match the nonscalars by using scalar expansion. Nonscalar arguments must be the same size.

```
syms w x y z a b c d
fz = ztrans(x,vars,transVars)
```

$$fz = \begin{pmatrix} \frac{ax}{a-1} & \frac{b}{(b-1)^2} \\ \frac{cx}{c-1} & \frac{dx}{d-1} \end{pmatrix}$$

### Z-Transform of Symbolic Function

Compute the Z-transform of symbolic functions. When the first argument contains symbolic functions, then the second argument must be a scalar.

```
syms f1(x) f2(x) a b
f1(x) = exp(x);
f2(x) = x;
fz = ztrans([f1 f2],x,[a b])
```

$$fz = \begin{pmatrix} \frac{a}{a-e} & \frac{b}{(b-1)^2} \end{pmatrix}$$

### If Z-Transform Cannot Be Found

If `ztrans` cannot transform the input then it returns an unevaluated call.

```
syms f(n) z
f(n) = 1/n;
fz = ztrans(f,n,z)
```

$$fz = ztrans\left(\frac{1}{n}, n, z\right)$$

Return the original expression by using `iztrans`.

```
f = iztrans(fz,z,n)
```

$$f = \frac{1}{n}$$

## Input Arguments

### f — Input

symbolic expression | symbolic function | symbolic vector | symbolic matrix

Input, specified as a symbolic expression, function, vector, or matrix.

**var — Independent variable**

`n` (default) | symbolic variable

Independent variable, specified as a symbolic variable. This variable is often called the "discrete time variable". If you do not specify the variable, then `ztrans` uses `n`. If `f` does not contain `n`, then `ztrans` uses the function `symvar`.

**ttransVar — Transformation variable**

`z` (default) | symbolic variable | symbolic expression | symbolic vector | symbolic matrix

Transformation variable, specified as a symbolic variable, expression, vector, or matrix. This variable is often called the "complex frequency variable." By default, `ztrans` uses `z`. If `z` is the independent variable of `f`, then `ztrans` uses `w`.

## More About

**Z-Transform**

The Z-transform  $F = F(z)$  of the expression  $f = f(n)$  with respect to the variable `n` at the point `z` is

$$F(z) = \sum_{n=0}^{\infty} \frac{f(n)}{z^n}.$$

**Tips**

- If any argument is an array, then `ztrans` acts element-wise on all elements of the array.
- If the first argument contains a symbolic function, then the second argument must be a scalar.
- To compute the inverse Z-transform, use `iztrans`.

## Version History

Introduced before R2006a

**See Also**

`fourier` | `ifourier` | `ilaplace` | `iztrans` | `kroneckerDelta` | `laplace`

**Topics**

"Solve Difference Equations Using Z-Transform" on page 3-205